Vulkan® 1.0.302 - A Specification (with all ratified extensions)

The Khronos® Vulkan Working Group

Version 1.0.302, 2024-11-21 05:51:01Z: from git branch: github-main commit: 310c86fb5a06544a84bce70867f7c038b748e51c

Table of Contents

1. Preamble . . . 1
2. Introduction . . . 3
   2.1. Document Conventions . . . 3
3. Fundamentals . . . 7
   3.1. Host and Device Environment . . . 7
   3.2. Execution Model . . . 7
   3.3. Object Model . . . 9
   3.4. Application Binary Interface . . . 13
   3.5. Command Syntax and Duration . . . 14
   3.6. Threading Behavior . . . 17
   3.7. Valid Usage . . . 32
   3.8. `VkResult` Return Codes . . . 39
   3.9. Numeric Representation and Computation . . . 43
   3.10. Fixed-Point Data Conversions . . . 45
   3.11. String Representation . . . 47
   3.12. Common Object Types . . . 47
   3.13. API Name Aliases . . . 79
4. Initialization . . . 80
   4.1. Command Function Pointers . . . 80
   4.2. Instances . . . 83
5. Devices and Queues . . . 92
   5.1. Physical Devices . . . 92
   5.2. Devices . . . 122
   5.3. Queues . . . 140
6. Command Buffers . . . 147
   6.1. Command Buffer Lifecycle . . . 147
   6.2. Command Pools . . . 149
   6.3. Command Buffer Allocation and Management . . . 156
   6.4. Command Buffer Recording . . . 160
   6.5. Command Buffer Submission . . . 169
   6.6. Queue Forward Progress . . . 192
   6.7. Secondary Command Buffer Execution . . . 192
   6.8. Nested Command Buffers . . . 199
   6.9. Command Buffer Device Mask . . . 200
7. Synchronization and Cache Control . . . 203
   7.1. Execution and Memory Dependencies . . . 203
   7.2. Implicit Synchronization Guarantees . . . 236
   7.3. Fences . . . 238

7.4. Semaphores . . . 263
7.5. Events . . . 292
7.6. Pipeline Barriers . . . 318
7.7. Memory Barriers . . . 328
7.8. Wait Idle Operations . . . 381
7.9. Host Write Ordering Guarantees . . . 382
7.10. Synchronization and Multiple Physical Devices . . . 383
7.11. Calibrated Timestamps . . . 383
8. Render Pass . . . 387
8.1. Render Pass Objects . . . 409
8.2. Render Pass Creation . . . 410
8.3. Render Pass Compatibility . . . 468
8.4. Framebuffers . . . 469
8.5. Render Pass Load Operations . . . 481
8.6. Render Pass Store Operations . . . 482
8.7. Render Pass Multisample Resolve Operations . . . 483
8.8. Render Pass Commands . . . 485
8.9. Common Render Pass Data Races (Informative) . . . 511
9. Shaders . . . 514
9.1. Shader Objects . . . 514
9.2. Shader Modules . . . 534
9.3. Shader Module Identifiers . . . 539
9.4. Binding Shaders . . . 542
9.5. Shader Execution . . . 542
9.6. Shader Memory Access Ordering . . . 543
9.7. Shader Inputs and Outputs . . . 544
9.8. Vertex Shaders . . . 544
9.9. Tessellation Control Shaders . . . 545
9.10. Tessellation Evaluation Shaders . . . 547
9.11. Geometry Shaders . . . 547
9.12. Fragment Shaders . . . 547
9.13. Compute Shaders . . . 547
9.14. Ray Generation Shaders . . . 548
9.15. Intersection Shaders . . . 548
9.16. Any-Hit Shaders . . . 548
9.17. Closest Hit Shaders . . . 549
9.18. Miss Shaders . . . 549
9.19. Callable Shaders . . . 549
9.20. Interpolation Decorations . . . 549
9.21. Static Use . . . 550
9.22. Scope . . . 550

9.23. Derivative Operations . . . 554
9.24. Helper Invocations . . . 556
9.25. Cooperative Matrices . . . 557
9.26. Validation Cache . . . 560
10. Pipelines . . . 568
10.1. Multiple Pipeline Creation . . . 569
10.2. Compute Pipelines . . . 570
10.3. Graphics Pipelines . . . 581
10.4. Ray Tracing Pipelines . . . 625
10.5. Pipeline Destruction . . . 642
10.6. Pipeline Derivatives . . . 643
10.7. Pipeline Cache . . . 644
10.8. Pipeline Binaries . . . 652
10.9. Specialization Constants . . . 664
10.10. Pipeline Libraries . . . 668
10.11. Pipeline Binding . . . 669
10.12. Dynamic State . . . 673
10.13. Pipeline Properties and Shader Information . . . 673
11. Memory Allocation . . . 683
11.1. Host Memory . . . 683
11.2. Device Memory . . . 690
12. Resource Creation . . . 761
12.1. Buffers . . . 761
12.2. Buffer Views . . . 775
12.3. Images . . . 780
12.4. Image Layouts . . . 822
12.5. Image Views . . . 828
12.6. Acceleration Structures . . . 848
12.7. Micromaps . . . 862
12.8. Resource Memory Association . . . 871
12.9. Resource Sharing Mode . . . 903
12.10. Memory Aliasing . . . 905
13. Samplers . . . 908
13.1. Sampler $Y'C_BC_R$ Conversion . . . 919
14. Resource Descriptors . . . 930
14.1. Descriptor Types . . . 930
14.2. Descriptor Sets . . . 934
14.3. Physical Storage Buffer Access . . . 1010
15. Shader Interfaces . . . 1014
15.1. Shader Input and Output Interfaces . . . 1014
15.2. Vertex Input Interface . . . 1018

15.3. Fragment Output Interface . . . 1018
15.4. Fragment Tile Image Interface . . . 1022
15.5. Fragment Input Attachment Interface . . . 1023
15.6. Ray Tracing Pipeline Interface . . . 1027
15.7. Shader Resource Interface . . . 1028
15.8. Built-In Variables . . . 1036
16. Image Operations . . . 1066
16.1. Image Operations Overview . . . 1066
16.2. Conversion Formulas . . . 1069
16.3. Texel Input Operations . . . 1071
16.4. Texel Output Operations . . . 1089
16.5. Normalized Texel Coordinate Operations . . . 1090
16.6. Unnormalized Texel Coordinate Operations . . . 1097
16.7. Integer Texel Coordinate Operations . . . 1099
16.8. Image Sample Operations . . . 1099
16.9. Image Operation Steps . . . 1103
16.10. Image Query Instructions . . . 1104
17. Fragment Density Map Operations . . . 1105
17.1. Fragment Density Map Operations Overview . . . 1105
17.2. Fetch Density Value . . . 1105
17.3. Fragment Area Conversion . . . 1106
18. Queries . . . 1108
18.1. Query Pools . . . 1108
18.2. Query Operation . . . 1115
18.3. Occlusion Queries . . . 1142
18.4. Pipeline Statistics Queries . . . 1143
18.5. Timestamp Queries . . . 1145
18.6. Performance Queries . . . 1152
18.7. Transform Feedback Queries . . . 1155
18.8. Primitives Generated Queries . . . 1155
18.9. Result Status Queries . . . 1156
18.10. Video Encode Feedback Queries . . . 1156
19. Clear Commands . . . 1159
19.1. Clearing Images Outside a Render Pass Instance . . . 1159
19.2. Clearing Images Inside a Render Pass Instance . . . 1164
19.3. Clear Values . . . 1168
19.4. Filling Buffers . . . 1170
19.5. Updating Buffers . . . 1172
20. Copy Commands . . . 1175
20.1. Copying Data Between Buffers . . . 1175
20.2. Copying Data Between Images . . . 1181

20.3. Copying Data Between Buffers and Images . . . 1199
20.4. Image Copies With Scaling . . . 1242
20.5. Resolving Multisample Images . . . 1258
21. Drawing Commands . . . 1270
21.1. Primitive Topologies . . . 1273
21.2. Primitive Order . . . 1284
21.3. Programmable Primitive Shading . . . 1285
22. Fixed-Function Vertex Processing . . . 1559
22.1. Vertex Attributes . . . 1559
22.2. Vertex Input Description . . . 1563
22.3. Vertex Attribute Divisor in Instanced Rendering . . . 1576
22.4. Vertex Input Address Calculation . . . 1578
23. Tessellation . . . 1580
23.1. Tessellator . . . 1580
23.2. Tessellator Patch Discard . . . 1583
23.3. Tessellator Spacing . . . 1584
23.4. Tessellation Primitive Ordering . . . 1584
23.5. Tessellator Vertex Winding Order . . . 1585
23.6. Triangle Tessellation . . . 1585
23.7. Quad Tessellation . . . 1587
23.8. Isoline Tessellation . . . 1589
23.9. Tessellation Point Mode . . . 1589
23.10. Tessellation Pipeline State . . . 1589
24. Geometry Shading . . . 1594
24.1. Geometry Shader Input Primitives . . . 1594
24.2. Geometry Shader Output Primitives . . . 1595
24.3. Multiple Invocations of Geometry Shaders . . . 1595
24.4. Geometry Shader Primitive Ordering . . . 1595
25. Fixed-Function Vertex Post-Processing . . . 1596
25.1. Transform Feedback . . . 1596
25.2. Flat Shading . . . 1605
25.3. Primitive Clipping . . . 1608
25.4. Clipping Shader Outputs . . . 1615
25.5. Coordinate Transformations . . . 1615
25.6. Controlling the Viewport . . . 1616
26. Rasterization . . . 1626
26.1. Discarding Primitives Before Rasterization . . . 1631
26.2. Controlling the Vertex Stream Used for Rasterization . . . 1632
26.3. Rasterization Order . . . 1635
26.4. Multisampling . . . 1635
26.5. Custom Sample Locations . . . 1641

26.6. Fragment Shading Rates . . . 1646
26.7. Sample Shading . . . 1654
26.8. Barycentric Interpolation . . . 1654
26.9. Points . . . 1657
26.10. Line Segments . . . 1658
26.11. Polygons . . . 1670
27. Fragment Operations . . . 1692
27.1. Discard Rectangles Test . . . 1693
27.2. Scissor Test . . . 1699
27.3. Sample Mask Test . . . 1701
27.4. Fragment Shading . . . 1703
27.5. Multisample Coverage . . . 1708
27.6. Depth and Stencil Operations . . . 1711
27.7. Depth Bounds Test . . . 1712
27.8. Stencil Test . . . 1715
27.9. Depth Test . . . 1725
27.10. Sample Counting . . . 1732
27.11. Coverage Reduction . . . 1732
28. The Framebuffer . . . 1734
28.1. Blending . . . 1734
28.2. Logical Operations . . . 1762
28.3. Color Write Mask . . . 1767
28.4. Color Write Enable . . . 1768
29. Dispatching Commands . . . 1771
30. Device-Generated Commands . . . 1791
30.1. Indirect Commands Layout . . . 1791
30.2. Indirect Commands Generation and Execution . . . 1811
31. Sparse Resources . . . 1865
31.1. Sparse Resource Features . . . 1865
31.2. Sparse Buffers and Fully-Resident Images . . . 1866
31.3. Sparse Partially-Resident Buffers . . . 1867
31.4. Sparse Partially-Resident Images . . . 1867
31.5. Sparse Memory Aliasing . . . 1875
31.6. Sparse Resource Implementation Guidelines (Informative) . . . 1876
31.7. Sparse Resource API . . . 1877
32. Window System Integration (WSI) . . . 1905
32.1. WSI Platform . . . 1905
32.2. WSI Surface . . . 1905
32.3. Presenting Directly to Display Devices . . . 1919
32.4. Querying for WSI Support . . . 1945
32.5. Surface Queries . . . 1949

32.6. Full Screen Exclusive Control . . . 1975
32.7. Device Group Queries . . . 1978
32.8. Present Wait . . . 1984
32.9. WSI Swapchain . . . 1984
32.10. HDR Metadata . . . 2025
33. Deferred Host Operations . . . 2028
33.1. Requesting Deferral . . . 2028
33.2. Deferred Host Operations API . . . 2029
34. Acceleration Structures . . . 2035
34.1. Acceleration Structures . . . 2035
34.2. Host Acceleration Structure Operations . . . 2085
35. Micromap . . . 2101
35.1. Micromaps . . . 2101
35.2. Host Micromap Operations . . . 2120
36. Ray Traversal . . . 2130
36.1. Ray Intersection Candidate Determination . . . 2130
36.2. Ray Intersection Culling . . . 2133
36.3. Ray Intersection Confirmation . . . 2137
36.4. Ray Closest Hit Determination . . . 2139
36.5. Ray Result Determination . . . 2139
37. Ray Tracing . . . 2140
37.1. Shader Call Instructions . . . 2140
37.2. Ray Tracing Commands . . . 2141
37.3. Shader Binding Table . . . 2171
37.4. Ray Tracing Pipeline Stack . . . 2173
37.5. Ray Tracing Capture Replay . . . 2174
38. Video Coding . . . 2175
38.1. Video Picture Resources . . . 2175
38.2. Decoded Picture Buffer . . . 2177
38.3. Video Profiles . . . 2179
38.4. Video Capabilities . . . 2189
38.5. Video Sessions . . . 2198
38.6. Video Profile Compatibility . . . 2211
38.7. Video Session Parameters . . . 2213
38.8. Video Coding Scope . . . 2237
38.9. Video Coding Control . . . 2248
38.10. Inline Queries . . . 2251
38.11. Video Decode Operations . . . 2252
38.12. H.264 Decode Operations . . . 2269
38.13. H.265 Decode Operations . . . 2283
38.14. AV1 Decode Operations . . . 2297

38.15. Video Encode Operations . . . 2308
38.16. Video Encode Rate Control . . . 2345
38.17. Video Encode Quantization Maps . . . 2354
38.18. H.264 Encode Operations . . . 2362
38.19. H.265 Encode Operations . . . 2398
38.20. AV1 Encode Operations . . . 2440
39. Extending Vulkan . . . 2480
39.1. Instance and Device Functionality . . . 2480
39.2. Core Versions . . . 2480
39.3. Layers . . . 2483
39.4. Extensions . . . 2487
39.5. Extension Dependencies . . . 2491
39.6. Compatibility Guarantees (Informative) . . . 2492
40. Features . . . 2497
40.1. Feature Requirements . . . 2590
41. Limits . . . 2595
41.1. Limit Requirements . . . 2648
41.2. Additional Multisampling Capabilities . . . 2664
42. Formats . . . 2666
42.1. Format Definition . . . 2666
42.2. Format Properties . . . 2711
42.3. Required Format Support . . . 2729
43. Additional Capabilities . . . 2751
43.1. Additional Image Capabilities . . . 2751
43.2. Additional Buffer Capabilities . . . 2772
43.3. Optional Semaphore Capabilities . . . 2775
43.4. Optional Fence Capabilities . . . 2780
43.5. Timestamp Calibration Capabilities . . . 2785
44. Debugging . . . 2787
44.1. Debug Utilities . . . 2790
44.2. Frame Boundary . . . 2811
Appendix A: Vulkan Environment for SPIR-V . . . 2814
Versions and Formats . . . 2814
Capabilities . . . 2814
Validation Rules Within a Module . . . 2823
Precision and Operation of SPIR-V Instructions . . . 2847
Signedness of SPIR-V Image Accesses . . . 2853
Image Format and Type Matching . . . 2853
Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types . . . 2855
Compatibility Between SPIR-V Image Formats and Vulkan Formats . . . 2856
Ray Query Precision and Operation . . . 2857

Appendix B: Memory Model . . . 2858
Agent . . . 2858
Memory Location . . . 2858
Allocation . . . 2858
Memory Operation . . . 2859
Reference . . . 2859
Program-Order . . . 2859
Shader Call Related . . . 2860
Shader Call Order . . . 2860
Scope . . . 2860
Atomic Operation . . . 2861
Scoped Modification Order . . . 2861
Memory Semantics . . . 2862
Release Sequence . . . 2863
Synchronizes-With . . . 2863
System-Synchronizes-With . . . 2865
Private vs. Non-Private . . . 2866
Inter-Thread-Happens-Before . . . 2866
Happens-Before . . . 2867
Availability and Visibility . . . 2867
Availability, Visibility, and Domain Operations . . . 2869
Availability and Visibility Semantics . . . 2870
Per-Instruction Availability and Visibility Semantics . . . 2870
Location-Ordered . . . 2871
Data Race . . . 2872
Visible-To . . . 2872
Acyclicity . . . 2872
Shader I/O . . . 2873
Deallocation . . . 2873
Descriptions (Informative) . . . 2874
Tessellation Output Ordering . . . 2874
Cooperative Matrix Memory Access . . . 2875
Appendix C: Compressed Image Formats . . . 2876
Block-Compressed Image Formats . . . 2877
ETC Compressed Image Formats . . . 2878
ASTC Compressed Image Formats . . . 2879
Appendix D: Core Revisions (Informative) . . . 2881
Vulkan Version 1.0 . . . 2881
Appendix E: Layers & Extensions (Informative) . . . 2895
Extension Dependencies . . . 2895
Extension Interactions . . . 2895

List of Current Extensions . . . 2896
List of Provisional Extensions . . . 3422
List of Deprecated Extensions . . . 3425
Appendix F: API Boilerplate . . . 3428
Vulkan Header Files . . . 3428
Window System-Specific Header Control (Informative) . . . 3432
Provisional Extension Header Control (Informative) . . . 3434
Video Std Headers . . . 3434
Appendix G: Invariance . . . 3436
Repeatability . . . 3436
Multi-Pass Algorithms . . . 3436
Invariance Rules . . . 3436
Tessellation Invariance . . . 3438
Appendix H: Lexicon . . . 3440
Glossary . . . 3440
Common Abbreviations . . . 3467
Video-Specific Abbreviations . . . 3468
Prefixes . . . 3470
Appendix I: Credits (Informative) . . . 3471
Working Group Contributors to Vulkan . . . 3471
Other Credits . . . 3479

Chapter 1. Preamble

Copyright 2014-2024 The Khronos Group Inc.

This Specification is protected by copyright laws and contains material proprietary to Khronos. Except as described by these terms, it or any components may not be reproduced, republished, distributed, transmitted, displayed, broadcast or otherwise exploited in any manner without the express prior written permission of Khronos.

Khronos grants a conditional copyright license to use and reproduce the unmodified Specification for any purpose, without fee or royalty, EXCEPT no licenses to any patent, trademark or other intellectual property rights are granted under these terms.

Khronos makes no, and expressly disclaims any, representations or warranties, express or implied, regarding this Specification, including, without limitation: merchantability, fitness for a particular purpose, non-infringement of any intellectual property, correctness, accuracy, completeness, timeliness, and reliability. Under no circumstances will Khronos, or any of its Promoters, Contributors or Members, or their respective partners, officers, directors, employees, agents or representatives be liable for any damages, whether direct, indirect, special or consequential damages for lost revenues, lost profits, or otherwise, arising from or in connection with these materials.

This Specification has been created under the Khronos Intellectual Property Rights Policy, which is Attachment A of the Khronos Group Membership Agreement available at https://www.khronos.org/files/member_agreement.pdf. Parties desiring to implement the Specification and make use of Khronos trademarks in relation to that implementation, and receive reciprocal patent license protection under the Khronos Intellectual Property Rights Policy must become Adopters and confirm the implementation as conformant under the process defined by Khronos for this Specification; see https://www.khronos.org/adopters.

This Specification contains substantially unmodified functionality from, and is a successor to, Khronos specifications including OpenGL, OpenGL ES and OpenCL.

The Khronos Intellectual Property Rights Policy defines the terms 'Scope', 'Compliant Portion', and 'Necessary Patent Claims'.

Some parts of this Specification are purely informative and so are EXCLUDED the Scope of this Specification. The Document Conventions section of the Introduction defines how these parts of the Specification are identified.

Where this Specification uses technical terminology, defined in the Glossary or otherwise, that refer to enabling technologies that are not expressly set forth in this Specification, those enabling technologies are EXCLUDED from the Scope of this Specification. For clarity, enabling technologies not disclosed with particularity in this Specification (e.g. semiconductor manufacturing technology, hardware architecture, processor architecture or microarchitecture, memory architecture, compiler technology, object oriented technology, basic operating system technology, compression technology, algorithms, and so on) are NOT to be considered expressly set forth; only those application program interfaces and data structures disclosed with particularity are included in the Scope of this Specification.

For purposes of the Khronos Intellectual Property Rights Policy as it relates to the definition of Necessary Patent Claims, all recommended or optional features, behaviors and functionality set forth in this Specification, if implemented, are considered to be included as Compliant Portions.

Where this Specification identifies specific sections of external references, only those specifically identified sections define normative functionality. The Khronos Intellectual Property Rights Policy excludes external references to materials and associated enabling technology not created by Khronos from the Scope of this Specification, and any licenses that may be required to implement such referenced materials and associated technologies must be obtained separately and may involve royalty payments.

Khronos and Vulkan are registered trademarks, and SPIR-V is a trademark of The Khronos Group Inc. OpenCL is a trademark of Apple Inc., used under license by Khronos. OpenGL is a registered trademark and the OpenGL ES logo is a trademark of Hewlett Packard Enterprise, used under license by Khronos. ASTC is a trademark of ARM Holdings PLC. All other product names, trademarks, and/or company names are used solely for identification and belong to their respective owners.

Chapter 2. Introduction

This document, referred to as the “Vulkan Specification” or just the “Specification” hereafter, describes the Vulkan Application Programming Interface (API). Vulkan is a C99 API designed for explicit control of low-level graphics and compute functionality.

The canonical version of the Specification is available in the official Vulkan Registry (https://registry.khronos.org/vulkan/). The source files used to generate the Vulkan specification are stored in the Vulkan Documentation Repository (https://github.com/KhronosGroup/Vulkan-Docs).

The source repository additionally has a public issue tracker and allows the submission of pull requests that improve the specification.

2.1. Document Conventions

The Vulkan specification is intended for use by both implementors of the API and application developers seeking to make use of the API, forming a contract between these parties. Specification text may address either party; typically the intended audience can be inferred from context, though some sections are defined to address only one of these parties. (For example, Valid Usage sections only address application developers). Any requirements, prohibitions, recommendations or options defined in specification text are imposed only on the audience of that text.

2.1.1. Normative Requirements

The Vulkan Specification uses a combination of normative terminology and normative descriptions to express the requirements that it imposes on applications and implementations. An application which complies with all normative requirements imposed on applications is said to make **valid use** of the API; failing to comply with such requirements results in undefined behavior, as discussed in the Valid Usage section below. In the context of this document, an implementation which complies with all normative requirements imposed on implementations is said to be **conformant**.

NOTE	The Khronos Group imposes additional requirements on implementors who wish to make public statements describing their Vulkan implementations as conformant. These include signing the Vulkan Adopter’s Agreement, paying the associated fee, and making a successful conformance test submission to the Khronos Conformance Process. For details see the Khronos Trademark Guidelines (https://www.khronos.org/legal/khronos-trademark-guidelines).

Normative Terminology

Within this specification, the key words **must**, **required**, **should**, **may**, and **optional** are to be interpreted as described in RFC 2119 - Key words for use in RFCs to Indicate Requirement Levels (https://www.ietf.org/rfc/rfc2119.txt). The additional key word **optionally** is an alternate form of **optional**, for use where grammatically appropriate. These key words are highlighted in the specification to indicate that they are being used in a specific technical sense.

The additional key words **can** and **cannot** are to be interpreted as describing the capabilities of an application, as follows:

can

This word means that the application is able to perform the action described.

cannot

This word means that the API and/or the execution environment provide no mechanism through which the application can express or accomplish the action described.

These key words are never used in text addressing implementors.

NOTE	There is an important distinction between **cannot** and **must not**, as used in this Specification. **Cannot** refers to something the API provides no way for the application to express or accomplish. **Must not** describes something that the application is able to express, but that is not valid use of the API, and will have undefined and potentially unrecoverable consequences.

Normative Descriptions

In the Vulkan Specification, the normative term **must** is primarily used to describe **application** behavior, and in particular to constrain what inputs or commands issued by the application to the implementation are considered valid.

To constrain **implementation** behavior, the specification sometimes uses **must**, but more often simply describes the behavior of the implementation in response to specified commands and inputs. Unless explicitly stated otherwise, such references to implementation behavior describe the behavior of **conformant** implementations, and express normative requirements which an implementation must satisfy in order to conform to the specification. For example, if the specification says “Under **specified condition**, the error code `VK_ERROR_FEATURE_NOT_PRESENT` is returned”, that behavior is a requirement of the specification, and an implementation which does not return that error code under that condition is not conformant.

When the normative terms **may**, **should**, or **optional** are used to describe implementation behavior, they define alternative or optional behaviors which a conformant implementation may or may not exhibit. Such statements are also normative. For example, if the specification says "Under **specified condition**, the implementation **should** return A but **may** instead return B", then an implementation that returns either A or B under that condition is conformant (assuming it does not violate other normative requirements), while an implementation that returns anything else is not.

2.1.2. Normative References

References to external documents are considered normative references if the Specification uses normative terminology or normative descriptions to refer to them or their requirements, either as a whole or in part.

The following documents are referenced by normative sections of the specification:

IEEE. August, 2008. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2008. https://dx.doi.org/10.1109/IEEESTD.2008.4610935 .

Andrew Garrard. *Khronos Data Format Specification, version 1.3*. https://registry.khronos.org/DataFormat/specs/1.3/dataformat.1.3.html .

John Kessenich. *SPIR-V Extended Instructions for GLSL, Version 1.00* (February 10, 2016). https://registry.khronos.org/spir-v/ .

John Kessenich, Boaz Ouriel, and Raun Krisch. *SPIR-V Specification, Version 1.5, Revision 3, Unified* (April 24, 2020). https://registry.khronos.org/spir-v/ .

ITU-T. *H.264 Advanced Video Coding for Generic Audiovisual Services* (August, 2021). https://www.itu.int/rec/T-REC-H.264-202108-I/ .

ITU-T. *H.265 High Efficiency Video Coding* (August, 2021). https://www.itu.int/rec/T-REC-H.265-202108-S/ .

Alliance for Open Media. *AV1 Bitstream & Decoding Process Specification* (January 8, 2019). https://aomediacodec.github.io/av1-spec/av1-spec.pdf .

Jon Leech. *The Khronos Vulkan API Registry* (February 26, 2023). https://registry.khronos.org/vulkan/specs/1.3/registry.html .

Jon Leech and Tobias Hector. *Vulkan Documentation and Extensions: Procedures and Conventions* (February 26, 2023). https://registry.khronos.org/vulkan/specs/1.3/styleguide.html .

Architecture of the Vulkan Loader Interfaces (October, 2021). https://github.com/KhronosGroup/Vulkan-Loader/blob/main/docs/LoaderInterfaceArchitecture.md .

2.1.3. Informative Language

Some language in the specification is purely informative, intended to provide background information or make suggestions to implementors or developers. Such language does not impose normative requirements on implementations or applications.

All NOTEs are implicitly informative.

If an entire chapter, section, or appendix contains only informative language, its title will be suffixed with “(Informative)”. Unless so noted in the title, all chapters, sections, and appendices in this document are normative.

2.1.4. Technical Terminology

The Vulkan Specification makes use of common engineering and graphics terms such as **Pipeline**, **Shader**, and **Host** to identify and describe Vulkan API constructs and their attributes, states, and behaviors. The Glossary defines the basic meanings of these terms in the context of the Specification. The Specification text provides fuller definitions of the terms and may elaborate, extend, or clarify the Glossary definitions. When a term defined in the Glossary is used in normative language within the Specification, the definitions within the Specification govern and supersede any meanings the terms may have in other technical contexts (i.e. outside the Specification).

2.1.5. Ratification

Ratification of a Vulkan core version or extension is a status conferred by vote of the Khronos Board of Promoters, bringing that core version or extension under the umbrella of the Khronos IP Rights Policy.

All Vulkan core versions and KHR extensions (including provisional specifications) are ratified, as are some multi-vendor EXT extensions. Ratification status of extensions is described in the Layers & Extensions (Informative) appendix.

NOTE

Ratification status is primarily of interest to IHVs developing GPU hardware and Vulkan implementations. For developers, ratification does not necessarily mean that an extension is “better”, has a more stable API, or is more widely supported than alternative ways of achieving that functionality.

Interactions between ratified and non-ratified extensions are not themselves ratified.

Chapter 3. Fundamentals

This chapter introduces fundamental concepts including the Vulkan architecture and execution model, API syntax, queues, pipeline configurations, numeric representation, state and state queries, and the different types of objects and shaders. It provides a framework for interpreting more specific descriptions of commands and behavior in the remainder of the Specification.

3.1. Host and Device Environment

The Vulkan Specification assumes and requires: the following properties of the host environment with respect to Vulkan implementations:

- The host **must** have runtime support for 8, 16, 32 and 64-bit signed and unsigned twos-complement integers, all addressable at the granularity of their size in bytes.
- The host **must** have runtime support for 32- and 64-bit floating-point types satisfying the range and precision constraints in the Floating-Point Computation section.
- The representation and endianness of these types on the host **must** match the representation and endianness of the same types on every physical device supported.

NOTE Since a variety of data types and structures in Vulkan **may** be accessible by both host and physical device operations, the implementation **should** be able to access such data efficiently in both paths in order to facilitate writing portable and performant applications.

3.2. Execution Model

This section outlines the execution model of a Vulkan system.

Vulkan exposes one or more *devices*, each of which exposes one or more *queues* which **may** process work asynchronously to one another. The set of queues supported by a device is partitioned into *families*. Each family supports one or more types of functionality and **may** contain multiple queues with similar characteristics. Queues within a single family are considered *compatible* with one another, and work produced for a family of queues **can** be executed on any queue within that family. This specification defines the following types of functionality that queues **may** support: graphics, compute, video decode, video encode, sparse memory management, and transfer.

NOTE A single device **may** report multiple similar queue families rather than, or as well as, reporting multiple members of one or more of those families. This indicates that while members of those families have similar capabilities, they are *not* directly compatible with one another.

Device memory is explicitly managed by the application. Each device **may** advertise one or more heaps, representing different areas of memory. Memory heaps are either device-local or host-local, but are always visible to the device. Further detail about memory heaps is exposed via memory types available on that heap. Examples of memory areas that **may** be available on an implementation include:

- *device-local* is memory that is physically connected to the device.
- *device-local, host visible* is device-local memory that is visible to the host.
- *host-local, host visible* is memory that is local to the host and visible to the device and host.

On other architectures, there **may** only be a single heap that **can** be used for any purpose.

3.2.1. Queue Operation

Vulkan queues provide an interface to the execution engines of a device. Commands for these execution engines are recorded into command buffers ahead of execution time, and then submitted to a queue for execution. Once submitted to a queue, command buffers will begin and complete execution without further application intervention, though the order of this execution is dependent on a number of implicit and explicit ordering constraints.

Work is submitted to queues using *queue submission commands* that typically take the form `vkQueue*` (e.g. vkQueueSubmit , vkQueueBindSparse), and **can** take a list of semaphores upon which to wait before work begins and a list of semaphores to signal once work has completed. The work itself, as well as signaling and waiting on the semaphores are all *queue operations*. Queue submission commands return control to the application once queue operations have been submitted - they do not wait for completion.

There are no implicit ordering constraints between queue operations on different queues, or between queues and the host, so these **may** operate in any order with respect to each other. Explicit ordering constraints between different queues or with the host **can** be expressed with semaphores and fences.

Command buffer submissions to a single queue respect submission order and other implicit ordering guarantees, but otherwise **may** overlap or execute out of order. Other types of batches and queue submissions against a single queue (e.g. sparse memory binding) have no implicit ordering constraints with any other queue submission or batch. Additional explicit ordering constraints between queue submissions and individual batches can be expressed with semaphores and fences.

Before a fence or semaphore is signaled, it is guaranteed that any previously submitted queue operations have completed execution, and that memory writes from those queue operations are available to future queue operations. Waiting on a signaled semaphore or fence guarantees that previous writes that are available are also visible to subsequent commands.

Command buffer boundaries, both between primary command buffers of the same or different batches or submissions as well as between primary and secondary command buffers, do not introduce any additional ordering constraints. In other words, submitting the set of command buffers (which **can** include executing secondary command buffers) between any semaphore or fence operations execute the recorded commands as if they had all been recorded into a single primary command buffer, except that the current state is reset on each boundary. Explicit ordering constraints **can** be expressed with explicit synchronization primitives.

There are a few implicit ordering guarantees between commands within a command buffer, but only covering a subset of execution. Additional explicit ordering constraints can be expressed with the various explicit synchronization primitives.

NOTE Implementations have significant freedom to overlap execution of work submitted to a queue, and this is common due to deep pipelining and parallelism in Vulkan devices.

Commands recorded in command buffers can perform actions, set state that persists across commands, synchronize other commands, or indirectly launch other commands, with some commands fulfilling several of these roles. The “Command Properties” section for each such command lists which of these roles the command takes:

Action

Action commands perform operations that can update values in memory. E.g. draw commands, dispatch commands.

State

State setting commands update the current state of a command buffer, affecting the operation of future action commands.

Synchronization

Synchronization commands impose ordering constraints on action commands, by introducing explicit execution and memory dependencies.

Indirection

Indirection commands execute other commands which were not directly recorded in the same command buffer.

NOTE In the absence of explicit synchronization or implicit ordering guarantees, action commands may overlap execution or execute out of order, potentially leading to data races. However, such reordering does not affect the current state observed by any action command. Each action command uses the state in effect at the point where the command occurs in the command buffer, regardless of when it is executed.

3.3. Object Model

The devices, queues, and other entities in Vulkan are represented by Vulkan objects. At the API level, all objects are referred to by handles. There are two classes of handles, dispatchable and non-dispatchable. *Dispatchable* handle types are a pointer to an opaque type. This pointer **may** be used by layers as part of intercepting API commands, and thus each API command takes a dispatchable type as its first parameter. Each object of a dispatchable type **must** have a unique handle value during its lifetime.

Non-dispatchable handle types are a 64-bit integer type whose meaning is implementation-dependent. Non-dispatchable handles **may** encode object information directly in the handle rather than acting as a reference to an underlying object, and thus **may** not have unique handle values. If handle values are not unique, then destroying one such handle **must** not cause identical handles of other types to become invalid, and **must** not cause identical handles of the same type to become invalid if that handle value has been created more times than it has been destroyed.

All objects created or allocated from a VkDevice (i.e. with a VkDevice as the first parameter) are private to that device, and **must** not be used on other devices.

3.3.1. Object Lifetime

Objects are created or allocated by vkCreate* and vkAllocate* commands, respectively. Once an object is created or allocated, its "structure" is considered to be immutable, though the contents of certain object types is still free to change. Objects are destroyed or freed by vkDestroy* and vkFree* commands, respectively.

Objects that are allocated (rather than created) take resources from an existing pool object or memory heap, and when freed return resources to that pool or heap. While object creation and destruction are generally expected to be low-frequency occurrences during runtime, allocating and freeing objects **can** occur at high frequency. Pool objects help accommodate improved performance of the allocations and frees.

It is an application's responsibility to track the lifetime of Vulkan objects, and not to destroy them while they are still in use.

The ownership of application-owned memory is immediately acquired by any Vulkan command it is passed into. Ownership of such memory **must** be released back to the application at the end of the duration of the command, unless that command was deferred, so that the application **can** alter or free this memory as soon as all the commands that acquired it have returned. If the command was deferred, ownership of such memory is released back to the application when the deferred operation is complete.

The following object types are consumed when they are passed into a Vulkan command and not further accessed by the objects they are used to create. They **must** not be destroyed in the duration of any API command they are passed into:

- VkShaderModule
- VkPipelineCache
- VkValidationCacheEXT

A VkRenderPass or VkPipelineLayout object passed as a parameter to create another object is not further accessed by that object after the duration of the command it is passed into. A VkRenderPass used in a command buffer follows the rules described below.

VkDescriptorSetLayout objects **may** be accessed by commands that operate on descriptor sets allocated using that layout, and those descriptor sets **must** not be updated with vkUpdateDescriptorSets after the descriptor set layout has been destroyed. Otherwise, a VkDescriptorSetLayout object is no longer referenced by an API command it is passed into once host execution of that command completes.

The application **must** not destroy any other type of Vulkan object until all uses of that object by the device (such as via command buffer execution) have completed.

The following Vulkan objects **must** not be destroyed while any command buffers using the object are in the pending state:

- VkEvent
- VkQueryPool
- VkBuffer
- VkBufferView
- VkImage
- VkImageView
- VkPipeline
- VkSampler
- VkSamplerYcbcrConversion
- VkDescriptorPool
- VkFramebuffer
- VkRenderPass
- VkCommandBuffer
- VkCommandPool
- VkDeviceMemory
- VkDescriptorSet
- VkIndirectCommandsLayoutEXT
- VkIndirectExecutionSetEXT
- VkAccelerationStructureKHR
- VkVideoSessionKHR
- VkVideoSessionParametersKHR

Destroying these objects will move any command buffers that are in the recording or executable state, and are using those objects, to the invalid state.

The following Vulkan objects **must** not be destroyed while any queue is executing commands that use the object:

- VkFence
- VkSemaphore
- VkCommandBuffer
- VkCommandPool

In general, objects **can** be destroyed or freed in any order, even if the object being freed is involved in the use of another object (e.g. use of a resource in a view, use of a view in a descriptor set, use of a pipeline library in another pipeline, use of a referenced object for additional shader-related states in a set, use of a bottom level acceleration structure in an instance referenced by a top level acceleration structure, use of an object in a command buffer, binding of a memory allocation to a resource), as long as any object that uses the freed object is not further used in any way except to be destroyed or to be reset in such a way that it no longer uses the other object (such as resetting a

command buffer). If the object has been reset, then it **can** be used as if it never used the freed object. An exception to this is when there is a parent/child relationship between objects. In this case, the application **must** not destroy a parent object before its children, except when the parent is explicitly defined to free its children when it is destroyed (e.g. for pool objects, as defined below).

`VkCommandPool` objects are parents of `VkCommandBuffer` objects. `VkDescriptorPool` objects are parents of `VkDescriptorSet` objects. `VkDevice` objects are parents of many object types (all that take a `VkDevice` as a parameter to their creation).

The following Vulkan objects have specific restrictions for when they **can** be destroyed:

- `VkQueue` objects **cannot** be explicitly destroyed. Instead, they are implicitly destroyed when the `VkDevice` object they are retrieved from is destroyed.
- Destroying a pool object implicitly frees all objects allocated from that pool. Specifically, destroying `VkCommandPool` frees all `VkCommandBuffer` objects that were allocated from it, and destroying `VkDescriptorPool` frees all `VkDescriptorSet` objects that were allocated from it.
- `VkDevice` objects **can** be destroyed when all `VkQueue` objects retrieved from them are idle, and all objects created from them have been destroyed.
    - This includes the following objects:
        - `VkFence`
        - `VkSemaphore`
        - `VkEvent`
        - `VkQueryPool`
        - `VkBuffer`
        - `VkBufferView`
        - `VkImage`
        - `VkImageView`
        - `VkShaderModule`
        - `VkPipelineCache`
        - `VkPipeline`
        - `VkPipelineLayout`
        - `VkSampler`
        - `VkSamplerYcbcrConversion`
        - `VkDescriptorSetLayout`
        - `VkDescriptorPool`
        - `VkFramebuffer`
        - `VkRenderPass`
        - `VkCommandPool`
        - `VkCommandBuffer`

    - VkDeviceMemory
    - VkValidationCacheEXT
    - VkAccelerationStructureKHR
    - VkVideoSessionKHR
    - VkVideoSessionParametersKHR

- VkPhysicalDevice objects **cannot** be explicitly destroyed. Instead, they are implicitly destroyed when the VkInstance object they are retrieved from is destroyed.
- VkInstance objects **can** be destroyed once all VkDevice objects created from any of its VkPhysicalDevice objects have been destroyed.

3.3.2. External Object Handles

As defined above, the scope of object handles created or allocated from a VkDevice is limited to that logical device. Objects which are not in scope are said to be external. To bring an external object into scope, an external handle **must** be exported from the object in the source scope and imported into the destination scope.

NOTE	The scope of external handles and their associated resources **may** vary according to their type, but they **can** generally be shared across process and API boundaries.

3.4. Application Binary Interface

The mechanism by which Vulkan is made available to applications is platform- or implementation-defined. On many platforms the C interface described in this Specification is provided by a shared library. Since shared libraries can be changed independently of the applications that use them, they present particular compatibility challenges, and this Specification places some requirements on them.

Shared library implementations **must** use the default Application Binary Interface (ABI) of the standard C compiler for the platform, or provide customized API headers that cause application code to use the implementation's non-default ABI. An ABI in this context means the size, alignment, and layout of C data types; the procedure calling convention; and the naming convention for shared library symbols corresponding to C functions. Customizing the calling convention for a platform is usually accomplished by defining calling convention macros appropriately in vk_platform.h.

On platforms where Vulkan is provided as a shared library, library symbols beginning with "vk" and followed by a digit or uppercase letter are reserved for use by the implementation. Applications which use Vulkan **must** not provide definitions of these symbols. This allows the Vulkan shared library to be updated with additional symbols for new API versions or extensions without causing symbol conflicts with existing applications.

Shared library implementations **should** provide library symbols for commands in the highest version of this Specification they support, and for Window System Integration extensions relevant to the platform. They **may** also provide library symbols for commands defined by additional extensions.

NOTE

These requirements and recommendations are intended to allow implementors to take advantage of platform-specific conventions for SDKs, ABIs, library versioning mechanisms, etc. while still minimizing the code changes necessary to port applications or libraries between platforms. Platform vendors, or providers of the *de facto* standard Vulkan shared library for a platform, are encouraged to document what symbols the shared library provides and how it will be versioned when new symbols are added.

Applications **should** only rely on shared library symbols for commands in the minimum core version required by the application. vkGetInstanceProcAddr and vkGetDeviceProcAddr **should** be used to obtain function pointers for commands in core versions beyond the application's minimum required version.

3.5. Command Syntax and Duration

The Specification describes Vulkan commands as functions or procedures using C99 syntax. Language bindings for other languages such as C++ and JavaScript **may** allow for stricter parameter passing, or object-oriented interfaces.

Vulkan uses the standard C types for the base type of scalar parameters (e.g. types from <stdint.h>), with exceptions described below, or elsewhere in the text when appropriate:

VkBool32 represents boolean True and False values, since C does not have a sufficiently portable built-in boolean type:

```
// Provided by VK_VERSION_1_0
typedef uint32_t VkBool32;
```

VK_TRUE represents a boolean **True** (unsigned integer 1) value, and VK_FALSE a boolean **False** (unsigned integer 0) value.

All values returned from a Vulkan implementation in a VkBool32 will be either VK_TRUE or VK_FALSE.

Applications **must** not pass any other values than VK_TRUE or VK_FALSE into a Vulkan implementation where a VkBool32 is expected.

VK_TRUE is a constant representing a VkBool32 **True** value.

```
#define VK_TRUE                         1U
```

VK_FALSE is a constant representing a VkBool32 **False** value.

```
#define VK_FALSE                        0U
```

VkDeviceSize represents device memory size and offset values:

```
// Provided by VK_VERSION_1_0
typedef uint64_t VkDeviceSize;
```

VkDeviceAddress represents device buffer address values:

```
// Provided by VK_VERSION_1_0
typedef uint64_t VkDeviceAddress;
```

Commands that create Vulkan objects are of the form vkCreate* and take Vk*CreateInfo structures with the parameters needed to create the object. These Vulkan objects are destroyed with commands of the form vkDestroy*.

The last in-parameter to each command that creates or destroys a Vulkan object is pAllocator. The pAllocator parameter **can** be a non-NULL value, in which case allocations for the given object are delegated to an application provided callback. Refer to the Memory Allocation chapter for further details.

Commands that allocate Vulkan objects owned by pool objects are of the form vkAllocate*, and take Vk*AllocateInfo structures. These Vulkan objects are freed with commands of the form vkFree*. These objects do not take allocators; if host memory is needed, they will use the allocator that was specified when their parent pool was created.

Commands are recorded into a command buffer by calling API commands of the form vkCmd*. Each such command **may** have different restrictions on where it **can** be used: in a primary and/or secondary command buffer, inside and/or outside a render pass, and in one or more of the supported queue types. These restrictions are documented together with the definition of each such command.

The *duration* of a Vulkan command refers to the interval between calling the command and its return to the caller.

3.5.1. Lifetime of Retrieved Results

Information is retrieved from the implementation with commands of the form vkGet* and vkEnumerate*.

Unless otherwise specified for an individual command, the results are *invariant*; that is, they will remain unchanged when retrieved again by calling the same command with the same parameters, so long as those parameters themselves all remain valid.

3.5.2. Array Results

Some query commands of the form vkGet* and vkEnumerate* enable retrieving multiple results in the form of a return array. Such commands typically have two pointer arguments as follows:

- An element count pointer pointing to an integer variable, conventionally named as p*Count where * is the capitalized singular form of the name of the retrieved values.

- A pointer to an array where the result array is retrieved, conventionally named as `p*` where `*` is the capitalized plural form of the name of the retrieved values.

If such commands are called with the array pointer set to `NULL`, then the number of retrievable elements is returned in the variable pointed to by the element count pointer. Otherwise, the element count pointer **must** point to a variable set by the application to the number of elements in the return array, and on return the variable is overwritten with the number of elements actually written to the return array. If the input element count is less than the number of retrievable array elements, the query will write only as many elements to the return array as specified by the element count variable set by the application, and the command will return `VK_INCOMPLETE` instead of `VK_SUCCESS`, to indicate that not all retrievable array elements were returned.

NOTE

In practice, this means that applications will typically call such query commands twice:

- First, with the array pointer set to `NULL`, to retrieve the number of retrievable elements.
- Second, with the array pointer pointing to an application allocated storage for at least as many elements as indicated by the variable pointed to by the element count pointer, to retrieve at most as many of the retrievable elements.

Query commands that return one or more structures, regardless of whether they return a single or an array of structures with or without a `pNext` chain, **may** also contain arrays within those structures. Such return arrays are typically defined in the form of two members as follows:

- An integer value specifying the element count, conventionally named as `*Count` where `*` is the singular form of the name of the retrieved values.
- A pointer to an array where the result array is retrieved, conventionally named as `p*` where `*` is the capitalized plural form of the name of the retrieved values.

Analogously to query commands that return multiple results, if the command is called with the array pointer member of the output structure in question set to `NULL`, then the number of retrievable elements is returned in the element count member of that output structure. Otherwise, the element count **must** specify the number of elements in the return array, and on return the element count member is overwritten with the number of elements actually written to the return array. If the input element count is less than the number of retrievable array elements, the query will write only as many elements to the return array as specified by the input element count, and the command will return `VK_INCOMPLETE` instead of `VK_SUCCESS`, if the query command has a VkResult return type, to indicate that not all retrievable array elements were returned.

NOTE

Applications need to separately track the value they provided as the input element count member for such arrays and compare those with the returned element counts in order to determine whether the actually returned element count is smaller than the size of the return array. Another side effect of this is that it is impossible for the application to determine if the number of retrievable elements has increased beyond the provided input element count so using return arrays in output structures **should** be limited to *invariant* array results. In practice, this means that applications will typically call such query commands multiple times:

- First, with the array pointer member(s) set to `NULL`, to retrieve the number(s) of retrievable elements.
- Second, with the array pointer(s) pointing to an application allocated storage for at least as many elements as indicated by the element count member(s), to retrieve at most as many of the retrievable elements.
- Then the process **may** need to be repeated for all other newly introduced return arrays in any nested output structures indirectly specified through the previously retrieved result arrays.

Regardless of the type of query command, any array pointer member of an output structure **must** either be `NULL`, or point to an application-allocated array. Query commands **must** not return a pointer to implementation allocated storage in any output structure.

3.6. Threading Behavior

Vulkan is intended to provide scalable performance when used on multiple host threads. All commands support being called concurrently from multiple threads, but certain parameters, or components of parameters are defined to be *externally synchronized*. This means that the caller **must** guarantee that no more than one thread is using such a parameter at a given time.

More precisely, Vulkan commands use simple stores to update the state of Vulkan objects. A parameter declared as externally synchronized **may** have its contents updated at any time during the host execution of the command. If two commands operate on the same object and at least one of the commands declares the object to be externally synchronized, then the caller **must** guarantee not only that the commands do not execute simultaneously, but also that the two commands are separated by an appropriate memory barrier (if needed).

NOTE

Memory barriers are particularly relevant for hosts based on the ARM CPU architecture, which is more weakly ordered than many developers are accustomed to from x86/x64 programming. Fortunately, most higher-level synchronization primitives (like the pthread library) perform memory barriers as a part of mutual exclusion, so mutexing Vulkan objects via these primitives will have the desired effect.

Similarly the application **must** avoid any potential data hazard of application-owned memory that has its ownership temporarily acquired by a Vulkan command. While the ownership of application-owned memory remains acquired by a command the implementation **may** read the memory at any point, and it **may** write non-`const` qualified memory at any point. Parameters referring to non-`const` qualified application-owned memory are not marked explicitly as *externally synchronized* in the Specification.

If an application is using deferred host operations in a command, and that operation is successfully deferred, object parameters and application-owned memory passed to that command **may** be accessed at any time until the deferred operation is complete.

Many object types are *immutable*, meaning the objects **cannot** change once they have been created. These types of objects never need external synchronization, except that they **must** not be destroyed

while they are in use on another thread. In certain special cases mutable object parameters are internally synchronized, making external synchronization unnecessary. Any command parameters that are not labeled as externally synchronized are either not mutated by the command or are internally synchronized. Additionally, certain objects related to a command's parameters (e.g. command pools and descriptor pools) **may** be affected by a command, and **must** also be externally synchronized. These implicit parameters are documented as described below.

Parameters of commands that are externally synchronized are listed below.

Externally Synchronized Parameters

- The `instance` parameter in vkDestroyInstance
- The `device` parameter in vkDestroyDevice
- The `queue` parameter in vkQueueSubmit
- The `fence` parameter in vkQueueSubmit
- The `queue` parameter in vkQueueWaitIdle
- The `memory` parameter in vkFreeMemory
- The `memory` parameter in vkMapMemory
- The `memory` parameter in vkUnmapMemory
- The `buffer` parameter in vkBindBufferMemory
- The `image` parameter in vkBindImageMemory
- The `queue` parameter in vkQueueBindSparse
- The `fence` parameter in vkQueueBindSparse
- The `fence` parameter in vkDestroyFence
- The `semaphore` parameter in vkDestroySemaphore
- The `event` parameter in vkDestroyEvent
- The `event` parameter in vkSetEvent
- The `event` parameter in vkResetEvent
- The `queryPool` parameter in vkDestroyQueryPool
- The `buffer` parameter in vkDestroyBuffer
- The `bufferView` parameter in vkDestroyBufferView
- The `image` parameter in vkDestroyImage
- The `imageView` parameter in vkDestroyImageView
- The `shaderModule` parameter in vkDestroyShaderModule
- The `pipelineCache` parameter in vkDestroyPipelineCache
- The `dstCache` parameter in vkMergePipelineCaches
- The `pipeline` parameter in vkDestroyPipeline
- The `pipelineLayout` parameter in vkDestroyPipelineLayout

- The `sampler` parameter in vkDestroySampler
- The `descriptorSetLayout` parameter in vkDestroyDescriptorSetLayout
- The `descriptorPool` parameter in vkDestroyDescriptorPool
- The `descriptorPool` parameter in vkResetDescriptorPool
- The `descriptorPool` member of the `pAllocateInfo` parameter in vkAllocateDescriptorSets
- The `descriptorPool` parameter in vkFreeDescriptorSets
- The `framebuffer` parameter in vkDestroyFramebuffer
- The `renderPass` parameter in vkDestroyRenderPass
- The `commandPool` parameter in vkDestroyCommandPool
- The `commandPool` parameter in vkResetCommandPool
- The `commandPool` member of the `pAllocateInfo` parameter in vkAllocateCommandBuffers
- The `commandPool` parameter in vkFreeCommandBuffers
- The `commandBuffer` parameter in vkBeginCommandBuffer
- The `commandBuffer` parameter in vkEndCommandBuffer
- The `commandBuffer` parameter in vkResetCommandBuffer
- The `commandBuffer` parameter in vkCmdBindPipeline
- The `commandBuffer` parameter in vkCmdSetViewport
- The `commandBuffer` parameter in vkCmdSetScissor
- The `commandBuffer` parameter in vkCmdSetLineWidth
- The `commandBuffer` parameter in vkCmdSetDepthBias
- The `commandBuffer` parameter in vkCmdSetBlendConstants
- The `commandBuffer` parameter in vkCmdSetDepthBounds
- The `commandBuffer` parameter in vkCmdSetStencilCompareMask
- The `commandBuffer` parameter in vkCmdSetStencilWriteMask
- The `commandBuffer` parameter in vkCmdSetStencilReference
- The `commandBuffer` parameter in vkCmdBindDescriptorSets
- The `commandBuffer` parameter in vkCmdBindIndexBuffer
- The `commandBuffer` parameter in vkCmdBindVertexBuffers
- The `commandBuffer` parameter in vkCmdDraw
- The `commandBuffer` parameter in vkCmdDrawIndexed
- The `commandBuffer` parameter in vkCmdDrawIndirect
- The `commandBuffer` parameter in vkCmdDrawIndexedIndirect
- The `commandBuffer` parameter in vkCmdDispatch
- The `commandBuffer` parameter in vkCmdDispatchIndirect
- The `commandBuffer` parameter in vkCmdCopyBuffer

- The `commandBuffer` parameter in vkCmdCopyImage
- The `commandBuffer` parameter in vkCmdBlitImage
- The `commandBuffer` parameter in vkCmdCopyBufferToImage
- The `commandBuffer` parameter in vkCmdCopyImageToBuffer
- The `commandBuffer` parameter in vkCmdUpdateBuffer
- The `commandBuffer` parameter in vkCmdFillBuffer
- The `commandBuffer` parameter in vkCmdClearColorImage
- The `commandBuffer` parameter in vkCmdClearDepthStencilImage
- The `commandBuffer` parameter in vkCmdClearAttachments
- The `commandBuffer` parameter in vkCmdResolveImage
- The `commandBuffer` parameter in vkCmdSetEvent
- The `commandBuffer` parameter in vkCmdResetEvent
- The `commandBuffer` parameter in vkCmdWaitEvents
- The `commandBuffer` parameter in vkCmdPipelineBarrier
- The `commandBuffer` parameter in vkCmdBeginQuery
- The `commandBuffer` parameter in vkCmdEndQuery
- The `commandBuffer` parameter in vkCmdResetQueryPool
- The `commandBuffer` parameter in vkCmdWriteTimestamp
- The `commandBuffer` parameter in vkCmdCopyQueryPoolResults
- The `commandBuffer` parameter in vkCmdPushConstants
- The `commandBuffer` parameter in vkCmdBeginRenderPass
- The `commandBuffer` parameter in vkCmdNextSubpass
- The `commandBuffer` parameter in vkCmdEndRenderPass
- The `commandBuffer` parameter in vkCmdExecuteCommands
- The `surface` parameter in vkDestroySurfaceKHR
- The `surface` member of the `pCreateInfo` parameter in vkCreateSwapchainKHR
- The `oldSwapchain` member of the `pCreateInfo` parameter in vkCreateSwapchainKHR
- The `swapchain` parameter in vkDestroySwapchainKHR
- The `swapchain` parameter in vkAcquireNextImageKHR
- The `semaphore` parameter in vkAcquireNextImageKHR
- The `fence` parameter in vkAcquireNextImageKHR
- The `queue` parameter in vkQueuePresentKHR
- The `surface` parameter in vkGetDeviceGroupSurfacePresentModesKHR
- The `surface` parameter in vkGetPhysicalDevicePresentRectanglesKHR
- The `display` parameter in vkCreateDisplayModeKHR

- The `mode` parameter in vkGetDisplayPlaneCapabilitiesKHR
- The `videoSession` parameter in vkDestroyVideoSessionKHR
- The `videoSession` parameter in vkBindVideoSessionMemoryKHR
- The `videoSessionParameters` parameter in vkDestroyVideoSessionParametersKHR
- The `commandBuffer` parameter in vkCmdBeginVideoCodingKHR
- The `commandBuffer` parameter in vkCmdEndVideoCodingKHR
- The `commandBuffer` parameter in vkCmdControlVideoCodingKHR
- The `commandBuffer` parameter in vkCmdDecodeVideoKHR
- The `commandBuffer` parameter in vkCmdBeginRenderingKHR
- The `commandBuffer` parameter in vkCmdEndRenderingKHR
- The `commandBuffer` parameter in vkCmdSetDeviceMaskKHR
- The `commandBuffer` parameter in vkCmdDispatchBaseKHR
- The `commandPool` parameter in vkTrimCommandPoolKHR
- The `commandBuffer` parameter in vkCmdPushDescriptorSetKHR
- The `commandBuffer` parameter in vkCmdPushDescriptorSetWithTemplateKHR
- The `descriptorUpdateTemplate` parameter in vkDestroyDescriptorUpdateTemplateKHR
- The `commandBuffer` parameter in vkCmdBeginRenderPass2KHR
- The `commandBuffer` parameter in vkCmdNextSubpass2KHR
- The `commandBuffer` parameter in vkCmdEndRenderPass2KHR
- The `swapchain` parameter in vkGetSwapchainStatusKHR
- The `ycbcrConversion` parameter in vkDestroySamplerYcbcrConversionKHR
- The `commandBuffer` parameter in vkCmdDrawIndirectCountKHR
- The `commandBuffer` parameter in vkCmdDrawIndexedIndirectCountKHR
- The `commandBuffer` parameter in vkCmdSetFragmentShadingRateKHR
- The `commandBuffer` parameter in vkCmdSetRenderingAttachmentLocationsKHR
- The `commandBuffer` parameter in vkCmdSetRenderingInputAttachmentIndicesKHR
- The `swapchain` parameter in vkWaitForPresentKHR
- The `operation` parameter in vkDestroyDeferredOperationKHR
- The `commandBuffer` parameter in vkCmdEncodeVideoKHR
- The `commandBuffer` parameter in vkCmdSetEvent2KHR
- The `commandBuffer` parameter in vkCmdResetEvent2KHR
- The `commandBuffer` parameter in vkCmdWaitEvents2KHR
- The `commandBuffer` parameter in vkCmdPipelineBarrier2KHR
- The `commandBuffer` parameter in vkCmdWriteTimestamp2KHR
- The `queue` parameter in vkQueueSubmit2KHR

- The `fence` parameter in vkQueueSubmit2KHR
- The `commandBuffer` parameter in vkCmdCopyBuffer2KHR
- The `commandBuffer` parameter in vkCmdCopyImage2KHR
- The `commandBuffer` parameter in vkCmdCopyBufferToImage2KHR
- The `commandBuffer` parameter in vkCmdCopyImageToBuffer2KHR
- The `commandBuffer` parameter in vkCmdBlitImage2KHR
- The `commandBuffer` parameter in vkCmdResolveImage2KHR
- The `commandBuffer` parameter in vkCmdTraceRaysIndirect2KHR
- The `commandBuffer` parameter in vkCmdBindIndexBuffer2KHR
- The `pipelineBinary` parameter in vkDestroyPipelineBinaryKHR
- The `pipeline` member of the `pInfo` parameter in vkReleaseCapturedPipelineDataKHR
- The `commandBuffer` parameter in vkCmdSetLineStippleKHR
- The `commandBuffer` parameter in vkCmdBindDescriptorSets2KHR
- The `commandBuffer` parameter in vkCmdPushConstants2KHR
- The `commandBuffer` parameter in vkCmdPushDescriptorSet2KHR
- The `commandBuffer` parameter in vkCmdPushDescriptorSetWithTemplate2KHR
- The `commandBuffer` parameter in vkCmdSetDescriptorBufferOffsets2EXT
- The `commandBuffer` parameter in vkCmdBindDescriptorBufferEmbeddedSamplers2EXT
- The `commandBuffer` parameter in vkCmdBindTransformFeedbackBuffersEXT
- The `commandBuffer` parameter in vkCmdBeginTransformFeedbackEXT
- The `commandBuffer` parameter in vkCmdEndTransformFeedbackEXT
- The `commandBuffer` parameter in vkCmdBeginQueryIndexedEXT
- The `commandBuffer` parameter in vkCmdEndQueryIndexedEXT
- The `commandBuffer` parameter in vkCmdDrawIndirectByteCountEXT
- The `commandBuffer` parameter in vkCmdSetDiscardRectangleEXT
- The `commandBuffer` parameter in vkCmdSetDiscardRectangleEnableEXT
- The `commandBuffer` parameter in vkCmdSetDiscardRectangleModeEXT
- The `objectHandle` member of the `pNameInfo` parameter in vkSetDebugUtilsObjectNameEXT
- The `objectHandle` member of the `pTagInfo` parameter in vkSetDebugUtilsObjectTagEXT
- The `commandBuffer` parameter in vkCmdBeginDebugUtilsLabelEXT
- The `commandBuffer` parameter in vkCmdEndDebugUtilsLabelEXT
- The `commandBuffer` parameter in vkCmdInsertDebugUtilsLabelEXT
- The `messenger` parameter in vkDestroyDebugUtilsMessengerEXT
- The `commandBuffer` parameter in vkCmdSetSampleLocationsEXT
- The `validationCache` parameter in vkDestroyValidationCacheEXT

- The `dstCache` parameter in vkMergeValidationCachesEXT
- The `commandBuffer` parameter in vkCmdSetDepthBias2EXT
- The `commandBuffer` parameter in vkCmdSetVertexInputEXT
- The `commandBuffer` parameter in vkCmdSetColorWriteEnableEXT
- The `commandBuffer` parameter in vkCmdDrawMultiEXT
- The `commandBuffer` parameter in vkCmdDrawMultiIndexedEXT
- The `micromap` parameter in vkDestroyMicromapEXT
- The `commandBuffer` parameter in vkCmdBuildMicromapsEXT
- The `commandBuffer` parameter in vkCmdCopyMicromapEXT
- The `commandBuffer` parameter in vkCmdCopyMicromapToMemoryEXT
- The `commandBuffer` parameter in vkCmdCopyMemoryToMicromapEXT
- The `commandBuffer` parameter in vkCmdWriteMicromapsPropertiesEXT
- The `commandBuffer` parameter in vkCmdSetDepthClampEnableEXT
- The `commandBuffer` parameter in vkCmdSetPolygonModeEXT
- The `commandBuffer` parameter in vkCmdSetRasterizationSamplesEXT
- The `commandBuffer` parameter in vkCmdSetSampleMaskEXT
- The `commandBuffer` parameter in vkCmdSetAlphaToCoverageEnableEXT
- The `commandBuffer` parameter in vkCmdSetAlphaToOneEnableEXT
- The `commandBuffer` parameter in vkCmdSetLogicOpEnableEXT
- The `commandBuffer` parameter in vkCmdSetColorBlendEnableEXT
- The `commandBuffer` parameter in vkCmdSetColorBlendEquationEXT
- The `commandBuffer` parameter in vkCmdSetColorWriteMaskEXT
- The `commandBuffer` parameter in vkCmdSetTessellationDomainOriginEXT
- The `commandBuffer` parameter in vkCmdSetRasterizationStreamEXT
- The `commandBuffer` parameter in vkCmdSetConservativeRasterizationModeEXT
- The `commandBuffer` parameter in vkCmdSetExtraPrimitiveOverestimationSizeEXT
- The `commandBuffer` parameter in vkCmdSetDepthClipEnableEXT
- The `commandBuffer` parameter in vkCmdSetSampleLocationsEnableEXT
- The `commandBuffer` parameter in vkCmdSetColorBlendAdvancedEXT
- The `commandBuffer` parameter in vkCmdSetProvokingVertexModeEXT
- The `commandBuffer` parameter in vkCmdSetLineRasterizationModeEXT
- The `commandBuffer` parameter in vkCmdSetLineStippleEnableEXT
- The `commandBuffer` parameter in vkCmdSetDepthClipNegativeOneToOneEXT
- The `commandBuffer` parameter in vkCmdSetViewportWScalingEnableNV
- The `commandBuffer` parameter in vkCmdSetViewportSwizzleNV

- The `commandBuffer` parameter in vkCmdSetCoverageToColorEnableNV
- The `commandBuffer` parameter in vkCmdSetCoverageToColorLocationNV
- The `commandBuffer` parameter in vkCmdSetCoverageModulationModeNV
- The `commandBuffer` parameter in vkCmdSetCoverageModulationTableEnableNV
- The `commandBuffer` parameter in vkCmdSetCoverageModulationTableNV
- The `commandBuffer` parameter in vkCmdSetShadingRateImageEnableNV
- The `commandBuffer` parameter in vkCmdSetRepresentativeFragmentTestEnableNV
- The `commandBuffer` parameter in vkCmdSetCoverageReductionModeNV
- The `shader` parameter in vkDestroyShaderEXT
- The `commandBuffer` parameter in vkCmdBindShadersEXT
- The `commandBuffer` parameter in vkCmdSetCullModeEXT
- The `commandBuffer` parameter in vkCmdSetFrontFaceEXT
- The `commandBuffer` parameter in vkCmdSetPrimitiveTopologyEXT
- The `commandBuffer` parameter in vkCmdSetViewportWithCountEXT
- The `commandBuffer` parameter in vkCmdSetScissorWithCountEXT
- The `commandBuffer` parameter in vkCmdBindVertexBuffers2EXT
- The `commandBuffer` parameter in vkCmdSetDepthTestEnableEXT
- The `commandBuffer` parameter in vkCmdSetDepthWriteEnableEXT
- The `commandBuffer` parameter in vkCmdSetDepthCompareOpEXT
- The `commandBuffer` parameter in vkCmdSetDepthBoundsTestEnableEXT
- The `commandBuffer` parameter in vkCmdSetStencilTestEnableEXT
- The `commandBuffer` parameter in vkCmdSetStencilOpEXT
- The `commandBuffer` parameter in vkCmdSetPatchControlPointsEXT
- The `commandBuffer` parameter in vkCmdSetRasterizerDiscardEnableEXT
- The `commandBuffer` parameter in vkCmdSetDepthBiasEnableEXT
- The `commandBuffer` parameter in vkCmdSetLogicOpEXT
- The `commandBuffer` parameter in vkCmdSetPrimitiveRestartEnableEXT
- The `commandBuffer` parameter in vkCmdSetDepthClampRangeEXT
- The `commandBuffer` parameter in vkCmdSetAttachmentFeedbackLoopEnableEXT
- The `commandBuffer` parameter in vkCmdPreprocessGeneratedCommandsEXT
- The `stateCommandBuffer` parameter in vkCmdPreprocessGeneratedCommandsEXT
- The `commandBuffer` parameter in vkCmdExecuteGeneratedCommandsEXT
- The `indirectCommandsLayout` parameter in vkDestroyIndirectCommandsLayoutEXT
- The `indirectExecutionSet` parameter in vkDestroyIndirectExecutionSetEXT
- The `indirectExecutionSet` parameter in vkUpdateIndirectExecutionSetPipelineEXT

- The `indirectExecutionSet` parameter in vkUpdateIndirectExecutionSetShaderEXT
- The `accelerationStructure` parameter in vkDestroyAccelerationStructureKHR
- The `commandBuffer` parameter in vkCmdBuildAccelerationStructuresKHR
- The `commandBuffer` parameter in vkCmdBuildAccelerationStructuresIndirectKHR
- The `commandBuffer` parameter in vkCmdCopyAccelerationStructureKHR
- The `commandBuffer` parameter in vkCmdCopyAccelerationStructureToMemoryKHR
- The `commandBuffer` parameter in vkCmdCopyMemoryToAccelerationStructureKHR
- The `commandBuffer` parameter in vkCmdWriteAccelerationStructuresPropertiesKHR
- The `commandBuffer` parameter in vkCmdTraceRaysKHR
- The `commandBuffer` parameter in vkCmdTraceRaysIndirectKHR
- The `commandBuffer` parameter in vkCmdSetRayTracingPipelineStackSizeKHR

There are also a few instances where a command **can** take in an application-allocated list whose contents are externally synchronized parameters. In these cases, the caller **must** guarantee that at most one thread is using a given element within the list at a given time. These parameters are listed below.

Externally Synchronized Parameter Lists

- Each element of the `pFences` parameter in vkResetFences
- Each element of the `pDescriptorSets` parameter in vkFreeDescriptorSets
- Each element of the `pCommandBuffers` parameter in vkFreeCommandBuffers
- Each element of the `pWaitSemaphores` member of the `pPresentInfo` parameter in vkQueuePresentKHR
- Each element of the `pSwapchains` member of the `pPresentInfo` parameter in vkQueuePresentKHR
- The `surface` member of each element of the `pCreateInfos` parameter in vkCreateSharedSwapchainsKHR
- The `oldSwapchain` member of each element of the `pCreateInfos` parameter in vkCreateSharedSwapchainsKHR

In addition, there are some implicit parameters that need to be externally synchronized. For example, when a `commandBuffer` parameter needs to be externally synchronized, it implies that the `commandPool` from which that command buffer was allocated also needs to be externally synchronized. The implicit parameters and their associated object are listed below.

Implicit Externally Synchronized Parameters

- All `VkPhysicalDevice` objects enumerated from `instance` in vkDestroyInstance

- All `VkQueue` objects created from `device` in vkDestroyDevice
- All `VkQueue` objects created from `device` in vkDeviceWaitIdle
- Any `VkDescriptorSet` objects allocated from `descriptorPool` in vkResetDescriptorPool
- The `VkCommandPool` that `commandBuffer` was allocated from in vkBeginCommandBuffer
- The `VkCommandPool` that `commandBuffer` was allocated from in vkEndCommandBuffer
- The `VkCommandPool` that `commandBuffer` was allocated from in vkResetCommandBuffer
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdBindPipeline
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetViewport
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetScissor
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetLineWidth
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetDepthBias
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetBlendConstants
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetDepthBounds
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetStencilCompareMask
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetStencilWriteMask
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdSetStencilReference
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdBindDescriptorSets
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdBindIndexBuffer
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdBindVertexBuffers
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdDraw
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdDrawIndexed
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdDrawIndirect
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdDrawIndexedIndirect
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdDispatch
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdDispatchIndirect
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdCopyBuffer
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdCopyImage
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdBlitImage
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdCopyBufferToImage
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdCopyImageToBuffer
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdUpdateBuffer
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdFillBuffer
- The `VkCommandPool` that `commandBuffer` was allocated from, in vkCmdClearColorImage
- The `VkCommandPool` that `commandBuffer` was allocated from, in

vkCmdClearDepthStencilImage

- The VkCommandPool that commandBuffer was allocated from, in vkCmdClearAttachments
- The VkCommandPool that commandBuffer was allocated from, in vkCmdResolveImage
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetEvent
- The VkCommandPool that commandBuffer was allocated from, in vkCmdResetEvent
- The VkCommandPool that commandBuffer was allocated from, in vkCmdWaitEvents
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPipelineBarrier
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBeginQuery
- The VkCommandPool that commandBuffer was allocated from, in vkCmdEndQuery
- The VkCommandPool that commandBuffer was allocated from, in vkCmdResetQueryPool
- The VkCommandPool that commandBuffer was allocated from, in vkCmdWriteTimestamp
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyQueryPoolResults
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPushConstants
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBeginRenderPass
- The VkCommandPool that commandBuffer was allocated from, in vkCmdNextSubpass
- The VkCommandPool that commandBuffer was allocated from, in vkCmdEndRenderPass
- The VkCommandPool that commandBuffer was allocated from, in vkCmdExecuteCommands
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBeginVideoCodingKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdEndVideoCodingKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdControlVideoCodingKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdDecodeVideoKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBeginRenderingKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdEndRenderingKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDeviceMaskKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdDispatchBaseKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPushDescriptorSetKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPushDescriptorSetWithTemplateKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBeginRenderPass2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdNextSubpass2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdEndRenderPass2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdDrawIndirectCountKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdDrawIndexedIndirectCountKHR

- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetFragmentShadingRateKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetRenderingAttachmentLocationsKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetRenderingInputAttachmentIndicesKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdEncodeVideoKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetEvent2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdResetEvent2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdWaitEvents2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPipelineBarrier2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdWriteTimestamp2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyBuffer2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyImage2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyBufferToImage2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyImageToBuffer2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBlitImage2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdResolveImage2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdTraceRaysIndirect2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBindIndexBuffer2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetLineStippleKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBindDescriptorSets2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPushConstants2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPushDescriptorSet2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPushDescriptorSetWithTemplate2KHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDescriptorBufferOffsets2EXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBindDescriptorBufferEmbeddedSamplers2EXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBindTransformFeedbackBuffersEXT
- The VkCommandPool that commandBuffer was allocated from, in

vkCmdBeginTransformFeedbackEXT

- The VkCommandPool that commandBuffer was allocated from, in vkCmdEndTransformFeedbackEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBeginQueryIndexedEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdEndQueryIndexedEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdDrawIndirectByteCountEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDiscardRectangleEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDiscardRectangleEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDiscardRectangleModeEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBeginDebugUtilsLabelEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdEndDebugUtilsLabelEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdInsertDebugUtilsLabelEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetSampleLocationsEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthBias2EXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetVertexInputEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetColorWriteEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdDrawMultiEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdDrawMultiIndexedEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBuildMicromapsEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyMicromapEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyMicromapToMemoryEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyMemoryToMicromapEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdWriteMicromapsPropertiesEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthClampEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetPolygonModeEXT

- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetRasterizationSamplesEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetSampleMaskEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetAlphaToCoverageEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetAlphaToOneEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetLogicOpEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetColorBlendEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetColorBlendEquationEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetColorWriteMaskEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetTessellationDomainOriginEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetRasterizationStreamEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetConservativeRasterizationModeEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetExtraPrimitiveOverestimationSizeEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthClipEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetSampleLocationsEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetColorBlendAdvancedEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetProvokingVertexModeEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetLineRasterizationModeEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetLineStippleEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthClipNegativeOneToOneEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetViewportWScalingEnableNV
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetViewportSwizzleNV
- The VkCommandPool that commandBuffer was allocated from, in

vkCmdSetCoverageToColorEnableNV

- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetCoverageToColorLocationNV
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetCoverageModulationModeNV
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetCoverageModulationTableEnableNV
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetCoverageModulationTableNV
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetShadingRateImageEnableNV
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetRepresentativeFragmentTestEnableNV
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetCoverageReductionModeNV
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBindShadersEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetCullModeEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetFrontFaceEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetPrimitiveTopologyEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetViewportWithCountEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetScissorWithCountEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBindVertexBuffers2EXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthTestEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthWriteEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthCompareOpEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthBoundsTestEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetStencilTestEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetStencilOpEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetPatchControlPointsEXT

- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetRasterizerDiscardEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthBiasEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetLogicOpEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetPrimitiveRestartEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetDepthClampRangeEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetAttachmentFeedbackLoopEnableEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdPreprocessGeneratedCommandsEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdExecuteGeneratedCommandsEXT
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBuildAccelerationStructuresKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdBuildAccelerationStructuresIndirectKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyAccelerationStructureKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyAccelerationStructureToMemoryKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdCopyMemoryToAccelerationStructureKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdWriteAccelerationStructuresPropertiesKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdTraceRaysKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdTraceRaysIndirectKHR
- The VkCommandPool that commandBuffer was allocated from, in vkCmdSetRayTracingPipelineStackSizeKHR

3.7. Valid Usage

Valid usage defines a set of conditions which **must** be met in order to achieve well-defined runtime behavior in an application. These conditions depend only on Vulkan state, and the parameters or objects whose usage is constrained by the condition.

The core layer assumes applications are using the API correctly. Except as documented elsewhere in the Specification, the behavior of the core layer to an application using the API incorrectly is undefined, and **may** include program termination. However, implementations **must** ensure that

incorrect usage by an application does not affect the integrity of the operating system, the Vulkan implementation, or other applications in the system using Vulkan. In particular, any guarantees made by an operating system about whether memory from one process **can** be visible to another process or not **must** not be violated by a Vulkan implementation for **any memory allocation**. Vulkan implementations are not **required** to make additional security or integrity guarantees beyond those provided by the OS unless explicitly directed by the application's use of a particular feature or extension.

NOTE

For instance, if an operating system guarantees that data in all its memory allocations are set to zero when newly allocated, the Vulkan implementation **must** make the same guarantees for any allocations it controls (e.g. VkDeviceMemory).

Similarly, if an operating system guarantees that use-after-free of host allocations will not result in values written by another process becoming visible, the same guarantees **must** be made by the Vulkan implementation for device memory.

Some valid usage conditions have dependencies on runtime limits or feature availability. It is possible to validate these conditions against Vulkan's minimum supported values for these limits and features, or some subset of other known values.

Valid usage conditions do not cover conditions where well-defined behavior (including returning an error code) exists.

Valid usage conditions **should** apply to the command or structure where complete information about the condition would be known during execution of an application. This is such that a validation layer or linter **can** be written directly against these statements at the point they are specified.

NOTE

This does lead to some non-obvious places for valid usage statements. For instance, the valid values for a structure might depend on a separate value in the calling command. In this case, the structure itself will not reference this valid usage as it is impossible to determine validity from the structure that it is invalid - instead this valid usage would be attached to the calling command.

Another example is draw state - the state setters are independent, and can cause a legitimately invalid state configuration between draw calls; so the valid usage statements are attached to the place where all state needs to be valid - at the drawing command.

Valid usage conditions are described in a block labeled "Valid Usage" following each command or structure they apply to.

3.7.1. Usage Validation

Vulkan is a layered API. The lowest layer is the core Vulkan layer, as defined by this Specification. The application **can** use additional layers above the core for debugging, validation, and other purposes.

One of the core principles of Vulkan is that building and submitting command buffers **should** be

highly efficient. Thus error checking and validation of state in the core layer is minimal, although more rigorous validation **can** be enabled through the use of layers.

Validation of correct API usage is left to validation layers. Applications **should** be developed with validation layers enabled, to help catch and eliminate errors. Once validated, released applications **should** not enable validation layers by default.

3.7.2. Implicit Valid Usage

Some valid usage conditions apply to all commands and structures in the API, unless explicitly denoted otherwise for a specific command or structure. These conditions are considered *implicit*, and are described in a block labeled "Valid Usage (Implicit)" following each command or structure they apply to. Implicit valid usage conditions are described in detail below.

Valid Usage for Object Handles

Any input parameter to a command that is an object handle **must** be a valid object handle, unless otherwise specified. An object handle is valid if:

- It has been created or allocated by a previous, successful call to the API. Such calls are noted in the Specification.
- It has not been deleted or freed by a previous call to the API. Such calls are noted in the Specification.
- Any objects used by that object, either as part of creation or execution, **must** also be valid.

The reserved values VK_NULL_HANDLE and `NULL` **can** be used in place of valid non-dispatchable handles and dispatchable handles, respectively, when *explicitly called out in the Specification*. Any command that creates an object successfully **must** not return these values. It is valid to pass these values to `vkDestroy*` or `vkFree*` commands, which will silently ignore these values.

Valid Usage for Pointers

Any parameter that is a pointer **must** be a *valid pointer* only if it is explicitly called out by a Valid Usage statement.

A pointer is "valid" if it points at memory containing values of the number and type(s) expected by the command, and all fundamental types accessed through the pointer (e.g. as elements of an array or as members of a structure) satisfy the alignment requirements of the host processor.

Valid Usage for Strings

Any parameter that is a pointer to `char` **must** be a finite sequence of values terminated by a null character, or if *explicitly called out in the Specification*, **can** be `NULL`.

Strings specified as UTF-8 encoded **must** not contain invalid UTF-8 sequences. See String Representation for additional information about strings.

Valid Usage for Enumerated Types

Any parameter of an enumerated type **must** be a valid enumerant for that type. Use of an enumerant is valid if the following conditions are true:

- The enumerant is defined as part of the enumerated type.
- The enumerant is not a value suffixed with `_MAX_ENUM`.
  - This value exists only to ensure that C `enum` types are 32 bits in size and **must** not be used by applications.
- If the enumerant is used in a function that has a VkInstance as its first parameter and either:
  - it was added by a core version that is supported and the value of VkApplicationInfo::`apiVersion` is greater than or equal to the version that added it; or
  - it was added by an instance extension that was enabled for the instance.
- If the enumerant is used in a function that has a VkPhysicalDevice object as its first parameter and either:
  - it was added by a core version that is supported by that device (as reported by VkPhysicalDeviceProperties::`apiVersion`);
  - it was added by an instance extension that was enabled for the instance; or
  - it was added by a device extension that is supported by that device.
- If the enumerant is used in a function that has any other dispatchable object as its first parameter and either:
  - it was added by a core version that is supported for the device (as reported by VkPhysicalDeviceProperties::`apiVersion`); or
  - it was added by a device extension that was enabled for the device.

Additionally, if the `maintenance5` feature is supported, any integer value representable in the range valid for the defined type is valid when used in a function that has a VkPhysicalDevice object as its first parameter. Physical device queries will either return results indicating lack of support, or ignore unsupported values when used as a bit flag in a `Vk*Flags*` parameter.

Any enumerated type returned from a query command or otherwise output from Vulkan to the application **must** not have a reserved value. Reserved values are values not defined by any extension for that enumerated type.

NOTE	In some special cases, an enumerant is only meaningful if a feature defined by an extension is also enabled, as well as the extension itself. The global “valid enumerant” rule described here does not address such cases.
NOTE	This language is intended to accommodate cases such as “hidden” extensions known only to driver internals, or layers enabling extensions without knowledge of the application, without allowing return of values not defined by any extension.
NOTE	Application developers are encouraged to be careful when using `switch` statements with Vulkan API enums. This is because new extensions can add new values to

existing enums. Using a `default:` statement within a `switch` may avoid future compilation issues.

Valid Usage for Flags

A collection of flags is represented by a bitmask using the type VkFlags:

```
// Provided by VK_VERSION_1_0
typedef uint32_t VkFlags;
```

Bitmasks are passed to many commands and structures to compactly represent options, but VkFlags is not used directly in the API. Instead, a `Vk*Flags` type which is an alias of VkFlags, and whose name matches the corresponding `Vk*FlagBits` that are valid for that type, is used.

Any `Vk*Flags` member or parameter used in the API as an input **must** be a valid combination of bit flags. A valid combination is either zero or the bitwise OR of valid bit flags.

An individual bit flag is valid for a `Vk*Flags` type if it would be a valid enumerant when used with the equivalent `Vk*FlagBits` type, where the bits type is obtained by taking the flag type and replacing the trailing `Flags` with `FlagBits`. For example, a flag value of type VkColorComponentFlags **must** contain only bit flags defined by VkColorComponentFlagBits.

Any `Vk*Flags` member or parameter returned from a query command or otherwise output from Vulkan to the application **may** contain bit flags undefined in its corresponding `Vk*FlagBits` type. An application **cannot** rely on the state of these unspecified bits.

Only the low-order 31 bits (bit positions zero through 30) are available for use as flag bits.

NOTE This restriction is due to poorly defined behavior by C compilers given a C enumerant value of `0x80000000`. In some cases adding this enumerant value may increase the size of the underlying `Vk*FlagBits` type, breaking the ABI.

A collection of 64-bit flags is represented by a bitmask using the type VkFlags64:

```
// Provided by VK_KHR_synchronization2
typedef uint64_t VkFlags64;
```

When the 31 bits available in VkFlags are insufficient, the VkFlags64 type can be passed to commands and structures to represent up to 64 options. VkFlags64 is not used directly in the API. Instead, a `Vk*Flags2` type which is an alias of VkFlags64, and whose name matches the corresponding `Vk*FlagBits2` that are valid for that type, is used.

Any `Vk*Flags2` member or parameter used in the API as an input **must** be a valid combination of bit flags. A valid combination is either zero or the bitwise OR of valid bit flags.

An individual bit flag is valid for a `Vk*Flags2` type if it would be a valid enumerant when used with the equivalent `Vk*FlagBits2` type, where the bits type is obtained by taking the flag type and

replacing the trailing `Flags2` with `FlagBits2`. For example, a flag value of type VkAccessFlags2KHR **must** contain only bit flags defined by VkAccessFlagBits2KHR.

Any `Vk*Flags2` member or parameter returned from a query command or otherwise output from Vulkan to the application **may** contain bit flags undefined in its corresponding `Vk*FlagBits2` type. An application **cannot** rely on the state of these unspecified bits.

NOTE

Both the `Vk*FlagBits2` type, and the individual bits defined for that type, are defined as `uint64_t` integers in the C API. This is in contrast to the 32-bit types, where the `Vk*FlagBits` type is defined as a C `enum` and the individual bits as enumerants belonging to that `enum`. As a result, there is less compile time type checking possible for the 64-bit types. This is unavoidable since there is no sufficiently portable way to define a 64-bit `enum` type in C99.

Valid Usage for Structure Types

Any parameter that is a structure containing a `sType` member **must** have a value of `sType` which is a valid VkStructureType value matching the type of the structure.

Valid Usage for Structure Pointer Chains

Any parameter that is a structure containing a `void* pNext` member **must** have a value of `pNext` that is either `NULL`, or is a pointer to a valid *extending structure*, containing `sType` and `pNext` members as described in the Vulkan Documentation and Extensions document in the section "Extending Structures". The set of structures connected by `pNext` pointers is referred to as a `pNext` *chain*.

Each structure included in the `pNext` chain **must** be defined at runtime by either:

- a core version which is supported
- an extension which is enabled
- a supported device extension in the case of physical-device-level functionality added by the device extension

Each type of extending structure **must** not appear more than once in a `pNext` chain, including any aliases. This general rule may be explicitly overridden for specific structures.

Any component of the implementation (the loader, any enabled layers, and drivers) **must** skip over, without processing (other than reading the `sType` and `pNext` members) any extending structures in the chain not defined by core versions or extensions supported by that component.

As a convenience to implementations and layers needing to iterate through a structure pointer chain, the Vulkan API provides two *base structures*. These structures allow for some type safety, and can be used by Vulkan API functions that operate on generic inputs and outputs.

The `VkBaseInStructure` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkBaseInStructure {
    VkStructureType                    sType;
    const struct VkBaseInStructure*    pNext;
} VkBaseInStructure;
```

- `sType` is the structure type of the structure being iterated through.
- `pNext` is `NULL` or a pointer to the next structure in a structure chain.

`VkBaseInStructure` can be used to facilitate iterating through a read-only structure pointer chain.

The `VkBaseOutStructure` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkBaseOutStructure {
    VkStructureType               sType;
    struct VkBaseOutStructure*    pNext;
} VkBaseOutStructure;
```

- `sType` is the structure type of the structure being iterated through.
- `pNext` is `NULL` or a pointer to the next structure in a structure chain.

`VkBaseOutStructure` can be used to facilitate iterating through a structure pointer chain that returns data back to the application.

Valid Usage for Nested Structures

The above conditions also apply recursively to members of structures provided as input to a command, either as a direct argument to the command, or themselves a member of another structure.

Specifics on valid usage of each command are covered in their individual sections.

Valid Usage for Extensions

Instance-level functionality or behavior added by an instance extension to the API **must** not be used unless that extension is supported by the instance as determined by vkEnumerateInstanceExtensionProperties, and that extension is enabled in VkInstanceCreateInfo.

Physical-device-level functionality or behavior added by an instance extension to the API **must** not be used unless that extension is supported by the instance as determined by vkEnumerateInstanceExtensionProperties, and that extension is enabled in VkInstanceCreateInfo.

Physical-device-level functionality or behavior added by a device extension to the API **must** not be used unless the conditions described in Extending Physical Device From Device Extensions are met.

Device-level functionality added by a device extension that is dispatched from a VkDevice, or from a child object of a VkDevice **must** not be used unless that extension is supported by the device as

determined by vkEnumerateDeviceExtensionProperties, and that extension is enabled in VkDeviceCreateInfo.

Valid Usage for Newer Core Versions

Physical-device-level functionality or behavior added by a new core version of the API **must** not be used unless it is supported by the physical device as determined by VkPhysicalDeviceProperties::`apiVersion` and the specified version of VkApplicationInfo::`apiVersion`.

Device-level functionality or behavior added by a new core version of the API **must** not be used unless it is supported by the device as determined by VkPhysicalDeviceProperties::`apiVersion` and the specified version of VkApplicationInfo::`apiVersion`.

3.8. `VkResult` Return Codes

While the core Vulkan API is not designed to capture incorrect usage, some circumstances still require return codes. Commands in Vulkan return their status via return codes that are in one of two categories:

- Successful completion codes are returned when a command needs to communicate success or status information. All successful completion codes are non-negative values.
- Runtime error codes are returned when a command needs to communicate a failure that could only be detected at runtime. All runtime error codes are negative values.

All return codes in Vulkan are reported via VkResult return values. The possible codes are:

```
// Provided by VK_VERSION_1_0
typedef enum VkResult {
    VK_SUCCESS = 0,
    VK_NOT_READY = 1,
    VK_TIMEOUT = 2,
    VK_EVENT_SET = 3,
    VK_EVENT_RESET = 4,
    VK_INCOMPLETE = 5,
    VK_ERROR_OUT_OF_HOST_MEMORY = -1,
    VK_ERROR_OUT_OF_DEVICE_MEMORY = -2,
    VK_ERROR_INITIALIZATION_FAILED = -3,
    VK_ERROR_DEVICE_LOST = -4,
    VK_ERROR_MEMORY_MAP_FAILED = -5,
    VK_ERROR_LAYER_NOT_PRESENT = -6,
    VK_ERROR_EXTENSION_NOT_PRESENT = -7,
    VK_ERROR_FEATURE_NOT_PRESENT = -8,
    VK_ERROR_INCOMPATIBLE_DRIVER = -9,
    VK_ERROR_TOO_MANY_OBJECTS = -10,
    VK_ERROR_FORMAT_NOT_SUPPORTED = -11,
    VK_ERROR_FRAGMENTED_POOL = -12,
    VK_ERROR_UNKNOWN = -13,
    VK_ERROR_OUT_OF_POOL_MEMORY = -1000069000,
    VK_ERROR_INVALID_EXTERNAL_HANDLE = -1000072003,
```

```
    VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS = -1000257000,
  // Provided by VK_KHR_surface
    VK_ERROR_SURFACE_LOST_KHR = -1000000000,
  // Provided by VK_KHR_surface
    VK_ERROR_NATIVE_WINDOW_IN_USE_KHR = -1000000001,
  // Provided by VK_KHR_swapchain
    VK_SUBOPTIMAL_KHR = 1000001003,
  // Provided by VK_KHR_swapchain
    VK_ERROR_OUT_OF_DATE_KHR = -1000001004,
  // Provided by VK_KHR_display_swapchain
    VK_ERROR_INCOMPATIBLE_DISPLAY_KHR = -1000003001,
  // Provided by VK_KHR_video_queue
    VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR = -1000023000,
  // Provided by VK_KHR_video_queue
    VK_ERROR_VIDEO_PICTURE_LAYOUT_NOT_SUPPORTED_KHR = -1000023001,
  // Provided by VK_KHR_video_queue
    VK_ERROR_VIDEO_PROFILE_OPERATION_NOT_SUPPORTED_KHR = -1000023002,
  // Provided by VK_KHR_video_queue
    VK_ERROR_VIDEO_PROFILE_FORMAT_NOT_SUPPORTED_KHR = -1000023003,
  // Provided by VK_KHR_video_queue
    VK_ERROR_VIDEO_PROFILE_CODEC_NOT_SUPPORTED_KHR = -1000023004,
  // Provided by VK_KHR_video_queue
    VK_ERROR_VIDEO_STD_VERSION_NOT_SUPPORTED_KHR = -1000023005,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_ERROR_INVALID_DRM_FORMAT_MODIFIER_PLANE_LAYOUT_EXT = -1000158000,
  // Provided by VK_KHR_global_priority
    VK_ERROR_NOT_PERMITTED_KHR = -1000174001,
  // Provided by VK_EXT_full_screen_exclusive
    VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT = -1000255000,
  // Provided by VK_KHR_deferred_host_operations
    VK_THREAD_IDLE_KHR = 1000268000,
  // Provided by VK_KHR_deferred_host_operations
    VK_THREAD_DONE_KHR = 1000268001,
  // Provided by VK_KHR_deferred_host_operations
    VK_OPERATION_DEFERRED_KHR = 1000268002,
  // Provided by VK_KHR_deferred_host_operations
    VK_OPERATION_NOT_DEFERRED_KHR = 1000268003,
  // Provided by VK_KHR_video_encode_queue
    VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR = -1000299000,
  // Provided by VK_EXT_shader_object
    VK_INCOMPATIBLE_SHADER_BINARY_EXT = 1000482000,
  // Provided by VK_KHR_pipeline_binary
    VK_PIPELINE_BINARY_MISSING_KHR = 1000483000,
  // Provided by VK_KHR_pipeline_binary
    VK_ERROR_NOT_ENOUGH_SPACE_KHR = -1000483000,
  // Provided by VK_KHR_maintenance1
    VK_ERROR_OUT_OF_POOL_MEMORY_KHR = VK_ERROR_OUT_OF_POOL_MEMORY,
  // Provided by VK_KHR_external_memory
    VK_ERROR_INVALID_EXTERNAL_HANDLE_KHR = VK_ERROR_INVALID_EXTERNAL_HANDLE,
  // Provided by VK_KHR_buffer_device_address
    VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR =
```

```
VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS,
  // Provided by VK_EXT_shader_object
  // VK_ERROR_INCOMPATIBLE_SHADER_BINARY_EXT is a deprecated alias
    VK_ERROR_INCOMPATIBLE_SHADER_BINARY_EXT = VK_INCOMPATIBLE_SHADER_BINARY_EXT,
} VkResult;
```

Success Codes

- VK_SUCCESS Command successfully completed
- VK_NOT_READY A fence or query has not yet completed
- VK_TIMEOUT A wait operation has not completed in the specified time
- VK_EVENT_SET An event is signaled
- VK_EVENT_RESET An event is unsignaled
- VK_INCOMPLETE A return array was too small for the result
- VK_SUBOPTIMAL_KHR A swapchain no longer matches the surface properties exactly, but **can** still be used to present to the surface successfully.
- VK_THREAD_IDLE_KHR A deferred operation is not complete but there is currently no work for this thread to do at the time of this call.
- VK_THREAD_DONE_KHR A deferred operation is not complete but there is no work remaining to assign to additional threads.
- VK_OPERATION_DEFERRED_KHR A deferred operation was requested and at least some of the work was deferred.
- VK_OPERATION_NOT_DEFERRED_KHR A deferred operation was requested and no operations were deferred.
- VK_PIPELINE_BINARY_MISSING_KHR The application attempted to create a pipeline binary by querying an internal cache, but the internal cache entry did not exist.
- VK_INCOMPATIBLE_SHADER_BINARY_EXT The provided binary shader code is not compatible with this device.

NOTE	In the initial version of the VK_EXT_shader_object extension, this return code was named VK_ERROR_INCOMPATIBLE_SHADER_BINARY_EXT and improperly described as an error code. The name has been changed, but the old name is retained as an alias for compatibility with old code.

Error Codes

- VK_ERROR_OUT_OF_HOST_MEMORY A host memory allocation has failed.
- VK_ERROR_OUT_OF_DEVICE_MEMORY A device memory allocation has failed.
- VK_ERROR_INITIALIZATION_FAILED Initialization of an object could not be completed for implementation-specific reasons.
- VK_ERROR_DEVICE_LOST The logical or physical device has been lost. See Lost Device
- VK_ERROR_MEMORY_MAP_FAILED Mapping of a memory object has failed.
- VK_ERROR_LAYER_NOT_PRESENT A requested layer is not present or could not be loaded.

- VK_ERROR_EXTENSION_NOT_PRESENT A requested extension is not supported.
- VK_ERROR_FEATURE_NOT_PRESENT A requested feature is not supported.
- VK_ERROR_INCOMPATIBLE_DRIVER The requested version of Vulkan is not supported by the driver or is otherwise incompatible for implementation-specific reasons.
- VK_ERROR_TOO_MANY_OBJECTS Too many objects of the type have already been created.
- VK_ERROR_FORMAT_NOT_SUPPORTED A requested format is not supported on this device.
- VK_ERROR_FRAGMENTED_POOL A pool allocation has failed due to fragmentation of the pool's memory. This **must** only be returned if no attempt to allocate host or device memory was made to accommodate the new allocation. This **should** be returned in preference to VK_ERROR_OUT_OF_POOL_MEMORY, but only if the implementation is certain that the pool allocation failure was due to fragmentation.
- VK_ERROR_SURFACE_LOST_KHR A surface is no longer available.
- VK_ERROR_NATIVE_WINDOW_IN_USE_KHR The requested window is already in use by Vulkan or another API in a manner which prevents it from being used again.
- VK_ERROR_OUT_OF_DATE_KHR A surface has changed in such a way that it is no longer compatible with the swapchain, and further presentation requests using the swapchain will fail. Applications **must** query the new surface properties and recreate their swapchain if they wish to continue presenting to the surface.
- VK_ERROR_INCOMPATIBLE_DISPLAY_KHR The display used by a swapchain does not use the same presentable image layout, or is incompatible in a way that prevents sharing an image.
- VK_ERROR_OUT_OF_POOL_MEMORY A pool memory allocation has failed. This **must** only be returned if no attempt to allocate host or device memory was made to accommodate the new allocation. If the failure was definitely due to fragmentation of the pool, VK_ERROR_FRAGMENTED_POOL **should** be returned instead.
- VK_ERROR_INVALID_EXTERNAL_HANDLE An external handle is not a valid handle of the specified type.
- VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS A buffer creation or memory allocation failed because the requested address is not available. A shader group handle assignment failed because the requested shader group handle information is no longer valid.
- VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT An operation on a swapchain created with VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT failed as it did not have exclusive full-screen access. This **may** occur due to implementation-dependent reasons, outside of the application's control.
- VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR The requested VkImageUsageFlags are not supported.
- VK_ERROR_VIDEO_PICTURE_LAYOUT_NOT_SUPPORTED_KHR The requested video picture layout is not supported.
- VK_ERROR_VIDEO_PROFILE_OPERATION_NOT_SUPPORTED_KHR A video profile operation specified via VkVideoProfileInfoKHR::videoCodecOperation is not supported.
- VK_ERROR_VIDEO_PROFILE_FORMAT_NOT_SUPPORTED_KHR Format parameters in a requested VkVideoProfileInfoKHR chain are not supported.
- VK_ERROR_VIDEO_PROFILE_CODEC_NOT_SUPPORTED_KHR Codec-specific parameters in a requested VkVideoProfileInfoKHR chain are not supported.

- VK_ERROR_VIDEO_STD_VERSION_NOT_SUPPORTED_KHR The specified video Std header version is not supported.
- VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR The specified Video Std parameters do not adhere to the syntactic or semantic requirements of the used video compression standard, or values derived from parameters according to the rules defined by the used video compression standard do not adhere to the capabilities of the video compression standard or the implementation.
- VK_ERROR_NOT_PERMITTED_KHR The driver implementation has denied a request to acquire a priority above the default priority (VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_EXT) because the application does not have sufficient privileges.
- VK_ERROR_NOT_ENOUGH_SPACE_KHR The application did not provide enough space to return all the required data.
- VK_ERROR_UNKNOWN An unknown error has occurred; either the application has provided invalid input, or an implementation failure has occurred.

If a command returns a runtime error, unless otherwise specified any output parameters will have undefined contents, except that if the output parameter is a structure with sType and pNext fields, those fields will be unmodified. Any structures chained from pNext will also have undefined contents, except that sType and pNext will be unmodified.

VK_ERROR_OUT_OF_*_MEMORY errors do not modify any currently existing Vulkan objects. Objects that have already been successfully created **can** still be used by the application.

NOTE	As a general rule, Free, Release, and Reset commands do not return VK_ERROR_OUT_OF_HOST_MEMORY, while any other command with a return code **may** return it. Any exceptions from this rule are described for those commands.

VK_ERROR_UNKNOWN will be returned by an implementation when an unexpected error occurs that cannot be attributed to valid behavior of the application and implementation. Under these conditions, it **may** be returned from any command returning a VkResult.

NOTE	VK_ERROR_UNKNOWN is not expected to ever be returned if the application behavior is valid, and if the implementation is bug-free. If VK_ERROR_UNKNOWN is received, the application should be checked against the latest validation layers to verify correct behavior as much as possible. If no issues are identified it could be an implementation issue, and the implementor should be contacted for support.

Performance-critical commands generally do not have return codes. If a runtime error occurs in such commands, the implementation will defer reporting the error until a specified point. For commands that record into command buffers (vkCmd*) runtime errors are reported by vkEndCommandBuffer.

3.9. Numeric Representation and Computation

Implementations normally perform computations in floating-point, and **must** meet the range and precision requirements defined under “Floating-Point Computation” below.

These requirements only apply to computations performed in Vulkan operations outside of shader execution, such as texture image specification and sampling, and per-fragment operations. Range and precision requirements during shader execution differ and are specified by the Precision and Operation of SPIR-V Instructions section.

In some cases, the representation and/or precision of operations is implicitly limited by the specified format of vertex or texel data consumed by Vulkan. Specific floating-point formats are described later in this section.

3.9.1. Floating-Point Computation

Most floating-point computation is performed in SPIR-V shader modules. The properties of computation within shaders are constrained as defined by the Precision and Operation of SPIR-V Instructions section.

Some floating-point computation is performed outside of shaders, such as viewport and depth range calculations. For these computations, we do not specify how floating-point numbers are to be represented, or the details of how operations on them are performed, but only place minimal requirements on representation and precision as described in the remainder of this section.

We require simply that numbers' floating-point parts contain enough bits and that their exponent fields are large enough so that individual results of floating-point operations are accurate to about 1 part in 10^5. The maximum representable magnitude for all floating-point values **must** be at least 2^{32}.

$x \times 0 = 0 \times x = 0$ for any non-infinite and non-NaN x.

$1 \times x = x \times 1 = x$.

$x + 0 = 0 + x = x$.

$0^0 = 1$.

Occasionally, further requirements will be specified. Most single-precision floating-point formats meet these requirements.

The special values Inf and -Inf encode values with magnitudes too large to be represented; the special value NaN encodes "Not A Number" values resulting from undefined arithmetic operations such as 0 / 0. Implementations **may** support Inf and NaN in their floating-point computations. Any computation which does not support either Inf or NaN, for which that value is an input or output will yield an undefined value.

3.9.2. Floating-Point Format Conversions

When a value is converted to a defined floating-point representation, finite values falling between two representable finite values are rounded to one or the other. The rounding mode is not defined.

Finite values whose magnitude is larger than that of any representable finite value may be rounded either to the closest representable finite value or to the appropriately signed infinity. For unsigned destination formats any negative values are converted to zero. Positive infinity is converted to positive infinity; negative infinity is converted to negative infinity in signed formats and to zero in unsigned formats; and any NaN is converted to a NaN.

3.9.3. 16-Bit Floating-Point Numbers

16-bit floating-point numbers are defined in the “16-bit floating-point numbers” section of the Khronos Data Format Specification.

3.9.4. Unsigned 11-Bit Floating-Point Numbers

Unsigned 11-bit floating-point numbers are defined in the “Unsigned 11-bit floating-point numbers” section of the Khronos Data Format Specification.

3.9.5. Unsigned 10-Bit Floating-Point Numbers

Unsigned 10-bit floating-point numbers are defined in the “Unsigned 10-bit floating-point numbers” section of the Khronos Data Format Specification.

3.9.6. General Requirements

Any representable floating-point value in the appropriate format is legal as input to a Vulkan command that requires floating-point data. The result of providing a value that is not a floating-point number to such a command is unspecified, but **must** not lead to Vulkan interruption or termination. For example, providing a negative zero (where applicable) or a denormalized number to a Vulkan command **must** yield deterministic results, while providing a NaN or Inf yields unspecified results.

Some calculations require division. In such cases (including implied divisions performed by vector normalization), division by zero produces an unspecified result but **must** not lead to Vulkan interruption or termination.

3.10. Fixed-Point Data Conversions

When generic vertex attributes and pixel color or depth *components* are represented as integers, they are often (but not always) considered to be *normalized*. Normalized integer values are treated specially when being converted to and from floating-point values, and are usually referred to as *normalized fixed-point*.

In the remainder of this section, b denotes the bit width of the fixed-point integer representation. When the integer is one of the types defined by the API, b is the bit width of that type. When the integer comes from an image containing color or depth component texels, b is the number of bits allocated to that component in its specified image format.

The signed and unsigned fixed-point representations are assumed to be b-bit binary two’s-complement integers and binary unsigned integers, respectively.

3.10.1. Conversion From Normalized Fixed-Point to Floating-Point

Unsigned normalized fixed-point integers represent numbers in the range [0,1]. The conversion from an unsigned normalized fixed-point value c to the corresponding floating-point value f is defined as

$$f = \frac{c}{2^b - 1}$$

Signed normalized fixed-point integers represent numbers in the range [-1,1]. The conversion from a signed normalized fixed-point value c to the corresponding floating-point value f is performed using

$$f = \max\left(\frac{c}{2^{b-1} - 1}, -1.0\right)$$

Only the range [$-2^{b-1} + 1$, $2^{b-1} - 1$] is used to represent signed fixed-point values in the range [-1,1]. For example, if b = 8, then the integer value -127 corresponds to -1.0 and the value 127 corresponds to 1.0. This equation is used everywhere that signed normalized fixed-point values are converted to floating-point.

Note that while zero is exactly expressible in this representation, one value (-128 in the example) is outside the representable range, and implementations **must** clamp it to -1.0. Where the value is subject to further processing by the implementation, e.g. during texture filtering, values less than -1.0 **may** be used but the result **must** be clamped before the value is returned to shaders.

3.10.2. Conversion From Floating-Point to Normalized Fixed-Point

The conversion from a floating-point value f to the corresponding unsigned normalized fixed-point value c is defined by first clamping f to the range [0,1], then computing

$$c = \text{convertFloatToUint}(f \times (2^b - 1), b)$$

where convertFloatToUint(r,b) returns one of the two unsigned binary integer values with exactly b bits which are closest to the floating-point value r. Implementations **should** round to nearest. If r is equal to an integer, then that integer value **must** be returned. In particular, if f is equal to 0.0 or 1.0, then c **must** be assigned 0 or $2^b - 1$, respectively.

The conversion from a floating-point value f to the corresponding signed normalized fixed-point value c is performed by clamping f to the range [-1,1], then computing

$$c = \text{convertFloatToInt}(f \times (2^{b-1} - 1), b)$$

where convertFloatToInt(r,b) returns one of the two signed two's-complement binary integer values with exactly b bits which are closest to the floating-point value r. Implementations **should** round to nearest. If r is equal to an integer, then that integer value **must** be returned. In particular, if f is equal to -1.0, 0.0, or 1.0, then c **must** be assigned $-(2^{b-1} - 1)$, 0, or $2^{b-1} - 1$, respectively.

This equation is used everywhere that floating-point values are converted to signed normalized

fixed-point.

3.11. String Representation

Strings passed into and returned from Vulkan API commands are usually defined to be null-terminated and UTF-8 encoded.

NOTE	Exceptions to this rule exist only when strings are defined or used by operating system APIs where that OS has a different convention. For example, VkExportMemoryWin32HandleInfoKHR::`name` is a null-terminated UTF-16 encoded string used in conjunction with Windows handles.

When a UTF-8 string is **returned from** a Vulkan API query, it is returned in a fixed-length buffer of C `char`. For example, a string returned in VkPhysicalDeviceProperties::`deviceName` has maximum length `VK_MAX_PHYSICAL_DEVICE_NAME_SIZE`, and a string returned in VkExtensionProperties ::`extensionName` has maximum length `VK_MAX_EXTENSION_NAME_SIZE`. The string, **including** its null terminator, will always fit completely within this buffer. If the string is shorter than the buffer size, the contents of `char` in the buffer following the null terminator are undefined.

When a UTF-8 string is **passed into** a Vulkan API, such as VkDeviceCreateInfo ::`ppEnabledExtensionNames`, there is no explicit limit on the length of that string. However, the string **must** contain a valid UTF-8 encoded string and **must** be null-terminated.

3.12. Common Object Types

Some types of Vulkan objects are used in many different structures and command parameters, and are described here. These types include *offsets*, *extents*, and *rectangles*.

3.12.1. Offsets

Offsets are used to describe a pixel location within an image or framebuffer, as an (x,y) location for two-dimensional images, or an (x,y,z) location for three-dimensional images.

A two-dimensional offset is defined by the structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkOffset2D {
    int32_t    x;
    int32_t    y;
} VkOffset2D;
```

- `x` is the x offset.
- `y` is the y offset.

A three-dimensional offset is defined by the structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkOffset3D {
    int32_t    x;
    int32_t    y;
    int32_t    z;
} VkOffset3D;
```

- `x` is the x offset.
- `y` is the y offset.
- `z` is the z offset.

3.12.2. Extents

Extents are used to describe the size of a rectangular region of pixels within an image or framebuffer, as (width,height) for two-dimensional images, or as (width,height,depth) for three-dimensional images.

A two-dimensional extent is defined by the structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkExtent2D {
    uint32_t    width;
    uint32_t    height;
} VkExtent2D;
```

- `width` is the width of the extent.
- `height` is the height of the extent.

A three-dimensional extent is defined by the structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkExtent3D {
    uint32_t    width;
    uint32_t    height;
    uint32_t    depth;
} VkExtent3D;
```

- `width` is the width of the extent.
- `height` is the height of the extent.
- `depth` is the depth of the extent.

3.12.3. Rectangles

Rectangles are used to describe a specified rectangular region of pixels within an image or framebuffer. Rectangles include both an offset and an extent of the same dimensionality, as

described above. Two-dimensional rectangles are defined by the structure

```
// Provided by VK_VERSION_1_0
typedef struct VkRect2D {
    VkOffset2D    offset;
    VkExtent2D    extent;
} VkRect2D;
```

- `offset` is a VkOffset2D specifying the rectangle offset.
- `extent` is a VkExtent2D specifying the rectangle extent.

3.12.4. Structure Types

Each value corresponds to a particular structure with a `sType` member with a matching name. As a general rule, the name of each VkStructureType value is obtained by taking the name of the structure, stripping the leading `Vk`, prefixing each capital letter with `_`, converting the entire resulting string to upper case, and prefixing it with `VK_STRUCTURE_TYPE_`. For example, structures of type VkImageCreateInfo correspond to a VkStructureType value of `VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO`, and thus a structure of this type **must** have its `sType` member set to this value before it is passed to the API.

The values `VK_STRUCTURE_TYPE_LOADER_INSTANCE_CREATE_INFO` and `VK_STRUCTURE_TYPE_LOADER_DEVICE_CREATE_INFO` are reserved for internal use by the loader, and do not have corresponding Vulkan structures in this Specification.

Structure types supported by the Vulkan API include:

```
// Provided by VK_VERSION_1_0
typedef enum VkStructureType {
    VK_STRUCTURE_TYPE_APPLICATION_INFO = 0,
    VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO = 1,
    VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO = 2,
    VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO = 3,
    VK_STRUCTURE_TYPE_SUBMIT_INFO = 4,
    VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO = 5,
    VK_STRUCTURE_TYPE_MAPPED_MEMORY_RANGE = 6,
    VK_STRUCTURE_TYPE_BIND_SPARSE_INFO = 7,
    VK_STRUCTURE_TYPE_FENCE_CREATE_INFO = 8,
    VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO = 9,
    VK_STRUCTURE_TYPE_EVENT_CREATE_INFO = 10,
    VK_STRUCTURE_TYPE_QUERY_POOL_CREATE_INFO = 11,
    VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO = 12,
    VK_STRUCTURE_TYPE_BUFFER_VIEW_CREATE_INFO = 13,
    VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO = 14,
    VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO = 15,
    VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO = 16,
    VK_STRUCTURE_TYPE_PIPELINE_CACHE_CREATE_INFO = 17,
    VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO = 18,
```

```
    VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO = 19,
    VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO = 20,
    VK_STRUCTURE_TYPE_PIPELINE_TESSELLATION_STATE_CREATE_INFO = 21,
    VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO = 22,
    VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO = 23,
    VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO = 24,
    VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO = 25,
    VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO = 26,
    VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO = 27,
    VK_STRUCTURE_TYPE_GRAPHICS_PIPELINE_CREATE_INFO = 28,
    VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO = 29,
    VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO = 30,
    VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO = 31,
    VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO = 32,
    VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO = 33,
    VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO = 34,
    VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET = 35,
    VK_STRUCTURE_TYPE_COPY_DESCRIPTOR_SET = 36,
    VK_STRUCTURE_TYPE_FRAMEBUFFER_CREATE_INFO = 37,
    VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO = 38,
    VK_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO = 39,
    VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO = 40,
    VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_INFO = 41,
    VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO = 42,
    VK_STRUCTURE_TYPE_RENDER_PASS_BEGIN_INFO = 43,
    VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER = 44,
    VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER = 45,
    VK_STRUCTURE_TYPE_MEMORY_BARRIER = 46,
    VK_STRUCTURE_TYPE_LOADER_INSTANCE_CREATE_INFO = 47,
    VK_STRUCTURE_TYPE_LOADER_DEVICE_CREATE_INFO = 48,
    VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_INFO = 1000157000,
    VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_INFO = 1000157001,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_16BIT_STORAGE_FEATURES = 1000083000,
    VK_STRUCTURE_TYPE_MEMORY_DEDICATED_REQUIREMENTS = 1000127000,
    VK_STRUCTURE_TYPE_MEMORY_DEDICATED_ALLOCATE_INFO = 1000127001,
    VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_FLAGS_INFO = 1000060000,
    VK_STRUCTURE_TYPE_DEVICE_GROUP_RENDER_PASS_BEGIN_INFO = 1000060003,
    VK_STRUCTURE_TYPE_DEVICE_GROUP_COMMAND_BUFFER_BEGIN_INFO = 1000060004,
    VK_STRUCTURE_TYPE_DEVICE_GROUP_SUBMIT_INFO = 1000060005,
    VK_STRUCTURE_TYPE_DEVICE_GROUP_BIND_SPARSE_INFO = 1000060006,
    VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_DEVICE_GROUP_INFO = 1000060013,
    VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_DEVICE_GROUP_INFO = 1000060014,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GROUP_PROPERTIES = 1000070000,
    VK_STRUCTURE_TYPE_DEVICE_GROUP_DEVICE_CREATE_INFO = 1000070001,
    VK_STRUCTURE_TYPE_BUFFER_MEMORY_REQUIREMENTS_INFO_2 = 1000146000,
    VK_STRUCTURE_TYPE_IMAGE_MEMORY_REQUIREMENTS_INFO_2 = 1000146001,
    VK_STRUCTURE_TYPE_IMAGE_SPARSE_MEMORY_REQUIREMENTS_INFO_2 = 1000146002,
    VK_STRUCTURE_TYPE_MEMORY_REQUIREMENTS_2 = 1000146003,
    VK_STRUCTURE_TYPE_SPARSE_IMAGE_MEMORY_REQUIREMENTS_2 = 1000146004,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2 = 1000059000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROPERTIES_2 = 1000059001,
```

```
    VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_2 = 1000059002,
    VK_STRUCTURE_TYPE_IMAGE_FORMAT_PROPERTIES_2 = 1000059003,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_FORMAT_INFO_2 = 1000059004,
    VK_STRUCTURE_TYPE_QUEUE_FAMILY_PROPERTIES_2 = 1000059005,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PROPERTIES_2 = 1000059006,
    VK_STRUCTURE_TYPE_SPARSE_IMAGE_FORMAT_PROPERTIES_2 = 1000059007,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SPARSE_IMAGE_FORMAT_INFO_2 = 1000059008,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_POINT_CLIPPING_PROPERTIES = 1000117000,
    VK_STRUCTURE_TYPE_RENDER_PASS_INPUT_ATTACHMENT_ASPECT_CREATE_INFO = 1000117001,
    VK_STRUCTURE_TYPE_IMAGE_VIEW_USAGE_CREATE_INFO = 1000117002,
    VK_STRUCTURE_TYPE_PIPELINE_TESSELLATION_DOMAIN_ORIGIN_STATE_CREATE_INFO =
1000117003,
    VK_STRUCTURE_TYPE_RENDER_PASS_MULTIVIEW_CREATE_INFO = 1000053000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_FEATURES = 1000053001,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_PROPERTIES = 1000053002,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VARIABLE_POINTERS_FEATURES = 1000120000,
    VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_CREATE_INFO = 1000156000,
    VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_INFO = 1000156001,
    VK_STRUCTURE_TYPE_BIND_IMAGE_PLANE_MEMORY_INFO = 1000156002,
    VK_STRUCTURE_TYPE_IMAGE_PLANE_MEMORY_REQUIREMENTS_INFO = 1000156003,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SAMPLER_YCBCR_CONVERSION_FEATURES = 1000156004,
    VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_IMAGE_FORMAT_PROPERTIES = 1000156005,
    VK_STRUCTURE_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_CREATE_INFO = 1000085000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_IMAGE_FORMAT_INFO = 1000071000,
    VK_STRUCTURE_TYPE_EXTERNAL_IMAGE_FORMAT_PROPERTIES = 1000071001,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_BUFFER_INFO = 1000071002,
    VK_STRUCTURE_TYPE_EXTERNAL_BUFFER_PROPERTIES = 1000071003,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ID_PROPERTIES = 1000071004,
    VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_BUFFER_CREATE_INFO = 1000072000,
    VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_IMAGE_CREATE_INFO = 1000072001,
    VK_STRUCTURE_TYPE_EXPORT_MEMORY_ALLOCATE_INFO = 1000072002,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_FENCE_INFO = 1000112000,
    VK_STRUCTURE_TYPE_EXTERNAL_FENCE_PROPERTIES = 1000112001,
    VK_STRUCTURE_TYPE_EXPORT_FENCE_CREATE_INFO = 1000113000,
    VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_CREATE_INFO = 1000077000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_SEMAPHORE_INFO = 1000076000,
    VK_STRUCTURE_TYPE_EXTERNAL_SEMAPHORE_PROPERTIES = 1000076001,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_3_PROPERTIES = 1000168000,
    VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_SUPPORT = 1000168001,
    VK_STRUCTURE_TYPE_IMAGE_FORMAT_LIST_CREATE_INFO = 1000147000,
    VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_2 = 1000109000,
    VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_2 = 1000109001,
    VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_2 = 1000109002,
    VK_STRUCTURE_TYPE_SUBPASS_DEPENDENCY_2 = 1000109003,
    VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO_2 = 1000109004,
    VK_STRUCTURE_TYPE_SUBPASS_BEGIN_INFO = 1000109005,
    VK_STRUCTURE_TYPE_SUBPASS_END_INFO = 1000109006,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_8BIT_STORAGE_FEATURES = 1000177000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DRIVER_PROPERTIES = 1000196000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_INT64_FEATURES = 1000180000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT16_INT8_FEATURES = 1000082000,
```

```
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FLOAT_CONTROLS_PROPERTIES = 1000197000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_STENCIL_RESOLVE_PROPERTIES = 1000199000,
    VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_DEPTH_STENCIL_RESOLVE = 1000199001,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_MEMORY_MODEL_FEATURES = 1000211000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGELESS_FRAMEBUFFER_FEATURES = 1000108000,
    VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENTS_CREATE_INFO = 1000108001,
    VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENT_IMAGE_INFO = 1000108002,
    VK_STRUCTURE_TYPE_RENDER_PASS_ATTACHMENT_BEGIN_INFO = 1000108003,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_UNIFORM_BUFFER_STANDARD_LAYOUT_FEATURES =
1000253000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_EXTENDED_TYPES_FEATURES =
1000175000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SEPARATE_DEPTH_STENCIL_LAYOUTS_FEATURES =
1000241000,
    VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_STENCIL_LAYOUT = 1000241001,
    VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_STENCIL_LAYOUT = 1000241002,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_FEATURES = 1000207000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_PROPERTIES = 1000207001,
    VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO = 1000207002,
    VK_STRUCTURE_TYPE_TIMELINE_SEMAPHORE_SUBMIT_INFO = 1000207003,
    VK_STRUCTURE_TYPE_SEMAPHORE_WAIT_INFO = 1000207004,
    VK_STRUCTURE_TYPE_SEMAPHORE_SIGNAL_INFO = 1000207005,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BUFFER_DEVICE_ADDRESS_FEATURES = 1000257000,
    VK_STRUCTURE_TYPE_BUFFER_DEVICE_ADDRESS_INFO = 1000244001,
    VK_STRUCTURE_TYPE_BUFFER_OPAQUE_CAPTURE_ADDRESS_CREATE_INFO = 1000257002,
    VK_STRUCTURE_TYPE_MEMORY_OPAQUE_CAPTURE_ADDRESS_ALLOCATE_INFO = 1000257003,
    VK_STRUCTURE_TYPE_DEVICE_MEMORY_OPAQUE_CAPTURE_ADDRESS_INFO = 1000257004,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TERMINATE_INVOCATION_FEATURES =
1000215000,
    VK_STRUCTURE_TYPE_MEMORY_BARRIER_2 = 1000314000,
    VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2 = 1000314001,
    VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2 = 1000314002,
    VK_STRUCTURE_TYPE_DEPENDENCY_INFO = 1000314003,
    VK_STRUCTURE_TYPE_SUBMIT_INFO_2 = 1000314004,
    VK_STRUCTURE_TYPE_SEMAPHORE_SUBMIT_INFO = 1000314005,
    VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO = 1000314006,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SYNCHRONIZATION_2_FEATURES = 1000314007,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ZERO_INITIALIZE_WORKGROUP_MEMORY_FEATURES =
1000325000,
    VK_STRUCTURE_TYPE_COPY_BUFFER_INFO_2 = 1000337000,
    VK_STRUCTURE_TYPE_COPY_IMAGE_INFO_2 = 1000337001,
    VK_STRUCTURE_TYPE_COPY_BUFFER_TO_IMAGE_INFO_2 = 1000337002,
    VK_STRUCTURE_TYPE_COPY_IMAGE_TO_BUFFER_INFO_2 = 1000337003,
    VK_STRUCTURE_TYPE_BLIT_IMAGE_INFO_2 = 1000337004,
    VK_STRUCTURE_TYPE_RESOLVE_IMAGE_INFO_2 = 1000337005,
    VK_STRUCTURE_TYPE_BUFFER_COPY_2 = 1000337006,
    VK_STRUCTURE_TYPE_IMAGE_COPY_2 = 1000337007,
    VK_STRUCTURE_TYPE_IMAGE_BLIT_2 = 1000337008,
    VK_STRUCTURE_TYPE_BUFFER_IMAGE_COPY_2 = 1000337009,
    VK_STRUCTURE_TYPE_IMAGE_RESOLVE_2 = 1000337010,
    VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_REQUIRED_SUBGROUP_SIZE_CREATE_INFO =
```

```
1000225001,
    VK_STRUCTURE_TYPE_RENDERING_INFO = 1000044000,
    VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO = 1000044001,
    VK_STRUCTURE_TYPE_PIPELINE_RENDERING_CREATE_INFO = 1000044002,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_FEATURES = 1000044003,
    VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_RENDERING_INFO = 1000044004,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_FEATURES =
1000280000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_PROPERTIES =
1000280001,
    VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_3 = 1000360000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_FEATURES = 1000413000,
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_PROPERTIES = 1000413001,
    VK_STRUCTURE_TYPE_DEVICE_BUFFER_MEMORY_REQUIREMENTS = 1000413002,
    VK_STRUCTURE_TYPE_DEVICE_IMAGE_MEMORY_REQUIREMENTS = 1000413003,
  // Provided by VK_KHR_swapchain
    VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR = 1000001000,
  // Provided by VK_KHR_swapchain
    VK_STRUCTURE_TYPE_PRESENT_INFO_KHR = 1000001001,
  // Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_surface
    VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_CAPABILITIES_KHR = 1000060007,
  // Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
    VK_STRUCTURE_TYPE_IMAGE_SWAPCHAIN_CREATE_INFO_KHR = 1000060008,
  // Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
    VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_SWAPCHAIN_INFO_KHR = 1000060009,
  // Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
    VK_STRUCTURE_TYPE_ACQUIRE_NEXT_IMAGE_INFO_KHR = 1000060010,
  // Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
    VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_INFO_KHR = 1000060011,
  // Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
    VK_STRUCTURE_TYPE_DEVICE_GROUP_SWAPCHAIN_CREATE_INFO_KHR = 1000060012,
  // Provided by VK_KHR_display
    VK_STRUCTURE_TYPE_DISPLAY_MODE_CREATE_INFO_KHR = 1000002000,
  // Provided by VK_KHR_display
    VK_STRUCTURE_TYPE_DISPLAY_SURFACE_CREATE_INFO_KHR = 1000002001,
  // Provided by VK_KHR_display_swapchain
    VK_STRUCTURE_TYPE_DISPLAY_PRESENT_INFO_KHR = 1000003000,
  // Provided by VK_KHR_xlib_surface
    VK_STRUCTURE_TYPE_XLIB_SURFACE_CREATE_INFO_KHR = 1000004000,
  // Provided by VK_KHR_xcb_surface
    VK_STRUCTURE_TYPE_XCB_SURFACE_CREATE_INFO_KHR = 1000005000,
  // Provided by VK_KHR_wayland_surface
    VK_STRUCTURE_TYPE_WAYLAND_SURFACE_CREATE_INFO_KHR = 1000006000,
  // Provided by VK_KHR_android_surface
    VK_STRUCTURE_TYPE_ANDROID_SURFACE_CREATE_INFO_KHR = 1000008000,
```

```
  // Provided by VK_KHR_win32_surface
    VK_STRUCTURE_TYPE_WIN32_SURFACE_CREATE_INFO_KHR = 1000009000,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_PROFILE_INFO_KHR = 1000023000,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_CAPABILITIES_KHR = 1000023001,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_PICTURE_RESOURCE_INFO_KHR = 1000023002,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_SESSION_MEMORY_REQUIREMENTS_KHR = 1000023003,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_BIND_VIDEO_SESSION_MEMORY_INFO_KHR = 1000023004,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_SESSION_CREATE_INFO_KHR = 1000023005,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_SESSION_PARAMETERS_CREATE_INFO_KHR = 1000023006,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_SESSION_PARAMETERS_UPDATE_INFO_KHR = 1000023007,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_BEGIN_CODING_INFO_KHR = 1000023008,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_END_CODING_INFO_KHR = 1000023009,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_CODING_CONTROL_INFO_KHR = 1000023010,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_REFERENCE_SLOT_INFO_KHR = 1000023011,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_QUEUE_FAMILY_VIDEO_PROPERTIES_KHR = 1000023012,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_PROFILE_LIST_INFO_KHR = 1000023013,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_FORMAT_INFO_KHR = 1000023014,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_VIDEO_FORMAT_PROPERTIES_KHR = 1000023015,
  // Provided by VK_KHR_video_queue
    VK_STRUCTURE_TYPE_QUEUE_FAMILY_QUERY_RESULT_STATUS_PROPERTIES_KHR = 1000023016,
  // Provided by VK_KHR_video_decode_queue
    VK_STRUCTURE_TYPE_VIDEO_DECODE_INFO_KHR = 1000024000,
  // Provided by VK_KHR_video_decode_queue
    VK_STRUCTURE_TYPE_VIDEO_DECODE_CAPABILITIES_KHR = 1000024001,
  // Provided by VK_KHR_video_decode_queue
    VK_STRUCTURE_TYPE_VIDEO_DECODE_USAGE_INFO_KHR = 1000024002,
  // Provided by VK_EXT_transform_feedback
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TRANSFORM_FEEDBACK_FEATURES_EXT = 1000028000,
  // Provided by VK_EXT_transform_feedback
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TRANSFORM_FEEDBACK_PROPERTIES_EXT = 1000028001,
  // Provided by VK_EXT_transform_feedback
    VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_STREAM_CREATE_INFO_EXT =
1000028002,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_CAPABILITIES_KHR = 1000038000,
```

```
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_CREATE_INFO_KHR =
1000038001,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_ADD_INFO_KHR = 1000038002,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_PICTURE_INFO_KHR = 1000038003,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_DPB_SLOT_INFO_KHR = 1000038004,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_NALU_SLICE_INFO_KHR = 1000038005,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_GOP_REMAINING_FRAME_INFO_KHR = 1000038006,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_PROFILE_INFO_KHR = 1000038007,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_RATE_CONTROL_INFO_KHR = 1000038008,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_RATE_CONTROL_LAYER_INFO_KHR = 1000038009,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_CREATE_INFO_KHR = 1000038010,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_QUALITY_LEVEL_PROPERTIES_KHR = 1000038011,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_GET_INFO_KHR = 1000038012,
  // Provided by VK_KHR_video_encode_h264
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_FEEDBACK_INFO_KHR =
1000038013,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_CAPABILITIES_KHR = 1000039000,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_CREATE_INFO_KHR =
1000039001,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_ADD_INFO_KHR = 1000039002,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_PICTURE_INFO_KHR = 1000039003,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_DPB_SLOT_INFO_KHR = 1000039004,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_NALU_SLICE_SEGMENT_INFO_KHR = 1000039005,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_GOP_REMAINING_FRAME_INFO_KHR = 1000039006,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_PROFILE_INFO_KHR = 1000039007,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_RATE_CONTROL_INFO_KHR = 1000039009,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_RATE_CONTROL_LAYER_INFO_KHR = 1000039010,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_CREATE_INFO_KHR = 1000039011,
```

```
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_QUALITY_LEVEL_PROPERTIES_KHR = 1000039012,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_GET_INFO_KHR = 1000039013,
  // Provided by VK_KHR_video_encode_h265
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_FEEDBACK_INFO_KHR =
1000039014,
  // Provided by VK_KHR_video_decode_h264
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_CAPABILITIES_KHR = 1000040000,
  // Provided by VK_KHR_video_decode_h264
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_PICTURE_INFO_KHR = 1000040001,
  // Provided by VK_KHR_video_decode_h264
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_PROFILE_INFO_KHR = 1000040003,
  // Provided by VK_KHR_video_decode_h264
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_SESSION_PARAMETERS_CREATE_INFO_KHR =
1000040004,
  // Provided by VK_KHR_video_decode_h264
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_SESSION_PARAMETERS_ADD_INFO_KHR = 1000040005,
  // Provided by VK_KHR_video_decode_h264
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_DPB_SLOT_INFO_KHR = 1000040006,
  // Provided by VK_EXT_astc_decode_mode
    VK_STRUCTURE_TYPE_IMAGE_VIEW_ASTC_DECODE_MODE_EXT = 1000067000,
  // Provided by VK_EXT_astc_decode_mode
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ASTC_DECODE_FEATURES_EXT = 1000067001,
  // Provided by VK_KHR_external_memory_win32
    VK_STRUCTURE_TYPE_IMPORT_MEMORY_WIN32_HANDLE_INFO_KHR = 1000073000,
  // Provided by VK_KHR_external_memory_win32
    VK_STRUCTURE_TYPE_EXPORT_MEMORY_WIN32_HANDLE_INFO_KHR = 1000073001,
  // Provided by VK_KHR_external_memory_win32
    VK_STRUCTURE_TYPE_MEMORY_WIN32_HANDLE_PROPERTIES_KHR = 1000073002,
  // Provided by VK_KHR_external_memory_win32
    VK_STRUCTURE_TYPE_MEMORY_GET_WIN32_HANDLE_INFO_KHR = 1000073003,
  // Provided by VK_KHR_external_memory_fd
    VK_STRUCTURE_TYPE_IMPORT_MEMORY_FD_INFO_KHR = 1000074000,
  // Provided by VK_KHR_external_memory_fd
    VK_STRUCTURE_TYPE_MEMORY_FD_PROPERTIES_KHR = 1000074001,
  // Provided by VK_KHR_external_memory_fd
    VK_STRUCTURE_TYPE_MEMORY_GET_FD_INFO_KHR = 1000074002,
  // Provided by VK_KHR_win32_keyed_mutex
    VK_STRUCTURE_TYPE_WIN32_KEYED_MUTEX_ACQUIRE_RELEASE_INFO_KHR = 1000075000,
  // Provided by VK_KHR_external_semaphore_win32
    VK_STRUCTURE_TYPE_IMPORT_SEMAPHORE_WIN32_HANDLE_INFO_KHR = 1000078000,
  // Provided by VK_KHR_external_semaphore_win32
    VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_WIN32_HANDLE_INFO_KHR = 1000078001,
  // Provided by VK_KHR_external_semaphore_win32
    VK_STRUCTURE_TYPE_D3D12_FENCE_SUBMIT_INFO_KHR = 1000078002,
  // Provided by VK_KHR_external_semaphore_win32
    VK_STRUCTURE_TYPE_SEMAPHORE_GET_WIN32_HANDLE_INFO_KHR = 1000078003,
  // Provided by VK_KHR_external_semaphore_fd
    VK_STRUCTURE_TYPE_IMPORT_SEMAPHORE_FD_INFO_KHR = 1000079000,
  // Provided by VK_KHR_external_semaphore_fd
```

```
    VK_STRUCTURE_TYPE_SEMAPHORE_GET_FD_INFO_KHR = 1000079001,
  // Provided by VK_KHR_push_descriptor
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PUSH_DESCRIPTOR_PROPERTIES_KHR = 1000080000,
  // Provided by VK_KHR_incremental_present
    VK_STRUCTURE_TYPE_PRESENT_REGIONS_KHR = 1000084000,
  // Provided by VK_EXT_display_surface_counter
    VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_2_EXT = 1000090000,
  // Provided by VK_EXT_display_control
    VK_STRUCTURE_TYPE_DISPLAY_POWER_INFO_EXT = 1000091000,
  // Provided by VK_EXT_display_control
    VK_STRUCTURE_TYPE_DEVICE_EVENT_INFO_EXT = 1000091001,
  // Provided by VK_EXT_display_control
    VK_STRUCTURE_TYPE_DISPLAY_EVENT_INFO_EXT = 1000091002,
  // Provided by VK_EXT_display_control
    VK_STRUCTURE_TYPE_SWAPCHAIN_COUNTER_CREATE_INFO_EXT = 1000091003,
  // Provided by VK_EXT_discard_rectangles
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DISCARD_RECTANGLE_PROPERTIES_EXT = 1000099000,
  // Provided by VK_EXT_discard_rectangles
    VK_STRUCTURE_TYPE_PIPELINE_DISCARD_RECTANGLE_STATE_CREATE_INFO_EXT = 1000099001,
  // Provided by VK_EXT_conservative_rasterization
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CONSERVATIVE_RASTERIZATION_PROPERTIES_EXT =
1000101000,
  // Provided by VK_EXT_conservative_rasterization
    VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_CONSERVATIVE_STATE_CREATE_INFO_EXT =
1000101001,
  // Provided by VK_EXT_depth_clip_enable
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLIP_ENABLE_FEATURES_EXT = 1000102000,
  // Provided by VK_EXT_depth_clip_enable
    VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_DEPTH_CLIP_STATE_CREATE_INFO_EXT =
1000102001,
  // Provided by VK_EXT_hdr_metadata
    VK_STRUCTURE_TYPE_HDR_METADATA_EXT = 1000105000,
  // Provided by VK_KHR_shared_presentable_image
    VK_STRUCTURE_TYPE_SHARED_PRESENT_SURFACE_CAPABILITIES_KHR = 1000111000,
  // Provided by VK_KHR_external_fence_win32
    VK_STRUCTURE_TYPE_IMPORT_FENCE_WIN32_HANDLE_INFO_KHR = 1000114000,
  // Provided by VK_KHR_external_fence_win32
    VK_STRUCTURE_TYPE_EXPORT_FENCE_WIN32_HANDLE_INFO_KHR = 1000114001,
  // Provided by VK_KHR_external_fence_win32
    VK_STRUCTURE_TYPE_FENCE_GET_WIN32_HANDLE_INFO_KHR = 1000114002,
  // Provided by VK_KHR_external_fence_fd
    VK_STRUCTURE_TYPE_IMPORT_FENCE_FD_INFO_KHR = 1000115000,
  // Provided by VK_KHR_external_fence_fd
    VK_STRUCTURE_TYPE_FENCE_GET_FD_INFO_KHR = 1000115001,
  // Provided by VK_KHR_performance_query
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PERFORMANCE_QUERY_FEATURES_KHR = 1000116000,
  // Provided by VK_KHR_performance_query
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PERFORMANCE_QUERY_PROPERTIES_KHR = 1000116001,
  // Provided by VK_KHR_performance_query
    VK_STRUCTURE_TYPE_QUERY_POOL_PERFORMANCE_CREATE_INFO_KHR = 1000116002,
  // Provided by VK_KHR_performance_query
```

```
    VK_STRUCTURE_TYPE_PERFORMANCE_QUERY_SUBMIT_INFO_KHR = 1000116003,
  // Provided by VK_KHR_performance_query
    VK_STRUCTURE_TYPE_ACQUIRE_PROFILING_LOCK_INFO_KHR = 1000116004,
  // Provided by VK_KHR_performance_query
    VK_STRUCTURE_TYPE_PERFORMANCE_COUNTER_KHR = 1000116005,
  // Provided by VK_KHR_performance_query
    VK_STRUCTURE_TYPE_PERFORMANCE_COUNTER_DESCRIPTION_KHR = 1000116006,
  // Provided by VK_KHR_get_surface_capabilities2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SURFACE_INFO_2_KHR = 1000119000,
  // Provided by VK_KHR_get_surface_capabilities2
    VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_2_KHR = 1000119001,
  // Provided by VK_KHR_get_surface_capabilities2
    VK_STRUCTURE_TYPE_SURFACE_FORMAT_2_KHR = 1000119002,
  // Provided by VK_KHR_get_display_properties2
    VK_STRUCTURE_TYPE_DISPLAY_PROPERTIES_2_KHR = 1000121000,
  // Provided by VK_KHR_get_display_properties2
    VK_STRUCTURE_TYPE_DISPLAY_PLANE_PROPERTIES_2_KHR = 1000121001,
  // Provided by VK_KHR_get_display_properties2
    VK_STRUCTURE_TYPE_DISPLAY_MODE_PROPERTIES_2_KHR = 1000121002,
  // Provided by VK_KHR_get_display_properties2
    VK_STRUCTURE_TYPE_DISPLAY_PLANE_INFO_2_KHR = 1000121003,
  // Provided by VK_KHR_get_display_properties2
    VK_STRUCTURE_TYPE_DISPLAY_PLANE_CAPABILITIES_2_KHR = 1000121004,
  // Provided by VK_EXT_debug_utils
    VK_STRUCTURE_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT = 1000128000,
  // Provided by VK_EXT_debug_utils
    VK_STRUCTURE_TYPE_DEBUG_UTILS_OBJECT_TAG_INFO_EXT = 1000128001,
  // Provided by VK_EXT_debug_utils
    VK_STRUCTURE_TYPE_DEBUG_UTILS_LABEL_EXT = 1000128002,
  // Provided by VK_EXT_debug_utils
    VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CALLBACK_DATA_EXT = 1000128003,
  // Provided by VK_EXT_debug_utils
    VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT = 1000128004,
  // Provided by VK_EXT_sample_locations
    VK_STRUCTURE_TYPE_SAMPLE_LOCATIONS_INFO_EXT = 1000143000,
  // Provided by VK_EXT_sample_locations
    VK_STRUCTURE_TYPE_RENDER_PASS_SAMPLE_LOCATIONS_BEGIN_INFO_EXT = 1000143001,
  // Provided by VK_EXT_sample_locations
    VK_STRUCTURE_TYPE_PIPELINE_SAMPLE_LOCATIONS_STATE_CREATE_INFO_EXT = 1000143002,
  // Provided by VK_EXT_sample_locations
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SAMPLE_LOCATIONS_PROPERTIES_EXT = 1000143003,
  // Provided by VK_EXT_sample_locations
    VK_STRUCTURE_TYPE_MULTISAMPLE_PROPERTIES_EXT = 1000143004,
  // Provided by VK_EXT_blend_operation_advanced
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BLEND_OPERATION_ADVANCED_FEATURES_EXT =
1000148000,
  // Provided by VK_EXT_blend_operation_advanced
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BLEND_OPERATION_ADVANCED_PROPERTIES_EXT =
1000148001,
  // Provided by VK_EXT_blend_operation_advanced
    VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_ADVANCED_STATE_CREATE_INFO_EXT =
```

```
1000148002,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET_ACCELERATION_STRUCTURE_KHR = 1000150007,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_BUILD_GEOMETRY_INFO_KHR = 1000150000,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_DEVICE_ADDRESS_INFO_KHR = 1000150002,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_AABBS_DATA_KHR = 1000150003,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_INSTANCES_DATA_KHR = 1000150004,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_TRIANGLES_DATA_KHR = 1000150005,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_KHR = 1000150006,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_VERSION_INFO_KHR = 1000150009,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_COPY_ACCELERATION_STRUCTURE_INFO_KHR = 1000150010,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_COPY_ACCELERATION_STRUCTURE_TO_MEMORY_INFO_KHR = 1000150011,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_COPY_MEMORY_TO_ACCELERATION_STRUCTURE_INFO_KHR = 1000150012,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ACCELERATION_STRUCTURE_FEATURES_KHR =
1000150013,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ACCELERATION_STRUCTURE_PROPERTIES_KHR =
1000150014,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_CREATE_INFO_KHR = 1000150017,
  // Provided by VK_KHR_acceleration_structure
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_BUILD_SIZES_INFO_KHR = 1000150020,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_PIPELINE_FEATURES_KHR = 1000347000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_PIPELINE_PROPERTIES_KHR =
1000347001,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_STRUCTURE_TYPE_RAY_TRACING_PIPELINE_CREATE_INFO_KHR = 1000150015,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_STRUCTURE_TYPE_RAY_TRACING_SHADER_GROUP_CREATE_INFO_KHR = 1000150016,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_STRUCTURE_TYPE_RAY_TRACING_PIPELINE_INTERFACE_CREATE_INFO_KHR = 1000150018,
  // Provided by VK_KHR_ray_query
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_QUERY_FEATURES_KHR = 1000348013,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_STRUCTURE_TYPE_DRM_FORMAT_MODIFIER_PROPERTIES_LIST_EXT = 1000158000,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_DRM_FORMAT_MODIFIER_INFO_EXT = 1000158002,
  // Provided by VK_EXT_image_drm_format_modifier
```

```
    VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_LIST_CREATE_INFO_EXT = 1000158003,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_EXPLICIT_CREATE_INFO_EXT = 1000158004,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_PROPERTIES_EXT = 1000158005,
  // Provided by VK_EXT_image_drm_format_modifier with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
    VK_STRUCTURE_TYPE_DRM_FORMAT_MODIFIER_PROPERTIES_LIST_2_EXT = 1000158006,
  // Provided by VK_EXT_validation_cache
    VK_STRUCTURE_TYPE_VALIDATION_CACHE_CREATE_INFO_EXT = 1000160000,
  // Provided by VK_EXT_validation_cache
    VK_STRUCTURE_TYPE_SHADER_MODULE_VALIDATION_CACHE_CREATE_INFO_EXT = 1000160001,
#ifdef VK_ENABLE_BETA_EXTENSIONS
  // Provided by VK_KHR_portability_subset
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PORTABILITY_SUBSET_FEATURES_KHR = 1000163000,
#endif
#ifdef VK_ENABLE_BETA_EXTENSIONS
  // Provided by VK_KHR_portability_subset
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PORTABILITY_SUBSET_PROPERTIES_KHR = 1000163001,
#endif
  // Provided by VK_EXT_filter_cubic
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_VIEW_IMAGE_FORMAT_INFO_EXT = 1000170000,
  // Provided by VK_EXT_filter_cubic
    VK_STRUCTURE_TYPE_FILTER_CUBIC_IMAGE_VIEW_IMAGE_FORMAT_PROPERTIES_EXT =
1000170001,
  // Provided by VK_EXT_external_memory_host
    VK_STRUCTURE_TYPE_IMPORT_MEMORY_HOST_POINTER_INFO_EXT = 1000178000,
  // Provided by VK_EXT_external_memory_host
    VK_STRUCTURE_TYPE_MEMORY_HOST_POINTER_PROPERTIES_EXT = 1000178001,
  // Provided by VK_EXT_external_memory_host
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_MEMORY_HOST_PROPERTIES_EXT =
1000178002,
  // Provided by VK_KHR_shader_clock
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_CLOCK_FEATURES_KHR = 1000181000,
  // Provided by VK_KHR_video_decode_h265
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_CAPABILITIES_KHR = 1000187000,
  // Provided by VK_KHR_video_decode_h265
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_SESSION_PARAMETERS_CREATE_INFO_KHR =
1000187001,
  // Provided by VK_KHR_video_decode_h265
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_SESSION_PARAMETERS_ADD_INFO_KHR = 1000187002,
  // Provided by VK_KHR_video_decode_h265
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_PROFILE_INFO_KHR = 1000187003,
  // Provided by VK_KHR_video_decode_h265
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_PICTURE_INFO_KHR = 1000187004,
  // Provided by VK_KHR_video_decode_h265
    VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_DPB_SLOT_INFO_KHR = 1000187005,
  // Provided by VK_KHR_global_priority
    VK_STRUCTURE_TYPE_DEVICE_QUEUE_GLOBAL_PRIORITY_CREATE_INFO_KHR = 1000174000,
  // Provided by VK_KHR_global_priority
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GLOBAL_PRIORITY_QUERY_FEATURES_KHR = 1000388000,
```

```
  // Provided by VK_KHR_global_priority
    VK_STRUCTURE_TYPE_QUEUE_FAMILY_GLOBAL_PRIORITY_PROPERTIES_KHR = 1000388001,
  // Provided by VK_EXT_pci_bus_info
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PCI_BUS_INFO_PROPERTIES_EXT = 1000212000,
  // Provided by VK_EXT_fragment_density_map
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_FEATURES_EXT = 1000218000,
  // Provided by VK_EXT_fragment_density_map
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_PROPERTIES_EXT =
1000218001,
  // Provided by VK_EXT_fragment_density_map
    VK_STRUCTURE_TYPE_RENDER_PASS_FRAGMENT_DENSITY_MAP_CREATE_INFO_EXT = 1000218002,
  // Provided by VK_EXT_fragment_density_map with VK_VERSION_1_3 or
VK_KHR_dynamic_rendering
    VK_STRUCTURE_TYPE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_INFO_EXT = 1000044007,
  // Provided by VK_KHR_fragment_shading_rate
    VK_STRUCTURE_TYPE_FRAGMENT_SHADING_RATE_ATTACHMENT_INFO_KHR = 1000226000,
  // Provided by VK_KHR_fragment_shading_rate
    VK_STRUCTURE_TYPE_PIPELINE_FRAGMENT_SHADING_RATE_STATE_CREATE_INFO_KHR =
1000226001,
  // Provided by VK_KHR_fragment_shading_rate
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_PROPERTIES_KHR =
1000226002,
  // Provided by VK_KHR_fragment_shading_rate
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_FEATURES_KHR = 1000226003,
  // Provided by VK_KHR_fragment_shading_rate
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_KHR = 1000226004,
  // Provided by VK_KHR_fragment_shading_rate with VK_VERSION_1_3 or
VK_KHR_dynamic_rendering
    VK_STRUCTURE_TYPE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_INFO_KHR =
1000044006,
  // Provided by VK_KHR_dynamic_rendering_local_read
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_LOCAL_READ_FEATURES_KHR =
1000232000,
  // Provided by VK_KHR_dynamic_rendering_local_read
    VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_LOCATION_INFO_KHR = 1000232001,
  // Provided by VK_KHR_dynamic_rendering_local_read
    VK_STRUCTURE_TYPE_RENDERING_INPUT_ATTACHMENT_INDEX_INFO_KHR = 1000232002,
  // Provided by VK_EXT_shader_image_atomic_int64
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_IMAGE_ATOMIC_INT64_FEATURES_EXT =
1000234000,
  // Provided by VK_KHR_shader_quad_control
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_QUAD_CONTROL_FEATURES_KHR = 1000235000,
  // Provided by VK_EXT_memory_budget
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_BUDGET_PROPERTIES_EXT = 1000237000,
  // Provided by VK_EXT_memory_priority
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PRIORITY_FEATURES_EXT = 1000238000,
  // Provided by VK_EXT_memory_priority
    VK_STRUCTURE_TYPE_MEMORY_PRIORITY_ALLOCATE_INFO_EXT = 1000238001,
  // Provided by VK_KHR_surface_protected_capabilities
    VK_STRUCTURE_TYPE_SURFACE_PROTECTED_CAPABILITIES_KHR = 1000239000,
  // Provided by VK_KHR_present_wait
```

```
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_WAIT_FEATURES_KHR = 1000248000,
  // Provided by VK_EXT_fragment_shader_interlock
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_INTERLOCK_FEATURES_EXT =
1000251000,
  // Provided by VK_EXT_ycbcr_image_arrays
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_YCBCR_IMAGE_ARRAYS_FEATURES_EXT = 1000252000,
  // Provided by VK_EXT_provoking_vertex
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROVOKING_VERTEX_FEATURES_EXT = 1000254000,
  // Provided by VK_EXT_provoking_vertex
    VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_PROVOKING_VERTEX_STATE_CREATE_INFO_EXT =
1000254001,
  // Provided by VK_EXT_provoking_vertex
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROVOKING_VERTEX_PROPERTIES_EXT = 1000254002,
  // Provided by VK_EXT_full_screen_exclusive
    VK_STRUCTURE_TYPE_SURFACE_FULL_SCREEN_EXCLUSIVE_INFO_EXT = 1000255000,
  // Provided by VK_EXT_full_screen_exclusive
    VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_FULL_SCREEN_EXCLUSIVE_EXT = 1000255002,
  // Provided by VK_KHR_win32_surface with VK_EXT_full_screen_exclusive
    VK_STRUCTURE_TYPE_SURFACE_FULL_SCREEN_EXCLUSIVE_WIN32_INFO_EXT = 1000255001,
  // Provided by VK_EXT_headless_surface
    VK_STRUCTURE_TYPE_HEADLESS_SURFACE_CREATE_INFO_EXT = 1000256000,
  // Provided by VK_EXT_shader_atomic_float
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_FLOAT_FEATURES_EXT = 1000260000,
  // Provided by VK_KHR_pipeline_executable_properties
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_EXECUTABLE_PROPERTIES_FEATURES_KHR =
1000269000,
  // Provided by VK_KHR_pipeline_executable_properties
    VK_STRUCTURE_TYPE_PIPELINE_INFO_KHR = 1000269001,
  // Provided by VK_KHR_pipeline_executable_properties
    VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_PROPERTIES_KHR = 1000269002,
  // Provided by VK_KHR_pipeline_executable_properties
    VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_INFO_KHR = 1000269003,
  // Provided by VK_KHR_pipeline_executable_properties
    VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_STATISTIC_KHR = 1000269004,
  // Provided by VK_KHR_pipeline_executable_properties
    VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_INTERNAL_REPRESENTATION_KHR = 1000269005,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_HOST_IMAGE_COPY_FEATURES_EXT = 1000270000,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_HOST_IMAGE_COPY_PROPERTIES_EXT = 1000270001,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_MEMORY_TO_IMAGE_COPY_EXT = 1000270002,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_IMAGE_TO_MEMORY_COPY_EXT = 1000270003,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_COPY_IMAGE_TO_MEMORY_INFO_EXT = 1000270004,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_COPY_MEMORY_TO_IMAGE_INFO_EXT = 1000270005,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_HOST_IMAGE_LAYOUT_TRANSITION_INFO_EXT = 1000270006,
  // Provided by VK_EXT_host_image_copy
```

```
    VK_STRUCTURE_TYPE_COPY_IMAGE_TO_IMAGE_INFO_EXT = 1000270007,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_SUBRESOURCE_HOST_MEMCPY_SIZE_EXT = 1000270008,
  // Provided by VK_EXT_host_image_copy
    VK_STRUCTURE_TYPE_HOST_IMAGE_COPY_DEVICE_PERFORMANCE_QUERY_EXT = 1000270009,
  // Provided by VK_KHR_map_memory2
    VK_STRUCTURE_TYPE_MEMORY_MAP_INFO_KHR = 1000271000,
  // Provided by VK_KHR_map_memory2
    VK_STRUCTURE_TYPE_MEMORY_UNMAP_INFO_KHR = 1000271001,
  // Provided by VK_EXT_map_memory_placed
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAP_MEMORY_PLACED_FEATURES_EXT = 1000272000,
  // Provided by VK_EXT_map_memory_placed
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAP_MEMORY_PLACED_PROPERTIES_EXT = 1000272001,
  // Provided by VK_EXT_map_memory_placed
    VK_STRUCTURE_TYPE_MEMORY_MAP_PLACED_INFO_EXT = 1000272002,
  // Provided by VK_EXT_shader_atomic_float2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_FLOAT_2_FEATURES_EXT = 1000273000,
  // Provided by VK_EXT_depth_bias_control
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_BIAS_CONTROL_FEATURES_EXT = 1000283000,
  // Provided by VK_EXT_depth_bias_control
    VK_STRUCTURE_TYPE_DEPTH_BIAS_INFO_EXT = 1000283001,
  // Provided by VK_EXT_depth_bias_control
    VK_STRUCTURE_TYPE_DEPTH_BIAS_REPRESENTATION_INFO_EXT = 1000283002,
  // Provided by VK_EXT_device_memory_report
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_MEMORY_REPORT_FEATURES_EXT = 1000284000,
  // Provided by VK_EXT_device_memory_report
    VK_STRUCTURE_TYPE_DEVICE_DEVICE_MEMORY_REPORT_CREATE_INFO_EXT = 1000284001,
  // Provided by VK_EXT_device_memory_report
    VK_STRUCTURE_TYPE_DEVICE_MEMORY_REPORT_CALLBACK_DATA_EXT = 1000284002,
  // Provided by VK_EXT_robustness2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ROBUSTNESS_2_FEATURES_EXT = 1000286000,
  // Provided by VK_EXT_robustness2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ROBUSTNESS_2_PROPERTIES_EXT = 1000286001,
  // Provided by VK_EXT_custom_border_color
    VK_STRUCTURE_TYPE_SAMPLER_CUSTOM_BORDER_COLOR_CREATE_INFO_EXT = 1000287000,
  // Provided by VK_EXT_custom_border_color
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CUSTOM_BORDER_COLOR_PROPERTIES_EXT = 1000287001,
  // Provided by VK_EXT_custom_border_color
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CUSTOM_BORDER_COLOR_FEATURES_EXT = 1000287002,
  // Provided by VK_KHR_pipeline_library
    VK_STRUCTURE_TYPE_PIPELINE_LIBRARY_CREATE_INFO_KHR = 1000290000,
  // Provided by VK_KHR_present_id
    VK_STRUCTURE_TYPE_PRESENT_ID_KHR = 1000294000,
  // Provided by VK_KHR_present_id
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_ID_FEATURES_KHR = 1000294001,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_INFO_KHR = 1000299000,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_RATE_CONTROL_INFO_KHR = 1000299001,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_RATE_CONTROL_LAYER_INFO_KHR = 1000299002,
```

```
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_CAPABILITIES_KHR = 1000299003,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_USAGE_INFO_KHR = 1000299004,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_QUERY_POOL_VIDEO_ENCODE_FEEDBACK_CREATE_INFO_KHR = 1000299005,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_QUALITY_LEVEL_INFO_KHR =
1000299006,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUALITY_LEVEL_PROPERTIES_KHR = 1000299007,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUALITY_LEVEL_INFO_KHR = 1000299008,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_SESSION_PARAMETERS_GET_INFO_KHR = 1000299009,
  // Provided by VK_KHR_video_encode_queue
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_SESSION_PARAMETERS_FEEDBACK_INFO_KHR = 1000299010,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_EXPORT_METAL_OBJECT_CREATE_INFO_EXT = 1000311000,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_EXPORT_METAL_OBJECTS_INFO_EXT = 1000311001,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_EXPORT_METAL_DEVICE_INFO_EXT = 1000311002,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_EXPORT_METAL_COMMAND_QUEUE_INFO_EXT = 1000311003,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_EXPORT_METAL_BUFFER_INFO_EXT = 1000311004,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_IMPORT_METAL_BUFFER_INFO_EXT = 1000311005,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_EXPORT_METAL_TEXTURE_INFO_EXT = 1000311006,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_IMPORT_METAL_TEXTURE_INFO_EXT = 1000311007,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_EXPORT_METAL_IO_SURFACE_INFO_EXT = 1000311008,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_IMPORT_METAL_IO_SURFACE_INFO_EXT = 1000311009,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_EXPORT_METAL_SHARED_EVENT_INFO_EXT = 1000311010,
  // Provided by VK_EXT_metal_objects
    VK_STRUCTURE_TYPE_IMPORT_METAL_SHARED_EVENT_INFO_EXT = 1000311011,
  // Provided by VK_KHR_fragment_shader_barycentric
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_BARYCENTRIC_FEATURES_KHR =
1000203000,
  // Provided by VK_KHR_fragment_shader_barycentric
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_BARYCENTRIC_PROPERTIES_KHR =
1000322000,
  // Provided by VK_KHR_shader_subgroup_uniform_control_flow

VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_UNIFORM_CONTROL_FLOW_FEATURES_KHR =
1000323000,
```

```
  // Provided by VK_EXT_fragment_density_map2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_2_FEATURES_EXT =
1000332000,
  // Provided by VK_EXT_fragment_density_map2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_2_PROPERTIES_EXT =
1000332001,
  // Provided by VK_KHR_workgroup_memory_explicit_layout
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_WORKGROUP_MEMORY_EXPLICIT_LAYOUT_FEATURES_KHR =
1000336000,
  // Provided by VK_EXT_attachment_feedback_loop_layout
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ATTACHMENT_FEEDBACK_LOOP_LAYOUT_FEATURES_EXT =
1000339000,
  // Provided by VK_EXT_vertex_input_dynamic_state
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_INPUT_DYNAMIC_STATE_FEATURES_EXT =
1000352000,
  // Provided by VK_EXT_shader_object, VK_EXT_vertex_input_dynamic_state
    VK_STRUCTURE_TYPE_VERTEX_INPUT_BINDING_DESCRIPTION_2_EXT = 1000352001,
  // Provided by VK_EXT_shader_object, VK_EXT_vertex_input_dynamic_state
    VK_STRUCTURE_TYPE_VERTEX_INPUT_ATTRIBUTE_DESCRIPTION_2_EXT = 1000352002,
  // Provided by VK_EXT_physical_device_drm
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DRM_PROPERTIES_EXT = 1000353000,
  // Provided by VK_EXT_depth_clip_control
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLIP_CONTROL_FEATURES_EXT = 1000355000,
  // Provided by VK_EXT_depth_clip_control
    VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_DEPTH_CLIP_CONTROL_CREATE_INFO_EXT =
1000355001,
  // Provided by VK_EXT_primitive_topology_list_restart
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRIMITIVE_TOPOLOGY_LIST_RESTART_FEATURES_EXT =
1000356000,
  // Provided by VK_EXT_present_mode_fifo_latest_ready
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_MODE_FIFO_LATEST_READY_FEATURES_EXT =
1000361000,
  // Provided by VK_EXT_frame_boundary
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAME_BOUNDARY_FEATURES_EXT = 1000375000,
  // Provided by VK_EXT_frame_boundary
    VK_STRUCTURE_TYPE_FRAME_BOUNDARY_EXT = 1000375001,
  // Provided by VK_EXT_color_write_enable
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COLOR_WRITE_ENABLE_FEATURES_EXT = 1000381000,
  // Provided by VK_EXT_color_write_enable
    VK_STRUCTURE_TYPE_PIPELINE_COLOR_WRITE_CREATE_INFO_EXT = 1000381001,
  // Provided by VK_EXT_primitives_generated_query
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRIMITIVES_GENERATED_QUERY_FEATURES_EXT =
1000382000,
  // Provided by VK_KHR_ray_tracing_maintenance1
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_MAINTENANCE_1_FEATURES_KHR =
1000386000,
  // Provided by VK_EXT_image_view_min_lod
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_VIEW_MIN_LOD_FEATURES_EXT = 1000391000,
  // Provided by VK_EXT_image_view_min_lod
    VK_STRUCTURE_TYPE_IMAGE_VIEW_MIN_LOD_CREATE_INFO_EXT = 1000391001,
  // Provided by VK_EXT_multi_draw
```

```
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTI_DRAW_FEATURES_EXT = 1000392000,
  // Provided by VK_EXT_multi_draw
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTI_DRAW_PROPERTIES_EXT = 1000392001,
  // Provided by VK_EXT_image_2d_view_of_3d
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_2D_VIEW_OF_3D_FEATURES_EXT = 1000393000,
  // Provided by VK_EXT_shader_tile_image
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TILE_IMAGE_FEATURES_EXT = 1000395000,
  // Provided by VK_EXT_shader_tile_image
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TILE_IMAGE_PROPERTIES_EXT = 1000395001,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_MICROMAP_BUILD_INFO_EXT = 1000396000,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_MICROMAP_VERSION_INFO_EXT = 1000396001,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_COPY_MICROMAP_INFO_EXT = 1000396002,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_COPY_MICROMAP_TO_MEMORY_INFO_EXT = 1000396003,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_COPY_MEMORY_TO_MICROMAP_INFO_EXT = 1000396004,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_OPACITY_MICROMAP_FEATURES_EXT = 1000396005,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_OPACITY_MICROMAP_PROPERTIES_EXT = 1000396006,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_MICROMAP_CREATE_INFO_EXT = 1000396007,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_MICROMAP_BUILD_SIZES_INFO_EXT = 1000396008,
  // Provided by VK_EXT_opacity_micromap
    VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_TRIANGLES_OPACITY_MICROMAP_EXT =
1000396009,
  // Provided by VK_EXT_border_color_swizzle
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BORDER_COLOR_SWIZZLE_FEATURES_EXT = 1000411000,
  // Provided by VK_EXT_border_color_swizzle
    VK_STRUCTURE_TYPE_SAMPLER_BORDER_COLOR_COMPONENT_MAPPING_CREATE_INFO_EXT =
1000411001,
  // Provided by VK_KHR_shader_subgroup_rotate
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_ROTATE_FEATURES_KHR =
1000416000,
  // Provided by VK_EXT_non_seamless_cube_map
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NON_SEAMLESS_CUBE_MAP_FEATURES_EXT = 1000422000,
  // Provided by VK_KHR_shader_maximal_reconvergence
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MAXIMAL_RECONVERGENCE_FEATURES_KHR =
1000434000,
  // Provided by VK_EXT_nested_command_buffer
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NESTED_COMMAND_BUFFER_FEATURES_EXT = 1000451000,
  // Provided by VK_EXT_nested_command_buffer
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NESTED_COMMAND_BUFFER_PROPERTIES_EXT =
1000451001,
  // Provided by VK_EXT_external_memory_acquire_unmodified
    VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_ACQUIRE_UNMODIFIED_EXT = 1000453000,
  // Provided by VK_EXT_extended_dynamic_state3
```

```
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTENDED_DYNAMIC_STATE_3_FEATURES_EXT =
1000455000,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTENDED_DYNAMIC_STATE_3_PROPERTIES_EXT =
1000455001,
  // Provided by VK_EXT_shader_module_identifier
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MODULE_IDENTIFIER_FEATURES_EXT =
1000462000,
  // Provided by VK_EXT_shader_module_identifier
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MODULE_IDENTIFIER_PROPERTIES_EXT =
1000462001,
  // Provided by VK_EXT_shader_module_identifier
    VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_MODULE_IDENTIFIER_CREATE_INFO_EXT =
1000462002,
  // Provided by VK_EXT_shader_module_identifier
    VK_STRUCTURE_TYPE_SHADER_MODULE_IDENTIFIER_EXT = 1000462003,
  // Provided by VK_KHR_maintenance5
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_5_FEATURES_KHR = 1000470000,
  // Provided by VK_KHR_maintenance5
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_5_PROPERTIES_KHR = 1000470001,
  // Provided by VK_KHR_maintenance5
    VK_STRUCTURE_TYPE_RENDERING_AREA_INFO_KHR = 1000470003,
  // Provided by VK_KHR_maintenance5
    VK_STRUCTURE_TYPE_DEVICE_IMAGE_SUBRESOURCE_INFO_KHR = 1000470004,
  // Provided by VK_KHR_maintenance5
    VK_STRUCTURE_TYPE_SUBRESOURCE_LAYOUT_2_KHR = 1000338002,
  // Provided by VK_KHR_maintenance5
    VK_STRUCTURE_TYPE_IMAGE_SUBRESOURCE_2_KHR = 1000338003,
  // Provided by VK_KHR_maintenance5
    VK_STRUCTURE_TYPE_PIPELINE_CREATE_FLAGS_2_CREATE_INFO_KHR = 1000470005,
  // Provided by VK_KHR_maintenance5
    VK_STRUCTURE_TYPE_BUFFER_USAGE_FLAGS_2_CREATE_INFO_KHR = 1000470006,
  // Provided by VK_KHR_ray_tracing_position_fetch
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_POSITION_FETCH_FEATURES_KHR =
1000481000,
  // Provided by VK_EXT_shader_object
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_OBJECT_FEATURES_EXT = 1000482000,
  // Provided by VK_EXT_shader_object
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_OBJECT_PROPERTIES_EXT = 1000482001,
  // Provided by VK_EXT_shader_object
    VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT = 1000482002,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_BINARY_FEATURES_KHR = 1000483000,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_PIPELINE_BINARY_CREATE_INFO_KHR = 1000483001,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_PIPELINE_BINARY_INFO_KHR = 1000483002,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_PIPELINE_BINARY_KEY_KHR = 1000483003,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_BINARY_PROPERTIES_KHR = 1000483004,
```

```
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_RELEASE_CAPTURED_PIPELINE_DATA_INFO_KHR = 1000483005,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_PIPELINE_BINARY_DATA_INFO_KHR = 1000483006,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_PIPELINE_CREATE_INFO_KHR = 1000483007,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_DEVICE_PIPELINE_BINARY_INTERNAL_CACHE_CONTROL_KHR = 1000483008,
  // Provided by VK_KHR_pipeline_binary
    VK_STRUCTURE_TYPE_PIPELINE_BINARY_HANDLES_INFO_KHR = 1000483009,
  // Provided by VK_EXT_legacy_vertex_attributes
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LEGACY_VERTEX_ATTRIBUTES_FEATURES_EXT =
1000495000,
  // Provided by VK_EXT_legacy_vertex_attributes
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LEGACY_VERTEX_ATTRIBUTES_PROPERTIES_EXT =
1000495001,
  // Provided by VK_EXT_layer_settings
    VK_STRUCTURE_TYPE_LAYER_SETTINGS_CREATE_INFO_EXT = 1000496000,
  // Provided by VK_EXT_dynamic_rendering_unused_attachments

VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_UNUSED_ATTACHMENTS_FEATURES_EXT =
1000499000,
  // Provided by VK_KHR_cooperative_matrix
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COOPERATIVE_MATRIX_FEATURES_KHR = 1000506000,
  // Provided by VK_KHR_cooperative_matrix
    VK_STRUCTURE_TYPE_COOPERATIVE_MATRIX_PROPERTIES_KHR = 1000506001,
  // Provided by VK_KHR_cooperative_matrix
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COOPERATIVE_MATRIX_PROPERTIES_KHR = 1000506002,
  // Provided by VK_KHR_compute_shader_derivatives
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COMPUTE_SHADER_DERIVATIVES_FEATURES_KHR =
1000201000,
  // Provided by VK_KHR_compute_shader_derivatives
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COMPUTE_SHADER_DERIVATIVES_PROPERTIES_KHR =
1000511000,
  // Provided by VK_KHR_video_decode_av1
    VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_CAPABILITIES_KHR = 1000512000,
  // Provided by VK_KHR_video_decode_av1
    VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_PICTURE_INFO_KHR = 1000512001,
  // Provided by VK_KHR_video_decode_av1
    VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_PROFILE_INFO_KHR = 1000512003,
  // Provided by VK_KHR_video_decode_av1
    VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_SESSION_PARAMETERS_CREATE_INFO_KHR =
1000512004,
  // Provided by VK_KHR_video_decode_av1
    VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_DPB_SLOT_INFO_KHR = 1000512005,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_CAPABILITIES_KHR = 1000513000,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_SESSION_PARAMETERS_CREATE_INFO_KHR =
1000513001,
  // Provided by VK_KHR_video_encode_av1
```

```
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_PICTURE_INFO_KHR = 1000513002,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_DPB_SLOT_INFO_KHR = 1000513003,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_AV1_FEATURES_KHR = 1000513004,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_PROFILE_INFO_KHR = 1000513005,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_RATE_CONTROL_INFO_KHR = 1000513006,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_RATE_CONTROL_LAYER_INFO_KHR = 1000513007,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_QUALITY_LEVEL_PROPERTIES_KHR = 1000513008,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_SESSION_CREATE_INFO_KHR = 1000513009,
  // Provided by VK_KHR_video_encode_av1
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_GOP_REMAINING_FRAME_INFO_KHR = 1000513010,
  // Provided by VK_KHR_video_maintenance1
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_MAINTENANCE_1_FEATURES_KHR = 1000515000,
  // Provided by VK_KHR_video_maintenance1
    VK_STRUCTURE_TYPE_VIDEO_INLINE_QUERY_INFO_KHR = 1000515001,
  // Provided by VK_EXT_attachment_feedback_loop_dynamic_state

VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ATTACHMENT_FEEDBACK_LOOP_DYNAMIC_STATE_FEATURES_EXT
= 1000524000,
  // Provided by VK_KHR_vertex_attribute_divisor
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_ATTRIBUTE_DIVISOR_PROPERTIES_KHR =
1000525000,
  // Provided by VK_KHR_vertex_attribute_divisor
    VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_DIVISOR_STATE_CREATE_INFO_KHR =
1000190001,
  // Provided by VK_KHR_vertex_attribute_divisor
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_ATTRIBUTE_DIVISOR_FEATURES_KHR =
1000190002,
  // Provided by VK_KHR_shader_float_controls2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT_CONTROLS_2_FEATURES_KHR =
1000528000,
  // Provided by VK_KHR_index_type_uint8
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_INDEX_TYPE_UINT8_FEATURES_KHR = 1000265000,
  // Provided by VK_KHR_line_rasterization
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LINE_RASTERIZATION_FEATURES_KHR = 1000259000,
  // Provided by VK_KHR_line_rasterization
    VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_LINE_STATE_CREATE_INFO_KHR = 1000259001,
  // Provided by VK_KHR_line_rasterization
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LINE_RASTERIZATION_PROPERTIES_KHR = 1000259002,
  // Provided by VK_KHR_calibrated_timestamps
    VK_STRUCTURE_TYPE_CALIBRATED_TIMESTAMP_INFO_KHR = 1000184000,
  // Provided by VK_KHR_shader_expect_assume
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_EXPECT_ASSUME_FEATURES_KHR = 1000544000,
  // Provided by VK_KHR_maintenance6
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_6_FEATURES_KHR = 1000545000,
```

```
  // Provided by VK_KHR_maintenance6
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_6_PROPERTIES_KHR = 1000545001,
  // Provided by VK_KHR_maintenance6
    VK_STRUCTURE_TYPE_BIND_MEMORY_STATUS_KHR = 1000545002,
  // Provided by VK_KHR_maintenance6
    VK_STRUCTURE_TYPE_BIND_DESCRIPTOR_SETS_INFO_KHR = 1000545003,
  // Provided by VK_KHR_maintenance6
    VK_STRUCTURE_TYPE_PUSH_CONSTANTS_INFO_KHR = 1000545004,
  // Provided by VK_KHR_maintenance6 with VK_KHR_push_descriptor
    VK_STRUCTURE_TYPE_PUSH_DESCRIPTOR_SET_INFO_KHR = 1000545005,
  // Provided by VK_KHR_maintenance6 with VK_KHR_push_descriptor
    VK_STRUCTURE_TYPE_PUSH_DESCRIPTOR_SET_WITH_TEMPLATE_INFO_KHR = 1000545006,
  // Provided by VK_KHR_maintenance6 with VK_EXT_descriptor_buffer
    VK_STRUCTURE_TYPE_SET_DESCRIPTOR_BUFFER_OFFSETS_INFO_EXT = 1000545007,
  // Provided by VK_KHR_maintenance6 with VK_EXT_descriptor_buffer
    VK_STRUCTURE_TYPE_BIND_DESCRIPTOR_BUFFER_EMBEDDED_SAMPLERS_INFO_EXT = 1000545008,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_CAPABILITIES_KHR = 1000553000,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_FORMAT_QUANTIZATION_MAP_PROPERTIES_KHR = 1000553001,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_INFO_KHR = 1000553002,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_SESSION_PARAMETERS_CREATE_INFO_KHR
= 1000553005,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_QUANTIZATION_MAP_FEATURES_KHR =
1000553009,
  // Provided by VK_KHR_video_encode_h264 with VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_QUANTIZATION_MAP_CAPABILITIES_KHR =
1000553003,
  // Provided by VK_KHR_video_encode_h265 with VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_QUANTIZATION_MAP_CAPABILITIES_KHR =
1000553004,
  // Provided by VK_KHR_video_encode_h265 with VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_FORMAT_H265_QUANTIZATION_MAP_PROPERTIES_KHR = 1000553006,
  // Provided by VK_KHR_video_encode_av1 with VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_QUANTIZATION_MAP_CAPABILITIES_KHR = 1000553007,
  // Provided by VK_KHR_video_encode_av1 with VK_KHR_video_encode_quantization_map
    VK_STRUCTURE_TYPE_VIDEO_FORMAT_AV1_QUANTIZATION_MAP_PROPERTIES_KHR = 1000553008,
  // Provided by VK_KHR_shader_relaxed_extended_instruction
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_RELAXED_EXTENDED_INSTRUCTION_FEATURES_KHR
= 1000558000,
  // Provided by VK_KHR_maintenance7
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_7_FEATURES_KHR = 1000562000,
  // Provided by VK_KHR_maintenance7
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_7_PROPERTIES_KHR = 1000562001,
  // Provided by VK_KHR_maintenance7
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_PROPERTIES_LIST_KHR = 1000562002,
  // Provided by VK_KHR_maintenance7
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_PROPERTIES_KHR = 1000562003,
```

```
  // Provided by VK_KHR_maintenance7
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_VULKAN_PROPERTIES_KHR = 1000562004,
  // Provided by VK_EXT_shader_replicated_composites
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_REPLICATED_COMPOSITES_FEATURES_EXT =
1000564000,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_GENERATED_COMMANDS_FEATURES_EXT =
1000572000,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_GENERATED_COMMANDS_PROPERTIES_EXT =
1000572001,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_GENERATED_COMMANDS_MEMORY_REQUIREMENTS_INFO_EXT = 1000572002,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_CREATE_INFO_EXT = 1000572003,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_GENERATED_COMMANDS_INFO_EXT = 1000572004,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_INDIRECT_COMMANDS_LAYOUT_CREATE_INFO_EXT = 1000572006,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_INDIRECT_COMMANDS_LAYOUT_TOKEN_EXT = 1000572007,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_WRITE_INDIRECT_EXECUTION_SET_PIPELINE_EXT = 1000572008,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_WRITE_INDIRECT_EXECUTION_SET_SHADER_EXT = 1000572009,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_PIPELINE_INFO_EXT = 1000572010,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_SHADER_INFO_EXT = 1000572011,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_SHADER_LAYOUT_INFO_EXT = 1000572012,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_GENERATED_COMMANDS_PIPELINE_INFO_EXT = 1000572013,
  // Provided by VK_EXT_device_generated_commands
    VK_STRUCTURE_TYPE_GENERATED_COMMANDS_SHADER_INFO_EXT = 1000572014,
  // Provided by VK_EXT_depth_clamp_control
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLAMP_CONTROL_FEATURES_EXT = 1000582000,
  // Provided by VK_EXT_depth_clamp_control
    VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_DEPTH_CLAMP_CONTROL_CREATE_INFO_EXT =
1000582001,
  // Provided by VK_KHR_dynamic_rendering
    VK_STRUCTURE_TYPE_RENDERING_INFO_KHR = VK_STRUCTURE_TYPE_RENDERING_INFO,
  // Provided by VK_KHR_dynamic_rendering
    VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO_KHR =
VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO,
  // Provided by VK_KHR_dynamic_rendering
    VK_STRUCTURE_TYPE_PIPELINE_RENDERING_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_PIPELINE_RENDERING_CREATE_INFO,
  // Provided by VK_KHR_dynamic_rendering
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_FEATURES,
```

```
  // Provided by VK_KHR_dynamic_rendering
    VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_RENDERING_INFO_KHR =
VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_RENDERING_INFO,
  // Provided by VK_KHR_multiview
    VK_STRUCTURE_TYPE_RENDER_PASS_MULTIVIEW_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_RENDER_PASS_MULTIVIEW_CREATE_INFO,
  // Provided by VK_KHR_multiview
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_FEATURES,
  // Provided by VK_KHR_multiview
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_PROPERTIES,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROPERTIES_2_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROPERTIES_2,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_2_KHR = VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_2,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_IMAGE_FORMAT_PROPERTIES_2_KHR =
VK_STRUCTURE_TYPE_IMAGE_FORMAT_PROPERTIES_2,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_FORMAT_INFO_2_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_FORMAT_INFO_2,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_QUEUE_FAMILY_PROPERTIES_2_KHR =
VK_STRUCTURE_TYPE_QUEUE_FAMILY_PROPERTIES_2,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PROPERTIES_2_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PROPERTIES_2,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_SPARSE_IMAGE_FORMAT_PROPERTIES_2_KHR =
VK_STRUCTURE_TYPE_SPARSE_IMAGE_FORMAT_PROPERTIES_2,
  // Provided by VK_KHR_get_physical_device_properties2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SPARSE_IMAGE_FORMAT_INFO_2_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SPARSE_IMAGE_FORMAT_INFO_2,
  // Provided by VK_KHR_device_group
    VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_FLAGS_INFO_KHR =
VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_FLAGS_INFO,
  // Provided by VK_KHR_device_group
    VK_STRUCTURE_TYPE_DEVICE_GROUP_RENDER_PASS_BEGIN_INFO_KHR =
VK_STRUCTURE_TYPE_DEVICE_GROUP_RENDER_PASS_BEGIN_INFO,
  // Provided by VK_KHR_device_group
    VK_STRUCTURE_TYPE_DEVICE_GROUP_COMMAND_BUFFER_BEGIN_INFO_KHR =
VK_STRUCTURE_TYPE_DEVICE_GROUP_COMMAND_BUFFER_BEGIN_INFO,
  // Provided by VK_KHR_device_group
    VK_STRUCTURE_TYPE_DEVICE_GROUP_SUBMIT_INFO_KHR =
VK_STRUCTURE_TYPE_DEVICE_GROUP_SUBMIT_INFO,
  // Provided by VK_KHR_device_group
```

```
    VK_STRUCTURE_TYPE_DEVICE_GROUP_BIND_SPARSE_INFO_KHR =
VK_STRUCTURE_TYPE_DEVICE_GROUP_BIND_SPARSE_INFO,
  // Provided by VK_KHR_bind_memory2 with VK_KHR_device_group
    VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_DEVICE_GROUP_INFO_KHR =
VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_DEVICE_GROUP_INFO,
  // Provided by VK_KHR_bind_memory2 with VK_KHR_device_group
    VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_DEVICE_GROUP_INFO_KHR =
VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_DEVICE_GROUP_INFO,
  // Provided by VK_KHR_device_group_creation
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GROUP_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GROUP_PROPERTIES,
  // Provided by VK_KHR_device_group_creation
    VK_STRUCTURE_TYPE_DEVICE_GROUP_DEVICE_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_DEVICE_GROUP_DEVICE_CREATE_INFO,
  // Provided by VK_KHR_external_memory_capabilities
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_IMAGE_FORMAT_INFO_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_IMAGE_FORMAT_INFO,
  // Provided by VK_KHR_external_memory_capabilities
    VK_STRUCTURE_TYPE_EXTERNAL_IMAGE_FORMAT_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_EXTERNAL_IMAGE_FORMAT_PROPERTIES,
  // Provided by VK_KHR_external_memory_capabilities
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_BUFFER_INFO_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_BUFFER_INFO,
  // Provided by VK_KHR_external_memory_capabilities
    VK_STRUCTURE_TYPE_EXTERNAL_BUFFER_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_EXTERNAL_BUFFER_PROPERTIES,
  // Provided by VK_KHR_external_fence_capabilities,
VK_KHR_external_memory_capabilities, VK_KHR_external_semaphore_capabilities
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ID_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ID_PROPERTIES,
  // Provided by VK_KHR_external_memory
    VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_BUFFER_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_BUFFER_CREATE_INFO,
  // Provided by VK_KHR_external_memory
    VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_IMAGE_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_IMAGE_CREATE_INFO,
  // Provided by VK_KHR_external_memory
    VK_STRUCTURE_TYPE_EXPORT_MEMORY_ALLOCATE_INFO_KHR =
VK_STRUCTURE_TYPE_EXPORT_MEMORY_ALLOCATE_INFO,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_SEMAPHORE_INFO_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_SEMAPHORE_INFO,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_STRUCTURE_TYPE_EXTERNAL_SEMAPHORE_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_EXTERNAL_SEMAPHORE_PROPERTIES,
  // Provided by VK_KHR_external_semaphore
    VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_CREATE_INFO,
  // Provided by VK_KHR_shader_float16_int8
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT16_INT8_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT16_INT8_FEATURES,
```

```
  // Provided by VK_KHR_shader_float16_int8
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FLOAT16_INT8_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT16_INT8_FEATURES,
  // Provided by VK_KHR_16bit_storage
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_16BIT_STORAGE_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_16BIT_STORAGE_FEATURES,
  // Provided by VK_KHR_descriptor_update_template
    VK_STRUCTURE_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_CREATE_INFO,
  // Provided by VK_EXT_display_surface_counter
  // VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES2_EXT is a deprecated alias
    VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES2_EXT =
VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_2_EXT,
  // Provided by VK_KHR_imageless_framebuffer
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGELESS_FRAMEBUFFER_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGELESS_FRAMEBUFFER_FEATURES,
  // Provided by VK_KHR_imageless_framebuffer
    VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENTS_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENTS_CREATE_INFO,
  // Provided by VK_KHR_imageless_framebuffer
    VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENT_IMAGE_INFO_KHR =
VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENT_IMAGE_INFO,
  // Provided by VK_KHR_imageless_framebuffer
    VK_STRUCTURE_TYPE_RENDER_PASS_ATTACHMENT_BEGIN_INFO_KHR =
VK_STRUCTURE_TYPE_RENDER_PASS_ATTACHMENT_BEGIN_INFO,
  // Provided by VK_KHR_create_renderpass2
    VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_2_KHR =
VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_2,
  // Provided by VK_KHR_create_renderpass2
    VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_2_KHR =
VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_2,
  // Provided by VK_KHR_create_renderpass2
    VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_2_KHR =
VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_2,
  // Provided by VK_KHR_create_renderpass2
    VK_STRUCTURE_TYPE_SUBPASS_DEPENDENCY_2_KHR =
VK_STRUCTURE_TYPE_SUBPASS_DEPENDENCY_2,
  // Provided by VK_KHR_create_renderpass2
    VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO_2_KHR =
VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO_2,
  // Provided by VK_KHR_create_renderpass2
    VK_STRUCTURE_TYPE_SUBPASS_BEGIN_INFO_KHR = VK_STRUCTURE_TYPE_SUBPASS_BEGIN_INFO,
  // Provided by VK_KHR_create_renderpass2
    VK_STRUCTURE_TYPE_SUBPASS_END_INFO_KHR = VK_STRUCTURE_TYPE_SUBPASS_END_INFO,
  // Provided by VK_KHR_external_fence_capabilities
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_FENCE_INFO_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_FENCE_INFO,
  // Provided by VK_KHR_external_fence_capabilities
    VK_STRUCTURE_TYPE_EXTERNAL_FENCE_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_EXTERNAL_FENCE_PROPERTIES,
  // Provided by VK_KHR_external_fence
```

```
    VK_STRUCTURE_TYPE_EXPORT_FENCE_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_EXPORT_FENCE_CREATE_INFO,
  // Provided by VK_KHR_maintenance2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_POINT_CLIPPING_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_POINT_CLIPPING_PROPERTIES,
  // Provided by VK_KHR_maintenance2
    VK_STRUCTURE_TYPE_RENDER_PASS_INPUT_ATTACHMENT_ASPECT_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_RENDER_PASS_INPUT_ATTACHMENT_ASPECT_CREATE_INFO,
  // Provided by VK_KHR_maintenance2
    VK_STRUCTURE_TYPE_IMAGE_VIEW_USAGE_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_IMAGE_VIEW_USAGE_CREATE_INFO,
  // Provided by VK_KHR_maintenance2
    VK_STRUCTURE_TYPE_PIPELINE_TESSELLATION_DOMAIN_ORIGIN_STATE_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_PIPELINE_TESSELLATION_DOMAIN_ORIGIN_STATE_CREATE_INFO,
  // Provided by VK_KHR_variable_pointers
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VARIABLE_POINTERS_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VARIABLE_POINTERS_FEATURES,
  // Provided by VK_KHR_variable_pointers
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VARIABLE_POINTER_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VARIABLE_POINTERS_FEATURES_KHR,
  // Provided by VK_KHR_dedicated_allocation
    VK_STRUCTURE_TYPE_MEMORY_DEDICATED_REQUIREMENTS_KHR =
VK_STRUCTURE_TYPE_MEMORY_DEDICATED_REQUIREMENTS,
  // Provided by VK_KHR_dedicated_allocation
    VK_STRUCTURE_TYPE_MEMORY_DEDICATED_ALLOCATE_INFO_KHR =
VK_STRUCTURE_TYPE_MEMORY_DEDICATED_ALLOCATE_INFO,
  // Provided by VK_KHR_get_memory_requirements2
    VK_STRUCTURE_TYPE_BUFFER_MEMORY_REQUIREMENTS_INFO_2_KHR =
VK_STRUCTURE_TYPE_BUFFER_MEMORY_REQUIREMENTS_INFO_2,
  // Provided by VK_KHR_get_memory_requirements2
    VK_STRUCTURE_TYPE_IMAGE_MEMORY_REQUIREMENTS_INFO_2_KHR =
VK_STRUCTURE_TYPE_IMAGE_MEMORY_REQUIREMENTS_INFO_2,
  // Provided by VK_KHR_get_memory_requirements2
    VK_STRUCTURE_TYPE_IMAGE_SPARSE_MEMORY_REQUIREMENTS_INFO_2_KHR =
VK_STRUCTURE_TYPE_IMAGE_SPARSE_MEMORY_REQUIREMENTS_INFO_2,
  // Provided by VK_KHR_get_memory_requirements2
    VK_STRUCTURE_TYPE_MEMORY_REQUIREMENTS_2_KHR =
VK_STRUCTURE_TYPE_MEMORY_REQUIREMENTS_2,
  // Provided by VK_KHR_get_memory_requirements2
    VK_STRUCTURE_TYPE_SPARSE_IMAGE_MEMORY_REQUIREMENTS_2_KHR =
VK_STRUCTURE_TYPE_SPARSE_IMAGE_MEMORY_REQUIREMENTS_2,
  // Provided by VK_KHR_image_format_list
    VK_STRUCTURE_TYPE_IMAGE_FORMAT_LIST_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_IMAGE_FORMAT_LIST_CREATE_INFO,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_CREATE_INFO,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_INFO_KHR =
VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_INFO,
  // Provided by VK_KHR_sampler_ycbcr_conversion
```

```
    VK_STRUCTURE_TYPE_BIND_IMAGE_PLANE_MEMORY_INFO_KHR =
VK_STRUCTURE_TYPE_BIND_IMAGE_PLANE_MEMORY_INFO,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_STRUCTURE_TYPE_IMAGE_PLANE_MEMORY_REQUIREMENTS_INFO_KHR =
VK_STRUCTURE_TYPE_IMAGE_PLANE_MEMORY_REQUIREMENTS_INFO,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SAMPLER_YCBCR_CONVERSION_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SAMPLER_YCBCR_CONVERSION_FEATURES,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_IMAGE_FORMAT_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_IMAGE_FORMAT_PROPERTIES,
  // Provided by VK_KHR_bind_memory2
    VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_INFO_KHR =
VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_INFO,
  // Provided by VK_KHR_bind_memory2
    VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_INFO_KHR =
VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_INFO,
  // Provided by VK_KHR_maintenance3
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_3_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_3_PROPERTIES,
  // Provided by VK_KHR_maintenance3
    VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_SUPPORT_KHR =
VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_SUPPORT,
  // Provided by VK_KHR_shader_subgroup_extended_types
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_EXTENDED_TYPES_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_EXTENDED_TYPES_FEATURES,
  // Provided by VK_KHR_8bit_storage
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_8BIT_STORAGE_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_8BIT_STORAGE_FEATURES,
  // Provided by VK_KHR_shader_atomic_int64
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_INT64_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_INT64_FEATURES,
  // Provided by VK_KHR_driver_properties
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DRIVER_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DRIVER_PROPERTIES,
  // Provided by VK_KHR_shader_float_controls
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FLOAT_CONTROLS_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FLOAT_CONTROLS_PROPERTIES,
  // Provided by VK_KHR_depth_stencil_resolve
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_STENCIL_RESOLVE_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_STENCIL_RESOLVE_PROPERTIES,
  // Provided by VK_KHR_depth_stencil_resolve
    VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_DEPTH_STENCIL_RESOLVE_KHR =
VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_DEPTH_STENCIL_RESOLVE,
  // Provided by VK_KHR_timeline_semaphore
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_FEATURES,
  // Provided by VK_KHR_timeline_semaphore
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_PROPERTIES,
  // Provided by VK_KHR_timeline_semaphore
```

```
    VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO,
  // Provided by VK_KHR_timeline_semaphore
    VK_STRUCTURE_TYPE_TIMELINE_SEMAPHORE_SUBMIT_INFO_KHR =
VK_STRUCTURE_TYPE_TIMELINE_SEMAPHORE_SUBMIT_INFO,
  // Provided by VK_KHR_timeline_semaphore
    VK_STRUCTURE_TYPE_SEMAPHORE_WAIT_INFO_KHR = VK_STRUCTURE_TYPE_SEMAPHORE_WAIT_INFO,
  // Provided by VK_KHR_timeline_semaphore
    VK_STRUCTURE_TYPE_SEMAPHORE_SIGNAL_INFO_KHR =
VK_STRUCTURE_TYPE_SEMAPHORE_SIGNAL_INFO,
  // Provided by VK_KHR_vulkan_memory_model
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_MEMORY_MODEL_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_MEMORY_MODEL_FEATURES,
  // Provided by VK_KHR_shader_terminate_invocation
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TERMINATE_INVOCATION_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TERMINATE_INVOCATION_FEATURES,
  // Provided by VK_KHR_separate_depth_stencil_layouts
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SEPARATE_DEPTH_STENCIL_LAYOUTS_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SEPARATE_DEPTH_STENCIL_LAYOUTS_FEATURES,
  // Provided by VK_KHR_separate_depth_stencil_layouts
    VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_STENCIL_LAYOUT_KHR =
VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_STENCIL_LAYOUT,
  // Provided by VK_KHR_separate_depth_stencil_layouts
    VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_STENCIL_LAYOUT_KHR =
VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_STENCIL_LAYOUT,
  // Provided by VK_KHR_uniform_buffer_standard_layout
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_UNIFORM_BUFFER_STANDARD_LAYOUT_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_UNIFORM_BUFFER_STANDARD_LAYOUT_FEATURES,
  // Provided by VK_KHR_buffer_device_address
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BUFFER_DEVICE_ADDRESS_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BUFFER_DEVICE_ADDRESS_FEATURES,
  // Provided by VK_KHR_buffer_device_address
    VK_STRUCTURE_TYPE_BUFFER_DEVICE_ADDRESS_INFO_KHR =
VK_STRUCTURE_TYPE_BUFFER_DEVICE_ADDRESS_INFO,
  // Provided by VK_KHR_buffer_device_address
    VK_STRUCTURE_TYPE_BUFFER_OPAQUE_CAPTURE_ADDRESS_CREATE_INFO_KHR =
VK_STRUCTURE_TYPE_BUFFER_OPAQUE_CAPTURE_ADDRESS_CREATE_INFO,
  // Provided by VK_KHR_buffer_device_address
    VK_STRUCTURE_TYPE_MEMORY_OPAQUE_CAPTURE_ADDRESS_ALLOCATE_INFO_KHR =
VK_STRUCTURE_TYPE_MEMORY_OPAQUE_CAPTURE_ADDRESS_ALLOCATE_INFO,
  // Provided by VK_KHR_buffer_device_address
    VK_STRUCTURE_TYPE_DEVICE_MEMORY_OPAQUE_CAPTURE_ADDRESS_INFO_KHR =
VK_STRUCTURE_TYPE_DEVICE_MEMORY_OPAQUE_CAPTURE_ADDRESS_INFO,
  // Provided by VK_KHR_shader_integer_dot_product
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_FEATURES,
  // Provided by VK_KHR_shader_integer_dot_product
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_PROPERTIES,
  // Provided by VK_KHR_synchronization2
    VK_STRUCTURE_TYPE_MEMORY_BARRIER_2_KHR = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
```

```
  // Provided by VK_KHR_synchronization2
    VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2_KHR =
VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2,
  // Provided by VK_KHR_synchronization2
    VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2_KHR =
VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
  // Provided by VK_KHR_synchronization2
    VK_STRUCTURE_TYPE_DEPENDENCY_INFO_KHR = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
  // Provided by VK_KHR_synchronization2
    VK_STRUCTURE_TYPE_SUBMIT_INFO_2_KHR = VK_STRUCTURE_TYPE_SUBMIT_INFO_2,
  // Provided by VK_KHR_synchronization2
    VK_STRUCTURE_TYPE_SEMAPHORE_SUBMIT_INFO_KHR =
VK_STRUCTURE_TYPE_SEMAPHORE_SUBMIT_INFO,
  // Provided by VK_KHR_synchronization2
    VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO_KHR =
VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO,
  // Provided by VK_KHR_synchronization2
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SYNCHRONIZATION_2_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SYNCHRONIZATION_2_FEATURES,
  // Provided by VK_KHR_zero_initialize_workgroup_memory
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ZERO_INITIALIZE_WORKGROUP_MEMORY_FEATURES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ZERO_INITIALIZE_WORKGROUP_MEMORY_FEATURES,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_COPY_BUFFER_INFO_2_KHR = VK_STRUCTURE_TYPE_COPY_BUFFER_INFO_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_COPY_IMAGE_INFO_2_KHR = VK_STRUCTURE_TYPE_COPY_IMAGE_INFO_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_COPY_BUFFER_TO_IMAGE_INFO_2_KHR =
VK_STRUCTURE_TYPE_COPY_BUFFER_TO_IMAGE_INFO_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_COPY_IMAGE_TO_BUFFER_INFO_2_KHR =
VK_STRUCTURE_TYPE_COPY_IMAGE_TO_BUFFER_INFO_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_BLIT_IMAGE_INFO_2_KHR = VK_STRUCTURE_TYPE_BLIT_IMAGE_INFO_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_RESOLVE_IMAGE_INFO_2_KHR =
VK_STRUCTURE_TYPE_RESOLVE_IMAGE_INFO_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_BUFFER_COPY_2_KHR = VK_STRUCTURE_TYPE_BUFFER_COPY_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_IMAGE_COPY_2_KHR = VK_STRUCTURE_TYPE_IMAGE_COPY_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_IMAGE_BLIT_2_KHR = VK_STRUCTURE_TYPE_IMAGE_BLIT_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_BUFFER_IMAGE_COPY_2_KHR = VK_STRUCTURE_TYPE_BUFFER_IMAGE_COPY_2,
  // Provided by VK_KHR_copy_commands2
    VK_STRUCTURE_TYPE_IMAGE_RESOLVE_2_KHR = VK_STRUCTURE_TYPE_IMAGE_RESOLVE_2,
  // Provided by VK_KHR_format_feature_flags2
    VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_3_KHR = VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_3,
  // Provided by VK_KHR_maintenance4
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_FEATURES_KHR =
```

```
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_FEATURES,
  // Provided by VK_KHR_maintenance4
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_PROPERTIES_KHR =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_PROPERTIES,
  // Provided by VK_KHR_maintenance4
    VK_STRUCTURE_TYPE_DEVICE_BUFFER_MEMORY_REQUIREMENTS_KHR =
VK_STRUCTURE_TYPE_DEVICE_BUFFER_MEMORY_REQUIREMENTS,
  // Provided by VK_KHR_maintenance4
    VK_STRUCTURE_TYPE_DEVICE_IMAGE_MEMORY_REQUIREMENTS_KHR =
VK_STRUCTURE_TYPE_DEVICE_IMAGE_MEMORY_REQUIREMENTS,
  // Provided by VK_EXT_shader_object
    VK_STRUCTURE_TYPE_SHADER_REQUIRED_SUBGROUP_SIZE_CREATE_INFO_EXT =
VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_REQUIRED_SUBGROUP_SIZE_CREATE_INFO,
} VkStructureType;
```

3.13. API Name Aliases

A small number of APIs did not follow the naming conventions when initially defined. For consistency, when we discover an API name that violates the naming conventions, we rename it in the Specification, XML, and header files. For backwards compatibility, the original (incorrect) name is retained as a “typo alias”. The alias is deprecated and should not be used, but will be retained indefinitely.

NOTE

VK_STENCIL_FRONT_AND_BACK is an example of a *typo alias*. It was initially defined as part of VkStencilFaceFlagBits. Once the naming inconsistency was noticed, it was renamed to VK_STENCIL_FACE_FRONT_AND_BACK, and the old name was aliased to the correct name.

Chapter 4. Initialization

Before using Vulkan, an application **must** initialize it by loading the Vulkan commands, and creating a `VkInstance` object.

4.1. Command Function Pointers

Vulkan commands are not necessarily exposed by static linking on a platform. Commands to query function pointers for Vulkan commands are described below.

NOTE

When extensions are promoted or otherwise incorporated into another extension or Vulkan core version, command aliases may be included. Whilst the behavior of each command alias is identical, the behavior of retrieving each alias's function pointer is not. A function pointer for a given alias can only be retrieved if the extension or version that introduced that alias is supported and enabled, irrespective of whether any other alias is available.

Function pointers for all Vulkan commands **can** be obtained by calling:

```
// Provided by VK_VERSION_1_0
PFN_vkVoidFunction vkGetInstanceProcAddr(
    VkInstance                                  instance,
    const char*                                 pName);
```

- `instance` is the instance that the function pointer will be compatible with, or `NULL` for commands not dependent on any instance.
- `pName` is the name of the command to obtain.

`vkGetInstanceProcAddr` itself is obtained in a platform- and loader- specific manner. Typically, the loader library will export this command as a function symbol, so applications **can** link against the loader library, or load it dynamically and look up the symbol using platform-specific APIs.

The table below defines the various use cases for `vkGetInstanceProcAddr` and expected return value ("fp" is "function pointer") for each case. A valid returned function pointer ("fp") **must** not be `NULL`.

The returned function pointer is of type PFN_vkVoidFunction, and **must** be cast to the type of the command being queried before use.

Table 1. `vkGetInstanceProcAddr` *behavior*

`instance`	`pName`	**return value**
*[1]	`NULL`	undefined
invalid non-`NULL` instance	*[1]	undefined
`NULL`	*global command*[2]	fp
instance	vkGetInstanceProcAddr	fp

instance	pName	return value
instance	core *dispatchable command*	fp[3]
instance	enabled instance extension dispatchable command for instance	fp[3]
instance	available device extension[4] dispatchable command for instance	fp[3]
any other case, not covered above		NULL

1

"*" means any representable value for the parameter (including valid values, invalid values, and NULL).

2

The global commands are: vkEnumerateInstanceExtensionProperties, vkEnumerateInstanceLayerProperties, and vkCreateInstance. Dispatchable commands are all other commands which are not global.

3

The returned function pointer **must** only be called with a dispatchable object (the first parameter) that is instance or a child of instance, e.g. VkInstance, VkPhysicalDevice, VkDevice, VkQueue, or VkCommandBuffer.

4

An “available device extension” is a device extension supported by any physical device enumerated by instance.

Valid Usage (Implicit)

- VUID-vkGetInstanceProcAddr-instance-parameter
  If instance is not NULL, instance **must** be a valid VkInstance handle
- VUID-vkGetInstanceProcAddr-pName-parameter
  pName **must** be a null-terminated UTF-8 string

In order to support systems with multiple Vulkan implementations, the function pointers returned by vkGetInstanceProcAddr **may** point to dispatch code that calls a different real implementation for different VkDevice objects or their child objects. The overhead of the internal dispatch for VkDevice objects can be avoided by obtaining device-specific function pointers for any commands that use a device or device-child object as their dispatchable object. Such function pointers **can** be obtained by calling:

```
// Provided by VK_VERSION_1_0
PFN_vkVoidFunction vkGetDeviceProcAddr(
    VkDevice                                    device,
    const char*                                 pName);
```

The table below defines the various use cases for `vkGetDeviceProcAddr` and expected return value ("fp" is "function pointer") for each case. A valid returned function pointer ("fp") **must** not be `NULL`.

The returned function pointer is of type PFN_vkVoidFunction, and **must** be cast to the type of the command being queried before use. The function pointer **must** only be called with a dispatchable object (the first parameter) that is `device` or a child of `device`.

Table 2. `vkGetDeviceProcAddr` behavior

`device`	`pName`	**return value**
`NULL`	*[1]	undefined
invalid device	*[1]	undefined
device	`NULL`	undefined
device	requested core version[2] device-level dispatchable command[3]	fp[4]
device	enabled extension device-level dispatchable command[3]	fp[4]
any other case, not covered above		`NULL`

1

"*" means any representable value for the parameter (including valid values, invalid values, and `NULL`).

2

Device-level commands which are part of the core version specified by VkApplicationInfo ::`apiVersion` when creating the instance will always return a valid function pointer. If the `maintenance5` feature is enabled, core commands beyond that version which are supported by the implementation will return `NULL`, otherwise the implementation **may** either return `NULL` or a function pointer. If a function pointer is returned, it **must** not be called.

3

In this function, device-level excludes all physical-device-level commands.

4

The returned function pointer **must** only be called with a dispatchable object (the first parameter) that is `device` or a child of `device` e.g. VkDevice, VkQueue, or VkCommandBuffer.

Valid Usage (Implicit)

- VUID-vkGetDeviceProcAddr-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceProcAddr-pName-parameter
  `pName` **must** be a null-terminated UTF-8 string

The definition of PFN_vkVoidFunction is:

```
// Provided by VK_VERSION_1_0
typedef void (VKAPI_PTR *PFN_vkVoidFunction)(void);
```

This type is returned from command function pointer queries, and **must** be cast to an actual command function pointer before use.

4.1.1. Extending Physical Device From Device Extensions

When the `VK_KHR_get_physical_device_properties2` extension is enabled, physical-device-level functionality of a device extension **can** be used with a physical device if the corresponding extension is enumerated by vkEnumerateDeviceExtensionProperties for that physical device, even before a logical device has been created.

To obtain a function pointer for a physical-device-level command from a device extension, an application **can** use vkGetInstanceProcAddr. This function pointer **may** point to dispatch code, which calls a different real implementation for different `VkPhysicalDevice` objects. Applications **must** not use a VkPhysicalDevice in any command added by an extension or core version that is not supported by that physical device.

Device extensions **may** define structures that **can** be added to the `pNext` chain of physical-device-level commands.

4.2. Instances

There is no global state in Vulkan and all per-application state is stored in a `VkInstance` object. Creating a `VkInstance` object initializes the Vulkan library and allows the application to pass information about itself to the implementation.

Instances are represented by `VkInstance` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_HANDLE(VkInstance)
```

To create an instance object, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateInstance(
    const VkInstanceCreateInfo*                 pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkInstance*                                 pInstance);
```

- `pCreateInfo` is a pointer to a VkInstanceCreateInfo structure controlling creation of the instance.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pInstance` points a VkInstance handle in which the resulting instance is returned.

`vkCreateInstance` verifies that the requested layers exist. If not, `vkCreateInstance` will return `VK_ERROR_LAYER_NOT_PRESENT`. Next `vkCreateInstance` verifies that the requested extensions are supported (e.g. in the implementation or in any enabled instance layer) and if any requested extension is not supported, `vkCreateInstance` **must** return `VK_ERROR_EXTENSION_NOT_PRESENT`. After verifying and enabling the instance layers and extensions the `VkInstance` object is created and returned to the application. If a requested extension is only supported by a layer, both the layer and the extension need to be specified at `vkCreateInstance` time for the creation to succeed.

Valid Usage

- VUID-vkCreateInstance-ppEnabledExtensionNames-01388
  All required extensions for each extension in the VkInstanceCreateInfo ::`ppEnabledExtensionNames` list **must** also be present in that list

Valid Usage (Implicit)

- VUID-vkCreateInstance-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkInstanceCreateInfo structure
- VUID-vkCreateInstance-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateInstance-pInstance-parameter
  `pInstance` **must** be a valid pointer to a VkInstance handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

- VK_ERROR_INITIALIZATION_FAILED
- VK_ERROR_LAYER_NOT_PRESENT
- VK_ERROR_EXTENSION_NOT_PRESENT
- VK_ERROR_INCOMPATIBLE_DRIVER

The VkInstanceCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkInstanceCreateInfo {
    VkStructureType             sType;
    const void*                 pNext;
    VkInstanceCreateFlags       flags;
    const VkApplicationInfo*    pApplicationInfo;
    uint32_t                    enabledLayerCount;
    const char* const*          ppEnabledLayerNames;
    uint32_t                    enabledExtensionCount;
    const char* const*          ppEnabledExtensionNames;
} VkInstanceCreateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkInstanceCreateFlagBits indicating the behavior of the instance.
- pApplicationInfo is NULL or a pointer to a VkApplicationInfo structure. If not NULL, this information helps implementations recognize behavior inherent to classes of applications. VkApplicationInfo is defined in detail below.
- enabledLayerCount is the number of global layers to enable.
- ppEnabledLayerNames is a pointer to an array of enabledLayerCount null-terminated UTF-8 strings containing the names of layers to enable for the created instance. The layers are loaded in the order they are listed in this array, with the first array element being the closest to the application, and the last array element being the closest to the driver. See the Layers section for further details.
- enabledExtensionCount is the number of global extensions to enable.
- ppEnabledExtensionNames is a pointer to an array of enabledExtensionCount null-terminated UTF-8 strings containing the names of extensions to enable.

To capture events that occur while creating or destroying an instance, an application **can** link a VkDebugUtilsMessengerCreateInfoEXT structure to the pNext element of the VkInstanceCreateInfo structure given to vkCreateInstance. This callback is only valid for the duration of the vkCreateInstance and the vkDestroyInstance call. Use vkCreateDebugUtilsMessengerEXT to create persistent callback objects.

Valid Usage

- VUID-VkInstanceCreateInfo-pNext-04926
  If the `pNext` chain of `VkInstanceCreateInfo` includes a VkDebugUtilsMessengerCreateInfoEXT structure, the list of enabled extensions in `ppEnabledExtensionNames` **must** contain `VK_EXT_debug_utils`
- VUID-VkInstanceCreateInfo-pNext-06779
  If the `pNext` chain includes a VkExportMetalObjectCreateInfoEXT structure, its `exportObjectType` member **must** be either `VK_EXPORT_METAL_OBJECT_TYPE_METAL_DEVICE_BIT_EXT` or `VK_EXPORT_METAL_OBJECT_TYPE_METAL_COMMAND_QUEUE_BIT_EXT`
- VUID-VkInstanceCreateInfo-flags-06559
  If `flags` has the `VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR` bit set, the list of enabled extensions in `ppEnabledExtensionNames` **must** contain `VK_KHR_portability_enumeration`
- VUID-VkInstanceCreateInfo-pNext-10242
  If the `pNext` chain of `VkInstanceCreateInfo` includes a VkLayerSettingsCreateInfoEXT structure, the list of enabled extensions in `ppEnabledExtensionNames` **must** contain VK_EXT_layer_settings

Valid Usage (Implicit)

- VUID-VkInstanceCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO`
- VUID-VkInstanceCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkDebugUtilsMessengerCreateInfoEXT, VkExportMetalObjectCreateInfoEXT, or VkLayerSettingsCreateInfoEXT
- VUID-VkInstanceCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique, with the exception of structures of type VkDebugUtilsMessengerCreateInfoEXT, VkExportMetalObjectCreateInfoEXT, or VkLayerSettingsCreateInfoEXT
- VUID-VkInstanceCreateInfo-flags-parameter
  `flags` **must** be a valid combination of VkInstanceCreateFlagBits values
- VUID-VkInstanceCreateInfo-pApplicationInfo-parameter
  If `pApplicationInfo` is not `NULL`, `pApplicationInfo` **must** be a valid pointer to a valid VkApplicationInfo structure
- VUID-VkInstanceCreateInfo-ppEnabledLayerNames-parameter
  If `enabledLayerCount` is not `0`, `ppEnabledLayerNames` **must** be a valid pointer to an array of `enabledLayerCount` null-terminated UTF-8 strings
- VUID-VkInstanceCreateInfo-ppEnabledExtensionNames-parameter
  If `enabledExtensionCount` is not `0`, `ppEnabledExtensionNames` **must** be a valid pointer to an

array of `enabledExtensionCount` null-terminated UTF-8 strings

```
// Provided by VK_VERSION_1_0
typedef enum VkInstanceCreateFlagBits {
  // Provided by VK_KHR_portability_enumeration
    VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR = 0x00000001,
} VkInstanceCreateFlagBits;
```

- `VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR` specifies that the instance will enumerate available Vulkan Portability-compliant physical devices and groups in addition to the Vulkan physical devices and groups that are enumerated by default.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkInstanceCreateFlags;
```

`VkInstanceCreateFlags` is a bitmask type for setting a mask of zero or more VkInstanceCreateFlagBits.

To create a Vulkan instance with a specific configuration of layer settings, add VkLayerSettingsCreateInfoEXT structures to the `pNext` chain of the VkInstanceCreateInfo structure, specifying the settings to be configured.

```
// Provided by VK_EXT_layer_settings
typedef struct VkLayerSettingsCreateInfoEXT {
    VkStructureType             sType;
    const void*                 pNext;
    uint32_t                    settingCount;
    const VkLayerSettingEXT*    pSettings;
} VkLayerSettingsCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `settingCount` is the number of settings to configure.
- `pSettings` is a pointer to an array of `settingCount` VkLayerSettingEXT values specifying the settings to be configured.

Valid Usage (Implicit)

- VUID-VkLayerSettingsCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_LAYER_SETTINGS_CREATE_INFO_EXT`
- VUID-VkLayerSettingsCreateInfoEXT-pSettings-parameter
  If `settingCount` is not `0`, `pSettings` **must** be a valid pointer to an array of `settingCount` valid VkLayerSettingEXT structures

The values of elements of the VkLayerSettingsCreateInfoEXT::pSettings array, specifying layer settings to be configured, are:

```
// Provided by VK_EXT_layer_settings
typedef struct VkLayerSettingEXT {
    const char*              pLayerName;
    const char*              pSettingName;
    VkLayerSettingTypeEXT    type;
    uint32_t                 valueCount;
    const void*              pValues;
} VkLayerSettingEXT;
```

- pLayerName is a pointer to a null-terminated UTF-8 string naming the layer to configure the setting from.
- pSettingName is a pointer to a null-terminated UTF-8 string naming the setting to configure. Values of pSettingName that are unknown to the layer are ignored.
- type is a VkLayerSettingTypeEXT value specifying the type of the pValues values.
- valueCount is the number of values used to configure the layer setting.
- pValues is a pointer to an array of valueCount values of the type indicated by type to configure the layer setting.

When multiple VkLayerSettingsCreateInfoEXT structures are chained and the same pSettingName is referenced for the same pLayerName, the value of the first reference of the layer setting is used.

Valid Usage

- VUID-VkLayerSettingEXT-valueCount-10070
  If valueCount is not 0, pValues **must** be a valid pointer to an array of valueCount values of the type indicated by type

Valid Usage (Implicit)

- VUID-VkLayerSettingEXT-pLayerName-parameter
  pLayerName **must** be a null-terminated UTF-8 string
- VUID-VkLayerSettingEXT-pSettingName-parameter
  pSettingName **must** be a null-terminated UTF-8 string
- VUID-VkLayerSettingEXT-type-parameter
  type **must** be a valid VkLayerSettingTypeEXT value

Possible values of VkLayerSettingEXT::type, specifying the type of the data returned in VkLayerSettingEXT::pValues, are:

```
// Provided by VK_EXT_layer_settings
typedef enum VkLayerSettingTypeEXT {
    VK_LAYER_SETTING_TYPE_BOOL32_EXT = 0,
    VK_LAYER_SETTING_TYPE_INT32_EXT = 1,
    VK_LAYER_SETTING_TYPE_INT64_EXT = 2,
    VK_LAYER_SETTING_TYPE_UINT32_EXT = 3,
    VK_LAYER_SETTING_TYPE_UINT64_EXT = 4,
    VK_LAYER_SETTING_TYPE_FLOAT32_EXT = 5,
    VK_LAYER_SETTING_TYPE_FLOAT64_EXT = 6,
    VK_LAYER_SETTING_TYPE_STRING_EXT = 7,
} VkLayerSettingTypeEXT;
```

- `VK_LAYER_SETTING_TYPE_BOOL32_EXT` specifies that the layer setting's type is VkBool32.
- `VK_LAYER_SETTING_TYPE_INT32_EXT` specifies that the layer setting's type is signed 32-bit integer.
- `VK_LAYER_SETTING_TYPE_INT64_EXT` specifies that the layer setting's type is signed 64-bit integer.
- `VK_LAYER_SETTING_TYPE_UINT32_EXT` specifies that the layer setting's type is unsigned 32-bit integer.
- `VK_LAYER_SETTING_TYPE_UINT64_EXT` specifies that the layer setting's type is unsigned 64-bit integer.
- `VK_LAYER_SETTING_TYPE_FLOAT32_EXT` specifies that the layer setting's type is 32-bit floating-point.
- `VK_LAYER_SETTING_TYPE_FLOAT64_EXT` specifies that the layer setting's type is 64-bit floating-point.
- `VK_LAYER_SETTING_TYPE_STRING_EXT` specifies that the layer setting's type is a pointer to a null-terminated UTF-8 string.

The `VkApplicationInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkApplicationInfo {
    VkStructureType    sType;
    const void*        pNext;
    const char*        pApplicationName;
    uint32_t           applicationVersion;
    const char*        pEngineName;
    uint32_t           engineVersion;
    uint32_t           apiVersion;
} VkApplicationInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pApplicationName` is `NULL` or is a pointer to a null-terminated UTF-8 string containing the name of the application.
- `applicationVersion` is an unsigned integer variable containing the developer-supplied version number of the application.

- `pEngineName` is `NULL` or is a pointer to a null-terminated UTF-8 string containing the name of the engine (if any) used to create the application.
- `engineVersion` is an unsigned integer variable containing the developer-supplied version number of the engine used to create the application.
- `apiVersion` is the version of the Vulkan API against which the application expects to run, encoded as described in Version Numbers. If `apiVersion` is 0 the implementation **must** ignore it, otherwise if the implementation does not support the requested `apiVersion`, or an effective substitute for `apiVersion`, it **must** return `VK_ERROR_INCOMPATIBLE_DRIVER`. The patch version number specified in `apiVersion` is ignored when creating an instance object. The variant version of the instance **must** match that requested in `apiVersion`.

Valid Usage

- VUID-VkApplicationInfo-apiVersion-04010
  If `apiVersion` is not `0`, then it **must** be greater than or equal to VK_API_VERSION_1_0

Valid Usage (Implicit)

- VUID-VkApplicationInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_APPLICATION_INFO`
- VUID-VkApplicationInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkApplicationInfo-pApplicationName-parameter
  If `pApplicationName` is not `NULL`, `pApplicationName` **must** be a null-terminated UTF-8 string
- VUID-VkApplicationInfo-pEngineName-parameter
  If `pEngineName` is not `NULL`, `pEngineName` **must** be a null-terminated UTF-8 string

To destroy an instance, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyInstance(
    VkInstance                                  instance,
    const VkAllocationCallbacks*                pAllocator);
```

- `instance` is the handle of the instance to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyInstance-instance-00629
  All child objects created using `instance` **must** have been destroyed prior to destroying `instance`

- VUID-vkDestroyInstance-instance-00630
  If `VkAllocationCallbacks` were provided when `instance` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyInstance-instance-00631
  If no `VkAllocationCallbacks` were provided when `instance` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyInstance-instance-parameter
  If `instance` is not `NULL`, `instance` **must** be a valid VkInstance handle
- VUID-vkDestroyInstance-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure

Host Synchronization

- Host access to `instance` **must** be externally synchronized
- Host access to all `VkPhysicalDevice` objects enumerated from `instance` **must** be externally synchronized

Chapter 5. Devices and Queues

Once Vulkan is initialized, devices and queues are the primary objects used to interact with a Vulkan implementation.

Vulkan separates the concept of *physical* and *logical* devices. A physical device usually represents a single complete implementation of Vulkan (excluding instance-level functionality) available to the host, of which there are a finite number. A logical device represents an instance of that implementation with its own state and resources independent of other logical devices.

Physical devices are represented by `VkPhysicalDevice` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_HANDLE(VkPhysicalDevice)
```

5.1. Physical Devices

To retrieve a list of physical device objects representing the physical devices installed in the system, call:

```
// Provided by VK_VERSION_1_0
VkResult vkEnumeratePhysicalDevices(
    VkInstance                                  instance,
    uint32_t*                                   pPhysicalDeviceCount,
    VkPhysicalDevice*                           pPhysicalDevices);
```

- `instance` is a handle to a Vulkan instance previously created with vkCreateInstance.
- `pPhysicalDeviceCount` is a pointer to an integer related to the number of physical devices available or queried, as described below.
- `pPhysicalDevices` is either `NULL` or a pointer to an array of `VkPhysicalDevice` handles.

If `pPhysicalDevices` is `NULL`, then the number of physical devices available is returned in `pPhysicalDeviceCount`. Otherwise, `pPhysicalDeviceCount` **must** point to a variable set by the application to the number of elements in the `pPhysicalDevices` array, and on return the variable is overwritten with the number of handles actually written to `pPhysicalDevices`. If `pPhysicalDeviceCount` is less than the number of physical devices available, at most `pPhysicalDeviceCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available physical devices were returned.

Valid Usage (Implicit)

- VUID-vkEnumeratePhysicalDevices-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkEnumeratePhysicalDevices-pPhysicalDeviceCount-parameter

pPhysicalDeviceCount **must** be a valid pointer to a uint32_t value

- VUID-vkEnumeratePhysicalDevices-pPhysicalDevices-parameter
  If the value referenced by pPhysicalDeviceCount is not 0, and pPhysicalDevices is not NULL, pPhysicalDevices **must** be a valid pointer to an array of pPhysicalDeviceCount VkPhysicalDevice handles

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INITIALIZATION_FAILED

To query general properties of physical devices once enumerated, call:

```
// Provided by VK_VERSION_1_0
void vkGetPhysicalDeviceProperties(
    VkPhysicalDevice                            physicalDevice,
    VkPhysicalDeviceProperties*                 pProperties);
```

- physicalDevice is the handle to the physical device whose properties will be queried.
- pProperties is a pointer to a VkPhysicalDeviceProperties structure in which properties are returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceProperties-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceProperties-pProperties-parameter
  pProperties **must** be a valid pointer to a VkPhysicalDeviceProperties structure

The VkPhysicalDeviceProperties structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPhysicalDeviceProperties {
    uint32_t                            apiVersion;
    uint32_t                            driverVersion;
    uint32_t                            vendorID;
    uint32_t                            deviceID;
    VkPhysicalDeviceType                deviceType;
    char                                deviceName[VK_MAX_PHYSICAL_DEVICE_NAME_SIZE];
    uint8_t                             pipelineCacheUUID[VK_UUID_SIZE];
    VkPhysicalDeviceLimits              limits;
    VkPhysicalDeviceSparseProperties    sparseProperties;
} VkPhysicalDeviceProperties;
```

- `apiVersion` is the version of Vulkan supported by the device, encoded as described in Version Numbers.
- `driverVersion` is the vendor-specified version of the driver.
- `vendorID` is a unique identifier for the *vendor* (see below) of the physical device.
- `deviceID` is a unique identifier for the physical device among devices available from the vendor.
- `deviceType` is a VkPhysicalDeviceType specifying the type of device.
- `deviceName` is an array of `VK_MAX_PHYSICAL_DEVICE_NAME_SIZE` `char` containing a null-terminated UTF-8 string which is the name of the device.
- `pipelineCacheUUID` is an array of `VK_UUID_SIZE` `uint8_t` values representing a universally unique identifier for the device.
- `limits` is the VkPhysicalDeviceLimits structure specifying device-specific limits of the physical device. See Limits for details.
- `sparseProperties` is the VkPhysicalDeviceSparseProperties structure specifying various sparse related properties of the physical device. See Sparse Properties for details.

NOTE The encoding of `driverVersion` is implementation-defined. It **may** not use the same encoding as `apiVersion`. Applications should follow information from the *vendor* on how to extract the version information from `driverVersion`.

The `vendorID` and `deviceID` fields are provided to allow applications to adapt to device characteristics that are not adequately exposed by other Vulkan queries.

NOTE These **may** include performance profiles, hardware errata, or other characteristics.

The *vendor* identified by `vendorID` is the entity responsible for the most salient characteristics of the underlying implementation of the VkPhysicalDevice being queried.

NOTE For example, in the case of a discrete GPU implementation, this **should** be the GPU chipset vendor. In the case of a hardware accelerator integrated into a system-on-chip (SoC), this **should** be the supplier of the silicon IP used to create the accelerator.

If the vendor has a PCI vendor ID, the low 16 bits of `vendorID` **must** contain that PCI vendor ID, and the remaining bits **must** be zero. Otherwise, the value returned **must** be a valid Khronos vendor ID, obtained as described in the Vulkan Documentation and Extensions: Procedures and Conventions document in the section "Registering a Vendor ID with Khronos". Khronos vendor IDs are allocated starting at 0x10000, to distinguish them from the PCI vendor ID namespace. Khronos vendor IDs are symbolically defined in the VkVendorId type.

The vendor is also responsible for the value returned in `deviceID`. If the implementation is driven primarily by a PCI device with a PCI device ID, the low 16 bits of `deviceID` **must** contain that PCI device ID, and the remaining bits **must** be zero. Otherwise, the choice of what values to return **may** be dictated by operating system or platform policies - but **should** uniquely identify both the device version and any major configuration options (for example, core count in the case of multicore devices).

NOTE

The same device ID **should** be used for all physical implementations of that device version and configuration. For example, all uses of a specific silicon IP GPU version and configuration **should** use the same device ID, even if those uses occur in different SoCs.

Khronos vendor IDs which **may** be returned in VkPhysicalDeviceProperties::`vendorID` are:

```
// Provided by VK_VERSION_1_0
typedef enum VkVendorId {
    VK_VENDOR_ID_KHRONOS = 0x10000,
    VK_VENDOR_ID_VIV = 0x10001,
    VK_VENDOR_ID_VSI = 0x10002,
    VK_VENDOR_ID_KAZAN = 0x10003,
    VK_VENDOR_ID_CODEPLAY = 0x10004,
    VK_VENDOR_ID_MESA = 0x10005,
    VK_VENDOR_ID_POCL = 0x10006,
    VK_VENDOR_ID_MOBILEYE = 0x10007,
} VkVendorId;
```

NOTE

Khronos vendor IDs may be allocated by vendors at any time. Only the latest canonical versions of this Specification, of the corresponding `vk.xml` API Registry, and of the corresponding `vulkan_core.h` header file **must** contain all reserved Khronos vendor IDs.

Only Khronos vendor IDs are given symbolic names at present. PCI vendor IDs returned by the implementation can be looked up in the PCI-SIG database.

`VK_MAX_PHYSICAL_DEVICE_NAME_SIZE` is the length in `char` values of an array containing a physical device name string, as returned in VkPhysicalDeviceProperties::`deviceName`.

```
#define VK_MAX_PHYSICAL_DEVICE_NAME_SIZE  256U
```

The physical device types which **may** be returned in VkPhysicalDeviceProperties::`deviceType` are:

```
// Provided by VK_VERSION_1_0
typedef enum VkPhysicalDeviceType {
    VK_PHYSICAL_DEVICE_TYPE_OTHER = 0,
    VK_PHYSICAL_DEVICE_TYPE_INTEGRATED_GPU = 1,
    VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU = 2,
    VK_PHYSICAL_DEVICE_TYPE_VIRTUAL_GPU = 3,
    VK_PHYSICAL_DEVICE_TYPE_CPU = 4,
} VkPhysicalDeviceType;
```

- `VK_PHYSICAL_DEVICE_TYPE_OTHER` - the device does not match any other available types.
- `VK_PHYSICAL_DEVICE_TYPE_INTEGRATED_GPU` - the device is typically one embedded in or tightly coupled with the host.
- `VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU` - the device is typically a separate processor connected to the host via an interlink.
- `VK_PHYSICAL_DEVICE_TYPE_VIRTUAL_GPU` - the device is typically a virtual node in a virtualization environment.
- `VK_PHYSICAL_DEVICE_TYPE_CPU` - the device is typically running on the same processors as the host.

The physical device type is advertised for informational purposes only, and does not directly affect the operation of the system. However, the device type **may** correlate with other advertised properties or capabilities of the system, such as how many memory heaps there are.

To query general properties of physical devices once enumerated, call:

```
// Provided by VK_KHR_get_physical_device_properties2
void vkGetPhysicalDeviceProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    VkPhysicalDeviceProperties2*                pProperties);
```

- `physicalDevice` is the handle to the physical device whose properties will be queried.
- `pProperties` is a pointer to a `VkPhysicalDeviceProperties2` structure in which properties are returned.

Each structure in `pProperties` and its `pNext` chain contains members corresponding to implementation-dependent properties, behaviors, or limits. `vkGetPhysicalDeviceProperties2KHR` fills in each member to specify the corresponding value for the implementation.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceProperties2-physicalDevice-parameter
  `physicalDevice` **must** be a valid `VkPhysicalDevice` handle
- VUID-vkGetPhysicalDeviceProperties2-pProperties-parameter
  `pProperties` **must** be a valid pointer to a `VkPhysicalDeviceProperties2` structure

The VkPhysicalDeviceProperties2 structure is defined as:

```
typedef struct VkPhysicalDeviceProperties2 {
    VkStructureType                 sType;
    void*                           pNext;
    VkPhysicalDeviceProperties      properties;
} VkPhysicalDeviceProperties2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkPhysicalDeviceProperties2 VkPhysicalDeviceProperties2KHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- properties is a VkPhysicalDeviceProperties structure describing properties of the physical device. This structure is written with the same values as if it were written by vkGetPhysicalDeviceProperties.

The pNext chain of this structure is used to extend the structure with properties defined by extensions.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceProperties2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROPERTIES_2
- VUID-VkPhysicalDeviceProperties2-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkPhysicalDeviceAccelerationStructurePropertiesKHR, VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT, VkPhysicalDeviceComputeShaderDerivativesPropertiesKHR, VkPhysicalDeviceConservativeRasterizationPropertiesEXT, VkPhysicalDeviceCooperativeMatrixPropertiesKHR, VkPhysicalDeviceCustomBorderColorPropertiesEXT, VkPhysicalDeviceDepthStencilResolveProperties, VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT, VkPhysicalDeviceDiscardRectanglePropertiesEXT, VkPhysicalDeviceDriverProperties, VkPhysicalDeviceDrmPropertiesEXT, VkPhysicalDeviceExtendedDynamicState3PropertiesEXT, VkPhysicalDeviceExternalMemoryHostPropertiesEXT, VkPhysicalDeviceFloatControlsProperties, VkPhysicalDeviceFragmentDensityMap2PropertiesEXT, VkPhysicalDeviceFragmentDensityMapPropertiesEXT, VkPhysicalDeviceFragmentShaderBarycentricPropertiesKHR,

VkPhysicalDeviceFragmentShadingRatePropertiesKHR,
VkPhysicalDeviceHostImageCopyPropertiesEXT, VkPhysicalDeviceIDProperties,
VkPhysicalDeviceLayeredApiPropertiesListKHR,
VkPhysicalDeviceLegacyVertexAttributesPropertiesEXT,
VkPhysicalDeviceLineRasterizationPropertiesKHR,
VkPhysicalDeviceMaintenance3Properties, VkPhysicalDeviceMaintenance4Properties,
VkPhysicalDeviceMaintenance5PropertiesKHR,
VkPhysicalDeviceMaintenance6PropertiesKHR,
VkPhysicalDeviceMaintenance7PropertiesKHR,
VkPhysicalDeviceMapMemoryPlacedPropertiesEXT,
VkPhysicalDeviceMultiDrawPropertiesEXT, VkPhysicalDeviceMultiviewProperties,
VkPhysicalDeviceNestedCommandBufferPropertiesEXT,
VkPhysicalDeviceOpacityMicromapPropertiesEXT,
VkPhysicalDevicePCIBusInfoPropertiesEXT,
VkPhysicalDevicePerformanceQueryPropertiesKHR,
VkPhysicalDevicePipelineBinaryPropertiesKHR,
VkPhysicalDevicePointClippingProperties,
VkPhysicalDevicePortabilitySubsetPropertiesKHR,
VkPhysicalDeviceProvokingVertexPropertiesEXT,
VkPhysicalDevicePushDescriptorPropertiesKHR,
VkPhysicalDeviceRayTracingPipelinePropertiesKHR,
VkPhysicalDeviceRobustness2PropertiesEXT,
VkPhysicalDeviceSampleLocationsPropertiesEXT,
VkPhysicalDeviceShaderIntegerDotProductProperties,
VkPhysicalDeviceShaderModuleIdentifierPropertiesEXT,
VkPhysicalDeviceShaderObjectPropertiesEXT,
VkPhysicalDeviceShaderTileImagePropertiesEXT,
VkPhysicalDeviceTimelineSemaphoreProperties,
VkPhysicalDeviceTransformFeedbackPropertiesEXT, or
VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR

- VUID-VkPhysicalDeviceProperties2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

The `VkPhysicalDeviceIDProperties` structure is defined as:

```
typedef struct VkPhysicalDeviceIDProperties {
    VkStructureType    sType;
    void*              pNext;
    uint8_t            deviceUUID[VK_UUID_SIZE];
    uint8_t            driverUUID[VK_UUID_SIZE];
    uint8_t            deviceLUID[VK_LUID_SIZE];
    uint32_t           deviceNodeMask;
    VkBool32           deviceLUIDValid;
} VkPhysicalDeviceIDProperties;
```

or the equivalent

```
// Provided by VK_KHR_external_fence_capabilities,
VK_KHR_external_memory_capabilities, VK_KHR_external_semaphore_capabilities
typedef VkPhysicalDeviceIDProperties VkPhysicalDeviceIDPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `deviceUUID` is an array of `VK_UUID_SIZE` `uint8_t` values representing a universally unique identifier for the device.
- `driverUUID` is an array of `VK_UUID_SIZE` `uint8_t` values representing a universally unique identifier for the driver build in use by the device.
- `deviceLUID` is an array of `VK_LUID_SIZE` `uint8_t` values representing a locally unique identifier for the device.
- `deviceNodeMask` is a `uint32_t` bitfield identifying the node within a linked device adapter corresponding to the device.
- `deviceLUIDValid` is a boolean value that will be `VK_TRUE` if `deviceLUID` contains a valid LUID and `deviceNodeMask` contains a valid node mask, and `VK_FALSE` if they do not.

If the `VkPhysicalDeviceIDProperties` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

`deviceUUID` **must** be immutable for a given device across instances, processes, driver APIs, driver versions, and system reboots.

Applications **can** compare the `driverUUID` value across instance and process boundaries, and **can** make similar queries in external APIs to determine whether they are capable of sharing memory objects and resources using them with the device.

`deviceUUID` and/or `driverUUID` **must** be used to determine whether a particular external object can be shared between driver components, where such a restriction exists as defined in the compatibility table for the particular object type:

- External memory handle types compatibility
- External semaphore handle types compatibility
- External fence handle types compatibility

If `deviceLUIDValid` is `VK_FALSE`, the values of `deviceLUID` and `deviceNodeMask` are undefined. If `deviceLUIDValid` is `VK_TRUE` and Vulkan is running on the Windows operating system, the contents of `deviceLUID` **can** be cast to an `LUID` object and **must** be equal to the locally unique identifier of a `IDXGIAdapter1` object that corresponds to `physicalDevice`. If `deviceLUIDValid` is `VK_TRUE`, `deviceNodeMask` **must** contain exactly one bit. If Vulkan is running on an operating system that supports the Direct3D 12 API and `physicalDevice` corresponds to an individual device in a linked device adapter, `deviceNodeMask` identifies the Direct3D 12 node corresponding to `physicalDevice`. Otherwise, `deviceNodeMask` **must** be `1`.

Although they have identical descriptions, VkPhysicalDeviceIDProperties::deviceUUID may differ from VkPhysicalDeviceProperties2::pipelineCacheUUID. The former is intended to identify and correlate devices across API and driver boundaries, while the latter is used to identify a compatible device and driver combination to use when serializing and de-serializing pipeline state.

Implementations **should** return deviceUUID values which are likely to be unique even in the presence of multiple Vulkan implementations (such as a GPU driver and a software renderer; two drivers for different GPUs; or the same Vulkan driver running on two logically different devices).

NOTE

Khronos' conformance testing is unable to guarantee that deviceUUID values are actually unique, so implementors **should** make their own best efforts to ensure this. In particular, hard-coded deviceUUID values, especially all-0 bits, **should** never be used.

A combination of values unique to the vendor, the driver, and the hardware environment can be used to provide a deviceUUID which is unique to a high degree of certainty. Some possible inputs to such a computation are:

- Information reported by vkGetPhysicalDeviceProperties
- PCI device ID (if defined)
- PCI bus ID, or similar system configuration information.
- Driver binary checksums.

NOTE

While VkPhysicalDeviceIDProperties::deviceUUID is specified to remain consistent across driver versions and system reboots, it is not intended to be usable as a serializable persistent identifier for a device. It may change when a device is physically added to, removed from, or moved to a different connector in a system while that system is powered down. Further, there is no reasonable way to verify with conformance testing that a given device retains the same UUID in a given system across all driver versions supported in that system. While implementations should make every effort to report consistent device UUIDs across driver versions, applications should avoid relying on the persistence of this value for uses other than identifying compatible devices for external object sharing purposes.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceIDProperties-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ID_PROPERTIES

VK_UUID_SIZE is the length in uint8_t values of an array containing a universally unique device or driver build identifier, as returned in VkPhysicalDeviceIDProperties::deviceUUID and VkPhysicalDeviceIDProperties::driverUUID.

```
#define VK_UUID_SIZE                      16U
```

VK_LUID_SIZE is the length in uint8_t values of an array containing a locally unique device identifier, as returned in VkPhysicalDeviceIDProperties::deviceLUID.

```
#define VK_LUID_SIZE                      8U
```

or the equivalent

```
#define VK_LUID_SIZE_KHR                  VK_LUID_SIZE
```

The VkPhysicalDeviceDriverProperties structure is defined as:

```
typedef struct VkPhysicalDeviceDriverProperties {
    VkStructureType         sType;
    void*                   pNext;
    VkDriverId              driverID;
    char                    driverName[VK_MAX_DRIVER_NAME_SIZE];
    char                    driverInfo[VK_MAX_DRIVER_INFO_SIZE];
    VkConformanceVersion    conformanceVersion;
} VkPhysicalDeviceDriverProperties;
```

or the equivalent

```
// Provided by VK_KHR_driver_properties
typedef VkPhysicalDeviceDriverProperties VkPhysicalDeviceDriverPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- driverID is a unique identifier for the driver of the physical device.
- driverName is an array of VK_MAX_DRIVER_NAME_SIZE char containing a null-terminated UTF-8 string which is the name of the driver.
- driverInfo is an array of VK_MAX_DRIVER_INFO_SIZE char containing a null-terminated UTF-8 string with additional information about the driver.
- conformanceVersion is the latest version of the Vulkan conformance test that the implementor has successfully tested this driver against prior to release (see VkConformanceVersion).

If the VkPhysicalDeviceDriverProperties structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

These are properties of the driver corresponding to a physical device.

`driverID` **must** be immutable for a given driver across instances, processes, driver versions, and system reboots.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDriverProperties-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DRIVER_PROPERTIES`

Khronos driver IDs which **may** be returned in VkPhysicalDeviceDriverProperties::`driverID` are:

```
typedef enum VkDriverId {
    VK_DRIVER_ID_AMD_PROPRIETARY = 1,
    VK_DRIVER_ID_AMD_OPEN_SOURCE = 2,
    VK_DRIVER_ID_MESA_RADV = 3,
    VK_DRIVER_ID_NVIDIA_PROPRIETARY = 4,
    VK_DRIVER_ID_INTEL_PROPRIETARY_WINDOWS = 5,
    VK_DRIVER_ID_INTEL_OPEN_SOURCE_MESA = 6,
    VK_DRIVER_ID_IMAGINATION_PROPRIETARY = 7,
    VK_DRIVER_ID_QUALCOMM_PROPRIETARY = 8,
    VK_DRIVER_ID_ARM_PROPRIETARY = 9,
    VK_DRIVER_ID_GOOGLE_SWIFTSHADER = 10,
    VK_DRIVER_ID_GGP_PROPRIETARY = 11,
    VK_DRIVER_ID_BROADCOM_PROPRIETARY = 12,
    VK_DRIVER_ID_MESA_LLVMPIPE = 13,
    VK_DRIVER_ID_MOLTENVK = 14,
    VK_DRIVER_ID_COREAVI_PROPRIETARY = 15,
    VK_DRIVER_ID_JUICE_PROPRIETARY = 16,
    VK_DRIVER_ID_VERISILICON_PROPRIETARY = 17,
    VK_DRIVER_ID_MESA_TURNIP = 18,
    VK_DRIVER_ID_MESA_V3DV = 19,
    VK_DRIVER_ID_MESA_PANVK = 20,
    VK_DRIVER_ID_SAMSUNG_PROPRIETARY = 21,
    VK_DRIVER_ID_MESA_VENUS = 22,
    VK_DRIVER_ID_MESA_DOZEN = 23,
    VK_DRIVER_ID_MESA_NVK = 24,
    VK_DRIVER_ID_IMAGINATION_OPEN_SOURCE_MESA = 25,
    VK_DRIVER_ID_MESA_HONEYKRISP = 26,
    VK_DRIVER_ID_RESERVED_27 = 27,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_AMD_PROPRIETARY_KHR = VK_DRIVER_ID_AMD_PROPRIETARY,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_AMD_OPEN_SOURCE_KHR = VK_DRIVER_ID_AMD_OPEN_SOURCE,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_MESA_RADV_KHR = VK_DRIVER_ID_MESA_RADV,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_NVIDIA_PROPRIETARY_KHR = VK_DRIVER_ID_NVIDIA_PROPRIETARY,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_INTEL_PROPRIETARY_WINDOWS_KHR =
VK_DRIVER_ID_INTEL_PROPRIETARY_WINDOWS,
```

```
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_INTEL_OPEN_SOURCE_MESA_KHR = VK_DRIVER_ID_INTEL_OPEN_SOURCE_MESA,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_IMAGINATION_PROPRIETARY_KHR = VK_DRIVER_ID_IMAGINATION_PROPRIETARY,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_QUALCOMM_PROPRIETARY_KHR = VK_DRIVER_ID_QUALCOMM_PROPRIETARY,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_ARM_PROPRIETARY_KHR = VK_DRIVER_ID_ARM_PROPRIETARY,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_GOOGLE_SWIFTSHADER_KHR = VK_DRIVER_ID_GOOGLE_SWIFTSHADER,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_GGP_PROPRIETARY_KHR = VK_DRIVER_ID_GGP_PROPRIETARY,
  // Provided by VK_KHR_driver_properties
    VK_DRIVER_ID_BROADCOM_PROPRIETARY_KHR = VK_DRIVER_ID_BROADCOM_PROPRIETARY,
} VkDriverId;
```

or the equivalent

```
// Provided by VK_KHR_driver_properties
typedef VkDriverId VkDriverIdKHR;
```

NOTE

Khronos driver IDs may be allocated by vendors at any time. There may be multiple driver IDs for the same vendor, representing different drivers (for e.g. different platforms, proprietary or open source, etc.). Only the latest canonical versions of this Specification, of the corresponding `vk.xml` API Registry, and of the corresponding `vulkan_core.h` header file **must** contain all reserved Khronos driver IDs.

Only driver IDs registered with Khronos are given symbolic names. There **may** be unregistered driver IDs returned.

`VK_MAX_DRIVER_NAME_SIZE` is the length in `char` values of an array containing a driver name string, as returned in VkPhysicalDeviceDriverProperties::`driverName`.

```
#define VK_MAX_DRIVER_NAME_SIZE           256U
```

or the equivalent

```
#define VK_MAX_DRIVER_NAME_SIZE_KHR       VK_MAX_DRIVER_NAME_SIZE
```

`VK_MAX_DRIVER_INFO_SIZE` is the length in `char` values of an array containing a driver information string, as returned in VkPhysicalDeviceDriverProperties::`driverInfo`.

```
#define VK_MAX_DRIVER_INFO_SIZE           256U
```

or the equivalent

```
#define VK_MAX_DRIVER_INFO_SIZE_KHR       VK_MAX_DRIVER_INFO_SIZE
```

The conformance test suite version an implementation is compliant with is described with the `VkConformanceVersion` structure:

```
typedef struct VkConformanceVersion {
    uint8_t    major;
    uint8_t    minor;
    uint8_t    subminor;
    uint8_t    patch;
} VkConformanceVersion;
```

or the equivalent

```
// Provided by VK_KHR_driver_properties
typedef VkConformanceVersion VkConformanceVersionKHR;
```

- `major` is the major version number of the conformance test suite.
- `minor` is the minor version number of the conformance test suite.
- `subminor` is the subminor version number of the conformance test suite.
- `patch` is the patch version number of the conformance test suite.

The `VkPhysicalDevicePCIBusInfoPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_pci_bus_info
typedef struct VkPhysicalDevicePCIBusInfoPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           pciDomain;
    uint32_t           pciBus;
    uint32_t           pciDevice;
    uint32_t           pciFunction;
} VkPhysicalDevicePCIBusInfoPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pciDomain` is the PCI bus domain.
- `pciBus` is the PCI bus identifier.
- `pciDevice` is the PCI device identifier.
- `pciFunction` is the PCI device function identifier.

If the VkPhysicalDevicePCIBusInfoPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

These are properties of the PCI bus information of a physical device.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePCIBusInfoPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PCI_BUS_INFO_PROPERTIES_EXT

The VkPhysicalDeviceDrmPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_physical_device_drm
typedef struct VkPhysicalDeviceDrmPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           hasPrimary;
    VkBool32           hasRender;
    int64_t            primaryMajor;
    int64_t            primaryMinor;
    int64_t            renderMajor;
    int64_t            renderMinor;
} VkPhysicalDeviceDrmPropertiesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- hasPrimary is a boolean indicating whether the physical device has a DRM primary node.
- hasRender is a boolean indicating whether the physical device has a DRM render node.
- primaryMajor is the DRM primary node major number, if any.
- primaryMinor is the DRM primary node minor number, if any.
- renderMajor is the DRM render node major number, if any.
- renderMinor is the DRM render node minor number, if any.

If the VkPhysicalDeviceDrmPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

These are properties of the DRM information of a physical device.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDrmPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DRM_PROPERTIES_EXT

The VkPhysicalDeviceShaderIntegerDotProductProperties structure is defined as:

```
typedef struct VkPhysicalDeviceShaderIntegerDotProductProperties {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           integerDotProduct8BitUnsignedAccelerated;
    VkBool32           integerDotProduct8BitSignedAccelerated;
    VkBool32           integerDotProduct8BitMixedSignednessAccelerated;
    VkBool32           integerDotProduct4x8BitPackedUnsignedAccelerated;
    VkBool32           integerDotProduct4x8BitPackedSignedAccelerated;
    VkBool32           integerDotProduct4x8BitPackedMixedSignednessAccelerated;
    VkBool32           integerDotProduct16BitUnsignedAccelerated;
    VkBool32           integerDotProduct16BitSignedAccelerated;
    VkBool32           integerDotProduct16BitMixedSignednessAccelerated;
    VkBool32           integerDotProduct32BitUnsignedAccelerated;
    VkBool32           integerDotProduct32BitSignedAccelerated;
    VkBool32           integerDotProduct32BitMixedSignednessAccelerated;
    VkBool32           integerDotProduct64BitUnsignedAccelerated;
    VkBool32           integerDotProduct64BitSignedAccelerated;
    VkBool32           integerDotProduct64BitMixedSignednessAccelerated;
    VkBool32           integerDotProductAccumulatingSaturating8BitUnsignedAccelerated;
    VkBool32           integerDotProductAccumulatingSaturating8BitSignedAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating8BitMixedSignednessAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating4x8BitPackedUnsignedAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating4x8BitPackedSignedAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating4x8BitPackedMixedSignednessAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating16BitUnsignedAccelerated;
    VkBool32           integerDotProductAccumulatingSaturating16BitSignedAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating16BitMixedSignednessAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating32BitUnsignedAccelerated;
    VkBool32           integerDotProductAccumulatingSaturating32BitSignedAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating32BitMixedSignednessAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating64BitUnsignedAccelerated;
    VkBool32           integerDotProductAccumulatingSaturating64BitSignedAccelerated;
    VkBool32
integerDotProductAccumulatingSaturating64BitMixedSignednessAccelerated;
} VkPhysicalDeviceShaderIntegerDotProductProperties;
```

or the equivalent

```
// Provided by VK_KHR_shader_integer_dot_product
typedef VkPhysicalDeviceShaderIntegerDotProductProperties
VkPhysicalDeviceShaderIntegerDotProductPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- integerDotProduct8BitUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 8-bit unsigned dot product operations using the OpUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct8BitSignedAccelerated is a boolean that will be VK_TRUE if the support for 8-bit signed dot product operations using the OpSDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct8BitMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 8-bit mixed signedness dot product operations using the OpSUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct4x8BitPackedUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 8-bit unsigned dot product operations from operands packed into 32-bit integers using the OpUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct4x8BitPackedSignedAccelerated is a boolean that will be VK_TRUE if the support for 8-bit signed dot product operations from operands packed into 32-bit integers using the OpSDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct4x8BitPackedMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 8-bit mixed signedness dot product operations from operands packed into 32-bit integers using the OpSUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct16BitUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 16-bit unsigned dot product operations using the OpUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct16BitSignedAccelerated is a boolean that will be VK_TRUE if the support for 16-bit signed dot product operations using the OpSDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct16BitMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 16-bit mixed signedness dot product operations using the OpSUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct32BitUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 32-bit unsigned dot product operations using the OpUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct32BitSignedAccelerated is a boolean that will be VK_TRUE if the support for 32-bit signed dot product operations using the OpSDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct32BitMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 32-bit mixed signedness dot product operations using the OpSUDotKHR SPIR-V

instruction is accelerated as defined below.

- integerDotProduct64BitUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 64-bit unsigned dot product operations using the OpUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct64BitSignedAccelerated is a boolean that will be VK_TRUE if the support for 64-bit signed dot product operations using the OpSDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProduct64BitMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 64-bit mixed signedness dot product operations using the OpSUDotKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating8BitUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 8-bit unsigned accumulating saturating dot product operations using the OpUDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating8BitSignedAccelerated is a boolean that will be VK_TRUE if the support for 8-bit signed accumulating saturating dot product operations using the OpSDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating8BitMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 8-bit mixed signedness accumulating saturating dot product operations using the OpSUDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating4x8BitPackedUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 8-bit unsigned accumulating saturating dot product operations from operands packed into 32-bit integers using the OpUDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating4x8BitPackedSignedAccelerated is a boolean that will be VK_TRUE if the support for 8-bit signed accumulating saturating dot product operations from operands packed into 32-bit integers using the OpSDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating4x8BitPackedMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 8-bit mixed signedness accumulating saturating dot product operations from operands packed into 32-bit integers using the OpSUDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating16BitUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 16-bit unsigned accumulating saturating dot product operations using the OpUDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating16BitSignedAccelerated is a boolean that will be VK_TRUE if the support for 16-bit signed accumulating saturating dot product operations using the OpSDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating16BitMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 16-bit mixed signedness accumulating saturating dot product operations using the OpSUDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating32BitUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 32-bit unsigned accumulating saturating dot product operations using the OpUDotAccSatKHR SPIR-V instruction is accelerated as defined below.

- integerDotProductAccumulatingSaturating32BitSignedAccelerated is a boolean that will be VK_TRUE if the support for 32-bit signed accumulating saturating dot product operations using the OpSDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating32BitMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 32-bit mixed signedness accumulating saturating dot product operations using the OpSUDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating64BitUnsignedAccelerated is a boolean that will be VK_TRUE if the support for 64-bit unsigned accumulating saturating dot product operations using the OpUDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating64BitSignedAccelerated is a boolean that will be VK_TRUE if the support for 64-bit signed accumulating saturating dot product operations using the OpSDotAccSatKHR SPIR-V instruction is accelerated as defined below.
- integerDotProductAccumulatingSaturating64BitMixedSignednessAccelerated is a boolean that will be VK_TRUE if the support for 64-bit mixed signedness accumulating saturating dot product operations using the OpSUDotAccSatKHR SPIR-V instruction is accelerated as defined below.

If the VkPhysicalDeviceShaderIntegerDotProductProperties structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

These are properties of the integer dot product acceleration information of a physical device.

NOTE

A dot product operation is deemed accelerated if its implementation provides a performance advantage over application-provided code composed from elementary instructions and/or other dot product instructions, either because the implementation uses optimized machine code sequences whose generation from application-provided code cannot be guaranteed or because it uses hardware features that cannot otherwise be targeted from application-provided code.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderIntegerDotProductProperties-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_PROPERTIES

The VkPhysicalDeviceShaderTileImagePropertiesEXT structure is defined as:

```
// Provided by VK_EXT_shader_tile_image
typedef struct VkPhysicalDeviceShaderTileImagePropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderTileImageCoherentReadAccelerated;
    VkBool32           shaderTileImageReadSampleFromPixelRateInvocation;
    VkBool32           shaderTileImageReadFromHelperInvocation;
} VkPhysicalDeviceShaderTileImagePropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `shaderTileImageCoherentReadAccelerated` is a boolean that will be `VK_TRUE` if coherent reads of tile image data is accelerated.
- `shaderTileImageReadSampleFromPixelRateInvocation` is a boolean that will be `VK_TRUE` if reading from samples from a pixel rate fragment invocation is supported when VkPipelineMultisampleStateCreateInfo::`rasterizationSamples` > 1.
- `shaderTileImageReadFromHelperInvocation` is a boolean that will be `VK_TRUE` if reads of tile image data from helper fragment invocations result in valid values.

If the `VkPhysicalDeviceShaderTileImagePropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

These are properties of the tile image information of a physical device.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderTileImagePropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TILE_IMAGE_PROPERTIES_EXT`

To query properties of queues available on a physical device, call:

```
// Provided by VK_VERSION_1_0
void vkGetPhysicalDeviceQueueFamilyProperties(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pQueueFamilyPropertyCount,
    VkQueueFamilyProperties*                    pQueueFamilyProperties);
```

- `physicalDevice` is the handle to the physical device whose properties will be queried.
- `pQueueFamilyPropertyCount` is a pointer to an integer related to the number of queue families available or queried, as described below.
- `pQueueFamilyProperties` is either `NULL` or a pointer to an array of VkQueueFamilyProperties structures.

If `pQueueFamilyProperties` is `NULL`, then the number of queue families available is returned in `pQueueFamilyPropertyCount`. Implementations **must** support at least one queue family. Otherwise, `pQueueFamilyPropertyCount` **must** point to a variable set by the application to the number of elements in the `pQueueFamilyProperties` array, and on return the variable is overwritten with the number of structures actually written to `pQueueFamilyProperties`. If `pQueueFamilyPropertyCount` is less than the number of queue families available, at most `pQueueFamilyPropertyCount` structures will be written.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceQueueFamilyProperties-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceQueueFamilyProperties-pQueueFamilyPropertyCount-parameter
  `pQueueFamilyPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceQueueFamilyProperties-pQueueFamilyProperties-parameter
  If the value referenced by `pQueueFamilyPropertyCount` is not `0`, and `pQueueFamilyProperties` is not `NULL`, `pQueueFamilyProperties` **must** be a valid pointer to an array of `pQueueFamilyPropertyCount` VkQueueFamilyProperties structures

The `VkQueueFamilyProperties` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkQueueFamilyProperties {
    VkQueueFlags    queueFlags;
    uint32_t        queueCount;
    uint32_t        timestampValidBits;
    VkExtent3D      minImageTransferGranularity;
} VkQueueFamilyProperties;
```

- `queueFlags` is a bitmask of VkQueueFlagBits indicating capabilities of the queues in this queue family.
- `queueCount` is the unsigned integer count of queues in this queue family. Each queue family **must** support at least one queue.
- `timestampValidBits` is the unsigned integer count of meaningful bits in the timestamps written via vkCmdWriteTimestamp2KHR or vkCmdWriteTimestamp. The valid range for the count is 36 to 64 bits, or a value of 0, indicating no support for timestamps. Bits outside the valid range are guaranteed to be zeros.
- `minImageTransferGranularity` is the minimum granularity supported for image transfer operations on the queues in this queue family.

The value returned in `minImageTransferGranularity` has a unit of compressed texel blocks for images having a block-compressed format, and a unit of texels otherwise.

Possible values of `minImageTransferGranularity` are:

- (0,0,0) specifies that only whole mip levels **must** be transferred using the image transfer operations on the corresponding queues. In this case, the following restrictions apply to all offset and extent parameters of image transfer operations:
  - The `x`, `y`, and `z` members of a VkOffset3D parameter **must** always be zero.
  - The `width`, `height`, and `depth` members of a VkExtent3D parameter **must** always match the width, height, and depth of the image subresource corresponding to the parameter,

respectively.

- (A_x, A_y, A_z) where A_x, A_y, and A_z are all integer powers of two. In this case the following restrictions apply to all image transfer operations:
  - `x`, `y`, and `z` of a VkOffset3D parameter **must** be integer multiples of A_x, A_y, and A_z, respectively.
  - `width` of a VkExtent3D parameter **must** be an integer multiple of A_x, or else `x` + `width` **must** equal the width of the image subresource corresponding to the parameter.
  - `height` of a VkExtent3D parameter **must** be an integer multiple of A_y, or else `y` + `height` **must** equal the height of the image subresource corresponding to the parameter.
  - `depth` of a VkExtent3D parameter **must** be an integer multiple of A_z, or else `z` + `depth` **must** equal the depth of the image subresource corresponding to the parameter.
  - If the format of the image corresponding to the parameters is one of the block-compressed formats then for the purposes of the above calculations the granularity **must** be scaled up by the compressed texel block dimensions.

Queues supporting graphics and/or compute operations **must** report (1,1,1) in `minImageTransferGranularity`, meaning that there are no additional restrictions on the granularity of image transfer operations for these queues. Other queues supporting image transfer operations are only **required** to support whole mip level transfers, thus `minImageTransferGranularity` for queues belonging to such queue families **may** be (0,0,0).

The Device Memory section describes memory properties queried from the physical device.

For physical device feature queries see the Features chapter.

Bits which **may** be set in VkQueueFamilyProperties::`queueFlags`, indicating capabilities of queues in a queue family are:

```
// Provided by VK_VERSION_1_0
typedef enum VkQueueFlagBits {
    VK_QUEUE_GRAPHICS_BIT = 0x00000001,
    VK_QUEUE_COMPUTE_BIT = 0x00000002,
    VK_QUEUE_TRANSFER_BIT = 0x00000004,
    VK_QUEUE_SPARSE_BINDING_BIT = 0x00000008,
  // Provided by VK_KHR_video_decode_queue
    VK_QUEUE_VIDEO_DECODE_BIT_KHR = 0x00000020,
  // Provided by VK_KHR_video_encode_queue
    VK_QUEUE_VIDEO_ENCODE_BIT_KHR = 0x00000040,
} VkQueueFlagBits;
```

- `VK_QUEUE_GRAPHICS_BIT` specifies that queues in this queue family support graphics operations.
- `VK_QUEUE_COMPUTE_BIT` specifies that queues in this queue family support compute operations.
- `VK_QUEUE_TRANSFER_BIT` specifies that queues in this queue family support transfer operations.
- `VK_QUEUE_SPARSE_BINDING_BIT` specifies that queues in this queue family support sparse memory management operations (see Sparse Resources). If any of the sparse resource features are

enabled, then at least one queue family **must** support this bit.

- VK_QUEUE_VIDEO_DECODE_BIT_KHR specifies that queues in this queue family support video decode operations.
- VK_QUEUE_VIDEO_ENCODE_BIT_KHR specifies that queues in this queue family support video encode operations.

If an implementation exposes any queue family that supports graphics operations, at least one queue family of at least one physical device exposed by the implementation **must** support both graphics and compute operations.

NOTE	All commands that are allowed on a queue that supports transfer operations are also allowed on a queue that supports either graphics or compute operations. Thus, if the capabilities of a queue family include VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT, then reporting the VK_QUEUE_TRANSFER_BIT capability separately for that queue family is **optional**.

For further details see Queues.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkQueueFlags;
```

VkQueueFlags is a bitmask type for setting a mask of zero or more VkQueueFlagBits.

To query properties of queues available on a physical device, call:

```
// Provided by VK_KHR_get_physical_device_properties2
void vkGetPhysicalDeviceQueueFamilyProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pQueueFamilyPropertyCount,
    VkQueueFamilyProperties2*                   pQueueFamilyProperties);
```

- physicalDevice is the handle to the physical device whose properties will be queried.
- pQueueFamilyPropertyCount is a pointer to an integer related to the number of queue families available or queried, as described in vkGetPhysicalDeviceQueueFamilyProperties.
- pQueueFamilyProperties is either NULL or a pointer to an array of VkQueueFamilyProperties2 structures.

vkGetPhysicalDeviceQueueFamilyProperties2KHR behaves similarly to vkGetPhysicalDeviceQueueFamilyProperties, with the ability to return extended information in a pNext chain of output structures.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceQueueFamilyProperties2-physicalDevice-parameter physicalDevice **must** be a valid VkPhysicalDevice handle

- VUID-vkGetPhysicalDeviceQueueFamilyProperties2-pQueueFamilyPropertyCount-parameter
  `pQueueFamilyPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceQueueFamilyProperties2-pQueueFamilyProperties-parameter
  If the value referenced by `pQueueFamilyPropertyCount` is not `0`, and `pQueueFamilyProperties` is not `NULL`, `pQueueFamilyProperties` **must** be a valid pointer to an array of `pQueueFamilyPropertyCount` VkQueueFamilyProperties2 structures

The `VkQueueFamilyProperties2` structure is defined as:

```
typedef struct VkQueueFamilyProperties2 {
    VkStructureType           sType;
    void*                     pNext;
    VkQueueFamilyProperties   queueFamilyProperties;
} VkQueueFamilyProperties2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkQueueFamilyProperties2 VkQueueFamilyProperties2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `queueFamilyProperties` is a VkQueueFamilyProperties structure which is populated with the same values as in vkGetPhysicalDeviceQueueFamilyProperties.

Valid Usage (Implicit)

- VUID-VkQueueFamilyProperties2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_QUEUE_FAMILY_PROPERTIES_2`
- VUID-VkQueueFamilyProperties2-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkQueueFamilyGlobalPriorityPropertiesKHR, VkQueueFamilyQueryResultStatusPropertiesKHR, or VkQueueFamilyVideoPropertiesKHR
- VUID-VkQueueFamilyProperties2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

The VkQueueFamilyGlobalPriorityPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_global_priority
typedef struct VkQueueFamilyGlobalPriorityPropertiesKHR {
    VkStructureType             sType;
    void*                       pNext;
    uint32_t                    priorityCount;
    VkQueueGlobalPriorityKHR    priorities[VK_MAX_GLOBAL_PRIORITY_SIZE_KHR];
} VkQueueFamilyGlobalPriorityPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `priorityCount` is the number of supported global queue priorities in this queue family, and it **must** be greater than 0.
- `priorities` is an array of `VK_MAX_GLOBAL_PRIORITY_SIZE_KHR` VkQueueGlobalPriorityKHR enums representing all supported global queue priorities in this queue family. The first `priorityCount` elements of the array will be valid.

If the `VkQueueFamilyGlobalPriorityPropertiesKHR` structure is included in the `pNext` chain of the VkQueueFamilyProperties2 structure passed to vkGetPhysicalDeviceQueueFamilyProperties2KHR, it is filled in with the list of supported global queue priorities for the indicated family.

The valid elements of `priorities` **must** not contain any duplicate values.

The valid elements of `priorities` **must** be a continuous sequence of VkQueueGlobalPriorityKHR enums in the ascending order.

NOTE	For example, returning `priorityCount` as 3 with supported `priorities` as `VK_QUEUE_GLOBAL_PRIORITY_LOW_KHR`, `VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_KHR` and `VK_QUEUE_GLOBAL_PRIORITY_REALTIME_KHR` is not allowed.

Valid Usage (Implicit)

- VUID-VkQueueFamilyGlobalPriorityPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_QUEUE_FAMILY_GLOBAL_PRIORITY_PROPERTIES_KHR`

`VK_MAX_GLOBAL_PRIORITY_SIZE_KHR` is the length of an array of VkQueueGlobalPriorityKHR enumerants representing supported queue priorities, as returned in VkQueueFamilyGlobalPriorityPropertiesKHR::`priorities`.

```
#define VK_MAX_GLOBAL_PRIORITY_SIZE_KHR   16U
```

The VkQueueFamilyVideoPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkQueueFamilyVideoPropertiesKHR {
    VkStructureType                 sType;
    void*                           pNext;
    VkVideoCodecOperationFlagsKHR   videoCodecOperations;
} VkQueueFamilyVideoPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `videoCodecOperations` is a bitmask of VkVideoCodecOperationFlagBitsKHR that indicates the set of video codec operations supported by the queue family.

If this structure is included in the `pNext` chain of the VkQueueFamilyProperties2 structure passed to vkGetPhysicalDeviceQueueFamilyProperties2KHR, then it is filled with the set of video codec operations supported by the specified queue family.

Valid Usage (Implicit)

- VUID-VkQueueFamilyVideoPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_QUEUE_FAMILY_VIDEO_PROPERTIES_KHR`

The VkQueueFamilyQueryResultStatusPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkQueueFamilyQueryResultStatusPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           queryResultStatusSupport;
} VkQueueFamilyQueryResultStatusPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `queryResultStatusSupport` reports `VK_TRUE` if query type `VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR` and use of `VK_QUERY_RESULT_WITH_STATUS_BIT_KHR` are supported.

If this structure is included in the `pNext` chain of the VkQueueFamilyProperties2 structure passed to vkGetPhysicalDeviceQueueFamilyProperties2KHR, then it is filled with information about whether result status queries are supported by the specified queue family.

Valid Usage (Implicit)

- VUID-VkQueueFamilyQueryResultStatusPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_QUEUE_FAMILY_QUERY_RESULT_STATUS_PROPERTIES_KHR`

To enumerate the performance query counters available on a queue family of a physical device, call:

```
// Provided by VK_KHR_performance_query
VkResult vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t                                    queueFamilyIndex,
    uint32_t*                                   pCounterCount,
    VkPerformanceCounterKHR*                    pCounters,
    VkPerformanceCounterDescriptionKHR*         pCounterDescriptions);
```

- `physicalDevice` is the handle to the physical device whose queue family performance query counter properties will be queried.
- `queueFamilyIndex` is the index into the queue family of the physical device we want to get properties for.
- `pCounterCount` is a pointer to an integer related to the number of counters available or queried, as described below.
- `pCounters` is either `NULL` or a pointer to an array of VkPerformanceCounterKHR structures.
- `pCounterDescriptions` is either `NULL` or a pointer to an array of VkPerformanceCounterDescriptionKHR structures.

If `pCounters` is `NULL` and `pCounterDescriptions` is `NULL`, then the number of counters available is returned in `pCounterCount`. Otherwise, `pCounterCount` **must** point to a variable set by the application to the number of elements in the `pCounters`, `pCounterDescriptions`, or both arrays and on return the variable is overwritten with the number of structures actually written out. If `pCounterCount` is less than the number of counters available, at most `pCounterCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available counters were returned.

Valid Usage (Implicit)

- VUID-vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR-pCounterCount-parameter
  `pCounterCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR-pCounters-parameter
  If the value referenced by `pCounterCount` is not `0`, and `pCounters` is not `NULL`, `pCounters` **must** be a valid pointer to an array of `pCounterCount` VkPerformanceCounterKHR structures
- VUID-vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR-pCounterDescriptions-parameter
  If the value referenced by `pCounterCount` is not `0`, and `pCounterDescriptions` is not `NULL`,

pCounterDescriptions **must** be a valid pointer to an array of pCounterCount VkPerformanceCounterDescriptionKHR structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INITIALIZATION_FAILED

The VkPerformanceCounterKHR structure is defined as:

```
// Provided by VK_KHR_performance_query
typedef struct VkPerformanceCounterKHR {
    VkStructureType                 sType;
    void*                           pNext;
    VkPerformanceCounterUnitKHR     unit;
    VkPerformanceCounterScopeKHR    scope;
    VkPerformanceCounterStorageKHR  storage;
    uint8_t                         uuid[VK_UUID_SIZE];
} VkPerformanceCounterKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- unit is a VkPerformanceCounterUnitKHR specifying the unit that the counter data will record.
- scope is a VkPerformanceCounterScopeKHR specifying the scope that the counter belongs to.
- storage is a VkPerformanceCounterStorageKHR specifying the storage type that the counter's data uses.
- uuid is an array of size VK_UUID_SIZE, containing 8-bit values that represent a universally unique identifier for the counter of the physical device.

Valid Usage (Implicit)

- VUID-VkPerformanceCounterKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PERFORMANCE_COUNTER_KHR
- VUID-VkPerformanceCounterKHR-pNext-pNext
  pNext **must** be NULL

Performance counters have an associated unit. This unit describes how to interpret the performance counter result.

The performance counter unit types which **may** be returned in VkPerformanceCounterKHR::unit are:

```
// Provided by VK_KHR_performance_query
typedef enum VkPerformanceCounterUnitKHR {
    VK_PERFORMANCE_COUNTER_UNIT_GENERIC_KHR = 0,
    VK_PERFORMANCE_COUNTER_UNIT_PERCENTAGE_KHR = 1,
    VK_PERFORMANCE_COUNTER_UNIT_NANOSECONDS_KHR = 2,
    VK_PERFORMANCE_COUNTER_UNIT_BYTES_KHR = 3,
    VK_PERFORMANCE_COUNTER_UNIT_BYTES_PER_SECOND_KHR = 4,
    VK_PERFORMANCE_COUNTER_UNIT_KELVIN_KHR = 5,
    VK_PERFORMANCE_COUNTER_UNIT_WATTS_KHR = 6,
    VK_PERFORMANCE_COUNTER_UNIT_VOLTS_KHR = 7,
    VK_PERFORMANCE_COUNTER_UNIT_AMPS_KHR = 8,
    VK_PERFORMANCE_COUNTER_UNIT_HERTZ_KHR = 9,
    VK_PERFORMANCE_COUNTER_UNIT_CYCLES_KHR = 10,
} VkPerformanceCounterUnitKHR;
```

- VK_PERFORMANCE_COUNTER_UNIT_GENERIC_KHR - the performance counter unit is a generic data point.
- VK_PERFORMANCE_COUNTER_UNIT_PERCENTAGE_KHR - the performance counter unit is a percentage (%).
- VK_PERFORMANCE_COUNTER_UNIT_NANOSECONDS_KHR - the performance counter unit is a value of nanoseconds (ns).
- VK_PERFORMANCE_COUNTER_UNIT_BYTES_KHR - the performance counter unit is a value of bytes.
- VK_PERFORMANCE_COUNTER_UNIT_BYTES_PER_SECOND_KHR - the performance counter unit is a value of bytes/s.
- VK_PERFORMANCE_COUNTER_UNIT_KELVIN_KHR - the performance counter unit is a temperature reported in Kelvin.
- VK_PERFORMANCE_COUNTER_UNIT_WATTS_KHR - the performance counter unit is a value of watts (W).
- VK_PERFORMANCE_COUNTER_UNIT_VOLTS_KHR - the performance counter unit is a value of volts (V).
- VK_PERFORMANCE_COUNTER_UNIT_AMPS_KHR - the performance counter unit is a value of amps (A).
- VK_PERFORMANCE_COUNTER_UNIT_HERTZ_KHR - the performance counter unit is a value of hertz (Hz).
- VK_PERFORMANCE_COUNTER_UNIT_CYCLES_KHR - the performance counter unit is a value of cycles.

Performance counters have an associated scope. This scope describes the granularity of a performance counter.

The performance counter scope types which **may** be returned in VkPerformanceCounterKHR::scope are:

```
// Provided by VK_KHR_performance_query
typedef enum VkPerformanceCounterScopeKHR {
    VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_BUFFER_KHR = 0,
    VK_PERFORMANCE_COUNTER_SCOPE_RENDER_PASS_KHR = 1,
    VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_KHR = 2,
  // VK_QUERY_SCOPE_COMMAND_BUFFER_KHR is a deprecated alias
    VK_QUERY_SCOPE_COMMAND_BUFFER_KHR =
VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_BUFFER_KHR,
  // VK_QUERY_SCOPE_RENDER_PASS_KHR is a deprecated alias
    VK_QUERY_SCOPE_RENDER_PASS_KHR = VK_PERFORMANCE_COUNTER_SCOPE_RENDER_PASS_KHR,
  // VK_QUERY_SCOPE_COMMAND_KHR is a deprecated alias
    VK_QUERY_SCOPE_COMMAND_KHR = VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_KHR,
} VkPerformanceCounterScopeKHR;
```

- VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_BUFFER_KHR - the performance counter scope is a single complete command buffer.
- VK_PERFORMANCE_COUNTER_SCOPE_RENDER_PASS_KHR - the performance counter scope is zero or more complete render passes. The performance query containing the performance counter **must** begin and end outside a render pass instance.
- VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_KHR - the performance counter scope is zero or more commands.

Performance counters have an associated storage. This storage describes the payload of a counter result.

The performance counter storage types which **may** be returned in VkPerformanceCounterKHR ::storage are:

```
// Provided by VK_KHR_performance_query
typedef enum VkPerformanceCounterStorageKHR {
    VK_PERFORMANCE_COUNTER_STORAGE_INT32_KHR = 0,
    VK_PERFORMANCE_COUNTER_STORAGE_INT64_KHR = 1,
    VK_PERFORMANCE_COUNTER_STORAGE_UINT32_KHR = 2,
    VK_PERFORMANCE_COUNTER_STORAGE_UINT64_KHR = 3,
    VK_PERFORMANCE_COUNTER_STORAGE_FLOAT32_KHR = 4,
    VK_PERFORMANCE_COUNTER_STORAGE_FLOAT64_KHR = 5,
} VkPerformanceCounterStorageKHR;
```

- VK_PERFORMANCE_COUNTER_STORAGE_INT32_KHR - the performance counter storage is a 32-bit signed integer.
- VK_PERFORMANCE_COUNTER_STORAGE_INT64_KHR - the performance counter storage is a 64-bit signed integer.
- VK_PERFORMANCE_COUNTER_STORAGE_UINT32_KHR - the performance counter storage is a 32-bit unsigned integer.
- VK_PERFORMANCE_COUNTER_STORAGE_UINT64_KHR - the performance counter storage is a 64-bit unsigned integer.

- VK_PERFORMANCE_COUNTER_STORAGE_FLOAT32_KHR - the performance counter storage is a 32-bit floating-point.
- VK_PERFORMANCE_COUNTER_STORAGE_FLOAT64_KHR - the performance counter storage is a 64-bit floating-point.

The VkPerformanceCounterDescriptionKHR structure is defined as:

```
// Provided by VK_KHR_performance_query
typedef struct VkPerformanceCounterDescriptionKHR {
    VkStructureType                            sType;
    void*                                      pNext;
    VkPerformanceCounterDescriptionFlagsKHR    flags;
    char                                       name[VK_MAX_DESCRIPTION_SIZE];
    char                                       category[VK_MAX_DESCRIPTION_SIZE];
    char                                       description[VK_MAX_DESCRIPTION_SIZE];
} VkPerformanceCounterDescriptionKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkPerformanceCounterDescriptionFlagBitsKHR indicating the usage behavior for the counter.
- name is an array of size VK_MAX_DESCRIPTION_SIZE, containing a null-terminated UTF-8 string specifying the name of the counter.
- category is an array of size VK_MAX_DESCRIPTION_SIZE, containing a null-terminated UTF-8 string specifying the category of the counter.
- description is an array of size VK_MAX_DESCRIPTION_SIZE, containing a null-terminated UTF-8 string specifying the description of the counter.

Valid Usage (Implicit)

- VUID-VkPerformanceCounterDescriptionKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PERFORMANCE_COUNTER_DESCRIPTION_KHR
- VUID-VkPerformanceCounterDescriptionKHR-pNext-pNext
  pNext **must** be NULL

Bits which **can** be set in VkPerformanceCounterDescriptionKHR::flags, specifying usage behavior of a performance counter, are:

```
// Provided by VK_KHR_performance_query
typedef enum VkPerformanceCounterDescriptionFlagBitsKHR {
    VK_PERFORMANCE_COUNTER_DESCRIPTION_PERFORMANCE_IMPACTING_BIT_KHR = 0x00000001,
    VK_PERFORMANCE_COUNTER_DESCRIPTION_CONCURRENTLY_IMPACTED_BIT_KHR = 0x00000002,
  // VK_PERFORMANCE_COUNTER_DESCRIPTION_PERFORMANCE_IMPACTING_KHR is a deprecated
alias
    VK_PERFORMANCE_COUNTER_DESCRIPTION_PERFORMANCE_IMPACTING_KHR =
VK_PERFORMANCE_COUNTER_DESCRIPTION_PERFORMANCE_IMPACTING_BIT_KHR,
  // VK_PERFORMANCE_COUNTER_DESCRIPTION_CONCURRENTLY_IMPACTED_KHR is a deprecated
alias
    VK_PERFORMANCE_COUNTER_DESCRIPTION_CONCURRENTLY_IMPACTED_KHR =
VK_PERFORMANCE_COUNTER_DESCRIPTION_CONCURRENTLY_IMPACTED_BIT_KHR,
} VkPerformanceCounterDescriptionFlagBitsKHR;
```

- VK_PERFORMANCE_COUNTER_DESCRIPTION_PERFORMANCE_IMPACTING_BIT_KHR specifies that recording the counter **may** have a noticeable performance impact.
- VK_PERFORMANCE_COUNTER_DESCRIPTION_CONCURRENTLY_IMPACTED_BIT_KHR specifies that concurrently recording the counter while other submitted command buffers are running **may** impact the accuracy of the recording.

```
// Provided by VK_KHR_performance_query
typedef VkFlags VkPerformanceCounterDescriptionFlagsKHR;
```

VkPerformanceCounterDescriptionFlagsKHR is a bitmask type for setting a mask of zero or more VkPerformanceCounterDescriptionFlagBitsKHR.

5.2. Devices

Device objects represent logical connections to physical devices. Each device exposes a number of *queue families* each having one or more *queues*. All queues in a queue family support the same operations.

As described in Physical Devices, a Vulkan application will first query for all physical devices in a system. Each physical device **can** then be queried for its capabilities, including its queue and queue family properties. Once an acceptable physical device is identified, an application will create a corresponding logical device. The created logical device is then the primary interface to the physical device.

How to enumerate the physical devices in a system and query those physical devices for their queue family properties is described in the Physical Device Enumeration section above.

A single logical device **can** be created from multiple physical devices, if those physical devices belong to the same device group. A *device group* is a set of physical devices that support accessing each other's memory and recording a single command buffer that **can** be executed on all the physical devices. Device groups are enumerated by calling vkEnumeratePhysicalDeviceGroupsKHR, and a logical device is created from a subset of the physical devices in a device group by passing the physical devices through VkDeviceGroupDeviceCreateInfo. For two physical devices to be in the

same device group, they **must** support identical extensions, features, and properties.

NOTE

Physical devices in the same device group **must** be so similar because there are no rules for how different features/properties would interact. They **must** return the same values for nearly every invariant `vkGetPhysicalDevice*` feature, property, capability, etc., but could potentially differ for certain queries based on things like having a different display connected, or a different compositor. The specification does not attempt to enumerate which state is in each category, because such a list would quickly become out of date.

To retrieve a list of the device groups present in the system, call:

```
// Provided by VK_KHR_device_group_creation
VkResult vkEnumeratePhysicalDeviceGroupsKHR(
    VkInstance                                  instance,
    uint32_t*                                   pPhysicalDeviceGroupCount,
    VkPhysicalDeviceGroupProperties*            pPhysicalDeviceGroupProperties);
```

- `instance` is a handle to a Vulkan instance previously created with vkCreateInstance.
- `pPhysicalDeviceGroupCount` is a pointer to an integer related to the number of device groups available or queried, as described below.
- `pPhysicalDeviceGroupProperties` is either `NULL` or a pointer to an array of VkPhysicalDeviceGroupProperties structures.

If `pPhysicalDeviceGroupProperties` is `NULL`, then the number of device groups available is returned in `pPhysicalDeviceGroupCount`. Otherwise, `pPhysicalDeviceGroupCount` **must** point to a variable set by the application to the number of elements in the `pPhysicalDeviceGroupProperties` array, and on return the variable is overwritten with the number of structures actually written to `pPhysicalDeviceGroupProperties`. If `pPhysicalDeviceGroupCount` is less than the number of device groups available, at most `pPhysicalDeviceGroupCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available device groups were returned.

Every physical device **must** be in exactly one device group.

Valid Usage (Implicit)

- VUID-vkEnumeratePhysicalDeviceGroups-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkEnumeratePhysicalDeviceGroups-pPhysicalDeviceGroupCount-parameter
  `pPhysicalDeviceGroupCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkEnumeratePhysicalDeviceGroups-pPhysicalDeviceGroupProperties-parameter
  If the value referenced by `pPhysicalDeviceGroupCount` is not `0`, and `pPhysicalDeviceGroupProperties` is not `NULL`, `pPhysicalDeviceGroupProperties` **must** be a valid pointer to an array of `pPhysicalDeviceGroupCount` VkPhysicalDeviceGroupProperties structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INITIALIZATION_FAILED

The VkPhysicalDeviceGroupProperties structure is defined as:

```
typedef struct VkPhysicalDeviceGroupProperties {
    VkStructureType     sType;
    void*               pNext;
    uint32_t            physicalDeviceCount;
    VkPhysicalDevice    physicalDevices[VK_MAX_DEVICE_GROUP_SIZE];
    VkBool32            subsetAllocation;
} VkPhysicalDeviceGroupProperties;
```

or the equivalent

```
// Provided by VK_KHR_device_group_creation
typedef VkPhysicalDeviceGroupProperties VkPhysicalDeviceGroupPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- physicalDeviceCount is the number of physical devices in the group.
- physicalDevices is an array of VK_MAX_DEVICE_GROUP_SIZE VkPhysicalDevice handles representing all physical devices in the group. The first physicalDeviceCount elements of the array will be valid.
- subsetAllocation specifies whether logical devices created from the group support allocating device memory on a subset of devices, via the deviceMask member of the VkMemoryAllocateFlagsInfo. If this is VK_FALSE, then all device memory allocations are made across all physical devices in the group. If physicalDeviceCount is 1, then subsetAllocation **must** be VK_FALSE.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceGroupProperties-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GROUP_PROPERTIES

- VUID-VkPhysicalDeviceGroupProperties-pNext-pNext
  `pNext` **must** be `NULL`

`VK_MAX_DEVICE_GROUP_SIZE` is the length of an array containing VkPhysicalDevice handle values representing all physical devices in a group, as returned in VkPhysicalDeviceGroupProperties ::`physicalDevices`.

```
#define VK_MAX_DEVICE_GROUP_SIZE          32U
```

or the equivalent

```
#define VK_MAX_DEVICE_GROUP_SIZE_KHR      VK_MAX_DEVICE_GROUP_SIZE
```

5.2.1. Device Creation

Logical devices are represented by `VkDevice` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_HANDLE(VkDevice)
```

A logical device is created as a *connection* to a physical device. To create a logical device, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateDevice(
    VkPhysicalDevice                            physicalDevice,
    const VkDeviceCreateInfo*                   pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkDevice*                                   pDevice);
```

- `physicalDevice` **must** be one of the device handles returned from a call to `vkEnumeratePhysicalDevices` (see Physical Device Enumeration).
- `pCreateInfo` is a pointer to a VkDeviceCreateInfo structure containing information about how to create the device.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pDevice` is a pointer to a handle in which the created VkDevice is returned.

`vkCreateDevice` verifies that extensions and features requested in the `ppEnabledExtensionNames` and `pEnabledFeatures` members of `pCreateInfo`, respectively, are supported by the implementation. If any requested extension is not supported, `vkCreateDevice` **must** return `VK_ERROR_EXTENSION_NOT_PRESENT`. If any requested feature is not supported, `vkCreateDevice` **must** return `VK_ERROR_FEATURE_NOT_PRESENT`. Support for extensions **can** be checked before creating a device by querying vkEnumerateDeviceExtensionProperties. Support for features **can** similarly be checked by querying vkGetPhysicalDeviceFeatures.

After verifying and enabling the extensions the VkDevice object is created and returned to the application.

Multiple logical devices **can** be created from the same physical device. Logical device creation **may** fail due to lack of device-specific resources (in addition to other errors). If that occurs, vkCreateDevice will return VK_ERROR_TOO_MANY_OBJECTS.

Valid Usage

- VUID-vkCreateDevice-ppEnabledExtensionNames-01387
  All required device extensions for each extension in the VkDeviceCreateInfo ::ppEnabledExtensionNames list **must** also be present in that list

Valid Usage (Implicit)

- VUID-vkCreateDevice-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkCreateDevice-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkDeviceCreateInfo structure
- VUID-vkCreateDevice-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateDevice-pDevice-parameter
  pDevice **must** be a valid pointer to a VkDevice handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INITIALIZATION_FAILED
- VK_ERROR_EXTENSION_NOT_PRESENT
- VK_ERROR_FEATURE_NOT_PRESENT
- VK_ERROR_TOO_MANY_OBJECTS
- VK_ERROR_DEVICE_LOST

The VkDeviceCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDeviceCreateInfo {
    VkStructureType                    sType;
    const void*                        pNext;
    VkDeviceCreateFlags                flags;
    uint32_t                           queueCreateInfoCount;
    const VkDeviceQueueCreateInfo*     pQueueCreateInfos;
    // enabledLayerCount is deprecated and should not be used
    uint32_t                           enabledLayerCount;
    // ppEnabledLayerNames is deprecated and should not be used
    const char* const*                 ppEnabledLayerNames;
    uint32_t                           enabledExtensionCount;
    const char* const*                 ppEnabledExtensionNames;
    const VkPhysicalDeviceFeatures*    pEnabledFeatures;
} VkDeviceCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `queueCreateInfoCount` is the unsigned integer size of the `pQueueCreateInfos` array. Refer to the Queue Creation section below for further details.
- `pQueueCreateInfos` is a pointer to an array of VkDeviceQueueCreateInfo structures describing the queues that are requested to be created along with the logical device. Refer to the Queue Creation section below for further details.
- `enabledLayerCount` is deprecated and ignored.
- `ppEnabledLayerNames` is deprecated and ignored. See Device Layer Deprecation.
- `enabledExtensionCount` is the number of device extensions to enable.
- `ppEnabledExtensionNames` is a pointer to an array of `enabledExtensionCount` null-terminated UTF-8 strings containing the names of extensions to enable for the created device. See the Extensions section for further details.
- `pEnabledFeatures` is `NULL` or a pointer to a VkPhysicalDeviceFeatures structure containing boolean indicators of all the features to be enabled. Refer to the Features section for further details.

Valid Usage

- VUID-VkDeviceCreateInfo-queueFamilyIndex-02802
  The `queueFamilyIndex` member of each element of `pQueueCreateInfos` **must** be unique within `pQueueCreateInfos`
- VUID-VkDeviceCreateInfo-pNext-00373
  If the `pNext` chain includes a VkPhysicalDeviceFeatures2 structure, then `pEnabledFeatures` **must** be `NULL`
- VUID-VkDeviceCreateInfo-pProperties-04451

If the VK_KHR_portability_subset extension is included in pProperties of vkEnumerateDeviceExtensionProperties, ppEnabledExtensionNames **must** include "VK_KHR_portability_subset"

- VUID-VkDeviceCreateInfo-fragmentDensityMap-04481
  If the fragmentDensityMap feature is enabled, the pipelineFragmentShadingRate feature **must** not be enabled
- VUID-VkDeviceCreateInfo-fragmentDensityMap-04482
  If the fragmentDensityMap feature is enabled, the primitiveFragmentShadingRate feature **must** not be enabled
- VUID-VkDeviceCreateInfo-fragmentDensityMap-04483
  If the fragmentDensityMap feature is enabled, the attachmentFragmentShadingRate feature **must** not be enabled
- VUID-VkDeviceCreateInfo-None-04896
  If the sparseImageInt64Atomics feature is enabled, shaderImageInt64Atomics **must** be enabled
- VUID-VkDeviceCreateInfo-None-04897
  If the sparseImageFloat32Atomics feature is enabled, shaderImageFloat32Atomics **must** be enabled
- VUID-VkDeviceCreateInfo-None-04898
  If the sparseImageFloat32AtomicAdd feature is enabled, shaderImageFloat32AtomicAdd **must** be enabled
- VUID-VkDeviceCreateInfo-sparseImageFloat32AtomicMinMax-04975
  If the sparseImageFloat32AtomicMinMax feature is enabled, shaderImageFloat32AtomicMinMax **must** be enabled

Valid Usage (Implicit)

- VUID-VkDeviceCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO
- VUID-VkDeviceCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkDeviceDeviceMemoryReportCreateInfoEXT, VkDeviceGroupDeviceCreateInfo, VkDevicePipelineBinaryInternalCacheControlKHR, VkPhysicalDevice16BitStorageFeatures, VkPhysicalDevice8BitStorageFeatures, VkPhysicalDeviceASTCDecodeFeaturesEXT, VkPhysicalDeviceAccelerationStructureFeaturesKHR, VkPhysicalDeviceAttachmentFeedbackLoopDynamicStateFeaturesEXT, VkPhysicalDeviceAttachmentFeedbackLoopLayoutFeaturesEXT, VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT, VkPhysicalDeviceBorderColorSwizzleFeaturesEXT, VkPhysicalDeviceBufferDeviceAddressFeatures, VkPhysicalDeviceColorWriteEnableFeaturesEXT, VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR,

VkPhysicalDeviceCooperativeMatrixFeaturesKHR,
VkPhysicalDeviceCustomBorderColorFeaturesEXT,
VkPhysicalDeviceDepthBiasControlFeaturesEXT,
VkPhysicalDeviceDepthClampControlFeaturesEXT,
VkPhysicalDeviceDepthClipControlFeaturesEXT,
VkPhysicalDeviceDepthClipEnableFeaturesEXT,
VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT,
VkPhysicalDeviceDeviceMemoryReportFeaturesEXT,
VkPhysicalDeviceDynamicRenderingFeatures,
VkPhysicalDeviceDynamicRenderingLocalReadFeaturesKHR,
VkPhysicalDeviceDynamicRenderingUnusedAttachmentsFeaturesEXT,
VkPhysicalDeviceExtendedDynamicState3FeaturesEXT, VkPhysicalDeviceFeatures2,
VkPhysicalDeviceFragmentDensityMap2FeaturesEXT,
VkPhysicalDeviceFragmentDensityMapFeaturesEXT,
VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR,
VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT,
VkPhysicalDeviceFragmentShadingRateFeaturesKHR,
VkPhysicalDeviceFrameBoundaryFeaturesEXT,
VkPhysicalDeviceGlobalPriorityQueryFeaturesKHR,
VkPhysicalDeviceHostImageCopyFeaturesEXT,
VkPhysicalDeviceImage2DViewOf3DFeaturesEXT,
VkPhysicalDeviceImageViewMinLodFeaturesEXT,
VkPhysicalDeviceImagelessFramebufferFeatures,
VkPhysicalDeviceIndexTypeUint8FeaturesKHR,
VkPhysicalDeviceLegacyVertexAttributesFeaturesEXT,
VkPhysicalDeviceLineRasterizationFeaturesKHR,
VkPhysicalDeviceMaintenance4Features, VkPhysicalDeviceMaintenance5FeaturesKHR,
VkPhysicalDeviceMaintenance6FeaturesKHR,
VkPhysicalDeviceMaintenance7FeaturesKHR,
VkPhysicalDeviceMapMemoryPlacedFeaturesEXT,
VkPhysicalDeviceMemoryPriorityFeaturesEXT, VkPhysicalDeviceMultiDrawFeaturesEXT,
VkPhysicalDeviceMultiviewFeatures,
VkPhysicalDeviceNestedCommandBufferFeaturesEXT,
VkPhysicalDeviceNonSeamlessCubeMapFeaturesEXT,
VkPhysicalDeviceOpacityMicromapFeaturesEXT,
VkPhysicalDevicePerformanceQueryFeaturesKHR,
VkPhysicalDevicePipelineBinaryFeaturesKHR,
VkPhysicalDevicePipelineExecutablePropertiesFeaturesKHR,
VkPhysicalDevicePortabilitySubsetFeaturesKHR, VkPhysicalDevicePresentIdFeaturesKHR,
VkPhysicalDevicePresentModeFifoLatestReadyFeaturesEXT,
VkPhysicalDevicePresentWaitFeaturesKHR,
VkPhysicalDevicePrimitiveTopologyListRestartFeaturesEXT,
VkPhysicalDevicePrimitivesGeneratedQueryFeaturesEXT,
VkPhysicalDeviceProvokingVertexFeaturesEXT, VkPhysicalDeviceRayQueryFeaturesKHR,
VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR,
VkPhysicalDeviceRayTracingPipelineFeaturesKHR,
VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR,
VkPhysicalDeviceRobustness2FeaturesEXT,

VkPhysicalDeviceSamplerYcbcrConversionFeatures,
VkPhysicalDeviceSeparateDepthStencilLayoutsFeatures,
VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT,
VkPhysicalDeviceShaderAtomicFloatFeaturesEXT,
VkPhysicalDeviceShaderAtomicInt64Features,
VkPhysicalDeviceShaderClockFeaturesKHR,
VkPhysicalDeviceShaderExpectAssumeFeaturesKHR,
VkPhysicalDeviceShaderFloat16Int8Features,
VkPhysicalDeviceShaderFloatControls2FeaturesKHR,
VkPhysicalDeviceShaderImageAtomicInt64FeaturesEXT,
VkPhysicalDeviceShaderIntegerDotProductFeatures,
VkPhysicalDeviceShaderMaximalReconvergenceFeaturesKHR,
VkPhysicalDeviceShaderModuleIdentifierFeaturesEXT,
VkPhysicalDeviceShaderObjectFeaturesEXT,
VkPhysicalDeviceShaderQuadControlFeaturesKHR,
VkPhysicalDeviceShaderRelaxedExtendedInstructionFeaturesKHR,
VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT,
VkPhysicalDeviceShaderSubgroupExtendedTypesFeatures,
VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR,
VkPhysicalDeviceShaderSubgroupUniformControlFlowFeaturesKHR,
VkPhysicalDeviceShaderTerminateInvocationFeatures,
VkPhysicalDeviceShaderTileImageFeaturesEXT,
VkPhysicalDeviceSynchronization2Features,
VkPhysicalDeviceTimelineSemaphoreFeatures,
VkPhysicalDeviceTransformFeedbackFeaturesEXT,
VkPhysicalDeviceUniformBufferStandardLayoutFeatures,
VkPhysicalDeviceVariablePointersFeatures,
VkPhysicalDeviceVertexAttributeDivisorFeaturesKHR,
VkPhysicalDeviceVertexInputDynamicStateFeaturesEXT,
VkPhysicalDeviceVideoEncodeAV1FeaturesKHR,
VkPhysicalDeviceVideoEncodeQuantizationMapFeaturesKHR,
VkPhysicalDeviceVideoMaintenance1FeaturesKHR,
VkPhysicalDeviceVulkanMemoryModelFeatures,
VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR,
VkPhysicalDeviceYcbcrImageArraysFeaturesEXT, or
VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeatures

- VUID-VkDeviceCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique, with the exception of structures of type VkDeviceDeviceMemoryReportCreateInfoEXT
- VUID-VkDeviceCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkDeviceCreateInfo-pQueueCreateInfos-parameter
  `pQueueCreateInfos` **must** be a valid pointer to an array of `queueCreateInfoCount` valid VkDeviceQueueCreateInfo structures
- VUID-VkDeviceCreateInfo-ppEnabledLayerNames-parameter
  If `enabledLayerCount` is not `0`, `ppEnabledLayerNames` **must** be a valid pointer to an array of

enabledLayerCount null-terminated UTF-8 strings

- VUID-VkDeviceCreateInfo-ppEnabledExtensionNames-parameter
  If enabledExtensionCount is not 0, ppEnabledExtensionNames **must** be a valid pointer to an array of enabledExtensionCount null-terminated UTF-8 strings
- VUID-VkDeviceCreateInfo-pEnabledFeatures-parameter
  If pEnabledFeatures is not NULL, pEnabledFeatures **must** be a valid pointer to a valid VkPhysicalDeviceFeatures structure
- VUID-VkDeviceCreateInfo-queueCreateInfoCount-arraylength
  queueCreateInfoCount **must** be greater than 0

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkDeviceCreateFlags;
```

VkDeviceCreateFlags is a bitmask type for setting a mask, but is currently reserved for future use.

A logical device **can** be created that connects to one or more physical devices by adding a VkDeviceGroupDeviceCreateInfo structure to the pNext chain of VkDeviceCreateInfo. The VkDeviceGroupDeviceCreateInfo structure is defined as:

```
typedef struct VkDeviceGroupDeviceCreateInfo {
    VkStructureType            sType;
    const void*                pNext;
    uint32_t                   physicalDeviceCount;
    const VkPhysicalDevice*    pPhysicalDevices;
} VkDeviceGroupDeviceCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_device_group_creation
typedef VkDeviceGroupDeviceCreateInfo VkDeviceGroupDeviceCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- physicalDeviceCount is the number of elements in the pPhysicalDevices array.
- pPhysicalDevices is a pointer to an array of physical device handles belonging to the same device group.

The elements of the pPhysicalDevices array are an ordered list of the physical devices that the logical device represents. These **must** be a subset of a single device group, and need not be in the same order as they were enumerated. The order of the physical devices in the pPhysicalDevices array determines the *device index* of each physical device, with element i being assigned a device index of i. Certain commands and structures refer to one or more physical devices by using device indices or *device masks* formed using device indices.

A logical device created without using VkDeviceGroupDeviceCreateInfo, or with physicalDeviceCount equal to zero, is equivalent to a physicalDeviceCount of one and pPhysicalDevices pointing to the physicalDevice parameter to vkCreateDevice. In particular, the device index of that physical device is zero.

Valid Usage

- VUID-VkDeviceGroupDeviceCreateInfo-pPhysicalDevices-00375
  Each element of pPhysicalDevices **must** be unique
- VUID-VkDeviceGroupDeviceCreateInfo-pPhysicalDevices-00376
  All elements of pPhysicalDevices **must** be in the same device group as enumerated by vkEnumeratePhysicalDeviceGroupsKHR
- VUID-VkDeviceGroupDeviceCreateInfo-physicalDeviceCount-00377
  If physicalDeviceCount is not 0, the physicalDevice parameter of vkCreateDevice **must** be an element of pPhysicalDevices

Valid Usage (Implicit)

- VUID-VkDeviceGroupDeviceCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEVICE_GROUP_DEVICE_CREATE_INFO
- VUID-VkDeviceGroupDeviceCreateInfo-pPhysicalDevices-parameter
  If physicalDeviceCount is not 0, pPhysicalDevices **must** be a valid pointer to an array of physicalDeviceCount valid VkPhysicalDevice handles

To register callbacks for underlying device memory events of type VkDeviceMemoryReportEventTypeEXT, add one or multiple VkDeviceDeviceMemoryReportCreateInfoEXT structures to the pNext chain of the VkDeviceCreateInfo structure.

```
// Provided by VK_EXT_device_memory_report
typedef struct VkDeviceDeviceMemoryReportCreateInfoEXT {
    VkStructureType                           sType;
    const void*                               pNext;
    VkDeviceMemoryReportFlagsEXT              flags;
    PFN_vkDeviceMemoryReportCallbackEXT       pfnUserCallback;
    void*                                     pUserData;
} VkDeviceDeviceMemoryReportCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is 0 and reserved for future use.
- pfnUserCallback is the application callback function to call.
- pUserData is user data to be passed to the callback.

The callback **may** be called from multiple threads simultaneously.

The callback **must** be called only once by the implementation when a VkDeviceMemoryReportEventTypeEXT event occurs.

NOTE	The callback could be called from a background thread other than the thread calling the Vulkan commands.

Valid Usage (Implicit)

- VUID-VkDeviceDeviceMemoryReportCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_DEVICE_MEMORY_REPORT_CREATE_INFO_EXT`
- VUID-VkDeviceDeviceMemoryReportCreateInfoEXT-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkDeviceDeviceMemoryReportCreateInfoEXT-pfnUserCallback-parameter
  `pfnUserCallback` **must** be a valid PFN_vkDeviceMemoryReportCallbackEXT value
- VUID-VkDeviceDeviceMemoryReportCreateInfoEXT-pUserData-parameter
  `pUserData` **must** be a pointer value

The prototype for the VkDeviceDeviceMemoryReportCreateInfoEXT::`pfnUserCallback` function implemented by the application is:

```
// Provided by VK_EXT_device_memory_report
typedef void (VKAPI_PTR *PFN_vkDeviceMemoryReportCallbackEXT)(
    const VkDeviceMemoryReportCallbackDataEXT*  pCallbackData,
    void*                                       pUserData);
```

- `pCallbackData` contains all the callback related data in the VkDeviceMemoryReportCallbackDataEXT structure.
- `pUserData` is the user data provided when the VkDeviceDeviceMemoryReportCreateInfoEXT was created.

The callback **must** not make calls to any Vulkan commands.

The definition of `VkDeviceMemoryReportCallbackDataEXT` is:

```
// Provided by VK_EXT_device_memory_report
typedef struct VkDeviceMemoryReportCallbackDataEXT {
    VkStructureType                      sType;
    void*                                pNext;
    VkDeviceMemoryReportFlagsEXT         flags;
    VkDeviceMemoryReportEventTypeEXT     type;
    uint64_t                             memoryObjectId;
    VkDeviceSize                         size;
    VkObjectType                         objectType;
    uint64_t                             objectHandle;
    uint32_t                             heapIndex;
} VkDeviceMemoryReportCallbackDataEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is 0 and reserved for future use.
- `type` is a VkDeviceMemoryReportEventTypeEXT type specifying the type of event reported in this `VkDeviceMemoryReportCallbackDataEXT` structure.
- `memoryObjectId` is the unique id for the underlying memory object as described below.
- `size` is the size of the memory object in bytes. If `type` is `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATE_EXT`, `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_IMPORT_EXT` or `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATION_FAILED_EXT`, `size` is a valid VkDeviceSize value. Otherwise, `size` is undefined.
- `objectType` is a VkObjectType value specifying the type of the object associated with this device memory report event. If `type` is `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATE_EXT`, `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_FREE_EXT`, `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_IMPORT_EXT`, `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_UNIMPORT_EXT` or `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATION_FAILED_EXT`, `objectType` is a valid VkObjectType enum. Otherwise, `objectType` is undefined.
- `objectHandle` is the object this device memory report event is attributed to. If `type` is `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATE_EXT`, `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_FREE_EXT`, `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_IMPORT_EXT` or `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_UNIMPORT_EXT`, `objectHandle` is a valid Vulkan handle of the type associated with `objectType` as defined in the `VkObjectType` and Vulkan Handle Relationship table. Otherwise, `objectHandle` is undefined.
- `heapIndex` describes which memory heap this device memory allocation is made from. If `type` is `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATE_EXT` or `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATION_FAILED_EXT`, `heapIndex` corresponds to one of the valid heaps from the VkPhysicalDeviceMemoryProperties structure. Otherwise, `heapIndex` is undefined.

`memoryObjectId` is used to avoid double-counting on the same memory object.

If an internally-allocated device memory object or a VkDeviceMemory **cannot** be exported,

`memoryObjectId` **must** be unique in the VkDevice.

If an internally-allocated device memory object or a VkDeviceMemory supports being exported, `memoryObjectId` **must** be unique system wide.

If an internal device memory object or a VkDeviceMemory is backed by an imported external memory object, `memoryObjectId` **must** be unique system wide.

Implementor's Note

If the heap backing an internally-allocated device memory **cannot** be used to back VkDeviceMemory, implementations **can** advertise that heap with no types.

NOTE

This structure should only be considered valid during the lifetime of the triggered callback.

For `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATE_EXT` and `VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_IMPORT_EXT` events, `objectHandle` usually will not yet exist when the application or tool receives the callback. `objectHandle` will only exist when the create or allocate call that triggered the event returns, and if the allocation or import ends up failing `objectHandle` will not ever exist.

Valid Usage (Implicit)

- VUID-VkDeviceMemoryReportCallbackDataEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_MEMORY_REPORT_CALLBACK_DATA_EXT`
- VUID-VkDeviceMemoryReportCallbackDataEXT-pNext-pNext
  `pNext` **must** be `NULL`

```
// Provided by VK_EXT_device_memory_report
typedef VkFlags VkDeviceMemoryReportFlagsEXT;
```

`VkDeviceMemoryReportFlagsEXT` is a bitmask type for setting a mask, but is currently reserved for future use.

Possible values of VkDeviceMemoryReportCallbackDataEXT::`type`, specifying event types which cause the device driver to call the callback, are:

```
// Provided by VK_EXT_device_memory_report
typedef enum VkDeviceMemoryReportEventTypeEXT {
    VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATE_EXT = 0,
    VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_FREE_EXT = 1,
    VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_IMPORT_EXT = 2,
    VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_UNIMPORT_EXT = 3,
    VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATION_FAILED_EXT = 4,
} VkDeviceMemoryReportEventTypeEXT;
```

- VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATE_EXT specifies this event corresponds to the allocation of an internal device memory object or a VkDeviceMemory.
- VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_FREE_EXT specifies this event corresponds to the deallocation of an internally-allocated device memory object or a VkDeviceMemory.
- VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_IMPORT_EXT specifies this event corresponds to the import of an external memory object.
- VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_UNIMPORT_EXT specifies this event is the release of an imported external memory object.
- VK_DEVICE_MEMORY_REPORT_EVENT_TYPE_ALLOCATION_FAILED_EXT specifies this event corresponds to the failed allocation of an internal device memory object or a VkDeviceMemory.

To disable the implementation's internal pipeline cache, add a VkDevicePipelineBinaryInternalCacheControlKHR structure to the pNext chain of the VkDeviceCreateInfo structure.

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkDevicePipelineBinaryInternalCacheControlKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           disableInternalCache;
} VkDevicePipelineBinaryInternalCacheControlKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- disableInternalCache specifies whether or not to disable the implementation's internal pipeline cache.

If the VkDeviceCreateInfo::pNext chain does not include this structure, then disableInternalCache defaults to VK_FALSE.

Valid Usage

- VUID-VkDevicePipelineBinaryInternalCacheControlKHR-disableInternalCache-09602
  If VkPhysicalDevicePipelineBinaryPropertiesKHR::pipelineBinaryInternalCacheControl is VK_FALSE, disableInternalCache **must** be VK_FALSE

Valid Usage (Implicit)

- VUID-VkDevicePipelineBinaryInternalCacheControlKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_PIPELINE_BINARY_INTERNAL_CACHE_CONTROL_KHR`

5.2.2. Device Use

The following is a high-level list of `VkDevice` uses along with references on where to find more information:

- Creation of queues. See the Queues section below for further details.
- Creation and tracking of various synchronization constructs. See Synchronization and Cache Control for further details.
- Allocating, freeing, and managing memory. See Memory Allocation and Resource Creation for further details.
- Creation and destruction of command buffers and command buffer pools. See Command Buffers for further details.
- Creation, destruction, and management of graphics state. See Pipelines and Resource Descriptors, among others, for further details.

5.2.3. Lost Device

A logical device **may** become *lost* for a number of implementation-specific reasons, indicating that pending and future command execution **may** fail and cause resources and backing memory to become undefined.

NOTE

Typical reasons for device loss will include things like execution timing out (to prevent denial of service), power management events, platform resource management, implementation errors.

Applications not adhering to valid usage may also result in device loss being reported, however this is not guaranteed. Even if device loss is reported, the system may be in an unrecoverable state, and further usage of the API is still considered invalid.

When this happens, certain commands will return `VK_ERROR_DEVICE_LOST`. After any such event, the logical device is considered *lost*. It is not possible to reset the logical device to a non-lost state, however the lost state is specific to a logical device (`VkDevice`), and the corresponding physical device (`VkPhysicalDevice`) **may** be otherwise unaffected.

In some cases, the physical device **may** also be lost, and attempting to create a new logical device will fail, returning `VK_ERROR_DEVICE_LOST`. This is usually indicative of a problem with the underlying implementation, or its connection to the host. If the physical device has not been lost, and a new logical device is successfully created from that physical device, it **must** be in the non-lost state.

NOTE

Whilst logical device loss **may** be recoverable, in the case of physical device loss, it

> is unlikely that an application will be able to recover unless additional, unaffected physical devices exist on the system. The error is largely informational and intended only to inform the application that a platform issue has occurred, and **should** be investigated further. For example, underlying hardware **may** have developed a fault or become physically disconnected from the rest of the system. In many cases, physical device loss **may** cause other more serious issues such as the operating system crashing; in which case it **may** not be reported via the Vulkan API.

When a device is lost, its child objects are not implicitly destroyed and their handles are still valid. Those objects **must** still be destroyed before their parents or the device **can** be destroyed (see the Object Lifetime section). The host address space corresponding to device memory mapped using vkMapMemory is still valid, and host memory accesses to these mapped regions are still valid, but the contents are undefined. It is still legal to call any API command on the device and child objects.

Once a device is lost, command execution **may** fail, and certain commands that return a VkResult **may** return `VK_ERROR_DEVICE_LOST`. These commands can be identified by the inclusion of `VK_ERROR_DEVICE_LOST` in the Return Codes section for each command. Commands that do not allow runtime errors **must** still operate correctly for valid usage and, if applicable, return valid data.

Commands that wait indefinitely for device execution (namely vkDeviceWaitIdle, vkQueueWaitIdle, vkWaitForFences or vkAcquireNextImageKHR with a maximum `timeout`, and vkGetQueryPoolResults with the `VK_QUERY_RESULT_WAIT_BIT` bit set in `flags`) **must** return in finite time even in the case of a lost device, and return either `VK_SUCCESS` or `VK_ERROR_DEVICE_LOST`. For any command that **may** return `VK_ERROR_DEVICE_LOST`, for the purpose of determining whether a command buffer is in the pending state, or whether resources are considered in-use by the device, a return value of `VK_ERROR_DEVICE_LOST` is equivalent to `VK_SUCCESS`.

If a device was created with the `maintenance5` feature enabled, and any device command returns `VK_ERROR_DEVICE_LOST`, then all device commands for which `VK_ERROR_DEVICE_LOST` is a valid return value and which happen-after it on the same host thread **must** return `VK_ERROR_DEVICE_LOST`.

Device commands executing on other threads **must** begin returning `VK_ERROR_DEVICE_LOST` within finite time.

The content of any external memory objects that have been exported from or imported to a lost device become undefined. Objects on other logical devices or in other APIs which are associated with the same underlying memory resource as the external memory objects on the lost device are unaffected other than their content becoming undefined. The layout of subresources of images on other logical devices that are bound to `VkDeviceMemory` objects associated with the same underlying memory resources as external memory objects on the lost device becomes `VK_IMAGE_LAYOUT_UNDEFINED`.

The state of `VkSemaphore` objects on other logical devices created by importing a semaphore payload with temporary permanence which was exported from the lost device is undefined. The state of `VkSemaphore` objects on other logical devices that permanently share a semaphore payload with a `VkSemaphore` object on the lost device is undefined, and remains undefined following any subsequent signal operations. Implementations **must** ensure pending and subsequently submitted wait operations on such semaphores behave as defined in Semaphore State Requirements For Wait Operations for external semaphores not in a valid state for a wait operation.

5.2.4. Device Destruction

To destroy a device, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyDevice(
    VkDevice                                    device,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

To ensure that no work is active on the device, vkDeviceWaitIdle **can** be used to gate the destruction of the device. Prior to destroying a device, an application is responsible for destroying/freeing any Vulkan objects that were created using that device as the first parameter of the corresponding `vkCreate*` or `vkAllocate*` command.

NOTE	The lifetime of each of these objects is bound by the lifetime of the `VkDevice` object. Therefore, to avoid resource leaks, it is critical that an application explicitly free all of these resources prior to calling `vkDestroyDevice`.

Valid Usage

- VUID-vkDestroyDevice-device-05137
  All child objects created on `device` **must** have been destroyed prior to destroying `device`
- VUID-vkDestroyDevice-device-00379
  If `VkAllocationCallbacks` were provided when `device` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyDevice-device-00380
  If no `VkAllocationCallbacks` were provided when `device` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyDevice-device-parameter
  If `device` is not `NULL`, `device` **must** be a valid VkDevice handle
- VUID-vkDestroyDevice-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure

Host Synchronization

- Host access to `device` **must** be externally synchronized

- Host access to all VkQueue objects created from device **must** be externally synchronized

5.3. Queues

5.3.1. Queue Family Properties

As discussed in the Physical Device Enumeration section above, the vkGetPhysicalDeviceQueueFamilyProperties command is used to retrieve details about the queue families and queues supported by a device.

Each index in the pQueueFamilyProperties array returned by vkGetPhysicalDeviceQueueFamilyProperties describes a unique queue family on that physical device. These indices are used when creating queues, and they correspond directly with the queueFamilyIndex that is passed to the vkCreateDevice command via the VkDeviceQueueCreateInfo structure as described in the Queue Creation section below.

Grouping of queue families within a physical device is implementation-dependent.

NOTE	The general expectation is that a physical device groups all queues of matching capabilities into a single family. However, while implementations **should** do this, it is possible that a physical device **may** return two separate queue families with the same capabilities.

Once an application has identified a physical device with the queue(s) that it desires to use, it will create those queues in conjunction with a logical device. This is described in the following section.

5.3.2. Queue Creation

Creating a logical device also creates the queues associated with that device. The queues to create are described by a set of VkDeviceQueueCreateInfo structures that are passed to vkCreateDevice in pQueueCreateInfos.

Queues are represented by VkQueue handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_HANDLE(VkQueue)
```

The VkDeviceQueueCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDeviceQueueCreateInfo {
    VkStructureType             sType;
    const void*                 pNext;
    VkDeviceQueueCreateFlags    flags;
    uint32_t                    queueFamilyIndex;
    uint32_t                    queueCount;
    const float*                pQueuePriorities;
} VkDeviceQueueCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `queueFamilyIndex` is an unsigned integer indicating the index of the queue family in which to create the queues on this device. This index corresponds to the index of an element of the `pQueueFamilyProperties` array that was returned by `vkGetPhysicalDeviceQueueFamilyProperties`.
- `queueCount` is an unsigned integer specifying the number of queues to create in the queue family indicated by `queueFamilyIndex`, and with the behavior specified by `flags`.
- `pQueuePriorities` is a pointer to an array of `queueCount` normalized floating-point values, specifying priorities of work that will be submitted to each created queue. See Queue Priority for more information.

Valid Usage

- VUID-VkDeviceQueueCreateInfo-queueFamilyIndex-00381
  `queueFamilyIndex` **must** be less than `pQueueFamilyPropertyCount` returned by `vkGetPhysicalDeviceQueueFamilyProperties`
- VUID-VkDeviceQueueCreateInfo-queueCount-00382
  `queueCount` **must** be less than or equal to the `queueCount` member of the `VkQueueFamilyProperties` structure, as returned by `vkGetPhysicalDeviceQueueFamilyProperties` in the `pQueueFamilyProperties`[queueFamilyIndex]
- VUID-VkDeviceQueueCreateInfo-pQueuePriorities-00383
  Each element of `pQueuePriorities` **must** be between `0.0` and `1.0` inclusive

Valid Usage (Implicit)

- VUID-VkDeviceQueueCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO`
- VUID-VkDeviceQueueCreateInfo-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkDeviceQueueGlobalPriorityCreateInfoKHR

- VUID-VkDeviceQueueCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkDeviceQueueCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkDeviceQueueCreateInfo-pQueuePriorities-parameter
  `pQueuePriorities` **must** be a valid pointer to an array of `queueCount` `float` values
- VUID-VkDeviceQueueCreateInfo-queueCount-arraylength
  `queueCount` **must** be greater than `0`

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkDeviceQueueCreateFlags;
```

`VkDeviceQueueCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

Queues **can** be created with a system-wide priority by adding a `VkDeviceQueueGlobalPriorityCreateInfoKHR` structure to the `pNext` chain of VkDeviceQueueCreateInfo.

The `VkDeviceQueueGlobalPriorityCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_global_priority
typedef struct VkDeviceQueueGlobalPriorityCreateInfoKHR {
    VkStructureType             sType;
    const void*                 pNext;
    VkQueueGlobalPriorityKHR    globalPriority;
} VkDeviceQueueGlobalPriorityCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `globalPriority` is the system-wide priority associated to these queues as specified by VkQueueGlobalPriorityKHR

Queues created without specifying `VkDeviceQueueGlobalPriorityCreateInfoKHR` will default to `VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_KHR`.

Valid Usage (Implicit)

- VUID-VkDeviceQueueGlobalPriorityCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_QUEUE_GLOBAL_PRIORITY_CREATE_INFO_KHR`
- VUID-VkDeviceQueueGlobalPriorityCreateInfoKHR-globalPriority-parameter
  `globalPriority` **must** be a valid VkQueueGlobalPriorityKHR value

Possible values of VkDeviceQueueGlobalPriorityCreateInfoKHR::globalPriority, specifying a system-wide priority level are:

```
// Provided by VK_KHR_global_priority
typedef enum VkQueueGlobalPriorityKHR {
    VK_QUEUE_GLOBAL_PRIORITY_LOW_KHR = 128,
    VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_KHR = 256,
    VK_QUEUE_GLOBAL_PRIORITY_HIGH_KHR = 512,
    VK_QUEUE_GLOBAL_PRIORITY_REALTIME_KHR = 1024,
    VK_QUEUE_GLOBAL_PRIORITY_LOW_EXT = VK_QUEUE_GLOBAL_PRIORITY_LOW_KHR,
    VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_EXT = VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_KHR,
    VK_QUEUE_GLOBAL_PRIORITY_HIGH_EXT = VK_QUEUE_GLOBAL_PRIORITY_HIGH_KHR,
    VK_QUEUE_GLOBAL_PRIORITY_REALTIME_EXT = VK_QUEUE_GLOBAL_PRIORITY_REALTIME_KHR,
} VkQueueGlobalPriorityKHR;
```

Priority values are sorted in ascending order. A comparison operation on the enum values can be used to determine the priority order.

- VK_QUEUE_GLOBAL_PRIORITY_LOW_KHR is below the system default. Useful for non-interactive tasks.
- VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_KHR is the system default priority.
- VK_QUEUE_GLOBAL_PRIORITY_HIGH_KHR is above the system default.
- VK_QUEUE_GLOBAL_PRIORITY_REALTIME_KHR is the highest priority. Useful for critical tasks.

Queues with higher system priority **may** be allotted more processing time than queues with lower priority. An implementation **may** allow a higher-priority queue to starve a lower-priority queue until the higher-priority queue has no further commands to execute.

Priorities imply no ordering or scheduling constraints.

No specific guarantees are made about higher priority queues receiving more processing time or better quality of service than lower priority queues.

The global priority level of a queue takes precedence over the per-process queue priority (VkDeviceQueueCreateInfo::pQueuePriorities).

Abuse of this feature **may** result in starving the rest of the system of implementation resources. Therefore, the driver implementation **may** deny requests to acquire a priority above the default priority (VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_KHR) if the caller does not have sufficient privileges. In this scenario VK_ERROR_NOT_PERMITTED_KHR is returned.

The driver implementation **may** fail the queue allocation request if resources required to complete the operation have been exhausted (either by the same process or a different process). In this scenario VK_ERROR_INITIALIZATION_FAILED is returned.

If the globalPriorityQuery feature is enabled and the requested global priority is not reported via VkQueueFamilyGlobalPriorityPropertiesKHR, the driver implementation **must** fail the queue creation. In this scenario, VK_ERROR_INITIALIZATION_FAILED is returned.

To retrieve a handle to a VkQueue object, call:

```
// Provided by VK_VERSION_1_0
void vkGetDeviceQueue(
    VkDevice                                    device,
    uint32_t                                    queueFamilyIndex,
    uint32_t                                    queueIndex,
    VkQueue*                                    pQueue);
```

- `device` is the logical device that owns the queue.
- `queueFamilyIndex` is the index of the queue family to which the queue belongs.
- `queueIndex` is the index within this queue family of the queue to retrieve.
- `pQueue` is a pointer to a VkQueue object that will be filled with the handle for the requested queue.

Valid Usage

- VUID-vkGetDeviceQueue-queueFamilyIndex-00384
  `queueFamilyIndex` **must** be one of the queue family indices specified when `device` was created, via the VkDeviceQueueCreateInfo structure
- VUID-vkGetDeviceQueue-queueIndex-00385
  `queueIndex` **must** be less than the value of VkDeviceQueueCreateInfo::`queueCount` for the queue family indicated by `queueFamilyIndex` when `device` was created
- VUID-vkGetDeviceQueue-flags-01841
  VkDeviceQueueCreateInfo::`flags` **must** have been zero when `device` was created

Valid Usage (Implicit)

- VUID-vkGetDeviceQueue-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceQueue-pQueue-parameter
  `pQueue` **must** be a valid pointer to a VkQueue handle

5.3.3. Queue Family Index

The queue family index is used in multiple places in Vulkan in order to tie operations to a specific family of queues.

When retrieving a handle to the queue via `vkGetDeviceQueue`, the queue family index is used to select which queue family to retrieve the `VkQueue` handle from as described in the previous section.

When creating a `VkCommandPool` object (see Command Pools), a queue family index is specified in the VkCommandPoolCreateInfo structure. Command buffers from this pool **can** only be submitted on

queues corresponding to this queue family.

When creating VkImage (see Images) and VkBuffer (see Buffers) resources, a set of queue families is included in the VkImageCreateInfo and VkBufferCreateInfo structures to specify the queue families that **can** access the resource.

When inserting a VkBufferMemoryBarrier or VkImageMemoryBarrier (see Pipeline Barriers), a source and destination queue family index is specified to allow the ownership of a buffer or image to be transferred from one queue family to another. See the Resource Sharing section for details.

5.3.4. Queue Priority

Each queue is assigned a priority, as set in the VkDeviceQueueCreateInfo structures when creating the device. The priority of each queue is a normalized floating-point value between 0.0 and 1.0, which is then translated to a discrete priority level by the implementation. Higher values indicate a higher priority, with 0.0 being the lowest priority and 1.0 being the highest.

Within the same device, queues with higher priority **may** be allotted more processing time than queues with lower priority. The implementation makes no guarantees with regards to ordering or scheduling among queues with the same priority, other than the constraints defined by any explicit synchronization primitives. The implementation makes no guarantees with regards to queues across different devices.

An implementation **may** allow a higher-priority queue to starve a lower-priority queue on the same VkDevice until the higher-priority queue has no further commands to execute. The relationship of queue priorities **must** not cause queues on one VkDevice to starve queues on another VkDevice.

No specific guarantees are made about higher priority queues receiving more processing time or better quality of service than lower priority queues.

5.3.5. Queue Submission

Work is submitted to a queue via *queue submission* commands such as vkQueueSubmit2KHR or vkQueueSubmit. Queue submission commands define a set of *queue operations* to be executed by the underlying physical device, including synchronization with semaphores and fences.

Submission commands take as parameters a target queue, zero or more *batches* of work, and an **optional** fence to signal upon completion. Each batch consists of three distinct parts:

1. Zero or more semaphores to wait on before execution of the rest of the batch.
    - If present, these describe a semaphore wait operation.
2. Zero or more work items to execute.
    - If present, these describe a *queue operation* matching the work described.
3. Zero or more semaphores to signal upon completion of the work items.
    - If present, these describe a semaphore signal operation.

If a fence is present in a queue submission, it describes a fence signal operation.

All work described by a queue submission command **must** be submitted to the queue before the command returns.

Sparse Memory Binding

In Vulkan it is possible to sparsely bind memory to buffers and images as described in the Sparse Resource chapter. Sparse memory binding is a queue operation. A queue whose flags include the `VK_QUEUE_SPARSE_BINDING_BIT` **must** be able to support the mapping of a virtual address to a physical address on the device. This causes an update to the page table mappings on the device. This update **must** be synchronized on a queue to avoid corrupting page table mappings during execution of graphics commands. By binding the sparse memory resources on queues, all commands that are dependent on the updated bindings are synchronized to only execute after the binding is updated. See the Synchronization and Cache Control chapter for how this synchronization is accomplished.

5.3.6. Queue Destruction

Queues are created along with a logical device during `vkCreateDevice`. All queues associated with a logical device are destroyed when `vkDestroyDevice` is called on that device.

Chapter 6. Command Buffers

Command buffers are objects used to record commands which **can** be subsequently submitted to a device queue for execution. There are two levels of command buffers - *primary command buffers*, which **can** execute secondary command buffers, and which are submitted to queues, and *secondary command buffers*, which **can** be executed by primary command buffers, and which are not directly submitted to queues.

Command buffers are represented by `VkCommandBuffer` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_HANDLE(VkCommandBuffer)
```

Recorded commands include commands to bind pipelines and descriptor sets to the command buffer, commands to modify dynamic state, commands to draw (for graphics rendering), commands to dispatch (for compute), commands to execute secondary command buffers (for primary command buffers only), commands to copy buffers and images, and other commands.

Each command buffer manages state independently of other command buffers. There is no inheritance of state across primary and secondary command buffers, or between secondary command buffers. When a command buffer begins recording, all state in that command buffer is undefined. When secondary command buffer(s) are recorded to execute on a primary command buffer, the secondary command buffer inherits no state from the primary command buffer, and all state of the primary command buffer is undefined after an execute secondary command buffer command is recorded. There is one exception to this rule - if the primary command buffer is inside a render pass instance, then the render pass and subpass state is not disturbed by executing secondary command buffers. For state dependent commands (such as draws and dispatches), any state consumed by those commands **must** not be undefined.

Unless otherwise specified, and without explicit synchronization, the various commands submitted to a queue via command buffers **may** execute in arbitrary order relative to each other, and/or concurrently. Also, the memory side effects of those commands **may** not be directly visible to other commands without explicit memory dependencies. This is true within a command buffer, and across command buffers submitted to a given queue. See the synchronization chapter for information on implicit and explicit synchronization between commands.

6.1. Command Buffer Lifecycle

Each command buffer is always in one of the following states:

Initial

When a command buffer is allocated, it is in the *initial state*. Some commands are able to *reset* a command buffer (or a set of command buffers) back to this state from any of the executable, recording or invalid state. Command buffers in the initial state **can** only be moved to the recording state, or freed.

Recording

vkBeginCommandBuffer changes the state of a command buffer from the initial state to the *recording state*. Once a command buffer is in the recording state, `vkCmd*` commands **can** be used to record to the command buffer.

Executable

vkEndCommandBuffer ends the recording of a command buffer, and moves it from the recording state to the *executable state*. Executable command buffers **can** be submitted, reset, or recorded to another command buffer.

Pending

Queue submission of a command buffer changes the state of a command buffer from the executable state to the *pending state*. Whilst in the pending state, applications **must** not attempt to modify the command buffer in any way - as the device **may** be processing the commands recorded to it. Once execution of a command buffer completes, the command buffer either reverts back to the *executable state*, or if it was recorded with `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT`, it moves to the *invalid state*. A synchronization command **should** be used to detect when this occurs.

Invalid

Some operations, such as modifying or deleting a resource that was used in a command recorded to a command buffer, will transition the state of that command buffer into the *invalid state*. Command buffers in the invalid state **can** only be reset or freed.

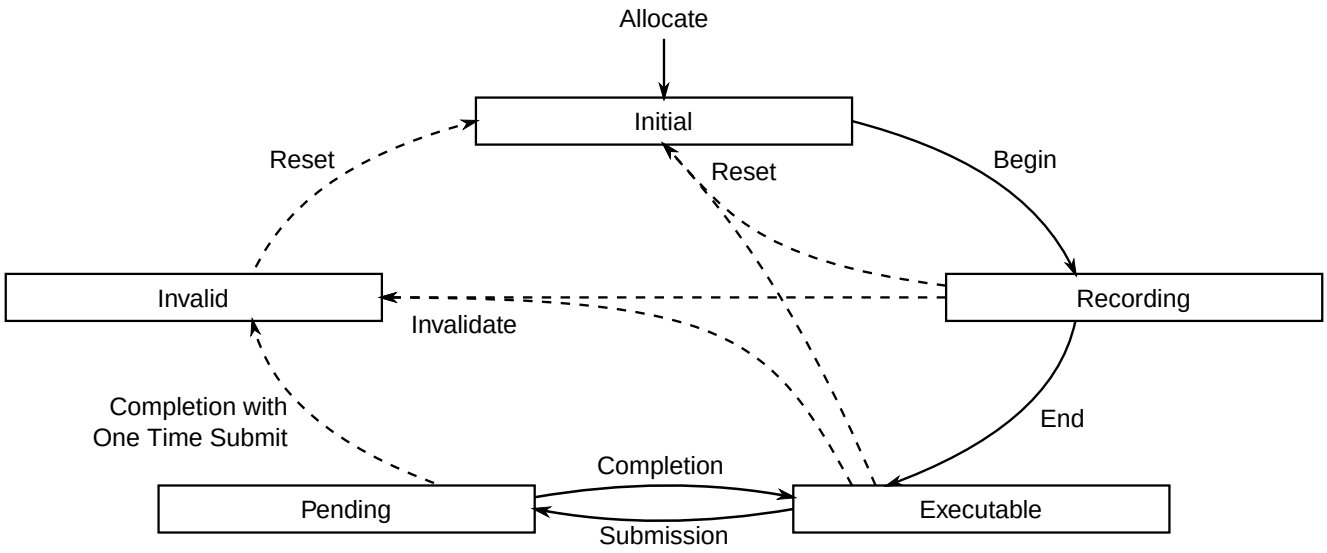


Figure 1. Lifecycle of a command buffer

Any given command that operates on a command buffer has its own requirements on what state a command buffer **must** be in, which are detailed in the valid usage constraints for that command.

Resetting a command buffer is an operation that discards any previously recorded commands and puts a command buffer in the *initial state*. Resetting occurs as a result of vkResetCommandBuffer or vkResetCommandPool, or as part of vkBeginCommandBuffer (which additionally puts the command buffer in the *recording state*).

Secondary command buffers **can** be recorded to a primary command buffer via

vkCmdExecuteCommands. This partially ties the lifecycle of the two command buffers together - if the primary is submitted to a queue, both the primary and any secondaries recorded to it move to the *pending state*. Once execution of the primary completes, so it does for any secondary recorded within it. After all executions of each command buffer complete, they each move to their appropriate completion state (either to the *executable state* or the *invalid state*, as specified above).

If a secondary moves to the *invalid state* or the *initial state*, then all primary buffers it is recorded in move to the *invalid state*. A primary moving to any other state does not affect the state of a secondary recorded in it.

NOTE	Resetting or freeing a primary command buffer removes the lifecycle linkage to all secondary command buffers that were recorded into it.

6.2. Command Pools

Command pools are opaque objects that command buffer memory is allocated from, and which allow the implementation to amortize the cost of resource creation across multiple command buffers. Command pools are externally synchronized, meaning that a command pool **must** not be used concurrently in multiple threads. That includes use via recording commands on any command buffers allocated from the pool, as well as operations that allocate, free, and reset command buffers or the pool itself.

Command pools are represented by `VkCommandPool` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkCommandPool)
```

To create a command pool, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateCommandPool(
    VkDevice                                    device,
    const VkCommandPoolCreateInfo*              pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkCommandPool*                              pCommandPool);
```

- `device` is the logical device that creates the command pool.
- `pCreateInfo` is a pointer to a VkCommandPoolCreateInfo structure specifying the state of the command pool object.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pCommandPool` is a pointer to a VkCommandPool handle in which the created pool is returned.

Valid Usage

- VUID-vkCreateCommandPool-queueFamilyIndex-01937

`pCreateInfo->queueFamilyIndex` **must** be the index of a queue family available in the logical device `device`

Valid Usage (Implicit)

- VUID-vkCreateCommandPool-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateCommandPool-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkCommandPoolCreateInfo structure
- VUID-vkCreateCommandPool-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateCommandPool-pCommandPool-parameter
  `pCommandPool` **must** be a valid pointer to a VkCommandPool handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkCommandPoolCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkCommandPoolCreateInfo {
    VkStructureType             sType;
    const void*                 pNext;
    VkCommandPoolCreateFlags    flags;
    uint32_t                    queueFamilyIndex;
} VkCommandPoolCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkCommandPoolCreateFlagBits indicating usage behavior for the pool and command buffers allocated from it.
- `queueFamilyIndex` designates a queue family as described in section Queue Family Properties. All command buffers allocated from this command pool **must** be submitted on queues from the same queue family.

Valid Usage

Valid Usage (Implicit)

- VUID-VkCommandPoolCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO`
- VUID-VkCommandPoolCreateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCommandPoolCreateInfo-flags-parameter
  `flags` **must** be a valid combination of VkCommandPoolCreateFlagBits values

Bits which **can** be set in VkCommandPoolCreateInfo::`flags`, specifying usage behavior for a command pool, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkCommandPoolCreateFlagBits {
    VK_COMMAND_POOL_CREATE_TRANSIENT_BIT = 0x00000001,
    VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT = 0x00000002,
} VkCommandPoolCreateFlagBits;
```

- `VK_COMMAND_POOL_CREATE_TRANSIENT_BIT` specifies that command buffers allocated from the pool will be short-lived, meaning that they will be reset or freed in a relatively short timeframe. This flag **may** be used by the implementation to control memory allocation behavior within the pool.
- `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT` allows any command buffer allocated from a pool to be individually reset to the initial state; either by calling vkResetCommandBuffer, or via the implicit reset when calling vkBeginCommandBuffer. If this flag is not set on a pool, then `vkResetCommandBuffer` **must** not be called for any command buffer allocated from that pool.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkCommandPoolCreateFlags;
```

`VkCommandPoolCreateFlags` is a bitmask type for setting a mask of zero or more VkCommandPoolCreateFlagBits.

To trim a command pool, call:

```
// Provided by VK_KHR_maintenance1
void vkTrimCommandPoolKHR(
    VkDevice                                    device,
    VkCommandPool                               commandPool,
    VkCommandPoolTrimFlags                      flags);
```

- `device` is the logical device that owns the command pool.
- `commandPool` is the command pool to trim.
- `flags` is reserved for future use.

Trimming a command pool recycles unused memory from the command pool back to the system. Command buffers allocated from the pool are not affected by the command.

NOTE

This command provides applications with some control over the internal memory allocations used by command pools.

Unused memory normally arises from command buffers that have been recorded and later reset, such that they are no longer using the memory. On reset, a command buffer can return memory to its command pool, but the only way to release memory from a command pool to the system requires calling vkResetCommandPool, which cannot be executed while any command buffers from that pool are still in use. Subsequent recording operations into command buffers will reuse this memory but since total memory requirements fluctuate over time, unused memory can accumulate.

In this situation, trimming a command pool **may** be useful to return unused memory back to the system, returning the total outstanding memory allocated by the pool back to a more “average” value.

Implementations utilize many internal allocation strategies that make it impossible to guarantee that all unused memory is released back to the system. For instance, an implementation of a command pool **may** involve allocating memory in bulk from the system and sub-allocating from that memory. In such an implementation any live command buffer that holds a reference to a bulk allocation would prevent that allocation from being freed, even if only a small proportion of the bulk allocation is in use.

In most cases trimming will result in a reduction in allocated but unused memory, but it does not guarantee the “ideal” behavior.

Trimming **may** be an expensive operation, and **should** not be called frequently. Trimming **should** be treated as a way to relieve memory pressure after application-known points when there exists enough unused memory that the cost of trimming is “worth” it.

Valid Usage (Implicit)

- VUID-vkTrimCommandPool-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkTrimCommandPool-commandPool-parameter
  `commandPool` **must** be a valid VkCommandPool handle
- VUID-vkTrimCommandPool-flags-zerobitmask
  `flags` **must** be `0`

- VUID-vkTrimCommandPool-commandPool-parent
  `commandPool` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `commandPool` **must** be externally synchronized

```
typedef VkFlags VkCommandPoolTrimFlags;
```

or the equivalent

```
// Provided by VK_KHR_maintenance1
typedef VkCommandPoolTrimFlags VkCommandPoolTrimFlagsKHR;
```

`VkCommandPoolTrimFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

To reset a command pool, call:

```
// Provided by VK_VERSION_1_0
VkResult vkResetCommandPool(
    VkDevice                                    device,
    VkCommandPool                               commandPool,
    VkCommandPoolResetFlags                     flags);
```

- `device` is the logical device that owns the command pool.
- `commandPool` is the command pool to reset.
- `flags` is a bitmask of VkCommandPoolResetFlagBits controlling the reset operation.

Resetting a command pool recycles all of the resources from all of the command buffers allocated from the command pool back to the command pool. All command buffers that have been allocated from the command pool are put in the initial state.

Any primary command buffer allocated from another VkCommandPool that is in the recording or executable state and has a secondary command buffer allocated from `commandPool` recorded into it, becomes invalid.

Valid Usage

- VUID-vkResetCommandPool-commandPool-00040
  All `VkCommandBuffer` objects allocated from `commandPool` **must** not be in the pending state

Valid Usage (Implicit)

- VUID-vkResetCommandPool-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkResetCommandPool-commandPool-parameter
  `commandPool` **must** be a valid VkCommandPool handle
- VUID-vkResetCommandPool-flags-parameter
  `flags` **must** be a valid combination of VkCommandPoolResetFlagBits values
- VUID-vkResetCommandPool-commandPool-parent
  `commandPool` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `commandPool` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Bits which **can** be set in vkResetCommandPool::`flags`, controlling the reset operation, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkCommandPoolResetFlagBits {
    VK_COMMAND_POOL_RESET_RELEASE_RESOURCES_BIT = 0x00000001,
} VkCommandPoolResetFlagBits;
```

- `VK_COMMAND_POOL_RESET_RELEASE_RESOURCES_BIT` specifies that resetting a command pool recycles all of the resources from the command pool back to the system.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkCommandPoolResetFlags;
```

`VkCommandPoolResetFlags` is a bitmask type for setting a mask of zero or more VkCommandPoolResetFlagBits.

To destroy a command pool, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyCommandPool(
    VkDevice                                    device,
    VkCommandPool                               commandPool,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the command pool.
- `commandPool` is the handle of the command pool to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

When a pool is destroyed, all command buffers allocated from the pool are freed.

Any primary command buffer allocated from another VkCommandPool that is in the recording or executable state and has a secondary command buffer allocated from `commandPool` recorded into it, becomes invalid.

Valid Usage

- VUID-vkDestroyCommandPool-commandPool-00041
  All `VkCommandBuffer` objects allocated from `commandPool` **must** not be in the pending state
- VUID-vkDestroyCommandPool-commandPool-00042
  If `VkAllocationCallbacks` were provided when `commandPool` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyCommandPool-commandPool-00043
  If no `VkAllocationCallbacks` were provided when `commandPool` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyCommandPool-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyCommandPool-commandPool-parameter
  If `commandPool` is not VK_NULL_HANDLE, `commandPool` **must** be a valid VkCommandPool handle
- VUID-vkDestroyCommandPool-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyCommandPool-commandPool-parent
  If `commandPool` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `commandPool` **must** be externally synchronized

6.3. Command Buffer Allocation and Management

To allocate command buffers, call:

```
// Provided by VK_VERSION_1_0
VkResult vkAllocateCommandBuffers(
    VkDevice                                    device,
    const VkCommandBufferAllocateInfo*          pAllocateInfo,
    VkCommandBuffer*                            pCommandBuffers);
```

- `device` is the logical device that owns the command pool.
- `pAllocateInfo` is a pointer to a VkCommandBufferAllocateInfo structure describing parameters of the allocation.
- `pCommandBuffers` is a pointer to an array of VkCommandBuffer handles in which the resulting command buffer objects are returned. The array **must** be at least the length specified by the `commandBufferCount` member of `pAllocateInfo`. Each allocated command buffer begins in the initial state.

`vkAllocateCommandBuffers` **can** be used to allocate multiple command buffers. If the allocation of any of those command buffers fails, the implementation **must** free all successfully allocated command buffer objects from this command, set all entries of the `pCommandBuffers` array to `NULL` and return the error.

NOTE Filling `pCommandBuffers` with `NULL` values on failure is an exception to the default error behavior that output parameters will have undefined contents.

When command buffers are first allocated, they are in the initial state.

Valid Usage (Implicit)

- VUID-vkAllocateCommandBuffers-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkAllocateCommandBuffers-pAllocateInfo-parameter
  `pAllocateInfo` **must** be a valid pointer to a valid VkCommandBufferAllocateInfo structure
- VUID-vkAllocateCommandBuffers-pCommandBuffers-parameter
  `pCommandBuffers` **must** be a valid pointer to an array of `pAllocateInfo->commandBufferCount` VkCommandBuffer handles
- VUID-vkAllocateCommandBuffers-pAllocateInfo::commandBufferCount-arraylength
  `pAllocateInfo->commandBufferCount` **must** be greater than `0`

Host Synchronization

- Host access to `pAllocateInfo->commandPool` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkCommandBufferAllocateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkCommandBufferAllocateInfo {
    VkStructureType         sType;
    const void*             pNext;
    VkCommandPool           commandPool;
    VkCommandBufferLevel    level;
    uint32_t                commandBufferCount;
} VkCommandBufferAllocateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `commandPool` is the command pool from which the command buffers are allocated.
- `level` is a VkCommandBufferLevel value specifying the command buffer level.
- `commandBufferCount` is the number of command buffers to allocate from the pool.

Valid Usage (Implicit)

- VUID-VkCommandBufferAllocateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO`
- VUID-VkCommandBufferAllocateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCommandBufferAllocateInfo-commandPool-parameter
  `commandPool` **must** be a valid VkCommandPool handle
- VUID-VkCommandBufferAllocateInfo-level-parameter
  `level` **must** be a valid VkCommandBufferLevel value

Possible values of VkCommandBufferAllocateInfo::`level`, specifying the command buffer level, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkCommandBufferLevel {
    VK_COMMAND_BUFFER_LEVEL_PRIMARY = 0,
    VK_COMMAND_BUFFER_LEVEL_SECONDARY = 1,
} VkCommandBufferLevel;
```

- `VK_COMMAND_BUFFER_LEVEL_PRIMARY` specifies a primary command buffer.
- `VK_COMMAND_BUFFER_LEVEL_SECONDARY` specifies a secondary command buffer.

To reset a command buffer, call:

```
// Provided by VK_VERSION_1_0
VkResult vkResetCommandBuffer(
    VkCommandBuffer                             commandBuffer,
    VkCommandBufferResetFlags                   flags);
```

- `commandBuffer` is the command buffer to reset. The command buffer **can** be in any state other than pending, and is moved into the initial state.
- `flags` is a bitmask of VkCommandBufferResetFlagBits controlling the reset operation.

Any primary command buffer that is in the recording or executable state and has `commandBuffer` recorded into it, becomes invalid.

Valid Usage

- VUID-vkResetCommandBuffer-commandBuffer-00045
  `commandBuffer` **must** not be in the pending state
- VUID-vkResetCommandBuffer-commandBuffer-00046
  `commandBuffer` **must** have been allocated from a pool that was created with the `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT`

Valid Usage (Implicit)

- VUID-vkResetCommandBuffer-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkResetCommandBuffer-flags-parameter
  `flags` **must** be a valid combination of VkCommandBufferResetFlagBits values

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized

- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_DEVICE_MEMORY

Bits which **can** be set in vkResetCommandBuffer::flags, controlling the reset operation, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkCommandBufferResetFlagBits {
    VK_COMMAND_BUFFER_RESET_RELEASE_RESOURCES_BIT = 0x00000001,
} VkCommandBufferResetFlagBits;
```

- VK_COMMAND_BUFFER_RESET_RELEASE_RESOURCES_BIT specifies that most or all memory resources currently owned by the command buffer **should** be returned to the parent command pool. If this flag is not set, then the command buffer **may** hold onto memory resources and reuse them when recording commands. commandBuffer is moved to the initial state.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkCommandBufferResetFlags;
```

VkCommandBufferResetFlags is a bitmask type for setting a mask of zero or more VkCommandBufferResetFlagBits.

To free command buffers, call:

```
// Provided by VK_VERSION_1_0
void vkFreeCommandBuffers(
    VkDevice                                    device,
    VkCommandPool                               commandPool,
    uint32_t                                    commandBufferCount,
    const VkCommandBuffer*                      pCommandBuffers);
```

- device is the logical device that owns the command pool.
- commandPool is the command pool from which the command buffers were allocated.
- commandBufferCount is the length of the pCommandBuffers array.
- pCommandBuffers is a pointer to an array of handles of command buffers to free.

Any primary command buffer that is in the recording or executable state and has any element of

`pCommandBuffers` recorded into it, becomes invalid.

Valid Usage

- VUID-vkFreeCommandBuffers-pCommandBuffers-00047
  All elements of `pCommandBuffers` **must** not be in the pending state
- VUID-vkFreeCommandBuffers-pCommandBuffers-00048
  `pCommandBuffers` **must** be a valid pointer to an array of `commandBufferCount` `VkCommandBuffer` handles, each element of which **must** either be a valid handle or `NULL`

Valid Usage (Implicit)

- VUID-vkFreeCommandBuffers-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkFreeCommandBuffers-commandPool-parameter
  `commandPool` **must** be a valid VkCommandPool handle
- VUID-vkFreeCommandBuffers-commandBufferCount-arraylength
  `commandBufferCount` **must** be greater than `0`
- VUID-vkFreeCommandBuffers-commandPool-parent
  `commandPool` **must** have been created, allocated, or retrieved from `device`
- VUID-vkFreeCommandBuffers-pCommandBuffers-parent
  Each element of `pCommandBuffers` that is a valid handle **must** have been created, allocated, or retrieved from `commandPool`

Host Synchronization

- Host access to `commandPool` **must** be externally synchronized
- Host access to each member of `pCommandBuffers` **must** be externally synchronized

6.4. Command Buffer Recording

To begin recording a command buffer, call:

```
// Provided by VK_VERSION_1_0
VkResult vkBeginCommandBuffer(
    VkCommandBuffer                             commandBuffer,
    const VkCommandBufferBeginInfo*             pBeginInfo);
```

- `commandBuffer` is the handle of the command buffer which is to be put in the recording state.
- `pBeginInfo` is a pointer to a VkCommandBufferBeginInfo structure defining additional information about how the command buffer begins recording.

Valid Usage

- VUID-vkBeginCommandBuffer-commandBuffer-00049
  `commandBuffer` **must** not be in the recording or pending state
- VUID-vkBeginCommandBuffer-commandBuffer-00050
  If `commandBuffer` was allocated from a VkCommandPool which did not have the `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT` flag set, `commandBuffer` **must** be in the initial state
- VUID-vkBeginCommandBuffer-commandBuffer-00051
  If `commandBuffer` is a secondary command buffer, the `pInheritanceInfo` member of `pBeginInfo` **must** be a valid `VkCommandBufferInheritanceInfo` structure
- VUID-vkBeginCommandBuffer-commandBuffer-00052
  If `commandBuffer` is a secondary command buffer and either the `occlusionQueryEnable` member of the `pInheritanceInfo` member of `pBeginInfo` is `VK_FALSE`, or the `occlusionQueryPrecise` feature is not enabled, then `pBeginInfo->pInheritanceInfo->queryFlags` **must** not contain `VK_QUERY_CONTROL_PRECISE_BIT`
- VUID-vkBeginCommandBuffer-commandBuffer-02840
  If `commandBuffer` is a primary command buffer, then `pBeginInfo->flags` **must** not set both the `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT` and the `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT` flags

Valid Usage (Implicit)

- VUID-vkBeginCommandBuffer-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkBeginCommandBuffer-pBeginInfo-parameter
  `pBeginInfo` **must** be a valid pointer to a valid VkCommandBufferBeginInfo structure

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkCommandBufferBeginInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkCommandBufferBeginInfo {
    VkStructureType                          sType;
    const void*                              pNext;
    VkCommandBufferUsageFlags                flags;
    const VkCommandBufferInheritanceInfo*    pInheritanceInfo;
} VkCommandBufferBeginInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkCommandBufferUsageFlagBits specifying usage behavior for the command buffer.
- pInheritanceInfo is a pointer to a VkCommandBufferInheritanceInfo structure, used if commandBuffer is a secondary command buffer. If this is a primary command buffer, then this value is ignored.

Valid Usage

- VUID-VkCommandBufferBeginInfo-flags-09123
  If flags contains VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT, the VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-VkCommandBufferBeginInfo-flags-00055
  If flags contains VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT, the framebuffer member of pInheritanceInfo **must** be either VK_NULL_HANDLE, or a valid VkFramebuffer that is compatible with the renderPass member of pInheritanceInfo
- VUID-VkCommandBufferBeginInfo-flags-09240
  If flags contains VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT and the dynamicRendering feature is not enabled, the renderPass member of pInheritanceInfo **must** not be VK_NULL_HANDLE
- VUID-VkCommandBufferBeginInfo-flags-06002
  If flags contains VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT and the renderPass member of pInheritanceInfo is VK_NULL_HANDLE, the pNext chain of pInheritanceInfo **must** include a VkCommandBufferInheritanceRenderingInfo structure
- VUID-VkCommandBufferBeginInfo-flags-06000
  If flags contains VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT and the renderPass member of pInheritanceInfo is not VK_NULL_HANDLE, the renderPass member of pInheritanceInfo **must** be a valid VkRenderPass
- VUID-VkCommandBufferBeginInfo-flags-06001
  If flags contains VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT and the renderPass

member of pInheritanceInfo is not VK_NULL_HANDLE, the subpass member of pInheritanceInfo **must** be a valid subpass index within the renderPass member of pInheritanceInfo

Valid Usage (Implicit)

- VUID-VkCommandBufferBeginInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO
- VUID-VkCommandBufferBeginInfo-pNext-pNext
  pNext **must** be NULL or a pointer to a valid instance of VkDeviceGroupCommandBufferBeginInfo
- VUID-VkCommandBufferBeginInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkCommandBufferBeginInfo-flags-parameter
  flags **must** be a valid combination of VkCommandBufferUsageFlagBits values

Bits which **can** be set in VkCommandBufferBeginInfo::flags, specifying usage behavior for a command buffer, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkCommandBufferUsageFlagBits {
    VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT = 0x00000001,
    VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT = 0x00000002,
    VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT = 0x00000004,
} VkCommandBufferUsageFlagBits;
```

- VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT specifies that each recording of the command buffer will only be submitted once, and the command buffer will be reset and recorded again between each submission.
- VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT specifies that a secondary command buffer is considered to be entirely inside a render pass. If this is a primary command buffer, then this bit is ignored.
- VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT specifies that a command buffer **can** be resubmitted to any queue of the same queue family while it is in the *pending state*, and recorded into multiple primary command buffers.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkCommandBufferUsageFlags;
```

VkCommandBufferUsageFlags is a bitmask type for setting a mask of zero or more VkCommandBufferUsageFlagBits.

If the command buffer is a secondary command buffer, then the VkCommandBufferInheritanceInfo

structure defines any state that will be inherited from the primary command buffer:

```
// Provided by VK_VERSION_1_0
typedef struct VkCommandBufferInheritanceInfo {
    VkStructureType                  sType;
    const void*                      pNext;
    VkRenderPass                     renderPass;
    uint32_t                         subpass;
    VkFramebuffer                    framebuffer;
    VkBool32                         occlusionQueryEnable;
    VkQueryControlFlags              queryFlags;
    VkQueryPipelineStatisticFlags    pipelineStatistics;
} VkCommandBufferInheritanceInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `renderPass` is a VkRenderPass object defining which render passes the `VkCommandBuffer` will be compatible with and **can** be executed within.
- `subpass` is the index of the subpass within the render pass instance that the `VkCommandBuffer` will be executed within.
- `framebuffer` **can** refer to the VkFramebuffer object that the `VkCommandBuffer` will be rendering to if it is executed within a render pass instance. It **can** be VK_NULL_HANDLE if the framebuffer is not known.

> **NOTE** Specifying the exact framebuffer that the secondary command buffer will be executed with **may** result in better performance at command buffer execution time.

- `occlusionQueryEnable` specifies whether the command buffer **can** be executed while an occlusion query is active in the primary command buffer. If this is `VK_TRUE`, then this command buffer **can** be executed whether the primary command buffer has an occlusion query active or not. If this is `VK_FALSE`, then the primary command buffer **must** not have an occlusion query active.
- `queryFlags` specifies the query flags that **can** be used by an active occlusion query in the primary command buffer when this secondary command buffer is executed. If this value includes the `VK_QUERY_CONTROL_PRECISE_BIT` bit, then the active query **can** return boolean results or actual sample counts. If this bit is not set, then the active query **must** not use the `VK_QUERY_CONTROL_PRECISE_BIT` bit.
- `pipelineStatistics` is a bitmask of VkQueryPipelineStatisticFlagBits specifying the set of pipeline statistics that **can** be counted by an active query in the primary command buffer when this secondary command buffer is executed. If this value includes a given bit, then this command buffer **can** be executed whether the primary command buffer has a pipeline statistics query active that includes this bit or not. If this value excludes a given bit, then the active pipeline statistics query **must** not be from a query pool that counts that statistic.

If the VkCommandBuffer will not be executed within a render pass instance, or if the render pass instance was begun with vkCmdBeginRenderingKHR, `renderPass`, `subpass`, and `framebuffer` are ignored.

Valid Usage

- VUID-VkCommandBufferInheritanceInfo-occlusionQueryEnable-00056
  If the `inheritedQueries` feature is not enabled, `occlusionQueryEnable` **must** be `VK_FALSE`
- VUID-VkCommandBufferInheritanceInfo-queryFlags-00057
  If the `inheritedQueries` feature is enabled, `queryFlags` **must** be a valid combination of VkQueryControlFlagBits values
- VUID-VkCommandBufferInheritanceInfo-queryFlags-02788
  If the `inheritedQueries` feature is not enabled, `queryFlags` **must** be `0`
- VUID-VkCommandBufferInheritanceInfo-pipelineStatistics-02789
  If the `pipelineStatisticsQuery` feature is enabled, `pipelineStatistics` **must** be a valid combination of VkQueryPipelineStatisticFlagBits values
- VUID-VkCommandBufferInheritanceInfo-pipelineStatistics-00058
  If the `pipelineStatisticsQuery` feature is not enabled, `pipelineStatistics` **must** be `0`

Valid Usage (Implicit)

- VUID-VkCommandBufferInheritanceInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_INFO`
- VUID-VkCommandBufferInheritanceInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkCommandBufferInheritanceRenderingInfo, VkRenderingAttachmentLocationInfoKHR, or VkRenderingInputAttachmentIndexInfoKHR
- VUID-VkCommandBufferInheritanceInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkCommandBufferInheritanceInfo-commonparent
  Both of `framebuffer`, and `renderPass` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

NOTE On some implementations, not using the `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT` bit enables command buffers to be patched in-place if needed, rather than creating a copy of the command buffer.

If a command buffer is in the invalid, or executable state, and the command buffer was allocated from a command pool with the `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT` flag set, then `vkBeginCommandBuffer` implicitly resets the command buffer, behaving as if `vkResetCommandBuffer` had been called with `VK_COMMAND_BUFFER_RESET_RELEASE_RESOURCES_BIT` not set. After the implicit reset, `commandBuffer` is moved to the recording state.

The VkCommandBufferInheritanceRenderingInfo structure is defined as:

```
typedef struct VkCommandBufferInheritanceRenderingInfo {
    VkStructureType          sType;
    const void*              pNext;
    VkRenderingFlags         flags;
    uint32_t                 viewMask;
    uint32_t                 colorAttachmentCount;
    const VkFormat*          pColorAttachmentFormats;
    VkFormat                 depthAttachmentFormat;
    VkFormat                 stencilAttachmentFormat;
    VkSampleCountFlagBits    rasterizationSamples;
} VkCommandBufferInheritanceRenderingInfo;
```

or the equivalent

```
// Provided by VK_KHR_dynamic_rendering
typedef VkCommandBufferInheritanceRenderingInfo
VkCommandBufferInheritanceRenderingInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure
- flags is a bitmask of VkRenderingFlagBits used by the render pass instance.
- viewMask is the view mask used for rendering.
- colorAttachmentCount is the number of color attachments specified in the render pass instance.
- pColorAttachmentFormats is a pointer to an array of VkFormat values defining the format of color attachments.
- depthAttachmentFormat is a VkFormat value defining the format of the depth attachment.
- stencilAttachmentFormat is a VkFormat value defining the format of the stencil attachment.
- rasterizationSamples is a VkSampleCountFlagBits specifying the number of samples used in rasterization.

If the pNext chain of VkCommandBufferInheritanceInfo includes a VkCommandBufferInheritanceRenderingInfo structure, then that structure controls parameters of dynamic render pass instances that the VkCommandBuffer **can** be executed within. If VkCommandBufferInheritanceInfo::renderPass is not VK_NULL_HANDLE, or VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT is not specified in VkCommandBufferBeginInfo::flags, parameters of this structure are ignored.

If colorAttachmentCount is 0 and the variableMultisampleRate feature is enabled, rasterizationSamples is ignored.

If depthAttachmentFormat, stencilAttachmentFormat, or any element of pColorAttachmentFormats is VK_FORMAT_UNDEFINED, it indicates that the corresponding attachment is unused within the render

pass and writes to those attachments are discarded.

Valid Usage

- VUID-VkCommandBufferInheritanceRenderingInfo-colorAttachmentCount-06004
  If `colorAttachmentCount` is not `0`, `rasterizationSamples` **must** be a valid VkSampleCountFlagBits value
- VUID-VkCommandBufferInheritanceRenderingInfo-variableMultisampleRate-06005
  If the `variableMultisampleRate` feature is not enabled, `rasterizationSamples` **must** be a valid VkSampleCountFlagBits value
- VUID-VkCommandBufferInheritanceRenderingInfo-depthAttachmentFormat-06540
  If `depthAttachmentFormat` is not `VK_FORMAT_UNDEFINED`, it **must** be a format that includes a depth component
- VUID-VkCommandBufferInheritanceRenderingInfo-depthAttachmentFormat-06007
  If `depthAttachmentFormat` is not `VK_FORMAT_UNDEFINED`, it **must** be a format with potential format features that include `VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-VkCommandBufferInheritanceRenderingInfo-pColorAttachmentFormats-06492
  If any element of `pColorAttachmentFormats` is not `VK_FORMAT_UNDEFINED`, it **must** be a format with potential format features that include `VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT`
- VUID-VkCommandBufferInheritanceRenderingInfo-stencilAttachmentFormat-06541
  If `stencilAttachmentFormat` is not `VK_FORMAT_UNDEFINED`, it **must** be a format that includes a stencil aspect
- VUID-VkCommandBufferInheritanceRenderingInfo-stencilAttachmentFormat-06199
  If `stencilAttachmentFormat` is not `VK_FORMAT_UNDEFINED`, it **must** be a format with potential format features that include `VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-VkCommandBufferInheritanceRenderingInfo-depthAttachmentFormat-06200
  If `depthAttachmentFormat` is not `VK_FORMAT_UNDEFINED` and `stencilAttachmentFormat` is not `VK_FORMAT_UNDEFINED`, `depthAttachmentFormat` **must** equal `stencilAttachmentFormat`
- VUID-VkCommandBufferInheritanceRenderingInfo-multiview-06008
  If the `multiview` feature is not enabled, `viewMask` **must** be `0`
- VUID-VkCommandBufferInheritanceRenderingInfo-viewMask-06009
  The index of the most significant bit in `viewMask` **must** be less than `maxMultiviewViewCount`

Valid Usage (Implicit)

- VUID-VkCommandBufferInheritanceRenderingInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_RENDERING_INFO`
- VUID-VkCommandBufferInheritanceRenderingInfo-flags-parameter
  `flags` **must** be a valid combination of VkRenderingFlagBits values
- VUID-VkCommandBufferInheritanceRenderingInfo-pColorAttachmentFormats-parameter
  If `colorAttachmentCount` is not `0`, `pColorAttachmentFormats` **must** be a valid pointer to an array of `colorAttachmentCount` valid VkFormat values

- VUID-VkCommandBufferInheritanceRenderingInfo-depthAttachmentFormat-parameter
  `depthAttachmentFormat` **must** be a valid VkFormat value
- VUID-VkCommandBufferInheritanceRenderingInfo-stencilAttachmentFormat-parameter
  `stencilAttachmentFormat` **must** be a valid VkFormat value
- VUID-VkCommandBufferInheritanceRenderingInfo-rasterizationSamples-parameter
  If `rasterizationSamples` is not `0`, `rasterizationSamples` **must** be a valid VkSampleCountFlagBits value

Once recording starts, an application records a sequence of commands (`vkCmd*`) to set state in the command buffer, draw, dispatch, and other commands.

Several commands can also be recorded indirectly from `VkBuffer` content, see Device-Generated Commands.

To complete recording of a command buffer, call:

```
// Provided by VK_VERSION_1_0
VkResult vkEndCommandBuffer(
    VkCommandBuffer                             commandBuffer);
```

- `commandBuffer` is the command buffer to complete recording.

The command buffer **must** have been in the recording state, and, if successful, is moved to the executable state.

If there was an error during recording, the application will be notified by an unsuccessful return code returned by `vkEndCommandBuffer`, and the command buffer will be moved to the invalid state.

In case the application recorded one or more video encode operations into the command buffer, implementations **may** return the `VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR` error if any of the specified Video Std parameters do not adhere to the syntactic or semantic requirements of the used video compression standard, or if values derived from parameters according to the rules defined by the used video compression standard do not adhere to the capabilities of the video compression standard or the implementation.

NOTE Applications **should** not rely on the `VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR` error being returned by any command as a means to verify Video Std parameters, as implementations are not required to report the error in any specific set of cases.

Valid Usage

- VUID-vkEndCommandBuffer-commandBuffer-00059
  `commandBuffer` **must** be in the recording state
- VUID-vkEndCommandBuffer-commandBuffer-00060
  If `commandBuffer` is a primary command buffer, there **must** not be an active render pass instance

- VUID-vkEndCommandBuffer-commandBuffer-00061
  All queries made active during the recording of `commandBuffer` **must** have been made inactive
- VUID-vkEndCommandBuffer-None-06991
  There **must** be no video session object bound
- VUID-vkEndCommandBuffer-commandBuffer-01815
  If `commandBuffer` is a secondary command buffer, there **must** not be an outstanding vkCmdBeginDebugUtilsLabelEXT command recorded to `commandBuffer` that has not previously been ended by a call to vkCmdEndDebugUtilsLabelEXT

Valid Usage (Implicit)

- VUID-vkEndCommandBuffer-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR`

When a command buffer is in the executable state, it **can** be submitted to a queue for execution.

6.5. Command Buffer Submission

NOTE	Submission can be a high overhead operation, and applications **should** attempt to batch work together into as few calls to `vkQueueSubmit` or `vkQueueSubmit2KHR` as possible.

To submit command buffers to a queue, call:

```
// Provided by VK_KHR_synchronization2
VkResult vkQueueSubmit2KHR(
    VkQueue                                     queue,
    uint32_t                                    submitCount,
    const VkSubmitInfo2*                        pSubmits,
    VkFence                                     fence);
```

- `queue` is the queue that the command buffers will be submitted to.
- `submitCount` is the number of elements in the `pSubmits` array.
- `pSubmits` is a pointer to an array of VkSubmitInfo2 structures, each specifying a command buffer submission batch.
- `fence` is an **optional** handle to a fence to be signaled once all submitted command buffers have completed execution. If `fence` is not VK_NULL_HANDLE, it defines a fence signal operation.

`vkQueueSubmit2KHR` is a queue submission command, with each batch defined by an element of `pSubmits`.

Semaphore operations submitted with vkQueueSubmit2KHR have additional ordering constraints compared to other submission commands, with dependencies involving previous and subsequent queue operations. Information about these additional constraints can be found in the semaphore section of the synchronization chapter.

If any command buffer submitted to this queue is in the executable state, it is moved to the pending state. Once execution of all submissions of a command buffer complete, it moves from the pending state, back to the executable state. If a command buffer was recorded with the `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT` flag, it instead moves back to the invalid state.

If `vkQueueSubmit2KHR` fails, it **may** return `VK_ERROR_OUT_OF_HOST_MEMORY` or `VK_ERROR_OUT_OF_DEVICE_MEMORY`. If it does, the implementation **must** ensure that the state and contents of any resources or synchronization primitives referenced by the submitted command buffers and any semaphores referenced by `pSubmits` is unaffected by the call or its failure. If `vkQueueSubmit2KHR` fails in such a way that the implementation is unable to make that guarantee, the implementation **must** return `VK_ERROR_DEVICE_LOST`. See Lost Device.

Valid Usage

- VUID-vkQueueSubmit2-fence-04894
  If `fence` is not VK_NULL_HANDLE, `fence` **must** be unsignaled
- VUID-vkQueueSubmit2-fence-04895
  If `fence` is not VK_NULL_HANDLE, `fence` **must** not be associated with any other queue command that has not yet completed execution on that queue
- VUID-vkQueueSubmit2-synchronization2-03866
  The `synchronization2` feature **must** be enabled
- VUID-vkQueueSubmit2-commandBuffer-03867
  If a command recorded into the `commandBuffer` member of any element of the

pCommandBufferInfos member of any element of pSubmits referenced a VkEvent, that event **must** not be referenced by a command that has been submitted to another queue and is still in the *pending state*

- VUID-vkQueueSubmit2-semaphore-03868
  The semaphore member of any binary semaphore element of the pSignalSemaphoreInfos member of any element of pSubmits **must** be unsignaled when the semaphore signal operation it defines is executed on the device
- VUID-vkQueueSubmit2-stageMask-03869
  The stageMask member of any element of the pSignalSemaphoreInfos member of any element of pSubmits **must** only include pipeline stages that are supported by the queue family which queue belongs to
- VUID-vkQueueSubmit2-stageMask-03870
  The stageMask member of any element of the pWaitSemaphoreInfos member of any element of pSubmits **must** only include pipeline stages that are supported by the queue family which queue belongs to
- VUID-vkQueueSubmit2-semaphore-03871
  When a semaphore wait operation for a binary semaphore is executed, as defined by the semaphore member of any element of the pWaitSemaphoreInfos member of any element of pSubmits, there **must** be no other queues waiting on the same semaphore
- VUID-vkQueueSubmit2-semaphore-03873
  The semaphore member of any element of the pWaitSemaphoreInfos member of any element of pSubmits that was created with a VkSemaphoreTypeKHR of VK_SEMAPHORE_TYPE_BINARY_KHR **must** reference a semaphore signal operation that has been submitted for execution and any semaphore signal operations on which it depends **must** have also been submitted for execution
- VUID-vkQueueSubmit2-commandBuffer-03874
  The commandBuffer member of any element of the pCommandBufferInfos member of any element of pSubmits **must** be in the pending or executable state
- VUID-vkQueueSubmit2-commandBuffer-03875
  If a command recorded into the commandBuffer member of any element of the pCommandBufferInfos member of any element of pSubmits was not recorded with the VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT, it **must** not be in the pending state
- VUID-vkQueueSubmit2-commandBuffer-03876
  Any secondary command buffers recorded into the commandBuffer member of any element of the pCommandBufferInfos member of any element of pSubmits **must** be in the pending or executable state
- VUID-vkQueueSubmit2-commandBuffer-03877
  If any secondary command buffers recorded into the commandBuffer member of any element of the pCommandBufferInfos member of any element of pSubmits was not recorded with the VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT, it **must** not be in the pending state
- VUID-vkQueueSubmit2-commandBuffer-03878
  The commandBuffer member of any element of the pCommandBufferInfos member of any element of pSubmits **must** have been allocated from a VkCommandPool that was created for

the same queue family `queue` belongs to

- VUID-vkQueueSubmit2-commandBuffer-03879
  If a command recorded into the `commandBuffer` member of any element of the `pCommandBufferInfos` member of any element of `pSubmits` includes a Queue Family Ownership Transfer Acquire Operation, there **must** exist a previously submitted Queue Family Ownership Transfer Release Operation on a queue in the queue family identified by the acquire operation, with parameters matching the acquire operation as defined in the definition of such acquire operations, and which happens before the acquire operation
- VUID-vkQueueSubmit2-commandBuffer-03880
  If a command recorded into the `commandBuffer` member of any element of the `pCommandBufferInfos` member of any element of `pSubmits` was a vkCmdBeginQuery whose `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, the profiling lock **must** have been held continuously on the `VkDevice` that `queue` was retrieved from, throughout recording of those command buffers

Valid Usage (Implicit)

- VUID-vkQueueSubmit2-queue-parameter
  `queue` **must** be a valid VkQueue handle
- VUID-vkQueueSubmit2-pSubmits-parameter
  If `submitCount` is not `0`, `pSubmits` **must** be a valid pointer to an array of `submitCount` valid VkSubmitInfo2 structures
- VUID-vkQueueSubmit2-fence-parameter
  If `fence` is not VK_NULL_HANDLE, `fence` **must** be a valid VkFence handle
- VUID-vkQueueSubmit2-commonparent
  Both of `fence`, and `queue` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `queue` **must** be externally synchronized
- Host access to `fence` **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
-	-	-	Any	-

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

The VkSubmitInfo2 structure is defined as:

```
typedef struct VkSubmitInfo2 {
    VkStructureType                       sType;
    const void*                           pNext;
    VkSubmitFlags                         flags;
    uint32_t                              waitSemaphoreInfoCount;
    const VkSemaphoreSubmitInfo*          pWaitSemaphoreInfos;
    uint32_t                              commandBufferInfoCount;
    const VkCommandBufferSubmitInfo*      pCommandBufferInfos;
    uint32_t                              signalSemaphoreInfoCount;
    const VkSemaphoreSubmitInfo*          pSignalSemaphoreInfos;
} VkSubmitInfo2;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkSubmitInfo2 VkSubmitInfo2KHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkSubmitFlagBits.
- waitSemaphoreInfoCount is the number of elements in pWaitSemaphoreInfos.
- pWaitSemaphoreInfos is a pointer to an array of VkSemaphoreSubmitInfo structures defining semaphore wait operations.
- commandBufferInfoCount is the number of elements in pCommandBufferInfos and the number of command buffers to execute in the batch.
- pCommandBufferInfos is a pointer to an array of VkCommandBufferSubmitInfo structures describing command buffers to execute in the batch.
- signalSemaphoreInfoCount is the number of elements in pSignalSemaphoreInfos.
- pSignalSemaphoreInfos is a pointer to an array of VkSemaphoreSubmitInfo describing semaphore signal operations.

Valid Usage

- VUID-VkSubmitInfo2-semaphore-03881
  If the same semaphore is used as the `semaphore` member of both an element of `pSignalSemaphoreInfos` and `pWaitSemaphoreInfos`, and that semaphore is a timeline semaphore, the `value` member of the `pSignalSemaphoreInfos` element **must** be greater than the `value` member of the `pWaitSemaphoreInfos` element
- VUID-VkSubmitInfo2-semaphore-03882
  If the `semaphore` member of any element of `pSignalSemaphoreInfos` is a timeline semaphore, the `value` member of that element **must** have a value greater than the current value of the semaphore when the semaphore signal operation is executed
- VUID-VkSubmitInfo2-semaphore-03883
  If the `semaphore` member of any element of `pSignalSemaphoreInfos` is a timeline semaphore, the `value` member of that element **must** have a value which does not differ from the current value of the semaphore or the value of any outstanding semaphore wait or signal operation on that semaphore by more than `maxTimelineSemaphoreValueDifference`
- VUID-VkSubmitInfo2-semaphore-03884
  If the `semaphore` member of any element of `pWaitSemaphoreInfos` is a timeline semaphore, the `value` member of that element **must** have a value which does not differ from the current value of the semaphore or the value of any outstanding semaphore wait or signal operation on that semaphore by more than `maxTimelineSemaphoreValueDifference`
- VUID-VkSubmitInfo2KHR-commandBuffer-06192
  If any `commandBuffer` member of an element of `pCommandBufferInfos` contains any resumed render pass instances, they **must** be suspended by a render pass instance earlier in submission order within `pCommandBufferInfos`
- VUID-VkSubmitInfo2KHR-commandBuffer-06010
  If any `commandBuffer` member of an element of `pCommandBufferInfos` contains any suspended render pass instances, they **must** be resumed by a render pass instance later in submission order within `pCommandBufferInfos`
- VUID-VkSubmitInfo2KHR-commandBuffer-06011
  If any `commandBuffer` member of an element of `pCommandBufferInfos` contains any suspended render pass instances, there **must** be no action or synchronization commands between that render pass instance and the render pass instance that resumes it
- VUID-VkSubmitInfo2KHR-commandBuffer-06012
  If any `commandBuffer` member of an element of `pCommandBufferInfos` contains any suspended render pass instances, there **must** be no render pass instances between that render pass instance and the render pass instance that resumes it
- VUID-VkSubmitInfo2KHR-variableSampleLocations-06013
  If the `variableSampleLocations` limit is not supported, and any `commandBuffer` member of an element of `pCommandBufferInfos` contains any suspended render pass instances, where a graphics pipeline has been bound, any pipelines bound in the render pass instance that resumes it, or any subsequent render pass instances that resume from that one and so on, **must** use the same sample locations

Valid Usage (Implicit)

- VUID-VkSubmitInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SUBMIT_INFO_2`
- VUID-VkSubmitInfo2-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkFrameBoundaryEXT, VkPerformanceQuerySubmitInfoKHR, or VkWin32KeyedMutexAcquireReleaseInfoKHR
- VUID-VkSubmitInfo2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkSubmitInfo2-flags-parameter
  `flags` **must** be a valid combination of VkSubmitFlagBits values
- VUID-VkSubmitInfo2-pWaitSemaphoreInfos-parameter
  If `waitSemaphoreInfoCount` is not `0`, `pWaitSemaphoreInfos` **must** be a valid pointer to an array of `waitSemaphoreInfoCount` valid VkSemaphoreSubmitInfo structures
- VUID-VkSubmitInfo2-pCommandBufferInfos-parameter
  If `commandBufferInfoCount` is not `0`, `pCommandBufferInfos` **must** be a valid pointer to an array of `commandBufferInfoCount` valid VkCommandBufferSubmitInfo structures
- VUID-VkSubmitInfo2-pSignalSemaphoreInfos-parameter
  If `signalSemaphoreInfoCount` is not `0`, `pSignalSemaphoreInfos` **must** be a valid pointer to an array of `signalSemaphoreInfoCount` valid VkSemaphoreSubmitInfo structures

Bits which **can** be set in VkSubmitInfo2::`flags`, specifying submission behavior, are:

```
typedef enum VkSubmitFlagBits {
    VK_SUBMIT_PROTECTED_BIT = 0x00000001,
    VK_SUBMIT_PROTECTED_BIT_KHR = VK_SUBMIT_PROTECTED_BIT,
} VkSubmitFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkSubmitFlagBits VkSubmitFlagBitsKHR;
```

- `VK_SUBMIT_PROTECTED_BIT` specifies that this batch is a protected submission.

```
typedef VkFlags VkSubmitFlags;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkSubmitFlags VkSubmitFlagsKHR;
```

VkSubmitFlags is a bitmask type for setting a mask of zero or more VkSubmitFlagBits.

The VkSemaphoreSubmitInfo structure is defined as:

```
typedef struct VkSemaphoreSubmitInfo {
    VkStructureType          sType;
    const void*              pNext;
    VkSemaphore              semaphore;
    uint64_t                 value;
    VkPipelineStageFlags2    stageMask;
    uint32_t                 deviceIndex;
} VkSemaphoreSubmitInfo;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkSemaphoreSubmitInfo VkSemaphoreSubmitInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- semaphore is a VkSemaphore affected by this operation.
- value is either the value used to signal semaphore or the value waited on by semaphore, if semaphore is a timeline semaphore. Otherwise it is ignored.
- stageMask is a VkPipelineStageFlags2 mask of pipeline stages which limit the first synchronization scope of a semaphore signal operation, or second synchronization scope of a semaphore wait operation as described in the semaphore wait operation and semaphore signal operation sections of the synchronization chapter.
- deviceIndex is the index of the device within a device group that executes the semaphore wait or signal operation.

Whether this structure defines a semaphore wait or signal operation is defined by how it is used.

Valid Usage

- VUID-VkSemaphoreSubmitInfo-stageMask-03929
  If the geometryShader feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT
- VUID-VkSemaphoreSubmitInfo-stageMask-03930
  If the tessellationShader feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT or

VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT

- VUID-VkSemaphoreSubmitInfo-stageMask-03932
  If the fragmentDensityMap feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-VkSemaphoreSubmitInfo-stageMask-03933
  If the transformFeedback feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-VkSemaphoreSubmitInfo-stageMask-07317
  If the attachmentFragmentShadingRate feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkSemaphoreSubmitInfo-stageMask-07947
  If the rayTracingPipeline feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkSemaphoreSubmitInfo-device-03888
  If the device that semaphore was created on is not a device group, deviceIndex **must** be 0
- VUID-VkSemaphoreSubmitInfo-device-03889
  If the device that semaphore was created on is a device group, deviceIndex **must** be a valid device index

Valid Usage (Implicit)

- VUID-VkSemaphoreSubmitInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SEMAPHORE_SUBMIT_INFO
- VUID-VkSemaphoreSubmitInfo-pNext-pNext
  pNext **must** be NULL
- VUID-VkSemaphoreSubmitInfo-semaphore-parameter
  semaphore **must** be a valid VkSemaphore handle
- VUID-VkSemaphoreSubmitInfo-stageMask-parameter
  stageMask **must** be a valid combination of VkPipelineStageFlagBits2 values

The VkCommandBufferSubmitInfo structure is defined as:

```
typedef struct VkCommandBufferSubmitInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkCommandBuffer    commandBuffer;
    uint32_t           deviceMask;
} VkCommandBufferSubmitInfo;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkCommandBufferSubmitInfo VkCommandBufferSubmitInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `commandBuffer` is a VkCommandBuffer to be submitted for execution.
- `deviceMask` is a bitmask indicating which devices in a device group execute the command buffer. A `deviceMask` of `0` is equivalent to setting all bits corresponding to valid devices in the group to `1`.

Valid Usage

- VUID-VkCommandBufferSubmitInfo-commandBuffer-03890
  `commandBuffer` **must** not have been allocated with `VK_COMMAND_BUFFER_LEVEL_SECONDARY`
- VUID-VkCommandBufferSubmitInfo-deviceMask-03891
  If `deviceMask` is not `0`, it **must** be a valid device mask

Valid Usage (Implicit)

- VUID-VkCommandBufferSubmitInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO`
- VUID-VkCommandBufferSubmitInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCommandBufferSubmitInfo-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle

To submit command buffers to a queue, call:

```
// Provided by VK_VERSION_1_0
VkResult vkQueueSubmit(
    VkQueue                                     queue,
    uint32_t                                    submitCount,
    const VkSubmitInfo*                         pSubmits,
    VkFence                                     fence);
```

- `queue` is the queue that the command buffers will be submitted to.
- `submitCount` is the number of elements in the `pSubmits` array.
- `pSubmits` is a pointer to an array of VkSubmitInfo structures, each specifying a command buffer submission batch.
- `fence` is an **optional** handle to a fence to be signaled once all submitted command buffers have completed execution. If `fence` is not VK_NULL_HANDLE, it defines a fence signal operation.

vkQueueSubmit is a queue submission command, with each batch defined by an element of pSubmits. Batches begin execution in the order they appear in pSubmits, but **may** complete out of order.

Fence and semaphore operations submitted with vkQueueSubmit have additional ordering constraints compared to other submission commands, with dependencies involving previous and subsequent queue operations. Information about these additional constraints can be found in the semaphore and fence sections of the synchronization chapter.

Details on the interaction of pWaitDstStageMask with synchronization are described in the semaphore wait operation section of the synchronization chapter.

The order that batches appear in pSubmits is used to determine submission order, and thus all the implicit ordering guarantees that respect it. Other than these implicit ordering guarantees and any explicit synchronization primitives, these batches **may** overlap or otherwise execute out of order.

If any command buffer submitted to this queue is in the executable state, it is moved to the pending state. Once execution of all submissions of a command buffer complete, it moves from the pending state, back to the executable state. If a command buffer was recorded with the VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT flag, it instead moves to the invalid state.

If vkQueueSubmit fails, it **may** return VK_ERROR_OUT_OF_HOST_MEMORY or VK_ERROR_OUT_OF_DEVICE_MEMORY. If it does, the implementation **must** ensure that the state and contents of any resources or synchronization primitives referenced by the submitted command buffers and any semaphores referenced by pSubmits is unaffected by the call or its failure. If vkQueueSubmit fails in such a way that the implementation is unable to make that guarantee, the implementation **must** return VK_ERROR_DEVICE_LOST. See Lost Device.

Valid Usage

- VUID-vkQueueSubmit-fence-00063
  If fence is not VK_NULL_HANDLE, fence **must** be unsignaled
- VUID-vkQueueSubmit-fence-00064
  If fence is not VK_NULL_HANDLE, fence **must** not be associated with any other queue command that has not yet completed execution on that queue
- VUID-vkQueueSubmit-pCommandBuffers-00065
  Any calls to vkCmdSetEvent, vkCmdResetEvent or vkCmdWaitEvents that have been recorded into any of the command buffer elements of the pCommandBuffers member of any element of pSubmits, **must** not reference any VkEvent that is referenced by any of those commands in a command buffer that has been submitted to another queue and is still in the *pending state*
- VUID-vkQueueSubmit-pWaitDstStageMask-00066
  Any stage flag included in any element of the pWaitDstStageMask member of any element of pSubmits **must** be a pipeline stage supported by one of the capabilities of queue, as specified in the table of supported pipeline stages
- VUID-vkQueueSubmit-pSignalSemaphores-00067
  Each binary semaphore element of the pSignalSemaphores member of any element of pSubmits **must** be unsignaled when the semaphore signal operation it defines is executed

on the device

- VUID-vkQueueSubmit-pWaitSemaphores-00068
  When a semaphore wait operation referring to a binary semaphore defined by any element of the pWaitSemaphores member of any element of pSubmits executes on queue, there **must** be no other queues waiting on the same semaphore
- VUID-vkQueueSubmit-pWaitSemaphores-03238
  All elements of the pWaitSemaphores member of all elements of pSubmits created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_BINARY **must** reference a semaphore signal operation that has been submitted for execution and any semaphore signal operations on which it depends **must** have also been submitted for execution
- VUID-vkQueueSubmit-pCommandBuffers-00070
  Each element of the pCommandBuffers member of each element of pSubmits **must** be in the pending or executable state
- VUID-vkQueueSubmit-pCommandBuffers-00071
  If any element of the pCommandBuffers member of any element of pSubmits was not recorded with the VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT, it **must** not be in the pending state
- VUID-vkQueueSubmit-pCommandBuffers-00072
  Any secondary command buffers recorded into any element of the pCommandBuffers member of any element of pSubmits **must** be in the pending or executable state
- VUID-vkQueueSubmit-pCommandBuffers-00073
  If any secondary command buffers recorded into any element of the pCommandBuffers member of any element of pSubmits was not recorded with the VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT, it **must** not be in the pending state
- VUID-vkQueueSubmit-pCommandBuffers-00074
  Each element of the pCommandBuffers member of each element of pSubmits **must** have been allocated from a VkCommandPool that was created for the same queue family queue belongs to
- VUID-vkQueueSubmit-pSubmits-02207
  If any element of pSubmits->pCommandBuffers includes a Queue Family Ownership Transfer Acquire Operation, there **must** exist a previously submitted Queue Family Ownership Transfer Release Operation on a queue in the queue family identified by the acquire operation, with parameters matching the acquire operation as defined in the definition of such acquire operations, and which happens-before the acquire operation
- VUID-vkQueueSubmit-pCommandBuffers-03220
  If a command recorded into any element of pCommandBuffers was a vkCmdBeginQuery whose queryPool was created with a queryType of VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, the profiling lock **must** have been held continuously on the VkDevice that queue was retrieved from, throughout recording of those command buffers
- VUID-vkQueueSubmit-pSubmits-02808
  Any resource created with VK_SHARING_MODE_EXCLUSIVE that is read by an operation specified by pSubmits **must** not be owned by any queue family other than the one which queue belongs to, at the time it is executed

- VUID-vkQueueSubmit-pSubmits-04626
  Any resource created with VK_SHARING_MODE_CONCURRENT that is accessed by an operation specified by pSubmits **must** have included the queue family of queue at resource creation time

Valid Usage (Implicit)

- VUID-vkQueueSubmit-queue-parameter
  queue **must** be a valid VkQueue handle
- VUID-vkQueueSubmit-pSubmits-parameter
  If submitCount is not 0, pSubmits **must** be a valid pointer to an array of submitCount valid VkSubmitInfo structures
- VUID-vkQueueSubmit-fence-parameter
  If fence is not VK_NULL_HANDLE, fence **must** be a valid VkFence handle
- VUID-vkQueueSubmit-commonparent
  Both of fence, and queue that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to queue **must** be externally synchronized
- Host access to fence **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
-	-	-	Any	-

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

The VkSubmitInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSubmitInfo {
    VkStructureType                sType;
    const void*                    pNext;
    uint32_t                       waitSemaphoreCount;
    const VkSemaphore*             pWaitSemaphores;
    const VkPipelineStageFlags*    pWaitDstStageMask;
    uint32_t                       commandBufferCount;
    const VkCommandBuffer*         pCommandBuffers;
    uint32_t                       signalSemaphoreCount;
    const VkSemaphore*             pSignalSemaphores;
} VkSubmitInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `waitSemaphoreCount` is the number of semaphores upon which to wait before executing the command buffers for the batch.
- `pWaitSemaphores` is a pointer to an array of VkSemaphore handles upon which to wait before the command buffers for this batch begin execution. If semaphores to wait on are provided, they define a semaphore wait operation.
- `pWaitDstStageMask` is a pointer to an array of pipeline stages at which each corresponding semaphore wait will occur.
- `commandBufferCount` is the number of command buffers to execute in the batch.
- `pCommandBuffers` is a pointer to an array of VkCommandBuffer handles to execute in the batch.
- `signalSemaphoreCount` is the number of semaphores to be signaled once the commands specified in `pCommandBuffers` have completed execution.
- `pSignalSemaphores` is a pointer to an array of VkSemaphore handles which will be signaled when the command buffers for this batch have completed execution. If semaphores to be signaled are provided, they define a semaphore signal operation.

The order that command buffers appear in `pCommandBuffers` is used to determine submission order, and thus all the implicit ordering guarantees that respect it. Other than these implicit ordering guarantees and any explicit synchronization primitives, these command buffers **may** overlap or otherwise execute out of order.

Valid Usage

- VUID-VkSubmitInfo-pWaitDstStageMask-04090
  If the `geometryShader` feature is not enabled, `pWaitDstStageMask` **must** not contain `VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT`
- VUID-VkSubmitInfo-pWaitDstStageMask-04091
  If the `tessellationShader` feature is not enabled, `pWaitDstStageMask` **must** not contain `VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT`

- VUID-VkSubmitInfo-pWaitDstStageMask-04093
  If the fragmentDensityMap feature is not enabled, pWaitDstStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-VkSubmitInfo-pWaitDstStageMask-04094
  If the transformFeedback feature is not enabled, pWaitDstStageMask **must** not contain VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-VkSubmitInfo-pWaitDstStageMask-07319
  If the attachmentFragmentShadingRate feature is not enabled, pWaitDstStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkSubmitInfo-pWaitDstStageMask-03937
  If the synchronization2 feature is not enabled, pWaitDstStageMask **must** not be 0
- VUID-VkSubmitInfo-pWaitDstStageMask-07950
  If the rayTracingPipeline feature is not enabled, pWaitDstStageMask **must** not contain VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkSubmitInfo-pCommandBuffers-00075
  Each element of pCommandBuffers **must** not have been allocated with VK_COMMAND_BUFFER_LEVEL_SECONDARY
- VUID-VkSubmitInfo-pWaitDstStageMask-00078
  Each element of pWaitDstStageMask **must** not include VK_PIPELINE_STAGE_HOST_BIT
- VUID-VkSubmitInfo-pWaitSemaphores-03239
  If any element of pWaitSemaphores or pSignalSemaphores was created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE, then the pNext chain **must** include a VkTimelineSemaphoreSubmitInfo structure
- VUID-VkSubmitInfo-pNext-03240
  If the pNext chain of this structure includes a VkTimelineSemaphoreSubmitInfo structure and any element of pWaitSemaphores was created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE, then its waitSemaphoreValueCount member **must** equal waitSemaphoreCount
- VUID-VkSubmitInfo-pNext-03241
  If the pNext chain of this structure includes a VkTimelineSemaphoreSubmitInfo structure and any element of pSignalSemaphores was created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE, then its signalSemaphoreValueCount member **must** equal signalSemaphoreCount
- VUID-VkSubmitInfo-pSignalSemaphores-03242
  For each element of pSignalSemaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE the corresponding element of VkTimelineSemaphoreSubmitInfo::pSignalSemaphoreValues **must** have a value greater than the current value of the semaphore when the semaphore signal operation is executed
- VUID-VkSubmitInfo-pWaitSemaphores-03243
  For each element of pWaitSemaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE the corresponding element of VkTimelineSemaphoreSubmitInfo::pWaitSemaphoreValues **must** have a value which does

not differ from the current value of the semaphore or the value of any outstanding semaphore wait or signal operation on that semaphore by more than maxTimelineSemaphoreValueDifference

- VUID-VkSubmitInfo-pSignalSemaphores-03244
  For each element of pSignalSemaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE the corresponding element of VkTimelineSemaphoreSubmitInfo::pSignalSemaphoreValues **must** have a value which does not differ from the current value of the semaphore or the value of any outstanding semaphore wait or signal operation on that semaphore by more than maxTimelineSemaphoreValueDifference
- VUID-VkSubmitInfo-pCommandBuffers-06193
  If pCommandBuffers contains any resumed render pass instances, they **must** be suspended by a render pass instance earlier in submission order within pCommandBuffers
- VUID-VkSubmitInfo-pCommandBuffers-06014
  If pCommandBuffers contains any suspended render pass instances, they **must** be resumed by a render pass instance later in submission order within pCommandBuffers
- VUID-VkSubmitInfo-pCommandBuffers-06015
  If pCommandBuffers contains any suspended render pass instances, there **must** be no action or synchronization commands executed in a primary or secondary command buffer between that render pass instance and the render pass instance that resumes it
- VUID-VkSubmitInfo-pCommandBuffers-06016
  If pCommandBuffers contains any suspended render pass instances, there **must** be no render pass instances between that render pass instance and the render pass instance that resumes it
- VUID-VkSubmitInfo-variableSampleLocations-06017
  If the variableSampleLocations limit is not supported, and any element of pCommandBuffers contains any suspended render pass instances, where a graphics pipeline has been bound, any pipelines bound in the render pass instance that resumes it, or any subsequent render pass instances that resume from that one and so on, **must** use the same sample locations

Valid Usage (Implicit)

- VUID-VkSubmitInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SUBMIT_INFO
- VUID-VkSubmitInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkD3D12FenceSubmitInfoKHR, VkDeviceGroupSubmitInfo, VkFrameBoundaryEXT, VkPerformanceQuerySubmitInfoKHR, VkTimelineSemaphoreSubmitInfo, or VkWin32KeyedMutexAcquireReleaseInfoKHR
- VUID-VkSubmitInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique

- VUID-VkSubmitInfo-pWaitSemaphores-parameter
  If `waitSemaphoreCount` is not `0`, `pWaitSemaphores` **must** be a valid pointer to an array of `waitSemaphoreCount` valid VkSemaphore handles
- VUID-VkSubmitInfo-pWaitDstStageMask-parameter
  If `waitSemaphoreCount` is not `0`, `pWaitDstStageMask` **must** be a valid pointer to an array of `waitSemaphoreCount` valid combinations of VkPipelineStageFlagBits values
- VUID-VkSubmitInfo-pCommandBuffers-parameter
  If `commandBufferCount` is not `0`, `pCommandBuffers` **must** be a valid pointer to an array of `commandBufferCount` valid VkCommandBuffer handles
- VUID-VkSubmitInfo-pSignalSemaphores-parameter
  If `signalSemaphoreCount` is not `0`, `pSignalSemaphores` **must** be a valid pointer to an array of `signalSemaphoreCount` valid VkSemaphore handles
- VUID-VkSubmitInfo-commonparent
  Each of the elements of `pCommandBuffers`, the elements of `pSignalSemaphores`, and the elements of `pWaitSemaphores` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

To specify the values to use when waiting for and signaling semaphores created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_TIMELINE`, add a VkTimelineSemaphoreSubmitInfo structure to the `pNext` chain of the VkSubmitInfo structure when using vkQueueSubmit or the VkBindSparseInfo structure when using vkQueueBindSparse . The `VkTimelineSemaphoreSubmitInfo` structure is defined as:

```
typedef struct VkTimelineSemaphoreSubmitInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           waitSemaphoreValueCount;
    const uint64_t*    pWaitSemaphoreValues;
    uint32_t           signalSemaphoreValueCount;
    const uint64_t*    pSignalSemaphoreValues;
} VkTimelineSemaphoreSubmitInfo;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkTimelineSemaphoreSubmitInfo VkTimelineSemaphoreSubmitInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `waitSemaphoreValueCount` is the number of semaphore wait values specified in `pWaitSemaphoreValues`.
- `pWaitSemaphoreValues` is a pointer to an array of `waitSemaphoreValueCount` values for the corresponding semaphores in VkSubmitInfo::`pWaitSemaphores` to wait for.

- `signalSemaphoreValueCount` is the number of semaphore signal values specified in `pSignalSemaphoreValues`.
- `pSignalSemaphoreValues` is a pointer to an array `signalSemaphoreValueCount` values for the corresponding semaphores in VkSubmitInfo::`pSignalSemaphores` to set when signaled.

If the semaphore in VkSubmitInfo::`pWaitSemaphores` or VkSubmitInfo::`pSignalSemaphores` corresponding to an entry in `pWaitSemaphoreValues` or `pSignalSemaphoreValues` respectively was not created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_TIMELINE`, the implementation **must** ignore the value in the `pWaitSemaphoreValues` or `pSignalSemaphoreValues` entry.

Valid Usage (Implicit)

- VUID-VkTimelineSemaphoreSubmitInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_TIMELINE_SEMAPHORE_SUBMIT_INFO`
- VUID-VkTimelineSemaphoreSubmitInfo-pWaitSemaphoreValues-parameter
  If `waitSemaphoreValueCount` is not `0`, and `pWaitSemaphoreValues` is not `NULL`, `pWaitSemaphoreValues` **must** be a valid pointer to an array of `waitSemaphoreValueCount` `uint64_t` values
- VUID-VkTimelineSemaphoreSubmitInfo-pSignalSemaphoreValues-parameter
  If `signalSemaphoreValueCount` is not `0`, and `pSignalSemaphoreValues` is not `NULL`, `pSignalSemaphoreValues` **must** be a valid pointer to an array of `signalSemaphoreValueCount` `uint64_t` values

To specify the values to use when waiting for and signaling semaphores whose current payload refers to a Direct3D 12 fence, add a VkD3D12FenceSubmitInfoKHR structure to the `pNext` chain of the VkSubmitInfo structure. The `VkD3D12FenceSubmitInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_semaphore_win32
typedef struct VkD3D12FenceSubmitInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           waitSemaphoreValuesCount;
    const uint64_t*    pWaitSemaphoreValues;
    uint32_t           signalSemaphoreValuesCount;
    const uint64_t*    pSignalSemaphoreValues;
} VkD3D12FenceSubmitInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `waitSemaphoreValuesCount` is the number of semaphore wait values specified in `pWaitSemaphoreValues`.
- `pWaitSemaphoreValues` is a pointer to an array of `waitSemaphoreValuesCount` values for the corresponding semaphores in VkSubmitInfo::`pWaitSemaphores` to wait for.
- `signalSemaphoreValuesCount` is the number of semaphore signal values specified in

pSignalSemaphoreValues.

- pSignalSemaphoreValues is a pointer to an array of signalSemaphoreValuesCount values for the corresponding semaphores in VkSubmitInfo::pSignalSemaphores to set when signaled.

If the semaphore in VkSubmitInfo::pWaitSemaphores or VkSubmitInfo::pSignalSemaphores corresponding to an entry in pWaitSemaphoreValues or pSignalSemaphoreValues respectively does not currently have a payload referring to a Direct3D 12 fence, the implementation **must** ignore the value in the pWaitSemaphoreValues or pSignalSemaphoreValues entry.

NOTE

As the introduction of the external semaphore handle type VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT predates that of timeline semaphores, support for importing semaphore payloads from external handles of that type into semaphores created (implicitly or explicitly) with a VkSemaphoreType of VK_SEMAPHORE_TYPE_BINARY is preserved for backwards compatibility. However, applications **should** prefer importing such handle types into semaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE, and use the VkTimelineSemaphoreSubmitInfo structure instead of the VkD3D12FenceSubmitInfoKHR structure to specify the values to use when waiting for and signaling such semaphores.

Valid Usage

- VUID-VkD3D12FenceSubmitInfoKHR-waitSemaphoreValuesCount-00079
  waitSemaphoreValuesCount **must** be the same value as VkSubmitInfo::waitSemaphoreCount, where this structure is in the pNext chain of a VkSubmitInfo structure
- VUID-VkD3D12FenceSubmitInfoKHR-signalSemaphoreValuesCount-00080
  signalSemaphoreValuesCount **must** be the same value as VkSubmitInfo::signalSemaphoreCount, where this structure is in the pNext chain of a VkSubmitInfo structure

Valid Usage (Implicit)

- VUID-VkD3D12FenceSubmitInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_D3D12_FENCE_SUBMIT_INFO_KHR
- VUID-VkD3D12FenceSubmitInfoKHR-pWaitSemaphoreValues-parameter
  If waitSemaphoreValuesCount is not 0, and pWaitSemaphoreValues is not NULL, pWaitSemaphoreValues **must** be a valid pointer to an array of waitSemaphoreValuesCount uint64_t values
- VUID-VkD3D12FenceSubmitInfoKHR-pSignalSemaphoreValues-parameter
  If signalSemaphoreValuesCount is not 0, and pSignalSemaphoreValues is not NULL, pSignalSemaphoreValues **must** be a valid pointer to an array of signalSemaphoreValuesCount uint64_t values

When submitting work that operates on memory imported from a Direct3D 11 resource to a queue, the keyed mutex mechanism **may** be used in addition to Vulkan semaphores to synchronize the

work. Keyed mutexes are a property of a properly created shareable Direct3D 11 resource. They **can** only be used if the imported resource was created with the `D3D11_RESOURCE_MISC_SHARED_KEYEDMUTEX` flag.

To acquire keyed mutexes before submitted work and/or release them after, add a `VkWin32KeyedMutexAcquireReleaseInfoKHR` structure to the `pNext` chain of the `VkSubmitInfo` structure.

The `VkWin32KeyedMutexAcquireReleaseInfoKHR` structure is defined as:

```
// Provided by VK_KHR_win32_keyed_mutex
typedef struct VkWin32KeyedMutexAcquireReleaseInfoKHR {
    VkStructureType          sType;
    const void*              pNext;
    uint32_t                 acquireCount;
    const VkDeviceMemory*    pAcquireSyncs;
    const uint64_t*          pAcquireKeys;
    const uint32_t*          pAcquireTimeouts;
    uint32_t                 releaseCount;
    const VkDeviceMemory*    pReleaseSyncs;
    const uint64_t*          pReleaseKeys;
} VkWin32KeyedMutexAcquireReleaseInfoKHR;
```

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `acquireCount` is the number of entries in the `pAcquireSyncs`, `pAcquireKeys`, and `pAcquireTimeouts` arrays.
- `pAcquireSyncs` is a pointer to an array of `VkDeviceMemory` objects which were imported from Direct3D 11 resources.
- `pAcquireKeys` is a pointer to an array of mutex key values to wait for prior to beginning the submitted work. Entries refer to the keyed mutex associated with the corresponding entries in `pAcquireSyncs`.
- `pAcquireTimeouts` is a pointer to an array of timeout values, in millisecond units, for each acquire specified in `pAcquireKeys`.
- `releaseCount` is the number of entries in the `pReleaseSyncs` and `pReleaseKeys` arrays.
- `pReleaseSyncs` is a pointer to an array of `VkDeviceMemory` objects which were imported from Direct3D 11 resources.
- `pReleaseKeys` is a pointer to an array of mutex key values to set when the submitted work has completed. Entries refer to the keyed mutex associated with the corresponding entries in `pReleaseSyncs`.

Valid Usage

- VUID-VkWin32KeyedMutexAcquireReleaseInfoKHR-pAcquireSyncs-00081
  Each member of `pAcquireSyncs` and `pReleaseSyncs` **must** be a device memory object

imported by setting VkImportMemoryWin32HandleInfoKHR::handleType to VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT or VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT

Valid Usage (Implicit)

- VUID-VkWin32KeyedMutexAcquireReleaseInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_WIN32_KEYED_MUTEX_ACQUIRE_RELEASE_INFO_KHR
- VUID-VkWin32KeyedMutexAcquireReleaseInfoKHR-pAcquireSyncs-parameter
  If acquireCount is not 0, pAcquireSyncs **must** be a valid pointer to an array of acquireCount valid VkDeviceMemory handles
- VUID-VkWin32KeyedMutexAcquireReleaseInfoKHR-pAcquireKeys-parameter
  If acquireCount is not 0, pAcquireKeys **must** be a valid pointer to an array of acquireCount uint64_t values
- VUID-VkWin32KeyedMutexAcquireReleaseInfoKHR-pAcquireTimeouts-parameter
  If acquireCount is not 0, pAcquireTimeouts **must** be a valid pointer to an array of acquireCount uint32_t values
- VUID-VkWin32KeyedMutexAcquireReleaseInfoKHR-pReleaseSyncs-parameter
  If releaseCount is not 0, pReleaseSyncs **must** be a valid pointer to an array of releaseCount valid VkDeviceMemory handles
- VUID-VkWin32KeyedMutexAcquireReleaseInfoKHR-pReleaseKeys-parameter
  If releaseCount is not 0, pReleaseKeys **must** be a valid pointer to an array of releaseCount uint64_t values
- VUID-VkWin32KeyedMutexAcquireReleaseInfoKHR-commonparent
  Both of the elements of pAcquireSyncs, and the elements of pReleaseSyncs that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

If the pNext chain of VkSubmitInfo includes a VkDeviceGroupSubmitInfo structure, then that structure includes device indices and masks specifying which physical devices execute semaphore operations and command buffers.

The VkDeviceGroupSubmitInfo structure is defined as:

```
typedef struct VkDeviceGroupSubmitInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           waitSemaphoreCount;
    const uint32_t*    pWaitSemaphoreDeviceIndices;
    uint32_t           commandBufferCount;
    const uint32_t*    pCommandBufferDeviceMasks;
    uint32_t           signalSemaphoreCount;
    const uint32_t*    pSignalSemaphoreDeviceIndices;
} VkDeviceGroupSubmitInfo;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkDeviceGroupSubmitInfo VkDeviceGroupSubmitInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `waitSemaphoreCount` is the number of elements in the `pWaitSemaphoreDeviceIndices` array.
- `pWaitSemaphoreDeviceIndices` is a pointer to an array of `waitSemaphoreCount` device indices indicating which physical device executes the semaphore wait operation in the corresponding element of VkSubmitInfo::`pWaitSemaphores`.
- `commandBufferCount` is the number of elements in the `pCommandBufferDeviceMasks` array.
- `pCommandBufferDeviceMasks` is a pointer to an array of `commandBufferCount` device masks indicating which physical devices execute the command buffer in the corresponding element of VkSubmitInfo::`pCommandBuffers`. A physical device executes the command buffer if the corresponding bit is set in the mask.
- `signalSemaphoreCount` is the number of elements in the `pSignalSemaphoreDeviceIndices` array.
- `pSignalSemaphoreDeviceIndices` is a pointer to an array of `signalSemaphoreCount` device indices indicating which physical device executes the semaphore signal operation in the corresponding element of VkSubmitInfo::`pSignalSemaphores`.

If this structure is not present, semaphore operations and command buffers execute on device index zero.

Valid Usage

- VUID-VkDeviceGroupSubmitInfo-waitSemaphoreCount-00082
  `waitSemaphoreCount` **must** equal VkSubmitInfo::`waitSemaphoreCount`
- VUID-VkDeviceGroupSubmitInfo-commandBufferCount-00083
  `commandBufferCount` **must** equal VkSubmitInfo::`commandBufferCount`
- VUID-VkDeviceGroupSubmitInfo-signalSemaphoreCount-00084
  `signalSemaphoreCount` **must** equal VkSubmitInfo::`signalSemaphoreCount`

- VUID-VkDeviceGroupSubmitInfo-pWaitSemaphoreDeviceIndices-00085
  All elements of `pWaitSemaphoreDeviceIndices` and `pSignalSemaphoreDeviceIndices` **must** be valid device indices
- VUID-VkDeviceGroupSubmitInfo-pCommandBufferDeviceMasks-00086
  All elements of `pCommandBufferDeviceMasks` **must** be valid device masks

Valid Usage (Implicit)

- VUID-VkDeviceGroupSubmitInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_GROUP_SUBMIT_INFO`
- VUID-VkDeviceGroupSubmitInfo-pWaitSemaphoreDeviceIndices-parameter
  If `waitSemaphoreCount` is not `0`, `pWaitSemaphoreDeviceIndices` **must** be a valid pointer to an array of `waitSemaphoreCount` `uint32_t` values
- VUID-VkDeviceGroupSubmitInfo-pCommandBufferDeviceMasks-parameter
  If `commandBufferCount` is not `0`, `pCommandBufferDeviceMasks` **must** be a valid pointer to an array of `commandBufferCount` `uint32_t` values
- VUID-VkDeviceGroupSubmitInfo-pSignalSemaphoreDeviceIndices-parameter
  If `signalSemaphoreCount` is not `0`, `pSignalSemaphoreDeviceIndices` **must** be a valid pointer to an array of `signalSemaphoreCount` `uint32_t` values

If the `pNext` chain of VkSubmitInfo includes a VkPerformanceQuerySubmitInfoKHR structure, then the structure indicates which counter pass is active for the batch in that submit.

The `VkPerformanceQuerySubmitInfoKHR` structure is defined as:

```
// Provided by VK_KHR_performance_query
typedef struct VkPerformanceQuerySubmitInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           counterPassIndex;
} VkPerformanceQuerySubmitInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `counterPassIndex` specifies which counter pass index is active.

If the `VkSubmitInfo`::`pNext` chain does not include this structure, the batch defaults to use counter pass index 0.

Valid Usage

- VUID-VkPerformanceQuerySubmitInfoKHR-counterPassIndex-03221
  `counterPassIndex` **must** be less than the number of counter passes required by any queries

within the batch. The required number of counter passes for a performance query is obtained by calling vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR

Valid Usage (Implicit)

- VUID-VkPerformanceQuerySubmitInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PERFORMANCE_QUERY_SUBMIT_INFO_KHR`

6.6. Queue Forward Progress

When using binary semaphores, the application **must** ensure that command buffer submissions will be able to complete without any subsequent operations by the application on any queue. After any call to `vkQueueSubmit` (or other queue operation), for every queued wait on a semaphore created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_BINARY` there **must** be a prior signal of that semaphore that will not be consumed by a different wait on the semaphore.

When using timeline semaphores, wait-before-signal behavior is well-defined and applications **can** submit work via `vkQueueSubmit` defining a timeline semaphore wait operation before submitting a corresponding semaphore signal operation. For each timeline semaphore wait operation defined by a call to `vkQueueSubmit`, the application **must** ensure that a corresponding semaphore signal operation is executed before forward progress can be made.

If a command buffer submission waits for any events to be signaled, the application **must** ensure that command buffer submissions will be able to complete without any subsequent operations by the application. Events signaled by the host **must** be signaled before the command buffer waits on those events.

NOTE

The ability for commands to wait on the host to set an events was originally added to allow low-latency updates to resources between host and device. However, to ensure quality of service, implementations would necessarily detect extended stalls in execution and timeout after a short period. As this period is not defined in the Vulkan specification, it is impossible to correctly validate any application with any wait period. Since the original users of this functionality were highly limited and platform-specific, this functionality is now considered defunct and should not be used.

6.7. Secondary Command Buffer Execution

Secondary command buffers **must** not be directly submitted to a queue. To record a secondary command buffer to execute as part of a primary command buffer, call:

```
// Provided by VK_VERSION_1_0
void vkCmdExecuteCommands(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    commandBufferCount,
    const VkCommandBuffer*                      pCommandBuffers);
```

- `commandBuffer` is a handle to a primary command buffer that the secondary command buffers are executed in.
- `commandBufferCount` is the length of the `pCommandBuffers` array.
- `pCommandBuffers` is a pointer to an array of `commandBufferCount` secondary command buffer handles, which are recorded to execute in the primary command buffer in the order they are listed in the array.

If any element of `pCommandBuffers` was not recorded with the `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT` flag, and it was recorded into any other primary command buffer which is currently in the executable or recording state, that primary command buffer becomes invalid.

If the `nestedCommandBuffer` feature is enabled it is valid usage for `vkCmdExecuteCommands` to also be recorded to a secondary command buffer.

Valid Usage

- VUID-vkCmdExecuteCommands-pCommandBuffers-00088
  Each element of `pCommandBuffers` **must** have been allocated with a `level` of `VK_COMMAND_BUFFER_LEVEL_SECONDARY`
- VUID-vkCmdExecuteCommands-pCommandBuffers-00089
  Each element of `pCommandBuffers` **must** be in the pending or executable state
- VUID-vkCmdExecuteCommands-pCommandBuffers-00091
  If any element of `pCommandBuffers` was not recorded with the `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT` flag, it **must** not be in the pending state
- VUID-vkCmdExecuteCommands-pCommandBuffers-00092
  If any element of `pCommandBuffers` was not recorded with the `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT` flag, it **must** not have already been recorded to `commandBuffer`
- VUID-vkCmdExecuteCommands-pCommandBuffers-00093
  If any element of `pCommandBuffers` was not recorded with the `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT` flag, it **must** not appear more than once in `pCommandBuffers`
- VUID-vkCmdExecuteCommands-pCommandBuffers-00094
  Each element of `pCommandBuffers` **must** have been allocated from a `VkCommandPool` that was created for the same queue family as the `VkCommandPool` from which `commandBuffer` was allocated
- VUID-vkCmdExecuteCommands-pCommandBuffers-00096

If vkCmdExecuteCommands is being called within a render pass instance, each element of pCommandBuffers **must** have been recorded with the VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT

- VUID-vkCmdExecuteCommands-pCommandBuffers-00099
  If vkCmdExecuteCommands is being called within a render pass instance, and any element of pCommandBuffers was recorded with VkCommandBufferInheritanceInfo::framebuffer not equal to VK_NULL_HANDLE, that VkFramebuffer **must** match the VkFramebuffer used in the current render pass instance
- VUID-vkCmdExecuteCommands-contents-09680
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderPass, and vkCmdNextSubpass has not been called in the current render pass instance, the contents parameter of vkCmdBeginRenderPass **must** have been VK_SUBPASS_CONTENTS_SECONDARY_COMMAND_BUFFERS , or VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_EXT
- VUID-vkCmdExecuteCommands-None-09681
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderPass, and vkCmdNextSubpass has been called in the current render pass instance, the contents parameter of the last call to vkCmdNextSubpass **must** have been VK_SUBPASS_CONTENTS_SECONDARY_COMMAND_BUFFERS , or VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_KHR
- VUID-vkCmdExecuteCommands-pCommandBuffers-06019
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderPass, each element of pCommandBuffers **must** have been recorded with VkCommandBufferInheritanceInfo::subpass set to the index of the subpass which the given command buffer will be executed in
- VUID-vkCmdExecuteCommands-pBeginInfo-06020
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderPass, the render passes specified in the pBeginInfo->pInheritanceInfo->renderPass members of the vkBeginCommandBuffer commands used to begin recording each element of pCommandBuffers **must** be compatible with the current render pass
- VUID-vkCmdExecuteCommands-pCommandBuffers-00100
  If vkCmdExecuteCommands is not being called within a render pass instance, each element of pCommandBuffers **must** not have been recorded with the VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT
- VUID-vkCmdExecuteCommands-commandBuffer-00101
  If the inheritedQueries feature is not enabled, commandBuffer **must** not have any queries active
- VUID-vkCmdExecuteCommands-commandBuffer-00102
  If commandBuffer has a VK_QUERY_TYPE_OCCLUSION query active, then each element of pCommandBuffers **must** have been recorded with VkCommandBufferInheritanceInfo::occlusionQueryEnable set to VK_TRUE
- VUID-vkCmdExecuteCommands-commandBuffer-00103
  If commandBuffer has a VK_QUERY_TYPE_OCCLUSION query active, then each element of pCommandBuffers **must** have been recorded with VkCommandBufferInheritanceInfo

::queryFlags having all bits set that are set for the query

- VUID-vkCmdExecuteCommands-commandBuffer-00104
  If commandBuffer has a VK_QUERY_TYPE_PIPELINE_STATISTICS query active, then each element of pCommandBuffers **must** have been recorded with VkCommandBufferInheritanceInfo ::pipelineStatistics having all bits set that are set in the VkQueryPool the query uses
- VUID-vkCmdExecuteCommands-pCommandBuffers-00105
  Each element of pCommandBuffers **must** not begin any query types that are active in commandBuffer
- VUID-vkCmdExecuteCommands-commandBuffer-07594
  commandBuffer **must** not have any queries other than VK_QUERY_TYPE_OCCLUSION and VK_QUERY_TYPE_PIPELINE_STATISTICS active
- VUID-vkCmdExecuteCommands-None-02286
  This command **must** not be recorded when transform feedback is active
- VUID-vkCmdExecuteCommands-commandBuffer-06533
  If vkCmdExecuteCommands is being called within a render pass instance and any recorded command in commandBuffer in the current subpass will write to an image subresource as an attachment, commands recorded in elements of pCommandBuffers **must** not read from the memory backing that image subresource in any other way
- VUID-vkCmdExecuteCommands-commandBuffer-06534
  If vkCmdExecuteCommands is being called within a render pass instance and any recorded command in commandBuffer in the current subpass will read from an image subresource used as an attachment in any way other than as an attachment, commands recorded in elements of pCommandBuffers **must** not write to that image subresource as an attachment
- VUID-vkCmdExecuteCommands-pCommandBuffers-06535
  If vkCmdExecuteCommands is being called within a render pass instance and any recorded command in a given element of pCommandBuffers will write to an image subresource as an attachment, commands recorded in elements of pCommandBuffers at a higher index **must** not read from the memory backing that image subresource in any other way
- VUID-vkCmdExecuteCommands-pCommandBuffers-06536
  If vkCmdExecuteCommands is being called within a render pass instance and any recorded command in a given element of pCommandBuffers will read from an image subresource used as an attachment in any way other than as an attachment, commands recorded in elements of pCommandBuffers at a higher index **must** not write to that image subresource as an attachment
- VUID-vkCmdExecuteCommands-pCommandBuffers-06021
  If pCommandBuffers contains any suspended render pass instances, there **must** be no action or synchronization commands between that render pass instance and any render pass instance that resumes it
- VUID-vkCmdExecuteCommands-pCommandBuffers-06022
  If pCommandBuffers contains any suspended render pass instances, there **must** be no render pass instances between that render pass instance and any render pass instance that resumes it
- VUID-vkCmdExecuteCommands-variableSampleLocations-06023

If the variableSampleLocations limit is not supported, and any element of pCommandBuffers contains any suspended render pass instances, where a graphics pipeline has been bound, any pipelines bound in the render pass instance that resumes it, or any subsequent render pass instances that resume from that one and so on, **must** use the same sample locations

- VUID-vkCmdExecuteCommands-flags-06024
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, its VkRenderingInfo::flags parameter **must** have included VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT
- VUID-vkCmdExecuteCommands-pBeginInfo-06025
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, the render passes specified in the pBeginInfo->pInheritanceInfo->renderPass members of the vkBeginCommandBuffer commands used to begin recording each element of pCommandBuffers **must** be VK_NULL_HANDLE
- VUID-vkCmdExecuteCommands-flags-06026
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, the flags member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be equal to the VkRenderingInfo::flags parameter to vkCmdBeginRenderingKHR, excluding VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT
- VUID-vkCmdExecuteCommands-colorAttachmentCount-06027
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, the colorAttachmentCount member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be equal to the VkRenderingInfo::colorAttachmentCount parameter to vkCmdBeginRenderingKHR
- VUID-vkCmdExecuteCommands-imageView-06028
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, if the imageView member of an element of the VkRenderingInfo::pColorAttachments parameter to vkCmdBeginRenderingKHR is not VK_NULL_HANDLE, the corresponding element of the pColorAttachmentFormats member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be equal to the format used to create that image view
- VUID-vkCmdExecuteCommands-imageView-07606
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, if the imageView member of an element of the VkRenderingInfo::pColorAttachments parameter to vkCmdBeginRenderingKHR is VK_NULL_HANDLE, the corresponding element of the pColorAttachmentFormats member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be VK_FORMAT_UNDEFINED

- VUID-vkCmdExecuteCommands-pDepthAttachment-06029
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, if the VkRenderingInfo::pDepthAttachment->imageView parameter to vkCmdBeginRenderingKHR is not VK_NULL_HANDLE, the value of the depthAttachmentFormat member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be equal to the format used to create that image view
- VUID-vkCmdExecuteCommands-pStencilAttachment-06030
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, if the VkRenderingInfo::pStencilAttachment->imageView parameter to vkCmdBeginRenderingKHR is not VK_NULL_HANDLE, the value of the stencilAttachmentFormat member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be equal to the format used to create that image view
- VUID-vkCmdExecuteCommands-pDepthAttachment-06774
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR and the VkRenderingInfo::pDepthAttachment->imageView parameter to vkCmdBeginRenderingKHR was VK_NULL_HANDLE, the value of the depthAttachmentFormat member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteCommands-pStencilAttachment-06775
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR and the VkRenderingInfo::pStencilAttachment->imageView parameter to vkCmdBeginRenderingKHR was VK_NULL_HANDLE, the value of the stencilAttachmentFormat member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteCommands-viewMask-06031
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, the viewMask member of the VkCommandBufferInheritanceRenderingInfo structure included in the pNext chain of VkCommandBufferBeginInfo::pInheritanceInfo used to begin recording each element of pCommandBuffers **must** be equal to the VkRenderingInfo::viewMask parameter to vkCmdBeginRenderingKHR
- VUID-vkCmdExecuteCommands-commandBuffer-09375
  commandBuffer **must** not be a secondary command buffer unless the nestedCommandBuffer feature is enabled
- VUID-vkCmdExecuteCommands-nestedCommandBuffer-09376
  If the nestedCommandBuffer feature is enabled, the command buffer nesting level of each element of pCommandBuffers **must** be less than maxCommandBufferNestingLevel
- VUID-vkCmdExecuteCommands-nestedCommandBufferRendering-09377
  If the nestedCommandBufferRendering feature is not enabled, and commandBuffer is a

secondary command buffer, commandBuffer **must** not have been recorded with VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT

- VUID-vkCmdExecuteCommands-nestedCommandBufferSimultaneousUse-09378
  If the nestedCommandBufferSimultaneousUse feature is not enabled, and commandBuffer is a secondary command buffer, each element of pCommandBuffers **must** not have been recorded with VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT
- VUID-vkCmdExecuteCommands-pCommandBuffers-09504
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, the color attachment mapping state specified by VkRenderingAttachmentLocationInfoKHR in the inheritance info of each element of pCommandBuffers and in the current state of commandBuffer **must** match
- VUID-vkCmdExecuteCommands-pCommandBuffers-09505
  If vkCmdExecuteCommands is being called within a render pass instance begun with vkCmdBeginRenderingKHR, the input attachment mapping state specified by VkRenderingInputAttachmentIndexInfoKHR in the inheritance info of each element of pCommandBuffers and in the current state of commandBuffer **must** match

Valid Usage (Implicit)

- VUID-vkCmdExecuteCommands-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdExecuteCommands-pCommandBuffers-parameter
  pCommandBuffers **must** be a valid pointer to an array of commandBufferCount valid VkCommandBuffer handles
- VUID-vkCmdExecuteCommands-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdExecuteCommands-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdExecuteCommands-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdExecuteCommands-commandBufferCount-arraylength
  commandBufferCount **must** be greater than 0
- VUID-vkCmdExecuteCommands-commonparent
  Both of commandBuffer, and the elements of pCommandBuffers **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Transfer Graphics Compute	Indirection

6.8. Nested Command Buffers

In addition to secondary command buffer execution from primary command buffers, an implementation **may** support nested command buffers, which enable secondary command buffers to be executed from other secondary command buffers. If the `nestedCommandBuffer` feature is enabled, the implementation supports nested command buffers.

Nested command buffer execution works the same as primary-to-secondary execution, except that it is subject to some additional implementation-defined limits.

Each secondary command buffer has a command buffer nesting level, which is determined at vkEndCommandBuffer time and evaluated at vkCmdExecuteCommands time. A secondary command buffer that executes no other secondary command buffers has a command buffer nesting level of zero. Otherwise, the command buffer nesting level of a secondary command buffer is equal to the maximum nesting level of all secondary command buffers executed by that command buffer plus one. Some implementations **may** have a limit on the maximum nesting level of secondary command buffers that **can** be recorded. This limit is advertised in `maxCommandBufferNestingLevel`.

If the `nestedCommandBufferRendering` feature is enabled, the implementation supports calling vkCmdExecuteCommands inside secondary command buffers recorded with `VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT`. If the `nestedCommandBufferSimultaneousUse` feature is enabled, the implementation supports calling vkCmdExecuteCommands with secondary command buffers recorded with `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT`.

Whenever vkCmdExecuteCommands is recorded inside a secondary command buffer recorded with `VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT`, each member of `pCommandBuffers` **must** have been recorded with a VkCommandBufferBeginInfo with VkCommandBufferInheritanceInfo compatible with the VkCommandBufferInheritanceInfo of the command buffer into which the vkCmdExecuteCommands call is being recorded. The VkCommandBufferInheritanceRenderingInfo structures are compatible when the `VkCommandBufferInheritanceRenderingInfo::renderpass` are compatible, or if they are VK_NULL_HANDLE then the VkCommandBufferInheritanceRenderingInfo members match, and all other members of `VkCommandBufferInheritanceRenderingInfo` match. This requirement applies recursively, down to the most nested command buffer and up to the command buffer where the render pass was originally begun.

6.9. Command Buffer Device Mask

Each command buffer has a piece of state storing the current device mask of the command buffer. This mask controls which physical devices within the logical device all subsequent commands will execute on, including state-setting commands, action commands, and synchronization commands.

Scissor and viewport state **can** be different values on each physical device (only when set as dynamic state), and each physical device will render using its local copy of the state. Other state is shared between physical devices, such that all physical devices use the most recently set values for the state. However, when recording an action command that uses a piece of state, the most recent command that set that state **must** have included all physical devices that execute the action command in its current device mask.

The command buffer's device mask is orthogonal to the `pCommandBufferDeviceMasks` member of VkDeviceGroupSubmitInfo. Commands only execute on a physical device if the device index is set in both device masks.

If the `pNext` chain of VkCommandBufferBeginInfo includes a `VkDeviceGroupCommandBufferBeginInfo` structure, then that structure includes an initial device mask for the command buffer.

The `VkDeviceGroupCommandBufferBeginInfo` structure is defined as:

```
typedef struct VkDeviceGroupCommandBufferBeginInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           deviceMask;
} VkDeviceGroupCommandBufferBeginInfo;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkDeviceGroupCommandBufferBeginInfo VkDeviceGroupCommandBufferBeginInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `deviceMask` is the initial value of the command buffer's device mask.

The initial device mask also acts as an upper bound on the set of devices that **can** ever be in the device mask in the command buffer.

If this structure is not present, the initial value of a command buffer's device mask includes all physical devices in the logical device when the command buffer begins recording.

Valid Usage

- VUID-VkDeviceGroupCommandBufferBeginInfo-deviceMask-00106

`deviceMask` **must** be a valid device mask value

- VUID-VkDeviceGroupCommandBufferBeginInfo-deviceMask-00107
  `deviceMask` **must** not be zero

Valid Usage (Implicit)

- VUID-VkDeviceGroupCommandBufferBeginInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_GROUP_COMMAND_BUFFER_BEGIN_INFO`

To update the current device mask of a command buffer, call:

```
// Provided by VK_KHR_device_group
void vkCmdSetDeviceMaskKHR(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    deviceMask);
```

- `commandBuffer` is command buffer whose current device mask is modified.
- `deviceMask` is the new value of the current device mask.

`deviceMask` is used to filter out subsequent commands from executing on all physical devices whose bit indices are not set in the mask, except commands beginning a render pass instance, commands transitioning to the next subpass in the render pass instance, and commands ending a render pass instance, which always execute on the set of physical devices whose bit indices are included in the `deviceMask` member of the VkDeviceGroupRenderPassBeginInfo structure passed to the command beginning the corresponding render pass instance.

Valid Usage

- VUID-vkCmdSetDeviceMask-deviceMask-00108
  `deviceMask` **must** be a valid device mask value
- VUID-vkCmdSetDeviceMask-deviceMask-00109
  `deviceMask` **must** not be zero
- VUID-vkCmdSetDeviceMask-deviceMask-00110
  `deviceMask` **must** not include any set bits that were not in the VkDeviceGroupCommandBufferBeginInfo::`deviceMask` value when the command buffer began recording
- VUID-vkCmdSetDeviceMask-deviceMask-00111
  If `vkCmdSetDeviceMaskKHR` is called inside a render pass instance, `deviceMask` **must** not include any set bits that were not in the VkDeviceGroupRenderPassBeginInfo::`deviceMask` value when the render pass instance began recording

Valid Usage (Implicit)

- VUID-vkCmdSetDeviceMask-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDeviceMask-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDeviceMask-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, or transfer operations

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Graphics Compute Transfer	State

Chapter 7. Synchronization and Cache Control

Synchronization of access to resources is primarily the responsibility of the application in Vulkan. The order of execution of commands with respect to the host and other commands on the device has few implicit guarantees, and needs to be explicitly specified. Memory caches and other optimizations are also explicitly managed, requiring that the flow of data through the system is largely under application control.

Whilst some implicit guarantees exist between commands, five explicit synchronization mechanisms are exposed by Vulkan:

Fences

Fences **can** be used to communicate to the host that execution of some task on the device has completed, controlling resource access between host and device.

Semaphores

Semaphores **can** be used to control resource access across multiple queues.

Events

Events provide a fine-grained synchronization primitive which **can** be signaled either within a command buffer or by the host, and **can** be waited upon within a command buffer or queried on the host. Events **can** be used to control resource access within a single queue.

Pipeline Barriers

Pipeline barriers also provide synchronization control within a command buffer, but at a single point, rather than with separate signal and wait operations. Pipeline barriers **can** be used to control resource access within a single queue.

Render Pass Objects

Render pass objects provide a synchronization framework for rendering tasks, built upon the concepts in this chapter. Many cases that would otherwise need an application to use other synchronization primitives **can** be expressed more efficiently as part of a render pass. Render pass objects **can** be used to control resource access within a single queue.

7.1. Execution and Memory Dependencies

An *operation* is an arbitrary amount of work to be executed on the host, a device, or an external entity such as a presentation engine. Synchronization commands introduce explicit *execution dependencies*, and *memory dependencies* between two sets of operations defined by the command's two *synchronization scopes*.

The synchronization scopes define which other operations a synchronization command is able to create execution dependencies with. Any type of operation that is not in a synchronization command's synchronization scopes will not be included in the resulting dependency. For example, for many synchronization commands, the synchronization scopes **can** be limited to just operations executing in specific pipeline stages, which allows other pipeline stages to be excluded from a

dependency. Other scoping options are possible, depending on the particular command.

An *execution dependency* is a guarantee that for two sets of operations, the first set **must** *happen-before* the second set. If an operation happens-before another operation, then the first operation **must** complete before the second operation is initiated. More precisely:

- Let $\textbf{Ops}_1$ and $\textbf{Ops}_2$ be separate sets of operations.
- Let **Sync** be a synchronization command.
- Let $\textbf{Scope}_{1st}$ and $\textbf{Scope}_{2nd}$ be the synchronization scopes of **Sync**.
- Let $\textbf{ScopedOps}_1$ be the intersection of sets $\textbf{Ops}_1$ and $\textbf{Scope}_{1st}$.
- Let $\textbf{ScopedOps}_2$ be the intersection of sets $\textbf{Ops}_2$ and $\textbf{Scope}_{2nd}$.
- Submitting $\textbf{Ops}_1$, **Sync** and $\textbf{Ops}_2$ for execution, in that order, will result in execution dependency **ExeDep** between $\textbf{ScopedOps}_1$ and $\textbf{ScopedOps}_2$.
- Execution dependency **ExeDep** guarantees that $\textbf{ScopedOps}_1$ happen-before $\textbf{ScopedOps}_2$.

An *execution dependency chain* is a sequence of execution dependencies that form a happens-before relation between the first dependency's $\textbf{ScopedOps}_1$ and the final dependency's $\textbf{ScopedOps}_2$. For each consecutive pair of execution dependencies, a chain exists if the intersection of $\textbf{Scope}_{2nd}$ in the first dependency and $\textbf{Scope}_{1st}$ in the second dependency is not an empty set. The formation of a single execution dependency from an execution dependency chain can be described by substituting the following in the description of execution dependencies:

- Let **Sync** be a set of synchronization commands that generate an execution dependency chain.
- Let $\textbf{Scope}_{1st}$ be the first synchronization scope of the first command in **Sync**.
- Let $\textbf{Scope}_{2nd}$ be the second synchronization scope of the last command in **Sync**.

Execution dependencies alone are not sufficient to guarantee that values resulting from writes in one set of operations **can** be read from another set of operations.

Three additional types of operations are used to control memory access. *Availability operations* cause the values generated by specified memory write accesses to become *available* to a memory domain for future access. Any available value remains available until a subsequent write to the same memory location occurs (whether it is made available or not) or the memory is freed. *Memory domain operations* cause writes that are available to a source memory domain to become available to a destination memory domain (an example of this is making writes available to the host domain available to the device domain). *Visibility operations* cause values available to a memory domain to become *visible* to specified memory accesses.

Availability, visibility, memory domains, and memory domain operations are formally defined in the Availability and Visibility section of the Memory Model chapter. Which API operations perform each of these operations is defined in Availability, Visibility, and Domain Operations.

A *memory dependency* is an execution dependency which includes availability and visibility operations such that:

- The first set of operations happens-before the availability operation.

- The availability operation happens-before the visibility operation.
- The visibility operation happens-before the second set of operations.

Once written values are made visible to a particular type of memory access, they **can** be read or written by that type of memory access. Most synchronization commands in Vulkan define a memory dependency.

The specific memory accesses that are made available and visible are defined by the *access scopes* of a memory dependency. Any type of access that is in a memory dependency's first access scope and occurs in $\textbf{ScopedOps}_1$ is made available. Any type of access that is in a memory dependency's second access scope and occurs in $\textbf{ScopedOps}_2$ has any available writes made visible to it. Any type of operation that is not in a synchronization command's access scopes will not be included in the resulting dependency.

A memory dependency enforces availability and visibility of memory accesses and execution order between two sets of operations. Adding to the description of execution dependency chains:

- Let $\textbf{MemOps}_1$ be the set of memory accesses performed by $\textbf{ScopedOps}_1$.
- Let $\textbf{MemOps}_2$ be the set of memory accesses performed by $\textbf{ScopedOps}_2$.
- Let $\textbf{AccessScope}_{1st}$ be the first access scope of the first command in the **Sync** chain.
- Let $\textbf{AccessScope}_{2nd}$ be the second access scope of the last command in the **Sync** chain.
- Let $\textbf{ScopedMemOps}_1$ be the intersection of sets $\textbf{MemOps}_1$ and $\textbf{AccessScope}_{1st}$.
- Let $\textbf{ScopedMemOps}_2$ be the intersection of sets $\textbf{MemOps}_2$ and $\textbf{AccessScope}_{2nd}$.
- Submitting $\textbf{Ops}_1$, **Sync**, and $\textbf{Ops}_2$ for execution, in that order, will result in a memory dependency **MemDep** between $\textbf{ScopedOps}_1$ and $\textbf{ScopedOps}_2$.
- Memory dependency **MemDep** guarantees that:
  - Memory writes in $\textbf{ScopedMemOps}_1$ are made available.
  - Available memory writes, including those from $\textbf{ScopedMemOps}_1$, are made visible to $\textbf{ScopedMemOps}_2$.

NOTE

Execution and memory dependencies are used to solve data hazards, i.e. to ensure that read and write operations occur in a well-defined order. Write-after-read hazards can be solved with just an execution dependency, but read-after-write and write-after-write hazards need appropriate memory dependencies to be included between them. If an application does not include dependencies to solve these hazards, the results and execution orders of memory accesses are undefined.

7.1.1. Image Layout Transitions

Image subresources **can** be transitioned from one layout to another as part of a memory dependency (e.g. by using an image memory barrier). When a layout transition is specified in a memory dependency, it happens-after the availability operations in the memory dependency, and happens-before the visibility operations. Image layout transitions **may** perform read and write accesses on all memory bound to the image subresource range, so applications **must** ensure that all memory writes have been made available before a layout transition is executed. Available memory

is automatically made visible to a layout transition, and writes performed by a layout transition are automatically made available.

Layout transitions always apply to a particular image subresource range, and specify both an old layout and new layout. The old layout **must** either be VK_IMAGE_LAYOUT_UNDEFINED, or match the current layout of the image subresource range. If the old layout matches the current layout of the image subresource range, the transition preserves the contents of that range. If the old layout is VK_IMAGE_LAYOUT_UNDEFINED, the contents of that range **may** be discarded.

NOTE

Image layout transitions with VK_IMAGE_LAYOUT_UNDEFINED allow the implementation to discard the image subresource range, which can provide performance or power benefits. Tile-based architectures may be able to avoid flushing tile data to memory, and immediate style renderers may be able to achieve fast metadata clears to reinitialize frame buffer compression state, or similar.

If the contents of an attachment are not needed after a render pass completes, then applications **should** use VK_ATTACHMENT_STORE_OP_DONT_CARE.

As image layout transitions **may** perform read and write accesses on the memory bound to the image, if the image subresource affected by the layout transition is bound to peer memory for any device in the current device mask then the memory heap the bound memory comes from **must** support the VK_PEER_MEMORY_FEATURE_GENERIC_SRC_BIT and VK_PEER_MEMORY_FEATURE_GENERIC_DST_BIT capabilities as returned by vkGetDeviceGroupPeerMemoryFeaturesKHR.

NOTE

Applications **must** ensure that layout transitions happen-after all operations accessing the image with the old layout, and happen-before any operations that will access the image with the new layout. Layout transitions are potentially read/write operations, so not defining appropriate memory dependencies to guarantee this will result in a data race.

Image layout transitions interact with memory aliasing.

Layout transitions that are performed via image memory barriers execute in their entirety in submission order, relative to other image layout transitions submitted to the same queue, including those performed by render passes. In effect there is an implicit execution dependency from each such layout transition to all layout transitions previously submitted to the same queue.

The image layout of each image subresource of a depth/stencil image created with VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT is dependent on the last sample locations used to render to the image subresource as a depth/stencil attachment, thus when the image member of an image memory barrier is an image created with this flag the application **can** chain a VkSampleLocationsInfoEXT structure to the pNext chain of VkImageMemoryBarrier2 or VkImageMemoryBarrier to specify the sample locations to use during any image layout transition.

If the VkSampleLocationsInfoEXT structure does not match the sample location state last used to render to the image subresource range specified by subresourceRange, or if no VkSampleLocationsInfoEXT structure is present, then the contents of the given image subresource range becomes undefined as if oldLayout would equal VK_IMAGE_LAYOUT_UNDEFINED.

7.1.2. Pipeline Stages

The work performed by an action command consists of multiple operations, which are performed as a sequence of logically independent steps known as *pipeline stages*. The exact pipeline stages executed depend on the particular command that is used, and current command buffer state when the command was recorded.

NOTE

Operations performed by synchronization commands (e.g. availability and visibility operations) are not executed by a defined pipeline stage. However other commands can still synchronize with them by using the synchronization scopes to create a dependency chain.

Execution of operations across pipeline stages **must** adhere to implicit ordering guarantees, particularly including pipeline stage order. Otherwise, execution across pipeline stages **may** overlap or execute out of order with regards to other stages, unless otherwise enforced by an execution dependency.

Several of the synchronization commands include pipeline stage parameters, restricting the synchronization scopes for that command to just those stages. This allows fine grained control over the exact execution dependencies and accesses performed by action commands. Implementations **should** use these pipeline stages to avoid unnecessary stalls or cache flushing.

Bits which **can** be set in a VkPipelineStageFlags2 mask, specifying stages of execution, are:

```
// Flag bits for VkPipelineStageFlagBits2
typedef VkFlags64 VkPipelineStageFlagBits2;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_NONE = 0ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_NONE_KHR = 0ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_TOP_OF_PIPE_BIT =
0x00000001ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_TOP_OF_PIPE_BIT_KHR =
0x00000001ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT =
0x00000002ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT_KHR =
0x00000002ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT =
0x00000004ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT_KHR =
0x00000004ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT =
0x00000008ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT_KHR =
0x00000008ULL;
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT = 0x00000010ULL;
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT_KHR = 0x00000010ULL;
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT = 0x00000020ULL;
```

```
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT_KHR = 0x00000020ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT =
0x00000040ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT_KHR =
0x00000040ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT =
0x00000080ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT_KHR =
0x00000080ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT =
0x00000100ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT_KHR
= 0x00000100ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT =
0x00000200ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT_KHR
= 0x00000200ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT
= 0x00000400ULL;
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT_KHR = 0x00000400ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT =
0x00000800ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT_KHR =
0x00000800ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT =
0x00001000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT_KHR =
0x00001000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_TRANSFER_BIT =
0x00001000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_TRANSFER_BIT_KHR =
0x00001000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_BOTTOM_OF_PIPE_BIT =
0x00002000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_BOTTOM_OF_PIPE_BIT_KHR =
0x00002000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_HOST_BIT = 0x00004000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_HOST_BIT_KHR =
0x00004000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT =
0x00008000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT_KHR =
0x00008000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT =
0x00010000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT_KHR =
0x00010000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_COPY_BIT = 0x100000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_COPY_BIT_KHR =
```

```
0x100000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_RESOLVE_BIT =
0x200000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_RESOLVE_BIT_KHR =
0x200000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_BLIT_BIT = 0x400000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_BLIT_BIT_KHR =
0x400000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_CLEAR_BIT = 0x800000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_CLEAR_BIT_KHR =
0x800000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT =
0x1000000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT_KHR =
0x1000000000ULL;
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT =
0x2000000000ULL;
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT_KHR = 0x2000000000ULL;
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_PRE_RASTERIZATION_SHADERS_BIT = 0x4000000000ULL;
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_PRE_RASTERIZATION_SHADERS_BIT_KHR = 0x4000000000ULL;
// Provided by VK_KHR_video_decode_queue
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR =
0x04000000ULL;
// Provided by VK_KHR_video_encode_queue
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR =
0x08000000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_transform_feedback
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT =
0x01000000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_conditional_rendering
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_CONDITIONAL_RENDERING_BIT_EXT = 0x00040000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_device_generated_commands
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_NV =
0x00020000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_device_generated_commands
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT =
0x00020000ULL;
// Provided by VK_KHR_fragment_shading_rate with VK_KHR_synchronization2
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR = 0x00400000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_shading_rate_image
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_SHADING_RATE_IMAGE_BIT_NV =
0x00400000ULL;
// Provided by VK_KHR_acceleration_structure with VK_KHR_synchronization2
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR = 0x02000000ULL;
// Provided by VK_KHR_ray_tracing_pipeline with VK_KHR_synchronization2
```

```
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR =
0x00200000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_ray_tracing
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_NV =
0x00200000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_ray_tracing
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_NV = 0x02000000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_fragment_density_map
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT = 0x00800000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_mesh_shader
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_TASK_SHADER_BIT_NV =
0x00080000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_mesh_shader
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_MESH_SHADER_BIT_NV =
0x00100000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_mesh_shader
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_TASK_SHADER_BIT_EXT =
0x00080000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_mesh_shader
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_MESH_SHADER_BIT_EXT =
0x00100000ULL;
// Provided by VK_KHR_ray_tracing_maintenance1 with VK_KHR_synchronization2 or
VK_VERSION_1_3
static const VkPipelineStageFlagBits2
VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR = 0x10000000ULL;
// Provided by VK_EXT_opacity_micromap
static const VkPipelineStageFlagBits2 VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT =
0x40000000ULL;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkPipelineStageFlagBits2 VkPipelineStageFlagBits2KHR;
```

- VK_PIPELINE_STAGE_2_NONE specifies no stages of execution.
- VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT specifies the stage of the pipeline where indirect command parameters are consumed. This stage also includes reading commands written by vkCmdPreprocessGeneratedCommandsEXT.
- VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT specifies the stage of the pipeline where index buffers are consumed.
- VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT specifies the stage of the pipeline where vertex buffers are consumed.
- VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT is equivalent to the logical OR of:
  - VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT

    - VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT
- VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT specifies the vertex shader stage.
- VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT specifies the tessellation control shader stage.
- VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT specifies the tessellation evaluation shader stage.
- VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT specifies the geometry shader stage.
- VK_PIPELINE_STAGE_2_PRE_RASTERIZATION_SHADERS_BIT is equivalent to specifying all supported pre-rasterization shader stages:
    - VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT
    - VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT
    - VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT
    - VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT
- VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT specifies the fragment shader stage.
- VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT specifies the stage of the pipeline where early fragment tests (depth and stencil tests before fragment shading) are performed. This stage also includes render pass load operations for framebuffer attachments with a depth/stencil format.
- VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT specifies the stage of the pipeline where late fragment tests (depth and stencil tests after fragment shading) are performed. This stage also includes render pass store operations for framebuffer attachments with a depth/stencil format.
- VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT specifies the stage of the pipeline where final color values are output from the pipeline. This stage includes blending, logic operations, render pass load and store operations for color attachments, render pass multisample resolve operations, and vkCmdClearAttachments.
- VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT specifies the compute shader stage.
- VK_PIPELINE_STAGE_2_HOST_BIT specifies a pseudo-stage indicating execution on the host of reads/writes of device memory. This stage is not invoked by any commands recorded in a command buffer.
- VK_PIPELINE_STAGE_2_COPY_BIT specifies the execution of all copy commands, including vkCmdCopyQueryPoolResults.
- VK_PIPELINE_STAGE_2_BLIT_BIT specifies the execution of vkCmdBlitImage.
- VK_PIPELINE_STAGE_2_RESOLVE_BIT specifies the execution of vkCmdResolveImage.
- VK_PIPELINE_STAGE_2_CLEAR_BIT specifies the execution of clear commands, with the exception of vkCmdClearAttachments.
- VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT is equivalent to specifying all of:
    - VK_PIPELINE_STAGE_2_COPY_BIT
    - VK_PIPELINE_STAGE_2_BLIT_BIT
    - VK_PIPELINE_STAGE_2_RESOLVE_BIT

    - VK_PIPELINE_STAGE_2_CLEAR_BIT
    - VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR
- VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR specifies the execution of the ray tracing shader stages.
- VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR specifies the execution of acceleration structure commands or acceleration structure copy commands.
- VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR specifies the execution of acceleration structure copy commands.
- VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT specifies the execution of all graphics pipeline stages, and is equivalent to the logical OR of:
    - VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT
    - VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT
    - VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT
    - VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT
    - VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT
    - VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT
    - VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT
    - VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT
    - VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT
    - VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT
    - VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT
    - VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
    - VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT specifies all operations performed by all commands supported on the queue it is used with.
- VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT specifies the stage of the pipeline where vertex attribute output values are written to the transform feedback buffers.
- VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT specifies the stage of the pipeline where device-side generation of commands via vkCmdPreprocessGeneratedCommandsEXT is handled.
- VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR specifies the stage of the pipeline where the fragment shading rate attachment is read to determine the fragment shading rate for portions of a rasterized primitive.
- VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT specifies the stage of the pipeline where the fragment density map is read to generate the fragment areas.
- VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR specifies the execution of video decode operations.
- VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR specifies the execution of video encode operations.
- VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT specifies the execution of micromap commands.

- VK_PIPELINE_STAGE_2_TOP_OF_PIPE_BIT is equivalent to VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT with VkAccessFlags2 set to 0 when specified in the second synchronization scope, but equivalent to VK_PIPELINE_STAGE_2_NONE in the first scope.
- VK_PIPELINE_STAGE_2_BOTTOM_OF_PIPE_BIT is equivalent to VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT with VkAccessFlags2 set to 0 when specified in the first synchronization scope, but equivalent to VK_PIPELINE_STAGE_2_NONE in the second scope.

NOTE	The TOP and BOTTOM pipeline stages are deprecated, and applications should prefer VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT and VK_PIPELINE_STAGE_2_NONE.

NOTE	The VkPipelineStageFlags2 bitmask goes beyond the 31 individual bit flags allowable within a C99 enum, which is how VkPipelineStageFlagBits is defined. The first 31 values are common to both, and are interchangeable.

VkPipelineStageFlags2 is a bitmask type for setting a mask of zero or more VkPipelineStageFlagBits2 flags:

```
typedef VkFlags64 VkPipelineStageFlags2;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkPipelineStageFlags2 VkPipelineStageFlags2KHR;
```

Bits which **can** be set in a VkPipelineStageFlags mask, specifying stages of execution, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkPipelineStageFlagBits {
    VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT = 0x00000001,
    VK_PIPELINE_STAGE_DRAW_INDIRECT_BIT = 0x00000002,
    VK_PIPELINE_STAGE_VERTEX_INPUT_BIT = 0x00000004,
    VK_PIPELINE_STAGE_VERTEX_SHADER_BIT = 0x00000008,
    VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT = 0x00000010,
    VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT = 0x00000020,
    VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT = 0x00000040,
    VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT = 0x00000080,
    VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT = 0x00000100,
    VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT = 0x00000200,
    VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT = 0x00000400,
    VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT = 0x00000800,
    VK_PIPELINE_STAGE_TRANSFER_BIT = 0x00001000,
    VK_PIPELINE_STAGE_BOTTOM_OF_PIPE_BIT = 0x00002000,
    VK_PIPELINE_STAGE_HOST_BIT = 0x00004000,
    VK_PIPELINE_STAGE_ALL_GRAPHICS_BIT = 0x00008000,
    VK_PIPELINE_STAGE_ALL_COMMANDS_BIT = 0x00010000,
    VK_PIPELINE_STAGE_NONE = 0,
  // Provided by VK_EXT_transform_feedback
    VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT = 0x01000000,
  // Provided by VK_KHR_acceleration_structure
    VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR = 0x02000000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR = 0x00200000,
  // Provided by VK_EXT_fragment_density_map
    VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT = 0x00800000,
  // Provided by VK_KHR_fragment_shading_rate
    VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR = 0x00400000,
    VK_PIPELINE_STAGE_COMMAND_PREPROCESS_BIT_NV = 0x00020000,
  // Provided by VK_KHR_synchronization2
    VK_PIPELINE_STAGE_NONE_KHR = VK_PIPELINE_STAGE_NONE,
  // Provided by VK_EXT_device_generated_commands
    VK_PIPELINE_STAGE_COMMAND_PREPROCESS_BIT_EXT =
VK_PIPELINE_STAGE_COMMAND_PREPROCESS_BIT_NV,
} VkPipelineStageFlagBits;
```

These values all have the same meaning as the equivalently named values for VkPipelineStageFlags2.

- VK_PIPELINE_STAGE_NONE specifies no stages of execution.
- VK_PIPELINE_STAGE_DRAW_INDIRECT_BIT specifies the stage of the pipeline where VkDrawIndirect* / VkDispatchIndirect* / VkTraceRaysIndirect* data structures are consumed. This stage also includes reading commands written by vkCmdExecuteGeneratedCommandsEXT.
- VK_PIPELINE_STAGE_VERTEX_INPUT_BIT specifies the stage of the pipeline where vertex and index buffers are consumed.
- VK_PIPELINE_STAGE_VERTEX_SHADER_BIT specifies the vertex shader stage.

- VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT specifies the tessellation control shader stage.
- VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT specifies the tessellation evaluation shader stage.
- VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT specifies the geometry shader stage.
- VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT specifies the fragment shader stage.
- VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT specifies the stage of the pipeline where early fragment tests (depth and stencil tests before fragment shading) are performed. This stage also includes render pass load operations for framebuffer attachments with a depth/stencil format.
- VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT specifies the stage of the pipeline where late fragment tests (depth and stencil tests after fragment shading) are performed. This stage also includes render pass store operations for framebuffer attachments with a depth/stencil format.
- VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT specifies the stage of the pipeline after blending where the final color values are output from the pipeline. This stage includes blending, logic operations, render pass load and store operations for color attachments, render pass multisample resolve operations, and vkCmdClearAttachments.
- VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT specifies the execution of a compute shader.
- VK_PIPELINE_STAGE_TRANSFER_BIT specifies the following commands:
  - All copy commands, including vkCmdCopyQueryPoolResults
  - vkCmdBlitImage2KHR and vkCmdBlitImage
  - vkCmdResolveImage2KHR and vkCmdResolveImage
  - All clear commands, with the exception of vkCmdClearAttachments
- VK_PIPELINE_STAGE_HOST_BIT specifies a pseudo-stage indicating execution on the host of reads/writes of device memory. This stage is not invoked by any commands recorded in a command buffer.
- VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR specifies the execution of vkCmdBuildAccelerationStructuresKHR, vkCmdBuildAccelerationStructuresIndirectKHR, vkCmdCopyAccelerationStructureKHR, vkCmdCopyAccelerationStructureToMemoryKHR, vkCmdCopyMemoryToAccelerationStructureKHR, and vkCmdWriteAccelerationStructuresPropertiesKHR.
- VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR specifies the execution of the ray tracing shader stages, via vkCmdTraceRaysKHR, or vkCmdTraceRaysIndirectKHR
- VK_PIPELINE_STAGE_ALL_GRAPHICS_BIT specifies the execution of all graphics pipeline stages, and is equivalent to the logical OR of:
  - VK_PIPELINE_STAGE_DRAW_INDIRECT_BIT
  - VK_PIPELINE_STAGE_VERTEX_INPUT_BIT
  - VK_PIPELINE_STAGE_VERTEX_SHADER_BIT
  - VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT
  - VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT

    - VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT
    - VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT
    - VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT
    - VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT
    - VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT
    - VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
    - VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
    - VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VK_PIPELINE_STAGE_ALL_COMMANDS_BIT specifies all operations performed by all commands supported on the queue it is used with.
- VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT specifies the stage of the pipeline where vertex attribute output values are written to the transform feedback buffers.
- VK_PIPELINE_STAGE_COMMAND_PREPROCESS_BIT_EXT specifies the stage of the pipeline where device-side preprocessing for generated commands via vkCmdPreprocessGeneratedCommandsEXT is handled.
- VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR specifies the stage of the pipeline where the fragment shading rate attachment is read to determine the fragment shading rate for portions of a rasterized primitive.
- VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT specifies the stage of the pipeline where the fragment density map is read to generate the fragment areas.
- VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT is equivalent to VK_PIPELINE_STAGE_ALL_COMMANDS_BIT with VkAccessFlags set to 0 when specified in the second synchronization scope, but specifies no stage of execution when specified in the first scope.
- VK_PIPELINE_STAGE_BOTTOM_OF_PIPE_BIT is equivalent to VK_PIPELINE_STAGE_ALL_COMMANDS_BIT with VkAccessFlags set to 0 when specified in the first synchronization scope, but specifies no stage of execution when specified in the second scope.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineStageFlags;
```

VkPipelineStageFlags is a bitmask type for setting a mask of zero or more VkPipelineStageFlagBits.

If a synchronization command includes a source stage mask, its first synchronization scope only includes execution of the pipeline stages specified in that mask and any logically earlier stages. Its first access scope only includes memory accesses performed by pipeline stages explicitly specified in the source stage mask.

If a synchronization command includes a destination stage mask, its second synchronization scope only includes execution of the pipeline stages specified in that mask and any logically later stages. Its second access scope only includes memory accesses performed by pipeline stages explicitly specified in the destination stage mask.

NOTE Note that access scopes do not interact with the logically earlier or later stages for either scope - only the stages the application specifies are considered part of each access scope.

Certain pipeline stages are only available on queues that support a particular set of operations. The following table lists, for each pipeline stage flag, which queue capability flag **must** be supported by the queue. When multiple flags are enumerated in the second column of the table, it means that the pipeline stage is supported on the queue if it supports any of the listed capability flags. For further details on queue capabilities see Physical Device Enumeration and Queues.

Table 3. Supported Pipeline Stage Flags

Pipeline stage flag	Required queue capability flag
VK_PIPELINE_STAGE_2_NONE	None required
VK_PIPELINE_STAGE_2_TOP_OF_PIPE_BIT	None required
VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT
VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT	VK_QUEUE_COMPUTE_BIT
VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT or VK_QUEUE_TRANSFER_BIT
VK_PIPELINE_STAGE_2_BOTTOM_OF_PIPE_BIT	None required
VK_PIPELINE_STAGE_2_HOST_BIT	None required
VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT	None required
VK_PIPELINE_STAGE_2_COPY_BIT	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT or VK_QUEUE_TRANSFER_BIT
VK_PIPELINE_STAGE_2_RESOLVE_BIT	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT or VK_QUEUE_TRANSFER_BIT

Pipeline stage flag	Required queue capability flag
VK_PIPELINE_STAGE_2_BLIT_BIT	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT or VK_QUEUE_TRANSFER_BIT
VK_PIPELINE_STAGE_2_CLEAR_BIT	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT or VK_QUEUE_TRANSFER_BIT
VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_PRE_RASTERIZATION_SHADERS_BIT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR	VK_QUEUE_VIDEO_DECODE_BIT_KHR
VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR	VK_QUEUE_VIDEO_ENCODE_BIT_KHR
VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_NV	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT
VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT
VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR	VK_QUEUE_COMPUTE_BIT
VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR	VK_QUEUE_COMPUTE_BIT
VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT	VK_QUEUE_GRAPHICS_BIT
VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR	VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT or VK_QUEUE_TRANSFER_BIT
VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT	VK_QUEUE_COMPUTE_BIT

Pipeline stages that execute as a result of a command logically complete execution in a specific order, such that completion of a logically later pipeline stage **must** not happen-before completion of a logically earlier stage. This means that including any stage in the source stage mask for a particular synchronization command also implies that any logically earlier stages are included in $\textbf{Scope}_{\text{1st}}$ for that command.

Similarly, initiation of a logically earlier pipeline stage **must** not happen-after initiation of a logically later pipeline stage. Including any given stage in the destination stage mask for a particular synchronization command also implies that any logically later stages are included in $\textbf{Scope}_{\text{2nd}}$ for that command.

NOTE

Implementations **may** not support synchronization at every pipeline stage for every synchronization operation. If a pipeline stage that an implementation does not support synchronization for appears in a source stage mask, it **may** substitute any logically later stage in its place for the first synchronization scope. If a pipeline stage that an implementation does not support synchronization for appears in a

> destination stage mask, it **may** substitute any logically earlier stage in its place for the second synchronization scope.
>
> For example, if an implementation is unable to signal an event immediately after vertex shader execution is complete, it **may** instead signal the event after color attachment output has completed.
>
> If an implementation makes such a substitution, it **must** not affect the semantics of execution or memory dependencies or image and buffer memory barriers.

Graphics pipelines are executable on queues supporting `VK_QUEUE_GRAPHICS_BIT`. Stages executed by graphics pipelines **can** only be specified in commands recorded for queues supporting `VK_QUEUE_GRAPHICS_BIT`.

The graphics pipeline executes the following stages, with the logical ordering of the stages matching the order specified here:

- `VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT`
- `VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT`
- `VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT`
- `VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT`
- `VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT`
- `VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT`
- `VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT`
- `VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT`
- `VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- `VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT`
- `VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT`
- `VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT`
- `VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT`

For the compute pipeline, the following stages occur in this order:

- `VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT`
- `VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT`

For graphics pipeline commands executing in a render pass with a fragment density map attachment, the following pipeline stage where the fragment density map read happens has no particular order relative to the other stages, except that it is logically earlier than `VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT`:

- `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- `VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT`

For the transfer pipeline, the following stages occur in this order:

- VK_PIPELINE_STAGE_2_TRANSFER_BIT

For host operations, only one pipeline stage occurs, so no order is guaranteed:

- VK_PIPELINE_STAGE_2_HOST_BIT

For the command preprocessing pipeline, the following stages occur in this order:

- VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT

For acceleration structure build operations, only one pipeline stage occurs, so no order is guaranteed:

- VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR

For acceleration structure copy operations, only one pipeline stage occurs, so no order is guaranteed:

- VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR

For opacity micromap build operations, only one pipeline stage occurs, so no order is guaranteed:

- VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT

For the ray tracing pipeline, the following stages occur in this order:

- VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT
- VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR

For the video decode pipeline, the following stages occur in this order:

- VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR

For the video encode pipeline, the following stages occur in this order:

- VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR

7.1.3. Access Types

Memory in Vulkan **can** be accessed from within shader invocations and via some fixed-function stages of the pipeline. The *access type* is a function of the descriptor type used, or how a fixed-function stage accesses memory.

Some synchronization commands take sets of access types as parameters to define the access scopes of a memory dependency. If a synchronization command includes a *source access mask*, its first access scope only includes accesses via the access types specified in that mask. Similarly, if a synchronization command includes a *destination access mask*, its second access scope only includes accesses via the access types specified in that mask.

Bits which **can** be set in the srcAccessMask and dstAccessMask members of VkMemoryBarrier2KHR,

VkImageMemoryBarrier2KHR, and VkBufferMemoryBarrier2KHR, specifying access behavior, are:

```
// Flag bits for VkAccessFlagBits2
typedef VkFlags64 VkAccessFlagBits2;
static const VkAccessFlagBits2 VK_ACCESS_2_NONE = 0ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_NONE_KHR = 0ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT = 0x00000001ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT_KHR =
0x00000001ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_INDEX_READ_BIT = 0x00000002ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_INDEX_READ_BIT_KHR = 0x00000002ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT = 0x00000004ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT_KHR =
0x00000004ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_UNIFORM_READ_BIT = 0x00000008ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_UNIFORM_READ_BIT_KHR = 0x00000008ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT = 0x00000010ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT_KHR =
0x00000010ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_READ_BIT = 0x00000020ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_READ_BIT_KHR = 0x00000020ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_WRITE_BIT = 0x00000040ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_WRITE_BIT_KHR = 0x00000040ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT = 0x00000080ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT_KHR =
0x00000080ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT = 0x00000100ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT_KHR =
0x00000100ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT =
0x00000200ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT_KHR =
0x00000200ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT =
0x00000400ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT_KHR =
0x00000400ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_TRANSFER_READ_BIT = 0x00000800ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_TRANSFER_READ_BIT_KHR = 0x00000800ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_TRANSFER_WRITE_BIT = 0x00001000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_TRANSFER_WRITE_BIT_KHR = 0x00001000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_HOST_READ_BIT = 0x00002000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_HOST_READ_BIT_KHR = 0x00002000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_HOST_WRITE_BIT = 0x00004000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_HOST_WRITE_BIT_KHR = 0x00004000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_MEMORY_READ_BIT = 0x00008000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_MEMORY_READ_BIT_KHR = 0x00008000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_MEMORY_WRITE_BIT = 0x00010000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_MEMORY_WRITE_BIT_KHR = 0x00010000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_SAMPLED_READ_BIT = 0x100000000ULL;
```

```
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_SAMPLED_READ_BIT_KHR =
0x100000000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_STORAGE_READ_BIT = 0x200000000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_STORAGE_READ_BIT_KHR =
0x200000000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT = 0x400000000ULL;
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT_KHR =
0x400000000ULL;
// Provided by VK_KHR_video_decode_queue
static const VkAccessFlagBits2 VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR = 0x800000000ULL;
// Provided by VK_KHR_video_decode_queue
static const VkAccessFlagBits2 VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR =
0x1000000000ULL;
// Provided by VK_KHR_video_encode_queue
static const VkAccessFlagBits2 VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR =
0x2000000000ULL;
// Provided by VK_KHR_video_encode_queue
static const VkAccessFlagBits2 VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR =
0x4000000000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_transform_feedback
static const VkAccessFlagBits2 VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT =
0x02000000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_transform_feedback
static const VkAccessFlagBits2 VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT =
0x04000000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_transform_feedback
static const VkAccessFlagBits2 VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT =
0x08000000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_conditional_rendering
static const VkAccessFlagBits2 VK_ACCESS_2_CONDITIONAL_RENDERING_READ_BIT_EXT =
0x00100000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_device_generated_commands
static const VkAccessFlagBits2 VK_ACCESS_2_COMMAND_PREPROCESS_READ_BIT_NV =
0x00020000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_device_generated_commands
static const VkAccessFlagBits2 VK_ACCESS_2_COMMAND_PREPROCESS_WRITE_BIT_NV =
0x00040000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_device_generated_commands
static const VkAccessFlagBits2 VK_ACCESS_2_COMMAND_PREPROCESS_READ_BIT_EXT =
0x00020000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_device_generated_commands
static const VkAccessFlagBits2 VK_ACCESS_2_COMMAND_PREPROCESS_WRITE_BIT_EXT =
0x00040000ULL;
// Provided by VK_KHR_fragment_shading_rate with VK_KHR_synchronization2
static const VkAccessFlagBits2
VK_ACCESS_2_FRAGMENT_SHADING_RATE_ATTACHMENT_READ_BIT_KHR = 0x00800000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_shading_rate_image
static const VkAccessFlagBits2 VK_ACCESS_2_SHADING_RATE_IMAGE_READ_BIT_NV =
0x00800000ULL;
// Provided by VK_KHR_acceleration_structure with VK_KHR_synchronization2
static const VkAccessFlagBits2 VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR =
```

```
0x00200000ULL;
// Provided by VK_KHR_acceleration_structure with VK_KHR_synchronization2
static const VkAccessFlagBits2 VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR =
0x00400000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_ray_tracing
static const VkAccessFlagBits2 VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_NV =
0x00200000ULL;
// Provided by VK_KHR_synchronization2 with VK_NV_ray_tracing
static const VkAccessFlagBits2 VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_NV =
0x00400000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_fragment_density_map
static const VkAccessFlagBits2 VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT =
0x01000000ULL;
// Provided by VK_KHR_synchronization2 with VK_EXT_blend_operation_advanced
static const VkAccessFlagBits2 VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT =
0x00080000ULL;
// Provided by VK_KHR_ray_tracing_maintenance1 with (VK_KHR_synchronization2 or
VK_VERSION_1_3) and VK_KHR_ray_tracing_pipeline
static const VkAccessFlagBits2 VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR =
0x10000000000ULL;
// Provided by VK_EXT_opacity_micromap
static const VkAccessFlagBits2 VK_ACCESS_2_MICROMAP_READ_BIT_EXT = 0x100000000000ULL;
// Provided by VK_EXT_opacity_micromap
static const VkAccessFlagBits2 VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT = 0x200000000000ULL;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkAccessFlagBits2 VkAccessFlagBits2KHR;
```

- VK_ACCESS_2_NONE specifies no accesses.
- VK_ACCESS_2_MEMORY_READ_BIT specifies all read accesses. It is always valid in any access mask, and is treated as equivalent to setting all READ access flags that are valid where it is used.
- VK_ACCESS_2_MEMORY_WRITE_BIT specifies all write accesses. It is always valid in any access mask, and is treated as equivalent to setting all WRITE access flags that are valid where it is used.
- VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT specifies read access to command data read from indirect buffers as part of an indirect build, trace, drawing or dispatch command. Such access occurs in the VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT pipeline stage.
- VK_ACCESS_2_INDEX_READ_BIT specifies read access to an index buffer as part of an indexed drawing command, bound by vkCmdBindIndexBuffer2KHR and vkCmdBindIndexBuffer. Such access occurs in the VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT pipeline stage.
- VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT specifies read access to a vertex buffer as part of a drawing command, bound by vkCmdBindVertexBuffers. Such access occurs in the VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT pipeline stage.
- VK_ACCESS_2_UNIFORM_READ_BIT specifies read access to a uniform buffer in any shader pipeline stage.

- VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT specifies read access to an input attachment within a render pass during fragment shading. Such access occurs in the VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT pipeline stage.
- VK_ACCESS_2_SHADER_SAMPLED_READ_BIT specifies read access to a uniform texel buffer or sampled image in any shader pipeline stage.
- VK_ACCESS_2_SHADER_STORAGE_READ_BIT specifies read access to a storage buffer, storage texel buffer, or storage image in any shader pipeline stage.
- VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR specifies read access to a shader binding table in any shader pipeline stage.
- VK_ACCESS_2_SHADER_READ_BIT is equivalent to the logical OR of:
  - VK_ACCESS_2_SHADER_SAMPLED_READ_BIT
  - VK_ACCESS_2_SHADER_STORAGE_READ_BIT
- VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT specifies write access to a storage buffer, storage texel buffer, or storage image in any shader pipeline stage.
- VK_ACCESS_2_SHADER_WRITE_BIT is equivalent to VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT.
- VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT specifies read access to a color attachment, such as via blending (other than advanced blend operations), logic operations or certain render pass load operations in the VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage or via fragment shader tile image reads in the VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT pipeline stage.
- VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT specifies write access to a color attachment during a render pass or via certain render pass load, store, and multisample resolve operations. Such access occurs in the VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage.
- VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT specifies read access to a depth/stencil attachment, via depth or stencil operations or certain render pass load operations in the VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT or VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT pipeline stages or via fragment shader tile image reads in the VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT pipeline stage.
- VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT specifies write access to a depth/stencil attachment, via depth or stencil operations or certain render pass load and store operations. Such access occurs in the VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT or VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT pipeline stages.
- VK_ACCESS_2_TRANSFER_READ_BIT specifies read access to an image or buffer in a copy operation. Such access occurs in the VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, or VK_PIPELINE_STAGE_2_RESOLVE_BIT pipeline stages.
- VK_ACCESS_2_TRANSFER_WRITE_BIT specifies write access to an image or buffer in a clear or copy operation. Such access occurs in the VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_CLEAR_BIT, or VK_PIPELINE_STAGE_2_RESOLVE_BIT pipeline stages.
- VK_ACCESS_2_HOST_READ_BIT specifies read access by a host operation. Accesses of this type are not performed through a resource, but directly on memory. Such access occurs in the VK_PIPELINE_STAGE_2_HOST_BIT pipeline stage.

- VK_ACCESS_2_HOST_WRITE_BIT specifies write access by a host operation. Accesses of this type are not performed through a resource, but directly on memory. Such access occurs in the VK_PIPELINE_STAGE_2_HOST_BIT pipeline stage.
- VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT specifies write access to a transform feedback buffer made when transform feedback is active. Such access occurs in the VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT pipeline stage.
- VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT specifies read access to a transform feedback counter buffer which is read when vkCmdBeginTransformFeedbackEXT executes. Such access occurs in the VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT pipeline stage.
- VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT specifies write access to a transform feedback counter buffer which is written when vkCmdEndTransformFeedbackEXT executes. Such access occurs in the VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT pipeline stage.
- VK_ACCESS_2_COMMAND_PREPROCESS_READ_BIT_EXT specifies reads from buffer inputs to vkCmdPreprocessGeneratedCommandsEXT. Such access occurs in the VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT pipeline stage.
- VK_ACCESS_2_COMMAND_PREPROCESS_WRITE_BIT_EXT specifies writes to the target command buffer preprocess outputs. Such access occurs in the VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT pipeline stage.
- VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT specifies read access to color attachments, including advanced blend operations. Such access occurs in the VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage.
- VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR specifies read access to an acceleration structure as part of a trace, build, or copy command, or to an acceleration structure scratch buffer as part of a build command. Such access occurs in the VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR pipeline stage or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stage.
- VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR specifies write access to an acceleration structure or acceleration structure scratch buffer as part of a build or copy command. Such access occurs in the VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stage.
- VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT specifies read access to a fragment density map attachment during dynamic fragment density map operations. Such access occurs in the VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT pipeline stage.
- VK_ACCESS_2_FRAGMENT_SHADING_RATE_ATTACHMENT_READ_BIT_KHR specifies read access to a fragment shading rate attachment during rasterization. Such access occurs in the VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR pipeline stage.
- VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR specifies read access to an image or buffer resource in a video decode operation. Such access occurs in the VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR pipeline stage.
- VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR specifies write access to an image or buffer resource in a video decode operation. Such access occurs in the VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR pipeline stage.
- VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR specifies read access to an image or buffer resource in a video encode operation. Such access occurs in the VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR

pipeline stage.

- VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR specifies write access to an image or buffer resource in a video encode operation. Such access occurs in the VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR pipeline stage.
- VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT specifies write access to a micromap object. Such access occurs in the VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT pipeline stage.
- VK_ACCESS_2_MICROMAP_READ_BIT_EXT specifies read access to a micromap object. Such access occurs in the VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT and VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stages.

NOTE	In situations where an application wishes to select all access types for a given set of pipeline stages, VK_ACCESS_2_MEMORY_READ_BIT or VK_ACCESS_2_MEMORY_WRITE_BIT can be used. This is particularly useful when specifying stages that only have a single access type.

NOTE	The VkAccessFlags2 bitmask goes beyond the 31 individual bit flags allowable within a C99 enum, which is how VkAccessFlagBits is defined. The first 31 values are common to both, and are interchangeable.

VkAccessFlags2 is a bitmask type for setting a mask of zero or more VkAccessFlagBits2:

```
typedef VkFlags64 VkAccessFlags2;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkAccessFlags2 VkAccessFlags2KHR;
```

Bits which **can** be set in the srcAccessMask and dstAccessMask members of VkSubpassDependency, VkSubpassDependency2, VkMemoryBarrier, VkBufferMemoryBarrier, and VkImageMemoryBarrier, specifying access behavior, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkAccessFlagBits {
    VK_ACCESS_INDIRECT_COMMAND_READ_BIT = 0x00000001,
    VK_ACCESS_INDEX_READ_BIT = 0x00000002,
    VK_ACCESS_VERTEX_ATTRIBUTE_READ_BIT = 0x00000004,
    VK_ACCESS_UNIFORM_READ_BIT = 0x00000008,
    VK_ACCESS_INPUT_ATTACHMENT_READ_BIT = 0x00000010,
    VK_ACCESS_SHADER_READ_BIT = 0x00000020,
    VK_ACCESS_SHADER_WRITE_BIT = 0x00000040,
    VK_ACCESS_COLOR_ATTACHMENT_READ_BIT = 0x00000080,
    VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT = 0x00000100,
    VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_READ_BIT = 0x00000200,
    VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT = 0x00000400,
    VK_ACCESS_TRANSFER_READ_BIT = 0x00000800,
    VK_ACCESS_TRANSFER_WRITE_BIT = 0x00001000,
    VK_ACCESS_HOST_READ_BIT = 0x00002000,
    VK_ACCESS_HOST_WRITE_BIT = 0x00004000,
    VK_ACCESS_MEMORY_READ_BIT = 0x00008000,
    VK_ACCESS_MEMORY_WRITE_BIT = 0x00010000,
    VK_ACCESS_NONE = 0,
  // Provided by VK_EXT_transform_feedback
    VK_ACCESS_TRANSFORM_FEEDBACK_WRITE_BIT_EXT = 0x02000000,
  // Provided by VK_EXT_transform_feedback
    VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT = 0x04000000,
  // Provided by VK_EXT_transform_feedback
    VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT = 0x08000000,
  // Provided by VK_EXT_blend_operation_advanced
    VK_ACCESS_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT = 0x00080000,
  // Provided by VK_KHR_acceleration_structure
    VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR = 0x00200000,
  // Provided by VK_KHR_acceleration_structure
    VK_ACCESS_ACCELERATION_STRUCTURE_WRITE_BIT_KHR = 0x00400000,
  // Provided by VK_EXT_fragment_density_map
    VK_ACCESS_FRAGMENT_DENSITY_MAP_READ_BIT_EXT = 0x01000000,
  // Provided by VK_KHR_fragment_shading_rate
    VK_ACCESS_FRAGMENT_SHADING_RATE_ATTACHMENT_READ_BIT_KHR = 0x00800000,
    VK_ACCESS_COMMAND_PREPROCESS_READ_BIT_NV = 0x00020000,
    VK_ACCESS_COMMAND_PREPROCESS_WRITE_BIT_NV = 0x00040000,
  // Provided by VK_KHR_synchronization2
    VK_ACCESS_NONE_KHR = VK_ACCESS_NONE,
  // Provided by VK_EXT_device_generated_commands
    VK_ACCESS_COMMAND_PREPROCESS_READ_BIT_EXT =
VK_ACCESS_COMMAND_PREPROCESS_READ_BIT_NV,
  // Provided by VK_EXT_device_generated_commands
    VK_ACCESS_COMMAND_PREPROCESS_WRITE_BIT_EXT =
VK_ACCESS_COMMAND_PREPROCESS_WRITE_BIT_NV,
} VkAccessFlagBits;
```

These values all have the same meaning as the equivalently named values for VkAccessFlags2.

- VK_ACCESS_NONE specifies no accesses.
- VK_ACCESS_MEMORY_READ_BIT specifies all read accesses. It is always valid in any access mask, and is treated as equivalent to setting all READ access flags that are valid where it is used.
- VK_ACCESS_MEMORY_WRITE_BIT specifies all write accesses. It is always valid in any access mask, and is treated as equivalent to setting all WRITE access flags that are valid where it is used.
- VK_ACCESS_INDIRECT_COMMAND_READ_BIT specifies read access to indirect command data read as part of an indirect build, trace, drawing or dispatching command. Such access occurs in the VK_PIPELINE_STAGE_DRAW_INDIRECT_BIT pipeline stage.
- VK_ACCESS_INDEX_READ_BIT specifies read access to an index buffer as part of an indexed drawing command, bound by vkCmdBindIndexBuffer2KHR and vkCmdBindIndexBuffer. Such access occurs in the VK_PIPELINE_STAGE_VERTEX_INPUT_BIT pipeline stage.
- VK_ACCESS_VERTEX_ATTRIBUTE_READ_BIT specifies read access to a vertex buffer as part of a drawing command, bound by vkCmdBindVertexBuffers. Such access occurs in the VK_PIPELINE_STAGE_VERTEX_INPUT_BIT pipeline stage.
- VK_ACCESS_UNIFORM_READ_BIT specifies read access to a uniform buffer in any shader pipeline stage.
- VK_ACCESS_INPUT_ATTACHMENT_READ_BIT specifies read access to an input attachment within a render pass during fragment shading. Such access occurs in the VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT pipeline stage.
- VK_ACCESS_SHADER_READ_BIT specifies read access to a uniform texel buffer, sampled image, storage buffer, physical storage buffer, shader binding table, storage texel buffer, or storage image in any shader pipeline stage.
- VK_ACCESS_SHADER_WRITE_BIT specifies write access to a storage buffer, physical storage buffer, storage texel buffer, or storage image in any shader pipeline stage.
- VK_ACCESS_COLOR_ATTACHMENT_READ_BIT specifies read access to a color attachment, such as via blending (other than advanced blend operations), logic operations or certain render pass load operations in the VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage or via fragment shader tile image reads in the VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT pipeline stage.
- VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT specifies write access to a color, resolve, or depth/stencil resolve attachment during a render pass or via certain render pass load and store operations. Such access occurs in the VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage.
- VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_READ_BIT specifies read access to a depth/stencil attachment, via depth or stencil operations or certain render pass load operations in the VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT or VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT pipeline stages or via fragment shader tile image reads in the VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT pipeline stage.
- VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT specifies write access to a depth/stencil attachment, via depth or stencil operations or certain render pass load and store operations. Such access occurs in the VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT or VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT pipeline stages.
- VK_ACCESS_TRANSFER_READ_BIT specifies read access to an image or buffer in a copy operation. Such access occurs in the VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT pipeline stage.

- VK_ACCESS_TRANSFER_WRITE_BIT specifies write access to an image or buffer in a clear or copy operation. Such access occurs in the VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT pipeline stage.
- VK_ACCESS_HOST_READ_BIT specifies read access by a host operation. Accesses of this type are not performed through a resource, but directly on memory. Such access occurs in the VK_PIPELINE_STAGE_HOST_BIT pipeline stage.
- VK_ACCESS_HOST_WRITE_BIT specifies write access by a host operation. Accesses of this type are not performed through a resource, but directly on memory. Such access occurs in the VK_PIPELINE_STAGE_HOST_BIT pipeline stage.
- VK_ACCESS_TRANSFORM_FEEDBACK_WRITE_BIT_EXT specifies write access to a transform feedback buffer made when transform feedback is active. Such access occurs in the VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT pipeline stage.
- VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT specifies read access to a transform feedback counter buffer which is read when vkCmdBeginTransformFeedbackEXT executes. Such access occurs in the VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT pipeline stage.
- VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT specifies write access to a transform feedback counter buffer which is written when vkCmdEndTransformFeedbackEXT executes. Such access occurs in the VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT pipeline stage.
- VK_ACCESS_COMMAND_PREPROCESS_READ_BIT_EXT specifies reads from buffer inputs to vkCmdPreprocessGeneratedCommandsEXT. Such access occurs in the VK_PIPELINE_STAGE_COMMAND_PREPROCESS_BIT_EXT pipeline stage.
- VK_ACCESS_COMMAND_PREPROCESS_WRITE_BIT_EXT specifies writes to the target command buffer preprocess outputs in vkCmdPreprocessGeneratedCommandsEXT. Such access occurs in the VK_PIPELINE_STAGE_COMMAND_PREPROCESS_BIT_EXT pipeline stage.
- VK_ACCESS_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT specifies read access to color attachments, including advanced blend operations. Such access occurs in the VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage.
- VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR specifies read access to an acceleration structure as part of a trace, build, or copy command, or to an acceleration structure scratch buffer as part of a build command. Such access occurs in the VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR pipeline stage or VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stage.
- VK_ACCESS_ACCELERATION_STRUCTURE_WRITE_BIT_KHR specifies write access to an acceleration structure or acceleration structure scratch buffer as part of a build or copy command. Such access occurs in the VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stage.
- VK_ACCESS_FRAGMENT_DENSITY_MAP_READ_BIT_EXT specifies read access to a fragment density map attachment during dynamic fragment density map operations Such access occurs in the VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT pipeline stage.
- VK_ACCESS_FRAGMENT_SHADING_RATE_ATTACHMENT_READ_BIT_KHR specifies read access to a fragment shading rate attachment during rasterization. Such access occurs in the VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR pipeline stage.

Certain access types are only performed by a subset of pipeline stages. Any synchronization command that takes both stage masks and access masks uses both to define the access scopes - only

the specified access types performed by the specified stages are included in the access scope. An application **must** not specify an access flag in a synchronization command if it does not include a pipeline stage in the corresponding stage mask that is able to perform accesses of that type. The following table lists, for each access flag, which pipeline stages **can** perform that type of access.

Table 4. Supported Access Types

Access flag	Supported pipeline stages
VK_ACCESS_2_NONE	Any
VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT	VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR
VK_ACCESS_2_INDEX_READ_BIT	VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT
VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT	VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT
VK_ACCESS_2_UNIFORM_READ_BIT	VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT, VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT, VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR,
VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT	VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT,
VK_ACCESS_2_SHADER_READ_BIT	VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT, VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT, VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT, VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR,
VK_ACCESS_2_SHADER_WRITE_BIT	VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT, VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT, VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR,

Access flag	Supported pipeline stages
VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT	VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT
VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT	VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT
VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT	VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT
VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT	VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT
VK_ACCESS_2_TRANSFER_READ_BIT	VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT
VK_ACCESS_2_TRANSFER_WRITE_BIT	VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT
VK_ACCESS_2_HOST_READ_BIT	VK_PIPELINE_STAGE_2_HOST_BIT
VK_ACCESS_2_HOST_WRITE_BIT	VK_PIPELINE_STAGE_2_HOST_BIT
VK_ACCESS_2_MEMORY_READ_BIT	Any
VK_ACCESS_2_MEMORY_WRITE_BIT	Any
VK_ACCESS_2_SHADER_SAMPLED_READ_BIT	VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT, VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT, VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR,

Access flag	Supported pipeline stages
VK_ACCESS_2_SHADER_STORAGE_READ_BIT	VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT, VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT, VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR,
VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT	VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT, VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT, VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR,
VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR	VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR	VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR	VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR	VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT	VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT
VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT	VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT
VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT	VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT
VK_ACCESS_2_COMMAND_PREPROCESS_READ_BIT_NV	VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_NV
VK_ACCESS_2_COMMAND_PREPROCESS_WRITE_BIT_NV	VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_NV
VK_ACCESS_2_COMMAND_PREPROCESS_READ_BIT_EXT	VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT
VK_ACCESS_2_COMMAND_PREPROCESS_WRITE_BIT_EXT	VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT
VK_ACCESS_2_FRAGMENT_SHADING_RATE_ATTACHMENT_READ_BIT_KHR	VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR

Access flag	Supported pipeline stages
VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR	VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT, VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT, VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT, VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR,
VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR	VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR
VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT	VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT
VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT	VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT
VK_ACCESS_2_MICROMAP_READ_BIT_EXT	VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR
VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT	VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkAccessFlags;
```

VkAccessFlags is a bitmask type for setting a mask of zero or more VkAccessFlagBits.

If a memory object does not have the VK_MEMORY_PROPERTY_HOST_COHERENT_BIT property, then vkFlushMappedMemoryRanges **must** be called in order to guarantee that writes to the memory object from the host are made available to the host domain, where they **can** be further made available to the device domain via a domain operation. Similarly, vkInvalidateMappedMemoryRanges **must** be called to guarantee that writes which are available to the host domain are made visible to host operations.

If the memory object does have the VK_MEMORY_PROPERTY_HOST_COHERENT_BIT property flag, writes to the memory object from the host are automatically made available to the host domain. Similarly, writes made available to the host domain are automatically made visible to the host.

NOTE

Queue submission commands automatically perform a domain operation from host to device for all writes performed before the command executes, so in most cases an explicit memory barrier is not needed for this case. In the few circumstances where a submit does not occur between the host write and the device read access,

writes **can** be made available by using an explicit memory barrier.

7.1.4. Framebuffer Region Dependencies

Pipeline stages that operate on, or with respect to, the framebuffer are collectively the *framebuffer-space* pipeline stages. These stages are:

- `VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT`
- `VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT`
- `VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT`
- `VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT`

For these pipeline stages, an execution or memory dependency from the first set of operations to the second set **can** either be a single *framebuffer-global* dependency, or split into multiple *framebuffer-local* dependencies. A dependency with non-framebuffer-space pipeline stages is neither framebuffer-global nor framebuffer-local.

A *framebuffer region* is a set of sample (x, y, layer, sample) coordinates that is a subset of the entire framebuffer.

Both synchronization scopes of a framebuffer-local dependency include only the operations performed within corresponding framebuffer regions (as defined below). No ordering guarantees are made between different framebuffer regions for a framebuffer-local dependency.

Both synchronization scopes of a framebuffer-global dependency include operations on all framebuffer-regions.

If the first synchronization scope includes operations on pixels/fragments with N samples and the second synchronization scope includes operations on pixels/fragments with M samples, where N does not equal M, then a framebuffer region containing all samples at a given (x, y, layer) coordinate in the first synchronization scope corresponds to a region containing all samples at the same coordinate in the second synchronization scope. In other words, it is a pixel granularity dependency. If N equals M, then a framebuffer region containing a single (x, y, layer, sample) coordinate in the first synchronization scope corresponds to a region containing the same sample at the same coordinate in the second synchronization scope. In other words, it is a sample granularity dependency.

NOTE Since fragment shader invocations are not specified to run in any particular groupings, the size of a framebuffer region is implementation-dependent, not known to the application, and **must** be assumed to be no larger than specified above.

NOTE Practically, the pixel vs. sample granularity dependency means that if an input attachment has a different number of samples than the pipeline's `rasterizationSamples`, then a fragment **can** access any sample in the input attachment's pixel even if it only uses framebuffer-local dependencies. If the input attachment has the same number of samples, then the fragment **can** only access the

> covered samples in its input `SampleMask` (i.e. the fragment operations happen-after a framebuffer-local dependency for each sample the fragment covers). To access samples that are not covered, a framebuffer-global dependency is required.

If a synchronization command includes a `dependencyFlags` parameter, and specifies the `VK_DEPENDENCY_BY_REGION_BIT` flag, then it defines framebuffer-local dependencies for the framebuffer-space pipeline stages in that synchronization command, for all framebuffer regions. If no `dependencyFlags` parameter is included, or the `VK_DEPENDENCY_BY_REGION_BIT` flag is not specified, then a framebuffer-global dependency is specified for those stages. The `VK_DEPENDENCY_BY_REGION_BIT` flag does not affect the dependencies between non-framebuffer-space pipeline stages, nor does it affect the dependencies between framebuffer-space and non-framebuffer-space pipeline stages.

NOTE

> Framebuffer-local dependencies are more efficient for most architectures; particularly tile-based architectures - which can keep framebuffer-regions entirely in on-chip registers and thus avoid external bandwidth across such a dependency. Including a framebuffer-global dependency in your rendering will usually force all implementations to flush data to memory, or to a higher level cache, breaking any potential locality optimizations.

7.1.5. View-Local Dependencies

In a render pass instance that has multiview enabled, dependencies **can** be either view-local or view-global.

A view-local dependency only includes operations from a single source view from the source subpass in the first synchronization scope, and only includes operations from a single destination view from the destination subpass in the second synchronization scope. A view-global dependency includes all views in the view mask of the source and destination subpasses in the corresponding synchronization scopes.

If a synchronization command includes a `dependencyFlags` parameter and specifies the `VK_DEPENDENCY_VIEW_LOCAL_BIT` flag, then it defines view-local dependencies for that synchronization command, for all views. If no `dependencyFlags` parameter is included or the `VK_DEPENDENCY_VIEW_LOCAL_BIT` flag is not specified, then a view-global dependency is specified.

7.1.6. Device-Local Dependencies

Dependencies **can** be either device-local or non-device-local. A device-local dependency acts as multiple separate dependencies, one for each physical device that executes the synchronization command, where each dependency only includes operations from that physical device in both synchronization scopes. A non-device-local dependency is a single dependency where both synchronization scopes include operations from all physical devices that participate in the synchronization command. For subpass dependencies, all physical devices in the VkDeviceGroupRenderPassBeginInfo::`deviceMask` participate in the dependency, and for pipeline barriers all physical devices that are set in the command buffer's current device mask participate in the dependency.

If a synchronization command includes a `dependencyFlags` parameter and specifies the `VK_DEPENDENCY_DEVICE_GROUP_BIT` flag, then it defines a non-device-local dependency for that synchronization command. If no `dependencyFlags` parameter is included or the `VK_DEPENDENCY_DEVICE_GROUP_BIT` flag is not specified, then it defines device-local dependencies for that synchronization command, for all participating physical devices.

Semaphore and event dependencies are device-local and only execute on the one physical device that performs the dependency.

7.2. Implicit Synchronization Guarantees

A small number of implicit ordering guarantees are provided by Vulkan, ensuring that the order in which commands are submitted is meaningful, and avoiding unnecessary complexity in common operations.

Submission order is a fundamental ordering in Vulkan, giving meaning to the order in which action and synchronization commands are recorded and submitted to a single queue. Explicit and implicit ordering guarantees between commands in Vulkan all work on the premise that this ordering is meaningful. This order does not itself define any execution or memory dependencies; synchronization commands and other orderings within the API use this ordering to define their scopes.

Submission order for any given set of commands is based on the order in which they were recorded to command buffers and then submitted. This order is determined as follows:

1. The initial order is determined by the order in which vkQueueSubmit and vkQueueSubmit2KHR commands are executed on the host, for a single queue, from first to last.
2. The order in which VkSubmitInfo structures are specified in the `pSubmits` parameter of vkQueueSubmit, or in which VkSubmitInfo2 structures are specified in the `pSubmits` parameter of vkQueueSubmit2KHR, from lowest index to highest.
3. The order in which command buffers are specified in the `pCommandBuffers` member of VkSubmitInfo or VkSubmitInfo2 from lowest index to highest.
4. The order in which commands outside of a render pass were recorded to a command buffer on the host, from first to last.
5. The order in which commands inside a single subpass were recorded to a command buffer on the host, from first to last.

NOTE

When using a render pass object with multiple subpasses, commands in different subpasses have no defined submission order relative to each other, regardless of the order in which the subpasses were recorded. Commands within a subpass are still ordered relative to other commands in the same subpass, and those outside of the render pass.

State commands do not execute any operations on the device, instead they set the state of the command buffer when they execute on the host, in the order that they are recorded. Action commands consume the current state of the command buffer when they are recorded, and will execute state changes on the device as required to match the recorded state.

The order of primitives passing through the graphics pipeline and image layout transitions as part of an image memory barrier provide additional guarantees based on submission order.

Execution of pipeline stages within a given command also has a loose ordering, dependent only on a single command.

Signal operation order is a fundamental ordering in Vulkan, giving meaning to the order in which semaphore and fence signal operations occur when submitted to a single queue. The signal operation order for queue operations is determined as follows:

1. The initial order is determined by the order in which vkQueueSubmit and vkQueueSubmit2KHR commands are executed on the host, for a single queue, from first to last.
2. The order in which VkSubmitInfo structures are specified in the `pSubmits` parameter of vkQueueSubmit, or in which VkSubmitInfo2 structures are specified in the `pSubmits` parameter of vkQueueSubmit2KHR, from lowest index to highest.
3. The fence signal operation defined by the `fence` parameter of a vkQueueSubmit or vkQueueSubmit2KHR or vkQueueBindSparse command is ordered after all semaphore signal operations defined by that command.

Semaphore signal operations defined by a single VkSubmitInfo or VkSubmitInfo2 or VkBindSparseInfo structure are unordered with respect to other semaphore signal operations defined within the same structure.

The vkSignalSemaphoreKHR command does not execute on a queue but instead performs the signal operation from the host. The semaphore signal operation defined by executing a vkSignalSemaphoreKHR command happens-after the vkSignalSemaphoreKHR command is invoked and happens-before the command returns.

NOTE

When signaling timeline semaphores, it is the responsibility of the application to ensure that they are ordered such that the semaphore value is strictly increasing. Because the first synchronization scope for a semaphore signal operation contains all semaphore signal operations which occur earlier in submission order, all semaphore signal operations contained in any given batch are guaranteed to happen-after all semaphore signal operations contained in any previous batches. However, no ordering guarantee is provided between the semaphore signal operations defined within a single batch. This, combined with the requirement that timeline semaphore values strictly increase, means that it is invalid to signal the same timeline semaphore twice within a single batch.

If an application wishes to ensure that some semaphore signal operation happens-after some other semaphore signal operation, it can submit a separate batch containing only semaphore signal operations, which will happen-after the semaphore signal operations in any earlier batches.

When signaling a semaphore from the host, the only ordering guarantee is that the signal operation happens-after when vkSignalSemaphoreKHR is called and happens-before it returns. Therefore, it is invalid to call `vkSignalSemaphoreKHR` while there are any outstanding signal operations on that semaphore from any queue

> submissions unless those queue submissions have some dependency which ensures that they happen-after the host signal operation. One example of this would be if the pending signal operation is, itself, waiting on the same semaphore at a lower value and the call to vkSignalSemaphoreKHR signals that lower value. Furthermore, if there are two or more processes or threads signaling the same timeline semaphore from the host, the application must ensure that the vkSignalSemaphoreKHR with the lower semaphore value returns before vkSignalSemaphoreKHR is called with the higher value.

7.3. Fences

Fences are a synchronization primitive that **can** be used to insert a dependency from a queue to the host. Fences have two states - signaled and unsignaled. A fence **can** be signaled as part of the execution of a queue submission command. Fences **can** be unsignaled on the host with vkResetFences. Fences **can** be waited on by the host with the vkWaitForFences command, and the current state **can** be queried with vkGetFenceStatus.

The internal data of a fence **may** include a reference to any resources and pending work associated with signal or unsignal operations performed on that fence object, collectively referred to as the fence's *payload*. Mechanisms to import and export that internal data to and from fences are provided below. These mechanisms indirectly enable applications to share fence state between two or more fences and other synchronization primitives across process and API boundaries.

Fences are represented by VkFence handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkFence)
```

To create a fence, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateFence(
    VkDevice                                    device,
    const VkFenceCreateInfo*                    pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkFence*                                    pFence);
```

- device is the logical device that creates the fence.
- pCreateInfo is a pointer to a VkFenceCreateInfo structure containing information about how the fence is to be created.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.
- pFence is a pointer to a handle in which the resulting fence object is returned.

Valid Usage (Implicit)

- VUID-vkCreateFence-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateFence-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkFenceCreateInfo structure
- VUID-vkCreateFence-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateFence-pFence-parameter
  `pFence` **must** be a valid pointer to a VkFence handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkFenceCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkFenceCreateInfo {
    VkStructureType       sType;
    const void*           pNext;
    VkFenceCreateFlags    flags;
} VkFenceCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkFenceCreateFlagBits specifying the initial state and behavior of the fence.

Valid Usage (Implicit)

- VUID-VkFenceCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_FENCE_CREATE_INFO`
- VUID-VkFenceCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkExportFenceCreateInfo or

VkExportFenceWin32HandleInfoKHR

- VUID-VkFenceCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkFenceCreateInfo-flags-parameter
  `flags` **must** be a valid combination of VkFenceCreateFlagBits values

```
// Provided by VK_VERSION_1_0
typedef enum VkFenceCreateFlagBits {
    VK_FENCE_CREATE_SIGNALED_BIT = 0x00000001,
} VkFenceCreateFlagBits;
```

- `VK_FENCE_CREATE_SIGNALED_BIT` specifies that the fence object is created in the signaled state. Otherwise, it is created in the unsignaled state.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkFenceCreateFlags;
```

`VkFenceCreateFlags` is a bitmask type for setting a mask of zero or more VkFenceCreateFlagBits.

To create a fence whose payload **can** be exported to external handles, add a VkExportFenceCreateInfo structure to the `pNext` chain of the VkFenceCreateInfo structure. The `VkExportFenceCreateInfo` structure is defined as:

```
typedef struct VkExportFenceCreateInfo {
    VkStructureType                   sType;
    const void*                       pNext;
    VkExternalFenceHandleTypeFlags    handleTypes;
} VkExportFenceCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_fence
typedef VkExportFenceCreateInfo VkExportFenceCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleTypes` is a bitmask of VkExternalFenceHandleTypeFlagBits specifying one or more fence handle types the application **can** export from the resulting fence. The application **can** request multiple handle types for the same fence.

Valid Usage

- VUID-VkExportFenceCreateInfo-handleTypes-01446
  The bits in `handleTypes` **must** be supported and compatible, as reported by VkExternalFenceProperties

Valid Usage (Implicit)

- VUID-VkExportFenceCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_FENCE_CREATE_INFO`
- VUID-VkExportFenceCreateInfo-handleTypes-parameter
  `handleTypes` **must** be a valid combination of VkExternalFenceHandleTypeFlagBits values

To specify additional attributes of NT handles exported from a fence, add a VkExportFenceWin32HandleInfoKHR structure to the `pNext` chain of the VkFenceCreateInfo structure. The `VkExportFenceWin32HandleInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_fence_win32
typedef struct VkExportFenceWin32HandleInfoKHR {
    VkStructureType               sType;
    const void*                   pNext;
    const SECURITY_ATTRIBUTES*    pAttributes;
    DWORD                         dwAccess;
    LPCWSTR                       name;
} VkExportFenceWin32HandleInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pAttributes` is a pointer to a Windows `SECURITY_ATTRIBUTES` structure specifying security attributes of the handle.
- `dwAccess` is a `DWORD` specifying access rights of the handle.
- `name` is a null-terminated UTF-16 string to associate with the underlying synchronization primitive referenced by NT handles exported from the created fence.

If VkExportFenceCreateInfo is not included in the same `pNext` chain, this structure is ignored.

If VkExportFenceCreateInfo is included in the `pNext` chain of VkFenceCreateInfo with a Windows `handleType`, but either `VkExportFenceWin32HandleInfoKHR` is not included in the `pNext` chain, or it is included but `pAttributes` is `NULL`, default security descriptor values will be used, and child processes created by the application will not inherit the handle, as described in the MSDN documentation for "Synchronization Object Security and Access Rights"[1]. Further, if the structure is not present, the access rights will be

`DXGI_SHARED_RESOURCE_READ` | `DXGI_SHARED_RESOURCE_WRITE`

for handles of the following types:

`VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT`

1

https://docs.microsoft.com/en-us/windows/win32/sync/synchronization-object-security-and-access-rights

Valid Usage

- VUID-VkExportFenceWin32HandleInfoKHR-handleTypes-01447
  If VkExportFenceCreateInfo::`handleTypes` does not include `VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT`, a `VkExportFenceWin32HandleInfoKHR` structure **must** not be included in the `pNext` chain of VkFenceCreateInfo

Valid Usage (Implicit)

- VUID-VkExportFenceWin32HandleInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_FENCE_WIN32_HANDLE_INFO_KHR`
- VUID-VkExportFenceWin32HandleInfoKHR-pAttributes-parameter
  If `pAttributes` is not `NULL`, `pAttributes` **must** be a valid pointer to a valid `SECURITY_ATTRIBUTES` value

To export a Windows handle representing the state of a fence, call:

```
// Provided by VK_KHR_external_fence_win32
VkResult vkGetFenceWin32HandleKHR(
    VkDevice                                    device,
    const VkFenceGetWin32HandleInfoKHR*         pGetWin32HandleInfo,
    HANDLE*                                     pHandle);
```

- `device` is the logical device that created the fence being exported.
- `pGetWin32HandleInfo` is a pointer to a VkFenceGetWin32HandleInfoKHR structure containing parameters of the export operation.
- `pHandle` will return the Windows handle representing the fence state.

For handle types defined as NT handles, the handles returned by `vkGetFenceWin32HandleKHR` are owned by the application. To avoid leaking resources, the application **must** release ownership of them using the `CloseHandle` system call when they are no longer needed.

Exporting a Windows handle from a fence **may** have side effects depending on the transference of the specified handle type, as described in Importing Fence Payloads.

Valid Usage (Implicit)

- VUID-vkGetFenceWin32HandleKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetFenceWin32HandleKHR-pGetWin32HandleInfo-parameter
  `pGetWin32HandleInfo` **must** be a valid pointer to a valid VkFenceGetWin32HandleInfoKHR structure
- VUID-vkGetFenceWin32HandleKHR-pHandle-parameter
  `pHandle` **must** be a valid pointer to a `HANDLE` value

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_TOO_MANY_OBJECTS`
- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkFenceGetWin32HandleInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_fence_win32
typedef struct VkFenceGetWin32HandleInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkFence                               fence;
    VkExternalFenceHandleTypeFlagBits     handleType;
} VkFenceGetWin32HandleInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fence` is the fence from which state will be exported.
- `handleType` is a VkExternalFenceHandleTypeFlagBits value specifying the type of handle requested.

The properties of the handle returned depend on the value of `handleType`. See VkExternalFenceHandleTypeFlagBits for a description of the properties of the defined external fence handle types.

Valid Usage

- VUID-VkFenceGetWin32HandleInfoKHR-handleType-01448
  `handleType` **must** have been included in VkExportFenceCreateInfo::`handleTypes` when the

fence's current payload was created

- VUID-VkFenceGetWin32HandleInfoKHR-handleType-01449
  If handleType is defined as an NT handle, vkGetFenceWin32HandleKHR **must** be called no more than once for each valid unique combination of fence and handleType
- VUID-VkFenceGetWin32HandleInfoKHR-fence-01450
  fence **must** not currently have its payload replaced by an imported payload as described below in Importing Fence Payloads unless that imported payload's handle type was included in VkExternalFenceProperties::exportFromImportedHandleTypes for handleType
- VUID-VkFenceGetWin32HandleInfoKHR-handleType-01451
  If handleType refers to a handle type with copy payload transference semantics, fence **must** be signaled, or have an associated fence signal operation pending execution
- VUID-VkFenceGetWin32HandleInfoKHR-handleType-01452
  handleType **must** be defined as an NT handle or a global share handle

Valid Usage (Implicit)

- VUID-VkFenceGetWin32HandleInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_FENCE_GET_WIN32_HANDLE_INFO_KHR
- VUID-VkFenceGetWin32HandleInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkFenceGetWin32HandleInfoKHR-fence-parameter
  fence **must** be a valid VkFence handle
- VUID-VkFenceGetWin32HandleInfoKHR-handleType-parameter
  handleType **must** be a valid VkExternalFenceHandleTypeFlagBits value

To export a POSIX file descriptor representing the payload of a fence, call:

```
// Provided by VK_KHR_external_fence_fd
VkResult vkGetFenceFdKHR(
    VkDevice                                    device,
    const VkFenceGetFdInfoKHR*                  pGetFdInfo,
    int*                                        pFd);
```

- device is the logical device that created the fence being exported.
- pGetFdInfo is a pointer to a VkFenceGetFdInfoKHR structure containing parameters of the export operation.
- pFd will return the file descriptor representing the fence payload.

Each call to vkGetFenceFdKHR **must** create a new file descriptor and transfer ownership of it to the application. To avoid leaking resources, the application **must** release ownership of the file descriptor when it is no longer needed.

NOTE	Ownership can be released in many ways. For example, the application can call `close`() on the file descriptor, or transfer ownership back to Vulkan by using the file descriptor to import a fence payload.

If `pGetFdInfo->handleType` is `VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT` and the fence is signaled at the time `vkGetFenceFdKHR` is called, `pFd` **may** return the value `-1` instead of a valid file descriptor.

Where supported by the operating system, the implementation **must** set the file descriptor to be closed automatically when an `execve` system call is made.

Exporting a file descriptor from a fence **may** have side effects depending on the transference of the specified handle type, as described in Importing Fence State.

Valid Usage (Implicit)

- VUID-vkGetFenceFdKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetFenceFdKHR-pGetFdInfo-parameter
  `pGetFdInfo` **must** be a valid pointer to a valid VkFenceGetFdInfoKHR structure
- VUID-vkGetFenceFdKHR-pFd-parameter
  `pFd` **must** be a valid pointer to an `int` value

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_TOO_MANY_OBJECTS`
- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkFenceGetFdInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_fence_fd
typedef struct VkFenceGetFdInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkFence                               fence;
    VkExternalFenceHandleTypeFlagBits     handleType;
} VkFenceGetFdInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fence` is the fence from which state will be exported.

- `handleType` is a VkExternalFenceHandleTypeFlagBits value specifying the type of handle requested.

The properties of the file descriptor returned depend on the value of `handleType`. See VkExternalFenceHandleTypeFlagBits for a description of the properties of the defined external fence handle types.

Valid Usage

- VUID-VkFenceGetFdInfoKHR-handleType-01453
  `handleType` **must** have been included in VkExportFenceCreateInfo::`handleTypes` when `fence`'s current payload was created
- VUID-VkFenceGetFdInfoKHR-handleType-01454
  If `handleType` refers to a handle type with copy payload transference semantics, `fence` **must** be signaled, or have an associated fence signal operation pending execution
- VUID-VkFenceGetFdInfoKHR-fence-01455
  `fence` **must** not currently have its payload replaced by an imported payload as described below in Importing Fence Payloads unless that imported payload's handle type was included in VkExternalFenceProperties::`exportFromImportedHandleTypes` for `handleType`
- VUID-VkFenceGetFdInfoKHR-handleType-01456
  `handleType` **must** be defined as a POSIX file descriptor handle

Valid Usage (Implicit)

- VUID-VkFenceGetFdInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_FENCE_GET_FD_INFO_KHR`
- VUID-VkFenceGetFdInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkFenceGetFdInfoKHR-fence-parameter
  `fence` **must** be a valid VkFence handle
- VUID-VkFenceGetFdInfoKHR-handleType-parameter
  `handleType` **must** be a valid VkExternalFenceHandleTypeFlagBits value

To destroy a fence, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyFence(
    VkDevice                                    device,
    VkFence                                     fence,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the fence.
- `fence` is the handle of the fence to destroy.

- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyFence-fence-01120
  All queue submission commands that refer to `fence` **must** have completed execution
- VUID-vkDestroyFence-fence-01121
  If `VkAllocationCallbacks` were provided when `fence` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyFence-fence-01122
  If no `VkAllocationCallbacks` were provided when `fence` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyFence-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyFence-fence-parameter
  If `fence` is not VK_NULL_HANDLE, `fence` **must** be a valid VkFence handle
- VUID-vkDestroyFence-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyFence-fence-parent
  If `fence` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `fence` **must** be externally synchronized

To query the status of a fence from the host, call:

```
// Provided by VK_VERSION_1_0
VkResult vkGetFenceStatus(
    VkDevice                                    device,
    VkFence                                     fence);
```

- `device` is the logical device that owns the fence.
- `fence` is the handle of the fence to query.

Upon success, `vkGetFenceStatus` returns the status of the fence object, with the following return codes:

Table 5. Fence Object Status Codes

Status	Meaning
VK_SUCCESS	The fence specified by fence is signaled.
VK_NOT_READY	The fence specified by fence is unsignaled.
VK_ERROR_DEVICE_LOST	The device has been lost. See Lost Device.

If a queue submission command is pending execution, then the value returned by this command **may** immediately be out of date.

If the device has been lost (see Lost Device), vkGetFenceStatus **may** return any of the above status codes. If the device has been lost and vkGetFenceStatus is called repeatedly, it will eventually return either VK_SUCCESS or VK_ERROR_DEVICE_LOST.

Valid Usage (Implicit)

- VUID-vkGetFenceStatus-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetFenceStatus-fence-parameter
  fence **must** be a valid VkFence handle
- VUID-vkGetFenceStatus-fence-parent
  fence **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS
- VK_NOT_READY

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

To set the state of fences to unsignaled from the host, call:

```
// Provided by VK_VERSION_1_0
VkResult vkResetFences(
    VkDevice                                    device,
    uint32_t                                    fenceCount,
    const VkFence*                              pFences);
```

- `device` is the logical device that owns the fences.
- `fenceCount` is the number of fences to reset.
- `pFences` is a pointer to an array of fence handles to reset.

If any member of `pFences` currently has its payload imported with temporary permanence, that fence's prior permanent payload is first restored. The remaining operations described therefore operate on the restored payload.

When vkResetFences is executed on the host, it defines a *fence unsignal operation* for each fence, which resets the fence to the unsignaled state.

If any member of `pFences` is already in the unsignaled state when vkResetFences is executed, then vkResetFences has no effect on that fence.

Valid Usage

- VUID-vkResetFences-pFences-01123
  Each element of `pFences` **must** not be currently associated with any queue command that has not yet completed execution on that queue

Valid Usage (Implicit)

- VUID-vkResetFences-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkResetFences-pFences-parameter
  `pFences` **must** be a valid pointer to an array of `fenceCount` valid VkFence handles
- VUID-vkResetFences-fenceCount-arraylength
  `fenceCount` **must** be greater than `0`
- VUID-vkResetFences-pFences-parent
  Each element of `pFences` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to each member of `pFences` **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_DEVICE_MEMORY

When a fence is submitted to a queue as part of a queue submission command, it defines a memory dependency on the batches that were submitted as part of that command, and defines a *fence signal operation* which sets the fence to the signaled state.

The first synchronization scope includes every batch submitted in the same queue submission command. Fence signal operations that are defined by vkQueueSubmit or vkQueueSubmit2KHR additionally include in the first synchronization scope all commands that occur earlier in submission order. Fence signal operations that are defined by vkQueueSubmit or vkQueueSubmit2KHR or vkQueueBindSparse additionally include in the first synchronization scope any semaphore and fence signal operations that occur earlier in signal operation order.

The second synchronization scope only includes the fence signal operation.

The first access scope includes all memory access performed by the device.

The second access scope is empty.

To wait for one or more fences to enter the signaled state on the host, call:

```
// Provided by VK_VERSION_1_0
VkResult vkWaitForFences(
    VkDevice                                    device,
    uint32_t                                    fenceCount,
    const VkFence*                              pFences,
    VkBool32                                    waitAll,
    uint64_t                                    timeout);
```

- device is the logical device that owns the fences.
- fenceCount is the number of fences to wait on.
- pFences is a pointer to an array of fenceCount fence handles.
- waitAll is the condition that **must** be satisfied to successfully unblock the wait. If waitAll is VK_TRUE, then the condition is that all fences in pFences are signaled. Otherwise, the condition is that at least one fence in pFences is signaled.
- timeout is the timeout period in units of nanoseconds. timeout is adjusted to the closest value allowed by the implementation-dependent timeout accuracy, which **may** be substantially longer than one nanosecond, and **may** be longer than the requested period.

If the condition is satisfied when vkWaitForFences is called, then vkWaitForFences returns

immediately. If the condition is not satisfied at the time vkWaitForFences is called, then vkWaitForFences will block and wait until the condition is satisfied or the timeout has expired, whichever is sooner.

If timeout is zero, then vkWaitForFences does not wait, but simply returns the current state of the fences. VK_TIMEOUT will be returned in this case if the condition is not satisfied, even though no actual wait was performed.

If the condition is satisfied before the timeout has expired, vkWaitForFences returns VK_SUCCESS. Otherwise, vkWaitForFences returns VK_TIMEOUT after the timeout has expired.

If device loss occurs (see Lost Device) before the timeout has expired, vkWaitForFences **must** return in finite time with either VK_SUCCESS or VK_ERROR_DEVICE_LOST.

NOTE	While we guarantee that vkWaitForFences **must** return in finite time, no guarantees are made that it returns immediately upon device loss. However, the application can reasonably expect that the delay will be on the order of seconds and that calling vkWaitForFences will not result in a permanently (or seemingly permanently) dead process.

Valid Usage (Implicit)

- VUID-vkWaitForFences-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkWaitForFences-pFences-parameter
  pFences **must** be a valid pointer to an array of fenceCount valid VkFence handles
- VUID-vkWaitForFences-fenceCount-arraylength
  fenceCount **must** be greater than 0
- VUID-vkWaitForFences-pFences-parent
  Each element of pFences **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS
- VK_TIMEOUT

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

An execution dependency is defined by waiting for a fence to become signaled, either via vkWaitForFences or by polling on vkGetFenceStatus.

The first synchronization scope includes only the fence signal operation.

The second synchronization scope includes the host operations of vkWaitForFences or vkGetFenceStatus indicating that the fence has become signaled.

NOTE	Signaling a fence and waiting on the host does not guarantee that the results of memory accesses will be visible to the host, as the access scope of a memory dependency defined by a fence only includes device access. A memory barrier or other memory dependency **must** be used to guarantee this. See the description of host access types for more information.

7.3.1. Alternate Methods to Signal Fences

Besides submitting a fence to a queue as part of a queue submission command, a fence **may** also be signaled when a particular event occurs on a device or display.

To create a fence that will be signaled when an event occurs on a device, call:

```
// Provided by VK_EXT_display_control
VkResult vkRegisterDeviceEventEXT(
    VkDevice                                    device,
    const VkDeviceEventInfoEXT*                 pDeviceEventInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkFence*                                    pFence);
```

- `device` is a logical device on which the event **may** occur.
- `pDeviceEventInfo` is a pointer to a VkDeviceEventInfoEXT structure describing the event of interest to the application.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pFence` is a pointer to a handle in which the resulting fence object is returned.

Valid Usage (Implicit)

- VUID-vkRegisterDeviceEventEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkRegisterDeviceEventEXT-pDeviceEventInfo-parameter
  `pDeviceEventInfo` **must** be a valid pointer to a valid VkDeviceEventInfoEXT structure
- VUID-vkRegisterDeviceEventEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkRegisterDeviceEventEXT-pFence-parameter
  `pFence` **must** be a valid pointer to a VkFence handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY

The VkDeviceEventInfoEXT structure is defined as:

```
// Provided by VK_EXT_display_control
typedef struct VkDeviceEventInfoEXT {
    VkStructureType         sType;
    const void*             pNext;
    VkDeviceEventTypeEXT    deviceEvent;
} VkDeviceEventInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- device is a VkDeviceEventTypeEXT value specifying when the fence will be signaled.

Valid Usage (Implicit)

- VUID-VkDeviceEventInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEVICE_EVENT_INFO_EXT
- VUID-VkDeviceEventInfoEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkDeviceEventInfoEXT-deviceEvent-parameter
  deviceEvent **must** be a valid VkDeviceEventTypeEXT value

Possible values of VkDeviceEventInfoEXT::device, specifying when a fence will be signaled, are:

```
// Provided by VK_EXT_display_control
typedef enum VkDeviceEventTypeEXT {
    VK_DEVICE_EVENT_TYPE_DISPLAY_HOTPLUG_EXT = 0,
} VkDeviceEventTypeEXT;
```

- VK_DEVICE_EVENT_TYPE_DISPLAY_HOTPLUG_EXT specifies that the fence is signaled when a display is plugged into or unplugged from the specified device. Applications **can** use this notification to determine when they need to re-enumerate the available displays on a device.

To create a fence that will be signaled when an event occurs on a VkDisplayKHR object, call:

```
// Provided by VK_EXT_display_control
VkResult vkRegisterDisplayEventEXT(
    VkDevice                                    device,
    VkDisplayKHR                                display,
    const VkDisplayEventInfoEXT*                pDisplayEventInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkFence*                                    pFence);
```

- `device` is a logical device associated with `display`
- `display` is the display on which the event **may** occur.
- `pDisplayEventInfo` is a pointer to a VkDisplayEventInfoEXT structure describing the event of interest to the application.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pFence` is a pointer to a handle in which the resulting fence object is returned.

Valid Usage (Implicit)

- VUID-vkRegisterDisplayEventEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkRegisterDisplayEventEXT-display-parameter
  `display` **must** be a valid VkDisplayKHR handle
- VUID-vkRegisterDisplayEventEXT-pDisplayEventInfo-parameter
  `pDisplayEventInfo` **must** be a valid pointer to a valid VkDisplayEventInfoEXT structure
- VUID-vkRegisterDisplayEventEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkRegisterDisplayEventEXT-pFence-parameter
  `pFence` **must** be a valid pointer to a VkFence handle
- VUID-vkRegisterDisplayEventEXT-commonparent
  Both of `device`, and `display` **must** have been created, allocated, or retrieved from the same VkPhysicalDevice

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkDisplayEventInfoEXT` structure is defined as:

```
// Provided by VK_EXT_display_control
typedef struct VkDisplayEventInfoEXT {
    VkStructureType          sType;
    const void*              pNext;
    VkDisplayEventTypeEXT    displayEvent;
} VkDisplayEventInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `displayEvent` is a VkDisplayEventTypeEXT specifying when the fence will be signaled.

Valid Usage (Implicit)

- VUID-VkDisplayEventInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_EVENT_INFO_EXT`
- VUID-VkDisplayEventInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDisplayEventInfoEXT-displayEvent-parameter
  `displayEvent` **must** be a valid VkDisplayEventTypeEXT value

Possible values of VkDisplayEventInfoEXT::`displayEvent`, specifying when a fence will be signaled, are:

```
// Provided by VK_EXT_display_control
typedef enum VkDisplayEventTypeEXT {
    VK_DISPLAY_EVENT_TYPE_FIRST_PIXEL_OUT_EXT = 0,
} VkDisplayEventTypeEXT;
```

- `VK_DISPLAY_EVENT_TYPE_FIRST_PIXEL_OUT_EXT` specifies that the fence is signaled when the first pixel of the next display refresh cycle leaves the display engine for the display.

7.3.2. Importing Fence Payloads

Applications **can** import a fence payload into an existing fence using an external fence handle. The effects of the import operation will be either temporary or permanent, as specified by the application. If the import is temporary, the fence will be *restored* to its permanent state the next time that fence is passed to vkResetFences.

NOTE Restoring a fence to its prior permanent payload is a distinct operation from resetting a fence payload. See vkResetFences for more detail.

Performing a subsequent temporary import on a fence before resetting it has no effect on this requirement; the next unsignal of the fence **must** still restore its last permanent state. A permanent payload import behaves as if the target fence was destroyed, and a new fence was created with the

same handle but the imported payload. Because importing a fence payload temporarily or permanently detaches the existing payload from a fence, similar usage restrictions to those applied to `vkDestroyFence` are applied to any command that imports a fence payload. Which of these import types is used is referred to as the import operation's *permanence*. Each handle type supports either one or both types of permanence.

The implementation **must** perform the import operation by either referencing or copying the payload referred to by the specified external fence handle, depending on the handle's type. The import method used is referred to as the handle type's *transference*. When using handle types with reference transference, importing a payload to a fence adds the fence to the set of all fences sharing that payload. This set includes the fence from which the payload was exported. Fence signaling, waiting, and resetting operations performed on any fence in the set **must** behave as if the set were a single fence. Importing a payload using handle types with copy transference creates a duplicate copy of the payload at the time of import, but makes no further reference to it. Fence signaling, waiting, and resetting operations performed on the target of copy imports **must** not affect any other fence or payload.

Export operations have the same transference as the specified handle type's import operations. Additionally, exporting a fence payload to a handle with copy transference has the same side effects on the source fence's payload as executing a fence reset operation. If the fence was using a temporarily imported payload, the fence's prior permanent payload will be restored.

NOTE	The tables Handle Types Supported by `VkImportFenceWin32HandleInfoKHR` and Handle Types Supported by `VkImportFenceFdInfoKHR` define the permanence and transference of each handle type.

External synchronization allows implementations to modify an object's internal state, i.e. payload, without internal synchronization. However, for fences sharing a payload across processes, satisfying the external synchronization requirements of `VkFence` parameters as if all fences in the set were the same object is sometimes infeasible. Satisfying valid usage constraints on the state of a fence would similarly require impractical coordination or levels of trust between processes. Therefore, these constraints only apply to a specific fence handle, not to its payload. For distinct fence objects which share a payload:

- If multiple commands which queue a signal operation, or which unsignal a fence, are called concurrently, behavior will be as if the commands were called in an arbitrary sequential order.
- If a queue submission command is called with a fence that is sharing a payload, and the payload is already associated with another queue command that has not yet completed execution, either one or both of the commands will cause the fence to become signaled when they complete execution.
- If a fence payload is reset while it is associated with a queue command that has not yet completed execution, the payload will become unsignaled, but **may** become signaled again when the command completes execution.
- In the preceding cases, any of the devices associated with the fences sharing the payload **may** be lost, or any of the queue submission or fence reset commands **may** return `VK_ERROR_INITIALIZATION_FAILED`.

Other than these non-deterministic results, behavior is well defined. In particular:

- The implementation **must** not crash or enter an internally inconsistent state where future valid Vulkan commands might cause undefined results,
- Timeouts on future wait commands on fences sharing the payload **must** be effective.

NOTE

These rules allow processes to synchronize access to shared memory without trusting each other. However, such processes must still be cautious not to use the shared fence for more than synchronizing access to the shared memory. For example, a process should not use a fence with shared payload to tell when commands it submitted to a queue have completed and objects used by those commands may be destroyed, since the other process can accidentally or maliciously cause the fence to signal before the commands actually complete.

When a fence is using an imported payload, its VkExportFenceCreateInfo::`handleTypes` value is specified when creating the fence from which the payload was exported, rather than specified when creating the fence. Additionally, VkExternalFenceProperties::`exportFromImportedHandleTypes` restricts which handle types **can** be exported from such a fence based on the specific handle type used to import the current payload. Passing a fence to vkAcquireNextImageKHR is equivalent to temporarily importing a fence payload to that fence.

NOTE

Because the exportable handle types of an imported fence correspond to its current imported payload, and vkAcquireNextImageKHR behaves the same as a temporary import operation for which the source fence is opaque to the application, applications have no way of determining whether any external handle types **can** be exported from a fence in this state. Therefore, applications **must** not attempt to export handles from fences using a temporarily imported payload from vkAcquireNextImageKHR.

When importing a fence payload, it is the responsibility of the application to ensure the external handles meet all valid usage requirements. However, implementations **must** perform sufficient validation of external handles to ensure that the operation results in a valid fence which will not cause program termination, device loss, queue stalls, host thread stalls, or corruption of other resources when used as allowed according to its import parameters. If the external handle provided does not meet these requirements, the implementation **must** fail the fence payload import operation with the error code `VK_ERROR_INVALID_EXTERNAL_HANDLE`.

To import a fence payload from a Windows handle, call:

```
// Provided by VK_KHR_external_fence_win32
VkResult vkImportFenceWin32HandleKHR(
    VkDevice                                    device,
    const VkImportFenceWin32HandleInfoKHR*      pImportFenceWin32HandleInfo);
```

- `device` is the logical device that created the fence.
- `pImportFenceWin32HandleInfo` is a pointer to a VkImportFenceWin32HandleInfoKHR structure specifying the fence and import parameters.

Importing a fence payload from Windows handles does not transfer ownership of the handle to the Vulkan implementation. For handle types defined as NT handles, the application **must** release ownership using the `CloseHandle` system call when the handle is no longer needed.

Applications **can** import the same fence payload into multiple instances of Vulkan, into the same instance from which it was exported, and multiple times into a given Vulkan instance.

Valid Usage

- VUID-vkImportFenceWin32HandleKHR-fence-04448
  `fence` **must** not be associated with any queue command that has not yet completed execution on that queue

Valid Usage (Implicit)

- VUID-vkImportFenceWin32HandleKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkImportFenceWin32HandleKHR-pImportFenceWin32HandleInfo-parameter
  `pImportFenceWin32HandleInfo` **must** be a valid pointer to a valid VkImportFenceWin32HandleInfoKHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_INVALID_EXTERNAL_HANDLE`

The `VkImportFenceWin32HandleInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_fence_win32
typedef struct VkImportFenceWin32HandleInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkFence                               fence;
    VkFenceImportFlags                    flags;
    VkExternalFenceHandleTypeFlagBits     handleType;
    HANDLE                                handle;
    LPCWSTR                               name;
} VkImportFenceWin32HandleInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fence` is the fence into which the state will be imported.
- `flags` is a bitmask of VkFenceImportFlagBits specifying additional parameters for the fence payload import operation.
- `handleType` is a VkExternalFenceHandleTypeFlagBits value specifying the type of `handle`.
- `handle` is `NULL` or the external handle to import.
- `name` is `NULL` or a null-terminated UTF-16 string naming the underlying synchronization primitive to import.

The handle types supported by `handleType` are:

Table 6. Handle Types Supported by `VkImportFenceWin32HandleInfoKHR`

Handle Type	Transference	Permanence Supported
`VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT`	Reference	Temporary,Permanent
`VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT`	Reference	Temporary,Permanent

Valid Usage

- VUID-VkImportFenceWin32HandleInfoKHR-handleType-01457
  `handleType` **must** be a value included in the Handle Types Supported by `VkImportFenceWin32HandleInfoKHR` table
- VUID-VkImportFenceWin32HandleInfoKHR-handleType-01459
  If `handleType` is not `VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT`, `name` **must** be `NULL`
- VUID-VkImportFenceWin32HandleInfoKHR-handleType-01460
  If `handle` is `NULL`, `name` **must** name a valid synchronization primitive of the type specified by `handleType`
- VUID-VkImportFenceWin32HandleInfoKHR-handleType-01461
  If `name` is `NULL`, `handle` **must** be a valid handle of the type specified by `handleType`
- VUID-VkImportFenceWin32HandleInfoKHR-handle-01462
  If `handle` is not `NULL`, `name` **must** be `NULL`
- VUID-VkImportFenceWin32HandleInfoKHR-handle-01539
  If `handle` is not `NULL`, it **must** obey any requirements listed for `handleType` in external fence handle types compatibility
- VUID-VkImportFenceWin32HandleInfoKHR-name-01540
  If `name` is not `NULL`, it **must** obey any requirements listed for `handleType` in external fence handle types compatibility

Valid Usage (Implicit)

- VUID-VkImportFenceWin32HandleInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_FENCE_WIN32_HANDLE_INFO_KHR`
- VUID-VkImportFenceWin32HandleInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImportFenceWin32HandleInfoKHR-fence-parameter
  `fence` **must** be a valid VkFence handle
- VUID-VkImportFenceWin32HandleInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkFenceImportFlagBits values

Host Synchronization

- Host access to `fence` **must** be externally synchronized

To import a fence payload from a POSIX file descriptor, call:

```
// Provided by VK_KHR_external_fence_fd
VkResult vkImportFenceFdKHR(
    VkDevice                                    device,
    const VkImportFenceFdInfoKHR*               pImportFenceFdInfo);
```

- `device` is the logical device that created the fence.
- `pImportFenceFdInfo` is a pointer to a VkImportFenceFdInfoKHR structure specifying the fence and import parameters.

Importing a fence payload from a file descriptor transfers ownership of the file descriptor from the application to the Vulkan implementation. The application **must** not perform any operations on the file descriptor after a successful import.

Applications **can** import the same fence payload into multiple instances of Vulkan, into the same instance from which it was exported, and multiple times into a given Vulkan instance.

Valid Usage

- VUID-vkImportFenceFdKHR-fence-01463
  `fence` **must** not be associated with any queue command that has not yet completed execution on that queue

Valid Usage (Implicit)

- VUID-vkImportFenceFdKHR-device-parameter

`device` **must** be a valid VkDevice handle

- VUID-vkImportFenceFdKHR-pImportFenceFdInfo-parameter
  `pImportFenceFdInfo` **must** be a valid pointer to a valid VkImportFenceFdInfoKHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_INVALID_EXTERNAL_HANDLE`

The `VkImportFenceFdInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_fence_fd
typedef struct VkImportFenceFdInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkFence                               fence;
    VkFenceImportFlags                    flags;
    VkExternalFenceHandleTypeFlagBits     handleType;
    int                                   fd;
} VkImportFenceFdInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fence` is the fence into which the payload will be imported.
- `flags` is a bitmask of VkFenceImportFlagBits specifying additional parameters for the fence payload import operation.
- `handleType` is a VkExternalFenceHandleTypeFlagBits value specifying the type of `fd`.
- `fd` is the external handle to import.

The handle types supported by `handleType` are:

Table 7. Handle Types Supported by `VkImportFenceFdInfoKHR`

Handle Type	Transference	Permanence Supported
`VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_FD_BIT`	Reference	Temporary,Permanent
`VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT`	Copy	Temporary

Valid Usage

- VUID-VkImportFenceFdInfoKHR-handleType-01464
  `handleType` **must** be a value included in the Handle Types Supported by `VkImportFenceFdInfoKHR` table
- VUID-VkImportFenceFdInfoKHR-fd-01541
  `fd` **must** obey any requirements listed for `handleType` in external fence handle types compatibility
- VUID-VkImportFenceFdInfoKHR-handleType-07306
  If `handleType` refers to a handle type with copy payload transference semantics, `flags` **must** contain `VK_FENCE_IMPORT_TEMPORARY_BIT`

If `handleType` is `VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT`, the special value `-1` for `fd` is treated like a valid sync file descriptor referring to an object that has already signaled. The import operation will succeed and the `VkFence` will have a temporarily imported payload as if a valid file descriptor had been provided.

NOTE

This special behavior for importing an invalid sync file descriptor allows easier interoperability with other system APIs which use the convention that an invalid sync file descriptor represents work that has already completed and does not need to be waited for. It is consistent with the option for implementations to return a `-1` file descriptor when exporting a `VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT` from a `VkFence` which is signaled.

Valid Usage (Implicit)

- VUID-VkImportFenceFdInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_FENCE_FD_INFO_KHR`
- VUID-VkImportFenceFdInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImportFenceFdInfoKHR-fence-parameter
  `fence` **must** be a valid VkFence handle
- VUID-VkImportFenceFdInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkFenceImportFlagBits values
- VUID-VkImportFenceFdInfoKHR-handleType-parameter
  `handleType` **must** be a valid VkExternalFenceHandleTypeFlagBits value

Host Synchronization

- Host access to `fence` **must** be externally synchronized

Bits which **can** be set in

- VkImportFenceWin32HandleInfoKHR::flags
- VkImportFenceFdInfoKHR::flags

specifying additional parameters of a fence import operation are:

```
typedef enum VkFenceImportFlagBits {
    VK_FENCE_IMPORT_TEMPORARY_BIT = 0x00000001,
  // Provided by VK_KHR_external_fence
    VK_FENCE_IMPORT_TEMPORARY_BIT_KHR = VK_FENCE_IMPORT_TEMPORARY_BIT,
} VkFenceImportFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_external_fence
typedef VkFenceImportFlagBits VkFenceImportFlagBitsKHR;
```

- VK_FENCE_IMPORT_TEMPORARY_BIT specifies that the fence payload will be imported only temporarily, as described in Importing Fence Payloads, regardless of the permanence of handleType.

```
typedef VkFlags VkFenceImportFlags;
```

or the equivalent

```
// Provided by VK_KHR_external_fence
typedef VkFenceImportFlags VkFenceImportFlagsKHR;
```

VkFenceImportFlags is a bitmask type for setting a mask of zero or more VkFenceImportFlagBits.

7.4. Semaphores

Semaphores are a synchronization primitive that **can** be used to insert a dependency between queue operations or between a queue operation and the host. Binary semaphores have two states - signaled and unsignaled. Timeline semaphores have a strictly increasing 64-bit unsigned integer payload and are signaled with respect to a particular reference value. A semaphore **can** be signaled after execution of a queue operation is completed, and a queue operation **can** wait for a semaphore to become signaled before it begins execution. A timeline semaphore **can** additionally be signaled from the host with the vkSignalSemaphoreKHR command and waited on from the host with the vkWaitSemaphoresKHR command.

The internal data of a semaphore **may** include a reference to any resources and pending work associated with signal or unsignal operations performed on that semaphore object, collectively referred to as the semaphore's *payload*. Mechanisms to import and export that internal data to and from semaphores are provided below. These mechanisms indirectly enable applications to share

semaphore state between two or more semaphores and other synchronization primitives across process and API boundaries.

Semaphores are represented by VkSemaphore handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkSemaphore)
```

To create a semaphore, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateSemaphore(
    VkDevice                                    device,
    const VkSemaphoreCreateInfo*                pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSemaphore*                                pSemaphore);
```

- device is the logical device that creates the semaphore.
- pCreateInfo is a pointer to a VkSemaphoreCreateInfo structure containing information about how the semaphore is to be created.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.
- pSemaphore is a pointer to a handle in which the resulting semaphore object is returned.

Valid Usage (Implicit)

- VUID-vkCreateSemaphore-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateSemaphore-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkSemaphoreCreateInfo structure
- VUID-vkCreateSemaphore-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateSemaphore-pSemaphore-parameter
  pSemaphore **must** be a valid pointer to a VkSemaphore handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkSemaphoreCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSemaphoreCreateInfo {
    VkStructureType           sType;
    const void*               pNext;
    VkSemaphoreCreateFlags    flags;
} VkSemaphoreCreateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.

Valid Usage

- VUID-VkSemaphoreCreateInfo-pNext-06789
  If the pNext chain includes a VkExportMetalObjectCreateInfoEXT structure, its exportObjectType member **must** be VK_EXPORT_METAL_OBJECT_TYPE_METAL_SHARED_EVENT_BIT_EXT

Valid Usage (Implicit)

- VUID-VkSemaphoreCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO
- VUID-VkSemaphoreCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkExportMetalObjectCreateInfoEXT, VkExportSemaphoreCreateInfo, VkExportSemaphoreWin32HandleInfoKHR, VkImportMetalSharedEventInfoEXT, or VkSemaphoreTypeCreateInfo
- VUID-VkSemaphoreCreateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique, with the exception of structures of type VkExportMetalObjectCreateInfoEXT
- VUID-VkSemaphoreCreateInfo-flags-zerobitmask
  flags **must** be 0

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkSemaphoreCreateFlags;
```

VkSemaphoreCreateFlags is a bitmask type for setting a mask, but is currently reserved for future use.

The VkSemaphoreTypeCreateInfo structure is defined as:

```
typedef struct VkSemaphoreTypeCreateInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkSemaphoreType    semaphoreType;
    uint64_t           initialValue;
} VkSemaphoreTypeCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkSemaphoreTypeCreateInfo VkSemaphoreTypeCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `semaphoreType` is a VkSemaphoreType value specifying the type of the semaphore.
- `initialValue` is the initial payload value if `semaphoreType` is `VK_SEMAPHORE_TYPE_TIMELINE`.

To create a semaphore of a specific type, add a `VkSemaphoreTypeCreateInfo` structure to the VkSemaphoreCreateInfo::`pNext` chain.

If no `VkSemaphoreTypeCreateInfo` structure is included in the `pNext` chain of VkSemaphoreCreateInfo, then the created semaphore will have a default VkSemaphoreType of `VK_SEMAPHORE_TYPE_BINARY`.

Valid Usage

- VUID-VkSemaphoreTypeCreateInfo-timelineSemaphore-03252
  If the `timelineSemaphore` feature is not enabled, `semaphoreType` **must** not equal `VK_SEMAPHORE_TYPE_TIMELINE`
- VUID-VkSemaphoreTypeCreateInfo-semaphoreType-03279
  If `semaphoreType` is `VK_SEMAPHORE_TYPE_BINARY`, `initialValue` **must** be zero

Valid Usage (Implicit)

- VUID-VkSemaphoreTypeCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO`
- VUID-VkSemaphoreTypeCreateInfo-semaphoreType-parameter
  `semaphoreType` **must** be a valid VkSemaphoreType value

Possible values of VkSemaphoreTypeCreateInfo::`semaphoreType`, specifying the type of a semaphore, are:

```
typedef enum VkSemaphoreType {
    VK_SEMAPHORE_TYPE_BINARY = 0,
    VK_SEMAPHORE_TYPE_TIMELINE = 1,
  // Provided by VK_KHR_timeline_semaphore
    VK_SEMAPHORE_TYPE_BINARY_KHR = VK_SEMAPHORE_TYPE_BINARY,
  // Provided by VK_KHR_timeline_semaphore
    VK_SEMAPHORE_TYPE_TIMELINE_KHR = VK_SEMAPHORE_TYPE_TIMELINE,
} VkSemaphoreType;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkSemaphoreType VkSemaphoreTypeKHR;
```

- VK_SEMAPHORE_TYPE_BINARY specifies a *binary semaphore* type that has a boolean payload indicating whether the semaphore is currently signaled or unsignaled. When created, the semaphore is in the unsignaled state.
- VK_SEMAPHORE_TYPE_TIMELINE specifies a *timeline semaphore* type that has a strictly increasing 64-bit unsigned integer payload indicating whether the semaphore is signaled with respect to a particular reference value. When created, the semaphore payload has the value given by the initialValue field of VkSemaphoreTypeCreateInfo.

To create a semaphore whose payload **can** be exported to external handles, add a VkExportSemaphoreCreateInfo structure to the pNext chain of the VkSemaphoreCreateInfo structure. The VkExportSemaphoreCreateInfo structure is defined as:

```
typedef struct VkExportSemaphoreCreateInfo {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExternalSemaphoreHandleTypeFlags    handleTypes;
} VkExportSemaphoreCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore
typedef VkExportSemaphoreCreateInfo VkExportSemaphoreCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- handleTypes is a bitmask of VkExternalSemaphoreHandleTypeFlagBits specifying one or more semaphore handle types the application **can** export from the resulting semaphore. The application **can** request multiple handle types for the same semaphore.

Valid Usage

- VUID-VkExportSemaphoreCreateInfo-handleTypes-01124
  The bits in `handleTypes` **must** be supported and compatible, as reported by VkExternalSemaphoreProperties

Valid Usage (Implicit)

- VUID-VkExportSemaphoreCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_CREATE_INFO`
- VUID-VkExportSemaphoreCreateInfo-handleTypes-parameter
  `handleTypes` **must** be a valid combination of VkExternalSemaphoreHandleTypeFlagBits values

To specify additional attributes of NT handles exported from a semaphore, add a `VkExportSemaphoreWin32HandleInfoKHR` structure to the `pNext` chain of the VkSemaphoreCreateInfo structure. The `VkExportSemaphoreWin32HandleInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_semaphore_win32
typedef struct VkExportSemaphoreWin32HandleInfoKHR {
    VkStructureType               sType;
    const void*                   pNext;
    const SECURITY_ATTRIBUTES*    pAttributes;
    DWORD                         dwAccess;
    LPCWSTR                       name;
} VkExportSemaphoreWin32HandleInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pAttributes` is a pointer to a Windows `SECURITY_ATTRIBUTES` structure specifying security attributes of the handle.
- `dwAccess` is a `DWORD` specifying access rights of the handle.
- `name` is a null-terminated UTF-16 string to associate with the underlying synchronization primitive referenced by NT handles exported from the created semaphore.

If VkExportSemaphoreCreateInfo is not included in the same `pNext` chain, this structure is ignored.

If VkExportSemaphoreCreateInfo is included in the `pNext` chain of VkSemaphoreCreateInfo with a Windows `handleType`, but either `VkExportSemaphoreWin32HandleInfoKHR` is not included in the `pNext` chain, or it is included but `pAttributes` is `NULL`, default security descriptor values will be used, and child processes created by the application will not inherit the handle, as described in the MSDN documentation for “Synchronization Object Security and Access Rights”[1]. Further, if the structure is not present, the access rights used depend on the handle type.

For handles of the following types:

`VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT`

The implementation **must** ensure the access rights allow both signal and wait operations on the semaphore.

For handles of the following types:

`VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT`

The access rights **must** be:

`GENERIC_ALL`

1

https://docs.microsoft.com/en-us/windows/win32/sync/synchronization-object-security-and-access-rights

Valid Usage

- VUID-VkExportSemaphoreWin32HandleInfoKHR-handleTypes-01125
  If VkExportSemaphoreCreateInfo::`handleTypes` does not include `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT` or `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT`, `VkExportSemaphoreWin32HandleInfoKHR` **must** not be included in the `pNext` chain of VkSemaphoreCreateInfo

Valid Usage (Implicit)

- VUID-VkExportSemaphoreWin32HandleInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_WIN32_HANDLE_INFO_KHR`
- VUID-VkExportSemaphoreWin32HandleInfoKHR-pAttributes-parameter
  If `pAttributes` is not `NULL`, `pAttributes` **must** be a valid pointer to a valid `SECURITY_ATTRIBUTES` value

To export a Windows handle representing the payload of a semaphore, call:

```
// Provided by VK_KHR_external_semaphore_win32
VkResult vkGetSemaphoreWin32HandleKHR(
    VkDevice                                    device,
    const VkSemaphoreGetWin32HandleInfoKHR*     pGetWin32HandleInfo,
    HANDLE*                                     pHandle);
```

- `device` is the logical device that created the semaphore being exported.
- `pGetWin32HandleInfo` is a pointer to a VkSemaphoreGetWin32HandleInfoKHR structure containing parameters of the export operation.

- pHandle will return the Windows handle representing the semaphore state.

For handle types defined as NT handles, the handles returned by vkGetSemaphoreWin32HandleKHR are owned by the application. To avoid leaking resources, the application **must** release ownership of them using the CloseHandle system call when they are no longer needed.

Exporting a Windows handle from a semaphore **may** have side effects depending on the transference of the specified handle type, as described in Importing Semaphore Payloads.

Valid Usage (Implicit)

- VUID-vkGetSemaphoreWin32HandleKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetSemaphoreWin32HandleKHR-pGetWin32HandleInfo-parameter
  pGetWin32HandleInfo **must** be a valid pointer to a valid VkSemaphoreGetWin32HandleInfoKHR structure
- VUID-vkGetSemaphoreWin32HandleKHR-pHandle-parameter
  pHandle **must** be a valid pointer to a HANDLE value

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_TOO_MANY_OBJECTS
- VK_ERROR_OUT_OF_HOST_MEMORY

The VkSemaphoreGetWin32HandleInfoKHR structure is defined as:

```
// Provided by VK_KHR_external_semaphore_win32
typedef struct VkSemaphoreGetWin32HandleInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkSemaphore                           semaphore;
    VkExternalSemaphoreHandleTypeFlagBits handleType;
} VkSemaphoreGetWin32HandleInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- semaphore is the semaphore from which state will be exported.
- handleType is a VkExternalSemaphoreHandleTypeFlagBits value specifying the type of handle requested.

The properties of the handle returned depend on the value of handleType. See VkExternalSemaphoreHandleTypeFlagBits for a description of the properties of the defined external semaphore handle types.

Valid Usage

- VUID-VkSemaphoreGetWin32HandleInfoKHR-handleType-01126
  handleType **must** have been included in VkExportSemaphoreCreateInfo::handleTypes when the semaphore's current payload was created
- VUID-VkSemaphoreGetWin32HandleInfoKHR-handleType-01127
  If handleType is defined as an NT handle, vkGetSemaphoreWin32HandleKHR **must** be called no more than once for each valid unique combination of semaphore and handleType
- VUID-VkSemaphoreGetWin32HandleInfoKHR-semaphore-01128
  semaphore **must** not currently have its payload replaced by an imported payload as described below in Importing Semaphore Payloads unless that imported payload's handle type was included in VkExternalSemaphoreProperties::exportFromImportedHandleTypes for handleType
- VUID-VkSemaphoreGetWin32HandleInfoKHR-handleType-01129
  If handleType refers to a handle type with copy payload transference semantics, as defined below in Importing Semaphore Payloads, there **must** be no queue waiting on semaphore
- VUID-VkSemaphoreGetWin32HandleInfoKHR-handleType-01130
  If handleType refers to a handle type with copy payload transference semantics, semaphore **must** be signaled, or have an associated semaphore signal operation pending execution
- VUID-VkSemaphoreGetWin32HandleInfoKHR-handleType-01131
  handleType **must** be defined as an NT handle or a global share handle

Valid Usage (Implicit)

- VUID-VkSemaphoreGetWin32HandleInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SEMAPHORE_GET_WIN32_HANDLE_INFO_KHR
- VUID-VkSemaphoreGetWin32HandleInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkSemaphoreGetWin32HandleInfoKHR-semaphore-parameter
  semaphore **must** be a valid VkSemaphore handle
- VUID-VkSemaphoreGetWin32HandleInfoKHR-handleType-parameter
  handleType **must** be a valid VkExternalSemaphoreHandleTypeFlagBits value

To export a POSIX file descriptor representing the payload of a semaphore, call:

```
// Provided by VK_KHR_external_semaphore_fd
VkResult vkGetSemaphoreFdKHR(
    VkDevice                                    device,
    const VkSemaphoreGetFdInfoKHR*              pGetFdInfo,
    int*                                        pFd);
```

- `device` is the logical device that created the semaphore being exported.
- `pGetFdInfo` is a pointer to a VkSemaphoreGetFdInfoKHR structure containing parameters of the export operation.
- `pFd` will return the file descriptor representing the semaphore payload.

Each call to `vkGetSemaphoreFdKHR` **must** create a new file descriptor and transfer ownership of it to the application. To avoid leaking resources, the application **must** release ownership of the file descriptor when it is no longer needed.

NOTE	Ownership can be released in many ways. For example, the application can call `close`() on the file descriptor, or transfer ownership back to Vulkan by using the file descriptor to import a semaphore payload.

Where supported by the operating system, the implementation **must** set the file descriptor to be closed automatically when an `execve` system call is made.

Exporting a file descriptor from a semaphore **may** have side effects depending on the transference of the specified handle type, as described in Importing Semaphore State.

Valid Usage (Implicit)

- VUID-vkGetSemaphoreFdKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetSemaphoreFdKHR-pGetFdInfo-parameter
  `pGetFdInfo` **must** be a valid pointer to a valid VkSemaphoreGetFdInfoKHR structure
- VUID-vkGetSemaphoreFdKHR-pFd-parameter
  `pFd` **must** be a valid pointer to an `int` value

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_TOO_MANY_OBJECTS`
- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkSemaphoreGetFdInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_semaphore_fd
typedef struct VkSemaphoreGetFdInfoKHR {
    VkStructureType                           sType;
    const void*                               pNext;
    VkSemaphore                               semaphore;
    VkExternalSemaphoreHandleTypeFlagBits     handleType;
} VkSemaphoreGetFdInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `semaphore` is the semaphore from which state will be exported.
- `handleType` is a VkExternalSemaphoreHandleTypeFlagBits value specifying the type of handle requested.

The properties of the file descriptor returned depend on the value of `handleType`. See VkExternalSemaphoreHandleTypeFlagBits for a description of the properties of the defined external semaphore handle types.

Valid Usage

- VUID-VkSemaphoreGetFdInfoKHR-handleType-01132
  `handleType` **must** have been included in VkExportSemaphoreCreateInfo::`handleTypes` when `semaphore`'s current payload was created
- VUID-VkSemaphoreGetFdInfoKHR-semaphore-01133
  `semaphore` **must** not currently have its payload replaced by an imported payload as described below in Importing Semaphore Payloads unless that imported payload's handle type was included in VkExternalSemaphoreProperties::`exportFromImportedHandleTypes` for `handleType`
- VUID-VkSemaphoreGetFdInfoKHR-handleType-01134
  If `handleType` refers to a handle type with copy payload transference semantics, as defined below in Importing Semaphore Payloads, there **must** be no queue waiting on `semaphore`
- VUID-VkSemaphoreGetFdInfoKHR-handleType-01135
  If `handleType` refers to a handle type with copy payload transference semantics, `semaphore` **must** be signaled, or have an associated semaphore signal operation pending execution
- VUID-VkSemaphoreGetFdInfoKHR-handleType-01136
  `handleType` **must** be defined as a POSIX file descriptor handle
- VUID-VkSemaphoreGetFdInfoKHR-handleType-03253
  If `handleType` refers to a handle type with copy payload transference semantics, `semaphore` **must** have been created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_BINARY`
- VUID-VkSemaphoreGetFdInfoKHR-handleType-03254
  If `handleType` refers to a handle type with copy payload transference semantics, `semaphore` **must** have an associated semaphore signal operation that has been submitted for execution and any semaphore signal operations on which it depends **must** have also been

submitted for execution

Valid Usage (Implicit)

- VUID-VkSemaphoreGetFdInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SEMAPHORE_GET_FD_INFO_KHR`
- VUID-VkSemaphoreGetFdInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkSemaphoreGetFdInfoKHR-semaphore-parameter
  `semaphore` **must** be a valid VkSemaphore handle
- VUID-VkSemaphoreGetFdInfoKHR-handleType-parameter
  `handleType` **must** be a valid VkExternalSemaphoreHandleTypeFlagBits value

To destroy a semaphore, call:

```
// Provided by VK_VERSION_1_0
void vkDestroySemaphore(
    VkDevice                                    device,
    VkSemaphore                                 semaphore,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the semaphore.
- `semaphore` is the handle of the semaphore to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroySemaphore-semaphore-05149
  All submitted batches that refer to `semaphore` **must** have completed execution
- VUID-vkDestroySemaphore-semaphore-01138
  If `VkAllocationCallbacks` were provided when `semaphore` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroySemaphore-semaphore-01139
  If no `VkAllocationCallbacks` were provided when `semaphore` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroySemaphore-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroySemaphore-semaphore-parameter

If `semaphore` is not VK_NULL_HANDLE, `semaphore` **must** be a valid VkSemaphore handle

- VUID-vkDestroySemaphore-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroySemaphore-semaphore-parent
  If `semaphore` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `semaphore` **must** be externally synchronized

7.4.1. Semaphore Signaling

When a batch is submitted to a queue via a queue submission, and it includes semaphores to be signaled, it defines a memory dependency on the batch, and defines *semaphore signal operations* which set the semaphores to the signaled state.

In case of semaphores created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_TIMELINE` the semaphore is considered signaled with respect to the counter value set to be signaled as specified in VkTimelineSemaphoreSubmitInfo or VkSemaphoreSignalInfo.

The first synchronization scope includes every command submitted in the same batch. In the case of vkQueueSubmit2KHR, the first synchronization scope is limited to the pipeline stage specified by VkSemaphoreSubmitInfo::`stageMask`. Semaphore signal operations that are defined by vkQueueSubmit or vkQueueSubmit2KHR additionally include all commands that occur earlier in submission order. Semaphore signal operations that are defined by vkQueueSubmit or vkQueueSubmit2KHR or vkQueueBindSparse additionally include in the first synchronization scope any semaphore and fence signal operations that occur earlier in signal operation order.

The second synchronization scope includes only the semaphore signal operation.

The first access scope includes all memory access performed by the device.

The second access scope is empty.

7.4.2. Semaphore Waiting

When a batch is submitted to a queue via a queue submission, and it includes semaphores to be waited on, it defines a memory dependency between prior semaphore signal operations and the batch, and defines *semaphore wait operations*.

Such semaphore wait operations set the semaphores created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_BINARY` to the unsignaled state. In case of semaphores created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_TIMELINE` a prior semaphore signal operation defines a memory dependency with a semaphore wait operation if the value the semaphore is signaled with is greater than or equal to the value the semaphore is waited with, thus the semaphore will

continue to be considered signaled with respect to the counter value waited on as specified in VkTimelineSemaphoreSubmitInfo.

The first synchronization scope includes one semaphore signal operation for each semaphore waited on by this batch. The specific signal operation waited on for each semaphore **must** meet the following criteria:

- for binary semaphores, the signal operation is either earlier in submission order on the same queue, or is submitted by a command whose host operation happens-before this batch is submitted on the host
- for binary semaphores, no wait operation exists that happens-after the signal operation and happens-before this wait operation
- the signal operation is not guaranteed to happen-after the semaphore wait operation in this batch
- for timeline semaphores, the signal value is greater than or equal to the wait value

If multiple semaphore signal operations meet these criteria, any of those operations **may** be included in the first synchronization scope. When waiting on a binary semaphore, applications **must** ensure that exactly one semaphore signal operation meets these criteria.

The second synchronization scope includes every command submitted in the same batch. In the case of vkQueueSubmit, the second synchronization scope is limited to operations on the pipeline stages determined by the destination stage mask specified by the corresponding element of `pWaitDstStageMask`. In the case of vkQueueSubmit2KHR, the second synchronization scope is limited to the pipeline stage specified by VkSemaphoreSubmitInfo::`stageMask`. Also, in the case of either vkQueueSubmit2KHR or vkQueueSubmit, the second synchronization scope additionally includes all commands that occur later in submission order.

The first access scope is empty.

The second access scope includes all memory access performed by the device.

The semaphore wait operation happens-after the first set of operations in the execution dependency, and happens-before the second set of operations in the execution dependency.

NOTE Unlike timeline semaphores, fences or events, waiting for a binary semaphore also unsignals that semaphore when the wait completes. Applications **must** ensure that between two such wait operations, the semaphore is signaled again, with execution dependencies used to ensure these occur in order. Binary semaphore waits and signals should thus occur in discrete 1:1 pairs.

NOTE A common scenario for using `pWaitDstStageMask` with values other than `VK_PIPELINE_STAGE_ALL_COMMANDS_BIT` is when synchronizing a window system presentation operation against subsequent command buffers which render the next frame. In this case, a presentation image **must** not be overwritten until the presentation operation completes, but other pipeline stages **can** execute without waiting. A mask of `VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT` prevents subsequent color attachment writes from executing until the semaphore signals.

Some implementations **may** be able to execute transfer operations and/or pre-rasterization work before the semaphore is signaled.

If an image layout transition needs to be performed on a presentable image before it is used in a framebuffer, that **can** be performed as the first operation submitted to the queue after acquiring the image, and **should** not prevent other work from overlapping with the presentation operation. For example, a `VkImageMemoryBarrier` could use:

- `srcStageMask` = `VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT`
- `srcAccessMask` = 0
- `dstStageMask` = `VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT`
- `dstAccessMask` = `VK_ACCESS_COLOR_ATTACHMENT_READ_BIT` | `VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT`.
- `oldLayout` = `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR`
- `newLayout` = `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`

Alternatively, `oldLayout` **can** be `VK_IMAGE_LAYOUT_UNDEFINED`, if the image's contents need not be preserved.

This barrier accomplishes a dependency chain between previous presentation operations and subsequent color attachment output operations, with the layout transition performed in between, and does not introduce a dependency between previous work and any pre-rasterization shader stages. More precisely, the semaphore signals after the presentation operation completes, the semaphore wait stalls the `VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT` stage, and there is a dependency from that same stage to itself with the layout transition performed in between.

7.4.3. Semaphore State Requirements for Wait Operations

Before waiting on a semaphore, the application **must** ensure the semaphore is in a valid state for a wait operation. Specifically, when a semaphore wait operation is submitted to a queue:

- A binary semaphore **must** be signaled, or have an associated semaphore signal operation that is pending execution.
- Any semaphore signal operations on which the pending binary semaphore signal operation depends **must** also be completed or pending execution.
- There **must** be no other queue waiting on the same binary semaphore when the operation executes.

7.4.4. Host Operations on Semaphores

In addition to semaphore signal operations and semaphore wait operations submitted to device queues, timeline semaphores support the following host operations:

- Query the current counter value of the semaphore using the vkGetSemaphoreCounterValueKHR command.
- Wait for a set of semaphores to reach particular counter values using the vkWaitSemaphoresKHR command.
- Signal the semaphore with a particular counter value from the host using the vkSignalSemaphoreKHR command.

To query the current counter value of a semaphore created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE from the host, call:

```
// Provided by VK_KHR_timeline_semaphore
VkResult vkGetSemaphoreCounterValueKHR(
    VkDevice                                    device,
    VkSemaphore                                 semaphore,
    uint64_t*                                   pValue);
```

- device is the logical device that owns the semaphore.
- semaphore is the handle of the semaphore to query.
- pValue is a pointer to a 64-bit integer value in which the current counter value of the semaphore is returned.

NOTE If a queue submission command is pending execution, then the value returned by this command **may** immediately be out of date.

Valid Usage

- VUID-vkGetSemaphoreCounterValue-semaphore-03255
  semaphore **must** have been created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE

Valid Usage (Implicit)

- VUID-vkGetSemaphoreCounterValue-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetSemaphoreCounterValue-semaphore-parameter
  semaphore **must** be a valid VkSemaphore handle
- VUID-vkGetSemaphoreCounterValue-pValue-parameter
  pValue **must** be a valid pointer to a uint64_t value
- VUID-vkGetSemaphoreCounterValue-semaphore-parent
  semaphore **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

To wait for a set of semaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE to reach particular counter values on the host, call:

```
// Provided by VK_KHR_timeline_semaphore
VkResult vkWaitSemaphoresKHR(
    VkDevice                                    device,
    const VkSemaphoreWaitInfo*                  pWaitInfo,
    uint64_t                                    timeout);
```

- device is the logical device that owns the semaphores.
- pWaitInfo is a pointer to a VkSemaphoreWaitInfo structure containing information about the wait condition.
- timeout is the timeout period in units of nanoseconds. timeout is adjusted to the closest value allowed by the implementation-dependent timeout accuracy, which **may** be substantially longer than one nanosecond, and **may** be longer than the requested period.

If the condition is satisfied when vkWaitSemaphoresKHR is called, then vkWaitSemaphoresKHR returns immediately. If the condition is not satisfied at the time vkWaitSemaphoresKHR is called, then vkWaitSemaphoresKHR will block and wait until the condition is satisfied or the timeout has expired, whichever is sooner.

If timeout is zero, then vkWaitSemaphoresKHR does not wait, but simply returns information about the current state of the semaphores. VK_TIMEOUT will be returned in this case if the condition is not satisfied, even though no actual wait was performed.

If the condition is satisfied before the timeout has expired, vkWaitSemaphoresKHR returns VK_SUCCESS. Otherwise, vkWaitSemaphoresKHR returns VK_TIMEOUT after the timeout has expired.

If device loss occurs (see Lost Device) before the timeout has expired, vkWaitSemaphoresKHR **must** return in finite time with either VK_SUCCESS or VK_ERROR_DEVICE_LOST.

Valid Usage (Implicit)

- VUID-vkWaitSemaphores-device-parameter
  device **must** be a valid VkDevice handle

- VUID-vkWaitSemaphores-pWaitInfo-parameter
  `pWaitInfo` **must** be a valid pointer to a valid VkSemaphoreWaitInfo structure

Return Codes

Success

- `VK_SUCCESS`
- `VK_TIMEOUT`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_DEVICE_LOST`

The `VkSemaphoreWaitInfo` structure is defined as:

```
typedef struct VkSemaphoreWaitInfo {
    VkStructureType         sType;
    const void*             pNext;
    VkSemaphoreWaitFlags    flags;
    uint32_t                semaphoreCount;
    const VkSemaphore*      pSemaphores;
    const uint64_t*         pValues;
} VkSemaphoreWaitInfo;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkSemaphoreWaitInfo VkSemaphoreWaitInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkSemaphoreWaitFlagBits specifying additional parameters for the semaphore wait operation.
- `semaphoreCount` is the number of semaphores to wait on.
- `pSemaphores` is a pointer to an array of `semaphoreCount` semaphore handles to wait on.
- `pValues` is a pointer to an array of `semaphoreCount` timeline semaphore values.

Valid Usage

- VUID-VkSemaphoreWaitInfo-pSemaphores-03256
  All of the elements of `pSemaphores` **must** reference a semaphore that was created with a

VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE

Valid Usage (Implicit)

- VUID-VkSemaphoreWaitInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SEMAPHORE_WAIT_INFO
- VUID-VkSemaphoreWaitInfo-pNext-pNext
  pNext **must** be NULL
- VUID-VkSemaphoreWaitInfo-flags-parameter
  flags **must** be a valid combination of VkSemaphoreWaitFlagBits values
- VUID-VkSemaphoreWaitInfo-pSemaphores-parameter
  pSemaphores **must** be a valid pointer to an array of semaphoreCount valid VkSemaphore handles
- VUID-VkSemaphoreWaitInfo-pValues-parameter
  pValues **must** be a valid pointer to an array of semaphoreCount uint64_t values
- VUID-VkSemaphoreWaitInfo-semaphoreCount-arraylength
  semaphoreCount **must** be greater than 0

Bits which **can** be set in VkSemaphoreWaitInfo::flags, specifying additional parameters of a semaphore wait operation, are:

```
typedef enum VkSemaphoreWaitFlagBits {
    VK_SEMAPHORE_WAIT_ANY_BIT = 0x00000001,
  // Provided by VK_KHR_timeline_semaphore
    VK_SEMAPHORE_WAIT_ANY_BIT_KHR = VK_SEMAPHORE_WAIT_ANY_BIT,
} VkSemaphoreWaitFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkSemaphoreWaitFlagBits VkSemaphoreWaitFlagBitsKHR;
```

- VK_SEMAPHORE_WAIT_ANY_BIT specifies that the semaphore wait condition is that at least one of the semaphores in VkSemaphoreWaitInfo::pSemaphores has reached the value specified by the corresponding element of VkSemaphoreWaitInfo::pValues. If VK_SEMAPHORE_WAIT_ANY_BIT is not set, the semaphore wait condition is that all of the semaphores in VkSemaphoreWaitInfo::pSemaphores have reached the value specified by the corresponding element of VkSemaphoreWaitInfo::pValues.

```
typedef VkFlags VkSemaphoreWaitFlags;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkSemaphoreWaitFlags VkSemaphoreWaitFlagsKHR;
```

VkSemaphoreWaitFlags is a bitmask type for setting a mask of zero or more VkSemaphoreWaitFlagBits.

To signal a semaphore created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE with a particular counter value, on the host, call:

```
// Provided by VK_KHR_timeline_semaphore
VkResult vkSignalSemaphoreKHR(
    VkDevice                                    device,
    const VkSemaphoreSignalInfo*                pSignalInfo);
```

- device is the logical device that owns the semaphore.
- pSignalInfo is a pointer to a VkSemaphoreSignalInfo structure containing information about the signal operation.

When vkSignalSemaphoreKHR is executed on the host, it defines and immediately executes a *semaphore signal operation* which sets the timeline semaphore to the given value.

The first synchronization scope is defined by the host execution model, but includes execution of vkSignalSemaphoreKHR on the host and anything that happened-before it.

The second synchronization scope is empty.

Valid Usage (Implicit)

- VUID-vkSignalSemaphore-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkSignalSemaphore-pSignalInfo-parameter
  pSignalInfo **must** be a valid pointer to a valid VkSemaphoreSignalInfo structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkSemaphoreSignalInfo structure is defined as:

```
typedef struct VkSemaphoreSignalInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkSemaphore        semaphore;
    uint64_t           value;
} VkSemaphoreSignalInfo;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkSemaphoreSignalInfo VkSemaphoreSignalInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `semaphore` is the handle of the semaphore to signal.
- `value` is the value to signal.

Valid Usage

- VUID-VkSemaphoreSignalInfo-semaphore-03257
  `semaphore` **must** have been created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_TIMELINE`
- VUID-VkSemaphoreSignalInfo-value-03258
  `value` **must** have a value greater than the current value of the semaphore
- VUID-VkSemaphoreSignalInfo-value-03259
  `value` **must** be less than the value of any pending semaphore signal operations
- VUID-VkSemaphoreSignalInfo-value-03260
  `value` **must** have a value which does not differ from the current value of the semaphore or the value of any outstanding semaphore wait or signal operation on `semaphore` by more than `maxTimelineSemaphoreValueDifference`

Valid Usage (Implicit)

- VUID-VkSemaphoreSignalInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SEMAPHORE_SIGNAL_INFO`
- VUID-VkSemaphoreSignalInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkSemaphoreSignalInfo-semaphore-parameter
  `semaphore` **must** be a valid VkSemaphore handle

7.4.5. Importing Semaphore Payloads

Applications **can** import a semaphore payload into an existing semaphore using an external semaphore handle. The effects of the import operation will be either temporary or permanent, as specified by the application. If the import is temporary, the implementation **must** restore the semaphore to its prior permanent state after submitting the next semaphore wait operation. Performing a subsequent temporary import on a semaphore before performing a semaphore wait has no effect on this requirement; the next wait submitted on the semaphore **must** still restore its last permanent state. A permanent payload import behaves as if the target semaphore was destroyed, and a new semaphore was created with the same handle but the imported payload. Because importing a semaphore payload temporarily or permanently detaches the existing payload from a semaphore, similar usage restrictions to those applied to `vkDestroySemaphore` are applied to any command that imports a semaphore payload. Which of these import types is used is referred to as the import operation's *permanence*. Each handle type supports either one or both types of permanence.

The implementation **must** perform the import operation by either referencing or copying the payload referred to by the specified external semaphore handle, depending on the handle's type. The import method used is referred to as the handle type's *transference*. When using handle types with reference transference, importing a payload to a semaphore adds the semaphore to the set of all semaphores sharing that payload. This set includes the semaphore from which the payload was exported. Semaphore signaling and waiting operations performed on any semaphore in the set **must** behave as if the set were a single semaphore. Importing a payload using handle types with copy transference creates a duplicate copy of the payload at the time of import, but makes no further reference to it. Semaphore signaling and waiting operations performed on the target of copy imports **must** not affect any other semaphore or payload.

Export operations have the same transference as the specified handle type's import operations. Additionally, exporting a semaphore payload to a handle with copy transference has the same side effects on the source semaphore's payload as executing a semaphore wait operation. If the semaphore was using a temporarily imported payload, the semaphore's prior permanent payload will be restored.

NOTE

The permanence and transference of handle types can be found in:

- Handle Types Supported by `VkImportSemaphoreWin32HandleInfoKHR`
- Handle Types Supported by `VkImportSemaphoreFdInfoKHR`

External synchronization allows implementations to modify an object's internal state, i.e. payload, without internal synchronization. However, for semaphores sharing a payload across processes, satisfying the external synchronization requirements of `VkSemaphore` parameters as if all semaphores in the set were the same object is sometimes infeasible. Satisfying the wait operation state requirements would similarly require impractical coordination or levels of trust between processes. Therefore, these constraints only apply to a specific semaphore handle, not to its payload. For distinct semaphore objects which share a payload, if the semaphores are passed to separate queue submission commands concurrently, behavior will be as if the commands were called in an arbitrary sequential order. If the wait operation state requirements are violated for the shared payload by a queue submission command, or if a signal operation is queued for a shared

payload that is already signaled or has a pending signal operation, effects **must** be limited to one or more of the following:

- Returning `VK_ERROR_INITIALIZATION_FAILED` from the command which resulted in the violation.
- Losing the logical device on which the violation occurred immediately or at a future time, resulting in a `VK_ERROR_DEVICE_LOST` error from subsequent commands, including the one causing the violation.
- Continuing execution of the violating command or operation as if the semaphore wait completed successfully after an implementation-dependent timeout. In this case, the state of the payload becomes undefined, and future operations on semaphores sharing the payload will be subject to these same rules. The semaphore **must** be destroyed or have its payload replaced by an import operation to again have a well-defined state.

NOTE

These rules allow processes to synchronize access to shared memory without trusting each other. However, such processes must still be cautious not to use the shared semaphore for more than synchronizing access to the shared memory. For example, a process should not use a shared semaphore as part of an execution dependency chain that, when complete, leads to objects being destroyed, if it does not trust other processes sharing the semaphore payload.

When a semaphore is using an imported payload, its VkExportSemaphoreCreateInfo::`handleTypes` value is specified when creating the semaphore from which the payload was exported, rather than specified when creating the semaphore. Additionally, VkExternalSemaphoreProperties::`exportFromImportedHandleTypes` restricts which handle types **can** be exported from such a semaphore based on the specific handle type used to import the current payload. Passing a semaphore to vkAcquireNextImageKHR is equivalent to temporarily importing a semaphore payload to that semaphore.

NOTE

Because the exportable handle types of an imported semaphore correspond to its current imported payload, and vkAcquireNextImageKHR behaves the same as a temporary import operation for which the source semaphore is opaque to the application, applications have no way of determining whether any external handle types **can** be exported from a semaphore in this state. Therefore, applications **must** not attempt to export external handles from semaphores using a temporarily imported payload from vkAcquireNextImageKHR.

When importing a semaphore payload, it is the responsibility of the application to ensure the external handles meet all valid usage requirements. However, implementations **must** perform sufficient validation of external handles to ensure that the operation results in a valid semaphore which will not cause program termination, device loss, queue stalls, or corruption of other resources when used as allowed according to its import parameters, and excepting those side effects allowed for violations of the valid semaphore state for wait operations rules. If the external handle provided does not meet these requirements, the implementation **must** fail the semaphore payload import operation with the error code `VK_ERROR_INVALID_EXTERNAL_HANDLE`.

In addition, when importing a semaphore payload that is not compatible with the payload type corresponding to the VkSemaphoreType the semaphore was created with, the implementation **may**

fail the semaphore payload import operation with the error code VK_ERROR_INVALID_EXTERNAL_HANDLE.

NOTE	As the introduction of the external semaphore handle type VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT predates that of timeline semaphores, support for importing semaphore payloads from external handles of that type into semaphores created (implicitly or explicitly) with a VkSemaphoreType of VK_SEMAPHORE_TYPE_BINARY is preserved for backwards compatibility. However, applications **should** prefer importing such handle types into semaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE.

To import a semaphore payload from a Windows handle, call:

```
// Provided by VK_KHR_external_semaphore_win32
VkResult vkImportSemaphoreWin32HandleKHR(
    VkDevice                                    device,
    const VkImportSemaphoreWin32HandleInfoKHR*  pImportSemaphoreWin32HandleInfo);
```

- device is the logical device that created the semaphore.
- pImportSemaphoreWin32HandleInfo is a pointer to a VkImportSemaphoreWin32HandleInfoKHR structure specifying the semaphore and import parameters.

Importing a semaphore payload from Windows handles does not transfer ownership of the handle to the Vulkan implementation. For handle types defined as NT handles, the application **must** release ownership using the CloseHandle system call when the handle is no longer needed.

Applications **can** import the same semaphore payload into multiple instances of Vulkan, into the same instance from which it was exported, and multiple times into a given Vulkan instance.

Valid Usage (Implicit)

- VUID-vkImportSemaphoreWin32HandleKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkImportSemaphoreWin32HandleKHR-pImportSemaphoreWin32HandleInfo-parameter
  pImportSemaphoreWin32HandleInfo **must** be a valid pointer to a valid VkImportSemaphoreWin32HandleInfoKHR structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_INVALID_EXTERNAL_HANDLE

The VkImportSemaphoreWin32HandleInfoKHR structure is defined as:

```
// Provided by VK_KHR_external_semaphore_win32
typedef struct VkImportSemaphoreWin32HandleInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    VkSemaphore                              semaphore;
    VkSemaphoreImportFlags                   flags;
    VkExternalSemaphoreHandleTypeFlagBits    handleType;
    HANDLE                                   handle;
    LPCWSTR                                  name;
} VkImportSemaphoreWin32HandleInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- semaphore is the semaphore into which the payload will be imported.
- flags is a bitmask of VkSemaphoreImportFlagBits specifying additional parameters for the semaphore payload import operation.
- handleType is a VkExternalSemaphoreHandleTypeFlagBits value specifying the type of handle.
- handle is NULL or the external handle to import.
- name is NULL or a null-terminated UTF-16 string naming the underlying synchronization primitive to import.

The handle types supported by handleType are:

Table 8. Handle Types Supported by VkImportSemaphoreWin32HandleInfoKHR

Handle Type	Transference	Permanence Supported
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT	Reference	Temporary,Permanent
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT	Reference	Temporary,Permanent
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT	Reference	Temporary,Permanent

Valid Usage

- VUID-VkImportSemaphoreWin32HandleInfoKHR-handleType-01140
  handleType **must** be a value included in the Handle Types Supported by VkImportSemaphoreWin32HandleInfoKHR table
- VUID-VkImportSemaphoreWin32HandleInfoKHR-handleType-01466
  If handleType is not VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT or VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT, name **must** be NULL

- VUID-VkImportSemaphoreWin32HandleInfoKHR-handleType-01467
  If `handle` is `NULL`, `name` **must** name a valid synchronization primitive of the type specified by `handleType`
- VUID-VkImportSemaphoreWin32HandleInfoKHR-handleType-01468
  If `name` is `NULL`, `handle` **must** be a valid handle of the type specified by `handleType`
- VUID-VkImportSemaphoreWin32HandleInfoKHR-handle-01469
  If `handle` is not `NULL`, `name` **must** be `NULL`
- VUID-VkImportSemaphoreWin32HandleInfoKHR-handle-01542
  If `handle` is not `NULL`, it **must** obey any requirements listed for `handleType` in external semaphore handle types compatibility
- VUID-VkImportSemaphoreWin32HandleInfoKHR-name-01543
  If `name` is not `NULL`, it **must** obey any requirements listed for `handleType` in external semaphore handle types compatibility
- VUID-VkImportSemaphoreWin32HandleInfoKHR-handleType-03261
  If `handleType` is `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT` or `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT`, the VkSemaphoreCreateInfo ::`flags` field **must** match that of the semaphore from which `handle` or `name` was exported
- VUID-VkImportSemaphoreWin32HandleInfoKHR-handleType-03262
  If `handleType` is `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT` or `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT`, the VkSemaphoreTypeCreateInfo::`semaphoreType` field **must** match that of the semaphore from which `handle` or `name` was exported
- VUID-VkImportSemaphoreWin32HandleInfoKHR-flags-03322
  If `flags` contains `VK_SEMAPHORE_IMPORT_TEMPORARY_BIT`, the VkSemaphoreTypeCreateInfo ::`semaphoreType` field of the semaphore from which `handle` or `name` was exported **must** not be `VK_SEMAPHORE_TYPE_TIMELINE`

Valid Usage (Implicit)

- VUID-VkImportSemaphoreWin32HandleInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_SEMAPHORE_WIN32_HANDLE_INFO_KHR`
- VUID-VkImportSemaphoreWin32HandleInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImportSemaphoreWin32HandleInfoKHR-semaphore-parameter
  `semaphore` **must** be a valid VkSemaphore handle
- VUID-VkImportSemaphoreWin32HandleInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkSemaphoreImportFlagBits values

Host Synchronization

- Host access to `semaphore` **must** be externally synchronized

To import a semaphore payload from a POSIX file descriptor, call:

```
// Provided by VK_KHR_external_semaphore_fd
VkResult vkImportSemaphoreFdKHR(
    VkDevice                                    device,
    const VkImportSemaphoreFdInfoKHR*           pImportSemaphoreFdInfo);
```

- `device` is the logical device that created the semaphore.
- `pImportSemaphoreFdInfo` is a pointer to a VkImportSemaphoreFdInfoKHR structure specifying the semaphore and import parameters.

Importing a semaphore payload from a file descriptor transfers ownership of the file descriptor from the application to the Vulkan implementation. The application **must** not perform any operations on the file descriptor after a successful import.

Applications **can** import the same semaphore payload into multiple instances of Vulkan, into the same instance from which it was exported, and multiple times into a given Vulkan instance.

Valid Usage

- VUID-vkImportSemaphoreFdKHR-semaphore-01142
  `semaphore` **must** not be associated with any queue command that has not yet completed execution on that queue

Valid Usage (Implicit)

- VUID-vkImportSemaphoreFdKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkImportSemaphoreFdKHR-pImportSemaphoreFdInfo-parameter
  `pImportSemaphoreFdInfo` **must** be a valid pointer to a valid VkImportSemaphoreFdInfoKHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_INVALID_EXTERNAL_HANDLE`

The `VkImportSemaphoreFdInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_semaphore_fd
typedef struct VkImportSemaphoreFdInfoKHR {
    VkStructureType                           sType;
    const void*                               pNext;
    VkSemaphore                               semaphore;
    VkSemaphoreImportFlags                    flags;
    VkExternalSemaphoreHandleTypeFlagBits     handleType;
    int                                       fd;
} VkImportSemaphoreFdInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `semaphore` is the semaphore into which the payload will be imported.
- `flags` is a bitmask of VkSemaphoreImportFlagBits specifying additional parameters for the semaphore payload import operation.
- `handleType` is a VkExternalSemaphoreHandleTypeFlagBits value specifying the type of `fd`.
- `fd` is the external handle to import.

The handle types supported by `handleType` are:

Table 9. Handle Types Supported by `VkImportSemaphoreFdInfoKHR`

Handle Type	Transference	Permanence Supported
`VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT`	Reference	Temporary,Permanent
`VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT`	Copy	Temporary

Valid Usage

- VUID-VkImportSemaphoreFdInfoKHR-handleType-01143
  `handleType` **must** be a value included in the Handle Types Supported by `VkImportSemaphoreFdInfoKHR` table
- VUID-VkImportSemaphoreFdInfoKHR-fd-01544
  `fd` **must** obey any requirements listed for `handleType` in external semaphore handle types compatibility
- VUID-VkImportSemaphoreFdInfoKHR-handleType-03263
  If `handleType` is `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT`, the VkSemaphoreCreateInfo::`flags` field **must** match that of the semaphore from which `fd` was exported
- VUID-VkImportSemaphoreFdInfoKHR-handleType-07307
  If `handleType` refers to a handle type with copy payload transference semantics, `flags` **must** contain `VK_SEMAPHORE_IMPORT_TEMPORARY_BIT`
- VUID-VkImportSemaphoreFdInfoKHR-handleType-03264

If `handleType` is `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT`, the VkSemaphoreTypeCreateInfo::`semaphoreType` field **must** match that of the semaphore from which `fd` was exported

- VUID-VkImportSemaphoreFdInfoKHR-flags-03323
  If `flags` contains `VK_SEMAPHORE_IMPORT_TEMPORARY_BIT`, the VkSemaphoreTypeCreateInfo::`semaphoreType` field of the semaphore from which `fd` was exported **must** not be `VK_SEMAPHORE_TYPE_TIMELINE`

If `handleType` is `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT`, the special value `-1` for `fd` is treated like a valid sync file descriptor referring to an object that has already signaled. The import operation will succeed and the `VkSemaphore` will have a temporarily imported payload as if a valid file descriptor had been provided.

NOTE

This special behavior for importing an invalid sync file descriptor allows easier interoperability with other system APIs which use the convention that an invalid sync file descriptor represents work that has already completed and does not need to be waited for. It is consistent with the option for implementations to return a `-1` file descriptor when exporting a `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT` from a `VkSemaphore` which is signaled.

Valid Usage (Implicit)

- VUID-VkImportSemaphoreFdInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_SEMAPHORE_FD_INFO_KHR`
- VUID-VkImportSemaphoreFdInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImportSemaphoreFdInfoKHR-semaphore-parameter
  `semaphore` **must** be a valid VkSemaphore handle
- VUID-VkImportSemaphoreFdInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkSemaphoreImportFlagBits values
- VUID-VkImportSemaphoreFdInfoKHR-handleType-parameter
  `handleType` **must** be a valid VkExternalSemaphoreHandleTypeFlagBits value

Host Synchronization

- Host access to `semaphore` **must** be externally synchronized

Bits which **can** be set in

- VkImportSemaphoreWin32HandleInfoKHR::`flags`
- VkImportSemaphoreFdInfoKHR::`flags`

specifying additional parameters of a semaphore import operation are:

```
typedef enum VkSemaphoreImportFlagBits {
    VK_SEMAPHORE_IMPORT_TEMPORARY_BIT = 0x00000001,
  // Provided by VK_KHR_external_semaphore
    VK_SEMAPHORE_IMPORT_TEMPORARY_BIT_KHR = VK_SEMAPHORE_IMPORT_TEMPORARY_BIT,
} VkSemaphoreImportFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore
typedef VkSemaphoreImportFlagBits VkSemaphoreImportFlagBitsKHR;
```

These bits have the following meanings:

- VK_SEMAPHORE_IMPORT_TEMPORARY_BIT specifies that the semaphore payload will be imported only temporarily, as described in Importing Semaphore Payloads, regardless of the permanence of handleType.

```
typedef VkFlags VkSemaphoreImportFlags;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore
typedef VkSemaphoreImportFlags VkSemaphoreImportFlagsKHR;
```

VkSemaphoreImportFlags is a bitmask type for setting a mask of zero or more VkSemaphoreImportFlagBits.

7.5. Events

Events are a synchronization primitive that **can** be used to insert a fine-grained dependency between commands submitted to the same queue, or between the host and a queue. Events **must** not be used to insert a dependency between commands submitted to different queues. Events have two states - signaled and unsignaled. An application **can** signal or unsignal an event either on the host or on the device. A device **can** be made to wait for an event to become signaled before executing further operations. No command exists to wait for an event to become signaled on the host, but the current state of an event **can** be queried.

Events are represented by VkEvent handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkEvent)
```

To create an event, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateEvent(
    VkDevice                                    device,
    const VkEventCreateInfo*                    pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkEvent*                                    pEvent);
```

- `device` is the logical device that creates the event.
- `pCreateInfo` is a pointer to a VkEventCreateInfo structure containing information about how the event is to be created.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pEvent` is a pointer to a handle in which the resulting event object is returned.

When created, the event object is in the unsignaled state.

Valid Usage

- VUID-vkCreateEvent-device-09672
  `device` **must** support at least one queue family with one of the `VK_QUEUE_VIDEO_ENCODE_BIT_KHR`, `VK_QUEUE_VIDEO_DECODE_BIT_KHR`, `VK_QUEUE_COMPUTE_BIT`, or `VK_QUEUE_GRAPHICS_BIT` capabilities
- VUID-vkCreateEvent-events-04468
  If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`events` is `VK_FALSE`, then the implementation does not support events, and vkCreateEvent **must** not be used

Valid Usage (Implicit)

- VUID-vkCreateEvent-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateEvent-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkEventCreateInfo structure
- VUID-vkCreateEvent-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateEvent-pEvent-parameter
  `pEvent` **must** be a valid pointer to a VkEvent handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkEventCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkEventCreateInfo {
    VkStructureType       sType;
    const void*           pNext;
    VkEventCreateFlags    flags;
} VkEventCreateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkEventCreateFlagBits defining additional creation parameters.

Valid Usage

- VUID-VkEventCreateInfo-pNext-06790
  If the pNext chain includes a VkExportMetalObjectCreateInfoEXT structure, its exportObjectType member **must** be VK_EXPORT_METAL_OBJECT_TYPE_METAL_SHARED_EVENT_BIT_EXT

Valid Usage (Implicit)

- VUID-VkEventCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_EVENT_CREATE_INFO
- VUID-VkEventCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkExportMetalObjectCreateInfoEXT or VkImportMetalSharedEventInfoEXT
- VUID-VkEventCreateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique, with the exception of structures of type VkExportMetalObjectCreateInfoEXT
- VUID-VkEventCreateInfo-flags-parameter
  flags **must** be a valid combination of VkEventCreateFlagBits values

```
// Provided by VK_VERSION_1_0
typedef enum VkEventCreateFlagBits {
    VK_EVENT_CREATE_DEVICE_ONLY_BIT = 0x00000001,
  // Provided by VK_KHR_synchronization2
    VK_EVENT_CREATE_DEVICE_ONLY_BIT_KHR = VK_EVENT_CREATE_DEVICE_ONLY_BIT,
} VkEventCreateFlagBits;
```

- `VK_EVENT_CREATE_DEVICE_ONLY_BIT` specifies that host event commands will not be used with this event.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkEventCreateFlags;
```

`VkEventCreateFlags` is a bitmask type for setting a mask of VkEventCreateFlagBits.

To destroy an event, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyEvent(
    VkDevice                                    device,
    VkEvent                                     event,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the event.
- `event` is the handle of the event to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyEvent-event-01145
  All submitted commands that refer to `event` **must** have completed execution
- VUID-vkDestroyEvent-event-01146
  If `VkAllocationCallbacks` were provided when `event` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyEvent-event-01147
  If no `VkAllocationCallbacks` were provided when `event` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyEvent-device-parameter
  `device` **must** be a valid VkDevice handle

- VUID-vkDestroyEvent-event-parameter
  If `event` is not VK_NULL_HANDLE, `event` **must** be a valid VkEvent handle
- VUID-vkDestroyEvent-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyEvent-event-parent
  If `event` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `event` **must** be externally synchronized

To query the state of an event from the host, call:

```
// Provided by VK_VERSION_1_0
VkResult vkGetEventStatus(
    VkDevice                                    device,
    VkEvent                                     event);
```

- `device` is the logical device that owns the event.
- `event` is the handle of the event to query.

Upon success, `vkGetEventStatus` returns the state of the event object with the following return codes:

Table 10. Event Object Status Codes

Status	Meaning
`VK_EVENT_SET`	The event specified by `event` is signaled.
`VK_EVENT_RESET`	The event specified by `event` is unsignaled.

If a `vkCmdSetEvent` or `vkCmdResetEvent` command is in a command buffer that is in the pending state, then the value returned by this command **may** immediately be out of date.

The state of an event **can** be updated by the host. The state of the event is immediately changed, and subsequent calls to `vkGetEventStatus` will return the new state. If an event is already in the requested state, then updating it to the same state has no effect.

Valid Usage

- VUID-vkGetEventStatus-event-03940
  `event` **must** not have been created with `VK_EVENT_CREATE_DEVICE_ONLY_BIT`

Valid Usage (Implicit)

- VUID-vkGetEventStatus-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetEventStatus-event-parameter
  `event` **must** be a valid VkEvent handle
- VUID-vkGetEventStatus-event-parent
  `event` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_EVENT_SET`
- `VK_EVENT_RESET`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_DEVICE_LOST`

To set the state of an event to signaled from the host, call:

```
// Provided by VK_VERSION_1_0
VkResult vkSetEvent(
    VkDevice                                    device,
    VkEvent                                     event);
```

- `device` is the logical device that owns the event.
- `event` is the event to set.

When vkSetEvent is executed on the host, it defines an *event signal operation* which sets the event to the signaled state.

If `event` is already in the signaled state when vkSetEvent is executed, then vkSetEvent has no effect, and no event signal operation occurs.

NOTE If a command buffer is waiting for an event to be signaled from the host, the application must signal the event before submitting the command buffer, as described in the queue forward progress section.

Valid Usage

- VUID-vkSetEvent-event-03941

`event` **must** not have been created with `VK_EVENT_CREATE_DEVICE_ONLY_BIT`

- VUID-vkSetEvent-event-09543
  `event` **must** not be waited on by a command buffer in the pending state

Valid Usage (Implicit)

- VUID-vkSetEvent-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkSetEvent-event-parameter
  `event` **must** be a valid VkEvent handle
- VUID-vkSetEvent-event-parent
  `event` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `event` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

To set the state of an event to unsignaled from the host, call:

```
// Provided by VK_VERSION_1_0
VkResult vkResetEvent(
    VkDevice                                    device,
    VkEvent                                     event);
```

- `device` is the logical device that owns the event.
- `event` is the event to reset.

When vkResetEvent is executed on the host, it defines an *event unsignal operation* which resets the event to the unsignaled state.

If `event` is already in the unsignaled state when vkResetEvent is executed, then vkResetEvent has no effect, and no event unsignal operation occurs.

Valid Usage

- VUID-vkResetEvent-event-03821
  There **must** be an execution dependency between `vkResetEvent` and the execution of any vkCmdWaitEvents that includes `event` in its `pEvents` parameter
- VUID-vkResetEvent-event-03822
  There **must** be an execution dependency between `vkResetEvent` and the execution of any vkCmdWaitEvents2KHR that includes `event` in its `pEvents` parameter
- VUID-vkResetEvent-event-03823
  `event` **must** not have been created with `VK_EVENT_CREATE_DEVICE_ONLY_BIT`

Valid Usage (Implicit)

- VUID-vkResetEvent-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkResetEvent-event-parameter
  `event` **must** be a valid VkEvent handle
- VUID-vkResetEvent-event-parent
  `event` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `event` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The state of an event **can** also be updated on the device by commands inserted in command buffers.

To signal an event from a device, call:

```
// Provided by VK_KHR_synchronization2
void vkCmdSetEvent2KHR(
    VkCommandBuffer                             commandBuffer,
    VkEvent                                     event,
    const VkDependencyInfo*                     pDependencyInfo);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `event` is the event that will be signaled.
- `pDependencyInfo` is a pointer to a VkDependencyInfo structure defining the first scopes of this operation.

When vkCmdSetEvent2KHR is submitted to a queue, it defines the first half of memory dependencies defined by `pDependencyInfo`, as well as an event signal operation which sets the event to the signaled state. A memory dependency is defined between the event signal operation and commands that occur earlier in submission order.

The first synchronization scope and access scope are defined by the union of all the memory dependencies defined by `pDependencyInfo`, and are applied to all operations that occur earlier in submission order. Queue family ownership transfers and image layout transitions defined by `pDependencyInfo` are also included in the first scopes.

The second synchronization scope includes only the event signal operation, and any queue family ownership transfers and image layout transitions defined by `pDependencyInfo`.

The second access scope includes only queue family ownership transfers and image layout transitions.

Future vkCmdWaitEvents2KHR commands rely on all values of each element in `pDependencyInfo` matching exactly with those used to signal the corresponding event. vkCmdWaitEvents **must** not be used to wait on the result of a signal operation defined by `vkCmdSetEvent2KHR`.

NOTE The extra information provided by vkCmdSetEvent2KHR compared to vkCmdSetEvent allows implementations to more efficiently schedule the operations required to satisfy the requested dependencies. With vkCmdSetEvent, the full dependency information is not known until vkCmdWaitEvents is recorded, forcing implementations to insert the required operations at that point and not before.

If `event` is already in the signaled state when vkCmdSetEvent2KHR is executed on the device, then vkCmdSetEvent2KHR has no effect, no event signal operation occurs, and no dependency is generated.

Valid Usage

- VUID-vkCmdSetEvent2-synchronization2-03824
  The `synchronization2` feature **must** be enabled
- VUID-vkCmdSetEvent2-dependencyFlags-03825

The dependencyFlags member of pDependencyInfo **must** be 0

- VUID-vkCmdSetEvent2-srcStageMask-09391
  The srcStageMask member of any element of the pMemoryBarriers, pBufferMemoryBarriers, or pImageMemoryBarriers members of pDependencyInfo **must** not include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-vkCmdSetEvent2-dstStageMask-09392
  The dstStageMask member of any element of the pMemoryBarriers, pBufferMemoryBarriers, or pImageMemoryBarriers members of pDependencyInfo **must** not include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-vkCmdSetEvent2-commandBuffer-03826
  The current device mask of commandBuffer **must** include exactly one physical device
- VUID-vkCmdSetEvent2-srcStageMask-03827
  The srcStageMask member of any element of the pMemoryBarriers, pBufferMemoryBarriers, or pImageMemoryBarriers members of pDependencyInfo **must** only include pipeline stages valid for the queue family that was used to create the command pool that commandBuffer was allocated from
- VUID-vkCmdSetEvent2-dstStageMask-03828
  The dstStageMask member of any element of the pMemoryBarriers, pBufferMemoryBarriers, or pImageMemoryBarriers members of pDependencyInfo **must** only include pipeline stages valid for the queue family that was used to create the command pool that commandBuffer was allocated from

Valid Usage (Implicit)

- VUID-vkCmdSetEvent2-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetEvent2-event-parameter
  event **must** be a valid VkEvent handle
- VUID-vkCmdSetEvent2-pDependencyInfo-parameter
  pDependencyInfo **must** be a valid pointer to a valid VkDependencyInfo structure
- VUID-vkCmdSetEvent2-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetEvent2-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdSetEvent2-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdSetEvent2-commonparent
  Both of commandBuffer, and event **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Both	Graphics Compute Decode Encode	Synchronization

The `VkDependencyInfo` structure is defined as:

```
typedef struct VkDependencyInfo {
    VkStructureType                  sType;
    const void*                      pNext;
    VkDependencyFlags                dependencyFlags;
    uint32_t                         memoryBarrierCount;
    const VkMemoryBarrier2*          pMemoryBarriers;
    uint32_t                         bufferMemoryBarrierCount;
    const VkBufferMemoryBarrier2*    pBufferMemoryBarriers;
    uint32_t                         imageMemoryBarrierCount;
    const VkImageMemoryBarrier2*     pImageMemoryBarriers;
} VkDependencyInfo;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkDependencyInfo VkDependencyInfoKHR;
```

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `dependencyFlags` is a bitmask of `VkDependencyFlagBits` specifying how execution and memory dependencies are formed.
- `memoryBarrierCount` is the length of the `pMemoryBarriers` array.
- `pMemoryBarriers` is a pointer to an array of `VkMemoryBarrier2` structures defining memory dependencies between any memory accesses.

- `bufferMemoryBarrierCount` is the length of the `pBufferMemoryBarriers` array.
- `pBufferMemoryBarriers` is a pointer to an array of VkBufferMemoryBarrier2 structures defining memory dependencies between buffer ranges.
- `imageMemoryBarrierCount` is the length of the `pImageMemoryBarriers` array.
- `pImageMemoryBarriers` is a pointer to an array of VkImageMemoryBarrier2 structures defining memory dependencies between image subresources.

This structure defines a set of memory dependencies, as well as queue family ownership transfer operations and image layout transitions.

Each member of `pMemoryBarriers`, `pBufferMemoryBarriers`, and `pImageMemoryBarriers` defines a separate memory dependency.

Valid Usage (Implicit)

- VUID-VkDependencyInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEPENDENCY_INFO`
- VUID-VkDependencyInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDependencyInfo-dependencyFlags-parameter
  `dependencyFlags` **must** be a valid combination of VkDependencyFlagBits values
- VUID-VkDependencyInfo-pMemoryBarriers-parameter
  If `memoryBarrierCount` is not `0`, `pMemoryBarriers` **must** be a valid pointer to an array of `memoryBarrierCount` valid VkMemoryBarrier2 structures
- VUID-VkDependencyInfo-pBufferMemoryBarriers-parameter
  If `bufferMemoryBarrierCount` is not `0`, `pBufferMemoryBarriers` **must** be a valid pointer to an array of `bufferMemoryBarrierCount` valid VkBufferMemoryBarrier2 structures
- VUID-VkDependencyInfo-pImageMemoryBarriers-parameter
  If `imageMemoryBarrierCount` is not `0`, `pImageMemoryBarriers` **must** be a valid pointer to an array of `imageMemoryBarrierCount` valid VkImageMemoryBarrier2 structures

To set the state of an event to signaled from a device, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetEvent(
    VkCommandBuffer                             commandBuffer,
    VkEvent                                     event,
    VkPipelineStageFlags                        stageMask);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `event` is the event that will be signaled.
- `stageMask` specifies the source stage mask used to determine the first synchronization scope.

vkCmdSetEvent behaves identically to vkCmdSetEvent2KHR, except that it does not define an access scope, and **must** only be used with vkCmdWaitEvents, not vkCmdWaitEvents2KHR.

Valid Usage

- VUID-vkCmdSetEvent-stageMask-04090
  If the geometryShader feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT
- VUID-vkCmdSetEvent-stageMask-04091
  If the tessellationShader feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-vkCmdSetEvent-stageMask-04093
  If the fragmentDensityMap feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-vkCmdSetEvent-stageMask-04094
  If the transformFeedback feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-vkCmdSetEvent-stageMask-07319
  If the attachmentFragmentShadingRate feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdSetEvent-stageMask-03937
  If the synchronization2 feature is not enabled, stageMask **must** not be 0
- VUID-vkCmdSetEvent-stageMask-07950
  If the rayTracingPipeline feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-vkCmdSetEvent-stageMask-06457
  Any pipeline stage included in stageMask **must** be supported by the capabilities of the queue family specified by the queueFamilyIndex member of the VkCommandPoolCreateInfo structure that was used to create the VkCommandPool that commandBuffer was allocated from, as specified in the table of supported pipeline stages
- VUID-vkCmdSetEvent-stageMask-01149
  stageMask **must** not include VK_PIPELINE_STAGE_HOST_BIT
- VUID-vkCmdSetEvent-commandBuffer-01152
  The current device mask of commandBuffer **must** include exactly one physical device

Valid Usage (Implicit)

- VUID-vkCmdSetEvent-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetEvent-event-parameter
  event **must** be a valid VkEvent handle

- VUID-vkCmdSetEvent-stageMask-parameter
  `stageMask` **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-vkCmdSetEvent-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetEvent-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdSetEvent-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdSetEvent-commonparent
  Both of `commandBuffer`, and `event` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Both	Graphics Compute Decode Encode	Synchronization

To unsignal the event from a device, call:

```
// Provided by VK_KHR_synchronization2
void vkCmdResetEvent2KHR(
    VkCommandBuffer                             commandBuffer,
    VkEvent                                     event,
    VkPipelineStageFlags2                       stageMask);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `event` is the event that will be unsignaled.
- `stageMask` is a VkPipelineStageFlags2 mask of pipeline stages used to determine the first synchronization scope.

When vkCmdResetEvent2KHR is submitted to a queue, it defines an execution dependency on commands that were submitted before it, and defines an event unsignal operation which resets the event to the unsignaled state.

The first synchronization scope includes all commands that occur earlier in submission order. The synchronization scope is limited to operations by stageMask or stages that are logically earlier than stageMask.

The second synchronization scope includes only the event unsignal operation.

If event is already in the unsignaled state when vkCmdResetEvent2KHR is executed on the device, then this command has no effect, no event unsignal operation occurs, and no execution dependency is generated.

Valid Usage

- VUID-vkCmdResetEvent2-stageMask-03929
  If the geometryShader feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT
- VUID-vkCmdResetEvent2-stageMask-03930
  If the tessellationShader feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-vkCmdResetEvent2-stageMask-03932
  If the fragmentDensityMap feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-vkCmdResetEvent2-stageMask-03933
  If the transformFeedback feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-vkCmdResetEvent2-stageMask-07317
  If the attachmentFragmentShadingRate feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdResetEvent2-stageMask-07947
  If the rayTracingPipeline feature is not enabled, stageMask **must** not contain VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-vkCmdResetEvent2-synchronization2-03829
  The synchronization2 feature **must** be enabled
- VUID-vkCmdResetEvent2-stageMask-03830
  stageMask **must** not include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-vkCmdResetEvent2-event-03831
  There **must** be an execution dependency between vkCmdResetEvent2KHR and the execution of any vkCmdWaitEvents that includes event in its pEvents parameter
- VUID-vkCmdResetEvent2-event-03832
  There **must** be an execution dependency between vkCmdResetEvent2KHR and the execution of any vkCmdWaitEvents2KHR that includes event in its pEvents parameter

- VUID-vkCmdResetEvent2-commandBuffer-03833
  `commandBuffer`'s current device mask **must** include exactly one physical device

Valid Usage (Implicit)

- VUID-vkCmdResetEvent2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdResetEvent2-event-parameter
  `event` **must** be a valid VkEvent handle
- VUID-vkCmdResetEvent2-stageMask-parameter
  `stageMask` **must** be a valid combination of VkPipelineStageFlagBits2 values
- VUID-vkCmdResetEvent2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdResetEvent2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdResetEvent2-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdResetEvent2-commonparent
  Both of `commandBuffer`, and `event` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Both	Graphics Compute Decode Encode	Synchronization

To set the state of an event to unsignaled from a device, call:

```
// Provided by VK_VERSION_1_0
void vkCmdResetEvent(
    VkCommandBuffer                             commandBuffer,
    VkEvent                                     event,
    VkPipelineStageFlags                        stageMask);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `event` is the event that will be unsignaled.
- `stageMask` is a bitmask of VkPipelineStageFlagBits specifying the source stage mask used to determine when the `event` is unsignaled.

`vkCmdResetEvent` behaves identically to vkCmdResetEvent2KHR.

Valid Usage

- VUID-vkCmdResetEvent-stageMask-04090
  If the `geometryShader` feature is not enabled, `stageMask` **must** not contain `VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT`
- VUID-vkCmdResetEvent-stageMask-04091
  If the `tessellationShader` feature is not enabled, `stageMask` **must** not contain `VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-vkCmdResetEvent-stageMask-04093
  If the `fragmentDensityMap` feature is not enabled, `stageMask` **must** not contain `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- VUID-vkCmdResetEvent-stageMask-04094
  If the `transformFeedback` feature is not enabled, `stageMask` **must** not contain `VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT`
- VUID-vkCmdResetEvent-stageMask-07319
  If the `attachmentFragmentShadingRate` feature is not enabled, `stageMask` **must** not contain `VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-vkCmdResetEvent-stageMask-03937
  If the `synchronization2` feature is not enabled, `stageMask` **must** not be `0`
- VUID-vkCmdResetEvent-stageMask-07950
  If the `rayTracingPipeline` feature is not enabled, `stageMask` **must** not contain `VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR`
- VUID-vkCmdResetEvent-stageMask-06458
  Any pipeline stage included in `stageMask` **must** be supported by the capabilities of the queue family specified by the `queueFamilyIndex` member of the VkCommandPoolCreateInfo structure that was used to create the `VkCommandPool` that `commandBuffer` was allocated from, as specified in the table of supported pipeline stages
- VUID-vkCmdResetEvent-stageMask-01153
  `stageMask` **must** not include `VK_PIPELINE_STAGE_HOST_BIT`

- VUID-vkCmdResetEvent-event-03834
  There **must** be an execution dependency between `vkCmdResetEvent` and the execution of any vkCmdWaitEvents that includes `event` in its `pEvents` parameter
- VUID-vkCmdResetEvent-event-03835
  There **must** be an execution dependency between `vkCmdResetEvent` and the execution of any vkCmdWaitEvents2KHR that includes `event` in its `pEvents` parameter
- VUID-vkCmdResetEvent-commandBuffer-01157
  `commandBuffer`'s current device mask **must** include exactly one physical device

Valid Usage (Implicit)

- VUID-vkCmdResetEvent-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdResetEvent-event-parameter
  `event` **must** be a valid VkEvent handle
- VUID-vkCmdResetEvent-stageMask-parameter
  `stageMask` **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-vkCmdResetEvent-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdResetEvent-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdResetEvent-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdResetEvent-commonparent
  Both of `commandBuffer`, and `event` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Both	Graphics Compute Decode Encode	Synchronization

To wait for one or more events to enter the signaled state on a device, call:

```
// Provided by VK_KHR_synchronization2
void vkCmdWaitEvents2KHR(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    eventCount,
    const VkEvent*                              pEvents,
    const VkDependencyInfo*                     pDependencyInfos);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `eventCount` is the length of the `pEvents` array.
- `pEvents` is a pointer to an array of `eventCount` events to wait on.
- `pDependencyInfos` is a pointer to an array of `eventCount` VkDependencyInfo structures, defining the second synchronization scope.

When `vkCmdWaitEvents2KHR` is submitted to a queue, it inserts memory dependencies according to the elements of `pDependencyInfos` and each corresponding element of `pEvents`. `vkCmdWaitEvents2KHR` **must** not be used to wait on event signal operations occurring on other queues, or signal operations executed by vkCmdSetEvent.

The first synchronization scope and access scope of each memory dependency defined by any element i of `pDependencyInfos` are applied to operations that occurred earlier in submission order than the last event signal operation on element i of `pEvents`.

Signal operations for an event at index i are only included if:

- The event was signaled by a vkCmdSetEvent2KHR command that occurred earlier in submission order with a `dependencyInfo` parameter exactly equal to the element of `pDependencyInfos` at index i ; or
- The event was created without `VK_EVENT_CREATE_DEVICE_ONLY_BIT`, and the first synchronization scope defined by the element of `pDependencyInfos` at index i only includes host operations (`VK_PIPELINE_STAGE_2_HOST_BIT`).

The second synchronization scope and access scope of each memory dependency defined by any element i of `pDependencyInfos` are applied to operations that occurred later in submission order than `vkCmdWaitEvents2KHR`.

NOTE vkCmdWaitEvents2KHR is used with vkCmdSetEvent2KHR to define a memory dependency between two sets of action commands, roughly in the same way as pipeline barriers, but split into two commands such that work between the two **may** execute unhindered.

NOTE Applications should be careful to avoid race conditions when using events. There is no direct ordering guarantee between `vkCmdSetEvent2KHR` and vkCmdResetEvent2KHR, vkCmdResetEvent, or vkCmdSetEvent. Another execution dependency (e.g. a pipeline barrier or semaphore with `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`) is needed to prevent such a race condition.

Valid Usage

- VUID-vkCmdWaitEvents2-synchronization2-03836
  The `synchronization2` feature **must** be enabled
- VUID-vkCmdWaitEvents2-pEvents-03837
  Members of `pEvents` **must** not have been signaled by vkCmdSetEvent
- VUID-vkCmdWaitEvents2-pEvents-03838
  For any element i of `pEvents`, if that event is signaled by vkCmdSetEvent2KHR, that command's `dependencyInfo` parameter **must** be exactly equal to the ith element of `pDependencyInfos`
- VUID-vkCmdWaitEvents2-pEvents-03839
  For any element i of `pEvents`, if that event is signaled by vkSetEvent, barriers in the ith element of `pDependencyInfos` **must** include only host operations in their first synchronization scope
- VUID-vkCmdWaitEvents2-pEvents-03840
  For any element i of `pEvents`, if barriers in the ith element of `pDependencyInfos` include only host operations, the ith element of `pEvents` **must** be signaled before vkCmdWaitEvents2KHR is executed
- VUID-vkCmdWaitEvents2-pEvents-03841
  For any element i of `pEvents`, if barriers in the ith element of `pDependencyInfos` do not include host operations, the ith element of `pEvents` **must** be signaled by a corresponding vkCmdSetEvent2KHR that occurred earlier in submission order
- VUID-vkCmdWaitEvents2-srcStageMask-03842
  The `srcStageMask` member of any element of the `pMemoryBarriers`, `pBufferMemoryBarriers`, or `pImageMemoryBarriers` members of `pDependencyInfos` **must** only include pipeline stages valid for the queue family that was used to create the command pool that `commandBuffer` was allocated from
- VUID-vkCmdWaitEvents2-dstStageMask-03843
  The `dstStageMask` member of any element of the `pMemoryBarriers`, `pBufferMemoryBarriers`, or `pImageMemoryBarriers` members of `pDependencyInfos` **must** only include pipeline stages valid for the queue family that was used to create the command pool that `commandBuffer` was allocated from

- VUID-vkCmdWaitEvents2-dependencyFlags-03844
  If `vkCmdWaitEvents2KHR` is being called inside a render pass instance, the `srcStageMask` member of any element of the `pMemoryBarriers`, `pBufferMemoryBarriers`, or `pImageMemoryBarriers` members of `pDependencyInfos` **must** not include `VK_PIPELINE_STAGE_2_HOST_BIT`
- VUID-vkCmdWaitEvents2-commandBuffer-03846
  `commandBuffer`'s current device mask **must** include exactly one physical device

Valid Usage (Implicit)

- VUID-vkCmdWaitEvents2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdWaitEvents2-pEvents-parameter
  `pEvents` **must** be a valid pointer to an array of `eventCount` valid VkEvent handles
- VUID-vkCmdWaitEvents2-pDependencyInfos-parameter
  `pDependencyInfos` **must** be a valid pointer to an array of `eventCount` valid VkDependencyInfo structures
- VUID-vkCmdWaitEvents2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdWaitEvents2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdWaitEvents2-eventCount-arraylength
  `eventCount` **must** be greater than `0`
- VUID-vkCmdWaitEvents2-commonparent
  Both of `commandBuffer`, and the elements of `pEvents` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Graphics Compute Decode Encode	Synchronization

To wait for one or more events to enter the signaled state on a device, call:

```
// Provided by VK_VERSION_1_0
void vkCmdWaitEvents(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    eventCount,
    const VkEvent*                              pEvents,
    VkPipelineStageFlags                        srcStageMask,
    VkPipelineStageFlags                        dstStageMask,
    uint32_t                                    memoryBarrierCount,
    const VkMemoryBarrier*                      pMemoryBarriers,
    uint32_t                                    bufferMemoryBarrierCount,
    const VkBufferMemoryBarrier*                pBufferMemoryBarriers,
    uint32_t                                    imageMemoryBarrierCount,
    const VkImageMemoryBarrier*                 pImageMemoryBarriers);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `eventCount` is the length of the `pEvents` array.
- `pEvents` is a pointer to an array of event object handles to wait on.
- `srcStageMask` is a bitmask of VkPipelineStageFlagBits specifying the source stage mask.
- `dstStageMask` is a bitmask of VkPipelineStageFlagBits specifying the destination stage mask.
- `memoryBarrierCount` is the length of the `pMemoryBarriers` array.
- `pMemoryBarriers` is a pointer to an array of VkMemoryBarrier structures.
- `bufferMemoryBarrierCount` is the length of the `pBufferMemoryBarriers` array.
- `pBufferMemoryBarriers` is a pointer to an array of VkBufferMemoryBarrier structures.
- `imageMemoryBarrierCount` is the length of the `pImageMemoryBarriers` array.
- `pImageMemoryBarriers` is a pointer to an array of VkImageMemoryBarrier structures.

`vkCmdWaitEvents` is largely similar to vkCmdWaitEvents2KHR, but **can** only wait on signal operations defined by vkCmdSetEvent. As vkCmdSetEvent does not define any access scopes, `vkCmdWaitEvents` defines the first access scope for each event signal operation in addition to its own access scopes.

NOTE Since vkCmdSetEvent does not have any dependency information beyond a stage

> mask, implementations do not have the same opportunity to perform availability and visibility operations or image layout transitions in advance as they do with vkCmdSetEvent2KHR and vkCmdWaitEvents2KHR.

When vkCmdWaitEvents is submitted to a queue, it defines a memory dependency between prior event signal operations on the same queue or the host, and subsequent commands. vkCmdWaitEvents **must** not be used to wait on event signal operations occurring on other queues.

The first synchronization scope only includes event signal operations that operate on members of pEvents, and the operations that happened-before the event signal operations. Event signal operations performed by vkCmdSetEvent that occur earlier in submission order are included in the first synchronization scope, if the logically latest pipeline stage in their stageMask parameter is logically earlier than or equal to the logically latest pipeline stage in srcStageMask. Event signal operations performed by vkSetEvent are only included in the first synchronization scope if VK_PIPELINE_STAGE_HOST_BIT is included in srcStageMask.

The second synchronization scope includes all commands that occur later in submission order. The second synchronization scope is limited to operations on the pipeline stages determined by the destination stage mask specified by dstStageMask.

The first access scope is limited to accesses in the pipeline stages determined by the source stage mask specified by srcStageMask. Within that, the first access scope only includes the first access scopes defined by elements of the pMemoryBarriers, pBufferMemoryBarriers and pImageMemoryBarriers arrays, which each define a set of memory barriers. If no memory barriers are specified, then the first access scope includes no accesses.

The second access scope is limited to accesses in the pipeline stages determined by the destination stage mask specified by dstStageMask. Within that, the second access scope only includes the second access scopes defined by elements of the pMemoryBarriers, pBufferMemoryBarriers and pImageMemoryBarriers arrays, which each define a set of memory barriers. If no memory barriers are specified, then the second access scope includes no accesses.

Valid Usage

- VUID-vkCmdWaitEvents-srcStageMask-04090
  If the geometryShader feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT
- VUID-vkCmdWaitEvents-srcStageMask-04091
  If the tessellationShader feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-vkCmdWaitEvents-srcStageMask-04093
  If the fragmentDensityMap feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-vkCmdWaitEvents-srcStageMask-04094
  If the transformFeedback feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT

- VUID-vkCmdWaitEvents-srcStageMask-07319
  If the attachmentFragmentShadingRate feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdWaitEvents-srcStageMask-03937
  If the synchronization2 feature is not enabled, srcStageMask **must** not be 0
- VUID-vkCmdWaitEvents-srcStageMask-07950
  If the rayTracingPipeline feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-vkCmdWaitEvents-srcAccessMask-06257
  If the rayQuery feature is not enabled and a memory barrier srcAccessMask includes VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR, srcStageMask **must** not include any of the VK_PIPELINE_STAGE_*_SHADER_BIT stages except VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-vkCmdWaitEvents-dstStageMask-04090
  If the geometryShader feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT
- VUID-vkCmdWaitEvents-dstStageMask-04091
  If the tessellationShader feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-vkCmdWaitEvents-dstStageMask-04093
  If the fragmentDensityMap feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-vkCmdWaitEvents-dstStageMask-04094
  If the transformFeedback feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-vkCmdWaitEvents-dstStageMask-07319
  If the attachmentFragmentShadingRate feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdWaitEvents-dstStageMask-03937
  If the synchronization2 feature is not enabled, dstStageMask **must** not be 0
- VUID-vkCmdWaitEvents-dstStageMask-07950
  If the rayTracingPipeline feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-vkCmdWaitEvents-dstAccessMask-06257
  If the rayQuery feature is not enabled and a memory barrier dstAccessMask includes VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR, dstStageMask **must** not include any of the VK_PIPELINE_STAGE_*_SHADER_BIT stages except VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-vkCmdWaitEvents-srcAccessMask-02815
  The srcAccessMask member of each element of pMemoryBarriers **must** only include access

flags that are supported by one or more of the pipeline stages in srcStageMask, as specified in the table of supported access types

- VUID-vkCmdWaitEvents-dstAccessMask-02816
  The dstAccessMask member of each element of pMemoryBarriers **must** only include access flags that are supported by one or more of the pipeline stages in dstStageMask, as specified in the table of supported access types
- VUID-vkCmdWaitEvents-pBufferMemoryBarriers-02817
  For any element of pBufferMemoryBarriers, if its srcQueueFamilyIndex and dstQueueFamilyIndex members are equal, or if its srcQueueFamilyIndex is the queue family index that was used to create the command pool that commandBuffer was allocated from, then its srcAccessMask member **must** only contain access flags that are supported by one or more of the pipeline stages in srcStageMask, as specified in the table of supported access types
- VUID-vkCmdWaitEvents-pBufferMemoryBarriers-02818
  For any element of pBufferMemoryBarriers, if its srcQueueFamilyIndex and dstQueueFamilyIndex members are equal, or if its dstQueueFamilyIndex is the queue family index that was used to create the command pool that commandBuffer was allocated from, then its dstAccessMask member **must** only contain access flags that are supported by one or more of the pipeline stages in dstStageMask, as specified in the table of supported access types
- VUID-vkCmdWaitEvents-pImageMemoryBarriers-02819
  For any element of pImageMemoryBarriers, if its srcQueueFamilyIndex and dstQueueFamilyIndex members are equal, or if its srcQueueFamilyIndex is the queue family index that was used to create the command pool that commandBuffer was allocated from, then its srcAccessMask member **must** only contain access flags that are supported by one or more of the pipeline stages in srcStageMask, as specified in the table of supported access types
- VUID-vkCmdWaitEvents-pImageMemoryBarriers-02820
  For any element of pImageMemoryBarriers, if its srcQueueFamilyIndex and dstQueueFamilyIndex members are equal, or if its dstQueueFamilyIndex is the queue family index that was used to create the command pool that commandBuffer was allocated from, then its dstAccessMask member **must** only contain access flags that are supported by one or more of the pipeline stages in dstStageMask, as specified in the table of supported access types
- VUID-vkCmdWaitEvents-srcStageMask-06459
  Any pipeline stage included in srcStageMask **must** be supported by the capabilities of the queue family specified by the queueFamilyIndex member of the VkCommandPoolCreateInfo structure that was used to create the VkCommandPool that commandBuffer was allocated from, as specified in the table of supported pipeline stages
- VUID-vkCmdWaitEvents-dstStageMask-06460
  Any pipeline stage included in dstStageMask **must** be supported by the capabilities of the queue family specified by the queueFamilyIndex member of the VkCommandPoolCreateInfo structure that was used to create the VkCommandPool that commandBuffer was allocated from, as specified in the table of supported pipeline stages
- VUID-vkCmdWaitEvents-srcStageMask-01158

srcStageMask **must** be the bitwise OR of the stageMask parameter used in previous calls to vkCmdSetEvent with any of the elements of pEvents and VK_PIPELINE_STAGE_HOST_BIT if any of the elements of pEvents was set using vkSetEvent

- VUID-vkCmdWaitEvents-srcStageMask-07308
  If vkCmdWaitEvents is being called inside a render pass instance, srcStageMask **must** not include VK_PIPELINE_STAGE_HOST_BIT
- VUID-vkCmdWaitEvents-srcQueueFamilyIndex-02803
  The srcQueueFamilyIndex and dstQueueFamilyIndex members of any element of pBufferMemoryBarriers or pImageMemoryBarriers **must** be equal
- VUID-vkCmdWaitEvents-commandBuffer-01167
  commandBuffer's current device mask **must** include exactly one physical device
- VUID-vkCmdWaitEvents-pEvents-03847
  Elements of pEvents **must** not have been signaled by vkCmdSetEvent2KHR

Valid Usage (Implicit)

- VUID-vkCmdWaitEvents-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdWaitEvents-pEvents-parameter
  pEvents **must** be a valid pointer to an array of eventCount valid VkEvent handles
- VUID-vkCmdWaitEvents-srcStageMask-parameter
  srcStageMask **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-vkCmdWaitEvents-dstStageMask-parameter
  dstStageMask **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-vkCmdWaitEvents-pMemoryBarriers-parameter
  If memoryBarrierCount is not 0, pMemoryBarriers **must** be a valid pointer to an array of memoryBarrierCount valid VkMemoryBarrier structures
- VUID-vkCmdWaitEvents-pBufferMemoryBarriers-parameter
  If bufferMemoryBarrierCount is not 0, pBufferMemoryBarriers **must** be a valid pointer to an array of bufferMemoryBarrierCount valid VkBufferMemoryBarrier structures
- VUID-vkCmdWaitEvents-pImageMemoryBarriers-parameter
  If imageMemoryBarrierCount is not 0, pImageMemoryBarriers **must** be a valid pointer to an array of imageMemoryBarrierCount valid VkImageMemoryBarrier structures
- VUID-vkCmdWaitEvents-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdWaitEvents-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdWaitEvents-eventCount-arraylength
  eventCount **must** be greater than 0
- VUID-vkCmdWaitEvents-commonparent

Both of `commandBuffer`, and the elements of `pEvents` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Graphics Compute Decode Encode	Synchronization

7.6. Pipeline Barriers

To record a pipeline barrier, call:

```
// Provided by VK_KHR_synchronization2
void vkCmdPipelineBarrier2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkDependencyInfo*                     pDependencyInfo);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `pDependencyInfo` is a pointer to a VkDependencyInfo structure defining the scopes of this operation.

When vkCmdPipelineBarrier2KHR is submitted to a queue, it defines memory dependencies between commands that were submitted to the same queue before it, and those submitted to the same queue after it.

The first synchronization scope and access scope of each memory dependency defined by `pDependencyInfo` are applied to operations that occurred earlier in submission order.

The second synchronization scope and access scope of each memory dependency defined by `pDependencyInfo` are applied to operations that occurred later in submission order.

If `vkCmdPipelineBarrier2KHR` is recorded within a render pass instance, the synchronization scopes are limited to a subset of operations within the same subpass or render pass instance.

Valid Usage

- VUID-vkCmdPipelineBarrier2-None-07889
  If vkCmdPipelineBarrier2KHR is called within a render pass instance using a VkRenderPass object, the render pass **must** have been created with at least one subpass dependency that expresses a dependency from the current subpass to itself, does not include VK_DEPENDENCY_BY_REGION_BIT if this command does not, does not include VK_DEPENDENCY_VIEW_LOCAL_BIT if this command does not, and has synchronization scopes and access scopes that are all supersets of the scopes defined in this command
- VUID-vkCmdPipelineBarrier2-bufferMemoryBarrierCount-01178
  If vkCmdPipelineBarrier2KHR is called within a render pass instance using a VkRenderPass object, it **must** not include any buffer memory barriers
- VUID-vkCmdPipelineBarrier2-image-04073
  If vkCmdPipelineBarrier2KHR is called within a render pass instance using a VkRenderPass object, the image member of any image memory barrier included in this command **must** be an attachment used in the current subpass both as an input attachment, and as either a color, or depth/stencil attachment
- VUID-vkCmdPipelineBarrier2-oldLayout-01181
  If vkCmdPipelineBarrier2KHR is called within a render pass instance, the oldLayout and newLayout members of any image memory barrier included in this command **must** be equal
- VUID-vkCmdPipelineBarrier2-srcQueueFamilyIndex-01182
  If vkCmdPipelineBarrier2KHR is called within a render pass instance, the srcQueueFamilyIndex and dstQueueFamilyIndex members of any memory barrier included in this command **must** be equal
- VUID-vkCmdPipelineBarrier2-None-07890
  If vkCmdPipelineBarrier2KHR is called within a render pass instance, and the source stage masks of any memory barriers include framebuffer-space stages, destination stage masks of all memory barriers **must** only include framebuffer-space stages
- VUID-vkCmdPipelineBarrier2-dependencyFlags-07891
  If vkCmdPipelineBarrier2KHR is called within a render pass instance, and the source stage masks of any memory barriers include framebuffer-space stages, then dependencyFlags **must** include VK_DEPENDENCY_BY_REGION_BIT
- VUID-vkCmdPipelineBarrier2-None-07892
  If vkCmdPipelineBarrier2KHR is called within a render pass instance, the source and destination stage masks of any memory barriers **must** only include graphics pipeline stages
- VUID-vkCmdPipelineBarrier2-dependencyFlags-01186
  If vkCmdPipelineBarrier2KHR is called outside of a render pass instance, the dependency flags **must** not include VK_DEPENDENCY_VIEW_LOCAL_BIT
- VUID-vkCmdPipelineBarrier2-None-07893
  If vkCmdPipelineBarrier2KHR is called inside a render pass instance, and there is more than one view in the current subpass, dependency flags **must** include VK_DEPENDENCY_VIEW_LOCAL_BIT

- VUID-vkCmdPipelineBarrier2-None-09553
  If none of the shaderTileImageColorReadAccess, shaderTileImageStencilReadAccess, or shaderTileImageDepthReadAccess features are enabled, and the dynamicRenderingLocalRead feature is not enabled, vkCmdPipelineBarrier2KHR **must** not be called within a render pass instance started with vkCmdBeginRenderingKHR
- VUID-vkCmdPipelineBarrier2-None-09554
  If the dynamicRenderingLocalRead feature is not enabled, and vkCmdPipelineBarrier2KHR is called within a render pass instance started with vkCmdBeginRenderingKHR, there **must** be no buffer or image memory barriers specified by this command
- VUID-vkCmdPipelineBarrier2-None-09586
  If the dynamicRenderingLocalRead feature is not enabled, and vkCmdPipelineBarrier2KHR is called within a render pass instance started with vkCmdBeginRenderingKHR, memory barriers specified by this command **must** only include VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT, VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT, or VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT in their access masks
- VUID-vkCmdPipelineBarrier2-image-09555
  If vkCmdPipelineBarrier2KHR is called within a render pass instance started with vkCmdBeginRenderingKHR, and the image member of any image memory barrier is used as an attachment in the current render pass instance, it **must** be in the VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR or VK_IMAGE_LAYOUT_GENERAL layout
- VUID-vkCmdPipelineBarrier2-srcStageMask-09556
  If vkCmdPipelineBarrier2KHR is called within a render pass instance started with vkCmdBeginRenderingKHR, this command **must** only specify framebuffer-space stages in srcStageMask and dstStageMask
- VUID-vkCmdPipelineBarrier2-synchronization2-03848
  The synchronization2 feature **must** be enabled
- VUID-vkCmdPipelineBarrier2-srcStageMask-09673
  The srcStageMask member of any element of the pMemoryBarriers member of pDependencyInfo **must** only include pipeline stages valid for the queue family that was used to create the command pool that commandBuffer was allocated from
- VUID-vkCmdPipelineBarrier2-dstStageMask-09674
  The dstStageMask member of any element of the pMemoryBarriers member of pDependencyInfo **must** only include pipeline stages valid for the queue family that was used to create the command pool that commandBuffer was allocated from
- VUID-vkCmdPipelineBarrier2-srcStageMask-09675
  If a buffer or image memory barrier does not specify an acquire operation, the respective srcStageMask member of the element of the pBufferMemoryBarriers or pImageMemoryBarriers members of pDependencyInfo **must** only include pipeline stages valid for the queue family that was used to create the command pool that commandBuffer was allocated from
- VUID-vkCmdPipelineBarrier2-dstStageMask-09676
  If a buffer or image memory barrier does not specify an release operation, the respective dstStageMask member of the element of the pBufferMemoryBarriers or pImageMemoryBarriers members of pDependencyInfo **must** only include pipeline stages valid for the queue family

that was used to create the command pool that `commandBuffer` was allocated from

Valid Usage (Implicit)

- VUID-vkCmdPipelineBarrier2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPipelineBarrier2-pDependencyInfo-parameter
  `pDependencyInfo` **must** be a valid pointer to a valid VkDependencyInfo structure
- VUID-vkCmdPipelineBarrier2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdPipelineBarrier2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, compute, decode, or encode operations

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Transfer Graphics Compute Decode Encode	Synchronization

To record a pipeline barrier, call:

```
// Provided by VK_VERSION_1_0
void vkCmdPipelineBarrier(
    VkCommandBuffer                             commandBuffer,
    VkPipelineStageFlags                        srcStageMask,
    VkPipelineStageFlags                        dstStageMask,
    VkDependencyFlags                           dependencyFlags,
    uint32_t                                    memoryBarrierCount,
    const VkMemoryBarrier*                      pMemoryBarriers,
    uint32_t                                    bufferMemoryBarrierCount,
    const VkBufferMemoryBarrier*                pBufferMemoryBarriers,
    uint32_t                                    imageMemoryBarrierCount,
    const VkImageMemoryBarrier*                 pImageMemoryBarriers);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `srcStageMask` is a bitmask of VkPipelineStageFlagBits specifying the source stages.
- `dstStageMask` is a bitmask of VkPipelineStageFlagBits specifying the destination stages.
- `dependencyFlags` is a bitmask of VkDependencyFlagBits specifying how execution and memory dependencies are formed.
- `memoryBarrierCount` is the length of the `pMemoryBarriers` array.
- `pMemoryBarriers` is a pointer to an array of VkMemoryBarrier structures.
- `bufferMemoryBarrierCount` is the length of the `pBufferMemoryBarriers` array.
- `pBufferMemoryBarriers` is a pointer to an array of VkBufferMemoryBarrier structures.
- `imageMemoryBarrierCount` is the length of the `pImageMemoryBarriers` array.
- `pImageMemoryBarriers` is a pointer to an array of VkImageMemoryBarrier structures.

`vkCmdPipelineBarrier` operates almost identically to vkCmdPipelineBarrier2KHR, except that the scopes and barriers are defined as direct parameters rather than being defined by a VkDependencyInfo.

When vkCmdPipelineBarrier is submitted to a queue, it defines a memory dependency between commands that were submitted to the same queue before it, and those submitted to the same queue after it.

If vkCmdPipelineBarrier was recorded outside a render pass instance, the first synchronization scope includes all commands that occur earlier in submission order. If vkCmdPipelineBarrier was recorded inside a render pass instance, the first synchronization scope includes only commands that occur earlier in submission order within the same subpass. In either case, the first synchronization scope is limited to operations on the pipeline stages determined by the source stage mask specified by `srcStageMask`.

If vkCmdPipelineBarrier was recorded outside a render pass instance, the second synchronization scope includes all commands that occur later in submission order. If vkCmdPipelineBarrier was recorded inside a render pass instance, the second synchronization scope includes only commands that occur later in submission order within the same subpass. In either case, the second synchronization scope is limited to operations on the pipeline stages determined by the destination

stage mask specified by dstStageMask.

The first access scope is limited to accesses in the pipeline stages determined by the source stage mask specified by srcStageMask. Within that, the first access scope only includes the first access scopes defined by elements of the pMemoryBarriers, pBufferMemoryBarriers and pImageMemoryBarriers arrays, which each define a set of memory barriers. If no memory barriers are specified, then the first access scope includes no accesses.

The second access scope is limited to accesses in the pipeline stages determined by the destination stage mask specified by dstStageMask. Within that, the second access scope only includes the second access scopes defined by elements of the pMemoryBarriers, pBufferMemoryBarriers and pImageMemoryBarriers arrays, which each define a set of memory barriers. If no memory barriers are specified, then the second access scope includes no accesses.

If dependencyFlags includes VK_DEPENDENCY_BY_REGION_BIT, then any dependency between framebuffer-space pipeline stages is framebuffer-local - otherwise it is framebuffer-global.

Valid Usage

- VUID-vkCmdPipelineBarrier-srcStageMask-04090
  If the geometryShader feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT
- VUID-vkCmdPipelineBarrier-srcStageMask-04091
  If the tessellationShader feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-vkCmdPipelineBarrier-srcStageMask-04093
  If the fragmentDensityMap feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-vkCmdPipelineBarrier-srcStageMask-04094
  If the transformFeedback feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-vkCmdPipelineBarrier-srcStageMask-07319
  If the attachmentFragmentShadingRate feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdPipelineBarrier-srcStageMask-03937
  If the synchronization2 feature is not enabled, srcStageMask **must** not be 0
- VUID-vkCmdPipelineBarrier-srcStageMask-07950
  If the rayTracingPipeline feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-vkCmdPipelineBarrier-srcAccessMask-06257
  If the rayQuery feature is not enabled and a memory barrier srcAccessMask includes VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR, srcStageMask **must** not include any of the VK_PIPELINE_STAGE_*_SHADER_BIT stages except VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR

- VUID-vkCmdPipelineBarrier-dstStageMask-04090
  If the `geometryShader` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT`
- VUID-vkCmdPipelineBarrier-dstStageMask-04091
  If the `tessellationShader` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-vkCmdPipelineBarrier-dstStageMask-04093
  If the `fragmentDensityMap` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- VUID-vkCmdPipelineBarrier-dstStageMask-04094
  If the `transformFeedback` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT`
- VUID-vkCmdPipelineBarrier-dstStageMask-07319
  If the `attachmentFragmentShadingRate` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-vkCmdPipelineBarrier-dstStageMask-03937
  If the `synchronization2` feature is not enabled, `dstStageMask` **must** not be `0`
- VUID-vkCmdPipelineBarrier-dstStageMask-07950
  If the `rayTracingPipeline` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR`
- VUID-vkCmdPipelineBarrier-dstAccessMask-06257
  If the `rayQuery` feature is not enabled and a memory barrier `dstAccessMask` includes `VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR`, `dstStageMask` **must** not include any of the `VK_PIPELINE_STAGE_*_SHADER_BIT` stages except `VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR`
- VUID-vkCmdPipelineBarrier-srcAccessMask-02815
  The `srcAccessMask` member of each element of `pMemoryBarriers` **must** only include access flags that are supported by one or more of the pipeline stages in `srcStageMask`, as specified in the table of supported access types
- VUID-vkCmdPipelineBarrier-dstAccessMask-02816
  The `dstAccessMask` member of each element of `pMemoryBarriers` **must** only include access flags that are supported by one or more of the pipeline stages in `dstStageMask`, as specified in the table of supported access types
- VUID-vkCmdPipelineBarrier-pBufferMemoryBarriers-02817
  For any element of `pBufferMemoryBarriers`, if its `srcQueueFamilyIndex` and `dstQueueFamilyIndex` members are equal, or if its `srcQueueFamilyIndex` is the queue family index that was used to create the command pool that `commandBuffer` was allocated from, then its `srcAccessMask` member **must** only contain access flags that are supported by one or more of the pipeline stages in `srcStageMask`, as specified in the table of supported access types
- VUID-vkCmdPipelineBarrier-pBufferMemoryBarriers-02818
  For any element of `pBufferMemoryBarriers`, if its `srcQueueFamilyIndex` and

dstQueueFamilyIndex members are equal, or if its dstQueueFamilyIndex is the queue family index that was used to create the command pool that commandBuffer was allocated from, then its dstAccessMask member **must** only contain access flags that are supported by one or more of the pipeline stages in dstStageMask, as specified in the table of supported access types

- VUID-vkCmdPipelineBarrier-pImageMemoryBarriers-02819
  For any element of pImageMemoryBarriers, if its srcQueueFamilyIndex and dstQueueFamilyIndex members are equal, or if its srcQueueFamilyIndex is the queue family index that was used to create the command pool that commandBuffer was allocated from, then its srcAccessMask member **must** only contain access flags that are supported by one or more of the pipeline stages in srcStageMask, as specified in the table of supported access types
- VUID-vkCmdPipelineBarrier-pImageMemoryBarriers-02820
  For any element of pImageMemoryBarriers, if its srcQueueFamilyIndex and dstQueueFamilyIndex members are equal, or if its dstQueueFamilyIndex is the queue family index that was used to create the command pool that commandBuffer was allocated from, then its dstAccessMask member **must** only contain access flags that are supported by one or more of the pipeline stages in dstStageMask, as specified in the table of supported access types
- VUID-vkCmdPipelineBarrier-None-07889
  If vkCmdPipelineBarrier is called within a render pass instance using a VkRenderPass object, the render pass **must** have been created with at least one subpass dependency that expresses a dependency from the current subpass to itself, does not include VK_DEPENDENCY_BY_REGION_BIT if this command does not, does not include VK_DEPENDENCY_VIEW_LOCAL_BIT if this command does not, and has synchronization scopes and access scopes that are all supersets of the scopes defined in this command
- VUID-vkCmdPipelineBarrier-bufferMemoryBarrierCount-01178
  If vkCmdPipelineBarrier is called within a render pass instance using a VkRenderPass object, it **must** not include any buffer memory barriers
- VUID-vkCmdPipelineBarrier-image-04073
  If vkCmdPipelineBarrier is called within a render pass instance using a VkRenderPass object, the image member of any image memory barrier included in this command **must** be an attachment used in the current subpass both as an input attachment, and as either a color, or depth/stencil attachment
- VUID-vkCmdPipelineBarrier-oldLayout-01181
  If vkCmdPipelineBarrier is called within a render pass instance, the oldLayout and newLayout members of any image memory barrier included in this command **must** be equal
- VUID-vkCmdPipelineBarrier-srcQueueFamilyIndex-01182
  If vkCmdPipelineBarrier is called within a render pass instance, the srcQueueFamilyIndex and dstQueueFamilyIndex members of any memory barrier included in this command **must** be equal
- VUID-vkCmdPipelineBarrier-None-07890
  If vkCmdPipelineBarrier is called within a render pass instance, and the source stage masks

of any memory barriers include framebuffer-space stages, destination stage masks of all memory barriers **must** only include framebuffer-space stages

- VUID-vkCmdPipelineBarrier-dependencyFlags-07891
  If `vkCmdPipelineBarrier` is called within a render pass instance, and the source stage masks of any memory barriers include framebuffer-space stages, then `dependencyFlags` **must** include `VK_DEPENDENCY_BY_REGION_BIT`
- VUID-vkCmdPipelineBarrier-None-07892
  If `vkCmdPipelineBarrier` is called within a render pass instance, the source and destination stage masks of any memory barriers **must** only include graphics pipeline stages
- VUID-vkCmdPipelineBarrier-dependencyFlags-01186
  If `vkCmdPipelineBarrier` is called outside of a render pass instance, the dependency flags **must** not include `VK_DEPENDENCY_VIEW_LOCAL_BIT`
- VUID-vkCmdPipelineBarrier-None-07893
  If `vkCmdPipelineBarrier` is called inside a render pass instance, and there is more than one view in the current subpass, dependency flags **must** include `VK_DEPENDENCY_VIEW_LOCAL_BIT`
- VUID-vkCmdPipelineBarrier-None-09553
  If none of the `shaderTileImageColorReadAccess`, `shaderTileImageStencilReadAccess`, or `shaderTileImageDepthReadAccess` features are enabled, and the `dynamicRenderingLocalRead` feature is not enabled, `vkCmdPipelineBarrier` **must** not be called within a render pass instance started with vkCmdBeginRenderingKHR
- VUID-vkCmdPipelineBarrier-None-09554
  If the `dynamicRenderingLocalRead` feature is not enabled, and `vkCmdPipelineBarrier` is called within a render pass instance started with vkCmdBeginRenderingKHR, there **must** be no buffer or image memory barriers specified by this command
- VUID-vkCmdPipelineBarrier-None-09586
  If the `dynamicRenderingLocalRead` feature is not enabled, and `vkCmdPipelineBarrier` is called within a render pass instance started with vkCmdBeginRenderingKHR, memory barriers specified by this command **must** only include `VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT`, `VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT`, `VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT`, or `VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT` in their access masks
- VUID-vkCmdPipelineBarrier-image-09555
  If `vkCmdPipelineBarrier` is called within a render pass instance started with vkCmdBeginRenderingKHR, and the `image` member of any image memory barrier is used as an attachment in the current render pass instance, it **must** be in the `VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR` or `VK_IMAGE_LAYOUT_GENERAL` layout
- VUID-vkCmdPipelineBarrier-srcStageMask-09556
  If `vkCmdPipelineBarrier` is called within a render pass instance started with vkCmdBeginRenderingKHR, this command **must** only specify framebuffer-space stages in `srcStageMask` and `dstStageMask`
- VUID-vkCmdPipelineBarrier-srcStageMask-06461
  Any pipeline stage included in `srcStageMask` **must** be supported by the capabilities of the queue family specified by the `queueFamilyIndex` member of the VkCommandPoolCreateInfo structure that was used to create the `VkCommandPool` that `commandBuffer` was allocated from, as specified in the table of supported pipeline stages

- VUID-vkCmdPipelineBarrier-dstStageMask-06462
  Any pipeline stage included in `dstStageMask` **must** be supported by the capabilities of the queue family specified by the `queueFamilyIndex` member of the VkCommandPoolCreateInfo structure that was used to create the `VkCommandPool` that `commandBuffer` was allocated from, as specified in the table of supported pipeline stages
- VUID-vkCmdPipelineBarrier-srcStageMask-09633
  If either `srcStageMask` or `dstStageMask` includes `VK_PIPELINE_STAGE_HOST_BIT`, for any element of `pImageMemoryBarriers`, `srcQueueFamilyIndex` and `dstQueueFamilyIndex` **must** be equal
- VUID-vkCmdPipelineBarrier-srcStageMask-09634
  If either `srcStageMask` or `dstStageMask` includes `VK_PIPELINE_STAGE_HOST_BIT`, for any element of `pBufferMemoryBarriers`, `srcQueueFamilyIndex` and `dstQueueFamilyIndex` **must** be equal

Valid Usage (Implicit)

- VUID-vkCmdPipelineBarrier-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPipelineBarrier-srcStageMask-parameter
  `srcStageMask` **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-vkCmdPipelineBarrier-dstStageMask-parameter
  `dstStageMask` **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-vkCmdPipelineBarrier-dependencyFlags-parameter
  `dependencyFlags` **must** be a valid combination of VkDependencyFlagBits values
- VUID-vkCmdPipelineBarrier-pMemoryBarriers-parameter
  If `memoryBarrierCount` is not `0`, `pMemoryBarriers` **must** be a valid pointer to an array of `memoryBarrierCount` valid VkMemoryBarrier structures
- VUID-vkCmdPipelineBarrier-pBufferMemoryBarriers-parameter
  If `bufferMemoryBarrierCount` is not `0`, `pBufferMemoryBarriers` **must** be a valid pointer to an array of `bufferMemoryBarrierCount` valid VkBufferMemoryBarrier structures
- VUID-vkCmdPipelineBarrier-pImageMemoryBarriers-parameter
  If `imageMemoryBarrierCount` is not `0`, `pImageMemoryBarriers` **must** be a valid pointer to an array of `imageMemoryBarrierCount` valid VkImageMemoryBarrier structures
- VUID-vkCmdPipelineBarrier-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdPipelineBarrier-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, compute, decode, or encode operations

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized

- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Transfer Graphics Compute Decode Encode	Synchronization

Bits which **can** be set in vkCmdPipelineBarrier::dependencyFlags, specifying how execution and memory dependencies are formed, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkDependencyFlagBits {
    VK_DEPENDENCY_BY_REGION_BIT = 0x00000001,
    VK_DEPENDENCY_DEVICE_GROUP_BIT = 0x00000004,
    VK_DEPENDENCY_VIEW_LOCAL_BIT = 0x00000002,
  // Provided by VK_EXT_attachment_feedback_loop_layout
    VK_DEPENDENCY_FEEDBACK_LOOP_BIT_EXT = 0x00000008,
  // Provided by VK_KHR_multiview
    VK_DEPENDENCY_VIEW_LOCAL_BIT_KHR = VK_DEPENDENCY_VIEW_LOCAL_BIT,
  // Provided by VK_KHR_device_group
    VK_DEPENDENCY_DEVICE_GROUP_BIT_KHR = VK_DEPENDENCY_DEVICE_GROUP_BIT,
} VkDependencyFlagBits;
```

- VK_DEPENDENCY_BY_REGION_BIT specifies that dependencies will be framebuffer-local.
- VK_DEPENDENCY_VIEW_LOCAL_BIT specifies that dependencies will be view-local.
- VK_DEPENDENCY_DEVICE_GROUP_BIT specifies that dependencies are non-device-local.
- VK_DEPENDENCY_FEEDBACK_LOOP_BIT_EXT specifies that the render pass will write to and read from the same image using the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT layout.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkDependencyFlags;
```

VkDependencyFlags is a bitmask type for setting a mask of zero or more VkDependencyFlagBits.

7.7. Memory Barriers

Memory barriers are used to explicitly control access to buffer and image subresource ranges.

Memory barriers are used to transfer ownership between queue families, change image layouts, and define availability and visibility operations. They explicitly define the access types and buffer and image subresource ranges that are included in the access scopes of a memory dependency that is created by a synchronization command that includes them.

7.7.1. Global Memory Barriers

Global memory barriers apply to memory accesses involving all memory objects that exist at the time of its execution.

The `VkMemoryBarrier2` structure is defined as:

```
typedef struct VkMemoryBarrier2 {
    VkStructureType          sType;
    const void*              pNext;
    VkPipelineStageFlags2    srcStageMask;
    VkAccessFlags2           srcAccessMask;
    VkPipelineStageFlags2    dstStageMask;
    VkAccessFlags2           dstAccessMask;
} VkMemoryBarrier2;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkMemoryBarrier2 VkMemoryBarrier2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcStageMask` is a VkPipelineStageFlags2 mask of pipeline stages to be included in the first synchronization scope.
- `srcAccessMask` is a VkAccessFlags2 mask of access flags to be included in the first access scope.
- `dstStageMask` is a VkPipelineStageFlags2 mask of pipeline stages to be included in the second synchronization scope.
- `dstAccessMask` is a VkAccessFlags2 mask of access flags to be included in the second access scope.

This structure defines a memory dependency affecting all device memory.

The first synchronization scope and access scope described by this structure include only operations and memory accesses specified by `srcStageMask` and `srcAccessMask`.

The second synchronization scope and access scope described by this structure include only operations and memory accesses specified by `dstStageMask` and `dstAccessMask`.

Valid Usage

- VUID-VkMemoryBarrier2-srcStageMask-03929
  If the `geometryShader` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT`
- VUID-VkMemoryBarrier2-srcStageMask-03930
  If the `tessellationShader` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-VkMemoryBarrier2-srcStageMask-03932
  If the `fragmentDensityMap` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- VUID-VkMemoryBarrier2-srcStageMask-03933
  If the `transformFeedback` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT`
- VUID-VkMemoryBarrier2-srcStageMask-07317
  If the `attachmentFragmentShadingRate` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-VkMemoryBarrier2-srcStageMask-07947
  If the `rayTracingPipeline` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR`
- VUID-VkMemoryBarrier2-srcAccessMask-03900
  If `srcAccessMask` includes `VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT`, `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkMemoryBarrier2-srcAccessMask-03901
  If `srcAccessMask` includes `VK_ACCESS_2_INDEX_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT`, `VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkMemoryBarrier2-srcAccessMask-03902
  If `srcAccessMask` includes `VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT`, `VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkMemoryBarrier2-srcAccessMask-03903
  If `srcAccessMask` includes `VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT`, `VK_PIPELINE_STAGE_2_SUBPASS_SHADER_BIT_HUAWEI`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkMemoryBarrier2-srcAccessMask-03904
  If `srcAccessMask` includes `VK_ACCESS_2_UNIFORM_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`, or one of

the VK_PIPELINE_STAGE_*_SHADER_BIT stages

- VUID-VkMemoryBarrier2-srcAccessMask-03905
  If srcAccessMask includes VK_ACCESS_2_SHADER_SAMPLED_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-srcAccessMask-03906
  If srcAccessMask includes VK_ACCESS_2_SHADER_STORAGE_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-srcAccessMask-03907
  If srcAccessMask includes VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-srcAccessMask-07454
  If srcAccessMask includes VK_ACCESS_2_SHADER_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-srcAccessMask-03909
  If srcAccessMask includes VK_ACCESS_2_SHADER_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-srcAccessMask-03910
  If srcAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03911
  If srcAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03912
  If srcAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03913
  If srcAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03914
  If srcAccessMask includes VK_ACCESS_2_TRANSFER_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT,

VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkMemoryBarrier2-srcAccessMask-03915
  If srcAccessMask includes VK_ACCESS_2_TRANSFER_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03916
  If srcAccessMask includes VK_ACCESS_2_HOST_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03917
  If srcAccessMask includes VK_ACCESS_2_HOST_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03919
  If srcAccessMask includes VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03920
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-04747
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03922
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03926
  If srcAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-srcAccessMask-03927
  If srcAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT

stages

- VUID-VkMemoryBarrier2-srcAccessMask-03928
  If `srcAccessMask` includes `VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR`, `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkMemoryBarrier2-srcAccessMask-06256
  If the `rayQuery` feature is not enabled and `srcAccessMask` includes `VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR`, `srcStageMask` **must** not include any of the `VK_PIPELINE_STAGE_*_SHADER_BIT` stages except `VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR`
- VUID-VkMemoryBarrier2-srcAccessMask-07272
  If `srcAccessMask` includes `VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT` or `VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR`
- VUID-VkMemoryBarrier2-srcAccessMask-04858
  If `srcAccessMask` includes `VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR`
- VUID-VkMemoryBarrier2-srcAccessMask-04859
  If `srcAccessMask` includes `VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR`
- VUID-VkMemoryBarrier2-srcAccessMask-04860
  If `srcAccessMask` includes `VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR`
- VUID-VkMemoryBarrier2-srcAccessMask-04861
  If `srcAccessMask` includes `VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR`
- VUID-VkMemoryBarrier2-srcAccessMask-07457
  If `srcAccessMask` includes `VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT`
- VUID-VkMemoryBarrier2-srcAccessMask-07458
  If `srcAccessMask` includes `VK_ACCESS_2_MICROMAP_READ_BIT_EXT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` or `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR`
- VUID-VkMemoryBarrier2-dstStageMask-03929
  If the `geometryShader` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT`
- VUID-VkMemoryBarrier2-dstStageMask-03930
  If the `tessellationShader` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-VkMemoryBarrier2-dstStageMask-03932
  If the `fragmentDensityMap` feature is not enabled, `dstStageMask` **must** not contain

VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT

- VUID-VkMemoryBarrier2-dstStageMask-03933
  If the transformFeedback feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-VkMemoryBarrier2-dstStageMask-07317
  If the attachmentFragmentShadingRate feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkMemoryBarrier2-dstStageMask-07947
  If the rayTracingPipeline feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkMemoryBarrier2-dstAccessMask-03900
  If dstAccessMask includes VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03901
  If dstAccessMask includes VK_ACCESS_2_INDEX_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03902
  If dstAccessMask includes VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03903
  If dstAccessMask includes VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_SUBPASS_SHADER_BIT_HUAWEI, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03904
  If dstAccessMask includes VK_ACCESS_2_UNIFORM_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-dstAccessMask-03905
  If dstAccessMask includes VK_ACCESS_2_SHADER_SAMPLED_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-dstAccessMask-03906
  If dstAccessMask includes VK_ACCESS_2_SHADER_STORAGE_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-dstAccessMask-03907
  If dstAccessMask includes VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT, dstStageMask **must**

include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages

- VUID-VkMemoryBarrier2-dstAccessMask-07454
  If dstAccessMask includes VK_ACCESS_2_SHADER_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-dstAccessMask-03909
  If dstAccessMask includes VK_ACCESS_2_SHADER_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-dstAccessMask-03910
  If dstAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03911
  If dstAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03912
  If dstAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03913
  If dstAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03914
  If dstAccessMask includes VK_ACCESS_2_TRANSFER_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03915
  If dstAccessMask includes VK_ACCESS_2_TRANSFER_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkMemoryBarrier2-dstAccessMask-03916
  If dstAccessMask includes VK_ACCESS_2_HOST_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03917
  If dstAccessMask includes VK_ACCESS_2_HOST_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03919
  If dstAccessMask includes VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03920
  If dstAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-04747
  If dstAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03922
  If dstAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03926
  If dstAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-03927
  If dstAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkMemoryBarrier2-dstAccessMask-03928
  If dstAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkMemoryBarrier2-dstAccessMask-06256
  If the rayQuery feature is not enabled and dstAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, dstStageMask **must** not include any of the VK_PIPELINE_STAGE_*_SHADER_BIT stages except VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkMemoryBarrier2-dstAccessMask-07272

If dstAccessMask includes VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT or VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR

- VUID-VkMemoryBarrier2-dstAccessMask-04858
  If dstAccessMask includes VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- VUID-VkMemoryBarrier2-dstAccessMask-04859
  If dstAccessMask includes VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- VUID-VkMemoryBarrier2-dstAccessMask-04860
  If dstAccessMask includes VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- VUID-VkMemoryBarrier2-dstAccessMask-04861
  If dstAccessMask includes VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- VUID-VkMemoryBarrier2-dstAccessMask-07457
  If dstAccessMask includes VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT
- VUID-VkMemoryBarrier2-dstAccessMask-07458
  If dstAccessMask includes VK_ACCESS_2_MICROMAP_READ_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR

Valid Usage (Implicit)

- VUID-VkMemoryBarrier2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_MEMORY_BARRIER_2
- VUID-VkMemoryBarrier2-srcStageMask-parameter
  srcStageMask **must** be a valid combination of VkPipelineStageFlagBits2 values
- VUID-VkMemoryBarrier2-srcAccessMask-parameter
  srcAccessMask **must** be a valid combination of VkAccessFlagBits2 values
- VUID-VkMemoryBarrier2-dstStageMask-parameter
  dstStageMask **must** be a valid combination of VkPipelineStageFlagBits2 values
- VUID-VkMemoryBarrier2-dstAccessMask-parameter
  dstAccessMask **must** be a valid combination of VkAccessFlagBits2 values

The VkMemoryBarrier structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkMemoryBarrier {
    VkStructureType    sType;
    const void*        pNext;
    VkAccessFlags      srcAccessMask;
    VkAccessFlags      dstAccessMask;
} VkMemoryBarrier;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcAccessMask` is a bitmask of VkAccessFlagBits specifying a source access mask.
- `dstAccessMask` is a bitmask of VkAccessFlagBits specifying a destination access mask.

The first access scope is limited to access types in the source access mask specified by `srcAccessMask`.

The second access scope is limited to access types in the destination access mask specified by `dstAccessMask`.

Valid Usage (Implicit)

- VUID-VkMemoryBarrier-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_BARRIER`
- VUID-VkMemoryBarrier-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkMemoryBarrier-srcAccessMask-parameter
  `srcAccessMask` **must** be a valid combination of VkAccessFlagBits values
- VUID-VkMemoryBarrier-dstAccessMask-parameter
  `dstAccessMask` **must** be a valid combination of VkAccessFlagBits values

7.7.2. Buffer Memory Barriers

Buffer memory barriers only apply to memory accesses involving a specific buffer range. That is, a memory dependency formed from a buffer memory barrier is scoped to access via the specified buffer range. Buffer memory barriers **can** also be used to define a queue family ownership transfer for the specified buffer range.

The `VkBufferMemoryBarrier2` structure is defined as:

```
typedef struct VkBufferMemoryBarrier2 {
    VkStructureType          sType;
    const void*              pNext;
    VkPipelineStageFlags2    srcStageMask;
    VkAccessFlags2           srcAccessMask;
    VkPipelineStageFlags2    dstStageMask;
    VkAccessFlags2           dstAccessMask;
    uint32_t                 srcQueueFamilyIndex;
    uint32_t                 dstQueueFamilyIndex;
    VkBuffer                 buffer;
    VkDeviceSize             offset;
    VkDeviceSize             size;
} VkBufferMemoryBarrier2;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkBufferMemoryBarrier2 VkBufferMemoryBarrier2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcStageMask` is a VkPipelineStageFlags2 mask of pipeline stages to be included in the first synchronization scope.
- `srcAccessMask` is a VkAccessFlags2 mask of access flags to be included in the first access scope.
- `dstStageMask` is a VkPipelineStageFlags2 mask of pipeline stages to be included in the second synchronization scope.
- `dstAccessMask` is a VkAccessFlags2 mask of access flags to be included in the second access scope.
- `srcQueueFamilyIndex` is the source queue family for a queue family ownership transfer.
- `dstQueueFamilyIndex` is the destination queue family for a queue family ownership transfer.
- `buffer` is a handle to the buffer whose backing memory is affected by the barrier.
- `offset` is an offset in bytes into the backing memory for `buffer`; this is relative to the base offset as bound to the buffer (see vkBindBufferMemory).
- `size` is a size in bytes of the affected area of backing memory for `buffer`, or `VK_WHOLE_SIZE` to use the range from `offset` to the end of the buffer.

This structure defines a memory dependency limited to a range of a buffer, and **can** define a queue family ownership transfer operation for that range.

The first synchronization scope and access scope described by this structure include only operations and memory accesses specified by `srcStageMask` and `srcAccessMask`.

The second synchronization scope and access scope described by this structure include only operations and memory accesses specified by `dstStageMask` and `dstAccessMask`.

Both access scopes are limited to only memory accesses to `buffer` in the range defined by `offset` and `size`.

If `buffer` was created with `VK_SHARING_MODE_EXCLUSIVE`, and `srcQueueFamilyIndex` is not equal to `dstQueueFamilyIndex`, this memory barrier defines a queue family ownership transfer operation. When executed on a queue in the family identified by `srcQueueFamilyIndex`, this barrier defines a queue family release operation for the specified buffer range, and the second synchronization scope does not apply to this operation. When executed on a queue in the family identified by `dstQueueFamilyIndex`, this barrier defines a queue family acquire operation for the specified buffer range, and the first synchronization scope does not apply to this operation.

A queue family ownership transfer operation is also defined if the values are not equal, and either is one of the special queue family values reserved for external memory ownership transfers, as described in Queue Family Ownership Transfer. A queue family release operation is defined when `dstQueueFamilyIndex` is one of those values, and a queue family acquire operation is defined when `srcQueueFamilyIndex` is one of those values.

Valid Usage

- VUID-VkBufferMemoryBarrier2-srcStageMask-03929
  If the `geometryShader` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT`
- VUID-VkBufferMemoryBarrier2-srcStageMask-03930
  If the `tessellationShader` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-VkBufferMemoryBarrier2-srcStageMask-03932
  If the `fragmentDensityMap` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- VUID-VkBufferMemoryBarrier2-srcStageMask-03933
  If the `transformFeedback` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT`
- VUID-VkBufferMemoryBarrier2-srcStageMask-07317
  If the `attachmentFragmentShadingRate` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-VkBufferMemoryBarrier2-srcStageMask-07947
  If the `rayTracingPipeline` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR`
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03900
  If `srcAccessMask` includes `VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT`, `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03901
  If `srcAccessMask` includes `VK_ACCESS_2_INDEX_READ_BIT`, `srcStageMask` **must** include

VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkBufferMemoryBarrier2-srcAccessMask-03902
  If srcAccessMask includes VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03903
  If srcAccessMask includes VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_SUBPASS_SHADER_BIT_HUAWEI, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03904
  If srcAccessMask includes VK_ACCESS_2_UNIFORM_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03905
  If srcAccessMask includes VK_ACCESS_2_SHADER_SAMPLED_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03906
  If srcAccessMask includes VK_ACCESS_2_SHADER_STORAGE_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03907
  If srcAccessMask includes VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-srcAccessMask-07454
  If srcAccessMask includes VK_ACCESS_2_SHADER_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03909
  If srcAccessMask includes VK_ACCESS_2_SHADER_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03910
  If srcAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03911
  If srcAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, srcStageMask **must**

include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkBufferMemoryBarrier2-srcAccessMask-03912
  If srcAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03913
  If srcAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03914
  If srcAccessMask includes VK_ACCESS_2_TRANSFER_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03915
  If srcAccessMask includes VK_ACCESS_2_TRANSFER_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03916
  If srcAccessMask includes VK_ACCESS_2_HOST_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03917
  If srcAccessMask includes VK_ACCESS_2_HOST_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03919
  If srcAccessMask includes VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03920
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-04747
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT,

VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkBufferMemoryBarrier2-srcAccessMask-03922
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03926
  If srcAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03927
  If srcAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-srcAccessMask-03928
  If srcAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-srcAccessMask-06256
  If the rayQuery feature is not enabled and srcAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, srcStageMask **must** not include any of the VK_PIPELINE_STAGE_*_SHADER_BIT stages except VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkBufferMemoryBarrier2-srcAccessMask-07272
  If srcAccessMask includes VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT or VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkBufferMemoryBarrier2-srcAccessMask-04858
  If srcAccessMask includes VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- VUID-VkBufferMemoryBarrier2-srcAccessMask-04859
  If srcAccessMask includes VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- VUID-VkBufferMemoryBarrier2-srcAccessMask-04860
  If srcAccessMask includes VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- VUID-VkBufferMemoryBarrier2-srcAccessMask-04861
  If srcAccessMask includes VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- VUID-VkBufferMemoryBarrier2-srcAccessMask-07457

If srcAccessMask includes VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT

- VUID-VkBufferMemoryBarrier2-srcAccessMask-07458
  If srcAccessMask includes VK_ACCESS_2_MICROMAP_READ_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR

- VUID-VkBufferMemoryBarrier2-dstStageMask-03929
  If the geometryShader feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT

- VUID-VkBufferMemoryBarrier2-dstStageMask-03930
  If the tessellationShader feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT

- VUID-VkBufferMemoryBarrier2-dstStageMask-03932
  If the fragmentDensityMap feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT

- VUID-VkBufferMemoryBarrier2-dstStageMask-03933
  If the transformFeedback feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT

- VUID-VkBufferMemoryBarrier2-dstStageMask-07317
  If the attachmentFragmentShadingRate feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR

- VUID-VkBufferMemoryBarrier2-dstStageMask-07947
  If the rayTracingPipeline feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR

- VUID-VkBufferMemoryBarrier2-dstAccessMask-03900
  If dstAccessMask includes VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkBufferMemoryBarrier2-dstAccessMask-03901
  If dstAccessMask includes VK_ACCESS_2_INDEX_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkBufferMemoryBarrier2-dstAccessMask-03902
  If dstAccessMask includes VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkBufferMemoryBarrier2-dstAccessMask-03903
  If dstAccessMask includes VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_SUBPASS_SHADER_BIT_HUAWEI, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or

VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkBufferMemoryBarrier2-dstAccessMask-03904
  If dstAccessMask includes VK_ACCESS_2_UNIFORM_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03905
  If dstAccessMask includes VK_ACCESS_2_SHADER_SAMPLED_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03906
  If dstAccessMask includes VK_ACCESS_2_SHADER_STORAGE_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03907
  If dstAccessMask includes VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-dstAccessMask-07454
  If dstAccessMask includes VK_ACCESS_2_SHADER_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03909
  If dstAccessMask includes VK_ACCESS_2_SHADER_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03910
  If dstAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03911
  If dstAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03912
  If dstAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03913
  If dstAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or

VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkBufferMemoryBarrier2-dstAccessMask-03914
  If dstAccessMask includes VK_ACCESS_2_TRANSFER_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03915
  If dstAccessMask includes VK_ACCESS_2_TRANSFER_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03916
  If dstAccessMask includes VK_ACCESS_2_HOST_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03917
  If dstAccessMask includes VK_ACCESS_2_HOST_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03919
  If dstAccessMask includes VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03920
  If dstAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-04747
  If dstAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03922
  If dstAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03926
  If dstAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-03927

If dstAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages

- VUID-VkBufferMemoryBarrier2-dstAccessMask-03928
  If dstAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-06256
  If the rayQuery feature is not enabled and dstAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, dstStageMask **must** not include any of the VK_PIPELINE_STAGE_*_SHADER_BIT stages except VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkBufferMemoryBarrier2-dstAccessMask-07272
  If dstAccessMask includes VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT or VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkBufferMemoryBarrier2-dstAccessMask-04858
  If dstAccessMask includes VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- VUID-VkBufferMemoryBarrier2-dstAccessMask-04859
  If dstAccessMask includes VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- VUID-VkBufferMemoryBarrier2-dstAccessMask-04860
  If dstAccessMask includes VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- VUID-VkBufferMemoryBarrier2-dstAccessMask-04861
  If dstAccessMask includes VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR, dstStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- VUID-VkBufferMemoryBarrier2-dstAccessMask-07457
  If dstAccessMask includes VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT
- VUID-VkBufferMemoryBarrier2-dstAccessMask-07458
  If dstAccessMask includes VK_ACCESS_2_MICROMAP_READ_BIT_EXT, dstStageMask **must** include VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR
- VUID-VkBufferMemoryBarrier2-offset-01187
  offset **must** be less than the size of buffer
- VUID-VkBufferMemoryBarrier2-size-01188
  If size is not equal to VK_WHOLE_SIZE, size **must** be greater than 0
- VUID-VkBufferMemoryBarrier2-size-01189

If `size` is not equal to `VK_WHOLE_SIZE`, `size` **must** be less than or equal to than the size of `buffer` minus `offset`

- VUID-VkBufferMemoryBarrier2-buffer-01931
  If `buffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkBufferMemoryBarrier2-buffer-09095
  If `buffer` was created with a sharing mode of `VK_SHARING_MODE_EXCLUSIVE`, and `srcQueueFamilyIndex` and `dstQueueFamilyIndex` are not equal, `srcQueueFamilyIndex` **must** be `VK_QUEUE_FAMILY_EXTERNAL`, `VK_QUEUE_FAMILY_FOREIGN_EXT`, or a valid queue family
- VUID-VkBufferMemoryBarrier2-buffer-09096
  If `buffer` was created with a sharing mode of `VK_SHARING_MODE_EXCLUSIVE`, and `srcQueueFamilyIndex` and `dstQueueFamilyIndex` are not equal, `dstQueueFamilyIndex` **must** be `VK_QUEUE_FAMILY_EXTERNAL`, `VK_QUEUE_FAMILY_FOREIGN_EXT`, or a valid queue family
- VUID-VkBufferMemoryBarrier2-srcQueueFamilyIndex-04087
  If `srcQueueFamilyIndex` is not equal to `dstQueueFamilyIndex`, at least one of `srcQueueFamilyIndex` or `dstQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_EXTERNAL` or `VK_QUEUE_FAMILY_FOREIGN_EXT`
- VUID-VkBufferMemoryBarrier2-None-09097
  If the VK_KHR_external_memory extension is not enabled, `srcQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_EXTERNAL`
- VUID-VkBufferMemoryBarrier2-None-09098
  If the VK_KHR_external_memory extension is not enabled, `dstQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_EXTERNAL`
- VUID-VkBufferMemoryBarrier2-srcQueueFamilyIndex-09099
  If the VK_EXT_queue_family_foreign extension is not enabled `srcQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_FOREIGN_EXT`
- VUID-VkBufferMemoryBarrier2-dstQueueFamilyIndex-09100
  If the VK_EXT_queue_family_foreign extension is not enabled `dstQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_FOREIGN_EXT`
- VUID-VkBufferMemoryBarrier2-srcStageMask-03851
  If either `srcStageMask` or `dstStageMask` includes `VK_PIPELINE_STAGE_2_HOST_BIT`, `srcQueueFamilyIndex` and `dstQueueFamilyIndex` **must** be equal

Valid Usage (Implicit)

- VUID-VkBufferMemoryBarrier2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2`
- VUID-VkBufferMemoryBarrier2-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkExternalMemoryAcquireUnmodifiedEXT
- VUID-VkBufferMemoryBarrier2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

- VUID-VkBufferMemoryBarrier2-srcStageMask-parameter
  `srcStageMask` **must** be a valid combination of VkPipelineStageFlagBits2 values
- VUID-VkBufferMemoryBarrier2-srcAccessMask-parameter
  `srcAccessMask` **must** be a valid combination of VkAccessFlagBits2 values
- VUID-VkBufferMemoryBarrier2-dstStageMask-parameter
  `dstStageMask` **must** be a valid combination of VkPipelineStageFlagBits2 values
- VUID-VkBufferMemoryBarrier2-dstAccessMask-parameter
  `dstAccessMask` **must** be a valid combination of VkAccessFlagBits2 values
- VUID-VkBufferMemoryBarrier2-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle

The `VkBufferMemoryBarrier` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkBufferMemoryBarrier {
    VkStructureType    sType;
    const void*        pNext;
    VkAccessFlags      srcAccessMask;
    VkAccessFlags      dstAccessMask;
    uint32_t           srcQueueFamilyIndex;
    uint32_t           dstQueueFamilyIndex;
    VkBuffer           buffer;
    VkDeviceSize       offset;
    VkDeviceSize       size;
} VkBufferMemoryBarrier;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcAccessMask` is a bitmask of VkAccessFlagBits specifying a source access mask.
- `dstAccessMask` is a bitmask of VkAccessFlagBits specifying a destination access mask.
- `srcQueueFamilyIndex` is the source queue family for a queue family ownership transfer.
- `dstQueueFamilyIndex` is the destination queue family for a queue family ownership transfer.
- `buffer` is a handle to the buffer whose backing memory is affected by the barrier.
- `offset` is an offset in bytes into the backing memory for `buffer`; this is relative to the base offset as bound to the buffer (see vkBindBufferMemory).
- `size` is a size in bytes of the affected area of backing memory for `buffer`, or `VK_WHOLE_SIZE` to use the range from `offset` to the end of the buffer.

The first access scope is limited to access to memory through the specified buffer range, via access types in the source access mask specified by `srcAccessMask`. If `srcAccessMask` includes `VK_ACCESS_HOST_WRITE_BIT`, a memory domain operation is performed where available memory in the host domain is also made available to the device domain.

The second access scope is limited to access to memory through the specified buffer range, via access types in the destination access mask specified by `dstAccessMask`. If `dstAccessMask` includes `VK_ACCESS_HOST_WRITE_BIT` or `VK_ACCESS_HOST_READ_BIT`, a memory domain operation is performed where available memory in the device domain is also made available to the host domain.

NOTE	When `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT` is used, available memory in host domain is automatically made visible to host domain, and any host write is automatically made available to host domain.

If `srcQueueFamilyIndex` is not equal to `dstQueueFamilyIndex`, and `srcQueueFamilyIndex` is equal to the current queue family, then the memory barrier defines a queue family release operation for the specified buffer range, and the second synchronization scope of the calling command does not apply to this operation.

If `dstQueueFamilyIndex` is not equal to `srcQueueFamilyIndex`, and `dstQueueFamilyIndex` is equal to the current queue family, then the memory barrier defines a queue family acquire operation for the specified buffer range, and the first synchronization scope of the calling command does not apply to this operation.

Valid Usage

- VUID-VkBufferMemoryBarrier-offset-01187
  `offset` **must** be less than the size of `buffer`
- VUID-VkBufferMemoryBarrier-size-01188
  If `size` is not equal to `VK_WHOLE_SIZE`, `size` **must** be greater than `0`
- VUID-VkBufferMemoryBarrier-size-01189
  If `size` is not equal to `VK_WHOLE_SIZE`, `size` **must** be less than or equal to than the size of `buffer` minus `offset`
- VUID-VkBufferMemoryBarrier-buffer-01931
  If `buffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkBufferMemoryBarrier-buffer-09095
  If `buffer` was created with a sharing mode of `VK_SHARING_MODE_EXCLUSIVE`, and `srcQueueFamilyIndex` and `dstQueueFamilyIndex` are not equal, `srcQueueFamilyIndex` **must** be `VK_QUEUE_FAMILY_EXTERNAL`, `VK_QUEUE_FAMILY_FOREIGN_EXT`, or a valid queue family
- VUID-VkBufferMemoryBarrier-buffer-09096
  If `buffer` was created with a sharing mode of `VK_SHARING_MODE_EXCLUSIVE`, and `srcQueueFamilyIndex` and `dstQueueFamilyIndex` are not equal, `dstQueueFamilyIndex` **must** be `VK_QUEUE_FAMILY_EXTERNAL`, `VK_QUEUE_FAMILY_FOREIGN_EXT`, or a valid queue family
- VUID-VkBufferMemoryBarrier-srcQueueFamilyIndex-04087
  If `srcQueueFamilyIndex` is not equal to `dstQueueFamilyIndex`, at least one of `srcQueueFamilyIndex` or `dstQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_EXTERNAL` or `VK_QUEUE_FAMILY_FOREIGN_EXT`
- VUID-VkBufferMemoryBarrier-None-09097
  If the VK_KHR_external_memory extension is not enabled, `srcQueueFamilyIndex` **must** not

be VK_QUEUE_FAMILY_EXTERNAL

- VUID-VkBufferMemoryBarrier-None-09098
  If the VK_KHR_external_memory extension is not enabled, dstQueueFamilyIndex **must** not be VK_QUEUE_FAMILY_EXTERNAL
- VUID-VkBufferMemoryBarrier-srcQueueFamilyIndex-09099
  If the VK_EXT_queue_family_foreign extension is not enabled srcQueueFamilyIndex **must** not be VK_QUEUE_FAMILY_FOREIGN_EXT
- VUID-VkBufferMemoryBarrier-dstQueueFamilyIndex-09100
  If the VK_EXT_queue_family_foreign extension is not enabled dstQueueFamilyIndex **must** not be VK_QUEUE_FAMILY_FOREIGN_EXT
- VUID-VkBufferMemoryBarrier-None-09049
  If the synchronization2 feature is not enabled, and buffer was created with a sharing mode of VK_SHARING_MODE_CONCURRENT, at least one of srcQueueFamilyIndex and dstQueueFamilyIndex **must** be VK_QUEUE_FAMILY_IGNORED
- VUID-VkBufferMemoryBarrier-None-09050
  If the synchronization2 feature is not enabled, and buffer was created with a sharing mode of VK_SHARING_MODE_CONCURRENT, srcQueueFamilyIndex **must** be VK_QUEUE_FAMILY_IGNORED or VK_QUEUE_FAMILY_EXTERNAL
- VUID-VkBufferMemoryBarrier-None-09051
  If the synchronization2 feature is not enabled, and buffer was created with a sharing mode of VK_SHARING_MODE_CONCURRENT, dstQueueFamilyIndex **must** be VK_QUEUE_FAMILY_IGNORED or VK_QUEUE_FAMILY_EXTERNAL

Valid Usage (Implicit)

- VUID-VkBufferMemoryBarrier-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER
- VUID-VkBufferMemoryBarrier-pNext-pNext
  pNext **must** be NULL or a pointer to a valid instance of VkExternalMemoryAcquireUnmodifiedEXT
- VUID-VkBufferMemoryBarrier-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkBufferMemoryBarrier-buffer-parameter
  buffer **must** be a valid VkBuffer handle

VK_WHOLE_SIZE is a special value indicating that the entire remaining length of a buffer following a given offset should be used. It **can** be specified for VkBufferMemoryBarrier::size and other structures.

```
#define VK_WHOLE_SIZE                   (~0ULL)
```

7.7.3. Image Memory Barriers

Image memory barriers only apply to memory accesses involving a specific image subresource range. That is, a memory dependency formed from an image memory barrier is scoped to access via the specified image subresource range. Image memory barriers **can** also be used to define image layout transitions or a queue family ownership transfer for the specified image subresource range.

The `VkImageMemoryBarrier2` structure is defined as:

```
typedef struct VkImageMemoryBarrier2 {
    VkStructureType            sType;
    const void*                pNext;
    VkPipelineStageFlags2      srcStageMask;
    VkAccessFlags2             srcAccessMask;
    VkPipelineStageFlags2      dstStageMask;
    VkAccessFlags2             dstAccessMask;
    VkImageLayout              oldLayout;
    VkImageLayout              newLayout;
    uint32_t                   srcQueueFamilyIndex;
    uint32_t                   dstQueueFamilyIndex;
    VkImage                    image;
    VkImageSubresourceRange    subresourceRange;
} VkImageMemoryBarrier2;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkImageMemoryBarrier2 VkImageMemoryBarrier2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcStageMask` is a VkPipelineStageFlags2 mask of pipeline stages to be included in the first synchronization scope.
- `srcAccessMask` is a VkAccessFlags2 mask of access flags to be included in the first access scope.
- `dstStageMask` is a VkPipelineStageFlags2 mask of pipeline stages to be included in the second synchronization scope.
- `dstAccessMask` is a VkAccessFlags2 mask of access flags to be included in the second access scope.
- `oldLayout` is the old layout in an image layout transition.
- `newLayout` is the new layout in an image layout transition.
- `srcQueueFamilyIndex` is the source queue family for a queue family ownership transfer.
- `dstQueueFamilyIndex` is the destination queue family for a queue family ownership transfer.
- `image` is a handle to the image affected by this barrier.

- `subresourceRange` describes the image subresource range within `image` that is affected by this barrier.

This structure defines a memory dependency limited to an image subresource range, and **can** define a queue family ownership transfer operation and image layout transition for that subresource range.

The first synchronization scope and access scope described by this structure include only operations and memory accesses specified by `srcStageMask` and `srcAccessMask`.

The second synchronization scope and access scope described by this structure include only operations and memory accesses specified by `dstStageMask` and `dstAccessMask`.

Both access scopes are limited to only memory accesses to `image` in the subresource range defined by `subresourceRange`.

If `image` was created with `VK_SHARING_MODE_EXCLUSIVE`, and `srcQueueFamilyIndex` is not equal to `dstQueueFamilyIndex`, this memory barrier defines a queue family ownership transfer operation. When executed on a queue in the family identified by `srcQueueFamilyIndex`, this barrier defines a queue family release operation for the specified image subresource range, and the second synchronization scope does not apply to this operation. When executed on a queue in the family identified by `dstQueueFamilyIndex`, this barrier defines a queue family acquire operation for the specified image subresource range, and the first synchronization, the first synchronization scope does not apply to this operation.

A queue family ownership transfer operation is also defined if the values are not equal, and either is one of the special queue family values reserved for external memory ownership transfers, as described in Queue Family Ownership Transfer. A queue family release operation is defined when `dstQueueFamilyIndex` is one of those values, and a queue family acquire operation is defined when `srcQueueFamilyIndex` is one of those values.

If `oldLayout` is not equal to `newLayout`, then the memory barrier defines an image layout transition for the specified image subresource range. If this memory barrier defines a queue family ownership transfer operation, the layout transition is only executed once between the queues.

NOTE	When the old and new layout are equal, the layout values are ignored - data is preserved no matter what values are specified, or what layout the image is currently in.

If `image` has a multi-planar format and the image is *disjoint*, then including `VK_IMAGE_ASPECT_COLOR_BIT` in the `aspectMask` member of `subresourceRange` is equivalent to including `VK_IMAGE_ASPECT_PLANE_0_BIT`, `VK_IMAGE_ASPECT_PLANE_1_BIT`, and (for three-plane formats only) `VK_IMAGE_ASPECT_PLANE_2_BIT`.

Valid Usage

- VUID-VkImageMemoryBarrier2-srcStageMask-03929
  If the `geometryShader` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT`

- VUID-VkImageMemoryBarrier2-srcStageMask-03930
  If the tessellationShader feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-VkImageMemoryBarrier2-srcStageMask-03932
  If the fragmentDensityMap feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-VkImageMemoryBarrier2-srcStageMask-03933
  If the transformFeedback feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-VkImageMemoryBarrier2-srcStageMask-07317
  If the attachmentFragmentShadingRate feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcStageMask-07947
  If the rayTracingPipeline feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcAccessMask-03900
  If srcAccessMask includes VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03901
  If srcAccessMask includes VK_ACCESS_2_INDEX_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03902
  If srcAccessMask includes VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03903
  If srcAccessMask includes VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_SUBPASS_SHADER_BIT_HUAWEI, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03904
  If srcAccessMask includes VK_ACCESS_2_UNIFORM_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkImageMemoryBarrier2-srcAccessMask-03905
  If srcAccessMask includes VK_ACCESS_2_SHADER_SAMPLED_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages

- VUID-VkImageMemoryBarrier2-srcAccessMask-03906
  If `srcAccessMask` includes `VK_ACCESS_2_SHADER_STORAGE_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`, or one of the `VK_PIPELINE_STAGE_*_SHADER_BIT` stages
- VUID-VkImageMemoryBarrier2-srcAccessMask-03907
  If `srcAccessMask` includes `VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`, or one of the `VK_PIPELINE_STAGE_*_SHADER_BIT` stages
- VUID-VkImageMemoryBarrier2-srcAccessMask-07454
  If `srcAccessMask` includes `VK_ACCESS_2_SHADER_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`, `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR`, `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT`, or one of the `VK_PIPELINE_STAGE_*_SHADER_BIT` stages
- VUID-VkImageMemoryBarrier2-srcAccessMask-03909
  If `srcAccessMask` includes `VK_ACCESS_2_SHADER_WRITE_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`, or one of the `VK_PIPELINE_STAGE_*_SHADER_BIT` stages
- VUID-VkImageMemoryBarrier2-srcAccessMask-03910
  If `srcAccessMask` includes `VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT` `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-srcAccessMask-03911
  If `srcAccessMask` includes `VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT` `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-srcAccessMask-03912
  If `srcAccessMask` includes `VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT`, `VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-srcAccessMask-03913
  If `srcAccessMask` includes `VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT`, `VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-srcAccessMask-03914
  If `srcAccessMask` includes `VK_ACCESS_2_TRANSFER_READ_BIT`, `srcStageMask` **must** include `VK_PIPELINE_STAGE_2_COPY_BIT`, `VK_PIPELINE_STAGE_2_BLIT_BIT`, `VK_PIPELINE_STAGE_2_RESOLVE_BIT`, `VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT`, `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR`, `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-srcAccessMask-03915

If srcAccessMask includes VK_ACCESS_2_TRANSFER_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkImageMemoryBarrier2-srcAccessMask-03916
  If srcAccessMask includes VK_ACCESS_2_HOST_READ_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03917
  If srcAccessMask includes VK_ACCESS_2_HOST_WRITE_BIT, srcStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03919
  If srcAccessMask includes VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03920
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-04747
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03922
  If srcAccessMask includes VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03926
  If srcAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-srcAccessMask-03927
  If srcAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkImageMemoryBarrier2-srcAccessMask-03928
  If srcAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR or

VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT

- VUID-VkImageMemoryBarrier2-srcAccessMask-06256
  If the rayQuery feature is not enabled and srcAccessMask includes VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR, srcStageMask **must** not include any of the VK_PIPELINE_STAGE_*_SHADER_BIT stages except VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcAccessMask-07272
  If srcAccessMask includes VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT or VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcAccessMask-04858
  If srcAccessMask includes VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcAccessMask-04859
  If srcAccessMask includes VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcAccessMask-04860
  If srcAccessMask includes VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcAccessMask-04861
  If srcAccessMask includes VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR, srcStageMask **must** include VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcAccessMask-07457
  If srcAccessMask includes VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT
- VUID-VkImageMemoryBarrier2-srcAccessMask-07458
  If srcAccessMask includes VK_ACCESS_2_MICROMAP_READ_BIT_EXT, srcStageMask **must** include VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR
- VUID-VkImageMemoryBarrier2-dstStageMask-03929
  If the geometryShader feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT
- VUID-VkImageMemoryBarrier2-dstStageMask-03930
  If the tessellationShader feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-VkImageMemoryBarrier2-dstStageMask-03932
  If the fragmentDensityMap feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-VkImageMemoryBarrier2-dstStageMask-03933
  If the transformFeedback feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT

- VUID-VkImageMemoryBarrier2-dstStageMask-07317
  If the attachmentFragmentShadingRate feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkImageMemoryBarrier2-dstStageMask-07947
  If the rayTracingPipeline feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkImageMemoryBarrier2-dstAccessMask-03900
  If dstAccessMask includes VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03901
  If dstAccessMask includes VK_ACCESS_2_INDEX_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03902
  If dstAccessMask includes VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT, VK_PIPELINE_STAGE_2_VERTEX_INPUT_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03903
  If dstAccessMask includes VK_ACCESS_2_INPUT_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT, VK_PIPELINE_STAGE_2_SUBPASS_SHADER_BIT_HUAWEI, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03904
  If dstAccessMask includes VK_ACCESS_2_UNIFORM_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkImageMemoryBarrier2-dstAccessMask-03905
  If dstAccessMask includes VK_ACCESS_2_SHADER_SAMPLED_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkImageMemoryBarrier2-dstAccessMask-03906
  If dstAccessMask includes VK_ACCESS_2_SHADER_STORAGE_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkImageMemoryBarrier2-dstAccessMask-03907
  If dstAccessMask includes VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkImageMemoryBarrier2-dstAccessMask-07454
  If dstAccessMask includes VK_ACCESS_2_SHADER_READ_BIT, dstStageMask **must** include

VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages

- VUID-VkImageMemoryBarrier2-dstAccessMask-03909
  If dstAccessMask includes VK_ACCESS_2_SHADER_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, or one of the VK_PIPELINE_STAGE_*_SHADER_BIT stages
- VUID-VkImageMemoryBarrier2-dstAccessMask-03910
  If dstAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03911
  If dstAccessMask includes VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03912
  If dstAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03913
  If dstAccessMask includes VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT, VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03914
  If dstAccessMask includes VK_ACCESS_2_TRANSFER_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, or VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03915
  If dstAccessMask includes VK_ACCESS_2_TRANSFER_WRITE_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_COPY_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT, VK_PIPELINE_STAGE_2_RESOLVE_BIT, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_PIPELINE_STAGE_2_ALL_TRANSFER_BIT, VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR, or VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT
- VUID-VkImageMemoryBarrier2-dstAccessMask-03916
  If dstAccessMask includes VK_ACCESS_2_HOST_READ_BIT, dstStageMask **must** include VK_PIPELINE_STAGE_2_HOST_BIT

- VUID-VkImageMemoryBarrier2-dstAccessMask-03917
  If `dstAccessMask` includes `VK_ACCESS_2_HOST_WRITE_BIT`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_HOST_BIT`
- VUID-VkImageMemoryBarrier2-dstAccessMask-03919
  If `dstAccessMask` includes `VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-dstAccessMask-03920
  If `dstAccessMask` includes `VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-dstAccessMask-04747
  If `dstAccessMask` includes `VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT`, `VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-dstAccessMask-03922
  If `dstAccessMask` includes `VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT`, `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-dstAccessMask-03926
  If `dstAccessMask` includes `VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT` `VK_PIPELINE_STAGE_2_ALL_GRAPHICS_BIT`, or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-dstAccessMask-03927
  If `dstAccessMask` includes `VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR`, `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR`, `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`, or one of the `VK_PIPELINE_STAGE_*_SHADER_BIT` stages
- VUID-VkImageMemoryBarrier2-dstAccessMask-03928
  If `dstAccessMask` includes `VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR`, `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` or `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`
- VUID-VkImageMemoryBarrier2-dstAccessMask-06256
  If the `rayQuery` feature is not enabled and `dstAccessMask` includes `VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR`, `dstStageMask` **must** not include any of the `VK_PIPELINE_STAGE_*_SHADER_BIT` stages except `VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR`
- VUID-VkImageMemoryBarrier2-dstAccessMask-07272
  If `dstAccessMask` includes `VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT` or `VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR`

- VUID-VkImageMemoryBarrier2-dstAccessMask-04858
  If `dstAccessMask` includes `VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR`
- VUID-VkImageMemoryBarrier2-dstAccessMask-04859
  If `dstAccessMask` includes `VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR`
- VUID-VkImageMemoryBarrier2-dstAccessMask-04860
  If `dstAccessMask` includes `VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR`
- VUID-VkImageMemoryBarrier2-dstAccessMask-04861
  If `dstAccessMask` includes `VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR`
- VUID-VkImageMemoryBarrier2-dstAccessMask-07457
  If `dstAccessMask` includes `VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT`
- VUID-VkImageMemoryBarrier2-dstAccessMask-07458
  If `dstAccessMask` includes `VK_ACCESS_2_MICROMAP_READ_BIT_EXT`, `dstStageMask` **must** include `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` or `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR`
- VUID-VkImageMemoryBarrier2-oldLayout-01208
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` then `image` **must** have been created with `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`
- VUID-VkImageMemoryBarrier2-oldLayout-01209
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL` then `image` **must** have been created with `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-VkImageMemoryBarrier2-oldLayout-01210
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL` then `image` **must** have been created with `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-VkImageMemoryBarrier2-oldLayout-01211
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` then `image` **must** have been created with `VK_IMAGE_USAGE_SAMPLED_BIT` or `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT`
- VUID-VkImageMemoryBarrier2-oldLayout-01212
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL` then `image` **must** have been created with

VK_IMAGE_USAGE_TRANSFER_SRC_BIT

- VUID-VkImageMemoryBarrier2-oldLayout-01213
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_TRANSFER_DST_BIT
- VUID-VkImageMemoryBarrier2-oldLayout-01197
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, oldLayout **must** be VK_IMAGE_LAYOUT_UNDEFINED or the current layout of the image subresources affected by the barrier
- VUID-VkImageMemoryBarrier2-newLayout-01198
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, newLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED
- VUID-VkImageMemoryBarrier2-oldLayout-01658
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier2-oldLayout-01659
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier2-synchronization2-07793
  If the synchronization2 feature is not enabled, oldLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR or VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkImageMemoryBarrier2-synchronization2-07794
  If the synchronization2 feature is not enabled, newLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR or VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-03938
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL, image **must** have been created with VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-03939
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL, image **must** have been created with at least one of VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT, VK_IMAGE_USAGE_SAMPLED_BIT, or VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier2-oldLayout-02088
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer

or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR then image **must** have been created with VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR set

- VUID-VkImageMemoryBarrier2-image-09117
  If image was created with a sharing mode of VK_SHARING_MODE_EXCLUSIVE, and srcQueueFamilyIndex and dstQueueFamilyIndex are not equal, srcQueueFamilyIndex **must** be VK_QUEUE_FAMILY_EXTERNAL, VK_QUEUE_FAMILY_FOREIGN_EXT, or a valid queue family
- VUID-VkImageMemoryBarrier2-image-09118
  If image was created with a sharing mode of VK_SHARING_MODE_EXCLUSIVE, and srcQueueFamilyIndex and dstQueueFamilyIndex are not equal, dstQueueFamilyIndex **must** be VK_QUEUE_FAMILY_EXTERNAL, VK_QUEUE_FAMILY_FOREIGN_EXT, or a valid queue family
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-04070
  If srcQueueFamilyIndex is not equal to dstQueueFamilyIndex, at least one of srcQueueFamilyIndex or dstQueueFamilyIndex **must** not be VK_QUEUE_FAMILY_EXTERNAL or VK_QUEUE_FAMILY_FOREIGN_EXT
- VUID-VkImageMemoryBarrier2-None-09119
  If the VK_KHR_external_memory extension is not enabled, srcQueueFamilyIndex **must** not be VK_QUEUE_FAMILY_EXTERNAL
- VUID-VkImageMemoryBarrier2-None-09120
  If the VK_KHR_external_memory extension is not enabled, dstQueueFamilyIndex **must** not be VK_QUEUE_FAMILY_EXTERNAL
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-09121
  If the VK_EXT_queue_family_foreign extension is not enabled srcQueueFamilyIndex **must** not be VK_QUEUE_FAMILY_FOREIGN_EXT
- VUID-VkImageMemoryBarrier2-dstQueueFamilyIndex-09122
  If the VK_EXT_queue_family_foreign extension is not enabled dstQueueFamilyIndex **must** not be VK_QUEUE_FAMILY_FOREIGN_EXT
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-07120
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_DECODE_SRC_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-07121
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_DECODE_DST_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-07122
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-07123

If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_ENCODE_SRC_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR

- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-07124
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_ENCODE_DST_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-07125
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-10287
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_ENCODE_QUANTIZATION_MAP_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-07006
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT then image **must** have been created with either the VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT usage bits, and the VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT or VK_IMAGE_USAGE_SAMPLED_BIT usage bits, and the VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT usage bit
- VUID-VkImageMemoryBarrier2-attachmentFeedbackLoopLayout-07313
  If the attachmentFeedbackLoopLayout feature is not enabled, newLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT
- VUID-VkImageMemoryBarrier2-srcQueueFamilyIndex-09550
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR then image **must** have been created with either VK_IMAGE_USAGE_STORAGE_BIT, or with both VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT and either of VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier2-dynamicRenderingLocalRead-09551
  If the dynamicRenderingLocalRead feature is not enabled, oldLayout **must** not be VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR
- VUID-VkImageMemoryBarrier2-dynamicRenderingLocalRead-09552
  If the dynamicRenderingLocalRead feature is not enabled, newLayout **must** not be VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR
- VUID-VkImageMemoryBarrier2-subresourceRange-01486

subresourceRange.baseMipLevel **must** be less than the mipLevels specified in VkImageCreateInfo when image was created

- VUID-VkImageMemoryBarrier2-subresourceRange-01724
  If subresourceRange.levelCount is not VK_REMAINING_MIP_LEVELS, subresourceRange.baseMipLevel + subresourceRange.levelCount **must** be less than or equal to the mipLevels specified in VkImageCreateInfo when image was created
- VUID-VkImageMemoryBarrier2-subresourceRange-01488
  subresourceRange.baseArrayLayer **must** be less than the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-VkImageMemoryBarrier2-subresourceRange-01725
  If subresourceRange.layerCount is not VK_REMAINING_ARRAY_LAYERS, subresourceRange.baseArrayLayer + subresourceRange.layerCount **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-VkImageMemoryBarrier2-image-01932
  If image is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkImageMemoryBarrier2-image-09241
  If image has a color format that is single-plane, then the aspectMask member of subresourceRange **must** be VK_IMAGE_ASPECT_COLOR_BIT
- VUID-VkImageMemoryBarrier2-image-09242
  If image has a color format and is not *disjoint*, then the aspectMask member of subresourceRange **must** be VK_IMAGE_ASPECT_COLOR_BIT
- VUID-VkImageMemoryBarrier2-image-01672
  If image has a multi-planar format and the image is *disjoint*, then the aspectMask member of subresourceRange **must** include at least one multi-planar aspect mask bit or VK_IMAGE_ASPECT_COLOR_BIT
- VUID-VkImageMemoryBarrier2-image-03320
  If image has a depth/stencil format with both depth and stencil and the separateDepthStencilLayouts feature is not enabled, then the aspectMask member of subresourceRange **must** include both VK_IMAGE_ASPECT_DEPTH_BIT and VK_IMAGE_ASPECT_STENCIL_BIT
- VUID-VkImageMemoryBarrier2-image-03319
  If image has a depth/stencil format with both depth and stencil and the separateDepthStencilLayouts feature is enabled, then the aspectMask member of subresourceRange **must** include either or both VK_IMAGE_ASPECT_DEPTH_BIT and VK_IMAGE_ASPECT_STENCIL_BIT
- VUID-VkImageMemoryBarrier2-aspectMask-08702
  If the aspectMask member of subresourceRange includes VK_IMAGE_ASPECT_DEPTH_BIT, oldLayout and newLayout **must** not be one of VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkImageMemoryBarrier2-aspectMask-08703
  If the aspectMask member of subresourceRange includes VK_IMAGE_ASPECT_STENCIL_BIT, oldLayout and newLayout **must** not be one of VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or

VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL

- VUID-VkImageMemoryBarrier2-subresourceRange-09601
  subresourceRange.aspectMask **must** be valid for the format the image was created with
- VUID-VkImageMemoryBarrier2-srcStageMask-03854
  If either srcStageMask or dstStageMask includes VK_PIPELINE_STAGE_2_HOST_BIT, srcQueueFamilyIndex and dstQueueFamilyIndex **must** be equal
- VUID-VkImageMemoryBarrier2-srcStageMask-03855
  If srcStageMask includes VK_PIPELINE_STAGE_2_HOST_BIT, and srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, oldLayout **must** be one of VK_IMAGE_LAYOUT_PREINITIALIZED, VK_IMAGE_LAYOUT_UNDEFINED, or VK_IMAGE_LAYOUT_GENERAL

Valid Usage (Implicit)

- VUID-VkImageMemoryBarrier2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2
- VUID-VkImageMemoryBarrier2-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkExternalMemoryAcquireUnmodifiedEXT or VkSampleLocationsInfoEXT
- VUID-VkImageMemoryBarrier2-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkImageMemoryBarrier2-srcStageMask-parameter
  srcStageMask **must** be a valid combination of VkPipelineStageFlagBits2 values
- VUID-VkImageMemoryBarrier2-srcAccessMask-parameter
  srcAccessMask **must** be a valid combination of VkAccessFlagBits2 values
- VUID-VkImageMemoryBarrier2-dstStageMask-parameter
  dstStageMask **must** be a valid combination of VkPipelineStageFlagBits2 values
- VUID-VkImageMemoryBarrier2-dstAccessMask-parameter
  dstAccessMask **must** be a valid combination of VkAccessFlagBits2 values
- VUID-VkImageMemoryBarrier2-oldLayout-parameter
  oldLayout **must** be a valid VkImageLayout value
- VUID-VkImageMemoryBarrier2-newLayout-parameter
  newLayout **must** be a valid VkImageLayout value
- VUID-VkImageMemoryBarrier2-image-parameter
  image **must** be a valid VkImage handle
- VUID-VkImageMemoryBarrier2-subresourceRange-parameter
  subresourceRange **must** be a valid VkImageSubresourceRange structure

The VkImageMemoryBarrier structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageMemoryBarrier {
    VkStructureType            sType;
    const void*                pNext;
    VkAccessFlags              srcAccessMask;
    VkAccessFlags              dstAccessMask;
    VkImageLayout              oldLayout;
    VkImageLayout              newLayout;
    uint32_t                   srcQueueFamilyIndex;
    uint32_t                   dstQueueFamilyIndex;
    VkImage                    image;
    VkImageSubresourceRange    subresourceRange;
} VkImageMemoryBarrier;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcAccessMask` is a bitmask of VkAccessFlagBits specifying a source access mask.
- `dstAccessMask` is a bitmask of VkAccessFlagBits specifying a destination access mask.
- `oldLayout` is the old layout in an image layout transition.
- `newLayout` is the new layout in an image layout transition.
- `srcQueueFamilyIndex` is the source queue family for a queue family ownership transfer.
- `dstQueueFamilyIndex` is the destination queue family for a queue family ownership transfer.
- `image` is a handle to the image affected by this barrier.
- `subresourceRange` describes the image subresource range within `image` that is affected by this barrier.

The first access scope is limited to access to memory through the specified image subresource range, via access types in the source access mask specified by `srcAccessMask`. If `srcAccessMask` includes `VK_ACCESS_HOST_WRITE_BIT`, memory writes performed by that access type are also made visible, as that access type is not performed through a resource.

The second access scope is limited to access to memory through the specified image subresource range, via access types in the destination access mask specified by `dstAccessMask`. If `dstAccessMask` includes `VK_ACCESS_HOST_WRITE_BIT` or `VK_ACCESS_HOST_READ_BIT`, available memory writes are also made visible to accesses of those types, as those access types are not performed through a resource.

If `srcQueueFamilyIndex` is not equal to `dstQueueFamilyIndex`, and `srcQueueFamilyIndex` is equal to the current queue family, then the memory barrier defines a queue family release operation for the specified image subresource range, and the second synchronization scope of the calling command does not apply to this operation.

If `dstQueueFamilyIndex` is not equal to `srcQueueFamilyIndex`, and `dstQueueFamilyIndex` is equal to the current queue family, then the memory barrier defines a queue family acquire operation for the specified image subresource range, and the first synchronization scope of the calling command does not apply to this operation.

If the synchronization2 feature is not enabled or oldLayout is not equal to newLayout, oldLayout and newLayout define an image layout transition for the specified image subresource range.

NOTE	If the synchronization2 feature is enabled, when the old and new layout are equal, the layout values are ignored - data is preserved no matter what values are specified, or what layout the image is currently in.

If image has a multi-planar format and the image is *disjoint*, then including VK_IMAGE_ASPECT_COLOR_BIT in the aspectMask member of subresourceRange is equivalent to including VK_IMAGE_ASPECT_PLANE_0_BIT, VK_IMAGE_ASPECT_PLANE_1_BIT, and (for three-plane formats only) VK_IMAGE_ASPECT_PLANE_2_BIT.

Valid Usage

- VUID-VkImageMemoryBarrier-oldLayout-01208
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-oldLayout-01209
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-oldLayout-01210
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-oldLayout-01211
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_SAMPLED_BIT or VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-oldLayout-01212
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_TRANSFER_SRC_BIT
- VUID-VkImageMemoryBarrier-oldLayout-01213
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_TRANSFER_DST_BIT
- VUID-VkImageMemoryBarrier-oldLayout-01197

If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, oldLayout **must** be VK_IMAGE_LAYOUT_UNDEFINED or the current layout of the image subresources affected by the barrier

- VUID-VkImageMemoryBarrier-newLayout-01198
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, newLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED
- VUID-VkImageMemoryBarrier-oldLayout-01658
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-oldLayout-01659
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL then image **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-synchronization2-07793
  If the synchronization2 feature is not enabled, oldLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR or VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkImageMemoryBarrier-synchronization2-07794
  If the synchronization2 feature is not enabled, newLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR or VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-03938
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL, image **must** have been created with VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-03939
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL, image **must** have been created with at least one of VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT, VK_IMAGE_USAGE_SAMPLED_BIT, or VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-oldLayout-02088
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR then image **must** have been created with VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR set
- VUID-VkImageMemoryBarrier-image-09117
  If image was created with a sharing mode of VK_SHARING_MODE_EXCLUSIVE, and srcQueueFamilyIndex and dstQueueFamilyIndex are not equal, srcQueueFamilyIndex **must** be VK_QUEUE_FAMILY_EXTERNAL, VK_QUEUE_FAMILY_FOREIGN_EXT, or a valid queue family

- VUID-VkImageMemoryBarrier-image-09118
  If `image` was created with a sharing mode of `VK_SHARING_MODE_EXCLUSIVE`, and `srcQueueFamilyIndex` and `dstQueueFamilyIndex` are not equal, `dstQueueFamilyIndex` **must** be `VK_QUEUE_FAMILY_EXTERNAL`, `VK_QUEUE_FAMILY_FOREIGN_EXT`, or a valid queue family
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-04070
  If `srcQueueFamilyIndex` is not equal to `dstQueueFamilyIndex`, at least one of `srcQueueFamilyIndex` or `dstQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_EXTERNAL` or `VK_QUEUE_FAMILY_FOREIGN_EXT`
- VUID-VkImageMemoryBarrier-None-09119
  If the VK_KHR_external_memory extension is not enabled, `srcQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_EXTERNAL`
- VUID-VkImageMemoryBarrier-None-09120
  If the VK_KHR_external_memory extension is not enabled, `dstQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_EXTERNAL`
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-09121
  If the VK_EXT_queue_family_foreign extension is not enabled `srcQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_FOREIGN_EXT`
- VUID-VkImageMemoryBarrier-dstQueueFamilyIndex-09122
  If the VK_EXT_queue_family_foreign extension is not enabled `dstQueueFamilyIndex` **must** not be `VK_QUEUE_FAMILY_FOREIGN_EXT`
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-07120
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_VIDEO_DECODE_SRC_KHR` then `image` **must** have been created with `VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR`
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-07121
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_VIDEO_DECODE_DST_KHR` then `image` **must** have been created with `VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR`
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-07122
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR` then `image` **must** have been created with `VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR`
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-07123
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_VIDEO_ENCODE_SRC_KHR` then `image` **must** have been created with `VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR`
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-07124
  If `srcQueueFamilyIndex` and `dstQueueFamilyIndex` define a queue family ownership transfer or `oldLayout` and `newLayout` define an image layout transition, and `oldLayout` or `newLayout` is `VK_IMAGE_LAYOUT_VIDEO_ENCODE_DST_KHR` then `image` **must** have been created with

VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR

- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-07125
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-10287
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_VIDEO_ENCODE_QUANTIZATION_MAP_KHR then image **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-07006
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT then image **must** have been created with either the VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT usage bits, and the VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT or VK_IMAGE_USAGE_SAMPLED_BIT usage bits, and the VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT usage bit
- VUID-VkImageMemoryBarrier-attachmentFeedbackLoopLayout-07313
  If the attachmentFeedbackLoopLayout feature is not enabled, newLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT
- VUID-VkImageMemoryBarrier-srcQueueFamilyIndex-09550
  If srcQueueFamilyIndex and dstQueueFamilyIndex define a queue family ownership transfer or oldLayout and newLayout define an image layout transition, and oldLayout or newLayout is VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR then image **must** have been created with either VK_IMAGE_USAGE_STORAGE_BIT, or with both VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT and either of VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageMemoryBarrier-dynamicRenderingLocalRead-09551
  If the dynamicRenderingLocalRead feature is not enabled, oldLayout **must** not be VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR
- VUID-VkImageMemoryBarrier-dynamicRenderingLocalRead-09552
  If the dynamicRenderingLocalRead feature is not enabled, newLayout **must** not be VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR
- VUID-VkImageMemoryBarrier-subresourceRange-01486
  subresourceRange.baseMipLevel **must** be less than the mipLevels specified in VkImageCreateInfo when image was created
- VUID-VkImageMemoryBarrier-subresourceRange-01724
  If subresourceRange.levelCount is not VK_REMAINING_MIP_LEVELS, subresourceRange.baseMipLevel + subresourceRange.levelCount **must** be less than or equal to the mipLevels specified in VkImageCreateInfo when image was created
- VUID-VkImageMemoryBarrier-subresourceRange-01488

`subresourceRange.baseArrayLayer` **must** be less than the `arrayLayers` specified in VkImageCreateInfo when `image` was created

- VUID-VkImageMemoryBarrier-subresourceRange-01725
  If `subresourceRange.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `subresourceRange.baseArrayLayer` + `subresourceRange.layerCount` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `image` was created
- VUID-VkImageMemoryBarrier-image-01932
  If `image` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkImageMemoryBarrier-image-09241
  If `image` has a color format that is single-plane, then the `aspectMask` member of `subresourceRange` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-VkImageMemoryBarrier-image-09242
  If `image` has a color format and is not *disjoint*, then the `aspectMask` member of `subresourceRange` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-VkImageMemoryBarrier-image-01672
  If `image` has a multi-planar format and the image is *disjoint*, then the `aspectMask` member of `subresourceRange` **must** include at least one multi-planar aspect mask bit or `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-VkImageMemoryBarrier-image-03320
  If `image` has a depth/stencil format with both depth and stencil and the `separateDepthStencilLayouts` feature is not enabled, then the `aspectMask` member of `subresourceRange` **must** include both `VK_IMAGE_ASPECT_DEPTH_BIT` and `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-VkImageMemoryBarrier-image-03319
  If `image` has a depth/stencil format with both depth and stencil and the `separateDepthStencilLayouts` feature is enabled, then the `aspectMask` member of `subresourceRange` **must** include either or both `VK_IMAGE_ASPECT_DEPTH_BIT` and `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-VkImageMemoryBarrier-aspectMask-08702
  If the `aspectMask` member of `subresourceRange` includes `VK_IMAGE_ASPECT_DEPTH_BIT`, `oldLayout` and `newLayout` **must** not be one of `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`
- VUID-VkImageMemoryBarrier-aspectMask-08703
  If the `aspectMask` member of `subresourceRange` includes `VK_IMAGE_ASPECT_STENCIL_BIT`, `oldLayout` and `newLayout` **must** not be one of `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`
- VUID-VkImageMemoryBarrier-subresourceRange-09601
  `subresourceRange.aspectMask` **must** be valid for the `format` the `image` was created with
- VUID-VkImageMemoryBarrier-None-09052
  If the `synchronization2` feature is not enabled, and `image` was created with a sharing mode of `VK_SHARING_MODE_CONCURRENT`, at least one of `srcQueueFamilyIndex` and `dstQueueFamilyIndex` **must** be `VK_QUEUE_FAMILY_IGNORED`

- VUID-VkImageMemoryBarrier-None-09053
  If the synchronization2 feature is not enabled, and image was created with a sharing mode of VK_SHARING_MODE_CONCURRENT, srcQueueFamilyIndex **must** be VK_QUEUE_FAMILY_IGNORED or VK_QUEUE_FAMILY_EXTERNAL
- VUID-VkImageMemoryBarrier-None-09054
  If the synchronization2 feature is not enabled, and image was created with a sharing mode of VK_SHARING_MODE_CONCURRENT, dstQueueFamilyIndex **must** be VK_QUEUE_FAMILY_IGNORED or VK_QUEUE_FAMILY_EXTERNAL

Valid Usage (Implicit)

- VUID-VkImageMemoryBarrier-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER
- VUID-VkImageMemoryBarrier-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkExternalMemoryAcquireUnmodifiedEXT or VkSampleLocationsInfoEXT
- VUID-VkImageMemoryBarrier-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkImageMemoryBarrier-oldLayout-parameter
  oldLayout **must** be a valid VkImageLayout value
- VUID-VkImageMemoryBarrier-newLayout-parameter
  newLayout **must** be a valid VkImageLayout value
- VUID-VkImageMemoryBarrier-image-parameter
  image **must** be a valid VkImage handle
- VUID-VkImageMemoryBarrier-subresourceRange-parameter
  subresourceRange **must** be a valid VkImageSubresourceRange structure

To facilitate usage of images whose memory is initialized on the host, Vulkan allows image layout transitions to be performed by the host as well, albeit supporting limited layouts.

To perform an image layout transition on the host, call:

```
// Provided by VK_EXT_host_image_copy
VkResult vkTransitionImageLayoutEXT(
    VkDevice                                    device,
    uint32_t                                    transitionCount,
    const VkHostImageLayoutTransitionInfoEXT*   pTransitions);
```

- device is the device which owns pTransitions[i].image.
- transitionCount is the number of image layout transitions to perform.
- pTransitions is a pointer to an array of VkHostImageLayoutTransitionInfoEXT structures

specifying the image and subresource ranges within them to transition.

Valid Usage (Implicit)

- VUID-vkTransitionImageLayoutEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkTransitionImageLayoutEXT-pTransitions-parameter
  `pTransitions` **must** be a valid pointer to an array of `transitionCount` valid VkHostImageLayoutTransitionInfoEXT structures
- VUID-vkTransitionImageLayoutEXT-transitionCount-arraylength
  `transitionCount` **must** be greater than `0`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_INITIALIZATION_FAILED`
- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_MEMORY_MAP_FAILED`

The `VkHostImageLayoutTransitionInfoEXT` structure is defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkHostImageLayoutTransitionInfoEXT {
    VkStructureType            sType;
    const void*                pNext;
    VkImage                    image;
    VkImageLayout              oldLayout;
    VkImageLayout              newLayout;
    VkImageSubresourceRange    subresourceRange;
} VkHostImageLayoutTransitionInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `image` is a handle to the image affected by this layout transition.
- `oldLayout` is the old layout in an image layout transition.
- `newLayout` is the new layout in an image layout transition.
- `subresourceRange` describes the image subresource range within `image` that is affected by this layout transition.

vkTransitionImageLayoutEXT does not check whether the device memory associated with an image is currently in use before performing the layout transition. The application **must** guarantee that any previously submitted command that reads from or writes to this subresource has completed before the host performs the layout transition. The memory of image is accessed by the host as if coherent.

NOTE Image layout transitions performed on the host do not require queue family ownership transfers as the physical layout of the image will not vary between queue families for the layouts supported by this function.

NOTE If the device has written to the image memory, it is not automatically made available to the host. Before this command can be called, a memory barrier for this image **must** have been issued on the device with the second synchronization scope including VK_PIPELINE_STAGE_HOST_BIT and VK_ACCESS_HOST_READ_BIT.

Because queue submissions automatically make host memory visible to the device, there would not be a need for a memory barrier before using the results of this layout transition on the device.

Valid Usage

- VUID-VkHostImageLayoutTransitionInfoEXT-image-09055
  image **must** have been created with VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT
- VUID-VkHostImageLayoutTransitionInfoEXT-subresourceRange-01486
  subresourceRange.baseMipLevel **must** be less than the mipLevels specified in VkImageCreateInfo when image was created
- VUID-VkHostImageLayoutTransitionInfoEXT-subresourceRange-01724
  If subresourceRange.levelCount is not VK_REMAINING_MIP_LEVELS, subresourceRange.baseMipLevel + subresourceRange.levelCount **must** be less than or equal to the mipLevels specified in VkImageCreateInfo when image was created
- VUID-VkHostImageLayoutTransitionInfoEXT-subresourceRange-01488
  subresourceRange.baseArrayLayer **must** be less than the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-VkHostImageLayoutTransitionInfoEXT-subresourceRange-01725
  If subresourceRange.layerCount is not VK_REMAINING_ARRAY_LAYERS, subresourceRange.baseArrayLayer + subresourceRange.layerCount **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-VkHostImageLayoutTransitionInfoEXT-image-01932
  If image is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkHostImageLayoutTransitionInfoEXT-image-09241
  If image has a color format that is single-plane, then the aspectMask member of subresourceRange **must** be VK_IMAGE_ASPECT_COLOR_BIT
- VUID-VkHostImageLayoutTransitionInfoEXT-image-09242
  If image has a color format and is not *disjoint*, then the aspectMask member of

subresourceRange **must** be VK_IMAGE_ASPECT_COLOR_BIT

- VUID-VkHostImageLayoutTransitionInfoEXT-image-01672
  If image has a multi-planar format and the image is *disjoint*, then the aspectMask member of subresourceRange **must** include at least one multi-planar aspect mask bit or VK_IMAGE_ASPECT_COLOR_BIT
- VUID-VkHostImageLayoutTransitionInfoEXT-image-03320
  If image has a depth/stencil format with both depth and stencil and the separateDepthStencilLayouts feature is not enabled, then the aspectMask member of subresourceRange **must** include both VK_IMAGE_ASPECT_DEPTH_BIT and VK_IMAGE_ASPECT_STENCIL_BIT
- VUID-VkHostImageLayoutTransitionInfoEXT-image-03319
  If image has a depth/stencil format with both depth and stencil and the separateDepthStencilLayouts feature is enabled, then the aspectMask member of subresourceRange **must** include either or both VK_IMAGE_ASPECT_DEPTH_BIT and VK_IMAGE_ASPECT_STENCIL_BIT
- VUID-VkHostImageLayoutTransitionInfoEXT-aspectMask-08702
  If the aspectMask member of subresourceRange includes VK_IMAGE_ASPECT_DEPTH_BIT, oldLayout and newLayout **must** not be one of VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkHostImageLayoutTransitionInfoEXT-aspectMask-08703
  If the aspectMask member of subresourceRange includes VK_IMAGE_ASPECT_STENCIL_BIT, oldLayout and newLayout **must** not be one of VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL
- VUID-VkHostImageLayoutTransitionInfoEXT-subresourceRange-09601
  subresourceRange.aspectMask **must** be valid for the format the image was created with
- VUID-VkHostImageLayoutTransitionInfoEXT-oldLayout-09229
  oldLayout **must** be either VK_IMAGE_LAYOUT_UNDEFINED or the current layout of the image subresources as specified in subresourceRange
- VUID-VkHostImageLayoutTransitionInfoEXT-oldLayout-09230
  If oldLayout is not VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED, it **must** be one of the layouts in VkPhysicalDeviceHostImageCopyPropertiesEXT::pCopySrcLayouts
- VUID-VkHostImageLayoutTransitionInfoEXT-newLayout-09057
  newLayout **must** be one of the layouts in VkPhysicalDeviceHostImageCopyPropertiesEXT::pCopyDstLayouts

Valid Usage (Implicit)

- VUID-VkHostImageLayoutTransitionInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_HOST_IMAGE_LAYOUT_TRANSITION_INFO_EXT
- VUID-VkHostImageLayoutTransitionInfoEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkHostImageLayoutTransitionInfoEXT-image-parameter

`image` **must** be a valid VkImage handle

- VUID-VkHostImageLayoutTransitionInfoEXT-oldLayout-parameter
  `oldLayout` **must** be a valid VkImageLayout value
- VUID-VkHostImageLayoutTransitionInfoEXT-newLayout-parameter
  `newLayout` **must** be a valid VkImageLayout value
- VUID-VkHostImageLayoutTransitionInfoEXT-subresourceRange-parameter
  `subresourceRange` **must** be a valid VkImageSubresourceRange structure

7.7.4. Queue Family Ownership Transfer

Resources created with a VkSharingMode of `VK_SHARING_MODE_EXCLUSIVE` **must** have their ownership explicitly transferred from one queue family to another in order to access their content in a well-defined manner on a queue in a different queue family.

The special queue family index `VK_QUEUE_FAMILY_IGNORED` indicates that a queue family parameter or member is ignored.

```
#define VK_QUEUE_FAMILY_IGNORED           (~0U)
```

Resources shared with external APIs or instances using external memory **must** also explicitly manage ownership transfers between local and external queues (or equivalent constructs in external APIs) regardless of the VkSharingMode specified when creating them.

The special queue family index `VK_QUEUE_FAMILY_EXTERNAL` represents any queue external to the resource's current Vulkan instance, as long as the queue uses the same underlying device group or physical device, and the same driver version as the resource's VkDevice, as indicated by VkPhysicalDeviceIDProperties::`deviceUUID` and VkPhysicalDeviceIDProperties::`driverUUID`.

```
#define VK_QUEUE_FAMILY_EXTERNAL          (~1U)
```

or the equivalent

```
#define VK_QUEUE_FAMILY_EXTERNAL_KHR      VK_QUEUE_FAMILY_EXTERNAL
```

The special queue family index `VK_QUEUE_FAMILY_FOREIGN_EXT` represents any queue external to the resource's current Vulkan instance, regardless of the queue's underlying physical device or driver version. This includes, for example, queues for fixed-function image processing devices, media codec devices, and display devices, as well as all queues that use the same underlying device group or physical device, and the same driver version as the resource's VkDevice.

```
#define VK_QUEUE_FAMILY_FOREIGN_EXT       (~2U)
```

If memory dependencies are correctly expressed between uses of such a resource between two

queues in different families, but no ownership transfer is defined, the contents of that resource are undefined for any read accesses performed by the second queue family.

NOTE If an application does not need the contents of a resource to remain valid when transferring from one queue family to another, then the ownership transfer **should** be skipped.

NOTE Applications should expect transfers to/from `VK_QUEUE_FAMILY_FOREIGN_EXT` to be more expensive than transfers to/from `VK_QUEUE_FAMILY_EXTERNAL_KHR`.

A queue family ownership transfer consists of two distinct parts:

1. Release exclusive ownership from the source queue family
2. Acquire exclusive ownership for the destination queue family

An application **must** ensure that these operations occur in the correct order by defining an execution dependency between them, e.g. using a semaphore.

A *release operation* is used to release exclusive ownership of a range of a buffer or image subresource range. A release operation is defined by executing a buffer memory barrier (for a buffer range) or an image memory barrier (for an image subresource range) using a pipeline barrier command, on a queue from the source queue family. The `srcQueueFamilyIndex` parameter of the barrier **must** be the source queue family index, and the `dstQueueFamilyIndex` parameter to the destination queue family index. `dstAccessMask` is ignored for such a barrier, such that no visibility operation is executed - the value of this mask does not affect the validity of the barrier. The release operation happens-after the availability operation. `dstStageMask` is also ignored for such a barrier as defined by buffer memory ownership transfer and image memory ownership transfer.

An *acquire operation* is used to acquire exclusive ownership of a range of a buffer or image subresource range. An acquire operation is defined by executing a buffer memory barrier (for a buffer range) or an image memory barrier (for an image subresource range) using a pipeline barrier command, on a queue from the destination queue family. The buffer range or image subresource range specified in an acquire operation **must** match exactly that of a previous release operation. The `srcQueueFamilyIndex` parameter of the barrier **must** be the source queue family index, and the `dstQueueFamilyIndex` parameter to the destination queue family index. `srcAccessMask` is ignored for such a barrier, such that no availability operation is executed - the value of this mask does not affect the validity of the barrier. The acquire operation happens-before the visibility operation. `srcStageMask` is also ignored for such a barrier as defined by buffer memory ownership transfer and image memory ownership transfer. As the first synchronization scope for an acquire operation is empty there is no happens-before dependency. Such a dependency **can** be introduced by using `VK_PIPELINE_STAGE_ALL_COMMANDS_BIT`.

NOTE Whilst it is not invalid to provide destination or source access masks for memory barriers used for release or acquire operations, respectively, they have no practical effect. Access after a release operation has undefined results, and so visibility for those accesses has no practical effect. Similarly, write access before an acquire operation will produce undefined results for future access, so availability of those writes has no practical use. In an earlier version of the specification, these were

required to match on both sides - but this was subsequently relaxed. These masks **should** be set to 0.

NOTE

Since a release and acquire operation does not synchronize with second and first scopes respectively, the `VK_PIPELINE_STAGE_ALL_COMMANDS_BIT` stage must be used to wait for a release operation to complete. Typically, a release and acquire pair is performed by a VkSemaphore signal and wait in their respective queues. Signaling a semaphore with vkQueueSubmit waits for `VK_PIPELINE_STAGE_ALL_COMMANDS_BIT`. With vkQueueSubmit2KHR, `stageMask` for the signal semaphore must be `VK_PIPELINE_STAGE_ALL_COMMANDS_BIT`. Similarly, for the acquire operation, waiting for a semaphore must use `VK_PIPELINE_STAGE_ALL_COMMANDS_BIT` to make sure the acquire operation is synchronized.

If the transfer is via an image memory barrier, and an image layout transition is desired, then the values of `oldLayout` and `newLayout` in the *release operation*'s memory barrier **must** be equal to values of `oldLayout` and `newLayout` in the *acquire operation*'s memory barrier. Although the image layout transition is submitted twice, it will only be executed once. A layout transition specified in this way happens-after the *release operation* and happens-before the *acquire operation*.

If the values of `srcQueueFamilyIndex` and `dstQueueFamilyIndex` are equal, no ownership transfer is performed, and the barrier operates as if they were both set to `VK_QUEUE_FAMILY_IGNORED`.

Queue family ownership transfers **may** perform read and write accesses on all memory bound to the image subresource or buffer range, so applications **must** ensure that all memory writes have been made available before a queue family ownership transfer is executed. Available memory is automatically made visible to queue family release and acquire operations, and writes performed by those operations are automatically made available.

Once a queue family has acquired ownership of a buffer range or image subresource range of a `VK_SHARING_MODE_EXCLUSIVE` resource, its contents are undefined to other queue families unless ownership is transferred. The contents of any portion of another resource which aliases memory that is bound to the transferred buffer or image subresource range are undefined after a release or acquire operation.

NOTE

Because events **cannot** be used directly for inter-queue synchronization, and because vkCmdSetEvent does not have the queue family index or memory barrier parameters needed by a *release operation*, the release and acquire operations of a queue family ownership transfer **can** only be performed using vkCmdPipelineBarrier.

An *acquire operation* **may** have a performance penalty when acquiring ownership of a subresource range from one of the special queue families reserved for external memory ownership transfers described above. The application **can** reduce the performance penalty in some cases by adding a VkExternalMemoryAcquireUnmodifiedEXT structure to the `pNext` chain of the *acquire operation*'s memory barrier structure.

The `VkExternalMemoryAcquireUnmodifiedEXT` structure is defined as:

```
// Provided by VK_EXT_external_memory_acquire_unmodified
typedef struct VkExternalMemoryAcquireUnmodifiedEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           acquireUnmodifiedMemory;
} VkExternalMemoryAcquireUnmodifiedEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `acquireUnmodifiedMemory` specifies, if `VK_TRUE`, that no range of VkDeviceMemory bound to the resource of the memory barrier's subresource range was modified at any time since the resource's most recent release of ownership to the queue family specified by the memory barrier's `srcQueueFamilyIndex`. If `VK_FALSE`, it specifies nothing.

If the application releases ownership of the subresource range to one of the special queue families reserved for external memory ownership transfers with a memory barrier structure, and later re-acquires ownership from the same queue family with a memory barrier structure, and if no range of VkDeviceMemory bound to the resource was modified at any time between the *release operation* and the *acquire operation*, then the application **should** add a VkExternalMemoryAcquireUnmodifiedEXT structure to the `pNext` chain of the *acquire operation*'s memory barrier structure because this **may** reduce the performance penalty.

This struct is ignored if `acquireUnmodifiedMemory` is `VK_FALSE`. In particular, `VK_FALSE` does *not* specify that memory was modified.

This struct is ignored if the memory barrier's `srcQueueFamilyIndex` is not a special queue family reserved for external memory ownership transfers.

NOTE

The method by which the application determines whether memory was modified between the *release operation* and *acquire operation* is outside the scope of Vulkan.

For any Vulkan operation that accesses a resource, the application **must** not assume the implementation accesses the resource's memory as read-only, even for *apparently* read-only operations such as transfer commands and shader reads.

The validity of VkExternalMemoryAcquireUnmodifiedEXT::`acquireUnmodifiedMemory` is independent of memory ranges outside the ranges of VkDeviceMemory bound to the resource. In particular, it is independent of any implementation-private memory associated with the resource.

Valid Usage

- VUID-VkExternalMemoryAcquireUnmodifiedEXT-acquireUnmodifiedMemory-08922
  If `acquireUnmodifiedMemory` is `VK_TRUE`, and the memory barrier's `srcQueueFamilyIndex` is a special queue family reserved for external memory ownership transfers (as described in Queue Family Ownership Transfer), then each range of VkDeviceMemory bound to the resource **must** have remained unmodified during all time since the resource's most

recent release of ownership to the queue family

Valid Usage (Implicit)

- VUID-VkExternalMemoryAcquireUnmodifiedEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_ACQUIRE_UNMODIFIED_EXT`

7.8. Wait Idle Operations

To wait on the host for the completion of outstanding queue operations for a given queue, call:

```
// Provided by VK_VERSION_1_0
VkResult vkQueueWaitIdle(
    VkQueue                                     queue);
```

- `queue` is the queue on which to wait.

`vkQueueWaitIdle` is equivalent to having submitted a valid fence to every previously executed queue submission command that accepts a fence, then waiting for all of those fences to signal using vkWaitForFences with an infinite timeout and `waitAll` set to `VK_TRUE`.

Valid Usage (Implicit)

- VUID-vkQueueWaitIdle-queue-parameter
  `queue` **must** be a valid VkQueue handle

Host Synchronization

- Host access to `queue` **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
-	-	-	Any	-

Return Codes

Success

- `VK_SUCCESS`

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

To wait on the host for the completion of outstanding queue operations for all queues on a given logical device, call:

```
// Provided by VK_VERSION_1_0
VkResult vkDeviceWaitIdle(
    VkDevice                                    device);
```

- device is the logical device to idle.

vkDeviceWaitIdle is equivalent to calling vkQueueWaitIdle for all queues owned by device.

Valid Usage (Implicit)

- VUID-vkDeviceWaitIdle-device-parameter
  device **must** be a valid VkDevice handle

Host Synchronization

- Host access to all VkQueue objects created from device **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

7.9. Host Write Ordering Guarantees

When batches of command buffers are submitted to a queue via a queue submission command, it defines a memory dependency with prior host operations, and execution of command buffers submitted to the queue.

The first synchronization scope includes execution of vkQueueSubmit on the host and anything

that happened-before it, as defined by the host memory model.

NOTE	Some systems allow writes that do not directly integrate with the host memory model; these have to be synchronized by the application manually. One example of this is non-temporal store instructions on x86; to ensure these happen-before submission, applications should call `_mm_sfence()`.

The second synchronization scope includes all commands submitted in the same queue submission, and all commands that occur later in submission order.

The first access scope includes all host writes to mappable device memory that are available to the host memory domain.

The second access scope includes all memory access performed by the device.

7.10. Synchronization and Multiple Physical Devices

If a logical device includes more than one physical device, then fences, semaphores, and events all still have a single instance of the signaled state.

A fence becomes signaled when all physical devices complete the necessary queue operations.

Semaphore wait and signal operations all include a device index that is the sole physical device that performs the operation. These indices are provided in the VkDeviceGroupSubmitInfo and VkDeviceGroupBindSparseInfo structures. Semaphores are not exclusively owned by any physical device. For example, a semaphore can be signaled by one physical device and then waited on by a different physical device.

An event **can** only be waited on by the same physical device that signaled it (or the host).

7.11. Calibrated Timestamps

In order to be able to correlate the time a particular operation took place at on timelines of different time domains (e.g. a device operation vs. a host operation), Vulkan allows querying calibrated timestamps from multiple time domains.

To query calibrated timestamps from a set of time domains, call:

```
// Provided by VK_KHR_calibrated_timestamps
VkResult vkGetCalibratedTimestampsKHR(
    VkDevice                                    device,
    uint32_t                                    timestampCount,
    const VkCalibratedTimestampInfoKHR*         pTimestampInfos,
    uint64_t*                                   pTimestamps,
    uint64_t*                                   pMaxDeviation);
```

- `device` is the logical device used to perform the query.
- `timestampCount` is the number of timestamps to query.

- `pTimestampInfos` is a pointer to an array of `timestampCount` VkCalibratedTimestampInfoKHR structures, describing the time domains the calibrated timestamps should be captured from.
- `pTimestamps` is a pointer to an array of `timestampCount` 64-bit unsigned integer values in which the requested calibrated timestamp values are returned.
- `pMaxDeviation` is a pointer to a 64-bit unsigned integer value in which the strictly positive maximum deviation, in nanoseconds, of the calibrated timestamp values is returned.

NOTE

The maximum deviation **may** vary between calls to `vkGetCalibratedTimestampsKHR` even for the same set of time domains due to implementation and platform specific reasons. It is the application's responsibility to assess whether the returned maximum deviation makes the timestamp values suitable for any particular purpose and **can** choose to re-issue the timestamp calibration call pursuing a lower deviation value.

Calibrated timestamp values **can** be extrapolated to estimate future coinciding timestamp values, however, depending on the nature of the time domains and other properties of the platform extrapolating values over a sufficiently long period of time **may** no longer be accurate enough to fit any particular purpose, so applications are expected to re-calibrate the timestamps on a regular basis.

Valid Usage

- VUID-vkGetCalibratedTimestampsEXT-timeDomain-09246
  The `timeDomain` value of each VkCalibratedTimestampInfoKHR in `pTimestampInfos` **must** be unique

Valid Usage (Implicit)

- VUID-vkGetCalibratedTimestampsKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetCalibratedTimestampsKHR-pTimestampInfos-parameter
  `pTimestampInfos` **must** be a valid pointer to an array of `timestampCount` valid VkCalibratedTimestampInfoKHR structures
- VUID-vkGetCalibratedTimestampsKHR-pTimestamps-parameter
  `pTimestamps` **must** be a valid pointer to an array of `timestampCount` `uint64_t` values
- VUID-vkGetCalibratedTimestampsKHR-pMaxDeviation-parameter
  `pMaxDeviation` **must** be a valid pointer to a `uint64_t` value
- VUID-vkGetCalibratedTimestampsKHR-timestampCount-arraylength
  `timestampCount` **must** be greater than `0`

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkCalibratedTimestampInfoKHR structure is defined as:

```
// Provided by VK_KHR_calibrated_timestamps
typedef struct VkCalibratedTimestampInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkTimeDomainKHR    timeDomain;
} VkCalibratedTimestampInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- timeDomain is a VkTimeDomainKHR value specifying the time domain from which the calibrated timestamp value should be returned.

Valid Usage

- VUID-VkCalibratedTimestampInfoEXT-timeDomain-02354
  timeDomain **must** be one of the VkTimeDomainKHR values returned by vkGetPhysicalDeviceCalibrateableTimeDomainsKHR

Valid Usage (Implicit)

- VUID-VkCalibratedTimestampInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_CALIBRATED_TIMESTAMP_INFO_KHR
- VUID-VkCalibratedTimestampInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkCalibratedTimestampInfoKHR-timeDomain-parameter
  timeDomain **must** be a valid VkTimeDomainKHR value

The set of supported time domains consists of:

```
// Provided by VK_KHR_calibrated_timestamps
typedef enum VkTimeDomainKHR {
    VK_TIME_DOMAIN_DEVICE_KHR = 0,
    VK_TIME_DOMAIN_CLOCK_MONOTONIC_KHR = 1,
    VK_TIME_DOMAIN_CLOCK_MONOTONIC_RAW_KHR = 2,
    VK_TIME_DOMAIN_QUERY_PERFORMANCE_COUNTER_KHR = 3,
} VkTimeDomainKHR;
```

- VK_TIME_DOMAIN_DEVICE_KHR specifies the device time domain. Timestamp values in this time domain use the same units and are comparable with device timestamp values captured using vkCmdWriteTimestamp or vkCmdWriteTimestamp2KHR and are defined to be incrementing according to the timestampPeriod of the device.
- VK_TIME_DOMAIN_CLOCK_MONOTONIC_KHR specifies the CLOCK_MONOTONIC time domain available on POSIX platforms. Timestamp values in this time domain are in units of nanoseconds and are comparable with platform timestamp values captured using the POSIX clock_gettime API as computed by this example:

NOTE

An implementation supporting VK_KHR_calibrated_timestamps will use the same time domain for all its VkQueue so that timestamp values reported for VK_TIME_DOMAIN_DEVICE_KHR can be matched to any timestamp captured through vkCmdWriteTimestamp or vkCmdWriteTimestamp2KHR .

```
struct timespec tv;
clock_gettime(CLOCK_MONOTONIC, &tv);
return tv.tv_nsec + tv.tv_sec*1000000000ull;
```

- VK_TIME_DOMAIN_CLOCK_MONOTONIC_RAW_KHR specifies the CLOCK_MONOTONIC_RAW time domain available on POSIX platforms. Timestamp values in this time domain are in units of nanoseconds and are comparable with platform timestamp values captured using the POSIX clock_gettime API as computed by this example:

```
struct timespec tv;
clock_gettime(CLOCK_MONOTONIC_RAW, &tv);
return tv.tv_nsec + tv.tv_sec*1000000000ull;
```

- VK_TIME_DOMAIN_QUERY_PERFORMANCE_COUNTER_KHR specifies the performance counter (QPC) time domain available on Windows. Timestamp values in this time domain are in the same units as those provided by the Windows QueryPerformanceCounter API and are comparable with platform timestamp values captured using that API as computed by this example:

```
LARGE_INTEGER counter;
QueryPerformanceCounter(&counter);
return counter.QuadPart;
```

Chapter 8. Render Pass

Draw commands **must** be recorded within a *render pass instance*. Each render pass instance defines a set of image resources, referred to as *attachments*, used during rendering.

To begin a render pass instance, call:

```
// Provided by VK_KHR_dynamic_rendering
void vkCmdBeginRenderingKHR(
    VkCommandBuffer                             commandBuffer,
    const VkRenderingInfo*                      pRenderingInfo);
```

- `commandBuffer` is the command buffer in which to record the command.
- `pRenderingInfo` is a pointer to a VkRenderingInfo structure specifying details of the render pass instance to begin.

After beginning a render pass instance, the command buffer is ready to record draw commands.

If `pRenderingInfo->flags` includes `VK_RENDERING_RESUMING_BIT` then this render pass is resumed from a render pass instance that has been suspended earlier in submission order.

Valid Usage

- VUID-vkCmdBeginRendering-dynamicRendering-06446
  The `dynamicRendering` feature **must** be enabled
- VUID-vkCmdBeginRendering-commandBuffer-06068
  If `commandBuffer` is a secondary command buffer, and the `nestedCommandBuffer` feature is not enabled, `pRenderingInfo->flags` **must** not include `VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT`
- VUID-vkCmdBeginRendering-pRenderingInfo-09588
  If `pRenderingInfo->pDepthAttachment` is not `NULL` and `pRenderingInfo->pDepthAttachment->imageView` is not `VK_NULL_HANDLE`, `pRenderingInfo->pDepthAttachment->imageView` **must** be in the layout specified by `pRenderingInfo->pDepthAttachment->imageLayout`
- VUID-vkCmdBeginRendering-pRenderingInfo-09589
  If `pRenderingInfo->pDepthAttachment` is not `NULL`, `pRenderingInfo->pDepthAttachment->imageView` is not `VK_NULL_HANDLE`, `pRenderingInfo->pDepthAttachment->imageResolveMode` is not `VK_RESOLVE_MODE_NONE`, and `pRenderingInfo->pDepthAttachment->resolveImageView` is not `VK_NULL_HANDLE`, `pRenderingInfo->pDepthAttachment->resolveImageView` **must** be in the layout specified by `pRenderingInfo->pDepthAttachment->resolveImageLayout`
- VUID-vkCmdBeginRendering-pRenderingInfo-09590
  If `pRenderingInfo->pStencilAttachment` is not `NULL` and `pRenderingInfo->pStencilAttachment->imageView` is not `VK_NULL_HANDLE`, `pRenderingInfo->pStencilAttachment->imageView` **must** be in the layout specified by `pRenderingInfo->pStencilAttachment->imageLayout`

- VUID-vkCmdBeginRendering-pRenderingInfo-09591
  If `pRenderingInfo->pStencilAttachment` is not `NULL`, `pRenderingInfo->pStencilAttachment->imageView` is not VK_NULL_HANDLE, `pRenderingInfo->pStencilAttachment->imageResolveMode` is not `VK_RESOLVE_MODE_NONE`, and `pRenderingInfo->pStencilAttachment->resolveImageView` is not VK_NULL_HANDLE, `pRenderingInfo->pStencilAttachment->resolveImageView` **must** be in the layout specified by `pRenderingInfo->pStencilAttachment->resolveImageLayout`
- VUID-vkCmdBeginRendering-pRenderingInfo-09592
  For any element of `pRenderingInfo->pColorAttachments`, if `imageView` is not VK_NULL_HANDLE, that image view **must** be in the layout specified by `imageLayout`
- VUID-vkCmdBeginRendering-pRenderingInfo-09593
  For any element of `pRenderingInfo->pColorAttachments`, if `imageView` is not VK_NULL_HANDLE and `resolveMode` is not `VK_RESOLVE_MODE_NONE`, and `resolveImageView` is not VK_NULL_HANDLE, `resolveImageView` **must** be in the layout specified by `resolveImageLayout`

Valid Usage (Implicit)

- VUID-vkCmdBeginRendering-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBeginRendering-pRenderingInfo-parameter
  `pRenderingInfo` **must** be a valid pointer to a valid VkRenderingInfo structure
- VUID-vkCmdBeginRendering-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBeginRendering-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBeginRendering-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBeginRendering-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics	Action State

The `VkRenderingInfo` structure is defined as:

```
typedef struct VkRenderingInfo {
    VkStructureType                     sType;
    const void*                         pNext;
    VkRenderingFlags                    flags;
    VkRect2D                            renderArea;
    uint32_t                            layerCount;
    uint32_t                            viewMask;
    uint32_t                            colorAttachmentCount;
    const VkRenderingAttachmentInfo*    pColorAttachments;
    const VkRenderingAttachmentInfo*    pDepthAttachment;
    const VkRenderingAttachmentInfo*    pStencilAttachment;
} VkRenderingInfo;
```

or the equivalent

```
// Provided by VK_KHR_dynamic_rendering
typedef VkRenderingInfo VkRenderingInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkRenderingFlagBits.
- `renderArea` is the render area that is affected by the render pass instance.
- `layerCount` is the number of layers rendered to in each attachment when `viewMask` is `0`.
- `viewMask` is the view mask indicating the indices of attachment layers that will be rendered when it is not `0`.
- `colorAttachmentCount` is the number of elements in `pColorAttachments`.
- `pColorAttachments` is a pointer to an array of `colorAttachmentCount` VkRenderingAttachmentInfo structures describing any color attachments used.
- `pDepthAttachment` is a pointer to a VkRenderingAttachmentInfo structure describing a depth attachment.
- `pStencilAttachment` is a pointer to a VkRenderingAttachmentInfo structure describing a stencil attachment.

If viewMask is not 0, multiview is enabled.

If there is an instance of VkDeviceGroupRenderPassBeginInfo included in the pNext chain and its deviceRenderAreaCount member is not 0, then renderArea is ignored, and the render area is defined per-device by that structure.

Each element of the pColorAttachments array corresponds to an output location in the shader, i.e. if the shader declares an output variable decorated with a Location value of **X**, then it uses the attachment provided in pColorAttachments[**X**]. If the imageView member of any element of pColorAttachments is VK_NULL_HANDLE, writes to the corresponding location by a fragment are discarded.

Valid Usage

- VUID-VkRenderingInfo-viewMask-06069
  If viewMask is 0, layerCount **must** not be 0
- VUID-VkRenderingInfo-multisampledRenderToSingleSampled-06857
  imageView members of pDepthAttachment, pStencilAttachment, and elements of pColorAttachments that are not VK_NULL_HANDLE **must** have been created with the same sampleCount
- VUID-VkRenderingInfo-imageView-09429
  imageView members of elements of pColorAttachments that are not VK_NULL_HANDLE **must** have been created with the same sampleCount
- VUID-VkRenderingInfo-None-08994
  If VkDeviceGroupRenderPassBeginInfo::deviceRenderAreaCount is 0, renderArea.extent.width **must** be greater than 0
- VUID-VkRenderingInfo-None-08995
  If VkDeviceGroupRenderPassBeginInfo::deviceRenderAreaCount is 0, renderArea.extent.height **must** be greater than 0
- VUID-VkRenderingInfo-pNext-06077
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, renderArea.offset.x **must** be greater than or equal to 0
- VUID-VkRenderingInfo-pNext-06078
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, renderArea.offset.y **must** be greater than or equal to 0
- VUID-VkRenderingInfo-pNext-07815
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, the sum of renderArea.extent.width and renderArea.offset.x **must** be less than or equal to maxFramebufferWidth
- VUID-VkRenderingInfo-pNext-07816
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, the sum of renderArea.extent.height and renderArea.offset.y **must** be less than or equal to maxFramebufferHeight

- VUID-VkRenderingInfo-pNext-06079
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, the width of the imageView member of any element of pColorAttachments, pDepthAttachment, or pStencilAttachment that is not VK_NULL_HANDLE **must** be greater than or equal to renderArea.offset.x + renderArea.extent.width
- VUID-VkRenderingInfo-pNext-06080
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, the height of the imageView member of any element of pColorAttachments, pDepthAttachment, or pStencilAttachment that is not VK_NULL_HANDLE **must** be greater than or equal to renderArea.offset.y + renderArea.extent.height
- VUID-VkRenderingInfo-pNext-06083
  If the pNext chain contains VkDeviceGroupRenderPassBeginInfo, the width of the imageView member of any element of pColorAttachments, pDepthAttachment, or pStencilAttachment that is not VK_NULL_HANDLE **must** be greater than or equal to the sum of the offset.x and extent.width members of each element of pDeviceRenderAreas
- VUID-VkRenderingInfo-pNext-06084
  If the pNext chain contains VkDeviceGroupRenderPassBeginInfo, the height of the imageView member of any element of pColorAttachments, pDepthAttachment, or pStencilAttachment that is not VK_NULL_HANDLE **must** be greater than or equal to the sum of the offset.y and extent.height members of each element of pDeviceRenderAreas
- VUID-VkRenderingInfo-pDepthAttachment-06085
  If neither pDepthAttachment or pStencilAttachment are NULL and the imageView member of either structure is not VK_NULL_HANDLE, the imageView member of each structure **must** be the same
- VUID-VkRenderingInfo-pDepthAttachment-06086
  If neither pDepthAttachment or pStencilAttachment are NULL, and the resolveMode member of each is not VK_RESOLVE_MODE_NONE, the resolveImageView member of each structure **must** be the same
- VUID-VkRenderingInfo-colorAttachmentCount-06087
  If colorAttachmentCount is not 0 and the imageView member of an element of pColorAttachments is not VK_NULL_HANDLE, that imageView **must** have been created with VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT
- VUID-VkRenderingInfo-colorAttachmentCount-09476
  If colorAttachmentCount is not 0 and there is an element of pColorAttachments with its imageView member not VK_NULL_HANDLE, and its resolveMode member not set to VK_RESOLVE_MODE_NONE, the resolveImageView member of that element of pColorAttachments **must** have been created with VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT
- VUID-VkRenderingInfo-pDepthAttachment-06547
  If pDepthAttachment is not NULL and pDepthAttachment->imageView is not VK_NULL_HANDLE, pDepthAttachment->imageView **must** have been created with a format that includes a depth component
- VUID-VkRenderingInfo-pDepthAttachment-06088

If pDepthAttachment is not NULL and pDepthAttachment->imageView is not VK_NULL_HANDLE, pDepthAttachment->imageView **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT

- VUID-VkRenderingInfo-pDepthAttachment-09477
  If pDepthAttachment is not NULL and pDepthAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pDepthAttachment->resolveImageView **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkRenderingInfo-pStencilAttachment-06548
  If pStencilAttachment is not NULL and pStencilAttachment->imageView is not VK_NULL_HANDLE, pStencilAttachment->imageView **must** have been created with a format that includes a stencil aspect
- VUID-VkRenderingInfo-pStencilAttachment-06089
  If pStencilAttachment is not NULL and pStencilAttachment->imageView is not VK_NULL_HANDLE, pStencilAttachment->imageView **must** have been created with a stencil usage including VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkRenderingInfo-pStencilAttachment-09478
  If pStencilAttachment is not NULL and pStencilAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pStencilAttachment->resolveImageView **must** have been created with VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkRenderingInfo-colorAttachmentCount-06090
  If colorAttachmentCount is not 0 and the imageView member of an element of pColorAttachments is not VK_NULL_HANDLE, the layout member of that element of pColorAttachments **must** not be VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-colorAttachmentCount-06091
  If colorAttachmentCount is not 0 and the imageView member of an element of pColorAttachments is not VK_NULL_HANDLE, if the resolveMode member of that element of pColorAttachments is not VK_RESOLVE_MODE_NONE, its resolveImageLayout member **must** not be VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-pDepthAttachment-06092
  If pDepthAttachment is not NULL and pDepthAttachment->imageView is not VK_NULL_HANDLE, pDepthAttachment->layout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL
- VUID-VkRenderingInfo-pDepthAttachment-06093
  If pDepthAttachment is not NULL, pDepthAttachment->imageView is not VK_NULL_HANDLE, and pDepthAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pDepthAttachment->resolveImageLayout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL
- VUID-VkRenderingInfo-pStencilAttachment-06094
  If pStencilAttachment is not NULL and pStencilAttachment->imageView is not VK_NULL_HANDLE, pStencilAttachment->layout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL
- VUID-VkRenderingInfo-pStencilAttachment-06095
  If pStencilAttachment is not NULL, pStencilAttachment->imageView is not VK_NULL_HANDLE, and pStencilAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pStencilAttachment-

>resolveImageLayout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL

- VUID-VkRenderingInfo-colorAttachmentCount-06096
  If colorAttachmentCount is not 0 and the imageView member of an element of pColorAttachments is not VK_NULL_HANDLE, the layout member of that element of pColorAttachments **must** not be VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-colorAttachmentCount-06097
  If colorAttachmentCount is not 0 and the imageView member of an element of pColorAttachments is not VK_NULL_HANDLE, if the resolveMode member of that element of pColorAttachments is not VK_RESOLVE_MODE_NONE, its resolveImageLayout member **must** not be VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-pDepthAttachment-06098
  If pDepthAttachment is not NULL, pDepthAttachment->imageView is not VK_NULL_HANDLE, and pDepthAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pDepthAttachment->resolveImageLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkRenderingInfo-pStencilAttachment-06099
  If pStencilAttachment is not NULL, pStencilAttachment->imageView is not VK_NULL_HANDLE, and pStencilAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pStencilAttachment->resolveImageLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-colorAttachmentCount-06100
  If colorAttachmentCount is not 0 and the imageView member of an element of pColorAttachments is not VK_NULL_HANDLE, the layout member of that element of pColorAttachments **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-colorAttachmentCount-06101
  If colorAttachmentCount is not 0 and the imageView member of an element of pColorAttachments is not VK_NULL_HANDLE, if the resolveMode member of that element of pColorAttachments is not VK_RESOLVE_MODE_NONE, its resolveImageLayout member **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-pDepthAttachment-07732
  If pDepthAttachment is not NULL and pDepthAttachment->imageView is not VK_NULL_HANDLE, pDepthAttachment->layout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-pDepthAttachment-07733
  If pDepthAttachment is not NULL, pDepthAttachment->imageView is not VK_NULL_HANDLE, and pDepthAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pDepthAttachment->resolveImageLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL

- VUID-VkRenderingInfo-pStencilAttachment-07734
  If pStencilAttachment is not NULL and pStencilAttachment->imageView is not VK_NULL_HANDLE, pStencilAttachment->layout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-pStencilAttachment-07735
  If pStencilAttachment is not NULL, pStencilAttachment->imageView is not VK_NULL_HANDLE, and pStencilAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pStencilAttachment->resolveImageLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL
- VUID-VkRenderingInfo-pDepthAttachment-06102
  If pDepthAttachment is not NULL and pDepthAttachment->imageView is not VK_NULL_HANDLE, pDepthAttachment->resolveMode **must** be one of the bits set in VkPhysicalDeviceDepthStencilResolveProperties::supportedDepthResolveModes
- VUID-VkRenderingInfo-pStencilAttachment-06103
  If pStencilAttachment is not NULL and pStencilAttachment->imageView is not VK_NULL_HANDLE, pStencilAttachment->resolveMode **must** be one of the bits set in VkPhysicalDeviceDepthStencilResolveProperties::supportedStencilResolveModes
- VUID-VkRenderingInfo-pDepthAttachment-06104
  If pDepthAttachment or pStencilAttachment are both not NULL, pDepthAttachment->imageView and pStencilAttachment->imageView are both not VK_NULL_HANDLE, and VkPhysicalDeviceDepthStencilResolveProperties::independentResolveNone is VK_FALSE, the resolveMode of both structures **must** be the same value
- VUID-VkRenderingInfo-pDepthAttachment-06105
  If pDepthAttachment or pStencilAttachment are both not NULL, pDepthAttachment->imageView and pStencilAttachment->imageView are both not VK_NULL_HANDLE, VkPhysicalDeviceDepthStencilResolveProperties::independentResolve is VK_FALSE, and the resolveMode of neither structure is VK_RESOLVE_MODE_NONE, the resolveMode of both structures **must** be the same value
- VUID-VkRenderingInfo-colorAttachmentCount-06106
  colorAttachmentCount **must** be less than or equal to VkPhysicalDeviceLimits::maxColorAttachments
- VUID-VkRenderingInfo-imageView-06107
  If the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain is not VK_NULL_HANDLE, and the fragmentDensityMapNonSubsampledImages feature is not enabled, valid imageView and resolveImageView members of pDepthAttachment, pStencilAttachment, and each element of pColorAttachments **must** be a VkImageView created with VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT
- VUID-VkRenderingInfo-imageView-06108
  If the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain is not VK_NULL_HANDLE, and viewMask is not 0, imageView **must** have a layerCount greater than or equal to the index of the most significant bit in viewMask
- VUID-VkRenderingInfo-imageView-06109

If the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain is not VK_NULL_HANDLE, and viewMask is 0, imageView **must** have a layerCount equal to 1

- VUID-VkRenderingInfo-pNext-06112
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0 and the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain is not VK_NULL_HANDLE, imageView **must** have a width greater than or equal to $\lceil \frac{renderArea_x + renderArea_{width}}{maxFragmentDensityTexelSize_{width}} \rceil$
- VUID-VkRenderingInfo-pNext-06114
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0 and the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain is not VK_NULL_HANDLE, imageView **must** have a height greater than or equal to $\lceil \frac{renderArea_y + renderArea_{height}}{maxFragmentDensityTexelSize_{height}} \rceil$
- VUID-VkRenderingInfo-pNext-06113
  If the pNext chain contains a VkDeviceGroupRenderPassBeginInfo structure, its deviceRenderAreaCount member is not 0, and the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain is not VK_NULL_HANDLE, imageView **must** have a width greater than or equal to $\lceil \frac{pDeviceRenderAreas_x + pDeviceRenderAreas_{width}}{maxFragmentDensityTexelSize_{width}} \rceil$ for each element of pDeviceRenderAreas
- VUID-VkRenderingInfo-pNext-06115
  If the pNext chain contains a VkDeviceGroupRenderPassBeginInfo structure, its deviceRenderAreaCount member is not 0, and the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain is not VK_NULL_HANDLE, imageView **must** have a height greater than or equal to $\lceil \frac{pDeviceRenderAreas_y + pDeviceRenderAreas_{height}}{maxFragmentDensityTexelSize_{height}} \rceil$ for each element of pDeviceRenderAreas
- VUID-VkRenderingInfo-imageView-06116
  If the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain is not VK_NULL_HANDLE, it **must** not be equal to the imageView or resolveImageView member of pDepthAttachment, pStencilAttachment, or any element of pColorAttachments
- VUID-VkRenderingInfo-pNext-06119
  If the maintenance7 feature is not enabled or the robustFragmentShadingRateAttachmentAccess limit is VK_FALSE or the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure was created with VkImageSubresourceRange::baseMipLevel greater than 0, the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, and the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, imageView **must** have a width greater than or equal to $\lceil \frac{renderArea_x + renderArea_{width}}{shadingRateAttachmentTexelSize_{width}} \rceil$
- VUID-VkRenderingInfo-pNext-06121
  If the maintenance7 feature is not enabled or the robustFragmentShadingRateAttachmentAccess limit is VK_FALSE or the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure was created with

VkImageSubresourceRange::baseMipLevel greater than 0, the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0 and the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, imageView **must** have a height greater than or equal to $\lceil \frac{renderArea_y + renderArea_{height}}{shadingRateAttachmentTexelSize_{height}} \rceil$

- VUID-VkRenderingInfo-pNext-06120
  If the maintenance7 feature is not enabled or the robustFragmentShadingRateAttachmentAccess limit is VK_FALSE or the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure was created with VkImageSubresourceRange::baseMipLevel greater than 0, the pNext chain contains a VkDeviceGroupRenderPassBeginInfo structure, its deviceRenderAreaCount member is not 0, and the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, imageView **must** have a width greater than or equal to $\lceil \frac{pDeviceRenderAreas_x + pDeviceRenderAreas_{width}}{shadingRateAttachmentTexelSize_{width}} \rceil$ for each element of pDeviceRenderAreas
- VUID-VkRenderingInfo-pNext-06122
  If the maintenance7 feature is not enabled or the robustFragmentShadingRateAttachmentAccess limit is VK_FALSE or the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure was created with VkImageSubresourceRange::baseMipLevel greater than 0, the pNext chain contains a VkDeviceGroupRenderPassBeginInfo structure, its deviceRenderAreaCount member is not 0, and the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, imageView **must** have a height greater than or equal to $\lceil \frac{pDeviceRenderAreas_y + pDeviceRenderAreas_{height}}{shadingRateAttachmentTexelSize_{height}} \rceil$ for each element of pDeviceRenderAreas
- VUID-VkRenderingInfo-layerCount-07817
  layerCount **must** be less than or equal to maxFramebufferLayers
- VUID-VkRenderingInfo-imageView-06123
  If the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, and viewMask is 0, imageView **must** have a layerCount that is either equal to 1 or greater than or equal to layerCount
- VUID-VkRenderingInfo-imageView-06124
  If the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, and viewMask is not 0, imageView **must** have a layerCount that either equal to 1 or greater than or equal to the index of the most significant bit in viewMask
- VUID-VkRenderingInfo-imageView-06125
  If the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, it **must** not be equal to the imageView or resolveImageView member of pDepthAttachment, pStencilAttachment, or any element of pColorAttachments
- VUID-VkRenderingInfo-imageView-06126
  If the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, it **must** not be equal to

the imageView member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the pNext chain

- VUID-VkRenderingInfo-multiview-06127
  If the multiview feature is not enabled, viewMask **must** be 0
- VUID-VkRenderingInfo-viewMask-06128
  The index of the most significant bit in viewMask **must** be less than maxMultiviewViewCount
- VUID-VkRenderingInfo-None-09044
  Valid attachments specified by this structure **must** not be bound to memory locations that are bound to any other valid attachments specified by this structure
- VUID-VkRenderingInfo-flags-10012
  If flags includes VK_RENDERING_CONTENTS_INLINE_BIT_KHR then at least one of the following features **must** be enabled
  - maintenance7
  - nestedCommandBuffer
- VUID-VkRenderingInfo-colorAttachmentCount-09479
  If colorAttachmentCount is not 0 and the imageView member of an element of pColorAttachments is not VK_NULL_HANDLE, that imageView **must** have been created with the identity swizzle
- VUID-VkRenderingInfo-colorAttachmentCount-09480
  If colorAttachmentCount is not 0, and there is an element of pColorAttachments with its imageView member not set to VK_NULL_HANDLE and its resolveMode member not set to VK_RESOLVE_MODE_NONE, the resolveImageView member of that element of pColorAttachments **must** have been created with the identity swizzle
- VUID-VkRenderingInfo-pDepthAttachment-09481
  If pDepthAttachment is not NULL and pDepthAttachment->imageView is not VK_NULL_HANDLE, pDepthAttachment->imageView **must** have been created with the identity swizzle
- VUID-VkRenderingInfo-pDepthAttachment-09482
  If pDepthAttachment is not NULL, pDepthAttachment->imageView is not VK_NULL_HANDLE, and pDepthAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pDepthAttachment->resolveImageView **must** have been created with the identity swizzle
- VUID-VkRenderingInfo-pStencilAttachment-09483
  If pStencilAttachment is not NULL and pStencilAttachment->imageView is not VK_NULL_HANDLE, pStencilAttachment->imageView **must** have been created with the identity swizzle
- VUID-VkRenderingInfo-pStencilAttachment-09484
  If pStencilAttachment is not NULL, pStencilAttachment->imageView is not VK_NULL_HANDLE, and pStencilAttachment->resolveMode is not VK_RESOLVE_MODE_NONE, pStencilAttachment->resolveImageView **must** have been created with the identity swizzle
- VUID-VkRenderingInfo-imageView-09485
  If the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure included in the pNext chain is not VK_NULL_HANDLE, it **must** have been created with the identity swizzle

- VUID-VkRenderingInfo-imageView-09486
  If the `imageView` member of a VkRenderingFragmentDensityMapAttachmentInfoEXT structure included in the `pNext` chain is not VK_NULL_HANDLE, it **must** have been created with the identity swizzle

Valid Usage (Implicit)

- VUID-VkRenderingInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDERING_INFO`
- VUID-VkRenderingInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkDeviceGroupRenderPassBeginInfo, VkRenderingFragmentDensityMapAttachmentInfoEXT, or VkRenderingFragmentShadingRateAttachmentInfoKHR
- VUID-VkRenderingInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkRenderingInfo-flags-parameter
  `flags` **must** be a valid combination of VkRenderingFlagBits values
- VUID-VkRenderingInfo-pColorAttachments-parameter
  If `colorAttachmentCount` is not `0`, `pColorAttachments` **must** be a valid pointer to an array of `colorAttachmentCount` valid VkRenderingAttachmentInfo structures
- VUID-VkRenderingInfo-pDepthAttachment-parameter
  If `pDepthAttachment` is not `NULL`, `pDepthAttachment` **must** be a valid pointer to a valid VkRenderingAttachmentInfo structure
- VUID-VkRenderingInfo-pStencilAttachment-parameter
  If `pStencilAttachment` is not `NULL`, `pStencilAttachment` **must** be a valid pointer to a valid VkRenderingAttachmentInfo structure

Bits which **can** be set in VkRenderingInfo::`flags` describing additional properties of the render pass are:

```
typedef enum VkRenderingFlagBits {
    VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT = 0x00000001,
    VK_RENDERING_SUSPENDING_BIT = 0x00000002,
    VK_RENDERING_RESUMING_BIT = 0x00000004,
  // Provided by VK_KHR_maintenance7
    VK_RENDERING_CONTENTS_INLINE_BIT_KHR = 0x00000010,
    VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT_KHR =
VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT,
    VK_RENDERING_SUSPENDING_BIT_KHR = VK_RENDERING_SUSPENDING_BIT,
    VK_RENDERING_RESUMING_BIT_KHR = VK_RENDERING_RESUMING_BIT,
  // Provided by VK_EXT_nested_command_buffer
    VK_RENDERING_CONTENTS_INLINE_BIT_EXT = VK_RENDERING_CONTENTS_INLINE_BIT_KHR,
} VkRenderingFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_dynamic_rendering
typedef VkRenderingFlagBits VkRenderingFlagBitsKHR;
```

- VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT specifies that draw calls for the render pass instance will be recorded in secondary command buffers. If the nestedCommandBuffer feature is enabled, the draw calls **can** come from both inline and vkCmdExecuteCommands.
- VK_RENDERING_RESUMING_BIT specifies that the render pass instance is resuming an earlier suspended render pass instance.
- VK_RENDERING_SUSPENDING_BIT specifies that the render pass instance will be suspended.
- VK_RENDERING_CONTENTS_INLINE_BIT_KHR specifies that draw calls for the render pass instance **can** be recorded inline within the current command buffer. This **can** be combined with the VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT bit to allow draw calls to be recorded both inline and in secondary command buffers.

The contents of pRenderingInfo **must** match between suspended render pass instances and the render pass instances that resume them, other than the presence or absence of the VK_RENDERING_RESUMING_BIT, VK_RENDERING_SUSPENDING_BIT, and VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT flags. No action or synchronization commands, or other render pass instances, are allowed between suspending and resuming render pass instances.

```
typedef VkFlags VkRenderingFlags;
```

or the equivalent

```
// Provided by VK_KHR_dynamic_rendering
typedef VkRenderingFlags VkRenderingFlagsKHR;
```

VkRenderingFlags is a bitmask type for setting a mask of zero or more VkRenderingFlagBits.

The VkRenderingAttachmentInfo structure is defined as:

```
typedef struct VkRenderingAttachmentInfo {
    VkStructureType          sType;
    const void*              pNext;
    VkImageView              imageView;
    VkImageLayout            imageLayout;
    VkResolveModeFlagBits    resolveMode;
    VkImageView              resolveImageView;
    VkImageLayout            resolveImageLayout;
    VkAttachmentLoadOp       loadOp;
    VkAttachmentStoreOp      storeOp;
    VkClearValue             clearValue;
} VkRenderingAttachmentInfo;
```

or the equivalent

```
// Provided by VK_KHR_dynamic_rendering
typedef VkRenderingAttachmentInfo VkRenderingAttachmentInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- imageView is the image view that will be used for rendering.
- imageLayout is the layout that imageView will be in during rendering.
- resolveMode is a VkResolveModeFlagBits value defining how data written to imageView will be resolved into resolveImageView.
- resolveImageView is an image view used to write resolved data at the end of rendering.
- resolveImageLayout is the layout that resolveImageView will be in during rendering.
- loadOp is a VkAttachmentLoadOp value defining the load operation for the attachment.
- storeOp is a VkAttachmentStoreOp value defining the store operation for the attachment.
- clearValue is a VkClearValue structure defining values used to clear imageView when loadOp is VK_ATTACHMENT_LOAD_OP_CLEAR.

Values in imageView are loaded and stored according to the values of loadOp and storeOp, within the render area for each device specified in VkRenderingInfo. If imageView is VK_NULL_HANDLE, other members of this structure are ignored; writes to this attachment will be discarded, and no load, store, or multisample resolve operations will be performed.

If resolveMode is VK_RESOLVE_MODE_NONE, then resolveImageView is ignored. If resolveMode is not VK_RESOLVE_MODE_NONE, and resolveImageView is not VK_NULL_HANDLE, a render pass multisample resolve operation is defined for the attachment subresource.

NOTE	The resolve mode and store operation are independent; it is valid to write both resolved and unresolved values, and equally valid to discard the unresolved values while writing the resolved ones.

Store and resolve operations are only performed at the end of a render pass instance that does not specify the VK_RENDERING_SUSPENDING_BIT_KHR flag.

Load operations are only performed at the beginning of a render pass instance that does not specify the VK_RENDERING_RESUMING_BIT_KHR flag.

Image contents at the end of a suspended render pass instance remain defined for access by a resuming render pass instance.

Valid Usage

- VUID-VkRenderingAttachmentInfo-imageView-06129
  If imageView is not VK_NULL_HANDLE and has a non-integer color format, resolveMode **must** be VK_RESOLVE_MODE_NONE or VK_RESOLVE_MODE_AVERAGE_BIT
- VUID-VkRenderingAttachmentInfo-imageView-06130
  If imageView is not VK_NULL_HANDLE and has an integer color format, resolveMode **must** be VK_RESOLVE_MODE_NONE or VK_RESOLVE_MODE_SAMPLE_ZERO_BIT
- VUID-VkRenderingAttachmentInfo-imageView-06861
  imageView **must** not have a sample count of VK_SAMPLE_COUNT_1_BIT if all of the following hold:
  - imageView is not VK_NULL_HANDLE
  - resolveMode is not VK_RESOLVE_MODE_NONE
- VUID-VkRenderingAttachmentInfo-imageView-06862
  resolveImageView **must** not be VK_NULL_HANDLE if all of the following hold:
  - imageView is not VK_NULL_HANDLE
  - resolveMode is not VK_RESOLVE_MODE_NONE
- VUID-VkRenderingAttachmentInfo-imageView-06864
  If imageView is not VK_NULL_HANDLE, resolveImageView is not VK_NULL_HANDLE, and resolveMode is not VK_RESOLVE_MODE_NONE, resolveImageView **must** have a sample count of VK_SAMPLE_COUNT_1_BIT
- VUID-VkRenderingAttachmentInfo-imageView-06865
  If imageView is not VK_NULL_HANDLE, resolveImageView is not VK_NULL_HANDLE, and resolveMode is not VK_RESOLVE_MODE_NONE, imageView and resolveImageView **must** have the same VkFormat
- VUID-VkRenderingAttachmentInfo-imageView-06135
  If imageView is not VK_NULL_HANDLE, imageLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, or VK_IMAGE_LAYOUT_PREINITIALIZED
- VUID-VkRenderingAttachmentInfo-imageView-06136

If imageView is not VK_NULL_HANDLE and resolveMode is not VK_RESOLVE_MODE_NONE, resolveImageLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, or VK_IMAGE_LAYOUT_PREINITIALIZED

- VUID-VkRenderingAttachmentInfo-imageView-06137
  If imageView is not VK_NULL_HANDLE and resolveMode is not VK_RESOLVE_MODE_NONE, resolveImageLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderingAttachmentInfo-imageView-06140
  If imageView is not VK_NULL_HANDLE, imageLayout **must** not be VK_IMAGE_LAYOUT_FRAGMENT_DENSITY_MAP_OPTIMAL_EXT
- VUID-VkRenderingAttachmentInfo-imageView-06141
  If imageView is not VK_NULL_HANDLE and resolveMode is not VK_RESOLVE_MODE_NONE, resolveImageLayout **must** not be VK_IMAGE_LAYOUT_FRAGMENT_DENSITY_MAP_OPTIMAL_EXT
- VUID-VkRenderingAttachmentInfo-imageView-06142
  If imageView is not VK_NULL_HANDLE and resolveMode is not VK_RESOLVE_MODE_NONE, resolveImageLayout **must** not be VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkRenderingAttachmentInfo-imageView-06143
  If imageView is not VK_NULL_HANDLE, imageLayout **must** not be VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR
- VUID-VkRenderingAttachmentInfo-imageView-06144
  If imageView is not VK_NULL_HANDLE and resolveMode is not VK_RESOLVE_MODE_NONE, resolveImageLayout **must** not be VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR
- VUID-VkRenderingAttachmentInfo-imageView-06145
  If imageView is not VK_NULL_HANDLE, imageLayout **must** not be VK_IMAGE_LAYOUT_PRESENT_SRC_KHR
- VUID-VkRenderingAttachmentInfo-imageView-06146
  If imageView is not VK_NULL_HANDLE and resolveMode is not VK_RESOLVE_MODE_NONE, resolveImageLayout **must** not be VK_IMAGE_LAYOUT_PRESENT_SRC_KHR

Valid Usage (Implicit)

- VUID-VkRenderingAttachmentInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO
- VUID-VkRenderingAttachmentInfo-pNext-pNext
  pNext **must** be NULL
- VUID-VkRenderingAttachmentInfo-imageView-parameter
  If imageView is not VK_NULL_HANDLE, imageView **must** be a valid VkImageView handle
- VUID-VkRenderingAttachmentInfo-imageLayout-parameter
  imageLayout **must** be a valid VkImageLayout value

- VUID-VkRenderingAttachmentInfo-resolveMode-parameter
  If `resolveMode` is not `0`, `resolveMode` **must** be a valid VkResolveModeFlagBits value
- VUID-VkRenderingAttachmentInfo-resolveImageView-parameter
  If `resolveImageView` is not VK_NULL_HANDLE, `resolveImageView` **must** be a valid VkImageView handle
- VUID-VkRenderingAttachmentInfo-resolveImageLayout-parameter
  `resolveImageLayout` **must** be a valid VkImageLayout value
- VUID-VkRenderingAttachmentInfo-loadOp-parameter
  `loadOp` **must** be a valid VkAttachmentLoadOp value
- VUID-VkRenderingAttachmentInfo-storeOp-parameter
  `storeOp` **must** be a valid VkAttachmentStoreOp value
- VUID-VkRenderingAttachmentInfo-clearValue-parameter
  `clearValue` **must** be a valid VkClearValue union
- VUID-VkRenderingAttachmentInfo-commonparent
  Both of `imageView`, and `resolveImageView` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The `VkRenderingFragmentShadingRateAttachmentInfoKHR` structure is defined as:

```
// Provided by VK_KHR_fragment_shading_rate with VK_VERSION_1_3 or
VK_KHR_dynamic_rendering
typedef struct VkRenderingFragmentShadingRateAttachmentInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkImageView        imageView;
    VkImageLayout      imageLayout;
    VkExtent2D         shadingRateAttachmentTexelSize;
} VkRenderingFragmentShadingRateAttachmentInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `imageView` is the image view that will be used as a fragment shading rate attachment.
- `imageLayout` is the layout that `imageView` will be in during rendering.
- `shadingRateAttachmentTexelSize` specifies the number of pixels corresponding to each texel in `imageView`.

This structure can be included in the `pNext` chain of VkRenderingInfo to define a fragment shading rate attachment. If `imageView` is VK_NULL_HANDLE, or if this structure is not specified, the implementation behaves as if a valid shading rate attachment was specified with all texels specifying a single pixel per fragment.

Valid Usage

- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06147
  If `imageView` is not `VK_NULL_HANDLE`, `layout` **must** be `VK_IMAGE_LAYOUT_GENERAL` or `VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR`
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06148
  If `imageView` is not `VK_NULL_HANDLE`, it **must** have been created with `VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06149
  If `imageView` is not `VK_NULL_HANDLE`, `shadingRateAttachmentTexelSize.width` **must** be a power of two value
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06150
  If `imageView` is not `VK_NULL_HANDLE`, `shadingRateAttachmentTexelSize.width` **must** be less than or equal to `maxFragmentShadingRateAttachmentTexelSize.width`
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06151
  If `imageView` is not `VK_NULL_HANDLE`, `shadingRateAttachmentTexelSize.width` **must** be greater than or equal to `minFragmentShadingRateAttachmentTexelSize.width`
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06152
  If `imageView` is not `VK_NULL_HANDLE`, `shadingRateAttachmentTexelSize.height` **must** be a power of two value
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06153
  If `imageView` is not `VK_NULL_HANDLE`, `shadingRateAttachmentTexelSize.height` **must** be less than or equal to `maxFragmentShadingRateAttachmentTexelSize.height`
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06154
  If `imageView` is not `VK_NULL_HANDLE`, `shadingRateAttachmentTexelSize.height` **must** be greater than or equal to `minFragmentShadingRateAttachmentTexelSize.height`
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06155
  If `imageView` is not `VK_NULL_HANDLE`, the quotient of `shadingRateAttachmentTexelSize.width` and `shadingRateAttachmentTexelSize.height` **must** be less than or equal to `maxFragmentShadingRateAttachmentTexelSizeAspectRatio`
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-06156
  If `imageView` is not `VK_NULL_HANDLE`, the quotient of `shadingRateAttachmentTexelSize.height` and `shadingRateAttachmentTexelSize.width` **must** be less than or equal to `maxFragmentShadingRateAttachmentTexelSizeAspectRatio`

Valid Usage (Implicit)

- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_INFO_KHR`
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageView-parameter
  If `imageView` is not `VK_NULL_HANDLE`, `imageView` **must** be a valid `VkImageView` handle
- VUID-VkRenderingFragmentShadingRateAttachmentInfoKHR-imageLayout-parameter

imageLayout **must** be a valid VkImageLayout value

The VkRenderingFragmentDensityMapAttachmentInfoEXT structure is defined as:

```
// Provided by VK_EXT_fragment_density_map with VK_VERSION_1_3 or
VK_KHR_dynamic_rendering
typedef struct VkRenderingFragmentDensityMapAttachmentInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkImageView        imageView;
    VkImageLayout      imageLayout;
} VkRenderingFragmentDensityMapAttachmentInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- imageView is the image view that will be used as a fragment density map attachment.
- imageLayout is the layout that imageView will be in during rendering.

This structure can be included in the pNext chain of VkRenderingInfo to define a fragment density map. If this structure is not included in the pNext chain, imageView is treated as VK_NULL_HANDLE.

Valid Usage

- VUID-VkRenderingFragmentDensityMapAttachmentInfoEXT-imageView-06157
  If imageView is not VK_NULL_HANDLE, imageLayout **must** be VK_IMAGE_LAYOUT_GENERAL or VK_IMAGE_LAYOUT_FRAGMENT_DENSITY_MAP_OPTIMAL_EXT
- VUID-VkRenderingFragmentDensityMapAttachmentInfoEXT-imageView-06158
  If imageView is not VK_NULL_HANDLE, it **must** have been created with VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT
- VUID-VkRenderingFragmentDensityMapAttachmentInfoEXT-imageView-06159
  If imageView is not VK_NULL_HANDLE, it **must** not have been created with VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT
- VUID-VkRenderingFragmentDensityMapAttachmentInfoEXT-apiVersion-07908
  If the multiview feature is not enabled, VkPhysicalDeviceProperties::apiVersion is less than Vulkan 1.1, and imageView is not VK_NULL_HANDLE, it **must** have a layerCount equal to 1

Valid Usage (Implicit)

- VUID-VkRenderingFragmentDensityMapAttachmentInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_INFO_EXT
- VUID-VkRenderingFragmentDensityMapAttachmentInfoEXT-imageView-parameter
  imageView **must** be a valid VkImageView handle

- VUID-VkRenderingFragmentDensityMapAttachmentInfoEXT-imageLayout-parameter
  `imageLayout` **must** be a valid VkImageLayout value

To query the render area granularity for a render pass instance, call:

```
// Provided by VK_KHR_maintenance5
void vkGetRenderingAreaGranularityKHR(
    VkDevice                                    device,
    const VkRenderingAreaInfoKHR*               pRenderingAreaInfo,
    VkExtent2D*                                 pGranularity);
```

- `device` is the logical device that owns the render pass instance.
- `pRenderingAreaInfo` is a pointer to a VkRenderingAreaInfoKHR structure specifying details of the render pass instance to query the render area granularity for.
- `pGranularity` is a pointer to a VkExtent2D structure in which the granularity is returned.

The conditions leading to an optimal `renderArea` are:

- the `offset.x` member in `renderArea` is a multiple of the `width` member of the returned VkExtent2D (the horizontal granularity).
- the `offset.y` member in `renderArea` is a multiple of the `height` member of the returned VkExtent2D (the vertical granularity).
- either the `extent.width` member in `renderArea` is a multiple of the horizontal granularity or `offset.x`+`extent.width` is equal to the `width` of each attachment used in the render pass instance.
- either the `extent.height` member in `renderArea` is a multiple of the vertical granularity or `offset.y`+`extent.height` is equal to the `height` of each attachment used in the render pass instance.

Valid Usage (Implicit)

- VUID-vkGetRenderingAreaGranularityKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetRenderingAreaGranularityKHR-pRenderingAreaInfo-parameter
  `pRenderingAreaInfo` **must** be a valid pointer to a valid VkRenderingAreaInfoKHR structure
- VUID-vkGetRenderingAreaGranularityKHR-pGranularity-parameter
  `pGranularity` **must** be a valid pointer to a VkExtent2D structure

The `VkRenderingAreaInfoKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance5
typedef struct VkRenderingAreaInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           viewMask;
    uint32_t           colorAttachmentCount;
    const VkFormat*    pColorAttachmentFormats;
    VkFormat           depthAttachmentFormat;
    VkFormat           stencilAttachmentFormat;
} VkRenderingAreaInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `viewMask` is the viewMask used for rendering.
- `colorAttachmentCount` is the number of entries in `pColorAttachmentFormats`
- `pColorAttachmentFormats` is a pointer to an array of VkFormat values defining the format of color attachments used in the render pass instance.
- `depthAttachmentFormat` is a VkFormat value defining the format of the depth attachment used in the render pass instance.
- `stencilAttachmentFormat` is a VkFormat value defining the format of the stencil attachment used in the render pass instance.

Valid Usage (Implicit)

- VUID-VkRenderingAreaInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDERING_AREA_INFO_KHR`
- VUID-VkRenderingAreaInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`

To end a render pass instance, call:

```
// Provided by VK_KHR_dynamic_rendering
void vkCmdEndRenderingKHR(
    VkCommandBuffer                             commandBuffer);
```

- `commandBuffer` is the command buffer in which to record the command.

If the value of `pRenderingInfo->flags` used to begin this render pass instance included `VK_RENDERING_SUSPENDING_BIT`, then this render pass is suspended and will be resumed later in submission order.

Valid Usage

- VUID-vkCmdEndRendering-None-06161
  The current render pass instance **must** have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdEndRendering-commandBuffer-06162
  The current render pass instance **must** have been begun in `commandBuffer`
- VUID-vkCmdEndRendering-None-06781
  This command **must** not be recorded when transform feedback is active
- VUID-vkCmdEndRendering-None-06999
  If `vkCmdBeginQuery*` was called within the render pass, the corresponding `vkCmdEndQuery*` **must** have been called subsequently within the same subpass

Valid Usage (Implicit)

- VUID-vkCmdEndRendering-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdEndRendering-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdEndRendering-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdEndRendering-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdEndRendering-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action State

NOTE

For more complex rendering graphs, it is possible to pre-define a static *render pass* object, which as well as allowing draw commands, allows the definition of framebuffer-local dependencies between multiple subpasses. These objects have a lot of setup cost compared to vkCmdBeginRenderingKHR, but use of subpass dependencies can confer important performance benefits on some devices.

8.1. Render Pass Objects

A render pass object represents a collection of attachments, subpasses, and dependencies between the subpasses, and describes how the attachments are used over the course of the subpasses.

Render passes are represented by `VkRenderPass` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkRenderPass)
```

An *attachment description* describes the properties of an attachment including its format, sample count, and how its contents are treated at the beginning and end of each render pass instance.

A *subpass* represents a phase of rendering that reads and writes a subset of the attachments in a render pass. Rendering commands are recorded into a particular subpass of a render pass instance.

A *subpass description* describes the subset of attachments that is involved in the execution of a subpass. Each subpass **can** read from some attachments as *input attachments*, write to some as *color attachments* or *depth/stencil attachments*, and perform *multisample resolve operations* to *resolve attachments*. A subpass description **can** also include a set of *preserve attachments*, which are attachments that are not read or written by the subpass but whose contents **must** be preserved throughout the subpass.

A subpass *uses an attachment* if the attachment is a color, depth/stencil, resolve, depth/stencil resolve, fragment shading rate, or input attachment for that subpass (as determined by the `pColorAttachments`, `pDepthStencilAttachment`, `pResolveAttachments`, VkSubpassDescriptionDepthStencilResolve::`pDepthStencilResolveAttachment`, VkFragmentShadingRateAttachmentInfoKHR::`pFragmentShadingRateAttachment->attachment`, and `pInputAttachments` members of VkSubpassDescription, respectively). A subpass does not use an attachment if that attachment is preserved by the subpass. The *first use of an attachment* is in the lowest numbered subpass that uses that attachment. Similarly, the *last use of an attachment* is in the highest numbered subpass that uses that attachment.

The subpasses in a render pass all render to the same dimensions, and fragments for pixel (x,y,layer) in one subpass **can** only read attachment contents written by previous subpasses at that same (x,y,layer) location. For multi-pixel fragments, the pixel read from an input attachment is selected from the pixels covered by that fragment in an implementation-dependent manner. However, this selection **must** be made consistently for any fragment with the same shading rate for the lifetime of the VkDevice.

NOTE

By describing a complete set of subpasses in advance, render passes provide the

> implementation an opportunity to optimize the storage and transfer of attachment data between subpasses.
>
> In practice, this means that subpasses with a simple framebuffer-space dependency **may** be merged into a single tiled rendering pass, keeping the attachment data on-chip for the duration of a render pass instance. However, it is also quite common for a render pass to only contain a single subpass.

Subpass dependencies describe execution and memory dependencies between subpasses.

A *subpass dependency chain* is a sequence of subpass dependencies in a render pass, where the source subpass of each subpass dependency (after the first) equals the destination subpass of the previous dependency.

Execution of subpasses **may** overlap or execute out of order with regards to other subpasses, unless otherwise enforced by an execution dependency. Each subpass only respects submission order for commands recorded in the same subpass, and the vkCmdBeginRenderPass and vkCmdEndRenderPass commands that delimit the render pass - commands within other subpasses are not included. This affects most other implicit ordering guarantees.

A render pass describes the structure of subpasses and attachments independent of any specific image views for the attachments. The specific image views that will be used for the attachments, and their dimensions, are specified in `VkFramebuffer` objects. Framebuffers are created with respect to a specific render pass that the framebuffer is compatible with (see Render Pass Compatibility). Collectively, a render pass and a framebuffer define the complete render target state for one or more subpasses as well as the algorithmic dependencies between the subpasses.

The various pipeline stages of the drawing commands for a given subpass **may** execute concurrently and/or out of order, both within and across drawing commands, whilst still respecting pipeline order. However for a given (x,y,layer,sample) sample location, certain per-sample operations are performed in rasterization order.

`VK_ATTACHMENT_UNUSED` is a constant indicating that a render pass attachment is not used.

```
#define VK_ATTACHMENT_UNUSED              (~0U)
```

8.2. Render Pass Creation

To create a render pass, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateRenderPass(
    VkDevice                                    device,
    const VkRenderPassCreateInfo*               pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkRenderPass*                               pRenderPass);
```

- `device` is the logical device that creates the render pass.
- `pCreateInfo` is a pointer to a VkRenderPassCreateInfo structure describing the parameters of the render pass.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pRenderPass` is a pointer to a VkRenderPass handle in which the resulting render pass object is returned.

Valid Usage

- VUID-vkCreateRenderPass-device-10000
  `device` **must** support at least one queue family with the `VK_QUEUE_GRAPHICS_BIT` capability

Valid Usage (Implicit)

- VUID-vkCreateRenderPass-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateRenderPass-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkRenderPassCreateInfo structure
- VUID-vkCreateRenderPass-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateRenderPass-pRenderPass-parameter
  `pRenderPass` **must** be a valid pointer to a VkRenderPass handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkRenderPassCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkRenderPassCreateInfo {
    VkStructureType                   sType;
    const void*                       pNext;
    VkRenderPassCreateFlags           flags;
    uint32_t                          attachmentCount;
    const VkAttachmentDescription*    pAttachments;
    uint32_t                          subpassCount;
    const VkSubpassDescription*       pSubpasses;
    uint32_t                          dependencyCount;
    const VkSubpassDependency*        pDependencies;
} VkRenderPassCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `attachmentCount` is the number of attachments used by this render pass.
- `pAttachments` is a pointer to an array of `attachmentCount` VkAttachmentDescription structures describing the attachments used by the render pass.
- `subpassCount` is the number of subpasses to create.
- `pSubpasses` is a pointer to an array of `subpassCount` VkSubpassDescription structures describing each subpass.
- `dependencyCount` is the number of memory dependencies between pairs of subpasses.
- `pDependencies` is a pointer to an array of `dependencyCount` VkSubpassDependency structures describing dependencies between pairs of subpasses.

NOTE Care should be taken to avoid a data race here; if any subpasses access attachments with overlapping memory locations, and one of those accesses is a write, a subpass dependency needs to be included between them.

Valid Usage

- VUID-VkRenderPassCreateInfo-attachment-00834
  If the `attachment` member of any element of `pInputAttachments`, `pColorAttachments`, `pResolveAttachments` or `pDepthStencilAttachment`, or any element of `pPreserveAttachments` in any element of `pSubpasses` is not `VK_ATTACHMENT_UNUSED`, then it **must** be less than `attachmentCount`
- VUID-VkRenderPassCreateInfo-fragmentDensityMapAttachment-06471
  If the pNext chain includes a VkRenderPassFragmentDensityMapCreateInfoEXT structure and the `fragmentDensityMapAttachment` member is not `VK_ATTACHMENT_UNUSED`, then `attachment` **must** be less than `attachmentCount`
- VUID-VkRenderPassCreateInfo-pAttachments-00836
  For any member of `pAttachments` with a `loadOp` equal to `VK_ATTACHMENT_LOAD_OP_CLEAR`, the

first use of that attachment **must** not specify a `layout` equal to `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`

- VUID-VkRenderPassCreateInfo-pAttachments-02511
  For any member of `pAttachments` with a `stencilLoadOp` equal to `VK_ATTACHMENT_LOAD_OP_CLEAR`, the first use of that attachment **must** not specify a `layout` equal to `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`
- VUID-VkRenderPassCreateInfo-pAttachments-01566
  For any member of `pAttachments` with a `loadOp` equal to `VK_ATTACHMENT_LOAD_OP_CLEAR`, the first use of that attachment **must** not specify a `layout` equal to `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`
- VUID-VkRenderPassCreateInfo-pAttachments-01567
  For any member of `pAttachments` with a `stencilLoadOp` equal to `VK_ATTACHMENT_LOAD_OP_CLEAR`, the first use of that attachment **must** not specify a `layout` equal to `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`
- VUID-VkRenderPassCreateInfo-pNext-01926
  If the `pNext` chain includes a VkRenderPassInputAttachmentAspectCreateInfo structure, the `subpass` member of each element of its `pAspectReferences` member **must** be less than `subpassCount`
- VUID-VkRenderPassCreateInfo-pNext-01927
  If the `pNext` chain includes a VkRenderPassInputAttachmentAspectCreateInfo structure, the `inputAttachmentIndex` member of each element of its `pAspectReferences` member **must** be less than the value of `inputAttachmentCount` in the element of `pSubpasses` identified by its `subpass` member
- VUID-VkRenderPassCreateInfo-pNext-01963
  If the `pNext` chain includes a VkRenderPassInputAttachmentAspectCreateInfo structure, for any element of the `pInputAttachments` member of any element of `pSubpasses` where the `attachment` member is not `VK_ATTACHMENT_UNUSED`, the `aspectMask` member of the corresponding element of VkRenderPassInputAttachmentAspectCreateInfo ::`pAspectReferences` **must** only include aspects that are present in images of the format specified by the element of `pAttachments` at `attachment`
- VUID-VkRenderPassCreateInfo-pNext-01928
  If the `pNext` chain includes a VkRenderPassMultiviewCreateInfo structure, and its `subpassCount` member is not zero, that member **must** be equal to the value of `subpassCount`
- VUID-VkRenderPassCreateInfo-pNext-01929
  If the `pNext` chain includes a VkRenderPassMultiviewCreateInfo structure, if its `dependencyCount` member is not zero, it **must** be equal to `dependencyCount`
- VUID-VkRenderPassCreateInfo-pNext-01930
  If the `pNext` chain includes a VkRenderPassMultiviewCreateInfo structure, for each non-zero element of `pViewOffsets`, the `srcSubpass` and `dstSubpass` members of `pDependencies` at the same index **must** not be equal
- VUID-VkRenderPassCreateInfo-pNext-02512
  If the `pNext` chain includes a VkRenderPassMultiviewCreateInfo structure, for any element

of pDependencies with a dependencyFlags member that does not include VK_DEPENDENCY_VIEW_LOCAL_BIT, the corresponding element of the pViewOffsets member of that VkRenderPassMultiviewCreateInfo instance **must** be 0

- VUID-VkRenderPassCreateInfo-pNext-02513
  If the pNext chain includes a VkRenderPassMultiviewCreateInfo structure, elements of its pViewMasks member **must** either all be 0, or all not be 0
- VUID-VkRenderPassCreateInfo-pNext-02514
  If the pNext chain includes a VkRenderPassMultiviewCreateInfo structure, and each element of its pViewMasks member is 0, the dependencyFlags member of each element of pDependencies **must** not include VK_DEPENDENCY_VIEW_LOCAL_BIT
- VUID-VkRenderPassCreateInfo-pNext-02515
  If the pNext chain includes a VkRenderPassMultiviewCreateInfo structure, and each element of its pViewMasks member is 0, its correlationMaskCount member **must** be 0
- VUID-VkRenderPassCreateInfo-pDependencies-00837
  For any element of pDependencies, if the srcSubpass is not VK_SUBPASS_EXTERNAL, all stage flags included in the srcStageMask member of that dependency **must** be a pipeline stage supported by the pipeline identified by the pipelineBindPoint member of the source subpass
- VUID-VkRenderPassCreateInfo-pDependencies-00838
  For any element of pDependencies, if the dstSubpass is not VK_SUBPASS_EXTERNAL, all stage flags included in the dstStageMask member of that dependency **must** be a pipeline stage supported by the pipeline identified by the pipelineBindPoint member of the destination subpass
- VUID-VkRenderPassCreateInfo-pDependencies-06866
  For any element of pDependencies, if its srcSubpass is not VK_SUBPASS_EXTERNAL, it **must** be less than subpassCount
- VUID-VkRenderPassCreateInfo-pDependencies-06867
  For any element of pDependencies, if its dstSubpass is not VK_SUBPASS_EXTERNAL, it **must** be less than subpassCount

Valid Usage (Implicit)

- VUID-VkRenderPassCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO
- VUID-VkRenderPassCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkRenderPassFragmentDensityMapCreateInfoEXT, VkRenderPassInputAttachmentAspectCreateInfo, or VkRenderPassMultiviewCreateInfo
- VUID-VkRenderPassCreateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkRenderPassCreateInfo-flags-zerobitmask
  flags **must** be 0

- VUID-VkRenderPassCreateInfo-pAttachments-parameter
  If `attachmentCount` is not `0`, `pAttachments` **must** be a valid pointer to an array of `attachmentCount` valid VkAttachmentDescription structures
- VUID-VkRenderPassCreateInfo-pSubpasses-parameter
  `pSubpasses` **must** be a valid pointer to an array of `subpassCount` valid VkSubpassDescription structures
- VUID-VkRenderPassCreateInfo-pDependencies-parameter
  If `dependencyCount` is not `0`, `pDependencies` **must** be a valid pointer to an array of `dependencyCount` valid VkSubpassDependency structures
- VUID-VkRenderPassCreateInfo-subpassCount-arraylength
  `subpassCount` **must** be greater than `0`

Bits which **can** be set in VkRenderPassCreateInfo::`flags`, describing additional properties of the render pass, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkRenderPassCreateFlagBits {
} VkRenderPassCreateFlagBits;
```

NOTE All bits for this type are defined by extensions, and none of those extensions are enabled in this build of the specification.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkRenderPassCreateFlags;
```

`VkRenderPassCreateFlags` is a bitmask type for setting a mask of zero or more VkRenderPassCreateFlagBits.

If the VkRenderPassCreateInfo::`pNext` chain includes a `VkRenderPassMultiviewCreateInfo` structure, then that structure includes an array of view masks, view offsets, and correlation masks for the render pass.

The `VkRenderPassMultiviewCreateInfo` structure is defined as:

```
typedef struct VkRenderPassMultiviewCreateInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           subpassCount;
    const uint32_t*    pViewMasks;
    uint32_t           dependencyCount;
    const int32_t*     pViewOffsets;
    uint32_t           correlationMaskCount;
    const uint32_t*    pCorrelationMasks;
} VkRenderPassMultiviewCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_multiview
typedef VkRenderPassMultiviewCreateInfo VkRenderPassMultiviewCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `subpassCount` is zero or the number of subpasses in the render pass.
- `pViewMasks` is a pointer to an array of `subpassCount` view masks, where each mask is a bitfield of view indices describing which views rendering is broadcast to in each subpass, when multiview is enabled. If `subpassCount` is zero, each view mask is treated as zero.
- `dependencyCount` is zero or the number of dependencies in the render pass.
- `pViewOffsets` is a pointer to an array of `dependencyCount` view offsets, one for each dependency. If `dependencyCount` is zero, each dependency's view offset is treated as zero. Each view offset controls which views in the source subpass the views in the destination subpass depend on.
- `correlationMaskCount` is zero or the number of correlation masks.
- `pCorrelationMasks` is a pointer to an array of `correlationMaskCount` view masks indicating sets of views that **may** be more efficient to render concurrently.

When a subpass uses a non-zero view mask, *multiview* functionality is considered to be enabled. Multiview is all-or-nothing for a render pass - that is, either all subpasses **must** have a non-zero view mask (though some subpasses **may** have only one view) or all **must** be zero. Multiview causes all drawing and clear commands in the subpass to behave as if they were broadcast to each view, where a view is represented by one layer of the framebuffer attachments. All draws and clears are broadcast to each *view index* whose bit is set in the view mask. The view index is provided in the `ViewIndex` shader input variable, and color, depth/stencil, and input attachments all read/write the layer of the framebuffer corresponding to the view index.

If the view mask is zero for all subpasses, multiview is considered to be disabled and all drawing commands execute normally, without this additional broadcasting.

Some implementations **may** not support multiview in conjunction with geometry shaders or tessellation shaders.

When multiview is enabled, the `VK_DEPENDENCY_VIEW_LOCAL_BIT` bit in a dependency **can** be used to express a view-local dependency, meaning that each view in the destination subpass depends on a single view in the source subpass. Unlike pipeline barriers, a subpass dependency **can** potentially have a different view mask in the source subpass and the destination subpass. If the dependency is view-local, then each view (dstView) in the destination subpass depends on the view dstView + `pViewOffsets`[dependency] in the source subpass. If there is not such a view in the source subpass, then this dependency does not affect that view in the destination subpass. If the dependency is not view-local, then all views in the destination subpass depend on all views in the source subpass, and the view offset is ignored. A non-zero view offset is not allowed in a self-dependency.

The elements of `pCorrelationMasks` are a set of masks of views indicating that views in the same

mask **may** exhibit spatial coherency between the views, making it more efficient to render them concurrently. Correlation masks **must** not have a functional effect on the results of the multiview rendering.

When multiview is enabled, at the beginning of each subpass all non-render pass state is undefined. In particular, each time vkCmdBeginRenderPass or vkCmdNextSubpass is called the graphics pipeline **must** be bound, any relevant descriptor sets or vertex/index buffers **must** be bound, and any relevant dynamic state or push constants **must** be set before they are used.

Valid Usage

- VUID-VkRenderPassMultiviewCreateInfo-pCorrelationMasks-00841
  Each view index **must** not be set in more than one element of `pCorrelationMasks`
- VUID-VkRenderPassMultiviewCreateInfo-multiview-06555
  If the `multiview` feature is not enabled, each element of `pViewMasks` **must** be `0`
- VUID-VkRenderPassMultiviewCreateInfo-pViewMasks-06697
  The index of the most significant bit in each element of `pViewMasks` **must** be less than `maxMultiviewViewCount`

Valid Usage (Implicit)

- VUID-VkRenderPassMultiviewCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDER_PASS_MULTIVIEW_CREATE_INFO`
- VUID-VkRenderPassMultiviewCreateInfo-pViewMasks-parameter
  If `subpassCount` is not `0`, `pViewMasks` **must** be a valid pointer to an array of `subpassCount` `uint32_t` values
- VUID-VkRenderPassMultiviewCreateInfo-pViewOffsets-parameter
  If `dependencyCount` is not `0`, `pViewOffsets` **must** be a valid pointer to an array of `dependencyCount` `int32_t` values
- VUID-VkRenderPassMultiviewCreateInfo-pCorrelationMasks-parameter
  If `correlationMaskCount` is not `0`, `pCorrelationMasks` **must** be a valid pointer to an array of `correlationMaskCount` `uint32_t` values

If the VkRenderPassCreateInfo::`pNext` chain includes a `VkRenderPassFragmentDensityMapCreateInfoEXT` structure, then that structure includes a fragment density map attachment for the render pass.

The `VkRenderPassFragmentDensityMapCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_fragment_density_map
typedef struct VkRenderPassFragmentDensityMapCreateInfoEXT {
    VkStructureType          sType;
    const void*              pNext;
    VkAttachmentReference    fragmentDensityMapAttachment;
} VkRenderPassFragmentDensityMapCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fragmentDensityMapAttachment` is the fragment density map to use for the render pass.

The fragment density map is read at an implementation-dependent time with the following constraints determined by the attachment's image view `flags`:

- `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DYNAMIC_BIT_EXT` specifies that the fragment density map will be read by the device during `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DEFERRED_BIT_EXT` specifies that the fragment density map will be read by the host during vkEndCommandBuffer of the primary command buffer that the render pass is recorded into
- Otherwise the fragment density map will be read by the host during vkCmdBeginRenderPass

The fragment density map **may** additionally be read by the device during `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT` for any mode.

If this structure is not present, it is as if `fragmentDensityMapAttachment` was given as `VK_ATTACHMENT_UNUSED`.

Valid Usage

- VUID-VkRenderPassFragmentDensityMapCreateInfoEXT-fragmentDensityMapAttachment-02548
  If `fragmentDensityMapAttachment` is not `VK_ATTACHMENT_UNUSED`, `fragmentDensityMapAttachment` **must** not be an element of `VkSubpassDescription`::`pInputAttachments`, `VkSubpassDescription`::`pColorAttachments`, `VkSubpassDescription`::`pResolveAttachments`, `VkSubpassDescription`::`pDepthStencilAttachment`, or `VkSubpassDescription`::`pPreserveAttachments` for any subpass
- VUID-VkRenderPassFragmentDensityMapCreateInfoEXT-fragmentDensityMapAttachment-02549
  If `fragmentDensityMapAttachment` is not `VK_ATTACHMENT_UNUSED`, `layout` **must** be equal to `VK_IMAGE_LAYOUT_FRAGMENT_DENSITY_MAP_OPTIMAL_EXT`, or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-VkRenderPassFragmentDensityMapCreateInfoEXT-fragmentDensityMapAttachment-02550
  If `fragmentDensityMapAttachment` is not `VK_ATTACHMENT_UNUSED`, `fragmentDensityMapAttachment` **must** reference an attachment with a `loadOp` equal to `VK_ATTACHMENT_LOAD_OP_LOAD` or `VK_ATTACHMENT_LOAD_OP_DONT_CARE`

- VUID-VkRenderPassFragmentDensityMapCreateInfoEXT-fragmentDensityMapAttachment-02551
  If `fragmentDensityMapAttachment` is not `VK_ATTACHMENT_UNUSED`, `fragmentDensityMapAttachment` **must** reference an attachment with a `storeOp` equal to `VK_ATTACHMENT_STORE_OP_DONT_CARE`

Valid Usage (Implicit)

- VUID-VkRenderPassFragmentDensityMapCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDER_PASS_FRAGMENT_DENSITY_MAP_CREATE_INFO_EXT`
- VUID-VkRenderPassFragmentDensityMapCreateInfoEXT-fragmentDensityMapAttachment-parameter
  `fragmentDensityMapAttachment` **must** be a valid VkAttachmentReference structure

The `VkAttachmentDescription` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkAttachmentDescription {
    VkAttachmentDescriptionFlags    flags;
    VkFormat                        format;
    VkSampleCountFlagBits           samples;
    VkAttachmentLoadOp              loadOp;
    VkAttachmentStoreOp             storeOp;
    VkAttachmentLoadOp              stencilLoadOp;
    VkAttachmentStoreOp             stencilStoreOp;
    VkImageLayout                   initialLayout;
    VkImageLayout                   finalLayout;
} VkAttachmentDescription;
```

- `flags` is a bitmask of VkAttachmentDescriptionFlagBits specifying additional properties of the attachment.
- `format` is a VkFormat value specifying the format of the image view that will be used for the attachment.
- `samples` is a VkSampleCountFlagBits value specifying the number of samples of the image.
- `loadOp` is a VkAttachmentLoadOp value specifying how the contents of color and depth components of the attachment are treated at the beginning of the subpass where it is first used.
- `storeOp` is a VkAttachmentStoreOp value specifying how the contents of color and depth components of the attachment are treated at the end of the subpass where it is last used.
- `stencilLoadOp` is a VkAttachmentLoadOp value specifying how the contents of stencil components of the attachment are treated at the beginning of the subpass where it is first used.
- `stencilStoreOp` is a VkAttachmentStoreOp value specifying how the contents of stencil components of the attachment are treated at the end of the last subpass where it is used.
- `initialLayout` is the layout the attachment image subresource will be in when a render pass instance begins.

- `finalLayout` is the layout the attachment image subresource will be transitioned to when a render pass instance ends.

If the attachment uses a color format, then `loadOp` and `storeOp` are used, and `stencilLoadOp` and `stencilStoreOp` are ignored. If the format has depth and/or stencil components, `loadOp` and `storeOp` apply only to the depth data, while `stencilLoadOp` and `stencilStoreOp` define how the stencil data is handled. `loadOp` and `stencilLoadOp` define the load operations for the attachment. `storeOp` and `stencilStoreOp` define the store operations for the attachment. If an attachment is not used by any subpass, `loadOp`, `storeOp`, `stencilStoreOp`, and `stencilLoadOp` will be ignored for that attachment, and no load or store ops will be performed. However, any transition specified by `initialLayout` and `finalLayout` will still be executed.

If `flags` includes `VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT`, then the attachment is treated as if it shares physical memory with another attachment in the same render pass. This information limits the ability of the implementation to reorder certain operations (like layout transitions and the `loadOp`) such that it is not improperly reordered against other uses of the same physical memory via a different attachment. This is described in more detail below.

If a render pass uses multiple attachments that alias the same device memory, those attachments **must** each include the `VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT` bit in their attachment description flags. Attachments aliasing the same memory occurs in multiple ways:

- Multiple attachments being assigned the same image view as part of framebuffer creation.
- Attachments using distinct image views that correspond to the same image subresource of an image.
- Attachments using views of distinct image subresources which are bound to overlapping memory ranges.

NOTE

Render passes **must** include subpass dependencies (either directly or via a subpass dependency chain) between any two subpasses that operate on the same attachment or aliasing attachments and those subpass dependencies **must** include execution and memory dependencies separating uses of the aliases, if at least one of those subpasses writes to one of the aliases. These dependencies **must** not include the `VK_DEPENDENCY_BY_REGION_BIT` if the aliases are views of distinct image subresources which overlap in memory.

Multiple attachments that alias the same memory **must** not be used in a single subpass. A given attachment index **must** not be used multiple times in a single subpass, with one exception: two subpass attachments **can** use the same attachment index if at least one use is as an input attachment and neither use is as a resolve or preserve attachment. In other words, the same view **can** be used simultaneously as an input and color or depth/stencil attachment, but **must** not be used as multiple color or depth/stencil attachments nor as resolve or preserve attachments.

If a set of attachments alias each other, then all except the first to be used in the render pass **must** use an `initialLayout` of `VK_IMAGE_LAYOUT_UNDEFINED`, since the earlier uses of the other aliases make their contents undefined. Once an alias has been used and a different alias has been used after it, the first alias **must** not be used in any later subpasses. However, an application **can** assign the same image view to multiple aliasing attachment indices, which allows that image view to be used

multiple times even if other aliases are used in between.

NOTE

Once an attachment needs the VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT bit, there **should** be no additional cost of introducing additional aliases, and using these additional aliases **may** allow more efficient clearing of the attachments on multiple uses via VK_ATTACHMENT_LOAD_OP_CLEAR.

Valid Usage

- VUID-VkAttachmentDescription-format-06699
  If format includes a color or depth component and loadOp is VK_ATTACHMENT_LOAD_OP_LOAD, then initialLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED
- VUID-VkAttachmentDescription-finalLayout-00843
  finalLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED
- VUID-VkAttachmentDescription-format-03280
  If format is a color format, initialLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription-format-03281
  If format is a depth/stencil format, initialLayout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescription-format-03282
  If format is a color format, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription-format-03283
  If format is a depth/stencil format, finalLayout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescription-format-06487
  If format is a color format, initialLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescription-format-06488
  If format is a color format, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescription-separateDepthStencilLayouts-03284
  If the separateDepthStencilLayouts feature is not enabled, initialLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL,
- VUID-VkAttachmentDescription-separateDepthStencilLayouts-03285
  If the separateDepthStencilLayouts feature is not enabled, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL,

VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL,

- VUID-VkAttachmentDescription-format-03286
  If format is a color format, initialLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription-format-03287
  If format is a color format, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription-format-06906
  If format is a depth/stencil format which includes both depth and stencil components, initialLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription-format-06907
  If format is a depth/stencil format which includes both depth and stencil components, finalLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription-format-03290
  If format is a depth/stencil format which includes only the depth component, initialLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription-format-03291
  If format is a depth/stencil format which includes only the depth component, finalLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription-synchronization2-06908
  If the synchronization2 feature is not enabled, initialLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR or VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkAttachmentDescription-synchronization2-06909
  If the synchronization2 feature is not enabled, finalLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR or VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkAttachmentDescription-attachmentFeedbackLoopLayout-07309
  If the attachmentFeedbackLoopLayout feature is not enabled, initialLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT
- VUID-VkAttachmentDescription-attachmentFeedbackLoopLayout-07310
  If the attachmentFeedbackLoopLayout feature is not enabled, finalLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT
- VUID-VkAttachmentDescription-samples-08745
  samples **must** be a valid VkSampleCountFlagBits value that is set in imageCreateSampleCounts (as defined in Image Creation Limits) for the given format

- VUID-VkAttachmentDescription-dynamicRenderingLocalRead-09544
  If the `dynamicRenderingLocalRead` feature is not enabled, `initialLayout` **must** not be `VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR`
- VUID-VkAttachmentDescription-dynamicRenderingLocalRead-09545
  If the `dynamicRenderingLocalRead` feature is not enabled, `finalLayout` **must** not be `VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR`
- VUID-VkAttachmentDescription-format-06698
  `format` **must** not be VK_FORMAT_UNDEFINED
- VUID-VkAttachmentDescription-format-06700
  If `format` includes a stencil component and `stencilLoadOp` is `VK_ATTACHMENT_LOAD_OP_LOAD`, then `initialLayout` **must** not be `VK_IMAGE_LAYOUT_UNDEFINED`
- VUID-VkAttachmentDescription-format-03292
  If `format` is a depth/stencil format which includes only the stencil component, `initialLayout` **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`
- VUID-VkAttachmentDescription-format-03293
  If `format` is a depth/stencil format which includes only the stencil component, `finalLayout` **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`
- VUID-VkAttachmentDescription-format-06242
  If `format` is a depth/stencil format which includes both depth and stencil components, `initialLayout` **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`
- VUID-VkAttachmentDescription-format-06243
  If `format` is a depth/stencil format which includes both depth and stencil components, `finalLayout` **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`

Valid Usage (Implicit)

- VUID-VkAttachmentDescription-flags-parameter
  `flags` **must** be a valid combination of VkAttachmentDescriptionFlagBits values
- VUID-VkAttachmentDescription-format-parameter
  `format` **must** be a valid VkFormat value
- VUID-VkAttachmentDescription-samples-parameter
  `samples` **must** be a valid VkSampleCountFlagBits value
- VUID-VkAttachmentDescription-loadOp-parameter
  `loadOp` **must** be a valid VkAttachmentLoadOp value
- VUID-VkAttachmentDescription-storeOp-parameter
  `storeOp` **must** be a valid VkAttachmentStoreOp value
- VUID-VkAttachmentDescription-stencilLoadOp-parameter
  `stencilLoadOp` **must** be a valid VkAttachmentLoadOp value

- VUID-VkAttachmentDescription-stencilStoreOp-parameter
  `stencilStoreOp` **must** be a valid VkAttachmentStoreOp value
- VUID-VkAttachmentDescription-initialLayout-parameter
  `initialLayout` **must** be a valid VkImageLayout value
- VUID-VkAttachmentDescription-finalLayout-parameter
  `finalLayout` **must** be a valid VkImageLayout value

Bits which **can** be set in VkAttachmentDescription::`flags`, describing additional properties of the attachment, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkAttachmentDescriptionFlagBits {
    VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT = 0x00000001,
} VkAttachmentDescriptionFlagBits;
```

- `VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT` specifies that the attachment aliases the same device memory as other attachments.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkAttachmentDescriptionFlags;
```

`VkAttachmentDescriptionFlags` is a bitmask type for setting a mask of zero or more VkAttachmentDescriptionFlagBits.

The `VkRenderPassInputAttachmentAspectCreateInfo` structure is defined as:

```
typedef struct VkRenderPassInputAttachmentAspectCreateInfo {
    VkStructureType                              sType;
    const void*                                  pNext;
    uint32_t                                     aspectReferenceCount;
    const VkInputAttachmentAspectReference*      pAspectReferences;
} VkRenderPassInputAttachmentAspectCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_maintenance2
typedef VkRenderPassInputAttachmentAspectCreateInfo
VkRenderPassInputAttachmentAspectCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `aspectReferenceCount` is the number of elements in the `pAspectReferences` array.
- `pAspectReferences` is a pointer to an array of `aspectReferenceCount`

VkInputAttachmentAspectReference structures containing a mask describing which aspect(s) **can** be accessed for a given input attachment within a given subpass.

To specify which aspects of an input attachment **can** be read, add a VkRenderPassInputAttachmentAspectCreateInfo structure to the pNext chain of the VkRenderPassCreateInfo structure:

An application **can** access any aspect of an input attachment that does not have a specified aspect mask in the pAspectReferences array. Otherwise, an application **must** not access aspect(s) of an input attachment other than those in its specified aspect mask.

Valid Usage (Implicit)

- VUID-VkRenderPassInputAttachmentAspectCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RENDER_PASS_INPUT_ATTACHMENT_ASPECT_CREATE_INFO
- VUID-VkRenderPassInputAttachmentAspectCreateInfo-pAspectReferences-parameter
  pAspectReferences **must** be a valid pointer to an array of aspectReferenceCount valid VkInputAttachmentAspectReference structures
- VUID-VkRenderPassInputAttachmentAspectCreateInfo-aspectReferenceCount-arraylength
  aspectReferenceCount **must** be greater than 0

The VkInputAttachmentAspectReference structure is defined as:

```
typedef struct VkInputAttachmentAspectReference {
    uint32_t              subpass;
    uint32_t              inputAttachmentIndex;
    VkImageAspectFlags    aspectMask;
} VkInputAttachmentAspectReference;
```

or the equivalent

```
// Provided by VK_KHR_maintenance2
typedef VkInputAttachmentAspectReference VkInputAttachmentAspectReferenceKHR;
```

- subpass is an index into the pSubpasses array of the parent VkRenderPassCreateInfo structure.
- inputAttachmentIndex is an index into the pInputAttachments of the specified subpass.
- aspectMask is a mask of which aspect(s) **can** be accessed within the specified subpass.

This structure specifies an aspect mask for a specific input attachment of a specific subpass in the render pass.

subpass and inputAttachmentIndex index into the render pass as:

```
pCreateInfo->pSubpasses[subpass].pInputAttachments[inputAttachmentIndex]
```

Valid Usage

- VUID-VkInputAttachmentAspectReference-aspectMask-01964
  `aspectMask` **must** not include `VK_IMAGE_ASPECT_METADATA_BIT`
- VUID-VkInputAttachmentAspectReference-aspectMask-02250
  `aspectMask` **must** not include `VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT` for any index *i*

Valid Usage (Implicit)

- VUID-VkInputAttachmentAspectReference-aspectMask-parameter
  `aspectMask` **must** be a valid combination of VkImageAspectFlagBits values
- VUID-VkInputAttachmentAspectReference-aspectMask-requiredbitmask
  `aspectMask` **must** not be `0`

The `VkSubpassDescription` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSubpassDescription {
    VkSubpassDescriptionFlags       flags;
    VkPipelineBindPoint             pipelineBindPoint;
    uint32_t                        inputAttachmentCount;
    const VkAttachmentReference*    pInputAttachments;
    uint32_t                        colorAttachmentCount;
    const VkAttachmentReference*    pColorAttachments;
    const VkAttachmentReference*    pResolveAttachments;
    const VkAttachmentReference*    pDepthStencilAttachment;
    uint32_t                        preserveAttachmentCount;
    const uint32_t*                 pPreserveAttachments;
} VkSubpassDescription;
```

- `flags` is a bitmask of VkSubpassDescriptionFlagBits specifying usage of the subpass.
- `pipelineBindPoint` is a VkPipelineBindPoint value specifying the pipeline type supported for this subpass.
- `inputAttachmentCount` is the number of input attachments.
- `pInputAttachments` is a pointer to an array of VkAttachmentReference structures defining the input attachments for this subpass and their layouts.
- `colorAttachmentCount` is the number of color attachments.
- `pColorAttachments` is a pointer to an array of `colorAttachmentCount` VkAttachmentReference structures defining the color attachments for this subpass and their layouts.
- `pResolveAttachments` is `NULL` or a pointer to an array of `colorAttachmentCount` VkAttachmentReference structures defining the resolve attachments for this subpass and their layouts.

- pDepthStencilAttachment is a pointer to a VkAttachmentReference structure specifying the depth/stencil attachment for this subpass and its layout.
- preserveAttachmentCount is the number of preserved attachments.
- pPreserveAttachments is a pointer to an array of preserveAttachmentCount render pass attachment indices identifying attachments that are not used by this subpass, but whose contents **must** be preserved throughout the subpass.

Each element of the pInputAttachments array corresponds to an input attachment index in a fragment shader, i.e. if a shader declares an image variable decorated with a InputAttachmentIndex value of **X**, then it uses the attachment provided in pInputAttachments[**X**]. Input attachments **must** also be bound to the pipeline in a descriptor set. If the attachment member of any element of pInputAttachments is VK_ATTACHMENT_UNUSED, the application **must** not read from the corresponding input attachment index. Fragment shaders **can** use subpass input variables to access the contents of an input attachment at the fragment's (x_f,y_f) framebuffer coordinates and layer.

Each element of the pColorAttachments array corresponds to an output location in the shader, i.e. if the shader declares an output variable decorated with a Location value of **X**, then it uses the attachment provided in pColorAttachments[**X**]. If the attachment member of any element of pColorAttachments is VK_ATTACHMENT_UNUSED, or if Color Write Enable has been disabled for the corresponding attachment index, then writes to the corresponding location by a fragment shader are discarded.

If pResolveAttachments is not NULL, each of its elements corresponds to a color attachment (the element in pColorAttachments at the same index), and a multisample resolve operation is defined for each attachment unless the resolve attachment index is VK_ATTACHMENT_UNUSED.

Similarly, if VkSubpassDescriptionDepthStencilResolve::pDepthStencilResolveAttachment is not NULL and does not have the value VK_ATTACHMENT_UNUSED, it corresponds to the depth/stencil attachment in pDepthStencilAttachment, and multisample resolve operation for depth and stencil are defined by VkSubpassDescriptionDepthStencilResolve::depthResolveMode and VkSubpassDescriptionDepthStencilResolve::stencilResolveMode, respectively. If VkSubpassDescriptionDepthStencilResolve::depthResolveMode is VK_RESOLVE_MODE_NONE or the pDepthStencilResolveAttachment does not have a depth aspect, no resolve operation is performed for the depth attachment. If VkSubpassDescriptionDepthStencilResolve::stencilResolveMode is VK_RESOLVE_MODE_NONE or the pDepthStencilResolveAttachment does not have a stencil aspect, no resolve operation is performed for the stencil attachment.

If the image subresource range referenced by the depth/stencil attachment is created with VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT, then the multisample resolve operation uses the sample locations state specified in the sampleLocationsInfo member of the element of the VkRenderPassSampleLocationsBeginInfoEXT::pPostSubpassSampleLocations for the subpass.

If pDepthStencilAttachment is NULL, or if its attachment index is VK_ATTACHMENT_UNUSED, it indicates that no depth/stencil attachment will be used in the subpass.

The contents of an attachment within the render area become undefined at the start of a subpass **S** if all of the following conditions are true:

- The attachment is used as a color, depth/stencil, or resolve attachment in any subpass in the render pass.
- There is a subpass $\mathbf{S}_1$ that uses or preserves the attachment, and a subpass dependency from $\mathbf{S}_1$ to **S**.
- The attachment is not used or preserved in subpass **S**.

Once the contents of an attachment become undefined in subpass **S**, they remain undefined for subpasses in subpass dependency chains starting with subpass **S** until they are written again. However, they remain valid for subpasses in other subpass dependency chains starting with subpass $\mathbf{S}_1$ if those subpasses use or preserve the attachment.

Valid Usage

- VUID-VkSubpassDescription-attachment-06912
  If the `attachment` member of an element of `pInputAttachments` is not `VK_ATTACHMENT_UNUSED`, its `layout` member **must** not be `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL`
- VUID-VkSubpassDescription-attachment-06913
  If the `attachment` member of an element of `pColorAttachments` is not `VK_ATTACHMENT_UNUSED`, its `layout` member **must** not be `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`
- VUID-VkSubpassDescription-attachment-06914
  If the `attachment` member of an element of `pResolveAttachments` is not `VK_ATTACHMENT_UNUSED`, its `layout` member **must** not be `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`
- VUID-VkSubpassDescription-attachment-06915
  If the `attachment` member of `pDepthStencilAttachment` is not `VK_ATTACHMENT_UNUSED`, ts `layout` member **must** not be `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`
- VUID-VkSubpassDescription-attachment-06916
  If the `attachment` member of an element of `pColorAttachments` is not `VK_ATTACHMENT_UNUSED`, its `layout` member **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`
- VUID-VkSubpassDescription-attachment-06917
  If the `attachment` member of an element of `pResolveAttachments` is not `VK_ATTACHMENT_UNUSED`, its `layout` member **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`
- VUID-VkSubpassDescription-attachment-06918
  If the `attachment` member of an element of `pInputAttachments` is not `VK_ATTACHMENT_UNUSED`, its `layout` member **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL`

- VUID-VkSubpassDescription-attachment-06919
  If the attachment member of an element of pColorAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkSubpassDescription-attachment-06920
  If the attachment member of an element of pResolveAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkSubpassDescription-attachment-06921
  If the attachment member of an element of pInputAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR
- VUID-VkSubpassDescription-attachment-06922
  If the attachment member of an element of pColorAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkSubpassDescription-attachment-06923
  If the attachment member of an element of pResolveAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkSubpassDescription-pipelineBindPoint-04952
  pipelineBindPoint **must** be VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-VkSubpassDescription-colorAttachmentCount-00845
  colorAttachmentCount **must** be less than or equal to VkPhysicalDeviceLimits ::maxColorAttachments
- VUID-VkSubpassDescription-loadOp-00846
  If the first use of an attachment in this render pass is as an input attachment, and the attachment is not also used as a color or depth/stencil attachment in the same subpass, then loadOp **must** not be VK_ATTACHMENT_LOAD_OP_CLEAR
- VUID-VkSubpassDescription-pResolveAttachments-00847
  If pResolveAttachments is not NULL, for each resolve attachment that is not VK_ATTACHMENT_UNUSED, the corresponding color attachment **must** not be VK_ATTACHMENT_UNUSED
- VUID-VkSubpassDescription-pResolveAttachments-00848
  If pResolveAttachments is not NULL, for each resolve attachment that is not VK_ATTACHMENT_UNUSED, the corresponding color attachment **must** not have a sample count of VK_SAMPLE_COUNT_1_BIT
- VUID-VkSubpassDescription-pResolveAttachments-00849
  If pResolveAttachments is not NULL, each resolve attachment that is not VK_ATTACHMENT_UNUSED **must** have a sample count of VK_SAMPLE_COUNT_1_BIT
- VUID-VkSubpassDescription-pResolveAttachments-00850
  If pResolveAttachments is not NULL, each resolve attachment that is not

VK_ATTACHMENT_UNUSED **must** have the same VkFormat as its corresponding color attachment

- VUID-VkSubpassDescription-pColorAttachments-09430
  All attachments in pColorAttachments that are not VK_ATTACHMENT_UNUSED **must** have the same sample count
- VUID-VkSubpassDescription-pInputAttachments-02647
  All attachments in pInputAttachments that are not VK_ATTACHMENT_UNUSED **must** have image formats whose potential format features contain at least VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT or VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkSubpassDescription-pColorAttachments-02648
  All attachments in pColorAttachments that are not VK_ATTACHMENT_UNUSED **must** have image formats whose potential format features contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT
- VUID-VkSubpassDescription-pResolveAttachments-02649
  All attachments in pResolveAttachments that are not VK_ATTACHMENT_UNUSED **must** have image formats whose potential format features contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT
- VUID-VkSubpassDescription-pDepthStencilAttachment-02650
  If pDepthStencilAttachment is not NULL and the attachment is not VK_ATTACHMENT_UNUSED then it **must** have an image format whose potential format features contain VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkSubpassDescription-pDepthStencilAttachment-01418
  If pDepthStencilAttachment is not VK_ATTACHMENT_UNUSED and any attachments in pColorAttachments are not VK_ATTACHMENT_UNUSED, they **must** have the same sample count
- VUID-VkSubpassDescription-attachment-00853
  Each element of pPreserveAttachments **must** not be VK_ATTACHMENT_UNUSED
- VUID-VkSubpassDescription-pPreserveAttachments-00854
  Each element of pPreserveAttachments **must** not also be an element of any other member of the subpass description
- VUID-VkSubpassDescription-layout-02519
  If any attachment is used by more than one VkAttachmentReference member, then each use **must** use the same layout
- VUID-VkSubpassDescription-pDepthStencilAttachment-04438
  pDepthStencilAttachment and pColorAttachments **must** not contain references to the same attachment

Valid Usage (Implicit)

- VUID-VkSubpassDescription-flags-zerobitmask
  flags **must** be 0
- VUID-VkSubpassDescription-pipelineBindPoint-parameter
  pipelineBindPoint **must** be a valid VkPipelineBindPoint value

- VUID-VkSubpassDescription-pInputAttachments-parameter
  If `inputAttachmentCount` is not `0`, `pInputAttachments` **must** be a valid pointer to an array of `inputAttachmentCount` valid VkAttachmentReference structures
- VUID-VkSubpassDescription-pColorAttachments-parameter
  If `colorAttachmentCount` is not `0`, `pColorAttachments` **must** be a valid pointer to an array of `colorAttachmentCount` valid VkAttachmentReference structures
- VUID-VkSubpassDescription-pResolveAttachments-parameter
  If `colorAttachmentCount` is not `0`, and `pResolveAttachments` is not `NULL`, `pResolveAttachments` **must** be a valid pointer to an array of `colorAttachmentCount` valid VkAttachmentReference structures
- VUID-VkSubpassDescription-pDepthStencilAttachment-parameter
  If `pDepthStencilAttachment` is not `NULL`, `pDepthStencilAttachment` **must** be a valid pointer to a valid VkAttachmentReference structure
- VUID-VkSubpassDescription-pPreserveAttachments-parameter
  If `preserveAttachmentCount` is not `0`, `pPreserveAttachments` **must** be a valid pointer to an array of `preserveAttachmentCount` `uint32_t` values

Bits which **can** be set in VkSubpassDescription::`flags`, specifying usage of the subpass, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSubpassDescriptionFlagBits {
} VkSubpassDescriptionFlagBits;
```

NOTE All bits for this type are defined by extensions, and none of those extensions are enabled in this build of the specification.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkSubpassDescriptionFlags;
```

`VkSubpassDescriptionFlags` is a bitmask type for setting a mask of zero or more VkSubpassDescriptionFlagBits.

The `VkAttachmentReference` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkAttachmentReference {
    uint32_t         attachment;
    VkImageLayout    layout;
} VkAttachmentReference;
```

- `attachment` is either an integer value identifying an attachment at the corresponding index in VkRenderPassCreateInfo::`pAttachments`, or `VK_ATTACHMENT_UNUSED` to signify that this attachment is not used.

- `layout` is a VkImageLayout value specifying the layout the attachment uses during the subpass.

Valid Usage

- VUID-VkAttachmentReference-layout-03077
  If `attachment` is not `VK_ATTACHMENT_UNUSED`, `layout` **must** not be `VK_IMAGE_LAYOUT_UNDEFINED`, `VK_IMAGE_LAYOUT_PREINITIALIZED`, or `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR`
- VUID-VkAttachmentReference-separateDepthStencilLayouts-03313
  If the `separateDepthStencilLayouts` feature is not enabled, and `attachment` is not `VK_ATTACHMENT_UNUSED`, `layout` **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`,
- VUID-VkAttachmentReference-synchronization2-06910
  If the `synchronization2` feature is not enabled, `layout` **must** not be `VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR` or `VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR`
- VUID-VkAttachmentReference-attachmentFeedbackLoopLayout-07311
  If the `attachmentFeedbackLoopLayout` feature is not enabled, `layout` **must** not be `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT`
- VUID-VkAttachmentReference-dynamicRenderingLocalRead-09546
  If the `dynamicRenderingLocalRead` feature is not enabled, `layout` **must** not be `VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR`

Valid Usage (Implicit)

- VUID-VkAttachmentReference-layout-parameter
  `layout` **must** be a valid VkImageLayout value

`VK_SUBPASS_EXTERNAL` is a special subpass index value expanding synchronization scope outside a subpass. It is described in more detail by VkSubpassDependency.

```
#define VK_SUBPASS_EXTERNAL             (~0U)
```

The `VkSubpassDependency` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSubpassDependency {
    uint32_t                srcSubpass;
    uint32_t                dstSubpass;
    VkPipelineStageFlags    srcStageMask;
    VkPipelineStageFlags    dstStageMask;
    VkAccessFlags           srcAccessMask;
    VkAccessFlags           dstAccessMask;
    VkDependencyFlags       dependencyFlags;
} VkSubpassDependency;
```

- `srcSubpass` is the subpass index of the first subpass in the dependency, or `VK_SUBPASS_EXTERNAL`.
- `dstSubpass` is the subpass index of the second subpass in the dependency, or `VK_SUBPASS_EXTERNAL`.
- `srcStageMask` is a bitmask of VkPipelineStageFlagBits specifying the source stage mask.
- `dstStageMask` is a bitmask of VkPipelineStageFlagBits specifying the destination stage mask
- `srcAccessMask` is a bitmask of VkAccessFlagBits specifying a source access mask.
- `dstAccessMask` is a bitmask of VkAccessFlagBits specifying a destination access mask.
- `dependencyFlags` is a bitmask of VkDependencyFlagBits.

If `srcSubpass` is equal to `dstSubpass` then the VkSubpassDependency does not directly define a dependency. Instead, it enables pipeline barriers to be used in a render pass instance within the identified subpass, where the scopes of one pipeline barrier **must** be a subset of those described by one subpass dependency. Subpass dependencies specified in this way that include framebuffer-space stages in the `srcStageMask` **must** only include framebuffer-space stages in `dstStageMask`, and **must** include `VK_DEPENDENCY_BY_REGION_BIT`. When a subpass dependency is specified in this way for a subpass that has more than one view in its view mask, its `dependencyFlags` **must** include `VK_DEPENDENCY_VIEW_LOCAL_BIT`.

If `srcSubpass` and `dstSubpass` are not equal, when a render pass instance which includes a subpass dependency is submitted to a queue, it defines a dependency between the subpasses identified by `srcSubpass` and `dstSubpass`.

If `srcSubpass` is equal to `VK_SUBPASS_EXTERNAL`, the first synchronization scope includes commands that occur earlier in submission order than the vkCmdBeginRenderPass used to begin the render pass instance. Otherwise, the first set of commands includes all commands submitted as part of the subpass instance identified by `srcSubpass` and any load, store, or multisample resolve operations on attachments used in `srcSubpass`. In either case, the first synchronization scope is limited to operations on the pipeline stages determined by the source stage mask specified by `srcStageMask`.

If `dstSubpass` is equal to `VK_SUBPASS_EXTERNAL`, the second synchronization scope includes commands that occur later in submission order than the vkCmdEndRenderPass used to end the render pass instance. Otherwise, the second set of commands includes all commands submitted as part of the subpass instance identified by `dstSubpass` and any load, store, and multisample resolve operations on attachments used in `dstSubpass`. In either case, the second synchronization scope is limited to operations on the pipeline stages determined by the destination stage mask specified by

dstStageMask.

The first access scope is limited to accesses in the pipeline stages determined by the source stage mask specified by srcStageMask. It is also limited to access types in the source access mask specified by srcAccessMask.

The second access scope is limited to accesses in the pipeline stages determined by the destination stage mask specified by dstStageMask. It is also limited to access types in the destination access mask specified by dstAccessMask.

The availability and visibility operations defined by a subpass dependency affect the execution of image layout transitions within the render pass.

NOTE

For non-attachment resources, the memory dependency expressed by subpass dependency is nearly identical to that of a VkMemoryBarrier (with matching srcAccessMask and dstAccessMask parameters) submitted as a part of a vkCmdPipelineBarrier (with matching srcStageMask and dstStageMask parameters). The only difference being that its scopes are limited to the identified subpasses rather than potentially affecting everything before and after.

For attachments however, subpass dependencies work more like a VkImageMemoryBarrier defined similarly to the VkMemoryBarrier above, the queue family indices set to VK_QUEUE_FAMILY_IGNORED, and layouts as follows:

- The equivalent to oldLayout is the attachment's layout according to the subpass description for srcSubpass.
- The equivalent to newLayout is the attachment's layout according to the subpass description for dstSubpass.

Valid Usage

- VUID-VkSubpassDependency-srcStageMask-04090
  If the geometryShader feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT
- VUID-VkSubpassDependency-srcStageMask-04091
  If the tessellationShader feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-VkSubpassDependency-srcStageMask-04093
  If the fragmentDensityMap feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-VkSubpassDependency-srcStageMask-04094
  If the transformFeedback feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-VkSubpassDependency-srcStageMask-07319
  If the attachmentFragmentShadingRate feature is not enabled, srcStageMask **must** not contain

VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR

- VUID-VkSubpassDependency-srcStageMask-03937
  If the synchronization2 feature is not enabled, srcStageMask **must** not be 0
- VUID-VkSubpassDependency-srcStageMask-07950
  If the rayTracingPipeline feature is not enabled, srcStageMask **must** not contain VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkSubpassDependency-dstStageMask-04090
  If the geometryShader feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT
- VUID-VkSubpassDependency-dstStageMask-04091
  If the tessellationShader feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT or VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT
- VUID-VkSubpassDependency-dstStageMask-04093
  If the fragmentDensityMap feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT
- VUID-VkSubpassDependency-dstStageMask-04094
  If the transformFeedback feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-VkSubpassDependency-dstStageMask-07319
  If the attachmentFragmentShadingRate feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkSubpassDependency-dstStageMask-03937
  If the synchronization2 feature is not enabled, dstStageMask **must** not be 0
- VUID-VkSubpassDependency-dstStageMask-07950
  If the rayTracingPipeline feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkSubpassDependency-srcSubpass-00864
  srcSubpass **must** be less than or equal to dstSubpass, unless one of them is VK_SUBPASS_EXTERNAL, to avoid cyclic dependencies and ensure a valid execution order
- VUID-VkSubpassDependency-srcSubpass-00865
  srcSubpass and dstSubpass **must** not both be equal to VK_SUBPASS_EXTERNAL
- VUID-VkSubpassDependency-srcSubpass-06809
  If srcSubpass is equal to dstSubpass and srcStageMask includes a framebuffer-space stage, dstStageMask **must** only contain framebuffer-space stages
- VUID-VkSubpassDependency-srcAccessMask-00868
  Any access flag included in srcAccessMask **must** be supported by one of the pipeline stages in srcStageMask, as specified in the table of supported access types
- VUID-VkSubpassDependency-dstAccessMask-00869
  Any access flag included in dstAccessMask **must** be supported by one of the pipeline stages in dstStageMask, as specified in the table of supported access types

- VUID-VkSubpassDependency-srcSubpass-02243
  If `srcSubpass` equals `dstSubpass`, and `srcStageMask` and `dstStageMask` both include a framebuffer-space stage, then `dependencyFlags` **must** include `VK_DEPENDENCY_BY_REGION_BIT`
- VUID-VkSubpassDependency-dependencyFlags-02520
  If `dependencyFlags` includes `VK_DEPENDENCY_VIEW_LOCAL_BIT`, `srcSubpass` **must** not be equal to `VK_SUBPASS_EXTERNAL`
- VUID-VkSubpassDependency-dependencyFlags-02521
  If `dependencyFlags` includes `VK_DEPENDENCY_VIEW_LOCAL_BIT`, `dstSubpass` **must** not be equal to `VK_SUBPASS_EXTERNAL`
- VUID-VkSubpassDependency-srcSubpass-00872
  If `srcSubpass` equals `dstSubpass` and that subpass has more than one bit set in the view mask, then `dependencyFlags` **must** include `VK_DEPENDENCY_VIEW_LOCAL_BIT`

Valid Usage (Implicit)

- VUID-VkSubpassDependency-srcStageMask-parameter
  `srcStageMask` **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-VkSubpassDependency-dstStageMask-parameter
  `dstStageMask` **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-VkSubpassDependency-srcAccessMask-parameter
  `srcAccessMask` **must** be a valid combination of VkAccessFlagBits values
- VUID-VkSubpassDependency-dstAccessMask-parameter
  `dstAccessMask` **must** be a valid combination of VkAccessFlagBits values
- VUID-VkSubpassDependency-dependencyFlags-parameter
  `dependencyFlags` **must** be a valid combination of VkDependencyFlagBits values

When multiview is enabled, the execution of the multiple views of one subpass **may** not occur simultaneously or even back-to-back, and rather **may** be interleaved with the execution of other subpasses. The load and store operations apply to attachments on a per-view basis. For example, an attachment using `VK_ATTACHMENT_LOAD_OP_CLEAR` will have each view cleared on first use, but the first use of one view may be temporally distant from the first use of another view.

NOTE

A good mental model for multiview is to think of a multiview subpass as if it were a collection of individual (per-view) subpasses that are logically grouped together and described as a single multiview subpass in the API. Similarly, a multiview attachment can be thought of like several individual attachments that happen to be layers in a single image. A view-local dependency between two multiview subpasses acts like a set of one-to-one dependencies between corresponding pairs of per-view subpasses. A view-global dependency between two multiview subpasses acts like a set of N × M dependencies between all pairs of per-view subpasses in the source and destination. Thus, it is a more compact representation which also makes clear the commonality and reuse that is present between views in a subpass. This interpretation motivates the answers to questions like “when does the load op

> apply" - it is on the first use of each view of an attachment, as if each view was a separate attachment.
>
> The content of each view follows the description in attachment content behavior. In particular, if an attachment is preserved, all views within the attachment are preserved.

If any two subpasses of a render pass activate transform feedback to the same bound transform feedback buffers, a subpass dependency **must** be included (either directly or via some intermediate subpasses) between them.

If there is no subpass dependency from `VK_SUBPASS_EXTERNAL` to the first subpass that uses an attachment, then an implicit subpass dependency exists from `VK_SUBPASS_EXTERNAL` to the first subpass it is used in. The implicit subpass dependency only exists if there exists an automatic layout transition away from `initialLayout`. The subpass dependency operates as if defined with the following parameters:

```
VkSubpassDependency implicitDependency = {
    .srcSubpass = VK_SUBPASS_EXTERNAL,
    .dstSubpass = firstSubpass, // First subpass attachment is used in
    .srcStageMask = VK_PIPELINE_STAGE_NONE,
    .dstStageMask = VK_PIPELINE_STAGE_ALL_COMMANDS_BIT,
    .srcAccessMask = 0,
    .dstAccessMask = VK_ACCESS_INPUT_ATTACHMENT_READ_BIT |
                     VK_ACCESS_COLOR_ATTACHMENT_READ_BIT |
                     VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT |
                     VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_READ_BIT |
                     VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT,
    .dependencyFlags = 0
};
```

Similarly, if there is no subpass dependency from the last subpass that uses an attachment to `VK_SUBPASS_EXTERNAL`, then an implicit subpass dependency exists from the last subpass it is used in to `VK_SUBPASS_EXTERNAL`. The implicit subpass dependency only exists if there exists an automatic layout transition into `finalLayout`. The subpass dependency operates as if defined with the following parameters:

```
VkSubpassDependency implicitDependency = {
    .srcSubpass = lastSubpass, // Last subpass attachment is used in
    .dstSubpass = VK_SUBPASS_EXTERNAL,
    .srcStageMask = VK_PIPELINE_STAGE_ALL_COMMANDS_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_NONE,
    .srcAccessMask = VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT |
                     VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT,
    .dstAccessMask = 0,
    .dependencyFlags = 0
};
```

As subpasses **may** overlap or execute out of order with regards to other subpasses unless a subpass dependency chain describes otherwise, the layout transitions required between subpasses **cannot** be known to an application. Instead, an application provides the layout that each attachment **must** be in at the start and end of a render pass, and the layout it **must** be in during each subpass it is used in. The implementation then **must** execute layout transitions between subpasses in order to guarantee that the images are in the layouts required by each subpass, and in the final layout at the end of the render pass.

Automatic layout transitions apply to the entire image subresource attached to the framebuffer. If multiview is not enabled and the attachment is a view of a 1D or 2D image, the automatic layout transitions apply to the number of layers specified by VkFramebufferCreateInfo::`layers`. If multiview is enabled and the attachment is a view of a 1D or 2D image, the automatic layout transitions apply to the layers corresponding to views which are used by some subpass in the render pass, even if that subpass does not reference the given attachment. If the attachment view is a 2D or 2D array view of a 3D image, even if the attachment view only refers to a subset of the slices of the selected mip level of the 3D image, automatic layout transitions apply to the entire subresource referenced which is the entire mip level in this case.

Automatic layout transitions away from the layout used in a subpass happen-after the availability operations for all dependencies with that subpass as the `srcSubpass`.

Automatic layout transitions into the layout used in a subpass happen-before the visibility operations for all dependencies with that subpass as the `dstSubpass`.

Automatic layout transitions away from `initialLayout` happen-after the availability operations for all dependencies with a `srcSubpass` equal to `VK_SUBPASS_EXTERNAL`, where `dstSubpass` uses the attachment that will be transitioned. For attachments created with `VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT`, automatic layout transitions away from `initialLayout` happen-after the availability operations for all dependencies with a `srcSubpass` equal to `VK_SUBPASS_EXTERNAL`, where `dstSubpass` uses any aliased attachment.

Automatic layout transitions into `finalLayout` happen-before the visibility operations for all dependencies with a `dstSubpass` equal to `VK_SUBPASS_EXTERNAL`, where `srcSubpass` uses the attachment that will be transitioned. For attachments created with `VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT`, automatic layout transitions into `finalLayout` happen-before the visibility operations for all dependencies with a `dstSubpass` equal to `VK_SUBPASS_EXTERNAL`, where `srcSubpass` uses any aliased attachment.

The image layout of the depth aspect of a depth/stencil attachment referring to an image created with `VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT` is dependent on the last sample locations used to render to the attachment, thus automatic layout transitions use the sample locations state specified in VkRenderPassSampleLocationsBeginInfoEXT.

Automatic layout transitions of an attachment referring to a depth/stencil image created with `VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT` use the sample locations the image subresource range referenced by the attachment was last rendered with. If the current render pass does not use the attachment as a depth/stencil attachment in any subpass that happens-before, the automatic layout transition uses the sample locations state specified in the `sampleLocationsInfo` member of the element of the `VkRenderPassSampleLocationsBeginInfoEXT`

::pAttachmentInitialSampleLocations array for which the attachmentIndex member equals the attachment index of the attachment, if one is specified. Otherwise, the automatic layout transition uses the sample locations state specified in the sampleLocationsInfo member of the element of the VkRenderPassSampleLocationsBeginInfoEXT::pPostSubpassSampleLocations array for which the subpassIndex member equals the index of the subpass that last used the attachment as a depth/stencil attachment, if one is specified.

If no sample locations state has been specified for an automatic layout transition performed on an attachment referring to a depth/stencil image created with VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT the contents of the depth aspect of the depth/stencil attachment become undefined as if the layout of the attachment was transitioned from the VK_IMAGE_LAYOUT_UNDEFINED layout.

If two subpasses use the same attachment, and both subpasses use the attachment in a read-only layout, no subpass dependency needs to be specified between those subpasses. If an implementation treats those layouts separately, it **must** insert an implicit subpass dependency between those subpasses to separate the uses in each layout. The subpass dependency operates as if defined with the following parameters:

```
// Used for input attachments
VkPipelineStageFlags inputAttachmentStages = VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT;
VkAccessFlags inputAttachmentDstAccess = VK_ACCESS_INPUT_ATTACHMENT_READ_BIT;

// Used for depth/stencil attachments
VkPipelineStageFlags depthStencilAttachmentStages =
VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT |
VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT;
VkAccessFlags depthStencilAttachmentDstAccess =
VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_READ_BIT;

VkSubpassDependency implicitDependency = {
    .srcSubpass = firstSubpass;
    .dstSubpass = secondSubpass;
    .srcStageMask = inputAttachmentStages | depthStencilAttachmentStages;
    .dstStageMask = inputAttachmentStages | depthStencilAttachmentStages;
    .srcAccessMask = 0;
    .dstAccessMask = inputAttachmentDstAccess | depthStencilAttachmentDstAccess;
    .dependencyFlags = 0;
};
```

When drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, the application **must** specify which types of attachments that are written to during a render pass will also be accessed as non-attachments in the render pass.

To dynamically set whether a pipeline **can** access a resource as a non-attachment while it is also used as an attachment that is written to, call:

```
// Provided by VK_EXT_attachment_feedback_loop_dynamic_state
void vkCmdSetAttachmentFeedbackLoopEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkImageAspectFlags                          aspectMask);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `aspectMask` specifies the types of attachments for which feedback loops will be enabled. Attachment types whose aspects are not included in `aspectMask` will have feedback loops disabled.

For attachments that are written to in a render pass, only attachments with the aspects specified in `aspectMask` **can** be accessed as non-attachments by subsequent drawing commands.

Valid Usage

- VUID-vkCmdSetAttachmentFeedbackLoopEnableEXT-attachmentFeedbackLoopDynamicState-08862
  The `attachmentFeedbackLoopDynamicState` feature **must** be enabled
- VUID-vkCmdSetAttachmentFeedbackLoopEnableEXT-aspectMask-08863
  `aspectMask` **must** only include `VK_IMAGE_ASPECT_NONE`, `VK_IMAGE_ASPECT_COLOR_BIT`, `VK_IMAGE_ASPECT_DEPTH_BIT`, and `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-vkCmdSetAttachmentFeedbackLoopEnableEXT-attachmentFeedbackLoopLayout-08864
  If the `attachmentFeedbackLoopLayout` feature is not enabled, `aspectMask` **must** be `VK_IMAGE_ASPECT_NONE`

Valid Usage (Implicit)

- VUID-vkCmdSetAttachmentFeedbackLoopEnableEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetAttachmentFeedbackLoopEnableEXT-aspectMask-parameter
  `aspectMask` **must** be a valid combination of VkImageAspectFlagBits values
- VUID-vkCmdSetAttachmentFeedbackLoopEnableEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetAttachmentFeedbackLoopEnableEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetAttachmentFeedbackLoopEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

A more extensible version of render pass creation is also defined below.

To create a render pass, call:

```
// Provided by VK_KHR_create_renderpass2
VkResult vkCreateRenderPass2KHR(
    VkDevice                                    device,
    const VkRenderPassCreateInfo2*              pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkRenderPass*                               pRenderPass);
```

- `device` is the logical device that creates the render pass.
- `pCreateInfo` is a pointer to a VkRenderPassCreateInfo2 structure describing the parameters of the render pass.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pRenderPass` is a pointer to a VkRenderPass handle in which the resulting render pass object is returned.

This command is functionally identical to vkCreateRenderPass, but includes extensible sub-structures that include `sType` and `pNext` parameters, allowing them to be more easily extended.

Valid Usage

- VUID-vkCreateRenderPass2-device-10001
  `device` **must** support at least one queue family with the `VK_QUEUE_GRAPHICS_BIT` capability

Valid Usage (Implicit)

- VUID-vkCreateRenderPass2-device-parameter

`device` **must** be a valid VkDevice handle

- VUID-vkCreateRenderPass2-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkRenderPassCreateInfo2 structure
- VUID-vkCreateRenderPass2-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateRenderPass2-pRenderPass-parameter
  `pRenderPass` **must** be a valid pointer to a VkRenderPass handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkRenderPassCreateInfo2` structure is defined as:

```
typedef struct VkRenderPassCreateInfo2 {
    VkStructureType                   sType;
    const void*                       pNext;
    VkRenderPassCreateFlags           flags;
    uint32_t                          attachmentCount;
    const VkAttachmentDescription2*   pAttachments;
    uint32_t                          subpassCount;
    const VkSubpassDescription2*      pSubpasses;
    uint32_t                          dependencyCount;
    const VkSubpassDependency2*       pDependencies;
    uint32_t                          correlatedViewMaskCount;
    const uint32_t*                   pCorrelatedViewMasks;
} VkRenderPassCreateInfo2;
```

or the equivalent

```
// Provided by VK_KHR_create_renderpass2
typedef VkRenderPassCreateInfo2 VkRenderPassCreateInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `attachmentCount` is the number of attachments used by this render pass.

- pAttachments is a pointer to an array of attachmentCount VkAttachmentDescription2 structures describing the attachments used by the render pass.
- subpassCount is the number of subpasses to create.
- pSubpasses is a pointer to an array of subpassCount VkSubpassDescription2 structures describing each subpass.
- dependencyCount is the number of dependencies between pairs of subpasses.
- pDependencies is a pointer to an array of dependencyCount VkSubpassDependency2 structures describing dependencies between pairs of subpasses.
- correlatedViewMaskCount is the number of correlation masks.
- pCorrelatedViewMasks is a pointer to an array of view masks indicating sets of views that **may** be more efficient to render concurrently.

Parameters defined by this structure with the same name as those in VkRenderPassCreateInfo have the identical effect to those parameters; the child structures are variants of those used in VkRenderPassCreateInfo which add sType and pNext parameters, allowing them to be extended.

If the VkSubpassDescription2::viewMask member of any element of pSubpasses is not zero, *multiview* functionality is considered to be enabled for this render pass.

correlatedViewMaskCount and pCorrelatedViewMasks have the same effect as VkRenderPassMultiviewCreateInfo::correlationMaskCount and VkRenderPassMultiviewCreateInfo ::pCorrelationMasks, respectively.

Valid Usage

- VUID-VkRenderPassCreateInfo2-None-03049
  If any two subpasses operate on attachments with overlapping ranges of the same VkDeviceMemory object, and at least one subpass writes to that area of VkDeviceMemory, a subpass dependency **must** be included (either directly or via some intermediate subpasses) between them
- VUID-VkRenderPassCreateInfo2-attachment-03050
  If the attachment member of any element of pInputAttachments, pColorAttachments, pResolveAttachments or pDepthStencilAttachment, or the attachment indexed by any element of pPreserveAttachments in any element of pSubpasses is bound to a range of a VkDeviceMemory object that overlaps with any other attachment in any subpass (including the same subpass), the VkAttachmentDescription2 structures describing them **must** include VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT in flags
- VUID-VkRenderPassCreateInfo2-attachment-03051
  If the attachment member of any element of pInputAttachments, pColorAttachments, pResolveAttachments or pDepthStencilAttachment, or any element of pPreserveAttachments in any element of pSubpasses is not VK_ATTACHMENT_UNUSED, then it **must** be less than attachmentCount
- VUID-VkRenderPassCreateInfo2-fragmentDensityMapAttachment-06472
  If the pNext chain includes a VkRenderPassFragmentDensityMapCreateInfoEXT structure and the fragmentDensityMapAttachment member is not VK_ATTACHMENT_UNUSED, then

attachment **must** be less than attachmentCount

- VUID-VkRenderPassCreateInfo2-pSubpasses-06473
  If the pSubpasses pNext chain includes a VkSubpassDescriptionDepthStencilResolve structure and the pDepthStencilResolveAttachment member is not NULL and does not have the value VK_ATTACHMENT_UNUSED, then attachment **must** be less than attachmentCount
- VUID-VkRenderPassCreateInfo2-pAttachments-02522
  For any member of pAttachments with a loadOp equal to VK_ATTACHMENT_LOAD_OP_CLEAR, the first use of that attachment **must** not specify a layout equal to VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkRenderPassCreateInfo2-pAttachments-02523
  For any member of pAttachments with a stencilLoadOp equal to VK_ATTACHMENT_LOAD_OP_CLEAR, the first use of that attachment **must** not specify a layout equal to VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, or VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkRenderPassCreateInfo2-pDependencies-03054
  For any element of pDependencies, if the srcSubpass is not VK_SUBPASS_EXTERNAL, all stage flags included in the srcStageMask member of that dependency **must** be a pipeline stage supported by the pipeline identified by the pipelineBindPoint member of the source subpass
- VUID-VkRenderPassCreateInfo2-pDependencies-03055
  For any element of pDependencies, if the dstSubpass is not VK_SUBPASS_EXTERNAL, all stage flags included in the dstStageMask member of that dependency **must** be a pipeline stage supported by the pipeline identified by the pipelineBindPoint member of the destination subpass
- VUID-VkRenderPassCreateInfo2-pCorrelatedViewMasks-03056
  The set of bits included in any element of pCorrelatedViewMasks **must** not overlap with the set of bits included in any other element of pCorrelatedViewMasks
- VUID-VkRenderPassCreateInfo2-viewMask-03057
  If the VkSubpassDescription2::viewMask member of all elements of pSubpasses is 0, correlatedViewMaskCount **must** be 0
- VUID-VkRenderPassCreateInfo2-viewMask-03058
  The VkSubpassDescription2::viewMask member of all elements of pSubpasses **must** either all be 0, or all not be 0
- VUID-VkRenderPassCreateInfo2-viewMask-03059
  If the VkSubpassDescription2::viewMask member of all elements of pSubpasses is 0, the dependencyFlags member of any element of pDependencies **must** not include VK_DEPENDENCY_VIEW_LOCAL_BIT
- VUID-VkRenderPassCreateInfo2-pDependencies-03060
  For any element of pDependencies where its srcSubpass member equals its dstSubpass member, if the viewMask member of the corresponding element of pSubpasses includes more than one bit, its dependencyFlags member **must** include

VK_DEPENDENCY_VIEW_LOCAL_BIT

- VUID-VkRenderPassCreateInfo2-attachment-02525
  If the attachment member of any element of the pInputAttachments member of any element of pSubpasses is not VK_ATTACHMENT_UNUSED, the aspectMask member of that element of pInputAttachments **must** only include aspects that are present in images of the format specified by the element of pAttachments specified by attachment
- VUID-VkRenderPassCreateInfo2-srcSubpass-02526
  The srcSubpass member of each element of pDependencies **must** be less than subpassCount
- VUID-VkRenderPassCreateInfo2-dstSubpass-02527
  The dstSubpass member of each element of pDependencies **must** be less than subpassCount
- VUID-VkRenderPassCreateInfo2-pAttachments-04585
  If any element of pAttachments is used as a fragment shading rate attachment in any subpass, it **must** not be used as any other attachment in the render pass
- VUID-VkRenderPassCreateInfo2-pAttachments-09387
  If any element of pAttachments is used as a fragment shading rate attachment, the loadOp for that attachment **must** not be VK_ATTACHMENT_LOAD_OP_CLEAR
- VUID-VkRenderPassCreateInfo2-pAttachments-04586
  If any element of pAttachments is used as a fragment shading rate attachment in any subpass, it **must** have an image format whose potential format features contain VK_FORMAT_FEATURE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkRenderPassCreateInfo2-attachment-06244
  If the attachment member of the pDepthStencilAttachment member of an element of pSubpasses is not VK_ATTACHMENT_UNUSED, the layout member of that same structure is either VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, and the pNext chain of that structure does not include a VkAttachmentReferenceStencilLayout structure, then the element of pAttachments with an index equal to attachment **must** not have a format that includes both depth and stencil components
- VUID-VkRenderPassCreateInfo2-attachment-06245
  If the attachment member of the pDepthStencilAttachment member of an element of pSubpasses is not VK_ATTACHMENT_UNUSED and the layout member of that same structure is either VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL, then the element of pAttachments with an index equal to attachment **must** have a format that includes only a stencil component
- VUID-VkRenderPassCreateInfo2-attachment-06246
  If the attachment member of the pDepthStencilAttachment member of an element of pSubpasses is not VK_ATTACHMENT_UNUSED and the layout member of that same structure is either VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, then the element of pAttachments with an index equal to attachment **must** not have a format that includes only a stencil component

Valid Usage (Implicit)

- VUID-VkRenderPassCreateInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO_2`
- VUID-VkRenderPassCreateInfo2-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkRenderPassFragmentDensityMapCreateInfoEXT
- VUID-VkRenderPassCreateInfo2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkRenderPassCreateInfo2-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkRenderPassCreateInfo2-pAttachments-parameter
  If `attachmentCount` is not `0`, `pAttachments` **must** be a valid pointer to an array of `attachmentCount` valid VkAttachmentDescription2 structures
- VUID-VkRenderPassCreateInfo2-pSubpasses-parameter
  `pSubpasses` **must** be a valid pointer to an array of `subpassCount` valid VkSubpassDescription2 structures
- VUID-VkRenderPassCreateInfo2-pDependencies-parameter
  If `dependencyCount` is not `0`, `pDependencies` **must** be a valid pointer to an array of `dependencyCount` valid VkSubpassDependency2 structures
- VUID-VkRenderPassCreateInfo2-pCorrelatedViewMasks-parameter
  If `correlatedViewMaskCount` is not `0`, `pCorrelatedViewMasks` **must** be a valid pointer to an array of `correlatedViewMaskCount` `uint32_t` values
- VUID-VkRenderPassCreateInfo2-subpassCount-arraylength
  `subpassCount` **must** be greater than `0`

The `VkAttachmentDescription2` structure is defined as:

```
typedef struct VkAttachmentDescription2 {
    VkStructureType                 sType;
    const void*                     pNext;
    VkAttachmentDescriptionFlags    flags;
    VkFormat                        format;
    VkSampleCountFlagBits           samples;
    VkAttachmentLoadOp              loadOp;
    VkAttachmentStoreOp             storeOp;
    VkAttachmentLoadOp              stencilLoadOp;
    VkAttachmentStoreOp             stencilStoreOp;
    VkImageLayout                   initialLayout;
    VkImageLayout                   finalLayout;
} VkAttachmentDescription2;
```

or the equivalent

```
// Provided by VK_KHR_create_renderpass2
typedef VkAttachmentDescription2 VkAttachmentDescription2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkAttachmentDescriptionFlagBits specifying additional properties of the attachment.
- `format` is a VkFormat value specifying the format of the image that will be used for the attachment.
- `samples` is a VkSampleCountFlagBits value specifying the number of samples of the image.
- `loadOp` is a VkAttachmentLoadOp value specifying how the contents of color and depth components of the attachment are treated at the beginning of the subpass where it is first used.
- `storeOp` is a VkAttachmentStoreOp value specifying how the contents of color and depth components of the attachment are treated at the end of the subpass where it is last used.
- `stencilLoadOp` is a VkAttachmentLoadOp value specifying how the contents of stencil components of the attachment are treated at the beginning of the subpass where it is first used.
- `stencilStoreOp` is a VkAttachmentStoreOp value specifying how the contents of stencil components of the attachment are treated at the end of the last subpass where it is used.
- `initialLayout` is the layout the attachment image subresource will be in when a render pass instance begins.
- `finalLayout` is the layout the attachment image subresource will be transitioned to when a render pass instance ends.

Parameters defined by this structure with the same name as those in VkAttachmentDescription have the identical effect to those parameters.

If the `separateDepthStencilLayouts` feature is enabled, and `format` is a depth/stencil format, `initialLayout` and `finalLayout` **can** be set to a layout that only specifies the layout of the depth aspect.

If the `pNext` chain includes a VkAttachmentDescriptionStencilLayout structure, then the `stencilInitialLayout` and `stencilFinalLayout` members specify the initial and final layouts of the stencil aspect of a depth/stencil format, and `initialLayout` and `finalLayout` only apply to the depth aspect. For depth-only formats, the VkAttachmentDescriptionStencilLayout structure is ignored. For stencil-only formats, the initial and final layouts of the stencil aspect are taken from the VkAttachmentDescriptionStencilLayout structure if present, or `initialLayout` and `finalLayout` if not present.

If `format` is a depth/stencil format, and either `initialLayout` or `finalLayout` does not specify a layout for the stencil aspect, then the application **must** specify the initial and final layouts of the stencil aspect by including a VkAttachmentDescriptionStencilLayout structure in the `pNext` chain.

`loadOp` and `storeOp` are ignored for fragment shading rate attachments. No access to the shading rate attachment is performed in `loadOp` and `storeOp`. Instead, access to

VK_ACCESS_FRAGMENT_SHADING_RATE_ATTACHMENT_READ_BIT_KHR is performed as fragments are rasterized.

Valid Usage

- VUID-VkAttachmentDescription2-format-06699
  If format includes a color or depth component and loadOp is VK_ATTACHMENT_LOAD_OP_LOAD, then initialLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED
- VUID-VkAttachmentDescription2-finalLayout-00843
  finalLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED
- VUID-VkAttachmentDescription2-format-03280
  If format is a color format, initialLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-format-03281
  If format is a depth/stencil format, initialLayout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescription2-format-03282
  If format is a color format, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-format-03283
  If format is a depth/stencil format, finalLayout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescription2-format-06487
  If format is a color format, initialLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescription2-format-06488
  If format is a color format, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescription2-separateDepthStencilLayouts-03284
  If the separateDepthStencilLayouts feature is not enabled, initialLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL,
- VUID-VkAttachmentDescription2-separateDepthStencilLayouts-03285
  If the separateDepthStencilLayouts feature is not enabled, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL,
- VUID-VkAttachmentDescription2-format-03286
  If format is a color format, initialLayout **must** not be

VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL

- VUID-VkAttachmentDescription2-format-03287
  If format is a color format, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-format-06906
  If format is a depth/stencil format which includes both depth and stencil components, initialLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-format-06907
  If format is a depth/stencil format which includes both depth and stencil components, finalLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-format-03290
  If format is a depth/stencil format which includes only the depth component, initialLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-format-03291
  If format is a depth/stencil format which includes only the depth component, finalLayout **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-synchronization2-06908
  If the synchronization2 feature is not enabled, initialLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR or VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkAttachmentDescription2-synchronization2-06909
  If the synchronization2 feature is not enabled, finalLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR or VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkAttachmentDescription2-attachmentFeedbackLoopLayout-07309
  If the attachmentFeedbackLoopLayout feature is not enabled, initialLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT
- VUID-VkAttachmentDescription2-attachmentFeedbackLoopLayout-07310
  If the attachmentFeedbackLoopLayout feature is not enabled, finalLayout **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT
- VUID-VkAttachmentDescription2-samples-08745
  samples **must** be a valid VkSampleCountFlagBits value that is set in imageCreateSampleCounts (as defined in Image Creation Limits) for the given format
- VUID-VkAttachmentDescription2-dynamicRenderingLocalRead-09544
  If the dynamicRenderingLocalRead feature is not enabled, initialLayout **must** not be VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR
- VUID-VkAttachmentDescription2-dynamicRenderingLocalRead-09545

If the dynamicRenderingLocalRead feature is not enabled, finalLayout **must** not be VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR

- VUID-VkAttachmentDescription2-pNext-06704
  If the pNext chain does not include a VkAttachmentDescriptionStencilLayout structure, format includes a stencil component, and stencilLoadOp is VK_ATTACHMENT_LOAD_OP_LOAD, then initialLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED
- VUID-VkAttachmentDescription2-pNext-06705
  If the pNext chain includes a VkAttachmentDescriptionStencilLayout structure, format includes a stencil component, and stencilLoadOp is VK_ATTACHMENT_LOAD_OP_LOAD, then VkAttachmentDescriptionStencilLayout::stencilInitialLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED
- VUID-VkAttachmentDescription2-format-06249
  If format is a depth/stencil format which includes both depth and stencil components, and initialLayout is VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, the pNext chain **must** include a VkAttachmentDescriptionStencilLayout structure
- VUID-VkAttachmentDescription2-format-06250
  If format is a depth/stencil format which includes both depth and stencil components, and finalLayout is VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, the pNext chain **must** include a VkAttachmentDescriptionStencilLayout structure
- VUID-VkAttachmentDescription2-format-06247
  If the pNext chain does not include a VkAttachmentDescriptionStencilLayout structure and format only includes a stencil component, initialLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-format-06248
  If the pNext chain does not include a VkAttachmentDescriptionStencilLayout structure and format only includes a stencil component, finalLayout **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL
- VUID-VkAttachmentDescription2-format-09332
  format **must** not be VK_FORMAT_UNDEFINED

Valid Usage (Implicit)

- VUID-VkAttachmentDescription2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_2
- VUID-VkAttachmentDescription2-pNext-pNext
  pNext **must** be NULL or a pointer to a valid instance of VkAttachmentDescriptionStencilLayout
- VUID-VkAttachmentDescription2-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkAttachmentDescription2-flags-parameter
  flags **must** be a valid combination of VkAttachmentDescriptionFlagBits values

- VUID-VkAttachmentDescription2-format-parameter
  `format` **must** be a valid VkFormat value
- VUID-VkAttachmentDescription2-samples-parameter
  `samples` **must** be a valid VkSampleCountFlagBits value
- VUID-VkAttachmentDescription2-loadOp-parameter
  `loadOp` **must** be a valid VkAttachmentLoadOp value
- VUID-VkAttachmentDescription2-storeOp-parameter
  `storeOp` **must** be a valid VkAttachmentStoreOp value
- VUID-VkAttachmentDescription2-stencilLoadOp-parameter
  `stencilLoadOp` **must** be a valid VkAttachmentLoadOp value
- VUID-VkAttachmentDescription2-stencilStoreOp-parameter
  `stencilStoreOp` **must** be a valid VkAttachmentStoreOp value
- VUID-VkAttachmentDescription2-initialLayout-parameter
  `initialLayout` **must** be a valid VkImageLayout value
- VUID-VkAttachmentDescription2-finalLayout-parameter
  `finalLayout` **must** be a valid VkImageLayout value

The `VkAttachmentDescriptionStencilLayout` structure is defined as:

```
typedef struct VkAttachmentDescriptionStencilLayout {
    VkStructureType    sType;
    void*              pNext;
    VkImageLayout      stencilInitialLayout;
    VkImageLayout      stencilFinalLayout;
} VkAttachmentDescriptionStencilLayout;
```

or the equivalent

```
// Provided by VK_KHR_separate_depth_stencil_layouts
typedef VkAttachmentDescriptionStencilLayout VkAttachmentDescriptionStencilLayoutKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `stencilInitialLayout` is the layout the stencil aspect of the attachment image subresource will be in when a render pass instance begins.
- `stencilFinalLayout` is the layout the stencil aspect of the attachment image subresource will be transitioned to when a render pass instance ends.

Valid Usage

- VUID-VkAttachmentDescriptionStencilLayout-stencilInitialLayout-03308
  `stencilInitialLayout` **must** not be `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`,

VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL, or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL

- VUID-VkAttachmentDescriptionStencilLayout-stencilFinalLayout-03309
  stencilFinalLayout **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL, or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkAttachmentDescriptionStencilLayout-stencilFinalLayout-03310
  stencilFinalLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED

Valid Usage (Implicit)

- VUID-VkAttachmentDescriptionStencilLayout-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_STENCIL_LAYOUT
- VUID-VkAttachmentDescriptionStencilLayout-stencilInitialLayout-parameter
  stencilInitialLayout **must** be a valid VkImageLayout value
- VUID-VkAttachmentDescriptionStencilLayout-stencilFinalLayout-parameter
  stencilFinalLayout **must** be a valid VkImageLayout value

The VkSubpassDescription2 structure is defined as:

```
typedef struct VkSubpassDescription2 {
    VkStructureType                 sType;
    const void*                     pNext;
    VkSubpassDescriptionFlags       flags;
    VkPipelineBindPoint             pipelineBindPoint;
    uint32_t                        viewMask;
    uint32_t                        inputAttachmentCount;
    const VkAttachmentReference2*   pInputAttachments;
    uint32_t                        colorAttachmentCount;
    const VkAttachmentReference2*   pColorAttachments;
    const VkAttachmentReference2*   pResolveAttachments;
    const VkAttachmentReference2*   pDepthStencilAttachment;
    uint32_t                        preserveAttachmentCount;
    const uint32_t*                 pPreserveAttachments;
} VkSubpassDescription2;
```

or the equivalent

```
// Provided by VK_KHR_create_renderpass2
typedef VkSubpassDescription2 VkSubpassDescription2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkSubpassDescriptionFlagBits specifying usage of the subpass.
- `pipelineBindPoint` is a VkPipelineBindPoint value specifying the pipeline type supported for this subpass.
- `viewMask` is a bitfield of view indices describing which views rendering is broadcast to in this subpass, when multiview is enabled.
- `inputAttachmentCount` is the number of input attachments.
- `pInputAttachments` is a pointer to an array of VkAttachmentReference2 structures defining the input attachments for this subpass and their layouts.
- `colorAttachmentCount` is the number of color attachments.
- `pColorAttachments` is a pointer to an array of `colorAttachmentCount` VkAttachmentReference2 structures defining the color attachments for this subpass and their layouts.
- `pResolveAttachments` is `NULL` or a pointer to an array of `colorAttachmentCount` VkAttachmentReference2 structures defining the resolve attachments for this subpass and their layouts.
- `pDepthStencilAttachment` is a pointer to a VkAttachmentReference2 structure specifying the depth/stencil attachment for this subpass and its layout.
- `preserveAttachmentCount` is the number of preserved attachments.
- `pPreserveAttachments` is a pointer to an array of `preserveAttachmentCount` render pass attachment indices identifying attachments that are not used by this subpass, but whose contents **must** be preserved throughout the subpass.

Parameters defined by this structure with the same name as those in VkSubpassDescription have the identical effect to those parameters.

`viewMask` has the same effect for the described subpass as VkRenderPassMultiviewCreateInfo ::`pViewMasks` has on each corresponding subpass.

If a VkFragmentShadingRateAttachmentInfoKHR structure is included in the `pNext` chain, `pFragmentShadingRateAttachment` is not `NULL`, and its `attachment` member is not `VK_ATTACHMENT_UNUSED`, the identified attachment defines a fragment shading rate attachment for that subpass.

Valid Usage

- VUID-VkSubpassDescription2-attachment-06912
  If the `attachment` member of an element of `pInputAttachments` is not `VK_ATTACHMENT_UNUSED`, its `layout` member **must** not be `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL`

- VUID-VkSubpassDescription2-attachment-06913
  If the attachment member of an element of pColorAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL
- VUID-VkSubpassDescription2-attachment-06914
  If the attachment member of an element of pResolveAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL
- VUID-VkSubpassDescription2-attachment-06915
  If the attachment member of pDepthStencilAttachment is not VK_ATTACHMENT_UNUSED, ts layout member **must** not be VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL
- VUID-VkSubpassDescription2-attachment-06916
  If the attachment member of an element of pColorAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkSubpassDescription2-attachment-06917
  If the attachment member of an element of pResolveAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL or VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkSubpassDescription2-attachment-06918
  If the attachment member of an element of pInputAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL
- VUID-VkSubpassDescription2-attachment-06919
  If the attachment member of an element of pColorAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkSubpassDescription2-attachment-06920
  If the attachment member of an element of pResolveAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkSubpassDescription2-attachment-06921
  If the attachment member of an element of pInputAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR
- VUID-VkSubpassDescription2-attachment-06922
  If the attachment member of an element of pColorAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR

- VUID-VkSubpassDescription2-attachment-06923
  If the attachment member of an element of pResolveAttachments is not VK_ATTACHMENT_UNUSED, its layout member **must** not be VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VUID-VkSubpassDescription2-attachment-06251
  If the attachment member of pDepthStencilAttachment is not VK_ATTACHMENT_UNUSED and its pNext chain includes a VkAttachmentReferenceStencilLayout structure, the layout member of pDepthStencilAttachment **must** not be VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VUID-VkSubpassDescription2-pipelineBindPoint-04953
  pipelineBindPoint **must** be VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-VkSubpassDescription2-colorAttachmentCount-03063
  colorAttachmentCount **must** be less than or equal to VkPhysicalDeviceLimits ::maxColorAttachments
- VUID-VkSubpassDescription2-loadOp-03064
  If the first use of an attachment in this render pass is as an input attachment, and the attachment is not also used as a color or depth/stencil attachment in the same subpass, then loadOp **must** not be VK_ATTACHMENT_LOAD_OP_CLEAR
- VUID-VkSubpassDescription2-pResolveAttachments-03065
  If pResolveAttachments is not NULL, for each resolve attachment that does not have the value VK_ATTACHMENT_UNUSED, the corresponding color attachment **must** not have the value VK_ATTACHMENT_UNUSED
- VUID-VkSubpassDescription2-pResolveAttachments-03066
  If pResolveAttachments is not NULL, for each resolve attachment that is not VK_ATTACHMENT_UNUSED, the corresponding color attachment **must** not have a sample count of VK_SAMPLE_COUNT_1_BIT
- VUID-VkSubpassDescription2-pResolveAttachments-03068
  Each element of pResolveAttachments **must** have the same VkFormat as its corresponding color attachment
- VUID-VkSubpassDescription2-pResolveAttachments-03067
  If pResolveAttachments is not NULL, each resolve attachment that is not VK_ATTACHMENT_UNUSED **must** have a sample count of VK_SAMPLE_COUNT_1_BIT
- VUID-VkSubpassDescription2-pInputAttachments-02897
  All attachments in pInputAttachments that are not VK_ATTACHMENT_UNUSED

  must have image formats whose potential format features contain at least VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT or VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkSubpassDescription2-pColorAttachments-02898
  All attachments in pColorAttachments that are not VK_ATTACHMENT_UNUSED **must** have image formats whose potential format features contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT
- VUID-VkSubpassDescription2-pResolveAttachments-02899
  All attachments in pResolveAttachments that are not VK_ATTACHMENT_UNUSED **must** have

image formats whose potential format features contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT

- VUID-VkSubpassDescription2-pDepthStencilAttachment-02900
  If pDepthStencilAttachment is not NULL and the attachment is not VK_ATTACHMENT_UNUSED then it **must** have an image format whose potential format features contain VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkSubpassDescription2-multisampledRenderToSingleSampled-06872
  All attachments in pDepthStencilAttachment or pColorAttachments that are not VK_ATTACHMENT_UNUSED **must** have the same sample count
- VUID-VkSubpassDescription2-attachment-03073
  Each element of pPreserveAttachments **must** not be VK_ATTACHMENT_UNUSED
- VUID-VkSubpassDescription2-pPreserveAttachments-03074
  Each element of pPreserveAttachments **must** not also be an element of any other member of the subpass description
- VUID-VkSubpassDescription2-layout-02528
  If any attachment is used by more than one VkAttachmentReference2 member, then each use **must** use the same layout
- VUID-VkSubpassDescription2-attachment-02799
  If the attachment member of any element of pInputAttachments is not VK_ATTACHMENT_UNUSED, then the aspectMask member **must** be a valid combination of VkImageAspectFlagBits
- VUID-VkSubpassDescription2-attachment-02800
  If the attachment member of any element of pInputAttachments is not VK_ATTACHMENT_UNUSED, then the aspectMask member **must** not be 0
- VUID-VkSubpassDescription2-attachment-02801
  If the attachment member of any element of pInputAttachments is not VK_ATTACHMENT_UNUSED, then the aspectMask member **must** not include VK_IMAGE_ASPECT_METADATA_BIT
- VUID-VkSubpassDescription2-attachment-04563
  If the attachment member of any element of pInputAttachments is not VK_ATTACHMENT_UNUSED, then the aspectMask member **must** not include VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT for any index *i*
- VUID-VkSubpassDescription2-pDepthStencilAttachment-04440
  An attachment **must** not be used in both pDepthStencilAttachment and pColorAttachments
- VUID-VkSubpassDescription2-multiview-06558
  If the multiview feature is not enabled, viewMask **must** be 0
- VUID-VkSubpassDescription2-viewMask-06706
  The index of the most significant bit in viewMask **must** be less than maxMultiviewViewCount

Valid Usage (Implicit)

- VUID-VkSubpassDescription2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_2
- VUID-VkSubpassDescription2-pNext-pNext

Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkFragmentShadingRateAttachmentInfoKHR or VkSubpassDescriptionDepthStencilResolve

- VUID-VkSubpassDescription2-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkSubpassDescription2-flags-zerobitmask
  flags **must** be 0
- VUID-VkSubpassDescription2-pipelineBindPoint-parameter
  pipelineBindPoint **must** be a valid VkPipelineBindPoint value
- VUID-VkSubpassDescription2-pInputAttachments-parameter
  If inputAttachmentCount is not 0, pInputAttachments **must** be a valid pointer to an array of inputAttachmentCount valid VkAttachmentReference2 structures
- VUID-VkSubpassDescription2-pColorAttachments-parameter
  If colorAttachmentCount is not 0, pColorAttachments **must** be a valid pointer to an array of colorAttachmentCount valid VkAttachmentReference2 structures
- VUID-VkSubpassDescription2-pResolveAttachments-parameter
  If colorAttachmentCount is not 0, and pResolveAttachments is not NULL, pResolveAttachments **must** be a valid pointer to an array of colorAttachmentCount valid VkAttachmentReference2 structures
- VUID-VkSubpassDescription2-pDepthStencilAttachment-parameter
  If pDepthStencilAttachment is not NULL, pDepthStencilAttachment **must** be a valid pointer to a valid VkAttachmentReference2 structure
- VUID-VkSubpassDescription2-pPreserveAttachments-parameter
  If preserveAttachmentCount is not 0, pPreserveAttachments **must** be a valid pointer to an array of preserveAttachmentCount uint32_t values

The VkSubpassDescriptionDepthStencilResolve structure is defined as:

```
typedef struct VkSubpassDescriptionDepthStencilResolve {
    VkStructureType                  sType;
    const void*                      pNext;
    VkResolveModeFlagBits            depthResolveMode;
    VkResolveModeFlagBits            stencilResolveMode;
    const VkAttachmentReference2*    pDepthStencilResolveAttachment;
} VkSubpassDescriptionDepthStencilResolve;
```

or the equivalent

```
// Provided by VK_KHR_depth_stencil_resolve
typedef VkSubpassDescriptionDepthStencilResolve
VkSubpassDescriptionDepthStencilResolveKHR;
```

- sType is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `depthResolveMode` is a VkResolveModeFlagBits value describing the depth resolve mode.
- `stencilResolveMode` is a VkResolveModeFlagBits value describing the stencil resolve mode.
- `pDepthStencilResolveAttachment` is `NULL` or a pointer to a VkAttachmentReference2 structure defining the depth/stencil resolve attachment for this subpass and its layout.

If the `pNext` chain of VkSubpassDescription2 includes a `VkSubpassDescriptionDepthStencilResolve` structure, then that structure describes multisample resolve operations for the depth/stencil attachment in a subpass. If this structure is not included in the `pNext` chain of VkSubpassDescription2, or if it is and either `pDepthStencilResolveAttachment` is `NULL` or its attachment index is `VK_ATTACHMENT_UNUSED`, it indicates that no depth/stencil resolve attachment will be used in the subpass.

Valid Usage

- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-03177
  If `pDepthStencilResolveAttachment` is not `NULL` and does not have the value `VK_ATTACHMENT_UNUSED`, `pDepthStencilAttachment` **must** not be `NULL` or have the value `VK_ATTACHMENT_UNUSED`
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-03179
  If `pDepthStencilResolveAttachment` is not `NULL` and does not have the value `VK_ATTACHMENT_UNUSED`, `pDepthStencilAttachment` **must** not have a sample count of `VK_SAMPLE_COUNT_1_BIT`
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-03180
  If `pDepthStencilResolveAttachment` is not `NULL` and does not have the value `VK_ATTACHMENT_UNUSED`, `pDepthStencilResolveAttachment` **must** have a sample count of `VK_SAMPLE_COUNT_1_BIT`
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-02651
  If `pDepthStencilResolveAttachment` is not `NULL` and does not have the value `VK_ATTACHMENT_UNUSED` then it **must** have an image format whose potential format features contain `VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-03181
  If `pDepthStencilResolveAttachment` is not `NULL` and does not have the value `VK_ATTACHMENT_UNUSED` and VkFormat of `pDepthStencilResolveAttachment` has a depth component, then the VkFormat of `pDepthStencilAttachment` **must** have a depth component with the same number of bits and numeric format
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-03182
  If `pDepthStencilResolveAttachment` is not `NULL` and does not have the value `VK_ATTACHMENT_UNUSED`, and VkFormat of `pDepthStencilResolveAttachment` has a stencil component, then the VkFormat of `pDepthStencilAttachment` **must** have a stencil component with the same number of bits and numeric format
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-03178
  If `pDepthStencilResolveAttachment` is not `NULL` and does not have the value `VK_ATTACHMENT_UNUSED`, `depthResolveMode` and `stencilResolveMode` **must** not both be

VK_RESOLVE_MODE_NONE

- VUID-VkSubpassDescriptionDepthStencilResolve-depthResolveMode-03183
  If pDepthStencilResolveAttachment is not NULL and does not have the value VK_ATTACHMENT_UNUSED and the VkFormat of pDepthStencilResolveAttachment has a depth component, then the value of depthResolveMode **must** be one of the bits set in VkPhysicalDeviceDepthStencilResolveProperties::supportedDepthResolveModes or VK_RESOLVE_MODE_NONE
- VUID-VkSubpassDescriptionDepthStencilResolve-stencilResolveMode-03184
  If pDepthStencilResolveAttachment is not NULL and does not have the value VK_ATTACHMENT_UNUSED and the VkFormat of pDepthStencilResolveAttachment has a stencil component, then the value of stencilResolveMode **must** be one of the bits set in VkPhysicalDeviceDepthStencilResolveProperties::supportedStencilResolveModes or VK_RESOLVE_MODE_NONE
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-03185
  If pDepthStencilResolveAttachment is not NULL and does not have the value VK_ATTACHMENT_UNUSED, the VkFormat of pDepthStencilResolveAttachment has both depth and stencil components, VkPhysicalDeviceDepthStencilResolveProperties::independentResolve is VK_FALSE, and VkPhysicalDeviceDepthStencilResolveProperties::independentResolveNone is VK_FALSE, then the values of depthResolveMode and stencilResolveMode **must** be identical
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-03186
  If pDepthStencilResolveAttachment is not NULL and does not have the value VK_ATTACHMENT_UNUSED, the VkFormat of pDepthStencilResolveAttachment has both depth and stencil components, VkPhysicalDeviceDepthStencilResolveProperties::independentResolve is VK_FALSE and VkPhysicalDeviceDepthStencilResolveProperties::independentResolveNone is VK_TRUE, then the values of depthResolveMode and stencilResolveMode **must** be identical or one of them **must** be VK_RESOLVE_MODE_NONE

Valid Usage (Implicit)

- VUID-VkSubpassDescriptionDepthStencilResolve-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_DEPTH_STENCIL_RESOLVE
- VUID-VkSubpassDescriptionDepthStencilResolve-pDepthStencilResolveAttachment-parameter
  If pDepthStencilResolveAttachment is not NULL, pDepthStencilResolveAttachment **must** be a valid pointer to a valid VkAttachmentReference2 structure

The VkFragmentShadingRateAttachmentInfoKHR structure is defined as:

```
// Provided by VK_KHR_fragment_shading_rate
typedef struct VkFragmentShadingRateAttachmentInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    const VkAttachmentReference2*   pFragmentShadingRateAttachment;
    VkExtent2D                      shadingRateAttachmentTexelSize;
} VkFragmentShadingRateAttachmentInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pFragmentShadingRateAttachment` is `NULL` or a pointer to a VkAttachmentReference2 structure defining the fragment shading rate attachment for this subpass.
- `shadingRateAttachmentTexelSize` specifies the size of the portion of the framebuffer corresponding to each texel in `pFragmentShadingRateAttachment`.

If no shading rate attachment is specified, or if this structure is not specified, the implementation behaves as if a valid shading rate attachment was specified with all texels specifying a single pixel per fragment.

Valid Usage

- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04524
  If `pFragmentShadingRateAttachment` is not `NULL` and its `attachment` member is not `VK_ATTACHMENT_UNUSED`, its `layout` member **must** be equal to `VK_IMAGE_LAYOUT_GENERAL` or `VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR`
- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04525
  If `pFragmentShadingRateAttachment` is not `NULL` and its `attachment` member is not `VK_ATTACHMENT_UNUSED`, `shadingRateAttachmentTexelSize.width` **must** be a power of two value
- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04526
  If `pFragmentShadingRateAttachment` is not `NULL` and its `attachment` member is not `VK_ATTACHMENT_UNUSED`, `shadingRateAttachmentTexelSize.width` **must** be less than or equal to `maxFragmentShadingRateAttachmentTexelSize.width`
- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04527
  If `pFragmentShadingRateAttachment` is not `NULL` and its `attachment` member is not `VK_ATTACHMENT_UNUSED`, `shadingRateAttachmentTexelSize.width` **must** be greater than or equal to `minFragmentShadingRateAttachmentTexelSize.width`
- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04528
  If `pFragmentShadingRateAttachment` is not `NULL` and its `attachment` member is not

VK_ATTACHMENT_UNUSED, shadingRateAttachmentTexelSize.height **must** be a power of two value

- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04529
  If pFragmentShadingRateAttachment is not NULL and its attachment member is not VK_ATTACHMENT_UNUSED, shadingRateAttachmentTexelSize.height **must** be less than or equal to maxFragmentShadingRateAttachmentTexelSize.height
- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04530
  If pFragmentShadingRateAttachment is not NULL and its attachment member is not VK_ATTACHMENT_UNUSED, shadingRateAttachmentTexelSize.height **must** be greater than or equal to minFragmentShadingRateAttachmentTexelSize.height
- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04531
  If pFragmentShadingRateAttachment is not NULL and its attachment member is not VK_ATTACHMENT_UNUSED, the quotient of shadingRateAttachmentTexelSize.width and shadingRateAttachmentTexelSize.height **must** be less than or equal to maxFragmentShadingRateAttachmentTexelSizeAspectRatio
- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-04532
  If pFragmentShadingRateAttachment is not NULL and its attachment member is not VK_ATTACHMENT_UNUSED, the quotient of shadingRateAttachmentTexelSize.height and shadingRateAttachmentTexelSize.width **must** be less than or equal to maxFragmentShadingRateAttachmentTexelSizeAspectRatio

Valid Usage (Implicit)

- VUID-VkFragmentShadingRateAttachmentInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_FRAGMENT_SHADING_RATE_ATTACHMENT_INFO_KHR
- VUID-VkFragmentShadingRateAttachmentInfoKHR-pFragmentShadingRateAttachment-parameter
  If pFragmentShadingRateAttachment is not NULL, pFragmentShadingRateAttachment **must** be a valid pointer to a valid VkAttachmentReference2 structure

The VkAttachmentReference2 structure is defined as:

```
typedef struct VkAttachmentReference2 {
    VkStructureType       sType;
    const void*           pNext;
    uint32_t              attachment;
    VkImageLayout         layout;
    VkImageAspectFlags    aspectMask;
} VkAttachmentReference2;
```

or the equivalent

```
// Provided by VK_KHR_create_renderpass2
typedef VkAttachmentReference2 VkAttachmentReference2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `attachment` is either an integer value identifying an attachment at the corresponding index in VkRenderPassCreateInfo2::`pAttachments`, or `VK_ATTACHMENT_UNUSED` to signify that this attachment is not used.
- `layout` is a VkImageLayout value specifying the layout the attachment uses during the subpass.
- `aspectMask` is a mask of which aspect(s) **can** be accessed within the specified subpass as an input attachment.

Parameters defined by this structure with the same name as those in VkAttachmentReference have the identical effect to those parameters.

`aspectMask` is ignored when this structure is used to describe anything other than an input attachment reference.

If the `separateDepthStencilLayouts` feature is enabled, and `attachment` has a depth/stencil format, `layout` **can** be set to a layout that only specifies the layout of the depth aspect.

If `layout` only specifies the layout of the depth aspect of the attachment, the layout of the stencil aspect is specified by the `stencilLayout` member of a VkAttachmentReferenceStencilLayout structure included in the `pNext` chain. Otherwise, `layout` describes the layout for all relevant image aspects.

Valid Usage

- VUID-VkAttachmentReference2-layout-03077
  If `attachment` is not `VK_ATTACHMENT_UNUSED`, `layout` **must** not be `VK_IMAGE_LAYOUT_UNDEFINED`, `VK_IMAGE_LAYOUT_PREINITIALIZED`, or `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR`
- VUID-VkAttachmentReference2-separateDepthStencilLayouts-03313
  If the `separateDepthStencilLayouts` feature is not enabled, and `attachment` is not `VK_ATTACHMENT_UNUSED`, `layout` **must** not be `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`,
- VUID-VkAttachmentReference2-synchronization2-06910
  If the `synchronization2` feature is not enabled, `layout` **must** not be `VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR` or `VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR`
- VUID-VkAttachmentReference2-attachmentFeedbackLoopLayout-07311
  If the `attachmentFeedbackLoopLayout` feature is not enabled, `layout` **must** not be `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT`

- VUID-VkAttachmentReference2-dynamicRenderingLocalRead-09546
  If the dynamicRenderingLocalRead feature is not enabled, layout **must** not be VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR

Valid Usage (Implicit)

- VUID-VkAttachmentReference2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_2
- VUID-VkAttachmentReference2-pNext-pNext
  pNext **must** be NULL or a pointer to a valid instance of VkAttachmentReferenceStencilLayout
- VUID-VkAttachmentReference2-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkAttachmentReference2-layout-parameter
  layout **must** be a valid VkImageLayout value

The VkAttachmentReferenceStencilLayout structure is defined as:

```
typedef struct VkAttachmentReferenceStencilLayout {
    VkStructureType    sType;
    void*              pNext;
    VkImageLayout      stencilLayout;
} VkAttachmentReferenceStencilLayout;
```

or the equivalent

```
// Provided by VK_KHR_separate_depth_stencil_layouts
typedef VkAttachmentReferenceStencilLayout VkAttachmentReferenceStencilLayoutKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- stencilLayout is a VkImageLayout value specifying the layout the stencil aspect of the attachment uses during the subpass.

Valid Usage

- VUID-VkAttachmentReferenceStencilLayout-stencilLayout-03318
  stencilLayout **must** not be VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_PREINITIALIZED, VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL,

```
    VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL,
    VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL,                or
    VK_IMAGE_LAYOUT_PRESENT_SRC_KHR
```

Valid Usage (Implicit)

- VUID-VkAttachmentReferenceStencilLayout-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_STENCIL_LAYOUT`
- VUID-VkAttachmentReferenceStencilLayout-stencilLayout-parameter
  `stencilLayout` **must** be a valid VkImageLayout value

The `VkSubpassDependency2` structure is defined as:

```
typedef struct VkSubpassDependency2 {
    VkStructureType         sType;
    const void*             pNext;
    uint32_t                srcSubpass;
    uint32_t                dstSubpass;
    VkPipelineStageFlags    srcStageMask;
    VkPipelineStageFlags    dstStageMask;
    VkAccessFlags           srcAccessMask;
    VkAccessFlags           dstAccessMask;
    VkDependencyFlags       dependencyFlags;
    int32_t                 viewOffset;
} VkSubpassDependency2;
```

or the equivalent

```
// Provided by VK_KHR_create_renderpass2
typedef VkSubpassDependency2 VkSubpassDependency2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcSubpass` is the subpass index of the first subpass in the dependency, or `VK_SUBPASS_EXTERNAL`.
- `dstSubpass` is the subpass index of the second subpass in the dependency, or `VK_SUBPASS_EXTERNAL`.
- `srcStageMask` is a bitmask of VkPipelineStageFlagBits specifying the source stage mask.
- `dstStageMask` is a bitmask of VkPipelineStageFlagBits specifying the destination stage mask
- `srcAccessMask` is a bitmask of VkAccessFlagBits specifying a source access mask.
- `dstAccessMask` is a bitmask of VkAccessFlagBits specifying a destination access mask.
- `dependencyFlags` is a bitmask of VkDependencyFlagBits.

- `viewOffset` controls which views in the source subpass the views in the destination subpass depend on.

Parameters defined by this structure with the same name as those in VkSubpassDependency have the identical effect to those parameters.

`viewOffset` has the same effect for the described subpass dependency as VkRenderPassMultiviewCreateInfo::`pViewOffsets` has on each corresponding subpass dependency.

If a VkMemoryBarrier2 is included in the `pNext` chain, `srcStageMask`, `dstStageMask`, `srcAccessMask`, and `dstAccessMask` parameters are ignored. The synchronization and access scopes instead are defined by the parameters of VkMemoryBarrier2.

Valid Usage

- VUID-VkSubpassDependency2-srcStageMask-04090
  If the `geometryShader` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT`
- VUID-VkSubpassDependency2-srcStageMask-04091
  If the `tessellationShader` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-VkSubpassDependency2-srcStageMask-04093
  If the `fragmentDensityMap` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- VUID-VkSubpassDependency2-srcStageMask-04094
  If the `transformFeedback` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT`
- VUID-VkSubpassDependency2-srcStageMask-07319
  If the `attachmentFragmentShadingRate` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-VkSubpassDependency2-srcStageMask-03937
  If the `synchronization2` feature is not enabled, `srcStageMask` **must** not be `0`
- VUID-VkSubpassDependency2-srcStageMask-07950
  If the `rayTracingPipeline` feature is not enabled, `srcStageMask` **must** not contain `VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR`
- VUID-VkSubpassDependency2-dstStageMask-04090
  If the `geometryShader` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT`
- VUID-VkSubpassDependency2-dstStageMask-04091
  If the `tessellationShader` feature is not enabled, `dstStageMask` **must** not contain `VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-VkSubpassDependency2-dstStageMask-04093

If the fragmentDensityMap feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT

- VUID-VkSubpassDependency2-dstStageMask-04094
  If the transformFeedback feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
- VUID-VkSubpassDependency2-dstStageMask-07319
  If the attachmentFragmentShadingRate feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkSubpassDependency2-dstStageMask-03937
  If the synchronization2 feature is not enabled, dstStageMask **must** not be 0
- VUID-VkSubpassDependency2-dstStageMask-07950
  If the rayTracingPipeline feature is not enabled, dstStageMask **must** not contain VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- VUID-VkSubpassDependency2-srcSubpass-03084
  srcSubpass **must** be less than or equal to dstSubpass, unless one of them is VK_SUBPASS_EXTERNAL, to avoid cyclic dependencies and ensure a valid execution order
- VUID-VkSubpassDependency2-srcSubpass-03085
  srcSubpass and dstSubpass **must** not both be equal to VK_SUBPASS_EXTERNAL
- VUID-VkSubpassDependency2-srcSubpass-06810
  If srcSubpass is equal to dstSubpass and srcStageMask includes a framebuffer-space stage, dstStageMask **must** only contain framebuffer-space stages
- VUID-VkSubpassDependency2-srcAccessMask-03088
  Any access flag included in srcAccessMask **must** be supported by one of the pipeline stages in srcStageMask, as specified in the table of supported access types
- VUID-VkSubpassDependency2-dstAccessMask-03089
  Any access flag included in dstAccessMask **must** be supported by one of the pipeline stages in dstStageMask, as specified in the table of supported access types
- VUID-VkSubpassDependency2-dependencyFlags-03090
  If dependencyFlags includes VK_DEPENDENCY_VIEW_LOCAL_BIT, srcSubpass **must** not be equal to VK_SUBPASS_EXTERNAL
- VUID-VkSubpassDependency2-dependencyFlags-03091
  If dependencyFlags includes VK_DEPENDENCY_VIEW_LOCAL_BIT, dstSubpass **must** not be equal to VK_SUBPASS_EXTERNAL
- VUID-VkSubpassDependency2-srcSubpass-02245
  If srcSubpass equals dstSubpass, and srcStageMask and dstStageMask both include a framebuffer-space stage, then dependencyFlags **must** include VK_DEPENDENCY_BY_REGION_BIT
- VUID-VkSubpassDependency2-viewOffset-02530
  If viewOffset is not equal to 0, srcSubpass **must** not be equal to dstSubpass
- VUID-VkSubpassDependency2-dependencyFlags-03092
  If dependencyFlags does not include VK_DEPENDENCY_VIEW_LOCAL_BIT, viewOffset **must** be 0

Valid Usage (Implicit)

- VUID-VkSubpassDependency2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SUBPASS_DEPENDENCY_2`
- VUID-VkSubpassDependency2-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkMemoryBarrier2
- VUID-VkSubpassDependency2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkSubpassDependency2-srcStageMask-parameter
  `srcStageMask` **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-VkSubpassDependency2-dstStageMask-parameter
  `dstStageMask` **must** be a valid combination of VkPipelineStageFlagBits values
- VUID-VkSubpassDependency2-srcAccessMask-parameter
  `srcAccessMask` **must** be a valid combination of VkAccessFlagBits values
- VUID-VkSubpassDependency2-dstAccessMask-parameter
  `dstAccessMask` **must** be a valid combination of VkAccessFlagBits values
- VUID-VkSubpassDependency2-dependencyFlags-parameter
  `dependencyFlags` **must** be a valid combination of VkDependencyFlagBits values

To destroy a render pass, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyRenderPass(
    VkDevice                                    device,
    VkRenderPass                                renderPass,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the render pass.
- `renderPass` is the handle of the render pass to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyRenderPass-renderPass-00873
  All submitted commands that refer to `renderPass` **must** have completed execution
- VUID-vkDestroyRenderPass-renderPass-00874
  If `VkAllocationCallbacks` were provided when `renderPass` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyRenderPass-renderPass-00875
  If no `VkAllocationCallbacks` were provided when `renderPass` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyRenderPass-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyRenderPass-renderPass-parameter
  If `renderPass` is not VK_NULL_HANDLE, `renderPass` **must** be a valid VkRenderPass handle
- VUID-vkDestroyRenderPass-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyRenderPass-renderPass-parent
  If `renderPass` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `renderPass` **must** be externally synchronized

8.3. Render Pass Compatibility

Framebuffers and graphics pipelines are created based on a specific render pass object. They **must** only be used with that render pass object, or one compatible with it.

Two attachment references are compatible if they have matching format and sample count, or are both `VK_ATTACHMENT_UNUSED` or the pointer that would contain the reference is `NULL`.

Two arrays of attachment references are compatible if all corresponding pairs of attachments are compatible. If the arrays are of different lengths, attachment references not present in the smaller array are treated as `VK_ATTACHMENT_UNUSED`.

Two render passes are compatible if their corresponding color, input, resolve, and depth/stencil attachment references are compatible and if they are otherwise identical except for:

- Initial and final image layout in attachment descriptions
- Load and store operations in attachment descriptions
- Image layout in attachment references

As an additional special case, if two render passes have a single subpass, the resolve attachment reference and depth/stencil resolve mode compatibility requirements are ignored.

A framebuffer is compatible with a render pass if it was created using the same render pass or a compatible render pass.

8.4. Framebuffers

Render passes operate in conjunction with *framebuffers*. Framebuffers represent a collection of specific memory attachments that a render pass instance uses.

Framebuffers are represented by `VkFramebuffer` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkFramebuffer)
```

To create a framebuffer, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateFramebuffer(
    VkDevice                                    device,
    const VkFramebufferCreateInfo*              pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkFramebuffer*                              pFramebuffer);
```

- `device` is the logical device that creates the framebuffer.
- `pCreateInfo` is a pointer to a VkFramebufferCreateInfo structure describing additional information about framebuffer creation.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pFramebuffer` is a pointer to a VkFramebuffer handle in which the resulting framebuffer object is returned.

Valid Usage

- VUID-vkCreateFramebuffer-device-10002
  `device` **must** support at least one queue family with the `VK_QUEUE_GRAPHICS_BIT` capability
- VUID-vkCreateFramebuffer-pCreateInfo-02777
  If `pCreateInfo->flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, and `attachmentCount` is not `0`, each element of `pCreateInfo->pAttachments` **must** have been created on `device`

Valid Usage (Implicit)

- VUID-vkCreateFramebuffer-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateFramebuffer-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkFramebufferCreateInfo structure
- VUID-vkCreateFramebuffer-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid

VkAllocationCallbacks structure

- VUID-vkCreateFramebuffer-pFramebuffer-parameter
  `pFramebuffer` **must** be a valid pointer to a VkFramebuffer handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkFramebufferCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkFramebufferCreateInfo {
    VkStructureType             sType;
    const void*                 pNext;
    VkFramebufferCreateFlags    flags;
    VkRenderPass                renderPass;
    uint32_t                    attachmentCount;
    const VkImageView*          pAttachments;
    uint32_t                    width;
    uint32_t                    height;
    uint32_t                    layers;
} VkFramebufferCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkFramebufferCreateFlagBits
- `renderPass` is a render pass defining what render passes the framebuffer will be compatible with. See Render Pass Compatibility for details.
- `attachmentCount` is the number of attachments.
- `pAttachments` is a pointer to an array of VkImageView handles, each of which will be used as the corresponding attachment in a render pass instance. If `flags` includes `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, this parameter is ignored.
- `width`, `height` and `layers` define the dimensions of the framebuffer. If the render pass uses multiview, then `layers` **must** be one and each attachment requires a number of layers that is greater than the maximum bit index set in the view mask in the subpasses in which it is used.

It is legal for a subpass to use no color or depth/stencil attachments, either because it has no attachment references or because all of them are `VK_ATTACHMENT_UNUSED`. This kind of subpass **can**

use shader side effects such as image stores and atomics to produce an output. In this case, the subpass continues to use the `width`, `height`, and `layers` of the framebuffer to define the dimensions of the rendering area, and the `rasterizationSamples` from each pipeline's VkPipelineMultisampleStateCreateInfo to define the number of samples used in rasterization; however, if VkPhysicalDeviceFeatures::`variableMultisampleRate` is `VK_FALSE`, then all pipelines to be bound with the subpass **must** have the same value for VkPipelineMultisampleStateCreateInfo ::`rasterizationSamples`. In all such cases, `rasterizationSamples` **must** be a valid VkSampleCountFlagBits value that is set in VkPhysicalDeviceLimits ::`framebufferNoAttachmentsSampleCounts`.

Valid Usage

- VUID-VkFramebufferCreateInfo-attachmentCount-00876
  `attachmentCount` **must** be equal to the attachment count specified in `renderPass`
- VUID-VkFramebufferCreateInfo-flags-02778
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT` and `attachmentCount` is not `0`, `pAttachments` **must** be a valid pointer to an array of `attachmentCount` valid VkImageView handles
- VUID-VkFramebufferCreateInfo-pAttachments-00877
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` that is used as a color attachment or resolve attachment by `renderPass` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`
- VUID-VkFramebufferCreateInfo-pAttachments-02633
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` that is used as a depth/stencil attachment by `renderPass` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-VkFramebufferCreateInfo-pAttachments-02634
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` that is used as a depth/stencil resolve attachment by `renderPass` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-VkFramebufferCreateInfo-pAttachments-00879
  If `renderpass` is not VK_NULL_HANDLE, `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` that is used as an input attachment by `renderPass` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT`
- VUID-VkFramebufferCreateInfo-pAttachments-02552
  Each element of `pAttachments` that is used as a fragment density map attachment by `renderPass` **must** not have been created with a `flags` value including `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- VUID-VkFramebufferCreateInfo-renderPass-02553
  If `renderPass` has a fragment density map attachment and the `fragmentDensityMapNonSubsampledImages` feature is not enabled, each element of `pAttachments` **must** have been created with a `flags` value including `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT` unless that element is the fragment density map attachment

- VUID-VkFramebufferCreateInfo-pAttachments-00880
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` **must** have been created with a VkFormat value that matches the VkFormat specified by the corresponding `VkAttachmentDescription` in `renderPass`
- VUID-VkFramebufferCreateInfo-pAttachments-00881
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` **must** have been created with a `samples` value that matches the `samples` value specified by the corresponding `VkAttachmentDescription` in `renderPass`
- VUID-VkFramebufferCreateInfo-flags-04533
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` that is used as an input, color, resolve, or depth/stencil attachment by `renderPass` **must** have been created with a VkImageCreateInfo::`extent.width` greater than or equal to `width`
- VUID-VkFramebufferCreateInfo-flags-04534
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` that is used as an input, color, resolve, or depth/stencil attachment by `renderPass` **must** have been created with a VkImageCreateInfo::`extent.height` greater than or equal to `height`
- VUID-VkFramebufferCreateInfo-flags-04535
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` that is used as an input, color, resolve, or depth/stencil attachment by `renderPass` **must** have been created with a VkImageViewCreateInfo ::`subresourceRange.layerCount` greater than or equal to `layers`
- VUID-VkFramebufferCreateInfo-renderPass-04536
  If `renderPass` was specified with non-zero view masks, each element of `pAttachments` that is used as an input, color, resolve, or depth/stencil attachment by `renderPass` **must** have a `layerCount` greater than the index of the most significant bit set in any of those view masks
- VUID-VkFramebufferCreateInfo-renderPass-02746
  Each element of `pAttachments` that is referenced by `fragmentDensityMapAttachment` **must** have a `layerCount` equal to `1` or if `renderPass` was specified with non-zero view masks, greater than the index of the most significant bit set in any of those view masks
- VUID-VkFramebufferCreateInfo-pAttachments-02555
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, an element of `pAttachments` that is referenced by `fragmentDensityMapAttachment` **must** have a width at least as large as $\lceil \frac{width}{maxFragmentDensityTexelSize_{width}} \rceil$
- VUID-VkFramebufferCreateInfo-pAttachments-02556
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, an element of `pAttachments` that is referenced by `fragmentDensityMapAttachment` **must** have a height at least as large as $\lceil \frac{height}{maxFragmentDensityTexelSize_{height}} \rceil$
- VUID-VkFramebufferCreateInfo-flags-04537
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, and `renderPass` was specified with non-zero view masks, each element of `pAttachments` that is used as a fragment shading rate attachment by `renderPass` **must** have a `layerCount` that is either `1`, or greater than the index of the most significant bit set in any of those view masks

- VUID-VkFramebufferCreateInfo-flags-04538
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, and `renderPass` was not specified with non-zero view masks, each element of `pAttachments` that is used as a fragment shading rate attachment by `renderPass` **must** have a `layerCount` that is either `1`, or greater than `layers`
- VUID-VkFramebufferCreateInfo-flags-04539
  If the `maintenance7` feature is not enabled or the `robustFragmentShadingRateAttachmentAccess` limit is `VK_FALSE` or the `imageView` member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure was created with VkImageSubresourceRange::`baseMipLevel` greater than 0, `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, an element of `pAttachments` that is used as a fragment shading rate attachment **must** have a width at least as large as ⌈`width` / `texelWidth`⌉, where `texelWidth` is the largest value of `shadingRateAttachmentTexelSize.width` in a VkFragmentShadingRateAttachmentInfoKHR which references that attachment
- VUID-VkFramebufferCreateInfo-flags-04540
  If the `maintenance7` feature is not enabled or the `robustFragmentShadingRateAttachmentAccess` limit is `VK_FALSE` or the `imageView` member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure was created with VkImageSubresourceRange::`baseMipLevel` greater than 0, `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, an element of `pAttachments` that is used as a fragment shading rate attachment **must** have a height at least as large as ⌈`height` / `texelHeight`⌉, where `texelHeight` is the largest value of `shadingRateAttachmentTexelSize.height` in a VkFragmentShadingRateAttachmentInfoKHR which references that attachment
- VUID-VkFramebufferCreateInfo-pAttachments-00883
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` **must** only specify a single mip level
- VUID-VkFramebufferCreateInfo-pAttachments-00884
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` **must** have been created with the identity swizzle
- VUID-VkFramebufferCreateInfo-width-00885
  `width` **must** be greater than `0`
- VUID-VkFramebufferCreateInfo-width-00886
  `width` **must** be less than or equal to `maxFramebufferWidth`
- VUID-VkFramebufferCreateInfo-height-00887
  `height` **must** be greater than `0`
- VUID-VkFramebufferCreateInfo-height-00888
  `height` **must** be less than or equal to `maxFramebufferHeight`
- VUID-VkFramebufferCreateInfo-layers-00889
  `layers` **must** be greater than `0`
- VUID-VkFramebufferCreateInfo-layers-00890
  `layers` **must** be less than or equal to `maxFramebufferLayers`

- VUID-VkFramebufferCreateInfo-renderPass-02531
  If `renderPass` was specified with non-zero view masks, `layers` **must** be `1`
- VUID-VkFramebufferCreateInfo-pAttachments-00891
  If `flags` does not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, each element of `pAttachments` that is a 2D or 2D array image view taken from a 3D image **must** not be a depth/stencil format
- VUID-VkFramebufferCreateInfo-flags-03189
  If the `imagelessFramebuffer` feature is not enabled, `flags` **must** not include `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`
- VUID-VkFramebufferCreateInfo-flags-03190
  If `flags` includes `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, the `pNext` chain **must** include a VkFramebufferAttachmentsCreateInfo structure
- VUID-VkFramebufferCreateInfo-flags-03191
  If `flags` includes `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, the `attachmentImageInfoCount` member of a VkFramebufferAttachmentsCreateInfo structure in the `pNext` chain **must** be equal to either zero or `attachmentCount`
- VUID-VkFramebufferCreateInfo-flags-04541
  If `flags` includes `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, the `width` member of any element of the `pAttachmentImageInfos` member of a VkFramebufferAttachmentsCreateInfo structure in the `pNext` chain that is used as an input, color, resolve or depth/stencil attachment in `renderPass` **must** be greater than or equal to `width`
- VUID-VkFramebufferCreateInfo-flags-04542
  If `flags` includes `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, the `height` member of any element of the `pAttachmentImageInfos` member of a VkFramebufferAttachmentsCreateInfo structure in the `pNext` chain that is used as an input, color, resolve or depth/stencil attachment in `renderPass` **must** be greater than or equal to `height`
- VUID-VkFramebufferCreateInfo-flags-03196
  If `flags` includes `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, the `width` member of any element of the `pAttachmentImageInfos` member of a VkFramebufferAttachmentsCreateInfo structure in the `pNext` chain that is referenced by VkRenderPassFragmentDensityMapCreateInfoEXT::`fragmentDensityMapAttachment` in `renderPass` **must** be greater than or equal to $\lceil \frac{width}{maxFragmentDensityTexelSize_{width}} \rceil$
- VUID-VkFramebufferCreateInfo-flags-03197
  If `flags` includes `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, the `height` member of any element of the `pAttachmentImageInfos` member of a VkFramebufferAttachmentsCreateInfo structure included in the `pNext` chain that is referenced by VkRenderPassFragmentDensityMapCreateInfoEXT::`fragmentDensityMapAttachment` in `renderPass` **must** be greater than or equal to $\lceil \frac{height}{maxFragmentDensityTexelSize_{height}} \rceil$
- VUID-VkFramebufferCreateInfo-flags-04543
  If the `maintenance7` feature is not enabled or the `robustFragmentShadingRateAttachmentAccess` limit is `VK_FALSE` or the `imageView` member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure was created with VkImageSubresourceRange::`baseMipLevel` greater than 0, and `flags` includes `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT`, the `width` member of any element of the

pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure in the pNext chain that is used as a fragment shading rate attachment **must** be greater than or equal to ⌈width / texelWidth⌉, where texelWidth is the largest value of shadingRateAttachmentTexelSize.width in a VkFragmentShadingRateAttachmentInfoKHR which references that attachment

- VUID-VkFramebufferCreateInfo-flags-04544
  If the maintenance7 feature is not enabled or the robustFragmentShadingRateAttachmentAccess limit is VK_FALSE or the imageView member of a VkRenderingFragmentShadingRateAttachmentInfoKHR structure was created with VkImageSubresourceRange::baseMipLevel greater than 0, and flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the height member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure in the pNext chain that is used as a fragment shading rate attachment **must** be greater than or equal to ⌈height / texelHeight⌉, where texelHeight is the largest value of shadingRateAttachmentTexelSize.height in a VkFragmentShadingRateAttachmentInfoKHR which references that attachment
- VUID-VkFramebufferCreateInfo-flags-04545
  If flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the layerCount member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure in the pNext chain that is used as a fragment shading rate attachment **must** be either 1, or greater than or equal to layers
- VUID-VkFramebufferCreateInfo-flags-04587
  If flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT and renderPass was specified with non-zero view masks, the layerCount member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure in the pNext chain that is used as a fragment shading rate attachment **must** be either 1, or greater than the index of the most significant bit set in any of those view masks
- VUID-VkFramebufferCreateInfo-renderPass-03198
  If multiview is enabled for renderPass and flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the layerCount member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure included in the pNext chain used as an input, color, resolve, or depth/stencil attachment in renderPass **must** be greater than the maximum bit index set in the view mask in the subpasses in which it is used in renderPass
- VUID-VkFramebufferCreateInfo-renderPass-04546
  If multiview is not enabled for renderPass and flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the layerCount member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure included in the pNext chain used as an input, color, resolve, or depth/stencil attachment in renderPass **must** be greater than or equal to layers
- VUID-VkFramebufferCreateInfo-flags-03201
  If flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the usage member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure included in the pNext chain that refers to an attachment used as a color attachment or resolve attachment by renderPass **must** include

VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT

- VUID-VkFramebufferCreateInfo-flags-03202
  If flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the usage member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure included in the pNext chain that refers to an attachment used as a depth/stencil attachment by renderPass **must** include VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkFramebufferCreateInfo-flags-03203
  If flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the usage member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure included in the pNext chain that refers to an attachment used as a depth/stencil resolve attachment by renderPass **must** include VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkFramebufferCreateInfo-flags-03204
  If flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the usage member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure included in the pNext chain that refers to an attachment used as an input attachment by renderPass **must** include VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT
- VUID-VkFramebufferCreateInfo-flags-03205
  If flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, at least one element of the pViewFormats member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure included in the pNext chain **must** be equal to the corresponding value of VkAttachmentDescription::format used to create renderPass
- VUID-VkFramebufferCreateInfo-flags-04113
  If flags does not include VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of pAttachments **must** have been created with VkImageViewCreateInfo::viewType not equal to VK_IMAGE_VIEW_TYPE_3D
- VUID-VkFramebufferCreateInfo-flags-04548
  If flags does not include VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of pAttachments that is used as a fragment shading rate attachment by renderPass **must** have been created with a usage value including VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-VkFramebufferCreateInfo-flags-04549
  If flags includes VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the usage member of any element of the pAttachmentImageInfos member of a VkFramebufferAttachmentsCreateInfo structure included in the pNext chain that refers to an attachment used as a fragment shading rate attachment by renderPass **must** include VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR

Valid Usage (Implicit)

- VUID-VkFramebufferCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_FRAMEBUFFER_CREATE_INFO

- VUID-VkFramebufferCreateInfo-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkFramebufferAttachmentsCreateInfo
- VUID-VkFramebufferCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkFramebufferCreateInfo-flags-parameter
  `flags` **must** be a valid combination of VkFramebufferCreateFlagBits values
- VUID-VkFramebufferCreateInfo-renderPass-parameter
  `renderPass` **must** be a valid VkRenderPass handle
- VUID-VkFramebufferCreateInfo-commonparent
  Both of `renderPass`, and the elements of `pAttachments` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The `VkFramebufferAttachmentsCreateInfo` structure is defined as:

```
typedef struct VkFramebufferAttachmentsCreateInfo {
    VkStructureType                           sType;
    const void*                               pNext;
    uint32_t                                  attachmentImageInfoCount;
    const VkFramebufferAttachmentImageInfo*   pAttachmentImageInfos;
} VkFramebufferAttachmentsCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_imageless_framebuffer
typedef VkFramebufferAttachmentsCreateInfo VkFramebufferAttachmentsCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `attachmentImageInfoCount` is the number of attachments being described.
- `pAttachmentImageInfos` is a pointer to an array of VkFramebufferAttachmentImageInfo structures, each structure describing a number of parameters of the corresponding attachment in a render pass instance.

Valid Usage (Implicit)

- VUID-VkFramebufferAttachmentsCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENTS_CREATE_INFO`
- VUID-VkFramebufferAttachmentsCreateInfo-pAttachmentImageInfos-parameter
  If `attachmentImageInfoCount` is not `0`, `pAttachmentImageInfos` **must** be a valid pointer to an array of `attachmentImageInfoCount` valid VkFramebufferAttachmentImageInfo structures

The VkFramebufferAttachmentImageInfo structure is defined as:

```
typedef struct VkFramebufferAttachmentImageInfo {
    VkStructureType       sType;
    const void*           pNext;
    VkImageCreateFlags    flags;
    VkImageUsageFlags     usage;
    uint32_t              width;
    uint32_t              height;
    uint32_t              layerCount;
    uint32_t              viewFormatCount;
    const VkFormat*       pViewFormats;
} VkFramebufferAttachmentImageInfo;
```

or the equivalent

```
// Provided by VK_KHR_imageless_framebuffer
typedef VkFramebufferAttachmentImageInfo VkFramebufferAttachmentImageInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkImageCreateFlagBits, matching the value of VkImageCreateInfo::flags used to create an image that will be used with this framebuffer.
- usage is a bitmask of VkImageUsageFlagBits, matching the value of VkImageCreateInfo::usage used to create an image used with this framebuffer.
- width is the width of the image view used for rendering.
- height is the height of the image view used for rendering.
- layerCount is the number of array layers of the image view used for rendering.
- viewFormatCount is the number of entries in the pViewFormats array, matching the value of VkImageFormatListCreateInfo::viewFormatCount used to create an image used with this framebuffer.
- pViewFormats is a pointer to an array of VkFormat values specifying all of the formats which **can** be used when creating views of the image, matching the value of VkImageFormatListCreateInfo::pViewFormats used to create an image used with this framebuffer.

Images that **can** be used with the framebuffer when beginning a render pass, as specified by VkRenderPassAttachmentBeginInfo, **must** be created with parameters that are identical to those specified here.

Valid Usage

- VUID-VkFramebufferAttachmentImageInfo-viewFormatCount-09536

If viewFormatCount is not 0, each element of pViewFormats **must** not be VK_FORMAT_UNDEFINED

Valid Usage (Implicit)

- VUID-VkFramebufferAttachmentImageInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENT_IMAGE_INFO
- VUID-VkFramebufferAttachmentImageInfo-pNext-pNext
  pNext **must** be NULL
- VUID-VkFramebufferAttachmentImageInfo-flags-parameter
  flags **must** be a valid combination of VkImageCreateFlagBits values
- VUID-VkFramebufferAttachmentImageInfo-usage-parameter
  usage **must** be a valid combination of VkImageUsageFlagBits values
- VUID-VkFramebufferAttachmentImageInfo-usage-requiredbitmask
  usage **must** not be 0
- VUID-VkFramebufferAttachmentImageInfo-pViewFormats-parameter
  If viewFormatCount is not 0, pViewFormats **must** be a valid pointer to an array of viewFormatCount valid VkFormat values

Bits which **can** be set in VkFramebufferCreateInfo::flags, specifying options for framebuffers, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkFramebufferCreateFlagBits {
    VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT = 0x00000001,
  // Provided by VK_KHR_imageless_framebuffer
    VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT_KHR = VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT,
} VkFramebufferCreateFlagBits;
```

- VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT specifies that image views are not specified, and only attachment compatibility information will be provided via a VkFramebufferAttachmentImageInfo structure.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkFramebufferCreateFlags;
```

VkFramebufferCreateFlags is a bitmask type for setting a mask of zero or more VkFramebufferCreateFlagBits.

To destroy a framebuffer, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyFramebuffer(
    VkDevice                                    device,
    VkFramebuffer                               framebuffer,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the framebuffer.
- `framebuffer` is the handle of the framebuffer to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyFramebuffer-framebuffer-00892
  All submitted commands that refer to `framebuffer` **must** have completed execution
- VUID-vkDestroyFramebuffer-framebuffer-00893
  If `VkAllocationCallbacks` were provided when `framebuffer` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyFramebuffer-framebuffer-00894
  If no `VkAllocationCallbacks` were provided when `framebuffer` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyFramebuffer-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyFramebuffer-framebuffer-parameter
  If `framebuffer` is not VK_NULL_HANDLE, `framebuffer` **must** be a valid VkFramebuffer handle
- VUID-vkDestroyFramebuffer-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyFramebuffer-framebuffer-parent
  If `framebuffer` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `framebuffer` **must** be externally synchronized

8.5. Render Pass Load Operations

Render pass load operations define the initial values of an attachment during a render pass instance.

Load operations for attachments with a depth/stencil format execute in the VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT pipeline stage. Load operations for attachments with a color format execute in the VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage. The load operation for each sample in an attachment happens-before any recorded command which accesses the sample in that render pass instance via that attachment or an alias.

NOTE

Because load operations always happen first, external synchronization with attachment access only needs to synchronize the load operations with previous commands; not the operations within the render pass instance. This does not apply when using VK_ATTACHMENT_LOAD_OP_NONE_KHR.

Load operations only update values within the defined render area for the render pass instance. However, any writes performed by a load operation (as defined by its access masks) to a given attachment **may** read and write back any memory locations within the image subresource bound for that attachment. For depth/stencil images, if the maintenance7 feature is not enabled or separateDepthStencilAttachmentAccess is VK_FALSE, writes to one aspect **may** also result in read-modify-write operations for the other aspect. If the subresource is in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT layout, implementations **must** not access pixels outside of the render area.

NOTE

As entire subresources could be accessed by load operations, applications cannot safely access values outside of the render area during a render pass instance when a load operation that modifies values is used.

Load operations that **can** be used for a render pass are:

```
// Provided by VK_VERSION_1_0
typedef enum VkAttachmentLoadOp {
    VK_ATTACHMENT_LOAD_OP_LOAD = 0,
    VK_ATTACHMENT_LOAD_OP_CLEAR = 1,
    VK_ATTACHMENT_LOAD_OP_DONT_CARE = 2,
  // Provided by VK_KHR_load_store_op_none
    VK_ATTACHMENT_LOAD_OP_NONE_KHR = 1000400000,
  // Provided by VK_EXT_load_store_op_none
    VK_ATTACHMENT_LOAD_OP_NONE_EXT = VK_ATTACHMENT_LOAD_OP_NONE_KHR,
} VkAttachmentLoadOp;
```

- VK_ATTACHMENT_LOAD_OP_LOAD specifies that the previous contents of the image within the render area will be preserved as the initial values. For attachments with a depth/stencil format, this uses the access type VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_READ_BIT. For attachments with a color format, this uses the access type VK_ACCESS_COLOR_ATTACHMENT_READ_BIT.
- VK_ATTACHMENT_LOAD_OP_CLEAR specifies that the contents within the render area will be cleared to

a uniform value, which is specified when a render pass instance is begun. For attachments with a depth/stencil format, this uses the access type VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT. For attachments with a color format, this uses the access type VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT.

- VK_ATTACHMENT_LOAD_OP_DONT_CARE specifies that the previous contents within the area need not be preserved; the contents of the attachment will be undefined inside the render area. For attachments with a depth/stencil format, this uses the access type VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT. For attachments with a color format, this uses the access type VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT.
- VK_ATTACHMENT_LOAD_OP_NONE_KHR specifies that the previous contents of the image will be undefined inside the render pass. No access type is used as the image is not accessed.

During a render pass instance, input and color attachments with color formats that have a component size of 8, 16, or 32 bits **must** be represented in the attachment's format throughout the instance. Attachments with other floating- or fixed-point color formats, or with depth components **may** be represented in a format with a precision higher than the attachment format, but **must** be represented with the same range. When such a component is loaded via the loadOp, it will be converted into an implementation-dependent format used by the render pass. Such components **must** be converted from the render pass format, to the format of the attachment, before they are resolved or stored at the end of a render pass instance via storeOp. Conversions occur as described in Numeric Representation and Computation and Fixed-Point Data Conversions.

8.6. Render Pass Store Operations

Render pass store operations define how values written to an attachment during a render pass instance are stored to memory.

Store operations for attachments with a depth/stencil format execute in the VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT pipeline stage. Store operations for attachments with a color format execute in the VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage. The store operation for each sample in an attachment happens-after any recorded command which accesses the sample via that attachment or an alias.

NOTE Because store operations always happen after other accesses in a render pass instance, external synchronization with attachment access in an earlier render pass only needs to synchronize with the store operations; not the operations within the render pass instance. This does not apply when using VK_ATTACHMENT_STORE_OP_NONE.

Store operations only update values within the defined render area for the render pass instance. However, any writes performed by a store operation (as defined by its access masks) to a given attachment **may** read and write back any memory locations within the image subresource bound for that attachment. For depth/stencil images, if separateDepthStencilAttachmentAccess is VK_FALSE, writes to one aspect **may** also result in read-modify-write operations for the other aspect. If the subresource is in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT layout, implementations **must** not access pixels outside of the render area.

NOTE As entire subresources could be accessed by store operations, applications cannot

safely access values outside of the render area via aliased resources during a render pass instance when a store operation that modifies values is used.

Possible values of VkAttachmentDescription::storeOp and stencilStoreOp, specifying how the contents of the attachment are treated, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkAttachmentStoreOp {
    VK_ATTACHMENT_STORE_OP_STORE = 0,
    VK_ATTACHMENT_STORE_OP_DONT_CARE = 1,
    VK_ATTACHMENT_STORE_OP_NONE = 1000301000,
  // Provided by VK_KHR_dynamic_rendering, VK_KHR_load_store_op_none
    VK_ATTACHMENT_STORE_OP_NONE_KHR = VK_ATTACHMENT_STORE_OP_NONE,
  // Provided by VK_EXT_load_store_op_none
    VK_ATTACHMENT_STORE_OP_NONE_EXT = VK_ATTACHMENT_STORE_OP_NONE,
} VkAttachmentStoreOp;
```

- VK_ATTACHMENT_STORE_OP_STORE specifies the contents generated during the render pass and within the render area are written to memory. For attachments with a depth/stencil format, this uses the access type VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT. For attachments with a color format, this uses the access type VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT.
- VK_ATTACHMENT_STORE_OP_DONT_CARE specifies the contents within the render area are not needed after rendering, and **may** be discarded; the contents of the attachment will be undefined inside the render area. For attachments with a depth/stencil format, this uses the access type VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT. For attachments with a color format, this uses the access type VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT.
- VK_ATTACHMENT_STORE_OP_NONE specifies the contents within the render area are not accessed by the store operation as long as no values are written to the attachment during the render pass. If values are written during the render pass, this behaves identically to VK_ATTACHMENT_STORE_OP_DONT_CARE and with matching access semantics.

NOTE VK_ATTACHMENT_STORE_OP_DONT_CARE **can** cause contents generated during previous render passes to be discarded before reaching memory, even if no write to the attachment occurs during the current render pass.

8.7. Render Pass Multisample Resolve Operations

Render pass multisample resolve operations combine sample values from a single pixel in a multisample attachment and store the result to the corresponding pixel in a single sample attachment.

Multisample resolve operations for attachments execute in the VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT pipeline stage. A final resolve operation for all pixels in the render area happens-after any recorded command which writes a pixel via the multisample attachment to be resolved or an explicit alias of it in the subpass that it is specified. Any single sample attachment specified for use in a multisample resolve operation **may** have its

contents modified at any point once rendering begins for the render pass instance. Reads from the multisample attachment can be synchronized with VK_ACCESS_COLOR_ATTACHMENT_READ_BIT. Access to the single sample attachment can be synchronized with VK_ACCESS_COLOR_ATTACHMENT_READ_BIT and VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT. These pipeline stage and access types are used whether the attachments are color or depth/stencil attachments.

When using render pass objects, a subpass dependency specified with the above pipeline stages and access flags will ensure synchronization with multisample resolve operations for any attachments that were last accessed by that subpass. This allows later subpasses to read resolved values as input attachments.

Resolve operations only update values within the defined render area for the render pass instance. However, any writes performed by a resolve operation (as defined by its access masks) to a given attachment **may** read and write back any memory locations within the image subresource bound for that attachment. For depth/stencil images, if separateDepthStencilAttachmentAccess is VK_FALSE, writes to one aspect **may** also result in read-modify-write operations for the other aspect. If the subresource is in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT layout, implementations **must** not access pixels outside of the render area.

NOTE

As entire subresources could be accessed by multisample resolve operations, applications cannot safely access values outside of the render area via aliased resources during a render pass instance when a multisample resolve operation is performed.

Multisample values in a multisample attachment are combined according to the resolve mode used:

```
typedef enum VkResolveModeFlagBits {
    VK_RESOLVE_MODE_NONE = 0,
    VK_RESOLVE_MODE_SAMPLE_ZERO_BIT = 0x00000001,
    VK_RESOLVE_MODE_AVERAGE_BIT = 0x00000002,
    VK_RESOLVE_MODE_MIN_BIT = 0x00000004,
    VK_RESOLVE_MODE_MAX_BIT = 0x00000008,
  // Provided by VK_KHR_depth_stencil_resolve
    VK_RESOLVE_MODE_NONE_KHR = VK_RESOLVE_MODE_NONE,
  // Provided by VK_KHR_depth_stencil_resolve
    VK_RESOLVE_MODE_SAMPLE_ZERO_BIT_KHR = VK_RESOLVE_MODE_SAMPLE_ZERO_BIT,
  // Provided by VK_KHR_depth_stencil_resolve
    VK_RESOLVE_MODE_AVERAGE_BIT_KHR = VK_RESOLVE_MODE_AVERAGE_BIT,
  // Provided by VK_KHR_depth_stencil_resolve
    VK_RESOLVE_MODE_MIN_BIT_KHR = VK_RESOLVE_MODE_MIN_BIT,
  // Provided by VK_KHR_depth_stencil_resolve
    VK_RESOLVE_MODE_MAX_BIT_KHR = VK_RESOLVE_MODE_MAX_BIT,
} VkResolveModeFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_depth_stencil_resolve
typedef VkResolveModeFlagBits VkResolveModeFlagBitsKHR;
```

- VK_RESOLVE_MODE_NONE specifies that no resolve operation is done.
- VK_RESOLVE_MODE_SAMPLE_ZERO_BIT specifies that result of the resolve operation is equal to the value of sample 0.
- VK_RESOLVE_MODE_AVERAGE_BIT specifies that result of the resolve operation is the average of the sample values.
- VK_RESOLVE_MODE_MIN_BIT specifies that result of the resolve operation is the minimum of the sample values.
- VK_RESOLVE_MODE_MAX_BIT specifies that result of the resolve operation is the maximum of the sample values.

If no resolve mode is otherwise specified, VK_RESOLVE_MODE_AVERAGE_BIT is used.

```
typedef VkFlags VkResolveModeFlags;
```

or the equivalent

```
// Provided by VK_KHR_depth_stencil_resolve
typedef VkResolveModeFlags VkResolveModeFlagsKHR;
```

VkResolveModeFlags is a bitmask type for setting a mask of zero or more VkResolveModeFlagBits.

8.8. Render Pass Commands

An application records the commands for a render pass instance one subpass at a time, by beginning a render pass instance, iterating over the subpasses to record commands for that subpass, and then ending the render pass instance.

To begin a render pass instance, call:

```
// Provided by VK_VERSION_1_0
void vkCmdBeginRenderPass(
    VkCommandBuffer                             commandBuffer,
    const VkRenderPassBeginInfo*                pRenderPassBegin,
    VkSubpassContents                           contents);
```

- commandBuffer is the command buffer in which to record the command.
- pRenderPassBegin is a pointer to a VkRenderPassBeginInfo structure specifying the render pass to begin an instance of, and the framebuffer the instance uses.
- contents is a VkSubpassContents value specifying how the commands in the first subpass will be

provided.

After beginning a render pass instance, the command buffer is ready to record the commands for the first subpass of that render pass.

Valid Usage

- VUID-vkCmdBeginRenderPass-initialLayout-00895
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`
- VUID-vkCmdBeginRenderPass-initialLayout-01758
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-vkCmdBeginRenderPass-initialLayout-02842
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-vkCmdBeginRenderPass-stencilInitialLayout-02843
  If any of the `stencilInitialLayout` or `stencilFinalLayout` member of the `VkAttachmentDescriptionStencilLayout` structures or the `stencilLayout` member of the `VkAttachmentReferenceStencilLayout` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-vkCmdBeginRenderPass-initialLayout-00897

If any of the initialLayout or finalLayout member of the VkAttachmentDescription structures or the layout member of the VkAttachmentReference structures specified when creating the render pass specified in the renderPass member of pRenderPassBegin is VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL then the corresponding attachment image view of the framebuffer specified in the framebuffer member of pRenderPassBegin **must** have been created with a usage value including VK_IMAGE_USAGE_SAMPLED_BIT or VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT

- VUID-vkCmdBeginRenderPass-initialLayout-00898
  If any of the initialLayout or finalLayout member of the VkAttachmentDescription structures or the layout member of the VkAttachmentReference structures specified when creating the render pass specified in the renderPass member of pRenderPassBegin is VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL then the corresponding attachment image view of the framebuffer specified in the framebuffer member of pRenderPassBegin **must** have been created with a usage value including VK_IMAGE_USAGE_TRANSFER_SRC_BIT
- VUID-vkCmdBeginRenderPass-initialLayout-00899
  If any of the initialLayout or finalLayout member of the VkAttachmentDescription structures or the layout member of the VkAttachmentReference structures specified when creating the render pass specified in the renderPass member of pRenderPassBegin is VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL then the corresponding attachment image view of the framebuffer specified in the framebuffer member of pRenderPassBegin **must** have been created with a usage value including VK_IMAGE_USAGE_TRANSFER_DST_BIT
- VUID-vkCmdBeginRenderPass-initialLayout-00900
  If the initialLayout member of any of the VkAttachmentDescription structures specified when creating the render pass specified in the renderPass member of pRenderPassBegin is not VK_IMAGE_LAYOUT_UNDEFINED, then each such initialLayout **must** be equal to the current layout of the corresponding attachment image subresource of the framebuffer specified in the framebuffer member of pRenderPassBegin
- VUID-vkCmdBeginRenderPass-srcStageMask-06451
  The srcStageMask members of any element of the pDependencies member of VkRenderPassCreateInfo used to create renderPass **must** be supported by the capabilities of the queue family identified by the queueFamilyIndex member of the VkCommandPoolCreateInfo used to create the command pool which commandBuffer was allocated from
- VUID-vkCmdBeginRenderPass-dstStageMask-06452
  The dstStageMask members of any element of the pDependencies member of VkRenderPassCreateInfo used to create renderPass **must** be supported by the capabilities of the queue family identified by the queueFamilyIndex member of the VkCommandPoolCreateInfo used to create the command pool which commandBuffer was allocated from
- VUID-vkCmdBeginRenderPass-framebuffer-02532
  For any attachment in framebuffer that is used by renderPass and is bound to memory locations that are also bound to another attachment used by renderPass, and if at least one of those uses causes either attachment to be written to, both attachments **must** have had the VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT set
- VUID-vkCmdBeginRenderPass-framebuffer-09045

If any attachments specified in framebuffer are used by renderPass and are bound to overlapping memory locations, there **must** be only one that is used as a color attachment, depth/stencil, or resolve attachment in any subpass

- VUID-vkCmdBeginRenderPass-initialLayout-07000
  If any of the initialLayout or finalLayout member of the VkAttachmentDescription structures or the layout member of the VkAttachmentReference structures specified when creating the render pass specified in the renderPass member of pRenderPassBegin is VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT then the corresponding attachment image view of the framebuffer specified in the framebuffer member of pRenderPassBegin **must** have been created with a usage value including either the VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT and either the VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT or VK_IMAGE_USAGE_SAMPLED_BIT usage bits
- VUID-vkCmdBeginRenderPass-initialLayout-07001
  If any of the initialLayout or finalLayout member of the VkAttachmentDescription structures or the layout member of the VkAttachmentReference structures specified when creating the render pass specified in the renderPass member of pRenderPassBegin is VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT then the corresponding attachment image view of the framebuffer specified in the framebuffer member of pRenderPassBegin **must** have been created with a usage value the VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT usage bit
- VUID-vkCmdBeginRenderPass-initialLayout-09537
  If any of the initialLayout or finalLayout member of the VkAttachmentDescription structures or the layout member of the VkAttachmentReference structures specified when creating the render pass specified in the renderPass member of pRenderPassBegin is VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR then the corresponding attachment image view of the framebuffer specified in the framebuffer member of pRenderPassBegin **must** have been created with a usage value including either VK_IMAGE_USAGE_STORAGE_BIT, or both VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT and either of VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-vkCmdBeginRenderPass-contents-09640
  If contents is VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_KHR, then at least one of the following features **must** be enabled:
  - maintenance7
  - nestedCommandBuffer

Valid Usage (Implicit)

- VUID-vkCmdBeginRenderPass-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBeginRenderPass-pRenderPassBegin-parameter
  pRenderPassBegin **must** be a valid pointer to a valid VkRenderPassBeginInfo structure
- VUID-vkCmdBeginRenderPass-contents-parameter
  contents **must** be a valid VkSubpassContents value

- VUID-vkCmdBeginRenderPass-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBeginRenderPass-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBeginRenderPass-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBeginRenderPass-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBeginRenderPass-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Outside	Outside	Graphics	Action State Synchronization

Alternatively to begin a render pass, call:

```
// Provided by VK_KHR_create_renderpass2
void vkCmdBeginRenderPass2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkRenderPassBeginInfo*                pRenderPassBegin,
    const VkSubpassBeginInfo*                   pSubpassBeginInfo);
```

- `commandBuffer` is the command buffer in which to record the command.
- `pRenderPassBegin` is a pointer to a VkRenderPassBeginInfo structure specifying the render pass to begin an instance of, and the framebuffer the instance uses.
- `pSubpassBeginInfo` is a pointer to a VkSubpassBeginInfo structure containing information about the subpass which is about to begin rendering.

After beginning a render pass instance, the command buffer is ready to record the commands for

the first subpass of that render pass.

Valid Usage

- VUID-vkCmdBeginRenderPass2-framebuffer-02779
  Both the `framebuffer` and `renderPass` members of `pRenderPassBegin` **must** have been created on the same VkDevice that `commandBuffer` was allocated on
- VUID-vkCmdBeginRenderPass2-initialLayout-03094
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`
- VUID-vkCmdBeginRenderPass2-initialLayout-03096
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-vkCmdBeginRenderPass2-initialLayout-02844
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
- VUID-vkCmdBeginRenderPass2-stencilInitialLayout-02845
  If any of the `stencilInitialLayout` or `stencilFinalLayout` member of the `VkAttachmentDescriptionStencilLayout` structures or the `stencilLayout` member of the `VkAttachmentReferenceStencilLayout` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL`, or `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`

- VUID-vkCmdBeginRenderPass2-initialLayout-03097
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_SAMPLED_BIT` or `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT`
- VUID-vkCmdBeginRenderPass2-initialLayout-03098
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_TRANSFER_SRC_BIT`
- VUID-vkCmdBeginRenderPass2-initialLayout-03099
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including `VK_IMAGE_USAGE_TRANSFER_DST_BIT`
- VUID-vkCmdBeginRenderPass2-initialLayout-03100
  If the `initialLayout` member of any of the `VkAttachmentDescription` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is not `VK_IMAGE_LAYOUT_UNDEFINED`, then each such `initialLayout` **must** be equal to the current layout of the corresponding attachment image subresource of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin`
- VUID-vkCmdBeginRenderPass2-srcStageMask-06453
  The `srcStageMask` members of any element of the `pDependencies` member of VkRenderPassCreateInfo used to create `renderPass` **must** be supported by the capabilities of the queue family identified by the `queueFamilyIndex` member of the VkCommandPoolCreateInfo used to create the command pool which `commandBuffer` was allocated from
- VUID-vkCmdBeginRenderPass2-dstStageMask-06454
  The `dstStageMask` members of any element of the `pDependencies` member of VkRenderPassCreateInfo used to create `renderPass` **must** be supported by the capabilities of the queue family identified by the `queueFamilyIndex` member of the VkCommandPoolCreateInfo used to create the command pool which `commandBuffer` was allocated from
- VUID-vkCmdBeginRenderPass2-framebuffer-02533
  For any attachment in `framebuffer` that is used by `renderPass` and is bound to memory locations that are also bound to another attachment used by `renderPass`, and if at least one of those uses causes either attachment to be written to, both attachments **must** have had the `VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT` set

- VUID-vkCmdBeginRenderPass2-framebuffer-09046
  If any attachments specified in `framebuffer` are used by `renderPass` and are bound to overlapping memory locations, there **must** be only one that is used as a color attachment, depth/stencil, or resolve attachment in any subpass
- VUID-vkCmdBeginRenderPass2-initialLayout-07002
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including either the `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT` or `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT` and either the `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT` or `VK_IMAGE_USAGE_SAMPLED_BIT` usage bits
- VUID-vkCmdBeginRenderPass2-initialLayout-07003
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value the `VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT` usage bit
- VUID-vkCmdBeginRenderPass2-initialLayout-09538
  If any of the `initialLayout` or `finalLayout` member of the `VkAttachmentDescription` structures or the `layout` member of the `VkAttachmentReference` structures specified when creating the render pass specified in the `renderPass` member of `pRenderPassBegin` is `VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR` then the corresponding attachment image view of the framebuffer specified in the `framebuffer` member of `pRenderPassBegin` **must** have been created with a `usage` value including either `VK_IMAGE_USAGE_STORAGE_BIT`, or both `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT` and either of `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT` or `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`

Valid Usage (Implicit)

- VUID-vkCmdBeginRenderPass2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBeginRenderPass2-pRenderPassBegin-parameter
  `pRenderPassBegin` **must** be a valid pointer to a valid VkRenderPassBeginInfo structure
- VUID-vkCmdBeginRenderPass2-pSubpassBeginInfo-parameter
  `pSubpassBeginInfo` **must** be a valid pointer to a valid VkSubpassBeginInfo structure
- VUID-vkCmdBeginRenderPass2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBeginRenderPass2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations

- VUID-vkCmdBeginRenderPass2-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBeginRenderPass2-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBeginRenderPass2-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Outside	Outside	Graphics	Action State Synchronization

The `VkRenderPassBeginInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkRenderPassBeginInfo {
    VkStructureType        sType;
    const void*            pNext;
    VkRenderPass           renderPass;
    VkFramebuffer          framebuffer;
    VkRect2D               renderArea;
    uint32_t               clearValueCount;
    const VkClearValue*    pClearValues;
} VkRenderPassBeginInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `renderPass` is the render pass to begin an instance of.
- `framebuffer` is the framebuffer containing the attachments that are used with the render pass.
- `renderArea` is the render area that is affected by the render pass instance, and is described in more detail below.
- `clearValueCount` is the number of elements in `pClearValues`.

- `pClearValues` is a pointer to an array of `clearValueCount` VkClearValue structures containing clear values for each attachment, if the attachment uses a `loadOp` value of `VK_ATTACHMENT_LOAD_OP_CLEAR` or if the attachment has a depth/stencil format and uses a `stencilLoadOp` value of `VK_ATTACHMENT_LOAD_OP_CLEAR`. The array is indexed by attachment number. Only elements corresponding to cleared attachments are used. Other elements of `pClearValues` are ignored.

`renderArea` is the render area that is affected by the render pass instance. The effects of attachment load, store and multisample resolve operations are restricted to the pixels whose x and y coordinates fall within the render area on all attachments. The render area extends to all layers of `framebuffer`. The application **must** ensure (using scissor if necessary) that all rendering is contained within the render area. The render area **must** be contained within the framebuffer dimensions.

NOTE There **may** be a performance cost for using a render area smaller than the framebuffer, unless it matches the render area granularity for the render pass.

Valid Usage

- VUID-VkRenderPassBeginInfo-clearValueCount-00902
  `clearValueCount` **must** be greater than the largest attachment index in `renderPass` specifying a `loadOp` (or `stencilLoadOp`, if the attachment has a depth/stencil format) of `VK_ATTACHMENT_LOAD_OP_CLEAR`
- VUID-VkRenderPassBeginInfo-clearValueCount-04962
  If `clearValueCount` is not `0`, `pClearValues` **must** be a valid pointer to an array of `clearValueCount` VkClearValue unions
- VUID-VkRenderPassBeginInfo-renderPass-00904
  `renderPass` **must** be compatible with the `renderPass` member of the VkFramebufferCreateInfo structure specified when creating `framebuffer`
- VUID-VkRenderPassBeginInfo-None-08996
  If the `pNext` chain does not contain VkDeviceGroupRenderPassBeginInfo or its `deviceRenderAreaCount` member is equal to 0, `renderArea.extent.width` **must** be greater than 0
- VUID-VkRenderPassBeginInfo-None-08997
  If the `pNext` chain does not contain VkDeviceGroupRenderPassBeginInfo or its `deviceRenderAreaCount` member is equal to 0, `renderArea.extent.height` **must** be greater than 0
- VUID-VkRenderPassBeginInfo-pNext-02850
  If the `pNext` chain does not contain VkDeviceGroupRenderPassBeginInfo or its `deviceRenderAreaCount` member is equal to 0, `renderArea.offset.x` **must** be greater than or equal to 0
- VUID-VkRenderPassBeginInfo-pNext-02851
  If the `pNext` chain does not contain VkDeviceGroupRenderPassBeginInfo or its `deviceRenderAreaCount` member is equal to 0, `renderArea.offset.y` **must** be greater than or equal to 0
- VUID-VkRenderPassBeginInfo-pNext-02852

If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, renderArea.offset.x + renderArea.extent.width **must** be less than or equal to VkFramebufferCreateInfo::width the framebuffer was created with

- VUID-VkRenderPassBeginInfo-pNext-02853
  If the pNext chain does not contain VkDeviceGroupRenderPassBeginInfo or its deviceRenderAreaCount member is equal to 0, renderArea.offset.y + renderArea.extent.height **must** be less than or equal to VkFramebufferCreateInfo::height the framebuffer was created with
- VUID-VkRenderPassBeginInfo-pNext-02856
  If the pNext chain contains VkDeviceGroupRenderPassBeginInfo, offset.x + extent.width of each element of pDeviceRenderAreas **must** be less than or equal to VkFramebufferCreateInfo::width the framebuffer was created with
- VUID-VkRenderPassBeginInfo-pNext-02857
  If the pNext chain contains VkDeviceGroupRenderPassBeginInfo, offset.y + extent.height of each element of pDeviceRenderAreas **must** be less than or equal to VkFramebufferCreateInfo::height the framebuffer was created with
- VUID-VkRenderPassBeginInfo-framebuffer-03207
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that did not include VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, and the pNext chain includes a VkRenderPassAttachmentBeginInfo structure, its attachmentCount **must** be zero
- VUID-VkRenderPassBeginInfo-framebuffer-03208
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, the attachmentCount of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be equal to the value of VkFramebufferAttachmentsCreateInfo::attachmentImageInfoCount used to create framebuffer
- VUID-VkRenderPassBeginInfo-framebuffer-02780
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** have been created on the same VkDevice as framebuffer and renderPass
- VUID-VkRenderPassBeginInfo-framebuffer-03209
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView of an image created with a value of VkImageCreateInfo::flags equal to the flags member of the corresponding element of VkFramebufferAttachmentsCreateInfo ::pAttachmentImageInfos used to create framebuffer
- VUID-VkRenderPassBeginInfo-framebuffer-04627
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView with an inherited usage equal to the usage member of the corresponding element of VkFramebufferAttachmentsCreateInfo::pAttachmentImageInfos used to create

framebuffer

- VUID-VkRenderPassBeginInfo-framebuffer-03211
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView with a width equal to the width member of the corresponding element of VkFramebufferAttachmentsCreateInfo::pAttachmentImageInfos used to create framebuffer
- VUID-VkRenderPassBeginInfo-framebuffer-03212
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView with a height equal to the height member of the corresponding element of VkFramebufferAttachmentsCreateInfo::pAttachmentImageInfos used to create framebuffer
- VUID-VkRenderPassBeginInfo-framebuffer-03213
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView of an image created with a value of VkImageViewCreateInfo::subresourceRange.layerCount equal to the layerCount member of the corresponding element of VkFramebufferAttachmentsCreateInfo::pAttachmentImageInfos used to create framebuffer
- VUID-VkRenderPassBeginInfo-framebuffer-03214
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView of an image created with a value of VkImageFormatListCreateInfo::viewFormatCount equal to the viewFormatCount member of the corresponding element of VkFramebufferAttachmentsCreateInfo::pAttachmentImageInfos used to create framebuffer
- VUID-VkRenderPassBeginInfo-framebuffer-03215
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView of an image created with a set of elements in VkImageFormatListCreateInfo::pViewFormats equal to the set of elements in the pViewFormats member of the corresponding element of VkFramebufferAttachmentsCreateInfo::pAttachmentImageInfos used to create framebuffer
- VUID-VkRenderPassBeginInfo-framebuffer-03216
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView of an image created with a value of VkImageViewCreateInfo::format equal to the corresponding value of VkAttachmentDescription::format in renderPass
- VUID-VkRenderPassBeginInfo-framebuffer-09047
  If framebuffer was created with a VkFramebufferCreateInfo::flags value that included VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT, each element of the pAttachments member of a

VkRenderPassAttachmentBeginInfo structure included in the pNext chain **must** be a VkImageView of an image created with a value of VkImageCreateInfo::samples equal to the corresponding value of VkAttachmentDescription::samples in renderPass

Valid Usage (Implicit)

- VUID-VkRenderPassBeginInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RENDER_PASS_BEGIN_INFO
- VUID-VkRenderPassBeginInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkDeviceGroupRenderPassBeginInfo, VkRenderPassAttachmentBeginInfo, or VkRenderPassSampleLocationsBeginInfoEXT
- VUID-VkRenderPassBeginInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkRenderPassBeginInfo-renderPass-parameter
  renderPass **must** be a valid VkRenderPass handle
- VUID-VkRenderPassBeginInfo-framebuffer-parameter
  framebuffer **must** be a valid VkFramebuffer handle
- VUID-VkRenderPassBeginInfo-commonparent
  Both of framebuffer, and renderPass **must** have been created, allocated, or retrieved from the same VkDevice

The image layout of the depth aspect of a depth/stencil attachment referring to an image created with VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT is dependent on the last sample locations used to render to the image subresource, thus preserving the contents of such depth/stencil attachments across subpass boundaries requires the application to specify these sample locations whenever a layout transition of the attachment **may** occur. This information **can** be provided by adding a VkRenderPassSampleLocationsBeginInfoEXT structure to the pNext chain of VkRenderPassBeginInfo.

The VkRenderPassSampleLocationsBeginInfoEXT structure is defined as:

```
// Provided by VK_EXT_sample_locations
typedef struct VkRenderPassSampleLocationsBeginInfoEXT {
    VkStructureType                             sType;
    const void*                                 pNext;
    uint32_t                                    attachmentInitialSampleLocationsCount;
    const VkAttachmentSampleLocationsEXT*       pAttachmentInitialSampleLocations;
    uint32_t                                    postSubpassSampleLocationsCount;
    const VkSubpassSampleLocationsEXT*          pPostSubpassSampleLocations;
} VkRenderPassSampleLocationsBeginInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- `attachmentInitialSampleLocationsCount` is the number of elements in the `pAttachmentInitialSampleLocations` array.
- `pAttachmentInitialSampleLocations` is a pointer to an array of `attachmentInitialSampleLocationsCount` VkAttachmentSampleLocationsEXT structures specifying the attachment indices and their corresponding sample location state. Each element of `pAttachmentInitialSampleLocations` **can** specify the sample location state to use in the automatic layout transition performed to transition a depth/stencil attachment from the initial layout of the attachment to the image layout specified for the attachment in the first subpass using it.
- `postSubpassSampleLocationsCount` is the number of elements in the `pPostSubpassSampleLocations` array.
- `pPostSubpassSampleLocations` is a pointer to an array of `postSubpassSampleLocationsCount` VkSubpassSampleLocationsEXT structures specifying the subpass indices and their corresponding sample location state. Each element of `pPostSubpassSampleLocations` **can** specify the sample location state to use in the automatic layout transition performed to transition the depth/stencil attachment used by the specified subpass to the image layout specified in a dependent subpass or to the final layout of the attachment in case the specified subpass is the last subpass using that attachment. In addition, if VkPhysicalDeviceSampleLocationsPropertiesEXT::`variableSampleLocations` is `VK_FALSE`, each element of `pPostSubpassSampleLocations` **must** specify the sample location state that matches the sample locations used by all pipelines that will be bound to a command buffer during the specified subpass. If `variableSampleLocations` is `VK_TRUE`, the sample locations used for rasterization do not depend on `pPostSubpassSampleLocations`.

Valid Usage (Implicit)

- VUID-VkRenderPassSampleLocationsBeginInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDER_PASS_SAMPLE_LOCATIONS_BEGIN_INFO_EXT`
- VUID-VkRenderPassSampleLocationsBeginInfoEXT-pAttachmentInitialSampleLocations-parameter
  If `attachmentInitialSampleLocationsCount` is not `0`, `pAttachmentInitialSampleLocations` **must** be a valid pointer to an array of `attachmentInitialSampleLocationsCount` valid VkAttachmentSampleLocationsEXT structures
- VUID-VkRenderPassSampleLocationsBeginInfoEXT-pPostSubpassSampleLocations-parameter
  If `postSubpassSampleLocationsCount` is not `0`, `pPostSubpassSampleLocations` **must** be a valid pointer to an array of `postSubpassSampleLocationsCount` valid VkSubpassSampleLocationsEXT structures

The `VkAttachmentSampleLocationsEXT` structure is defined as:

```
// Provided by VK_EXT_sample_locations
typedef struct VkAttachmentSampleLocationsEXT {
    uint32_t                    attachmentIndex;
    VkSampleLocationsInfoEXT    sampleLocationsInfo;
} VkAttachmentSampleLocationsEXT;
```

- `attachmentIndex` is the index of the attachment for which the sample locations state is provided.
- `sampleLocationsInfo` is the sample locations state to use for the layout transition of the given attachment from the initial layout of the attachment to the image layout specified for the attachment in the first subpass using it.

If the image referenced by the framebuffer attachment at index `attachmentIndex` was not created with `VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT` then the values specified in `sampleLocationsInfo` are ignored.

Valid Usage

- VUID-VkAttachmentSampleLocationsEXT-attachmentIndex-01531 `attachmentIndex` **must** be less than the `attachmentCount` specified in VkRenderPassCreateInfo the render pass specified by VkRenderPassBeginInfo::`renderPass` was created with

Valid Usage (Implicit)

- VUID-VkAttachmentSampleLocationsEXT-sampleLocationsInfo-parameter `sampleLocationsInfo` **must** be a valid VkSampleLocationsInfoEXT structure

The `VkSubpassSampleLocationsEXT` structure is defined as:

```
// Provided by VK_EXT_sample_locations
typedef struct VkSubpassSampleLocationsEXT {
    uint32_t                    subpassIndex;
    VkSampleLocationsInfoEXT    sampleLocationsInfo;
} VkSubpassSampleLocationsEXT;
```

- `subpassIndex` is the index of the subpass for which the sample locations state is provided.
- `sampleLocationsInfo` is the sample locations state to use for the layout transition of the depth/stencil attachment away from the image layout the attachment is used with in the subpass specified in `subpassIndex`.

If the image referenced by the depth/stencil attachment used in the subpass identified by `subpassIndex` was not created with `VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT` or if the subpass does not use a depth/stencil attachment, and VkPhysicalDeviceSampleLocationsPropertiesEXT::`variableSampleLocations` is `VK_TRUE` then the

values specified in sampleLocationsInfo are ignored.

Valid Usage

- VUID-VkSubpassSampleLocationsEXT-subpassIndex-01532
  subpassIndex **must** be less than the subpassCount specified in VkRenderPassCreateInfo the render pass specified by VkRenderPassBeginInfo::renderPass was created with

Valid Usage (Implicit)

- VUID-VkSubpassSampleLocationsEXT-sampleLocationsInfo-parameter
  sampleLocationsInfo **must** be a valid VkSampleLocationsInfoEXT structure

The VkSubpassBeginInfo structure is defined as:

```
typedef struct VkSubpassBeginInfo {
    VkStructureType      sType;
    const void*          pNext;
    VkSubpassContents    contents;
} VkSubpassBeginInfo;
```

or the equivalent

```
// Provided by VK_KHR_create_renderpass2
typedef VkSubpassBeginInfo VkSubpassBeginInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- contents is a VkSubpassContents value specifying how the commands in the next subpass will be provided.

Valid Usage

- VUID-VkSubpassBeginInfo-contents-09382
  If contents is VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_KHR, then at least one of the following features **must** be enabled:
  - maintenance7
  - nestedCommandBuffer

Valid Usage (Implicit)

- VUID-VkSubpassBeginInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SUBPASS_BEGIN_INFO`
- VUID-VkSubpassBeginInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkSubpassBeginInfo-contents-parameter
  `contents` **must** be a valid VkSubpassContents value

Possible values of vkCmdBeginRenderPass::`contents`, specifying how the commands in the first subpass will be provided, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSubpassContents {
    VK_SUBPASS_CONTENTS_INLINE = 0,
    VK_SUBPASS_CONTENTS_SECONDARY_COMMAND_BUFFERS = 1,
  // Provided by VK_KHR_maintenance7
    VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_KHR = 1000451000,
  // Provided by VK_EXT_nested_command_buffer
    VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_EXT =
VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_KHR,
} VkSubpassContents;
```

- `VK_SUBPASS_CONTENTS_INLINE` specifies that the contents of the subpass will be recorded inline in the primary command buffer, and secondary command buffers **must** not be executed within the subpass.
- `VK_SUBPASS_CONTENTS_SECONDARY_COMMAND_BUFFERS` specifies that the contents are recorded in secondary command buffers that will be called from the primary command buffer, and vkCmdExecuteCommands is the only valid command in the command buffer until vkCmdNextSubpass or vkCmdEndRenderPass.
- `VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_KHR` specifies that the contents of the subpass **can** be recorded both inline and in secondary command buffers executed from this command buffer with vkCmdExecuteCommands.

If the `pNext` chain of VkRenderPassBeginInfo or VkRenderingInfo includes a `VkDeviceGroupRenderPassBeginInfo` structure, then that structure includes a device mask and set of render areas for the render pass instance.

The `VkDeviceGroupRenderPassBeginInfo` structure is defined as:

```
typedef struct VkDeviceGroupRenderPassBeginInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           deviceMask;
    uint32_t           deviceRenderAreaCount;
    const VkRect2D*    pDeviceRenderAreas;
} VkDeviceGroupRenderPassBeginInfo;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkDeviceGroupRenderPassBeginInfo VkDeviceGroupRenderPassBeginInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `deviceMask` is the device mask for the render pass instance.
- `deviceRenderAreaCount` is the number of elements in the `pDeviceRenderAreas` array.
- `pDeviceRenderAreas` is a pointer to an array of VkRect2D structures defining the render area for each physical device.

The `deviceMask` serves several purposes. It is an upper bound on the set of physical devices that **can** be used during the render pass instance, and the initial device mask when the render pass instance begins. In addition, commands transitioning to the next subpass in a render pass instance and commands ending the render pass instance, and, accordingly render pass load, store, and multisample resolve operations and subpass dependencies corresponding to the render pass instance, are executed on the physical devices included in the device mask provided here.

If `deviceRenderAreaCount` is not zero, then the elements of `pDeviceRenderAreas` override the value of VkRenderPassBeginInfo::`renderArea`, and provide a render area specific to each physical device. These render areas serve the same purpose as VkRenderPassBeginInfo::`renderArea`, including controlling the region of attachments that are cleared by `VK_ATTACHMENT_LOAD_OP_CLEAR` and that are resolved into resolve attachments.

If this structure is not present, the render pass instance's device mask is the value of VkDeviceGroupCommandBufferBeginInfo::`deviceMask`. If this structure is not present or if `deviceRenderAreaCount` is zero, VkRenderPassBeginInfo::`renderArea` is used for all physical devices.

Valid Usage

- VUID-VkDeviceGroupRenderPassBeginInfo-deviceMask-00905
  `deviceMask` **must** be a valid device mask value
- VUID-VkDeviceGroupRenderPassBeginInfo-deviceMask-00906
  `deviceMask` **must** not be zero
- VUID-VkDeviceGroupRenderPassBeginInfo-deviceMask-00907

deviceMask **must** be a subset of the command buffer's initial device mask

- VUID-VkDeviceGroupRenderPassBeginInfo-deviceRenderAreaCount-00908
  deviceRenderAreaCount **must** either be zero or equal to the number of physical devices in the logical device
- VUID-VkDeviceGroupRenderPassBeginInfo-offset-06166
  The offset.x member of any element of pDeviceRenderAreas **must** be greater than or equal to 0
- VUID-VkDeviceGroupRenderPassBeginInfo-offset-06167
  The offset.y member of any element of pDeviceRenderAreas **must** be greater than or equal to 0
- VUID-VkDeviceGroupRenderPassBeginInfo-offset-06168
  The sum of the offset.x and extent.width members of any element of pDeviceRenderAreas **must** be less than or equal to maxFramebufferWidth
- VUID-VkDeviceGroupRenderPassBeginInfo-offset-06169
  The sum of the offset.y and extent.height members of any element of pDeviceRenderAreas **must** be less than or equal to maxFramebufferHeight
- VUID-VkDeviceGroupRenderPassBeginInfo-extent-08998
  The extent.width member of any element of pDeviceRenderAreas **must** be greater than 0
- VUID-VkDeviceGroupRenderPassBeginInfo-extent-08999
  The extent.height member of any element of pDeviceRenderAreas **must** be greater than 0

Valid Usage (Implicit)

- VUID-VkDeviceGroupRenderPassBeginInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEVICE_GROUP_RENDER_PASS_BEGIN_INFO
- VUID-VkDeviceGroupRenderPassBeginInfo-pDeviceRenderAreas-parameter
  If deviceRenderAreaCount is not 0, pDeviceRenderAreas **must** be a valid pointer to an array of deviceRenderAreaCount VkRect2D structures

The VkRenderPassAttachmentBeginInfo structure is defined as:

```
typedef struct VkRenderPassAttachmentBeginInfo {
    VkStructureType       sType;
    const void*           pNext;
    uint32_t              attachmentCount;
    const VkImageView*    pAttachments;
} VkRenderPassAttachmentBeginInfo;
```

or the equivalent

```
// Provided by VK_KHR_imageless_framebuffer
typedef VkRenderPassAttachmentBeginInfo VkRenderPassAttachmentBeginInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `attachmentCount` is the number of attachments.
- `pAttachments` is a pointer to an array of `VkImageView` handles, each of which will be used as the corresponding attachment in the render pass instance.

Valid Usage

- VUID-VkRenderPassAttachmentBeginInfo-pAttachments-03218
  Each element of `pAttachments` **must** only specify a single mip level
- VUID-VkRenderPassAttachmentBeginInfo-pAttachments-03219
  Each element of `pAttachments` **must** have been created with the identity swizzle
- VUID-VkRenderPassAttachmentBeginInfo-pAttachments-04114
  Each element of `pAttachments` **must** have been created with VkImageViewCreateInfo ::`viewType` not equal to `VK_IMAGE_VIEW_TYPE_3D`

Valid Usage (Implicit)

- VUID-VkRenderPassAttachmentBeginInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDER_PASS_ATTACHMENT_BEGIN_INFO`
- VUID-VkRenderPassAttachmentBeginInfo-pAttachments-parameter
  If `attachmentCount` is not `0`, `pAttachments` **must** be a valid pointer to an array of `attachmentCount` valid VkImageView handles

To query the render area granularity, call:

```
// Provided by VK_VERSION_1_0
void vkGetRenderAreaGranularity(
    VkDevice                                    device,
    VkRenderPass                                renderPass,
    VkExtent2D*                                 pGranularity);
```

- `device` is the logical device that owns the render pass.
- `renderPass` is a handle to a render pass.
- `pGranularity` is a pointer to a VkExtent2D structure in which the granularity is returned.

The conditions leading to an optimal `renderArea` are:

- the `offset.x` member in `renderArea` is a multiple of the `width` member of the returned VkExtent2D (the horizontal granularity).
- the `offset.y` member in `renderArea` is a multiple of the `height` member of the returned VkExtent2D (the vertical granularity).
- either the `extent.width` member in `renderArea` is a multiple of the horizontal granularity or `offset.x`+`extent.width` is equal to the `width` of the `framebuffer` in the VkRenderPassBeginInfo.
- either the `extent.height` member in `renderArea` is a multiple of the vertical granularity or `offset.y`+`extent.height` is equal to the `height` of the `framebuffer` in the VkRenderPassBeginInfo.

Subpass dependencies are not affected by the render area, and apply to the entire image subresources attached to the framebuffer as specified in the description of automatic layout transitions. Similarly, pipeline barriers are valid even if their effect extends outside the render area.

Valid Usage (Implicit)

- VUID-vkGetRenderAreaGranularity-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetRenderAreaGranularity-renderPass-parameter
  `renderPass` **must** be a valid VkRenderPass handle
- VUID-vkGetRenderAreaGranularity-pGranularity-parameter
  `pGranularity` **must** be a valid pointer to a VkExtent2D structure
- VUID-vkGetRenderAreaGranularity-renderPass-parent
  `renderPass` **must** have been created, allocated, or retrieved from `device`

To transition to the next subpass in the render pass instance after recording the commands for a subpass, call:

```
// Provided by VK_VERSION_1_0
void vkCmdNextSubpass(
    VkCommandBuffer                             commandBuffer,
    VkSubpassContents                           contents);
```

- `commandBuffer` is the command buffer in which to record the command.
- `contents` specifies how the commands in the next subpass will be provided, in the same fashion as the corresponding parameter of vkCmdBeginRenderPass.

The subpass index for a render pass begins at zero when `vkCmdBeginRenderPass` is recorded, and increments each time `vkCmdNextSubpass` is recorded.

After transitioning to the next subpass, the application **can** record the commands for that subpass.

Valid Usage

- VUID-vkCmdNextSubpass-None-00909
  The current subpass index **must** be less than the number of subpasses in the render pass minus one
- VUID-vkCmdNextSubpass-None-02349
  This command **must** not be recorded when transform feedback is active

Valid Usage (Implicit)

- VUID-vkCmdNextSubpass-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdNextSubpass-contents-parameter
  `contents` **must** be a valid VkSubpassContents value
- VUID-vkCmdNextSubpass-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdNextSubpass-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdNextSubpass-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdNextSubpass-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdNextSubpass-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Inside	Outside	Graphics	Action State Synchronization

To transition to the next subpass in the render pass instance after recording the commands for a subpass, call:

```
// Provided by VK_KHR_create_renderpass2
void vkCmdNextSubpass2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkSubpassBeginInfo*                   pSubpassBeginInfo,
    const VkSubpassEndInfo*                     pSubpassEndInfo);
```

- `commandBuffer` is the command buffer in which to record the command.
- `pSubpassBeginInfo` is a pointer to a VkSubpassBeginInfo structure containing information about the subpass which is about to begin rendering.
- `pSubpassEndInfo` is a pointer to a VkSubpassEndInfo structure containing information about how the previous subpass will be ended.

`vkCmdNextSubpass2KHR` is semantically identical to vkCmdNextSubpass, except that it is extensible, and that `contents` is provided as part of an extensible structure instead of as a flat parameter.

Valid Usage

- VUID-vkCmdNextSubpass2-None-03102
  The current subpass index **must** be less than the number of subpasses in the render pass minus one
- VUID-vkCmdNextSubpass2-None-02350
  This command **must** not be recorded when transform feedback is active

Valid Usage (Implicit)

- VUID-vkCmdNextSubpass2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdNextSubpass2-pSubpassBeginInfo-parameter
  `pSubpassBeginInfo` **must** be a valid pointer to a valid VkSubpassBeginInfo structure
- VUID-vkCmdNextSubpass2-pSubpassEndInfo-parameter
  `pSubpassEndInfo` **must** be a valid pointer to a valid VkSubpassEndInfo structure
- VUID-vkCmdNextSubpass2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdNextSubpass2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdNextSubpass2-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdNextSubpass2-videocoding

This command **must** only be called outside of a video coding scope

- VUID-vkCmdNextSubpass2-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Inside	Outside	Graphics	Action State Synchronization

To record a command to end a render pass instance after recording the commands for the last subpass, call:

```
// Provided by VK_VERSION_1_0
void vkCmdEndRenderPass(
    VkCommandBuffer                             commandBuffer);
```

- `commandBuffer` is the command buffer in which to end the current render pass instance.

Ending a render pass instance performs any multisample resolve operations on the final subpass.

Valid Usage

- VUID-vkCmdEndRenderPass-None-00910
  The current subpass index **must** be equal to the number of subpasses in the render pass minus one
- VUID-vkCmdEndRenderPass-None-02351
  This command **must** not be recorded when transform feedback is active
- VUID-vkCmdEndRenderPass-None-06170
  The current render pass instance **must** not have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdEndRenderPass-None-07004
  If `vkCmdBeginQuery*` was called within a subpass of the render pass, the corresponding

vkCmdEndQuery* **must** have been called subsequently within the same subpass

Valid Usage (Implicit)

- VUID-vkCmdEndRenderPass-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdEndRenderPass-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdEndRenderPass-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdEndRenderPass-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdEndRenderPass-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdEndRenderPass-bufferlevel
  commandBuffer **must** be a primary VkCommandBuffer

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Inside	Outside	Graphics	Action State Synchronization

To record a command to end a render pass instance after recording the commands for the last subpass, call:

```
// Provided by VK_KHR_create_renderpass2
void vkCmdEndRenderPass2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkSubpassEndInfo*                     pSubpassEndInfo);
```

- `commandBuffer` is the command buffer in which to end the current render pass instance.
- `pSubpassEndInfo` is a pointer to a VkSubpassEndInfo structure containing information about how the last subpass will be ended.

`vkCmdEndRenderPass2KHR` is semantically identical to vkCmdEndRenderPass, except that it is extensible.

Valid Usage

- VUID-vkCmdEndRenderPass2-None-03103
  The current subpass index **must** be equal to the number of subpasses in the render pass minus one
- VUID-vkCmdEndRenderPass2-None-02352
  This command **must** not be recorded when transform feedback is active
- VUID-vkCmdEndRenderPass2-None-06171
  The current render pass instance **must** not have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdEndRenderPass2-None-07005
  If `vkCmdBeginQuery`* was called within a subpass of the render pass, the corresponding `vkCmdEndQuery`* **must** have been called subsequently within the same subpass

Valid Usage (Implicit)

- VUID-vkCmdEndRenderPass2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdEndRenderPass2-pSubpassEndInfo-parameter
  `pSubpassEndInfo` **must** be a valid pointer to a valid VkSubpassEndInfo structure
- VUID-vkCmdEndRenderPass2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdEndRenderPass2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdEndRenderPass2-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdEndRenderPass2-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdEndRenderPass2-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized

- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Inside	Outside	Graphics	Action State Synchronization

The VkSubpassEndInfo structure is defined as:

```
typedef struct VkSubpassEndInfo {
    VkStructureType    sType;
    const void*        pNext;
} VkSubpassEndInfo;
```

or the equivalent

```
// Provided by VK_KHR_create_renderpass2
typedef VkSubpassEndInfo VkSubpassEndInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

Valid Usage (Implicit)

- VUID-VkSubpassEndInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SUBPASS_END_INFO
- VUID-VkSubpassEndInfo-pNext-pNext
  pNext **must** be NULL

8.9. Common Render Pass Data Races (Informative)

Due to the complexity of how rendering is performed, there are several ways an application can accidentally introduce a data race, usually by doing something that may seem benign but actually cannot be supported. This section indicates a number of the more common cases as guidelines to help avoid them.

8.9.1. Sampling From a Read-only Attachment

Vulkan includes read-only layouts for depth/stencil images, that allow the images to be both read during a render pass for the purposes of depth/stencil tests, and read as a non-attachment.

However, because VK_ATTACHMENT_STORE_OP_STORE and VK_ATTACHMENT_STORE_OP_DONT_CARE may perform write operations, even if no recorded command writes to an attachment, reading from an image while also using it as an attachment with these store operations can result in a data race. If the reads from the non-attachment are performed in a fragment shader where the accessed samples match those covered by the fragment shader, no data race will occur as store operations are guaranteed to operate after fragment shader execution for the set of samples the fragment covers. Notably, input attachments can also be used for this case. Reading other samples or in any other shader stage can result in unexpected behavior due to the potential for a data race, and validation errors should be generated for doing so. In practice, many applications have shipped reading samples outside of the covered fragment without any observable issue, but there is no guarantee that this will always work, and it is not advisable to rely on this in new or re-worked code bases. As VK_ATTACHMENT_STORE_OP_NONE is guaranteed to perform no writes, applications wishing to read an image as both an attachment and a non-attachment should make use of this store operation, coupled with a load operation that also performs no writes.

8.9.2. Non-overlapping Access Between Resources

When relying on non-overlapping accesses between attachments and other resources, it is important to note that load and store operations have fairly wide alignment requirements - potentially affecting entire subresources and adjacent depth/stencil aspects. This makes it invalid to access a non-attachment subresource that is simultaneously being used as an attachment where either access performs a write operation.

The only exception to this is if a subresource is in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, in which case the overlap is defined to occur at a per-pixel granularity, and applications can read data from pixels outside the render area without introducing a data race.

8.9.3. Depth/Stencil and Input Attachments

When rendering to only the depth OR stencil aspect of an image, an input attachment accessing the other aspect will not cause a data race only under very specific conditions. To avoid a data race, the aspect not being written **must** be in a read-only layout, and writes to it **must** be disabled in the draw state. For example, to read from stencil while writing depth, the attachment **must** be in VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL (or equivalent), and the stencil write mask **must** be 0. Similarly to read from depth while writing stencil, the attachment **must** be in VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL (or equivalent), and depth write enable **must** be VK_FALSE.

8.9.4. Synchronization Options

There are several synchronization options available to synchronize between accesses to resources within a render pass. Some of the options are outlined below:

- A VkSubpassDependency in a render pass object can synchronize attachment writes and multisample resolve operations from a prior subpass for subsequent input attachment reads.
- A vkCmdPipelineBarrier inside a subpass can synchronize prior attachment writes in the subpass with subsequent input attachment reads.
- A vkCmdPipelineBarrier inside a subpass can synchronize prior attachment writes in the subpass with subsequent non-attachment reads if the attachment is in the `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` image layout.
- If a subresource is used as two separate non-attachment resources, writes to a pixel or individual sample in a fragment shader can be synchronized with access to the same pixel or sample in another fragment shader by using one of the fragment interlock execution modes.

Chapter 9. Shaders

A shader specifies programmable operations that execute for each vertex, control point, tessellated vertex, primitive, fragment, or workgroup in the corresponding stage(s) of the graphics and compute pipelines.

Graphics pipelines include vertex shader execution as a result of primitive assembly, followed, if enabled, by tessellation control and evaluation shaders operating on patches, geometry shaders, if enabled, operating on primitives, and fragment shaders, if present, operating on fragments generated by Rasterization. In this specification, vertex, tessellation control, tessellation evaluation and geometry shaders are collectively referred to as pre-rasterization shader stages and occur in the logical pipeline before rasterization. The fragment shader occurs logically after rasterization.

Only the compute shader stage is included in a compute pipeline. Compute shaders operate on compute invocations in a workgroup.

Shaders **can** read from input variables, and read from and write to output variables. Input and output variables **can** be used to transfer data between shader stages, or to allow the shader to interact with values that exist in the execution environment. Similarly, the execution environment provides constants describing capabilities.

Shader variables are associated with execution environment-provided inputs and outputs using *built-in* decorations in the shader. The available decorations for each stage are documented in the following subsections.

9.1. Shader Objects

Shaders **may** be compiled and linked into pipeline objects as described in Pipelines chapter, or if the `shaderObject` feature is enabled they **may** be compiled into individual per-stage *shader objects* which **can** be bound on a command buffer independently from one another. Unlike pipelines, shader objects are not intrinsically tied to any specific set of state. Instead, state is specified dynamically in the command buffer.

Each shader object represents a single compiled shader stage, which **may optionally** be linked with one or more other stages.

Shader objects are represented by `VkShaderEXT` handles:

```
// Provided by VK_EXT_shader_object
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkShaderEXT)
```

9.1.1. Shader Object Creation

Shader objects **may** be created from shader code provided as SPIR-V, or in an opaque, implementation-defined binary format specific to the physical device.

To create one or more shader objects, call:

```
// Provided by VK_EXT_shader_object
VkResult vkCreateShadersEXT(
    VkDevice                                    device,
    uint32_t                                    createInfoCount,
    const VkShaderCreateInfoEXT*                pCreateInfos,
    const VkAllocationCallbacks*                pAllocator,
    VkShaderEXT*                                pShaders);
```

- `device` is the logical device that creates the shader objects.
- `createInfoCount` is the length of the `pCreateInfos` and `pShaders` arrays.
- `pCreateInfos` is a pointer to an array of VkShaderCreateInfoEXT structures.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pShaders` is a pointer to an array of VkShaderEXT handles in which the resulting shader objects are returned.

When this function returns, whether or not it succeeds, it is guaranteed that every element of `pShaders` will have been overwritten by either VK_NULL_HANDLE or a valid `VkShaderEXT` handle.

This means that whenever shader creation fails, the application **can** determine which shader the returned error pertains to by locating the first VK_NULL_HANDLE element in `pShaders`. It also means that an application **can** reliably clean up from a failed call by iterating over the `pShaders` array and destroying every element that is not VK_NULL_HANDLE.

Valid Usage

- VUID-vkCreateShadersEXT-stage-09670
  If the `stage` member of any element of `pCreateInfos` is `VK_SHADER_STAGE_COMPUTE_BIT`, `device` **must** support at least one queue family with the `VK_QUEUE_COMPUTE_BIT` capability
- VUID-vkCreateShadersEXT-stage-09671
  If the `stage` member of any element of `pCreateInfos` is `VK_SHADER_STAGE_VERTEX_BIT`, `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT`, `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT`, `VK_SHADER_STAGE_GEOMETRY_BIT`, or `VK_SHADER_STAGE_FRAGMENT_BIT`, `device` **must** support at least one queue family with the `VK_QUEUE_GRAPHICS_BIT` capability
- VUID-vkCreateShadersEXT-None-08400
  The `shaderObject` feature **must** be enabled
- VUID-vkCreateShadersEXT-pCreateInfos-08402
  If the `flags` member of any element of `pCreateInfos` includes `VK_SHADER_CREATE_LINK_STAGE_BIT_EXT`, the `flags` member of all other elements of `pCreateInfos` whose `stage` is `VK_SHADER_STAGE_VERTEX_BIT`, `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT`, `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT`, `VK_SHADER_STAGE_GEOMETRY_BIT`, or `VK_SHADER_STAGE_FRAGMENT_BIT` **must** also include `VK_SHADER_CREATE_LINK_STAGE_BIT_EXT`
- VUID-vkCreateShadersEXT-pCreateInfos-08409
  For each element of `pCreateInfos` whose `flags` member includes

VK_SHADER_CREATE_LINK_STAGE_BIT_EXT, if there is any other element of pCreateInfos whose stage is logically later than the stage of the former and whose flags member also includes VK_SHADER_CREATE_LINK_STAGE_BIT_EXT, the nextStage of the former **must** be equal to the stage of the element with the logically earliest stage following the stage of the former whose flags member also includes VK_SHADER_CREATE_LINK_STAGE_BIT_EXT

- VUID-vkCreateShadersEXT-pCreateInfos-08410
  The stage member of each element of pCreateInfos whose flags member includes VK_SHADER_CREATE_LINK_STAGE_BIT_EXT **must** be unique
- VUID-vkCreateShadersEXT-pCreateInfos-08411
  The codeType member of all elements of pCreateInfos whose flags member includes VK_SHADER_CREATE_LINK_STAGE_BIT_EXT **must** be the same
- VUID-vkCreateShadersEXT-pCreateInfos-08867
  If pCreateInfos contains elements with both VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT and VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, both elements' flags include VK_SHADER_CREATE_LINK_STAGE_BIT_EXT, both elements' codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT stage's pCode contains an OpExecutionMode instruction specifying the type of subdivision, it **must** match the subdivision type specified in the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage
- VUID-vkCreateShadersEXT-pCreateInfos-08868
  If pCreateInfos contains elements with both VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT and VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, both elements' flags include VK_SHADER_CREATE_LINK_STAGE_BIT_EXT, both elements' codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT stage's pCode contains an OpExecutionMode instruction specifying the orientation of triangles, it **must** match the triangle orientation specified in the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage
- VUID-vkCreateShadersEXT-pCreateInfos-08869
  If pCreateInfos contains elements with both VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT and VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, both elements' flags include VK_SHADER_CREATE_LINK_STAGE_BIT_EXT, both elements' codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT stage's pCode contains an OpExecutionMode instruction specifying PointMode, the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage **must** also contain an OpExecutionMode instruction specifying PointMode
- VUID-vkCreateShadersEXT-pCreateInfos-08870
  If pCreateInfos contains elements with both VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT and VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, both elements' flags include VK_SHADER_CREATE_LINK_STAGE_BIT_EXT, both elements' codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT stage's pCode contains an OpExecutionMode instruction specifying the spacing of segments on the edges of tessellated primitives, it **must** match the segment spacing specified in the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage
- VUID-vkCreateShadersEXT-pCreateInfos-08871
  If pCreateInfos contains elements with both VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT

and VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, both elements' flags include VK_SHADER_CREATE_LINK_STAGE_BIT_EXT, both elements' codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT stage's pCode contains an OpExecutionMode instruction specifying the output patch size, it **must** match the output patch size specified in the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage

Valid Usage (Implicit)

- VUID-vkCreateShadersEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateShadersEXT-pCreateInfos-parameter
  pCreateInfos **must** be a valid pointer to an array of createInfoCount valid VkShaderCreateInfoEXT structures
- VUID-vkCreateShadersEXT-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateShadersEXT-pShaders-parameter
  pShaders **must** be a valid pointer to an array of createInfoCount VkShaderEXT handles
- VUID-vkCreateShadersEXT-createInfoCount-arraylength
  createInfoCount **must** be greater than 0

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPATIBLE_SHADER_BINARY_EXT

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INITIALIZATION_FAILED

The VkShaderCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_shader_object
typedef struct VkShaderCreateInfoEXT {
    VkStructureType                 sType;
    const void*                     pNext;
    VkShaderCreateFlagsEXT          flags;
    VkShaderStageFlagBits           stage;
    VkShaderStageFlags              nextStage;
    VkShaderCodeTypeEXT             codeType;
    size_t                          codeSize;
    const void*                     pCode;
    const char*                     pName;
    uint32_t                        setLayoutCount;
    const VkDescriptorSetLayout*    pSetLayouts;
    uint32_t                        pushConstantRangeCount;
    const VkPushConstantRange*      pPushConstantRanges;
    const VkSpecializationInfo*     pSpecializationInfo;
} VkShaderCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkShaderCreateFlagBitsEXT describing additional parameters of the shader.
- `stage` is a VkShaderStageFlagBits value specifying a single shader stage.
- `nextStage` is a bitmask of VkShaderStageFlagBits specifying zero or stages which **may** be used as a logically next bound stage when drawing with the shader bound.
- `codeType` is a VkShaderCodeTypeEXT value specifying the type of the shader code pointed to be `pCode`.
- `codeSize` is the size in bytes of the shader code pointed to be `pCode`.
- `pCode` is a pointer to the shader code to use to create the shader.
- `pName` is a pointer to a null-terminated UTF-8 string specifying the entry point name of the shader for this stage.
- `setLayoutCount` is the number of descriptor set layouts pointed to by `pSetLayouts`.
- `pSetLayouts` is a pointer to an array of VkDescriptorSetLayout objects used by the shader stage.
- `pushConstantRangeCount` is the number of push constant ranges pointed to by `pPushConstantRanges`.
- `pPushConstantRanges` is a pointer to an array of VkPushConstantRange structures used by the shader stage.
- `pSpecializationInfo` is a pointer to a VkSpecializationInfo structure, as described in Specialization Constants, or `NULL`.

Valid Usage

- VUID-VkShaderCreateInfoEXT-codeSize-08735
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, `codeSize` **must** be a multiple of 4
- VUID-VkShaderCreateInfoEXT-pCode-08736
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, `pCode` **must** point to valid SPIR-V code, formatted and packed as described by the Khronos SPIR-V Specification
- VUID-VkShaderCreateInfoEXT-pCode-08737
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, `pCode` **must** adhere to the validation rules described by the Validation Rules within a Module section of the SPIR-V Environment appendix
- VUID-VkShaderCreateInfoEXT-pCode-08738
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, `pCode` **must** declare the `Shader` capability for SPIR-V code
- VUID-VkShaderCreateInfoEXT-pCode-08739
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, `pCode` **must** not declare any capability that is not supported by the API, as described by the Capabilities section of the SPIR-V Environment appendix
- VUID-VkShaderCreateInfoEXT-pCode-08740
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, and `pCode` declares any of the capabilities listed in the SPIR-V Environment appendix, one of the corresponding requirements **must** be satisfied
- VUID-VkShaderCreateInfoEXT-pCode-08741
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, `pCode` **must** not declare any SPIR-V extension that is not supported by the API, as described by the Extension section of the SPIR-V Environment appendix
- VUID-VkShaderCreateInfoEXT-pCode-08742
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, and `pCode` declares any of the SPIR-V extensions listed in the SPIR-V Environment appendix, one of the corresponding requirements **must** be satisfied
- VUID-VkShaderCreateInfoEXT-flags-08412
  If `stage` is not `VK_SHADER_STAGE_TASK_BIT_EXT`, `VK_SHADER_STAGE_MESH_BIT_EXT`, `VK_SHADER_STAGE_VERTEX_BIT`, `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT`, `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT`, `VK_SHADER_STAGE_GEOMETRY_BIT`, or `VK_SHADER_STAGE_FRAGMENT_BIT`, `flags` **must** not include `VK_SHADER_CREATE_LINK_STAGE_BIT_EXT`
- VUID-VkShaderCreateInfoEXT-flags-08486
  If `stage` is not `VK_SHADER_STAGE_FRAGMENT_BIT`, `flags` **must** not include `VK_SHADER_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_EXT`
- VUID-VkShaderCreateInfoEXT-flags-08487
  If the `attachmentFragmentShadingRate` feature is not enabled, `flags` **must** not include `VK_SHADER_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_EXT`
- VUID-VkShaderCreateInfoEXT-flags-08488

If stage is not VK_SHADER_STAGE_FRAGMENT_BIT, flags **must** not include VK_SHADER_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT

- VUID-VkShaderCreateInfoEXT-flags-08489
  If the fragmentDensityMap feature is not enabled, flags **must** not include VK_SHADER_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-VkShaderCreateInfoEXT-flags-08485
  If stage is not VK_SHADER_STAGE_COMPUTE_BIT, flags **must** not include VK_SHADER_CREATE_DISPATCH_BASE_BIT_EXT
- VUID-VkShaderCreateInfoEXT-stage-08418
  stage **must** not be VK_SHADER_STAGE_ALL_GRAPHICS or VK_SHADER_STAGE_ALL
- VUID-VkShaderCreateInfoEXT-stage-08419
  If the tessellationShader feature is not enabled, stage **must** not be VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT or VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-VkShaderCreateInfoEXT-stage-08420
  If the geometryShader feature is not enabled, stage **must** not be VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-VkShaderCreateInfoEXT-nextStage-08427
  If stage is VK_SHADER_STAGE_VERTEX_BIT, nextStage **must** not include any bits other than VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT, VK_SHADER_STAGE_GEOMETRY_BIT, and VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-VkShaderCreateInfoEXT-nextStage-08428
  If the tessellationShader feature is not enabled, nextStage **must** not include VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT or VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-VkShaderCreateInfoEXT-nextStage-08429
  If the geometryShader feature is not enabled, nextStage **must** not include VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-VkShaderCreateInfoEXT-nextStage-08430
  If stage is VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT, nextStage **must** not include any bits other than VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-VkShaderCreateInfoEXT-nextStage-08431
  If stage is VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, nextStage **must** not include any bits other than VK_SHADER_STAGE_GEOMETRY_BIT and VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-VkShaderCreateInfoEXT-nextStage-08433
  If stage is VK_SHADER_STAGE_GEOMETRY_BIT, nextStage **must** not include any bits other than VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-VkShaderCreateInfoEXT-nextStage-08434
  If stage is VK_SHADER_STAGE_FRAGMENT_BIT or VK_SHADER_STAGE_COMPUTE_BIT, nextStage **must** be 0
- VUID-VkShaderCreateInfoEXT-pName-08440
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, pName **must** be the name of an OpEntryPoint in pCode with an execution model that matches stage

- VUID-VkShaderCreateInfoEXT-pCode-08492
  If codeType is VK_SHADER_CODE_TYPE_BINARY_EXT, pCode **must** be aligned to 16 bytes
- VUID-VkShaderCreateInfoEXT-pCode-08493
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, pCode **must** be aligned to 4 bytes
- VUID-VkShaderCreateInfoEXT-pCode-08448
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the identified entry point includes any variable in its interface that is declared with the ClipDistance BuiltIn decoration, that variable **must** not have an array size greater than VkPhysicalDeviceLimits ::maxClipDistances
- VUID-VkShaderCreateInfoEXT-pCode-08449
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the identified entry point includes any variable in its interface that is declared with the CullDistance BuiltIn decoration, that variable **must** not have an array size greater than VkPhysicalDeviceLimits ::maxCullDistances
- VUID-VkShaderCreateInfoEXT-pCode-08450
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the identified entry point includes variables in its interface that are declared with the ClipDistance BuiltIn decoration and variables in its interface that are declared with the CullDistance BuiltIn decoration, those variables **must** not have array sizes which sum to more than VkPhysicalDeviceLimits ::maxCombinedClipAndCullDistances
- VUID-VkShaderCreateInfoEXT-pCode-08451
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and the identified entry point includes any variable in its interface that is declared with the SampleMask BuiltIn decoration, that variable **must** not have an array size greater than VkPhysicalDeviceLimits ::maxSampleMaskWords
- VUID-VkShaderCreateInfoEXT-pCode-08452
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_VERTEX_BIT, the identified entry point **must** not include any input variable in its interface that is decorated with CullDistance
- VUID-VkShaderCreateInfoEXT-pCode-08453
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT or VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, and the identified entry point has an OpExecutionMode instruction specifying a patch size with OutputVertices, the patch size **must** be greater than 0 and less than or equal to VkPhysicalDeviceLimits ::maxTessellationPatchSize
- VUID-VkShaderCreateInfoEXT-pCode-08454
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_GEOMETRY_BIT, the identified entry point **must** have an OpExecutionMode instruction specifying a maximum output vertex count that is greater than 0 and less than or equal to VkPhysicalDeviceLimits::maxGeometryOutputVertices
- VUID-VkShaderCreateInfoEXT-pCode-08455
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_GEOMETRY_BIT, the identified entry point **must** have an OpExecutionMode instruction specifying an

invocation count that is greater than 0 and less than or equal to VkPhysicalDeviceLimits ::maxGeometryShaderInvocations

- VUID-VkShaderCreateInfoEXT-pCode-08456
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is a pre-rasterization shader stage, and the identified entry point writes to Layer for any primitive, it **must** write the same value to Layer for all vertices of a given primitive
- VUID-VkShaderCreateInfoEXT-pCode-08457
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is a pre-rasterization shader stage, and the identified entry point writes to ViewportIndex for any primitive, it **must** write the same value to ViewportIndex for all vertices of a given primitive
- VUID-VkShaderCreateInfoEXT-pCode-08458
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_FRAGMENT_BIT, the identified entry point **must** not include any output variables in its interface decorated with CullDistance
- VUID-VkShaderCreateInfoEXT-pCode-08459
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_FRAGMENT_BIT, and the identified entry point writes to FragDepth in any execution path, all execution paths that are not exclusive to helper invocations **must** either discard the fragment, or write or initialize the value of FragDepth
- VUID-VkShaderCreateInfoEXT-pCode-08460
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, the shader code in pCode **must** be valid as described by the Khronos SPIR-V Specification after applying the specializations provided in pSpecializationInfo, if any, and then converting all specialization constants into fixed constants
- VUID-VkShaderCreateInfoEXT-codeType-08872
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, pCode **must** contain an OpExecutionMode instruction specifying the type of subdivision
- VUID-VkShaderCreateInfoEXT-codeType-08873
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, pCode **must** contain an OpExecutionMode instruction specifying the orientation of triangles generated by the tessellator
- VUID-VkShaderCreateInfoEXT-codeType-08874
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, pCode **must** contain an OpExecutionMode instruction specifying the spacing of segments on the edges of tessellated primitives
- VUID-VkShaderCreateInfoEXT-codeType-08875
  If codeType is VK_SHADER_CODE_TYPE_SPIRV_EXT, and stage is VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, pCode **must** contain an OpExecutionMode instruction specifying the output patch size
- VUID-VkShaderCreateInfoEXT-pPushConstantRanges-10063
  Any two elements of pPushConstantRanges **must** not include the same stage in stageFlags
- VUID-VkShaderCreateInfoEXT-codeType-10064

If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, and if a push constant block is declared in a shader, then an element of `pPushConstantRanges`::`stageFlags` **must** match `stage`

- VUID-VkShaderCreateInfoEXT-codeType-10065
  If `codeType` is `VK_SHADER_CODE_TYPE_SPIRV_EXT`, and if a push constant block is declared in a shader, the block must be contained inside the element of `pPushConstantRanges` that matches the stage

Valid Usage (Implicit)

- VUID-VkShaderCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT`
- VUID-VkShaderCreateInfoEXT-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineShaderStageRequiredSubgroupSizeCreateInfo
- VUID-VkShaderCreateInfoEXT-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkShaderCreateInfoEXT-flags-parameter
  `flags` **must** be a valid combination of VkShaderCreateFlagBitsEXT values
- VUID-VkShaderCreateInfoEXT-stage-parameter
  `stage` **must** be a valid VkShaderStageFlagBits value
- VUID-VkShaderCreateInfoEXT-nextStage-parameter
  `nextStage` **must** be a valid combination of VkShaderStageFlagBits values
- VUID-VkShaderCreateInfoEXT-codeType-parameter
  `codeType` **must** be a valid VkShaderCodeTypeEXT value
- VUID-VkShaderCreateInfoEXT-pCode-parameter
  `pCode` **must** be a valid pointer to an array of `codeSize` bytes
- VUID-VkShaderCreateInfoEXT-pName-parameter
  If `pName` is not `NULL`, `pName` **must** be a null-terminated UTF-8 string
- VUID-VkShaderCreateInfoEXT-pSetLayouts-parameter
  If `setLayoutCount` is not `0`, and `pSetLayouts` is not `NULL`, `pSetLayouts` **must** be a valid pointer to an array of `setLayoutCount` valid VkDescriptorSetLayout handles
- VUID-VkShaderCreateInfoEXT-pPushConstantRanges-parameter
  If `pushConstantRangeCount` is not `0`, and `pPushConstantRanges` is not `NULL`, `pPushConstantRanges` **must** be a valid pointer to an array of `pushConstantRangeCount` valid VkPushConstantRange structures
- VUID-VkShaderCreateInfoEXT-pSpecializationInfo-parameter
  If `pSpecializationInfo` is not `NULL`, `pSpecializationInfo` **must** be a valid pointer to a valid VkSpecializationInfo structure
- VUID-VkShaderCreateInfoEXT-codeSize-arraylength
  `codeSize` **must** be greater than `0`

```
// Provided by VK_EXT_shader_object
typedef VkFlags VkShaderCreateFlagsEXT;
```

VkShaderCreateFlagsEXT is a bitmask type for setting a mask of zero or more VkShaderCreateFlagBitsEXT.

Possible values of the flags member of VkShaderCreateInfoEXT specifying how a shader object is created, are:

```
// Provided by VK_EXT_shader_object
typedef enum VkShaderCreateFlagBitsEXT {
    VK_SHADER_CREATE_LINK_STAGE_BIT_EXT = 0x00000001,
  // Provided by VK_EXT_shader_object with VK_EXT_subgroup_size_control or
VK_VERSION_1_3
    VK_SHADER_CREATE_ALLOW_VARYING_SUBGROUP_SIZE_BIT_EXT = 0x00000002,
  // Provided by VK_EXT_shader_object with VK_EXT_subgroup_size_control or
VK_VERSION_1_3
    VK_SHADER_CREATE_REQUIRE_FULL_SUBGROUPS_BIT_EXT = 0x00000004,
  // Provided by VK_EXT_shader_object with VK_EXT_mesh_shader or VK_NV_mesh_shader
    VK_SHADER_CREATE_NO_TASK_SHADER_BIT_EXT = 0x00000008,
  // Provided by VK_EXT_shader_object with VK_KHR_device_group or VK_VERSION_1_1
    VK_SHADER_CREATE_DISPATCH_BASE_BIT_EXT = 0x00000010,
  // Provided by VK_KHR_fragment_shading_rate with VK_EXT_shader_object
    VK_SHADER_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_EXT = 0x00000020,
  // Provided by VK_EXT_fragment_density_map with VK_EXT_shader_object
    VK_SHADER_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT = 0x00000040,
  // Provided by VK_EXT_device_generated_commands
    VK_SHADER_CREATE_INDIRECT_BINDABLE_BIT_EXT = 0x00000080,
} VkShaderCreateFlagBitsEXT;
```

- VK_SHADER_CREATE_LINK_STAGE_BIT_EXT specifies that a shader is linked to all other shaders created in the same vkCreateShadersEXT call whose VkShaderCreateInfoEXT structures' flags include VK_SHADER_CREATE_LINK_STAGE_BIT_EXT.
- VK_SHADER_CREATE_DISPATCH_BASE_BIT_EXT specifies that a compute shader **can** be used with vkCmdDispatchBaseKHR with a non-zero base workgroup.
- VK_SHADER_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_EXT specifies that a fragment shader **can** be used with a fragment shading rate attachment.
- VK_SHADER_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT specifies that a fragment shader **can** be used with a fragment density map attachment.
- VK_SHADER_CREATE_INDIRECT_BINDABLE_BIT_EXT specifies that the shader **can** be used in combination with Device-Generated Commands.

NOTE

The behavior of VK_SHADER_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_EXT and VK_SHADER_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT differs subtly from the behavior of VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR and

> VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT in that the shader bit allows, but does not require the shader to be used with that type of attachment. This means that the application need not create multiple shaders when it does not know in advance whether the shader will be used with or without the attachment type, or when it needs the same shader to be compatible with usage both with and without. This **may** come at some performance cost on some implementations, so applications **should** still only set bits that are actually necessary.

Shader objects **can** be created using different types of shader code. Possible values of VkShaderCreateInfoEXT::codeType, are:

```
// Provided by VK_EXT_shader_object
typedef enum VkShaderCodeTypeEXT {
    VK_SHADER_CODE_TYPE_BINARY_EXT = 0,
    VK_SHADER_CODE_TYPE_SPIRV_EXT = 1,
} VkShaderCodeTypeEXT;
```

- VK_SHADER_CODE_TYPE_BINARY_EXT specifies shader code in an opaque, implementation-defined binary format specific to the physical device.
- VK_SHADER_CODE_TYPE_SPIRV_EXT specifies shader code in SPIR-V format.

9.1.2. Binary Shader Code

Binary shader code **can** be retrieved from a shader object using the command:

```
// Provided by VK_EXT_shader_object
VkResult vkGetShaderBinaryDataEXT(
    VkDevice                                    device,
    VkShaderEXT                                 shader,
    size_t*                                     pDataSize,
    void*                                       pData);
```

- device is the logical device that shader object was created from.
- shader is the shader object to retrieve binary shader code from.
- pDataSize is a pointer to a size_t value related to the size of the binary shader code, as described below.
- pData is either NULL or a pointer to a buffer.

If pData is NULL, then the size of the binary shader code of the shader object, in bytes, is returned in pDataSize. Otherwise, pDataSize **must** point to a variable set by the application to the size of the buffer, in bytes, pointed to by pData, and on return the variable is overwritten with the amount of data actually written to pData. If pDataSize is less than the size of the binary shader code, nothing is written to pData, and VK_INCOMPLETE will be returned instead of VK_SUCCESS.

NOTE The behavior of this command when `pDataSize` is too small differs from how some other getter-type commands work in Vulkan. Because shader binary data is only usable in its entirety, it would never be useful for the implementation to return partial data. Because of this, nothing is written to `pData` unless `pDataSize` is large enough to fit the data in its entirety.

Binary shader code retrieved using `vkGetShaderBinaryDataEXT` **can** be passed to a subsequent call to vkCreateShadersEXT on a compatible physical device by specifying `VK_SHADER_CODE_TYPE_BINARY_EXT` in the `codeType` member of `VkShaderCreateInfoEXT`.

The shader code returned by repeated calls to this function with the same `VkShaderEXT` is guaranteed to be invariant for the lifetime of the `VkShaderEXT` object.

Valid Usage

- VUID-vkGetShaderBinaryDataEXT-None-08461
  The `shaderObject` feature **must** be enabled
- VUID-vkGetShaderBinaryDataEXT-None-08499
  If `pData` is not `NULL`, it **must** be aligned to `16` bytes

Valid Usage (Implicit)

- VUID-vkGetShaderBinaryDataEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetShaderBinaryDataEXT-shader-parameter
  `shader` **must** be a valid VkShaderEXT handle
- VUID-vkGetShaderBinaryDataEXT-pDataSize-parameter
  `pDataSize` **must** be a valid pointer to a `size_t` value
- VUID-vkGetShaderBinaryDataEXT-pData-parameter
  If the value referenced by `pDataSize` is not `0`, and `pData` is not `NULL`, `pData` **must** be a valid pointer to an array of `pDataSize` bytes
- VUID-vkGetShaderBinaryDataEXT-shader-parent
  `shader` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

9.1.3. Binary Shader Compatibility

Binary shader compatibility means that binary shader code returned from a call to vkGetShaderBinaryDataEXT **can** be passed to a later call to vkCreateShadersEXT, potentially on a different logical and/or physical device, and that this will result in the successful creation of a shader object functionally equivalent to the shader object that the code was originally queried from.

Binary shader code queried from vkGetShaderBinaryDataEXT is not guaranteed to be compatible across all devices, but implementations are required to provide some compatibility guarantees. Applications **may** determine binary shader compatibility using either (or both) of two mechanisms.

Guaranteed compatibility of shader binaries is expressed through a combination of the `shaderBinaryUUID` and `shaderBinaryVersion` members of the VkPhysicalDeviceShaderObjectPropertiesEXT structure queried from a physical device. Binary shaders retrieved from a physical device with a certain `shaderBinaryUUID` are guaranteed to be compatible with all other physical devices reporting the same `shaderBinaryUUID` and the same or higher `shaderBinaryVersion`.

Whenever a new version of an implementation incorporates any changes that affect the output of vkGetShaderBinaryDataEXT, the implementation **should** either increment `shaderBinaryVersion` if binary shader code retrieved from older versions remains compatible with the new implementation, or else replace `shaderBinaryUUID` with a new value if backward compatibility has been broken. Binary shader code queried from a device with a matching `shaderBinaryUUID` and lower `shaderBinaryVersion` relative to the device on which vkCreateShadersEXT is being called **may** be suboptimal for the new device in ways that do not change shader functionality, but it is still guaranteed to be usable to successfully create the shader object(s).

NOTE

Implementations are encouraged to share `shaderBinaryUUID` between devices and driver versions to the maximum extent their hardware naturally allows, and are **strongly** discouraged from ever changing the `shaderBinaryUUID` for the same hardware except unless absolutely necessary.

In addition to the shader compatibility guarantees described above, it is valid for an application to call vkCreateShadersEXT with binary shader code created on a device with a different or unknown `shaderBinaryUUID` and/or higher `shaderBinaryVersion`. In this case, the implementation **may** use any unspecified means of its choosing to determine whether the provided binary shader code is usable. If it is, vkCreateShadersEXT **must** return `VK_SUCCESS`, and the created shader object is guaranteed to be valid. Otherwise, in the absence of some error, vkCreateShadersEXT **must** return `VK_INCOMPATIBLE_SHADER_BINARY_EXT` to indicate that the provided binary shader code is not compatible with the device.

9.1.4. Binding Shader Objects

Once shader objects have been created, they **can** be bound to the command buffer using the command:

```
// Provided by VK_EXT_shader_object
void vkCmdBindShadersEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    stageCount,
    const VkShaderStageFlagBits*                pStages,
    const VkShaderEXT*                          pShaders);
```

- `commandBuffer` is the command buffer that the shader object will be bound to.
- `stageCount` is the length of the `pStages` and `pShaders` arrays.
- `pStages` is a pointer to an array of VkShaderStageFlagBits values specifying one stage per array index that is affected by the corresponding value in the `pShaders` array.
- `pShaders` is a pointer to an array of `VkShaderEXT` handles and/or VK_NULL_HANDLE values describing the shader binding operations to be performed on each stage in `pStages`.

When binding linked shaders, an application **may** bind them in any combination of one or more calls to `vkCmdBindShadersEXT` (i.e., shaders that were created linked together do not need to be bound in the same `vkCmdBindShadersEXT` call).

Any shader object bound to a particular stage **may** be unbound by setting its value in `pShaders` to VK_NULL_HANDLE. If `pShaders` is `NULL`, `vkCmdBindShadersEXT` behaves as if `pShaders` was an array of `stageCount` VK_NULL_HANDLE values (i.e., any shaders bound to the stages specified in `pStages` are unbound).

Valid Usage

- VUID-vkCmdBindShadersEXT-None-08462
  The `shaderObject` feature **must** be enabled
- VUID-vkCmdBindShadersEXT-pStages-08463
  Every element of `pStages` **must** be unique
- VUID-vkCmdBindShadersEXT-pStages-08464
  `pStages` **must** not contain `VK_SHADER_STAGE_ALL_GRAPHICS` or `VK_SHADER_STAGE_ALL`
- VUID-vkCmdBindShadersEXT-pStages-08465
  `pStages` **must** not contain `VK_SHADER_STAGE_RAYGEN_BIT_KHR`, `VK_SHADER_STAGE_ANY_HIT_BIT_KHR`, `VK_SHADER_STAGE_CLOSEST_HIT_BIT_KHR`, `VK_SHADER_STAGE_MISS_BIT_KHR`, `VK_SHADER_STAGE_INTERSECTION_BIT_KHR`, or `VK_SHADER_STAGE_CALLABLE_BIT_KHR`
- VUID-vkCmdBindShadersEXT-pShaders-08469
  For each element of `pStages`, if `pShaders` is not `NULL`, and the element of the `pShaders` array with the same index is not VK_NULL_HANDLE, it **must** have been created with a `stage` equal to the corresponding element of `pStages`
- VUID-vkCmdBindShadersEXT-pShaders-08474
  If the `tessellationShader` feature is not enabled, and `pStages` contains `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT` or `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT`, and `pShaders` is not `NULL`, the same index or

indices in pShaders **must** be VK_NULL_HANDLE

- VUID-vkCmdBindShadersEXT-pShaders-08475
  If the geometryShader feature is not enabled, and pStages contains VK_SHADER_STAGE_GEOMETRY_BIT, and pShaders is not NULL, the same index in pShaders **must** be VK_NULL_HANDLE
- VUID-vkCmdBindShadersEXT-pShaders-08476
  If pStages contains VK_SHADER_STAGE_COMPUTE_BIT, the VkCommandPool that commandBuffer was allocated from **must** support compute operations
- VUID-vkCmdBindShadersEXT-pShaders-08477
  If pStages contains VK_SHADER_STAGE_VERTEX_BIT, VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT, VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT, VK_SHADER_STAGE_GEOMETRY_BIT, or VK_SHADER_STAGE_FRAGMENT_BIT, the VkCommandPool that commandBuffer was allocated from **must** support graphics operations

Valid Usage (Implicit)

- VUID-vkCmdBindShadersEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindShadersEXT-pStages-parameter
  pStages **must** be a valid pointer to an array of stageCount valid VkShaderStageFlagBits values
- VUID-vkCmdBindShadersEXT-pShaders-parameter
  If pShaders is not NULL, pShaders **must** be a valid pointer to an array of stageCount valid or VK_NULL_HANDLE VkShaderEXT handles
- VUID-vkCmdBindShadersEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdBindShadersEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics, or compute operations
- VUID-vkCmdBindShadersEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBindShadersEXT-stageCount-arraylength
  stageCount **must** be greater than 0
- VUID-vkCmdBindShadersEXT-commonparent
  Both of commandBuffer, and the elements of pShaders that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally

synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

9.1.5. Setting State

Whenever shader objects are used to issue drawing commands, the appropriate dynamic state setting commands **must** have been called to set the relevant state in the command buffer prior to drawing:

- vkCmdSetViewportWithCountEXT
- vkCmdSetScissorWithCountEXT
- vkCmdSetRasterizerDiscardEnableEXT
- vkCmdSetVertexInputEXT
- vkCmdSetPrimitiveTopologyEXT
- vkCmdSetPrimitiveRestartEnableEXT

If a shader is bound to the `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT` stage, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetPatchControlPointsEXT, if `primitiveTopology` is `VK_PRIMITIVE_TOPOLOGY_PATCH_LIST`

If a shader is bound to the `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT` stage, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetTessellationDomainOriginEXT

If `rasterizerDiscardEnable` is `VK_FALSE`, the following commands **must** have been called in the command buffer prior to drawing:

- vkCmdSetRasterizationSamplesEXT
- vkCmdSetSampleMaskEXT
- vkCmdSetAlphaToCoverageEnableEXT
- vkCmdSetAlphaToOneEnableEXT, if the alphaToOne feature is enabled
- vkCmdSetPolygonModeEXT
- vkCmdSetLineWidth, if `polygonMode` is `VK_POLYGON_MODE_LINE`, or if `primitiveTopology` is a line topology, or if a shader which outputs line primitives is bound to the `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT` or `VK_SHADER_STAGE_GEOMETRY_BIT` stage

- vkCmdSetCullModeEXT
- vkCmdSetFrontFaceEXT
- vkCmdSetDepthTestEnableEXT
- vkCmdSetDepthWriteEnableEXT
- vkCmdSetDepthCompareOpEXT, if `depthTestEnable` is `VK_TRUE`
- vkCmdSetDepthBoundsTestEnableEXT, if the depthBounds feature is enabled
- vkCmdSetDepthBounds, if `depthBoundsTestEnable` is `VK_TRUE`
- vkCmdSetDepthBiasEnableEXT
- vkCmdSetDepthBias or vkCmdSetDepthBias2EXT, if `depthBiasEnable` is `VK_TRUE`
- vkCmdSetDepthClampEnableEXT, if the depthClamp feature is enabled
- vkCmdSetStencilTestEnableEXT
- vkCmdSetStencilOpEXT, if `stencilTestEnable` is `VK_TRUE`
- vkCmdSetStencilCompareMask, if `stencilTestEnable` is `VK_TRUE`
- vkCmdSetStencilWriteMask, if `stencilTestEnable` is `VK_TRUE`
- vkCmdSetStencilReference, if `stencilTestEnable` is `VK_TRUE`

If a shader is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, and `rasterizerDiscardEnable` is `VK_FALSE`, the following commands **must** have been called in the command buffer prior to drawing:

- vkCmdSetLogicOpEnableEXT, if the `logicOp` feature is enabled
- vkCmdSetLogicOpEXT, if `logicOpEnable` is `VK_TRUE`
- vkCmdSetColorBlendEnableEXT and vkCmdSetColorWriteMaskEXT, if color attachments are bound, with values set for every color attachment in the render pass instance active at draw time
- vkCmdSetColorBlendEquationEXT or vkCmdSetColorBlendAdvancedEXT, if color attachments are bound, for every attachment whose index in `pColorBlendEnables` is a pointer to a value of `VK_TRUE`
- vkCmdSetBlendConstants, if any index in `pColorBlendEnables` is `VK_TRUE`, and the same index in `pColorBlendEquations` is a `VkColorBlendEquationEXT` structure with any VkBlendFactor member with a value of `VK_BLEND_FACTOR_CONSTANT_COLOR`, `VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR`, `VK_BLEND_FACTOR_CONSTANT_ALPHA`, or `VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA`

If the `pipelineFragmentShadingRate` feature is enabled, and a shader is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, and `rasterizerDiscardEnable` is `VK_FALSE`, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetFragmentShadingRateKHR

If the `geometryStreams` feature is enabled, and a shader is bound to the `VK_SHADER_STAGE_GEOMETRY_BIT` stage, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetRasterizationStreamEXT

If the VK_EXT_discard_rectangles extension is enabled, and rasterizerDiscardEnable is VK_FALSE, the following commands **must** have been called in the command buffer prior to drawing:

- vkCmdSetDiscardRectangleEnableEXT
- vkCmdSetDiscardRectangleModeEXT, if discardRectangleEnable is VK_TRUE
- vkCmdSetDiscardRectangleEXT, if discardRectangleEnable is VK_TRUE

If the VK_EXT_conservative_rasterization extension is enabled, and rasterizerDiscardEnable is VK_FALSE, the following commands **must** have been called in the command buffer prior to drawing:

- vkCmdSetConservativeRasterizationModeEXT
- vkCmdSetExtraPrimitiveOverestimationSizeEXT, if conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT

If the depthClipEnable feature is enabled, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetDepthClipEnableEXT

If the VK_EXT_sample_locations extension is enabled, and rasterizerDiscardEnable is VK_FALSE, the following commands **must** have been called in the command buffer prior to drawing:

- vkCmdSetSampleLocationsEnableEXT
- vkCmdSetSampleLocationsEXT, if sampleLocationsEnable is VK_TRUE

If the VK_EXT_provoking_vertex extension is enabled, and rasterizerDiscardEnable is VK_FALSE, and a shader is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetProvokingVertexModeEXT

If any of the <features-stippledRectangularLines, stippledRectangularLines>>, <features-stippledBresenhamLines, stippledBresenhamLines>>, or <features-stippledSmoothLines, stippledSmoothLines>> features are enabled, and rasterizerDiscardEnable is VK_FALSE, and if polygonMode is VK_POLYGON_MODE_LINE or a shader is bound to the VK_SHADER_STAGE_VERTEX_BIT stage and primitiveTopology is a line topology or a shader which outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, the following commands **must** have been called in the command buffer prior to drawing:

- vkCmdSetLineRasterizationModeEXT
- vkCmdSetLineStippleEnableEXT
- vkCmdSetLineStippleKHR, if stippledLineEnable is VK_TRUE

If the depthClipControl feature is enabled, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetDepthClipNegativeOneToOneEXT

If the colorWriteEnable feature is enabled, and a shader is bound to the

VK_SHADER_STAGE_FRAGMENT_BIT stage, and rasterizerDiscardEnable is VK_FALSE, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetColorWriteEnableEXT, with values set for every color attachment in the render pass instance active at draw time

If the attachmentFeedbackLoopDynamicState feature is enabled, and a shader is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and rasterizerDiscardEnable is VK_FALSE, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetAttachmentFeedbackLoopEnableEXT

If the depthClamp and depthClampControl features are enabled, and depthClampEnable is VK_TRUE, the following command **must** have been called in the command buffer prior to drawing:

- vkCmdSetDepthClampRangeEXT

State **can** be set either at any time before or after shader objects are bound, but all required state **must** be set prior to issuing drawing commands.

9.1.6. Interaction With Pipelines

Calling vkCmdBindShadersEXT causes the pipeline bind points corresponding to each stage in pStages to be disturbed, meaning that any pipelines that had previously been bound to those pipeline bind points are no longer bound.

If VK_PIPELINE_BIND_POINT_GRAPHICS is disturbed (i.e., if pStages contains any graphics stage), any graphics pipeline state that the previously bound pipeline did not specify as dynamic becomes undefined, and **must** be set in the command buffer before issuing drawing commands using shader objects.

Calls to vkCmdBindPipeline likewise disturb the shader stage(s) corresponding to pipelineBindPoint, meaning that any shaders that had previously been bound to any of those stages are no longer bound, even if the pipeline was created without shaders for some of those stages.

9.1.7. Shader Object Destruction

To destroy a shader object, call:

```
// Provided by VK_EXT_shader_object
void vkDestroyShaderEXT(
    VkDevice                                    device,
    VkShaderEXT                                 shader,
    const VkAllocationCallbacks*                pAllocator);
```

- device is the logical device that destroys the shader object.
- shader is the handle of the shader object to destroy.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.

Destroying a shader object used by one or more command buffers in the recording or executable state causes those command buffers to move into the *invalid state*.

Valid Usage

- VUID-vkDestroyShaderEXT-None-08481
  The `shaderObject` feature **must** be enabled
- VUID-vkDestroyShaderEXT-shader-08482
  All submitted commands that refer to `shader` **must** have completed execution
- VUID-vkDestroyShaderEXT-pAllocator-08483
  If `VkAllocationCallbacks` were provided when `shader` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyShaderEXT-pAllocator-08484
  If no `VkAllocationCallbacks` were provided when `shader` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyShaderEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyShaderEXT-shader-parameter
  If `shader` is not VK_NULL_HANDLE, `shader` **must** be a valid VkShaderEXT handle
- VUID-vkDestroyShaderEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyShaderEXT-shader-parent
  If `shader` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `shader` **must** be externally synchronized

9.2. Shader Modules

Shader modules contain *shader code* and one or more entry points. Shaders are selected from a shader module by specifying an entry point as part of pipeline creation. The stages of a pipeline **can** use shaders that come from different modules. The shader code defining a shader module **must** be in the SPIR-V format, as described by the Vulkan Environment for SPIR-V appendix.

Shader modules are represented by `VkShaderModule` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkShaderModule)
```

To create a shader module, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateShaderModule(
    VkDevice                                    device,
    const VkShaderModuleCreateInfo*             pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkShaderModule*                             pShaderModule);
```

- `device` is the logical device that creates the shader module.
- `pCreateInfo` is a pointer to a VkShaderModuleCreateInfo structure.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pShaderModule` is a pointer to a VkShaderModule handle in which the resulting shader module object is returned.

Once a shader module has been created, any entry points it contains **can** be used in pipeline shader stages as described in Compute Pipelines and Graphics Pipelines.

NOTE If the `maintenance5` feature is enabled, shader module creation can be omitted entirely. Instead, applications should provide the VkShaderModuleCreateInfo structure directly in to pipeline creation by chaining it to VkPipelineShaderStageCreateInfo. This avoids the overhead of creating and managing an additional object.

Valid Usage

- VUID-vkCreateShaderModule-pCreateInfo-06904
  If `pCreateInfo` is not `NULL`, `pCreateInfo->pNext` **must** be `NULL` or a pointer to a valid instance of
  - VkShaderModuleValidationCacheCreateInfoEXT

Valid Usage (Implicit)

- VUID-vkCreateShaderModule-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateShaderModule-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkShaderModuleCreateInfo structure
- VUID-vkCreateShaderModule-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid

VkAllocationCallbacks structure

- VUID-vkCreateShaderModule-pShaderModule-parameter
  `pShaderModule` **must** be a valid pointer to a VkShaderModule handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkShaderModuleCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkShaderModuleCreateInfo {
    VkStructureType              sType;
    const void*                  pNext;
    VkShaderModuleCreateFlags    flags;
    size_t                       codeSize;
    const uint32_t*              pCode;
} VkShaderModuleCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `codeSize` is the size, in bytes, of the code pointed to by `pCode`.
- `pCode` is a pointer to code that is used to create the shader module. The type and format of the code is determined from the content of the memory addressed by `pCode`.

Valid Usage

- VUID-VkShaderModuleCreateInfo-codeSize-08735
  `codeSize` **must** be a multiple of 4
- VUID-VkShaderModuleCreateInfo-pCode-08736
  `pCode` **must** point to valid SPIR-V code, formatted and packed as described by the Khronos SPIR-V Specification
- VUID-VkShaderModuleCreateInfo-pCode-08737
  `pCode` **must** adhere to the validation rules described by the Validation Rules within a Module section of the SPIR-V Environment appendix
- VUID-VkShaderModuleCreateInfo-pCode-08738

`pCode` **must** declare the `Shader` capability for SPIR-V code

- VUID-VkShaderModuleCreateInfo-pCode-08739
  `pCode` **must** not declare any capability that is not supported by the API, as described by the Capabilities section of the SPIR-V Environment appendix
- VUID-VkShaderModuleCreateInfo-pCode-08740
  and `pCode` declares any of the capabilities listed in the SPIR-V Environment appendix, one of the corresponding requirements **must** be satisfied
- VUID-VkShaderModuleCreateInfo-pCode-08741
  `pCode` **must** not declare any SPIR-V extension that is not supported by the API, as described by the Extension section of the SPIR-V Environment appendix
- VUID-VkShaderModuleCreateInfo-pCode-08742
  and `pCode` declares any of the SPIR-V extensions listed in the SPIR-V Environment appendix, one of the corresponding requirements **must** be satisfied
- VUID-VkShaderModuleCreateInfo-codeSize-01085
  `codeSize` **must** be greater than 0

Valid Usage (Implicit)

- VUID-VkShaderModuleCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO`
- VUID-VkShaderModuleCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkShaderModuleCreateInfo-pCode-parameter
  `pCode` **must** be a valid pointer to an array of $\frac{\text{codeSize}}{4}$ `uint32_t` values

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkShaderModuleCreateFlags;
```

`VkShaderModuleCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

To use a VkValidationCacheEXT to cache shader validation results, add a VkShaderModuleValidationCacheCreateInfoEXT structure to the `pNext` chain of the VkShaderModuleCreateInfo structure, specifying the cache object to use.

The `VkShaderModuleValidationCacheCreateInfoEXT` struct is defined as:

```
// Provided by VK_EXT_validation_cache
typedef struct VkShaderModuleValidationCacheCreateInfoEXT {
    VkStructureType         sType;
    const void*             pNext;
    VkValidationCacheEXT    validationCache;
} VkShaderModuleValidationCacheCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `validationCache` is the validation cache object from which the results of prior validation attempts will be written, and to which new validation results for this VkShaderModule will be written (if not already present).

Valid Usage (Implicit)

- VUID-VkShaderModuleValidationCacheCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SHADER_MODULE_VALIDATION_CACHE_CREATE_INFO_EXT`
- VUID-VkShaderModuleValidationCacheCreateInfoEXT-validationCache-parameter
  `validationCache` **must** be a valid VkValidationCacheEXT handle

To destroy a shader module, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyShaderModule(
    VkDevice                                    device,
    VkShaderModule                              shaderModule,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the shader module.
- `shaderModule` is the handle of the shader module to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

A shader module **can** be destroyed while pipelines created using its shaders are still in use.

Valid Usage

- VUID-vkDestroyShaderModule-shaderModule-01092
  If `VkAllocationCallbacks` were provided when `shaderModule` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyShaderModule-shaderModule-01093
  If no `VkAllocationCallbacks` were provided when `shaderModule` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyShaderModule-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyShaderModule-shaderModule-parameter
  If `shaderModule` is not VK_NULL_HANDLE, `shaderModule` **must** be a valid VkShaderModule handle
- VUID-vkDestroyShaderModule-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyShaderModule-shaderModule-parent
  If `shaderModule` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `shaderModule` **must** be externally synchronized

9.3. Shader Module Identifiers

Shader modules have unique identifiers associated with them. To query an implementation provided identifier, call:

```
// Provided by VK_EXT_shader_module_identifier
void vkGetShaderModuleIdentifierEXT(
    VkDevice                                    device,
    VkShaderModule                              shaderModule,
    VkShaderModuleIdentifierEXT*                pIdentifier);
```

- `device` is the logical device that created the shader module.
- `shaderModule` is the handle of the shader module.
- `pIdentifier` is a pointer to the returned VkShaderModuleIdentifierEXT.

The identifier returned by the implementation **must** only depend on `shaderIdentifierAlgorithmUUID` and information provided in the VkShaderModuleCreateInfo which created `shaderModule`. The implementation **may** return equal identifiers for two different VkShaderModuleCreateInfo structures if the difference does not affect pipeline compilation. Identifiers are only meaningful on different VkDevice objects if the device the identifier was queried from had the same `shaderModuleIdentifierAlgorithmUUID` as the device consuming the identifier.

Valid Usage

- VUID-vkGetShaderModuleIdentifierEXT-shaderModuleIdentifier-06884
  `shaderModuleIdentifier` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkGetShaderModuleIdentifierEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetShaderModuleIdentifierEXT-shaderModule-parameter
  `shaderModule` **must** be a valid VkShaderModule handle
- VUID-vkGetShaderModuleIdentifierEXT-pIdentifier-parameter
  `pIdentifier` **must** be a valid pointer to a VkShaderModuleIdentifierEXT structure
- VUID-vkGetShaderModuleIdentifierEXT-shaderModule-parent
  `shaderModule` **must** have been created, allocated, or retrieved from `device`

VkShaderModuleCreateInfo structures have unique identifiers associated with them. To query an implementation provided identifier, call:

```
// Provided by VK_EXT_shader_module_identifier
void vkGetShaderModuleCreateInfoIdentifierEXT(
    VkDevice                                    device,
    const VkShaderModuleCreateInfo*             pCreateInfo,
    VkShaderModuleIdentifierEXT*                pIdentifier);
```

- `device` is the logical device that **can** create a VkShaderModule from `pCreateInfo`.
- `pCreateInfo` is a pointer to a VkShaderModuleCreateInfo structure.
- `pIdentifier` is a pointer to the returned VkShaderModuleIdentifierEXT.

The identifier returned by implementation **must** only depend on `shaderIdentifierAlgorithmUUID` and information provided in the VkShaderModuleCreateInfo. The implementation **may** return equal identifiers for two different VkShaderModuleCreateInfo structures if the difference does not affect pipeline compilation. Identifiers are only meaningful on different VkDevice objects if the device the identifier was queried from had the same `shaderModuleIdentifierAlgorithmUUID` as the device consuming the identifier.

The identifier returned by the implementation in vkGetShaderModuleCreateInfoIdentifierEXT **must** be equal to the identifier returned by vkGetShaderModuleIdentifierEXT given equivalent definitions of VkShaderModuleCreateInfo and any chained `pNext` structures.

Valid Usage

- VUID-vkGetShaderModuleCreateInfoIdentifierEXT-shaderModuleIdentifier-06885

`shaderModuleIdentifier` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkGetShaderModuleCreateInfoIdentifierEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetShaderModuleCreateInfoIdentifierEXT-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkShaderModuleCreateInfo structure
- VUID-vkGetShaderModuleCreateInfoIdentifierEXT-pIdentifier-parameter
  `pIdentifier` **must** be a valid pointer to a VkShaderModuleIdentifierEXT structure

VkShaderModuleIdentifierEXT represents a shader module identifier returned by the implementation.

```
// Provided by VK_EXT_shader_module_identifier
typedef struct VkShaderModuleIdentifierEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           identifierSize;
    uint8_t            identifier[VK_MAX_SHADER_MODULE_IDENTIFIER_SIZE_EXT];
} VkShaderModuleIdentifierEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `identifierSize` is the size, in bytes, of valid data returned in `identifier`.
- `identifier` is a buffer of opaque data specifying an identifier.

Any returned values beyond the first `identifierSize` bytes are undefined. Implementations **must** return an `identifierSize` greater than 0, and less-or-equal to `VK_MAX_SHADER_MODULE_IDENTIFIER_SIZE_EXT`.

Two identifiers are considered equal if `identifierSize` is equal and the first `identifierSize` bytes of `identifier` compare equal.

Implementations **may** return a different `identifierSize` for different modules. Implementations **should** ensure that `identifierSize` is large enough to uniquely define a shader module.

Valid Usage (Implicit)

- VUID-VkShaderModuleIdentifierEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SHADER_MODULE_IDENTIFIER_EXT`
- VUID-VkShaderModuleIdentifierEXT-pNext-pNext
  `pNext` **must** be `NULL`

VK_MAX_SHADER_MODULE_IDENTIFIER_SIZE_EXT is the length in bytes of a shader module identifier, as returned in VkShaderModuleIdentifierEXT::identifierSize.

```
#define VK_MAX_SHADER_MODULE_IDENTIFIER_SIZE_EXT 32U
```

9.4. Binding Shaders

Before a shader can be used it **must** be first bound to the command buffer.

Calling vkCmdBindPipeline binds all stages corresponding to the VkPipelineBindPoint. Calling vkCmdBindShadersEXT binds all stages in pStages

The following table describes the relationship between shader stages and pipeline bind points:

Shader stage	Pipeline bind point	behavior controlled
• VK_SHADER_STAGE_VERTEX_BIT • VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT • VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT • VK_SHADER_STAGE_GEOMETRY_BIT • VK_SHADER_STAGE_FRAGMENT_BIT	VK_PIPELINE_BIND_POINT_GRAPHICS	all drawing commands
• VK_SHADER_STAGE_COMPUTE_BIT	VK_PIPELINE_BIND_POINT_COMPUTE	all dispatch commands
• VK_SHADER_STAGE_ANY_HIT_BIT_KHR • VK_SHADER_STAGE_CALLABLE_BIT_KHR • VK_SHADER_STAGE_CLOSEST_HIT_BIT_KHR • VK_SHADER_STAGE_INTERSECTION_BIT_KHR • VK_SHADER_STAGE_MISS_BIT_KHR • VK_SHADER_STAGE_RAYGEN_BIT_KHR	VK_PIPELINE_BIND_POINT_RAY_TRACING_KHR	vkCmdTraceRaysKHR and vkCmdTraceRaysIndirectKHR

9.5. Shader Execution

At each stage of the pipeline, multiple invocations of a shader **may** execute simultaneously. Further, invocations of a single shader produced as the result of different commands **may** execute

simultaneously. The relative execution order of invocations of the same shader type is undefined. Shader invocations **may** complete in a different order than that in which the primitives they originated from were drawn or dispatched by the application. However, fragment shader outputs are written to attachments in rasterization order.

The relative execution order of invocations of different shader types is largely undefined. However, when invoking a shader whose inputs are generated from a previous pipeline stage, the shader invocations from the previous stage are guaranteed to have executed far enough to generate input values for all required inputs.

9.5.1. Shader Termination

A shader invocation that is *terminated* has finished executing instructions.

Executing `OpReturn` in the entry point, or executing `OpTerminateInvocation` in any function will terminate an invocation. Implementations **may** also terminate a shader invocation when `OpKill` is executed in any function; otherwise it becomes a helper invocation.

In addition to the above conditions, helper invocations **may** be terminated when all non-helper invocations in the same derivative group either terminate or become helper invocations.

A shader stage for a given command completes execution when all invocations for that stage have terminated.

NOTE	Depending on the implementation, `OpKill` will be functionally equivalent to either `OpTerminateInvocation` or `OpDemoteToHelperInvocation`. To obtain the most predictable behavior, shader authors should use `OpTerminateInvocation` or `OpDemoteToHelperInvocation` rather than `OpKill` wherever possible.

9.6. Shader Memory Access Ordering

The order in which image or buffer memory is read or written by shaders is largely undefined. For some shader types (vertex, tessellation evaluation, and in some cases, fragment), even the number of shader invocations that **may** perform loads and stores is undefined.

In particular, the following rules apply:

- Vertex and tessellation evaluation shaders will be invoked at least once for each unique vertex, as defined in those sections.
- Fragment shaders will be invoked zero or more times, as defined in that section.
- The relative execution order of invocations of the same shader type is undefined. A store issued by a shader when working on primitive B might complete prior to a store for primitive A, even if primitive A is specified prior to primitive B. This applies even to fragment shaders; while fragment shader outputs are always written to the framebuffer in rasterization order, stores executed by fragment shader invocations are not.
- The relative execution order of invocations of different shader types is largely undefined.

NOTE	The above limitations on shader invocation order make some forms of synchronization between shader invocations within a single set of primitives unimplementable. For example, having one invocation poll memory written by another invocation assumes that the other invocation has been launched and will complete its writes in finite time.

The Memory Model appendix defines the terminology and rules for how to correctly communicate between shader invocations, such as when a write is Visible-To a read, and what constitutes a Data Race.

Applications **must** not cause a data race.

The SPIR-V **SubgroupMemory**, **CrossWorkgroupMemory**, and **AtomicCounterMemory** memory semantics are ignored. Sequentially consistent atomics and barriers are not supported and **SequentiallyConsistent** is treated as **AcquireRelease**. **SequentiallyConsistent should** not be used.

9.7. Shader Inputs and Outputs

Data is passed into and out of shaders using variables with input or output storage class, respectively. User-defined inputs and outputs are connected between stages by matching their `Location` decorations. Additionally, data **can** be provided by or communicated to special functions provided by the execution environment using `BuiltIn` decorations.

In many cases, the same `BuiltIn` decoration **can** be used in multiple shader stages with similar meaning. The specific behavior of variables decorated as `BuiltIn` is documented in the following sections.

9.8. Vertex Shaders

Each vertex shader invocation operates on one vertex and its associated vertex attribute data, and outputs one vertex and associated data. Graphics pipelines **must** include a vertex shader, and the vertex shader stage is always the first shader stage in the graphics pipeline.

9.8.1. Vertex Shader Execution

A vertex shader **must** be executed at least once for each vertex specified by a drawing command. If the subpass includes multiple views in its view mask, the shader **may** be invoked separately for each view. During execution, the shader is presented with the index of the vertex and instance for which it has been invoked. Input variables declared in the vertex shader are filled by the implementation with the values of vertex attributes associated with the invocation being executed.

If the same vertex is specified multiple times in a drawing command (e.g. by including the same index value multiple times in an index buffer) the implementation **may** reuse the results of vertex shading if it can statically determine that the vertex shader invocations will produce identical results.

NOTE	It is implementation-dependent when and if results of vertex shading are reused,

> and thus how many times the vertex shader will be executed. This is true also if the vertex shader contains stores or atomic operations (see vertexPipelineStoresAndAtomics).

9.9. Tessellation Control Shaders

The tessellation control shader is used to read an input patch provided by the application and to produce an output patch. Each tessellation control shader invocation operates on an input patch (after all control points in the patch are processed by a vertex shader) and its associated data, and outputs a single control point of the output patch and its associated data, and **can** also output additional per-patch data. The input patch is sized according to the patchControlPoints member of VkPipelineTessellationStateCreateInfo, as part of input assembly.

The input patch can also be dynamically sized with patchControlPoints parameter of vkCmdSetPatchControlPointsEXT.

To dynamically set the number of control points per patch, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetPatchControlPointsEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    patchControlPoints);
```

- commandBuffer is the command buffer into which the command will be recorded.
- patchControlPoints specifies the number of control points per patch.

This command sets the number of control points per patch for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineTessellationStateCreateInfo::patchControlPoints value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetPatchControlPointsEXT-None-09422
  At least one of the following **must** be true:
  - The shaderObject feature is enabled
- VUID-vkCmdSetPatchControlPointsEXT-patchControlPoints-04874
  patchControlPoints **must** be greater than zero and less than or equal to VkPhysicalDeviceLimits::maxTessellationPatchSize

Valid Usage (Implicit)

- VUID-vkCmdSetPatchControlPointsEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle

- VUID-vkCmdSetPatchControlPointsEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetPatchControlPointsEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetPatchControlPointsEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

The size of the output patch is controlled by the `OpExecutionMode OutputVertices` specified in the tessellation control or tessellation evaluation shaders, which **must** be specified in at least one of the shaders. The size of the input and output patches **must** each be greater than zero and less than or equal to `VkPhysicalDeviceLimits`::`maxTessellationPatchSize`.

9.9.1. Tessellation Control Shader Execution

A tessellation control shader is invoked at least once for each *output* vertex in a patch. If the subpass includes multiple views in its view mask, the shader **may** be invoked separately for each view.

Inputs to the tessellation control shader are generated by the vertex shader. Each invocation of the tessellation control shader **can** read the attributes of any incoming vertices and their associated data. The invocations corresponding to a given patch execute logically in parallel, with undefined relative execution order. However, the `OpControlBarrier` instruction **can** be used to provide limited control of the execution order by synchronizing invocations within a patch, effectively dividing tessellation control shader execution into a set of phases. Tessellation control shaders will read undefined values if one invocation reads a per-vertex or per-patch output written by another invocation at any point during the same phase, or if two invocations attempt to write different values to the same per-patch output in a single phase.

9.10. Tessellation Evaluation Shaders

The Tessellation Evaluation Shader operates on an input patch of control points and their associated data, and a single input barycentric coordinate indicating the invocation's relative position within the subdivided patch, and outputs a single vertex and its associated data.

9.10.1. Tessellation Evaluation Shader Execution

A tessellation evaluation shader is invoked at least once for each unique vertex generated by the tessellator. If the subpass includes multiple views in its view mask, the shader **may** be invoked separately for each view.

9.11. Geometry Shaders

The geometry shader operates on a group of vertices and their associated data assembled from a single input primitive, and emits zero or more output primitives and the group of vertices and their associated data required for each output primitive.

9.11.1. Geometry Shader Execution

A geometry shader is invoked at least once for each primitive produced by the tessellation stages, or at least once for each primitive generated by primitive assembly when tessellation is not in use. A shader can request that the geometry shader runs multiple instances. A geometry shader is invoked at least once for each instance. If the subpass includes multiple views in its view mask, the shader **may** be invoked separately for each view.

9.12. Fragment Shaders

Fragment shaders are invoked as a fragment operation in a graphics pipeline. Each fragment shader invocation operates on a single fragment and its associated data. With few exceptions, fragment shaders do not have access to any data associated with other fragments and are considered to execute in isolation of fragment shader invocations associated with other fragments.

9.13. Compute Shaders

Compute shaders are invoked via _dispatching commands. In general, they have access to similar resources as shader stages executing as part of a graphics pipeline.

Compute workloads are formed from groups of work items called workgroups and processed by the compute shader in the current compute pipeline. A workgroup is a collection of shader invocations that execute the same shader, potentially in parallel. Compute shaders execute in *global workgroups* which are divided into a number of *local workgroups* with a size that **can** be set by assigning a value to the `LocalSize` or `LocalSizeId` execution mode or via an object decorated by the `WorkgroupSize` decoration. An invocation within a local workgroup **can** share data with other members of the local workgroup through shared variables and issue memory and control flow barriers to synchronize with other members of the local workgroup.

9.14. Ray Generation Shaders

A ray generation shader is similar to a compute shader. Its main purpose is to execute ray tracing queries using pipeline trace ray instructions (such as `OpTraceRayKHR`) and process the results.

9.14.1. Ray Generation Shader Execution

One ray generation shader is executed per ray tracing dispatch. Its location in the shader binding table (see Shader Binding Table for details) is passed directly into vkCmdTraceRaysKHR using the `pRaygenShaderBindingTable` parameter or .

9.15. Intersection Shaders

Intersection shaders enable the implementation of arbitrary, application defined geometric primitives. An intersection shader for a primitive is executed whenever its axis-aligned bounding box is hit by a ray.

Like other ray tracing shader domains, an intersection shader operates on a single ray at a time. It also operates on a single primitive at a time. It is therefore the purpose of an intersection shader to compute the ray-primitive intersections and report them. To report an intersection, the shader calls the `OpReportIntersectionKHR` instruction.

An intersection shader communicates with any-hit and closest shaders by generating attribute values that they **can** read. Intersection shaders **cannot** read or modify the ray payload.

9.15.1. Intersection Shader Execution

The order in which intersections are found along a ray, and therefore the order in which intersection shaders are executed, is unspecified.

The intersection shader of the closest AABB which intersects the ray is guaranteed to be executed at some point during traversal, unless the ray is forcibly terminated.

9.16. Any-Hit Shaders

The any-hit shader is executed after the intersection shader reports an intersection that lies within the current $[t_{min},t_{max}]$ of the ray. The main use of any-hit shaders is to programmatically decide whether or not an intersection will be accepted. The intersection will be accepted unless the shader calls the `OpIgnoreIntersectionKHR` instruction. Any-hit shaders have read-only access to the attributes generated by the corresponding intersection shader, and **can** read or modify the ray payload.

9.16.1. Any-Hit Shader Execution

The order in which intersections are found along a ray, and therefore the order in which any-hit shaders are executed, is unspecified.

The any-hit shader of the closest hit is guaranteed to be executed at some point during traversal,

unless the ray is forcibly terminated.

9.17. Closest Hit Shaders

Closest hit shaders have read-only access to the attributes generated by the corresponding intersection shader, and **can** read or modify the ray payload. They also have access to a number of system-generated values. Closest hit shaders **can** call pipeline trace ray instructions to recursively trace rays.

9.17.1. Closest Hit Shader Execution

Exactly one closest hit shader is executed when traversal is finished and an intersection has been found and accepted.

9.18. Miss Shaders

Miss shaders **can** access the ray payload and **can** trace new rays through the pipeline trace ray instructions, but **cannot** access attributes since they are not associated with an intersection.

9.18.1. Miss Shader Execution

A miss shader is executed instead of a closest hit shader if no intersection was found during traversal.

9.19. Callable Shaders

Callable shaders **can** access a callable payload that works similarly to ray payloads to do subroutine work.

9.19.1. Callable Shader Execution

A callable shader is executed by calling `OpExecuteCallableKHR` from an allowed shader stage.

9.20. Interpolation Decorations

Variables in the `Input` storage class in a fragment shader's interface are interpolated from the values specified by the primitive being rasterized.

NOTE	Interpolation decorations can be present on input and output variables in pre-rasterization shaders but have no effect on the interpolation performed.

An undecorated input variable will be interpolated with perspective-correct interpolation according to the primitive type being rasterized. Lines and polygons are interpolated in the same way as the primitive's clip coordinates. If the `NoPerspective` decoration is present, linear interpolation is instead used for lines and polygons. For points, as there is only a single vertex, input values are never interpolated and instead take the value written for the single vertex.

If the `Flat` decoration is present on an input variable, the value is not interpolated, and instead takes its value directly from the provoking vertex. Fragment shader inputs that are signed or unsigned integers, integer vectors, or any double-precision floating-point type **must** be decorated with `Flat`.

Interpolation of input variables is performed at an implementation-defined position within the fragment area being shaded. The position is further constrained as follows:

- If the `Centroid` decoration is used, the interpolation position used for the variable **must** also fall within the bounds of the primitive being rasterized.
- If the `Sample` decoration is used, the interpolation position used for the variable **must** be at the position of the sample being shaded by the current fragment shader invocation.
- If a sample count of 1 is used, the interpolation position **must** be at the center of the fragment area.

NOTE

As `Centroid` constrains the interpolation position to lie within the covered area of the primitive, using it may cause the position to differ between neighboring fragments when it otherwise would not. Derivatives calculated based on these differing locations can produce inconsistent results compared to undecorated inputs. Thus using `Centroid` with input variables used in derivative calculations is not recommended.

If the `PerVertexKHR` decoration is present on an input variable, the value is not interpolated, and instead values from all input vertices are available in an array. Each index of the array corresponds to one of the vertices of the primitive that produced the fragment.

9.21. Static Use

A SPIR-V module declares a global object in memory using the `OpVariable` instruction, which results in a pointer `x` to that object. A specific entry point in a SPIR-V module is said to *statically use* that object if that entry point's call tree contains a function containing a instruction with `x` as an `id` operand. A shader entry point also *statically uses* any variables explicitly declared in its interface.

9.22. Scope

A *scope* describes a set of shader invocations, where each such set is a *scope instance*. Each invocation belongs to one or more scope instances, but belongs to no more than one scope instance for each scope.

The operations available between invocations in a given scope instance vary, with smaller scopes generally able to perform more operations, and with greater efficiency.

9.22.1. Cross Device

All invocations executed in a Vulkan instance fall into a single *cross device scope instance*.

Whilst the `CrossDevice` scope is defined in SPIR-V, it is disallowed in Vulkan. API synchronization

commands **can** be used to communicate between devices.

9.22.2. Device

All invocations executed on a single device form a *device scope instance*.

If the `vulkanMemoryModel` and `vulkanMemoryModelDeviceScope` features are enabled, this scope is represented in SPIR-V by the `Device Scope`, which **can** be used as a `Memory Scope` for barrier and atomic operations.

If both the `shaderDeviceClock` and `vulkanMemoryModelDeviceScope` features are enabled, using the `Device Scope` with the `OpReadClockKHR` instruction will read from a clock that is consistent across invocations in the same device scope instance.

There is no method to synchronize the execution of these invocations within SPIR-V, and this **can** only be done with API synchronization primitives.

Invocations executing on different devices in a device group operate in separate device scope instances.

9.22.3. Queue Family

Invocations executed by queues in a given queue family form a *queue family scope instance*.

This scope is identified in SPIR-V as the `QueueFamily Scope` if the `vulkanMemoryModel` feature is enabled, or if not, the `Device Scope`, which **can** be used as a `Memory Scope` for barrier and atomic operations.

If the `shaderDeviceClock` feature is enabled, but the `vulkanMemoryModelDeviceScope` feature is not enabled, using the `Device Scope` with the `OpReadClockKHR` instruction will read from a clock that is consistent across invocations in the same queue family scope instance.

There is no method to synchronize the execution of these invocations within SPIR-V, and this **can** only be done with API synchronization primitives.

Each invocation in a queue family scope instance **must** be in the same device scope instance.

9.22.4. Command

Any shader invocations executed as the result of a single command such as vkCmdDispatch or vkCmdDraw form a *command scope instance*. For indirect drawing commands with `drawCount` greater than one, invocations from separate draws are in separate command scope instances. For ray tracing shaders, an invocation group is an implementation-dependent subset of the set of shader invocations of a given shader stage which are produced by a single trace rays command.

There is no specific `Scope` for communication across invocations in a command scope instance. As this has a clear boundary at the API level, coordination here **can** be performed in the API, rather than in SPIR-V.

Each invocation in a command scope instance **must** be in the same queue-family scope instance.

For shaders without defined workgroups, this set of invocations forms an *invocation group* as defined in the SPIR-V specification.

9.22.5. Primitive

Any fragment shader invocations executed as the result of rasterization of a single primitive form a *primitive scope instance*.

There is no specific `Scope` for communication across invocations in a primitive scope instance.

Any generated helper invocations are included in this scope instance.

Each invocation in a primitive scope instance **must** be in the same command scope instance.

Any input variables decorated with `Flat` are uniform within a primitive scope instance.

9.22.6. Shader Call

Any shader-call-related invocations that are executed in one or more ray tracing execution models form a *shader call scope instance*.

The `ShaderCallKHR Scope` can be used as `Memory Scope` for barrier and atomic operations.

Each invocation in a shader call scope instance **must** be in the same queue family scope instance.

9.22.7. Workgroup

A *local workgroup* is a set of invocations that can synchronize and share data with each other using memory in the `Workgroup` storage class.

The `Workgroup Scope` can be used as both an `Execution Scope` and `Memory Scope` for barrier and atomic operations.

Each invocation in a local workgroup **must** be in the same command scope instance.

Only compute shaders have defined workgroups - other shader types **cannot** use workgroup functionality. For shaders that have defined workgroups, this set of invocations forms an *invocation group* as defined in the SPIR-V specification.

When variables declared with the `Workgroup` storage class are explicitly laid out (hence they are also decorated with `Block`), the amount of storage consumed is the size of the largest Block variable, not counting any padding at the end. The amount of storage consumed by the non-Block variables declared with the `Workgroup` storage class is implementation-dependent. However, the amount of storage consumed may not exceed the largest block size that would be obtained if all active non-Block variables declared with `Workgroup` storage class were assigned offsets in an arbitrary order by successively taking the smallest valid offset according to the Standard Storage Buffer Layout rules, and with `Boolean` values considered as 32-bit integer values for the purpose of this calculation. (This is equivalent to using the GLSL std430 layout rules.)

9.22.8. Quad

A *quad scope instance* is formed of four shader invocations.

In a fragment shader, each invocation in a quad scope instance is formed of invocations in neighboring framebuffer locations (x_i, y_i), where:

- i is the index of the invocation within the scope instance.
- w and h are the number of pixels the fragment covers in the x and y axes.
- w and h are identical for all participating invocations.
- (x_0) = (x_1 - w) = (x_2) = (x_3 - w)
- (y_0) = (y_1) = (y_2 - h) = (y_3 - h)
- Each invocation has the same layer and sample indices.

In a mesh, task, or compute shader, if the `DerivativeGroupQuadsKHR` execution mode is specified, each invocation in a quad scope instance is formed of invocations with adjacent local invocation IDs (x_i, y_i), where:

- i is the index of the invocation within the quad scope instance.
- (x_0) = (x_1 - 1) = (x_2) = (x_3 - 1)
- (y_0) = (y_1) = (y_2 - 1) = (y_3 - 1)
- x_0 and y_0 are integer multiples of 2.
- Each invocation has the same z coordinate.

In a mesh, task, or compute shader, if the `DerivativeGroupLinearKHR` execution mode is specified, each invocation in a quad scope instance is formed of invocations with adjacent local invocation indices (l_i), where:

- i is the index of the invocation within the quad scope instance.
- (l_0) = (l_1 - 1) = (l_2 - 2) = (l_3 - 3)
- l_0 is an integer multiple of 4.

The specific set of invocations that make up a quad scope instance in other shader stages is undefined.

In a fragment shader, each invocation in a quad scope instance **must** be in the same primitive scope instance.

For shaders that have defined workgroups, each invocation in a quad scope instance **must** be in the same local workgroup.

In other shader stages, each invocation in a quad scope instance **must** be in the same device scope instance.

Fragment , mesh, task, and compute shaders have defined quad scope instances.

9.22.9. Fragment Interlock

A *fragment interlock scope instance* is formed of fragment shader invocations based on their framebuffer locations (x,y,layer,sample), executed by commands inside a single subpass.

The specific set of invocations included varies based on the execution mode as follows:

- If the `SampleInterlockOrderedEXT` or `SampleInterlockUnorderedEXT` execution modes are used, only invocations with identical framebuffer locations (x,y,layer,sample) are included.
- If the `PixelInterlockOrderedEXT` or `PixelInterlockUnorderedEXT` execution modes are used, fragments with different sample ids are also included.
- If the `ShadingRateInterlockOrderedEXT` or `ShadingRateInterlockUnorderedEXT` execution modes are used, fragments from neighboring framebuffer locations are also included. The fragment shading rate determines these fragments.

Only fragment shaders with one of the above execution modes have defined fragment interlock scope instances.

There is no specific `Scope` value for communication across invocations in a fragment interlock scope instance. However, this is implicitly used as a memory scope by `OpBeginInvocationInterlockEXT` and `OpEndInvocationInterlockEXT`.

Each invocation in a fragment interlock scope instance **must** be in the same queue family scope instance.

9.22.10. Invocation

The smallest *scope* is a single invocation; this is represented by the `Invocation Scope` in SPIR-V.

Fragment shader invocations **must** be in a primitive scope instance.

Invocations in fragment shaders that have a defined fragment interlock scope **must** be in a fragment interlock scope instance.

Invocations in shaders that have defined workgroups **must** be in a local workgroup.

Invocations in shaders that have a defined quad scope **must** be in a quad scope instance.

All invocations in all stages **must** be in a command scope instance.

9.23. Derivative Operations

Derivative operations calculate the partial derivative for an expression P as a function of an invocation's x and y coordinates.

Derivative operations operate on a set of invocations known as a *derivative group* as defined in the SPIR-V specification.

A derivative group in a fragment shader is equivalent to the quad scope instance if the `QuadDerivativesKHR` execution mode is specified, otherwise it is equivalent to the primitive scope

instance. A derivative group in a mesh, task, or compute shader is equivalent to the quad scope instance.

Derivatives are calculated assuming that P is piecewise linear and continuous within the derivative group.

The following control-flow restrictions apply to derivative operations:

- If the `QuadDerivativesKHR` execution mode is specified, dynamic instances of any derivative operations **must** be executed in control flow that is uniform within the current quad scope instance.
- If the `QuadDerivativesKHR` execution mode is not specified:
  - dynamic instances of explicit derivative instructions (`OpDPdx*`, `OpDPdy*`, and `OpFwidth*`) **must** be executed in control flow that is uniform within a derivative group.
  - dynamic instances of implicit derivative operations **can** be executed in control flow that is not uniform within the derivative group, but results are undefined.

Fragment shaders that statically execute derivative operations **must** launch sufficient invocations to ensure their correct operation; additional helper invocations are launched for framebuffer locations not covered by rasterized fragments if necessary.

NOTE	In a mesh, task, or compute shader, it is the application's responsibility to ensure that sufficient invocations are launched.

Derivative operations calculate their results as the difference between the result of P across invocations in the quad. For fine derivative operations (`OpDPdxFine` and `OpDPdyFine`), the values of $DPdx(P_i)$ are calculated as

$$DPdx(P_0) = DPdx(P_1) = P_1 - P_0$$

$$DPdx(P_2) = DPdx(P_3) = P_3 - P_2$$

and the values of $DPdy(P_i)$ are calculated as

$$DPdy(P_0) = DPdy(P_2) = P_2 - P_0$$

$$DPdy(P_1) = DPdy(P_3) = P_3 - P_1$$

where i is the index of each invocation as described in Quad.

Coarse derivative operations (`OpDPdxCoarse` and `OpDPdyCoarse`), calculate their results in roughly the same manner, but **may** only calculate two values instead of four (one for each of DPdx and DPdy), reusing the same result no matter the originating invocation. If an implementation does this, it **should** use the fine derivative calculations described for P_0.

NOTE

Derivative values are calculated between fragments rather than pixels. If the fragment shader invocations involved in the calculation cover multiple pixels, these operations cover a wider area, resulting in larger derivative values. This in turn will result in a coarser LOD being selected for image sampling operations using derivatives.

Applications may want to account for this when using multi-pixel fragments; if pixel derivatives are desired, applications should use explicit derivative operations and divide the results by the size of the fragment in each dimension as follows:

$$DPdx(P_n)' = DPdx(P_n) / w$$

$$DPdy(P_n)' = DPdy(P_n) / h$$

where w and h are the size of the fragments in the quad, and $DPdx(P_n)'$ and $DPdy(P_n)'$ are the pixel derivatives.

The results for `OpDPdx` and `OpDPdy` **may** be calculated as either fine or coarse derivatives, with implementations favoring the most efficient approach. Implementations **must** choose coarse or fine consistently between the two.

Executing `OpFwidthFine`, `OpFwidthCoarse`, or `OpFwidth` is equivalent to executing the corresponding `OpDPdx*` and `OpDPdy*` instructions, taking the absolute value of the results, and summing them.

Executing an `OpImage*Sample*ImplicitLod` instruction is equivalent to executing `OpDPdx`(`Coordinate`) and `OpDPdy`(`Coordinate`), and passing the results as the `Grad` operands `dx` and `dy`.

NOTE

It is expected that using the `ImplicitLod` variants of sampling functions will be substantially more efficient than using the `ExplicitLod` variants with explicitly generated derivatives.

9.24. Helper Invocations

When performing derivative operations in a fragment shader, additional invocations **may** be spawned in order to ensure correct results. These additional invocations are known as *helper invocations* and **can** be identified by a non-zero value in the `HelperInvocation` built-in. Stores and atomics performed by helper invocations **must** not have any effect on memory except for the `Function`, `Private` and `Output` storage classes, and values returned by atomic instructions in helper invocations are undefined.

NOTE

While storage to `Output` storage class has an effect even in helper invocations, it does not mean that helper invocations have an effect on the framebuffer. `Output` variables in fragment shaders can be read from as well, and they behave more like `Private` variables for the duration of the shader invocation.

9.25. Cooperative Matrices

A *cooperative matrix* type is a SPIR-V type where the storage for and computations performed on the matrix are spread across the invocations in a scope instance. These types give the implementation freedom in how to optimize matrix multiplies.

SPIR-V defines the types and instructions, but does not specify rules about what sizes/combinations are valid, and it is expected that different implementations **may** support different sizes.

To enumerate the supported cooperative matrix types and operations, call:

```
// Provided by VK_KHR_cooperative_matrix
VkResult vkGetPhysicalDeviceCooperativeMatrixPropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pPropertyCount,
    VkCooperativeMatrixPropertiesKHR*           pProperties);
```

- `physicalDevice` is the physical device.
- `pPropertyCount` is a pointer to an integer related to the number of cooperative matrix properties available or queried.
- `pProperties` is either `NULL` or a pointer to an array of VkCooperativeMatrixPropertiesKHR structures.

If `pProperties` is `NULL`, then the number of cooperative matrix properties available is returned in `pPropertyCount`. Otherwise, `pPropertyCount` **must** point to a variable set by the application to the number of elements in the `pProperties` array, and on return the variable is overwritten with the number of structures actually written to `pProperties`. If `pPropertyCount` is less than the number of cooperative matrix properties available, at most `pPropertyCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available cooperative matrix properties were returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceCooperativeMatrixPropertiesKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceCooperativeMatrixPropertiesKHR-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceCooperativeMatrixPropertiesKHR-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkCooperativeMatrixPropertiesKHR structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

Each VkCooperativeMatrixPropertiesKHR structure describes a single supported combination of types for a matrix multiply/add operation (OpCooperativeMatrixMulAddKHR). The multiply **can** be described in terms of the following variables and types (in SPIR-V pseudocode):

```
    %A is of type OpTypeCooperativeMatrixKHR %AType %scope %MSize %KSize %MatrixAKHR
    %B is of type OpTypeCooperativeMatrixKHR %BType %scope %KSize %NSize %MatrixBKHR
    %C is of type OpTypeCooperativeMatrixKHR %CType %scope %MSize %NSize
%MatrixAccumulatorKHR
    %Result is of type OpTypeCooperativeMatrixKHR %ResultType %scope %MSize %NSize
%MatrixAccumulatorKHR

    %Result = %A * %B + %C // using OpCooperativeMatrixMulAddKHR
```

A matrix multiply with these dimensions is known as an *MxNxK* matrix multiply.

The VkCooperativeMatrixPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_cooperative_matrix
typedef struct VkCooperativeMatrixPropertiesKHR {
    VkStructureType       sType;
    void*                 pNext;
    uint32_t              MSize;
    uint32_t              NSize;
    uint32_t              KSize;
    VkComponentTypeKHR    AType;
    VkComponentTypeKHR    BType;
    VkComponentTypeKHR    CType;
    VkComponentTypeKHR    ResultType;
    VkBool32              saturatingAccumulation;
    VkScopeKHR            scope;
} VkCooperativeMatrixPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- MSize is the number of rows in matrices A, C, and Result.

- KSize is the number of columns in matrix A and rows in matrix B.
- NSize is the number of columns in matrices B, C, Result.
- AType is the component type of matrix A, of type VkComponentTypeKHR.
- BType is the component type of matrix B, of type VkComponentTypeKHR.
- CType is the component type of matrix C, of type VkComponentTypeKHR.
- ResultType is the component type of matrix Result, of type VkComponentTypeKHR.
- saturatingAccumulation indicates whether the SaturatingAccumulation operand to OpCooperativeMatrixMulAddKHR **must** be present or not. If it is VK_TRUE, the SaturatingAccumulation operand **must** be present. If it is VK_FALSE, the SaturatingAccumulation operand **must** not be present.
- scope is the scope of all the matrix types, of type VkScopeKHR.

If some types are preferred over other types (e.g. for performance), they **should** appear earlier in the list enumerated by vkGetPhysicalDeviceCooperativeMatrixPropertiesKHR.

At least one entry in the list **must** have power of two values for all of MSize, KSize, and NSize.

scope **must** be VK_SCOPE_SUBGROUP_KHR.

Valid Usage (Implicit)

- VUID-VkCooperativeMatrixPropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_COOPERATIVE_MATRIX_PROPERTIES_KHR
- VUID-VkCooperativeMatrixPropertiesKHR-pNext-pNext
  pNext **must** be NULL

Possible values for VkScopeKHR include:

```
// Provided by VK_KHR_cooperative_matrix
typedef enum VkScopeKHR {
    VK_SCOPE_DEVICE_KHR = 1,
    VK_SCOPE_WORKGROUP_KHR = 2,
    VK_SCOPE_SUBGROUP_KHR = 3,
    VK_SCOPE_QUEUE_FAMILY_KHR = 5,
} VkScopeKHR;
```

- VK_SCOPE_DEVICE_KHR corresponds to SPIR-V Device scope.
- VK_SCOPE_WORKGROUP_KHR corresponds to SPIR-V Workgroup scope.
- VK_SCOPE_SUBGROUP_KHR corresponds to SPIR-V Subgroup scope.
- VK_SCOPE_QUEUE_FAMILY_KHR corresponds to SPIR-V QueueFamily scope.

All enum values match the corresponding SPIR-V value.

Possible values for VkComponentTypeKHR include:

```
// Provided by VK_KHR_cooperative_matrix
typedef enum VkComponentTypeKHR {
    VK_COMPONENT_TYPE_FLOAT16_KHR = 0,
    VK_COMPONENT_TYPE_FLOAT32_KHR = 1,
    VK_COMPONENT_TYPE_FLOAT64_KHR = 2,
    VK_COMPONENT_TYPE_SINT8_KHR = 3,
    VK_COMPONENT_TYPE_SINT16_KHR = 4,
    VK_COMPONENT_TYPE_SINT32_KHR = 5,
    VK_COMPONENT_TYPE_SINT64_KHR = 6,
    VK_COMPONENT_TYPE_UINT8_KHR = 7,
    VK_COMPONENT_TYPE_UINT16_KHR = 8,
    VK_COMPONENT_TYPE_UINT32_KHR = 9,
    VK_COMPONENT_TYPE_UINT64_KHR = 10,
} VkComponentTypeKHR;
```

- VK_COMPONENT_TYPE_FLOAT16_KHR corresponds to SPIR-V OpTypeFloat 16.
- VK_COMPONENT_TYPE_FLOAT32_KHR corresponds to SPIR-V OpTypeFloat 32.
- VK_COMPONENT_TYPE_FLOAT64_KHR corresponds to SPIR-V OpTypeFloat 64.
- VK_COMPONENT_TYPE_SINT8_KHR corresponds to SPIR-V OpTypeInt 8 0/1.
- VK_COMPONENT_TYPE_SINT16_KHR corresponds to SPIR-V OpTypeInt 16 0/1.
- VK_COMPONENT_TYPE_SINT32_KHR corresponds to SPIR-V OpTypeInt 32 0/1.
- VK_COMPONENT_TYPE_SINT64_KHR corresponds to SPIR-V OpTypeInt 64 0/1.
- VK_COMPONENT_TYPE_UINT8_KHR corresponds to SPIR-V OpTypeInt 8 0/1.
- VK_COMPONENT_TYPE_UINT16_KHR corresponds to SPIR-V OpTypeInt 16 0/1.
- VK_COMPONENT_TYPE_UINT32_KHR corresponds to SPIR-V OpTypeInt 32 0/1.
- VK_COMPONENT_TYPE_UINT64_KHR corresponds to SPIR-V OpTypeInt 64 0/1.

9.26. Validation Cache

Validation cache objects allow the result of internal validation to be reused, both within a single application run and between multiple runs. Reuse within a single run is achieved by passing the same validation cache object when creating supported Vulkan objects. Reuse across runs of an application is achieved by retrieving validation cache contents in one run of an application, saving the contents, and using them to preinitialize a validation cache on a subsequent run. The contents of the validation cache objects are managed by the validation layers. Applications **can** manage the host memory consumed by a validation cache object and control the amount of data retrieved from a validation cache object.

Validation cache objects are represented by VkValidationCacheEXT handles:

```
// Provided by VK_EXT_validation_cache
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkValidationCacheEXT)
```

To create validation cache objects, call:

```
// Provided by VK_EXT_validation_cache
VkResult vkCreateValidationCacheEXT(
    VkDevice                                    device,
    const VkValidationCacheCreateInfoEXT*       pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkValidationCacheEXT*                       pValidationCache);
```

- `device` is the logical device that creates the validation cache object.
- `pCreateInfo` is a pointer to a VkValidationCacheCreateInfoEXT structure containing the initial parameters for the validation cache object.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pValidationCache` is a pointer to a VkValidationCacheEXT handle in which the resulting validation cache object is returned.

NOTE Applications **can** track and manage the total host memory size of a validation cache object using the `pAllocator`. Applications **can** limit the amount of data retrieved from a validation cache object in `vkGetValidationCacheDataEXT`. Implementations **should** not internally limit the total number of entries added to a validation cache object or the total host memory consumed.

Once created, a validation cache **can** be passed to the `vkCreateShaderModule` command by adding this object to the VkShaderModuleCreateInfo structure's `pNext` chain. If a VkShaderModuleValidationCacheCreateInfoEXT object is included in the VkShaderModuleCreateInfo::`pNext` chain, and its `validationCache` field is not VK_NULL_HANDLE, the implementation will query it for possible reuse opportunities and update it with new content. The use of the validation cache object in these commands is internally synchronized, and the same validation cache object **can** be used in multiple threads simultaneously.

NOTE Implementations **should** make every effort to limit any critical sections to the actual accesses to the cache, which is expected to be significantly shorter than the duration of the `vkCreateShaderModule` command.

Valid Usage (Implicit)

- VUID-vkCreateValidationCacheEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateValidationCacheEXT-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkValidationCacheCreateInfoEXT structure

- VUID-vkCreateValidationCacheEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateValidationCacheEXT-pValidationCache-parameter
  `pValidationCache` **must** be a valid pointer to a VkValidationCacheEXT handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkValidationCacheCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_validation_cache
typedef struct VkValidationCacheCreateInfoEXT {
    VkStructureType                 sType;
    const void*                     pNext;
    VkValidationCacheCreateFlagsEXT flags;
    size_t                          initialDataSize;
    const void*                     pInitialData;
} VkValidationCacheCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `initialDataSize` is the number of bytes in `pInitialData`. If `initialDataSize` is zero, the validation cache will initially be empty.
- `pInitialData` is a pointer to previously retrieved validation cache data. If the validation cache data is incompatible (as defined below) with the device, the validation cache will be initially empty. If `initialDataSize` is zero, `pInitialData` is ignored.

Valid Usage

- VUID-VkValidationCacheCreateInfoEXT-initialDataSize-01534
  If `initialDataSize` is not `0`, it **must** be equal to the size of `pInitialData`, as returned by `vkGetValidationCacheDataEXT` when `pInitialData` was originally retrieved
- VUID-VkValidationCacheCreateInfoEXT-initialDataSize-01535
  If `initialDataSize` is not `0`, `pInitialData` **must** have been retrieved from a previous call to `vkGetValidationCacheDataEXT`

Valid Usage (Implicit)

- VUID-VkValidationCacheCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VALIDATION_CACHE_CREATE_INFO_EXT`
- VUID-VkValidationCacheCreateInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkValidationCacheCreateInfoEXT-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkValidationCacheCreateInfoEXT-pInitialData-parameter
  If `initialDataSize` is not `0`, `pInitialData` **must** be a valid pointer to an array of `initialDataSize` bytes

```
// Provided by VK_EXT_validation_cache
typedef VkFlags VkValidationCacheCreateFlagsEXT;
```

`VkValidationCacheCreateFlagsEXT` is a bitmask type for setting a mask, but is currently reserved for future use.

Validation cache objects **can** be merged using the command:

```
// Provided by VK_EXT_validation_cache
VkResult vkMergeValidationCachesEXT(
    VkDevice                                    device,
    VkValidationCacheEXT                        dstCache,
    uint32_t                                    srcCacheCount,
    const VkValidationCacheEXT*                 pSrcCaches);
```

- `device` is the logical device that owns the validation cache objects.
- `dstCache` is the handle of the validation cache to merge results into.
- `srcCacheCount` is the length of the `pSrcCaches` array.
- `pSrcCaches` is a pointer to an array of validation cache handles, which will be merged into `dstCache`. The previous contents of `dstCache` are included after the merge.

NOTE The details of the merge operation are implementation-dependent, but implementations **should** merge the contents of the specified validation caches and prune duplicate entries.

Valid Usage

- VUID-vkMergeValidationCachesEXT-dstCache-01536
  `dstCache` **must** not appear in the list of source caches

Valid Usage (Implicit)

- VUID-vkMergeValidationCachesEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkMergeValidationCachesEXT-dstCache-parameter
  `dstCache` **must** be a valid VkValidationCacheEXT handle
- VUID-vkMergeValidationCachesEXT-pSrcCaches-parameter
  `pSrcCaches` **must** be a valid pointer to an array of `srcCacheCount` valid VkValidationCacheEXT handles
- VUID-vkMergeValidationCachesEXT-srcCacheCount-arraylength
  `srcCacheCount` **must** be greater than `0`
- VUID-vkMergeValidationCachesEXT-dstCache-parent
  `dstCache` **must** have been created, allocated, or retrieved from `device`
- VUID-vkMergeValidationCachesEXT-pSrcCaches-parent
  Each element of `pSrcCaches` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `dstCache` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Data **can** be retrieved from a validation cache object using the command:

```
// Provided by VK_EXT_validation_cache
VkResult vkGetValidationCacheDataEXT(
    VkDevice                                    device,
    VkValidationCacheEXT                        validationCache,
    size_t*                                     pDataSize,
    void*                                       pData);
```

- `device` is the logical device that owns the validation cache.
- `validationCache` is the validation cache to retrieve data from.
- `pDataSize` is a pointer to a value related to the amount of data in the validation cache, as

described below.

- pData is either NULL or a pointer to a buffer.

If pData is NULL, then the maximum size of the data that **can** be retrieved from the validation cache, in bytes, is returned in pDataSize. Otherwise, pDataSize **must** point to a variable set by the application to the size of the buffer, in bytes, pointed to by pData, and on return the variable is overwritten with the amount of data actually written to pData. If pDataSize is less than the maximum size that **can** be retrieved by the validation cache, at most pDataSize bytes will be written to pData, and vkGetValidationCacheDataEXT will return VK_INCOMPLETE instead of VK_SUCCESS, to indicate that not all of the validation cache was returned.

Any data written to pData is valid and **can** be provided as the pInitialData member of the VkValidationCacheCreateInfoEXT structure passed to vkCreateValidationCacheEXT.

Two calls to vkGetValidationCacheDataEXT with the same parameters **must** retrieve the same data unless a command that modifies the contents of the cache is called between them.

Applications **can** store the data retrieved from the validation cache, and use these data, possibly in a future run of the application, to populate new validation cache objects. The results of validation, however, **may** depend on the vendor ID, device ID, driver version, and other details of the device. To enable applications to detect when previously retrieved data is incompatible with the device, the initial bytes written to pData **must** be a header consisting of the following members:

Table 11. Layout for Validation Cache Header Version
VK_VALIDATION_CACHE_HEADER_VERSION_ONE_EXT

Offset	Size	Meaning
0	4	length in bytes of the entire validation cache header written as a stream of bytes, with the least significant byte first
4	4	a VkValidationCacheHeaderVersionEXT value written as a stream of bytes, with the least significant byte first
8	VK_UUID_SIZE	a layer commit ID expressed as a UUID, which uniquely identifies the version of the validation layers used to generate these validation results

The first four bytes encode the length of the entire validation cache header, in bytes. This value includes all fields in the header including the validation cache version field and the size of the length field.

The next four bytes encode the validation cache version, as described for VkValidationCacheHeaderVersionEXT. A consumer of the validation cache **should** use the cache version to interpret the remainder of the cache header.

If pDataSize is less than what is necessary to store this header, nothing will be written to pData and zero will be written to pDataSize.

Valid Usage (Implicit)

- VUID-vkGetValidationCacheDataEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetValidationCacheDataEXT-validationCache-parameter
  `validationCache` **must** be a valid VkValidationCacheEXT handle
- VUID-vkGetValidationCacheDataEXT-pDataSize-parameter
  `pDataSize` **must** be a valid pointer to a `size_t` value
- VUID-vkGetValidationCacheDataEXT-pData-parameter
  If the value referenced by `pDataSize` is not `0`, and `pData` is not `NULL`, `pData` **must** be a valid pointer to an array of `pDataSize` bytes
- VUID-vkGetValidationCacheDataEXT-validationCache-parent
  `validationCache` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Possible values of the second group of four bytes in the header returned by vkGetValidationCacheDataEXT, encoding the validation cache version, are:

```
// Provided by VK_EXT_validation_cache
typedef enum VkValidationCacheHeaderVersionEXT {
    VK_VALIDATION_CACHE_HEADER_VERSION_ONE_EXT = 1,
} VkValidationCacheHeaderVersionEXT;
```

- `VK_VALIDATION_CACHE_HEADER_VERSION_ONE_EXT` specifies version one of the validation cache.

To destroy a validation cache, call:

```
// Provided by VK_EXT_validation_cache
void vkDestroyValidationCacheEXT(
    VkDevice                                    device,
    VkValidationCacheEXT                        validationCache,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the validation cache object.
- `validationCache` is the handle of the validation cache to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyValidationCacheEXT-validationCache-01537
  If `VkAllocationCallbacks` were provided when `validationCache` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyValidationCacheEXT-validationCache-01538
  If no `VkAllocationCallbacks` were provided when `validationCache` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyValidationCacheEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyValidationCacheEXT-validationCache-parameter
  If `validationCache` is not VK_NULL_HANDLE, `validationCache` **must** be a valid VkValidationCacheEXT handle
- VUID-vkDestroyValidationCacheEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyValidationCacheEXT-validationCache-parent
  If `validationCache` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `validationCache` **must** be externally synchronized

Chapter 10. Pipelines

The following figure shows a block diagram of the Vulkan pipelines. Some Vulkan commands specify geometric objects to be drawn or computational work to be performed, while others specify state controlling how objects are handled by the various pipeline stages, or control data transfer between memory organized as images and buffers. Commands are effectively sent through a processing pipeline, either a *graphics pipeline*, a *ray tracing pipeline*, or a *compute pipeline*.

The first stage of the graphics pipeline (Input Assembler) assembles vertices to form geometric primitives such as points, lines, and triangles, based on a requested primitive topology. In the next stage (Vertex Shader) vertices **can** be transformed, computing positions and attributes for each vertex. If tessellation and/or geometry shaders are supported, they **can** then generate multiple primitives from a single input primitive, possibly changing the primitive topology or generating additional attribute data in the process.

The final resulting primitives are clipped to a clip volume in preparation for the next stage, Rasterization. The rasterizer produces a series of *fragments* associated with a region of the framebuffer, from a two-dimensional description of a point, line segment, or triangle. These fragments are processed by fragment operations to determine whether generated values will be written to the framebuffer. Fragment shading determines the values to be written to the framebuffer attachments. Framebuffer operations then read and write the color and depth/stencil attachments of the framebuffer for a given subpass of a render pass instance. The attachments **can** be used as input attachments in the fragment shader in a later subpass of the same render pass.

The compute pipeline is a separate pipeline from the graphics pipeline, which operates on one-, two-, or three-dimensional workgroups which **can** read from and write to buffer and image memory.

This ordering is meant only as a tool for describing Vulkan, not as a strict rule of how Vulkan is implemented, and we present it only as a means to organize the various operations of the pipelines. Actual ordering guarantees between pipeline stages are explained in detail in the synchronization chapter.

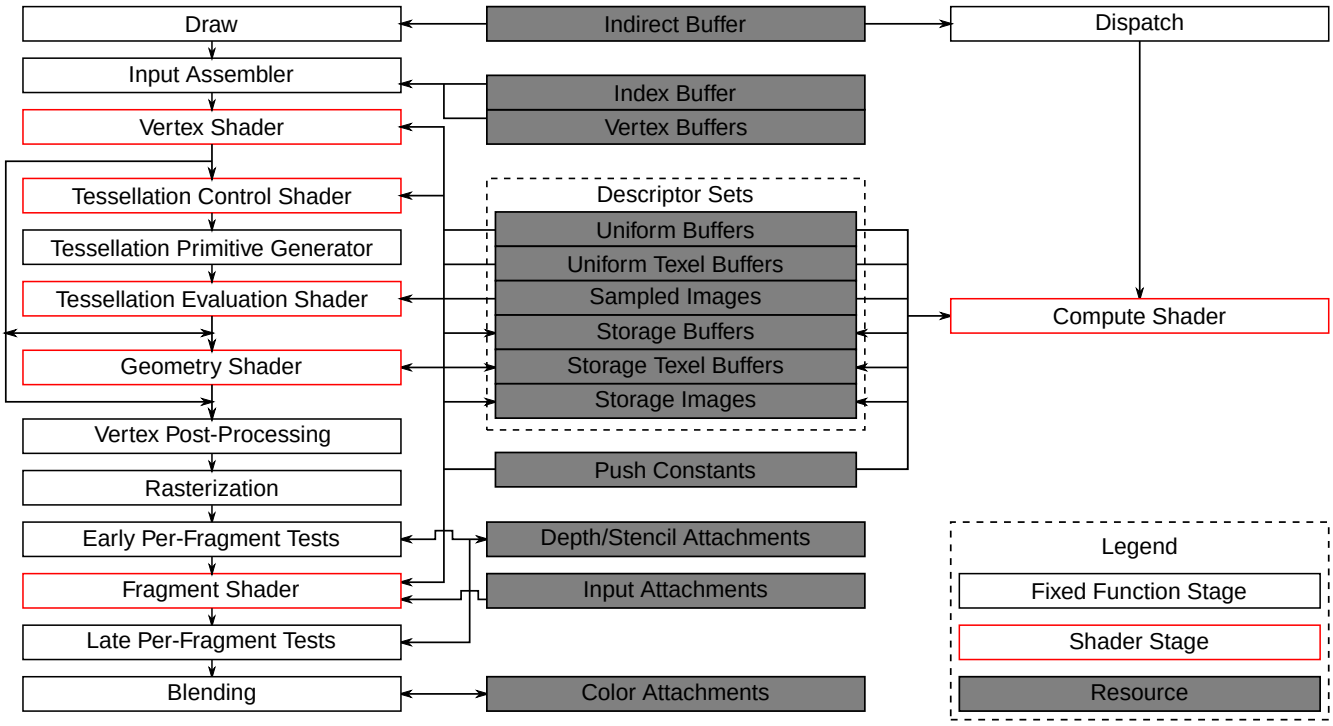


Figure 2. Block diagram of the Vulkan pipeline

Each pipeline is controlled by a monolithic object created from a description of all of the shader stages and any relevant fixed-function stages. Linking the whole pipeline together allows the optimization of shaders based on their input/outputs and eliminates expensive draw time state validation.

A pipeline object is bound to the current state using vkCmdBindPipeline. Any pipeline object state that is specified as dynamic is not applied to the current state when the pipeline object is bound, but is instead set by dynamic state setting commands.

No state, including dynamic state, is inherited from one command buffer to another.

Compute, ray tracing, and graphics pipelines are each represented by `VkPipeline` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkPipeline)
```

10.1. Multiple Pipeline Creation

Multiple pipelines **can** be created in a single call by commands such as vkCreateRayTracingPipelinesKHR, vkCreateComputePipelines, and vkCreateGraphicsPipelines.

The creation commands are passed an array `pCreateInfos` of `Vk*PipelineCreateInfo` structures specifying parameters of each pipeline to be created, and return a corresponding array of handles in `pPipelines`. Each element index *i* of `pPipelines` is created based on the corresponding element *i* of `pCreateInfos`.

Applications **can** group together similar pipelines to be created in a single call, and implementations are encouraged to look for reuse opportunities when creating a group.

When attempting to create many pipelines in a single command, it is possible that creation **may** fail for a subset of them. In this case, the corresponding elements of `pPipelines` will be VK_NULL_HANDLE. If creation fails for a pipeline despite valid arguments (for example, due to out of memory errors), the VkResult code returned by the pipeline creation command will indicate why. The implementation will attempt to create all pipelines, and only return VK_NULL_HANDLE values for those that actually failed.

If creation fails for multiple pipelines, the returned VkResult **must** be the return value of any one of the pipelines which did not succeed. An application **can** reliably clean up from a failed call by iterating over the `pPipelines` array and destroying every element that is not VK_NULL_HANDLE.

If the entire command fails and no pipelines are created, all elements of `pPipelines` will be VK_NULL_HANDLE.

10.2. Compute Pipelines

Compute pipelines consist of a single static compute shader stage and the pipeline layout.

The compute pipeline represents a compute shader and is created by calling `vkCreateComputePipelines` with `module` and `pName` selecting an entry point from a shader module, where that entry point defines a valid compute shader, in the VkPipelineShaderStageCreateInfo structure contained within the VkComputePipelineCreateInfo structure.

To create compute pipelines, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateComputePipelines(
    VkDevice                                    device,
    VkPipelineCache                             pipelineCache,
    uint32_t                                    createInfoCount,
    const VkComputePipelineCreateInfo*          pCreateInfos,
    const VkAllocationCallbacks*                pAllocator,
    VkPipeline*                                 pPipelines);
```

- `device` is the logical device that creates the compute pipelines.
- `pipelineCache` is either VK_NULL_HANDLE, indicating that pipeline caching is disabled; or the handle of a valid pipeline cache object, in which case use of that cache is enabled for the duration of the command.
- `createInfoCount` is the length of the `pCreateInfos` and `pPipelines` arrays.
- `pCreateInfos` is a pointer to an array of VkComputePipelineCreateInfo structures.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pPipelines` is a pointer to an array of VkPipeline handles in which the resulting compute pipeline objects are returned.

Pipelines are created and returned as described for Multiple Pipeline Creation.

Valid Usage

- VUID-vkCreateComputePipelines-device-09661
  `device` **must** support at least one queue family with the `VK_QUEUE_COMPUTE_BIT` capability
- VUID-vkCreateComputePipelines-flags-00695
  If the `flags` member of any element of `pCreateInfos` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, and the `basePipelineIndex` member of that same element is not `-1`, `basePipelineIndex` **must** be less than the index into `pCreateInfos` that corresponds to that element
- VUID-vkCreateComputePipelines-flags-00696
  If the `flags` member of any element of `pCreateInfos` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, the base pipeline **must** have been created with the `VK_PIPELINE_CREATE_ALLOW_DERIVATIVES_BIT` flag set
- VUID-vkCreateComputePipelines-pNext-09616
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `pipelineCache` **must** be VK_NULL_HANDLE
- VUID-vkCreateComputePipelines-pNext-09617
  If a VkPipelineCreateFlags2CreateInfoKHR structure with the `VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR` flag set is included in the `pNext` chain of any element of `pCreateInfos`, `pipelineCache` **must** be VK_NULL_HANDLE
- VUID-vkCreateComputePipelines-binaryCount-09620
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATION_FEEDBACK_APPLICATION_PIPELINE_CACHE_HIT_BIT` **must** not be set in the `flags` of that element
- VUID-vkCreateComputePipelines-binaryCount-09621
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATION_FEEDBACK_BASE_PIPELINE_ACCELERATION_BIT` **must** not be set in the `flags` of that element
- VUID-vkCreateComputePipelines-binaryCount-09622
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT_EXT` **must** not be set in the `flags` of that element

Valid Usage (Implicit)

- VUID-vkCreateComputePipelines-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateComputePipelines-pipelineCache-parameter
  If `pipelineCache` is not VK_NULL_HANDLE, `pipelineCache` **must** be a valid VkPipelineCache handle
- VUID-vkCreateComputePipelines-pCreateInfos-parameter
  `pCreateInfos` **must** be a valid pointer to an array of `createInfoCount` valid

VkComputePipelineCreateInfo structures

- VUID-vkCreateComputePipelines-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateComputePipelines-pPipelines-parameter
  `pPipelines` **must** be a valid pointer to an array of `createInfoCount` VkPipeline handles
- VUID-vkCreateComputePipelines-createInfoCount-arraylength
  `createInfoCount` **must** be greater than `0`
- VUID-vkCreateComputePipelines-pipelineCache-parent
  If `pipelineCache` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkComputePipelineCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkComputePipelineCreateInfo {
    VkStructureType                    sType;
    const void*                        pNext;
    VkPipelineCreateFlags              flags;
    VkPipelineShaderStageCreateInfo    stage;
    VkPipelineLayout                   layout;
    VkPipeline                         basePipelineHandle;
    int32_t                            basePipelineIndex;
} VkComputePipelineCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkPipelineCreateFlagBits specifying how the pipeline will be generated.
- `stage` is a VkPipelineShaderStageCreateInfo structure describing the compute shader.
- `layout` is the description of binding locations used by both the pipeline and descriptor sets used with the pipeline.
- `basePipelineHandle` is a pipeline to derive from.
- `basePipelineIndex` is an index into the `pCreateInfos` parameter to use as a pipeline to derive

from.

The parameters basePipelineHandle and basePipelineIndex are described in more detail in Pipeline Derivatives.

If the pNext chain includes a VkPipelineCreateFlags2CreateInfoKHR structure, VkPipelineCreateFlags2CreateInfoKHR::flags from that structure is used instead of flags from this structure.

Valid Usage

- VUID-VkComputePipelineCreateInfo-None-09497
  If the pNext chain does not include a VkPipelineCreateFlags2CreateInfoKHR structure, flags **must** be a valid combination of VkPipelineCreateFlagBits values
- VUID-VkComputePipelineCreateInfo-flags-07984
  If flags contains the VK_PIPELINE_CREATE_DERIVATIVE_BIT flag, and basePipelineIndex is -1, basePipelineHandle **must** be a valid compute VkPipeline handle
- VUID-VkComputePipelineCreateInfo-flags-07985
  If flags contains the VK_PIPELINE_CREATE_DERIVATIVE_BIT flag, and basePipelineHandle is VK_NULL_HANDLE, basePipelineIndex **must** be a valid index into the calling command's pCreateInfos parameter
- VUID-VkComputePipelineCreateInfo-flags-07986
  If flags contains the VK_PIPELINE_CREATE_DERIVATIVE_BIT flag, basePipelineIndex **must** be -1 or basePipelineHandle **must** be VK_NULL_HANDLE
- VUID-VkComputePipelineCreateInfo-layout-07987
  If a push constant block is declared in a shader, a push constant range in layout **must** match the shader stage
- VUID-VkComputePipelineCreateInfo-layout-10069
  If a push constant block is declared in a shader, the block must be contained inside the push constant range in layout that matches the stage
- VUID-VkComputePipelineCreateInfo-layout-07988
  If a resource variables is declared in a shader, a descriptor slot in layout **must** match the shader stage
- VUID-VkComputePipelineCreateInfo-layout-07990
  If a resource variables is declared in a shader, a descriptor slot in layout **must** match the descriptor type
- VUID-VkComputePipelineCreateInfo-layout-07991
  If a resource variables is declared in a shader as an array, a descriptor slot in layout **must** match the descriptor count
- VUID-VkComputePipelineCreateInfo-flags-03365
  flags **must** not include VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR
- VUID-VkComputePipelineCreateInfo-flags-03366
  flags **must** not include VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR

- VUID-VkComputePipelineCreateInfo-flags-03367
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR`
- VUID-VkComputePipelineCreateInfo-flags-03368
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR`
- VUID-VkComputePipelineCreateInfo-flags-03369
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR`
- VUID-VkComputePipelineCreateInfo-flags-03370
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR`
- VUID-VkComputePipelineCreateInfo-flags-03576
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR`
- VUID-VkComputePipelineCreateInfo-flags-11007
  If `flags` includes `VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT`, then the `VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT::deviceGeneratedCommands` feature **must** be enabled
- VUID-VkComputePipelineCreateInfo-stage-00701
  The `stage` member of `stage` **must** be `VK_SHADER_STAGE_COMPUTE_BIT`
- VUID-VkComputePipelineCreateInfo-stage-00702
  The shader code for the entry point identified by `stage` and the rest of the state identified by this structure **must** adhere to the pipeline linking rules described in the Shader Interfaces chapter
- VUID-VkComputePipelineCreateInfo-layout-01687
  The number of resources in `layout` accessible to the compute shader stage **must** be less than or equal to `VkPhysicalDeviceLimits::maxPerStageResources`
- VUID-VkComputePipelineCreateInfo-shaderEnqueue-09177
  `flags` **must** not include `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR`
- VUID-VkComputePipelineCreateInfo-flags-07367
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT`

Valid Usage (Implicit)

- VUID-VkComputePipelineCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO`
- VUID-VkComputePipelineCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkPipelineBinaryInfoKHR or VkPipelineCreateFlags2CreateInfoKHR
- VUID-VkComputePipelineCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkComputePipelineCreateInfo-stage-parameter
  `stage` **must** be a valid VkPipelineShaderStageCreateInfo structure

- VUID-VkComputePipelineCreateInfo-layout-parameter
  `layout` **must** be a valid VkPipelineLayout handle
- VUID-VkComputePipelineCreateInfo-commonparent
  Both of `basePipelineHandle`, and `layout` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The `VkPipelineShaderStageCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineShaderStageCreateInfo {
    VkStructureType                     sType;
    const void*                         pNext;
    VkPipelineShaderStageCreateFlags    flags;
    VkShaderStageFlagBits               stage;
    VkShaderModule                      module;
    const char*                         pName;
    const VkSpecializationInfo*         pSpecializationInfo;
} VkPipelineShaderStageCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkPipelineShaderStageCreateFlagBits specifying how the pipeline shader stage will be generated.
- `stage` is a VkShaderStageFlagBits value specifying a single pipeline stage.
- `module` is optionally a VkShaderModule object containing the shader code for this stage.
- `pName` is a pointer to a null-terminated UTF-8 string specifying the entry point name of the shader for this stage.
- `pSpecializationInfo` is a pointer to a VkSpecializationInfo structure, as described in Specialization Constants, or `NULL`.

If `module` is not VK_NULL_HANDLE, the shader code used by the pipeline is defined by `module`. If `module` is VK_NULL_HANDLE, the shader code is defined by the chained VkShaderModuleCreateInfo if present.

If the `shaderModuleIdentifier` feature is enabled, applications **can** omit shader code for `stage` and instead provide a module identifier. This is done by including a VkPipelineShaderStageModuleIdentifierCreateInfoEXT struct with `identifierSize` not equal to 0 in the `pNext` chain. A shader stage created in this way is equivalent to one created using a shader module with the same identifier. The identifier allows an implementation to look up a pipeline without consuming a valid SPIR-V module. If a pipeline is not found, pipeline compilation is not possible and the implementation **must** fail as specified by `VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT`.

When an identifier is used in lieu of a shader module, implementations **may** fail pipeline compilation with `VK_PIPELINE_COMPILE_REQUIRED` for any reason.

NOTE The rationale for the relaxed requirement on implementations to return a pipeline with VkPipelineShaderStageModuleIdentifierCreateInfoEXT is that layers or tools may intercept pipeline creation calls and require the full SPIR-V context to operate correctly. ICDs are not expected to fail pipeline compilation if the pipeline exists in a cache somewhere.

Applications **can** use identifiers when creating pipelines with VK_PIPELINE_CREATE_LIBRARY_BIT_KHR. When creating such pipelines, VK_SUCCESS **may** be returned, but subsequently fail when referencing the pipeline in a VkPipelineLibraryCreateInfoKHR struct. Applications **must** allow pipeline compilation to fail during link steps with VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT as it **may** not be possible to determine if a pipeline **can** be created from identifiers until the link step.

Valid Usage

- VUID-VkPipelineShaderStageCreateInfo-stage-00704
  If the geometryShader feature is not enabled, stage **must** not be VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-VkPipelineShaderStageCreateInfo-stage-00705
  If the tessellationShader feature is not enabled, stage **must** not be VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT or VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-VkPipelineShaderStageCreateInfo-stage-00706
  stage **must** not be VK_SHADER_STAGE_ALL_GRAPHICS, or VK_SHADER_STAGE_ALL
- VUID-VkPipelineShaderStageCreateInfo-pName-00707
  pName **must** be the name of an OpEntryPoint in module with an execution model that matches stage
- VUID-VkPipelineShaderStageCreateInfo-maxClipDistances-00708
  If the identified entry point includes any variable in its interface that is declared with the ClipDistance BuiltIn decoration, that variable **must** not have an array size greater than VkPhysicalDeviceLimits::maxClipDistances
- VUID-VkPipelineShaderStageCreateInfo-maxCullDistances-00709
  If the identified entry point includes any variable in its interface that is declared with the CullDistance BuiltIn decoration, that variable **must** not have an array size greater than VkPhysicalDeviceLimits::maxCullDistances
- VUID-VkPipelineShaderStageCreateInfo-maxCombinedClipAndCullDistances-00710
  If the identified entry point includes variables in its interface that are declared with the ClipDistance BuiltIn decoration and variables in its interface that are declared with the CullDistance BuiltIn decoration, those variables **must** not have array sizes which sum to more than VkPhysicalDeviceLimits::maxCombinedClipAndCullDistances
- VUID-VkPipelineShaderStageCreateInfo-maxSampleMaskWords-00711
  If the identified entry point includes any variable in its interface that is declared with the SampleMask BuiltIn decoration, that variable **must** not have an array size greater than VkPhysicalDeviceLimits::maxSampleMaskWords

- VUID-VkPipelineShaderStageCreateInfo-stage-00713
  If `stage` is `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT` or `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT`, and the identified entry point has an `OpExecutionMode` instruction specifying a patch size with `OutputVertices`, the patch size **must** be greater than `0` and less than or equal to `VkPhysicalDeviceLimits::maxTessellationPatchSize`
- VUID-VkPipelineShaderStageCreateInfo-stage-00714
  If `stage` is `VK_SHADER_STAGE_GEOMETRY_BIT`, the identified entry point **must** have an `OpExecutionMode` instruction specifying a maximum output vertex count that is greater than `0` and less than or equal to `VkPhysicalDeviceLimits::maxGeometryOutputVertices`
- VUID-VkPipelineShaderStageCreateInfo-stage-00715
  If `stage` is `VK_SHADER_STAGE_GEOMETRY_BIT`, the identified entry point **must** have an `OpExecutionMode` instruction specifying an invocation count that is greater than `0` and less than or equal to `VkPhysicalDeviceLimits::maxGeometryShaderInvocations`
- VUID-VkPipelineShaderStageCreateInfo-stage-02596
  If `stage` is either `VK_SHADER_STAGE_VERTEX_BIT`, `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT`, `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT`, or `VK_SHADER_STAGE_GEOMETRY_BIT`, and the identified entry point writes to `Layer` for any primitive, it **must** write the same value to `Layer` for all vertices of a given primitive
- VUID-VkPipelineShaderStageCreateInfo-stage-02597
  If `stage` is either `VK_SHADER_STAGE_VERTEX_BIT`, `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT`, `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT`, or `VK_SHADER_STAGE_GEOMETRY_BIT`, and the identified entry point writes to `ViewportIndex` for any primitive, it **must** write the same value to `ViewportIndex` for all vertices of a given primitive
- VUID-VkPipelineShaderStageCreateInfo-stage-06685
  If `stage` is `VK_SHADER_STAGE_FRAGMENT_BIT`, and the identified entry point writes to `FragDepth` in any execution path, all execution paths that are not exclusive to helper invocations **must** either discard the fragment, or write or initialize the value of `FragDepth`
- VUID-VkPipelineShaderStageCreateInfo-stage-06686
  If `stage` is `VK_SHADER_STAGE_FRAGMENT_BIT`, and the identified entry point writes to `FragStencilRefEXT` in any execution path, all execution paths that are not exclusive to helper invocations **must** either discard the fragment, or write or initialize the value of `FragStencilRefEXT`
- VUID-VkPipelineShaderStageCreateInfo-stage-08771
  If a shader module identifier is not specified for this `stage`, `module` **must** be a valid VkShaderModule , or the `pNext` chain of the parent `Vk*CreateInfo` structure **must** set VkPipelineBinaryInfoKHR::`binaryCount` to a value greater than `0`, if none of the following features are enabled:
  - `maintenance5`
- VUID-VkPipelineShaderStageCreateInfo-stage-06845
  If a shader module identifier is not specified for this `stage`, `module` **must** be a valid VkShaderModule, or there **must** be a valid VkShaderModuleCreateInfo structure in the `pNext` chain , or the `pNext` chain of the parent `Vk*CreateInfo` structure **must** set VkPipelineBinaryInfoKHR::`binaryCount` to a value greater than `0`,

- VUID-VkPipelineShaderStageCreateInfo-stage-06844
  If a shader module identifier is specified for this `stage`, the `pNext` chain **must** not include a VkShaderModuleCreateInfo structure
- VUID-VkPipelineShaderStageCreateInfo-stage-06848
  If a shader module identifier is specified for this `stage`, `module` **must** be VK_NULL_HANDLE
- VUID-VkPipelineShaderStageCreateInfo-pSpecializationInfo-06849
  If a shader module identifier is not specified, the shader code used by the pipeline **must** be valid as described by the Khronos SPIR-V Specification after applying the specializations provided in `pSpecializationInfo`, if any, and then converting all specialization constants into fixed constants

Valid Usage (Implicit)

- VUID-VkPipelineShaderStageCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO`
- VUID-VkPipelineShaderStageCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkDebugUtilsObjectNameInfoEXT, VkPipelineShaderStageModuleIdentifierCreateInfoEXT, VkPipelineShaderStageRequiredSubgroupSizeCreateInfo, VkShaderModuleCreateInfo, or VkShaderModuleValidationCacheCreateInfoEXT
- VUID-VkPipelineShaderStageCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPipelineShaderStageCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineShaderStageCreateInfo-stage-parameter
  `stage` **must** be a valid VkShaderStageFlagBits value
- VUID-VkPipelineShaderStageCreateInfo-module-parameter
  If `module` is not VK_NULL_HANDLE, `module` **must** be a valid VkShaderModule handle
- VUID-VkPipelineShaderStageCreateInfo-pName-parameter
  `pName` **must** be a null-terminated UTF-8 string
- VUID-VkPipelineShaderStageCreateInfo-pSpecializationInfo-parameter
  If `pSpecializationInfo` is not `NULL`, `pSpecializationInfo` **must** be a valid pointer to a valid VkSpecializationInfo structure

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineShaderStageCreateFlags;
```

`VkPipelineShaderStageCreateFlags` is a bitmask type for setting a mask of zero or more VkPipelineShaderStageCreateFlagBits.

Possible values of the `flags` member of VkPipelineShaderStageCreateInfo specifying how a pipeline

shader stage is created, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkPipelineShaderStageCreateFlagBits {
} VkPipelineShaderStageCreateFlagBits;
```

Bits which **can** be set by commands and structures, specifying one or more shader stages, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkShaderStageFlagBits {
    VK_SHADER_STAGE_VERTEX_BIT = 0x00000001,
    VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT = 0x00000002,
    VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT = 0x00000004,
    VK_SHADER_STAGE_GEOMETRY_BIT = 0x00000008,
    VK_SHADER_STAGE_FRAGMENT_BIT = 0x00000010,
    VK_SHADER_STAGE_COMPUTE_BIT = 0x00000020,
    VK_SHADER_STAGE_ALL_GRAPHICS = 0x0000001F,
    VK_SHADER_STAGE_ALL = 0x7FFFFFFF,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_SHADER_STAGE_RAYGEN_BIT_KHR = 0x00000100,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_SHADER_STAGE_ANY_HIT_BIT_KHR = 0x00000200,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_SHADER_STAGE_CLOSEST_HIT_BIT_KHR = 0x00000400,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_SHADER_STAGE_MISS_BIT_KHR = 0x00000800,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_SHADER_STAGE_INTERSECTION_BIT_KHR = 0x00001000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_SHADER_STAGE_CALLABLE_BIT_KHR = 0x00002000,
} VkShaderStageFlagBits;
```

- VK_SHADER_STAGE_VERTEX_BIT specifies the vertex stage.
- VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT specifies the tessellation control stage.
- VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT specifies the tessellation evaluation stage.
- VK_SHADER_STAGE_GEOMETRY_BIT specifies the geometry stage.
- VK_SHADER_STAGE_FRAGMENT_BIT specifies the fragment stage.
- VK_SHADER_STAGE_COMPUTE_BIT specifies the compute stage.
- VK_SHADER_STAGE_ALL_GRAPHICS is a combination of bits used as shorthand to specify all graphics stages defined above (excluding the compute stage).
- VK_SHADER_STAGE_ALL is a combination of bits used as shorthand to specify all shader stages supported by the device, including all additional stages which are introduced by extensions.
- VK_SHADER_STAGE_RAYGEN_BIT_KHR specifies the ray generation stage.
- VK_SHADER_STAGE_ANY_HIT_BIT_KHR specifies the any-hit stage.

- VK_SHADER_STAGE_CLOSEST_HIT_BIT_KHR specifies the closest hit stage.
- VK_SHADER_STAGE_MISS_BIT_KHR specifies the miss stage.
- VK_SHADER_STAGE_INTERSECTION_BIT_KHR specifies the intersection stage.
- VK_SHADER_STAGE_CALLABLE_BIT_KHR specifies the callable stage.

NOTE	VK_SHADER_STAGE_ALL_GRAPHICS only includes the original five graphics stages included in Vulkan 1.0, and not any stages added by extensions. Thus, it may not have the desired effect in all cases.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkShaderStageFlags;
```

VkShaderStageFlags is a bitmask type for setting a mask of zero or more VkShaderStageFlagBits.

An identifier **can** be provided instead of shader code in an attempt to compile pipelines without providing complete SPIR-V to the implementation.

The VkPipelineShaderStageModuleIdentifierCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_shader_module_identifier
typedef struct VkPipelineShaderStageModuleIdentifierCreateInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           identifierSize;
    const uint8_t*     pIdentifier;
} VkPipelineShaderStageModuleIdentifierCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- identifierSize is the size, in bytes, of the buffer pointed to by pIdentifier.
- pIdentifier is a pointer to a buffer of opaque data specifying an identifier.

Any identifier **can** be used. If the pipeline being created with identifier requires compilation to complete the pipeline creation call, pipeline compilation **must** fail as defined by VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT.

pIdentifier and identifierSize **can** be obtained from an VkShaderModuleIdentifierEXT queried earlier.

Valid Usage

- VUID-VkPipelineShaderStageModuleIdentifierCreateInfoEXT-pNext-06850
  If this structure is included in a pNext chain and identifierSize is not equal to 0, the shaderModuleIdentifier feature **must** be enabled

- VUID-VkPipelineShaderStageModuleIdentifierCreateInfoEXT-pNext-06851
  If this struct is included in a `pNext` chain of VkPipelineShaderStageCreateInfo and `identifierSize` is not equal to 0, the pipeline **must** be created with the `VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT` flag set
- VUID-VkPipelineShaderStageModuleIdentifierCreateInfoEXT-identifierSize-06852
  `identifierSize` **must** be less-or-equal to `VK_MAX_SHADER_MODULE_IDENTIFIER_SIZE_EXT`

Valid Usage (Implicit)

- VUID-VkPipelineShaderStageModuleIdentifierCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_MODULE_IDENTIFIER_CREATE_INFO_EXT`
- VUID-VkPipelineShaderStageModuleIdentifierCreateInfoEXT-pIdentifier-parameter
  If `identifierSize` is not `0`, `pIdentifier` **must** be a valid pointer to an array of `identifierSize` `uint8_t` values

10.3. Graphics Pipelines

Graphics pipelines consist of multiple shader stages, multiple fixed-function pipeline stages, and a pipeline layout.

To create graphics pipelines, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateGraphicsPipelines(
    VkDevice                                    device,
    VkPipelineCache                             pipelineCache,
    uint32_t                                    createInfoCount,
    const VkGraphicsPipelineCreateInfo*         pCreateInfos,
    const VkAllocationCallbacks*                pAllocator,
    VkPipeline*                                 pPipelines);
```

- `device` is the logical device that creates the graphics pipelines.
- `pipelineCache` is either VK_NULL_HANDLE, indicating that pipeline caching is disabled; or the handle of a valid pipeline cache object, in which case use of that cache is enabled for the duration of the command.
- `createInfoCount` is the length of the `pCreateInfos` and `pPipelines` arrays.
- `pCreateInfos` is a pointer to an array of VkGraphicsPipelineCreateInfo structures.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pPipelines` is a pointer to an array of VkPipeline handles in which the resulting graphics pipeline objects are returned.

The VkGraphicsPipelineCreateInfo structure includes an array of VkPipelineShaderStageCreateInfo

structures for each of the desired active shader stages, as well as creation information for all relevant fixed-function stages, and a pipeline layout.

Pipelines are created and returned as described for Multiple Pipeline Creation.

Valid Usage

- VUID-vkCreateGraphicsPipelines-device-09662
  `device` **must** support at least one queue family with the `VK_QUEUE_GRAPHICS_BIT` capability
- VUID-vkCreateGraphicsPipelines-flags-00720
  If the `flags` member of any element of `pCreateInfos` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, and the `basePipelineIndex` member of that same element is not `-1`, `basePipelineIndex` **must** be less than the index into `pCreateInfos` that corresponds to that element
- VUID-vkCreateGraphicsPipelines-flags-00721
  If the `flags` member of any element of `pCreateInfos` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, the base pipeline **must** have been created with the `VK_PIPELINE_CREATE_ALLOW_DERIVATIVES_BIT` flag set
- VUID-vkCreateGraphicsPipelines-pNext-09616
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `pipelineCache` **must** be VK_NULL_HANDLE
- VUID-vkCreateGraphicsPipelines-pNext-09617
  If a VkPipelineCreateFlags2CreateInfoKHR structure with the `VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR` flag set is included in the `pNext` chain of any element of `pCreateInfos`, `pipelineCache` **must** be VK_NULL_HANDLE
- VUID-vkCreateGraphicsPipelines-binaryCount-09620
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATION_FEEDBACK_APPLICATION_PIPELINE_CACHE_HIT_BIT` **must** not be set in the `flags` of that element
- VUID-vkCreateGraphicsPipelines-binaryCount-09621
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATION_FEEDBACK_BASE_PIPELINE_ACCELERATION_BIT` **must** not be set in the `flags` of that element
- VUID-vkCreateGraphicsPipelines-binaryCount-09622
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT_EXT` **must** not be set in the `flags` of that element

Valid Usage (Implicit)

- VUID-vkCreateGraphicsPipelines-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateGraphicsPipelines-pipelineCache-parameter

If pipelineCache is not VK_NULL_HANDLE, pipelineCache **must** be a valid VkPipelineCache handle

- VUID-vkCreateGraphicsPipelines-pCreateInfos-parameter
  pCreateInfos **must** be a valid pointer to an array of createInfoCount valid VkGraphicsPipelineCreateInfo structures
- VUID-vkCreateGraphicsPipelines-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateGraphicsPipelines-pPipelines-parameter
  pPipelines **must** be a valid pointer to an array of createInfoCount VkPipeline handles
- VUID-vkCreateGraphicsPipelines-createInfoCount-arraylength
  createInfoCount **must** be greater than 0
- VUID-vkCreateGraphicsPipelines-pipelineCache-parent
  If pipelineCache is a valid handle, it **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkGraphicsPipelineCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkGraphicsPipelineCreateInfo {
    VkStructureType                                 sType;
    const void*                                     pNext;
    VkPipelineCreateFlags                           flags;
    uint32_t                                        stageCount;
    const VkPipelineShaderStageCreateInfo*          pStages;
    const VkPipelineVertexInputStateCreateInfo*     pVertexInputState;
    const VkPipelineInputAssemblyStateCreateInfo*   pInputAssemblyState;
    const VkPipelineTessellationStateCreateInfo*    pTessellationState;
    const VkPipelineViewportStateCreateInfo*        pViewportState;
    const VkPipelineRasterizationStateCreateInfo*   pRasterizationState;
    const VkPipelineMultisampleStateCreateInfo*     pMultisampleState;
    const VkPipelineDepthStencilStateCreateInfo*    pDepthStencilState;
    const VkPipelineColorBlendStateCreateInfo*      pColorBlendState;
    const VkPipelineDynamicStateCreateInfo*         pDynamicState;
    VkPipelineLayout                                layout;
    VkRenderPass                                    renderPass;
    uint32_t                                        subpass;
    VkPipeline                                      basePipelineHandle;
    int32_t                                         basePipelineIndex;
} VkGraphicsPipelineCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkPipelineCreateFlagBits specifying how the pipeline will be generated.
- `stageCount` is the number of entries in the `pStages` array.
- `pStages` is a pointer to an array of `stageCount` VkPipelineShaderStageCreateInfo structures describing the set of the shader stages to be included in the graphics pipeline.
- `pVertexInputState` is a pointer to a VkPipelineVertexInputStateCreateInfo structure. It **can** be `NULL` if the pipeline is created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic state set.
- `pInputAssemblyState` is a pointer to a VkPipelineInputAssemblyStateCreateInfo structure which determines input assembly behavior for vertex shading, as described in Drawing Commands. If the `VK_EXT_extended_dynamic_state3` extension is enabled, it **can** be `NULL` if the pipeline is created with both `VK_DYNAMIC_STATE_PRIMITIVE_RESTART_ENABLE`, and `VK_DYNAMIC_STATE_PRIMITIVE_TOPOLOGY` dynamic states set and `dynamicPrimitiveTopologyUnrestricted` is `VK_TRUE`.
- `pTessellationState` is a pointer to a VkPipelineTessellationStateCreateInfo structure defining tessellation state used by tessellation shaders.
- `pViewportState` is a pointer to a VkPipelineViewportStateCreateInfo structure defining viewport state used when rasterization is enabled. If the `VK_EXT_extended_dynamic_state3` extension is enabled, it **can** be `NULL` if the pipeline is created with both `VK_DYNAMIC_STATE_VIEWPORT_WITH_COUNT`, and `VK_DYNAMIC_STATE_SCISSOR_WITH_COUNT` dynamic states set.
- `pRasterizationState` is a pointer to a VkPipelineRasterizationStateCreateInfo structure defining

rasterization state. If the VK_EXT_extended_dynamic_state3 extension is enabled, it **can** be NULL if the pipeline is created with all of VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT, VK_DYNAMIC_STATE_RASTERIZER_DISCARD_ENABLE, VK_DYNAMIC_STATE_POLYGON_MODE_EXT, VK_DYNAMIC_STATE_CULL_MODE, VK_DYNAMIC_STATE_FRONT_FACE, VK_DYNAMIC_STATE_DEPTH_BIAS_ENABLE, VK_DYNAMIC_STATE_DEPTH_BIAS, and VK_DYNAMIC_STATE_LINE_WIDTH dynamic states set.

- pMultisampleState is a pointer to a VkPipelineMultisampleStateCreateInfo structure defining multisample state used when rasterization is enabled. If the VK_EXT_extended_dynamic_state3 extension is enabled, it **can** be NULL if the pipeline is created with all of VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT, VK_DYNAMIC_STATE_SAMPLE_MASK_EXT, and VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic states set, and either the alphaToOne feature is not enabled or VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT is set, in which case VkPipelineMultisampleStateCreateInfo::sampleShadingEnable is assumed to be VK_FALSE.
- pDepthStencilState is a pointer to a VkPipelineDepthStencilStateCreateInfo structure defining depth/stencil state used when rasterization is enabled for depth or stencil attachments accessed during rendering. If the VK_EXT_extended_dynamic_state3 extension is enabled, it **can** be NULL if the pipeline is created with all of VK_DYNAMIC_STATE_DEPTH_TEST_ENABLE, VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE, VK_DYNAMIC_STATE_DEPTH_COMPARE_OP, VK_DYNAMIC_STATE_DEPTH_BOUNDS_TEST_ENABLE, VK_DYNAMIC_STATE_STENCIL_TEST_ENABLE, VK_DYNAMIC_STATE_STENCIL_OP, and VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic states set.
- pColorBlendState is a pointer to a VkPipelineColorBlendStateCreateInfo structure defining color blend state used when rasterization is enabled for any color attachments accessed during rendering. If the VK_EXT_extended_dynamic_state3 extension is enabled, it **can** be NULL if the pipeline is created with all of VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT, VK_DYNAMIC_STATE_LOGIC_OP_EXT, VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT, VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT, VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic states set.
- pDynamicState is a pointer to a VkPipelineDynamicStateCreateInfo structure defining which properties of the pipeline state object are dynamic and **can** be changed independently of the pipeline state. This **can** be NULL, which means no state in the pipeline is considered dynamic.
- layout is the description of binding locations used by both the pipeline and descriptor sets used with the pipeline.
- renderPass is a handle to a render pass object describing the environment in which the pipeline will be used. The pipeline **must** only be used with a render pass instance compatible with the one provided. See Render Pass Compatibility for more information.
- subpass is the index of the subpass in the render pass where this pipeline will be used.
- basePipelineHandle is a pipeline to derive from.
- basePipelineIndex is an index into the pCreateInfos parameter to use as a pipeline to derive from.

The parameters basePipelineHandle and basePipelineIndex are described in more detail in Pipeline Derivatives.

NOTE	With VK_EXT_extended_dynamic_state3, it is possible that many of the VkGraphicsPipelineCreateInfo members above **can** be NULL because all their state is

> dynamic and therefore ignored. This is optional so the application **can** still use a valid pointer if it needs to set the `pNext` or `flags` fields to specify state for other extensions.

The state required for a graphics pipeline is divided into vertex input state, pre-rasterization shader state, fragment shader state, and fragment output state.

Vertex Input State

Vertex input state is defined by:

- VkPipelineVertexInputStateCreateInfo
- VkPipelineInputAssemblyStateCreateInfo

This state **must** be specified to create a complete graphics pipeline.

Pre-Rasterization Shader State

Pre-rasterization shader state is defined by:

- VkPipelineShaderStageCreateInfo entries for:
  - Vertex shaders
  - Tessellation control shaders
  - Tessellation evaluation shaders
  - Geometry shaders
- Within the VkPipelineLayout, the full pipeline layout **must** be specified.
- VkPipelineViewportStateCreateInfo
- VkPipelineRasterizationStateCreateInfo
- VkPipelineTessellationStateCreateInfo
- VkRenderPass and `subpass` parameter
- The `viewMask` parameter of VkPipelineRenderingCreateInfo (formats are ignored)
- VkPipelineDiscardRectangleStateCreateInfoEXT
- VkPipelineFragmentShadingRateStateCreateInfoKHR

This state **must** be specified to create a complete graphics pipeline.

Fragment Shader State

Fragment shader state is defined by:

- A VkPipelineShaderStageCreateInfo entry for the fragment shader
- Within the VkPipelineLayout, the full pipeline layout **must** be specified.
- VkPipelineMultisampleStateCreateInfo if sample shading is enabled or `renderpass` is not VK_NULL_HANDLE
- VkPipelineDepthStencilStateCreateInfo

- VkRenderPass and `subpass` parameter
- The `viewMask` parameter of VkPipelineRenderingCreateInfo (formats are ignored)
- VkPipelineFragmentShadingRateStateCreateInfoKHR
- Inclusion/omission of the `VK_PIPELINE_RASTERIZATION_STATE_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR` flag
- Inclusion/omission of the `VK_PIPELINE_RASTERIZATION_STATE_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT` flag
- VkRenderingInputAttachmentIndexInfoKHR

If `rasterizerDiscardEnable` is `VK_FALSE` this state **must** be specified to create a complete graphics pipeline.

Fragment Output State

Fragment output state is defined by:

- VkPipelineColorBlendStateCreateInfo
- VkRenderPass and `subpass` parameter
- VkPipelineMultisampleStateCreateInfo
- VkPipelineRenderingCreateInfo
- Inclusion/omission of the `VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT` and `VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT` flags
- VkRenderingAttachmentLocationInfoKHR

If `rasterizerDiscardEnable` is `VK_FALSE` this state **must** be specified to create a complete graphics pipeline.

Dynamic State

Dynamic state values set via `pDynamicState` **must** be ignored if the state they correspond to is not otherwise statically set by one of the state subsets used to create the pipeline. For example, if a pipeline only included pre-rasterization shader state, then any dynamic state value corresponding to depth or stencil testing has no effect.

Complete Graphics Pipelines

A complete graphics pipeline always includes pre-rasterization shader state, with other subsets included depending on that state as specified in the above sections.

If the `pNext` chain includes a VkPipelineCreateFlags2CreateInfoKHR structure, VkPipelineCreateFlags2CreateInfoKHR::`flags` from that structure is used instead of `flags` from this structure.

Valid Usage

- VUID-VkGraphicsPipelineCreateInfo-None-09497
  If the `pNext` chain does not include a VkPipelineCreateFlags2CreateInfoKHR structure,

`flags` **must** be a valid combination of VkPipelineCreateFlagBits values

- VUID-VkGraphicsPipelineCreateInfo-flags-07984
  If `flags` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, and `basePipelineIndex` is -1, `basePipelineHandle` **must** be a valid graphics `VkPipeline` handle
- VUID-VkGraphicsPipelineCreateInfo-flags-07985
  If `flags` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, and `basePipelineHandle` is VK_NULL_HANDLE, `basePipelineIndex` **must** be a valid index into the calling command's `pCreateInfos` parameter
- VUID-VkGraphicsPipelineCreateInfo-flags-07986
  If `flags` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, `basePipelineIndex` **must** be -1 or `basePipelineHandle` **must** be VK_NULL_HANDLE
- VUID-VkGraphicsPipelineCreateInfo-layout-07987
  If a push constant block is declared in a shader, a push constant range in `layout` **must** match the shader stage
- VUID-VkGraphicsPipelineCreateInfo-layout-10069
  If a push constant block is declared in a shader, the block must be contained inside the push constant range in `layout` that matches the stage
- VUID-VkGraphicsPipelineCreateInfo-layout-07988
  If a resource variables is declared in a shader, a descriptor slot in `layout` **must** match the shader stage
- VUID-VkGraphicsPipelineCreateInfo-layout-07990
  If a resource variables is declared in a shader, a descriptor slot in `layout` **must** match the descriptor type
- VUID-VkGraphicsPipelineCreateInfo-layout-07991
  If a resource variables is declared in a shader as an array, a descriptor slot in `layout` **must** match the descriptor count
- VUID-VkGraphicsPipelineCreateInfo-stage-02096
  If the pipeline requires pre-rasterization shader state the `stage` member of one element of `pStages` **must** be `VK_SHADER_STAGE_VERTEX_BIT`
- VUID-VkGraphicsPipelineCreateInfo-pStages-00729
  If the pipeline requires pre-rasterization shader state and `pStages` includes a tessellation control shader stage, it **must** include a tessellation evaluation shader stage
- VUID-VkGraphicsPipelineCreateInfo-pStages-00730
  If the pipeline requires pre-rasterization shader state and `pStages` includes a tessellation evaluation shader stage, it **must** include a tessellation control shader stage
- VUID-VkGraphicsPipelineCreateInfo-pStages-09022
  If the pipeline requires pre-rasterization shader state and `pStages` includes a tessellation control shader stage, and the `VK_EXT_extended_dynamic_state3` extension is not enabled or the `VK_DYNAMIC_STATE_PATCH_CONTROL_POINTS_EXT` dynamic state is not set, `pTessellationState` **must** be a valid pointer to a valid VkPipelineTessellationStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pTessellationState-09023
  If `pTessellationState` is not `NULL` it **must** be a pointer to a valid

VkPipelineTessellationStateCreateInfo structure

- VUID-VkGraphicsPipelineCreateInfo-pStages-00732
  If the pipeline requires pre-rasterization shader state and pStages includes tessellation shader stages, the shader code of at least one stage **must** contain an OpExecutionMode instruction specifying the type of subdivision in the pipeline
- VUID-VkGraphicsPipelineCreateInfo-pStages-00733
  If the pipeline requires pre-rasterization shader state and pStages includes tessellation shader stages, and the shader code of both stages contain an OpExecutionMode instruction specifying the type of subdivision in the pipeline, they **must** both specify the same subdivision mode
- VUID-VkGraphicsPipelineCreateInfo-pStages-00734
  If the pipeline requires pre-rasterization shader state and pStages includes tessellation shader stages, the shader code of at least one stage **must** contain an OpExecutionMode instruction specifying the output patch size in the pipeline
- VUID-VkGraphicsPipelineCreateInfo-pStages-00735
  If the pipeline requires pre-rasterization shader state and pStages includes tessellation shader stages, and the shader code of both contain an OpExecutionMode instruction specifying the out patch size in the pipeline, they **must** both specify the same patch size
- VUID-VkGraphicsPipelineCreateInfo-pStages-08888
  If the pipeline is being created with pre-rasterization shader state and vertex input state and pStages includes tessellation shader stages, and either VK_DYNAMIC_STATE_PRIMITIVE_TOPOLOGY dynamic state is not enabled or dynamicPrimitiveTopologyUnrestricted is VK_FALSE, the topology member of pInputAssembly **must** be VK_PRIMITIVE_TOPOLOGY_PATCH_LIST
- VUID-VkGraphicsPipelineCreateInfo-topology-08889
  If the pipeline is being created with pre-rasterization shader state and vertex input state and the topology member of pInputAssembly is VK_PRIMITIVE_TOPOLOGY_PATCH_LIST, and either VK_DYNAMIC_STATE_PRIMITIVE_TOPOLOGY dynamic state is not enabled or dynamicPrimitiveTopologyUnrestricted is VK_FALSE, then pStages **must** include tessellation shader stages
- VUID-VkGraphicsPipelineCreateInfo-TessellationEvaluation-07723
  If the pipeline is being created with a TessellationEvaluation Execution Model, no Geometry Execution Model, uses the PointMode Execution Mode, and the shaderTessellationAndGeometryPointSize feature is enabled, a PointSize decorated variable **must** be written to if the maintenance5 feature is not enabled
- VUID-VkGraphicsPipelineCreateInfo-topology-08773
  If the pipeline is being created with a Vertex Execution Model and no TessellationEvaluation or Geometry Execution Model, and the topology member of pInputAssembly is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, and either VK_DYNAMIC_STATE_PRIMITIVE_TOPOLOGY dynamic state is not enabled or dynamicPrimitiveTopologyUnrestricted is VK_FALSE, a PointSize decorated variable **must** be written to if the maintenance5 feature is not enabled
- VUID-VkGraphicsPipelineCreateInfo-maintenance5-08775
  If the maintenance5 feature is enabled and a PointSize decorated variable is written to, all

execution paths **must** write to a PointSize decorated variable

- VUID-VkGraphicsPipelineCreateInfo-TessellationEvaluation-07724
  If the pipeline is being created with a TessellationEvaluation Execution Model, no Geometry Execution Model, uses the PointMode Execution Mode, and the shaderTessellationAndGeometryPointSize feature is not enabled, a PointSize decorated variable **must** not be written to
- VUID-VkGraphicsPipelineCreateInfo-shaderTessellationAndGeometryPointSize-08776
  If the pipeline is being created with a Geometry Execution Model, uses the OutputPoints Execution Mode, and the shaderTessellationAndGeometryPointSize feature is enabled, a PointSize decorated variable **must** be written to for every vertex emitted if the maintenance5 feature is not enabled
- VUID-VkGraphicsPipelineCreateInfo-Geometry-07726
  If the pipeline is being created with a Geometry Execution Model, uses the OutputPoints Execution Mode, and the shaderTessellationAndGeometryPointSize feature is not enabled, a PointSize decorated variable **must** not be written to
- VUID-VkGraphicsPipelineCreateInfo-pStages-00738
  If the pipeline requires pre-rasterization shader state and pStages includes a geometry shader stage, and does not include any tessellation shader stages, its shader code **must** contain an OpExecutionMode instruction specifying an input primitive type that is compatible with the primitive topology specified in pInputAssembly
- VUID-VkGraphicsPipelineCreateInfo-pStages-00739
  If the pipeline requires pre-rasterization shader state and pStages includes a geometry shader stage, and also includes tessellation shader stages, its shader code **must** contain an OpExecutionMode instruction specifying an input primitive type that is compatible with the primitive topology that is output by the tessellation stages
- VUID-VkGraphicsPipelineCreateInfo-pStages-00740
  If the pipeline requires pre-rasterization shader state and fragment shader state, it includes both a fragment shader and a geometry shader, and the fragment shader code reads from an input variable that is decorated with PrimitiveId, then the geometry shader code **must** write to a matching output variable, decorated with PrimitiveId, in all execution paths
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06038
  If renderPass is not VK_NULL_HANDLE and the pipeline is being created with fragment shader state the fragment shader **must** not read from any input attachment that is defined as VK_ATTACHMENT_UNUSED in subpass
- VUID-VkGraphicsPipelineCreateInfo-pStages-00742
  If the pipeline requires pre-rasterization shader state and multiple pre-rasterization shader stages are included in pStages, the shader code for the entry points identified by those pStages and the rest of the state identified by this structure **must** adhere to the pipeline linking rules described in the Shader Interfaces chapter
- VUID-VkGraphicsPipelineCreateInfo-None-04889
  If the pipeline requires pre-rasterization shader state and fragment shader state, the fragment shader and last pre-rasterization shader stage and any relevant state **must** adhere to the pipeline linking rules described in the Shader Interfaces chapter

- VUID-VkGraphicsPipelineCreateInfo-renderPass-06041
  If `renderPass` is not `VK_NULL_HANDLE`, and the pipeline is being created with fragment output interface state, then for each color attachment in the subpass, if the potential format features of the format of the corresponding attachment description do not contain `VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT`, then the `blendEnable` member of the corresponding element of the `pAttachments` member of `pColorBlendState` **must** be `VK_FALSE`
- VUID-VkGraphicsPipelineCreateInfo-renderPass-07609
  If `renderPass` is not `VK_NULL_HANDLE`, the pipeline is being created with fragment output interface state, the `pColorBlendState` pointer is not `NULL`, the `attachmentCount` member of `pColorBlendState` is not ignored, and the subpass uses color attachments, the `attachmentCount` member of `pColorBlendState` **must** be equal to the `colorAttachmentCount` used to create `subpass`
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-04130
  If the pipeline requires pre-rasterization shader state, and `pViewportState->pViewports` is not dynamic, then `pViewportState->pViewports` **must** be a valid pointer to an array of `pViewportState->viewportCount` valid `VkViewport` structures
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-04131
  If the pipeline requires pre-rasterization shader state, and `pViewportState->pScissors` is not dynamic, then `pViewportState->pScissors` **must** be a valid pointer to an array of `pViewportState->scissorCount` `VkRect2D` structures
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-00749
  If the pipeline requires pre-rasterization shader state, and the `wideLines` feature is not enabled, and no element of the `pDynamicStates` member of `pDynamicState` is `VK_DYNAMIC_STATE_LINE_WIDTH`, the `lineWidth` member of `pRasterizationState` **must** be `1.0`
- VUID-VkGraphicsPipelineCreateInfo-rasterizerDiscardEnable-09024
  If the pipeline requires pre-rasterization shader state, and the `rasterizerDiscardEnable` member of `pRasterizationState` is `VK_FALSE`, and related dynamic state is not set, `pViewportState` **must** be a valid pointer to a valid VkPipelineViewportStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pViewportState-09025
  If `pViewportState` is not `NULL` it **must** be a valid pointer to a valid VkPipelineViewportStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pMultisampleState-09026
  If the pipeline requires fragment output interface state, and the `VK_EXT_extended_dynamic_state3` extension is not enabled or any of the `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT`, `VK_DYNAMIC_STATE_SAMPLE_MASK_EXT`, or `VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT` dynamic states is not set, or the alphaToOne feature is enabled and `VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT` is not set, `pMultisampleState` **must** be a valid pointer to a valid VkPipelineMultisampleStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pMultisampleState-09027
  If `pMultisampleState` is not `NULL` it **must** be a valid pointer to a valid VkPipelineMultisampleStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-alphaToCoverageEnable-08891

If the pipeline is being created with fragment shader state, the VkPipelineMultisampleStateCreateInfo::alphaToCoverageEnable is not ignored and is VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0

- VUID-VkGraphicsPipelineCreateInfo-renderPass-09028
  If renderPass is not VK_NULL_HANDLE, the pipeline is being created with fragment shader state, and subpass uses a depth/stencil attachment, and related dynamic state is not set, pDepthStencilState **must** be a valid pointer to a valid VkPipelineDepthStencilStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pDepthStencilState-09029
  If pDepthStencilState is not NULL it **must** be a valid pointer to a valid VkPipelineDepthStencilStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-renderPass-09030
  If renderPass is not VK_NULL_HANDLE, the pipeline is being created with fragment output interface state, and subpass uses color attachments, and related dynamic state is not set, pColorBlendState **must** be a valid pointer to a valid VkPipelineColorBlendStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-00754
  If the pipeline requires pre-rasterization shader state, the depthBiasClamp feature is not enabled, no element of the pDynamicStates member of pDynamicState is VK_DYNAMIC_STATE_DEPTH_BIAS, and the depthBiasEnable member of pRasterizationState is VK_TRUE, the depthBiasClamp member of pRasterizationState **must** be 0.0
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-02510
  If the pipeline requires fragment shader state, the VK_EXT_depth_range_unrestricted extension is not enabled and no element of the pDynamicStates member of pDynamicState is VK_DYNAMIC_STATE_DEPTH_BOUNDS, and the depthBoundsTestEnable member of pDepthStencilState is VK_TRUE, the minDepthBounds and maxDepthBounds members of pDepthStencilState **must** be between 0.0 and 1.0, inclusive
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-07610
  If the pipeline requires fragment shader state or fragment output interface state, and rasterizationSamples and sampleLocationsInfo are not dynamic, and VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable included in the pNext chain of pMultisampleState is VK_TRUE, sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-07611
  If the pipeline requires fragment shader state or fragment output interface state, and rasterizationSamples and sampleLocationsInfo are not dynamic, and VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable the included in the pNext chain of pMultisampleState is VK_TRUE or VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT is used, sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by

vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples

- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-07612
  If the pipeline requires fragment shader state or fragment output interface state, and rasterizationSamples and sampleLocationsInfo are not dynamic, and VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable included in the pNext chain of pMultisampleState is VK_TRUE or VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT is used, sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples
- VUID-VkGraphicsPipelineCreateInfo-sampleLocationsEnable-01524
  If the pipeline requires fragment shader state, and the sampleLocationsEnable member of a VkPipelineSampleLocationsStateCreateInfoEXT structure included in the pNext chain of pMultisampleState is VK_TRUE, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-VkGraphicsPipelineCreateInfo-subpass-00758
  If the pipeline requires fragment output interface state, rasterizationSamples is not dynamic, and subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples member of pMultisampleState **must** follow the rules for a zero-attachment subpass
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06046
  If renderPass is not VK_NULL_HANDLE, subpass **must** be a valid subpass within renderPass
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06047
  If renderPass is not VK_NULL_HANDLE, the pipeline is being created with pre-rasterization shader state, subpass viewMask is not 0, and multiviewTessellationShader is not enabled, then pStages **must** not include tessellation shaders
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06048
  If renderPass is not VK_NULL_HANDLE, the pipeline is being created with pre-rasterization shader state, subpass viewMask is not 0, and multiviewGeometryShader is not enabled, then pStages **must** not include a geometry shader
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06050
  If renderPass is not VK_NULL_HANDLE and the pipeline is being created with pre-rasterization shader state, and subpass viewMask is not 0, then all of the shaders in the pipeline **must** not include variables decorated with the Layer built-in decoration in their interfaces
- VUID-VkGraphicsPipelineCreateInfo-flags-00764
  flags **must** not contain the VK_PIPELINE_CREATE_DISPATCH_BASE flag
- VUID-VkGraphicsPipelineCreateInfo-pStages-01565
  If the pipeline requires fragment shader state and an input attachment was referenced by an aspectMask at renderPass creation time, the fragment shader **must** only read from the aspects that were specified for that input attachment
- VUID-VkGraphicsPipelineCreateInfo-layout-01688
  The number of resources in layout accessible to each shader stage that is used by the pipeline **must** be less than or equal to VkPhysicalDeviceLimits::maxPerStageResources

- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-04058
  If the pipeline requires pre-rasterization shader state, and no element of the pDynamicStates member of pDynamicState is VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT, and if pNext chain includes a VkPipelineDiscardRectangleStateCreateInfoEXT structure, and if its discardRectangleCount member is not 0, then its pDiscardRectangles member **must** be a valid pointer to an array of discardRectangleCount VkRect2D structures
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-07855
  If VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT is included in the pDynamicStates array then the implementation **must** support at least specVersion 2 of the VK_EXT_discard_rectangles extension
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-07856
  If VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT is included in the pDynamicStates array then the implementation **must** support at least specVersion 2 of the VK_EXT_discard_rectangles extension
- VUID-VkGraphicsPipelineCreateInfo-pStages-02097
  If the pipeline requires vertex input state, and pVertexInputState is not dynamic, then pVertexInputState **must** be a valid pointer to a valid VkPipelineVertexInputStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-Input-07904
  If the pipeline is being created with vertex input state and pVertexInputState is not dynamic, then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription::location
- VUID-VkGraphicsPipelineCreateInfo-Input-08733
  If the pipeline requires vertex input state and pVertexInputState is not dynamic, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription::format
- VUID-VkGraphicsPipelineCreateInfo-pVertexInputState-08929
  If the pipeline is being created with vertex input state and pVertexInputState is not dynamic, and VkVertexInputAttributeDescription::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit
- VUID-VkGraphicsPipelineCreateInfo-pVertexInputState-08930
  If the pipeline is being created with vertex input state and pVertexInputState is not dynamic, and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription::format **must** have a 64-bit component
- VUID-VkGraphicsPipelineCreateInfo-pVertexInputState-09198
  If the pipeline is being created with vertex input state and pVertexInputState is not dynamic, and VkVertexInputAttributeDescription::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format
- VUID-VkGraphicsPipelineCreateInfo-dynamicPrimitiveTopologyUnrestricted-09031

If the pipeline requires vertex input state, and related dynamic state is not set, pInputAssemblyState **must** be a valid pointer to a valid VkPipelineInputAssemblyStateCreateInfo structure

- VUID-VkGraphicsPipelineCreateInfo-pInputAssemblyState-09032
  If pInputAssemblyState is not NULL it **must** be a valid pointer to a valid VkPipelineInputAssemblyStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pStages-02317
  If the pipeline requires pre-rasterization shader state, the Xfb execution mode **can** be specified by no more than one shader stage in pStages
- VUID-VkGraphicsPipelineCreateInfo-pStages-02318
  If the pipeline requires pre-rasterization shader state, and any shader stage in pStages specifies Xfb execution mode it **must** be the last pre-rasterization shader stage
- VUID-VkGraphicsPipelineCreateInfo-rasterizationStream-02319
  If the pipeline requires pre-rasterization shader state, and a VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream value other than zero is specified, all variables in the output interface of the entry point being compiled decorated with Position, PointSize, ClipDistance, or CullDistance **must** be decorated with identical Stream values that match the rasterizationStream
- VUID-VkGraphicsPipelineCreateInfo-rasterizationStream-02320
  If the pipeline requires pre-rasterization shader state, and VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream is zero, or not specified, all variables in the output interface of the entry point being compiled decorated with Position, PointSize, ClipDistance, or CullDistance **must** be decorated with a Stream value of zero, or **must** not specify the Stream decoration
- VUID-VkGraphicsPipelineCreateInfo-geometryStreams-02321
  If the pipeline requires pre-rasterization shader state, and the last pre-rasterization shader stage is a geometry shader, and that geometry shader uses the GeometryStreams capability, then VkPhysicalDeviceTransformFeedbackFeaturesEXT::geometryStreams feature **must** be enabled
- VUID-VkGraphicsPipelineCreateInfo-lineRasterizationMode-02766
  If the pipeline requires pre-rasterization shader state and at least one of fragment output interface state or fragment shader state, and pMultisampleState is not NULL, the lineRasterizationMode member of a VkPipelineRasterizationLineStateCreateInfoKHR structure included in the pNext chain of pRasterizationState is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR or VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the alphaToCoverageEnable, alphaToOneEnable, and sampleShadingEnable members of pMultisampleState **must** all be VK_FALSE
- VUID-VkGraphicsPipelineCreateInfo-stippledLineEnable-02767
  If the pipeline requires pre-rasterization shader state, the stippledLineEnable member of VkPipelineRasterizationLineStateCreateInfoKHR is VK_TRUE, and no element of the pDynamicStates member of pDynamicState is VK_DYNAMIC_STATE_LINE_STIPPLE_EXT, then the lineStippleFactor member of VkPipelineRasterizationLineStateCreateInfoKHR **must** be in the range [1,256]

- VUID-VkGraphicsPipelineCreateInfo-flags-03372
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR`
- VUID-VkGraphicsPipelineCreateInfo-flags-03373
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR`
- VUID-VkGraphicsPipelineCreateInfo-flags-03374
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR`
- VUID-VkGraphicsPipelineCreateInfo-flags-03375
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR`
- VUID-VkGraphicsPipelineCreateInfo-flags-03376
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR`
- VUID-VkGraphicsPipelineCreateInfo-flags-03377
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR`
- VUID-VkGraphicsPipelineCreateInfo-flags-03577
  `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR`
- VUID-VkGraphicsPipelineCreateInfo-flags-11000
  If `flags` includes `VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT`, then the `VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT::deviceGeneratedCommands` feature **must** be enabled
- VUID-VkGraphicsPipelineCreateInfo-flags-11001
  If the pipeline requires pre-rasterization shader state and `flags` includes `VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT`, then all stages **must** not specify `Xfb` execution mode
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04494
  If the pipeline requires pre-rasterization shader state or fragment shader state and `VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR` is not included in `pDynamicState->pDynamicStates`, `VkPipelineFragmentShadingRateStateCreateInfoKHR::fragmentSize.width` **must** be greater than or equal to `1`
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04495
  If the pipeline requires pre-rasterization shader state or fragment shader state and `VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR` is not included in `pDynamicState->pDynamicStates`, `VkPipelineFragmentShadingRateStateCreateInfoKHR::fragmentSize.height` **must** be greater than or equal to `1`
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04496
  If the pipeline requires pre-rasterization shader state or fragment shader state and `VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR` is not included in `pDynamicState->pDynamicStates`, `VkPipelineFragmentShadingRateStateCreateInfoKHR::fragmentSize.width` **must** be a power-of-two value
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04497
  If the pipeline requires pre-rasterization shader state or fragment shader state and `VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR` is not included in `pDynamicState-`

>pDynamicStates, VkPipelineFragmentShadingRateStateCreateInfoKHR::fragmentSize.height **must** be a power-of-two value

- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04498
  If the pipeline requires pre-rasterization shader state or fragment shader state and VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR is not included in pDynamicState->pDynamicStates, VkPipelineFragmentShadingRateStateCreateInfoKHR::fragmentSize.width **must** be less than or equal to 4
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04499
  If the pipeline requires pre-rasterization shader state or fragment shader state and VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR is not included in pDynamicState->pDynamicStates, VkPipelineFragmentShadingRateStateCreateInfoKHR::fragmentSize.height **must** be less than or equal to 4
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04500
  If the pipeline requires pre-rasterization shader state or fragment shader state and VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR is not included in pDynamicState->pDynamicStates, and the pipelineFragmentShadingRate feature is not enabled, VkPipelineFragmentShadingRateStateCreateInfoKHR::fragmentSize.width and VkPipelineFragmentShadingRateStateCreateInfoKHR::fragmentSize.height **must** both be equal to 1
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-06567
  If the pipeline requires pre-rasterization shader state or fragment shader state and VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR is not included in pDynamicState->pDynamicStates, VkPipelineFragmentShadingRateStateCreateInfoKHR::combinerOps[0] **must** be a valid VkFragmentShadingRateCombinerOpKHR value
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-06568
  If the pipeline requires pre-rasterization shader state or fragment shader state and VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR is not included in pDynamicState->pDynamicStates, VkPipelineFragmentShadingRateStateCreateInfoKHR::combinerOps[1] **must** be a valid VkFragmentShadingRateCombinerOpKHR value
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04501
  If the pipeline requires pre-rasterization shader state or fragment shader state and VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR is not included in pDynamicState->pDynamicStates, and the primitiveFragmentShadingRate feature is not enabled, VkPipelineFragmentShadingRateStateCreateInfoKHR::combinerOps[0] **must** be VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-04502
  If the pipeline requires pre-rasterization shader state or fragment shader state and VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR is not included in pDynamicState->pDynamicStates, and the attachmentFragmentShadingRate feature is not enabled, VkPipelineFragmentShadingRateStateCreateInfoKHR::combinerOps[1] **must** be VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR
- VUID-VkGraphicsPipelineCreateInfo-primitiveFragmentShadingRateWithMultipleViewports-04504
  If the pipeline requires pre-rasterization shader state and the primitiveFragmentShadingRateWithMultipleViewports limit is not supported, and entry

points specified in pStages write to the ViewportIndex built-in, they **must** not also write to the PrimitiveShadingRateKHR built-in

- VUID-VkGraphicsPipelineCreateInfo-fragmentShadingRateNonTrivialCombinerOps-04506
  If the pipeline requires pre-rasterization shader state or fragment shader state, the fragmentShadingRateNonTrivialCombinerOps limit is not supported, and VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR is not included in pDynamicState->pDynamicStates, elements of VkPipelineFragmentShadingRateStateCreateInfoKHR::combinerOps **must** be VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR or VK_FRAGMENT_SHADING_RATE_COMBINER_OP_REPLACE_KHR
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-03578
  All elements of the pDynamicStates member of pDynamicState **must** not be VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-04807
  If the pipeline requires pre-rasterization shader state and the vertexInputDynamicState feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_VERTEX_INPUT_EXT
- VUID-VkGraphicsPipelineCreateInfo-pDynamicStates-04800
  If the colorWriteEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT
- VUID-VkGraphicsPipelineCreateInfo-dynamicRendering-06576
  If the dynamicRendering feature is not enabled and the pipeline requires pre-rasterization shader state, fragment shader state, or fragment output interface state, renderPass **must** not be VK_NULL_HANDLE
- VUID-VkGraphicsPipelineCreateInfo-multiview-06577
  If the multiview feature is not enabled, the pipeline requires pre-rasterization shader state, fragment shader state, or fragment output interface state, and renderPass is VK_NULL_HANDLE, VkPipelineRenderingCreateInfo::viewMask **must** be 0
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06578
  If the pipeline requires pre-rasterization shader state, fragment shader state, or fragment output interface state, and renderPass is VK_NULL_HANDLE, the index of the most significant bit in VkPipelineRenderingCreateInfo::viewMask **must** be less than maxMultiviewViewCount
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06579
  If the pipeline requires fragment output interface state, and renderPass is VK_NULL_HANDLE, and VkPipelineRenderingCreateInfo::colorAttachmentCount is not 0, VkPipelineRenderingCreateInfo::pColorAttachmentFormats **must** be a valid pointer to an array of colorAttachmentCount valid VkFormat values
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06580
  If the pipeline requires fragment output interface state, and renderPass is VK_NULL_HANDLE, each element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats **must** be a valid VkFormat value
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06582
  If the pipeline requires fragment output interface state, renderPass is VK_NULL_HANDLE, and any element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats is not

VK_FORMAT_UNDEFINED, that format **must** be a format with potential format features that include VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT

- VUID-VkGraphicsPipelineCreateInfo-renderPass-06583
  If the pipeline requires fragment output interface state, and renderPass is VK_NULL_HANDLE, VkPipelineRenderingCreateInfo::depthAttachmentFormat **must** be a valid VkFormat value
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06584
  If the pipeline requires fragment output interface state, and renderPass is VK_NULL_HANDLE, VkPipelineRenderingCreateInfo::stencilAttachmentFormat **must** be a valid VkFormat value
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06585
  If the pipeline requires fragment output interface state, renderPass is VK_NULL_HANDLE, and VkPipelineRenderingCreateInfo::depthAttachmentFormat is not VK_FORMAT_UNDEFINED, it **must** be a format with potential format features that include VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06586
  If the pipeline requires fragment output interface state, renderPass is VK_NULL_HANDLE, and VkPipelineRenderingCreateInfo::stencilAttachmentFormat is not VK_FORMAT_UNDEFINED, it **must** be a format with potential format features that include VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06587
  If the pipeline requires fragment output interface state, renderPass is VK_NULL_HANDLE, and VkPipelineRenderingCreateInfo::depthAttachmentFormat is not VK_FORMAT_UNDEFINED, it **must** be a format that includes a depth component
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06588
  If the pipeline requires fragment output interface state, renderPass is VK_NULL_HANDLE, and VkPipelineRenderingCreateInfo::stencilAttachmentFormat is not VK_FORMAT_UNDEFINED, it **must** be a format that includes a stencil component
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06589
  If the pipeline requires fragment output interface state, renderPass is VK_NULL_HANDLE, VkPipelineRenderingCreateInfo::depthAttachmentFormat is not VK_FORMAT_UNDEFINED, and VkPipelineRenderingCreateInfo::stencilAttachmentFormat is not VK_FORMAT_UNDEFINED, depthAttachmentFormat **must** equal stencilAttachmentFormat
- VUID-VkGraphicsPipelineCreateInfo-renderPass-09033
  If renderPass is VK_NULL_HANDLE, the pipeline is being created with fragment shader state and fragment output interface state, and either of VkPipelineRenderingCreateInfo ::depthAttachmentFormat or VkPipelineRenderingCreateInfo::stencilAttachmentFormat are not VK_FORMAT_UNDEFINED, and the VK_EXT_extended_dynamic_state3 extension is not enabled or any of the VK_DYNAMIC_STATE_DEPTH_TEST_ENABLE, VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE, VK_DYNAMIC_STATE_DEPTH_COMPARE_OP, VK_DYNAMIC_STATE_DEPTH_BOUNDS_TEST_ENABLE, VK_DYNAMIC_STATE_STENCIL_TEST_ENABLE, VK_DYNAMIC_STATE_STENCIL_OP, or VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic states are not set, pDepthStencilState **must** be a valid pointer to a valid VkPipelineDepthStencilStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pDepthStencilState-09034

If pDepthStencilState is not NULL it **must** be a valid pointer to a valid VkPipelineDepthStencilStateCreateInfo structure

- VUID-VkGraphicsPipelineCreateInfo-renderPass-09037
  If renderPass is VK_NULL_HANDLE, the pipeline is being created with fragment output interface state, and any element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats is not VK_FORMAT_UNDEFINED, and the VK_EXT_extended_dynamic_state3 extension is not enabled, or any of the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT, VK_DYNAMIC_STATE_LOGIC_OP_EXT, VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT, VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT, VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, or VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic states are not set, pColorBlendState **must** be a valid pointer to a valid VkPipelineColorBlendStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pColorBlendState-09038
  If pColorBlendState is not NULL it **must** be a valid pointer to a valid VkPipelineColorBlendStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06055
  If renderPass is VK_NULL_HANDLE, pColorBlendState is not dynamic, and the pipeline is being created with fragment output interface state, pColorBlendState->attachmentCount **must** be equal to VkPipelineRenderingCreateInfo::colorAttachmentCount
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06057
  If renderPass is VK_NULL_HANDLE, the pipeline is being created with pre-rasterization shader state, VkPipelineRenderingCreateInfo::viewMask is not 0, and the multiviewTessellationShader feature is not enabled, then pStages **must** not include tessellation shaders
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06058
  If renderPass is VK_NULL_HANDLE, the pipeline is being created with pre-rasterization shader state, VkPipelineRenderingCreateInfo::viewMask is not 0, and the multiviewGeometryShader feature is not enabled, then pStages **must** not include a geometry shader
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06059
  If renderPass is VK_NULL_HANDLE, the pipeline is being created with pre-rasterization shader state, and VkPipelineRenderingCreateInfo::viewMask is not 0, all of the shaders in the pipeline **must** not include variables decorated with the Layer built-in decoration in their interfaces
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06061
  If the dynamicRenderingLocalRead feature is not enabled, the pipeline requires fragment shader state, and renderPass is VK_NULL_HANDLE, fragment shaders in pStages **must** not include the InputAttachment capability
- VUID-VkGraphicsPipelineCreateInfo-renderPass-08710
  If the pipeline requires fragment shader state and renderPass is not VK_NULL_HANDLE, fragment shaders in pStages **must** not include any of the TileImageColorReadAccessEXT, TileImageDepthReadAccessEXT, or TileImageStencilReadAccessEXT capabilities
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06062
  If the pipeline requires fragment output interface state and renderPass is

VK_NULL_HANDLE, for each color attachment format defined by the pColorAttachmentFormats member of VkPipelineRenderingCreateInfo, if its potential format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE

- VUID-VkGraphicsPipelineCreateInfo-pStages-06600
  If the pipeline requires pre-rasterization shader state or fragment shader state, pStages **must** be a valid pointer to an array of stageCount valid VkPipelineShaderStageCreateInfo structures
- VUID-VkGraphicsPipelineCreateInfo-stageCount-09587
  If the pipeline does not require pre-rasterization shader state or fragment shader state, stageCount **must** be zero
- VUID-VkGraphicsPipelineCreateInfo-pRasterizationState-06601
  If the pipeline requires pre-rasterization shader state, and related dynamic state is not set, pRasterizationState **must** be a valid pointer to a valid VkPipelineRasterizationStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pRasterizationState-09039
  If related dynamic state is not set, then pMultisampleState **must** be a valid pointer to a valid VkPipelineMultisampleStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-pRasterizationState-09040
  If pRasterizationState is not NULL it **must** be a valid pointer to a valid VkPipelineRasterizationStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-layout-06602
  If the pipeline requires fragment shader state or pre-rasterization shader state, layout **must** be a valid VkPipelineLayout handle
- VUID-VkGraphicsPipelineCreateInfo-renderPass-06603
  If the pipeline requires pre-rasterization shader state, fragment shader state, or fragment output state, and renderPass is not VK_NULL_HANDLE, renderPass **must** be a valid VkRenderPass handle
- VUID-VkGraphicsPipelineCreateInfo-stageCount-09530
  If the pipeline requires pre-rasterization shader state, stageCount **must** be greater than 0
- VUID-VkGraphicsPipelineCreateInfo-graphicsPipelineLibrary-06606
  flags **must** not include VK_PIPELINE_CREATE_LIBRARY_BIT_KHR
- VUID-VkGraphicsPipelineCreateInfo-conservativePointAndLineRasterization-08892
  If conservativePointAndLineRasterization is not supported; the pipeline is being created with vertex input state and pre-rasterization shader state; the pipeline does not include a geometry shader; and the value of VkPipelineInputAssemblyStateCreateInfo::topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, or VK_PRIMITIVE_TOPOLOGY_LINE_STRIP, and either VK_DYNAMIC_STATE_PRIMITIVE_TOPOLOGY dynamic state is not enabled or dynamicPrimitiveTopologyUnrestricted is VK_FALSE, then VkPipelineRasterizationConservativeStateCreateInfoEXT::conservativeRasterizationMode **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-VkGraphicsPipelineCreateInfo-conservativePointAndLineRasterization-06760

If conservativePointAndLineRasterization is not supported, the pipeline requires pre-rasterization shader state, and the pipeline includes a geometry shader with either the OutputPoints or OutputLineStrip execution modes, VkPipelineRasterizationConservativeStateCreateInfoEXT::conservativeRasterizationMode **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT

- VUID-VkGraphicsPipelineCreateInfo-pStages-06894
  If the pipeline requires pre-rasterization shader state but not fragment shader state, elements of pStages **must** not have stage set to VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-VkGraphicsPipelineCreateInfo-pStages-06895
  If the pipeline requires fragment shader state but not pre-rasterization shader state, elements of pStages **must** not have stage set to a shader stage which participates in pre-rasterization
- VUID-VkGraphicsPipelineCreateInfo-pStages-06896
  If the pipeline requires pre-rasterization shader state, all elements of pStages **must** have a stage set to a shader stage which participates in fragment shader state or pre-rasterization shader state
- VUID-VkGraphicsPipelineCreateInfo-stage-06897
  If the pipeline requires fragment shader state and/or pre-rasterization shader state, any value of stage **must** not be set in more than one element of pStages
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3TessellationDomainOrigin-07370
  If the extendedDynamicState3TessellationDomainOrigin feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3DepthClampEnable-07371
  If the extendedDynamicState3DepthClampEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3PolygonMode-07372
  If the extendedDynamicState3PolygonMode feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_POLYGON_MODE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3RasterizationSamples-07373
  If the extendedDynamicState3RasterizationSamples feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3SampleMask-07374
  If the extendedDynamicState3SampleMask feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_SAMPLE_MASK_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3AlphaToCoverageEnable-07375
  If the extendedDynamicState3AlphaToCoverageEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT

- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3AlphaToOneEnable-07376
  If the extendedDynamicState3AlphaToOneEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3LogicOpEnable-07377
  If the extendedDynamicState3LogicOpEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ColorBlendEnable-07378
  If the extendedDynamicState3ColorBlendEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ColorBlendEquation-07379
  If the extendedDynamicState3ColorBlendEquation feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ColorWriteMask-07380
  If the extendedDynamicState3ColorWriteMask feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3RasterizationStream-07381
  If the extendedDynamicState3RasterizationStream feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ConservativeRasterizationMode-07382
  If the extendedDynamicState3ConservativeRasterizationMode feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ExtraPrimitiveOverestimationSize-07383
  If the extendedDynamicState3ExtraPrimitiveOverestimationSize feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-09639
  If the pipeline requires pre-rasterization shader state, pDynamicState includes VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT, and pDynamicState does not include VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT, pRasterizationState **must** include a VkPipelineRasterizationConservativeStateCreateInfoEXT in its pNext chain
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3DepthClipEnable-07384
  If the extendedDynamicState3DepthClipEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3SampleLocationsEnable-

07385
If the extendedDynamicState3SampleLocationsEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT

- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ColorBlendAdvanced-07386
If the extendedDynamicState3ColorBlendAdvanced feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ProvokingVertexMode-07387
If the extendedDynamicState3ProvokingVertexMode feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3LineRasterizationMode-07388
If the extendedDynamicState3LineRasterizationMode feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3LineStippleEnable-07389
If the extendedDynamicState3LineStippleEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3DepthClipNegativeOneToOne-07390
If the extendedDynamicState3DepthClipNegativeOneToOne feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ViewportWScalingEnable-07391
If the extendedDynamicState3ViewportWScalingEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_VIEWPORT_W_SCALING_ENABLE_NV
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ViewportSwizzle-07392
If the extendedDynamicState3ViewportSwizzle feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_VIEWPORT_SWIZZLE_NV
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3CoverageToColorEnable-07393
If the extendedDynamicState3CoverageToColorEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COVERAGE_TO_COLOR_ENABLE_NV
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3CoverageToColorLocation-07394
If the extendedDynamicState3CoverageToColorLocation feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to

VK_DYNAMIC_STATE_COVERAGE_TO_COLOR_LOCATION_NV

- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3CoverageModulationMode-07395
  If the extendedDynamicState3CoverageModulationMode feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COVERAGE_MODULATION_MODE_NV
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3CoverageModulationTableEnable-07396
  If the extendedDynamicState3CoverageModulationTableEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COVERAGE_MODULATION_TABLE_ENABLE_NV
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3CoverageModulationTable-07397
  If the extendedDynamicState3CoverageModulationTable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COVERAGE_MODULATION_TABLE_NV
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3CoverageReductionMode-07398
  If the extendedDynamicState3CoverageReductionMode feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_COVERAGE_REDUCTION_MODE_NV
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3RepresentativeFragmentTestEnable-07399
  If the extendedDynamicState3RepresentativeFragmentTestEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_REPRESENTATIVE_FRAGMENT_TEST_ENABLE_NV
- VUID-VkGraphicsPipelineCreateInfo-extendedDynamicState3ShadingRateImageEnable-07400
  If the extendedDynamicState3ShadingRateImageEnable feature is not enabled, there **must** be no element of the pDynamicStates member of pDynamicState set to VK_DYNAMIC_STATE_SHADING_RATE_IMAGE_ENABLE_NV
- VUID-VkGraphicsPipelineCreateInfo-flags-07401
  flags **must** not include VK_PIPELINE_CREATE_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT
- VUID-VkGraphicsPipelineCreateInfo-pStages-08711
  If pStages includes a fragment shader stage, VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE is not set in VkPipelineDynamicStateCreateInfo::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable member of VkPipelineDepthStencilStateCreateInfo **must** be VK_FALSE
- VUID-VkGraphicsPipelineCreateInfo-pStages-08712
  If pStages includes a fragment shader stage, VK_DYNAMIC_STATE_STENCIL_WRITE_MASK is not set in VkPipelineDynamicStateCreateInfo::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the value of VkStencilOpState::writeMask for both front and back in VkPipelineDepthStencilStateCreateInfo **must** be 0

- VUID-VkGraphicsPipelineCreateInfo-renderPass-08744
  If `renderPass` is VK_NULL_HANDLE, the pipeline requires fragment output state or fragment shader state, the pipeline enables sample shading, `rasterizationSamples` is not dynamic, and the `pNext` chain includes a VkPipelineRenderingCreateInfo structure, `rasterizationSamples` **must** be a valid VkSampleCountFlagBits value that is set in `imageCreateSampleCounts` (as defined in Image Creation Limits) for every element of `depthAttachmentFormat`, `stencilAttachmentFormat` and the `pColorAttachmentFormats` array which is not `VK_FORMAT_UNDEFINED`
- VUID-VkGraphicsPipelineCreateInfo-None-08893
  The pipeline **must** be created with pre-rasterization shader state
- VUID-VkGraphicsPipelineCreateInfo-pStages-08894
  If `pStages` includes a vertex shader stage, the pipeline **must** be created with vertex input state
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-08896
  If `pRasterizationState->rasterizerDiscardEnable` is `VK_FALSE`, the pipeline **must** be created with fragment shader state and fragment output interface state
- VUID-VkGraphicsPipelineCreateInfo-None-09043
  If `pDynamicState->pDynamicStates` does not include `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT`, and the format of any color attachment is `VK_FORMAT_E5B9G9R9_UFLOAT_PACK32`, the `colorWriteMask` member of the corresponding element of `pColorBlendState->pAttachments` **must** either include all of `VK_COLOR_COMPONENT_R_BIT`, `VK_COLOR_COMPONENT_G_BIT`, and `VK_COLOR_COMPONENT_B_BIT`, or none of them
- VUID-VkGraphicsPipelineCreateInfo-renderPass-09531
  If the pipeline is being created with fragment shader state and fragment output state, the value of `renderPass` is VK_NULL_HANDLE, and VkRenderingInputAttachmentIndexInfoKHR is included, VkRenderingInputAttachmentIndexInfoKHR::`colorAttachmentCount` **must** be equal to VkPipelineRenderingCreateInfo::`colorAttachmentCount`
- VUID-VkGraphicsPipelineCreateInfo-renderPass-09652
  If the pipeline is being created with fragment shader state and fragment output state, the value of `renderPass` is VK_NULL_HANDLE, and VkRenderingInputAttachmentIndexInfoKHR is not included, the fragment shader **must** not contain any input attachments with a `InputAttachmentIndex` greater than or equal to VkPipelineRenderingCreateInfo::`colorAttachmentCount`
- VUID-VkGraphicsPipelineCreateInfo-renderPass-09532
  If the pipeline is being created with fragment output state, and the value of `renderPass` is VK_NULL_HANDLE, VkRenderingAttachmentLocationInfoKHR::`colorAttachmentCount` **must** be equal to VkPipelineRenderingCreateInfo::`colorAttachmentCount`

Valid Usage (Implicit)

- VUID-VkGraphicsPipelineCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_GRAPHICS_PIPELINE_CREATE_INFO`

- VUID-VkGraphicsPipelineCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkPipelineBinaryInfoKHR, VkPipelineCreateFlags2CreateInfoKHR, VkPipelineDiscardRectangleStateCreateInfoEXT, VkPipelineFragmentShadingRateStateCreateInfoKHR, VkPipelineLibraryCreateInfoKHR, VkPipelineRenderingCreateInfo, VkRenderingAttachmentLocationInfoKHR, or VkRenderingInputAttachmentIndexInfoKHR
- VUID-VkGraphicsPipelineCreateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkGraphicsPipelineCreateInfo-pDynamicState-parameter
  If pDynamicState is not NULL, pDynamicState **must** be a valid pointer to a valid VkPipelineDynamicStateCreateInfo structure
- VUID-VkGraphicsPipelineCreateInfo-commonparent
  Each of basePipelineHandle, layout, and renderPass that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The VkPipelineRenderingCreateInfo structure is defined as:

```
typedef struct VkPipelineRenderingCreateInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           viewMask;
    uint32_t           colorAttachmentCount;
    const VkFormat*    pColorAttachmentFormats;
    VkFormat           depthAttachmentFormat;
    VkFormat           stencilAttachmentFormat;
} VkPipelineRenderingCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_dynamic_rendering
typedef VkPipelineRenderingCreateInfo VkPipelineRenderingCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- viewMask is the viewMask used for rendering.
- colorAttachmentCount is the number of entries in pColorAttachmentFormats
- pColorAttachmentFormats is a pointer to an array of VkFormat values defining the format of color attachments used in this pipeline.
- depthAttachmentFormat is a VkFormat value defining the format of the depth attachment used in this pipeline.
- stencilAttachmentFormat is a VkFormat value defining the format of the stencil attachment used in this pipeline.

When a pipeline is created without a VkRenderPass, if the pNext chain of VkGraphicsPipelineCreateInfo includes this structure, it specifies the view mask and format of attachments used for rendering. If this structure is not specified, and the pipeline does not include a VkRenderPass, viewMask and colorAttachmentCount are 0, and depthAttachmentFormat and stencilAttachmentFormat are VK_FORMAT_UNDEFINED. If a graphics pipeline is created with a valid VkRenderPass, parameters of this structure are ignored.

If depthAttachmentFormat, stencilAttachmentFormat, or any element of pColorAttachmentFormats is VK_FORMAT_UNDEFINED, it indicates that the corresponding attachment is unused within the render pass. Valid formats indicate that an attachment **can** be used - but it is still valid to set the attachment to NULL when beginning rendering.

Valid Usage

- VUID-VkPipelineRenderingCreateInfo-colorAttachmentCount-09533
  colorAttachmentCount **must** be less than or equal to maxColorAttachments

Valid Usage (Implicit)

- VUID-VkPipelineRenderingCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_RENDERING_CREATE_INFO

The VkPipelineCreateFlags2CreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_maintenance5
typedef struct VkPipelineCreateFlags2CreateInfoKHR {
    VkStructureType              sType;
    const void*                  pNext;
    VkPipelineCreateFlags2KHR    flags;
} VkPipelineCreateFlags2CreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkPipelineCreateFlagBits2KHR specifying how a pipeline will be generated.

If this structure is included in the pNext chain of a pipeline creation structure, flags is used instead of the corresponding flags value passed in that creation structure, allowing additional creation flags to be specified.

Valid Usage (Implicit)

- VUID-VkPipelineCreateFlags2CreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_CREATE_FLAGS_2_CREATE_INFO_KHR
- VUID-VkPipelineCreateFlags2CreateInfoKHR-flags-parameter

`flags` **must** be a valid combination of VkPipelineCreateFlagBits2KHR values

- VUID-VkPipelineCreateFlags2CreateInfoKHR-flags-requiredbitmask
  `flags` **must** not be `0`

Bits which **can** be set in VkPipelineCreateFlags2CreateInfoKHR::`flags`, specifying how a pipeline is created, are:

```
// Provided by VK_KHR_maintenance5
// Flag bits for VkPipelineCreateFlagBits2KHR
typedef VkFlags64 VkPipelineCreateFlagBits2KHR;
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_DISABLE_OPTIMIZATION_BIT_KHR = 0x00000001ULL;
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_ALLOW_DERIVATIVES_BIT_KHR = 0x00000002ULL;
static const VkPipelineCreateFlagBits2KHR VK_PIPELINE_CREATE_2_DERIVATIVE_BIT_KHR =
0x00000004ULL;
// Provided by VK_KHR_maintenance5 with VK_VERSION_1_1 or VK_KHR_device_group
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_VIEW_INDEX_FROM_DEVICE_INDEX_BIT_KHR = 0x00000008ULL;
// Provided by VK_KHR_maintenance5 with VK_VERSION_1_1 or VK_KHR_device_group
static const VkPipelineCreateFlagBits2KHR VK_PIPELINE_CREATE_2_DISPATCH_BASE_BIT_KHR =
0x00000010ULL;
// Provided by VK_KHR_maintenance5 with VK_NV_ray_tracing
static const VkPipelineCreateFlagBits2KHR VK_PIPELINE_CREATE_2_DEFER_COMPILE_BIT_NV =
0x00000020ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_pipeline_executable_properties
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_CAPTURE_STATISTICS_BIT_KHR = 0x00000040ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_pipeline_executable_properties
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_CAPTURE_INTERNAL_REPRESENTATIONS_BIT_KHR = 0x00000080ULL;
// Provided by VK_KHR_maintenance5 with VK_VERSION_1_3 or
VK_EXT_pipeline_creation_cache_control
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT_KHR = 0x00000100ULL;
// Provided by VK_KHR_maintenance5 with VK_VERSION_1_3 or
VK_EXT_pipeline_creation_cache_control
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_EARLY_RETURN_ON_FAILURE_BIT_KHR = 0x00000200ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_graphics_pipeline_library
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_LINK_TIME_OPTIMIZATION_BIT_EXT = 0x00000400ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_graphics_pipeline_library
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RETAIN_LINK_TIME_OPTIMIZATION_INFO_BIT_EXT = 0x00800000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_pipeline_library
static const VkPipelineCreateFlagBits2KHR VK_PIPELINE_CREATE_2_LIBRARY_BIT_KHR =
0x00000800ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_ray_tracing_pipeline
```

```
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR = 0x00001000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_ray_tracing_pipeline
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_SKIP_AABBS_BIT_KHR = 0x00002000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_ray_tracing_pipeline
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR = 0x00004000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_ray_tracing_pipeline
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR = 0x00008000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_ray_tracing_pipeline
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR = 0x00010000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_ray_tracing_pipeline
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR = 0x00020000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_ray_tracing_pipeline
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR =
0x00080000ULL;
// Provided by VK_KHR_maintenance5 with VK_NV_device_generated_commands
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_NV = 0x00040000ULL;
// Provided by VK_KHR_maintenance5 with VK_NV_ray_tracing_motion_blur
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_ALLOW_MOTION_BIT_NV = 0x00100000ULL;
// Provided by VK_KHR_maintenance5 with (VK_KHR_dynamic_rendering or VK_VERSION_1_3)
and VK_KHR_fragment_shading_rate
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR =
0x00200000ULL;
// Provided by VK_KHR_maintenance5 with (VK_KHR_dynamic_rendering or VK_VERSION_1_3)
and VK_EXT_fragment_density_map
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT =
0x00400000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_opacity_micromap
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT = 0x01000000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_attachment_feedback_loop_layout
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT = 0x02000000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_attachment_feedback_loop_layout
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT = 0x04000000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_pipeline_protected_access
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_NO_PROTECTED_ACCESS_BIT_EXT = 0x08000000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_pipeline_protected_access
static const VkPipelineCreateFlagBits2KHR
```

```
VK_PIPELINE_CREATE_2_PROTECTED_ACCESS_ONLY_BIT_EXT = 0x40000000ULL;
// Provided by VK_KHR_maintenance5 with VK_NV_displacement_micromap
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_RAY_TRACING_DISPLACEMENT_MICROMAP_BIT_NV = 0x10000000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_descriptor_buffer
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_DESCRIPTOR_BUFFER_BIT_EXT = 0x20000000ULL;
// Provided by VK_KHR_pipeline_binary
static const VkPipelineCreateFlagBits2KHR VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR =
0x80000000ULL;
// Provided by VK_EXT_device_generated_commands
static const VkPipelineCreateFlagBits2KHR
VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT = 0x4000000000ULL;
```

- VK_PIPELINE_CREATE_2_DISABLE_OPTIMIZATION_BIT_KHR specifies that the created pipeline will not be optimized. Using this flag **may** reduce the time taken to create the pipeline.
- VK_PIPELINE_CREATE_2_ALLOW_DERIVATIVES_BIT_KHR specifies that the pipeline to be created is allowed to be the parent of a pipeline that will be created in a subsequent pipeline creation call.
- VK_PIPELINE_CREATE_2_DERIVATIVE_BIT_KHR specifies that the pipeline to be created will be a child of a previously created parent pipeline.
- VK_PIPELINE_CREATE_2_VIEW_INDEX_FROM_DEVICE_INDEX_BIT_KHR specifies that any shader input variables decorated as ViewIndex will be assigned values as if they were decorated as DeviceIndex.
- VK_PIPELINE_CREATE_2_DISPATCH_BASE_BIT_KHR specifies that a compute pipeline **can** be used with vkCmdDispatchBaseKHR with a non-zero base workgroup.
- VK_PIPELINE_CREATE_2_CAPTURE_STATISTICS_BIT_KHR specifies that the shader compiler should capture statistics for the pipeline executables produced by the compile process which **can** later be retrieved by calling vkGetPipelineExecutableStatisticsKHR. Enabling this flag **must** not affect the final compiled pipeline but **may** disable pipeline caching or otherwise affect pipeline creation time.
- VK_PIPELINE_CREATE_2_CAPTURE_INTERNAL_REPRESENTATIONS_BIT_KHR specifies that the shader compiler should capture the internal representations of pipeline executables produced by the compile process which **can** later be retrieved by calling vkGetPipelineExecutableInternalRepresentationsKHR. Enabling this flag **must** not affect the final compiled pipeline but **may** disable pipeline caching or otherwise affect pipeline creation time. When capturing IR from pipelines created with pipeline libraries, there is no guarantee that IR from libraries **can** be retrieved from the linked pipeline. Applications **should** retrieve IR from each library, and any linked pipelines, separately.
- VK_PIPELINE_CREATE_2_LIBRARY_BIT_KHR specifies that the pipeline **cannot** be used directly, and instead defines a *pipeline library* that **can** be combined with other pipelines using the VkPipelineLibraryCreateInfoKHR structure. This is available in ray tracing pipelines.
- VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR specifies that an any-hit shader will always be present when an any-hit shader would be executed. A NULL any-hit shader is an any-hit shader which is effectively VK_SHADER_UNUSED_KHR, such as from a shader group consisting entirely of zeros.

- VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR specifies that a closest hit shader will always be present when a closest hit shader would be executed. A NULL closest hit shader is a closest hit shader which is effectively VK_SHADER_UNUSED_KHR, such as from a shader group consisting entirely of zeros.
- VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR specifies that a miss shader will always be present when a miss shader would be executed. A NULL miss shader is a miss shader which is effectively VK_SHADER_UNUSED_KHR, such as from a shader group consisting entirely of zeros.
- VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR specifies that an intersection shader will always be present when an intersection shader would be executed. A NULL intersection shader is an intersection shader which is effectively VK_SHADER_UNUSED_KHR, such as from a shader group consisting entirely of zeros.
- VK_PIPELINE_CREATE_2_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR specifies that triangle primitives will be skipped during traversal using pipeline trace ray instructions.
- VK_PIPELINE_CREATE_2_RAY_TRACING_SKIP_AABBS_BIT_KHR specifies that AABB primitives will be skipped during traversal using pipeline trace ray instructions.
- VK_PIPELINE_CREATE_2_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR specifies that the shader group handles **can** be saved and reused on a subsequent run (e.g. for trace capture and replay).
- VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT specifies that the pipeline **can** be used in a VkIndirectExecutionSetEXT.
- VK_PIPELINE_CREATE_2_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR specifies that the pipeline will be used with a fragment shading rate attachment.
- VK_PIPELINE_CREATE_2_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT specifies that the pipeline will be used with a fragment density map attachment.
- VK_PIPELINE_CREATE_2_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT specifies that the pipeline **may** be used with an attachment feedback loop including color attachments.
- VK_PIPELINE_CREATE_2_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT specifies that the pipeline **may** be used with an attachment feedback loop including depth-stencil attachments.
- VK_PIPELINE_CREATE_2_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT specifies that the ray tracing pipeline **can** be used with acceleration structures which reference an opacity micromap array.
- VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR specifies that VkPipelineBinaryKHR objects **can** be created from the pipeline. If VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR is used, implementations **should** not store pipeline data to an internal cache, if such a cache exists as stated by pipelineBinaryInternalCache. If pipelineBinaryPrefersInternalCache is VK_TRUE, applications **should** not use VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR.

It is valid to set both VK_PIPELINE_CREATE_2_ALLOW_DERIVATIVES_BIT_KHR and VK_PIPELINE_CREATE_2_DERIVATIVE_BIT_KHR. This allows a pipeline to be both a parent and possibly a child in a pipeline hierarchy. See Pipeline Derivatives for more information.

```
// Provided by VK_KHR_maintenance5
typedef VkFlags64 VkPipelineCreateFlags2KHR;
```

VkPipelineCreateFlags2KHR is a bitmask type for setting a mask of zero or more VkPipelineCreateFlagBits2KHR.

Bits which **can** be set in

- VkGraphicsPipelineCreateInfo::flags
- VkComputePipelineCreateInfo::flags
- VkRayTracingPipelineCreateInfoKHR::flags

specify how a pipeline is created, and are:

```
// Provided by VK_VERSION_1_0
typedef enum VkPipelineCreateFlagBits {
    VK_PIPELINE_CREATE_DISABLE_OPTIMIZATION_BIT = 0x00000001,
    VK_PIPELINE_CREATE_ALLOW_DERIVATIVES_BIT = 0x00000002,
    VK_PIPELINE_CREATE_DERIVATIVE_BIT = 0x00000004,
    VK_PIPELINE_CREATE_VIEW_INDEX_FROM_DEVICE_INDEX_BIT = 0x00000008,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR = 0x00004000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR = 0x00008000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR = 0x00010000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR = 0x00020000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR = 0x00001000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR = 0x00002000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR =
0x00080000,
  // Provided by VK_EXT_fragment_density_map with VK_VERSION_1_3 or
VK_KHR_dynamic_rendering
    VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT = 0x00400000,
  // Provided by VK_KHR_fragment_shading_rate with VK_VERSION_1_3 or
VK_KHR_dynamic_rendering
    VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR =
0x00200000,
  // Provided by VK_KHR_pipeline_executable_properties
    VK_PIPELINE_CREATE_CAPTURE_STATISTICS_BIT_KHR = 0x00000040,
  // Provided by VK_KHR_pipeline_executable_properties
    VK_PIPELINE_CREATE_CAPTURE_INTERNAL_REPRESENTATIONS_BIT_KHR = 0x00000080,
  // Provided by VK_KHR_pipeline_library
    VK_PIPELINE_CREATE_LIBRARY_BIT_KHR = 0x00000800,
```

```
  // Provided by VK_EXT_attachment_feedback_loop_layout
    VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT = 0x02000000,
  // Provided by VK_EXT_attachment_feedback_loop_layout
    VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT = 0x04000000,
  // Provided by VK_EXT_opacity_micromap
    VK_PIPELINE_CREATE_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT = 0x01000000,
    VK_PIPELINE_CREATE_DISPATCH_BASE = VK_PIPELINE_CREATE_DISPATCH_BASE_BIT,
  // Provided by VK_KHR_device_group
    VK_PIPELINE_CREATE_VIEW_INDEX_FROM_DEVICE_INDEX_BIT_KHR =
VK_PIPELINE_CREATE_VIEW_INDEX_FROM_DEVICE_INDEX_BIT,
  // Provided by VK_KHR_device_group
    VK_PIPELINE_CREATE_DISPATCH_BASE_KHR = VK_PIPELINE_CREATE_DISPATCH_BASE,
  // Provided by VK_EXT_fragment_density_map with VK_VERSION_1_3 or
VK_KHR_dynamic_rendering
  // VK_PIPELINE_RASTERIZATION_STATE_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT is
a deprecated alias
    VK_PIPELINE_RASTERIZATION_STATE_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT =
VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT,
  // Provided by VK_KHR_fragment_shading_rate with VK_VERSION_1_3 or
VK_KHR_dynamic_rendering
  // VK_PIPELINE_RASTERIZATION_STATE_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
is a deprecated alias
    VK_PIPELINE_RASTERIZATION_STATE_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR =
VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR,
} VkPipelineCreateFlagBits;
```

- VK_PIPELINE_CREATE_DISABLE_OPTIMIZATION_BIT specifies that the created pipeline will not be optimized. Using this flag **may** reduce the time taken to create the pipeline.
- VK_PIPELINE_CREATE_ALLOW_DERIVATIVES_BIT specifies that the pipeline to be created is allowed to be the parent of a pipeline that will be created in a subsequent pipeline creation call.
- VK_PIPELINE_CREATE_DERIVATIVE_BIT specifies that the pipeline to be created will be a child of a previously created parent pipeline.
- VK_PIPELINE_CREATE_VIEW_INDEX_FROM_DEVICE_INDEX_BIT specifies that any shader input variables decorated as ViewIndex will be assigned values as if they were decorated as DeviceIndex.
- VK_PIPELINE_CREATE_DISPATCH_BASE specifies that a compute pipeline **can** be used with vkCmdDispatchBaseKHR with a non-zero base workgroup.
- VK_PIPELINE_CREATE_CAPTURE_STATISTICS_BIT_KHR specifies that the shader compiler should capture statistics for the pipeline executables produced by the compile process which **can** later be retrieved by calling vkGetPipelineExecutableStatisticsKHR. Enabling this flag **must** not affect the final compiled pipeline but **may** disable pipeline caching or otherwise affect pipeline creation time.
- VK_PIPELINE_CREATE_CAPTURE_INTERNAL_REPRESENTATIONS_BIT_KHR specifies that the shader compiler should capture the internal representations of pipeline executables produced by the compile process which **can** later be retrieved by calling vkGetPipelineExecutableInternalRepresentationsKHR. Enabling this flag **must** not affect the final compiled pipeline but **may** disable pipeline caching or otherwise affect pipeline creation

time. When capturing IR from pipelines created with pipeline libraries, there is no guarantee that IR from libraries **can** be retrieved from the linked pipeline. Applications **should** retrieve IR from each library, and any linked pipelines, separately.

- VK_PIPELINE_CREATE_LIBRARY_BIT_KHR specifies that the pipeline **cannot** be used directly, and instead defines a *pipeline library* that **can** be combined with other pipelines using the VkPipelineLibraryCreateInfoKHR structure. This is available in ray tracing pipelines.
- VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR specifies that an any-hit shader will always be present when an any-hit shader would be executed. A NULL any-hit shader is an any-hit shader which is effectively VK_SHADER_UNUSED_KHR, such as from a shader group consisting entirely of zeros.
- VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR specifies that a closest hit shader will always be present when a closest hit shader would be executed. A NULL closest hit shader is a closest hit shader which is effectively VK_SHADER_UNUSED_KHR, such as from a shader group consisting entirely of zeros.
- VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR specifies that a miss shader will always be present when a miss shader would be executed. A NULL miss shader is a miss shader which is effectively VK_SHADER_UNUSED_KHR, such as from a shader group consisting entirely of zeros.
- VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR specifies that an intersection shader will always be present when an intersection shader would be executed. A NULL intersection shader is an intersection shader which is effectively VK_SHADER_UNUSED_KHR, such as from a shader group consisting entirely of zeros.
- VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR specifies that triangle primitives will be skipped during traversal using pipeline trace ray instructions.
- VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR specifies that AABB primitives will be skipped during traversal using pipeline trace ray instructions.
- VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR specifies that the shader group handles **can** be saved and reused on a subsequent run (e.g. for trace capture and replay).
- VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR specifies that the pipeline will be used with a fragment shading rate attachment and dynamic rendering.
- VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT specifies that the pipeline will be used with a fragment density map attachment and dynamic rendering.
- VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT specifies that the pipeline **may** be used with an attachment feedback loop including color attachments. It is ignored if VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT is set in pDynamicStates.
- VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT specifies that the pipeline **may** be used with an attachment feedback loop including depth-stencil attachments. It is ignored if VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT is set in pDynamicStates.
- VK_PIPELINE_CREATE_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT specifies that the ray tracing pipeline **can** be used with acceleration structures which reference an opacity micromap array.

It is valid to set both VK_PIPELINE_CREATE_ALLOW_DERIVATIVES_BIT and

VK_PIPELINE_CREATE_DERIVATIVE_BIT. This allows a pipeline to be both a parent and possibly a child in a pipeline hierarchy. See Pipeline Derivatives for more information.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineCreateFlags;
```

VkPipelineCreateFlags is a bitmask type for setting a mask of zero or more VkPipelineCreateFlagBits.

The VkPipelineBinaryInfoKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPipelineBinaryInfoKHR {
    VkStructureType               sType;
    const void*                   pNext;
    uint32_t                      binaryCount;
    const VkPipelineBinaryKHR*    pPipelineBinaries;
} VkPipelineBinaryInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- binaryCount is the number of elements in the pPipelineBinaries array.
- pPipelineBinaries is a pointer to an array of VkPipelineBinaryKHR handles.

If a VkPipelineBinaryInfoKHR structure with a binaryCount greater than 0 is included in the pNext chain of any Vk*PipelineCreateInfo structure when creating a pipeline, implementations **must** use the data in pPipelineBinaries instead of recalculating it. Any shader module identifiers or shader modules declared in VkPipelineShaderStageCreateInfo instances are ignored.

If this structure is not included in the pNext chain, it is equivalent to specifying this structure with a binaryCount of 0.

Valid Usage

- VUID-VkPipelineBinaryInfoKHR-binaryCount-09603
  binaryCount and the order of the elements in pPipelineBinaries **must** exactly match that returned by vkCreatePipelineBinariesKHR for the matching Vk*PipelineCreateInfo structure and its pNext chain, ignoring the presence of the VkPipelineBinaryInfoKHR structure, the presence of the VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR flag, and absence of any shader module identifiers or shader modules, for the same global pipeline key, from either:
  - VkPipelineBinaryCreateInfoKHR::pPipelineCreateInfo, or
  - VkPipelineBinaryCreateInfoKHR::pipeline

Valid Usage (Implicit)

- VUID-VkPipelineBinaryInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_BINARY_INFO_KHR`
- VUID-VkPipelineBinaryInfoKHR-pPipelineBinaries-parameter
  If `binaryCount` is not `0`, `pPipelineBinaries` **must** be a valid pointer to an array of `binaryCount` valid VkPipelineBinaryKHR handles

The `VkPipelineDynamicStateCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineDynamicStateCreateInfo {
    VkStructureType                      sType;
    const void*                          pNext;
    VkPipelineDynamicStateCreateFlags    flags;
    uint32_t                             dynamicStateCount;
    const VkDynamicState*                pDynamicStates;
} VkPipelineDynamicStateCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `dynamicStateCount` is the number of elements in the `pDynamicStates` array.
- `pDynamicStates` is a pointer to an array of VkDynamicState values specifying which pieces of pipeline state will use the values from dynamic state commands rather than from pipeline state creation information.

Valid Usage

- VUID-VkPipelineDynamicStateCreateInfo-pDynamicStates-01442
  Each element of `pDynamicStates` **must** be unique

Valid Usage (Implicit)

- VUID-VkPipelineDynamicStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO`
- VUID-VkPipelineDynamicStateCreateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPipelineDynamicStateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineDynamicStateCreateInfo-pDynamicStates-parameter
  If `dynamicStateCount` is not `0`, `pDynamicStates` **must** be a valid pointer to an array of

`dynamicStateCount` valid `VkDynamicState` values

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineDynamicStateCreateFlags;
```

`VkPipelineDynamicStateCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

The source of different pieces of dynamic state is specified by the `VkPipelineDynamicStateCreateInfo`::`pDynamicStates` property of the currently active pipeline, each of whose elements **must** be one of the values:

```
// Provided by VK_VERSION_1_0
typedef enum VkDynamicState {
    VK_DYNAMIC_STATE_VIEWPORT = 0,
    VK_DYNAMIC_STATE_SCISSOR = 1,
    VK_DYNAMIC_STATE_LINE_WIDTH = 2,
    VK_DYNAMIC_STATE_DEPTH_BIAS = 3,
    VK_DYNAMIC_STATE_BLEND_CONSTANTS = 4,
    VK_DYNAMIC_STATE_DEPTH_BOUNDS = 5,
    VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK = 6,
    VK_DYNAMIC_STATE_STENCIL_WRITE_MASK = 7,
    VK_DYNAMIC_STATE_STENCIL_REFERENCE = 8,
  // Provided by VK_EXT_discard_rectangles
    VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT = 1000099000,
  // Provided by VK_EXT_discard_rectangles
    VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT = 1000099001,
  // Provided by VK_EXT_discard_rectangles
    VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT = 1000099002,
  // Provided by VK_EXT_sample_locations
    VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT = 1000143000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR = 1000347000,
  // Provided by VK_KHR_fragment_shading_rate
    VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR = 1000226000,
  // Provided by VK_EXT_vertex_input_dynamic_state
    VK_DYNAMIC_STATE_VERTEX_INPUT_EXT = 1000352000,
  // Provided by VK_EXT_color_write_enable
    VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT = 1000381000,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT = 1000455003,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_POLYGON_MODE_EXT = 1000455004,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT = 1000455005,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_SAMPLE_MASK_EXT = 1000455006,
  // Provided by VK_EXT_extended_dynamic_state3
```

```
    VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT = 1000455007,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT = 1000455008,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT = 1000455009,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT = 1000455010,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT = 1000455011,
  // Provided by VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT = 1000455012,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_KHR_maintenance2 or
VK_VERSION_1_1
    VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT = 1000455002,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_transform_feedback
    VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT = 1000455013,
  // Provided by VK_EXT_conservative_rasterization with VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT = 1000455014,
  // Provided by VK_EXT_conservative_rasterization with VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT = 1000455015,
  // Provided by VK_EXT_depth_clip_enable with VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT = 1000455016,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_sample_locations
    VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT = 1000455017,
  // Provided by VK_EXT_blend_operation_advanced with VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT = 1000455018,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_provoking_vertex
    VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT = 1000455019,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_line_rasterization
    VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT = 1000455020,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_line_rasterization
    VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT = 1000455021,
  // Provided by VK_EXT_depth_clip_control with VK_EXT_extended_dynamic_state3
    VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT = 1000455022,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_clip_space_w_scaling
    VK_DYNAMIC_STATE_VIEWPORT_W_SCALING_ENABLE_NV = 1000455023,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_viewport_swizzle
    VK_DYNAMIC_STATE_VIEWPORT_SWIZZLE_NV = 1000455024,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_fragment_coverage_to_color
    VK_DYNAMIC_STATE_COVERAGE_TO_COLOR_ENABLE_NV = 1000455025,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_fragment_coverage_to_color
    VK_DYNAMIC_STATE_COVERAGE_TO_COLOR_LOCATION_NV = 1000455026,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_framebuffer_mixed_samples
    VK_DYNAMIC_STATE_COVERAGE_MODULATION_MODE_NV = 1000455027,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_framebuffer_mixed_samples
    VK_DYNAMIC_STATE_COVERAGE_MODULATION_TABLE_ENABLE_NV = 1000455028,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_framebuffer_mixed_samples
    VK_DYNAMIC_STATE_COVERAGE_MODULATION_TABLE_NV = 1000455029,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_shading_rate_image
    VK_DYNAMIC_STATE_SHADING_RATE_IMAGE_ENABLE_NV = 1000455030,
  // Provided by VK_EXT_extended_dynamic_state3 with
```

```
// VK_NV_representative_fragment_test
    VK_DYNAMIC_STATE_REPRESENTATIVE_FRAGMENT_TEST_ENABLE_NV = 1000455031,
  // Provided by VK_EXT_extended_dynamic_state3 with VK_NV_coverage_reduction_mode
    VK_DYNAMIC_STATE_COVERAGE_REDUCTION_MODE_NV = 1000455032,
  // Provided by VK_EXT_attachment_feedback_loop_dynamic_state
    VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT = 1000524000,
  // Provided by VK_KHR_line_rasterization
    VK_DYNAMIC_STATE_LINE_STIPPLE_KHR = 1000259000,
  // Provided by VK_EXT_depth_clamp_control
    VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT = 1000582000,
} VkDynamicState;
```

- VK_DYNAMIC_STATE_VIEWPORT specifies that the pViewports state in VkPipelineViewportStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetViewport before any drawing commands. The number of viewports used by a pipeline is still specified by the viewportCount member of VkPipelineViewportStateCreateInfo.
- VK_DYNAMIC_STATE_SCISSOR specifies that the pScissors state in VkPipelineViewportStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetScissor before any drawing commands. The number of scissor rectangles used by a pipeline is still specified by the scissorCount member of VkPipelineViewportStateCreateInfo.
- VK_DYNAMIC_STATE_LINE_WIDTH specifies that the lineWidth state in VkPipelineRasterizationStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetLineWidth before any drawing commands that generate line primitives for the rasterizer.
- VK_DYNAMIC_STATE_DEPTH_BIAS specifies that any instance of VkDepthBiasRepresentationInfoEXT included in the pNext chain of VkPipelineRasterizationStateCreateInfo as well as the depthBiasConstantFactor, depthBiasClamp and depthBiasSlopeFactor states in VkPipelineRasterizationStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetDepthBias or vkCmdSetDepthBias2EXT before any draws are performed with depth bias enabled.
- VK_DYNAMIC_STATE_BLEND_CONSTANTS specifies that the blendConstants state in VkPipelineColorBlendStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetBlendConstants before any draws are performed with a pipeline state with VkPipelineColorBlendAttachmentState member blendEnable set to VK_TRUE and any of the blend functions using a constant blend color.
- VK_DYNAMIC_STATE_DEPTH_BOUNDS specifies that the minDepthBounds and maxDepthBounds states of VkPipelineDepthStencilStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetDepthBounds before any draws are performed with a pipeline state with VkPipelineDepthStencilStateCreateInfo member depthBoundsTestEnable set to VK_TRUE.
- VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK specifies that the compareMask state in VkPipelineDepthStencilStateCreateInfo for both front and back will be ignored and **must** be set dynamically with vkCmdSetStencilCompareMask before any draws are performed with a pipeline state with VkPipelineDepthStencilStateCreateInfo member stencilTestEnable set to VK_TRUE
- VK_DYNAMIC_STATE_STENCIL_WRITE_MASK specifies that the writeMask state in

VkPipelineDepthStencilStateCreateInfo for both `front` and `back` will be ignored and **must** be set dynamically with vkCmdSetStencilWriteMask before any draws are performed with a pipeline state with VkPipelineDepthStencilStateCreateInfo member `stencilTestEnable` set to `VK_TRUE`

- `VK_DYNAMIC_STATE_STENCIL_REFERENCE` specifies that the `reference` state in VkPipelineDepthStencilStateCreateInfo for both `front` and `back` will be ignored and **must** be set dynamically with vkCmdSetStencilReference before any draws are performed with a pipeline state with VkPipelineDepthStencilStateCreateInfo member `stencilTestEnable` set to `VK_TRUE`
- `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT` specifies that the `pDiscardRectangles` state in VkPipelineDiscardRectangleStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetDiscardRectangleEXT before any draw or clear commands.
- `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT` specifies that the presence of the VkPipelineDiscardRectangleStateCreateInfoEXT structure in the VkGraphicsPipelineCreateInfo chain with a `discardRectangleCount` greater than zero does not implicitly enable discard rectangles and they **must** be enabled dynamically with vkCmdSetDiscardRectangleEnableEXT before any draw commands. This is available on implementations that support at least `specVersion` 2 of the `VK_EXT_discard_rectangles` extension.
- `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT` specifies that the `discardRectangleMode` state in VkPipelineDiscardRectangleStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetDiscardRectangleModeEXT before any draw commands. This is available on implementations that support at least `specVersion` 2 of the `VK_EXT_discard_rectangles` extension.
- `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` specifies that the `sampleLocationsInfo` state in VkPipelineSampleLocationsStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetSampleLocationsEXT before any draw or clear commands. Enabling custom sample locations is still indicated by the `sampleLocationsEnable` member of VkPipelineSampleLocationsStateCreateInfoEXT.
- `VK_DYNAMIC_STATE_LINE_STIPPLE_EXT` specifies that the `lineStippleFactor` and `lineStipplePattern` state in VkPipelineRasterizationLineStateCreateInfoKHR will be ignored and **must** be set dynamically with vkCmdSetLineStippleKHR before any draws are performed with a pipeline state with VkPipelineRasterizationLineStateCreateInfoKHR member `stippledLineEnable` set to `VK_TRUE`.
- `VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR` specifies that state in VkPipelineFragmentShadingRateStateCreateInfoKHR will be ignored and **must** be set dynamically with vkCmdSetFragmentShadingRateKHR before any drawing commands.
- `VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR` specifies that the default stack size computation for the pipeline will be ignored and **must** be set dynamically with vkCmdSetRayTracingPipelineStackSizeKHR before any ray tracing calls are performed.
- `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` specifies that the `pVertexInputState` state will be ignored and **must** be set dynamically with vkCmdSetVertexInputEXT before any drawing commands
- `VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT` specifies that the `pColorWriteEnables` state in VkPipelineColorWriteCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetColorWriteEnableEXT before any draw call.
- `VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT` specifies that the `domainOrigin` state in VkPipelineTessellationDomainOriginStateCreateInfo will be ignored and **must** be set

dynamically with vkCmdSetTessellationDomainOriginEXT before any draw call.

- VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT specifies that the depthClampEnable state in VkPipelineRasterizationStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetDepthClampEnableEXT before any draw call.
- VK_DYNAMIC_STATE_POLYGON_MODE_EXT specifies that the polygonMode state in VkPipelineRasterizationStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetPolygonModeEXT before any draw call.
- VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT specifies that the rasterizationSamples state in VkPipelineMultisampleStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetRasterizationSamplesEXT before any draw call.
- VK_DYNAMIC_STATE_SAMPLE_MASK_EXT specifies that the pSampleMask state in VkPipelineMultisampleStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetSampleMaskEXT before any draw call.
- VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT specifies that the alphaToCoverageEnable state in VkPipelineMultisampleStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetAlphaToCoverageEnableEXT before any draw call.
- VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT specifies that the alphaToOneEnable state in VkPipelineMultisampleStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetAlphaToOneEnableEXT before any draw call.
- VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT specifies that the logicOpEnable state in VkPipelineColorBlendStateCreateInfo will be ignored and **must** be set dynamically with vkCmdSetLogicOpEnableEXT before any draw call.
- VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT specifies that the blendEnable state in VkPipelineColorBlendAttachmentState will be ignored and **must** be set dynamically with vkCmdSetColorBlendEnableEXT before any draw call.
- VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT specifies that the srcColorBlendFactor, dstColorBlendFactor, colorBlendOp, srcAlphaBlendFactor, dstAlphaBlendFactor, and alphaBlendOp states in VkPipelineColorBlendAttachmentState will be ignored and **must** be set dynamically with vkCmdSetColorBlendEquationEXT before any draw call.
- VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT specifies that the colorWriteMask state in VkPipelineColorBlendAttachmentState will be ignored and **must** be set dynamically with vkCmdSetColorWriteMaskEXT before any draw call.
- VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT specifies that the rasterizationStream state in VkPipelineRasterizationStateStreamCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetRasterizationStreamEXT before any draw call.
- VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT specifies that the conservativeRasterizationMode state in VkPipelineRasterizationConservativeStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetConservativeRasterizationModeEXT before any draw call.
- VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT specifies that the extraPrimitiveOverestimationSize state in VkPipelineRasterizationConservativeStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetExtraPrimitiveOverestimationSizeEXT before any draw call.

- VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT specifies that the depthClipEnable state in VkPipelineRasterizationDepthClipStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetDepthClipEnableEXT before any draw call.
- VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT specifies that the sampleLocationsEnable state in VkPipelineSampleLocationsStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetSampleLocationsEnableEXT before any draw call.
- VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT specifies that the colorBlendOp state in VkPipelineColorBlendAttachmentState, and srcPremultiplied, dstPremultiplied, and blendOverlap states in VkPipelineColorBlendAdvancedStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetColorBlendAdvancedEXT before any draw call.
- VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT specifies that the provokingVertexMode state in VkPipelineRasterizationProvokingVertexStateCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetProvokingVertexModeEXT before any draw call.
- VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT specifies that the lineRasterizationMode state in VkPipelineRasterizationLineStateCreateInfoKHR will be ignored and **must** be set dynamically with vkCmdSetLineRasterizationModeEXT before any draw call.
- VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT specifies that the stippledLineEnable state in VkPipelineRasterizationLineStateCreateInfoKHR will be ignored and **must** be set dynamically with vkCmdSetLineStippleEnableEXT before any draw call.
- VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT specifies that the negativeOneToOne state in VkPipelineViewportDepthClipControlCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetDepthClipNegativeOneToOneEXT before any draw call.
- VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT specifies that the depthClampMode and pDepthClampRange state in VkPipelineViewportDepthClampControlCreateInfoEXT will be ignored and **must** be set dynamically with vkCmdSetDepthClampRangeEXT before any draw call.
- VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT specifies that the VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT and VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT flags will be ignored and **must** be set dynamically with vkCmdSetAttachmentFeedbackLoopEnableEXT before any draw call.

10.3.1. Valid Combinations of Stages for Graphics Pipelines

If tessellation shader stages are omitted, the tessellation shading and fixed-function stages of the pipeline are skipped.

If a geometry shader is omitted, the geometry shading stage is skipped.

If a fragment shader is omitted, fragment color outputs have undefined values, and the fragment depth value is determined by Fragment Operations state. This **can** be useful for depth-only rendering.

Presence of a shader stage in a pipeline is indicated by including a valid VkPipelineShaderStageCreateInfo with module and pName selecting an entry point from a shader module, where that entry point is valid for the stage specified by stage.

Presence of some of the fixed-function stages in the pipeline is implicitly derived from enabled shaders and provided state. For example, the fixed-function tessellator is always present when the pipeline has valid Tessellation Control and Tessellation Evaluation shaders.

For example:

- Depth/stencil-only rendering in a subpass with no color attachments
  - Active Pipeline Shader Stages
    - Vertex Shader
  - Required: Fixed-Function Pipeline Stages
    - VkPipelineVertexInputStateCreateInfo
    - VkPipelineInputAssemblyStateCreateInfo
    - VkPipelineViewportStateCreateInfo
    - VkPipelineRasterizationStateCreateInfo
    - VkPipelineMultisampleStateCreateInfo
    - VkPipelineDepthStencilStateCreateInfo
- Color-only rendering in a subpass with no depth/stencil attachment
  - Active Pipeline Shader Stages
    - Vertex Shader
    - Fragment Shader
  - Required: Fixed-Function Pipeline Stages
    - VkPipelineVertexInputStateCreateInfo
    - VkPipelineInputAssemblyStateCreateInfo
    - VkPipelineViewportStateCreateInfo
    - VkPipelineRasterizationStateCreateInfo
    - VkPipelineMultisampleStateCreateInfo
    - VkPipelineColorBlendStateCreateInfo
- Rendering pipeline with tessellation and geometry shaders
  - Active Pipeline Shader Stages
    - Vertex Shader
    - Tessellation Control Shader
    - Tessellation Evaluation Shader
    - Geometry Shader
    - Fragment Shader
  - Required: Fixed-Function Pipeline Stages
    - VkPipelineVertexInputStateCreateInfo
    - VkPipelineInputAssemblyStateCreateInfo

- VkPipelineTessellationStateCreateInfo
- VkPipelineViewportStateCreateInfo
- VkPipelineRasterizationStateCreateInfo
- VkPipelineMultisampleStateCreateInfo
- VkPipelineDepthStencilStateCreateInfo
- VkPipelineColorBlendStateCreateInfo

10.4. Ray Tracing Pipelines

Ray tracing pipelines consist of multiple shader stages, fixed-function traversal stages, and a pipeline layout.

VK_SHADER_UNUSED_KHR is a special shader index used to indicate that a ray generation, miss, or callable shader member is not used.

```
#define VK_SHADER_UNUSED_KHR              (~0U)
```

To create ray tracing pipelines, call:

```
// Provided by VK_KHR_ray_tracing_pipeline
VkResult vkCreateRayTracingPipelinesKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    VkPipelineCache                             pipelineCache,
    uint32_t                                    createInfoCount,
    const VkRayTracingPipelineCreateInfoKHR*    pCreateInfos,
    const VkAllocationCallbacks*                pAllocator,
    VkPipeline*                                 pPipelines);
```

- device is the logical device that creates the ray tracing pipelines.
- deferredOperation is VK_NULL_HANDLE or the handle of a valid VkDeferredOperationKHR request deferral object for this command.
- pipelineCache is either VK_NULL_HANDLE, indicating that pipeline caching is disabled, or the handle of a valid pipeline cache object, in which case use of that cache is enabled for the duration of the command.
- createInfoCount is the length of the pCreateInfos and pPipelines arrays.
- pCreateInfos is a pointer to an array of VkRayTracingPipelineCreateInfoKHR structures.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.
- pPipelines is a pointer to an array in which the resulting ray tracing pipeline objects are returned.

The VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS error is returned if the implementation is unable to

reuse the shader group handles provided in VkRayTracingShaderGroupCreateInfoKHR::pShaderGroupCaptureReplayHandle when VkPhysicalDeviceRayTracingPipelineFeaturesKHR::rayTracingPipelineShaderGroupHandleCaptureReplay is enabled.

Pipelines are created and returned as described for Multiple Pipeline Creation.

Valid Usage

- VUID-vkCreateRayTracingPipelinesKHR-device-09677
  `device` **must** support at least one queue family with the `VK_QUEUE_COMPUTE_BIT` capability
- VUID-vkCreateRayTracingPipelinesKHR-flags-03415
  If the `flags` member of any element of `pCreateInfos` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, and the `basePipelineIndex` member of that same element is not `-1`, `basePipelineIndex` **must** be less than the index into `pCreateInfos` that corresponds to that element
- VUID-vkCreateRayTracingPipelinesKHR-flags-03416
  If the `flags` member of any element of `pCreateInfos` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, the base pipeline **must** have been created with the `VK_PIPELINE_CREATE_ALLOW_DERIVATIVES_BIT` flag set
- VUID-vkCreateRayTracingPipelinesKHR-flags-03816
  `flags` **must** not contain the `VK_PIPELINE_CREATE_DISPATCH_BASE` flag
- VUID-vkCreateRayTracingPipelinesKHR-deferredOperation-03678
  Any previous deferred operation that was associated with `deferredOperation` **must** be complete
- VUID-vkCreateRayTracingPipelinesKHR-pNext-09616
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `pipelineCache` **must** be VK_NULL_HANDLE
- VUID-vkCreateRayTracingPipelinesKHR-pNext-09617
  If a VkPipelineCreateFlags2CreateInfoKHR structure with the `VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR` flag set is included in the `pNext` chain of any element of `pCreateInfos`, `pipelineCache` **must** be VK_NULL_HANDLE
- VUID-vkCreateRayTracingPipelinesKHR-binaryCount-09620
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATION_FEEDBACK_APPLICATION_PIPELINE_CACHE_HIT_BIT` **must** not be set in the `flags` of that element
- VUID-vkCreateRayTracingPipelinesKHR-binaryCount-09621
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATION_FEEDBACK_BASE_PIPELINE_ACCELERATION_BIT` **must** not be set in the `flags` of that element
- VUID-vkCreateRayTracingPipelinesKHR-binaryCount-09622
  If VkPipelineBinaryInfoKHR::`binaryCount` is not `0` for any element of `pCreateInfos`, `VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT_EXT` **must** not be set in the `flags` of that element

- VUID-vkCreateRayTracingPipelinesKHR-rayTracingPipeline-03586
  The `rayTracingPipeline` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCreateRayTracingPipelinesKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateRayTracingPipelinesKHR-deferredOperation-parameter
  If `deferredOperation` is not VK_NULL_HANDLE, `deferredOperation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkCreateRayTracingPipelinesKHR-pipelineCache-parameter
  If `pipelineCache` is not VK_NULL_HANDLE, `pipelineCache` **must** be a valid VkPipelineCache handle
- VUID-vkCreateRayTracingPipelinesKHR-pCreateInfos-parameter
  `pCreateInfos` **must** be a valid pointer to an array of `createInfoCount` valid VkRayTracingPipelineCreateInfoKHR structures
- VUID-vkCreateRayTracingPipelinesKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateRayTracingPipelinesKHR-pPipelines-parameter
  `pPipelines` **must** be a valid pointer to an array of `createInfoCount` VkPipeline handles
- VUID-vkCreateRayTracingPipelinesKHR-createInfoCount-arraylength
  `createInfoCount` **must** be greater than `0`
- VUID-vkCreateRayTracingPipelinesKHR-deferredOperation-parent
  If `deferredOperation` is a valid handle, it **must** have been created, allocated, or retrieved from `device`
- VUID-vkCreateRayTracingPipelinesKHR-pipelineCache-parent
  If `pipelineCache` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_OPERATION_DEFERRED_KHR`
- `VK_OPERATION_NOT_DEFERRED_KHR`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS`

The VkRayTracingPipelineCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef struct VkRayTracingPipelineCreateInfoKHR {
    VkStructureType                                 sType;
    const void*                                     pNext;
    VkPipelineCreateFlags                           flags;
    uint32_t                                        stageCount;
    const VkPipelineShaderStageCreateInfo*          pStages;
    uint32_t                                        groupCount;
    const VkRayTracingShaderGroupCreateInfoKHR*     pGroups;
    uint32_t                                        maxPipelineRayRecursionDepth;
    const VkPipelineLibraryCreateInfoKHR*           pLibraryInfo;
    const VkRayTracingPipelineInterfaceCreateInfoKHR* pLibraryInterface;
    const VkPipelineDynamicStateCreateInfo*         pDynamicState;
    VkPipelineLayout                                layout;
    VkPipeline                                      basePipelineHandle;
    int32_t                                         basePipelineIndex;
} VkRayTracingPipelineCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkPipelineCreateFlagBits specifying how the pipeline will be generated.
- stageCount is the number of entries in the pStages array.
- pStages is a pointer to an array of stageCount VkPipelineShaderStageCreateInfo structures describing the set of the shader stages to be included in the ray tracing pipeline.
- groupCount is the number of entries in the pGroups array.
- pGroups is a pointer to an array of groupCount VkRayTracingShaderGroupCreateInfoKHR structures describing the set of the shader stages to be included in each shader group in the ray tracing pipeline.
- maxPipelineRayRecursionDepth is the maximum recursion depth of shaders executed by this pipeline.
- pLibraryInfo is a pointer to a VkPipelineLibraryCreateInfoKHR structure defining pipeline libraries to include.
- pLibraryInterface is a pointer to a VkRayTracingPipelineInterfaceCreateInfoKHR structure defining additional information when using pipeline libraries.
- pDynamicState is a pointer to a VkPipelineDynamicStateCreateInfo structure, and is used to indicate which properties of the pipeline state object are dynamic and **can** be changed independently of the pipeline state. This **can** be NULL, which means no state in the pipeline is considered dynamic.
- layout is the description of binding locations used by both the pipeline and descriptor sets used with the pipeline.
- basePipelineHandle is a pipeline to derive from.

- `basePipelineIndex` is an index into the `pCreateInfos` parameter to use as a pipeline to derive from.

The parameters `basePipelineHandle` and `basePipelineIndex` are described in more detail in Pipeline Derivatives.

When `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR` is specified, this pipeline defines a *pipeline library* which **cannot** be bound as a ray tracing pipeline directly. Instead, pipeline libraries define common shaders and shader groups which **can** be included in future pipeline creation.

If pipeline libraries are included in `pLibraryInfo`, shaders defined in those libraries are treated as if they were defined as additional entries in `pStages`, appended in the order they appear in the `pLibraries` array and in the `pStages` array when those libraries were defined.

When referencing shader groups in order to obtain a shader group handle, groups defined in those libraries are treated as if they were defined as additional entries in `pGroups`, appended in the order they appear in the `pLibraries` array and in the `pGroups` array when those libraries were defined. The shaders these groups reference are set when the pipeline library is created, referencing those specified in the pipeline library, not in the pipeline that includes it.

The default stack size for a pipeline if `VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR` is not provided is computed as described in Ray Tracing Pipeline Stack.

If the `pNext` chain includes a VkPipelineCreateFlags2CreateInfoKHR structure, VkPipelineCreateFlags2CreateInfoKHR::`flags` from that structure is used instead of `flags` from this structure.

Valid Usage

- VUID-VkRayTracingPipelineCreateInfoKHR-None-09497
  If the `pNext` chain does not include a VkPipelineCreateFlags2CreateInfoKHR structure, `flags` **must** be a valid combination of VkPipelineCreateFlagBits values
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-07984
  If `flags` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, and `basePipelineIndex` is -1, `basePipelineHandle` **must** be a valid ray tracing `VkPipeline` handle
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-07985
  If `flags` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, and `basePipelineHandle` is VK_NULL_HANDLE, `basePipelineIndex` **must** be a valid index into the calling command's `pCreateInfos` parameter
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-07986
  If `flags` contains the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag, `basePipelineIndex` **must** be -1 or `basePipelineHandle` **must** be VK_NULL_HANDLE
- VUID-VkRayTracingPipelineCreateInfoKHR-layout-07987
  If a push constant block is declared in a shader, a push constant range in `layout` **must** match the shader stage
- VUID-VkRayTracingPipelineCreateInfoKHR-layout-10069
  If a push constant block is declared in a shader, the block must be contained inside the

push constant range in `layout` that matches the stage

- VUID-VkRayTracingPipelineCreateInfoKHR-layout-07988
  If a resource variables is declared in a shader, a descriptor slot in `layout` **must** match the shader stage
- VUID-VkRayTracingPipelineCreateInfoKHR-layout-07990
  If a resource variables is declared in a shader, a descriptor slot in `layout` **must** match the descriptor type
- VUID-VkRayTracingPipelineCreateInfoKHR-layout-07991
  If a resource variables is declared in a shader as an array, a descriptor slot in `layout` **must** match the descriptor count
- VUID-VkRayTracingPipelineCreateInfoKHR-pStages-03426
  The shader code for the entry points identified by `pStages`, and the rest of the state identified by this structure **must** adhere to the pipeline linking rules described in the Shader Interfaces chapter
- VUID-VkRayTracingPipelineCreateInfoKHR-layout-03428
  The number of resources in `layout` accessible to each shader stage that is used by the pipeline **must** be less than or equal to VkPhysicalDeviceLimits::`maxPerStageResources`
- VUID-VkRayTracingPipelineCreateInfoKHR-stage-03425
  If `flags` does not include `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR`, the `stage` member of at least one element of `pStages`, including those implicitly added by `pLibraryInfo`, **must** be `VK_SHADER_STAGE_RAYGEN_BIT_KHR`
- VUID-VkRayTracingPipelineCreateInfoKHR-maxPipelineRayRecursionDepth-03589
  `maxPipelineRayRecursionDepth` **must** be less than or equal to VkPhysicalDeviceRayTracingPipelinePropertiesKHR::`maxRayRecursionDepth`
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-03465
  If `flags` includes `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR`, `pLibraryInterface` **must** not be `NULL`
- VUID-VkRayTracingPipelineCreateInfoKHR-pLibraryInfo-03590
  If `pLibraryInfo` is not `NULL` and its `libraryCount` member is greater than `0`, `pLibraryInterface` **must** not be `NULL`
- VUID-VkRayTracingPipelineCreateInfoKHR-pLibraries-03591
  Each element of `pLibraryInfo->pLibraries` **must** have been created with the value of `maxPipelineRayRecursionDepth` equal to that in this pipeline
- VUID-VkRayTracingPipelineCreateInfoKHR-pLibraryInfo-03592
  If `pLibraryInfo` is not `NULL`, each element of its `pLibraries` member **must** have been created with a `layout` that is compatible with the `layout` in this pipeline
- VUID-VkRayTracingPipelineCreateInfoKHR-pLibraryInfo-03593
  If `pLibraryInfo` is not `NULL`, each element of its `pLibraries` member **must** have been created with values of the `maxPipelineRayPayloadSize` and `maxPipelineRayHitAttributeSize` members of `pLibraryInterface` equal to those in this pipeline
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-03594
  If `flags` includes `VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR`, each

element of pLibraryInfo->pLibraries **must** have been created with the VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR bit set

- VUID-VkRayTracingPipelineCreateInfoKHR-flags-04718
  If flags includes VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR, each element of pLibraryInfo->pLibraries **must** have been created with the VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR bit set
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-04719
  If flags includes VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR, each element of pLibraryInfo->pLibraries **must** have been created with the VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR bit set
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-04720
  If flags includes VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR, each element of pLibraryInfo->pLibraries **must** have been created with the VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR bit set
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-04721
  If flags includes VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR, each element of pLibraryInfo->pLibraries **must** have been created with the VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR bit set
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-04722
  If flags includes VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR, each element of pLibraryInfo->pLibraries **must** have been created with the VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR bit set
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-04723
  If flags includes VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR, each element of pLibraryInfo->pLibraries **must** have been created with the VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR bit set
- VUID-VkRayTracingPipelineCreateInfoKHR-pLibraryInfo-03595
  If the VK_KHR_pipeline_library extension is not enabled, pLibraryInfo and pLibraryInterface **must** be NULL
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-03470
  If flags includes VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR, for any element of pGroups with a type of VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR or VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR, the anyHitShader of that element **must** not be VK_SHADER_UNUSED_KHR
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-03471
  If flags includes VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR, for any element of pGroups with a type of VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR or VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR, the closestHitShader of that element **must** not be VK_SHADER_UNUSED_KHR
- VUID-VkRayTracingPipelineCreateInfoKHR-rayTraversalPrimitiveCulling-03596
  If the rayTraversalPrimitiveCulling feature is not enabled, flags **must** not include VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR

- VUID-VkRayTracingPipelineCreateInfoKHR-rayTraversalPrimitiveCulling-03597
  If the `rayTraversalPrimitiveCulling` feature is not enabled, `flags` **must** not include `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR`
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-06546
  `flags` **must** not include both `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR` and `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR`
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-03598
  If `flags` includes `VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR`, `rayTracingPipelineShaderGroupHandleCaptureReplay` **must** be enabled
- VUID-VkRayTracingPipelineCreateInfoKHR-rayTracingPipelineShaderGroupHandleCaptureReplay-03599
  If VkPhysicalDeviceRayTracingPipelineFeaturesKHR ::`rayTracingPipelineShaderGroupHandleCaptureReplay` is `VK_TRUE` and the `pShaderGroupCaptureReplayHandle` member of any element of `pGroups` is not `NULL`, `flags` **must** include `VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR`
- VUID-VkRayTracingPipelineCreateInfoKHR-pLibraryInfo-07999
  If `pLibraryInfo` is `NULL` or its `libraryCount` is `0`, `stageCount` **must** not be `0`
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-08700
  If `flags` does not include `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR` and either `pLibraryInfo` is `NULL` or its `libraryCount` is `0`, `groupCount` **must** not be `0`
- VUID-VkRayTracingPipelineCreateInfoKHR-pDynamicStates-03602
  Any element of the `pDynamicStates` member of `pDynamicState` **must** be `VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR`
- VUID-VkRayTracingPipelineCreateInfoKHR-stage-06899
  The `stage` value in all `pStages` elements **must** be one of `VK_SHADER_STAGE_RAYGEN_BIT_KHR`, `VK_SHADER_STAGE_ANY_HIT_BIT_KHR`, `VK_SHADER_STAGE_CLOSEST_HIT_BIT_KHR`, `VK_SHADER_STAGE_MISS_BIT_KHR`, `VK_SHADER_STAGE_INTERSECTION_BIT_KHR`, or `VK_SHADER_STAGE_CALLABLE_BIT_KHR`
- VUID-VkRayTracingPipelineCreateInfoKHR-flags-07403
  If `flags` includes `VK_PIPELINE_CREATE_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT`, each element of `pLibraryInfo->pLibraries` **must** have been created with the `VK_PIPELINE_CREATE_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT` bit set

Valid Usage (Implicit)

- VUID-VkRayTracingPipelineCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RAY_TRACING_PIPELINE_CREATE_INFO_KHR`
- VUID-VkRayTracingPipelineCreateInfoKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkPipelineBinaryInfoKHR or VkPipelineCreateFlags2CreateInfoKHR

- VUID-VkRayTracingPipelineCreateInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkRayTracingPipelineCreateInfoKHR-pStages-parameter
  If `stageCount` is not `0`, `pStages` **must** be a valid pointer to an array of `stageCount` valid VkPipelineShaderStageCreateInfo structures
- VUID-VkRayTracingPipelineCreateInfoKHR-pGroups-parameter
  If `groupCount` is not `0`, `pGroups` **must** be a valid pointer to an array of `groupCount` valid VkRayTracingShaderGroupCreateInfoKHR structures
- VUID-VkRayTracingPipelineCreateInfoKHR-pLibraryInfo-parameter
  If `pLibraryInfo` is not `NULL`, `pLibraryInfo` **must** be a valid pointer to a valid VkPipelineLibraryCreateInfoKHR structure
- VUID-VkRayTracingPipelineCreateInfoKHR-pLibraryInterface-parameter
  If `pLibraryInterface` is not `NULL`, `pLibraryInterface` **must** be a valid pointer to a valid VkRayTracingPipelineInterfaceCreateInfoKHR structure
- VUID-VkRayTracingPipelineCreateInfoKHR-pDynamicState-parameter
  If `pDynamicState` is not `NULL`, `pDynamicState` **must** be a valid pointer to a valid VkPipelineDynamicStateCreateInfo structure
- VUID-VkRayTracingPipelineCreateInfoKHR-layout-parameter
  `layout` **must** be a valid VkPipelineLayout handle
- VUID-VkRayTracingPipelineCreateInfoKHR-commonparent
  Both of `basePipelineHandle`, and `layout` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The `VkRayTracingShaderGroupCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef struct VkRayTracingShaderGroupCreateInfoKHR {
    VkStructureType                   sType;
    const void*                       pNext;
    VkRayTracingShaderGroupTypeKHR    type;
    uint32_t                          generalShader;
    uint32_t                          closestHitShader;
    uint32_t                          anyHitShader;
    uint32_t                          intersectionShader;
    const void*                       pShaderGroupCaptureReplayHandle;
} VkRayTracingShaderGroupCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `type` is the type of hit group specified in this structure.
- `generalShader` is the index of the ray generation, miss, or callable shader from VkRayTracingPipelineCreateInfoKHR::`pStages` in the group if the shader group has `type` of `VK_RAY_TRACING_SHADER_GROUP_TYPE_GENERAL_KHR`, and `VK_SHADER_UNUSED_KHR` otherwise.

- `closestHitShader` is the optional index of the closest hit shader from VkRayTracingPipelineCreateInfoKHR::`pStages` in the group if the shader group has `type` of `VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR` or `VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR`, and `VK_SHADER_UNUSED_KHR` otherwise.
- `anyHitShader` is the optional index of the any-hit shader from VkRayTracingPipelineCreateInfoKHR::`pStages` in the group if the shader group has `type` of `VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR` or `VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR`, and `VK_SHADER_UNUSED_KHR` otherwise.
- `intersectionShader` is the index of the intersection shader from VkRayTracingPipelineCreateInfoKHR::`pStages` in the group if the shader group has `type` of `VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR`, and `VK_SHADER_UNUSED_KHR` otherwise.
- `pShaderGroupCaptureReplayHandle` is `NULL` or a pointer to replay information for this shader group queried from vkGetRayTracingCaptureReplayShaderGroupHandlesKHR, as described in Ray Tracing Capture Replay. Ignored if VkPhysicalDeviceRayTracingPipelineFeaturesKHR::`rayTracingPipelineShaderGroupHandleCaptureReplay` is `VK_FALSE`.

Valid Usage

- VUID-VkRayTracingShaderGroupCreateInfoKHR-type-03474
  If `type` is `VK_RAY_TRACING_SHADER_GROUP_TYPE_GENERAL_KHR` then `generalShader` **must** be a valid index into VkRayTracingPipelineCreateInfoKHR::`pStages` referring to a shader of `VK_SHADER_STAGE_RAYGEN_BIT_KHR`, `VK_SHADER_STAGE_MISS_BIT_KHR`, or `VK_SHADER_STAGE_CALLABLE_BIT_KHR`
- VUID-VkRayTracingShaderGroupCreateInfoKHR-type-03475
  If `type` is `VK_RAY_TRACING_SHADER_GROUP_TYPE_GENERAL_KHR` then `closestHitShader`, `anyHitShader`, and `intersectionShader` **must** be `VK_SHADER_UNUSED_KHR`
- VUID-VkRayTracingShaderGroupCreateInfoKHR-type-03476
  If `type` is `VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR` then `intersectionShader` **must** be a valid index into VkRayTracingPipelineCreateInfoKHR::`pStages` referring to a shader of `VK_SHADER_STAGE_INTERSECTION_BIT_KHR`
- VUID-VkRayTracingShaderGroupCreateInfoKHR-type-03477
  If `type` is `VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR` then `intersectionShader` **must** be `VK_SHADER_UNUSED_KHR`
- VUID-VkRayTracingShaderGroupCreateInfoKHR-closestHitShader-03478
  `closestHitShader` **must** be either `VK_SHADER_UNUSED_KHR` or a valid index into VkRayTracingPipelineCreateInfoKHR::`pStages` referring to a shader of `VK_SHADER_STAGE_CLOSEST_HIT_BIT_KHR`
- VUID-VkRayTracingShaderGroupCreateInfoKHR-anyHitShader-03479
  `anyHitShader` **must** be either `VK_SHADER_UNUSED_KHR` or a valid index into VkRayTracingPipelineCreateInfoKHR::`pStages` referring to a shader of `VK_SHADER_STAGE_ANY_HIT_BIT_KHR`

- VUID-VkRayTracingShaderGroupCreateInfoKHR-rayTracingPipelineShaderGroupHandleCaptureReplayMixed-03603
  If VkPhysicalDeviceRayTracingPipelineFeaturesKHR::rayTracingPipelineShaderGroupHandleCaptureReplayMixed is VK_FALSE then pShaderGroupCaptureReplayHandle **must** not be provided if it has not been provided on a previous call to ray tracing pipeline creation
- VUID-VkRayTracingShaderGroupCreateInfoKHR-rayTracingPipelineShaderGroupHandleCaptureReplayMixed-03604
  If VkPhysicalDeviceRayTracingPipelineFeaturesKHR::rayTracingPipelineShaderGroupHandleCaptureReplayMixed is VK_FALSE then the caller **must** guarantee that no ray tracing pipeline creation commands with pShaderGroupCaptureReplayHandle provided execute simultaneously with ray tracing pipeline creation commands without pShaderGroupCaptureReplayHandle provided

Valid Usage (Implicit)

- VUID-VkRayTracingShaderGroupCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RAY_TRACING_SHADER_GROUP_CREATE_INFO_KHR
- VUID-VkRayTracingShaderGroupCreateInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkRayTracingShaderGroupCreateInfoKHR-type-parameter
  type **must** be a valid VkRayTracingShaderGroupTypeKHR value

The VkRayTracingShaderGroupTypeKHR enumeration is defined as:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef enum VkRayTracingShaderGroupTypeKHR {
    VK_RAY_TRACING_SHADER_GROUP_TYPE_GENERAL_KHR = 0,
    VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR = 1,
    VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR = 2,
} VkRayTracingShaderGroupTypeKHR;
```

- VK_RAY_TRACING_SHADER_GROUP_TYPE_GENERAL_KHR specifies that a shader group with a single VK_SHADER_STAGE_RAYGEN_BIT_KHR, VK_SHADER_STAGE_MISS_BIT_KHR, or VK_SHADER_STAGE_CALLABLE_BIT_KHR shader in it.
- VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR specifies that a shader group that only hits triangles and **must** not contain an intersection shader, only closest hit and any-hit shaders.
- VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR specifies that a shader group that only intersects with custom geometry and **must** contain an intersection shader and **may** contain closest hit and any-hit shaders.

NOTE For current group types, the hit group type could be inferred from the presence or absence of the intersection shader, but we provide the type explicitly for future hit

groups that do not have that property.

The VkRayTracingPipelineInterfaceCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef struct VkRayTracingPipelineInterfaceCreateInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           maxPipelineRayPayloadSize;
    uint32_t           maxPipelineRayHitAttributeSize;
} VkRayTracingPipelineInterfaceCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxPipelineRayPayloadSize is the maximum payload size in bytes used by any shader in the pipeline.
- maxPipelineRayHitAttributeSize is the maximum attribute structure size in bytes used by any shader in the pipeline.

maxPipelineRayPayloadSize is calculated as the maximum number of bytes used by any block declared in the RayPayloadKHR or IncomingRayPayloadKHR storage classes. maxPipelineRayHitAttributeSize is calculated as the maximum number of bytes used by any block declared in the HitAttributeKHR storage class. As variables in these storage classes do not have explicit offsets, the size should be calculated as if each variable has a scalar alignment equal to the largest scalar alignment of any of the block's members.

NOTE There is no explicit upper limit for maxPipelineRayPayloadSize, but in practice it should be kept as small as possible. Similar to invocation local memory, it must be allocated for each shader invocation and for devices which support many simultaneous invocations, this storage can rapidly be exhausted, resulting in failure.

Valid Usage

- VUID-VkRayTracingPipelineInterfaceCreateInfoKHR-maxPipelineRayHitAttributeSize-03605
  maxPipelineRayHitAttributeSize **must** be less than or equal to VkPhysicalDeviceRayTracingPipelinePropertiesKHR::maxRayHitAttributeSize

Valid Usage (Implicit)

- VUID-VkRayTracingPipelineInterfaceCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RAY_TRACING_PIPELINE_INTERFACE_CREATE_INFO_KHR
- VUID-VkRayTracingPipelineInterfaceCreateInfoKHR-pNext-pNext

`pNext` **must** be `NULL`

To query the opaque handles of shaders in the ray tracing pipeline, call:

```
// Provided by VK_KHR_ray_tracing_pipeline
VkResult vkGetRayTracingShaderGroupHandlesKHR(
    VkDevice                                    device,
    VkPipeline                                  pipeline,
    uint32_t                                    firstGroup,
    uint32_t                                    groupCount,
    size_t                                      dataSize,
    void*                                       pData);
```

- `device` is the logical device containing the ray tracing pipeline.
- `pipeline` is the ray tracing pipeline object containing the shaders.
- `firstGroup` is the index of the first group to retrieve a handle for from the VkRayTracingPipelineCreateInfoKHR::`pGroups` array.
- `groupCount` is the number of shader handles to retrieve.
- `dataSize` is the size in bytes of the buffer pointed to by `pData`.
- `pData` is a pointer to an application-allocated buffer where the results will be written.

On success, an array of `groupCount` shader handles will be written to `pData`, with each element being of size VkPhysicalDeviceRayTracingPipelinePropertiesKHR::`shaderGroupHandleSize`.

Valid Usage

- VUID-vkGetRayTracingShaderGroupHandlesKHR-pipeline-04619
  `pipeline` **must** be a ray tracing pipeline
- VUID-vkGetRayTracingShaderGroupHandlesKHR-firstGroup-04050
  `firstGroup` **must** be less than the number of shader groups in `pipeline`
- VUID-vkGetRayTracingShaderGroupHandlesKHR-firstGroup-02419
  The sum of `firstGroup` and `groupCount` **must** be less than or equal to the number of shader groups in `pipeline`
- VUID-vkGetRayTracingShaderGroupHandlesKHR-dataSize-02420
  `dataSize` **must** be at least VkPhysicalDeviceRayTracingPipelinePropertiesKHR::`shaderGroupHandleSize` × `groupCount`
- VUID-vkGetRayTracingShaderGroupHandlesKHR-pipeline-07828
  `pipeline` **must** not have been created with `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR`

Valid Usage (Implicit)

- VUID-vkGetRayTracingShaderGroupHandlesKHR-device-parameter

`device` **must** be a valid VkDevice handle

- VUID-vkGetRayTracingShaderGroupHandlesKHR-pipeline-parameter
  `pipeline` **must** be a valid VkPipeline handle
- VUID-vkGetRayTracingShaderGroupHandlesKHR-pData-parameter
  `pData` **must** be a valid pointer to an array of `dataSize` bytes
- VUID-vkGetRayTracingShaderGroupHandlesKHR-dataSize-arraylength
  `dataSize` **must** be greater than `0`
- VUID-vkGetRayTracingShaderGroupHandlesKHR-pipeline-parent
  `pipeline` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

To query the opaque capture data of shader groups in a ray tracing pipeline, call:

```
// Provided by VK_KHR_ray_tracing_pipeline
VkResult vkGetRayTracingCaptureReplayShaderGroupHandlesKHR(
    VkDevice                                    device,
    VkPipeline                                  pipeline,
    uint32_t                                    firstGroup,
    uint32_t                                    groupCount,
    size_t                                      dataSize,
    void*                                       pData);
```

- `device` is the logical device containing the ray tracing pipeline.
- `pipeline` is the ray tracing pipeline object containing the shaders.
- `firstGroup` is the index of the first group to retrieve a handle for from the VkRayTracingPipelineCreateInfoKHR::`pGroups` array.
- `groupCount` is the number of shader handles to retrieve.
- `dataSize` is the size in bytes of the buffer pointed to by `pData`.
- `pData` is a pointer to an application-allocated buffer where the results will be written.

On success, an array of `groupCount` shader handles will be written to `pData`, with each element being of size VkPhysicalDeviceRayTracingPipelinePropertiesKHR::`shaderGroupHandleCaptureReplaySize`.

Once queried, this opaque data **can** be provided at pipeline creation time (in a subsequent

execution), using VkRayTracingShaderGroupCreateInfoKHR::pShaderGroupCaptureReplayHandle, as described in Ray Tracing Capture Replay.

Valid Usage

- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-pipeline-04620
  `pipeline` **must** be a ray tracing pipeline
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-firstGroup-04051
  `firstGroup` **must** be less than the number of shader groups in `pipeline`
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-firstGroup-03483
  The sum of `firstGroup` and `groupCount` **must** be less than or equal to the number of shader groups in `pipeline`
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-dataSize-03484
  `dataSize` **must** be at least VkPhysicalDeviceRayTracingPipelinePropertiesKHR::`shaderGroupHandleCaptureReplaySize` × `groupCount`
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-rayTracingPipelineShaderGroupHandleCaptureReplay-03606
  `VkPhysicalDeviceRayTracingPipelineFeaturesKHR`::`rayTracingPipelineShaderGroupHandleCaptureReplay` **must** be enabled to call this function
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-pipeline-03607
  `pipeline` **must** have been created with a `flags` that included `VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR`
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-pipeline-07829
  `pipeline` **must** not have been created with `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR`

Valid Usage (Implicit)

- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-pipeline-parameter
  `pipeline` **must** be a valid VkPipeline handle
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-pData-parameter
  `pData` **must** be a valid pointer to an array of `dataSize` bytes
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-dataSize-arraylength
  `dataSize` **must** be greater than `0`
- VUID-vkGetRayTracingCaptureReplayShaderGroupHandlesKHR-pipeline-parent
  `pipeline` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

To query the pipeline stack size of shaders in a shader group in the ray tracing pipeline, call:

```
// Provided by VK_KHR_ray_tracing_pipeline
VkDeviceSize vkGetRayTracingShaderGroupStackSizeKHR(
    VkDevice                                    device,
    VkPipeline                                  pipeline,
    uint32_t                                    group,
    VkShaderGroupShaderKHR                      groupShader);
```

- `device` is the logical device containing the ray tracing pipeline.
- `pipeline` is the ray tracing pipeline object containing the shaders groups.
- `group` is the index of the shader group to query.
- `groupShader` is the type of shader from the group to query.

The return value is the ray tracing pipeline stack size in bytes for the specified shader as called from the specified shader group.

Valid Usage

- VUID-vkGetRayTracingShaderGroupStackSizeKHR-pipeline-04622
  `pipeline` **must** be a ray tracing pipeline
- VUID-vkGetRayTracingShaderGroupStackSizeKHR-group-03608
  The value of `group` **must** be less than the number of shader groups in `pipeline`
- VUID-vkGetRayTracingShaderGroupStackSizeKHR-groupShader-03609
  The shader identified by `groupShader` in `group` **must** not be `VK_SHADER_UNUSED_KHR`

Valid Usage (Implicit)

- VUID-vkGetRayTracingShaderGroupStackSizeKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetRayTracingShaderGroupStackSizeKHR-pipeline-parameter
  `pipeline` **must** be a valid VkPipeline handle
- VUID-vkGetRayTracingShaderGroupStackSizeKHR-groupShader-parameter

`groupShader` **must** be a valid VkShaderGroupShaderKHR value

- VUID-vkGetRayTracingShaderGroupStackSizeKHR-pipeline-parent
  `pipeline` **must** have been created, allocated, or retrieved from `device`

Possible values of `groupShader` in vkGetRayTracingShaderGroupStackSizeKHR are:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef enum VkShaderGroupShaderKHR {
    VK_SHADER_GROUP_SHADER_GENERAL_KHR = 0,
    VK_SHADER_GROUP_SHADER_CLOSEST_HIT_KHR = 1,
    VK_SHADER_GROUP_SHADER_ANY_HIT_KHR = 2,
    VK_SHADER_GROUP_SHADER_INTERSECTION_KHR = 3,
} VkShaderGroupShaderKHR;
```

- `VK_SHADER_GROUP_SHADER_GENERAL_KHR` uses the shader specified in the group with VkRayTracingShaderGroupCreateInfoKHR::`generalShader`
- `VK_SHADER_GROUP_SHADER_CLOSEST_HIT_KHR` uses the shader specified in the group with VkRayTracingShaderGroupCreateInfoKHR::`closestHitShader`
- `VK_SHADER_GROUP_SHADER_ANY_HIT_KHR` uses the shader specified in the group with VkRayTracingShaderGroupCreateInfoKHR::`anyHitShader`
- `VK_SHADER_GROUP_SHADER_INTERSECTION_KHR` uses the shader specified in the group with VkRayTracingShaderGroupCreateInfoKHR::`intersectionShader`

To dynamically set the stack size for a ray tracing pipeline, call:

```
// Provided by VK_KHR_ray_tracing_pipeline
void vkCmdSetRayTracingPipelineStackSizeKHR(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    pipelineStackSize);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pipelineStackSize` is the stack size to use for subsequent ray tracing trace commands.

This command sets the stack size for subsequent ray tracing commands when the ray tracing pipeline is created with `VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, the stack size is computed as described in Ray Tracing Pipeline Stack.

Valid Usage

- VUID-vkCmdSetRayTracingPipelineStackSizeKHR-pipelineStackSize-03610
  `pipelineStackSize` **must** be large enough for any dynamic execution through the shaders in the ray tracing pipeline used by a subsequent trace call

Valid Usage (Implicit)

- VUID-vkCmdSetRayTracingPipelineStackSizeKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetRayTracingPipelineStackSizeKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetRayTracingPipelineStackSizeKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdSetRayTracingPipelineStackSizeKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdSetRayTracingPipelineStackSizeKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	State

10.5. Pipeline Destruction

To destroy a pipeline, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyPipeline(
    VkDevice                                    device,
    VkPipeline                                  pipeline,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the pipeline.
- `pipeline` is the handle of the pipeline to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyPipeline-pipeline-00765
  All submitted commands that refer to `pipeline` **must** have completed execution
- VUID-vkDestroyPipeline-pipeline-00766
  If `VkAllocationCallbacks` were provided when `pipeline` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyPipeline-pipeline-00767
  If no `VkAllocationCallbacks` were provided when `pipeline` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyPipeline-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyPipeline-pipeline-parameter
  If `pipeline` is not VK_NULL_HANDLE, `pipeline` **must** be a valid VkPipeline handle
- VUID-vkDestroyPipeline-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyPipeline-pipeline-parent
  If `pipeline` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `pipeline` **must** be externally synchronized

10.6. Pipeline Derivatives

A pipeline derivative is a child pipeline created from a parent pipeline, where the child and parent are expected to have much commonality.

The goal of derivative pipelines is that they be cheaper to create using the parent as a starting point, and that it be more efficient (on either host or device) to switch/bind between children of the same parent.

A derivative pipeline is created by setting the `VK_PIPELINE_CREATE_DERIVATIVE_BIT` flag in the `Vk*PipelineCreateInfo` structure. If this is set, then exactly one of `basePipelineHandle` or `basePipelineIndex` members of the structure **must** have a valid handle/index, and specifies the parent pipeline. If `basePipelineHandle` is used, the parent pipeline **must** have already been created. If `basePipelineIndex` is used, then the parent is being created in the same command. VK_NULL_HANDLE acts as the invalid handle for `basePipelineHandle`, and -1 is the invalid index for

basePipelineIndex. If basePipelineIndex is used, the base pipeline **must** appear earlier in the array. The base pipeline **must** have been created with the VK_PIPELINE_CREATE_ALLOW_DERIVATIVES_BIT flag set.

10.7. Pipeline Cache

Pipeline cache objects allow the result of pipeline construction to be reused between pipelines and between runs of an application. Reuse between pipelines is achieved by passing the same pipeline cache object when creating multiple related pipelines. Reuse across runs of an application is achieved by retrieving pipeline cache contents in one run of an application, saving the contents, and using them to preinitialize a pipeline cache on a subsequent run. The contents of the pipeline cache objects are managed by the implementation. Applications **can** manage the host memory consumed by a pipeline cache object and control the amount of data retrieved from a pipeline cache object.

Pipeline cache objects are represented by VkPipelineCache handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkPipelineCache)
```

10.7.1. Creating a Pipeline Cache

To create pipeline cache objects, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreatePipelineCache(
    VkDevice                                    device,
    const VkPipelineCacheCreateInfo*            pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkPipelineCache*                            pPipelineCache);
```

- device is the logical device that creates the pipeline cache object.
- pCreateInfo is a pointer to a VkPipelineCacheCreateInfo structure containing initial parameters for the pipeline cache object.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.
- pPipelineCache is a pointer to a VkPipelineCache handle in which the resulting pipeline cache object is returned.

NOTE	Applications **can** track and manage the total host memory size of a pipeline cache object using the pAllocator. Applications **can** limit the amount of data retrieved from a pipeline cache object in vkGetPipelineCacheData. Implementations **should** not internally limit the total number of entries added to a pipeline cache object or the total host memory consumed.

Once created, a pipeline cache **can** be passed to the vkCreateGraphicsPipelines

vkCreateRayTracingPipelinesKHR, and vkCreateComputePipelines commands. If the pipeline cache passed into these commands is not VK_NULL_HANDLE, the implementation will query it for possible reuse opportunities and update it with new content. The use of the pipeline cache object in these commands is internally synchronized, and the same pipeline cache object **can** be used in multiple threads simultaneously.

NOTE	Implementations **should** make every effort to limit any critical sections to the actual accesses to the cache, which is expected to be significantly shorter than the duration of the `vkCreate*Pipelines` commands.

Valid Usage (Implicit)

- VUID-vkCreatePipelineCache-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreatePipelineCache-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkPipelineCacheCreateInfo structure
- VUID-vkCreatePipelineCache-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreatePipelineCache-pPipelineCache-parameter
  `pPipelineCache` **must** be a valid pointer to a VkPipelineCache handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkPipelineCacheCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineCacheCreateInfo {
    VkStructureType              sType;
    const void*                  pNext;
    VkPipelineCacheCreateFlags   flags;
    size_t                       initialDataSize;
    const void*                  pInitialData;
} VkPipelineCacheCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `initialDataSize` is the number of bytes in `pInitialData`. If `initialDataSize` is zero, the pipeline cache will initially be empty.
- `pInitialData` is a pointer to previously retrieved pipeline cache data. If the pipeline cache data is incompatible (as defined below) with the device, the pipeline cache will be initially empty. If `initialDataSize` is zero, `pInitialData` is ignored.

Valid Usage

- VUID-VkPipelineCacheCreateInfo-initialDataSize-00768
  If `initialDataSize` is not `0`, it **must** be equal to the size of `pInitialData`, as returned by `vkGetPipelineCacheData` when `pInitialData` was originally retrieved
- VUID-VkPipelineCacheCreateInfo-initialDataSize-00769
  If `initialDataSize` is not `0`, `pInitialData` **must** have been retrieved from a previous call to `vkGetPipelineCacheData`

Valid Usage (Implicit)

- VUID-VkPipelineCacheCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_CACHE_CREATE_INFO`
- VUID-VkPipelineCacheCreateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPipelineCacheCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineCacheCreateInfo-pInitialData-parameter
  If `initialDataSize` is not `0`, `pInitialData` **must** be a valid pointer to an array of `initialDataSize` bytes

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineCacheCreateFlags;
```

`VkPipelineCacheCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

10.7.2. Merging Pipeline Caches

Pipeline cache objects **can** be merged using the command:

```
// Provided by VK_VERSION_1_0
VkResult vkMergePipelineCaches(
    VkDevice                                    device,
    VkPipelineCache                             dstCache,
    uint32_t                                    srcCacheCount,
    const VkPipelineCache*                      pSrcCaches);
```

- `device` is the logical device that owns the pipeline cache objects.
- `dstCache` is the handle of the pipeline cache to merge results into.
- `srcCacheCount` is the length of the `pSrcCaches` array.
- `pSrcCaches` is a pointer to an array of pipeline cache handles, which will be merged into `dstCache`. The previous contents of `dstCache` are included after the merge.

NOTE	The details of the merge operation are implementation-dependent, but implementations **should** merge the contents of the specified pipelines and prune duplicate entries.

Valid Usage

- VUID-vkMergePipelineCaches-dstCache-00770
  `dstCache` **must** not appear in the list of source caches

Valid Usage (Implicit)

- VUID-vkMergePipelineCaches-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkMergePipelineCaches-dstCache-parameter
  `dstCache` **must** be a valid VkPipelineCache handle
- VUID-vkMergePipelineCaches-pSrcCaches-parameter
  `pSrcCaches` **must** be a valid pointer to an array of `srcCacheCount` valid VkPipelineCache handles
- VUID-vkMergePipelineCaches-srcCacheCount-arraylength
  `srcCacheCount` **must** be greater than `0`
- VUID-vkMergePipelineCaches-dstCache-parent
  `dstCache` **must** have been created, allocated, or retrieved from `device`
- VUID-vkMergePipelineCaches-pSrcCaches-parent
  Each element of `pSrcCaches` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `dstCache` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

10.7.3. Retrieving Pipeline Cache Data

Data **can** be retrieved from a pipeline cache object using the command:

```
// Provided by VK_VERSION_1_0
VkResult vkGetPipelineCacheData(
    VkDevice                                    device,
    VkPipelineCache                             pipelineCache,
    size_t*                                     pDataSize,
    void*                                       pData);
```

- `device` is the logical device that owns the pipeline cache.
- `pipelineCache` is the pipeline cache to retrieve data from.
- `pDataSize` is a pointer to a `size_t` value related to the amount of data in the pipeline cache, as described below.
- `pData` is either `NULL` or a pointer to a buffer.

If `pData` is `NULL`, then the maximum size of the data that **can** be retrieved from the pipeline cache, in bytes, is returned in `pDataSize`. Otherwise, `pDataSize` **must** point to a variable set by the application to the size of the buffer, in bytes, pointed to by `pData`, and on return the variable is overwritten with the amount of data actually written to `pData`. If `pDataSize` is less than the maximum size that **can** be retrieved by the pipeline cache, at most `pDataSize` bytes will be written to `pData`, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all of the pipeline cache was returned.

Any data written to `pData` is valid and **can** be provided as the `pInitialData` member of the VkPipelineCacheCreateInfo structure passed to `vkCreatePipelineCache`.

Two calls to `vkGetPipelineCacheData` with the same parameters **must** retrieve the same data unless a command that modifies the contents of the cache is called between them.

The initial bytes written to `pData` **must** be a header as described in the Pipeline Cache Header section.

If `pDataSize` is less than what is necessary to store this header, nothing will be written to `pData` and zero will be written to `pDataSize`.

Valid Usage (Implicit)

- VUID-vkGetPipelineCacheData-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetPipelineCacheData-pipelineCache-parameter
  `pipelineCache` **must** be a valid VkPipelineCache handle
- VUID-vkGetPipelineCacheData-pDataSize-parameter
  `pDataSize` **must** be a valid pointer to a `size_t` value
- VUID-vkGetPipelineCacheData-pData-parameter
  If the value referenced by `pDataSize` is not `0`, and `pData` is not `NULL`, `pData` **must** be a valid pointer to an array of `pDataSize` bytes
- VUID-vkGetPipelineCacheData-pipelineCache-parent
  `pipelineCache` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

10.7.4. Pipeline Cache Header

Applications **can** store the data retrieved from the pipeline cache, and use these data, possibly in a future run of the application, to populate new pipeline cache objects. The results of pipeline compiles, however, **may** depend on the vendor ID, device ID, driver version, and other details of the device. To enable applications to detect when previously retrieved data is incompatible with the device, the pipeline cache data **must** begin with a valid pipeline cache header.

NOTE

Structures described in this section are not part of the Vulkan API and are only used to describe the representation of data elements in pipeline cache data. Accordingly, the valid usage clauses defined for structures defined in this section do not define valid usage conditions for APIs accepting pipeline cache data as input, as providing invalid pipeline cache data as input to any Vulkan API commands will result in the provided pipeline cache data being ignored.

Version one of the pipeline cache header is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineCacheHeaderVersionOne {
    uint32_t                        headerSize;
    VkPipelineCacheHeaderVersion    headerVersion;
    uint32_t                        vendorID;
    uint32_t                        deviceID;
    uint8_t                         pipelineCacheUUID[VK_UUID_SIZE];
} VkPipelineCacheHeaderVersionOne;
```

- `headerSize` is the length in bytes of the pipeline cache header.
- `headerVersion` is a VkPipelineCacheHeaderVersion value specifying the version of the header. A consumer of the pipeline cache **should** use the cache version to interpret the remainder of the cache header.
- `vendorID` is the `VkPhysicalDeviceProperties`::`vendorID` of the implementation.
- `deviceID` is the `VkPhysicalDeviceProperties`::`deviceID` of the implementation.
- `pipelineCacheUUID` is the `VkPhysicalDeviceProperties`::`pipelineCacheUUID` of the implementation.

Unlike most structures declared by the Vulkan API, all fields of this structure are written with the least significant byte first, regardless of host byte-order.

The C language specification does not define the packing of structure members. This layout assumes tight structure member packing, with members laid out in the order listed in the structure, and the intended size of the structure is 32 bytes. If a compiler produces code that diverges from that pattern, applications **must** employ another method to set values at the correct offsets.

Valid Usage

- VUID-VkPipelineCacheHeaderVersionOne-headerSize-04967
  `headerSize` **must** be 32
- VUID-VkPipelineCacheHeaderVersionOne-headerVersion-04968
  `headerVersion` **must** be `VK_PIPELINE_CACHE_HEADER_VERSION_ONE`
- VUID-VkPipelineCacheHeaderVersionOne-headerSize-08990
  `headerSize` **must** not exceed the size of the pipeline cache

Valid Usage (Implicit)

- VUID-VkPipelineCacheHeaderVersionOne-headerVersion-parameter
  `headerVersion` **must** be a valid VkPipelineCacheHeaderVersion value

Possible values of the `headerVersion` value of the pipeline cache header are:

```
// Provided by VK_VERSION_1_0
typedef enum VkPipelineCacheHeaderVersion {
    VK_PIPELINE_CACHE_HEADER_VERSION_ONE = 1,
} VkPipelineCacheHeaderVersion;
```

- VK_PIPELINE_CACHE_HEADER_VERSION_ONE specifies version one of the pipeline cache, described by VkPipelineCacheHeaderVersionOne.

10.7.5. Destroying a Pipeline Cache

To destroy a pipeline cache, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyPipelineCache(
    VkDevice                                    device,
    VkPipelineCache                             pipelineCache,
    const VkAllocationCallbacks*                pAllocator);
```

- device is the logical device that destroys the pipeline cache object.
- pipelineCache is the handle of the pipeline cache to destroy.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyPipelineCache-pipelineCache-00771
  If VkAllocationCallbacks were provided when pipelineCache was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyPipelineCache-pipelineCache-00772
  If no VkAllocationCallbacks were provided when pipelineCache was created, pAllocator **must** be NULL

Valid Usage (Implicit)

- VUID-vkDestroyPipelineCache-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkDestroyPipelineCache-pipelineCache-parameter
  If pipelineCache is not VK_NULL_HANDLE, pipelineCache **must** be a valid VkPipelineCache handle
- VUID-vkDestroyPipelineCache-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyPipelineCache-pipelineCache-parent
  If pipelineCache is a valid handle, it **must** have been created, allocated, or retrieved from

device

Host Synchronization

- Host access to pipelineCache **must** be externally synchronized

10.8. Pipeline Binaries

Pipeline binary objects allow the result of pipeline construction to be reused between pipelines and between runs of an application. Reuse is achieved by extracting pipeline binaries from a VkPipeline object, associating them with a corresponding VkPipelineBinaryKeyKHR and then adding a VkPipelineBinaryInfoKHR to the pNext chain of any Vk*PipelineCreateInfo when creating a pipeline. Pipeline binaries can be reused between runs by extracting VkPipelineBinaryDataKHR from VkPipelineBinaryKHR objects, saving the contents, and then using them to create a VkPipelineBinaryKHR object on subsequent runs.

When creating a pipeline that includes VkPipelineBinaryInfoKHR in the pNext chain, or has the VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR flag set, the use of VkPipelineCache objects is not allowed.

Pipeline binary objects are represented by VkPipelineBinaryKHR handles:

```
// Provided by VK_KHR_pipeline_binary
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkPipelineBinaryKHR)
```

10.8.1. Generating the Pipeline Key

To generate the key for a particular pipeline creation info, call:

```
// Provided by VK_KHR_pipeline_binary
VkResult vkGetPipelineKeyKHR(
    VkDevice                                    device,
    const VkPipelineCreateInfoKHR*              pPipelineCreateInfo,
    VkPipelineBinaryKeyKHR*                     pPipelineKey);
```

- device is the logical device that creates the pipeline object.
- pPipelineCreateInfo is NULL or a pointer to a VkPipelineCreateInfoKHR structure.
- pPipelineKey is a pointer to a VkPipelineBinaryKeyKHR structure in which the resulting key is returned.

If pPipelineCreateInfo is NULL, then the implementation **must** return the global key that applies to all pipelines. If the key obtained in this way changes between saving and restoring data obtained from vkGetPipelineBinaryDataKHR in a different VkDevice, then the application **must** assume that the restored data is invalid and cannot be passed to vkCreatePipelineBinariesKHR. Otherwise the

application **can** assume the data is still valid.

If pPipelineCreateInfo is not NULL, the key obtained functions as a method to compare two pipeline creation info structures. Implementations **may** not compare parts of a pipeline creation info which would not contribute to the final binary output. If a shader module identifier is used instead of a shader module, the pPipelineKey generated **must** be equal to the key generated when using the shader module from which the identifier was queried. If the content of two pPipelineKey are equal, pipelines created with the two pPipelineCreateInfo->pname:pNext create infos **must** produce the same VkPipelineBinaryKHR contents.

The pipeline key is distinct from pipeline binary key. Pipeline binary keys **can** only be obtained after compilation. The pipeline key is intended to optionally allow associating pipeline create info with multiple pipeline binary keys.

Valid Usage

- VUID-vkGetPipelineKeyKHR-pNext-09605
  The pNext chain of pPipelineCreateInfo **must** not set VkPipelineBinaryInfoKHR ::binaryCount to a value greater than 0

Valid Usage (Implicit)

- VUID-vkGetPipelineKeyKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetPipelineKeyKHR-pPipelineCreateInfo-parameter
  If pPipelineCreateInfo is not NULL, pPipelineCreateInfo **must** be a valid pointer to a valid VkPipelineCreateInfoKHR structure
- VUID-vkGetPipelineKeyKHR-pPipelineKey-parameter
  pPipelineKey **must** be a valid pointer to a VkPipelineBinaryKeyKHR structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkPipelineCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPipelineCreateInfoKHR {
    VkStructureType    sType;
    void*              pNext;
} VkPipelineCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is a pointer to a structure extending this structure.

Valid Usage

- VUID-VkPipelineCreateInfoKHR-pNext-09604
  `pNext` **must** be pointer to a valid instance of VkGraphicsPipelineCreateInfo, VkRayTracingPipelineCreateInfoKHR, or VkComputePipelineCreateInfo

Valid Usage (Implicit)

- VUID-VkPipelineCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_CREATE_INFO_KHR`

The `VkPipelineBinaryKeyKHR` structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPipelineBinaryKeyKHR {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           keySize;
    uint8_t            key[VK_MAX_PIPELINE_BINARY_KEY_SIZE_KHR];
} VkPipelineBinaryKeyKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `keySize` is the size, in bytes, of valid data returned in `key`.
- `key` is a buffer of opaque data specifying a pipeline binary key.

Any returned values beyond the first `keySize` bytes are undefined. Implementations **must** return a `keySize` greater than 0, and less-or-equal to `VK_MAX_PIPELINE_BINARY_KEY_SIZE_KHR`.

Two keys are considered equal if `keySize` is equal and the first `keySize` bytes of `key` compare equal.

Implementations **may** return a different `keySize` for different binaries.

Implementations **should** ensure that `keySize` is large enough to uniquely identify a pipeline binary.

Valid Usage (Implicit)

- VUID-VkPipelineBinaryKeyKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_BINARY_KEY_KHR`
- VUID-VkPipelineBinaryKeyKHR-pNext-pNext
  `pNext` **must** be `NULL`

`VK_MAX_PIPELINE_BINARY_KEY_SIZE_KHR` is the length in bytes of a binary key, as returned in VkPipelineBinaryKeyKHR::`keySize`.

```
#define VK_MAX_PIPELINE_BINARY_KEY_SIZE_KHR 32U
```

10.8.2. Creating Pipeline Binaries

To create pipeline binary objects, call:

```
// Provided by VK_KHR_pipeline_binary
VkResult vkCreatePipelineBinariesKHR(
    VkDevice                                    device,
    const VkPipelineBinaryCreateInfoKHR*        pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkPipelineBinaryHandlesInfoKHR*             pBinaries);
```

- `device` is the logical device that creates the pipeline binary objects.
- `pCreateInfo` is a pointer to a VkPipelineBinaryCreateInfoKHR structure that contains the data to create the pipeline binaries from.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pBinaries` is a pointer to a VkPipelineBinaryHandlesInfoKHR structure in which the resulting pipeline binaries are returned.

The implementation will attempt to create all pipeline binaries. If creation fails for any pipeline binary, then:

- The corresponding entry in the `pPipelineBinaries` output array will be filled with VK_NULL_HANDLE.
- The `VkResult` returned by vkCreatePipelineBinariesKHR will contain the error value for the first entry in the output array in `pBinaries` containing VK_NULL_HANDLE.

Valid Usage (Implicit)

- VUID-vkCreatePipelineBinariesKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreatePipelineBinariesKHR-pCreateInfo-parameter

pCreateInfo **must** be a valid pointer to a valid VkPipelineBinaryCreateInfoKHR structure

- VUID-vkCreatePipelineBinariesKHR-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreatePipelineBinariesKHR-pBinaries-parameter
  pBinaries **must** be a valid pointer to a VkPipelineBinaryHandlesInfoKHR structure

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE
- VK_PIPELINE_BINARY_MISSING_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INITIALIZATION_FAILED

The VkPipelineBinaryHandlesInfoKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPipelineBinaryHandlesInfoKHR {
    VkStructureType         sType;
    const void*             pNext;
    uint32_t                pipelineBinaryCount;
    VkPipelineBinaryKHR*    pPipelineBinaries;
} VkPipelineBinaryHandlesInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pipelineBinaryCount is the number of binaries associated with this pipeline or the number of entries in the pPipelineBinaries array.
- pPipelineBinaries is NULL or a pointer to an array of VkPipelineBinaryKHR handles in which the resulting pipeline binaries are returned.

If pPipelineBinaries is NULL, the number of binaries that would be created is returned in pipelineBinaryCount. Otherwise, pipelineBinaryCount **must** be the number of entries in the pPipelineBinaries array, and on return from vkCreatePipelineBinariesKHR pipelineBinaryCount is overwritten with the number of handles actually written to pPipelineBinaries. If the value of pipelineBinaryCount is less than the number of binaries that would have been created, at most pipelineBinaryCount handles will be written to pPipelineBinaries and VK_INCOMPLETE will be returned instead of VK_SUCCESS, to indicate that pPipelineBinaries was not large enough to create all

the binaries.

Valid Usage (Implicit)

- VUID-VkPipelineBinaryHandlesInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_BINARY_HANDLES_INFO_KHR`
- VUID-VkPipelineBinaryHandlesInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPipelineBinaryHandlesInfoKHR-pPipelineBinaries-parameter
  If `pipelineBinaryCount` is not `0`, and `pPipelineBinaries` is not `NULL`, `pPipelineBinaries` **must** be a valid pointer to an array of `pipelineBinaryCount` VkPipelineBinaryKHR handles

The `VkPipelineBinaryCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPipelineBinaryCreateInfoKHR {
    VkStructureType                           sType;
    const void*                               pNext;
    const VkPipelineBinaryKeysAndDataKHR*     pKeysAndDataInfo;
    VkPipeline                                pipeline;
    const VkPipelineCreateInfoKHR*            pPipelineCreateInfo;
} VkPipelineBinaryCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pKeysAndDataInfo` is `NULL` or a pointer to a VkPipelineBinaryKeysAndDataKHR structure that contains keys and data to create the pipeline binaries from.
- `pipeline` is VK_NULL_HANDLE or a `VkPipeline` that contains data to create the pipeline binaries from.
- `pPipelineCreateInfo` is `NULL` or a pointer to a VkPipelineCreateInfoKHR structure with the pipeline creation info. This is used to probe the implementation's internal cache for pipeline binaries.

When `pPipelineCreateInfo` is not `NULL`, an implementation will attempt to retrieve pipeline binary data from an internal cache external to the application if `pipelineBinaryInternalCache` is `VK_TRUE`. Applications **can** use this to determine if a pipeline **can** be created without compilation. If the implementation fails to create a pipeline binary due to missing an internal cache entry, `VK_PIPELINE_BINARY_MISSING_KHR` is returned. If creation succeeds, the resulting binary **can** be used to create a pipeline. `VK_PIPELINE_BINARY_MISSING_KHR` **may** be returned for any reason in this situation, even if creating a pipeline binary with the same parameters that succeeded earlier.

If `pipelineBinaryPrecompiledInternalCache` is `VK_TRUE`, the implementation **may** be able to create pipeline binaries even when `pPipelineCreateInfo` has not been used to create binaries before by the application.

NOTE On some platforms, internal pipeline caches may be pre-populated before running the application.

Valid Usage

- VUID-VkPipelineBinaryCreateInfoKHR-pipeline-09607
  If `pipeline` is not VK_NULL_HANDLE, `pipeline` **must** have been created with `VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR`
- VUID-VkPipelineBinaryCreateInfoKHR-pipeline-09608
  If `pipeline` is not VK_NULL_HANDLE, vkReleaseCapturedPipelineDataKHR **must** not have been called on `pipeline` prior to this command
- VUID-VkPipelineBinaryCreateInfoKHR-pipelineBinaryInternalCache-09609
  If `pipelineBinaryInternalCache` is `VK_FALSE` pPipelineCreateInfo **must** be `NULL`
- VUID-VkPipelineBinaryCreateInfoKHR-device-09610
  If `device` was created with VkDevicePipelineBinaryInternalCacheControlKHR ::`disableInternalCache` set to `VK_TRUE`, `pPipelineCreateInfo` **must** be `NULL`
- VUID-VkPipelineBinaryCreateInfoKHR-pKeysAndDataInfo-09619
  One and only one of `pKeysAndDataInfo`, `pipeline`, or `pPipelineCreateInfo` **must** be non-`NULL`
- VUID-VkPipelineBinaryCreateInfoKHR-pPipelineCreateInfo-09606
  If `pPipelineCreateInfo` is not `NULL`, the `pNext` chain of `pPipelineCreateInfo` **must** not set VkPipelineBinaryInfoKHR::`binaryCount` to a value greater than `0`

Valid Usage (Implicit)

- VUID-VkPipelineBinaryCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_BINARY_CREATE_INFO_KHR`
- VUID-VkPipelineBinaryCreateInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPipelineBinaryCreateInfoKHR-pKeysAndDataInfo-parameter
  If `pKeysAndDataInfo` is not `NULL`, `pKeysAndDataInfo` **must** be a valid pointer to a valid VkPipelineBinaryKeysAndDataKHR structure
- VUID-VkPipelineBinaryCreateInfoKHR-pipeline-parameter
  If `pipeline` is not VK_NULL_HANDLE, `pipeline` **must** be a valid VkPipeline handle
- VUID-VkPipelineBinaryCreateInfoKHR-pPipelineCreateInfo-parameter
  If `pPipelineCreateInfo` is not `NULL`, `pPipelineCreateInfo` **must** be a valid pointer to a valid VkPipelineCreateInfoKHR structure

The `VkPipelineBinaryKeysAndDataKHR` structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPipelineBinaryKeysAndDataKHR {
    uint32_t                            binaryCount;
    const VkPipelineBinaryKeyKHR*       pPipelineBinaryKeys;
    const VkPipelineBinaryDataKHR*      pPipelineBinaryData;
} VkPipelineBinaryKeysAndDataKHR;
```

- `binaryCount` is the size of the `pPipelineBinaryKeys` and `pPipelineBinaryData` arrays
- `pPipelineBinaryKeys` is a pointer to an array of VkPipelineBinaryKeyKHR structures containing the pipeline binary keys
- `pPipelineBinaryData` is a pointer to an array of VkPipelineBinaryDataKHR structures containing the pipeline binary data

Valid Usage (Implicit)

- VUID-VkPipelineBinaryKeysAndDataKHR-pPipelineBinaryKeys-parameter
  `pPipelineBinaryKeys` **must** be a valid pointer to an array of `binaryCount` valid VkPipelineBinaryKeyKHR structures
- VUID-VkPipelineBinaryKeysAndDataKHR-pPipelineBinaryData-parameter
  `pPipelineBinaryData` **must** be a valid pointer to an array of `binaryCount` valid VkPipelineBinaryDataKHR structures
- VUID-VkPipelineBinaryKeysAndDataKHR-binaryCount-arraylength
  `binaryCount` **must** be greater than `0`

The `VkPipelineBinaryDataKHR` structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPipelineBinaryDataKHR {
    size_t    dataSize;
    void*     pData;
} VkPipelineBinaryDataKHR;
```

- `dataSize` is the size of the `pData` buffer in bytes.
- `pData` is a pointer to a buffer of `size` bytes that contains pipeline binary data obtained from `vkGetPipelineBinaryDataKHR`.

Valid Usage (Implicit)

- VUID-VkPipelineBinaryDataKHR-pData-parameter
  `pData` **must** be a valid pointer to an array of `dataSize` bytes
- VUID-VkPipelineBinaryDataKHR-dataSize-arraylength
  `dataSize` **must** be greater than `0`

10.8.3. Retrieving Pipeline Binary Data

Data **can** be retrieved from a pipeline binary object using the command:

```
// Provided by VK_KHR_pipeline_binary
VkResult vkGetPipelineBinaryDataKHR(
    VkDevice                                    device,
    const VkPipelineBinaryDataInfoKHR*          pInfo,
    VkPipelineBinaryKeyKHR*                     pPipelineBinaryKey,
    size_t*                                     pPipelineBinaryDataSize,
    void*                                       pPipelineBinaryData);
```

- `device` is the logical device that created the pipeline binary.
- `pInfo` is a pointer to a VkPipelineBinaryDataInfoKHR structure which describes the pipeline binary to get data from.
- `pPipelineBinaryKey` is a pointer to a VkPipelineBinaryKeyKHR structure where the key for this binary will be written.
- `pPipelineBinaryDataSize` is a pointer to a `size_t` value related to the amount of data in the pipeline binary, as described below.
- `pPipelineBinaryData` is either `NULL` or a pointer to a buffer.

If `pPipelineBinaryData` is `NULL`, then the size of the data, in bytes, that is required to store the binary is returned in `pPipelineBinaryDataSize`. Otherwise, `pPipelineBinaryDataSize` **must** contain the size of the buffer, in bytes, pointed to by `pPipelineBinaryData`, and on return `pPipelineBinaryDataSize` is overwritten with the size of the data, in bytes, that is required to store the binary. If `pPipelineBinaryDataSize` is less than the size that is required to store the binary, nothing is written to `pPipelineBinaryData` and `VK_ERROR_NOT_ENOUGH_SPACE_KHR` will be returned, instead of `VK_SUCCESS`.

If pipelineBinaryCompressedData is `VK_FALSE`, implementations **should** not return compressed pipeline binary data to the application.

Valid Usage (Implicit)

- VUID-vkGetPipelineBinaryDataKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetPipelineBinaryDataKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkPipelineBinaryDataInfoKHR structure
- VUID-vkGetPipelineBinaryDataKHR-pPipelineBinaryKey-parameter
  `pPipelineBinaryKey` **must** be a valid pointer to a VkPipelineBinaryKeyKHR structure
- VUID-vkGetPipelineBinaryDataKHR-pPipelineBinaryDataSize-parameter
  `pPipelineBinaryDataSize` **must** be a valid pointer to a `size_t` value
- VUID-vkGetPipelineBinaryDataKHR-pPipelineBinaryData-parameter
  If the value referenced by `pPipelineBinaryDataSize` is not `0`, and `pPipelineBinaryData` is not `NULL`, `pPipelineBinaryData` **must** be a valid pointer to an array of `pPipelineBinaryDataSize`

bytes

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_NOT_ENOUGH_SPACE_KHR

The VkPipelineBinaryDataInfoKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPipelineBinaryDataInfoKHR {
    VkStructureType        sType;
    void*                  pNext;
    VkPipelineBinaryKHR    pipelineBinary;
} VkPipelineBinaryDataInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pipelineBinary is the pipeline binary to get data from.

Valid Usage (Implicit)

- VUID-VkPipelineBinaryDataInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_BINARY_DATA_INFO_KHR
- VUID-VkPipelineBinaryDataInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkPipelineBinaryDataInfoKHR-pipelineBinary-parameter
  pipelineBinary **must** be a valid VkPipelineBinaryKHR handle

10.8.4. Releasing Captured Pipeline Binary Data

To release pipeline resources captured with VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR, call:

```
// Provided by VK_KHR_pipeline_binary
VkResult vkReleaseCapturedPipelineDataKHR(
    VkDevice                                    device,
    const VkReleaseCapturedPipelineDataInfoKHR* pInfo,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that created the pipeline object.
- `pInfo` is a pointer to a VkReleaseCapturedPipelineDataInfoKHR structure which describes the pipeline to release the data from.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

The implementation **may** free any resources captured as a result of creating the pipeline with `VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR` and put the pipeline into a state as if `VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR` had not been provided at pipeline creation time.

NOTE	Any resources captured as a result of creating the pipeline with `VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR` are implicitly freed by vkDestroyPipeline.

Valid Usage

- VUID-vkReleaseCapturedPipelineDataKHR-pipeline-09611
  If `VkAllocationCallbacks` were provided when `pipeline` was created, a compatible set of callbacks **must** be provided in `pAllocator`
- VUID-vkReleaseCapturedPipelineDataKHR-pipeline-09612
  If no VkAllocationCallbacks were provided when `pipeline` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkReleaseCapturedPipelineDataKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkReleaseCapturedPipelineDataKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkReleaseCapturedPipelineDataInfoKHR structure
- VUID-vkReleaseCapturedPipelineDataKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure

Host Synchronization

- Host access to `pInfo->pipeline` **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS

Failure

None

The VkReleaseCapturedPipelineDataInfoKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkReleaseCapturedPipelineDataInfoKHR {
    VkStructureType    sType;
    void*              pNext;
    VkPipeline         pipeline;
} VkReleaseCapturedPipelineDataInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pipeline the handle of the pipeline object to release the data from.

Valid Usage

- VUID-VkReleaseCapturedPipelineDataInfoKHR-pipeline-09613
  pipeline **must** have been created with VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR
- VUID-VkReleaseCapturedPipelineDataInfoKHR-pipeline-09618
  pipeline **must** not have been used in a previous call to vkReleaseCapturedPipelineDataKHR

Valid Usage (Implicit)

- VUID-VkReleaseCapturedPipelineDataInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RELEASE_CAPTURED_PIPELINE_DATA_INFO_KHR
- VUID-VkReleaseCapturedPipelineDataInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkReleaseCapturedPipelineDataInfoKHR-pipeline-parameter
  pipeline **must** be a valid VkPipeline handle

10.8.5. Destroying Pipeline Binaries

To destroy a VkPipelineBinaryKHR, call:

```
// Provided by VK_KHR_pipeline_binary
void vkDestroyPipelineBinaryKHR(
    VkDevice                                    device,
    VkPipelineBinaryKHR                         pipelineBinary,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that created the pipeline binary object.
- `pipelineBinary` is the handle of the pipeline binary object to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyPipelineBinaryKHR-pipelineBinary-09614
  If `VkAllocationCallbacks` were provided when `pipelineBinary` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyPipelineBinaryKHR-pipelineBinary-09615
  If no VkAllocationCallbacks were provided when `pipelineBinary` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyPipelineBinaryKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyPipelineBinaryKHR-pipelineBinary-parameter
  If `pipelineBinary` is not VK_NULL_HANDLE, `pipelineBinary` **must** be a valid VkPipelineBinaryKHR handle
- VUID-vkDestroyPipelineBinaryKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyPipelineBinaryKHR-pipelineBinary-parent
  If `pipelineBinary` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `pipelineBinary` **must** be externally synchronized

10.9. Specialization Constants

Specialization constants are a mechanism whereby constants in a SPIR-V module **can** have their constant value specified at the time the `VkPipeline` is created. This allows a SPIR-V module to have

constants that **can** be modified while executing an application that uses the Vulkan API.

NOTE	Specialization constants are useful to allow a compute shader to have its local workgroup size changed at runtime by the user, for example.

Each VkPipelineShaderStageCreateInfo structure contains a `pSpecializationInfo` member, which **can** be `NULL` to indicate no specialization constants, or point to a `VkSpecializationInfo` structure.

The `VkSpecializationInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSpecializationInfo {
    uint32_t                           mapEntryCount;
    const VkSpecializationMapEntry*    pMapEntries;
    size_t                             dataSize;
    const void*                        pData;
} VkSpecializationInfo;
```

- `mapEntryCount` is the number of entries in the `pMapEntries` array.
- `pMapEntries` is a pointer to an array of `VkSpecializationMapEntry` structures, which map constant IDs to offsets in `pData`.
- `dataSize` is the byte size of the `pData` buffer.
- `pData` contains the actual constant values to specialize with.

Valid Usage

- VUID-VkSpecializationInfo-offset-00773
  The `offset` member of each element of `pMapEntries` **must** be less than `dataSize`
- VUID-VkSpecializationInfo-pMapEntries-00774
  The `size` member of each element of `pMapEntries` **must** be less than or equal to `dataSize` minus `offset`
- VUID-VkSpecializationInfo-constantID-04911
  The `constantID` value of each element of `pMapEntries` **must** be unique within `pMapEntries`

Valid Usage (Implicit)

- VUID-VkSpecializationInfo-pMapEntries-parameter
  If `mapEntryCount` is not `0`, `pMapEntries` **must** be a valid pointer to an array of `mapEntryCount` valid VkSpecializationMapEntry structures
- VUID-VkSpecializationInfo-pData-parameter
  If `dataSize` is not `0`, `pData` **must** be a valid pointer to an array of `dataSize` bytes

The `VkSpecializationMapEntry` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSpecializationMapEntry {
    uint32_t    constantID;
    uint32_t    offset;
    size_t      size;
} VkSpecializationMapEntry;
```

- `constantID` is the ID of the specialization constant in SPIR-V.
- `offset` is the byte offset of the specialization constant value within the supplied data buffer.
- `size` is the byte size of the specialization constant value within the supplied data buffer.

If a `constantID` value is not a specialization constant ID used in the shader, that map entry does not affect the behavior of the pipeline.

Valid Usage

- VUID-VkSpecializationMapEntry-constantID-00776
  For a `constantID` specialization constant declared in a shader, `size` **must** match the byte size of the `constantID`. If the specialization constant is of type `boolean`, `size` **must** be the byte size of VkBool32

In human readable SPIR-V:

```
OpDecorate %x SpecId 13 ; decorate .x component of WorkgroupSize with ID 13
OpDecorate %y SpecId 42 ; decorate .y component of WorkgroupSize with ID 42
OpDecorate %z SpecId 3  ; decorate .z component of WorkgroupSize with ID 3
OpDecorate %wgsize BuiltIn WorkgroupSize ; decorate WorkgroupSize onto constant
%i32 = OpTypeInt 32 0 ; declare an unsigned 32-bit type
%uvec3 = OpTypeVector %i32 3 ; declare a 3 element vector type of unsigned 32-bit
%x = OpSpecConstant %i32 1 ; declare the .x component of WorkgroupSize
%y = OpSpecConstant %i32 1 ; declare the .y component of WorkgroupSize
%z = OpSpecConstant %i32 1 ; declare the .z component of WorkgroupSize
%wgsize = OpSpecConstantComposite %uvec3 %x %y %z ; declare WorkgroupSize
```

From the above we have three specialization constants, one for each of the x, y & z elements of the WorkgroupSize vector.

Now to specialize the above via the specialization constants mechanism:

```
const VkSpecializationMapEntry entries[] =
{
    {
        .constantID = 13,
        .offset = 0 * sizeof(uint32_t),
        .size = sizeof(uint32_t)
```

```
    },
    {
        .constantID = 42,
        .offset = 1 * sizeof(uint32_t),
        .size = sizeof(uint32_t)
    },
    {
        .constantID = 3,
        .offset = 2 * sizeof(uint32_t),
        .size = sizeof(uint32_t)
    }
};

const uint32_t data[] = { 16, 8, 4 }; // our workgroup size is 16x8x4

const VkSpecializationInfo info =
{
    .mapEntryCount = 3,
    .pMapEntries  = entries,
    .dataSize = 3 * sizeof(uint32_t),
    .pData = data,
};
```

Then when calling vkCreateComputePipelines, and passing the VkSpecializationInfo we defined as the pSpecializationInfo parameter of VkPipelineShaderStageCreateInfo, we will create a compute pipeline with the runtime specified local workgroup size.

Another example would be that an application has a SPIR-V module that has some platform-dependent constants they wish to use.

In human readable SPIR-V:

```
OpDecorate %1 SpecId 0  ; decorate our signed 32-bit integer constant
OpDecorate %2 SpecId 12 ; decorate our 32-bit floating-point constant
%i32 = OpTypeInt 32 1   ; declare a signed 32-bit type
%float = OpTypeFloat 32 ; declare a 32-bit floating-point type
%1 = OpSpecConstant %i32 -1 ; some signed 32-bit integer constant
%2 = OpSpecConstant %float 0.5 ; some 32-bit floating-point constant
```

From the above we have two specialization constants, one is a signed 32-bit integer and the second is a 32-bit floating-point value.

Now to specialize the above via the specialization constants mechanism:

```
struct SpecializationData {
    int32_t data0;
    float data1;
};
```

```
const VkSpecializationMapEntry entries[] =
{
    {
        .constantID = 0,
        .offset = offsetof(SpecializationData, data0),
        .size = sizeof(SpecializationData::data0)
    },
    {
        .constantID = 12,
        .offset = offsetof(SpecializationData, data1),
        .size = sizeof(SpecializationData::data1)
    }
};

SpecializationData data;
data.data0 = -42;    // set the data for the 32-bit integer
data.data1 = 42.0f;  // set the data for the 32-bit floating-point

const VkSpecializationInfo info =
{
    .mapEntryCount = 2,
    .pMapEntries = entries,
    .dataSize = sizeof(data),
    .pdata = &data,
};
```

It is legal for a SPIR-V module with specializations to be compiled into a pipeline where no specialization information was provided. SPIR-V specialization constants contain default values such that if a specialization is not provided, the default value will be used. In the examples above, it would be valid for an application to only specialize some of the specialization constants within the SPIR-V module, and let the other constants use their default values encoded within the OpSpecConstant declarations.

10.10. Pipeline Libraries

A pipeline library is a special pipeline that was created using the `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR` and cannot be bound, instead it defines a set of pipeline state which can be linked into other pipelines. For ray tracing pipelines this includes shaders and shader groups. The application **must** maintain the lifetime of a pipeline library based on the pipelines that link with it.

This linkage is achieved by using the following structure within the appropriate creation mechanisms:

The `VkPipelineLibraryCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_pipeline_library
typedef struct VkPipelineLibraryCreateInfoKHR {
    VkStructureType      sType;
    const void*          pNext;
    uint32_t             libraryCount;
    const VkPipeline*    pLibraries;
} VkPipelineLibraryCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `libraryCount` is the number of pipeline libraries in `pLibraries`.
- `pLibraries` is a pointer to an array of VkPipeline structures specifying pipeline libraries to use when creating a pipeline.

Valid Usage

- VUID-VkPipelineLibraryCreateInfoKHR-pLibraries-03381
  Each element of `pLibraries` **must** have been created with `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR`
- VUID-VkPipelineLibraryCreateInfoKHR-pLibraries-06855
  If any library in `pLibraries` was created with a shader stage with VkPipelineShaderStageModuleIdentifierCreateInfoEXT and `identifierSize` not equal to 0, the pipeline **must** be created with the `VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT` flag set

Valid Usage (Implicit)

- VUID-VkPipelineLibraryCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_LIBRARY_CREATE_INFO_KHR`
- VUID-VkPipelineLibraryCreateInfoKHR-pLibraries-parameter
  If `libraryCount` is not `0`, `pLibraries` **must** be a valid pointer to an array of `libraryCount` valid VkPipeline handles

Pipelines created with `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR` libraries **can** depend on other pipeline libraries in VkPipelineLibraryCreateInfoKHR.

A pipeline library is considered in-use, as long as one of the linking pipelines is in-use. This applies recursively if a pipeline library includes other pipeline libraries.

10.11. Pipeline Binding

Once a pipeline has been created, it **can** be bound to the command buffer using the command:

```
// Provided by VK_VERSION_1_0
void vkCmdBindPipeline(
    VkCommandBuffer                             commandBuffer,
    VkPipelineBindPoint                         pipelineBindPoint,
    VkPipeline                                  pipeline);
```

- `commandBuffer` is the command buffer that the pipeline will be bound to.
- `pipelineBindPoint` is a VkPipelineBindPoint value specifying to which bind point the pipeline is bound. Binding one does not disturb the others.
- `pipeline` is the pipeline to be bound.

Once bound, a pipeline binding affects subsequent commands that interact with the given pipeline type in the command buffer until a different pipeline of the same type is bound to the bind point, or until the pipeline bind point is disturbed by binding a shader object as described in Interaction with Pipelines. Commands that do not interact with the given pipeline type **must** not be affected by the pipeline state.

Valid Usage

- VUID-vkCmdBindPipeline-pipelineBindPoint-00777
  If `pipelineBindPoint` is `VK_PIPELINE_BIND_POINT_COMPUTE`, the `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdBindPipeline-pipelineBindPoint-00778
  If `pipelineBindPoint` is `VK_PIPELINE_BIND_POINT_GRAPHICS`, the `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBindPipeline-pipelineBindPoint-00779
  If `pipelineBindPoint` is `VK_PIPELINE_BIND_POINT_COMPUTE`, `pipeline` **must** be a compute pipeline
- VUID-vkCmdBindPipeline-pipelineBindPoint-00780
  If `pipelineBindPoint` is `VK_PIPELINE_BIND_POINT_GRAPHICS`, `pipeline` **must** be a graphics pipeline
- VUID-vkCmdBindPipeline-pipeline-00781
  If the `variableMultisampleRate` feature is not supported, `pipeline` is a graphics pipeline, the current subpass uses no attachments, and this is not the first call to this function with a graphics pipeline after transitioning to the current subpass, then the sample count specified by this pipeline **must** match that set in the previous pipeline
- VUID-vkCmdBindPipeline-variableSampleLocations-01525
  If VkPhysicalDeviceSampleLocationsPropertiesEXT::`variableSampleLocations` is `VK_FALSE`, and `pipeline` is a graphics pipeline created with a VkPipelineSampleLocationsStateCreateInfoEXT structure having its `sampleLocationsEnable` member set to `VK_TRUE` but without `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` enabled then the current render pass instance **must** have been begun by specifying a VkRenderPassSampleLocationsBeginInfoEXT structure whose `pPostSubpassSampleLocations` member contains an element with a `subpassIndex` matching

the current subpass index and the sampleLocationsInfo member of that element **must** match the sampleLocationsInfo specified in VkPipelineSampleLocationsStateCreateInfoEXT when the pipeline was created

- VUID-vkCmdBindPipeline-None-02323
  This command **must** not be recorded when transform feedback is active
- VUID-vkCmdBindPipeline-pipelineBindPoint-02391
  If pipelineBindPoint is VK_PIPELINE_BIND_POINT_RAY_TRACING_KHR, the VkCommandPool that commandBuffer was allocated from **must** support compute operations
- VUID-vkCmdBindPipeline-pipelineBindPoint-02392
  If pipelineBindPoint is VK_PIPELINE_BIND_POINT_RAY_TRACING_KHR, pipeline **must** be a ray tracing pipeline
- VUID-vkCmdBindPipeline-pipelineBindPoint-06721
  If pipelineBindPoint is VK_PIPELINE_BIND_POINT_RAY_TRACING_KHR, commandBuffer **must** not be a protected command buffer
- VUID-vkCmdBindPipeline-pipeline-03382
  pipeline **must** not have been created with VK_PIPELINE_CREATE_LIBRARY_BIT_KHR set
- VUID-vkCmdBindPipeline-commandBuffer-04809
  If commandBuffer is a secondary command buffer with VkCommandBufferInheritanceViewportScissorInfoNV::viewportScissor2D enabled and pipelineBindPoint is VK_PIPELINE_BIND_POINT_GRAPHICS and pipeline was created with VkPipelineDiscardRectangleStateCreateInfoEXT structure and its discardRectangleCount member is not 0, or the pipeline was created with VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT enabled, then the pipeline **must** have been created with VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT enabled
- VUID-vkCmdBindPipeline-pipelineBindPoint-04881
  If pipelineBindPoint is VK_PIPELINE_BIND_POINT_GRAPHICS and the provokingVertexModePerPipeline limit is VK_FALSE, then pipeline's VkPipelineRasterizationProvokingVertexStateCreateInfoEXT::provokingVertexMode **must** be the same as that of any other pipelines previously bound to this bind point within the current render pass instance, including any pipeline already bound when beginning the render pass instance

Valid Usage (Implicit)

- VUID-vkCmdBindPipeline-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindPipeline-pipelineBindPoint-parameter
  pipelineBindPoint **must** be a valid VkPipelineBindPoint value
- VUID-vkCmdBindPipeline-pipeline-parameter
  pipeline **must** be a valid VkPipeline handle
- VUID-vkCmdBindPipeline-commandBuffer-recording
  commandBuffer **must** be in the recording state

- VUID-vkCmdBindPipeline-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics, or compute operations
- VUID-vkCmdBindPipeline-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBindPipeline-commonparent
  Both of commandBuffer, and pipeline **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

Possible values of vkCmdBindPipeline::pipelineBindPoint, specifying the bind point of a pipeline object, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkPipelineBindPoint {
    VK_PIPELINE_BIND_POINT_GRAPHICS = 0,
    VK_PIPELINE_BIND_POINT_COMPUTE = 1,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_PIPELINE_BIND_POINT_RAY_TRACING_KHR = 1000165000,
} VkPipelineBindPoint;
```

- VK_PIPELINE_BIND_POINT_COMPUTE specifies binding as a compute pipeline.
- VK_PIPELINE_BIND_POINT_GRAPHICS specifies binding as a graphics pipeline.
- VK_PIPELINE_BIND_POINT_RAY_TRACING_KHR specifies binding as a ray tracing pipeline.

10.11.1. Interaction With Shader Objects

If the shaderObject feature is enabled, applications **can** use both pipelines and shader objects at the same time. The interaction between pipelines and shader objects is described in Interaction with Pipelines.

10.12. Dynamic State

When a pipeline object is bound, any pipeline object state that is not specified as dynamic is applied to the command buffer state. Pipeline object state that is specified as dynamic is not applied to the command buffer state at this time.

Instead, dynamic state **can** be modified at any time and persists for the lifetime of the command buffer, or until modified by another dynamic state setting command, or made invalid by binding a pipeline in which that state is statically specified.

When a pipeline object is bound, the following applies to each state parameter:

- If the state is not specified as dynamic in the new pipeline object, then that command buffer state is overwritten by the state in the new pipeline object. Before any draw or dispatch call with this pipeline there **must** not have been any calls to any of the corresponding dynamic state setting commands after this pipeline was bound.
- If the state is specified as dynamic in the new pipeline object, then that command buffer state is not disturbed. Before any draw or dispatch call with this pipeline there **must** have been at least one call to each of the corresponding dynamic state setting commands. The state-setting commands **must** be recorded after command buffer recording was begun, or after the last command binding a pipeline object with that state specified as static, whichever was the latter.
- If the state is not included (corresponding pointer in VkGraphicsPipelineCreateInfo was `NULL` or was ignored) in the new pipeline object, then that command buffer state is not disturbed.

Dynamic state that does not affect the result of operations **can** be left undefined.

NOTE	For example, if blending is disabled by the pipeline object state then the dynamic color blend constants do not need to be specified in the command buffer, even if this state is specified as dynamic in the pipeline object.

NOTE	Applications running on Vulkan implementations advertising an VkPhysicalDeviceDriverProperties::`conformanceVersion` less than 1.3.8.0 should be aware that rebinding the bound pipeline object may not reapply static state.

10.13. Pipeline Properties and Shader Information

When a pipeline is created, its state and shaders are compiled into zero or more device-specific executables, which are used when executing commands against that pipeline. To query the properties of these pipeline executables, call:

```
// Provided by VK_KHR_pipeline_executable_properties
VkResult vkGetPipelineExecutablePropertiesKHR(
    VkDevice                                    device,
    const VkPipelineInfoKHR*                    pPipelineInfo,
    uint32_t*                                   pExecutableCount,
    VkPipelineExecutablePropertiesKHR*          pProperties);
```

- `device` is the device that created the pipeline.
- `pPipelineInfo` describes the pipeline being queried.
- `pExecutableCount` is a pointer to an integer related to the number of pipeline executables available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of VkPipelineExecutablePropertiesKHR structures.

If `pProperties` is `NULL`, then the number of pipeline executables associated with the pipeline is returned in `pExecutableCount`. Otherwise, `pExecutableCount` **must** point to a variable set by the application to the number of elements in the `pProperties` array, and on return the variable is overwritten with the number of structures actually written to `pProperties`. If `pExecutableCount` is less than the number of pipeline executables associated with the pipeline, at most `pExecutableCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available properties were returned.

Valid Usage

- VUID-vkGetPipelineExecutablePropertiesKHR-pipelineExecutableInfo-03270
  The `pipelineExecutableInfo` feature **must** be enabled
- VUID-vkGetPipelineExecutablePropertiesKHR-pipeline-03271
  The `pipeline` member of `pPipelineInfo` **must** have been created with `device`

Valid Usage (Implicit)

- VUID-vkGetPipelineExecutablePropertiesKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetPipelineExecutablePropertiesKHR-pPipelineInfo-parameter
  `pPipelineInfo` **must** be a valid pointer to a valid VkPipelineInfoKHR structure
- VUID-vkGetPipelineExecutablePropertiesKHR-pExecutableCount-parameter
  `pExecutableCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPipelineExecutablePropertiesKHR-pProperties-parameter
  If the value referenced by `pExecutableCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pExecutableCount` VkPipelineExecutablePropertiesKHR structures

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkPipelineExecutablePropertiesKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_executable_properties
typedef struct VkPipelineExecutablePropertiesKHR {
    VkStructureType       sType;
    void*                 pNext;
    VkShaderStageFlags    stages;
    char                  name[VK_MAX_DESCRIPTION_SIZE];
    char                  description[VK_MAX_DESCRIPTION_SIZE];
    uint32_t              subgroupSize;
} VkPipelineExecutablePropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- stages is a bitmask of zero or more VkShaderStageFlagBits indicating which shader stages (if any) were principally used as inputs to compile this pipeline executable.
- name is an array of VK_MAX_DESCRIPTION_SIZE char containing a null-terminated UTF-8 string which is a short human readable name for this pipeline executable.
- description is an array of VK_MAX_DESCRIPTION_SIZE char containing a null-terminated UTF-8 string which is a human readable description for this pipeline executable.
- subgroupSize is the subgroup size with which this pipeline executable is dispatched.

Not all implementations have a 1:1 mapping between shader stages and pipeline executables and some implementations **may** reduce a given shader stage to fixed function hardware programming such that no pipeline executable is available. No guarantees are provided about the mapping between shader stages and pipeline executables and stages **should** be considered a best effort hint. Because the application **cannot** rely on the stages field to provide an exact description, name and description provide a human readable name and description which more accurately describes the given pipeline executable.

Valid Usage (Implicit)

- VUID-VkPipelineExecutablePropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_PROPERTIES_KHR
- VUID-VkPipelineExecutablePropertiesKHR-pNext-pNext
  pNext **must** be NULL

The VkPipelineInfoKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_executable_properties
typedef struct VkPipelineInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkPipeline         pipeline;
} VkPipelineInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pipeline` is a `VkPipeline` handle.

Valid Usage (Implicit)

- VUID-VkPipelineInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_INFO_KHR`
- VUID-VkPipelineInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPipelineInfoKHR-pipeline-parameter
  `pipeline` **must** be a valid VkPipeline handle

Each pipeline executable **may** have a set of statistics associated with it that are generated by the pipeline compilation process. These statistics **may** include things such as instruction counts, amount of spilling (if any), maximum number of simultaneous threads, or anything else which **may** aid developers in evaluating the expected performance of a shader. To query the compile time statistics associated with a pipeline executable, call:

```
// Provided by VK_KHR_pipeline_executable_properties
VkResult vkGetPipelineExecutableStatisticsKHR(
    VkDevice                                    device,
    const VkPipelineExecutableInfoKHR*          pExecutableInfo,
    uint32_t*                                   pStatisticCount,
    VkPipelineExecutableStatisticKHR*           pStatistics);
```

- `device` is the device that created the pipeline.
- `pExecutableInfo` describes the pipeline executable being queried.
- `pStatisticCount` is a pointer to an integer related to the number of statistics available or queried, as described below.
- `pStatistics` is either `NULL` or a pointer to an array of VkPipelineExecutableStatisticKHR structures.

If `pStatistics` is `NULL`, then the number of statistics associated with the pipeline executable is returned in `pStatisticCount`. Otherwise, `pStatisticCount` **must** point to a variable set by the application to the number of elements in the `pStatistics` array, and on return the variable is

overwritten with the number of structures actually written to pStatistics. If pStatisticCount is less than the number of statistics associated with the pipeline executable, at most pStatisticCount structures will be written, and VK_INCOMPLETE will be returned instead of VK_SUCCESS, to indicate that not all the available statistics were returned.

Valid Usage

- VUID-vkGetPipelineExecutableStatisticsKHR-pipelineExecutableInfo-03272
  The pipelineExecutableInfo feature **must** be enabled
- VUID-vkGetPipelineExecutableStatisticsKHR-pipeline-03273
  The pipeline member of pExecutableInfo **must** have been created with device
- VUID-vkGetPipelineExecutableStatisticsKHR-pipeline-03274
  The pipeline member of pExecutableInfo **must** have been created with VK_PIPELINE_CREATE_CAPTURE_STATISTICS_BIT_KHR

Valid Usage (Implicit)

- VUID-vkGetPipelineExecutableStatisticsKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetPipelineExecutableStatisticsKHR-pExecutableInfo-parameter
  pExecutableInfo **must** be a valid pointer to a valid VkPipelineExecutableInfoKHR structure
- VUID-vkGetPipelineExecutableStatisticsKHR-pStatisticCount-parameter
  pStatisticCount **must** be a valid pointer to a uint32_t value
- VUID-vkGetPipelineExecutableStatisticsKHR-pStatistics-parameter
  If the value referenced by pStatisticCount is not 0, and pStatistics is not NULL, pStatistics **must** be a valid pointer to an array of pStatisticCount VkPipelineExecutableStatisticKHR structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkPipelineExecutableInfoKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_executable_properties
typedef struct VkPipelineExecutableInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkPipeline         pipeline;
    uint32_t           executableIndex;
} VkPipelineExecutableInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pipeline` is the pipeline to query.
- `executableIndex` is the index of the pipeline executable to query in the array of executable properties returned by vkGetPipelineExecutablePropertiesKHR.

Valid Usage

- VUID-VkPipelineExecutableInfoKHR-executableIndex-03275
  `executableIndex` **must** be less than the number of pipeline executables associated with `pipeline` as returned in the `pExecutableCount` parameter of `vkGetPipelineExecutablePropertiesKHR`

Valid Usage (Implicit)

- VUID-VkPipelineExecutableInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_INFO_KHR`
- VUID-VkPipelineExecutableInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPipelineExecutableInfoKHR-pipeline-parameter
  `pipeline` **must** be a valid VkPipeline handle

The `VkPipelineExecutableStatisticKHR` structure is defined as:

```
// Provided by VK_KHR_pipeline_executable_properties
typedef struct VkPipelineExecutableStatisticKHR {
    VkStructureType                           sType;
    void*                                     pNext;
    char                                      name[VK_MAX_DESCRIPTION_SIZE];
    char                                      description[VK_MAX_DESCRIPTION_SIZE];
    VkPipelineExecutableStatisticFormatKHR    format;
    VkPipelineExecutableStatisticValueKHR     value;
} VkPipelineExecutableStatisticKHR;
```

- `sType` is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `name` is an array of `VK_MAX_DESCRIPTION_SIZE` `char` containing a null-terminated UTF-8 string which is a short human readable name for this statistic.
- `description` is an array of `VK_MAX_DESCRIPTION_SIZE` `char` containing a null-terminated UTF-8 string which is a human readable description for this statistic.
- `format` is a VkPipelineExecutableStatisticFormatKHR value specifying the format of the data found in `value`.
- `value` is the value of this statistic.

Valid Usage (Implicit)

- VUID-VkPipelineExecutableStatisticKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_STATISTIC_KHR`
- VUID-VkPipelineExecutableStatisticKHR-pNext-pNext
  `pNext` **must** be `NULL`

The `VkPipelineExecutableStatisticFormatKHR` enum is defined as:

```
// Provided by VK_KHR_pipeline_executable_properties
typedef enum VkPipelineExecutableStatisticFormatKHR {
    VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_BOOL32_KHR = 0,
    VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_INT64_KHR = 1,
    VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_UINT64_KHR = 2,
    VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_FLOAT64_KHR = 3,
} VkPipelineExecutableStatisticFormatKHR;
```

- `VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_BOOL32_KHR` specifies that the statistic is returned as a 32-bit boolean value which **must** be either `VK_TRUE` or `VK_FALSE` and **should** be read from the `b32` field of `VkPipelineExecutableStatisticValueKHR`.
- `VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_INT64_KHR` specifies that the statistic is returned as a signed 64-bit integer and **should** be read from the `i64` field of `VkPipelineExecutableStatisticValueKHR`.
- `VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_UINT64_KHR` specifies that the statistic is returned as an unsigned 64-bit integer and **should** be read from the `u64` field of `VkPipelineExecutableStatisticValueKHR`.
- `VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_FLOAT64_KHR` specifies that the statistic is returned as a 64-bit floating-point value and **should** be read from the `f64` field of `VkPipelineExecutableStatisticValueKHR`.

The `VkPipelineExecutableStatisticValueKHR` union is defined as:

```
// Provided by VK_KHR_pipeline_executable_properties
typedef union VkPipelineExecutableStatisticValueKHR {
    VkBool32    b32;
    int64_t     i64;
    uint64_t    u64;
    double      f64;
} VkPipelineExecutableStatisticValueKHR;
```

- `b32` is the 32-bit boolean value if the `VkPipelineExecutableStatisticFormatKHR` is `VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_BOOL32_KHR`.
- `i64` is the signed 64-bit integer value if the `VkPipelineExecutableStatisticFormatKHR` is `VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_INT64_KHR`.
- `u64` is the unsigned 64-bit integer value if the `VkPipelineExecutableStatisticFormatKHR` is `VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_UINT64_KHR`.
- `f64` is the 64-bit floating-point value if the `VkPipelineExecutableStatisticFormatKHR` is `VK_PIPELINE_EXECUTABLE_STATISTIC_FORMAT_FLOAT64_KHR`.

Each pipeline executable **may** have one or more text or binary internal representations associated with it which are generated as part of the compile process. These **may** include the final shader assembly, a binary form of the compiled shader, or the shader compiler's internal representation at any number of intermediate compile steps. To query the internal representations associated with a pipeline executable, call:

```
// Provided by VK_KHR_pipeline_executable_properties
VkResult vkGetPipelineExecutableInternalRepresentationsKHR(
    VkDevice                                    device,
    const VkPipelineExecutableInfoKHR*          pExecutableInfo,
    uint32_t*                                   pInternalRepresentationCount,
    VkPipelineExecutableInternalRepresentationKHR* pInternalRepresentations);
```

- `device` is the device that created the pipeline.
- `pExecutableInfo` describes the pipeline executable being queried.
- `pInternalRepresentationCount` is a pointer to an integer related to the number of internal representations available or queried, as described below.
- `pInternalRepresentations` is either `NULL` or a pointer to an array of `VkPipelineExecutableInternalRepresentationKHR` structures.

If `pInternalRepresentations` is `NULL`, then the number of internal representations associated with the pipeline executable is returned in `pInternalRepresentationCount`. Otherwise, `pInternalRepresentationCount` **must** point to a variable set by the application to the number of elements in the `pInternalRepresentations` array, and on return the variable is overwritten with the number of structures actually written to `pInternalRepresentations`. If `pInternalRepresentationCount` is less than the number of internal representations associated with the pipeline executable, at most `pInternalRepresentationCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available representations were returned.

While the details of the internal representations remain implementation-dependent, the implementation **should** order the internal representations in the order in which they occur in the compiled pipeline with the final shader assembly (if any) last.

Valid Usage

- VUID-vkGetPipelineExecutableInternalRepresentationsKHR-pipelineExecutableInfo-03276
  The `pipelineExecutableInfo` feature **must** be enabled
- VUID-vkGetPipelineExecutableInternalRepresentationsKHR-pipeline-03277
  The `pipeline` member of `pExecutableInfo` **must** have been created with `device`
- VUID-vkGetPipelineExecutableInternalRepresentationsKHR-pipeline-03278
  The `pipeline` member of `pExecutableInfo` **must** have been created with `VK_PIPELINE_CREATE_CAPTURE_INTERNAL_REPRESENTATIONS_BIT_KHR`

Valid Usage (Implicit)

- VUID-vkGetPipelineExecutableInternalRepresentationsKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetPipelineExecutableInternalRepresentationsKHR-pExecutableInfo-parameter
  `pExecutableInfo` **must** be a valid pointer to a valid VkPipelineExecutableInfoKHR structure
- VUID-vkGetPipelineExecutableInternalRepresentationsKHR-pInternalRepresentationCount-parameter
  `pInternalRepresentationCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPipelineExecutableInternalRepresentationsKHR-pInternalRepresentations-parameter
  If the value referenced by `pInternalRepresentationCount` is not `0`, and `pInternalRepresentations` is not `NULL`, `pInternalRepresentations` **must** be a valid pointer to an array of `pInternalRepresentationCount` VkPipelineExecutableInternalRepresentationKHR structures

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkPipelineExecutableInternalRepresentationKHR` structure is defined as:

```
// Provided by VK_KHR_pipeline_executable_properties
typedef struct VkPipelineExecutableInternalRepresentationKHR {
    VkStructureType    sType;
    void*              pNext;
    char               name[VK_MAX_DESCRIPTION_SIZE];
    char               description[VK_MAX_DESCRIPTION_SIZE];
    VkBool32           isText;
    size_t             dataSize;
    void*              pData;
} VkPipelineExecutableInternalRepresentationKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `name` is an array of `VK_MAX_DESCRIPTION_SIZE` `char` containing a null-terminated UTF-8 string which is a short human readable name for this internal representation.
- `description` is an array of `VK_MAX_DESCRIPTION_SIZE` `char` containing a null-terminated UTF-8 string which is a human readable description for this internal representation.
- `isText` specifies whether the returned data is text or opaque data. If `isText` is `VK_TRUE` then the data returned in `pData` is text and is guaranteed to be a null-terminated UTF-8 string.
- `dataSize` is an integer related to the size, in bytes, of the internal representation's data, as described below.
- `pData` is either `NULL` or a pointer to a block of data into which the implementation will write the internal representation.

If `pData` is `NULL`, then the size, in bytes, of the internal representation data is returned in `dataSize`. Otherwise, `dataSize` **must** be the size of the buffer, in bytes, pointed to by `pData` and on return `dataSize` is overwritten with the number of bytes of data actually written to `pData` including any trailing null character. If `dataSize` is less than the size, in bytes, of the internal representation's data, at most `dataSize` bytes of data will be written to `pData`, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available representation was returned.

If `isText` is `VK_TRUE` and `pData` is not `NULL` and `dataSize` is not zero, the last byte written to `pData` will be a null character.

Valid Usage (Implicit)

- VUID-VkPipelineExecutableInternalRepresentationKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_INTERNAL_REPRESENTATION_KHR`
- VUID-VkPipelineExecutableInternalRepresentationKHR-pNext-pNext
  `pNext` **must** be `NULL`

Chapter 11. Memory Allocation

Vulkan memory is broken up into two categories, *host memory* and *device memory*.

11.1. Host Memory

Host memory is memory needed by the Vulkan implementation for non-device-visible storage.

> **NOTE** This memory **may** be used to store the implementation's representation and state of Vulkan objects.

Vulkan provides applications the opportunity to perform host memory allocations on behalf of the Vulkan implementation. If this feature is not used, the implementation will perform its own memory allocations. Since most memory allocations are off the critical path, this is not meant as a performance feature. Rather, this **can** be useful for certain embedded systems, for debugging purposes (e.g. putting a guard page after all host allocations), or for memory allocation logging.

Allocators are provided by the application as a pointer to a `VkAllocationCallbacks` structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkAllocationCallbacks {
    void*                                   pUserData;
    PFN_vkAllocationFunction                pfnAllocation;
    PFN_vkReallocationFunction              pfnReallocation;
    PFN_vkFreeFunction                      pfnFree;
    PFN_vkInternalAllocationNotification    pfnInternalAllocation;
    PFN_vkInternalFreeNotification          pfnInternalFree;
} VkAllocationCallbacks;
```

- `pUserData` is a value to be interpreted by the implementation of the callbacks. When any of the callbacks in `VkAllocationCallbacks` are called, the Vulkan implementation will pass this value as the first parameter to the callback. This value **can** vary each time an allocator is passed into a command, even when the same object takes an allocator in multiple commands.
- `pfnAllocation` is a `PFN_vkAllocationFunction` pointer to an application-defined memory allocation function.
- `pfnReallocation` is a `PFN_vkReallocationFunction` pointer to an application-defined memory reallocation function.
- `pfnFree` is a `PFN_vkFreeFunction` pointer to an application-defined memory free function.
- `pfnInternalAllocation` is a `PFN_vkInternalAllocationNotification` pointer to an application-defined function that is called by the implementation when the implementation makes internal allocations.
- `pfnInternalFree` is a `PFN_vkInternalFreeNotification` pointer to an application-defined function that is called by the implementation when the implementation frees internal allocations.

Valid Usage

- VUID-VkAllocationCallbacks-pfnAllocation-00632
  `pfnAllocation` **must** be a valid pointer to a valid application-defined PFN_vkAllocationFunction
- VUID-VkAllocationCallbacks-pfnReallocation-00633
  `pfnReallocation` **must** be a valid pointer to a valid application-defined PFN_vkReallocationFunction
- VUID-VkAllocationCallbacks-pfnFree-00634
  `pfnFree` **must** be a valid pointer to a valid application-defined PFN_vkFreeFunction
- VUID-VkAllocationCallbacks-pfnInternalAllocation-00635
  If either of `pfnInternalAllocation` or `pfnInternalFree` is not `NULL`, both **must** be valid callbacks

The type of `pfnAllocation` is:

```
// Provided by VK_VERSION_1_0
typedef void* (VKAPI_PTR *PFN_vkAllocationFunction)(
    void*                                       pUserData,
    size_t                                      size,
    size_t                                      alignment,
    VkSystemAllocationScope                     allocationScope);
```

- `pUserData` is the value specified for VkAllocationCallbacks::`pUserData` in the allocator specified by the application.
- `size` is the size in bytes of the requested allocation.
- `alignment` is the requested alignment of the allocation in bytes and **must** be a power of two.
- `allocationScope` is a VkSystemAllocationScope value specifying the allocation scope of the lifetime of the allocation, as described here.

If `pfnAllocation` is unable to allocate the requested memory, it **must** return `NULL`. If the allocation was successful, it **must** return a valid pointer to memory allocation containing at least `size` bytes, and with the pointer value being a multiple of `alignment`.

NOTE

Correct Vulkan operation **cannot** be assumed if the application does not follow these rules.

For example, `pfnAllocation` (or `pfnReallocation`) could cause termination of running Vulkan instance(s) on a failed allocation for debugging purposes, either directly or indirectly. In these circumstances, it **cannot** be assumed that any part of any affected VkInstance objects are going to operate correctly (even vkDestroyInstance), and the application **must** ensure it cleans up properly via other means (e.g. process termination).

If pfnAllocation returns NULL, and if the implementation is unable to continue correct processing of the current command without the requested allocation, it **must** treat this as a runtime error, and generate VK_ERROR_OUT_OF_HOST_MEMORY at the appropriate time for the command in which the condition was detected, as described in Return Codes.

If the implementation is able to continue correct processing of the current command without the requested allocation, then it **may** do so, and **must** not generate VK_ERROR_OUT_OF_HOST_MEMORY as a result of this failed allocation.

The type of pfnReallocation is:

```
// Provided by VK_VERSION_1_0
typedef void* (VKAPI_PTR *PFN_vkReallocationFunction)(
    void*                                       pUserData,
    void*                                       pOriginal,
    size_t                                      size,
    size_t                                      alignment,
    VkSystemAllocationScope                     allocationScope);
```

- pUserData is the value specified for VkAllocationCallbacks::pUserData in the allocator specified by the application.
- pOriginal **must** be either NULL or a pointer previously returned by pfnReallocation or pfnAllocation of a compatible allocator.
- size is the size in bytes of the requested allocation.
- alignment is the requested alignment of the allocation in bytes and **must** be a power of two.
- allocationScope is a VkSystemAllocationScope value specifying the allocation scope of the lifetime of the allocation, as described here.

If the reallocation was successful, pfnReallocation **must** return an allocation with enough space for size bytes, and the contents of the original allocation from bytes zero to min(original size, new size) - 1 **must** be preserved in the returned allocation. If size is larger than the old size, the contents of the additional space are undefined. If satisfying these requirements involves creating a new allocation, then the old allocation **should** be freed.

If pOriginal is NULL, then pfnReallocation **must** behave equivalently to a call to PFN_vkAllocationFunction with the same parameter values (without pOriginal).

If size is zero, then pfnReallocation **must** behave equivalently to a call to PFN_vkFreeFunction with the same pUserData parameter value, and pMemory equal to pOriginal.

If pOriginal is non-NULL, the implementation **must** ensure that alignment is equal to the alignment used to originally allocate pOriginal.

If this function fails and pOriginal is non-NULL the application **must** not free the old allocation.

pfnReallocation **must** follow the same rules for return values as PFN_vkAllocationFunction.

The type of pfnFree is:

```
// Provided by VK_VERSION_1_0
typedef void (VKAPI_PTR *PFN_vkFreeFunction)(
    void*                                       pUserData,
    void*                                       pMemory);
```

- `pUserData` is the value specified for VkAllocationCallbacks::`pUserData` in the allocator specified by the application.
- `pMemory` is the allocation to be freed.

`pMemory` **may** be `NULL`, which the callback **must** handle safely. If `pMemory` is non-`NULL`, it **must** be a pointer previously allocated by `pfnAllocation` or `pfnReallocation`. The application **should** free this memory.

The type of `pfnInternalAllocation` is:

```
// Provided by VK_VERSION_1_0
typedef void (VKAPI_PTR *PFN_vkInternalAllocationNotification)(
    void*                                       pUserData,
    size_t                                      size,
    VkInternalAllocationType                    allocationType,
    VkSystemAllocationScope                     allocationScope);
```

- `pUserData` is the value specified for VkAllocationCallbacks::`pUserData` in the allocator specified by the application.
- `size` is the requested size of an allocation.
- `allocationType` is a VkInternalAllocationType value specifying the requested type of an allocation.
- `allocationScope` is a VkSystemAllocationScope value specifying the allocation scope of the lifetime of the allocation, as described here.

This is a purely informational callback.

The type of `pfnInternalFree` is:

```
// Provided by VK_VERSION_1_0
typedef void (VKAPI_PTR *PFN_vkInternalFreeNotification)(
    void*                                       pUserData,
    size_t                                      size,
    VkInternalAllocationType                    allocationType,
    VkSystemAllocationScope                     allocationScope);
```

- `pUserData` is the value specified for VkAllocationCallbacks::`pUserData` in the allocator specified by the application.
- `size` is the requested size of an allocation.

- `allocationType` is a VkInternalAllocationType value specifying the requested type of an allocation.
- `allocationScope` is a VkSystemAllocationScope value specifying the allocation scope of the lifetime of the allocation, as described here.

Each allocation has an *allocation scope* defining its lifetime and which object it is associated with. Possible values passed to the `allocationScope` parameter of the callback functions specified by VkAllocationCallbacks, indicating the allocation scope, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSystemAllocationScope {
    VK_SYSTEM_ALLOCATION_SCOPE_COMMAND = 0,
    VK_SYSTEM_ALLOCATION_SCOPE_OBJECT = 1,
    VK_SYSTEM_ALLOCATION_SCOPE_CACHE = 2,
    VK_SYSTEM_ALLOCATION_SCOPE_DEVICE = 3,
    VK_SYSTEM_ALLOCATION_SCOPE_INSTANCE = 4,
} VkSystemAllocationScope;
```

- `VK_SYSTEM_ALLOCATION_SCOPE_COMMAND` specifies that the allocation is scoped to the duration of the Vulkan command.
- `VK_SYSTEM_ALLOCATION_SCOPE_OBJECT` specifies that the allocation is scoped to the lifetime of the Vulkan object that is being created or used.
- `VK_SYSTEM_ALLOCATION_SCOPE_CACHE` specifies that the allocation is scoped to the lifetime of a `VkPipelineCache` or `VkValidationCacheEXT` object.
- `VK_SYSTEM_ALLOCATION_SCOPE_DEVICE` specifies that the allocation is scoped to the lifetime of the Vulkan device.
- `VK_SYSTEM_ALLOCATION_SCOPE_INSTANCE` specifies that the allocation is scoped to the lifetime of the Vulkan instance.

Most Vulkan commands operate on a single object, or there is a sole object that is being created or manipulated. When an allocation uses an allocation scope of `VK_SYSTEM_ALLOCATION_SCOPE_OBJECT` or `VK_SYSTEM_ALLOCATION_SCOPE_CACHE`, the allocation is scoped to the object being created or manipulated.

When an implementation requires host memory, it will make callbacks to the application using the most specific allocator and allocation scope available:

- If an allocation is scoped to the duration of a command, the allocator will use the `VK_SYSTEM_ALLOCATION_SCOPE_COMMAND` allocation scope. The most specific allocator available is used: if the object being created or manipulated has an allocator, that object's allocator will be used, else if the parent `VkDevice` has an allocator it will be used, else if the parent `VkInstance` has an allocator it will be used. Else,
- If an allocation is associated with a `VkValidationCacheEXT` or `VkPipelineCache` object, the allocator will use the `VK_SYSTEM_ALLOCATION_SCOPE_CACHE` allocation scope. The most specific allocator available is used (cache, else device, else instance). Else,

- If an allocation is scoped to the lifetime of an object, that object is being created or manipulated by the command, and that object's type is not `VkDevice` or `VkInstance`, the allocator will use an allocation scope of `VK_SYSTEM_ALLOCATION_SCOPE_OBJECT`. The most specific allocator available is used (object, else device, else instance). Else,
- If an allocation is scoped to the lifetime of a device, the allocator will use an allocation scope of `VK_SYSTEM_ALLOCATION_SCOPE_DEVICE`. The most specific allocator available is used (device, else instance). Else,
- If the allocation is scoped to the lifetime of an instance and the instance has an allocator, its allocator will be used with an allocation scope of `VK_SYSTEM_ALLOCATION_SCOPE_INSTANCE`.
- Otherwise an implementation will allocate memory through an alternative mechanism that is unspecified.

Objects that are allocated from pools do not specify their own allocator. When an implementation requires host memory for such an object, that memory is sourced from the object's parent pool's allocator.

The application is not expected to handle allocating memory that is intended for execution by the host due to the complexities of differing security implementations across multiple platforms. The implementation will allocate such memory internally and invoke an application provided informational callback when these *internal allocations* are allocated and freed. Upon allocation of executable memory, `pfnInternalAllocation` will be called. Upon freeing executable memory, `pfnInternalFree` will be called. An implementation will only call an informational callback for executable memory allocations and frees.

The `allocationType` parameter to the `pfnInternalAllocation` and `pfnInternalFree` functions **may** be one of the following values:

```
// Provided by VK_VERSION_1_0
typedef enum VkInternalAllocationType {
    VK_INTERNAL_ALLOCATION_TYPE_EXECUTABLE = 0,
} VkInternalAllocationType;
```

- `VK_INTERNAL_ALLOCATION_TYPE_EXECUTABLE` specifies that the allocation is intended for execution by the host.

An implementation **must** only make calls into an application-provided allocator during the execution of an API command. An implementation **must** only make calls into an application-provided allocator from the same thread that called the provoking API command. The implementation **should** not synchronize calls to any of the callbacks. If synchronization is needed, the callbacks **must** provide it themselves. The informational callbacks are subject to the same restrictions as the allocation callbacks.

If an implementation intends to make calls through a `VkAllocationCallbacks` structure between the time a `vkCreate*` command returns and the time a corresponding `vkDestroy*` command begins, that implementation **must** save a copy of the allocator before the `vkCreate*` command returns. The callback functions and any data structures they rely upon **must** remain valid for the lifetime of the object they are associated with.

If an allocator is provided to a vkCreate* command, a *compatible* allocator **must** be provided to the corresponding vkDestroy* command. Two VkAllocationCallbacks structures are compatible if memory allocated with pfnAllocation or pfnReallocation in each **can** be freed with pfnReallocation or pfnFree in the other. An allocator **must** not be provided to a vkDestroy* command if an allocator was not provided to the corresponding vkCreate* command.

If a non-NULL allocator is used, the pfnAllocation, pfnReallocation and pfnFree members **must** be non-NULL and point to valid implementations of the callbacks. An application **can** choose to not provide informational callbacks by setting both pfnInternalAllocation and pfnInternalFree to NULL. pfnInternalAllocation and pfnInternalFree **must** either both be NULL or both be non-NULL.

If pfnAllocation or pfnReallocation fail, the implementation **may** fail object creation and/or generate a VK_ERROR_OUT_OF_HOST_MEMORY error, as appropriate.

Allocation callbacks **must** not call any Vulkan commands.

The following sets of rules define when an implementation is permitted to call the allocator callbacks.

pfnAllocation or pfnReallocation **may** be called in the following situations:

- Allocations scoped to a VkDevice or VkInstance **may** be allocated from any API command.
- Allocations scoped to a command **may** be allocated from any API command.
- Allocations scoped to a VkPipelineCache **may** only be allocated from:
  - vkCreatePipelineCache
  - vkMergePipelineCaches for dstCache
  - vkCreateGraphicsPipelines for pipelineCache
  - vkCreateComputePipelines for pipelineCache
- Allocations scoped to a VkValidationCacheEXT **may** only be allocated from:
  - vkCreateValidationCacheEXT
  - vkMergeValidationCachesEXT for dstCache
  - vkCreateShaderModule for validationCache in VkShaderModuleValidationCacheCreateInfoEXT
- Allocations scoped to a VkDescriptorPool **may** only be allocated from:
  - any command that takes the pool as a direct argument
  - vkAllocateDescriptorSets for the descriptorPool member of its pAllocateInfo parameter
  - vkCreateDescriptorPool
- Allocations scoped to a VkCommandPool **may** only be allocated from:
  - any command that takes the pool as a direct argument
  - vkCreateCommandPool
  - vkAllocateCommandBuffers for the commandPool member of its pAllocateInfo parameter
  - any vkCmd* command whose commandBuffer was allocated from that VkCommandPool

- Allocations scoped to any other object **may** only be allocated in that object's `vkCreate*` command.

`pfnFree`, or `pfnReallocation` with zero `size`, **may** be called in the following situations:

- Allocations scoped to a `VkDevice` or `VkInstance` **may** be freed from any API command.
- Allocations scoped to a command **must** be freed by any API command which allocates such memory.
- Allocations scoped to a `VkPipelineCache` **may** be freed from `vkDestroyPipelineCache`.
- Allocations scoped to a `VkValidationCacheEXT` **may** be freed from `vkDestroyValidationCacheEXT`.
- Allocations scoped to a `VkDescriptorPool` **may** be freed from
  - any command that takes the pool as a direct argument
- Allocations scoped to a `VkCommandPool` **may** be freed from:
  - any command that takes the pool as a direct argument
  - `vkResetCommandBuffer` whose `commandBuffer` was allocated from that `VkCommandPool`
- Allocations scoped to any other object **may** be freed in that object's `vkDestroy*` command.
- Any command that allocates host memory **may** also free host memory of the same scope.

11.2. Device Memory

Device memory is memory that is visible to the device — for example the contents of the image or buffer objects, which **can** be natively used by the device.

11.2.1. Device Memory Properties

Memory properties of a physical device describe the memory heaps and memory types available.

To query memory properties, call:

```
// Provided by VK_VERSION_1_0
void vkGetPhysicalDeviceMemoryProperties(
    VkPhysicalDevice                            physicalDevice,
    VkPhysicalDeviceMemoryProperties*           pMemoryProperties);
```

- `physicalDevice` is the handle to the device to query.
- `pMemoryProperties` is a pointer to a VkPhysicalDeviceMemoryProperties structure in which the properties are returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceMemoryProperties-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceMemoryProperties-pMemoryProperties-parameter

pMemoryProperties **must** be a valid pointer to a VkPhysicalDeviceMemoryProperties structure

The VkPhysicalDeviceMemoryProperties structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPhysicalDeviceMemoryProperties {
    uint32_t        memoryTypeCount;
    VkMemoryType    memoryTypes[VK_MAX_MEMORY_TYPES];
    uint32_t        memoryHeapCount;
    VkMemoryHeap    memoryHeaps[VK_MAX_MEMORY_HEAPS];
} VkPhysicalDeviceMemoryProperties;
```

- memoryTypeCount is the number of valid elements in the memoryTypes array.
- memoryTypes is an array of VK_MAX_MEMORY_TYPES VkMemoryType structures describing the *memory types* that **can** be used to access memory allocated from the heaps specified by memoryHeaps.
- memoryHeapCount is the number of valid elements in the memoryHeaps array.
- memoryHeaps is an array of VK_MAX_MEMORY_HEAPS VkMemoryHeap structures describing the *memory heaps* from which memory **can** be allocated.

The VkPhysicalDeviceMemoryProperties structure describes a number of *memory heaps* as well as a number of *memory types* that **can** be used to access memory allocated in those heaps. Each heap describes a memory resource of a particular size, and each memory type describes a set of memory properties (e.g. host cached vs. uncached) that **can** be used with a given memory heap. Allocations using a particular memory type will consume resources from the heap indicated by that memory type's heap index. More than one memory type **may** share each heap, and the heaps and memory types provide a mechanism to advertise an accurate size of the physical memory resources while allowing the memory to be used with a variety of different properties.

The number of memory heaps is given by memoryHeapCount and is less than or equal to VK_MAX_MEMORY_HEAPS. Each heap is described by an element of the memoryHeaps array as a VkMemoryHeap structure. The number of memory types available across all memory heaps is given by memoryTypeCount and is less than or equal to VK_MAX_MEMORY_TYPES. Each memory type is described by an element of the memoryTypes array as a VkMemoryType structure.

At least one heap **must** include VK_MEMORY_HEAP_DEVICE_LOCAL_BIT in VkMemoryHeap::flags. If there are multiple heaps that all have similar performance characteristics, they **may** all include VK_MEMORY_HEAP_DEVICE_LOCAL_BIT. In a unified memory architecture (UMA) system there is often only a single memory heap which is considered to be equally "local" to the host and to the device, and such an implementation **must** advertise the heap as device-local.

Each memory type returned by vkGetPhysicalDeviceMemoryProperties **must** have its propertyFlags set to one of the following values:

- 0

- VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT
- VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_CACHED_BIT
- VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_CACHED_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT
- VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT
- VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT | VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT
- VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT | VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_CACHED_BIT
- VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT | VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_CACHED_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT
- VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT | VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT

There **must** be at least one memory type with both the VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT and VK_MEMORY_PROPERTY_HOST_COHERENT_BIT bits set in its propertyFlags. There **must** be at least one memory type with the VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT bit set in its propertyFlags.

For each pair of elements **X** and **Y** returned in memoryTypes, **X** **must** be placed at a lower index position than **Y** if:

- the set of bit flags returned in the propertyFlags member of **X** is a strict subset of the set of bit flags returned in the propertyFlags member of **Y**; or
- the propertyFlags members of **X** and **Y** are equal, and **X** belongs to a memory heap with greater performance (as determined in an implementation-specific manner)

NOTE

There is no ordering requirement between **X** and **Y** elements for the case their propertyFlags members are not in a subset relation. That potentially allows more than one possible way to order the same set of memory types. Notice that the list of all allowed memory property flag combinations is written in a valid order. But if instead VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT was before VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, the list would still be in a valid order.

This ordering requirement enables applications to use a simple search loop to select the desired memory type along the lines of:

```
// Find a memory in `memoryTypeBitsRequirement` that includes all of
```

```
`requiredProperties`
int32_t findProperties(const VkPhysicalDeviceMemoryProperties* pMemoryProperties,
                       uint32_t memoryTypeBitsRequirement,
                       VkMemoryPropertyFlags requiredProperties) {
    const uint32_t memoryCount = pMemoryProperties->memoryTypeCount;
    for (uint32_t memoryIndex = 0; memoryIndex < memoryCount; ++memoryIndex) {
        const uint32_t memoryTypeBits = (1 << memoryIndex);
        const bool isRequiredMemoryType = memoryTypeBitsRequirement & memoryTypeBits;

        const VkMemoryPropertyFlags properties =
            pMemoryProperties->memoryTypes[memoryIndex].propertyFlags;
        const bool hasRequiredProperties =
            (properties & requiredProperties) == requiredProperties;

        if (isRequiredMemoryType && hasRequiredProperties)
            return static_cast<int32_t>(memoryIndex);
    }

    // failed to find memory type
    return -1;
}

// Try to find an optimal memory type, or if it does not exist try fallback memory
type
// `device` is the VkDevice
// `image` is the VkImage that requires memory to be bound
// `memoryProperties` properties as returned by vkGetPhysicalDeviceMemoryProperties
// `requiredProperties` are the property flags that must be present
// `optimalProperties` are the property flags that are preferred by the application
VkMemoryRequirements memoryRequirements;
vkGetImageMemoryRequirements(device, image, &memoryRequirements);
int32_t memoryType =
    findProperties(&memoryProperties, memoryRequirements.memoryTypeBits,
optimalProperties);
if (memoryType == -1) // not found; try fallback properties
    memoryType =
        findProperties(&memoryProperties, memoryRequirements.memoryTypeBits,
requiredProperties);
```

VK_MAX_MEMORY_TYPES is the length of an array of VkMemoryType structures describing memory types, as returned in VkPhysicalDeviceMemoryProperties::memoryTypes.

```
#define VK_MAX_MEMORY_TYPES             32U
```

VK_MAX_MEMORY_HEAPS is the length of an array of VkMemoryHeap structures describing memory heaps, as returned in VkPhysicalDeviceMemoryProperties::memoryHeaps.

```
#define VK_MAX_MEMORY_HEAPS             16U
```

To query memory properties, call:

```
// Provided by VK_KHR_get_physical_device_properties2
void vkGetPhysicalDeviceMemoryProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    VkPhysicalDeviceMemoryProperties2*          pMemoryProperties);
```

- `physicalDevice` is the handle to the device to query.
- `pMemoryProperties` is a pointer to a VkPhysicalDeviceMemoryProperties2 structure in which the properties are returned.

`vkGetPhysicalDeviceMemoryProperties2KHR` behaves similarly to vkGetPhysicalDeviceMemoryProperties, with the ability to return extended information in a `pNext` chain of output structures.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceMemoryProperties2-physicalDevice-parameter `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceMemoryProperties2-pMemoryProperties-parameter `pMemoryProperties` **must** be a valid pointer to a VkPhysicalDeviceMemoryProperties2 structure

The `VkPhysicalDeviceMemoryProperties2` structure is defined as:

```
typedef struct VkPhysicalDeviceMemoryProperties2 {
    VkStructureType                     sType;
    void*                               pNext;
    VkPhysicalDeviceMemoryProperties    memoryProperties;
} VkPhysicalDeviceMemoryProperties2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkPhysicalDeviceMemoryProperties2 VkPhysicalDeviceMemoryProperties2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memoryProperties` is a VkPhysicalDeviceMemoryProperties structure which is populated with the same values as in vkGetPhysicalDeviceMemoryProperties.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMemoryProperties2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PROPERTIES_2`
- VUID-VkPhysicalDeviceMemoryProperties2-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPhysicalDeviceMemoryBudgetPropertiesEXT
- VUID-VkPhysicalDeviceMemoryProperties2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

The `VkMemoryHeap` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkMemoryHeap {
    VkDeviceSize         size;
    VkMemoryHeapFlags    flags;
} VkMemoryHeap;
```

- `size` is the total memory size in bytes in the heap.
- `flags` is a bitmask of VkMemoryHeapFlagBits specifying attribute flags for the heap.

Bits which **may** be set in VkMemoryHeap::`flags`, indicating attribute flags for the heap, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkMemoryHeapFlagBits {
    VK_MEMORY_HEAP_DEVICE_LOCAL_BIT = 0x00000001,
    VK_MEMORY_HEAP_MULTI_INSTANCE_BIT = 0x00000002,
  // Provided by VK_KHR_device_group_creation
    VK_MEMORY_HEAP_MULTI_INSTANCE_BIT_KHR = VK_MEMORY_HEAP_MULTI_INSTANCE_BIT,
} VkMemoryHeapFlagBits;
```

- `VK_MEMORY_HEAP_DEVICE_LOCAL_BIT` specifies that the heap corresponds to device-local memory. Device-local memory **may** have different performance characteristics than host-local memory, and **may** support different memory property flags.
- `VK_MEMORY_HEAP_MULTI_INSTANCE_BIT` specifies that in a logical device representing more than one physical device, there is a per-physical device instance of the heap memory. By default, an allocation from such a heap will be replicated to each physical device's instance of the heap.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkMemoryHeapFlags;
```

`VkMemoryHeapFlags` is a bitmask type for setting a mask of zero or more VkMemoryHeapFlagBits.

The `VkMemoryType` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkMemoryType {
    VkMemoryPropertyFlags    propertyFlags;
    uint32_t                 heapIndex;
} VkMemoryType;
```

- `heapIndex` describes which memory heap this memory type corresponds to, and **must** be less than `memoryHeapCount` from the VkPhysicalDeviceMemoryProperties structure.
- `propertyFlags` is a bitmask of VkMemoryPropertyFlagBits of properties for this memory type.

Bits which **may** be set in VkMemoryType::`propertyFlags`, indicating properties of a memory type, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkMemoryPropertyFlagBits {
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT = 0x00000001,
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT = 0x00000002,
    VK_MEMORY_PROPERTY_HOST_COHERENT_BIT = 0x00000004,
    VK_MEMORY_PROPERTY_HOST_CACHED_BIT = 0x00000008,
    VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT = 0x00000010,
} VkMemoryPropertyFlagBits;
```

- `VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT` bit specifies that memory allocated with this type is the most efficient for device access. This property will be set if and only if the memory type belongs to a heap with the `VK_MEMORY_HEAP_DEVICE_LOCAL_BIT` set.
- `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT` bit specifies that memory allocated with this type **can** be mapped for host access using vkMapMemory.
- `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT` bit specifies that the host cache management commands vkFlushMappedMemoryRanges and vkInvalidateMappedMemoryRanges are not needed to manage availability and visibility on the host.
- `VK_MEMORY_PROPERTY_HOST_CACHED_BIT` bit specifies that memory allocated with this type is cached on the host. Host memory accesses to uncached memory are slower than to cached memory, however uncached memory is always host coherent.
- `VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT` bit specifies that the memory type only allows device access to the memory. Memory types **must** not have both `VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT` and `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT` set. Additionally, the object's backing memory **may** be provided by the implementation lazily as specified in Lazily Allocated Memory.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkMemoryPropertyFlags;
```

`VkMemoryPropertyFlags` is a bitmask type for setting a mask of zero or more VkMemoryPropertyFlagBits.

If the VkPhysicalDeviceMemoryBudgetPropertiesEXT structure is included in the pNext chain of VkPhysicalDeviceMemoryProperties2, it is filled with the current memory budgets and usages.

The VkPhysicalDeviceMemoryBudgetPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_memory_budget
typedef struct VkPhysicalDeviceMemoryBudgetPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkDeviceSize       heapBudget[VK_MAX_MEMORY_HEAPS];
    VkDeviceSize       heapUsage[VK_MAX_MEMORY_HEAPS];
} VkPhysicalDeviceMemoryBudgetPropertiesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- heapBudget is an array of VK_MAX_MEMORY_HEAPS VkDeviceSize values in which memory budgets are returned, with one element for each memory heap. A heap's budget is a rough estimate of how much memory the process **can** allocate from that heap before allocations **may** fail or cause performance degradation. The budget includes any currently allocated device memory.
- heapUsage is an array of VK_MAX_MEMORY_HEAPS VkDeviceSize values in which memory usages are returned, with one element for each memory heap. A heap's usage is an estimate of how much memory the process is currently using in that heap.

The values returned in this structure are not invariant. The heapBudget and heapUsage values **must** be zero for array elements greater than or equal to VkPhysicalDeviceMemoryProperties::memoryHeapCount. The heapBudget value **must** be non-zero for array elements less than VkPhysicalDeviceMemoryProperties::memoryHeapCount. The heapBudget value **must** be less than or equal to VkMemoryHeap::size for each heap.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMemoryBudgetPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_BUDGET_PROPERTIES_EXT

11.2.2. Device Memory Objects

A Vulkan device operates on data in device memory via memory objects that are represented in the API by a VkDeviceMemory handle:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDeviceMemory)
```

11.2.3. Device Memory Allocation

To allocate memory objects, call:

```
// Provided by VK_VERSION_1_0
VkResult vkAllocateMemory(
    VkDevice                                    device,
    const VkMemoryAllocateInfo*                 pAllocateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkDeviceMemory*                             pMemory);
```

- `device` is the logical device that owns the memory.
- `pAllocateInfo` is a pointer to a VkMemoryAllocateInfo structure describing parameters of the allocation. A successfully returned allocation **must** use the requested parameters — no substitution is permitted by the implementation.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pMemory` is a pointer to a VkDeviceMemory handle in which information about the allocated memory is returned.

Allocations returned by `vkAllocateMemory` are guaranteed to meet any alignment requirement of the implementation. For example, if an implementation requires 128 byte alignment for images and 64 byte alignment for buffers, the device memory returned through this mechanism would be 128-byte aligned. This ensures that applications **can** correctly suballocate objects of different types (with potentially different alignment requirements) in the same memory object.

When memory is allocated, its contents are undefined.

The maximum number of valid memory allocations that **can** exist simultaneously within a VkDevice **may** be restricted by implementation- or platform-dependent limits. The `maxMemoryAllocationCount` feature describes the number of allocations that **can** exist simultaneously before encountering these internal limits.

NOTE

For historical reasons, if `maxMemoryAllocationCount` is exceeded, some implementations may return `VK_ERROR_TOO_MANY_OBJECTS`. Exceeding this limit will result in undefined behavior, and an application should not rely on the use of the returned error code in order to identify when the limit is reached.

NOTE

Many protected memory implementations involve complex hardware and system software support, and often have additional and much lower limits on the number of simultaneous protected memory allocations (from memory types with the `VK_MEMORY_PROPERTY_PROTECTED_BIT` property) than for non-protected memory allocations. These limits can be system-wide, and depend on a variety of factors outside of the Vulkan implementation, so they cannot be queried in Vulkan. Applications **should** use as few allocations as possible from such memory types by suballocating aggressively, and be prepared for allocation failure even when there is apparently plenty of capacity remaining in the memory heap. As a guideline, the Vulkan conformance test suite requires that at least 80 minimum-size allocations

> can exist concurrently when no other uses of protected memory are active in the system.

Some platforms **may** have a limit on the maximum size of a single allocation. For example, certain systems **may** fail to create allocations with a size greater than or equal to 4GB. Such a limit is implementation-dependent, and if such a failure occurs then the error `VK_ERROR_OUT_OF_DEVICE_MEMORY` **must** be returned. This limit is advertised in VkPhysicalDeviceMaintenance3Properties::`maxMemoryAllocationSize`.

Valid Usage

- VUID-vkAllocateMemory-pAllocateInfo-01713
  `pAllocateInfo->allocationSize` **must** be less than or equal to VkPhysicalDeviceMemoryProperties::`memoryHeaps`[memindex].`size` where `memindex` = VkPhysicalDeviceMemoryProperties::`memoryTypes`[`pAllocateInfo->memoryTypeIndex`].`heapIndex` as returned by vkGetPhysicalDeviceMemoryProperties for the VkPhysicalDevice that `device` was created from
- VUID-vkAllocateMemory-pAllocateInfo-01714
  `pAllocateInfo->memoryTypeIndex` **must** be less than VkPhysicalDeviceMemoryProperties::`memoryTypeCount` as returned by vkGetPhysicalDeviceMemoryProperties for the VkPhysicalDevice that `device` was created from
- VUID-vkAllocateMemory-maxMemoryAllocationCount-04101
  There **must** be less than `VkPhysicalDeviceLimits`::`maxMemoryAllocationCount` device memory allocations currently allocated on the device

Valid Usage (Implicit)

- VUID-vkAllocateMemory-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkAllocateMemory-pAllocateInfo-parameter
  `pAllocateInfo` **must** be a valid pointer to a valid VkMemoryAllocateInfo structure
- VUID-vkAllocateMemory-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkAllocateMemory-pMemory-parameter
  `pMemory` **must** be a valid pointer to a VkDeviceMemory handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INVALID_EXTERNAL_HANDLE
- VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR

The VkMemoryAllocateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkMemoryAllocateInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkDeviceSize       allocationSize;
    uint32_t           memoryTypeIndex;
} VkMemoryAllocateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- allocationSize is the size of the allocation in bytes.
- memoryTypeIndex is an index identifying a memory type from the memoryTypes array of the VkPhysicalDeviceMemoryProperties structure.

The internal data of an allocated device memory object **must** include a reference to implementation-specific resources, referred to as the memory object's *payload*. Applications **can** also import and export that internal data to and from device memory objects to share data between Vulkan instances and other compatible APIs. A VkMemoryAllocateInfo structure defines a memory import operation if its pNext chain includes one of the following structures:

- VkImportMemoryWin32HandleInfoKHR with a non-zero handleType value
- VkImportMemoryFdInfoKHR with a non-zero handleType value
- VkImportMemoryHostPointerInfoEXT with a non-zero handleType value

If the parameters define an import operation and the external handle type is VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT, VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT, or VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT, allocationSize is ignored. The implementation **must** query the size of these allocations from the OS.

Whether device memory objects constructed via a memory import operation hold a reference to their payload depends on the properties of the handle type used to perform the import, as defined below for each valid handle type. Importing memory **must** not modify the content of the memory. Implementations **must** ensure that importing memory does not enable the importing Vulkan instance to access any memory or resources in other Vulkan instances other than that corresponding to the memory object imported. Implementations **must** also ensure accessing

imported memory which has not been initialized does not allow the importing Vulkan instance to obtain data from the exporting Vulkan instance or vice-versa.

NOTE	How exported and imported memory is isolated is left to the implementation, but applications should be aware that such isolation **may** prevent implementations from placing multiple exportable memory objects in the same physical or virtual page. Hence, applications **should** avoid creating many small external memory objects whenever possible.

Importing memory **must** not increase overall heap usage within a system. However, it **must** affect the following per-process values:

- VkPhysicalDeviceLimits::maxMemoryAllocationCount
- VkPhysicalDeviceMemoryBudgetPropertiesEXT::heapUsage

When performing a memory import operation, it is the responsibility of the application to ensure the external handles and their associated payloads meet all valid usage requirements. However, implementations **must** perform sufficient validation of external handles and payloads to ensure that the operation results in a valid memory object which will not cause program termination, device loss, queue stalls, or corruption of other resources when used as allowed according to its allocation parameters. If the external handle provided does not meet these requirements, the implementation **must** fail the memory import operation with the error code VK_ERROR_INVALID_EXTERNAL_HANDLE.

Valid Usage

- VUID-VkMemoryAllocateInfo-allocationSize-07897
  If the parameters do not define an import or export operation, allocationSize **must** be greater than 0
- VUID-VkMemoryAllocateInfo-None-06657
  The parameters **must** not define more than one import operation
- VUID-VkMemoryAllocateInfo-allocationSize-07899
  If the parameters define an export operation , allocationSize **must** be greater than 0
- VUID-VkMemoryAllocateInfo-pNext-00639
  If the pNext chain includes a VkExportMemoryAllocateInfo structure, and any of the handle types specified in VkExportMemoryAllocateInfo::handleTypes require a dedicated allocation, as reported by vkGetPhysicalDeviceImageFormatProperties2KHR in VkExternalImageFormatProperties::externalMemoryProperties.externalMemoryFeatures, or by vkGetPhysicalDeviceExternalBufferPropertiesKHR in VkExternalBufferProperties::externalMemoryProperties.externalMemoryFeatures, the pNext chain **must** include a VkMemoryDedicatedAllocateInfo or VkDedicatedAllocationMemoryAllocateInfoNV structure with either its image or buffer member set to a value other than VK_NULL_HANDLE
- VUID-VkMemoryAllocateInfo-allocationSize-01742
  If the parameters define an import operation, the external handle specified was created by the Vulkan API, and the external handle type is VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT, then the values of allocationSize and

memoryTypeIndex **must** match those specified when the payload being imported was created

- VUID-VkMemoryAllocateInfo-None-00643
  If the parameters define an import operation and the external handle specified was created by the Vulkan API, the device mask specified by VkMemoryAllocateFlagsInfo **must** match the mask specified when the payload being imported was allocated
- VUID-VkMemoryAllocateInfo-None-00644
  If the parameters define an import operation and the external handle specified was created by the Vulkan API, the list of physical devices that comprise the logical device passed to vkAllocateMemory **must** match the list of physical devices that comprise the logical device on which the payload was originally allocated
- VUID-VkMemoryAllocateInfo-memoryTypeIndex-00645
  If the parameters define an import operation and the external handle is an NT handle or a global share handle created outside of the Vulkan API, the value of memoryTypeIndex **must** be one of those returned by vkGetMemoryWin32HandlePropertiesKHR
- VUID-VkMemoryAllocateInfo-allocationSize-01743
  If the parameters define an import operation, the external handle was created by the Vulkan API, and the external handle type is VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT or VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT, then the values of allocationSize and memoryTypeIndex **must** match those specified when the payload being imported was created
- VUID-VkMemoryAllocateInfo-allocationSize-00647
  If the parameters define an import operation and the external handle type is VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT, allocationSize **must** match the size specified when creating the Direct3D 12 heap from which the payload was extracted
- VUID-VkMemoryAllocateInfo-memoryTypeIndex-00648
  If the parameters define an import operation and the external handle is a POSIX file descriptor created outside of the Vulkan API, the value of memoryTypeIndex **must** be one of those returned by vkGetMemoryFdPropertiesKHR
- VUID-VkMemoryAllocateInfo-memoryTypeIndex-01744
  If the parameters define an import operation and the external handle is a host pointer, the value of memoryTypeIndex **must** be one of those returned by vkGetMemoryHostPointerPropertiesEXT
- VUID-VkMemoryAllocateInfo-allocationSize-01745
  If the parameters define an import operation and the external handle is a host pointer, allocationSize **must** be an integer multiple of VkPhysicalDeviceExternalMemoryHostPropertiesEXT::minImportedHostPointerAlignment
- VUID-VkMemoryAllocateInfo-pNext-02806
  If the parameters define an import operation and the external handle is a host pointer, the pNext chain **must** not include a VkMemoryDedicatedAllocateInfo structure with either its image or buffer field set to a value other than VK_NULL_HANDLE
- VUID-VkMemoryAllocateInfo-opaqueCaptureAddress-03329
  If VkMemoryOpaqueCaptureAddressAllocateInfo::opaqueCaptureAddress is not zero,

VkMemoryAllocateFlagsInfo::flags **must** include VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT

- VUID-VkMemoryAllocateInfo-flags-03330
  If VkMemoryAllocateFlagsInfo::flags includes VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT, the bufferDeviceAddressCaptureReplay feature **must** be enabled
- VUID-VkMemoryAllocateInfo-flags-03331
  If VkMemoryAllocateFlagsInfo::flags includes VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT, the bufferDeviceAddress feature **must** be enabled
- VUID-VkMemoryAllocateInfo-pNext-03332
  If the pNext chain includes a VkImportMemoryHostPointerInfoEXT structure, VkMemoryOpaqueCaptureAddressAllocateInfo::opaqueCaptureAddress **must** be zero
- VUID-VkMemoryAllocateInfo-opaqueCaptureAddress-03333
  If the parameters define an import operation, VkMemoryOpaqueCaptureAddressAllocateInfo::opaqueCaptureAddress **must** be zero
- VUID-VkMemoryAllocateInfo-pNext-06780
  If the pNext chain includes a VkExportMetalObjectCreateInfoEXT structure, its exportObjectType member **must** be VK_EXPORT_METAL_OBJECT_TYPE_METAL_BUFFER_BIT_EXT

Valid Usage (Implicit)

- VUID-VkMemoryAllocateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO
- VUID-VkMemoryAllocateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkExportMemoryAllocateInfo, VkExportMemoryWin32HandleInfoKHR, VkExportMetalObjectCreateInfoEXT, VkImportMemoryFdInfoKHR, VkImportMemoryHostPointerInfoEXT, VkImportMemoryWin32HandleInfoKHR, VkImportMetalBufferInfoEXT, VkMemoryAllocateFlagsInfo, VkMemoryDedicatedAllocateInfo, VkMemoryOpaqueCaptureAddressAllocateInfo, or VkMemoryPriorityAllocateInfoEXT
- VUID-VkMemoryAllocateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique, with the exception of structures of type VkExportMetalObjectCreateInfoEXT

If the pNext chain includes a VkMemoryDedicatedAllocateInfo structure, then that structure includes a handle of the sole buffer or image resource that the memory **can** be bound to.

The VkMemoryDedicatedAllocateInfo structure is defined as:

```
typedef struct VkMemoryDedicatedAllocateInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkImage            image;
    VkBuffer           buffer;
} VkMemoryDedicatedAllocateInfo;
```

or the equivalent

```
// Provided by VK_KHR_dedicated_allocation
typedef VkMemoryDedicatedAllocateInfo VkMemoryDedicatedAllocateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `image` is VK_NULL_HANDLE or a handle of an image which this memory will be bound to.
- `buffer` is VK_NULL_HANDLE or a handle of a buffer which this memory will be bound to.

Valid Usage

- VUID-VkMemoryDedicatedAllocateInfo-image-01432
  At least one of `image` and `buffer` **must** be VK_NULL_HANDLE
- VUID-VkMemoryDedicatedAllocateInfo-image-02964
  If `image` is not VK_NULL_HANDLE , `VkMemoryAllocateInfo`::`allocationSize` **must** equal the `VkMemoryRequirements`::`size` of the image
- VUID-VkMemoryDedicatedAllocateInfo-image-01434
  If `image` is not VK_NULL_HANDLE, `image` **must** have been created without `VK_IMAGE_CREATE_SPARSE_BINDING_BIT` set in VkImageCreateInfo::`flags`
- VUID-VkMemoryDedicatedAllocateInfo-buffer-02965
  If `buffer` is not VK_NULL_HANDLE , `VkMemoryAllocateInfo`::`allocationSize` **must** equal the `VkMemoryRequirements`::`size` of the buffer
- VUID-VkMemoryDedicatedAllocateInfo-buffer-01436
  If `buffer` is not VK_NULL_HANDLE, `buffer` **must** have been created without `VK_BUFFER_CREATE_SPARSE_BINDING_BIT` set in VkBufferCreateInfo::`flags`
- VUID-VkMemoryDedicatedAllocateInfo-image-01876
  If `image` is not VK_NULL_HANDLE and VkMemoryAllocateInfo defines a memory import operation with handle type `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT`, or `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT`, and the external handle was created by the Vulkan API, then the memory being imported **must** also be a dedicated image allocation and `image` **must** be identical to the image associated with the imported memory

- VUID-VkMemoryDedicatedAllocateInfo-buffer-01877
  If `buffer` is not VK_NULL_HANDLE and VkMemoryAllocateInfo defines a memory import operation with handle type `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT`, or `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT`, and the external handle was created by the Vulkan API, then the memory being imported **must** also be a dedicated buffer allocation and `buffer` **must** be identical to the buffer associated with the imported memory
- VUID-VkMemoryDedicatedAllocateInfo-image-01878
  If `image` is not VK_NULL_HANDLE and VkMemoryAllocateInfo defines a memory import operation with handle type `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT`, the memory being imported **must** also be a dedicated image allocation and `image` **must** be identical to the image associated with the imported memory
- VUID-VkMemoryDedicatedAllocateInfo-buffer-01879
  If `buffer` is not VK_NULL_HANDLE and VkMemoryAllocateInfo defines a memory import operation with handle type `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT`, the memory being imported **must** also be a dedicated buffer allocation and `buffer` **must** be identical to the buffer associated with the imported memory
- VUID-VkMemoryDedicatedAllocateInfo-image-01797
  If `image` is not VK_NULL_HANDLE, `image` **must** not have been created with `VK_IMAGE_CREATE_DISJOINT_BIT` set in VkImageCreateInfo::`flags`

Valid Usage (Implicit)

- VUID-VkMemoryDedicatedAllocateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_DEDICATED_ALLOCATE_INFO`
- VUID-VkMemoryDedicatedAllocateInfo-image-parameter
  If `image` is not VK_NULL_HANDLE, `image` **must** be a valid VkImage handle
- VUID-VkMemoryDedicatedAllocateInfo-buffer-parameter
  If `buffer` is not VK_NULL_HANDLE, `buffer` **must** be a valid VkBuffer handle
- VUID-VkMemoryDedicatedAllocateInfo-commonparent
  Both of `buffer`, and `image` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

If the `pNext` chain includes a `VkMemoryPriorityAllocateInfoEXT` structure, then that structure includes a priority for the memory.

The `VkMemoryPriorityAllocateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_memory_priority
typedef struct VkMemoryPriorityAllocateInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    float              priority;
} VkMemoryPriorityAllocateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `priority` is a floating-point value between `0` and `1`, indicating the priority of the allocation relative to other memory allocations. Larger values are higher priority. The granularity of the priorities is implementation-dependent.

Memory allocations with higher priority **may** be more likely to stay in device-local memory when the system is under memory pressure.

If this structure is not included, it is as if the `priority` value were `0.5`.

Valid Usage

- VUID-VkMemoryPriorityAllocateInfoEXT-priority-02602
  `priority` **must** be between `0` and `1`, inclusive

Valid Usage (Implicit)

- VUID-VkMemoryPriorityAllocateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_PRIORITY_ALLOCATE_INFO_EXT`

When allocating memory whose payload **may** be exported to another process or Vulkan instance, add a VkExportMemoryAllocateInfo structure to the `pNext` chain of the VkMemoryAllocateInfo structure, specifying the handle types that **may** be exported.

The VkExportMemoryAllocateInfo structure is defined as:

```
typedef struct VkExportMemoryAllocateInfo {
    VkStructureType                    sType;
    const void*                        pNext;
    VkExternalMemoryHandleTypeFlags    handleTypes;
} VkExportMemoryAllocateInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_memory
typedef VkExportMemoryAllocateInfo VkExportMemoryAllocateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleTypes` is zero or a bitmask of VkExternalMemoryHandleTypeFlagBits specifying one or more memory handle types the application **can** export from the resulting allocation. The application **can** request multiple handle types for the same allocation.

Valid Usage

- VUID-VkExportMemoryAllocateInfo-handleTypes-00656
  The bits in `handleTypes` **must** be supported and compatible, as reported by VkExternalImageFormatProperties or VkExternalBufferProperties

Valid Usage (Implicit)

- VUID-VkExportMemoryAllocateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_MEMORY_ALLOCATE_INFO`
- VUID-VkExportMemoryAllocateInfo-handleTypes-parameter
  `handleTypes` **must** be a valid combination of VkExternalMemoryHandleTypeFlagBits values

11.2.4. Win32 External Memory

To specify additional attributes of NT handles exported from a memory object, add a VkExportMemoryWin32HandleInfoKHR structure to the `pNext` chain of the VkMemoryAllocateInfo structure. The `VkExportMemoryWin32HandleInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_memory_win32
typedef struct VkExportMemoryWin32HandleInfoKHR {
    VkStructureType               sType;
    const void*                   pNext;
    const SECURITY_ATTRIBUTES*    pAttributes;
    DWORD                         dwAccess;
    LPCWSTR                       name;
} VkExportMemoryWin32HandleInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pAttributes` is a pointer to a Windows `SECURITY_ATTRIBUTES` structure specifying security attributes of the handle.

- `dwAccess` is a `DWORD` specifying access rights of the handle.
- `name` is a null-terminated UTF-16 string to associate with the payload referenced by NT handles exported from the created memory.

If VkExportMemoryAllocateInfo is not included in the same `pNext` chain, this structure is ignored.

If VkExportMemoryAllocateInfo is included in the `pNext` chain of VkMemoryAllocateInfo with a Windows `handleType`, but either `VkExportMemoryWin32HandleInfoKHR` is not included in the `pNext` chain, or it is included but `pAttributes` is set to `NULL`, default security descriptor values will be used, and child processes created by the application will not inherit the handle, as described in the MSDN documentation for "Synchronization Object Security and Access Rights"[1]. Further, if the structure is not present, the access rights used depend on the handle type.

For handles of the following types:

- `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT`

The implementation **must** ensure the access rights allow read and write access to the memory.

1

https://docs.microsoft.com/en-us/windows/win32/sync/synchronization-object-security-and-access-rights

Valid Usage

- VUID-VkExportMemoryWin32HandleInfoKHR-handleTypes-00657
  If VkExportMemoryAllocateInfo::`handleTypes` does not include `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT`, a `VkExportMemoryWin32HandleInfoKHR` structure **must** not be included in the `pNext` chain of VkMemoryAllocateInfo

Valid Usage (Implicit)

- VUID-VkExportMemoryWin32HandleInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_MEMORY_WIN32_HANDLE_INFO_KHR`
- VUID-VkExportMemoryWin32HandleInfoKHR-pAttributes-parameter
  If `pAttributes` is not `NULL`, `pAttributes` **must** be a valid pointer to a valid `SECURITY_ATTRIBUTES` value

To import memory from a Windows handle, add a VkImportMemoryWin32HandleInfoKHR structure to the `pNext` chain of the VkMemoryAllocateInfo structure.

The `VkImportMemoryWin32HandleInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_memory_win32
typedef struct VkImportMemoryWin32HandleInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExternalMemoryHandleTypeFlagBits    handleType;
    HANDLE                                handle;
    LPCWSTR                               name;
} VkImportMemoryWin32HandleInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleType` is a VkExternalMemoryHandleTypeFlagBits value specifying the type of `handle` or `name`.
- `handle` is `NULL` or the external handle to import.
- `name` is `NULL` or a null-terminated UTF-16 string naming the payload to import.

Importing memory object payloads from Windows handles does not transfer ownership of the handle to the Vulkan implementation. For handle types defined as NT handles, the application **must** release handle ownership using the `CloseHandle` system call when the handle is no longer needed. For handle types defined as NT handles, the imported memory object holds a reference to its payload.

NOTE	Non-NT handle import operations do not add a reference to their associated payload. If the original object owning the payload is destroyed, all resources and handles sharing that payload will become invalid.

Applications **can** import the same payload into multiple instances of Vulkan, into the same instance from which it was exported, and multiple times into a given Vulkan instance. In all cases, each import operation **must** create a distinct `VkDeviceMemory` object.

Valid Usage

- VUID-VkImportMemoryWin32HandleInfoKHR-handleType-00658
  If `handleType` is not `0`, it **must** be supported for import, as reported by VkExternalImageFormatProperties or VkExternalBufferProperties
- VUID-VkImportMemoryWin32HandleInfoKHR-handle-00659
  The memory from which `handle` was exported, or the memory named by `name` **must** have been created on the same underlying physical device as `device`
- VUID-VkImportMemoryWin32HandleInfoKHR-handleType-00660
  If `handleType` is not `0`, it **must** be defined as an NT handle or a global share handle
- VUID-VkImportMemoryWin32HandleInfoKHR-handleType-01439
  If `handleType` is not `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT`, `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT`, or

VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT, name **must** be NULL

- VUID-VkImportMemoryWin32HandleInfoKHR-handleType-01440
  If handleType is not 0 and handle is NULL, name **must** name a valid memory resource of the type specified by handleType
- VUID-VkImportMemoryWin32HandleInfoKHR-handleType-00661
  If handleType is not 0 and name is NULL, handle **must** be a valid handle of the type specified by handleType
- VUID-VkImportMemoryWin32HandleInfoKHR-handle-01441
  If handle is not NULL, name **must** be NULL
- VUID-VkImportMemoryWin32HandleInfoKHR-handle-01518
  If handle is not NULL, it **must** obey any requirements listed for handleType in external memory handle types compatibility
- VUID-VkImportMemoryWin32HandleInfoKHR-name-01519
  If name is not NULL, it **must** obey any requirements listed for handleType in external memory handle types compatibility

Valid Usage (Implicit)

- VUID-VkImportMemoryWin32HandleInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMPORT_MEMORY_WIN32_HANDLE_INFO_KHR
- VUID-VkImportMemoryWin32HandleInfoKHR-handleType-parameter
  If handleType is not 0, handleType **must** be a valid VkExternalMemoryHandleTypeFlagBits value

To export a Windows handle representing the payload of a Vulkan device memory object, call:

```
// Provided by VK_KHR_external_memory_win32
VkResult vkGetMemoryWin32HandleKHR(
    VkDevice                                    device,
    const VkMemoryGetWin32HandleInfoKHR*        pGetWin32HandleInfo,
    HANDLE*                                     pHandle);
```

- device is the logical device that created the device memory being exported.
- pGetWin32HandleInfo is a pointer to a VkMemoryGetWin32HandleInfoKHR structure containing parameters of the export operation.
- pHandle will return the Windows handle representing the payload of the device memory object.

For handle types defined as NT handles, the handles returned by vkGetMemoryWin32HandleKHR are owned by the application and hold a reference to their payload. To avoid leaking resources, the application **must** release ownership of them using the CloseHandle system call when they are no longer needed.

NOTE Non-NT handle types do not add a reference to their associated payload. If the

> original object owning the payload is destroyed, all resources and handles sharing that payload will become invalid.

Valid Usage (Implicit)

- VUID-vkGetMemoryWin32HandleKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetMemoryWin32HandleKHR-pGetWin32HandleInfo-parameter
  `pGetWin32HandleInfo` **must** be a valid pointer to a valid VkMemoryGetWin32HandleInfoKHR structure
- VUID-vkGetMemoryWin32HandleKHR-pHandle-parameter
  `pHandle` **must** be a valid pointer to a `HANDLE` value

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_TOO_MANY_OBJECTS`
- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkMemoryGetWin32HandleInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_memory_win32
typedef struct VkMemoryGetWin32HandleInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkDeviceMemory                        memory;
    VkExternalMemoryHandleTypeFlagBits    handleType;
} VkMemoryGetWin32HandleInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memory` is the memory object from which the handle will be exported.
- `handleType` is a VkExternalMemoryHandleTypeFlagBits value specifying the type of handle requested.

The properties of the handle returned depend on the value of `handleType`. See VkExternalMemoryHandleTypeFlagBits for a description of the properties of the defined external memory handle types.

Valid Usage

- VUID-VkMemoryGetWin32HandleInfoKHR-handleType-00662
  `handleType` **must** have been included in VkExportMemoryAllocateInfo::`handleTypes` when `memory` was created
- VUID-VkMemoryGetWin32HandleInfoKHR-handleType-00663
  If `handleType` is defined as an NT handle, vkGetMemoryWin32HandleKHR **must** be called no more than once for each valid unique combination of `memory` and `handleType`
- VUID-VkMemoryGetWin32HandleInfoKHR-handleType-00664
  `handleType` **must** be defined as an NT handle or a global share handle

Valid Usage (Implicit)

- VUID-VkMemoryGetWin32HandleInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_GET_WIN32_HANDLE_INFO_KHR`
- VUID-VkMemoryGetWin32HandleInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkMemoryGetWin32HandleInfoKHR-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle
- VUID-VkMemoryGetWin32HandleInfoKHR-handleType-parameter
  `handleType` **must** be a valid VkExternalMemoryHandleTypeFlagBits value

Windows memory handles compatible with Vulkan **may** also be created by non-Vulkan APIs using methods beyond the scope of this specification. To determine the correct parameters to use when importing such handles, call:

```
// Provided by VK_KHR_external_memory_win32
VkResult vkGetMemoryWin32HandlePropertiesKHR(
    VkDevice                                    device,
    VkExternalMemoryHandleTypeFlagBits          handleType,
    HANDLE                                      handle,
    VkMemoryWin32HandlePropertiesKHR*           pMemoryWin32HandleProperties);
```

- `device` is the logical device that will be importing `handle`.
- `handleType` is a VkExternalMemoryHandleTypeFlagBits value specifying the type of the handle `handle`.
- `handle` is the handle which will be imported.
- `pMemoryWin32HandleProperties` is a pointer to a VkMemoryWin32HandlePropertiesKHR structure in which properties of `handle` are returned.

Valid Usage

- VUID-vkGetMemoryWin32HandlePropertiesKHR-handle-00665
  `handle` **must** point to a valid Windows memory handle
- VUID-vkGetMemoryWin32HandlePropertiesKHR-handleType-00666
  `handleType` **must** not be one of the handle types defined as opaque

Valid Usage (Implicit)

- VUID-vkGetMemoryWin32HandlePropertiesKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetMemoryWin32HandlePropertiesKHR-handleType-parameter
  `handleType` **must** be a valid VkExternalMemoryHandleTypeFlagBits value
- VUID-vkGetMemoryWin32HandlePropertiesKHR-pMemoryWin32HandleProperties-parameter
  `pMemoryWin32HandleProperties` **must** be a valid pointer to a VkMemoryWin32HandlePropertiesKHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_INVALID_EXTERNAL_HANDLE`

The `VkMemoryWin32HandlePropertiesKHR` structure returned is defined as:

```
// Provided by VK_KHR_external_memory_win32
typedef struct VkMemoryWin32HandlePropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           memoryTypeBits;
} VkMemoryWin32HandlePropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memoryTypeBits` is a bitmask containing one bit set for every memory type which the specified windows handle **can** be imported as.

Valid Usage (Implicit)

- VUID-VkMemoryWin32HandlePropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_WIN32_HANDLE_PROPERTIES_KHR`
- VUID-VkMemoryWin32HandlePropertiesKHR-pNext-pNext
  `pNext` **must** be `NULL`

11.2.5. File Descriptor External Memory

To import memory from a POSIX file descriptor handle, add a VkImportMemoryFdInfoKHR structure to the `pNext` chain of the VkMemoryAllocateInfo structure. The `VkImportMemoryFdInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_memory_fd
typedef struct VkImportMemoryFdInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExternalMemoryHandleTypeFlagBits    handleType;
    int                                   fd;
} VkImportMemoryFdInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleType` is a VkExternalMemoryHandleTypeFlagBits value specifying the handle type of `fd`.
- `fd` is the external handle to import.

Importing memory from a file descriptor transfers ownership of the file descriptor from the application to the Vulkan implementation. The application **must** not perform any operations on the file descriptor after a successful import. The imported memory object holds a reference to its payload.

Applications **can** import the same payload into multiple instances of Vulkan, into the same instance from which it was exported, and multiple times into a given Vulkan instance. In all cases, each import operation **must** create a distinct `VkDeviceMemory` object.

Valid Usage

- VUID-VkImportMemoryFdInfoKHR-handleType-00667
  If `handleType` is not `0`, it **must** be supported for import, as reported by VkExternalImageFormatProperties or VkExternalBufferProperties
- VUID-VkImportMemoryFdInfoKHR-fd-00668
  The memory from which `fd` was exported **must** have been created on the same underlying physical device as `device`
- VUID-VkImportMemoryFdInfoKHR-handleType-00669

If `handleType` is not `0`, it **must** be `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT` or `VK_EXTERNAL_MEMORY_HANDLE_TYPE_DMA_BUF_BIT_EXT`

- VUID-VkImportMemoryFdInfoKHR-handleType-00670
  If `handleType` is not `0`, `fd` **must** be a valid handle of the type specified by `handleType`
- VUID-VkImportMemoryFdInfoKHR-fd-01746
  The memory represented by `fd` **must** have been created from a physical device and driver that is compatible with `device` and `handleType`, as described in External Memory Handle Types Compatibility
- VUID-VkImportMemoryFdInfoKHR-fd-01520
  `fd` **must** obey any requirements listed for `handleType` in external memory handle types compatibility

Valid Usage (Implicit)

- VUID-VkImportMemoryFdInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_MEMORY_FD_INFO_KHR`
- VUID-VkImportMemoryFdInfoKHR-handleType-parameter
  If `handleType` is not `0`, `handleType` **must** be a valid VkExternalMemoryHandleTypeFlagBits value

To export a POSIX file descriptor referencing the payload of a Vulkan device memory object, call:

```
// Provided by VK_KHR_external_memory_fd
VkResult vkGetMemoryFdKHR(
    VkDevice                                    device,
    const VkMemoryGetFdInfoKHR*                 pGetFdInfo,
    int*                                        pFd);
```

- `device` is the logical device that created the device memory being exported.
- `pGetFdInfo` is a pointer to a VkMemoryGetFdInfoKHR structure containing parameters of the export operation.
- `pFd` will return a file descriptor referencing the payload of the device memory object.

Each call to `vkGetMemoryFdKHR` **must** create a new file descriptor holding a reference to the memory object's payload and transfer ownership of the file descriptor to the application. To avoid leaking resources, the application **must** release ownership of the file descriptor using the `close` system call when it is no longer needed, or by importing a Vulkan memory object from it. Where supported by the operating system, the implementation **must** set the file descriptor to be closed automatically when an `execve` system call is made.

Valid Usage (Implicit)

- VUID-vkGetMemoryFdKHR-device-parameter

`device` **must** be a valid VkDevice handle

- VUID-vkGetMemoryFdKHR-pGetFdInfo-parameter
  `pGetFdInfo` **must** be a valid pointer to a valid VkMemoryGetFdInfoKHR structure
- VUID-vkGetMemoryFdKHR-pFd-parameter
  `pFd` **must** be a valid pointer to an `int` value

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_TOO_MANY_OBJECTS`
- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkMemoryGetFdInfoKHR` structure is defined as:

```
// Provided by VK_KHR_external_memory_fd
typedef struct VkMemoryGetFdInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkDeviceMemory                        memory;
    VkExternalMemoryHandleTypeFlagBits    handleType;
} VkMemoryGetFdInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memory` is the memory object from which the handle will be exported.
- `handleType` is a VkExternalMemoryHandleTypeFlagBits value specifying the type of handle requested.

The properties of the file descriptor exported depend on the value of `handleType`. See VkExternalMemoryHandleTypeFlagBits for a description of the properties of the defined external memory handle types.

NOTE The size of the exported file **may** be larger than the size requested by VkMemoryAllocateInfo::`allocationSize`. If `handleType` is `VK_EXTERNAL_MEMORY_HANDLE_TYPE_DMA_BUF_BIT_EXT`, then the application **can** query the file's actual size with `lseek`.

Valid Usage

- VUID-VkMemoryGetFdInfoKHR-handleType-00671

handleType **must** have been included in VkExportMemoryAllocateInfo::handleTypes when memory was created

- VUID-VkMemoryGetFdInfoKHR-handleType-00672
handleType **must** be VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT or VK_EXTERNAL_MEMORY_HANDLE_TYPE_DMA_BUF_BIT_EXT

Valid Usage (Implicit)

- VUID-VkMemoryGetFdInfoKHR-sType-sType
sType **must** be VK_STRUCTURE_TYPE_MEMORY_GET_FD_INFO_KHR
- VUID-VkMemoryGetFdInfoKHR-pNext-pNext
pNext **must** be NULL
- VUID-VkMemoryGetFdInfoKHR-memory-parameter
memory **must** be a valid VkDeviceMemory handle
- VUID-VkMemoryGetFdInfoKHR-handleType-parameter
handleType **must** be a valid VkExternalMemoryHandleTypeFlagBits value

POSIX file descriptor memory handles compatible with Vulkan **may** also be created by non-Vulkan APIs using methods beyond the scope of this specification. To determine the correct parameters to use when importing such handles, call:

```
// Provided by VK_KHR_external_memory_fd
VkResult vkGetMemoryFdPropertiesKHR(
    VkDevice                                    device,
    VkExternalMemoryHandleTypeFlagBits          handleType,
    int                                         fd,
    VkMemoryFdPropertiesKHR*                    pMemoryFdProperties);
```

- device is the logical device that will be importing fd.
- handleType is a VkExternalMemoryHandleTypeFlagBits value specifying the type of the handle fd.
- fd is the handle which will be imported.
- pMemoryFdProperties is a pointer to a VkMemoryFdPropertiesKHR structure in which the properties of the handle fd are returned.

Valid Usage

- VUID-vkGetMemoryFdPropertiesKHR-fd-00673
fd **must** point to a valid POSIX file descriptor memory handle
- VUID-vkGetMemoryFdPropertiesKHR-handleType-00674
handleType **must** not be VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT

Valid Usage (Implicit)

- VUID-vkGetMemoryFdPropertiesKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetMemoryFdPropertiesKHR-handleType-parameter
  `handleType` **must** be a valid VkExternalMemoryHandleTypeFlagBits value
- VUID-vkGetMemoryFdPropertiesKHR-pMemoryFdProperties-parameter
  `pMemoryFdProperties` **must** be a valid pointer to a VkMemoryFdPropertiesKHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_INVALID_EXTERNAL_HANDLE`

The `VkMemoryFdPropertiesKHR` structure returned is defined as:

```
// Provided by VK_KHR_external_memory_fd
typedef struct VkMemoryFdPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           memoryTypeBits;
} VkMemoryFdPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memoryTypeBits` is a bitmask containing one bit set for every memory type which the specified file descriptor **can** be imported as.

Valid Usage (Implicit)

- VUID-VkMemoryFdPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_FD_PROPERTIES_KHR`
- VUID-VkMemoryFdPropertiesKHR-pNext-pNext
  `pNext` **must** be `NULL`

11.2.6. Host External Memory

To import memory from a host pointer, add a VkImportMemoryHostPointerInfoEXT structure to

the pNext chain of the VkMemoryAllocateInfo structure. The VkImportMemoryHostPointerInfoEXT structure is defined as:

```
// Provided by VK_EXT_external_memory_host
typedef struct VkImportMemoryHostPointerInfoEXT {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExternalMemoryHandleTypeFlagBits    handleType;
    void*                                 pHostPointer;
} VkImportMemoryHostPointerInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- handleType is a VkExternalMemoryHandleTypeFlagBits value specifying the handle type.
- pHostPointer is the host pointer to import from.

Importing memory from a host pointer shares ownership of the memory between the host and the Vulkan implementation. The application **can** continue to access the memory through the host pointer but it is the application's responsibility to synchronize device and non-device access to the payload as defined in Host Access to Device Memory Objects.

Applications **can** import the same payload into multiple instances of Vulkan and multiple times into a given Vulkan instance. However, implementations **may** fail to import the same payload multiple times into a given physical device due to platform constraints.

Importing memory from a particular host pointer **may** not be possible due to additional platform-specific restrictions beyond the scope of this specification in which case the implementation **must** fail the memory import operation with the error code VK_ERROR_INVALID_EXTERNAL_HANDLE_KHR.

Whether device memory objects imported from a host pointer hold a reference to their payload is undefined. As such, the application **must** ensure that the imported memory range remains valid and accessible for the lifetime of the imported memory object.

Implementations **may** support importing host pointers for memory types which are not host-visible. In this case, after a successful call to vkAllocateMemory, the memory range imported from pHostPointer **must** not be accessed by the application until the VkDeviceMemory has been destroyed. Memory contents for the host memory becomes undefined on import, and is left undefined after the VkDeviceMemory has been destroyed. Applications **must** also not access host memory which is mapped to the same physical memory as pHostPointer, but mapped to a different host pointer while the VkDeviceMemory handle is valid. Implementations running on general-purpose operating systems **should** not support importing host pointers for memory types which are not host-visible.

NOTE	Using host pointers to back non-host visible allocations is a platform-specific use case, and applications should not attempt to do this unless instructed by the platform.

Valid Usage

- VUID-VkImportMemoryHostPointerInfoEXT-handleType-01747
  If `handleType` is not `0`, it **must** be supported for import, as reported in VkExternalMemoryProperties
- VUID-VkImportMemoryHostPointerInfoEXT-handleType-01748
  If `handleType` is not `0`, it **must** be `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT` or `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT`
- VUID-VkImportMemoryHostPointerInfoEXT-pHostPointer-01749
  `pHostPointer` **must** be a pointer aligned to an integer multiple of `VkPhysicalDeviceExternalMemoryHostPropertiesEXT`::`minImportedHostPointerAlignment`
- VUID-VkImportMemoryHostPointerInfoEXT-handleType-01750
  If `handleType` is `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT`, `pHostPointer` **must** be a pointer to `allocationSize` number of bytes of host memory, where `allocationSize` is the member of the `VkMemoryAllocateInfo` structure this structure is chained to
- VUID-VkImportMemoryHostPointerInfoEXT-handleType-01751
  If `handleType` is `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT`, `pHostPointer` **must** be a pointer to `allocationSize` number of bytes of host mapped foreign memory, where `allocationSize` is the member of the `VkMemoryAllocateInfo` structure this structure is chained to

Valid Usage (Implicit)

- VUID-VkImportMemoryHostPointerInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_MEMORY_HOST_POINTER_INFO_EXT`
- VUID-VkImportMemoryHostPointerInfoEXT-handleType-parameter
  `handleType` **must** be a valid VkExternalMemoryHandleTypeFlagBits value
- VUID-VkImportMemoryHostPointerInfoEXT-pHostPointer-parameter
  `pHostPointer` **must** be a pointer value

To determine the correct parameters to use when importing host pointers, call:

```
// Provided by VK_EXT_external_memory_host
VkResult vkGetMemoryHostPointerPropertiesEXT(
    VkDevice                                    device,
    VkExternalMemoryHandleTypeFlagBits          handleType,
    const void*                                 pHostPointer,
    VkMemoryHostPointerPropertiesEXT*           pMemoryHostPointerProperties);
```

- `device` is the logical device that will be importing `pHostPointer`.
- `handleType` is a VkExternalMemoryHandleTypeFlagBits value specifying the type of the handle

pHostPointer.

- pHostPointer is the host pointer to import from.
- pMemoryHostPointerProperties is a pointer to a VkMemoryHostPointerPropertiesEXT structure in which the host pointer properties are returned.

Valid Usage

- VUID-vkGetMemoryHostPointerPropertiesEXT-handleType-01752
  handleType **must** be VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT or VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT
- VUID-vkGetMemoryHostPointerPropertiesEXT-pHostPointer-01753
  pHostPointer **must** be a pointer aligned to an integer multiple of VkPhysicalDeviceExternalMemoryHostPropertiesEXT::minImportedHostPointerAlignment
- VUID-vkGetMemoryHostPointerPropertiesEXT-handleType-01754
  If handleType is VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT, pHostPointer **must** be a pointer to host memory
- VUID-vkGetMemoryHostPointerPropertiesEXT-handleType-01755
  If handleType is VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT, pHostPointer **must** be a pointer to host mapped foreign memory

Valid Usage (Implicit)

- VUID-vkGetMemoryHostPointerPropertiesEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetMemoryHostPointerPropertiesEXT-handleType-parameter
  handleType **must** be a valid VkExternalMemoryHandleTypeFlagBits value
- VUID-vkGetMemoryHostPointerPropertiesEXT-pHostPointer-parameter
  pHostPointer **must** be a pointer value
- VUID-vkGetMemoryHostPointerPropertiesEXT-pMemoryHostPointerProperties-parameter
  pMemoryHostPointerProperties **must** be a valid pointer to a VkMemoryHostPointerPropertiesEXT structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_INVALID_EXTERNAL_HANDLE

The VkMemoryHostPointerPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_external_memory_host
typedef struct VkMemoryHostPointerPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           memoryTypeBits;
} VkMemoryHostPointerPropertiesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- memoryTypeBits is a bitmask containing one bit set for every memory type which the specified host pointer **can** be imported as.

The value returned by memoryTypeBits **should** only include bits that identify memory types which are host visible. Implementations **may** include bits that identify memory types which are not host visible. Behavior for imported pointers of such types is defined by VkImportMemoryHostPointerInfoEXT.

Valid Usage (Implicit)

- VUID-VkMemoryHostPointerPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_MEMORY_HOST_POINTER_PROPERTIES_EXT
- VUID-VkMemoryHostPointerPropertiesEXT-pNext-pNext
  pNext **must** be NULL

11.2.7. Metal Objects

A Vulkan implementation that is layered on top of Metal on Apple device platform, and implements the VK_EXT_metal_objects extension, supports the ability to import and export the underlying Metal objects associated with specific Vulkan objects.

The underlying Metal objects associated with certain Vulkan objects can be exported from those Vulkan objects using the pNext chain of the VkExportMetalObjectsInfoEXT parameter of the vkExportMetalObjectsEXT command.

An VkDeviceMemory object can be allocated on an existing MTLBuffer object, by including the MTLBuffer object in a VkImportMetalBufferInfoEXT structure in the pNext chain of the VkMemoryAllocateInfo structure in the vkAllocateMemory command.

A new VkImage object can be created on an existing IOSurface object, or one or more existing Metal MTLTexture objects, by including those Metal objects in either VkImportMetalIOSurfaceInfoEXT or VkImportMetalTextureInfoEXT structures in the pNext chain of the VkImageCreateInfo structure in the vkCreateImage command.

To export Metal objects from Vulkan objects, the application **must** first indicate the intention to do

so during the creation of the Vulkan object, by including one or more VkExportMetalObjectCreateInfoEXT structures in the pNext chain of the VkInstanceCreateInfo, VkMemoryAllocateInfo, VkImageCreateInfo, VkImageViewCreateInfo, VkBufferViewCreateInfo, VkSemaphoreCreateInfo, or VkEventCreateInfo, in the corresponding Vulkan object creation command.

The VkExportMetalObjectCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkExportMetalObjectCreateInfoEXT {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExportMetalObjectTypeFlagBitsEXT    exportObjectType;
} VkExportMetalObjectCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- exportObjectType is a VkExportMetalObjectTypeFlagBitsEXT indicating the type of Metal object that the application may request to be exported from the Vulkan object.

Valid Usage (Implicit)

- VUID-VkExportMetalObjectCreateInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_EXPORT_METAL_OBJECT_CREATE_INFO_EXT
- VUID-VkExportMetalObjectCreateInfoEXT-exportObjectType-parameter
  If exportObjectType is not 0, exportObjectType **must** be a valid VkExportMetalObjectTypeFlagBitsEXT value

Bits which indicate the types of Metal objects that may be exported from a corresponding Vulkan object are:

```
// Provided by VK_EXT_metal_objects
typedef enum VkExportMetalObjectTypeFlagBitsEXT {
    VK_EXPORT_METAL_OBJECT_TYPE_METAL_DEVICE_BIT_EXT = 0x00000001,
    VK_EXPORT_METAL_OBJECT_TYPE_METAL_COMMAND_QUEUE_BIT_EXT = 0x00000002,
    VK_EXPORT_METAL_OBJECT_TYPE_METAL_BUFFER_BIT_EXT = 0x00000004,
    VK_EXPORT_METAL_OBJECT_TYPE_METAL_TEXTURE_BIT_EXT = 0x00000008,
    VK_EXPORT_METAL_OBJECT_TYPE_METAL_IOSURFACE_BIT_EXT = 0x00000010,
    VK_EXPORT_METAL_OBJECT_TYPE_METAL_SHARED_EVENT_BIT_EXT = 0x00000020,
} VkExportMetalObjectTypeFlagBitsEXT;
```

- VK_EXPORT_METAL_OBJECT_TYPE_METAL_DEVICE_BIT_EXT specifies that a Metal MTLDevice may be exported.
- VK_EXPORT_METAL_OBJECT_TYPE_METAL_COMMAND_QUEUE_BIT_EXT specifies that a Metal MTLCommandQueue may be exported.

- VK_EXPORT_METAL_OBJECT_TYPE_METAL_BUFFER_BIT_EXT specifies that a Metal MTLBuffer may be exported.
- VK_EXPORT_METAL_OBJECT_TYPE_METAL_TEXTURE_BIT_EXT specifies that a Metal MTLTexture may be exported.
- VK_EXPORT_METAL_OBJECT_TYPE_METAL_IOSURFACE_BIT_EXT specifies that a Metal IOSurface may be exported.
- VK_EXPORT_METAL_OBJECT_TYPE_METAL_SHARED_EVENT_BIT_EXT specifies that a Metal MTLSharedEvent may be exported.

```
// Provided by VK_EXT_metal_objects
typedef VkFlags VkExportMetalObjectTypeFlagsEXT;
```

VkExportMetalObjectTypeFlagsEXT is a bitmask type for setting a mask of zero or more VkExportMetalObjectTypeFlagBitsEXT.

To export Metal objects that underlie Vulkan objects, call:

```
// Provided by VK_EXT_metal_objects
void vkExportMetalObjectsEXT(
    VkDevice                                    device,
    VkExportMetalObjectsInfoEXT*                pMetalObjectsInfo);
```

- device is the device that created the Vulkan objects.
- pMetalObjectsInfo is a pointer to a VkExportMetalObjectsInfoEXT structure whose pNext chain contains structures, each identifying a Vulkan object and providing a pointer through which the Metal object will be returned.

Valid Usage (Implicit)

- VUID-vkExportMetalObjectsEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkExportMetalObjectsEXT-pMetalObjectsInfo-parameter
  pMetalObjectsInfo **must** be a valid pointer to a VkExportMetalObjectsInfoEXT structure

The VkExportMetalObjectsInfoEXT structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkExportMetalObjectsInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
} VkExportMetalObjectsInfoEXT;
```

- sType is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.

Valid Usage

- VUID-VkExportMetalObjectsInfoEXT-pNext-06791
  If the pNext chain includes a VkExportMetalDeviceInfoEXT structure, the VkInstance **must** have been created with VK_EXPORT_METAL_OBJECT_TYPE_METAL_DEVICE_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkInstanceCreateInfo structure in the vkCreateInstance command
- VUID-VkExportMetalObjectsInfoEXT-pNext-06792
  If the pNext chain includes a VkExportMetalCommandQueueInfoEXT structure, the VkInstance **must** have been created with VK_EXPORT_METAL_OBJECT_TYPE_METAL_COMMAND_QUEUE_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkInstanceCreateInfo structure in the vkCreateInstance command
- VUID-VkExportMetalObjectsInfoEXT-pNext-06793
  If the pNext chain includes a VkExportMetalBufferInfoEXT structure, the VkDeviceMemory in its memory member **must** have been allocated with VK_EXPORT_METAL_OBJECT_TYPE_METAL_BUFFER_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkMemoryAllocateInfo structure in the vkAllocateMemory command
- VUID-VkExportMetalObjectsInfoEXT-pNext-06794
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, exactly one of its image, imageView, or bufferView members **must** not be VK_NULL_HANDLE
- VUID-VkExportMetalObjectsInfoEXT-pNext-06795
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, and its image member is not VK_NULL_HANDLE, the VkImage in its image member **must** have been created with VK_EXPORT_METAL_OBJECT_TYPE_METAL_TEXTURE_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkImageCreateInfo structure in the vkCreateImage command
- VUID-VkExportMetalObjectsInfoEXT-pNext-06796
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, and its imageView member is not VK_NULL_HANDLE, the VkImageView in its imageView member **must** have been created with VK_EXPORT_METAL_OBJECT_TYPE_METAL_TEXTURE_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkImageViewCreateInfo structure in the vkCreateImageView command
- VUID-VkExportMetalObjectsInfoEXT-pNext-06797
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, and its bufferView member is not VK_NULL_HANDLE, the VkBufferView in its bufferView member **must** have been created with VK_EXPORT_METAL_OBJECT_TYPE_METAL_TEXTURE_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkBufferViewCreateInfo structure in the vkCreateBufferView command
- VUID-VkExportMetalObjectsInfoEXT-pNext-06798
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, and if either its image or imageView member is not VK_NULL_HANDLE, then plane **must** be

VK_IMAGE_ASPECT_PLANE_0_BIT, VK_IMAGE_ASPECT_PLANE_1_BIT, or VK_IMAGE_ASPECT_PLANE_2_BIT

- VUID-VkExportMetalObjectsInfoEXT-pNext-06799
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, and if the VkImage in its image member does not have a multi-planar format, then its plane member **must** be VK_IMAGE_ASPECT_PLANE_0_BIT
- VUID-VkExportMetalObjectsInfoEXT-pNext-06800
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, and if the VkImage in its image member has a multi-planar format with only two planes, then its plane member **must** not be VK_IMAGE_ASPECT_PLANE_2_BIT
- VUID-VkExportMetalObjectsInfoEXT-pNext-06801
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, and if the VkImageView in its imageView member does not have a multi-planar format, then its plane member **must** be VK_IMAGE_ASPECT_PLANE_0_BIT
- VUID-VkExportMetalObjectsInfoEXT-pNext-06802
  If the pNext chain includes a VkExportMetalTextureInfoEXT structure, and if the VkImageView in its imageView member has a multi-planar format with only two planes, then its plane member **must** not be VK_IMAGE_ASPECT_PLANE_2_BIT
- VUID-VkExportMetalObjectsInfoEXT-pNext-06803
  If the pNext chain includes a VkExportMetalIOSurfaceInfoEXT structure, the VkImage in its image member **must** have been created with VK_EXPORT_METAL_OBJECT_TYPE_METAL_IOSURFACE_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkImageCreateInfo structure in the vkCreateImage command
- VUID-VkExportMetalObjectsInfoEXT-pNext-06804
  If the pNext chain includes a VkExportMetalSharedEventInfoEXT structure, exactly one of its semaphore or event members **must** not be VK_NULL_HANDLE
- VUID-VkExportMetalObjectsInfoEXT-pNext-06805
  If the pNext chain includes a VkExportMetalSharedEventInfoEXT structure, and its semaphore member is not VK_NULL_HANDLE, the VkSemaphore in its semaphore member **must** have been created with VK_EXPORT_METAL_OBJECT_TYPE_METAL_SHARED_EVENT_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkSemaphoreCreateInfo structure in the vkCreateSemaphore command
- VUID-VkExportMetalObjectsInfoEXT-pNext-06806
  If the pNext chain includes a VkExportMetalSharedEventInfoEXT structure, and its event member is not VK_NULL_HANDLE, the VkEvent in its event member **must** have been created with VK_EXPORT_METAL_OBJECT_TYPE_METAL_SHARED_EVENT_BIT_EXT in the exportObjectType member of a VkExportMetalObjectCreateInfoEXT structure in the pNext chain of the VkEventCreateInfo structure in the vkCreateEvent command

Valid Usage (Implicit)

- VUID-VkExportMetalObjectsInfoEXT-sType-sType

sType **must** be VK_STRUCTURE_TYPE_EXPORT_METAL_OBJECTS_INFO_EXT

- VUID-VkExportMetalObjectsInfoEXT-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkExportMetalBufferInfoEXT, VkExportMetalCommandQueueInfoEXT, VkExportMetalDeviceInfoEXT, VkExportMetalIOSurfaceInfoEXT, VkExportMetalSharedEventInfoEXT, or VkExportMetalTextureInfoEXT
- VUID-VkExportMetalObjectsInfoEXT-sType-unique
  The sType value of each struct in the pNext chain **must** be unique, with the exception of structures of type VkExportMetalBufferInfoEXT, VkExportMetalCommandQueueInfoEXT, VkExportMetalIOSurfaceInfoEXT, VkExportMetalSharedEventInfoEXT, or VkExportMetalTextureInfoEXT

To export the Metal MTLDevice object underlying the VkPhysicalDevice associated with a VkDevice object, include a VkExportMetalDeviceInfoEXT structure in the pNext chain of the pMetalObjectsInfo parameter of a vkExportMetalObjectsEXT call.

The VkExportMetalDeviceInfoEXT structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkExportMetalDeviceInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    MTLDevice_id       mtlDevice;
} VkExportMetalDeviceInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- mtlDevice is the Metal id<MTLDevice> object underlying the VkPhysicalDevice associated with the VkDevice object identified in the call. The implementation will return the MTLDevice in this member, or it will return NULL if no MTLDevice could be found underlying the VkPhysicalDevice object.

Valid Usage (Implicit)

- VUID-VkExportMetalDeviceInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_EXPORT_METAL_DEVICE_INFO_EXT

The type id<MTLDevice> is defined in Apple's Metal framework, but to remove an unnecessary compile time dependency, an incomplete type definition of MTLDevice_id is provided in the Vulkan headers:

```
// Provided by VK_EXT_metal_objects
#ifdef __OBJC__
@protocol MTLDevice;
typedef __unsafe_unretained id<MTLDevice> MTLDevice_id;
#else
typedef void* MTLDevice_id;
#endif
```

To export the Metal MTLCommandQueue object underlying a VkQueue object, include a VkExportMetalCommandQueueInfoEXT structure in the pNext chain of the pMetalObjectsInfo parameter of a vkExportMetalObjectsEXT call.

The VkExportMetalCommandQueueInfoEXT structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkExportMetalCommandQueueInfoEXT {
    VkStructureType       sType;
    const void*           pNext;
    VkQueue               queue;
    MTLCommandQueue_id    mtlCommandQueue;
} VkExportMetalCommandQueueInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- queue is a VkQueue.
- mtlCommandQueue is the Metal id<MTLCommandQueue> object underlying the VkQueue object in queue. The implementation will return the MTLCommandQueue in this member, or it will return NULL if no MTLCommandQueue could be found underlying the VkQueue object.

Valid Usage (Implicit)

- VUID-VkExportMetalCommandQueueInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_EXPORT_METAL_COMMAND_QUEUE_INFO_EXT
- VUID-VkExportMetalCommandQueueInfoEXT-queue-parameter
  queue **must** be a valid VkQueue handle

The type id<MTLCommandQueue> is defined in Apple's Metal framework, but to remove an unnecessary compile time dependency, an incomplete type definition of MTLCommandQueue_id is provided in the Vulkan headers:

```
// Provided by VK_EXT_metal_objects
#ifdef __OBJC__
@protocol MTLCommandQueue;
typedef __unsafe_unretained id<MTLCommandQueue> MTLCommandQueue_id;
#else
typedef void* MTLCommandQueue_id;
#endif
```

To export the Metal `MTLBuffer` object underlying a VkDeviceMemory object, include a `VkExportMetalBufferInfoEXT` structure in the `pNext` chain of the `pMetalObjectsInfo` parameter of a vkExportMetalObjectsEXT call.

The `VkExportMetalBufferInfoEXT` structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkExportMetalBufferInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkDeviceMemory     memory;
    MTLBuffer_id       mtlBuffer;
} VkExportMetalBufferInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memory` is a VkDeviceMemory.
- `mtlBuffer` is the Metal `id<MTLBuffer>` object underlying the VkDeviceMemory object in `memory`. The implementation will return the `MTLBuffer` in this member, or it will return `NULL` if no `MTLBuffer` could be found underlying the VkDeviceMemory object.

Valid Usage (Implicit)

- VUID-VkExportMetalBufferInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_METAL_BUFFER_INFO_EXT`
- VUID-VkExportMetalBufferInfoEXT-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle

To import a Metal `MTLBuffer` object to underlie a VkDeviceMemory object, include a `VkImportMetalBufferInfoEXT` structure in the `pNext` chain of the VkMemoryAllocateInfo structure in a vkAllocateMemory command.

The `VkImportMetalBufferInfoEXT` structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkImportMetalBufferInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    MTLBuffer_id       mtlBuffer;
} VkImportMetalBufferInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `mtlBuffer` is the Metal `id<MTLBuffer>` object that is to underlie the VkDeviceMemory.

The application **must** ensure that the configuration of the `id<MTLBuffer>` object is compatible with the configuration of the VkDeviceMemory. Failure to do so results in undefined behavior.

Valid Usage (Implicit)

- VUID-VkImportMetalBufferInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_METAL_BUFFER_INFO_EXT`

The type `id<MTLBuffer>` is defined in Apple's Metal framework, but to remove an unnecessary compile time dependency, an incomplete type definition of MTLBuffer_id is provided in the Vulkan headers:

```
// Provided by VK_EXT_metal_objects
#ifdef __OBJC__
@protocol MTLBuffer;
typedef __unsafe_unretained id<MTLBuffer> MTLBuffer_id;
#else
typedef void* MTLBuffer_id;
#endif
```

To export a Metal `MTLTexture` object underlying a VkImage, VkImageView, or VkBufferView object, include a `VkExportMetalTextureInfoEXT` structure in the `pNext` chain of the `pMetalObjectsInfo` parameter of a vkExportMetalObjectsEXT call.

The `VkExportMetalTextureInfoEXT` structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkExportMetalTextureInfoEXT {
    VkStructureType          sType;
    const void*              pNext;
    VkImage                  image;
    VkImageView              imageView;
    VkBufferView             bufferView;
    VkImageAspectFlagBits    plane;
    MTLTexture_id            mtlTexture;
} VkExportMetalTextureInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `image` is VK_NULL_HANDLE or a VkImage.
- `imageView` is VK_NULL_HANDLE or a VkImageView.
- `bufferView` is VK_NULL_HANDLE or a VkBufferView.
- `plane` specifies the plane of a multi-planar VkImage or VkImageView.
- `mtlTexture` is the Metal `id<MTLTexture>` object underlying the VkImage, VkImageView, or VkBufferView object in `image`, `imageView`, or `bufferView`, respectively, at the plane indicated in `aspectMask`. The implementation will return the `MTLTexture` in this member, or it will return `NULL` if no `MTLTexture` could be found underlying the VkImage, VkImageView, or VkBufferView object, at the plane indicated in `aspectMask`.

Valid Usage (Implicit)

- VUID-VkExportMetalTextureInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_METAL_TEXTURE_INFO_EXT`
- VUID-VkExportMetalTextureInfoEXT-image-parameter
  If `image` is not VK_NULL_HANDLE, `image` **must** be a valid VkImage handle
- VUID-VkExportMetalTextureInfoEXT-imageView-parameter
  If `imageView` is not VK_NULL_HANDLE, `imageView` **must** be a valid VkImageView handle
- VUID-VkExportMetalTextureInfoEXT-bufferView-parameter
  If `bufferView` is not VK_NULL_HANDLE, `bufferView` **must** be a valid VkBufferView handle
- VUID-VkExportMetalTextureInfoEXT-plane-parameter
  `plane` **must** be a valid VkImageAspectFlagBits value
- VUID-VkExportMetalTextureInfoEXT-commonparent
  Each of `bufferView`, `image`, and `imageView` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

To import one or more existing Metal `MTLTexture` objects to underlie a VkImage object, include one or more `VkImportMetalTextureInfoEXT` structures in the `pNext` chain of the VkImageCreateInfo structure in a vkCreateImage command.

The VkImportMetalTextureInfoEXT structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkImportMetalTextureInfoEXT {
    VkStructureType          sType;
    const void*              pNext;
    VkImageAspectFlagBits    plane;
    MTLTexture_id            mtlTexture;
} VkImportMetalTextureInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- plane specifies the plane of the VkImage that the id<MTLTexture> object should be attached to.
- mtlTexture is a the Metal id<MTLTexture> object that is to underlie the VkImage plane.

The pNext chain **must** include one VkImportMetalTextureInfoEXT structure for each plane in the VkImage. The application **must** ensure that the configuration of the Metal id<MTLTexture> objects are compatible with the configuration of the VkImage. Failure to do so results in undefined behavior.

Valid Usage (Implicit)

- VUID-VkImportMetalTextureInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMPORT_METAL_TEXTURE_INFO_EXT
- VUID-VkImportMetalTextureInfoEXT-plane-parameter
  plane **must** be a valid VkImageAspectFlagBits value

The type id<MTLTexture> is defined in Apple's Metal framework, but to remove an unnecessary compile time dependency, an incomplete type definition of MTLTexture_id is provided in the Vulkan headers:

```
// Provided by VK_EXT_metal_objects
#ifdef __OBJC__
@protocol MTLTexture;
typedef __unsafe_unretained id<MTLTexture> MTLTexture_id;
#else
typedef void* MTLTexture_id;
#endif
```

To export the Metal IOSurfaceRef object underlying a VkImage object, include a VkExportMetalIOSurfaceInfoEXT structure in the pNext chain of the pMetalObjectsInfo parameter of a vkExportMetalObjectsEXT call.

The VkExportMetalIOSurfaceInfoEXT structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkExportMetalIOSurfaceInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkImage            image;
    IOSurfaceRef       ioSurface;
} VkExportMetalIOSurfaceInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `image` is a VkImage.
- `ioSurface` is the Metal IOSurfaceRef object underlying the VkImage object in `image`. The implementation will return the IOSurfaceRef in this member, or it will return `NULL` if no IOSurfaceRef could be found underlying the VkImage object.

Valid Usage (Implicit)

- VUID-VkExportMetalIOSurfaceInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_METAL_IO_SURFACE_INFO_EXT`
- VUID-VkExportMetalIOSurfaceInfoEXT-image-parameter
  `image` **must** be a valid VkImage handle

To import, or create, a Metal IOSurfaceRef object to underlie a VkImage object, include a `VkImportMetalIOSurfaceInfoEXT` structure in the `pNext` chain of the VkImageCreateInfo structure in a vkCreateImage command.

The `VkImportMetalIOSurfaceInfoEXT` structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkImportMetalIOSurfaceInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    IOSurfaceRef       ioSurface;
} VkImportMetalIOSurfaceInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `ioSurface` is VK_NULL_HANDLE or the Metal IOSurfaceRef object that is to underlie the VkImage.

If `ioSurface` is not VK_NULL_HANDLE, it will be used to underlie the VkImage. If `ioSurface` is VK_NULL_HANDLE, the implementation will create a new `IOSurface` to underlie the VkImage.

If provided, the application **must** ensure that the configuration of the IOSurfaceRef object is

compatible with the configuration of the VkImage. Failure to do so results in undefined behavior.

Valid Usage (Implicit)

- VUID-VkImportMetalIOSurfaceInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_METAL_IO_SURFACE_INFO_EXT`

The type IOSurfaceRef is defined in Apple's CoreGraphics framework, but to remove an unnecessary compile time dependency, an incomplete type definition of IOSurfaceRef is provided in the Vulkan headers:

```
// Provided by VK_EXT_metal_objects
typedef struct __IOSurface* IOSurfaceRef;
```

To export the Metal `MTLSharedEvent` object underlying a VkSemaphore or VkEvent object, include a `VkExportMetalSharedEventInfoEXT` structure in the `pNext` chain of the `pMetalObjectsInfo` parameter of a vkExportMetalObjectsEXT call.

The `VkExportMetalSharedEventInfoEXT` structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkExportMetalSharedEventInfoEXT {
    VkStructureType      sType;
    const void*          pNext;
    VkSemaphore          semaphore;
    VkEvent              event;
    MTLSharedEvent_id    mtlSharedEvent;
} VkExportMetalSharedEventInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `semaphore` is VK_NULL_HANDLE or a VkSemaphore.
- `event` is VK_NULL_HANDLE or a VkEvent.
- `mtlSharedEvent` is the Metal `id<MTLSharedEvent>` object underlying the VkSemaphore or VkEvent object in `semaphore` or `event`, respectively. The implementation will return the `MTLSharedEvent` in this member, or it will return `NULL` if no `MTLSharedEvent` could be found underlying the VkSemaphore or VkEvent object.

Valid Usage (Implicit)

- VUID-VkExportMetalSharedEventInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXPORT_METAL_SHARED_EVENT_INFO_EXT`
- VUID-VkExportMetalSharedEventInfoEXT-semaphore-parameter
  If `semaphore` is not VK_NULL_HANDLE, `semaphore` **must** be a valid VkSemaphore handle

- VUID-VkExportMetalSharedEventInfoEXT-event-parameter
  If `event` is not VK_NULL_HANDLE, `event` **must** be a valid VkEvent handle
- VUID-VkExportMetalSharedEventInfoEXT-commonparent
  Both of `event`, and `semaphore` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

To import a Metal `id<MTLSharedEvent>` object to underlie a VkSemaphore or VkEvent object, include a `VkImportMetalSharedEventInfoEXT` structure in the `pNext` chain of the VkSemaphoreCreateInfo or VkEventCreateInfo structure in a vkCreateSemaphore or vkCreateEvent command, respectively.

The `VkImportMetalSharedEventInfoEXT` structure is defined as:

```
// Provided by VK_EXT_metal_objects
typedef struct VkImportMetalSharedEventInfoEXT {
    VkStructureType      sType;
    const void*          pNext;
    MTLSharedEvent_id    mtlSharedEvent;
} VkImportMetalSharedEventInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `mtlSharedEvent` is the Metal `id<MTLSharedEvent>` object that is to underlie the VkSemaphore or VkEvent.

If the `pNext` chain of the VkSemaphoreCreateInfo structure includes both `VkImportMetalSharedEventInfoEXT` and VkSemaphoreTypeCreateInfo, the `signaledValue` property of the imported `id<MTLSharedEvent>` object will be set to VkSemaphoreTypeCreateInfo::`initialValue`.

Valid Usage (Implicit)

- VUID-VkImportMetalSharedEventInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMPORT_METAL_SHARED_EVENT_INFO_EXT`

The type `id<MTLSharedEvent>` is defined in Apple's Metal framework, but to remove an unnecessary compile time dependency, an incomplete type definition of MTLSharedEvent_id is provided in the Vulkan headers:

```
// Provided by VK_EXT_metal_objects
#ifdef __OBJC__
@protocol MTLSharedEvent;
typedef __unsafe_unretained id<MTLSharedEvent> MTLSharedEvent_id;
#else
typedef void* MTLSharedEvent_id;
#endif
```

11.2.8. Device Group Memory Allocations

If the pNext chain of VkMemoryAllocateInfo includes a VkMemoryAllocateFlagsInfo structure, then that structure includes flags and a device mask controlling how many instances of the memory will be allocated.

The VkMemoryAllocateFlagsInfo structure is defined as:

```
typedef struct VkMemoryAllocateFlagsInfo {
    VkStructureType          sType;
    const void*              pNext;
    VkMemoryAllocateFlags    flags;
    uint32_t                 deviceMask;
} VkMemoryAllocateFlagsInfo;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkMemoryAllocateFlagsInfo VkMemoryAllocateFlagsInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkMemoryAllocateFlagBits controlling the allocation.
- deviceMask is a mask of physical devices in the logical device, indicating that memory **must** be allocated on each device in the mask, if VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT is set in flags.

If VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT is not set, the number of instances allocated depends on whether VK_MEMORY_HEAP_MULTI_INSTANCE_BIT is set in the memory heap. If VK_MEMORY_HEAP_MULTI_INSTANCE_BIT is set, then memory is allocated for every physical device in the logical device (as if deviceMask has bits set for all device indices). If VK_MEMORY_HEAP_MULTI_INSTANCE_BIT is not set, then a single instance of memory is allocated (as if deviceMask is set to one).

On some implementations, allocations from a multi-instance heap **may** consume memory on all physical devices even if the deviceMask excludes some devices. If VkPhysicalDeviceGroupProperties::subsetAllocation is VK_TRUE, then memory is only consumed for the devices in the device mask.

NOTE	In practice, most allocations on a multi-instance heap will be allocated across all physical devices. Unicast allocation support is an optional optimization for a minority of allocations.

Valid Usage

- VUID-VkMemoryAllocateFlagsInfo-deviceMask-00675

If VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT is set, deviceMask **must** be a valid device mask

- VUID-VkMemoryAllocateFlagsInfo-deviceMask-00676
  If VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT is set, deviceMask **must** not be zero

Valid Usage (Implicit)

- VUID-VkMemoryAllocateFlagsInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_FLAGS_INFO
- VUID-VkMemoryAllocateFlagsInfo-flags-parameter
  flags **must** be a valid combination of VkMemoryAllocateFlagBits values

Bits which **can** be set in VkMemoryAllocateFlagsInfo::flags, controlling device memory allocation, are:

```
typedef enum VkMemoryAllocateFlagBits {
    VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT = 0x00000001,
    VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT = 0x00000002,
    VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT = 0x00000004,
  // Provided by VK_KHR_device_group
    VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT_KHR = VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT,
  // Provided by VK_KHR_buffer_device_address
    VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT_KHR = VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT,
  // Provided by VK_KHR_buffer_device_address
    VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_KHR =
VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT,
} VkMemoryAllocateFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkMemoryAllocateFlagBits VkMemoryAllocateFlagBitsKHR;
```

- VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT specifies that memory will be allocated for the devices in VkMemoryAllocateFlagsInfo::deviceMask.
- VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT specifies that the memory **can** be attached to a buffer object created with the VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT bit set in usage, and that the memory handle **can** be used to retrieve an opaque address via vkGetDeviceMemoryOpaqueCaptureAddressKHR.
- VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT specifies that the memory's address **can** be saved and reused on a subsequent run (e.g. for trace capture and replay), see VkBufferOpaqueCaptureAddressCreateInfo for more detail.

```
typedef VkFlags VkMemoryAllocateFlags;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkMemoryAllocateFlags VkMemoryAllocateFlagsKHR;
```

VkMemoryAllocateFlags is a bitmask type for setting a mask of zero or more VkMemoryAllocateFlagBits.

11.2.9. Opaque Capture Address Allocation

To request a specific device address for a memory allocation, add a VkMemoryOpaqueCaptureAddressAllocateInfo structure to the pNext chain of the VkMemoryAllocateInfo structure. The VkMemoryOpaqueCaptureAddressAllocateInfo structure is defined as:

```
typedef struct VkMemoryOpaqueCaptureAddressAllocateInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint64_t           opaqueCaptureAddress;
} VkMemoryOpaqueCaptureAddressAllocateInfo;
```

or the equivalent

```
// Provided by VK_KHR_buffer_device_address
typedef VkMemoryOpaqueCaptureAddressAllocateInfo
VkMemoryOpaqueCaptureAddressAllocateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- opaqueCaptureAddress is the opaque capture address requested for the memory allocation.

If opaqueCaptureAddress is zero, no specific address is requested.

If opaqueCaptureAddress is not zero, it **should** be an address retrieved from vkGetDeviceMemoryOpaqueCaptureAddressKHR on an identically created memory allocation on the same implementation.

NOTE

In most cases, it is expected that a non-zero opaqueAddress is an address retrieved from vkGetDeviceMemoryOpaqueCaptureAddressKHR on an identically created memory allocation. If this is not the case, it is likely that VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS errors will occur.

This is, however, not a strict requirement because trace capture/replay tools may need to adjust memory allocation parameters for imported memory.

If this structure is not present, it is as if `opaqueCaptureAddress` is zero.

Valid Usage (Implicit)

- VUID-VkMemoryOpaqueCaptureAddressAllocateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_OPAQUE_CAPTURE_ADDRESS_ALLOCATE_INFO`

11.2.10. Freeing Device Memory

To free a memory object, call:

```
// Provided by VK_VERSION_1_0
void vkFreeMemory(
    VkDevice                                    device,
    VkDeviceMemory                              memory,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that owns the memory.
- `memory` is the VkDeviceMemory object to be freed.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Before freeing a memory object, an application **must** ensure the memory object is no longer in use by the device — for example by command buffers in the *pending state*. Memory **can** be freed whilst still bound to resources, but those resources **must** not be used afterwards. Freeing a memory object releases the reference it held, if any, to its payload. If there are still any bound images or buffers, the memory object's payload **may** not be immediately released by the implementation, but **must** be released by the time all bound images and buffers have been destroyed. Once all references to a payload are released, it is returned to the heap from which it was allocated.

How memory objects are bound to Images and Buffers is described in detail in the Resource Memory Association section.

If a memory object is mapped at the time it is freed, it is implicitly unmapped.

NOTE As described below, host writes are not implicitly flushed when the memory object is unmapped, but the implementation **must** guarantee that writes that have not been flushed do not affect any other memory.

Valid Usage

- VUID-vkFreeMemory-memory-00677
  All submitted commands that refer to `memory` (via images or buffers) **must** have completed

execution

Valid Usage (Implicit)

- VUID-vkFreeMemory-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkFreeMemory-memory-parameter
  If `memory` is not VK_NULL_HANDLE, `memory` **must** be a valid VkDeviceMemory handle
- VUID-vkFreeMemory-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkFreeMemory-memory-parent
  If `memory` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `memory` **must** be externally synchronized

11.2.11. Host Access to Device Memory Objects

Memory objects created with vkAllocateMemory are not directly host accessible.

Memory objects created with the memory property `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT` are considered *mappable*. Memory objects **must** be mappable in order to be successfully mapped on the host.

To retrieve a host virtual address pointer to a region of a mappable memory object, call:

```
// Provided by VK_VERSION_1_0
VkResult vkMapMemory(
    VkDevice                                    device,
    VkDeviceMemory                              memory,
    VkDeviceSize                                offset,
    VkDeviceSize                                size,
    VkMemoryMapFlags                            flags,
    void**                                      ppData);
```

- `device` is the logical device that owns the memory.
- `memory` is the VkDeviceMemory object to be mapped.
- `offset` is a zero-based byte offset from the beginning of the memory object.
- `size` is the size of the memory range to map, or `VK_WHOLE_SIZE` to map from `offset` to the end of the allocation.

- `flags` is a bitmask of VkMemoryMapFlagBits specifying additional parameters of the memory map operation.
- `ppData` is a pointer to a `void*` variable in which a host-accessible pointer to the beginning of the mapped range is returned. The value of the returned pointer minus `offset` **must** be aligned to VkPhysicalDeviceLimits::`minMemoryMapAlignment`.

After a successful call to `vkMapMemory` the memory object `memory` is considered to be currently *host mapped*.

NOTE	It is an application error to call `vkMapMemory` on a memory object that is already *host mapped*.

NOTE	`vkMapMemory` will fail if the implementation is unable to allocate an appropriately sized contiguous virtual address range, e.g. due to virtual address space fragmentation or platform limits. In such cases, `vkMapMemory` **must** return `VK_ERROR_MEMORY_MAP_FAILED`. The application **can** improve the likelihood of success by reducing the size of the mapped range and/or removing unneeded mappings using vkUnmapMemory.

`vkMapMemory` does not check whether the device memory is currently in use before returning the host-accessible pointer. The application **must** guarantee that any previously submitted command that writes to this range has completed before the host reads from or writes to that range, and that any previously submitted command that reads from that range has completed before the host writes to that region (see here for details on fulfilling such a guarantee). If the device memory was allocated without the `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT` set, these guarantees **must** be made for an extended range: the application **must** round down the start of the range to the nearest multiple of VkPhysicalDeviceLimits::`nonCoherentAtomSize`, and round the end of the range up to the nearest multiple of VkPhysicalDeviceLimits::`nonCoherentAtomSize`.

While a range of device memory is host mapped, the application is responsible for synchronizing both device and host access to that memory range.

NOTE	It is important for the application developer to become meticulously familiar with all of the mechanisms described in the chapter on Synchronization and Cache Control as they are crucial to maintaining memory access ordering.

Calling `vkMapMemory` is equivalent to calling vkMapMemory2KHR with an empty `pNext` chain.

Valid Usage

- VUID-vkMapMemory-memory-00678
  `memory` **must** not be currently host mapped
- VUID-vkMapMemory-offset-00679
  `offset` **must** be less than the size of `memory`
- VUID-vkMapMemory-size-00680
  If `size` is not equal to `VK_WHOLE_SIZE`, `size` **must** be greater than `0`

- VUID-vkMapMemory-size-00681
  If `size` is not equal to `VK_WHOLE_SIZE`, `size` **must** be less than or equal to the size of the `memory` minus `offset`
- VUID-vkMapMemory-memory-00682
  `memory` **must** have been created with a memory type that reports `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`
- VUID-vkMapMemory-memory-00683
  `memory` **must** not have been allocated with multiple instances
- VUID-vkMapMemory-flags-09568
  `VK_MEMORY_MAP_PLACED_BIT_EXT` **must** not be set in `flags`

Valid Usage (Implicit)

- VUID-vkMapMemory-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkMapMemory-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle
- VUID-vkMapMemory-flags-parameter
  `flags` **must** be a valid combination of VkMemoryMapFlagBits values
- VUID-vkMapMemory-ppData-parameter
  `ppData` **must** be a valid pointer to a pointer value
- VUID-vkMapMemory-memory-parent
  `memory` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `memory` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_MEMORY_MAP_FAILED`

Bits which **can** be set in vkMapMemory::`flags` and VkMemoryMapInfoKHR::`flags`, specifying additional properties of a memory map, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkMemoryMapFlagBits {
  // Provided by VK_EXT_map_memory_placed
    VK_MEMORY_MAP_PLACED_BIT_EXT = 0x00000001,
} VkMemoryMapFlagBits;
```

- VK_MEMORY_MAP_PLACED_BIT_EXT requests that the implementation place the memory map at the virtual address specified by the application via VkMemoryMapPlacedInfoEXT::pPlacedAddress, replacing any existing mapping at that address. This flag **must** not be used with vkMapMemory as there is no way to specify the placement address.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkMemoryMapFlags;
```

VkMemoryMapFlags is a bitmask type for setting a mask of zero or more VkMemoryMapFlagBits.

Alternatively, to retrieve a host virtual address pointer to a region of a mappable memory object, call:

```
// Provided by VK_KHR_map_memory2
VkResult vkMapMemory2KHR(
    VkDevice                                    device,
    const VkMemoryMapInfoKHR*                   pMemoryMapInfo,
    void**                                      ppData);
```

- device is the logical device that owns the memory.
- pMemoryMapInfo is a pointer to a VkMemoryMapInfoKHR structure describing parameters of the map.
- ppData is a pointer to a void * variable in which is returned a host-accessible pointer to the beginning of the mapped range. This pointer minus VkMemoryMapInfoKHR::offset **must** be aligned to at least VkPhysicalDeviceLimits::minMemoryMapAlignment.

This function behaves identically to vkMapMemory except that it gets its parameters via an extensible structure pointer rather than directly as function arguments.

Valid Usage (Implicit)

- VUID-vkMapMemory2KHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkMapMemory2KHR-pMemoryMapInfo-parameter
  pMemoryMapInfo **must** be a valid pointer to a valid VkMemoryMapInfoKHR structure
- VUID-vkMapMemory2KHR-ppData-parameter
  ppData **must** be a valid pointer to a pointer value

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_MEMORY_MAP_FAILED

The VkMemoryMapInfoKHR structure is defined as:

```
// Provided by VK_KHR_map_memory2
typedef struct VkMemoryMapInfoKHR {
    VkStructureType     sType;
    const void*         pNext;
    VkMemoryMapFlags    flags;
    VkDeviceMemory      memory;
    VkDeviceSize        offset;
    VkDeviceSize        size;
} VkMemoryMapInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkMemoryMapFlagBits specifying additional parameters of the memory map operation.
- memory is the VkDeviceMemory object to be mapped.
- offset is a zero-based byte offset from the beginning of the memory object.
- size is the size of the memory range to map, or VK_WHOLE_SIZE to map from offset to the end of the allocation.

Valid Usage

- VUID-VkMemoryMapInfoKHR-memory-07958
  memory **must** not be currently host mapped
- VUID-VkMemoryMapInfoKHR-offset-07959
  offset **must** be less than the size of memory
- VUID-VkMemoryMapInfoKHR-size-07960
  If size is not equal to VK_WHOLE_SIZE, size **must** be greater than 0
- VUID-VkMemoryMapInfoKHR-size-07961
  If size is not equal to VK_WHOLE_SIZE, size **must** be less than or equal to the size of the memory minus offset

- VUID-VkMemoryMapInfoKHR-memory-07962
  `memory` **must** have been created with a memory type that reports `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`
- VUID-VkMemoryMapInfoKHR-memory-07963
  `memory` **must** not have been allocated with multiple instances
- VUID-VkMemoryMapInfoKHR-flags-09569
  If `VK_MEMORY_MAP_PLACED_BIT_EXT` is set in `flags`, the `memoryMapPlaced` feature **must** be enabled
- VUID-VkMemoryMapInfoKHR-flags-09570
  If `VK_MEMORY_MAP_PLACED_BIT_EXT` is set in `flags`, the `pNext` chain **must** include a VkMemoryMapPlacedInfoEXT structure and `VkMemoryMapPlacedInfoEXT`::`pPlacedAddress` **must** not be `NULL`
- VUID-VkMemoryMapInfoKHR-flags-09571
  If `VK_MEMORY_MAP_PLACED_BIT_EXT` is set in `flags` and the `memoryMapRangePlaced` feature is not enabled, `offset` **must** be zero
- VUID-VkMemoryMapInfoKHR-flags-09572
  If `VK_MEMORY_MAP_PLACED_BIT_EXT` is set in `flags` and the `memoryMapRangePlaced` feature is not enabled, `size` **must** be `VK_WHOLE_SIZE` or `VkMemoryAllocateInfo`::`allocationSize`
- VUID-VkMemoryMapInfoKHR-flags-09573
  If `VK_MEMORY_MAP_PLACED_BIT_EXT` is set in `flags` and the `memoryMapRangePlaced` feature is enabled, `offset` **must** be aligned to an integer multiple of `VkPhysicalDeviceMapMemoryPlacedPropertiesEXT`::`minPlacedMemoryMapAlignment`
- VUID-VkMemoryMapInfoKHR-flags-09574
  If `VK_MEMORY_MAP_PLACED_BIT_EXT` is set in `flags` and `size` is not `VK_WHOLE_SIZE`, `size` **must** be aligned to an integer multiple of `VkPhysicalDeviceMapMemoryPlacedPropertiesEXT`::`minPlacedMemoryMapAlignment`
- VUID-VkMemoryMapInfoKHR-flags-09651
  If `VK_MEMORY_MAP_PLACED_BIT_EXT` is set in `flags` and `size` is `VK_WHOLE_SIZE`, `VkMemoryAllocateInfo`::`allocationSize` **must** be aligned to an integer multiple of `VkPhysicalDeviceMapMemoryPlacedPropertiesEXT`::`minPlacedMemoryMapAlignment`
- VUID-VkMemoryMapInfoKHR-flags-09575
  If `VK_MEMORY_MAP_PLACED_BIT_EXT` is set in `flags`, the memory object **must** not have been imported from a handle type of `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT` or `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT`

Valid Usage (Implicit)

- VUID-VkMemoryMapInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_MAP_INFO_KHR`
- VUID-VkMemoryMapInfoKHR-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkMemoryMapPlacedInfoEXT
- VUID-VkMemoryMapInfoKHR-sType-unique

The sType value of each struct in the pNext chain **must** be unique

- VUID-VkMemoryMapInfoKHR-flags-parameter
  flags **must** be a valid combination of VkMemoryMapFlagBits values
- VUID-VkMemoryMapInfoKHR-memory-parameter
  memory **must** be a valid VkDeviceMemory handle

Host Synchronization

- Host access to memory **must** be externally synchronized

If VK_MEMORY_MAP_PLACED_BIT_EXT is set in VkMemoryMapInfoKHR::flags and the pNext chain of VkMemoryMapInfoKHR includes a VkMemoryMapPlacedInfoEXT structure, then that structure specifies the placement address of the memory map. The implementation will place the memory map at the specified address, replacing any existing maps in the specified memory range. Replacing memory maps in this way does not implicitly unmap Vulkan memory objects. Instead, the application **must** ensure no other Vulkan memory objects are mapped anywhere in the specified virtual address range. If successful, ppData will be set to the same value as VkMemoryMapPlacedInfoEXT::pPlacedAddress and vkMapMemory2KHR will return VK_SUCCESS. If it cannot place the map at the requested address for any reason, the memory object is left unmapped and vkMapMemory2KHR will return VK_ERROR_MEMORY_MAP_FAILED.

The VkMemoryMapPlacedInfoEXT structure is defined as:

```
// Provided by VK_EXT_map_memory_placed
typedef struct VkMemoryMapPlacedInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    void*              pPlacedAddress;
} VkMemoryMapPlacedInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pPlacedAddress is the virtual address at which to place the address. If VkMemoryMapInfoKHR::flags does not contain VK_MEMORY_MAP_PLACED_BIT_EXT, this value is ignored.

Valid Usage

- VUID-VkMemoryMapPlacedInfoEXT-flags-09576
  If VkMemoryMapInfoKHR::flags contains VK_MEMORY_MAP_PLACED_BIT_EXT, pPlacedAddress **must** not be NULL
- VUID-VkMemoryMapPlacedInfoEXT-pPlacedAddress-09577
  pPlacedAddress **must** be aligned to an integer multiple of VkPhysicalDeviceMapMemoryPlacedPropertiesEXT::minPlacedMemoryMapAlignment

- VUID-VkMemoryMapPlacedInfoEXT-pPlacedAddress-09578
  The address range specified by pPlacedAddress and VkMemoryMapInfoKHR::size **must** not overlap any existing Vulkan memory object mapping

Valid Usage (Implicit)

- VUID-VkMemoryMapPlacedInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_MEMORY_MAP_PLACED_INFO_EXT

Two commands are provided to enable applications to work with non-coherent memory allocations: vkFlushMappedMemoryRanges and vkInvalidateMappedMemoryRanges.

NOTE

If the memory object was created with the VK_MEMORY_PROPERTY_HOST_COHERENT_BIT set, vkFlushMappedMemoryRanges and vkInvalidateMappedMemoryRanges are unnecessary and **may** have a performance cost. However, availability and visibility operations still need to be managed on the device. See the description of host access types for more information.

NOTE

While memory objects imported from a handle type of VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT or VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT are inherently mapped to host address space, they are not considered to be host mapped device memory unless they are explicitly host mapped using vkMapMemory. That means flushing or invalidating host caches with respect to host accesses performed on such memory through the original host pointer specified at import time is the responsibility of the application and **must** be performed with appropriate synchronization primitives provided by the platform which are outside the scope of Vulkan. vkFlushMappedMemoryRanges and vkInvalidateMappedMemoryRanges, however, **can** still be used on such memory objects to synchronize host accesses performed through the host pointer of the host mapped device memory range returned by vkMapMemory.

After a successful call to vkMapMemory or vkMapMemory2KHR the memory object memory is considered to be currently *host mapped*.

To flush ranges of non-coherent memory from the host caches, call:

```
// Provided by VK_VERSION_1_0
VkResult vkFlushMappedMemoryRanges(
    VkDevice                                    device,
    uint32_t                                    memoryRangeCount,
    const VkMappedMemoryRange*                  pMemoryRanges);
```

- device is the logical device that owns the memory ranges.
- memoryRangeCount is the length of the pMemoryRanges array.

- `pMemoryRanges` is a pointer to an array of VkMappedMemoryRange structures describing the memory ranges to flush.

`vkFlushMappedMemoryRanges` guarantees that host writes to the memory ranges described by `pMemoryRanges` are made available to the host memory domain, such that they **can** be made available to the device memory domain via memory domain operations using the `VK_ACCESS_HOST_WRITE_BIT` access type.

The first synchronization scope includes all host operations that happened-before it, as defined by the host memory model.

NOTE

Note

Some systems allow writes that do not directly integrate with the host memory model; these have to be synchronized by the application manually. One example of this is non-temporal store instructions on x86; to ensure these happen-before submission, applications should call `_mm_sfence()`.

The second synchronization scope is empty.

The first access scope includes host writes to the specified memory ranges.

NOTE

Note

When a host write to a memory location is made available in this way, each whole aligned set of `nonCoherentAtomSize` bytes that the memory location exists in will also be made available as if they were written by the host. For example, with a `nonCoherentAtomSize` of 128, if an application writes to the first byte of a memory object via a host mapping, the first 128 bytes of the memory object will be made available by this command. While the value of the following 127 bytes will be unchanged, this does count as an access for the purpose of synchronization, so care must be taken to avoid data races.

The second access scope is empty.

Unmapping non-coherent memory does not implicitly flush the host mapped memory, and host writes that have not been flushed **may** not ever be visible to the device. However, implementations **must** ensure that writes that have not been flushed do not become visible to any other memory.

NOTE

The above guarantee avoids a potential memory corruption in scenarios where host writes to a mapped memory object have not been flushed before the memory is unmapped (or freed), and the virtual address range is subsequently reused for a different mapping (or memory allocation).

Valid Usage (Implicit)

- VUID-vkFlushMappedMemoryRanges-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkFlushMappedMemoryRanges-pMemoryRanges-parameter

pMemoryRanges **must** be a valid pointer to an array of memoryRangeCount valid VkMappedMemoryRange structures

- VUID-vkFlushMappedMemoryRanges-memoryRangeCount-arraylength
  memoryRangeCount **must** be greater than 0

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

To invalidate ranges of non-coherent memory from the host caches, call:

```
// Provided by VK_VERSION_1_0
VkResult vkInvalidateMappedMemoryRanges(
    VkDevice                                    device,
    uint32_t                                    memoryRangeCount,
    const VkMappedMemoryRange*                  pMemoryRanges);
```

- device is the logical device that owns the memory ranges.
- memoryRangeCount is the length of the pMemoryRanges array.
- pMemoryRanges is a pointer to an array of VkMappedMemoryRange structures describing the memory ranges to invalidate.

vkInvalidateMappedMemoryRanges guarantees that device writes to the memory ranges described by pMemoryRanges, which have been made available to the host memory domain using the VK_ACCESS_HOST_WRITE_BIT and VK_ACCESS_HOST_READ_BIT access types, are made visible to the host. If a range of non-coherent memory is written by the host and then invalidated without first being flushed, its contents are undefined.

The first synchronization scope includes all host operations that happened-before it, as defined by the host memory model.

NOTE

Note

This function does not synchronize with device operations directly - other host synchronization operations that depend on device operations such as vkWaitForFences must be executed beforehand. So for any non-coherent device write to be made visible to the host, there has to be a dependency chain along the following lines:

1. Device write

2. Device memory barrier including host reads in its second scope
3. Signal on the device (e.g. a fence signal operation)
4. Wait on the host (e.g. vkWaitForFences)
5. vkInvalidateMappedMemoryRanges

The second synchronization scope includes all host operations that happen-after it, as defined by the host memory model.

The first access scope is empty.

The second access scope includes host reads to the specified memory ranges.

NOTE

Note

When a device write to a memory location is made visible to the host in this way, each whole aligned set of `nonCoherentAtomSize` bytes that the memory location exists in will also be made visible as if they were written by the device. For example, with a `nonCoherentAtomSize` of 128, if an application writes to the first byte of a memory object on the device, the first 128 bytes of the memory object will be made visible by this command. While the value of the following 127 bytes will be unchanged, this does count as an access for the purpose of synchronization, so care must be taken to avoid data races.

NOTE Mapping non-coherent memory does not implicitly invalidate that memory.

Valid Usage (Implicit)

- VUID-vkInvalidateMappedMemoryRanges-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkInvalidateMappedMemoryRanges-pMemoryRanges-parameter
  `pMemoryRanges` **must** be a valid pointer to an array of `memoryRangeCount` valid VkMappedMemoryRange structures
- VUID-vkInvalidateMappedMemoryRanges-memoryRangeCount-arraylength
  `memoryRangeCount` **must** be greater than `0`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The VkMappedMemoryRange structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkMappedMemoryRange {
    VkStructureType    sType;
    const void*        pNext;
    VkDeviceMemory     memory;
    VkDeviceSize       offset;
    VkDeviceSize       size;
} VkMappedMemoryRange;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- memory is the memory object to which this range belongs.
- offset is the zero-based byte offset from the beginning of the memory object.
- size is either the size of range, or VK_WHOLE_SIZE to affect the range from offset to the end of the current mapping of the allocation.

Valid Usage

- VUID-VkMappedMemoryRange-memory-00684
  memory **must** be currently host mapped
- VUID-VkMappedMemoryRange-size-00685
  If size is not equal to VK_WHOLE_SIZE, offset and size **must** specify a range contained within the currently mapped range of memory
- VUID-VkMappedMemoryRange-size-00686
  If size is equal to VK_WHOLE_SIZE, offset **must** be within the currently mapped range of memory
- VUID-VkMappedMemoryRange-offset-00687
  offset **must** be a multiple of VkPhysicalDeviceLimits::nonCoherentAtomSize
- VUID-VkMappedMemoryRange-size-01389
  If size is equal to VK_WHOLE_SIZE, the end of the current mapping of memory **must** either be a multiple of VkPhysicalDeviceLimits::nonCoherentAtomSize bytes from the beginning of the memory object, or be equal to the end of the memory object
- VUID-VkMappedMemoryRange-size-01390
  If size is not equal to VK_WHOLE_SIZE, size **must** either be a multiple of VkPhysicalDeviceLimits::nonCoherentAtomSize, or offset plus size **must** equal the size of memory

Valid Usage (Implicit)

- VUID-VkMappedMemoryRange-sType-sType

`sType` **must** be `VK_STRUCTURE_TYPE_MAPPED_MEMORY_RANGE`

- VUID-VkMappedMemoryRange-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkMappedMemoryRange-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle

To unmap a memory object once host access to it is no longer needed by the application, call:

```
// Provided by VK_VERSION_1_0
void vkUnmapMemory(
    VkDevice                                    device,
    VkDeviceMemory                              memory);
```

- `device` is the logical device that owns the memory.
- `memory` is the memory object to be unmapped.

Calling `vkUnmapMemory` is equivalent to calling vkUnmapMemory2KHR with an empty `pNext` chain and `flags` set to zero.

Valid Usage

- VUID-vkUnmapMemory-memory-00689
  `memory` **must** be currently host mapped

Valid Usage (Implicit)

- VUID-vkUnmapMemory-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkUnmapMemory-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle
- VUID-vkUnmapMemory-memory-parent
  `memory` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `memory` **must** be externally synchronized

Alternatively, to unmap a memory object once host access to it is no longer needed by the application, call:

```
// Provided by VK_KHR_map_memory2
VkResult vkUnmapMemory2KHR(
    VkDevice                                    device,
    const VkMemoryUnmapInfoKHR*                 pMemoryUnmapInfo);
```

- `device` is the logical device that owns the memory.
- `pMemoryUnmapInfo` is a pointer to a VkMemoryUnmapInfoKHR structure describing parameters of the unmap.

This function behaves identically to vkUnmapMemory except that it gets its parameters via an extensible structure pointer rather than directly as function arguments.

Valid Usage (Implicit)

- VUID-vkUnmapMemory2KHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkUnmapMemory2KHR-pMemoryUnmapInfo-parameter
  `pMemoryUnmapInfo` **must** be a valid pointer to a valid VkMemoryUnmapInfoKHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_MEMORY_MAP_FAILED`

The `VkMemoryUnmapInfoKHR` structure is defined as:

```
// Provided by VK_KHR_map_memory2
typedef struct VkMemoryUnmapInfoKHR {
    VkStructureType          sType;
    const void*              pNext;
    VkMemoryUnmapFlagsKHR    flags;
    VkDeviceMemory           memory;
} VkMemoryUnmapInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkMemoryUnmapFlagBitsKHR specifying additional parameters of the memory map operation.
- `memory` is the VkDeviceMemory object to be unmapped.

Valid Usage

- VUID-VkMemoryUnmapInfoKHR-memory-07964
  `memory` **must** be currently host mapped
- VUID-VkMemoryUnmapInfoKHR-flags-09579
  If `VK_MEMORY_UNMAP_RESERVE_BIT_EXT` is set in `flags`, the `memoryUnmapReserve` **must** be enabled
- VUID-VkMemoryUnmapInfoKHR-flags-09580
  If `VK_MEMORY_UNMAP_RESERVE_BIT_EXT` is set in `flags`, the memory object **must** not have been imported from a handle type of `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT` or `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT`

Valid Usage (Implicit)

- VUID-VkMemoryUnmapInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_UNMAP_INFO_KHR`
- VUID-VkMemoryUnmapInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkMemoryUnmapInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkMemoryUnmapFlagBitsKHR values
- VUID-VkMemoryUnmapInfoKHR-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle

Host Synchronization

- Host access to `memory` **must** be externally synchronized

Bits which **can** be set in VkMemoryUnmapInfoKHR::`flags`, specifying additional properties of a memory unmap, are:

```
// Provided by VK_KHR_map_memory2
typedef enum VkMemoryUnmapFlagBitsKHR {
  // Provided by VK_EXT_map_memory_placed
    VK_MEMORY_UNMAP_RESERVE_BIT_EXT = 0x00000001,
} VkMemoryUnmapFlagBitsKHR;
```

- `VK_MEMORY_UNMAP_RESERVE_BIT_EXT` requests that virtual address range currently occupied by the memory map remain reserved after the vkUnmapMemory2KHR call completes. Future system memory map operations or calls to vkMapMemory or vkMapMemory2KHR will not return addresses in that range unless the range has since been unreserved by the application or the mapping is explicitly placed in that range by calling vkMapMemory2KHR with `VK_MEMORY_MAP_PLACED_BIT_EXT`, or doing the system memory map equivalent. When `VK_MEMORY_UNMAP_RESERVE_BIT_EXT` is set, the memory unmap operation **may** fail, in which case

the memory object will remain host mapped and vkUnmapMemory2KHR will return VK_ERROR_MEMORY_MAP_FAILED.

```
// Provided by VK_KHR_map_memory2
typedef VkFlags VkMemoryUnmapFlagsKHR;
```

VkMemoryUnmapFlagsKHR is a bitmask type for setting a mask of zero or more VkMemoryUnmapFlagBitsKHR.

11.2.12. Lazily Allocated Memory

If the memory object is allocated from a heap with the VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT bit set, that object's backing memory **may** be provided by the implementation lazily. The actual committed size of the memory **may** initially be as small as zero (or as large as the requested size), and monotonically increases as additional memory is needed.

A memory type with this flag set is only allowed to be bound to a VkImage whose usage flags include VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT.

NOTE	Using lazily allocated memory objects for framebuffer attachments that are not needed once a render pass instance has completed **may** allow some implementations to never allocate memory for such attachments.

To determine the amount of lazily-allocated memory that is currently committed for a memory object, call:

```
// Provided by VK_VERSION_1_0
void vkGetDeviceMemoryCommitment(
    VkDevice                                    device,
    VkDeviceMemory                              memory,
    VkDeviceSize*                               pCommittedMemoryInBytes);
```

- device is the logical device that owns the memory.
- memory is the memory object being queried.
- pCommittedMemoryInBytes is a pointer to a VkDeviceSize value in which the number of bytes currently committed is returned, on success.

The implementation **may** update the commitment at any time, and the value returned by this query **may** be out of date.

The implementation guarantees to allocate any committed memory from the heapIndex indicated by the memory type that the memory object was created with.

Valid Usage

- VUID-vkGetDeviceMemoryCommitment-memory-00690

`memory` **must** have been created with a memory type that reports `VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT`

Valid Usage (Implicit)

- VUID-vkGetDeviceMemoryCommitment-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceMemoryCommitment-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle
- VUID-vkGetDeviceMemoryCommitment-pCommittedMemoryInBytes-parameter
  `pCommittedMemoryInBytes` **must** be a valid pointer to a VkDeviceSize value
- VUID-vkGetDeviceMemoryCommitment-memory-parent
  `memory` **must** have been created, allocated, or retrieved from `device`

11.2.13. Peer Memory Features

Peer memory is memory that is allocated for a given physical device and then bound to a resource and accessed by a different physical device, in a logical device that represents multiple physical devices. Some ways of reading and writing peer memory **may** not be supported by a device.

To determine how peer memory **can** be accessed, call:

```
// Provided by VK_KHR_device_group
void vkGetDeviceGroupPeerMemoryFeaturesKHR(
    VkDevice                                    device,
    uint32_t                                    heapIndex,
    uint32_t                                    localDeviceIndex,
    uint32_t                                    remoteDeviceIndex,
    VkPeerMemoryFeatureFlags*                   pPeerMemoryFeatures);
```

- `device` is the logical device that owns the memory.
- `heapIndex` is the index of the memory heap from which the memory is allocated.
- `localDeviceIndex` is the device index of the physical device that performs the memory access.
- `remoteDeviceIndex` is the device index of the physical device that the memory is allocated for.
- `pPeerMemoryFeatures` is a pointer to a VkPeerMemoryFeatureFlags bitmask indicating which types of memory accesses are supported for the combination of heap, local, and remote devices.

Valid Usage

- VUID-vkGetDeviceGroupPeerMemoryFeatures-heapIndex-00691
  `heapIndex` **must** be less than `memoryHeapCount`
- VUID-vkGetDeviceGroupPeerMemoryFeatures-localDeviceIndex-00692

`localDeviceIndex` **must** be a valid device index

- VUID-vkGetDeviceGroupPeerMemoryFeatures-remoteDeviceIndex-00693
  `remoteDeviceIndex` **must** be a valid device index
- VUID-vkGetDeviceGroupPeerMemoryFeatures-localDeviceIndex-00694
  `localDeviceIndex` **must** not equal `remoteDeviceIndex`

Valid Usage (Implicit)

- VUID-vkGetDeviceGroupPeerMemoryFeatures-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceGroupPeerMemoryFeatures-pPeerMemoryFeatures-parameter
  `pPeerMemoryFeatures` **must** be a valid pointer to a VkPeerMemoryFeatureFlags value

Bits which **may** be set in vkGetDeviceGroupPeerMemoryFeaturesKHR::`pPeerMemoryFeatures`, indicating supported peer memory features, are:

```
typedef enum VkPeerMemoryFeatureFlagBits {
    VK_PEER_MEMORY_FEATURE_COPY_SRC_BIT = 0x00000001,
    VK_PEER_MEMORY_FEATURE_COPY_DST_BIT = 0x00000002,
    VK_PEER_MEMORY_FEATURE_GENERIC_SRC_BIT = 0x00000004,
    VK_PEER_MEMORY_FEATURE_GENERIC_DST_BIT = 0x00000008,
  // Provided by VK_KHR_device_group
    VK_PEER_MEMORY_FEATURE_COPY_SRC_BIT_KHR = VK_PEER_MEMORY_FEATURE_COPY_SRC_BIT,
  // Provided by VK_KHR_device_group
    VK_PEER_MEMORY_FEATURE_COPY_DST_BIT_KHR = VK_PEER_MEMORY_FEATURE_COPY_DST_BIT,
  // Provided by VK_KHR_device_group
    VK_PEER_MEMORY_FEATURE_GENERIC_SRC_BIT_KHR =
VK_PEER_MEMORY_FEATURE_GENERIC_SRC_BIT,
  // Provided by VK_KHR_device_group
    VK_PEER_MEMORY_FEATURE_GENERIC_DST_BIT_KHR =
VK_PEER_MEMORY_FEATURE_GENERIC_DST_BIT,
} VkPeerMemoryFeatureFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkPeerMemoryFeatureFlagBits VkPeerMemoryFeatureFlagBitsKHR;
```

- `VK_PEER_MEMORY_FEATURE_COPY_SRC_BIT` specifies that the memory **can** be accessed as the source of any `vkCmdCopy*` command.
- `VK_PEER_MEMORY_FEATURE_COPY_DST_BIT` specifies that the memory **can** be accessed as the destination of any `vkCmdCopy*` command.
- `VK_PEER_MEMORY_FEATURE_GENERIC_SRC_BIT` specifies that the memory **can** be read as any memory

access type.

- VK_PEER_MEMORY_FEATURE_GENERIC_DST_BIT specifies that the memory **can** be written as any memory access type. Shader atomics are considered to be writes.

NOTE	The peer memory features of a memory heap also apply to any accesses that **may** be performed during image layout transitions.

VK_PEER_MEMORY_FEATURE_COPY_DST_BIT **must** be supported for all host local heaps and for at least one device-local memory heap.

If a device does not support a peer memory feature, it is still valid to use a resource that includes both local and peer memory bindings with the corresponding access type as long as only the local bindings are actually accessed. For example, an application doing split-frame rendering would use framebuffer attachments that include both local and peer memory bindings, but would scissor the rendering to only update local memory.

```
typedef VkFlags VkPeerMemoryFeatureFlags;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkPeerMemoryFeatureFlags VkPeerMemoryFeatureFlagsKHR;
```

VkPeerMemoryFeatureFlags is a bitmask type for setting a mask of zero or more VkPeerMemoryFeatureFlagBits.

11.2.14. Opaque Capture Address Query

To query a 64-bit opaque capture address value from a memory object, call:

```
// Provided by VK_KHR_buffer_device_address
uint64_t vkGetDeviceMemoryOpaqueCaptureAddressKHR(
    VkDevice                                    device,
    const VkDeviceMemoryOpaqueCaptureAddressInfo* pInfo);
```

- device is the logical device that the memory object was allocated on.
- pInfo is a pointer to a VkDeviceMemoryOpaqueCaptureAddressInfo structure specifying the memory object to retrieve an address for.

The 64-bit return value is an opaque address representing the start of pInfo->memory.

If the memory object was allocated with a non-zero value of VkMemoryOpaqueCaptureAddressAllocateInfo::opaqueCaptureAddress, the return value **must** be the same address.

NOTE	The expected usage for these opaque addresses is only for trace capture/replay tools to store these addresses in a trace and subsequently specify them during replay.

Valid Usage

- VUID-vkGetDeviceMemoryOpaqueCaptureAddress-None-03334
  The `bufferDeviceAddress` feature **must** be enabled
- VUID-vkGetDeviceMemoryOpaqueCaptureAddress-device-03335
  If `device` was created with multiple physical devices, then the `bufferDeviceAddressMultiDevice` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkGetDeviceMemoryOpaqueCaptureAddress-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceMemoryOpaqueCaptureAddress-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkDeviceMemoryOpaqueCaptureAddressInfo structure

The `VkDeviceMemoryOpaqueCaptureAddressInfo` structure is defined as:

```
typedef struct VkDeviceMemoryOpaqueCaptureAddressInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkDeviceMemory     memory;
} VkDeviceMemoryOpaqueCaptureAddressInfo;
```

or the equivalent

```
// Provided by VK_KHR_buffer_device_address
typedef VkDeviceMemoryOpaqueCaptureAddressInfo
VkDeviceMemoryOpaqueCaptureAddressInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memory` specifies the memory whose address is being queried.

Valid Usage

- VUID-VkDeviceMemoryOpaqueCaptureAddressInfo-memory-03336
  `memory` **must** have been allocated with `VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT`

Valid Usage (Implicit)

- VUID-VkDeviceMemoryOpaqueCaptureAddressInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_MEMORY_OPAQUE_CAPTURE_ADDRESS_INFO`
- VUID-VkDeviceMemoryOpaqueCaptureAddressInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDeviceMemoryOpaqueCaptureAddressInfo-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle

Chapter 12. Resource Creation

Vulkan supports two primary resource types: *buffers* and *images*. Resources are views of memory with associated formatting and dimensionality. Buffers provide access to raw arrays of bytes, whereas images **can** be multidimensional and **may** have associated metadata.

Other resource types, such as acceleration structures and micromaps use buffers as the backing store for opaque data structures.

12.1. Buffers

Buffers represent linear arrays of data which are used for various purposes by binding them to a graphics or compute pipeline via descriptor sets or certain commands, or by directly specifying them as parameters to certain commands.

Buffers are represented by `VkBuffer` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkBuffer)
```

To create buffers, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateBuffer(
    VkDevice                                    device,
    const VkBufferCreateInfo*                   pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkBuffer*                                   pBuffer);
```

- `device` is the logical device that creates the buffer object.
- `pCreateInfo` is a pointer to a VkBufferCreateInfo structure containing parameters affecting creation of the buffer.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pBuffer` is a pointer to a VkBuffer handle in which the resulting buffer object is returned.

Valid Usage

- VUID-vkCreateBuffer-device-09664
  `device` **must** support at least one queue family with one of the `VK_QUEUE_VIDEO_ENCODE_BIT_KHR`, `VK_QUEUE_VIDEO_DECODE_BIT_KHR`, `VK_QUEUE_SPARSE_BINDING_BIT`, `VK_QUEUE_TRANSFER_BIT`, `VK_QUEUE_COMPUTE_BIT`, or `VK_QUEUE_GRAPHICS_BIT` capabilities
- VUID-vkCreateBuffer-flags-00911
  If the `flags` member of `pCreateInfo` includes `VK_BUFFER_CREATE_SPARSE_BINDING_BIT`,

creating this VkBuffer **must** not cause the total required sparse memory for all currently valid sparse resources on the device to exceed VkPhysicalDeviceLimits ::sparseAddressSpaceSize

Valid Usage (Implicit)

- VUID-vkCreateBuffer-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateBuffer-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkBufferCreateInfo structure
- VUID-vkCreateBuffer-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateBuffer-pBuffer-parameter
  pBuffer **must** be a valid pointer to a VkBuffer handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR

The VkBufferCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkBufferCreateInfo {
    VkStructureType        sType;
    const void*            pNext;
    VkBufferCreateFlags    flags;
    VkDeviceSize           size;
    VkBufferUsageFlags     usage;
    VkSharingMode          sharingMode;
    uint32_t               queueFamilyIndexCount;
    const uint32_t*        pQueueFamilyIndices;
} VkBufferCreateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- `flags` is a bitmask of VkBufferCreateFlagBits specifying additional parameters of the buffer.
- `size` is the size in bytes of the buffer to be created.
- `usage` is a bitmask of VkBufferUsageFlagBits specifying allowed usages of the buffer.
- `sharingMode` is a VkSharingMode value specifying the sharing mode of the buffer when it will be accessed by multiple queue families.
- `queueFamilyIndexCount` is the number of entries in the `pQueueFamilyIndices` array.
- `pQueueFamilyIndices` is a pointer to an array of queue families that will access this buffer. It is ignored if `sharingMode` is not `VK_SHARING_MODE_CONCURRENT`.

If the `pNext` chain includes a VkBufferUsageFlags2CreateInfoKHR structure, VkBufferUsageFlags2CreateInfoKHR::`usage` from that structure is used instead of `usage` from this structure.

Valid Usage

- VUID-VkBufferCreateInfo-None-09499
  If the `pNext` chain does not include a VkBufferUsageFlags2CreateInfoKHR structure, `usage` **must** be a valid combination of VkBufferUsageFlagBits values
- VUID-VkBufferCreateInfo-None-09500
  If the `pNext` chain does not include a VkBufferUsageFlags2CreateInfoKHR structure, `usage` **must** not be 0
- VUID-VkBufferCreateInfo-size-00912
  `size` **must** be greater than `0`
- VUID-VkBufferCreateInfo-sharingMode-00913
  If `sharingMode` is `VK_SHARING_MODE_CONCURRENT`, `pQueueFamilyIndices` **must** be a valid pointer to an array of `queueFamilyIndexCount` `uint32_t` values
- VUID-VkBufferCreateInfo-sharingMode-00914
  If `sharingMode` is `VK_SHARING_MODE_CONCURRENT`, `queueFamilyIndexCount` **must** be greater than `1`
- VUID-VkBufferCreateInfo-sharingMode-01419
  If `sharingMode` is `VK_SHARING_MODE_CONCURRENT`, each element of `pQueueFamilyIndices` **must** be unique and **must** be less than `pQueueFamilyPropertyCount` returned by either vkGetPhysicalDeviceQueueFamilyProperties2KHR or vkGetPhysicalDeviceQueueFamilyProperties for the `physicalDevice` that was used to create `device`
- VUID-VkBufferCreateInfo-flags-00915
  If the `sparseBinding` feature is not enabled, `flags` **must** not contain `VK_BUFFER_CREATE_SPARSE_BINDING_BIT`
- VUID-VkBufferCreateInfo-flags-00916
  If the `sparseResidencyBuffer` feature is not enabled, `flags` **must** not contain `VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkBufferCreateInfo-flags-00917
  If the `sparseResidencyAliased` feature is not enabled, `flags` **must** not contain

VK_BUFFER_CREATE_SPARSE_ALIASED_BIT

- VUID-VkBufferCreateInfo-flags-00918
  If flags contains VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT or VK_BUFFER_CREATE_SPARSE_ALIASED_BIT, it **must** also contain VK_BUFFER_CREATE_SPARSE_BINDING_BIT
- VUID-VkBufferCreateInfo-pNext-00920
  If the pNext chain includes a VkExternalMemoryBufferCreateInfo structure, its handleTypes member **must** only contain bits that are also in VkExternalBufferProperties::externalMemoryProperties.compatibleHandleTypes, as returned by vkGetPhysicalDeviceExternalBufferPropertiesKHR with pExternalBufferInfo->handleType equal to any one of the handle types specified in VkExternalMemoryBufferCreateInfo::handleTypes
- VUID-VkBufferCreateInfo-opaqueCaptureAddress-03337
  If VkBufferOpaqueCaptureAddressCreateInfo::opaqueCaptureAddress is not zero, flags **must** include VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT
- VUID-VkBufferCreateInfo-flags-03338
  If flags includes VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT, the bufferDeviceAddressCaptureReplay feature **must** be enabled
- VUID-VkBufferCreateInfo-usage-04813
  If usage includes VK_BUFFER_USAGE_VIDEO_DECODE_SRC_BIT_KHR or VK_BUFFER_USAGE_VIDEO_DECODE_DST_BIT_KHR, and flags does not include VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then the pNext chain **must** include a VkVideoProfileListInfoKHR structure with profileCount greater than 0 and pProfiles including at least one VkVideoProfileInfoKHR structure with a videoCodecOperation member specifying a decode operation
- VUID-VkBufferCreateInfo-usage-04814
  If usage includes VK_BUFFER_USAGE_VIDEO_ENCODE_SRC_BIT_KHR or VK_BUFFER_USAGE_VIDEO_ENCODE_DST_BIT_KHR, and flags does not include VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then the pNext chain **must** include a VkVideoProfileListInfoKHR structure with profileCount greater than 0 and pProfiles including at least one VkVideoProfileInfoKHR structure with a videoCodecOperation member specifying an encode operation
- VUID-VkBufferCreateInfo-flags-08325
  If flags includes VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then videoMaintenance1 **must** be enabled
- VUID-VkBufferCreateInfo-pNext-10249
  If the pNext chain includes a VkVideoProfileListInfoKHR structure and for any element of its pProfiles member videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the videoEncodeAV1 feature **must** be enabled
- VUID-VkBufferCreateInfo-size-06409
  size **must** be less than or equal to VkPhysicalDeviceMaintenance4Properties::maxBufferSize
- VUID-VkBufferCreateInfo-flags-09641

If `flags` includes `VK_BUFFER_CREATE_PROTECTED_BIT`, then `usage` **must** not contain any of the following bits

- `VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_BUFFER_BIT_EXT`
- `VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT`
- `VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR`
- `VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR`
- `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR`
- `VK_BUFFER_USAGE_MICROMAP_BUILD_INPUT_READ_ONLY_BIT_EXT`
- `VK_BUFFER_USAGE_MICROMAP_STORAGE_BIT_EXT`

Valid Usage (Implicit)

- VUID-VkBufferCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO`
- VUID-VkBufferCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkBufferOpaqueCaptureAddressCreateInfo, VkBufferUsageFlags2CreateInfoKHR, VkExternalMemoryBufferCreateInfo, or VkVideoProfileListInfoKHR
- VUID-VkBufferCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkBufferCreateInfo-flags-parameter
  `flags` **must** be a valid combination of VkBufferCreateFlagBits values
- VUID-VkBufferCreateInfo-sharingMode-parameter
  `sharingMode` **must** be a valid VkSharingMode value

The `VkBufferUsageFlags2CreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance5
typedef struct VkBufferUsageFlags2CreateInfoKHR {
    VkStructureType           sType;
    const void*               pNext;
    VkBufferUsageFlags2KHR    usage;
} VkBufferUsageFlags2CreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `usage` is a bitmask of VkBufferUsageFlagBits2KHR specifying allowed usages of the buffer.

If this structure is included in the `pNext` chain of a buffer creation structure, `usage` is used instead of the corresponding `usage` value passed in that creation structure, allowing additional usage flags to

be specified. If this structure is included in the pNext chain of a buffer query structure, the usage flags of the buffer are returned in usage of this structure, and the usage flags representable in usage of the buffer query structure are also returned in that field.

Valid Usage (Implicit)

- VUID-VkBufferUsageFlags2CreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_BUFFER_USAGE_FLAGS_2_CREATE_INFO_KHR
- VUID-VkBufferUsageFlags2CreateInfoKHR-usage-parameter
  usage **must** be a valid combination of VkBufferUsageFlagBits2KHR values
- VUID-VkBufferUsageFlags2CreateInfoKHR-usage-requiredbitmask
  usage **must** not be 0

Bits which **can** be set in VkBufferUsageFlags2CreateInfoKHR::usage, specifying usage behavior of a buffer, are:

```
// Provided by VK_KHR_maintenance5
// Flag bits for VkBufferUsageFlagBits2KHR
typedef VkFlags64 VkBufferUsageFlagBits2KHR;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_TRANSFER_SRC_BIT_KHR =
0x00000001ULL;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_TRANSFER_DST_BIT_KHR =
0x00000002ULL;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_UNIFORM_TEXEL_BUFFER_BIT_KHR
= 0x00000004ULL;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_STORAGE_TEXEL_BUFFER_BIT_KHR
= 0x00000008ULL;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_UNIFORM_BUFFER_BIT_KHR =
0x00000010ULL;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_STORAGE_BUFFER_BIT_KHR =
0x00000020ULL;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_INDEX_BUFFER_BIT_KHR =
0x00000040ULL;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_VERTEX_BUFFER_BIT_KHR =
0x00000080ULL;
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_INDIRECT_BUFFER_BIT_KHR =
0x00000100ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_conditional_rendering
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_CONDITIONAL_RENDERING_BIT_EXT
= 0x00000200ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_ray_tracing_pipeline
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_SHADER_BINDING_TABLE_BIT_KHR
= 0x00000400ULL;
// Provided by VK_KHR_maintenance5 with VK_NV_ray_tracing
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_RAY_TRACING_BIT_NV =
0x00000400ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_transform_feedback
static const VkBufferUsageFlagBits2KHR
```

```
VK_BUFFER_USAGE_2_TRANSFORM_FEEDBACK_BUFFER_BIT_EXT = 0x00000800ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_transform_feedback
static const VkBufferUsageFlagBits2KHR
VK_BUFFER_USAGE_2_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT = 0x00001000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_video_decode_queue
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_VIDEO_DECODE_SRC_BIT_KHR =
0x00002000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_video_decode_queue
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_VIDEO_DECODE_DST_BIT_KHR =
0x00004000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_video_encode_queue
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_VIDEO_ENCODE_DST_BIT_KHR =
0x00008000ULL;
// Provided by VK_KHR_maintenance5 with VK_KHR_video_encode_queue
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_VIDEO_ENCODE_SRC_BIT_KHR =
0x00010000ULL;
// Provided by VK_KHR_maintenance5 with (VK_VERSION_1_2 or
VK_KHR_buffer_device_address) or VK_EXT_buffer_device_address
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_SHADER_DEVICE_ADDRESS_BIT_KHR
= 0x00020000ULL;
// Provided by VK_KHR_acceleration_structure with VK_KHR_maintenance5
static const VkBufferUsageFlagBits2KHR
VK_BUFFER_USAGE_2_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR =
0x00080000ULL;
// Provided by VK_KHR_acceleration_structure with VK_KHR_maintenance5
static const VkBufferUsageFlagBits2KHR
VK_BUFFER_USAGE_2_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR = 0x00100000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_descriptor_buffer
static const VkBufferUsageFlagBits2KHR
VK_BUFFER_USAGE_2_SAMPLER_DESCRIPTOR_BUFFER_BIT_EXT = 0x00200000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_descriptor_buffer
static const VkBufferUsageFlagBits2KHR
VK_BUFFER_USAGE_2_RESOURCE_DESCRIPTOR_BUFFER_BIT_EXT = 0x00400000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_descriptor_buffer
static const VkBufferUsageFlagBits2KHR
VK_BUFFER_USAGE_2_PUSH_DESCRIPTORS_DESCRIPTOR_BUFFER_BIT_EXT = 0x04000000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_opacity_micromap
static const VkBufferUsageFlagBits2KHR
VK_BUFFER_USAGE_2_MICROMAP_BUILD_INPUT_READ_ONLY_BIT_EXT = 0x00800000ULL;
// Provided by VK_KHR_maintenance5 with VK_EXT_opacity_micromap
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_MICROMAP_STORAGE_BIT_EXT =
0x01000000ULL;
// Provided by VK_EXT_device_generated_commands
static const VkBufferUsageFlagBits2KHR VK_BUFFER_USAGE_2_PREPROCESS_BUFFER_BIT_EXT =
0x80000000ULL;
```

- VK_BUFFER_USAGE_2_TRANSFER_SRC_BIT_KHR specifies that the buffer **can** be used as the source of a *transfer command* (see the definition of VK_PIPELINE_STAGE_TRANSFER_BIT).
- VK_BUFFER_USAGE_2_TRANSFER_DST_BIT_KHR specifies that the buffer **can** be used as the destination of a transfer command.

- VK_BUFFER_USAGE_2_UNIFORM_TEXEL_BUFFER_BIT_KHR specifies that the buffer **can** be used to create a VkBufferView suitable for occupying a VkDescriptorSet slot of type VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER.
- VK_BUFFER_USAGE_2_STORAGE_TEXEL_BUFFER_BIT_KHR specifies that the buffer **can** be used to create a VkBufferView suitable for occupying a VkDescriptorSet slot of type VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER.
- VK_BUFFER_USAGE_2_UNIFORM_BUFFER_BIT_KHR specifies that the buffer **can** be used in a VkDescriptorBufferInfo suitable for occupying a VkDescriptorSet slot either of type VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER or VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC.
- VK_BUFFER_USAGE_2_STORAGE_BUFFER_BIT_KHR specifies that the buffer **can** be used in a VkDescriptorBufferInfo suitable for occupying a VkDescriptorSet slot either of type VK_DESCRIPTOR_TYPE_STORAGE_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC.
- VK_BUFFER_USAGE_2_INDEX_BUFFER_BIT_KHR specifies that the buffer is suitable for passing as the buffer parameter to vkCmdBindIndexBuffer2KHR and vkCmdBindIndexBuffer.
- VK_BUFFER_USAGE_2_VERTEX_BUFFER_BIT_KHR specifies that the buffer is suitable for passing as an element of the pBuffers array to vkCmdBindVertexBuffers.
- VK_BUFFER_USAGE_2_INDIRECT_BUFFER_BIT_KHR specifies that the buffer is suitable for passing as the buffer parameter to vkCmdDrawIndirect, vkCmdDrawIndexedIndirect, or vkCmdDispatchIndirect. It is also suitable for passing as the underlying buffer of either the preprocessAddress or sequenceCountAddress members of VkGeneratedCommandsInfoEXT.
- VK_BUFFER_USAGE_2_TRANSFORM_FEEDBACK_BUFFER_BIT_EXT specifies that the buffer is suitable for using for binding as a transform feedback buffer with vkCmdBindTransformFeedbackBuffersEXT.
- VK_BUFFER_USAGE_2_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT specifies that the buffer is suitable for using as a counter buffer with vkCmdBeginTransformFeedbackEXT and vkCmdEndTransformFeedbackEXT.
- VK_BUFFER_USAGE_2_SHADER_BINDING_TABLE_BIT_KHR specifies that the buffer is suitable for use as a Shader Binding Table.
- VK_BUFFER_USAGE_2_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR specifies that the buffer is suitable for use as a read-only input to an acceleration structure build.
- VK_BUFFER_USAGE_2_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR specifies that the buffer is suitable for storage space for a VkAccelerationStructureKHR.
- VK_BUFFER_USAGE_2_SHADER_DEVICE_ADDRESS_BIT_KHR specifies that the buffer **can** be used to retrieve a buffer device address via vkGetBufferDeviceAddressKHR and use that address to access the buffer's memory from a shader.
- VK_BUFFER_USAGE_2_VIDEO_DECODE_SRC_BIT_KHR specifies that the buffer **can** be used as the source video bitstream buffer in a video decode operation.
- VK_BUFFER_USAGE_2_VIDEO_DECODE_DST_BIT_KHR is reserved for future use.
- VK_BUFFER_USAGE_2_VIDEO_ENCODE_DST_BIT_KHR specifies that the buffer **can** be used as the destination video bitstream buffer in a video encode operation.
- VK_BUFFER_USAGE_2_VIDEO_ENCODE_SRC_BIT_KHR is reserved for future use.

- VK_BUFFER_USAGE_2_PREPROCESS_BUFFER_BIT_EXT specifies that the buffer **can** be used as a preprocess buffer for Device-Generated Commands.

```
// Provided by VK_KHR_maintenance5
typedef VkFlags64 VkBufferUsageFlags2KHR;
```

VkBufferUsageFlags2KHR is a bitmask type for setting a mask of zero or more VkBufferUsageFlagBits2KHR.

Bits which **can** be set in VkBufferCreateInfo::usage, specifying usage behavior of a buffer, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkBufferUsageFlagBits {
    VK_BUFFER_USAGE_TRANSFER_SRC_BIT = 0x00000001,
    VK_BUFFER_USAGE_TRANSFER_DST_BIT = 0x00000002,
    VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT = 0x00000004,
    VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT = 0x00000008,
    VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT = 0x00000010,
    VK_BUFFER_USAGE_STORAGE_BUFFER_BIT = 0x00000020,
    VK_BUFFER_USAGE_INDEX_BUFFER_BIT = 0x00000040,
    VK_BUFFER_USAGE_VERTEX_BUFFER_BIT = 0x00000080,
    VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT = 0x00000100,
    VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT = 0x00020000,
  // Provided by VK_KHR_video_decode_queue
    VK_BUFFER_USAGE_VIDEO_DECODE_SRC_BIT_KHR = 0x00002000,
  // Provided by VK_KHR_video_decode_queue
    VK_BUFFER_USAGE_VIDEO_DECODE_DST_BIT_KHR = 0x00004000,
  // Provided by VK_EXT_transform_feedback
    VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_BUFFER_BIT_EXT = 0x00000800,
  // Provided by VK_EXT_transform_feedback
    VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT = 0x00001000,
  // Provided by VK_KHR_acceleration_structure
    VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR = 0x00080000,
  // Provided by VK_KHR_acceleration_structure
    VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR = 0x00100000,
  // Provided by VK_KHR_ray_tracing_pipeline
    VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR = 0x00000400,
  // Provided by VK_KHR_video_encode_queue
    VK_BUFFER_USAGE_VIDEO_ENCODE_DST_BIT_KHR = 0x00008000,
  // Provided by VK_KHR_video_encode_queue
    VK_BUFFER_USAGE_VIDEO_ENCODE_SRC_BIT_KHR = 0x00010000,
  // Provided by VK_EXT_opacity_micromap
    VK_BUFFER_USAGE_MICROMAP_BUILD_INPUT_READ_ONLY_BIT_EXT = 0x00800000,
  // Provided by VK_EXT_opacity_micromap
    VK_BUFFER_USAGE_MICROMAP_STORAGE_BIT_EXT = 0x01000000,
  // Provided by VK_KHR_buffer_device_address
    VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT_KHR =
VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT,
} VkBufferUsageFlagBits;
```

- VK_BUFFER_USAGE_TRANSFER_SRC_BIT specifies that the buffer **can** be used as the source of a *transfer command* (see the definition of VK_PIPELINE_STAGE_TRANSFER_BIT).
- VK_BUFFER_USAGE_TRANSFER_DST_BIT specifies that the buffer **can** be used as the destination of a transfer command.
- VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT specifies that the buffer **can** be used to create a VkBufferView suitable for occupying a VkDescriptorSet slot of type VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER.
- VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT specifies that the buffer **can** be used to create a VkBufferView suitable for occupying a VkDescriptorSet slot of type

VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER.

- VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT specifies that the buffer **can** be used in a VkDescriptorBufferInfo suitable for occupying a VkDescriptorSet slot either of type VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER or VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC.
- VK_BUFFER_USAGE_STORAGE_BUFFER_BIT specifies that the buffer **can** be used in a VkDescriptorBufferInfo suitable for occupying a VkDescriptorSet slot either of type VK_DESCRIPTOR_TYPE_STORAGE_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC.
- VK_BUFFER_USAGE_INDEX_BUFFER_BIT specifies that the buffer is suitable for passing as the buffer parameter to vkCmdBindIndexBuffer2KHR and vkCmdBindIndexBuffer.
- VK_BUFFER_USAGE_VERTEX_BUFFER_BIT specifies that the buffer is suitable for passing as an element of the pBuffers array to vkCmdBindVertexBuffers.
- VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT specifies that the buffer is suitable for passing as the buffer parameter to vkCmdDrawIndirect, vkCmdDrawIndexedIndirect, or vkCmdDispatchIndirect. It is also suitable for passing as the underlying buffer of either the preprocessAddress or sequenceCountAddress members of VkGeneratedCommandsInfoEXT.
- VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_BUFFER_BIT_EXT specifies that the buffer is suitable for using for binding as a transform feedback buffer with vkCmdBindTransformFeedbackBuffersEXT.
- VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT specifies that the buffer is suitable for using as a counter buffer with vkCmdBeginTransformFeedbackEXT and vkCmdEndTransformFeedbackEXT.
- VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR specifies that the buffer is suitable for use as a Shader Binding Table.
- VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR specifies that the buffer is suitable for use as a read-only input to an acceleration structure build.
- VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR specifies that the buffer is suitable for storage space for a VkAccelerationStructureKHR.
- VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT specifies that the buffer **can** be used to retrieve a buffer device address via vkGetBufferDeviceAddressKHR and use that address to access the buffer's memory from a shader.
- VK_BUFFER_USAGE_VIDEO_DECODE_SRC_BIT_KHR specifies that the buffer **can** be used as the source video bitstream buffer in a video decode operation.
- VK_BUFFER_USAGE_VIDEO_DECODE_DST_BIT_KHR is reserved for future use.
- VK_BUFFER_USAGE_VIDEO_ENCODE_DST_BIT_KHR specifies that the buffer **can** be used as the destination video bitstream buffer in a video encode operation.
- VK_BUFFER_USAGE_VIDEO_ENCODE_SRC_BIT_KHR is reserved for future use.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkBufferUsageFlags;
```

VkBufferUsageFlags is a bitmask type for setting a mask of zero or more VkBufferUsageFlagBits.

Bits which **can** be set in VkBufferCreateInfo::flags, specifying additional parameters of a buffer, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkBufferCreateFlagBits {
    VK_BUFFER_CREATE_SPARSE_BINDING_BIT = 0x00000001,
    VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT = 0x00000002,
    VK_BUFFER_CREATE_SPARSE_ALIASED_BIT = 0x00000004,
    VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT = 0x00000010,
  // Provided by VK_KHR_video_maintenance1
    VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR = 0x00000040,
  // Provided by VK_KHR_buffer_device_address
    VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_KHR =
VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT,
} VkBufferCreateFlagBits;
```

- VK_BUFFER_CREATE_SPARSE_BINDING_BIT specifies that the buffer will be backed using sparse memory binding.
- VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT specifies that the buffer **can** be partially backed using sparse memory binding. Buffers created with this flag **must** also be created with the VK_BUFFER_CREATE_SPARSE_BINDING_BIT flag.
- VK_BUFFER_CREATE_SPARSE_ALIASED_BIT specifies that the buffer will be backed using sparse memory binding with memory ranges that might also simultaneously be backing another buffer (or another portion of the same buffer). Buffers created with this flag **must** also be created with the VK_BUFFER_CREATE_SPARSE_BINDING_BIT flag.
- VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT specifies that the buffer's address **can** be saved and reused on a subsequent run (e.g. for trace capture and replay), see VkBufferOpaqueCaptureAddressCreateInfo for more detail.
- VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR specifies that the buffer **can** be used in video coding operations without having to specify at buffer creation time the set of video profiles the buffer will be used with.

See Sparse Resource Features and Physical Device Features for details of the sparse memory features supported on a device.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkBufferCreateFlags;
```

VkBufferCreateFlags is a bitmask type for setting a mask of zero or more VkBufferCreateFlagBits.

To define a set of external memory handle types that **may** be used as backing store for a buffer, add a VkExternalMemoryBufferCreateInfo structure to the pNext chain of the VkBufferCreateInfo structure. The VkExternalMemoryBufferCreateInfo structure is defined as:

```
typedef struct VkExternalMemoryBufferCreateInfo {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExternalMemoryHandleTypeFlags       handleTypes;
} VkExternalMemoryBufferCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_memory
typedef VkExternalMemoryBufferCreateInfo VkExternalMemoryBufferCreateInfoKHR;
```

NOTE A `VkExternalMemoryBufferCreateInfo` structure with a non-zero `handleTypes` field must be included in the creation parameters for a buffer that will be bound to memory that is either exported or imported.

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleTypes` is zero or a bitmask of VkExternalMemoryHandleTypeFlagBits specifying one or more external memory handle types.

Valid Usage (Implicit)

- VUID-VkExternalMemoryBufferCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_BUFFER_CREATE_INFO`
- VUID-VkExternalMemoryBufferCreateInfo-handleTypes-parameter
  `handleTypes` **must** be a valid combination of VkExternalMemoryHandleTypeFlagBits values

To request a specific device address for a buffer, add a VkBufferOpaqueCaptureAddressCreateInfo structure to the `pNext` chain of the VkBufferCreateInfo structure. The `VkBufferOpaqueCaptureAddressCreateInfo` structure is defined as:

```
typedef struct VkBufferOpaqueCaptureAddressCreateInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint64_t           opaqueCaptureAddress;
} VkBufferOpaqueCaptureAddressCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_buffer_device_address
typedef VkBufferOpaqueCaptureAddressCreateInfo
VkBufferOpaqueCaptureAddressCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `opaqueCaptureAddress` is the opaque capture address requested for the buffer.

If `opaqueCaptureAddress` is zero, no specific address is requested.

If `opaqueCaptureAddress` is not zero, then it **should** be an address retrieved from vkGetBufferOpaqueCaptureAddressKHR for an identically created buffer on the same implementation.

If this structure is not present, it is as if `opaqueCaptureAddress` is zero.

Apps **should** avoid creating buffers with app-provided addresses and implementation-provided addresses in the same process, to reduce the likelihood of `VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS` errors.

NOTE

The expected usage for this is that a trace capture/replay tool will add the `VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT` flag to all buffers that use `VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT`, and during capture will save the queried opaque device addresses in the trace. During replay, the buffers will be created specifying the original address so any address values stored in the trace data will remain valid.

Implementations are expected to separate such buffers in the GPU address space so normal allocations will avoid using these addresses. Apps/tools should avoid mixing app-provided and implementation-provided addresses for buffers created with `VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT`, to avoid address space allocation conflicts.

Valid Usage (Implicit)

- VUID-VkBufferOpaqueCaptureAddressCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BUFFER_OPAQUE_CAPTURE_ADDRESS_CREATE_INFO`

To destroy a buffer, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyBuffer(
    VkDevice                                    device,
    VkBuffer                                    buffer,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the buffer.
- `buffer` is the buffer to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyBuffer-buffer-00922
  All submitted commands that refer to `buffer`, either directly or via a `VkBufferView`, **must** have completed execution
- VUID-vkDestroyBuffer-buffer-00923
  If `VkAllocationCallbacks` were provided when `buffer` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyBuffer-buffer-00924
  If no `VkAllocationCallbacks` were provided when `buffer` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyBuffer-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyBuffer-buffer-parameter
  If `buffer` is not VK_NULL_HANDLE, `buffer` **must** be a valid VkBuffer handle
- VUID-vkDestroyBuffer-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyBuffer-buffer-parent
  If `buffer` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `buffer` **must** be externally synchronized

12.2. Buffer Views

A *buffer view* represents a contiguous range of a buffer and a specific format to be used to interpret the data. Buffer views are used to enable shaders to access buffer contents using image operations. In order to create a valid buffer view, the buffer **must** have been created with at least one of the following usage flags:

- `VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT`
- `VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT`

Buffer views are represented by VkBufferView handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkBufferView)
```

To create a buffer view, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateBufferView(
    VkDevice                                    device,
    const VkBufferViewCreateInfo*               pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkBufferView*                               pView);
```

- device is the logical device that creates the buffer view.
- pCreateInfo is a pointer to a VkBufferViewCreateInfo structure containing parameters to be used to create the buffer view.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.
- pView is a pointer to a VkBufferView handle in which the resulting buffer view object is returned.

Valid Usage

- VUID-vkCreateBufferView-device-09665
  device **must** support at least one queue family with one of the VK_QUEUE_COMPUTE_BIT or VK_QUEUE_GRAPHICS_BIT capabilities

Valid Usage (Implicit)

- VUID-vkCreateBufferView-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateBufferView-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkBufferViewCreateInfo structure
- VUID-vkCreateBufferView-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateBufferView-pView-parameter
  pView **must** be a valid pointer to a VkBufferView handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkBufferViewCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkBufferViewCreateInfo {
    VkStructureType            sType;
    const void*                pNext;
    VkBufferViewCreateFlags    flags;
    VkBuffer                   buffer;
    VkFormat                   format;
    VkDeviceSize               offset;
    VkDeviceSize               range;
} VkBufferViewCreateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- buffer is a VkBuffer on which the view will be created.
- format is a VkFormat describing the format of the data elements in the buffer.
- offset is an offset in bytes from the base address of the buffer. Accesses to the buffer view from shaders use addressing that is relative to this starting offset.
- range is a size in bytes of the buffer view. If range is equal to VK_WHOLE_SIZE, the range from offset to the end of the buffer is used. If VK_WHOLE_SIZE is used and the remaining size of the buffer is not a multiple of the texel block size of format, the nearest smaller multiple is used.

The buffer view has a *buffer view usage* identifying which descriptor types can be created from it. This usage **can** be defined by including the VkBufferUsageFlags2CreateInfoKHR structure in the pNext chain, and specifying the usage value there. If this structure is not included, it is equal to the VkBufferCreateInfo::usage value used to create buffer.

Valid Usage

- VUID-VkBufferViewCreateInfo-offset-00925
  offset **must** be less than the size of buffer
- VUID-VkBufferViewCreateInfo-range-00928

If range is not equal to VK_WHOLE_SIZE, range **must** be greater than 0

- VUID-VkBufferViewCreateInfo-range-00929
  If range is not equal to VK_WHOLE_SIZE, range **must** be an integer multiple of the texel block size of format
- VUID-VkBufferViewCreateInfo-range-00930
  If range is not equal to VK_WHOLE_SIZE, the number of texel buffer elements given by (⌊range / (texel block size)⌋ × (texels per block)) where texel block size and texels per block are as defined in the Compatible Formats table for format, **must** be less than or equal to VkPhysicalDeviceLimits::maxTexelBufferElements
- VUID-VkBufferViewCreateInfo-offset-00931
  If range is not equal to VK_WHOLE_SIZE, the sum of offset and range **must** be less than or equal to the size of buffer
- VUID-VkBufferViewCreateInfo-range-04059
  If range is equal to VK_WHOLE_SIZE, the number of texel buffer elements given by (⌊(size - offset) / (texel block size)⌋ × (texels per block)) where size is the size of buffer, and texel block size and texels per block are as defined in the Compatible Formats table for format, **must** be less than or equal to VkPhysicalDeviceLimits::maxTexelBufferElements
- VUID-VkBufferViewCreateInfo-buffer-00932
  buffer **must** have been created with a usage value containing at least one of VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT or VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT
- VUID-VkBufferViewCreateInfo-format-08778
  If the buffer view usage contains VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT, then format features of format **must** contain VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT
- VUID-VkBufferViewCreateInfo-format-08779
  If the buffer view usage contains VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT, then format features of format **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT
- VUID-VkBufferViewCreateInfo-buffer-00935
  If buffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkBufferViewCreateInfo-offset-02749
  offset **must** be a multiple of VkPhysicalDeviceLimits::minTexelBufferOffsetAlignment
- VUID-VkBufferViewCreateInfo-pNext-06782
  If the pNext chain includes a VkExportMetalObjectCreateInfoEXT structure, its exportObjectType member **must** be VK_EXPORT_METAL_OBJECT_TYPE_METAL_TEXTURE_BIT_EXT
- VUID-VkBufferViewCreateInfo-pNext-08780
  If the pNext chain includes a VkBufferUsageFlags2CreateInfoKHR, its usage **must** not contain any other bit than VK_BUFFER_USAGE_2_UNIFORM_TEXEL_BUFFER_BIT_KHR or VK_BUFFER_USAGE_2_STORAGE_TEXEL_BUFFER_BIT_KHR
- VUID-VkBufferViewCreateInfo-pNext-08781
  If the pNext chain includes a VkBufferUsageFlags2CreateInfoKHR, its usage **must** be a subset of the VkBufferCreateInfo::usage specified or VkBufferUsageFlags2CreateInfoKHR ::usage from VkBufferCreateInfo::pNext when creating buffer

Valid Usage (Implicit)

- VUID-VkBufferViewCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BUFFER_VIEW_CREATE_INFO`
- VUID-VkBufferViewCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkBufferUsageFlags2CreateInfoKHR or VkExportMetalObjectCreateInfoEXT
- VUID-VkBufferViewCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique, with the exception of structures of type VkExportMetalObjectCreateInfoEXT
- VUID-VkBufferViewCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkBufferViewCreateInfo-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle
- VUID-VkBufferViewCreateInfo-format-parameter
  `format` **must** be a valid VkFormat value

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkBufferViewCreateFlags;
```

`VkBufferViewCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

To destroy a buffer view, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyBufferView(
    VkDevice                                    device,
    VkBufferView                                bufferView,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the buffer view.
- `bufferView` is the buffer view to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyBufferView-bufferView-00936
  All submitted commands that refer to `bufferView` **must** have completed execution
- VUID-vkDestroyBufferView-bufferView-00937
  If `VkAllocationCallbacks` were provided when `bufferView` was created, a compatible set of

callbacks **must** be provided here

- VUID-vkDestroyBufferView-bufferView-00938
  If no `VkAllocationCallbacks` were provided when `bufferView` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyBufferView-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyBufferView-bufferView-parameter
  If `bufferView` is not VK_NULL_HANDLE, `bufferView` **must** be a valid VkBufferView handle
- VUID-vkDestroyBufferView-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyBufferView-bufferView-parent
  If `bufferView` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `bufferView` **must** be externally synchronized

12.2.1. Buffer View Format Features

Valid uses of a VkBufferView **may** depend on the buffer view's *format features*, defined below. Such constraints are documented in the affected valid usage statement.

- If Vulkan 1.3 is supported or the `VK_KHR_format_feature_flags2` extension is supported, then the buffer view's set of *format features* is the value of VkFormatProperties3::`bufferFeatures` found by calling vkGetPhysicalDeviceFormatProperties2KHR on the same `format` as VkBufferViewCreateInfo::`format`.

12.3. Images

Images represent multidimensional - up to 3 - arrays of data which **can** be used for various purposes (e.g. attachments, textures), by binding them to a graphics or compute pipeline via descriptor sets, or by directly specifying them as parameters to certain commands.

Images are represented by `VkImage` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkImage)
```

To create images, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateImage(
    VkDevice                                    device,
    const VkImageCreateInfo*                    pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkImage*                                    pImage);
```

- `device` is the logical device that creates the image.
- `pCreateInfo` is a pointer to a VkImageCreateInfo structure containing parameters to be used to create the image.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pImage` is a pointer to a VkImage handle in which the resulting image object is returned.

Valid Usage

- VUID-vkCreateImage-device-09666
  `device` **must** support at least one queue family with one of the `VK_QUEUE_VIDEO_ENCODE_BIT_KHR`, `VK_QUEUE_VIDEO_DECODE_BIT_KHR`, `VK_QUEUE_SPARSE_BINDING_BIT`, `VK_QUEUE_TRANSFER_BIT`, `VK_QUEUE_COMPUTE_BIT`, or `VK_QUEUE_GRAPHICS_BIT` capabilities
- VUID-vkCreateImage-flags-00939
  If the `flags` member of `pCreateInfo` includes `VK_IMAGE_CREATE_SPARSE_BINDING_BIT`, creating this `VkImage` **must** not cause the total required sparse memory for all currently valid sparse resources on the device to exceed `VkPhysicalDeviceLimits`::`sparseAddressSpaceSize`

Valid Usage (Implicit)

- VUID-vkCreateImage-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateImage-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkImageCreateInfo structure
- VUID-vkCreateImage-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateImage-pImage-parameter
  `pImage` **must** be a valid pointer to a VkImage handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR

The VkImageCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageCreateInfo {
    VkStructureType          sType;
    const void*              pNext;
    VkImageCreateFlags       flags;
    VkImageType              imageType;
    VkFormat                 format;
    VkExtent3D               extent;
    uint32_t                 mipLevels;
    uint32_t                 arrayLayers;
    VkSampleCountFlagBits    samples;
    VkImageTiling            tiling;
    VkImageUsageFlags        usage;
    VkSharingMode            sharingMode;
    uint32_t                 queueFamilyIndexCount;
    const uint32_t*          pQueueFamilyIndices;
    VkImageLayout            initialLayout;
} VkImageCreateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkImageCreateFlagBits describing additional parameters of the image.
- imageType is a VkImageType value specifying the basic dimensionality of the image. Layers in array textures do not count as a dimension for the purposes of the image type.
- format is a VkFormat describing the format and type of the texel blocks that will be contained in the image.
- extent is a VkExtent3D describing the number of data elements in each dimension of the base level.
- mipLevels describes the number of levels of detail available for minified sampling of the image.
- arrayLayers is the number of layers in the image.
- samples is a VkSampleCountFlagBits value specifying the number of samples per texel.

- `tiling` is a VkImageTiling value specifying the tiling arrangement of the texel blocks in memory.
- `usage` is a bitmask of VkImageUsageFlagBits describing the intended usage of the image.
- `sharingMode` is a VkSharingMode value specifying the sharing mode of the image when it will be accessed by multiple queue families.
- `queueFamilyIndexCount` is the number of entries in the `pQueueFamilyIndices` array.
- `pQueueFamilyIndices` is a pointer to an array of queue families that will access this image. It is ignored if `sharingMode` is not `VK_SHARING_MODE_CONCURRENT`.
- `initialLayout` is a VkImageLayout value specifying the initial VkImageLayout of all image subresources of the image. See Image Layouts.

Images created with `tiling` equal to `VK_IMAGE_TILING_LINEAR` have further restrictions on their limits and capabilities compared to images created with `tiling` equal to `VK_IMAGE_TILING_OPTIMAL`. Creation of images with tiling `VK_IMAGE_TILING_LINEAR` **may** not be supported unless other parameters meet all of the constraints:

- `imageType` is `VK_IMAGE_TYPE_2D`
- `format` is not a depth/stencil format
- `mipLevels` is 1
- `arrayLayers` is 1
- `samples` is `VK_SAMPLE_COUNT_1_BIT`
- `usage` only includes `VK_IMAGE_USAGE_TRANSFER_SRC_BIT` and/or `VK_IMAGE_USAGE_TRANSFER_DST_BIT`

Images created with one of the formats that require a sampler $Y'C_BC_R$ conversion, have further restrictions on their limits and capabilities compared to images created with other formats. Creation of images with a format requiring $Y'C_BC_R$ conversion **may** not be supported unless other parameters meet all of the constraints:

- `imageType` is `VK_IMAGE_TYPE_2D`
- `mipLevels` is 1
- `arrayLayers` is 1, unless the `ycbcrImageArrays` feature is enabled, or otherwise indicated by VkImageFormatProperties::`maxArrayLayers`, as returned by vkGetPhysicalDeviceImageFormatProperties
- `samples` is `VK_SAMPLE_COUNT_1_BIT`

Implementations **may** support additional limits and capabilities beyond those listed above.

To determine the set of valid `usage` bits for a given format, call vkGetPhysicalDeviceFormatProperties.

If the size of the resultant image would exceed `maxResourceSize`, then vkCreateImage **must** fail and return `VK_ERROR_OUT_OF_DEVICE_MEMORY`. This failure **may** occur even when all image creation parameters satisfy their valid usage requirements.

If the implementation reports `VK_TRUE` in VkPhysicalDeviceHostImageCopyPropertiesEXT ::`identicalMemoryTypeRequirements`, usage of `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT` **must** not affect

the memory type requirements of the image as described in Sparse Resource Memory Requirements and Resource Memory Association.

NOTE

For images created without VK_IMAGE_CREATE_EXTENDED_USAGE_BIT a usage bit is valid if it is supported for the format the image is created with.

For images created with VK_IMAGE_CREATE_EXTENDED_USAGE_BIT a usage bit is valid if it is supported for at least one of the formats a VkImageView created from the image **can** have (see Image Views for more detail).

Image Creation Limits

Valid values for some image creation parameters are limited by a numerical upper bound or by inclusion in a bitset. For example, VkImageCreateInfo::arrayLayers is limited by imageCreateMaxArrayLayers, defined below; and VkImageCreateInfo::samples is limited by imageCreateSampleCounts, also defined below.

Several limiting values are defined below, as well as assisting values from which the limiting values are derived. The limiting values are referenced by the relevant valid usage statements of VkImageCreateInfo.

- Let uint64_t imageCreateDrmFormatModifiers[] be the set of Linux DRM format modifiers that the resultant image **may** have.
  - If tiling is not VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then imageCreateDrmFormatModifiers is empty.
  - If VkImageCreateInfo::pNext contains VkImageDrmFormatModifierExplicitCreateInfoEXT, then imageCreateDrmFormatModifiers contains exactly one modifier, VkImageDrmFormatModifierExplicitCreateInfoEXT::drmFormatModifier.
  - If VkImageCreateInfo::pNext contains VkImageDrmFormatModifierListCreateInfoEXT, then imageCreateDrmFormatModifiers contains the entire array VkImageDrmFormatModifierListCreateInfoEXT::pDrmFormatModifiers.
- Let VkBool32 imageCreateMaybeLinear indicate if the resultant image may be linear.
  - If tiling is VK_IMAGE_TILING_LINEAR, then imageCreateMaybeLinear is VK_TRUE.
  - If tiling is VK_IMAGE_TILING_OPTIMAL, then imageCreateMaybeLinear is VK_FALSE.
  - If tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then imageCreateMaybeLinear is VK_TRUE if and only if imageCreateDrmFormatModifiers contains DRM_FORMAT_MOD_LINEAR.
- Let VkFormatFeatureFlags imageCreateFormatFeatures be the set of valid *format features* available during image creation.
  - If tiling is VK_IMAGE_TILING_LINEAR, then imageCreateFormatFeatures is the value of VkFormatProperties::linearTilingFeatures found by calling vkGetPhysicalDeviceFormatProperties with parameter format equal to VkImageCreateInfo::format.
  - If tiling is VK_IMAGE_TILING_OPTIMAL, then imageCreateFormatFeatures is the value of

VkFormatProperties::optimalTilingFeatures found by calling vkGetPhysicalDeviceFormatProperties with parameter format equal to VkImageCreateInfo::format.

  - If tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then the value of imageCreateFormatFeatures is found by calling vkGetPhysicalDeviceFormatProperties2KHR with VkImageFormatProperties::format equal to VkImageCreateInfo::format and with VkDrmFormatModifierPropertiesListEXT chained into VkFormatProperties2; by collecting all members of the returned array VkDrmFormatModifierPropertiesListEXT::pDrmFormatModifierProperties whose drmFormatModifier belongs to imageCreateDrmFormatModifiers; and by taking the bitwise intersection, over the collected array members, of drmFormatModifierTilingFeatures. (The resultant imageCreateFormatFeatures **may** be empty).

- Let VkImageFormatProperties2 imageCreateImageFormatPropertiesList[] be the list of structures obtained by calling vkGetPhysicalDeviceImageFormatProperties2KHR, possibly multiple times, as follows:
  - The parameters VkPhysicalDeviceImageFormatInfo2::format, imageType, tiling, usage, and flags **must** be equal to those in VkImageCreateInfo.
  - If VkImageCreateInfo::pNext contains a VkExternalMemoryImageCreateInfo structure whose handleTypes is not 0, then VkPhysicalDeviceImageFormatInfo2::pNext **must** contain a VkPhysicalDeviceExternalImageFormatInfo structure whose handleType is not 0; and vkGetPhysicalDeviceImageFormatProperties2KHR **must** be called for each handle type in VkExternalMemoryImageCreateInfo::handleTypes, successively setting VkPhysicalDeviceExternalImageFormatInfo::handleType on each call.
  - If VkImageCreateInfo::pNext contains no VkExternalMemoryImageCreateInfo structure, or contains a structure whose handleTypes is 0, then VkPhysicalDeviceImageFormatInfo2::pNext **must** either contain no VkPhysicalDeviceExternalImageFormatInfo structure, or contain a structure whose handleType is 0.
  - If VkImageCreateInfo::pNext contains a VkVideoProfileListInfoKHR structure then VkPhysicalDeviceImageFormatInfo2::pNext **must** also contain the same VkVideoProfileListInfoKHR structure on each call.
  - If tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then:
    - VkPhysicalDeviceImageFormatInfo2::pNext **must** contain a VkPhysicalDeviceImageDrmFormatModifierInfoEXT structure where sharingMode is equal to VkImageCreateInfo::sharingMode;
    - if sharingMode is VK_SHARING_MODE_CONCURRENT, then queueFamilyIndexCount and pQueueFamilyIndices **must** be equal to those in VkImageCreateInfo;
    - if flags contains VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT, then the VkImageFormatListCreateInfo structure included in the pNext chain of VkPhysicalDeviceImageFormatInfo2 **must** be equivalent to the one included in the pNext chain of VkImageCreateInfo;
    - vkGetPhysicalDeviceImageFormatProperties2KHR **must** be called for each

modifier in imageCreateDrmFormatModifiers, successively setting VkPhysicalDeviceImageDrmFormatModifierInfoEXT::drmFormatModifier on each call.

  - If tiling is not VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then VkPhysicalDeviceImageFormatInfo2::pNext **must** contain no VkPhysicalDeviceImageDrmFormatModifierInfoEXT structure.
  - If any call to vkGetPhysicalDeviceImageFormatProperties2KHR returns an error, then imageCreateImageFormatPropertiesList is defined to be the empty list.

- Let uint32_t imageCreateMaxMipLevels be the minimum value of VkImageFormatProperties::maxMipLevels in imageCreateImageFormatPropertiesList. The value is undefined if imageCreateImageFormatPropertiesList is empty.
- Let uint32_t imageCreateMaxArrayLayers be the minimum value of VkImageFormatProperties::maxArrayLayers in imageCreateImageFormatPropertiesList. The value is undefined if imageCreateImageFormatPropertiesList is empty.
- Let VkExtent3D imageCreateMaxExtent be the component-wise minimum over all VkImageFormatProperties::maxExtent values in imageCreateImageFormatPropertiesList. The value is undefined if imageCreateImageFormatPropertiesList is empty.
- Let VkSampleCountFlags imageCreateSampleCounts be the intersection of each VkImageFormatProperties::sampleCounts in imageCreateImageFormatPropertiesList. The value is undefined if imageCreateImageFormatPropertiesList is empty.
- Let VkVideoFormatPropertiesKHR videoFormatProperties[] be defined as follows.
  - If VkImageCreateInfo::pNext contains a VkVideoProfileListInfoKHR structure, then videoFormatProperties is the list of structures obtained by calling vkGetPhysicalDeviceVideoFormatPropertiesKHR with VkPhysicalDeviceVideoFormatInfoKHR::imageUsage equal to the usage member of VkImageCreateInfo and VkPhysicalDeviceVideoFormatInfoKHR::pNext containing the same VkVideoProfileListInfoKHR structure chained to VkImageCreateInfo.
  - If VkImageCreateInfo::pNext contains no VkVideoProfileListInfoKHR structure, then videoFormatProperties is an empty list.
- Let VkBool32 supportedVideoFormat indicate if the image parameters are supported by the specified video profiles.
  - supportedVideoFormat is VK_TRUE if there exists an element in the videoFormatProperties list for which all of the following conditions are true:
    - VkImageCreateInfo::format equals VkVideoFormatPropertiesKHR::format.
    - VkImageCreateInfo::flags only contains VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR and/or bits also set in VkVideoFormatPropertiesKHR::imageCreateFlags.
    - VkImageCreateInfo::imageType equals VkVideoFormatPropertiesKHR::imageType.
    - VkImageCreateInfo::tiling equals VkVideoFormatPropertiesKHR::imageTiling.
    - VkImageCreateInfo::usage only contains bits also set in VkVideoFormatPropertiesKHR::imageUsageFlags, or VkImageCreateInfo::flags

includes VK_IMAGE_CREATE_EXTENDED_USAGE_BIT.

- Otherwise supportedVideoFormat is VK_FALSE.

Valid Usage

- VUID-VkImageCreateInfo-imageCreateMaxMipLevels-02251
  Each of the following values (as described in Image Creation Limits) **must** not be undefined : imageCreateMaxMipLevels, imageCreateMaxArrayLayers, imageCreateMaxExtent, and imageCreateSampleCounts
- VUID-VkImageCreateInfo-sharingMode-00941
  If sharingMode is VK_SHARING_MODE_CONCURRENT, pQueueFamilyIndices **must** be a valid pointer to an array of queueFamilyIndexCount uint32_t values
- VUID-VkImageCreateInfo-sharingMode-00942
  If sharingMode is VK_SHARING_MODE_CONCURRENT, queueFamilyIndexCount **must** be greater than 1
- VUID-VkImageCreateInfo-sharingMode-01420
  If sharingMode is VK_SHARING_MODE_CONCURRENT, each element of pQueueFamilyIndices **must** be unique and **must** be less than pQueueFamilyPropertyCount returned by either vkGetPhysicalDeviceQueueFamilyProperties or vkGetPhysicalDeviceQueueFamilyProperties2KHR for the physicalDevice that was used to create device
- VUID-VkImageCreateInfo-format-00943
  format **must** not be VK_FORMAT_UNDEFINED
- VUID-VkImageCreateInfo-extent-00944
  extent.width **must** be greater than 0
- VUID-VkImageCreateInfo-extent-00945
  extent.height **must** be greater than 0
- VUID-VkImageCreateInfo-extent-00946
  extent.depth **must** be greater than 0
- VUID-VkImageCreateInfo-mipLevels-00947
  mipLevels **must** be greater than 0
- VUID-VkImageCreateInfo-arrayLayers-00948
  arrayLayers **must** be greater than 0
- VUID-VkImageCreateInfo-flags-00949
  If flags contains VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT, imageType **must** be VK_IMAGE_TYPE_2D
- VUID-VkImageCreateInfo-flags-08865
  If flags contains VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT, extent.width and extent.height **must** be equal
- VUID-VkImageCreateInfo-flags-08866
  If flags contains VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT, arrayLayers **must** be greater than or equal to 6

- VUID-VkImageCreateInfo-flags-02557
  If `flags` contains `VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT`, `imageType` **must** be `VK_IMAGE_TYPE_2D`
- VUID-VkImageCreateInfo-flags-00950
  If `flags` contains `VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT`, `imageType` **must** be `VK_IMAGE_TYPE_3D`
- VUID-VkImageCreateInfo-flags-09403
  If `flags` contains `VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT`, `flags` **must** not include `VK_IMAGE_CREATE_SPARSE_ALIASED_BIT`, `VK_IMAGE_CREATE_SPARSE_BINDING_BIT`, or `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-flags-07755
  If `flags` contains `VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT`, `imageType` **must** be `VK_IMAGE_TYPE_3D`
- VUID-VkImageCreateInfo-imageType-10197
  If `flags` contains `VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT`, `flags` **must** not include `VK_IMAGE_CREATE_SPARSE_ALIASED_BIT`, `VK_IMAGE_CREATE_SPARSE_BINDING_BIT`, or `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-extent-02252
  `extent.width` **must** be less than or equal to `imageCreateMaxExtent.width` (as defined in Image Creation Limits)
- VUID-VkImageCreateInfo-extent-02253
  `extent.height` **must** be less than or equal to `imageCreateMaxExtent.height` (as defined in Image Creation Limits)
- VUID-VkImageCreateInfo-extent-02254
  `extent.depth` **must** be less than or equal to `imageCreateMaxExtent.depth` (as defined in Image Creation Limits)
- VUID-VkImageCreateInfo-imageType-00956
  If `imageType` is `VK_IMAGE_TYPE_1D`, both `extent.height` and `extent.depth` **must** be `1`
- VUID-VkImageCreateInfo-imageType-00957
  If `imageType` is `VK_IMAGE_TYPE_2D`, `extent.depth` **must** be `1`
- VUID-VkImageCreateInfo-mipLevels-00958
  `mipLevels` **must** be less than or equal to the number of levels in the complete mipmap chain based on `extent.width`, `extent.height`, and `extent.depth`
- VUID-VkImageCreateInfo-mipLevels-02255
  `mipLevels` **must** be less than or equal to `imageCreateMaxMipLevels` (as defined in Image Creation Limits)
- VUID-VkImageCreateInfo-arrayLayers-02256
  `arrayLayers` **must** be less than or equal to `imageCreateMaxArrayLayers` (as defined in Image Creation Limits)
- VUID-VkImageCreateInfo-imageType-00961
  If `imageType` is `VK_IMAGE_TYPE_3D`, `arrayLayers` **must** be `1`
- VUID-VkImageCreateInfo-samples-02257

If `samples` is not `VK_SAMPLE_COUNT_1_BIT`, then `imageType` **must** be `VK_IMAGE_TYPE_2D`, `flags` **must** not contain `VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT`, `mipLevels` **must** be equal to `1`, and `imageCreateMaybeLinear` (as defined in Image Creation Limits) **must** be `VK_FALSE`,

- VUID-VkImageCreateInfo-samples-02558
  If `samples` is not `VK_SAMPLE_COUNT_1_BIT`, `usage` **must** not contain `VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT`
- VUID-VkImageCreateInfo-usage-00963
  If `usage` includes `VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT`, then bits other than `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`, `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`, and `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT` **must** not be set
- VUID-VkImageCreateInfo-usage-00964
  If `usage` includes `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`, `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`, `VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT`, or `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT`, `extent.width` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxFramebufferWidth`
- VUID-VkImageCreateInfo-usage-00965
  If `usage` includes `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`, `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`, `VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT`, or `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT`, `extent.height` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxFramebufferHeight`
- VUID-VkImageCreateInfo-usage-02559
  If `usage` includes `VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT`, `extent.width` **must** be less than or equal to $\lceil \frac{maxFramebufferWidth}{minFragmentDensityTexelSize_{width}} \rceil$
- VUID-VkImageCreateInfo-usage-02560
  If `usage` includes `VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT`, `extent.height` **must** be less than or equal to $\lceil \frac{maxFramebufferHeight}{minFragmentDensityTexelSize_{height}} \rceil$
- VUID-VkImageCreateInfo-usage-00966
  If `usage` includes `VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT`, `usage` **must** also contain at least one of `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`, `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`, or `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT`
- VUID-VkImageCreateInfo-samples-02258
  `samples` **must** be a valid VkSampleCountFlagBits value that is set in `imageCreateSampleCounts` (as defined in Image Creation Limits)
- VUID-VkImageCreateInfo-usage-00968
  If the `shaderStorageImageMultisample` feature is not enabled, and `usage` contains `VK_IMAGE_USAGE_STORAGE_BIT`, `samples` **must** be `VK_SAMPLE_COUNT_1_BIT`
- VUID-VkImageCreateInfo-flags-00969
  If the `sparseBinding` feature is not enabled, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_BINDING_BIT`
- VUID-VkImageCreateInfo-flags-01924
  If the `sparseResidencyAliased` feature is not enabled, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_ALIASED_BIT`
- VUID-VkImageCreateInfo-tiling-04121

If `tiling` is `VK_IMAGE_TILING_LINEAR`, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`

- VUID-VkImageCreateInfo-imageType-00970
  If `imageType` is `VK_IMAGE_TYPE_1D`, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-imageType-00971
  If the `sparseResidencyImage2D` feature is not enabled, and `imageType` is `VK_IMAGE_TYPE_2D`, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-imageType-00972
  If the `sparseResidencyImage3D` feature is not enabled, and `imageType` is `VK_IMAGE_TYPE_3D`, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-imageType-00973
  If the `sparseResidency2Samples` feature is not enabled, `imageType` is `VK_IMAGE_TYPE_2D`, and `samples` is `VK_SAMPLE_COUNT_2_BIT`, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-imageType-00974
  If the `sparseResidency4Samples` feature is not enabled, `imageType` is `VK_IMAGE_TYPE_2D`, and `samples` is `VK_SAMPLE_COUNT_4_BIT`, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-imageType-00975
  If the `sparseResidency8Samples` feature is not enabled, `imageType` is `VK_IMAGE_TYPE_2D`, and `samples` is `VK_SAMPLE_COUNT_8_BIT`, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-imageType-00976
  If the `sparseResidency16Samples` feature is not enabled, `imageType` is `VK_IMAGE_TYPE_2D`, and `samples` is `VK_SAMPLE_COUNT_16_BIT`, `flags` **must** not contain `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`
- VUID-VkImageCreateInfo-flags-00987
  If `flags` contains `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` or `VK_IMAGE_CREATE_SPARSE_ALIASED_BIT`, it **must** also contain `VK_IMAGE_CREATE_SPARSE_BINDING_BIT`
- VUID-VkImageCreateInfo-None-01925
  If any of the bits `VK_IMAGE_CREATE_SPARSE_BINDING_BIT`, `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`, or `VK_IMAGE_CREATE_SPARSE_ALIASED_BIT` are set, `VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT` **must** not also be set
- VUID-VkImageCreateInfo-pNext-00990
  If the `pNext` chain includes a VkExternalMemoryImageCreateInfo structure, its `handleTypes` member **must** only contain bits that are also in VkExternalImageFormatProperties::`externalMemoryProperties.compatibleHandleTypes`, as returned by vkGetPhysicalDeviceImageFormatProperties2KHR with `format`, `imageType`, `tiling`, `usage`, and `flags` equal to those in this structure, and with a VkPhysicalDeviceExternalImageFormatInfo structure included in the `pNext` chain, with a `handleType` equal to any one of the handle types specified in VkExternalMemoryImageCreateInfo::`handleTypes`

- VUID-VkImageCreateInfo-physicalDeviceCount-01421
  If the logical device was created with VkDeviceGroupDeviceCreateInfo ::physicalDeviceCount equal to 1, flags **must** not contain VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT
- VUID-VkImageCreateInfo-flags-02259
  If flags contains VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT, then mipLevels **must** be one, arrayLayers **must** be one, imageType **must** be VK_IMAGE_TYPE_2D. and imageCreateMaybeLinear (as defined in Image Creation Limits) **must** be VK_FALSE
- VUID-VkImageCreateInfo-flags-01572
  If flags contains VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT, then format **must** be a compressed image format
- VUID-VkImageCreateInfo-flags-01573
  If flags contains VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT, then flags **must** also contain VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT
- VUID-VkImageCreateInfo-initialLayout-00993
  initialLayout **must** be VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED
- VUID-VkImageCreateInfo-pNext-01443
  If the pNext chain includes a VkExternalMemoryImageCreateInfo or VkExternalMemoryImageCreateInfoNV structure whose handleTypes member is not 0, initialLayout **must** be VK_IMAGE_LAYOUT_UNDEFINED
- VUID-VkImageCreateInfo-format-06410
  If the image format is one of the formats that require a sampler $Y'C_BC_R$ conversion, mipLevels **must** be 1
- VUID-VkImageCreateInfo-format-06411
  If the image format is one of the formats that require a sampler $Y'C_BC_R$ conversion, samples **must** be VK_SAMPLE_COUNT_1_BIT
- VUID-VkImageCreateInfo-format-06412
  If the image format is one of the formats that require a sampler $Y'C_BC_R$ conversion, imageType **must** be VK_IMAGE_TYPE_2D
- VUID-VkImageCreateInfo-imageCreateFormatFeatures-02260
  If format is a *multi-planar* format, and if imageCreateFormatFeatures (as defined in Image Creation Limits) does not contain VK_FORMAT_FEATURE_DISJOINT_BIT, then flags **must** not contain VK_IMAGE_CREATE_DISJOINT_BIT
- VUID-VkImageCreateInfo-format-01577
  If format is not a *multi-planar* format, and flags does not include VK_IMAGE_CREATE_ALIAS_BIT, flags **must** not contain VK_IMAGE_CREATE_DISJOINT_BIT
- VUID-VkImageCreateInfo-format-04712
  If format has a _422 or _420 suffix, extent.width **must** be a multiple of 2
- VUID-VkImageCreateInfo-format-04713
  If format has a _420 suffix, extent.height **must** be a multiple of 2
- VUID-VkImageCreateInfo-tiling-02261
  If tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then the pNext chain **must** include exactly one of VkImageDrmFormatModifierListCreateInfoEXT or

VkImageDrmFormatModifierExplicitCreateInfoEXT structures

- VUID-VkImageCreateInfo-pNext-02262
  If the pNext chain includes a VkImageDrmFormatModifierListCreateInfoEXT or VkImageDrmFormatModifierExplicitCreateInfoEXT structure, then tiling **must** be VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT
- VUID-VkImageCreateInfo-tiling-02353
  If tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT and flags contains VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT, then the pNext chain **must** include a VkImageFormatListCreateInfo structure with non-zero viewFormatCount
- VUID-VkImageCreateInfo-flags-01533
  If flags contains VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT format **must** be a depth or depth/stencil format
- VUID-VkImageCreateInfo-imageType-02082
  If usage includes VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR, imageType **must** be VK_IMAGE_TYPE_2D
- VUID-VkImageCreateInfo-samples-02083
  If usage includes VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR, samples **must** be VK_SAMPLE_COUNT_1_BIT
- VUID-VkImageCreateInfo-flags-02565
  If flags contains VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT, tiling **must** be VK_IMAGE_TILING_OPTIMAL
- VUID-VkImageCreateInfo-flags-02566
  If flags contains VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT, imageType **must** be VK_IMAGE_TYPE_2D
- VUID-VkImageCreateInfo-flags-02567
  If flags contains VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT, flags **must** not contain VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT
- VUID-VkImageCreateInfo-flags-02568
  If flags contains VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT, mipLevels **must** be 1
- VUID-VkImageCreateInfo-imageView2DOn3DImage-04459
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::imageView2DOn3DImage is VK_FALSE, flags **must** not contain VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT
- VUID-VkImageCreateInfo-multisampleArrayImage-04460
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::multisampleArrayImage is VK_FALSE, and samples is not VK_SAMPLE_COUNT_1_BIT, then arrayLayers **must** be 1
- VUID-VkImageCreateInfo-pNext-06722
  If a VkImageFormatListCreateInfo structure was included in the pNext chain and format is not a multi-planar format and VkImageFormatListCreateInfo::viewFormatCount is not zero, then each format in VkImageFormatListCreateInfo::pViewFormats **must** either be compatible with the format as described in the compatibility table or, if flags contains VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT, be an uncompressed format that is size-compatible with format

- VUID-VkImageCreateInfo-pNext-10062
  If a VkImageFormatListCreateInfo structure was included in the pNext chain and format is a multi-planar format and flags contains VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT and VkImageFormatListCreateInfo::viewFormatCount is not zero, then each format in VkImageFormatListCreateInfo::pViewFormats **must** be compatible with the VkFormat for the plane of the image format
- VUID-VkImageCreateInfo-flags-04738
  If flags does not contain VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT and the pNext chain includes a VkImageFormatListCreateInfo structure, then VkImageFormatListCreateInfo::viewFormatCount **must** be 0 or 1
- VUID-VkImageCreateInfo-usage-04815
  If usage includes VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR, VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR, or VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR, and flags does not include VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then the pNext chain **must** include a VkVideoProfileListInfoKHR structure with profileCount greater than 0 and pProfiles including at least one VkVideoProfileInfoKHR structure with a videoCodecOperation member specifying a decode operation
- VUID-VkImageCreateInfo-usage-04816
  If usage includes VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR, VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR, or VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR, and flags does not include VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then the pNext chain **must** include a VkVideoProfileListInfoKHR structure with profileCount greater than 0 and pProfiles including at least one VkVideoProfileInfoKHR structure with a videoCodecOperation member specifying an encode operation
- VUID-VkImageCreateInfo-flags-08328
  If flags includes VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then videoMaintenance1 **must** be enabled
- VUID-VkImageCreateInfo-flags-08329
  If flags includes VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR and usage does not include VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR, then usage **must** not include VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR
- VUID-VkImageCreateInfo-flags-08331
  If flags includes VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then usage **must** not include VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR, VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR, or VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR
- VUID-VkImageCreateInfo-pNext-06811
  If the pNext chain includes a VkVideoProfileListInfoKHR structure with profileCount greater than 0, then supportedVideoFormat **must** be VK_TRUE
- VUID-VkImageCreateInfo-pNext-10250
  If the pNext chain includes a VkVideoProfileListInfoKHR structure and for any element of its pProfiles member videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the videoEncodeAV1 feature **must** be enabled

- VUID-VkImageCreateInfo-usage-10251
  If `usage` includes `VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR` or `VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR`, then the `videoEncodeQuantizationMap` feature **must** be enabled
- VUID-VkImageCreateInfo-usage-10252
  If `usage` includes `VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR` or `VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR`, `imageType` **must** be `VK_IMAGE_TYPE_2D`
- VUID-VkImageCreateInfo-usage-10253
  If `usage` includes `VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR` or `VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR`, `samples` **must** be `VK_SAMPLE_COUNT_1_BIT`
- VUID-VkImageCreateInfo-usage-10254
  If `usage` includes `VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR` or `VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR`, then the `pNext` chain **must** include a VkVideoProfileListInfoKHR structure with `profileCount` equal to `1` and `pProfiles` pointing to a VkVideoProfileInfoKHR structure with a `videoCodecOperation` member specifying an encode operation
- VUID-VkImageCreateInfo-usage-10255
  If `usage` includes `VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR`, then VkVideoEncodeCapabilitiesKHR::`flags` **must** include `VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified in the `pProfiles` member of the VkVideoProfileListInfoKHR structure included in the `pNext` chain
- VUID-VkImageCreateInfo-usage-10256
  If `usage` includes `VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR`, then VkVideoEncodeCapabilitiesKHR::`flags` **must** include `VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified in the `pProfiles` member of the VkVideoProfileListInfoKHR structure included in the `pNext` chain
- VUID-VkImageCreateInfo-usage-10257
  If `usage` includes `VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR` or `VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR`, `extent.width` **must** be less than or equal to VkVideoEncodeQuantizationMapCapabilitiesKHR::`maxQuantizationMapExtent.width`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified in the `pProfiles` member of the VkVideoProfileListInfoKHR structure included in the `pNext` chain
- VUID-VkImageCreateInfo-usage-10258
  If `usage` includes `VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR` or `VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR`, `extent.height` **must** be less than or equal to VkVideoEncodeQuantizationMapCapabilitiesKHR::`maxQuantizationMapExtent.height`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified in the `pProfiles` member of the VkVideoProfileListInfoKHR structure included in the `pNext` chain
- VUID-VkImageCreateInfo-pNext-06783
  If the `pNext` chain includes a VkExportMetalObjectCreateInfoEXT structure, its

exportObjectType member **must** be either VK_EXPORT_METAL_OBJECT_TYPE_METAL_TEXTURE_BIT_EXT or VK_EXPORT_METAL_OBJECT_TYPE_METAL_IOSURFACE_BIT_EXT

- VUID-VkImageCreateInfo-pNext-06784
  If the pNext chain includes a VkImportMetalTextureInfoEXT structure its plane member **must** be VK_IMAGE_ASPECT_PLANE_0_BIT, VK_IMAGE_ASPECT_PLANE_1_BIT, or VK_IMAGE_ASPECT_PLANE_2_BIT
- VUID-VkImageCreateInfo-pNext-06785
  If the pNext chain includes a VkImportMetalTextureInfoEXT structure and the image does not have a multi-planar format, then VkImportMetalTextureInfoEXT::plane **must** be VK_IMAGE_ASPECT_PLANE_0_BIT
- VUID-VkImageCreateInfo-pNext-06786
  If the pNext chain includes a VkImportMetalTextureInfoEXT structure and the image has a multi-planar format with only two planes, then VkImportMetalTextureInfoEXT::plane **must** not be VK_IMAGE_ASPECT_PLANE_2_BIT
- VUID-VkImageCreateInfo-imageCreateFormatFeatures-09048
  If imageCreateFormatFeatures (as defined in Image Creation Limits) does not contain VK_FORMAT_FEATURE_2_HOST_IMAGE_TRANSFER_BIT_EXT, then usage **must** not contain VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT
- VUID-VkImageCreateInfo-usage-10245
  If usage includes VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT, then the hostImageCopy feature **must** be enabled

Valid Usage (Implicit)

- VUID-VkImageCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO
- VUID-VkImageCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkExportMetalObjectCreateInfoEXT, VkExternalMemoryImageCreateInfo, VkImageDrmFormatModifierExplicitCreateInfoEXT, VkImageDrmFormatModifierListCreateInfoEXT, VkImageFormatListCreateInfo, VkImageSwapchainCreateInfoKHR, VkImportMetalIOSurfaceInfoEXT, VkImportMetalTextureInfoEXT, or VkVideoProfileListInfoKHR
- VUID-VkImageCreateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique, with the exception of structures of type VkExportMetalObjectCreateInfoEXT or VkImportMetalTextureInfoEXT
- VUID-VkImageCreateInfo-flags-parameter
  flags **must** be a valid combination of VkImageCreateFlagBits values
- VUID-VkImageCreateInfo-imageType-parameter
  imageType **must** be a valid VkImageType value
- VUID-VkImageCreateInfo-format-parameter
  format **must** be a valid VkFormat value

- VUID-VkImageCreateInfo-samples-parameter
  `samples` **must** be a valid VkSampleCountFlagBits value
- VUID-VkImageCreateInfo-tiling-parameter
  `tiling` **must** be a valid VkImageTiling value
- VUID-VkImageCreateInfo-usage-parameter
  `usage` **must** be a valid combination of VkImageUsageFlagBits values
- VUID-VkImageCreateInfo-usage-requiredbitmask
  `usage` **must** not be `0`
- VUID-VkImageCreateInfo-sharingMode-parameter
  `sharingMode` **must** be a valid VkSharingMode value
- VUID-VkImageCreateInfo-initialLayout-parameter
  `initialLayout` **must** be a valid VkImageLayout value

To define a set of external memory handle types that **may** be used as backing store for an image, add a VkExternalMemoryImageCreateInfo structure to the `pNext` chain of the VkImageCreateInfo structure. The `VkExternalMemoryImageCreateInfo` structure is defined as:

```
typedef struct VkExternalMemoryImageCreateInfo {
    VkStructureType                   sType;
    const void*                       pNext;
    VkExternalMemoryHandleTypeFlags   handleTypes;
} VkExternalMemoryImageCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_memory
typedef VkExternalMemoryImageCreateInfo VkExternalMemoryImageCreateInfoKHR;
```

NOTE A `VkExternalMemoryImageCreateInfo` structure with a non-zero `handleTypes` field must be included in the creation parameters for an image that will be bound to memory that is either exported or imported.

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleTypes` is zero or a bitmask of VkExternalMemoryHandleTypeFlagBits specifying one or more external memory handle types.

Valid Usage (Implicit)

- VUID-VkExternalMemoryImageCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_IMAGE_CREATE_INFO`
- VUID-VkExternalMemoryImageCreateInfo-handleTypes-parameter

handleTypes **must** be a valid combination of VkExternalMemoryHandleTypeFlagBits values

If the pNext chain of VkImageCreateInfo includes a VkImageSwapchainCreateInfoKHR structure, then that structure includes a swapchain handle indicating that the image will be bound to memory from that swapchain.

The VkImageSwapchainCreateInfoKHR structure is defined as:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
typedef struct VkImageSwapchainCreateInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkSwapchainKHR     swapchain;
} VkImageSwapchainCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- swapchain is VK_NULL_HANDLE or a handle of a swapchain that the image will be bound to.

Valid Usage

- VUID-VkImageSwapchainCreateInfoKHR-swapchain-00995
  If swapchain is not VK_NULL_HANDLE, the fields of VkImageCreateInfo **must** match the implied image creation parameters of the swapchain

Valid Usage (Implicit)

- VUID-VkImageSwapchainCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMAGE_SWAPCHAIN_CREATE_INFO_KHR
- VUID-VkImageSwapchainCreateInfoKHR-swapchain-parameter
  If swapchain is not VK_NULL_HANDLE, swapchain **must** be a valid VkSwapchainKHR handle

If the pNext chain of VkImageCreateInfo includes a VkImageFormatListCreateInfo structure, then that structure contains a list of all formats that **can** be used when creating views of this image.

The VkImageFormatListCreateInfo structure is defined as:

```
typedef struct VkImageFormatListCreateInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           viewFormatCount;
    const VkFormat*    pViewFormats;
} VkImageFormatListCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_image_format_list
typedef VkImageFormatListCreateInfo VkImageFormatListCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `viewFormatCount` is the number of entries in the `pViewFormats` array.
- `pViewFormats` is a pointer to an array of VkFormat values specifying all formats which **can** be used when creating views of this image.

If `viewFormatCount` is zero, `pViewFormats` is ignored and the image is created as if the `VkImageFormatListCreateInfo` structure were not included in the `pNext` chain of VkImageCreateInfo.

Valid Usage

- VUID-VkImageFormatListCreateInfo-viewFormatCount-09540
  If `viewFormatCount` is not 0, each element of `pViewFormats` **must** not be `VK_FORMAT_UNDEFINED`

Valid Usage (Implicit)

- VUID-VkImageFormatListCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_FORMAT_LIST_CREATE_INFO`
- VUID-VkImageFormatListCreateInfo-pViewFormats-parameter
  If `viewFormatCount` is not `0`, `pViewFormats` **must** be a valid pointer to an array of `viewFormatCount` valid VkFormat values

If the `pNext` chain of VkImageCreateInfo includes a VkImageDrmFormatModifierListCreateInfoEXT structure, then the image will be created with one of the Linux DRM format modifiers listed in the structure. The choice of modifier is implementation-dependent.

The VkImageDrmFormatModifierListCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_image_drm_format_modifier
typedef struct VkImageDrmFormatModifierListCreateInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           drmFormatModifierCount;
    const uint64_t*    pDrmFormatModifiers;
} VkImageDrmFormatModifierListCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `drmFormatModifierCount` is the length of the `pDrmFormatModifiers` array.
- `pDrmFormatModifiers` is a pointer to an array of *Linux DRM format modifiers*.

Valid Usage

- VUID-VkImageDrmFormatModifierListCreateInfoEXT-pDrmFormatModifiers-02263
  Each *modifier* in `pDrmFormatModifiers` **must** be compatible with the parameters in VkImageCreateInfo and its `pNext` chain, as determined by querying VkPhysicalDeviceImageFormatInfo2 extended with VkPhysicalDeviceImageDrmFormatModifierInfoEXT

Valid Usage (Implicit)

- VUID-VkImageDrmFormatModifierListCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_LIST_CREATE_INFO_EXT`
- VUID-VkImageDrmFormatModifierListCreateInfoEXT-pDrmFormatModifiers-parameter
  `pDrmFormatModifiers` **must** be a valid pointer to an array of `drmFormatModifierCount` `uint64_t` values
- VUID-VkImageDrmFormatModifierListCreateInfoEXT-drmFormatModifierCount-arraylength
  `drmFormatModifierCount` **must** be greater than `0`

If the `pNext` chain of VkImageCreateInfo includes a VkImageDrmFormatModifierExplicitCreateInfoEXT structure, then the image will be created with the Linux DRM format modifier and memory layout defined by the structure.

The VkImageDrmFormatModifierExplicitCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_image_drm_format_modifier
typedef struct VkImageDrmFormatModifierExplicitCreateInfoEXT {
    VkStructureType                sType;
    const void*                    pNext;
    uint64_t                       drmFormatModifier;
    uint32_t                       drmFormatModifierPlaneCount;
    const VkSubresourceLayout*     pPlaneLayouts;
} VkImageDrmFormatModifierExplicitCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `drmFormatModifier` is the *Linux DRM format modifier* with which the image will be created.
- `drmFormatModifierPlaneCount` is the number of *memory planes* in the image (as reported by VkDrmFormatModifierPropertiesEXT) as well as the length of the `pPlaneLayouts` array.
- `pPlaneLayouts` is a pointer to an array of VkSubresourceLayout structures describing the image's *memory planes*.

The i^{th} member of `pPlaneLayouts` describes the layout of the image's i^{th} *memory plane* (that is, `VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT`). In each element of `pPlaneLayouts`, the implementation **must** ignore `size`. The implementation calculates the size of each plane, which the application **can** query with vkGetImageSubresourceLayout.

When creating an image with VkImageDrmFormatModifierExplicitCreateInfoEXT, it is the application's responsibility to satisfy all valid usage requirements. However, the implementation **must** validate that the provided `pPlaneLayouts`, when combined with the provided `drmFormatModifier` and other creation parameters in VkImageCreateInfo and its `pNext` chain, produce a valid image. (This validation is necessarily implementation-dependent and outside the scope of Vulkan, and therefore not described by valid usage requirements). If this validation fails, then vkCreateImage returns `VK_ERROR_INVALID_DRM_FORMAT_MODIFIER_PLANE_LAYOUT_EXT`.

Valid Usage

- VUID-VkImageDrmFormatModifierExplicitCreateInfoEXT-drmFormatModifier-02264
  `drmFormatModifier` **must** be compatible with the parameters in VkImageCreateInfo and its `pNext` chain, as determined by querying VkPhysicalDeviceImageFormatInfo2 extended with VkPhysicalDeviceImageDrmFormatModifierInfoEXT
- VUID-VkImageDrmFormatModifierExplicitCreateInfoEXT-drmFormatModifierPlaneCount-02265
  `drmFormatModifierPlaneCount` **must** be equal to the VkDrmFormatModifierPropertiesEXT::`drmFormatModifierPlaneCount` associated with VkImageCreateInfo::`format` and `drmFormatModifier`, as found by querying VkDrmFormatModifierPropertiesListEXT
- VUID-VkImageDrmFormatModifierExplicitCreateInfoEXT-size-02267
  For each element of `pPlaneLayouts`, `size` **must** be 0
- VUID-VkImageDrmFormatModifierExplicitCreateInfoEXT-arrayPitch-02268

For each element of pPlaneLayouts, arrayPitch **must** be 0 if VkImageCreateInfo ::arrayLayers is 1

- VUID-VkImageDrmFormatModifierExplicitCreateInfoEXT-depthPitch-02269
  For each element of pPlaneLayouts, depthPitch **must** be 0 if VkImageCreateInfo ::extent.depth is 1

Valid Usage (Implicit)

- VUID-VkImageDrmFormatModifierExplicitCreateInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_EXPLICIT_CREATE_INFO_EXT
- VUID-VkImageDrmFormatModifierExplicitCreateInfoEXT-pPlaneLayouts-parameter
  pPlaneLayouts **must** be a valid pointer to an array of drmFormatModifierPlaneCount VkSubresourceLayout structures
- VUID-VkImageDrmFormatModifierExplicitCreateInfoEXT-drmFormatModifierPlaneCount-arraylength
  drmFormatModifierPlaneCount **must** be greater than 0

Bits which **can** be set in

- VkImageViewUsageCreateInfo::usage
- VkImageCreateInfo::usage

specify intended usage of an image, and are:

```
// Provided by VK_VERSION_1_0
typedef enum VkImageUsageFlagBits {
    VK_IMAGE_USAGE_TRANSFER_SRC_BIT = 0x00000001,
    VK_IMAGE_USAGE_TRANSFER_DST_BIT = 0x00000002,
    VK_IMAGE_USAGE_SAMPLED_BIT = 0x00000004,
    VK_IMAGE_USAGE_STORAGE_BIT = 0x00000008,
    VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT = 0x00000010,
    VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT = 0x00000020,
    VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT = 0x00000040,
    VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT = 0x00000080,
  // Provided by VK_KHR_video_decode_queue
    VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR = 0x00000400,
  // Provided by VK_KHR_video_decode_queue
    VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR = 0x00000800,
  // Provided by VK_KHR_video_decode_queue
    VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR = 0x00001000,
  // Provided by VK_EXT_fragment_density_map
    VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT = 0x00000200,
  // Provided by VK_KHR_fragment_shading_rate
    VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR = 0x00000100,
  // Provided by VK_EXT_host_image_copy
    VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT = 0x00400000,
  // Provided by VK_KHR_video_encode_queue
    VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR = 0x00002000,
  // Provided by VK_KHR_video_encode_queue
    VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR = 0x00004000,
  // Provided by VK_KHR_video_encode_queue
    VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR = 0x00008000,
  // Provided by VK_EXT_attachment_feedback_loop_layout
    VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT = 0x00080000,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR = 0x02000000,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR = 0x04000000,
} VkImageUsageFlagBits;
```

- VK_IMAGE_USAGE_TRANSFER_SRC_BIT specifies that the image **can** be used as the source of a transfer command.
- VK_IMAGE_USAGE_TRANSFER_DST_BIT specifies that the image **can** be used as the destination of a transfer command.
- VK_IMAGE_USAGE_SAMPLED_BIT specifies that the image **can** be used to create a VkImageView suitable for occupying a VkDescriptorSet slot either of type VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, and be sampled by a shader.
- VK_IMAGE_USAGE_STORAGE_BIT specifies that the image **can** be used to create a VkImageView suitable for occupying a VkDescriptorSet slot of type VK_DESCRIPTOR_TYPE_STORAGE_IMAGE.
- VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT specifies that the image **can** be used to create a VkImageView suitable for use as a color or resolve attachment in a VkFramebuffer.

- VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT specifies that the image **can** be used to create a VkImageView suitable for use as a depth/stencil or depth/stencil resolve attachment in a VkFramebuffer.
- VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT specifies that implementations **may** support using memory allocations with the VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT to back an image with this usage. This bit **can** be set for any image that **can** be used to create a VkImageView suitable for use as a color, resolve, depth/stencil, or input attachment.
- VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT specifies that the image **can** be used to create a VkImageView suitable for occupying VkDescriptorSet slot of type VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT; be read from a shader as an input attachment; and be used as an input attachment in a framebuffer.
- VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT specifies that the image **can** be used to create a VkImageView suitable for use as a fragment density map image.
- VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR specifies that the image **can** be used to create a VkImageView suitable for use as a fragment shading rate attachment
- VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR specifies that the image **can** be used as a decode output picture in a video decode operation.
- VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR is reserved for future use.
- VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR specifies that the image **can** be used as an output reconstructed picture or an input reference picture in a video decode operation.
- VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR is reserved for future use.
- VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR specifies that the image **can** be used as an encode input picture in a video encode operation.
- VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR specifies that the image **can** be used as an output reconstructed picture or an input reference picture in a video encode operation.
- VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT specifies that the image **can** be transitioned to the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT layout to be used as a color or depth/stencil attachment in a VkFramebuffer and/or as a read-only input resource in a shader (sampled image, combined image sampler or input attachment) in the same render pass.
- VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT specifies that the image **can** be used with host copy commands and host layout transitions.
- VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR specifies that the image **can** be used as a quantization delta map in a video encode operation.
- VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR specifies that the image **can** be used as an emphasis map in a video encode operation.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkImageUsageFlags;
```

VkImageUsageFlags is a bitmask type for setting a mask of zero or more VkImageUsageFlagBits.

When creating a VkImageView one of the following VkImageUsageFlagBits **must** be set:

- VK_IMAGE_USAGE_SAMPLED_BIT
- VK_IMAGE_USAGE_STORAGE_BIT
- VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT
- VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
- VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT
- VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT
- VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT
- VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR

Bits which **can** be set in VkImageCreateInfo::`flags`, specifying additional parameters of an image, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkImageCreateFlagBits {
    VK_IMAGE_CREATE_SPARSE_BINDING_BIT = 0x00000001,
    VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT = 0x00000002,
    VK_IMAGE_CREATE_SPARSE_ALIASED_BIT = 0x00000004,
    VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT = 0x00000008,
    VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT = 0x00000010,
    VK_IMAGE_CREATE_ALIAS_BIT = 0x00000400,
    VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT = 0x00000040,
    VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT = 0x00000020,
    VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT = 0x00000080,
    VK_IMAGE_CREATE_EXTENDED_USAGE_BIT = 0x00000100,
    VK_IMAGE_CREATE_DISJOINT_BIT = 0x00000200,
  // Provided by VK_EXT_sample_locations
    VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT = 0x00001000,
  // Provided by VK_EXT_fragment_density_map
    VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT = 0x00004000,
  // Provided by VK_EXT_image_2d_view_of_3d
    VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT = 0x00020000,
  // Provided by VK_KHR_video_maintenance1
    VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR = 0x00100000,
  // Provided by VK_KHR_bind_memory2 with VK_KHR_device_group
    VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR =
VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT,
  // Provided by VK_KHR_maintenance1
    VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT_KHR =
VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT,
  // Provided by VK_KHR_maintenance2
    VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT_KHR =
VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT,
  // Provided by VK_KHR_maintenance2
    VK_IMAGE_CREATE_EXTENDED_USAGE_BIT_KHR = VK_IMAGE_CREATE_EXTENDED_USAGE_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_IMAGE_CREATE_DISJOINT_BIT_KHR = VK_IMAGE_CREATE_DISJOINT_BIT,
  // Provided by VK_KHR_bind_memory2
    VK_IMAGE_CREATE_ALIAS_BIT_KHR = VK_IMAGE_CREATE_ALIAS_BIT,
} VkImageCreateFlagBits;
```

- VK_IMAGE_CREATE_SPARSE_BINDING_BIT specifies that the image will be backed using sparse memory binding.
- VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT specifies that the image **can** be partially backed using sparse memory binding. Images created with this flag **must** also be created with the VK_IMAGE_CREATE_SPARSE_BINDING_BIT flag.
- VK_IMAGE_CREATE_SPARSE_ALIASED_BIT specifies that the image will be backed using sparse memory binding with memory ranges that might also simultaneously be backing another image (or another portion of the same image). Images created with this flag **must** also be created with the VK_IMAGE_CREATE_SPARSE_BINDING_BIT flag.
- VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT specifies that the image **can** be used to create a VkImageView

with a different format from the image. For multi-planar formats, VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT specifies that a VkImageView can be created of a *plane* of the image.

- VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT specifies that the image **can** be used to create a VkImageView of type VK_IMAGE_VIEW_TYPE_CUBE or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY.
- VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT specifies that the image **can** be used to create a VkImageView of type VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY.
- VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT specifies that the image **can** be used to create a VkImageView of type VK_IMAGE_VIEW_TYPE_2D.
- VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT specifies that the image **can** be used with a non-zero value of the splitInstanceBindRegionCount member of a VkBindImageMemoryDeviceGroupInfo structure passed into vkBindImageMemory2KHR. This flag also has the effect of making the image use the standard sparse image block dimensions.
- VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT specifies that the image having a compressed format **can** be used to create a VkImageView with an uncompressed format where each texel in the image view corresponds to a compressed texel block of the image.
- VK_IMAGE_CREATE_EXTENDED_USAGE_BIT specifies that the image **can** be created with usage flags that are not supported for the format the image is created with but are supported for at least one format a VkImageView created from the image **can** have.
- VK_IMAGE_CREATE_DISJOINT_BIT specifies that an image with a multi-planar format **must** have each plane separately bound to memory, rather than having a single memory binding for the whole image; the presence of this bit distinguishes a *disjoint image* from an image without this bit set.
- VK_IMAGE_CREATE_ALIAS_BIT specifies that two images created with the same creation parameters and aliased to the same memory **can** interpret the contents of the memory consistently with each other, subject to the rules described in the Memory Aliasing section. This flag further specifies that each plane of a *disjoint* image **can** share an in-memory non-linear representation with single-plane images, and that a single-plane image **can** share an in-memory non-linear representation with a plane of a multi-planar disjoint image, according to the rules in Compatible Formats of Planes of Multi-Planar Formats. If the pNext chain includes a VkExternalMemoryImageCreateInfo structure whose handleTypes member is not 0, it is as if VK_IMAGE_CREATE_ALIAS_BIT is set.
- VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT specifies that an image with a depth or depth/stencil format **can** be used with custom sample locations when used as a depth/stencil attachment.
- VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT specifies that an image **can** be in a subsampled format which **may** be more optimal when written as an attachment by a render pass that has a fragment density map attachment. Accessing a subsampled image has additional considerations:
  - Image data read as an image sampler will have undefined values if the sampler was not created with flags containing VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT or was not sampled through the use of a combined image sampler with an immutable sampler in VkDescriptorSetLayoutBinding.

- Image data read with an input attachment will have undefined values if the contents were not written as an attachment in an earlier subpass of the same render pass.
- Image data read as an image sampler in the fragment shader will be additionally be read by the device during VK_PIPELINE_STAGE_VERTEX_SHADER_BIT if VkPhysicalDeviceFragmentDensityMap2PropertiesEXT::subsampledCoarseReconstructionEarlyAccess is VK_TRUE and the sampler was created with flags containing VK_SAMPLER_CREATE_SUBSAMPLED_COARSE_RECONSTRUCTION_BIT_EXT.
- Image data read with load operations are resampled to the fragment density of the render pass if VkPhysicalDeviceFragmentDensityMap2PropertiesEXT::subsampledLoads is VK_TRUE. Otherwise, values of image data are undefined.
- Image contents outside of the render area take on undefined values if the image is stored as a render pass attachment.

- VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR specifies that the image **can** be used in video coding operations without having to specify at image creation time the set of video profiles the image will be used with, except for images used only as DPB pictures, as long as the image is otherwise compatible with the video profile in question.

  NOTE

  This enables exchanging video picture data without additional copies or conversions when used as:

  - Decode output pictures, indifferent of the video profile used to produce them.
  - Encode input pictures, indifferent of the video profile used to consume them.

  This includes images created with both VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR and VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR, which is necessary to use the same video picture as the reconstructed picture and decode output picture in a video decode operation on implementations supporting VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR.

  However, images with only DPB usage remain tied to the video profiles the image was created with, as the data layout of such DPB-only images **may** be implementation- and codec-dependent.

  If an application would like to share or reuse the device memory backing such images (e.g. for the purposes of temporal aliasing), then it **should** create separate image objects for each video profile and bind them to the same underlying device memory range, similar to how memory resources **can** be shared across separate video sessions or any other memory-backed resource.

See Sparse Resource Features and Sparse Physical Device Features for more details.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkImageCreateFlags;
```

VkImageCreateFlags is a bitmask type for setting a mask of zero or more VkImageCreateFlagBits.

Possible values of VkImageCreateInfo::imageType, specifying the basic dimensionality of an image, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkImageType {
    VK_IMAGE_TYPE_1D = 0,
    VK_IMAGE_TYPE_2D = 1,
    VK_IMAGE_TYPE_3D = 2,
} VkImageType;
```

- VK_IMAGE_TYPE_1D specifies a one-dimensional image.
- VK_IMAGE_TYPE_2D specifies a two-dimensional image.
- VK_IMAGE_TYPE_3D specifies a three-dimensional image.

Possible values of VkImageCreateInfo::tiling, specifying the tiling arrangement of texel blocks in an image, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkImageTiling {
    VK_IMAGE_TILING_OPTIMAL = 0,
    VK_IMAGE_TILING_LINEAR = 1,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT = 1000158000,
} VkImageTiling;
```

- VK_IMAGE_TILING_OPTIMAL specifies optimal tiling (texels are laid out in an implementation-dependent arrangement, for more efficient memory access).
- VK_IMAGE_TILING_LINEAR specifies linear tiling (texels are laid out in memory in row-major order, possibly with some padding on each row).
- VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT specifies that the image's tiling is defined by a Linux DRM format modifier. The modifier is specified at image creation with VkImageDrmFormatModifierListCreateInfoEXT or VkImageDrmFormatModifierExplicitCreateInfoEXT, and **can** be queried with vkGetImageDrmFormatModifierPropertiesEXT.

To query the memory layout of an image subresource, call:

```
// Provided by VK_VERSION_1_0
void vkGetImageSubresourceLayout(
    VkDevice                                    device,
    VkImage                                     image,
    const VkImageSubresource*                   pSubresource,
    VkSubresourceLayout*                        pLayout);
```

- device is the logical device that owns the image.

- `image` is the image whose layout is being queried.
- `pSubresource` is a pointer to a VkImageSubresource structure selecting a specific image subresource from the image.
- `pLayout` is a pointer to a VkSubresourceLayout structure in which the layout is returned.

If the image is linear, then the returned layout is valid for host access.

If the image's tiling is `VK_IMAGE_TILING_LINEAR` and its format is a multi-planar format, then `vkGetImageSubresourceLayout` describes one *format plane* of the image. If the image's tiling is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`, then `vkGetImageSubresourceLayout` describes one *memory plane* of the image. If the image's tiling is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT` and the image is non-linear, then the returned layout has an implementation-dependent meaning; the vendor of the image's DRM format modifier **may** provide documentation that explains how to interpret the returned layout.

`vkGetImageSubresourceLayout` is invariant for the lifetime of a single image.

Valid Usage

- VUID-vkGetImageSubresourceLayout-image-07790
  `image` **must** have been created with `tiling` equal to `VK_IMAGE_TILING_LINEAR` or `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`
- VUID-vkGetImageSubresourceLayout-aspectMask-00997
  The `aspectMask` member of `pSubresource` **must** only have a single bit set
- VUID-vkGetImageSubresourceLayout-mipLevel-01716
  The `mipLevel` member of `pSubresource` **must** be less than the `mipLevels` specified in `image`
- VUID-vkGetImageSubresourceLayout-arrayLayer-01717
  The `arrayLayer` member of `pSubresource` **must** be less than the `arrayLayers` specified in `image`
- VUID-vkGetImageSubresourceLayout-format-08886
  If `format` of the `image` is a color format that is not a multi-planar image format, and `tiling` of the `image` is `VK_IMAGE_TILING_LINEAR` or `VK_IMAGE_TILING_OPTIMAL`, the `aspectMask` member of `pSubresource` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-vkGetImageSubresourceLayout-format-04462
  If `format` of the `image` has a depth component, the `aspectMask` member of `pSubresource` **must** contain `VK_IMAGE_ASPECT_DEPTH_BIT`
- VUID-vkGetImageSubresourceLayout-format-04463
  If `format` of the `image` has a stencil component, the `aspectMask` member of `pSubresource` **must** contain `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-vkGetImageSubresourceLayout-format-04464
  If `format` of the `image` does not contain a stencil or depth component, the `aspectMask` member of `pSubresource` **must** not contain `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-vkGetImageSubresourceLayout-tiling-08717

If the `tiling` of the `image` is `VK_IMAGE_TILING_LINEAR` and has a multi-planar image format, then the `aspectMask` member of `pSubresource` **must** be a single valid multi-planar aspect mask bit

- VUID-vkGetImageSubresourceLayout-tiling-09433
  If the `tiling` of the `image` is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`, then the `aspectMask` member of `pSubresource` **must** be `VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT` and the index *i* **must** be less than the VkDrmFormatModifierPropertiesEXT::`drmFormatModifierPlaneCount` associated with the image's `format` and VkImageDrmFormatModifierPropertiesEXT::`drmFormatModifier`

Valid Usage (Implicit)

- VUID-vkGetImageSubresourceLayout-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetImageSubresourceLayout-image-parameter
  `image` **must** be a valid VkImage handle
- VUID-vkGetImageSubresourceLayout-pSubresource-parameter
  `pSubresource` **must** be a valid pointer to a valid VkImageSubresource structure
- VUID-vkGetImageSubresourceLayout-pLayout-parameter
  `pLayout` **must** be a valid pointer to a VkSubresourceLayout structure
- VUID-vkGetImageSubresourceLayout-image-parent
  `image` **must** have been created, allocated, or retrieved from `device`

The `VkImageSubresource` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageSubresource {
    VkImageAspectFlags    aspectMask;
    uint32_t              mipLevel;
    uint32_t              arrayLayer;
} VkImageSubresource;
```

- `aspectMask` is a VkImageAspectFlags value selecting the image *aspect*.
- `mipLevel` selects the mipmap level.
- `arrayLayer` selects the array layer.

Valid Usage (Implicit)

- VUID-VkImageSubresource-aspectMask-parameter
  `aspectMask` **must** be a valid combination of VkImageAspectFlagBits values
- VUID-VkImageSubresource-aspectMask-requiredbitmask
  `aspectMask` **must** not be `0`

Information about the layout of the image subresource is returned in a VkSubresourceLayout structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkSubresourceLayout {
    VkDeviceSize    offset;
    VkDeviceSize    size;
    VkDeviceSize    rowPitch;
    VkDeviceSize    arrayPitch;
    VkDeviceSize    depthPitch;
} VkSubresourceLayout;
```

- offset is the byte offset from the start of the image or the plane where the image subresource begins.
- size is the size in bytes of the image subresource. size includes any extra memory that is required based on rowPitch.
- rowPitch describes the number of bytes between each row of texels in an image.
- arrayPitch describes the number of bytes between each array layer of an image.
- depthPitch describes the number of bytes between each slice of 3D image.

If the image is linear, then rowPitch, arrayPitch and depthPitch describe the layout of the image subresource in linear memory. For uncompressed formats, rowPitch is the number of bytes between texels with the same x coordinate in adjacent rows (y coordinates differ by one). arrayPitch is the number of bytes between texels with the same x and y coordinate in adjacent array layers of the image (array layer values differ by one). depthPitch is the number of bytes between texels with the same x and y coordinate in adjacent slices of a 3D image (z coordinates differ by one). Expressed as an addressing formula, the starting byte of a texel in the image subresource has address:

```
// (x,y,z,layer) are in texel coordinates
address(x,y,z,layer) = layer*arrayPitch + z*depthPitch + y*rowPitch + x*elementSize +
offset
```

For compressed formats, the rowPitch is the number of bytes between compressed texel blocks in adjacent rows. arrayPitch is the number of bytes between compressed texel blocks in adjacent array layers. depthPitch is the number of bytes between compressed texel blocks in adjacent slices of a 3D image.

```
// (x,y,z,layer) are in compressed texel block coordinates
address(x,y,z,layer) = layer*arrayPitch + z*depthPitch + y*rowPitch + x
*compressedTexelBlockByteSize + offset;
```

The value of arrayPitch is undefined for images that were not created as arrays. depthPitch is defined only for 3D images.

If the image has a *single-plane* color format and its tiling is VK_IMAGE_TILING_LINEAR , then the

aspectMask member of VkImageSubresource **must** be VK_IMAGE_ASPECT_COLOR_BIT.

If the image has a depth/stencil format and its tiling is VK_IMAGE_TILING_LINEAR , then aspectMask **must** be either VK_IMAGE_ASPECT_DEPTH_BIT or VK_IMAGE_ASPECT_STENCIL_BIT. On implementations that store depth and stencil aspects separately, querying each of these image subresource layouts will return a different offset and size representing the region of memory used for that aspect. On implementations that store depth and stencil aspects interleaved, the same offset and size are returned and represent the interleaved memory allocation.

If the image has a multi-planar format and its tiling is VK_IMAGE_TILING_LINEAR , then the aspectMask member of VkImageSubresource **must** be VK_IMAGE_ASPECT_PLANE_0_BIT, VK_IMAGE_ASPECT_PLANE_1_BIT, or (for 3-plane formats only) VK_IMAGE_ASPECT_PLANE_2_BIT. Querying each of these image subresource layouts will return a different offset and size representing the region of memory used for that plane. If the image is *disjoint*, then the offset is relative to the base address of the plane. If the image is *non-disjoint*, then the offset is relative to the base address of the image.

If the image's tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then the aspectMask member of VkImageSubresource **must** be one of VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT, where the maximum allowed plane index *i* is defined by the VkDrmFormatModifierPropertiesEXT ::drmFormatModifierPlaneCount associated with the image's VkImageCreateInfo::format and modifier. The memory range used by the subresource is described by offset and size. If the image is *disjoint*, then the offset is relative to the base address of the *memory plane*. If the image is *non-disjoint*, then the offset is relative to the base address of the image. If the image is non-linear, then rowPitch, arrayPitch, and depthPitch have an implementation-dependent meaning.

To query the memory layout of an image subresource, call:

```
// Provided by VK_KHR_maintenance5
void vkGetImageSubresourceLayout2KHR(
    VkDevice                                    device,
    VkImage                                     image,
    const VkImageSubresource2KHR*               pSubresource,
    VkSubresourceLayout2KHR*                    pLayout);
```

or the equivalent command

```
// Provided by VK_EXT_host_image_copy
void vkGetImageSubresourceLayout2EXT(
    VkDevice                                    device,
    VkImage                                     image,
    const VkImageSubresource2KHR*               pSubresource,
    VkSubresourceLayout2KHR*                    pLayout);
```

- device is the logical device that owns the image.
- image is the image whose layout is being queried.
- pSubresource is a pointer to a VkImageSubresource2KHR structure selecting a specific image for the image subresource.

- `pLayout` is a pointer to a VkSubresourceLayout2KHR structure in which the layout is returned.

`vkGetImageSubresourceLayout2KHR` behaves similarly to vkGetImageSubresourceLayout, with the ability to specify extended inputs via chained input structures, and to return extended information via chained output structures.

It is legal to call `vkGetImageSubresourceLayout2KHR` with an `image` created with `tiling` equal to `VK_IMAGE_TILING_OPTIMAL`, but the members of VkSubresourceLayout2KHR::`subresourceLayout` will have undefined values in this case.

NOTE Structures chained from VkImageSubresource2KHR::`pNext` will also be updated when `tiling` is equal to `VK_IMAGE_TILING_OPTIMAL`.

Valid Usage

- VUID-vkGetImageSubresourceLayout2KHR-aspectMask-00997
  The `aspectMask` member of `pSubresource` **must** only have a single bit set
- VUID-vkGetImageSubresourceLayout2KHR-mipLevel-01716
  The `mipLevel` member of `pSubresource` **must** be less than the `mipLevels` specified in `image`
- VUID-vkGetImageSubresourceLayout2KHR-arrayLayer-01717
  The `arrayLayer` member of `pSubresource` **must** be less than the `arrayLayers` specified in `image`
- VUID-vkGetImageSubresourceLayout2KHR-format-08886
  If `format` of the `image` is a color format that is not a multi-planar image format, and `tiling` of the `image` is `VK_IMAGE_TILING_LINEAR` or `VK_IMAGE_TILING_OPTIMAL`, the `aspectMask` member of `pSubresource` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-vkGetImageSubresourceLayout2KHR-format-04462
  If `format` of the `image` has a depth component, the `aspectMask` member of `pSubresource` **must** contain `VK_IMAGE_ASPECT_DEPTH_BIT`
- VUID-vkGetImageSubresourceLayout2KHR-format-04463
  If `format` of the `image` has a stencil component, the `aspectMask` member of `pSubresource` **must** contain `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-vkGetImageSubresourceLayout2KHR-format-04464
  If `format` of the `image` does not contain a stencil or depth component, the `aspectMask` member of `pSubresource` **must** not contain `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-vkGetImageSubresourceLayout2KHR-tiling-08717
  If the `tiling` of the `image` is `VK_IMAGE_TILING_LINEAR` and has a multi-planar image format, then the `aspectMask` member of `pSubresource` **must** be a single valid multi-planar aspect mask bit
- VUID-vkGetImageSubresourceLayout2KHR-tiling-09435
  If the `tiling` of the `image` is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`, then the `aspectMask` member of `pSubresource` **must** be `VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT` and the index *i* **must** be less than the VkDrmFormatModifierPropertiesEXT::`drmFormatModifierPlaneCount`

associated with the image's `format` and VkImageDrmFormatModifierPropertiesEXT ::`drmFormatModifier`

Valid Usage (Implicit)

- VUID-vkGetImageSubresourceLayout2KHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetImageSubresourceLayout2KHR-image-parameter
  `image` **must** be a valid VkImage handle
- VUID-vkGetImageSubresourceLayout2KHR-pSubresource-parameter
  `pSubresource` **must** be a valid pointer to a valid VkImageSubresource2KHR structure
- VUID-vkGetImageSubresourceLayout2KHR-pLayout-parameter
  `pLayout` **must** be a valid pointer to a VkSubresourceLayout2KHR structure
- VUID-vkGetImageSubresourceLayout2KHR-image-parent
  `image` **must** have been created, allocated, or retrieved from `device`

The `VkImageSubresource2KHR` structure is defined as:

```
// Provided by VK_KHR_maintenance5
typedef struct VkImageSubresource2KHR {
    VkStructureType       sType;
    void*                 pNext;
    VkImageSubresource    imageSubresource;
} VkImageSubresource2KHR;
```

or the equivalent

```
// Provided by VK_EXT_host_image_copy
typedef VkImageSubresource2KHR VkImageSubresource2EXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `imageSubresource` is a VkImageSubresource structure.

Valid Usage (Implicit)

- VUID-VkImageSubresource2KHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_SUBRESOURCE_2_KHR`
- VUID-VkImageSubresource2KHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImageSubresource2KHR-imageSubresource-parameter

imageSubresource **must** be a valid VkImageSubresource structure

Information about the layout of the image subresource is returned in a VkSubresourceLayout2KHR structure:

```
// Provided by VK_KHR_maintenance5
typedef struct VkSubresourceLayout2KHR {
    VkStructureType         sType;
    void*                   pNext;
    VkSubresourceLayout     subresourceLayout;
} VkSubresourceLayout2KHR;
```

or the equivalent

```
// Provided by VK_EXT_host_image_copy
typedef VkSubresourceLayout2KHR VkSubresourceLayout2EXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- subresourceLayout is a VkSubresourceLayout structure.

Valid Usage (Implicit)

- VUID-VkSubresourceLayout2KHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SUBRESOURCE_LAYOUT_2_KHR
- VUID-VkSubresourceLayout2KHR-pNext-pNext
  pNext **must** be NULL or a pointer to a valid instance of VkSubresourceHostMemcpySizeEXT
- VUID-VkSubresourceLayout2KHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique

To query the memory size needed to copy to or from an image using vkCopyMemoryToImageEXT or vkCopyImageToMemoryEXT when the VK_HOST_IMAGE_COPY_MEMCPY_EXT flag is specified, add a VkSubresourceHostMemcpySizeEXT structure to the pNext chain of the VkSubresourceLayout2EXT structure in a call to vkGetImageSubresourceLayout2EXT.

The VkSubresourceHostMemcpySizeEXT structure is defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkSubresourceHostMemcpySizeEXT {
    VkStructureType    sType;
    void*              pNext;
    VkDeviceSize       size;
} VkSubresourceHostMemcpySizeEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `size` is the size in bytes of the image subresource.

Valid Usage (Implicit)

- VUID-VkSubresourceHostMemcpySizeEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SUBRESOURCE_HOST_MEMCPY_SIZE_EXT`

To query the memory layout of an image subresource, without an image object, call:

```
// Provided by VK_KHR_maintenance5
void vkGetDeviceImageSubresourceLayoutKHR(
    VkDevice                                    device,
    const VkDeviceImageSubresourceInfoKHR*      pInfo,
    VkSubresourceLayout2KHR*                    pLayout);
```

- `device` is the logical device that owns the image.
- `pInfo` is a pointer to a VkDeviceImageSubresourceInfoKHR structure containing parameters required for the subresource layout query.
- `pLayout` is a pointer to a VkSubresourceLayout2KHR structure in which the layout is returned.

`vkGetDeviceImageSubresourceLayoutKHR` behaves similarly to vkGetImageSubresourceLayout2KHR, but uses a VkImageCreateInfo structure to specify the image rather than a VkImage object.

Valid Usage (Implicit)

- VUID-vkGetDeviceImageSubresourceLayoutKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceImageSubresourceLayoutKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkDeviceImageSubresourceInfoKHR structure
- VUID-vkGetDeviceImageSubresourceLayoutKHR-pLayout-parameter
  `pLayout` **must** be a valid pointer to a VkSubresourceLayout2KHR structure

The `VkDeviceImageSubresourceInfoKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance5
typedef struct VkDeviceImageSubresourceInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    const VkImageCreateInfo*        pCreateInfo;
    const VkImageSubresource2KHR*   pSubresource;
} VkDeviceImageSubresourceInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pCreateInfo` is a pointer to a VkImageCreateInfo structure containing parameters affecting creation of the image to query.
- `pSubresource` pSubresource is a pointer to a VkImageSubresource2KHR structure selecting a specific image subresource for the query.

Valid Usage

- VUID-VkDeviceImageSubresourceInfoKHR-aspectMask-00997
  The `aspectMask` member of `pSubresource` **must** only have a single bit set
- VUID-VkDeviceImageSubresourceInfoKHR-mipLevel-01716
  The `mipLevel` member of `pSubresource` **must** be less than the `mipLevels` specified in `pCreateInfo`
- VUID-VkDeviceImageSubresourceInfoKHR-arrayLayer-01717
  The `arrayLayer` member of `pSubresource` **must** be less than the `arrayLayers` specified in `pCreateInfo`
- VUID-VkDeviceImageSubresourceInfoKHR-format-08886
  If `format` of the `image` is a color format that is not a multi-planar image format, and `tiling` of the `pCreateInfo` is `VK_IMAGE_TILING_LINEAR` or `VK_IMAGE_TILING_OPTIMAL`, the `aspectMask` member of `pSubresource` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-VkDeviceImageSubresourceInfoKHR-format-04462
  If `format` of the `pCreateInfo` has a depth component, the `aspectMask` member of `pSubresource` **must** contain `VK_IMAGE_ASPECT_DEPTH_BIT`
- VUID-VkDeviceImageSubresourceInfoKHR-format-04463
  If `format` of the `pCreateInfo` has a stencil component, the `aspectMask` member of `pSubresource` **must** contain `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-VkDeviceImageSubresourceInfoKHR-format-04464
  If `format` of the `pCreateInfo` does not contain a stencil or depth component, the `aspectMask` member of `pSubresource` **must** not contain `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-VkDeviceImageSubresourceInfoKHR-tiling-08717
  If the `tiling` of the `pCreateInfo` is `VK_IMAGE_TILING_LINEAR` and has a multi-planar image format, then the `aspectMask` member of `pSubresource` **must** be a single valid multi-planar aspect mask bit

Valid Usage (Implicit)

- VUID-VkDeviceImageSubresourceInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_IMAGE_SUBRESOURCE_INFO_KHR`
- VUID-VkDeviceImageSubresourceInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDeviceImageSubresourceInfoKHR-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkImageCreateInfo structure
- VUID-VkDeviceImageSubresourceInfoKHR-pSubresource-parameter
  `pSubresource` **must** be a valid pointer to a valid VkImageSubresource2KHR structure

If an image was created with `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`, then the image has a Linux DRM format modifier. To query the *modifier*, call:

```
// Provided by VK_EXT_image_drm_format_modifier
VkResult vkGetImageDrmFormatModifierPropertiesEXT(
    VkDevice                                    device,
    VkImage                                     image,
    VkImageDrmFormatModifierPropertiesEXT*      pProperties);
```

- `device` is the logical device that owns the image.
- `image` is the queried image.
- `pProperties` is a pointer to a VkImageDrmFormatModifierPropertiesEXT structure in which properties of the image's *DRM format modifier* are returned.

Valid Usage

- VUID-vkGetImageDrmFormatModifierPropertiesEXT-image-02272
  `image` **must** have been created with `tiling` equal to `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`

Valid Usage (Implicit)

- VUID-vkGetImageDrmFormatModifierPropertiesEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetImageDrmFormatModifierPropertiesEXT-image-parameter
  `image` **must** be a valid VkImage handle
- VUID-vkGetImageDrmFormatModifierPropertiesEXT-pProperties-parameter
  `pProperties` **must** be a valid pointer to a VkImageDrmFormatModifierPropertiesEXT structure
- VUID-vkGetImageDrmFormatModifierPropertiesEXT-image-parent

image **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY

The VkImageDrmFormatModifierPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_image_drm_format_modifier
typedef struct VkImageDrmFormatModifierPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint64_t           drmFormatModifier;
} VkImageDrmFormatModifierPropertiesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- drmFormatModifier returns the image's Linux DRM format modifier.

If the image was created with VkImageDrmFormatModifierListCreateInfoEXT, then the returned drmFormatModifier **must** belong to the list of modifiers provided at time of image creation in VkImageDrmFormatModifierListCreateInfoEXT::pDrmFormatModifiers. If the image was created with VkImageDrmFormatModifierExplicitCreateInfoEXT, then the returned drmFormatModifier **must** be the modifier provided at time of image creation in VkImageDrmFormatModifierExplicitCreateInfoEXT::drmFormatModifier.

Valid Usage (Implicit)

- VUID-VkImageDrmFormatModifierPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_PROPERTIES_EXT
- VUID-VkImageDrmFormatModifierPropertiesEXT-pNext-pNext
  pNext **must** be NULL

To destroy an image, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyImage(
    VkDevice                                    device,
    VkImage                                     image,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the image.
- `image` is the image to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyImage-image-01000
  All submitted commands that refer to `image`, either directly or via a `VkImageView`, **must** have completed execution
- VUID-vkDestroyImage-image-01001
  If `VkAllocationCallbacks` were provided when `image` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyImage-image-01002
  If no `VkAllocationCallbacks` were provided when `image` was created, `pAllocator` **must** be `NULL`
- VUID-vkDestroyImage-image-04882
  `image` **must** not have been acquired from vkGetSwapchainImagesKHR

Valid Usage (Implicit)

- VUID-vkDestroyImage-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyImage-image-parameter
  If `image` is not VK_NULL_HANDLE, `image` **must** be a valid VkImage handle
- VUID-vkDestroyImage-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyImage-image-parent
  If `image` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `image` **must** be externally synchronized

12.3.1. Image Format Features

Valid uses of a VkImage **may** depend on the image's *format features*, defined below. Such constraints are documented in the affected valid usage statement.

- If the image was created with VK_IMAGE_TILING_LINEAR, then its set of *format features* is the value of VkFormatProperties::linearTilingFeatures found by calling vkGetPhysicalDeviceFormatProperties on the same format as VkImageCreateInfo::format.
- If the image was created with VK_IMAGE_TILING_OPTIMAL, then its set of *format features* is the value of VkFormatProperties::optimalTilingFeatures found by calling vkGetPhysicalDeviceFormatProperties on the same format as VkImageCreateInfo::format.
- If the image was created with VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then:
  - The image's DRM format modifier is the value of VkImageDrmFormatModifierPropertiesEXT::drmFormatModifier found by calling vkGetImageDrmFormatModifierPropertiesEXT.
  - Let VkDrmFormatModifierPropertiesListEXT::pDrmFormatModifierProperties be the array found by calling vkGetPhysicalDeviceFormatProperties2KHR on the same format as VkImageCreateInfo::format.
  - Let VkDrmFormatModifierPropertiesEXT prop be the array element whose drmFormatModifier member is the value of the image's DRM format modifier.
  - Then the image's set of *format features* is the value of prop::drmFormatModifierTilingFeatures.

12.3.2. Image Mip Level Sizing

A *complete mipmap chain* is the full set of mip levels, from the largest mip level provided, down to the *minimum mip level size*.

Conventional Images

For conventional images, the dimensions of each successive mip level, n+1, are:

$$width_{n+1} = \max(\lfloor width_n/2 \rfloor, 1)$$

$$height_{n+1} = \max(\lfloor height_n/2 \rfloor, 1)$$

$$depth_{n+1} = \max(\lfloor depth_n/2 \rfloor, 1)$$

where $width_n$, $height_n$, and $depth_n$ are the dimensions of the next larger mip level, n.

The minimum mip level size is:

- 1 for one-dimensional images,
- 1x1 for two-dimensional images, and

- 1x1x1 for three-dimensional images.

The number of levels in a complete mipmap chain is:

$$\lfloor \log_2(\max(\texttt{width}_0, \texttt{height}_0, \texttt{depth}_0)) \rfloor + 1$$

where $\texttt{width}_0$, $\texttt{height}_0$, and $\texttt{depth}_0$ are the dimensions of the largest (most detailed) mip level, `0`.

12.4. Image Layouts

Images are stored in implementation-dependent opaque layouts in memory. Each layout has limitations on what kinds of operations are supported for image subresources using the layout. At any given time, the data representing an image subresource in memory exists in a particular layout which is determined by the most recent layout transition that was performed on that image subresource. Applications have control over which layout each image subresource uses, and **can** transition an image subresource from one layout to another. Transitions **can** happen with an image memory barrier, included as part of a vkCmdPipelineBarrier or a vkCmdWaitEvents command buffer command (see Image Memory Barriers), or as part of a subpass dependency within a render pass (see `VkSubpassDependency`).

Image layout is per-image subresource. Separate image subresources of the same image **can** be in different layouts at the same time, with the exception that depth and stencil aspects of a given image subresource **can** only be in different layouts if the `separateDepthStencilLayouts` feature is enabled.

NOTE

Each layout **may** offer optimal performance for a specific usage of image memory. For example, an image with a layout of `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` **may** provide optimal performance for use as a color attachment, but be unsupported for use in transfer commands. Applications **can** transition an image subresource from one layout to another in order to achieve optimal performance when the image subresource is used for multiple kinds of operations. After initialization, applications need not use any layout other than the general layout, though this **may** produce suboptimal performance on some implementations.

Upon creation, all image subresources of an image are initially in the same layout, where that layout is selected by the `VkImageCreateInfo`::`initialLayout` member. The `initialLayout` **must** be either `VK_IMAGE_LAYOUT_UNDEFINED` or `VK_IMAGE_LAYOUT_PREINITIALIZED`. If it is `VK_IMAGE_LAYOUT_PREINITIALIZED`, then the image data **can** be preinitialized by the host while using this layout, and the transition away from this layout will preserve that data. If it is `VK_IMAGE_LAYOUT_UNDEFINED`, then the contents of the data are considered to be undefined, and the transition away from this layout is not guaranteed to preserve that data. For either of these initial layouts, any image subresources **must** be transitioned to another layout before they are accessed by the device.

Host access to image memory is only well-defined for linear images and for image subresources of those images which are currently in either the `VK_IMAGE_LAYOUT_PREINITIALIZED` or `VK_IMAGE_LAYOUT_GENERAL` layout. Calling vkGetImageSubresourceLayout for a linear image returns a

subresource layout mapping that is valid for either of those image layouts.

The set of image layouts consists of:

```
// Provided by VK_VERSION_1_0
typedef enum VkImageLayout {
    VK_IMAGE_LAYOUT_UNDEFINED = 0,
    VK_IMAGE_LAYOUT_GENERAL = 1,
    VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL = 2,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL = 3,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL = 4,
    VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL = 5,
    VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL = 6,
    VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL = 7,
    VK_IMAGE_LAYOUT_PREINITIALIZED = 8,
    VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL = 1000117000,
    VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL = 1000117001,
    VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL = 1000241000,
    VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL = 1000241001,
    VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL = 1000241002,
    VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL = 1000241003,
    VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL = 1000314000,
    VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL = 1000314001,
  // Provided by VK_KHR_swapchain
    VK_IMAGE_LAYOUT_PRESENT_SRC_KHR = 1000001002,
  // Provided by VK_KHR_video_decode_queue
    VK_IMAGE_LAYOUT_VIDEO_DECODE_DST_KHR = 1000024000,
  // Provided by VK_KHR_video_decode_queue
    VK_IMAGE_LAYOUT_VIDEO_DECODE_SRC_KHR = 1000024001,
  // Provided by VK_KHR_video_decode_queue
    VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR = 1000024002,
  // Provided by VK_KHR_shared_presentable_image
    VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR = 1000111000,
  // Provided by VK_EXT_fragment_density_map
    VK_IMAGE_LAYOUT_FRAGMENT_DENSITY_MAP_OPTIMAL_EXT = 1000218000,
  // Provided by VK_KHR_fragment_shading_rate
    VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR = 1000164003,
  // Provided by VK_KHR_dynamic_rendering_local_read
    VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR = 1000232000,
  // Provided by VK_KHR_video_encode_queue
    VK_IMAGE_LAYOUT_VIDEO_ENCODE_DST_KHR = 1000299000,
  // Provided by VK_KHR_video_encode_queue
    VK_IMAGE_LAYOUT_VIDEO_ENCODE_SRC_KHR = 1000299001,
  // Provided by VK_KHR_video_encode_queue
    VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR = 1000299002,
  // Provided by VK_EXT_attachment_feedback_loop_layout
    VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT = 1000339000,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_IMAGE_LAYOUT_VIDEO_ENCODE_QUANTIZATION_MAP_KHR = 1000553000,
  // Provided by VK_KHR_maintenance2
```

```
    VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL_KHR =
VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL,
  // Provided by VK_KHR_maintenance2
    VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL_KHR =
VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL,
  // Provided by VK_KHR_separate_depth_stencil_layouts
    VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL_KHR =
VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL,
  // Provided by VK_KHR_separate_depth_stencil_layouts
    VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL_KHR =
VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL,
  // Provided by VK_KHR_separate_depth_stencil_layouts
    VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL_KHR =
VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL,
  // Provided by VK_KHR_separate_depth_stencil_layouts
    VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL_KHR =
VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL,
  // Provided by VK_KHR_synchronization2
    VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR = VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL,
  // Provided by VK_KHR_synchronization2
    VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR = VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL,
} VkImageLayout;
```

The type(s) of device access supported by each layout are:

- `VK_IMAGE_LAYOUT_UNDEFINED` specifies that the layout is unknown. Image memory **cannot** be transitioned into this layout. This layout **can** be used as the `initialLayout` member of VkImageCreateInfo. This layout **can** be used in place of the current image layout in a layout transition, but doing so will cause the contents of the image's memory to be undefined.
- `VK_IMAGE_LAYOUT_PREINITIALIZED` specifies that an image's memory is in a defined layout and **can** be populated by data, but that it has not yet been initialized by the driver. Image memory **cannot** be transitioned into this layout. This layout **can** be used as the `initialLayout` member of VkImageCreateInfo. This layout is intended to be used as the initial layout for an image whose contents are written by the host, and hence the data **can** be written to memory immediately, without first executing a layout transition. Currently, `VK_IMAGE_LAYOUT_PREINITIALIZED` is only useful with linear images because there is not a standard layout defined for `VK_IMAGE_TILING_OPTIMAL` images.
- `VK_IMAGE_LAYOUT_GENERAL` supports all types of device access, unless specified otherwise.
- `VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL` specifies a layout that **must** only be used with attachment accesses in the graphics pipeline.
- `VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL` specifies a layout allowing read only access as an attachment, or in shaders as a sampled image, combined image/sampler, or input attachment.
- `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` **must** only be used as a color or resolve attachment in a `VkFramebuffer`. This layout is valid only for image subresources of images created with the `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT` usage bit enabled.
- `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL` specifies a layout for both the depth and stencil aspects of a depth/stencil format image allowing read and write access as a depth/stencil

attachment. It is equivalent to VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL and VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL.

- VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL specifies a layout for both the depth and stencil aspects of a depth/stencil format image allowing read only access as a depth/stencil attachment or in shaders as a sampled image, combined image/sampler, or input attachment. It is equivalent to VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL and VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL.
- VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL specifies a layout for depth/stencil format images allowing read and write access to the stencil aspect as a stencil attachment, and read only access to the depth aspect as a depth attachment or in shaders as a sampled image, combined image/sampler, or input attachment. It is equivalent to VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL and VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL.
- VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL specifies a layout for depth/stencil format images allowing read and write access to the depth aspect as a depth attachment, and read only access to the stencil aspect as a stencil attachment or in shaders as a sampled image, combined image/sampler, or input attachment. It is equivalent to VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL and VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL.
- VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL specifies a layout for the depth aspect of a depth/stencil format image allowing read and write access as a depth attachment.
- VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL specifies a layout for the depth aspect of a depth/stencil format image allowing read-only access as a depth attachment or in shaders as a sampled image, combined image/sampler, or input attachment.
- VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL specifies a layout for the stencil aspect of a depth/stencil format image allowing read and write access as a stencil attachment.
- VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL specifies a layout for the stencil aspect of a depth/stencil format image allowing read-only access as a stencil attachment or in shaders as a sampled image, combined image/sampler, or input attachment.
- VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL specifies a layout allowing read-only access in a shader as a sampled image, combined image/sampler, or input attachment. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_SAMPLED_BIT or VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT usage bits enabled.
- VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL **must** only be used as a source image of a transfer command (see the definition of VK_PIPELINE_STAGE_TRANSFER_BIT). This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_TRANSFER_SRC_BIT usage bit enabled.
- VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL **must** only be used as a destination image of a transfer command. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_TRANSFER_DST_BIT usage bit enabled.
- VK_IMAGE_LAYOUT_PRESENT_SRC_KHR **must** only be used for presenting a presentable image for display.
- VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR is valid only for shared presentable images, and **must** be used for any usage the image supports.
- VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR **must** only be used as a

fragment shading rate attachment or This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR usage bit enabled.

- VK_IMAGE_LAYOUT_FRAGMENT_DENSITY_MAP_OPTIMAL_EXT **must** only be used as a fragment density map attachment in a VkRenderPass. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT usage bit enabled.
- VK_IMAGE_LAYOUT_VIDEO_DECODE_DST_KHR **must** only be used as a decode output picture in a video decode operation. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR usage bit enabled.
- VK_IMAGE_LAYOUT_VIDEO_DECODE_SRC_KHR is reserved for future use.
- VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR **must** only be used as an output reconstructed picture or an input reference picture in a video decode operation. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR usage bit enabled.
- VK_IMAGE_LAYOUT_VIDEO_ENCODE_DST_KHR is reserved for future use.
- VK_IMAGE_LAYOUT_VIDEO_ENCODE_SRC_KHR **must** only be used as an encode input picture in a video encode operation. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR usage bit enabled.
- VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR **must** only be used as an output reconstructed picture or an input reference picture in a video encode operation. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR usage bit enabled.
- VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT **must** only be used as either a color attachment or depth/stencil attachment in a VkFramebuffer and/or read-only access in a shader as a sampled image, combined image/sampler, or input attachment. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT usage bit enabled and either the VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT and either the VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT or VK_IMAGE_USAGE_SAMPLED_BIT usage bits enabled.
- VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR **must** only be used as either a storage image, or a color or depth/stencil attachment and an input attachment. This layout is valid only for image subresources of images created with either VK_IMAGE_USAGE_STORAGE_BIT, or both VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT and either of VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT or VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT.
- VK_IMAGE_LAYOUT_VIDEO_ENCODE_QUANTIZATION_MAP_KHR **must** only be used as a quantization map in a video encode operation. This layout is valid only for image subresources of images created with the VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR usage bit enabled.

The layout of each image subresource is not a state of the image subresource itself, but is rather a property of how the data in memory is organized, and thus for each mechanism of accessing an image in the API the application **must** specify a parameter or structure member that indicates which image layout the image subresource(s) are considered to be in when the image will be accessed. For transfer commands, this is a parameter to the command (see Clear Commands and Copy Commands). For use as a framebuffer attachment, this is a member in the substructures of the

VkRenderPassCreateInfo (see Render Pass). For use in a descriptor set, this is a member in the VkDescriptorImageInfo structure (see Descriptor Set Updates).

NOTE

VK_IMAGE_LAYOUT_GENERAL can be a useful catch-all image layout, but there are situations where a dedicated image layout must be used instead. Some examples include:

- VK_IMAGE_LAYOUT_PRESENT_SRC_KHR
- VK_IMAGE_LAYOUT_VIDEO_DECODE_SRC_KHR
- VK_IMAGE_LAYOUT_VIDEO_DECODE_DST_KHR
- VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR
- VK_IMAGE_LAYOUT_VIDEO_ENCODE_SRC_KHR
- VK_IMAGE_LAYOUT_VIDEO_ENCODE_DST_KHR
- VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR

NOTE

While VK_IMAGE_LAYOUT_GENERAL suggests that all types of device access is possible, it does not mean that all patterns of memory accesses are safe in all situations. Common Render Pass Data Races outlines some situations where data races are unavoidable. For example, when a subresource is used as both an attachment and a sampled image (i.e., not an input attachment), VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT adds extra guarantees which VK_IMAGE_LAYOUT_GENERAL does not.

12.4.1. Image Layout Matching Rules

At the time that any command buffer command accessing an image executes on any queue, the layouts of the image subresources that are accessed **must** all match exactly the layout specified via the API controlling those accesses, except in case of accesses to an image with a depth/stencil format performed through descriptors referring to only a single aspect of the image, where the following relaxed matching rules apply:

- Descriptors referring just to the depth aspect of a depth/stencil image only need to match in the image layout of the depth aspect, thus VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL and VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL are considered to match.
- Descriptors referring just to the stencil aspect of a depth/stencil image only need to match in the image layout of the stencil aspect, thus VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL and VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL are considered to match.

When performing a layout transition on an image subresource, the old layout value **must** either equal the current layout of the image subresource (at the time the transition executes), or else be VK_IMAGE_LAYOUT_UNDEFINED (implying that the contents of the image subresource need not be preserved). The new layout used in a transition **must** not be VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED.

The image layout of each image subresource of a depth/stencil image created with

VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT is dependent on the last sample locations used to render to the image subresource as a depth/stencil attachment, thus applications **must** provide the same sample locations that were last used to render to the given image subresource whenever a layout transition of the image subresource happens, otherwise the contents of the depth aspect of the image subresource become undefined.

In addition, depth reads from a depth/stencil attachment referring to an image subresource range of a depth/stencil image created with VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT using different sample locations than what have been last used to perform depth writes to the image subresources of the same image subresource range return undefined values.

Similarly, depth writes to a depth/stencil attachment referring to an image subresource range of a depth/stencil image created with VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT using different sample locations than what have been last used to perform depth writes to the image subresources of the same image subresource range make the contents of the depth aspect of those image subresources undefined.

12.5. Image Views

Image objects are not directly accessed by pipeline shaders for reading or writing image data. Instead, *image views* representing contiguous ranges of the image subresources and containing additional metadata are used for that purpose. Views **must** be created on images of compatible types, and **must** represent a valid subset of image subresources.

Image views are represented by VkImageView handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkImageView)
```

VK_REMAINING_ARRAY_LAYERS is a special constant value used for image views to indicate that all remaining array layers in an image after the base layer should be included in the view.

```
#define VK_REMAINING_ARRAY_LAYERS         (~0U)
```

VK_REMAINING_MIP_LEVELS is a special constant value used for image views to indicate that all remaining mipmap levels in an image after the base level should be included in the view.

```
#define VK_REMAINING_MIP_LEVELS           (~0U)
```

The types of image views that **can** be created are:

```
// Provided by VK_VERSION_1_0
typedef enum VkImageViewType {
    VK_IMAGE_VIEW_TYPE_1D = 0,
    VK_IMAGE_VIEW_TYPE_2D = 1,
    VK_IMAGE_VIEW_TYPE_3D = 2,
    VK_IMAGE_VIEW_TYPE_CUBE = 3,
    VK_IMAGE_VIEW_TYPE_1D_ARRAY = 4,
    VK_IMAGE_VIEW_TYPE_2D_ARRAY = 5,
    VK_IMAGE_VIEW_TYPE_CUBE_ARRAY = 6,
} VkImageViewType;
```

To create an image view, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateImageView(
    VkDevice                                    device,
    const VkImageViewCreateInfo*                pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkImageView*                                pView);
```

- `device` is the logical device that creates the image view.
- `pCreateInfo` is a pointer to a `VkImageViewCreateInfo` structure containing parameters to be used to create the image view.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pView` is a pointer to a VkImageView handle in which the resulting image view object is returned.

Valid Usage

- VUID-vkCreateImageView-device-09667
  `device` **must** support at least one queue family with one of the `VK_QUEUE_VIDEO_ENCODE_BIT_KHR`, `VK_QUEUE_VIDEO_DECODE_BIT_KHR`, `VK_QUEUE_COMPUTE_BIT`, or `VK_QUEUE_GRAPHICS_BIT` capabilities
- VUID-vkCreateImageView-image-09179
  VkImageViewCreateInfo::`image` **must** have been created from `device`

Valid Usage (Implicit)

- VUID-vkCreateImageView-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateImageView-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkImageViewCreateInfo structure
- VUID-vkCreateImageView-pAllocator-parameter

If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure

- VUID-vkCreateImageView-pView-parameter
  `pView` **must** be a valid pointer to a VkImageView handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR`

The `VkImageViewCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageViewCreateInfo {
    VkStructureType            sType;
    const void*                pNext;
    VkImageViewCreateFlags     flags;
    VkImage                    image;
    VkImageViewType            viewType;
    VkFormat                   format;
    VkComponentMapping         components;
    VkImageSubresourceRange    subresourceRange;
} VkImageViewCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkImageViewCreateFlagBits specifying additional parameters of the image view.
- `image` is a VkImage on which the view will be created.
- `viewType` is a VkImageViewType value specifying the type of the image view.
- `format` is a VkFormat specifying the format and type used to interpret texel blocks of the image.
- `components` is a VkComponentMapping structure specifying a remapping of color components (or of depth or stencil components after they have been converted into color components).
- `subresourceRange` is a VkImageSubresourceRange structure selecting the set of mipmap levels and array layers to be accessible to the view.

Some of the `image` creation parameters are inherited by the view. In particular, image view creation

inherits the implicit parameter `usage` specifying the allowed usages of the image view that, by default, takes the value of the corresponding `usage` parameter specified in VkImageCreateInfo at image creation time. The implicit `usage` **can** be overridden by adding a VkImageViewUsageCreateInfo structure to the `pNext` chain, but the view usage **must** be a subset of the image usage.

If `image` was created with the `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` flag, and if the `format` of the image is not multi-planar, `format` **can** be different from the image's format, but if `image` was created without the `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT` flag and they are not equal they **must** be *compatible*. Image format compatibility is defined in the Format Compatibility Classes section. Views of compatible formats will have the same mapping between texel coordinates and memory locations irrespective of the `format`, with only the interpretation of the bit pattern changing.

If `image` was created with a multi-planar format, and the image view's `aspectMask` is one of `VK_IMAGE_ASPECT_PLANE_0_BIT`, `VK_IMAGE_ASPECT_PLANE_1_BIT` or `VK_IMAGE_ASPECT_PLANE_2_BIT`, the view's aspect mask is considered to be equivalent to `VK_IMAGE_ASPECT_COLOR_BIT` when used as a framebuffer attachment.

NOTE

Values intended to be used with one view format **may** not be exactly preserved when written or read through a different format. For example, an integer value that happens to have the bit pattern of a floating-point denorm or NaN **may** be flushed or canonicalized when written or read through a view with a floating-point format. Similarly, a value written through a signed normalized format that has a bit pattern exactly equal to -2^b **may** be changed to $-2^b + 1$ as described in Conversion from Normalized Fixed-Point to Floating-Point.

If `image` was created with the `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT` flag, `format` **must** be *compatible* with the image's format as described above; or **must** be an uncompressed format, in which case it **must** be *size-compatible* with the image's format. In this case, the resulting image view's texel dimensions equal the dimensions of the selected mip level divided by the compressed texel block size and rounded up.

The VkComponentMapping `components` member describes a remapping from components of the image to components of the vector returned by shader image instructions. This remapping **must** be the identity swizzle for storage image descriptors, input attachment descriptors, framebuffer attachments, and any `VkImageView` used with a combined image sampler that enables sampler $Y'C_BC_R$ conversion.

If the image view is to be used with a sampler which supports sampler $Y'C_BC_R$ conversion, an *identically defined object* of type VkSamplerYcbcrConversion to that used to create the sampler **must** be passed to vkCreateImageView in a VkSamplerYcbcrConversionInfo included in the `pNext` chain of VkImageViewCreateInfo. Conversely, if a VkSamplerYcbcrConversion object is passed to vkCreateImageView, an identically defined VkSamplerYcbcrConversion object **must** be used when sampling the image.

If the image has a multi-planar `format`, `subresourceRange.aspectMask` is `VK_IMAGE_ASPECT_COLOR_BIT`, and `usage` includes `VK_IMAGE_USAGE_SAMPLED_BIT`, then the `format` **must** be identical to the image `format` and the sampler to be used with the image view **must** enable sampler $Y'C_BC_R$ conversion.

When such an image is used in a video coding operation, the sampler $Y'C_BC_R$ conversion has no effect.

If image was created with the VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT and the image has a multi-planar format, and if subresourceRange.aspectMask is VK_IMAGE_ASPECT_PLANE_0_BIT, VK_IMAGE_ASPECT_PLANE_1_BIT, or VK_IMAGE_ASPECT_PLANE_2_BIT, format **must** be compatible with the corresponding plane of the image, and the sampler to be used with the image view **must** not enable sampler $Y'C_BC_R$ conversion. The width and height of the single-plane image view **must** be derived from the multi-planar image's dimensions in the manner listed for plane compatibility for the plane.

Any view of an image plane will have the same mapping between texel coordinates and memory locations as used by the components of the color aspect, subject to the formulae relating texel coordinates to lower-resolution planes as described in Chroma Reconstruction. That is, if an R or B plane has a reduced resolution relative to the G plane of the multi-planar image, the image view operates using the (u_{plane}, v_{plane}) unnormalized coordinates of the reduced-resolution plane, and these coordinates access the same memory locations as the (u_{color}, v_{color}) unnormalized coordinates of the color aspect for which chroma reconstruction operations operate on the same (u_{plane}, v_{plane}) or (i_{plane}, j_{plane}) coordinates.

Table 12. Image Type and Image View Type Compatibility Requirements

Image View Type	Compatible Image Types
VK_IMAGE_VIEW_TYPE_1D	VK_IMAGE_TYPE_1D
VK_IMAGE_VIEW_TYPE_1D_ARRAY	VK_IMAGE_TYPE_1D
VK_IMAGE_VIEW_TYPE_2D	VK_IMAGE_TYPE_2D , VK_IMAGE_TYPE_3D
VK_IMAGE_VIEW_TYPE_2D_ARRAY	VK_IMAGE_TYPE_2D , VK_IMAGE_TYPE_3D
VK_IMAGE_VIEW_TYPE_CUBE	VK_IMAGE_TYPE_2D
VK_IMAGE_VIEW_TYPE_CUBE_ARRAY	VK_IMAGE_TYPE_2D
VK_IMAGE_VIEW_TYPE_3D	VK_IMAGE_TYPE_3D

Valid Usage

- VUID-VkImageViewCreateInfo-image-01003
  If image was not created with VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT then viewType **must** not be VK_IMAGE_VIEW_TYPE_CUBE or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-VkImageViewCreateInfo-viewType-01004
  If the imageCubeArray feature is not enabled, viewType **must** not be VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-VkImageViewCreateInfo-image-06723
  If image was created with VK_IMAGE_TYPE_3D but without VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT set then viewType **must** not be VK_IMAGE_VIEW_TYPE_2D_ARRAY
- VUID-VkImageViewCreateInfo-image-06728
  If image was created with VK_IMAGE_TYPE_3D but without

VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT or VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT set, then viewType **must** not be VK_IMAGE_VIEW_TYPE_2D

- VUID-VkImageViewCreateInfo-image-04970
  If image was created with VK_IMAGE_TYPE_3D and viewType is VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY then subresourceRange.levelCount **must** be 1
- VUID-VkImageViewCreateInfo-image-04972
  If image was created with a samples value not equal to VK_SAMPLE_COUNT_1_BIT then viewType **must** be either VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY
- VUID-VkImageViewCreateInfo-image-04441
  image **must** have been created with a usage value containing at least one of the usages defined in the valid image usage list for image views
- VUID-VkImageViewCreateInfo-None-02273
  The format features of the resultant image view **must** contain at least one bit
- VUID-VkImageViewCreateInfo-usage-02274
  If usage contains VK_IMAGE_USAGE_SAMPLED_BIT, then the format features of the resultant image view **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT
- VUID-VkImageViewCreateInfo-usage-02275
  If usage contains VK_IMAGE_USAGE_STORAGE_BIT, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT
- VUID-VkImageViewCreateInfo-usage-02276
  If usage contains VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT, then the image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT
- VUID-VkImageViewCreateInfo-usage-02277
  If usage contains VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT, then the image view's format features **must** contain VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageViewCreateInfo-image-08333
  If image was created with VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR and usage contains VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR, then the image view's format features **must** contain VK_FORMAT_FEATURE_VIDEO_DECODE_OUTPUT_BIT_KHR
- VUID-VkImageViewCreateInfo-image-08334
  If image was created with VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR and usage contains VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR, then the image view's format features **must** contain VK_FORMAT_FEATURE_VIDEO_DECODE_DPB_BIT_KHR
- VUID-VkImageViewCreateInfo-image-08335
  If image was created with VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then usage **must** not include VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR
- VUID-VkImageViewCreateInfo-image-08336
  If image was created with VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR and usage contains VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR, then the image view's format features **must** contain VK_FORMAT_FEATURE_VIDEO_ENCODE_INPUT_BIT_KHR
- VUID-VkImageViewCreateInfo-image-08337
  If image was created with VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR and usage contains VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR, then the image view's format features

must contain VK_FORMAT_FEATURE_VIDEO_ENCODE_DPB_BIT_KHR

- VUID-VkImageViewCreateInfo-image-08338
  If image was created with VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR, then usage **must** not include VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR
- VUID-VkImageViewCreateInfo-usage-10259
  If usage contains VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR, then the image view's format features **must** contain VK_FORMAT_FEATURE_2_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- VUID-VkImageViewCreateInfo-usage-10260
  If usage contains VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR, then the image view's format features **must** contain VK_FORMAT_FEATURE_2_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR
- VUID-VkImageViewCreateInfo-usage-08932
  If usage contains VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT,

  then the image view's format features **must** contain at least one of VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT or VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
- VUID-VkImageViewCreateInfo-subresourceRange-01478
  subresourceRange.baseMipLevel **must** be less than the mipLevels specified in VkImageCreateInfo when image was created
- VUID-VkImageViewCreateInfo-subresourceRange-01718
  If subresourceRange.levelCount is not VK_REMAINING_MIP_LEVELS, subresourceRange.baseMipLevel + subresourceRange.levelCount **must** be less than or equal to the mipLevels specified in VkImageCreateInfo when image was created
- VUID-VkImageViewCreateInfo-image-02571
  If image was created with usage containing VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT, subresourceRange.levelCount **must** be 1
- VUID-VkImageViewCreateInfo-image-06724
  If image is not a 3D image created with VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT or VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT set, or viewType is not VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY, subresourceRange.baseArrayLayer **must** be less than the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-VkImageViewCreateInfo-subresourceRange-06725
  If subresourceRange.layerCount is not VK_REMAINING_ARRAY_LAYERS, image is not a 3D image created with VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT or VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT set, or viewType is not VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY, subresourceRange.layerCount **must** be non-zero and subresourceRange.baseArrayLayer + subresourceRange.layerCount **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-VkImageViewCreateInfo-image-02724
  If image is a 3D image created with VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT set, and viewType is VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY, subresourceRange.baseArrayLayer **must** be less than the depth computed from baseMipLevel and extent.depth specified in VkImageCreateInfo when image was created, according to

the formula defined in Image Mip Level Sizing

- VUID-VkImageViewCreateInfo-subresourceRange-02725
  If `subresourceRange.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `image` is a 3D image created with `VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT` set, and `viewType` is `VK_IMAGE_VIEW_TYPE_2D` or `VK_IMAGE_VIEW_TYPE_2D_ARRAY`, `subresourceRange.layerCount` **must** be non-zero and `subresourceRange.baseArrayLayer` + `subresourceRange.layerCount` **must** be less than or equal to the depth computed from `baseMipLevel` and `extent.depth` specified in VkImageCreateInfo when `image` was created, according to the formula defined in Image Mip Level Sizing
- VUID-VkImageViewCreateInfo-image-01761
  If `image` was created with the `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` flag, but without the `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT` flag, and if the `format` of the `image` is not a multi-planar format, `format` **must** be compatible with the `format` used to create `image`, as defined in Format Compatibility Classes
- VUID-VkImageViewCreateInfo-image-01583
  If `image` was created with the `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT` flag, `format` **must** be compatible with, or **must** be an uncompressed format that is size-compatible with, the `format` used to create `image`
- VUID-VkImageViewCreateInfo-image-07072
  If `image` was created with the `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT` flag and `format` is a non-compressed format, the `levelCount` member of `subresourceRange` **must** be `1`
- VUID-VkImageViewCreateInfo-image-09487
  If `image` was created with the `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT` flag, the `VkPhysicalDeviceMaintenance6PropertiesKHR`::`blockTexelViewCompatibleMultipleLayers` property is not `VK_TRUE`, and `format` is a non-compressed format, then the `layerCount` member of `subresourceRange` **must** be `1`
- VUID-VkImageViewCreateInfo-pNext-01585
  If a VkImageFormatListCreateInfo structure was included in the `pNext` chain of the VkImageCreateInfo structure used when creating `image` and VkImageFormatListCreateInfo::`viewFormatCount` is not zero then `format` **must** be one of the formats in VkImageFormatListCreateInfo::`pViewFormats`
- VUID-VkImageViewCreateInfo-image-01586
  If `image` was created with the `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` flag, if the `format` of the `image` is a multi-planar format, and if `subresourceRange.aspectMask` is one of the multi-planar aspect mask bits, then `format` **must** be compatible with the VkFormat for the plane of the `image` `format` indicated by `subresourceRange.aspectMask`, as defined in Compatible Formats of Planes of Multi-Planar Formats
- VUID-VkImageViewCreateInfo-subresourceRange-07818
  `subresourceRange.aspectMask` **must** only have at most 1 valid multi-planar aspect mask bit
- VUID-VkImageViewCreateInfo-image-01762
  If `image` was not created with the `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` flag, or if the `format` of the `image` is a multi-planar format and if `subresourceRange.aspectMask` is `VK_IMAGE_ASPECT_COLOR_BIT`, `format` **must** be identical to the `format` used to create `image`
- VUID-VkImageViewCreateInfo-format-06415

If the image view requires a sampler $Y'C_BC_R$ conversion and `usage` contains `VK_IMAGE_USAGE_SAMPLED_BIT`, then the `pNext` chain **must** include a VkSamplerYcbcrConversionInfo structure with a conversion value other than VK_NULL_HANDLE

- VUID-VkImageViewCreateInfo-format-04714
  If `format` has a `_422` or `_420` suffix then `image` **must** have been created with a width that is a multiple of 2
- VUID-VkImageViewCreateInfo-format-04715
  If `format` has a `_420` suffix then `image` **must** have been created with a height that is a multiple of 2
- VUID-VkImageViewCreateInfo-pNext-01970
  If the `pNext` chain includes a VkSamplerYcbcrConversionInfo structure with a `conversion` value other than VK_NULL_HANDLE, all members of `components` **must** have the identity swizzle
- VUID-VkImageViewCreateInfo-pNext-06658
  If the `pNext` chain includes a VkSamplerYcbcrConversionInfo structure with a `conversion` value other than VK_NULL_HANDLE, `format` **must** be the same used in VkSamplerYcbcrConversionCreateInfo::`format`
- VUID-VkImageViewCreateInfo-image-01020
  If `image` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkImageViewCreateInfo-subResourceRange-01021
  `viewType` **must** be compatible with the type of `image` as shown in the view type compatibility table
- VUID-VkImageViewCreateInfo-image-02086
  If `image` was created with `usage` containing `VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`, `viewType` **must** be `VK_IMAGE_VIEW_TYPE_2D` or `VK_IMAGE_VIEW_TYPE_2D_ARRAY`
- VUID-VkImageViewCreateInfo-usage-04550
  If the `attachmentFragmentShadingRate` feature is enabled, and the `usage` for the image view includes `VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`, then the image view's format features **must** contain `VK_FORMAT_FEATURE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-VkImageViewCreateInfo-usage-04551
  If the `attachmentFragmentShadingRate` feature is enabled, the `usage` for the image view includes `VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`, and `layeredShadingRateAttachments` is `VK_FALSE`, `subresourceRange.layerCount` **must** be `1`
- VUID-VkImageViewCreateInfo-flags-02572
  If the `fragmentDensityMapDynamic` feature is not enabled, `flags` **must** not contain `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DYNAMIC_BIT_EXT`
- VUID-VkImageViewCreateInfo-flags-03567
  If the `fragmentDensityMapDeferred` feature is not enabled, `flags` **must** not contain `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DEFERRED_BIT_EXT`

- VUID-VkImageViewCreateInfo-flags-03568
  If `flags` contains `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DEFERRED_BIT_EXT`, `flags` **must** not contain `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DYNAMIC_BIT_EXT`
- VUID-VkImageViewCreateInfo-image-03569
  If `image` was created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT` and `usage` containing `VK_IMAGE_USAGE_SAMPLED_BIT`, `subresourceRange.layerCount` **must** be less than or equal to `VkPhysicalDeviceFragmentDensityMap2PropertiesEXT::maxSubsampledArrayLayers`
- VUID-VkImageViewCreateInfo-flags-04116
  If `flags` does not contain `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DYNAMIC_BIT_EXT` and `image` was created with `usage` containing `VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT`, its `flags` **must** not contain any of `VK_IMAGE_CREATE_PROTECTED_BIT`, `VK_IMAGE_CREATE_SPARSE_BINDING_BIT`, `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`, or `VK_IMAGE_CREATE_SPARSE_ALIASED_BIT`
- VUID-VkImageViewCreateInfo-pNext-02661
  If the `pNext` chain includes a VkImageViewUsageCreateInfo structure, its `usage` member **must** not include any bits that were not set in the `usage` member of the VkImageCreateInfo structure used to create `image`
- VUID-VkImageViewCreateInfo-imageViewType-04973
  If `viewType` is `VK_IMAGE_VIEW_TYPE_1D`, `VK_IMAGE_VIEW_TYPE_2D`, or `VK_IMAGE_VIEW_TYPE_3D`; and `subresourceRange.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, then `subresourceRange.layerCount` **must** be 1
- VUID-VkImageViewCreateInfo-imageViewType-04974
  If `viewType` is `VK_IMAGE_VIEW_TYPE_1D`, `VK_IMAGE_VIEW_TYPE_2D`, or `VK_IMAGE_VIEW_TYPE_3D`; and `subresourceRange.layerCount` is `VK_REMAINING_ARRAY_LAYERS`, then the remaining number of layers **must** be 1
- VUID-VkImageViewCreateInfo-viewType-02960
  If `viewType` is `VK_IMAGE_VIEW_TYPE_CUBE` and `subresourceRange.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `subresourceRange.layerCount` **must** be `6`
- VUID-VkImageViewCreateInfo-viewType-02961
  If `viewType` is `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY` and `subresourceRange.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `subresourceRange.layerCount` **must** be a multiple of `6`
- VUID-VkImageViewCreateInfo-viewType-02962
  If `viewType` is `VK_IMAGE_VIEW_TYPE_CUBE` and `subresourceRange.layerCount` is `VK_REMAINING_ARRAY_LAYERS`, the remaining number of layers **must** be `6`
- VUID-VkImageViewCreateInfo-viewType-02963
  If `viewType` is `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY` and `subresourceRange.layerCount` is `VK_REMAINING_ARRAY_LAYERS`, the remaining number of layers **must** be a multiple of `6`
- VUID-VkImageViewCreateInfo-imageViewFormatSwizzle-04465
  If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`imageViewFormatSwizzle` is `VK_FALSE`, all elements of `components` **must** have the identity swizzle
- VUID-VkImageViewCreateInfo-imageViewFormatReinterpretation-04466
  If the `VK_KHR_portability_subset` extension is enabled, and

VkPhysicalDevicePortabilitySubsetFeaturesKHR::imageViewFormatReinterpretation is VK_FALSE, the VkFormat in format **must** not contain a different number of components, or a different number of bits in each component, than the format of the VkImage in image

- VUID-VkImageViewCreateInfo-image-04817
  If image was created with usage containing VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR, VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR, or VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR, then the viewType **must** be VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY
- VUID-VkImageViewCreateInfo-image-04818
  If image was created with usage containing VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR, VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR, or VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR, then the viewType **must** be VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY
- VUID-VkImageViewCreateInfo-image-10261
  If image was created with usage containing VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR, then viewType **must** be VK_IMAGE_VIEW_TYPE_2D or VK_IMAGE_VIEW_TYPE_2D_ARRAY
- VUID-VkImageViewCreateInfo-pNext-06787
  If the pNext chain includes a VkExportMetalObjectCreateInfoEXT structure, its exportObjectType member **must** be VK_EXPORT_METAL_OBJECT_TYPE_METAL_TEXTURE_BIT_EXT
- VUID-VkImageViewCreateInfo-subresourceRange-09594
  subresourceRange.aspectMask **must** be valid for the format the image was created with

Valid Usage (Implicit)

- VUID-VkImageViewCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO
- VUID-VkImageViewCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkExportMetalObjectCreateInfoEXT, VkImageViewASTCDecodeModeEXT, VkImageViewMinLodCreateInfoEXT, VkImageViewUsageCreateInfo, or VkSamplerYcbcrConversionInfo
- VUID-VkImageViewCreateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique, with the exception of structures of type VkExportMetalObjectCreateInfoEXT
- VUID-VkImageViewCreateInfo-flags-parameter
  flags **must** be a valid combination of VkImageViewCreateFlagBits values
- VUID-VkImageViewCreateInfo-image-parameter
  image **must** be a valid VkImage handle
- VUID-VkImageViewCreateInfo-viewType-parameter
  viewType **must** be a valid VkImageViewType value
- VUID-VkImageViewCreateInfo-format-parameter
  format **must** be a valid VkFormat value

- VUID-VkImageViewCreateInfo-components-parameter
  `components` **must** be a valid VkComponentMapping structure
- VUID-VkImageViewCreateInfo-subresourceRange-parameter
  `subresourceRange` **must** be a valid VkImageSubresourceRange structure

Bits which **can** be set in VkImageViewCreateInfo::`flags`, specifying additional parameters of an image view, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkImageViewCreateFlagBits {
  // Provided by VK_EXT_fragment_density_map
    VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DYNAMIC_BIT_EXT = 0x00000001,
  // Provided by VK_EXT_fragment_density_map2
    VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DEFERRED_BIT_EXT = 0x00000002,
} VkImageViewCreateFlagBits;
```

- `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DYNAMIC_BIT_EXT` specifies that the fragment density map will be read by device during `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DEFERRED_BIT_EXT` specifies that the fragment density map will be read by the host during vkEndCommandBuffer for the primary command buffer that the render pass is recorded into

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkImageViewCreateFlags;
```

`VkImageViewCreateFlags` is a bitmask type for setting a mask of zero or more VkImageViewCreateFlagBits.

The set of usages for the created image view **can** be restricted compared to the parent image's `usage` flags by adding a `VkImageViewUsageCreateInfo` structure to the `pNext` chain of VkImageViewCreateInfo.

The `VkImageViewUsageCreateInfo` structure is defined as:

```
typedef struct VkImageViewUsageCreateInfo {
    VkStructureType      sType;
    const void*          pNext;
    VkImageUsageFlags    usage;
} VkImageViewUsageCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_maintenance2
typedef VkImageViewUsageCreateInfo VkImageViewUsageCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `usage` is a bitmask of VkImageUsageFlagBits specifying allowed usages of the image view.

When this structure is chained to VkImageViewCreateInfo the `usage` field overrides the implicit `usage` parameter inherited from image creation time and its value is used instead for the purposes of determining the valid usage conditions of VkImageViewCreateInfo.

Valid Usage (Implicit)

- VUID-VkImageViewUsageCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_VIEW_USAGE_CREATE_INFO`
- VUID-VkImageViewUsageCreateInfo-usage-parameter
  `usage` **must** be a valid combination of VkImageUsageFlagBits values
- VUID-VkImageViewUsageCreateInfo-usage-requiredbitmask
  `usage` **must** not be `0`

The `VkImageSubresourceRange` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageSubresourceRange {
    VkImageAspectFlags    aspectMask;
    uint32_t              baseMipLevel;
    uint32_t              levelCount;
    uint32_t              baseArrayLayer;
    uint32_t              layerCount;
} VkImageSubresourceRange;
```

- `aspectMask` is a bitmask of VkImageAspectFlagBits specifying which aspect(s) of the image are included in the view.
- `baseMipLevel` is the first mipmap level accessible to the view.
- `levelCount` is the number of mipmap levels (starting from `baseMipLevel`) accessible to the view.
- `baseArrayLayer` is the first array layer accessible to the view.
- `layerCount` is the number of array layers (starting from `baseArrayLayer`) accessible to the view.

The number of mipmap levels and array layers **must** be a subset of the image subresources in the image. If an application wants to use all mip levels or layers in an image after the `baseMipLevel` or `baseArrayLayer`, it **can** set `levelCount` and `layerCount` to the special values `VK_REMAINING_MIP_LEVELS` and `VK_REMAINING_ARRAY_LAYERS` without knowing the exact number of mip levels or layers.

For cube and cube array image views, the layers of the image view starting at `baseArrayLayer` correspond to faces in the order +X, -X, +Y, -Y, +Z, -Z. For cube arrays, each set of six sequential layers is a single cube, so the number of cube maps in a cube map array view is `layerCount` / *6*, and image array layer (`baseArrayLayer` + i) is face index (i mod 6) of cube *i* / *6*. If the number of layers in

the view, whether set explicitly in `layerCount` or implied by `VK_REMAINING_ARRAY_LAYERS`, is not a multiple of 6, the last cube map in the array **must** not be accessed.

`aspectMask` **must** be only `VK_IMAGE_ASPECT_COLOR_BIT`, `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT` if `format` is a color, depth-only or stencil-only format, respectively, except if `format` is a multi-planar format. If using a depth/stencil format with both depth and stencil components, `aspectMask` **must** include at least one of `VK_IMAGE_ASPECT_DEPTH_BIT` and `VK_IMAGE_ASPECT_STENCIL_BIT`, and **can** include both.

When the `VkImageSubresourceRange` structure is used to select a subset of the slices of a 3D image's mip level in order to create a 2D or 2D array image view of a 3D image created with `VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT`, `baseArrayLayer` and `layerCount` specify the first slice index and the number of slices to include in the created image view. Such an image view **can** be used as a framebuffer attachment that refers only to the specified range of slices of the selected mip level. However, any layout transitions performed on such an attachment view during a render pass instance still apply to the entire subresource referenced which includes all the slices of the selected mip level.

When using an image view of a depth/stencil image to populate a descriptor set (e.g. for sampling in the shader, or for use as an input attachment), the `aspectMask` **must** only include one bit, which selects whether the image view is used for depth reads (i.e. using a floating-point sampler or input attachment in the shader) or stencil reads (i.e. using an unsigned integer sampler or input attachment in the shader). When an image view of a depth/stencil image is used as a depth/stencil framebuffer attachment, the `aspectMask` is ignored and both depth and stencil image subresources are used.

When creating a `VkImageView`, if sampler $Y'C_BC_R$ conversion is enabled in the sampler, the `aspectMask` of a `subresourceRange` used by the `VkImageView` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`.

When creating a `VkImageView`, if sampler $Y'C_BC_R$ conversion is not enabled in the sampler and the image `format` is multi-planar, the image **must** have been created with `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT`, and the `aspectMask` of the `VkImageView`'s `subresourceRange` **must** be `VK_IMAGE_ASPECT_PLANE_0_BIT`, `VK_IMAGE_ASPECT_PLANE_1_BIT` or `VK_IMAGE_ASPECT_PLANE_2_BIT`.

Valid Usage

- VUID-VkImageSubresourceRange-levelCount-01720
  If `levelCount` is not `VK_REMAINING_MIP_LEVELS`, it **must** be greater than `0`
- VUID-VkImageSubresourceRange-layerCount-01721
  If `layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, it **must** be greater than `0`
- VUID-VkImageSubresourceRange-aspectMask-01670
  If `aspectMask` includes `VK_IMAGE_ASPECT_COLOR_BIT`, then it **must** not include any of `VK_IMAGE_ASPECT_PLANE_0_BIT`, `VK_IMAGE_ASPECT_PLANE_1_BIT`, or `VK_IMAGE_ASPECT_PLANE_2_BIT`
- VUID-VkImageSubresourceRange-aspectMask-02278
  `aspectMask` **must** not include `VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT` for any index *i*

Valid Usage (Implicit)

- VUID-VkImageSubresourceRange-aspectMask-parameter
  `aspectMask` **must** be a valid combination of VkImageAspectFlagBits values
- VUID-VkImageSubresourceRange-aspectMask-requiredbitmask
  `aspectMask` **must** not be `0`

Bits which **can** be set in an aspect mask to specify aspects of an image for purposes such as identifying a subresource, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkImageAspectFlagBits {
    VK_IMAGE_ASPECT_COLOR_BIT = 0x00000001,
    VK_IMAGE_ASPECT_DEPTH_BIT = 0x00000002,
    VK_IMAGE_ASPECT_STENCIL_BIT = 0x00000004,
    VK_IMAGE_ASPECT_METADATA_BIT = 0x00000008,
    VK_IMAGE_ASPECT_PLANE_0_BIT = 0x00000010,
    VK_IMAGE_ASPECT_PLANE_1_BIT = 0x00000020,
    VK_IMAGE_ASPECT_PLANE_2_BIT = 0x00000040,
    VK_IMAGE_ASPECT_NONE = 0,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_IMAGE_ASPECT_MEMORY_PLANE_0_BIT_EXT = 0x00000080,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_IMAGE_ASPECT_MEMORY_PLANE_1_BIT_EXT = 0x00000100,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_IMAGE_ASPECT_MEMORY_PLANE_2_BIT_EXT = 0x00000200,
  // Provided by VK_EXT_image_drm_format_modifier
    VK_IMAGE_ASPECT_MEMORY_PLANE_3_BIT_EXT = 0x00000400,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_IMAGE_ASPECT_PLANE_0_BIT_KHR = VK_IMAGE_ASPECT_PLANE_0_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_IMAGE_ASPECT_PLANE_1_BIT_KHR = VK_IMAGE_ASPECT_PLANE_1_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_IMAGE_ASPECT_PLANE_2_BIT_KHR = VK_IMAGE_ASPECT_PLANE_2_BIT,
  // Provided by VK_KHR_maintenance4
    VK_IMAGE_ASPECT_NONE_KHR = VK_IMAGE_ASPECT_NONE,
} VkImageAspectFlagBits;
```

- `VK_IMAGE_ASPECT_NONE` specifies no image aspect, or the image aspect is not applicable.
- `VK_IMAGE_ASPECT_COLOR_BIT` specifies the color aspect.
- `VK_IMAGE_ASPECT_DEPTH_BIT` specifies the depth aspect.
- `VK_IMAGE_ASPECT_STENCIL_BIT` specifies the stencil aspect.
- `VK_IMAGE_ASPECT_METADATA_BIT` specifies the metadata aspect used for sparse resource operations.
- `VK_IMAGE_ASPECT_PLANE_0_BIT` specifies plane 0 of a *multi-planar* image format.
- `VK_IMAGE_ASPECT_PLANE_1_BIT` specifies plane 1 of a *multi-planar* image format.

- VK_IMAGE_ASPECT_PLANE_2_BIT specifies plane 2 of a *multi-planar* image format.
- VK_IMAGE_ASPECT_MEMORY_PLANE_0_BIT_EXT specifies *memory plane* 0.
- VK_IMAGE_ASPECT_MEMORY_PLANE_1_BIT_EXT specifies *memory plane* 1.
- VK_IMAGE_ASPECT_MEMORY_PLANE_2_BIT_EXT specifies *memory plane* 2.
- VK_IMAGE_ASPECT_MEMORY_PLANE_3_BIT_EXT specifies *memory plane* 3.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkImageAspectFlags;
```

VkImageAspectFlags is a bitmask type for setting a mask of zero or more VkImageAspectFlagBits.

The VkComponentMapping structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkComponentMapping {
    VkComponentSwizzle    r;
    VkComponentSwizzle    g;
    VkComponentSwizzle    b;
    VkComponentSwizzle    a;
} VkComponentMapping;
```

- r is a VkComponentSwizzle specifying the component value placed in the R component of the output vector.
- g is a VkComponentSwizzle specifying the component value placed in the G component of the output vector.
- b is a VkComponentSwizzle specifying the component value placed in the B component of the output vector.
- a is a VkComponentSwizzle specifying the component value placed in the A component of the output vector.

Valid Usage (Implicit)

- VUID-VkComponentMapping-r-parameter
  r **must** be a valid VkComponentSwizzle value
- VUID-VkComponentMapping-g-parameter
  g **must** be a valid VkComponentSwizzle value
- VUID-VkComponentMapping-b-parameter
  b **must** be a valid VkComponentSwizzle value
- VUID-VkComponentMapping-a-parameter
  a **must** be a valid VkComponentSwizzle value

Possible values of the members of VkComponentMapping, specifying the component values placed

in each component of the output vector, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkComponentSwizzle {
    VK_COMPONENT_SWIZZLE_IDENTITY = 0,
    VK_COMPONENT_SWIZZLE_ZERO = 1,
    VK_COMPONENT_SWIZZLE_ONE = 2,
    VK_COMPONENT_SWIZZLE_R = 3,
    VK_COMPONENT_SWIZZLE_G = 4,
    VK_COMPONENT_SWIZZLE_B = 5,
    VK_COMPONENT_SWIZZLE_A = 6,
} VkComponentSwizzle;
```

- VK_COMPONENT_SWIZZLE_IDENTITY specifies that the component is set to the identity swizzle.
- VK_COMPONENT_SWIZZLE_ZERO specifies that the component is set to zero.
- VK_COMPONENT_SWIZZLE_ONE specifies that the component is set to either 1 or 1.0, depending on whether the type of the image view format is integer or floating-point respectively, as determined by the Format Definition section for each VkFormat.
- VK_COMPONENT_SWIZZLE_R specifies that the component is set to the value of the R component of the image.
- VK_COMPONENT_SWIZZLE_G specifies that the component is set to the value of the G component of the image.
- VK_COMPONENT_SWIZZLE_B specifies that the component is set to the value of the B component of the image.
- VK_COMPONENT_SWIZZLE_A specifies that the component is set to the value of the A component of the image.

Setting the identity swizzle on a component is equivalent to setting the identity mapping on that component. That is:

Table 13. Component Mappings Equivalent To VK_COMPONENT_SWIZZLE_IDENTITY

Component	Identity Mapping
components.r	VK_COMPONENT_SWIZZLE_R
components.g	VK_COMPONENT_SWIZZLE_G
components.b	VK_COMPONENT_SWIZZLE_B
components.a	VK_COMPONENT_SWIZZLE_A

If the pNext chain includes a VkImageViewASTCDecodeModeEXT structure, then that structure includes a parameter specifying the decode mode for image views using ASTC compressed formats.

The VkImageViewASTCDecodeModeEXT structure is defined as:

```
// Provided by VK_EXT_astc_decode_mode
typedef struct VkImageViewASTCDecodeModeEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkFormat           decodeMode;
} VkImageViewASTCDecodeModeEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `decodeMode` is the intermediate format used to decode ASTC compressed formats.

Valid Usage

- VUID-VkImageViewASTCDecodeModeEXT-decodeMode-02230
  `decodeMode` **must** be one of `VK_FORMAT_R16G16B16A16_SFLOAT`, `VK_FORMAT_R8G8B8A8_UNORM`, or `VK_FORMAT_E5B9G9R9_UFLOAT_PACK32`
- VUID-VkImageViewASTCDecodeModeEXT-decodeMode-02231
  If the `decodeModeSharedExponent` feature is not enabled, `decodeMode` **must** not be `VK_FORMAT_E5B9G9R9_UFLOAT_PACK32`
- VUID-VkImageViewASTCDecodeModeEXT-decodeMode-02232
  If `decodeMode` is `VK_FORMAT_R8G8B8A8_UNORM` the image view **must** not include blocks using any of the ASTC HDR modes
- VUID-VkImageViewASTCDecodeModeEXT-format-04084
  `format` of the image view **must** be one of the ASTC Compressed Image Formats

If `format` uses sRGB encoding then the `decodeMode` has no effect.

Valid Usage (Implicit)

- VUID-VkImageViewASTCDecodeModeEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_VIEW_ASTC_DECODE_MODE_EXT`
- VUID-VkImageViewASTCDecodeModeEXT-decodeMode-parameter
  `decodeMode` **must** be a valid VkFormat value

To destroy an image view, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyImageView(
    VkDevice                                    device,
    VkImageView                                 imageView,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the image view.
- `imageView` is the image view to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyImageView-imageView-01026
  All submitted commands that refer to `imageView` **must** have completed execution
- VUID-vkDestroyImageView-imageView-01027
  If `VkAllocationCallbacks` were provided when `imageView` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyImageView-imageView-01028
  If no `VkAllocationCallbacks` were provided when `imageView` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyImageView-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyImageView-imageView-parameter
  If `imageView` is not VK_NULL_HANDLE, `imageView` **must** be a valid VkImageView handle
- VUID-vkDestroyImageView-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyImageView-imageView-parent
  If `imageView` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `imageView` **must** be externally synchronized

12.5.1. Image View Format Features

Valid uses of a VkImageView **may** depend on the image view's *format features*, defined below. Such constraints are documented in the affected valid usage statement.

- If Vulkan 1.3 is supported or the `VK_KHR_format_feature_flags2` extension is supported, and VkImageViewCreateInfo::`image` was created with `VK_IMAGE_TILING_LINEAR`, then the image view's set of *format features* is the value of VkFormatProperties3::`linearTilingFeatures` found by calling vkGetPhysicalDeviceFormatProperties2KHR on the same `format` as VkImageViewCreateInfo::`format`.

- If Vulkan 1.3 is not supported and the VK_KHR_format_feature_flags2 extension is not supported, and VkImageViewCreateInfo::image was created with VK_IMAGE_TILING_LINEAR, then the image view's set of *format features* is the union of the value of VkFormatProperties::linearTilingFeatures found by calling vkGetPhysicalDeviceFormatProperties on the same format as VkImageViewCreateInfo::format, with:
  - VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT if the format is a depth/stencil format and the image view features also contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_BIT.
  - VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT if the format is one of the extended storage formats and the shaderStorageImageReadWithoutFormat feature is enabled.
  - VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT if the format is one of the extended storage formats and the shaderStorageImageWriteWithoutFormat feature is enabled.
- If Vulkan 1.3 is supported or the VK_KHR_format_feature_flags2 extension is supported, and VkImageViewCreateInfo::image was created with VK_IMAGE_TILING_OPTIMAL, then the image view's set of *format features* is the value of VkFormatProperties::optimalTilingFeatures or VkFormatProperties3::optimalTilingFeatures found by calling vkGetPhysicalDeviceFormatProperties or vkGetPhysicalDeviceImageFormatProperties2KHR on the same format as VkImageViewCreateInfo::format.
- If Vulkan 1.3 is not supported and the VK_KHR_format_feature_flags2 extension is not supported, and VkImageViewCreateInfo::image was created with VK_IMAGE_TILING_OPTIMAL, then the image view's set of *format features* is the union of the value of VkFormatProperties::optimalTilingFeatures found by calling vkGetPhysicalDeviceFormatProperties on the same format as VkImageViewCreateInfo::format, with:
  - VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT if the format is a depth/stencil format and the image view features also contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_BIT.
  - VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT if the format is one of the extended storage formats and the shaderStorageImageReadWithoutFormat is enabled.
  - VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT if the format is one of the extended storage formats and the shaderStorageImageWriteWithoutFormat is enabled.
- If VkImageViewCreateInfo::image was created with VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then:
  - The image's DRM format modifier is the value of VkImageDrmFormatModifierPropertiesEXT::drmFormatModifier found by calling vkGetImageDrmFormatModifierPropertiesEXT.
  - Let VkDrmFormatModifierPropertiesListEXT::pDrmFormatModifierProperties be the array found by calling vkGetPhysicalDeviceFormatProperties2KHR on the same format as VkImageViewCreateInfo::format.
  - Let VkDrmFormatModifierPropertiesEXT prop be the array element whose drmFormatModifier member is the value of the image's DRM format modifier.
  - Then the image view's set of *format features* is prop::drmFormatModifierTilingFeatures.

The VkImageViewMinLodCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_image_view_min_lod
typedef struct VkImageViewMinLodCreateInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    float              minLod;
} VkImageViewMinLodCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `minLod` is the value to clamp the minimum LOD accessible by this VkImageView.

If the `pNext` chain includes a `VkImageViewMinLodCreateInfoEXT` structure, then that structure includes a parameter specifying a value to clamp the minimum LOD value during Image Level(s) Selection, Texel Gathering and Integer Texel Coordinate Operations.

If the image view contains `VkImageViewMinLodCreateInfoEXT` and it is used as part of a sampling operation:

$\text{minLodFloat}_{\text{imageView}}$ = `minLod`

otherwise:

$\text{minLodFloat}_{\text{imageView}}$ = 0.0

An integer variant of this parameter is also defined for sampling operations which access integer mipmap levels:

$\text{minLodInteger}_{\text{imageView}} = \lfloor \text{minLodFloat}_{\text{imageView}} \rfloor$

Valid Usage

- VUID-VkImageViewMinLodCreateInfoEXT-minLod-06455
  If the `minLod` feature is not enabled, `minLod` **must** be `0.0`
- VUID-VkImageViewMinLodCreateInfoEXT-minLod-06456
  `minLod` **must** be less or equal to the index of the last mipmap level accessible to the view

Valid Usage (Implicit)

- VUID-VkImageViewMinLodCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_VIEW_MIN_LOD_CREATE_INFO_EXT`

12.6. Acceleration Structures

Acceleration structures are opaque data structures that are built by the implementation to more efficiently perform spatial queries on the provided geometric data. For these extensions, an

acceleration structure is either a top-level acceleration structure containing a set of bottom-level acceleration structures or a bottom-level acceleration structure containing either a set of axis-aligned bounding boxes for custom geometry or a set of triangles.

Each instance in the top-level acceleration structure contains a reference to a bottom-level acceleration structure as well as an instance transform plus information required to index into the shader bindings. The top-level acceleration structure is what is bound to the acceleration descriptor, for example to trace inside the shader in the ray tracing pipeline.

Acceleration structures for the `VK_KHR_acceleration_structure` extension are represented by `VkAccelerationStructureKHR` handles:

```
// Provided by VK_KHR_acceleration_structure
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkAccelerationStructureKHR)
```

To create an acceleration structure, call:

```
// Provided by VK_KHR_acceleration_structure
VkResult vkCreateAccelerationStructureKHR(
    VkDevice                                    device,
    const VkAccelerationStructureCreateInfoKHR* pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkAccelerationStructureKHR*                 pAccelerationStructure);
```

- `device` is the logical device that creates the acceleration structure object.
- `pCreateInfo` is a pointer to a VkAccelerationStructureCreateInfoKHR structure containing parameters affecting creation of the acceleration structure.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pAccelerationStructure` is a pointer to a `VkAccelerationStructureKHR` handle in which the resulting acceleration structure object is returned.

Similar to other objects in Vulkan, the acceleration structure creation merely creates an object with a specific “shape”. The type and quantity of geometry that can be built into an acceleration structure is determined by the parameters of VkAccelerationStructureCreateInfoKHR.

The acceleration structure data is stored in the object referred to by `VkAccelerationStructureCreateInfoKHR`::`buffer`. Once memory has been bound to that buffer, it **must** be populated by acceleration structure build or acceleration structure copy commands such as vkCmdBuildAccelerationStructuresKHR, vkBuildAccelerationStructuresKHR, vkCmdCopyAccelerationStructureKHR, and vkCopyAccelerationStructureKHR.

NOTE

The expected usage for a trace capture/replay tool is that it will serialize and later deserialize the acceleration structure data using acceleration structure copy commands. During capture the tool will use vkCopyAccelerationStructureToMemoryKHR or vkCmdCopyAccelerationStructureToMemoryKHR with a `mode` of

VK_COPY_ACCELERATION_STRUCTURE_MODE_SERIALIZE_KHR, and vkCopyMemoryToAccelerationStructureKHR or vkCmdCopyMemoryToAccelerationStructureKHR with a mode of VK_COPY_ACCELERATION_STRUCTURE_MODE_DESERIALIZE_KHR during replay.

NOTE Memory does not need to be bound to the underlying buffer when vkCreateAccelerationStructureKHR is called.

The input buffers passed to acceleration structure build commands will be referenced by the implementation for the duration of the command. After the command completes, the acceleration structure **may** hold a reference to any acceleration structure specified by an active instance contained therein. Apart from this referencing, acceleration structures **must** be fully self-contained. The application **can** reuse or free any memory which was used by the command as an input or as scratch without affecting the results of ray traversal.

Valid Usage

- VUID-vkCreateAccelerationStructureKHR-accelerationStructure-03611
  The VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure feature **must** be enabled
- VUID-vkCreateAccelerationStructureKHR-deviceAddress-03488
  If VkAccelerationStructureCreateInfoKHR::deviceAddress is not zero, the accelerationStructureCaptureReplay feature **must** be enabled
- VUID-vkCreateAccelerationStructureKHR-device-03489
  If device was created with multiple physical devices, then the bufferDeviceAddressMultiDevice feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCreateAccelerationStructureKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateAccelerationStructureKHR-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkAccelerationStructureCreateInfoKHR structure
- VUID-vkCreateAccelerationStructureKHR-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateAccelerationStructureKHR-pAccelerationStructure-parameter
  pAccelerationStructure **must** be a valid pointer to a VkAccelerationStructureKHR handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR

The VkAccelerationStructureCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureCreateInfoKHR {
    VkStructureType                           sType;
    const void*                               pNext;
    VkAccelerationStructureCreateFlagsKHR     createFlags;
    VkBuffer                                  buffer;
    VkDeviceSize                              offset;
    VkDeviceSize                              size;
    VkAccelerationStructureTypeKHR            type;
    VkDeviceAddress                           deviceAddress;
} VkAccelerationStructureCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- createFlags is a bitmask of VkAccelerationStructureCreateFlagBitsKHR specifying additional creation parameters of the acceleration structure.
- buffer is the buffer on which the acceleration structure will be stored.
- offset is an offset in bytes from the base address of the buffer at which the acceleration structure will be stored, and **must** be a multiple of 256.
- size is the size required for the acceleration structure.
- type is a VkAccelerationStructureTypeKHR value specifying the type of acceleration structure that will be created.
- deviceAddress is the device address requested for the acceleration structure if the accelerationStructureCaptureReplay feature is being used. If deviceAddress is zero, no specific address is requested.

Applications **should** avoid creating acceleration structures with application-provided addresses and implementation-provided addresses in the same process, to reduce the likelihood of VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR errors.

NOTE	The expected usage for this is that a trace capture/replay tool will add the VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT flag to all buffers that use VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT, and will add

VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT to all buffers used as storage for an acceleration structure where deviceAddress is not zero. This also means that the tool will need to add VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT to memory allocations to allow the flag to be set where the application may not have otherwise required it. During capture the tool will save the queried opaque device addresses in the trace. During replay, the buffers will be created specifying the original address so any address values stored in the trace data will remain valid.

Implementations are expected to separate such buffers in the GPU address space so normal allocations will avoid using these addresses. Apps/tools should avoid mixing app-provided and implementation-provided addresses for buffers created with VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT, to avoid address space allocation conflicts.

Applications **should** create an acceleration structure with a specific VkAccelerationStructureTypeKHR other than VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR.

NOTE

VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR is intended to be used by API translation layers. This can be used at acceleration structure creation time in cases where the actual acceleration structure type (top or bottom) is not yet known. The actual acceleration structure type must be specified as VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR or VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR when the build is performed.

If the acceleration structure will be the target of a build operation, the required size for an acceleration structure **can** be queried with vkGetAccelerationStructureBuildSizesKHR. If the acceleration structure is going to be the target of a compacting copy, vkCmdWriteAccelerationStructuresPropertiesKHR or vkWriteAccelerationStructuresPropertiesKHR **can** be used to obtain the compacted size required.

Valid Usage

- VUID-VkAccelerationStructureCreateInfoKHR-deviceAddress-03612
  If deviceAddress is not zero, createFlags **must** include VK_ACCELERATION_STRUCTURE_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_KHR
- VUID-VkAccelerationStructureCreateInfoKHR-deviceAddress-09488
  If deviceAddress is not zero, it **must** have been retrieved from an identically created acceleration structure, except for buffer and deviceAddress
- VUID-VkAccelerationStructureCreateInfoKHR-deviceAddress-09489
  If deviceAddress is not zero, buffer **must** have been created identically to the buffer used to create the acceleration structure from which deviceAddress was retrieved, except for VkBufferOpaqueCaptureAddressCreateInfo::opaqueCaptureAddress
- VUID-VkAccelerationStructureCreateInfoKHR-deviceAddress-09490
  If deviceAddress is not zero, buffer **must** have been created with a VkBufferOpaqueCaptureAddressCreateInfo::opaqueCaptureAddress that was retrieved from vkGetBufferOpaqueCaptureAddressKHR for the buffer that was used to create the

acceleration structure from which deviceAddress was retrieved

- VUID-VkAccelerationStructureCreateInfoKHR-createFlags-03613
  If createFlags includes VK_ACCELERATION_STRUCTURE_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_KHR, VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructureCaptureReplay **must** be VK_TRUE
- VUID-VkAccelerationStructureCreateInfoKHR-buffer-03614
  buffer **must** have been created with a usage value containing VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR
- VUID-VkAccelerationStructureCreateInfoKHR-buffer-03615
  buffer **must** not have been created with VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT
- VUID-VkAccelerationStructureCreateInfoKHR-offset-03616
  The sum of offset and size **must** be less than the size of buffer
- VUID-VkAccelerationStructureCreateInfoKHR-offset-03734
  offset **must** be a multiple of 256 bytes

Valid Usage (Implicit)

- VUID-VkAccelerationStructureCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_CREATE_INFO_KHR
- VUID-VkAccelerationStructureCreateInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkAccelerationStructureCreateInfoKHR-createFlags-parameter
  createFlags **must** be a valid combination of VkAccelerationStructureCreateFlagBitsKHR values
- VUID-VkAccelerationStructureCreateInfoKHR-buffer-parameter
  buffer **must** be a valid VkBuffer handle
- VUID-VkAccelerationStructureCreateInfoKHR-type-parameter
  type **must** be a valid VkAccelerationStructureTypeKHR value

To get the build sizes for an acceleration structure, call:

```
// Provided by VK_KHR_acceleration_structure
void vkGetAccelerationStructureBuildSizesKHR(
    VkDevice                                    device,
    VkAccelerationStructureBuildTypeKHR         buildType,
    const VkAccelerationStructureBuildGeometryInfoKHR* pBuildInfo,
    const uint32_t*                             pMaxPrimitiveCounts,
    VkAccelerationStructureBuildSizesInfoKHR*   pSizeInfo);
```

- device is the logical device that will be used for creating the acceleration structure.
- buildType defines whether host or device operations (or both) are being queried for.

- `pBuildInfo` is a pointer to a VkAccelerationStructureBuildGeometryInfoKHR structure describing parameters of a build operation.
- `pMaxPrimitiveCounts` is a pointer to an array of `pBuildInfo->geometryCount` `uint32_t` values defining the number of primitives built into each geometry.
- `pSizeInfo` is a pointer to a VkAccelerationStructureBuildSizesInfoKHR structure which returns the size required for an acceleration structure and the sizes required for the scratch buffers, given the build parameters.

The `srcAccelerationStructure`, `dstAccelerationStructure`, and `mode` members of `pBuildInfo` are ignored. Any VkDeviceOrHostAddressKHR or VkDeviceOrHostAddressConstKHR members of `pBuildInfo` are ignored by this command, except that the `hostAddress` member of VkAccelerationStructureGeometryTrianglesDataKHR::`transformData` will be examined to check if it is `NULL`.

An acceleration structure created with the `accelerationStructureSize` returned by this command supports any build or update with a VkAccelerationStructureBuildGeometryInfoKHR structure and array of VkAccelerationStructureBuildRangeInfoKHR structures subject to the following properties:

- The build command is a host build command, and `buildType` is `VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_KHR` or `VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_OR_DEVICE_KHR`
- The build command is a device build command, and `buildType` is `VK_ACCELERATION_STRUCTURE_BUILD_TYPE_DEVICE_KHR` or `VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_OR_DEVICE_KHR`
- For VkAccelerationStructureBuildGeometryInfoKHR:
  - Its `type`, and `flags` members are equal to `pBuildInfo->type` and `pBuildInfo->flags`, respectively.
  - `geometryCount` is less than or equal to `pBuildInfo->geometryCount`.
  - For each element of either `pGeometries` or `ppGeometries` at a given index, its `geometryType` member is equal to `pBuildInfo->geometryType`.
  - For each element of either `pGeometries` or `ppGeometries` at a given index, its `flags` member is equal to the corresponding member of the same element in `pBuildInfo`.
  - For each element of either `pGeometries` or `ppGeometries` at a given index, with a `geometryType` member equal to `VK_GEOMETRY_TYPE_TRIANGLES_KHR`, the `vertexFormat` and `indexType` members of `geometry.triangles` are equal to the corresponding members of the same element in `pBuildInfo`.
  - For each element of either `pGeometries` or `ppGeometries` at a given index, with a `geometryType` member equal to `VK_GEOMETRY_TYPE_TRIANGLES_KHR`, the `maxVertex` member of `geometry.triangles` is less than or equal to the corresponding member of the same element in `pBuildInfo`.
  - For each element of either `pGeometries` or `ppGeometries` at a given index, with a `geometryType` member equal to `VK_GEOMETRY_TYPE_TRIANGLES_KHR`, if the applicable address in the `transformData` member of `geometry.triangles` is not `NULL`, the corresponding `transformData.hostAddress` parameter in `pBuildInfo` is not `NULL`.

- For each VkAccelerationStructureBuildRangeInfoKHR corresponding to the VkAccelerationStructureBuildGeometryInfoKHR:
  - Its primitiveCount member is less than or equal to the corresponding element of pMaxPrimitiveCounts.
  - For each element of either pGeometries or ppGeometries at a given index, with a geometryType member equal to VK_GEOMETRY_TYPE_TRIANGLES_KHR, if the pNext chain contains VkAccelerationStructureTrianglesOpacityMicromapEXT the corresponding member of pBuildInfo also contains VkAccelerationStructureTrianglesOpacityMicromapEXT and with an equivalent micromap.

Similarly, the updateScratchSize value will support any build command specifying the VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR mode under the above conditions, and the buildScratchSize value will support any build command specifying the VK_BUILD_ACCELERATION_STRUCTURE_MODE_BUILD_KHR mode under the above conditions.

Valid Usage

- VUID-vkGetAccelerationStructureBuildSizesKHR-accelerationStructure-08933
  The VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure feature **must** be enabled
- VUID-vkGetAccelerationStructureBuildSizesKHR-device-03618
  If device was created with multiple physical devices, then the bufferDeviceAddressMultiDevice feature **must** be enabled
- VUID-vkGetAccelerationStructureBuildSizesKHR-pBuildInfo-03619
  If pBuildInfo->geometryCount is not 0, pMaxPrimitiveCounts **must** be a valid pointer to an array of pBuildInfo->geometryCount uint32_t values
- VUID-vkGetAccelerationStructureBuildSizesKHR-pBuildInfo-03785
  If pBuildInfo->pGeometries or pBuildInfo->ppGeometries has a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, each pMaxPrimitiveCounts[i] **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxInstanceCount

Valid Usage (Implicit)

- VUID-vkGetAccelerationStructureBuildSizesKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetAccelerationStructureBuildSizesKHR-buildType-parameter
  buildType **must** be a valid VkAccelerationStructureBuildTypeKHR value
- VUID-vkGetAccelerationStructureBuildSizesKHR-pBuildInfo-parameter
  pBuildInfo **must** be a valid pointer to a valid VkAccelerationStructureBuildGeometryInfoKHR structure
- VUID-vkGetAccelerationStructureBuildSizesKHR-pMaxPrimitiveCounts-parameter
  If pMaxPrimitiveCounts is not NULL, pMaxPrimitiveCounts **must** be a valid pointer to an array of pBuildInfo->geometryCount uint32_t values

- VUID-vkGetAccelerationStructureBuildSizesKHR-pSizeInfo-parameter
  `pSizeInfo` **must** be a valid pointer to a VkAccelerationStructureBuildSizesInfoKHR structure

The `VkAccelerationStructureBuildSizesInfoKHR` structure describes the required build sizes for an acceleration structure and scratch buffers and is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureBuildSizesInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkDeviceSize       accelerationStructureSize;
    VkDeviceSize       updateScratchSize;
    VkDeviceSize       buildScratchSize;
} VkAccelerationStructureBuildSizesInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `accelerationStructureSize` is the size in bytes required in a VkAccelerationStructureKHR for a build or update operation.
- `updateScratchSize` is the size in bytes required in a scratch buffer for an update operation.
- `buildScratchSize` is the size in bytes required in a scratch buffer for a build operation.

Valid Usage (Implicit)

- VUID-VkAccelerationStructureBuildSizesInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_BUILD_SIZES_INFO_KHR`
- VUID-VkAccelerationStructureBuildSizesInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`

Values which **can** be set in VkAccelerationStructureCreateInfoKHR::`type` specifying the type of acceleration structure, are:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkAccelerationStructureTypeKHR {
    VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR = 0,
    VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR = 1,
    VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR = 2,
} VkAccelerationStructureTypeKHR;
```

- `VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR` is a top-level acceleration structure containing instance data referring to bottom-level acceleration structures.
- `VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR` is a bottom-level acceleration structure

containing the AABBs or geometry to be intersected.

- VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR is an acceleration structure whose type is determined at build time used for special circumstances. In these cases, the acceleration structure type is not known at creation time, but **must** be specified at build time as either top or bottom.

Bits which **can** be set in VkAccelerationStructureCreateInfoKHR::createFlags, specifying additional creation parameters for acceleration structures, are:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkAccelerationStructureCreateFlagBitsKHR {
    VK_ACCELERATION_STRUCTURE_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_KHR =
0x00000001,
} VkAccelerationStructureCreateFlagBitsKHR;
```

- VK_ACCELERATION_STRUCTURE_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_KHR specifies that the acceleration structure's address **can** be saved and reused on a subsequent run.

```
// Provided by VK_KHR_acceleration_structure
typedef VkFlags VkAccelerationStructureCreateFlagsKHR;
```

VkAccelerationStructureCreateFlagsKHR is a bitmask type for setting a mask of zero or more VkAccelerationStructureCreateFlagBitsKHR.

Bits which **can** be set in VkAccelerationStructureBuildGeometryInfoKHR::flags specifying additional parameters for acceleration structure builds, are:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkBuildAccelerationStructureFlagBitsKHR {
    VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_UPDATE_BIT_KHR = 0x00000001,
    VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_COMPACTION_BIT_KHR = 0x00000002,
    VK_BUILD_ACCELERATION_STRUCTURE_PREFER_FAST_TRACE_BIT_KHR = 0x00000004,
    VK_BUILD_ACCELERATION_STRUCTURE_PREFER_FAST_BUILD_BIT_KHR = 0x00000008,
    VK_BUILD_ACCELERATION_STRUCTURE_LOW_MEMORY_BIT_KHR = 0x00000010,
  // Provided by VK_EXT_opacity_micromap
    VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_UPDATE_EXT = 0x00000040,
  // Provided by VK_EXT_opacity_micromap
    VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DISABLE_OPACITY_MICROMAPS_EXT = 0x00000080,
  // Provided by VK_EXT_opacity_micromap
    VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_DATA_UPDATE_EXT =
0x00000100,
  // Provided by VK_KHR_ray_tracing_position_fetch
    VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DATA_ACCESS_KHR = 0x00000800,
} VkBuildAccelerationStructureFlagBitsKHR;
```

- VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_UPDATE_BIT_KHR specifies that the specified acceleration

structure **can** be updated with a `mode` of `VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR` in VkAccelerationStructureBuildGeometryInfoKHR .

- `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_COMPACTION_BIT_KHR` specifies that the specified acceleration structure **can** act as the source for a copy acceleration structure command with `mode` of `VK_COPY_ACCELERATION_STRUCTURE_MODE_COMPACT_KHR` to produce a compacted acceleration structure.
- `VK_BUILD_ACCELERATION_STRUCTURE_PREFER_FAST_TRACE_BIT_KHR` specifies that the given acceleration structure build **should** prioritize trace performance over build time.
- `VK_BUILD_ACCELERATION_STRUCTURE_PREFER_FAST_BUILD_BIT_KHR` specifies that the given acceleration structure build **should** prioritize build time over trace performance.
- `VK_BUILD_ACCELERATION_STRUCTURE_LOW_MEMORY_BIT_KHR` specifies that this acceleration structure **should** minimize the size of the scratch memory and the final result acceleration structure, potentially at the expense of build time or trace performance.
- `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_UPDATE_EXT` specifies that the opacity micromaps associated with the specified acceleration structure **may** change with an acceleration structure update.
- `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_DATA_UPDATE_EXT` specifies that the data of the opacity micromaps associated with the specified acceleration structure **may** change with an acceleration structure update.
- `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DISABLE_OPACITY_MICROMAPS_EXT` specifies that the specified acceleration structure **may** be referenced in an instance with `VK_GEOMETRY_INSTANCE_DISABLE_OPACITY_MICROMAPS_EXT` set.
- `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DATA_ACCESS_KHR` specifies that the specified acceleration structure **can** be used when fetching the vertex positions of a hit triangle.

NOTE `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_UPDATE_BIT_KHR` and `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_COMPACTION_BIT_KHR` **may** take more time and memory than a normal build, and so **should** only be used when those features are needed.

NOTE `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_UPDATE_BIT_KHR` and `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_COMPACTION_BIT_KHR` are allowed to be used together. In that case, the result of the compaction copy is used as the source of a build with `mode` of `VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR` to perform the compacted update.

```
// Provided by VK_KHR_acceleration_structure
typedef VkFlags VkBuildAccelerationStructureFlagsKHR;
```

`VkBuildAccelerationStructureFlagsKHR` is a bitmask type for setting a mask of zero or more VkBuildAccelerationStructureFlagBitsKHR.

Geometry types are specified by VkGeometryTypeKHR, which takes values:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkGeometryTypeKHR {
    VK_GEOMETRY_TYPE_TRIANGLES_KHR = 0,
    VK_GEOMETRY_TYPE_AABBS_KHR = 1,
    VK_GEOMETRY_TYPE_INSTANCES_KHR = 2,
} VkGeometryTypeKHR;
```

- VK_GEOMETRY_TYPE_TRIANGLES_KHR specifies a geometry type consisting of triangles.
- VK_GEOMETRY_TYPE_AABBS_KHR specifies a geometry type consisting of axis-aligned bounding boxes.
- VK_GEOMETRY_TYPE_INSTANCES_KHR specifies a geometry type consisting of acceleration structure instances.

Bits specifying additional parameters for geometries in acceleration structure builds, are:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkGeometryFlagBitsKHR {
    VK_GEOMETRY_OPAQUE_BIT_KHR = 0x00000001,
    VK_GEOMETRY_NO_DUPLICATE_ANY_HIT_INVOCATION_BIT_KHR = 0x00000002,
} VkGeometryFlagBitsKHR;
```

- VK_GEOMETRY_OPAQUE_BIT_KHR specifies that this geometry does not invoke the any-hit shaders even if present in a hit group.
- VK_GEOMETRY_NO_DUPLICATE_ANY_HIT_INVOCATION_BIT_KHR specifies that the implementation **must** only call the any-hit shader a single time for each primitive in this geometry. If this bit is absent an implementation **may** invoke the any-hit shader more than once for this geometry.

```
// Provided by VK_KHR_acceleration_structure
typedef VkFlags VkGeometryFlagsKHR;
```

VkGeometryFlagsKHR is a bitmask type for setting a mask of zero or more VkGeometryFlagBitsKHR.

To destroy an acceleration structure, call:

```
// Provided by VK_KHR_acceleration_structure
void vkDestroyAccelerationStructureKHR(
    VkDevice                                    device,
    VkAccelerationStructureKHR                  accelerationStructure,
    const VkAllocationCallbacks*                pAllocator);
```

- device is the logical device that destroys the acceleration structure.
- accelerationStructure is the acceleration structure to destroy.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyAccelerationStructureKHR-accelerationStructure-08934
  The `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure` feature **must** be enabled
- VUID-vkDestroyAccelerationStructureKHR-accelerationStructure-02442
  All submitted commands that refer to `accelerationStructure` **must** have completed execution
- VUID-vkDestroyAccelerationStructureKHR-accelerationStructure-02443
  If `VkAllocationCallbacks` were provided when `accelerationStructure` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyAccelerationStructureKHR-accelerationStructure-02444
  If no `VkAllocationCallbacks` were provided when `accelerationStructure` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyAccelerationStructureKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyAccelerationStructureKHR-accelerationStructure-parameter
  If `accelerationStructure` is not VK_NULL_HANDLE, `accelerationStructure` **must** be a valid VkAccelerationStructureKHR handle
- VUID-vkDestroyAccelerationStructureKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyAccelerationStructureKHR-accelerationStructure-parent
  If `accelerationStructure` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `accelerationStructure` **must** be externally synchronized

Possible values of `buildType` in vkGetAccelerationStructureBuildSizesKHR are:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkAccelerationStructureBuildTypeKHR {
    VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_KHR = 0,
    VK_ACCELERATION_STRUCTURE_BUILD_TYPE_DEVICE_KHR = 1,
    VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_OR_DEVICE_KHR = 2,
} VkAccelerationStructureBuildTypeKHR;
```

- VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_KHR requests the memory requirement for operations performed by the host.
- VK_ACCELERATION_STRUCTURE_BUILD_TYPE_DEVICE_KHR requests the memory requirement for operations performed by the device.
- VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_OR_DEVICE_KHR requests the memory requirement for operations performed by either the host, or the device.

To query the 64-bit device address for an acceleration structure, call:

```
// Provided by VK_KHR_acceleration_structure
VkDeviceAddress vkGetAccelerationStructureDeviceAddressKHR(
    VkDevice                                    device,
    const VkAccelerationStructureDeviceAddressInfoKHR* pInfo);
```

- device is the logical device that the acceleration structure was created on.
- pInfo is a pointer to a VkAccelerationStructureDeviceAddressInfoKHR structure specifying the acceleration structure to retrieve an address for.

The 64-bit return value is an address of the acceleration structure, which can be used for device and shader operations that involve acceleration structures, such as ray traversal and acceleration structure building.

If the acceleration structure was created with a non-zero value of VkAccelerationStructureCreateInfoKHR::deviceAddress, the return value will be the same address.

If the acceleration structure was created with a type of VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR, the returned address **must** be consistent with the relative offset to other acceleration structures with type VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR allocated with the same VkBuffer. That is, the difference in returned addresses between the two **must** be the same as the difference in offsets provided at acceleration structure creation.

The returned address **must** be aligned to 256 bytes.

NOTE The acceleration structure device address **may** be different from the buffer device address corresponding to the acceleration structure's start offset in its storage buffer for acceleration structure types other than VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR.

Valid Usage

- VUID-vkGetAccelerationStructureDeviceAddressKHR-accelerationStructure-08935
  The VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure feature **must** be enabled
- VUID-vkGetAccelerationStructureDeviceAddressKHR-device-03504
  If device was created with multiple physical devices, then the bufferDeviceAddressMultiDevice feature **must** be enabled

- VUID-vkGetAccelerationStructureDeviceAddressKHR-pInfo-09541
  If the buffer on which `pInfo->accelerationStructure` was placed is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkGetAccelerationStructureDeviceAddressKHR-pInfo-09542
  The buffer on which `pInfo->accelerationStructure` was placed **must** have been created with the `VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT` usage flag

Valid Usage (Implicit)

- VUID-vkGetAccelerationStructureDeviceAddressKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetAccelerationStructureDeviceAddressKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkAccelerationStructureDeviceAddressInfoKHR structure

The `VkAccelerationStructureDeviceAddressInfoKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureDeviceAddressInfoKHR {
    VkStructureType               sType;
    const void*                   pNext;
    VkAccelerationStructureKHR    accelerationStructure;
} VkAccelerationStructureDeviceAddressInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `accelerationStructure` specifies the acceleration structure whose address is being queried.

Valid Usage (Implicit)

- VUID-VkAccelerationStructureDeviceAddressInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_DEVICE_ADDRESS_INFO_KHR`
- VUID-VkAccelerationStructureDeviceAddressInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkAccelerationStructureDeviceAddressInfoKHR-accelerationStructure-parameter
  `accelerationStructure` **must** be a valid VkAccelerationStructureKHR handle

12.7. Micromaps

Micromaps are opaque data structures that are built by the implementation to encode sub-triangle data to be included in an acceleration structure.

Micromaps are represented by VkMicromapEXT handles:

```
// Provided by VK_EXT_opacity_micromap
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkMicromapEXT)
```

To create a micromap, call:

```
// Provided by VK_EXT_opacity_micromap
VkResult vkCreateMicromapEXT(
    VkDevice                                    device,
    const VkMicromapCreateInfoEXT*              pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkMicromapEXT*                              pMicromap);
```

- device is the logical device that creates the acceleration structure object.
- pCreateInfo is a pointer to a VkMicromapCreateInfoEXT structure containing parameters affecting creation of the micromap.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.
- pMicromap is a pointer to a VkMicromapEXT handle in which the resulting micromap object is returned.

Similar to other objects in Vulkan, the micromap creation merely creates an object with a specific "shape". The type and quantity of geometry that can be built into a micromap is determined by the parameters of VkMicromapCreateInfoEXT.

The micromap data is stored in the object referred to by VkMicromapCreateInfoEXT::buffer. Once memory has been bound to that buffer, it **must** be populated by micromap build or micromap copy commands such as vkCmdBuildMicromapsEXT, vkBuildMicromapsEXT, vkCmdCopyMicromapEXT, and vkCopyMicromapEXT.

NOTE The expected usage for a trace capture/replay tool is that it will serialize and later deserialize the micromap data using micromap copy commands. During capture the tool will use vkCopyMicromapToMemoryEXT or vkCmdCopyMicromapToMemoryEXT with a mode of VK_COPY_MICROMAP_MODE_SERIALIZE_EXT, and vkCopyMemoryToMicromapEXT or vkCmdCopyMemoryToMicromapEXT with a mode of VK_COPY_MICROMAP_MODE_DESERIALIZE_EXT during replay.

The input buffers passed to micromap build commands will be referenced by the implementation for the duration of the command. Micromaps **must** be fully self-contained. The application **can** reuse or free any memory which was used by the command as an input or as scratch without affecting the results of a subsequent acceleration structure build using the micromap or traversal of that acceleration structure.

Valid Usage

- VUID-vkCreateMicromapEXT-micromap-07430
  The micromap feature **must** be enabled
- VUID-vkCreateMicromapEXT-deviceAddress-07431
  If VkMicromapCreateInfoEXT::deviceAddress is not zero, the micromapCaptureReplay feature **must** be enabled
- VUID-vkCreateMicromapEXT-device-07432
  If device was created with multiple physical devices, then the bufferDeviceAddressMultiDevice feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCreateMicromapEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateMicromapEXT-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkMicromapCreateInfoEXT structure
- VUID-vkCreateMicromapEXT-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateMicromapEXT-pMicromap-parameter
  pMicromap **must** be a valid pointer to a VkMicromapEXT handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR

The VkMicromapCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkMicromapCreateInfoEXT {
    VkStructureType             sType;
    const void*                 pNext;
    VkMicromapCreateFlagsEXT    createFlags;
    VkBuffer                    buffer;
    VkDeviceSize                offset;
    VkDeviceSize                size;
    VkMicromapTypeEXT           type;
    VkDeviceAddress             deviceAddress;
} VkMicromapCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `createFlags` is a bitmask of VkMicromapCreateFlagBitsEXT specifying additional creation parameters of the micromap.
- `buffer` is the buffer on which the micromap will be stored.
- `offset` is an offset in bytes from the base address of the buffer at which the micromap will be stored, and **must** be a multiple of `256`.
- `size` is the size required for the micromap.
- `type` is a VkMicromapTypeEXT value specifying the type of micromap that will be created.
- `deviceAddress` is the device address requested for the micromap if the `micromapCaptureReplay` feature is being used.

If `deviceAddress` is zero, no specific address is requested.

If `deviceAddress` is not zero, `deviceAddress` **must** be an address retrieved from an identically created micromap on the same implementation. The micromap **must** also be placed on an identically created `buffer` and at the same `offset`.

Applications **should** avoid creating micromaps with application-provided addresses and implementation-provided addresses in the same process, to reduce the likelihood of `VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR` errors.

NOTE

The expected usage for this is that a trace capture/replay tool will add the `VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT` flag to all buffers that use `VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT`, and will add `VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT` to all buffers used as storage for a micromap where `deviceAddress` is not zero. This also means that the tool will need to add `VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT` to memory allocations to allow the flag to be set where the application may not have otherwise required it. During capture the tool will save the queried opaque device addresses in the trace. During replay, the buffers will be created specifying the original address so any address values stored in the trace data will remain valid.

> Implementations are expected to separate such buffers in the GPU address space so normal allocations will avoid using these addresses. Apps/tools should avoid mixing app-provided and implementation-provided addresses for buffers created with VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT, to avoid address space allocation conflicts.

If the micromap will be the target of a build operation, the required size for a micromap **can** be queried with vkGetMicromapBuildSizesEXT.

Valid Usage

- VUID-VkMicromapCreateInfoEXT-deviceAddress-07433
  If deviceAddress is not zero, createFlags **must** include VK_MICROMAP_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_EXT
- VUID-VkMicromapCreateInfoEXT-createFlags-07434
  If createFlags includes VK_MICROMAP_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_EXT, VkPhysicalDeviceOpacityMicromapFeaturesEXT::micromapCaptureReplay **must** be VK_TRUE
- VUID-VkMicromapCreateInfoEXT-buffer-07435
  buffer **must** have been created with a usage value containing VK_BUFFER_USAGE_MICROMAP_STORAGE_BIT_EXT
- VUID-VkMicromapCreateInfoEXT-buffer-07436
  buffer **must** not have been created with VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT
- VUID-VkMicromapCreateInfoEXT-offset-07437
  The sum of offset and size **must** be less than the size of buffer
- VUID-VkMicromapCreateInfoEXT-offset-07438
  offset **must** be a multiple of 256 bytes

Valid Usage (Implicit)

- VUID-VkMicromapCreateInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_MICROMAP_CREATE_INFO_EXT
- VUID-VkMicromapCreateInfoEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkMicromapCreateInfoEXT-createFlags-parameter
  createFlags **must** be a valid combination of VkMicromapCreateFlagBitsEXT values
- VUID-VkMicromapCreateInfoEXT-buffer-parameter
  buffer **must** be a valid VkBuffer handle
- VUID-VkMicromapCreateInfoEXT-type-parameter
  type **must** be a valid VkMicromapTypeEXT value

To get the build sizes for a micromap, call:

```
// Provided by VK_EXT_opacity_micromap
void vkGetMicromapBuildSizesEXT(
    VkDevice                                    device,
    VkAccelerationStructureBuildTypeKHR         buildType,
    const VkMicromapBuildInfoEXT*               pBuildInfo,
    VkMicromapBuildSizesInfoEXT*                pSizeInfo);
```

- `device` is the logical device that will be used for creating the micromap.
- `buildType` defines whether host or device operations (or both) are being queried for.
- `pBuildInfo` is a pointer to a VkMicromapBuildInfoEXT structure describing parameters of a build operation.
- `pSizeInfo` is a pointer to a VkMicromapBuildSizesInfoEXT structure which returns the size required for a micromap and the sizes required for the scratch buffers, given the build parameters.

The `dstMicromap` and `mode` members of `pBuildInfo` are ignored. Any VkDeviceOrHostAddressKHR members of `pBuildInfo` are ignored by this command.

A micromap created with the `micromapSize` returned by this command supports any build with a VkMicromapBuildInfoEXT structure subject to the following properties:

- The build command is a host build command, and `buildType` is `VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_KHR` or `VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_OR_DEVICE_KHR`
- The build command is a device build command, and `buildType` is `VK_ACCELERATION_STRUCTURE_BUILD_TYPE_DEVICE_KHR` or `VK_ACCELERATION_STRUCTURE_BUILD_TYPE_HOST_OR_DEVICE_KHR`
- For VkMicromapBuildInfoEXT:
  - Its `type`, and `flags` members are equal to `pBuildInfo->type` and `pBuildInfo->flags`, respectively.
  - The sum of usage information in either `pUsageCounts` or `ppUsageCounts` is equal to the sum of usage information in either `pBuildInfo->pUsageCounts` or `pBuildInfo->ppUsageCounts`.

Similarly, the `buildScratchSize` value will support any build command specifying the `VK_BUILD_MICROMAP_MODE_BUILD_EXT` `mode` under the above conditions.

Valid Usage

- VUID-vkGetMicromapBuildSizesEXT-dstMicromap-09180
  VkMicromapBuildInfoEXT::`dstMicromap` **must** have been created from `device`
- VUID-vkGetMicromapBuildSizesEXT-micromap-07439
  The `micromap` feature **must** be enabled
- VUID-vkGetMicromapBuildSizesEXT-device-07440
  If `device` was created with multiple physical devices, then the

`bufferDeviceAddressMultiDevice` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkGetMicromapBuildSizesEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetMicromapBuildSizesEXT-buildType-parameter
  `buildType` **must** be a valid VkAccelerationStructureBuildTypeKHR value
- VUID-vkGetMicromapBuildSizesEXT-pBuildInfo-parameter
  `pBuildInfo` **must** be a valid pointer to a valid VkMicromapBuildInfoEXT structure
- VUID-vkGetMicromapBuildSizesEXT-pSizeInfo-parameter
  `pSizeInfo` **must** be a valid pointer to a VkMicromapBuildSizesInfoEXT structure

The `VkMicromapBuildSizesInfoEXT` structure describes the required build sizes for a micromap and scratch buffers and is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkMicromapBuildSizesInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkDeviceSize       micromapSize;
    VkDeviceSize       buildScratchSize;
    VkBool32           discardable;
} VkMicromapBuildSizesInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `micromapSize` is the size in bytes required in a VkMicromapEXT for a build or update operation.
- `buildScratchSize` is the size in bytes required in a scratch buffer for a build operation.
- `discardable` indicates whether or not the micromap object may be destroyed after an acceleration structure build or update. A false value means that acceleration structures built with this micromap **may** contain references to the data contained therein, and the application **must** not destroy the micromap until ray traversal has concluded. A true value means that the information in the micromap will be copied by value into the acceleration structure, and the micromap **may** be destroyed after the acceleration structure build concludes.

Valid Usage (Implicit)

- VUID-VkMicromapBuildSizesInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MICROMAP_BUILD_SIZES_INFO_EXT`
- VUID-VkMicromapBuildSizesInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`

Values which **can** be set in VkMicromapCreateInfoEXT::type specifying the type of micromap, are:

```
// Provided by VK_EXT_opacity_micromap
typedef enum VkMicromapTypeEXT {
    VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT = 0,
} VkMicromapTypeEXT;
```

- VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT is a micromap containing data to control the opacity of a triangle.

Bits which **can** be set in VkMicromapCreateInfoEXT::createFlags, specifying additional creation parameters for micromaps, are:

```
// Provided by VK_EXT_opacity_micromap
typedef enum VkMicromapCreateFlagBitsEXT {
    VK_MICROMAP_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_EXT = 0x00000001,
} VkMicromapCreateFlagBitsEXT;
```

- VK_MICROMAP_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_EXT specifies that the micromap's address **can** be saved and reused on a subsequent run.

```
// Provided by VK_EXT_opacity_micromap
typedef VkFlags VkMicromapCreateFlagsEXT;
```

VkMicromapCreateFlagsEXT is a bitmask type for setting a mask of zero or more VkMicromapCreateFlagBitsEXT.

Bits which **can** be set in VkMicromapBuildInfoEXT::flags specifying additional parameters for micromap builds, are:

```
// Provided by VK_EXT_opacity_micromap
typedef enum VkBuildMicromapFlagBitsEXT {
    VK_BUILD_MICROMAP_PREFER_FAST_TRACE_BIT_EXT = 0x00000001,
    VK_BUILD_MICROMAP_PREFER_FAST_BUILD_BIT_EXT = 0x00000002,
    VK_BUILD_MICROMAP_ALLOW_COMPACTION_BIT_EXT = 0x00000004,
} VkBuildMicromapFlagBitsEXT;
```

- VK_BUILD_MICROMAP_PREFER_FAST_TRACE_BIT_EXT specifies that the given micromap build **should** prioritize trace performance over build time.
- VK_BUILD_MICROMAP_PREFER_FAST_BUILD_BIT_EXT specifies that the given micromap build **should** prioritize build time over trace performance.

```
// Provided by VK_EXT_opacity_micromap
typedef VkFlags VkBuildMicromapFlagsEXT;
```

VkBuildMicromapFlagsEXT is a bitmask type for setting a mask of zero or more VkBuildMicromapFlagBitsEXT.

To destroy a micromap, call:

```
// Provided by VK_EXT_opacity_micromap
void vkDestroyMicromapEXT(
    VkDevice                                    device,
    VkMicromapEXT                               micromap,
    const VkAllocationCallbacks*                pAllocator);
```

- device is the logical device that destroys the micromap.
- micromap is the micromap to destroy.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyMicromapEXT-micromap-07441
  All submitted commands that refer to micromap **must** have completed execution
- VUID-vkDestroyMicromapEXT-micromap-07442
  If VkAllocationCallbacks were provided when micromap was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyMicromapEXT-micromap-07443
  If no VkAllocationCallbacks were provided when micromap was created, pAllocator **must** be NULL

Valid Usage (Implicit)

- VUID-vkDestroyMicromapEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkDestroyMicromapEXT-micromap-parameter
  If micromap is not VK_NULL_HANDLE, micromap **must** be a valid VkMicromapEXT handle
- VUID-vkDestroyMicromapEXT-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyMicromapEXT-micromap-parent
  If micromap is a valid handle, it **must** have been created, allocated, or retrieved from device

Host Synchronization

- Host access to micromap **must** be externally synchronized

12.8. Resource Memory Association

Resources are initially created as *virtual allocations* with no backing memory. Device memory is allocated separately (see Device Memory) and then associated with the resource. This association is done differently for sparse and non-sparse resources.

Resources created with any of the sparse creation flags are considered sparse resources. Resources created without these flags are non-sparse. The details on resource memory association for sparse resources is described in Sparse Resources.

Non-sparse resources **must** be bound completely and contiguously to a single `VkDeviceMemory` object before the resource is passed as a parameter to any of the following operations:

- creating image or buffer views
- updating descriptor sets
- recording commands in a command buffer

Once bound, the memory binding is immutable for the lifetime of the resource.

In a logical device representing more than one physical device, buffer and image resources exist on all physical devices but **can** be bound to memory differently on each. Each such replicated resource is an *instance* of the resource. For sparse resources, each instance **can** be bound to memory arbitrarily differently. For non-sparse resources, each instance **can** either be bound to the local or a peer instance of the memory, or for images **can** be bound to rectangular regions from the local and/or peer instances. When a resource is used in a descriptor set, each physical device interprets the descriptor according to its own instance's binding to memory.

NOTE	There are no new copy commands to transfer data between physical devices. Instead, an application **can** create a resource with a peer mapping and use it as the source or destination of a transfer command executed by a single physical device to copy the data from one physical device to another.

To determine the memory requirements for a buffer resource, call:

```
// Provided by VK_VERSION_1_0
void vkGetBufferMemoryRequirements(
    VkDevice                                    device,
    VkBuffer                                    buffer,
    VkMemoryRequirements*                       pMemoryRequirements);
```

- `device` is the logical device that owns the buffer.
- `buffer` is the buffer to query.
- `pMemoryRequirements` is a pointer to a VkMemoryRequirements structure in which the memory requirements of the buffer object are returned.

Valid Usage (Implicit)

- VUID-vkGetBufferMemoryRequirements-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetBufferMemoryRequirements-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle
- VUID-vkGetBufferMemoryRequirements-pMemoryRequirements-parameter
  `pMemoryRequirements` **must** be a valid pointer to a VkMemoryRequirements structure
- VUID-vkGetBufferMemoryRequirements-buffer-parent
  `buffer` **must** have been created, allocated, or retrieved from `device`

To determine the memory requirements for an image resource which is not created with the `VK_IMAGE_CREATE_DISJOINT_BIT` flag set, call:

```
// Provided by VK_VERSION_1_0
void vkGetImageMemoryRequirements(
    VkDevice                                    device,
    VkImage                                     image,
    VkMemoryRequirements*                       pMemoryRequirements);
```

- `device` is the logical device that owns the image.
- `image` is the image to query.
- `pMemoryRequirements` is a pointer to a VkMemoryRequirements structure in which the memory requirements of the image object are returned.

Valid Usage

- VUID-vkGetImageMemoryRequirements-image-01588
  `image` **must** not have been created with the `VK_IMAGE_CREATE_DISJOINT_BIT` flag set

Valid Usage (Implicit)

- VUID-vkGetImageMemoryRequirements-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetImageMemoryRequirements-image-parameter
  `image` **must** be a valid VkImage handle
- VUID-vkGetImageMemoryRequirements-pMemoryRequirements-parameter
  `pMemoryRequirements` **must** be a valid pointer to a VkMemoryRequirements structure
- VUID-vkGetImageMemoryRequirements-image-parent
  `image` **must** have been created, allocated, or retrieved from `device`

The VkMemoryRequirements structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkMemoryRequirements {
    VkDeviceSize    size;
    VkDeviceSize    alignment;
    uint32_t        memoryTypeBits;
} VkMemoryRequirements;
```

- size is the size, in bytes, of the memory allocation **required** for the resource.
- alignment is the alignment, in bytes, of the offset within the allocation **required** for the resource.
- memoryTypeBits is a bitmask and contains one bit set for every supported memory type for the resource. Bit i is set if and only if the memory type i in the VkPhysicalDeviceMemoryProperties structure for the physical device is supported for the resource.

If the resource being queried was created with the VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT, VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT, or VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT external memory handle type, the value of size has no meaning and **should** be ignored.

The implementation guarantees certain properties about the memory requirements returned by vkGetBufferMemoryRequirements2KHR, vkGetImageMemoryRequirements2KHR, vkGetDeviceBufferMemoryRequirementsKHR, vkGetDeviceImageMemoryRequirementsKHR, vkGetBufferMemoryRequirements and vkGetImageMemoryRequirements:

- The memoryTypeBits member always contains at least one bit set.
- If buffer is a VkBuffer not created with the VK_BUFFER_CREATE_SPARSE_BINDING_BIT or VK_BUFFER_CREATE_PROTECTED_BIT bits set, or if image is a linear image that was not created with the VK_IMAGE_CREATE_PROTECTED_BIT bit set, then the memoryTypeBits member always contains at least one bit set corresponding to a VkMemoryType with a propertyFlags that has both the VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT bit and the VK_MEMORY_PROPERTY_HOST_COHERENT_BIT bit set. In other words, mappable coherent memory **can** always be attached to these objects.
- If buffer was created with VkExternalMemoryBufferCreateInfo::handleTypes set to 0 or image was created with VkExternalMemoryImageCreateInfo::handleTypes set to 0, the memoryTypeBits member always contains at least one bit set corresponding to a VkMemoryType with a propertyFlags that has the VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT bit set.
- The memoryTypeBits member is identical for all VkBuffer objects created with the same value for the flags and usage members in the VkBufferCreateInfo structure and the handleTypes member of the VkExternalMemoryBufferCreateInfo structure passed to vkCreateBuffer. Further, if usage1 and usage2 of type VkBufferUsageFlags are such that the bits set in usage2 are a subset of the bits set in usage1, and they have the same flags and VkExternalMemoryBufferCreateInfo ::handleTypes, then the bits set in memoryTypeBits returned for usage1 **must** be a subset of the bits set in memoryTypeBits returned for usage2, for all values of flags.

- The alignment member is a power of two.
- The alignment member is identical for all VkBuffer objects created with the same combination of values for the usage and flags members in the VkBufferCreateInfo structure passed to vkCreateBuffer.
- If the maintenance4 feature is enabled, then the alignment member is identical for all VkImage objects created with the same combination of values for the flags, imageType, format, extent, mipLevels, arrayLayers, samples, tiling and usage members in the VkImageCreateInfo structure passed to vkCreateImage.
- The alignment member satisfies the buffer descriptor offset alignment requirements associated with the VkBuffer's usage:
  - If usage included VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT or VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT, alignment **must** be an integer multiple of VkPhysicalDeviceLimits::minTexelBufferOffsetAlignment.
  - If usage included VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT, alignment **must** be an integer multiple of VkPhysicalDeviceLimits::minUniformBufferOffsetAlignment.
  - If usage included VK_BUFFER_USAGE_STORAGE_BUFFER_BIT, alignment **must** be an integer multiple of VkPhysicalDeviceLimits::minStorageBufferOffsetAlignment.
- For images created with a color format, the memoryTypeBits member is identical for all VkImage objects created with the same combination of values for the tiling member, the VK_IMAGE_CREATE_SPARSE_BINDING_BIT bit of the flags member, the VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT bit of the flags member, the VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT bit of the usage member if the VkPhysicalDeviceHostImageCopyPropertiesEXT::identicalMemoryTypeRequirements property is VK_FALSE, handleTypes member of VkExternalMemoryImageCreateInfo, and the VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT of the usage member in the VkImageCreateInfo structure passed to vkCreateImage.
- For images created with a depth/stencil format, the memoryTypeBits member is identical for all VkImage objects created with the same combination of values for the format member, the tiling member, the VK_IMAGE_CREATE_SPARSE_BINDING_BIT bit of the flags member, the VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT bit of the flags member, the VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT bit of the usage member if the VkPhysicalDeviceHostImageCopyPropertiesEXT::identicalMemoryTypeRequirements property is VK_FALSE, handleTypes member of VkExternalMemoryImageCreateInfo, and the VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT of the usage member in the VkImageCreateInfo structure passed to vkCreateImage.
- If the memory requirements are for a VkImage, the memoryTypeBits member **must** not refer to a VkMemoryType with a propertyFlags that has the VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT bit set if the image did not have VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT bit set in the usage member of the VkImageCreateInfo structure passed to vkCreateImage.
- If the memory requirements are for a VkBuffer, the memoryTypeBits member **must** not refer to a VkMemoryType with a propertyFlags that has the VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT bit set.

NOTE	The implication of this requirement is that lazily allocated memory is disallowed

for buffers in all cases.

- The `size` member is identical for all `VkBuffer` objects created with the same combination of creation parameters specified in VkBufferCreateInfo and its `pNext` chain.
- The `size` member is identical for all `VkImage` objects created with the same combination of creation parameters specified in VkImageCreateInfo and its `pNext` chain.

> **NOTE** This, however, does not imply that they interpret the contents of the bound memory identically with each other. That additional guarantee, however, **can** be explicitly requested using `VK_IMAGE_CREATE_ALIAS_BIT`.

- If the `maintenance4` feature is enabled, these additional guarantees apply:
  - For a `VkBuffer`, the `size` memory requirement is never greater than that of another `VkBuffer` created with a greater or equal `size` specified in VkBufferCreateInfo, all other creation parameters being identical.
  - For a `VkBuffer`, the `size` memory requirement is never greater than the result of aligning VkBufferCreateInfo::`size` with the `alignment` memory requirement.
  - For a VkImage, the `size` memory requirement is never greater than that of another VkImage created with a greater or equal value in each of `extent.width`, `extent.height`, and `extent.depth`; all other creation parameters being identical.
  - The memory requirements returned by vkGetDeviceBufferMemoryRequirementsKHR are identical to those that would be returned by vkGetBufferMemoryRequirements2KHR if it were called with a `VkBuffer` created with the same VkBufferCreateInfo values.
  - The memory requirements returned by vkGetDeviceImageMemoryRequirementsKHR are identical to those that would be returned by vkGetImageMemoryRequirements2KHR if it were called with a `VkImage` created with the same VkImageCreateInfo values.

To determine the memory requirements for a buffer resource, call:

```
// Provided by VK_KHR_get_memory_requirements2
void vkGetBufferMemoryRequirements2KHR(
    VkDevice                                    device,
    const VkBufferMemoryRequirementsInfo2*      pInfo,
    VkMemoryRequirements2*                      pMemoryRequirements);
```

- `device` is the logical device that owns the buffer.
- `pInfo` is a pointer to a VkBufferMemoryRequirementsInfo2 structure containing parameters required for the memory requirements query.
- `pMemoryRequirements` is a pointer to a VkMemoryRequirements2 structure in which the memory requirements of the buffer object are returned.

Valid Usage (Implicit)

- VUID-vkGetBufferMemoryRequirements2-device-parameter

`device` **must** be a valid VkDevice handle

- VUID-vkGetBufferMemoryRequirements2-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkBufferMemoryRequirementsInfo2 structure
- VUID-vkGetBufferMemoryRequirements2-pMemoryRequirements-parameter
  `pMemoryRequirements` **must** be a valid pointer to a VkMemoryRequirements2 structure

To determine the memory requirements for a buffer resource without creating an object, call:

```
// Provided by VK_KHR_maintenance4
void vkGetDeviceBufferMemoryRequirementsKHR(
    VkDevice                                    device,
    const VkDeviceBufferMemoryRequirements*     pInfo,
    VkMemoryRequirements2*                      pMemoryRequirements);
```

- `device` is the logical device intended to own the buffer.
- `pInfo` is a pointer to a VkDeviceBufferMemoryRequirements structure containing parameters required for the memory requirements query.
- `pMemoryRequirements` is a pointer to a VkMemoryRequirements2 structure in which the memory requirements of the buffer object are returned.

Valid Usage (Implicit)

- VUID-vkGetDeviceBufferMemoryRequirements-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceBufferMemoryRequirements-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkDeviceBufferMemoryRequirements structure
- VUID-vkGetDeviceBufferMemoryRequirements-pMemoryRequirements-parameter
  `pMemoryRequirements` **must** be a valid pointer to a VkMemoryRequirements2 structure

The `VkBufferMemoryRequirementsInfo2` structure is defined as:

```
typedef struct VkBufferMemoryRequirementsInfo2 {
    VkStructureType    sType;
    const void*        pNext;
    VkBuffer           buffer;
} VkBufferMemoryRequirementsInfo2;
```

or the equivalent

```
// Provided by VK_KHR_get_memory_requirements2
typedef VkBufferMemoryRequirementsInfo2 VkBufferMemoryRequirementsInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `buffer` is the buffer to query.

Valid Usage (Implicit)

- VUID-VkBufferMemoryRequirementsInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BUFFER_MEMORY_REQUIREMENTS_INFO_2`
- VUID-VkBufferMemoryRequirementsInfo2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkBufferMemoryRequirementsInfo2-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle

The `VkDeviceBufferMemoryRequirements` structure is defined as:

```
typedef struct VkDeviceBufferMemoryRequirements {
    VkStructureType              sType;
    const void*                  pNext;
    const VkBufferCreateInfo*    pCreateInfo;
} VkDeviceBufferMemoryRequirements;
```

or the equivalent

```
// Provided by VK_KHR_maintenance4
typedef VkDeviceBufferMemoryRequirements VkDeviceBufferMemoryRequirementsKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pCreateInfo` is a pointer to a VkBufferCreateInfo structure containing parameters affecting creation of the buffer to query.

Valid Usage (Implicit)

- VUID-VkDeviceBufferMemoryRequirements-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_BUFFER_MEMORY_REQUIREMENTS`
- VUID-VkDeviceBufferMemoryRequirements-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDeviceBufferMemoryRequirements-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkBufferCreateInfo structure

To determine the memory requirements for an image resource, call:

```
// Provided by VK_KHR_get_memory_requirements2
void vkGetImageMemoryRequirements2KHR(
    VkDevice                                    device,
    const VkImageMemoryRequirementsInfo2*       pInfo,
    VkMemoryRequirements2*                      pMemoryRequirements);
```

- `device` is the logical device that owns the image.
- `pInfo` is a pointer to a VkImageMemoryRequirementsInfo2 structure containing parameters required for the memory requirements query.
- `pMemoryRequirements` is a pointer to a VkMemoryRequirements2 structure in which the memory requirements of the image object are returned.

Valid Usage (Implicit)

- VUID-vkGetImageMemoryRequirements2-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetImageMemoryRequirements2-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkImageMemoryRequirementsInfo2 structure
- VUID-vkGetImageMemoryRequirements2-pMemoryRequirements-parameter
  `pMemoryRequirements` **must** be a valid pointer to a VkMemoryRequirements2 structure

To determine the memory requirements for an image resource without creating an object, call:

```
// Provided by VK_KHR_maintenance4
void vkGetDeviceImageMemoryRequirementsKHR(
    VkDevice                                    device,
    const VkDeviceImageMemoryRequirements*      pInfo,
    VkMemoryRequirements2*                      pMemoryRequirements);
```

- `device` is the logical device intended to own the image.
- `pInfo` is a pointer to a VkDeviceImageMemoryRequirements structure containing parameters required for the memory requirements query.
- `pMemoryRequirements` is a pointer to a VkMemoryRequirements2 structure in which the memory requirements of the image object are returned.

Valid Usage (Implicit)

- VUID-vkGetDeviceImageMemoryRequirements-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceImageMemoryRequirements-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkDeviceImageMemoryRequirements structure
- VUID-vkGetDeviceImageMemoryRequirements-pMemoryRequirements-parameter

pMemoryRequirements **must** be a valid pointer to a VkMemoryRequirements2 structure

The VkImageMemoryRequirementsInfo2 structure is defined as:

```
typedef struct VkImageMemoryRequirementsInfo2 {
    VkStructureType    sType;
    const void*        pNext;
    VkImage            image;
} VkImageMemoryRequirementsInfo2;
```

or the equivalent

```
// Provided by VK_KHR_get_memory_requirements2
typedef VkImageMemoryRequirementsInfo2 VkImageMemoryRequirementsInfo2KHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- image is the image to query.

Valid Usage

- VUID-VkImageMemoryRequirementsInfo2-image-01589
  If image was created with a *multi-planar* format and the VK_IMAGE_CREATE_DISJOINT_BIT flag, there **must** be a VkImagePlaneMemoryRequirementsInfo included in the pNext chain of the VkImageMemoryRequirementsInfo2 structure
- VUID-VkImageMemoryRequirementsInfo2-image-02279
  If image was created with VK_IMAGE_CREATE_DISJOINT_BIT and with VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then there **must** be a VkImagePlaneMemoryRequirementsInfo included in the pNext chain of the VkImageMemoryRequirementsInfo2 structure
- VUID-VkImageMemoryRequirementsInfo2-image-01590
  If image was not created with the VK_IMAGE_CREATE_DISJOINT_BIT flag, there **must** not be a VkImagePlaneMemoryRequirementsInfo included in the pNext chain of the VkImageMemoryRequirementsInfo2 structure
- VUID-VkImageMemoryRequirementsInfo2-image-02280
  If image was created with a single-plane format and with any tiling other than VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then there **must** not be a VkImagePlaneMemoryRequirementsInfo included in the pNext chain of the VkImageMemoryRequirementsInfo2 structure

Valid Usage (Implicit)

- VUID-VkImageMemoryRequirementsInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_MEMORY_REQUIREMENTS_INFO_2`
- VUID-VkImageMemoryRequirementsInfo2-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkImagePlaneMemoryRequirementsInfo
- VUID-VkImageMemoryRequirementsInfo2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkImageMemoryRequirementsInfo2-image-parameter
  `image` **must** be a valid VkImage handle

The `VkDeviceImageMemoryRequirements` structure is defined as:

```
typedef struct VkDeviceImageMemoryRequirements {
    VkStructureType             sType;
    const void*                 pNext;
    const VkImageCreateInfo*    pCreateInfo;
    VkImageAspectFlagBits       planeAspect;
} VkDeviceImageMemoryRequirements;
```

or the equivalent

```
// Provided by VK_KHR_maintenance4
typedef VkDeviceImageMemoryRequirements VkDeviceImageMemoryRequirementsKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pCreateInfo` is a pointer to a VkImageCreateInfo structure containing parameters affecting creation of the image to query.
- `planeAspect` is a VkImageAspectFlagBits value specifying the aspect corresponding to the image plane to query. This parameter is ignored unless `pCreateInfo->tiling` is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`, or `pCreateInfo->flags` has `VK_IMAGE_CREATE_DISJOINT_BIT` set.

Valid Usage

- VUID-VkDeviceImageMemoryRequirements-pCreateInfo-06416
  The `pCreateInfo->pNext` chain **must** not contain a VkImageSwapchainCreateInfoKHR structure
- VUID-VkDeviceImageMemoryRequirements-pCreateInfo-06776
  The `pCreateInfo->pNext` chain **must** not contain a

VkImageDrmFormatModifierExplicitCreateInfoEXT structure

- VUID-VkDeviceImageMemoryRequirements-pCreateInfo-06417
  If pCreateInfo->format specifies a *multi-planar* format and pCreateInfo->flags has VK_IMAGE_CREATE_DISJOINT_BIT set then planeAspect **must** not be VK_IMAGE_ASPECT_NONE_KHR
- VUID-VkDeviceImageMemoryRequirements-pCreateInfo-06419
  If pCreateInfo->flags has VK_IMAGE_CREATE_DISJOINT_BIT set and if the pCreateInfo->tiling is VK_IMAGE_TILING_LINEAR or VK_IMAGE_TILING_OPTIMAL, then planeAspect **must** be a single valid multi-planar aspect mask bit
- VUID-VkDeviceImageMemoryRequirements-pCreateInfo-06420
  If pCreateInfo->tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then planeAspect **must** be a single valid *memory plane* for the image (that is, aspectMask **must** specify a plane index that is less than the VkDrmFormatModifierPropertiesEXT::drmFormatModifierPlaneCount associated with the image's format and VkImageDrmFormatModifierPropertiesEXT::drmFormatModifier)

Valid Usage (Implicit)

- VUID-VkDeviceImageMemoryRequirements-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEVICE_IMAGE_MEMORY_REQUIREMENTS
- VUID-VkDeviceImageMemoryRequirements-pNext-pNext
  pNext **must** be NULL
- VUID-VkDeviceImageMemoryRequirements-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkImageCreateInfo structure
- VUID-VkDeviceImageMemoryRequirements-planeAspect-parameter
  If planeAspect is not 0, planeAspect **must** be a valid VkImageAspectFlagBits value

To determine the memory requirements for a plane of a disjoint image, add a VkImagePlaneMemoryRequirementsInfo structure to the pNext chain of the VkImageMemoryRequirementsInfo2 structure.

The VkImagePlaneMemoryRequirementsInfo structure is defined as:

```
typedef struct VkImagePlaneMemoryRequirementsInfo {
    VkStructureType          sType;
    const void*              pNext;
    VkImageAspectFlagBits    planeAspect;
} VkImagePlaneMemoryRequirementsInfo;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkImagePlaneMemoryRequirementsInfo VkImagePlaneMemoryRequirementsInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `planeAspect` is a VkImageAspectFlagBits value specifying the aspect corresponding to the image plane to query.

Valid Usage

- VUID-VkImagePlaneMemoryRequirementsInfo-planeAspect-02281
  If the image's `tiling` is `VK_IMAGE_TILING_LINEAR` or `VK_IMAGE_TILING_OPTIMAL`, then `planeAspect` **must** be a single valid multi-planar aspect mask bit
- VUID-VkImagePlaneMemoryRequirementsInfo-planeAspect-02282
  If the image's `tiling` is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`, then `planeAspect` **must** be a single valid *memory plane* for the image (that is, `aspectMask` **must** specify a plane index that is less than the VkDrmFormatModifierPropertiesEXT::`drmFormatModifierPlaneCount` associated with the image's `format` and VkImageDrmFormatModifierPropertiesEXT::`drmFormatModifier`)

Valid Usage (Implicit)

- VUID-VkImagePlaneMemoryRequirementsInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_PLANE_MEMORY_REQUIREMENTS_INFO`
- VUID-VkImagePlaneMemoryRequirementsInfo-planeAspect-parameter
  `planeAspect` **must** be a valid VkImageAspectFlagBits value

The `VkMemoryRequirements2` structure is defined as:

```
typedef struct VkMemoryRequirements2 {
    VkStructureType         sType;
    void*                   pNext;
    VkMemoryRequirements    memoryRequirements;
} VkMemoryRequirements2;
```

or the equivalent

```
// Provided by VK_KHR_get_memory_requirements2
typedef VkMemoryRequirements2 VkMemoryRequirements2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memoryRequirements` is a VkMemoryRequirements structure describing the memory requirements of the resource.

Valid Usage (Implicit)

- VUID-VkMemoryRequirements2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_REQUIREMENTS_2`
- VUID-VkMemoryRequirements2-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkMemoryDedicatedRequirements
- VUID-VkMemoryRequirements2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

The `VkMemoryDedicatedRequirements` structure is defined as:

```
typedef struct VkMemoryDedicatedRequirements {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           prefersDedicatedAllocation;
    VkBool32           requiresDedicatedAllocation;
} VkMemoryDedicatedRequirements;
```

or the equivalent

```
// Provided by VK_KHR_dedicated_allocation
typedef VkMemoryDedicatedRequirements VkMemoryDedicatedRequirementsKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `prefersDedicatedAllocation` specifies that the implementation would prefer a dedicated allocation for this resource. The application is still free to suballocate the resource but it **may** get better performance if a dedicated allocation is used.
- `requiresDedicatedAllocation` specifies that a dedicated allocation is required for this resource.

To determine the dedicated allocation requirements of a buffer or image resource, add a VkMemoryDedicatedRequirements structure to the `pNext` chain of the VkMemoryRequirements2 structure passed as the `pMemoryRequirements` parameter of vkGetBufferMemoryRequirements2KHR or vkGetImageMemoryRequirements2KHR, respectively.

Constraints on the values returned for buffer resources are:

- `requiresDedicatedAllocation` **may** be `VK_TRUE` if the `pNext` chain of VkBufferCreateInfo for the call to `vkCreateBuffer` used to create the buffer being queried included a VkExternalMemoryBufferCreateInfo structure, and any of the handle types specified in VkExternalMemoryBufferCreateInfo::`handleTypes` requires dedicated allocation, as reported by vkGetPhysicalDeviceExternalBufferPropertiesKHR in `VkExternalBufferProperties`::`externalMemoryProperties.externalMemoryFeatures`. Otherwise, `requiresDedicatedAllocation` will be `VK_FALSE`.

- When the implementation sets `requiresDedicatedAllocation` to `VK_TRUE`, it **must** also set `prefersDedicatedAllocation` to `VK_TRUE`.
- If `VK_BUFFER_CREATE_SPARSE_BINDING_BIT` was set in VkBufferCreateInfo::`flags` when `buffer` was created, then both `prefersDedicatedAllocation` and `requiresDedicatedAllocation` will be `VK_FALSE`.

Constraints on the values returned for image resources are:

- `requiresDedicatedAllocation` **may** be `VK_TRUE` if the `pNext` chain of VkImageCreateInfo for the call to vkCreateImage used to create the image being queried included a VkExternalMemoryImageCreateInfo structure, and any of the handle types specified in VkExternalMemoryImageCreateInfo::`handleTypes` requires dedicated allocation, as reported by vkGetPhysicalDeviceImageFormatProperties2KHR in `VkExternalImageFormatProperties`::`externalMemoryProperties.externalMemoryFeatures`.
- `requiresDedicatedAllocation` **may** be `VK_TRUE` if the image's tiling is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`.
- `requiresDedicatedAllocation` will otherwise be `VK_FALSE`
- If `VK_IMAGE_CREATE_SPARSE_BINDING_BIT` was set in VkImageCreateInfo::`flags` when `image` was created, then both `prefersDedicatedAllocation` and `requiresDedicatedAllocation` will be `VK_FALSE`.

Valid Usage (Implicit)

- VUID-VkMemoryDedicatedRequirements-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_DEDICATED_REQUIREMENTS`

To attach memory to a buffer object, call:

```
// Provided by VK_VERSION_1_0
VkResult vkBindBufferMemory(
    VkDevice                                    device,
    VkBuffer                                    buffer,
    VkDeviceMemory                              memory,
    VkDeviceSize                                memoryOffset);
```

- `device` is the logical device that owns the buffer and memory.
- `buffer` is the buffer to be attached to memory.
- `memory` is a VkDeviceMemory object describing the device memory to attach.
- `memoryOffset` is the start offset of the region of `memory` which is to be bound to the buffer. The number of bytes returned in the `VkMemoryRequirements`::`size` member in `memory`, starting from `memoryOffset` bytes, will be bound to the specified buffer.

`vkBindBufferMemory` is equivalent to passing the same parameters through VkBindBufferMemoryInfo to vkBindBufferMemory2KHR.

Valid Usage

- VUID-vkBindBufferMemory-buffer-07459
  `buffer` **must** not have been bound to a memory object
- VUID-vkBindBufferMemory-buffer-01030
  `buffer` **must** not have been created with any sparse memory binding flags
- VUID-vkBindBufferMemory-memoryOffset-01031
  `memoryOffset` **must** be less than the size of `memory`
- VUID-vkBindBufferMemory-memory-01035
  `memory` **must** have been allocated using one of the memory types allowed in the `memoryTypeBits` member of the `VkMemoryRequirements` structure returned from a call to `vkGetBufferMemoryRequirements` with `buffer`
- VUID-vkBindBufferMemory-memoryOffset-01036
  `memoryOffset` **must** be an integer multiple of the `alignment` member of the `VkMemoryRequirements` structure returned from a call to `vkGetBufferMemoryRequirements` with `buffer`
- VUID-vkBindBufferMemory-size-01037
  The `size` member of the `VkMemoryRequirements` structure returned from a call to `vkGetBufferMemoryRequirements` with `buffer` **must** be less than or equal to the size of `memory` minus `memoryOffset`
- VUID-vkBindBufferMemory-buffer-01444
  If `buffer` requires a dedicated allocation (as reported by vkGetBufferMemoryRequirements2KHR in VkMemoryDedicatedRequirements::`requiresDedicatedAllocation` for `buffer`), `memory` **must** have been allocated with VkMemoryDedicatedAllocateInfo::`buffer` equal to `buffer`
- VUID-vkBindBufferMemory-memory-01508
  If the `VkMemoryAllocateInfo` provided when `memory` was allocated included a VkMemoryDedicatedAllocateInfo structure in its `pNext` chain, and VkMemoryDedicatedAllocateInfo::`buffer` was not VK_NULL_HANDLE, then `buffer` **must** equal VkMemoryDedicatedAllocateInfo::`buffer`, and `memoryOffset` **must** be zero
- VUID-vkBindBufferMemory-memory-02726
  If the value of VkExportMemoryAllocateInfo::`handleTypes` used to allocate `memory` is not `0`, it **must** include at least one of the handles set in VkExternalMemoryBufferCreateInfo::`handleTypes` when `buffer` was created
- VUID-vkBindBufferMemory-memory-02985
  If `memory` was allocated by a memory import operation, the external handle type of the imported memory **must** also have been set in VkExternalMemoryBufferCreateInfo::`handleTypes` when `buffer` was created
- VUID-vkBindBufferMemory-bufferDeviceAddress-03339
  If the VkPhysicalDeviceBufferDeviceAddressFeatures::`bufferDeviceAddress` feature is enabled and `buffer` was created with the `VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT` bit set, `memory` **must** have been allocated with the `VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT` bit set

- VUID-vkBindBufferMemory-bufferDeviceAddressCaptureReplay-09200
  If the VkPhysicalDeviceBufferDeviceAddressFeatures::bufferDeviceAddressCaptureReplay feature is enabled and buffer was created with the VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT bit set, memory **must** have been allocated with the VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT bit set

Valid Usage (Implicit)

- VUID-vkBindBufferMemory-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkBindBufferMemory-buffer-parameter
  buffer **must** be a valid VkBuffer handle
- VUID-vkBindBufferMemory-memory-parameter
  memory **must** be a valid VkDeviceMemory handle
- VUID-vkBindBufferMemory-buffer-parent
  buffer **must** have been created, allocated, or retrieved from device
- VUID-vkBindBufferMemory-memory-parent
  memory **must** have been created, allocated, or retrieved from device

Host Synchronization

- Host access to buffer **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR

To attach memory to buffer objects for one or more buffers at a time, call:

```
// Provided by VK_KHR_bind_memory2
VkResult vkBindBufferMemory2KHR(
    VkDevice                                    device,
    uint32_t                                    bindInfoCount,
    const VkBindBufferMemoryInfo*               pBindInfos);
```

- `device` is the logical device that owns the buffers and memory.
- `bindInfoCount` is the number of elements in `pBindInfos`.
- `pBindInfos` is a pointer to an array of `bindInfoCount` VkBindBufferMemoryInfo structures describing buffers and memory to bind.

On some implementations, it **may** be more efficient to batch memory bindings into a single command.

If the `maintenance6` feature is enabled, this command **must** attempt to perform all of the memory binding operations described by `pBindInfos`, and **must** not early exit on the first failure.

If any of the memory binding operations described by `pBindInfos` fail, the VkResult returned by this command **must** be the return value of any one of the memory binding operations which did not return `VK_SUCCESS`.

NOTE

If the `vkBindBufferMemory2KHR` command failed, VkBindMemoryStatusKHR structures were not included in the `pNext` chains of each element of `pBindInfos`, and `bindInfoCount` was greater than one, then the buffers referenced by `pBindInfos` will be in an indeterminate state, and must not be used.

Applications should destroy these buffers.

Valid Usage (Implicit)

- VUID-vkBindBufferMemory2-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkBindBufferMemory2-pBindInfos-parameter
  `pBindInfos` **must** be a valid pointer to an array of `bindInfoCount` valid VkBindBufferMemoryInfo structures
- VUID-vkBindBufferMemory2-bindInfoCount-arraylength
  `bindInfoCount` **must** be greater than `0`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR`

`VkBindBufferMemoryInfo` contains members corresponding to the parameters of vkBindBufferMemory.

The VkBindBufferMemoryInfo structure is defined as:

```
typedef struct VkBindBufferMemoryInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkBuffer           buffer;
    VkDeviceMemory     memory;
    VkDeviceSize       memoryOffset;
} VkBindBufferMemoryInfo;
```

or the equivalent

```
// Provided by VK_KHR_bind_memory2
typedef VkBindBufferMemoryInfo VkBindBufferMemoryInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- buffer is the buffer to be attached to memory.
- memory is a VkDeviceMemory object describing the device memory to attach.
- memoryOffset is the start offset of the region of memory which is to be bound to the buffer. The number of bytes returned in the VkMemoryRequirements::size member in memory, starting from memoryOffset bytes, will be bound to the specified buffer.

Valid Usage

- VUID-VkBindBufferMemoryInfo-buffer-07459
  buffer **must** not have been bound to a memory object
- VUID-VkBindBufferMemoryInfo-buffer-01030
  buffer **must** not have been created with any sparse memory binding flags
- VUID-VkBindBufferMemoryInfo-memoryOffset-01031
  memoryOffset **must** be less than the size of memory
- VUID-VkBindBufferMemoryInfo-memory-01035
  memory **must** have been allocated using one of the memory types allowed in the memoryTypeBits member of the VkMemoryRequirements structure returned from a call to vkGetBufferMemoryRequirements with buffer
- VUID-VkBindBufferMemoryInfo-memoryOffset-01036
  memoryOffset **must** be an integer multiple of the alignment member of the VkMemoryRequirements structure returned from a call to vkGetBufferMemoryRequirements with buffer
- VUID-VkBindBufferMemoryInfo-size-01037
  The size member of the VkMemoryRequirements structure returned from a call to vkGetBufferMemoryRequirements with buffer **must** be less than or equal to the size of memory minus memoryOffset

- VUID-VkBindBufferMemoryInfo-buffer-01444
  If `buffer` requires a dedicated allocation (as reported by vkGetBufferMemoryRequirements2KHR in VkMemoryDedicatedRequirements::`requiresDedicatedAllocation` for `buffer`), `memory` **must** have been allocated with VkMemoryDedicatedAllocateInfo::`buffer` equal to `buffer`
- VUID-VkBindBufferMemoryInfo-memory-01508
  If the `VkMemoryAllocateInfo` provided when `memory` was allocated included a VkMemoryDedicatedAllocateInfo structure in its `pNext` chain, and VkMemoryDedicatedAllocateInfo::`buffer` was not VK_NULL_HANDLE, then `buffer` **must** equal VkMemoryDedicatedAllocateInfo::`buffer`, and `memoryOffset` **must** be zero
- VUID-VkBindBufferMemoryInfo-memory-02726
  If the value of VkExportMemoryAllocateInfo::`handleTypes` used to allocate `memory` is not `0`, it **must** include at least one of the handles set in VkExternalMemoryBufferCreateInfo::`handleTypes` when `buffer` was created
- VUID-VkBindBufferMemoryInfo-memory-02985
  If `memory` was allocated by a memory import operation, the external handle type of the imported memory **must** also have been set in VkExternalMemoryBufferCreateInfo::`handleTypes` when `buffer` was created
- VUID-VkBindBufferMemoryInfo-bufferDeviceAddress-03339
  If the VkPhysicalDeviceBufferDeviceAddressFeatures::`bufferDeviceAddress` feature is enabled and `buffer` was created with the `VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT` bit set, `memory` **must** have been allocated with the `VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT` bit set
- VUID-VkBindBufferMemoryInfo-bufferDeviceAddressCaptureReplay-09200
  If the VkPhysicalDeviceBufferDeviceAddressFeatures::`bufferDeviceAddressCaptureReplay` feature is enabled and `buffer` was created with the `VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT` bit set, `memory` **must** have been allocated with the `VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT` bit set
- VUID-VkBindBufferMemoryInfo-pNext-01605
  If the `pNext` chain includes a VkBindBufferMemoryDeviceGroupInfo structure, all instances of `memory` specified by VkBindBufferMemoryDeviceGroupInfo::`pDeviceIndices` **must** have been allocated

Valid Usage (Implicit)

- VUID-VkBindBufferMemoryInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_INFO`
- VUID-VkBindBufferMemoryInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkBindBufferMemoryDeviceGroupInfo or VkBindMemoryStatusKHR
- VUID-VkBindBufferMemoryInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

- VUID-VkBindBufferMemoryInfo-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle
- VUID-VkBindBufferMemoryInfo-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle
- VUID-VkBindBufferMemoryInfo-commonparent
  Both of `buffer`, and `memory` **must** have been created, allocated, or retrieved from the same VkDevice

The `VkBindBufferMemoryDeviceGroupInfo` structure is defined as:

```
typedef struct VkBindBufferMemoryDeviceGroupInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           deviceIndexCount;
    const uint32_t*    pDeviceIndices;
} VkBindBufferMemoryDeviceGroupInfo;
```

or the equivalent

```
// Provided by VK_KHR_bind_memory2 with VK_KHR_device_group
typedef VkBindBufferMemoryDeviceGroupInfo VkBindBufferMemoryDeviceGroupInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `deviceIndexCount` is the number of elements in `pDeviceIndices`.
- `pDeviceIndices` is a pointer to an array of device indices.

If the `pNext` chain of VkBindBufferMemoryInfo includes a `VkBindBufferMemoryDeviceGroupInfo` structure, then that structure determines how memory is bound to buffers across multiple devices in a device group.

If `deviceIndexCount` is greater than zero, then on device index i the buffer is attached to the instance of `memory` on the physical device with device index `pDeviceIndices`[i].

If `deviceIndexCount` is zero and `memory` comes from a memory heap with the `VK_MEMORY_HEAP_MULTI_INSTANCE_BIT` bit set, then it is as if `pDeviceIndices` contains consecutive indices from zero to the number of physical devices in the logical device, minus one. In other words, by default each physical device attaches to its own instance of `memory`.

If `deviceIndexCount` is zero and `memory` comes from a memory heap without the `VK_MEMORY_HEAP_MULTI_INSTANCE_BIT` bit set, then it is as if `pDeviceIndices` contains an array of zeros. In other words, by default each physical device attaches to instance zero.

Valid Usage

- VUID-VkBindBufferMemoryDeviceGroupInfo-deviceIndexCount-01606
  `deviceIndexCount` **must** either be zero or equal to the number of physical devices in the logical device
- VUID-VkBindBufferMemoryDeviceGroupInfo-pDeviceIndices-01607
  All elements of `pDeviceIndices` **must** be valid device indices

Valid Usage (Implicit)

- VUID-VkBindBufferMemoryDeviceGroupInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_DEVICE_GROUP_INFO`
- VUID-VkBindBufferMemoryDeviceGroupInfo-pDeviceIndices-parameter
  If `deviceIndexCount` is not `0`, `pDeviceIndices` **must** be a valid pointer to an array of `deviceIndexCount` `uint32_t` values

The `VkBindMemoryStatusKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance6
typedef struct VkBindMemoryStatusKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkResult*          pResult;
} VkBindMemoryStatusKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pResult` is a pointer to a `VkResult` value.

If the `pNext` chain of VkBindBufferMemoryInfo or VkBindImageMemoryInfo includes a `VkBindMemoryStatusKHR` structure, then the `VkBindMemoryStatusKHR`::`pResult` will be populated with a value describing the result of the corresponding memory binding operation.

Valid Usage (Implicit)

- VUID-VkBindMemoryStatusKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BIND_MEMORY_STATUS_KHR`
- VUID-VkBindMemoryStatusKHR-pResult-parameter
  `pResult` **must** be a valid pointer to a VkResult value

To attach memory to a `VkImage` object created without the `VK_IMAGE_CREATE_DISJOINT_BIT` set, call:

```
// Provided by VK_VERSION_1_0
VkResult vkBindImageMemory(
    VkDevice                                    device,
    VkImage                                     image,
    VkDeviceMemory                              memory,
    VkDeviceSize                                memoryOffset);
```

- `device` is the logical device that owns the image and memory.
- `image` is the image.
- `memory` is the VkDeviceMemory object describing the device memory to attach.
- `memoryOffset` is the start offset of the region of `memory` which is to be bound to the image. The number of bytes returned in the `VkMemoryRequirements`::`size` member in `memory`, starting from `memoryOffset` bytes, will be bound to the specified image.

`vkBindImageMemory` is equivalent to passing the same parameters through VkBindImageMemoryInfo to vkBindImageMemory2KHR.

Valid Usage

- VUID-vkBindImageMemory-image-07460
  `image` **must** not have been bound to a memory object
- VUID-vkBindImageMemory-image-01045
  `image` **must** not have been created with any sparse memory binding flags
- VUID-vkBindImageMemory-memoryOffset-01046
  `memoryOffset` **must** be less than the size of `memory`
- VUID-vkBindImageMemory-image-01445
  If `image` requires a dedicated allocation (as reported by vkGetImageMemoryRequirements2KHR in VkMemoryDedicatedRequirements::`requiresDedicatedAllocation` for `image`), `memory` **must** have been created with VkMemoryDedicatedAllocateInfo::`image` equal to `image`
- VUID-vkBindImageMemory-memory-02628
  If the `VkMemoryAllocateInfo` provided when `memory` was allocated included a VkMemoryDedicatedAllocateInfo structure in its `pNext` chain, and VkMemoryDedicatedAllocateInfo::`image` was not VK_NULL_HANDLE, then `image` **must** equal VkMemoryDedicatedAllocateInfo::`image` and `memoryOffset` **must** be zero
- VUID-vkBindImageMemory-memory-02728
  If the value of VkExportMemoryAllocateInfo::`handleTypes` used to allocate `memory` is not `0`, it **must** include at least one of the handles set in VkExternalMemoryImageCreateInfo::`handleTypes` when `image` was created
- VUID-vkBindImageMemory-memory-02989
  If `memory` was created by a memory import operation, the external handle type of the imported memory **must** also have been set in VkExternalMemoryImageCreateInfo::`handleTypes` when `image` was created

- VUID-vkBindImageMemory-image-01608
  `image` **must** not have been created with the `VK_IMAGE_CREATE_DISJOINT_BIT` set
- VUID-vkBindImageMemory-memory-01047
  `memory` **must** have been allocated using one of the memory types allowed in the `memoryTypeBits` member of the `VkMemoryRequirements` structure returned from a call to vkGetImageMemoryRequirements with `image`
- VUID-vkBindImageMemory-memoryOffset-01048
  `memoryOffset` **must** be an integer multiple of the `alignment` member of the `VkMemoryRequirements` structure returned from a call to vkGetImageMemoryRequirements with `image`
- VUID-vkBindImageMemory-size-01049
  The difference of the size of `memory` and `memoryOffset` **must** be greater than or equal to the `size` member of the VkMemoryRequirements structure returned from a call to vkGetImageMemoryRequirements with the same `image`

Valid Usage (Implicit)

- VUID-vkBindImageMemory-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkBindImageMemory-image-parameter
  `image` **must** be a valid VkImage handle
- VUID-vkBindImageMemory-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle
- VUID-vkBindImageMemory-image-parent
  `image` **must** have been created, allocated, or retrieved from `device`
- VUID-vkBindImageMemory-memory-parent
  `memory` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `image` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

To attach memory to image objects for one or more images at a time, call:

```
// Provided by VK_KHR_bind_memory2
VkResult vkBindImageMemory2KHR(
    VkDevice                                    device,
    uint32_t                                    bindInfoCount,
    const VkBindImageMemoryInfo*                pBindInfos);
```

- `device` is the logical device that owns the images and memory.
- `bindInfoCount` is the number of elements in `pBindInfos`.
- `pBindInfos` is a pointer to an array of VkBindImageMemoryInfo structures, describing images and memory to bind.

On some implementations, it **may** be more efficient to batch memory bindings into a single command.

If the `maintenance6` feature is enabled, this command **must** attempt to perform all of the memory binding operations described by `pBindInfos`, and **must** not early exit on the first failure.

If any of the memory binding operations described by `pBindInfos` fail, the VkResult returned by this command **must** be the return value of any one of the memory binding operations which did not return `VK_SUCCESS`.

NOTE

If the `vkBindImageMemory2KHR` command failed, VkBindMemoryStatusKHR structures were not included in the `pNext` chains of each element of `pBindInfos`, and `bindInfoCount` was greater than one, then the images referenced by `pBindInfos` will be in an indeterminate state, and must not be used.

Applications should destroy these images.

Valid Usage

- VUID-vkBindImageMemory2-pBindInfos-02858
  If any VkBindImageMemoryInfo::`image` was created with `VK_IMAGE_CREATE_DISJOINT_BIT` then all planes of VkBindImageMemoryInfo::`image` **must** be bound individually in separate `pBindInfos`
- VUID-vkBindImageMemory2-pBindInfos-04006
  `pBindInfos` **must** not refer to the same image subresource more than once

Valid Usage (Implicit)

- VUID-vkBindImageMemory2-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkBindImageMemory2-pBindInfos-parameter
  `pBindInfos` **must** be a valid pointer to an array of `bindInfoCount` valid

VkBindImageMemoryInfo structures

- VUID-vkBindImageMemory2-bindInfoCount-arraylength
  `bindInfoCount` **must** be greater than `0`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

`VkBindImageMemoryInfo` contains members corresponding to the parameters of vkBindImageMemory.

The `VkBindImageMemoryInfo` structure is defined as:

```
typedef struct VkBindImageMemoryInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkImage            image;
    VkDeviceMemory     memory;
    VkDeviceSize       memoryOffset;
} VkBindImageMemoryInfo;
```

or the equivalent

```
// Provided by VK_KHR_bind_memory2
typedef VkBindImageMemoryInfo VkBindImageMemoryInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `image` is the image to be attached to memory.
- `memory` is a VkDeviceMemory object describing the device memory to attach.
- `memoryOffset` is the start offset of the region of `memory` which is to be bound to the image. The number of bytes returned in the `VkMemoryRequirements`::`size` member in `memory`, starting from `memoryOffset` bytes, will be bound to the specified image.

Valid Usage

- VUID-VkBindImageMemoryInfo-image-07460
  `image` **must** not have been bound to a memory object

- VUID-VkBindImageMemoryInfo-image-01045
  `image` **must** not have been created with any sparse memory binding flags
- VUID-VkBindImageMemoryInfo-memoryOffset-01046
  `memoryOffset` **must** be less than the size of `memory`
- VUID-VkBindImageMemoryInfo-image-01445
  If `image` requires a dedicated allocation (as reported by vkGetImageMemoryRequirements2KHR in VkMemoryDedicatedRequirements::`requiresDedicatedAllocation` for `image`), `memory` **must** have been created with VkMemoryDedicatedAllocateInfo::`image` equal to `image`
- VUID-VkBindImageMemoryInfo-memory-02628
  If the `VkMemoryAllocateInfo` provided when `memory` was allocated included a VkMemoryDedicatedAllocateInfo structure in its `pNext` chain, and VkMemoryDedicatedAllocateInfo::`image` was not VK_NULL_HANDLE, then `image` **must** equal VkMemoryDedicatedAllocateInfo::`image` and `memoryOffset` **must** be zero
- VUID-VkBindImageMemoryInfo-memory-02728
  If the value of VkExportMemoryAllocateInfo::`handleTypes` used to allocate `memory` is not `0`, it **must** include at least one of the handles set in VkExternalMemoryImageCreateInfo::`handleTypes` when `image` was created
- VUID-VkBindImageMemoryInfo-memory-02989
  If `memory` was created by a memory import operation, the external handle type of the imported memory **must** also have been set in VkExternalMemoryImageCreateInfo::`handleTypes` when `image` was created
- VUID-VkBindImageMemoryInfo-pNext-01615
  If the `pNext` chain does not include a VkBindImagePlaneMemoryInfo structure, `memory` **must** have been allocated using one of the memory types allowed in the `memoryTypeBits` member of the VkMemoryRequirements structure returned from a call to vkGetImageMemoryRequirements2KHR with `image`
- VUID-VkBindImageMemoryInfo-pNext-01616
  If the `pNext` chain does not include a VkBindImagePlaneMemoryInfo structure, `memoryOffset` **must** be an integer multiple of the `alignment` member of the VkMemoryRequirements structure returned from a call to vkGetImageMemoryRequirements2KHR with `image`
- VUID-VkBindImageMemoryInfo-pNext-01617
  If the `pNext` chain does not include a VkBindImagePlaneMemoryInfo structure, the difference of the size of `memory` and `memoryOffset` **must** be greater than or equal to the `size` member of the VkMemoryRequirements structure returned from a call to vkGetImageMemoryRequirements2KHR with the same `image`
- VUID-VkBindImageMemoryInfo-pNext-01618
  If the `pNext` chain includes a VkBindImagePlaneMemoryInfo structure, `image` **must** have been created with the `VK_IMAGE_CREATE_DISJOINT_BIT` bit set
- VUID-VkBindImageMemoryInfo-image-07736
  If `image` was created with the `VK_IMAGE_CREATE_DISJOINT_BIT` bit set, then the `pNext` chain **must** include a VkBindImagePlaneMemoryInfo structure

- VUID-VkBindImageMemoryInfo-pNext-01619
  If the pNext chain includes a VkBindImagePlaneMemoryInfo structure, memory **must** have been allocated using one of the memory types allowed in the memoryTypeBits member of the VkMemoryRequirements structure returned from a call to vkGetImageMemoryRequirements2KHR with image and where VkBindImagePlaneMemoryInfo::planeAspect corresponds to the VkImagePlaneMemoryRequirementsInfo::planeAspect in the VkImageMemoryRequirementsInfo2 structure's pNext chain
- VUID-VkBindImageMemoryInfo-pNext-01620
  If the pNext chain includes a VkBindImagePlaneMemoryInfo structure, memoryOffset **must** be an integer multiple of the alignment member of the VkMemoryRequirements structure returned from a call to vkGetImageMemoryRequirements2KHR with image and where VkBindImagePlaneMemoryInfo::planeAspect corresponds to the VkImagePlaneMemoryRequirementsInfo::planeAspect in the VkImageMemoryRequirementsInfo2 structure's pNext chain
- VUID-VkBindImageMemoryInfo-pNext-01621
  If the pNext chain includes a VkBindImagePlaneMemoryInfo structure, the difference of the size of memory and memoryOffset **must** be greater than or equal to the size member of the VkMemoryRequirements structure returned from a call to vkGetImageMemoryRequirements2KHR with the same image and where VkBindImagePlaneMemoryInfo::planeAspect corresponds to the VkImagePlaneMemoryRequirementsInfo::planeAspect in the VkImageMemoryRequirementsInfo2 structure's pNext chain
- VUID-VkBindImageMemoryInfo-pNext-01626
  If the pNext chain includes a VkBindImageMemoryDeviceGroupInfo structure, all instances of memory specified by VkBindImageMemoryDeviceGroupInfo::pDeviceIndices **must** have been allocated
- VUID-VkBindImageMemoryInfo-pNext-01627
  If the pNext chain includes a VkBindImageMemoryDeviceGroupInfo structure, and VkBindImageMemoryDeviceGroupInfo::splitInstanceBindRegionCount is not zero, then image **must** have been created with the VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT bit set
- VUID-VkBindImageMemoryInfo-pNext-01628
  If the pNext chain includes a VkBindImageMemoryDeviceGroupInfo structure, all elements of VkBindImageMemoryDeviceGroupInfo::pSplitInstanceBindRegions **must** be valid rectangles contained within the dimensions of image
- VUID-VkBindImageMemoryInfo-pNext-01629
  If the pNext chain includes a VkBindImageMemoryDeviceGroupInfo structure, the union of the areas of all elements of VkBindImageMemoryDeviceGroupInfo::pSplitInstanceBindRegions that correspond to the same instance of image **must** cover the entire image
- VUID-VkBindImageMemoryInfo-image-01630
  If image was created with a valid swapchain handle in VkImageSwapchainCreateInfoKHR::swapchain, then the pNext chain **must** include a VkBindImageMemorySwapchainInfoKHR structure containing the same swapchain

handle

- VUID-VkBindImageMemoryInfo-pNext-01631
  If the pNext chain includes a VkBindImageMemorySwapchainInfoKHR structure, memory **must** be VK_NULL_HANDLE
- VUID-VkBindImageMemoryInfo-pNext-01632
  If the pNext chain does not include a VkBindImageMemorySwapchainInfoKHR structure, memory **must** be a valid VkDeviceMemory handle

Valid Usage (Implicit)

- VUID-VkBindImageMemoryInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_INFO
- VUID-VkBindImageMemoryInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkBindImageMemoryDeviceGroupInfo, VkBindImageMemorySwapchainInfoKHR, VkBindImagePlaneMemoryInfo, or VkBindMemoryStatusKHR
- VUID-VkBindImageMemoryInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkBindImageMemoryInfo-image-parameter
  image **must** be a valid VkImage handle
- VUID-VkBindImageMemoryInfo-commonparent
  Both of image, and memory that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The VkBindImageMemoryDeviceGroupInfo structure is defined as:

```
typedef struct VkBindImageMemoryDeviceGroupInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           deviceIndexCount;
    const uint32_t*    pDeviceIndices;
    uint32_t           splitInstanceBindRegionCount;
    const VkRect2D*    pSplitInstanceBindRegions;
} VkBindImageMemoryDeviceGroupInfo;
```

or the equivalent

```
// Provided by VK_KHR_bind_memory2 with VK_KHR_device_group
typedef VkBindImageMemoryDeviceGroupInfo VkBindImageMemoryDeviceGroupInfoKHR;
```

- sType is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `deviceIndexCount` is the number of elements in `pDeviceIndices`.
- `pDeviceIndices` is a pointer to an array of device indices.
- `splitInstanceBindRegionCount` is the number of elements in `pSplitInstanceBindRegions`.
- `pSplitInstanceBindRegions` is a pointer to an array of VkRect2D structures describing which regions of the image are attached to each instance of memory.

If the `pNext` chain of VkBindImageMemoryInfo includes a `VkBindImageMemoryDeviceGroupInfo` structure, then that structure determines how memory is bound to images across multiple devices in a device group.

If `deviceIndexCount` is greater than zero, then on device index i `image` is attached to the instance of the memory on the physical device with device index `pDeviceIndices`[i].

Let N be the number of physical devices in the logical device. If `splitInstanceBindRegionCount` is greater than zero, then `pSplitInstanceBindRegions` is a pointer to an array of N^2 rectangles, where the image region specified by the rectangle at element i*N+j in resource instance i is bound to the memory instance j. The blocks of the memory that are bound to each sparse image block region use an offset in memory, relative to `memoryOffset`, computed as if the whole image was being bound to a contiguous range of memory. In other words, horizontally adjacent image blocks use consecutive blocks of memory, vertically adjacent image blocks are separated by the number of bytes per block multiplied by the width in blocks of `image`, and the block at (0,0) corresponds to memory starting at `memoryOffset`.

If `splitInstanceBindRegionCount` and `deviceIndexCount` are zero and the memory comes from a memory heap with the `VK_MEMORY_HEAP_MULTI_INSTANCE_BIT` bit set, then it is as if `pDeviceIndices` contains consecutive indices from zero to the number of physical devices in the logical device, minus one. In other words, by default each physical device attaches to its own instance of the memory.

If `splitInstanceBindRegionCount` and `deviceIndexCount` are zero and the memory comes from a memory heap without the `VK_MEMORY_HEAP_MULTI_INSTANCE_BIT` bit set, then it is as if `pDeviceIndices` contains an array of zeros. In other words, by default each physical device attaches to instance zero.

Valid Usage

- VUID-VkBindImageMemoryDeviceGroupInfo-deviceIndexCount-01633
  At least one of `deviceIndexCount` and `splitInstanceBindRegionCount` **must** be zero
- VUID-VkBindImageMemoryDeviceGroupInfo-deviceIndexCount-01634
  `deviceIndexCount` **must** either be zero or equal to the number of physical devices in the logical device
- VUID-VkBindImageMemoryDeviceGroupInfo-pDeviceIndices-01635
  All elements of `pDeviceIndices` **must** be valid device indices
- VUID-VkBindImageMemoryDeviceGroupInfo-splitInstanceBindRegionCount-01636
  `splitInstanceBindRegionCount` **must** either be zero or equal to the number of physical devices in the logical device squared

- VUID-VkBindImageMemoryDeviceGroupInfo-pSplitInstanceBindRegions-01637
  Elements of `pSplitInstanceBindRegions` that correspond to the same instance of an image **must** not overlap
- VUID-VkBindImageMemoryDeviceGroupInfo-offset-01638
  The `offset.x` member of any element of `pSplitInstanceBindRegions` **must** be a multiple of the sparse image block width (`VkSparseImageFormatProperties`::`imageGranularity.width`) of all non-metadata aspects of the image
- VUID-VkBindImageMemoryDeviceGroupInfo-offset-01639
  The `offset.y` member of any element of `pSplitInstanceBindRegions` **must** be a multiple of the sparse image block height (`VkSparseImageFormatProperties`::`imageGranularity.height`) of all non-metadata aspects of the image
- VUID-VkBindImageMemoryDeviceGroupInfo-extent-01640
  The `extent.width` member of any element of `pSplitInstanceBindRegions` **must** either be a multiple of the sparse image block width of all non-metadata aspects of the image, or else `extent.width` + `offset.x` **must** equal the width of the image subresource
- VUID-VkBindImageMemoryDeviceGroupInfo-extent-01641
  The `extent.height` member of any element of `pSplitInstanceBindRegions` **must** either be a multiple of the sparse image block height of all non-metadata aspects of the image, or else `extent.height` + `offset.y` **must** equal the height of the image subresource

Valid Usage (Implicit)

- VUID-VkBindImageMemoryDeviceGroupInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_DEVICE_GROUP_INFO`
- VUID-VkBindImageMemoryDeviceGroupInfo-pDeviceIndices-parameter
  If `deviceIndexCount` is not `0`, `pDeviceIndices` **must** be a valid pointer to an array of `deviceIndexCount` `uint32_t` values
- VUID-VkBindImageMemoryDeviceGroupInfo-pSplitInstanceBindRegions-parameter
  If `splitInstanceBindRegionCount` is not `0`, `pSplitInstanceBindRegions` **must** be a valid pointer to an array of `splitInstanceBindRegionCount` VkRect2D structures

If the `pNext` chain of VkBindImageMemoryInfo includes a `VkBindImageMemorySwapchainInfoKHR` structure, then that structure includes a swapchain handle and image index indicating that the image will be bound to memory from that swapchain.

The `VkBindImageMemorySwapchainInfoKHR` structure is defined as:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
typedef struct VkBindImageMemorySwapchainInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkSwapchainKHR     swapchain;
    uint32_t           imageIndex;
} VkBindImageMemorySwapchainInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `swapchain` is VK_NULL_HANDLE or a swapchain handle.
- `imageIndex` is an image index within `swapchain`.

If `swapchain` is not `NULL`, the `swapchain` and `imageIndex` are used to determine the memory that the image is bound to, instead of `memory` and `memoryOffset`.

Memory **can** be bound to a swapchain and use the `pDeviceIndices` or `pSplitInstanceBindRegions` members of VkBindImageMemoryDeviceGroupInfo.

Valid Usage

- VUID-VkBindImageMemorySwapchainInfoKHR-imageIndex-01644
  `imageIndex` **must** be less than the number of images in `swapchain`

Valid Usage (Implicit)

- VUID-VkBindImageMemorySwapchainInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_SWAPCHAIN_INFO_KHR`
- VUID-VkBindImageMemorySwapchainInfoKHR-swapchain-parameter
  `swapchain` **must** be a valid VkSwapchainKHR handle

Host Synchronization

- Host access to `swapchain` **must** be externally synchronized

In order to bind *planes* of a *disjoint image*, add a `VkBindImagePlaneMemoryInfo` structure to the `pNext` chain of VkBindImageMemoryInfo.

The `VkBindImagePlaneMemoryInfo` structure is defined as:

```
typedef struct VkBindImagePlaneMemoryInfo {
    VkStructureType          sType;
    const void*              pNext;
    VkImageAspectFlagBits    planeAspect;
} VkBindImagePlaneMemoryInfo;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkBindImagePlaneMemoryInfo VkBindImagePlaneMemoryInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `planeAspect` is a `VkImageAspectFlagBits` value specifying the aspect of the disjoint image plane to bind.

Valid Usage

- VUID-VkBindImagePlaneMemoryInfo-planeAspect-02283
  If the image's `tiling` is `VK_IMAGE_TILING_LINEAR` or `VK_IMAGE_TILING_OPTIMAL`, then `planeAspect` **must** be a single valid multi-planar aspect mask bit
- VUID-VkBindImagePlaneMemoryInfo-planeAspect-02284
  If the image's `tiling` is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`, then `planeAspect` **must** be a single valid *memory plane* for the image (that is, `aspectMask` **must** specify a plane index that is less than the VkDrmFormatModifierPropertiesEXT::`drmFormatModifierPlaneCount` associated with the image's `format` and VkImageDrmFormatModifierPropertiesEXT::`drmFormatModifier`)

Valid Usage (Implicit)

- VUID-VkBindImagePlaneMemoryInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BIND_IMAGE_PLANE_MEMORY_INFO`
- VUID-VkBindImagePlaneMemoryInfo-planeAspect-parameter
  `planeAspect` **must** be a valid VkImageAspectFlagBits value

Buffer-Image Granularity

The implementation-dependent limit `bufferImageGranularity` specifies a page-like granularity at which linear and non-linear resources **must** be placed in adjacent memory locations to avoid aliasing. Two resources which do not satisfy this granularity requirement are said to alias. `bufferImageGranularity` is specified in bytes, and **must** be a power of two. Implementations which do not impose a granularity restriction **may** report a `bufferImageGranularity` value of one.

NOTE Despite its name, `bufferImageGranularity` is really a granularity between "linear" and "non-linear" resources.

Given resourceA at the lower memory offset and resourceB at the higher memory offset in the same `VkDeviceMemory` object, where one resource is linear and the other is non-linear (as defined in the Glossary), and the following:

```
resourceA.end       = resourceA.memoryOffset + resourceA.size - 1
resourceA.endPage   = resourceA.end & ~(bufferImageGranularity-1)
resourceB.start     = resourceB.memoryOffset
resourceB.startPage = resourceB.start & ~(bufferImageGranularity-1)
```

The following property **must** hold:

```
resourceA.endPage < resourceB.startPage
```

That is, the end of the first resource (A) and the beginning of the second resource (B) **must** be on separate "pages" of size `bufferImageGranularity`. `bufferImageGranularity` **may** be different than the physical page size of the memory heap. This restriction is only needed when a linear resource and a non-linear resource are adjacent in memory and will be used simultaneously. The memory ranges of adjacent resources **can** be closer than `bufferImageGranularity`, provided they meet the `alignment` requirement for the objects in question.

Sparse block size in bytes and sparse image and buffer memory alignments **must** all be multiples of the `bufferImageGranularity`. Therefore, memory bound to sparse resources naturally satisfies the `bufferImageGranularity`.

12.9. Resource Sharing Mode

Buffer and image objects are created with a *sharing mode* controlling how they **can** be accessed from queues. The supported sharing modes are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSharingMode {
    VK_SHARING_MODE_EXCLUSIVE = 0,
    VK_SHARING_MODE_CONCURRENT = 1,
} VkSharingMode;
```

- `VK_SHARING_MODE_EXCLUSIVE` specifies that access to any range or image subresource of the object will be exclusive to a single queue family at a time.
- `VK_SHARING_MODE_CONCURRENT` specifies that concurrent access to any range or image subresource of the object from multiple queue families is supported.

NOTE `VK_SHARING_MODE_CONCURRENT` **may** result in lower performance access to the buffer or

image than VK_SHARING_MODE_EXCLUSIVE.

Ranges of buffers and image subresources of image objects created using VK_SHARING_MODE_EXCLUSIVE **must** only be accessed by queues in the queue family that has *ownership* of the resource. Upon creation, such resources are not owned by any queue family; ownership is implicitly acquired upon first use within a queue. Once a resource using VK_SHARING_MODE_EXCLUSIVE is owned by some queue family, the application **must** perform a queue family ownership transfer to make the memory contents of a range or image subresource accessible to a different queue family.

NOTE Images still require a layout transition from VK_IMAGE_LAYOUT_UNDEFINED or VK_IMAGE_LAYOUT_PREINITIALIZED before being used on the first queue.

A queue family **can** take ownership of an image subresource or buffer range of a resource created with VK_SHARING_MODE_EXCLUSIVE, without an ownership transfer, in the same way as for a resource that was just created; however, taking ownership in this way has the effect that the contents of the image subresource or buffer range are undefined.

Ranges of buffers and image subresources of image objects created using VK_SHARING_MODE_CONCURRENT **must** only be accessed by queues from the queue families specified through the queueFamilyIndexCount and pQueueFamilyIndices members of the corresponding create info structures.

12.9.1. External Resource Sharing

Resources **should** only be accessed in the Vulkan instance that has exclusive ownership of their underlying memory. Only one Vulkan instance has exclusive ownership of a resource's underlying memory at a given time, regardless of whether the resource was created using VK_SHARING_MODE_EXCLUSIVE or VK_SHARING_MODE_CONCURRENT. Applications can transfer ownership of a resource's underlying memory only if the memory has been imported from or exported to another instance or external API using external memory handles. The semantics for transferring ownership outside of the instance are similar to those used for transferring ownership of VK_SHARING_MODE_EXCLUSIVE resources between queues, and is also accomplished using VkBufferMemoryBarrier or VkImageMemoryBarrier operations. To make the contents of the underlying memory accessible in the destination instance or API, applications **must**

1. Release exclusive ownership from the source instance or API.
2. Ensure the release operation has completed using semaphores or fences.
3. Acquire exclusive ownership in the destination instance or API

Unlike queue family ownership transfers, the destination instance or API is not specified explicitly when releasing ownership, nor is the source instance or API specified when acquiring ownership. Instead, the image or memory barrier's dstQueueFamilyIndex or srcQueueFamilyIndex parameters are set to the reserved queue family index VK_QUEUE_FAMILY_EXTERNAL or VK_QUEUE_FAMILY_FOREIGN_EXT to represent the external destination or source respectively.

Binding a resource to a memory object shared between multiple Vulkan instances or other APIs does not change the ownership of the underlying memory. The first entity to access the resource implicitly acquires ownership. An entity **can** also implicitly take ownership from another entity in

the same way without an explicit ownership transfer. However, taking ownership in this way has the effect that the contents of the underlying memory are undefined.

Accessing a resource backed by memory that is owned by a particular instance or API has the same semantics as accessing a VK_SHARING_MODE_EXCLUSIVE resource, with one exception: Implementations **must** ensure layout transitions performed on one member of a set of identical subresources of identical images that alias the same range of an underlying memory object affect the layout of all the subresources in the set.

As a corollary, writes to any image subresources in such a set **must** not make the contents of memory used by other subresources in the set undefined. An application **can** define the content of a subresource of one image by performing device writes to an identical subresource of another image provided both images are bound to the same region of external memory. Applications **may** also add resources to such a set after the content of the existing set members has been defined without making the content undefined by creating a new image with the initial layout VK_IMAGE_LAYOUT_UNDEFINED and binding it to the same region of external memory as the existing images.

NOTE Because layout transitions apply to all identical images aliasing the same region of external memory, the actual layout of the memory backing a new image as well as an existing image with defined content will not be undefined. Such an image is not usable until it acquires ownership of its memory from the existing owner. Therefore, the layout specified as part of this transition will be the true initial layout of the image. The undefined layout specified when creating it is a placeholder to simplify valid usage requirements.

12.10. Memory Aliasing

A range of a VkDeviceMemory allocation is *aliased* if it is bound to multiple resources simultaneously, as described below, via vkBindImageMemory, vkBindBufferMemory, via sparse memory bindings, or by binding the memory to resources in multiple Vulkan instances or external APIs using external memory handle export and import mechanisms.

Consider two resources, resource_A and resource_B, bound respectively to memory range_A and range_B. Let paddedRange_A and paddedRange_B be, respectively, range_A and range_B aligned to bufferImageGranularity. If the resources are both linear or both non-linear (as defined in the Glossary), then the resources *alias* the memory in the intersection of range_A and range_B. If one resource is linear and the other is non-linear, then the resources *alias* the memory in the intersection of paddedRange_A and paddedRange_B.

Applications **can** alias memory, but use of multiple aliases is subject to several constraints.

NOTE Memory aliasing **can** be useful to reduce the total device memory footprint of an application, if some large resources are used for disjoint periods of time.

When a non-linear, non-VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT image is bound to an aliased range, all image subresources of the image *overlap* the range. When a linear image is bound to an aliased range, the image subresources that (according to the image's advertised layout) include bytes from

the aliased range overlap the range. When a VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT image has sparse image blocks bound to an aliased range, only image subresources including those sparse image blocks overlap the range, and when the memory bound to the image's mip tail overlaps an aliased range all image subresources in the mip tail overlap the range.

Buffers, and linear image subresources in either the VK_IMAGE_LAYOUT_PREINITIALIZED or VK_IMAGE_LAYOUT_GENERAL layouts, are *host-accessible subresources*. That is, the host has a well-defined addressing scheme to interpret the contents, and thus the layout of the data in memory **can** be consistently interpreted across aliases if each of those aliases is a host-accessible subresource. Non-linear images, and linear image subresources in other layouts, are not host-accessible.

If two aliases are both host-accessible, then they interpret the contents of the memory in consistent ways, and data written to one alias **can** be read by the other alias.

If two aliases are both images that were created with identical creation parameters, both were created with the VK_IMAGE_CREATE_ALIAS_BIT flag set, and both are bound identically to memory except for VkBindImageMemoryDeviceGroupInfo::pDeviceIndices and VkBindImageMemoryDeviceGroupInfo::pSplitInstanceBindRegions, then they interpret the contents of the memory in consistent ways, and data written to one alias **can** be read by the other alias.

Additionally, if an individual plane of a multi-planar image and a single-plane image alias the same memory, then they also interpret the contents of the memory in consistent ways under the same conditions, but with the following modifications:

- Both **must** have been created with the VK_IMAGE_CREATE_DISJOINT_BIT flag.
- The single-plane image **must** have a VkFormat that is equivalent to that of the multi-planar image's individual plane.
- The single-plane image and the individual plane of the multi-planar image **must** be bound identically to memory except for VkBindImageMemoryDeviceGroupInfo::pDeviceIndices and VkBindImageMemoryDeviceGroupInfo::pSplitInstanceBindRegions.
- The width and height of the single-plane image are derived from the multi-planar image's dimensions in the manner listed for plane compatibility for the aliased plane.
- If either image's tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then both images **must** be linear.
- All other creation parameters **must** be identical

Aliases created by binding the same memory to resources in multiple Vulkan instances or external APIs using external memory handle export and import mechanisms interpret the contents of the memory in consistent ways, and data written to one alias **can** be read by the other alias.

Otherwise, the aliases interpret the contents of the memory differently, and writes via one alias make the contents of memory partially or completely undefined to the other alias. If the first alias is a host-accessible subresource, then the bytes affected are those written by the memory operations according to its addressing scheme. If the first alias is not host-accessible, then the bytes affected are those overlapped by the image subresources that were written. If the second alias is a host-accessible subresource, the affected bytes become undefined. If the second alias is not host-accessible, all sparse image blocks (for sparse partially-resident images) or all image subresources

(for non-sparse image and fully resident sparse images) that overlap the affected bytes become undefined.

If any image subresources are made undefined due to writes to an alias, then each of those image subresources **must** have its layout transitioned from `VK_IMAGE_LAYOUT_UNDEFINED` to a valid layout before it is used, or from `VK_IMAGE_LAYOUT_PREINITIALIZED` if the memory has been written by the host. If any sparse blocks of a sparse image have been made undefined, then only the image subresources containing them **must** be transitioned.

Use of an overlapping range by two aliases **must** be separated by a memory dependency using the appropriate access types if at least one of those uses performs writes, whether the aliases interpret memory consistently or not. If buffer or image memory barriers are used, the scope of the barrier **must** contain the entire range and/or set of image subresources that overlap.

If two aliasing image views are used in the same framebuffer, then the render pass **must** declare the attachments using the `VK_ATTACHMENT_DESCRIPTION_MAY_ALIAS_BIT`, and follow the other rules listed in that section.

NOTE	Memory recycled via an application suballocator (i.e. without freeing and reallocating the memory objects) is not substantially different from memory aliasing. However, a suballocator usually waits on a fence before recycling a region of memory, and signaling a fence involves sufficient implicit dependencies to satisfy all the above requirements.

12.10.1. Resource Memory Overlap

Applications **can** safely access resources concurrently via separate device and host operations as long as the accessed memory locations are guaranteed to not overlap, as defined in Memory Location, and the operation, resource, and access are otherwise independently valid.

Some operations have alignment requirements or access ambiguous memory locations, so the semantics of a particular operation **should** be considered when determining the overlap. Such requirements will be described alongside the operation. Operations between host and device when using non-coherent memory are aligned to `nonCoherentAtomSize`, as defined by vkFlushMappedMemoryRanges and vkInvalidateMappedMemoryRanges.

NOTE	The intent is that buffers (or linear images) can be accessed concurrently, even when they share cache lines, but otherwise do not access the same memory range. The concept of a device cache line size is not exposed in the memory model.

Chapter 13. Samplers

VkSampler objects represent the state of an image sampler which is used by the implementation to read image data and apply filtering and other transformations for the shader.

Samplers are represented by VkSampler handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkSampler)
```

To create a sampler object, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateSampler(
    VkDevice                                    device,
    const VkSamplerCreateInfo*                  pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSampler*                                  pSampler);
```

- device is the logical device that creates the sampler.
- pCreateInfo is a pointer to a VkSamplerCreateInfo structure specifying the state of the sampler object.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.
- pSampler is a pointer to a VkSampler handle in which the resulting sampler object is returned.

Valid Usage

- VUID-vkCreateSampler-device-09668
  device **must** support at least one queue family with one of the VK_QUEUE_COMPUTE_BIT or VK_QUEUE_GRAPHICS_BIT capabilities
- VUID-vkCreateSampler-maxSamplerAllocationCount-04110
  There **must** be less than VkPhysicalDeviceLimits::maxSamplerAllocationCount VkSampler objects currently created on the device

Valid Usage (Implicit)

- VUID-vkCreateSampler-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateSampler-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkSamplerCreateInfo structure
- VUID-vkCreateSampler-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid

VkAllocationCallbacks structure

- VUID-vkCreateSampler-pSampler-parameter
  `pSampler` **must** be a valid pointer to a VkSampler handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR

The VkSamplerCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSamplerCreateInfo {
    VkStructureType         sType;
    const void*             pNext;
    VkSamplerCreateFlags    flags;
    VkFilter                magFilter;
    VkFilter                minFilter;
    VkSamplerMipmapMode     mipmapMode;
    VkSamplerAddressMode    addressModeU;
    VkSamplerAddressMode    addressModeV;
    VkSamplerAddressMode    addressModeW;
    float                   mipLodBias;
    VkBool32                anisotropyEnable;
    float                   maxAnisotropy;
    VkBool32                compareEnable;
    VkCompareOp             compareOp;
    float                   minLod;
    float                   maxLod;
    VkBorderColor           borderColor;
    VkBool32                unnormalizedCoordinates;
} VkSamplerCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkSamplerCreateFlagBits describing additional parameters of the sampler.
- `magFilter` is a VkFilter value specifying the magnification filter to apply to lookups.
- `minFilter` is a VkFilter value specifying the minification filter to apply to lookups.
- `mipmapMode` is a VkSamplerMipmapMode value specifying the mipmap filter to apply to lookups.

- `addressModeU` is a VkSamplerAddressMode value specifying the addressing mode for U coordinates outside [0,1).
- `addressModeV` is a VkSamplerAddressMode value specifying the addressing mode for V coordinates outside [0,1).
- `addressModeW` is a VkSamplerAddressMode value specifying the addressing mode for W coordinates outside [0,1).
- `mipLodBias` is the bias to be added to mipmap LOD calculation and bias provided by image sampling functions in SPIR-V, as described in the LOD Operation section.
- `anisotropyEnable` is `VK_TRUE` to enable anisotropic filtering, as described in the Texel Anisotropic Filtering section, or `VK_FALSE` otherwise.
- `maxAnisotropy` is the anisotropy value clamp used by the sampler when `anisotropyEnable` is `VK_TRUE`. If `anisotropyEnable` is `VK_FALSE`, `maxAnisotropy` is ignored.
- `compareEnable` is `VK_TRUE` to enable comparison against a reference value during lookups, or `VK_FALSE` otherwise.
  - Note: Some implementations will default to shader state if this member does not match.
- `compareOp` is a VkCompareOp value specifying the comparison operator to apply to fetched data before filtering as described in the Depth Compare Operation section.
- `minLod` is used to clamp the minimum of the computed LOD value.
- `maxLod` is used to clamp the maximum of the computed LOD value. To avoid clamping the maximum value, set `maxLod` to the constant `VK_LOD_CLAMP_NONE`.
- `borderColor` is a VkBorderColor value specifying the predefined border color to use.
- `unnormalizedCoordinates` controls whether to use unnormalized or normalized texel coordinates to address texels of the image. When `unnormalizedCoordinates` is `VK_TRUE`, the range of the image coordinates used to lookup the texel is in the range of zero to the image size in each dimension. When `unnormalizedCoordinates` is `VK_FALSE`, the range of image coordinates is zero to one.

  When `unnormalizedCoordinates` is `VK_TRUE`, images the sampler is used with in the shader have the following requirements:

  - The `viewType` **must** be either `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`.
  - The image view **must** have a single layer and a single mip level.

  When `unnormalizedCoordinates` is `VK_TRUE`, image built-in functions in the shader that use the sampler have the following requirements:

  - The functions **must** not use projection.
  - The functions **must** not use offsets.

NOTE

Mapping of OpenGL to Vulkan Filter Modes

`magFilter` values of `VK_FILTER_NEAREST` and `VK_FILTER_LINEAR` directly correspond to `GL_NEAREST` and `GL_LINEAR` magnification filters. `minFilter` and `mipmapMode` combine to correspond to the similarly named OpenGL minification filter of `GL_minFilter_MIPMAP_mipmapMode` (e.g. `minFilter` of `VK_FILTER_LINEAR` and `mipmapMode` of

> VK_SAMPLER_MIPMAP_MODE_NEAREST correspond to GL_LINEAR_MIPMAP_NEAREST).
>
> There are no Vulkan filter modes that directly correspond to OpenGL minification filters of GL_LINEAR or GL_NEAREST, but they **can** be emulated using VK_SAMPLER_MIPMAP_MODE_NEAREST, minLod = 0, and maxLod = 0.25, and using minFilter = VK_FILTER_LINEAR or minFilter = VK_FILTER_NEAREST, respectively.
>
> Note that using a maxLod of zero would cause magnification to always be performed, and the magFilter to always be used. This is valid, just not an exact match for OpenGL behavior. Clamping the maximum LOD to 0.25 allows the λ value to be non-zero and minification to be performed, while still always rounding down to the base level. If the minFilter and magFilter are equal, then using a maxLod of zero also works.

The maximum number of sampler objects which **can** be simultaneously created on a device is implementation-dependent and specified by the maxSamplerAllocationCount member of the VkPhysicalDeviceLimits structure.

NOTE

For historical reasons, if maxSamplerAllocationCount is exceeded, some implementations may return VK_ERROR_TOO_MANY_OBJECTS. Exceeding this limit will result in undefined behavior, and an application should not rely on the use of the returned error code in order to identify when the limit is reached.

Since VkSampler is a non-dispatchable handle type, implementations **may** return the same handle for sampler state vectors that are identical. In such cases, all such objects would only count once against the maxSamplerAllocationCount limit.

Valid Usage

- VUID-VkSamplerCreateInfo-mipLodBias-01069
  The absolute value of mipLodBias **must** be less than or equal to VkPhysicalDeviceLimits::maxSamplerLodBias
- VUID-VkSamplerCreateInfo-samplerMipLodBias-04467
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::samplerMipLodBias is VK_FALSE, mipLodBias **must** be zero
- VUID-VkSamplerCreateInfo-maxLod-01973
  maxLod **must** be greater than or equal to minLod
- VUID-VkSamplerCreateInfo-anisotropyEnable-01070
  If the samplerAnisotropy feature is not enabled, anisotropyEnable **must** be VK_FALSE
- VUID-VkSamplerCreateInfo-anisotropyEnable-01071
  If anisotropyEnable is VK_TRUE, maxAnisotropy **must** be between 1.0 and VkPhysicalDeviceLimits::maxSamplerAnisotropy, inclusive
- VUID-VkSamplerCreateInfo-minFilter-01645
  If sampler $Y'C_BC_R$ conversion is enabled and the potential format features of the sampler $Y'C_BC_R$ conversion do not support

VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT, minFilter and magFilter **must** be equal to the sampler $Y'C_BC_R$ conversion's chromaFilter

- VUID-VkSamplerCreateInfo-unnormalizedCoordinates-01072
  If unnormalizedCoordinates is VK_TRUE, minFilter and magFilter **must** be equal
- VUID-VkSamplerCreateInfo-unnormalizedCoordinates-01073
  If unnormalizedCoordinates is VK_TRUE, mipmapMode **must** be VK_SAMPLER_MIPMAP_MODE_NEAREST
- VUID-VkSamplerCreateInfo-unnormalizedCoordinates-01074
  If unnormalizedCoordinates is VK_TRUE, minLod and maxLod **must** be zero
- VUID-VkSamplerCreateInfo-unnormalizedCoordinates-01075
  If unnormalizedCoordinates is VK_TRUE, addressModeU and addressModeV **must** each be either VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE or VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER
- VUID-VkSamplerCreateInfo-unnormalizedCoordinates-01076
  If unnormalizedCoordinates is VK_TRUE, anisotropyEnable **must** be VK_FALSE
- VUID-VkSamplerCreateInfo-unnormalizedCoordinates-01077
  If unnormalizedCoordinates is VK_TRUE, compareEnable **must** be VK_FALSE
- VUID-VkSamplerCreateInfo-addressModeU-01078
  If any of addressModeU, addressModeV or addressModeW are VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER, borderColor **must** be a valid VkBorderColor value
- VUID-VkSamplerCreateInfo-addressModeU-01646
  If sampler $Y'C_BC_R$ conversion is enabled, addressModeU, addressModeV, and addressModeW **must** be VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE, anisotropyEnable **must** be VK_FALSE, and unnormalizedCoordinates **must** be VK_FALSE
- VUID-VkSamplerCreateInfo-addressModeU-01079
  If the samplerMirrorClampToEdge feature is not enabled, and if the VK_KHR_sampler_mirror_clamp_to_edge extension is not enabled, addressModeU, addressModeV and addressModeW **must** not be VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE
- VUID-VkSamplerCreateInfo-compareEnable-01080
  If compareEnable is VK_TRUE, compareOp **must** be a valid VkCompareOp value
- VUID-VkSamplerCreateInfo-magFilter-01081
  If either magFilter or minFilter is VK_FILTER_CUBIC_EXT, anisotropyEnable **must** be VK_FALSE
- VUID-VkSamplerCreateInfo-flags-02574
  If flags includes VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT, then minFilter and magFilter **must** be equal
- VUID-VkSamplerCreateInfo-flags-02575
  If flags includes VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT, then mipmapMode **must** be VK_SAMPLER_MIPMAP_MODE_NEAREST
- VUID-VkSamplerCreateInfo-flags-02576
  If flags includes VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT, then minLod and maxLod **must** be zero
- VUID-VkSamplerCreateInfo-flags-02577
  If flags includes VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT, then addressModeU and

addressModeV **must** each be either VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE or VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER

- VUID-VkSamplerCreateInfo-flags-02578
  If flags includes VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT, then anisotropyEnable **must** be VK_FALSE
- VUID-VkSamplerCreateInfo-flags-02579
  If flags includes VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT, then compareEnable **must** be VK_FALSE
- VUID-VkSamplerCreateInfo-flags-02580
  If flags includes VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT, then unnormalizedCoordinates **must** be VK_FALSE
- VUID-VkSamplerCreateInfo-nonSeamlessCubeMap-06788
  If the nonSeamlessCubeMap feature is not enabled, flags **must** not include VK_SAMPLER_CREATE_NON_SEAMLESS_CUBE_MAP_BIT_EXT
- VUID-VkSamplerCreateInfo-borderColor-04011
  If borderColor is one of VK_BORDER_COLOR_FLOAT_CUSTOM_EXT or VK_BORDER_COLOR_INT_CUSTOM_EXT, then a VkSamplerCustomBorderColorCreateInfoEXT **must** be included in the pNext chain
- VUID-VkSamplerCreateInfo-customBorderColors-04085
  If the customBorderColors feature is not enabled, borderColor **must** not be VK_BORDER_COLOR_FLOAT_CUSTOM_EXT or VK_BORDER_COLOR_INT_CUSTOM_EXT
- VUID-VkSamplerCreateInfo-borderColor-04442
  If borderColor is one of VK_BORDER_COLOR_FLOAT_CUSTOM_EXT or VK_BORDER_COLOR_INT_CUSTOM_EXT, and VkSamplerCustomBorderColorCreateInfoEXT::format is not VK_FORMAT_UNDEFINED, VkSamplerCustomBorderColorCreateInfoEXT ::customBorderColor **must** be within the range of values representable in format
- VUID-VkSamplerCreateInfo-None-04012
  The maximum number of samplers with custom border colors which **can** be simultaneously created on a device is implementation-dependent and specified by the maxCustomBorderColorSamplers member of the VkPhysicalDeviceCustomBorderColorPropertiesEXT structure

Valid Usage (Implicit)

- VUID-VkSamplerCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO
- VUID-VkSamplerCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkSamplerBorderColorComponentMappingCreateInfoEXT, VkSamplerCustomBorderColorCreateInfoEXT, or VkSamplerYcbcrConversionInfo
- VUID-VkSamplerCreateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique

- VUID-VkSamplerCreateInfo-flags-parameter
  `flags` **must** be a valid combination of VkSamplerCreateFlagBits values
- VUID-VkSamplerCreateInfo-magFilter-parameter
  `magFilter` **must** be a valid VkFilter value
- VUID-VkSamplerCreateInfo-minFilter-parameter
  `minFilter` **must** be a valid VkFilter value
- VUID-VkSamplerCreateInfo-mipmapMode-parameter
  `mipmapMode` **must** be a valid VkSamplerMipmapMode value
- VUID-VkSamplerCreateInfo-addressModeU-parameter
  `addressModeU` **must** be a valid VkSamplerAddressMode value
- VUID-VkSamplerCreateInfo-addressModeV-parameter
  `addressModeV` **must** be a valid VkSamplerAddressMode value
- VUID-VkSamplerCreateInfo-addressModeW-parameter
  `addressModeW` **must** be a valid VkSamplerAddressMode value

`VK_LOD_CLAMP_NONE` is a special constant value used for VkSamplerCreateInfo::`maxLod` to indicate that maximum LOD clamping should not be performed.

```
#define VK_LOD_CLAMP_NONE                 1000.0F
```

Bits which **can** be set in VkSamplerCreateInfo::`flags`, specifying additional parameters of a sampler, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSamplerCreateFlagBits {
  // Provided by VK_EXT_fragment_density_map
    VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT = 0x00000001,
  // Provided by VK_EXT_fragment_density_map
    VK_SAMPLER_CREATE_SUBSAMPLED_COARSE_RECONSTRUCTION_BIT_EXT = 0x00000002,
  // Provided by VK_EXT_non_seamless_cube_map
    VK_SAMPLER_CREATE_NON_SEAMLESS_CUBE_MAP_BIT_EXT = 0x00000004,
} VkSamplerCreateFlagBits;
```

- `VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT` specifies that the sampler will read from an image created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`.
- `VK_SAMPLER_CREATE_SUBSAMPLED_COARSE_RECONSTRUCTION_BIT_EXT` specifies that the implementation **may** use approximations when reconstructing a full color value for texture access from a subsampled image.
- `VK_SAMPLER_CREATE_NON_SEAMLESS_CUBE_MAP_BIT_EXT` specifies that cube map edge handling is not performed.

NOTE	The approximations used when `VK_SAMPLER_CREATE_SUBSAMPLED_COARSE_RECONSTRUCTION_BIT_EXT` is specified are

> implementation defined. Some implementations **may** interpolate between fragment density levels in a subsampled image. In that case, this bit **may** be used to decide whether the interpolation factors are calculated per fragment or at a coarser granularity.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkSamplerCreateFlags;
```

VkSamplerCreateFlags is a bitmask type for setting a mask of zero or more VkSamplerCreateFlagBits.

Possible values of the VkSamplerCreateInfo::magFilter and minFilter parameters, specifying filters used for texture lookups, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkFilter {
    VK_FILTER_NEAREST = 0,
    VK_FILTER_LINEAR = 1,
  // Provided by VK_EXT_filter_cubic
    VK_FILTER_CUBIC_EXT = 1000015000,
} VkFilter;
```

- VK_FILTER_NEAREST specifies nearest filtering.
- VK_FILTER_LINEAR specifies linear filtering.
- VK_FILTER_CUBIC_EXT specifies cubic filtering.

These filters are described in detail in Texel Filtering.

Possible values of the VkSamplerCreateInfo::mipmapMode, specifying the mipmap mode used for texture lookups, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSamplerMipmapMode {
    VK_SAMPLER_MIPMAP_MODE_NEAREST = 0,
    VK_SAMPLER_MIPMAP_MODE_LINEAR = 1,
} VkSamplerMipmapMode;
```

- VK_SAMPLER_MIPMAP_MODE_NEAREST specifies nearest filtering.
- VK_SAMPLER_MIPMAP_MODE_LINEAR specifies linear filtering.

These modes are described in detail in Texel Filtering.

Possible values of the VkSamplerCreateInfo::addressMode* parameters, specifying the behavior of sampling with coordinates outside the range [0,1] for the respective u, v, or w coordinate as defined in the Wrapping Operation section, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSamplerAddressMode {
    VK_SAMPLER_ADDRESS_MODE_REPEAT = 0,
    VK_SAMPLER_ADDRESS_MODE_MIRRORED_REPEAT = 1,
    VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE = 2,
    VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER = 3,
  // Provided by VK_KHR_sampler_mirror_clamp_to_edge
    VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE = 4,
  // Provided by VK_KHR_sampler_mirror_clamp_to_edge
  // VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE_KHR is a deprecated alias
    VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE_KHR =
VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE,
} VkSamplerAddressMode;
```

- VK_SAMPLER_ADDRESS_MODE_REPEAT specifies that the repeat wrap mode will be used.
- VK_SAMPLER_ADDRESS_MODE_MIRRORED_REPEAT specifies that the mirrored repeat wrap mode will be used.
- VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE specifies that the clamp to edge wrap mode will be used.
- VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER specifies that the clamp to border wrap mode will be used.
- VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE specifies that the mirror clamp to edge wrap mode will be used. This is only valid if the VK_KHR_sampler_mirror_clamp_to_edge extension is enabled.

Comparison operators compare a *reference* and a *test* value, and return a true (“passed”) or false (“failed”) value depending on the comparison operator chosen. The supported operators are:

```
// Provided by VK_VERSION_1_0
typedef enum VkCompareOp {
    VK_COMPARE_OP_NEVER = 0,
    VK_COMPARE_OP_LESS = 1,
    VK_COMPARE_OP_EQUAL = 2,
    VK_COMPARE_OP_LESS_OR_EQUAL = 3,
    VK_COMPARE_OP_GREATER = 4,
    VK_COMPARE_OP_NOT_EQUAL = 5,
    VK_COMPARE_OP_GREATER_OR_EQUAL = 6,
    VK_COMPARE_OP_ALWAYS = 7,
} VkCompareOp;
```

- VK_COMPARE_OP_NEVER specifies that the comparison always evaluates false.
- VK_COMPARE_OP_LESS specifies that the comparison evaluates *reference* < *test*.
- VK_COMPARE_OP_EQUAL specifies that the comparison evaluates *reference* = *test*.
- VK_COMPARE_OP_LESS_OR_EQUAL specifies that the comparison evaluates *reference* ≤ *test*.
- VK_COMPARE_OP_GREATER specifies that the comparison evaluates *reference* > *test*.

- VK_COMPARE_OP_NOT_EQUAL specifies that the comparison evaluates *reference* ≠ *test*.
- VK_COMPARE_OP_GREATER_OR_EQUAL specifies that the comparison evaluates *reference* ≥ *test*.
- VK_COMPARE_OP_ALWAYS specifies that the comparison always evaluates true.

Comparison operators are used for:

- The Depth Compare Operation operator for a sampler, specified by VkSamplerCreateInfo::compareOp.
- The stencil comparison operator for the stencil test, specified by VkStencilOpState::compareOp.
- The Depth Comparison operator for the depth test, specified by VkPipelineDepthStencilStateCreateInfo::depthCompareOp.

Each such use describes how the *reference* and *test* values for that comparison are determined.

Possible values of VkSamplerCreateInfo::borderColor, specifying the border color used for texture lookups, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkBorderColor {
    VK_BORDER_COLOR_FLOAT_TRANSPARENT_BLACK = 0,
    VK_BORDER_COLOR_INT_TRANSPARENT_BLACK = 1,
    VK_BORDER_COLOR_FLOAT_OPAQUE_BLACK = 2,
    VK_BORDER_COLOR_INT_OPAQUE_BLACK = 3,
    VK_BORDER_COLOR_FLOAT_OPAQUE_WHITE = 4,
    VK_BORDER_COLOR_INT_OPAQUE_WHITE = 5,
  // Provided by VK_EXT_custom_border_color
    VK_BORDER_COLOR_FLOAT_CUSTOM_EXT = 1000287003,
  // Provided by VK_EXT_custom_border_color
    VK_BORDER_COLOR_INT_CUSTOM_EXT = 1000287004,
} VkBorderColor;
```

- VK_BORDER_COLOR_FLOAT_TRANSPARENT_BLACK specifies a transparent, floating-point format, black color.
- VK_BORDER_COLOR_INT_TRANSPARENT_BLACK specifies a transparent, integer format, black color.
- VK_BORDER_COLOR_FLOAT_OPAQUE_BLACK specifies an opaque, floating-point format, black color.
- VK_BORDER_COLOR_INT_OPAQUE_BLACK specifies an opaque, integer format, black color.
- VK_BORDER_COLOR_FLOAT_OPAQUE_WHITE specifies an opaque, floating-point format, white color.
- VK_BORDER_COLOR_INT_OPAQUE_WHITE specifies an opaque, integer format, white color.
- VK_BORDER_COLOR_FLOAT_CUSTOM_EXT specifies that a VkSamplerCustomBorderColorCreateInfoEXT structure is included in the VkSamplerCreateInfo::pNext chain containing the color data in floating-point format.
- VK_BORDER_COLOR_INT_CUSTOM_EXT specifies that a VkSamplerCustomBorderColorCreateInfoEXT structure is included in the VkSamplerCreateInfo::pNext chain containing the color data in integer format.

These colors are described in detail in Texel Replacement.

To destroy a sampler, call:

```
// Provided by VK_VERSION_1_0
void vkDestroySampler(
    VkDevice                                    device,
    VkSampler                                   sampler,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the sampler.
- `sampler` is the sampler to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroySampler-sampler-01082
  All submitted commands that refer to `sampler` **must** have completed execution
- VUID-vkDestroySampler-sampler-01083
  If `VkAllocationCallbacks` were provided when `sampler` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroySampler-sampler-01084
  If no `VkAllocationCallbacks` were provided when `sampler` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroySampler-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroySampler-sampler-parameter
  If `sampler` is not VK_NULL_HANDLE, `sampler` **must** be a valid VkSampler handle
- VUID-vkDestroySampler-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroySampler-sampler-parent
  If `sampler` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `sampler` **must** be externally synchronized

13.1. Sampler $Y'C_BC_R$ Conversion

To create a sampler with $Y'C_BC_R$ conversion enabled, add a VkSamplerYcbcrConversionInfo structure to the pNext chain of the VkSamplerCreateInfo structure. To create a sampler $Y'C_BC_R$ conversion, the samplerYcbcrConversion feature **must** be enabled. Conversion **must** be fixed at pipeline creation time, through use of a combined image sampler with an immutable sampler in VkDescriptorSetLayoutBinding.

A VkSamplerYcbcrConversionInfo **must** be provided for samplers to be used with image views that access VK_IMAGE_ASPECT_COLOR_BIT if the format is one of the formats that require a sampler $Y'C_BC_R$ conversion .

The VkSamplerYcbcrConversionInfo structure is defined as:

```
typedef struct VkSamplerYcbcrConversionInfo {
    VkStructureType             sType;
    const void*                 pNext;
    VkSamplerYcbcrConversion    conversion;
} VkSamplerYcbcrConversionInfo;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkSamplerYcbcrConversionInfo VkSamplerYcbcrConversionInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- conversion is a VkSamplerYcbcrConversion handle created with vkCreateSamplerYcbcrConversionKHR.

Valid Usage (Implicit)

- VUID-VkSamplerYcbcrConversionInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_INFO
- VUID-VkSamplerYcbcrConversionInfo-conversion-parameter
  conversion **must** be a valid VkSamplerYcbcrConversion handle

A sampler $Y'C_BC_R$ conversion is an opaque representation of a device-specific sampler $Y'C_BC_R$ conversion description, represented as a VkSamplerYcbcrConversion handle:

```
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkSamplerYcbcrConversion)
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkSamplerYcbcrConversion VkSamplerYcbcrConversionKHR;
```

To create a VkSamplerYcbcrConversion, call:

```
// Provided by VK_KHR_sampler_ycbcr_conversion
VkResult vkCreateSamplerYcbcrConversionKHR(
    VkDevice                                    device,
    const VkSamplerYcbcrConversionCreateInfo*   pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSamplerYcbcrConversion*                   pYcbcrConversion);
```

- `device` is the logical device that creates the sampler $Y'C_BC_R$ conversion.
- `pCreateInfo` is a pointer to a VkSamplerYcbcrConversionCreateInfo structure specifying the requested sampler $Y'C_BC_R$ conversion.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pYcbcrConversion` is a pointer to a VkSamplerYcbcrConversion handle in which the resulting sampler $Y'C_BC_R$ conversion is returned.

The interpretation of the configured sampler $Y'C_BC_R$ conversion is described in more detail in the description of sampler $Y'C_BC_R$ conversion in the Image Operations chapter.

Valid Usage

- VUID-vkCreateSamplerYcbcrConversion-None-01648
  The `samplerYcbcrConversion` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCreateSamplerYcbcrConversion-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateSamplerYcbcrConversion-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkSamplerYcbcrConversionCreateInfo structure
- VUID-vkCreateSamplerYcbcrConversion-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateSamplerYcbcrConversion-pYcbcrConversion-parameter
  `pYcbcrConversion` **must** be a valid pointer to a VkSamplerYcbcrConversion handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkSamplerYcbcrConversionCreateInfo structure is defined as:

```
typedef struct VkSamplerYcbcrConversionCreateInfo {
    VkStructureType                 sType;
    const void*                     pNext;
    VkFormat                        format;
    VkSamplerYcbcrModelConversion   ycbcrModel;
    VkSamplerYcbcrRange             ycbcrRange;
    VkComponentMapping              components;
    VkChromaLocation                xChromaOffset;
    VkChromaLocation                yChromaOffset;
    VkFilter                        chromaFilter;
    VkBool32                        forceExplicitReconstruction;
} VkSamplerYcbcrConversionCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkSamplerYcbcrConversionCreateInfo VkSamplerYcbcrConversionCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- format is the format of the image from which color information will be retrieved.
- ycbcrModel describes the color matrix for conversion between color models.
- ycbcrRange describes whether the encoded values have headroom and foot room, or whether the encoding uses the full numerical range.
- components applies a *swizzle* based on VkComponentSwizzle enums prior to range expansion and color model conversion.
- xChromaOffset describes the sample location associated with downsampled chroma components in the x dimension. xChromaOffset has no effect for formats in which chroma components are not downsampled horizontally.
- yChromaOffset describes the sample location associated with downsampled chroma components in the y dimension. yChromaOffset has no effect for formats in which the chroma components are not downsampled vertically.

- `chromaFilter` is the filter for chroma reconstruction.
- `forceExplicitReconstruction` **can** be used to ensure that reconstruction is done explicitly, if supported.

NOTE

Setting `forceExplicitReconstruction` to `VK_TRUE` **may** have a performance penalty on implementations where explicit reconstruction is not the default mode of operation.

If `format` supports `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT` the `forceExplicitReconstruction` value behaves as if it were `VK_TRUE`.

Sampler $Y'C_BC_R$ conversion objects do not support *external format conversion* without additional extensions defining *external formats*.

Valid Usage

- VUID-VkSamplerYcbcrConversionCreateInfo-format-04061
  `format` **must** represent unsigned normalized values (i.e. the format **must** be a `UNORM` format)
- VUID-VkSamplerYcbcrConversionCreateInfo-format-01650
  The potential format features of the sampler $Y'C_BC_R$ conversion **must** support `VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT` or `VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT`
- VUID-VkSamplerYcbcrConversionCreateInfo-xChromaOffset-01651
  If the potential format features of the sampler $Y'C_BC_R$ conversion do not support `VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT`, `xChromaOffset` and `yChromaOffset` **must** not be `VK_CHROMA_LOCATION_COSITED_EVEN` if the corresponding components are downsampled
- VUID-VkSamplerYcbcrConversionCreateInfo-xChromaOffset-01652
  If the potential format features of the sampler $Y'C_BC_R$ conversion do not support `VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT`, `xChromaOffset` and `yChromaOffset` **must** not be `VK_CHROMA_LOCATION_MIDPOINT` if the corresponding components are downsampled
- VUID-VkSamplerYcbcrConversionCreateInfo-components-02581
  If the format has a `_422` or `_420` suffix, then `components.g` **must** be the identity swizzle
- VUID-VkSamplerYcbcrConversionCreateInfo-components-02582
  If the format has a `_422` or `_420` suffix, then `components.a` **must** be the identity swizzle, `VK_COMPONENT_SWIZZLE_ONE`, or `VK_COMPONENT_SWIZZLE_ZERO`
- VUID-VkSamplerYcbcrConversionCreateInfo-components-02583
  If the format has a `_422` or `_420` suffix, then `components.r` **must** be the identity swizzle or `VK_COMPONENT_SWIZZLE_B`
- VUID-VkSamplerYcbcrConversionCreateInfo-components-02584
  If the format has a `_422` or `_420` suffix, then `components.b` **must** be the identity swizzle or `VK_COMPONENT_SWIZZLE_R`
- VUID-VkSamplerYcbcrConversionCreateInfo-components-02585
  If the format has a `_422` or `_420` suffix, and if either `components.r` or `components.b` is the

identity swizzle, both values **must** be the identity swizzle

- VUID-VkSamplerYcbcrConversionCreateInfo-ycbcrModel-01655
  If `ycbcrModel` is not `VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY`, then `components.r`, `components.g`, and `components.b` **must** correspond to components of the `format`; that is, `components.r`, `components.g`, and `components.b` **must** not be `VK_COMPONENT_SWIZZLE_ZERO` or `VK_COMPONENT_SWIZZLE_ONE`, and **must** not correspond to a component containing zero or one as a consequence of conversion to RGBA
- VUID-VkSamplerYcbcrConversionCreateInfo-ycbcrRange-02748
  If `ycbcrRange` is `VK_SAMPLER_YCBCR_RANGE_ITU_NARROW` then the R, G and B components obtained by applying the `component` swizzle to `format` **must** each have a bit-depth greater than or equal to 8
- VUID-VkSamplerYcbcrConversionCreateInfo-forceExplicitReconstruction-01656
  If the potential format features of the sampler $Y'C_BC_R$ conversion do not support `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEABLE_BIT` `forceExplicitReconstruction` **must** be `VK_FALSE`
- VUID-VkSamplerYcbcrConversionCreateInfo-chromaFilter-01657
  If the potential format features of the sampler $Y'C_BC_R$ conversion do not support `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT`, `chromaFilter` **must** not be `VK_FILTER_LINEAR`

Valid Usage (Implicit)

- VUID-VkSamplerYcbcrConversionCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_CREATE_INFO`
- VUID-VkSamplerYcbcrConversionCreateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkSamplerYcbcrConversionCreateInfo-format-parameter
  `format` **must** be a valid VkFormat value
- VUID-VkSamplerYcbcrConversionCreateInfo-ycbcrModel-parameter
  `ycbcrModel` **must** be a valid VkSamplerYcbcrModelConversion value
- VUID-VkSamplerYcbcrConversionCreateInfo-ycbcrRange-parameter
  `ycbcrRange` **must** be a valid VkSamplerYcbcrRange value
- VUID-VkSamplerYcbcrConversionCreateInfo-components-parameter
  `components` **must** be a valid VkComponentMapping structure
- VUID-VkSamplerYcbcrConversionCreateInfo-xChromaOffset-parameter
  `xChromaOffset` **must** be a valid VkChromaLocation value
- VUID-VkSamplerYcbcrConversionCreateInfo-yChromaOffset-parameter
  `yChromaOffset` **must** be a valid VkChromaLocation value
- VUID-VkSamplerYcbcrConversionCreateInfo-chromaFilter-parameter
  `chromaFilter` **must** be a valid VkFilter value

If chromaFilter is VK_FILTER_NEAREST, chroma samples are reconstructed to luma component resolution using nearest-neighbour sampling. Otherwise, chroma samples are reconstructed using interpolation. More details can be found in the description of sampler $Y'C_BC_R$ conversion in the Image Operations chapter.

VkSamplerYcbcrModelConversion defines the conversion from the source color model to the shader color model. Possible values are:

```
typedef enum VkSamplerYcbcrModelConversion {
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY = 0,
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_IDENTITY = 1,
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_709 = 2,
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_601 = 3,
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_2020 = 4,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY_KHR =
VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_IDENTITY_KHR =
VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_IDENTITY,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_709_KHR =
VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_709,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_601_KHR =
VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_601,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_2020_KHR =
VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_2020,
} VkSamplerYcbcrModelConversion;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkSamplerYcbcrModelConversion VkSamplerYcbcrModelConversionKHR;
```

- VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY specifies that the input values to the conversion are unmodified.
- VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_IDENTITY specifies no model conversion but the inputs are range expanded as for $Y'C_BC_R$.
- VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_709 specifies the color model conversion from $Y'C_BC_R$ to R'G'B' defined in BT.709 and described in the “BT.709 $Y'C_BC_R$ conversion” section of the Khronos Data Format Specification.
- VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_601 specifies the color model conversion from $Y'C_BC_R$ to R'G'B' defined in BT.601 and described in the “BT.601 $Y'C_BC_R$ conversion” section of the Khronos Data Format Specification.

- VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_2020 specifies the color model conversion from $Y'C_BC_R$ to R'G'B' defined in BT.2020 and described in the "BT.2020 $Y'C_BC_R$ conversion" section of the Khronos Data Format Specification.

In the VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_* color models, for the input to the sampler $Y'C_BC_R$ range expansion and model conversion:

- the Y (Y' luma) component corresponds to the G component of an RGB image.
- the CB (C_B or "U" blue color difference) component corresponds to the B component of an RGB image.
- the CR (C_R or "V" red color difference) component corresponds to the R component of an RGB image.
- the alpha component, if present, is not modified by color model conversion.

These rules reflect the mapping of components after the component swizzle operation (controlled by VkSamplerYcbcrConversionCreateInfo::components).

NOTE

For example, an "YUVA" 32-bit format comprising four 8-bit components can be implemented as VK_FORMAT_R8G8B8A8_UNORM with a component mapping:

- components.a = VK_COMPONENT_SWIZZLE_IDENTITY
- components.r = VK_COMPONENT_SWIZZLE_B
- components.g = VK_COMPONENT_SWIZZLE_R
- components.b = VK_COMPONENT_SWIZZLE_G

The VkSamplerYcbcrRange enum describes whether color components are encoded using the full range of numerical values or whether values are reserved for headroom and foot room. VkSamplerYcbcrRange is defined as:

```
typedef enum VkSamplerYcbcrRange {
    VK_SAMPLER_YCBCR_RANGE_ITU_FULL = 0,
    VK_SAMPLER_YCBCR_RANGE_ITU_NARROW = 1,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_SAMPLER_YCBCR_RANGE_ITU_FULL_KHR = VK_SAMPLER_YCBCR_RANGE_ITU_FULL,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_SAMPLER_YCBCR_RANGE_ITU_NARROW_KHR = VK_SAMPLER_YCBCR_RANGE_ITU_NARROW,
} VkSamplerYcbcrRange;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkSamplerYcbcrRange VkSamplerYcbcrRangeKHR;
```

- VK_SAMPLER_YCBCR_RANGE_ITU_FULL specifies that the full range of the encoded values are valid and interpreted according to the ITU "full range" quantization rules.

- VK_SAMPLER_YCBCR_RANGE_ITU_NARROW specifies that headroom and foot room are reserved in the numerical range of encoded values, and the remaining values are expanded according to the ITU “narrow range” quantization rules.

The formulae for these conversions is described in the Sampler $Y'C_BC_R$ Range Expansion section of the Image Operations chapter.

No range modification takes place if ycbcrModel is VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY; the ycbcrRange field of VkSamplerYcbcrConversionCreateInfo is ignored in this case.

The VkChromaLocation enum defines the location of downsampled chroma component samples relative to the luma samples, and is defined as:

```
typedef enum VkChromaLocation {
    VK_CHROMA_LOCATION_COSITED_EVEN = 0,
    VK_CHROMA_LOCATION_MIDPOINT = 1,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_CHROMA_LOCATION_COSITED_EVEN_KHR = VK_CHROMA_LOCATION_COSITED_EVEN,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_CHROMA_LOCATION_MIDPOINT_KHR = VK_CHROMA_LOCATION_MIDPOINT,
} VkChromaLocation;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkChromaLocation VkChromaLocationKHR;
```

- VK_CHROMA_LOCATION_COSITED_EVEN specifies that downsampled chroma samples are aligned with luma samples with even coordinates.
- VK_CHROMA_LOCATION_MIDPOINT specifies that downsampled chroma samples are located half way between each even luma sample and the nearest higher odd luma sample.

To destroy a sampler $Y'C_BC_R$ conversion, call:

```
// Provided by VK_KHR_sampler_ycbcr_conversion
void vkDestroySamplerYcbcrConversionKHR(
    VkDevice                                    device,
    VkSamplerYcbcrConversion                    ycbcrConversion,
    const VkAllocationCallbacks*                pAllocator);
```

- device is the logical device that destroys the $Y'C_BC_R$ conversion.
- ycbcrConversion is the conversion to destroy.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage (Implicit)

- VUID-vkDestroySamplerYcbcrConversion-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroySamplerYcbcrConversion-ycbcrConversion-parameter
  If `ycbcrConversion` is not VK_NULL_HANDLE, `ycbcrConversion` **must** be a valid VkSamplerYcbcrConversion handle
- VUID-vkDestroySamplerYcbcrConversion-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroySamplerYcbcrConversion-ycbcrConversion-parent
  If `ycbcrConversion` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `ycbcrConversion` **must** be externally synchronized

In addition to the predefined border color values, applications **can** provide a custom border color value by including the `VkSamplerCustomBorderColorCreateInfoEXT` structure in the VkSamplerCreateInfo::`pNext` chain.

The `VkSamplerCustomBorderColorCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_custom_border_color
typedef struct VkSamplerCustomBorderColorCreateInfoEXT {
    VkStructureType      sType;
    const void*          pNext;
    VkClearColorValue    customBorderColor;
    VkFormat             format;
} VkSamplerCustomBorderColorCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `customBorderColor` is a VkClearColorValue representing the desired custom sampler border color.
- `format` is a VkFormat representing the format of the sampled image view(s). This field may be `VK_FORMAT_UNDEFINED` if the `customBorderColorWithoutFormat` feature is enabled.

NOTE

If `format` is a depth/stencil format, the aspect is determined by the value of VkSamplerCreateInfo::`borderColor`. If VkSamplerCreateInfo::`borderColor` is `VK_BORDER_COLOR_FLOAT_CUSTOM_EXT`, the depth aspect is considered. If VkSamplerCreateInfo::`borderColor` is `VK_BORDER_COLOR_INT_CUSTOM_EXT`, the stencil

aspect is considered.

If `format` is `VK_FORMAT_UNDEFINED`, the VkSamplerCreateInfo::`borderColor` is `VK_BORDER_COLOR_INT_CUSTOM_EXT`, and the sampler is used with an image with a stencil format, then the implementation **must** source the custom border color from either the first or second components of VkSamplerCreateInfo::`customBorderColor` and **should** source it from the first component.

Valid Usage

- VUID-VkSamplerCustomBorderColorCreateInfoEXT-format-07605
  If `format` is not `VK_FORMAT_UNDEFINED` and `format` is not a depth/stencil format then the VkSamplerCreateInfo::`borderColor` type **must** match the sampled type of the provided `format`, as shown in the *SPIR-V Type* column of the Interpretation of Numeric Format table
- VUID-VkSamplerCustomBorderColorCreateInfoEXT-format-04014
  If the `customBorderColorWithoutFormat` feature is not enabled then `format` **must** not be `VK_FORMAT_UNDEFINED`
- VUID-VkSamplerCustomBorderColorCreateInfoEXT-format-04015
  If the sampler is used to sample an image view of `VK_FORMAT_B4G4R4A4_UNORM_PACK16`, `VK_FORMAT_B5G6R5_UNORM_PACK16`, `VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR`, or `VK_FORMAT_B5G5R5A1_UNORM_PACK16` format then `format` **must** not be `VK_FORMAT_UNDEFINED`

Valid Usage (Implicit)

- VUID-VkSamplerCustomBorderColorCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SAMPLER_CUSTOM_BORDER_COLOR_CREATE_INFO_EXT`
- VUID-VkSamplerCustomBorderColorCreateInfoEXT-format-parameter
  `format` **must** be a valid VkFormat value

If the sampler is created with `VK_BORDER_COLOR_FLOAT_OPAQUE_BLACK`, `VK_BORDER_COLOR_INT_OPAQUE_BLACK`, `VK_BORDER_COLOR_FLOAT_CUSTOM_EXT`, or `VK_BORDER_COLOR_INT_CUSTOM_EXT` `borderColor`, and that sampler will be combined with an image view that does not have an identity swizzle, and VkPhysicalDeviceBorderColorSwizzleFeaturesEXT::`borderColorSwizzleFromImage` is not enabled, then it is necessary to specify the component mapping of the border color, by including the `VkSamplerBorderColorComponentMappingCreateInfoEXT` structure in the VkSamplerCreateInfo::`pNext` chain, to get defined results.

The `VkSamplerBorderColorComponentMappingCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_border_color_swizzle
typedef struct VkSamplerBorderColorComponentMappingCreateInfoEXT {
    VkStructureType       sType;
    const void*           pNext;
    VkComponentMapping    components;
    VkBool32              srgb;
} VkSamplerBorderColorComponentMappingCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `components` is a VkComponentMapping structure specifying a remapping of the border color components.
- `srgb` indicates that the sampler will be combined with an image view that has an image format which is sRGB encoded.

The VkComponentMapping `components` member describes a remapping from components of the border color to components of the vector returned by shader image instructions when the border color is used.

Valid Usage

- VUID-VkSamplerBorderColorComponentMappingCreateInfoEXT-borderColorSwizzle-06437
  The `borderColorSwizzle` feature **must** be enabled

Valid Usage (Implicit)

- VUID-VkSamplerBorderColorComponentMappingCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SAMPLER_BORDER_COLOR_COMPONENT_MAPPING_CREATE_INFO_EXT`
- VUID-VkSamplerBorderColorComponentMappingCreateInfoEXT-components-parameter
  `components` **must** be a valid VkComponentMapping structure

Chapter 14. Resource Descriptors

A *descriptor* is an opaque data structure representing a shader resource such as a buffer, buffer view, image view, sampler, or combined image sampler. Descriptors are organized into *descriptor sets*, which are bound during command recording for use in subsequent drawing commands. The arrangement of content in each descriptor set is determined by a *descriptor set layout*, which determines what descriptors can be stored within it. The sequence of descriptor set layouts that **can** be used by a pipeline is specified in a *pipeline layout*. Each pipeline object **can** use up to `maxBoundDescriptorSets` (see Limits) descriptor sets.

Shaders access resources via variables decorated with a descriptor set and binding number that link them to a descriptor in a descriptor set. The shader interface mapping to bound descriptor sets is described in the Shader Resource Interface section.

Shaders **can** also access buffers without going through descriptors by using Physical Storage Buffer Access to access them through 64-bit addresses.

14.1. Descriptor Types

There are a number of different types of descriptor supported by Vulkan, corresponding to different resources or usage. The following sections describe the API definitions of each descriptor type. The mapping of each type to SPIR-V is listed in the Shader Resource and Descriptor Type Correspondence and Shader Resource and Storage Class Correspondence tables in the Shader Interfaces chapter.

14.1.1. Storage Image

A *storage image* (`VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`) is a descriptor type associated with an image resource via an image view that load, store, and atomic operations **can** be performed on.

Storage image loads are supported in all shader stages for image views whose format features contain `VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT`.

Stores to storage images are supported in compute shaders for image views whose format features contain `VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT`.

Atomic operations on storage images are supported in compute shaders for image views whose format features contain `VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT`.

When the `fragmentStoresAndAtomics` feature is enabled, stores and atomic operations are also supported for storage images in fragment shaders with the same set of image formats as supported in compute shaders. When the `vertexPipelineStoresAndAtomics` feature is enabled, stores and atomic operations are also supported in vertex, tessellation, and geometry shaders with the same set of image formats as supported in compute shaders.

The image subresources for a storage image **must** be in the `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR` or `VK_IMAGE_LAYOUT_GENERAL` layout in order to access its data in a shader.

14.1.2. Sampler

A *sampler descriptor* (VK_DESCRIPTOR_TYPE_SAMPLER) is a descriptor type associated with a sampler object, used to control the behavior of sampling operations performed on a sampled image.

14.1.3. Sampled Image

A *sampled image* (VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE) is a descriptor type associated with an image resource via an image view that sampling operations **can** be performed on.

Shaders combine a sampled image variable and a sampler variable to perform sampling operations.

Sampled images are supported in all shader stages for image views whose format features contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT.

An image subresources for a sampled image **must** be in one of the following layouts:

- VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_GENERAL
- VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR
- VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT

14.1.4. Combined Image Sampler

A *combined image sampler* (VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER) is a single descriptor type associated with both a sampler and an image resource, combining both a sampler and sampled image descriptor into a single descriptor.

If the descriptor refers to a sampler that performs $Y'C_BC_R$ conversion or samples a subsampled image, the sampler **must** only be used to sample the image in the same descriptor. Otherwise, the sampler and image in this type of descriptor **can** be used freely with any other samplers and images.

An image subresources for a combined image sampler **must** be in one of the following layouts:

- VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_GENERAL

- VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR
- VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT

NOTE	On some implementations, it **may** be more efficient to sample from an image using a combination of sampler and sampled image that are stored together in the descriptor set in a combined descriptor.

14.1.5. Uniform Texel Buffer

A *uniform texel buffer* (VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER) is a descriptor type associated with a buffer resource via a buffer view that image sampling operations **can** be performed on.

Uniform texel buffers define a tightly-packed 1-dimensional linear array of texels, with texels going through format conversion when read in a shader in the same way as they are for an image.

Load operations from uniform texel buffers are supported in all shader stages for buffer view formats which report format features support for VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT

14.1.6. Storage Texel Buffer

A *storage texel buffer* (VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER) is a descriptor type associated with a buffer resource via a buffer view that image load, store, and atomic operations **can** be performed on.

Storage texel buffers define a tightly-packed 1-dimensional linear array of texels, with texels going through format conversion when read in a shader in the same way as they are for an image. Unlike uniform texel buffers, these buffers can also be written to in the same way as for storage images.

Storage texel buffer loads are supported in all shader stages for texel buffer view formats which report format features support for VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT

Stores to storage texel buffers are supported in compute shaders for texel buffer formats which report format features support for VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT

Atomic operations on storage texel buffers are supported in compute shaders for texel buffer formats which report format features support for VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT

When the fragmentStoresAndAtomics feature is enabled, stores and atomic operations are also supported for storage texel buffers in fragment shaders with the same set of texel buffer formats as supported in compute shaders. When the vertexPipelineStoresAndAtomics feature is enabled, stores and atomic operations are also supported in vertex, tessellation, and geometry shaders with the

same set of texel buffer formats as supported in compute shaders.

14.1.7. Storage Buffer

A *storage buffer* (VK_DESCRIPTOR_TYPE_STORAGE_BUFFER) is a descriptor type associated with a buffer resource directly, described in a shader as a structure with various members that load, store, and atomic operations **can** be performed on.

NOTE	Atomic operations **can** only be performed on members of certain types as defined in the SPIR-V environment appendix.

14.1.8. Uniform Buffer

A *uniform buffer* (VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER) is a descriptor type associated with a buffer resource directly, described in a shader as a structure with various members that load operations **can** be performed on.

14.1.9. Dynamic Uniform Buffer

A *dynamic uniform buffer* (VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC) is almost identical to a uniform buffer, and differs only in how the offset into the buffer is specified. The base offset calculated by the VkDescriptorBufferInfo when initially updating the descriptor set is added to a dynamic offset when binding the descriptor set.

14.1.10. Dynamic Storage Buffer

A *dynamic storage buffer* (VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC) is almost identical to a storage buffer, and differs only in how the offset into the buffer is specified. The base offset calculated by the VkDescriptorBufferInfo when initially updating the descriptor set is added to a dynamic offset when binding the descriptor set.

14.1.11. Input Attachment

An *input attachment* (VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT) is a descriptor type associated with an image resource via an image view that **can** be used for framebuffer local load operations in fragment shaders.

All image formats that are supported for color attachments (VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT) or depth/stencil attachments (VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT) for a given image tiling mode are also supported for input attachments.

An image view used as an input attachment **must** be in one of the following layouts:

- VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_GENERAL
- VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR

- VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL
- VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL
- VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR
- VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT
- VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR

14.1.12. Acceleration Structure

An *acceleration structure* (VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR) is a descriptor type that is used to retrieve scene geometry from within shaders that are used for ray traversal. Shaders have read-only access to the memory.

14.2. Descriptor Sets

Descriptors are grouped together into descriptor set objects. A descriptor set object is an opaque object containing storage for a set of descriptors, where the types and number of descriptors is defined by a descriptor set layout. The layout object **may** be used to define the association of each descriptor binding with memory or other implementation resources. The layout is used both for determining the resources that need to be associated with the descriptor set, and determining the interface between shader stages and shader resources.

14.2.1. Descriptor Set Layout

A descriptor set layout object is defined by an array of zero or more descriptor bindings. Each individual descriptor binding is specified by a descriptor type, a count (array size) of the number of descriptors in the binding, a set of shader stages that **can** access the binding, and (if using immutable samplers) an array of sampler descriptors.

Descriptor set layout objects are represented by VkDescriptorSetLayout handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDescriptorSetLayout)
```

To create descriptor set layout objects, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateDescriptorSetLayout(
    VkDevice                                    device,
    const VkDescriptorSetLayoutCreateInfo*      pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkDescriptorSetLayout*                      pSetLayout);
```

- device is the logical device that creates the descriptor set layout.
- pCreateInfo is a pointer to a VkDescriptorSetLayoutCreateInfo structure specifying the state of the descriptor set layout object.

- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pSetLayout` is a pointer to a VkDescriptorSetLayout handle in which the resulting descriptor set layout object is returned.

Valid Usage

- VUID-vkCreateDescriptorSetLayout-support-09582
  If the descriptor layout exceeds the limits reported through the physical device limits, then vkGetDescriptorSetLayoutSupportKHR **must** have returned VkDescriptorSetLayoutSupport with `support` equal to `VK_TRUE` for `pCreateInfo`

Valid Usage (Implicit)

- VUID-vkCreateDescriptorSetLayout-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateDescriptorSetLayout-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkDescriptorSetLayoutCreateInfo structure
- VUID-vkCreateDescriptorSetLayout-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateDescriptorSetLayout-pSetLayout-parameter
  `pSetLayout` **must** be a valid pointer to a VkDescriptorSetLayout handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Information about the descriptor set layout is passed in a `VkDescriptorSetLayoutCreateInfo` structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkDescriptorSetLayoutCreateInfo {
    VkStructureType                        sType;
    const void*                            pNext;
    VkDescriptorSetLayoutCreateFlags       flags;
    uint32_t                               bindingCount;
    const VkDescriptorSetLayoutBinding*    pBindings;
} VkDescriptorSetLayoutCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkDescriptorSetLayoutCreateFlagBits specifying options for descriptor set layout creation.
- `bindingCount` is the number of elements in `pBindings`.
- `pBindings` is a pointer to an array of VkDescriptorSetLayoutBinding structures.

Valid Usage

- VUID-VkDescriptorSetLayoutCreateInfo-binding-00279
  The VkDescriptorSetLayoutBinding::`binding` members of the elements of the `pBindings` array **must** each have different values
- VUID-VkDescriptorSetLayoutCreateInfo-flags-00280
  If `flags` contains `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR`, then all elements of `pBindings` **must** not have a `descriptorType` of `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` or `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC`
- VUID-VkDescriptorSetLayoutCreateInfo-flags-00281
  If `flags` contains `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR`, then the total number of elements of all bindings **must** be less than or equal to VkPhysicalDevicePushDescriptorPropertiesKHR::`maxPushDescriptors`

Valid Usage (Implicit)

- VUID-VkDescriptorSetLayoutCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO`
- VUID-VkDescriptorSetLayoutCreateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDescriptorSetLayoutCreateInfo-flags-parameter
  `flags` **must** be a valid combination of VkDescriptorSetLayoutCreateFlagBits values
- VUID-VkDescriptorSetLayoutCreateInfo-pBindings-parameter
  If `bindingCount` is not `0`, `pBindings` **must** be a valid pointer to an array of `bindingCount` valid VkDescriptorSetLayoutBinding structures

Bits which **can** be set in VkDescriptorSetLayoutCreateInfo::flags, specifying options for descriptor set layout, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkDescriptorSetLayoutCreateFlagBits {
  // Provided by VK_KHR_push_descriptor
    VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR = 0x00000001,
} VkDescriptorSetLayoutCreateFlagBits;
```

- VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR specifies that descriptor sets **must** not be allocated using this layout, and descriptors are instead pushed by vkCmdPushDescriptorSetKHR.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkDescriptorSetLayoutCreateFlags;
```

VkDescriptorSetLayoutCreateFlags is a bitmask type for setting a mask of zero or more VkDescriptorSetLayoutCreateFlagBits.

The VkDescriptorSetLayoutBinding structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDescriptorSetLayoutBinding {
    uint32_t              binding;
    VkDescriptorType      descriptorType;
    uint32_t              descriptorCount;
    VkShaderStageFlags    stageFlags;
    const VkSampler*      pImmutableSamplers;
} VkDescriptorSetLayoutBinding;
```

- binding is the binding number of this entry and corresponds to a resource of the same binding number in the shader stages.
- descriptorType is a VkDescriptorType specifying which type of resource descriptors are used for this binding.
- descriptorCount is the number of descriptors contained in the binding, accessed in a shader as an array. If descriptorCount is zero this binding entry is reserved and the resource **must** not be accessed from any stage via this binding within any pipeline using the set layout.
- stageFlags member is a bitmask of VkShaderStageFlagBits specifying which pipeline shader stages **can** access a resource for this binding. VK_SHADER_STAGE_ALL is a shorthand specifying that all defined shader stages, including any additional stages defined by extensions, **can** access the resource.

  If a shader stage is not included in stageFlags, then a resource **must** not be accessed from that stage via this binding within any pipeline using the set layout. Other than input attachments which are limited to the fragment shader, there are no limitations on what combinations of

stages **can** use a descriptor binding, and in particular a binding **can** be used by both graphics stages and the compute stage.

- `pImmutableSamplers` affects initialization of samplers. If `descriptorType` specifies a `VK_DESCRIPTOR_TYPE_SAMPLER` or `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` type descriptor, then `pImmutableSamplers` **can** be used to initialize a set of *immutable samplers*. Immutable samplers are permanently bound into the set layout and **must** not be changed; updating a `VK_DESCRIPTOR_TYPE_SAMPLER` descriptor with immutable samplers is not allowed and updates to a `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` descriptor with immutable samplers does not modify the samplers (the image views are updated, but the sampler updates are ignored). If `pImmutableSamplers` is not `NULL`, then it is a pointer to an array of sampler handles that will be copied into the set layout and used for the corresponding binding. Only the sampler handles are copied; the sampler objects **must** not be destroyed before the final use of the set layout and any descriptor pools and sets created using it. If `pImmutableSamplers` is `NULL`, then the sampler slots are dynamic and sampler handles **must** be bound into descriptor sets using this layout. If `descriptorType` is not one of these descriptor types, then `pImmutableSamplers` is ignored.

The above layout definition allows the descriptor bindings to be specified sparsely such that not all binding numbers between 0 and the maximum binding number need to be specified in the `pBindings` array. Bindings that are not specified have a `descriptorCount` and `stageFlags` of zero, and the value of `descriptorType` is undefined. However, all binding numbers between 0 and the maximum binding number in the VkDescriptorSetLayoutCreateInfo::`pBindings` array **may** consume memory in the descriptor set layout even if not all descriptor bindings are used, though it **should** not consume additional memory from the descriptor pool.

NOTE	The maximum binding number specified **should** be as compact as possible to avoid wasted memory.

Valid Usage

- VUID-VkDescriptorSetLayoutBinding-descriptorType-00282
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_SAMPLER` or `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, and `descriptorCount` is not `0` and `pImmutableSamplers` is not `NULL`, `pImmutableSamplers` **must** be a valid pointer to an array of `descriptorCount` valid `VkSampler` handles
- VUID-VkDescriptorSetLayoutBinding-descriptorCount-09465
  If `descriptorCount` is not `0`, `stageFlags` **must** be `VK_SHADER_STAGE_ALL` or a valid combination of other VkShaderStageFlagBits values
- VUID-VkDescriptorSetLayoutBinding-descriptorType-01510
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` and `descriptorCount` is not `0`, then `stageFlags` **must** be `0` or `VK_SHADER_STAGE_FRAGMENT_BIT`
- VUID-VkDescriptorSetLayoutBinding-pImmutableSamplers-04009
  The sampler objects indicated by `pImmutableSamplers` **must** not have a `borderColor` with one of the values `VK_BORDER_COLOR_FLOAT_CUSTOM_EXT` or `VK_BORDER_COLOR_INT_CUSTOM_EXT`

Valid Usage (Implicit)

- VUID-VkDescriptorSetLayoutBinding-descriptorType-parameter
  `descriptorType` **must** be a valid VkDescriptorType value

To query information about whether a descriptor set layout **can** be created, call:

```
// Provided by VK_KHR_maintenance3
void vkGetDescriptorSetLayoutSupportKHR(
    VkDevice                                    device,
    const VkDescriptorSetLayoutCreateInfo*      pCreateInfo,
    VkDescriptorSetLayoutSupport*               pSupport);
```

- `device` is the logical device that would create the descriptor set layout.
- `pCreateInfo` is a pointer to a VkDescriptorSetLayoutCreateInfo structure specifying the state of the descriptor set layout object.
- `pSupport` is a pointer to a VkDescriptorSetLayoutSupport structure, in which information about support for the descriptor set layout object is returned.

Some implementations have limitations on what fits in a descriptor set which are not easily expressible in terms of existing limits like `maxDescriptorSet*`, for example if all descriptor types share a limited space in memory but each descriptor is a different size or alignment. This command returns information about whether a descriptor set satisfies this limit. If the descriptor set layout satisfies the VkPhysicalDeviceMaintenance3Properties::`maxPerSetDescriptors` limit, this command is guaranteed to return `VK_TRUE` in VkDescriptorSetLayoutSupport::`supported`. If the descriptor set layout exceeds the VkPhysicalDeviceMaintenance3Properties::`maxPerSetDescriptors` limit, whether the descriptor set layout is supported is implementation-dependent and **may** depend on whether the descriptor sizes and alignments cause the layout to exceed an internal limit.

This command does not consider other limits such as `maxPerStageDescriptor*`, and so a descriptor set layout that is supported according to this command **must** still satisfy the pipeline layout limits such as `maxPerStageDescriptor*` in order to be used in a pipeline layout.

NOTE This is a `VkDevice` query rather than `VkPhysicalDevice` because the answer **may** depend on enabled features.

Valid Usage (Implicit)

- VUID-vkGetDescriptorSetLayoutSupport-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDescriptorSetLayoutSupport-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkDescriptorSetLayoutCreateInfo structure
- VUID-vkGetDescriptorSetLayoutSupport-pSupport-parameter
  `pSupport` **must** be a valid pointer to a VkDescriptorSetLayoutSupport structure

Information about support for the descriptor set layout is returned in a VkDescriptorSetLayoutSupport structure:

```
typedef struct VkDescriptorSetLayoutSupport {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           supported;
} VkDescriptorSetLayoutSupport;
```

or the equivalent

```
// Provided by VK_KHR_maintenance3
typedef VkDescriptorSetLayoutSupport VkDescriptorSetLayoutSupportKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- supported specifies whether the descriptor set layout **can** be created.

supported will be VK_TRUE if the descriptor set **can** be created, or else VK_FALSE.

Valid Usage (Implicit)

- VUID-VkDescriptorSetLayoutSupport-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_SUPPORT
- VUID-VkDescriptorSetLayoutSupport-pNext-pNext
  pNext **must** be NULL

The following examples show a shader snippet using two descriptor sets, and application code that creates corresponding descriptor set layouts.

GLSL Example

```
//
// binding to a single sampled image descriptor in set 0
//
layout (set=0, binding=0) uniform texture2D mySampledImage;

//
// binding to an array of sampled image descriptors in set 0
//
layout (set=0, binding=1) uniform texture2D myArrayOfSampledImages[12];

//
// binding to a single uniform buffer descriptor in set 1
//
layout (set=1, binding=0) uniform myUniformBuffer
```

```
{
    vec4 myElement[32];
};
```

SPIR-V Example

```
            ...
         %1 = OpExtInstImport "GLSL.std.450"
            ...
            OpName %9 "mySampledImage"
            OpName %14 "myArrayOfSampledImages"
            OpName %18 "myUniformBuffer"
            OpMemberName %18 0 "myElement"
            OpName %20 ""
            OpDecorate %9 DescriptorSet 0
            OpDecorate %9 Binding 0
            OpDecorate %14 DescriptorSet 0
            OpDecorate %14 Binding 1
            OpDecorate %17 ArrayStride 16
            OpMemberDecorate %18 0 Offset 0
            OpDecorate %18 Block
            OpDecorate %20 DescriptorSet 1
            OpDecorate %20 Binding 0
         %2 = OpTypeVoid
         %3 = OpTypeFunction %2
         %6 = OpTypeFloat 32
         %7 = OpTypeImage %6 2D 0 0 0 1 Unknown
         %8 = OpTypePointer UniformConstant %7
         %9 = OpVariable %8 UniformConstant
        %10 = OpTypeInt 32 0
        %11 = OpConstant %10 12
        %12 = OpTypeArray %7 %11
        %13 = OpTypePointer UniformConstant %12
        %14 = OpVariable %13 UniformConstant
        %15 = OpTypeVector %6 4
        %16 = OpConstant %10 32
        %17 = OpTypeArray %15 %16
        %18 = OpTypeStruct %17
        %19 = OpTypePointer Uniform %18
        %20 = OpVariable %19 Uniform
            ...
```

API Example

```
VkResult myResult;

const VkDescriptorSetLayoutBinding myDescriptorSetLayoutBinding[] =
{
    // binding to a single image descriptor
    {
```

```
        .binding = 0,
        .descriptorType = VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE,
        .descriptorCount = 1,
        .stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT,
        .pImmutableSamplers = NULL
    },

    // binding to an array of image descriptors
    {
        .binding = 1,
        .descriptorType = VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE,
        .descriptorCount = 12,
        .stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT,
        .pImmutableSamplers = NULL
    },

    // binding to a single uniform buffer descriptor
    {
        .binding = 0,
        .descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER,
        .descriptorCount = 1,
        .stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT,
        .pImmutableSamplers = NULL
    }
};

const VkDescriptorSetLayoutCreateInfo myDescriptorSetLayoutCreateInfo[] =
{
    // Information for first descriptor set with two descriptor bindings
    {
        .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO,
        .pNext = NULL,
        .flags = 0,
        .bindingCount = 2,
        .pBindings = &myDescriptorSetLayoutBinding[0]
    },

    // Information for second descriptor set with one descriptor binding
    {
        .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO,
        .pNext = NULL,
        .flags = 0,
        .bindingCount = 1,
        .pBindings = &myDescriptorSetLayoutBinding[2]
    }
};

VkDescriptorSetLayout myDescriptorSetLayout[2];

//
// Create first descriptor set layout
```

```
//
myResult = vkCreateDescriptorSetLayout(
    myDevice,
    &myDescriptorSetLayoutCreateInfo[0],
    NULL,
    &myDescriptorSetLayout[0]);

//
// Create second descriptor set layout
//
myResult = vkCreateDescriptorSetLayout(
    myDevice,
    &myDescriptorSetLayoutCreateInfo[1],
    NULL,
    &myDescriptorSetLayout[1]);
```

To destroy a descriptor set layout, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyDescriptorSetLayout(
    VkDevice                                    device,
    VkDescriptorSetLayout                       descriptorSetLayout,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the descriptor set layout.
- `descriptorSetLayout` is the descriptor set layout to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyDescriptorSetLayout-descriptorSetLayout-00284
  If `VkAllocationCallbacks` were provided when `descriptorSetLayout` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyDescriptorSetLayout-descriptorSetLayout-00285
  If no `VkAllocationCallbacks` were provided when `descriptorSetLayout` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyDescriptorSetLayout-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyDescriptorSetLayout-descriptorSetLayout-parameter
  If `descriptorSetLayout` is not VK_NULL_HANDLE, `descriptorSetLayout` **must** be a valid VkDescriptorSetLayout handle

- VUID-vkDestroyDescriptorSetLayout-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyDescriptorSetLayout-descriptorSetLayout-parent
  If `descriptorSetLayout` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `descriptorSetLayout` **must** be externally synchronized

14.2.2. Pipeline Layouts

Access to descriptor sets from a pipeline is accomplished through a *pipeline layout*. Zero or more descriptor set layouts and zero or more push constant ranges are combined to form a pipeline layout object describing the complete set of resources that **can** be accessed by a pipeline. The pipeline layout represents a sequence of descriptor sets with each having a specific layout. This sequence of layouts is used to determine the interface between shader stages and shader resources. Each pipeline is created using a pipeline layout.

Pipeline layout objects are represented by `VkPipelineLayout` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkPipelineLayout)
```

To create a pipeline layout, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreatePipelineLayout(
    VkDevice                                    device,
    const VkPipelineLayoutCreateInfo*           pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkPipelineLayout*                           pPipelineLayout);
```

- `device` is the logical device that creates the pipeline layout.
- `pCreateInfo` is a pointer to a VkPipelineLayoutCreateInfo structure specifying the state of the pipeline layout object.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pPipelineLayout` is a pointer to a VkPipelineLayout handle in which the resulting pipeline layout object is returned.

Valid Usage (Implicit)

- VUID-vkCreatePipelineLayout-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreatePipelineLayout-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkPipelineLayoutCreateInfo structure
- VUID-vkCreatePipelineLayout-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreatePipelineLayout-pPipelineLayout-parameter
  `pPipelineLayout` **must** be a valid pointer to a VkPipelineLayout handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The VkPipelineLayoutCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineLayoutCreateInfo {
    VkStructureType                 sType;
    const void*                     pNext;
    VkPipelineLayoutCreateFlags     flags;
    uint32_t                        setLayoutCount;
    const VkDescriptorSetLayout*    pSetLayouts;
    uint32_t                        pushConstantRangeCount;
    const VkPushConstantRange*      pPushConstantRanges;
} VkPipelineLayoutCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkPipelineLayoutCreateFlagBits specifying options for pipeline layout creation.
- `setLayoutCount` is the number of descriptor sets included in the pipeline layout.
- `pSetLayouts` is a pointer to an array of `VkDescriptorSetLayout` objects.
- `pushConstantRangeCount` is the number of push constant ranges included in the pipeline layout.
- `pPushConstantRanges` is a pointer to an array of `VkPushConstantRange` structures defining a set of

push constant ranges for use in a single pipeline layout. In addition to descriptor set layouts, a pipeline layout also describes how many push constants **can** be accessed by each stage of the pipeline.

NOTE	Push constants represent a high speed path to modify constant data in pipelines that is expected to outperform memory-backed resource updates.

Valid Usage

- VUID-VkPipelineLayoutCreateInfo-setLayoutCount-00286
  `setLayoutCount` **must** be less than or equal to `VkPhysicalDeviceLimits::maxBoundDescriptorSets`
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03016
  The total number of descriptors in descriptor set layouts with a `descriptorType` of `VK_DESCRIPTOR_TYPE_SAMPLER` and `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` accessible to any given shader stage across all elements of `pSetLayouts` **must** be less than or equal to `VkPhysicalDeviceLimits::maxPerStageDescriptorSamplers`
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03017
  The total number of descriptors in descriptor set layouts with a `descriptorType` of `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER` and `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` accessible to any given shader stage across all elements of `pSetLayouts` **must** be less than or equal to `VkPhysicalDeviceLimits::maxPerStageDescriptorUniformBuffers`
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03018
  The total number of descriptors in descriptor set layouts with a `descriptorType` of `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER` and `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` accessible to any given shader stage across all elements of `pSetLayouts` **must** be less than or equal to `VkPhysicalDeviceLimits::maxPerStageDescriptorStorageBuffers`
- VUID-VkPipelineLayoutCreateInfo-descriptorType-06939
  The total number of descriptors in descriptor set layouts with a `descriptorType` of `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, and `VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER`, accessible to any given shader stage across all elements of `pSetLayouts` **must** be less than or equal to `VkPhysicalDeviceLimits::maxPerStageDescriptorSampledImages`
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03020
  The total number of descriptors in descriptor set layouts with a `descriptorType` of `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, and `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER` accessible to any given shader stage across all elements of `pSetLayouts` **must** be less than or equal to `VkPhysicalDeviceLimits::maxPerStageDescriptorStorageImages`
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03021
  The total number of descriptors in descriptor set layouts with a `descriptorType` of `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` accessible to any given shader stage across all elements of `pSetLayouts` **must** be less than or equal to `VkPhysicalDeviceLimits::maxPerStageDescriptorInputAttachments`
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03028

The total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_SAMPLER and VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceLimits::maxDescriptorSetSamplers

- VUID-VkPipelineLayoutCreateInfo-descriptorType-03029
  The total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceLimits::maxDescriptorSetUniformBuffers
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03030
  If the maintenance7 feature is not enabled, the total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceLimits::maxDescriptorSetUniformBuffersDynamic
- VUID-VkPipelineLayoutCreateInfo-maintenance7-10003
  If the maintenance7 feature is enabled, the total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceMaintenance7PropertiesKHR::maxDescriptorSetTotalUniformBuffersDynamic
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03031
  The total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_STORAGE_BUFFER accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceLimits::maxDescriptorSetStorageBuffers
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03032
  If the maintenance7 feature is not enabled, the total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceLimits::maxDescriptorSetStorageBuffersDynamic
- VUID-VkPipelineLayoutCreateInfo-maintenance7-10004
  If the maintenance7 feature is enabled, the total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceMaintenance7PropertiesKHR::maxDescriptorSetTotalStorageBuffersDynamic
- VUID-VkPipelineLayoutCreateInfo-None-10005
  The total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceMaintenance7PropertiesKHR::maxDescriptorSetTotalBuffersDynamic
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03033
  The total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, and

VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceLimits::maxDescriptorSetSampledImages

- VUID-VkPipelineLayoutCreateInfo-descriptorType-03034
  The total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, and VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceLimits::maxDescriptorSetStorageImages
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03035
  The total number of descriptors in descriptor set layouts with a descriptorType of VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceLimits::maxDescriptorSetInputAttachments
- VUID-VkPipelineLayoutCreateInfo-pPushConstantRanges-00292
  Any two elements of pPushConstantRanges **must** not include the same stage in stageFlags
- VUID-VkPipelineLayoutCreateInfo-pSetLayouts-00293
  pSetLayouts **must** not contain more than one descriptor set layout that was created with VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR set
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03571
  The total number of bindings in descriptor set layouts created without the VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT bit set with a descriptorType of VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR accessible to any given shader stage across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxPerStageDescriptorAccelerationStructures
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03572
  The total number of bindings with a descriptorType of VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR accessible to any given shader stage across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxPerStageDescriptorUpdateAfterBindAccelerationStructures
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03573
  The total number of bindings in descriptor set layouts created without the VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT bit set with a descriptorType of VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxDescriptorSetAccelerationStructures
- VUID-VkPipelineLayoutCreateInfo-descriptorType-03574
  The total number of bindings with a descriptorType of VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR accessible across all shader stages and across all elements of pSetLayouts **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxDescriptorSetUpdateAfterBindAccelerationStructures

- VUID-VkPipelineLayoutCreateInfo-pImmutableSamplers-03566
  The total number of `pImmutableSamplers` created with `flags` containing `VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT` or `VK_SAMPLER_CREATE_SUBSAMPLED_COARSE_RECONSTRUCTION_BIT_EXT` across all shader stages and across all elements of `pSetLayouts` **must** be less than or equal to `VkPhysicalDeviceFragmentDensityMap2PropertiesEXT::maxDescriptorSetSubsampledSamplers`
- VUID-VkPipelineLayoutCreateInfo-graphicsPipelineLibrary-06753
  Elements of `pSetLayouts` **must** be valid VkDescriptorSetLayout objects

Valid Usage (Implicit)

- VUID-VkPipelineLayoutCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO`
- VUID-VkPipelineLayoutCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineLayoutCreateInfo-pSetLayouts-parameter
  If `setLayoutCount` is not `0`, `pSetLayouts` **must** be a valid pointer to an array of `setLayoutCount` valid or VK_NULL_HANDLE VkDescriptorSetLayout handles
- VUID-VkPipelineLayoutCreateInfo-pPushConstantRanges-parameter
  If `pushConstantRangeCount` is not `0`, `pPushConstantRanges` **must** be a valid pointer to an array of `pushConstantRangeCount` valid VkPushConstantRange structures

```
typedef enum VkPipelineLayoutCreateFlagBits {
} VkPipelineLayoutCreateFlagBits;
```

All values for this enum are defined by extensions.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineLayoutCreateFlags;
```

`VkPipelineLayoutCreateFlags` is a bitmask type for setting a mask of VkPipelineLayoutCreateFlagBits.

The `VkPushConstantRange` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPushConstantRange {
    VkShaderStageFlags    stageFlags;
    uint32_t              offset;
    uint32_t              size;
} VkPushConstantRange;
```

- `stageFlags` is a set of stage flags describing the shader stages that will access a range of push

constants. If a particular stage is not included in the range, then accessing members of that range of push constants from the corresponding shader stage will return undefined values.

- `offset` and `size` are the start offset and size, respectively, consumed by the range. Both `offset` and `size` are in units of bytes and **must** be a multiple of 4. The layout of the push constant variables is specified in the shader.

Valid Usage

- VUID-VkPushConstantRange-offset-00294
  `offset` **must** be less than `VkPhysicalDeviceLimits`::`maxPushConstantsSize`
- VUID-VkPushConstantRange-offset-00295
  `offset` **must** be a multiple of `4`
- VUID-VkPushConstantRange-size-00296
  `size` **must** be greater than `0`
- VUID-VkPushConstantRange-size-00297
  `size` **must** be a multiple of `4`
- VUID-VkPushConstantRange-size-00298
  `size` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxPushConstantsSize` minus `offset`

Valid Usage (Implicit)

- VUID-VkPushConstantRange-stageFlags-parameter
  `stageFlags` **must** be a valid combination of VkShaderStageFlagBits values
- VUID-VkPushConstantRange-stageFlags-requiredbitmask
  `stageFlags` **must** not be `0`

Once created, pipeline layouts are used as part of pipeline creation (see Pipelines), as part of binding descriptor sets (see Descriptor Set Binding), and as part of setting push constants (see Push Constant Updates). Pipeline creation accepts a pipeline layout as input, and the layout **may** be used to map (set, binding, arrayElement) tuples to implementation resources or memory locations within a descriptor set. The assignment of implementation resources depends only on the bindings defined in the descriptor sets that comprise the pipeline layout, and not on any shader source.

All resource variables statically used in all shaders in a pipeline **must** be declared with a (set, binding, arrayElement) that exists in the corresponding descriptor set layout and is of an appropriate descriptor type and includes the set of shader stages it is used by in `stageFlags`. The pipeline layout **can** include entries that are not used by a particular pipeline. The pipeline layout allows the application to provide a consistent set of bindings across multiple pipeline compiles, which enables those pipelines to be compiled in a way that the implementation **may** cheaply switch pipelines without reprogramming the bindings.

Similarly, the push constant block declared in each shader (if present) **must** only place variables at offsets that are each included in a push constant range with `stageFlags` including the bit

corresponding to the shader stage that uses it. The pipeline layout **can** include ranges or portions of ranges that are not used by a particular pipeline.

There is a limit on the total number of resources of each type that **can** be included in bindings in all descriptor set layouts in a pipeline layout as shown in Pipeline Layout Resource Limits. The "Total Resources Available" column gives the limit on the number of each type of resource that **can** be included in bindings in all descriptor sets in the pipeline layout. Some resource types count against multiple limits. Additionally, there are limits on the total number of each type of resource that **can** be used in any pipeline stage as described in Shader Resource Limits.

Table 14. Pipeline Layout Resource Limits

Total Resources Available	Resource Types
`maxDescriptorSetSamplers`	sampler
	combined image sampler
`maxDescriptorSetSampledImages`	sampled image
	combined image sampler
	uniform texel buffer
`maxDescriptorSetStorageImages`	storage image
	storage texel buffer
`maxDescriptorSetUniformBuffers`	uniform buffer
	uniform buffer dynamic
`maxDescriptorSetUniformBuffersDynamic` or `maxDescriptorSetUpdateAfterBindTotalUniformBuffersDynamic`	uniform buffer dynamic
`maxDescriptorSetStorageBuffers`	storage buffer
	storage buffer dynamic
`maxDescriptorSetStorageBuffersDynamic` or `maxDescriptorSetUpdateAfterBindTotalStorageBuffersDynamic`	storage buffer dynamic
`maxDescriptorSetInputAttachments`	input attachment
`maxDescriptorSetAccelerationStructures` or `maxDescriptorSetUpdateAfterBindAccelerationStructures`	acceleration structure

To destroy a pipeline layout, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyPipelineLayout(
    VkDevice                                    device,
    VkPipelineLayout                            pipelineLayout,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the pipeline layout.

- `pipelineLayout` is the pipeline layout to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyPipelineLayout-pipelineLayout-00299
  If `VkAllocationCallbacks` were provided when `pipelineLayout` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyPipelineLayout-pipelineLayout-00300
  If no `VkAllocationCallbacks` were provided when `pipelineLayout` was created, `pAllocator` **must** be `NULL`
- VUID-vkDestroyPipelineLayout-pipelineLayout-02004
  `pipelineLayout` **must** not have been passed to any `vkCmd*` command for any command buffers that are still in the recording state when `vkDestroyPipelineLayout` is called

Valid Usage (Implicit)

- VUID-vkDestroyPipelineLayout-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyPipelineLayout-pipelineLayout-parameter
  If `pipelineLayout` is not VK_NULL_HANDLE, `pipelineLayout` **must** be a valid VkPipelineLayout handle
- VUID-vkDestroyPipelineLayout-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyPipelineLayout-pipelineLayout-parent
  If `pipelineLayout` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `pipelineLayout` **must** be externally synchronized

Pipeline Layout Compatibility

Two pipeline layouts are defined to be “compatible for push constants” if they were created with identical push constant ranges. Two pipeline layouts are defined to be “compatible for set N” if they were created with *identically defined* descriptor set layouts for sets zero through N, and if they were created with identical push constant ranges.

When binding a descriptor set (see Descriptor Set Binding) to set number N, a previously bound descriptor set bound with lower index M than N is disturbed if the pipeline layouts for set M and N are not compatible for set M. Otherwise, the bound descriptor set in M is not disturbed.

If, additionally, the previously bound descriptor set for set N was bound using a pipeline layout not compatible for set N, then all bindings in sets numbered greater than N are disturbed.

When binding a pipeline, the pipeline **can** correctly access any previously bound descriptor set N if it was bound with compatible pipeline layout for set N, and it was not disturbed.

Layout compatibility means that descriptor sets **can** be bound to a command buffer for use by any pipeline created with a compatible pipeline layout, and without having bound a particular pipeline first. It also means that descriptor sets **can** remain valid across a pipeline change, and the same resources will be accessible to the newly bound pipeline.

When a descriptor set is disturbed by binding descriptor sets, the disturbed set is considered to contain undefined descriptors bound with the same pipeline layout as the disturbing descriptor set.

Implementor's Note

A consequence of layout compatibility is that when the implementation compiles a pipeline layout and maps pipeline resources to implementation resources, the mechanism for set N **should** only be a function of sets [0..N].

NOTE Place the least frequently changing descriptor sets near the start of the pipeline layout, and place the descriptor sets representing the most frequently changing resources near the end. When pipelines are switched, only the descriptor set bindings that have been invalidated will need to be updated and the remainder of the descriptor set bindings will remain in place.

The maximum number of descriptor sets that **can** be bound to a pipeline layout is queried from physical device properties (see `maxBoundDescriptorSets` in Limits).

API Example

```
const VkDescriptorSetLayout layouts[] = { layout1, layout2 };

const VkPushConstantRange ranges[] =
{
    {
        .stageFlags = VK_SHADER_STAGE_VERTEX_BIT,
        .offset = 0,
        .size = 4
    },
    {
        .stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT,
        .offset = 4,
        .size = 4
    },
};

const VkPipelineLayoutCreateInfo createInfo =
{
```

```
    .sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO,
    .pNext = NULL,
    .flags = 0,
    .setLayoutCount = 2,
    .pSetLayouts = layouts,
    .pushConstantRangeCount = 2,
    .pPushConstantRanges = ranges
};

VkPipelineLayout myPipelineLayout;
myResult = vkCreatePipelineLayout(
    myDevice,
    &createInfo,
    NULL,
    &myPipelineLayout);
```

14.2.3. Allocation of Descriptor Sets

A *descriptor pool* maintains a pool of descriptors, from which descriptor sets are allocated. Descriptor pools are externally synchronized, meaning that the application **must** not allocate and/or free descriptor sets from the same pool in multiple threads simultaneously.

Descriptor pools are represented by `VkDescriptorPool` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDescriptorPool)
```

To create a descriptor pool object, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateDescriptorPool(
    VkDevice                                    device,
    const VkDescriptorPoolCreateInfo*           pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkDescriptorPool*                           pDescriptorPool);
```

- `device` is the logical device that creates the descriptor pool.
- `pCreateInfo` is a pointer to a VkDescriptorPoolCreateInfo structure specifying the state of the descriptor pool object.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pDescriptorPool` is a pointer to a VkDescriptorPool handle in which the resulting descriptor pool object is returned.

The created descriptor pool is returned in `pDescriptorPool`.

Valid Usage (Implicit)

- VUID-vkCreateDescriptorPool-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateDescriptorPool-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkDescriptorPoolCreateInfo structure
- VUID-vkCreateDescriptorPool-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateDescriptorPool-pDescriptorPool-parameter
  `pDescriptorPool` **must** be a valid pointer to a VkDescriptorPool handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Additional information about the pool is passed in a `VkDescriptorPoolCreateInfo` structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkDescriptorPoolCreateInfo {
    VkStructureType                sType;
    const void*                    pNext;
    VkDescriptorPoolCreateFlags    flags;
    uint32_t                       maxSets;
    uint32_t                       poolSizeCount;
    const VkDescriptorPoolSize*    pPoolSizes;
} VkDescriptorPoolCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkDescriptorPoolCreateFlagBits specifying certain supported operations on the pool.
- `maxSets` is the maximum number of descriptor sets that **can** be allocated from the pool.
- `poolSizeCount` is the number of elements in `pPoolSizes`.
- `pPoolSizes` is a pointer to an array of VkDescriptorPoolSize structures, each containing a descriptor type and number of descriptors of that type to be allocated in the pool.

If multiple VkDescriptorPoolSize structures containing the same descriptor type appear in the pPoolSizes array then the pool will be created with enough storage for the total number of descriptors of each type.

Fragmentation of a descriptor pool is possible and **may** lead to descriptor set allocation failures. A failure due to fragmentation is defined as failing a descriptor set allocation despite the sum of all outstanding descriptor set allocations from the pool plus the requested allocation requiring no more than the total number of descriptors requested at pool creation. Implementations provide certain guarantees of when fragmentation **must** not cause allocation failure, as described below.

If a descriptor pool has not had any descriptor sets freed since it was created or most recently reset then fragmentation **must** not cause an allocation failure (note that this is always the case for a pool created without the VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT bit set). Additionally, if all sets allocated from the pool since it was created or most recently reset use the same number of descriptors (of each type) and the requested allocation also uses that same number of descriptors (of each type), then fragmentation **must** not cause an allocation failure.

If an allocation failure occurs due to fragmentation, an application **can** create an additional descriptor pool to perform further descriptor set allocations.

Valid Usage

- VUID-VkDescriptorPoolCreateInfo-descriptorPoolOverallocation-09227
  maxSets **must** be greater than 0

Valid Usage (Implicit)

- VUID-VkDescriptorPoolCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO
- VUID-VkDescriptorPoolCreateInfo-pNext-pNext
  pNext **must** be NULL
- VUID-VkDescriptorPoolCreateInfo-flags-parameter
  flags **must** be a valid combination of VkDescriptorPoolCreateFlagBits values
- VUID-VkDescriptorPoolCreateInfo-pPoolSizes-parameter
  If poolSizeCount is not 0, pPoolSizes **must** be a valid pointer to an array of poolSizeCount valid VkDescriptorPoolSize structures

Bits which **can** be set in VkDescriptorPoolCreateInfo::flags, enabling operations on a descriptor pool, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkDescriptorPoolCreateFlagBits {
    VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT = 0x00000001,
} VkDescriptorPoolCreateFlagBits;
```

- VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT specifies that descriptor sets **can** return their individual allocations to the pool, i.e. all of vkAllocateDescriptorSets, vkFreeDescriptorSets, and vkResetDescriptorPool are allowed. Otherwise, descriptor sets allocated from the pool **must** not be individually freed back to the pool, i.e. only vkAllocateDescriptorSets and vkResetDescriptorPool are allowed.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkDescriptorPoolCreateFlags;
```

VkDescriptorPoolCreateFlags is a bitmask type for setting a mask of zero or more VkDescriptorPoolCreateFlagBits.

The VkDescriptorPoolSize structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDescriptorPoolSize {
    VkDescriptorType    type;
    uint32_t            descriptorCount;
} VkDescriptorPoolSize;
```

- type is the type of descriptor.
- descriptorCount is the number of descriptors of that type to allocate.

NOTE

When creating a descriptor pool that will contain descriptors for combined image samplers of multi-planar formats, an application needs to account for non-trivial descriptor consumption when choosing the descriptorCount value, as indicated by VkSamplerYcbcrConversionImageFormatProperties::combinedImageSamplerDescriptorCount.

For simplicity the application **can** use the VkPhysicalDeviceMaintenance6PropertiesKHR::maxCombinedImageSamplerDescriptorCount property, which is sized to accommodate any and all formats that require a sampler $Y'C_BC_R$ conversion supported by the implementation.

Valid Usage

- VUID-VkDescriptorPoolSize-descriptorCount-00302
  descriptorCount **must** be greater than 0

Valid Usage (Implicit)

- VUID-VkDescriptorPoolSize-type-parameter
  type **must** be a valid VkDescriptorType value

To destroy a descriptor pool, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyDescriptorPool(
    VkDevice                                    device,
    VkDescriptorPool                            descriptorPool,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the descriptor pool.
- `descriptorPool` is the descriptor pool to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

When a pool is destroyed, all descriptor sets allocated from the pool are implicitly freed and become invalid. Descriptor sets allocated from a given pool do not need to be freed before destroying that descriptor pool.

Valid Usage

- VUID-vkDestroyDescriptorPool-descriptorPool-00303
  All submitted commands that refer to `descriptorPool` (via any allocated descriptor sets) **must** have completed execution
- VUID-vkDestroyDescriptorPool-descriptorPool-00304
  If `VkAllocationCallbacks` were provided when `descriptorPool` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyDescriptorPool-descriptorPool-00305
  If no `VkAllocationCallbacks` were provided when `descriptorPool` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyDescriptorPool-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyDescriptorPool-descriptorPool-parameter
  If `descriptorPool` is not VK_NULL_HANDLE, `descriptorPool` **must** be a valid VkDescriptorPool handle
- VUID-vkDestroyDescriptorPool-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyDescriptorPool-descriptorPool-parent
  If `descriptorPool` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `descriptorPool` **must** be externally synchronized

Descriptor sets are allocated from descriptor pool objects, and are represented by `VkDescriptorSet` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDescriptorSet)
```

To allocate descriptor sets from a descriptor pool, call:

```
// Provided by VK_VERSION_1_0
VkResult vkAllocateDescriptorSets(
    VkDevice                                    device,
    const VkDescriptorSetAllocateInfo*          pAllocateInfo,
    VkDescriptorSet*                            pDescriptorSets);
```

- `device` is the logical device that owns the descriptor pool.
- `pAllocateInfo` is a pointer to a VkDescriptorSetAllocateInfo structure describing parameters of the allocation.
- `pDescriptorSets` is a pointer to an array of VkDescriptorSet handles in which the resulting descriptor set objects are returned.

The allocated descriptor sets are returned in `pDescriptorSets`.

When a descriptor set is allocated, the initial state is largely uninitialized and all descriptors are undefined, with the exception that samplers with a non-null `pImmutableSamplers` are initialized on allocation. Descriptors also become undefined if the underlying resource or view object is destroyed. Descriptor sets containing undefined descriptors **can** still be bound and used, subject to the following conditions:

- Descriptors that are statically used **must** have been populated before the descriptor set is consumed.
- Entries that are not used by a pipeline **can** have undefined descriptors.

If a call to `vkAllocateDescriptorSets` would cause the total number of descriptor sets allocated from the pool to exceed the value of VkDescriptorPoolCreateInfo::`maxSets` used to create `pAllocateInfo->descriptorPool`, then the allocation **may** fail due to lack of space in the descriptor pool. Similarly, the allocation **may** fail due to lack of space if the call to `vkAllocateDescriptorSets` would cause the number of any given descriptor type to exceed the sum of all the `descriptorCount` members of each element of VkDescriptorPoolCreateInfo::`pPoolSizes` with a `type` equal to that type.

If the allocation fails due to no more space in the descriptor pool, and not because of system or device memory exhaustion, then `VK_ERROR_OUT_OF_POOL_MEMORY` **must** be returned.

vkAllocateDescriptorSets **can** be used to create multiple descriptor sets. If the creation of any of those descriptor sets fails, then the implementation **must** destroy all successfully created descriptor set objects from this command, set all entries of the pDescriptorSets array to VK_NULL_HANDLE and return the error.

Valid Usage (Implicit)

- VUID-vkAllocateDescriptorSets-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkAllocateDescriptorSets-pAllocateInfo-parameter
  pAllocateInfo **must** be a valid pointer to a valid VkDescriptorSetAllocateInfo structure
- VUID-vkAllocateDescriptorSets-pDescriptorSets-parameter
  pDescriptorSets **must** be a valid pointer to an array of pAllocateInfo->descriptorSetCount VkDescriptorSet handles
- VUID-vkAllocateDescriptorSets-pAllocateInfo::descriptorSetCount-arraylength
  pAllocateInfo->descriptorSetCount **must** be greater than 0

Host Synchronization

- Host access to pAllocateInfo->descriptorPool **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_FRAGMENTED_POOL
- VK_ERROR_OUT_OF_POOL_MEMORY

The VkDescriptorSetAllocateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDescriptorSetAllocateInfo {
    VkStructureType                 sType;
    const void*                     pNext;
    VkDescriptorPool                descriptorPool;
    uint32_t                        descriptorSetCount;
    const VkDescriptorSetLayout*    pSetLayouts;
} VkDescriptorSetAllocateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `descriptorPool` is the pool which the sets will be allocated from.
- `descriptorSetCount` determines the number of descriptor sets to be allocated from the pool.
- `pSetLayouts` is a pointer to an array of descriptor set layouts, with each member specifying how the corresponding descriptor set is allocated.

Valid Usage

- VUID-VkDescriptorSetAllocateInfo-apiVersion-07895
  If the VK_KHR_maintenance1 extension is not enabled and VkPhysicalDeviceProperties ::`apiVersion` is less than Vulkan 1.1, `descriptorSetCount` **must** not be greater than the number of sets that are currently available for allocation in `descriptorPool`
- VUID-VkDescriptorSetAllocateInfo-apiVersion-07896
  If the VK_KHR_maintenance1 extension is not enabled and VkPhysicalDeviceProperties ::`apiVersion` is less than Vulkan 1.1, `descriptorPool` **must** have enough free descriptor capacity remaining to allocate the descriptor sets of the specified layouts
- VUID-VkDescriptorSetAllocateInfo-pSetLayouts-00308
  Each element of `pSetLayouts` **must** not have been created with `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR` set

Valid Usage (Implicit)

- VUID-VkDescriptorSetAllocateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO`
- VUID-VkDescriptorSetAllocateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDescriptorSetAllocateInfo-descriptorPool-parameter
  `descriptorPool` **must** be a valid VkDescriptorPool handle
- VUID-VkDescriptorSetAllocateInfo-pSetLayouts-parameter
  `pSetLayouts` **must** be a valid pointer to an array of `descriptorSetCount` valid VkDescriptorSetLayout handles
- VUID-VkDescriptorSetAllocateInfo-descriptorSetCount-arraylength
  `descriptorSetCount` **must** be greater than `0`
- VUID-VkDescriptorSetAllocateInfo-commonparent
  Both of `descriptorPool`, and the elements of `pSetLayouts` **must** have been created, allocated, or retrieved from the same VkDevice

To free allocated descriptor sets, call:

```
// Provided by VK_VERSION_1_0
VkResult vkFreeDescriptorSets(
    VkDevice                                    device,
    VkDescriptorPool                            descriptorPool,
    uint32_t                                    descriptorSetCount,
    const VkDescriptorSet*                      pDescriptorSets);
```

- `device` is the logical device that owns the descriptor pool.
- `descriptorPool` is the descriptor pool from which the descriptor sets were allocated.
- `descriptorSetCount` is the number of elements in the `pDescriptorSets` array.
- `pDescriptorSets` is a pointer to an array of handles to VkDescriptorSet objects.

After calling `vkFreeDescriptorSets`, all descriptor sets in `pDescriptorSets` are invalid.

Valid Usage

- VUID-vkFreeDescriptorSets-pDescriptorSets-00309
  All submitted commands that refer to any element of `pDescriptorSets` **must** have completed execution
- VUID-vkFreeDescriptorSets-pDescriptorSets-00310
  `pDescriptorSets` **must** be a valid pointer to an array of `descriptorSetCount` `VkDescriptorSet` handles, each element of which **must** either be a valid handle or VK_NULL_HANDLE
- VUID-vkFreeDescriptorSets-descriptorPool-00312
  `descriptorPool` **must** have been created with the `VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT` flag

Valid Usage (Implicit)

- VUID-vkFreeDescriptorSets-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkFreeDescriptorSets-descriptorPool-parameter
  `descriptorPool` **must** be a valid VkDescriptorPool handle
- VUID-vkFreeDescriptorSets-descriptorSetCount-arraylength
  `descriptorSetCount` **must** be greater than `0`
- VUID-vkFreeDescriptorSets-descriptorPool-parent
  `descriptorPool` **must** have been created, allocated, or retrieved from `device`
- VUID-vkFreeDescriptorSets-pDescriptorSets-parent
  Each element of `pDescriptorSets` that is a valid handle **must** have been created, allocated, or retrieved from `descriptorPool`

Host Synchronization

- Host access to `descriptorPool` **must** be externally synchronized
- Host access to each member of `pDescriptorSets` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

None

To return all descriptor sets allocated from a given pool to the pool, rather than freeing individual descriptor sets, call:

```
// Provided by VK_VERSION_1_0
VkResult vkResetDescriptorPool(
    VkDevice                                    device,
    VkDescriptorPool                            descriptorPool,
    VkDescriptorPoolResetFlags                  flags);
```

- `device` is the logical device that owns the descriptor pool.
- `descriptorPool` is the descriptor pool to be reset.
- `flags` is reserved for future use.

Resetting a descriptor pool recycles all of the resources from all of the descriptor sets allocated from the descriptor pool back to the descriptor pool, and the descriptor sets are implicitly freed.

Valid Usage

- VUID-vkResetDescriptorPool-descriptorPool-00313
  All uses of `descriptorPool` (via any allocated descriptor sets) **must** have completed execution

Valid Usage (Implicit)

- VUID-vkResetDescriptorPool-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkResetDescriptorPool-descriptorPool-parameter
  `descriptorPool` **must** be a valid VkDescriptorPool handle
- VUID-vkResetDescriptorPool-flags-zerobitmask

`flags` **must** be `0`

- VUID-vkResetDescriptorPool-descriptorPool-parent
  `descriptorPool` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `descriptorPool` **must** be externally synchronized
- Host access to any `VkDescriptorSet` objects allocated from `descriptorPool` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

None

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkDescriptorPoolResetFlags;
```

`VkDescriptorPoolResetFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

14.2.4. Descriptor Set Updates

Once allocated, descriptor sets **can** be updated with a combination of write and copy operations. To update descriptor sets, call:

```
// Provided by VK_VERSION_1_0
void vkUpdateDescriptorSets(
    VkDevice                                    device,
    uint32_t                                    descriptorWriteCount,
    const VkWriteDescriptorSet*                 pDescriptorWrites,
    uint32_t                                    descriptorCopyCount,
    const VkCopyDescriptorSet*                  pDescriptorCopies);
```

- `device` is the logical device that updates the descriptor sets.
- `descriptorWriteCount` is the number of elements in the `pDescriptorWrites` array.
- `pDescriptorWrites` is a pointer to an array of VkWriteDescriptorSet structures describing the descriptor sets to write to.
- `descriptorCopyCount` is the number of elements in the `pDescriptorCopies` array.

- pDescriptorCopies is a pointer to an array of VkCopyDescriptorSet structures describing the descriptor sets to copy between.

The operations described by pDescriptorWrites are performed first, followed by the operations described by pDescriptorCopies. Within each array, the operations are performed in the order they appear in the array.

Each element in the pDescriptorWrites array describes an operation updating the descriptor set using descriptors for resources specified in the structure.

Each element in the pDescriptorCopies array is a VkCopyDescriptorSet structure describing an operation copying descriptors between sets.

If the dstSet member of any element of pDescriptorWrites or pDescriptorCopies is bound, accessed, or modified by any command that was recorded to a command buffer which is currently in the recording or executable state, that command buffer becomes invalid.

Valid Usage

- VUID-vkUpdateDescriptorSets-pDescriptorWrites-06236
  For each element i where pDescriptorWrites[i].descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER, elements of the pTexelBufferView member of pDescriptorWrites[i] **must** have been created on device
- VUID-vkUpdateDescriptorSets-pDescriptorWrites-06237
  For each element i where pDescriptorWrites[i].descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC, or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC, the buffer member of any element of the pBufferInfo member of pDescriptorWrites[i] **must** have been created on device
- VUID-vkUpdateDescriptorSets-pDescriptorWrites-06238
  For each element i where pDescriptorWrites[i].descriptorType is VK_DESCRIPTOR_TYPE_SAMPLER or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, and dstSet was not allocated with a layout that included immutable samplers for dstBinding with descriptorType, the sampler member of any element of the pImageInfo member of pDescriptorWrites[i] **must** have been created on device
- VUID-vkUpdateDescriptorSets-pDescriptorWrites-06239
  For each element i where pDescriptorWrites[i].descriptorType is VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT, or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER the imageView member of any element of pDescriptorWrites[i] **must** have been created on device
- VUID-vkUpdateDescriptorSets-pDescriptorWrites-06240
  For each element i where pDescriptorWrites[i].descriptorType is VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR, elements of the pAccelerationStructures member of a VkWriteDescriptorSetAccelerationStructureKHR structure in the pNext chain of pDescriptorWrites[i] **must** have been created on device

- VUID-vkUpdateDescriptorSets-pDescriptorWrites-06493
  For each element i where pDescriptorWrites[i].descriptorType is VK_DESCRIPTOR_TYPE_SAMPLER, VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT, pDescriptorWrites[i].pImageInfo **must** be a valid pointer to an array of pDescriptorWrites[i].descriptorCount valid VkDescriptorImageInfo structures
- VUID-vkUpdateDescriptorSets-None-03047
  The dstSet member of each element of pDescriptorWrites or pDescriptorCopies **must** not be used by any command that was recorded to a command buffer which is in the pending state
- VUID-vkUpdateDescriptorSets-pDescriptorWrites-06993
  Host access to pDescriptorWrites[i].dstSet and pDescriptorCopies[i].dstSet **must** be externally synchronized

Valid Usage (Implicit)

- VUID-vkUpdateDescriptorSets-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkUpdateDescriptorSets-pDescriptorWrites-parameter
  If descriptorWriteCount is not 0, pDescriptorWrites **must** be a valid pointer to an array of descriptorWriteCount valid VkWriteDescriptorSet structures
- VUID-vkUpdateDescriptorSets-pDescriptorCopies-parameter
  If descriptorCopyCount is not 0, pDescriptorCopies **must** be a valid pointer to an array of descriptorCopyCount valid VkCopyDescriptorSet structures

The VkWriteDescriptorSet structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkWriteDescriptorSet {
    VkStructureType                  sType;
    const void*                      pNext;
    VkDescriptorSet                  dstSet;
    uint32_t                         dstBinding;
    uint32_t                         dstArrayElement;
    uint32_t                         descriptorCount;
    VkDescriptorType                 descriptorType;
    const VkDescriptorImageInfo*     pImageInfo;
    const VkDescriptorBufferInfo*    pBufferInfo;
    const VkBufferView*              pTexelBufferView;
} VkWriteDescriptorSet;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- `dstSet` is the destination descriptor set to update.
- `dstBinding` is the descriptor binding within that set.
- `dstArrayElement` is the starting element in that array.
- `descriptorCount` is the number of descriptors to update. `descriptorCount` is one of
  - the number of elements in `pImageInfo`
  - the number of elements in `pBufferInfo`
  - the number of elements in `pTexelBufferView`
  - a value matching the `accelerationStructureCount` of a VkWriteDescriptorSetAccelerationStructureKHR structure in the `pNext` chain
- `descriptorType` is a VkDescriptorType specifying the type of each descriptor in `pImageInfo`, `pBufferInfo`, or `pTexelBufferView`, as described below. It **must** be the same type as the `descriptorType` specified in `VkDescriptorSetLayoutBinding` for `dstSet` at `dstBinding`. The type of the descriptor also controls which array the descriptors are taken from.
- `pImageInfo` is a pointer to an array of VkDescriptorImageInfo structures or is ignored, as described below.
- `pBufferInfo` is a pointer to an array of VkDescriptorBufferInfo structures or is ignored, as described below.
- `pTexelBufferView` is a pointer to an array of VkBufferView handles as described in the Buffer Views section or is ignored, as described below.

Only one of `pImageInfo`, `pBufferInfo`, or `pTexelBufferView` members is used according to the descriptor type specified in the `descriptorType` member of the containing `VkWriteDescriptorSet` structure, or if `descriptorType` is `VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR`, in which case the source data for the descriptor writes is taken from the VkWriteDescriptorSetAccelerationStructureKHR structure in the `pNext` chain of `VkWriteDescriptorSet`, as specified below.

If the `nullDescriptor` feature is enabled, the buffer, acceleration structure, imageView, or bufferView **can** be VK_NULL_HANDLE. Loads from a null descriptor return zero values and stores and atomics to a null descriptor are discarded. A null acceleration structure descriptor results in the miss shader being invoked.

Consecutive Binding Updates

If the `dstBinding` has fewer than `descriptorCount` array elements remaining starting from `dstArrayElement`, then the remainder will be used to update the subsequent binding - `dstBinding`+1 starting at array element zero. If a binding has a `descriptorCount` of zero, it is skipped. This behavior applies recursively, with the update affecting consecutive bindings as needed to update all `descriptorCount` descriptors. Consecutive bindings **must** have identical VkDescriptorType, VkShaderStageFlags, and immutable samplers references.

Valid Usage

- VUID-VkWriteDescriptorSet-dstBinding-00315
  `dstBinding` **must** be less than or equal to the maximum value of `binding` of all

VkDescriptorSetLayoutBinding structures specified when dstSet's descriptor set layout was created

- VUID-VkWriteDescriptorSet-dstBinding-00316
  dstBinding **must** be a binding with a non-zero descriptorCount
- VUID-VkWriteDescriptorSet-dstBinding-10009
  dstBinding **must** be a binding with a non-zero VkDescriptorSetLayoutCreateInfo ::bindingCount
- VUID-VkWriteDescriptorSet-descriptorCount-00317
  All consecutive bindings updated via a single VkWriteDescriptorSet structure, except those with a descriptorCount of zero, **must** have identical descriptorType and stageFlags
- VUID-VkWriteDescriptorSet-descriptorCount-00318
  All consecutive bindings updated via a single VkWriteDescriptorSet structure, except those with a descriptorCount of zero, **must** all either use immutable samplers or **must** all not use immutable samplers
- VUID-VkWriteDescriptorSet-descriptorType-00319
  descriptorType **must** match the type of dstBinding within dstSet
- VUID-VkWriteDescriptorSet-dstSet-00320
  dstSet **must** be a valid VkDescriptorSet handle
- VUID-VkWriteDescriptorSet-dstArrayElement-00321
  The sum of dstArrayElement and descriptorCount **must** be less than or equal to the number of array elements in the descriptor set binding specified by dstBinding, and all applicable consecutive bindings
- VUID-VkWriteDescriptorSet-descriptorType-02994
  If descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER, each element of pTexelBufferView **must** be either a valid VkBufferView handle or VK_NULL_HANDLE
- VUID-VkWriteDescriptorSet-descriptorType-02995
  If descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER and the nullDescriptor feature is not enabled, each element of pTexelBufferView **must** not be VK_NULL_HANDLE
- VUID-VkWriteDescriptorSet-descriptorType-00324
  If descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC, or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC, pBufferInfo **must** be a valid pointer to an array of descriptorCount valid VkDescriptorBufferInfo structures
- VUID-VkWriteDescriptorSet-descriptorType-00325
  If descriptorType is VK_DESCRIPTOR_TYPE_SAMPLER or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, and dstSet was not allocated with a layout that included immutable samplers for dstBinding with descriptorType, the sampler member of each element of pImageInfo **must** be a valid VkSampler object
- VUID-VkWriteDescriptorSet-descriptorType-02996
  If descriptorType is VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, or VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, the imageView

member of each element of pImageInfo **must** be either a valid VkImageView handle or VK_NULL_HANDLE

- VUID-VkWriteDescriptorSet-descriptorType-02997
  If descriptorType is VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, or VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, and the nullDescriptor feature is not enabled, the imageView member of each element of pImageInfo **must** not be VK_NULL_HANDLE
- VUID-VkWriteDescriptorSet-descriptorType-07683
  If descriptorType is VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT, the imageView member of each element of pImageInfo **must** not be VK_NULL_HANDLE
- VUID-VkWriteDescriptorSet-descriptorType-02382
  If descriptorType is VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR, the pNext chain **must** include a VkWriteDescriptorSetAccelerationStructureKHR structure whose accelerationStructureCount member equals descriptorCount
- VUID-VkWriteDescriptorSet-descriptorType-01946
  If descriptorType is VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, then the imageView member of each pImageInfo element **must** have been created without a VkSamplerYcbcrConversionInfo structure in its pNext chain
- VUID-VkWriteDescriptorSet-descriptorType-02738
  If descriptorType is VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, and if any element of pImageInfo has an imageView member that was created with a VkSamplerYcbcrConversionInfo structure in its pNext chain, then dstSet **must** have been allocated with a layout that included immutable samplers for dstBinding, and the corresponding immutable sampler **must** have been created with an *identically defined* VkSamplerYcbcrConversionInfo object
- VUID-VkWriteDescriptorSet-descriptorType-01948
  If descriptorType is VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, and dstSet was allocated with a layout that included immutable samplers for dstBinding, then the imageView member of each element of pImageInfo which corresponds to an immutable sampler that enables sampler $Y'C_BC_R$ conversion **must** have been created with a VkSamplerYcbcrConversionInfo structure in its pNext chain with an *identically defined* VkSamplerYcbcrConversionInfo to the corresponding immutable sampler
- VUID-VkWriteDescriptorSet-descriptorType-09506
  If descriptorType is VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, dstSet was allocated with a layout that included immutable samplers for dstBinding, and those samplers enable sampler $Y'C_BC_R$ conversion, then imageView **must** not be VK_NULL_HANDLE
- VUID-VkWriteDescriptorSet-descriptorType-00327
  If descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER or VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC, the offset member of each element of pBufferInfo **must** be a multiple of VkPhysicalDeviceLimits ::minUniformBufferOffsetAlignment
- VUID-VkWriteDescriptorSet-descriptorType-00328
  If descriptorType is VK_DESCRIPTOR_TYPE_STORAGE_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC, the offset member of each element of

pBufferInfo **must** be a multiple of VkPhysicalDeviceLimits ::minStorageBufferOffsetAlignment

- VUID-VkWriteDescriptorSet-descriptorType-00329
  If descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER, VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC, and the buffer member of any element of pBufferInfo is the handle of a non-sparse buffer, then that buffer **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkWriteDescriptorSet-descriptorType-00330
  If descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER or VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC, the buffer member of each element of pBufferInfo **must** have been created with VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT set
- VUID-VkWriteDescriptorSet-descriptorType-00331
  If descriptorType is VK_DESCRIPTOR_TYPE_STORAGE_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC, the buffer member of each element of pBufferInfo **must** have been created with VK_BUFFER_USAGE_STORAGE_BUFFER_BIT set
- VUID-VkWriteDescriptorSet-descriptorType-00332
  If descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER or VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC, the range member of each element of pBufferInfo, or the effective range if range is VK_WHOLE_SIZE, **must** be less than or equal to VkPhysicalDeviceLimits::maxUniformBufferRange
- VUID-VkWriteDescriptorSet-descriptorType-00333
  If descriptorType is VK_DESCRIPTOR_TYPE_STORAGE_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC, the range member of each element of pBufferInfo, or the effective range if range is VK_WHOLE_SIZE, **must** be less than or equal to VkPhysicalDeviceLimits::maxStorageBufferRange
- VUID-VkWriteDescriptorSet-descriptorType-08765
  If descriptorType is VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER, the pTexelBufferView buffer view usage **must** include VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT
- VUID-VkWriteDescriptorSet-descriptorType-08766
  If descriptorType is VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER, the pTexelBufferView buffer view usage **must** include VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT
- VUID-VkWriteDescriptorSet-descriptorType-00336
  If descriptorType is VK_DESCRIPTOR_TYPE_STORAGE_IMAGE or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT, the imageView member of each element of pImageInfo **must** have been created with the identity swizzle
- VUID-VkWriteDescriptorSet-descriptorType-00337
  If descriptorType is VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, the imageView member of each element of pImageInfo **must** have been created with VK_IMAGE_USAGE_SAMPLED_BIT set
- VUID-VkWriteDescriptorSet-descriptorType-04149
  If descriptorType is VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE the imageLayout member of each element of pImageInfo **must** be a member of the list given in Sampled Image

- VUID-VkWriteDescriptorSet-descriptorType-04150
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` the `imageLayout` member of each element of `pImageInfo` **must** be a member of the list given in Combined Image Sampler
- VUID-VkWriteDescriptorSet-descriptorType-04151
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` the `imageLayout` member of each element of `pImageInfo` **must** be a member of the list given in Input Attachment
- VUID-VkWriteDescriptorSet-descriptorType-04152
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE` the `imageLayout` member of each element of `pImageInfo` **must** be a member of the list given in Storage Image
- VUID-VkWriteDescriptorSet-descriptorType-00338
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT`, the `imageView` member of each element of `pImageInfo` **must** have been created with `VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT` set
- VUID-VkWriteDescriptorSet-descriptorType-00339
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, the `imageView` member of each element of `pImageInfo` **must** have been created with `VK_IMAGE_USAGE_STORAGE_BIT` set
- VUID-VkWriteDescriptorSet-descriptorType-02752
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_SAMPLER`, then `dstSet` **must** not have been allocated with a layout that included immutable samplers for `dstBinding`
- VUID-VkWriteDescriptorSet-descriptorType-06450
  If `descriptorType` is `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT`, the `imageView` member of each element of `pImageInfo` **must** have either been created without a VkImageViewMinLodCreateInfoEXT included in the `pNext` chain or with a VkImageViewMinLodCreateInfoEXT::`minLod` of `0.0`

Valid Usage (Implicit)

- VUID-VkWriteDescriptorSet-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET`
- VUID-VkWriteDescriptorSet-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkWriteDescriptorSetAccelerationStructureKHR
- VUID-VkWriteDescriptorSet-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkWriteDescriptorSet-descriptorType-parameter
  `descriptorType` **must** be a valid VkDescriptorType value
- VUID-VkWriteDescriptorSet-descriptorCount-arraylength
  `descriptorCount` **must** be greater than `0`
- VUID-VkWriteDescriptorSet-commonparent
  Both of `dstSet`, and the elements of `pTexelBufferView` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The type of descriptors in a descriptor set is specified by VkWriteDescriptorSet::`descriptorType`, which **must** be one of the values:

```
// Provided by VK_VERSION_1_0
typedef enum VkDescriptorType {
    VK_DESCRIPTOR_TYPE_SAMPLER = 0,
    VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER = 1,
    VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE = 2,
    VK_DESCRIPTOR_TYPE_STORAGE_IMAGE = 3,
    VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER = 4,
    VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER = 5,
    VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER = 6,
    VK_DESCRIPTOR_TYPE_STORAGE_BUFFER = 7,
    VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC = 8,
    VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC = 9,
    VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT = 10,
  // Provided by VK_KHR_acceleration_structure
    VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR = 1000150000,
} VkDescriptorType;
```

- `VK_DESCRIPTOR_TYPE_SAMPLER` specifies a sampler descriptor.
- `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` specifies a combined image sampler descriptor.
- `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE` specifies a sampled image descriptor.
- `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE` specifies a storage image descriptor.
- `VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER` specifies a uniform texel buffer descriptor.
- `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER` specifies a storage texel buffer descriptor.
- `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER` specifies a uniform buffer descriptor.
- `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER` specifies a storage buffer descriptor.
- `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` specifies a dynamic uniform buffer descriptor.
- `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` specifies a dynamic storage buffer descriptor.
- `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` specifies an input attachment descriptor.

When a descriptor set is updated via elements of VkWriteDescriptorSet, members of `pImageInfo`, `pBufferInfo` and `pTexelBufferView` are only accessed by the implementation when they correspond to descriptor type being defined - otherwise they are ignored. The members accessed are as follows for each descriptor type:

- For `VK_DESCRIPTOR_TYPE_SAMPLER`, only the `sampler` member of each element of VkWriteDescriptorSet::`pImageInfo` is accessed.
- For `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT`, only the `imageView` and `imageLayout` members of each element of VkWriteDescriptorSet::`pImageInfo` are accessed.
- For `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, all members of each element of VkWriteDescriptorSet::`pImageInfo` are accessed.

- For VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC, or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC, all members of each element of VkWriteDescriptorSet::pBufferInfo are accessed.
- For VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER, each element of VkWriteDescriptorSet::pTexelBufferView is accessed.

When updating descriptors with a descriptorType of VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR, none of the pImageInfo, pBufferInfo, or pTexelBufferView members are accessed, instead the source data of the descriptor update operation is taken from the VkWriteDescriptorSetAccelerationStructureKHR structure in the pNext chain of VkWriteDescriptorSet.

The VkDescriptorBufferInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDescriptorBufferInfo {
    VkBuffer        buffer;
    VkDeviceSize    offset;
    VkDeviceSize    range;
} VkDescriptorBufferInfo;
```

- buffer is VK_NULL_HANDLE or the buffer resource.
- offset is the offset in bytes from the start of buffer. Access to buffer memory via this descriptor uses addressing that is relative to this starting offset.
- range is the size in bytes that is used for this descriptor update, or VK_WHOLE_SIZE to use the range from offset to the end of the buffer.

NOTE

When setting range to VK_WHOLE_SIZE, the effective range **must** not be larger than the maximum range for the descriptor type (maxUniformBufferRange or maxStorageBufferRange). This means that VK_WHOLE_SIZE is not typically useful in the common case where uniform buffer descriptors are suballocated from a buffer that is much larger than maxUniformBufferRange.

For VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC and VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC descriptor types, offset is the base offset from which the dynamic offset is applied and range is the static size used for all dynamic offsets.

When range is VK_WHOLE_SIZE the effective range is calculated at vkUpdateDescriptorSets is by taking the size of buffer minus the offset.

Valid Usage

- VUID-VkDescriptorBufferInfo-offset-00340
  offset **must** be less than the size of buffer
- VUID-VkDescriptorBufferInfo-range-00341
  If range is not equal to VK_WHOLE_SIZE, range **must** be greater than 0

- VUID-VkDescriptorBufferInfo-range-00342
  If `range` is not equal to `VK_WHOLE_SIZE`, `range` **must** be less than or equal to the size of `buffer` minus `offset`
- VUID-VkDescriptorBufferInfo-buffer-02998
  If the `nullDescriptor` feature is not enabled, `buffer` **must** not be VK_NULL_HANDLE
- VUID-VkDescriptorBufferInfo-buffer-02999
  If `buffer` is VK_NULL_HANDLE, `offset` **must** be zero and `range` **must** be `VK_WHOLE_SIZE`

Valid Usage (Implicit)

- VUID-VkDescriptorBufferInfo-buffer-parameter
  If `buffer` is not VK_NULL_HANDLE, `buffer` **must** be a valid VkBuffer handle

The `VkDescriptorImageInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDescriptorImageInfo {
    VkSampler        sampler;
    VkImageView      imageView;
    VkImageLayout    imageLayout;
} VkDescriptorImageInfo;
```

- `sampler` is a sampler handle, and is used in descriptor updates for types `VK_DESCRIPTOR_TYPE_SAMPLER` and `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` if the binding being updated does not use immutable samplers.
- `imageView` is VK_NULL_HANDLE or an image view handle, and is used in descriptor updates for types `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, and `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT`.
- `imageLayout` is the layout that the image subresources accessible from `imageView` will be in at the time this descriptor is accessed. `imageLayout` is used in descriptor updates for types `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, and `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT`.

Members of `VkDescriptorImageInfo` that are not used in an update (as described above) are ignored.

Valid Usage

- VUID-VkDescriptorImageInfo-imageView-06712
  `imageView` **must** not be a 2D array image view created from a 3D image
- VUID-VkDescriptorImageInfo-imageView-07795
  If `imageView` is a 2D view created from a 3D image, then `descriptorType` **must** be `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, or `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`

- VUID-VkDescriptorImageInfo-imageView-07796
  If `imageView` is a 2D view created from a 3D image, then the image **must** have been created with `VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT` set
- VUID-VkDescriptorImageInfo-descriptorType-06713
  If the `image2DViewOf3D` feature is not enabled or `descriptorType` is not `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE` then `imageView` **must** not be a 2D view created from a 3D image
- VUID-VkDescriptorImageInfo-descriptorType-06714
  If the `sampler2DViewOf3D` feature is not enabled or `descriptorType` is not `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE` or `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` then `imageView` **must** not be a 2D view created from a 3D image
- VUID-VkDescriptorImageInfo-imageView-01976
  If `imageView` is created from a depth/stencil image, the `aspectMask` used to create the `imageView` **must** include either `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT` but not both
- VUID-VkDescriptorImageInfo-imageLayout-09425
  If `imageLayout` is `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`, then the `aspectMask` used to create `imageView` **must** not include either `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-VkDescriptorImageInfo-imageLayout-09426
  If `imageLayout` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL`, `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`, `VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL`, `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL` or `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, then the `aspectMask` used to create `imageView` **must** not include `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-VkDescriptorImageInfo-imageLayout-00344
  `imageLayout` **must** match the actual VkImageLayout of each subresource accessible from `imageView` at the time this descriptor is accessed as defined by the image layout matching rules
- VUID-VkDescriptorImageInfo-sampler-01564
  If `sampler` is used and the VkFormat of the image is a multi-planar format, the image **must** have been created with `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT`, and the `aspectMask` of the `imageView` **must** be a valid multi-planar aspect mask bit
- VUID-VkDescriptorImageInfo-mutableComparisonSamplers-04450
  If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`mutableComparisonSamplers` is `VK_FALSE`, then `sampler` **must** have been created with VkSamplerCreateInfo::`compareEnable` set to `VK_FALSE`

Valid Usage (Implicit)

- VUID-VkDescriptorImageInfo-commonparent
  Both of `imageView`, and `sampler` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The `VkWriteDescriptorSetAccelerationStructureKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkWriteDescriptorSetAccelerationStructureKHR {
    VkStructureType                      sType;
    const void*                          pNext;
    uint32_t                             accelerationStructureCount;
    const VkAccelerationStructureKHR*    pAccelerationStructures;
} VkWriteDescriptorSetAccelerationStructureKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `accelerationStructureCount` is the number of elements in `pAccelerationStructures`.
- `pAccelerationStructures` is a pointer to an array of VkAccelerationStructureKHR structures specifying the acceleration structures to update.

Valid Usage

- VUID-VkWriteDescriptorSetAccelerationStructureKHR-accelerationStructureCount-02236
  `accelerationStructureCount` **must** be equal to `descriptorCount` in the extended structure
- VUID-VkWriteDescriptorSetAccelerationStructureKHR-pAccelerationStructures-03579
  Each acceleration structure in `pAccelerationStructures` **must** have been created with a `type` of `VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR` or `VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR`
- VUID-VkWriteDescriptorSetAccelerationStructureKHR-pAccelerationStructures-03580
  If the `nullDescriptor` feature is not enabled, each element of `pAccelerationStructures` **must** not be VK_NULL_HANDLE

Valid Usage (Implicit)

- VUID-VkWriteDescriptorSetAccelerationStructureKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET_ACCELERATION_STRUCTURE_KHR`
- VUID-VkWriteDescriptorSetAccelerationStructureKHR-pAccelerationStructures-parameter
  `pAccelerationStructures` **must** be a valid pointer to an array of `accelerationStructureCount` valid or VK_NULL_HANDLE VkAccelerationStructureKHR handles

- VUID-VkWriteDescriptorSetAccelerationStructureKHR-accelerationStructureCount-arraylength
  `accelerationStructureCount` **must** be greater than `0`

The `VkCopyDescriptorSet` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkCopyDescriptorSet {
    VkStructureType    sType;
    const void*        pNext;
    VkDescriptorSet    srcSet;
    uint32_t           srcBinding;
    uint32_t           srcArrayElement;
    VkDescriptorSet    dstSet;
    uint32_t           dstBinding;
    uint32_t           dstArrayElement;
    uint32_t           descriptorCount;
} VkCopyDescriptorSet;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcSet`, `srcBinding`, and `srcArrayElement` are the source set, binding, and array element, respectively.
- `dstSet`, `dstBinding`, and `dstArrayElement` are the destination set, binding, and array element, respectively.
- `descriptorCount` is the number of descriptors to copy from the source to destination. If `descriptorCount` is greater than the number of remaining array elements in the source or destination binding, those affect consecutive bindings in a manner similar to VkWriteDescriptorSet above.

Valid Usage

- VUID-VkCopyDescriptorSet-srcBinding-00345
  `srcBinding` **must** be a valid binding within `srcSet`
- VUID-VkCopyDescriptorSet-srcArrayElement-00346
  The sum of `srcArrayElement` and `descriptorCount` **must** be less than or equal to the number of array elements in the descriptor set binding specified by `srcBinding`, and all applicable consecutive bindings
- VUID-VkCopyDescriptorSet-dstBinding-00347
  `dstBinding` **must** be a valid binding within `dstSet`
- VUID-VkCopyDescriptorSet-dstArrayElement-00348
  The sum of `dstArrayElement` and `descriptorCount` **must** be less than or equal to the number of array elements in the descriptor set binding specified by `dstBinding`, and all applicable consecutive bindings

- VUID-VkCopyDescriptorSet-dstBinding-02632
  The type of `dstBinding` within `dstSet` **must** be equal to the type of `srcBinding` within `srcSet`
- VUID-VkCopyDescriptorSet-srcSet-00349
  If `srcSet` is equal to `dstSet`, then the source and destination ranges of descriptors **must** not overlap, where the ranges **may** include array elements from consecutive bindings
- VUID-VkCopyDescriptorSet-dstBinding-02753
  If the descriptor type of the descriptor set binding specified by `dstBinding` is `VK_DESCRIPTOR_TYPE_SAMPLER`, then `dstSet` **must** not have been allocated with a layout that included immutable samplers for `dstBinding`

Valid Usage (Implicit)

- VUID-VkCopyDescriptorSet-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_DESCRIPTOR_SET`
- VUID-VkCopyDescriptorSet-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyDescriptorSet-srcSet-parameter
  `srcSet` **must** be a valid VkDescriptorSet handle
- VUID-VkCopyDescriptorSet-dstSet-parameter
  `dstSet` **must** be a valid VkDescriptorSet handle
- VUID-VkCopyDescriptorSet-commonparent
  Both of `dstSet`, and `srcSet` **must** have been created, allocated, or retrieved from the same VkDevice

14.2.5. Descriptor Update Templates

A descriptor update template specifies a mapping from descriptor update information in host memory to descriptors in a descriptor set. It is designed to avoid passing redundant information to the driver when frequently updating the same set of descriptors in descriptor sets.

Descriptor update template objects are represented by `VkDescriptorUpdateTemplate` handles:

```
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDescriptorUpdateTemplate)
```

or the equivalent

```
// Provided by VK_KHR_descriptor_update_template
typedef VkDescriptorUpdateTemplate VkDescriptorUpdateTemplateKHR;
```

14.2.6. Descriptor Set Updates With Templates

Updating a large `VkDescriptorSet` array **can** be an expensive operation since an application **must**

specify one VkWriteDescriptorSet structure for each descriptor or descriptor array to update, each of which re-specifies the same state when updating the same descriptor in multiple descriptor sets. For cases when an application wishes to update the same set of descriptors in multiple descriptor sets allocated using the same VkDescriptorSetLayout, vkUpdateDescriptorSetWithTemplateKHR **can** be used as a replacement for vkUpdateDescriptorSets.

VkDescriptorUpdateTemplate allows implementations to convert a set of descriptor update operations on a single descriptor set to an internal format that, in conjunction with vkUpdateDescriptorSetWithTemplateKHR or vkCmdPushDescriptorSetWithTemplateKHR , **can** be more efficient compared to calling vkUpdateDescriptorSets or vkCmdPushDescriptorSetKHR . The descriptors themselves are not specified in the VkDescriptorUpdateTemplate, rather, offsets into an application provided pointer to host memory are specified, which are combined with a pointer passed to vkUpdateDescriptorSetWithTemplateKHR or vkCmdPushDescriptorSetWithTemplateKHR . This allows large batches of updates to be executed without having to convert application data structures into a strictly-defined Vulkan data structure.

To create a descriptor update template, call:

```
// Provided by VK_KHR_descriptor_update_template
VkResult vkCreateDescriptorUpdateTemplateKHR(
    VkDevice                                    device,
    const VkDescriptorUpdateTemplateCreateInfo* pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkDescriptorUpdateTemplate*                 pDescriptorUpdateTemplate);
```

- device is the logical device that creates the descriptor update template.
- pCreateInfo is a pointer to a VkDescriptorUpdateTemplateCreateInfo structure specifying the set of descriptors to update with a single call to vkCmdPushDescriptorSetWithTemplateKHR or vkUpdateDescriptorSetWithTemplateKHR.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.
- pDescriptorUpdateTemplate is a pointer to a VkDescriptorUpdateTemplate handle in which the resulting descriptor update template object is returned.

Valid Usage (Implicit)

- VUID-vkCreateDescriptorUpdateTemplate-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateDescriptorUpdateTemplate-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkDescriptorUpdateTemplateCreateInfo structure
- VUID-vkCreateDescriptorUpdateTemplate-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateDescriptorUpdateTemplate-pDescriptorUpdateTemplate-parameter
  pDescriptorUpdateTemplate **must** be a valid pointer to a VkDescriptorUpdateTemplate

handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkDescriptorUpdateTemplateCreateInfo structure is defined as:

```
typedef struct VkDescriptorUpdateTemplateCreateInfo {
    VkStructureType                           sType;
    const void*                               pNext;
    VkDescriptorUpdateTemplateCreateFlags     flags;
    uint32_t                                  descriptorUpdateEntryCount;
    const VkDescriptorUpdateTemplateEntry*    pDescriptorUpdateEntries;
    VkDescriptorUpdateTemplateType            templateType;
    VkDescriptorSetLayout                     descriptorSetLayout;
    VkPipelineBindPoint                       pipelineBindPoint;
    VkPipelineLayout                          pipelineLayout;
    uint32_t                                  set;
} VkDescriptorUpdateTemplateCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_descriptor_update_template
typedef VkDescriptorUpdateTemplateCreateInfo VkDescriptorUpdateTemplateCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- descriptorUpdateEntryCount is the number of elements in the pDescriptorUpdateEntries array.
- pDescriptorUpdateEntries is a pointer to an array of VkDescriptorUpdateTemplateEntry structures describing the descriptors to be updated by the descriptor update template.
- templateType Specifies the type of the descriptor update template. If set to VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET it **can** only be used to update descriptor sets with a fixed descriptorSetLayout. If set to VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR it **can** only be used to push descriptor sets using the provided pipelineBindPoint, pipelineLayout, and set number.
- descriptorSetLayout is the descriptor set layout used to build the descriptor update template. All

descriptor sets which are going to be updated through the newly created descriptor update template **must** be created with a layout that matches (is the same as, or defined identically to) this layout. This parameter is ignored if `templateType` is not `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET`.

- `pipelineBindPoint` is a VkPipelineBindPoint indicating the type of the pipeline that will use the descriptors. This parameter is ignored if `templateType` is not `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`
- `pipelineLayout` is a VkPipelineLayout object used to program the bindings. This parameter is ignored if `templateType` is not `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`
- `set` is the set number of the descriptor set in the pipeline layout that will be updated. This parameter is ignored if `templateType` is not `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`

Valid Usage

- VUID-VkDescriptorUpdateTemplateCreateInfo-templateType-00350
  If `templateType` is `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET`, `descriptorSetLayout` **must** be a valid `VkDescriptorSetLayout` handle
- VUID-VkDescriptorUpdateTemplateCreateInfo-templateType-00351
  If `templateType` is `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`, `pipelineBindPoint` **must** be a valid VkPipelineBindPoint value
- VUID-VkDescriptorUpdateTemplateCreateInfo-templateType-00352
  If `templateType` is `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`, `pipelineLayout` **must** be a valid `VkPipelineLayout` handle
- VUID-VkDescriptorUpdateTemplateCreateInfo-templateType-00353
  If `templateType` is `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`, `set` **must** be the unique set number in the pipeline layout that uses a descriptor set layout that was created with `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR`

Valid Usage (Implicit)

- VUID-VkDescriptorUpdateTemplateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_CREATE_INFO`
- VUID-VkDescriptorUpdateTemplateCreateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDescriptorUpdateTemplateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkDescriptorUpdateTemplateCreateInfo-pDescriptorUpdateEntries-parameter
  `pDescriptorUpdateEntries` **must** be a valid pointer to an array of `descriptorUpdateEntryCount` valid VkDescriptorUpdateTemplateEntry structures
- VUID-VkDescriptorUpdateTemplateCreateInfo-templateType-parameter
  `templateType` **must** be a valid VkDescriptorUpdateTemplateType value

- VUID-VkDescriptorUpdateTemplateCreateInfo-descriptorUpdateEntryCount-arraylength
  `descriptorUpdateEntryCount` **must** be greater than `0`
- VUID-VkDescriptorUpdateTemplateCreateInfo-commonparent
  Both of `descriptorSetLayout`, and `pipelineLayout` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

```
typedef VkFlags VkDescriptorUpdateTemplateCreateFlags;
```

or the equivalent

```
// Provided by VK_KHR_descriptor_update_template
typedef VkDescriptorUpdateTemplateCreateFlags
VkDescriptorUpdateTemplateCreateFlagsKHR;
```

`VkDescriptorUpdateTemplateCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

The descriptor update template type is determined by the VkDescriptorUpdateTemplateCreateInfo ::`templateType` property, which takes the following values:

```
typedef enum VkDescriptorUpdateTemplateType {
    VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET = 0,
  // Provided by VK_KHR_descriptor_update_template with VK_KHR_push_descriptor,
VK_KHR_push_descriptor with VK_VERSION_1_1 or VK_KHR_descriptor_update_template
    VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR = 1,
  // Provided by VK_KHR_descriptor_update_template
    VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET_KHR =
VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET,
} VkDescriptorUpdateTemplateType;
```

or the equivalent

```
// Provided by VK_KHR_descriptor_update_template
typedef VkDescriptorUpdateTemplateType VkDescriptorUpdateTemplateTypeKHR;
```

- `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET` specifies that the descriptor update template will be used for descriptor set updates only.
- `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR` specifies that the descriptor update template will be used for push descriptor updates only.

The `VkDescriptorUpdateTemplateEntry` structure is defined as:

```
typedef struct VkDescriptorUpdateTemplateEntry {
    uint32_t            dstBinding;
    uint32_t            dstArrayElement;
    uint32_t            descriptorCount;
    VkDescriptorType    descriptorType;
    size_t              offset;
    size_t              stride;
} VkDescriptorUpdateTemplateEntry;
```

or the equivalent

```
// Provided by VK_KHR_descriptor_update_template
typedef VkDescriptorUpdateTemplateEntry VkDescriptorUpdateTemplateEntryKHR;
```

- `dstBinding` is the descriptor binding to update when using this descriptor update template.
- `dstArrayElement` is the starting element in the array belonging to `dstBinding`.
- `descriptorCount` is the number of descriptors to update. If `descriptorCount` is greater than the number of remaining array elements in the destination binding, those affect consecutive bindings in a manner similar to VkWriteDescriptorSet above.
- `descriptorType` is a VkDescriptorType specifying the type of the descriptor.
- `offset` is the offset in bytes of the first binding in the raw data structure.
- `stride` is the stride in bytes between two consecutive array elements of the descriptor update information in the raw data structure. The actual pointer ptr for each array element j of update entry i is computed using the following formula:

  ```
      const char *ptr = (const char *)pData + pDescriptorUpdateEntries[i].offset + j
  * pDescriptorUpdateEntries[i].stride
  ```

  The stride is useful in case the bindings are stored in structs along with other data.

Valid Usage

- VUID-VkDescriptorUpdateTemplateEntry-dstBinding-00354
  `dstBinding` **must** be a valid binding in the descriptor set layout implicitly specified when using a descriptor update template to update descriptors
- VUID-VkDescriptorUpdateTemplateEntry-dstArrayElement-00355
  `dstArrayElement` and `descriptorCount` **must** be less than or equal to the number of array elements in the descriptor set binding implicitly specified when using a descriptor update template to update descriptors, and all applicable consecutive bindings

Valid Usage (Implicit)

- VUID-VkDescriptorUpdateTemplateEntry-descriptorType-parameter
  `descriptorType` **must** be a valid VkDescriptorType value

To destroy a descriptor update template, call:

```
// Provided by VK_KHR_descriptor_update_template
void vkDestroyDescriptorUpdateTemplateKHR(
    VkDevice                                    device,
    VkDescriptorUpdateTemplate                  descriptorUpdateTemplate,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that has been used to create the descriptor update template
- `descriptorUpdateTemplate` is the descriptor update template to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyDescriptorUpdateTemplate-descriptorSetLayout-00356
  If `VkAllocationCallbacks` were provided when `descriptorUpdateTemplate` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyDescriptorUpdateTemplate-descriptorSetLayout-00357
  If no `VkAllocationCallbacks` were provided when `descriptorUpdateTemplate` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyDescriptorUpdateTemplate-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyDescriptorUpdateTemplate-descriptorUpdateTemplate-parameter
  If `descriptorUpdateTemplate` is not VK_NULL_HANDLE, `descriptorUpdateTemplate` **must** be a valid VkDescriptorUpdateTemplate handle
- VUID-vkDestroyDescriptorUpdateTemplate-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyDescriptorUpdateTemplate-descriptorUpdateTemplate-parent
  If `descriptorUpdateTemplate` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `descriptorUpdateTemplate` **must** be externally synchronized

Once a `VkDescriptorUpdateTemplate` has been created, descriptor sets **can** be updated by calling:

```
// Provided by VK_KHR_descriptor_update_template
void vkUpdateDescriptorSetWithTemplateKHR(
    VkDevice                                    device,
    VkDescriptorSet                             descriptorSet,
    VkDescriptorUpdateTemplate                  descriptorUpdateTemplate,
    const void*                                 pData);
```

- `device` is the logical device that updates the descriptor set.
- `descriptorSet` is the descriptor set to update
- `descriptorUpdateTemplate` is a VkDescriptorUpdateTemplate object specifying the update mapping between `pData` and the descriptor set to update.
- `pData` is a pointer to memory containing one or more VkDescriptorImageInfo, VkDescriptorBufferInfo, or VkBufferView structures or VkAccelerationStructureKHR handles used to write the descriptors.

Valid Usage

- VUID-vkUpdateDescriptorSetWithTemplate-pData-01685
  `pData` **must** be a valid pointer to a memory containing one or more valid instances of VkDescriptorImageInfo, VkDescriptorBufferInfo, or VkBufferView in a layout defined by `descriptorUpdateTemplate` when it was created with vkCreateDescriptorUpdateTemplateKHR
- VUID-vkUpdateDescriptorSetWithTemplate-descriptorSet-06995
  Host access to `descriptorSet` **must** be externally synchronized

Valid Usage (Implicit)

- VUID-vkUpdateDescriptorSetWithTemplate-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkUpdateDescriptorSetWithTemplate-descriptorSet-parameter
  `descriptorSet` **must** be a valid VkDescriptorSet handle
- VUID-vkUpdateDescriptorSetWithTemplate-descriptorUpdateTemplate-parameter
  `descriptorUpdateTemplate` **must** be a valid VkDescriptorUpdateTemplate handle
- VUID-vkUpdateDescriptorSetWithTemplate-descriptorSet-parent
  `descriptorSet` **must** have been created, allocated, or retrieved from `device`
- VUID-vkUpdateDescriptorSetWithTemplate-descriptorUpdateTemplate-parent

`descriptorUpdateTemplate` **must** have been created, allocated, or retrieved from `device`

API Example

```
struct AppBufferView {
    VkBufferView bufferView;
    uint32_t     applicationRelatedInformation;
};

struct AppDataStructure
{
    VkDescriptorImageInfo  imageInfo;          // a single image info
    VkDescriptorBufferInfo bufferInfoArray[3]; // 3 buffer infos in an array
    AppBufferView          bufferView[2];      // An application-defined structure
containing a bufferView
    // ... some more application-related data
};

const VkDescriptorUpdateTemplateEntry descriptorUpdateTemplateEntries[] =
{
    // binding to a single image descriptor
    {
        .binding = 0,
        .dstArrayElement = 0,
        .descriptorCount = 1,
        .descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER,
        .offset = offsetof(AppDataStructure, imageInfo),
        .stride = 0         // stride not required if descriptorCount is 1
    },

    // binding to an array of buffer descriptors
    {
        .binding = 1,
        .dstArrayElement = 0,
        .descriptorCount = 3,
        .descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER,
        .offset = offsetof(AppDataStructure, bufferInfoArray),
        .stride = sizeof(VkDescriptorBufferInfo)    // descriptor buffer infos are
compact
    },

    // binding to an array of buffer views
    {
        .binding = 2,
        .dstArrayElement = 0,
        .descriptorCount = 2,
        .descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER,
        .offset = offsetof(AppDataStructure, bufferView) +
                  offsetof(AppBufferView, bufferView),
        .stride = sizeof(AppBufferView)             // bufferViews do not have to be
```

```
compact
    },
};

// create a descriptor update template for descriptor set updates
const VkDescriptorUpdateTemplateCreateInfo createInfo =
{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_CREATE_INFO,
    .pNext = NULL,
    .flags = 0,
    .descriptorUpdateEntryCount = 3,
    .pDescriptorUpdateEntries = descriptorUpdateTemplateEntries,
    .templateType = VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET,
    .descriptorSetLayout = myLayout,
    .pipelineBindPoint = 0,     // ignored by given templateType
    .pipelineLayout = 0,        // ignored by given templateType
    .set = 0,                   // ignored by given templateType
};

VkDescriptorUpdateTemplate myDescriptorUpdateTemplate;
myResult = vkCreateDescriptorUpdateTemplate(
    myDevice,
    &createInfo,
    NULL,
    &myDescriptorUpdateTemplate);

AppDataStructure appData;

// fill appData here or cache it in your engine
vkUpdateDescriptorSetWithTemplate(myDevice, myDescriptorSet,
myDescriptorUpdateTemplate, &appData);
```

14.2.7. Descriptor Set Binding

To bind one or more descriptor sets to a command buffer, call:

```
// Provided by VK_VERSION_1_0
void vkCmdBindDescriptorSets(
    VkCommandBuffer                             commandBuffer,
    VkPipelineBindPoint                         pipelineBindPoint,
    VkPipelineLayout                            layout,
    uint32_t                                    firstSet,
    uint32_t                                    descriptorSetCount,
    const VkDescriptorSet*                      pDescriptorSets,
    uint32_t                                    dynamicOffsetCount,
    const uint32_t*                             pDynamicOffsets);
```

- `commandBuffer` is the command buffer that the descriptor sets will be bound to.

- `pipelineBindPoint` is a VkPipelineBindPoint indicating the type of the pipeline that will use the descriptors. There is a separate set of bind points for each pipeline type, so binding one does not disturb the others.
- `layout` is a VkPipelineLayout object used to program the bindings.
- `firstSet` is the set number of the first descriptor set to be bound.
- `descriptorSetCount` is the number of elements in the `pDescriptorSets` array.
- `pDescriptorSets` is a pointer to an array of handles to VkDescriptorSet objects describing the descriptor sets to bind to.
- `dynamicOffsetCount` is the number of dynamic offsets in the `pDynamicOffsets` array.
- `pDynamicOffsets` is a pointer to an array of `uint32_t` values specifying dynamic offsets.

`vkCmdBindDescriptorSets` binds descriptor sets `pDescriptorSets`[0..`descriptorSetCount`-1] to set numbers [`firstSet`..`firstSet`+`descriptorSetCount`-1] for subsequent bound pipeline commands set by `pipelineBindPoint`. Any bindings that were previously applied via these sets are no longer valid.

Once bound, a descriptor set affects rendering of subsequent commands that interact with the given pipeline type in the command buffer until either a different set is bound to the same set number, or the set is disturbed as described in Pipeline Layout Compatibility.

A compatible descriptor set **must** be bound for all set numbers that any shaders in a pipeline access, at the time that a drawing or dispatching command is recorded to execute using that pipeline. However, if none of the shaders in a pipeline statically use any bindings with a particular set number, then no descriptor set need be bound for that set number, even if the pipeline layout includes a non-trivial descriptor set layout for that set number.

When consuming a descriptor, a descriptor is considered valid if the descriptor is not undefined as described by descriptor set allocation. If the `nullDescriptor` feature is enabled, a null descriptor is also considered valid. A descriptor that was disturbed by Pipeline Layout Compatibility, or was never bound by `vkCmdBindDescriptorSets` is not considered valid. If a pipeline accesses a descriptor either statically or dynamically depending on the `VkDescriptorBindingFlagBits`, the consuming descriptor type in the pipeline **must** match the VkDescriptorType in VkDescriptorSetLayoutCreateInfo for the descriptor to be considered valid.

NOTE	Further validation may be carried out beyond validation for descriptor types, e.g. Texel Input Validation.

If any of the sets being bound include dynamic uniform or storage buffers, then `pDynamicOffsets` includes one element for each array element in each dynamic descriptor type binding in each set. Values are taken from `pDynamicOffsets` in an order such that all entries for set N come before set N+1; within a set, entries are ordered by the binding numbers in the descriptor set layouts; and within a binding array, elements are in order. `dynamicOffsetCount` **must** equal the total number of dynamic descriptors in the sets being bound.

The effective offset used for dynamic uniform and storage buffer bindings is the sum of the relative offset taken from `pDynamicOffsets`, and the base address of the buffer plus base offset in the descriptor set. The range of the dynamic uniform and storage buffer bindings is the buffer range as

specified in the descriptor set.

Each of the pDescriptorSets **must** be compatible with the pipeline layout specified by layout. The layout used to program the bindings **must** also be compatible with the pipeline used in subsequent bound pipeline commands with that pipeline type, as defined in the Pipeline Layout Compatibility section.

The descriptor set contents bound by a call to vkCmdBindDescriptorSets **may** be consumed at the following times:

- during host execution of the command, or during shader execution of the resulting draws and dispatches, or any time in between.

Thus, the contents of a descriptor set binding **must** not be altered (overwritten by an update command, or freed) between the first point in time that it **may** be consumed, and when the command completes executing on the queue.

The contents of pDynamicOffsets are consumed immediately during execution of vkCmdBindDescriptorSets. Once all pending uses have completed, it is legal to update and reuse a descriptor set.

Valid Usage

- VUID-vkCmdBindDescriptorSets-pDescriptorSets-00358
  Each element of pDescriptorSets **must** have been allocated with a VkDescriptorSetLayout that matches (is the same as, or identically defined as) the VkDescriptorSetLayout at set *n* in layout, where *n* is the sum of firstSet and the index into pDescriptorSets
- VUID-vkCmdBindDescriptorSets-dynamicOffsetCount-00359
  dynamicOffsetCount **must** be equal to the total number of dynamic descriptors in pDescriptorSets
- VUID-vkCmdBindDescriptorSets-firstSet-00360
  The sum of firstSet and descriptorSetCount **must** be less than or equal to VkPipelineLayoutCreateInfo::setLayoutCount provided when layout was created
- VUID-vkCmdBindDescriptorSets-pDynamicOffsets-01971
  Each element of pDynamicOffsets which corresponds to a descriptor binding with type VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC **must** be a multiple of VkPhysicalDeviceLimits::minUniformBufferOffsetAlignment
- VUID-vkCmdBindDescriptorSets-pDynamicOffsets-01972
  Each element of pDynamicOffsets which corresponds to a descriptor binding with type VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC **must** be a multiple of VkPhysicalDeviceLimits::minStorageBufferOffsetAlignment
- VUID-vkCmdBindDescriptorSets-pDescriptorSets-01979
  For each dynamic uniform or storage buffer binding in pDescriptorSets, the sum of the effective offset and the range of the binding **must** be less than or equal to the size of the buffer
- VUID-vkCmdBindDescriptorSets-pDescriptorSets-06715

For each dynamic uniform or storage buffer binding in pDescriptorSets, if the range was set with VK_WHOLE_SIZE then pDynamicOffsets which corresponds to the descriptor binding **must** be 0

- VUID-vkCmdBindDescriptorSets-pDescriptorSets-06563
  Each element of pDescriptorSets **must** be a valid VkDescriptorSet
- VUID-vkCmdBindDescriptorSets-pipelineBindPoint-00361
  pipelineBindPoint **must** be supported by the commandBuffer's parent VkCommandPool's queue family

Valid Usage (Implicit)

- VUID-vkCmdBindDescriptorSets-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindDescriptorSets-pipelineBindPoint-parameter
  pipelineBindPoint **must** be a valid VkPipelineBindPoint value
- VUID-vkCmdBindDescriptorSets-layout-parameter
  layout **must** be a valid VkPipelineLayout handle
- VUID-vkCmdBindDescriptorSets-pDescriptorSets-parameter
  pDescriptorSets **must** be a valid pointer to an array of descriptorSetCount valid or VK_NULL_HANDLE VkDescriptorSet handles
- VUID-vkCmdBindDescriptorSets-pDynamicOffsets-parameter
  If dynamicOffsetCount is not 0, pDynamicOffsets **must** be a valid pointer to an array of dynamicOffsetCount uint32_t values
- VUID-vkCmdBindDescriptorSets-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdBindDescriptorSets-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics, or compute operations
- VUID-vkCmdBindDescriptorSets-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBindDescriptorSets-descriptorSetCount-arraylength
  descriptorSetCount **must** be greater than 0
- VUID-vkCmdBindDescriptorSets-commonparent
  Each of commandBuffer, layout, and the elements of pDescriptorSets that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally

synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

Alternatively, to bind one or more descriptor sets to a command buffer, call:

```
// Provided by VK_KHR_maintenance6
void vkCmdBindDescriptorSets2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkBindDescriptorSetsInfoKHR*          pBindDescriptorSetsInfo);
```

- `commandBuffer` is the command buffer that the descriptor sets will be bound to.
- `pBindDescriptorSetsInfo` is a pointer to a `VkBindDescriptorSetsInfoKHR` structure.

Valid Usage

- VUID-vkCmdBindDescriptorSets2KHR-pBindDescriptorSetsInfo-09467
  Each bit in `pBindDescriptorSetsInfo->stageFlags` **must** be a stage supported by the `commandBuffer`'s parent `VkCommandPool`'s queue family

Valid Usage (Implicit)

- VUID-vkCmdBindDescriptorSets2KHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindDescriptorSets2KHR-pBindDescriptorSetsInfo-parameter
  `pBindDescriptorSetsInfo` **must** be a valid pointer to a valid VkBindDescriptorSetsInfoKHR structure
- VUID-vkCmdBindDescriptorSets2KHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBindDescriptorSets2KHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdBindDescriptorSets2KHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

The `VkBindDescriptorSetsInfoKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance6
typedef struct VkBindDescriptorSetsInfoKHR {
    VkStructureType           sType;
    const void*               pNext;
    VkShaderStageFlags        stageFlags;
    VkPipelineLayout          layout;
    uint32_t                  firstSet;
    uint32_t                  descriptorSetCount;
    const VkDescriptorSet*    pDescriptorSets;
    uint32_t                  dynamicOffsetCount;
    const uint32_t*           pDynamicOffsets;
} VkBindDescriptorSetsInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `stageFlags` is a bitmask of VkShaderStageFlagBits specifying the shader stages the descriptor sets will be bound to.
- `layout` is a VkPipelineLayout object used to program the bindings.
- `firstSet` is the set number of the first descriptor set to be bound.
- `descriptorSetCount` is the number of elements in the `pDescriptorSets` array.
- `pDescriptorSets` is a pointer to an array of handles to VkDescriptorSet objects describing the descriptor sets to bind to.
- `dynamicOffsetCount` is the number of dynamic offsets in the `pDynamicOffsets` array.
- `pDynamicOffsets` is a pointer to an array of `uint32_t` values specifying dynamic offsets.

If `stageFlags` specifies a subset of all stages corresponding to one or more pipeline bind points, the

binding operation still affects all stages corresponding to the given pipeline bind point(s) as if the equivalent original version of this command had been called with the same parameters. For example, specifying a `stageFlags` value of `VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT | VK_SHADER_STAGE_COMPUTE_BIT` is equivalent to calling the original version of this command once with `VK_PIPELINE_BIND_POINT_GRAPHICS` and once with `VK_PIPELINE_BIND_POINT_COMPUTE`.

Valid Usage

- VUID-VkBindDescriptorSetsInfoKHR-pDescriptorSets-00358
  Each element of `pDescriptorSets` **must** have been allocated with a `VkDescriptorSetLayout` that matches (is the same as, or identically defined as) the `VkDescriptorSetLayout` at set *n* in `layout`, where *n* is the sum of `firstSet` and the index into `pDescriptorSets`
- VUID-VkBindDescriptorSetsInfoKHR-dynamicOffsetCount-00359
  `dynamicOffsetCount` **must** be equal to the total number of dynamic descriptors in `pDescriptorSets`
- VUID-VkBindDescriptorSetsInfoKHR-firstSet-00360
  The sum of `firstSet` and `descriptorSetCount` **must** be less than or equal to VkPipelineLayoutCreateInfo::`setLayoutCount` provided when `layout` was created
- VUID-VkBindDescriptorSetsInfoKHR-pDynamicOffsets-01971
  Each element of `pDynamicOffsets` which corresponds to a descriptor binding with type `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` **must** be a multiple of `VkPhysicalDeviceLimits`::`minUniformBufferOffsetAlignment`
- VUID-VkBindDescriptorSetsInfoKHR-pDynamicOffsets-01972
  Each element of `pDynamicOffsets` which corresponds to a descriptor binding with type `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` **must** be a multiple of `VkPhysicalDeviceLimits`::`minStorageBufferOffsetAlignment`
- VUID-VkBindDescriptorSetsInfoKHR-pDescriptorSets-01979
  For each dynamic uniform or storage buffer binding in `pDescriptorSets`, the sum of the effective offset and the range of the binding **must** be less than or equal to the size of the buffer
- VUID-VkBindDescriptorSetsInfoKHR-pDescriptorSets-06715
  For each dynamic uniform or storage buffer binding in `pDescriptorSets`, if the range was set with `VK_WHOLE_SIZE` then `pDynamicOffsets` which corresponds to the descriptor binding **must** be 0
- VUID-VkBindDescriptorSetsInfoKHR-pDescriptorSets-06563
  Each element of `pDescriptorSets` **must** be a valid VkDescriptorSet
- VUID-VkBindDescriptorSetsInfoKHR-None-09495
  `layout` **must** be a valid VkPipelineLayout handle

Valid Usage (Implicit)

- VUID-VkBindDescriptorSetsInfoKHR-sType-sType

`sType` **must** be `VK_STRUCTURE_TYPE_BIND_DESCRIPTOR_SETS_INFO_KHR`

- VUID-VkBindDescriptorSetsInfoKHR-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineLayoutCreateInfo
- VUID-VkBindDescriptorSetsInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkBindDescriptorSetsInfoKHR-stageFlags-parameter
  `stageFlags` **must** be a valid combination of VkShaderStageFlagBits values
- VUID-VkBindDescriptorSetsInfoKHR-stageFlags-requiredbitmask
  `stageFlags` **must** not be `0`
- VUID-VkBindDescriptorSetsInfoKHR-layout-parameter
  If `layout` is not VK_NULL_HANDLE, `layout` **must** be a valid VkPipelineLayout handle
- VUID-VkBindDescriptorSetsInfoKHR-pDescriptorSets-parameter
  `pDescriptorSets` **must** be a valid pointer to an array of `descriptorSetCount` valid VkDescriptorSet handles
- VUID-VkBindDescriptorSetsInfoKHR-pDynamicOffsets-parameter
  If `dynamicOffsetCount` is not `0`, and `pDynamicOffsets` is not `NULL`, `pDynamicOffsets` **must** be a valid pointer to an array of `dynamicOffsetCount` or VK_NULL_HANDLE `uint32_t` values
- VUID-VkBindDescriptorSetsInfoKHR-descriptorSetCount-arraylength
  `descriptorSetCount` **must** be greater than `0`
- VUID-VkBindDescriptorSetsInfoKHR-commonparent
  Both of `layout`, and the elements of `pDescriptorSets` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

14.2.8. Push Descriptor Updates

In addition to allocating descriptor sets and binding them to a command buffer, an application **can** record descriptor updates into the command buffer.

To push descriptor updates into a command buffer, call:

```
// Provided by VK_KHR_push_descriptor
void vkCmdPushDescriptorSetKHR(
    VkCommandBuffer                             commandBuffer,
    VkPipelineBindPoint                         pipelineBindPoint,
    VkPipelineLayout                            layout,
    uint32_t                                    set,
    uint32_t                                    descriptorWriteCount,
    const VkWriteDescriptorSet*                 pDescriptorWrites);
```

- `commandBuffer` is the command buffer that the descriptors will be recorded in.
- `pipelineBindPoint` is a VkPipelineBindPoint indicating the type of the pipeline that will use the descriptors. There is a separate set of push descriptor bindings for each pipeline type, so binding one does not disturb the others.

- `layout` is a VkPipelineLayout object used to program the bindings.
- `set` is the set number of the descriptor set in the pipeline layout that will be updated.
- `descriptorWriteCount` is the number of elements in the `pDescriptorWrites` array.
- `pDescriptorWrites` is a pointer to an array of VkWriteDescriptorSet structures describing the descriptors to be updated.

Push descriptors are a small bank of descriptors whose storage is internally managed by the command buffer rather than being written into a descriptor set and later bound to a command buffer. Push descriptors allow for incremental updates of descriptors without managing the lifetime of descriptor sets.

When a command buffer begins recording, all push descriptors are undefined. Push descriptors **can** be updated incrementally and cause shaders to use the updated descriptors for subsequent bound pipeline commands with the pipeline type set by `pipelineBindPoint` until the descriptor is overwritten, or else until the set is disturbed as described in Pipeline Layout Compatibility. When the set is disturbed or push descriptors with a different descriptor set layout are set, all push descriptors are undefined.

Push descriptors that are statically used by a pipeline **must** not be undefined at the time that a drawing or dispatching command is recorded to execute using that pipeline. This includes immutable sampler descriptors, which **must** be pushed before they are accessed by a pipeline (the immutable samplers are pushed, rather than the samplers in `pDescriptorWrites`). Push descriptors that are not statically used **can** remain undefined.

Push descriptors do not use dynamic offsets. Instead, the corresponding non-dynamic descriptor types **can** be used and the `offset` member of VkDescriptorBufferInfo **can** be changed each time the descriptor is written.

Each element of `pDescriptorWrites` is interpreted as in VkWriteDescriptorSet, except the `dstSet` member is ignored.

To push an immutable sampler, use a VkWriteDescriptorSet with `dstBinding` and `dstArrayElement` selecting the immutable sampler's binding. If the descriptor type is `VK_DESCRIPTOR_TYPE_SAMPLER`, the `pImageInfo` parameter is ignored and the immutable sampler is taken from the push descriptor set layout in the pipeline layout. If the descriptor type is `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, the `sampler` member of the `pImageInfo` parameter is ignored and the immutable sampler is taken from the push descriptor set layout in the pipeline layout.

Valid Usage

- VUID-vkCmdPushDescriptorSetKHR-set-00364
  `set` **must** be less than VkPipelineLayoutCreateInfo::`setLayoutCount` provided when `layout` was created
- VUID-vkCmdPushDescriptorSetKHR-set-00365
  `set` **must** be the unique set number in the pipeline layout that uses a descriptor set layout that was created with `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR`
- VUID-vkCmdPushDescriptorSetKHR-pDescriptorWrites-06494

For each element i where pDescriptorWrites[i].descriptorType is VK_DESCRIPTOR_TYPE_SAMPLER, VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT, pDescriptorWrites[i].pImageInfo **must** be a valid pointer to an array of pDescriptorWrites[i].descriptorCount valid VkDescriptorImageInfo structures

- VUID-vkCmdPushDescriptorSetKHR-pipelineBindPoint-00363
  pipelineBindPoint **must** be supported by the commandBuffer's parent VkCommandPool's queue family

Valid Usage (Implicit)

- VUID-vkCmdPushDescriptorSetKHR-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPushDescriptorSetKHR-pipelineBindPoint-parameter
  pipelineBindPoint **must** be a valid VkPipelineBindPoint value
- VUID-vkCmdPushDescriptorSetKHR-layout-parameter
  layout **must** be a valid VkPipelineLayout handle
- VUID-vkCmdPushDescriptorSetKHR-pDescriptorWrites-parameter
  pDescriptorWrites **must** be a valid pointer to an array of descriptorWriteCount valid VkWriteDescriptorSet structures
- VUID-vkCmdPushDescriptorSetKHR-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdPushDescriptorSetKHR-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics, or compute operations
- VUID-vkCmdPushDescriptorSetKHR-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdPushDescriptorSetKHR-descriptorWriteCount-arraylength
  descriptorWriteCount **must** be greater than 0
- VUID-vkCmdPushDescriptorSetKHR-commonparent
  Both of commandBuffer, and layout **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

Alternatively, to push descriptor updates into a command buffer, call:

```
// Provided by VK_KHR_maintenance6 with VK_KHR_push_descriptor
void vkCmdPushDescriptorSet2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkPushDescriptorSetInfoKHR*           pPushDescriptorSetInfo);
```

- `commandBuffer` is the command buffer that the descriptors will be recorded in.
- `pPushDescriptorSetInfo` is a pointer to a `VkPushDescriptorSetInfoKHR` structure.

Valid Usage

- VUID-vkCmdPushDescriptorSet2KHR-pPushDescriptorSetInfo-09468
  Each bit in `pPushDescriptorSetInfo->stageFlags` **must** be a stage supported by the `commandBuffer`'s parent `VkCommandPool`'s queue family

Valid Usage (Implicit)

- VUID-vkCmdPushDescriptorSet2KHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPushDescriptorSet2KHR-pPushDescriptorSetInfo-parameter
  `pPushDescriptorSetInfo` **must** be a valid pointer to a valid VkPushDescriptorSetInfoKHR structure
- VUID-vkCmdPushDescriptorSet2KHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdPushDescriptorSet2KHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdPushDescriptorSet2KHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized

- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

The VkPushDescriptorSetInfoKHR structure is defined as:

```
// Provided by VK_KHR_maintenance6 with VK_KHR_push_descriptor
typedef struct VkPushDescriptorSetInfoKHR {
    VkStructureType                sType;
    const void*                    pNext;
    VkShaderStageFlags             stageFlags;
    VkPipelineLayout               layout;
    uint32_t                       set;
    uint32_t                       descriptorWriteCount;
    const VkWriteDescriptorSet*    pDescriptorWrites;
} VkPushDescriptorSetInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- stageFlags is a bitmask of VkShaderStageFlagBits specifying the shader stages that will use the descriptors.
- layout is a VkPipelineLayout object used to program the bindings.
- set is the set number of the descriptor set in the pipeline layout that will be updated.
- descriptorWriteCount is the number of elements in the pDescriptorWrites array.
- pDescriptorWrites is a pointer to an array of VkWriteDescriptorSet structures describing the descriptors to be updated.

If stageFlags specifies a subset of all stages corresponding to one or more pipeline bind points, the binding operation still affects all stages corresponding to the given pipeline bind point(s) as if the equivalent original version of this command had been called with the same parameters. For example, specifying a stageFlags value of VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT | VK_SHADER_STAGE_COMPUTE_BIT is equivalent to calling the original version of this command once with VK_PIPELINE_BIND_POINT_GRAPHICS and once with VK_PIPELINE_BIND_POINT_COMPUTE.

Valid Usage

- VUID-VkPushDescriptorSetInfoKHR-set-00364
  `set` **must** be less than VkPipelineLayoutCreateInfo::`setLayoutCount` provided when `layout` was created
- VUID-VkPushDescriptorSetInfoKHR-set-00365
  `set` **must** be the unique set number in the pipeline layout that uses a descriptor set layout that was created with `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR`
- VUID-VkPushDescriptorSetInfoKHR-pDescriptorWrites-06494
  For each element i where `pDescriptorWrites`[i].`descriptorType` is `VK_DESCRIPTOR_TYPE_SAMPLER`, `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT`, `pDescriptorWrites`[i].`pImageInfo` **must** be a valid pointer to an array of `pDescriptorWrites`[i].`descriptorCount` valid `VkDescriptorImageInfo` structures
- VUID-VkPushDescriptorSetInfoKHR-None-09495
  `layout` **must** be a valid VkPipelineLayout handle

Valid Usage (Implicit)

- VUID-VkPushDescriptorSetInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PUSH_DESCRIPTOR_SET_INFO_KHR`
- VUID-VkPushDescriptorSetInfoKHR-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineLayoutCreateInfo
- VUID-VkPushDescriptorSetInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPushDescriptorSetInfoKHR-stageFlags-parameter
  `stageFlags` **must** be a valid combination of VkShaderStageFlagBits values
- VUID-VkPushDescriptorSetInfoKHR-stageFlags-requiredbitmask
  `stageFlags` **must** not be `0`
- VUID-VkPushDescriptorSetInfoKHR-layout-parameter
  If `layout` is not VK_NULL_HANDLE, `layout` **must** be a valid VkPipelineLayout handle
- VUID-VkPushDescriptorSetInfoKHR-pDescriptorWrites-parameter
  `pDescriptorWrites` **must** be a valid pointer to an array of `descriptorWriteCount` valid VkWriteDescriptorSet structures
- VUID-VkPushDescriptorSetInfoKHR-descriptorWriteCount-arraylength
  `descriptorWriteCount` **must** be greater than `0`

14.2.9. Push Descriptor Updates With Descriptor Update Templates

It is also possible to use a descriptor update template to specify the push descriptors to update. To

do so, call:

```
// Provided by VK_KHR_descriptor_update_template with VK_KHR_push_descriptor,
VK_KHR_push_descriptor with VK_VERSION_1_1 or VK_KHR_descriptor_update_template
void vkCmdPushDescriptorSetWithTemplateKHR(
    VkCommandBuffer                             commandBuffer,
    VkDescriptorUpdateTemplate                  descriptorUpdateTemplate,
    VkPipelineLayout                            layout,
    uint32_t                                    set,
    const void*                                 pData);
```

- `commandBuffer` is the command buffer that the descriptors will be recorded in.
- `descriptorUpdateTemplate` is a descriptor update template defining how to interpret the descriptor information in `pData`.
- `layout` is a VkPipelineLayout object used to program the bindings. It **must** be compatible with the layout used to create the `descriptorUpdateTemplate` handle.
- `set` is the set number of the descriptor set in the pipeline layout that will be updated. This **must** be the same number used to create the `descriptorUpdateTemplate` handle.
- `pData` is a pointer to memory containing descriptors for the templated update.

Valid Usage

- VUID-vkCmdPushDescriptorSetWithTemplateKHR-commandBuffer-00366
  The `pipelineBindPoint` specified during the creation of the descriptor update template **must** be supported by the `commandBuffer`'s parent `VkCommandPool`'s queue family
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-pData-01686
  `pData` **must** be a valid pointer to a memory containing one or more valid instances of VkDescriptorImageInfo, VkDescriptorBufferInfo, or VkBufferView in a layout defined by `descriptorUpdateTemplate` when it was created with vkCreateDescriptorUpdateTemplateKHR
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-layout-07993
  `layout` **must** be compatible with the layout used to create `descriptorUpdateTemplate`
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-descriptorUpdateTemplate-07994
  `descriptorUpdateTemplate` **must** have been created with a `templateType` of `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-set-07995
  `set` **must** be the same value used to create `descriptorUpdateTemplate`
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-set-07304
  `set` **must** be less than VkPipelineLayoutCreateInfo::`setLayoutCount` provided when `layout` was created
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-set-07305
  `set` **must** be the unique set number in the pipeline layout that uses a descriptor set layout that was created with `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR`

Valid Usage (Implicit)

- VUID-vkCmdPushDescriptorSetWithTemplateKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-descriptorUpdateTemplate-parameter
  `descriptorUpdateTemplate` **must** be a valid VkDescriptorUpdateTemplate handle
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-layout-parameter
  `layout` **must** be a valid VkPipelineLayout handle
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdPushDescriptorSetWithTemplateKHR-commonparent
  Each of `commandBuffer`, `descriptorUpdateTemplate`, and `layout` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

API Example

```
struct AppDataStructure
{
    VkDescriptorImageInfo  imageInfo;          // a single image info
    // ... some more application-related data
};

const VkDescriptorUpdateTemplateEntry descriptorUpdateTemplateEntries[] =
{
```

```
    // binding to a single image descriptor
    {
        .binding = 0,
        .dstArrayElement = 0,
        .descriptorCount = 1,
        .descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER,
        .offset = offsetof(AppDataStructure, imageInfo),
        .stride = 0     // not required if descriptorCount is 1
    }
};

// create a descriptor update template for push descriptor set updates
const VkDescriptorUpdateTemplateCreateInfo createInfo =
{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_CREATE_INFO,
    .pNext = NULL,
    .flags = 0,
    .descriptorUpdateEntryCount = 1,
    .pDescriptorUpdateEntries = descriptorUpdateTemplateEntries,
    .templateType = VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR,
    .descriptorSetLayout = 0,   // ignored by given templateType
    .pipelineBindPoint = VK_PIPELINE_BIND_POINT_GRAPHICS,
    .pipelineLayout = myPipelineLayout,
    .set = 0,
};

VkDescriptorUpdateTemplate myDescriptorUpdateTemplate;
myResult = vkCreateDescriptorUpdateTemplate(
    myDevice,
    &createInfo,
    NULL,
    &myDescriptorUpdateTemplate);

AppDataStructure appData;
// fill appData here or cache it in your engine
vkCmdPushDescriptorSetWithTemplateKHR(myCmdBuffer, myDescriptorUpdateTemplate,
myPipelineLayout, 0,&appData);
```

Alternatively, to use a descriptor update template to specify the push descriptors to update, call:

```
// Provided by VK_KHR_maintenance6 with VK_KHR_push_descriptor
void vkCmdPushDescriptorSetWithTemplate2KHR(
    VkCommandBuffer                                commandBuffer,
    const VkPushDescriptorSetWithTemplateInfoKHR* pPushDescriptorSetWithTemplateInfo);
```

- `commandBuffer` is the command buffer that the descriptors will be recorded in.
- `pPushDescriptorSetWithTemplateInfo` is a pointer to a `VkPushDescriptorSetWithTemplateInfoKHR` structure.

Valid Usage (Implicit)

- VUID-vkCmdPushDescriptorSetWithTemplate2KHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPushDescriptorSetWithTemplate2KHR-pPushDescriptorSetWithTemplateInfo-parameter
  `pPushDescriptorSetWithTemplateInfo` **must** be a valid pointer to a valid VkPushDescriptorSetWithTemplateInfoKHR structure
- VUID-vkCmdPushDescriptorSetWithTemplate2KHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdPushDescriptorSetWithTemplate2KHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdPushDescriptorSetWithTemplate2KHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

The `VkPushDescriptorSetWithTemplateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance6 with VK_KHR_push_descriptor
typedef struct VkPushDescriptorSetWithTemplateInfoKHR {
    VkStructureType               sType;
    const void*                   pNext;
    VkDescriptorUpdateTemplate    descriptorUpdateTemplate;
    VkPipelineLayout              layout;
    uint32_t                      set;
    const void*                   pData;
} VkPushDescriptorSetWithTemplateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `descriptorUpdateTemplate` is a descriptor update template defining how to interpret the descriptor information in `pData`.
- `layout` is a VkPipelineLayout object used to program the bindings. It **must** be compatible with the layout used to create the `descriptorUpdateTemplate` handle.
- `set` is the set number of the descriptor set in the pipeline layout that will be updated. This **must** be the same number used to create the `descriptorUpdateTemplate` handle.
- `pData` is a pointer to memory containing descriptors for the templated update.

Valid Usage

- VUID-VkPushDescriptorSetWithTemplateInfoKHR-commandBuffer-00366
  The `pipelineBindPoint` specified during the creation of the descriptor update template **must** be supported by the `commandBuffer`'s parent `VkCommandPool`'s queue family
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-pData-01686
  `pData` **must** be a valid pointer to a memory containing one or more valid instances of VkDescriptorImageInfo, VkDescriptorBufferInfo, or VkBufferView in a layout defined by `descriptorUpdateTemplate` when it was created with vkCreateDescriptorUpdateTemplateKHR
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-layout-07993
  `layout` **must** be compatible with the layout used to create `descriptorUpdateTemplate`
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-descriptorUpdateTemplate-07994
  `descriptorUpdateTemplate` **must** have been created with a `templateType` of `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-set-07995
  `set` **must** be the same value used to create `descriptorUpdateTemplate`
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-set-07304
  `set` **must** be less than VkPipelineLayoutCreateInfo::`setLayoutCount` provided when `layout` was created
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-set-07305
  `set` **must** be the unique set number in the pipeline layout that uses a descriptor set layout that was created with `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR`
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-None-09495
  `layout` **must** be a valid VkPipelineLayout handle

Valid Usage (Implicit)

- VUID-VkPushDescriptorSetWithTemplateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PUSH_DESCRIPTOR_SET_WITH_TEMPLATE_INFO_KHR`
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-pNext-pNext

`pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineLayoutCreateInfo

- VUID-VkPushDescriptorSetWithTemplateInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-descriptorUpdateTemplate-parameter
  `descriptorUpdateTemplate` **must** be a valid VkDescriptorUpdateTemplate handle
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-layout-parameter
  If `layout` is not VK_NULL_HANDLE, `layout` **must** be a valid VkPipelineLayout handle
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-pData-parameter
  `pData` **must** be a pointer value
- VUID-VkPushDescriptorSetWithTemplateInfoKHR-commonparent
  Both of `descriptorUpdateTemplate`, and `layout` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

14.2.10. Push Constant Updates

As described above in section Pipeline Layouts, the pipeline layout defines shader push constants which are updated via Vulkan commands rather than via writes to memory or copy commands.

NOTE Push constants represent a high speed path to modify constant data in pipelines that is expected to outperform memory-backed resource updates.

To update push constants, call:

```
// Provided by VK_VERSION_1_0
void vkCmdPushConstants(
    VkCommandBuffer                             commandBuffer,
    VkPipelineLayout                            layout,
    VkShaderStageFlags                          stageFlags,
    uint32_t                                    offset,
    uint32_t                                    size,
    const void*                                 pValues);
```

- `commandBuffer` is the command buffer in which the push constant update will be recorded.
- `layout` is the pipeline layout used to program the push constant updates.
- `stageFlags` is a bitmask of VkShaderStageFlagBits specifying the shader stages that will use the push constants in the updated range.
- `offset` is the start offset of the push constant range to update, in units of bytes.
- `size` is the size of the push constant range to update, in units of bytes.
- `pValues` is a pointer to an array of `size` bytes containing the new push constant values.

When a command buffer begins recording, all push constant values are undefined. Reads of undefined push constant values by the executing shader return undefined values.

Push constant values **can** be updated incrementally, causing shader stages in `stageFlags` to read the new data from `pValues` for push constants modified by this command, while still reading the previous data for push constants not modified by this command. When a bound pipeline command is issued, the bound pipeline's layout **must** be compatible with the layouts used to set the values of all push constants in the pipeline layout's push constant ranges, as described in Pipeline Layout Compatibility. Binding a pipeline with a layout that is not compatible with the push constant layout does not disturb the push constant values.

NOTE As `stageFlags` needs to include all flags the relevant push constant ranges were created with, any flags that are not supported by the queue family that the VkCommandPool used to allocate `commandBuffer` was created on are ignored.

Valid Usage

- VUID-vkCmdPushConstants-offset-01795
  For each byte in the range specified by `offset` and `size` and for each shader stage in `stageFlags`, there **must** be a push constant range in `layout` that includes that byte and that stage
- VUID-vkCmdPushConstants-offset-01796
  For each byte in the range specified by `offset` and `size` and for each push constant range that overlaps that byte, `stageFlags` **must** include all stages in that push constant range's VkPushConstantRange::`stageFlags`
- VUID-vkCmdPushConstants-offset-00368
  `offset` **must** be a multiple of `4`
- VUID-vkCmdPushConstants-size-00369
  `size` **must** be a multiple of `4`
- VUID-vkCmdPushConstants-offset-00370
  `offset` **must** be less than `VkPhysicalDeviceLimits`::`maxPushConstantsSize`
- VUID-vkCmdPushConstants-size-00371
  `size` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxPushConstantsSize` minus `offset`

Valid Usage (Implicit)

- VUID-vkCmdPushConstants-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPushConstants-layout-parameter
  `layout` **must** be a valid VkPipelineLayout handle
- VUID-vkCmdPushConstants-stageFlags-parameter
  `stageFlags` **must** be a valid combination of VkShaderStageFlagBits values
- VUID-vkCmdPushConstants-stageFlags-requiredbitmask
  `stageFlags` **must** not be `0`

- VUID-vkCmdPushConstants-pValues-parameter
  `pValues` **must** be a valid pointer to an array of `size` bytes
- VUID-vkCmdPushConstants-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdPushConstants-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdPushConstants-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdPushConstants-size-arraylength
  `size` **must** be greater than `0`
- VUID-vkCmdPushConstants-commonparent
  Both of `commandBuffer`, and `layout` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

Alternatively, to update push constants, call:

```
// Provided by VK_KHR_maintenance6
void vkCmdPushConstants2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkPushConstantsInfoKHR*               pPushConstantsInfo);
```

- `commandBuffer` is the command buffer in which the push constant update will be recorded.
- `pPushConstantsInfo` is a pointer to a `VkPushConstantsInfoKHR` structure.

Valid Usage (Implicit)

- VUID-vkCmdPushConstants2KHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPushConstants2KHR-pPushConstantsInfo-parameter
  `pPushConstantsInfo` **must** be a valid pointer to a valid VkPushConstantsInfoKHR structure
- VUID-vkCmdPushConstants2KHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdPushConstants2KHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdPushConstants2KHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	State

The `VkPushConstantsInfoKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance6
typedef struct VkPushConstantsInfoKHR {
    VkStructureType       sType;
    const void*           pNext;
    VkPipelineLayout      layout;
    VkShaderStageFlags    stageFlags;
    uint32_t              offset;
    uint32_t              size;
    const void*           pValues;
} VkPushConstantsInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `layout` is the pipeline layout used to program the push constant updates.

- `stageFlags` is a bitmask of VkShaderStageFlagBits specifying the shader stages that will use the push constants in the updated range.
- `offset` is the start offset of the push constant range to update, in units of bytes.
- `size` is the size of the push constant range to update, in units of bytes.
- `pValues` is a pointer to an array of `size` bytes containing the new push constant values.

Valid Usage

- VUID-VkPushConstantsInfoKHR-offset-01795
  For each byte in the range specified by `offset` and `size` and for each shader stage in `stageFlags`, there **must** be a push constant range in `layout` that includes that byte and that stage
- VUID-VkPushConstantsInfoKHR-offset-01796
  For each byte in the range specified by `offset` and `size` and for each push constant range that overlaps that byte, `stageFlags` **must** include all stages in that push constant range's VkPushConstantRange::`stageFlags`
- VUID-VkPushConstantsInfoKHR-offset-00368
  `offset` **must** be a multiple of `4`
- VUID-VkPushConstantsInfoKHR-size-00369
  `size` **must** be a multiple of `4`
- VUID-VkPushConstantsInfoKHR-offset-00370
  `offset` **must** be less than `VkPhysicalDeviceLimits`::`maxPushConstantsSize`
- VUID-VkPushConstantsInfoKHR-size-00371
  `size` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxPushConstantsSize` minus `offset`
- VUID-VkPushConstantsInfoKHR-None-09495
  `layout` **must** be a valid VkPipelineLayout handle

Valid Usage (Implicit)

- VUID-VkPushConstantsInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PUSH_CONSTANTS_INFO_KHR`
- VUID-VkPushConstantsInfoKHR-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineLayoutCreateInfo
- VUID-VkPushConstantsInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPushConstantsInfoKHR-layout-parameter
  If `layout` is not VK_NULL_HANDLE, `layout` **must** be a valid VkPipelineLayout handle
- VUID-VkPushConstantsInfoKHR-stageFlags-parameter
  `stageFlags` **must** be a valid combination of VkShaderStageFlagBits values

- VUID-VkPushConstantsInfoKHR-stageFlags-requiredbitmask
  `stageFlags` **must** not be `0`
- VUID-VkPushConstantsInfoKHR-pValues-parameter
  `pValues` **must** be a valid pointer to an array of `size` bytes
- VUID-VkPushConstantsInfoKHR-size-arraylength
  `size` **must** be greater than `0`

14.3. Physical Storage Buffer Access

To query a 64-bit buffer device address value through which buffer memory **can** be accessed in a shader, call:

```
// Provided by VK_KHR_buffer_device_address
VkDeviceAddress vkGetBufferDeviceAddressKHR(
    VkDevice                                    device,
    const VkBufferDeviceAddressInfo*            pInfo);
```

- `device` is the logical device that the buffer was created on.
- `pInfo` is a pointer to a VkBufferDeviceAddressInfo structure specifying the buffer to retrieve an address for.

The 64-bit return value is an address of the start of `pInfo->buffer`. The address range starting at this value and whose size is the size of the buffer **can** be used in a shader to access the memory bound to that buffer, using the `SPV_KHR_physical_storage_buffer` extension and the `PhysicalStorageBuffer` storage class. For example, this value **can** be stored in a uniform buffer, and the shader **can** read the value from the uniform buffer and use it to do a dependent read/write to this buffer. A value of zero is reserved as a "null" pointer and **must** not be returned as a valid buffer device address. All loads, stores, and atomics in a shader through `PhysicalStorageBuffer` pointers **must** access addresses in the address range of some buffer.

If the buffer was created with a non-zero value of VkBufferOpaqueCaptureAddressCreateInfo::`opaqueCaptureAddress`, the return value will be the same address that was returned at capture time.

The returned address **must** satisfy the alignment requirement specified by VkMemoryRequirements::`alignment` for the buffer in VkBufferDeviceAddressInfo::`buffer`.

If multiple VkBuffer objects are bound to overlapping ranges of VkDeviceMemory, implementations **may** return address ranges which overlap. In this case, it is ambiguous which VkBuffer is associated with any given device address. For purposes of valid usage, if multiple VkBuffer objects **can** be attributed to a device address, a VkBuffer is selected such that valid usage passes, if it exists.

Valid Usage

- VUID-vkGetBufferDeviceAddress-bufferDeviceAddress-03324

The bufferDeviceAddress feature **must** be enabled

- VUID-vkGetBufferDeviceAddress-device-03325
  If device was created with multiple physical devices, then the bufferDeviceAddressMultiDevice feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkGetBufferDeviceAddress-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetBufferDeviceAddress-pInfo-parameter
  pInfo **must** be a valid pointer to a valid VkBufferDeviceAddressInfo structure

The VkBufferDeviceAddressInfo structure is defined as:

```
typedef struct VkBufferDeviceAddressInfo {
    VkStructureType    sType;
    const void*        pNext;
    VkBuffer           buffer;
} VkBufferDeviceAddressInfo;
```

or the equivalent

```
// Provided by VK_KHR_buffer_device_address
typedef VkBufferDeviceAddressInfo VkBufferDeviceAddressInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- buffer specifies the buffer whose address is being queried.

Valid Usage

- VUID-VkBufferDeviceAddressInfo-buffer-02600
  If buffer is non-sparse and was not created with the VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT flag, then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkBufferDeviceAddressInfo-buffer-02601
  buffer **must** have been created with VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT

Valid Usage (Implicit)

- VUID-VkBufferDeviceAddressInfo-sType-sType

`sType` **must** be `VK_STRUCTURE_TYPE_BUFFER_DEVICE_ADDRESS_INFO`

- VUID-VkBufferDeviceAddressInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkBufferDeviceAddressInfo-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle

To query a 64-bit buffer opaque capture address, call:

```
// Provided by VK_KHR_buffer_device_address
uint64_t vkGetBufferOpaqueCaptureAddressKHR(
    VkDevice                                    device,
    const VkBufferDeviceAddressInfo*            pInfo);
```

- `device` is the logical device that the buffer was created on.
- `pInfo` is a pointer to a VkBufferDeviceAddressInfo structure specifying the buffer to retrieve an address for.

The 64-bit return value is an opaque capture address of the start of `pInfo->buffer`.

If the buffer was created with a non-zero value of VkBufferOpaqueCaptureAddressCreateInfo ::`opaqueCaptureAddress` the return value **must** be the same address.

Valid Usage

- VUID-vkGetBufferOpaqueCaptureAddress-None-03326
  The `bufferDeviceAddress` feature **must** be enabled
- VUID-vkGetBufferOpaqueCaptureAddress-device-03327
  If `device` was created with multiple physical devices, then the `bufferDeviceAddressMultiDevice` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkGetBufferOpaqueCaptureAddress-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetBufferOpaqueCaptureAddress-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkBufferDeviceAddressInfo structure

The `VkStridedDeviceAddressRegionKHR` structure is defined as:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef struct VkStridedDeviceAddressRegionKHR {
    VkDeviceAddress    deviceAddress;
    VkDeviceSize       stride;
    VkDeviceSize       size;
} VkStridedDeviceAddressRegionKHR;
```

- `deviceAddress` is the device address (as returned by the vkGetBufferDeviceAddressKHR command) at which the region starts, or zero if the region is unused.
- `stride` is the byte stride between consecutive elements.
- `size` is the size in bytes of the region starting at `deviceAddress`.

Valid Usage

- VUID-VkStridedDeviceAddressRegionKHR-size-04631
  If `size` is not zero, all addresses between `deviceAddress` and `deviceAddress` + `size` - 1 **must** be in the buffer device address range of the same buffer
- VUID-VkStridedDeviceAddressRegionKHR-size-04632
  If `size` is not zero, `stride` **must** be less than or equal to the size of the buffer from which `deviceAddress` was queried

Chapter 15. Shader Interfaces

When a pipeline is created, the set of shaders specified in the corresponding `VkPipelineCreateInfo` structure are implicitly linked at a number of different interfaces.

- Shader Input and Output Interface
- Vertex Input Interface
- Fragment Output Interface
- Fragment Tile Image Interface
- Fragment Input Attachment Interface
- Ray Tracing Pipeline Interface
- Shader Resource Interface

This chapter describes valid uses for a set of SPIR-V decorations. Any other use of one of these decorations is invalid, with the exception that, when using SPIR-V versions 1.4 and earlier: `Block`, `BufferBlock`, `Offset`, `ArrayStride`, and `MatrixStride` can also decorate types and type members used by variables in the `Private` and `Function` storage classes.

NOTE In this chapter, there are references to SPIR-V terms such as the `MeshNV` execution model. These terms will appear even in a build of the specification which does not support any extensions. This is as intended, since these terms appear in the unified SPIR-V specification without such qualifiers.

15.1. Shader Input and Output Interfaces

When multiple stages are present in a pipeline, the outputs of one stage form an interface with the inputs of the next stage. When such an interface involves a shader, shader outputs are matched against the inputs of the next stage, and shader inputs are matched against the outputs of the previous stage.

All the variables forming the shader input and output *interfaces* are listed as operands to the `OpEntryPoint` instruction and are declared with the `Input` or `Output` storage classes, respectively, in the SPIR-V module. These generally form the interfaces between consecutive shader stages, regardless of any non-shader stages between the consecutive shader stages.

There are two classes of variables that **can** be matched between shader stages, built-in variables and user-defined variables. Each class has a different set of matching criteria.

`Output` variables of a shader stage have undefined values until the shader writes to them or uses the `Initializer` operand when declaring the variable.

15.1.1. Built-in Interface Block

Shader built-in variables meeting the following requirements define the *built-in interface block*. They **must**

- be explicitly declared (there are no implicit built-ins),
- be identified with a `BuiltIn` decoration,
- form object types as described in the Built-in Variables section, and
- be declared in a block whose top-level members are the built-ins.

There **must** be no more than one built-in interface block per shader per interface .

Built-ins **must** not have any `Location` or `Component` decorations.

15.1.2. User-defined Variable Interface

The non-built-in variables listed by `OpEntryPoint` with the `Input` or `Output` storage class form the *user-defined variable interface*. These **must** have numeric type or, recursively, composite types of such types. If an implementation supports `storageInputOutput16`, components **can** have a width of 16 bits. These variables **must** be identified with a `Location` decoration and **can** also be identified with a `Component` decoration.

15.1.3. Interface Matching

An output variable, block, or structure member in a given shader stage has an interface match with an input variable, block, or structure member in a subsequent shader stage if they both adhere to the following conditions:

- They have equivalent decorations, other than:
  - `XfbBuffer`, `XfbStride`, `Offset`, and `Stream`
  - one is not decorated with `Component` and the other is declared with a `Component` of `0`
  - Interpolation decorations
  - `RelaxedPrecision` if one is an input variable and the other an output variable
- Their types match as follows:
  - if the input is declared in a tessellation control or geometry shader as an `OpTypeArray` with an `Element Type` equivalent to the `OpType*` declaration of the output, and neither is a structure member; or
  - if the `maintenance4` feature is enabled, they are declared as `OpTypeVector` variables, and the output has a `Component Count` value higher than that of the input but the same `Component Type`; or
  - if the input is decorated with `PerVertexKHR`, and is declared in a fragment shader as an `OpTypeArray` with an `Element Type` equivalent to the `OpType*` declaration of the output, and neither the input nor the output is a structure member; or
  - if in any other case they are declared with an equivalent `OpType*` declaration.
- If both are structures and every member has an interface match.

NOTE	The word “structure” above refers to both variables that have an `OpTypeStruct` type and interface blocks (which are also declared as `OpTypeStruct`).

All input variables and blocks **must** have an interface match in the preceding shader stage, except for built-in variables in fragment shaders. Shaders **can** declare and write to output variables that are not declared or read by the subsequent stage.

The value of an input variable is undefined if the preceding stage does not write to a matching output variable, as described above.

15.1.4. Location Assignment

This section describes `Location` assignments for user-defined variables and how many `Location` slots are consumed by a given user-variable type. As mentioned above, some inputs and outputs have an additional level of arrayness relative to other shader inputs and outputs. This outer array level is removed from the type before considering how many `Location` slots the type consumes.

The `Location` value specifies an interface slot comprised of a 32-bit four-component vector conveyed between stages. The `Component` specifies word components within these vector `Location` slots. Only types with widths of 16, 32 or 64 are supported in shader interfaces.

Inputs and outputs of the following types consume a single interface `Location`:

- 16-bit scalar and vector types, and
- 32-bit scalar and vector types, and
- 64-bit scalar and 2-component vector types.

64-bit three- and four-component vectors consume two consecutive `Location` slots.

If a declared input or output is an array of size n and each element takes m `Location` slots, it will be assigned $m \times n$ consecutive `Location` slots starting with the specified `Location`.

If the declared input or output is an $n \times m$ 16-, 32- or 64-bit matrix, it will be assigned multiple `Location` slots starting with the specified `Location`. The number of `Location` slots assigned for each matrix will be the same as for an n-element array of m-component vectors.

An `OpVariable` with a structure type that is not a block **must** be decorated with a `Location`.

When an `OpVariable` with a structure type (either block or non-block) is decorated with a `Location`, the members in the structure type **must** not be decorated with a `Location`. The `OpVariable`'s members are assigned consecutive `Location` slots in declaration order, starting from the first member, which is assigned the `Location` decoration from the `OpVariable`.

When a block-type `OpVariable` is declared without a `Location` decoration, each member in its structure type **must** be decorated with a `Location`. Types nested deeper than the top-level members **must** not have `Location` decorations.

The `Location` slots consumed by block and structure members are determined by applying the rules above in a depth-first traversal of the instantiated members as though the structure or block member were declared as an input or output variable of the same type.

Any two inputs listed as operands on the same `OpEntryPoint` **must** not be assigned the same `Location` slot and `Component` word, either explicitly or implicitly. Any two outputs listed as operands

on the same OpEntryPoint **must** not be assigned the same Location slot and Component word, either explicitly or implicitly.

The number of input and output Location slots available for a shader input or output interface is limited, and dependent on the shader stage as described in Shader Input and Output Locations. All variables in both the built-in interface block and the user-defined variable interface count against these limits. Each effective Location **must** have a value less than the number of Location slots available for the given interface, as specified in the "Locations Available" column in Shader Input and Output Locations.

Table 15. Shader Input and Output Locations

Shader Interface	Locations Available
vertex input	maxVertexInputAttributes
vertex output	maxVertexOutputComponents / 4
tessellation control input	maxTessellationControlPerVertexInputComponents / 4
tessellation control output	maxTessellationControlPerVertexOutputComponents / 4
tessellation evaluation input	maxTessellationEvaluationInputComponents / 4
tessellation evaluation output	maxTessellationEvaluationOutputComponents / 4
geometry input	maxGeometryInputComponents / 4
geometry output	maxGeometryOutputComponents / 4
fragment input	maxFragmentInputComponents / 4
fragment output	maxFragmentOutputAttachments

15.1.5. Component Assignment

The Component decoration allows the Location to be more finely specified for scalars and vectors, down to the individual Component word within a Location slot that are consumed. The Component word within a Location are 0, 1, 2, and 3. A variable or block member starting at Component N will consume Component words N, N+1, N+2, ... up through its size. For 16-, and 32-bit types, it is invalid if this sequence of Component words gets larger than 3. A scalar 64-bit type will consume two of these Component words in sequence, and a two-component 64-bit vector type will consume all four Component words available within a Location. A three- or four-component 64-bit vector type **must** not specify a non-zero Component decoration. A three-component 64-bit vector type will consume all four Component words of the first Location and Component 0 and 1 of the second Location. This leaves Component 2 and 3 available for other component-qualified declarations.

A scalar or two-component 64-bit data type **must** not specify a Component decoration of 1 or 3. A Component decoration **must** not be specified for any type that is not a scalar or vector.

A four-component 64-bit data type will consume all four Component words of the first Location and all four Component words of the second Location.

15.2. Vertex Input Interface

When the vertex stage is present in a pipeline, the vertex shader input variables form an interface with the vertex input attributes. The vertex shader input variables are matched by the `Location` and `Component` decorations to the vertex input attributes specified in the `pVertexInputState` member of the VkGraphicsPipelineCreateInfo structure.

The vertex shader input variables listed by `OpEntryPoint` with the `Input` storage class form the *vertex input interface*. These variables **must** be identified with a `Location` decoration and **can** also be identified with a `Component` decoration.

For the purposes of interface matching: variables declared without a `Component` decoration are considered to have a `Component` decoration of zero. The number of available vertex input `Location` slots is given by the `maxVertexInputAttributes` member of the `VkPhysicalDeviceLimits` structure.

See Attribute Location and Component Assignment for details.

All vertex shader inputs declared as above **must** have a corresponding attribute and binding in the pipeline.

15.3. Fragment Output Interface

When the fragment stage is present in a pipeline, the fragment shader outputs form an interface with the output attachments defined by a render pass instance. The fragment shader output variables are matched by the `Location` and `Component` decorations to specified color attachments.

The fragment shader output variables listed by `OpEntryPoint` with the `Output` storage class form the *fragment output interface*. These variables **must** be identified with a `Location` decoration. They **can** also be identified with a `Component` decoration and/or an `Index` decoration. For the purposes of interface matching: variables declared without a `Component` decoration are considered to have a `Component` decoration of zero, and variables declared without an `Index` decoration are considered to have an `Index` decoration of zero.

A fragment shader output variable identified with a `Location` decoration of *i* is associated with the color attachment indicated by VkRenderingInfo::`pColorAttachments`[*i*]. When using render pass objects, it is associated with the color attachment indicated by VkSubpassDescription ::`pColorAttachments`[*i*]. Values are written to those attachments after passing through the blending unit as described in Blending, if enabled. Locations are consumed as described in Location Assignment. The number of available fragment output `Location` slots is given by the `maxFragmentOutputAttachments` member of the `VkPhysicalDeviceLimits` structure.

If the `dynamicRenderingLocalRead` feature is supported, fragment output locations **can** be remapped when using dynamic rendering.

To set the fragment output location mappings during rendering, call:

```
// Provided by VK_KHR_dynamic_rendering_local_read
void vkCmdSetRenderingAttachmentLocationsKHR(
    VkCommandBuffer                             commandBuffer,
    const VkRenderingAttachmentLocationInfoKHR* pLocationInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pLocationInfo` is a VkRenderingAttachmentLocationInfoKHR structure indicating the new mappings.

This command sets the attachment location mappings for subsequent drawing commands, and **must** match the mappings provided to the bound pipeline, if one is bound, which **can** be set by chaining VkRenderingAttachmentLocationInfoKHR to VkGraphicsPipelineCreateInfo.

Until this command is called, mappings in the command buffer state are treated as each color attachment specified in vkCmdBeginRenderingKHR having a location equal to its index in VkRenderingInfo::`pColorAttachments`. This state is reset whenever vkCmdBeginRenderingKHR is called.

Valid Usage

- VUID-vkCmdSetRenderingAttachmentLocationsKHR-dynamicRenderingLocalRead-09509
  `dynamicRenderingLocalRead` **must** be enabled
- VUID-vkCmdSetRenderingAttachmentLocationsKHR-pLocationInfo-09510
  `pLocationInfo->colorAttachmentCount` **must** be equal to the value of VkRenderingInfo::`colorAttachmentCount` used to begin the current render pass instance
- VUID-vkCmdSetRenderingAttachmentLocationsKHR-commandBuffer-09511
  The current render pass instance **must** have been started or resumed by vkCmdBeginRenderingKHR in this `commandBuffer`

Valid Usage (Implicit)

- VUID-vkCmdSetRenderingAttachmentLocationsKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetRenderingAttachmentLocationsKHR-pLocationInfo-parameter
  `pLocationInfo` **must** be a valid pointer to a valid VkRenderingAttachmentLocationInfoKHR structure
- VUID-vkCmdSetRenderingAttachmentLocationsKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetRenderingAttachmentLocationsKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetRenderingAttachmentLocationsKHR-renderpass
  This command **must** only be called inside of a render pass instance

- VUID-vkCmdSetRenderingAttachmentLocationsKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	State

The `VkRenderingAttachmentLocationInfoKHR` structure is defined as:

```
// Provided by VK_KHR_dynamic_rendering_local_read
typedef struct VkRenderingAttachmentLocationInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           colorAttachmentCount;
    const uint32_t*    pColorAttachmentLocations;
} VkRenderingAttachmentLocationInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `colorAttachmentCount` is the number of elements in `pColorAttachmentLocations`.
- `pColorAttachmentLocations` is a pointer to an array of `colorAttachmentCount uint32_t` values defining remapped locations for color attachments.

This structure allows applications to remap the locations of color attachments to different fragment shader output locations.

Each element of `pColorAttachmentLocations` set to `VK_ATTACHMENT_UNUSED` will be inaccessible to this pipeline as a color attachment; no location will map to it. Each element of `pColorAttachmentLocations` set to any other value will map the specified location value to the color attachment specified in the render pass at the corresponding index in the `pColorAttachmentLocations` array. Any writes to a fragment output location that is not mapped to an attachment **must** be discarded.

If `pColorAttachmentLocations` is `NULL`, it is equivalent to setting each element to its index within the

array.

This structure **can** be included in the pNext chain of a VkGraphicsPipelineCreateInfo structure to set this state for a pipeline. If this structure is not included in the pNext chain of VkGraphicsPipelineCreateInfo, it is equivalent to specifying this structure with the following properties:

- colorAttachmentCount set to VkPipelineRenderingCreateInfo::colorAttachmentCount.
- pColorAttachmentLocations set to NULL.

This structure **can** be included in the pNext chain of a VkCommandBufferInheritanceInfo structure to specify inherited state from the primary command buffer. If VkCommandBufferInheritanceInfo::renderPass is not VK_NULL_HANDLE, or VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT is not specified in VkCommandBufferBeginInfo::flags, members of this structure are ignored. If this structure is not included in the pNext chain of VkCommandBufferInheritanceInfo, it is equivalent to specifying this structure with the following properties:

- colorAttachmentCount set to VkCommandBufferInheritanceRenderingInfo::colorAttachmentCount.
- pColorAttachmentLocations set to NULL.

Valid Usage

- VUID-VkRenderingAttachmentLocationInfoKHR-dynamicRenderingLocalRead-09512
  If the dynamicRenderingLocalRead feature is not enabled, and pColorAttachmentLocations is not NULL, each element **must** be the value of its index within the array
- VUID-VkRenderingAttachmentLocationInfoKHR-pColorAttachmentLocations-09513
  Elements of pColorAttachmentLocations that are not VK_ATTACHMENT_UNUSED **must** each be unique
- VUID-VkRenderingAttachmentLocationInfoKHR-colorAttachmentCount-09514
  colorAttachmentCount **must** be less than or equal to maxColorAttachments
- VUID-VkRenderingAttachmentLocationInfoKHR-pColorAttachmentLocations-09515
  Each element of pColorAttachmentLocations **must** be less than maxColorAttachments

Valid Usage (Implicit)

- VUID-VkRenderingAttachmentLocationInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_LOCATION_INFO_KHR

When an active fragment shader invocation finishes, the values of all fragment shader outputs are copied out and used as blend inputs or color attachments writes. If the invocation does not set a value for them, the input values to those blending or color attachment writes are undefined.

Components of the output variables are assigned as described in Component Assignment. Output Component words identified as 0, 1, 2, and 3 will be directed to the R, G, B, and A inputs to the

blending unit, respectively, or to the output attachment if blending is disabled. If two variables are placed within the same `Location`, they **must** have the same underlying type (floating-point or integer). `Component` words which do not correspond to any fragment shader output will also result in undefined values for blending or color attachment writes.

Fragment outputs identified with an `Index` of zero are directed to the first input of the blending unit associated with the corresponding `Location`. Outputs identified with an `Index` of one are directed to the second input of the corresponding blending unit.

There **must** be no output variable which has the same `Location`, `Component`, and `Index` as any other, either explicitly declared or implied.

Output values written by a fragment shader **must** be declared with either `OpTypeFloat` or `OpTypeInt`, and a `Width` of 32. If `storageInputOutput16` is supported, output values written by a fragment shader **can** be also declared with either `OpTypeFloat` or `OpTypeInt` and a `Width` of 16. Composites of these types are also permitted. If the color attachment has a signed or unsigned normalized fixed-point format, color values are assumed to be floating-point and are converted to fixed-point as described in Conversion From Floating-Point to Normalized Fixed-Point; If the color attachment has an integer format, color values are assumed to be integers and converted to the bit-depth of the target. Any value that cannot be represented in the attachment's format is undefined. For any other attachment format no conversion is performed. If the type of the values written by the fragment shader do not match the format of the corresponding color attachment, the resulting values are undefined for those components.

15.4. Fragment Tile Image Interface

When a fragment stage is present in a pipeline, the fragment shader tile image variables decorated with `Location` form an interface with the color attachments defined by the render pass instance. The fragment shader tile image variables are matched by `Location` decorations to the color attachments specified in the `pColorAttachments` array of the VkRenderingInfoKHR structure describing the render pass instance the fragment shader is executed in.

The fragment shader variables listed by `OpEntryPoint` with the `TileImageEXT` storage class and a decoration of `Location` form the *fragment tile image interface*. These variables **must** be declared with a type of `OpTypeImage`, and a `Dim` operand of `TileImageDataEXT`. The `Component` decoration is not supported for these variables.

Reading from a tile image variable with a `Location` decoration of *i* reads from the color attachment identified by the element of VkRenderingInfoKHR::`pColorAttachments` with a `location` equal to *i*. If the tile image variable is declared as an array of size N, it consumes N consecutive tile image locations, starting with the index specified. There **must** not be more than one tile image variable with the same `Location` whether explicitly declared or implied by an array declaration. The number of available tile image locations is the same as the number of available fragment output locations as given by the `maxFragmentOutputAttachments` member of the `VkPhysicalDeviceLimits` structure.

The basic data type (floating-point, integer, unsigned integer) of the tile image variable **must** match the basic format of the corresponding color attachment, or the values read from the tile image variables are undefined.

15.5. Fragment Input Attachment Interface

When a fragment stage is present in a pipeline, the fragment shader subpass inputs form an interface with the input attachments of the current subpass. The fragment shader subpass input variables are matched by InputAttachmentIndex decorations to the input attachments specified in the pInputAttachments array of the VkSubpassDescription structure describing the subpass that the fragment shader is executed in.

The fragment shader subpass input variables with the UniformConstant storage class and a decoration of InputAttachmentIndex that are statically used by OpEntryPoint form the *fragment input attachment interface*. These variables **must** be declared with a type of OpTypeImage, a Dim operand of SubpassData, an Arrayed operand of 0, and a Sampled operand of 2. The MS operand of the OpTypeImage **must** be 0 if the samples field of the corresponding VkAttachmentDescription is VK_SAMPLE_COUNT_1_BIT and 1 otherwise.

A subpass input variable identified with an InputAttachmentIndex decoration of *i* reads from the input attachment indicated by pInputAttachments[*i*] member of VkSubpassDescription. If the subpass input variable is declared as an array of size N, it consumes N consecutive input attachments, starting with the index specified. There **must** not be more than one input variable with the same InputAttachmentIndex whether explicitly declared or implied by an array declaration per image aspect. A multi-aspect image (e.g. a depth/stencil format) **can** use the same input variable. The number of available input attachment indices is given by the maxPerStageDescriptorInputAttachments member of the VkPhysicalDeviceLimits structure.

When using dynamic rendering with the dynamicRenderingLocalRead feature enabled, a subpass input variable with a InputAttachmentIndex decoration of *i* can be mapped to a color, depth, or stencil attachment.

To set the input attachment index mappings during dynamic rendering, call:

```
// Provided by VK_KHR_dynamic_rendering_local_read
void vkCmdSetRenderingInputAttachmentIndicesKHR(
    VkCommandBuffer                                 commandBuffer,
    const VkRenderingInputAttachmentIndexInfoKHR* pInputAttachmentIndexInfo);
```

- commandBuffer is the command buffer into which the command will be recorded.
- pInputAttachmentIndexInfo is a VkRenderingInputAttachmentIndexInfoKHR structure indicating the new mappings.

This command sets the input attachment index mappings for subsequent drawing commands, and **must** match the mappings provided to the bound pipeline, if one is bound, which **can** be set by chaining VkRenderingInputAttachmentIndexInfoKHR to VkGraphicsPipelineCreateInfo.

Until this command is called, mappings in the command buffer state are treated as each color attachment specified in vkCmdBeginRenderingKHR mapping to subpass inputs with a InputAttachmentIndex equal to its index in VkRenderingInfo::pColorAttachments, and depth/stencil attachments mapping to input attachments without these decorations. This state is reset whenever vkCmdBeginRenderingKHR is called.

Valid Usage

- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-dynamicRenderingLocalRead-09516
  `dynamicRenderingLocalRead` **must** be enabled
- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-pInputAttachmentIndexInfo-09517
  `pInputAttachmentIndexInfo->colorAttachmentCount` **must** be equal to the value of VkRenderingInfo::`colorAttachmentCount` used to begin the current render pass instance
- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-commandBuffer-09518
  The current render pass instance **must** have been started or resumed by vkCmdBeginRenderingKHR in this `commandBuffer`

Valid Usage (Implicit)

- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-pInputAttachmentIndexInfo-parameter
  `pInputAttachmentIndexInfo` **must** be a valid pointer to a valid VkRenderingInputAttachmentIndexInfoKHR structure
- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdSetRenderingInputAttachmentIndicesKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	State

The VkRenderingInputAttachmentIndexInfoKHR structure is defined as:

```
// Provided by VK_KHR_dynamic_rendering_local_read
typedef struct VkRenderingInputAttachmentIndexInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           colorAttachmentCount;
    const uint32_t*    pColorAttachmentInputIndices;
    const uint32_t*    pDepthInputAttachmentIndex;
    const uint32_t*    pStencilInputAttachmentIndex;
} VkRenderingInputAttachmentIndexInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- colorAttachmentCount is the number of elements in pColorAttachmentInputIndices.
- pColorAttachmentInputIndices is a pointer to an array of colorAttachmentCount uint32_t values defining indices for color attachments to be used as input attachments.
- pDepthInputAttachmentIndex is either NULL, or a pointer to a uint32_t value defining the index for the depth attachment to be used as an input attachment.
- pStencilInputAttachmentIndex is either NULL, or a pointer to a uint32_t value defining the index for the stencil attachment to be used as an input attachment.

This structure allows applications to remap attachments to different input attachment indices.

Each element of pColorAttachmentInputIndices set to a value of VK_ATTACHMENT_UNUSED indicates that the corresponding attachment will not be used as an input attachment in this pipeline. Any other value in each of those elements will map the corresponding attachment to a InputAttachmentIndex value defined in shader code.

If pColorAttachmentInputIndices is NULL, it is equivalent to setting each element to its index within the array.

If pDepthInputAttachmentIndex or pStencilInputAttachmentIndex are set to NULL, they map to input attachments without a InputAttachmentIndex decoration. If they point to a value of VK_ATTACHMENT_UNUSED, it indicates that the corresponding attachment will not be used as an input attachment in this pipeline. If they point to any other value it maps the corresponding attachment to a InputAttachmentIndex value defined in shader code.

This structure **can** be included in the pNext chain of a VkGraphicsPipelineCreateInfo structure to set this state for a pipeline. If this structure is not included in the pNext chain of VkGraphicsPipelineCreateInfo, it is equivalent to specifying this structure with the following properties:

- colorAttachmentCount set to VkPipelineRenderingCreateInfo::colorAttachmentCount.
- pColorAttachmentInputIndices set to NULL.
- pDepthInputAttachmentIndex set to NULL.
- pStencilInputAttachmentIndex set to NULL.

This structure **can** be included in the pNext chain of a VkCommandBufferInheritanceInfo structure to specify inherited state from the primary command buffer. If this structure is not included in the pNext chain of VkCommandBufferInheritanceInfo, it is equivalent to specifying this structure with the following properties:

- colorAttachmentCount set to VkCommandBufferInheritanceRenderingInfo::colorAttachmentCount.
- pColorAttachmentInputIndices set to NULL.
- pDepthInputAttachmentIndex set to NULL.
- pStencilInputAttachmentIndex set to NULL.

Valid Usage

- VUID-VkRenderingInputAttachmentIndexInfoKHR-dynamicRenderingLocalRead-09519
  If the dynamicRenderingLocalRead feature is not enabled, and pColorAttachmentInputIndices is not NULL, each element **must** be VK_ATTACHMENT_UNUSED
- VUID-VkRenderingInputAttachmentIndexInfoKHR-dynamicRenderingLocalRead-09520
  If the dynamicRenderingLocalRead feature is not enabled, pDepthInputAttachmentIndex **must** be a valid pointer to a value of VK_ATTACHMENT_UNUSED
- VUID-VkRenderingInputAttachmentIndexInfoKHR-dynamicRenderingLocalRead-09521
  If the dynamicRenderingLocalRead feature is not enabled, pStencilInputAttachmentIndex **must** be a valid pointer to a value of VK_ATTACHMENT_UNUSED
- VUID-VkRenderingInputAttachmentIndexInfoKHR-pColorAttachmentInputIndices-09522
  Elements of pColorAttachmentInputIndices that are not VK_ATTACHMENT_UNUSED **must** each be unique
- VUID-VkRenderingInputAttachmentIndexInfoKHR-pColorAttachmentInputIndices-09523
  Elements of pColorAttachmentInputIndices that are not VK_ATTACHMENT_UNUSED **must** not take the same value as the content of pDepthInputAttachmentIndex
- VUID-VkRenderingInputAttachmentIndexInfoKHR-pColorAttachmentInputIndices-09524
  Elements of pColorAttachmentInputIndices that are not VK_ATTACHMENT_UNUSED **must** not take the same value as the content of pStencilInputAttachmentIndex
- VUID-VkRenderingInputAttachmentIndexInfoKHR-colorAttachmentCount-09525
  colorAttachmentCount **must** be less than or equal to maxColorAttachments

Valid Usage (Implicit)

- VUID-VkRenderingInputAttachmentIndexInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_RENDERING_INPUT_ATTACHMENT_INDEX_INFO_KHR`
- VUID-VkRenderingInputAttachmentIndexInfoKHR-pColorAttachmentInputIndices-parameter
  If `colorAttachmentCount` is not `0`, and `pColorAttachmentInputIndices` is not `NULL`, `pColorAttachmentInputIndices` **must** be a valid pointer to an array of `colorAttachmentCount` `uint32_t` values
- VUID-VkRenderingInputAttachmentIndexInfoKHR-pDepthInputAttachmentIndex-parameter
  If `pDepthInputAttachmentIndex` is not `NULL`, `pDepthInputAttachmentIndex` **must** be a valid pointer to a valid `uint32_t` value
- VUID-VkRenderingInputAttachmentIndexInfoKHR-pStencilInputAttachmentIndex-parameter
  If `pStencilInputAttachmentIndex` is not `NULL`, `pStencilInputAttachmentIndex` **must** be a valid pointer to a valid `uint32_t` value

Variables identified with the `InputAttachmentIndex` **must** only be used by a fragment stage. The numeric format of the subpass input **must** match the format of the corresponding input attachment, or the values of subpass loads from these variables are undefined. If the framebuffer attachment contains both depth and stencil aspects, the numeric format of the subpass input determines if depth or stencil aspect is accessed by the shader.

See Input Attachment for more details.

15.5.1. Fragment Input Attachment Compatibility

An input attachment that is statically accessed by a fragment shader **must** be backed by a descriptor that is equivalent to the VkImageView in the VkFramebuffer, except for `subresourceRange.aspectMask`. The `aspectMask` **must** be equal to the aspect accessed by the shader.

15.6. Ray Tracing Pipeline Interface

Ray tracing pipelines **may** have more stages than other pipelines with multiple instances of each stage and more dynamic interactions between the stages, but still have interface structures that obey the same general rules as interfaces between shader stages in other pipelines. The three types of inter-stage interface variables for ray tracing pipelines are:

- Ray payloads containing data tracked for the entire lifetime of the ray.
- Hit attributes containing data about a specific hit for the duration of its processing.
- Callable data for passing data into and out of a callable shader.

Ray payloads and callable data are used in explicit shader call instructions, so they have an incoming variant to distinguish the parameter passed to the invocation from any other payloads or

data being used by subsequent shader call instructions.

An interface structure used between stages **must** match between the stages using it. Specifically:

- The hit attribute structure read in an any-hit or closest hit shader **must** be the same structure as the hit attribute structure written in the corresponding intersection shader in the same hit group.
- The incoming callable data for a callable shader **must** be the same structure as the callable data referenced by the execute callable instruction in the calling shader.
- The ray payload for a shader invoked by a ray tracing command **must** be the same structure for all shader stages using the payload for that ray.

Any shader with an incoming ray payload, incoming callable data, or hit attribute **must** only declare one variable of that type.

Table 16. Ray Pipeline Shader Interface

Shader Stage	Ray Payload	Incoming Ray Payload	Hit Attribute	Callable Data	Incoming Callable Data
Ray Generation	r/w			r/w	
Intersection			r/w		
Any-Hit		r/w	r		
Closest Hit	r/w	r/w	r	r/w	
Miss	r/w	r/w		r/w	
Callable				r/w	r/w

15.7. Shader Resource Interface

When a shader stage accesses buffer or image resources, as described in the Resource Descriptors section, the shader resource variables **must** be matched with the pipeline layout that is provided at pipeline creation time.

The set of shader variables that form the *shader resource interface* for a stage are the variables statically used by that stage's `OpEntryPoint` with a storage class of `Uniform`, `UniformConstant`, `StorageBuffer`, or `PushConstant`. For the fragment shader, this includes the fragment input attachment interface.

The shader resource interface consists of two sub-interfaces: the push constant interface and the descriptor set interface.

15.7.1. Push Constant Interface

The shader variables defined with a storage class of `PushConstant` that are statically used by the shader entry points for the pipeline define the *push constant interface*. They **must** be:

- typed as `OpTypeStruct`,
- identified with a `Block` decoration, and
- laid out explicitly using the `Offset`, `ArrayStride`, and `MatrixStride` decorations as specified in Offset and Stride Assignment.

There **must** be no more than one push constant block statically used per shader entry point.

Each statically used member of a push constant block **must** be placed at an `Offset` such that the entire member is entirely contained within the VkPushConstantRange for each `OpEntryPoint` that uses it, and the `stageFlags` for that range **must** specify the appropriate VkShaderStageFlagBits for that stage. The `Offset` decoration for any member of a push constant block **must** not cause the space required for that member to extend outside the range [0, `maxPushConstantsSize`).

Any member of a push constant block that is declared as an array **must** only be accessed with *dynamically uniform* indices.

15.7.2. Descriptor Set Interface

The *descriptor set interface* is comprised of the shader variables with the storage class of `StorageBuffer`, `Uniform` or `UniformConstant` (including the variables in the fragment input attachment interface) that are statically used by the shader entry points for the pipeline.

These variables **must** have `DescriptorSet` and `Binding` decorations specified, which are assigned and matched with the `VkDescriptorSetLayout` objects in the pipeline layout as described in DescriptorSet and Binding Assignment.

The `Image Format` of an `OpTypeImage` declaration **must** not be **Unknown**, for variables which are used for `OpImageRead`, `OpImageSparseRead`, or `OpImageWrite` operations, except under the following conditions:

- For `OpImageWrite`, if the image format is listed in the storage without format list and if the `shaderStorageImageWriteWithoutFormat` feature is enabled and the shader module declares the `StorageImageWriteWithoutFormat` capability.
- For `OpImageWrite`, if the image format supports `VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT` and the shader module declares the `StorageImageWriteWithoutFormat` capability.
- For `OpImageRead` or `OpImageSparseRead`, if the image format is listed in the storage without format list and if the `shaderStorageImageReadWithoutFormat` feature is enabled and the shader module declares the `StorageImageReadWithoutFormat` capability.
- For `OpImageRead` or `OpImageSparseRead`, if the image format supports `VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT` and the shader module declares the `StorageImageReadWithoutFormat` capability.
- For `OpImageRead`, if `Dim` is `SubpassData` (indicating a read from an input attachment).

The `Image Format` of an `OpTypeImage` declaration **must** not be **Unknown**, for variables which are used for `OpAtomic*` operations.

Variables identified with the Uniform storage class are used to access transparent buffer backed resources. Such variables **must** be:

- typed as OpTypeStruct, or an array of this type,
- identified with a Block or BufferBlock decoration, and
- laid out explicitly using the Offset, ArrayStride, and MatrixStride decorations as specified in Offset and Stride Assignment.

Variables identified with the StorageBuffer storage class are used to access transparent buffer backed resources. Such variables **must** be:

- typed as OpTypeStruct, or an array of this type,
- identified with a Block decoration, and
- laid out explicitly using the Offset, ArrayStride, and MatrixStride decorations as specified in Offset and Stride Assignment.

The Offset decoration for any member of a Block-decorated variable in the Uniform storage class **must** not cause the space required for that variable to extend outside the range [0, maxUniformBufferRange). The Offset decoration for any member of a Block-decorated variable in the StorageBuffer storage class **must** not cause the space required for that variable to extend outside the range [0, maxStorageBufferRange).

Variables identified with a storage class of UniformConstant and a decoration of InputAttachmentIndex **must** be declared as described in Fragment Input Attachment Interface.

SPIR-V variables decorated with a descriptor set and binding that identify a combined image sampler descriptor **can** have a type of OpTypeImage, OpTypeSampler (Sampled=1), or OpTypeSampledImage.

When accessing a resource through such a variable, the resource **must** be selected via compile time constant expressions unless features are enabled to allow dynamically uniform or non-uniform expressions, as described below:

- Storage images (except storage texel buffers and input attachments):
  - Dynamically uniform: shaderStorageImageArrayDynamicIndexing and StorageImageArrayDynamicIndexing
- Sampled images (except uniform texel buffers), samplers and combined image samplers:
  - Dynamically uniform: shaderSampledImageArrayDynamicIndexing and SampledImageArrayDynamicIndexing
- Uniform buffers:
  - Dynamically uniform: shaderUniformBufferArrayDynamicIndexing and UniformBufferArrayDynamicIndexing
- Storage buffers:
  - Dynamically uniform: shaderStorageBufferArrayDynamicIndexing and StorageBufferArrayDynamicIndexing
- Acceleration structures:

- Dynamically uniform: Always supported.
- Non-uniform: Always supported.

A combined image sampler in an array that enables sampler $Y'C_BC_R$ conversion or samples a subsampled image **must** only be indexed by constant integral expressions.

Table 17. Shader Resource and Descriptor Type Correspondence

Resource type	Descriptor Type
sampler	VK_DESCRIPTOR_TYPE_SAMPLER or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER
sampled image	VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER
storage image	VK_DESCRIPTOR_TYPE_STORAGE_IMAGE
combined image sampler	VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER
uniform texel buffer	VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER
storage texel buffer	VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER
uniform buffer	VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER or VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC
storage buffer	VK_DESCRIPTOR_TYPE_STORAGE_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC
input attachment	VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT
acceleration structure	VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR

Table 18. Shader Resource and Storage Class Correspondence

Resource type	Storage Class	Type[1]	Decoration(s)[2]
sampler	UniformConstant	OpTypeSampler	
sampled image	UniformConstant	OpTypeImage (Sampled=1)	
storage image	UniformConstant	OpTypeImage (Sampled=2)	
combined image sampler	UniformConstant	OpTypeSampledImage OpTypeImage (Sampled=1) OpTypeSampler	
uniform texel buffer	UniformConstant	OpTypeImage (Dim=Buffer, Sampled=1)	
storage texel buffer	UniformConstant	OpTypeImage (Dim=Buffer, Sampled=2)	
uniform buffer	Uniform	OpTypeStruct	Block, Offset, (ArrayStride), (MatrixStride)

Resource type	Storage Class	Type[1]	Decoration(s)[2]
storage buffer	Uniform	OpTypeStruct	BufferBlock, Offset, (ArrayStride), (MatrixStride)
	StorageBuffer		Block, Offset, (ArrayStride), (MatrixStride)
input attachment	UniformConstant	OpTypeImage (Dim =SubpassData, Sampled=2)	InputAttachmentIndex
acceleration structure	UniformConstant	OpTypeAccelerationStructureKHR	

1

Where OpTypeImage is referenced, the Dim values Buffer and Subpassdata are only accepted where they are specifically referenced. They do not correspond to resource types where a generic OpTypeImage is specified.

2

In addition to DescriptorSet and Binding.

15.7.3. DescriptorSet and Binding Assignment

A variable decorated with a DescriptorSet decoration of s and a Binding decoration of b indicates that this variable is associated with the VkDescriptorSetLayoutBinding that has a binding equal to b in pSetLayouts[*s*] that was specified in VkPipelineLayoutCreateInfo.

DescriptorSet decoration values **must** be between zero and maxBoundDescriptorSets minus one, inclusive. Binding decoration values **can** be any 32-bit unsigned integer value, as described in Descriptor Set Layout. Each descriptor set has its own binding name space.

If the Binding decoration is used with an array, the entire array is assigned that binding value. The array **must** be a single-dimensional array and size of the array **must** be no larger than the number of descriptors in the binding. The array **must** not be runtime-sized. The index of each element of the array is referred to as the *arrayElement*. For the purposes of interface matching and descriptor set operations, if a resource variable is not an array, it is treated as if it has an arrayElement of zero.

There is a limit on the number of resources of each type that **can** be accessed by a pipeline stage as shown in Shader Resource Limits. The “Resources Per Stage” column gives the limit on the number each type of resource that **can** be statically used for an entry point in any given stage in a pipeline. The “Resource Types” column lists which resource types are counted against the limit. Some resource types count against multiple limits.

The pipeline layout **may** include descriptor sets and bindings which are not referenced by any variables statically used by the entry points for the shader stages in the binding’s stageFlags.

However, if a variable assigned to a given DescriptorSet and Binding is statically used by the entry point for a shader stage, the pipeline layout **must** contain a descriptor set layout binding in that descriptor set layout and for that binding number, and that binding’s stageFlags **must** include the appropriate VkShaderStageFlagBits for that stage. The variable **must** be of a valid resource type

determined by its SPIR-V type and storage class, as defined in Shader Resource and Storage Class Correspondence. The descriptor set layout binding **must** be of a corresponding descriptor type, as defined in Shader Resource and Descriptor Type Correspondence.

NOTE

There are no limits on the number of shader variables that can have overlapping set and binding values in a shader; but which resources are statically used has an impact. If any shader variable identifying a resource is statically used in a shader, then the underlying descriptor bound at the declared set and binding **must** support the declared type in the shader when the shader executes.

If multiple shader variables are declared with the same set and binding values, and with the same underlying descriptor type, they can all be statically used within the same shader. However, accesses are not automatically synchronized, and `Aliased` decorations should be used to avoid data hazards (see section 2.18.2 Aliasing in the SPIR-V specification).

If multiple shader variables with the same set and binding values are declared in a single shader, but with different declared types, where any of those are not supported by the relevant bound descriptor, that shader can only be executed if the variables with the unsupported type are not statically used.

A noteworthy example of using multiple statically-used shader variables sharing the same descriptor set and binding values is a descriptor of type `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` that has multiple corresponding shader variables in the `UniformConstant` storage class, where some could be `OpTypeImage` (`Sampled`=1), some could be `OpTypeSampler`, and some could be `OpTypeSampledImage`.

Table 19. Shader Resource Limits

Resources per Stage	Resource Types
`maxPerStageDescriptorSamplers`	sampler
	combined image sampler
`maxPerStageDescriptorSampledImages`	sampled image
	combined image sampler
	uniform texel buffer
`maxPerStageDescriptorStorageImages`	storage image
	storage texel buffer
`maxPerStageDescriptorUniformBuffers`	uniform buffer
	uniform buffer dynamic
`maxPerStageDescriptorStorageBuffers`	storage buffer
	storage buffer dynamic
`maxPerStageDescriptorInputAttachments`	input attachment[1]

Resources per Stage	Resource Types
`maxPerStageDescriptorAccelerationStructures` or `maxPerStageDescriptorUpdateAfterBindAccelerationStructures`	acceleration structure

1

Input attachments **can** only be used in the fragment shader stage

15.7.4. Offset and Stride Assignment

Certain objects **must** be explicitly laid out using the `Offset`, `ArrayStride`, and `MatrixStride`, as described in SPIR-V explicit layout validation rules. All such layouts also **must** conform to the following requirements.

NOTE	The numeric order of `Offset` decorations does not need to follow member declaration order.

Alignment Requirements

There are different alignment requirements depending on the specific resources and on the features enabled.

Matrix types are defined in terms of arrays as follows:

- A column-major matrix with C columns and R rows is equivalent to a C element array of vectors with R components.
- A row-major matrix with C columns and R rows is equivalent to an R element array of vectors with C components.

The *scalar alignment* of the type of an `OpTypeStruct` member is defined recursively as follows:

- A scalar of size N has a scalar alignment of N.
- A vector type has a scalar alignment equal to that of its component type.
- An array type has a scalar alignment equal to that of its element type.
- A structure has a scalar alignment equal to the largest scalar alignment of any of its members.
- A matrix type inherits *scalar alignment* from the equivalent array declaration.

The *base alignment* of the type of an `OpTypeStruct` member is defined recursively as follows:

- A scalar has a base alignment equal to its scalar alignment.
- A two-component vector has a base alignment equal to twice its scalar alignment.
- A three- or four-component vector has a base alignment equal to four times its scalar alignment.
- An array has a base alignment equal to the base alignment of its element type.
- A structure has a base alignment equal to the largest base alignment of any of its members. An empty structure has a base alignment equal to the size of the smallest scalar type permitted by

the capabilities declared in the SPIR-V module. (e.g., for a 1 byte aligned empty struct in the `StorageBuffer` storage class, `StorageBuffer8BitAccess` or `UniformAndStorageBuffer8BitAccess` **must** be declared in the SPIR-V module.)

- A matrix type inherits *base alignment* from the equivalent array declaration.

The *extended alignment* of the type of an `OpTypeStruct` member is similarly defined as follows:

- A scalar or vector type has an extended alignment equal to its base alignment.
- An array or structure type has an extended alignment equal to the largest extended alignment of any of its members, rounded up to a multiple of 16.
- A matrix type inherits extended alignment from the equivalent array declaration.

A member is defined to *improperly straddle* if either of the following are true:

- It is a vector with total size less than or equal to 16 bytes, and has `Offset` decorations placing its first byte at F and its last byte at L, where floor(F / 16) != floor(L / 16).
- It is a vector with total size greater than 16 bytes and has its `Offset` decorations placing its first byte at a non-integer multiple of 16.

Standard Buffer Layout

Every member of an `OpTypeStruct` that is required to be explicitly laid out **must** be aligned according to the first matching rule as follows. If the struct is contained in pointer types of multiple storage classes, it **must** satisfy the requirements for every storage class used to reference it.

1. If the `workgroupMemoryExplicitLayoutScalarBlockLayout` feature is enabled and the storage class is `Workgroup` then every member **must** be aligned according to its scalar alignment.
2. All vectors **must** be aligned according to their scalar alignment.
3. If the `uniformBufferStandardLayout` feature is not enabled, then any member of an `OpTypeStruct` with a storage class of `Uniform` and a decoration of `Block` **must** be aligned according to its extended alignment.
4. Every other member **must** be aligned according to its base alignment.

The memory layout **must** obey the following rules:

- The `Offset` decoration of any member **must** be a multiple of its alignment.
- Any `ArrayStride` or `MatrixStride` decoration **must** be a multiple of the alignment of the array or matrix as defined above.

If one of the conditions below applies

- The storage class is `Workgroup`, and either the struct member is not part of a `Block` or the `workgroupMemoryExplicitLayoutScalarBlockLayout` feature is not enabled.
- The storage class is any other storage class.

the memory layout **must** also obey the following rules:

- Vectors **must** not improperly straddle, as defined above.
- The `Offset` decoration of a member **must** not place it between the end of a structure, an array or a matrix and the next multiple of the alignment of that structure, array or matrix.

NOTE	The **std430 layout** in GLSL satisfies these rules for types using the base alignment. The **std140 layout** satisfies the rules for types using the extended alignment.

15.8. Built-In Variables

Built-in variables are accessed in shaders by declaring a variable decorated with a `BuiltIn` SPIR-V decoration. The meaning of each `BuiltIn` decoration is as follows. In the remainder of this section, the name of a built-in is used interchangeably with a term equivalent to a variable decorated with that particular built-in. Built-ins that represent integer values **can** be declared as either signed or unsigned 32-bit integers.

As mentioned above, some inputs and outputs have an additional level of arrayness relative to other shader inputs and outputs. This level of arrayness is not included in the type descriptions below, but **must** be included when declaring the built-in.

Any two `Input` storage class `OpVariable` declarations listed as operands on the same `OpEntryPoint` **must** not have the same `BuiltIn` decoration. Any two `Output` storage class `OpVariable` declarations listed as operands on the same `OpEntryPoint` **must** not have the same `BuiltIn` decoration.

`BaryCoordKHR`

The `BaryCoordKHR` decoration **can** be used to decorate a fragment shader input variable. This variable will contain a three-component floating-point vector with barycentric weights that indicate the location of the fragment relative to the screen-space locations of vertices of its primitive, obtained using perspective interpolation.

Valid Usage

- VUID-BaryCoordKHR-BaryCoordKHR-04154
  The `BaryCoordKHR` decoration **must** be used only within the `Fragment Execution Model`
- VUID-BaryCoordKHR-BaryCoordKHR-04155
  The variable decorated with `BaryCoordKHR` **must** be declared using the `Input Storage Class`
- VUID-BaryCoordKHR-BaryCoordKHR-04156
  The variable decorated with `BaryCoordKHR` **must** be declared as a three-component vector of 32-bit floating-point values

`BaryCoordNoPerspKHR`

The `BaryCoordNoPerspKHR` decoration **can** be used to decorate a fragment shader input variable. This variable will contain a three-component floating-point vector with barycentric weights that indicate the location of the fragment relative to the screen-space locations of vertices of its primitive, obtained using linear interpolation.

Valid Usage

- VUID-BaryCoordNoPerspKHR-BaryCoordNoPerspKHR-04160
  The `BaryCoordNoPerspKHR` decoration **must** be used only within the `Fragment` `Execution` `Model`
- VUID-BaryCoordNoPerspKHR-BaryCoordNoPerspKHR-04161
  The variable decorated with `BaryCoordNoPerspKHR` **must** be declared using the `Input` `Storage` `Class`
- VUID-BaryCoordNoPerspKHR-BaryCoordNoPerspKHR-04162
  The variable decorated with `BaryCoordNoPerspKHR` **must** be declared as a three-component vector of 32-bit floating-point values

`BaseInstance`

Decorating a variable with the `BaseInstance` built-in will make that variable contain the integer value corresponding to the first instance that was passed to the command that invoked the current vertex shader invocation. `BaseInstance` is the `firstInstance` parameter to a *direct drawing command* or the `firstInstance` member of a structure consumed by an *indirect drawing command*.

Valid Usage

- VUID-BaseInstance-BaseInstance-04181
  The `BaseInstance` decoration **must** be used only within the `Vertex` `Execution` `Model`
- VUID-BaseInstance-BaseInstance-04182
  The variable decorated with `BaseInstance` **must** be declared using the `Input` `Storage` `Class`
- VUID-BaseInstance-BaseInstance-04183
  The variable decorated with `BaseInstance` **must** be declared as a scalar 32-bit integer value

`BaseVertex`

Decorating a variable with the `BaseVertex` built-in will make that variable contain the integer value corresponding to the first vertex or vertex offset that was passed to the command that invoked the current vertex shader invocation. For *non-indexed drawing commands*, this variable is the `firstVertex` parameter to a *direct drawing command* or the `firstVertex` member of the structure consumed by an *indirect drawing command*. For *indexed drawing commands*, this variable is the `vertexOffset` parameter to a *direct drawing command* or the `vertexOffset` member of the structure consumed by an *indirect drawing command*.

Valid Usage

- VUID-BaseVertex-BaseVertex-04184
  The `BaseVertex` decoration **must** be used only within the `Vertex` `Execution` `Model`
- VUID-BaseVertex-BaseVertex-04185
  The variable decorated with `BaseVertex` **must** be declared using the `Input` `Storage` `Class`

- VUID-BaseVertex-BaseVertex-04186
  The variable decorated with `BaseVertex` **must** be declared as a scalar 32-bit integer value

`ClipDistance`

Decorating a variable with the `ClipDistance` built-in decoration will make that variable contain the mechanism for controlling user clipping. `ClipDistance` is an array such that the i^{th} element of the array specifies the clip distance for plane i. A clip distance of 0 means the vertex is on the plane, a positive distance means the vertex is inside the clip half-space, and a negative distance means the vertex is outside the clip half-space.

NOTE The array variable decorated with `ClipDistance` is explicitly sized by the shader.

NOTE In the last pre-rasterization shader stage, these values will be linearly interpolated across the primitive and the portion of the primitive with interpolated distances less than 0 will be considered outside the clip volume. If `ClipDistance` is then used by a fragment shader, `ClipDistance` contains these linearly interpolated values.

Valid Usage

- VUID-ClipDistance-ClipDistance-04187
  The `ClipDistance` decoration **must** be used only within the `MeshEXT`, `MeshNV`, `Vertex`, `Fragment`, `TessellationControl`, `TessellationEvaluation`, or `Geometry` `Execution Model`
- VUID-ClipDistance-ClipDistance-04188
  The variable decorated with `ClipDistance` within the `MeshEXT`, `MeshNV`, or `Vertex` `Execution Model` **must** be declared using the `Output` `Storage Class`
- VUID-ClipDistance-ClipDistance-04189
  The variable decorated with `ClipDistance` within the `Fragment` `Execution Model` **must** be declared using the `Input` `Storage Class`
- VUID-ClipDistance-ClipDistance-04190
  The variable decorated with `ClipDistance` within the `TessellationControl`, `TessellationEvaluation`, or `Geometry` `Execution Model` **must** not be declared in a `Storage Class` other than `Input` or `Output`
- VUID-ClipDistance-ClipDistance-04191
  The variable decorated with `ClipDistance` **must** be declared as an array of 32-bit floating-point values

`CullDistance`

Decorating a variable with the `CullDistance` built-in decoration will make that variable contain the mechanism for controlling user culling. If any member of this array is assigned a negative value for all vertices belonging to a primitive, then the primitive is discarded before rasterization.

NOTE In fragment shaders, the values of the `CullDistance` array are linearly interpolated

across each primitive.

NOTE If `CullDistance` decorates an input variable, that variable will contain the corresponding value from the `CullDistance` decorated output variable from the previous shader stage.

Valid Usage

- VUID-CullDistance-CullDistance-04196
  The `CullDistance` decoration **must** be used only within the `MeshEXT`, `MeshNV`, `Vertex`, `Fragment`, `TessellationControl`, `TessellationEvaluation`, or `Geometry` `Execution Model`
- VUID-CullDistance-CullDistance-04197
  The variable decorated with `CullDistance` within the `MeshEXT`, `MeshNV` or `Vertex` `Execution Model` **must** be declared using the `Output` `Storage Class`
- VUID-CullDistance-CullDistance-04198
  The variable decorated with `CullDistance` within the `Fragment` `Execution Model` **must** be declared using the `Input` `Storage Class`
- VUID-CullDistance-CullDistance-04199
  The variable decorated with `CullDistance` within the `TessellationControl`, `TessellationEvaluation`, or `Geometry` `Execution Model` **must** not be declared using a `Storage Class` other than `Input` or `Output`
- VUID-CullDistance-CullDistance-04200
  The variable decorated with `CullDistance` **must** be declared as an array of 32-bit floating-point values

`CullMaskKHR`

A variable decorated with the `CullMaskKHR` decoration will specify the cull mask of the ray being processed. The value is given by the `Cull Mask` parameter passed into one of the `OpTrace*` instructions.

Valid Usage

- VUID-CullMaskKHR-CullMaskKHR-06735
  The `CullMaskKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, or `MissKHR` `Execution Model`
- VUID-CullMaskKHR-CullMaskKHR-06736
  The variable decorated with `CullMaskKHR` **must** be declared using the `Input` `Storage Class`
- VUID-CullMaskKHR-CullMaskKHR-06737
  The variable decorated with `CullMaskKHR` **must** be declared as a scalar 32-bit integer value

`DeviceIndex`

The `DeviceIndex` decoration **can** be applied to a shader input which will be filled with the device index of the physical device that is executing the current shader invocation. This value will be in

the range $[0, max(1, physicalDeviceCount))$, where physicalDeviceCount is the `physicalDeviceCount` member of VkDeviceGroupDeviceCreateInfo.

Valid Usage

- VUID-DeviceIndex-DeviceIndex-04205
  The variable decorated with `DeviceIndex` **must** be declared using the `Input` `Storage` `Class`
- VUID-DeviceIndex-DeviceIndex-04206
  The variable decorated with `DeviceIndex` **must** be declared as a scalar 32-bit integer value

`DrawIndex`

Decorating a variable with the `DrawIndex` built-in will make that variable contain the integer value corresponding to the zero-based index of the draw that invoked the current vertex shader invocation. For *indirect drawing commands*, `DrawIndex` begins at zero and increments by one for each draw executed. The number of draws is given by the `drawCount` parameter. For *direct drawing commands*, if vkCmdDrawMultiEXT or vkCmdDrawMultiIndexedEXT is used, this variable contains the integer value corresponding to the zero-based index of the draw. Otherwise `DrawIndex` is always zero. `DrawIndex` is dynamically uniform.

Valid Usage

- VUID-DrawIndex-DrawIndex-04207
  The `DrawIndex` decoration **must** be used only within the `Vertex`, `MeshEXT`, `TaskEXT`, `MeshNV`, or `TaskNV` `Execution` `Model`
- VUID-DrawIndex-DrawIndex-04208
  The variable decorated with `DrawIndex` **must** be declared using the `Input` `Storage` `Class`
- VUID-DrawIndex-DrawIndex-04209
  The variable decorated with `DrawIndex` **must** be declared as a scalar 32-bit integer value

`FragCoord`

Decorating a variable with the `FragCoord` built-in decoration will make that variable contain the coordinates (x,y,z,1/w) of the fragment being processed.

The (x,y) values are the framebuffer coordinates ($x_{f,y}f$) of the fragment.

When Sample Shading is enabled, the x and y components of `FragCoord` reflect the location of one of the samples corresponding to the shader invocation.

Otherwise, the x and y components of `FragCoord` reflect the location of the center of the fragment.

The z component of `FragCoord` is the interpolated depth value of the primitive.

The w component is the interpolated $\frac{1}{w}$.

The `Centroid` interpolation decoration is ignored, but allowed, on `FragCoord`.

Valid Usage

- VUID-FragCoord-FragCoord-04210
  The `FragCoord` decoration **must** be used only within the `Fragment Execution Model`
- VUID-FragCoord-FragCoord-04211
  The variable decorated with `FragCoord` **must** be declared using the `Input Storage Class`
- VUID-FragCoord-FragCoord-04212
  The variable decorated with `FragCoord` **must** be declared as a four-component vector of 32-bit floating-point values

`FragDepth`

To have a shader supply a fragment-depth value, the shader **must** declare the `DepthReplacing` execution mode. Such a shader's fragment-depth value will come from the variable decorated with the `FragDepth` built-in decoration.

This value will be used for any subsequent depth testing performed by the implementation or writes to the depth attachment. See fragment shader depth replacement for details.

Valid Usage

- VUID-FragDepth-FragDepth-04213
  The `FragDepth` decoration **must** be used only within the `Fragment Execution Model`
- VUID-FragDepth-FragDepth-04214
  The variable decorated with `FragDepth` **must** be declared using the `Output Storage Class`
- VUID-FragDepth-FragDepth-04215
  The variable decorated with `FragDepth` **must** be declared as a scalar 32-bit floating-point value
- VUID-FragDepth-FragDepth-04216
  If the shader dynamically writes to the variable decorated with `FragDepth`, the `DepthReplacing Execution Mode` **must** be declared

`FragInvocationCountEXT`

Decorating a variable with the `FragInvocationCountEXT` built-in decoration will make that variable contain the maximum number of fragment shader invocations for the fragment, as determined by `minSampleShading`.

If Sample Shading is not enabled, `FragInvocationCountEXT` will be filled with a value of 1.

Valid Usage

- VUID-FragInvocationCountEXT-FragInvocationCountEXT-04217
  The `FragInvocationCountEXT` decoration **must** be used only within the `Fragment Execution Model`

- VUID-FragInvocationCountEXT-FragInvocationCountEXT-04218
  The variable decorated with `FragInvocationCountEXT` **must** be declared using the `Input` `Storage Class`
- VUID-FragInvocationCountEXT-FragInvocationCountEXT-04219
  The variable decorated with `FragInvocationCountEXT` **must** be declared as a scalar 32-bit integer value

`FragSizeEXT`

Decorating a variable with the `FragSizeEXT` built-in decoration will make that variable contain the dimensions in pixels of the area that the fragment covers for that invocation.

If fragment density map is not enabled, `FragSizeEXT` will be filled with a value of (1,1).

Valid Usage

- VUID-FragSizeEXT-FragSizeEXT-04220
  The `FragSizeEXT` decoration **must** be used only within the `Fragment` `Execution Model`
- VUID-FragSizeEXT-FragSizeEXT-04221
  The variable decorated with `FragSizeEXT` **must** be declared using the `Input` `Storage Class`
- VUID-FragSizeEXT-FragSizeEXT-04222
  The variable decorated with `FragSizeEXT` **must** be declared as a two-component vector of 32-bit integer values

`FragStencilRefEXT`

Decorating a variable with the `FragStencilRefEXT` built-in decoration will make that variable contain the new stencil reference value for all samples covered by the fragment. This value will be used as the stencil reference value used in stencil testing.

To write to `FragStencilRefEXT`, a shader **must** declare the `StencilRefReplacingEXT` execution mode. If a shader declares the `StencilRefReplacingEXT` execution mode and there is an execution path through the shader that does not set `FragStencilRefEXT`, then the fragment's stencil reference value is undefined for executions of the shader that take that path.

Only the least significant **s** bits of the integer value of the variable decorated with `FragStencilRefEXT` are considered for stencil testing, where **s** is the number of bits in the stencil framebuffer attachment, and higher order bits are discarded.

See fragment shader stencil reference replacement for more details.

Valid Usage

- VUID-FragStencilRefEXT-FragStencilRefEXT-04223
  The `FragStencilRefEXT` decoration **must** be used only within the `Fragment` `Execution Model`
- VUID-FragStencilRefEXT-FragStencilRefEXT-04224
  The variable decorated with `FragStencilRefEXT` **must** be declared using the `Output` `Storage`

Class

- VUID-FragStencilRefEXT-FragStencilRefEXT-04225
  The variable decorated with FragStencilRefEXT **must** be declared as a scalar integer value

FrontFacing

Decorating a variable with the FrontFacing built-in decoration will make that variable contain whether the fragment is front or back facing. This variable is non-zero if the current fragment is considered to be part of a front-facing polygon primitive or of a non-polygon primitive and is zero if the fragment is considered to be part of a back-facing polygon primitive.

Valid Usage

- VUID-FrontFacing-FrontFacing-04229
  The FrontFacing decoration **must** be used only within the Fragment Execution Model
- VUID-FrontFacing-FrontFacing-04230
  The variable decorated with FrontFacing **must** be declared using the Input Storage Class
- VUID-FrontFacing-FrontFacing-04231
  The variable decorated with FrontFacing **must** be declared as a boolean value

FullyCoveredEXT

Decorating a variable with the FullyCoveredEXT built-in decoration will make that variable indicate whether the fragment area is fully covered by the generating primitive. This variable is non-zero if conservative rasterization is enabled and the current fragment area is fully covered by the generating primitive, and is zero if the fragment is not covered or partially covered, or conservative rasterization is disabled.

Valid Usage

- VUID-FullyCoveredEXT-FullyCoveredEXT-04232
  The FullyCoveredEXT decoration **must** be used only within the Fragment Execution Model
- VUID-FullyCoveredEXT-FullyCoveredEXT-04233
  The variable decorated with FullyCoveredEXT **must** be declared using the Input Storage Class
- VUID-FullyCoveredEXT-FullyCoveredEXT-04234
  The variable decorated with FullyCoveredEXT **must** be declared as a boolean value
- VUID-FullyCoveredEXT-conservativeRasterizationPostDepthCoverage-04235
  If VkPhysicalDeviceConservativeRasterizationPropertiesEXT ::conservativeRasterizationPostDepthCoverage is not supported the PostDepthCoverage Execution Mode **must** not be declared, when a variable with the FullyCoveredEXT decoration is declared

GlobalInvocationId

Decorating a variable with the GlobalInvocationId built-in decoration will make that variable contain the location of the current invocation within the global workgroup. Each component is equal to the index of the local workgroup multiplied by the size of the local workgroup plus LocalInvocationId.

Valid Usage

- VUID-GlobalInvocationId-GlobalInvocationId-04236
  The GlobalInvocationId decoration **must** be used only within the GLCompute, MeshEXT, TaskEXT, MeshNV, or TaskNV Execution Model
- VUID-GlobalInvocationId-GlobalInvocationId-04237
  The variable decorated with GlobalInvocationId **must** be declared using the Input Storage Class
- VUID-GlobalInvocationId-GlobalInvocationId-04238
  The variable decorated with GlobalInvocationId **must** be declared as a three-component vector of 32-bit integer values

HelperInvocation

Decorating a variable with the HelperInvocation built-in decoration will make that variable contain whether the current invocation is a helper invocation. This variable is non-zero if the current fragment being shaded is a helper invocation and zero otherwise. A helper invocation is an invocation of the shader that is produced to satisfy internal requirements such as the generation of derivatives.

NOTE It is very likely that a helper invocation will have a value of SampleMask fragment shader input value that is zero.

Valid Usage

- VUID-HelperInvocation-HelperInvocation-04239
  The HelperInvocation decoration **must** be used only within the Fragment Execution Model
- VUID-HelperInvocation-HelperInvocation-04240
  The variable decorated with HelperInvocation **must** be declared using the Input Storage Class
- VUID-HelperInvocation-HelperInvocation-04241
  The variable decorated with HelperInvocation **must** be declared as a boolean value

HitKindKHR

A variable decorated with the HitKindKHR decoration will describe the intersection that triggered the execution of the current shader. The values are determined by the intersection shader. For user-defined intersection shaders this is the value that was passed to the “Hit Kind” operand of OpReportIntersectionKHR. For triangle intersection candidates, this will be one of HitKindFrontFacingTriangleKHR or HitKindBackFacingTriangleKHR.

Valid Usage

- VUID-HitKindKHR-HitKindKHR-04242
  The `HitKindKHR` decoration **must** be used only within the `AnyHitKHR` or `ClosestHitKHR` `Execution Model`
- VUID-HitKindKHR-HitKindKHR-04243
  The variable decorated with `HitKindKHR` **must** be declared using the `Input` `Storage Class`
- VUID-HitKindKHR-HitKindKHR-04244
  The variable decorated with `HitKindKHR` **must** be declared as a scalar 32-bit integer value

`HitTriangleVertexPositionsKHR`

A variable decorated with the `HitTriangleVertexPositionsKHR` decoration will specify the object space vertices of the triangle at the current intersection in application-provided order. The positions returned are transformed by the geometry transform, which is performed at standard floating-point precision, but without a specifically defined order of floating-point operations to perform the matrix multiplication.

Valid Usage

- VUID-HitTriangleVertexPositionsKHR-HitTriangleVertexPositionsKHR-08747
  The `HitTriangleVertexPositionsKHR` decoration **must** be used only within the `AnyHitKHR` or `ClosestHitKHR` `Execution Model`
- VUID-HitTriangleVertexPositionsKHR-HitTriangleVertexPositionsKHR-08748
  The variable decorated with `HitTriangleVertexPositionsKHR` **must** be declared using the `Input` `Storage Class`
- VUID-HitTriangleVertexPositionsKHR-HitTriangleVertexPositionsKHR-08749
  The variable decorated with `HitTriangleVertexPositionsKHR` **must** be declared as an array of three vectors of three 32-bit float values
- VUID-HitTriangleVertexPositionsKHR-HitTriangleVertexPositionsKHR-08750
  The variable decorated with `HitTriangleVertexPositionsKHR` **must** be used only if the value of `HitKindKHR` is `HitKindFrontFacingTriangleKHR` or `HitKindBackFacingTriangleKHR`
- VUID-HitTriangleVertexPositionsKHR-None-08751
  The acceleration structure corresponding to the current intersection **must** have been built with `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DATA_ACCESS_KHR`

`IncomingRayFlagsKHR`

A variable with the `IncomingRayFlagsKHR` decoration will contain the ray flags passed in to the trace call that invoked this particular shader. Setting pipeline flags on the ray tracing pipeline **must** not cause any corresponding flags to be set in variables with this decoration.

Valid Usage

- VUID-IncomingRayFlagsKHR-IncomingRayFlagsKHR-04248

The IncomingRayFlagsKHR decoration **must** be used only within the IntersectionKHR, AnyHitKHR, ClosestHitKHR, or MissKHR Execution Model

- VUID-IncomingRayFlagsKHR-IncomingRayFlagsKHR-04249
  The variable decorated with IncomingRayFlagsKHR **must** be declared using the Input Storage Class
- VUID-IncomingRayFlagsKHR-IncomingRayFlagsKHR-04250
  The variable decorated with IncomingRayFlagsKHR **must** be declared as a scalar 32-bit integer value

InstanceCustomIndexKHR

A variable decorated with the InstanceCustomIndexKHR decoration will contain the application-defined value of the instance that intersects the current ray. This variable contains the value that was specified in VkAccelerationStructureInstanceKHR::instanceCustomIndex for the current acceleration structure instance in the lower 24 bits and the upper 8 bits will be zero.

Valid Usage

- VUID-InstanceCustomIndexKHR-InstanceCustomIndexKHR-04251
  The InstanceCustomIndexKHR decoration **must** be used only within the IntersectionKHR, AnyHitKHR, or ClosestHitKHR Execution Model
- VUID-InstanceCustomIndexKHR-InstanceCustomIndexKHR-04252
  The variable decorated with InstanceCustomIndexKHR **must** be declared using the Input Storage Class
- VUID-InstanceCustomIndexKHR-InstanceCustomIndexKHR-04253
  The variable decorated with InstanceCustomIndexKHR **must** be declared as a scalar 32-bit integer value

InstanceId

Decorating a variable in an intersection, any-hit, or closest hit shader with the InstanceId decoration will make that variable contain the index of the instance that intersects the current ray.

Valid Usage

- VUID-InstanceId-InstanceId-04254
  The InstanceId decoration **must** be used only within the IntersectionKHR, AnyHitKHR, or ClosestHitKHR Execution Model
- VUID-InstanceId-InstanceId-04255
  The variable decorated with InstanceId **must** be declared using the Input Storage Class
- VUID-InstanceId-InstanceId-04256
  The variable decorated with InstanceId **must** be declared as a scalar 32-bit integer value

`InvocationId`

Decorating a variable with the `InvocationId` built-in decoration will make that variable contain the index of the current shader invocation in a geometry shader, or the index of the output patch vertex in a tessellation control shader.

In a geometry shader, the index of the current shader invocation ranges from zero to the number of instances declared in the shader minus one. If the instance count of the geometry shader is one or is not specified, then `InvocationId` will be zero.

Valid Usage

- VUID-InvocationId-InvocationId-04257
  The `InvocationId` decoration **must** be used only within the `TessellationControl` or `Geometry` `Execution Model`
- VUID-InvocationId-InvocationId-04258
  The variable decorated with `InvocationId` **must** be declared using the `Input` `Storage Class`
- VUID-InvocationId-InvocationId-04259
  The variable decorated with `InvocationId` **must** be declared as a scalar 32-bit integer value

`InstanceIndex`

Decorating a variable in a vertex shader with the `InstanceIndex` built-in decoration will make that variable contain the index of the instance that is being processed by the current vertex shader invocation. `InstanceIndex` begins at the `firstInstance` parameter to vkCmdDraw or vkCmdDrawIndexed or at the `firstInstance` member of a structure consumed by vkCmdDrawIndirect or vkCmdDrawIndexedIndirect.

Valid Usage

- VUID-InstanceIndex-InstanceIndex-04263
  The `InstanceIndex` decoration **must** be used only within the `Vertex` `Execution Model`
- VUID-InstanceIndex-InstanceIndex-04264
  The variable decorated with `InstanceIndex` **must** be declared using the `Input` `Storage Class`
- VUID-InstanceIndex-InstanceIndex-04265
  The variable decorated with `InstanceIndex` **must** be declared as a scalar 32-bit integer value

`LaunchIdKHR`

A variable decorated with the `LaunchIdKHR` decoration will specify the index of the work item being processed. One work item is generated for each of the `width` × `height` × `depth` items dispatched by a vkCmdTraceRaysKHR command. All shader invocations inherit the same value for variables decorated with `LaunchIdKHR`.

Valid Usage

- VUID-LaunchIdKHR-LaunchIdKHR-04266
  The `LaunchIdKHR` decoration **must** be used only within the `RayGenerationKHR`, `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, `MissKHR`, or `CallableKHR` `Execution Model`
- VUID-LaunchIdKHR-LaunchIdKHR-04267
  The variable decorated with `LaunchIdKHR` **must** be declared using the `Input` `Storage Class`
- VUID-LaunchIdKHR-LaunchIdKHR-04268
  The variable decorated with `LaunchIdKHR` **must** be declared as a three-component vector of 32-bit integer values

`LaunchSizeKHR`

A variable decorated with the `LaunchSizeKHR` decoration will contain the `width`, `height`, and `depth` dimensions passed to the vkCmdTraceRaysKHR command that initiated this shader execution. The `width` is in the first component, the `height` is in the second component, and the `depth` is in the third component.

Valid Usage

- VUID-LaunchSizeKHR-LaunchSizeKHR-04269
  The `LaunchSizeKHR` decoration **must** be used only within the `RayGenerationKHR`, `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, `MissKHR`, or `CallableKHR` `Execution Model`
- VUID-LaunchSizeKHR-LaunchSizeKHR-04270
  The variable decorated with `LaunchSizeKHR` **must** be declared using the `Input` `Storage Class`
- VUID-LaunchSizeKHR-LaunchSizeKHR-04271
  The variable decorated with `LaunchSizeKHR` **must** be declared as a three-component vector of 32-bit integer values

`Layer`

Decorating a variable with the `Layer` built-in decoration will make that variable contain the select layer of a multi-layer framebuffer attachment.

In a geometry shader, any variable decorated with `Layer` can be written with the framebuffer layer index to which the primitive produced by that shader will be directed.

If the last active pre-rasterization shader stage shader entry point's interface does not include a variable decorated with `Layer`, then the first layer is used. If a pre-rasterization shader stage shader entry point's interface includes a variable decorated with `Layer`, it **must** write the same value to `Layer` for all output vertices of a given primitive. If the `Layer` value is less than 0 or greater than or equal to the number of layers in the framebuffer, then primitives **may** still be rasterized, fragment shaders **may** be executed, and the framebuffer values for all layers are undefined.

In a fragment shader, a variable decorated with `Layer` contains the layer index of the primitive

that the fragment invocation belongs to.

Valid Usage

- VUID-Layer-Layer-04272
  The `Layer` decoration **must** be used only within the `MeshEXT`, `MeshNV`, `Vertex`, `TessellationEvaluation`, `Geometry`, or `Fragment` `Execution Model`
- VUID-Layer-Layer-04274
  The variable decorated with `Layer` within the `MeshEXT`, `MeshNV`, `Vertex`, `TessellationEvaluation`, or `Geometry` `Execution Model` **must** be declared using the `Output` `Storage Class`
- VUID-Layer-Layer-04275
  The variable decorated with `Layer` within the `Fragment` `Execution Model` **must** be declared using the `Input` `Storage Class`
- VUID-Layer-Layer-04276
  The variable decorated with `Layer` **must** be declared as a scalar 32-bit integer value
- VUID-Layer-Layer-07039
  The variable decorated with `Layer` within the `MeshEXT` `Execution Model` **must** also be decorated with the `PerPrimitiveEXT` decoration

`LocalInvocationId`

Decorating a variable with the `LocalInvocationId` built-in decoration will make that variable contain the location of the current compute shader invocation within the local workgroup. Each component ranges from zero through to the size of the workgroup in that dimension minus one.

NOTE If the size of the workgroup in a particular dimension is one, then the `LocalInvocationId` in that dimension will be zero. If the workgroup is effectively two-dimensional, then `LocalInvocationId.z` will be zero. If the workgroup is effectively one-dimensional, then both `LocalInvocationId.y` and `LocalInvocationId.z` will be zero.

Valid Usage

- VUID-LocalInvocationId-LocalInvocationId-04281
  The `LocalInvocationId` decoration **must** be used only within the `GLCompute`, `MeshEXT`, `TaskEXT`, `MeshNV`, or `TaskNV` `Execution Model`
- VUID-LocalInvocationId-LocalInvocationId-04282
  The variable decorated with `LocalInvocationId` **must** be declared using the `Input` `Storage Class`
- VUID-LocalInvocationId-LocalInvocationId-04283
  The variable decorated with `LocalInvocationId` **must** be declared as a three-component vector of 32-bit integer values

LocalInvocationIndex

Decorating a variable with the LocalInvocationIndex built-in decoration will make that variable contain a one-dimensional representation of LocalInvocationId. This is computed as:

```
LocalInvocationIndex =
    LocalInvocationId.z * WorkgroupSize.x * WorkgroupSize.y +
    LocalInvocationId.y * WorkgroupSize.x +
    LocalInvocationId.x;
```

Valid Usage

- VUID-LocalInvocationIndex-LocalInvocationIndex-04284
  The LocalInvocationIndex decoration **must** be used only within the GLCompute, MeshEXT, TaskEXT, MeshNV, or TaskNV Execution Model
- VUID-LocalInvocationIndex-LocalInvocationIndex-04285
  The variable decorated with LocalInvocationIndex **must** be declared using the Input Storage Class
- VUID-LocalInvocationIndex-LocalInvocationIndex-04286
  The variable decorated with LocalInvocationIndex **must** be declared as a scalar 32-bit integer value

NumWorkgroups

Decorating a variable with the NumWorkgroups built-in decoration will make that variable contain the number of local workgroups that are part of the dispatch that the invocation belongs to. Each component is equal to the values of the workgroup count parameters passed into the dispatching commands.

Valid Usage

- VUID-NumWorkgroups-NumWorkgroups-04296
  The NumWorkgroups decoration **must** be used only within the GLCompute, MeshEXT, or TaskEXT Execution Model
- VUID-NumWorkgroups-NumWorkgroups-04297
  The variable decorated with NumWorkgroups **must** be declared using the Input Storage Class
- VUID-NumWorkgroups-NumWorkgroups-04298
  The variable decorated with NumWorkgroups **must** be declared as a three-component vector of 32-bit integer values

ObjectRayDirectionKHR

A variable decorated with the ObjectRayDirectionKHR decoration will specify the direction of the ray being processed, in object space.

Valid Usage

- VUID-ObjectRayDirectionKHR-ObjectRayDirectionKHR-04299
  The `ObjectRayDirectionKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, or `ClosestHitKHR` `Execution Model`
- VUID-ObjectRayDirectionKHR-ObjectRayDirectionKHR-04300
  The variable decorated with `ObjectRayDirectionKHR` **must** be declared using the `Input` `Storage Class`
- VUID-ObjectRayDirectionKHR-ObjectRayDirectionKHR-04301
  The variable decorated with `ObjectRayDirectionKHR` **must** be declared as a three-component vector of 32-bit floating-point values

`ObjectRayOriginKHR`

A variable decorated with the `ObjectRayOriginKHR` decoration will specify the origin of the ray being processed, in object space.

Valid Usage

- VUID-ObjectRayOriginKHR-ObjectRayOriginKHR-04302
  The `ObjectRayOriginKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, or `ClosestHitKHR` `Execution Model`
- VUID-ObjectRayOriginKHR-ObjectRayOriginKHR-04303
  The variable decorated with `ObjectRayOriginKHR` **must** be declared using the `Input` `Storage Class`
- VUID-ObjectRayOriginKHR-ObjectRayOriginKHR-04304
  The variable decorated with `ObjectRayOriginKHR` **must** be declared as a three-component vector of 32-bit floating-point values

`ObjectToWorldKHR`

A variable decorated with the `ObjectToWorldKHR` decoration will contain the current object-to-world transformation matrix, which is determined by the instance of the current intersection.

Valid Usage

- VUID-ObjectToWorldKHR-ObjectToWorldKHR-04305
  The `ObjectToWorldKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, or `ClosestHitKHR` `Execution Model`
- VUID-ObjectToWorldKHR-ObjectToWorldKHR-04306
  The variable decorated with `ObjectToWorldKHR` **must** be declared using the `Input` `Storage Class`
- VUID-ObjectToWorldKHR-ObjectToWorldKHR-04307
  The variable decorated with `ObjectToWorldKHR` **must** be declared as a matrix with four columns of three-component vectors of 32-bit floating-point values

PatchVertices

Decorating a variable with the PatchVertices built-in decoration will make that variable contain the number of vertices in the input patch being processed by the shader. In a Tessellation Control Shader, this is the same as the name:patchControlPoints member of VkPipelineTessellationStateCreateInfo. In a Tessellation Evaluation Shader, PatchVertices is equal to the tessellation control output patch size. When the same shader is used in different pipelines where the patch sizes are configured differently, the value of the PatchVertices variable will also differ.

Valid Usage

- VUID-PatchVertices-PatchVertices-04308
  The PatchVertices decoration **must** be used only within the TessellationControl or TessellationEvaluation Execution Model
- VUID-PatchVertices-PatchVertices-04309
  The variable decorated with PatchVertices **must** be declared using the Input Storage Class
- VUID-PatchVertices-PatchVertices-04310
  The variable decorated with PatchVertices **must** be declared as a scalar 32-bit integer value

PointCoord

Decorating a variable with the PointCoord built-in decoration will make that variable contain the coordinate of the current fragment within the point being rasterized, normalized to the size of the point with origin in the upper left corner of the point, as described in Basic Point Rasterization. If the primitive the fragment shader invocation belongs to is not a point, then the variable decorated with PointCoord contains an undefined value.

NOTE Depending on how the point is rasterized, PointCoord **may** never reach (0,0) or (1,1).

Valid Usage

- VUID-PointCoord-PointCoord-04311
  The PointCoord decoration **must** be used only within the Fragment Execution Model
- VUID-PointCoord-PointCoord-04312
  The variable decorated with PointCoord **must** be declared using the Input Storage Class
- VUID-PointCoord-PointCoord-04313
  The variable decorated with PointCoord **must** be declared as a two-component vector of 32-bit floating-point values

PointSize

Decorating a variable with the PointSize built-in decoration will make that variable contain the size of point primitives or the final rasterization of polygons if polygon mode is VK_POLYGON_MODE_POINT when VkPhysicalDeviceMaintenance5PropertiesKHR::polygonModePointSize is set to VK_TRUE . The value written to the variable decorated with PointSize by the last pre-

rasterization shader stage in the pipeline is used as the framebuffer-space size of points produced by rasterization. If the maintenance5 feature is enabled and a value is not written to a variable decorated with PointSize, a value of 1.0 is used as the size of points.

NOTE When PointSize decorates a variable in the Input Storage Class, it contains the data written to the output variable decorated with PointSize from the previous shader stage.

Valid Usage

- VUID-PointSize-PointSize-04314
  The PointSize decoration **must** be used only within the MeshEXT, MeshNV, Vertex, TessellationControl, TessellationEvaluation, or Geometry Execution Model
- VUID-PointSize-PointSize-04315
  The variable decorated with PointSize within the MeshEXT, MeshNV, or Vertex Execution Model **must** be declared using the Output Storage Class
- VUID-PointSize-PointSize-04316
  The variable decorated with PointSize within the TessellationControl, TessellationEvaluation, or Geometry Execution Model **must** not be declared using a Storage Class other than Input or Output
- VUID-PointSize-PointSize-04317
  The variable decorated with PointSize **must** be declared as a scalar 32-bit floating-point value

Position

Decorating a variable with the Position built-in decoration will make that variable contain the position of the current vertex. In the last pre-rasterization shader stage, the value of the variable decorated with Position is used in subsequent primitive assembly, clipping, and rasterization operations.

NOTE When Position decorates a variable in the Input Storage Class, it contains the data written to the output variable decorated with Position from the previous shader stage.

Valid Usage

- VUID-Position-Position-04318
  The Position decoration **must** be used only within the MeshEXT, MeshNV, Vertex, TessellationControl, TessellationEvaluation, or Geometry Execution Model
- VUID-Position-Position-04319
  The variable decorated with Position within the MeshEXT, MeshNV, or Vertex Execution Model **must** be declared using the Output Storage Class
- VUID-Position-Position-04320
  The variable decorated with Position within the TessellationControl,

`TessellationEvaluation`, or `Geometry` `Execution Model` **must** not be declared using a `Storage Class` other than `Input` or `Output`

- VUID-Position-Position-04321
  The variable decorated with `Position` **must** be declared as a four-component vector of 32-bit floating-point values

`PrimitiveId`

Decorating a variable with the `PrimitiveId` built-in decoration will make that variable contain the index of the current primitive.

The index of the first primitive generated by a drawing command is zero, and the index is incremented after every individual point, line, or triangle primitive is processed.

For triangles drawn as points or line segments (see Polygon Mode), the primitive index is incremented only once, even if multiple points or lines are eventually drawn.

Variables decorated with `PrimitiveId` are reset to zero between each instance drawn.

Restarting a primitive topology using primitive restart has no effect on the value of variables decorated with `PrimitiveId`.

In tessellation control and tessellation evaluation shaders, it will contain the index of the patch within the current set of rendering primitives that corresponds to the shader invocation.

In a geometry shader, it will contain the number of primitives presented as input to the shader since the current set of rendering primitives was started.

In a fragment shader, it will contain the primitive index written by the geometry shader if a geometry shader is present, or with the value that would have been presented as input to the geometry shader had it been present.

In an intersection, any-hit, or closest hit shader, it will contain the index within the geometry of the triangle or bounding box being processed.

NOTE

When the `PrimitiveId` decoration is applied to an output variable in the geometry shader, the resulting value is seen through the `PrimitiveId` decorated input variable in the fragment shader.

The fragment shader using `PrimitiveId` will need to declare either the `Geometry` or `Tessellation` capability to satisfy the requirement SPIR-V has to use `PrimitiveId`.

Valid Usage

- VUID-PrimitiveId-PrimitiveId-04330
  The `PrimitiveId` decoration **must** be used only within the `MeshEXT`, `MeshNV`, `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, `TessellationControl`, `TessellationEvaluation`, `Geometry`, or `Fragment` `Execution Model`
- VUID-PrimitiveId-Fragment-04331

If pipeline contains both the Fragment and Geometry Execution Model and a variable decorated with PrimitiveId is read from Fragment shader, then the Geometry shader **must** write to the output variables decorated with PrimitiveId in all execution paths

- VUID-PrimitiveId-Fragment-04332
  If pipeline contains both the Fragment and MeshEXT or MeshNV Execution Model and a variable decorated with PrimitiveId is read from Fragment shader, then the MeshEXT or MeshNV shader **must** write to the output variables decorated with PrimitiveId in all execution paths
- VUID-PrimitiveId-Fragment-04333
  If Fragment Execution Model contains a variable decorated with PrimitiveId, then either the MeshShadingEXT, MeshShadingNV, Geometry or Tessellation capability **must** also be declared
- VUID-PrimitiveId-PrimitiveId-04334
  The variable decorated with PrimitiveId within the TessellationControl, TessellationEvaluation, Fragment, IntersectionKHR, AnyHitKHR, or ClosestHitKHR Execution Model **must** be declared using the Input Storage Class
- VUID-PrimitiveId-PrimitiveId-04335
  The variable decorated with PrimitiveId within the Geometry Execution Model **must** be declared using the Input or Output Storage Class
- VUID-PrimitiveId-PrimitiveId-04336
  The variable decorated with PrimitiveId within the MeshEXT or MeshNV Execution Model **must** be declared using the Output Storage Class
- VUID-PrimitiveId-PrimitiveId-04337
  The variable decorated with PrimitiveId **must** be declared as a scalar 32-bit integer value
- VUID-PrimitiveId-PrimitiveId-07040
  The variable decorated with PrimitiveId within the MeshEXT Execution Model **must** also be decorated with the PerPrimitiveEXT decoration

PrimitiveShadingRateKHR

Decorating a variable with the PrimitiveShadingRateKHR built-in decoration will make that variable contain the primitive fragment shading rate.

The value written to the variable decorated with PrimitiveShadingRateKHR by the last pre-rasterization shader stage in the pipeline is used as the primitive fragment shading rate. Outputs in previous shader stages are ignored.

If the last active pre-rasterization shader stage shader entry point's interface does not include a variable decorated with PrimitiveShadingRateKHR, then it is as if the shader specified a fragment shading rate value of 0, indicating a horizontal and vertical rate of 1 pixel.

If a shader has PrimitiveShadingRateKHR in the output interface and there is an execution path through the shader that does not write to it, its value is undefined for executions of the shader that take that path.

Valid Usage

- VUID-PrimitiveShadingRateKHR-PrimitiveShadingRateKHR-04484
  The `PrimitiveShadingRateKHR` decoration **must** be used only within the `MeshEXT`, `MeshNV`, `Vertex`, or `Geometry` `Execution Model`
- VUID-PrimitiveShadingRateKHR-PrimitiveShadingRateKHR-04485
  The variable decorated with `PrimitiveShadingRateKHR` **must** be declared using the `Output` `Storage Class`
- VUID-PrimitiveShadingRateKHR-PrimitiveShadingRateKHR-04486
  The variable decorated with `PrimitiveShadingRateKHR` **must** be declared as a scalar 32-bit integer value
- VUID-PrimitiveShadingRateKHR-PrimitiveShadingRateKHR-04487
  The value written to `PrimitiveShadingRateKHR` **must** include no more than one of `Vertical2Pixels` and `Vertical4Pixels`
- VUID-PrimitiveShadingRateKHR-PrimitiveShadingRateKHR-04488
  The value written to `PrimitiveShadingRateKHR` **must** include no more than one of `Horizontal2Pixels` and `Horizontal4Pixels`
- VUID-PrimitiveShadingRateKHR-PrimitiveShadingRateKHR-04489
  The value written to `PrimitiveShadingRateKHR` **must** not have any bits set other than those defined by **Fragment Shading Rate Flags** enumerants in the SPIR-V specification
- VUID-PrimitiveShadingRateKHR-PrimitiveShadingRateKHR-07059
  The variable decorated with `PrimitiveShadingRateKHR` within the `MeshEXT` `Execution Model` **must** also be decorated with the `PerPrimitiveEXT` decoration

`RayGeometryIndexKHR`

A variable decorated with the `RayGeometryIndexKHR` decoration will contain the geometry index for the acceleration structure geometry currently being shaded.

Valid Usage

- VUID-RayGeometryIndexKHR-RayGeometryIndexKHR-04345
  The `RayGeometryIndexKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, or `ClosestHitKHR` `Execution Model`
- VUID-RayGeometryIndexKHR-RayGeometryIndexKHR-04346
  The variable decorated with `RayGeometryIndexKHR` **must** be declared using the `Input` `Storage Class`
- VUID-RayGeometryIndexKHR-RayGeometryIndexKHR-04347
  The variable decorated with `RayGeometryIndexKHR` **must** be declared as a scalar 32-bit integer value

`RayTmaxKHR`

A variable decorated with the `RayTmaxKHR` decoration will contain the parametric t_{max} value of the

ray being processed. The value is independent of the space in which the ray origin and direction exist. The value is initialized to the parameter passed into the pipeline trace ray instruction.

The t_{max} value changes throughout the lifetime of the ray that produced the intersection. In the closest hit shader, the value reflects the closest distance to the intersected primitive. In the any-hit shader, it reflects the distance to the primitive currently being intersected. In the intersection shader, it reflects the distance to the closest primitive intersected so far or the initial value. The value can change in the intersection shader after calling `OpReportIntersectionKHR` if the corresponding any-hit shader does not ignore the intersection. In a miss shader, the value is identical to the parameter passed into the pipeline trace ray instruction.

Valid Usage

- VUID-RayTmaxKHR-RayTmaxKHR-04348
  The `RayTmaxKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, or `MissKHR` `Execution Model`
- VUID-RayTmaxKHR-RayTmaxKHR-04349
  The variable decorated with `RayTmaxKHR` **must** be declared using the `Input` `Storage Class`
- VUID-RayTmaxKHR-RayTmaxKHR-04350
  The variable decorated with `RayTmaxKHR` **must** be declared as a scalar 32-bit floating-point value

`RayTminKHR`

A variable decorated with the `RayTminKHR` decoration will contain the parametric t_{min} value of the ray being processed. The value is independent of the space in which the ray origin and direction exist. The value is the parameter passed into the pipeline trace ray instruction.

The t_{min} value remains constant for the duration of the ray query.

Valid Usage

- VUID-RayTminKHR-RayTminKHR-04351
  The `RayTminKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, or `MissKHR` `Execution Model`
- VUID-RayTminKHR-RayTminKHR-04352
  The variable decorated with `RayTminKHR` **must** be declared using the `Input` `Storage Class`
- VUID-RayTminKHR-RayTminKHR-04353
  The variable decorated with `RayTminKHR` **must** be declared as a scalar 32-bit floating-point value

`SampleId`

Decorating a variable with the `SampleId` built-in decoration will make that variable contain the coverage index for the current fragment shader invocation. `SampleId` ranges from zero to the number of samples in the framebuffer minus one. If a fragment shader entry point's interface

includes an input variable decorated with SampleId, Sample Shading is considered enabled with a minSampleShading value of 1.0.

Valid Usage

- VUID-SampleId-SampleId-04354
  The SampleId decoration **must** be used only within the Fragment Execution Model
- VUID-SampleId-SampleId-04355
  The variable decorated with SampleId **must** be declared using the Input Storage Class
- VUID-SampleId-SampleId-04356
  The variable decorated with SampleId **must** be declared as a scalar 32-bit integer value

SampleMask

Decorating a variable with the SampleMask built-in decoration will make any variable contain the sample mask for the current fragment shader invocation.

A variable in the Input storage class decorated with SampleMask will contain a bitmask of the set of samples covered by the primitive generating the fragment during rasterization. It has a sample bit set if and only if the sample is considered covered for this fragment shader invocation. SampleMask[] is an array of integers. Bits are mapped to samples in a manner where bit B of mask M (SampleMask[M]) corresponds to sample 32 × M + B.

A variable in the Output storage class decorated with SampleMask is an array of integers forming a bit array in a manner similar to an input variable decorated with SampleMask, but where each bit represents coverage as computed by the shader. This computed SampleMask is combined with the generated coverage mask in the multisample coverage operation.

Variables decorated with SampleMask **must** be either an unsized array, or explicitly sized to be no larger than the implementation-dependent maximum sample-mask (as an array of 32-bit elements), determined by the maximum number of samples.

If a fragment shader entry point's interface includes an output variable decorated with SampleMask, the sample mask will be undefined for any array elements of any fragment shader invocations that fail to assign a value. If a fragment shader entry point's interface does not include an output variable decorated with SampleMask, the sample mask has no effect on the processing of a fragment.

Valid Usage

- VUID-SampleMask-SampleMask-04357
  The SampleMask decoration **must** be used only within the Fragment Execution Model
- VUID-SampleMask-SampleMask-04358
  The variable decorated with SampleMask **must** be declared using the Input or Output Storage Class
- VUID-SampleMask-SampleMask-04359
  The variable decorated with SampleMask **must** be declared as an array of 32-bit integer

values

`SamplePosition`

Decorating a variable with the `SamplePosition` built-in decoration will make that variable contain the sub-pixel position of the sample being shaded. The top left of the pixel is considered to be at coordinate (0,0) and the bottom right of the pixel is considered to be at coordinate (1,1).

If the render pass has a fragment density map attachment, the variable will instead contain the sub-fragment position of the sample being shaded. The top left of the fragment is considered to be at coordinate (0,0) and the bottom right of the fragment is considered to be at coordinate (1,1) for any fragment area.

If a fragment shader entry point's interface includes an input variable decorated with `SamplePosition`, Sample Shading is considered enabled with a `minSampleShading` value of 1.0.

If the current pipeline uses custom sample locations the value of any variable decorated with the `SamplePosition` built-in decoration is undefined.

Valid Usage

- VUID-SamplePosition-SamplePosition-04360
  The `SamplePosition` decoration **must** be used only within the `Fragment` `Execution Model`
- VUID-SamplePosition-SamplePosition-04361
  The variable decorated with `SamplePosition` **must** be declared using the `Input` `Storage Class`
- VUID-SamplePosition-SamplePosition-04362
  The variable decorated with `SamplePosition` **must** be declared as a two-component vector of 32-bit floating-point values

`ShadingRateKHR`

Decorating a variable with the `ShadingRateKHR` built-in decoration will make that variable contain the fragment shading rate for the current fragment invocation.

Valid Usage

- VUID-ShadingRateKHR-ShadingRateKHR-04490
  The `ShadingRateKHR` decoration **must** be used only within the `Fragment` `Execution Model`
- VUID-ShadingRateKHR-ShadingRateKHR-04491
  The variable decorated with `ShadingRateKHR` **must** be declared using the `Input` `Storage Class`
- VUID-ShadingRateKHR-ShadingRateKHR-04492
  The variable decorated with `ShadingRateKHR` **must** be declared as a scalar 32-bit integer value

`TessCoord`

Decorating a variable with the `TessCoord` built-in decoration will make that variable contain the three-dimensional (u,v,w) barycentric coordinate of the tessellated vertex within the patch. u, v, and w are in the range [0,1] and vary linearly across the primitive being subdivided. For the tessellation modes of `Quads` or `IsoLines`, the third component is always zero.

Valid Usage

- VUID-TessCoord-TessCoord-04387
  The `TessCoord` decoration **must** be used only within the `TessellationEvaluation` `Execution Model`
- VUID-TessCoord-TessCoord-04388
  The variable decorated with `TessCoord` **must** be declared using the `Input` `Storage Class`
- VUID-TessCoord-TessCoord-04389
  The variable decorated with `TessCoord` **must** be declared as a three-component vector of 32-bit floating-point values

`TessLevelOuter`

Decorating a variable with the `TessLevelOuter` built-in decoration will make that variable contain the outer tessellation levels for the current patch.

In tessellation control shaders, the variable decorated with `TessLevelOuter` **can** be written to, controlling the tessellation factors for the resulting patch. These values are used by the tessellator to control primitive tessellation and **can** be read by tessellation evaluation shaders.

In tessellation evaluation shaders, the variable decorated with `TessLevelOuter` **can** read the values written by the tessellation control shader.

Valid Usage

- VUID-TessLevelOuter-TessLevelOuter-04390
  The `TessLevelOuter` decoration **must** be used only within the `TessellationControl` or `TessellationEvaluation` `Execution Model`
- VUID-TessLevelOuter-TessLevelOuter-04391
  The variable decorated with `TessLevelOuter` within the `TessellationControl` `Execution Model` **must** be declared using the `Output` `Storage Class`
- VUID-TessLevelOuter-TessLevelOuter-04392
  The variable decorated with `TessLevelOuter` within the `TessellationEvaluation` `Execution Model` **must** be declared using the `Input` `Storage Class`
- VUID-TessLevelOuter-TessLevelOuter-04393
  The variable decorated with `TessLevelOuter` **must** be declared as an array of size four, containing 32-bit floating-point values

`TessLevelInner`

Decorating a variable with the `TessLevelInner` built-in decoration will make that variable contain the inner tessellation levels for the current patch.

In tessellation control shaders, the variable decorated with `TessLevelInner` **can** be written to, controlling the tessellation factors for the resulting patch. These values are used by the tessellator to control primitive tessellation and **can** be read by tessellation evaluation shaders.

In tessellation evaluation shaders, the variable decorated with `TessLevelInner` **can** read the values written by the tessellation control shader.

Valid Usage

- VUID-TessLevelInner-TessLevelInner-04394
  The `TessLevelInner` decoration **must** be used only within the `TessellationControl` or `TessellationEvaluation` `Execution Model`
- VUID-TessLevelInner-TessLevelInner-04395
  The variable decorated with `TessLevelInner` within the `TessellationControl` `Execution Model` **must** be declared using the `Output` `Storage Class`
- VUID-TessLevelInner-TessLevelInner-04396
  The variable decorated with `TessLevelInner` within the `TessellationEvaluation` `Execution Model` **must** be declared using the `Input` `Storage Class`
- VUID-TessLevelInner-TessLevelInner-04397
  The variable decorated with `TessLevelInner` **must** be declared as an array of size two, containing 32-bit floating-point values

`VertexIndex`

Decorating a variable with the `VertexIndex` built-in decoration will make that variable contain the index of the vertex that is being processed by the current vertex shader invocation. For non-indexed draws, this variable begins at the `firstVertex` parameter to vkCmdDraw or the `firstVertex` member of a structure consumed by vkCmdDrawIndirect and increments by one for each vertex in the draw. For indexed draws, its value is the content of the index buffer for the vertex plus the `vertexOffset` parameter to vkCmdDrawIndexed or the `vertexOffset` member of the structure consumed by vkCmdDrawIndexedIndirect.

NOTE `VertexIndex` starts at the same starting value for each instance.

Valid Usage

- VUID-VertexIndex-VertexIndex-04398
  The `VertexIndex` decoration **must** be used only within the `Vertex` `Execution Model`
- VUID-VertexIndex-VertexIndex-04399
  The variable decorated with `VertexIndex` **must** be declared using the `Input` `Storage Class`
- VUID-VertexIndex-VertexIndex-04400

The variable decorated with VertexIndex **must** be declared as a scalar 32-bit integer value

ViewIndex

The ViewIndex decoration **can** be applied to a shader input which will be filled with the index of the view that is being processed by the current shader invocation.

If multiview is enabled in the render pass, this value will be the index of one of the bits set in the view mask of the subpass the pipeline is compiled against. If multiview is not enabled in the render pass, this value will be zero.

Valid Usage

- VUID-ViewIndex-ViewIndex-04401
  The ViewIndex decoration **must** be used only within the MeshEXT, Vertex, Geometry, TessellationControl, TessellationEvaluation or Fragment Execution Model
- VUID-ViewIndex-ViewIndex-04402
  The variable decorated with ViewIndex **must** be declared using the Input Storage Class
- VUID-ViewIndex-ViewIndex-04403
  The variable decorated with ViewIndex **must** be declared as a scalar 32-bit integer value

ViewportIndex

Decorating a variable with the ViewportIndex built-in decoration will make that variable contain the index of the viewport.

In a geometry shader, the variable decorated with ViewportIndex can be written to with the viewport index to which the primitive produced by that shader will be directed.

The selected viewport index is used to select the viewport transform and scissor rectangle.

If the last active pre-rasterization shader stage shader entry point's interface does not include a variable decorated with ViewportIndex then the first viewport is used. If a pre-rasterization shader stage shader entry point's interface includes a variable decorated with ViewportIndex, it **must** write the same value to ViewportIndex for all output vertices of a given primitive.

In a fragment shader, the variable decorated with ViewportIndex contains the viewport index of the primitive that the fragment invocation belongs to.

Valid Usage

- VUID-ViewportIndex-ViewportIndex-04404
  The ViewportIndex decoration **must** be used only within the MeshEXT, MeshNV, Vertex, TessellationEvaluation, Geometry, or Fragment Execution Model
- VUID-ViewportIndex-ViewportIndex-04406
  The variable decorated with ViewportIndex within the MeshEXT, MeshNV, Vertex, TessellationEvaluation, or Geometry Execution Model **must** be declared using the Output Storage Class

- VUID-ViewportIndex-ViewportIndex-04407
  The variable decorated with `ViewportIndex` within the `Fragment Execution Model` **must** be declared using the `Input Storage Class`
- VUID-ViewportIndex-ViewportIndex-04408
  The variable decorated with `ViewportIndex` **must** be declared as a scalar 32-bit integer value
- VUID-ViewportIndex-ViewportIndex-07060
  The variable decorated with `ViewportIndex` within the `MeshEXT Execution Model` **must** also be decorated with the `PerPrimitiveEXT` decoration

`WorkgroupId`

Decorating a variable with the `WorkgroupId` built-in decoration will make that variable contain the global workgroup that the current invocation is a member of. Each component ranges from a base value to a base + count value, based on the parameters passed into the dispatching commands.

Valid Usage

- VUID-WorkgroupId-WorkgroupId-04422
  The `WorkgroupId` decoration **must** be used only within the `GLCompute`, `MeshEXT`, `TaskEXT`, `MeshNV`, or `TaskNV Execution Model`
- VUID-WorkgroupId-WorkgroupId-04423
  The variable decorated with `WorkgroupId` **must** be declared using the `Input Storage Class`
- VUID-WorkgroupId-WorkgroupId-04424
  The variable decorated with `WorkgroupId` **must** be declared as a three-component vector of 32-bit integer values

`WorkgroupSize`

Decorating an object with the `WorkgroupSize` built-in decoration will make that object contain the dimensions of a local workgroup. If an object is decorated with the `WorkgroupSize` decoration, this takes precedence over any `LocalSize` or `LocalSizeId` execution mode.

Valid Usage

- VUID-WorkgroupSize-WorkgroupSize-04425
  The `WorkgroupSize` decoration **must** be used only within the `GLCompute`, `MeshEXT`, `TaskEXT`, `MeshNV`, or `TaskNV Execution Model`
- VUID-WorkgroupSize-WorkgroupSize-04426
  The variable decorated with `WorkgroupSize` **must** be a specialization constant or a constant
- VUID-WorkgroupSize-WorkgroupSize-04427
  The variable decorated with `WorkgroupSize` **must** be declared as a three-component vector of 32-bit integer values

`WorldRayDirectionKHR`

A variable decorated with the `WorldRayDirectionKHR` decoration will specify the direction of the ray being processed, in world space. The value is the parameter passed into the pipeline trace ray instruction.

Valid Usage

- VUID-WorldRayDirectionKHR-WorldRayDirectionKHR-04428
  The `WorldRayDirectionKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, or `MissKHR` `Execution Model`
- VUID-WorldRayDirectionKHR-WorldRayDirectionKHR-04429
  The variable decorated with `WorldRayDirectionKHR` **must** be declared using the `Input` `Storage Class`
- VUID-WorldRayDirectionKHR-WorldRayDirectionKHR-04430
  The variable decorated with `WorldRayDirectionKHR` **must** be declared as a three-component vector of 32-bit floating-point values

`WorldRayOriginKHR`

A variable decorated with the `WorldRayOriginKHR` decoration will specify the origin of the ray being processed, in world space. The value is the parameter passed into the pipeline trace ray instruction.

Valid Usage

- VUID-WorldRayOriginKHR-WorldRayOriginKHR-04431
  The `WorldRayOriginKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, or `MissKHR` `Execution Model`
- VUID-WorldRayOriginKHR-WorldRayOriginKHR-04432
  The variable decorated with `WorldRayOriginKHR` **must** be declared using the `Input` `Storage Class`
- VUID-WorldRayOriginKHR-WorldRayOriginKHR-04433
  The variable decorated with `WorldRayOriginKHR` **must** be declared as a three-component vector of 32-bit floating-point values

`WorldToObjectKHR`

A variable decorated with the `WorldToObjectKHR` decoration will contain the current world-to-object transformation matrix, which is determined by the instance of the current intersection.

Valid Usage

- VUID-WorldToObjectKHR-WorldToObjectKHR-04434
  The `WorldToObjectKHR` decoration **must** be used only within the `IntersectionKHR`, `AnyHitKHR`, or `ClosestHitKHR` `Execution Model`

- VUID-WorldToObjectKHR-WorldToObjectKHR-04435
  The variable decorated with `WorldToObjectKHR` **must** be declared using the `Input Storage Class`
- VUID-WorldToObjectKHR-WorldToObjectKHR-04436
  The variable decorated with `WorldToObjectKHR` **must** be declared as a matrix with four columns of three-component vectors of 32-bit floating-point values

Chapter 16. Image Operations

16.1. Image Operations Overview

Vulkan Image Operations are operations performed by those SPIR-V Image Instructions which take an `OpTypeImage` (representing a `VkImageView`) or `OpTypeSampledImage` (representing a (`VkImageView`, `VkSampler`) pair). Read, write, and atomic operations also take texel coordinates as operands, and return a value based on a neighborhood of texture elements (*texels*) within the image. Query operations return properties of the bound image or of the lookup itself. The "Depth" operand of `OpTypeImage` is ignored.

NOTE	Texel is a term which is a combination of the words texture and element. Early interactive computer graphics supported texture operations on textures, a small subset of the image operations on images described here. The discrete samples remain essentially equivalent, however, so we retain the historical term texel to refer to them.

Image Operations include the functionality of the following SPIR-V Image Instructions:

- `OpImageSample*` and `OpImageSparseSample*` read one or more neighboring texels of the image, and filter the texel values based on the state of the sampler.
  - Instructions with `ImplicitLod` in the name determine the LOD used in the sampling operation based on the coordinates used in neighboring fragments.
  - Instructions with `ExplicitLod` in the name determine the LOD used in the sampling operation based on additional coordinates.
  - Instructions with `Proj` in the name apply homogeneous projection to the coordinates.
- `OpImageFetch` and `OpImageSparseFetch` return a single texel of the image. No sampler is used.
- `OpImage*Gather` and `OpImageSparse*Gather` read neighboring texels and return a single component of each.
- `OpImageRead` (and `OpImageSparseRead`) and `OpImageWrite` read and write, respectively, a texel in the image. No sampler is used.
- `OpImage*Dref*` instructions apply depth comparison on the texel values.
- `OpImageSparse*` instructions additionally return a sparse residency code.
- `OpImageQuerySize`, `OpImageQuerySizeLod`, `OpImageQueryLevels`, and `OpImageQuerySamples` return properties of the image descriptor that would be accessed. The image itself is not accessed.
- `OpImageQueryLod` returns the LOD parameters that would be used in a sample operation. The actual operation is not performed.

16.1.1. Texel Coordinate Systems

Images are addressed by *texel coordinates*. There are three *texel coordinate systems*:

- normalized texel coordinates [0.0, 1.0]

- unnormalized texel coordinates [0.0, width / height / depth)
- integer texel coordinates [0, width / height / depth)

SPIR-V `OpImageFetch`, `OpImageSparseFetch`, `OpImageRead`, `OpImageSparseRead`, and `OpImageWrite` instructions use integer texel coordinates.

Other image instructions **can** use either normalized or unnormalized texel coordinates (selected by the `unnormalizedCoordinates` state of the sampler used in the instruction), but there are limitations on what operations, image state, and sampler state is supported. Normalized coordinates are logically converted to unnormalized as part of image operations, and certain steps are only performed on normalized coordinates. The array layer coordinate is always treated as unnormalized even when other coordinates are normalized.

Normalized texel coordinates are referred to as (s,t,r,q,a), with the coordinates having the following meanings:

- s: Coordinate in the first dimension of an image.
- t: Coordinate in the second dimension of an image.
- r: Coordinate in the third dimension of an image.
  - (s,t,r) are interpreted as a direction vector for Cube images.
- q: Fourth coordinate, for homogeneous (projective) coordinates.
- a: Coordinate for array layer.

The coordinates are extracted from the SPIR-V operand based on the dimensionality of the image variable and type of instruction. For `Proj` instructions, the components are in order (s, [t,] [r,] q), with t and r being conditionally present based on the `Dim` of the image. For non-`Proj` instructions, the coordinates are (s [,t] [,r] [,a]), with t and r being conditionally present based on the `Dim` of the image and a being conditionally present based on the `Arrayed` property of the image. Projective image instructions are not supported on `Arrayed` images.

Unnormalized texel coordinates are referred to as (u,v,w,a), with the coordinates having the following meanings:

- u: Coordinate in the first dimension of an image.
- v: Coordinate in the second dimension of an image.
- w: Coordinate in the third dimension of an image.
- a: Coordinate for array layer.

Only the u and v coordinates are directly extracted from the SPIR-V operand, because only 1D and 2D (non-`Arrayed`) dimensionalities support unnormalized coordinates. The components are in order (u [,v]), with v being conditionally present when the dimensionality is 2D. When normalized coordinates are converted to unnormalized coordinates, all four coordinates are used.

Integer texel coordinates are referred to as (i,j,k,l,n), with the coordinates having the following meanings:

- i: Coordinate in the first dimension of an image.

- j: Coordinate in the second dimension of an image.
- k: Coordinate in the third dimension of an image.
- l: Coordinate for array layer.
- n: Index of the sample within the texel.

They are extracted from the SPIR-V operand in order (i [,j] [,k] [,l] [,n]), with j and k conditionally present based on the `Dim` of the image, and l conditionally present based on the `Arrayed` property of the image. n is conditionally present and is taken from the `Sample` image operand.

For all coordinate types, unused coordinates are assigned a value of zero.

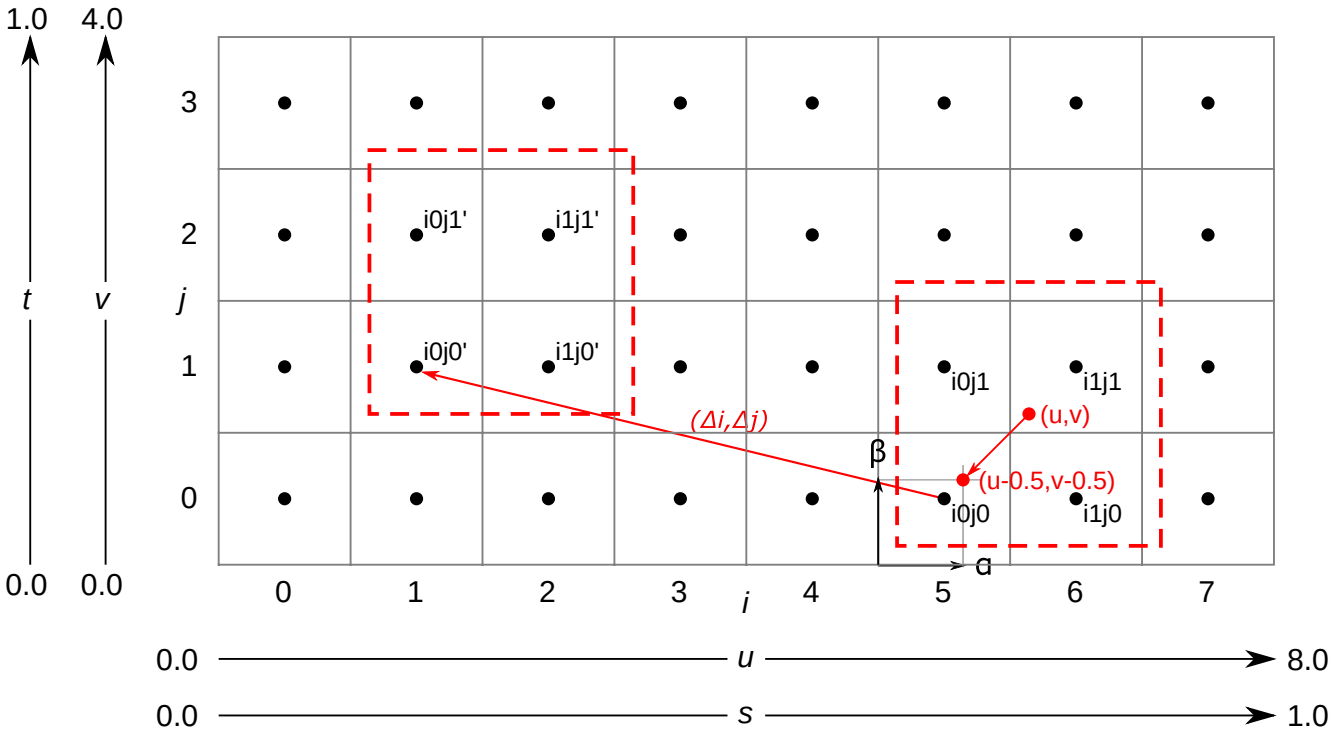


Figure 3. Texel Coordinate Systems, Linear Filtering

The Texel Coordinate Systems - For the example shown of an 8×4 texel two dimensional image.

- Normalized texel coordinates:
  - The s coordinate goes from 0.0 to 1.0.
  - The t coordinate goes from 0.0 to 1.0.
- Unnormalized texel coordinates:
  - The u coordinate within the range 0.0 to 8.0 is within the image, otherwise it is outside the image.
  - The v coordinate within the range 0.0 to 4.0 is within the image, otherwise it is outside the image.
- Integer texel coordinates:
  - The i coordinate within the range 0 to 7 addresses texels within the image, otherwise it is outside the image.
  - The j coordinate within the range 0 to 3 addresses texels within the image, otherwise it is

outside the image.

- Also shown for linear filtering:
  - Given the unnormalized coordinates (u,v), the four texels selected are i_0j_0, i_1j_0, i_0j_1, and i_1j_1.
  - The fractions α and β.
  - Given the offset Δ_i and Δ_j, the four texels selected by the offset are $i_0j'_0$, $i_1j'_0$, $i_0j'_1$, and $i_1j'_1$.

NOTE

For formats with reduced-resolution components, Δ_i and Δ_j are relative to the resolution of the highest-resolution component, and therefore may be divided by two relative to the unnormalized coordinate space of the lower-resolution components.

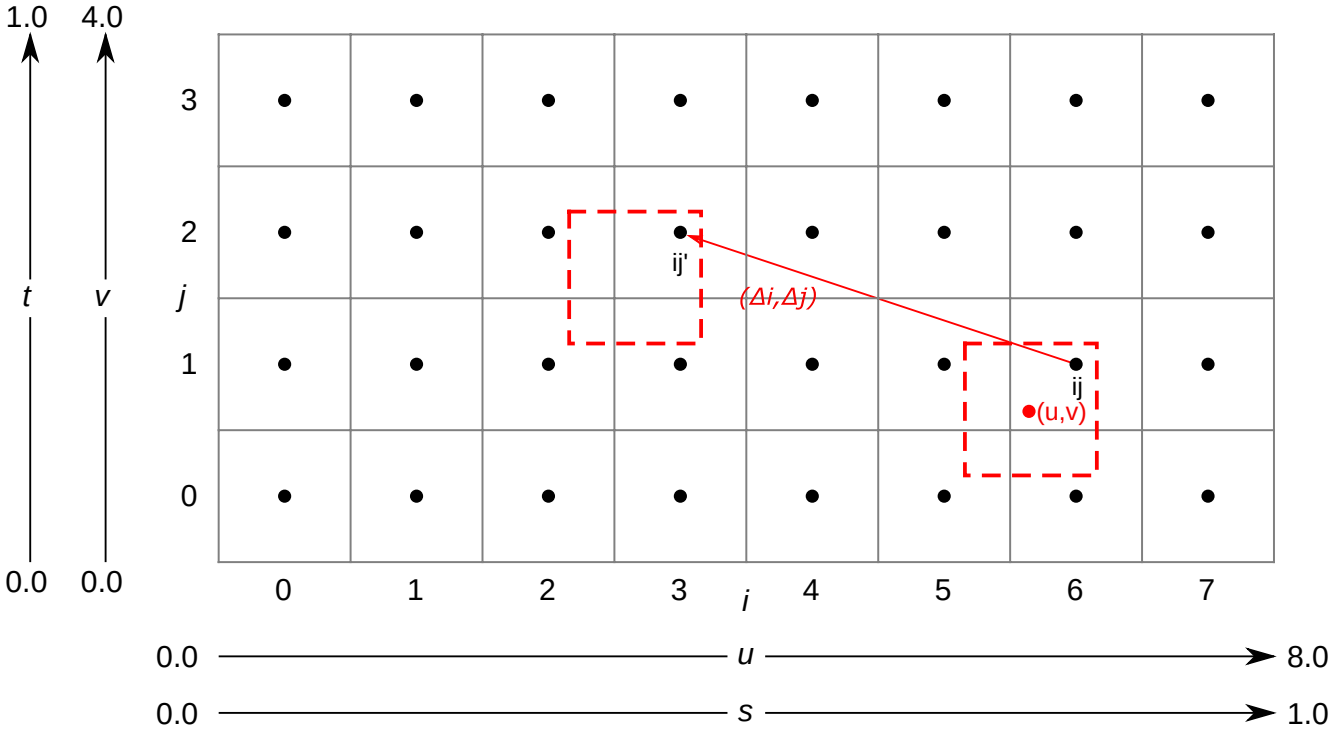


Figure 4. Texel Coordinate Systems, Nearest Filtering

The Texel Coordinate Systems - For the example shown of an 8×4 texel two dimensional image.

- Texel coordinates as above. Also shown for nearest filtering:
  - Given the unnormalized coordinates (u,v), the texel selected is ij.
  - Given the offset Δ_i and Δ_j, the texel selected by the offset is ij'.

16.2. Conversion Formulas

16.2.1. RGB to Shared Exponent Conversion

An RGB color (red, green, blue) is transformed to a shared exponent color (red_{shared}, $green_{shared}$, $blue_{shared}$, exp_{shared}) as follows:

First, the components (red, green, blue) are clamped to ($red_{clamped}$, $green_{clamped}$, $blue_{clamped}$) as:

$red_{clamped} = \max(0, \min(sharedexp_{max}, red))$

$green_{clamped} = \max(0, \min(sharedexp_{max}, green))$

$blue_{clamped} = \max(0, \min(sharedexp_{max}, blue))$

where:

$$N = 9 \quad \text{number of mantissa bits per component}$$
$$B = 15 \quad \text{exponent bias}$$
$$E_{max} = 31 \quad \text{maximum possible biased exponent value}$$
$$sharedexp_{max} = \frac{(2^N - 1)}{2^N} \times 2^{(E_{max} - B)}$$

NOTE NaN, if supported, is handled as in IEEE 754-2008 `minNum()` and `maxNum()`. This results in any NaN being mapped to zero.

The largest clamped component, $max_{clamped}$ is determined:

$max_{clamped} = \max(red_{clamped}, green_{clamped}, blue_{clamped})$

A preliminary shared exponent exp' is computed:

$$exp' = \begin{cases} \lfloor \log_2(max_{clamped}) \rfloor + (B+1) & \text{for } max_{clamped} > 2^{-(B+1)} \\ 0 & \text{for } max_{clamped} \le 2^{-(B+1)} \end{cases}$$

The shared exponent exp_{shared} is computed:

$$max_{shared} = \lfloor \frac{max_{clamped}}{2^{(exp' - B - N)}} + \frac{1}{2} \rfloor$$

$$exp_{shared} = \begin{cases} exp' & \text{for } 0 \le max_{shared} < 2^N \\ exp' + 1 & \text{for } max_{shared} = 2^N \end{cases}$$

Finally, three integer values in the range 0 to 2^N are computed:

$$red_{shared} = \lfloor \frac{red_{clamped}}{2^{(exp_{shared} - B - N)}} + \frac{1}{2} \rfloor$$
$$green_{shared} = \lfloor \frac{green_{clamped}}{2^{(exp_{shared} - B - N)}} + \frac{1}{2} \rfloor$$
$$blue_{shared} = \lfloor \frac{blue_{clamped}}{2^{(exp_{shared} - B - N)}} + \frac{1}{2} \rfloor$$

16.2.2. Shared Exponent to RGB

A shared exponent color (red_{shared}, $green_{shared}$, $blue_{shared}$, exp_{shared}) is transformed to an RGB color (red, green, blue) as follows:

$$red = red_{shared} \times 2^{(exp_{shared} - B - N)}$$

$$green = green_{shared} \times 2^{(exp_{shared} - B - N)}$$

$$blue = blue_{shared} \times 2^{(exp_{shared} - B - N)}$$

where:

N = 9 (number of mantissa bits per component)

B = 15 (exponent bias)

16.3. Texel Input Operations

Texel input instructions are SPIR-V image instructions that read from an image. *Texel input operations* are a set of steps that are performed on state, coordinates, and texel values while processing a texel input instruction, and which are common to some or all texel input instructions. They include the following steps, which are performed in the listed order:

- Validation operations
  - Instruction/Sampler/Image validation
  - Coordinate validation
  - Sparse validation
  - Layout validation
- Format conversion
- Texel replacement
- Depth comparison

- Conversion to RGBA
- Component swizzle
- Chroma reconstruction
- Y'C_BC_R conversion

For texel input instructions involving multiple texels (for sampling or gathering), these steps are applied for each texel that is used in the instruction. Depending on the type of image instruction, other steps are conditionally performed between these steps or involving multiple coordinate or texel values.

If Chroma Reconstruction is implicit, Texel Filtering instead takes place during chroma reconstruction, before sampler Y'C_BC_R conversion occurs.

16.3.1. Texel Input Validation Operations

Texel input validation operations inspect instruction/image/sampler state or coordinates, and in certain circumstances cause the texel value to be replaced or become undefined. There are a series of validations that the texel undergoes.

Instruction/Sampler/Image View Validation

There are a number of cases where a SPIR-V instruction **can** mismatch with the sampler, the image view, or both, and a number of further cases where the sampler **can** mismatch with the image view. In such cases the value of the texel returned is undefined.

These cases include:

- The sampler `borderColor` is an integer type and the image view `format` is not one of the VkFormat integer types or a stencil component of a depth/stencil format.
- The sampler `borderColor` is a float type and the image view `format` is not one of the VkFormat float types or a depth component of a depth/stencil format.
- The sampler `borderColor` is one of the opaque black colors (`VK_BORDER_COLOR_FLOAT_OPAQUE_BLACK` or `VK_BORDER_COLOR_INT_OPAQUE_BLACK`) and the image view VkComponentSwizzle for any of the VkComponentMapping components is not the identity swizzle, and VkPhysicalDeviceBorderColorSwizzleFeaturesEXT::`borderColorSwizzleFromImage` feature is not enabled, and VkSamplerBorderColorComponentMappingCreateInfoEXT is not specified.
- VkSamplerBorderColorComponentMappingCreateInfoEXT::`components`, if specified, has a component swizzle that does not match the component swizzle of the image view, and either component swizzle is not a form of identity swizzle.
- VkSamplerBorderColorComponentMappingCreateInfoEXT::`srgb`, if specified, does not match the sRGB encoding of the image view.
- The sampler `borderColor` is a custom color (`VK_BORDER_COLOR_FLOAT_CUSTOM_EXT` or `VK_BORDER_COLOR_INT_CUSTOM_EXT`) and the supplied VkSamplerCustomBorderColorCreateInfoEXT::`customBorderColor` is outside the bounds of the values representable in the image view's `format`.
- The sampler `borderColor` is a custom color (`VK_BORDER_COLOR_FLOAT_CUSTOM_EXT` or

VK_BORDER_COLOR_INT_CUSTOM_EXT) and the image view VkComponentSwizzle for any of the VkComponentMapping components is not the identity swizzle, and VkPhysicalDeviceBorderColorSwizzleFeaturesEXT::borderColorSwizzleFromImage feature is not enabled, and VkSamplerBorderColorComponentMappingCreateInfoEXT is not specified.

- The VkImageLayout of any subresource in the image view does not match the VkDescriptorImageInfo::imageLayout used to write the image descriptor.
- The SPIR-V Image Format is not compatible with the image view's format.
- The sampler unnormalizedCoordinates is VK_TRUE and any of the limitations of unnormalized coordinates are violated.
- The sampler was created with flags containing VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT and the image was not created with flags containing VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT.
- The sampler was not created with flags containing VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT and the image was created with flags containing VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT.
- The sampler was created with flags containing VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT and is used with a function that is not OpImageSampleImplicitLod or OpImageSampleExplicitLod, or is used with operands Offset or ConstOffsets.
- The SPIR-V instruction is one of the OpImage*Dref* instructions and the sampler compareEnable is VK_FALSE
- The SPIR-V instruction is not one of the OpImage*Dref* instructions and the sampler compareEnable is VK_TRUE
- The SPIR-V instruction is one of the OpImage*Dref* instructions, the image view format is one of the depth/stencil formats, and the image view aspect is not VK_IMAGE_ASPECT_DEPTH_BIT.
- The SPIR-V instruction's image variable's properties are not compatible with the image view:
  - If the image view's viewType is one of VK_IMAGE_VIEW_TYPE_1D_ARRAY, VK_IMAGE_VIEW_TYPE_2D_ARRAY, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY then the instruction **must** have Arrayed = 1. Otherwise the instruction **must** have Arrayed = 0.
  - If the image was created with VkImageCreateInfo::samples equal to VK_SAMPLE_COUNT_1_BIT, the instruction **must** have MS = 0.
  - If the image was created with VkImageCreateInfo::samples not equal to VK_SAMPLE_COUNT_1_BIT, the instruction **must** have MS = 1.
  - If the Sampled Type of the OpTypeImage does not match the SPIR-V Type.
  - If the signedness of any read or sample operation does not match the signedness of the image's format.
- The sampler was created with a specified VkSamplerCustomBorderColorCreateInfoEXT::format which does not match the VkFormat of the image view(s) it is sampling.
- The sampler is sampling an image view of VK_FORMAT_B4G4R4A4_UNORM_PACK16, VK_FORMAT_B5G6R5_UNORM_PACK16, or VK_FORMAT_B5G5R5A1_UNORM_PACK16 format without a specified VkSamplerCustomBorderColorCreateInfoEXT::format.

Only OpImageSample* and OpImageSparseSample* **can** be used with a sampler or image view that enables sampler $Y'C_BC_R$ conversion.

`OpImageFetch`, `OpImageSparseFetch`, `OpImage*Gather`, and `OpImageSparse*Gather` **must** not be used with a sampler or image view that enables sampler $Y'C_BC_R$ conversion.

The `ConstOffset` and `Offset` operands **must** not be used with a sampler or image view that enables sampler $Y'C_BC_R$ conversion.

If the underlying `VkImage` format has an X component in its format description, undefined values are read from those bits.

NOTE

If the `VkImage` format and `VkImageView` format are the same, these bits will be unused by format conversion and this will have no effect. However, if the `VkImageView` format is different, then some bits of the result may be undefined. For example, when a `VK_FORMAT_R10X6_UNORM_PACK16` `VkImage` is sampled via a `VK_FORMAT_R16_UNORM` `VkImageView`, the low 6 bits of the value before format conversion are undefined and format conversion may return a range of different values.

NOTE

Some implementations will return undefined values in the case where a sampler uses a VkSamplerAddressMode of `VK_SAMPLER_ADDRESS_MODE_MIRRORED_REPEAT`, the sampler is used with operands `Offset`, `ConstOffset`, or `ConstOffsets`, and the value of the offset is larger than or equal to the corresponding width, height, or depth of any accessed image level.

This behavior was not tested prior to Vulkan conformance test suite version 1.3.8.0. Affected implementations will have a conformance test waiver for this issue.

Integer Texel Coordinate Validation

Integer texel coordinates are validated against the size of the image level, and the number of layers and number of samples in the image. For SPIR-V instructions that use integer texel coordinates, this is performed directly on the integer coordinates. For instructions that use normalized or unnormalized texel coordinates, this is performed on the coordinates that result after conversion to integer texel coordinates.

If the integer texel coordinates do not satisfy all of the conditions

$0 \leq i < w_s$

$0 \leq j < h_s$

$0 \leq k < d_s$

$0 \leq l < \text{layers}$

$0 \leq n < \text{samples}$

where:

w_s = width of the image level

h_s = height of the image level

d_s = depth of the image level

layers = number of layers in the image

samples = number of samples per texel in the image

then the texel fails integer texel coordinate validation.

There are four cases to consider:

1. Valid Texel Coordinates
   - If the texel coordinates pass validation (that is, the coordinates lie within the image),

     then the texel value comes from the value in image memory.
2. Border Texel
   - If the texel coordinates fail validation, and
   - If the read is the result of an image sample instruction or image gather instruction, and
   - If the image is not a cube image, or if a sampler created with `VK_SAMPLER_CREATE_NON_SEAMLESS_CUBE_MAP_BIT_EXT` is used,

   then the texel is a border texel and texel replacement is performed.
3. Invalid Texel
   - If the texel coordinates fail validation, and
   - If the read is the result of an image fetch instruction, image read instruction, or atomic instruction,

     then the texel is an invalid texel and texel replacement is performed.
4. Cube Map Edge or Corner

   Otherwise the texel coordinates lie beyond the edges or corners of the selected cube map face, and Cube map edge handling is performed.

Cube Map Edge Handling

If the texel coordinates lie beyond the edges or corners of the selected cube map face (as described in the prior section), the following steps are performed. Note that this does not occur when using `VK_FILTER_NEAREST` filtering within a mip level, since `VK_FILTER_NEAREST` is treated as using `VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE`.

- Cube Map Edge Texel
  - If the texel lies beyond the selected cube map face in either only i or only j, then the coordinates (i,j) and the array layer l are transformed to select the adjacent texel from the appropriate neighboring face.
- Cube Map Corner Texel
  - If the texel lies beyond the selected cube map face in both i and j, then there is no unique neighboring face from which to read that texel. The texel **should** be replaced by the average of the three values of the adjacent texels in each incident face. However, implementations **may** replace the cube map corner texel by other methods. The methods are subject to the constraint that for linear filtering if the three available texels have the same value, the resulting filtered texel **must** have that value, and for cubic filtering if the twelve available samples have the same value, the resulting filtered texel **must** have that value.

Sparse Validation

If the texel reads from an unbound region of a sparse image, the texel is a *sparse unbound texel*, and processing continues with texel replacement.

Layout Validation

If all planes of a *disjoint multi-planar* image are not in the same image layout, the image **must** not be sampled with sampler $Y'C_BC_R$ conversion enabled.

16.3.2. Format Conversion

Texels undergo a format conversion from the VkFormat of the image view to a vector of either floating-point or signed or unsigned integer components, with the number of components based on the number of components present in the format.

- Color formats have one, two, three, or four components, according to the format.
- Depth/stencil formats are one component. The depth or stencil component is selected by the `aspectMask` of the image view.

Each component is converted based on its type and size (as defined in the Format Definition section for each VkFormat), using the appropriate equations in 16-Bit Floating-Point Numbers, Unsigned 11-Bit Floating-Point Numbers, Unsigned 10-Bit Floating-Point Numbers, Fixed-Point Data Conversion, and Shared Exponent to RGB. Signed integer components smaller than 32 bits are sign-extended.

If the image view format is sRGB, the color components are first converted as if they are UNORM, and then sRGB to linear conversion is applied to the R, G, and B components as described in the

“sRGB EOTF” section of the Khronos Data Format Specification. The A component, if present, is unchanged.

If the image view format is block-compressed, then the texel value is first decoded, then converted based on the type and number of components defined by the compressed format.

16.3.3. Texel Replacement

A texel is replaced if it is one (and only one) of:

- a border texel,
- an invalid texel, or
- a sparse unbound texel.

Border texels are replaced with a value based on the image format and the `borderColor` of the sampler. The border color is:

Table 20. Border Color B, Custom Border Color VkSamplerCustomBorderColorCreateInfoEXT ::customBorderColor U

Sampler `borderColor`	**Corresponding Border Color**
`VK_BORDER_COLOR_FLOAT_TRANSPARENT_BLACK`	$[B_r, B_g, B_b, B_a] = [0.0, 0.0, 0.0, 0.0]$
`VK_BORDER_COLOR_FLOAT_OPAQUE_BLACK`	$[B_r, B_g, B_b, B_a] = [0.0, 0.0, 0.0, 1.0]$
`VK_BORDER_COLOR_FLOAT_OPAQUE_WHITE`	$[B_r, B_g, B_b, B_a] = [1.0, 1.0, 1.0, 1.0]$
`VK_BORDER_COLOR_INT_TRANSPARENT_BLACK`	$[B_r, B_g, B_b, B_a] = [0, 0, 0, 0]$
`VK_BORDER_COLOR_INT_OPAQUE_BLACK`	$[B_r, B_g, B_b, B_a] = [0, 0, 0, 1]$
`VK_BORDER_COLOR_INT_OPAQUE_WHITE`	$[B_r, B_g, B_b, B_a] = [1, 1, 1, 1]$
`VK_BORDER_COLOR_FLOAT_CUSTOM_EXT`	$[B_r, B_g, B_b, B_a] = [U_r, U_g, U_b, U_a]$
`VK_BORDER_COLOR_INT_CUSTOM_EXT`	$[B_r, B_g, B_b, B_a] = [U_r, U_g, U_b, U_a]$

The custom border color (U) **may** be rounded by implementations prior to texel replacement, but the error introduced by such a rounding **must** not exceed one ULP of the image’s `format`.

NOTE The names `VK_BORDER_COLOR_*_TRANSPARENT_BLACK`, `VK_BORDER_COLOR_*_OPAQUE_BLACK`, and `VK_BORDER_COLOR_*_OPAQUE_WHITE` are meant to describe which components are zeros and ones in the vocabulary of compositing, and are not meant to imply that the numerical value of `VK_BORDER_COLOR_INT_OPAQUE_WHITE` is a saturating value for integers.

This is substituted for the texel value by replacing the number of components in the image format

Table 21. Border Texel Components After Replacement

Texel Aspect or Format	**Component Assignment**
Depth aspect	$D = B_r$

Texel Aspect or Format	Component Assignment
Stencil aspect	$S = B_r$†
One component color format	$Color_r = B_r$
Two component color format	$[Color_r,Color_g] = [B_r,B_g]$
Three component color format	$[Color_r,Color_g,Color_b] = [B_r,B_g,B_b]$
Four component color format	$[Color_r,Color_g,Color_b,Color_a] = [B_r,B_g,B_b,B_a]$
Single component alpha format	$[Color_r,Color_g,Color_b, Color_a] = [0,0,0,B_a]$

† $S = B_g$ **may** be substituted as the replacement method by the implementation when VkSamplerCreateInfo::`borderColor` is `VK_BORDER_COLOR_INT_CUSTOM_EXT` and VkSamplerCustomBorderColorCreateInfoEXT::`format` is `VK_FORMAT_UNDEFINED`. Implementations **should** use $S = B_r$ as the replacement method.

The value returned by a read of an invalid texel is undefined, unless that read operation is from a buffer resource and the `robustBufferAccess` feature is enabled. In that case, an invalid texel is replaced as described by the `robustBufferAccess` feature. If the access is to an image resource and the x, y, z, or layer coordinate validation fails and If the `robustImageAccess2` feature is enabled, zero values **must** be returned. If only the sample index was invalid, the values returned are undefined.

Loads from a null descriptor return a four component color value of all zeros. However, for storage images and storage texel buffers using an explicit SPIR-V Image Format, loads from a null descriptor **may** return an alpha value of 1 (float or integer, depending on format) if the format does not include alpha.

If the VkPhysicalDeviceSparseProperties::`residencyNonResidentStrict` property is `VK_TRUE`, a sparse unbound texel is replaced with 0 or 0.0 values for integer and floating-point components of the image format, respectively.

If `residencyNonResidentStrict` is `VK_FALSE`, the value of the sparse unbound texel is undefined.

16.3.4. Depth Compare Operation

If the image view has a depth/stencil format, the depth component is selected by the `aspectMask`, and the operation is an `OpImage*Dref*` instruction, a depth comparison is performed. The result is 1.0 if the comparison evaluates to true, and 0.0 otherwise. This value replaces the depth component D.

The compare operation is selected by the VkCompareOp value set by VkSamplerCreateInfo ::`compareOp`. The reference value from the SPIR-V operand D_{ref} and the texel depth value D_{tex} are used as the *reference* and *test* values, respectively, in that operation.

If the image being sampled has an unsigned normalized fixed-point format, then D_{ref} is clamped to [0,1] before the compare operation.

16.3.5. Conversion to RGBA

The texel is expanded from one, two, or three components to four components based on the image base color:

Table 22. Texel Color After Conversion To RGBA

Texel Aspect or Format	RGBA Color
Depth aspect	[$Color_r$,$Color_g$,$Color_b$, $Color_a$] = [D,0,0,one]
Stencil aspect	[$Color_r$,$Color_g$,$Color_b$, $Color_a$] = [S,0,0,one]
One component color format	[$Color_r$,$Color_g$,$Color_b$, $Color_a$] = [$Color_r$,0,0,one]
Two component color format	[$Color_r$,$Color_g$,$Color_b$, $Color_a$] = [$Color_r$,$Color_g$,0,one]
Three component color format	[$Color_r$,$Color_g$,$Color_b$, $Color_a$] = [$Color_r$,$Color_g$,$Color_b$,one]
Four component color format	[$Color_r$,$Color_g$,$Color_b$, $Color_a$] = [$Color_r$,$Color_g$,$Color_b$,$Color_a$]
One alpha component color format	[$Color_r$,$Color_g$,$Color_b$, $Color_a$] = [0,0,0,$Color_a$]

where one = 1.0f for floating-point formats and depth aspects, and one = 1 for integer formats and stencil aspects.

16.3.6. Component Swizzle

All texel input instructions apply a *swizzle* based on:

- the VkComponentSwizzle enums in the `components` member of the VkImageViewCreateInfo structure for the image being read if sampler $Y'C_BC_R$ conversion is not enabled, and
- the VkComponentSwizzle enums in the `components` member of the VkSamplerYcbcrConversionCreateInfo structure for the sampler $Y'C_BC_R$ conversion if sampler $Y'C_BC_R$ conversion is enabled.

The swizzle **can** rearrange the components of the texel, or substitute zero or one for any components. It is defined as follows for each color component:

$$Color'_{component} = \begin{cases} Color_r & \text{for RED swizzle} \\ Color_g & \text{for GREEN swizzle} \\ Color_b & \text{for BLUE swizzle} \\ Color_a & \text{for ALPHA swizzle} \\ 0 & \text{for ZERO swizzle} \\ one & \text{for ONE swizzle} \\ identity & \text{for IDENTITY swizzle} \end{cases}$$

where:

$$one = \begin{cases} 1.0f & \text{for floating-point components} \\ 1 & \text{for integer components} \end{cases}$$

$$identity = \begin{cases} Color_r & \text{for } component = r \\ Color_g & \text{for } component = g \\ Color_b & \text{for } component = b \\ Color_a & \text{for } component = a \end{cases}$$

If the border color is one of the VK_BORDER_COLOR_*_OPAQUE_BLACK enums and the VkComponentSwizzle is not the identity swizzle for all components, the value of the texel after swizzle is undefined.

If the image view has a depth/stencil format and the VkComponentSwizzle is VK_COMPONENT_SWIZZLE_ONE, and VkPhysicalDeviceMaintenance5PropertiesKHR ::depthStencilSwizzleOneSupport is not VK_TRUE, the value of the texel after swizzle is undefined.

16.3.7. Sparse Residency

OpImageSparse* instructions return a structure which includes a *residency code* indicating whether any texels accessed by the instruction are sparse unbound texels. This code **can** be interpreted by the OpImageSparseTexelsResident instruction which converts the residency code to a boolean value.

16.3.8. Chroma Reconstruction

In some color models, the color representation is defined in terms of monochromatic light intensity (often called "luma") and color differences relative to this intensity, often called "chroma". It is common for color models other than RGB to represent the chroma components at lower spatial resolution than the luma component. This approach is used to take advantage of the eye's lower spatial sensitivity to color compared with its sensitivity to brightness. Less commonly, the same approach is used with additive color, since the green component dominates the eye's sensitivity to light intensity and the spatial sensitivity to color introduced by red and blue is lower.

Lower-resolution components are "downsampled" by resizing them to a lower spatial resolution than the component representing luminance. This process is also commonly known as "chroma subsampling". There is one luminance sample in each texture texel, but each chrominance sample may be shared among several texels in one or both texture dimensions.

- "_444" formats do not spatially downsample chroma values compared with luma: there are unique chroma samples for each texel.
- "_422" formats have downsampling in the x dimension (corresponding to u or s coordinates): they are sampled at half the resolution of luma in that dimension.
- "_420" formats have downsampling in the x dimension (corresponding to u or s coordinates) and the y dimension (corresponding to v or t coordinates): they are sampled at half the resolution of luma in both dimensions.

The process of reconstructing a full color value for texture access involves accessing both chroma and luma values at the same location. To generate the color accurately, the values of the lower-resolution components at the location of the luma samples are reconstructed from the lower-resolution sample locations, an operation known here as "chroma reconstruction" irrespective of the actual color model.

The location of the chroma samples relative to the luma coordinates is determined by the xChromaOffset and yChromaOffset members of the VkSamplerYcbcrConversionCreateInfo structure used to create the sampler $Y'C_BC_R$ conversion.

The following diagrams show the relationship between unnormalized (u,v) coordinates and (i,j)

integer texel positions in the luma component (shown in black, with circles showing integer sample positions) and the texel coordinates of reduced-resolution chroma components, shown as crosses in red.

NOTE

If the chroma values are reconstructed at the locations of the luma samples by means of interpolation, chroma samples from outside the image bounds are needed; these are determined according to Wrapping Operation. These diagrams represent this by showing the bounds of the “chroma texel” extending beyond the image bounds, and including additional chroma sample positions where required for interpolation. The limits of a sample for `NEAREST` sampling is shown as a grid.

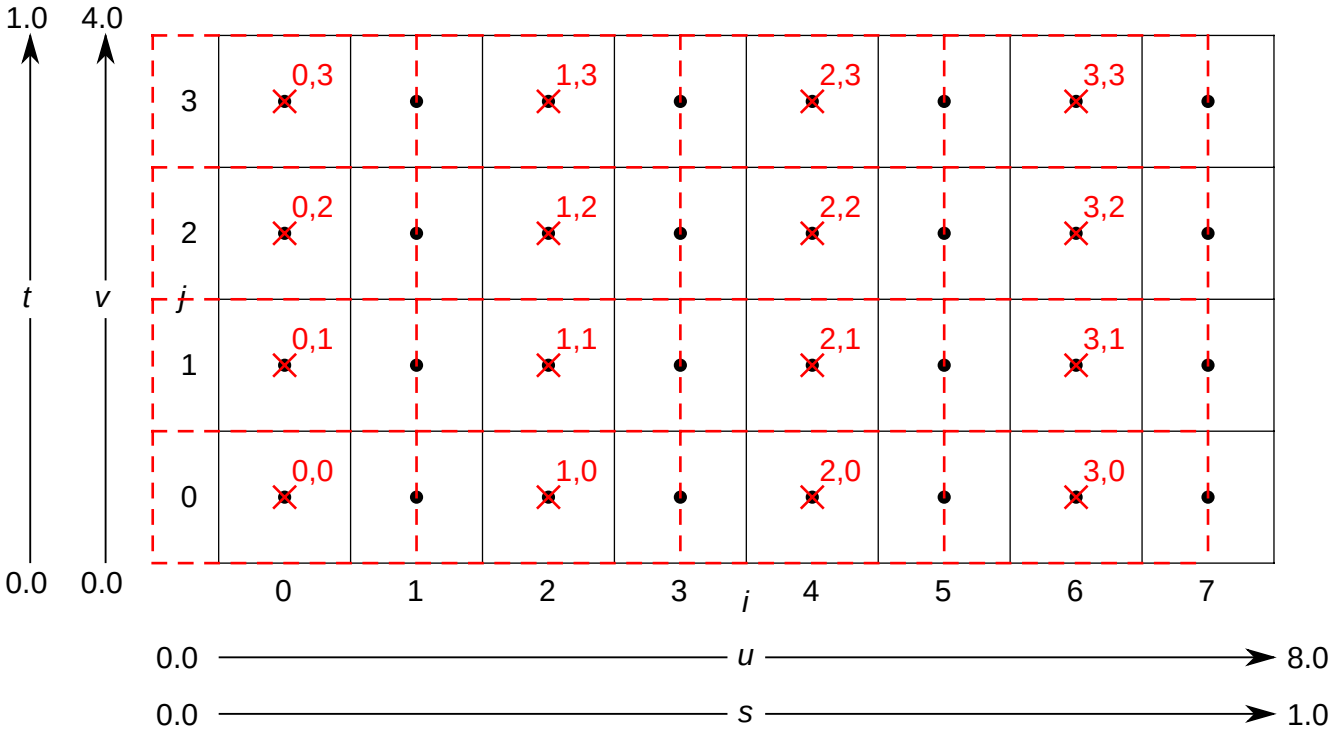


Figure 5. 422 downsampling, xChromaOffset=COSITED_EVEN

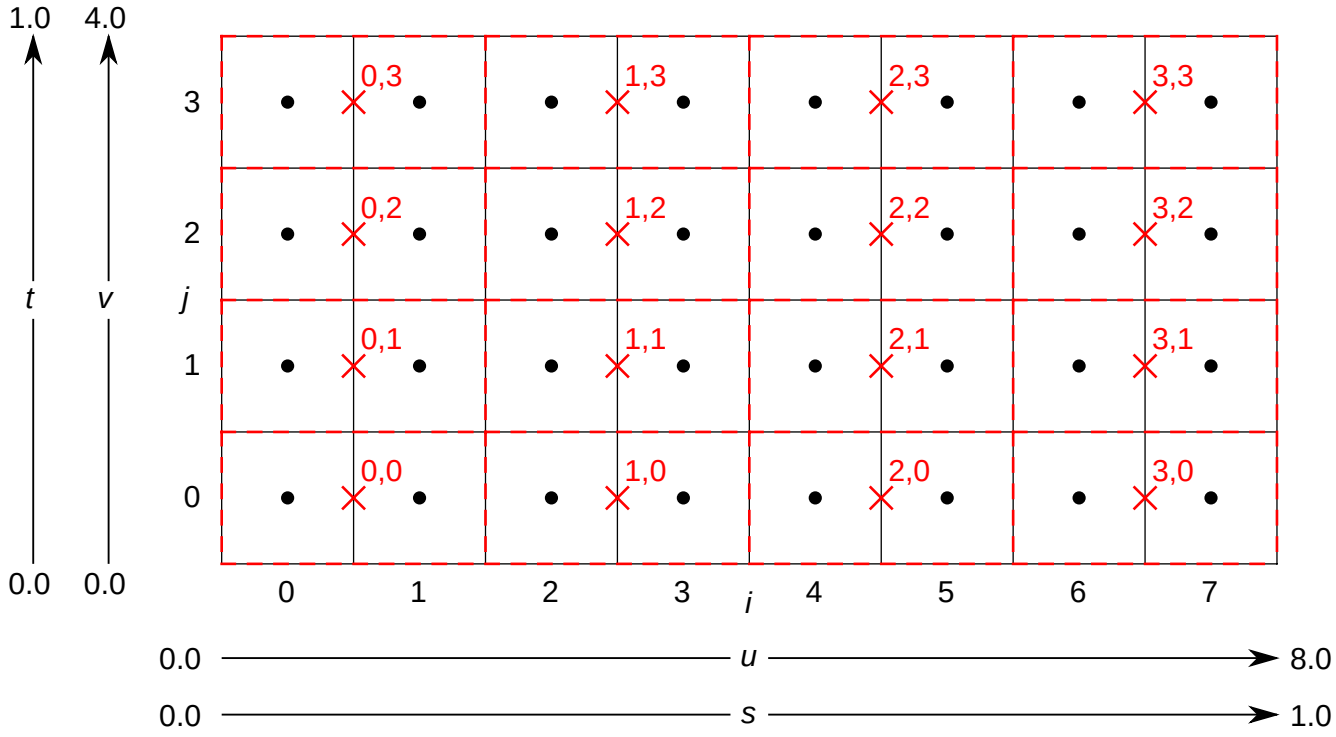


Figure 6. 422 downsampling, xChromaOffset=MIDPOINT

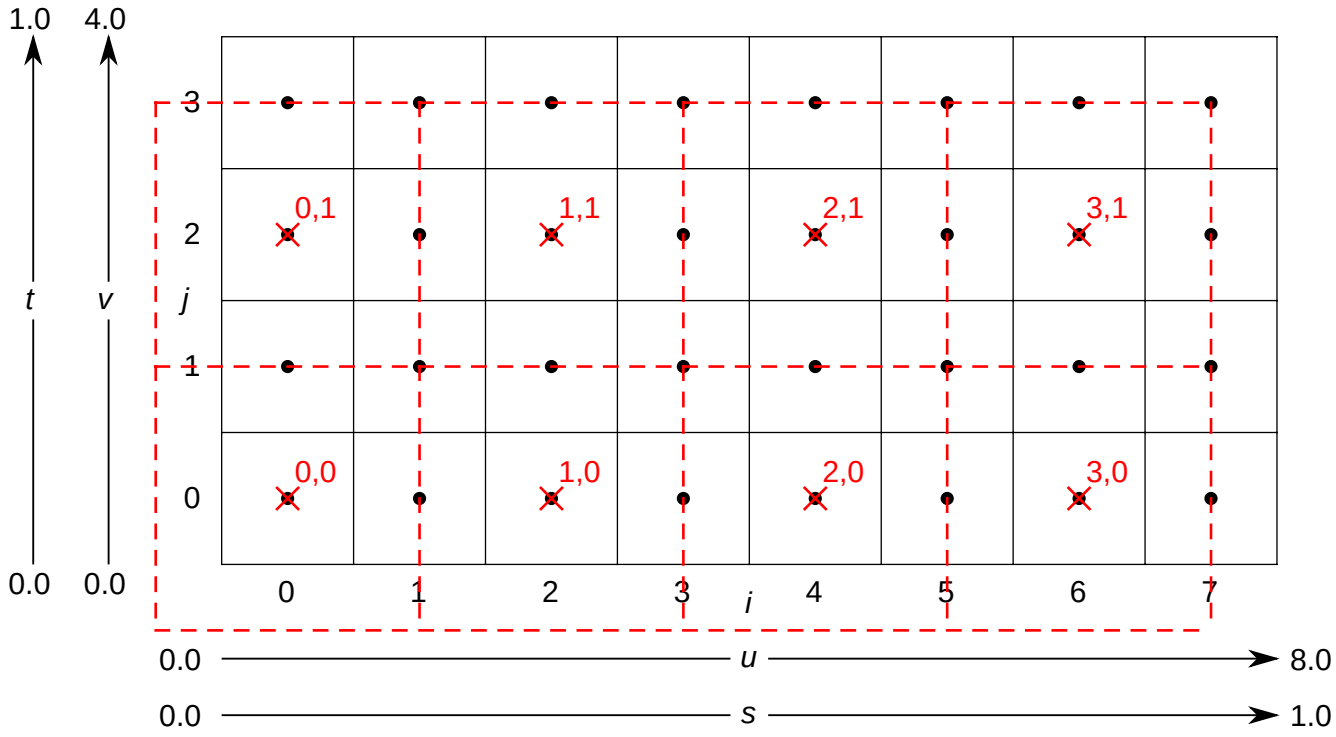


Figure 7. 420 downsampling, xChromaOffset=COSITED_EVEN, yChromaOffset=COSITED_EVEN

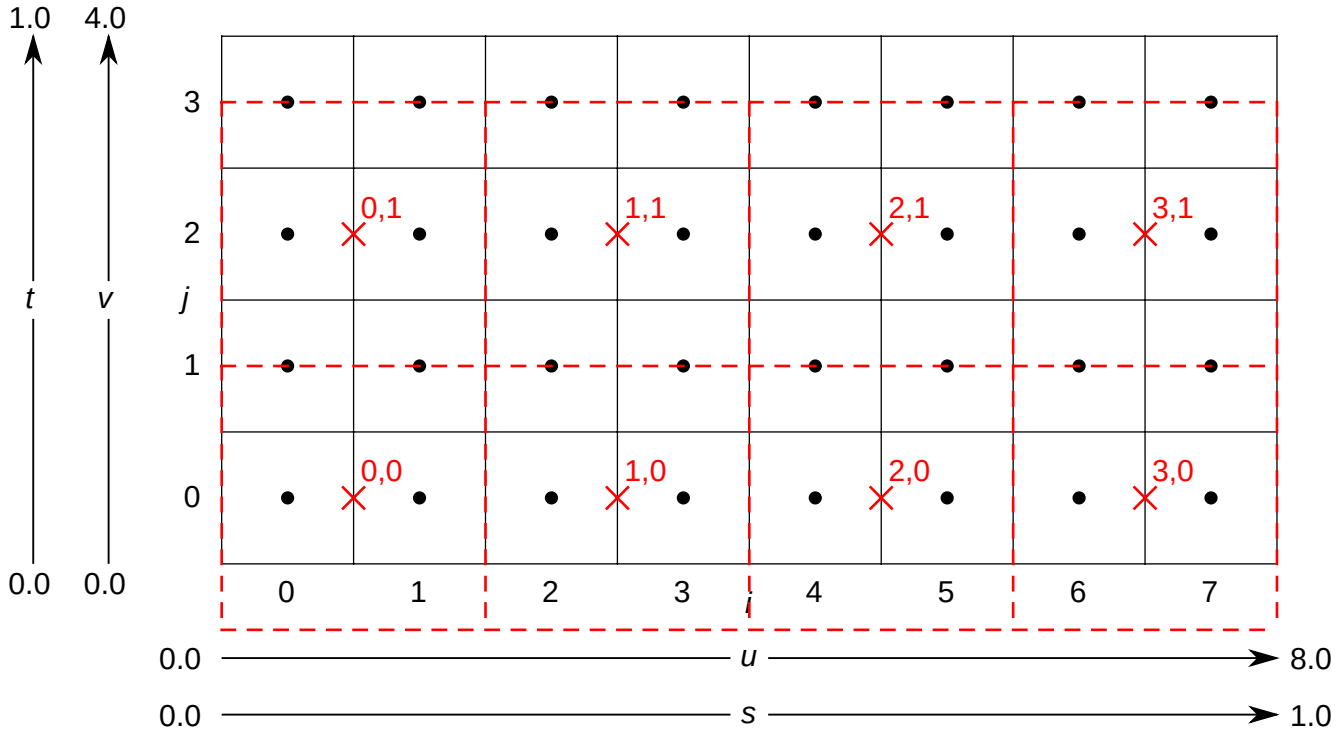


Figure 8. 420 downsampling, xChromaOffset=MIDPOINT, yChromaOffset=COSITED_EVEN

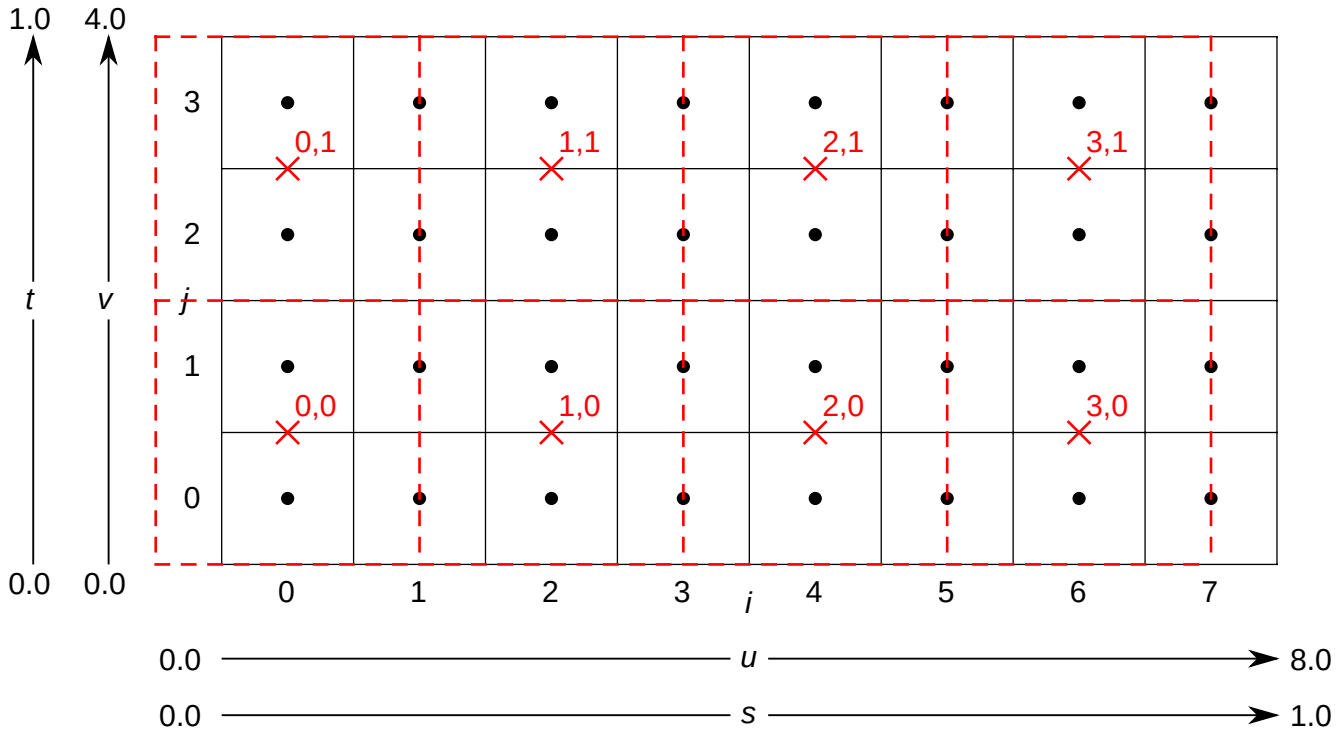


Figure 9. 420 downsampling, xChromaOffset=COSITED_EVEN, yChromaOffset=MIDPOINT

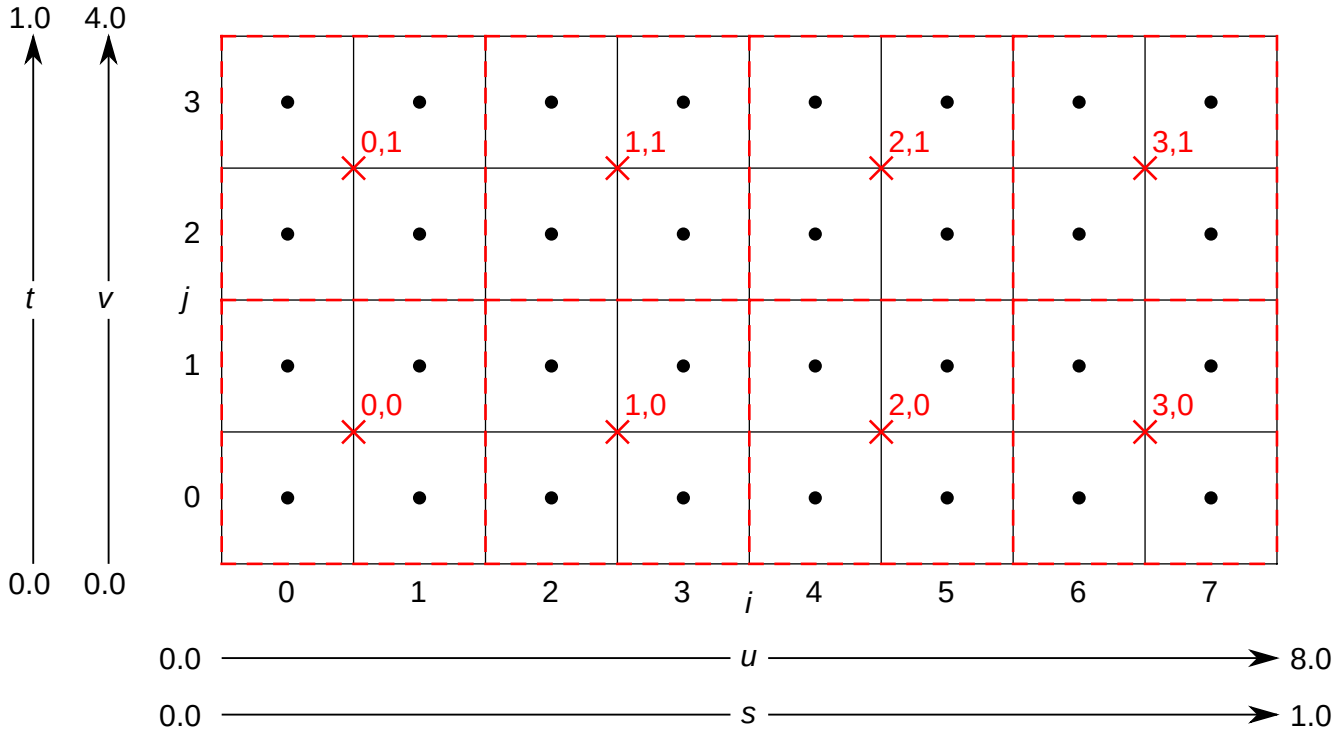


Figure 10. 420 downsampling, xChromaOffset=MIDPOINT, yChromaOffset=MIDPOINT

Reconstruction is implemented in one of two ways:

If the format of the image that is to be sampled sets `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT`, or the VkSamplerYcbcrConversionCreateInfo's `forceExplicitReconstruction` is `VK_TRUE`, reconstruction is performed as an explicit step independent of filtering, described in the Explicit Reconstruction section.

If the format of the image that is to be sampled does not set `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT` and if the VkSamplerYcbcrConversionCreateInfo's `forceExplicitReconstruction` is `VK_FALSE`, reconstruction is performed as an implicit part of filtering prior to color model conversion, with no separate post-conversion texel filtering step, as described in the Implicit Reconstruction section.

Explicit Reconstruction

- If the `chromaFilter` member of the VkSamplerYcbcrConversionCreateInfo structure is `VK_FILTER_NEAREST`:
  - If the format's R and B components are reduced in resolution in just width by a factor of two relative to the G component (i.e. this is a "`_422`" format), the $\tau_{ijk}[level]$ values accessed by texel filtering are reconstructed as follows:

$$\tau_R{}'(i,\ j) = \tau_R(\lfloor i \times 0.5 \rfloor,\ j)[level]$$
$$\tau_B{}'(i,\ j) = \tau_B(\lfloor i \times 0.5 \rfloor,\ j)[level]$$

  - If the format's R and B components are reduced in resolution in width and height by a factor of two relative to the G component (i.e. this is a "`_420`" format), the $\tau_{ijk}[level]$ values accessed by texel filtering are reconstructed as follows:

$$\tau_R{}'(i,\ j) = \tau_R(\lfloor i \times 0.5 \rfloor,\ \lfloor j \times 0.5 \rfloor)[level]$$
$$\tau_B{}'(i,\ j) = \tau_B(\lfloor i \times 0.5 \rfloor,\ \lfloor j \times 0.5 \rfloor)[level]$$

NOTE `xChromaOffset` and `yChromaOffset` have no effect if `chromaFilter` is `VK_FILTER_NEAREST` for explicit reconstruction.

- If the `chromaFilter` member of the VkSamplerYcbcrConversionCreateInfo structure is `VK_FILTER_LINEAR`:
  - If the format's R and B components are reduced in resolution in just width by a factor of two relative to the G component (i.e. this is a "`_422`" format):
    - If `xChromaOffset` is `VK_CHROMA_LOCATION_COSITED_EVEN`:

$$\tau_{RB}{}'(i,\ j) = \begin{cases} \tau_{RB}(\lfloor i \times 0.5 \rfloor,\ j)[level], & 0.5 \times i = \lfloor 0.5 \times i \rfloor \\ 0.5 \times \tau_{RB}(\lfloor i \times 0.5 \rfloor,\ j)[level] + \\ 0.5 \times \tau_{RB}(\lfloor i \times 0.5 \rfloor + 1,\ j)[level], & 0.5 \times i \neq \lfloor 0.5 \times i \rfloor \end{cases}$$

    - If `xChromaOffset` is `VK_CHROMA_LOCATION_MIDPOINT`:

$$\tau_{RB}{}'(i,\ j) = \begin{cases} 0.25 \times \tau_{RB}(\lfloor i \times 0.5 \rfloor - 1,\ j)[level] + \\ 0.75 \times \tau_{RB}(\lfloor i \times 0.5 \rfloor,\ j)[level], & 0.5 \times i = \lfloor 0.5 \times i \rfloor \\ 0.75 \times \tau_{RB}(\lfloor i \times 0.5 \rfloor,\ j)[level] + \\ 0.25 \times \tau_{RB}(\lfloor i \times 0.5 \rfloor + 1,\ j)[level], & 0.5 \times i \neq \lfloor 0.5 \times i \rfloor \end{cases}$$

  - If the format's R and B components are reduced in resolution in width and height by a factor of two relative to the G component (i.e. this is a "`_420`" format), a similar relationship applies. Due to the number of options, these formulae are expressed more concisely as follows:

$$i_{RB} = \begin{cases} 0.5 \times (i) & \text{xChromaOffset=COSITED_EVEN} \\ 0.5 \times (i - 0.5) & \text{xChromaOffset=MIDPOINT} \end{cases}$$
$$j_{RB} = \begin{cases} 0.5 \times (j) & \text{yChromaOffset=COSITED_EVEN} \\ 0.5 \times (j - 0.5) & \text{yChromaOffset=MIDPOINT} \end{cases}$$
$$i_{floor} = \lfloor i_{RB} \rfloor$$
$$j_{floor} = \lfloor j_{RB} \rfloor$$
$$i_{frac} = i_{RB} - i_{floor}$$
$$j_{frac} = j_{RB} - j_{floor}$$

$$\begin{aligned} \tau_{RB}{}'(i,\ j) = {} & \tau_{RB}(i_{floor},\ j_{floor})[level] && \times (1 - i_{frac}) && \times && (1 - j_{frac}) + \\ & \tau_{RB}(1 + i_{floor},\ j_{floor})[level] && \times (i_{frac}) && \times && (1 - j_{frac}) + \\ & \tau_{RB}(i_{floor},\ 1 + j_{floor})[level] && \times (1 - i_{frac}) && \times && (j_{frac}) + \\ & \tau_{RB}(1 + i_{floor},\ 1 + j_{floor})[level] && \times (i_{frac}) && \times && (j_{frac}) \end{aligned}$$

NOTE In the case where the texture itself is bilinearly interpolated as described in Texel Filtering, thus requiring four full-color samples for the filtering operation, and where the reconstruction of these samples uses bilinear interpolation in the chroma

components due to `chromaFilter`=`VK_FILTER_LINEAR`, up to nine chroma samples may be required, depending on the sample location.

Implicit Reconstruction

Implicit reconstruction takes place by the samples being interpolated, as required by the filter settings of the sampler, except that `chromaFilter` takes precedence for the chroma samples.

If `chromaFilter` is `VK_FILTER_NEAREST`, an implementation **may** behave as if `xChromaOffset` and `yChromaOffset` were both `VK_CHROMA_LOCATION_MIDPOINT`, irrespective of the values set.

NOTE This will not have any visible effect if the locations of the luma samples coincide with the location of the samples used for rasterization.

The sample coordinates are adjusted by the downsample factor of the component (such that, for example, the sample coordinates are divided by two if the component has a downsample factor of two relative to the luma component):

$$u_{RB}'(422/420) = \begin{cases} 0.5 \times (u + 0.5), & \text{xChromaOffset=COSITED_EVEN} \\ 0.5 \times u, & \text{xChromaOffset=MIDPOINT} \end{cases}$$

$$v_{RB}'(420) = \begin{cases} 0.5 \times (v + 0.5), & \text{yChromaOffset=COSITED_EVEN} \\ 0.5 \times v, & \text{yChromaOffset=MIDPOINT} \end{cases}$$

16.3.9. Sampler $Y'C_BC_R$ Conversion

Sampler $Y'C_BC_R$ conversion performs the following operations, which an implementation **may** combine into a single mathematical operation:

- Sampler $Y'C_BC_R$ Range Expansion
- Sampler $Y'C_BC_R$ Model Conversion

Sampler $Y'C_BC_R$ Range Expansion

Sampler $Y'C_BC_R$ range expansion is applied to color component values after all texel input operations which are not specific to sampler $Y'C_BC_R$ conversion. For example, the input values to this stage have been converted using the normal format conversion rules.

Sampler $Y'C_BC_R$ range expansion is not applied if `ycbcrModel` is `VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY`. That is, the shader receives the vector C'_{rgba} as output by the Component Swizzle stage without further modification.

For other values of `ycbcrModel`, range expansion is applied to the texel component values output by the Component Swizzle defined by the `components` member of VkSamplerYcbcrConversionCreateInfo. Range expansion applies independently to each component of the image. For the purposes of range expansion and $Y'C_BC_R$ model conversion, the R and B components contain color difference (chroma) values and the G component contains luma. The A component is not modified by sampler $Y'C_BC_R$ range expansion.

The range expansion to be applied is defined by the ycbcrRange member of the VkSamplerYcbcrConversionCreateInfo structure:

- If ycbcrRange is VK_SAMPLER_YCBCR_RANGE_ITU_FULL, the following transformations are applied:

$$Y' = C'_{rgba}[G]$$
$$C_B = C'_{rgba}[B] - \frac{2^{(n-1)}}{(2^n)-1}$$
$$C_R = C'_{rgba}[R] - \frac{2^{(n-1)}}{(2^n)-1}$$

> **NOTE**
>
> These formulae correspond to the “full range” encoding in the “Quantization schemes” chapter of the Khronos Data Format Specification.
>
> Should any future amendments be made to the ITU specifications from which these equations are derived, the formulae used by Vulkan **may** also be updated to maintain parity.

- If ycbcrRange is VK_SAMPLER_YCBCR_RANGE_ITU_NARROW, the following transformations are applied:

$$Y' = \frac{C'_{rgba}[G] \times (2^n - 1) - 16 \times 2^{n-8}}{219 \times 2^{n-8}}$$
$$C_B = \frac{C'_{rgba}[B] \times (2^n - 1) - 128 \times 2^{n-8}}{224 \times 2^{n-8}}$$
$$C_R = \frac{C'_{rgba}[R] \times (2^n - 1) - 128 \times 2^{n-8}}{224 \times 2^{n-8}}$$

> **NOTE**
>
> These formulae correspond to the “narrow range” encoding in the “Quantization schemes” chapter of the Khronos Data Format Specification.

- *n* is the bit-depth of the components in the format.

The precision of the operations performed during range expansion **must** be at least that of the source format.

An implementation **may** clamp the results of these range expansion operations such that Y' falls in the range [0,1], and/or such that C_B and C_R fall in the range [-0.5,0.5].

Sampler Y'$C_B C_R$ Model Conversion

The range-expanded values are converted between color models, according to the color model conversion specified in the ycbcrModel member:

VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY
: The color components are not modified by the color model conversion since they are assumed already to represent the desired color model in which the shader is operating; Y'$C_B C_R$ range expansion is also ignored.

`VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_IDENTITY`

The color components are not modified by the color model conversion and are assumed to be treated as though in $Y'C_BC_R$ form both in memory and in the shader; $Y'C_BC_R$ range expansion is applied to the components as for other $Y'C_BC_R$ models, with the vector (C_R,Y',C_B,A) provided to the shader.

`VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_709`

The color components are transformed from a $Y'C_BC_R$ representation to an R'G'B' representation as described in the "BT.709 $Y'C_BC_R$ conversion" section of the Khronos Data Format Specification.

`VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_601`

The color components are transformed from a $Y'C_BC_R$ representation to an R'G'B' representation as described in the "BT.601 $Y'C_BC_R$ conversion" section of the Khronos Data Format Specification.

`VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_2020`

The color components are transformed from a $Y'C_BC_R$ representation to an R'G'B' representation as described in the "BT.2020 $Y'C_BC_R$ conversion" section of the Khronos Data Format Specification.

In this operation, each output component is dependent on each input component.

An implementation **may** clamp the R'G'B' results of these conversions to the range [0,1].

The precision of the operations performed during model conversion **must** be at least that of the source format.

The alpha component is not modified by these model conversions.

NOTE

Sampling operations in a non-linear color space can introduce color and intensity shifts at sharp transition boundaries. To avoid this issue, the technically precise color correction sequence described in the "Introduction to Color Conversions" chapter of the Khronos Data Format Specification may be performed as follows:

- Calculate the unnormalized texel coordinates corresponding to the desired sample position.
- For a `minFilter` or `magFilter` of `VK_FILTER_NEAREST`:
  1. Calculate (i,j) for the sample location as described under the "nearest filtering" formulae in (u,v,w,a) to (i,j,k,l,n) Transformation and Array Layer Selection
  2. Calculate the normalized texel coordinates corresponding to these integer coordinates.
  3. Sample using sampler $Y'C_BC_R$ conversion at this location.
- For a `minFilter` or `magFilter` of `VK_FILTER_LINEAR`:
  1. Calculate ($i_{[0,1]}$,$j_{[0,1]}$) for the sample location as described under the "linear filtering" formulae in (u,v,w,a) to (i,j,k,l,n) Transformation and Array Layer Selection

2. Calculate the normalized texel coordinates corresponding to these integer coordinates.
3. Sample using sampler $Y'C_BC_R$ conversion at each of these locations.
4. Convert the non-linear A'R'G'B' outputs of the $Y'C_BC_R$ conversions to linear ARGB values as described in the "Transfer Functions" chapter of the Khronos Data Format Specification.
5. Interpolate the linear ARGB values using the α and β values described in the "linear filtering" section of (u,v,w,a) to (i,j,k,l,n) Transformation and Array Layer Selection and the equations in Texel Filtering.

The additional calculations and, especially, additional number of sampling operations in the `VK_FILTER_LINEAR` case can be expected to have a performance impact compared with using the outputs directly. Since the variations from "correct" results are subtle for most content, the application author should determine whether a more costly implementation is strictly necessary.

If `chromaFilter`, and `minFilter` or `magFilter` are both `VK_FILTER_NEAREST`, these operations are redundant and sampling using sampler $Y'C_BC_R$ conversion at the desired sample coordinates will produce the "correct" results without further processing.

16.4. Texel Output Operations

Texel output instructions are SPIR-V image instructions that write to an image. *Texel output operations* are a set of steps that are performed on state, coordinates, and texel values while processing a texel output instruction, and which are common to some or all texel output instructions. They include the following steps, which are performed in the listed order:

- Validation operations
  - Format validation
  - Type validation
  - Coordinate validation
  - Sparse validation
- Texel output format conversion

16.4.1. Texel Output Validation Operations

Texel output validation operations inspect instruction/image state or coordinates, and in certain circumstances cause the write to have no effect. There are a series of validations that the texel undergoes.

Texel Format Validation

If the image format of the `OpTypeImage` is not compatible with the `VkImageView`'s `format`, the write causes the contents of the image's memory to become undefined.

Texel Type Validation

If the `Sampled Type` of the `OpTypeImage` does not match the SPIR-V Type, the write causes the value of the texel to become undefined. For integer types, if the signedness of the access does not match the signedness of the accessed resource, the write causes the value of the texel to become undefined.

16.4.2. Integer Texel Coordinate Validation

The integer texel coordinates are validated according to the same rules as for texel input coordinate validation.

If the texel fails integer texel coordinate validation, then the write has no effect.

16.4.3. Sparse Texel Operation

If the texel attempts to write to an unbound region of a sparse image, the texel is a sparse unbound texel. In such a case, if the VkPhysicalDeviceSparseProperties::`residencyNonResidentStrict` property is `VK_TRUE`, the sparse unbound texel write has no effect. If `residencyNonResidentStrict` is `VK_FALSE`, the write **may** have a side effect that becomes visible to other accesses to unbound texels in any resource, but will not be visible to any device memory allocated by the application.

16.4.4. Texel Output Format Conversion

If the image format is sRGB, a linear to sRGB conversion is applied to the R, G, and B components as described in the “sRGB EOTF” section of the Khronos Data Format Specification. The A component, if present, is unchanged.

Texels then undergo a format conversion from the floating-point, signed, or unsigned integer type of the texel data to the VkFormat of the image view. If the number of components in the texel data is larger than the number of components in the format, additional components are discarded.

Each component is converted based on its type and size (as defined in the Format Definition section for each VkFormat). Floating-point outputs are converted as described in Floating-Point Format Conversions and Fixed-Point Data Conversion. Integer outputs are converted such that their value is preserved. The converted value of any integer that cannot be represented in the target format is undefined.

If the `VkImageView` format has an X component in its format description, undefined values are written to those bits.

If the underlying `VkImage` format has an X component in its format description, undefined values are also written to those bits, even if result format conversion produces a valid value for those bits because the `VkImageView` format is different.

16.5. Normalized Texel Coordinate Operations

If the image sampler instruction provides normalized texel coordinates, some of the following operations are performed.

16.5.1. Projection Operation

For `Proj` image operations, the normalized texel coordinates (s,t,r,q,a) and (if present) the D_{ref} coordinate are transformed as follows:

$$
\begin{aligned}
s &= \frac{s}{q}, && \text{for 1D, 2D, or 3D image} \\
t &= \frac{t}{q}, && \text{for 2D or 3D image} \\
r &= \frac{r}{q}, && \text{for 3D image} \\
D_{ref} &= \frac{D_{ref}}{q}, && \text{if provided}
\end{aligned}
$$

16.5.2. Derivative Image Operations

Derivatives are used for LOD selection. These derivatives are either implicit (in an `ImplicitLod` image instruction in a mesh, task, compute, or fragment shader) or explicit (provided explicitly by shader to the image instruction in any shader).

For implicit derivatives image instructions, the derivatives of texel coordinates are calculated in the same manner as derivative operations. That is:

$$
\begin{aligned}
\partial s/\partial x &= dPdx(s), & \partial s/\partial y &= dPdy(s), && \text{for 1D, 2D, Cube, or 3D image} \\
\partial t/\partial x &= dPdx(t), & \partial t/\partial y &= dPdy(t), && \text{for 2D, Cube, or 3D image} \\
\partial r/\partial x &= dPdx(r), & \partial r/\partial y &= dPdy(r), && \text{for Cube or 3D image}
\end{aligned}
$$

Partial derivatives not defined above for certain image dimensionalities are set to zero.

For explicit LOD image instructions, if the **optional** SPIR-V operand `Grad` is provided, then the operand values are used for the derivatives. The number of components present in each derivative for a given image dimensionality matches the number of partial derivatives computed above.

If the **optional** SPIR-V operand `Lod` is provided, then derivatives are set to zero, the cube map derivative transformation is skipped, and the scale factor operation is skipped. Instead, the floating-point scalar coordinate is directly assigned to λ_{base} as described in LOD Operation.

16.5.3. Cube Map Face Selection and Transformations

For cube map image instructions, the (s,t,r) coordinates are treated as a direction vector (r_x,r_y,r_z). The direction vector is used to select a cube map face. The direction vector is transformed to a per-face texel coordinate system (s_{face},t_{face}), The direction vector is also used to transform the derivatives to per-face derivatives.

16.5.4. Cube Map Face Selection

The direction vector selects one of the cube map's faces based on the largest magnitude coordinate direction (the major axis direction). Since two or more coordinates **can** have identical magnitude,

the implementation **must** have rules to disambiguate this situation.

The rules **should** have as the first rule that r_z wins over r_y and r_x, and the second rule that r_y wins over r_x. An implementation **may** choose other rules, but the rules **must** be deterministic and depend only on (r_x,r_y,r_z).

The layer number (corresponding to a cube map face), the coordinate selections for s_c, t_c, r_c, and the selection of derivatives, are determined by the major axis direction as specified in the following two tables.

Table 23. Cube Map Face and Coordinate Selection

Major Axis Direction	**Layer Number**	**Cube Map Face**	**s_c**	**t_c**	**r_c**
$+r_x$	0	Positive X	$-r_z$	$-r_y$	r_x
$-r_x$	1	Negative X	$+r_z$	$-r_y$	r_x
$+r_y$	2	Positive Y	$+r_x$	$+r_z$	r_y
$-r_y$	3	Negative Y	$+r_x$	$-r_z$	r_y
$+r_z$	4	Positive Z	$+r_x$	$-r_y$	r_z
$-r_z$	5	Negative Z	$-r_x$	$-r_y$	r_z

Table 24. Cube Map Derivative Selection

Major Axis Direction	**$\partial s_c / \partial x$**	**$\partial s_c / \partial y$**	**$\partial t_c / \partial x$**	**$\partial t_c / \partial y$**	**$\partial r_c / \partial x$**	**$\partial r_c / \partial y$**
$+r_x$	$-\partial r_z / \partial x$	$-\partial r_z / \partial y$	$-\partial r_y / \partial x$	$-\partial r_y / \partial y$	$+\partial r_x / \partial x$	$+\partial r_x / \partial y$
$-r_x$	$+\partial r_z / \partial x$	$+\partial r_z / \partial y$	$-\partial r_y / \partial x$	$-\partial r_y / \partial y$	$-\partial r_x / \partial x$	$-\partial r_x / \partial y$
$+r_y$	$+\partial r_x / \partial x$	$+\partial r_x / \partial y$	$+\partial r_z / \partial x$	$+\partial r_z / \partial y$	$+\partial r_y / \partial x$	$+\partial r_y / \partial y$
$-r_y$	$+\partial r_x / \partial x$	$+\partial r_x / \partial y$	$-\partial r_z / \partial x$	$-\partial r_z / \partial y$	$-\partial r_y / \partial x$	$-\partial r_y / \partial y$
$+r_z$	$+\partial r_x / \partial x$	$+\partial r_x / \partial y$	$-\partial r_y / \partial x$	$-\partial r_y / \partial y$	$+\partial r_z / \partial x$	$+\partial r_z / \partial y$
$-r_z$	$-\partial r_x / \partial x$	$-\partial r_x / \partial y$	$-\partial r_y / \partial x$	$-\partial r_y / \partial y$	$-\partial r_z / \partial x$	$-\partial r_z / \partial y$

16.5.5. Cube Map Coordinate Transformation

$$s_{face} = \frac{1}{2} \times \frac{s_c}{|r_c|} + \frac{1}{2}$$
$$t_{face} = \frac{1}{2} \times \frac{t_c}{|r_c|} + \frac{1}{2}$$

16.5.6. Cube Map Derivative Transformation

The partial derivatives of the Cube Map Coordinate Transformations can be computed as:

$$\begin{aligned}\frac{\partial s_{face}}{\partial x} &= \frac{\partial}{\partial x}\left(\frac{1}{2} \times \frac{s_c}{|r_c|} + \frac{1}{2}\right) \\ &= \frac{1}{2} \times \frac{\partial}{\partial x}\left(\frac{s_c}{|r_c|}\right) \\ &= \frac{1}{2} \times \left(\frac{|r_c| \times \partial s_c / \partial x - s_c \times \partial r_c / \partial x}{(r_c)^2}\right)\end{aligned}$$

The other derivatives are simplified similarly, resulting in

$$\begin{aligned}\frac{\partial s_{face}}{\partial y} &= \frac{1}{2} \times \left(\frac{|r_c| \times \partial s_c / \partial y - s_c \times \partial r_c / \partial y}{(r_c)^2}\right) \\ \frac{\partial t_{face}}{\partial x} &= \frac{1}{2} \times \left(\frac{|r_c| \times \partial t_c / \partial x - t_c \times \partial r_c / \partial x}{(r_c)^2}\right) \\ \frac{\partial t_{face}}{\partial y} &= \frac{1}{2} \times \left(\frac{|r_c| \times \partial t_c / \partial y - t_c \times \partial r_c / \partial y}{(r_c)^2}\right)\end{aligned}$$

16.5.7. Scale Factor Operation, LOD Operation and Image Level(s) Selection

LOD selection **can** be either explicit (provided explicitly by the image instruction) or implicit (determined from a scale factor calculated from the derivatives). The LOD **must** be computed with `mipmapPrecisionBits` of accuracy.

Scale Factor Operation

The magnitude of the derivatives are calculated by:

$m_{ux} = |\partial s/\partial x| \times w_{base}$

$m_{vx} = |\partial t/\partial x| \times h_{base}$

$m_{wx} = |\partial r/\partial x| \times d_{base}$

$m_{uy} = |\partial s/\partial y| \times w_{base}$

$m_{vy} = |\partial t/\partial y| \times h_{base}$

$m_{wy} = |\partial r/\partial y| \times d_{base}$

where:

$\partial t/\partial x = \partial t/\partial y = 0$ (for 1D images)

$\partial r/\partial x = \partial r/\partial y = 0$ (for 1D, 2D or Cube images)

and:

w_{base} = image.w

h_{base} = image.h

d_{base} = image.d

(for the `baseMipLevel`, from the image descriptor).

A point sampled in screen space has an elliptical footprint in texture space. The minimum and maximum scale factors (ρ_{min}, ρ_{max}) **should** be the minor and major axes of this ellipse.

The *scale factors* ρ_x and ρ_y, calculated from the magnitude of the derivatives in x and y, are used to compute the minimum and maximum scale factors.

ρ_x and ρ_y **may** be approximated with functions f_x and f_y, subject to the following constraints:

$$f_x \text{is continuous and monotonically increasing in each of} m_{ux},\ m_{vx},\ \text{and} m_{wx}$$
$$f_y \text{is continuous and monotonically increasing in each of} m_{uy},\ m_{vy},\ \text{and} m_{wy}$$

$$\max(|m_{ux}|, |m_{vx}|, |m_{wx}|) \le f_x \le \sqrt{2}(|m_{ux}| + |m_{vx}| + |m_{wx}|)$$
$$\max(|m_{uy}|, |m_{vy}|, |m_{wy}|) \le f_y \le \sqrt{2}(|m_{uy}| + |m_{vy}| + |m_{wy}|)$$

The minimum and maximum scale factors (ρ_{min},ρ_{max}) are determined by:

$\rho_{max} = \max(\rho_x, \rho_y)$

$\rho_{min} = \min(\rho_x, \rho_y)$

The ratio of anisotropy is determined by:

$\eta = \min(\rho_{max}/\rho_{min}, \max_{Aniso})$

where:

$sampler.max_{Aniso}$ = `maxAnisotropy` (from sampler descriptor)

$limits.max_{Aniso}$ = `maxSamplerAnisotropy` (from physical device limits)

$max_{Aniso} = \min(sampler.max_{Aniso}, limits.max_{Aniso})$

If $\rho_{max} = \rho_{min} = 0$, then all the partial derivatives are zero, the fragment's footprint in texel space is a point, and η **should** be treated as 1. If $\rho_{max} \neq 0$ and $\rho_{min} = 0$ then all partial derivatives along one axis are zero, the fragment's footprint in texel space is a line segment, and η **should** be treated as max_{Aniso}. However, anytime the footprint is small in texel space the implementation **may** use a smaller value of η, even when ρ_{min} is zero or close to zero. If either VkPhysicalDeviceFeatures ::`samplerAnisotropy` or VkSamplerCreateInfo::`anisotropyEnable` are `VK_FALSE`, max_{Aniso} is set to 1.

If $\eta = 1$, sampling is isotropic. If $\eta > 1$, sampling is anisotropic.

The sampling rate (N) is derived as:

$N = \lceil \eta \rceil$

An implementation **may** round N up to the nearest supported sampling rate. An implementation **may** use the value of N as an approximation of η.

LOD Operation

The LOD parameter λ is computed as follows:

$$\lambda_{base}(x, y) = \begin{cases} shaderOp.Lod & \text{(from optional SPIR-V operand)} \\ \log_2\left(\frac{\rho_{max}}{\eta}\right) & \text{otherwise} \end{cases}$$

$$\lambda'(x, y) = \lambda_{base} + \text{clamp}\left(sampler.bias + shaderOp.bias, -maxSamplerLodBias, maxSamplerLodBias\right)$$

$$\lambda = \begin{cases} lod_{max}, & \lambda' > lod_{max} \\ \lambda', & lod_{min} \leq \lambda' \leq lod_{max} \\ lod_{min}, & \lambda' < lod_{min} \\ undefined, & lod_{min} > lod_{max} \end{cases}$$

where:

$$sampler.bias = mipLodBias \quad \text{(from sampler descriptor)}$$

$$shaderOp.bias = \begin{cases} Bias & \text{(from optional SPIR-V operand)} \\ 0 & \text{otherwise} \end{cases}$$

$$sampler.lod_{min} = minLod \quad \text{(from sampler descriptor)}$$

$$shaderOp.lod_{min} = \begin{cases} MinLod & \text{(from optional SPIR-V operand)} \\ 0 & \text{otherwise} \end{cases}$$

$$lod_{min} = \max(sampler.lod_{min}, shaderOp.lod_{min})$$

$$lod_{max} = maxLod \quad \text{(from sampler descriptor)}$$

and maxSamplerLodBias is the value of the VkPhysicalDeviceLimits feature `maxSamplerLodBias`.

Image Level(s) Selection

The image level(s) d, d_{hi}, and d_{lo} which texels are read from are determined by an image-level parameter d_l, which is computed based on the LOD parameter, as follows:

$$d_l = \begin{cases} nearest(d'), & \text{mipmapMode is VK_SAMPLER_MIPMAP_MODE_NEAREST} \\ d', & \text{otherwise} \end{cases}$$

where:

$$d' = max(level_{base} + \text{clamp}(\lambda, 0, q), minLod_{imageView})$$

$$nearest(d') = \begin{cases} \lceil d' + 0.5 \rceil - 1, & \text{preferred} \\ \lfloor d' + 0.5 \rfloor, & \text{alternative} \end{cases}$$

and:

$$minLod_{imageView} = \begin{cases} minLodFloat_{imageView}, & \text{preferred} \\ minLodInteger_{imageView}, & \text{alternative} \end{cases}$$
$$level_{base} = baseMipLevel$$
$$q = levelCount - 1$$

`baseMipLevel` and `levelCount` are taken from the `subresourceRange` of the image view.

$minLod_{imageView}$ **must** be less or equal to $level_{base}$ + q.

If the sampler's `mipmapMode` is `VK_SAMPLER_MIPMAP_MODE_NEAREST`, then the level selected is $d = d_l$.

If the sampler's `mipmapMode` is `VK_SAMPLER_MIPMAP_MODE_LINEAR`, two neighboring levels are selected:

$$d_{hi} = \lfloor d_l \rfloor$$
$$d_{lo} = min(d_{hi} + 1, level_{base} + q)$$
$$\delta = d_l - d_{hi}$$

δ is the fractional value, quantized to the number of mipmap precision bits, used for linear filtering between levels.

16.5.8. (s,t,r,q,a) to (u,v,w,a) Transformation

The normalized texel coordinates are scaled by the image level dimensions and the array layer is selected.

This transformation is performed once for each level used in filtering (either d, or d_{hi} and d_{lo}).

$$u(x,\ y) = s(x,\ y) \times width_{scale} + \Delta_i$$

$$v(x,\ y) = \begin{cases} 0 & \text{for 1D images} \\ t(x,\ y) \times height_{scale} + \Delta_j & \text{otherwise} \end{cases}$$

$$w(x,\ y) = \begin{cases} 0 & \text{for 2D or Cube images} \\ r(x,\ y) \times depth_{scale} + \Delta_k & \text{otherwise} \end{cases}$$

$$a(x,\ y) = \begin{cases} a(x,\ y) & \text{for array images} \\ 0 & \text{otherwise} \end{cases}$$

where:

$width_{scale} = width_{level}$

$height_{scale} = height_{level}$

$depth_{scale} = depth_{level}$

and where (Δ_i, Δ_j, Δ_k) are taken from the image instruction if it includes a `ConstOffset` or `Offset` operand, otherwise they are taken to be zero.

Operations then proceed to Unnormalized Texel Coordinate Operations.

16.6. Unnormalized Texel Coordinate Operations

16.6.1. (u,v,w,a) to (i,j,k,l,n) Transformation and Array Layer Selection

The unnormalized texel coordinates are transformed to integer texel coordinates relative to the selected mipmap level.

The layer index l is computed as:

l = clamp(RNE(a), 0, `layerCount` - 1) + `baseArrayLayer`

where `layerCount` is the number of layers in the image subresource range of the image view, `baseArrayLayer` is the first layer from the subresource range, and where:

$$\text{RNE}\,(a) = \begin{cases} \text{roundTiesToEven}\,(a) & \text{preferred, from IEEE Std 754-2008 Floating-Point Arithmetic} \\ \lfloor a + 0.5 \rfloor & \text{alternative} \end{cases}$$

The sample index n is assigned the value 0.

Nearest filtering (`VK_FILTER_NEAREST`) computes the integer texel coordinates that the unnormalized coordinates lie within:

$$
\begin{aligned}
i &= \lfloor u + shift \rfloor \\
j &= \lfloor v + shift \rfloor \\
k &= \lfloor w + shift \rfloor
\end{aligned}
$$

where:

shift = 0.0

Linear filtering (VK_FILTER_LINEAR) computes a set of neighboring coordinates which bound the unnormalized coordinates. The integer texel coordinates are combinations of i_0 or i_1, j_0 or j_1, k_0 or k_1, as well as weights α, β, and γ.

$$
\begin{aligned}
i_0 &= \lfloor u - shift \rfloor \\
i_1 &= i_0 + 1 \\
j_0 &= \lfloor v - shift \rfloor \\
j_1 &= j_0 + 1 \\
k_0 &= \lfloor w - shift \rfloor \\
k_1 &= k_0 + 1
\end{aligned}
$$

$$
\begin{aligned}
\alpha &= \text{frac}\,(u - shift) \\
\beta &= \text{frac}\,(v - shift) \\
\gamma &= \text{frac}\,(w - shift)
\end{aligned}
$$

where:

shift = 0.5

and where:

$$
\text{frac}\,(x) = x - \lfloor x \rfloor
$$

where the number of fraction bits retained is specified by VkPhysicalDeviceLimits ::subTexelPrecisionBits.

Cubic filtering (VK_FILTER_CUBIC_EXT) computes a set of neighboring coordinates which bound the unnormalized coordinates. The integer texel coordinates are combinations of i_0, i_1, i_2 or i_3, j_0, j_1, j_2 or j_3, k_0, k_1, k_2 or k_3, as well as weights α, β, and γ.

$$
\begin{aligned}
i_0 &= \lfloor u - \frac{3}{2} \rfloor & i_1 &= i_0 + 1 & i_2 &= i_1 + 1 & i_3 &= i_2 + 1 \\
j_0 &= \lfloor v - \frac{3}{2} \rfloor & j_1 &= j_0 + 1 & j_2 &= j_1 + 1 & j_3 &= j_2 + 1 \\
k_0 &= \lfloor w - \frac{3}{2} \rfloor & k_1 &= k_0 + 1 & k_2 &= k_1 + 1 & k_3 &= k_2 + 1
\end{aligned}
$$

$$\alpha = \text{frac}\left(u - \frac{1}{2}\right)$$

$$\beta = \text{frac}\left(v - \frac{1}{2}\right)$$

$$\gamma = \text{frac}\left(w - \frac{1}{2}\right)$$

where:

$$\text{frac}(x) = x - \lfloor x \rfloor$$

where the number of fraction bits retained is specified by `VkPhysicalDeviceLimits::subTexelPrecisionBits`.

16.7. Integer Texel Coordinate Operations

The `OpImageFetch` and `OpImageFetchSparse` SPIR-V instructions **may** supply a LOD from which texels are to be fetched using the optional SPIR-V operand `Lod`. Other integer-coordinate operations **must** not. If the `Lod` is provided then it **must** be an integer.

The image level selected is:

$$d = level_{base} + \begin{cases} Lod & \text{(from optional SPIR-V operand)} \\ 0 & \text{otherwise} \end{cases}$$

If d does not lie in the range [`baseMipLevel`, `baseMipLevel` + `levelCount`) or d is less than $\text{minLodInteger}_{\text{imageView}}$, then any values fetched are zero if the `robustImageAccess2` feature is enabled, otherwise are undefined, and any writes (if supported) are discarded.

16.8. Image Sample Operations

16.8.1. Wrapping Operation

If the used sampler was created without `VK_SAMPLER_CREATE_NON_SEAMLESS_CUBE_MAP_BIT_EXT`, `Cube` images ignore the wrap modes specified in the sampler. Instead, if `VK_FILTER_NEAREST` is used within a mip level then `VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE` is used, and if `VK_FILTER_LINEAR` is used within a mip level then sampling at the edges is performed as described earlier in the Cube map edge handling section.

The first integer texel coordinate i is transformed based on the `addressModeU` parameter of the sampler.

$$i = \begin{cases} i \bmod size & \text{for repeat} \\ (size-1) - \text{mirror}\left((i \bmod (2 \times size)) - size\right) & \text{for mirrored repeat} \\ \text{clamp}\left(i,\ 0,\ size-1\right) & \text{for clamp to edge} \\ \text{clamp}\left(i,\ -1,\ size\right) & \text{for clamp to border} \\ \text{clamp}\left(\text{mirror}\left(i\right),\ 0,\ size-1\right) & \text{for mirror clamp to edge} \end{cases}$$

where:

$$\text{mirror}\left(n\right) = \begin{cases} n & \text{for } n \geq 0 \\ -(1+n) & \text{otherwise} \end{cases}$$

j (for 2D and Cube image) and k (for 3D image) are similarly transformed based on the `addressModeV` and `addressModeW` parameters of the sampler, respectively.

16.8.2. Texel Gathering

SPIR-V instructions with `Gather` in the name return a vector derived from 4 texels in the base level of the image view. The rules for the `VK_FILTER_LINEAR` minification filter are applied to identify the four selected texels. Each texel is then converted to an RGBA value according to conversion to RGBA and then swizzled. A four-component vector is then assembled by taking the component indicated by the `Component` value in the instruction from the swizzled color value of the four texels. If the operation does not use the `ConstOffsets` image operand then the four texels form the 2 × 2 rectangle used for texture filtering:

$$\begin{aligned} \tau[R] &= \tau_{i0\,j1}[level_{base}][comp] \\ \tau[G] &= \tau_{i1\,j1}[level_{base}][comp] \\ \tau[B] &= \tau_{i1\,j0}[level_{base}][comp] \\ \tau[A] &= \tau_{i0\,j0}[level_{base}][comp] \end{aligned}$$

If the operation does use the `ConstOffsets` image operand then the offsets allow a custom filter to be defined:

$$\begin{aligned} \tau[R] &= \tau_{i0\,j0\,+\,\Delta_0}[level_{base}][comp] \\ \tau[G] &= \tau_{i0\,j0\,+\,\Delta_1}[level_{base}][comp] \\ \tau[B] &= \tau_{i0\,j0\,+\,\Delta_2}[level_{base}][comp] \\ \tau[A] &= \tau_{i0\,j0\,+\,\Delta_3}[level_{base}][comp] \end{aligned}$$

where:

$$\tau[level_{base}][comp] = \begin{cases} \tau[level_{base}][R], & \text{for } comp = 0 \\ \tau[level_{base}][G], & \text{for } comp = 1 \\ \tau[level_{base}][B], & \text{for } comp = 2 \\ \tau[level_{base}][A], & \text{for } comp = 3 \end{cases}$$

$comp$ from SPIR-V operand Component

`OpImage*Gather` **must** not be used on a sampled image with sampler $Y'C_BC_R$ conversion enabled.

If $level_{base} < minLodInteger_{imageView}$, then any values fetched are zero if the `robustImageAccess2` feature is enabled. Otherwise values are undefined.

16.8.3. Texel Filtering

Texel filtering is first performed for each level (either d or d_{hi} and d_{lo}).

If λ is less than or equal to zero, the texture is said to be *magnified*, and the filter mode within a mip level is selected by the `magFilter` in the sampler. If λ is greater than zero, the texture is said to be *minified*, and the filter mode within a mip level is selected by the `minFilter` in the sampler.

Texel Nearest Filtering

Within a mip level, `VK_FILTER_NEAREST` filtering selects a single value using the (i, j, k) texel coordinates, with all texels taken from layer l.

$$\tau[level] = \begin{cases} \tau_{ijk}[level], & \text{for 3D image} \\ \tau_{ij}[level], & \text{for 2D or Cube image} \\ \tau_{i}[level], & \text{for 1D image} \end{cases}$$

Texel Linear Filtering

Within a mip level, `VK_FILTER_LINEAR` filtering combines 8 (for 3D), 4 (for 2D or Cube), or 2 (for 1D) texel values, together with their linear weights. The linear weights are derived from the fractions computed earlier:

$$\begin{aligned} w_{i_0} &= (1-\alpha) \\ w_{i_1} &= (\alpha) \\ w_{j_0} &= (1-\beta) \\ w_{j_1} &= (\beta) \\ w_{k_0} &= (1-\gamma) \\ w_{k_1} &= (\gamma) \end{aligned}$$

The values of multiple texels, together with their weights, are combined using a weighted average to produce a filtered value:

$$\tau_{3D} = \sum_{k=k_0}^{k_1} \sum_{j=j_0}^{j_1} \sum_{i=i_0}^{i_1} (w_i)(w_j)(w_k)\tau_{ijk}$$

$$\tau_{2D} = \sum_{j=j_0}^{j_1} \sum_{i=i_0}^{i_1} (w_i)(w_j)\tau_{ij}$$

$$\tau_{1D} = \sum_{i=i_0}^{i_1} (w_i)\tau_i$$

Texel Cubic Filtering

Within a mip level, VK_FILTER_CUBIC_EXT, filtering computes a weighted average of 64 (for 3D), 16 (for 2D), or 4 (for 1D) texel values, together with their Catmull-Rom weights.

Catmull-Rom weights are derived from the fractions computed earlier.

$$\begin{bmatrix} w_{i_0} & w_{i_1} & w_{i_2} & w_{i_3} \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 1 & \alpha & \alpha^2 & \alpha^3 \end{bmatrix}\begin{bmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{bmatrix}$$

$$\begin{bmatrix} w_{j_0} & w_{j_1} & w_{j_2} & w_{j_3} \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 1 & \beta & \beta^2 & \beta^3 \end{bmatrix}\begin{bmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{bmatrix}$$

$$\begin{bmatrix} w_{k_0} & w_{k_1} & w_{k_2} & w_{k_3} \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 1 & \gamma & \gamma^2 & \gamma^3 \end{bmatrix}\begin{bmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{bmatrix}$$

The values of multiple texels, together with their weights, are combined to produce a filtered value.

The VkSamplerReductionModeCreateInfo::reductionMode **can** control the process by which multiple texels, together with their weights, are combined to produce a filtered texture value.

When the reductionMode is set (explicitly or implicitly) to VK_SAMPLER_REDUCTION_MODE_WEIGHTED_AVERAGE , a weighted average is computed:

$$\tau_{3D} = \sum_{k=j_0}^{k_3} \sum_{j=j_0}^{j_3} \sum_{i=i_0}^{i_3} (w_i)(w_j)(w_k)\tau_{ijk}$$

$$\tau_{2D} = \sum_{j=j_0}^{j_3} \sum_{i=i_0}^{i_3} (w_i)(w_j)\tau_{ij}$$

$$\tau_{1D} = \sum_{i=i_0}^{i_3} (w_i)\tau_i$$

Texel Mipmap Filtering

VK_SAMPLER_MIPMAP_MODE_NEAREST filtering returns the value of a single mipmap level,

$\tau = \tau[d]$.

VK_SAMPLER_MIPMAP_MODE_LINEAR filtering combines the values of multiple mipmap levels (τ[hi] and τ[lo]), together with their linear weights.

The linear weights are derived from the fraction computed earlier:

$$w_{hi} = (1 - \delta)$$
$$w_{lo} = (\delta)$$

The values of multiple mipmap levels together with their linear weights, are combined using a weighted average to produce a final filtered value:

$$\tau = (w_{hi})\tau[hi] + (w_{lo})\tau[lo]$$

Texel Anisotropic Filtering

Anisotropic filtering is enabled by the `anisotropyEnable` in the sampler. When enabled, the image filtering scheme accounts for a degree of anisotropy.

The particular scheme for anisotropic texture filtering is implementation-dependent. Implementations **should** consider the `magFilter`, `minFilter` and `mipmapMode` of the sampler to control the specifics of the anisotropic filtering scheme used. In addition, implementations **should** consider `minLod` and `maxLod` of the sampler.

NOTE

For historical reasons, vendor implementations of anisotropic filtering interpret these sampler parameters in different ways, particularly in corner cases such as `magFilter`, `minFilter` of `NEAREST` or `maxAnisotropy` equal to 1.0. Applications should not expect consistent behavior in such cases, and should use anisotropic filtering only with parameters which are expected to give a quality improvement relative to `LINEAR` filtering.

The following describes one particular approach to implementing anisotropic filtering for the 2D Image case; implementations **may** choose other methods:

Given a `magFilter`, `minFilter` of `VK_FILTER_LINEAR` and a `mipmapMode` of `VK_SAMPLER_MIPMAP_MODE_NEAREST`:

Instead of a single isotropic sample, N isotropic samples are sampled within the image footprint of the image level d to approximate an anisotropic filter. The sum $\tau_{2Daniso}$ is defined using the single isotropic τ_{2D}(u,v) at level d.

$$\tau_{2Daniso} = \frac{1}{N}\sum_{i=1}^{N} \tau_{2D}\left(u\left(x - \frac{1}{2} + \frac{i}{N+1},\ y\right),\ v\left(x - \frac{1}{2} + \frac{i}{N+1},\ y\right)\right), \qquad \text{when } \rho_x > \rho_y$$

$$\tau_{2Daniso} = \frac{1}{N}\sum_{i=1}^{N} \tau_{2D}\left(u\left(x,\ y - \frac{1}{2} + \frac{i}{N+1}\right),\ v\left(x,\ y - \frac{1}{2} + \frac{i}{N+1}\right)\right), \qquad \text{when } \rho_y \geq \rho_x$$

16.9. Image Operation Steps

Each step described in this chapter is performed by a subset of the image instructions:

- Texel Input Validation Operations, Format Conversion, Texel Replacement, Conversion to RGBA, and Component Swizzle: Performed by all instructions except `OpImageWrite`.
- Depth Comparison: Performed by `OpImage*Dref` instructions.

- All Texel output operations: Performed by `OpImageWrite`.
- Projection: Performed by all `OpImage*Proj` instructions.
- Derivative Image Operations, Cube Map Operations, Scale Factor Operation, LOD Operation and Image Level(s) Selection, and Texel Anisotropic Filtering: Performed by all `OpImageSample*` and `OpImageSparseSample*` instructions.
- (s,t,r,q,a) to (u,v,w,a) Transformation, Wrapping, and (u,v,w,a) to (i,j,k,l,n) Transformation And Array Layer Selection: Performed by all `OpImageSample`, `OpImageSparseSample`, and `OpImage*Gather` instructions.
- Texel Gathering: Performed by `OpImage*Gather` instructions.
- Texel Filtering: Performed by all `OpImageSample*` and `OpImageSparseSample*` instructions.
- Sparse Residency: Performed by all `OpImageSparse*` instructions.

16.10. Image Query Instructions

16.10.1. Image Property Queries

`OpImageQuerySize`, `OpImageQuerySizeLod`, `OpImageQueryLevels`, and `OpImageQuerySamples` query properties of the image descriptor that would be accessed by a shader image operation. They return 0 if the bound descriptor is a null descriptor.

`OpImageQuerySizeLod` returns the size of the image level identified by the `Level of Detail` operand. If that level does not exist in the image, and the descriptor is not null, then the value returned is undefined.

16.10.2. LOD Query

`OpImageQueryLod` returns the Lod parameters that would be used in an image operation with the given image and coordinates. If the descriptor that would be accessed is a null descriptor then (0,0) is returned. Otherwise, the steps described in this chapter are performed as if for `OpImageSampleImplicitLod`, up to Scale Factor Operation, LOD Operation and Image Level(s) Selection. The return value is the vector (λ', d_l - $level_{base}$). These values **may** be subject to implementation-specific maxima and minima for very large, out-of-range values.

Chapter 17. Fragment Density Map Operations

17.1. Fragment Density Map Operations Overview

When a fragment is generated in a render pass that has a fragment density map attachment, its area is determined by the properties of the local framebuffer region that the fragment occupies. The framebuffer is divided into a uniform grid of these local regions, and their fragment area property is derived from the density map with the following operations:

- Fetch density value
  - Component swizzle
  - Component mapping
- Fragment area conversion
  - Fragment area filter
  - Fragment area clamp

17.2. Fetch Density Value

Each local framebuffer region at center coordinate (x,y) fetches a texel from the fragment density map at integer coordinates:

$$i = \lfloor \frac{x}{fragmentDensityTexelSize_{width}} \rfloor$$

$$j = \lfloor \frac{y}{fragmentDensityTexelSize_{height}} \rfloor$$

Where the size of each region in the framebuffer is:

$$fragmentDensityTexelSize'_{width} = 2^{\lceil \log_2 (\frac{framebuffer_{width}}{fragmentDensityMap_{width}}) \rceil}$$

$$fragmentDensityTexelSize'_{height} = 2^{\lceil \log_2 (\frac{framebuffer_{height}}{fragmentDensityMap_{height}}) \rceil}$$

This region is subject to the limits in `VkPhysicalDeviceFragmentDensityMapPropertiesEXT` and therefore the final region size is clamped:

$$fragmentDensityTexelSize_{width} = \text{clamp}(fragmentDensityTexelSize'_{width}, minFragmentDensityTexelSize_{width}, maxFragmentDensityTexelSize_{width})$$

$$fragmentDensityTexelSize_{height} = \text{clamp}(fragmentDensityTexelSize'_{height}, minFragmentDensityTexelSize_{height}, maxFragmentDensityTexelSize_{height})$$

When multiview is enabled for the render pass and the fragment density map attachment view was created with `layerCount` greater than `1`, the layer used for offsets and for fetching from the fragment density map is:

$$layer = baseArrayLayer + ViewIndex$$

Otherwise:

$$layer = baseArrayLayer$$

The texel fetched from the density map at (i,j,layer) is next converted to density with the following operations.

17.2.1. Component Swizzle

The `components` member of VkImageViewCreateInfo is applied to the fetched texel as defined in Image component swizzle.

17.2.2. Component Mapping

The swizzled texel's components are mapped to a density value:

$$densityValue_{xy} = (C'_r, C'_g)$$

17.3. Fragment Area Conversion

Fragment area for the framebuffer region is undefined if the density fetched is not a normalized floating-point value greater than `0.0`. Otherwise, the fetched fragment area for that region is derived as:

$$fragmentArea_{wh} = \frac{1.0}{densityValue_{xy}}$$

17.3.1. Fragment Area Filter

Optionally, the implementation **may** fetch additional density map texels in an implementation defined window around (i,j). The texels follow the standard conversion steps up to and including fragment area conversion.

A single fetched fragment area for the framebuffer region is chosen by the implementation and **must** have an area between the *min* and *max* areas of the fetched set.

17.3.2. Fragment Area Clamp

The implementation **may** clamp the fetched fragment area to one that it supports. The clamped fragment area **must** have a size less than or equal to the original fetched value. Implementations **may** vary the supported set of fragment areas per framebuffer region. Fragment area (1,1) **must** always be in the supported set.

NOTE	For example, if the fetched fragment area is (1,4) but the implementation only supports areas of {(1,1),(2,2)}, it could choose to clamp the area to (2,2) since it has the same size as (1,4). While this would produce fragments that have lower quality strictly in the x-axis, the overall density is maintained.

The clamped fragment area is assigned to the corresponding framebuffer region.

Chapter 18. Queries

Queries provide a mechanism to return information about the processing of a sequence of Vulkan commands. Query operations are asynchronous, and as such, their results are not returned immediately. Instead, their results, and their availability status are stored in a Query Pool. The state of these queries **can** be read back on the host, or copied to a buffer object on the device.

The supported query types are Occlusion Queries, Pipeline Statistics Queries, Result Status Queries, Video Encode Feedback Queries and Timestamp Queries. Performance Queries are supported if the associated extension is available. Transform Feedback Queries are supported if the associated extension is available.

Several additional queries with specific purposes associated with ray tracing are available if the corresponding extensions are supported, as described for VkQueryType.

18.1. Query Pools

Queries are managed using *query pool* objects. Each query pool is a collection of a specific number of queries of a particular type.

Query pools are represented by `VkQueryPool` handles:

```
// Provided by VK_VERSION_1_0
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkQueryPool)
```

To create a query pool, call:

```
// Provided by VK_VERSION_1_0
VkResult vkCreateQueryPool(
    VkDevice                                    device,
    const VkQueryPoolCreateInfo*                pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkQueryPool*                                pQueryPool);
```

- `device` is the logical device that creates the query pool.
- `pCreateInfo` is a pointer to a VkQueryPoolCreateInfo structure containing the number and type of queries to be managed by the pool.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pQueryPool` is a pointer to a VkQueryPool handle in which the resulting query pool object is returned.

Valid Usage

- VUID-vkCreateQueryPool-device-09663
  `device` **must** support at least one queue family with one of the

VK_QUEUE_VIDEO_ENCODE_BIT_KHR, VK_QUEUE_VIDEO_DECODE_BIT_KHR, VK_QUEUE_COMPUTE_BIT, or VK_QUEUE_GRAPHICS_BIT capabilities

Valid Usage (Implicit)

- VUID-vkCreateQueryPool-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateQueryPool-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkQueryPoolCreateInfo structure
- VUID-vkCreateQueryPool-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateQueryPool-pQueryPool-parameter
  pQueryPool **must** be a valid pointer to a VkQueryPool handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkQueryPoolCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkQueryPoolCreateInfo {
    VkStructureType                 sType;
    const void*                     pNext;
    VkQueryPoolCreateFlags          flags;
    VkQueryType                     queryType;
    uint32_t                        queryCount;
    VkQueryPipelineStatisticFlags   pipelineStatistics;
} VkQueryPoolCreateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- queryType is a VkQueryType value specifying the type of queries managed by the pool.
- queryCount is the number of queries managed by the pool.

- `pipelineStatistics` is a bitmask of VkQueryPipelineStatisticFlagBits specifying which counters will be returned in queries on the new pool, as described below in Pipeline Statistics Queries.

`pipelineStatistics` is ignored if `queryType` is not `VK_QUERY_TYPE_PIPELINE_STATISTICS`.

Valid Usage

- VUID-VkQueryPoolCreateInfo-queryType-00791
  If the `pipelineStatisticsQuery` feature is not enabled, `queryType` **must** not be `VK_QUERY_TYPE_PIPELINE_STATISTICS`
- VUID-VkQueryPoolCreateInfo-queryType-00792
  If `queryType` is `VK_QUERY_TYPE_PIPELINE_STATISTICS`, `pipelineStatistics` **must** be a valid combination of VkQueryPipelineStatisticFlagBits values
- VUID-VkQueryPoolCreateInfo-queryType-09534
  If `queryType` is `VK_QUERY_TYPE_PIPELINE_STATISTICS`, `pipelineStatistics` **must** not be zero
- VUID-VkQueryPoolCreateInfo-queryType-03222
  If `queryType` is `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, the `pNext` chain **must** include a VkQueryPoolPerformanceCreateInfoKHR structure
- VUID-VkQueryPoolCreateInfo-queryCount-02763
  `queryCount` **must** be greater than 0
- VUID-VkQueryPoolCreateInfo-queryType-07133
  If `queryType` is `VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR`, then the `pNext` chain **must** include a VkVideoProfileInfoKHR structure with `videoCodecOperation` specifying an encode operation
- VUID-VkQueryPoolCreateInfo-queryType-07906
  If `queryType` is `VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR`, then the `pNext` chain **must** include a VkQueryPoolVideoEncodeFeedbackCreateInfoKHR structure
- VUID-VkQueryPoolCreateInfo-queryType-07907
  If `queryType` is `VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR`, and the `pNext` chain includes a VkVideoProfileInfoKHR structure and a VkQueryPoolVideoEncodeFeedbackCreateInfoKHR structure, then VkQueryPoolVideoEncodeFeedbackCreateInfoKHR::`encodeFeedbackFlags` **must** not contain any bits that are not set in VkVideoEncodeCapabilitiesKHR::`supportedEncodeFeedbackFlags`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile described by VkVideoProfileInfoKHR and its `pNext` chain
- VUID-VkQueryPoolCreateInfo-pNext-10248
  If the `pNext` chain includes a VkVideoProfileInfoKHR structure and its `videoCodecOperation` member is `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR`, then the `videoEncodeAV1` feature **must** be enabled

Valid Usage (Implicit)

- VUID-VkQueryPoolCreateInfo-sType-sType

sType **must** be VK_STRUCTURE_TYPE_QUERY_POOL_CREATE_INFO

- VUID-VkQueryPoolCreateInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkQueryPoolPerformanceCreateInfoKHR, VkQueryPoolVideoEncodeFeedbackCreateInfoKHR, VkVideoDecodeAV1ProfileInfoKHR, VkVideoDecodeH264ProfileInfoKHR, VkVideoDecodeH265ProfileInfoKHR, VkVideoDecodeUsageInfoKHR, VkVideoEncodeAV1ProfileInfoKHR, VkVideoEncodeH264ProfileInfoKHR, VkVideoEncodeH265ProfileInfoKHR, VkVideoEncodeUsageInfoKHR, or VkVideoProfileInfoKHR
- VUID-VkQueryPoolCreateInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkQueryPoolCreateInfo-flags-zerobitmask
  flags **must** be 0
- VUID-VkQueryPoolCreateInfo-queryType-parameter
  queryType **must** be a valid VkQueryType value

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkQueryPoolCreateFlags;
```

VkQueryPoolCreateFlags is a bitmask type for setting a mask, but is currently reserved for future use.

The VkQueryPoolPerformanceCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_performance_query
typedef struct VkQueryPoolPerformanceCreateInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           queueFamilyIndex;
    uint32_t           counterIndexCount;
    const uint32_t*    pCounterIndices;
} VkQueryPoolPerformanceCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- queueFamilyIndex is the queue family index to create this performance query pool for.
- counterIndexCount is the length of the pCounterIndices array.
- pCounterIndices is a pointer to an array of indices into the vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR::pCounters to enable in this performance query pool.

Valid Usage

- VUID-VkQueryPoolPerformanceCreateInfoKHR-queueFamilyIndex-03236

queueFamilyIndex **must** be a valid queue family index of the device

- VUID-VkQueryPoolPerformanceCreateInfoKHR-performanceCounterQueryPools-03237
  The performanceCounterQueryPools feature **must** be enabled
- VUID-VkQueryPoolPerformanceCreateInfoKHR-pCounterIndices-03321
  Each element of pCounterIndices **must** be in the range of counters reported by vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR for the queue family specified in queueFamilyIndex

Valid Usage (Implicit)

- VUID-VkQueryPoolPerformanceCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_QUERY_POOL_PERFORMANCE_CREATE_INFO_KHR
- VUID-VkQueryPoolPerformanceCreateInfoKHR-pCounterIndices-parameter
  pCounterIndices **must** be a valid pointer to an array of counterIndexCount uint32_t values
- VUID-VkQueryPoolPerformanceCreateInfoKHR-counterIndexCount-arraylength
  counterIndexCount **must** be greater than 0

To query the number of passes required to query a performance query pool on a physical device, call:

```
// Provided by VK_KHR_performance_query
void vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR(
    VkPhysicalDevice                            physicalDevice,
    const VkQueryPoolPerformanceCreateInfoKHR*  pPerformanceQueryCreateInfo,
    uint32_t*                                   pNumPasses);
```

- physicalDevice is the handle to the physical device whose queue family performance query counter properties will be queried.
- pPerformanceQueryCreateInfo is a pointer to a VkQueryPoolPerformanceCreateInfoKHR of the performance query that is to be created.
- pNumPasses is a pointer to an integer related to the number of passes required to query the performance query pool, as described below.

The pPerformanceQueryCreateInfo member VkQueryPoolPerformanceCreateInfoKHR::queueFamilyIndex **must** be a queue family of physicalDevice. The number of passes required to capture the counters specified in the pPerformanceQueryCreateInfo member VkQueryPoolPerformanceCreateInfoKHR ::pCounters is returned in pNumPasses.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle

- VUID-vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR-pPerformanceQueryCreateInfo-parameter
  `pPerformanceQueryCreateInfo` **must** be a valid pointer to a valid VkQueryPoolPerformanceCreateInfoKHR structure
- VUID-vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR-pNumPasses-parameter
  `pNumPasses` **must** be a valid pointer to a `uint32_t` value

To destroy a query pool, call:

```
// Provided by VK_VERSION_1_0
void vkDestroyQueryPool(
    VkDevice                                    device,
    VkQueryPool                                 queryPool,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the query pool.
- `queryPool` is the query pool to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyQueryPool-queryPool-00793
  All submitted commands that refer to `queryPool` **must** have completed execution
- VUID-vkDestroyQueryPool-queryPool-00794
  If `VkAllocationCallbacks` were provided when `queryPool` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyQueryPool-queryPool-00795
  If no `VkAllocationCallbacks` were provided when `queryPool` was created, `pAllocator` **must** be `NULL`

NOTE

Applications **can** verify that `queryPool` **can** be destroyed by checking that `vkGetQueryPoolResults`() without the `VK_QUERY_RESULT_PARTIAL_BIT` flag returns `VK_SUCCESS` for all queries that are used in command buffers submitted for execution.

Valid Usage (Implicit)

- VUID-vkDestroyQueryPool-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyQueryPool-queryPool-parameter
  If `queryPool` is not VK_NULL_HANDLE, `queryPool` **must** be a valid VkQueryPool handle

- VUID-vkDestroyQueryPool-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyQueryPool-queryPool-parent
  If `queryPool` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `queryPool` **must** be externally synchronized

Possible values of VkQueryPoolCreateInfo::`queryType`, specifying the type of queries managed by the pool, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkQueryType {
    VK_QUERY_TYPE_OCCLUSION = 0,
    VK_QUERY_TYPE_PIPELINE_STATISTICS = 1,
    VK_QUERY_TYPE_TIMESTAMP = 2,
  // Provided by VK_KHR_video_queue
    VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR = 1000023000,
  // Provided by VK_EXT_transform_feedback
    VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT = 1000028004,
  // Provided by VK_KHR_performance_query
    VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR = 1000116000,
  // Provided by VK_KHR_acceleration_structure
    VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR = 1000150000,
  // Provided by VK_KHR_acceleration_structure
    VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR = 1000150001,
  // Provided by VK_KHR_video_encode_queue
    VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR = 1000299000,
  // Provided by VK_EXT_primitives_generated_query
    VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT = 1000382000,
  // Provided by VK_KHR_ray_tracing_maintenance1
    VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR =
1000386000,
  // Provided by VK_KHR_ray_tracing_maintenance1
    VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR = 1000386001,
  // Provided by VK_EXT_opacity_micromap
    VK_QUERY_TYPE_MICROMAP_SERIALIZATION_SIZE_EXT = 1000396000,
  // Provided by VK_EXT_opacity_micromap
    VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT = 1000396001,
} VkQueryType;
```

- `VK_QUERY_TYPE_OCCLUSION` specifies an occlusion query.
- `VK_QUERY_TYPE_PIPELINE_STATISTICS` specifies a pipeline statistics query.

- VK_QUERY_TYPE_TIMESTAMP specifies a timestamp query.
- VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR specifies a performance query.
- VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT specifies a transform feedback query.
- VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT specifies a primitives generated query.
- VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR specifies a acceleration structure size query for use with vkCmdWriteAccelerationStructuresPropertiesKHR or vkWriteAccelerationStructuresPropertiesKHR.
- VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR specifies a serialization acceleration structure size query.
- VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR specifies an acceleration structure size query for use with vkCmdWriteAccelerationStructuresPropertiesKHR or vkWriteAccelerationStructuresPropertiesKHR.
- VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR specifies a serialization acceleration structure pointer count query.
- VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR specifies a result status query.
- VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR specifies a video encode feedback query.

18.2. Query Operation

The operation of queries is controlled by the commands vkCmdBeginQuery, vkCmdEndQuery, vkCmdBeginQueryIndexedEXT, vkCmdEndQueryIndexedEXT, vkCmdResetQueryPool, vkCmdCopyQueryPoolResults, vkCmdWriteTimestamp2KHR, and vkCmdWriteTimestamp.

In order for a VkCommandBuffer to record query management commands, the queue family for which its VkCommandPool was created **must** support the appropriate type of operations (graphics, compute) suitable for the query type of a given query pool.

Each query in a query pool has a status that is either *unavailable* or *available*, and also has state to store the numerical results of a query operation of the type requested when the query pool was created. Resetting a query via vkCmdResetQueryPool sets the status to unavailable and makes the numerical results undefined. A query is made available by the operation of vkCmdEndQuery, vkCmdEndQueryIndexedEXT, vkCmdWriteTimestamp2KHR, or vkCmdWriteTimestamp. Both the availability status and numerical results **can** be retrieved by calling either vkGetQueryPoolResults or vkCmdCopyQueryPoolResults.

After query pool creation, each query is in an uninitialized state and **must** be reset before it is used. Queries **must** also be reset between uses.

If a logical device includes multiple physical devices, then each command that writes a query **must** execute on a single physical device, and any call to vkCmdBeginQuery **must** execute the corresponding vkCmdEndQuery command on the same physical device.

To reset a range of queries in a query pool on a queue, call:

```
// Provided by VK_VERSION_1_0
void vkCmdResetQueryPool(
    VkCommandBuffer                             commandBuffer,
    VkQueryPool                                 queryPool,
    uint32_t                                    firstQuery,
    uint32_t                                    queryCount);
```

- `commandBuffer` is the command buffer into which this command will be recorded.
- `queryPool` is the handle of the query pool managing the queries being reset.
- `firstQuery` is the initial query index to reset.
- `queryCount` is the number of queries to reset.

When executed on a queue, this command sets the status of query indices [`firstQuery`, `firstQuery` + `queryCount` - 1] to unavailable.

This command defines an execution dependency between other query commands that reference the same query.

The first synchronization scope includes all commands which reference the queries in `queryPool` indicated by `firstQuery` and `queryCount` that occur earlier in submission order.

The second synchronization scope includes all commands which reference the queries in `queryPool` indicated by `firstQuery` and `queryCount` that occur later in submission order.

The operation of this command happens after the first scope and happens before the second scope.

If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, this command sets the status of query indices [`firstQuery`, `firstQuery` + `queryCount` - 1] to unavailable for each pass of `queryPool`, as indicated by a call to vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR.

NOTE

Because `vkCmdResetQueryPool` resets all the passes of the indicated queries, applications must not record a `vkCmdResetQueryPool` command for a `queryPool` created with `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR` in a command buffer that needs to be submitted multiple times as indicated by a call to vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR. Otherwise applications will never be able to complete the recorded queries.

Valid Usage

- VUID-vkCmdResetQueryPool-firstQuery-09436
  `firstQuery` **must** be less than the number of queries in `queryPool`
- VUID-vkCmdResetQueryPool-firstQuery-09437
  The sum of `firstQuery` and `queryCount` **must** be less than or equal to the number of queries in `queryPool`

- VUID-vkCmdResetQueryPool-None-02841
  All queries used by the command **must** not be active
- VUID-vkCmdResetQueryPool-firstQuery-02862
  If `queryPool` was created with `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, this command **must** not be recorded in a command buffer that, either directly or through secondary command buffers, also contains begin commands for a query from the set of queries [`firstQuery`, `firstQuery` + `queryCount` - 1]

Valid Usage (Implicit)

- VUID-vkCmdResetQueryPool-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdResetQueryPool-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdResetQueryPool-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdResetQueryPool-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdResetQueryPool-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdResetQueryPool-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdResetQueryPool-commonparent
  Both of `commandBuffer`, and `queryPool` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics Compute Decode Encode	Action

Once queries are reset and ready for use, query commands **can** be issued to a command buffer. Occlusion queries and pipeline statistics queries count events - drawn samples and pipeline stage invocations, respectively - resulting from commands that are recorded between a vkCmdBeginQuery command and a vkCmdEndQuery command within a specified command buffer, effectively scoping a set of drawing and/or dispatching commands. Timestamp queries write timestamps to a query pool. Performance queries record performance counters to a query pool.

A query **must** begin and end in the same command buffer, although if it is a primary command buffer, and the `inheritedQueries` feature is enabled, it **can** execute secondary command buffers during the query operation. For a secondary command buffer to be executed while a query is active, it **must** set the `occlusionQueryEnable`, `queryFlags`, and/or `pipelineStatistics` members of VkCommandBufferInheritanceInfo to conservative values, as described in the Command Buffer Recording section. A query **must** either begin and end inside the same subpass of a render pass instance, or **must** both begin and end outside of a render pass instance (i.e. contain entire render pass instances).

If queries are used while executing a render pass instance that has multiview enabled, the query uses N consecutive query indices in the query pool (starting at `query`) where N is the number of bits set in the view mask in the subpass the query is used in. How the numerical results of the query are distributed among the queries is implementation-dependent. For example, some implementations **may** write each view's results to a distinct query, while other implementations **may** write the total result to the first query and write zero to the other queries. However, the sum of the results in all the queries **must** accurately reflect the total result of the query summed over all views. Applications **can** sum the results from all the queries to compute the total result.

Queries used with multiview rendering **must** not span subpasses, i.e. they **must** begin and end in the same subpass.

A query **must** either begin and end inside the same video coding scope, or **must** both begin and end outside of a video coding scope and **must** not contain entire video coding scopes.

To begin a query, call:

```
// Provided by VK_VERSION_1_0
void vkCmdBeginQuery(
    VkCommandBuffer                             commandBuffer,
    VkQueryPool                                 queryPool,
    uint32_t                                    query,
    VkQueryControlFlags                         flags);
```

- `commandBuffer` is the command buffer into which this command will be recorded.
- `queryPool` is the query pool that will manage the results of the query.
- `query` is the query index within the query pool that will contain the results.
- `flags` is a bitmask of VkQueryControlFlagBits specifying constraints on the types of queries that **can** be performed.

If the `queryType` of the pool is `VK_QUERY_TYPE_OCCLUSION` and `flags` contains `VK_QUERY_CONTROL_PRECISE_BIT`, an implementation **must** return a result that matches the actual number of samples passed. This is described in more detail in Occlusion Queries.

Calling `vkCmdBeginQuery` is equivalent to calling vkCmdBeginQueryIndexedEXT with the `index` parameter set to zero.

After beginning a query, that query is considered *active* within the command buffer it was called in until that same query is ended. Queries active in a primary command buffer when secondary command buffers are executed are considered active for those secondary command buffers.

Furthermore, if the query is started within a video coding scope, the following command buffer states are initialized for the query type:

- The *active_query_index* is set to the value specified by `query`.
- The *last activatable query index* is also set to the value specified by `query`.

Each video coding operation stores a result to the query corresponding to the current active query index, followed by incrementing the active query index. If the active query index gets incremented past the last activatable query index, issuing any further video coding operations results in undefined behavior.

NOTE	In practice, this means that currently no more than a single video coding operation **must** be issued between a begin and end query pair.

This command defines an execution dependency between other query commands that reference the same query.

The first synchronization scope includes all commands which reference the queries in `queryPool` indicated by `query` that occur earlier in submission order.

The second synchronization scope includes all commands which reference the queries in `queryPool` indicated by `query` that occur later in submission order.

The operation of this command happens after the first scope and happens before the second scope.

Valid Usage

- VUID-vkCmdBeginQuery-None-00807
  All queries used by the command **must** be *unavailable*
- VUID-vkCmdBeginQuery-queryType-02804
  The `queryType` used to create `queryPool` **must** not be `VK_QUERY_TYPE_TIMESTAMP`
- VUID-vkCmdBeginQuery-queryType-04728
  The `queryType` used to create `queryPool` **must** not be `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR` or `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR`
- VUID-vkCmdBeginQuery-queryType-06741
  The `queryType` used to create `queryPool` **must** not be `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR` or `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR`
- VUID-vkCmdBeginQuery-queryType-00800
  If the `occlusionQueryPrecise` feature is not enabled, or the `queryType` used to create `queryPool` was not `VK_QUERY_TYPE_OCCLUSION`, `flags` **must** not contain `VK_QUERY_CONTROL_PRECISE_BIT`
- VUID-vkCmdBeginQuery-query-00802
  `query` **must** be less than the number of queries in `queryPool`
- VUID-vkCmdBeginQuery-queryType-00803
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_OCCLUSION`, the `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBeginQuery-queryType-00804
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_PIPELINE_STATISTICS` and any of the `pipelineStatistics` indicate graphics operations, the `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBeginQuery-queryType-00805
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_PIPELINE_STATISTICS` and any of the `pipelineStatistics` indicate compute operations, the `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdBeginQuery-query-00808
  If called within a render pass instance, the sum of `query` and the number of bits set in the current subpass's view mask **must** be less than or equal to the number of queries in `queryPool`
- VUID-vkCmdBeginQuery-queryType-07126
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR`, then the `VkCommandPool` that `commandBuffer` was allocated from **must** have been created with a queue family index that supports result status queries, as indicated by VkQueueFamilyQueryResultStatusPropertiesKHR::`queryResultStatusSupport`
- VUID-vkCmdBeginQuery-None-07127

If there is a bound video session, then there **must** be no active queries

- VUID-vkCmdBeginQuery-None-08370
  If there is a bound video session, then it **must** not have been created with `VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR`
- VUID-vkCmdBeginQuery-queryType-07128
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR` and there is a bound video session, then `queryPool` **must** have been created with a VkVideoProfileInfoKHR structure included in the `pNext` chain of VkQueryPoolCreateInfo identical to the one specified in VkVideoSessionCreateInfoKHR::`pVideoProfile` the bound video session was created with
- VUID-vkCmdBeginQuery-queryType-07129
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR`, then there **must** be a bound video session
- VUID-vkCmdBeginQuery-queryType-07130
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR` and there is a bound video session, then `queryPool` **must** have been created with a VkVideoProfileInfoKHR structure included in the `pNext` chain of VkQueryPoolCreateInfo identical to the one specified in VkVideoSessionCreateInfoKHR::`pVideoProfile` the bound video session was created with
- VUID-vkCmdBeginQuery-queryType-07131
  If the `queryType` used to create `queryPool` was not `VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR` or `VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR`, then there **must** be no bound video session
- VUID-vkCmdBeginQuery-queryPool-01922
  `queryPool` **must** have been created with a `queryType` that differs from that of any queries that are active within `commandBuffer`
- VUID-vkCmdBeginQuery-queryType-02327
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT` the `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBeginQuery-queryType-02328
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT` then `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`transformFeedbackQueries` **must** be supported
- VUID-vkCmdBeginQuery-queryType-06687
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT` the `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBeginQuery-queryType-06688
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT` then `primitivesGeneratedQuery` **must** be enabled
- VUID-vkCmdBeginQuery-queryPool-07289
  If `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, then the VkQueryPoolPerformanceCreateInfoKHR::`queueFamilyIndex` `queryPool` was created with **must** equal the queue family index of the `VkCommandPool` that `commandBuffer` was allocated

from

- VUID-vkCmdBeginQuery-queryPool-03223
  If `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, the profiling lock **must** have been held before vkBeginCommandBuffer was called on `commandBuffer`
- VUID-vkCmdBeginQuery-queryPool-03224
  If `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR` and one of the counters used to create `queryPool` was `VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_BUFFER_KHR`, the query begin **must** be the first recorded command in `commandBuffer`
- VUID-vkCmdBeginQuery-queryPool-03225
  If `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR` and one of the counters used to create `queryPool` was `VK_PERFORMANCE_COUNTER_SCOPE_RENDER_PASS_KHR`, the begin command **must** not be recorded within a render pass instance
- VUID-vkCmdBeginQuery-queryPool-03226
  If `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR` and another query pool with a `queryType` `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR` has been used within `commandBuffer`, its parent primary command buffer or secondary command buffer recorded within the same parent primary command buffer as `commandBuffer`, the `performanceCounterMultipleQueryPools` feature **must** be enabled
- VUID-vkCmdBeginQuery-None-02863
  If `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, this command **must** not be recorded in a command buffer that, either directly or through secondary command buffers, also contains a `vkCmdResetQueryPool` command affecting the same query

Valid Usage (Implicit)

- VUID-vkCmdBeginQuery-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBeginQuery-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdBeginQuery-flags-parameter
  `flags` **must** be a valid combination of VkQueryControlFlagBits values
- VUID-vkCmdBeginQuery-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBeginQuery-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdBeginQuery-commonparent
  Both of `commandBuffer`, and `queryPool` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Graphics Compute Decode Encode	Action State

To begin an indexed query, call:

```
// Provided by VK_EXT_transform_feedback
void vkCmdBeginQueryIndexedEXT(
    VkCommandBuffer                             commandBuffer,
    VkQueryPool                                 queryPool,
    uint32_t                                    query,
    VkQueryControlFlags                         flags,
    uint32_t                                    index);
```

- `commandBuffer` is the command buffer into which this command will be recorded.
- `queryPool` is the query pool that will manage the results of the query.
- `query` is the query index within the query pool that will contain the results.
- `flags` is a bitmask of VkQueryControlFlagBits specifying constraints on the types of queries that **can** be performed.
- `index` is the query type specific index. When the query type is `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT` or `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT`, the index represents the vertex stream.

The `vkCmdBeginQueryIndexedEXT` command operates the same as the vkCmdBeginQuery command, except that it also accepts a query type specific `index` parameter.

This command defines an execution dependency between other query commands that reference the same query index.

The first synchronization scope includes all commands which reference the queries in `queryPool` indicated by `query` and `index` that occur earlier in submission order.

The second synchronization scope includes all commands which reference the queries in queryPool indicated by query and index that occur later in submission order.

The operation of this command happens after the first scope and happens before the second scope.

Valid Usage

- VUID-vkCmdBeginQueryIndexedEXT-None-00807
  All queries used by the command **must** be *unavailable*
- VUID-vkCmdBeginQueryIndexedEXT-queryType-02804
  The queryType used to create queryPool **must** not be VK_QUERY_TYPE_TIMESTAMP
- VUID-vkCmdBeginQueryIndexedEXT-queryType-04728
  The queryType used to create queryPool **must** not be VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR or VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR
- VUID-vkCmdBeginQueryIndexedEXT-queryType-06741
  The queryType used to create queryPool **must** not be VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR or VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR
- VUID-vkCmdBeginQueryIndexedEXT-queryType-00800
  If the occlusionQueryPrecise feature is not enabled, or the queryType used to create queryPool was not VK_QUERY_TYPE_OCCLUSION, flags **must** not contain VK_QUERY_CONTROL_PRECISE_BIT
- VUID-vkCmdBeginQueryIndexedEXT-query-00802
  query **must** be less than the number of queries in queryPool
- VUID-vkCmdBeginQueryIndexedEXT-queryType-00803
  If the queryType used to create queryPool was VK_QUERY_TYPE_OCCLUSION, the VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdBeginQueryIndexedEXT-queryType-00804
  If the queryType used to create queryPool was VK_QUERY_TYPE_PIPELINE_STATISTICS and any of the pipelineStatistics indicate graphics operations, the VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdBeginQueryIndexedEXT-queryType-00805
  If the queryType used to create queryPool was VK_QUERY_TYPE_PIPELINE_STATISTICS and any of the pipelineStatistics indicate compute operations, the VkCommandPool that commandBuffer was allocated from **must** support compute operations
- VUID-vkCmdBeginQueryIndexedEXT-query-00808
  If called within a render pass instance, the sum of query and the number of bits set in the current subpass's view mask **must** be less than or equal to the number of queries in queryPool
- VUID-vkCmdBeginQueryIndexedEXT-queryType-07126
  If the queryType used to create queryPool was VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR, then the VkCommandPool that commandBuffer was allocated from **must** have been created with a queue family index that supports result status queries, as indicated by

VkQueueFamilyQueryResultStatusPropertiesKHR::queryResultStatusSupport

- VUID-vkCmdBeginQueryIndexedEXT-None-07127
  If there is a bound video session, then there **must** be no active queries
- VUID-vkCmdBeginQueryIndexedEXT-None-08370
  If there is a bound video session, then it **must** not have been created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR
- VUID-vkCmdBeginQueryIndexedEXT-queryType-07128
  If the queryType used to create queryPool was VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR and there is a bound video session, then queryPool **must** have been created with a VkVideoProfileInfoKHR structure included in the pNext chain of VkQueryPoolCreateInfo identical to the one specified in VkVideoSessionCreateInfoKHR::pVideoProfile the bound video session was created with
- VUID-vkCmdBeginQueryIndexedEXT-queryType-07129
  If the queryType used to create queryPool was VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR, then there **must** be a bound video session
- VUID-vkCmdBeginQueryIndexedEXT-queryType-07130
  If the queryType used to create queryPool was VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR and there is a bound video session, then queryPool **must** have been created with a VkVideoProfileInfoKHR structure included in the pNext chain of VkQueryPoolCreateInfo identical to the one specified in VkVideoSessionCreateInfoKHR::pVideoProfile the bound video session was created with
- VUID-vkCmdBeginQueryIndexedEXT-queryType-07131
  If the queryType used to create queryPool was not VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR or VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR, then there **must** be no bound video session
- VUID-vkCmdBeginQueryIndexedEXT-queryPool-04753
  If the queryPool was created with the same queryType as that of another active query within commandBuffer, then index **must** not match the index used for the active query
- VUID-vkCmdBeginQueryIndexedEXT-queryType-02338
  If the queryType used to create queryPool was VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT the VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdBeginQueryIndexedEXT-queryType-02339
  If the queryType used to create queryPool was VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT the index parameter **must** be less than VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackStreams
- VUID-vkCmdBeginQueryIndexedEXT-queryType-06692
  If the queryType used to create queryPool was not VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT and not VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT, the index **must** be zero
- VUID-vkCmdBeginQueryIndexedEXT-queryType-06689
  If the queryType used to create queryPool was VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT the VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdBeginQueryIndexedEXT-queryType-06690

If the queryType used to create queryPool was VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT the index parameter **must** be less than VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackStreams

- VUID-vkCmdBeginQueryIndexedEXT-queryType-06691
  If the queryType used to create queryPool was VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT and the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled, the index parameter **must** be zero
- VUID-vkCmdBeginQueryIndexedEXT-queryType-06693
  If the queryType used to create queryPool was VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT then primitivesGeneratedQuery **must** be enabled
- VUID-vkCmdBeginQueryIndexedEXT-queryType-02341
  If the queryType used to create queryPool was VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT then VkPhysicalDeviceTransformFeedbackPropertiesEXT::transformFeedbackQueries **must** be supported
- VUID-vkCmdBeginQueryIndexedEXT-queryPool-07289
  If queryPool was created with a queryType of VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, then the VkQueryPoolPerformanceCreateInfoKHR::queueFamilyIndex queryPool was created with **must** equal the queue family index of the VkCommandPool that commandBuffer was allocated from
- VUID-vkCmdBeginQueryIndexedEXT-queryPool-03223
  If queryPool was created with a queryType of VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, the profiling lock **must** have been held before vkBeginCommandBuffer was called on commandBuffer
- VUID-vkCmdBeginQueryIndexedEXT-queryPool-03224
  If queryPool was created with a queryType of VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR and one of the counters used to create queryPool was VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_BUFFER_KHR, the query begin **must** be the first recorded command in commandBuffer
- VUID-vkCmdBeginQueryIndexedEXT-queryPool-03225
  If queryPool was created with a queryType of VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR and one of the counters used to create queryPool was VK_PERFORMANCE_COUNTER_SCOPE_RENDER_PASS_KHR, the begin command **must** not be recorded within a render pass instance
- VUID-vkCmdBeginQueryIndexedEXT-queryPool-03226
  If queryPool was created with a queryType of VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR and another query pool with a queryType VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR has been used within commandBuffer, its parent primary command buffer or secondary command buffer recorded within the same parent primary command buffer as commandBuffer, the performanceCounterMultipleQueryPools feature **must** be enabled
- VUID-vkCmdBeginQueryIndexedEXT-None-02863
  If queryPool was created with a queryType of VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, this command **must** not be recorded in a command buffer that, either directly or through secondary command buffers, also contains a vkCmdResetQueryPool command affecting the same query

Valid Usage (Implicit)

- VUID-vkCmdBeginQueryIndexedEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBeginQueryIndexedEXT-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdBeginQueryIndexedEXT-flags-parameter
  `flags` **must** be a valid combination of VkQueryControlFlagBits values
- VUID-vkCmdBeginQueryIndexedEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBeginQueryIndexedEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdBeginQueryIndexedEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBeginQueryIndexedEXT-commonparent
  Both of `commandBuffer`, and `queryPool` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute Decode Encode	Action State

Bits which **can** be set in vkCmdBeginQuery::`flags`, specifying constraints on the types of queries that **can** be performed, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkQueryControlFlagBits {
    VK_QUERY_CONTROL_PRECISE_BIT = 0x00000001,
} VkQueryControlFlagBits;
```

- VK_QUERY_CONTROL_PRECISE_BIT specifies the precision of occlusion queries.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkQueryControlFlags;
```

VkQueryControlFlags is a bitmask type for setting a mask of zero or more VkQueryControlFlagBits.

To end a query after the set of desired drawing or dispatching commands is executed, call:

```
// Provided by VK_VERSION_1_0
void vkCmdEndQuery(
    VkCommandBuffer                             commandBuffer,
    VkQueryPool                                 queryPool,
    uint32_t                                    query);
```

- commandBuffer is the command buffer into which this command will be recorded.
- queryPool is the query pool that is managing the results of the query.
- query is the query index within the query pool where the result is stored.

The command completes the query in queryPool identified by query, and marks it as available.

This command defines an execution dependency between other query commands that reference the same query.

The first synchronization scope includes all commands which reference the queries in queryPool indicated by query that occur earlier in submission order.

The second synchronization scope includes only the operation of this command.

Calling vkCmdEndQuery is equivalent to calling vkCmdEndQueryIndexedEXT with the index parameter set to zero.

Valid Usage

- VUID-vkCmdEndQuery-None-01923
  All queries used by the command **must** be active
- VUID-vkCmdEndQuery-query-00810
  query **must** be less than the number of queries in queryPool
- VUID-vkCmdEndQuery-query-00812
  If vkCmdEndQuery is called within a render pass instance, the sum of query and the number

of bits set in the current subpass's view mask **must** be less than or equal to the number of queries in `queryPool`

- VUID-vkCmdEndQuery-queryPool-03227
  If `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR` and one or more of the counters used to create `queryPool` was `VK_PERFORMANCE_COUNTER_SCOPE_COMMAND_BUFFER_KHR`, the vkCmdEndQuery **must** be the last recorded command in `commandBuffer`
- VUID-vkCmdEndQuery-queryPool-03228
  If `queryPool` was created with a `queryType` of `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR` and one or more of the counters used to create `queryPool` was `VK_PERFORMANCE_COUNTER_SCOPE_RENDER_PASS_KHR`, the vkCmdEndQuery **must** not be recorded within a render pass instance
- VUID-vkCmdEndQuery-None-07007
  If called within a subpass of a render pass instance, the corresponding `vkCmdBeginQuery*` command **must** have been called previously within the same subpass

Valid Usage (Implicit)

- VUID-vkCmdEndQuery-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdEndQuery-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdEndQuery-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdEndQuery-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdEndQuery-commonparent
  Both of `commandBuffer`, and `queryPool` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Graphics Compute Decode Encode	Action State

To end an indexed query after the set of desired drawing or dispatching commands is recorded, call:

```
// Provided by VK_EXT_transform_feedback
void vkCmdEndQueryIndexedEXT(
    VkCommandBuffer                             commandBuffer,
    VkQueryPool                                 queryPool,
    uint32_t                                    query,
    uint32_t                                    index);
```

- `commandBuffer` is the command buffer into which this command will be recorded.
- `queryPool` is the query pool that is managing the results of the query.
- `query` is the query index within the query pool where the result is stored.
- `index` is the query type specific index.

The command completes the query in `queryPool` identified by `query` and `index`, and marks it as available.

The `vkCmdEndQueryIndexedEXT` command operates the same as the vkCmdEndQuery command, except that it also accepts a query type specific `index` parameter.

This command defines an execution dependency between other query commands that reference the same query index.

The first synchronization scope includes all commands which reference the queries in `queryPool` indicated by `query` that occur earlier in submission order.

The second synchronization scope includes only the operation of this command.

Valid Usage

- VUID-vkCmdEndQueryIndexedEXT-None-02342
  All queries used by the command **must** be active
- VUID-vkCmdEndQueryIndexedEXT-query-02343
  `query` **must** be less than the number of queries in `queryPool`

- VUID-vkCmdEndQueryIndexedEXT-query-02345
  If `vkCmdEndQueryIndexedEXT` is called within a render pass instance, the sum of `query` and the number of bits set in the current subpass's view mask **must** be less than or equal to the number of queries in `queryPool`
- VUID-vkCmdEndQueryIndexedEXT-queryType-06694
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT` or `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT`, the `index` parameter **must** be less than `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackStreams`
- VUID-vkCmdEndQueryIndexedEXT-queryType-06695
  If the `queryType` used to create `queryPool` was not `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT` and not `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT`, the `index` **must** be zero
- VUID-vkCmdEndQueryIndexedEXT-queryType-06696
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT` or `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT`, `index` **must** equal the `index` used to begin the query
- VUID-vkCmdEndQueryIndexedEXT-None-07007
  If called within a subpass of a render pass instance, the corresponding `vkCmdBeginQuery`* command **must** have been called previously within the same subpass

Valid Usage (Implicit)

- VUID-vkCmdEndQueryIndexedEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdEndQueryIndexedEXT-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdEndQueryIndexedEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdEndQueryIndexedEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, compute, decode, or encode operations
- VUID-vkCmdEndQueryIndexedEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdEndQueryIndexedEXT-commonparent
  Both of `commandBuffer`, and `queryPool` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally

synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute Decode Encode	Action State

An application **can** retrieve results either by requesting they be written into application-provided memory, or by requesting they be copied into a `VkBuffer`. In either case, the layout in memory is defined as follows:

- The first query's result is written starting at the first byte requested by the command, and each subsequent query's result begins `stride` bytes later.
- Occlusion queries, pipeline statistics queries, transform feedback queries, primitives generated queries, video encode feedback queries, and timestamp queries store results in a tightly packed array of unsigned integers, either 32- or 64-bits as requested by the command, storing the numerical results and, if requested, the availability status.
- Performance queries store results in a tightly packed array whose type is determined by the `unit` member of the corresponding VkPerformanceCounterKHR.
- If `VK_QUERY_RESULT_WITH_AVAILABILITY_BIT` is used, the final element of each query's result is an integer indicating whether the query's result is available, with any non-zero value indicating that it is available.
- If `VK_QUERY_RESULT_WITH_STATUS_BIT_KHR` is used, the final element of each query's result is an integer value indicating that status of the query result. Positive values indicate success, negative values indicate failure, and 0 indicates that the result is not yet available. Specific error codes are encoded in the VkQueryResultStatusKHR enumeration.
- Occlusion queries write one integer value - the number of samples passed. Pipeline statistics queries write one integer value for each bit that is enabled in the `pipelineStatistics` when the pool is created, and the statistics values are written in bit order starting from the least significant bit. Timestamp queries write one integer value. Performance queries write one VkPerformanceCounterResultKHR value for each VkPerformanceCounterKHR in the query. Transform feedback queries write two integers; the first integer is the number of primitives successfully written to the corresponding transform feedback buffer and the second is the number of primitives output to the vertex stream, regardless of whether they were successfully captured or not. In other words, if the transform feedback buffer was sized too small for the number of primitives output by the vertex stream, the first integer represents the number of primitives actually written and the second is the number that would have been written if all the transform feedback buffers associated with that vertex stream were large enough. Primitives generated queries write the number of primitives output to the vertex stream, regardless of whether transform feedback is active or not, or whether they were successfully captured by

transform feedback or not. This is identical to the second integer of the transform feedback queries if transform feedback is active. Video encode feedback queries write one or more integer values for each bit that is enabled in VkQueryPoolVideoEncodeFeedbackCreateInfoKHR::encodeFeedbackFlags when the pool is created, and the feedback values are written in bit order starting from the least significant bit, as described here.

- If more than one query is retrieved and stride is not at least as large as the size of the array of values corresponding to a single query, the values written to memory are undefined.

To retrieve status and results for a set of queries, call:

```
// Provided by VK_VERSION_1_0
VkResult vkGetQueryPoolResults(
    VkDevice                                    device,
    VkQueryPool                                 queryPool,
    uint32_t                                    firstQuery,
    uint32_t                                    queryCount,
    size_t                                      dataSize,
    void*                                       pData,
    VkDeviceSize                                stride,
    VkQueryResultFlags                          flags);
```

- device is the logical device that owns the query pool.
- queryPool is the query pool managing the queries containing the desired results.
- firstQuery is the initial query index.
- queryCount is the number of queries to read.
- dataSize is the size in bytes of the buffer pointed to by pData.
- pData is a pointer to an application-allocated buffer where the results will be written
- stride is the stride in bytes between results for individual queries within pData.
- flags is a bitmask of VkQueryResultFlagBits specifying how and when results are returned.

Any results written for a query are written according to a layout dependent on the query type.

If no bits are set in flags, and all requested queries are in the available state, results are written as an array of 32-bit unsigned integer values. Behavior when not all queries are available is described below.

If VK_QUERY_RESULT_WITH_AVAILABILITY_BIT is set, results for all queries in queryPool identified by firstQuery and queryCount are copied to pData, along with an extra availability or status value written directly after the results of each query and interpreted as an unsigned integer. A value of zero indicates that the results are not yet available, otherwise the query is complete and results are available. The size of the availability or status values is 64 bits if VK_QUERY_RESULT_64_BIT is set in flags. Otherwise, it is 32 bits.

If VK_QUERY_RESULT_WITH_STATUS_BIT_KHR is set, results for all queries in queryPool identified by

`firstQuery` and `queryCount` are copied to `pData`, along with an extra status value written directly after the results of each query and interpreted as a signed integer. A value of zero indicates that the results are not yet available. Positive values indicate that the operations within the query completed successfully, and the query results are valid. Negative values indicate that the operations within the query completed unsuccessfully.

VkQueryResultStatusKHR defines specific meaning for values returned here, though implementations are free to return other values.

If the status value written is negative, indicating that the operations within the query completed unsuccessfully, then all other results written by this command are undefined unless otherwise specified for any of the results of the used query type.

NOTE

If `VK_QUERY_RESULT_WITH_AVAILABILITY_BIT` or `VK_QUERY_RESULT_WITH_STATUS_BIT_KHR` is set, the layout of data in the buffer is a *(result,availability)* or *(result,status)* pair for each query returned, and `stride` is the stride between each pair.

Results for any available query written by this command are final and represent the final result of the query. If `VK_QUERY_RESULT_PARTIAL_BIT` is set, then for any query that is unavailable, an intermediate result between zero and the final result value is written for that query. Otherwise, any result written by this command is undefined.

If `VK_QUERY_RESULT_64_BIT` is set, results and, if returned, availability or status values for all queries are written as an array of 64-bit values. If the `queryPool` was created with `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, results for each query are written as an array of the type indicated by VkPerformanceCounterKHR::`storage` for the counter being queried. Otherwise, results and availability or status values are written as an array of 32-bit values. If an unsigned integer query's value overflows the result type, the value **may** either wrap or saturate. If the `maintenance7` feature is enabled, for an unsigned integer query, the 32-bit result value **must** be equal to the 32 least significant bits of the equivalent 64-bit result value. If a signed integer query's value overflows the result type, the value is undefined. If a floating-point query's value is not representable as the result type, the value is undefined.

If `VK_QUERY_RESULT_WAIT_BIT` is set, this command defines an execution dependency with any earlier commands that writes one of the identified queries. The first synchronization scope includes all instances of vkCmdEndQuery, vkCmdEndQueryIndexedEXT, vkCmdWriteTimestamp2KHR, and vkCmdWriteTimestamp that reference any query in `queryPool` indicated by `firstQuery` and `queryCount`. The second synchronization scope includes the host operations of this command.

If `VK_QUERY_RESULT_WAIT_BIT` is not set, `vkGetQueryPoolResults` **may** return `VK_NOT_READY` if there are queries in the unavailable state.

NOTE

Applications **must** take care to ensure that use of the `VK_QUERY_RESULT_WAIT_BIT` bit has the desired effect.

For example, if a query has been used previously and a command buffer records the commands `vkCmdResetQueryPool`, `vkCmdBeginQuery`, and `vkCmdEndQuery` for that query, then the query will remain in the available state until the `vkCmdResetQueryPool` command executes on a queue. Applications **can** use fences or

events to ensure that a query has already been reset before checking for its results or availability status. Otherwise, a stale value could be returned from a previous use of the query.

The above also applies when VK_QUERY_RESULT_WAIT_BIT is used in combination with VK_QUERY_RESULT_WITH_AVAILABILITY_BIT. In this case, the returned availability status **may** reflect the result of a previous use of the query unless the vkCmdResetQueryPool command has been executed since the last use of the query.

A similar situation can arise with the VK_QUERY_RESULT_WITH_STATUS_BIT_KHR flag.

NOTE

Applications **can** double-buffer query pool usage, with a pool per frame, and reset queries at the end of the frame in which they are read.

Valid Usage

- VUID-vkGetQueryPoolResults-firstQuery-09436
  firstQuery **must** be less than the number of queries in queryPool
- VUID-vkGetQueryPoolResults-firstQuery-09437
  The sum of firstQuery and queryCount **must** be less than or equal to the number of queries in queryPool
- VUID-vkGetQueryPoolResults-queryCount-09438
  If queryCount is greater than 1, stride **must** not be zero
- VUID-vkGetQueryPoolResults-queryType-09439
  If the queryType used to create queryPool was VK_QUERY_TYPE_TIMESTAMP, flags **must** not contain VK_QUERY_RESULT_PARTIAL_BIT
- VUID-vkGetQueryPoolResults-queryType-09440
  If the queryType used to create queryPool was VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, flags **must** not contain VK_QUERY_RESULT_WITH_AVAILABILITY_BIT, VK_QUERY_RESULT_WITH_STATUS_BIT_KHR, VK_QUERY_RESULT_PARTIAL_BIT, or VK_QUERY_RESULT_64_BIT
- VUID-vkGetQueryPoolResults-queryType-09441
  If the queryType used to create queryPool was VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, the queryPool **must** have been recorded once for each pass as retrieved via a call to vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR
- VUID-vkGetQueryPoolResults-queryType-09442
  If the queryType used to create queryPool was VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR, then flags **must** include VK_QUERY_RESULT_WITH_STATUS_BIT_KHR
- VUID-vkGetQueryPoolResults-flags-09443
  If flags includes VK_QUERY_RESULT_WITH_STATUS_BIT_KHR, then it **must** not include VK_QUERY_RESULT_WITH_AVAILABILITY_BIT
- VUID-vkGetQueryPoolResults-None-09401
  All queries used by the command **must** not be uninitialized

- VUID-vkGetQueryPoolResults-flags-02828
  If `VK_QUERY_RESULT_64_BIT` is not set in `flags` and the `queryType` used to create `queryPool` was not `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, then `pData` and `stride` **must** be multiples of `4`
- VUID-vkGetQueryPoolResults-flags-00815
  If `VK_QUERY_RESULT_64_BIT` is set in `flags` then `pData` and `stride` **must** be multiples of `8`
- VUID-vkGetQueryPoolResults-stride-08993
  If `VK_QUERY_RESULT_WITH_AVAILABILITY_BIT` is set, `stride` **must** be large enough to contain the unsigned integer representing availability or status in addition to the query result
- VUID-vkGetQueryPoolResults-queryType-03229
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, then `pData` and `stride` **must** be multiples of the size of VkPerformanceCounterResultKHR
- VUID-vkGetQueryPoolResults-queryType-04519
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, then `stride` **must** be large enough to contain the VkQueryPoolPerformanceCreateInfoKHR ::`counterIndexCount` used to create `queryPool` times the size of VkPerformanceCounterResultKHR
- VUID-vkGetQueryPoolResults-dataSize-00817
  `dataSize` **must** be large enough to contain the result of each query, as described here

Valid Usage (Implicit)

- VUID-vkGetQueryPoolResults-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetQueryPoolResults-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkGetQueryPoolResults-pData-parameter
  `pData` **must** be a valid pointer to an array of `dataSize` bytes
- VUID-vkGetQueryPoolResults-flags-parameter
  `flags` **must** be a valid combination of VkQueryResultFlagBits values
- VUID-vkGetQueryPoolResults-dataSize-arraylength
  `dataSize` **must** be greater than `0`
- VUID-vkGetQueryPoolResults-queryPool-parent
  `queryPool` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_NOT_READY`

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

Bits which **can** be set in vkGetQueryPoolResults::flags and vkCmdCopyQueryPoolResults::flags, specifying how and when results are returned, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkQueryResultFlagBits {
    VK_QUERY_RESULT_64_BIT = 0x00000001,
    VK_QUERY_RESULT_WAIT_BIT = 0x00000002,
    VK_QUERY_RESULT_WITH_AVAILABILITY_BIT = 0x00000004,
    VK_QUERY_RESULT_PARTIAL_BIT = 0x00000008,
  // Provided by VK_KHR_video_queue
    VK_QUERY_RESULT_WITH_STATUS_BIT_KHR = 0x00000010,
} VkQueryResultFlagBits;
```

- VK_QUERY_RESULT_64_BIT specifies the results will be written as an array of 64-bit unsigned integer values. If this bit is not set, the results will be written as an array of 32-bit unsigned integer values.
- VK_QUERY_RESULT_WAIT_BIT specifies that Vulkan will wait for each query's status to become available before retrieving its results.
- VK_QUERY_RESULT_WITH_AVAILABILITY_BIT specifies that the availability status accompanies the results.
- VK_QUERY_RESULT_PARTIAL_BIT specifies that returning partial results is acceptable.
- VK_QUERY_RESULT_WITH_STATUS_BIT_KHR specifies that the last value returned in the query is a VkQueryResultStatusKHR value. See result status query for information on how an application can determine whether the use of this flag bit is supported.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkQueryResultFlags;
```

VkQueryResultFlags is a bitmask type for setting a mask of zero or more VkQueryResultFlagBits.

Specific status codes that **can** be returned from a query are:

```
// Provided by VK_KHR_video_queue
typedef enum VkQueryResultStatusKHR {
    VK_QUERY_RESULT_STATUS_ERROR_KHR = -1,
    VK_QUERY_RESULT_STATUS_NOT_READY_KHR = 0,
    VK_QUERY_RESULT_STATUS_COMPLETE_KHR = 1,
  // Provided by VK_KHR_video_encode_queue
    VK_QUERY_RESULT_STATUS_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_KHR = -1000299000,
} VkQueryResultStatusKHR;
```

- VK_QUERY_RESULT_STATUS_NOT_READY_KHR specifies that the query result is not yet available.
- VK_QUERY_RESULT_STATUS_ERROR_KHR specifies that operations did not complete successfully.
- VK_QUERY_RESULT_STATUS_COMPLETE_KHR specifies that operations completed successfully and the query result is available.
- VK_QUERY_RESULT_STATUS_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_KHR specifies that a video encode operation did not complete successfully due to the destination video bitstream buffer range not being sufficiently large to fit the encoded bitstream data.

To copy query statuses and numerical results directly to buffer memory, call:

```
// Provided by VK_VERSION_1_0
void vkCmdCopyQueryPoolResults(
    VkCommandBuffer                             commandBuffer,
    VkQueryPool                                 queryPool,
    uint32_t                                    firstQuery,
    uint32_t                                    queryCount,
    VkBuffer                                    dstBuffer,
    VkDeviceSize                                dstOffset,
    VkDeviceSize                                stride,
    VkQueryResultFlags                          flags);
```

- commandBuffer is the command buffer into which this command will be recorded.
- queryPool is the query pool managing the queries containing the desired results.
- firstQuery is the initial query index.
- queryCount is the number of queries. firstQuery and queryCount together define a range of queries.
- dstBuffer is a VkBuffer object that will receive the results of the copy command.
- dstOffset is an offset into dstBuffer.
- stride is the stride in bytes between results for individual queries within dstBuffer. The required size of the backing memory for dstBuffer is determined as described above for vkGetQueryPoolResults.
- flags is a bitmask of VkQueryResultFlagBits specifying how and when results are returned.

Any results written for a query are written according to a layout dependent on the query type.

Results for any query in queryPool identified by firstQuery and queryCount that is available are copied to dstBuffer.

If VK_QUERY_RESULT_WITH_AVAILABILITY_BIT is set, results for all queries in queryPool identified by firstQuery and queryCount are copied to dstBuffer, along with an extra availability value written directly after the results of each query and interpreted as an unsigned integer. A value of zero indicates that the results are not yet available, otherwise the query is complete and results are available.

If VK_QUERY_RESULT_WITH_STATUS_BIT_KHR is set, results for all queries in queryPool identified by firstQuery and queryCount are copied to dstBuffer, along with an extra status value written directly after the results of each query and interpreted as a signed integer. A value of zero indicates that the results are not yet available. Positive values indicate that the operations within the query completed successfully, and the query results are valid. Negative values indicate that the operations within the query completed unsuccessfully.

VkQueryResultStatusKHR defines specific meaning for values returned here, though implementations are free to return other values.

If the status value written is negative, indicating that the operations within the query completed unsuccessfully, then all other results written by this command are undefined unless otherwise specified for any of the results of the used query type.

Results for any available query written by this command are final and represent the final result of the query. If VK_QUERY_RESULT_PARTIAL_BIT is set, then for any query that is unavailable, an intermediate result between zero and the final result value is written for that query. Otherwise, any result written by this command is undefined.

If VK_QUERY_RESULT_64_BIT is set, results and availability or status values for all queries are written as an array of 64-bit values. If the queryPool was created with VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, results for each query are written as an array of the type indicated by VkPerformanceCounterKHR ::storage for the counter being queried. Otherwise, results and availability or status values are written as an array of 32-bit values. If an unsigned integer query's value overflows the result type, the value **may** either wrap or saturate. If the maintenance7 feature is enabled, for an unsigned integer query, the 32-bit result value **must** be equal to the 32 least significant bits of the equivalent 64-bit result value. If a signed integer query's value overflows the result type, the value is undefined. If a floating-point query's value is not representable as the result type, the value is undefined.

This command defines an execution dependency between other query commands that reference the same query.

The first synchronization scope includes all commands which reference the queries in queryPool indicated by query that occur earlier in submission order. If flags does not include VK_QUERY_RESULT_WAIT_BIT, vkCmdEndQueryIndexedEXT, vkCmdWriteTimestamp2KHR, vkCmdEndQuery, and vkCmdWriteTimestamp are excluded from this scope.

The second synchronization scope includes all commands which reference the queries in queryPool indicated by query that occur later in submission order.

The operation of this command happens after the first scope and happens before the second scope.

vkCmdCopyQueryPoolResults is considered to be a transfer operation, and its writes to buffer memory **must** be synchronized using VK_PIPELINE_STAGE_TRANSFER_BIT and VK_ACCESS_TRANSFER_WRITE_BIT before using the results.

Valid Usage

- VUID-vkCmdCopyQueryPoolResults-firstQuery-09436
  firstQuery **must** be less than the number of queries in queryPool
- VUID-vkCmdCopyQueryPoolResults-firstQuery-09437
  The sum of firstQuery and queryCount **must** be less than or equal to the number of queries in queryPool
- VUID-vkCmdCopyQueryPoolResults-queryCount-09438
  If queryCount is greater than 1, stride **must** not be zero
- VUID-vkCmdCopyQueryPoolResults-queryType-09439
  If the queryType used to create queryPool was VK_QUERY_TYPE_TIMESTAMP, flags **must** not contain VK_QUERY_RESULT_PARTIAL_BIT
- VUID-vkCmdCopyQueryPoolResults-queryType-09440
  If the queryType used to create queryPool was VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, flags **must** not contain VK_QUERY_RESULT_WITH_AVAILABILITY_BIT, VK_QUERY_RESULT_WITH_STATUS_BIT_KHR, VK_QUERY_RESULT_PARTIAL_BIT, or VK_QUERY_RESULT_64_BIT
- VUID-vkCmdCopyQueryPoolResults-queryType-09441
  If the queryType used to create queryPool was VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR, the queryPool **must** have been recorded once for each pass as retrieved via a call to vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR
- VUID-vkCmdCopyQueryPoolResults-queryType-09442
  If the queryType used to create queryPool was VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR, then flags **must** include VK_QUERY_RESULT_WITH_STATUS_BIT_KHR
- VUID-vkCmdCopyQueryPoolResults-flags-09443
  If flags includes VK_QUERY_RESULT_WITH_STATUS_BIT_KHR, then it **must** not include VK_QUERY_RESULT_WITH_AVAILABILITY_BIT
- VUID-vkCmdCopyQueryPoolResults-None-09402
  All queries used by the command **must** not be uninitialized when the command is executed
- VUID-vkCmdCopyQueryPoolResults-dstOffset-00819
  dstOffset **must** be less than the size of dstBuffer
- VUID-vkCmdCopyQueryPoolResults-flags-00822
  If VK_QUERY_RESULT_64_BIT is not set in flags then dstOffset and stride **must** be multiples of 4
- VUID-vkCmdCopyQueryPoolResults-flags-00823
  If VK_QUERY_RESULT_64_BIT is set in flags then dstOffset and stride **must** be multiples of 8

- VUID-vkCmdCopyQueryPoolResults-dstBuffer-00824
  `dstBuffer` **must** have enough storage, from `dstOffset`, to contain the result of each query, as described here
- VUID-vkCmdCopyQueryPoolResults-dstBuffer-00825
  `dstBuffer` **must** have been created with `VK_BUFFER_USAGE_TRANSFER_DST_BIT` usage flag
- VUID-vkCmdCopyQueryPoolResults-dstBuffer-00826
  If `dstBuffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdCopyQueryPoolResults-queryType-03232
  If the `queryType` used to create `queryPool` was `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`, VkPhysicalDevicePerformanceQueryPropertiesKHR::`allowCommandBufferQueryCopies` **must** be `VK_TRUE`
- VUID-vkCmdCopyQueryPoolResults-None-07429
  All queries used by the command **must** not be active
- VUID-vkCmdCopyQueryPoolResults-None-08752
  All queries used by the command **must** have been made *available* by prior executed commands

Valid Usage (Implicit)

- VUID-vkCmdCopyQueryPoolResults-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyQueryPoolResults-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdCopyQueryPoolResults-dstBuffer-parameter
  `dstBuffer` **must** be a valid VkBuffer handle
- VUID-vkCmdCopyQueryPoolResults-flags-parameter
  `flags` **must** be a valid combination of VkQueryResultFlagBits values
- VUID-vkCmdCopyQueryPoolResults-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyQueryPoolResults-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdCopyQueryPoolResults-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyQueryPoolResults-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdCopyQueryPoolResults-commonparent
  Each of `commandBuffer`, `dstBuffer`, and `queryPool` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics Compute	Action

Rendering operations such as clears, MSAA resolves, attachment load/store operations, and blits **may** count towards the results of queries. This behavior is implementation-dependent and **may** vary depending on the path used within an implementation. For example, some implementations have several types of clears, some of which **may** include vertices and some not.

18.3. Occlusion Queries

Occlusion queries track the number of samples that pass the per-fragment tests for a set of drawing commands. As such, occlusion queries are only available on queue families supporting graphics operations. The application **can** then use these results to inform future rendering decisions. An occlusion query is begun and ended by calling `vkCmdBeginQuery` and `vkCmdEndQuery`, respectively. When an occlusion query begins, the count of passing samples always starts at zero. For each drawing command, the count is incremented as described in Sample Counting. If `flags` does not contain `VK_QUERY_CONTROL_PRECISE_BIT` an implementation **may** generate any non-zero result value for the query if the count of passing samples is non-zero.

NOTE

Not setting `VK_QUERY_CONTROL_PRECISE_BIT` mode **may** be more efficient on some implementations, and **should** be used where it is sufficient to know a boolean result on whether any samples passed the per-fragment tests. In this case, some implementations **may** only return zero or one, indifferent to the actual number of samples passing the per-fragment tests.

Setting `VK_QUERY_CONTROL_PRECISE_BIT` does not guarantee that different implementations return the same number of samples in an occlusion query. Some implementations may kill fragments in the pre-rasterization shader stage, and these killed fragments do not contribute to the final result of the query. It is possible that some implementations generate a zero result value for the query, while others generate a non-zero value.

When an occlusion query finishes, the result for that query is marked as available. The application **can** then either copy the result to a buffer (via `vkCmdCopyQueryPoolResults`) or request it be put into

host memory (via vkGetQueryPoolResults).

NOTE	If occluding geometry is not drawn first, samples **can** pass the depth test, but still not be visible in a final image.

18.4. Pipeline Statistics Queries

Pipeline statistics queries allow the application to sample a specified set of VkPipeline counters. These counters are accumulated by Vulkan for a set of either drawing or dispatching commands while a pipeline statistics query is active. As such, pipeline statistics queries are available on queue families supporting either graphics or compute operations. The availability of pipeline statistics queries is indicated by the pipelineStatisticsQuery member of the VkPhysicalDeviceFeatures object (see vkGetPhysicalDeviceFeatures and vkCreateDevice for detecting and requesting this query type on a VkDevice).

A pipeline statistics query is begun and ended by calling vkCmdBeginQuery and vkCmdEndQuery, respectively. When a pipeline statistics query begins, all statistics counters are set to zero. While the query is active, the pipeline type determines which set of statistics are available, but these **must** be configured on the query pool when it is created. If a statistic counter is issued on a command buffer that does not support the corresponding operation, or the counter corresponds to a shading stage which is missing from any of the pipelines used while the query is active, the value of that counter is undefined after the query has been made available. At least one statistic counter relevant to the operations supported on the recording command buffer **must** be enabled.

Bits which **can** be set in VkQueryPoolCreateInfo::pipelineStatistics for query pools and in VkCommandBufferInheritanceInfo::pipelineStatistics for secondary command buffers, individually enabling pipeline statistics counters, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkQueryPipelineStatisticFlagBits {
    VK_QUERY_PIPELINE_STATISTIC_INPUT_ASSEMBLY_VERTICES_BIT = 0x00000001,
    VK_QUERY_PIPELINE_STATISTIC_INPUT_ASSEMBLY_PRIMITIVES_BIT = 0x00000002,
    VK_QUERY_PIPELINE_STATISTIC_VERTEX_SHADER_INVOCATIONS_BIT = 0x00000004,
    VK_QUERY_PIPELINE_STATISTIC_GEOMETRY_SHADER_INVOCATIONS_BIT = 0x00000008,
    VK_QUERY_PIPELINE_STATISTIC_GEOMETRY_SHADER_PRIMITIVES_BIT = 0x00000010,
    VK_QUERY_PIPELINE_STATISTIC_CLIPPING_INVOCATIONS_BIT = 0x00000020,
    VK_QUERY_PIPELINE_STATISTIC_CLIPPING_PRIMITIVES_BIT = 0x00000040,
    VK_QUERY_PIPELINE_STATISTIC_FRAGMENT_SHADER_INVOCATIONS_BIT = 0x00000080,
    VK_QUERY_PIPELINE_STATISTIC_TESSELLATION_CONTROL_SHADER_PATCHES_BIT = 0x00000100,
    VK_QUERY_PIPELINE_STATISTIC_TESSELLATION_EVALUATION_SHADER_INVOCATIONS_BIT =
0x00000200,
    VK_QUERY_PIPELINE_STATISTIC_COMPUTE_SHADER_INVOCATIONS_BIT = 0x00000400,
} VkQueryPipelineStatisticFlagBits;
```

- VK_QUERY_PIPELINE_STATISTIC_INPUT_ASSEMBLY_VERTICES_BIT specifies that queries managed by the pool will count the number of vertices processed by the input assembly stage. Vertices corresponding to incomplete primitives **may** contribute to the count.

- VK_QUERY_PIPELINE_STATISTIC_INPUT_ASSEMBLY_PRIMITIVES_BIT specifies that queries managed by the pool will count the number of primitives processed by the input assembly stage. If primitive restart is enabled, restarting the primitive topology has no effect on the count. Incomplete primitives **may** be counted.
- VK_QUERY_PIPELINE_STATISTIC_VERTEX_SHADER_INVOCATIONS_BIT specifies that queries managed by the pool will count the number of vertex shader invocations. This counter's value is incremented each time a vertex shader is invoked.
- VK_QUERY_PIPELINE_STATISTIC_GEOMETRY_SHADER_INVOCATIONS_BIT specifies that queries managed by the pool will count the number of geometry shader invocations. This counter's value is incremented each time a geometry shader is invoked. In the case of instanced geometry shaders, the geometry shader invocations count is incremented for each separate instanced invocation.
- VK_QUERY_PIPELINE_STATISTIC_GEOMETRY_SHADER_PRIMITIVES_BIT specifies that queries managed by the pool will count the number of primitives generated by geometry shader invocations. The counter's value is incremented each time the geometry shader emits a primitive. Restarting primitive topology using the SPIR-V instructions OpEndPrimitive or OpEndStreamPrimitive has no effect on the geometry shader output primitives count.
- VK_QUERY_PIPELINE_STATISTIC_CLIPPING_INVOCATIONS_BIT specifies that queries managed by the pool will count the number of primitives processed by the Primitive Clipping stage of the pipeline. The counter's value is incremented each time a primitive reaches the primitive clipping stage.
- VK_QUERY_PIPELINE_STATISTIC_CLIPPING_PRIMITIVES_BIT specifies that queries managed by the pool will count the number of primitives output by the Primitive Clipping stage of the pipeline. The counter's value is incremented each time a primitive passes the primitive clipping stage. The actual number of primitives output by the primitive clipping stage for a particular input primitive is implementation-dependent but **must** satisfy the following conditions:
  - If at least one vertex of the input primitive lies inside the clipping volume, the counter is incremented by one or more.
  - Otherwise, the counter is incremented by zero or more.
- VK_QUERY_PIPELINE_STATISTIC_FRAGMENT_SHADER_INVOCATIONS_BIT specifies that queries managed by the pool will count the number of fragment shader invocations. The counter's value is incremented each time the fragment shader is invoked.
- VK_QUERY_PIPELINE_STATISTIC_TESSELLATION_CONTROL_SHADER_PATCHES_BIT specifies that queries managed by the pool will count the number of patches processed by the tessellation control shader. The counter's value is incremented once for each patch for which a tessellation control shader is invoked.
- VK_QUERY_PIPELINE_STATISTIC_TESSELLATION_EVALUATION_SHADER_INVOCATIONS_BIT specifies that queries managed by the pool will count the number of invocations of the tessellation evaluation shader. The counter's value is incremented each time the tessellation evaluation shader is invoked.
- VK_QUERY_PIPELINE_STATISTIC_COMPUTE_SHADER_INVOCATIONS_BIT specifies that queries managed by the pool will count the number of compute shader invocations. The counter's value is incremented every time the compute shader is invoked. Implementations **may** skip the

execution of certain compute shader invocations or execute additional compute shader invocations for implementation-dependent reasons as long as the results of rendering otherwise remain unchanged.

These values are intended to measure relative statistics on one implementation. Various device architectures will count these values differently. Any or all counters **may** be affected by the issues described in Query Operation.

NOTE	For example, tile-based rendering devices **may** need to replay the scene multiple times, affecting some of the counts.

If a pipeline has `rasterizerDiscardEnable` enabled, implementations **may** discard primitives after the final pre-rasterization shader stage. As a result, if `rasterizerDiscardEnable` is enabled, the clipping input and output primitives counters **may** not be incremented.

When a pipeline statistics query finishes, the result for that query is marked as available. The application **can** copy the result to a buffer (via `vkCmdCopyQueryPoolResults`), or request it be put into host memory (via `vkGetQueryPoolResults`).

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkQueryPipelineStatisticFlags;
```

`VkQueryPipelineStatisticFlags` is a bitmask type for setting a mask of zero or more VkQueryPipelineStatisticFlagBits.

18.5. Timestamp Queries

Timestamps provide applications with a mechanism for timing the execution of commands. A timestamp is an integer value generated by the `VkPhysicalDevice`. Unlike other queries, timestamps do not operate over a range, and so do not use vkCmdBeginQuery or vkCmdEndQuery. The mechanism is built around a set of commands that allow the application to tell the `VkPhysicalDevice` to write timestamp values to a query pool and then either read timestamp values on the host (using vkGetQueryPoolResults) or copy timestamp values to a `VkBuffer` (using vkCmdCopyQueryPoolResults). The application **can** then compute differences between timestamps to determine execution time.

The number of valid bits in a timestamp value is determined by the `VkQueueFamilyProperties` ::`timestampValidBits` property of the queue on which the timestamp is written. Timestamps are supported on any queue which reports a non-zero value for `timestampValidBits` via vkGetPhysicalDeviceQueueFamilyProperties. If the `timestampComputeAndGraphics` limit is `VK_TRUE`, timestamps are supported by every queue family that supports either graphics or compute operations (see VkQueueFamilyProperties).

The number of nanoseconds it takes for a timestamp value to be incremented by 1 **can** be obtained from `VkPhysicalDeviceLimits`::`timestampPeriod` after a call to `vkGetPhysicalDeviceProperties`.

To request a timestamp and write the value to memory, call:

```
// Provided by VK_KHR_synchronization2
void vkCmdWriteTimestamp2KHR(
    VkCommandBuffer                             commandBuffer,
    VkPipelineStageFlags2                       stage,
    VkQueryPool                                 queryPool,
    uint32_t                                    query);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `stage` specifies a stage of the pipeline.
- `queryPool` is the query pool that will manage the timestamp.
- `query` is the query within the query pool that will contain the timestamp.

When `vkCmdWriteTimestamp2KHR` is submitted to a queue, it defines an execution dependency on commands that were submitted before it, and writes a timestamp to a query pool.

The first synchronization scope includes all commands that occur earlier in submission order. The synchronization scope is limited to operations on the pipeline stage specified by `stage`.

The second synchronization scope includes only the timestamp write operation.

NOTE Implementations may write the timestamp at any stage that is logically later than `stage`.

Any timestamp write that happens-after another timestamp write in the same submission **must** not have a lower value unless its value overflows the maximum supported integer bit width of the query. If `VK_KHR_calibrated_timestamps` is enabled, this extends to timestamp writes across all submissions on the same logical device: any timestamp write that happens-after another **must** not have a lower value unless its value overflows the maximum supported integer bit width of the query. Timestamps written by this command **must** be in the `VK_TIME_DOMAIN_DEVICE_KHR` time domain. If an overflow occurs, the timestamp value **must** wrap back to zero.

NOTE Comparisons between timestamps should be done between timestamps where they are guaranteed to not decrease. For example, subtracting an older timestamp from a newer one to determine the execution time of a sequence of commands is only a reliable measurement if the two timestamp writes were performed in the same submission, or if the writes were performed on the same logical device and `VK_KHR_calibrated_timestamps` is enabled.

If `vkCmdWriteTimestamp2KHR` is called while executing a render pass instance that has multiview enabled, the timestamp uses N consecutive query indices in the query pool (starting at `query`) where N is the number of bits set in the view mask of the subpass the command is executed in. The resulting query values are determined by an implementation-dependent choice of one of the following behaviors:

- The first query is a timestamp value and (if more than one bit is set in the view mask) zero is written to the remaining queries. If two timestamps are written in the same subpass, the sum of the execution time of all views between those commands is the difference between the first

query written by each command.

- All N queries are timestamp values. If two timestamps are written in the same subpass, the sum of the execution time of all views between those commands is the sum of the difference between corresponding queries written by each command. The difference between corresponding queries **may** be the execution time of a single view.

In either case, the application **can** sum the differences between all N queries to determine the total execution time.

Valid Usage

- VUID-vkCmdWriteTimestamp2-stage-03929
  If the `geometryShader` feature is not enabled, `stage` **must** not contain `VK_PIPELINE_STAGE_2_GEOMETRY_SHADER_BIT`
- VUID-vkCmdWriteTimestamp2-stage-03930
  If the `tessellationShader` feature is not enabled, `stage` **must** not contain `VK_PIPELINE_STAGE_2_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_2_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-vkCmdWriteTimestamp2-stage-03932
  If the `fragmentDensityMap` feature is not enabled, `stage` **must** not contain `VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- VUID-vkCmdWriteTimestamp2-stage-03933
  If the `transformFeedback` feature is not enabled, `stage` **must** not contain `VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT`
- VUID-vkCmdWriteTimestamp2-stage-07317
  If the `attachmentFragmentShadingRate` feature is not enabled, `stage` **must** not contain `VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-vkCmdWriteTimestamp2-stage-07947
  If the `rayTracingPipeline` feature is not enabled, `stage` **must** not contain `VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR`
- VUID-vkCmdWriteTimestamp2-synchronization2-03858
  The `synchronization2` feature **must** be enabled
- VUID-vkCmdWriteTimestamp2-stage-03859
  `stage` **must** only include a single pipeline stage
- VUID-vkCmdWriteTimestamp2-stage-03860
  `stage` **must** only include stages valid for the queue family that was used to create the command pool that `commandBuffer` was allocated from
- VUID-vkCmdWriteTimestamp2-queryPool-03861
  `queryPool` **must** have been created with a `queryType` of `VK_QUERY_TYPE_TIMESTAMP`
- VUID-vkCmdWriteTimestamp2-timestampValidBits-03863
  The command pool's queue family **must** support a non-zero `timestampValidBits`
- VUID-vkCmdWriteTimestamp2-query-04903
  `query` **must** be less than the number of queries in `queryPool`

- VUID-vkCmdWriteTimestamp2-None-03864
  All queries used by the command **must** be *unavailable*
- VUID-vkCmdWriteTimestamp2-query-03865
  If `vkCmdWriteTimestamp2KHR` is called within a render pass instance, the sum of `query` and the number of bits set in the current subpass's view mask **must** be less than or equal to the number of queries in `queryPool`

Valid Usage (Implicit)

- VUID-vkCmdWriteTimestamp2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdWriteTimestamp2-stage-parameter
  `stage` **must** be a valid combination of VkPipelineStageFlagBits2 values
- VUID-vkCmdWriteTimestamp2-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdWriteTimestamp2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdWriteTimestamp2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, compute, decode, or encode operations
- VUID-vkCmdWriteTimestamp2-commonparent
  Both of `commandBuffer`, and `queryPool` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Transfer Graphics Compute Decode Encode	Action

To request a timestamp and write the value to memory, call:

```
// Provided by VK_VERSION_1_0
void vkCmdWriteTimestamp(
    VkCommandBuffer                             commandBuffer,
    VkPipelineStageFlagBits                     pipelineStage,
    VkQueryPool                                 queryPool,
    uint32_t                                    query);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pipelineStage` is a VkPipelineStageFlagBits value, specifying a stage of the pipeline.
- `queryPool` is the query pool that will manage the timestamp.
- `query` is the query within the query pool that will contain the timestamp.

When `vkCmdWriteTimestamp` is submitted to a queue, it defines an execution dependency on commands that were submitted before it, and writes a timestamp to a query pool.

The first synchronization scope includes all commands that occur earlier in submission order. The synchronization scope is limited to operations on the pipeline stage specified by `pipelineStage`.

The second synchronization scope includes only the timestamp write operation.

NOTE Implementations may write the timestamp at any stage that is logically later than `stage`.

Any timestamp write that happens-after another timestamp write in the same submission **must** not have a lower value unless its value overflows the maximum supported integer bit width of the query. If `VK_KHR_calibrated_timestamps` is enabled, this extends to timestamp writes across all submissions on the same logical device: any timestamp write that happens-after another **must** not have a lower value unless its value overflows the maximum supported integer bit width of the query. Timestamps written by this command **must** be in the `VK_TIME_DOMAIN_DEVICE_KHR` time domain. If an overflow occurs, the timestamp value **must** wrap back to zero.

NOTE Comparisons between timestamps should be done between timestamps where they are guaranteed to not decrease. For example, subtracting an older timestamp from a newer one to determine the execution time of a sequence of commands is only a reliable measurement if the two timestamp writes were performed in the same submission, or if the writes were performed on the same logical device and `VK_KHR_calibrated_timestamps` is enabled.

If `vkCmdWriteTimestamp` is called while executing a render pass instance that has multiview enabled, the timestamp uses N consecutive query indices in the query pool (starting at `query`) where N is the number of bits set in the view mask of the subpass the command is executed in. The resulting query values are determined by an implementation-dependent choice of one of the following behaviors:

- The first query is a timestamp value and (if more than one bit is set in the view mask) zero is written to the remaining queries. If two timestamps are written in the same subpass, the sum of

the execution time of all views between those commands is the difference between the first query written by each command.

- All N queries are timestamp values. If two timestamps are written in the same subpass, the sum of the execution time of all views between those commands is the sum of the difference between corresponding queries written by each command. The difference between corresponding queries written by each command. The difference between corresponding queries **may** be the execution time of a single view.

In either case, the application **can** sum the differences between all N queries to determine the total execution time.

Valid Usage

- VUID-vkCmdWriteTimestamp-pipelineStage-04074
  `pipelineStage` **must** be a valid stage for the queue family that was used to create the command pool that `commandBuffer` was allocated from
- VUID-vkCmdWriteTimestamp-pipelineStage-04075
  If the `geometryShader` feature is not enabled, `pipelineStage` **must** not be `VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT`
- VUID-vkCmdWriteTimestamp-pipelineStage-04076
  If the `tessellationShader` feature is not enabled, `pipelineStage` **must** not be `VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT` or `VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT`
- VUID-vkCmdWriteTimestamp-pipelineStage-04078
  If the `fragmentDensityMap` feature is not enabled, `pipelineStage` **must** not be `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT`
- VUID-vkCmdWriteTimestamp-pipelineStage-04079
  If the `transformFeedback` feature is not enabled, `pipelineStage` **must** not be `VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT`
- VUID-vkCmdWriteTimestamp-fragmentShadingRate-07315
  If the `attachmentFragmentShadingRate` feature is not enabled, `pipelineStage` **must** not be `VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR`
- VUID-vkCmdWriteTimestamp-synchronization2-06489
  If the `synchronization2` feature is not enabled, `pipelineStage` **must** not be `VK_PIPELINE_STAGE_NONE`
- VUID-vkCmdWriteTimestamp-rayTracingPipeline-07944
  If the `rayTracingPipeline` feature is not enabled, `pipelineStage` **must** not be `VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR`
- VUID-vkCmdWriteTimestamp-queryPool-01416
  `queryPool` **must** have been created with a `queryType` of `VK_QUERY_TYPE_TIMESTAMP`
- VUID-vkCmdWriteTimestamp-timestampValidBits-00829
  The command pool's queue family **must** support a non-zero `timestampValidBits`
- VUID-vkCmdWriteTimestamp-query-04904
  `query` **must** be less than the number of queries in `queryPool`

- VUID-vkCmdWriteTimestamp-None-00830
  All queries used by the command **must** be *unavailable*
- VUID-vkCmdWriteTimestamp-query-00831
  If `vkCmdWriteTimestamp` is called within a render pass instance, the sum of `query` and the number of bits set in the current subpass's view mask **must** be less than or equal to the number of queries in `queryPool`

Valid Usage (Implicit)

- VUID-vkCmdWriteTimestamp-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdWriteTimestamp-pipelineStage-parameter
  `pipelineStage` **must** be a valid VkPipelineStageFlagBits value
- VUID-vkCmdWriteTimestamp-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdWriteTimestamp-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdWriteTimestamp-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, compute, decode, or encode operations
- VUID-vkCmdWriteTimestamp-commonparent
  Both of `commandBuffer`, and `queryPool` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Both	Transfer Graphics Compute Decode Encode	Action

18.6. Performance Queries

Performance queries provide applications with a mechanism for getting performance counter information about the execution of command buffers, render passes, and commands.

Each queue family advertises the performance counters that **can** be queried on a queue of that family via a call to vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR. Implementations **may** limit access to performance counters based on platform requirements or only to specialized drivers for development purposes.

NOTE	This may include no performance counters being enumerated, or a reduced set. Please refer to platform-specific documentation for guidance on any such restrictions.

Performance queries use the existing vkCmdBeginQuery and vkCmdEndQuery to control what command buffers, render passes, or commands to get performance information for.

Implementations **may** require multiple passes where the command buffer, render passes, or commands being recorded are the same and are executed on the same queue to record performance counter data. This is achieved by submitting the same batch and providing a VkPerformanceQuerySubmitInfoKHR structure containing a counter pass index. The number of passes required for a given performance query pool **can** be queried via a call to vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR.

NOTE	Command buffers created with VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT **must** not be re-submitted. Changing command buffer usage bits **may** affect performance. To avoid this, the application **should** re-record any command buffers with the VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT when multiple counter passes are required.

Performance counter results from a performance query pool **can** be obtained with the command vkGetQueryPoolResults.

The VkPerformanceCounterResultKHR union is defined as:

```
// Provided by VK_KHR_performance_query
typedef union VkPerformanceCounterResultKHR {
    int32_t     int32;
    int64_t     int64;
    uint32_t    uint32;
    uint64_t    uint64;
    float       float32;
    double      float64;
} VkPerformanceCounterResultKHR;
```

- int32 is a 32-bit signed integer value.
- int64 is a 64-bit signed integer value.

- `uint32` is a 32-bit unsigned integer value.
- `uint64` is a 64-bit unsigned integer value.
- `float32` is a 32-bit floating-point value.
- `float64` is a 64-bit floating-point value.

Performance query results are returned in an array of `VkPerformanceCounterResultKHR` unions containing the data associated with each counter in the query, stored in the same order as the counters supplied in `pCounterIndices` when creating the performance query. VkPerformanceCounterKHR::`storage` specifies how to parse the counter data.

18.6.1. Profiling Lock

To record and submit a command buffer containing a performance query pool the profiling lock **must** be held. The profiling lock **must** be acquired prior to any call to vkBeginCommandBuffer that will be using a performance query pool. The profiling lock **must** be held while any command buffer containing a performance query pool is in the *recording*, *executable*, or *pending state*. To acquire the profiling lock, call:

```
// Provided by VK_KHR_performance_query
VkResult vkAcquireProfilingLockKHR(
    VkDevice                                    device,
    const VkAcquireProfilingLockInfoKHR*        pInfo);
```

- `device` is the logical device to profile.
- `pInfo` is a pointer to a `VkAcquireProfilingLockInfoKHR` structure containing information about how the profiling is to be acquired.

Implementations **may** allow multiple actors to hold the profiling lock concurrently.

Valid Usage (Implicit)

- VUID-vkAcquireProfilingLockKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkAcquireProfilingLockKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkAcquireProfilingLockInfoKHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_TIMEOUT`

The VkAcquireProfilingLockInfoKHR structure is defined as:

```
// Provided by VK_KHR_performance_query
typedef struct VkAcquireProfilingLockInfoKHR {
    VkStructureType                   sType;
    const void*                       pNext;
    VkAcquireProfilingLockFlagsKHR    flags;
    uint64_t                          timeout;
} VkAcquireProfilingLockInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- timeout indicates how long the function waits, in nanoseconds, if the profiling lock is not available.

Valid Usage (Implicit)

- VUID-VkAcquireProfilingLockInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_ACQUIRE_PROFILING_LOCK_INFO_KHR
- VUID-VkAcquireProfilingLockInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkAcquireProfilingLockInfoKHR-flags-zerobitmask
  flags **must** be 0

If timeout is 0, vkAcquireProfilingLockKHR will not block while attempting to acquire the profiling lock. If timeout is UINT64_MAX, the function will not return until the profiling lock was acquired.

```
// Provided by VK_KHR_performance_query
typedef enum VkAcquireProfilingLockFlagBitsKHR {
} VkAcquireProfilingLockFlagBitsKHR;
```

```
// Provided by VK_KHR_performance_query
typedef VkFlags VkAcquireProfilingLockFlagsKHR;
```

VkAcquireProfilingLockFlagsKHR is a bitmask type for setting a mask, but is currently reserved for future use.

To release the profiling lock, call:

```
// Provided by VK_KHR_performance_query
void vkReleaseProfilingLockKHR(
    VkDevice                                    device);
```

- `device` is the logical device to cease profiling on.

Valid Usage

- VUID-vkReleaseProfilingLockKHR-device-03235
  The profiling lock of `device` **must** have been held via a previous successful call to vkAcquireProfilingLockKHR

Valid Usage (Implicit)

- VUID-vkReleaseProfilingLockKHR-device-parameter
  `device` **must** be a valid VkDevice handle

18.7. Transform Feedback Queries

Transform feedback queries track the number of primitives attempted to be written and actually written, by the vertex stream being captured, to a transform feedback buffer. This query is updated during drawing commands while transform feedback is active. The number of primitives actually written will be less than the number attempted to be written if the bound transform feedback buffer size was too small for the number of primitives actually drawn. Primitives are not written beyond the bound range of the transform feedback buffer. A transform feedback query is begun and ended by calling `vkCmdBeginQuery` and `vkCmdEndQuery`, respectively to query for vertex stream zero. `vkCmdBeginQueryIndexedEXT` and `vkCmdEndQueryIndexedEXT` **can** be used to begin and end transform feedback queries for any supported vertex stream. When a transform feedback query begins, the count of primitives written and primitives needed starts from zero. For each drawing command, the count is incremented as vertex attribute outputs are captured to the transform feedback buffers while transform feedback is active.

When a transform feedback query finishes, the result for that query is marked as available. The application **can** then either copy the result to a buffer (via `vkCmdCopyQueryPoolResults`) or request it be put into host memory (via `vkGetQueryPoolResults`).

18.8. Primitives Generated Queries

When a generated primitive query for a vertex stream is active, the primitives-generated count is incremented every time a primitive emitted to that stream reaches the transform feedback stage, whether or not transform feedback is active. A primitives generated query is begun and ended by calling `vkCmdBeginQuery` and `vkCmdEndQuery`, respectively to query for vertex stream zero. `vkCmdBeginQueryIndexedEXT` and `vkCmdEndQueryIndexedEXT` **can** be used to begin and end primitives generated queries for any supported vertex stream. When a primitives generated query begins, the

count of primitives generated starts from zero.

When a primitives generated query finishes, the result for that query is marked as available. The application **can** then either copy the result to a buffer (via `vkCmdCopyQueryPoolResults`) or request it be put into host memory (via `vkGetQueryPoolResults`).

NOTE	The result of this query is typically identical to `VK_QUERY_PIPELINE_STATISTIC_CLIPPING_INVOCATIONS_BIT`, but the primitives generated query is deterministic, i.e. it must be identical to the number of primitives processed. `VK_QUERY_PIPELINE_STATISTIC_CLIPPING_INVOCATIONS_BIT` may vary for implementation-dependent reasons, e.g. the same primitive may be processed multiple times for purposes of clipping.

18.9. Result Status Queries

Result status queries serve a single purpose: allowing the application to determine whether a set of operations have completed successfully or not, as indicated by the VkQueryResultStatusKHR value written when retrieving the result of a query using the `VK_QUERY_RESULT_WITH_STATUS_BIT_KHR` flag.

Unlike other query types, result status queries do not track or maintain any other data beyond the completion status, thus no other data is written when retrieving their results.

Support for result status queries is indicated by VkQueueFamilyQueryResultStatusPropertiesKHR ::`queryResultStatusSupport` , as returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR for the queue family in question.

18.10. Video Encode Feedback Queries

Video encode feedback queries allow the application to capture feedback values generated by video encode operations. As such, video encode feedback queries are available on queue families supporting video encode operations. The availability of individual video encode feedback values is indicated by the bits of VkVideoEncodeCapabilitiesKHR::`supportedEncodeFeedbackFlags`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the queries are intended to be used with.

The set of enabled video encode feedback values **must** be configured on the query pool when it is created using the `encodeFeedbackFlags` member of the VkQueryPoolVideoEncodeFeedbackCreateInfoKHR included in the `pNext` chain of VkQueryPoolCreateInfo.

The `VkQueryPoolVideoEncodeFeedbackCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkQueryPoolVideoEncodeFeedbackCreateInfoKHR {
    VkStructureType                  sType;
    const void*                      pNext;
    VkVideoEncodeFeedbackFlagsKHR    encodeFeedbackFlags;
} VkQueryPoolVideoEncodeFeedbackCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `encodeFeedbackFlags` is a bitmask of VkVideoEncodeFeedbackFlagBitsKHR values specifying the set of enabled video encode feedback values captured by queries of the new pool.

Valid Usage (Implicit)

- VUID-VkQueryPoolVideoEncodeFeedbackCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_QUERY_POOL_VIDEO_ENCODE_FEEDBACK_CREATE_INFO_KHR`
- VUID-VkQueryPoolVideoEncodeFeedbackCreateInfoKHR-encodeFeedbackFlags-parameter
  `encodeFeedbackFlags` **must** be a valid combination of VkVideoEncodeFeedbackFlagBitsKHR values
- VUID-VkQueryPoolVideoEncodeFeedbackCreateInfoKHR-encodeFeedbackFlags-requiredbitmask
  `encodeFeedbackFlags` **must** not be `0`

Bits which **can** be set in VkQueryPoolVideoEncodeFeedbackCreateInfoKHR::`encodeFeedbackFlags` for video encode feedback query pools are:

```
// Provided by VK_KHR_video_encode_queue
typedef enum VkVideoEncodeFeedbackFlagBitsKHR {
    VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BUFFER_OFFSET_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BYTES_WRITTEN_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR = 0x00000004,
} VkVideoEncodeFeedbackFlagBitsKHR;
```

- `VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BUFFER_OFFSET_BIT_KHR` specifies that queries managed by the pool will capture the byte offset of the bitstream data written by the video encode operation to the bitstream buffer specified in VkVideoEncodeInfoKHR::`dstBuffer` relative to the offset specified in VkVideoEncodeInfoKHR::`dstBufferOffset`. For the first video encode operation issued by any video encode command, this value will always be zero, meaning that bitstream data is always written to the buffer specified in VkVideoEncodeInfoKHR::`dstBuffer` starting from the offset specified in VkVideoEncodeInfoKHR::`dstBufferOffset`.
- `VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BYTES_WRITTEN_BIT_KHR` specifies that queries managed by the pool will capture the number of bytes written by the video encode operation to the bitstream buffer specified in VkVideoEncodeInfoKHR::`dstBuffer`.

- VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR specifies that queries managed by the pool will capture a boolean value indicating that the data written to the bitstream buffer specified in VkVideoEncodeInfoKHR::dstBuffer contains overridden parameters.

When retrieving the results of video encode feedback queries, the values corresponding to each enabled video encode feedback are written in the order of the bits defined above, followed by an optional value indicating availability or result status if VK_QUERY_RESULT_WITH_AVAILABILITY_BIT or VK_QUERY_RESULT_WITH_STATUS_BIT_KHR is specified, respectively.

If the result status of a video encode feedback query is negative, then the results of all enabled video encode feedback values will be undefined.

NOTE Applications should always specify VK_QUERY_RESULT_WITH_STATUS_BIT_KHR when retrieving the results of video encode feedback queries and ignore such undefined video encode feedback values for any unsuccessfully completed video encode operations.

```
// Provided by VK_KHR_video_encode_queue
typedef VkFlags VkVideoEncodeFeedbackFlagsKHR;
```

VkVideoEncodeFeedbackFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeFeedbackFlagBitsKHR.

Chapter 19. Clear Commands

19.1. Clearing Images Outside a Render Pass Instance

Color and depth/stencil images **can** be cleared outside a render pass instance using vkCmdClearColorImage or vkCmdClearDepthStencilImage, respectively. These commands are only allowed outside of a render pass instance.

To clear one or more subranges of a color image, call:

```
// Provided by VK_VERSION_1_0
void vkCmdClearColorImage(
    VkCommandBuffer                             commandBuffer,
    VkImage                                     image,
    VkImageLayout                               imageLayout,
    const VkClearColorValue*                    pColor,
    uint32_t                                    rangeCount,
    const VkImageSubresourceRange*              pRanges);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `image` is the image to be cleared.
- `imageLayout` specifies the current layout of the image subresource ranges to be cleared, and **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_GENERAL` or `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL`.
- `pColor` is a pointer to a VkClearColorValue structure containing the values that the image subresource ranges will be cleared to (see Clear Values below).
- `rangeCount` is the number of image subresource range structures in `pRanges`.
- `pRanges` is a pointer to an array of VkImageSubresourceRange structures describing a range of mipmap levels, array layers, and aspects to be cleared, as described in Image Views.

Each specified range in `pRanges` is cleared to the value specified by `pColor`.

Valid Usage

- VUID-vkCmdClearColorImage-image-01993
  The format features of `image` **must** contain `VK_FORMAT_FEATURE_TRANSFER_DST_BIT`
- VUID-vkCmdClearColorImage-image-00002
  `image` **must** have been created with `VK_IMAGE_USAGE_TRANSFER_DST_BIT` usage flag
- VUID-vkCmdClearColorImage-image-01545
  `image` **must** not use any of the formats that require a sampler $Y'C_BC_R$ conversion
- VUID-vkCmdClearColorImage-image-00003
  If `image` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object

- VUID-vkCmdClearColorImage-imageLayout-00004
  `imageLayout` **must** specify the layout of the image subresource ranges of `image` specified in `pRanges` at the time this command is executed on a `VkDevice`
- VUID-vkCmdClearColorImage-imageLayout-01394
  `imageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-vkCmdClearColorImage-aspectMask-02498
  The VkImageSubresourceRange::`aspectMask` members of the elements of the `pRanges` array **must** each only include `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-vkCmdClearColorImage-baseMipLevel-01470
  The VkImageSubresourceRange::`baseMipLevel` members of the elements of the `pRanges` array **must** each be less than the `mipLevels` specified in VkImageCreateInfo when `image` was created
- VUID-vkCmdClearColorImage-pRanges-01692
  For each VkImageSubresourceRange element of `pRanges`, if the `levelCount` member is not `VK_REMAINING_MIP_LEVELS`, then `baseMipLevel` + `levelCount` **must** be less than or equal to the `mipLevels` specified in VkImageCreateInfo when `image` was created
- VUID-vkCmdClearColorImage-baseArrayLayer-01472
  The VkImageSubresourceRange::`baseArrayLayer` members of the elements of the `pRanges` array **must** each be less than the `arrayLayers` specified in VkImageCreateInfo when `image` was created
- VUID-vkCmdClearColorImage-pRanges-01693
  For each VkImageSubresourceRange element of `pRanges`, if the `layerCount` member is not `VK_REMAINING_ARRAY_LAYERS`, then `baseArrayLayer` + `layerCount` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `image` was created
- VUID-vkCmdClearColorImage-image-00007
  `image` **must** not have a compressed or depth/stencil format
- VUID-vkCmdClearColorImage-pColor-04961
  `pColor` **must** be a valid pointer to a VkClearColorValue union
- VUID-vkCmdClearColorImage-image-09678
  If `image`'s format has components other than R and G, it **must** not have a 64-bit component width

Valid Usage (Implicit)

- VUID-vkCmdClearColorImage-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdClearColorImage-image-parameter
  `image` **must** be a valid VkImage handle
- VUID-vkCmdClearColorImage-imageLayout-parameter
  `imageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdClearColorImage-pRanges-parameter

`pRanges` **must** be a valid pointer to an array of `rangeCount` valid VkImageSubresourceRange structures

- VUID-vkCmdClearColorImage-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdClearColorImage-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdClearColorImage-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdClearColorImage-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdClearColorImage-rangeCount-arraylength
  `rangeCount` **must** be greater than `0`
- VUID-vkCmdClearColorImage-commonparent
  Both of `commandBuffer`, and `image` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics Compute	Action

To clear one or more subranges of a depth/stencil image, call:

```
// Provided by VK_VERSION_1_0
void vkCmdClearDepthStencilImage(
    VkCommandBuffer                             commandBuffer,
    VkImage                                     image,
    VkImageLayout                               imageLayout,
    const VkClearDepthStencilValue*             pDepthStencil,
    uint32_t                                    rangeCount,
    const VkImageSubresourceRange*              pRanges);
```

- commandBuffer is the command buffer into which the command will be recorded.
- image is the image to be cleared.
- imageLayout specifies the current layout of the image subresource ranges to be cleared, and **must** be VK_IMAGE_LAYOUT_GENERAL or VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL.
- pDepthStencil is a pointer to a VkClearDepthStencilValue structure containing the values that the depth and stencil image subresource ranges will be cleared to (see Clear Values below).
- rangeCount is the number of image subresource range structures in pRanges.
- pRanges is a pointer to an array of VkImageSubresourceRange structures describing a range of mipmap levels, array layers, and aspects to be cleared, as described in Image Views.

Valid Usage

- VUID-vkCmdClearDepthStencilImage-image-01994
  The format features of image **must** contain VK_FORMAT_FEATURE_TRANSFER_DST_BIT
- VUID-vkCmdClearDepthStencilImage-pRanges-02659
  If the aspect member of any element of pRanges includes VK_IMAGE_ASPECT_STENCIL_BIT, VK_IMAGE_USAGE_TRANSFER_DST_BIT **must** have been included in the VkImageCreateInfo ::usage used to create image
- VUID-vkCmdClearDepthStencilImage-pRanges-02660
  If the aspect member of any element of pRanges includes VK_IMAGE_ASPECT_DEPTH_BIT, VK_IMAGE_USAGE_TRANSFER_DST_BIT **must** have been included in the VkImageCreateInfo ::usage used to create image
- VUID-vkCmdClearDepthStencilImage-image-00010
  If image is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdClearDepthStencilImage-imageLayout-00011
  imageLayout **must** specify the layout of the image subresource ranges of image specified in pRanges at the time this command is executed on a VkDevice
- VUID-vkCmdClearDepthStencilImage-imageLayout-00012
  imageLayout **must** be either of VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL or VK_IMAGE_LAYOUT_GENERAL
- VUID-vkCmdClearDepthStencilImage-aspectMask-02824
  The VkImageSubresourceRange::aspectMask member of each element of the pRanges array **must** not include bits other than VK_IMAGE_ASPECT_DEPTH_BIT or VK_IMAGE_ASPECT_STENCIL_BIT
- VUID-vkCmdClearDepthStencilImage-image-02825
  If the image's format does not have a stencil component, then the VkImageSubresourceRange::aspectMask member of each element of the pRanges array **must** not include the VK_IMAGE_ASPECT_STENCIL_BIT bit
- VUID-vkCmdClearDepthStencilImage-image-02826
  If the image's format does not have a depth component, then the VkImageSubresourceRange::aspectMask member of each element of the pRanges array

must not include the VK_IMAGE_ASPECT_DEPTH_BIT bit

- VUID-vkCmdClearDepthStencilImage-baseMipLevel-01474
  The VkImageSubresourceRange::baseMipLevel members of the elements of the pRanges array **must** each be less than the mipLevels specified in VkImageCreateInfo when image was created
- VUID-vkCmdClearDepthStencilImage-pRanges-01694
  For each VkImageSubresourceRange element of pRanges, if the levelCount member is not VK_REMAINING_MIP_LEVELS, then baseMipLevel + levelCount **must** be less than or equal to the mipLevels specified in VkImageCreateInfo when image was created
- VUID-vkCmdClearDepthStencilImage-baseArrayLayer-01476
  The VkImageSubresourceRange::baseArrayLayer members of the elements of the pRanges array **must** each be less than the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-vkCmdClearDepthStencilImage-pRanges-01695
  For each VkImageSubresourceRange element of pRanges, if the layerCount member is not VK_REMAINING_ARRAY_LAYERS, then baseArrayLayer + layerCount **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-vkCmdClearDepthStencilImage-image-00014
  image **must** have a depth/stencil format

Valid Usage (Implicit)

- VUID-vkCmdClearDepthStencilImage-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdClearDepthStencilImage-image-parameter
  image **must** be a valid VkImage handle
- VUID-vkCmdClearDepthStencilImage-imageLayout-parameter
  imageLayout **must** be a valid VkImageLayout value
- VUID-vkCmdClearDepthStencilImage-pDepthStencil-parameter
  pDepthStencil **must** be a valid pointer to a valid VkClearDepthStencilValue structure
- VUID-vkCmdClearDepthStencilImage-pRanges-parameter
  pRanges **must** be a valid pointer to an array of rangeCount valid VkImageSubresourceRange structures
- VUID-vkCmdClearDepthStencilImage-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdClearDepthStencilImage-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdClearDepthStencilImage-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdClearDepthStencilImage-videocoding
  This command **must** only be called outside of a video coding scope

- VUID-vkCmdClearDepthStencilImage-rangeCount-arraylength
  `rangeCount` **must** be greater than `0`
- VUID-vkCmdClearDepthStencilImage-commonparent
  Both of `commandBuffer`, and `image` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics	Action

Clears outside render pass instances are treated as transfer operations for the purposes of memory barriers.

19.2. Clearing Images Inside a Render Pass Instance

To clear one or more regions of color and depth/stencil attachments inside a render pass instance, call:

```
// Provided by VK_VERSION_1_0
void vkCmdClearAttachments(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    attachmentCount,
    const VkClearAttachment*                    pAttachments,
    uint32_t                                    rectCount,
    const VkClearRect*                          pRects);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `attachmentCount` is the number of entries in the `pAttachments` array.
- `pAttachments` is a pointer to an array of VkClearAttachment structures defining the attachments to clear and the clear values to use.
- `rectCount` is the number of entries in the `pRects` array.
- `pRects` is a pointer to an array of VkClearRect structures defining regions within each selected

attachment to clear.

If the render pass has a fragment density map attachment, clears follow the operations of fragment density maps as if each clear region was a primitive which generates fragments. The clear color is applied to all pixels inside each fragment's area regardless if the pixels lie outside of the clear region. Clears **may** have a different set of supported fragment areas than draws.

Unlike other clear commands, vkCmdClearAttachments is not a transfer command. It performs its operations in rasterization order. For color attachments, the operations are executed as color attachment writes, by the VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT stage. For depth/stencil attachments, the operations are executed as depth writes and stencil writes by the VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT and VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT stages.

vkCmdClearAttachments is not affected by the bound pipeline state.

NOTE	It is generally preferable to clear attachments by using the VK_ATTACHMENT_LOAD_OP_CLEAR load operation at the start of rendering, as it is more efficient on some implementations.

If any attachment's aspectMask to be cleared is not backed by an image view, the clear has no effect on that aspect.

If an attachment being cleared refers to an image view created with an aspectMask equal to one of VK_IMAGE_ASPECT_PLANE_0_BIT, VK_IMAGE_ASPECT_PLANE_1_BIT or VK_IMAGE_ASPECT_PLANE_2_BIT, it is considered to be VK_IMAGE_ASPECT_COLOR_BIT for purposes of this command, and **must** be cleared with the VK_IMAGE_ASPECT_COLOR_BIT aspect as specified by image view creation.

Valid Usage

- VUID-vkCmdClearAttachments-aspectMask-07884
  If the current render pass instance does not use dynamic rendering, and the aspectMask member of any element of pAttachments contains VK_IMAGE_ASPECT_DEPTH_BIT, the current subpass instance's depth-stencil attachment **must** be either VK_ATTACHMENT_UNUSED or the attachment format **must** contain a depth component
- VUID-vkCmdClearAttachments-aspectMask-07885
  If the current render pass instance does not use dynamic rendering, and the aspectMask member of any element of pAttachments contains VK_IMAGE_ASPECT_STENCIL_BIT, the current subpass instance's depth-stencil attachment **must** be either VK_ATTACHMENT_UNUSED or the attachment format **must** contain a stencil component
- VUID-vkCmdClearAttachments-aspectMask-07271
  If the aspectMask member of any element of pAttachments contains VK_IMAGE_ASPECT_COLOR_BIT, the colorAttachment **must** be a valid color attachment index in the current render pass instance
- VUID-vkCmdClearAttachments-rect-02682
  The rect member of each element of pRects **must** have an extent.width greater than 0
- VUID-vkCmdClearAttachments-rect-02683
  The rect member of each element of pRects **must** have an extent.height greater than 0

- VUID-vkCmdClearAttachments-pRects-00016
  The rectangular region specified by each element of `pRects` **must** be contained within the render area of the current render pass instance
- VUID-vkCmdClearAttachments-pRects-06937
  The layers specified by each element of `pRects` **must** be contained within every attachment that `pAttachments` refers to, i.e. for each element of `pRects`, VkClearRect ::`baseArrayLayer` + VkClearRect::`layerCount` **must** be less than or equal to the number of layers rendered to in the current render pass instance
- VUID-vkCmdClearAttachments-layerCount-01934
  The `layerCount` member of each element of `pRects` **must** not be `0`
- VUID-vkCmdClearAttachments-baseArrayLayer-00018
  If the render pass instance this is recorded in uses multiview, then `baseArrayLayer` **must** be zero and `layerCount` **must** be one
- VUID-vkCmdClearAttachments-colorAttachment-09503
  The `colorAttachment` member of each element of `pAttachments` **must** not identify a color attachment that is currently mapped to `VK_ATTACHMENT_UNUSED` in `commandBuffer` via VkRenderingAttachmentLocationInfoKHR
- VUID-vkCmdClearAttachments-None-09679
  If the attachment format has components other than R and G, it **must** not have a 64-bit component width

Valid Usage (Implicit)

- VUID-vkCmdClearAttachments-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdClearAttachments-pAttachments-parameter
  `pAttachments` **must** be a valid pointer to an array of `attachmentCount` valid VkClearAttachment structures
- VUID-vkCmdClearAttachments-pRects-parameter
  `pRects` **must** be a valid pointer to an array of `rectCount` VkClearRect structures
- VUID-vkCmdClearAttachments-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdClearAttachments-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdClearAttachments-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdClearAttachments-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdClearAttachments-attachmentCount-arraylength
  `attachmentCount` **must** be greater than `0`
- VUID-vkCmdClearAttachments-rectCount-arraylength

`rectCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

The `VkClearRect` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkClearRect {
    VkRect2D    rect;
    uint32_t    baseArrayLayer;
    uint32_t    layerCount;
} VkClearRect;
```

- `rect` is the two-dimensional region to be cleared.
- `baseArrayLayer` is the first layer to be cleared.
- `layerCount` is the number of layers to clear.

The layers [`baseArrayLayer`, `baseArrayLayer` + `layerCount`) counting from the base layer of the attachment image view are cleared.

The `VkClearAttachment` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkClearAttachment {
    VkImageAspectFlags    aspectMask;
    uint32_t              colorAttachment;
    VkClearValue          clearValue;
} VkClearAttachment;
```

- `aspectMask` is a mask selecting the color, depth and/or stencil aspects of the attachment to be cleared.

- `colorAttachment` is only meaningful if `VK_IMAGE_ASPECT_COLOR_BIT` is set in `aspectMask`, in which case it is an index into the bound color attachments.
- `clearValue` is the color or depth/stencil value to clear the attachment to, as described in Clear Values below.

Valid Usage

- VUID-VkClearAttachment-aspectMask-00019
  If `aspectMask` includes `VK_IMAGE_ASPECT_COLOR_BIT`, it **must** not include `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-VkClearAttachment-aspectMask-00020
  `aspectMask` **must** not include `VK_IMAGE_ASPECT_METADATA_BIT`
- VUID-VkClearAttachment-aspectMask-02246
  `aspectMask` **must** not include `VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT` for any index *i*

Valid Usage (Implicit)

- VUID-VkClearAttachment-aspectMask-parameter
  `aspectMask` **must** be a valid combination of VkImageAspectFlagBits values
- VUID-VkClearAttachment-aspectMask-requiredbitmask
  `aspectMask` **must** not be `0`

19.3. Clear Values

The `VkClearColorValue` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef union VkClearColorValue {
    float       float32[4];
    int32_t     int32[4];
    uint32_t    uint32[4];
} VkClearColorValue;
```

- `float32` are the color clear values when the format of the image or attachment is one of the numeric formats with a numeric type that is floating-point. Floating-point values are automatically converted to the format of the image, with the clear value being treated as linear if the image is sRGB.
- `int32` are the color clear values when the format of the image or attachment has a numeric type that is signed integer (`SINT`). Signed integer values are converted to the format of the image by casting to the smaller type (with negative 32-bit values mapping to negative values in the smaller type). If the integer clear value is not representable in the target type (e.g. would overflow in conversion to that type), the clear value is undefined.

- `uint32` are the color clear values when the format of the image or attachment has a numeric type that is unsigned integer (`UINT`). Unsigned integer values are converted to the format of the image by casting to the integer type with fewer bits.

The four array elements of the clear color map to R, G, B, and A components of image formats, in order.

If the image has more than one sample, the same value is written to all samples for any pixels being cleared.

If the image or attachment format has a 64-bit component width, the first 2 array elements of each of the arrays above are reinterpreted as a single 64-bit element for the R component. The next 2 array elements are used in the same way for the G component. In other words, the union behaves as if it had the following additional members:

```
double float64[2];
int64_t int64[2];
uint64_t uint64[2];
```

The `VkClearDepthStencilValue` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkClearDepthStencilValue {
    float       depth;
    uint32_t    stencil;
} VkClearDepthStencilValue;
```

- `depth` is the clear value for the depth aspect of the depth/stencil attachment. It is a floating-point value which is automatically converted to the attachment's format.
- `stencil` is the clear value for the stencil aspect of the depth/stencil attachment. It is a 32-bit integer value which is converted to the attachment's format by taking the appropriate number of LSBs.

Valid Usage

- VUID-VkClearDepthStencilValue-depth-00022
  Unless the `VK_EXT_depth_range_unrestricted` extension is enabled `depth` **must** be between `0.0` and `1.0`, inclusive

The `VkClearValue` union is defined as:

```
// Provided by VK_VERSION_1_0
typedef union VkClearValue {
    VkClearColorValue           color;
    VkClearDepthStencilValue    depthStencil;
} VkClearValue;
```

- `color` specifies the color image clear values to use when clearing a color image or attachment.
- `depthStencil` specifies the depth and stencil clear values to use when clearing a depth/stencil image or attachment.

This union is used where part of the API requires either color or depth/stencil clear values, depending on the attachment, and defines the initial clear values in the VkRenderPassBeginInfo structure.

19.4. Filling Buffers

To clear buffer data, call:

```
// Provided by VK_VERSION_1_0
void vkCmdFillBuffer(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    dstBuffer,
    VkDeviceSize                                dstOffset,
    VkDeviceSize                                size,
    uint32_t                                    data);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `dstBuffer` is the buffer to be filled.
- `dstOffset` is the byte offset into the buffer at which to start filling, and **must** be a multiple of 4.
- `size` is the number of bytes to fill, and **must** be either a multiple of 4, or `VK_WHOLE_SIZE` to fill the range from `offset` to the end of the buffer. If `VK_WHOLE_SIZE` is used and the remaining size of the buffer is not a multiple of 4, then the nearest smaller multiple is used.
- `data` is the 4-byte word written repeatedly to the buffer to fill `size` bytes of data. The data word is written to memory according to the host endianness.

`vkCmdFillBuffer` is treated as a “transfer” operation for the purposes of synchronization barriers. The `VK_BUFFER_USAGE_TRANSFER_DST_BIT` **must** be specified in `usage` of VkBufferCreateInfo in order for the buffer to be compatible with `vkCmdFillBuffer`.

Valid Usage

- VUID-vkCmdFillBuffer-dstOffset-00024
  `dstOffset` **must** be less than the size of `dstBuffer`
- VUID-vkCmdFillBuffer-dstOffset-00025

`dstOffset` **must** be a multiple of `4`

- VUID-vkCmdFillBuffer-size-00026
  If `size` is not equal to `VK_WHOLE_SIZE`, `size` **must** be greater than `0`
- VUID-vkCmdFillBuffer-size-00027
  If `size` is not equal to `VK_WHOLE_SIZE`, `size` **must** be less than or equal to the size of `dstBuffer` minus `dstOffset`
- VUID-vkCmdFillBuffer-size-00028
  If `size` is not equal to `VK_WHOLE_SIZE`, `size` **must** be a multiple of `4`
- VUID-vkCmdFillBuffer-dstBuffer-00029
  `dstBuffer` **must** have been created with `VK_BUFFER_USAGE_TRANSFER_DST_BIT` usage flag
- VUID-vkCmdFillBuffer-apiVersion-07894
  If the VK_KHR_maintenance1 extension is not enabled and VkPhysicalDeviceProperties ::`apiVersion` is less than Vulkan 1.1, the VkCommandPool that `commandBuffer` was allocated from **must** support graphics or compute operations
- VUID-vkCmdFillBuffer-dstBuffer-00031
  If `dstBuffer` is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object

Valid Usage (Implicit)

- VUID-vkCmdFillBuffer-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdFillBuffer-dstBuffer-parameter
  `dstBuffer` **must** be a valid VkBuffer handle
- VUID-vkCmdFillBuffer-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdFillBuffer-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics or compute operations
- VUID-vkCmdFillBuffer-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdFillBuffer-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdFillBuffer-commonparent
  Both of `commandBuffer`, and `dstBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized

- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

19.5. Updating Buffers

To update buffer data inline in a command buffer, call:

```
// Provided by VK_VERSION_1_0
void vkCmdUpdateBuffer(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    dstBuffer,
    VkDeviceSize                                dstOffset,
    VkDeviceSize                                dataSize,
    const void*                                 pData);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `dstBuffer` is a handle to the buffer to be updated.
- `dstOffset` is the byte offset into the buffer to start updating, and **must** be a multiple of 4.
- `dataSize` is the number of bytes to update, and **must** be a multiple of 4.
- `pData` is a pointer to the source data for the buffer update, and **must** be at least `dataSize` bytes in size.

`dataSize` **must** be less than or equal to 65536 bytes. For larger updates, applications **can** use buffer to buffer copies.

NOTE

Buffer updates performed with `vkCmdUpdateBuffer` first copy the data into command buffer memory when the command is recorded (which requires additional storage and may incur an additional allocation), and then copy the data from the command buffer into `dstBuffer` when the command is executed on a device.

The additional cost of this functionality compared to buffer to buffer copies means it should only be used for very small amounts of data, and is why it is limited to at most 65536 bytes. Applications **can** work around this restriction by issuing multiple `vkCmdUpdateBuffer` commands to different ranges of the same buffer, but doing so is not recommended.

The source data is copied from pData to the command buffer when the command is called.

vkCmdUpdateBuffer is only allowed outside of a render pass. This command is treated as a "transfer" operation for the purposes of synchronization barriers. The VK_BUFFER_USAGE_TRANSFER_DST_BIT **must** be specified in usage of VkBufferCreateInfo in order for the buffer to be compatible with vkCmdUpdateBuffer.

Valid Usage

- VUID-vkCmdUpdateBuffer-dstOffset-00032
  dstOffset **must** be less than the size of dstBuffer
- VUID-vkCmdUpdateBuffer-dataSize-00033
  dataSize **must** be less than or equal to the size of dstBuffer minus dstOffset
- VUID-vkCmdUpdateBuffer-dstBuffer-00034
  dstBuffer **must** have been created with VK_BUFFER_USAGE_TRANSFER_DST_BIT usage flag
- VUID-vkCmdUpdateBuffer-dstBuffer-00035
  If dstBuffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdUpdateBuffer-dstOffset-00036
  dstOffset **must** be a multiple of 4
- VUID-vkCmdUpdateBuffer-dataSize-00037
  dataSize **must** be less than or equal to 65536
- VUID-vkCmdUpdateBuffer-dataSize-00038
  dataSize **must** be a multiple of 4

Valid Usage (Implicit)

- VUID-vkCmdUpdateBuffer-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdUpdateBuffer-dstBuffer-parameter
  dstBuffer **must** be a valid VkBuffer handle
- VUID-vkCmdUpdateBuffer-pData-parameter
  pData **must** be a valid pointer to an array of dataSize bytes
- VUID-vkCmdUpdateBuffer-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdUpdateBuffer-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdUpdateBuffer-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdUpdateBuffer-videocoding
  This command **must** only be called outside of a video coding scope

- VUID-vkCmdUpdateBuffer-dataSize-arraylength
  `dataSize` **must** be greater than `0`
- VUID-vkCmdUpdateBuffer-commonparent
  Both of `commandBuffer`, and `dstBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

NOTE The `pData` parameter was of type `uint32_t*` instead of `void*` prior to version 1.0.19 of the Specification and VK_HEADER_VERSION 19 of the Vulkan Header Files. This was a historical anomaly, as the source data may be of other types.

Chapter 20. Copy Commands

An application **can** copy buffer and image data using several methods described in this chapter, depending on the type of data transfer.

All copy commands are treated as “transfer” operations for the purposes of synchronization barriers.

All copy commands that have a source format with an X component in its format description read undefined values from those bits.

All copy commands that have a destination format with an X component in its format description write undefined values to those bits.

20.1. Copying Data Between Buffers

To copy data between buffer objects, call:

```
// Provided by VK_VERSION_1_0
void vkCmdCopyBuffer(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    srcBuffer,
    VkBuffer                                    dstBuffer,
    uint32_t                                    regionCount,
    const VkBufferCopy*                         pRegions);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `srcBuffer` is the source buffer.
- `dstBuffer` is the destination buffer.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkBufferCopy structures specifying the regions to copy.

Each source region specified by `pRegions` is copied from the source buffer to the destination region of the destination buffer. If any of the specified regions in `srcBuffer` overlaps in memory with any of the specified regions in `dstBuffer`, values read from those overlapping regions are undefined.

Valid Usage

- VUID-vkCmdCopyBuffer-srcOffset-00113
  The `srcOffset` member of each element of `pRegions` **must** be less than the size of `srcBuffer`
- VUID-vkCmdCopyBuffer-dstOffset-00114
  The `dstOffset` member of each element of `pRegions` **must** be less than the size of `dstBuffer`
- VUID-vkCmdCopyBuffer-size-00115
  The `size` member of each element of `pRegions` **must** be less than or equal to the size of `srcBuffer` minus `srcOffset`

- VUID-vkCmdCopyBuffer-size-00116
  The `size` member of each element of `pRegions` **must** be less than or equal to the size of `dstBuffer` minus `dstOffset`
- VUID-vkCmdCopyBuffer-pRegions-00117
  The union of the source regions, and the union of the destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-vkCmdCopyBuffer-srcBuffer-00118
  `srcBuffer` **must** have been created with `VK_BUFFER_USAGE_TRANSFER_SRC_BIT` usage flag
- VUID-vkCmdCopyBuffer-srcBuffer-00119
  If `srcBuffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdCopyBuffer-dstBuffer-00120
  `dstBuffer` **must** have been created with `VK_BUFFER_USAGE_TRANSFER_DST_BIT` usage flag
- VUID-vkCmdCopyBuffer-dstBuffer-00121
  If `dstBuffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object

Valid Usage (Implicit)

- VUID-vkCmdCopyBuffer-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyBuffer-srcBuffer-parameter
  `srcBuffer` **must** be a valid VkBuffer handle
- VUID-vkCmdCopyBuffer-dstBuffer-parameter
  `dstBuffer` **must** be a valid VkBuffer handle
- VUID-vkCmdCopyBuffer-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkBufferCopy structures
- VUID-vkCmdCopyBuffer-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyBuffer-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdCopyBuffer-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyBuffer-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdCopyBuffer-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-vkCmdCopyBuffer-commonparent
  Each of `commandBuffer`, `dstBuffer`, and `srcBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

The `VkBufferCopy` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkBufferCopy {
    VkDeviceSize    srcOffset;
    VkDeviceSize    dstOffset;
    VkDeviceSize    size;
} VkBufferCopy;
```

- `srcOffset` is the starting offset in bytes from the start of `srcBuffer`.
- `dstOffset` is the starting offset in bytes from the start of `dstBuffer`.
- `size` is the number of bytes to copy.

Valid Usage

- VUID-VkBufferCopy-size-01988
  The `size` **must** be greater than `0`

A more extensible version of the copy buffer command is defined below.

To copy data between buffer objects, call:

```
// Provided by VK_KHR_copy_commands2
void vkCmdCopyBuffer2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkCopyBufferInfo2*                    pCopyBufferInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.

- `pCopyBufferInfo` is a pointer to a VkCopyBufferInfo2 structure describing the copy parameters.

Each source region specified by `pCopyBufferInfo->pRegions` is copied from the source buffer to the destination region of the destination buffer. If any of the specified regions in `pCopyBufferInfo->srcBuffer` overlaps in memory with any of the specified regions in `pCopyBufferInfo->dstBuffer`, values read from those overlapping regions are undefined.

Valid Usage

Valid Usage (Implicit)

- VUID-vkCmdCopyBuffer2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyBuffer2-pCopyBufferInfo-parameter
  `pCopyBufferInfo` **must** be a valid pointer to a valid VkCopyBufferInfo2 structure
- VUID-vkCmdCopyBuffer2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyBuffer2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdCopyBuffer2-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyBuffer2-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

The `VkCopyBufferInfo2` structure is defined as:

```
typedef struct VkCopyBufferInfo2 {
    VkStructureType         sType;
    const void*             pNext;
    VkBuffer                srcBuffer;
    VkBuffer                dstBuffer;
    uint32_t                regionCount;
    const VkBufferCopy2*    pRegions;
} VkCopyBufferInfo2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkCopyBufferInfo2 VkCopyBufferInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcBuffer` is the source buffer.
- `dstBuffer` is the destination buffer.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkBufferCopy2 structures specifying the regions to copy.

Valid Usage

- VUID-VkCopyBufferInfo2-srcOffset-00113
  The `srcOffset` member of each element of `pRegions` **must** be less than the size of `srcBuffer`
- VUID-VkCopyBufferInfo2-dstOffset-00114
  The `dstOffset` member of each element of `pRegions` **must** be less than the size of `dstBuffer`
- VUID-VkCopyBufferInfo2-size-00115
  The `size` member of each element of `pRegions` **must** be less than or equal to the size of `srcBuffer` minus `srcOffset`
- VUID-VkCopyBufferInfo2-size-00116
  The `size` member of each element of `pRegions` **must** be less than or equal to the size of `dstBuffer` minus `dstOffset`
- VUID-VkCopyBufferInfo2-pRegions-00117
  The union of the source regions, and the union of the destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-VkCopyBufferInfo2-srcBuffer-00118
  `srcBuffer` **must** have been created with `VK_BUFFER_USAGE_TRANSFER_SRC_BIT` usage flag
- VUID-VkCopyBufferInfo2-srcBuffer-00119
  If `srcBuffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object

- VUID-VkCopyBufferInfo2-dstBuffer-00120
  `dstBuffer` **must** have been created with `VK_BUFFER_USAGE_TRANSFER_DST_BIT` usage flag
- VUID-VkCopyBufferInfo2-dstBuffer-00121
  If `dstBuffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object

Valid Usage (Implicit)

- VUID-VkCopyBufferInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_BUFFER_INFO_2`
- VUID-VkCopyBufferInfo2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyBufferInfo2-srcBuffer-parameter
  `srcBuffer` **must** be a valid VkBuffer handle
- VUID-VkCopyBufferInfo2-dstBuffer-parameter
  `dstBuffer` **must** be a valid VkBuffer handle
- VUID-VkCopyBufferInfo2-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkBufferCopy2 structures
- VUID-VkCopyBufferInfo2-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-VkCopyBufferInfo2-commonparent
  Both of `dstBuffer`, and `srcBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

The `VkBufferCopy2` structure is defined as:

```
typedef struct VkBufferCopy2 {
    VkStructureType    sType;
    const void*        pNext;
    VkDeviceSize       srcOffset;
    VkDeviceSize       dstOffset;
    VkDeviceSize       size;
} VkBufferCopy2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkBufferCopy2 VkBufferCopy2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.

- `srcOffset` is the starting offset in bytes from the start of `srcBuffer`.
- `dstOffset` is the starting offset in bytes from the start of `dstBuffer`.
- `size` is the number of bytes to copy.

Valid Usage

- VUID-VkBufferCopy2-size-01988
  The `size` **must** be greater than `0`

Valid Usage (Implicit)

- VUID-VkBufferCopy2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BUFFER_COPY_2`
- VUID-VkBufferCopy2-pNext-pNext
  `pNext` **must** be `NULL`

20.2. Copying Data Between Images

To copy data between image objects, call:

```
// Provided by VK_VERSION_1_0
void vkCmdCopyImage(
    VkCommandBuffer                             commandBuffer,
    VkImage                                     srcImage,
    VkImageLayout                               srcImageLayout,
    VkImage                                     dstImage,
    VkImageLayout                               dstImageLayout,
    uint32_t                                    regionCount,
    const VkImageCopy*                          pRegions);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `srcImage` is the source image.
- `srcImageLayout` is the current layout of the source image subresource.
- `dstImage` is the destination image.
- `dstImageLayout` is the current layout of the destination image subresource.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkImageCopy structures specifying the regions to copy.

Each source region specified by `pRegions` is copied from the source image to the destination region of the destination image. If any of the specified regions in `srcImage` overlaps in memory with any of the specified regions in `dstImage`, values read from those overlapping regions are undefined.

Multi-planar images **can** only be copied on a per-plane basis, and the subresources used in each region when copying to or from such images **must** specify only one plane, though different regions **can** specify different planes. When copying planes of multi-planar images, the format considered is the compatible format for that plane, rather than the format of the multi-planar image.

If the format of the destination image has a different block extent than the source image (e.g. one is a compressed format), the offset and extent for each of the regions specified is scaled according to the block extents of each format to match in size. Copy regions for each image **must** be aligned to a multiple of the texel block extent in each dimension, except at the edges of the image, where region extents **must** match the edge of the image.

Image data **can** be copied between images with different image types. If one image is `VK_IMAGE_TYPE_3D` and the other image is `VK_IMAGE_TYPE_2D` with multiple layers, then each slice is copied to or from a different layer; `depth` slices in the 3D image correspond to `layerCount` layers in the 2D image, with an effective `depth` of `1` used for the 2D image. If the `maintenance5` feature is enabled, all other combinations are allowed and function as if 1D images are 2D images with a height of 1. Otherwise, other combinations of image types are disallowed.

Valid Usage

- VUID-vkCmdCopyImage-pRegions-00124
  The union of all source regions, and the union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-vkCmdCopyImage-srcImage-01995
  The format features of `srcImage` **must** contain `VK_FORMAT_FEATURE_TRANSFER_SRC_BIT`
- VUID-vkCmdCopyImage-srcImageLayout-00128
  `srcImageLayout` **must** specify the layout of the image subresources of `srcImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-vkCmdCopyImage-srcImageLayout-01917
  `srcImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL`, or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-vkCmdCopyImage-srcImage-09460
  If `srcImage` and `dstImage` are the same, and any elements of `pRegions` contains the `srcSubresource` and `dstSubresource` with matching `mipLevel` and overlapping array layers, then the `srcImageLayout` and `dstImageLayout` **must** be `VK_IMAGE_LAYOUT_GENERAL` or `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`
- VUID-vkCmdCopyImage-dstImage-01996
  The format features of `dstImage` **must** contain `VK_FORMAT_FEATURE_TRANSFER_DST_BIT`
- VUID-vkCmdCopyImage-dstImageLayout-00133
  `dstImageLayout` **must** specify the layout of the image subresources of `dstImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-vkCmdCopyImage-dstImageLayout-01395
  `dstImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL`, or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-vkCmdCopyImage-srcImage-01548

If the VkFormat of each of `srcImage` and `dstImage` is not a *multi-planar format*, the VkFormat of each of `srcImage` and `dstImage` **must** be size-compatible

- VUID-vkCmdCopyImage-None-01549
  In a copy to or from a plane of a multi-planar image, the VkFormat of the image and plane **must** be compatible according to the description of compatible planes for the plane being copied
- VUID-vkCmdCopyImage-srcImage-09247
  If the VkFormat of each of `srcImage` and `dstImage` is a compressed image format, the formats **must** have the same texel block extent
- VUID-vkCmdCopyImage-srcImage-00136
  The sample count of `srcImage` and `dstImage` **must** match
- VUID-vkCmdCopyImage-srcOffset-01783
  The `srcOffset` and `extent` members of each element of `pRegions` **must** respect the image transfer granularity requirements of `commandBuffer`'s command pool's queue family, as described in VkQueueFamilyProperties
- VUID-vkCmdCopyImage-dstOffset-01784
  The `dstOffset` and `extent` members of each element of `pRegions` **must** respect the image transfer granularity requirements of `commandBuffer`'s command pool's queue family, as described in VkQueueFamilyProperties
- VUID-vkCmdCopyImage-srcImage-01551
  If neither `srcImage` nor `dstImage` has a multi-planar image format then for each element of `pRegions`, `srcSubresource.aspectMask` and `dstSubresource.aspectMask` **must** match
- VUID-vkCmdCopyImage-srcImage-08713
  If `srcImage` has a multi-planar image format, then for each element of `pRegions`, `srcSubresource.aspectMask` **must** be a single valid multi-planar aspect mask bit
- VUID-vkCmdCopyImage-dstImage-08714
  If `dstImage` has a multi-planar image format, then for each element of `pRegions`, `dstSubresource.aspectMask` **must** be a single valid multi-planar aspect mask bit
- VUID-vkCmdCopyImage-srcImage-01556
  If `srcImage` has a multi-planar image format and the `dstImage` does not have a multi-planar image format, then for each element of `pRegions`, `dstSubresource.aspectMask` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-vkCmdCopyImage-dstImage-01557
  If `dstImage` has a multi-planar image format and the `srcImage` does not have a multi-planar image format, then for each element of `pRegions`, `srcSubresource.aspectMask` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-vkCmdCopyImage-apiVersion-07932
  If the VK_KHR_maintenance1 extension is not enabled, or and either `srcImage` or `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `srcSubresource.baseArrayLayer` and `dstSubresource.baseArrayLayer` **must** both be `0`, and `srcSubresource.layerCount` and `dstSubresource.layerCount` **must** both be `1`
- VUID-vkCmdCopyImage-srcImage-04443
  If `srcImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`,

srcSubresource.baseArrayLayer **must** be 0 and srcSubresource.layerCount **must** be 1

- VUID-vkCmdCopyImage-dstImage-04444
  If dstImage is of type VK_IMAGE_TYPE_3D, then for each element of pRegions, dstSubresource.baseArrayLayer **must** be 0 and dstSubresource.layerCount **must** be 1
- VUID-vkCmdCopyImage-aspectMask-00142
  For each element of pRegions, srcSubresource.aspectMask **must** specify aspects present in srcImage
- VUID-vkCmdCopyImage-aspectMask-00143
  For each element of pRegions, dstSubresource.aspectMask **must** specify aspects present in dstImage
- VUID-vkCmdCopyImage-srcOffset-00144
  For each element of pRegions, srcOffset.x and (extent.width + srcOffset.x) **must** both be greater than or equal to 0 and less than or equal to the width of the specified srcSubresource of srcImage
- VUID-vkCmdCopyImage-srcOffset-00145
  For each element of pRegions, srcOffset.y and (extent.height + srcOffset.y) **must** both be greater than or equal to 0 and less than or equal to the height of the specified srcSubresource of srcImage
- VUID-vkCmdCopyImage-srcImage-00146
  If srcImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, srcOffset.y **must** be 0 and extent.height **must** be 1
- VUID-vkCmdCopyImage-srcOffset-00147
  If srcImage is of type VK_IMAGE_TYPE_3D, then for each element of pRegions, srcOffset.z and (extent.depth + srcOffset.z) **must** both be greater than or equal to 0 and less than or equal to the depth of the specified srcSubresource of srcImage
- VUID-vkCmdCopyImage-srcImage-01785
  If srcImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, srcOffset.z **must** be 0 and extent.depth **must** be 1
- VUID-vkCmdCopyImage-dstImage-01786
  If dstImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, dstOffset.z **must** be 0 and extent.depth **must** be 1
- VUID-vkCmdCopyImage-srcImage-01787
  If srcImage is of type VK_IMAGE_TYPE_2D, then for each element of pRegions, srcOffset.z **must** be 0
- VUID-vkCmdCopyImage-dstImage-01788
  If dstImage is of type VK_IMAGE_TYPE_2D, then for each element of pRegions, dstOffset.z **must** be 0
- VUID-vkCmdCopyImage-apiVersion-07933
  If the VK_KHR_maintenance1 extension is not enabled, and srcImage and dstImage **must** have the same VkImageType
- VUID-vkCmdCopyImage-apiVersion-08969
  If the VK_KHR_maintenance1 extension is not enabled, and srcImage or dstImage is of type VK_IMAGE_TYPE_2D, then for each element of pRegions, extent.depth **must** be 1

- VUID-vkCmdCopyImage-srcImage-07743
  If `srcImage` and `dstImage` have a different VkImageType, and the `maintenance5` feature is not enabled, one **must** be `VK_IMAGE_TYPE_3D` and the other **must** be `VK_IMAGE_TYPE_2D`
- VUID-vkCmdCopyImage-srcImage-08793
  If `srcImage` and `dstImage` have the same VkImageType, for each element of `pRegions`, if neither of the `layerCount` members of `srcSubresource` or `dstSubresource` are `VK_REMAINING_ARRAY_LAYERS`, the `layerCount` members of `srcSubresource` or `dstSubresource` **must** match
- VUID-vkCmdCopyImage-srcImage-08794
  If `srcImage` and `dstImage` have the same VkImageType, and one of the `layerCount` members of `srcSubresource` or `dstSubresource` is `VK_REMAINING_ARRAY_LAYERS`, the other member **must** be either `VK_REMAINING_ARRAY_LAYERS` or equal to the `arrayLayers` member of the VkImageCreateInfo used to create the image minus `baseArrayLayer`
- VUID-vkCmdCopyImage-srcImage-01790
  If `srcImage` and `dstImage` are both of type `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `extent.depth` **must** be `1`
- VUID-vkCmdCopyImage-srcImage-01791
  If `srcImage` is of type `VK_IMAGE_TYPE_2D`, and `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `extent.depth` **must** equal `srcSubresource.layerCount`
- VUID-vkCmdCopyImage-dstImage-01792
  If `dstImage` is of type `VK_IMAGE_TYPE_2D`, and `srcImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `extent.depth` **must** equal `dstSubresource.layerCount`
- VUID-vkCmdCopyImage-dstOffset-00150
  For each element of `pRegions`, `dstOffset.x` and (`extent.width` + `dstOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdCopyImage-dstOffset-00151
  For each element of `pRegions`, `dstOffset.y` and (`extent.height` + `dstOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdCopyImage-dstImage-00152
  If `dstImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `dstOffset.y` **must** be `0` and `extent.height` **must** be `1`
- VUID-vkCmdCopyImage-dstOffset-00153
  If `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `dstOffset.z` and (`extent.depth` + `dstOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdCopyImage-pRegions-07278
  For each element of `pRegions`, `srcOffset.x` **must** be a multiple of the texel block extent width of the VkFormat of `srcImage`
- VUID-vkCmdCopyImage-pRegions-07279
  For each element of `pRegions`, `srcOffset.y` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`

- VUID-vkCmdCopyImage-pRegions-07280
  For each element of pRegions, srcOffset.z **must** be a multiple of the texel block extent depth of the VkFormat of srcImage
- VUID-vkCmdCopyImage-pRegions-07281
  For each element of pRegions, dstOffset.x **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-vkCmdCopyImage-pRegions-07282
  For each element of pRegions, dstOffset.y **must** be a multiple of the texel block extent height of the VkFormat of dstImage
- VUID-vkCmdCopyImage-pRegions-07283
  For each element of pRegions, dstOffset.z **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-vkCmdCopyImage-srcImage-01728
  For each element of pRegions, if the sum of srcOffset.x and extent.width does not equal the width of the subresource specified by srcSubresource, extent.width **must** be a multiple of the texel block extent width of the VkFormat of srcImage
- VUID-vkCmdCopyImage-srcImage-01729
  For each element of pRegions, if the sum of srcOffset.y and extent.height does not equal the height of the subresource specified by srcSubresource, extent.height **must** be a multiple of the texel block extent height of the VkFormat of srcImage
- VUID-vkCmdCopyImage-srcImage-01730
  For each element of pRegions, if the sum of srcOffset.z and extent.depth does not equal the depth of the subresource specified by srcSubresource, extent.depth **must** be a multiple of the texel block extent depth of the VkFormat of srcImage
- VUID-vkCmdCopyImage-dstImage-01732
  For each element of pRegions, if the sum of dstOffset.x and extent.width does not equal the width of the subresource specified by dstSubresource, extent.width **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-vkCmdCopyImage-dstImage-01733
  For each element of pRegions, if the sum of dstOffset.y and extent.height does not equal the height of the subresource specified by dstSubresource, extent.height **must** be a multiple of the texel block extent height of the VkFormat of dstImage
- VUID-vkCmdCopyImage-dstImage-01734
  For each element of pRegions, if the sum of dstOffset.z and extent.depth does not equal the depth of the subresource specified by dstSubresource, extent.depth **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-vkCmdCopyImage-aspect-06662
  VK_IMAGE_USAGE_TRANSFER_SRC_BIT **must** have been included in the VkImageCreateInfo ::usage used to create srcImage
- VUID-vkCmdCopyImage-aspect-06663
  VK_IMAGE_USAGE_TRANSFER_DST_BIT **must** have been included in the VkImageCreateInfo ::usage used to create dstImage
- VUID-vkCmdCopyImage-srcImage-07966

If `srcImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object

- VUID-vkCmdCopyImage-srcSubresource-07967
  The `srcSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `srcImage` was created
- VUID-vkCmdCopyImage-srcSubresource-07968
  If `srcSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `srcSubresource.baseArrayLayer` + `srcSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `srcImage` was created
- VUID-vkCmdCopyImage-srcImage-07969
  `srcImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- VUID-vkCmdCopyImage-dstImage-07966
  If `dstImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdCopyImage-dstSubresource-07967
  The `dstSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `dstImage` was created
- VUID-vkCmdCopyImage-dstSubresource-07968
  If `dstSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `dstSubresource.baseArrayLayer` + `dstSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `dstImage` was created
- VUID-vkCmdCopyImage-dstImage-07969
  `dstImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`

Valid Usage (Implicit)

- VUID-vkCmdCopyImage-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyImage-srcImage-parameter
  `srcImage` **must** be a valid VkImage handle
- VUID-vkCmdCopyImage-srcImageLayout-parameter
  `srcImageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdCopyImage-dstImage-parameter
  `dstImage` **must** be a valid VkImage handle
- VUID-vkCmdCopyImage-dstImageLayout-parameter
  `dstImageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdCopyImage-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkImageCopy structures

- VUID-vkCmdCopyImage-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyImage-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdCopyImage-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyImage-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdCopyImage-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-vkCmdCopyImage-commonparent
  Each of `commandBuffer`, `dstImage`, and `srcImage` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

The `VkImageCopy` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageCopy {
    VkImageSubresourceLayers    srcSubresource;
    VkOffset3D                  srcOffset;
    VkImageSubresourceLayers    dstSubresource;
    VkOffset3D                  dstOffset;
    VkExtent3D                  extent;
} VkImageCopy;
```

- `srcSubresource` and `dstSubresource` are VkImageSubresourceLayers structures specifying the

image subresources of the images used for the source and destination image data, respectively.

- `srcOffset` and `dstOffset` select the initial `x`, `y`, and `z` offsets in texels of the sub-regions of the source and destination image data.
- `extent` is the size in texels of the image to copy in `width`, `height` and `depth`.

Valid Usage

- VUID-VkImageCopy-apiVersion-07940
  If the VK_KHR_sampler_ycbcr_conversion extension is not enabled, and the `aspectMask` member of `srcSubresource` and `dstSubresource` **must** match
- VUID-VkImageCopy-apiVersion-07941
  If the VK_KHR_maintenance1 extension is not enabled, and the `layerCount` member of `srcSubresource` and `dstSubresource` **must** match
- VUID-VkImageCopy-extent-06668
  `extent.width` **must** not be 0
- VUID-VkImageCopy-extent-06669
  `extent.height` **must** not be 0
- VUID-VkImageCopy-extent-06670
  `extent.depth` **must** not be 0

Valid Usage (Implicit)

- VUID-VkImageCopy-srcSubresource-parameter
  `srcSubresource` **must** be a valid VkImageSubresourceLayers structure
- VUID-VkImageCopy-dstSubresource-parameter
  `dstSubresource` **must** be a valid VkImageSubresourceLayers structure

The `VkImageSubresourceLayers` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageSubresourceLayers {
    VkImageAspectFlags    aspectMask;
    uint32_t              mipLevel;
    uint32_t              baseArrayLayer;
    uint32_t              layerCount;
} VkImageSubresourceLayers;
```

- `aspectMask` is a combination of VkImageAspectFlagBits, selecting the color, depth and/or stencil aspects to be copied.
- `mipLevel` is the mipmap level to copy
- `baseArrayLayer` and `layerCount` are the starting layer and number of layers to copy.

Valid Usage

- VUID-VkImageSubresourceLayers-aspectMask-00167
  If `aspectMask` contains `VK_IMAGE_ASPECT_COLOR_BIT`, it **must** not contain either of `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-VkImageSubresourceLayers-aspectMask-00168
  `aspectMask` **must** not contain `VK_IMAGE_ASPECT_METADATA_BIT`
- VUID-VkImageSubresourceLayers-aspectMask-02247
  `aspectMask` **must** not include `VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT` for any index *i*
- VUID-VkImageSubresourceLayers-layerCount-09243
  If the `maintenance5` feature is not enabled, `layerCount` **must** not be `VK_REMAINING_ARRAY_LAYERS`
- VUID-VkImageSubresourceLayers-layerCount-01700
  If `layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, it **must** be greater than 0

Valid Usage (Implicit)

- VUID-VkImageSubresourceLayers-aspectMask-parameter
  `aspectMask` **must** be a valid combination of VkImageAspectFlagBits values
- VUID-VkImageSubresourceLayers-aspectMask-requiredbitmask
  `aspectMask` **must** not be `0`

A more extensible version of the copy image command is defined below.

To copy data between image objects, call:

```
// Provided by VK_KHR_copy_commands2
void vkCmdCopyImage2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkCopyImageInfo2*                     pCopyImageInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pCopyImageInfo` is a pointer to a VkCopyImageInfo2 structure describing the copy parameters.

This command is functionally identical to vkCmdCopyImage, but includes extensible sub-structures that include `sType` and `pNext` parameters, allowing them to be more easily extended.

Valid Usage

Valid Usage (Implicit)

- VUID-vkCmdCopyImage2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyImage2-pCopyImageInfo-parameter
  `pCopyImageInfo` **must** be a valid pointer to a valid VkCopyImageInfo2 structure
- VUID-vkCmdCopyImage2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyImage2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdCopyImage2-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyImage2-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

The `VkCopyImageInfo2` structure is defined as:

```
typedef struct VkCopyImageInfo2 {
    VkStructureType        sType;
    const void*            pNext;
    VkImage                srcImage;
    VkImageLayout          srcImageLayout;
    VkImage                dstImage;
    VkImageLayout          dstImageLayout;
    uint32_t               regionCount;
    const VkImageCopy2*    pRegions;
} VkCopyImageInfo2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkCopyImageInfo2 VkCopyImageInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcImage` is the source image.
- `srcImageLayout` is the current layout of the source image subresource.
- `dstImage` is the destination image.
- `dstImageLayout` is the current layout of the destination image subresource.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkImageCopy2 structures specifying the regions to copy.

Valid Usage

- VUID-VkCopyImageInfo2-pRegions-00124
  The union of all source regions, and the union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-VkCopyImageInfo2-srcImage-01995
  The format features of `srcImage` **must** contain `VK_FORMAT_FEATURE_TRANSFER_SRC_BIT`
- VUID-VkCopyImageInfo2-srcImageLayout-00128
  `srcImageLayout` **must** specify the layout of the image subresources of `srcImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-VkCopyImageInfo2-srcImageLayout-01917
  `srcImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL`, or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-VkCopyImageInfo2-srcImage-09460
  If `srcImage` and `dstImage` are the same, and any elements of `pRegions` contains the `srcSubresource` and `dstSubresource` with matching `mipLevel` and overlapping array layers, then the `srcImageLayout` and `dstImageLayout` **must** be `VK_IMAGE_LAYOUT_GENERAL` or

VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR

- VUID-VkCopyImageInfo2-dstImage-01996
  The format features of dstImage **must** contain VK_FORMAT_FEATURE_TRANSFER_DST_BIT
- VUID-VkCopyImageInfo2-dstImageLayout-00133
  dstImageLayout **must** specify the layout of the image subresources of dstImage specified in pRegions at the time this command is executed on a VkDevice
- VUID-VkCopyImageInfo2-dstImageLayout-01395
  dstImageLayout **must** be VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, or VK_IMAGE_LAYOUT_GENERAL
- VUID-VkCopyImageInfo2-srcImage-01548
  If the VkFormat of each of srcImage and dstImage is not a *multi-planar format*, the VkFormat of each of srcImage and dstImage **must** be size-compatible
- VUID-VkCopyImageInfo2-None-01549
  In a copy to or from a plane of a multi-planar image, the VkFormat of the image and plane **must** be compatible according to the description of compatible planes for the plane being copied
- VUID-VkCopyImageInfo2-srcImage-09247
  If the VkFormat of each of srcImage and dstImage is a compressed image format, the formats **must** have the same texel block extent
- VUID-VkCopyImageInfo2-srcImage-00136
  The sample count of srcImage and dstImage **must** match
- VUID-VkCopyImageInfo2-srcOffset-01783
  The srcOffset and extent members of each element of pRegions **must** respect the image transfer granularity requirements of commandBuffer's command pool's queue family, as described in VkQueueFamilyProperties
- VUID-VkCopyImageInfo2-dstOffset-01784
  The dstOffset and extent members of each element of pRegions **must** respect the image transfer granularity requirements of commandBuffer's command pool's queue family, as described in VkQueueFamilyProperties
- VUID-VkCopyImageInfo2-srcImage-01551
  If neither srcImage nor dstImage has a multi-planar image format then for each element of pRegions, srcSubresource.aspectMask and dstSubresource.aspectMask **must** match
- VUID-VkCopyImageInfo2-srcImage-08713
  If srcImage has a multi-planar image format, then for each element of pRegions, srcSubresource.aspectMask **must** be a single valid multi-planar aspect mask bit
- VUID-VkCopyImageInfo2-dstImage-08714
  If dstImage has a multi-planar image format, then for each element of pRegions, dstSubresource.aspectMask **must** be a single valid multi-planar aspect mask bit
- VUID-VkCopyImageInfo2-srcImage-01556
  If srcImage has a multi-planar image format and the dstImage does not have a multi-planar image format, then for each element of pRegions, dstSubresource.aspectMask **must** be VK_IMAGE_ASPECT_COLOR_BIT

- VUID-VkCopyImageInfo2-dstImage-01557
  If `dstImage` has a multi-planar image format and the `srcImage` does not have a multi-planar image format, then for each element of `pRegions`, `srcSubresource.aspectMask` **must** be `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-VkCopyImageInfo2-apiVersion-07932
  If the VK_KHR_maintenance1 extension is not enabled, or and either `srcImage` or `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `srcSubresource.baseArrayLayer` and `dstSubresource.baseArrayLayer` **must** both be `0`, and `srcSubresource.layerCount` and `dstSubresource.layerCount` **must** both be `1`
- VUID-VkCopyImageInfo2-srcImage-04443
  If `srcImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `srcSubresource.baseArrayLayer` **must** be `0` and `srcSubresource.layerCount` **must** be `1`
- VUID-VkCopyImageInfo2-dstImage-04444
  If `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `dstSubresource.baseArrayLayer` **must** be `0` and `dstSubresource.layerCount` **must** be `1`
- VUID-VkCopyImageInfo2-aspectMask-00142
  For each element of `pRegions`, `srcSubresource.aspectMask` **must** specify aspects present in `srcImage`
- VUID-VkCopyImageInfo2-aspectMask-00143
  For each element of `pRegions`, `dstSubresource.aspectMask` **must** specify aspects present in `dstImage`
- VUID-VkCopyImageInfo2-srcOffset-00144
  For each element of `pRegions`, `srcOffset.x` and (`extent.width` + `srcOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `srcSubresource` of `srcImage`
- VUID-VkCopyImageInfo2-srcOffset-00145
  For each element of `pRegions`, `srcOffset.y` and (`extent.height` + `srcOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `srcSubresource` of `srcImage`
- VUID-VkCopyImageInfo2-srcImage-00146
  If `srcImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `srcOffset.y` **must** be `0` and `extent.height` **must** be `1`
- VUID-VkCopyImageInfo2-srcOffset-00147
  If `srcImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `srcOffset.z` and (`extent.depth` + `srcOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `srcSubresource` of `srcImage`
- VUID-VkCopyImageInfo2-srcImage-01785
  If `srcImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `srcOffset.z` **must** be `0` and `extent.depth` **must** be `1`
- VUID-VkCopyImageInfo2-dstImage-01786
  If `dstImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `dstOffset.z` **must** be `0` and `extent.depth` **must** be `1`
- VUID-VkCopyImageInfo2-srcImage-01787

If srcImage is of type VK_IMAGE_TYPE_2D, then for each element of pRegions, srcOffset.z **must** be 0

- VUID-VkCopyImageInfo2-dstImage-01788
  If dstImage is of type VK_IMAGE_TYPE_2D, then for each element of pRegions, dstOffset.z **must** be 0
- VUID-VkCopyImageInfo2-apiVersion-07933
  If the VK_KHR_maintenance1 extension is not enabled, and srcImage and dstImage **must** have the same VkImageType
- VUID-VkCopyImageInfo2-apiVersion-08969
  If the VK_KHR_maintenance1 extension is not enabled, and srcImage or dstImage is of type VK_IMAGE_TYPE_2D, then for each element of pRegions, extent.depth **must** be 1
- VUID-VkCopyImageInfo2-srcImage-07743
  If srcImage and dstImage have a different VkImageType, and the maintenance5 feature is not enabled, one **must** be VK_IMAGE_TYPE_3D and the other **must** be VK_IMAGE_TYPE_2D
- VUID-VkCopyImageInfo2-srcImage-08793
  If srcImage and dstImage have the same VkImageType, for each element of pRegions, if neither of the layerCount members of srcSubresource or dstSubresource are VK_REMAINING_ARRAY_LAYERS, the layerCount members of srcSubresource or dstSubresource **must** match
- VUID-VkCopyImageInfo2-srcImage-08794
  If srcImage and dstImage have the same VkImageType, and one of the layerCount members of srcSubresource or dstSubresource is VK_REMAINING_ARRAY_LAYERS, the other member **must** be either VK_REMAINING_ARRAY_LAYERS or equal to the arrayLayers member of the VkImageCreateInfo used to create the image minus baseArrayLayer
- VUID-VkCopyImageInfo2-srcImage-01790
  If srcImage and dstImage are both of type VK_IMAGE_TYPE_2D, then for each element of pRegions, extent.depth **must** be 1
- VUID-VkCopyImageInfo2-srcImage-01791
  If srcImage is of type VK_IMAGE_TYPE_2D, and dstImage is of type VK_IMAGE_TYPE_3D, then for each element of pRegions, extent.depth **must** equal srcSubresource.layerCount
- VUID-VkCopyImageInfo2-dstImage-01792
  If dstImage is of type VK_IMAGE_TYPE_2D, and srcImage is of type VK_IMAGE_TYPE_3D, then for each element of pRegions, extent.depth **must** equal dstSubresource.layerCount
- VUID-VkCopyImageInfo2-dstOffset-00150
  For each element of pRegions, dstOffset.x and (extent.width + dstOffset.x) **must** both be greater than or equal to 0 and less than or equal to the width of the specified dstSubresource of dstImage
- VUID-VkCopyImageInfo2-dstOffset-00151
  For each element of pRegions, dstOffset.y and (extent.height + dstOffset.y) **must** both be greater than or equal to 0 and less than or equal to the height of the specified dstSubresource of dstImage
- VUID-VkCopyImageInfo2-dstImage-00152
  If dstImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, dstOffset.y

must be `0` and `extent.height` **must** be `1`

- VUID-VkCopyImageInfo2-dstOffset-00153
  If `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `dstOffset.z` and (`extent.depth` + `dstOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `dstSubresource` of `dstImage`
- VUID-VkCopyImageInfo2-pRegions-07278
  For each element of `pRegions`, `srcOffset.x` **must** be a multiple of the texel block extent width of the VkFormat of `srcImage`
- VUID-VkCopyImageInfo2-pRegions-07279
  For each element of `pRegions`, `srcOffset.y` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`
- VUID-VkCopyImageInfo2-pRegions-07280
  For each element of `pRegions`, `srcOffset.z` **must** be a multiple of the texel block extent depth of the VkFormat of `srcImage`
- VUID-VkCopyImageInfo2-pRegions-07281
  For each element of `pRegions`, `dstOffset.x` **must** be a multiple of the texel block extent width of the VkFormat of `dstImage`
- VUID-VkCopyImageInfo2-pRegions-07282
  For each element of `pRegions`, `dstOffset.y` **must** be a multiple of the texel block extent height of the VkFormat of `dstImage`
- VUID-VkCopyImageInfo2-pRegions-07283
  For each element of `pRegions`, `dstOffset.z` **must** be a multiple of the texel block extent depth of the VkFormat of `dstImage`
- VUID-VkCopyImageInfo2-srcImage-01728
  For each element of `pRegions`, if the sum of `srcOffset.x` and `extent.width` does not equal the width of the subresource specified by `srcSubresource`, `extent.width` **must** be a multiple of the texel block extent width of the VkFormat of `srcImage`
- VUID-VkCopyImageInfo2-srcImage-01729
  For each element of `pRegions`, if the sum of `srcOffset.y` and `extent.height` does not equal the height of the subresource specified by `srcSubresource`, `extent.height` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`
- VUID-VkCopyImageInfo2-srcImage-01730
  For each element of `pRegions`, if the sum of `srcOffset.z` and `extent.depth` does not equal the depth of the subresource specified by `srcSubresource`, `extent.depth` **must** be a multiple of the texel block extent depth of the VkFormat of `srcImage`
- VUID-VkCopyImageInfo2-dstImage-01732
  For each element of `pRegions`, if the sum of `dstOffset.x` and `extent.width` does not equal the width of the subresource specified by `dstSubresource`, `extent.width` **must** be a multiple of the texel block extent width of the VkFormat of `dstImage`
- VUID-VkCopyImageInfo2-dstImage-01733
  For each element of `pRegions`, if the sum of `dstOffset.y` and `extent.height` does not equal the height of the subresource specified by `dstSubresource`, `extent.height` **must** be a multiple of the texel block extent height of the VkFormat of `dstImage`

- VUID-VkCopyImageInfo2-dstImage-01734
  For each element of pRegions, if the sum of dstOffset.z and extent.depth does not equal the depth of the subresource specified by dstSubresource, extent.depth **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-VkCopyImageInfo2-aspect-06662
  VK_IMAGE_USAGE_TRANSFER_SRC_BIT **must** have been included in the VkImageCreateInfo ::usage used to create srcImage
- VUID-VkCopyImageInfo2-aspect-06663
  VK_IMAGE_USAGE_TRANSFER_DST_BIT **must** have been included in the VkImageCreateInfo ::usage used to create dstImage
- VUID-VkCopyImageInfo2-srcImage-07966
  If srcImage is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkCopyImageInfo2-srcSubresource-07967
  The srcSubresource.mipLevel member of each element of pRegions **must** be less than the mipLevels specified in VkImageCreateInfo when srcImage was created
- VUID-VkCopyImageInfo2-srcSubresource-07968
  If srcSubresource.layerCount is not VK_REMAINING_ARRAY_LAYERS, srcSubresource.baseArrayLayer + srcSubresource.layerCount of each element of pRegions **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when srcImage was created
- VUID-VkCopyImageInfo2-srcImage-07969
  srcImage **must** not have been created with flags containing VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT
- VUID-VkCopyImageInfo2-dstImage-07966
  If dstImage is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkCopyImageInfo2-dstSubresource-07967
  The dstSubresource.mipLevel member of each element of pRegions **must** be less than the mipLevels specified in VkImageCreateInfo when dstImage was created
- VUID-VkCopyImageInfo2-dstSubresource-07968
  If dstSubresource.layerCount is not VK_REMAINING_ARRAY_LAYERS, dstSubresource.baseArrayLayer + dstSubresource.layerCount of each element of pRegions **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when dstImage was created
- VUID-VkCopyImageInfo2-dstImage-07969
  dstImage **must** not have been created with flags containing VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT

Valid Usage (Implicit)

- VUID-VkCopyImageInfo2-sType-sType

`sType` **must** be `VK_STRUCTURE_TYPE_COPY_IMAGE_INFO_2`

- VUID-VkCopyImageInfo2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyImageInfo2-srcImage-parameter
  `srcImage` **must** be a valid VkImage handle
- VUID-VkCopyImageInfo2-srcImageLayout-parameter
  `srcImageLayout` **must** be a valid VkImageLayout value
- VUID-VkCopyImageInfo2-dstImage-parameter
  `dstImage` **must** be a valid VkImage handle
- VUID-VkCopyImageInfo2-dstImageLayout-parameter
  `dstImageLayout` **must** be a valid VkImageLayout value
- VUID-VkCopyImageInfo2-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkImageCopy2 structures
- VUID-VkCopyImageInfo2-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-VkCopyImageInfo2-commonparent
  Both of `dstImage`, and `srcImage` **must** have been created, allocated, or retrieved from the same VkDevice

The `VkImageCopy2` structure is defined as:

```
typedef struct VkImageCopy2 {
    VkStructureType             sType;
    const void*                 pNext;
    VkImageSubresourceLayers    srcSubresource;
    VkOffset3D                  srcOffset;
    VkImageSubresourceLayers    dstSubresource;
    VkOffset3D                  dstOffset;
    VkExtent3D                  extent;
} VkImageCopy2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkImageCopy2 VkImageCopy2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcSubresource` and `dstSubresource` are VkImageSubresourceLayers structures specifying the image subresources of the images used for the source and destination image data, respectively.
- `srcOffset` and `dstOffset` select the initial `x`, `y`, and `z` offsets in texels of the sub-regions of the

source and destination image data.

- `extent` is the size in texels of the image to copy in `width`, `height` and `depth`.

Valid Usage

- VUID-VkImageCopy2-apiVersion-07940
  If the VK_KHR_sampler_ycbcr_conversion extension is not enabled, and the `aspectMask` member of `srcSubresource` and `dstSubresource` **must** match
- VUID-VkImageCopy2-apiVersion-07941
  If the VK_KHR_maintenance1 extension is not enabled, and the `layerCount` member of `srcSubresource` and `dstSubresource` **must** match
- VUID-VkImageCopy2-extent-06668
  `extent.width` **must** not be 0
- VUID-VkImageCopy2-extent-06669
  `extent.height` **must** not be 0
- VUID-VkImageCopy2-extent-06670
  `extent.depth` **must** not be 0

Valid Usage (Implicit)

- VUID-VkImageCopy2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_COPY_2`
- VUID-VkImageCopy2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImageCopy2-srcSubresource-parameter
  `srcSubresource` **must** be a valid VkImageSubresourceLayers structure
- VUID-VkImageCopy2-dstSubresource-parameter
  `dstSubresource` **must** be a valid VkImageSubresourceLayers structure

20.3. Copying Data Between Buffers and Images

Data **can** be copied between buffers and images, enabling applications to load and store data between images and application-defined offsets in buffer memory.

When copying between a buffer and an image, texels in the image and bytes in the buffer are accessed as follows.

Texels at each coordinate (x,y,z,layer) in the image subresource are accessed, where:

x is in the range [`imageOffset.x`, `imageOffset.x` + `imageExtent.width`),

y is in the range [`imageOffset.y`, `imageOffset.y` + `imageExtent.height`),

z is in the range [`imageOffset.z`, `imageOffset.z` + `imageExtent.depth`),

layer is in the range [eq]#[`imageSubresource.baseArrayLayer`, `imageSubresource.baseArrayLayer` + `imageSubresource.layerCount`)

For each (x,y,z,layer) coordinate in the image, bytes in the buffer are accessed at offsets in the range [texelOffset, texelOffset + blockSize), where:

texelOffset = `bufferOffset` + (⌊(x - imageOffset.x) / blockWidth⌋ × blockSize) + (⌊(y - imageOffset.y) / blockHeight⌋ × rowExtent) + (⌊(z - imageOffset.z) / blockDepth⌋ × sliceExtent) + ((layer - `imageSubresource.baseArrayLayer`) × layerExtent)

rowExtent = ⌈ max(`bufferRowLength`, `imageExtent.width`) / blockWidth ⌉ × blockSize

sliceExtent = ⌈ max(`bufferImageHeight`, `imageExtent.height`) / blockHeight ⌉ × rowExtent

layerExtent = ⌈ `imageExtent.depth` / blockDepth ⌉ × sliceExtent

and where blockSize, blockWidth, blockHeight, and blockDepth are the texel block size and extents of the image's format.

When copying between a buffer and the depth or stencil aspect of an image, data in the buffer is assumed to be laid out as separate planes rather than interleaved. Addressing calculations are thus performed for a different format than the base image, according to the aspect, as described in the following table:

Table 25. Depth/Stencil Aspect Copy Table

Base Format	Depth Aspect Format	Stencil Aspect Format
VK_FORMAT_D16_UNORM	VK_FORMAT_D16_UNORM	-
VK_FORMAT_X8_D24_UNORM_PACK32	VK_FORMAT_X8_D24_UNORM_PACK32	-
VK_FORMAT_D32_SFLOAT	VK_FORMAT_D32_SFLOAT	-
VK_FORMAT_S8_UINT	-	VK_FORMAT_S8_UINT
VK_FORMAT_D16_UNORM_S8_UINT	VK_FORMAT_D16_UNORM	VK_FORMAT_S8_UINT
VK_FORMAT_D24_UNORM_S8_UINT	VK_FORMAT_X8_D24_UNORM_PACK32	VK_FORMAT_S8_UINT
VK_FORMAT_D32_SFLOAT_S8_UINT	VK_FORMAT_D32_SFLOAT	VK_FORMAT_S8_UINT

When copying between a buffer and any plane of a multi-planar image, addressing calculations are

performed using the compatible format for that plane, rather than the format of the multi-planar image.

Each texel block is copied from one resource to the other according to the above addressing equations.

To copy data from a buffer object to an image object, call:

```
// Provided by VK_VERSION_1_0
void vkCmdCopyBufferToImage(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    srcBuffer,
    VkImage                                     dstImage,
    VkImageLayout                               dstImageLayout,
    uint32_t                                    regionCount,
    const VkBufferImageCopy*                    pRegions);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `srcBuffer` is the source buffer.
- `dstImage` is the destination image.
- `dstImageLayout` is the layout of the destination image subresources for the copy.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkBufferImageCopy structures specifying the regions to copy.

Each source region specified by `pRegions` is copied from the source buffer to the destination region of the destination image according to the addressing calculations for each resource. If any of the specified regions in `srcBuffer` overlaps in memory with any of the specified regions in `dstImage`, values read from those overlapping regions are undefined. If any region accesses a depth aspect in `dstImage` and the `VK_EXT_depth_range_unrestricted` extension is not enabled, values copied from `srcBuffer` outside of the range [0,1] will be written as undefined values to the destination image.

Copy regions for the image **must** be aligned to a multiple of the texel block extent in each dimension, except at the edges of the image, where region extents **must** match the edge of the image.

Valid Usage

- VUID-vkCmdCopyBufferToImage-dstImage-07966
  If `dstImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdCopyBufferToImage-imageSubresource-07967
  The `imageSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `dstImage` was created
- VUID-vkCmdCopyBufferToImage-imageSubresource-07968
  If `imageSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`,

`imageSubresource.baseArrayLayer` + `imageSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `dstImage` was created

- VUID-vkCmdCopyBufferToImage-dstImage-07969
  `dstImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- VUID-vkCmdCopyBufferToImage-imageSubresource-07970
  The image region specified by each element of `pRegions` **must** be contained within the specified `imageSubresource` of `dstImage`
- VUID-vkCmdCopyBufferToImage-imageSubresource-07971
  For each element of `pRegions`, `imageOffset.x` and (`imageExtent.width` + `imageOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `imageSubresource` of `dstImage`
- VUID-vkCmdCopyBufferToImage-imageSubresource-07972
  For each element of `pRegions`, `imageOffset.y` and (`imageExtent.height` + `imageOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `imageSubresource` of `dstImage`
- VUID-vkCmdCopyBufferToImage-dstImage-07973
  `dstImage` **must** have a sample count equal to `VK_SAMPLE_COUNT_1_BIT`
- VUID-vkCmdCopyBufferToImage-commandBuffer-07737
  If the queue family used to create the VkCommandPool which `commandBuffer` was allocated from does not support `VK_QUEUE_GRAPHICS_BIT` or `VK_QUEUE_COMPUTE_BIT`, the `bufferOffset` member of any element of `pRegions` **must** be a multiple of `4`
- VUID-vkCmdCopyBufferToImage-imageOffset-07738
  The `imageOffset` and `imageExtent` members of each element of `pRegions` **must** respect the image transfer granularity requirements of `commandBuffer`'s command pool's queue family, as described in VkQueueFamilyProperties
- VUID-vkCmdCopyBufferToImage-commandBuffer-07739
  If the queue family used to create the VkCommandPool which `commandBuffer` was allocated from does not support `VK_QUEUE_GRAPHICS_BIT`, for each element of `pRegions`, the `aspectMask` member of `imageSubresource` **must** not be `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT`
- VUID-vkCmdCopyBufferToImage-pRegions-00171
  `srcBuffer` **must** be large enough to contain all buffer locations that are accessed according to Buffer and Image Addressing, for each element of `pRegions`
- VUID-vkCmdCopyBufferToImage-pRegions-00173
  The union of all source regions, and the union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-vkCmdCopyBufferToImage-srcBuffer-00174
  `srcBuffer` **must** have been created with `VK_BUFFER_USAGE_TRANSFER_SRC_BIT` usage flag
- VUID-vkCmdCopyBufferToImage-dstImage-01997

The format features of dstImage **must** contain VK_FORMAT_FEATURE_TRANSFER_DST_BIT

- VUID-vkCmdCopyBufferToImage-srcBuffer-00176
  If srcBuffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdCopyBufferToImage-dstImage-00177
  dstImage **must** have been created with VK_IMAGE_USAGE_TRANSFER_DST_BIT usage flag
- VUID-vkCmdCopyBufferToImage-dstImageLayout-00180
  dstImageLayout **must** specify the layout of the image subresources of dstImage specified in pRegions at the time this command is executed on a VkDevice
- VUID-vkCmdCopyBufferToImage-dstImageLayout-01396
  dstImageLayout **must** be VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, or VK_IMAGE_LAYOUT_GENERAL
- VUID-vkCmdCopyBufferToImage-pRegions-07931
  If VK_EXT_depth_range_unrestricted is not enabled, for each element of pRegions whose imageSubresource contains a depth aspect, the data in srcBuffer **must** be in the range [0,1]
- VUID-vkCmdCopyBufferToImage-dstImage-07979
  If dstImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, imageOffset.y **must** be 0 and imageExtent.height **must** be 1
- VUID-vkCmdCopyBufferToImage-imageOffset-09104
  For each element of pRegions, imageOffset.z and (imageExtent.depth + imageOffset.z) **must** both be greater than or equal to 0 and less than or equal to the depth of the specified imageSubresource of dstImage
- VUID-vkCmdCopyBufferToImage-dstImage-07980
  If dstImage is of type VK_IMAGE_TYPE_1D or VK_IMAGE_TYPE_2D, then for each element of pRegions, imageOffset.z **must** be 0 and imageExtent.depth **must** be 1
- VUID-vkCmdCopyBufferToImage-dstImage-07274
  For each element of pRegions, imageOffset.x **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-vkCmdCopyBufferToImage-dstImage-07275
  For each element of pRegions, imageOffset.y **must** be a multiple of the texel block extent height of the VkFormat of dstImage
- VUID-vkCmdCopyBufferToImage-dstImage-07276
  For each element of pRegions, imageOffset.z **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-vkCmdCopyBufferToImage-dstImage-00207
  For each element of pRegions, if the sum of imageOffset.x and extent.width does not equal the width of the subresource specified by imageSubresource, extent.width **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-vkCmdCopyBufferToImage-dstImage-00208
  For each element of pRegions, if the sum of imageOffset.y and extent.height does not equal the height of the subresource specified by imageSubresource, extent.height **must** be a multiple of the texel block extent height of the VkFormat of dstImage

- VUID-vkCmdCopyBufferToImage-dstImage-00209
  For each element of `pRegions`, if the sum of `imageOffset.z` and `extent.depth` does not equal the depth of the subresource specified by `srcSubresource`, `extent.depth` **must** be a multiple of the texel block extent depth of the VkFormat of `dstImage`
- VUID-vkCmdCopyBufferToImage-imageSubresource-09105
  For each element of `pRegions`, `imageSubresource.aspectMask` **must** specify aspects present in `dstImage`
- VUID-vkCmdCopyBufferToImage-dstImage-07981
  If `dstImage` has a multi-planar image format, then for each element of `pRegions`, `imageSubresource.aspectMask` **must** be a single valid multi-planar aspect mask bit
- VUID-vkCmdCopyBufferToImage-dstImage-07983
  If `dstImage` is of type `VK_IMAGE_TYPE_3D`, for each element of `pRegions`, `imageSubresource.baseArrayLayer` **must** be `0` and `imageSubresource.layerCount` **must** be `1`
- VUID-vkCmdCopyBufferToImage-bufferRowLength-09106
  For each element of `pRegions`, `bufferRowLength` **must** be a multiple of the texel block extent width of the VkFormat of `dstImage`
- VUID-vkCmdCopyBufferToImage-bufferImageHeight-09107
  For each element of `pRegions`, `bufferImageHeight` **must** be a multiple of the texel block extent height of the VkFormat of `dstImage`
- VUID-vkCmdCopyBufferToImage-bufferRowLength-09108
  For each element of `pRegions`, `bufferRowLength` divided by the texel block extent width and then multiplied by the texel block size of `dstImage` **must** be less than or equal to 2^{31}-1
- VUID-vkCmdCopyBufferToImage-dstImage-07975
  If `dstImage` does not have either a depth/stencil format or a multi-planar format, then for each element of `pRegions`, `bufferOffset` **must** be a multiple of the texel block size
- VUID-vkCmdCopyBufferToImage-dstImage-07976
  If `dstImage` has a multi-planar format, then for each element of `pRegions`, `bufferOffset` **must** be a multiple of the element size of the compatible format for the format and the `aspectMask` of the `imageSubresource` as defined in Compatible Formats of Planes of Multi-Planar Formats
- VUID-vkCmdCopyBufferToImage-dstImage-07978
  If `dstImage` has a depth/stencil format, the `bufferOffset` member of any element of `pRegions` **must** be a multiple of `4`

Valid Usage (Implicit)

- VUID-vkCmdCopyBufferToImage-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyBufferToImage-srcBuffer-parameter
  `srcBuffer` **must** be a valid VkBuffer handle
- VUID-vkCmdCopyBufferToImage-dstImage-parameter
  `dstImage` **must** be a valid VkImage handle

- VUID-vkCmdCopyBufferToImage-dstImageLayout-parameter
  `dstImageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdCopyBufferToImage-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkBufferImageCopy structures
- VUID-vkCmdCopyBufferToImage-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyBufferToImage-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdCopyBufferToImage-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyBufferToImage-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdCopyBufferToImage-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-vkCmdCopyBufferToImage-commonparent
  Each of `commandBuffer`, `dstImage`, and `srcBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

To copy data from an image object to a buffer object, call:

```
// Provided by VK_VERSION_1_0
void vkCmdCopyImageToBuffer(
    VkCommandBuffer                             commandBuffer,
    VkImage                                     srcImage,
    VkImageLayout                               srcImageLayout,
    VkBuffer                                    dstBuffer,
    uint32_t                                    regionCount,
    const VkBufferImageCopy*                    pRegions);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `srcImage` is the source image.
- `srcImageLayout` is the layout of the source image subresources for the copy.
- `dstBuffer` is the destination buffer.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkBufferImageCopy structures specifying the regions to copy.

Each source region specified by `pRegions` is copied from the source image to the destination region of the destination buffer according to the addressing calculations for each resource. If any of the specified regions in `srcImage` overlaps in memory with any of the specified regions in `dstBuffer`, values read from those overlapping regions are undefined.

Copy regions for the image **must** be aligned to a multiple of the texel block extent in each dimension, except at the edges of the image, where region extents **must** match the edge of the image.

Valid Usage

- VUID-vkCmdCopyImageToBuffer-srcImage-07966
  If `srcImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdCopyImageToBuffer-imageSubresource-07967
  The `imageSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `srcImage` was created
- VUID-vkCmdCopyImageToBuffer-imageSubresource-07968
  If `imageSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `imageSubresource.baseArrayLayer + imageSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `srcImage` was created
- VUID-vkCmdCopyImageToBuffer-srcImage-07969
  `srcImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- VUID-vkCmdCopyImageToBuffer-imageSubresource-07970
  The image region specified by each element of `pRegions` **must** be contained within the

specified imageSubresource of srcImage

- VUID-vkCmdCopyImageToBuffer-imageSubresource-07971
  For each element of pRegions, imageOffset.x and (imageExtent.width + imageOffset.x) **must** both be greater than or equal to 0 and less than or equal to the width of the specified imageSubresource of srcImage
- VUID-vkCmdCopyImageToBuffer-imageSubresource-07972
  For each element of pRegions, imageOffset.y and (imageExtent.height + imageOffset.y) **must** both be greater than or equal to 0 and less than or equal to the height of the specified imageSubresource of srcImage
- VUID-vkCmdCopyImageToBuffer-srcImage-07973
  srcImage **must** have a sample count equal to VK_SAMPLE_COUNT_1_BIT
- VUID-vkCmdCopyImageToBuffer-commandBuffer-07746
  If the queue family used to create the VkCommandPool which commandBuffer was allocated from does not support VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT, the bufferOffset member of any element of pRegions **must** be a multiple of 4
- VUID-vkCmdCopyImageToBuffer-imageOffset-07747
  The imageOffset and imageExtent members of each element of pRegions **must** respect the image transfer granularity requirements of commandBuffer's command pool's queue family, as described in VkQueueFamilyProperties
- VUID-vkCmdCopyImageToBuffer-pRegions-00183
  dstBuffer **must** be large enough to contain all buffer locations that are accessed according to Buffer and Image Addressing, for each element of pRegions
- VUID-vkCmdCopyImageToBuffer-pRegions-00184
  The union of all source regions, and the union of all destination regions, specified by the elements of pRegions, **must** not overlap in memory
- VUID-vkCmdCopyImageToBuffer-srcImage-00186
  srcImage **must** have been created with VK_IMAGE_USAGE_TRANSFER_SRC_BIT usage flag
- VUID-vkCmdCopyImageToBuffer-srcImage-01998
  The format features of srcImage **must** contain VK_FORMAT_FEATURE_TRANSFER_SRC_BIT
- VUID-vkCmdCopyImageToBuffer-dstBuffer-00191
  dstBuffer **must** have been created with VK_BUFFER_USAGE_TRANSFER_DST_BIT usage flag
- VUID-vkCmdCopyImageToBuffer-dstBuffer-00192
  If dstBuffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdCopyImageToBuffer-srcImageLayout-00189
  srcImageLayout **must** specify the layout of the image subresources of srcImage specified in pRegions at the time this command is executed on a VkDevice
- VUID-vkCmdCopyImageToBuffer-srcImageLayout-01397
  srcImageLayout **must** be VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, or VK_IMAGE_LAYOUT_GENERAL
- VUID-vkCmdCopyImageToBuffer-srcImage-07979

If srcImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, imageOffset.y **must** be 0 and imageExtent.height **must** be 1

- VUID-vkCmdCopyImageToBuffer-imageOffset-09104
  For each element of pRegions, imageOffset.z and (imageExtent.depth + imageOffset.z) **must** both be greater than or equal to 0 and less than or equal to the depth of the specified imageSubresource of srcImage
- VUID-vkCmdCopyImageToBuffer-srcImage-07980
  If srcImage is of type VK_IMAGE_TYPE_1D or VK_IMAGE_TYPE_2D, then for each element of pRegions, imageOffset.z **must** be 0 and imageExtent.depth **must** be 1
- VUID-vkCmdCopyImageToBuffer-srcImage-07274
  For each element of pRegions, imageOffset.x **must** be a multiple of the texel block extent width of the VkFormat of srcImage
- VUID-vkCmdCopyImageToBuffer-srcImage-07275
  For each element of pRegions, imageOffset.y **must** be a multiple of the texel block extent height of the VkFormat of srcImage
- VUID-vkCmdCopyImageToBuffer-srcImage-07276
  For each element of pRegions, imageOffset.z **must** be a multiple of the texel block extent depth of the VkFormat of srcImage
- VUID-vkCmdCopyImageToBuffer-srcImage-00207
  For each element of pRegions, if the sum of imageOffset.x and extent.width does not equal the width of the subresource specified by imageSubresource, extent.width **must** be a multiple of the texel block extent width of the VkFormat of srcImage
- VUID-vkCmdCopyImageToBuffer-srcImage-00208
  For each element of pRegions, if the sum of imageOffset.y and extent.height does not equal the height of the subresource specified by imageSubresource, extent.height **must** be a multiple of the texel block extent height of the VkFormat of srcImage
- VUID-vkCmdCopyImageToBuffer-srcImage-00209
  For each element of pRegions, if the sum of imageOffset.z and extent.depth does not equal the depth of the subresource specified by srcSubresource, extent.depth **must** be a multiple of the texel block extent depth of the VkFormat of srcImage
- VUID-vkCmdCopyImageToBuffer-imageSubresource-09105
  For each element of pRegions, imageSubresource.aspectMask **must** specify aspects present in srcImage
- VUID-vkCmdCopyImageToBuffer-srcImage-07981
  If srcImage has a multi-planar image format, then for each element of pRegions, imageSubresource.aspectMask **must** be a single valid multi-planar aspect mask bit
- VUID-vkCmdCopyImageToBuffer-srcImage-07983
  If srcImage is of type VK_IMAGE_TYPE_3D, for each element of pRegions, imageSubresource.baseArrayLayer **must** be 0 and imageSubresource.layerCount **must** be 1
- VUID-vkCmdCopyImageToBuffer-bufferRowLength-09106
  For each element of pRegions, bufferRowLength **must** be a multiple of the texel block extent width of the VkFormat of srcImage

- VUID-vkCmdCopyImageToBuffer-bufferImageHeight-09107
  For each element of `pRegions`, `bufferImageHeight` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`
- VUID-vkCmdCopyImageToBuffer-bufferRowLength-09108
  For each element of `pRegions`, `bufferRowLength` divided by the texel block extent width and then multiplied by the texel block size of `srcImage` **must** be less than or equal to 2^{31}-1
- VUID-vkCmdCopyImageToBuffer-srcImage-07975
  If `srcImage` does not have either a depth/stencil format or a multi-planar format, then for each element of `pRegions`, `bufferOffset` **must** be a multiple of the texel block size
- VUID-vkCmdCopyImageToBuffer-srcImage-07976
  If `srcImage` has a multi-planar format, then for each element of `pRegions`, `bufferOffset` **must** be a multiple of the element size of the compatible format for the format and the `aspectMask` of the `imageSubresource` as defined in Compatible Formats of Planes of Multi-Planar Formats
- VUID-vkCmdCopyImageToBuffer-srcImage-07978
  If `srcImage` has a depth/stencil format, the `bufferOffset` member of any element of `pRegions` **must** be a multiple of 4

Valid Usage (Implicit)

- VUID-vkCmdCopyImageToBuffer-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyImageToBuffer-srcImage-parameter
  `srcImage` **must** be a valid VkImage handle
- VUID-vkCmdCopyImageToBuffer-srcImageLayout-parameter
  `srcImageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdCopyImageToBuffer-dstBuffer-parameter
  `dstBuffer` **must** be a valid VkBuffer handle
- VUID-vkCmdCopyImageToBuffer-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkBufferImageCopy structures
- VUID-vkCmdCopyImageToBuffer-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyImageToBuffer-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdCopyImageToBuffer-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyImageToBuffer-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdCopyImageToBuffer-regionCount-arraylength

`regionCount` **must** be greater than `0`

- VUID-vkCmdCopyImageToBuffer-commonparent
  Each of `commandBuffer`, `dstBuffer`, and `srcImage` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

For both vkCmdCopyBufferToImage and vkCmdCopyImageToBuffer, each element of `pRegions` is a structure defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkBufferImageCopy {
    VkDeviceSize                bufferOffset;
    uint32_t                    bufferRowLength;
    uint32_t                    bufferImageHeight;
    VkImageSubresourceLayers    imageSubresource;
    VkOffset3D                  imageOffset;
    VkExtent3D                  imageExtent;
} VkBufferImageCopy;
```

- `bufferOffset` is the offset in bytes from the start of the buffer object where the image data is copied from or to.
- `bufferRowLength` and `bufferImageHeight` specify in texels a subregion of a larger two- or three-dimensional image in buffer memory, and control the addressing calculations. If either of these values is zero, that aspect of the buffer memory is considered to be tightly packed according to the `imageExtent`.
- `imageSubresource` is a VkImageSubresourceLayers used to specify the specific image subresources of the image used for the source or destination image data.
- `imageOffset` selects the initial `x`, `y`, `z` offsets in texels of the sub-region of the source or destination image data.

- `imageExtent` is the size in texels of the image to copy in `width`, `height` and `depth`.

Valid Usage

- VUID-VkBufferImageCopy-bufferRowLength-09101
  `bufferRowLength` **must** be `0`, or greater than or equal to the `width` member of `imageExtent`
- VUID-VkBufferImageCopy-bufferImageHeight-09102
  `bufferImageHeight` **must** be `0`, or greater than or equal to the `height` member of `imageExtent`
- VUID-VkBufferImageCopy-aspectMask-09103
  The `aspectMask` member of `imageSubresource` **must** only have a single bit set
- VUID-VkBufferImageCopy-imageExtent-06659
  `imageExtent.width` **must** not be 0
- VUID-VkBufferImageCopy-imageExtent-06660
  `imageExtent.height` **must** not be 0
- VUID-VkBufferImageCopy-imageExtent-06661
  `imageExtent.depth` **must** not be 0

Valid Usage (Implicit)

- VUID-VkBufferImageCopy-imageSubresource-parameter
  `imageSubresource` **must** be a valid VkImageSubresourceLayers structure

More extensible versions of the commands to copy between buffers and images are defined below.

To copy data from a buffer object to an image object, call:

```
// Provided by VK_KHR_copy_commands2
void vkCmdCopyBufferToImage2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkCopyBufferToImageInfo2*             pCopyBufferToImageInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pCopyBufferToImageInfo` is a pointer to a VkCopyBufferToImageInfo2 structure describing the copy parameters.

This command is functionally identical to vkCmdCopyBufferToImage, but includes extensible sub-structures that include `sType` and `pNext` parameters, allowing them to be more easily extended.

Valid Usage

- VUID-vkCmdCopyBufferToImage2-commandBuffer-07737
  If the queue family used to create the VkCommandPool which `commandBuffer` was allocated from does not support `VK_QUEUE_GRAPHICS_BIT` or `VK_QUEUE_COMPUTE_BIT`, the `bufferOffset`

member of any element of pCopyBufferToImageInfo->pRegions **must** be a multiple of 4

- VUID-vkCmdCopyBufferToImage2-imageOffset-07738
  The imageOffset and imageExtent members of each element of pCopyBufferToImageInfo->pRegions **must** respect the image transfer granularity requirements of commandBuffer's command pool's queue family, as described in VkQueueFamilyProperties
- VUID-vkCmdCopyBufferToImage2-commandBuffer-07739
  If the queue family used to create the VkCommandPool which commandBuffer was allocated from does not support VK_QUEUE_GRAPHICS_BIT, for each element of pCopyBufferToImageInfo->pRegions, the aspectMask member of imageSubresource **must** not be VK_IMAGE_ASPECT_DEPTH_BIT or VK_IMAGE_ASPECT_STENCIL_BIT

Valid Usage (Implicit)

- VUID-vkCmdCopyBufferToImage2-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyBufferToImage2-pCopyBufferToImageInfo-parameter
  pCopyBufferToImageInfo **must** be a valid pointer to a valid VkCopyBufferToImageInfo2 structure
- VUID-vkCmdCopyBufferToImage2-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdCopyBufferToImage2-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdCopyBufferToImage2-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyBufferToImage2-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

The `VkCopyBufferToImageInfo2` structure is defined as:

```
typedef struct VkCopyBufferToImageInfo2 {
    VkStructureType             sType;
    const void*                 pNext;
    VkBuffer                    srcBuffer;
    VkImage                     dstImage;
    VkImageLayout               dstImageLayout;
    uint32_t                    regionCount;
    const VkBufferImageCopy2*   pRegions;
} VkCopyBufferToImageInfo2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkCopyBufferToImageInfo2 VkCopyBufferToImageInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcBuffer` is the source buffer.
- `dstImage` is the destination image.
- `dstImageLayout` is the layout of the destination image subresources for the copy.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkBufferImageCopy2 structures specifying the regions to copy.

Valid Usage

- VUID-VkCopyBufferToImageInfo2-pRegions-04565
  The image region specified by each element of `pRegions` **must** be contained within the specified `imageSubresource` of `dstImage`
- VUID-VkCopyBufferToImageInfo2-pRegions-00171
  `srcBuffer` **must** be large enough to contain all buffer locations that are accessed according

to Buffer and Image Addressing, for each element of pRegions

- VUID-VkCopyBufferToImageInfo2-pRegions-00173
  The union of all source regions, and the union of all destination regions, specified by the elements of pRegions, **must** not overlap in memory
- VUID-VkCopyBufferToImageInfo2-srcBuffer-00174
  srcBuffer **must** have been created with VK_BUFFER_USAGE_TRANSFER_SRC_BIT usage flag
- VUID-VkCopyBufferToImageInfo2-dstImage-01997
  The format features of dstImage **must** contain VK_FORMAT_FEATURE_TRANSFER_DST_BIT
- VUID-VkCopyBufferToImageInfo2-srcBuffer-00176
  If srcBuffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkCopyBufferToImageInfo2-dstImage-00177
  dstImage **must** have been created with VK_IMAGE_USAGE_TRANSFER_DST_BIT usage flag
- VUID-VkCopyBufferToImageInfo2-dstImageLayout-00180
  dstImageLayout **must** specify the layout of the image subresources of dstImage specified in pRegions at the time this command is executed on a VkDevice
- VUID-VkCopyBufferToImageInfo2-dstImageLayout-01396
  dstImageLayout **must** be VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, or VK_IMAGE_LAYOUT_GENERAL
- VUID-VkCopyBufferToImageInfo2-pRegions-07931
  If VK_EXT_depth_range_unrestricted is not enabled, for each element of pRegions whose imageSubresource contains a depth aspect, the data in srcBuffer **must** be in the range [0,1]
- VUID-VkCopyBufferToImageInfo2-dstImage-07966
  If dstImage is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkCopyBufferToImageInfo2-imageSubresource-07967
  The imageSubresource.mipLevel member of each element of pRegions **must** be less than the mipLevels specified in VkImageCreateInfo when dstImage was created
- VUID-VkCopyBufferToImageInfo2-imageSubresource-07968
  If imageSubresource.layerCount is not VK_REMAINING_ARRAY_LAYERS, imageSubresource.baseArrayLayer + imageSubresource.layerCount of each element of pRegions **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when dstImage was created
- VUID-VkCopyBufferToImageInfo2-dstImage-07969
  dstImage **must** not have been created with flags containing VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT
- VUID-VkCopyBufferToImageInfo2-dstImage-07973
  dstImage **must** have a sample count equal to VK_SAMPLE_COUNT_1_BIT
- VUID-VkCopyBufferToImageInfo2-dstImage-07979
  If dstImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, imageOffset.y **must** be 0 and imageExtent.height **must** be 1

- VUID-VkCopyBufferToImageInfo2-imageOffset-09104
  For each element of pRegions, imageOffset.z and (imageExtent.depth + imageOffset.z) **must** both be greater than or equal to 0 and less than or equal to the depth of the specified imageSubresource of dstImage
- VUID-VkCopyBufferToImageInfo2-dstImage-07980
  If dstImage is of type VK_IMAGE_TYPE_1D or VK_IMAGE_TYPE_2D, then for each element of pRegions, imageOffset.z **must** be 0 and imageExtent.depth **must** be 1
- VUID-VkCopyBufferToImageInfo2-dstImage-07274
  For each element of pRegions, imageOffset.x **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-VkCopyBufferToImageInfo2-dstImage-07275
  For each element of pRegions, imageOffset.y **must** be a multiple of the texel block extent height of the VkFormat of dstImage
- VUID-VkCopyBufferToImageInfo2-dstImage-07276
  For each element of pRegions, imageOffset.z **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-VkCopyBufferToImageInfo2-dstImage-00207
  For each element of pRegions, if the sum of imageOffset.x and extent.width does not equal the width of the subresource specified by imageSubresource, extent.width **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-VkCopyBufferToImageInfo2-dstImage-00208
  For each element of pRegions, if the sum of imageOffset.y and extent.height does not equal the height of the subresource specified by imageSubresource, extent.height **must** be a multiple of the texel block extent height of the VkFormat of dstImage
- VUID-VkCopyBufferToImageInfo2-dstImage-00209
  For each element of pRegions, if the sum of imageOffset.z and extent.depth does not equal the depth of the subresource specified by srcSubresource, extent.depth **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-VkCopyBufferToImageInfo2-imageSubresource-09105
  For each element of pRegions, imageSubresource.aspectMask **must** specify aspects present in dstImage
- VUID-VkCopyBufferToImageInfo2-dstImage-07981
  If dstImage has a multi-planar image format, then for each element of pRegions, imageSubresource.aspectMask **must** be a single valid multi-planar aspect mask bit
- VUID-VkCopyBufferToImageInfo2-dstImage-07983
  If dstImage is of type VK_IMAGE_TYPE_3D, for each element of pRegions, imageSubresource.baseArrayLayer **must** be 0 and imageSubresource.layerCount **must** be 1
- VUID-VkCopyBufferToImageInfo2-bufferRowLength-09106
  For each element of pRegions, bufferRowLength **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-VkCopyBufferToImageInfo2-bufferImageHeight-09107
  For each element of pRegions, bufferImageHeight **must** be a multiple of the texel block extent height of the VkFormat of dstImage

- VUID-VkCopyBufferToImageInfo2-bufferRowLength-09108
  For each element of `pRegions`, `bufferRowLength` divided by the texel block extent width and then multiplied by the texel block size of `dstImage` **must** be less than or equal to 2^{31}-1
- VUID-VkCopyBufferToImageInfo2-dstImage-07975
  If `dstImage` does not have either a depth/stencil format or a multi-planar format, then for each element of `pRegions`, `bufferOffset` **must** be a multiple of the texel block size
- VUID-VkCopyBufferToImageInfo2-dstImage-07976
  If `dstImage` has a multi-planar format, then for each element of `pRegions`, `bufferOffset` **must** be a multiple of the element size of the compatible format for the format and the `aspectMask` of the `imageSubresource` as defined in Compatible Formats of Planes of Multi-Planar Formats
- VUID-VkCopyBufferToImageInfo2-dstImage-07978
  If `dstImage` has a depth/stencil format, the `bufferOffset` member of any element of `pRegions` **must** be a multiple of `4`
- VUID-VkCopyBufferToImageInfo2-pRegions-06223
  For each element of `pRegions` not containing `VkCopyCommandTransformInfoQCOM` in its `pNext` chain, `imageOffset.x` and (`imageExtent.width` + `imageOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `imageSubresource` of `dstImage`
- VUID-VkCopyBufferToImageInfo2-pRegions-06224
  For each element of `pRegions` not containing `VkCopyCommandTransformInfoQCOM` in its `pNext` chain, `imageOffset.y` and (`imageExtent.height` + `imageOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `imageSubresource` of `dstImage`

Valid Usage (Implicit)

- VUID-VkCopyBufferToImageInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_BUFFER_TO_IMAGE_INFO_2`
- VUID-VkCopyBufferToImageInfo2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyBufferToImageInfo2-srcBuffer-parameter
  `srcBuffer` **must** be a valid VkBuffer handle
- VUID-VkCopyBufferToImageInfo2-dstImage-parameter
  `dstImage` **must** be a valid VkImage handle
- VUID-VkCopyBufferToImageInfo2-dstImageLayout-parameter
  `dstImageLayout` **must** be a valid VkImageLayout value
- VUID-VkCopyBufferToImageInfo2-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkBufferImageCopy2 structures
- VUID-VkCopyBufferToImageInfo2-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-VkCopyBufferToImageInfo2-commonparent

Both of dstImage, and srcBuffer **must** have been created, allocated, or retrieved from the same VkDevice

To copy data from an image object to a buffer object, call:

```
// Provided by VK_KHR_copy_commands2
void vkCmdCopyImageToBuffer2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkCopyImageToBufferInfo2*             pCopyImageToBufferInfo);
```

- commandBuffer is the command buffer into which the command will be recorded.
- pCopyImageToBufferInfo is a pointer to a VkCopyImageToBufferInfo2 structure describing the copy parameters.

This command is functionally identical to vkCmdCopyImageToBuffer, but includes extensible sub-structures that include sType and pNext parameters, allowing them to be more easily extended.

Valid Usage

- VUID-vkCmdCopyImageToBuffer2-commandBuffer-07746
  If the queue family used to create the VkCommandPool which commandBuffer was allocated from does not support VK_QUEUE_GRAPHICS_BIT or VK_QUEUE_COMPUTE_BIT, the bufferOffset member of any element of pCopyImageToBufferInfo->pRegions **must** be a multiple of 4
- VUID-vkCmdCopyImageToBuffer2-imageOffset-07747
  The imageOffset and imageExtent members of each element of pCopyImageToBufferInfo->pRegions **must** respect the image transfer granularity requirements of commandBuffer's command pool's queue family, as described in VkQueueFamilyProperties

Valid Usage (Implicit)

- VUID-vkCmdCopyImageToBuffer2-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyImageToBuffer2-pCopyImageToBufferInfo-parameter
  pCopyImageToBufferInfo **must** be a valid pointer to a valid VkCopyImageToBufferInfo2 structure
- VUID-vkCmdCopyImageToBuffer2-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdCopyImageToBuffer2-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support transfer, graphics, or compute operations
- VUID-vkCmdCopyImageToBuffer2-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyImageToBuffer2-videocoding

This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Transfer Graphics Compute	Action

The `VkCopyImageToBufferInfo2` structure is defined as:

```
typedef struct VkCopyImageToBufferInfo2 {
    VkStructureType              sType;
    const void*                  pNext;
    VkImage                      srcImage;
    VkImageLayout                srcImageLayout;
    VkBuffer                     dstBuffer;
    uint32_t                     regionCount;
    const VkBufferImageCopy2*    pRegions;
} VkCopyImageToBufferInfo2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkCopyImageToBufferInfo2 VkCopyImageToBufferInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcImage` is the source image.
- `srcImageLayout` is the layout of the source image subresources for the copy.
- `dstBuffer` is the destination buffer.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkBufferImageCopy2 structures specifying the regions to

copy.

Valid Usage

- VUID-VkCopyImageToBufferInfo2-pRegions-04566
  The image region specified by each element of `pRegions` **must** be contained within the specified `imageSubresource` of `srcImage`
- VUID-VkCopyImageToBufferInfo2-pRegions-00183
  `dstBuffer` **must** be large enough to contain all buffer locations that are accessed according to Buffer and Image Addressing, for each element of `pRegions`
- VUID-VkCopyImageToBufferInfo2-pRegions-00184
  The union of all source regions, and the union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-VkCopyImageToBufferInfo2-srcImage-00186
  `srcImage` **must** have been created with `VK_IMAGE_USAGE_TRANSFER_SRC_BIT` usage flag
- VUID-VkCopyImageToBufferInfo2-srcImage-01998
  The format features of `srcImage` **must** contain `VK_FORMAT_FEATURE_TRANSFER_SRC_BIT`
- VUID-VkCopyImageToBufferInfo2-dstBuffer-00191
  `dstBuffer` **must** have been created with `VK_BUFFER_USAGE_TRANSFER_DST_BIT` usage flag
- VUID-VkCopyImageToBufferInfo2-dstBuffer-00192
  If `dstBuffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkCopyImageToBufferInfo2-srcImageLayout-00189
  `srcImageLayout` **must** specify the layout of the image subresources of `srcImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-VkCopyImageToBufferInfo2-srcImageLayout-01397
  `srcImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL`, or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-VkCopyImageToBufferInfo2-srcImage-07966
  If `srcImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkCopyImageToBufferInfo2-imageSubresource-07967
  The `imageSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `srcImage` was created
- VUID-VkCopyImageToBufferInfo2-imageSubresource-07968
  If `imageSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `imageSubresource.baseArrayLayer` + `imageSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `srcImage` was created
- VUID-VkCopyImageToBufferInfo2-srcImage-07969
  `srcImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`

- VUID-VkCopyImageToBufferInfo2-srcImage-07973
  `srcImage` **must** have a sample count equal to `VK_SAMPLE_COUNT_1_BIT`
- VUID-VkCopyImageToBufferInfo2-srcImage-07979
  If `srcImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `imageOffset.y` **must** be `0` and `imageExtent.height` **must** be `1`
- VUID-VkCopyImageToBufferInfo2-imageOffset-09104
  For each element of `pRegions`, `imageOffset.z` and (`imageExtent.depth` + `imageOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `imageSubresource` of `srcImage`
- VUID-VkCopyImageToBufferInfo2-srcImage-07980
  If `srcImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `imageOffset.z` **must** be `0` and `imageExtent.depth` **must** be `1`
- VUID-VkCopyImageToBufferInfo2-srcImage-07274
  For each element of `pRegions`, `imageOffset.x` **must** be a multiple of the texel block extent width of the VkFormat of `srcImage`
- VUID-VkCopyImageToBufferInfo2-srcImage-07275
  For each element of `pRegions`, `imageOffset.y` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`
- VUID-VkCopyImageToBufferInfo2-srcImage-07276
  For each element of `pRegions`, `imageOffset.z` **must** be a multiple of the texel block extent depth of the VkFormat of `srcImage`
- VUID-VkCopyImageToBufferInfo2-srcImage-00207
  For each element of `pRegions`, if the sum of `imageOffset.x` and `extent.width` does not equal the width of the subresource specified by `imageSubresource`, `extent.width` **must** be a multiple of the texel block extent width of the VkFormat of `srcImage`
- VUID-VkCopyImageToBufferInfo2-srcImage-00208
  For each element of `pRegions`, if the sum of `imageOffset.y` and `extent.height` does not equal the height of the subresource specified by `imageSubresource`, `extent.height` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`
- VUID-VkCopyImageToBufferInfo2-srcImage-00209
  For each element of `pRegions`, if the sum of `imageOffset.z` and `extent.depth` does not equal the depth of the subresource specified by `srcSubresource`, `extent.depth` **must** be a multiple of the texel block extent depth of the VkFormat of `srcImage`
- VUID-VkCopyImageToBufferInfo2-imageSubresource-09105
  For each element of `pRegions`, `imageSubresource.aspectMask` **must** specify aspects present in `srcImage`
- VUID-VkCopyImageToBufferInfo2-srcImage-07981
  If `srcImage` has a multi-planar image format, then for each element of `pRegions`, `imageSubresource.aspectMask` **must** be a single valid multi-planar aspect mask bit
- VUID-VkCopyImageToBufferInfo2-srcImage-07983
  If `srcImage` is of type `VK_IMAGE_TYPE_3D`, for each element of `pRegions`, `imageSubresource.baseArrayLayer` **must** be `0` and `imageSubresource.layerCount` **must** be `1`

- VUID-VkCopyImageToBufferInfo2-bufferRowLength-09106
  For each element of `pRegions`, `bufferRowLength` **must** be a multiple of the texel block extent width of the VkFormat of `srcImage`
- VUID-VkCopyImageToBufferInfo2-bufferImageHeight-09107
  For each element of `pRegions`, `bufferImageHeight` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`
- VUID-VkCopyImageToBufferInfo2-bufferRowLength-09108
  For each element of `pRegions`, `bufferRowLength` divided by the texel block extent width and then multiplied by the texel block size of `srcImage` **must** be less than or equal to 2^{31}-1
- VUID-VkCopyImageToBufferInfo2-srcImage-07975
  If `srcImage` does not have either a depth/stencil format or a multi-planar format, then for each element of `pRegions`, `bufferOffset` **must** be a multiple of the texel block size
- VUID-VkCopyImageToBufferInfo2-srcImage-07976
  If `srcImage` has a multi-planar format, then for each element of `pRegions`, `bufferOffset` **must** be a multiple of the element size of the compatible format for the format and the `aspectMask` of the `imageSubresource` as defined in Compatible Formats of Planes of Multi-Planar Formats
- VUID-VkCopyImageToBufferInfo2-srcImage-07978
  If `srcImage` has a depth/stencil format, the `bufferOffset` member of any element of `pRegions` **must** be a multiple of `4`
- VUID-VkCopyImageToBufferInfo2-imageOffset-00197
  For each element of `pRegions` not containing `VkCopyCommandTransformInfoQCOM` in its `pNext` chain, `imageOffset.x` and (`imageExtent.width` + `imageOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `imageSubresource` of `srcImage`
- VUID-VkCopyImageToBufferInfo2-imageOffset-00198
  For each element of `pRegions` not containing `VkCopyCommandTransformInfoQCOM` in its `pNext` chain, `imageOffset.y` and (`imageExtent.height` + `imageOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `imageSubresource` of `srcImage`

Valid Usage (Implicit)

- VUID-VkCopyImageToBufferInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_IMAGE_TO_BUFFER_INFO_2`
- VUID-VkCopyImageToBufferInfo2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyImageToBufferInfo2-srcImage-parameter
  `srcImage` **must** be a valid VkImage handle
- VUID-VkCopyImageToBufferInfo2-srcImageLayout-parameter
  `srcImageLayout` **must** be a valid VkImageLayout value
- VUID-VkCopyImageToBufferInfo2-dstBuffer-parameter
  `dstBuffer` **must** be a valid VkBuffer handle

- VUID-VkCopyImageToBufferInfo2-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkBufferImageCopy2 structures
- VUID-VkCopyImageToBufferInfo2-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-VkCopyImageToBufferInfo2-commonparent
  Both of `dstBuffer`, and `srcImage` **must** have been created, allocated, or retrieved from the same VkDevice

For both vkCmdCopyBufferToImage2KHR and vkCmdCopyImageToBuffer2KHR, each element of `pRegions` is a structure defined as:

```
typedef struct VkBufferImageCopy2 {
    VkStructureType             sType;
    const void*                 pNext;
    VkDeviceSize                bufferOffset;
    uint32_t                    bufferRowLength;
    uint32_t                    bufferImageHeight;
    VkImageSubresourceLayers    imageSubresource;
    VkOffset3D                  imageOffset;
    VkExtent3D                  imageExtent;
} VkBufferImageCopy2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkBufferImageCopy2 VkBufferImageCopy2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `bufferOffset` is the offset in bytes from the start of the buffer object where the image data is copied from or to.
- `bufferRowLength` and `bufferImageHeight` specify in texels a subregion of a larger two- or three-dimensional image in buffer memory, and control the addressing calculations. If either of these values is zero, that aspect of the buffer memory is considered to be tightly packed according to the `imageExtent`.
- `imageSubresource` is a VkImageSubresourceLayers used to specify the specific image subresources of the image used for the source or destination image data.
- `imageOffset` selects the initial `x`, `y`, `z` offsets in texels of the sub-region of the source or destination image data.
- `imageExtent` is the size in texels of the image to copy in `width`, `height` and `depth`.

This structure is functionally identical to VkBufferImageCopy, but adds `sType` and `pNext` parameters,

allowing it to be more easily extended.

Valid Usage

- VUID-VkBufferImageCopy2-bufferRowLength-09101
  `bufferRowLength` **must** be `0`, or greater than or equal to the `width` member of `imageExtent`
- VUID-VkBufferImageCopy2-bufferImageHeight-09102
  `bufferImageHeight` **must** be `0`, or greater than or equal to the `height` member of `imageExtent`
- VUID-VkBufferImageCopy2-aspectMask-09103
  The `aspectMask` member of `imageSubresource` **must** only have a single bit set
- VUID-VkBufferImageCopy2-imageExtent-06659
  `imageExtent.width` **must** not be 0
- VUID-VkBufferImageCopy2-imageExtent-06660
  `imageExtent.height` **must** not be 0
- VUID-VkBufferImageCopy2-imageExtent-06661
  `imageExtent.depth` **must** not be 0

Valid Usage (Implicit)

- VUID-VkBufferImageCopy2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BUFFER_IMAGE_COPY_2`
- VUID-VkBufferImageCopy2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkBufferImageCopy2-imageSubresource-parameter
  `imageSubresource` **must** be a valid VkImageSubresourceLayers structure

The following commands can be used to copy between host memory and images. Bytes in host memory and texels in images are accessed as specified in Copying Data Between Buffers and Images, with buffers replaced with host memory.

NOTE Copies to and from an image on the host are not internally synchronized. Simultaneous access (involving writes) to overlapping image memory on the host constitutes a data race.

To copy data from host memory to an image object, call:

```
// Provided by VK_EXT_host_image_copy
VkResult vkCopyMemoryToImageEXT(
    VkDevice                                    device,
    const VkCopyMemoryToImageInfoEXT*           pCopyMemoryToImageInfo);
```

- `device` is the device which owns `pCopyMemoryToImageInfo->dstImage`.

- pCopyMemoryToImageInfo is a pointer to a VkCopyMemoryToImageInfoEXT structure describing the copy parameters.

This command is functionally similar to vkCmdCopyBufferToImage2KHR, except it is executed on the host and reads from host memory instead of a buffer. The memory of pCopyMemoryToImageInfo->dstImage is accessed by the host as if coherent.

NOTE	Because queue submissions automatically make host memory visible to the device, there would not be a need for a memory barrier before using the results of this copy operation on the device.

Valid Usage (Implicit)

- VUID-vkCopyMemoryToImageEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCopyMemoryToImageEXT-pCopyMemoryToImageInfo-parameter
  pCopyMemoryToImageInfo **must** be a valid pointer to a valid VkCopyMemoryToImageInfoEXT structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_INITIALIZATION_FAILED
- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_MEMORY_MAP_FAILED

The VkCopyMemoryToImageInfoEXT structure is defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkCopyMemoryToImageInfoEXT {
    VkStructureType                 sType;
    const void*                     pNext;
    VkHostImageCopyFlagsEXT         flags;
    VkImage                         dstImage;
    VkImageLayout                   dstImageLayout;
    uint32_t                        regionCount;
    const VkMemoryToImageCopyEXT*   pRegions;
} VkCopyMemoryToImageInfoEXT;
```

- sType is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkHostImageCopyFlagBitsEXT values describing additional copy parameters.
- `dstImage` is the destination image.
- `dstImageLayout` is the layout of the destination image subresources for the copy.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkMemoryToImageCopyEXT structures specifying the regions to copy.

`vkCopyMemoryToImageEXT` does not check whether the device memory associated with `dstImage` is currently in use before performing the copy. The application **must** guarantee that any previously submitted command that reads from or writes to the copy regions has completed before the host performs the copy.

Copy regions for the image **must** be aligned to a multiple of the texel block extent in each dimension, except at the edges of the image, where region extents **must** match the edge of the image.

Valid Usage

- VUID-VkCopyMemoryToImageInfoEXT-dstImage-09109
  If `dstImage` is sparse then all memory ranges accessed by the copy command **must** be bound as described in Binding Resource Memory
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-09111
  `dstImage` **must** have been created with `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT` set in VkImageCreateInfo::`usage`
- VUID-VkCopyMemoryToImageInfoEXT-imageOffset-09114
  If `flags` contains `VK_HOST_IMAGE_COPY_MEMCPY_EXT`, the `x`, `y`, and `z` members of the `imageOffset` member of each element of `pRegions` **must** be `0`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-09115
  If `flags` contains `VK_HOST_IMAGE_COPY_MEMCPY_EXT`, the `imageExtent` member of each element of `pRegions` **must** equal the extents of `dstImage` identified by `imageSubresource`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07966
  If `dstImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkCopyMemoryToImageInfoEXT-imageSubresource-07967
  The `imageSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `dstImage` was created
- VUID-VkCopyMemoryToImageInfoEXT-imageSubresource-07968
  If `imageSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `imageSubresource.baseArrayLayer` + `imageSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `dstImage` was created

- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07969
  `dstImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- VUID-VkCopyMemoryToImageInfoEXT-imageSubresource-07970
  The image region specified by each element of `pRegions` **must** be contained within the specified `imageSubresource` of `dstImage`
- VUID-VkCopyMemoryToImageInfoEXT-imageSubresource-07971
  For each element of `pRegions`, `imageOffset.x` and (`imageExtent.width` + `imageOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `imageSubresource` of `dstImage`
- VUID-VkCopyMemoryToImageInfoEXT-imageSubresource-07972
  For each element of `pRegions`, `imageOffset.y` and (`imageExtent.height` + `imageOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `imageSubresource` of `dstImage`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07973
  `dstImage` **must** have a sample count equal to `VK_SAMPLE_COUNT_1_BIT`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07979
  If `dstImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `imageOffset.y` **must** be `0` and `imageExtent.height` **must** be `1`
- VUID-VkCopyMemoryToImageInfoEXT-imageOffset-09104
  For each element of `pRegions`, `imageOffset.z` and (`imageExtent.depth` + `imageOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `imageSubresource` of `dstImage`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07980
  If `dstImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `imageOffset.z` **must** be `0` and `imageExtent.depth` **must** be `1`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07274
  For each element of `pRegions`, `imageOffset.x` **must** be a multiple of the texel block extent width of the VkFormat of `dstImage`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07275
  For each element of `pRegions`, `imageOffset.y` **must** be a multiple of the texel block extent height of the VkFormat of `dstImage`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07276
  For each element of `pRegions`, `imageOffset.z` **must** be a multiple of the texel block extent depth of the VkFormat of `dstImage`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-00207
  For each element of `pRegions`, if the sum of `imageOffset.x` and `extent.width` does not equal the width of the subresource specified by `imageSubresource`, `extent.width` **must** be a multiple of the texel block extent width of the VkFormat of `dstImage`
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-00208
  For each element of `pRegions`, if the sum of `imageOffset.y` and `extent.height` does not equal the height of the subresource specified by `imageSubresource`, `extent.height` **must** be a

multiple of the texel block extent height of the VkFormat of dstImage

- VUID-VkCopyMemoryToImageInfoEXT-dstImage-00209
  For each element of pRegions, if the sum of imageOffset.z and extent.depth does not equal the depth of the subresource specified by srcSubresource, extent.depth **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-VkCopyMemoryToImageInfoEXT-imageSubresource-09105
  For each element of pRegions, imageSubresource.aspectMask **must** specify aspects present in dstImage
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07981
  If dstImage has a multi-planar image format, then for each element of pRegions, imageSubresource.aspectMask **must** be a single valid multi-planar aspect mask bit
- VUID-VkCopyMemoryToImageInfoEXT-dstImage-07983
  If dstImage is of type VK_IMAGE_TYPE_3D, for each element of pRegions, imageSubresource.baseArrayLayer **must** be 0 and imageSubresource.layerCount **must** be 1
- VUID-VkCopyMemoryToImageInfoEXT-memoryRowLength-09106
  For each element of pRegions, memoryRowLength **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-VkCopyMemoryToImageInfoEXT-memoryImageHeight-09107
  For each element of pRegions, memoryImageHeight **must** be a multiple of the texel block extent height of the VkFormat of dstImage
- VUID-VkCopyMemoryToImageInfoEXT-memoryRowLength-09108
  For each element of pRegions, memoryRowLength divided by the texel block extent width and then multiplied by the texel block size of dstImage **must** be less than or equal to 2^{31}-1
- VUID-VkCopyMemoryToImageInfoEXT-dstImageLayout-09059
  dstImageLayout **must** specify the current layout of the image subresources of dstImage specified in pRegions
- VUID-VkCopyMemoryToImageInfoEXT-dstImageLayout-09060
  dstImageLayout **must** be one of the image layouts returned in VkPhysicalDeviceHostImageCopyPropertiesEXT::pCopyDstLayouts
- VUID-VkCopyMemoryToImageInfoEXT-flags-09393
  If flags includes VK_HOST_IMAGE_COPY_MEMCPY_EXT, for each region in pRegions, memoryRowLength and memoryImageHeight **must** both be 0

Valid Usage (Implicit)

- VUID-VkCopyMemoryToImageInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_COPY_MEMORY_TO_IMAGE_INFO_EXT
- VUID-VkCopyMemoryToImageInfoEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkCopyMemoryToImageInfoEXT-flags-parameter
  flags **must** be a valid combination of VkHostImageCopyFlagBitsEXT values

- VUID-VkCopyMemoryToImageInfoEXT-dstImage-parameter
  `dstImage` **must** be a valid VkImage handle
- VUID-VkCopyMemoryToImageInfoEXT-dstImageLayout-parameter
  `dstImageLayout` **must** be a valid VkImageLayout value
- VUID-VkCopyMemoryToImageInfoEXT-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkMemoryToImageCopyEXT structures
- VUID-VkCopyMemoryToImageInfoEXT-regionCount-arraylength
  `regionCount` **must** be greater than `0`

Each element of VkCopyMemoryToImageInfoEXT::`pRegions` is a structure defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkMemoryToImageCopyEXT {
    VkStructureType             sType;
    const void*                 pNext;
    const void*                 pHostPointer;
    uint32_t                    memoryRowLength;
    uint32_t                    memoryImageHeight;
    VkImageSubresourceLayers    imageSubresource;
    VkOffset3D                  imageOffset;
    VkExtent3D                  imageExtent;
} VkMemoryToImageCopyEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pHostPointer` is the host memory address which is the source of the copy.
- `memoryRowLength` and `memoryImageHeight` specify in texels a subregion of a larger two- or three-dimensional image in host memory, and control the addressing calculations. If either of these values is zero, that aspect of the host memory is considered to be tightly packed according to the `imageExtent`.
- `imageSubresource` is a VkImageSubresourceLayers used to specify the specific image subresources of the image used for the source or destination image data.
- `imageOffset` selects the initial `x`, `y`, `z` offsets in texels of the sub-region of the destination image data.
- `imageExtent` is the size in texels of the image to copy in `width`, `height` and `depth`.

This structure is functionally similar to VkBufferImageCopy2, except it defines host memory as the source of copy instead of a buffer. In particular, the same data packing rules and restrictions as that structure apply here as well.

Valid Usage

- VUID-VkMemoryToImageCopyEXT-pHostPointer-09061
  `pHostPointer` **must** point to memory that is large enough to contain all memory locations that are accessed according to Buffer and Image Addressing, for each element of `pRegions`
- VUID-VkMemoryToImageCopyEXT-pRegions-09062
  The union of all source regions, and the union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-VkMemoryToImageCopyEXT-memoryRowLength-09101
  `memoryRowLength` **must** be `0`, or greater than or equal to the `width` member of `imageExtent`
- VUID-VkMemoryToImageCopyEXT-memoryImageHeight-09102
  `memoryImageHeight` **must** be `0`, or greater than or equal to the `height` member of `imageExtent`
- VUID-VkMemoryToImageCopyEXT-aspectMask-09103
  The `aspectMask` member of `imageSubresource` **must** only have a single bit set
- VUID-VkMemoryToImageCopyEXT-imageExtent-06659
  `imageExtent.width` **must** not be 0
- VUID-VkMemoryToImageCopyEXT-imageExtent-06660
  `imageExtent.height` **must** not be 0
- VUID-VkMemoryToImageCopyEXT-imageExtent-06661
  `imageExtent.depth` **must** not be 0

Valid Usage (Implicit)

- VUID-VkMemoryToImageCopyEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MEMORY_TO_IMAGE_COPY_EXT`
- VUID-VkMemoryToImageCopyEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkMemoryToImageCopyEXT-pHostPointer-parameter
  `pHostPointer` **must** be a pointer value
- VUID-VkMemoryToImageCopyEXT-imageSubresource-parameter
  `imageSubresource` **must** be a valid VkImageSubresourceLayers structure

To copy data from an image object to host memory, call:

```
// Provided by VK_EXT_host_image_copy
VkResult vkCopyImageToMemoryEXT(
    VkDevice                                    device,
    const VkCopyImageToMemoryInfoEXT*           pCopyImageToMemoryInfo);
```

- `device` is the device which owns `pCopyImageToMemoryInfo->srcImage`.

- pCopyImageToMemoryInfo is a pointer to a VkCopyImageToMemoryInfoEXT structure describing the copy parameters.

This command is functionally similar to vkCmdCopyImageToBuffer2KHR, except it is executed on the host and writes to host memory instead of a buffer. The memory of pCopyImageToMemoryInfo->srcImage is accessed by the host as if coherent.

NOTE If the device has written to the image memory, it is not automatically made available to the host. Before this copy command can be called, a memory barrier for this image **must** have been issued on the device with the second synchronization scope including VK_PIPELINE_STAGE_HOST_BIT and VK_ACCESS_HOST_READ_BIT.

Valid Usage (Implicit)

- VUID-vkCopyImageToMemoryEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCopyImageToMemoryEXT-pCopyImageToMemoryInfo-parameter
  pCopyImageToMemoryInfo **must** be a valid pointer to a valid VkCopyImageToMemoryInfoEXT structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_INITIALIZATION_FAILED
- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_MEMORY_MAP_FAILED

The VkCopyImageToMemoryInfoEXT structure is defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkCopyImageToMemoryInfoEXT {
    VkStructureType                 sType;
    const void*                     pNext;
    VkHostImageCopyFlagsEXT         flags;
    VkImage                         srcImage;
    VkImageLayout                   srcImageLayout;
    uint32_t                        regionCount;
    const VkImageToMemoryCopyEXT*   pRegions;
} VkCopyImageToMemoryInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkHostImageCopyFlagBitsEXT values describing additional copy parameters.
- `srcImage` is the source image.
- `srcImageLayout` is the layout of the source image subresources for the copy.
- `regionCount` is the number of regions to copy.
- `pRegions` is a pointer to an array of VkImageToMemoryCopyEXT structures specifying the regions to copy.

`vkCopyImageToMemoryEXT` does not check whether the device memory associated with `srcImage` is currently in use before performing the copy. The application **must** guarantee that any previously submitted command that writes to the copy regions has completed before the host performs the copy.

Copy regions for the image **must** be aligned to a multiple of the texel block extent in each dimension, except at the edges of the image, where region extents **must** match the edge of the image.

Valid Usage

- VUID-VkCopyImageToMemoryInfoEXT-srcImage-09109
  If `srcImage` is sparse then all memory ranges accessed by the copy command **must** be bound as described in Binding Resource Memory
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-09111
  `srcImage` **must** have been created with `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT` set in VkImageCreateInfo::`usage`
- VUID-VkCopyImageToMemoryInfoEXT-imageOffset-09114
  If `flags` contains `VK_HOST_IMAGE_COPY_MEMCPY_EXT`, the `x`, `y`, and `z` members of the `imageOffset` member of each element of `pRegions` **must** be `0`
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-09115
  If `flags` contains `VK_HOST_IMAGE_COPY_MEMCPY_EXT`, the `imageExtent` member of each element of `pRegions` **must** equal the extents of `srcImage` identified by `imageSubresource`
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07966
  If `srcImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkCopyImageToMemoryInfoEXT-imageSubresource-07967
  The `imageSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `srcImage` was created
- VUID-VkCopyImageToMemoryInfoEXT-imageSubresource-07968
  If `imageSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `imageSubresource.baseArrayLayer` + `imageSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo

when srcImage was created

- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07969
  srcImage **must** not have been created with flags containing VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT
- VUID-VkCopyImageToMemoryInfoEXT-imageSubresource-07970
  The image region specified by each element of pRegions **must** be contained within the specified imageSubresource of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-imageSubresource-07971
  For each element of pRegions, imageOffset.x and (imageExtent.width + imageOffset.x) **must** both be greater than or equal to 0 and less than or equal to the width of the specified imageSubresource of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-imageSubresource-07972
  For each element of pRegions, imageOffset.y and (imageExtent.height + imageOffset.y) **must** both be greater than or equal to 0 and less than or equal to the height of the specified imageSubresource of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07973
  srcImage **must** have a sample count equal to VK_SAMPLE_COUNT_1_BIT
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07979
  If srcImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, imageOffset.y **must** be 0 and imageExtent.height **must** be 1
- VUID-VkCopyImageToMemoryInfoEXT-imageOffset-09104
  For each element of pRegions, imageOffset.z and (imageExtent.depth + imageOffset.z) **must** both be greater than or equal to 0 and less than or equal to the depth of the specified imageSubresource of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07980
  If srcImage is of type VK_IMAGE_TYPE_1D or VK_IMAGE_TYPE_2D, then for each element of pRegions, imageOffset.z **must** be 0 and imageExtent.depth **must** be 1
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07274
  For each element of pRegions, imageOffset.x **must** be a multiple of the texel block extent width of the VkFormat of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07275
  For each element of pRegions, imageOffset.y **must** be a multiple of the texel block extent height of the VkFormat of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07276
  For each element of pRegions, imageOffset.z **must** be a multiple of the texel block extent depth of the VkFormat of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-00207
  For each element of pRegions, if the sum of imageOffset.x and extent.width does not equal the width of the subresource specified by imageSubresource, extent.width **must** be a multiple of the texel block extent width of the VkFormat of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-00208

For each element of pRegions, if the sum of imageOffset.y and extent.height does not equal the height of the subresource specified by imageSubresource, extent.height **must** be a multiple of the texel block extent height of the VkFormat of srcImage

- VUID-VkCopyImageToMemoryInfoEXT-srcImage-00209
  For each element of pRegions, if the sum of imageOffset.z and extent.depth does not equal the depth of the subresource specified by srcSubresource, extent.depth **must** be a multiple of the texel block extent depth of the VkFormat of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-imageSubresource-09105
  For each element of pRegions, imageSubresource.aspectMask **must** specify aspects present in srcImage
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07981
  If srcImage has a multi-planar image format, then for each element of pRegions, imageSubresource.aspectMask **must** be a single valid multi-planar aspect mask bit
- VUID-VkCopyImageToMemoryInfoEXT-srcImage-07983
  If srcImage is of type VK_IMAGE_TYPE_3D, for each element of pRegions, imageSubresource.baseArrayLayer **must** be 0 and imageSubresource.layerCount **must** be 1
- VUID-VkCopyImageToMemoryInfoEXT-memoryRowLength-09106
  For each element of pRegions, memoryRowLength **must** be a multiple of the texel block extent width of the VkFormat of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-memoryImageHeight-09107
  For each element of pRegions, memoryImageHeight **must** be a multiple of the texel block extent height of the VkFormat of srcImage
- VUID-VkCopyImageToMemoryInfoEXT-memoryRowLength-09108
  For each element of pRegions, memoryRowLength divided by the texel block extent width and then multiplied by the texel block size of srcImage **must** be less than or equal to 2^{31}-1
- VUID-VkCopyImageToMemoryInfoEXT-srcImageLayout-09064
  srcImageLayout **must** specify the current layout of the image subresources of srcImage specified in pRegions
- VUID-VkCopyImageToMemoryInfoEXT-srcImageLayout-09065
  srcImageLayout **must** be one of the image layouts returned in VkPhysicalDeviceHostImageCopyPropertiesEXT::pCopySrcLayouts
- VUID-VkCopyImageToMemoryInfoEXT-flags-09394
  If flags includes VK_HOST_IMAGE_COPY_MEMCPY_EXT, for each region in pRegions, memoryRowLength and memoryImageHeight **must** both be 0

Valid Usage (Implicit)

- VUID-VkCopyImageToMemoryInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_COPY_IMAGE_TO_MEMORY_INFO_EXT
- VUID-VkCopyImageToMemoryInfoEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkCopyImageToMemoryInfoEXT-flags-parameter

`flags` **must** be a valid combination of VkHostImageCopyFlagBitsEXT values

- VUID-VkCopyImageToMemoryInfoEXT-srcImage-parameter
  `srcImage` **must** be a valid VkImage handle
- VUID-VkCopyImageToMemoryInfoEXT-srcImageLayout-parameter
  `srcImageLayout` **must** be a valid VkImageLayout value
- VUID-VkCopyImageToMemoryInfoEXT-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkImageToMemoryCopyEXT structures
- VUID-VkCopyImageToMemoryInfoEXT-regionCount-arraylength
  `regionCount` **must** be greater than `0`

Each element of VkCopyImageToMemoryInfoEXT::`pRegions` is a structure defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkImageToMemoryCopyEXT {
    VkStructureType             sType;
    const void*                 pNext;
    void*                       pHostPointer;
    uint32_t                    memoryRowLength;
    uint32_t                    memoryImageHeight;
    VkImageSubresourceLayers    imageSubresource;
    VkOffset3D                  imageOffset;
    VkExtent3D                  imageExtent;
} VkImageToMemoryCopyEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pHostPointer` is the host memory address which is the destination of the copy.
- `memoryRowLength` and `memoryImageHeight` specify in texels a subregion of a larger two- or three-dimensional image in host memory, and control the addressing calculations. If either of these values is zero, that aspect of the host memory is considered to be tightly packed according to the `imageExtent`.
- `imageSubresource` is a VkImageSubresourceLayers used to specify the specific image subresources of the image used for the source or destination image data.
- `imageOffset` selects the initial `x`, `y`, `z` offsets in texels of the sub-region of the source image data.
- `imageExtent` is the size in texels of the image to copy in `width`, `height` and `depth`.

This structure is functionally similar to VkBufferImageCopy2, except it defines host memory as the target of copy instead of a buffer. In particular, the same data packing rules and restrictions as that structure apply here as well.

Valid Usage

- VUID-VkImageToMemoryCopyEXT-pHostPointer-09066
  `pHostPointer` **must** point to memory that is large enough to contain all memory locations that are accessed according to Buffer and Image Addressing, for each element of `pRegions`
- VUID-VkImageToMemoryCopyEXT-pRegions-09067
  The union of all source regions, and the union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-VkImageToMemoryCopyEXT-memoryRowLength-09101
  `memoryRowLength` **must** be `0`, or greater than or equal to the `width` member of `imageExtent`
- VUID-VkImageToMemoryCopyEXT-memoryImageHeight-09102
  `memoryImageHeight` **must** be `0`, or greater than or equal to the `height` member of `imageExtent`
- VUID-VkImageToMemoryCopyEXT-aspectMask-09103
  The `aspectMask` member of `imageSubresource` **must** only have a single bit set
- VUID-VkImageToMemoryCopyEXT-imageExtent-06659
  `imageExtent.width` **must** not be 0
- VUID-VkImageToMemoryCopyEXT-imageExtent-06660
  `imageExtent.height` **must** not be 0
- VUID-VkImageToMemoryCopyEXT-imageExtent-06661
  `imageExtent.depth` **must** not be 0

Valid Usage (Implicit)

- VUID-VkImageToMemoryCopyEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_TO_MEMORY_COPY_EXT`
- VUID-VkImageToMemoryCopyEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImageToMemoryCopyEXT-pHostPointer-parameter
  `pHostPointer` **must** be a pointer value
- VUID-VkImageToMemoryCopyEXT-imageSubresource-parameter
  `imageSubresource` **must** be a valid VkImageSubresourceLayers structure

Bits which **can** be set in VkCopyMemoryToImageInfoEXT::`flags`, VkCopyImageToMemoryInfoEXT::`flags`, and VkCopyImageToImageInfoEXT::`flags`, specifying additional copy parameters are:

```
// Provided by VK_EXT_host_image_copy
typedef enum VkHostImageCopyFlagBitsEXT {
    VK_HOST_IMAGE_COPY_MEMCPY_EXT = 0x00000001,
} VkHostImageCopyFlagBitsEXT;
```

- `VK_HOST_IMAGE_COPY_MEMCPY_EXT` specifies that no memory layout swizzling is to be applied during

data copy. For copies between memory and images, this flag indicates that image data in host memory is swizzled in exactly the same way as the image data on the device. Using this flag indicates that the implementations **may** use a simple memory copy to transfer the data between the host memory and the device memory. The format of the swizzled data in host memory is platform dependent and is not defined in this specification.

```
// Provided by VK_EXT_host_image_copy
typedef VkFlags VkHostImageCopyFlagsEXT;
```

VkHostImageCopyFlagsEXT is a bitmask type for setting a mask of zero or more VkHostImageCopyFlagBitsEXT.

To copy data from an image object to another image object using the host, call:

```
// Provided by VK_EXT_host_image_copy
VkResult vkCopyImageToImageEXT(
    VkDevice                                    device,
    const VkCopyImageToImageInfoEXT*            pCopyImageToImageInfo);
```

- device is the device which owns pCopyImageToImageInfo->srcImage and pCopyImageToImageInfo->dstImage.
- pCopyImageToImageInfo is a pointer to a VkCopyImageToImageInfoEXT structure describing the copy parameters.

This command is functionally similar to vkCmdCopyImage2KHR, except it is executed on the host. The memory of pCopyImageToImageInfo->srcImage and pCopyImageToImageInfo->dstImage is accessed by the host as if coherent.

NOTE

If the device has written to the memory of pCopyImageToImageInfo->srcImage, it is not automatically made available to the host. Before this copy command can be called, a memory barrier for this image **must** have been issued on the device with the second synchronization scope including VK_PIPELINE_STAGE_HOST_BIT and VK_ACCESS_HOST_READ_BIT.

Because queue submissions automatically make host memory visible to the device, there would not be a need for a memory barrier before using the results of this copy operation in pCopyMemoryToImageInfo->dstImage on the device.

Valid Usage (Implicit)

- VUID-vkCopyImageToImageEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCopyImageToImageEXT-pCopyImageToImageInfo-parameter
  pCopyImageToImageInfo **must** be a valid pointer to a valid VkCopyImageToImageInfoEXT structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_INITIALIZATION_FAILED
- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_MEMORY_MAP_FAILED

The VkCopyImageToImageInfoEXT structure is defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkCopyImageToImageInfoEXT {
    VkStructureType            sType;
    const void*                pNext;
    VkHostImageCopyFlagsEXT    flags;
    VkImage                    srcImage;
    VkImageLayout              srcImageLayout;
    VkImage                    dstImage;
    VkImageLayout              dstImageLayout;
    uint32_t                   regionCount;
    const VkImageCopy2*        pRegions;
} VkCopyImageToImageInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkHostImageCopyFlagBitsEXT values describing additional copy parameters.
- srcImage is the source image.
- srcImageLayout is the layout of the source image subresources for the copy.
- dstImage is the destination image.
- dstImageLayout is the layout of the destination image subresources for the copy.
- regionCount is the number of regions to copy.
- pRegions is a pointer to an array of VkImageCopy2 structures specifying the regions to copy.

vkCopyImageToImageEXT does not check whether the device memory associated with srcImage or dstImage is currently in use before performing the copy. The application **must** guarantee that any previously submitted command that writes to the copy regions has completed before the host performs the copy.

Valid Usage

- VUID-VkCopyImageToImageInfoEXT-srcImage-09069
  `srcImage` and `dstImage` **must** have been created with identical image creation parameters
- VUID-VkCopyImageToImageInfoEXT-srcImage-09109
  If `srcImage` is sparse then all memory ranges accessed by the copy command **must** be bound as described in Binding Resource Memory
- VUID-VkCopyImageToImageInfoEXT-srcImage-09111
  `srcImage` **must** have been created with `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT` set in VkImageCreateInfo::`usage`
- VUID-VkCopyImageToImageInfoEXT-srcOffset-09114
  If `flags` contains `VK_HOST_IMAGE_COPY_MEMCPY_EXT`, the `x`, `y`, and `z` members of the `srcOffset` member of each element of `pRegions` **must** be `0`
- VUID-VkCopyImageToImageInfoEXT-srcImage-09115
  If `flags` contains `VK_HOST_IMAGE_COPY_MEMCPY_EXT`, the `extent` member of each element of `pRegions` **must** equal the extents of `srcImage` identified by `srcSubresource`
- VUID-VkCopyImageToImageInfoEXT-srcImage-07966
  If `srcImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkCopyImageToImageInfoEXT-srcSubresource-07967
  The `srcSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `srcImage` was created
- VUID-VkCopyImageToImageInfoEXT-srcSubresource-07968
  If `srcSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `srcSubresource.baseArrayLayer` + `srcSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `srcImage` was created
- VUID-VkCopyImageToImageInfoEXT-srcImage-07969
  `srcImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- VUID-VkCopyImageToImageInfoEXT-srcSubresource-07970
  The image region specified by each element of `pRegions` **must** be contained within the specified `srcSubresource` of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcSubresource-07971
  For each element of `pRegions`, `srcOffset.x` and (`extent.width` + `srcOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `srcSubresource` of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcSubresource-07972
  For each element of `pRegions`, `srcOffset.y` and (`extent.height` + `srcOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `srcSubresource` of `srcImage`

- VUID-VkCopyImageToImageInfoEXT-srcImage-07979
  If `srcImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `srcOffset.y` **must** be `0` and `extent.height` **must** be `1`
- VUID-VkCopyImageToImageInfoEXT-srcOffset-09104
  For each element of `pRegions`, `srcOffset.z` and (`extent.depth` + `srcOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `srcSubresource` of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcImage-07980
  If `srcImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `srcOffset.z` **must** be `0` and `extent.depth` **must** be `1`
- VUID-VkCopyImageToImageInfoEXT-srcImage-07274
  For each element of `pRegions`, `srcOffset.x` **must** be a multiple of the texel block extent width of the VkFormat of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcImage-07275
  For each element of `pRegions`, `srcOffset.y` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcImage-07276
  For each element of `pRegions`, `srcOffset.z` **must** be a multiple of the texel block extent depth of the VkFormat of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcImage-00207
  For each element of `pRegions`, if the sum of `srcOffset.x` and `extent.width` does not equal the width of the subresource specified by `srcSubresource`, `extent.width` **must** be a multiple of the texel block extent width of the VkFormat of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcImage-00208
  For each element of `pRegions`, if the sum of `srcOffset.y` and `extent.height` does not equal the height of the subresource specified by `srcSubresource`, `extent.height` **must** be a multiple of the texel block extent height of the VkFormat of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcImage-00209
  For each element of `pRegions`, if the sum of `srcOffset.z` and `extent.depth` does not equal the depth of the subresource specified by `srcSubresource`, `extent.depth` **must** be a multiple of the texel block extent depth of the VkFormat of `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcSubresource-09105
  For each element of `pRegions`, `srcSubresource.aspectMask` **must** specify aspects present in `srcImage`
- VUID-VkCopyImageToImageInfoEXT-srcImage-07981
  If `srcImage` has a multi-planar image format, then for each element of `pRegions`, `srcSubresource.aspectMask` **must** be a single valid multi-planar aspect mask bit
- VUID-VkCopyImageToImageInfoEXT-srcImage-07983
  If `srcImage` is of type `VK_IMAGE_TYPE_3D`, for each element of `pRegions`, `srcSubresource.baseArrayLayer` **must** be `0` and `srcSubresource.layerCount` **must** be `1`
- VUID-VkCopyImageToImageInfoEXT-dstImage-09109
  If `dstImage` is sparse then all memory ranges accessed by the copy command **must** be bound as described in Binding Resource Memory

- VUID-VkCopyImageToImageInfoEXT-dstImage-09111
  `dstImage` **must** have been created with `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT` set in VkImageCreateInfo::`usage`
- VUID-VkCopyImageToImageInfoEXT-dstOffset-09114
  If `flags` contains `VK_HOST_IMAGE_COPY_MEMCPY_EXT`, the `x`, `y`, and `z` members of the `dstOffset` member of each element of `pRegions` **must** be `0`
- VUID-VkCopyImageToImageInfoEXT-dstImage-09115
  If `flags` contains `VK_HOST_IMAGE_COPY_MEMCPY_EXT`, the `extent` member of each element of `pRegions` **must** equal the extents of `dstImage` identified by `dstSubresource`
- VUID-VkCopyImageToImageInfoEXT-dstImage-07966
  If `dstImage` is non-sparse then the image or the specified *disjoint* plane **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkCopyImageToImageInfoEXT-dstSubresource-07967
  The `dstSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `dstImage` was created
- VUID-VkCopyImageToImageInfoEXT-dstSubresource-07968
  If `dstSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `dstSubresource.baseArrayLayer` + `dstSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `dstImage` was created
- VUID-VkCopyImageToImageInfoEXT-dstImage-07969
  `dstImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- VUID-VkCopyImageToImageInfoEXT-dstSubresource-07970
  The image region specified by each element of `pRegions` **must** be contained within the specified `dstSubresource` of `dstImage`
- VUID-VkCopyImageToImageInfoEXT-dstSubresource-07971
  For each element of `pRegions`, `dstOffset.x` and (`extent.width` + `dstOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `dstSubresource` of `dstImage`
- VUID-VkCopyImageToImageInfoEXT-dstSubresource-07972
  For each element of `pRegions`, `dstOffset.y` and (`extent.height` + `dstOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `dstSubresource` of `dstImage`
- VUID-VkCopyImageToImageInfoEXT-dstImage-07979
  If `dstImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `dstOffset.y` **must** be `0` and `extent.height` **must** be `1`
- VUID-VkCopyImageToImageInfoEXT-dstOffset-09104
  For each element of `pRegions`, `dstOffset.z` and (`extent.depth` + `dstOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `dstSubresource` of `dstImage`
- VUID-VkCopyImageToImageInfoEXT-dstImage-07980

If dstImage is of type VK_IMAGE_TYPE_1D or VK_IMAGE_TYPE_2D, then for each element of pRegions, dstOffset.z **must** be 0 and extent.depth **must** be 1

- VUID-VkCopyImageToImageInfoEXT-dstImage-07274
  For each element of pRegions, dstOffset.x **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-VkCopyImageToImageInfoEXT-dstImage-07275
  For each element of pRegions, dstOffset.y **must** be a multiple of the texel block extent height of the VkFormat of dstImage
- VUID-VkCopyImageToImageInfoEXT-dstImage-07276
  For each element of pRegions, dstOffset.z **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-VkCopyImageToImageInfoEXT-dstImage-00207
  For each element of pRegions, if the sum of dstOffset.x and extent.width does not equal the width of the subresource specified by dstSubresource, extent.width **must** be a multiple of the texel block extent width of the VkFormat of dstImage
- VUID-VkCopyImageToImageInfoEXT-dstImage-00208
  For each element of pRegions, if the sum of dstOffset.y and extent.height does not equal the height of the subresource specified by dstSubresource, extent.height **must** be a multiple of the texel block extent height of the VkFormat of dstImage
- VUID-VkCopyImageToImageInfoEXT-dstImage-00209
  For each element of pRegions, if the sum of dstOffset.z and extent.depth does not equal the depth of the subresource specified by srcSubresource, extent.depth **must** be a multiple of the texel block extent depth of the VkFormat of dstImage
- VUID-VkCopyImageToImageInfoEXT-dstSubresource-09105
  For each element of pRegions, dstSubresource.aspectMask **must** specify aspects present in dstImage
- VUID-VkCopyImageToImageInfoEXT-dstImage-07981
  If dstImage has a multi-planar image format, then for each element of pRegions, dstSubresource.aspectMask **must** be a single valid multi-planar aspect mask bit
- VUID-VkCopyImageToImageInfoEXT-dstImage-07983
  If dstImage is of type VK_IMAGE_TYPE_3D, for each element of pRegions, dstSubresource.baseArrayLayer **must** be 0 and dstSubresource.layerCount **must** be 1
- VUID-VkCopyImageToImageInfoEXT-srcImageLayout-09070
  srcImageLayout **must** specify the current layout of the image subresources of srcImage specified in pRegions
- VUID-VkCopyImageToImageInfoEXT-dstImageLayout-09071
  dstImageLayout **must** specify the current layout of the image subresources of dstImage specified in pRegions
- VUID-VkCopyImageToImageInfoEXT-srcImageLayout-09072
  srcImageLayout **must** be one of the image layouts returned in VkPhysicalDeviceHostImageCopyPropertiesEXT::pCopySrcLayouts
- VUID-VkCopyImageToImageInfoEXT-dstImageLayout-09073
  dstImageLayout **must** be one of the image layouts returned in

VkPhysicalDeviceHostImageCopyPropertiesEXT::pCopyDstLayouts

Valid Usage (Implicit)

- VUID-VkCopyImageToImageInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_COPY_IMAGE_TO_IMAGE_INFO_EXT
- VUID-VkCopyImageToImageInfoEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkCopyImageToImageInfoEXT-flags-parameter
  flags **must** be a valid combination of VkHostImageCopyFlagBitsEXT values
- VUID-VkCopyImageToImageInfoEXT-srcImage-parameter
  srcImage **must** be a valid VkImage handle
- VUID-VkCopyImageToImageInfoEXT-srcImageLayout-parameter
  srcImageLayout **must** be a valid VkImageLayout value
- VUID-VkCopyImageToImageInfoEXT-dstImage-parameter
  dstImage **must** be a valid VkImage handle
- VUID-VkCopyImageToImageInfoEXT-dstImageLayout-parameter
  dstImageLayout **must** be a valid VkImageLayout value
- VUID-VkCopyImageToImageInfoEXT-pRegions-parameter
  pRegions **must** be a valid pointer to an array of regionCount valid VkImageCopy2 structures
- VUID-VkCopyImageToImageInfoEXT-regionCount-arraylength
  regionCount **must** be greater than 0
- VUID-VkCopyImageToImageInfoEXT-commonparent
  Both of dstImage, and srcImage **must** have been created, allocated, or retrieved from the same VkDevice

20.4. Image Copies With Scaling

To copy regions of a source image into a destination image, potentially performing format conversion, arbitrary scaling, and filtering, call:

```
// Provided by VK_VERSION_1_0
void vkCmdBlitImage(
    VkCommandBuffer                             commandBuffer,
    VkImage                                     srcImage,
    VkImageLayout                               srcImageLayout,
    VkImage                                     dstImage,
    VkImageLayout                               dstImageLayout,
    uint32_t                                    regionCount,
    const VkImageBlit*                          pRegions,
    VkFilter                                    filter);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `srcImage` is the source image.
- `srcImageLayout` is the layout of the source image subresources for the blit.
- `dstImage` is the destination image.
- `dstImageLayout` is the layout of the destination image subresources for the blit.
- `regionCount` is the number of regions to blit.
- `pRegions` is a pointer to an array of VkImageBlit structures specifying the regions to blit.
- `filter` is a VkFilter specifying the filter to apply if the blits require scaling.

`vkCmdBlitImage` **must** not be used for multisampled source or destination images. Use vkCmdResolveImage for this purpose.

As the sizes of the source and destination extents **can** differ in any dimension, texels in the source extent are scaled and filtered to the destination extent. Scaling occurs via the following operations:

- For each destination texel, the integer coordinate of that texel is converted to an unnormalized texture coordinate, using the effective inverse of the equations described in unnormalized to integer conversion:

$$u_{base} = i + \frac{1}{2}$$

$$v_{base} = j + \frac{1}{2}$$

$$w_{base} = k + \frac{1}{2}$$

- These base coordinates are then offset by the first destination offset:

$$u_{offset} = u_{base} - x_{dst0}$$

$$v_{offset} = v_{base} - y_{dst0}$$

$$w_{offset} = w_{base} - z_{dst0}$$

$$a_{offset} = a - \texttt{baseArrayCount}_{dst}$$

- The scale is determined from the source and destination regions, and applied to the offset coordinates:

$$scale_u = (x_{src1} - x_{src0}) / (x_{dst1} - x_{dst0})$$

$$scale_v = (y_{src1} - y_{src0}) / (y_{dst1} - y_{dst0})$$

$$scale_w = (z_{src1} - z_{src0}) / (z_{dst1} - z_{dst0})$$

$$u_{scaled} = u_{offset} \times scale_u$$

$$v_{scaled} = v_{offset} \times scale_v$$

$$w_{scaled} = w_{offset} \times scale_w$$

- Finally the source offset is added to the scaled coordinates, to determine the final unnormalized coordinates used to sample from `srcImage`:

$$u = u_{scaled} + x_{src0}$$

$$v = v_{scaled} + y_{src0}$$

$$w = w_{scaled} + z_{src0}$$

$$q = \texttt{mipLevel}$$

$$a = a_{offset} + \texttt{baseArrayCount}_{src}$$

These coordinates are used to sample from the source image, as described in Image Operations chapter, with the filter mode equal to that of `filter`, a mipmap mode of `VK_SAMPLER_MIPMAP_MODE_NEAREST` and an address mode of `VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE`. Implementations **must** clamp at the edge of the source image, and **may** additionally clamp to the

edge of the source region.

NOTE	Due to allowable rounding errors in the generation of the source texture coordinates, it is not always possible to guarantee exactly which source texels will be sampled for a given blit. As rounding errors are implementation-dependent, the exact results of a blitting operation are also implementation-dependent.

Blits are done layer by layer starting with the `baseArrayLayer` member of `srcSubresource` for the source and `dstSubresource` for the destination. `layerCount` layers are blitted to the destination image.

When blitting 3D textures, slices in the destination region bounded by `dstOffsets`[0].z and `dstOffsets`[1].z are sampled from slices in the source region bounded by `srcOffsets`[0].z and `srcOffsets`[1].z. If the `filter` parameter is `VK_FILTER_LINEAR` then the value sampled from the source image is taken by doing linear filtering using the interpolated **z** coordinate represented by **w** in the previous equations. If the `filter` parameter is `VK_FILTER_NEAREST` then the value sampled from the source image is taken from the single nearest slice, with an implementation-dependent arithmetic rounding mode.

The following filtering and conversion rules apply:

- Integer formats **can** only be converted to other integer formats with the same signedness.
- No format conversion is supported between depth/stencil images. The formats **must** match.
- Format conversions on unorm, snorm, scaled and packed float formats of the copied aspect of the image are performed by first converting the pixels to float values.
- For sRGB source formats, nonlinear RGB values are converted to linear representation prior to filtering.
- After filtering, the float values are first clamped and then cast to the destination image format. In case of sRGB destination format, linear RGB values are converted to nonlinear representation before writing the pixel to the image.

Signed and unsigned integers are converted by first clamping to the representable range of the destination format, then casting the value.

Valid Usage

- VUID-vkCmdBlitImage-pRegions-00215
  The source region specified by each element of `pRegions` **must** be a region that is contained within `srcImage`
- VUID-vkCmdBlitImage-pRegions-00216
  The destination region specified by each element of `pRegions` **must** be a region that is contained within `dstImage`
- VUID-vkCmdBlitImage-pRegions-00217
  The union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory with any texel that **may** be sampled during the blit operation
- VUID-vkCmdBlitImage-srcImage-01999
  The format features of `srcImage` **must** contain `VK_FORMAT_FEATURE_BLIT_SRC_BIT`

- VUID-vkCmdBlitImage-srcImage-06421
  `srcImage` **must** not use a format that requires a sampler $Y'C_BC_R$ conversion
- VUID-vkCmdBlitImage-srcImage-00219
  `srcImage` **must** have been created with `VK_IMAGE_USAGE_TRANSFER_SRC_BIT` usage flag
- VUID-vkCmdBlitImage-srcImage-00220
  If `srcImage` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdBlitImage-srcImageLayout-00221
  `srcImageLayout` **must** specify the layout of the image subresources of `srcImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-vkCmdBlitImage-srcImageLayout-01398
  `srcImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL` or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-vkCmdBlitImage-srcImage-09459
  If `srcImage` and `dstImage` are the same, and an elements of `pRegions` contains the `srcSubresource` and `dstSubresource` with matching `mipLevel` and overlapping array layers, then the `srcImageLayout` and `dstImageLayout` **must** be `VK_IMAGE_LAYOUT_GENERAL` or `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`
- VUID-vkCmdBlitImage-dstImage-02000
  The format features of `dstImage` **must** contain `VK_FORMAT_FEATURE_BLIT_DST_BIT`
- VUID-vkCmdBlitImage-dstImage-06422
  `dstImage` **must** not use a format that requires a sampler $Y'C_BC_R$ conversion
- VUID-vkCmdBlitImage-dstImage-00224
  `dstImage` **must** have been created with `VK_IMAGE_USAGE_TRANSFER_DST_BIT` usage flag
- VUID-vkCmdBlitImage-dstImage-00225
  If `dstImage` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdBlitImage-dstImageLayout-00226
  `dstImageLayout` **must** specify the layout of the image subresources of `dstImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-vkCmdBlitImage-dstImageLayout-01399
  `dstImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-vkCmdBlitImage-srcImage-00229
  If either of `srcImage` or `dstImage` was created with a signed integer VkFormat, the other **must** also have been created with a signed integer VkFormat
- VUID-vkCmdBlitImage-srcImage-00230
  If either of `srcImage` or `dstImage` was created with an unsigned integer VkFormat, the other **must** also have been created with an unsigned integer VkFormat
- VUID-vkCmdBlitImage-srcImage-00231
  If either of `srcImage` or `dstImage` was created with a depth/stencil format, the other **must** have exactly the same format

- VUID-vkCmdBlitImage-srcImage-00232
  If `srcImage` was created with a depth/stencil format, `filter` **must** be `VK_FILTER_NEAREST`
- VUID-vkCmdBlitImage-srcImage-00233
  `srcImage` **must** have been created with a `samples` value of `VK_SAMPLE_COUNT_1_BIT`
- VUID-vkCmdBlitImage-dstImage-00234
  `dstImage` **must** have been created with a `samples` value of `VK_SAMPLE_COUNT_1_BIT`
- VUID-vkCmdBlitImage-filter-02001
  If `filter` is `VK_FILTER_LINEAR`, then the format features of `srcImage` **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdBlitImage-filter-02002
  If `filter` is `VK_FILTER_CUBIC_EXT`, then the format features of `srcImage` **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT`
- VUID-vkCmdBlitImage-filter-00237
  If `filter` is `VK_FILTER_CUBIC_EXT`, `srcImage` **must** be of type `VK_IMAGE_TYPE_2D`
- VUID-vkCmdBlitImage-srcSubresource-01705
  The `srcSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `srcImage` was created
- VUID-vkCmdBlitImage-dstSubresource-01706
  The `dstSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `dstImage` was created
- VUID-vkCmdBlitImage-srcSubresource-01707
  If `srcSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `srcSubresource.baseArrayLayer` + `srcSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `srcImage` was created
- VUID-vkCmdBlitImage-dstSubresource-01708
  If `srcSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `dstSubresource.baseArrayLayer` + `dstSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `dstImage` was created
- VUID-vkCmdBlitImage-dstImage-02545
  `dstImage` and `srcImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- VUID-vkCmdBlitImage-srcImage-00240
  If either `srcImage` or `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `srcSubresource.baseArrayLayer` and `dstSubresource.baseArrayLayer` **must** each be `0`, and `srcSubresource.layerCount` and `dstSubresource.layerCount` **must** each be `1`
- VUID-vkCmdBlitImage-aspectMask-00241
  For each element of `pRegions`, `srcSubresource.aspectMask` **must** specify aspects present in `srcImage`
- VUID-vkCmdBlitImage-aspectMask-00242
  For each element of `pRegions`, `dstSubresource.aspectMask` **must** specify aspects present in `dstImage`

- VUID-vkCmdBlitImage-srcOffset-00243
  For each element of `pRegions`, `srcOffsets`[0].x and `srcOffsets`[1].x **must** both be greater than or equal to `0` and less than or equal to the width of the specified `srcSubresource` of `srcImage`
- VUID-vkCmdBlitImage-srcOffset-00244
  For each element of `pRegions`, `srcOffsets`[0].y and `srcOffsets`[1].y **must** both be greater than or equal to `0` and less than or equal to the height of the specified `srcSubresource` of `srcImage`
- VUID-vkCmdBlitImage-srcImage-00245
  If `srcImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `srcOffsets`[0].y **must** be `0` and `srcOffsets`[1].y **must** be `1`
- VUID-vkCmdBlitImage-srcOffset-00246
  For each element of `pRegions`, `srcOffsets`[0].z and `srcOffsets`[1].z **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `srcSubresource` of `srcImage`
- VUID-vkCmdBlitImage-srcImage-00247
  If `srcImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `srcOffsets`[0].z **must** be `0` and `srcOffsets`[1].z **must** be `1`
- VUID-vkCmdBlitImage-dstOffset-00248
  For each element of `pRegions`, `dstOffsets`[0].x and `dstOffsets`[1].x **must** both be greater than or equal to `0` and less than or equal to the width of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdBlitImage-dstOffset-00249
  For each element of `pRegions`, `dstOffsets`[0].y and `dstOffsets`[1].y **must** both be greater than or equal to `0` and less than or equal to the height of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdBlitImage-dstImage-00250
  If `dstImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `dstOffsets`[0].y **must** be `0` and `dstOffsets`[1].y **must** be `1`
- VUID-vkCmdBlitImage-dstOffset-00251
  For each element of `pRegions`, `dstOffsets`[0].z and `dstOffsets`[1].z **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdBlitImage-dstImage-00252
  If `dstImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `dstOffsets`[0].z **must** be `0` and `dstOffsets`[1].z **must** be `1`

Valid Usage (Implicit)

- VUID-vkCmdBlitImage-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBlitImage-srcImage-parameter
  `srcImage` **must** be a valid VkImage handle

- VUID-vkCmdBlitImage-srcImageLayout-parameter
  `srcImageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdBlitImage-dstImage-parameter
  `dstImage` **must** be a valid VkImage handle
- VUID-vkCmdBlitImage-dstImageLayout-parameter
  `dstImageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdBlitImage-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkImageBlit structures
- VUID-vkCmdBlitImage-filter-parameter
  `filter` **must** be a valid VkFilter value
- VUID-vkCmdBlitImage-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBlitImage-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBlitImage-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBlitImage-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBlitImage-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-vkCmdBlitImage-commonparent
  Each of `commandBuffer`, `dstImage`, and `srcImage` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics	Action

The `VkImageBlit` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageBlit {
    VkImageSubresourceLayers    srcSubresource;
    VkOffset3D                  srcOffsets[2];
    VkImageSubresourceLayers    dstSubresource;
    VkOffset3D                  dstOffsets[2];
} VkImageBlit;
```

- `srcSubresource` is the subresource to blit from.
- `srcOffsets` is a pointer to an array of two VkOffset3D structures specifying the bounds of the source region within `srcSubresource`.
- `dstSubresource` is the subresource to blit into.
- `dstOffsets` is a pointer to an array of two VkOffset3D structures specifying the bounds of the destination region within `dstSubresource`.

For each element of the `pRegions` array, a blit operation is performed for the specified source and destination regions.

Valid Usage

- VUID-VkImageBlit-aspectMask-00238
  The `aspectMask` member of `srcSubresource` and `dstSubresource` **must** match
- VUID-VkImageBlit-layerCount-08800
  If neither of the `layerCount` members of `srcSubresource` or `dstSubresource` are `VK_REMAINING_ARRAY_LAYERS`, the `layerCount` members of `srcSubresource` or `dstSubresource` **must** match
- VUID-VkImageBlit-layerCount-08801
  If one of the `layerCount` members of `srcSubresource` or `dstSubresource` is `VK_REMAINING_ARRAY_LAYERS`, the other member **must** be either `VK_REMAINING_ARRAY_LAYERS` or equal to the `arrayLayers` member of the VkImageCreateInfo used to create the image minus `baseArrayLayer`

Valid Usage (Implicit)

- VUID-VkImageBlit-srcSubresource-parameter
  `srcSubresource` **must** be a valid VkImageSubresourceLayers structure
- VUID-VkImageBlit-dstSubresource-parameter
  `dstSubresource` **must** be a valid VkImageSubresourceLayers structure

A more extensible version of the blit image command is defined below.

To copy regions of a source image into a destination image, potentially performing format conversion, arbitrary scaling, and filtering, call:

```
// Provided by VK_KHR_copy_commands2
void vkCmdBlitImage2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkBlitImageInfo2*                     pBlitImageInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pBlitImageInfo` is a pointer to a VkBlitImageInfo2 structure describing the blit parameters.

This command is functionally identical to vkCmdBlitImage, but includes extensible sub-structures that include `sType` and `pNext` parameters, allowing them to be more easily extended.

Valid Usage

Valid Usage (Implicit)

- VUID-vkCmdBlitImage2-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBlitImage2-pBlitImageInfo-parameter
  `pBlitImageInfo` **must** be a valid pointer to a valid VkBlitImageInfo2 structure
- VUID-vkCmdBlitImage2-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBlitImage2-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBlitImage2-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBlitImage2-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics	Action

The `VkBlitImageInfo2` structure is defined as:

```
typedef struct VkBlitImageInfo2 {
    VkStructureType        sType;
    const void*            pNext;
    VkImage                srcImage;
    VkImageLayout          srcImageLayout;
    VkImage                dstImage;
    VkImageLayout          dstImageLayout;
    uint32_t               regionCount;
    const VkImageBlit2*    pRegions;
    VkFilter               filter;
} VkBlitImageInfo2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkBlitImageInfo2 VkBlitImageInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcImage` is the source image.
- `srcImageLayout` is the layout of the source image subresources for the blit.
- `dstImage` is the destination image.
- `dstImageLayout` is the layout of the destination image subresources for the blit.
- `regionCount` is the number of regions to blit.
- `pRegions` is a pointer to an array of VkImageBlit2 structures specifying the regions to blit.
- `filter` is a VkFilter specifying the filter to apply if the blits require scaling.

Valid Usage

- VUID-VkBlitImageInfo2-pRegions-00215
  The source region specified by each element of `pRegions` **must** be a region that is contained within `srcImage`

- VUID-VkBlitImageInfo2-pRegions-00216
  The destination region specified by each element of pRegions **must** be a region that is contained within dstImage
- VUID-VkBlitImageInfo2-pRegions-00217
  The union of all destination regions, specified by the elements of pRegions, **must** not overlap in memory with any texel that **may** be sampled during the blit operation
- VUID-VkBlitImageInfo2-srcImage-01999
  The format features of srcImage **must** contain VK_FORMAT_FEATURE_BLIT_SRC_BIT
- VUID-VkBlitImageInfo2-srcImage-06421
  srcImage **must** not use a format that requires a sampler $Y'C_BC_R$ conversion
- VUID-VkBlitImageInfo2-srcImage-00219
  srcImage **must** have been created with VK_IMAGE_USAGE_TRANSFER_SRC_BIT usage flag
- VUID-VkBlitImageInfo2-srcImage-00220
  If srcImage is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkBlitImageInfo2-srcImageLayout-00221
  srcImageLayout **must** specify the layout of the image subresources of srcImage specified in pRegions at the time this command is executed on a VkDevice
- VUID-VkBlitImageInfo2-srcImageLayout-01398
  srcImageLayout **must** be VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL or VK_IMAGE_LAYOUT_GENERAL
- VUID-VkBlitImageInfo2-srcImage-09459
  If srcImage and dstImage are the same, and an elements of pRegions contains the srcSubresource and dstSubresource with matching mipLevel and overlapping array layers, then the srcImageLayout and dstImageLayout **must** be VK_IMAGE_LAYOUT_GENERAL or VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR
- VUID-VkBlitImageInfo2-dstImage-02000
  The format features of dstImage **must** contain VK_FORMAT_FEATURE_BLIT_DST_BIT
- VUID-VkBlitImageInfo2-dstImage-06422
  dstImage **must** not use a format that requires a sampler $Y'C_BC_R$ conversion
- VUID-VkBlitImageInfo2-dstImage-00224
  dstImage **must** have been created with VK_IMAGE_USAGE_TRANSFER_DST_BIT usage flag
- VUID-VkBlitImageInfo2-dstImage-00225
  If dstImage is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-VkBlitImageInfo2-dstImageLayout-00226
  dstImageLayout **must** specify the layout of the image subresources of dstImage specified in pRegions at the time this command is executed on a VkDevice
- VUID-VkBlitImageInfo2-dstImageLayout-01399
  dstImageLayout **must** be VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL or VK_IMAGE_LAYOUT_GENERAL
- VUID-VkBlitImageInfo2-srcImage-00229

If either of srcImage or dstImage was created with a signed integer VkFormat, the other **must** also have been created with a signed integer VkFormat

- VUID-VkBlitImageInfo2-srcImage-00230
  If either of srcImage or dstImage was created with an unsigned integer VkFormat, the other **must** also have been created with an unsigned integer VkFormat
- VUID-VkBlitImageInfo2-srcImage-00231
  If either of srcImage or dstImage was created with a depth/stencil format, the other **must** have exactly the same format
- VUID-VkBlitImageInfo2-srcImage-00232
  If srcImage was created with a depth/stencil format, filter **must** be VK_FILTER_NEAREST
- VUID-VkBlitImageInfo2-srcImage-00233
  srcImage **must** have been created with a samples value of VK_SAMPLE_COUNT_1_BIT
- VUID-VkBlitImageInfo2-dstImage-00234
  dstImage **must** have been created with a samples value of VK_SAMPLE_COUNT_1_BIT
- VUID-VkBlitImageInfo2-filter-02001
  If filter is VK_FILTER_LINEAR, then the format features of srcImage **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
- VUID-VkBlitImageInfo2-filter-02002
  If filter is VK_FILTER_CUBIC_EXT, then the format features of srcImage **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-VkBlitImageInfo2-filter-00237
  If filter is VK_FILTER_CUBIC_EXT, srcImage **must** be of type VK_IMAGE_TYPE_2D
- VUID-VkBlitImageInfo2-srcSubresource-01705
  The srcSubresource.mipLevel member of each element of pRegions **must** be less than the mipLevels specified in VkImageCreateInfo when srcImage was created
- VUID-VkBlitImageInfo2-dstSubresource-01706
  The dstSubresource.mipLevel member of each element of pRegions **must** be less than the mipLevels specified in VkImageCreateInfo when dstImage was created
- VUID-VkBlitImageInfo2-srcSubresource-01707
  If srcSubresource.layerCount is not VK_REMAINING_ARRAY_LAYERS, srcSubresource.baseArrayLayer + srcSubresource.layerCount of each element of pRegions **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when srcImage was created
- VUID-VkBlitImageInfo2-dstSubresource-01708
  If srcSubresource.layerCount is not VK_REMAINING_ARRAY_LAYERS, dstSubresource.baseArrayLayer + dstSubresource.layerCount of each element of pRegions **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when dstImage was created
- VUID-VkBlitImageInfo2-dstImage-02545
  dstImage and srcImage **must** not have been created with flags containing VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT
- VUID-VkBlitImageInfo2-srcImage-00240

If either `srcImage` or `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `srcSubresource.baseArrayLayer` and `dstSubresource.baseArrayLayer` **must** each be `0`, and `srcSubresource.layerCount` and `dstSubresource.layerCount` **must** each be `1`

- VUID-VkBlitImageInfo2-aspectMask-00241
  For each element of `pRegions`, `srcSubresource.aspectMask` **must** specify aspects present in `srcImage`
- VUID-VkBlitImageInfo2-aspectMask-00242
  For each element of `pRegions`, `dstSubresource.aspectMask` **must** specify aspects present in `dstImage`
- VUID-VkBlitImageInfo2-srcOffset-00243
  For each element of `pRegions`, `srcOffsets`[0].x and `srcOffsets`[1].x **must** both be greater than or equal to `0` and less than or equal to the width of the specified `srcSubresource` of `srcImage`
- VUID-VkBlitImageInfo2-srcOffset-00244
  For each element of `pRegions`, `srcOffsets`[0].y and `srcOffsets`[1].y **must** both be greater than or equal to `0` and less than or equal to the height of the specified `srcSubresource` of `srcImage`
- VUID-VkBlitImageInfo2-srcImage-00245
  If `srcImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `srcOffsets`[0].y **must** be `0` and `srcOffsets`[1].y **must** be `1`
- VUID-VkBlitImageInfo2-srcOffset-00246
  For each element of `pRegions`, `srcOffsets`[0].z and `srcOffsets`[1].z **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `srcSubresource` of `srcImage`
- VUID-VkBlitImageInfo2-srcImage-00247
  If `srcImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `srcOffsets`[0].z **must** be `0` and `srcOffsets`[1].z **must** be `1`
- VUID-VkBlitImageInfo2-dstOffset-00248
  For each element of `pRegions`, `dstOffsets`[0].x and `dstOffsets`[1].x **must** both be greater than or equal to `0` and less than or equal to the width of the specified `dstSubresource` of `dstImage`
- VUID-VkBlitImageInfo2-dstOffset-00249
  For each element of `pRegions`, `dstOffsets`[0].y and `dstOffsets`[1].y **must** both be greater than or equal to `0` and less than or equal to the height of the specified `dstSubresource` of `dstImage`
- VUID-VkBlitImageInfo2-dstImage-00250
  If `dstImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `dstOffsets`[0].y **must** be `0` and `dstOffsets`[1].y **must** be `1`
- VUID-VkBlitImageInfo2-dstOffset-00251
  For each element of `pRegions`, `dstOffsets`[0].z and `dstOffsets`[1].z **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `dstSubresource` of `dstImage`
- VUID-VkBlitImageInfo2-dstImage-00252

If `dstImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `dstOffsets`[0].z **must** be `0` and `dstOffsets`[1].z **must** be `1`

Valid Usage (Implicit)

- VUID-VkBlitImageInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BLIT_IMAGE_INFO_2`
- VUID-VkBlitImageInfo2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkBlitImageInfo2-srcImage-parameter
  `srcImage` **must** be a valid VkImage handle
- VUID-VkBlitImageInfo2-srcImageLayout-parameter
  `srcImageLayout` **must** be a valid VkImageLayout value
- VUID-VkBlitImageInfo2-dstImage-parameter
  `dstImage` **must** be a valid VkImage handle
- VUID-VkBlitImageInfo2-dstImageLayout-parameter
  `dstImageLayout` **must** be a valid VkImageLayout value
- VUID-VkBlitImageInfo2-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkImageBlit2 structures
- VUID-VkBlitImageInfo2-filter-parameter
  `filter` **must** be a valid VkFilter value
- VUID-VkBlitImageInfo2-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-VkBlitImageInfo2-commonparent
  Both of `dstImage`, and `srcImage` **must** have been created, allocated, or retrieved from the same VkDevice

The `VkImageBlit2` structure is defined as:

```
typedef struct VkImageBlit2 {
    VkStructureType             sType;
    const void*                 pNext;
    VkImageSubresourceLayers    srcSubresource;
    VkOffset3D                  srcOffsets[2];
    VkImageSubresourceLayers    dstSubresource;
    VkOffset3D                  dstOffsets[2];
} VkImageBlit2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkImageBlit2 VkImageBlit2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcSubresource` is the subresource to blit from.
- `srcOffsets` is a pointer to an array of two VkOffset3D structures specifying the bounds of the source region within `srcSubresource`.
- `dstSubresource` is the subresource to blit into.
- `dstOffsets` is a pointer to an array of two VkOffset3D structures specifying the bounds of the destination region within `dstSubresource`.

For each element of the `pRegions` array, a blit operation is performed for the specified source and destination regions.

Valid Usage

- VUID-VkImageBlit2-aspectMask-00238
  The `aspectMask` member of `srcSubresource` and `dstSubresource` **must** match
- VUID-VkImageBlit2-layerCount-08800
  If neither of the `layerCount` members of `srcSubresource` or `dstSubresource` are `VK_REMAINING_ARRAY_LAYERS`, the `layerCount` members of `srcSubresource` or `dstSubresource` **must** match
- VUID-VkImageBlit2-layerCount-08801
  If one of the `layerCount` members of `srcSubresource` or `dstSubresource` is `VK_REMAINING_ARRAY_LAYERS`, the other member **must** be either `VK_REMAINING_ARRAY_LAYERS` or equal to the `arrayLayers` member of the VkImageCreateInfo used to create the image minus `baseArrayLayer`

Valid Usage (Implicit)

- VUID-VkImageBlit2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_BLIT_2`
- VUID-VkImageBlit2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImageBlit2-srcSubresource-parameter
  `srcSubresource` **must** be a valid VkImageSubresourceLayers structure
- VUID-VkImageBlit2-dstSubresource-parameter
  `dstSubresource` **must** be a valid VkImageSubresourceLayers structure

20.5. Resolving Multisample Images

To resolve a multisample color image to a non-multisample color image, call:

```
// Provided by VK_VERSION_1_0
void vkCmdResolveImage(
    VkCommandBuffer                             commandBuffer,
    VkImage                                     srcImage,
    VkImageLayout                               srcImageLayout,
    VkImage                                     dstImage,
    VkImageLayout                               dstImageLayout,
    uint32_t                                    regionCount,
    const VkImageResolve*                       pRegions);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `srcImage` is the source image.
- `srcImageLayout` is the layout of the source image subresources for the resolve.
- `dstImage` is the destination image.
- `dstImageLayout` is the layout of the destination image subresources for the resolve.
- `regionCount` is the number of regions to resolve.
- `pRegions` is a pointer to an array of VkImageResolve structures specifying the regions to resolve.

During the resolve the samples corresponding to each pixel location in the source are converted to a single sample before being written to the destination. If the source formats are floating-point or normalized types, the sample values for each pixel are resolved in an implementation-dependent manner. If the source formats are integer types, a single sample's value is selected for each pixel.

`srcOffset` and `dstOffset` select the initial `x`, `y`, and `z` offsets in texels of the sub-regions of the source and destination image data. `extent` is the size in texels of the source image to resolve in `width`, `height` and `depth`. Each element of `pRegions` **must** be a region that is contained within its corresponding image.

Resolves are done layer by layer starting with `baseArrayLayer` member of `srcSubresource` for the source and `dstSubresource` for the destination. `layerCount` layers are resolved to the destination image.

Valid Usage

- VUID-vkCmdResolveImage-pRegions-00255
  The union of all source regions, and the union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-vkCmdResolveImage-srcImage-00256
  If `srcImage` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object

- VUID-vkCmdResolveImage-srcImage-00257
  `srcImage` **must** have a sample count equal to any valid sample count value other than `VK_SAMPLE_COUNT_1_BIT`
- VUID-vkCmdResolveImage-dstImage-00258
  If `dstImage` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdResolveImage-dstImage-00259
  `dstImage` **must** have a sample count equal to `VK_SAMPLE_COUNT_1_BIT`
- VUID-vkCmdResolveImage-srcImageLayout-00260
  `srcImageLayout` **must** specify the layout of the image subresources of `srcImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-vkCmdResolveImage-srcImageLayout-01400
  `srcImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL` or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-vkCmdResolveImage-dstImageLayout-00262
  `dstImageLayout` **must** specify the layout of the image subresources of `dstImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-vkCmdResolveImage-dstImageLayout-01401
  `dstImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-vkCmdResolveImage-dstImage-02003
  The format features of `dstImage` **must** contain `VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT`
- VUID-vkCmdResolveImage-srcImage-01386
  `srcImage` and `dstImage` **must** have been created with the same image format
- VUID-vkCmdResolveImage-srcSubresource-01709
  The `srcSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `srcImage` was created
- VUID-vkCmdResolveImage-dstSubresource-01710
  The `dstSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `dstImage` was created
- VUID-vkCmdResolveImage-srcSubresource-01711
  If `srcSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `srcSubresource.baseArrayLayer` + `srcSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `srcImage` was created
- VUID-vkCmdResolveImage-dstSubresource-01712
  If `dstSubresource.layerCount` is not `VK_REMAINING_ARRAY_LAYERS`, `dstSubresource.baseArrayLayer` + `dstSubresource.layerCount` of each element of `pRegions` **must** be less than or equal to the `arrayLayers` specified in VkImageCreateInfo when `dstImage` was created
- VUID-vkCmdResolveImage-dstImage-02546
  `dstImage` and `srcImage` **must** not have been created with `flags` containing `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`

- VUID-vkCmdResolveImage-srcImage-04446
  If `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `srcSubresource.layerCount` **must** be `1`
- VUID-vkCmdResolveImage-srcImage-04447
  If `dstImage` is of type `VK_IMAGE_TYPE_3D`, then for each element of `pRegions`, `dstSubresource.baseArrayLayer` **must** be `0` and `dstSubresource.layerCount` **must** be `1`
- VUID-vkCmdResolveImage-srcOffset-00269
  For each element of `pRegions`, `srcOffset.x` and (`extent.width` + `srcOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `srcSubresource` of `srcImage`
- VUID-vkCmdResolveImage-srcOffset-00270
  For each element of `pRegions`, `srcOffset.y` and (`extent.height` + `srcOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `srcSubresource` of `srcImage`
- VUID-vkCmdResolveImage-srcImage-00271
  If `srcImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `srcOffset.y` **must** be `0` and `extent.height` **must** be `1`
- VUID-vkCmdResolveImage-srcOffset-00272
  For each element of `pRegions`, `srcOffset.z` and (`extent.depth` + `srcOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `srcSubresource` of `srcImage`
- VUID-vkCmdResolveImage-srcImage-00273
  If `srcImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `srcOffset.z` **must** be `0` and `extent.depth` **must** be `1`
- VUID-vkCmdResolveImage-dstOffset-00274
  For each element of `pRegions`, `dstOffset.x` and (`extent.width` + `dstOffset.x`) **must** both be greater than or equal to `0` and less than or equal to the width of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdResolveImage-dstOffset-00275
  For each element of `pRegions`, `dstOffset.y` and (`extent.height` + `dstOffset.y`) **must** both be greater than or equal to `0` and less than or equal to the height of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdResolveImage-dstImage-00276
  If `dstImage` is of type `VK_IMAGE_TYPE_1D`, then for each element of `pRegions`, `dstOffset.y` **must** be `0` and `extent.height` **must** be `1`
- VUID-vkCmdResolveImage-dstOffset-00277
  For each element of `pRegions`, `dstOffset.z` and (`extent.depth` + `dstOffset.z`) **must** both be greater than or equal to `0` and less than or equal to the depth of the specified `dstSubresource` of `dstImage`
- VUID-vkCmdResolveImage-dstImage-00278
  If `dstImage` is of type `VK_IMAGE_TYPE_1D` or `VK_IMAGE_TYPE_2D`, then for each element of `pRegions`, `dstOffset.z` **must** be `0` and `extent.depth` **must** be `1`
- VUID-vkCmdResolveImage-srcImage-06762

`srcImage` **must** have been created with `VK_IMAGE_USAGE_TRANSFER_SRC_BIT` usage flag

- VUID-vkCmdResolveImage-srcImage-06763
  The format features of `srcImage` **must** contain `VK_FORMAT_FEATURE_TRANSFER_SRC_BIT`
- VUID-vkCmdResolveImage-dstImage-06764
  `dstImage` **must** have been created with `VK_IMAGE_USAGE_TRANSFER_DST_BIT` usage flag
- VUID-vkCmdResolveImage-dstImage-06765
  The format features of `dstImage` **must** contain `VK_FORMAT_FEATURE_TRANSFER_DST_BIT`

Valid Usage (Implicit)

- VUID-vkCmdResolveImage-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdResolveImage-srcImage-parameter
  `srcImage` **must** be a valid VkImage handle
- VUID-vkCmdResolveImage-srcImageLayout-parameter
  `srcImageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdResolveImage-dstImage-parameter
  `dstImage` **must** be a valid VkImage handle
- VUID-vkCmdResolveImage-dstImageLayout-parameter
  `dstImageLayout` **must** be a valid VkImageLayout value
- VUID-vkCmdResolveImage-pRegions-parameter
  `pRegions` **must** be a valid pointer to an array of `regionCount` valid VkImageResolve structures
- VUID-vkCmdResolveImage-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdResolveImage-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdResolveImage-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdResolveImage-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdResolveImage-regionCount-arraylength
  `regionCount` **must** be greater than `0`
- VUID-vkCmdResolveImage-commonparent
  Each of `commandBuffer`, `dstImage`, and `srcImage` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics	Action

The `VkImageResolve` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageResolve {
    VkImageSubresourceLayers    srcSubresource;
    VkOffset3D                  srcOffset;
    VkImageSubresourceLayers    dstSubresource;
    VkOffset3D                  dstOffset;
    VkExtent3D                  extent;
} VkImageResolve;
```

- `srcSubresource` and `dstSubresource` are VkImageSubresourceLayers structures specifying the image subresources of the images used for the source and destination image data, respectively. Resolve of depth/stencil images is not supported.
- `srcOffset` and `dstOffset` select the initial `x`, `y`, and `z` offsets in texels of the sub-regions of the source and destination image data.
- `extent` is the size in texels of the source image to resolve in `width`, `height` and `depth`.

Valid Usage

- VUID-VkImageResolve-aspectMask-00266
  The `aspectMask` member of `srcSubresource` and `dstSubresource` **must** only contain `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-VkImageResolve-layerCount-08803
  If neither of the `layerCount` members of `srcSubresource` or `dstSubresource` are `VK_REMAINING_ARRAY_LAYERS`, the `layerCount` member of `srcSubresource` and `dstSubresource` **must** match
- VUID-VkImageResolve-layerCount-08804
  If one of the `layerCount` members of `srcSubresource` or `dstSubresource` is

VK_REMAINING_ARRAY_LAYERS, the other member **must** be either VK_REMAINING_ARRAY_LAYERS or equal to the arrayLayers member of the VkImageCreateInfo used to create the image minus baseArrayLayer

Valid Usage (Implicit)

- VUID-VkImageResolve-srcSubresource-parameter
  srcSubresource **must** be a valid VkImageSubresourceLayers structure
- VUID-VkImageResolve-dstSubresource-parameter
  dstSubresource **must** be a valid VkImageSubresourceLayers structure

A more extensible version of the resolve image command is defined below.

To resolve a multisample image to a non-multisample image, call:

```
// Provided by VK_KHR_copy_commands2
void vkCmdResolveImage2KHR(
    VkCommandBuffer                             commandBuffer,
    const VkResolveImageInfo2*                  pResolveImageInfo);
```

- commandBuffer is the command buffer into which the command will be recorded.
- pResolveImageInfo is a pointer to a VkResolveImageInfo2 structure describing the resolve parameters.

This command is functionally identical to vkCmdResolveImage, but includes extensible sub-structures that include sType and pNext parameters, allowing them to be more easily extended.

Valid Usage

Valid Usage (Implicit)

- VUID-vkCmdResolveImage2-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdResolveImage2-pResolveImageInfo-parameter
  pResolveImageInfo **must** be a valid pointer to a valid VkResolveImageInfo2 structure
- VUID-vkCmdResolveImage2-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdResolveImage2-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdResolveImage2-renderpass

This command **must** only be called outside of a render pass instance

- VUID-vkCmdResolveImage2-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Graphics	Action

The `VkResolveImageInfo2` structure is defined as:

```
typedef struct VkResolveImageInfo2 {
    VkStructureType           sType;
    const void*               pNext;
    VkImage                   srcImage;
    VkImageLayout             srcImageLayout;
    VkImage                   dstImage;
    VkImageLayout             dstImageLayout;
    uint32_t                  regionCount;
    const VkImageResolve2*    pRegions;
} VkResolveImageInfo2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkResolveImageInfo2 VkResolveImageInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcImage` is the source image.
- `srcImageLayout` is the layout of the source image subresources for the resolve.
- `dstImage` is the destination image.

- `dstImageLayout` is the layout of the destination image subresources for the resolve.
- `regionCount` is the number of regions to resolve.
- `pRegions` is a pointer to an array of VkImageResolve2 structures specifying the regions to resolve.

Valid Usage

- VUID-VkResolveImageInfo2-pRegions-00255
  The union of all source regions, and the union of all destination regions, specified by the elements of `pRegions`, **must** not overlap in memory
- VUID-VkResolveImageInfo2-srcImage-00256
  If `srcImage` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkResolveImageInfo2-srcImage-00257
  `srcImage` **must** have a sample count equal to any valid sample count value other than `VK_SAMPLE_COUNT_1_BIT`
- VUID-VkResolveImageInfo2-dstImage-00258
  If `dstImage` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkResolveImageInfo2-dstImage-00259
  `dstImage` **must** have a sample count equal to `VK_SAMPLE_COUNT_1_BIT`
- VUID-VkResolveImageInfo2-srcImageLayout-00260
  `srcImageLayout` **must** specify the layout of the image subresources of `srcImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-VkResolveImageInfo2-srcImageLayout-01400
  `srcImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL` or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-VkResolveImageInfo2-dstImageLayout-00262
  `dstImageLayout` **must** specify the layout of the image subresources of `dstImage` specified in `pRegions` at the time this command is executed on a `VkDevice`
- VUID-VkResolveImageInfo2-dstImageLayout-01401
  `dstImageLayout` **must** be `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR`, `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` or `VK_IMAGE_LAYOUT_GENERAL`
- VUID-VkResolveImageInfo2-dstImage-02003
  The format features of `dstImage` **must** contain `VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT`
- VUID-VkResolveImageInfo2-srcImage-01386
  `srcImage` and `dstImage` **must** have been created with the same image format
- VUID-VkResolveImageInfo2-srcSubresource-01709
  The `srcSubresource.mipLevel` member of each element of `pRegions` **must** be less than the `mipLevels` specified in VkImageCreateInfo when `srcImage` was created
- VUID-VkResolveImageInfo2-dstSubresource-01710
  The `dstSubresource.mipLevel` member of each element of `pRegions` **must** be less than the

mipLevels specified in VkImageCreateInfo when dstImage was created

- VUID-VkResolveImageInfo2-srcSubresource-01711
  If srcSubresource.layerCount is not VK_REMAINING_ARRAY_LAYERS, srcSubresource.baseArrayLayer + srcSubresource.layerCount of each element of pRegions **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when srcImage was created
- VUID-VkResolveImageInfo2-dstSubresource-01712
  If dstSubresource.layerCount is not VK_REMAINING_ARRAY_LAYERS, dstSubresource.baseArrayLayer + dstSubresource.layerCount of each element of pRegions **must** be less than or equal to the arrayLayers specified in VkImageCreateInfo when dstImage was created
- VUID-VkResolveImageInfo2-dstImage-02546
  dstImage and srcImage **must** not have been created with flags containing VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT
- VUID-VkResolveImageInfo2-srcImage-04446
  If dstImage is of type VK_IMAGE_TYPE_3D, then for each element of pRegions, srcSubresource.layerCount **must** be 1
- VUID-VkResolveImageInfo2-srcImage-04447
  If dstImage is of type VK_IMAGE_TYPE_3D, then for each element of pRegions, dstSubresource.baseArrayLayer **must** be 0 and dstSubresource.layerCount **must** be 1
- VUID-VkResolveImageInfo2-srcOffset-00269
  For each element of pRegions, srcOffset.x and (extent.width + srcOffset.x) **must** both be greater than or equal to 0 and less than or equal to the width of the specified srcSubresource of srcImage
- VUID-VkResolveImageInfo2-srcOffset-00270
  For each element of pRegions, srcOffset.y and (extent.height + srcOffset.y) **must** both be greater than or equal to 0 and less than or equal to the height of the specified srcSubresource of srcImage
- VUID-VkResolveImageInfo2-srcImage-00271
  If srcImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, srcOffset.y **must** be 0 and extent.height **must** be 1
- VUID-VkResolveImageInfo2-srcOffset-00272
  For each element of pRegions, srcOffset.z and (extent.depth + srcOffset.z) **must** both be greater than or equal to 0 and less than or equal to the depth of the specified srcSubresource of srcImage
- VUID-VkResolveImageInfo2-srcImage-00273
  If srcImage is of type VK_IMAGE_TYPE_1D or VK_IMAGE_TYPE_2D, then for each element of pRegions, srcOffset.z **must** be 0 and extent.depth **must** be 1
- VUID-VkResolveImageInfo2-dstOffset-00274
  For each element of pRegions, dstOffset.x and (extent.width + dstOffset.x) **must** both be greater than or equal to 0 and less than or equal to the width of the specified dstSubresource of dstImage
- VUID-VkResolveImageInfo2-dstOffset-00275

For each element of pRegions, dstOffset.y and (extent.height + dstOffset.y) **must** both be greater than or equal to 0 and less than or equal to the height of the specified dstSubresource of dstImage

- VUID-VkResolveImageInfo2-dstImage-00276
  If dstImage is of type VK_IMAGE_TYPE_1D, then for each element of pRegions, dstOffset.y **must** be 0 and extent.height **must** be 1
- VUID-VkResolveImageInfo2-dstOffset-00277
  For each element of pRegions, dstOffset.z and (extent.depth + dstOffset.z) **must** both be greater than or equal to 0 and less than or equal to the depth of the specified dstSubresource of dstImage
- VUID-VkResolveImageInfo2-dstImage-00278
  If dstImage is of type VK_IMAGE_TYPE_1D or VK_IMAGE_TYPE_2D, then for each element of pRegions, dstOffset.z **must** be 0 and extent.depth **must** be 1
- VUID-VkResolveImageInfo2-srcImage-06762
  srcImage **must** have been created with VK_IMAGE_USAGE_TRANSFER_SRC_BIT usage flag
- VUID-VkResolveImageInfo2-srcImage-06763
  The format features of srcImage **must** contain VK_FORMAT_FEATURE_TRANSFER_SRC_BIT
- VUID-VkResolveImageInfo2-dstImage-06764
  dstImage **must** have been created with VK_IMAGE_USAGE_TRANSFER_DST_BIT usage flag
- VUID-VkResolveImageInfo2-dstImage-06765
  The format features of dstImage **must** contain VK_FORMAT_FEATURE_TRANSFER_DST_BIT

Valid Usage (Implicit)

- VUID-VkResolveImageInfo2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_RESOLVE_IMAGE_INFO_2
- VUID-VkResolveImageInfo2-pNext-pNext
  pNext **must** be NULL
- VUID-VkResolveImageInfo2-srcImage-parameter
  srcImage **must** be a valid VkImage handle
- VUID-VkResolveImageInfo2-srcImageLayout-parameter
  srcImageLayout **must** be a valid VkImageLayout value
- VUID-VkResolveImageInfo2-dstImage-parameter
  dstImage **must** be a valid VkImage handle
- VUID-VkResolveImageInfo2-dstImageLayout-parameter
  dstImageLayout **must** be a valid VkImageLayout value
- VUID-VkResolveImageInfo2-pRegions-parameter
  pRegions **must** be a valid pointer to an array of regionCount valid VkImageResolve2 structures
- VUID-VkResolveImageInfo2-regionCount-arraylength
  regionCount **must** be greater than 0

- VUID-VkResolveImageInfo2-commonparent
  Both of `dstImage`, and `srcImage` **must** have been created, allocated, or retrieved from the same VkDevice

The `VkImageResolve2` structure is defined as:

```
typedef struct VkImageResolve2 {
    VkStructureType             sType;
    const void*                 pNext;
    VkImageSubresourceLayers    srcSubresource;
    VkOffset3D                  srcOffset;
    VkImageSubresourceLayers    dstSubresource;
    VkOffset3D                  dstOffset;
    VkExtent3D                  extent;
} VkImageResolve2;
```

or the equivalent

```
// Provided by VK_KHR_copy_commands2
typedef VkImageResolve2 VkImageResolve2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `srcSubresource` and `dstSubresource` are VkImageSubresourceLayers structures specifying the image subresources of the images used for the source and destination image data, respectively. Resolve of depth/stencil images is not supported.
- `srcOffset` and `dstOffset` select the initial `x`, `y`, and `z` offsets in texels of the sub-regions of the source and destination image data.
- `extent` is the size in texels of the source image to resolve in `width`, `height` and `depth`.

Valid Usage

- VUID-VkImageResolve2-aspectMask-00266
  The `aspectMask` member of `srcSubresource` and `dstSubresource` **must** only contain `VK_IMAGE_ASPECT_COLOR_BIT`
- VUID-VkImageResolve2-layerCount-08803
  If neither of the `layerCount` members of `srcSubresource` or `dstSubresource` are `VK_REMAINING_ARRAY_LAYERS`, the `layerCount` member of `srcSubresource` and `dstSubresource` **must** match
- VUID-VkImageResolve2-layerCount-08804
  If one of the `layerCount` members of `srcSubresource` or `dstSubresource` is `VK_REMAINING_ARRAY_LAYERS`, the other member **must** be either `VK_REMAINING_ARRAY_LAYERS` or equal to the `arrayLayers` member of the VkImageCreateInfo used to create the image

minus `baseArrayLayer`

Valid Usage (Implicit)

- VUID-VkImageResolve2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_RESOLVE_2`
- VUID-VkImageResolve2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkImageResolve2-srcSubresource-parameter
  `srcSubresource` **must** be a valid VkImageSubresourceLayers structure
- VUID-VkImageResolve2-dstSubresource-parameter
  `dstSubresource` **must** be a valid VkImageSubresourceLayers structure

Chapter 21. Drawing Commands

Drawing commands (commands with `Draw` in the name) provoke work in a graphics pipeline. Drawing commands are recorded into a command buffer and when executed by a queue, will produce work which executes according to the bound graphics pipeline, or if the `shaderObject` feature is enabled, any shader objects bound to graphics stages. A graphics pipeline or a combination of one or more graphics shader objects **must** be bound to a command buffer before any drawing commands are recorded in that command buffer.

Each draw is made up of zero or more vertices and zero or more instances, which are processed by the device and result in the assembly of primitives. Primitives are assembled according to the `pInputAssemblyState` member of the VkGraphicsPipelineCreateInfo structure, which is of type `VkPipelineInputAssemblyStateCreateInfo`:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineInputAssemblyStateCreateInfo {
    VkStructureType                            sType;
    const void*                                pNext;
    VkPipelineInputAssemblyStateCreateFlags    flags;
    VkPrimitiveTopology                        topology;
    VkBool32                                   primitiveRestartEnable;
} VkPipelineInputAssemblyStateCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `topology` is a VkPrimitiveTopology defining the primitive topology, as described below.
- `primitiveRestartEnable` controls whether a special vertex index value is treated as restarting the assembly of primitives. This enable only applies to indexed draws (vkCmdDrawIndexed, vkCmdDrawMultiIndexedEXT, and vkCmdDrawIndexedIndirect), and the special index value is either 0xFFFFFFFF when the `indexType` parameter of `vkCmdBindIndexBuffer2KHR` or `vkCmdBindIndexBuffer` is equal to `VK_INDEX_TYPE_UINT32`, 0xFF when `indexType` is equal to `VK_INDEX_TYPE_UINT8_KHR`, or 0xFFFF when `indexType` is equal to `VK_INDEX_TYPE_UINT16`. Primitive restart is not allowed for "list" topologies, unless one of the features `primitiveTopologyPatchListRestart` (for `VK_PRIMITIVE_TOPOLOGY_PATCH_LIST`) or `primitiveTopologyListRestart` (for all other list topologies) is enabled.

Restarting the assembly of primitives discards the most recent index values if those elements formed an incomplete primitive, and restarts the primitive assembly using the subsequent indices, but only assembling the immediately following element through the end of the originally specified elements. The primitive restart index value comparison is performed before adding the `vertexOffset` value to the index value.

Valid Usage

- VUID-VkPipelineInputAssemblyStateCreateInfo-topology-06252
  If the primitiveTopologyListRestart feature is not enabled, and topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, primitiveRestartEnable **must** be VK_FALSE
- VUID-VkPipelineInputAssemblyStateCreateInfo-topology-06253
  If the primitiveTopologyPatchListRestart feature is not enabled, and topology is VK_PRIMITIVE_TOPOLOGY_PATCH_LIST, primitiveRestartEnable **must** be VK_FALSE
- VUID-VkPipelineInputAssemblyStateCreateInfo-topology-00429
  If the geometryShader feature is not enabled, topology **must** not be any of VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, VK_PRIMITIVE_TOPOLOGY_LINE_STRIP_WITH_ADJACENCY, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY
- VUID-VkPipelineInputAssemblyStateCreateInfo-topology-00430
  If the tessellationShader feature is not enabled, topology **must** not be VK_PRIMITIVE_TOPOLOGY_PATCH_LIST
- VUID-VkPipelineInputAssemblyStateCreateInfo-triangleFans-04452
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::triangleFans is VK_FALSE, topology **must** not be VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN

Valid Usage (Implicit)

- VUID-VkPipelineInputAssemblyStateCreateInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO
- VUID-VkPipelineInputAssemblyStateCreateInfo-pNext-pNext
  pNext **must** be NULL
- VUID-VkPipelineInputAssemblyStateCreateInfo-flags-zerobitmask
  flags **must** be 0
- VUID-VkPipelineInputAssemblyStateCreateInfo-topology-parameter
  topology **must** be a valid VkPrimitiveTopology value

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineInputAssemblyStateCreateFlags;
```

VkPipelineInputAssemblyStateCreateFlags is a bitmask type for setting a mask, but is currently reserved for future use.

To dynamically control whether a special vertex index value is treated as restarting the assembly of primitives, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetPrimitiveRestartEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    primitiveRestartEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `primitiveRestartEnable` controls whether a special vertex index value is treated as restarting the assembly of primitives. It behaves in the same way as `VkPipelineInputAssemblyStateCreateInfo` ::`primitiveRestartEnable`

This command sets the primitive restart enable for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineInputAssemblyStateCreateInfo::`primitiveRestartEnable` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetPrimitiveRestartEnable-None-08970
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetPrimitiveRestartEnable-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetPrimitiveRestartEnable-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetPrimitiveRestartEnable-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetPrimitiveRestartEnable-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

21.1. Primitive Topologies

Primitive topology determines how consecutive vertices are organized into primitives, and determines the type of primitive that is used at the beginning of the graphics pipeline. The effective topology for later stages of the pipeline is altered by tessellation or geometry shading (if either is in use) and depends on the execution modes of those shaders.

The primitive topologies defined by VkPrimitiveTopology are:

```
// Provided by VK_VERSION_1_0
typedef enum VkPrimitiveTopology {
    VK_PRIMITIVE_TOPOLOGY_POINT_LIST = 0,
    VK_PRIMITIVE_TOPOLOGY_LINE_LIST = 1,
    VK_PRIMITIVE_TOPOLOGY_LINE_STRIP = 2,
    VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST = 3,
    VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP = 4,
    VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN = 5,
    VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY = 6,
    VK_PRIMITIVE_TOPOLOGY_LINE_STRIP_WITH_ADJACENCY = 7,
    VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY = 8,
    VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY = 9,
    VK_PRIMITIVE_TOPOLOGY_PATCH_LIST = 10,
} VkPrimitiveTopology;
```

- VK_PRIMITIVE_TOPOLOGY_POINT_LIST specifies a series of separate point primitives.
- VK_PRIMITIVE_TOPOLOGY_LINE_LIST specifies a series of separate line primitives.
- VK_PRIMITIVE_TOPOLOGY_LINE_STRIP specifies a series of connected line primitives with consecutive lines sharing a vertex.
- VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST specifies a series of separate triangle primitives.
- VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP specifies a series of connected triangle primitives with consecutive triangles sharing an edge.
- VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN specifies a series of connected triangle primitives with all triangles sharing a common vertex. If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::triangleFans is VK_FALSE, then triangle fans are not supported by the implementation, and VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN **must** not be used.

- VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY specifies a series of separate line primitives with adjacency.
- VK_PRIMITIVE_TOPOLOGY_LINE_STRIP_WITH_ADJACENCY specifies a series of connected line primitives with adjacency, with consecutive primitives sharing three vertices.
- VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY specifies a series of separate triangle primitives with adjacency.
- VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY specifies connected triangle primitives with adjacency, with consecutive triangles sharing an edge.
- VK_PRIMITIVE_TOPOLOGY_PATCH_LIST specifies separate patch primitives.

Each primitive topology, and its construction from a list of vertices, is described in detail below with a supporting diagram, according to the following key:

•	Vertex	A point in 3-dimensional space. Positions chosen within the diagrams are arbitrary and for illustration only.
5	Vertex Number	Sequence position of a vertex within the provided vertex data.
	Provoking Vertex	Provoking vertex within the main primitive. The tail is angled towards the relevant primitive. Used in flat shading.
	Primitive Edge	An edge connecting the points of a main primitive.
	Adjacency Edge	Points connected by these lines do not contribute to a main primitive, and are only accessible in a geometry shader.
	Winding Order	The relative order in which vertices are defined within a primitive, used in the facing determination. This ordering has no specific start or end point.

The diagrams are supported with mathematical definitions where the vertices (v) and primitives (p) are numbered starting from 0; v_0 is the first vertex in the provided data and p_0 is the first primitive in the set of primitives defined by the vertices and topology.

To dynamically set primitive topology, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetPrimitiveTopologyEXT(
    VkCommandBuffer                             commandBuffer,
    VkPrimitiveTopology                         primitiveTopology);
```

- commandBuffer is the command buffer into which the command will be recorded.
- primitiveTopology specifies the primitive topology to use for drawing.

This command sets the primitive topology for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineInputAssemblyStateCreateInfo ::topology value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetPrimitiveTopology-None-08971
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetPrimitiveTopology-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetPrimitiveTopology-primitiveTopology-parameter
  `primitiveTopology` **must** be a valid VkPrimitiveTopology value
- VUID-vkCmdSetPrimitiveTopology-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetPrimitiveTopology-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetPrimitiveTopology-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

21.1.1. Topology Class

The primitive topologies are grouped into the following topology classes:

Table 26. Topology Classes

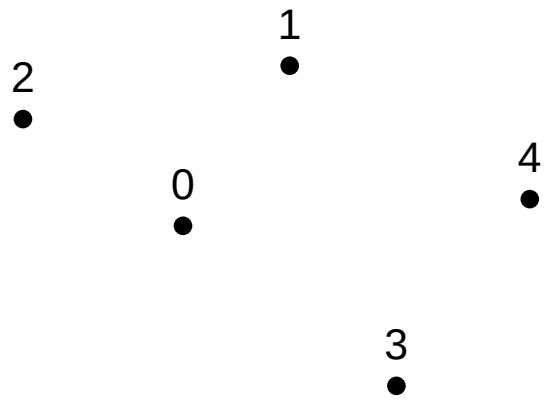
Topology Class	Primitive Topology
Point	VK_PRIMITIVE_TOPOLOGY_POINT_LIST
Line	VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_STRIP, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, VK_PRIMITIVE_TOPOLOGY_LINE_STRIP_WITH_ADJACENCY
Triangle	VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY
Patch	VK_PRIMITIVE_TOPOLOGY_PATCH_LIST

21.1.2. Point Lists

When the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, each consecutive vertex defines a single point primitive, according to the equation:

$$p_i = \{v_i\}$$

As there is only one vertex, that vertex is the provoking vertex. The number of primitives generated is equal to vertexCount.



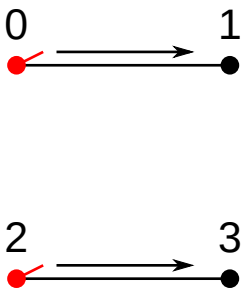
21.1.3. Line Lists

When the primitive topology is VK_PRIMITIVE_TOPOLOGY_LINE_LIST, each consecutive pair of vertices defines a single line primitive, according to the equation:

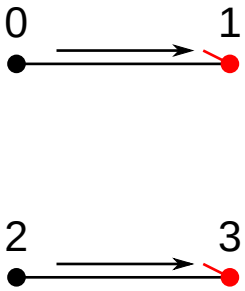
$$p_i = \{v_{2i}, v_{2i+1}\}$$

The number of primitives generated is equal to ⌊vertexCount/2⌋.

When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT, the provoking vertex for p_i is v_{2i}.



When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT, the provoking vertex for p_i is v_{2i+1}.



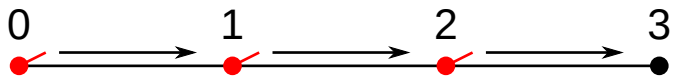
21.1.4. Line Strips

When the primitive topology is VK_PRIMITIVE_TOPOLOGY_LINE_STRIP, one line primitive is defined by each vertex and the following vertex, according to the equation:

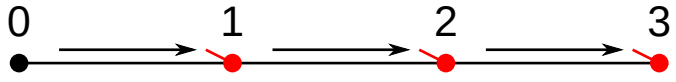
$$p_i = \{v_i, v_{i+1}\}$$

The number of primitives generated is equal to max(0,vertexCount-1).

When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT, the provoking vertex for p_i is v_i.



When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT, the provoking vertex for p_i is v_{i+1}.



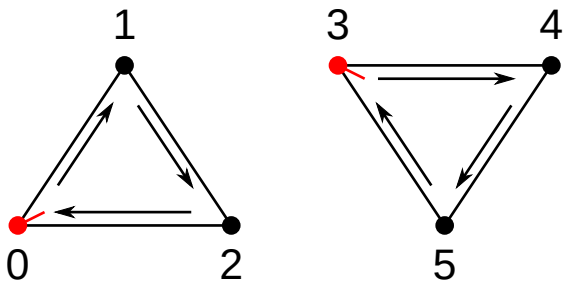
21.1.5. Triangle Lists

When the primitive topology is VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, each consecutive set of three vertices defines a single triangle primitive, according to the equation:

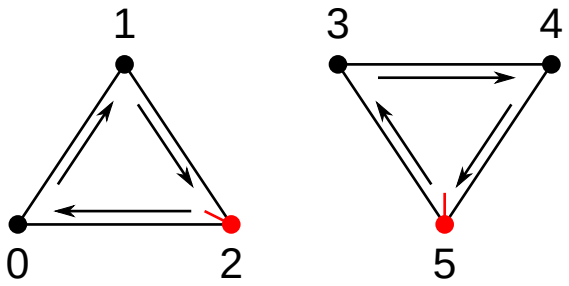
$$p_i = \{v_{3i}, v_{3i+1}, v_{3i+2}\}$$

The number of primitives generated is equal to ⌊vertexCount/3⌋.

When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT, the provoking vertex for p_i is v_{3i}.



When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT, the provoking vertex for p_i is v_{3i+2}.



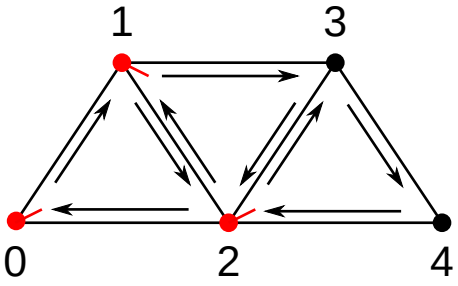
21.1.6. Triangle Strips

When the primitive topology is VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP, one triangle primitive is defined by each vertex and the two vertices that follow it, according to the equation:

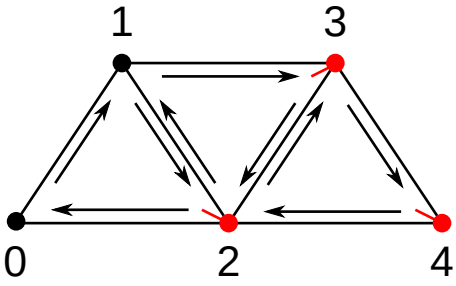
$$p_i = \{v_i, v_{i+(1+i\%2)}, v_{i+(2-i\%2)}\}$$

The number of primitives generated is equal to max(0,vertexCount-2).

When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT, the provoking vertex for p_i is v_i.



When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT, the provoking vertex for p_i is v_{i+2}.



NOTE The ordering of the vertices in each successive triangle is reversed, so that the winding order is consistent throughout the strip.

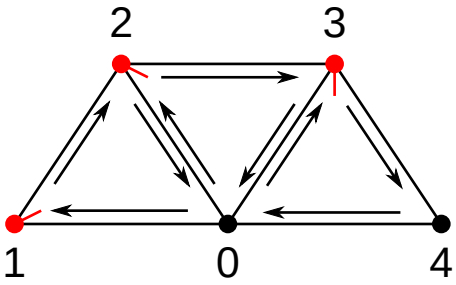
21.1.7. Triangle Fans

When the primitive topology is `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN`, triangle primitives are defined around a shared common vertex, according to the equation:

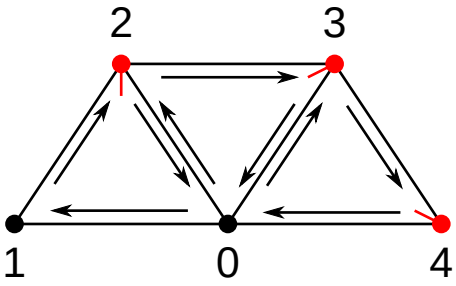
$$p_i = \{v_{i+1}, v_{i+2}, v_0\}$$

The number of primitives generated is equal to max(0,`vertexCount`-2).

When the `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT`, the provoking vertex for p_i is v_{i+1}.



When the `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT`, the provoking vertex for p_i is v_{i+2}.



NOTE If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`triangleFans` is `VK_FALSE`, then triangle fans are not supported by the implementation, and `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN` **must** not be used.

21.1.8. Line Lists With Adjacency

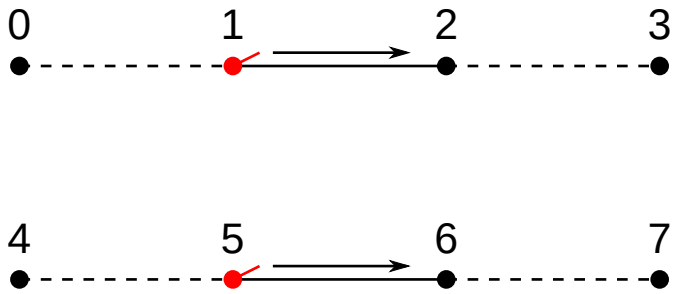
When the primitive topology is VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, each consecutive set of four vertices defines a single line primitive with adjacency, according to the equation:

$$p_i = \{v_{4i}, v_{4i+1}, v_{4i+2}, v_{4i+3}\}$$

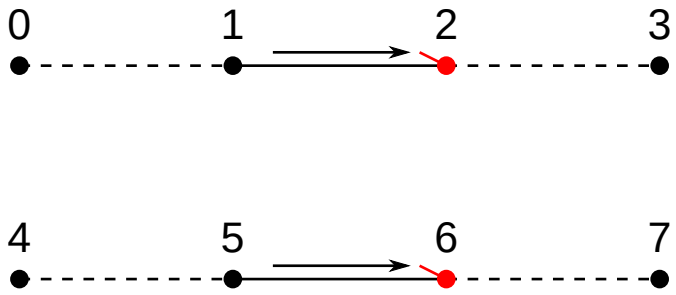
A line primitive is described by the second and third vertices of the total primitive, with the remaining two vertices only accessible in a geometry shader.

The number of primitives generated is equal to ⌊vertexCount/4⌋.

When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT, the provoking vertex for p_i is v_{4i+1}.



When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT, the provoking vertex for p_i is v_{4i+2}.



21.1.9. Line Strips With Adjacency

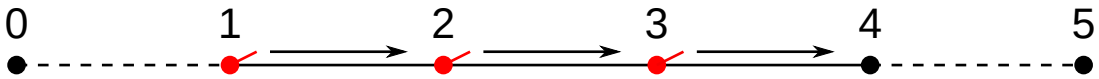
When the primitive topology is VK_PRIMITIVE_TOPOLOGY_LINE_STRIP_WITH_ADJACENCY, one line primitive with adjacency is defined by each vertex and the following vertex, according to the equation:

$$p_i = \{v_i, v_{i+1}, v_{i+2}, v_{i+3}\}$$

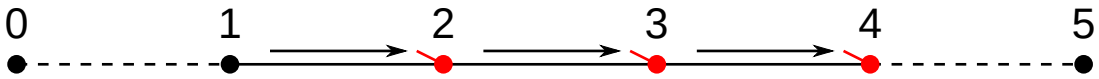
A line primitive is described by the second and third vertices of the total primitive, with the remaining two vertices only accessible in a geometry shader.

The number of primitives generated is equal to max(0,vertexCount-3).

When the provokingVertexMode is VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT, the provoking vertex for p_i is v_{i+1}.



When the `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT`, the provoking vertex for p_i is v_{i+2}.



21.1.10. Triangle Lists With Adjacency

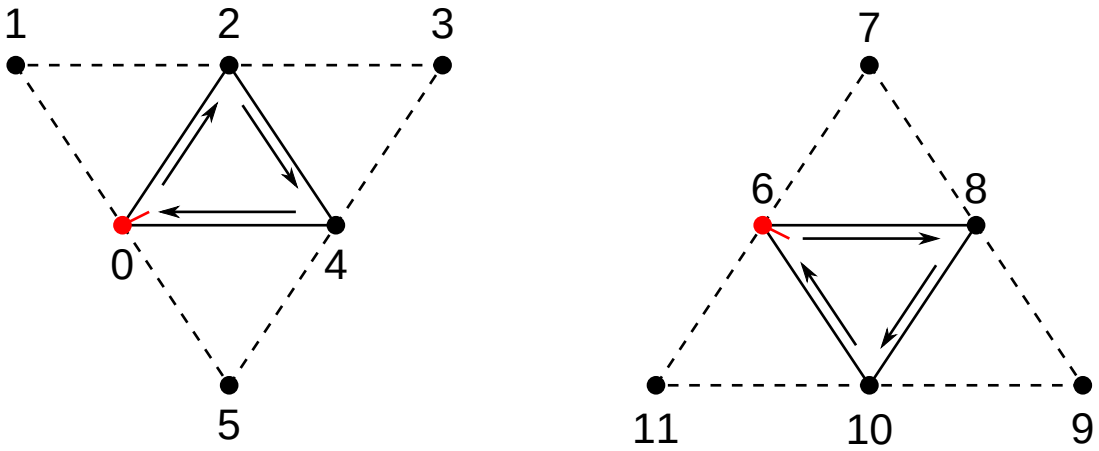
When the primitive topology is `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY`, each consecutive set of six vertices defines a single triangle primitive with adjacency, according to the equations:

$$p_i = \{v_{6i}, v_{6i+1}, v_{6i+2}, v_{6i+3}, v_{6i+4}, v_{6i+5}\}$$

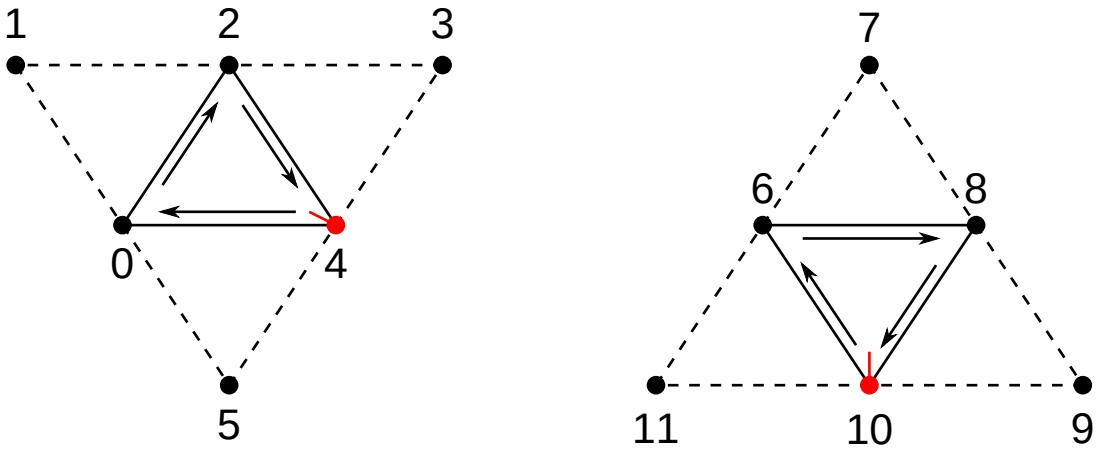
A triangle primitive is described by the first, third, and fifth vertices of the total primitive, with the remaining three vertices only accessible in a geometry shader.

The number of primitives generated is equal to ⌊`vertexCount`/6⌋.

When the `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT`, the provoking vertex for p_i is v_{6i}.



When the `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT`, the provoking vertex for p_i is v_{6i+4}.



21.1.11. Triangle Strips With Adjacency

When the primitive topology is `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY`, one triangle primitive with adjacency is defined by each vertex and the following 5 vertices.

The number of primitives generated, n, is equal to ⌊max(0, `vertexCount` - 4)/2⌋.

If n=1, the primitive is defined as:

$$p = \{v_0, v_1, v_2, v_5, v_4, v_3\}$$

If n>1, the total primitive consists of different vertices according to where it is in the strip:

$$p_i = \{v_{2i}, v_{2i+1}, v_{2i+2}, v_{2i+6}, v_{2i+4}, v_{2i+3}\} \text{ when } i=0$$

$$p_i = \{v_{2i}, v_{2i+3}, v_{2i+4}, v_{2i+6}, v_{2i+2}, v_{2i-2}\} \text{ when } i>0,\ i<n-1, \text{ and } i\%2=1$$

$$p_i = \{v_{2i}, v_{2i-2}, v_{2i+2}, v_{2i+6}, v_{2i+4}, v_{2i+3}\} \text{ when } i>0,\ i<n-1, \text{ and } i\%2=0$$

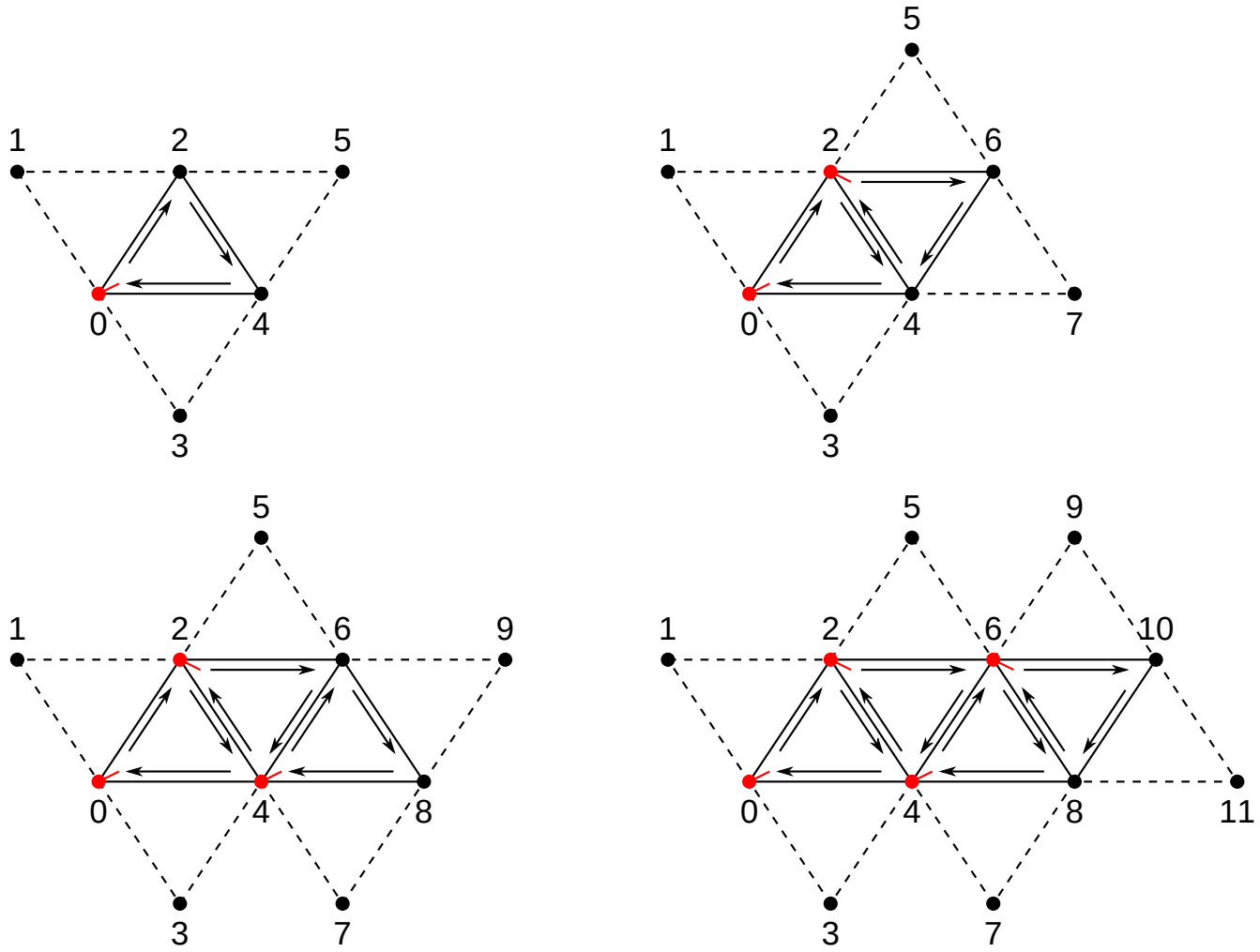
$$p_i = \{v_{2i}, v_{2i+3}, v_{2i+4}, v_{2i+5}, v_{2i+2}, v_{2i-2}\} \text{ when } i=n-1 \text{ and } i\%2=1$$

$$p_i = \{v_{2i}, v_{2i-2}, v_{2i+2}, v_{2i+5}, v_{2i+4}, v_{2i+3}\} \text{ when } i=n-1 \text{ and } i\%2=0$$

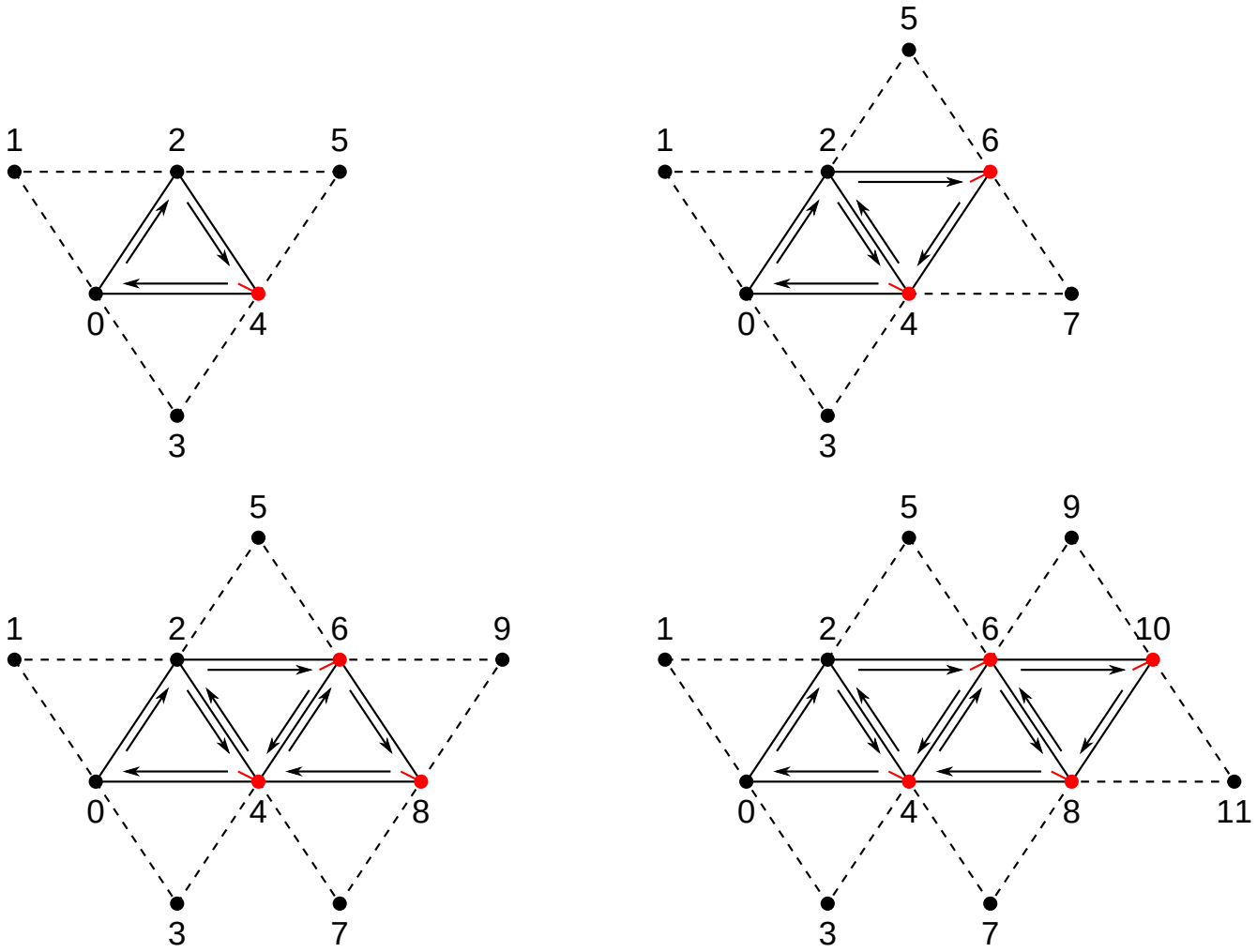
A triangle primitive is described by the first, third, and fifth vertices of the total primitive in all cases, with the remaining three vertices only accessible in a geometry shader.

NOTE	The ordering of the vertices in each successive triangle is altered so that the winding order is consistent throughout the strip.

When the `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT`, the provoking vertex for p_i is always v_{2i}.



When the `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT`, the provoking vertex for p_i is always v_{2i+4}.



21.1.12. Patch Lists

When the primitive topology is `VK_PRIMITIVE_TOPOLOGY_PATCH_LIST`, each consecutive set of m vertices defines a single patch primitive, according to the equation:

$$p_i = \{v_{mi}, v_{mi+1}, \ldots, v_{mi+(m-2)}, v_{mi+(m-1)}\}$$

where m is equal to VkPipelineTessellationStateCreateInfo::`patchControlPoints`.

Patch lists are never passed to vertex post-processing, and as such no provoking vertex is defined for patch primitives. The number of primitives generated is equal to ⌊`vertexCount`/m⌋.

The vertices comprising a patch have no implied geometry, and are used as inputs to tessellation shaders and the fixed-function tessellator to generate new point, line, or triangle primitives.

21.2. Primitive Order

Primitives generated by drawing commands progress through the stages of the graphics pipeline in *primitive order*. Primitive order is initially determined in the following way:

1. Submission order determines the initial ordering
2. For indirect drawing commands, the order in which accessed instances of the

VkDrawIndirectCommand are stored in `buffer`, from lower indirect buffer addresses to higher addresses.

3. If a drawing command includes multiple instances, the order in which instances are executed, from lower numbered instances to higher.
4. The order in which primitives are specified by a drawing command:
    - For non-indexed draws, from vertices with a lower numbered `vertexIndex` to a higher numbered `vertexIndex`.
    - For indexed draws, vertices sourced from a lower index buffer addresses to higher addresses.

Within this order implementations further sort primitives:

5. If tessellation shading is active, by an implementation-dependent order of new primitives generated by tessellation.
6. If geometry shading is active, by the order new primitives are generated by geometry shading.
7. If the polygon mode is not `VK_POLYGON_MODE_FILL`, by an implementation-dependent ordering of the new primitives generated within the original primitive.

Primitive order is later used to define rasterization order, which determines the order in which fragments output results to a framebuffer.

21.3. Programmable Primitive Shading

Once primitives are assembled, they proceed to the vertex shading stage of the pipeline. If the draw includes multiple instances, then the set of primitives is sent to the vertex shading stage multiple times, once for each instance.

It is implementation-dependent whether vertex shading occurs on vertices that are discarded as part of incomplete primitives, but if it does occur then it operates as if they were vertices in complete primitives and such invocations **can** have side effects.

Vertex shading receives two per-vertex inputs from the primitive assembly stage - the `vertexIndex` and the `instanceIndex`. How these values are generated is defined below, with each command.

Drawing commands fall roughly into two categories:

- Non-indexed drawing commands present a sequential `vertexIndex` to the vertex shader. The sequential index is generated automatically by the device (see Fixed-Function Vertex Processing for details on both specifying the vertex attributes indexed by `vertexIndex`, as well as binding vertex buffers containing those attributes to a command buffer). These commands are:
    - vkCmdDraw
    - vkCmdDrawIndirect
    - vkCmdDrawIndirectCountKHR
    - vkCmdDrawMultiEXT
- Indexed drawing commands read index values from an *index buffer* and use this to compute the

`vertexIndex` value for the vertex shader. These commands are:

- vkCmdDrawIndexed
- vkCmdDrawIndexedIndirect
- vkCmdDrawIndexedIndirectCountKHR
- vkCmdDrawMultiIndexedEXT

To bind an index buffer to a command buffer, call:

```
// Provided by VK_VERSION_1_0
void vkCmdBindIndexBuffer(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    buffer,
    VkDeviceSize                                offset,
    VkIndexType                                 indexType);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `buffer` is the buffer being bound.
- `offset` is the starting offset in bytes within `buffer` used in index buffer address calculations.
- `indexType` is a VkIndexType value specifying the size of the indices.

If the `maintenance6` feature is enabled, `buffer` **can** be VK_NULL_HANDLE. If `buffer` is VK_NULL_HANDLE and the `nullDescriptor` feature is enabled, every index fetched results in a value of zero.

Valid Usage

- VUID-vkCmdBindIndexBuffer-offset-08782
  `offset` **must** be less than the size of `buffer`
- VUID-vkCmdBindIndexBuffer-offset-08783
  The sum of `offset` and the base address of the range of `VkDeviceMemory` object that is backing `buffer`, **must** be a multiple of the size of the type indicated by `indexType`
- VUID-vkCmdBindIndexBuffer-buffer-08784
  `buffer` **must** have been created with the `VK_BUFFER_USAGE_INDEX_BUFFER_BIT` flag
- VUID-vkCmdBindIndexBuffer-buffer-08785
  If `buffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdBindIndexBuffer-indexType-08786
  `indexType` **must** not be `VK_INDEX_TYPE_NONE_KHR`
- VUID-vkCmdBindIndexBuffer-indexType-08787
  If `indexType` is `VK_INDEX_TYPE_UINT8_KHR`, the `indexTypeUint8` feature **must** be enabled
- VUID-vkCmdBindIndexBuffer-None-09493
  If the `maintenance6` feature is not enabled, `buffer` **must** not be VK_NULL_HANDLE

- VUID-vkCmdBindIndexBuffer-buffer-09494
  If `buffer` is VK_NULL_HANDLE, offset **must** be zero

Valid Usage (Implicit)

- VUID-vkCmdBindIndexBuffer-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindIndexBuffer-buffer-parameter
  If `buffer` is not VK_NULL_HANDLE, `buffer` **must** be a valid VkBuffer handle
- VUID-vkCmdBindIndexBuffer-indexType-parameter
  `indexType` **must** be a valid VkIndexType value
- VUID-vkCmdBindIndexBuffer-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBindIndexBuffer-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBindIndexBuffer-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBindIndexBuffer-commonparent
  Both of `buffer`, and `commandBuffer` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To bind an index buffer, along with its size, to a command buffer, call:

```
// Provided by VK_KHR_maintenance5
void vkCmdBindIndexBuffer2KHR(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    buffer,
    VkDeviceSize                                offset,
    VkDeviceSize                                size,
    VkIndexType                                 indexType);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `buffer` is the buffer being bound.
- `offset` is the starting offset in bytes within `buffer` used in index buffer address calculations.
- `size` is the size in bytes of index data bound from `buffer`.
- `indexType` is a VkIndexType value specifying the size of the indices.

`size` specifies the bound size of the index buffer starting from `offset`. If `size` is `VK_WHOLE_SIZE` then the bound size is from `offset` to the end of the `buffer`.

If the `maintenance6` feature is enabled, `buffer` **can** be VK_NULL_HANDLE. If `buffer` is VK_NULL_HANDLE and the `nullDescriptor` feature is enabled, every index fetched results in a value of zero.

Valid Usage

- VUID-vkCmdBindIndexBuffer2KHR-offset-08782
  `offset` **must** be less than the size of `buffer`
- VUID-vkCmdBindIndexBuffer2KHR-offset-08783
  The sum of `offset` and the base address of the range of `VkDeviceMemory` object that is backing `buffer`, **must** be a multiple of the size of the type indicated by `indexType`
- VUID-vkCmdBindIndexBuffer2KHR-buffer-08784
  `buffer` **must** have been created with the `VK_BUFFER_USAGE_INDEX_BUFFER_BIT` flag
- VUID-vkCmdBindIndexBuffer2KHR-buffer-08785
  If `buffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdBindIndexBuffer2KHR-indexType-08786
  `indexType` **must** not be `VK_INDEX_TYPE_NONE_KHR`
- VUID-vkCmdBindIndexBuffer2KHR-indexType-08787
  If `indexType` is `VK_INDEX_TYPE_UINT8_KHR`, the `indexTypeUint8` feature **must** be enabled
- VUID-vkCmdBindIndexBuffer2KHR-None-09493
  If the `maintenance6` feature is not enabled, `buffer` **must** not be VK_NULL_HANDLE
- VUID-vkCmdBindIndexBuffer2KHR-buffer-09494
  If `buffer` is VK_NULL_HANDLE, offset **must** be zero
- VUID-vkCmdBindIndexBuffer2KHR-size-08767

If `size` is not `VK_WHOLE_SIZE`, `size` **must** be a multiple of the size of the type indicated by `indexType`

- VUID-vkCmdBindIndexBuffer2KHR-size-08768
  If `size` is not `VK_WHOLE_SIZE`, the sum of `offset` and `size` **must** be less than or equal to the size of `buffer`

Valid Usage (Implicit)

- VUID-vkCmdBindIndexBuffer2KHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindIndexBuffer2KHR-buffer-parameter
  If `buffer` is not VK_NULL_HANDLE, `buffer` **must** be a valid VkBuffer handle
- VUID-vkCmdBindIndexBuffer2KHR-indexType-parameter
  `indexType` **must** be a valid VkIndexType value
- VUID-vkCmdBindIndexBuffer2KHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBindIndexBuffer2KHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBindIndexBuffer2KHR-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBindIndexBuffer2KHR-commonparent
  Both of `buffer`, and `commandBuffer` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Possible values of vkCmdBindIndexBuffer2KHR::`indexType` and vkCmdBindIndexBuffer::`indexType`, specifying the size of indices, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkIndexType {
    VK_INDEX_TYPE_UINT16 = 0,
    VK_INDEX_TYPE_UINT32 = 1,
  // Provided by VK_KHR_acceleration_structure
    VK_INDEX_TYPE_NONE_KHR = 1000165000,
  // Provided by VK_KHR_index_type_uint8
    VK_INDEX_TYPE_UINT8_KHR = 1000265000,
} VkIndexType;
```

- `VK_INDEX_TYPE_UINT16` specifies that indices are 16-bit unsigned integer values.
- `VK_INDEX_TYPE_UINT32` specifies that indices are 32-bit unsigned integer values.
- `VK_INDEX_TYPE_NONE_KHR` specifies that no indices are provided.
- `VK_INDEX_TYPE_UINT8_KHR` specifies that indices are 8-bit unsigned integer values.

The parameters for each drawing command are specified directly in the command or read from buffer memory, depending on the command. Drawing commands that source their parameters from buffer memory are known as *indirect* drawing commands.

All drawing commands interact with the `robustBufferAccess` feature.

To record a non-indexed draw, call:

```
// Provided by VK_VERSION_1_0
void vkCmdDraw(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    vertexCount,
    uint32_t                                    instanceCount,
    uint32_t                                    firstVertex,
    uint32_t                                    firstInstance);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `vertexCount` is the number of vertices to draw.
- `instanceCount` is the number of instances to draw.
- `firstVertex` is the index of the first vertex to draw.
- `firstInstance` is the instance ID of the first instance to draw.

When the command is executed, primitives are assembled using the current primitive topology and `vertexCount` consecutive vertex indices with the first `vertexIndex` value equal to `firstVertex`. The primitives are drawn `instanceCount` times with `instanceIndex` starting with `firstInstance` and increasing sequentially for each instance. The assembled primitives execute the bound graphics pipeline.

Valid Usage

- VUID-vkCmdDraw-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDraw-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDraw-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDraw-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`
- VUID-vkCmdDraw-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain `VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT`
- VUID-vkCmdDraw-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT`
- VUID-vkCmdDraw-None-07888
  If a `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER` descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT`
- VUID-vkCmdDraw-None-02692
  If a VkImageView is sampled with `VK_FILTER_CUBIC_EXT` as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT`
- VUID-vkCmdDraw-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with `VK_FILTER_CUBIC_EXT` as a result of this command, it **must** not have a VkImageViewType of `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`
- VUID-vkCmdDraw-filterCubic-02694
  Any VkImageView being sampled with `VK_FILTER_CUBIC_EXT` as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::`filterCubic` returned by vkGetPhysicalDeviceImageFormatProperties2KHR

- VUID-vkCmdDraw-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDraw-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDraw-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDraw-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDraw-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDraw-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDraw-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDraw-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDraw-maintenance4-08602
  If the maintenance4 feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in

Pipeline Layout Compatibility

- VUID-vkCmdDraw-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDraw-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDraw-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDraw-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdDraw-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDraw-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDraw-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDraw-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDraw-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform

buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point

- VUID-vkCmdDraw-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDraw-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDraw-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdDraw-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdDraw-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDraw-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdDraw-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDraw-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdDraw-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDraw-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDraw-SampledType-04471

If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32

- VUID-vkCmdDraw-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDraw-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDraw-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDraw-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDraw-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDraw-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDraw-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDraw-subpass-02685
  The subpass index of the current render pass **must** be equal to the subpass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDraw-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdDraw-OpTypeImage-07468
  If any shader executed by this pipeline accesses an OpTypeImage variable with a Dim operand of SubpassData, it **must** be decorated with an InputAttachmentIndex that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDraw-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat

as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' pInputAttachments [InputAttachmentIndex] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility

- VUID-vkCmdDraw-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a InputAttachmentIndex referenced by VkRenderingInputAttachmentIndexInfoKHR, or no InputAttachmentIndex if VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex or VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are NULL, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdDraw-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an InputAttachmentIndex if both VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex and VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are non-NULL
- VUID-vkCmdDraw-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an InputAttachmentIndex, the InputAttachmentIndex **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDraw-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDraw-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_COLOR_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDraw-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or

- the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
  - there is no bound graphics pipeline or
  - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDraw-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDraw-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdDraw-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDraw-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdDraw-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back writeMask are not zero, and stencil test is enabled, all stencil ops **must** be VK_STENCIL_OP_KEEP
- VUID-vkCmdDraw-None-07831
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VIEWPORT dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07832
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SCISSOR dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDraw-None-07833
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_WIDTH dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08619
  If a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor

member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDraw-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties::maxMultiviewInstanceIndex
- VUID-vkCmdDraw-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDraw-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDraw-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdDraw-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdDraw-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDraw-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments
- VUID-vkCmdDraw-None-08876
  If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdDraw-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command

must not write any values to the depth attachment

- VUID-vkCmdDraw-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDraw-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDraw-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDraw-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDraw-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDraw-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`viewMask` equal to VkRenderingInfo::`viewMask`
- VUID-vkCmdDraw-colorAttachmentCount-06179
  If the `dynamicRenderingUnusedAttachments` feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`colorAttachmentCount` equal to VkRenderingInfo::`colorAttachmentCount`
- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08910
  If the `dynamicRenderingUnusedAttachments` feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::`colorAttachmentCount` greater than `0`, then each element of the VkRenderingInfo::`pColorAttachments` array with an `imageView` not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::`pColorAttachmentFormats` used to create the bound graphics pipeline

- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDraw-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdDraw-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance
- VUID-vkCmdDraw-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle

in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount

- VUID-vkCmdDraw-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the

dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED

- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDraw-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDraw-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdDraw-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline
- VUID-vkCmdDraw-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDraw-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDraw-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass

equal to VK_NULL_HANDLE

- VUID-vkCmdDraw-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDraw-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDraw-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDraw-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdDraw-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDraw-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the

current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDraw-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDraw-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDraw-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior

to this drawing command

- VUID-vkCmdDraw-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDraw-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines

features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDraw-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDraw-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT
- VUID-vkCmdDraw-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdDraw-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in

the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline

- VUID-vkCmdDraw-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT

- VUID-vkCmdDraw-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments

- VUID-vkCmdDraw-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass

- VUID-vkCmdDraw-rasterizerDiscardEnable-09417
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass

- VUID-vkCmdDraw-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled

- VUID-vkCmdDraw-rasterizerDiscardEnable-09418
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and

attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled

- VUID-vkCmdDraw-firstAttachment-07478
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDraw-rasterizerDiscardEnable-09419
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDraw-firstAttachment-07479
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendAdvancedEXT calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDraw-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT and VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed advancedBlendMaxColorAttachments
- VUID-vkCmdDraw-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, and the bound graphics pipeline was created with VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the rasterizationStream to zero
- VUID-vkCmdDraw-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples member of the VkPipelineMultisampleStateCreateInfo structure the

bound graphics pipeline has been created with

- VUID-vkCmdDraw-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDraw-sampleLocationsEnable-07484
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDraw-sampleLocationsEnable-07485
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDraw-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDraw-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-vkCmdDraw-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the

VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT

- VUID-vkCmdDraw-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDraw-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDraw-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDraw-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled
- VUID-vkCmdDraw-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-vkCmdDraw-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the

VK_DYNAMIC_STATE_LINE_STIPPLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE

- VUID-vkCmdDraw-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdDraw-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT
- VUID-vkCmdDraw-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDraw-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDraw-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-vkCmdDraw-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdDraw-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdDraw-None-08699

If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound

- VUID-vkCmdDraw-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdDraw-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDraw-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE
- VUID-vkCmdDraw-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to vkCmdSetStencilWriteMask **must** be 0
- VUID-vkCmdDraw-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and the format of any color attachment is VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, the corresponding element of the pColorWriteMasks parameter of vkCmdSetColorWriteMaskEXT **must** either include all of VK_COLOR_COMPONENT_R_BIT, VK_COLOR_COMPONENT_G_BIT, and VK_COLOR_COMPONENT_B_BIT, or none of them
- VUID-vkCmdDraw-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of Location for any output attachment statically used in the Fragment Execution Model executed by this command **must** be less than maxFragmentDualSrcAttachments
- VUID-vkCmdDraw-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::pColorAttachmentLocations set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdDraw-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR

- VUID-vkCmdDraw-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDraw-None-04008
  If the nullDescriptor feature is not enabled, all vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE
- VUID-vkCmdDraw-None-02721
  If the robustBufferAccess feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDraw-None-04914
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command
- VUID-vkCmdDraw-Input-07939
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription2EXT::location
- VUID-vkCmdDraw-Input-08734
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and either the legacyVertexAttributes feature is not enabled or the SPIR-V Type associated with a given Input variable of the corresponding Location in the Vertex Execution Model OpEntryPoint is 64-bit, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription2EXT::format
- VUID-vkCmdDraw-format-08936
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit
- VUID-vkCmdDraw-format-08937
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component
- VUID-vkCmdDraw-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound

graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format

- VUID-vkCmdDraw-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDraw-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE
- VUID-vkCmdDraw-pNext-09461
  If the bound graphics pipeline state was created with VkPipelineVertexInputDivisorStateCreateInfoKHR in the pNext chain of VkGraphicsPipelineCreateInfo::pVertexInputState, any member of VkPipelineVertexInputDivisorStateCreateInfoKHR::pVertexBindingDivisors has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-vkCmdDraw-None-09462
  If shader objects are used for drawing or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled, any member of the pVertexBindingDescriptions parameter to the vkCmdSetVertexInputEXT call that sets this dynamic state has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0

Valid Usage (Implicit)

- VUID-vkCmdDraw-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDraw-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdDraw-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdDraw-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDraw-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

To record an indexed draw, call:

```
// Provided by VK_VERSION_1_0
void vkCmdDrawIndexed(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    indexCount,
    uint32_t                                    instanceCount,
    uint32_t                                    firstIndex,
    int32_t                                     vertexOffset,
    uint32_t                                    firstInstance);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `indexCount` is the number of vertices to draw.
- `instanceCount` is the number of instances to draw.
- `firstIndex` is the base index within the index buffer.
- `vertexOffset` is the value added to the vertex index before indexing into the vertex buffer.
- `firstInstance` is the instance ID of the first instance to draw.

When the command is executed, primitives are assembled using the current primitive topology and `indexCount` vertices whose indices are retrieved from the index buffer. The index buffer is treated as an array of tightly packed unsigned integers of size defined by the vkCmdBindIndexBuffer2KHR::`indexType` or the vkCmdBindIndexBuffer::`indexType` parameter with which the buffer was bound.

The first vertex index is at an offset of `firstIndex` × `indexSize` + `offset` within the bound index buffer, where `offset` is the offset specified by `vkCmdBindIndexBuffer` or `vkCmdBindIndexBuffer2KHR`, and `indexSize` is the byte size of the type specified by `indexType`. Subsequent index values are retrieved from consecutive locations in the index buffer. Indices are first compared to the primitive restart value, then zero extended to 32 bits (if the `indexType` is `VK_INDEX_TYPE_UINT8_KHR` or `VK_INDEX_TYPE_UINT16`) and have `vertexOffset` added to them, before being supplied as the

vertexIndex value.

The primitives are drawn instanceCount times with instanceIndex starting with firstInstance and increasing sequentially for each instance. The assembled primitives execute the bound graphics pipeline.

Valid Usage

- VUID-vkCmdDrawIndexed-magFilter-04553
  If a VkSampler created with magFilter or minFilter equal to VK_FILTER_LINEAR, and compareEnable equal to VK_FALSE is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
- VUID-vkCmdDrawIndexed-mipmapMode-04770
  If a VkSampler created with mipmapMode equal to VK_SAMPLER_MIPMAP_MODE_LINEAR, and compareEnable equal to VK_FALSE is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
- VUID-vkCmdDrawIndexed-unnormalizedCoordinates-09635
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's levelCount and layerCount **must** be 1
- VUID-vkCmdDrawIndexed-unnormalizedCoordinates-09636
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's viewType **must** be VK_IMAGE_VIEW_TYPE_1D or VK_IMAGE_VIEW_TYPE_2D
- VUID-vkCmdDrawIndexed-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdDrawIndexed-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdDrawIndexed-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdDrawIndexed-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdDrawIndexed-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of

VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY

- VUID-vkCmdDrawIndexed-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndexed-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndexed-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexed-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexed-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexed-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexed-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndexed-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndexed-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in

the descriptor sets used by this command

- VUID-vkCmdDrawIndexed-maintenance4-08602
  If the `maintenance4` feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndexed-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDrawIndexed-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDrawIndexed-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDrawIndexed-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdDrawIndexed-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDrawIndexed-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDrawIndexed-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDrawIndexed-uniformBuffers-06935

If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point

- VUID-vkCmdDrawIndexed-None-08612
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexed-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexed-None-08613
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexed-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with OpImageSample* or OpImageSparseSample* instructions
- VUID-vkCmdDrawIndexed-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the ConstOffset and Offset operands
- VUID-vkCmdDrawIndexed-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's viewType **must** match the Dim operand of the OpTypeImage as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDrawIndexed-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's format and the Sampled Type operand of the OpTypeImage **must** match
- VUID-vkCmdDrawIndexed-OpImageWrite-08795
  If a VkImageView created with a format other than VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDrawIndexed-OpImageWrite-08796
  If a VkImageView created with the format VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have four components
- VUID-vkCmdDrawIndexed-OpImageWrite-04469
  If a VkBufferView is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as

the buffer view's format

- VUID-vkCmdDrawIndexed-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawIndexed-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawIndexed-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawIndexed-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawIndexed-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndexed-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndexed-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDrawIndexed-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDrawIndexed-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndexed-subpass-02685
  The subpass index of the current render pass **must** be equal to the subpass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndexed-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set

- VUID-vkCmdDrawIndexed-OpTypeImage-07468
  If any shader executed by this pipeline accesses an `OpTypeImage` variable with a `Dim` operand of `SubpassData`, it **must** be decorated with an `InputAttachmentIndex` that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDrawIndexed-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' `pInputAttachments` [`InputAttachmentIndex`] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility
- VUID-vkCmdDrawIndexed-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a `InputAttachmentIndex` referenced by VkRenderingInputAttachmentIndexInfoKHR, or no `InputAttachmentIndex` if VkRenderingInputAttachmentIndexInfoKHR:`pDepthInputAttachmentIndex` or VkRenderingInputAttachmentIndexInfoKHR:`pStencilInputAttachmentIndex` are `NULL`, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdDrawIndexed-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an `InputAttachmentIndex` if both VkRenderingInputAttachmentIndexInfoKHR:`pDepthInputAttachmentIndex` and VkRenderingInputAttachmentIndexInfoKHR:`pStencilInputAttachmentIndex` are non-`NULL`
- VUID-vkCmdDrawIndexed-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an `InputAttachmentIndex`, the `InputAttachmentIndex` **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndexed-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDrawIndexed-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` image layout, and either:
  - the `VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT` is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included `VK_IMAGE_ASPECT_COLOR_BIT` and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with `VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT`

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawIndexed-None-09001

If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:

  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndexed-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:

  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndexed-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdDrawIndexed-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDrawIndexed-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdDrawIndexed-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back writeMask are not zero, and stencil test is enabled, all stencil ops **must** be VK_STENCIL_OP_KEEP
- VUID-vkCmdDrawIndexed-None-07831
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VIEWPORT

dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexed-None-07832
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SCISSOR dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07833
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_WIDTH dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08619
  If a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent

call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexed-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties ::maxMultiviewInstanceIndex
- VUID-vkCmdDrawIndexed-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with

VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set

- VUID-vkCmdDrawIndexed-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdDrawIndexed-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdDrawIndexed-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndexed-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndexed-None-08876

If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR

- VUID-vkCmdDrawIndexed-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndexed-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndexed-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndexed-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndexed-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndexed-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndexed-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`viewMask` equal to VkRenderingInfo::`viewMask`
- VUID-vkCmdDrawIndexed-colorAttachmentCount-06179
  If the `dynamicRenderingUnusedAttachments` feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`colorAttachmentCount` equal to VkRenderingInfo::`colorAttachmentCount`
- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08910
  If the `dynamicRenderingUnusedAttachments` feature is not enabled, and the current render

pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline

- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexed-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdDrawIndexed-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than

or equal to the number of color attachments in the current render pass instance

- VUID-vkCmdDrawIndexed-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount
- VUID-vkCmdDrawIndexed-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo

::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo ::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo ::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo ::pStencilAttachment->imageView
- VUID-vkCmdDrawIndexed-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo ::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexed-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDrawIndexed-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdDrawIndexed-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline
- VUID-vkCmdDrawIndexed-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndexed-multisampledRenderToSingleSampled-07287

If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView

- VUID-vkCmdDrawIndexed-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass equal to VK_NULL_HANDLE
- VUID-vkCmdDrawIndexed-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexed-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexed-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexed-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdDrawIndexed-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDrawIndexed-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of

rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexed-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndexed-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndexed-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexed-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created

with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexed-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07637

If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexed-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08670

If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexed-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT
- VUID-vkCmdDrawIndexed-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does

not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass

- VUID-vkCmdDrawIndexed-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline
- VUID-vkCmdDrawIndexed-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexed-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndexed-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexed-rasterizerDiscardEnable-09417
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexed-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is

enabled

- VUID-vkCmdDrawIndexed-rasterizerDiscardEnable-09418
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndexed-firstAttachment-07478
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexed-rasterizerDiscardEnable-09419
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexed-firstAttachment-07479
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendAdvancedEXT calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndexed-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT and VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed advancedBlendMaxColorAttachments
- VUID-vkCmdDrawIndexed-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, and the bound graphics pipeline was created with VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the rasterizationStream to zero

- VUID-vkCmdDrawIndexed-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples member of the VkPipelineMultisampleStateCreateInfo structure the bound graphics pipeline has been created with
- VUID-vkCmdDrawIndexed-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexed-sampleLocationsEnable-07484
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndexed-sampleLocationsEnable-07485
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndexed-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndexed-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound

graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample

- VUID-vkCmdDrawIndexed-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexed-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexed-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexed-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDrawIndexed-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled
- VUID-vkCmdDrawIndexed-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or

VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled

- VUID-vkCmdDrawIndexed-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE
- VUID-vkCmdDrawIndexed-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdDrawIndexed-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT
- VUID-vkCmdDrawIndexed-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDrawIndexed-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDrawIndexed-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-vkCmdDrawIndexed-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT

- VUID-vkCmdDrawIndexed-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdDrawIndexed-None-08699
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound
- VUID-vkCmdDrawIndexed-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdDrawIndexed-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDrawIndexed-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndexed-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to vkCmdSetStencilWriteMask **must** be 0
- VUID-vkCmdDrawIndexed-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and the format of any color attachment is VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, the corresponding element of the pColorWriteMasks parameter of vkCmdSetColorWriteMaskEXT **must** either include all of VK_COLOR_COMPONENT_R_BIT, VK_COLOR_COMPONENT_G_BIT, and VK_COLOR_COMPONENT_B_BIT, or none of them
- VUID-vkCmdDrawIndexed-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of Location for any output attachment statically used in the Fragment Execution Model executed by this command **must** be less than maxFragmentDualSrcAttachments
- VUID-vkCmdDrawIndexed-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::pColorAttachmentLocations set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the

corresponding element in the bound pipeline

- VUID-vkCmdDrawIndexed-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndexed-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDrawIndexed-None-04008
  If the `nullDescriptor` feature is not enabled, all vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE
- VUID-vkCmdDrawIndexed-None-02721
  If the `robustBufferAccess` feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDrawIndexed-None-04914
  If there is a shader object bound to the `VK_SHADER_STAGE_VERTEX_BIT` stage or the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command
- VUID-vkCmdDrawIndexed-Input-07939
  If there is a shader object bound to the `VK_SHADER_STAGE_VERTEX_BIT` stage or the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic state enabled then all variables with the `Input` storage class decorated with `Location` in the `Vertex` `Execution` `Model` `OpEntryPoint` **must** contain a location in VkVertexInputAttributeDescription2EXT::`location`
- VUID-vkCmdDrawIndexed-Input-08734
  If there is a shader object bound to the `VK_SHADER_STAGE_VERTEX_BIT` stage or the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic state enabled and either the `legacyVertexAttributes` feature is not enabled or the SPIR-V Type associated with a given `Input` variable of the corresponding `Location` in the `Vertex` `Execution` `Model` `OpEntryPoint` is 64-bit, then the numeric type associated with all `Input` variables of the corresponding `Location` in the `Vertex` `Execution` `Model` `OpEntryPoint` **must** be the same as VkVertexInputAttributeDescription2EXT::`format`
- VUID-vkCmdDrawIndexed-format-08936
  If there is a shader object bound to the `VK_SHADER_STAGE_VERTEX_BIT` stage or the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic state enabled and VkVertexInputAttributeDescription2EXT::`format` has a 64-bit component, then the scalar width associated with all `Input` variables of the corresponding `Location` in the `Vertex` `Execution` `Model` `OpEntryPoint` **must** be 64-bit
- VUID-vkCmdDrawIndexed-format-08937

If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component

- VUID-vkCmdDrawIndexed-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format
- VUID-vkCmdDrawIndexed-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexed-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndexed-None-07312
  If the maintenance6 feature is not enabled, a valid index buffer **must** be bound
- VUID-vkCmdDrawIndexed-pNext-09461
  If the bound graphics pipeline state was created with VkPipelineVertexInputDivisorStateCreateInfoKHR in the pNext chain of VkGraphicsPipelineCreateInfo::pVertexInputState, any member of VkPipelineVertexInputDivisorStateCreateInfoKHR::pVertexBindingDivisors has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-vkCmdDrawIndexed-None-09462
  If shader objects are used for drawing or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled, any member of the pVertexBindingDescriptions parameter to the vkCmdSetVertexInputEXT call that sets this dynamic state has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-vkCmdDrawIndexed-robustBufferAccess2-08798
  If the robustBufferAccess2 feature is not enabled, (indexSize × (firstIndex + indexCount) + offset) **must** be less than or equal to the size of the bound index buffer, with indexSize being based on the type specified by indexType, where the index buffer, indexType, and offset are specified via vkCmdBindIndexBuffer or vkCmdBindIndexBuffer2KHR. If vkCmdBindIndexBuffer2KHR is used to bind the index buffer, the size of the bound index

buffer is vkCmdBindIndexBuffer2KHR::size

Valid Usage (Implicit)

- VUID-vkCmdDrawIndexed-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDrawIndexed-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdDrawIndexed-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdDrawIndexed-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDrawIndexed-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

To record an ordered sequence of draws which have no state changes between them, call:

```
// Provided by VK_EXT_multi_draw
void vkCmdDrawMultiEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    drawCount,
    const VkMultiDrawInfoEXT*                   pVertexInfo,
    uint32_t                                    instanceCount,
    uint32_t                                    firstInstance,
    uint32_t                                    stride);
```

- commandBuffer is the command buffer into which the command is recorded.

- `drawCount` is the number of draws to execute, and **can** be zero.
- `pVertexInfo` is a pointer to an array of VkMultiDrawInfoEXT with vertex information to be drawn.
- `instanceCount` is the number of instances per draw.
- `firstInstance` is the instance ID of the first instance in each draw.
- `stride` is the byte stride between consecutive elements of `pVertexInfo`.

The number of draws recorded is `drawCount`, with each draw reading, sequentially, a `firstVertex` and a `vertexCount` from `pVertexInfo`. For each recorded draw, primitives are assembled as for vkCmdDraw, and drawn `instanceCount` times with `instanceIndex` starting with `firstInstance` and sequentially for each instance.

Valid Usage

- VUID-vkCmdDrawMultiEXT-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawMultiEXT-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawMultiEXT-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDrawMultiEXT-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`
- VUID-vkCmdDrawMultiEXT-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain `VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT`
- VUID-vkCmdDrawMultiEXT-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT`
- VUID-vkCmdDrawMultiEXT-None-07888
  If a `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER` descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT`

- VUID-vkCmdDrawMultiEXT-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdDrawMultiEXT-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdDrawMultiEXT-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawMultiEXT-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawMultiEXT-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawMultiEXT-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawMultiEXT-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawMultiEXT-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawMultiEXT-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawMultiEXT-None-08601
  For each push constant that is statically used by a bound shader, a push constant value

must have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility

- VUID-vkCmdDrawMultiEXT-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDrawMultiEXT-maintenance4-08602
  If the `maintenance4` feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawMultiEXT-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDrawMultiEXT-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDrawMultiEXT-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDrawMultiEXT-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdDrawMultiEXT-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDrawMultiEXT-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage

- VUID-vkCmdDrawMultiEXT-None-08607
  If the shaderObject feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDrawMultiEXT-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawMultiEXT-None-08612
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawMultiEXT-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawMultiEXT-None-08613
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawMultiEXT-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with OpImageSample* or OpImageSparseSample* instructions
- VUID-vkCmdDrawMultiEXT-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the ConstOffset and Offset operands
- VUID-vkCmdDrawMultiEXT-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's viewType **must** match the Dim operand of the OpTypeImage as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDrawMultiEXT-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's format and the Sampled Type operand of the OpTypeImage **must** match
- VUID-vkCmdDrawMultiEXT-OpImageWrite-08795
  If a VkImageView created with a format other than VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDrawMultiEXT-OpImageWrite-08796

If a VkImageView created with the format VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have four components

- VUID-vkCmdDrawMultiEXT-OpImageWrite-04469
  If a VkBufferView is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDrawMultiEXT-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawMultiEXT-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawMultiEXT-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawMultiEXT-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawMultiEXT-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawMultiEXT-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawMultiEXT-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDrawMultiEXT-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDrawMultiEXT-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawMultiEXT-subpass-02685

The subpass index of the current render pass **must** be equal to the `subpass` member of the `VkGraphicsPipelineCreateInfo` structure specified when creating the `VkPipeline` bound to `VK_PIPELINE_BIND_POINT_GRAPHICS`

- VUID-vkCmdDrawMultiEXT-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdDrawMultiEXT-OpTypeImage-07468
  If any shader executed by this pipeline accesses an `OpTypeImage` variable with a `Dim` operand of `SubpassData`, it **must** be decorated with an `InputAttachmentIndex` that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDrawMultiEXT-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' `pInputAttachments` [`InputAttachmentIndex`] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility
- VUID-vkCmdDrawMultiEXT-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a `InputAttachmentIndex` referenced by VkRenderingInputAttachmentIndexInfoKHR, or no `InputAttachmentIndex` if VkRenderingInputAttachmentIndexInfoKHR:`pDepthInputAttachmentIndex` or VkRenderingInputAttachmentIndexInfoKHR:`pStencilInputAttachmentIndex` are `NULL`, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdDrawMultiEXT-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an `InputAttachmentIndex` if both VkRenderingInputAttachmentIndexInfoKHR:`pDepthInputAttachmentIndex` and VkRenderingInputAttachmentIndexInfoKHR:`pStencilInputAttachmentIndex` are non-`NULL`
- VUID-vkCmdDrawMultiEXT-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an `InputAttachmentIndex`, the `InputAttachmentIndex` **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawMultiEXT-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDrawMultiEXT-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` image layout, and either:
  - the `VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT` is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included `VK_IMAGE_ASPECT_COLOR_BIT` and

- there is no bound graphics pipeline or
- the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawMultiEXT-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawMultiEXT-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawMultiEXT-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdDrawMultiEXT-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDrawMultiEXT-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled

- VUID-vkCmdDrawMultiEXT-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back `writeMask` are not zero, and stencil test is enabled, all stencil ops **must** be `VK_STENCIL_OP_KEEP`
- VUID-vkCmdDrawMultiEXT-None-07831
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VIEWPORT` dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07832
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SCISSOR` dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07833
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_LINE_WIDTH` dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set `polygonMode` to `VK_POLYGON_MODE_LINE`, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set `primitiveTopology` to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08619
  If a shader object that outputs line primitives is bound to the `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT` or `VK_SHADER_STAGE_GEOMETRY_BIT` stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the `VK_DYNAMIC_STATE_DEPTH_BIAS` dynamic state enabled, the current value of `rasterizerDiscardEnable` is `VK_FALSE`, and the current value of `depthBiasEnable` is `VK_TRUE`, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07835

If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiEXT-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiEXT-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties::maxMultiviewInstanceIndex
- VUID-vkCmdDrawMultiEXT-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawMultiEXT-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdDrawMultiEXT-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdDrawMultiEXT-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawMultiEXT-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set

rasterizerDiscardEnable to VK_FALSE,

then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments

- VUID-vkCmdDrawMultiEXT-None-08876
  If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdDrawMultiEXT-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawMultiEXT-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawMultiEXT-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawMultiEXT-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawMultiEXT-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawMultiEXT-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawMultiEXT-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo ::viewMask equal to VkRenderingInfo::viewMask
- VUID-vkCmdDrawMultiEXT-colorAttachmentCount-06179

If the dynamicRenderingUnusedAttachments feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::colorAttachmentCount equal to VkRenderingInfo::colorAttachmentCount

- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08910
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline
- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline

- VUID-vkCmdDrawMultiEXT-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance
- VUID-vkCmdDrawMultiEXT-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount
- VUID-vkCmdDrawMultiEXT-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of

VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView

- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawMultiEXT-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDrawMultiEXT-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdDrawMultiEXT-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of

rasterizationSamples for the bound graphics pipeline

- VUID-vkCmdDrawMultiEXT-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawMultiEXT-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawMultiEXT-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass equal to VK_NULL_HANDLE
- VUID-vkCmdDrawMultiEXT-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiEXT-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdDrawMultiEXT-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDrawMultiEXT-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiEXT-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawMultiEXT-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawMultiEXT-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiEXT-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most

recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiEXT-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07636

If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiEXT-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most

recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiEXT-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the

VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT

- VUID-vkCmdDrawMultiEXT-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdDrawMultiEXT-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline
- VUID-vkCmdDrawMultiEXT-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiEXT-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawMultiEXT-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawMultiEXT-rasterizerDiscardEnable-09417
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass

- VUID-vkCmdDrawMultiEXT-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawMultiEXT-rasterizerDiscardEnable-09418
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawMultiEXT-firstAttachment-07478
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawMultiEXT-rasterizerDiscardEnable-09419
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawMultiEXT-firstAttachment-07479
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendAdvancedEXT calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawMultiEXT-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT and VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed

advancedBlendMaxColorAttachments

- VUID-vkCmdDrawMultiEXT-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, and the bound graphics pipeline was created with VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the rasterizationStream to zero
- VUID-vkCmdDrawMultiEXT-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples member of the VkPipelineMultisampleStateCreateInfo structure the bound graphics pipeline has been created with
- VUID-vkCmdDrawMultiEXT-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiEXT-sampleLocationsEnable-07484
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawMultiEXT-sampleLocationsEnable-07485
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawMultiEXT-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the

sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples

- VUID-vkCmdDrawMultiEXT-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-vkCmdDrawMultiEXT-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiEXT-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiEXT-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiEXT-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDrawMultiEXT-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or

VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled

- VUID-vkCmdDrawMultiEXT-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-vkCmdDrawMultiEXT-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE
- VUID-vkCmdDrawMultiEXT-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdDrawMultiEXT-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT
- VUID-vkCmdDrawMultiEXT-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDrawMultiEXT-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDrawMultiEXT-None-08687

If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT

- VUID-vkCmdDrawMultiEXT-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdDrawMultiEXT-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdDrawMultiEXT-None-08699
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound
- VUID-vkCmdDrawMultiEXT-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdDrawMultiEXT-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDrawMultiEXT-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawMultiEXT-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to vkCmdSetStencilWriteMask **must** be 0
- VUID-vkCmdDrawMultiEXT-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and the format of any color attachment is VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, the corresponding element of the pColorWriteMasks parameter of vkCmdSetColorWriteMaskEXT **must** either include all of VK_COLOR_COMPONENT_R_BIT, VK_COLOR_COMPONENT_G_BIT, and VK_COLOR_COMPONENT_B_BIT, or none of them
- VUID-vkCmdDrawMultiEXT-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of Location

for any output attachment statically used in the Fragment Execution Model executed by this command **must** be less than maxFragmentDualSrcAttachments

- VUID-vkCmdDrawMultiEXT-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::pColorAttachmentLocations set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdDrawMultiEXT-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawMultiEXT-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDrawMultiEXT-None-04008
  If the nullDescriptor feature is not enabled, all vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE
- VUID-vkCmdDrawMultiEXT-None-02721
  If the robustBufferAccess feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDrawMultiEXT-None-04914
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command
- VUID-vkCmdDrawMultiEXT-Input-07939
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription2EXT::location
- VUID-vkCmdDrawMultiEXT-Input-08734
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and either the legacyVertexAttributes feature is not enabled or the SPIR-V Type associated with a given Input variable of the corresponding Location in the Vertex Execution Model OpEntryPoint is 64-bit, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription2EXT::format

- VUID-vkCmdDrawMultiEXT-format-08936
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit
- VUID-vkCmdDrawMultiEXT-format-08937
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component
- VUID-vkCmdDrawMultiEXT-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format
- VUID-vkCmdDrawMultiEXT-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiEXT-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawMultiEXT-pNext-09461
  If the bound graphics pipeline state was created with VkPipelineVertexInputDivisorStateCreateInfoKHR in the pNext chain of VkGraphicsPipelineCreateInfo::pVertexInputState, any member of VkPipelineVertexInputDivisorStateCreateInfoKHR::pVertexBindingDivisors has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-vkCmdDrawMultiEXT-None-09462
  If shader objects are used for drawing or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled, any member of the pVertexBindingDescriptions parameter to the vkCmdSetVertexInputEXT call that sets this dynamic state has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-vkCmdDrawMultiEXT-None-04933

The `multiDraw` feature **must** be enabled

- VUID-vkCmdDrawMultiEXT-drawCount-04934
  `drawCount` **must** be less than `VkPhysicalDeviceMultiDrawPropertiesEXT`::`maxMultiDrawCount`
- VUID-vkCmdDrawMultiEXT-drawCount-04935
  If `drawCount` is greater than zero, `pVertexInfo` **must** be a valid pointer to memory containing one or more valid instances of VkMultiDrawInfoEXT structures
- VUID-vkCmdDrawMultiEXT-drawCount-09628
  If `drawCount` is greater than `1`, `stride` **must** be a multiple of `4` and **must** be greater than or equal to `sizeof`(`VkMultiDrawInfoEXT`)

Valid Usage (Implicit)

- VUID-vkCmdDrawMultiEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDrawMultiEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdDrawMultiEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdDrawMultiEXT-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDrawMultiEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

To record an ordered sequence of indexed draws which have no state changes between them, call:

```
// Provided by VK_EXT_multi_draw
void vkCmdDrawMultiIndexedEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    drawCount,
    const VkMultiDrawIndexedInfoEXT*            pIndexInfo,
    uint32_t                                    instanceCount,
    uint32_t                                    firstInstance,
    uint32_t                                    stride,
    const int32_t*                              pVertexOffset);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `drawCount` is the number of draws to execute, and **can** be zero.
- `pIndexInfo` is a pointer to an array of VkMultiDrawIndexedInfoEXT with index information to be drawn.
- `instanceCount` is the number of instances per draw.
- `firstInstance` is the instance ID of the first instance in each draw.
- `stride` is the byte stride between consecutive elements of `pIndexInfo`.
- `pVertexOffset` is `NULL` or a pointer to the value added to the vertex index before indexing into the vertex buffer. When specified, `VkMultiDrawIndexedInfoEXT::offset` is ignored.

The number of draws recorded is `drawCount`, with each draw reading, sequentially, a `firstIndex` and an `indexCount` from `pIndexInfo`. For each recorded draw, primitives are assembled as for vkCmdDrawIndexed, and drawn `instanceCount` times with `instanceIndex` starting with `firstInstance` and sequentially for each instance. If `pVertexOffset` is `NULL`, a `vertexOffset` is also read from `pIndexInfo`, otherwise the value from dereferencing `pVertexOffset` is used.

Valid Usage

- VUID-vkCmdDrawMultiIndexedEXT-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawMultiIndexedEXT-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawMultiIndexedEXT-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDrawMultiIndexedEXT-unnormalizedCoordinates-09636

If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's viewType **must** be VK_IMAGE_VIEW_TYPE_1D or VK_IMAGE_VIEW_TYPE_2D

- VUID-vkCmdDrawMultiIndexedEXT-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdDrawMultiIndexedEXT-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdDrawMultiIndexedEXT-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawMultiIndexedEXT-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawMultiIndexedEXT-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawMultiIndexedEXT-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawMultiIndexedEXT-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format

field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT

- VUID-vkCmdDrawMultiIndexedEXT-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawMultiIndexedEXT-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawMultiIndexedEXT-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDrawMultiIndexedEXT-maintenance4-08602
  If the maintenance4 feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawMultiIndexedEXT-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDrawMultiIndexedEXT-None-08606
  If the shaderObject feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDrawMultiIndexedEXT-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDrawMultiIndexedEXT-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler

must not be used to sample from any VkImage with a VkImageView of the type VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, VK_IMAGE_VIEW_TYPE_1D_ARRAY, VK_IMAGE_VIEW_TYPE_2D_ARRAY or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY, in any shader stage

- VUID-vkCmdDrawMultiIndexedEXT-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V OpImageSample* or OpImageSparseSample* instructions with ImplicitLod, Dref or Proj in their name, in any shader stage
- VUID-vkCmdDrawMultiIndexedEXT-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V OpImageSample* or OpImageSparseSample* instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDrawMultiIndexedEXT-None-08607
  If the shaderObject feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDrawMultiIndexedEXT-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawMultiIndexedEXT-None-08612
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawMultiIndexedEXT-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawMultiIndexedEXT-None-08613
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawMultiIndexedEXT-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with OpImageSample* or OpImageSparseSample* instructions

- VUID-vkCmdDrawMultiIndexedEXT-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdDrawMultiIndexedEXT-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDrawMultiIndexedEXT-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdDrawMultiIndexedEXT-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDrawMultiIndexedEXT-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdDrawMultiIndexedEXT-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDrawMultiIndexedEXT-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDrawMultiIndexedEXT-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDrawMultiIndexedEXT-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDrawMultiIndexedEXT-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDrawMultiIndexedEXT-sparseImageInt64Atomics-04474
  If the `sparseImageInt64Atomics` feature is not enabled, VkImage objects created with the `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command
- VUID-vkCmdDrawMultiIndexedEXT-sparseImageInt64Atomics-04475

If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command

- VUID-vkCmdDrawMultiIndexedEXT-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDrawMultiIndexedEXT-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDrawMultiIndexedEXT-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawMultiIndexedEXT-subpass-02685
  The subpass index of the current render pass **must** be equal to the subpass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawMultiIndexedEXT-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdDrawMultiIndexedEXT-OpTypeImage-07468
  If any shader executed by this pipeline accesses an OpTypeImage variable with a Dim operand of SubpassData, it **must** be decorated with an InputAttachmentIndex that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDrawMultiIndexedEXT-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' pInputAttachments [InputAttachmentIndex] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility
- VUID-vkCmdDrawMultiIndexedEXT-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a InputAttachmentIndex referenced by VkRenderingInputAttachmentIndexInfoKHR, or no InputAttachmentIndex if VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex or VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are NULL, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdDrawMultiIndexedEXT-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an InputAttachmentIndex if both VkRenderingInputAttachmentIndexInfoKHR :pDepthInputAttachmentIndex and VkRenderingInputAttachmentIndexInfoKHR :pStencilInputAttachmentIndex are non-NULL
- VUID-vkCmdDrawMultiIndexedEXT-InputAttachmentIndex-09597

If an input attachment view accessed in a dynamic render pass via a shader object has an InputAttachmentIndex, the InputAttachmentIndex **must** match an index in VkRenderingInputAttachmentIndexInfoKHR

- VUID-vkCmdDrawMultiIndexedEXT-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDrawMultiIndexedEXT-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_COLOR_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawMultiIndexedEXT-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawMultiIndexedEXT-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or

- the bound graphics pipeline was created with `VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT`

it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawMultiIndexedEXT-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdDrawMultiIndexedEXT-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDrawMultiIndexedEXT-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdDrawMultiIndexedEXT-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back `writeMask` are not zero, and stencil test is enabled, all stencil ops **must** be `VK_STENCIL_OP_KEEP`
- VUID-vkCmdDrawMultiIndexedEXT-None-07831
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VIEWPORT` dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07832
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SCISSOR` dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07833
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_LINE_WIDTH` dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set `polygonMode` to `VK_POLYGON_MODE_LINE`, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set `primitiveTopology` to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently

invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-None-08619
  If a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called

and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties::maxMultiviewInstanceIndex
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawMultiIndexedEXT-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE

- VUID-vkCmdDrawMultiIndexedEXT-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdDrawMultiIndexedEXT-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawMultiIndexedEXT-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments
- VUID-vkCmdDrawMultiIndexedEXT-None-08876
  If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdDrawMultiIndexedEXT-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawMultiIndexedEXT-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawMultiIndexedEXT-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawMultiIndexedEXT-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment

- VUID-vkCmdDrawMultiIndexedEXT-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawMultiIndexedEXT-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawMultiIndexedEXT-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo ::viewMask equal to VkRenderingInfo::viewMask
- VUID-vkCmdDrawMultiIndexedEXT-colorAttachmentCount-06179
  If the dynamicRenderingUnusedAttachments feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::colorAttachmentCount equal to VkRenderingInfo::colorAttachmentCount
- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08910
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline
- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiIndexedEXT-None-07749
  If the bound graphics pipeline state was created with the

VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdDrawMultiIndexedEXT-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance
- VUID-vkCmdDrawMultiIndexedEXT-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount
- VUID-vkCmdDrawMultiIndexedEXT-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the

current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawMultiIndexedEXT-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawMultiIndexedEXT-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDrawMultiIndexedEXT-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdDrawMultiIndexedEXT-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline
- VUID-vkCmdDrawMultiIndexedEXT-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawMultiIndexedEXT-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawMultiIndexedEXT-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass equal to VK_NULL_HANDLE
- VUID-vkCmdDrawMultiIndexedEXT-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiIndexedEXT-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawMultiIndexedEXT-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the

VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawMultiIndexedEXT-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdDrawMultiIndexedEXT-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDrawMultiIndexedEXT-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-alphaToCoverageEnable-08919

If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0

- VUID-vkCmdDrawMultiIndexedEXT-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawMultiIndexedEXT-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently

invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then

vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT
- VUID-vkCmdDrawMultiIndexedEXT-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdDrawMultiIndexedEXT-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline
- VUID-vkCmdDrawMultiIndexedEXT-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiIndexedEXT-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments

- VUID-vkCmdDrawMultiIndexedEXT-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawMultiIndexedEXT-rasterizerDiscardEnable-09417
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawMultiIndexedEXT-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawMultiIndexedEXT-rasterizerDiscardEnable-09418
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawMultiIndexedEXT-firstAttachment-07478
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawMultiIndexedEXT-rasterizerDiscardEnable-09419
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color

attachments in the current subpass

- VUID-vkCmdDrawMultiIndexedEXT-firstAttachment-07479
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendAdvancedEXT calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawMultiIndexedEXT-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT and VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed advancedBlendMaxColorAttachments
- VUID-vkCmdDrawMultiIndexedEXT-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, and the bound graphics pipeline was created with VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the rasterizationStream to zero
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples member of the VkPipelineMultisampleStateCreateInfo structure the bound graphics pipeline has been created with
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsEnable-07484
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsEnable-07485

If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples

- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawMultiIndexedEXT-sampleLocationsEnable-07938

If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT

- VUID-vkCmdDrawMultiIndexedEXT-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDrawMultiIndexedEXT-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled
- VUID-vkCmdDrawMultiIndexedEXT-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-vkCmdDrawMultiIndexedEXT-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE
- VUID-vkCmdDrawMultiIndexedEXT-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdDrawMultiIndexedEXT-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the

current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawMultiIndexedEXT-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdDrawMultiIndexedEXT-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdDrawMultiIndexedEXT-None-08699
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound
- VUID-vkCmdDrawMultiIndexedEXT-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdDrawMultiIndexedEXT-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDrawMultiIndexedEXT-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE

- VUID-vkCmdDrawMultiIndexedEXT-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to vkCmdSetStencilWriteMask **must** be 0
- VUID-vkCmdDrawMultiIndexedEXT-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and the format of any color attachment is VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, the corresponding element of the pColorWriteMasks parameter of vkCmdSetColorWriteMaskEXT **must** either include all of VK_COLOR_COMPONENT_R_BIT, VK_COLOR_COMPONENT_G_BIT, and VK_COLOR_COMPONENT_B_BIT, or none of them
- VUID-vkCmdDrawMultiIndexedEXT-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of Location for any output attachment statically used in the Fragment Execution Model executed by this command **must** be less than maxFragmentDualSrcAttachments
- VUID-vkCmdDrawMultiIndexedEXT-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::pColorAttachmentLocations set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdDrawMultiIndexedEXT-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawMultiIndexedEXT-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDrawMultiIndexedEXT-None-04008
  If the nullDescriptor feature is not enabled, all vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE
- VUID-vkCmdDrawMultiIndexedEXT-None-02721
  If the robustBufferAccess feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDrawMultiIndexedEXT-None-04914
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic

state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command

- VUID-vkCmdDrawMultiIndexedEXT-Input-07939
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription2EXT::location
- VUID-vkCmdDrawMultiIndexedEXT-Input-08734
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and either the legacyVertexAttributes feature is not enabled or the SPIR-V Type associated with a given Input variable of the corresponding Location in the Vertex Execution Model OpEntryPoint is 64-bit, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription2EXT::format
- VUID-vkCmdDrawMultiIndexedEXT-format-08936
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit
- VUID-vkCmdDrawMultiIndexedEXT-format-08937
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component
- VUID-vkCmdDrawMultiIndexedEXT-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format
- VUID-vkCmdDrawMultiIndexedEXT-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawMultiIndexedEXT-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE

- VUID-vkCmdDrawMultiIndexedEXT-None-07312
  If the maintenance6 feature is not enabled, a valid index buffer **must** be bound

- VUID-vkCmdDrawMultiIndexedEXT-pNext-09461
  If the bound graphics pipeline state was created with VkPipelineVertexInputDivisorStateCreateInfoKHR in the pNext chain of VkGraphicsPipelineCreateInfo::pVertexInputState, any member of VkPipelineVertexInputDivisorStateCreateInfoKHR::pVertexBindingDivisors has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR ::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0

- VUID-vkCmdDrawMultiIndexedEXT-None-09462
  If shader objects are used for drawing or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled, any member of the pVertexBindingDescriptions parameter to the vkCmdSetVertexInputEXT call that sets this dynamic state has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0

- VUID-vkCmdDrawMultiIndexedEXT-robustBufferAccess2-08798
  If the robustBufferAccess2 feature is not enabled, (indexSize × (firstIndex + indexCount) + offset) **must** be less than or equal to the size of the bound index buffer, with indexSize being based on the type specified by indexType, where the index buffer, indexType, and offset are specified via vkCmdBindIndexBuffer or vkCmdBindIndexBuffer2KHR. If vkCmdBindIndexBuffer2KHR is used to bind the index buffer, the size of the bound index buffer is vkCmdBindIndexBuffer2KHR::size

- VUID-vkCmdDrawMultiIndexedEXT-None-04937
  The multiDraw feature **must** be enabled

- VUID-vkCmdDrawMultiIndexedEXT-drawCount-04939
  drawCount **must** be less than VkPhysicalDeviceMultiDrawPropertiesEXT::maxMultiDrawCount

- VUID-vkCmdDrawMultiIndexedEXT-drawCount-04940
  If drawCount is greater than zero, pIndexInfo **must** be a valid pointer to memory containing one or more valid instances of VkMultiDrawIndexedInfoEXT structures

- VUID-vkCmdDrawMultiIndexedEXT-drawCount-09629
  If drawCount is greater than 1, stride **must** be a multiple of 4 and **must** be greater than or equal to sizeof(VkMultiDrawIndexedInfoEXT)

Valid Usage (Implicit)

- VUID-vkCmdDrawMultiIndexedEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle

- VUID-vkCmdDrawMultiIndexedEXT-pVertexOffset-parameter
  If pVertexOffset is not NULL, pVertexOffset **must** be a valid pointer to a valid int32_t value

- VUID-vkCmdDrawMultiIndexedEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state

- VUID-vkCmdDrawMultiIndexedEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdDrawMultiIndexedEXT-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDrawMultiIndexedEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

The VkMultiDrawInfoEXT structure is defined as:

```
// Provided by VK_EXT_multi_draw
typedef struct VkMultiDrawInfoEXT {
    uint32_t    firstVertex;
    uint32_t    vertexCount;
} VkMultiDrawInfoEXT;
```

- firstVertex is the first vertex to draw.
- vertexCount is the number of vertices to draw.

The members of VkMultiDrawInfoEXT have the same meaning as the firstVertex and vertexCount parameters in vkCmdDraw.

The VkMultiDrawIndexedInfoEXT structure is defined as:

```
// Provided by VK_EXT_multi_draw
typedef struct VkMultiDrawIndexedInfoEXT {
    uint32_t    firstIndex;
    uint32_t    indexCount;
    int32_t     vertexOffset;
} VkMultiDrawIndexedInfoEXT;
```

- `firstIndex` is the first index to draw.
- `indexCount` is the number of vertices to draw.
- `vertexOffset` is the value added to the vertex index before indexing into the vertex buffer for indexed multidraws.

The `firstIndex`, `indexCount`, and `vertexOffset` members of `VkMultiDrawIndexedInfoEXT` have the same meaning as the `firstIndex`, `indexCount`, and `vertexOffset` parameters, respectively, of vkCmdDrawIndexed.

To record a non-indexed indirect drawing command, call:

```
// Provided by VK_VERSION_1_0
void vkCmdDrawIndirect(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    buffer,
    VkDeviceSize                                offset,
    uint32_t                                    drawCount,
    uint32_t                                    stride);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `buffer` is the buffer containing draw parameters.
- `offset` is the byte offset into `buffer` where parameters begin.
- `drawCount` is the number of draws to execute, and **can** be zero.
- `stride` is the byte stride between successive sets of draw parameters.

`vkCmdDrawIndirect` behaves similarly to vkCmdDraw except that the parameters are read by the device from a buffer during execution. `drawCount` draws are executed by the command, with parameters taken from `buffer` starting at `offset` and increasing by `stride` bytes for each successive draw. The parameters of each draw are encoded in an array of VkDrawIndirectCommand structures. If `drawCount` is less than or equal to one, `stride` is ignored.

Valid Usage

- VUID-vkCmdDrawIndirect-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain

VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT

- VUID-vkCmdDrawIndirect-mipmapMode-04770
  If a VkSampler created with mipmapMode equal to VK_SAMPLER_MIPMAP_MODE_LINEAR, and compareEnable equal to VK_FALSE is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
- VUID-vkCmdDrawIndirect-unnormalizedCoordinates-09635
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's levelCount and layerCount **must** be 1
- VUID-vkCmdDrawIndirect-unnormalizedCoordinates-09636
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's viewType **must** be VK_IMAGE_VIEW_TYPE_1D or VK_IMAGE_VIEW_TYPE_2D
- VUID-vkCmdDrawIndirect-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdDrawIndirect-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdDrawIndirect-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdDrawIndirect-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdDrawIndirect-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdDrawIndirect-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndirect-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by

vkGetPhysicalDeviceImageFormatProperties2KHR

- VUID-vkCmdDrawIndirect-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the `OpTypeImage` is `Unknown`, the view's format features **must** contain `VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT`
- VUID-vkCmdDrawIndirect-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the `OpTypeImage` is `Unknown`, the view's format features **must** contain `VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT`
- VUID-vkCmdDrawIndirect-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the `OpTypeImage` is `Unknown`, the view's buffer features **must** contain `VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT`
- VUID-vkCmdDrawIndirect-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the `OpTypeImage` is `Unknown` then the view's buffer features **must** contain `VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT`
- VUID-vkCmdDrawIndirect-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndirect-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndirect-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDrawIndirect-maintenance4-08602
  If the `maintenance4` feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndirect-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDrawIndirect-None-08606

If the shaderObject feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command

- VUID-vkCmdDrawIndirect-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDrawIndirect-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, VK_IMAGE_VIEW_TYPE_1D_ARRAY, VK_IMAGE_VIEW_TYPE_2D_ARRAY or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY, in any shader stage
- VUID-vkCmdDrawIndirect-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V OpImageSample* or OpImageSparseSample* instructions with ImplicitLod, Dref or Proj in their name, in any shader stage
- VUID-vkCmdDrawIndirect-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V OpImageSample* or OpImageSparseSample* instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDrawIndirect-None-08607
  If the shaderObject feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDrawIndirect-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirect-None-08612
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirect-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the

descriptor set bound to the same pipeline bind point

- VUID-vkCmdDrawIndirect-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirect-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdDrawIndirect-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdDrawIndirect-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDrawIndirect-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdDrawIndirect-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDrawIndirect-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdDrawIndirect-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDrawIndirect-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDrawIndirect-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDrawIndirect-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction

must have a `Width` of 64

- VUID-vkCmdDrawIndirect-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDrawIndirect-sparseImageInt64Atomics-04474
  If the `sparseImageInt64Atomics` feature is not enabled, VkImage objects created with the `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command
- VUID-vkCmdDrawIndirect-sparseImageInt64Atomics-04475
  If the `sparseImageInt64Atomics` feature is not enabled, VkBuffer objects created with the `VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command
- VUID-vkCmdDrawIndirect-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDrawIndirect-None-09600
  If a descriptor with type equal to any of `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDrawIndirect-renderPass-02684
  The current render pass **must** be compatible with the `renderPass` member of the `VkGraphicsPipelineCreateInfo` structure specified when creating the `VkPipeline` bound to `VK_PIPELINE_BIND_POINT_GRAPHICS`
- VUID-vkCmdDrawIndirect-subpass-02685
  The subpass index of the current render pass **must** be equal to the `subpass` member of the `VkGraphicsPipelineCreateInfo` structure specified when creating the `VkPipeline` bound to `VK_PIPELINE_BIND_POINT_GRAPHICS`
- VUID-vkCmdDrawIndirect-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdDrawIndirect-OpTypeImage-07468
  If any shader executed by this pipeline accesses an `OpTypeImage` variable with a `Dim` operand of `SubpassData`, it **must** be decorated with an `InputAttachmentIndex` that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDrawIndirect-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' `pInputAttachments` [`InputAttachmentIndex`] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility
- VUID-vkCmdDrawIndirect-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a `InputAttachmentIndex`

referenced by VkRenderingInputAttachmentIndexInfoKHR, or no InputAttachmentIndex if VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex or VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are NULL, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo

- VUID-vkCmdDrawIndirect-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an InputAttachmentIndex if both VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex and VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are non-NULL
- VUID-vkCmdDrawIndirect-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an InputAttachmentIndex, the InputAttachmentIndex **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndirect-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDrawIndirect-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_COLOR_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawIndirect-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndirect-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawIndirect-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdDrawIndirect-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDrawIndirect-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdDrawIndirect-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back writeMask are not zero, and stencil test is enabled, all stencil ops **must** be VK_STENCIL_OP_KEEP
- VUID-vkCmdDrawIndirect-None-07831
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VIEWPORT dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07832
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SCISSOR dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07833
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_WIDTH dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set

rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirect-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08619
  If a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which

was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirect-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties::maxMultiviewInstanceIndex
- VUID-vkCmdDrawIndirect-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndirect-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirect-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdDrawIndirect-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdDrawIndirect-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndirect-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndirect-None-08876
  If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdDrawIndirect-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndirect-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment

- VUID-vkCmdDrawIndirect-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndirect-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndirect-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndirect-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndirect-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::viewMask equal to VkRenderingInfo::viewMask
- VUID-vkCmdDrawIndirect-colorAttachmentCount-06179
  If the dynamicRenderingUnusedAttachments feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::colorAttachmentCount equal to VkRenderingInfo::colorAttachmentCount
- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08910
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline
- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirect-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdDrawIndirect-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance
- VUID-vkCmdDrawIndirect-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount
- VUID-vkCmdDrawIndirect-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior

to this drawing command

- VUID-vkCmdDrawIndirect-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo

::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo ::pStencilAttachment->imageView

- VUID-vkCmdDrawIndirect-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo ::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirect-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDrawIndirect-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdDrawIndirect-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline
- VUID-vkCmdDrawIndirect-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndirect-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndirect-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass equal to VK_NULL_HANDLE
- VUID-vkCmdDrawIndirect-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo

::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawIndirect-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirect-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirect-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdDrawIndirect-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDrawIndirect-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then

vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirect-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndirect-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndirect-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08657

If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirect-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirect-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirect-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior

to this drawing command

- VUID-vkCmdDrawIndirect-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT
- VUID-vkCmdDrawIndirect-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdDrawIndirect-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline
- VUID-vkCmdDrawIndirect-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT

must be greater or equal to the `rasterizationSamples` parameter in the last call to vkCmdSetRasterizationSamplesEXT

- VUID-vkCmdDrawIndirect-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT` state enabled, and neither the `VK_AMD_mixed_attachment_samples` nor the `VK_NV_framebuffer_mixed_samples` extensions are enabled, then the `rasterizationSamples` in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndirect-firstAttachment-07476
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT` dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendEnableEXT` calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirect-rasterizerDiscardEnable-09417
  If a shader object is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendEnableEXT` calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirect-firstAttachment-07477
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT` dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendEquationEXT` calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndirect-rasterizerDiscardEnable-09418
  If a shader object is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE` and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendEquationEXT` calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndirect-firstAttachment-07478
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT` dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer

prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorWriteMaskEXT` calls **must** specify the color write mask for all active color attachments in the current subpass

- VUID-vkCmdDrawIndirect-rasterizerDiscardEnable-09419
  If a shader object is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorWriteMaskEXT` calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirect-firstAttachment-07479
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendAdvancedEXT` calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndirect-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` and `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT` dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed `advancedBlendMaxColorAttachments`
- VUID-vkCmdDrawIndirect-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the `primitivesGeneratedQueryWithNonZeroStreams` feature is not enabled and the `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT` query is active, and the bound graphics pipeline was created with `VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT` state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the `rasterizationStream` to zero
- VUID-vkCmdDrawIndirect-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` state enabled and the `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT` state disabled, and the current value of `sampleLocationsEnable` is `VK_TRUE`, then the `sampleLocationsPerPixel` member of `pSampleLocationsInfo` in the last call to vkCmdSetSampleLocationsEXT **must** equal the `rasterizationSamples` member of the VkPipelineMultisampleStateCreateInfo structure the bound graphics pipeline has been created with
- VUID-vkCmdDrawIndirect-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` state enabled and the `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT` state enabled, and the current value of `sampleLocationsEnable` is `VK_TRUE`, then the `sampleLocationsPerPixel` member of `pSampleLocationsInfo` in the last call to vkCmdSetSampleLocationsEXT **must** equal the

rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT

- VUID-vkCmdDrawIndirect-sampleLocationsEnable-07484
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndirect-sampleLocationsEnable-07485
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndirect-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndirect-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-vkCmdDrawIndirect-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT

- VUID-vkCmdDrawIndirect-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirect-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirect-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDrawIndirect-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled
- VUID-vkCmdDrawIndirect-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-vkCmdDrawIndirect-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE
- VUID-vkCmdDrawIndirect-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the

VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT

- VUID-vkCmdDrawIndirect-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirect-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT
- VUID-vkCmdDrawIndirect-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDrawIndirect-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDrawIndirect-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-vkCmdDrawIndirect-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdDrawIndirect-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdDrawIndirect-None-08699
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound
- VUID-vkCmdDrawIndirect-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges

- VUID-vkCmdDrawIndirect-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDrawIndirect-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with `VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE` set in VkPipelineDynamicStateCreateInfo ::`pDynamicStates`, and the fragment shader declares the `EarlyFragmentTests` execution mode and uses `OpDepthAttachmentReadEXT`, the `depthWriteEnable` parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be `VK_FALSE`
- VUID-vkCmdDrawIndirect-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with `VK_DYNAMIC_STATE_STENCIL_WRITE_MASK` set in VkPipelineDynamicStateCreateInfo ::`pDynamicStates`, and the fragment shader declares the `EarlyFragmentTests` execution mode and uses `OpStencilAttachmentReadEXT`, the `writeMask` parameter in the last call to vkCmdSetStencilWriteMask **must** be `0`
- VUID-vkCmdDrawIndirect-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT`, and the format of any color attachment is `VK_FORMAT_E5B9G9R9_UFLOAT_PACK32`, the corresponding element of the `pColorWriteMasks` parameter of vkCmdSetColorWriteMaskEXT **must** either include all of `VK_COLOR_COMPONENT_R_BIT`, `VK_COLOR_COMPONENT_G_BIT`, and `VK_COLOR_COMPONENT_B_BIT`, or none of them
- VUID-vkCmdDrawIndirect-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of `Location` for any output attachment statically used in the `Fragment Execution Model` executed by this command **must** be less than `maxFragmentDualSrcAttachments`
- VUID-vkCmdDrawIndirect-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::`pColorAttachmentLocations` set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdDrawIndirect-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndirect-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDrawIndirect-None-04008
  If the `nullDescriptor` feature is not enabled, all vertex input bindings accessed via vertex

input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE

- VUID-vkCmdDrawIndirect-None-02721
  If the robustBufferAccess feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDrawIndirect-None-04914
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command
- VUID-vkCmdDrawIndirect-Input-07939
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription2EXT::location
- VUID-vkCmdDrawIndirect-Input-08734
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and either the legacyVertexAttributes feature is not enabled or the SPIR-V Type associated with a given Input variable of the corresponding Location in the Vertex Execution Model OpEntryPoint is 64-bit, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription2EXT::format
- VUID-vkCmdDrawIndirect-format-08936
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit
- VUID-vkCmdDrawIndirect-format-08937
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component
- VUID-vkCmdDrawIndirect-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format
- VUID-vkCmdDrawIndirect-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then

vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirect-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndirect-buffer-02708
  If buffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdDrawIndirect-buffer-02709
  buffer **must** have been created with the VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT bit set
- VUID-vkCmdDrawIndirect-offset-02710
  offset **must** be a multiple of 4
- VUID-vkCmdDrawIndirect-drawCount-02718
  If the multiDrawIndirect feature is not enabled, drawCount **must** be 0 or 1
- VUID-vkCmdDrawIndirect-drawCount-02719
  drawCount **must** be less than or equal to VkPhysicalDeviceLimits::maxDrawIndirectCount
- VUID-vkCmdDrawIndirect-drawCount-00476
  If drawCount is greater than 1, stride **must** be a multiple of 4 and **must** be greater than or equal to sizeof(VkDrawIndirectCommand)
- VUID-vkCmdDrawIndirect-drawCount-00487
  If drawCount is equal to 1, (offset + sizeof(VkDrawIndirectCommand)) **must** be less than or equal to the size of buffer
- VUID-vkCmdDrawIndirect-drawCount-00488
  If drawCount is greater than 1, (stride × (drawCount - 1) + offset + sizeof(VkDrawIndirectCommand)) **must** be less than or equal to the size of buffer

Valid Usage (Implicit)

- VUID-vkCmdDrawIndirect-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDrawIndirect-buffer-parameter
  buffer **must** be a valid VkBuffer handle
- VUID-vkCmdDrawIndirect-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdDrawIndirect-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations

- VUID-vkCmdDrawIndirect-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDrawIndirect-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdDrawIndirect-commonparent
  Both of `buffer`, and `commandBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

The `VkDrawIndirectCommand` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDrawIndirectCommand {
    uint32_t    vertexCount;
    uint32_t    instanceCount;
    uint32_t    firstVertex;
    uint32_t    firstInstance;
} VkDrawIndirectCommand;
```

- `vertexCount` is the number of vertices to draw.
- `instanceCount` is the number of instances to draw.
- `firstVertex` is the index of the first vertex to draw.
- `firstInstance` is the instance ID of the first instance to draw.

The members of `VkDrawIndirectCommand` have the same meaning as the similarly named parameters of vkCmdDraw.

Valid Usage

- VUID-VkDrawIndirectCommand-pNext-09461
  If the bound graphics pipeline state was created with VkPipelineVertexInputDivisorStateCreateInfoKHR in the pNext chain of VkGraphicsPipelineCreateInfo::pVertexInputState, any member of VkPipelineVertexInputDivisorStateCreateInfoKHR::pVertexBindingDivisors has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-VkDrawIndirectCommand-None-09462
  If shader objects are used for drawing or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled, any member of the pVertexBindingDescriptions parameter to the vkCmdSetVertexInputEXT call that sets this dynamic state has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-VkDrawIndirectCommand-firstInstance-00501
  If the drawIndirectFirstInstance feature is not enabled, firstInstance **must** be 0

To record a non-indexed draw call with a draw call count sourced from a buffer, call:

```
// Provided by VK_KHR_draw_indirect_count
void vkCmdDrawIndirectCountKHR(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    buffer,
    VkDeviceSize                                offset,
    VkBuffer                                    countBuffer,
    VkDeviceSize                                countBufferOffset,
    uint32_t                                    maxDrawCount,
    uint32_t                                    stride);
```

- commandBuffer is the command buffer into which the command is recorded.
- buffer is the buffer containing draw parameters.
- offset is the byte offset into buffer where parameters begin.
- countBuffer is the buffer containing the draw count.
- countBufferOffset is the byte offset into countBuffer where the draw count begins.
- maxDrawCount specifies the maximum number of draws that will be executed. The actual number of executed draw calls is the minimum of the count specified in countBuffer and maxDrawCount.
- stride is the byte stride between successive sets of draw parameters.

vkCmdDrawIndirectCountKHR behaves similarly to vkCmdDrawIndirect except that the draw count is read by the device from a buffer during execution. The command will read an unsigned 32-bit integer from countBuffer located at countBufferOffset and use this as the draw count.

Valid Usage

- VUID-vkCmdDrawIndirectCount-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawIndirectCount-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawIndirectCount-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDrawIndirectCount-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`
- VUID-vkCmdDrawIndirectCount-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain `VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT`
- VUID-vkCmdDrawIndirectCount-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT`
- VUID-vkCmdDrawIndirectCount-None-07888
  If a `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER` descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT`
- VUID-vkCmdDrawIndirectCount-None-02692
  If a VkImageView is sampled with `VK_FILTER_CUBIC_EXT` as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT`
- VUID-vkCmdDrawIndirectCount-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with `VK_FILTER_CUBIC_EXT` as a result of this command, it **must** not have a VkImageViewType of `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`
- VUID-vkCmdDrawIndirectCount-filterCubic-02694
  Any VkImageView being sampled with `VK_FILTER_CUBIC_EXT` as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::`filterCubic` returned by vkGetPhysicalDeviceImageFormatProperties2KHR

- VUID-vkCmdDrawIndirectCount-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndirectCount-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndirectCount-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndirectCount-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndirectCount-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndirectCount-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndirectCount-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndirectCount-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDrawIndirectCount-maintenance4-08602
  If the maintenance4 feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in

Pipeline Layout Compatibility

- VUID-vkCmdDrawIndirectCount-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDrawIndirectCount-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDrawIndirectCount-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDrawIndirectCount-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdDrawIndirectCount-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDrawIndirectCount-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDrawIndirectCount-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDrawIndirectCount-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirectCount-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform

buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point

- VUID-vkCmdDrawIndirectCount-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirectCount-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirectCount-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdDrawIndirectCount-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdDrawIndirectCount-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDrawIndirectCount-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdDrawIndirectCount-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDrawIndirectCount-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdDrawIndirectCount-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDrawIndirectCount-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDrawIndirectCount-SampledType-04471

If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32

- VUID-vkCmdDrawIndirectCount-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawIndirectCount-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawIndirectCount-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndirectCount-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndirectCount-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDrawIndirectCount-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDrawIndirectCount-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndirectCount-subpass-02685
  The subpass index of the current render pass **must** be equal to the subpass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndirectCount-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdDrawIndirectCount-OpTypeImage-07468
  If any shader executed by this pipeline accesses an OpTypeImage variable with a Dim operand of SubpassData, it **must** be decorated with an InputAttachmentIndex that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDrawIndirectCount-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat

as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' pInputAttachments [InputAttachmentIndex] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility

- VUID-vkCmdDrawIndirectCount-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a InputAttachmentIndex referenced by VkRenderingInputAttachmentIndexInfoKHR, or no InputAttachmentIndex if VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex or VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are NULL, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdDrawIndirectCount-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an InputAttachmentIndex if both VkRenderingInputAttachmentIndexInfoKHR :pDepthInputAttachmentIndex and VkRenderingInputAttachmentIndexInfoKHR :pStencilInputAttachmentIndex are non-NULL
- VUID-vkCmdDrawIndirectCount-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an InputAttachmentIndex, the InputAttachmentIndex **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndirectCount-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDrawIndirectCount-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_COLOR_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawIndirectCount-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or

- the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
  - there is no bound graphics pipeline or
  - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndirectCount-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndirectCount-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdDrawIndirectCount-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDrawIndirectCount-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdDrawIndirectCount-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back writeMask are not zero, and stencil test is enabled, all stencil ops **must** be VK_STENCIL_OP_KEEP
- VUID-vkCmdDrawIndirectCount-None-07831
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VIEWPORT dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07832
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SCISSOR dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectCount-None-07833
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_WIDTH dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08619
  If a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor

member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectCount-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties::maxMultiviewInstanceIndex
- VUID-vkCmdDrawIndirectCount-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndirectCount-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectCount-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdDrawIndirectCount-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdDrawIndirectCount-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndirectCount-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndirectCount-None-08876
  If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdDrawIndirectCount-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command

must not write any values to the depth attachment

- VUID-vkCmdDrawIndirectCount-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndirectCount-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndirectCount-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndirectCount-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndirectCount-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndirectCount-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`viewMask` equal to VkRenderingInfo::`viewMask`
- VUID-vkCmdDrawIndirectCount-colorAttachmentCount-06179
  If the `dynamicRenderingUnusedAttachments` feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`colorAttachmentCount` equal to VkRenderingInfo::`colorAttachmentCount`
- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08910
  If the `dynamicRenderingUnusedAttachments` feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::`colorAttachmentCount` greater than `0`, then each element of the VkRenderingInfo::`pColorAttachments` array with an `imageView` not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::`pColorAttachmentFormats` used to create the bound graphics pipeline

- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectCount-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdDrawIndirectCount-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance
- VUID-vkCmdDrawIndirectCount-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle

in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount

- VUID-vkCmdDrawIndirectCount-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the

dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndirectCount-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectCount-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDrawIndirectCount-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdDrawIndirectCount-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline
- VUID-vkCmdDrawIndirectCount-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndirectCount-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndirectCount-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass

equal to VK_NULL_HANDLE

- VUID-vkCmdDrawIndirectCount-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectCount-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectCount-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectCount-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdDrawIndirectCount-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDrawIndirectCount-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the

current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectCount-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndirectCount-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndirectCount-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior

to this drawing command

- VUID-vkCmdDrawIndirectCount-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectCount-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines

features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectCount-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectCount-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT
- VUID-vkCmdDrawIndirectCount-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdDrawIndirectCount-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in

the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline

- VUID-vkCmdDrawIndirectCount-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectCount-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndirectCount-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirectCount-rasterizerDiscardEnable-09417
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirectCount-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndirectCount-rasterizerDiscardEnable-09418
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and

attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled

- VUID-vkCmdDrawIndirectCount-firstAttachment-07478
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirectCount-rasterizerDiscardEnable-09419
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirectCount-firstAttachment-07479
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendAdvancedEXT calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndirectCount-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT and VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed advancedBlendMaxColorAttachments
- VUID-vkCmdDrawIndirectCount-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, and the bound graphics pipeline was created with VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the rasterizationStream to zero
- VUID-vkCmdDrawIndirectCount-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples member of the VkPipelineMultisampleStateCreateInfo structure the

bound graphics pipeline has been created with

- VUID-vkCmdDrawIndirectCount-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectCount-sampleLocationsEnable-07484
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndirectCount-sampleLocationsEnable-07485
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndirectCount-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndirectCount-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-vkCmdDrawIndirectCount-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the

VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT

- VUID-vkCmdDrawIndirectCount-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectCount-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectCount-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDrawIndirectCount-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled
- VUID-vkCmdDrawIndirectCount-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-vkCmdDrawIndirectCount-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the

VK_DYNAMIC_STATE_LINE_STIPPLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE

- VUID-vkCmdDrawIndirectCount-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdDrawIndirectCount-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT
- VUID-vkCmdDrawIndirectCount-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDrawIndirectCount-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDrawIndirectCount-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-vkCmdDrawIndirectCount-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdDrawIndirectCount-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdDrawIndirectCount-None-08699

If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound

- VUID-vkCmdDrawIndirectCount-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdDrawIndirectCount-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDrawIndirectCount-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndirectCount-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to vkCmdSetStencilWriteMask **must** be 0
- VUID-vkCmdDrawIndirectCount-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and the format of any color attachment is VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, the corresponding element of the pColorWriteMasks parameter of vkCmdSetColorWriteMaskEXT **must** either include all of VK_COLOR_COMPONENT_R_BIT, VK_COLOR_COMPONENT_G_BIT, and VK_COLOR_COMPONENT_B_BIT, or none of them
- VUID-vkCmdDrawIndirectCount-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of Location for any output attachment statically used in the Fragment Execution Model executed by this command **must** be less than maxFragmentDualSrcAttachments
- VUID-vkCmdDrawIndirectCount-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::pColorAttachmentLocations set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdDrawIndirectCount-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR

- VUID-vkCmdDrawIndirectCount-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDrawIndirectCount-None-04008
  If the nullDescriptor feature is not enabled, all vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE
- VUID-vkCmdDrawIndirectCount-None-02721
  If the robustBufferAccess feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDrawIndirectCount-None-04914
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command
- VUID-vkCmdDrawIndirectCount-Input-07939
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription2EXT::location
- VUID-vkCmdDrawIndirectCount-Input-08734
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and either the legacyVertexAttributes feature is not enabled or the SPIR-V Type associated with a given Input variable of the corresponding Location in the Vertex Execution Model OpEntryPoint is 64-bit, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription2EXT::format
- VUID-vkCmdDrawIndirectCount-format-08936
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit
- VUID-vkCmdDrawIndirectCount-format-08937
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component
- VUID-vkCmdDrawIndirectCount-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound

graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format

- VUID-vkCmdDrawIndirectCount-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectCount-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndirectCount-buffer-02708
  If buffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdDrawIndirectCount-buffer-02709
  buffer **must** have been created with the VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT bit set
- VUID-vkCmdDrawIndirectCount-offset-02710
  offset **must** be a multiple of 4
- VUID-vkCmdDrawIndirectCount-countBuffer-02714
  If countBuffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdDrawIndirectCount-countBuffer-02715
  countBuffer **must** have been created with the VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT bit set
- VUID-vkCmdDrawIndirectCount-countBufferOffset-02716
  countBufferOffset **must** be a multiple of 4
- VUID-vkCmdDrawIndirectCount-countBuffer-02717
  The count stored in countBuffer **must** be less than or equal to VkPhysicalDeviceLimits::maxDrawIndirectCount
- VUID-vkCmdDrawIndirectCount-countBufferOffset-04129
  (countBufferOffset + sizeof(uint32_t)) **must** be less than or equal to the size of countBuffer
- VUID-vkCmdDrawIndirectCount-stride-03110
  stride **must** be a multiple of 4 and **must** be greater than or equal to sizeof(VkDrawIndirectCommand)
- VUID-vkCmdDrawIndirectCount-maxDrawCount-03111
  If maxDrawCount is greater than or equal to 1, (stride × (maxDrawCount - 1) + offset + sizeof(VkDrawIndirectCommand)) **must** be less than or equal to the size of buffer

Valid Usage (Implicit)

- VUID-vkCmdDrawIndirectCount-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDrawIndirectCount-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle
- VUID-vkCmdDrawIndirectCount-countBuffer-parameter
  `countBuffer` **must** be a valid VkBuffer handle
- VUID-vkCmdDrawIndirectCount-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdDrawIndirectCount-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdDrawIndirectCount-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDrawIndirectCount-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdDrawIndirectCount-commonparent
  Each of `buffer`, `commandBuffer`, and `countBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

To record an indexed indirect drawing command, call:

```
// Provided by VK_VERSION_1_0
void vkCmdDrawIndexedIndirect(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    buffer,
    VkDeviceSize                                offset,
    uint32_t                                    drawCount,
    uint32_t                                    stride);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `buffer` is the buffer containing draw parameters.
- `offset` is the byte offset into `buffer` where parameters begin.
- `drawCount` is the number of draws to execute, and **can** be zero.
- `stride` is the byte stride between successive sets of draw parameters.

`vkCmdDrawIndexedIndirect` behaves similarly to vkCmdDrawIndexed except that the parameters are read by the device from a buffer during execution. `drawCount` draws are executed by the command, with parameters taken from `buffer` starting at `offset` and increasing by `stride` bytes for each successive draw. The parameters of each draw are encoded in an array of VkDrawIndexedIndirectCommand structures. If `drawCount` is less than or equal to one, `stride` is ignored.

Valid Usage

- VUID-vkCmdDrawIndexedIndirect-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawIndexedIndirect-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawIndexedIndirect-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDrawIndexedIndirect-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`
- VUID-vkCmdDrawIndexedIndirect-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain `VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT`

- VUID-vkCmdDrawIndexedIndirect-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdDrawIndexedIndirect-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdDrawIndexedIndirect-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdDrawIndexedIndirect-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdDrawIndexedIndirect-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndexedIndirect-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndexedIndirect-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexedIndirect-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexedIndirect-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexedIndirect-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT

- VUID-vkCmdDrawIndexedIndirect-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndexedIndirect-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndexedIndirect-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDrawIndexedIndirect-maintenance4-08602
  If the `maintenance4` feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndexedIndirect-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDrawIndexedIndirect-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDrawIndexedIndirect-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDrawIndexedIndirect-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdDrawIndexedIndirect-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler

must not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage

- VUID-vkCmdDrawIndexedIndirect-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDrawIndexedIndirect-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDrawIndexedIndirect-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexedIndirect-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexedIndirect-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexedIndirect-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexedIndirect-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdDrawIndexedIndirect-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdDrawIndexedIndirect-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types

- VUID-vkCmdDrawIndexedIndirect-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's format and the Sampled Type operand of the OpTypeImage **must** match
- VUID-vkCmdDrawIndexedIndirect-OpImageWrite-08795
  If a VkImageView created with a format other than VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDrawIndexedIndirect-OpImageWrite-08796
  If a VkImageView created with the format VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have four components
- VUID-vkCmdDrawIndexedIndirect-OpImageWrite-04469
  If a VkBufferView is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDrawIndexedIndirect-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawIndexedIndirect-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawIndexedIndirect-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawIndexedIndirect-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawIndexedIndirect-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndexedIndirect-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndexedIndirect-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDrawIndexedIndirect-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE,

VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written

- VUID-vkCmdDrawIndexedIndirect-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndexedIndirect-subpass-02685
  The subpass index of the current render pass **must** be equal to the subpass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndexedIndirect-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdDrawIndexedIndirect-OpTypeImage-07468
  If any shader executed by this pipeline accesses an OpTypeImage variable with a Dim operand of SubpassData, it **must** be decorated with an InputAttachmentIndex that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDrawIndexedIndirect-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' pInputAttachments [InputAttachmentIndex] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility
- VUID-vkCmdDrawIndexedIndirect-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a InputAttachmentIndex referenced by VkRenderingInputAttachmentIndexInfoKHR, or no InputAttachmentIndex if VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex or VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are NULL, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdDrawIndexedIndirect-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an InputAttachmentIndex if both VkRenderingInputAttachmentIndexInfoKHR :pDepthInputAttachmentIndex and VkRenderingInputAttachmentIndexInfoKHR :pStencilInputAttachmentIndex are non-NULL
- VUID-vkCmdDrawIndexedIndirect-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an InputAttachmentIndex, the InputAttachmentIndex **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndexedIndirect-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDrawIndexedIndirect-None-09000

If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:

- the VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
- the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_COLOR_BIT and
  - there is no bound graphics pipeline or
  - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndexedIndirect-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndexedIndirect-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndexedIndirect-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an

attachment, storage image, or sampled image by this command

- VUID-vkCmdDrawIndexedIndirect-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDrawIndexedIndirect-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdDrawIndexedIndirect-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back `writeMask` are not zero, and stencil test is enabled, all stencil ops **must** be `VK_STENCIL_OP_KEEP`
- VUID-vkCmdDrawIndexedIndirect-None-07831
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VIEWPORT` dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07832
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SCISSOR` dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07833
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_LINE_WIDTH` dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set `polygonMode` to `VK_POLYGON_MODE_LINE`, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set `primitiveTopology` to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08619
  If a shader object that outputs line primitives is bound to the `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT` or `VK_SHADER_STAGE_GEOMETRY_BIT` stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this

drawing command

- VUID-vkCmdDrawIndexedIndirect-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and

not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirect-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties ::maxMultiviewInstanceIndex
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndexedIndirect-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdDrawIndexedIndirect-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current

command buffer that affected that attachment index **must** have been VK_FALSE

- VUID-vkCmdDrawIndexedIndirect-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments

- VUID-vkCmdDrawIndexedIndirect-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments

- VUID-vkCmdDrawIndexedIndirect-None-08876
  If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR

- VUID-vkCmdDrawIndexedIndirect-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the depth attachment

- VUID-vkCmdDrawIndexedIndirect-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment

- VUID-vkCmdDrawIndexedIndirect-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL, this command **must** not write any values to the depth attachment

- VUID-vkCmdDrawIndexedIndirect-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment

- VUID-vkCmdDrawIndexedIndirect-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, this command **must** not write any values to the depth attachment

- VUID-vkCmdDrawIndexedIndirect-imageView-06177

If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pStencilAttachment is not VK_NULL_HANDLE, and the layout member of pStencilAttachment is VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL, this command **must** not write any values to the stencil attachment

- VUID-vkCmdDrawIndexedIndirect-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::viewMask equal to VkRenderingInfo::viewMask
- VUID-vkCmdDrawIndexedIndirect-colorAttachmentCount-06179
  If the dynamicRenderingUnusedAttachments feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::colorAttachmentCount equal to VkRenderingInfo::colorAttachmentCount
- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08910
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline
- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirect-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to

vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirect-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdDrawIndexedIndirect-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance
- VUID-vkCmdDrawIndexedIndirect-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount
- VUID-vkCmdDrawIndexedIndirect-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08913

If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndexedIndirect-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirect-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDrawIndexedIndirect-imageView-06184

If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT

- VUID-vkCmdDrawIndexedIndirect-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline
- VUID-vkCmdDrawIndexedIndirect-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndexedIndirect-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndexedIndirect-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass equal to VK_NULL_HANDLE
- VUID-vkCmdDrawIndexedIndirect-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirect-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirect-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirect-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be

enabled

- VUID-vkCmdDrawIndexedIndirect-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDrawIndexedIndirect-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndexedIndirect-alphaToCoverageEnable-08920

If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0

- VUID-vkCmdDrawIndexedIndirect-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in

pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirect-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently

invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirect-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT

must have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirect-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently

invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirect-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT
- VUID-vkCmdDrawIndexedIndirect-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdDrawIndexedIndirect-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline
- VUID-vkCmdDrawIndexedIndirect-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirect-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndexedIndirect-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and

attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass

- VUID-vkCmdDrawIndexedIndirect-rasterizerDiscardEnable-09417
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexedIndirect-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndexedIndirect-rasterizerDiscardEnable-09418
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndexedIndirect-firstAttachment-07478
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexedIndirect-rasterizerDiscardEnable-09419
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexedIndirect-firstAttachment-07479
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then

vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendAdvancedEXT` calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled

- VUID-vkCmdDrawIndexedIndirect-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` and `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT` dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed `advancedBlendMaxColorAttachments`
- VUID-vkCmdDrawIndexedIndirect-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the `primitivesGeneratedQueryWithNonZeroStreams` feature is not enabled and the `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT` query is active, and the bound graphics pipeline was created with `VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT` state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the `rasterizationStream` to zero
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` state enabled and the `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT` state disabled, and the current value of `sampleLocationsEnable` is `VK_TRUE`, then the `sampleLocationsPerPixel` member of `pSampleLocationsInfo` in the last call to vkCmdSetSampleLocationsEXT **must** equal the `rasterizationSamples` member of the VkPipelineMultisampleStateCreateInfo structure the bound graphics pipeline has been created with
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` state enabled and the `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT` state enabled, and the current value of `sampleLocationsEnable` is `VK_TRUE`, then the `sampleLocationsPerPixel` member of `pSampleLocationsInfo` in the last call to vkCmdSetSampleLocationsEXT **must** equal the `rasterizationSamples` parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsEnable-07484
  If a shader object is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, or the bound graphics pipeline was created with the `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT` state enabled, and `sampleLocationsEnable` was `VK_TRUE` in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the `VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT` bit set
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsEnable-07485
  If a shader object is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, or the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` state enabled and the `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT` state enabled, and if `sampleLocationsEnable` was `VK_TRUE` in the last call to

vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples

- VUID-vkCmdDrawIndexedIndirect-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirect-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must**

equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT

- VUID-vkCmdDrawIndexedIndirect-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDrawIndexedIndirect-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled
- VUID-vkCmdDrawIndexedIndirect-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-vkCmdDrawIndexedIndirect-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE
- VUID-vkCmdDrawIndexedIndirect-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdDrawIndexedIndirect-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-08684

If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT

- VUID-vkCmdDrawIndexedIndirect-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDrawIndexedIndirect-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDrawIndexedIndirect-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-vkCmdDrawIndexedIndirect-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdDrawIndexedIndirect-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdDrawIndexedIndirect-None-08699
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound
- VUID-vkCmdDrawIndexedIndirect-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdDrawIndexedIndirect-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDrawIndexedIndirect-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndexedIndirect-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to

vkCmdSetStencilWriteMask **must** be `0`

- VUID-vkCmdDrawIndexedIndirect-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT`, and the format of any color attachment is `VK_FORMAT_E5B9G9R9_UFLOAT_PACK32`, the corresponding element of the `pColorWriteMasks` parameter of vkCmdSetColorWriteMaskEXT **must** either include all of `VK_COLOR_COMPONENT_R_BIT`, `VK_COLOR_COMPONENT_G_BIT`, and `VK_COLOR_COMPONENT_B_BIT`, or none of them
- VUID-vkCmdDrawIndexedIndirect-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of `Location` for any output attachment statically used in the `Fragment Execution Model` executed by this command **must** be less than `maxFragmentDualSrcAttachments`
- VUID-vkCmdDrawIndexedIndirect-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::`pColorAttachmentLocations` set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdDrawIndexedIndirect-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndexedIndirect-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDrawIndexedIndirect-None-04008
  If the `nullDescriptor` feature is not enabled, all vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE
- VUID-vkCmdDrawIndexedIndirect-None-02721
  If the `robustBufferAccess` feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDrawIndexedIndirect-None-04914
  If there is a shader object bound to the `VK_SHADER_STAGE_VERTEX_BIT` stage or the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command
- VUID-vkCmdDrawIndexedIndirect-Input-07939
  If there is a shader object bound to the `VK_SHADER_STAGE_VERTEX_BIT` stage or the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic

state enabled then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription2EXT::location

- VUID-vkCmdDrawIndexedIndirect-Input-08734
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and either the legacyVertexAttributes feature is not enabled or the SPIR-V Type associated with a given Input variable of the corresponding Location in the Vertex Execution Model OpEntryPoint is 64-bit, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription2EXT::format
- VUID-vkCmdDrawIndexedIndirect-format-08936
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit
- VUID-vkCmdDrawIndexedIndirect-format-08937
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component
- VUID-vkCmdDrawIndexedIndirect-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format
- VUID-vkCmdDrawIndexedIndirect-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirect-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndexedIndirect-buffer-02708
  If buffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdDrawIndexedIndirect-buffer-02709

`buffer` **must** have been created with the `VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT` bit set

- VUID-vkCmdDrawIndexedIndirect-offset-02710
  `offset` **must** be a multiple of `4`
- VUID-vkCmdDrawIndexedIndirect-drawCount-02718
  If the `multiDrawIndirect` feature is not enabled, `drawCount` **must** be `0` or `1`
- VUID-vkCmdDrawIndexedIndirect-drawCount-02719
  `drawCount` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxDrawIndirectCount`
- VUID-vkCmdDrawIndexedIndirect-None-07312
  If the `maintenance6` feature is not enabled, a valid index buffer **must** be bound
- VUID-vkCmdDrawIndexedIndirect-drawCount-00528
  If `drawCount` is greater than `1`, `stride` **must** be a multiple of `4` and **must** be greater than or equal to `sizeof`(`VkDrawIndexedIndirectCommand`)
- VUID-vkCmdDrawIndexedIndirect-drawCount-00539
  If `drawCount` is equal to `1`, (`offset` + `sizeof`(`VkDrawIndexedIndirectCommand`)) **must** be less than or equal to the size of `buffer`
- VUID-vkCmdDrawIndexedIndirect-drawCount-00540
  If `drawCount` is greater than `1`, (`stride` × (`drawCount` - 1) + `offset` + `sizeof`(`VkDrawIndexedIndirectCommand`)) **must** be less than or equal to the size of `buffer`

Valid Usage (Implicit)

- VUID-vkCmdDrawIndexedIndirect-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDrawIndexedIndirect-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle
- VUID-vkCmdDrawIndexedIndirect-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdDrawIndexedIndirect-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdDrawIndexedIndirect-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDrawIndexedIndirect-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdDrawIndexedIndirect-commonparent
  Both of `buffer`, and `commandBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

The `VkDrawIndexedIndirectCommand` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDrawIndexedIndirectCommand {
    uint32_t    indexCount;
    uint32_t    instanceCount;
    uint32_t    firstIndex;
    int32_t     vertexOffset;
    uint32_t    firstInstance;
} VkDrawIndexedIndirectCommand;
```

- `indexCount` is the number of vertices to draw.
- `instanceCount` is the number of instances to draw.
- `firstIndex` is the base index within the index buffer.
- `vertexOffset` is the value added to the vertex index before indexing into the vertex buffer.
- `firstInstance` is the instance ID of the first instance to draw.

The members of `VkDrawIndexedIndirectCommand` have the same meaning as the similarly named parameters of vkCmdDrawIndexed.

Valid Usage

- VUID-VkDrawIndexedIndirectCommand-pNext-09461
  If the bound graphics pipeline state was created with VkPipelineVertexInputDivisorStateCreateInfoKHR in the `pNext` chain of VkGraphicsPipelineCreateInfo::`pVertexInputState`, any member of VkPipelineVertexInputDivisorStateCreateInfoKHR::`pVertexBindingDivisors` has a value other than `1` in `divisor`, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR ::`supportsNonZeroFirstInstance` is `VK_FALSE`, then `firstInstance` **must** be `0`

- VUID-VkDrawIndexedIndirectCommand-None-09462
  If shader objects are used for drawing or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled, any member of the pVertexBindingDescriptions parameter to the vkCmdSetVertexInputEXT call that sets this dynamic state has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0

- VUID-VkDrawIndexedIndirectCommand-robustBufferAccess2-08798
  If the robustBufferAccess2 feature is not enabled, (indexSize × (firstIndex + indexCount) + offset) **must** be less than or equal to the size of the bound index buffer, with indexSize being based on the type specified by indexType, where the index buffer, indexType, and offset are specified via vkCmdBindIndexBuffer or vkCmdBindIndexBuffer2KHR. If vkCmdBindIndexBuffer2KHR is used to bind the index buffer, the size of the bound index buffer is vkCmdBindIndexBuffer2KHR::size

- VUID-VkDrawIndexedIndirectCommand-firstInstance-00554
  If the drawIndirectFirstInstance feature is not enabled, firstInstance **must** be 0

To record an indexed draw call with a draw call count sourced from a buffer, call:

```
// Provided by VK_KHR_draw_indirect_count
void vkCmdDrawIndexedIndirectCountKHR(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    buffer,
    VkDeviceSize                                offset,
    VkBuffer                                    countBuffer,
    VkDeviceSize                                countBufferOffset,
    uint32_t                                    maxDrawCount,
    uint32_t                                    stride);
```

- commandBuffer is the command buffer into which the command is recorded.
- buffer is the buffer containing draw parameters.
- offset is the byte offset into buffer where parameters begin.
- countBuffer is the buffer containing the draw count.
- countBufferOffset is the byte offset into countBuffer where the draw count begins.
- maxDrawCount specifies the maximum number of draws that will be executed. The actual number of executed draw calls is the minimum of the count specified in countBuffer and maxDrawCount.
- stride is the byte stride between successive sets of draw parameters.

vkCmdDrawIndexedIndirectCountKHR behaves similarly to vkCmdDrawIndexedIndirect except that the draw count is read by the device from a buffer during execution. The command will read an unsigned 32-bit integer from countBuffer located at countBufferOffset and use this as the draw count.

Valid Usage

- VUID-vkCmdDrawIndexedIndirectCount-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawIndexedIndirectCount-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDrawIndexedIndirectCount-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDrawIndexedIndirectCount-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`
- VUID-vkCmdDrawIndexedIndirectCount-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain `VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT`
- VUID-vkCmdDrawIndexedIndirectCount-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT`
- VUID-vkCmdDrawIndexedIndirectCount-None-07888
  If a `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER` descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT`
- VUID-vkCmdDrawIndexedIndirectCount-None-02692
  If a VkImageView is sampled with `VK_FILTER_CUBIC_EXT` as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT`
- VUID-vkCmdDrawIndexedIndirectCount-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with `VK_FILTER_CUBIC_EXT` as a result of this command, it **must** not have a VkImageViewType of `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`
- VUID-vkCmdDrawIndexedIndirectCount-filterCubic-02694
  Any VkImageView being sampled with `VK_FILTER_CUBIC_EXT` as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::`filterCubic` returned by vkGetPhysicalDeviceImageFormatProperties2KHR

- VUID-vkCmdDrawIndexedIndirectCount-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndexedIndirectCount-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexedIndirectCount-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexedIndirectCount-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexedIndirectCount-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndexedIndirectCount-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndexedIndirectCount-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndexedIndirectCount-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDrawIndexedIndirectCount-maintenance4-08602
  If the maintenance4 feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in

Pipeline Layout Compatibility

- VUID-vkCmdDrawIndexedIndirectCount-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDrawIndexedIndirectCount-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDrawIndexedIndirectCount-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDrawIndexedIndirectCount-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdDrawIndexedIndirectCount-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDrawIndexedIndirectCount-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDrawIndexedIndirectCount-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDrawIndexedIndirectCount-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexedIndirectCount-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform

buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point

- VUID-vkCmdDrawIndexedIndirectCount-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexedIndirectCount-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndexedIndirectCount-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdDrawIndexedIndirectCount-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdDrawIndexedIndirectCount-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDrawIndexedIndirectCount-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdDrawIndexedIndirectCount-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDrawIndexedIndirectCount-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdDrawIndexedIndirectCount-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDrawIndexedIndirectCount-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDrawIndexedIndirectCount-SampledType-04471

If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32

- VUID-vkCmdDrawIndexedIndirectCount-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawIndexedIndirectCount-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawIndexedIndirectCount-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndexedIndirectCount-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndexedIndirectCount-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDrawIndexedIndirectCount-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDrawIndexedIndirectCount-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndexedIndirectCount-subpass-02685
  The subpass index of the current render pass **must** be equal to the subpass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndexedIndirectCount-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdDrawIndexedIndirectCount-OpTypeImage-07468
  If any shader executed by this pipeline accesses an OpTypeImage variable with a Dim operand of SubpassData, it **must** be decorated with an InputAttachmentIndex that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDrawIndexedIndirectCount-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat

as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' pInputAttachments [InputAttachmentIndex] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility

- VUID-vkCmdDrawIndexedIndirectCount-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a InputAttachmentIndex referenced by VkRenderingInputAttachmentIndexInfoKHR, or no InputAttachmentIndex if VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex or VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are NULL, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdDrawIndexedIndirectCount-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an InputAttachmentIndex if both VkRenderingInputAttachmentIndexInfoKHR :pDepthInputAttachmentIndex and VkRenderingInputAttachmentIndexInfoKHR :pStencilInputAttachmentIndex are non-NULL
- VUID-vkCmdDrawIndexedIndirectCount-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an InputAttachmentIndex, the InputAttachmentIndex **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndexedIndirectCount-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDrawIndexedIndirectCount-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_COLOR_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawIndexedIndirectCount-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or

- the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
  - there is no bound graphics pipeline or
  - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndexedIndirectCount-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndexedIndirectCount-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdDrawIndexedIndirectCount-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDrawIndexedIndirectCount-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdDrawIndexedIndirectCount-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back writeMask are not zero, and stencil test is enabled, all stencil ops **must** be VK_STENCIL_OP_KEEP
- VUID-vkCmdDrawIndexedIndirectCount-None-07831
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VIEWPORT dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07832
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SCISSOR dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirectCount-None-07833
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_WIDTH dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08619
  If a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor

member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirectCount-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties::maxMultiviewInstanceIndex
- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndexedIndirectCount-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirectCount-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdDrawIndexedIndirectCount-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdDrawIndexedIndirectCount-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndexedIndirectCount-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndexedIndirectCount-None-08876
  If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdDrawIndexedIndirectCount-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the imageView member of pDepthAttachment is not VK_NULL_HANDLE, and the layout member of pDepthAttachment is VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL, this command

must not write any values to the depth attachment

- VUID-vkCmdDrawIndexedIndirectCount-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndexedIndirectCount-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndexedIndirectCount-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndexedIndirectCount-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndexedIndirectCount-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndexedIndirectCount-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`viewMask` equal to VkRenderingInfo::`viewMask`
- VUID-vkCmdDrawIndexedIndirectCount-colorAttachmentCount-06179
  If the `dynamicRenderingUnusedAttachments` feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`colorAttachmentCount` equal to VkRenderingInfo::`colorAttachmentCount`
- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08910
  If the `dynamicRenderingUnusedAttachments` feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo::`colorAttachmentCount` greater than `0`, then each element of the VkRenderingInfo::`pColorAttachments` array with an `imageView` not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::`pColorAttachmentFormats` used to create the bound graphics pipeline

- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirectCount-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdDrawIndexedIndirectCount-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance
- VUID-vkCmdDrawIndexedIndirectCount-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle

in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount

- VUID-vkCmdDrawIndexedIndirectCount-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the

dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndexedIndirectCount-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirectCount-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDrawIndexedIndirectCount-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdDrawIndexedIndirectCount-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline
- VUID-vkCmdDrawIndexedIndirectCount-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndexedIndirectCount-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndexedIndirectCount-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass

equal to VK_NULL_HANDLE

- VUID-vkCmdDrawIndexedIndirectCount-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirectCount-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirectCount-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndexedIndirectCount-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdDrawIndexedIndirectCount-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDrawIndexedIndirectCount-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07621

If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirectCount-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndexedIndirectCount-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndexedIndirectCount-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current

value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirectCount-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07631

If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirectCount-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirectCount-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to

any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndexedIndirectCount-None-08671
  If any of the `stippledRectangularLines`, `stippledBresenhamLines` or `stippledSmoothLines` features are enabled, and a shader object that outputs line primitives is bound to the `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT` or `VK_SHADER_STAGE_GEOMETRY_BIT` stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07849
  If any of the `stippledRectangularLines`, `stippledBresenhamLines` or `stippledSmoothLines` features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_LINE_STIPPLE_KHR` dynamic state enabled, the current value of `rasterizerDiscardEnable` is `VK_FALSE`, and the current value of `stippledLineEnable` is `VK_TRUE`, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-07639
  If the `depthClipControl` feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the `VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT` dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-09650
  If the `depthClampControl` feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the `VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT` dynamic state enabled, and the current value of `depthClampEnable` is `VK_TRUE`, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT` state enabled and the last call to vkCmdSetColorBlendEnableEXT set `pColorBlendEnables` for any attachment to `VK_TRUE`, then for those attachments in the subpass the corresponding image view's format features **must** contain `VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT`
- VUID-vkCmdDrawIndexedIndirectCount-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT` state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the `rasterizationSamples` in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdDrawIndexedIndirectCount-samples-07472
  If the bound graphics pipeline state was created with the

VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline

- VUID-vkCmdDrawIndexedIndirectCount-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirectCount-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndexedIndirectCount-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexedIndirectCount-rasterizerDiscardEnable-09417
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexedIndirectCount-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndexedIndirectCount-rasterizerDiscardEnable-09418
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then

vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendEquationEXT` calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled

- VUID-vkCmdDrawIndexedIndirectCount-firstAttachment-07478
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT` dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorWriteMaskEXT` calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexedIndirectCount-rasterizerDiscardEnable-09419
  If a shader object is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorWriteMaskEXT` calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndexedIndirectCount-firstAttachment-07479
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendAdvancedEXT` calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndexedIndirectCount-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` and `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT` dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed `advancedBlendMaxColorAttachments`
- VUID-vkCmdDrawIndexedIndirectCount-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the `primitivesGeneratedQueryWithNonZeroStreams` feature is not enabled and the `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT` query is active, and the bound graphics pipeline was created with `VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT` state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the `rasterizationStream` to zero
- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` state enabled and the `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT` state disabled, and the current value of

sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples member of the VkPipelineMultisampleStateCreateInfo structure the bound graphics pipeline has been created with

- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsEnable-07484
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsEnable-07485
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample

- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirectCount-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndexedIndirectCount-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDrawIndexedIndirectCount-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled
- VUID-vkCmdDrawIndexedIndirectCount-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled

- VUID-vkCmdDrawIndexedIndirectCount-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE
- VUID-vkCmdDrawIndexedIndirectCount-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdDrawIndexedIndirectCount-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT
- VUID-vkCmdDrawIndexedIndirectCount-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDrawIndexedIndirectCount-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDrawIndexedIndirectCount-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-vkCmdDrawIndexedIndirectCount-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdDrawIndexedIndirectCount-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound

- VUID-vkCmdDrawIndexedIndirectCount-None-08699
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound
- VUID-vkCmdDrawIndexedIndirectCount-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdDrawIndexedIndirectCount-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDrawIndexedIndirectCount-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndexedIndirectCount-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to vkCmdSetStencilWriteMask **must** be 0
- VUID-vkCmdDrawIndexedIndirectCount-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and the format of any color attachment is VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, the corresponding element of the pColorWriteMasks parameter of vkCmdSetColorWriteMaskEXT **must** either include all of VK_COLOR_COMPONENT_R_BIT, VK_COLOR_COMPONENT_G_BIT, and VK_COLOR_COMPONENT_B_BIT, or none of them
- VUID-vkCmdDrawIndexedIndirectCount-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of Location for any output attachment statically used in the Fragment Execution Model executed by this command **must** be less than maxFragmentDualSrcAttachments
- VUID-vkCmdDrawIndexedIndirectCount-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::pColorAttachmentLocations set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdDrawIndexedIndirectCount-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via

VkRenderingInputAttachmentIndexInfoKHR

- VUID-vkCmdDrawIndexedIndirectCount-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDrawIndexedIndirectCount-None-04008
  If the nullDescriptor feature is not enabled, all vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE
- VUID-vkCmdDrawIndexedIndirectCount-None-02721
  If the robustBufferAccess feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDrawIndexedIndirectCount-None-04914
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command
- VUID-vkCmdDrawIndexedIndirectCount-Input-07939
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription2EXT::location
- VUID-vkCmdDrawIndexedIndirectCount-Input-08734
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and either the legacyVertexAttributes feature is not enabled or the SPIR-V Type associated with a given Input variable of the corresponding Location in the Vertex Execution Model OpEntryPoint is 64-bit, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription2EXT::format
- VUID-vkCmdDrawIndexedIndirectCount-format-08936
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit
- VUID-vkCmdDrawIndexedIndirectCount-format-08937
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component

- VUID-vkCmdDrawIndexedIndirectCount-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format
- VUID-vkCmdDrawIndexedIndirectCount-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndexedIndirectCount-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndexedIndirectCount-buffer-02708
  If buffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdDrawIndexedIndirectCount-buffer-02709
  buffer **must** have been created with the VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT bit set
- VUID-vkCmdDrawIndexedIndirectCount-offset-02710
  offset **must** be a multiple of 4
- VUID-vkCmdDrawIndexedIndirectCount-countBuffer-02714
  If countBuffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdDrawIndexedIndirectCount-countBuffer-02715
  countBuffer **must** have been created with the VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT bit set
- VUID-vkCmdDrawIndexedIndirectCount-countBufferOffset-02716
  countBufferOffset **must** be a multiple of 4
- VUID-vkCmdDrawIndexedIndirectCount-countBuffer-02717
  The count stored in countBuffer **must** be less than or equal to VkPhysicalDeviceLimits::maxDrawIndirectCount
- VUID-vkCmdDrawIndexedIndirectCount-countBufferOffset-04129
  (countBufferOffset + sizeof(uint32_t)) **must** be less than or equal to the size of countBuffer
- VUID-vkCmdDrawIndexedIndirectCount-None-07312
  If the maintenance6 feature is not enabled, a valid index buffer **must** be bound
- VUID-vkCmdDrawIndexedIndirectCount-stride-03142
  stride **must** be a multiple of 4 and **must** be greater than or equal to sizeof(VkDrawIndexedIndirectCommand)

- VUID-vkCmdDrawIndexedIndirectCount-maxDrawCount-03143
  If `maxDrawCount` is greater than or equal to `1`, (`stride` × (`maxDrawCount` - 1) + `offset` + sizeof(`VkDrawIndexedIndirectCommand`)) **must** be less than or equal to the size of `buffer`

Valid Usage (Implicit)

- VUID-vkCmdDrawIndexedIndirectCount-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDrawIndexedIndirectCount-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle
- VUID-vkCmdDrawIndexedIndirectCount-countBuffer-parameter
  `countBuffer` **must** be a valid VkBuffer handle
- VUID-vkCmdDrawIndexedIndirectCount-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdDrawIndexedIndirectCount-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdDrawIndexedIndirectCount-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDrawIndexedIndirectCount-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdDrawIndexedIndirectCount-commonparent
  Each of `buffer`, `commandBuffer`, and `countBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

21.3.1. Drawing Transform Feedback

It is possible to draw vertex data that was previously captured during active transform feedback by binding one or more of the transform feedback buffers as vertex buffers. A pipeline barrier is required between using the buffers as transform feedback buffers and vertex buffers to ensure all writes to the transform feedback buffers are visible when the data is read as vertex attributes. The source access is VK_ACCESS_TRANSFORM_FEEDBACK_WRITE_BIT_EXT and the destination access is VK_ACCESS_VERTEX_ATTRIBUTE_READ_BIT for the pipeline stages VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT and VK_PIPELINE_STAGE_VERTEX_INPUT_BIT respectively. The value written to the counter buffer by vkCmdEndTransformFeedbackEXT **can** be used to determine the vertex count for the draw. A pipeline barrier is required between using the counter buffer for vkCmdEndTransformFeedbackEXT and vkCmdDrawIndirectByteCountEXT where the source access is VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT and the destination access is VK_ACCESS_INDIRECT_COMMAND_READ_BIT for the pipeline stages VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT and VK_PIPELINE_STAGE_DRAW_INDIRECT_BIT respectively.

To record a non-indexed draw call, where the vertex count is based on a byte count read from a buffer and the passed in vertex stride parameter, call:

```
// Provided by VK_EXT_transform_feedback
void vkCmdDrawIndirectByteCountEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    instanceCount,
    uint32_t                                    firstInstance,
    VkBuffer                                    counterBuffer,
    VkDeviceSize                                counterBufferOffset,
    uint32_t                                    counterOffset,
    uint32_t                                    vertexStride);
```

- commandBuffer is the command buffer into which the command is recorded.
- instanceCount is the number of instances to draw.
- firstInstance is the instance ID of the first instance to draw.
- counterBuffer is the buffer handle from where the byte count is read.
- counterBufferOffset is the offset into the buffer used to read the byte count, which is used to calculate the vertex count for this draw call.
- counterOffset is subtracted from the byte count read from the counterBuffer at the counterBufferOffset
- vertexStride is the stride in bytes between each element of the vertex data that is used to calculate the vertex count from the counter value. This value is typically the same value that was used in the graphics pipeline state when the transform feedback was captured as the XfbStride.

When the command is executed, primitives are assembled in the same way as done with vkCmdDraw except the vertexCount is calculated based on the byte count read from counterBuffer at offset counterBufferOffset. The assembled primitives execute the bound graphics pipeline.

The effective vertexCount is calculated as follows:

```
const uint32_t * counterBufferPtr = (const uint8_t *)counterBuffer.address +
counterBufferOffset;
vertexCount = floor(max(0, (*counterBufferPtr - counterOffset)) / vertexStride);
```

The effective firstVertex is zero.

Valid Usage

- VUID-vkCmdDrawIndirectByteCountEXT-magFilter-04553
  If a VkSampler created with magFilter or minFilter equal to VK_FILTER_LINEAR, and compareEnable equal to VK_FALSE is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-mipmapMode-04770
  If a VkSampler created with mipmapMode equal to VK_SAMPLER_MIPMAP_MODE_LINEAR, and compareEnable equal to VK_FALSE is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-unnormalizedCoordinates-09635
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's levelCount and layerCount **must** be 1
- VUID-vkCmdDrawIndirectByteCountEXT-unnormalizedCoordinates-09636
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's viewType **must** be VK_IMAGE_VIEW_TYPE_1D or VK_IMAGE_VIEW_TYPE_2D
- VUID-vkCmdDrawIndirectByteCountEXT-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT

- VUID-vkCmdDrawIndirectByteCountEXT-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdDrawIndirectByteCountEXT-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndirectByteCountEXT-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDrawIndirectByteCountEXT-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndirectByteCountEXT-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility

- VUID-vkCmdDrawIndirectByteCountEXT-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDrawIndirectByteCountEXT-maintenance4-08602
  If the `maintenance4` feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDrawIndirectByteCountEXT-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDrawIndirectByteCountEXT-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDrawIndirectByteCountEXT-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdDrawIndirectByteCountEXT-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDrawIndirectByteCountEXT-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDrawIndirectByteCountEXT-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage

corresponding to the pipeline bind point used by this command

- VUID-vkCmdDrawIndirectByteCountEXT-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirectByteCountEXT-None-08612
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirectByteCountEXT-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirectByteCountEXT-None-08613
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDrawIndirectByteCountEXT-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with OpImageSample* or OpImageSparseSample* instructions
- VUID-vkCmdDrawIndirectByteCountEXT-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the ConstOffset and Offset operands
- VUID-vkCmdDrawIndirectByteCountEXT-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's viewType **must** match the Dim operand of the OpTypeImage as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDrawIndirectByteCountEXT-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's format and the Sampled Type operand of the OpTypeImage **must** match
- VUID-vkCmdDrawIndirectByteCountEXT-OpImageWrite-08795
  If a VkImageView created with a format other than VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDrawIndirectByteCountEXT-OpImageWrite-08796
  If a VkImageView created with the format VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have four components

- VUID-vkCmdDrawIndirectByteCountEXT-OpImageWrite-04469
  If a VkBufferView is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDrawIndirectByteCountEXT-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawIndirectByteCountEXT-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawIndirectByteCountEXT-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdDrawIndirectByteCountEXT-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdDrawIndirectByteCountEXT-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndirectByteCountEXT-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDrawIndirectByteCountEXT-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDrawIndirectByteCountEXT-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdDrawIndirectByteCountEXT-subpass-02685
  The subpass index of the current render pass **must** be equal to the subpass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS

- VUID-vkCmdDrawIndirectByteCountEXT-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdDrawIndirectByteCountEXT-OpTypeImage-07468
  If any shader executed by this pipeline accesses an `OpTypeImage` variable with a `Dim` operand of `SubpassData`, it **must** be decorated with an `InputAttachmentIndex` that corresponds to a valid input attachment in the current subpass
- VUID-vkCmdDrawIndirectByteCountEXT-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' `pInputAttachments` [`InputAttachmentIndex`] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility
- VUID-vkCmdDrawIndirectByteCountEXT-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a `InputAttachmentIndex` referenced by VkRenderingInputAttachmentIndexInfoKHR, or no `InputAttachmentIndex` if VkRenderingInputAttachmentIndexInfoKHR:`pDepthInputAttachmentIndex` or VkRenderingInputAttachmentIndexInfoKHR:`pStencilInputAttachmentIndex` are `NULL`, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdDrawIndirectByteCountEXT-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an `InputAttachmentIndex` if both VkRenderingInputAttachmentIndexInfoKHR:`pDepthInputAttachmentIndex` and VkRenderingInputAttachmentIndexInfoKHR:`pStencilInputAttachmentIndex` are non-`NULL`
- VUID-vkCmdDrawIndirectByteCountEXT-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an `InputAttachmentIndex`, the `InputAttachmentIndex` **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndirectByteCountEXT-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdDrawIndirectByteCountEXT-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` image layout, and either:
  - the `VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT` is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included `VK_IMAGE_ASPECT_COLOR_BIT` and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with `VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT`

it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdDrawIndirectByteCountEXT-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` image layout, and either:
  - the `VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT` is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included `VK_IMAGE_ASPECT_DEPTH_BIT` and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with `VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT`

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawIndirectByteCountEXT-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` image layout, and either:
  - the `VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT` is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included `VK_IMAGE_ASPECT_STENCIL_BIT` and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with `VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT`

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdDrawIndirectByteCountEXT-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdDrawIndirectByteCountEXT-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdDrawIndirectByteCountEXT-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdDrawIndirectByteCountEXT-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back `writeMask` are not zero, and stencil test is enabled, all stencil ops **must** be `VK_STENCIL_OP_KEEP`

- VUID-vkCmdDrawIndirectByteCountEXT-None-07831
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VIEWPORT dynamic state enabled then vkCmdSetViewport **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07832
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SCISSOR dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07833
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_WIDTH dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08619
  If a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectByteCountEXT-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties ::maxMultiviewInstanceIndex

- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndirectByteCountEXT-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdDrawIndirectByteCountEXT-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdDrawIndirectByteCountEXT-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndirectByteCountEXT-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for

the current render pass color and/or depth/stencil attachments

- VUID-vkCmdDrawIndirectByteCountEXT-None-08876
  If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR
- VUID-vkCmdDrawIndirectByteCountEXT-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndirectByteCountEXT-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndirectByteCountEXT-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndirectByteCountEXT-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndirectByteCountEXT-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdDrawIndirectByteCountEXT-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdDrawIndirectByteCountEXT-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`viewMask` equal to VkRenderingInfo::`viewMask`
- VUID-vkCmdDrawIndirectByteCountEXT-colorAttachmentCount-06179
  If the `dynamicRenderingUnusedAttachments` feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`colorAttachmentCount` equal to VkRenderingInfo::`colorAttachmentCount`

- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08910
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline
- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectByteCountEXT-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdDrawIndirectByteCountEXT-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set

rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance

- VUID-vkCmdDrawIndirectByteCountEXT-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount
- VUID-vkCmdDrawIndirectByteCountEXT-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08915

If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED

- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndirectByteCountEXT-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectByteCountEXT-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdDrawIndirectByteCountEXT-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdDrawIndirectByteCountEXT-multisampledRenderToSingleSampled-07285
  If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline
- VUID-vkCmdDrawIndirectByteCountEXT-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count

used to create VkRenderingInfo::pDepthAttachment->imageView

- VUID-vkCmdDrawIndirectByteCountEXT-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdDrawIndirectByteCountEXT-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass equal to VK_NULL_HANDLE
- VUID-vkCmdDrawIndirectByteCountEXT-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectByteCountEXT-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectByteCountEXT-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdDrawIndirectByteCountEXT-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdDrawIndirectByteCountEXT-primitivesGeneratedQueryWithNonZeroStreams-06709
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream
- VUID-vkCmdDrawIndirectByteCountEXT-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectByteCountEXT-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndirectByteCountEXT-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdDrawIndirectByteCountEXT-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the

VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectByteCountEXT-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in

the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectByteCountEXT-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current

value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectByteCountEXT-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07638
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to

VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdDrawIndirectByteCountEXT-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features

must contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT

- VUID-vkCmdDrawIndirectByteCountEXT-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdDrawIndirectByteCountEXT-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline
- VUID-vkCmdDrawIndirectByteCountEXT-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectByteCountEXT-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdDrawIndirectByteCountEXT-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirectByteCountEXT-rasterizerDiscardEnable-09417
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirectByteCountEXT-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then

vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendEquationEXT` calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled

- VUID-vkCmdDrawIndirectByteCountEXT-rasterizerDiscardEnable-09418
  If a shader object is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE` and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendEquationEXT` calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndirectByteCountEXT-firstAttachment-07478
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT` dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorWriteMaskEXT` calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirectByteCountEXT-rasterizerDiscardEnable-09419
  If a shader object is bound to the `VK_SHADER_STAGE_FRAGMENT_BIT` stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set `rasterizerDiscardEnable` to `VK_FALSE`, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorWriteMaskEXT` calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdDrawIndirectByteCountEXT-firstAttachment-07479
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the `firstAttachment` and `attachmentCount` parameters of `vkCmdSetColorBlendAdvancedEXT` calls **must** specify the advanced blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdDrawIndirectByteCountEXT-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` and `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT` dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed `advancedBlendMaxColorAttachments`
- VUID-vkCmdDrawIndirectByteCountEXT-

primitivesGeneratedQueryWithNonZeroStreams-07481
If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, and the bound graphics pipeline was created with VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the rasterizationStream to zero

- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsPerPixel-07482
If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples member of the VkPipelineMultisampleStateCreateInfo structure the bound graphics pipeline has been created with
- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsPerPixel-07483
If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsEnable-07484
If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsEnable-07485
If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT ::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsEnable-07486
If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT

::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples

- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectByteCountEXT-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdDrawIndirectByteCountEXT-stippledLineEnable-07495
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-vkCmdDrawIndirectByteCountEXT-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is

VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled

- VUID-vkCmdDrawIndirectByteCountEXT-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-vkCmdDrawIndirectByteCountEXT-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE
- VUID-vkCmdDrawIndirectByteCountEXT-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdDrawIndirectByteCountEXT-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages

with an element of VK_SHADER_STAGE_GEOMETRY_BIT

- VUID-vkCmdDrawIndirectByteCountEXT-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdDrawIndirectByteCountEXT-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdDrawIndirectByteCountEXT-None-08699
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound
- VUID-vkCmdDrawIndirectByteCountEXT-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdDrawIndirectByteCountEXT-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdDrawIndirectByteCountEXT-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndirectByteCountEXT-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to vkCmdSetStencilWriteMask **must** be 0
- VUID-vkCmdDrawIndirectByteCountEXT-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and the format of any color attachment is VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, the corresponding element of the pColorWriteMasks parameter of vkCmdSetColorWriteMaskEXT **must** either include all of VK_COLOR_COMPONENT_R_BIT, VK_COLOR_COMPONENT_G_BIT, and VK_COLOR_COMPONENT_B_BIT, or none of them
- VUID-vkCmdDrawIndirectByteCountEXT-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of Location for any output attachment statically used in the Fragment Execution Model executed by this command **must** be less than maxFragmentDualSrcAttachments

- VUID-vkCmdDrawIndirectByteCountEXT-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::pColorAttachmentLocations set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdDrawIndirectByteCountEXT-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdDrawIndirectByteCountEXT-None-04007
  All vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** have either valid or VK_NULL_HANDLE buffers bound
- VUID-vkCmdDrawIndirectByteCountEXT-None-04008
  If the nullDescriptor feature is not enabled, all vertex input bindings accessed via vertex input variables declared in the vertex shader entry point's interface **must** not be VK_NULL_HANDLE
- VUID-vkCmdDrawIndirectByteCountEXT-None-02721
  If the robustBufferAccess feature is not enabled, then for a given vertex buffer binding, any attribute data fetched **must** be entirely contained within the corresponding vertex buffer binding, as described in Vertex Input Description
- VUID-vkCmdDrawIndirectByteCountEXT-None-04914
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then vkCmdSetVertexInputEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this draw command
- VUID-vkCmdDrawIndirectByteCountEXT-Input-07939
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled then all variables with the Input storage class decorated with Location in the Vertex Execution Model OpEntryPoint **must** contain a location in VkVertexInputAttributeDescription2EXT::location
- VUID-vkCmdDrawIndirectByteCountEXT-Input-08734
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and either the legacyVertexAttributes feature is not enabled or the SPIR-V Type associated with a given Input variable of the corresponding Location in the Vertex Execution Model OpEntryPoint is 64-bit, then the numeric type associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be the same as VkVertexInputAttributeDescription2EXT::format
- VUID-vkCmdDrawIndirectByteCountEXT-format-08936
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound

graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then the scalar width associated with all Input variables of the corresponding Location in the Vertex Execution Model OpEntryPoint **must** be 64-bit

- VUID-vkCmdDrawIndirectByteCountEXT-format-08937
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and the scalar width associated with a Location decorated Input variable in the Vertex Execution Model OpEntryPoint is 64-bit, then the corresponding VkVertexInputAttributeDescription2EXT::format **must** have a 64-bit component
- VUID-vkCmdDrawIndirectByteCountEXT-None-09203
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled and VkVertexInputAttributeDescription2EXT::format has a 64-bit component, then all Input variables at the corresponding Location in the Vertex Execution Model OpEntryPoint **must** not use components that are not present in the format
- VUID-vkCmdDrawIndirectByteCountEXT-None-04879
  If there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdDrawIndirectByteCountEXT-None-09637
  If the primitiveTopologyListRestart feature is not enabled, the topology is VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY, or VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, there is a shader object bound to the VK_SHADER_STAGE_VERTEX_BIT stage then vkCmdSetPrimitiveRestartEnableEXT **must** be VK_FALSE
- VUID-vkCmdDrawIndirectByteCountEXT-pNext-09461
  If the bound graphics pipeline state was created with VkPipelineVertexInputDivisorStateCreateInfoKHR in the pNext chain of VkGraphicsPipelineCreateInfo::pVertexInputState, any member of VkPipelineVertexInputDivisorStateCreateInfoKHR::pVertexBindingDivisors has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-vkCmdDrawIndirectByteCountEXT-None-09462
  If shader objects are used for drawing or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VERTEX_INPUT_EXT dynamic state enabled, any member of the pVertexBindingDescriptions parameter to the vkCmdSetVertexInputEXT call that sets this dynamic state has a value other than 1 in divisor, and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::supportsNonZeroFirstInstance is VK_FALSE, then firstInstance **must** be 0
- VUID-vkCmdDrawIndirectByteCountEXT-transformFeedback-02287
  VkPhysicalDeviceTransformFeedbackFeaturesEXT::transformFeedback **must** be enabled
- VUID-vkCmdDrawIndirectByteCountEXT-transformFeedbackDraw-02288

The implementation **must** support VkPhysicalDeviceTransformFeedbackPropertiesEXT::transformFeedbackDraw

- VUID-vkCmdDrawIndirectByteCountEXT-vertexStride-02289
  vertexStride **must** be greater than 0 and less than or equal to VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackBufferDataStride
- VUID-vkCmdDrawIndirectByteCountEXT-counterBuffer-04567
  If counterBuffer is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdDrawIndirectByteCountEXT-counterBuffer-02290
  counterBuffer **must** have been created with the VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT bit set
- VUID-vkCmdDrawIndirectByteCountEXT-counterBufferOffset-04568
  counterBufferOffset **must** be a multiple of 4
- VUID-vkCmdDrawIndirectByteCountEXT-counterOffset-09474
  counterOffset **must** be a multiple of 4
- VUID-vkCmdDrawIndirectByteCountEXT-vertexStride-09475
  vertexStride **must** be a multiple of 4

Valid Usage (Implicit)

- VUID-vkCmdDrawIndirectByteCountEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDrawIndirectByteCountEXT-counterBuffer-parameter
  counterBuffer **must** be a valid VkBuffer handle
- VUID-vkCmdDrawIndirectByteCountEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdDrawIndirectByteCountEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdDrawIndirectByteCountEXT-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdDrawIndirectByteCountEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdDrawIndirectByteCountEXT-commonparent
  Both of commandBuffer, and counterBuffer **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized

- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	Action

Chapter 22. Fixed-Function Vertex Processing

Vertex fetching is controlled via configurable state, as a logically distinct graphics pipeline stage.

22.1. Vertex Attributes

Vertex shaders **can** define input variables, which receive *vertex attribute* data transferred from one or more VkBuffer(s) by drawing commands. Vertex shader input variables are bound to buffers via an indirect binding where the vertex shader associates a *vertex input attribute* number with each variable, vertex input attributes are associated to *vertex input bindings* on a per-pipeline basis, and vertex input bindings are associated with specific buffers on a per-draw basis via the vkCmdBindVertexBuffers command. Vertex input attribute and vertex input binding descriptions also contain format information controlling how data is extracted from buffer memory and converted to the format expected by the vertex shader.

There are VkPhysicalDeviceLimits::maxVertexInputAttributes number of vertex input attributes and VkPhysicalDeviceLimits::maxVertexInputBindings number of vertex input bindings (each referred to by zero-based indices), where there are at least as many vertex input attributes as there are vertex input bindings. Applications **can** store multiple vertex input attributes interleaved in a single buffer, and use a single vertex input binding to access those attributes.

In GLSL, vertex shaders associate input variables with a vertex input attribute number using the location layout qualifier. The Component layout qualifier associates components of a vertex shader input variable with components of a vertex input attribute.

GLSL Example

```
// Assign location M to variableName
layout (location=M, component=2) in vec2 variableName;

// Assign locations [N,N+L) to the array elements of variableNameArray
layout (location=N) in vec4 variableNameArray[L];
```

In SPIR-V, vertex shaders associate input variables with a vertex input attribute number using the Location decoration. The Component decoration associates components of a vertex shader input variable with components of a vertex input attribute. The Location and Component decorations are specified via the OpDecorate instruction.

SPIR-V Example

```
          ...
     %1 = OpExtInstImport "GLSL.std.450"
          ...
          OpName %9 "variableName"
          OpName %15 "variableNameArray"
          OpDecorate %18 BuiltIn VertexIndex
```

```
               OpDecorate %19 BuiltIn InstanceIndex
               OpDecorate %9 Location M
               OpDecorate %9 Component 2
               OpDecorate %15 Location N
               ...
        %2 = OpTypeVoid
        %3 = OpTypeFunction %2
        %6 = OpTypeFloat 32
        %7 = OpTypeVector %6 2
        %8 = OpTypePointer Input %7
        %9 = OpVariable %8 Input
       %10 = OpTypeVector %6 4
       %11 = OpTypeInt 32 0
       %12 = OpConstant %11 L
       %13 = OpTypeArray %10 %12
       %14 = OpTypePointer Input %13
       %15 = OpVariable %14 Input
               ...
```

22.1.1. Attribute Location and Component Assignment

The `Location` decoration specifies which vertex input attribute is used to read and interpret the data that a variable will consume.

When a vertex shader input variable declared using a 16- or 32-bit scalar or vector data type is assigned a `Location`, its value(s) are taken from the components of the input attribute specified with the corresponding `VkVertexInputAttributeDescription`::`location`. The components used depend on the type of variable and the `Component` decoration specified in the variable declaration, as identified in Input Attribute Components Accessed By 16-Bit and 32-Bit Input Variables. Any 16-bit or 32-bit scalar or vector input will consume a single `Location`. For 16-bit and 32-bit data types, missing components are filled in with default values as described below.

If an implementation supports `storageInputOutput16`, vertex shader input variables **can** have a width of 16 bits.

Table 27. Input Attribute Components Accessed By 16-Bit and 32-Bit Input Variables

16-bit or 32-bit data type	`Component` **decoration**	**Components consumed**
scalar	0 or unspecified	(x, 0, 0, 0)
scalar	1	(0, y, 0, 0)
scalar	2	(0, 0, z, 0)
scalar	3	(0, 0, 0, w)
two-component vector	0 or unspecified	(x, y, 0, 0)
two-component vector	1	(0, y, z, 0)
two-component vector	2	(0, 0, z, w)

16-bit or 32-bit data type	`Component` decoration	Components consumed
three-component vector	0 or unspecified	(x, y, z, o)
three-component vector	1	(o, y, z, w)
four-component vector	0 or unspecified	(x, y, z, w)

Components indicated by “o” are available for use by other input variables which are sourced from the same attribute, and if used, are either filled with the corresponding component from the input format (if present), or the default value.

When a vertex shader input variable declared using a 32-bit floating-point matrix type is assigned a `Location` *i*, its values are taken from consecutive input attributes starting with the corresponding `VkVertexInputAttributeDescription`::`location`. Such matrices are treated as an array of column vectors with values taken from the input attributes identified in Input Attributes Accessed by 32-Bit Input Matrix Variables. The `VkVertexInputAttributeDescription`::`format` **must** be specified with a VkFormat that corresponds to the appropriate type of column vector. The `Component` decoration **must** not be used with matrix types.

Table 28. Input Attributes Accessed by 32-Bit Input Matrix Variables

Data type	Column vector type	Locations consumed	Components consumed
mat2	two-component vector	i, i+1	(x, y, o, o), (x, y, o, o)
mat2x3	three-component vector	i, i+1	(x, y, z, o), (x, y, z, o)
mat2x4	four-component vector	i, i+1	(x, y, z, w), (x, y, z, w)
mat3x2	two-component vector	i, i+1, i+2	(x, y, o, o), (x, y, o, o), (x, y, o, o)
mat3	three-component vector	i, i+1, i+2	(x, y, z, o), (x, y, z, o), (x, y, z, o)
mat3x4	four-component vector	i, i+1, i+2	(x, y, z, w), (x, y, z, w), (x, y, z, w)
mat4x2	two-component vector	i, i+1, i+2, i+3	(x, y, o, o), (x, y, o, o), (x, y, o, o), (x, y, o, o)
mat4x3	three-component vector	i, i+1, i+2, i+3	(x, y, z, o), (x, y, z, o), (x, y, z, o), (x, y, z, o)
mat4	four-component vector	i, i+1, i+2, i+3	(x, y, z, w), (x, y, z, w), (x, y, z, w), (x, y, z, w)

Components indicated by “o” are available for use by other input variables which are sourced from the same attribute, and if used, are either filled with the corresponding component from the input (if present), or the default value.

When a vertex shader input variable declared using a scalar or vector 64-bit data type is assigned a `Location` *i*, its values are taken from consecutive input attributes starting with the corresponding

VkVertexInputAttributeDescription::location. The Location slots and Component words used depend on the type of variable and the Component decoration specified in the variable declaration, as identified in Input Attribute Locations and Components Accessed by 64-Bit Input Variables. For 64-bit data types, no default attribute values are provided. Input variables **must** not use more components than provided by the attribute.

Table 29. Input Attribute Locations and Components Accessed by 64-Bit Input Variables

Input format	Locations consumed	64-bit data type	Location decoration	Component decoration	32-bit components consumed
R64	i	scalar	i	0 or unspecified	(x, y, -, -)
R64G64	i	scalar	i	0 or unspecified	(x, y, o, o)
		scalar	i	2	(o, o, z, w)
		two-component vector	i	0 or unspecified	(x, y, z, w)
R64G64B64	i, i+1	scalar	i	0 or unspecified	(x, y, o, o), (o, o, -, -)
		scalar	i	2	(o, o, z, w), (o, o, -, -)
		scalar	i+1	0 or unspecified	(o, o, o, o), (x, y, -, -)
		two-component vector	i	0 or unspecified	(x, y, z, w), (o, o, -, -)
		three-component vector	i	unspecified	(x, y, z, w), (x, y, -, -)
R64G64B64A64	i, i+1	scalar	i	0 or unspecified	(x, y, o, o), (o, o, o, o)
		scalar	i	2	(o, o, z, w), (o, o, o, o)
		scalar	i+1	0 or unspecified	(o, o, o, o), (x, y, o, o)
		scalar	i+1	2	(o, o, o, o), (o, o, z, w)
		two-component vector	i	0 or unspecified	(x, y, z, w), (o, o, o, o)
		two-component vector	i+1	0 or unspecified	(o, o, o, o), (x, y, z, w)
		three-component vector	i	unspecified	(x, y, z, w), (x, y, o, o)
		four-component vector	i	unspecified	(x, y, z, w), (x, y, z, w)

Components indicated by “o” are available for use by other input variables which are sourced from the same attribute. Components indicated by “-” are not available for input variables as there are no default values provided for 64-bit data types, and there is no data provided by the input format.

When a vertex shader input variable declared using a 64-bit floating-point matrix type is assigned a `Location` *i*, its values are taken from consecutive input attribute locations. Such matrices are treated as an array of column vectors with values taken from the input attributes as shown in Input Attribute Locations and Components Accessed by 64-Bit Input Variables. Each column vector starts at the `Location` immediately following the last `Location` of the previous column vector. The number of attributes and components assigned to each matrix is determined by the matrix dimensions and ranges from two to eight locations.

When a vertex shader input variable declared using an array type is assigned a location, its values are taken from consecutive input attributes starting with the corresponding `VkVertexInputAttributeDescription`::`location`. The number of attributes and components assigned to each element are determined according to the data type of the array elements and `Component` decoration (if any) specified in the declaration of the array, as described above. Each element of the array, in order, is assigned to consecutive locations, but all at the same specified component within each location.

Only input variables declared with the data types and component decorations as specified above are supported. Two variables are allowed to share the same `Location` slot only if their `Component` words do not overlap. If multiple variables share the same `Location` slot, they **must** all have the same SPIR-V floating-point component type or all have the same width scalar type components.

22.2. Vertex Input Description

Applications specify vertex input attribute and vertex input binding descriptions as part of graphics pipeline creation by setting the VkGraphicsPipelineCreateInfo::`pVertexInputState` pointer to a VkPipelineVertexInputStateCreateInfo structure. Alternatively, if the graphics pipeline is created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic state enabled, then the vertex input attribute and vertex input binding descriptions are specified dynamically with vkCmdSetVertexInputEXT, and the VkGraphicsPipelineCreateInfo::`pVertexInputState` pointer is ignored.

The `VkPipelineVertexInputStateCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineVertexInputStateCreateInfo {
    VkStructureType                             sType;
    const void*                                 pNext;
    VkPipelineVertexInputStateCreateFlags       flags;
    uint32_t                                    vertexBindingDescriptionCount;
    const VkVertexInputBindingDescription*      pVertexBindingDescriptions;
    uint32_t                                    vertexAttributeDescriptionCount;
    const VkVertexInputAttributeDescription*    pVertexAttributeDescriptions;
} VkPipelineVertexInputStateCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `vertexBindingDescriptionCount` is the number of vertex binding descriptions provided in `pVertexBindingDescriptions`.
- `pVertexBindingDescriptions` is a pointer to an array of VkVertexInputBindingDescription structures.
- `vertexAttributeDescriptionCount` is the number of vertex attribute descriptions provided in `pVertexAttributeDescriptions`.
- `pVertexAttributeDescriptions` is a pointer to an array of VkVertexInputAttributeDescription structures.

Valid Usage

- VUID-VkPipelineVertexInputStateCreateInfo-vertexBindingDescriptionCount-00613
  `vertexBindingDescriptionCount` **must** be less than or equal to `VkPhysicalDeviceLimits::maxVertexInputBindings`
- VUID-VkPipelineVertexInputStateCreateInfo-vertexAttributeDescriptionCount-00614
  `vertexAttributeDescriptionCount` **must** be less than or equal to `VkPhysicalDeviceLimits::maxVertexInputAttributes`
- VUID-VkPipelineVertexInputStateCreateInfo-binding-00615
  For every `binding` specified by each element of `pVertexAttributeDescriptions`, a VkVertexInputBindingDescription **must** exist in `pVertexBindingDescriptions` with the same value of `binding`
- VUID-VkPipelineVertexInputStateCreateInfo-pVertexBindingDescriptions-00616
  All elements of `pVertexBindingDescriptions` **must** describe distinct binding numbers
- VUID-VkPipelineVertexInputStateCreateInfo-pVertexAttributeDescriptions-00617
  All elements of `pVertexAttributeDescriptions` **must** describe distinct attribute locations

Valid Usage (Implicit)

- VUID-VkPipelineVertexInputStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO`
- VUID-VkPipelineVertexInputStateCreateInfo-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineVertexInputDivisorStateCreateInfoKHR
- VUID-VkPipelineVertexInputStateCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPipelineVertexInputStateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineVertexInputStateCreateInfo-pVertexBindingDescriptions-parameter
  If `vertexBindingDescriptionCount` is not `0`, `pVertexBindingDescriptions` **must** be a valid

pointer to an array of vertexBindingDescriptionCount valid VkVertexInputBindingDescription structures

- VUID-VkPipelineVertexInputStateCreateInfo-pVertexAttributeDescriptions-parameter
  If vertexAttributeDescriptionCount is not 0, pVertexAttributeDescriptions **must** be a valid pointer to an array of vertexAttributeDescriptionCount valid VkVertexInputAttributeDescription structures

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineVertexInputStateCreateFlags;
```

VkPipelineVertexInputStateCreateFlags is a bitmask type for setting a mask, but is currently reserved for future use.

Each vertex input binding is specified by the VkVertexInputBindingDescription structure, defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkVertexInputBindingDescription {
    uint32_t             binding;
    uint32_t             stride;
    VkVertexInputRate    inputRate;
} VkVertexInputBindingDescription;
```

- binding is the binding number that this structure describes.
- stride is the byte stride between consecutive elements within the buffer.
- inputRate is a VkVertexInputRate value specifying whether vertex attribute addressing is a function of the vertex index or of the instance index.

Valid Usage

- VUID-VkVertexInputBindingDescription-binding-00618
  binding **must** be less than VkPhysicalDeviceLimits::maxVertexInputBindings
- VUID-VkVertexInputBindingDescription-stride-00619
  stride **must** be less than or equal to VkPhysicalDeviceLimits::maxVertexInputBindingStride
- VUID-VkVertexInputBindingDescription-stride-04456
  If the VK_KHR_portability_subset extension is enabled, stride **must** be a multiple of, and at least as large as, VkPhysicalDevicePortabilitySubsetPropertiesKHR::minVertexInputBindingStrideAlignment

Valid Usage (Implicit)

- VUID-VkVertexInputBindingDescription-inputRate-parameter
  inputRate **must** be a valid VkVertexInputRate value

Possible values of VkVertexInputBindingDescription::inputRate, specifying the rate at which vertex attributes are pulled from buffers, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkVertexInputRate {
    VK_VERTEX_INPUT_RATE_VERTEX = 0,
    VK_VERTEX_INPUT_RATE_INSTANCE = 1,
} VkVertexInputRate;
```

- VK_VERTEX_INPUT_RATE_VERTEX specifies that vertex attribute addressing is a function of the vertex index.
- VK_VERTEX_INPUT_RATE_INSTANCE specifies that vertex attribute addressing is a function of the instance index.

Each vertex input attribute is specified by the VkVertexInputAttributeDescription structure, defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkVertexInputAttributeDescription {
    uint32_t    location;
    uint32_t    binding;
    VkFormat    format;
    uint32_t    offset;
} VkVertexInputAttributeDescription;
```

- location is the shader input location number for this attribute.
- binding is the binding number which this attribute takes its data from.
- format is the size and type of the vertex attribute data.
- offset is a byte offset of this attribute relative to the start of an element in the vertex input binding.

Valid Usage

- VUID-VkVertexInputAttributeDescription-location-00620
  location **must** be less than VkPhysicalDeviceLimits::maxVertexInputAttributes
- VUID-VkVertexInputAttributeDescription-binding-00621
  binding **must** be less than VkPhysicalDeviceLimits::maxVertexInputBindings
- VUID-VkVertexInputAttributeDescription-offset-00622
  offset **must** be less than or equal to VkPhysicalDeviceLimits::maxVertexInputAttributeOffset
- VUID-VkVertexInputAttributeDescription-format-00623
  The format features of format **must** contain VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT
- VUID-VkVertexInputAttributeDescription-vertexAttributeAccessBeyondStride-04457

If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::vertexAttributeAccessBeyondStride is VK_FALSE, the sum of offset plus the size of the vertex attribute data described by format **must** not be greater than stride in the VkVertexInputBindingDescription referenced in binding

Valid Usage (Implicit)

- VUID-VkVertexInputAttributeDescription-format-parameter
  format **must** be a valid VkFormat value

To dynamically set the vertex input attribute and vertex input binding descriptions, call:

```
// Provided by VK_EXT_shader_object, VK_EXT_vertex_input_dynamic_state
void vkCmdSetVertexInputEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    vertexBindingDescriptionCount,
    const VkVertexInputBindingDescription2EXT*  pVertexBindingDescriptions,
    uint32_t                                    vertexAttributeDescriptionCount,
    const VkVertexInputAttributeDescription2EXT* pVertexAttributeDescriptions);
```

- commandBuffer is the command buffer into which the command will be recorded.
- vertexBindingDescriptionCount is the number of vertex binding descriptions provided in pVertexBindingDescriptions.
- pVertexBindingDescriptions is a pointer to an array of VkVertexInputBindingDescription2EXT structures.
- vertexAttributeDescriptionCount is the number of vertex attribute descriptions provided in pVertexAttributeDescriptions.
- pVertexAttributeDescriptions is a pointer to an array of VkVertexInputAttributeDescription2EXT structures.

This command sets the vertex input attribute and vertex input binding descriptions state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_VERTEX_INPUT_EXT set in VkPipelineDynamicStateCreateInfo ::pDynamicStates. Otherwise, this state is specified by the VkGraphicsPipelineCreateInfo ::pVertexInputState values used to create the currently active pipeline.

If drawing using shader objects, then vkCmdBindVertexBuffers2EXT can be used instead of vkCmdSetVertexInputEXT to dynamically set the stride.

The vertex attribute description for any location in the range [0,VkPhysicalDeviceLimits ::maxVertexInputAttributes) not specified in the pVertexAttributeDescriptions array becomes undefined.

Valid Usage

- VUID-vkCmdSetVertexInputEXT-None-08546
  Either the `vertexInputDynamicState` feature or the `shaderObject` feature or both **must** be enabled
- VUID-vkCmdSetVertexInputEXT-vertexBindingDescriptionCount-04791
  `vertexBindingDescriptionCount` **must** be less than or equal to `VkPhysicalDeviceLimits::maxVertexInputBindings`
- VUID-vkCmdSetVertexInputEXT-vertexAttributeDescriptionCount-04792
  `vertexAttributeDescriptionCount` **must** be less than or equal to `VkPhysicalDeviceLimits::maxVertexInputAttributes`
- VUID-vkCmdSetVertexInputEXT-binding-04793
  For every `binding` specified by each element of `pVertexAttributeDescriptions`, a VkVertexInputBindingDescription2EXT **must** exist in `pVertexBindingDescriptions` with the same value of `binding`
- VUID-vkCmdSetVertexInputEXT-pVertexBindingDescriptions-04794
  All elements of `pVertexBindingDescriptions` **must** describe distinct binding numbers
- VUID-vkCmdSetVertexInputEXT-pVertexAttributeDescriptions-04795
  All elements of `pVertexAttributeDescriptions` **must** describe distinct attribute locations

Valid Usage (Implicit)

- VUID-vkCmdSetVertexInputEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetVertexInputEXT-pVertexBindingDescriptions-parameter
  If `vertexBindingDescriptionCount` is not `0`, `pVertexBindingDescriptions` **must** be a valid pointer to an array of `vertexBindingDescriptionCount` valid VkVertexInputBindingDescription2EXT structures
- VUID-vkCmdSetVertexInputEXT-pVertexAttributeDescriptions-parameter
  If `vertexAttributeDescriptionCount` is not `0`, `pVertexAttributeDescriptions` **must** be a valid pointer to an array of `vertexAttributeDescriptionCount` valid VkVertexInputAttributeDescription2EXT structures
- VUID-vkCmdSetVertexInputEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetVertexInputEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetVertexInputEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

The `VkVertexInputBindingDescription2EXT` structure is defined as:

```
// Provided by VK_EXT_shader_object, VK_EXT_vertex_input_dynamic_state
typedef struct VkVertexInputBindingDescription2EXT {
    VkStructureType      sType;
    void*                pNext;
    uint32_t             binding;
    uint32_t             stride;
    VkVertexInputRate    inputRate;
    uint32_t             divisor;
} VkVertexInputBindingDescription2EXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `binding` is the binding number that this structure describes.
- `stride` is the byte stride between consecutive elements within the buffer.
- `inputRate` is a VkVertexInputRate value specifying whether vertex attribute addressing is a function of the vertex index or of the instance index.
- `divisor` is the number of successive instances that will use the same value of the vertex attribute when instanced rendering is enabled. This member **can** be a value other than `1` if the `vertexAttributeInstanceRateDivisor` feature is enabled. For example, if the divisor is N, the same vertex attribute will be applied to N successive instances before moving on to the next vertex attribute. The maximum value of `divisor` is implementation-dependent and can be queried using `VkPhysicalDeviceVertexAttributeDivisorPropertiesEXT`::`maxVertexAttribDivisor`. A value of `0` **can** be used for the divisor if the `vertexAttributeInstanceRateZeroDivisor` feature is enabled. In this case, the same vertex attribute will be applied to all instances.

Valid Usage

- VUID-VkVertexInputBindingDescription2EXT-binding-04796
  `binding` **must** be less than `VkPhysicalDeviceLimits`::`maxVertexInputBindings`
- VUID-VkVertexInputBindingDescription2EXT-stride-04797
  `stride` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxVertexInputBindingStride`
- VUID-VkVertexInputBindingDescription2EXT-divisor-04798
  If the `vertexAttributeInstanceRateZeroDivisor` feature is not enabled, `divisor` **must** not be `0`
- VUID-VkVertexInputBindingDescription2EXT-divisor-04799
  If the `vertexAttributeInstanceRateDivisor` feature is not enabled, `divisor` **must** be `1`
- VUID-VkVertexInputBindingDescription2EXT-divisor-06226
  `divisor` **must** be a value between `0` and `VkPhysicalDeviceVertexAttributeDivisorPropertiesEXT`::`maxVertexAttribDivisor`, inclusive
- VUID-VkVertexInputBindingDescription2EXT-divisor-06227
  If `divisor` is not `1` then `inputRate` **must** be of type `VK_VERTEX_INPUT_RATE_INSTANCE`

Valid Usage (Implicit)

- VUID-VkVertexInputBindingDescription2EXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VERTEX_INPUT_BINDING_DESCRIPTION_2_EXT`
- VUID-VkVertexInputBindingDescription2EXT-inputRate-parameter
  `inputRate` **must** be a valid VkVertexInputRate value

The `VkVertexInputAttributeDescription2EXT` structure is defined as:

```
// Provided by VK_EXT_shader_object, VK_EXT_vertex_input_dynamic_state
typedef struct VkVertexInputAttributeDescription2EXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           location;
    uint32_t           binding;
    VkFormat           format;
    uint32_t           offset;
} VkVertexInputAttributeDescription2EXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `location` is the shader input location number for this attribute.
- `binding` is the binding number which this attribute takes its data from.
- `format` is the size and type of the vertex attribute data.

- `offset` is a byte offset of this attribute relative to the start of an element in the vertex input binding.

Valid Usage

- VUID-VkVertexInputAttributeDescription2EXT-location-06228
  `location` **must** be less than `VkPhysicalDeviceLimits`::`maxVertexInputAttributes`
- VUID-VkVertexInputAttributeDescription2EXT-binding-06229
  `binding` **must** be less than `VkPhysicalDeviceLimits`::`maxVertexInputBindings`
- VUID-VkVertexInputAttributeDescription2EXT-offset-06230
  `offset` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxVertexInputAttributeOffset`
- VUID-VkVertexInputAttributeDescription2EXT-format-04805
  The format features of `format` **must** contain `VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT`
- VUID-VkVertexInputAttributeDescription2EXT-vertexAttributeAccessBeyondStride-04806
  If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`vertexAttributeAccessBeyondStride` is `VK_FALSE`, the sum of `offset` plus the size of the vertex attribute data described by `format` **must** not be greater than `stride` in the VkVertexInputBindingDescription2EXT referenced in `binding`

Valid Usage (Implicit)

- VUID-VkVertexInputAttributeDescription2EXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VERTEX_INPUT_ATTRIBUTE_DESCRIPTION_2_EXT`
- VUID-VkVertexInputAttributeDescription2EXT-format-parameter
  `format` **must** be a valid VkFormat value

To bind vertex buffers to a command buffer for use in subsequent drawing commands, call:

```
// Provided by VK_VERSION_1_0
void vkCmdBindVertexBuffers(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstBinding,
    uint32_t                                    bindingCount,
    const VkBuffer*                             pBuffers,
    const VkDeviceSize*                         pOffsets);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `firstBinding` is the index of the first vertex input binding whose state is updated by the command.
- `bindingCount` is the number of vertex input bindings whose state is updated by the command.

- `pBuffers` is a pointer to an array of buffer handles.
- `pOffsets` is a pointer to an array of buffer offsets.

The values taken from elements i of `pBuffers` and `pOffsets` replace the current state for the vertex input binding `firstBinding` + i, for i in [0, `bindingCount`). The vertex input binding is updated to start at the offset indicated by `pOffsets`[i] from the start of the buffer `pBuffers`[i]. All vertex input attributes that use each of these bindings will use these updated addresses in their address calculations for subsequent drawing commands. If the `nullDescriptor` feature is enabled, elements of `pBuffers` **can** be VK_NULL_HANDLE, and **can** be used by the vertex shader. If a vertex input attribute is bound to a vertex input binding that is VK_NULL_HANDLE, the values taken from memory are considered to be zero, and missing G, B, or A components are filled with (0,0,1).

Valid Usage

- VUID-vkCmdBindVertexBuffers-firstBinding-00624
  `firstBinding` **must** be less than `VkPhysicalDeviceLimits`::`maxVertexInputBindings`
- VUID-vkCmdBindVertexBuffers-firstBinding-00625
  The sum of `firstBinding` and `bindingCount` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxVertexInputBindings`
- VUID-vkCmdBindVertexBuffers-pOffsets-00626
  All elements of `pOffsets` **must** be less than the size of the corresponding element in `pBuffers`
- VUID-vkCmdBindVertexBuffers-pBuffers-00627
  All elements of `pBuffers` **must** have been created with the `VK_BUFFER_USAGE_VERTEX_BUFFER_BIT` flag
- VUID-vkCmdBindVertexBuffers-pBuffers-00628
  Each element of `pBuffers` that is non-sparse **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdBindVertexBuffers-pBuffers-04001
  If the `nullDescriptor` feature is not enabled, all elements of `pBuffers` **must** not be VK_NULL_HANDLE
- VUID-vkCmdBindVertexBuffers-pBuffers-04002
  If an element of `pBuffers` is VK_NULL_HANDLE, then the corresponding element of `pOffsets` **must** be zero

Valid Usage (Implicit)

- VUID-vkCmdBindVertexBuffers-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindVertexBuffers-pBuffers-parameter
  `pBuffers` **must** be a valid pointer to an array of `bindingCount` valid or VK_NULL_HANDLE VkBuffer handles
- VUID-vkCmdBindVertexBuffers-pOffsets-parameter

pOffsets **must** be a valid pointer to an array of bindingCount VkDeviceSize values

- VUID-vkCmdBindVertexBuffers-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdBindVertexBuffers-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdBindVertexBuffers-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBindVertexBuffers-bindingCount-arraylength
  bindingCount **must** be greater than 0
- VUID-vkCmdBindVertexBuffers-commonparent
  Both of commandBuffer, and the elements of pBuffers that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Alternatively, to bind vertex buffers, along with their sizes and strides, to a command buffer for use in subsequent drawing commands, call:

```
// Provided by VK_EXT_shader_object
void vkCmdBindVertexBuffers2EXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstBinding,
    uint32_t                                    bindingCount,
    const VkBuffer*                             pBuffers,
    const VkDeviceSize*                         pOffsets,
    const VkDeviceSize*                         pSizes,
    const VkDeviceSize*                         pStrides);
```

- commandBuffer is the command buffer into which the command is recorded.

- `firstBinding` is the index of the first vertex input binding whose state is updated by the command.
- `bindingCount` is the number of vertex input bindings whose state is updated by the command.
- `pBuffers` is a pointer to an array of buffer handles.
- `pOffsets` is a pointer to an array of buffer offsets.
- `pSizes` is `NULL` or a pointer to an array of the size in bytes of vertex data bound from `pBuffers`.
- `pStrides` is `NULL` or a pointer to an array of buffer strides.

The values taken from elements i of `pBuffers` and `pOffsets` replace the current state for the vertex input binding `firstBinding` + i, for i in [0, `bindingCount`). The vertex input binding is updated to start at the offset indicated by `pOffsets`[i] from the start of the buffer `pBuffers`[i]. If `pSizes` is not `NULL` then `pSizes`[i] specifies the bound size of the vertex buffer starting from the corresponding elements of `pBuffers`[i] plus `pOffsets`[i]. If `pSizes`[i] is `VK_WHOLE_SIZE` then the bound size is from `pBuffers`[i] plus `pOffsets`[i] to the end of the buffer `pBuffers`[i]. All vertex input attributes that use each of these bindings will use these updated addresses in their address calculations for subsequent drawing commands. If the `nullDescriptor` feature is enabled, elements of `pBuffers` **can** be VK_NULL_HANDLE, and **can** be used by the vertex shader. If a vertex input attribute is bound to a vertex input binding that is VK_NULL_HANDLE, the values taken from memory are considered to be zero, and missing G, B, or A components are filled with (0,0,1).

This command also dynamically sets the byte strides between consecutive elements within buffer `pBuffers`[i] to the corresponding `pStrides`[i] value when drawing using shader objects. Otherwise, strides are specified by the VkVertexInputBindingDescription::`stride` values used to create the currently active pipeline.

If drawing using shader objects or if the bound pipeline state object was also created with the `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT` dynamic state enabled then vkCmdSetVertexInputEXT **can** be used instead of `vkCmdBindVertexBuffers2EXT` to set the stride.

NOTE	Unlike the static state to set the same, `pStrides` must be between 0 and the maximum extent of the attributes in the binding. vkCmdSetVertexInputEXT does not have this restriction so can be used if other stride values are desired.

Valid Usage

- VUID-vkCmdBindVertexBuffers2-firstBinding-03355
  `firstBinding` **must** be less than `VkPhysicalDeviceLimits`::`maxVertexInputBindings`
- VUID-vkCmdBindVertexBuffers2-firstBinding-03356
  The sum of `firstBinding` and `bindingCount` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxVertexInputBindings`
- VUID-vkCmdBindVertexBuffers2-pOffsets-03357
  If `pSizes` is not `NULL`, all elements of `pOffsets` **must** be less than the size of the corresponding element in `pBuffers`
- VUID-vkCmdBindVertexBuffers2-pSizes-03358
  If `pSizes` is not `NULL`, all elements of `pOffsets` plus `pSizes` , where `pSizes` is not

VK_WHOLE_SIZE, **must** be less than or equal to the size of the corresponding element in pBuffers

- VUID-vkCmdBindVertexBuffers2-pBuffers-03359
  All elements of pBuffers **must** have been created with the VK_BUFFER_USAGE_VERTEX_BUFFER_BIT flag
- VUID-vkCmdBindVertexBuffers2-pBuffers-03360
  Each element of pBuffers that is non-sparse **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBindVertexBuffers2-pBuffers-04111
  If the nullDescriptor feature is not enabled, all elements of pBuffers **must** not be VK_NULL_HANDLE
- VUID-vkCmdBindVertexBuffers2-pBuffers-04112
  If an element of pBuffers is VK_NULL_HANDLE, then the corresponding element of pOffsets **must** be zero
- VUID-vkCmdBindVertexBuffers2-pStrides-03362
  If pStrides is not NULL each element of pStrides **must** be less than or equal to VkPhysicalDeviceLimits::maxVertexInputBindingStride
- VUID-vkCmdBindVertexBuffers2-pStrides-06209
  If pStrides is not NULL each element of pStrides **must** be either 0 or greater than or equal to the maximum extent of all vertex input attributes fetched from the corresponding binding, where the extent is calculated as the VkVertexInputAttributeDescription::offset plus VkVertexInputAttributeDescription::format size

Valid Usage (Implicit)

- VUID-vkCmdBindVertexBuffers2-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindVertexBuffers2-pBuffers-parameter
  pBuffers **must** be a valid pointer to an array of bindingCount valid or VK_NULL_HANDLE VkBuffer handles
- VUID-vkCmdBindVertexBuffers2-pOffsets-parameter
  pOffsets **must** be a valid pointer to an array of bindingCount VkDeviceSize values
- VUID-vkCmdBindVertexBuffers2-pSizes-parameter
  If pSizes is not NULL, pSizes **must** be a valid pointer to an array of bindingCount VkDeviceSize values
- VUID-vkCmdBindVertexBuffers2-pStrides-parameter
  If pStrides is not NULL, pStrides **must** be a valid pointer to an array of bindingCount VkDeviceSize values
- VUID-vkCmdBindVertexBuffers2-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdBindVertexBuffers2-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics

operations

- VUID-vkCmdBindVertexBuffers2-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBindVertexBuffers2-bindingCount-arraylength
  If any of `pSizes`, or `pStrides` are not `NULL`, `bindingCount` **must** be greater than `0`
- VUID-vkCmdBindVertexBuffers2-commonparent
  Both of `commandBuffer`, and the elements of `pBuffers` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

22.3. Vertex Attribute Divisor in Instanced Rendering

If the `vertexAttributeInstanceRateDivisor` feature is enabled and the `pNext` chain of VkPipelineVertexInputStateCreateInfo includes a `VkPipelineVertexInputDivisorStateCreateInfoKHR` structure, then that structure controls how vertex attributes are assigned to an instance when instanced rendering is enabled.

The `VkPipelineVertexInputDivisorStateCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_vertex_attribute_divisor
typedef struct VkPipelineVertexInputDivisorStateCreateInfoKHR {
    VkStructureType                                    sType;
    const void*                                        pNext;
    uint32_t                                           vertexBindingDivisorCount;
    const VkVertexInputBindingDivisorDescriptionKHR*   pVertexBindingDivisors;
} VkPipelineVertexInputDivisorStateCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.

- `vertexBindingDivisorCount` is the number of elements in the `pVertexBindingDivisors` array.
- `pVertexBindingDivisors` is a pointer to an array of VkVertexInputBindingDivisorDescriptionKHR structures specifying the divisor value for each binding.

Valid Usage (Implicit)

- VUID-VkPipelineVertexInputDivisorStateCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_DIVISOR_STATE_CREATE_INFO_KHR`
- VUID-VkPipelineVertexInputDivisorStateCreateInfoKHR-pVertexBindingDivisors-parameter
  `pVertexBindingDivisors` **must** be a valid pointer to an array of `vertexBindingDivisorCount` VkVertexInputBindingDivisorDescriptionKHR structures
- VUID-VkPipelineVertexInputDivisorStateCreateInfoKHR-vertexBindingDivisorCount-arraylength
  `vertexBindingDivisorCount` **must** be greater than `0`

The individual divisor values per binding are specified using the `VkVertexInputBindingDivisorDescriptionKHR` structure which is defined as:

```
// Provided by VK_KHR_vertex_attribute_divisor
typedef struct VkVertexInputBindingDivisorDescriptionKHR {
    uint32_t    binding;
    uint32_t    divisor;
} VkVertexInputBindingDivisorDescriptionKHR;
```

- `binding` is the binding number for which the divisor is specified.
- `divisor` is the number of successive instances that will use the same value of the vertex attribute when instanced rendering is enabled. For example, if the divisor is N, the same vertex attribute will be applied to N successive instances before moving on to the next vertex attribute. The maximum value of `divisor` is implementation-dependent and can be queried using VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::`maxVertexAttribDivisor`. A value of `0` **can** be used for the divisor if the `vertexAttributeInstanceRateZeroDivisor` feature is enabled. In this case, the same vertex attribute will be applied to all instances.

If this structure is not used to define a divisor value for an attribute, then the divisor has a logical default value of 1.

Valid Usage

- VUID-VkVertexInputBindingDivisorDescriptionKHR-binding-01869
  `binding` **must** be less than VkPhysicalDeviceLimits::`maxVertexInputBindings`
- VUID-VkVertexInputBindingDivisorDescriptionKHR-vertexAttributeInstanceRateZeroDivisor-02228
  If the `vertexAttributeInstanceRateZeroDivisor` feature is not enabled, `divisor` **must** not be

0

- VUID-VkVertexInputBindingDivisorDescriptionKHR-vertexAttributeInstanceRateDivisor-02229
  If the vertexAttributeInstanceRateDivisor feature is not enabled, divisor **must** be 1
- VUID-VkVertexInputBindingDivisorDescriptionKHR-divisor-01870
  divisor **must** be a value between 0 and VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR::maxVertexAttribDivisor, inclusive
- VUID-VkVertexInputBindingDivisorDescriptionKHR-inputRate-01871
  VkVertexInputBindingDescription::inputRate **must** be of type VK_VERTEX_INPUT_RATE_INSTANCE for this binding

22.4. Vertex Input Address Calculation

The address of each attribute for each vertexIndex and instanceIndex is calculated as follows:

- Let attribDesc be the member of VkPipelineVertexInputStateCreateInfo::pVertexAttributeDescriptions with VkVertexInputAttributeDescription::location equal to the vertex input attribute number.
- Let bindingDesc be the member of VkPipelineVertexInputStateCreateInfo::pVertexBindingDescriptions with VkVertexInputAttributeDescription::binding equal to attribDesc.binding.
- Let vertexIndex be the index of the vertex within the draw (a value between firstVertex and firstVertex+vertexCount for vkCmdDraw, or a value taken from the index buffer plus vertexOffset for vkCmdDrawIndexed), and let instanceIndex be the instance number of the draw (a value between firstInstance and firstInstance+instanceCount).
- Let offset be an array of offsets into the bound vertex buffers specified during vkCmdBindVertexBuffers with pOffsets.
- Let divisor be the member of VkPipelineVertexInputDivisorStateCreateInfoKHR::pVertexBindingDivisors with VkVertexInputBindingDivisorDescriptionKHR::binding equal to attribDesc.binding. If the vertex binding state is dynamically set, instead let divisor be the member of the pVertexBindingDescriptions parameter to the vkCmdSetVertexInputEXT call with VkVertexInputBindingDescription2EXT::binding equal to attribDesc.binding.
- Let stride be the member of VkPipelineVertexInputStateCreateInfo::pVertexBindingDescriptions->stride unless there is dynamic state causing the value to be ignored. In this case the value is set from the last value from one of the following
  - vkCmdSetVertexInputEXT::pVertexBindingDescriptions->stride
  - vkCmdBindVertexBuffers2EXT::pStride, if not NULL

```
bufferBindingAddress = buffer[binding].baseAddress + offset[binding];

if (bindingDesc.inputRate == VK_VERTEX_INPUT_RATE_VERTEX)
    effectiveVertexOffset = vertexIndex * stride;
else
```

```
    if (divisor == 0)
        effectiveVertexOffset = firstInstance * stride;
    else
        effectiveVertexOffset = (firstInstance + ((instanceIndex - firstInstance) /
divisor)) * stride;

attribAddress = bufferBindingAddress + effectiveVertexOffset + attribDesc.offset;
```

22.4.1. Vertex Input Extraction

For each attribute, raw data is extracted starting at `attribAddress` and is converted from the VkVertexInputAttributeDescription's `format` to either floating-point, unsigned integer, or signed integer based on the numeric type of `format`. The numeric type of `format` **must** match the numeric type of the input variable in the shader. The input variable in the shader **must** be declared as a 64-bit data type if and only if `format` is a 64-bit data type. If either `format` is a 64-bit format or the `legacyVertexAttributes` feature is not enabled, and `format` is a packed format, `attribAddress` **must** be a multiple of the size in bytes of the whole attribute data type as described in Packed Formats. Otherwise, if either `format` is a 64-bit format or the `legacyVertexAttributes` feature is not enabled, `attribAddress` **must** be a multiple of the size in bytes of the component type indicated by `format` (see Formats). For attributes that are not 64-bit data types, each component is converted to the format of the input variable based on its type and size (as defined in the Format Definition section for each VkFormat), using the appropriate equations in 16-Bit Floating-Point Numbers, Unsigned 11-Bit Floating-Point Numbers, Unsigned 10-Bit Floating-Point Numbers, Fixed-Point Data Conversion, and Shared Exponent to RGB. Signed integer components smaller than 32 bits are sign-extended. Attributes that are not 64-bit data types are expanded to four components in the same way as described in conversion to RGBA. The number of components in the vertex shader input variable need not exactly match the number of components in the format. If the vertex shader has fewer components, the extra components are discarded.

Chapter 23. Tessellation

Tessellation involves three pipeline stages. First, a tessellation control shader transforms control points of a patch and **can** produce per-patch data. Second, a fixed-function tessellator generates multiple primitives corresponding to a tessellation of the patch in (u,v) or (u,v,w) parameter space. Third, a tessellation evaluation shader transforms the vertices of the tessellated patch, for example to compute their positions and attributes as part of the tessellated surface. The tessellator is enabled when the pipeline contains both a tessellation control shader and a tessellation evaluation shader.

23.1. Tessellator

If a pipeline includes both tessellation shaders (control and evaluation), the tessellator consumes each input patch (after vertex shading) and produces a new set of independent primitives (points, lines, or triangles). These primitives are logically produced by subdividing a geometric primitive (rectangle or triangle) according to the per-patch outer and inner tessellation levels written by the tessellation control shader. These levels are specified using the built-in variables `TessLevelOuter` and `TessLevelInner`, respectively. This subdivision is performed in an implementation-dependent manner. If no tessellation shaders are present in the pipeline, the tessellator is disabled and incoming primitives are passed through without modification.

The type of subdivision performed by the tessellator is specified by an `OpExecutionMode` instruction using one of the `Triangles`, `Quads`, or `IsoLines` execution modes. When using shader objects, this instruction **must** be specified in the tessellation evaluation shader, and **may** also be specified in the tessellation control shader. When using pipelines, this instruction **may** be specified in either the tessellation evaluation or tessellation control shader. When using shader objects, tessellation-related modes that are **required must** be specified in the tessellation evaluation shader, and **may** also be specified in the tessellation control shader. Other tessellation-related modes **may** be specified in the tessellation evaluation shader. When using pipelines, other tessellation-related execution modes **can** also be specified in either the tessellation control or tessellation evaluation shaders.

Any tessellation-related modes specified in both the tessellation control and tessellation evaluation shaders **must** be the same.

Tessellation execution modes include:

- `Triangles`, `Quads`, and `IsoLines`. These control the type of subdivision and topology of the output primitives. When using shader objects, one mode **must** be set in at least the tessellation evaluation stage. When using pipelines, one mode **must** be set in at least one of the tessellation shader stages. If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`tessellationIsolines` is `VK_FALSE`, then isoline tessellation is not supported by the implementation, and `IsoLines` **must** not be used in either tessellation shader stage.
- `VertexOrderCw` and `VertexOrderCcw`. These control the orientation of triangles generated by the tessellator. When using shader objects, one mode **must** be set in at least the tessellation evaluation stage. When using pipelines, one mode **must** be set in at least one of the tessellation

shader stages.

- PointMode. Controls generation of points rather than triangles or lines. This functionality defaults to disabled, and is enabled if either shader stage includes the execution mode. When using shader objects, if PointMode is set in the tessellation control stage, it **must** be identically set in the tessellation evaluation stage. If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::tessellationPointMode is VK_FALSE, then point mode tessellation is not supported by the implementation, and PointMode **must** not be used in either tessellation shader stage.
- SpacingEqual, SpacingFractionalEven, and SpacingFractionalOdd. Controls the spacing of segments on the edges of tessellated primitives. When using shader objects, one mode **must** be set in at least the tessellation evaluation stage. When using pipelines, one mode **must** be set in at least one of the tessellation shader stages.
- OutputVertices. Controls the size of the output patch of the tessellation control shader. When using shader objects, one value **must** be set in at least the tessellation control stage. When using pipelines, one value **must** be set in at least one of the tessellation shader stages.

For triangles, the tessellator subdivides a triangle primitive into smaller triangles. For quads, the tessellator subdivides a rectangle primitive into smaller triangles. For isolines, the tessellator subdivides a rectangle primitive into a collection of line segments arranged in strips stretching across the rectangle in the u dimension (i.e. the coordinates in TessCoord are of the form (0,x) through (1,x) for all tessellation evaluation shader invocations that share a line).

Each vertex produced by the tessellator has an associated (u,v,w) or (u,v) position in a normalized parameter space, with parameter values in the range [0,1], as illustrated in figures Domain parameterization for tessellation primitive modes (upper-left origin) and Domain parameterization for tessellation primitive modes (lower-left origin). The domain space **can** have either an upper-left or lower-left origin, selected by the domainOrigin member of VkPipelineTessellationDomainOriginStateCreateInfo.

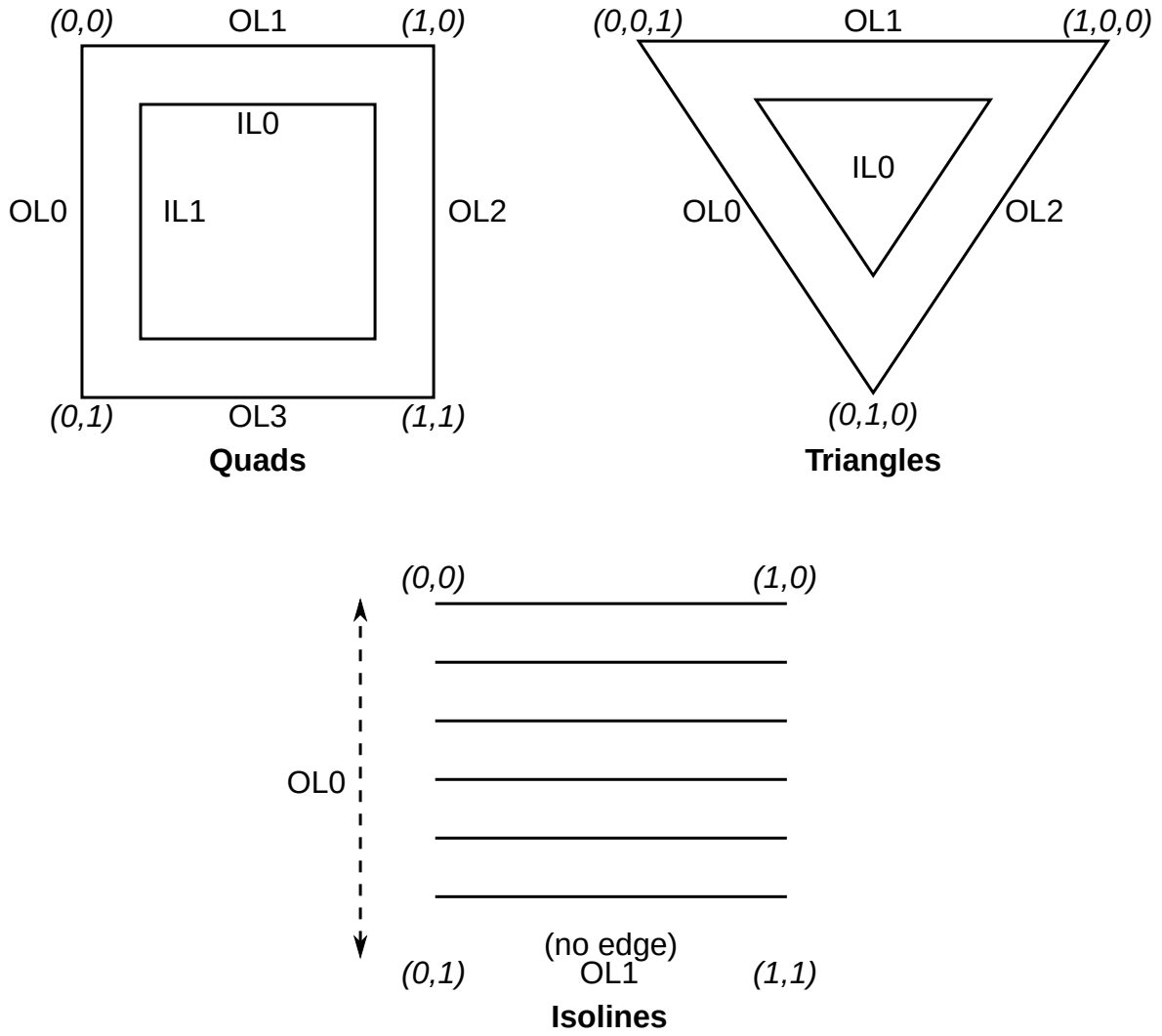


Figure 11. Domain parameterization for tessellation primitive modes (upper-left origin)

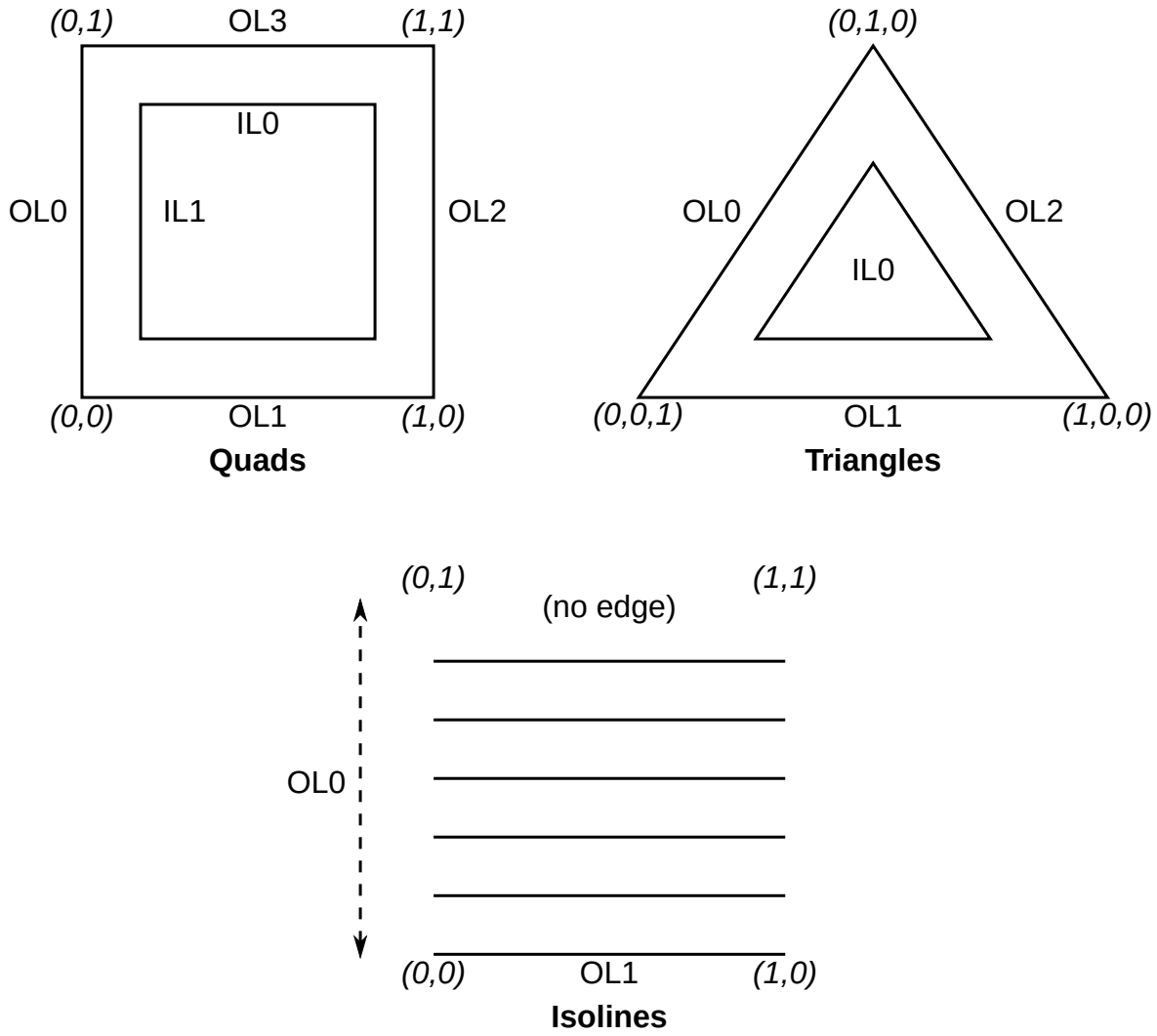


Figure 12. Domain parameterization for tessellation primitive modes (lower-left origin)

Caption

In the domain parameterization diagrams, the coordinates illustrate the value of `TessCoord` at the corners of the domain. The labels on the edges indicate the inner (IL0 and IL1) and outer (OL0 through OL3) tessellation level values used to control the number of subdivisions along each edge of the domain.

For triangles, the vertex's position is a barycentric coordinate (u,v,w), where u + v + w = 1.0, and indicates the relative influence of the three vertices of the triangle on the position of the vertex. For quads and isolines, the position is a (u,v) coordinate indicating the relative horizontal and vertical position of the vertex relative to the subdivided rectangle. The subdivision process is explained in more detail in subsequent sections.

23.2. Tessellator Patch Discard

A patch is discarded by the tessellator if any relevant outer tessellation level is less than or equal to zero.

Patches will also be discarded if any relevant outer tessellation level corresponds to a floating-point

NaN (not a number) in implementations supporting NaN.

No new primitives are generated and the tessellation evaluation shader is not executed for patches that are discarded. For `Quads`, all four outer levels are relevant. For `Triangles` and `IsoLines`, only the first three or two outer levels, respectively, are relevant. Negative inner levels will not cause a patch to be discarded; they will be clamped as described below.

23.3. Tessellator Spacing

Each of the tessellation levels is used to determine the number and spacing of segments used to subdivide a corresponding edge. The method used to derive the number and spacing of segments is specified by an `OpExecutionMode` in the tessellation control or tessellation evaluation shader using one of the identifiers `SpacingEqual`, `SpacingFractionalEven`, or `SpacingFractionalOdd`.

If `SpacingEqual` is used, the floating-point tessellation level is first clamped to [1, `maxLevel`], where `maxLevel` is the implementation-dependent maximum tessellation level (`VkPhysicalDeviceLimits ::maxTessellationGenerationLevel`). The result is rounded up to the nearest integer n, and the corresponding edge is divided into n segments of equal length in (u,v) space.

If `SpacingFractionalEven` is used, the tessellation level is first clamped to [2, `maxLevel`] and then rounded up to the nearest even integer n. If `SpacingFractionalOdd` is used, the tessellation level is clamped to [1, `maxLevel` - 1] and then rounded up to the nearest odd integer n. If n is one, the edge will not be subdivided. Otherwise, the corresponding edge will be divided into n - 2 segments of equal length, and two additional segments of equal length that are typically shorter than the other segments. The length of the two additional segments relative to the others will decrease monotonically with n - f, where f is the clamped floating-point tessellation level. When n - f is zero, the additional segments will have equal length to the other segments. As n - f approaches 2.0, the relative length of the additional segments approaches zero. The two additional segments **must** be placed symmetrically on opposite sides of the subdivided edge. The relative location of these two segments is implementation-dependent, but **must** be identical for any pair of subdivided edges with identical values of f.

When tessellating triangles or quads using point mode with fractional odd spacing, the tessellator **may** produce *interior vertices* that are positioned on the edge of the patch if an inner tessellation level is less than or equal to one. Such vertices are considered distinct from vertices produced by subdividing the outer edge of the patch, even if there are pairs of vertices with identical coordinates.

23.4. Tessellation Primitive Ordering

Few guarantees are provided for the relative ordering of primitives produced by tessellation, as they pertain to primitive order.

- The output primitives generated from each input primitive are passed to subsequent pipeline stages in an implementation-dependent order.
- All output primitives generated from a given input primitive are passed to subsequent pipeline stages before any output primitives generated from subsequent input primitives.

23.5. Tessellator Vertex Winding Order

When the tessellator produces triangles (in the `Triangles` or `Quads` modes), the orientation of all triangles is specified with an `OpExecutionMode` of `VertexOrderCw` or `VertexOrderCcw` in the tessellation control or tessellation evaluation shaders. If the order is `VertexOrderCw`, the vertices of all generated triangles will have clockwise ordering in (u,v) or (u,v,w) space. If the order is `VertexOrderCcw`, the vertices will have counter-clockwise ordering in that space.

If the tessellation domain has an upper-left origin, the vertices of a triangle have counter-clockwise ordering if

$$a = u_0 v_1 - u_1 v_0 + u_1 v_2 - u_2 v_1 + u_2 v_0 - u_0 v_2$$

is negative, and clockwise ordering if a is positive. u_i and v_i are the u and v coordinates in normalized parameter space of the ith vertex of the triangle. If the tessellation domain has a lower-left origin, the vertices of a triangle have counter-clockwise ordering if a is positive, and clockwise ordering if a is negative.

NOTE

The value a is proportional (with a positive factor) to the signed area of the triangle.

In `Triangles` mode, even though the vertex coordinates have a w value, it does not participate directly in the computation of a, being an affine combination of u and v.

23.6. Triangle Tessellation

If the tessellation primitive mode is `Triangles`, an equilateral triangle is subdivided into a collection of triangles covering the area of the original triangle. First, the original triangle is subdivided into a collection of concentric equilateral triangles. The edges of each of these triangles are subdivided, and the area between each triangle pair is filled by triangles produced by joining the vertices on the subdivided edges. The number of concentric triangles and the number of subdivisions along each triangle except the outermost is derived from the first inner tessellation level. The edges of the outermost triangle are subdivided independently, using the first, second, and third outer tessellation levels to control the number of subdivisions of the u = 0 (left), v = 0 (bottom), and w = 0 (right) edges, respectively. The second inner tessellation level and the fourth outer tessellation level have no effect in this mode.

If the first inner tessellation level and all three outer tessellation levels are exactly one after clamping and rounding, only a single triangle with (u,v,w) coordinates of (0,0,1), (1,0,0), and (0,1,0) is generated. If the inner tessellation level is one and any of the outer tessellation levels is greater than one, the inner tessellation level is treated as though it were originally specified as 1 + ε and will result in a two- or three-segment subdivision depending on the tessellation spacing. When used with fractional odd spacing, the three-segment subdivision **may** produce *inner vertices* positioned on the edge of the triangle.

If any tessellation level is greater than one, tessellation begins by producing a set of concentric inner triangles and subdividing their edges. First, the three outer edges are temporarily subdivided using the clamped and rounded first inner tessellation level and the specified tessellation spacing,

generating n segments. For the outermost inner triangle, the inner triangle is degenerate — a single point at the center of the triangle — if n is two. Otherwise, for each corner of the outer triangle, an inner triangle corner is produced at the intersection of two lines extended perpendicular to the corner's two adjacent edges running through the vertex of the subdivided outer edge nearest that corner. If n is three, the edges of the inner triangle are not subdivided and it is the final triangle in the set of concentric triangles. Otherwise, each edge of the inner triangle is divided into n - 2 segments, with the n - 1 vertices of this subdivision produced by intersecting the inner edge with lines perpendicular to the edge running through the n - 1 innermost vertices of the subdivision of the outer edge. Once the outermost inner triangle is subdivided, the previous subdivision process repeats itself, using the generated triangle as an outer triangle. This subdivision process is illustrated in Inner Triangle Tessellation.

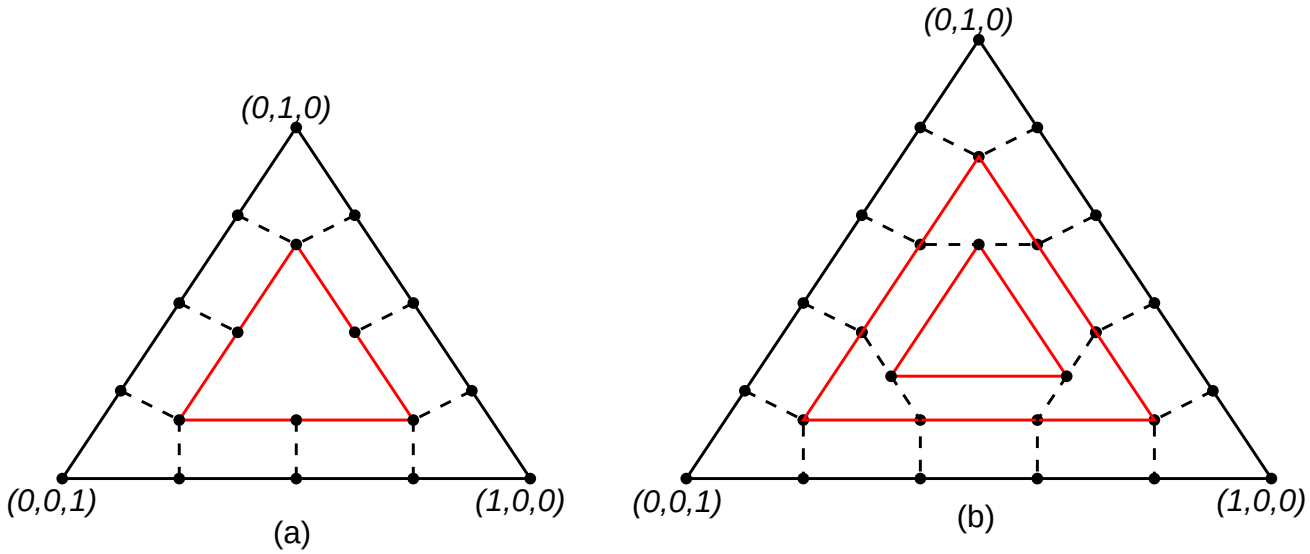


Figure 13. Inner Triangle Tessellation

Caption

In the Inner Triangle Tessellation diagram, inner tessellation levels of (a) four and (b) five are shown (not to scale). Solid black circles depict vertices along the edges of the concentric triangles. The edges of inner triangles are subdivided by intersecting the edge with segments perpendicular to the edge passing through each inner vertex of the subdivided outer edge. Dotted lines depict edges connecting corresponding vertices on the inner and outer triangle edges.

Once all the concentric triangles are produced and their edges are subdivided, the area between each pair of adjacent inner triangles is filled completely with a set of non-overlapping triangles. In this subdivision, two of the three vertices of each triangle are taken from adjacent vertices on a subdivided edge of one triangle; the third is one of the vertices on the corresponding edge of the other triangle. If the innermost triangle is degenerate (i.e., a point), the triangle containing it is subdivided into six triangles by connecting each of the six vertices on that triangle with the center point. If the innermost triangle is not degenerate, that triangle is added to the set of generated triangles as-is.

After the area corresponding to any inner triangles is filled, the tessellator generates triangles to cover the area between the outermost triangle and the outermost inner triangle. To do this, the

temporary subdivision of the outer triangle edge above is discarded. Instead, the u = 0, v = 0, and w = 0 edges are subdivided according to the first, second, and third outer tessellation levels, respectively, and the tessellation spacing. The original subdivision of the first inner triangle is retained. The area between the outer and first inner triangles is completely filled by non-overlapping triangles as described above. If the first (and only) inner triangle is degenerate, a set of triangles is produced by connecting each vertex on the outer triangle edges with the center point.

After all triangles are generated, each vertex in the subdivided triangle is assigned a barycentric (u,v,w) coordinate based on its location relative to the three vertices of the outer triangle.

The algorithm used to subdivide the triangular domain in (u,v,w) space into individual triangles is implementation-dependent. However, the set of triangles produced will completely cover the domain, and no portion of the domain will be covered by multiple triangles.

Output triangles are generated with a topology similar to triangle lists, except that the order in which each triangle is generated, and the order in which the vertices are generated for each triangle, are implementation-dependent. However, the order of vertices in each triangle is consistent across the domain as described in Tessellator Vertex Winding Order.

23.7. Quad Tessellation

If the tessellation primitive mode is `Quads`, a rectangle is subdivided into a collection of triangles covering the area of the original rectangle. First, the original rectangle is subdivided into a regular mesh of rectangles, where the number of rectangles along the u = 0 and u = 1 (vertical) and v = 0 and v = 1 (horizontal) edges are derived from the first and second inner tessellation levels, respectively. All rectangles, except those adjacent to one of the outer rectangle edges, are decomposed into triangle pairs. The outermost rectangle edges are subdivided independently, using the first, second, third, and fourth outer tessellation levels to control the number of subdivisions of the u = 0 (left), v = 0 (bottom), u = 1 (right), and v = 1 (top) edges, respectively. The area between the inner rectangles of the mesh and the outer rectangle edges are filled by triangles produced by joining the vertices on the subdivided outer edges to the vertices on the edge of the inner rectangle mesh.

If both clamped inner tessellation levels and all four clamped outer tessellation levels are exactly one, only a single triangle pair covering the outer rectangle is generated. Otherwise, if either clamped inner tessellation level is one, that tessellation level is treated as though it was originally specified as 1 + ε and will result in a two- or three-segment subdivision depending on the tessellation spacing. When used with fractional odd spacing, the three-segment subdivision **may** produce *inner vertices* positioned on the edge of the rectangle.

If any tessellation level is greater than one, tessellation begins by subdividing the u = 0 and u = 1 edges of the outer rectangle into m segments using the clamped and rounded first inner tessellation level and the tessellation spacing. The v = 0 and v = 1 edges are subdivided into n segments using the second inner tessellation level. Each vertex on the u = 0 and v = 0 edges are joined with the corresponding vertex on the u = 1 and v = 1 edges to produce a set of vertical and horizontal lines that divide the rectangle into a grid of smaller rectangles. The primitive generator emits a pair of non-overlapping triangles covering each such rectangle not adjacent to an edge of the outer rectangle. The boundary of the region covered by these triangles forms an inner rectangle, the

edges of which are subdivided by the grid vertices that lie on the edge. If either m or n is two, the inner rectangle is degenerate, and one or both of the rectangle's *edges* consist of a single point. This subdivision is illustrated in Figure Inner Quad Tessellation.

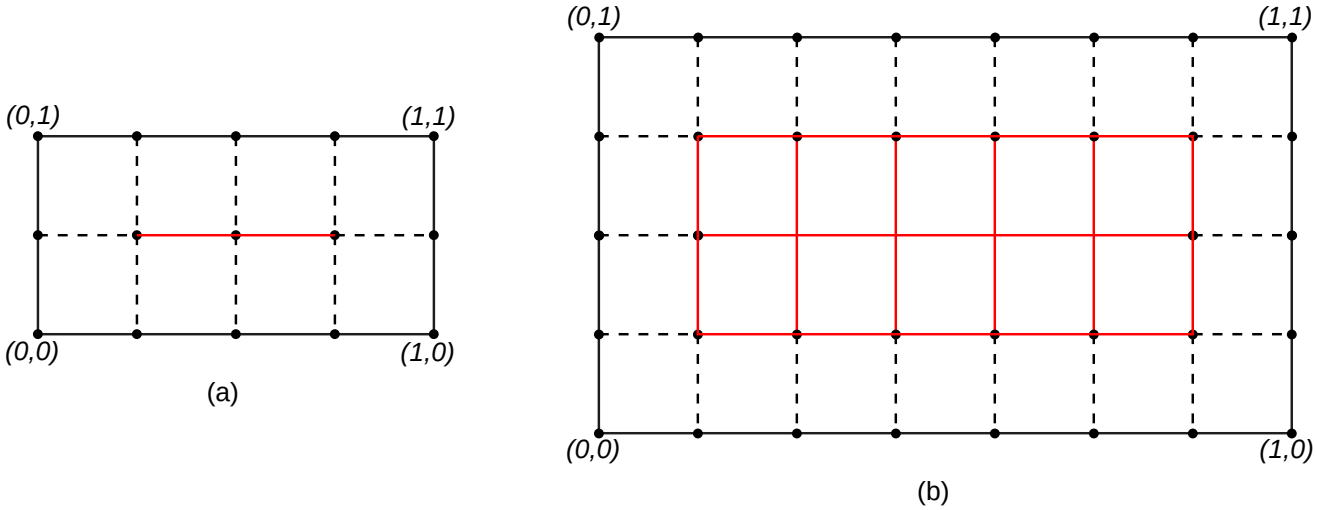


Figure 14. Inner Quad Tessellation

Caption

In the Inner Quad Tessellation diagram, inner quad tessellation levels of (a) (4,2) and (b) (7,4) are shown. The regions highlighted in red in figure (b) depict the 10 inner rectangles, each of which will be subdivided into two triangles. Solid black circles depict vertices on the boundary of the outer and inner rectangles, where the inner rectangle of figure (a) is degenerate (a single line segment). Dotted lines depict the horizontal and vertical edges connecting corresponding vertices on the inner and outer rectangle edges.

After the area corresponding to the inner rectangle is filled, the tessellator **must** produce triangles to cover the area between the inner and outer rectangles. To do this, the subdivision of the outer rectangle edge above is discarded. Instead, the u = 0, v = 0, u = 1, and v = 1 edges are subdivided according to the first, second, third, and fourth outer tessellation levels, respectively, and the tessellation spacing. The original subdivision of the inner rectangle is retained. The area between the outer and inner rectangles is completely filled by non-overlapping triangles. Two of the three vertices of each triangle are adjacent vertices on a subdivided edge of one rectangle; the third is one of the vertices on the corresponding edge of the other rectangle. If either edge of the innermost rectangle is degenerate, the area near the corresponding outer edges is filled by connecting each vertex on the outer edge with the single vertex making up the *inner edge*.

The algorithm used to subdivide the rectangular domain in (u,v) space into individual triangles is implementation-dependent. However, the set of triangles produced will completely cover the domain, and no portion of the domain will be covered by multiple triangles.

Output triangles are generated with a topology similar to triangle lists, except that the order in which each triangle is generated, and the order in which the vertices are generated for each triangle, are implementation-dependent. However, the order of vertices in each triangle is consistent across the domain as described in Tessellator Vertex Winding Order.

23.8. Isoline Tessellation

If the tessellation primitive mode is `IsoLines`, a set of independent horizontal line segments is drawn. The segments are arranged into connected strips called *isolines*, where the vertices of each isoline have a constant v coordinate and u coordinates covering the full range [0,1]. The number of isolines generated is derived from the first outer tessellation level; the number of segments in each isoline is derived from the second outer tessellation level. Both inner tessellation levels and the third and fourth outer tessellation levels have no effect in this mode.

As with quad tessellation above, isoline tessellation begins with a rectangle. The u = 0 and u = 1 edges of the rectangle are subdivided according to the first outer tessellation level. For the purposes of this subdivision, the tessellation spacing mode is ignored and treated as equal_spacing. An isoline is drawn connecting each vertex on the u = 0 rectangle edge to the corresponding vertex on the u = 1 rectangle edge, except that no line is drawn between (0,1) and (1,1). If the number of isolines on the subdivided u = 0 and u = 1 edges is n, this process will result in n equally spaced lines with constant v coordinates of 0, $\frac{1}{n}$, $\frac{2}{n}$, ..., $\frac{n-1}{n}$.

Each of the n isolines is then subdivided according to the second outer tessellation level and the tessellation spacing, resulting in m line segments. Each segment of each line is emitted by the tessellator. These line segments are generated with a topology similar to line lists, except that the order in which each line is generated, and the order in which the vertices are generated for each line segment, are implementation-dependent.

NOTE If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`tessellationIsolines` is `VK_FALSE`, then isoline tessellation is not supported by the implementation.

23.9. Tessellation Point Mode

For all primitive modes, the tessellator is capable of generating points instead of lines or triangles. If the tessellation control or tessellation evaluation shader specifies the `OpExecutionMode PointMode`, the primitive generator will generate one point for each distinct vertex produced by tessellation, rather than emitting triangles or lines. Otherwise, the tessellator will produce a collection of line segments or triangles according to the primitive mode. These points are generated with a topology similar to point lists, except the order in which the points are generated for each input primitive is undefined.

NOTE If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`tessellationPointMode` is `VK_FALSE`, then tessellation point mode is not supported by the implementation.

23.10. Tessellation Pipeline State

The `pTessellationState` member of VkGraphicsPipelineCreateInfo is a pointer to a `VkPipelineTessellationStateCreateInfo` structure.

The `VkPipelineTessellationStateCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineTessellationStateCreateInfo {
    VkStructureType                            sType;
    const void*                                pNext;
    VkPipelineTessellationStateCreateFlags     flags;
    uint32_t                                   patchControlPoints;
} VkPipelineTessellationStateCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `patchControlPoints` is the number of control points per patch.

Valid Usage

- VUID-VkPipelineTessellationStateCreateInfo-patchControlPoints-01214
  `patchControlPoints` **must** be greater than zero and less than or equal to `VkPhysicalDeviceLimits`::`maxTessellationPatchSize`

Valid Usage (Implicit)

- VUID-VkPipelineTessellationStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_TESSELLATION_STATE_CREATE_INFO`
- VUID-VkPipelineTessellationStateCreateInfo-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineTessellationDomainOriginStateCreateInfo
- VUID-VkPipelineTessellationStateCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPipelineTessellationStateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineTessellationStateCreateFlags;
```

`VkPipelineTessellationStateCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

The `VkPipelineTessellationDomainOriginStateCreateInfo` structure is defined as:

```
typedef struct VkPipelineTessellationDomainOriginStateCreateInfo {
    VkStructureType              sType;
    const void*                  pNext;
    VkTessellationDomainOrigin   domainOrigin;
} VkPipelineTessellationDomainOriginStateCreateInfo;
```

or the equivalent

```
// Provided by VK_KHR_maintenance2
typedef VkPipelineTessellationDomainOriginStateCreateInfo
VkPipelineTessellationDomainOriginStateCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `domainOrigin` is a VkTessellationDomainOrigin value controlling the origin of the tessellation domain space.

If the `VkPipelineTessellationDomainOriginStateCreateInfo` structure is included in the `pNext` chain of VkPipelineTessellationStateCreateInfo, it controls the origin of the tessellation domain. If this structure is not present, it is as if `domainOrigin` was `VK_TESSELLATION_DOMAIN_ORIGIN_UPPER_LEFT`.

Valid Usage (Implicit)

- VUID-VkPipelineTessellationDomainOriginStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_TESSELLATION_DOMAIN_ORIGIN_STATE_CREATE_INFO`
- VUID-VkPipelineTessellationDomainOriginStateCreateInfo-domainOrigin-parameter
  `domainOrigin` **must** be a valid VkTessellationDomainOrigin value

The possible tessellation domain origins are specified by the VkTessellationDomainOrigin enumeration:

```
typedef enum VkTessellationDomainOrigin {
    VK_TESSELLATION_DOMAIN_ORIGIN_UPPER_LEFT = 0,
    VK_TESSELLATION_DOMAIN_ORIGIN_LOWER_LEFT = 1,
  // Provided by VK_KHR_maintenance2
    VK_TESSELLATION_DOMAIN_ORIGIN_UPPER_LEFT_KHR =
VK_TESSELLATION_DOMAIN_ORIGIN_UPPER_LEFT,
  // Provided by VK_KHR_maintenance2
    VK_TESSELLATION_DOMAIN_ORIGIN_LOWER_LEFT_KHR =
VK_TESSELLATION_DOMAIN_ORIGIN_LOWER_LEFT,
} VkTessellationDomainOrigin;
```

or the equivalent

```
// Provided by VK_KHR_maintenance2
typedef VkTessellationDomainOrigin VkTessellationDomainOriginKHR;
```

- VK_TESSELLATION_DOMAIN_ORIGIN_UPPER_LEFT specifies that the origin of the domain space is in the upper left corner, as shown in figure Domain parameterization for tessellation primitive modes (upper-left origin).
- VK_TESSELLATION_DOMAIN_ORIGIN_LOWER_LEFT specifies that the origin of the domain space is in the lower left corner, as shown in figure Domain parameterization for tessellation primitive modes (lower-left origin).

This enum affects how the VertexOrderCw and VertexOrderCcw tessellation execution modes are interpreted, since the winding is defined relative to the orientation of the domain.

To dynamically set the origin of the tessellation domain space, call:

```
// Provided by VK_EXT_extended_dynamic_state3 with VK_KHR_maintenance2 or
VK_VERSION_1_1, VK_EXT_shader_object
void vkCmdSetTessellationDomainOriginEXT(
    VkCommandBuffer                             commandBuffer,
    VkTessellationDomainOrigin                  domainOrigin);
```

- commandBuffer is the command buffer into which the command will be recorded.
- domainOrigin specifies the origin of the tessellation domain space.

This command sets the origin of the tessellation domain space for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT set in VkPipelineDynamicStateCreateInfo ::pDynamicStates. Otherwise, this state is specified by the VkPipelineTessellationDomainOriginStateCreateInfo::domainOrigin value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetTessellationDomainOriginEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3TessellationDomainOrigin feature is enabled
  - The shaderObject feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetTessellationDomainOriginEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetTessellationDomainOriginEXT-domainOrigin-parameter

`domainOrigin` **must** be a valid VkTessellationDomainOrigin value

- VUID-vkCmdSetTessellationDomainOriginEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetTessellationDomainOriginEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetTessellationDomainOriginEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Chapter 24. Geometry Shading

The geometry shader operates on a group of vertices and their associated data assembled from a single input primitive, and emits zero or more output primitives and the group of vertices and their associated data required for each output primitive. Geometry shading is enabled when a geometry shader is included in the pipeline.

24.1. Geometry Shader Input Primitives

Each geometry shader invocation has access to all vertices in the primitive (and their associated data), which are presented to the shader as an array of inputs.

The input primitive type expected by the geometry shader is specified with an `OpExecutionMode` instruction in the geometry shader, and **must** match the incoming primitive type specified by either the pipeline's primitive topology if tessellation is inactive, or the tessellation mode if tessellation is active, as follows:

- An input primitive type of `InputPoints` **must** only be used with a pipeline topology of `VK_PRIMITIVE_TOPOLOGY_POINT_LIST`, or with a tessellation shader specifying `PointMode`. The input arrays always contain one element, as described by the point list topology or tessellation in point mode.
- An input primitive type of `InputLines` **must** only be used with a pipeline topology of `VK_PRIMITIVE_TOPOLOGY_LINE_LIST` or `VK_PRIMITIVE_TOPOLOGY_LINE_STRIP`, or with a tessellation shader specifying `IsoLines` that does not specify `PointMode`. The input arrays always contain two elements, as described by the line list topology or line strip topology, or by isoline tessellation.
- An input primitive type of `InputLinesAdjacency` **must** only be used when tessellation is inactive, with a pipeline topology of `VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY` or `VK_PRIMITIVE_TOPOLOGY_LINE_STRIP_WITH_ADJACENCY`. The input arrays always contain four elements, as described by the line list with adjacency topology or line strip with adjacency topology.
- An input primitive type of `Triangles` **must** only be used with a pipeline topology of `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST`, `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP`, or `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN`; or with a tessellation shader specifying `Quads` or `Triangles` that does not specify `PointMode`. The input arrays always contain three elements, as described by the triangle list topology, triangle strip topology, or triangle fan topology, or by triangle or quad tessellation. Vertices **may** be in a different absolute order than specified by the topology, but **must** adhere to the specified winding order.
- An input primitive type of `InputTrianglesAdjacency` **must** only be used when tessellation is inactive, with a pipeline topology of `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY` or `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY`. The input arrays always contain six elements, as described by the triangle list with adjacency topology or triangle strip with adjacency topology. Vertices **may** be in a different absolute order than specified by the topology, but **must** adhere to the specified winding order, and the vertices making up the main primitive **must** still occur at the first, third, and fifth index.

24.2. Geometry Shader Output Primitives

A geometry shader generates primitives in one of three output modes: points, line strips, or triangle strips. The primitive mode is specified in the shader using an `OpExecutionMode` instruction with the `OutputPoints`, `OutputLineStrip` or `OutputTriangleStrip` modes, respectively. Each geometry shader **must** include exactly one output primitive mode.

The vertices output by the geometry shader are assembled into points, lines, or triangles based on the output primitive type and the resulting primitives are then further processed as described in Rasterization. If the number of vertices emitted by the geometry shader is not sufficient to produce a single primitive, vertices corresponding to incomplete primitives are not processed by subsequent pipeline stages. The number of vertices output by the geometry shader is limited to a maximum count specified in the shader.

The maximum output vertex count is specified in the shader using an `OpExecutionMode` instruction with the mode set to `OutputVertices` and the maximum number of vertices that will be produced by the geometry shader specified as a literal. Each geometry shader **must** specify a maximum output vertex count.

24.3. Multiple Invocations of Geometry Shaders

Geometry shaders **can** be invoked more than one time for each input primitive. This is known as *geometry shader instancing* and is requested by including an `OpExecutionMode` instruction with `mode` specified as `Invocations` and the number of invocations specified as an integer literal.

In this mode, the geometry shader will execute at least n times for each input primitive, where n is the number of invocations specified in the `OpExecutionMode` instruction. The instance number is available to each invocation as a built-in input using `InvocationId`.

24.4. Geometry Shader Primitive Ordering

Limited guarantees are provided for the relative ordering of primitives produced by a geometry shader, as they pertain to primitive order.

- For instanced geometry shaders, the output primitives generated from each input primitive are passed to subsequent pipeline stages using the invocation number to order the primitives, from least to greatest.
- All output primitives generated from a given input primitive are passed to subsequent pipeline stages before any output primitives generated from subsequent input primitives.

Chapter 25. Fixed-Function Vertex Post-Processing

After pre-rasterization shader stages, the following fixed-function operations are applied to vertices of the resulting primitives:

- Transform feedback (see Transform Feedback)
- Flat shading (see Flat Shading).
- Primitive clipping, including application-defined half-spaces (see Primitive Clipping).
- Shader output attribute clipping (see Clipping Shader Outputs).
- Perspective division on clip coordinates (see Coordinate Transformations).
- Viewport mapping, including depth range scaling (see Controlling the Viewport).
- Front face determination for polygon primitives (see Basic Polygon Rasterization).

Next, rasterization is performed on primitives as described in chapter Rasterization.

25.1. Transform Feedback

Before any other fixed-function vertex post-processing, vertex outputs from the last shader in the pre-rasterization shader stage **can** be written out to one or more transform feedback buffers bound to the command buffer. To capture vertex outputs the last pre-rasterization shader stage shader **must** be declared with the `Xfb` execution mode. Outputs decorated with `XfbBuffer` will be written out to the corresponding transform feedback buffers bound to the command buffer when transform feedback is active. Transform feedback buffers are bound to the command buffer by using vkCmdBindTransformFeedbackBuffersEXT. Transform feedback is made active by calling vkCmdBeginTransformFeedbackEXT and made inactive by calling vkCmdEndTransformFeedbackEXT. After vertex data is written it is possible to use vkCmdDrawIndirectByteCountEXT to start a new draw where the `vertexCount` is derived from the number of bytes written by a previous transform feedback.

When an individual point, line, or triangle primitive reaches the transform feedback stage while transform feedback is active, the values of the specified output variables are assembled into primitives and appended to the bound transform feedback buffers. After activating transform feedback, the values of the first assembled primitive are written at the starting offsets of the bound transform feedback buffers, and subsequent primitives are appended to the buffer. If the optional `pCounterBuffers` and `pCounterBufferOffsets` parameters are specified, the starting points within the transform feedback buffers are adjusted so data is appended to the previously written values indicated by the value stored by the implementation in the counter buffer.

For multi-vertex primitives, all values for a given vertex are written before writing values for any other vertex. When the `transformFeedbackPreservesProvokingVertex` feature is not enabled, implementations **may** write out any vertex within the primitive first, but all subsequent vertices for that primitive **must** be written out in a consistent winding order defined as follows:

- If neither geometry or tessellation shading is active, vertices within a primitive are appended

according to the winding order described by the primitive topology defined by the VkPipelineInputAssemblyStateCreateInfo:topology used to execute the drawing command.

- If geometry shading is active, vertices within a primitive are appended according to the winding order described by the primitive topology defined by the OutputPoints, OutputLineStrip, or OutputTriangleStrip execution mode.
- If tessellation shading is active but geometry shading is not, vertices within a primitive are appended according to the winding order defined by triangle tessellation, quad tessellation, and isoline tessellation.

When the transformFeedbackPreservesProvokingVertex feature is enabled, then in addition to writing vertices with a consistent winding order, the vertex order **must** preserve the provoking vertex of each primitive:

- When the pipeline's provoking vertex mode is VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT, the primitive's provoking vertex **must** be the first vertex written.
- When the pipeline's provoking vertex mode is VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT, the primitive's provoking vertex **must** be the last vertex written.

If transformFeedbackPreservesTriangleFanProvokingVertex is VK_FALSE, neither geometry nor tessellation shading is active, and the primitive topology is VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN, then the first vertex written from each primitive is implementation-defined even when the transformFeedbackPreservesProvokingVertex feature is enabled.

When capturing vertices, the stride associated with each transform feedback buffer, as indicated by the XfbStride decoration, indicates the number of bytes of storage reserved for each vertex in the transform feedback buffer. For every vertex captured, each output attribute with a Offset decoration will be written to the storage reserved for the vertex at the associated transform feedback buffer. When writing output variables that are arrays or structures, individual array elements or structure members are written tightly packed in order. For vector types, individual components are written in order. For matrix types, outputs are written as an array of column vectors.

If any component of an output with an assigned transform feedback offset was not written to by its shader, the value recorded for that component is undefined. All components of an output variable **must** be written at an offset aligned to the size of the component. The size of each component of an output variable **must** be at least 32-bits. When capturing a vertex, any portion of the reserved storage not associated with an output variable with an assigned transform feedback offset will be unmodified.

When transform feedback is inactive, no vertices are recorded. If there is a valid counter buffer handle and counter buffer offset in the pCounterBuffers and pCounterBufferOffsets arrays, writes to the corresponding transform feedback buffer will start at the byte offset represented by the value stored in the counter buffer location.

Individual lines or triangles of a strip or fan primitive will be extracted and recorded separately. Incomplete primitives are not recorded.

When using a geometry shader that emits vertices to multiple vertex streams, a primitive will be

assembled and output for each stream when there are enough vertices emitted for the output primitive type. All outputs assigned to a given transform feedback buffer are required to come from a single vertex stream.

The sizes of the transform feedback buffers are defined by the vkCmdBindTransformFeedbackBuffersEXT `pSizes` parameter for each of the bound buffers, or the size of the bound buffer, whichever is the lesser. If there is less space remaining in any of the transform feedback buffers than the size of all of the vertex data for that primitive based on the `XfbStride` for that `XfbBuffer` then no vertex data of that primitive is recorded in any transform feedback buffer, and the value for the number of primitives written in the corresponding `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT` query for all transform feedback buffers is no longer incremented.

Any outputs made to a `XfbBuffer` that is not bound to a transform feedback buffer is ignored.

To bind transform feedback buffers to a command buffer for use in subsequent drawing commands, call:

```
// Provided by VK_EXT_transform_feedback
void vkCmdBindTransformFeedbackBuffersEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstBinding,
    uint32_t                                    bindingCount,
    const VkBuffer*                             pBuffers,
    const VkDeviceSize*                         pOffsets,
    const VkDeviceSize*                         pSizes);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `firstBinding` is the index of the first transform feedback binding whose state is updated by the command.
- `bindingCount` is the number of transform feedback bindings whose state is updated by the command.
- `pBuffers` is a pointer to an array of buffer handles.
- `pOffsets` is a pointer to an array of buffer offsets.
- `pSizes` is `NULL` or a pointer to an array of VkDeviceSize buffer sizes, specifying the maximum number of bytes to capture to the corresponding transform feedback buffer. If `pSizes` is `NULL`, or the value of the `pSizes` array element is `VK_WHOLE_SIZE`, then the maximum number of bytes captured will be the size of the corresponding buffer minus the buffer offset.

The values taken from elements i of `pBuffers`, `pOffsets` and `pSizes` replace the current state for the transform feedback binding `firstBinding` + i, for i in [0, `bindingCount`). The transform feedback binding is updated to start at the offset indicated by `pOffsets`[i] from the start of the buffer `pBuffers`[i].

Valid Usage

- VUID-vkCmdBindTransformFeedbackBuffersEXT-transformFeedback-02355
  `VkPhysicalDeviceTransformFeedbackFeaturesEXT`::`transformFeedback` **must** be enabled
- VUID-vkCmdBindTransformFeedbackBuffersEXT-firstBinding-02356
  `firstBinding` **must** be less than `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackBuffers`
- VUID-vkCmdBindTransformFeedbackBuffersEXT-firstBinding-02357
  The sum of `firstBinding` and `bindingCount` **must** be less than or equal to `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackBuffers`
- VUID-vkCmdBindTransformFeedbackBuffersEXT-pOffsets-02358
  All elements of `pOffsets` **must** be less than the size of the corresponding element in `pBuffers`
- VUID-vkCmdBindTransformFeedbackBuffersEXT-pOffsets-02359
  All elements of `pOffsets` **must** be a multiple of 4
- VUID-vkCmdBindTransformFeedbackBuffersEXT-pBuffers-02360
  All elements of `pBuffers` **must** have been created with the `VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_BUFFER_BIT_EXT` flag
- VUID-vkCmdBindTransformFeedbackBuffersEXT-pSize-02361
  If the optional `pSize` array is specified, each element of `pSizes` **must** either be `VK_WHOLE_SIZE`, or be less than or equal to `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackBufferSize`
- VUID-vkCmdBindTransformFeedbackBuffersEXT-pSizes-02362
  All elements of `pSizes` **must** be either `VK_WHOLE_SIZE`, or less than or equal to the size of the corresponding buffer in `pBuffers`
- VUID-vkCmdBindTransformFeedbackBuffersEXT-pOffsets-02363
  All elements of `pOffsets` plus `pSizes`, where the `pSizes`, element is not `VK_WHOLE_SIZE`, **must** be less than or equal to the size of the corresponding buffer in `pBuffers`
- VUID-vkCmdBindTransformFeedbackBuffersEXT-pBuffers-02364
  Each element of `pBuffers` that is non-sparse **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdBindTransformFeedbackBuffersEXT-None-02365
  Transform feedback **must** not be active when the `vkCmdBindTransformFeedbackBuffersEXT` command is recorded

Valid Usage (Implicit)

- VUID-vkCmdBindTransformFeedbackBuffersEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBindTransformFeedbackBuffersEXT-pBuffers-parameter
  `pBuffers` **must** be a valid pointer to an array of `bindingCount` valid VkBuffer handles

- VUID-vkCmdBindTransformFeedbackBuffersEXT-pOffsets-parameter
  `pOffsets` **must** be a valid pointer to an array of `bindingCount` VkDeviceSize values
- VUID-vkCmdBindTransformFeedbackBuffersEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBindTransformFeedbackBuffersEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBindTransformFeedbackBuffersEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBindTransformFeedbackBuffersEXT-bindingCount-arraylength
  `bindingCount` **must** be greater than `0`
- VUID-vkCmdBindTransformFeedbackBuffersEXT-commonparent
  Both of `commandBuffer`, and the elements of `pBuffers` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Transform feedback for specific transform feedback buffers is made active by calling:

```
// Provided by VK_EXT_transform_feedback
void vkCmdBeginTransformFeedbackEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstCounterBuffer,
    uint32_t                                    counterBufferCount,
    const VkBuffer*                             pCounterBuffers,
    const VkDeviceSize*                         pCounterBufferOffsets);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `firstCounterBuffer` is the index of the first transform feedback buffer corresponding to `pCounterBuffers`[0] and `pCounterBufferOffsets`[0].

- `counterBufferCount` is the size of the `pCounterBuffers` and `pCounterBufferOffsets` arrays.
- `pCounterBuffers` is `NULL` or a pointer to an array of VkBuffer handles to counter buffers. Each buffer contains a 4 byte integer value representing the byte offset from the start of the corresponding transform feedback buffer from where to start capturing vertex data. If the byte offset stored to the counter buffer location was done using vkCmdEndTransformFeedbackEXT it can be used to resume transform feedback from the previous location. If `pCounterBuffers` is `NULL`, then transform feedback will start capturing vertex data to byte offset zero in all bound transform feedback buffers. For each element of `pCounterBuffers` that is VK_NULL_HANDLE, transform feedback will start capturing vertex data to byte zero in the corresponding bound transform feedback buffer.
- `pCounterBufferOffsets` is `NULL` or a pointer to an array of VkDeviceSize values specifying offsets within each of the `pCounterBuffers` where the counter values were previously written. The location in each counter buffer at these offsets **must** be large enough to contain 4 bytes of data. This data is the number of bytes captured by the previous transform feedback to this buffer. If `pCounterBufferOffsets` is `NULL`, then it is assumed the offsets are zero.

The active transform feedback buffers will capture primitives emitted from the corresponding `XfbBuffer` in the bound graphics pipeline. Any `XfbBuffer` emitted that does not output to an active transform feedback buffer will not be captured.

Valid Usage

- VUID-vkCmdBeginTransformFeedbackEXT-transformFeedback-02366
  `VkPhysicalDeviceTransformFeedbackFeaturesEXT`::`transformFeedback` **must** be enabled
- VUID-vkCmdBeginTransformFeedbackEXT-None-02367
  Transform feedback **must** not be active
- VUID-vkCmdBeginTransformFeedbackEXT-firstCounterBuffer-02368
  `firstCounterBuffer` **must** be less than `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackBuffers`
- VUID-vkCmdBeginTransformFeedbackEXT-firstCounterBuffer-02369
  The sum of `firstCounterBuffer` and `counterBufferCount` **must** be less than or equal to `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackBuffers`
- VUID-vkCmdBeginTransformFeedbackEXT-counterBufferCount-02607
  If `counterBufferCount` is not `0`, and `pCounterBuffers` is not `NULL`, `pCounterBuffers` **must** be a valid pointer to an array of `counterBufferCount` `VkBuffer` handles that are either valid or VK_NULL_HANDLE
- VUID-vkCmdBeginTransformFeedbackEXT-pCounterBufferOffsets-02370
  For each buffer handle in the array, if it is not VK_NULL_HANDLE it **must** reference a buffer large enough to hold 4 bytes at the corresponding offset from the `pCounterBufferOffsets` array
- VUID-vkCmdBeginTransformFeedbackEXT-pCounterBuffer-02371
  If `pCounterBuffer` is `NULL`, then `pCounterBufferOffsets` **must** also be `NULL`
- VUID-vkCmdBeginTransformFeedbackEXT-pCounterBuffers-02372
  For each buffer handle in the `pCounterBuffers` array that is not VK_NULL_HANDLE it **must**

have been created with a `usage` value containing `VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT`

- VUID-vkCmdBeginTransformFeedbackEXT-firstCounterBuffer-09630
  The sum of `firstCounterBuffer` and `counterBufferCount` **must** be less than or equal to the number of transform feedback buffers bound by vkCmdBindTransformFeedbackBuffersEXT
- VUID-vkCmdBeginTransformFeedbackEXT-None-06233
  If the `shaderObject` feature is not enabled, a valid graphics pipeline **must** be bound to `VK_PIPELINE_BIND_POINT_GRAPHICS`
- VUID-vkCmdBeginTransformFeedbackEXT-None-04128
  The last pre-rasterization shader stage of the bound graphics pipeline **must** have been declared with the `Xfb` execution mode
- VUID-vkCmdBeginTransformFeedbackEXT-None-02373
  Transform feedback **must** not be made active in a render pass instance with multiview enabled

Valid Usage (Implicit)

- VUID-vkCmdBeginTransformFeedbackEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBeginTransformFeedbackEXT-pCounterBufferOffsets-parameter
  If `counterBufferCount` is not `0`, and `pCounterBufferOffsets` is not `NULL`, `pCounterBufferOffsets` **must** be a valid pointer to an array of `counterBufferCount` VkDeviceSize values
- VUID-vkCmdBeginTransformFeedbackEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBeginTransformFeedbackEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdBeginTransformFeedbackEXT-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdBeginTransformFeedbackEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBeginTransformFeedbackEXT-commonparent
  Both of `commandBuffer`, and the elements of `pCounterBuffers` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally

synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	State

Transform feedback for specific transform feedback buffers is made inactive by calling:

```
// Provided by VK_EXT_transform_feedback
void vkCmdEndTransformFeedbackEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstCounterBuffer,
    uint32_t                                    counterBufferCount,
    const VkBuffer*                             pCounterBuffers,
    const VkDeviceSize*                         pCounterBufferOffsets);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `firstCounterBuffer` is the index of the first transform feedback buffer corresponding to `pCounterBuffers`[0] and `pCounterBufferOffsets`[0].
- `counterBufferCount` is the size of the `pCounterBuffers` and `pCounterBufferOffsets` arrays.
- `pCounterBuffers` is `NULL` or a pointer to an array of VkBuffer handles to counter buffers. The counter buffers are used to record the current byte positions of each transform feedback buffer where the next vertex output data would be captured. This **can** be used by a subsequent vkCmdBeginTransformFeedbackEXT call to resume transform feedback capture from this position. It can also be used by vkCmdDrawIndirectByteCountEXT to determine the vertex count of the draw call.
- `pCounterBufferOffsets` is `NULL` or a pointer to an array of VkDeviceSize values specifying offsets within each of the `pCounterBuffers` where the counter values can be written. The location in each counter buffer at these offsets **must** be large enough to contain 4 bytes of data. The data stored at this location is the byte offset from the start of the transform feedback buffer binding where the next vertex data would be written. If `pCounterBufferOffsets` is `NULL`, then it is assumed the offsets are zero.

Valid Usage

- VUID-vkCmdEndTransformFeedbackEXT-transformFeedback-02374
  `VkPhysicalDeviceTransformFeedbackFeaturesEXT`::`transformFeedback` **must** be enabled
- VUID-vkCmdEndTransformFeedbackEXT-None-02375
  Transform feedback **must** be active

- VUID-vkCmdEndTransformFeedbackEXT-firstCounterBuffer-02376
  `firstCounterBuffer` **must** be less than `VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackBuffers`
- VUID-vkCmdEndTransformFeedbackEXT-firstCounterBuffer-02377
  The sum of `firstCounterBuffer` and `counterBufferCount` **must** be less than or equal to `VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackBuffers`
- VUID-vkCmdEndTransformFeedbackEXT-counterBufferCount-02608
  If `counterBufferCount` is not `0`, and `pCounterBuffers` is not `NULL`, `pCounterBuffers` **must** be a valid pointer to an array of `counterBufferCount` `VkBuffer` handles that are either valid or VK_NULL_HANDLE
- VUID-vkCmdEndTransformFeedbackEXT-pCounterBufferOffsets-02378
  For each buffer handle in the array, if it is not VK_NULL_HANDLE it **must** reference a buffer large enough to hold 4 bytes at the corresponding offset from the `pCounterBufferOffsets` array
- VUID-vkCmdEndTransformFeedbackEXT-pCounterBuffer-02379
  If `pCounterBuffer` is `NULL`, then `pCounterBufferOffsets` **must** also be `NULL`
- VUID-vkCmdEndTransformFeedbackEXT-pCounterBuffers-02380
  For each buffer handle in the `pCounterBuffers` array that is not VK_NULL_HANDLE it **must** have been created with a `usage` value containing `VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT`

Valid Usage (Implicit)

- VUID-vkCmdEndTransformFeedbackEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdEndTransformFeedbackEXT-pCounterBufferOffsets-parameter
  If `counterBufferCount` is not `0`, and `pCounterBufferOffsets` is not `NULL`, `pCounterBufferOffsets` **must** be a valid pointer to an array of `counterBufferCount` VkDeviceSize values
- VUID-vkCmdEndTransformFeedbackEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdEndTransformFeedbackEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdEndTransformFeedbackEXT-renderpass
  This command **must** only be called inside of a render pass instance
- VUID-vkCmdEndTransformFeedbackEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdEndTransformFeedbackEXT-commonparent
  Both of `commandBuffer`, and the elements of `pCounterBuffers` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Inside	Outside	Graphics	State

25.2. Flat Shading

Flat shading a vertex output attribute means to assign all vertices of the primitive the same value for that output. The output values assigned are those of the *provoking vertex* of the primitive. Flat shading is applied to those vertex attributes that match fragment input attributes which are decorated as `Flat`.

If neither geometry nor tessellation shading is active, the provoking vertex is determined by the primitive topology defined by VkPipelineInputAssemblyStateCreateInfo:`topology` used to execute the drawing command.

If geometry shading is active, the provoking vertex is determined by the primitive topology defined by the `OutputPoints`, `OutputLineStrip`, or `OutputTriangleStrip` execution mode.

If tessellation shading is active but geometry shading is not, the provoking vertex **may** be any of the vertices in each primitive.

For a given primitive topology, the pipeline's provoking vertex mode determines which vertex is the provoking vertex. To specify the provoking vertex mode, include a `VkPipelineRasterizationProvokingVertexStateCreateInfoEXT` structure in the VkPipelineRasterizationStateCreateInfo::`pNext` chain when creating the pipeline.

The `VkPipelineRasterizationProvokingVertexStateCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_provoking_vertex
typedef struct VkPipelineRasterizationProvokingVertexStateCreateInfoEXT {
    VkStructureType             sType;
    const void*                 pNext;
    VkProvokingVertexModeEXT    provokingVertexMode;
} VkPipelineRasterizationProvokingVertexStateCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `provokingVertexMode` is a VkProvokingVertexModeEXT value selecting the provoking vertex mode.

If this struct is not provided when creating the pipeline, the pipeline will use the `VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT` mode.

If the `provokingVertexModePerPipeline` limit is `VK_FALSE`, then all pipelines bound within a render pass instance **must** have the same `provokingVertexMode`.

Valid Usage

- VUID-VkPipelineRasterizationProvokingVertexStateCreateInfoEXT-provokingVertexMode-04883
  If `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT`, then the `provokingVertexLast` feature **must** be enabled

Valid Usage (Implicit)

- VUID-VkPipelineRasterizationProvokingVertexStateCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_PROVOKING_VERTEX_STATE_CREATE_INFO_EXT`
- VUID-VkPipelineRasterizationProvokingVertexStateCreateInfoEXT-provokingVertexMode-parameter
  `provokingVertexMode` **must** be a valid VkProvokingVertexModeEXT value

Possible values of VkPipelineRasterizationProvokingVertexStateCreateInfoEXT::`provokingVertexMode` are:

```
// Provided by VK_EXT_provoking_vertex
typedef enum VkProvokingVertexModeEXT {
    VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT = 0,
    VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT = 1,
} VkProvokingVertexModeEXT;
```

- `VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT` specifies that the provoking vertex is the first non-adjacency vertex in the list of vertices used by a primitive.
- `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT` specifies that the provoking vertex is the last non-adjacency vertex in the list of vertices used by a primitive.

These modes are described more precisely in Primitive Topologies.

To dynamically set the `provokingVertexMode` state, call:

```
// Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_provoking_vertex,
VK_EXT_provoking_vertex with VK_EXT_shader_object
void vkCmdSetProvokingVertexModeEXT(
    VkCommandBuffer                             commandBuffer,
    VkProvokingVertexModeEXT                    provokingVertexMode);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `provokingVertexMode` specifies the `provokingVertexMode` state.

This command sets the `provokingVertexMode` state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineRasterizationProvokingVertexStateCreateInfoEXT::`provokingVertexMode` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetProvokingVertexModeEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3ProvokingVertexMode` feature is enabled
  - The `shaderObject` feature is enabled
- VUID-vkCmdSetProvokingVertexModeEXT-provokingVertexMode-07447
  If `provokingVertexMode` is `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT`, then the `provokingVertexLast` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCmdSetProvokingVertexModeEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetProvokingVertexModeEXT-provokingVertexMode-parameter
  `provokingVertexMode` **must** be a valid VkProvokingVertexModeEXT value
- VUID-vkCmdSetProvokingVertexModeEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetProvokingVertexModeEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetProvokingVertexModeEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

25.3. Primitive Clipping

Primitives are culled against the *cull volume* and then clipped to the *clip volume*. In clip coordinates, the *view volume* is defined by:

$$-w_c \leq x_c \leq w_c$$
$$-w_c \leq y_c \leq w_c$$
$$z_m \leq z_c \leq w_c$$

where if VkPipelineViewportDepthClipControlCreateInfoEXT::`negativeOneToOne` is `VK_TRUE` z_m is equal to $-w_c$ otherwise z_m is equal to zero.

This view volume **can** be further restricted by as many as `VkPhysicalDeviceLimits`::`maxClipDistances` application-defined half-spaces.

The cull volume is the intersection of up to `VkPhysicalDeviceLimits`::`maxCullDistances` application-defined half-spaces (if no application-defined cull half-spaces are enabled, culling against the cull volume is skipped).

A shader **must** write a single cull distance for each enabled cull half-space to elements of the `CullDistance` array. If the cull distance for any enabled cull half-space is negative for all of the vertices of the primitive under consideration, the primitive is discarded. Otherwise the primitive is clipped against the clip volume as defined below.

The clip volume is the intersection of up to `VkPhysicalDeviceLimits`::`maxClipDistances` application-defined half-spaces with the view volume (if no application-defined clip half-spaces are enabled, the clip volume is the view volume).

A shader **must** write a single clip distance for each enabled clip half-space to elements of the `ClipDistance` array. Clip half-space i is then given by the set of points satisfying the inequality

$c_i(\mathbf{P}) \geq 0$

where $c_i(\mathbf{P})$ is the clip distance i at point **P**. For point primitives, $c_i(\mathbf{P})$ is simply the clip distance for the vertex in question. For line and triangle primitives, per-vertex clip distances are interpolated using a weighted mean, with weights derived according to the algorithms described in sections Basic Line Segment Rasterization and Basic Polygon Rasterization, using the perspective interpolation equations.

The number of application-defined clip and cull half-spaces that are enabled is determined by the explicit size of the built-in arrays `ClipDistance` and `CullDistance`, respectively, declared as an output in the interface of the entry point of the final shader stage before clipping.

If VkPipelineRasterizationDepthClipStateCreateInfoEXT is present in the graphics pipeline state then depth clipping is disabled if VkPipelineRasterizationDepthClipStateCreateInfoEXT ::`depthClipEnable` is `VK_FALSE`. Otherwise, if VkPipelineRasterizationDepthClipStateCreateInfoEXT is not present, depth clipping is disabled when VkPipelineRasterizationStateCreateInfo ::`depthClampEnable` is `VK_TRUE`.

To dynamically set enable or disable depth clamping, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetDepthClampEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    depthClampEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `depthClampEnable` specifies whether depth clamping is enabled.

This command sets whether depth clamping is enabled or disabled for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineRasterizationStateCreateInfo::`depthClampEnable` value used to create the currently active pipeline.

If the depth clamping state is changed dynamically, and the pipeline was not created with `VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT` enabled, then depth clipping is enabled when depth clamping is disabled and vice versa.

Valid Usage

- VUID-vkCmdSetDepthClampEnableEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3DepthClampEnable` feature is enabled
  - The `shaderObject` feature is enabled
- VUID-vkCmdSetDepthClampEnableEXT-depthClamp-07449
  If the `depthClamp` feature is not enabled, `depthClampEnable` **must** be `VK_FALSE`

Valid Usage (Implicit)

- VUID-vkCmdSetDepthClampEnableEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthClampEnableEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthClampEnableEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthClampEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set enable or disable depth clipping, call:

```
// Provided by VK_EXT_depth_clip_enable with VK_EXT_extended_dynamic_state3,
VK_EXT_depth_clip_enable with VK_EXT_shader_object
void vkCmdSetDepthClipEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    depthClipEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `depthClipEnable` specifies whether depth clipping is enabled.

This command sets whether depth clipping is enabled or disabled for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineRasterizationDepthClipStateCreateInfoEXT ::`depthClipEnable` value used to create the currently active pipeline, or by the inverse of

VkPipelineRasterizationStateCreateInfo::depthClampEnable if VkPipelineRasterizationDepthClipStateCreateInfoEXT is not specified.

Valid Usage

- VUID-vkCmdSetDepthClipEnableEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3DepthClipEnable feature is enabled
  - The shaderObject feature is enabled
- VUID-vkCmdSetDepthClipEnableEXT-depthClipEnable-07451
  The depthClipEnable feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCmdSetDepthClipEnableEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthClipEnableEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetDepthClipEnableEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthClipEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

When depth clipping is disabled, the plane equation

$$z_m \leq z_c \leq w_c$$

(see the clip volume definition above) is ignored by view volume clipping (effectively, there is no near or far plane clipping).

If the primitive under consideration is a point or line segment, then clipping passes it unchanged if its vertices lie entirely within the clip volume.

Possible values of VkPhysicalDevicePointClippingProperties::pointClippingBehavior, specifying clipping behavior of a point primitive whose vertex lies outside the clip volume, are:

```
typedef enum VkPointClippingBehavior {
    VK_POINT_CLIPPING_BEHAVIOR_ALL_CLIP_PLANES = 0,
    VK_POINT_CLIPPING_BEHAVIOR_USER_CLIP_PLANES_ONLY = 1,
  // Provided by VK_KHR_maintenance2
    VK_POINT_CLIPPING_BEHAVIOR_ALL_CLIP_PLANES_KHR =
VK_POINT_CLIPPING_BEHAVIOR_ALL_CLIP_PLANES,
  // Provided by VK_KHR_maintenance2
    VK_POINT_CLIPPING_BEHAVIOR_USER_CLIP_PLANES_ONLY_KHR =
VK_POINT_CLIPPING_BEHAVIOR_USER_CLIP_PLANES_ONLY,
} VkPointClippingBehavior;
```

or the equivalent

```
// Provided by VK_KHR_maintenance2
typedef VkPointClippingBehavior VkPointClippingBehaviorKHR;
```

- VK_POINT_CLIPPING_BEHAVIOR_ALL_CLIP_PLANES specifies that the primitive is discarded if the vertex lies outside any clip plane, including the planes bounding the view volume.
- VK_POINT_CLIPPING_BEHAVIOR_USER_CLIP_PLANES_ONLY specifies that the primitive is discarded only if the vertex lies outside any user clip plane.

If either of a line segment's vertices lie outside of the clip volume, the line segment **may** be clipped, with new vertex coordinates computed for each vertex that lies outside the clip volume. A clipped line segment endpoint lies on both the original line segment and the boundary of the clip volume.

This clipping produces a value, $0 \leq t \leq 1$, for each clipped vertex. If the coordinates of a clipped vertex are $\mathbf{P}$ and the unclipped line segment's vertex coordinates are $\mathbf{P}_1$ and $\mathbf{P}_2$, then t satisfies the following equation

$$\mathbf{P} = t\,\mathbf{P}_1 + (1\text{-}t)\,\mathbf{P}_2.$$

t is used to clip vertex output attributes as described in Clipping Shader Outputs.

If the primitive is a polygon, it passes unchanged if every one of its edges lies entirely inside the clip volume, and is either clipped or discarded otherwise. If the edges of the polygon intersect the

boundary of the clip volume, the intersecting edges are reconnected by new edges that lie along the boundary of the clip volume - in some cases requiring the introduction of new vertices into a polygon.

If a polygon intersects an edge of the clip volume's boundary, the clipped polygon **must** include a point on this boundary edge.

Primitives rendered with application-defined half-spaces **must** satisfy a complementarity criterion. Suppose a series of primitives is drawn where each vertex i has a single specified clip distance d_i (or a number of similarly specified clip distances, if multiple half-spaces are enabled). Next, suppose that the same series of primitives are drawn again with each such clip distance replaced by $-d_i$ (and the graphics pipeline is otherwise the same). In this case, primitives **must** not be missing any pixels, and pixels **must** not be drawn twice in regions where those primitives are cut by the clip planes.

The `VkPipelineViewportDepthClipControlCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_depth_clip_control
typedef struct VkPipelineViewportDepthClipControlCreateInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           negativeOneToOne;
} VkPipelineViewportDepthClipControlCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `negativeOneToOne` sets the z_m in the *view volume* to $-w_c$

Valid Usage

- VUID-VkPipelineViewportDepthClipControlCreateInfoEXT-negativeOneToOne-06470
  If the `depthClipControl` feature is not enabled, `negativeOneToOne` **must** be `VK_FALSE`

Valid Usage (Implicit)

- VUID-VkPipelineViewportDepthClipControlCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_DEPTH_CLIP_CONTROL_CREATE_INFO_EXT`

To dynamically set `negativeOneToOne`, call:

```
// Provided by VK_EXT_depth_clip_control with VK_EXT_extended_dynamic_state3,
VK_EXT_depth_clip_control with VK_EXT_shader_object
void vkCmdSetDepthClipNegativeOneToOneEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    negativeOneToOne);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `negativeOneToOne` specifies the `negativeOneToOne` state.

This command sets the `negativeOneToOne` state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineViewportDepthClipControlCreateInfoEXT::`negativeOneToOne` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDepthClipNegativeOneToOneEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3DepthClipNegativeOneToOne` feature is enabled
  - The `shaderObject` feature is enabled
- VUID-vkCmdSetDepthClipNegativeOneToOneEXT-depthClipControl-07453
  The `depthClipControl` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCmdSetDepthClipNegativeOneToOneEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthClipNegativeOneToOneEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthClipNegativeOneToOneEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthClipNegativeOneToOneEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

25.4. Clipping Shader Outputs

Next, vertex output attributes are clipped. The output values associated with a vertex that lies within the clip volume are unaffected by clipping. If a primitive is clipped, however, the output values assigned to vertices produced by clipping are clipped.

Let the output values assigned to the two vertices $\mathbf{P}_1$ and $\mathbf{P}_2$ of an unclipped edge be $\mathbf{c}_1$ and $\mathbf{c}_2$. The value of t (see Primitive Clipping) for a clipped point $\mathbf{P}$ is used to obtain the output value associated with $\mathbf{P}$ as

$$\mathbf{c} = t\,\mathbf{c}_1 + (1-t)\,\mathbf{c}_2.$$

(Multiplying an output value by a scalar means multiplying each of *x*, *y*, *z*, and *w* by the scalar.)

Since this computation is performed in clip space before division by w_c, clipped output values are perspective-correct.

Polygon clipping creates a clipped vertex along an edge of the clip volume's boundary. This situation is handled by noting that polygon clipping proceeds by clipping against one half-space at a time. Output value clipping is done in the same way, so that clipped points always occur at the intersection of polygon edges (possibly already clipped) with the clip volume's boundary.

For vertex output attributes whose matching fragment input attributes are decorated with `NoPerspective`, the value of t used to obtain the output value associated with $\mathbf{P}$ will be adjusted to produce results that vary linearly in framebuffer space.

Output attributes of integer or unsigned integer type **must** always be flat shaded. Flat shaded attributes are constant over the primitive being rasterized (see Basic Line Segment Rasterization and Basic Polygon Rasterization), and no interpolation is performed. The output value $\mathbf{c}$ is taken from either $\mathbf{c}_1$ or $\mathbf{c}_2$, since flat shading has already occurred and the two values are identical.

25.5. Coordinate Transformations

Clip coordinates for a vertex result from shader execution, which yields a vertex coordinate `Position`.

Perspective division on clip coordinates yields *normalized device coordinates*, followed by a *viewport* transformation (see Controlling the Viewport) to convert these coordinates into *framebuffer coordinates*.

If a vertex in clip coordinates has a position given by

$$\begin{pmatrix} x_c \\ y_c \\ z_c \\ w_c \end{pmatrix}$$

then the vertex's normalized device coordinates are

$$\begin{pmatrix} x_d \\ y_d \\ z_d \end{pmatrix} = \begin{pmatrix} \frac{x_c}{w_c} \\ \frac{y_c}{w_c} \\ \frac{z_c}{w_c} \end{pmatrix}$$

25.6. Controlling the Viewport

The viewport transformation is determined by the selected viewport's width and height in pixels, p_x and p_y, respectively, and its center (o_x, o_y) (also in pixels), as well as its depth range min and max determining a depth range scale value p_z and a depth range bias value o_z (defined below). The vertex's framebuffer coordinates (x_f, y_f) and depth z_f are given by

$$x_f = (p_x / 2)\, x_d + o_x$$

$$y_f = (p_y / 2)\, y_d + o_y$$

$$z_f = p_z \times z_d + o_z$$

Multiple viewports are available, numbered zero up to `VkPhysicalDeviceLimits`::`maxViewports` minus one. The number of viewports used by a pipeline is controlled by the `viewportCount` member of the `VkPipelineViewportStateCreateInfo` structure used in pipeline creation.

x_f and y_f have limited precision, where the number of fractional bits retained is specified by `VkPhysicalDeviceLimits`::`subPixelPrecisionBits`. When rasterizing line segments, the number of fractional bits is specified by `VkPhysicalDeviceLineRasterizationPropertiesKHR`::`lineSubPixelPrecisionBits`.

The `VkPipelineViewportStateCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineViewportStateCreateInfo {
    VkStructureType                       sType;
    const void*                           pNext;
    VkPipelineViewportStateCreateFlags    flags;
    uint32_t                              viewportCount;
    const VkViewport*                     pViewports;
    uint32_t                              scissorCount;
    const VkRect2D*                       pScissors;
} VkPipelineViewportStateCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `viewportCount` is the number of viewports used by the pipeline.
- `pViewports` is a pointer to an array of VkViewport structures, defining the viewport transforms. If the viewport state is dynamic, this member is ignored.
- `scissorCount` is the number of scissors and **must** match the number of viewports.
- `pScissors` is a pointer to an array of VkRect2D structures defining the rectangular bounds of the scissor for the corresponding viewport. If the scissor state is dynamic, this member is ignored.

Valid Usage

- VUID-VkPipelineViewportStateCreateInfo-viewportCount-01216
  If the `multiViewport` feature is not enabled, `viewportCount` **must** not be greater than `1`
- VUID-VkPipelineViewportStateCreateInfo-scissorCount-01217
  If the `multiViewport` feature is not enabled, `scissorCount` **must** not be greater than `1`
- VUID-VkPipelineViewportStateCreateInfo-viewportCount-01218
  `viewportCount` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxViewports`
- VUID-VkPipelineViewportStateCreateInfo-scissorCount-01219
  `scissorCount` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxViewports`
- VUID-VkPipelineViewportStateCreateInfo-x-02821
  The `x` and `y` members of `offset` member of any element of `pScissors` **must** be greater than or equal to `0`
- VUID-VkPipelineViewportStateCreateInfo-offset-02822
  Evaluation of (`offset.x` + `extent.width`) **must** not cause a signed integer addition overflow for any element of `pScissors`
- VUID-VkPipelineViewportStateCreateInfo-offset-02823
  Evaluation of (`offset.y` + `extent.height`) **must** not cause a signed integer addition overflow for any element of `pScissors`
- VUID-VkPipelineViewportStateCreateInfo-scissorCount-04134
  If `scissorCount` and `viewportCount` are both not dynamic, then `scissorCount` and

`viewportCount` **must** be identical

- VUID-VkPipelineViewportStateCreateInfo-viewportCount-04135
  `viewportCount` **must** be greater than `0`
- VUID-VkPipelineViewportStateCreateInfo-scissorCount-04136
  `scissorCount` **must** be greater than `0`

Valid Usage (Implicit)

- VUID-VkPipelineViewportStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO`
- VUID-VkPipelineViewportStateCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkPipelineViewportDepthClampControlCreateInfoEXT or VkPipelineViewportDepthClipControlCreateInfoEXT
- VUID-VkPipelineViewportStateCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPipelineViewportStateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`

To dynamically set the viewport count and viewports, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetViewportWithCountEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    viewportCount,
    const VkViewport*                           pViewports);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `viewportCount` specifies the viewport count.
- `pViewports` specifies the viewports to use for drawing.

This command sets the viewport count and viewports state for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the corresponding VkPipelineViewportStateCreateInfo::`viewportCount` and `pViewports` values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetViewportWithCount-None-08971
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

- VUID-vkCmdSetViewportWithCount-viewportCount-03394
  `viewportCount` **must** be between `1` and `VkPhysicalDeviceLimits`::`maxViewports`, inclusive
- VUID-vkCmdSetViewportWithCount-viewportCount-03395
  If the `multiViewport` feature is not enabled, `viewportCount` **must** be `1`

Valid Usage (Implicit)

- VUID-vkCmdSetViewportWithCount-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetViewportWithCount-pViewports-parameter
  `pViewports` **must** be a valid pointer to an array of `viewportCount` valid VkViewport structures
- VUID-vkCmdSetViewportWithCount-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetViewportWithCount-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetViewportWithCount-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetViewportWithCount-viewportCount-arraylength
  `viewportCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the scissor count and scissor rectangular bounds, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetScissorWithCountEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    scissorCount,
    const VkRect2D*                             pScissors);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `scissorCount` specifies the scissor count.
- `pScissors` specifies the scissors to use for drawing.

This command sets the scissor count and scissor rectangular bounds state for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the corresponding VkPipelineViewportStateCreateInfo::`scissorCount` and `pScissors` values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetScissorWithCount-None-08971
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled
- VUID-vkCmdSetScissorWithCount-scissorCount-03397
  `scissorCount` **must** be between `1` and `VkPhysicalDeviceLimits`::`maxViewports`, inclusive
- VUID-vkCmdSetScissorWithCount-scissorCount-03398
  If the `multiViewport` feature is not enabled, `scissorCount` **must** be `1`
- VUID-vkCmdSetScissorWithCount-x-03399
  The `x` and `y` members of `offset` member of any element of `pScissors` **must** be greater than or equal to `0`
- VUID-vkCmdSetScissorWithCount-offset-03400
  Evaluation of (`offset.x` + `extent.width`) **must** not cause a signed integer addition overflow for any element of `pScissors`
- VUID-vkCmdSetScissorWithCount-offset-03401
  Evaluation of (`offset.y` + `extent.height`) **must** not cause a signed integer addition overflow for any element of `pScissors`

Valid Usage (Implicit)

- VUID-vkCmdSetScissorWithCount-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetScissorWithCount-pScissors-parameter
  `pScissors` **must** be a valid pointer to an array of `scissorCount` VkRect2D structures
- VUID-vkCmdSetScissorWithCount-commandBuffer-recording
  `commandBuffer` **must** be in the recording state

- VUID-vkCmdSetScissorWithCount-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetScissorWithCount-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetScissorWithCount-scissorCount-arraylength
  `scissorCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineViewportStateCreateFlags;
```

`VkPipelineViewportStateCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

If a geometry shader is active and has an output variable decorated with `ViewportIndex`, the viewport transformation uses the viewport corresponding to the value assigned to `ViewportIndex` taken from an implementation-dependent vertex of each primitive. If `ViewportIndex` is outside the range zero to `viewportCount` minus one for a primitive, or if the geometry shader did not assign a value to `ViewportIndex` for all vertices of a primitive due to flow control, the values resulting from the viewport transformation of the vertices of such primitives are undefined. If no geometry shader is active, or if the geometry shader does not have an output decorated with `ViewportIndex`, the viewport numbered zero is used by the viewport transformation.

A single vertex **can** be used in more than one individual primitive, in primitives such as `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP`. In this case, the viewport transformation is applied separately for each primitive.

To dynamically set the viewport transformation parameters, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetViewport(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstViewport,
    uint32_t                                    viewportCount,
    const VkViewport*                           pViewports);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `firstViewport` is the index of the first viewport whose parameters are updated by the command.
- `viewportCount` is the number of viewports whose parameters are updated by the command.
- `pViewports` is a pointer to an array of VkViewport structures specifying viewport parameters.

This command sets the viewport transformation parameters state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_VIEWPORT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the `VkPipelineViewportStateCreateInfo`::`pViewports` values used to create the currently active pipeline.

The viewport parameters taken from element i of `pViewports` replace the current state for the viewport index `firstViewport` + i, for i in [0, `viewportCount`).

Valid Usage

- VUID-vkCmdSetViewport-firstViewport-01223
  The sum of `firstViewport` and `viewportCount` **must** be between `1` and `VkPhysicalDeviceLimits`::`maxViewports`, inclusive
- VUID-vkCmdSetViewport-firstViewport-01224
  If the `multiViewport` feature is not enabled, `firstViewport` **must** be `0`
- VUID-vkCmdSetViewport-viewportCount-01225
  If the `multiViewport` feature is not enabled, `viewportCount` **must** be `1`

Valid Usage (Implicit)

- VUID-vkCmdSetViewport-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetViewport-pViewports-parameter
  `pViewports` **must** be a valid pointer to an array of `viewportCount` valid VkViewport structures
- VUID-vkCmdSetViewport-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetViewport-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations

- VUID-vkCmdSetViewport-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetViewport-viewportCount-arraylength
  `viewportCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Both VkPipelineViewportStateCreateInfo and vkCmdSetViewport use `VkViewport` to set the viewport transformation parameters.

The `VkViewport` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkViewport {
    float    x;
    float    y;
    float    width;
    float    height;
    float    minDepth;
    float    maxDepth;
} VkViewport;
```

- `x` and `y` are the viewport's upper left corner (x,y).
- `width` and `height` are the viewport's width and height, respectively.
- `minDepth` and `maxDepth` are the depth range for the viewport.

NOTE Despite their names, `minDepth` **can** be less than, equal to, or greater than `maxDepth`.

The framebuffer depth coordinate z_f **may** be represented using either a fixed-point or floating-point representation. However, a floating-point representation **must** be used if the depth/stencil attachment has a floating-point depth component. If an m-bit fixed-point representation is used, we

assume that it represents each value $\frac{k}{2^m - 1}$, where k ∈ { 0, 1, ..., 2^m-1 }, as k (e.g. 1.0 is represented in binary as a string of all ones).

The viewport parameters shown in the above equations are found from these values as

o_x = `x` + `width` / 2

o_y = `y` + `height` / 2

o_z = `minDepth` (or (`maxDepth` + `minDepth`) / 2 if VkPipelineViewportDepthClipControlCreateInfoEXT::`negativeOneToOne` is `VK_TRUE`)

p_x = `width`

p_y = `height`

p_z = `maxDepth` - `minDepth` (or (`maxDepth` - `minDepth`) / 2 if VkPipelineViewportDepthClipControlCreateInfoEXT::`negativeOneToOne` is `VK_TRUE`)

The application **can** specify a negative term for `height`, which has the effect of negating the y coordinate in clip space before performing the transform. When using a negative `height`, the application **should** also adjust the `y` value to point to the lower left corner of the viewport instead of the upper left corner. Using the negative `height` allows the application to avoid having to negate the y component of the `Position` output from the last pre-rasterization shader stage.

The width and height of the implementation-dependent maximum viewport dimensions **must** be greater than or equal to the width and height of the largest image which **can** be created and attached to a framebuffer.

The floating-point viewport bounds are represented with an implementation-dependent precision.

Valid Usage

- VUID-VkViewport-width-01770
  `width` **must** be greater than `0.0`
- VUID-VkViewport-width-01771
  `width` **must** be less than or equal to VkPhysicalDeviceLimits::`maxViewportDimensions`[0]
- VUID-VkViewport-apiVersion-07917
  If the VK_KHR_maintenance1 extension is not enabled, the `VK_AMD_negative_viewport_height` extension is not enabled, and VkPhysicalDeviceProperties::`apiVersion` is less than Vulkan 1.1, `height` **must** be greater than `0.0`

- VUID-VkViewport-height-01773
  The absolute value of `height` **must** be less than or equal to `VkPhysicalDeviceLimits::maxViewportDimensions`[1]
- VUID-VkViewport-x-01774
  `x` **must** be greater than or equal to `viewportBoundsRange`[0]
- VUID-VkViewport-x-01232
  (`x` + `width`) **must** be less than or equal to `viewportBoundsRange`[1]
- VUID-VkViewport-y-01775
  `y` **must** be greater than or equal to `viewportBoundsRange`[0]
- VUID-VkViewport-y-01776
  `y` **must** be less than or equal to `viewportBoundsRange`[1]
- VUID-VkViewport-y-01777
  (`y` + `height`) **must** be greater than or equal to `viewportBoundsRange`[0]
- VUID-VkViewport-y-01233
  (`y` + `height`) **must** be less than or equal to `viewportBoundsRange`[1]
- VUID-VkViewport-minDepth-01234
  If the `VK_EXT_depth_range_unrestricted` extension is not enabled, `minDepth` **must** be between `0.0` and `1.0`, inclusive
- VUID-VkViewport-maxDepth-01235
  If the `VK_EXT_depth_range_unrestricted` extension is not enabled, `maxDepth` **must** be between `0.0` and `1.0`, inclusive

Chapter 26. Rasterization

Rasterization is the process by which a primitive is converted to a two-dimensional image. Each discrete location of this image contains associated data such as depth, color, or other attributes.

Rasterizing a primitive begins by determining which squares of an integer grid in framebuffer coordinates are occupied by the primitive, and assigning one or more depth values to each such square. This process is described below for points, lines, and polygons.

A grid square, including its (x,y) framebuffer coordinates, z (depth), and associated data added by fragment shaders, is called a fragment. A fragment is located by its upper left corner, which lies on integer grid coordinates.

Rasterization operations also refer to a fragment's sample locations, which are offset by fractional values from its upper left corner. The rasterization rules for points, lines, and triangles involve testing whether each sample location is inside the primitive. Fragments need not actually be square, and rasterization rules are not affected by the aspect ratio of fragments. Display of non-square grids, however, will cause rasterized points and line segments to appear fatter in one direction than the other.

We assume that fragments are square, since it simplifies antialiasing and texturing. After rasterization, fragments are processed by fragment operations.

Several factors affect rasterization, including the members of VkPipelineRasterizationStateCreateInfo and VkPipelineMultisampleStateCreateInfo.

The `VkPipelineRasterizationStateCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineRasterizationStateCreateInfo {
    VkStructureType                            sType;
    const void*                                pNext;
    VkPipelineRasterizationStateCreateFlags    flags;
    VkBool32                                   depthClampEnable;
    VkBool32                                   rasterizerDiscardEnable;
    VkPolygonMode                              polygonMode;
    VkCullModeFlags                            cullMode;
    VkFrontFace                                frontFace;
    VkBool32                                   depthBiasEnable;
    float                                      depthBiasConstantFactor;
    float                                      depthBiasClamp;
    float                                      depthBiasSlopeFactor;
    float                                      lineWidth;
} VkPipelineRasterizationStateCreateInfo;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.

- `depthClampEnable` controls whether to clamp the fragment's depth values as described in Depth Test. If the pipeline is not created with VkPipelineRasterizationDepthClipStateCreateInfoEXT present then enabling depth clamp will also disable clipping primitives to the z planes of the frustum as described in Primitive Clipping. Otherwise depth clipping is controlled by the state set in VkPipelineRasterizationDepthClipStateCreateInfoEXT.
- `rasterizerDiscardEnable` controls whether primitives are discarded immediately before the rasterization stage.
- `polygonMode` is the triangle rendering mode. See VkPolygonMode.
- `cullMode` is the triangle facing direction used for primitive culling. See VkCullModeFlagBits.
- `frontFace` is a VkFrontFace value specifying the front-facing triangle orientation to be used for culling.
- `depthBiasEnable` controls whether to bias fragment depth values.
- `depthBiasConstantFactor` is a scalar factor controlling the constant depth value added to each fragment.
- `depthBiasClamp` is the maximum (or minimum) depth bias of a fragment.
- `depthBiasSlopeFactor` is a scalar factor applied to a fragment's slope in depth bias calculations.
- `lineWidth` is the width of rasterized line segments.

Valid Usage

- VUID-VkPipelineRasterizationStateCreateInfo-depthClampEnable-00782
  If the `depthClamp` feature is not enabled, `depthClampEnable` **must** be `VK_FALSE`
- VUID-VkPipelineRasterizationStateCreateInfo-polygonMode-01507
  If the `fillModeNonSolid` feature is not enabled, `polygonMode` **must** be `VK_POLYGON_MODE_FILL`
- VUID-VkPipelineRasterizationStateCreateInfo-pointPolygons-04458
  If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`pointPolygons` is `VK_FALSE`, and `rasterizerDiscardEnable` is `VK_FALSE`, `polygonMode` **must** not be `VK_POLYGON_MODE_POINT`

Valid Usage (Implicit)

- VUID-VkPipelineRasterizationStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO`
- VUID-VkPipelineRasterizationStateCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkDepthBiasRepresentationInfoEXT, VkPipelineRasterizationConservativeStateCreateInfoEXT, VkPipelineRasterizationDepthClipStateCreateInfoEXT, VkPipelineRasterizationLineStateCreateInfoKHR, VkPipelineRasterizationProvokingVertexStateCreateInfoEXT, or VkPipelineRasterizationStateStreamCreateInfoEXT

- VUID-VkPipelineRasterizationStateCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPipelineRasterizationStateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineRasterizationStateCreateInfo-polygonMode-parameter
  `polygonMode` **must** be a valid VkPolygonMode value
- VUID-VkPipelineRasterizationStateCreateInfo-cullMode-parameter
  `cullMode` **must** be a valid combination of VkCullModeFlagBits values
- VUID-VkPipelineRasterizationStateCreateInfo-frontFace-parameter
  `frontFace` **must** be a valid VkFrontFace value

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineRasterizationStateCreateFlags;
```

`VkPipelineRasterizationStateCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

If the `pNext` chain of VkPipelineRasterizationStateCreateInfo includes a `VkPipelineRasterizationDepthClipStateCreateInfoEXT` structure, then that structure controls whether depth clipping is enabled or disabled.

The `VkPipelineRasterizationDepthClipStateCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_depth_clip_enable
typedef struct VkPipelineRasterizationDepthClipStateCreateInfoEXT {
    VkStructureType                                        sType;
    const void*                                            pNext;
    VkPipelineRasterizationDepthClipStateCreateFlagsEXT    flags;
    VkBool32                                               depthClipEnable;
} VkPipelineRasterizationDepthClipStateCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `depthClipEnable` controls whether depth clipping is enabled as described in Primitive Clipping.

Valid Usage (Implicit)

- VUID-VkPipelineRasterizationDepthClipStateCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_DEPTH_CLIP_STATE_CREATE_INFO_EXT`
- VUID-VkPipelineRasterizationDepthClipStateCreateInfoEXT-flags-zerobitmask
  `flags` **must** be `0`

```
// Provided by VK_EXT_depth_clip_enable
typedef VkFlags VkPipelineRasterizationDepthClipStateCreateFlagsEXT;
```

VkPipelineRasterizationDepthClipStateCreateFlagsEXT is a bitmask type for setting a mask, but is currently reserved for future use.

The VkPipelineMultisampleStateCreateInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineMultisampleStateCreateInfo {
    VkStructureType                         sType;
    const void*                             pNext;
    VkPipelineMultisampleStateCreateFlags   flags;
    VkSampleCountFlagBits                   rasterizationSamples;
    VkBool32                                sampleShadingEnable;
    float                                   minSampleShading;
    const VkSampleMask*                     pSampleMask;
    VkBool32                                alphaToCoverageEnable;
    VkBool32                                alphaToOneEnable;
} VkPipelineMultisampleStateCreateInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- rasterizationSamples is a VkSampleCountFlagBits value specifying the number of samples used in rasterization. This value is ignored for the purposes of setting the number of samples used in rasterization if the pipeline is created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state set, but if VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state is not set, it is still used to define the size of the pSampleMask array as described below.
- sampleShadingEnable **can** be used to enable Sample Shading.
- minSampleShading specifies a minimum fraction of sample shading if sampleShadingEnable is VK_TRUE.
- pSampleMask is a pointer to an array of VkSampleMask values used in the sample mask test.
- alphaToCoverageEnable controls whether a temporary coverage value is generated based on the alpha component of the fragment's first color output as specified in the Multisample Coverage section.
- alphaToOneEnable controls whether the alpha component of the fragment's first color output is replaced with one as described in Multisample Coverage.

Each bit in the sample mask is associated with a unique sample index as defined for the coverage mask. Each bit b for mask word w in the sample mask corresponds to sample index i, where i = 32 × w + b. pSampleMask has a length equal to ⌈ rasterizationSamples / 32 ⌉ words.

If pSampleMask is NULL, it is treated as if the mask has all bits set to 1.

Valid Usage

- VUID-VkPipelineMultisampleStateCreateInfo-sampleShadingEnable-00784
  If the `sampleRateShading` feature is not enabled, `sampleShadingEnable` **must** be `VK_FALSE`
- VUID-VkPipelineMultisampleStateCreateInfo-alphaToOneEnable-00785
  If the `alphaToOne` feature is not enabled, `alphaToOneEnable` **must** be `VK_FALSE`
- VUID-VkPipelineMultisampleStateCreateInfo-minSampleShading-00786
  `minSampleShading` **must** be in the range [0,1]

Valid Usage (Implicit)

- VUID-VkPipelineMultisampleStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO`
- VUID-VkPipelineMultisampleStateCreateInfo-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineSampleLocationsStateCreateInfoEXT
- VUID-VkPipelineMultisampleStateCreateInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPipelineMultisampleStateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineMultisampleStateCreateInfo-rasterizationSamples-parameter
  `rasterizationSamples` **must** be a valid VkSampleCountFlagBits value
- VUID-VkPipelineMultisampleStateCreateInfo-pSampleMask-parameter
  If `pSampleMask` is not `NULL`, `pSampleMask` **must** be a valid pointer to an array of $\lceil \frac{rasterizationSamples}{32} \rceil$ VkSampleMask values

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineMultisampleStateCreateFlags;
```

`VkPipelineMultisampleStateCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

The elements of the sample mask array are of type VkSampleMask, each representing 32 bits of coverage information:

```
// Provided by VK_VERSION_1_0
typedef uint32_t VkSampleMask;
```

Rasterization only generates fragments which cover one or more pixels inside the framebuffer. Pixels outside the framebuffer are never considered covered in the fragment. Fragments which would be produced by application of any of the primitive rasterization rules described below but

which lie outside the framebuffer are not produced, nor are they processed by any later stage of the pipeline, including any of the fragment operations.

Surviving fragments are processed by fragment shaders. Fragment shaders determine associated data for fragments, and **can** also modify or replace their assigned depth values.

26.1. Discarding Primitives Before Rasterization

Primitives are discarded before rasterization if the `rasterizerDiscardEnable` member of VkPipelineRasterizationStateCreateInfo is enabled. When enabled, primitives are discarded after they are processed by the last active shader stage in the pipeline before rasterization.

To dynamically enable whether primitives are discarded before the rasterization stage, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetRasterizerDiscardEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    rasterizerDiscardEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `rasterizerDiscardEnable` controls whether primitives are discarded immediately before the rasterization stage.

This command sets the discard enable for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineRasterizationStateCreateInfo ::`rasterizerDiscardEnable` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetRasterizerDiscardEnable-None-08970
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetRasterizerDiscardEnable-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetRasterizerDiscardEnable-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetRasterizerDiscardEnable-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetRasterizerDiscardEnable-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

26.2. Controlling the Vertex Stream Used for Rasterization

By default vertex data output from the last pre-rasterization shader stage are directed to vertex stream zero. Geometry shaders **can** emit primitives to multiple independent vertex streams. Each vertex emitted by the geometry shader is directed at one of the vertex streams. As vertices are received on each vertex stream, they are arranged into primitives of the type specified by the geometry shader output primitive type. The shading language instructions `OpEndPrimitive` and `OpEndStreamPrimitive` **can** be used to end the primitive being assembled on a given vertex stream and start a new empty primitive of the same type. An implementation supports up to `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackStreams` streams, which is at least 1. The individual streams are numbered 0 through `maxTransformFeedbackStreams` minus 1. There is no requirement on the order of the streams to which vertices are emitted, and the number of vertices emitted to each vertex stream **can** be completely independent, subject only to the `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackStreamDataSize` and `VkPhysicalDeviceTransformFeedbackPropertiesEXT`::`maxTransformFeedbackBufferDataSize` limits. The primitives output from all vertex streams are passed to the transform feedback stage to be captured to transform feedback buffers in the manner specified by the last pre-rasterization shader stage shader's `XfbBuffer`, `XfbStride`, and `Offsets` decorations on the output interface variables in the graphics pipeline. To use a vertex stream other than zero, or to use multiple streams, the `GeometryStreams` capability **must** be specified.

By default, the primitives output from vertex stream zero are rasterized. If the implementation supports the VkPhysicalDeviceTransformFeedbackPropertiesEXT::`transformFeedbackRasterizationStreamSelect` property it is possible to rasterize a vertex stream other than zero.

By default, geometry shaders that emit vertices to multiple vertex streams are limited to using only the `OutputPoints` output primitive type. If the implementation supports the VkPhysicalDeviceTransformFeedbackPropertiesEXT::`transformFeedbackStreamsLinesTriangles` property it is possible to emit `OutputLineStrip` or `OutputTriangleStrip` in addition to `OutputPoints`.

The vertex stream used for rasterization is specified by adding a VkPipelineRasterizationStateStreamCreateInfoEXT structure to the pNext chain of a VkPipelineRasterizationStateCreateInfo structure.

The VkPipelineRasterizationStateStreamCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_transform_feedback
typedef struct VkPipelineRasterizationStateStreamCreateInfoEXT {
    VkStructureType                                    sType;
    const void*                                        pNext;
    VkPipelineRasterizationStateStreamCreateFlagsEXT   flags;
    uint32_t                                           rasterizationStream;
} VkPipelineRasterizationStateStreamCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- rasterizationStream is the vertex stream selected for rasterization.

If this structure is not present, rasterizationStream is assumed to be zero.

Valid Usage

- VUID-VkPipelineRasterizationStateStreamCreateInfoEXT-geometryStreams-02324 VkPhysicalDeviceTransformFeedbackFeaturesEXT::geometryStreams **must** be enabled
- VUID-VkPipelineRasterizationStateStreamCreateInfoEXT-rasterizationStream-02325 rasterizationStream **must** be less than VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackStreams
- VUID-VkPipelineRasterizationStateStreamCreateInfoEXT-rasterizationStream-02326 rasterizationStream **must** be zero if VkPhysicalDeviceTransformFeedbackPropertiesEXT::transformFeedbackRasterizationStreamSelect is VK_FALSE

Valid Usage (Implicit)

- VUID-VkPipelineRasterizationStateStreamCreateInfoEXT-sType-sType sType **must** be VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_STREAM_CREATE_INFO_EXT
- VUID-VkPipelineRasterizationStateStreamCreateInfoEXT-flags-zerobitmask flags **must** be 0

```
// Provided by VK_EXT_transform_feedback
typedef VkFlags VkPipelineRasterizationStateStreamCreateFlagsEXT;
```

VkPipelineRasterizationStateStreamCreateFlagsEXT is a bitmask type for setting a mask, but is currently reserved for future use.

To dynamically set the rasterizationStream state, call:

```
// Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_transform_feedback,
VK_EXT_shader_object with VK_EXT_transform_feedback
void vkCmdSetRasterizationStreamEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    rasterizationStream);
```

- commandBuffer is the command buffer into which the command will be recorded.
- rasterizationStream specifies the rasterizationStream state.

This command sets the rasterizationStream state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetRasterizationStreamEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3RasterizationStream feature is enabled
  - The shaderObject feature is enabled
- VUID-vkCmdSetRasterizationStreamEXT-transformFeedback-07411
  The transformFeedback feature **must** be enabled
- VUID-vkCmdSetRasterizationStreamEXT-rasterizationStream-07412
  rasterizationStream **must** be less than VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackStreams
- VUID-vkCmdSetRasterizationStreamEXT-rasterizationStream-07413
  rasterizationStream **must** be zero if VkPhysicalDeviceTransformFeedbackPropertiesEXT::transformFeedbackRasterizationStreamSelect is VK_FALSE

Valid Usage (Implicit)

- VUID-vkCmdSetRasterizationStreamEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetRasterizationStreamEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetRasterizationStreamEXT-commandBuffer-cmdpool

The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations

- VUID-vkCmdSetRasterizationStreamEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

26.3. Rasterization Order

Within a subpass of a render pass instance, for a given (x,y,layer,sample) sample location, the following operations are guaranteed to execute in *rasterization order*, for each separate primitive that includes that sample location:

1. Fragment operations, in the order defined
2. Blending, logic operations, and color writes

Execution of these operations for each primitive in a subpass occurs in primitive order.

26.4. Multisampling

Multisampling is a mechanism to antialias all Vulkan primitives: points, lines, and polygons. The technique is to sample all primitives multiple times at each pixel. Each sample in each framebuffer attachment has storage for a color, depth, and/or stencil value, such that per-fragment operations apply to each sample independently. The color sample values **can** be later *resolved* to a single color (see Resolving Multisample Images and the Render Pass chapter for more details on how to resolve multisample images to non-multisample images).

Vulkan defines rasterization rules for single-sample modes in a way that is equivalent to a multisample mode with a single sample in the center of each fragment.

Each fragment includes a coverage mask with a single bit for each sample in the fragment, and a number of depth values and associated data for each sample.

It is understood that each pixel has rasterizationSamples locations associated with it. These locations are exact positions, rather than regions or areas, and each is referred to as a sample point. The sample points associated with a pixel **must** be located inside or on the boundary of the unit square that is considered to bound the pixel. Furthermore, the relative locations of sample points **may** be identical for each pixel in the framebuffer, or they **may** differ.

If the render pass has a fragment density map attachment, each fragment only has rasterizationSamples locations associated with it regardless of how many pixels are covered in the fragment area. Fragment sample locations are defined as if the fragment had an area of (1,1) and its sample points **must** be located within these bounds. Their actual location in the framebuffer is calculated by scaling the sample location by the fragment area. Attachments with storage for multiple samples per pixel are located at the pixel sample locations. Otherwise, the fragment's sample locations are generally used for evaluation of associated data and fragment operations.

If the current pipeline includes a fragment shader with one or more variables in its interface decorated with Sample and Input, the data associated with those variables will be assigned independently for each sample. The values for each sample **must** be evaluated at the location of the sample. The data associated with any other variables not decorated with Sample and Input need not be evaluated independently for each sample.

A *coverage mask* is generated for each fragment, based on which samples within that fragment are determined to be within the area of the primitive that generated the fragment.

Single pixel fragments and multi-pixel fragments defined by a fragment density map have one set of samples. Multi-pixel fragments defined by setting the fragment shading rate have one set of samples per pixel. Each set of samples has a number of samples determined by VkPipelineMultisampleStateCreateInfo::rasterizationSamples. Each sample in a set is assigned a unique *sample index* i in the range [0, rasterizationSamples).

To dynamically set the rasterizationSamples, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetRasterizationSamplesEXT(
    VkCommandBuffer                             commandBuffer,
    VkSampleCountFlagBits                       rasterizationSamples);
```

- commandBuffer is the command buffer into which the command will be recorded.
- rasterizationSamples specifies rasterizationSamples.

This command sets the rasterizationSamples for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineMultisampleStateCreateInfo::rasterizationSamples value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetRasterizationSamplesEXT-None-09423

At least one of the following **must** be true:

- The `extendedDynamicState3RasterizationSamples` feature is enabled
- The `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetRasterizationSamplesEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetRasterizationSamplesEXT-rasterizationSamples-parameter
  `rasterizationSamples` **must** be a valid VkSampleCountFlagBits value
- VUID-vkCmdSetRasterizationSamplesEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetRasterizationSamplesEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetRasterizationSamplesEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Each sample in a fragment is also assigned a unique *coverage index* j in the range [0, n × `rasterizationSamples`), where n is the number of sets in the fragment. If the fragment contains a single set of samples, the *coverage index* is always equal to the *sample index*.

If the fragment shading rate is set, the coverage index j is determined as a function of the *pixel index* p, the *sample index* i, and the number of rasterization samples r as:

$$j = i + r \times ((f_w \times f_h) - 1 - p)$$

where the pixel index p is determined as a function of the pixel's framebuffer location (x,y) and the fragment size (f_w,f_h):

$$p_x = x \% f_w$$

$$p_y = y \% f_h$$

$$p = p_x + (p_y \times f_w)$$

The tables below illustrate the pixel index for multi-pixel fragments:

Table 30. Pixel Indices - 1 Wide

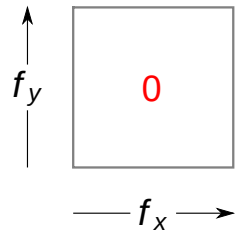
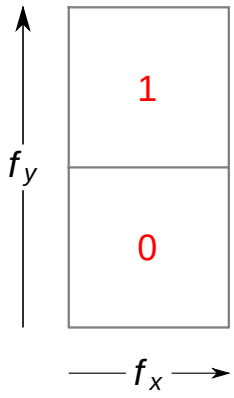
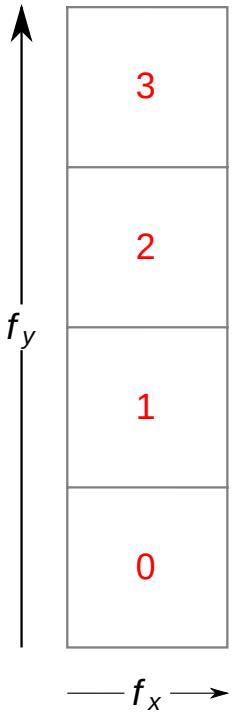
1x1	1x2	1x4
		

Table 31. Pixel Indices - 2 Wide

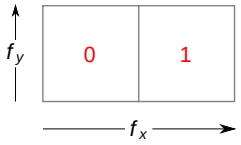
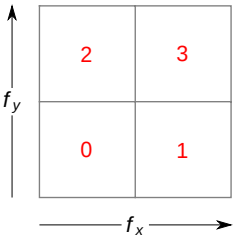
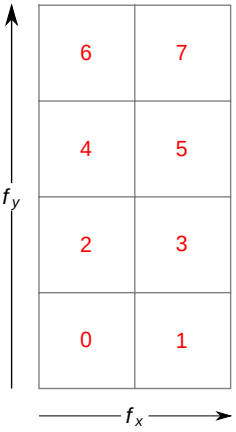
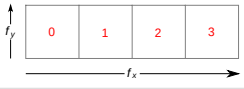
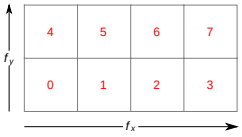
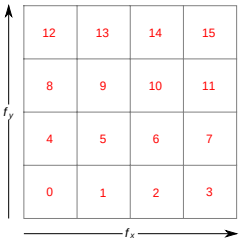
2x1	2x2	2x4
		

Table 32. Pixel Indices - 4 Wide

4x1	4x2	4x4
		

The coverage mask includes B bits packed into W words, defined as:

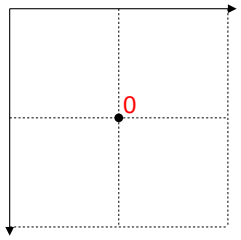
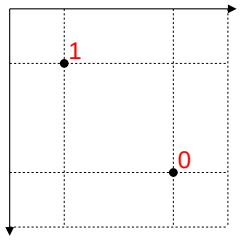
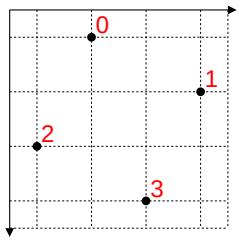
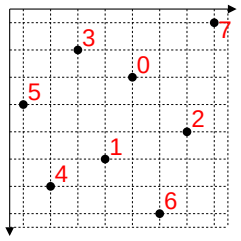
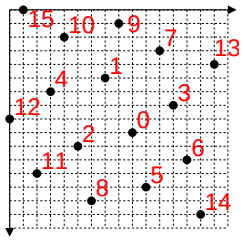
$$B = n \times \texttt{rasterizationSamples}$$

$$W = \lceil B/32 \rceil$$

Bit b in coverage mask word w is `1` if the sample with coverage index $j = 32 \times w + b$ is covered, and `0` otherwise.

If the `standardSampleLocations` member of VkPhysicalDeviceLimits is `VK_TRUE`, then the sample counts `VK_SAMPLE_COUNT_1_BIT`, `VK_SAMPLE_COUNT_2_BIT`, `VK_SAMPLE_COUNT_4_BIT`, `VK_SAMPLE_COUNT_8_BIT`, and `VK_SAMPLE_COUNT_16_BIT` have sample locations as listed in the following table, with the ith entry in the table corresponding to sample index i. `VK_SAMPLE_COUNT_32_BIT` and `VK_SAMPLE_COUNT_64_BIT` do not have standard sample locations. Locations are defined relative to an origin in the upper left corner of the fragment.

Table 33. Standard Sample Locations

Sample count	Sample Locations	
VK_SAMPLE_COUNT_1_BIT	(0.5,0.5)	
VK_SAMPLE_COUNT_2_BIT	(0.75,0.75) (0.25,0.25)	
VK_SAMPLE_COUNT_4_BIT	(0.375, 0.125) (0.875, 0.375) (0.125, 0.625) (0.625, 0.875)	
VK_SAMPLE_COUNT_8_BIT	(0.5625, 0.3125) (0.4375, 0.6875) (0.8125, 0.5625) (0.3125, 0.1875) (0.1875, 0.8125) (0.0625, 0.4375) (0.6875, 0.9375) (0.9375, 0.0625)	
VK_SAMPLE_COUNT_16_BIT	(0.5625, 0.5625) (0.4375, 0.3125) (0.3125, 0.625) (0.75, 0.4375) (0.1875, 0.375) (0.625, 0.8125) (0.8125, 0.6875) (0.6875, 0.1875) (0.375, 0.875) (0.5, 0.0625) (0.25, 0.125) (0.125, 0.75) (0.0, 0.5) (0.9375, 0.25) (0.875, 0.9375) (0.0625, 0.0)	

26.5. Custom Sample Locations

Applications **can** also control the sample locations used for rasterization.

If the pNext chain of the VkPipelineMultisampleStateCreateInfo structure specified at pipeline creation time includes a VkPipelineSampleLocationsStateCreateInfoEXT structure, then that structure controls the sample locations used when rasterizing primitives with the pipeline.

The VkPipelineSampleLocationsStateCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_sample_locations
typedef struct VkPipelineSampleLocationsStateCreateInfoEXT {
    VkStructureType             sType;
    const void*                 pNext;
    VkBool32                    sampleLocationsEnable;
    VkSampleLocationsInfoEXT    sampleLocationsInfo;
} VkPipelineSampleLocationsStateCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- sampleLocationsEnable controls whether custom sample locations are used. If sampleLocationsEnable is VK_FALSE, the default sample locations are used and the values specified in sampleLocationsInfo are ignored.
- sampleLocationsInfo is the sample locations to use during rasterization if sampleLocationsEnable is VK_TRUE and the graphics pipeline is not created with VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT.

Valid Usage (Implicit)

- VUID-VkPipelineSampleLocationsStateCreateInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_SAMPLE_LOCATIONS_STATE_CREATE_INFO_EXT
- VUID-VkPipelineSampleLocationsStateCreateInfoEXT-sampleLocationsInfo-parameter
  sampleLocationsInfo **must** be a valid VkSampleLocationsInfoEXT structure

The VkSampleLocationsInfoEXT structure is defined as:

```
// Provided by VK_EXT_sample_locations
typedef struct VkSampleLocationsInfoEXT {
    VkStructureType               sType;
    const void*                   pNext;
    VkSampleCountFlagBits         sampleLocationsPerPixel;
    VkExtent2D                    sampleLocationGridSize;
    uint32_t                      sampleLocationsCount;
    const VkSampleLocationEXT*    pSampleLocations;
} VkSampleLocationsInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `sampleLocationsPerPixel` is a VkSampleCountFlagBits value specifying the number of sample locations per pixel.
- `sampleLocationGridSize` is the size of the sample location grid to select custom sample locations for.
- `sampleLocationsCount` is the number of sample locations in `pSampleLocations`.
- `pSampleLocations` is a pointer to an array of `sampleLocationsCount` VkSampleLocationEXT structures.

This structure **can** be used either to specify the sample locations to be used for rendering or to specify the set of sample locations an image subresource has been last rendered with for the purposes of layout transitions of depth/stencil images created with `VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT`.

The sample locations in `pSampleLocations` specify `sampleLocationsPerPixel` number of sample locations for each pixel in the grid of the size specified in `sampleLocationGridSize`. The sample location for sample i at the pixel grid location (x,y) is taken from `pSampleLocations`[(x + y × `sampleLocationGridSize.width`) × `sampleLocationsPerPixel` + i].

If the render pass has a fragment density map, the implementation will choose the sample locations for the fragment and the contents of `pSampleLocations` **may** be ignored.

Valid Usage

- VUID-VkSampleLocationsInfoEXT-sampleLocationsPerPixel-01526
  `sampleLocationsPerPixel` **must** be a valid VkSampleCountFlagBits value that is set in VkPhysicalDeviceSampleLocationsPropertiesEXT::`sampleLocationSampleCounts`
- VUID-VkSampleLocationsInfoEXT-sampleLocationsCount-01527
  `sampleLocationsCount` **must** equal `sampleLocationsPerPixel` × `sampleLocationGridSize.width` × `sampleLocationGridSize.height`

Valid Usage (Implicit)

- VUID-VkSampleLocationsInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SAMPLE_LOCATIONS_INFO_EXT`
- VUID-VkSampleLocationsInfoEXT-pSampleLocations-parameter
  If `sampleLocationsCount` is not `0`, `pSampleLocations` **must** be a valid pointer to an array of `sampleLocationsCount` VkSampleLocationEXT structures

The `VkSampleLocationEXT` structure is defined as:

```
// Provided by VK_EXT_sample_locations
typedef struct VkSampleLocationEXT {
    float    x;
    float    y;
} VkSampleLocationEXT;
```

- `x` is the horizontal coordinate of the sample's location.
- `y` is the vertical coordinate of the sample's location.

The domain space of the sample location coordinates has an upper-left origin within the pixel in framebuffer space.

The values specified in a `VkSampleLocationEXT` structure are always clamped to the implementation-dependent sample location coordinate range [`sampleLocationCoordinateRange`[0],`sampleLocationCoordinateRange`[1]] that **can** be queried using VkPhysicalDeviceSampleLocationsPropertiesEXT.

To dynamically set the `sampleLocationsEnable` state, call:

```
// Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_sample_locations,
VK_EXT_sample_locations with VK_EXT_shader_object
void vkCmdSetSampleLocationsEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    sampleLocationsEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `sampleLocationsEnable` specifies the `sampleLocationsEnable` state.

This command sets the `sampleLocationsEnable` state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineSampleLocationsStateCreateInfoEXT::`sampleLocationsEnable` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetSampleLocationsEnableEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3SampleLocationsEnable` feature is enabled
  - The `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetSampleLocationsEnableEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetSampleLocationsEnableEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetSampleLocationsEnableEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetSampleLocationsEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the sample locations used for rasterization, call:

```
// Provided by VK_EXT_sample_locations
void vkCmdSetSampleLocationsEXT(
    VkCommandBuffer                             commandBuffer,
    const VkSampleLocationsInfoEXT*             pSampleLocationsInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pSampleLocationsInfo` is the sample locations state to set.

This command sets the custom sample locations for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`, and when the VkPipelineSampleLocationsStateCreateInfoEXT::`sampleLocationsEnable` property of the bound graphics pipeline is `VK_TRUE`. Otherwise, this state is specified by the VkPipelineSampleLocationsStateCreateInfoEXT::`sampleLocationsInfo` values used to create the

currently active pipeline.

Valid Usage

- VUID-vkCmdSetSampleLocationsEXT-variableSampleLocations-01530
  If VkPhysicalDeviceSampleLocationsPropertiesEXT::`variableSampleLocations` is `VK_FALSE` then the current render pass **must** have been begun by specifying a VkRenderPassSampleLocationsBeginInfoEXT structure whose `pPostSubpassSampleLocations` member contains an element with a `subpassIndex` matching the current subpass index and the `sampleLocationsInfo` member of that element **must** match the sample locations state pointed to by `pSampleLocationsInfo`

Valid Usage (Implicit)

- VUID-vkCmdSetSampleLocationsEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetSampleLocationsEXT-pSampleLocationsInfo-parameter
  `pSampleLocationsInfo` **must** be a valid pointer to a valid VkSampleLocationsInfoEXT structure
- VUID-vkCmdSetSampleLocationsEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetSampleLocationsEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetSampleLocationsEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

26.6. Fragment Shading Rates

The features advertised by VkPhysicalDeviceFragmentShadingRateFeaturesKHR allow an application to control the shading rate of a given fragment shader invocation.

The fragment shading rate strongly interacts with Multisampling, and the set of available rates for an implementation **may** be restricted by sample rate.

To query available shading rates, call:

```
// Provided by VK_KHR_fragment_shading_rate
VkResult vkGetPhysicalDeviceFragmentShadingRatesKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pFragmentShadingRateCount,
    VkPhysicalDeviceFragmentShadingRateKHR*     pFragmentShadingRates);
```

- `physicalDevice` is the handle to the physical device whose properties will be queried.
- `pFragmentShadingRateCount` is a pointer to an integer related to the number of fragment shading rates available or queried, as described below.
- `pFragmentShadingRates` is either `NULL` or a pointer to an array of VkPhysicalDeviceFragmentShadingRateKHR structures.

If `pFragmentShadingRates` is `NULL`, then the number of fragment shading rates available is returned in `pFragmentShadingRateCount`. Otherwise, `pFragmentShadingRateCount` **must** point to a variable set by the application to the number of elements in the `pFragmentShadingRates` array, and on return the variable is overwritten with the number of structures actually written to `pFragmentShadingRates`. If `pFragmentShadingRateCount` is less than the number of fragment shading rates available, at most `pFragmentShadingRateCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available fragment shading rates were returned.

The returned array of fragment shading rates **must** be ordered from largest `fragmentSize.width` value to smallest, and each set of fragment shading rates with the same `fragmentSize.width` value **must** be ordered from largest `fragmentSize.height` to smallest. Any two entries in the array **must** not have the same `fragmentSize` values.

For any entry in the array, the following rules also apply:

- The value of `fragmentSize.width` **must** be less than or equal to `maxFragmentSize.width`.
- The value of `fragmentSize.width` **must** be greater than or equal to `1`.
- The value of `fragmentSize.width` **must** be a power-of-two.
- The value of `fragmentSize.height` **must** be less than or equal to `maxFragmentSize.height`.
- The value of `fragmentSize.height` **must** be greater than or equal to `1`.
- The value of `fragmentSize.height` **must** be a power-of-two.
- The highest sample count in `sampleCounts` **must** be less than or equal to `maxFragmentShadingRateRasterizationSamples`.

- The product of `fragmentSize.width`, `fragmentSize.height`, and the highest sample count in `sampleCounts` **must** be less than or equal to `maxFragmentShadingRateCoverageSamples`.

Implementations **must** support at least the following shading rates:

`sampleCounts`	`fragmentSize`
`VK_SAMPLE_COUNT_1_BIT` | `VK_SAMPLE_COUNT_4_BIT`	{2,2}
`VK_SAMPLE_COUNT_1_BIT` | `VK_SAMPLE_COUNT_4_BIT`	{2,1}
~0	{1,1}

If `framebufferColorSampleCounts`, includes `VK_SAMPLE_COUNT_2_BIT`, the required rates **must** also include `VK_SAMPLE_COUNT_2_BIT`.

NOTE Including the {1,1} fragment size is done for completeness; it has no actual effect on the support of rendering without setting the fragment size. All sample counts are supported for this rate.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceFragmentShadingRatesKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceFragmentShadingRatesKHR-pFragmentShadingRateCount-parameter
  `pFragmentShadingRateCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceFragmentShadingRatesKHR-pFragmentShadingRates-parameter
  If the value referenced by `pFragmentShadingRateCount` is not `0`, and `pFragmentShadingRates` is not `NULL`, `pFragmentShadingRates` **must** be a valid pointer to an array of `pFragmentShadingRateCount` VkPhysicalDeviceFragmentShadingRateKHR structures

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkPhysicalDeviceFragmentShadingRateKHR` structure is defined as

```
// Provided by VK_KHR_fragment_shading_rate
typedef struct VkPhysicalDeviceFragmentShadingRateKHR {
    VkStructureType       sType;
    void*                 pNext;
    VkSampleCountFlags    sampleCounts;
    VkExtent2D            fragmentSize;
} VkPhysicalDeviceFragmentShadingRateKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `sampleCounts` is a bitmask of sample counts for which the shading rate described by `fragmentSize` is supported.
- `fragmentSize` is a VkExtent2D describing the width and height of a supported shading rate.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentShadingRateKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_KHR`
- VUID-VkPhysicalDeviceFragmentShadingRateKHR-pNext-pNext
  `pNext` **must** be `NULL`

Fragment shading rates **can** be set at three points, with the three rates combined to determine the final shading rate.

26.6.1. Pipeline Fragment Shading Rate

The *pipeline fragment shading rate* **can** be set on a per-draw basis by either setting the rate in a graphics pipeline, or dynamically via vkCmdSetFragmentShadingRateKHR.

The `VkPipelineFragmentShadingRateStateCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_fragment_shading_rate
typedef struct VkPipelineFragmentShadingRateStateCreateInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExtent2D                            fragmentSize;
    VkFragmentShadingRateCombinerOpKHR    combinerOps[2];
} VkPipelineFragmentShadingRateStateCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fragmentSize` specifies a VkExtent2D structure containing the fragment size used to define the pipeline fragment shading rate for drawing commands using this pipeline.

- `combinerOps` specifies a VkFragmentShadingRateCombinerOpKHR value determining how the pipeline, primitive, and attachment shading rates are combined for fragments generated by drawing commands using the created pipeline.

If the `pNext` chain of VkGraphicsPipelineCreateInfo includes a `VkPipelineFragmentShadingRateStateCreateInfoKHR` structure, then that structure includes parameters controlling the pipeline fragment shading rate.

If this structure is not present, `fragmentSize` is considered to be equal to (1,1), and both elements of `combinerOps` are considered to be equal to `VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR`.

Valid Usage (Implicit)

- VUID-VkPipelineFragmentShadingRateStateCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_FRAGMENT_SHADING_RATE_STATE_CREATE_INFO_KHR`

To dynamically set the pipeline fragment shading rate and combiner operation, call:

```
// Provided by VK_KHR_fragment_shading_rate
void vkCmdSetFragmentShadingRateKHR(
    VkCommandBuffer                             commandBuffer,
    const VkExtent2D*                           pFragmentSize,
    const VkFragmentShadingRateCombinerOpKHR    combinerOps[2]);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pFragmentSize` specifies the pipeline fragment shading rate for subsequent drawing commands.
- `combinerOps` specifies a VkFragmentShadingRateCombinerOpKHR determining how the pipeline, primitive, and attachment shading rates are combined for fragments generated by subsequent drawing commands.

This command sets the pipeline fragment shading rate and combiner operation for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR` set in VkPipelineDynamicStateCreateInfo ::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineFragmentShadingRateStateCreateInfoKHR values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetFragmentShadingRateKHR-pipelineFragmentShadingRate-04507
  If the `pipelineFragmentShadingRate` feature is not enabled, `pFragmentSize->width` **must** be `1`
- VUID-vkCmdSetFragmentShadingRateKHR-pipelineFragmentShadingRate-04508
  If the `pipelineFragmentShadingRate` feature is not enabled, `pFragmentSize->height` **must** be `1`
- VUID-vkCmdSetFragmentShadingRateKHR-pipelineFragmentShadingRate-04509
  One of the `pipelineFragmentShadingRate`, `primitiveFragmentShadingRate`, or

attachmentFragmentShadingRate features **must** be enabled

- VUID-vkCmdSetFragmentShadingRateKHR-primitiveFragmentShadingRate-04510
  If the primitiveFragmentShadingRate feature is not enabled, combinerOps[0] **must** be VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR
- VUID-vkCmdSetFragmentShadingRateKHR-attachmentFragmentShadingRate-04511
  If the attachmentFragmentShadingRate feature is not enabled, combinerOps[1] **must** be VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR
- VUID-vkCmdSetFragmentShadingRateKHR-fragmentSizeNonTrivialCombinerOps-04512
  If the fragmentSizeNonTrivialCombinerOps limit is not supported, elements of combinerOps **must** be either VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR or VK_FRAGMENT_SHADING_RATE_COMBINER_OP_REPLACE_KHR
- VUID-vkCmdSetFragmentShadingRateKHR-pFragmentSize-04513
  pFragmentSize->width **must** be greater than or equal to 1
- VUID-vkCmdSetFragmentShadingRateKHR-pFragmentSize-04514
  pFragmentSize->height **must** be greater than or equal to 1
- VUID-vkCmdSetFragmentShadingRateKHR-pFragmentSize-04515
  pFragmentSize->width **must** be a power-of-two value
- VUID-vkCmdSetFragmentShadingRateKHR-pFragmentSize-04516
  pFragmentSize->height **must** be a power-of-two value
- VUID-vkCmdSetFragmentShadingRateKHR-pFragmentSize-04517
  pFragmentSize->width **must** be less than or equal to 4
- VUID-vkCmdSetFragmentShadingRateKHR-pFragmentSize-04518
  pFragmentSize->height **must** be less than or equal to 4

Valid Usage (Implicit)

- VUID-vkCmdSetFragmentShadingRateKHR-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetFragmentShadingRateKHR-pFragmentSize-parameter
  pFragmentSize **must** be a valid pointer to a valid VkExtent2D structure
- VUID-vkCmdSetFragmentShadingRateKHR-combinerOps-parameter
  Each element of combinerOps **must** be a valid VkFragmentShadingRateCombinerOpKHR value
- VUID-vkCmdSetFragmentShadingRateKHR-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetFragmentShadingRateKHR-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetFragmentShadingRateKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

26.6.2. Primitive Fragment Shading Rate

The *primitive fragment shading rate* **can** be set via the `PrimitiveShadingRateKHR` built-in in the last active pre-rasterization shader stage. The rate associated with a given primitive is sourced from the value written to `PrimitiveShadingRateKHR` by that primitive's provoking vertex.

26.6.3. Attachment Fragment Shading Rate

The *attachment shading rate* **can** be set by including VkFragmentShadingRateAttachmentInfoKHR in a subpass to define a *fragment shading rate attachment*. Each pixel in the framebuffer is assigned an attachment fragment shading rate by the corresponding texel in the fragment shading rate attachment, according to:

$$x' = \text{floor}(x / \text{region}_x)$$

$$y' = \text{floor}(y / \text{region}_y)$$

where x' and y' are the coordinates of a texel in the fragment shading rate attachment, x and y are the coordinates of the pixel in the framebuffer, and region_x and region_y are the size of the region each texel corresponds to, as defined by the `shadingRateAttachmentTexelSize` member of VkFragmentShadingRateAttachmentInfoKHR.

If multiview is enabled and the shading rate attachment has multiple layers, the shading rate attachment texel is selected using layer = `ViewIndex`. If multiview is disabled, and both the shading rate attachment and the framebuffer have multiple layers, the shading rate attachment texel is selected using layer = `Layer`. Otherwise, layer = 0.

The texel is read from the fragment shading rate attachment image as a texture input operation without a sampler, using integer coordinates i = x', j = y', k = 0, l = layer, and s = 0. The fragment size is encoded into the first component of the result of that operation as follows:

$$size_w = 2^{((texel/4)\&3)}$$

$$size_h = 2^{(texel\&3)}$$

where texel is the value in the first component of the returned value, and $size_w$ and $size_h$ are the width and height of the fragment size, decoded from the texel.

If no fragment shading rate attachment is specified, this size is calculated as $size_w = size_h = 1$. Applications **must** not specify a width or height greater than 4 by this method.

The *Fragment Shading Rate* enumeration in SPIR-V adheres to the above encoding.

26.6.4. Combining the Fragment Shading Rates

The final rate (C_{xy}') used for fragment shading **must** be one of the rates returned by vkGetPhysicalDeviceFragmentShadingRatesKHR for the sample count used by rasterization.

If any of the following conditions are met, C_{xy}' is set to {1,1} by the implementation:

- If Sample Shading is enabled.
- The `fragmentShadingRateWithSampleMask` limit is not supported, and VkPipelineMultisampleStateCreateInfo::`pSampleMask` contains a zero value in any bit used by fragment operations.
- The `fragmentShadingRateWithShaderSampleMask` is not supported, and the fragment shader has `SampleMask` in the input or output interface.
- The `fragmentShadingRateWithShaderDepthStencilWrites` limit is not supported, and the fragment shader declares the `FragDepth` or `FragStencilRefEXT` built-in.
- The `fragmentShadingRateWithConservativeRasterization` limit is not supported, and VkPipelineRasterizationConservativeStateCreateInfoEXT::`conservativeRasterizationMode` is not `VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT`.
- The `fragmentShadingRateWithFragmentShaderInterlock` limit is not supported, and the fragment shader declares any of the fragment shader interlock execution modes.
- The `fragmentShadingRateWithCustomSampleLocations` limit is not supported, and VkPipelineSampleLocationsStateCreateInfoEXT::`sampleLocationsEnable` is `VK_TRUE`.
- The fragment shader declares any of the `TileImageColorReadAccessEXT`, `TileImageDepthReadAccessEXT`, or `TileImageStencilReadAccessEXT` capabilities.

Otherwise, each of the specified shading rates are combined and then used to derive the value of C_{xy}'. As there are three ways to specify shading rates, two combiner operations are specified - between the pipeline and primitive shading rates, and between the result of that and the attachment shading rate.

The equation used for each combiner operation is defined by `VkFragmentShadingRateCombinerOpKHR`:

```
// Provided by VK_KHR_fragment_shading_rate
typedef enum VkFragmentShadingRateCombinerOpKHR {
    VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR = 0,
    VK_FRAGMENT_SHADING_RATE_COMBINER_OP_REPLACE_KHR = 1,
    VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MIN_KHR = 2,
    VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MAX_KHR = 3,
    VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MUL_KHR = 4,
} VkFragmentShadingRateCombinerOpKHR;
```

- VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR specifies a combiner operation of combine(A_{xy},B_{xy}) = A_{xy}.
- VK_FRAGMENT_SHADING_RATE_COMBINER_OP_REPLACE_KHR specifies a combiner operation of combine(A_{xy},B_{xy}) = B_{xy}.
- VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MIN_KHR specifies a combiner operation of combine(A_{xy},B_{xy}) = min(A_{xy},B_{xy}).
- VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MAX_KHR specifies a combiner operation of combine(A_{xy},B_{xy}) = max(A_{xy},B_{xy}).
- VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MUL_KHR specifies a combiner operation of combine(A_{xy},B_{xy}) = A_{xy}*B_{xy}.

where combine(A_{xy},B_{xy}) is the combine operation, and A_{xy} and B_{xy} are the inputs to the operation.

If fragmentShadingRateStrictMultiplyCombiner is VK_FALSE, using VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MUL_KHR with values of 1 for both A and B in the same dimension results in the value 2 being produced for that dimension. See the definition of fragmentShadingRateStrictMultiplyCombiner for more information.

These operations are performed in a component-wise fashion.

This is used to generate a combined fragment area using the equation:

$$C_{xy} = \text{combine}(A_{xy},B_{xy})$$

where C_{xy} is the combined fragment area result, and A_{xy} and B_{xy} are the fragment areas of the fragment shading rates being combined.

Two combine operations are performed, first with A_{xy} equal to the pipeline fragment shading rate and B_{xy} equal to the primitive fragment shading rate, with the combine() operation selected by combinerOps[0]. A second combination is then performed, with A_{xy} equal to the result of the first combination and B_{xy} equal to the attachment fragment shading rate, with the combine() operation selected by combinerOps[1]. The result of the second combination is used as the final fragment shading rate, reported via the ShadingRateKHR built-in.

Implementations **should** clamp the inputs to the combiner operations A_{xy} and B_{xy}, and **must** do so if VkPhysicalDeviceMaintenance6PropertiesKHR::fragmentShadingRateClampCombinerInputs is VK_TRUE. All implementations **must** clamp the result of the second combiner operation.

A fragment shading rate R_{xy} representing any of A_{xy}, B_{xy} or C_{xy} is clamped as follows. If R_{xy} is one of the rates returned by vkGetPhysicalDeviceFragmentShadingRatesKHR for the sample count used by rasterization, the clamped shading rate R_{xy}' is R_{xy}. Otherwise, the clamped shading rate is selected from the rates returned by vkGetPhysicalDeviceFragmentShadingRatesKHR for the sample count used by rasterization. From this list of supported rates, the following steps are applied in order, to select a single value:

1. Keep only rates where $R_x' \leq R_x$ and $R_y' \leq R_y$.
   - Implementations **may** also keep rates where $R_x' \leq R_y$ and $R_y' \leq R_x$.
2. Keep only rates with the highest area ($R_x' \times R_y'$).
3. Keep only rates with the lowest aspect ratio ($R_x' + R_y'$).
4. In cases where a wide (e.g. 4x1) and tall (e.g. 1x4) rate remain, the implementation **may** choose either rate. However, it **must** choose this rate consistently for the same shading rates, and combiner operations for the lifetime of the VkDevice.

26.7. Sample Shading

Sample shading **can** be used to specify a minimum number of unique samples to process for each fragment. If sample shading is enabled, an implementation **must** invoke the fragment shader at least max(⌈ VkPipelineMultisampleStateCreateInfo::`minSampleShading` × VkPipelineMultisampleStateCreateInfo::`rasterizationSamples` ⌉, 1) times per fragment. If VkPipelineMultisampleStateCreateInfo::`sampleShadingEnable` is `VK_TRUE`, sample shading is enabled.

If a fragment shader entry point statically uses an input variable decorated with a `BuiltIn` of `SampleId` or `SamplePosition`, sample shading is enabled and a value of `1.0` is used instead of `minSampleShading`. If a fragment shader entry point statically uses an input variable decorated with `Sample`, sample shading **may** be enabled and a value of `1.0` will be used instead of `minSampleShading` if it is.

NOTE

If a shader decorates an input variable with `Sample` and that value meaningfully impacts the output of a shader, sample shading will be enabled to ensure that the input is in fact interpolated per-sample. This is inherent to the specification and not spelled out here - if an application simply declares such a variable it is implementation-defined whether sample shading is enabled or not. It is possible to see the effects of this by using atomics in the shader or using a pipeline statistics query to query the number of fragment invocations, even if the shader itself does not use any per-sample variables.

If there are fewer fragment invocations than covered samples, implementations **may** include those samples in fragment shader invocations in any manner as long as covered samples are all shaded at least once, and each invocation that is not a helper invocation covers at least one sample.

26.8. Barycentric Interpolation

When the `fragmentShaderBarycentric` feature is enabled, the `PerVertexKHR` interpolation decoration **can** be used with fragment shader inputs to indicate that the decorated inputs do not have

associated data in the fragment. Such inputs **can** only be accessed in a fragment shader using an array index whose value (0, 1, or 2) identifies one of the vertices of the primitive that produced the fragment. Reads of per-vertex values for missing vertices, such as the third vertex of a line primitive, will return values from the valid vertex with the highest index. This means that the per-vertex values of indices 1 and 2 for point primitives will be equal to those of index 0, and the per-vertex values of index 2 for line primitives will be equal to those of index 1.

When tessellation and geometry shading are not active, fragment shader inputs decorated with `PerVertexKHR` will take values from one of the vertices of the primitive that produced the fragment, identified by the extra index provided in SPIR-V code accessing the input. If the *n* vertices passed to a draw call are numbered 0 through *n*-1, and the point, line, and triangle primitives produced by the draw call are numbered with consecutive integers beginning with zero, the following table indicates the original vertex numbers used when the provoking vertex mode is `VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT` for index values of 0, 1, and 2. If an input decorated with `PerVertexKHR` is accessed with any other vertex index value, or is accessed while rasterizing a polygon when the VkPipelineRasterizationStateCreateInfo::`polygonMode` property of the currently active pipeline is not `VK_POLYGON_MODE_FILL`, an undefined value is returned.

Primitive Topology	Vertex 0	Vertex 1	Vertex 2
`VK_PRIMITIVE_TOPOLOGY_POINT_LIST`	i	i	i
`VK_PRIMITIVE_TOPOLOGY_LINE_LIST`	2i	2i+1	2i+1
`VK_PRIMITIVE_TOPOLOGY_LINE_STRIP`	i	i+1	i+1
`VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST`	3i	3i+1	3i+2
`VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP` (even)	i	i+1	i+2
`VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP` (odd)	i	i+2	i+1
`VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN`	i+1	i+2	0
`VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY`	4i+1	4i+2	4i+2
`VK_PRIMITIVE_TOPOLOGY_LINE_STRIP_WITH_ADJACENCY`	i+1	i+2	i+2
`VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY`	6i	6i+2	6i+4
`VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY` (even)	2i	2i+2	2i+4

Primitive Topology	Vertex 0	Vertex 1	Vertex 2
VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY (odd)	2i	2i+4	2i+2

When the provoking vertex mode is VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT, the original vertex numbers used are the same as above except as indicated in the table below.

Primitive Topology	Vertex 0	Vertex 1	Vertex 2
VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP (odd, and triStripVertexOrderIndependentOfProvokingVertex of VkPhysicalDeviceFragmentShaderBarycentricPropertiesKHR is VK_FALSE)	i+1	i	i+2
VK_PRIMITIVE_TOPOLOGY_TRIANGLE_FAN	0	i+1	i+2
VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP_WITH_ADJACENCY (odd)	2i+2	2i	2i+4

When geometry shading is active, primitives processed by fragment shaders are assembled from the vertices emitted by the geometry shader. In this case, the vertices used for fragment shader inputs decorated with PerVertexKHR are derived by treating the primitives produced by the shader as though they were specified by a draw call and consulting the table above.

When using tessellation without geometry shading, the tessellator produces primitives in an implementation-dependent manner. While there is no defined vertex ordering for inputs decorated with PerVertexKHR, the vertex ordering used in this case will be consistent with the ordering used to derive the values of inputs decorated with BaryCoordKHR or BaryCoordNoPerspKHR.

Fragment shader inputs decorated with BaryCoordKHR or BaryCoordNoPerspKHR hold three-component vectors with barycentric weights that indicate the location of the fragment relative to the screen-space locations of vertices of its primitive. For point primitives, such variables are always assigned the value (1,0,0). For line primitives, the built-ins are obtained by interpolating an attribute whose values for the vertices numbered 0 and 1 are (1,0,0) and (0,1,0), respectively. For polygon primitives, the built-ins are obtained by interpolating an attribute whose values for the vertices numbered 0, 1, and 2 are (1,0,0), (0,1,0), and (0,0,1), respectively. For BaryCoordKHR, the values are obtained using perspective interpolation. For BaryCoordNoPerspKHR, the values are obtained using linear interpolation. The values of BaryCoordKHR and BaryCoordNoPerspKHR are undefined while rasterizing a polygon when the VkPipelineRasterizationStateCreateInfo::polygonMode property of the currently active pipeline is not VK_POLYGON_MODE_FILL.

26.9. Points

A point is drawn by generating a set of fragments in the shape of a square centered around the vertex of the point. Each vertex has an associated point size controlling the width/height of that square. The point size is taken from the (potentially clipped) shader built-in `PointSize` written by:

- the geometry shader, if active;
- the tessellation evaluation shader, if active and no geometry shader is active;
- the vertex shader, otherwise

and clamped to the implementation-dependent point size range [`pointSizeRange`[0], `pointSizeRange`[1]]. The value written to `PointSize` **must** be greater than zero. If the `maintenance5` feature is enabled, and a value is not written to `PointSize`, the point size takes a default value of 1.0.

Not all point sizes need be supported, but the size 1.0 **must** be supported. The range of supported sizes and the size of evenly-spaced gradations within that range are implementation-dependent. The range and gradations are obtained from the `pointSizeRange` and `pointSizeGranularity` members of VkPhysicalDeviceLimits. If, for instance, the size range is from 0.1 to 2.0 and the gradation size is 0.1, then the sizes 0.1, 0.2, ..., 1.9, 2.0 are supported. Additional point sizes **may** also be supported. There is no requirement that these sizes be equally spaced. If an unsupported size is requested, the nearest supported size is used instead.

Further, if the render pass has a fragment density map attachment, point size **may** be rounded by the implementation to a multiple of the fragment's width or height.

26.9.1. Basic Point Rasterization

Point rasterization produces a fragment for each fragment area group of framebuffer pixels with one or more sample points that intersect a region centered at the point's (x_f,y_f). This region is a square with side equal to the current point size. Coverage bits that correspond to sample points that intersect the region are 1, other coverage bits are 0. All fragments produced in rasterizing a point are assigned the same associated data, which are those of the vertex corresponding to the point. However, the fragment shader built-in `PointCoord` contains point sprite texture coordinates. The s and t point sprite texture coordinates vary from zero to one across the point horizontally left-to-right and vertically top-to-bottom, respectively. The following formulas are used to evaluate s and t:

$$s = \frac{1}{2} + \frac{\left(x_p - x_f\right)}{\text{size}}$$

$$t = \frac{1}{2} + \frac{\left(y_p - y_f\right)}{\text{size}}$$

where size is the point's size; (x_p,y_p) is the location at which the point sprite coordinates are evaluated - this **may** be the framebuffer coordinates of the fragment center, or the location of a sample; and (x_f,y_f) is the exact, unrounded framebuffer coordinate of the vertex for the point.

26.10. Line Segments

Line segment rasterization options are controlled by the VkPipelineRasterizationLineStateCreateInfoKHR structure.

The VkPipelineRasterizationLineStateCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_line_rasterization
typedef struct VkPipelineRasterizationLineStateCreateInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkLineRasterizationModeKHR      lineRasterizationMode;
    VkBool32                        stippledLineEnable;
    uint32_t                        lineStippleFactor;
    uint16_t                        lineStipplePattern;
} VkPipelineRasterizationLineStateCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- lineRasterizationMode is a VkLineRasterizationModeKHR value selecting the style of line rasterization.
- stippledLineEnable enables stippled line rasterization.
- lineStippleFactor is the repeat factor used in stippled line rasterization.
- lineStipplePattern is the bit pattern used in stippled line rasterization.

If stippledLineEnable is VK_FALSE, the values of lineStippleFactor and lineStipplePattern are ignored.

Valid Usage

- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-lineRasterizationMode-02768
  If lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the rectangularLines feature **must** be enabled
- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-lineRasterizationMode-02769
  If lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the bresenhamLines feature **must** be enabled
- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-lineRasterizationMode-02770
  If lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the smoothLines feature **must** be enabled
- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-stippledLineEnable-02771
  If stippledLineEnable is VK_TRUE and lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled
- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-stippledLineEnable-02772

If stippledLineEnable is VK_TRUE and lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled

- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-stippledLineEnable-02773
  If stippledLineEnable is VK_TRUE and lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-stippledLineEnable-02774
  If stippledLineEnable is VK_TRUE and lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE

Valid Usage (Implicit)

- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_LINE_STATE_CREATE_INFO_KHR
- VUID-VkPipelineRasterizationLineStateCreateInfoKHR-lineRasterizationMode-parameter
  lineRasterizationMode **must** be a valid VkLineRasterizationModeKHR value

Possible values of VkPipelineRasterizationLineStateCreateInfoKHR::lineRasterizationMode are:

```
// Provided by VK_KHR_line_rasterization
typedef enum VkLineRasterizationModeKHR {
    VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR = 0,
    VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR = 1,
    VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR = 2,
    VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR = 3,
    VK_LINE_RASTERIZATION_MODE_DEFAULT_EXT = VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR,
    VK_LINE_RASTERIZATION_MODE_RECTANGULAR_EXT =
VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR,
    VK_LINE_RASTERIZATION_MODE_BRESENHAM_EXT =
VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR,
    VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_EXT =
VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR,
} VkLineRasterizationModeKHR;
```

- VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR is equivalent to VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR if VkPhysicalDeviceLimits::strictLines is VK_TRUE, otherwise lines are drawn as non-strictLines parallelograms. Both of these modes are defined in Basic Line Segment Rasterization.
- VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR specifies lines drawn as if they were rectangles extruded from the line
- VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR specifies lines drawn by determining which pixel diamonds the line intersects and exits, as defined in Bresenham Line Segment Rasterization.

- VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR specifies lines drawn if they were rectangles extruded from the line, with alpha falloff, as defined in Smooth Lines.

To dynamically set the lineRasterizationMode state, call:

```
// Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_line_rasterization,
VK_EXT_line_rasterization with VK_EXT_shader_object
void vkCmdSetLineRasterizationModeEXT(
    VkCommandBuffer                             commandBuffer,
    VkLineRasterizationModeEXT                  lineRasterizationMode);
```

- commandBuffer is the command buffer into which the command will be recorded.
- lineRasterizationMode specifies the lineRasterizationMode state.

This command sets the lineRasterizationMode state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT set in VkPipelineDynamicStateCreateInfo ::pDynamicStates. Otherwise, this state is specified by the VkPipelineRasterizationLineStateCreateInfoKHR::lineRasterizationMode value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetLineRasterizationModeEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3LineRasterizationMode feature is enabled
  - The shaderObject feature is enabled
- VUID-vkCmdSetLineRasterizationModeEXT-lineRasterizationMode-07418
  If lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the rectangularLines feature **must** be enabled
- VUID-vkCmdSetLineRasterizationModeEXT-lineRasterizationMode-07419
  If lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the bresenhamLines feature **must** be enabled
- VUID-vkCmdSetLineRasterizationModeEXT-lineRasterizationMode-07420
  If lineRasterizationMode is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the smoothLines feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCmdSetLineRasterizationModeEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetLineRasterizationModeEXT-lineRasterizationMode-parameter
  lineRasterizationMode **must** be a valid VkLineRasterizationModeEXT value

- VUID-vkCmdSetLineRasterizationModeEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetLineRasterizationModeEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetLineRasterizationModeEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the `stippledLineEnable` state, call:

```
// Provided by VK_EXT_extended_dynamic_state3 with VK_EXT_line_rasterization,
VK_EXT_line_rasterization with VK_EXT_shader_object
void vkCmdSetLineStippleEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    stippledLineEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `stippledLineEnable` specifies the `stippledLineEnable` state.

This command sets the `stippledLineEnable` state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineRasterizationLineStateCreateInfoKHR::`stippledLineEnable` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetLineStippleEnableEXT-None-09423

At least one of the following **must** be true:

- The `extendedDynamicState3LineStippleEnable` feature is enabled
- The `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetLineStippleEnableEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetLineStippleEnableEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetLineStippleEnableEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetLineStippleEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the line width, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetLineWidth(
    VkCommandBuffer                             commandBuffer,
    float                                       lineWidth);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `lineWidth` is the width of rasterized line segments.

This command sets the line width for subsequent drawing commands when drawing using shader

objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_LINE_WIDTH set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineRasterizationStateCreateInfo::lineWidth value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetLineWidth-lineWidth-00788
  If the wideLines feature is not enabled, lineWidth **must** be 1.0

Valid Usage (Implicit)

- VUID-vkCmdSetLineWidth-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetLineWidth-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetLineWidth-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetLineWidth-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Not all line widths need be supported for line segment rasterization, but width 1.0 antialiased segments **must** be provided. The range and gradations are obtained from the lineWidthRange and lineWidthGranularity members of VkPhysicalDeviceLimits. If, for instance, the size range is from 0.1 to 2.0 and the gradation size is 0.1, then the sizes 0.1, 0.2, ..., 1.9, 2.0 are supported. Additional line widths **may** also be supported. There is no requirement that these widths be equally spaced. If an unsupported width is requested, the nearest supported width is used instead.

Further, if the render pass has a fragment density map attachment, line width **may** be rounded by the implementation to a multiple of the fragment's width or height.

26.10.1. Basic Line Segment Rasterization

If the `lineRasterizationMode` member of VkPipelineRasterizationLineStateCreateInfoKHR is `VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR`, rasterized line segments produce fragments which intersect a rectangle centered on the line segment. Two of the edges are parallel to the specified line segment; each is at a distance of one-half the current width from that segment in directions perpendicular to the direction of the line. The other two edges pass through the line endpoints and are perpendicular to the direction of the specified line segment. Coverage bits that correspond to sample points that intersect the rectangle are 1, other coverage bits are 0.

Next we specify how the data associated with each rasterized fragment are obtained. Let $\mathbf{p}_r = (x_d, y_d)$ be the framebuffer coordinates at which associated data are evaluated. This **may** be the center of a fragment or the location of a sample within the fragment. When `rasterizationSamples` is `VK_SAMPLE_COUNT_1_BIT`, the fragment center **must** be used. Let $\mathbf{p}_a = (x_a, y_a)$ and $\mathbf{p}_b = (x_b, y_b)$ be initial and final endpoints of the line segment, respectively. Set

$$t = \frac{(\mathbf{p}_r - \mathbf{p}_a) \cdot (\mathbf{p}_b - \mathbf{p}_a)}{\| \mathbf{p}_b - \mathbf{p}_a \|^2}$$

(Note that t = 0 at $\mathbf{p}_a$ and t = 1 at $\mathbf{p}_b$. Also note that this calculation projects the vector from $\mathbf{p}_a$ to $\mathbf{p}_r$ onto the line, and thus computes the normalized distance of the fragment along the line.)

If `strictLines` is `VK_TRUE`, line segments are rasterized using perspective or linear interpolation.

Perspective interpolation for a line segment interpolates two values in a manner that is correct when taking the perspective of the viewport into consideration, by way of the line segment's clip coordinates. An interpolated value f can be determined by

$$f = \frac{(1-t) f_a / w_a + t f_b / w_b}{(1-t) / w_a + t / w_b}$$

where f_a and f_b are the data associated with the starting and ending endpoints of the segment, respectively; w_a and w_b are the clip w coordinates of the starting and ending endpoints of the segment, respectively.

Linear interpolation for a line segment directly interpolates two values, and an interpolated value f can be determined by

$$f = (1 - t) f_a + t f_b$$

where f_a and f_b are the data associated with the starting and ending endpoints of the segment, respectively.

The clip coordinate w for a sample is determined using perspective interpolation. The depth value z for a sample is determined using linear interpolation. Interpolation of fragment shader input values are determined by Interpolation decorations.

The above description documents the preferred method of line rasterization, and **must** be used when `lineRasterizationMode` is `VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR`.

By default, when `strictLines` is `VK_FALSE`, and when the `lineRasterizationMode` is `VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR`, the edges of the lines are generated as a parallelogram surrounding the original line. The major axis is chosen by noting the axis in which there is the greatest distance between the line start and end points. If the difference is equal in both directions then the X axis is chosen as the major axis. Edges 2 and 3 are aligned to the minor axis and are centered on the endpoints of the line as in Non strict lines, and each is `lineWidth` long. Edges 0 and 1 are parallel to the line and connect the endpoints of edges 2 and 3. Coverage bits that correspond to sample points that intersect the parallelogram are 1, other coverage bits are 0.

Samples that fall exactly on the edge of the parallelogram follow the polygon rasterization rules.

Interpolation occurs as if the parallelogram was decomposed into two triangles where each pair of vertices at each end of the line has identical attributes.

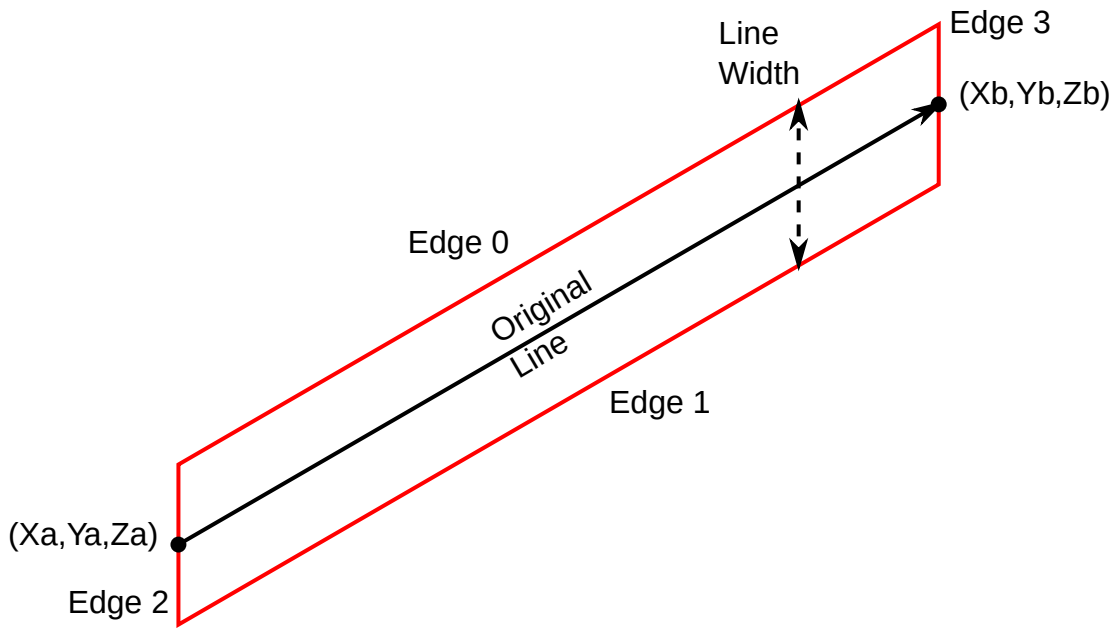


Figure 15. Non strict lines

Only when `strictLines` is `VK_FALSE` implementations **may** deviate from the non-strict line algorithm described above in the following ways:

- Implementations **may** instead interpolate each fragment according to the formula in Basic Line Segment Rasterization using the original line segment endpoints.
- Rasterization of non-antialiased non-strict line segments **may** be performed using the rules defined in Bresenham Line Segment Rasterization.

If `VkPhysicalDeviceMaintenance5PropertiesKHR`::`nonStrictSinglePixelWideLinesUseParallelogram` is `VK_TRUE`, and `strictLines` is `VK_FALSE`, non-strict lines of width 1.0 are rasterized as parallelograms, otherwise they are rasterized using Bresenham's algorithm.

If `VkPhysicalDeviceMaintenance5PropertiesKHR`::`nonStrictWideLinesUseParallelogram` is `VK_TRUE`, and `strictLines` is `VK_FALSE`, non-strict lines of width greater than 1.0 are rasterized as parallelograms, otherwise they are rasterized using Bresenham's algorithm.

26.10.2. Bresenham Line Segment Rasterization

If `lineRasterizationMode` is `VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR`, then the following rules replace the line rasterization rules defined in Basic Line Segment Rasterization.

Non-strict lines **may** also follow these rasterization rules for non-antialiased lines.

Line segment rasterization begins by characterizing the segment as either *x-major* or *y-major*. x-major line segments have slope in the closed interval [-1,1]; all other line segments are y-major (slope is determined by the segment's endpoints). We specify rasterization only for x-major segments except in cases where the modifications for y-major segments are not self-evident.

Ideally, Vulkan uses a *diamond-exit* rule to determine those fragments that are produced by rasterizing a line segment. For each fragment f with center at framebuffer coordinates x_f and y_f, define a diamond-shaped region that is the intersection of four half planes:

$$R_f = \{(x,\ y)\ |\ |x - x_f| + |y - y_f| < \frac{1}{2}\}$$

Essentially, a line segment starting at p_a and ending at p_b produces those fragments f for which the segment intersects R_f, except if p_b is contained in R_f.

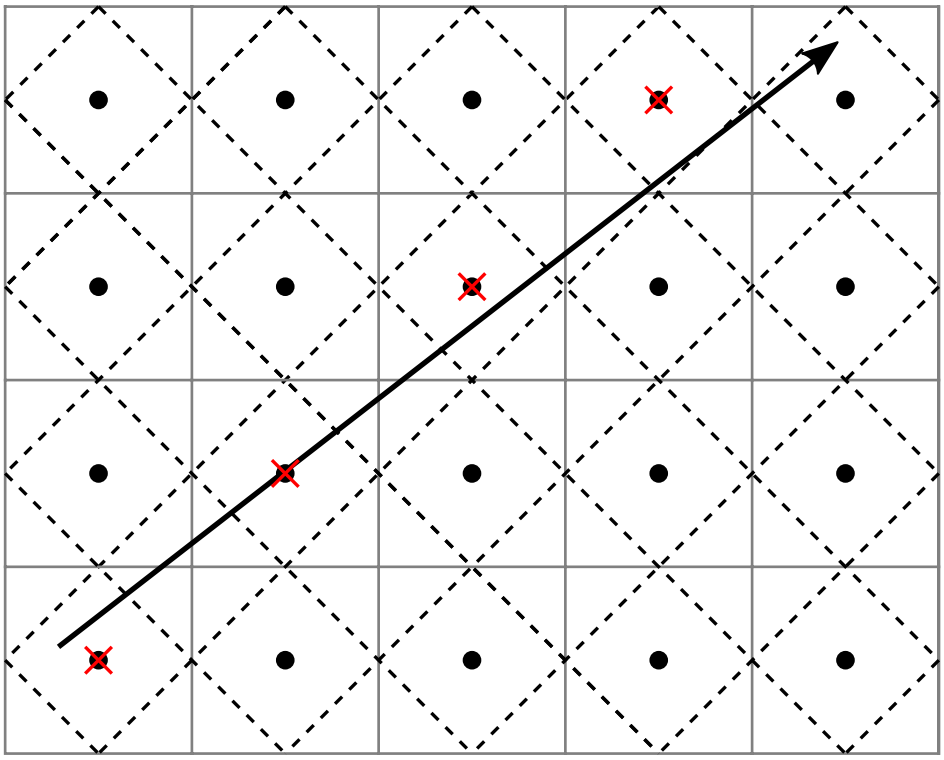


Figure 16. Visualization of Bresenham's algorithm

To avoid difficulties when an endpoint lies on a boundary of R_f we (in principle) perturb the supplied endpoints by a tiny amount. Let p_a and p_b have framebuffer coordinates (x_a, y_a) and (x_b, y_b), respectively. Obtain the perturbed endpoints p_a' given by $(x_a, y_a) - (\varepsilon, \varepsilon^2)$ and p_b' given by $(x_b, y_b) - (\varepsilon, \varepsilon^2)$. Rasterizing the line segment starting at p_a and ending at p_b produces those fragments f for which the segment starting at p_a' and ending on p_b' intersects R_f, except if p_b' is contained in R_f. ε is

chosen to be so small that rasterizing the line segment produces the same fragments when δ is substituted for ε for any $0 < \delta \leq \varepsilon$.

When p_a and p_b lie on fragment centers, this characterization of fragments reduces to Bresenham's algorithm with one modification: lines produced in this description are "half-open", meaning that the final fragment (corresponding to p_b) is not drawn. This means that when rasterizing a series of connected line segments, shared endpoints will be produced only once rather than twice (as would occur with Bresenham's algorithm).

Implementations **may** use other line segment rasterization algorithms, subject to the following rules:

- The coordinates of a fragment produced by the algorithm **must** not deviate by more than one unit in either x or y framebuffer coordinates from a corresponding fragment produced by the diamond-exit rule.
- The total number of fragments produced by the algorithm **must** not differ from that produced by the diamond-exit rule by more than one.
- For an x-major line, two fragments that lie in the same framebuffer-coordinate column **must** not be produced (for a y-major line, two fragments that lie in the same framebuffer-coordinate row **must** not be produced).
- If two line segments share a common endpoint, and both segments are either x-major (both left-to-right or both right-to-left) or y-major (both bottom-to-top or both top-to-bottom), then rasterizing both segments **must** not produce duplicate fragments. Fragments also **must** not be omitted so as to interrupt continuity of the connected segments.

The actual width w of Bresenham lines is determined by rounding the line width to the nearest integer, clamping it to the implementation-dependent `lineWidthRange` (with both values rounded to the nearest integer), then clamping it to be no less than 1.

Bresenham line segments of width other than one are rasterized by offsetting them in the minor direction (for an x-major line, the minor direction is y, and for a y-major line, the minor direction is x) and producing a row or column of fragments in the minor direction. If the line segment has endpoints given by (x_0, y_0) and (x_1, y_1) in framebuffer coordinates, the segment with endpoints $(x_0, y_0 - \frac{w-1}{2})$ and $(x_1, y_1 - \frac{w-1}{2})$ is rasterized, but instead of a single fragment, a column of fragments of height w (a row of fragments of length w for a y-major segment) is produced at each x (y for y-major) location. The lowest fragment of this column is the fragment that would be produced by rasterizing the segment of width 1 with the modified coordinates.

The preferred method of attribute interpolation for a wide line is to generate the same attribute values for all fragments in the row or column described above, as if the adjusted line was used for interpolation and those values replicated to the other fragments, except for `FragCoord` which is interpolated as usual. Implementations **may** instead interpolate each fragment according to the formula in Basic Line Segment Rasterization, using the original line segment endpoints.

When Bresenham lines are being rasterized, sample locations **may** all be treated as being at the pixel center (this **may** affect attribute and depth interpolation).

NOTE	The sample locations described above are **not** used for determining coverage, they

> are only used for things like attribute interpolation. The rasterization rules that determine coverage are defined in terms of whether the line intersects **pixels**, as opposed to the point sampling rules used for other primitive types. So these rules are independent of the sample locations. One consequence of this is that Bresenham lines cover the same pixels regardless of the number of rasterization samples, and cover all samples in those pixels (unless masked out or killed).

26.10.3. Line Stipple

If the `stippledLineEnable` member of VkPipelineRasterizationLineStateCreateInfoKHR is `VK_TRUE`, then lines are rasterized with a *line stipple* determined by `lineStippleFactor` and `lineStipplePattern`. `lineStipplePattern` is an unsigned 16-bit integer that determines which fragments are to be drawn or discarded when the line is rasterized. `lineStippleFactor` is a count that is used to modify the effective line stipple by causing each bit in `lineStipplePattern` to be used `lineStippleFactor` times.

Line stippling discards certain fragments that are produced by rasterization. The masking is achieved using three parameters: the 16-bit line stipple pattern p, the line stipple factor r, and an integer stipple counter s. Let

$$b = \lfloor \frac{s}{r} \rfloor \bmod 16$$

Then a fragment is produced if the b'th bit of p is 1, and discarded otherwise. The bits of p are numbered with 0 being the least significant and 15 being the most significant.

The initial value of s is zero. For `VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR` lines, s is incremented after production of each fragment of a line segment (fragments are produced in order, beginning at the starting point and working towards the ending point). For `VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR` and `VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR` lines, the rectangular region is subdivided into adjacent unit-length rectangles, and s is incremented once for each rectangle. Rectangles with a value of s such that the b'th bit of p is zero are discarded. If the last rectangle in a line segment is shorter than unit-length, then the remainder **may** carry over to the next line segment in the line strip using the same value of s (this is the preferred behavior, for the stipple pattern to appear more consistent through the strip).

s is reset to 0 at the start of each strip (for line strips), and before every line segment in a group of independent segments.

If the line segment has been clipped, then the value of s at the beginning of the line segment is implementation-dependent.

To dynamically set the line stipple state, call:

```
// Provided by VK_KHR_line_rasterization
void vkCmdSetLineStippleKHR(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    lineStippleFactor,
    uint16_t                                    lineStipplePattern);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `lineStippleFactor` is the repeat factor used in stippled line rasterization.
- `lineStipplePattern` is the bit pattern used in stippled line rasterization.

This command sets the line stipple state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_LINE_STIPPLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineRasterizationLineStateCreateInfoKHR::`lineStippleFactor` and VkPipelineRasterizationLineStateCreateInfoKHR::`lineStipplePattern` values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetLineStippleKHR-lineStippleFactor-02776
  `lineStippleFactor` **must** be in the range [1,256]

Valid Usage (Implicit)

- VUID-vkCmdSetLineStippleKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetLineStippleKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetLineStippleKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetLineStippleKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

26.10.4. Smooth Lines

If the `lineRasterizationMode` member of VkPipelineRasterizationLineStateCreateInfoKHR is `VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR`, then lines are considered to be rectangles using the same geometry as for `VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR` lines. The rules for determining which pixels are covered are implementation-dependent, and **may** include nearby pixels where no sample locations are covered or where the rectangle does not intersect the pixel at all. For each pixel that is considered covered, the fragment computes a coverage value that approximates the area of the intersection of the rectangle with the pixel square, and this coverage value is multiplied into the color location 0's alpha value after fragment shading, as described in Multisample Coverage.

NOTE The details of the rasterization rules and area calculation are left intentionally vague, to allow implementations to generate coverage and values that are aesthetically pleasing.

26.11. Polygons

A polygon results from the decomposition of a triangle strip, triangle fan or a series of independent triangles. Like points and line segments, polygon rasterization is controlled by several variables in the VkPipelineRasterizationStateCreateInfo structure.

26.11.1. Basic Polygon Rasterization

The first step of polygon rasterization is to determine whether the triangle is *back-facing* or *front-facing*. This determination is made based on the sign of the (clipped or unclipped) polygon's area computed in framebuffer coordinates. One way to compute this area is:

$$a = -\frac{1}{2}\sum_{i=0}^{n-1} x_f^i y_f^{i \oplus 1} - x_f^{i \oplus 1} y_f^i$$

where x_f^i and y_f^i are the x and y framebuffer coordinates of the ith vertex of the n-vertex polygon (vertices are numbered starting at zero for the purposes of this computation) and i ⊕ 1 is (i + 1) mod n.

The interpretation of the sign of a is determined by the VkPipelineRasterizationStateCreateInfo ::`frontFace` property of the currently active pipeline. Possible values are:

```
// Provided by VK_VERSION_1_0
typedef enum VkFrontFace {
    VK_FRONT_FACE_COUNTER_CLOCKWISE = 0,
    VK_FRONT_FACE_CLOCKWISE = 1,
} VkFrontFace;
```

- VK_FRONT_FACE_COUNTER_CLOCKWISE specifies that a triangle with positive area is considered front-facing.
- VK_FRONT_FACE_CLOCKWISE specifies that a triangle with negative area is considered front-facing.

Any triangle which is not front-facing is back-facing, including zero-area triangles.

To dynamically set the front face orientation, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetFrontFaceEXT(
    VkCommandBuffer                             commandBuffer,
    VkFrontFace                                 frontFace);
```

- commandBuffer is the command buffer into which the command will be recorded.
- frontFace is a VkFrontFace value specifying the front-facing triangle orientation to be used for culling.

This command sets the front face orientation for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineRasterizationStateCreateInfo::frontFace value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetFrontFace-None-08971
  At least one of the following **must** be true:
  - the shaderObject feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetFrontFace-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetFrontFace-frontFace-parameter
  frontFace **must** be a valid VkFrontFace value
- VUID-vkCmdSetFrontFace-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetFrontFace-commandBuffer-cmdpool

The VkCommandPool that commandBuffer was allocated from **must** support graphics operations

- VUID-vkCmdSetFrontFace-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Once the orientation of triangles is determined, they are culled according to the VkPipelineRasterizationStateCreateInfo::cullMode property of the currently active pipeline. Possible values are:

```
// Provided by VK_VERSION_1_0
typedef enum VkCullModeFlagBits {
    VK_CULL_MODE_NONE = 0,
    VK_CULL_MODE_FRONT_BIT = 0x00000001,
    VK_CULL_MODE_BACK_BIT = 0x00000002,
    VK_CULL_MODE_FRONT_AND_BACK = 0x00000003,
} VkCullModeFlagBits;
```

- VK_CULL_MODE_NONE specifies that no triangles are discarded
- VK_CULL_MODE_FRONT_BIT specifies that front-facing triangles are discarded
- VK_CULL_MODE_BACK_BIT specifies that back-facing triangles are discarded
- VK_CULL_MODE_FRONT_AND_BACK specifies that all triangles are discarded.

Following culling, fragments are produced for any triangles which have not been discarded.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkCullModeFlags;
```

VkCullModeFlags is a bitmask type for setting a mask of zero or more VkCullModeFlagBits.

To dynamically set the cull mode, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetCullModeEXT(
    VkCommandBuffer                             commandBuffer,
    VkCullModeFlags                             cullMode);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `cullMode` specifies the cull mode property to use for drawing.

This command sets the cull mode for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineRasterizationStateCreateInfo::`cullMode` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetCullMode-None-08971
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetCullMode-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetCullMode-cullMode-parameter
  `cullMode` **must** be a valid combination of VkCullModeFlagBits values
- VUID-vkCmdSetCullMode-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetCullMode-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetCullMode-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

The rule for determining which fragments are produced by polygon rasterization is called *point sampling*. The two-dimensional projection obtained by taking the x and y framebuffer coordinates of the polygon's vertices is formed. Fragments are produced for any fragment area groups of pixels for which any sample points lie inside of this polygon. Coverage bits that correspond to sample points that satisfy the point sampling criteria are 1, other coverage bits are 0. Special treatment is given to a sample whose sample location lies on a polygon edge. In such a case, if two polygons lie on either side of a common edge (with identical endpoints) on which a sample point lies, then exactly one of the polygons **must** result in a covered sample for that fragment during rasterization. As for the data associated with each fragment produced by rasterizing a polygon, we begin by specifying how these values are produced for fragments in a triangle.

Barycentric coordinates are a set of three numbers, a, b, and c, each in the range [0,1], with a + b + c = 1. These coordinates uniquely specify any point p within the triangle or on the triangle's boundary as

$$p = a\,p_a + b\,p_b + c\,p_c$$

where p_a, p_b, and p_c are the vertices of the triangle. a, b, and c are determined by:

$$a = \frac{\mathrm{A}(pp_bp_c)}{\mathrm{A}(p_ap_bp_c)}, \quad b = \frac{\mathrm{A}(pp_ap_c)}{\mathrm{A}(p_ap_bp_c)}, \quad c = \frac{\mathrm{A}(pp_ap_b)}{\mathrm{A}(p_ap_bp_c)},$$

where A(lmn) denotes the area in framebuffer coordinates of the triangle with vertices l, m, and n.

Denote an associated datum at p_a, p_b, or p_c as f_a, f_b, or f_c, respectively.

Perspective interpolation for a triangle interpolates three values in a manner that is correct when taking the perspective of the viewport into consideration, by way of the triangle's clip coordinates. An interpolated value f can be determined by

$$f = \frac{af_a / w_a + bf_b / w_b + cf_c / w_c}{a / w_a + b / w_b + c / w_c}$$

where w_a, w_b, and w_c are the clip w coordinates of p_a, p_b, and p_c, respectively. a, b, and c are the barycentric coordinates of the location at which the data are produced.

Linear interpolation for a triangle directly interpolates three values, and an interpolated value f can be determined by

$$f = a\,f_a + b\,f_b + c\,f_c$$

where f_a, f_b, and f_c are the data associated with p_a, p_b, and p_c, respectively.

The clip coordinate w for a sample is determined using perspective interpolation. The depth value z for a sample is determined using linear interpolation. Interpolation of fragment shader input values are determined by Interpolation decorations.

For a polygon with more than three edges, such as are produced by clipping a triangle, a convex combination of the values of the datum at the polygon's vertices **must** be used to obtain the value assigned to each fragment produced by the rasterization algorithm. That is, it **must** be the case that at every fragment

$$f = \sum_{i=1}^{n} a_i f_i$$

where n is the number of vertices in the polygon and f_i is the value of f at vertex i. For each i, $0 \le a_i \le 1$ and $\sum_{i=1}^{n} a_i = 1$. The values of a_i **may** differ from fragment to fragment, but at vertex i, $a_i = 1$ and $a_j = 0$ for $j \neq i$.

NOTE

One algorithm that achieves the required behavior is to triangulate a polygon (without adding any vertices) and then treat each triangle individually as already discussed. A scan-line rasterizer that linearly interpolates data along each edge and then linearly interpolates data across each horizontal span from edge to edge also satisfies the restrictions (in this case the numerator and denominator of perspective interpolation are iterated independently, and a division is performed for each fragment).

26.11.2. Polygon Mode

Possible values of the VkPipelineRasterizationStateCreateInfo::`polygonMode` property of the currently active pipeline, specifying the method of rasterization for polygons, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkPolygonMode {
    VK_POLYGON_MODE_FILL = 0,
    VK_POLYGON_MODE_LINE = 1,
    VK_POLYGON_MODE_POINT = 2,
} VkPolygonMode;
```

- `VK_POLYGON_MODE_POINT` specifies that polygon vertices are drawn as points.
- `VK_POLYGON_MODE_LINE` specifies that polygon edges are drawn as line segments.
- `VK_POLYGON_MODE_FILL` specifies that polygons are rendered using the polygon rasterization rules in this section.

These modes affect only the final rasterization of polygons: in particular, a polygon's vertices are

shaded and the polygon is clipped and possibly culled before these modes are applied.

If VkPhysicalDeviceMaintenance5PropertiesKHR::polygonModePointSize is VK_TRUE, the point size of the final rasterization of polygons is taken from PointSize when polygon mode is VK_POLYGON_MODE_POINT.

Otherwise, if VkPhysicalDeviceMaintenance5PropertiesKHR::polygonModePointSize is VK_FALSE, the point size of the final rasterization of polygons is 1.0 when polygon mode is VK_POLYGON_MODE_POINT.

To dynamically set the polygon mode, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetPolygonModeEXT(
    VkCommandBuffer                             commandBuffer,
    VkPolygonMode                               polygonMode);
```

- commandBuffer is the command buffer into which the command will be recorded.
- polygonMode specifies polygon mode.

This command sets the polygon mode for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_POLYGON_MODE_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineRasterizationStateCreateInfo::polygonMode value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetPolygonModeEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3PolygonMode feature is enabled
  - The shaderObject feature is enabled
- VUID-vkCmdSetPolygonModeEXT-fillModeNonSolid-07424
  If the fillModeNonSolid feature is not enabled, polygonMode **must** be VK_POLYGON_MODE_FILL

Valid Usage (Implicit)

- VUID-vkCmdSetPolygonModeEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetPolygonModeEXT-polygonMode-parameter
  polygonMode **must** be a valid VkPolygonMode value
- VUID-vkCmdSetPolygonModeEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetPolygonModeEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics

operations

- VUID-vkCmdSetPolygonModeEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

26.11.3. Depth Bias

The depth values of all fragments generated by the rasterization of a polygon **can** be biased (offset) by a single depth bias value o that is computed for that polygon.

Depth Bias Enable

The depth bias computation is enabled by the VkPipelineRasterizationStateCreateInfo ::`depthBiasEnable` value used to create the currently active pipeline. If the depth bias enable is `VK_FALSE`, no bias is applied and the fragment's depth values are unchanged.

To dynamically enable whether to bias fragment depth values, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetDepthBiasEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    depthBiasEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `depthBiasEnable` controls whether to bias fragment depth values.

This command sets the depth bias enable for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineRasterizationStateCreateInfo ::`depthBiasEnable` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDepthBiasEnable-None-08970
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetDepthBiasEnable-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthBiasEnable-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthBiasEnable-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthBiasEnable-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Depth Bias Computation

The depth bias depends on three parameters:

- `depthBiasSlopeFactor` scales the maximum depth slope m of the polygon
- `depthBiasConstantFactor` scales the parameter r of the depth attachment
- the scaled terms are summed to produce a value which is then clamped to a minimum or maximum value specified by `depthBiasClamp`

depthBiasSlopeFactor, depthBiasConstantFactor, and depthBiasClamp **can** each be positive, negative, or zero. These parameters are set as described for vkCmdSetDepthBias and vkCmdSetDepthBias2EXT below.

The maximum depth slope m of a triangle is

$$m = \sqrt{\left(\frac{\partial z_f}{\partial x_f}\right)^2 + \left(\frac{\partial z_f}{\partial y_f}\right)^2}$$

where (x_f, y_f, z_f) is a point on the triangle. m **may** be approximated as

$$m = \max\left(\left|\frac{\partial z_f}{\partial x_f}\right|, \left|\frac{\partial z_f}{\partial y_f}\right|\right).$$

In a pipeline with a depth bias representation of VK_DEPTH_BIAS_REPRESENTATION_FLOAT_EXT, r, for the given primitive is defined as

$$r = 1$$

Otherwise r is the minimum resolvable difference that depends on the depth attachment representation. If VkDepthBiasRepresentationInfoEXT::depthBiasExact is VK_FALSE it is the smallest difference in a sample's depth z_f values that is guaranteed to remain distinct throughout polygon rasterization and in the depth attachment. All pairs of fragments generated by the rasterization of two polygons with otherwise identical vertices, but z_f values that differ by r, will have distinct depth values.

For fixed-point depth attachment representations, or in a pipeline with a depth bias representation of VK_DEPTH_BIAS_REPRESENTATION_LEAST_REPRESENTABLE_VALUE_FORCE_UNORM_EXT, r is constant throughout the range of the entire depth attachment. If VkDepthBiasRepresentationInfoEXT::depthBiasExact is VK_TRUE, then its value **must** be

$$r = 2^{-n}$$

Otherwise its value is implementation-dependent but **must** be at most

$$r = 2 \times 2^{-n}$$

where n is the number of bits used for the depth aspect when using a fixed-point attachment, or the number of mantissa bits plus one when using a floating-point attachment.

Otherwise for floating-point depth attachment, there is no single minimum resolvable difference. In this case, the minimum resolvable difference for a given polygon is dependent on the maximum exponent, e, in the range of z values spanned by the primitive. If n is the number of bits in the floating-point mantissa, the minimum resolvable difference, r, for the given primitive is defined as

$r = 2^{e-n}$

If no depth attachment is present, r is undefined.

The bias value o for a polygon is

$$o = \text{dbclamp}(m \times depthBiasSlopeFactor + r \times depthBiasConstantFactor)$$

$$\text{where} \quad \text{dbclamp}(x) = \begin{cases} x & depthBiasClamp = 0 \text{ or } NaN \\ \min(x, depthBiasClamp) & depthBiasClamp > 0 \\ \max(x, depthBiasClamp) & depthBiasClamp < 0 \end{cases}$$

m is computed as described above. If the depth attachment uses a fixed-point representation, m is a function of depth values in the range [0,1], and o is applied to depth values in the same range.

Depth bias is applied to triangle topology primitives received by the rasterizer regardless of polygon mode. Depth bias **may** also be applied to line and point topology primitives received by the rasterizer.

To dynamically set the depth bias parameters, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetDepthBias(
    VkCommandBuffer                             commandBuffer,
    float                                       depthBiasConstantFactor,
    float                                       depthBiasClamp,
    float                                       depthBiasSlopeFactor);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `depthBiasConstantFactor` is a scalar factor controlling the constant depth value added to each fragment.
- `depthBiasClamp` is the maximum (or minimum) depth bias of a fragment.
- `depthBiasSlopeFactor` is a scalar factor applied to a fragment's slope in depth bias calculations.

This command sets the depth bias parameters for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_DEPTH_BIAS` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the corresponding VkPipelineRasterizationStateCreateInfo::`depthBiasConstantFactor`, `depthBiasClamp`, and `depthBiasSlopeFactor` values used to create the currently active pipeline.

Calling this function is equivalent to calling `vkCmdSetDepthBias2EXT` without a `VkDepthBiasRepresentationInfoEXT` in the pNext chain of `VkDepthBiasInfoEXT`.

Valid Usage

- VUID-vkCmdSetDepthBias-depthBiasClamp-00790

If the depthBiasClamp feature is not enabled, depthBiasClamp **must** be 0.0

Valid Usage (Implicit)

- VUID-vkCmdSetDepthBias-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthBias-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetDepthBias-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthBias-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

The VkDepthBiasRepresentationInfoEXT structure is defined as:

```
// Provided by VK_EXT_depth_bias_control
typedef struct VkDepthBiasRepresentationInfoEXT {
    VkStructureType                 sType;
    const void*                     pNext;
    VkDepthBiasRepresentationEXT    depthBiasRepresentation;
    VkBool32                        depthBiasExact;
} VkDepthBiasRepresentationInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- depthBiasRepresentation is a VkDepthBiasRepresentationEXT value specifying the depth bias

representation.

- `depthBiasExact` specifies that the implementation is not allowed to scale the depth bias value to ensure a minimum resolvable distance.

Valid Usage

- VUID-VkDepthBiasRepresentationInfoEXT-leastRepresentableValueForceUnormRepresentation-08947
  If the `leastRepresentableValueForceUnormRepresentation` feature is not enabled, `depthBiasRepresentation` **must** not be `VK_DEPTH_BIAS_REPRESENTATION_LEAST_REPRESENTABLE_VALUE_FORCE_UNORM_EXT`
- VUID-VkDepthBiasRepresentationInfoEXT-floatRepresentation-08948
  If the `floatRepresentation` feature is not enabled, `depthBiasRepresentation` **must** not be `VK_DEPTH_BIAS_REPRESENTATION_FLOAT_EXT`
- VUID-VkDepthBiasRepresentationInfoEXT-depthBiasExact-08949
  If the `depthBiasExact` feature is not enabled, `depthBiasExact` **must** be `VK_FALSE`

Valid Usage (Implicit)

- VUID-VkDepthBiasRepresentationInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEPTH_BIAS_REPRESENTATION_INFO_EXT`
- VUID-VkDepthBiasRepresentationInfoEXT-depthBiasRepresentation-parameter
  `depthBiasRepresentation` **must** be a valid VkDepthBiasRepresentationEXT value

Possible values of VkDepthBiasRepresentationInfoEXT::`depthBiasRepresentation`, specifying the depth bias representation are:

```
// Provided by VK_EXT_depth_bias_control
typedef enum VkDepthBiasRepresentationEXT {
    VK_DEPTH_BIAS_REPRESENTATION_LEAST_REPRESENTABLE_VALUE_FORMAT_EXT = 0,
    VK_DEPTH_BIAS_REPRESENTATION_LEAST_REPRESENTABLE_VALUE_FORCE_UNORM_EXT = 1,
    VK_DEPTH_BIAS_REPRESENTATION_FLOAT_EXT = 2,
} VkDepthBiasRepresentationEXT;
```

- `VK_DEPTH_BIAS_REPRESENTATION_LEAST_REPRESENTABLE_VALUE_FORMAT_EXT` specifies that the depth bias representation is a factor of the format's r as described in Depth Bias Computation.
- `VK_DEPTH_BIAS_REPRESENTATION_LEAST_REPRESENTABLE_VALUE_FORCE_UNORM_EXT` specifies that the depth bias representation is a factor of a constant r defined by the bit-size or mantissa of the format as described in Depth Bias Computation.
- `VK_DEPTH_BIAS_REPRESENTATION_FLOAT_EXT` specifies that the depth bias representation is a factor of constant r equal to 1.

The `VkDepthBiasInfoEXT` structure is defined as:

```
// Provided by VK_EXT_depth_bias_control
typedef struct VkDepthBiasInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    float              depthBiasConstantFactor;
    float              depthBiasClamp;
    float              depthBiasSlopeFactor;
} VkDepthBiasInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `depthBiasConstantFactor` is a scalar factor controlling the constant depth value added to each fragment.
- `depthBiasClamp` is the maximum (or minimum) depth bias of a fragment.
- `depthBiasSlopeFactor` is a scalar factor applied to a fragment's slope in depth bias calculations.

If `pNext` does not contain a VkDepthBiasRepresentationInfoEXT structure, then this command is equivalent to including a VkDepthBiasRepresentationInfoEXT with `depthBiasExact` set to `VK_FALSE` and `depthBiasRepresentation` set to `VK_DEPTH_BIAS_REPRESENTATION_LEAST_REPRESENTABLE_VALUE_FORMAT_EXT`.

Valid Usage

- VUID-VkDepthBiasInfoEXT-depthBiasClamp-08950
  If the `depthBiasClamp` feature is not enabled, `depthBiasClamp` **must** be `0.0`

Valid Usage (Implicit)

- VUID-VkDepthBiasInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEPTH_BIAS_INFO_EXT`
- VUID-VkDepthBiasInfoEXT-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkDepthBiasRepresentationInfoEXT
- VUID-VkDepthBiasInfoEXT-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

To dynamically set the depth bias parameters, call:

```
// Provided by VK_EXT_depth_bias_control
void vkCmdSetDepthBias2EXT(
    VkCommandBuffer                             commandBuffer,
    const VkDepthBiasInfoEXT*                   pDepthBiasInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pDepthBiasInfo` is a pointer to a VkDepthBiasInfoEXT structure specifying depth bias parameters.

This command is functionally identical to vkCmdSetDepthBias, but includes extensible sub-structures that include `sType` and `pNext` parameters, allowing them to be more easily extended.

Valid Usage (Implicit)

- VUID-vkCmdSetDepthBias2EXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthBias2EXT-pDepthBiasInfo-parameter
  `pDepthBiasInfo` **must** be a valid pointer to a valid VkDepthBiasInfoEXT structure
- VUID-vkCmdSetDepthBias2EXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthBias2EXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthBias2EXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

26.11.4. Conservative Rasterization

If the `pNext` chain of VkPipelineRasterizationStateCreateInfo includes a `VkPipelineRasterizationConservativeStateCreateInfoEXT` structure, then that structure includes parameters controlling conservative rasterization.

`VkPipelineRasterizationConservativeStateCreateInfoEXT` is defined as:

```
// Provided by VK_EXT_conservative_rasterization
typedef struct VkPipelineRasterizationConservativeStateCreateInfoEXT {
    VkStructureType                                           sType;
    const void*                                               pNext;
    VkPipelineRasterizationConservativeStateCreateFlagsEXT    flags;
    VkConservativeRasterizationModeEXT
conservativeRasterizationMode;
    float
extraPrimitiveOverestimationSize;
} VkPipelineRasterizationConservativeStateCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `conservativeRasterizationMode` is the conservative rasterization mode to use.
- `extraPrimitiveOverestimationSize` is the extra size in pixels to increase the generating primitive during conservative rasterization at each of its edges in `X` and `Y` equally in screen space beyond the base overestimation specified in `VkPhysicalDeviceConservativeRasterizationPropertiesEXT` ::`primitiveOverestimationSize`. If `conservativeRasterizationMode` is not `VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT`, this value is ignored.

If this structure is not included in the `pNext` chain, `conservativeRasterizationMode` is considered to be `VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT`, and conservative rasterization is disabled.

Polygon rasterization **can** be made conservative by setting `conservativeRasterizationMode` to `VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT` or `VK_CONSERVATIVE_RASTERIZATION_MODE_UNDERESTIMATE_EXT` in `VkPipelineRasterizationConservativeStateCreateInfoEXT`.

NOTE If `conservativePointAndLineRasterization` is supported, conservative rasterization can be applied to line and point primitives, otherwise it must be disabled.

Valid Usage

- VUID-VkPipelineRasterizationConservativeStateCreateInfoEXT-extraPrimitiveOverestimationSize-01769
  `extraPrimitiveOverestimationSize` **must** be in the range of `0.0` to `VkPhysicalDeviceConservativeRasterizationPropertiesEXT`::`maxExtraPrimitiveOverestimationSize` inclusive

Valid Usage (Implicit)

- VUID-VkPipelineRasterizationConservativeStateCreateInfoEXT-sType-sType
  `sType` **must** be

VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_CONSERVATIVE_STATE_CREATE_INFO_EXT

- VUID-VkPipelineRasterizationConservativeStateCreateInfoEXT-flags-zerobitmask flags **must** be 0
- VUID-VkPipelineRasterizationConservativeStateCreateInfoEXT-conservativeRasterizationMode-parameter conservativeRasterizationMode **must** be a valid VkConservativeRasterizationModeEXT value

```
// Provided by VK_EXT_conservative_rasterization
typedef VkFlags VkPipelineRasterizationConservativeStateCreateFlagsEXT;
```

VkPipelineRasterizationConservativeStateCreateFlagsEXT is a bitmask type for setting a mask, but is currently reserved for future use.

Possible values of VkPipelineRasterizationConservativeStateCreateInfoEXT ::conservativeRasterizationMode, specifying the conservative rasterization mode are:

```
// Provided by VK_EXT_conservative_rasterization
typedef enum VkConservativeRasterizationModeEXT {
    VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT = 0,
    VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT = 1,
    VK_CONSERVATIVE_RASTERIZATION_MODE_UNDERESTIMATE_EXT = 2,
} VkConservativeRasterizationModeEXT;
```

- VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT specifies that conservative rasterization is disabled and rasterization proceeds as normal.
- VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT specifies that conservative rasterization is enabled in overestimation mode.
- VK_CONSERVATIVE_RASTERIZATION_MODE_UNDERESTIMATE_EXT specifies that conservative rasterization is enabled in underestimation mode.

To dynamically set the conservativeRasterizationMode, call:

```
// Provided by VK_EXT_conservative_rasterization with VK_EXT_extended_dynamic_state3,
VK_EXT_conservative_rasterization with VK_EXT_shader_object
void vkCmdSetConservativeRasterizationModeEXT(
    VkCommandBuffer                             commandBuffer,
    VkConservativeRasterizationModeEXT          conservativeRasterizationMode);
```

- commandBuffer is the command buffer into which the command will be recorded.
- conservativeRasterizationMode specifies the conservativeRasterizationMode state.

This command sets the conservativeRasterizationMode state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with

VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineRasterizationConservativeStateCreateInfoEXT::conservativeRasterizationMode value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetConservativeRasterizationModeEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3ConservativeRasterizationMode feature is enabled
  - The shaderObject feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetConservativeRasterizationModeEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetConservativeRasterizationModeEXT-conservativeRasterizationMode-parameter
  conservativeRasterizationMode **must** be a valid VkConservativeRasterizationModeEXT value
- VUID-vkCmdSetConservativeRasterizationModeEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetConservativeRasterizationModeEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetConservativeRasterizationModeEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the extraPrimitiveOverestimationSize, call:

```
// Provided by VK_EXT_conservative_rasterization with VK_EXT_extended_dynamic_state3,
VK_EXT_conservative_rasterization with VK_EXT_shader_object
void vkCmdSetExtraPrimitiveOverestimationSizeEXT(
    VkCommandBuffer                             commandBuffer,
    float                                       extraPrimitiveOverestimationSize);
```

- commandBuffer is the command buffer into which the command will be recorded.
- extraPrimitiveOverestimationSize specifies the extraPrimitiveOverestimationSize.

This command sets the extraPrimitiveOverestimationSize for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineRasterizationConservativeStateCreateInfoEXT::extraPrimitiveOverestimationSize value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetExtraPrimitiveOverestimationSizeEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3ExtraPrimitiveOverestimationSize feature is enabled
  - The shaderObject feature is enabled
- VUID-vkCmdSetExtraPrimitiveOverestimationSizeEXT-extraPrimitiveOverestimationSize-07428
  extraPrimitiveOverestimationSize **must** be in the range of 0.0 to VkPhysicalDeviceConservativeRasterizationPropertiesEXT::maxExtraPrimitiveOverestimationSize inclusive

Valid Usage (Implicit)

- VUID-vkCmdSetExtraPrimitiveOverestimationSizeEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetExtraPrimitiveOverestimationSizeEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetExtraPrimitiveOverestimationSizeEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetExtraPrimitiveOverestimationSizeEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

When overestimate conservative rasterization is enabled, rather than evaluating coverage at individual sample locations, a determination is made whether any portion of the pixel (including its edges and corners) is covered by the primitive. If any portion of the pixel is covered, then all bits of the coverage mask for the fragment corresponding to that pixel are enabled. If the render pass has a fragment density map attachment and any bit of the coverage mask for the fragment is enabled, then all bits of the coverage mask for the fragment are enabled.

For the purposes of evaluating which pixels are covered by the primitive, implementations **can** increase the size of the primitive by up to `VkPhysicalDeviceConservativeRasterizationPropertiesEXT::primitiveOverestimationSize` pixels at each of the primitive edges. This **may** increase the number of fragments generated by this primitive and represents an overestimation of the pixel coverage.

This overestimation size can be increased further by setting the `extraPrimitiveOverestimationSize` value above `0.0` in steps of `VkPhysicalDeviceConservativeRasterizationPropertiesEXT::extraPrimitiveOverestimationSizeGranularity` up to and including `VkPhysicalDeviceConservativeRasterizationPropertiesEXT::extraPrimitiveOverestimationSize`. This **may** further increase the number of fragments generated by this primitive.

The actual precision of the overestimation size used for conservative rasterization **may** vary between implementations and produce results that only approximate the `primitiveOverestimationSize` and `extraPrimitiveOverestimationSizeGranularity` properties. Implementations **may** especially vary these approximations when the render pass has a fragment density map and the fragment area covers multiple pixels.

For triangles if `VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT` is enabled, fragments will be generated if the primitive area covers any portion of any pixel inside the fragment area, including their edges or corners. The tie-breaking rule described in Basic Polygon Rasterization does not apply during conservative rasterization and coverage is set for all fragments generated from shared edges of polygons. Degenerate triangles that evaluate to zero area after rasterization, even for pixels containing a vertex or edge of the zero-area polygon, will be culled if `VkPhysicalDeviceConservativeRasterizationPropertiesEXT::degenerateTrianglesRasterized` is `VK_FALSE` or will generate fragments if `degenerateTrianglesRasterized` is `VK_TRUE`. The fragment input values

for these degenerate triangles take their attribute and depth values from the provoking vertex. Degenerate triangles are considered backfacing and the application **can** enable backface culling if desired. Triangles that are zero area before rasterization **may** be culled regardless.

For lines if VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT is enabled, and the implementation sets VkPhysicalDeviceConservativeRasterizationPropertiesEXT ::conservativePointAndLineRasterization to VK_TRUE, fragments will be generated if the line covers any portion of any pixel inside the fragment area, including their edges or corners. Degenerate lines that evaluate to zero length after rasterization will be culled if VkPhysicalDeviceConservativeRasterizationPropertiesEXT::degenerateLinesRasterized is VK_FALSE or will generate fragments if degenerateLinesRasterized is VK_TRUE. The fragments input values for these degenerate lines take their attribute and depth values from the provoking vertex. Lines that are zero length before rasterization **may** be culled regardless.

For points if VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT is enabled, and the implementation sets VkPhysicalDeviceConservativeRasterizationPropertiesEXT ::conservativePointAndLineRasterization to VK_TRUE, fragments will be generated if the point square covers any portion of any pixel inside the fragment area, including their edges or corners.

When underestimate conservative rasterization is enabled, rather than evaluating coverage at individual sample locations, a determination is made whether all of the pixel (including its edges and corners) is covered by the primitive. If the entire pixel is covered, then a fragment is generated with all bits of its coverage mask corresponding to the pixel enabled, otherwise the pixel is not considered covered even if some portion of the pixel is covered. The fragment is discarded if no pixels inside the fragment area are considered covered. If the render pass has a fragment density map attachment and any pixel inside the fragment area is not considered covered, then the fragment is discarded even if some pixels are considered covered.

For triangles, if VK_CONSERVATIVE_RASTERIZATION_MODE_UNDERESTIMATE_EXT is enabled, fragments will only be generated if any pixel inside the fragment area is fully covered by the generating primitive, including its edges and corners.

For lines, if VK_CONSERVATIVE_RASTERIZATION_MODE_UNDERESTIMATE_EXT is enabled, fragments will be generated if any pixel inside the fragment area, including its edges and corners, are entirely covered by the line.

For points, if VK_CONSERVATIVE_RASTERIZATION_MODE_UNDERESTIMATE_EXT is enabled, fragments will only be generated if the point square covers the entirety of any pixel square inside the fragment area, including its edges or corners.

If the render pass has a fragment density map and VK_CONSERVATIVE_RASTERIZATION_MODE_UNDERESTIMATE_EXT is enabled, fragments will only be generated if the entirety of all pixels inside the fragment area are covered by the generating primitive, line, or point.

For both overestimate and underestimate conservative rasterization modes a fragment has all of its pixel squares fully covered by the generating primitive **must** set FullyCoveredEXT to VK_TRUE if the implementation enables the VkPhysicalDeviceConservativeRasterizationPropertiesEXT ::fullyCoveredFragmentShaderInputVariable feature.

When setting the fragment shading rate results in fragments covering multiple pixels, coverage for conservative rasterization is still evaluated on a per-pixel basis and may result in fragments with partial coverage. For fragment shader inputs decorated with `FullyCoveredEXT`, a fragment is considered fully covered if and only if all pixels in the fragment are fully covered by the generating primitive.

Chapter 27. Fragment Operations

Fragments produced by rasterization go through a number of operations to determine whether or how values produced by fragment shading are written to the framebuffer.

The following fragment operations adhere to rasterization order, and are typically performed in this order:

1. Discard rectangles test
2. Scissor test
3. Sample mask test
4. Certain Fragment shading operations:
    - Sample Mask Accesses
    - Tile Image Reads
    - Depth Replacement
    - Stencil Reference Replacement
    - Interlocked Operations
5. Multisample coverage
6. Depth bounds test
7. Stencil test
8. Depth test
9. Sample counting
10. Coverage reduction

The coverage mask generated by rasterization describes the initial coverage of each sample covered by the fragment. Fragment operations will update the coverage mask to add or subtract coverage where appropriate. If a fragment operation results in all bits of the coverage mask being `0`, the fragment is discarded, and no further operations are performed.

When one of the fragment operations in this chapter is described as “replacing” a fragment shader output, that output is replaced unconditionally, even if no fragment shader previously wrote to that output.

If there is a fragment shader and it declares the `PostDepthCoverage` execution mode, the sample mask test is instead performed after the depth test.

If `VkPhysicalDeviceMaintenance5PropertiesKHR`::`earlyFragmentMultisampleCoverageAfterSampleCounting` is `VK_TRUE` and there is a fragment shader which declares the `EarlyFragmentTests` execution mode, fragment shading and multisample coverage operations **must** be performed after sample counting.

Otherwise, if `VkPhysicalDeviceMaintenance5PropertiesKHR`::`earlyFragmentMultisampleCoverageAfterSampleCounting` is `VK_FALSE` and there is a fragment shader

which declares the EarlyFragmentTests execution mode, fragment shading and multisample coverage operations **should** instead be performed after sample counting, but **may** be performed before sample counting.

If VkPhysicalDeviceMaintenance5PropertiesKHR::earlyFragmentSampleMaskTestBeforeSampleCounting is VK_TRUE and there is a fragment shader which declares the EarlyFragmentTests execution mode sample mask test operations **must** follow the order of fragment operations from above.

Otherwise, if VkPhysicalDeviceMaintenance5PropertiesKHR::earlyFragmentSampleMaskTestBeforeSampleCounting is VK_FALSE and there is a fragment shader which declares the EarlyFragmentTests execution mode, sample mask test operations **should** follow the order of fragment operations from above but **may** instead be performed after sample counting.

For a pipeline with the following properties:

- a fragment shader is specified
- the fragment shader does not write to storage resources;
- the fragment shader specifies the DepthReplacing execution mode; and
- either
  - the fragment shader specifies the DepthUnchanged execution mode;
  - the fragment shader specifies the DepthLess execution mode and the pipeline uses a VkPipelineDepthStencilStateCreateInfo::depthCompareOp of VK_COMPARE_OP_GREATER or VK_COMPARE_OP_GREATER_OR_EQUAL; or
  - the fragment shader specifies the DepthGreater execution mode and the pipeline uses a VkPipelineDepthStencilStateCreateInfo::depthCompareOp of VK_COMPARE_OP_LESS or VK_COMPARE_OP_LESS_OR_EQUAL

the implementation **may** perform depth bounds test before fragment shading and perform an additional depth test immediately after that using the interpolated depth value generated by rasterization.

Once all fragment operations have completed, fragment shader outputs for covered color attachment samples pass through framebuffer operations.

27.1. Discard Rectangles Test

The discard rectangle test compares the framebuffer coordinates (x_f,y_f) of each sample covered by a fragment against a set of *discard rectangles*.

Each discard rectangle is defined by a VkRect2D. These values are either set by the VkPipelineDiscardRectangleStateCreateInfoEXT structure during pipeline creation, or dynamically by the vkCmdSetDiscardRectangleEXT command.

A given sample is considered inside a discard rectangle if the x_f is in the range [VkRect2D::offset.x, VkRect2D::offset.x + VkRect2D::extent.x), and y_f is in the range [VkRect2D::offset.y, VkRect2D::offset.y + VkRect2D::extent.y). If the test is set to be inclusive, samples that are not inside any of the discard rectangles will have their coverage set to 0. If the test is set to be exclusive, samples that

are inside any of the discard rectangles will have their coverage set to `0`.

If no discard rectangles are specified, the coverage mask is unmodified by this operation.

The `VkPipelineDiscardRectangleStateCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_discard_rectangles
typedef struct VkPipelineDiscardRectangleStateCreateInfoEXT {
    VkStructureType                                    sType;
    const void*                                        pNext;
    VkPipelineDiscardRectangleStateCreateFlagsEXT      flags;
    VkDiscardRectangleModeEXT                          discardRectangleMode;
    uint32_t                                           discardRectangleCount;
    const VkRect2D*                                    pDiscardRectangles;
} VkPipelineDiscardRectangleStateCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `discardRectangleMode` is a VkDiscardRectangleModeEXT value determining whether the discard rectangle test is inclusive or exclusive.
- `discardRectangleCount` is the number of discard rectangles to use.
- `pDiscardRectangles` is a pointer to an array of VkRect2D structures defining discard rectangles.

If the `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT` dynamic state is enabled for a pipeline, the `pDiscardRectangles` member is ignored. If the `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT` dynamic state is not enabled for the pipeline the presence of this structure in the VkGraphicsPipelineCreateInfo chain, and a `discardRectangleCount` greater than zero, implicitly enables discard rectangles in the pipeline, otherwise discard rectangles **must** enabled or disabled by vkCmdSetDiscardRectangleEnableEXT. If the `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT` dynamic state is enabled for the pipeline, the `discardRectangleMode` member is ignored, and the discard rectangle mode **must** be set by vkCmdSetDiscardRectangleModeEXT.

When this structure is included in the `pNext` chain of VkGraphicsPipelineCreateInfo, it defines parameters of the discard rectangle test. If the `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT` dynamic state is not enabled, and this structure is not included in the `pNext` chain, it is equivalent to specifying this structure with a `discardRectangleCount` of `0`.

Valid Usage

- VUID-VkPipelineDiscardRectangleStateCreateInfoEXT-discardRectangleCount-00582 `discardRectangleCount` **must** be less than or equal to `VkPhysicalDeviceDiscardRectanglePropertiesEXT::maxDiscardRectangles`

Valid Usage (Implicit)

- VUID-VkPipelineDiscardRectangleStateCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_DISCARD_RECTANGLE_STATE_CREATE_INFO_EXT`
- VUID-VkPipelineDiscardRectangleStateCreateInfoEXT-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineDiscardRectangleStateCreateInfoEXT-discardRectangleMode-parameter
  `discardRectangleMode` **must** be a valid VkDiscardRectangleModeEXT value

```
// Provided by VK_EXT_discard_rectangles
typedef VkFlags VkPipelineDiscardRectangleStateCreateFlagsEXT;
```

`VkPipelineDiscardRectangleStateCreateFlagsEXT` is a bitmask type for setting a mask, but is currently reserved for future use.

`VkDiscardRectangleModeEXT` values are:

```
// Provided by VK_EXT_discard_rectangles
typedef enum VkDiscardRectangleModeEXT {
    VK_DISCARD_RECTANGLE_MODE_INCLUSIVE_EXT = 0,
    VK_DISCARD_RECTANGLE_MODE_EXCLUSIVE_EXT = 1,
} VkDiscardRectangleModeEXT;
```

- `VK_DISCARD_RECTANGLE_MODE_INCLUSIVE_EXT` specifies that the discard rectangle test is inclusive.
- `VK_DISCARD_RECTANGLE_MODE_EXCLUSIVE_EXT` specifies that the discard rectangle test is exclusive.

To dynamically set the discard rectangles, call:

```
// Provided by VK_EXT_discard_rectangles
void vkCmdSetDiscardRectangleEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstDiscardRectangle,
    uint32_t                                    discardRectangleCount,
    const VkRect2D*                             pDiscardRectangles);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `firstDiscardRectangle` is the index of the first discard rectangle whose state is updated by the command.
- `discardRectangleCount` is the number of discard rectangles whose state are updated by the command.
- `pDiscardRectangles` is a pointer to an array of VkRect2D structures specifying discard rectangles.

The discard rectangle taken from element i of `pDiscardRectangles` replace the current state for the

discard rectangle at index `firstDiscardRectangle` + i, for i in [0, `discardRectangleCount`).

This command sets the discard rectangles for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineDiscardRectangleStateCreateInfoEXT::`pDiscardRectangles` values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDiscardRectangleEXT-firstDiscardRectangle-00585
  The sum of `firstDiscardRectangle` and `discardRectangleCount` **must** be less than or equal to VkPhysicalDeviceDiscardRectanglePropertiesEXT::`maxDiscardRectangles`
- VUID-vkCmdSetDiscardRectangleEXT-x-00587
  The `x` and `y` member of `offset` in each VkRect2D element of `pDiscardRectangles` **must** be greater than or equal to `0`
- VUID-vkCmdSetDiscardRectangleEXT-offset-00588
  Evaluation of (`offset.x` + `extent.width`) in each VkRect2D element of `pDiscardRectangles` **must** not cause a signed integer addition overflow
- VUID-vkCmdSetDiscardRectangleEXT-offset-00589
  Evaluation of (`offset.y` + `extent.height`) in each VkRect2D element of `pDiscardRectangles` **must** not cause a signed integer addition overflow

Valid Usage (Implicit)

- VUID-vkCmdSetDiscardRectangleEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDiscardRectangleEXT-pDiscardRectangles-parameter
  `pDiscardRectangles` **must** be a valid pointer to an array of `discardRectangleCount` VkRect2D structures
- VUID-vkCmdSetDiscardRectangleEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDiscardRectangleEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDiscardRectangleEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetDiscardRectangleEXT-discardRectangleCount-arraylength
  `discardRectangleCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set whether discard rectangles are enabled, call:

```
// Provided by VK_EXT_discard_rectangles
void vkCmdSetDiscardRectangleEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    discardRectangleEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `discardRectangleEnable` specifies whether discard rectangles are enabled or not.

This command sets the discard rectangle enable for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is implied by the VkPipelineDiscardRectangleStateCreateInfoEXT::`discardRectangleCount` value used to create the currently active pipeline, where a non-zero `discardRectangleCount` implicitly enables discard rectangles, otherwise they are disabled.

Valid Usage

- VUID-vkCmdSetDiscardRectangleEnableEXT-specVersion-07851
  The `VK_EXT_discard_rectangles` extension **must** be enabled, and the implementation **must** support at least `specVersion` `2` of this extension

Valid Usage (Implicit)

- VUID-vkCmdSetDiscardRectangleEnableEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDiscardRectangleEnableEXT-commandBuffer-recording

commandBuffer **must** be in the recording state

- VUID-vkCmdSetDiscardRectangleEnableEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetDiscardRectangleEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the discard rectangle mode, call:

```
// Provided by VK_EXT_discard_rectangles
void vkCmdSetDiscardRectangleModeEXT(
    VkCommandBuffer                             commandBuffer,
    VkDiscardRectangleModeEXT                   discardRectangleMode);
```

- commandBuffer is the command buffer into which the command will be recorded.
- discardRectangleMode specifies the discard rectangle mode for all discard rectangles, either inclusive or exclusive.

This command sets the discard rectangle mode for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleMode value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDiscardRectangleModeEXT-specVersion-07852
  The VK_EXT_discard_rectangles extension **must** be enabled, and the implementation **must**

support at least specVersion 2 of this extension

Valid Usage (Implicit)

- VUID-vkCmdSetDiscardRectangleModeEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDiscardRectangleModeEXT-discardRectangleMode-parameter
  discardRectangleMode **must** be a valid VkDiscardRectangleModeEXT value
- VUID-vkCmdSetDiscardRectangleModeEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetDiscardRectangleModeEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetDiscardRectangleModeEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

27.2. Scissor Test

The scissor test compares the framebuffer coordinates (x_f,y_f) of each sample covered by a fragment against a *scissor rectangle* at the index equal to the fragment's ViewportIndex.

Each scissor rectangle is defined by a VkRect2D. These values are either set by the VkPipelineViewportStateCreateInfo structure during pipeline creation, or dynamically by the vkCmdSetScissor command.

A given sample is considered inside a scissor rectangle if x_f is in the range [VkRect2D::offset.x, VkRect2D::offset.x + VkRect2D::extent.x), and y_f is in the range [VkRect2D::offset.y, VkRect2D ::offset.y + VkRect2D::extent.y). Samples with coordinates outside the scissor rectangle at the

corresponding ViewportIndex will have their coverage set to 0.

To dynamically set the scissor rectangles, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetScissor(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstScissor,
    uint32_t                                    scissorCount,
    const VkRect2D*                             pScissors);
```

- commandBuffer is the command buffer into which the command will be recorded.
- firstScissor is the index of the first scissor whose state is updated by the command.
- scissorCount is the number of scissors whose rectangles are updated by the command.
- pScissors is a pointer to an array of VkRect2D structures defining scissor rectangles.

The scissor rectangles taken from element i of pScissors replace the current state for the scissor index firstScissor + i, for i in [0, scissorCount).

This command sets the scissor rectangles for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_SCISSOR set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineViewportStateCreateInfo::pScissors values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetScissor-firstScissor-00592
  The sum of firstScissor and scissorCount **must** be between 1 and VkPhysicalDeviceLimits::maxViewports, inclusive
- VUID-vkCmdSetScissor-firstScissor-00593
  If the multiViewport feature is not enabled, firstScissor **must** be 0
- VUID-vkCmdSetScissor-scissorCount-00594
  If the multiViewport feature is not enabled, scissorCount **must** be 1
- VUID-vkCmdSetScissor-x-00595
  The x and y members of offset member of any element of pScissors **must** be greater than or equal to 0
- VUID-vkCmdSetScissor-offset-00596
  Evaluation of (offset.x + extent.width) **must** not cause a signed integer addition overflow for any element of pScissors
- VUID-vkCmdSetScissor-offset-00597
  Evaluation of (offset.y + extent.height) **must** not cause a signed integer addition overflow for any element of pScissors

Valid Usage (Implicit)

- VUID-vkCmdSetScissor-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetScissor-pScissors-parameter
  `pScissors` **must** be a valid pointer to an array of `scissorCount` VkRect2D structures
- VUID-vkCmdSetScissor-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetScissor-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetScissor-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetScissor-scissorCount-arraylength
  `scissorCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

27.3. Sample Mask Test

The sample mask test compares the coverage mask for a fragment with the *sample mask* defined by VkPipelineMultisampleStateCreateInfo::`pSampleMask`.

To dynamically set the sample mask, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetSampleMaskEXT(
    VkCommandBuffer                             commandBuffer,
    VkSampleCountFlagBits                       samples,
    const VkSampleMask*                         pSampleMask);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `samples` specifies the number of sample bits in the `pSampleMask`.
- `pSampleMask` is a pointer to an array of VkSampleMask values, where the array size is based on the `samples` parameter.

This command sets the sample mask for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_SAMPLE_MASK_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineMultisampleStateCreateInfo::`pSampleMask` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetSampleMaskEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3SampleMask` feature is enabled
  - The `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetSampleMaskEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetSampleMaskEXT-samples-parameter
  `samples` **must** be a valid VkSampleCountFlagBits value
- VUID-vkCmdSetSampleMaskEXT-pSampleMask-parameter
  `pSampleMask` **must** be a valid pointer to an array of $\lceil \frac{samples}{32} \rceil$ VkSampleMask values
- VUID-vkCmdSetSampleMaskEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetSampleMaskEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetSampleMaskEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Each bit of the coverage mask is associated with a sample index as described in the rasterization chapter. If the bit in VkPipelineMultisampleStateCreateInfo::`pSampleMask` which is associated with that same sample index is set to `0`, the coverage mask bit is set to `0`.

27.4. Fragment Shading

Fragment shaders are invoked for each fragment, or as helper invocations.

Most operations in the fragment shader are not performed in rasterization order, with exceptions called out in the following sections.

For fragment shaders invoked by fragments, the following rules apply:

- A fragment shader **must** not be executed if a fragment operation that executes before fragment shading discards the fragment.
- A fragment shader **may** not be executed if:
  - An implementation determines that another fragment shader, invoked by a subsequent primitive in primitive order, overwrites all results computed by the shader (including writes to storage resources).
  - Any other fragment operation discards the fragment, and the shader does not write to any storage resources.
  - If a fragment shader statically computes the same values for different framebuffer locations, and does not write to any storage resources, multiple fragments **may** be shaded by one fragment shader invocation. This **may** affect `VK_QUERY_PIPELINE_STATISTIC_FRAGMENT_SHADER_INVOCATIONS_BIT` results, but **must** otherwise not be visible behavior to applications.
- Otherwise, at least one fragment shader **must** be executed.
  - If sample shading is enabled and multiple invocations per fragment are **required**, additional invocations **must** be executed as specified.

- Each covered sample **must** be included in at least one fragment shader invocation.

If no fragment shader is included in the pipeline, no fragment shader is executed, and undefined values **may** be written to all color attachment outputs during this fragment operation.

NOTE

Multiple fragment shader invocations may be executed for the same fragment for any number of implementation-dependent reasons. When there is more than one fragment shader invocation per fragment, the association of samples to invocations is implementation-dependent. Stores and atomics performed by these additional invocations have the normal effect.

For example, if the subpass includes multiple views in its view mask, a fragment shader may be invoked separately for each view.

Similarly, if the render pass has a fragment density map attachment, more than one fragment shader invocation may be invoked for each covered sample. Such additional invocations are only produced if `VkPhysicalDeviceFragmentDensityMapPropertiesEXT`::`fragmentDensityInvocations` is `VK_TRUE`. Implementations may generate these additional fragment shader invocations in order to make transitions between fragment areas with different fragment densities more smooth.

27.4.1. Sample Mask

Reading from the `SampleMask` built-in in the `Input` storage class will return the coverage mask for the current fragment as calculated by fragment operations that executed prior to fragment shading.

If sample shading is enabled, fragment shaders will only see values of `1` for samples being shaded - other bits will be `0`.

Each bit of the coverage mask is associated with a sample index as described in the rasterization chapter. If the bit in `SampleMask` which is associated with that same sample index is set to `0`, that coverage mask bit is set to `0`.

Values written to the `SampleMask` built-in in the `Output` storage class will be used by the multisample coverage operation, with the same encoding as the input built-in.

27.4.2. Shader Termination and Demotion

If a fragment shader is terminated early or demoted to a helper invocation by using `OpKill`, `OpTerminateInvocation`, or `OpDemoteToHelperInvocation`, samples covered by that shader invocation have their coverage mask bits set to `0`.

27.4.3. Fragment Shader Tile Image Reads

If the `VK_EXT_shader_tile_image` extension is enabled, implementations divide the framebuffer into a grid of tiles. A tile image is a view of a framebuffer attachment tile for fragments with locations within the tile.

Within a render pass instance initiated by vkCmdBeginRenderingKHR, fragment shader invocations **can** read the framebuffer color, depth, and stencil values at the fragment location via tile images.

NOTE

Even though fragment shader invocation can only read from the corresponding fragment location, the abstraction of a tile image is introduced for the following reasons:

- Tile dimensions will be exposed in a future extension
- Future functionality such as executing compute dispatches within render passes via tile shaders can leverage tile images.

Enabling shaderTileImageColorReadAccess, shaderTileImageDepthReadAccess, shaderTileImageStencilReadAccess enables fragment shader invocations to read from color, depth, and stencil, respectively.

Color values are read from tile image variables with `OpColorAttachmentReadEXT`. Tile image variables are linked to specific color attachments using `Location` decoration. See Fragment Tile Image Interface for more details.

Depth values are read with `OpDepthAttachmentReadEXT`.

Stencil values are read with `OpStencilAttachmentReadEXT`.

The sample to read is specified by a sample index value specified as the `Sample` operand to `OpColorAttachmentReadEXT`, `OpDepthAttachmentReadEXT`, or `OpStencilAttachmentReadEXT`.

If sample shading is disabled, a fragment invocation **can** read from all sample locations associated with the fragment regardless of the fragment's coverage. This functionality is supported for VkPipelineMultisampleStateCreateInfo::`rasterizationSamples` > 1 when VkPhysicalDeviceShaderTileImagePropertiesEXT::`shaderTileImageReadSampleFromPixelRateInvocation` is `VK_TRUE`.

If sample shading is enabled, and `minSampleShading` is 1.0, a fragment invocation **must** only read from the coverage index sample. Tile image access **must** not be used if the value of `minSampleShading` is not 1.0.

If the fragment shader declares the `EarlyFragmentTests` execution mode, depth reads are allowed only if depth writes are disabled and stencil reads are allowed only if stencil writes are disabled.

If VkPhysicalDeviceShaderTileImagePropertiesEXT::`shaderTileImageReadFromHelperInvocation` is `VK_FALSE`, values read from helper invocations are undefined otherwise the values read are subject to the coherency guarantees described below.

`OpDepthAttachmentReadEXT` returns an undefined value if no depth attachment is present. `OpStencilAttachmentReadEXT` returns an undefined value if no stencil attachment is present.

Tile image reads from color, depth and stencil attachments are said to be coherent when the accesses happen in raster order and without data race with respect to accesses to the attachments from framebuffer-space pipeline stages. The samples which qualify for coherent access and the

enabling conditions are described below.

- Let Rc be the set of components being read from an attachment A in a draw call
- Let Wc be the set of components being written to A by the draw call

The samples which qualify for coherent tile image reads from an attachment A are:

- All samples in a pixel when Rc is disjoint with Wc.
- The samples with coverage in a fragment when Rc is not disjoint with Wc. The samples with coverage are determined by the coverage mask for the fragment as calculated by fragment operations that executed prior to fragment shading, including early fragment tests if enabled for the draw call.

A fragment shader **can** declare `NonCoherentColorAttachmentReadEXT`, `NonCoherentDepthAttachmentReadEXT`, or `NonCoherentStencilAttachmentReadEXT` execution modes to enable non-coherent tile image reads which require an explicit vkCmdPipelineBarrier2KHR call for the writes to an attachment to be made visible via tile image reads.

When VkPhysicalDeviceShaderTileImagePropertiesEXT::`shaderTileImageCoherentReadAccelerated` is `VK_TRUE`, the implementation prefers that coherent tile image reads are used, otherwise the implementation prefers that non-coherent tile image reads are used.

NOTE

In practice, the most common tile image reads usage patterns fall under one of the following:

- Programmable blending - each fragment reads from a single sample (SampleID) at its location. Per-sample shading is typically enabled when multisampled rendertargets are used.
- G-buffer generation and shading in one render pass - in the shading phase a fragment reads from a single sample at its location.
- Programmable resolve - a fragment reads from all samples at its location (per-sample shading is disabled). This requires the use of a "full-screen triangle" instead of a rectangle composed of two triangles in order to avoid data races along the shared edge of the triangles.
- 1:1 texturing with LOD - in use cases such a deferred screen space decals a fragment reads a single sample (SampleID) from depth buffer, but requires being able to read from helper threads to derive the texture LOD. This use case is supported as long as the attachment components being read are not overwritten by color, depth, or stencil attachment writes.

All of the above use cases are supported by coherent tile image reads, but only the latter three are supported when non-coherent reads are used as there is no mechanism to synchronize non-coherent reads with writes within a draw call.

27.4.4. Depth Replacement

Writing to the `FragDepth` built-in will replace the fragment's calculated depth values for each sample

in the input SampleMask. Depth testing performed after the fragment shader for this fragment will use this new value as z_f.

27.4.5. Stencil Reference Replacement

Writing to the FragStencilRefEXT built-in will replace the fragment's stencil reference value for each sample in the input SampleMask. Stencil testing performed after the fragment shader for this fragment will use this new value as s_r.

27.4.6. Interlocked Operations

OpBeginInvocationInterlockEXT and OpEndInvocationInterlockEXT define a section of a fragment shader which imposes additional ordering constraints on operations performed within them. These operations are defined as *interlocked operations*. How interlocked operations are ordered against other fragment shader invocations depends on the specified execution modes.

If the ShadingRateInterlockOrderedEXT execution mode is specified, any interlocked operations in a fragment shader **must** happen before interlocked operations in fragment shader invocations that execute later in rasterization order and cover at least one sample in the same fragment area, and **must** happen after interlocked operations in a fragment shader that executes earlier in rasterization order and cover at least one sample in the same fragment area.

If the ShadingRateInterlockUnorderedEXT execution mode is specified, any interlocked operations in a fragment shader **must** happen before or after interlocked operations in fragment shader invocations that execute earlier or later in rasterization order and cover at least one sample in the same fragment area.

If the PixelInterlockOrderedEXT execution mode is specified, any interlocked operations in a fragment shader **must** happen before interlocked operations in fragment shader invocations that execute later in rasterization order and cover at least one sample in the same pixel, and **must** happen after interlocked operations in a fragment shader that executes earlier in rasterization order and cover at least one sample in the same pixel.

If the PixelInterlockUnorderedEXT execution mode is specified, any interlocked operations in a fragment shader **must** happen before or after interlocked operations in fragment shader invocations that execute earlier or later in rasterization order and cover at least one sample in the same pixel.

If the SampleInterlockOrderedEXT execution mode is specified, any interlocked operations in a fragment shader **must** happen before interlocked operations in fragment shader invocations that execute later in rasterization order and cover at least one of the same samples, and **must** happen after interlocked operations in a fragment shader that executes earlier in rasterization order and cover at least one of the same samples.

If the SampleInterlockUnorderedEXT execution mode is specified, any interlocked operations in a fragment shader **must** happen before or after interlocked operations in fragment shader invocations that execute earlier or later in rasterization order and cover at least one of the same samples.

27.5. Multisample Coverage

If a fragment shader is active and its entry point's interface includes a built-in output variable decorated with SampleMask, the coverage mask is ANDed with the bits of the SampleMask built-in to generate a new coverage mask. If sample shading is enabled, bits written to SampleMask corresponding to samples that are not being shaded by the fragment shader invocation are ignored. If no fragment shader is active, or if the active fragment shader does not include SampleMask in its interface, the coverage mask is not modified.

Next, the fragment alpha value and coverage mask are modified based on the line coverage factor if the lineRasterizationMode member of the VkPipelineRasterizationStateCreateInfo structure is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, and the alphaToCoverageEnable and alphaToOneEnable members of the VkPipelineMultisampleStateCreateInfo structure.

To dynamically set the alphaToCoverageEnable state, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetAlphaToCoverageEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    alphaToCoverageEnable);
```

- commandBuffer is the command buffer into which the command will be recorded.
- alphaToCoverageEnable specifies the alphaToCoverageEnable state.

This command sets the alphaToCoverageEnable state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT set in VkPipelineDynamicStateCreateInfo ::pDynamicStates. Otherwise, this state is specified by the VkPipelineMultisampleStateCreateInfo ::alphaToCoverageEnable value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetAlphaToCoverageEnableEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3AlphaToCoverageEnable feature is enabled
  - The shaderObject feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetAlphaToCoverageEnableEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetAlphaToCoverageEnableEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetAlphaToCoverageEnableEXT-commandBuffer-cmdpool

The VkCommandPool that commandBuffer was allocated from **must** support graphics operations

- VUID-vkCmdSetAlphaToCoverageEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the alphaToOneEnable state, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetAlphaToOneEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    alphaToOneEnable);
```

- commandBuffer is the command buffer into which the command will be recorded.
- alphaToOneEnable specifies the alphaToOneEnable state.

This command sets the alphaToOneEnable state for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineMultisampleStateCreateInfo::alphaToOneEnable value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetAlphaToOneEnableEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3AlphaToOneEnable feature is enabled
  - The shaderObject feature is enabled
- VUID-vkCmdSetAlphaToOneEnableEXT-alphaToOne-07607

If the alphaToOne feature is not enabled, alphaToOneEnable **must** be VK_FALSE

Valid Usage (Implicit)

- VUID-vkCmdSetAlphaToOneEnableEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetAlphaToOneEnableEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetAlphaToOneEnableEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetAlphaToOneEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

All alpha values in this section refer only to the alpha component of the fragment shader output that has a Location and Index decoration of zero (see the Fragment Output Interface section). If that shader output has an integer or unsigned integer type, then these operations are skipped.

If the lineRasterizationMode member of the VkPipelineRasterizationStateCreateInfo structure is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR and the fragment came from a line segment, then the alpha value is replaced by multiplying it by the coverage factor for the fragment computed during smooth line rasterization.

If alphaToCoverageEnable is enabled, a temporary coverage mask is generated where each bit is determined by the fragment's alpha value, which is ANDed with the fragment coverage mask.

No specific algorithm is specified for converting the alpha value to a temporary coverage mask. It is intended that the number of 1's in this value be proportional to the alpha value (clamped to [0,1]), with all 1's corresponding to a value of 1.0 and all 0's corresponding to 0.0. The algorithm **may** be

different at different framebuffer coordinates.

NOTE	Using different algorithms at different framebuffer coordinates **may** help to avoid artifacts caused by regular coverage sample locations.

Finally, if `alphaToOneEnable` is enabled, each alpha value is replaced by the maximum representable alpha value for fixed-point color attachments, or by 1.0 for floating-point attachments. Otherwise, the alpha values are not changed.

27.6. Depth and Stencil Operations

Pipeline state controlling the depth bounds tests, stencil test, and depth test is specified through the members of the `VkPipelineDepthStencilStateCreateInfo` structure.

The `VkPipelineDepthStencilStateCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineDepthStencilStateCreateInfo {
    VkStructureType                           sType;
    const void*                               pNext;
    VkPipelineDepthStencilStateCreateFlags    flags;
    VkBool32                                  depthTestEnable;
    VkBool32                                  depthWriteEnable;
    VkCompareOp                               depthCompareOp;
    VkBool32                                  depthBoundsTestEnable;
    VkBool32                                  stencilTestEnable;
    VkStencilOpState                          front;
    VkStencilOpState                          back;
    float                                     minDepthBounds;
    float                                     maxDepthBounds;
} VkPipelineDepthStencilStateCreateInfo;
```

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `depthTestEnable` controls whether depth testing is enabled.
- `depthWriteEnable` controls whether depth writes are enabled when `depthTestEnable` is `VK_TRUE`. Depth writes are always disabled when `depthTestEnable` is `VK_FALSE`.
- `depthCompareOp` is a `VkCompareOp` value specifying the comparison operator to use in the Depth Comparison step of the depth test.
- `depthBoundsTestEnable` controls whether depth bounds testing is enabled.
- `stencilTestEnable` controls whether stencil testing is enabled.
- `front` and `back` are `VkStencilOpState` values controlling the corresponding parameters of the stencil test.

- `minDepthBounds` is the minimum depth bound used in the depth bounds test.
- `maxDepthBounds` is the maximum depth bound used in the depth bounds test.

Valid Usage

- VUID-VkPipelineDepthStencilStateCreateInfo-depthBoundsTestEnable-00598
  If the `depthBounds` feature is not enabled, `depthBoundsTestEnable` **must** be `VK_FALSE`
- VUID-VkPipelineDepthStencilStateCreateInfo-separateStencilMaskRef-04453
  If the `VK_KHR_portability_subset` extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::`separateStencilMaskRef` is `VK_FALSE`, and the value of VkPipelineDepthStencilStateCreateInfo::`stencilTestEnable` is `VK_TRUE`, and the value of VkPipelineRasterizationStateCreateInfo::`cullMode` is `VK_CULL_MODE_NONE`, the value of `reference` in each of the VkStencilOpState structs in `front` and `back` **must** be the same

Valid Usage (Implicit)

- VUID-VkPipelineDepthStencilStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO`
- VUID-VkPipelineDepthStencilStateCreateInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPipelineDepthStencilStateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineDepthStencilStateCreateInfo-depthCompareOp-parameter
  `depthCompareOp` **must** be a valid VkCompareOp value
- VUID-VkPipelineDepthStencilStateCreateInfo-front-parameter
  `front` **must** be a valid VkStencilOpState structure
- VUID-VkPipelineDepthStencilStateCreateInfo-back-parameter
  `back` **must** be a valid VkStencilOpState structure

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineDepthStencilStateCreateFlags;
```

`VkPipelineDepthStencilStateCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

27.7. Depth Bounds Test

The depth bounds test compares the depth value z_a in the depth/stencil attachment at each sample's framebuffer coordinates (x_f, y_f) and sample index i against a set of *depth bounds*.

The depth bounds are determined by two floating-point values defining a minimum (`minDepthBounds`) and maximum (`maxDepthBounds`) depth value. These values are either set by the

VkPipelineDepthStencilStateCreateInfo structure during pipeline creation, or dynamically by vkCmdSetDepthBoundsTestEnableEXT and vkCmdSetDepthBounds.

A given sample is considered within the depth bounds if z_a is in the range [minDepthBounds ,maxDepthBounds]. Samples with depth attachment values outside of the depth bounds will have their coverage set to 0.

If the depth bounds test is disabled, or if there is no depth attachment, the coverage mask is unmodified by this operation.

To dynamically enable or disable the depth bounds test, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetDepthBoundsTestEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    depthBoundsTestEnable);
```

- commandBuffer is the command buffer into which the command will be recorded.
- depthBoundsTestEnable specifies if the depth bounds test is enabled.

This command sets the depth bounds enable for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineDepthStencilStateCreateInfo::depthBoundsTestEnable value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDepthBoundsTestEnable-None-08971
  At least one of the following **must** be true:
  - the shaderObject feature is enabled
- VUID-vkCmdSetDepthBoundsTestEnable-depthBounds-10010
  If the depthBounds feature is not enabled, depthBoundsTestEnable **must** be VK_FALSE

Valid Usage (Implicit)

- VUID-vkCmdSetDepthBoundsTestEnable-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthBoundsTestEnable-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetDepthBoundsTestEnable-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthBoundsTestEnable-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the depth bounds range, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetDepthBounds(
    VkCommandBuffer                             commandBuffer,
    float                                       minDepthBounds,
    float                                       maxDepthBounds);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `minDepthBounds` is the minimum depth bound.
- `maxDepthBounds` is the maximum depth bound.

This command sets the depth bounds range for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_DEPTH_BOUNDS` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineDepthStencilStateCreateInfo::`minDepthBounds` and VkPipelineDepthStencilStateCreateInfo::`maxDepthBounds` values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDepthBounds-minDepthBounds-00600
  If the `VK_EXT_depth_range_unrestricted` extension is not enabled `minDepthBounds` **must** be between `0.0` and `1.0`, inclusive
- VUID-vkCmdSetDepthBounds-maxDepthBounds-00601
  If the `VK_EXT_depth_range_unrestricted` extension is not enabled `maxDepthBounds` **must** be between `0.0` and `1.0`, inclusive

Valid Usage (Implicit)

- VUID-vkCmdSetDepthBounds-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthBounds-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthBounds-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthBounds-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

27.8. Stencil Test

The stencil test compares the stencil attachment value s_a in the depth/stencil attachment at each sample's framebuffer coordinates (x_f,y_f) and sample index i against a *stencil reference value*.

If the render pass has a fragment density map attachment and the fragment covers multiple pixels, there is an implementation-dependent association of coverage samples to stencil attachment samples within the fragment. However, if all samples in the fragment are covered, and the stencil attachment value is updated as a result of this test, all stencil attachment samples will be updated.

If the stencil test is not enabled, as specified by VkPipelineDepthStencilStateCreateInfo ::`stencilTestEnable`, or if there is no stencil attachment, the coverage mask is unmodified by this operation.

The stencil test is controlled by one of two sets of stencil-related state, the front stencil state and the back stencil state. Stencil tests and writes use the back stencil state when processing fragments generated by back-facing polygons, and the front stencil state when processing fragments

generated by front-facing polygons or any other primitives.

The comparison operation performed is determined by the VkCompareOp value set by VkStencilOpState::compareOp during pipeline creation.

The compare mask s_c and stencil reference value s_r of the front or the back stencil state set determine arguments of the comparison operation. s_c is set by the VkPipelineDepthStencilStateCreateInfo structure during pipeline creation, or by the vkCmdSetStencilCompareMask command. s_r is set by VkPipelineDepthStencilStateCreateInfo or by vkCmdSetStencilReference.

s_r and s_a are each independently combined with s_c using a bitwise AND operation to create masked reference and attachment values s'_r and s'_a. s'_r and s'_a are used as the *reference* and *test* values, respectively, in the operation specified by the VkCompareOp.

If the comparison evaluates to false, the coverage for the sample is set to 0.

A new stencil value s_g is generated according to a stencil operation defined by VkStencilOp parameters set by VkPipelineDepthStencilStateCreateInfo. If the stencil test fails, failOp defines the stencil operation used. If the stencil test passes however, the stencil op used is based on the depth test - if it passes, VkPipelineDepthStencilStateCreateInfo::passOp is used, otherwise VkPipelineDepthStencilStateCreateInfo::depthFailOp is used.

The stencil attachment value s_a is then updated with the generated stencil value s_g according to the write mask s_w defined by writeMask in VkPipelineDepthStencilStateCreateInfo::front and VkPipelineDepthStencilStateCreateInfo::back as:

$$s_a = (s_a \,\&\, \neg s_w) \mid (s_g \,\&\, s_w)$$

To dynamically enable or disable the stencil test, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetStencilTestEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    stencilTestEnable);
```

- commandBuffer is the command buffer into which the command will be recorded.
- stencilTestEnable specifies if the stencil test is enabled.

This command sets the stencil test enable for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineDepthStencilStateCreateInfo ::stencilTestEnable value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetStencilTestEnable-None-08971
  At least one of the following **must** be true:

- the shaderObject feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetStencilTestEnable-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetStencilTestEnable-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetStencilTestEnable-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetStencilTestEnable-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the stencil operation, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetStencilOpEXT(
    VkCommandBuffer                             commandBuffer,
    VkStencilFaceFlags                          faceMask,
    VkStencilOp                                 failOp,
    VkStencilOp                                 passOp,
    VkStencilOp                                 depthFailOp,
    VkCompareOp                                 compareOp);
```

- commandBuffer is the command buffer into which the command will be recorded.
- faceMask is a bitmask of VkStencilFaceFlagBits specifying the set of stencil state for which to update the stencil operation.

- `failOp` is a VkStencilOp value specifying the action performed on samples that fail the stencil test.
- `passOp` is a VkStencilOp value specifying the action performed on samples that pass both the depth and stencil tests.
- `depthFailOp` is a VkStencilOp value specifying the action performed on samples that pass the stencil test and fail the depth test.
- `compareOp` is a VkCompareOp value specifying the comparison operator used in the stencil test.

This command sets the stencil operation for subsequent drawing commands when when drawing using shader objects. Otherwise, this state is specified by the corresponding `VkPipelineDepthStencilStateCreateInfo`::`failOp`, `passOp`, `depthFailOp`, and `compareOp` values used to create the currently active pipeline, for both front and back faces.

Valid Usage

- VUID-vkCmdSetStencilOp-None-08971
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetStencilOp-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetStencilOp-faceMask-parameter
  `faceMask` **must** be a valid combination of VkStencilFaceFlagBits values
- VUID-vkCmdSetStencilOp-faceMask-requiredbitmask
  `faceMask` **must** not be `0`
- VUID-vkCmdSetStencilOp-failOp-parameter
  `failOp` **must** be a valid VkStencilOp value
- VUID-vkCmdSetStencilOp-passOp-parameter
  `passOp` **must** be a valid VkStencilOp value
- VUID-vkCmdSetStencilOp-depthFailOp-parameter
  `depthFailOp` **must** be a valid VkStencilOp value
- VUID-vkCmdSetStencilOp-compareOp-parameter
  `compareOp` **must** be a valid VkCompareOp value
- VUID-vkCmdSetStencilOp-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetStencilOp-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetStencilOp-videocoding

This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

The `VkStencilOpState` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkStencilOpState {
    VkStencilOp    failOp;
    VkStencilOp    passOp;
    VkStencilOp    depthFailOp;
    VkCompareOp    compareOp;
    uint32_t       compareMask;
    uint32_t       writeMask;
    uint32_t       reference;
} VkStencilOpState;
```

- `failOp` is a VkStencilOp value specifying the action performed on samples that fail the stencil test.
- `passOp` is a VkStencilOp value specifying the action performed on samples that pass both the depth and stencil tests.
- `depthFailOp` is a VkStencilOp value specifying the action performed on samples that pass the stencil test and fail the depth test.
- `compareOp` is a VkCompareOp value specifying the comparison operator used in the stencil test.
- `compareMask` selects the bits of the unsigned integer stencil values participating in the stencil test.
- `writeMask` selects the bits of the unsigned integer stencil values updated by the stencil test in the stencil framebuffer attachment.
- `reference` is an integer stencil reference value that is used in the unsigned stencil comparison.

Valid Usage (Implicit)

- VUID-VkStencilOpState-failOp-parameter
  `failOp` **must** be a valid VkStencilOp value
- VUID-VkStencilOpState-passOp-parameter
  `passOp` **must** be a valid VkStencilOp value
- VUID-VkStencilOpState-depthFailOp-parameter
  `depthFailOp` **must** be a valid VkStencilOp value
- VUID-VkStencilOpState-compareOp-parameter
  `compareOp` **must** be a valid VkCompareOp value

To dynamically set the stencil compare mask, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetStencilCompareMask(
    VkCommandBuffer                             commandBuffer,
    VkStencilFaceFlags                          faceMask,
    uint32_t                                    compareMask);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `faceMask` is a bitmask of VkStencilFaceFlagBits specifying the set of stencil state for which to update the compare mask.
- `compareMask` is the new value to use as the stencil compare mask.

This command sets the stencil compare mask for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkStencilOpState::`compareMask` value used to create the currently active pipeline, for both front and back faces.

Valid Usage (Implicit)

- VUID-vkCmdSetStencilCompareMask-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetStencilCompareMask-faceMask-parameter
  `faceMask` **must** be a valid combination of VkStencilFaceFlagBits values
- VUID-vkCmdSetStencilCompareMask-faceMask-requiredbitmask
  `faceMask` **must** not be `0`
- VUID-vkCmdSetStencilCompareMask-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetStencilCompareMask-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics

operations

- VUID-vkCmdSetStencilCompareMask-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

`VkStencilFaceFlagBits` values are:

```
// Provided by VK_VERSION_1_0
typedef enum VkStencilFaceFlagBits {
    VK_STENCIL_FACE_FRONT_BIT = 0x00000001,
    VK_STENCIL_FACE_BACK_BIT = 0x00000002,
    VK_STENCIL_FACE_FRONT_AND_BACK = 0x00000003,
  // VK_STENCIL_FRONT_AND_BACK is a deprecated alias
    VK_STENCIL_FRONT_AND_BACK = VK_STENCIL_FACE_FRONT_AND_BACK,
} VkStencilFaceFlagBits;
```

- `VK_STENCIL_FACE_FRONT_BIT` specifies that only the front set of stencil state is updated.
- `VK_STENCIL_FACE_BACK_BIT` specifies that only the back set of stencil state is updated.
- `VK_STENCIL_FACE_FRONT_AND_BACK` is the combination of `VK_STENCIL_FACE_FRONT_BIT` and `VK_STENCIL_FACE_BACK_BIT`, and specifies that both sets of stencil state are updated.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkStencilFaceFlags;
```

`VkStencilFaceFlags` is a bitmask type for setting a mask of zero or more VkStencilFaceFlagBits.

To dynamically set the stencil write mask, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetStencilWriteMask(
    VkCommandBuffer                             commandBuffer,
    VkStencilFaceFlags                          faceMask,
    uint32_t                                    writeMask);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `faceMask` is a bitmask of VkStencilFaceFlagBits specifying the set of stencil state for which to update the write mask, as described above for vkCmdSetStencilCompareMask.
- `writeMask` is the new value to use as the stencil write mask.

This command sets the stencil write mask for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_STENCIL_WRITE_MASK` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the `writeMask` value used to create the currently active pipeline, for both VkPipelineDepthStencilStateCreateInfo::`front` and VkPipelineDepthStencilStateCreateInfo::`back` faces.

Valid Usage (Implicit)

- VUID-vkCmdSetStencilWriteMask-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetStencilWriteMask-faceMask-parameter
  `faceMask` **must** be a valid combination of VkStencilFaceFlagBits values
- VUID-vkCmdSetStencilWriteMask-faceMask-requiredbitmask
  `faceMask` **must** not be `0`
- VUID-vkCmdSetStencilWriteMask-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetStencilWriteMask-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetStencilWriteMask-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the stencil reference value, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetStencilReference(
    VkCommandBuffer                             commandBuffer,
    VkStencilFaceFlags                          faceMask,
    uint32_t                                    reference);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `faceMask` is a bitmask of VkStencilFaceFlagBits specifying the set of stencil state for which to update the reference value, as described above for vkCmdSetStencilCompareMask.
- `reference` is the new value to use as the stencil reference value.

This command sets the stencil reference value for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_STENCIL_REFERENCE` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineDepthStencilStateCreateInfo::`reference` value used to create the currently active pipeline, for both front and back faces.

Valid Usage (Implicit)

- VUID-vkCmdSetStencilReference-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetStencilReference-faceMask-parameter
  `faceMask` **must** be a valid combination of VkStencilFaceFlagBits values
- VUID-vkCmdSetStencilReference-faceMask-requiredbitmask
  `faceMask` **must** not be `0`
- VUID-vkCmdSetStencilReference-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetStencilReference-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetStencilReference-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Possible values of the `failOp`, `passOp`, and `depthFailOp` members of VkStencilOpState, specifying what happens to the stored stencil value if this or certain subsequent tests fail or pass, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkStencilOp {
    VK_STENCIL_OP_KEEP = 0,
    VK_STENCIL_OP_ZERO = 1,
    VK_STENCIL_OP_REPLACE = 2,
    VK_STENCIL_OP_INCREMENT_AND_CLAMP = 3,
    VK_STENCIL_OP_DECREMENT_AND_CLAMP = 4,
    VK_STENCIL_OP_INVERT = 5,
    VK_STENCIL_OP_INCREMENT_AND_WRAP = 6,
    VK_STENCIL_OP_DECREMENT_AND_WRAP = 7,
} VkStencilOp;
```

- `VK_STENCIL_OP_KEEP` keeps the current value.
- `VK_STENCIL_OP_ZERO` sets the value to 0.
- `VK_STENCIL_OP_REPLACE` sets the value to `reference`.
- `VK_STENCIL_OP_INCREMENT_AND_CLAMP` increments the current value and clamps to the maximum representable unsigned value.
- `VK_STENCIL_OP_DECREMENT_AND_CLAMP` decrements the current value and clamps to 0.
- `VK_STENCIL_OP_INVERT` bitwise-inverts the current value.
- `VK_STENCIL_OP_INCREMENT_AND_WRAP` increments the current value and wraps to 0 when the maximum value would have been exceeded.
- `VK_STENCIL_OP_DECREMENT_AND_WRAP` decrements the current value and wraps to the maximum possible value when the value would go below 0.

For purposes of increment and decrement, the stencil bits are considered as an unsigned integer.

27.9. Depth Test

The depth test compares the depth value z_a in the depth/stencil attachment at each sample's framebuffer coordinates (x_f,y_f) and sample index i against the sample's depth value z_f. If there is no depth attachment then the depth test is skipped.

If the render pass has a fragment density map attachment and the fragment covers multiple pixels, there is an implementation-dependent association of rasterization samples to depth attachment samples within the fragment. However, if all samples in the fragment are covered, and the depth attachment value is updated as a result of this test, all depth attachment samples will be updated.

The depth test occurs in three stages, as detailed in the following sections.

27.9.1. Depth Clamping and Range Adjustment

If VkPipelineRasterizationStateCreateInfo::depthClampEnable is enabled, z_f is clamped to [z_{min}, z_{max}], where z_{min} = min(n,f), z_{max} = max(n,f), and n and f are the minDepth and maxDepth depth range values of the viewport used by this fragment, respectively. If the depthClampControl feature is enabled and VkPipelineViewportDepthClampControlCreateInfoEXT::depthClampMode is VK_DEPTH_CLAMP_MODE_USER_DEFINED_RANGE_EXT, then z_{min} and z_{max} are equal to the minDepthClamp and maxDepthClamp values of the VkDepthClampRangeEXT structure pointed to by pDepthClampRange.

Following depth clamping:

- If z_f is not in the range [z_{min}, z_{max}], then z_f is undefined following this step.
- If the depth attachment has a fixed-point format and z_f is not in the range [0, 1], then z_f is undefined following this step.

The VkPipelineViewportDepthClampControlCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_depth_clamp_control
typedef struct VkPipelineViewportDepthClampControlCreateInfoEXT {
    VkStructureType                sType;
    const void*                    pNext;
    VkDepthClampModeEXT            depthClampMode;
    const VkDepthClampRangeEXT*    pDepthClampRange;
} VkPipelineViewportDepthClampControlCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- depthClampMode determines how the clamp range is determined for each viewport.
- pDepthClampRange sets the depth clamp range for all viewports if depthClampMode is VK_DEPTH_CLAMP_MODE_USER_DEFINED_RANGE_EXT.

This structure extends VkPipelineViewportStateCreateInfo and specifies the depth clamp range used in the pipeline. If this structure is not provided in the next chain then depthClampMode defaults to VK_DEPTH_CLAMP_MODE_VIEWPORT_RANGE_EXT.

Valid Usage

- VUID-VkPipelineViewportDepthClampControlCreateInfoEXT-pDepthClampRange-09646
  If `depthClampMode` is `VK_DEPTH_CLAMP_MODE_USER_DEFINED_RANGE_EXT`, and the pipeline is not created with `VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT`, then `pDepthClampRange` **must** be a valid pointer to a valid `VkDepthClampRangeEXT` structure

Valid Usage (Implicit)

- VUID-VkPipelineViewportDepthClampControlCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_DEPTH_CLAMP_CONTROL_CREATE_INFO_EXT`
- VUID-VkPipelineViewportDepthClampControlCreateInfoEXT-depthClampMode-parameter
  `depthClampMode` **must** be a valid VkDepthClampModeEXT value
- VUID-VkPipelineViewportDepthClampControlCreateInfoEXT-pDepthClampRange-parameter
  If `pDepthClampRange` is not `NULL`, `pDepthClampRange` **must** be a valid pointer to a valid VkDepthClampRangeEXT structure

Possible values of VkPipelineViewportDepthClampControlCreateInfoEXT::`depthClampMode`, specifying which range should be used for depth clamping, are:

```
// Provided by VK_EXT_depth_clamp_control
typedef enum VkDepthClampModeEXT {
    VK_DEPTH_CLAMP_MODE_VIEWPORT_RANGE_EXT = 0,
    VK_DEPTH_CLAMP_MODE_USER_DEFINED_RANGE_EXT = 1,
} VkDepthClampModeEXT;
```

- `VK_DEPTH_CLAMP_MODE_VIEWPORT_RANGE_EXT` specifies that the depth clamp range follows the viewport depth range. The depth clamp range of each viewport will implicitly be set to z_{min} = min(n,f) and z_{max} = max(n,f), where n and f are the `minDepth` and `maxDepth` depth range values of the viewport.
- `VK_DEPTH_CLAMP_MODE_USER_DEFINED_RANGE_EXT` specifies that a single user-defined depth clamp range will be used for all viewports. The user-defined depth clamp range is defined by the `minDepthClamp` and `maxDepthClamp` members of VkDepthClampRangeEXT.

To dynamically set the viewport depth clamp range parameters, call:

```
// Provided by VK_EXT_depth_clamp_control, VK_EXT_depth_clamp_control with
VK_EXT_shader_object
void vkCmdSetDepthClampRangeEXT(
    VkCommandBuffer                             commandBuffer,
    VkDepthClampModeEXT                         depthClampMode,
    const VkDepthClampRangeEXT*                 pDepthClampRange);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `depthClampMode` determines how the clamp range is determined for each viewport.
- `pDepthClampRange` sets the depth clamp range for all viewports if `depthClampMode` is `VK_DEPTH_CLAMP_MODE_USER_DEFINED_RANGE_EXT`.

This command sets the viewport depth clamp range for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineViewportDepthClampControlCreateInfoEXT ::`depthClampMode` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDepthClampRangeEXT-pDepthClampRange-09647
  If `depthClampMode` is `VK_DEPTH_CLAMP_MODE_USER_DEFINED_RANGE_EXT`, then `pDepthClampRange` must be a valid pointer to a valid `VkDepthClampRangeEXT` structure

Valid Usage (Implicit)

- VUID-vkCmdSetDepthClampRangeEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthClampRangeEXT-depthClampMode-parameter
  `depthClampMode` **must** be a valid VkDepthClampModeEXT value
- VUID-vkCmdSetDepthClampRangeEXT-pDepthClampRange-parameter
  If `pDepthClampRange` is not `NULL`, `pDepthClampRange` **must** be a valid pointer to a valid VkDepthClampRangeEXT structure
- VUID-vkCmdSetDepthClampRangeEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthClampRangeEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthClampRangeEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Both VkPipelineViewportDepthClampControlCreateInfoEXT and vkCmdSetDepthClampRangeEXT use VkDepthClampRangeEXT to set the viewport depth clamp range parameters.

The VkDepthClampRangeEXT structure is defined as:

```
// Provided by VK_EXT_depth_clamp_control
typedef struct VkDepthClampRangeEXT {
    float    minDepthClamp;
    float    maxDepthClamp;
} VkDepthClampRangeEXT;
```

- minDepthClamp sets z_{min} in the depth clamp range of the viewport.
- maxDepthClamp sets z_{max} in the depth clamp range of the viewport.

Valid Usage

- VUID-VkDepthClampRangeEXT-pDepthClampRange-00999
  minDepthClamp **must** be less than or equal to maxDepthClamp
- VUID-VkDepthClampRangeEXT-pDepthClampRange-09648
  If the VK_EXT_depth_range_unrestricted extension is not enabled, minDepthClamp **must** be greater than or equal to 0.0
- VUID-VkDepthClampRangeEXT-pDepthClampRange-09649
  If the VK_EXT_depth_range_unrestricted extension is not enabled, maxDepthClamp **must** be less than or equal to 1.0

27.9.2. Depth Comparison

If the depth test is not enabled, as specified by vkCmdSetDepthTestEnableEXT or VkPipelineDepthStencilStateCreateInfo::depthTestEnable, then this step is skipped.

The comparison operation performed is determined by the VkCompareOp value set by vkCmdSetDepthCompareOpEXT, or by VkPipelineDepthStencilStateCreateInfo::depthCompareOp during pipeline creation. z_f and z_a are used as the *reference* and *test* values, respectively, in the operation specified by the VkCompareOp.

If the comparison evaluates to false, the coverage for the sample is set to 0.

27.9.3. Depth Attachment Writes

If depth writes are enabled, as specified by vkCmdSetDepthWriteEnableEXT or VkPipelineDepthStencilStateCreateInfo::depthWriteEnable, and the comparison evaluated to true, the depth attachment value z_a is set to the sample's depth value z_f. If there is no depth attachment, no value is written.

To dynamically enable or disable the depth test, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetDepthTestEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    depthTestEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `depthTestEnable` specifies if the depth test is enabled.

This command sets the depth test enable for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineDepthStencilStateCreateInfo ::depthTestEnable value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDepthTestEnable-None-08971
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetDepthTestEnable-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthTestEnable-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthTestEnable-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthTestEnable-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally

synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the depth compare operator, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetDepthCompareOpEXT(
    VkCommandBuffer                             commandBuffer,
    VkCompareOp                                 depthCompareOp);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `depthCompareOp` is a VkCompareOp value specifying the comparison operator used for the Depth Comparison step of the depth test.

This command sets the depth comparison operator for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineDepthStencilStateCreateInfo::`depthCompareOp` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDepthCompareOp-None-08971
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetDepthCompareOp-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetDepthCompareOp-depthCompareOp-parameter
  `depthCompareOp` **must** be a valid VkCompareOp value
- VUID-vkCmdSetDepthCompareOp-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthCompareOp-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics

operations

- VUID-vkCmdSetDepthCompareOp-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the depth write enable, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetDepthWriteEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    depthWriteEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `depthWriteEnable` specifies if depth writes are enabled.

This command sets the depth write enable for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineDepthStencilStateCreateInfo ::`depthWriteEnable` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetDepthWriteEnable-None-08971
  At least one of the following **must** be true:
  - the `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetDepthWriteEnable-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle

- VUID-vkCmdSetDepthWriteEnable-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetDepthWriteEnable-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetDepthWriteEnable-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

27.10. Sample Counting

Occlusion queries use query pool entries to track the number of samples that pass all the per-fragment tests. The mechanism of collecting an occlusion query value is described in Occlusion Queries.

The occlusion query sample counter increments by one for each sample with a coverage value of 1 in each fragment that survives all the per-fragment tests, including scissor, sample mask, alpha to coverage, stencil, and depth tests.

27.11. Coverage Reduction

Coverage reduction takes the coverage information for a fragment and converts that to a boolean coverage value for each color sample in each pixel covered by the fragment.

27.11.1. Pixel Coverage

Coverage for each pixel is first extracted from the total fragment coverage mask. This consists of `rasterizationSamples` unique coverage samples for each pixel in the fragment area, each with a unique sample index. If the fragment only contains a single pixel, coverage for the pixel is equivalent to the fragment coverage.

If the render pass has a fragment density map attachment and the fragment covers multiple pixels, pixel coverage is generated in an implementation-dependent manner. If all samples in the fragment are covered, all samples will be covered in each pixel coverage.

If the fragment shading rate is set, and the fragment covers multiple pixels, each pixel's coverage consists of the coverage samples with a pixel index matching that pixel, and each sample retains its unique sample index i.

27.11.2. Color Sample Coverage

Once pixel coverage is determined, coverage for each individual color sample corresponding to that pixel is determined.

The number of `rasterizationSamples` is identical to the number of samples in the color attachments. A color sample is covered if the pixel coverage sample with the same sample index i is covered.

Chapter 28. The Framebuffer

28.1. Blending

Blending combines the incoming *source* fragment's R, G, B, and A values with the *destination* R, G, B, and A values of each sample stored in the framebuffer at the fragment's (x_f,y_f) location. Blending is performed for each color sample covered by the fragment, rather than just once for each fragment.

Source and destination values are combined according to the blend operation, quadruplets of source and destination weighting factors determined by the blend factors, and a blend constant, to obtain a new set of R, G, B, and A values, as described below.

Blending is computed and applied separately to each color attachment used by the subpass, with separate controls for each attachment.

Prior to performing the blend operation, signed and unsigned normalized fixed-point color components undergo an implied conversion to floating-point as specified by Conversion from Normalized Fixed-Point to Floating-Point. Blending computations are treated as if carried out in floating-point, and basic blend operations are performed with a precision and dynamic range no lower than that used to represent destination components. Advanced blending operations are performed with a precision and dynamic range no lower than the smaller of that used to represent destination components or that used to represent 16-bit floating-point values.

NOTE

Blending is only defined for floating-point, UNORM, SNORM, and sRGB formats. Within those formats, the implementation may only support blending on some subset of them. Which formats support blending is indicated by `VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT`.

The pipeline blend state is included in the `VkPipelineColorBlendStateCreateInfo` structure during graphics pipeline creation:

The `VkPipelineColorBlendStateCreateInfo` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineColorBlendStateCreateInfo {
    VkStructureType                               sType;
    const void*                                   pNext;
    VkPipelineColorBlendStateCreateFlags          flags;
    VkBool32                                      logicOpEnable;
    VkLogicOp                                     logicOp;
    uint32_t                                      attachmentCount;
    const VkPipelineColorBlendAttachmentState*    pAttachments;
    float                                         blendConstants[4];
} VkPipelineColorBlendStateCreateInfo;
```

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.

- `flags` is reserved for future use.
- `logicOpEnable` controls whether to apply Logical Operations.
- `logicOp` selects which logical operation to apply.
- `attachmentCount` is the number of VkPipelineColorBlendAttachmentState elements in `pAttachments`. It is ignored if the pipeline is created with `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT`, `VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT`, and `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT` dynamic states set, and either `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` set or the advancedBlendCoherentOperations feature is not enabled.
- `pAttachments` is a pointer to an array of VkPipelineColorBlendAttachmentState structures defining blend state for each color attachment. It is ignored if the pipeline is created with `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT`, `VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT`, and `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT` dynamic states set, and either `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT` set or the advancedBlendCoherentOperations feature is not enabled.
- `blendConstants` is a pointer to an array of four values used as the R, G, B, and A components of the blend constant that are used in blending, depending on the blend factor.

Valid Usage

- VUID-VkPipelineColorBlendStateCreateInfo-pAttachments-00605
  If the `independentBlend` feature is not enabled, all elements of `pAttachments` **must** be identical
- VUID-VkPipelineColorBlendStateCreateInfo-logicOpEnable-00606
  If the `logicOp` feature is not enabled, `logicOpEnable` **must** be `VK_FALSE`
- VUID-VkPipelineColorBlendStateCreateInfo-logicOpEnable-00607
  If `logicOpEnable` is `VK_TRUE`, `logicOp` **must** be a valid VkLogicOp value
- VUID-VkPipelineColorBlendStateCreateInfo-pAttachments-07353
  If `attachmentCount` is not `0` , and any of `VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT`, `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT`, `VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT`, or `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT` are not set, `pAttachments` **must** be a valid pointer to an array of `attachmentCount` valid VkPipelineColorBlendAttachmentState structures

Valid Usage (Implicit)

- VUID-VkPipelineColorBlendStateCreateInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO`
- VUID-VkPipelineColorBlendStateCreateInfo-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkPipelineColorBlendAdvancedStateCreateInfoEXT or VkPipelineColorWriteCreateInfoEXT
- VUID-VkPipelineColorBlendStateCreateInfo-sType-unique

The `sType` value of each struct in the `pNext` chain **must** be unique

- VUID-VkPipelineColorBlendStateCreateInfo-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkPipelineColorBlendStateCreateInfo-pAttachments-parameter
  If `attachmentCount` is not `0`, and `pAttachments` is not `NULL`, `pAttachments` **must** be a valid pointer to an array of `attachmentCount` valid VkPipelineColorBlendAttachmentState structures

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkPipelineColorBlendStateCreateFlags;
```

`VkPipelineColorBlendStateCreateFlags` is a bitmask type for setting a mask, but is currently reserved for future use.

The `VkPipelineColorBlendAttachmentState` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPipelineColorBlendAttachmentState {
    VkBool32                 blendEnable;
    VkBlendFactor            srcColorBlendFactor;
    VkBlendFactor            dstColorBlendFactor;
    VkBlendOp                colorBlendOp;
    VkBlendFactor            srcAlphaBlendFactor;
    VkBlendFactor            dstAlphaBlendFactor;
    VkBlendOp                alphaBlendOp;
    VkColorComponentFlags    colorWriteMask;
} VkPipelineColorBlendAttachmentState;
```

- `blendEnable` controls whether blending is enabled for the corresponding color attachment. If blending is not enabled, the source fragment's color for that attachment is passed through unmodified.
- `srcColorBlendFactor` selects which blend factor is used to determine the source factors (S_r,S_g,S_b).
- `dstColorBlendFactor` selects which blend factor is used to determine the destination factors (D_r,D_g,D_b).
- `colorBlendOp` selects which blend operation is used to calculate the RGB values to write to the color attachment.
- `srcAlphaBlendFactor` selects which blend factor is used to determine the source factor S_a.
- `dstAlphaBlendFactor` selects which blend factor is used to determine the destination factor D_a.
- `alphaBlendOp` selects which blend operation is used to calculate the alpha values to write to the color attachment.
- `colorWriteMask` is a bitmask of VkColorComponentFlagBits specifying which of the R, G, B, and/or A components are enabled for writing, as described for the Color Write Mask.

Valid Usage

- VUID-VkPipelineColorBlendAttachmentState-srcColorBlendFactor-00608
  If the dualSrcBlend feature is not enabled, srcColorBlendFactor **must** not be VK_BLEND_FACTOR_SRC1_COLOR, VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR, VK_BLEND_FACTOR_SRC1_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA
- VUID-VkPipelineColorBlendAttachmentState-dstColorBlendFactor-00609
  If the dualSrcBlend feature is not enabled, dstColorBlendFactor **must** not be VK_BLEND_FACTOR_SRC1_COLOR, VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR, VK_BLEND_FACTOR_SRC1_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA
- VUID-VkPipelineColorBlendAttachmentState-srcAlphaBlendFactor-00610
  If the dualSrcBlend feature is not enabled, srcAlphaBlendFactor **must** not be VK_BLEND_FACTOR_SRC1_COLOR, VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR, VK_BLEND_FACTOR_SRC1_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA
- VUID-VkPipelineColorBlendAttachmentState-dstAlphaBlendFactor-00611
  If the dualSrcBlend feature is not enabled, dstAlphaBlendFactor **must** not be VK_BLEND_FACTOR_SRC1_COLOR, VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR, VK_BLEND_FACTOR_SRC1_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA
- VUID-VkPipelineColorBlendAttachmentState-colorBlendOp-01406
  If either of colorBlendOp or alphaBlendOp is an advanced blend operation, then colorBlendOp **must** equal alphaBlendOp
- VUID-VkPipelineColorBlendAttachmentState-advancedBlendIndependentBlend-01407
  If VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT::advancedBlendIndependentBlend is VK_FALSE and colorBlendOp is an advanced blend operation, then colorBlendOp **must** be the same for all attachments
- VUID-VkPipelineColorBlendAttachmentState-advancedBlendIndependentBlend-01408
  If VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT::advancedBlendIndependentBlend is VK_FALSE and alphaBlendOp is an advanced blend operation, then alphaBlendOp **must** be the same for all attachments
- VUID-VkPipelineColorBlendAttachmentState-advancedBlendAllOperations-01409
  If VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT::advancedBlendAllOperations is VK_FALSE, then colorBlendOp **must** not be VK_BLEND_OP_ZERO_EXT, VK_BLEND_OP_SRC_EXT, VK_BLEND_OP_DST_EXT, VK_BLEND_OP_SRC_OVER_EXT, VK_BLEND_OP_DST_OVER_EXT, VK_BLEND_OP_SRC_IN_EXT, VK_BLEND_OP_DST_IN_EXT, VK_BLEND_OP_SRC_OUT_EXT, VK_BLEND_OP_DST_OUT_EXT, VK_BLEND_OP_SRC_ATOP_EXT, VK_BLEND_OP_DST_ATOP_EXT, VK_BLEND_OP_XOR_EXT, VK_BLEND_OP_INVERT_EXT, VK_BLEND_OP_INVERT_RGB_EXT, VK_BLEND_OP_LINEARDODGE_EXT, VK_BLEND_OP_LINEARBURN_EXT, VK_BLEND_OP_VIVIDLIGHT_EXT, VK_BLEND_OP_LINEARLIGHT_EXT, VK_BLEND_OP_PINLIGHT_EXT, VK_BLEND_OP_HARDMIX_EXT, VK_BLEND_OP_PLUS_EXT, VK_BLEND_OP_PLUS_CLAMPED_EXT, VK_BLEND_OP_PLUS_CLAMPED_ALPHA_EXT, VK_BLEND_OP_PLUS_DARKER_EXT, VK_BLEND_OP_MINUS_EXT, VK_BLEND_OP_MINUS_CLAMPED_EXT, VK_BLEND_OP_CONTRAST_EXT, VK_BLEND_OP_INVERT_OVG_EXT, VK_BLEND_OP_RED_EXT, VK_BLEND_OP_GREEN_EXT, or VK_BLEND_OP_BLUE_EXT
- VUID-VkPipelineColorBlendAttachmentState-colorBlendOp-01410
  If colorBlendOp or alphaBlendOp is an advanced blend operation, then colorAttachmentCount

of the subpass this pipeline is compiled against **must** be less than or equal to VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT::advancedBlendMaxColorAttachments

- VUID-VkPipelineColorBlendAttachmentState-constantAlphaColorBlendFactors-04454
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::constantAlphaColorBlendFactors is VK_FALSE, srcColorBlendFactor **must** not be VK_BLEND_FACTOR_CONSTANT_ALPHA or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA
- VUID-VkPipelineColorBlendAttachmentState-constantAlphaColorBlendFactors-04455
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::constantAlphaColorBlendFactors is VK_FALSE, dstColorBlendFactor **must** not be VK_BLEND_FACTOR_CONSTANT_ALPHA or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA

Valid Usage (Implicit)

- VUID-VkPipelineColorBlendAttachmentState-srcColorBlendFactor-parameter
  srcColorBlendFactor **must** be a valid VkBlendFactor value
- VUID-VkPipelineColorBlendAttachmentState-dstColorBlendFactor-parameter
  dstColorBlendFactor **must** be a valid VkBlendFactor value
- VUID-VkPipelineColorBlendAttachmentState-colorBlendOp-parameter
  colorBlendOp **must** be a valid VkBlendOp value
- VUID-VkPipelineColorBlendAttachmentState-srcAlphaBlendFactor-parameter
  srcAlphaBlendFactor **must** be a valid VkBlendFactor value
- VUID-VkPipelineColorBlendAttachmentState-dstAlphaBlendFactor-parameter
  dstAlphaBlendFactor **must** be a valid VkBlendFactor value
- VUID-VkPipelineColorBlendAttachmentState-alphaBlendOp-parameter
  alphaBlendOp **must** be a valid VkBlendOp value
- VUID-VkPipelineColorBlendAttachmentState-colorWriteMask-parameter
  colorWriteMask **must** be a valid combination of VkColorComponentFlagBits values

To dynamically set blendEnable, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetColorBlendEnableEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstAttachment,
    uint32_t                                    attachmentCount,
    const VkBool32*                             pColorBlendEnables);
```

- commandBuffer is the command buffer into which the command will be recorded.
- firstAttachment the first color attachment the color blending enable applies.

- `attachmentCount` the number of color blending enables in the `pColorBlendEnables` array.
- `pColorBlendEnables` an array of booleans to indicate whether color blending is enabled for the corresponding attachment.

This command sets the color blending enable of the specified color attachments for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT` set in VkPipelineDynamicStateCreateInfo ::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineColorBlendAttachmentState ::`blendEnable` values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetColorBlendEnableEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3ColorBlendEnable` feature is enabled
  - The `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetColorBlendEnableEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetColorBlendEnableEXT-pColorBlendEnables-parameter
  `pColorBlendEnables` **must** be a valid pointer to an array of `attachmentCount` VkBool32 values
- VUID-vkCmdSetColorBlendEnableEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetColorBlendEnableEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetColorBlendEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetColorBlendEnableEXT-attachmentCount-arraylength
  `attachmentCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set color blend factors and operations, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetColorBlendEquationEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstAttachment,
    uint32_t                                    attachmentCount,
    const VkColorBlendEquationEXT*              pColorBlendEquations);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `firstAttachment` the first color attachment the color blend factors and operations apply to.
- `attachmentCount` the number of VkColorBlendEquationEXT elements in the `pColorBlendEquations` array.
- `pColorBlendEquations` an array of VkColorBlendEquationEXT structs that specify the color blend factors and operations for the corresponding attachments.

This command sets the color blending factors and operations of the specified attachments for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineColorBlendAttachmentState::`srcColorBlendFactor`, VkPipelineColorBlendAttachmentState::`dstColorBlendFactor`, VkPipelineColorBlendAttachmentState::`colorBlendOp`, VkPipelineColorBlendAttachmentState::`srcAlphaBlendFactor`, VkPipelineColorBlendAttachmentState::`dstAlphaBlendFactor`, and VkPipelineColorBlendAttachmentState::`alphaBlendOp` values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetColorBlendEquationEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3ColorBlendEquation` feature is enabled
  - The `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetColorBlendEquationEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetColorBlendEquationEXT-pColorBlendEquations-parameter
  `pColorBlendEquations` **must** be a valid pointer to an array of `attachmentCount` valid VkColorBlendEquationEXT structures
- VUID-vkCmdSetColorBlendEquationEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetColorBlendEquationEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetColorBlendEquationEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetColorBlendEquationEXT-attachmentCount-arraylength
  `attachmentCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

The `VkColorBlendEquationEXT` structure is defined as:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
typedef struct VkColorBlendEquationEXT {
    VkBlendFactor    srcColorBlendFactor;
    VkBlendFactor    dstColorBlendFactor;
    VkBlendOp        colorBlendOp;
    VkBlendFactor    srcAlphaBlendFactor;
    VkBlendFactor    dstAlphaBlendFactor;
    VkBlendOp        alphaBlendOp;
} VkColorBlendEquationEXT;
```

- `srcColorBlendFactor` selects which blend factor is used to determine the source factors (S_r,S_g,S_b).
- `dstColorBlendFactor` selects which blend factor is used to determine the destination factors (D_r,D_g,D_b).
- `colorBlendOp` selects which blend operation is used to calculate the RGB values to write to the color attachment.
- `srcAlphaBlendFactor` selects which blend factor is used to determine the source factor S_a.
- `dstAlphaBlendFactor` selects which blend factor is used to determine the destination factor D_a.
- `alphaBlendOp` selects which blend operation is use to calculate the alpha values to write to the color attachment.

Valid Usage

- VUID-VkColorBlendEquationEXT-dualSrcBlend-07357
  If the `dualSrcBlend` feature is not enabled, `srcColorBlendFactor` **must** not be `VK_BLEND_FACTOR_SRC1_COLOR`, `VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR`, `VK_BLEND_FACTOR_SRC1_ALPHA`, or `VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA`
- VUID-VkColorBlendEquationEXT-dualSrcBlend-07358
  If the `dualSrcBlend` feature is not enabled, `dstColorBlendFactor` **must** not be `VK_BLEND_FACTOR_SRC1_COLOR`, `VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR`, `VK_BLEND_FACTOR_SRC1_ALPHA`, or `VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA`
- VUID-VkColorBlendEquationEXT-dualSrcBlend-07359
  If the `dualSrcBlend` feature is not enabled, `srcAlphaBlendFactor` **must** not be `VK_BLEND_FACTOR_SRC1_COLOR`, `VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR`, `VK_BLEND_FACTOR_SRC1_ALPHA`, or `VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA`
- VUID-VkColorBlendEquationEXT-dualSrcBlend-07360
  If the `dualSrcBlend` feature is not enabled, `dstAlphaBlendFactor` **must** not be `VK_BLEND_FACTOR_SRC1_COLOR`, `VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR`, `VK_BLEND_FACTOR_SRC1_ALPHA`, or `VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA`
- VUID-VkColorBlendEquationEXT-colorBlendOp-07361
  `colorBlendOp` and `alphaBlendOp` **must** not be `VK_BLEND_OP_ZERO_EXT`, `VK_BLEND_OP_SRC_EXT`, `VK_BLEND_OP_DST_EXT`, `VK_BLEND_OP_SRC_OVER_EXT`, `VK_BLEND_OP_DST_OVER_EXT`, `VK_BLEND_OP_SRC_IN_EXT`, `VK_BLEND_OP_DST_IN_EXT`, `VK_BLEND_OP_SRC_OUT_EXT`, `VK_BLEND_OP_DST_OUT_EXT`, `VK_BLEND_OP_SRC_ATOP_EXT`, `VK_BLEND_OP_DST_ATOP_EXT`, `VK_BLEND_OP_XOR_EXT`, `VK_BLEND_OP_MULTIPLY_EXT`, `VK_BLEND_OP_SCREEN_EXT`, `VK_BLEND_OP_OVERLAY_EXT`, `VK_BLEND_OP_DARKEN_EXT`, `VK_BLEND_OP_LIGHTEN_EXT`, `VK_BLEND_OP_COLORDODGE_EXT`, `VK_BLEND_OP_COLORBURN_EXT`, `VK_BLEND_OP_HARDLIGHT_EXT`, `VK_BLEND_OP_SOFTLIGHT_EXT`, `VK_BLEND_OP_DIFFERENCE_EXT`, `VK_BLEND_OP_EXCLUSION_EXT`, `VK_BLEND_OP_INVERT_EXT`, `VK_BLEND_OP_INVERT_RGB_EXT`, `VK_BLEND_OP_LINEARDODGE_EXT`, `VK_BLEND_OP_LINEARBURN_EXT`, `VK_BLEND_OP_VIVIDLIGHT_EXT`, `VK_BLEND_OP_LINEARLIGHT_EXT`, `VK_BLEND_OP_PINLIGHT_EXT`, `VK_BLEND_OP_HARDMIX_EXT`, `VK_BLEND_OP_HSL_HUE_EXT`, `VK_BLEND_OP_HSL_SATURATION_EXT`, `VK_BLEND_OP_HSL_COLOR_EXT`, `VK_BLEND_OP_HSL_LUMINOSITY_EXT`, `VK_BLEND_OP_PLUS_EXT`, `VK_BLEND_OP_PLUS_CLAMPED_EXT`, `VK_BLEND_OP_PLUS_CLAMPED_ALPHA_EXT`, `VK_BLEND_OP_PLUS_DARKER_EXT`, `VK_BLEND_OP_MINUS_EXT`, `VK_BLEND_OP_MINUS_CLAMPED_EXT`, `VK_BLEND_OP_CONTRAST_EXT`, `VK_BLEND_OP_INVERT_OVG_EXT`,

VK_BLEND_OP_RED_EXT, VK_BLEND_OP_GREEN_EXT, or VK_BLEND_OP_BLUE_EXT

- VUID-VkColorBlendEquationEXT-constantAlphaColorBlendFactors-07362
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::constantAlphaColorBlendFactors is VK_FALSE, srcColorBlendFactor **must** not be VK_BLEND_FACTOR_CONSTANT_ALPHA or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA
- VUID-VkColorBlendEquationEXT-constantAlphaColorBlendFactors-07363
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::constantAlphaColorBlendFactors is VK_FALSE, dstColorBlendFactor **must** not be VK_BLEND_FACTOR_CONSTANT_ALPHA or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA

Valid Usage (Implicit)

- VUID-VkColorBlendEquationEXT-srcColorBlendFactor-parameter
  srcColorBlendFactor **must** be a valid VkBlendFactor value
- VUID-VkColorBlendEquationEXT-dstColorBlendFactor-parameter
  dstColorBlendFactor **must** be a valid VkBlendFactor value
- VUID-VkColorBlendEquationEXT-colorBlendOp-parameter
  colorBlendOp **must** be a valid VkBlendOp value
- VUID-VkColorBlendEquationEXT-srcAlphaBlendFactor-parameter
  srcAlphaBlendFactor **must** be a valid VkBlendFactor value
- VUID-VkColorBlendEquationEXT-dstAlphaBlendFactor-parameter
  dstAlphaBlendFactor **must** be a valid VkBlendFactor value
- VUID-VkColorBlendEquationEXT-alphaBlendOp-parameter
  alphaBlendOp **must** be a valid VkBlendOp value

To dynamically set the color write masks, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetColorWriteMaskEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstAttachment,
    uint32_t                                    attachmentCount,
    const VkColorComponentFlags*                pColorWriteMasks);
```

- commandBuffer is the command buffer into which the command will be recorded.
- firstAttachment the first color attachment the color write masks apply to.
- attachmentCount the number of VkColorComponentFlags values in the pColorWriteMasks array.
- pColorWriteMasks an array of VkColorComponentFlags values that specify the color write masks of the corresponding attachments.

This command sets the color write masks of the specified attachments for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineColorBlendAttachmentState::colorWriteMask values used to create the currently active pipeline.

NOTE	Formats with bits that are shared between components specified by VkColorComponentFlagBits, such as VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, cannot have their channels individually masked by this functionality; either all components that share bits have to be enabled, or none of them.

Valid Usage

- VUID-vkCmdSetColorWriteMaskEXT-None-09423
  At least one of the following **must** be true:
  - The extendedDynamicState3ColorWriteMask feature is enabled
  - The shaderObject feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetColorWriteMaskEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetColorWriteMaskEXT-pColorWriteMasks-parameter
  pColorWriteMasks **must** be a valid pointer to an array of attachmentCount valid combinations of VkColorComponentFlagBits values
- VUID-vkCmdSetColorWriteMaskEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdSetColorWriteMaskEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support graphics operations
- VUID-vkCmdSetColorWriteMaskEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetColorWriteMaskEXT-attachmentCount-arraylength
  attachmentCount **must** be greater than 0

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

28.1.1. Blend Factors

The source and destination color and alpha blending factors are selected from the enum:

```
// Provided by VK_VERSION_1_0
typedef enum VkBlendFactor {
    VK_BLEND_FACTOR_ZERO = 0,
    VK_BLEND_FACTOR_ONE = 1,
    VK_BLEND_FACTOR_SRC_COLOR = 2,
    VK_BLEND_FACTOR_ONE_MINUS_SRC_COLOR = 3,
    VK_BLEND_FACTOR_DST_COLOR = 4,
    VK_BLEND_FACTOR_ONE_MINUS_DST_COLOR = 5,
    VK_BLEND_FACTOR_SRC_ALPHA = 6,
    VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA = 7,
    VK_BLEND_FACTOR_DST_ALPHA = 8,
    VK_BLEND_FACTOR_ONE_MINUS_DST_ALPHA = 9,
    VK_BLEND_FACTOR_CONSTANT_COLOR = 10,
    VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR = 11,
    VK_BLEND_FACTOR_CONSTANT_ALPHA = 12,
    VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA = 13,
    VK_BLEND_FACTOR_SRC_ALPHA_SATURATE = 14,
    VK_BLEND_FACTOR_SRC1_COLOR = 15,
    VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR = 16,
    VK_BLEND_FACTOR_SRC1_ALPHA = 17,
    VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA = 18,
} VkBlendFactor;
```

The semantics of the enum values are described in the table below:

Table 34. Blend Factors

VkBlendFactor	RGB Blend Factors (S_r,S_g,S_b) or (D_r,D_g,D_b)	Alpha Blend Factor (S_a or D_a)
VK_BLEND_FACTOR_ZERO	(0,0,0)	0
VK_BLEND_FACTOR_ONE	(1,1,1)	1
VK_BLEND_FACTOR_SRC_COLOR	(R_{s0},G_{s0},B_{s0})	A_{s0}

VkBlendFactor	RGB Blend Factors (S_r,S_g,S_b) or (D_r,D_g,D_b)	Alpha Blend Factor (S_a or D_a)
VK_BLEND_FACTOR_ONE_MINUS_SRC_COLOR	$(1-R_{s0},1-G_{s0},1-B_{s0})$	$1-A_{s0}$
VK_BLEND_FACTOR_DST_COLOR	(R_d,G_d,B_d)	A_d
VK_BLEND_FACTOR_ONE_MINUS_DST_COLOR	$(1-R_d,1-G_d,1-B_d)$	$1-A_d$
VK_BLEND_FACTOR_SRC_ALPHA	(A_{s0},A_{s0},A_{s0})	A_{s0}
VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA	$(1-A_{s0},1-A_{s0},1-A_{s0})$	$1-A_{s0}$
VK_BLEND_FACTOR_DST_ALPHA	(A_d,A_d,A_d)	A_d
VK_BLEND_FACTOR_ONE_MINUS_DST_ALPHA	$(1-A_d,1-A_d,1-A_d)$	$1-A_d$
VK_BLEND_FACTOR_CONSTANT_COLOR	(R_c,G_c,B_c)	A_c
VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR	$(1-R_c,1-G_c,1-B_c)$	$1-A_c$
VK_BLEND_FACTOR_CONSTANT_ALPHA	(A_c,A_c,A_c)	A_c
VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA	$(1-A_c,1-A_c,1-A_c)$	$1-A_c$
VK_BLEND_FACTOR_SRC_ALPHA_SATURATE	(f,f,f); $f = \min(A_{s0},1-A_d)$	1
VK_BLEND_FACTOR_SRC1_COLOR	(R_{s1},G_{s1},B_{s1})	A_{s1}
VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR	$(1-R_{s1},1-G_{s1},1-B_{s1})$	$1-A_{s1}$
VK_BLEND_FACTOR_SRC1_ALPHA	(A_{s1},A_{s1},A_{s1})	A_{s1}
VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA	$(1-A_{s1},1-A_{s1},1-A_{s1})$	$1-A_{s1}$

In this table, the following conventions are used:

- R_{s0},G_{s0},B_{s0} and A_{s0} represent the first source color R, G, B, and A components, respectively, for the fragment output location corresponding to the color attachment being blended.
- R_{s1},G_{s1},B_{s1} and A_{s1} represent the second source color R, G, B, and A components, respectively, used in dual source blending modes, for the fragment output location corresponding to the color attachment being blended.
- R_d,G_d,B_d and A_d represent the R, G, B, and A components of the destination color. That is, the color currently in the corresponding color attachment for this fragment/sample.
- R_c,G_c,B_c and A_c represent the blend constant R, G, B, and A components, respectively.

To dynamically set and change the blend constants, call:

```
// Provided by VK_VERSION_1_0
void vkCmdSetBlendConstants(
    VkCommandBuffer                             commandBuffer,
    const float                                 blendConstants[4]);
```

- commandBuffer is the command buffer into which the command will be recorded.

- `blendConstants` is a pointer to an array of four values specifying the R_c, G_c, B_c, and A_c components of the blend constant color used in blending, depending on the blend factor.

This command sets blend constants for subsequent drawing commands when when drawing using shader objects, or the graphics pipeline is created with `VK_DYNAMIC_STATE_BLEND_CONSTANTS` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineColorBlendStateCreateInfo::`blendConstants` values used to create the currently active pipeline.

Valid Usage (Implicit)

- VUID-vkCmdSetBlendConstants-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetBlendConstants-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetBlendConstants-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetBlendConstants-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

28.1.2. Dual-Source Blending

Blend factors that use the secondary color input (R_{s1},G_{s1},B_{s1},A_{s1}) (`VK_BLEND_FACTOR_SRC1_COLOR`, `VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR`, `VK_BLEND_FACTOR_SRC1_ALPHA`, and `VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA`) **may** consume implementation resources that could otherwise be used for rendering to multiple color attachments. Therefore, the number of color attachments that **can** be used in a framebuffer **may** be lower when using dual-source blending.

Dual-source blending is only supported if the `dualSrcBlend` feature is enabled.

The maximum number of color attachments that **can** be used in a subpass when using dual-source blending functions is implementation-dependent and is reported as the `maxFragmentDualSrcAttachments` member of `VkPhysicalDeviceLimits`.

Color outputs **can** be bound to the first and second inputs of the blender using the `Index` decoration, as described in Fragment Output Interface. If the second color input to the blender is not written in the shader, or if no output is bound to the second input of a blender, the value of the second input is undefined.

28.1.3. Blend Operations

Once the source and destination blend factors have been selected, they along with the source and destination components are passed to the blending operations. RGB and alpha components **can** use different operations. Possible values of VkBlendOp, specifying the operations, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkBlendOp {
    VK_BLEND_OP_ADD = 0,
    VK_BLEND_OP_SUBTRACT = 1,
    VK_BLEND_OP_REVERSE_SUBTRACT = 2,
    VK_BLEND_OP_MIN = 3,
    VK_BLEND_OP_MAX = 4,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_ZERO_EXT = 1000148000,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_SRC_EXT = 1000148001,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_DST_EXT = 1000148002,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_SRC_OVER_EXT = 1000148003,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_DST_OVER_EXT = 1000148004,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_SRC_IN_EXT = 1000148005,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_DST_IN_EXT = 1000148006,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_SRC_OUT_EXT = 1000148007,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_DST_OUT_EXT = 1000148008,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_SRC_ATOP_EXT = 1000148009,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_DST_ATOP_EXT = 1000148010,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_XOR_EXT = 1000148011,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_MULTIPLY_EXT = 1000148012,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_SCREEN_EXT = 1000148013,
```

```
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_OVERLAY_EXT = 1000148014,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_DARKEN_EXT = 1000148015,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_LIGHTEN_EXT = 1000148016,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_COLORDODGE_EXT = 1000148017,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_COLORBURN_EXT = 1000148018,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_HARDLIGHT_EXT = 1000148019,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_SOFTLIGHT_EXT = 1000148020,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_DIFFERENCE_EXT = 1000148021,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_EXCLUSION_EXT = 1000148022,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_INVERT_EXT = 1000148023,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_INVERT_RGB_EXT = 1000148024,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_LINEARDODGE_EXT = 1000148025,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_LINEARBURN_EXT = 1000148026,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_VIVIDLIGHT_EXT = 1000148027,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_LINEARLIGHT_EXT = 1000148028,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_PINLIGHT_EXT = 1000148029,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_HARDMIX_EXT = 1000148030,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_HSL_HUE_EXT = 1000148031,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_HSL_SATURATION_EXT = 1000148032,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_HSL_COLOR_EXT = 1000148033,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_HSL_LUMINOSITY_EXT = 1000148034,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_PLUS_EXT = 1000148035,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_PLUS_CLAMPED_EXT = 1000148036,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_PLUS_CLAMPED_ALPHA_EXT = 1000148037,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_PLUS_DARKER_EXT = 1000148038,
  // Provided by VK_EXT_blend_operation_advanced
```

```
    VK_BLEND_OP_MINUS_EXT = 1000148039,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_MINUS_CLAMPED_EXT = 1000148040,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_CONTRAST_EXT = 1000148041,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_INVERT_OVG_EXT = 1000148042,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_RED_EXT = 1000148043,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_GREEN_EXT = 1000148044,
  // Provided by VK_EXT_blend_operation_advanced
    VK_BLEND_OP_BLUE_EXT = 1000148045,
} VkBlendOp;
```

The semantics of the basic blend operations are described in the table below:

Table 35. Basic Blend Operations

VkBlendOp	RGB Components	Alpha Component
VK_BLEND_OP_ADD	$R = R_{s0} \times S_r + R_d \times D_r$ $G = G_{s0} \times S_g + G_d \times D_g$ $B = B_{s0} \times S_b + B_d \times D_b$	$A = A_{s0} \times S_a + A_d \times D_a$
VK_BLEND_OP_SUBTRACT	$R = R_{s0} \times S_r - R_d \times D_r$ $G = G_{s0} \times S_g - G_d \times D_g$ $B = B_{s0} \times S_b - B_d \times D_b$	$A = A_{s0} \times S_a - A_d \times D_a$
VK_BLEND_OP_REVERSE_SUBTRACT	$R = R_d \times D_r - R_{s0} \times S_r$ $G = G_d \times D_g - G_{s0} \times S_g$ $B = B_d \times D_b - B_{s0} \times S_b$	$A = A_d \times D_a - A_{s0} \times S_a$
VK_BLEND_OP_MIN	$R = \min(R_{s0},R_d)$ $G = \min(G_{s0},G_d)$ $B = \min(B_{s0},B_d)$	$A = \min(A_{s0},A_d)$
VK_BLEND_OP_MAX	$R = \max(R_{s0},R_d)$ $G = \max(G_{s0},G_d)$ $B = \max(B_{s0},B_d)$	$A = \max(A_{s0},A_d)$

In this table, the following conventions are used:

- R_{s0}, G_{s0}, B_{s0} and A_{s0} represent the first source color R, G, B, and A components, respectively.
- R_d, G_d, B_d and A_d represent the R, G, B, and A components of the destination color. That is, the color currently in the corresponding color attachment for this fragment/sample.
- S_r, S_g, S_b and S_a represent the source blend factor R, G, B, and A components, respectively.
- D_r, D_g, D_b and D_a represent the destination blend factor R, G, B, and A components, respectively.

The blending operation produces a new set of values R, G, B and A, which are written to the framebuffer attachment. If blending is not enabled for this attachment, then R, G, B and A are assigned R_{s0}, G_{s0}, B_{s0} and A_{s0}, respectively.

If the color attachment is fixed-point, the components of the source and destination values and blend factors are each clamped to [0,1] or [-1,1] respectively for an unsigned normalized or signed normalized color attachment prior to evaluating the blend operations. If the color attachment is floating-point, no clamping occurs.

If the numeric format of a framebuffer attachment uses sRGB encoding, the R, G, and B destination color values (after conversion from fixed-point to floating-point) are considered to be encoded for the sRGB color space and hence are linearized prior to their use in blending. Each R, G, and B component is converted from nonlinear to linear as described in the “sRGB EOTF” section of the Khronos Data Format Specification. If the format is not sRGB, no linearization is performed.

If the numeric format of a framebuffer attachment uses sRGB encoding, then the final R, G and B values are converted into the nonlinear sRGB representation before being written to the framebuffer attachment as described in the “sRGB $EOTF^{-1}$” section of the Khronos Data Format

Specification.

If the numeric format of a framebuffer color attachment is not sRGB encoded then the resulting c_s values for R, G and B are unmodified. The value of A is never sRGB encoded. That is, the alpha component is always stored in memory as linear.

If the framebuffer color attachment is VK_ATTACHMENT_UNUSED, no writes are performed through that attachment. Writes are not performed to framebuffer color attachments greater than or equal to the VkSubpassDescription::colorAttachmentCount or VkSubpassDescription2::colorAttachmentCount value.

28.1.4. Advanced Blend Operations

The *advanced blend operations* are those listed in tables f/X/Y/Z Advanced Blend Operations, Hue-Saturation-Luminosity Advanced Blend Operations, and Additional RGB Blend Operations.

If the pNext chain of VkPipelineColorBlendStateCreateInfo includes a VkPipelineColorBlendAdvancedStateCreateInfoEXT structure, then that structure includes parameters that affect advanced blend operations.

The VkPipelineColorBlendAdvancedStateCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_blend_operation_advanced
typedef struct VkPipelineColorBlendAdvancedStateCreateInfoEXT {
    VkStructureType      sType;
    const void*          pNext;
    VkBool32             srcPremultiplied;
    VkBool32             dstPremultiplied;
    VkBlendOverlapEXT    blendOverlap;
} VkPipelineColorBlendAdvancedStateCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- srcPremultiplied specifies whether the source color of the blend operation is treated as premultiplied.
- dstPremultiplied specifies whether the destination color of the blend operation is treated as premultiplied.
- blendOverlap is a VkBlendOverlapEXT value specifying how the source and destination sample's coverage is correlated.

If this structure is not present, srcPremultiplied and dstPremultiplied are both considered to be VK_TRUE, and blendOverlap is considered to be VK_BLEND_OVERLAP_UNCORRELATED_EXT.

Valid Usage

- VUID-VkPipelineColorBlendAdvancedStateCreateInfoEXT-srcPremultiplied-01424
  If the non-premultiplied source color property is not supported, srcPremultiplied **must** be

VK_TRUE

- VUID-VkPipelineColorBlendAdvancedStateCreateInfoEXT-dstPremultiplied-01425
  If the non-premultiplied destination color property is not supported, dstPremultiplied **must** be VK_TRUE
- VUID-VkPipelineColorBlendAdvancedStateCreateInfoEXT-blendOverlap-01426
  If the correlated overlap property is not supported, blendOverlap **must** be VK_BLEND_OVERLAP_UNCORRELATED_EXT

Valid Usage (Implicit)

- VUID-VkPipelineColorBlendAdvancedStateCreateInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_ADVANCED_STATE_CREATE_INFO_EXT
- VUID-VkPipelineColorBlendAdvancedStateCreateInfoEXT-blendOverlap-parameter
  blendOverlap **must** be a valid VkBlendOverlapEXT value

To dynamically set the advanced blend state, call:

```
// Provided by VK_EXT_blend_operation_advanced with VK_EXT_extended_dynamic_state3,
VK_EXT_blend_operation_advanced with VK_EXT_shader_object
void vkCmdSetColorBlendAdvancedEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    firstAttachment,
    uint32_t                                    attachmentCount,
    const VkColorBlendAdvancedEXT*              pColorBlendAdvanced);
```

- commandBuffer is the command buffer into which the command will be recorded.
- firstAttachment the first color attachment the advanced blend parameters apply to.
- attachmentCount the number of VkColorBlendAdvancedEXT elements in the pColorBlendAdvanced array.
- pColorBlendAdvanced an array of VkColorBlendAdvancedEXT structs that specify the advanced color blend parameters for the corresponding attachments.

This command sets the advanced blend operation parameters of the specified attachments for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineColorBlendAdvancedStateCreateInfoEXT::srcPremultiplied, VkPipelineColorBlendAdvancedStateCreateInfoEXT::dstPremultiplied, and VkPipelineColorBlendAdvancedStateCreateInfoEXT::blendOverlap values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetColorBlendAdvancedEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3ColorBlendAdvanced` feature is enabled
  - The `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetColorBlendAdvancedEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetColorBlendAdvancedEXT-pColorBlendAdvanced-parameter
  `pColorBlendAdvanced` **must** be a valid pointer to an array of `attachmentCount` valid VkColorBlendAdvancedEXT structures
- VUID-vkCmdSetColorBlendAdvancedEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetColorBlendAdvancedEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetColorBlendAdvancedEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetColorBlendAdvancedEXT-attachmentCount-arraylength
  `attachmentCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

The `VkColorBlendAdvancedEXT` structure is defined as:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
typedef struct VkColorBlendAdvancedEXT {
    VkBlendOp            advancedBlendOp;
    VkBool32             srcPremultiplied;
    VkBool32             dstPremultiplied;
    VkBlendOverlapEXT    blendOverlap;
    VkBool32             clampResults;
} VkColorBlendAdvancedEXT;
```

- `advancedBlendOp` selects which blend operation is used to calculate the RGB values to write to the color attachment.
- `srcPremultiplied` specifies whether the source color of the blend operation is treated as premultiplied.
- `dstPremultiplied` specifies whether the destination color of the blend operation is treated as premultiplied.
- `blendOverlap` is a VkBlendOverlapEXT value specifying how the source and destination sample's coverage is correlated.
- `clampResults` specifies that results **must** be clamped to the [0,1] range before writing to the attachment, which is useful when the attachment format is not normalized fixed-point.

Valid Usage

- VUID-VkColorBlendAdvancedEXT-srcPremultiplied-07505
  If the non-premultiplied source color property is not supported, `srcPremultiplied` **must** be `VK_TRUE`
- VUID-VkColorBlendAdvancedEXT-dstPremultiplied-07506
  If the non-premultiplied destination color property is not supported, `dstPremultiplied` **must** be `VK_TRUE`
- VUID-VkColorBlendAdvancedEXT-blendOverlap-07507
  If the correlated overlap property is not supported, `blendOverlap` **must** be `VK_BLEND_OVERLAP_UNCORRELATED_EXT`

Valid Usage (Implicit)

- VUID-VkColorBlendAdvancedEXT-advancedBlendOp-parameter
  `advancedBlendOp` **must** be a valid VkBlendOp value
- VUID-VkColorBlendAdvancedEXT-blendOverlap-parameter
  `blendOverlap` **must** be a valid VkBlendOverlapEXT value

When using one of the operations in table f/X/Y/Z Advanced Blend Operations or Hue-Saturation-Luminosity Advanced Blend Operations, blending is performed according to the following equations:

$$
\begin{aligned}
R = \quad & f(R_s', R_d') * p_0(A_s, A_d) + \quad Y * R_s' * p_1(A_s, A_d) + \quad Z * R_d' * p_2(A_s, A_d) \\
G = \quad & f(G_s', G_d') * p_0(A_s, A_d) + \quad Y * G_s' * p_1(A_s, A_d) + \quad Z * G_d' * p_2(A_s, A_d) \\
B = \quad & f(B_s', B_d') * p_0(A_s, A_d) + \quad Y * B_s' * p_1(A_s, A_d) + \quad Z * B_d' * p_2(A_s, A_d) \\
A = \quad & X * p_0(A_s, A_d) + \quad Y * p_1(A_s, A_d) + \quad Z * p_2(A_s, A_d)
\end{aligned}
$$

where the function f and terms X, Y, and Z are specified in the table. The R, G, and B components of the source color used for blending are derived according to `srcPremultiplied`. If `srcPremultiplied` is `VK_TRUE`, the fragment color components are considered to have been premultiplied by the A component prior to blending. The base source color (R_s',G_s',B_s') is obtained by dividing through by the A component:

$$
(R_s', G_s', B_s') = \begin{cases} (0, 0, 0) & A_s = 0 \\ (\frac{R_s}{A_s}, \frac{G_s}{A_s}, \frac{B_s}{A_s}) & \text{otherwise} \end{cases}
$$

If `srcPremultiplied` is `VK_FALSE`, the fragment color components are used as the base color:

$$
(R_s', G_s', B_s') = (R_s, G_s, B_s)
$$

The R, G, and B components of the destination color used for blending are derived according to `dstPremultiplied`. If `dstPremultiplied` is `VK_TRUE`, the destination components are considered to have been premultiplied by the A component prior to blending. The base destination color (R_d',G_d',B_d') is obtained by dividing through by the A component:

$$
(R_d', G_d', B_d') = \begin{cases} (0, 0, 0) & A_d = 0 \\ (\frac{R_d}{A_d}, \frac{G_d}{A_d}, \frac{B_d}{A_d}) & \text{otherwise} \end{cases}
$$

If `dstPremultiplied` is `VK_FALSE`, the destination color components are used as the base color:

$$
(R_d', G_d', B_d') = (R_d, G_d, B_d)
$$

When blending using advanced blend operations, we expect that the R, G, and B components of premultiplied source and destination color inputs be stored as the product of non-premultiplied R, G, and B component values and the A component of the color. If any R, G, or B component of a premultiplied input color is non-zero and the A component is zero, the color is considered ill-formed, and the corresponding component of the blend result is undefined.

All of the advanced blend operation formulas in this chapter compute the result as a premultiplied color. If `dstPremultiplied` is `VK_FALSE`, that result color's R, G, and B components are divided by the A component before being written to the framebuffer. If any R, G, or B component of the color is non-zero and the A component is zero, the result is considered ill-formed, and the corresponding component of the blend result is undefined. If all components are zero, that value is unchanged.

If the A component of any input or result color is less than zero, the color is considered ill-formed, and all components of the blend result are undefined.

The weighting functions p_0, p_1, and p_2 are defined in table Advanced Blend Overlap Modes. In these

functions, the A components of the source and destination colors are taken to indicate the portion of the pixel covered by the fragment (source) and the fragments previously accumulated in the pixel (destination). The functions p_0, p_1, and p_2 approximate the relative portion of the pixel covered by the intersection of the source and destination, covered only by the source, and covered only by the destination, respectively.

Possible values of VkPipelineColorBlendAdvancedStateCreateInfoEXT::blendOverlap, specifying the blend overlap functions, are:

```
// Provided by VK_EXT_blend_operation_advanced
typedef enum VkBlendOverlapEXT {
    VK_BLEND_OVERLAP_UNCORRELATED_EXT = 0,
    VK_BLEND_OVERLAP_DISJOINT_EXT = 1,
    VK_BLEND_OVERLAP_CONJOINT_EXT = 2,
} VkBlendOverlapEXT;
```

- VK_BLEND_OVERLAP_UNCORRELATED_EXT specifies that there is no correlation between the source and destination coverage.
- VK_BLEND_OVERLAP_CONJOINT_EXT specifies that the source and destination coverage are considered to have maximal overlap.
- VK_BLEND_OVERLAP_DISJOINT_EXT specifies that the source and destination coverage are considered to have minimal overlap.

Table 36. Advanced Blend Overlap Modes

Overlap Mode	Weighting Equations
VK_BLEND_OVERLAP_UNCORRELATED_EXT	$p_0(A_s, A_d) = A_s A_d$ $p_1(A_s, A_d) = A_s(1 - A_d)$ $p_2(A_s, A_d) = A_d(1 - A_s)$
VK_BLEND_OVERLAP_CONJOINT_EXT	$p_0(A_s, A_d) = min(A_s, A_d)$ $p_1(A_s, A_d) = max(A_s - A_d, 0)$ $p_2(A_s, A_d) = max(A_d - A_s, 0)$
VK_BLEND_OVERLAP_DISJOINT_EXT	$p_0(A_s, A_d) = max(A_s + A_d - 1, 0)$ $p_1(A_s, A_d) = min(A_s, 1 - A_d)$ $p_2(A_s, A_d) = min(A_d, 1 - A_s)$

Table 37. f/X/Y/Z Advanced Blend Operations

Mode	Blend Coefficients
VK_BLEND_OP_ZERO_EXT	$(X, Y, Z) = (0, 0, 0)$ $f(C_s, C_d) = 0$
VK_BLEND_OP_SRC_EXT	$(X, Y, Z) = (1, 1, 0)$ $f(C_s, C_d) = C_s$

Mode	Blend Coefficients
VK_BLEND_OP_DST_EXT	$(X, Y, Z) = (1, 0, 1)$ $f(C_s, C_d) = C_d$
VK_BLEND_OP_SRC_OVER_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = C_s$
VK_BLEND_OP_DST_OVER_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = C_d$
VK_BLEND_OP_SRC_IN_EXT	$(X, Y, Z) = (1, 0, 0)$ $f(C_s, C_d) = C_s$
VK_BLEND_OP_DST_IN_EXT	$(X, Y, Z) = (1, 0, 0)$ $f(C_s, C_d) = C_d$
VK_BLEND_OP_SRC_OUT_EXT	$(X, Y, Z) = (0, 1, 0)$ $f(C_s, C_d) = 0$
VK_BLEND_OP_DST_OUT_EXT	$(X, Y, Z) = (0, 0, 1)$ $f(C_s, C_d) = 0$
VK_BLEND_OP_SRC_ATOP_EXT	$(X, Y, Z) = (1, 0, 1)$ $f(C_s, C_d) = C_s$
VK_BLEND_OP_DST_ATOP_EXT	$(X, Y, Z) = (1, 1, 0)$ $f(C_s, C_d) = C_d$
VK_BLEND_OP_XOR_EXT	$(X, Y, Z) = (0, 1, 1)$ $f(C_s, C_d) = 0$
VK_BLEND_OP_MULTIPLY_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = C_s C_d$
VK_BLEND_OP_SCREEN_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = C_s + C_d - C_s C_d$
VK_BLEND_OP_OVERLAY_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} 2C_s C_d & C_d \le 0.5 \\ 1 - 2(1 - C_s)(1 - C_d) & \text{otherwise} \end{cases}$
VK_BLEND_OP_DARKEN_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = min(C_s, C_d)$
VK_BLEND_OP_LIGHTEN_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = max(C_s, C_d)$
VK_BLEND_OP_COLORDODGE_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} 0 & C_d \le 0 \\ min(1, \frac{C_d}{1 - C_s}) & C_d > 0 \text{and} C_s < 1 \\ 1 & C_d > 0 \text{and} C_s \ge 1 \end{cases}$

Mode	Blend Coefficients
VK_BLEND_OP_COLORBURN_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} 1 & C_d \geq 1 \\ 1 - min(1, \frac{1-C_d}{C_s}) & C_d < 1 \text{and} C_s > 0 \\ 0 & C_d < 1 \text{and} C_s \leq 0 \end{cases}$
VK_BLEND_OP_HARDLIGHT_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} 2C_sC_d & C_s \leq 0.5 \\ 1 - 2(1-C_s)(1-C_d) & \text{otherwise} \end{cases}$
VK_BLEND_OP_SOFTLIGHT_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} C_d - (1-2C_s)C_d(1-C_d) & C_s \leq 0.5 \\ C_d + (2C_s-1)C_d((16C_d-12)C_d+3) & C_s > 0.5 \text{and} C_d \leq 0.25 \\ C_d + (2C_s-1)(\sqrt{C_d} - C_d) & C_s > 0.5 \text{and} C_d > 0.25 \end{cases}$
VK_BLEND_OP_DIFFERENCE_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \lvert C_d - C_s \rvert$
VK_BLEND_OP_EXCLUSION_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = C_s + C_d - 2C_sC_d$
VK_BLEND_OP_INVERT_EXT	$(X, Y, Z) = (1, 0, 1)$ $f(C_s, C_d) = 1 - C_d$
VK_BLEND_OP_INVERT_RGB_EXT	$(X, Y, Z) = (1, 0, 1)$ $f(C_s, C_d) = C_s(1 - C_d)$
VK_BLEND_OP_LINEARDODGE_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} C_s + C_d & C_s + C_d \leq 1 \\ 1 & \text{otherwise} \end{cases}$
VK_BLEND_OP_LINEARBURN_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} C_s + C_d - 1 & C_s + C_d > 1 \\ 0 & \text{otherwise} \end{cases}$
VK_BLEND_OP_VIVIDLIGHT_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} 1 - min(1, \frac{1-C_d}{2C_s}) & 0 < C_s < 0.5 \\ 0 & C_s \leq 0 \\ min(1, \frac{C_d}{2(1-C_s)}) & 0.5 \leq C_s < 1 \\ 1 & C_s \geq 1 \end{cases}$
VK_BLEND_OP_LINEARLIGHT_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} 1 & 2C_s + C_d > 2 \\ 2C_s + C_d - 1 & 1 < 2C_s + C_d \leq 2 \\ 0 & 2C_s + C_d \leq 1 \end{cases}$
VK_BLEND_OP_PINLIGHT_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} 0 & 2C_s - 1 > C_d \text{and} C_s < 0.5 \\ 2C_s - 1 & 2C_s - 1 > C_d \text{and} C_s \geq 0.5 \\ 2C_s & 2C_s - 1 \leq C_d \text{and} C_s < 0.5 C_d \\ C_d & 2C_s - 1 \leq C_d \text{and} C_s \geq 0.5 C_d \end{cases}$
VK_BLEND_OP_HARDMIX_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = \begin{cases} 0 & C_s + C_d < 1 \\ 1 & \text{otherwise} \end{cases}$

When using one of the HSL blend operations in table Hue-Saturation-Luminosity Advanced Blend

Operations as the blend operation, the RGB color components produced by the function f are effectively obtained by converting both the non-premultiplied source and destination colors to the HSL (hue, saturation, luminosity) color space, generating a new HSL color by selecting H, S, and L components from the source or destination according to the blend operation, and then converting the result back to RGB. In the equations below, a blended RGB color is produced according to the following pseudocode:

```
float minv3(vec3 c) {
  return min(min(c.r, c.g), c.b);
}
float maxv3(vec3 c) {
  return max(max(c.r, c.g), c.b);
}
float lumv3(vec3 c) {
  return dot(c, vec3(0.30, 0.59, 0.11));
}
float satv3(vec3 c) {
  return maxv3(c) - minv3(c);
}

// If any color components are outside [0,1], adjust the color to
// get the components in range.
vec3 ClipColor(vec3 color) {
  float lum = lumv3(color);
  float mincol = minv3(color);
  float maxcol = maxv3(color);
  if (mincol < 0.0) {
    color = lum + ((color-lum)*lum) / (lum-mincol);
  }
  if (maxcol > 1.0) {
    color = lum + ((color-lum)*(1-lum)) / (maxcol-lum);
  }
  return color;
}

// Take the base RGB color <cbase> and override its luminosity
// with that of the RGB color <clum>.
vec3 SetLum(vec3 cbase, vec3 clum) {
  float lbase = lumv3(cbase);
  float llum = lumv3(clum);
  float ldiff = llum - lbase;
  vec3 color = cbase + vec3(ldiff);
  return ClipColor(color);
}

// Take the base RGB color <cbase> and override its saturation with
// that of the RGB color <csat>.  The override the luminosity of the
// result with that of the RGB color <clum>.
vec3 SetLumSat(vec3 cbase, vec3 csat, vec3 clum)
{
```

```
    float minbase = minv3(cbase);
    float sbase = satv3(cbase);
    float ssat = satv3(csat);
    vec3 color;
    if (sbase > 0) {
      // Equivalent (modulo rounding errors) to setting the
      // smallest (R,G,B) component to 0, the largest to <ssat>,
      // and interpolating the "middle" component based on its
      // original value relative to the smallest/largest.
      color = (cbase - minbase) * ssat / sbase;
    } else {
      color = vec3(0.0);
    }
    return SetLum(color, clum);
  }
```

Table 38. Hue-Saturation-Luminosity Advanced Blend Operations

Mode	Result
VK_BLEND_OP_HSL_HUE_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = SetLumSat(C_s, C_d, C_d)$
VK_BLEND_OP_HSL_SATURATION_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = SetLumSat(C_d, C_s, C_d)$
VK_BLEND_OP_HSL_COLOR_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = SetLum(C_s, C_d)$
VK_BLEND_OP_HSL_LUMINOSITY_EXT	$(X, Y, Z) = (1, 1, 1)$ $f(C_s, C_d) = SetLum(C_d, C_s)$

When using one of the operations in table Additional RGB Blend Operations as the blend operation, the source and destination colors used by these blending operations are interpreted according to srcPremultiplied and dstPremultiplied. The blending operations below are evaluated where the RGB source and destination color components are both considered to have been premultiplied by the corresponding A component.

$$(R_s', G_s', B_s') = \begin{cases} (R_s, G_s, B_s) & \text{if srcPremultiplied is VK_TRUE} \\ (R_s A_s, G_s A_s, B_s A_s) & \text{if srcPremultiplied is VK_FALSE} \end{cases}$$

$$(R_d', G_d', B_d') = \begin{cases} (R_d, G_d, B_d) & \text{if dstPremultiplied is VK_TRUE} \\ (R_d A_d, G_d A_d, B_d A_d) & \text{if dstPremultiplied is VK_FALSE} \end{cases}$$

Table 39. Additional RGB Blend Operations

Mode	Result
VK_BLEND_OP_PLUS_EXT	$(R, G, B, A) = (R_s' + R_d', G_s' + G_d', B_s' + B_d', A_s + A_d)$
VK_BLEND_OP_PLUS_CLAMPED_EXT	$(R, G, B, A) = (min(1, R_s' + R_d'), min(1, G_s' + G_d'), min(1, B_s' + B_d'), min(1, A_s + A_d))$
VK_BLEND_OP_PLUS_CLAMPED_ALPHA_EXT	$(R, G, B, A) = (min(min(1, A_s + A_d), R_s' + R_d'), min(min(1, A_s + A_d), G_s' + G_d'), min(min(1, A_s + A_d), B_s' + B_d'), min(1, A_s + A_d))$
VK_BLEND_OP_PLUS_DARKER_EXT	$(R, G, B, A) = (max(0, min(1, A_s + A_d) - ((A_s - R_s') + (A_d - R_d'))), max(0, min(1, A_s + A_d) - ((A_s - G_s') + (A_d - G_d'))), max(0, min(1, A_s + A_d) - ((A_s - B_s') + (A_d - B_d'))), min(1, A_s + A_d))$
VK_BLEND_OP_MINUS_EXT	$(R, G, B, A) = (R_d' - R_s', G_d' - G_s', B_d' - B_s', A_d - A_s)$
VK_BLEND_OP_MINUS_CLAMPED_EXT	$(R, G, B, A) = (max(0, R_d' - R_s'), max(0, G_d' - G_s'), max(0, B_d' - B_s'), max(0, A_d - A_s))$
VK_BLEND_OP_CONTRAST_EXT	$(R, G, B, A) = (\frac{A_d}{2} + 2(R_d' - \frac{A_d}{2})(R_s' - \frac{A_s}{2}), \frac{A_d}{2} + 2(G_d' - \frac{A_d}{2})(G_s' - \frac{A_s}{2}), \frac{A_d}{2} + 2(B_d' - \frac{A_d}{2})(B_s' - \frac{A_s}{2}), A_d)$
VK_BLEND_OP_INVERT_OVG_EXT	$(R, G, B, A) = (A_s(1 - R_d') + (1 - A_s)R_d', A_s(1 - G_d') + (1 - A_s)G_d', A_s(1 - B_d') + (1 - A_s)B_d', A_s + A_d - A_sA_d)$
VK_BLEND_OP_RED_EXT	$(R, G, B, A) = (R_s', G_d', B_d', A_d)$
VK_BLEND_OP_GREEN_EXT	$(R, G, B, A) = (R_d', G_s', B_d', A_d)$
VK_BLEND_OP_BLUE_EXT	$(R, G, B, A) = (R_d', G_d', B_s', A_d)$

28.2. Logical Operations

The application **can** enable a *logical operation* between the fragment's color values and the existing value in the framebuffer attachment. This logical operation is applied prior to updating the framebuffer attachment. Logical operations are applied only for signed and unsigned integer and normalized integer framebuffers. Logical operations are not applied to floating-point or sRGB format color attachments.

Logical operations are controlled by the `logicOpEnable` and `logicOp` members of VkPipelineColorBlendStateCreateInfo. The `logicOpEnable` state can also be controlled by vkCmdSetLogicOpEnableEXT if graphics pipeline is created with `VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. If `logicOpEnable` is `VK_TRUE`, then a logical operation selected by `logicOp` is applied between each color attachment and the fragment's corresponding output value, and blending of all attachments is treated as if it were disabled. Any attachments using color formats for which logical operations are not supported simply pass through the color values unmodified. The logical operation is applied independently for each of the red, green, blue, and alpha components. The `logicOp` is selected from the following operations:

```
// Provided by VK_VERSION_1_0
typedef enum VkLogicOp {
    VK_LOGIC_OP_CLEAR = 0,
    VK_LOGIC_OP_AND = 1,
    VK_LOGIC_OP_AND_REVERSE = 2,
    VK_LOGIC_OP_COPY = 3,
    VK_LOGIC_OP_AND_INVERTED = 4,
    VK_LOGIC_OP_NO_OP = 5,
    VK_LOGIC_OP_XOR = 6,
    VK_LOGIC_OP_OR = 7,
    VK_LOGIC_OP_NOR = 8,
    VK_LOGIC_OP_EQUIVALENT = 9,
    VK_LOGIC_OP_INVERT = 10,
    VK_LOGIC_OP_OR_REVERSE = 11,
    VK_LOGIC_OP_COPY_INVERTED = 12,
    VK_LOGIC_OP_OR_INVERTED = 13,
    VK_LOGIC_OP_NAND = 14,
    VK_LOGIC_OP_SET = 15,
} VkLogicOp;
```

The logical operations supported by Vulkan are summarized in the following table in which

- ¬ is bitwise invert,
- ∧ is bitwise and,
- ∨ is bitwise or,
- ⊕ is bitwise exclusive or,
- s is the fragment's R_{s0}, G_{s0}, B_{s0} or A_{s0} component value for the fragment output corresponding to the color attachment being updated, and
- d is the color attachment's R, G, B or A component value:

Table 40. Logical Operations

Mode	Operation
VK_LOGIC_OP_CLEAR	0
VK_LOGIC_OP_AND	s ∧ d
VK_LOGIC_OP_AND_REVERSE	s ∧ ¬ d
VK_LOGIC_OP_COPY	s
VK_LOGIC_OP_AND_INVERTED	¬ s ∧ d
VK_LOGIC_OP_NO_OP	d
VK_LOGIC_OP_XOR	s ⊕ d
VK_LOGIC_OP_OR	s ∨ d
VK_LOGIC_OP_NOR	¬ (s ∨ d)
VK_LOGIC_OP_EQUIVALENT	¬ (s ⊕ d)
VK_LOGIC_OP_INVERT	¬ d
VK_LOGIC_OP_OR_REVERSE	s ∨ ¬ d
VK_LOGIC_OP_COPY_INVERTED	¬ s
VK_LOGIC_OP_OR_INVERTED	¬ s ∨ d
VK_LOGIC_OP_NAND	¬ (s ∧ d)
VK_LOGIC_OP_SET	all 1s

The result of the logical operation is then written to the color attachment as controlled by the component write mask, described in Blend Operations.

To dynamically set whether logical operations are enabled, call:

```
// Provided by VK_EXT_extended_dynamic_state3, VK_EXT_shader_object
void vkCmdSetLogicOpEnableEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    logicOpEnable);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `logicOpEnable` specifies whether logical operations are enabled.

This command sets whether logical operations are enabled for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with `VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT` set in VkPipelineDynamicStateCreateInfo::`pDynamicStates`. Otherwise, this state is specified by the VkPipelineColorBlendStateCreateInfo::`logicOpEnable` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetLogicOpEnableEXT-None-09423
  At least one of the following **must** be true:
  - The `extendedDynamicState3LogicOpEnable` feature is enabled
  - The `shaderObject` feature is enabled
- VUID-vkCmdSetLogicOpEnableEXT-logicOp-07366
  If the `logicOp` feature is not enabled, `logicOpEnable` **must** be `VK_FALSE`

Valid Usage (Implicit)

- VUID-vkCmdSetLogicOpEnableEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetLogicOpEnableEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetLogicOpEnableEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetLogicOpEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

To dynamically set the logical operation to apply for blend state, call:

```
// Provided by VK_EXT_shader_object
void vkCmdSetLogicOpEXT(
    VkCommandBuffer                             commandBuffer,
    VkLogicOp                                   logicOp);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `logicOp` specifies the logical operation to apply for blend state.

This command sets the logical operation for blend state for subsequent drawing commands when drawing using shader objects. Otherwise, this state is specified by the VkPipelineColorBlendStateCreateInfo::`logicOp` value used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetLogicOpEXT-None-09422
  At least one of the following **must** be true:
  - The `shaderObject` feature is enabled

Valid Usage (Implicit)

- VUID-vkCmdSetLogicOpEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetLogicOpEXT-logicOp-parameter
  `logicOp` **must** be a valid VkLogicOp value
- VUID-vkCmdSetLogicOpEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetLogicOpEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetLogicOpEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

28.3. Color Write Mask

Bits which **can** be set in VkPipelineColorBlendAttachmentState::`colorWriteMask`, determining whether the final color values R, G, B and A are written to the framebuffer attachment, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkColorComponentFlagBits {
    VK_COLOR_COMPONENT_R_BIT = 0x00000001,
    VK_COLOR_COMPONENT_G_BIT = 0x00000002,
    VK_COLOR_COMPONENT_B_BIT = 0x00000004,
    VK_COLOR_COMPONENT_A_BIT = 0x00000008,
} VkColorComponentFlagBits;
```

- `VK_COLOR_COMPONENT_R_BIT` specifies that the R value is written to the color attachment for the appropriate sample. Otherwise, the value in memory is unmodified.
- `VK_COLOR_COMPONENT_G_BIT` specifies that the G value is written to the color attachment for the appropriate sample. Otherwise, the value in memory is unmodified.
- `VK_COLOR_COMPONENT_B_BIT` specifies that the B value is written to the color attachment for the appropriate sample. Otherwise, the value in memory is unmodified.
- `VK_COLOR_COMPONENT_A_BIT` specifies that the A value is written to the color attachment for the appropriate sample. Otherwise, the value in memory is unmodified.

The color write mask operation is applied regardless of whether blending is enabled.

The color write mask operation is applied only if Color Write Enable is enabled for the respective attachment. Otherwise the color write mask is ignored and writes to all components of the attachment are disabled.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkColorComponentFlags;
```

VkColorComponentFlags is a bitmask type for setting a mask of zero or more VkColorComponentFlagBits.

28.4. Color Write Enable

The VkPipelineColorWriteCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_color_write_enable
typedef struct VkPipelineColorWriteCreateInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           attachmentCount;
    const VkBool32*    pColorWriteEnables;
} VkPipelineColorWriteCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- attachmentCount is the number of VkBool32 elements in pColorWriteEnables.
- pColorWriteEnables is a pointer to an array of per target attachment boolean values specifying whether color writes are enabled for the given attachment.

When this structure is included in the pNext chain of VkPipelineColorBlendStateCreateInfo, it defines per-attachment color write state. If this structure is not included in the pNext chain, it is equivalent to specifying this structure with attachmentCount equal to the attachmentCount member of VkPipelineColorBlendStateCreateInfo, and pColorWriteEnables pointing to an array of as many VK_TRUE values.

If the colorWriteEnable feature is not enabled, all VkBool32 elements in the pColorWriteEnables array **must** be VK_TRUE.

Color Write Enable interacts with the Color Write Mask as follows:

- If colorWriteEnable is VK_TRUE, writes to the attachment are determined by the colorWriteMask.
- If colorWriteEnable is VK_FALSE, the colorWriteMask is ignored and writes to all components of the attachment are disabled. This is equivalent to specifying a colorWriteMask of 0.

Valid Usage

- VUID-VkPipelineColorWriteCreateInfoEXT-pAttachments-04801
  If the colorWriteEnable feature is not enabled, all elements of pColorWriteEnables **must** be VK_TRUE
- VUID-VkPipelineColorWriteCreateInfoEXT-attachmentCount-07608

If the pipeline is being created with VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT, VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT, VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT, or VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic states not set, attachmentCount **must** be equal to the attachmentCount member of the VkPipelineColorBlendStateCreateInfo structure specified during pipeline creation

- VUID-VkPipelineColorWriteCreateInfoEXT-attachmentCount-06655
  attachmentCount **must** be less than or equal to the maxColorAttachments member of VkPhysicalDeviceLimits

Valid Usage (Implicit)

- VUID-VkPipelineColorWriteCreateInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PIPELINE_COLOR_WRITE_CREATE_INFO_EXT
- VUID-VkPipelineColorWriteCreateInfoEXT-pColorWriteEnables-parameter
  If attachmentCount is not 0, pColorWriteEnables **must** be a valid pointer to an array of attachmentCount VkBool32 values

To dynamically enable or disable writes to a color attachment, call:

```
// Provided by VK_EXT_color_write_enable
void                                        vkCmdSetColorWriteEnableEXT(
    VkCommandBuffer                                 commandBuffer,
    uint32_t                                        attachmentCount,
    const VkBool32*                                 pColorWriteEnables);
```

- commandBuffer is the command buffer into which the command will be recorded.
- attachmentCount is the number of VkBool32 elements in pColorWriteEnables.
- pColorWriteEnables is a pointer to an array of per target attachment boolean values specifying whether color writes are enabled for the given attachment.

This command sets the color write enables for subsequent drawing commands when drawing using shader objects, or when the graphics pipeline is created with VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT set in VkPipelineDynamicStateCreateInfo::pDynamicStates. Otherwise, this state is specified by the VkPipelineColorWriteCreateInfoEXT::pColorWriteEnables values used to create the currently active pipeline.

Valid Usage

- VUID-vkCmdSetColorWriteEnableEXT-None-04803
  The colorWriteEnable feature **must** be enabled
- VUID-vkCmdSetColorWriteEnableEXT-attachmentCount-06656
  attachmentCount **must** be less than or equal to the maxColorAttachments member of VkPhysicalDeviceLimits

Valid Usage (Implicit)

- VUID-vkCmdSetColorWriteEnableEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdSetColorWriteEnableEXT-pColorWriteEnables-parameter
  `pColorWriteEnables` **must** be a valid pointer to an array of `attachmentCount` VkBool32 values
- VUID-vkCmdSetColorWriteEnableEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdSetColorWriteEnableEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics operations
- VUID-vkCmdSetColorWriteEnableEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdSetColorWriteEnableEXT-attachmentCount-arraylength
  `attachmentCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics	State

Chapter 29. Dispatching Commands

The *dispatching commands* described in this chapter provoke work in a compute pipeline. Dispatching commands are recorded into a command buffer and when executed by a queue, will produce work which executes according to the bound compute pipeline. A compute pipeline **must** be bound to a command buffer before any dispatching commands are recorded in that command buffer.

To record a dispatch, call:

```
// Provided by VK_VERSION_1_0
void vkCmdDispatch(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    groupCountX,
    uint32_t                                    groupCountY,
    uint32_t                                    groupCountZ);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `groupCountX` is the number of local workgroups to dispatch in the X dimension.
- `groupCountY` is the number of local workgroups to dispatch in the Y dimension.
- `groupCountZ` is the number of local workgroups to dispatch in the Z dimension.

When the command is executed, a global workgroup consisting of `groupCountX` × `groupCountY` × `groupCountZ` local workgroups is assembled.

Valid Usage

- VUID-vkCmdDispatch-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDispatch-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDispatch-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDispatch-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be

VK_IMAGE_VIEW_TYPE_1D or VK_IMAGE_VIEW_TYPE_2D

- VUID-vkCmdDispatch-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdDispatch-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdDispatch-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdDispatch-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdDispatch-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdDispatch-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDispatch-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDispatch-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDispatch-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDispatch-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT

- VUID-vkCmdDispatch-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the `OpTypeImage` is `Unknown` then the view's buffer features **must** contain `VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT`
- VUID-vkCmdDispatch-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatch-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatch-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDispatch-maintenance4-08602
  If the `maintenance4` feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatch-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDispatch-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDispatch-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDispatch-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage

- VUID-vkCmdDispatch-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDispatch-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDispatch-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDispatch-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatch-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatch-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatch-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatch-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdDispatch-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands

- VUID-vkCmdDispatch-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDispatch-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdDispatch-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDispatch-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdDispatch-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDispatch-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDispatch-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDispatch-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDispatch-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDispatch-sparseImageInt64Atomics-04474
  If the `sparseImageInt64Atomics` feature is not enabled, VkImage objects created with the `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command
- VUID-vkCmdDispatch-sparseImageInt64Atomics-04475
  If the `sparseImageInt64Atomics` feature is not enabled, VkBuffer objects created with the `VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command

- VUID-vkCmdDispatch-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDispatch-None-09600
  If a descriptor with type equal to any of `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDispatch-groupCountX-00386
  `groupCountX` **must** be less than or equal to `VkPhysicalDeviceLimits::maxComputeWorkGroupCount`[0]
- VUID-vkCmdDispatch-groupCountY-00387
  `groupCountY` **must** be less than or equal to `VkPhysicalDeviceLimits::maxComputeWorkGroupCount`[1]
- VUID-vkCmdDispatch-groupCountZ-00388
  `groupCountZ` **must** be less than or equal to `VkPhysicalDeviceLimits::maxComputeWorkGroupCount`[2]

Valid Usage (Implicit)

- VUID-vkCmdDispatch-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDispatch-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdDispatch-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdDispatch-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdDispatch-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

To record an indirect dispatching command, call:

```
// Provided by VK_VERSION_1_0
void vkCmdDispatchIndirect(
    VkCommandBuffer                             commandBuffer,
    VkBuffer                                    buffer,
    VkDeviceSize                                offset);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `buffer` is the buffer containing dispatch parameters.
- `offset` is the byte offset into `buffer` where parameters begin.

`vkCmdDispatchIndirect` behaves similarly to vkCmdDispatch except that the parameters are read by the device from a buffer during execution. The parameters of the dispatch are encoded in a VkDispatchIndirectCommand structure taken from `buffer` starting at `offset`.

Valid Usage

- VUID-vkCmdDispatchIndirect-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDispatchIndirect-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDispatchIndirect-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDispatchIndirect-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`

- VUID-vkCmdDispatchIndirect-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdDispatchIndirect-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdDispatchIndirect-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdDispatchIndirect-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdDispatchIndirect-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdDispatchIndirect-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDispatchIndirect-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDispatchIndirect-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDispatchIndirect-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDispatchIndirect-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDispatchIndirect-OpTypeImage-07030

Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT

- VUID-vkCmdDispatchIndirect-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatchIndirect-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatchIndirect-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDispatchIndirect-maintenance4-08602
  If the maintenance4 feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatchIndirect-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDispatchIndirect-None-08606
  If the shaderObject feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDispatchIndirect-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDispatchIndirect-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, VK_IMAGE_VIEW_TYPE_1D_ARRAY, VK_IMAGE_VIEW_TYPE_2D_ARRAY or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY, in any shader stage

- VUID-vkCmdDispatchIndirect-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDispatchIndirect-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDispatchIndirect-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDispatchIndirect-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatchIndirect-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatchIndirect-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatchIndirect-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatchIndirect-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdDispatchIndirect-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands

- VUID-vkCmdDispatchIndirect-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDispatchIndirect-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdDispatchIndirect-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDispatchIndirect-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdDispatchIndirect-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDispatchIndirect-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDispatchIndirect-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDispatchIndirect-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDispatchIndirect-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDispatchIndirect-sparseImageInt64Atomics-04474
  If the `sparseImageInt64Atomics` feature is not enabled, VkImage objects created with the `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command
- VUID-vkCmdDispatchIndirect-sparseImageInt64Atomics-04475
  If the `sparseImageInt64Atomics` feature is not enabled, VkBuffer objects created with the `VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command

- VUID-vkCmdDispatchIndirect-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDispatchIndirect-None-09600
  If a descriptor with type equal to any of `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDispatchIndirect-buffer-02708
  If `buffer` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdDispatchIndirect-buffer-02709
  `buffer` **must** have been created with the `VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT` bit set
- VUID-vkCmdDispatchIndirect-offset-02710
  `offset` **must** be a multiple of `4`
- VUID-vkCmdDispatchIndirect-offset-00407
  The sum of `offset` and the size of `VkDispatchIndirectCommand` **must** be less than or equal to the size of `buffer`

Valid Usage (Implicit)

- VUID-vkCmdDispatchIndirect-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDispatchIndirect-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle
- VUID-vkCmdDispatchIndirect-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdDispatchIndirect-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdDispatchIndirect-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdDispatchIndirect-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdDispatchIndirect-commonparent
  Both of `buffer`, and `commandBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized

- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

The VkDispatchIndirectCommand structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkDispatchIndirectCommand {
    uint32_t    x;
    uint32_t    y;
    uint32_t    z;
} VkDispatchIndirectCommand;
```

- x is the number of local workgroups to dispatch in the X dimension.
- y is the number of local workgroups to dispatch in the Y dimension.
- z is the number of local workgroups to dispatch in the Z dimension.

The members of VkDispatchIndirectCommand have the same meaning as the corresponding parameters of vkCmdDispatch.

Valid Usage

- VUID-VkDispatchIndirectCommand-x-00417
  x **must** be less than or equal to VkPhysicalDeviceLimits::maxComputeWorkGroupCount[0]
- VUID-VkDispatchIndirectCommand-y-00418
  y **must** be less than or equal to VkPhysicalDeviceLimits::maxComputeWorkGroupCount[1]
- VUID-VkDispatchIndirectCommand-z-00419
  z **must** be less than or equal to VkPhysicalDeviceLimits::maxComputeWorkGroupCount[2]

To record a dispatch using non-zero base values for the components of WorkgroupId, call:

```
// Provided by VK_KHR_device_group
void vkCmdDispatchBaseKHR(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    baseGroupX,
    uint32_t                                    baseGroupY,
    uint32_t                                    baseGroupZ,
    uint32_t                                    groupCountX,
    uint32_t                                    groupCountY,
    uint32_t                                    groupCountZ);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `baseGroupX` is the start value for the X component of `WorkgroupId`.
- `baseGroupY` is the start value for the Y component of `WorkgroupId`.
- `baseGroupZ` is the start value for the Z component of `WorkgroupId`.
- `groupCountX` is the number of local workgroups to dispatch in the X dimension.
- `groupCountY` is the number of local workgroups to dispatch in the Y dimension.
- `groupCountZ` is the number of local workgroups to dispatch in the Z dimension.

When the command is executed, a global workgroup consisting of `groupCountX` × `groupCountY` × `groupCountZ` local workgroups is assembled, with `WorkgroupId` values ranging from [`baseGroup*`, `baseGroup*` + `groupCount*`) in each component. vkCmdDispatch is equivalent to `vkCmdDispatchBase(0,0,0,groupCountX,groupCountY,groupCountZ)`.

Valid Usage

- VUID-vkCmdDispatchBase-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDispatchBase-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdDispatchBase-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdDispatchBase-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`

- VUID-vkCmdDispatchBase-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdDispatchBase-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdDispatchBase-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdDispatchBase-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdDispatchBase-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdDispatchBase-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDispatchBase-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdDispatchBase-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDispatchBase-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdDispatchBase-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdDispatchBase-OpTypeImage-07030

Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT

- VUID-vkCmdDispatchBase-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatchBase-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatchBase-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdDispatchBase-maintenance4-08602
  If the maintenance4 feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdDispatchBase-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdDispatchBase-None-08606
  If the shaderObject feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdDispatchBase-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdDispatchBase-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, VK_IMAGE_VIEW_TYPE_1D_ARRAY, VK_IMAGE_VIEW_TYPE_2D_ARRAY or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY, in any shader stage

- VUID-vkCmdDispatchBase-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdDispatchBase-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdDispatchBase-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdDispatchBase-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatchBase-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatchBase-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatchBase-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdDispatchBase-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdDispatchBase-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands

- VUID-vkCmdDispatchBase-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdDispatchBase-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdDispatchBase-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdDispatchBase-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdDispatchBase-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdDispatchBase-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDispatchBase-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDispatchBase-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdDispatchBase-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdDispatchBase-sparseImageInt64Atomics-04474
  If the `sparseImageInt64Atomics` feature is not enabled, VkImage objects created with the `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command
- VUID-vkCmdDispatchBase-sparseImageInt64Atomics-04475
  If the `sparseImageInt64Atomics` feature is not enabled, VkBuffer objects created with the `VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command

- VUID-vkCmdDispatchBase-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdDispatchBase-None-09600
  If a descriptor with type equal to any of `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdDispatchBase-baseGroupX-00421
  `baseGroupX` **must** be less than `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[0]
- VUID-vkCmdDispatchBase-baseGroupX-00422
  `baseGroupY` **must** be less than `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[1]
- VUID-vkCmdDispatchBase-baseGroupZ-00423
  `baseGroupZ` **must** be less than `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[2]
- VUID-vkCmdDispatchBase-groupCountX-00424
  `groupCountX` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[0] minus `baseGroupX`
- VUID-vkCmdDispatchBase-groupCountY-00425
  `groupCountY` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[1] minus `baseGroupY`
- VUID-vkCmdDispatchBase-groupCountZ-00426
  `groupCountZ` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[2] minus `baseGroupZ`
- VUID-vkCmdDispatchBase-baseGroupX-00427
  If any of `baseGroupX`, `baseGroupY`, or `baseGroupZ` are not zero, then the bound compute pipeline **must** have been created with the `VK_PIPELINE_CREATE_DISPATCH_BASE` flag or the bound compute shader object **must** have been created with the `VK_SHADER_CREATE_DISPATCH_BASE_BIT_EXT` flag

Valid Usage (Implicit)

- VUID-vkCmdDispatchBase-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDispatchBase-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdDispatchBase-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdDispatchBase-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdDispatchBase-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

Chapter 30. Device-Generated Commands

This chapter discusses the generation of command buffer content on the device, for which these principle steps are to be taken:

- Define a layout describing the sequence of commands which should be generated.
- Optionally set up device-bindable shaders.
- Retrieve device addresses by `vkGetBufferDeviceAddressEXT` for setting buffers on the device.
- Fill one or more `VkBuffer` with the appropriate content that gets interpreted by the command layout.
- Create a `preprocess` `VkBuffer` using the device-queried allocation information.
- Optionally preprocess the input data in a separate action.
- Generate and execute the actual commands.

The preprocessing step executes in a separate logical pipeline from either graphics or compute. When preprocessing commands in a separate step they **must** be explicitly synchronized against the command execution. When not preprocessing in a separate step, the preprocessing is automatically synchronized against the command execution.

30.1. Indirect Commands Layout

The device-side command generation happens through an iterative processing of an atomic sequence comprised of command tokens, which are represented by:

```
// Provided by VK_EXT_device_generated_commands
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkIndirectCommandsLayoutEXT)
```

Each indirect command layout **must** have exactly one action command token and it **must** be the last token in the sequence.

NOTE	If the indirect commands layout contains only 1 token, it will be an action command token, and the contents of the indirect buffer will be a sequence of indirect command structures, similar to the ones used for indirect draws and dispatches. On some implementations, using indirect draws and dispatches for these cases will result in increased performance compared to using device-generated commands, due to the overhead that results from using the latter.

30.1.1. Creation and Deletion

Indirect command layouts for `VK_EXT_device_generated_commands` are created by:

```
// Provided by VK_EXT_device_generated_commands
VkResult vkCreateIndirectCommandsLayoutEXT(
    VkDevice                                    device,
    const VkIndirectCommandsLayoutCreateInfoEXT* pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkIndirectCommandsLayoutEXT*                pIndirectCommandsLayout);
```

- `device` is the logical device that creates the indirect command layout.
- `pCreateInfo` is a pointer to a VkIndirectCommandsLayoutCreateInfoEXT structure containing parameters affecting creation of the indirect command layout.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pIndirectCommandsLayout` is a pointer to a `VkIndirectCommandsLayoutEXT` handle in which the resulting indirect command layout is returned.

Valid Usage

- VUID-vkCreateIndirectCommandsLayoutEXT-deviceGeneratedCommands-11089
  The `VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT::deviceGeneratedCommands` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCreateIndirectCommandsLayoutEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateIndirectCommandsLayoutEXT-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkIndirectCommandsLayoutCreateInfoEXT structure
- VUID-vkCreateIndirectCommandsLayoutEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateIndirectCommandsLayoutEXT-pIndirectCommandsLayout-parameter
  `pIndirectCommandsLayout` **must** be a valid pointer to a VkIndirectCommandsLayoutEXT handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The VkIndirectCommandsLayoutCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectCommandsLayoutCreateInfoEXT {
    VkStructureType                                 sType;
    const void*                                     pNext;
    VkIndirectCommandsLayoutUsageFlagsEXT           flags;
    VkShaderStageFlags                              shaderStages;
    uint32_t                                        indirectStride;
    VkPipelineLayout                                pipelineLayout;
    uint32_t                                        tokenCount;
    const VkIndirectCommandsLayoutTokenEXT*         pTokens;
} VkIndirectCommandsLayoutCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkIndirectCommandsLayoutUsageFlagBitsEXT specifying usage rules for this layout.
- shaderStages is the VkShaderStageFlags that this layout supports.
- indirectStride is the distance in bytes between sequences in the indirect buffer
- pipelineLayout is the optional VkPipelineLayout that tokens in this layout use. If the dynamicGeneratedPipelineLayout feature is enabled, pipelineLayout **can** be VK_NULL_HANDLE and the layout **must** be specified by chaining the VkPipelineLayoutCreateInfo structure off the pNext
- tokenCount is the length of the individual command sequence.
- pTokens is a pointer to an array of VkIndirectCommandsLayoutTokenEXT describing each command token in detail.

The following code illustrates some of the flags:

```
void cmdProcessAllSequences(cmd, indirectExecutionSet, indirectCommandsLayout,
indirectAddress, sequencesCount)
{
  for (s = 0; s < sequencesCount; s++)
  {
    sUsed = s;

    if (indirectCommandsLayout.flags &
VK_INDIRECT_COMMANDS_LAYOUT_USAGE_UNORDERED_SEQUENCES_BIT_EXT) {
      sUsed = incoherent_implementation_dependent_permutation[ sUsed ];
    }

    cmdProcessSequence( cmd, indirectExecutionSet, indirectCommandsLayout,
indirectAddress, sUsed );
  }
```

```
}
```

When tokens are consumed, an offset is computed based on token offset and stream stride. The resulting offset is required to be aligned. The alignment for a specific token is equal to the scalar alignment of the data type as defined in Alignment Requirements, or 4, whichever is lower.

Valid Usage

- VUID-VkIndirectCommandsLayoutCreateInfoEXT-indirectStride-11090
  `indirectStride` **must** be less than or equal to VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`maxIndirectCommandsIndirectStride`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-shaderStages-11091
  `shaderStages` **must** only contain stages supported by VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`supportedIndirectCommandsShaderStages`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-tokenCount-11092
  `tokenCount` **must** less than or equal to VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`maxIndirectCommandsTokenCount`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11093
  The number of tokens in the `pTokens` array with `type` equal to `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` **must** be less than or equal to `1`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11145
  The number of tokens in the `pTokens` array with `type` equal to `VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT` **must** be less than or equal to `1`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11094
  The number of tokens in the `pTokens` array with `type` equal to `VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT` **must** be less than or equal to `1`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11095
  If the action command token in the `pTokens` array is not an indexed draw token, then `pTokens` **must** not contain a member with `type` set to `VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11096
  If the action command token in the `pTokens` array is not a non-mesh draw token, then `pTokens` **must** not contain a member with `type` set to `VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11097
  If the `pTokens` array contains multiple tokens with `type` equal to `VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT`, then there **must** be no duplicate VkIndirectCommandsVertexBufferTokenEXT::`vertexBindingUnit` values
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11099
  For all `VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT` and

VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT type tokens in pTokens, there **must** be no overlapping ranges between any specified push constant ranges

- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11100
  The action command token **must** be the last token in the pTokens array
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11139
  If the pTokens array contains a VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT token, then this token **must** be the first token in the array
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11101
  For any element of pTokens, if type is VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT or VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT and the dynamicGeneratedPipelineLayout feature is not enabled, then the pipelineLayout **must** not be VK_NULL_HANDLE
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11102
  For any element of pTokens, if type is either VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT or VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT and pipelineLayout is VK_NULL_HANDLE, then the pNext chain **must** include a VkPipelineLayoutCreateInfo struct
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11103
  For any element of pTokens, the offset **must** be greater than or equal to the offset member of the previous tokens
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11104
  For any element of pTokens, if type is VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT, VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT, VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_COUNT_EXT, VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_EXT, or VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_EXT, then shaderStages **must** contain graphics stages
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11105
  For any element of pTokens, if type is VK_INDIRECT_COMMANDS_TOKEN_TYPE_DISPATCH_EXT, then shaderStages **must** be VK_SHADER_STAGE_COMPUTE_BIT
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-11108
  For any element of pTokens, if type is VK_INDIRECT_COMMANDS_TOKEN_TYPE_TRACE_RAYS2_EXT, then shaderStages **must** contain ray tracing stages
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-shaderStages-11109
  If shaderStages contains graphics stages then the state tokens in pTokens **must** not include VK_INDIRECT_COMMANDS_TOKEN_TYPE_TRACE_RAYS2_EXT, VK_INDIRECT_COMMANDS_TOKEN_TYPE_DISPATCH_EXT
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-shaderStages-11110
  If shaderStages is VK_SHADER_STAGE_COMPUTE_BIT then the state tokens in pTokens **must** only include VK_INDIRECT_COMMANDS_TOKEN_TYPE_DISPATCH_EXT, VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT, VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT, or VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT

- VUID-VkIndirectCommandsLayoutCreateInfoEXT-shaderStages-11111
  If `shaderStages` contains ray tracing stages then the state tokens in `pTokens` **must** only include `VK_INDIRECT_COMMANDS_TOKEN_TYPE_TRACE_RAYS2_EXT`, `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT`, `VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT`, or `VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-shaderStages-11112
  The `shaderStages` **must** only contain stages from one of the following:
  - graphics stages
  - `VK_SHADER_STAGE_COMPUTE_BIT`
  - ray tracing stages
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-shaderStages-11113
  If `shaderStages` contains `VK_SHADER_STAGE_FRAGMENT_BIT`, then `shaderStages` **must** also contain `VK_SHADER_STAGE_VERTEX_BIT`

Valid Usage (Implicit)

- VUID-VkIndirectCommandsLayoutCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_INDIRECT_COMMANDS_LAYOUT_CREATE_INFO_EXT`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPipelineLayoutCreateInfo
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-flags-parameter
  `flags` **must** be a valid combination of VkIndirectCommandsLayoutUsageFlagBitsEXT values
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-shaderStages-parameter
  `shaderStages` **must** be a valid combination of VkShaderStageFlagBits values
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-shaderStages-requiredbitmask
  `shaderStages` **must** not be `0`
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pipelineLayout-parameter
  If `pipelineLayout` is not VK_NULL_HANDLE, `pipelineLayout` **must** be a valid VkPipelineLayout handle
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-pTokens-parameter
  `pTokens` **must** be a valid pointer to an array of `tokenCount` valid VkIndirectCommandsLayoutTokenEXT structures
- VUID-VkIndirectCommandsLayoutCreateInfoEXT-tokenCount-arraylength
  `tokenCount` **must** be greater than `0`

Bits which **can** be set in VkIndirectCommandsLayoutCreateInfoEXT::`flags`, specifying usage hints of an indirect command layout, are:

```
// Provided by VK_EXT_device_generated_commands
typedef enum VkIndirectCommandsLayoutUsageFlagBitsEXT {
    VK_INDIRECT_COMMANDS_LAYOUT_USAGE_EXPLICIT_PREPROCESS_BIT_EXT = 0x00000001,
    VK_INDIRECT_COMMANDS_LAYOUT_USAGE_UNORDERED_SEQUENCES_BIT_EXT = 0x00000002,
} VkIndirectCommandsLayoutUsageFlagBitsEXT;
```

- VK_INDIRECT_COMMANDS_LAYOUT_USAGE_EXPLICIT_PREPROCESS_BIT_EXT specifies that the layout is always used with the manual preprocessing step through calling vkCmdPreprocessGeneratedCommandsEXT and executed by vkCmdExecuteGeneratedCommandsEXT with isPreprocessed set to VK_TRUE.
- VK_INDIRECT_COMMANDS_LAYOUT_USAGE_UNORDERED_SEQUENCES_BIT_EXT specifies that the processing of sequences will happen at an implementation-dependent order, which is not guaranteed to be deterministic using the same input data. This flag is ignored when the shaderStages is VK_SHADER_STAGE_COMPUTE_BIT as it is implied that the dispatch sequence is always unordered.

```
// Provided by VK_EXT_device_generated_commands
typedef VkFlags VkIndirectCommandsLayoutUsageFlagsEXT;
```

VkIndirectCommandsLayoutUsageFlagsEXT is a bitmask type for setting a mask of zero or more VkIndirectCommandsLayoutUsageFlagBitsEXT.

Indirect command layouts for VK_EXT_device_generated_commands are destroyed by:

```
// Provided by VK_EXT_device_generated_commands
void vkDestroyIndirectCommandsLayoutEXT(
    VkDevice                                    device,
    VkIndirectCommandsLayoutEXT                 indirectCommandsLayout,
    const VkAllocationCallbacks*                pAllocator);
```

- device is the logical device that destroys the layout.
- indirectCommandsLayout is the layout to destroy.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyIndirectCommandsLayoutEXT-indirectCommandsLayout-11114
  All submitted commands that refer to indirectCommandsLayout **must** have completed execution
- VUID-vkDestroyIndirectCommandsLayoutEXT-indirectCommandsLayout-11115
  If VkAllocationCallbacks were provided when indirectCommandsLayout was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyIndirectCommandsLayoutEXT-indirectCommandsLayout-11116
  If no VkAllocationCallbacks were provided when indirectCommandsLayout was created,

pAllocator **must** be NULL

Valid Usage (Implicit)

- VUID-vkDestroyIndirectCommandsLayoutEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkDestroyIndirectCommandsLayoutEXT-indirectCommandsLayout-parameter
  If indirectCommandsLayout is not VK_NULL_HANDLE, indirectCommandsLayout **must** be a valid VkIndirectCommandsLayoutEXT handle
- VUID-vkDestroyIndirectCommandsLayoutEXT-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyIndirectCommandsLayoutEXT-indirectCommandsLayout-parent
  If indirectCommandsLayout is a valid handle, it **must** have been created, allocated, or retrieved from device

Host Synchronization

- Host access to indirectCommandsLayout **must** be externally synchronized

30.1.2. Token Input Streams

For VK_EXT_device_generated_commands, the input streams **can** contain raw uint32_t values, existing indirect commands such as:

- VkDrawIndirectCommand
- VkDrawIndexedIndirectCommand
- VkDispatchIndirectCommand
- VkTraceRaysIndirectCommandKHR
- VkTraceRaysIndirectCommand2KHR

or additional commands as listed below. How the data is used is described in the next section.

The VkBindIndexBufferIndirectCommandEXT structure specifies the input data for the VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT token.

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkBindIndexBufferIndirectCommandEXT {
    VkDeviceAddress    bufferAddress;
    uint32_t           size;
    VkIndexType        indexType;
} VkBindIndexBufferIndirectCommandEXT;
```

- `bufferAddress` specifies a physical address of the VkBuffer used as index buffer.
- `size` is the byte size range which is available for this operation from the provided address.
- `indexType` is a VkIndexType value specifying how indices are treated.

Valid Usage

- VUID-VkBindIndexBufferIndirectCommandEXT-None-11117
  The buffer's usage flags from which the address was acquired **must** have the `VK_BUFFER_USAGE_INDEX_BUFFER_BIT` bit set
- VUID-VkBindIndexBufferIndirectCommandEXT-bufferAddress-11118
  The `bufferAddress` **must** be aligned to the VkIndexType of the `indexType` used
- VUID-VkBindIndexBufferIndirectCommandEXT-None-11119
  Each element of the buffer from which the address was acquired and that is non-sparse **must** be bound completely and contiguously to a single `VkDeviceMemory` object

Valid Usage (Implicit)

- VUID-VkBindIndexBufferIndirectCommandEXT-indexType-parameter
  `indexType` **must** be a valid VkIndexType value

The `VkBindVertexBufferIndirectCommandEXT` structure specifies the input data for the `VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT` token.

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkBindVertexBufferIndirectCommandEXT {
    VkDeviceAddress    bufferAddress;
    uint32_t           size;
    uint32_t           stride;
} VkBindVertexBufferIndirectCommandEXT;
```

- `bufferAddress` specifies a physical address of the VkBuffer used as vertex input binding.
- `size` is the byte size range which is available for this operation from the provided address.
- `stride` is the byte size stride for this vertex input binding as in `VkVertexInputBindingDescription` ::`stride`.

Valid Usage

- VUID-VkBindVertexBufferIndirectCommandEXT-None-11120
  The buffer's usage flag from which the address was acquired **must** have the `VK_BUFFER_USAGE_VERTEX_BUFFER_BIT` bit set
- VUID-VkBindVertexBufferIndirectCommandEXT-None-11121
  Each element of the buffer from which the address was acquired and that is non-sparse

must be bound completely and contiguously to a single VkDeviceMemory object

The VkDrawIndirectCountIndirectCommandEXT structure specifies the input data for all draw-type tokens.

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkDrawIndirectCountIndirectCommandEXT {
    VkDeviceAddress    bufferAddress;
    uint32_t           stride;
    uint32_t           commandCount;
} VkDrawIndirectCountIndirectCommandEXT;
```

- bufferAddress specifies a physical address of the VkBuffer used for draw commands.
- stride is the byte size stride for the command arguments
- commandCount is the number of commands to execute

The corresponding indirect draw struct data will be read from the buffer address.

Valid Usage

- VUID-VkDrawIndirectCountIndirectCommandEXT-None-11122
  The buffer's usage flag from which the address was acquired **must** have the VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT bit set
- VUID-VkDrawIndirectCountIndirectCommandEXT-None-11123
  Each element of the buffer from which the address was acquired and that is non-sparse **must** be bound completely and contiguously to a single VkDeviceMemory object

30.1.3. Tokenized Command Processing

The processing for VK_EXT_device_generated_commands is in principle illustrated below:

```
void cmdProcessSequence(cmd, indirectExecutionSet, indirectCommandsLayout,
indirectAddress, s)
{
  for (t = 0; t < indirectCommandsLayout.tokenCount; t++)
  {
    uint32_t offset  = indirectCommandsLayout.pTokens[t].offset;
    uint32_t stride  = indirectCommandsLayout.indirectStride;
    VkDeviceAddress streamData = indirectAddress;
    const void* input = streamData + stride * s + offset;

    // further details later
    indirectCommandsLayout.pTokens[t].command (cmd, indirectExecutionSet, input, s);
  }
}
```

```
void cmdProcessAllSequences(cmd, indirectExecutionSet, indirectCommandsLayout,
indirectAddress, sequencesCount)
{
  for (s = 0; s < sequencesCount; s++)
  {
    sUsed = s;

    if (indirectCommandsLayout.flags &
VK_INDIRECT_COMMANDS_LAYOUT_USAGE_UNORDERED_SEQUENCES_BIT_EXT) {
      sUsed = incoherent_implementation_dependent_permutation[ sUsed ];
    }

    cmdProcessSequence( cmd, indirectExecutionSet, indirectCommandsLayout,
indirectAddress, sUsed );
  }
}
```

The processing of each sequence is considered stateless, therefore all state changes **must** occur prior to action commands within the sequence. A single sequence is strictly targeting the VkShaderStageFlags it was created with.

The primary input data for each token is provided through VkBuffer content at preprocessing using vkCmdPreprocessGeneratedCommandsEXT or execution time using vkCmdExecuteGeneratedCommandsEXT, however some functional arguments, for example push constant layouts, are specified at layout creation time. The input size is different for each token.

Possible values of those elements of the VkIndirectCommandsLayoutCreateInfoEXT::pTokens array specifying command tokens (other elements of the array specify command parameters) are:

```
// Provided by VK_EXT_device_generated_commands
typedef enum VkIndirectCommandsTokenTypeEXT {
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT = 0,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT = 1,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT = 2,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT = 3,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT = 4,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_EXT = 5,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_EXT = 6,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_COUNT_EXT = 7,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_COUNT_EXT = 8,
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_DISPATCH_EXT = 9,
  // Provided by VK_KHR_ray_tracing_maintenance1 with VK_EXT_device_generated_commands
    VK_INDIRECT_COMMANDS_TOKEN_TYPE_TRACE_RAYS2_EXT = 1000386004,
} VkIndirectCommandsTokenTypeEXT;
```

Table 41. Supported Indirect Command Tokens

Common Tokens	Command Data
VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT	u32[] array of indices into the indirect execution set
VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT	u32[] raw data
VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT	u32 placeholder data (not accessed by shader)
Compute Tokens	
VK_INDIRECT_COMMANDS_TOKEN_TYPE_DISPATCH_EXT	VkDispatchIndirectCommand
Ray Tracing Tokens	
VK_INDIRECT_COMMANDS_TOKEN_TYPE_TRACE_RAYS2_EXT	VkTraceRaysIndirectCommand2KHR
Graphics State Tokens	
VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT	VkBindIndexBufferIndirectCommandEXT
VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT	VkBindVertexBufferIndirectCommandEXT
Graphics Draw Tokens	
VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_EXT	VkDrawIndexedIndirectCommand
VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_EXT	VkDrawIndirectCommand
Graphics Draw Count Tokens	
VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_COUNT_EXT	VkDrawIndirectCountIndirectCommandEXT with VkDrawIndexedIndirectCommand
VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_COUNT_EXT	VkDrawIndirectCountIndirectCommandEXT with VkDrawIndirectCommand

The VkIndirectCommandsLayoutTokenEXT structure specifies details to the function arguments that need to be known at layout creation time:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectCommandsLayoutTokenEXT {
    VkStructureType                    sType;
    const void*                        pNext;
    VkIndirectCommandsTokenTypeEXT     type;
    VkIndirectCommandsTokenDataEXT     data;
    uint32_t                           offset;
} VkIndirectCommandsLayoutTokenEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `type` specifies the VkIndirectCommandsTokenTypeEXT for `data`.
- `data` specifies a VkIndirectCommandsTokenDataEXT containing token-specific details for command execution. It is ignored if `type` does not match any member of the VkIndirectCommandsTokenDataEXT union.
- `offset` is the relative byte offset for the token within one sequence of the indirect buffer. The data stored at that offset is the command data for the token, e.g. `VkDispatchIndirectCommand`.

Valid Usage

- VUID-VkIndirectCommandsLayoutTokenEXT-offset-11124
  `offset` **must** be less than or equal to VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`maxIndirectCommandsTokenOffset`
- VUID-VkIndirectCommandsLayoutTokenEXT-offset-11125
  `offset` **must** be aligned to `4`
- VUID-VkIndirectCommandsLayoutTokenEXT-rayTracingMaintenance1-11128
  If the `rayTracingMaintenance1` feature is not enabled, `type` **must** not be `VK_INDIRECT_COMMANDS_TOKEN_TYPE_TRACE_RAYS2_EXT`
- VUID-VkIndirectCommandsLayoutTokenEXT-deviceGeneratedCommandsMultiDrawIndirectCount-11129
  If VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`deviceGeneratedCommandsMultiDrawIndirectCount` is not supported, `type` **must** not be `VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_COUNT_EXT` or `VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_COUNT_EXT`

Valid Usage (Implicit)

- VUID-VkIndirectCommandsLayoutTokenEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_INDIRECT_COMMANDS_LAYOUT_TOKEN_EXT`
- VUID-VkIndirectCommandsLayoutTokenEXT-type-parameter
  `type` **must** be a valid VkIndirectCommandsTokenTypeEXT value

- VUID-VkIndirectCommandsLayoutTokenEXT-pPushConstant-parameter
  If `type` is `VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT`,VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT, the `pPushConstant` member of `data` **must** be a valid pointer to a valid VkIndirectCommandsPushConstantTokenEXT structure
- VUID-VkIndirectCommandsLayoutTokenEXT-pVertexBuffer-parameter
  If `type` is `VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT`, the `pVertexBuffer` member of `data` **must** be a valid pointer to a valid VkIndirectCommandsVertexBufferTokenEXT structure
- VUID-VkIndirectCommandsLayoutTokenEXT-pIndexBuffer-parameter
  If `type` is `VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT`, the `pIndexBuffer` member of `data` **must** be a valid pointer to a valid VkIndirectCommandsIndexBufferTokenEXT structure
- VUID-VkIndirectCommandsLayoutTokenEXT-pExecutionSet-parameter
  If `type` is `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT`, the `pExecutionSet` member of `data` **must** be a valid pointer to a valid VkIndirectCommandsExecutionSetTokenEXT structure

The `VkIndirectCommandsTokenDataEXT` structure provides token-specific details used to generate the indirect execution layout.

```
// Provided by VK_EXT_device_generated_commands
typedef union VkIndirectCommandsTokenDataEXT {
    const VkIndirectCommandsPushConstantTokenEXT*    pPushConstant;
    const VkIndirectCommandsVertexBufferTokenEXT*    pVertexBuffer;
    const VkIndirectCommandsIndexBufferTokenEXT*     pIndexBuffer;
    const VkIndirectCommandsExecutionSetTokenEXT*    pExecutionSet;
} VkIndirectCommandsTokenDataEXT;
```

- `pPushConstant` is a pointer to a VkIndirectCommandsPushConstantTokenEXT struct needed for `VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT` and `VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT` tokens
- `pVertexBuffer` is a pointer to a VkIndirectCommandsVertexBufferTokenEXT struct needed for `VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT` tokens
- `pIndexBuffer` is a pointer to a VkIndirectCommandsIndexBufferTokenEXT struct needed for `VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT` tokens
- `pExecutionSet` is a pointer to a VkIndirectCommandsExecutionSetTokenEXT struct needed for `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` tokens

The appropriate member of the union **must** be set for each token.

The following code provides detailed information on how an individual sequence is processed. For valid usage, all restrictions from the regular commands apply.

```
void cmdProcessSequence(cmd, indirectExecutionSet, indirectCommandsLayout,
indirectAddress, s)
{
  for (uint32_t t = 0; t < indirectCommandsLayout.tokenCount; t++) {
    VkIndirectCommandsLayoutTokenEXT *token = &indirectCommandsLayout.pTokens[t];

    uint32_t offset  = token->offset;
    uint32_t stride  = indirectCommandsLayout.indirectStride;
    VkDeviceAddress streamData = indirectAddress;
    const void* input = streamData + stride * s + offset;

    switch (token->tokenType) {
    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT:
      uint32_t *bind = input;
      VkIndirectCommandsExecutionSetTokenEXT *info = token->data.pExecutionSet;

      if (info->type == VK_INDIRECT_EXECUTION_SET_INFO_TYPE_PIPELINES_EXT) {
        vkCmdBindPipeline(cmd, indirectExecutionSet.pipelineBindPoint,
indirectExecutionSet.pipelines[*bind]);
      } else {
        VkShaderStageFlagBits stages[];
        VkShaderEXT shaders[];
        uint32_t i = 0;
        IterateBitmaskLSBToMSB(iter, info->shaderStages) {
            stages[i] = iter;
            shaders[i] = indirectExecutionSet.shaders[bind[i]].shaderObject;
            i++;
        }
        vkCmdBindShadersEXT(cmd, i, stages, shaders);
      }
      break;

    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT:
      uint32_t* data = input;
      VkPushConstantsInfoKHR info = {
        VK_STRUCTURE_TYPE_PUSH_CONSTANTS_INFO_KHR,
        // this can also use `dynamicGeneratedPipelineLayout' to pass a
VkPipelineLayoutCreateInfo from pNext
        indirectCommandsLayout.pipelineLayout,
        token->token.pushConstant.updateRange.shaderStages,
        token->token.pushConstant.updateRange.offset,
        token->token.pushConstant.updateRange.size,
        data
      };

      vkCmdPushConstants2KHR(cmd, &info);
      break;

    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT:
      VkPushConstantsInfoKHR info = {
```

```
        VK_STRUCTURE_TYPE_PUSH_CONSTANTS_INFO_KHR,
        // this can also use `dynamicGeneratedPipelineLayout' to pass a
VkPipelineLayoutCreateInfo from pNext
        indirectCommandsLayout.pipelineLayout,
        token->token.pushConstant.updateRange.shaderStages,
        token->token.pushConstant.updateRange.offset,
        // this must be 4
        token->token.pushConstant.updateRange.size,
        // this just updates the sequence index
        &s
      };

      vkCmdPushConstants2KHR(cmd, &info);
      break;

    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT:
      VkBindIndexBufferIndirectCommandEXT* data = input;

      vkCmdBindIndexBuffer(cmd, deriveBuffer(data->bufferAddress), deriveOffset(data-
>bufferAddress), data->indexType);
      break;

    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT:
      VkBindVertexBufferIndirectCommandEXT* data = input;

      vkCmdBindVertexBuffers2(cmd, token->token.vertexBuffer->vertexBindingUnit, 1,
&deriveBuffer(data->bufferAddress),
                              &deriveOffset(data->bufferAddress), data->size, data-
>stride);
      break;

    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_EXT:
      VkDrawIndexedIndirectCommand *data = input;

      vkCmdDrawIndexed(cmd, data->indexCount, data->instanceCount, data->firstIndex,
data->vertexOffset, data->firstInstance);
      break;
    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_INDEXED_COUNT_EXT:
      VkDrawIndirectCountIndirectCommandEXT* data = input;

      vkCmdDrawIndexedIndirect(cmd, deriveBuffer(data->bufferAddress),  deriveoffset
(data->bufferAddress), min(data->commandCount, indirectCommandsLayout.maxDrawCount),
data->stride);
      break;

    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_EXT:
      VkDrawIndirectCommand* data = input;

      vkCmdDraw(cmd, data->vertex_count, data->instanceCount, data->firstVertex, data
->firstIndex);
      break;
```

```
    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_COUNT_EXT:
      VkDrawIndirectCountIndirectCommandEXT* data = input;

      vkCmdDrawIndirect(cmd, deriveBuffer(data->bufferAddress), deriveoffset(data-
>bufferAddress), min(data->commandCount, indirectCommandsLayout.maxDrawCount), data-
>stride);
      break;

    // only available if VK_NV_mesh_shader is enabled
    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_MESH_TASKS_NV_EXT:
      VkDrawMeshTasksIndirectCommandNV *data = input;

      vkCmdDrawMeshTasksNV(cmd, data->taskCount, data->firstTask);
     break;

    // only available if VK_NV_mesh_shader is enabled
    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_MESH_TASKS_COUNT_NV_EXT:
      VkDrawIndirectCountIndirectCommandEXT* data = input;

      vkCmdDrawMeshTasksIndirectCountNV(cmd, deriveBuffer(data->bufferAddress),
deriveoffset(data->bufferAddress), min(data->commandCount, indirectCommandsLayout
.maxDrawCount), data->stride);
      break;

    // only available if VK_EXT_mesh_shader is enabled
    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_MESH_TASKS_EXT:
      VkDrawMeshTasksIndirectCommandEXT *data = input;

      vkCmdDrawMeshTasksEXT(cmd, data->groupCountX, data->groupCountY, data-
>groupCountZ);
     break;

    // only available if VK_EXT_mesh_shader is enabled
    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DRAW_MESH_TASKS_COUNT_EXT:
      VkDrawIndirectCountIndirectCommandEXT* data = input;

      vkCmdDrawMeshTasksIndirectCountEXT(cmd, deriveBuffer(data->bufferAddress),
deriveoffset(data->bufferAddress), min(data->commandCount, indirectCommandsLayout
.maxDrawCount), data->stride);
      break;

    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_DISPATCH_EXT:
      VkDispatchIndirectCommand *data = input;

      vkCmdDispatch(cmd, data->x, data->y, data->z);
      break;

    // only available if VK_KHR_ray_tracing_maintenance1 is enabled
    case VK_INDIRECT_COMMANDS_TOKEN_TYPE_TRACE_RAYS2_EXT:
      vkCmdTraceRaysIndirect2KHR(cmd, deriveBuffer(input));
```

```
      break;
    }
  }
}
```

The VkIndirectCommandsPushConstantTokenEXT structure specifies the layout token info for VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT and VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT tokens.

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectCommandsPushConstantTokenEXT {
    VkPushConstantRange    updateRange;
} VkIndirectCommandsPushConstantTokenEXT;
```

- updateRange is the push constant range that will be updated by the token.

The stageFlags member of updateRange is ignored.

Valid Usage

- VUID-VkIndirectCommandsPushConstantTokenEXT-updateRange-11132
  updateRange **must** be contained within the push constant info used by VkIndirectCommandsLayoutCreateInfoEXT
- VUID-VkIndirectCommandsPushConstantTokenEXT-size-11133
  If the token type is VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT, the size member of updateRange **must** be 4

Valid Usage (Implicit)

- VUID-VkIndirectCommandsPushConstantTokenEXT-updateRange-parameter
  updateRange **must** be a valid VkPushConstantRange structure

The VkIndirectCommandsVertexBufferTokenEXT structure specifies the layout token info for the VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT token.

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectCommandsVertexBufferTokenEXT {
    uint32_t    vertexBindingUnit;
} VkIndirectCommandsVertexBufferTokenEXT;
```

- vertexBindingUnit is the vertex input binding number to be bound.

Valid Usage

- VUID-VkIndirectCommandsVertexBufferTokenEXT-vertexBindingUnit-11134
  `vertexBindingUnit` **must** be less than the total number of vertex input bindings in use by the current graphics state

The `VkIndirectCommandsIndexBufferTokenEXT` structure specifies the layout token info for the `VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT` token.

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectCommandsIndexBufferTokenEXT {
    VkIndirectCommandsInputModeFlagBitsEXT    mode;
} VkIndirectCommandsIndexBufferTokenEXT;
```

- `mode` specifies the mode to use with this token.

This allows for easy layering of Vulkan atop other APIs. When `VK_INDIRECT_COMMANDS_INPUT_MODE_DXGI_INDEX_BUFFER_EXT` is specified, the indirect buffer can contain a `D3D12_INDEX_BUFFER_VIEW` instead of VkBindIndexBufferIndirectCommandEXT as D3D's DXGI format value is mapped to the VkIndexType. It works as both structs are otherwise binary compatible.

Valid Usage

- VUID-VkIndirectCommandsIndexBufferTokenEXT-mode-11135
  `mode` **must** be non-zero
- VUID-VkIndirectCommandsIndexBufferTokenEXT-mode-11136
  `mode` **must** be one of the bits supported in VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`supportedIndirectCommandsInputModes`

Valid Usage (Implicit)

- VUID-VkIndirectCommandsIndexBufferTokenEXT-mode-parameter
  `mode` **must** be a valid VkIndirectCommandsInputModeFlagBitsEXT value

Bits which are set in VkIndirectCommandsIndexBufferTokenEXT::`mode`, specifying how an index buffer is used, are:

```
// Provided by VK_EXT_device_generated_commands
typedef enum VkIndirectCommandsInputModeFlagBitsEXT {
    VK_INDIRECT_COMMANDS_INPUT_MODE_VULKAN_INDEX_BUFFER_EXT = 0x00000001,
    VK_INDIRECT_COMMANDS_INPUT_MODE_DXGI_INDEX_BUFFER_EXT = 0x00000002,
} VkIndirectCommandsInputModeFlagBitsEXT;
```

- VK_INDIRECT_COMMANDS_INPUT_MODE_VULKAN_INDEX_BUFFER_EXT specifies that the indirect buffer contains VkBindIndexBufferIndirectCommandEXT.
- VK_INDIRECT_COMMANDS_INPUT_MODE_DXGI_INDEX_BUFFER_EXT specifies that the indirect buffer contains D3D12_INDEX_BUFFER_VIEW.

```
// Provided by VK_EXT_device_generated_commands
typedef VkFlags VkIndirectCommandsInputModeFlagsEXT;
```

VkIndirectCommandsInputModeFlagsEXT is a bitmask type for setting a mask of zero or more VkIndirectCommandsInputModeFlagBitsEXT.

The VkIndirectCommandsExecutionSetTokenEXT structure specifies the input data for the VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT token.

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectCommandsExecutionSetTokenEXT {
    VkIndirectExecutionSetInfoTypeEXT    type;
    VkShaderStageFlags                   shaderStages;
} VkIndirectCommandsExecutionSetTokenEXT;
```

- type describes the type of indirect execution set in use.
- shaderStages specifies the shaders that will be changed by this token.

Valid Usage

- VUID-VkIndirectCommandsExecutionSetTokenEXT-shaderStages-11137
  Each bit in shaderStages **must** be supported by VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::supportedIndirectCommandsShaderStagesPipelineBinding or VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::supportedIndirectCommandsShaderStagesShaderBinding

Valid Usage (Implicit)

- VUID-VkIndirectCommandsExecutionSetTokenEXT-type-parameter
  type **must** be a valid VkIndirectExecutionSetInfoTypeEXT value
- VUID-VkIndirectCommandsExecutionSetTokenEXT-shaderStages-parameter

`shaderStages` **must** be a valid combination of VkShaderStageFlagBits values

- VUID-VkIndirectCommandsExecutionSetTokenEXT-shaderStages-requiredbitmask
  `shaderStages` **must** not be `0`

30.2. Indirect Commands Generation and Execution

The generation of commands on the device requires a `preprocess` buffer.

With `VK_EXT_device_generated_commands`, to retrieve the memory size and alignment requirements of a particular execution state call:

```
// Provided by VK_EXT_device_generated_commands
void vkGetGeneratedCommandsMemoryRequirementsEXT(
    VkDevice                                    device,
    const VkGeneratedCommandsMemoryRequirementsInfoEXT* pInfo,
    VkMemoryRequirements2*                      pMemoryRequirements);
```

- `device` is the logical device that owns the buffer.
- `pInfo` is a pointer to a VkGeneratedCommandsMemoryRequirementsInfoEXT structure containing parameters required for the memory requirements query.
- `pMemoryRequirements` is a pointer to a VkMemoryRequirements2 structure in which the memory requirements of the buffer object are returned.

If the size returned is zero, the preprocessing step can be skipped for this layout.

Valid Usage (Implicit)

- VUID-vkGetGeneratedCommandsMemoryRequirementsEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetGeneratedCommandsMemoryRequirementsEXT-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkGeneratedCommandsMemoryRequirementsInfoEXT structure
- VUID-vkGetGeneratedCommandsMemoryRequirementsEXT-pMemoryRequirements-parameter
  `pMemoryRequirements` **must** be a valid pointer to a VkMemoryRequirements2 structure

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkGeneratedCommandsMemoryRequirementsInfoEXT {
    VkStructureType                sType;
    const void*                    pNext;
    VkIndirectExecutionSetEXT      indirectExecutionSet;
    VkIndirectCommandsLayoutEXT    indirectCommandsLayout;
    uint32_t                       maxSequenceCount;
    uint32_t                       maxDrawCount;
} VkGeneratedCommandsMemoryRequirementsInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `indirectExecutionSet` is the indirect execution set to be used for binding shaders.
- `indirectCommandsLayout` is the VkIndirectCommandsLayoutEXT that this buffer memory is intended to be used with.
- `maxSequenceCount` is the maximum number of sequences that this buffer memory can be used with.
- `maxDrawCount` is the maximum number of indirect draws that can be executed by any COUNT-type multi-draw indirect tokens. The draw count in the indirect buffer is clamped to this value for these token types.

If the action command token for the layout is not a COUNT-type multi-draw indirect token, `maxDrawCount` is ignored.

Valid Usage

- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-maxSequencesCount-11009
  `maxSequencesCount` **must** be less or equal to VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`maxIndirectSequenceCount`
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-indirectCommandsLayout-11010
  If `indirectCommandsLayout` was created with a token sequence that contained the `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` token, `indirectExecutionSet` **must** not be VK_NULL_HANDLE
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-indirectCommandsLayout-11151
  If `indirectCommandsLayout` was created with a token sequence that contained the `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` token, the shader stages used to create the initial shader state of `indirectExecutionSet` **must** equal the VkIndirectCommandsExecutionSetTokenEXT::`shaderStages` used to create `indirectCommandsLayout`
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-indirectCommandsLayout-11011
  If `indirectCommandsLayout` was not created with a token sequence that contained the

VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT token, indirectExecutionSet **must** be VK_NULL_HANDLE

- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-maxDrawCount-11146
  When not ignored, maxDrawCount × maxSequenceCount **must** be less than 2^24
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-indirectExecutionSet-11012
  If indirectExecutionSet is VK_NULL_HANDLE, either a VkGeneratedCommandsPipelineInfoEXT or a VkGeneratedCommandsShaderInfoEXT **must** be included in the pNext chain

Valid Usage (Implicit)

- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_GENERATED_COMMANDS_MEMORY_REQUIREMENTS_INFO_EXT
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkGeneratedCommandsPipelineInfoEXT or VkGeneratedCommandsShaderInfoEXT
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-indirectExecutionSet-parameter
  If indirectExecutionSet is not VK_NULL_HANDLE, indirectExecutionSet **must** be a valid VkIndirectExecutionSetEXT handle
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-indirectCommandsLayout-parameter
  indirectCommandsLayout **must** be a valid VkIndirectCommandsLayoutEXT handle
- VUID-VkGeneratedCommandsMemoryRequirementsInfoEXT-commonparent
  Both of indirectCommandsLayout, and indirectExecutionSet that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkGeneratedCommandsPipelineInfoEXT {
    VkStructureType    sType;
    void*              pNext;
    VkPipeline         pipeline;
} VkGeneratedCommandsPipelineInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pipeline is a valid pipeline object.

Valid Usage (Implicit)

- VUID-VkGeneratedCommandsPipelineInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_GENERATED_COMMANDS_PIPELINE_INFO_EXT`
- VUID-VkGeneratedCommandsPipelineInfoEXT-pipeline-parameter
  `pipeline` **must** be a valid VkPipeline handle

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkGeneratedCommandsShaderInfoEXT {
    VkStructureType       sType;
    void*                 pNext;
    uint32_t              shaderCount;
    const VkShaderEXT*    pShaders;
} VkGeneratedCommandsShaderInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `shaderCount` is the size of the `pShaders` array.
- `pShaders` is a pointer to an array of shader objects.

Valid Usage

- VUID-VkGeneratedCommandsShaderInfoEXT-pShaders-11127
  `pShaders` **must** not contain more than one shader object for a given VkShaderStageFlagBits stage

Valid Usage (Implicit)

- VUID-VkGeneratedCommandsShaderInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_GENERATED_COMMANDS_SHADER_INFO_EXT`
- VUID-VkGeneratedCommandsShaderInfoEXT-pShaders-parameter
  `pShaders` **must** be a valid pointer to an array of `shaderCount` valid VkShaderEXT handles
- VUID-VkGeneratedCommandsShaderInfoEXT-shaderCount-arraylength
  `shaderCount` **must** be greater than `0`

30.2.1. Indirect Execution Sets

Indirect Execution Sets contain sets of pipelines or shader objects which can be bound individually.

```
// Provided by VK_EXT_device_generated_commands
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkIndirectExecutionSetEXT)
```

Indirect Execution Sets allow the device to bind different shaders and pipeline states using Device-Generated Commands.

Indirect Execution Sets are created by calling:

```
// Provided by VK_EXT_device_generated_commands
VkResult vkCreateIndirectExecutionSetEXT(
    VkDevice                                    device,
    const VkIndirectExecutionSetCreateInfoEXT*  pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkIndirectExecutionSetEXT*                  pIndirectExecutionSet);
```

- `device` is the logical device that creates the indirect execution set.
- `pCreateInfo` is a pointer to a VkIndirectExecutionSetCreateInfoEXT structure containing parameters affecting creation of the indirect execution set.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pIndirectExecutionSet` is a pointer to a VkIndirectExecutionSetEXT handle in which the resulting indirect execution set is returned.

Valid Usage

- VUID-vkCreateIndirectExecutionSetEXT-deviceGeneratedCommands-11013
  The `VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT::deviceGeneratedCommands` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCreateIndirectExecutionSetEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateIndirectExecutionSetEXT-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkIndirectExecutionSetCreateInfoEXT structure
- VUID-vkCreateIndirectExecutionSetEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateIndirectExecutionSetEXT-pIndirectExecutionSet-parameter
  `pIndirectExecutionSet` **must** be a valid pointer to a VkIndirectExecutionSetEXT handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkIndirectExecutionSetCreateInfoEXT structure is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectExecutionSetCreateInfoEXT {
    VkStructureType                       sType;
    const void*                           pNext;
    VkIndirectExecutionSetInfoTypeEXT     type;
    VkIndirectExecutionSetInfoEXT         info;
} VkIndirectExecutionSetCreateInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- type is a VkIndirectExecutionSetInfoTypeEXT describing the type of set being created and determining which field of the info union will be used.
- info is a VkIndirectExecutionSetInfoEXT union containing layout information for the set.

Valid Usage

- VUID-VkIndirectExecutionSetCreateInfoEXT-maxIndirectShaderObjectCount-11014
  If VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::maxIndirectShaderObjectCount is zero or the shaderObject feature is not enabled type **must** not be VK_INDIRECT_EXECUTION_SET_INFO_TYPE_SHADER_OBJECTS_EXT

Valid Usage (Implicit)

- VUID-VkIndirectExecutionSetCreateInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_CREATE_INFO_EXT
- VUID-VkIndirectExecutionSetCreateInfoEXT-type-parameter
  type **must** be a valid VkIndirectExecutionSetInfoTypeEXT value
- VUID-VkIndirectExecutionSetCreateInfoEXT-pPipelineInfo-parameter
  If type is VK_INDIRECT_EXECUTION_SET_INFO_TYPE_PIPELINES_EXT, the pPipelineInfo member of info **must** be a valid pointer to a valid VkIndirectExecutionSetPipelineInfoEXT structure

- VUID-VkIndirectExecutionSetCreateInfoEXT-pShaderInfo-parameter
  If `type` is `VK_INDIRECT_EXECUTION_SET_INFO_TYPE_SHADER_OBJECTS_EXT`, the `pShaderInfo` member of `info` **must** be a valid pointer to a valid VkIndirectExecutionSetShaderInfoEXT structure

Values which **can** be set in VkIndirectExecutionSetCreateInfoEXT::`type`, specifying contents of an indirect execution set, are:

```
// Provided by VK_EXT_device_generated_commands
typedef enum VkIndirectExecutionSetInfoTypeEXT {
    VK_INDIRECT_EXECUTION_SET_INFO_TYPE_PIPELINES_EXT = 0,
    VK_INDIRECT_EXECUTION_SET_INFO_TYPE_SHADER_OBJECTS_EXT = 1,
} VkIndirectExecutionSetInfoTypeEXT;
```

- `VK_INDIRECT_EXECUTION_SET_INFO_TYPE_PIPELINES_EXT` specifies that the indirect execution set contains VkPipeline objects.
- `VK_INDIRECT_EXECUTION_SET_INFO_TYPE_SHADER_OBJECTS_EXT` specifies that the indirect execution set contains VkShaderEXT objects.

The `VkIndirectExecutionSetInfoEXT` union is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef union VkIndirectExecutionSetInfoEXT {
    const VkIndirectExecutionSetPipelineInfoEXT*    pPipelineInfo;
    const VkIndirectExecutionSetShaderInfoEXT*      pShaderInfo;
} VkIndirectExecutionSetInfoEXT;
```

- `pPipelineInfo` is a pointer to a VkIndirectExecutionSetPipelineInfoEXT struct containing pipeline layout information for the set.
- `pShaderInfo` is a pointer to a VkIndirectExecutionSetShaderInfoEXT struct containing shader object layout information for the set.

The `VkIndirectExecutionSetPipelineInfoEXT` structure is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectExecutionSetPipelineInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkPipeline         initialPipeline;
    uint32_t           maxPipelineCount;
} VkIndirectExecutionSetPipelineInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.

- `initialPipeline` is the initial pipeline for the set. This pipeline will be automatically added to the set at index `0`.
- `maxPipelineCount` is the maximum number of pipelines stored in the set.

The characteristics of `initialPipeline` will be used to validate all pipelines added to the set even if they are removed from the set or destroyed.

When an Indirect Execution Set created with pipelines is used, `initialPipeline` constitutes the initial shader state.

Valid Usage

- VUID-VkIndirectExecutionSetPipelineInfoEXT-supportedIndirectCommandsShaderStagesPipelineBinding-11015
  If VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`supportedIndirectCommandsShaderStagesPipelineBinding` does not contain `VK_SHADER_STAGE_COMPUTE_BIT`, the `VkPipelineBindPoint` of `initialPipeline` **must** not be `VK_PIPELINE_BIND_POINT_COMPUTE`
- VUID-VkIndirectExecutionSetPipelineInfoEXT-supportedIndirectCommandsShaderStagesPipelineBinding-11016
  If VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`supportedIndirectCommandsShaderStagesPipelineBinding` does not contain `VK_SHADER_STAGE_FRAGMENT_BIT`, the `VkPipelineBindPoint` of `initialPipeline` **must** not be `VK_PIPELINE_BIND_POINT_GRAPHICS`
- VUID-VkIndirectExecutionSetPipelineInfoEXT-supportedIndirectCommandsShaderStagesPipelineBinding-11017
  If VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`supportedIndirectCommandsShaderStagesPipelineBinding` does not contain ray tracing stages, the `VkPipelineBindPoint` of `initialPipeline` **must** not be `VK_PIPELINE_BIND_POINT_RAY_TRACING_KHR`
- VUID-VkIndirectExecutionSetPipelineInfoEXT-maxPipelineCount-11018
  `maxPipelineCount` **must** be between `1` and `VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT`::`maxIndirectPipelineCount`
- VUID-VkIndirectExecutionSetPipelineInfoEXT-initialPipeline-11019
  `initialPipeline` **must** not use descriptors of type `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` or `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC`
- VUID-VkIndirectExecutionSetPipelineInfoEXT-initialPipeline-11153
  `initialPipeline` **must** have been created with `VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT`

Valid Usage (Implicit)

- VUID-VkIndirectExecutionSetPipelineInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_PIPELINE_INFO_EXT`

- VUID-VkIndirectExecutionSetPipelineInfoEXT-initialPipeline-parameter
  `initialPipeline` **must** be a valid VkPipeline handle

The `VkIndirectExecutionSetShaderInfoEXT` structure is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectExecutionSetShaderInfoEXT {
    VkStructureType                                 sType;
    const void*                                     pNext;
    uint32_t                                        shaderCount;
    const VkShaderEXT*                              pInitialShaders;
    const VkIndirectExecutionSetShaderLayoutInfoEXT*    pSetLayoutInfos;
    uint32_t                                        maxShaderCount;
    uint32_t                                        pushConstantRangeCount;
    const VkPushConstantRange*                      pPushConstantRanges;
} VkIndirectExecutionSetShaderInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `shaderCount` is the number of members in the `pInitialShaders` and `pSetLayoutInfos` arrays.
- `pInitialShaders` is a pointer to an array containing a VkShaderEXT object for each shader stage that will be used in the set. These shaders will be automatically added to the set beginning at index `0`.
- `pSetLayoutInfos` is a pointer to an array containing a VkIndirectExecutionSetShaderLayoutInfoEXT used by each corresponding `pInitialShaders` shader stage in the set.
- `maxShaderCount` is the maximum number of shader objects stored in the set.
- `pushConstantRangeCount` is the number of members in the `pPushConstantRanges` array.
- `pPushConstantRanges` is a pointer to the array of VkPushConstantRange ranges used by all shaders in the set.

The characteristics of `pInitialShaders` will be used to validate all shaders added to the set even if they are removed from the set or destroyed.

When an Indirect Execution Set created with shader objects is used, `pInitialShaders` constitutes the initial shader state.

Valid Usage

- VUID-VkIndirectExecutionSetShaderInfoEXT-pInitialShaders-11020
  All members of `pInitialShaders` **must** have a `stage` supported by VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`supportedIndirectCommandsShaderStagesShaderBinding`
- VUID-VkIndirectExecutionSetShaderInfoEXT-maxShaderCount-11021

maxShaderCount **must** not be zero

- VUID-VkIndirectExecutionSetShaderInfoEXT-maxShaderCount-11022
  maxShaderCount **must** be less than or equal to VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::maxIndirectShaderObjectCount
- VUID-VkIndirectExecutionSetShaderInfoEXT-maxShaderCount-11036
  maxShaderCount **must** be greater than or equal to shaderCount
- VUID-VkIndirectExecutionSetShaderInfoEXT-stage-11023
  The stage of each element in the pInitialShaders array **must** be unique
- VUID-VkIndirectExecutionSetShaderInfoEXT-pInitialShaders-11154
  Each member of pInitialShaders **must** have been created with VK_SHADER_CREATE_INDIRECT_BINDABLE_BIT_EXT

Valid Usage (Implicit)

- VUID-VkIndirectExecutionSetShaderInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_SHADER_INFO_EXT
- VUID-VkIndirectExecutionSetShaderInfoEXT-pInitialShaders-parameter
  pInitialShaders **must** be a valid pointer to an array of shaderCount valid VkShaderEXT handles
- VUID-VkIndirectExecutionSetShaderInfoEXT-pSetLayoutInfos-parameter
  If pSetLayoutInfos is not NULL, pSetLayoutInfos **must** be a valid pointer to an array of shaderCount valid VkIndirectExecutionSetShaderLayoutInfoEXT structures
- VUID-VkIndirectExecutionSetShaderInfoEXT-pPushConstantRanges-parameter
  If pushConstantRangeCount is not 0, pPushConstantRanges **must** be a valid pointer to an array of pushConstantRangeCount valid VkPushConstantRange structures
- VUID-VkIndirectExecutionSetShaderInfoEXT-shaderCount-arraylength
  shaderCount **must** be greater than 0

The VkIndirectExecutionSetShaderLayoutInfoEXT structure is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkIndirectExecutionSetShaderLayoutInfoEXT {
    VkStructureType                 sType;
    const void*                     pNext;
    uint32_t                        setLayoutCount;
    const VkDescriptorSetLayout*    pSetLayouts;
} VkIndirectExecutionSetShaderLayoutInfoEXT;
```

- setLayoutCount is the number of members in the pSetLayouts array
- pSetLayouts is a pointer to an array containing VkDescriptorSetLayout objects used by the shader stage.

Valid Usage

- VUID-VkIndirectExecutionSetShaderLayoutInfoEXT-pSetLayouts-11024
  All members of `pSetLayouts` **must** not contain descriptors of type `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` or `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC`

Valid Usage (Implicit)

- VUID-VkIndirectExecutionSetShaderLayoutInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_SHADER_LAYOUT_INFO_EXT`
- VUID-VkIndirectExecutionSetShaderLayoutInfoEXT-pSetLayouts-parameter
  If `setLayoutCount` is not `0`, `pSetLayouts` **must** be a valid pointer to an array of `setLayoutCount` valid or VK_NULL_HANDLE VkDescriptorSetLayout handles

Destroy an Indirect Execution Set by calling:

```
// Provided by VK_EXT_device_generated_commands
void vkDestroyIndirectExecutionSetEXT(
    VkDevice                                    device,
    VkIndirectExecutionSetEXT                   indirectExecutionSet,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that owns the indirect execution set.
- `indirectExecutionSet` is the indirect execution set to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyIndirectExecutionSetEXT-indirectExecutionSet-11025
  All submitted commands that refer to `indirectExecutionSet` **must** have completed execution

Valid Usage (Implicit)

- VUID-vkDestroyIndirectExecutionSetEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyIndirectExecutionSetEXT-indirectExecutionSet-parameter
  If `indirectExecutionSet` is not VK_NULL_HANDLE, `indirectExecutionSet` **must** be a valid VkIndirectExecutionSetEXT handle
- VUID-vkDestroyIndirectExecutionSetEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid

VkAllocationCallbacks structure

- VUID-vkDestroyIndirectExecutionSetEXT-indirectExecutionSet-parent
  If `indirectExecutionSet` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `indirectExecutionSet` **must** be externally synchronized

The `VkWriteIndirectExecutionSetPipelineEXT` struct is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkWriteIndirectExecutionSetPipelineEXT {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           index;
    VkPipeline         pipeline;
} VkWriteIndirectExecutionSetPipelineEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `index` is the element of the set to update
- `pipeline` is the pipeline to store in the indirect execution set

Valid Usage

- VUID-VkWriteIndirectExecutionSetPipelineEXT-index-11026
  `index` **must** be less than the value of `VkIndirectExecutionSetPipelineInfoEXT::maxPipelineCount` used to create the set
- VUID-VkWriteIndirectExecutionSetPipelineEXT-pipeline-11027
  `pipeline` **must** have been created with `VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT`
- VUID-VkWriteIndirectExecutionSetPipelineEXT-index-11029
  `index` **must** not be referenced by submitted command buffers
- VUID-VkWriteIndirectExecutionSetPipelineEXT-pipeline-11030
  The shader stages contained in `pipeline` **must** be supported by VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`supportedIndirectCommandsShaderStagesPipelineBinding`

Valid Usage (Implicit)

- VUID-VkWriteIndirectExecutionSetPipelineEXT-sType-sType

`sType` **must** be `VK_STRUCTURE_TYPE_WRITE_INDIRECT_EXECUTION_SET_PIPELINE_EXT`

- VUID-VkWriteIndirectExecutionSetPipelineEXT-pipeline-parameter
  `pipeline` **must** be a valid VkPipeline handle

The `VkWriteIndirectExecutionSetShaderEXT` struct is defined as:

```
// Provided by VK_EXT_device_generated_commands with VK_EXT_shader_object
typedef struct VkWriteIndirectExecutionSetShaderEXT {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           index;
    VkShaderEXT        shader;
} VkWriteIndirectExecutionSetShaderEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `index` is the element of the set to update
- `shader` is the shader to store in the indirect execution set

Shaders need not be stored in the Indirect Execution Set according to their stage. The only restriction for shader indices within a set is that the value of the index **must** be less than the maximum number of shaders in the set.

Valid Usage

- VUID-VkWriteIndirectExecutionSetShaderEXT-index-11031
  `index` **must** be less than `VkIndirectExecutionSetShaderInfoEXT`::`maxShaderCount`
- VUID-VkWriteIndirectExecutionSetShaderEXT-shader-11032
  `shader` **must** have been created with `VK_SHADER_CREATE_INDIRECT_BINDABLE_BIT_EXT`
- VUID-VkWriteIndirectExecutionSetShaderEXT-pInitialShaders-11033
  A shader created with the same VkShaderStageFlagBits **must** have been passed in the `VkIndirectExecutionSetShaderInfoEXT`::`pInitialShaders` array
- VUID-VkWriteIndirectExecutionSetShaderEXT-index-11034
  `index` **must** not be in use by submitted command buffers

Valid Usage (Implicit)

- VUID-VkWriteIndirectExecutionSetShaderEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_WRITE_INDIRECT_EXECUTION_SET_SHADER_EXT`
- VUID-VkWriteIndirectExecutionSetShaderEXT-shader-parameter
  `shader` **must** be a valid VkShaderEXT handle

Pipeline elements in an Indirect Execution Set can be updated by calling:

```
// Provided by VK_EXT_device_generated_commands
void vkUpdateIndirectExecutionSetPipelineEXT(
    VkDevice                                    device,
    VkIndirectExecutionSetEXT                   indirectExecutionSet,
    uint32_t                                    executionSetWriteCount,
    const VkWriteIndirectExecutionSetPipelineEXT* pExecutionSetWrites);
```

- `device` is the logical device that owns the indirect execution set.
- `indirectExecutionSet` is the indirect execution set being updated.
- `executionSetWriteCount` is the number of elements in the `pExecutionSetWrites` array.
- `pExecutionSetWrites` is a pointer to an array of VkWriteIndirectExecutionSetPipelineEXT structures describing the elements to update.

Valid Usage

- VUID-vkUpdateIndirectExecutionSetPipelineEXT-indirectExecutionSet-11035
  `indirectExecutionSet` **must** have been created with type `VK_INDIRECT_EXECUTION_SET_INFO_TYPE_PIPELINES_EXT`
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-executionSetWriteCount-11037
  `executionSetWriteCount` **must** be less than or equal to `VkIndirectExecutionSetPipelineInfoEXT`::`maxPipelineCount`
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-pExecutionSetWrites-11042
  Each element in the `pExecutionSetWrites` array must have a unique `VkWriteIndirectExecutionSetPipelineEXT`::`index`
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-None-11038
  Each member of the Indirect Execution Set referenced by the update command **must** not be in use by the device
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-None-11039
  The layout of each pipeline in `pExecutionSetWrites` **must** be compatible with the `initialPipeline` used to create the Indirect Execution Set
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-None-11040
  Each pipeline in the Indirect Execution Set **must** have identically defined static and dynamic state values to the `initialPipeline` used to create the Indirect Execution Set
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-initialPipeline-11147
  Each pipeline in the Indirect Execution Set **must** have identically defined fragment outputs interface to the `initialPipeline` used to create the Indirect Execution Set
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-initialPipeline-11152
  Each pipeline in the Indirect Execution Set **must** match the `initialPipeline` used to create the Indirect Execution Set in its included shader stages
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-initialPipeline-11098
  Each pipeline in the Indirect Execution Set **must** match the `initialPipeline` used to create

the Indirect Execution Set in its use of `FragDepth`

- VUID-vkUpdateIndirectExecutionSetPipelineEXT-initialPipeline-11086
  Each pipeline in the Indirect Execution Set **must** match the `initialPipeline` used to create the Indirect Execution Set in its use of `SampleMask`
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-initialPipeline-11085
  Each pipeline in the Indirect Execution Set **must** match the `initialPipeline` used to create the Indirect Execution Set in its use of `StencilExportEXT`

Valid Usage (Implicit)

- VUID-vkUpdateIndirectExecutionSetPipelineEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-indirectExecutionSet-parameter
  `indirectExecutionSet` **must** be a valid VkIndirectExecutionSetEXT handle
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-pExecutionSetWrites-parameter
  `pExecutionSetWrites` **must** be a valid pointer to an array of `executionSetWriteCount` valid VkWriteIndirectExecutionSetPipelineEXT structures
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-executionSetWriteCount-arraylength
  `executionSetWriteCount` **must** be greater than `0`
- VUID-vkUpdateIndirectExecutionSetPipelineEXT-indirectExecutionSet-parent
  `indirectExecutionSet` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `indirectExecutionSet` **must** be externally synchronized

Shader object elements in an Indirect Execution Set can be updated by calling:

```
// Provided by VK_EXT_device_generated_commands
void vkUpdateIndirectExecutionSetShaderEXT(
    VkDevice                                    device,
    VkIndirectExecutionSetEXT                   indirectExecutionSet,
    uint32_t                                    executionSetWriteCount,
    const VkWriteIndirectExecutionSetShaderEXT* pExecutionSetWrites);
```

- `device` is the logical device that owns the indirect execution set.
- `indirectExecutionSet` is the indirect execution set being updated.
- `executionSetWriteCount` is the number of elements in the `pExecutionSetWrites` array.
- `pExecutionSetWrites` is a pointer to an array of VkWriteIndirectExecutionSetShaderEXT structures describing the elements to update.

Valid Usage

- VUID-vkUpdateIndirectExecutionSetShaderEXT-indirectExecutionSet-11041
  `indirectExecutionSet` **must** have been created with type `VK_INDIRECT_EXECUTION_SET_INFO_TYPE_SHADER_OBJECTS_EXT`
- VUID-vkUpdateIndirectExecutionSetShaderEXT-pExecutionSetWrites-11043
  Each element in the `pExecutionSetWrites` array must have a unique `VkWriteIndirectExecutionSetShaderEXT`::`index`
- VUID-vkUpdateIndirectExecutionSetShaderEXT-None-11044
  Each member of the Indirect Execution Set referenced by the update command **must** not be in use by the device
- VUID-vkUpdateIndirectExecutionSetShaderEXT-pExecutionSetWrites-11140
  The descriptor layout of each shader in `pExecutionSetWrites` **must** be compatible with the initial layout info used to create the Indirect Execution Set
- VUID-vkUpdateIndirectExecutionSetShaderEXT-None-11148
  Each fragment shader element in the Indirect Execution Set **must** have identically defined fragment outputs interface to the initial shader state used to create the Indirect Execution Set
- VUID-vkUpdateIndirectExecutionSetShaderEXT-FragDepth-11054
  Each fragment shader element in the Indirect Execution Set **must** match the initial shader state used to create the Indirect Execution Set in its use of `FragDepth`
- VUID-vkUpdateIndirectExecutionSetShaderEXT-SampleMask-11050
  Each fragment shader element in the Indirect Execution Set **must** match the initial shader state used to create the Indirect Execution Set in its use of `SampleMask`
- VUID-vkUpdateIndirectExecutionSetShaderEXT-StencilExportEXT-11003
  Each fragment shader element in the Indirect Execution Set **must** match the initial shader state used to create the Indirect Execution Set in its use of `StencilExportEXT`

Valid Usage (Implicit)

- VUID-vkUpdateIndirectExecutionSetShaderEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkUpdateIndirectExecutionSetShaderEXT-indirectExecutionSet-parameter
  `indirectExecutionSet` **must** be a valid VkIndirectExecutionSetEXT handle
- VUID-vkUpdateIndirectExecutionSetShaderEXT-pExecutionSetWrites-parameter
  `pExecutionSetWrites` **must** be a valid pointer to an array of `executionSetWriteCount` valid VkWriteIndirectExecutionSetShaderEXT structures
- VUID-vkUpdateIndirectExecutionSetShaderEXT-executionSetWriteCount-arraylength
  `executionSetWriteCount` **must** be greater than `0`
- VUID-vkUpdateIndirectExecutionSetShaderEXT-indirectExecutionSet-parent
  `indirectExecutionSet` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `indirectExecutionSet` **must** be externally synchronized

It is legal to update an Indirect Execution Set that is in flight as long as the element indices in `pExecutionSetWrites` are not in use. Any change to an indirect execution set requires recalculating memory requirements by calling vkGetGeneratedCommandsMemoryRequirementsEXT for commands that use that modified state. Commands that are in flight or those not using updated elements require no changes.

The lifetimes of pipelines and shader objects contained in a set **must** match or exceed the lifetime of the set.

With `VK_EXT_device_generated_commands`, the actual generation of commands as well as their execution on the device is handled as single action with:

```
// Provided by VK_EXT_device_generated_commands
void vkCmdExecuteGeneratedCommandsEXT(
    VkCommandBuffer                             commandBuffer,
    VkBool32                                    isPreprocessed,
    const VkGeneratedCommandsInfoEXT*           pGeneratedCommandsInfo);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `isPreprocessed` represents whether the input data has already been preprocessed on the device. If it is `VK_FALSE` this command will implicitly trigger the preprocessing step, otherwise not.
- `pGeneratedCommandsInfo` is a pointer to a VkGeneratedCommandsInfoEXT structure containing parameters affecting the generation of commands.

If the `VK_INDIRECT_COMMANDS_LAYOUT_USAGE_UNORDERED_SEQUENCES_BIT_EXT` flag was used to create the VkGeneratedCommandsInfoEXT::`indirectCommandsLayout` then the execution of sequences through this command **may** use implementation-defined ordering which is not guaranteed to be coherent using the same input data. It does not affect the order of token processing within a sequence. This is the implied ordering with `VK_INDIRECT_COMMANDS_TOKEN_TYPE_DISPATCH_EXT`.

After a call to `vkCmdExecuteGeneratedCommandsEXT`, command buffer state will become undefined according to the tokens executed. This table specifies the relationship between tokens used and state invalidation.

Table 42. Indirect Execution State Invalidation

Common Tokens	States Invalidated
`VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT`	Bound shaders and pipelines
`VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT`	Push constant data
`VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT`	Push constant data

Common Tokens	States Invalidated
VK_INDIRECT_COMMANDS_TOKEN_TYPE_INDEX_BUFFER_EXT	Index buffer
VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT	Vertex buffer

Valid Usage

- VUID-vkCmdExecuteGeneratedCommandsEXT-magFilter-04553
  If a VkSampler created with magFilter or minFilter equal to VK_FILTER_LINEAR, and compareEnable equal to VK_FALSE is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-mipmapMode-04770
  If a VkSampler created with mipmapMode equal to VK_SAMPLER_MIPMAP_MODE_LINEAR, and compareEnable equal to VK_FALSE is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-unnormalizedCoordinates-09635
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's levelCount and layerCount **must** be 1
- VUID-vkCmdExecuteGeneratedCommandsEXT-unnormalizedCoordinates-09636
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's viewType **must** be VK_IMAGE_VIEW_TYPE_1D or VK_IMAGE_VIEW_TYPE_2D
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY

- VUID-vkCmdExecuteGeneratedCommandsEXT-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdExecuteGeneratedCommandsEXT-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdExecuteGeneratedCommandsEXT-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command

- VUID-vkCmdExecuteGeneratedCommandsEXT-maintenance4-08602
  If the `maintenance4` feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdExecuteGeneratedCommandsEXT-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled,

that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdExecuteGeneratedCommandsEXT-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdExecuteGeneratedCommandsEXT-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdExecuteGeneratedCommandsEXT-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdExecuteGeneratedCommandsEXT-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdExecuteGeneratedCommandsEXT-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdExecuteGeneratedCommandsEXT-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdExecuteGeneratedCommandsEXT-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format

- VUID-vkCmdExecuteGeneratedCommandsEXT-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdExecuteGeneratedCommandsEXT-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdExecuteGeneratedCommandsEXT-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdExecuteGeneratedCommandsEXT-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdExecuteGeneratedCommandsEXT-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdExecuteGeneratedCommandsEXT-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdExecuteGeneratedCommandsEXT-renderPass-02684
  The current render pass **must** be compatible with the renderPass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdExecuteGeneratedCommandsEXT-subpass-02685
  The subpass index of the current render pass **must** be equal to the subpass member of the VkGraphicsPipelineCreateInfo structure specified when creating the VkPipeline bound to VK_PIPELINE_BIND_POINT_GRAPHICS
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07748
  If any shader statically accesses an input attachment, a valid descriptor **must** be bound to the pipeline via a descriptor set
- VUID-vkCmdExecuteGeneratedCommandsEXT-OpTypeImage-07468

If any shader executed by this pipeline accesses an OpTypeImage variable with a Dim operand of SubpassData, it **must** be decorated with an InputAttachmentIndex that corresponds to a valid input attachment in the current subpass

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07469
  Input attachment views accessed in a subpass **must** be created with the same VkFormat as the corresponding subpass definition, and be created with a VkImageView that is compatible with the attachment referenced by the subpass' pInputAttachments [InputAttachmentIndex] in the bound VkFramebuffer as specified by Fragment Input Attachment Compatibility
- VUID-vkCmdExecuteGeneratedCommandsEXT-pDepthInputAttachmentIndex-09595
  Input attachment views accessed in a dynamic render pass with a InputAttachmentIndex referenced by VkRenderingInputAttachmentIndexInfoKHR, or no InputAttachmentIndex if VkRenderingInputAttachmentIndexInfoKHR:pDepthInputAttachmentIndex or VkRenderingInputAttachmentIndexInfoKHR:pStencilInputAttachmentIndex are NULL, **must** be created with a VkImageView that is compatible with the corresponding color, depth, or stencil attachment in VkRenderingInfo
- VUID-vkCmdExecuteGeneratedCommandsEXT-pDepthInputAttachmentIndex-09596
  Input attachment views accessed in a dynamic render pass via a shader object **must** have an InputAttachmentIndex if both VkRenderingInputAttachmentIndexInfoKHR :pDepthInputAttachmentIndex and VkRenderingInputAttachmentIndexInfoKHR :pStencilInputAttachmentIndex are non-NULL
- VUID-vkCmdExecuteGeneratedCommandsEXT-InputAttachmentIndex-09597
  If an input attachment view accessed in a dynamic render pass via a shader object has an InputAttachmentIndex, the InputAttachmentIndex **must** match an index in VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-06537
  Memory backing image subresources used as attachments in the current render pass **must** not be written in any way other than as an attachment by this command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09000
  If a color attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:
  - the VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_COLOR_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09001
  If a depth attachment is written by any prior command in this subpass or by the load,

store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:

  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_DEPTH_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09002
  If a stencil attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it is not in the VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT image layout, and either:

  - the VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT is set on the bound pipeline or
  - the last call to vkCmdSetAttachmentFeedbackLoopEnableEXT included VK_IMAGE_ASPECT_STENCIL_BIT and
    - there is no bound graphics pipeline or
    - the bound graphics pipeline was created with VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT

  it **must** not be accessed in any way other than as an attachment by this command

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09003
  If an attachment is written by any prior command in this subpass or by the load, store, or resolve operations for this subpass, it **must** not be accessed in any way other than as an attachment, storage image, or sampled image by this command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-06539
  If any previously recorded command in the current subpass accessed an image subresource used as an attachment in this subpass in any way other than as an attachment, this command **must** not write to that image subresource as an attachment
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-06886
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the depth aspect, depth writes **must** be disabled
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-06887
  If the current render pass instance uses a depth/stencil attachment with a read-only layout for the stencil aspect, both front and back writeMask are not zero, and stencil test is enabled, all stencil ops **must** be VK_STENCIL_OP_KEEP
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07831
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_VIEWPORT dynamic state enabled then vkCmdSetViewport **must** have been called and not

subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07832
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SCISSOR dynamic state enabled then vkCmdSetScissor **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07833
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_WIDTH dynamic state enabled then vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08617
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08618
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08619
  If a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, vkCmdSetLineWidth **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07834
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BIAS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBiasEnable is VK_TRUE, then vkCmdSetDepthBounds or vkCmdSetDepthBias2EXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07835
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_BLEND_CONSTANTS dynamic state enabled then vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08621
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set

rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer set any element of pColorBlendEnables to VK_TRUE, and the most recent call to vkCmdSetColorBlendEquationEXT in the current command buffer set the same element of pColorBlendEquations to a VkColorBlendEquationEXT structure with any VkBlendFactor member with a value of VK_BLEND_FACTOR_CONSTANT_COLOR, VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_COLOR, VK_BLEND_FACTOR_CONSTANT_ALPHA, or VK_BLEND_FACTOR_ONE_MINUS_CONSTANT_ALPHA, vkCmdSetBlendConstants **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07836
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_BOUNDS dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of depthBoundsTestEnable is VK_TRUE, then vkCmdSetDepthBounds **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07837
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_COMPARE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilCompareMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07838
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_WRITE_MASK dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilWriteMask **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07839
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_STENCIL_REFERENCE dynamic state enabled, the current value of and rasterizerDiscardEnable is VK_FALSE, the current value of stencilTestEnable is VK_TRUE, then vkCmdSetStencilReference **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-maxMultiviewInstanceIndex-02688
  If the draw is recorded in a render pass instance with multiview enabled, the maximum instance index **must** be less than or equal to VkPhysicalDeviceMultiviewProperties::maxMultiviewInstanceIndex
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsEnable-02689
  If the bound graphics pipeline was created with VkPipelineSampleLocationsStateCreateInfoEXT::sampleLocationsEnable set to VK_TRUE and

the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07634
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleLocationsEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-06666
  If the VK_EXT_sample_locations extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of sampleLocationsEnable is VK_TRUE, then vkCmdSetSampleLocationsEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-blendEnable-04727
  If rasterization is not disabled in the bound graphics pipeline, then for each color attachment in the subpass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the blendEnable member of the corresponding element of the pAttachments member of pColorBlendState **must** be VK_FALSE
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08643
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then for each color attachment in the render pass, if the corresponding image view's format features do not contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT, then the corresponding member of pColorBlendEnables in the most recent call to vkCmdSetColorBlendEnableEXT in the current command buffer that affected that attachment index **must** have been VK_FALSE
- VUID-vkCmdExecuteGeneratedCommandsEXT-multisampledRenderToSingleSampled-07284
  If rasterization is not disabled in the bound graphics pipeline,

  then rasterizationSamples for the bound graphics pipeline **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08644
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE,

  then the most recent call to vkCmdSetRasterizationSamplesEXT in the current command buffer **must** have set rasterizationSamples to be the same as the number of samples for the current render pass color and/or depth/stencil attachments
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08876

If a shader object is bound to any graphics stage, the current render pass instance **must** have been begun with vkCmdBeginRenderingKHR

- VUID-vkCmdExecuteGeneratedCommandsEXT-imageView-06172
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdExecuteGeneratedCommandsEXT-imageView-06173
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdExecuteGeneratedCommandsEXT-imageView-06174
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdExecuteGeneratedCommandsEXT-imageView-06175
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdExecuteGeneratedCommandsEXT-imageView-06176
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pDepthAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pDepthAttachment` is `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL`, this command **must** not write any values to the depth attachment
- VUID-vkCmdExecuteGeneratedCommandsEXT-imageView-06177
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the `imageView` member of `pStencilAttachment` is not VK_NULL_HANDLE, and the `layout` member of `pStencilAttachment` is `VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL`, this command **must** not write any values to the stencil attachment
- VUID-vkCmdExecuteGeneratedCommandsEXT-viewMask-06178
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo ::`viewMask` equal to VkRenderingInfo::`viewMask`
- VUID-vkCmdExecuteGeneratedCommandsEXT-colorAttachmentCount-06179
  If the `dynamicRenderingUnusedAttachments` feature is not enabled and the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound graphics pipeline **must** have been created with a VkPipelineRenderingCreateInfo::`colorAttachmentCount` equal to VkRenderingInfo::`colorAttachmentCount`
- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08910

If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline

- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08912
  If the dynamicRenderingUnusedAttachments feature is not enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView equal to VK_NULL_HANDLE **must** have the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound pipeline equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08911
  If the dynamicRenderingUnusedAttachments feature is enabled, and the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingInfo ::colorAttachmentCount greater than 0, then each element of the VkRenderingInfo ::pColorAttachments array with an imageView not equal to VK_NULL_HANDLE **must** have been created with a VkFormat equal to the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the bound graphics pipeline, or the corresponding element of VkPipelineRenderingCreateInfo ::pColorAttachmentFormats, if it exists, **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07749
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08646
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-attachmentCount-07750
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT dynamic state enabled then the attachmentCount parameter of vkCmdSetColorWriteEnableEXT **must** be greater than or equal to the VkPipelineColorBlendStateCreateInfo::attachmentCount of the bound graphics pipeline
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08647
  If the colorWriteEnable feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to

vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then the attachmentCount parameter of most recent call to vkCmdSetColorWriteEnableEXT in the current command buffer **must** be greater than or equal to the number of color attachments in the current render pass instance

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07751
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT dynamic state enabled then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command for each discard rectangle in VkPipelineDiscardRectangleStateCreateInfoEXT::discardRectangleCount
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07880
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDiscardRectangleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07881
  If the VK_EXT_discard_rectangles extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of discardRectangleEnable is VK_TRUE, then vkCmdSetDiscardRectangleModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-rasterizerDiscardEnable-09236
  If the VK_EXT_discard_rectangles extension is enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetDiscardRectangleEnableEXT in the current command buffer set discardRectangleEnable to VK_TRUE, then vkCmdSetDiscardRectangleEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08913
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08914
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics

pipeline **must** be equal to the VkFormat used to create VkRenderingInfo ::pDepthAttachment->imageView

- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08915
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo ::pDepthAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo ::pDepthAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08916
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo ::pStencilAttachment->imageView was VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08917
  If current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is not enabled, and VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline **must** be equal to the VkFormat used to create VkRenderingInfo ::pStencilAttachment->imageView
- VUID-vkCmdExecuteGeneratedCommandsEXT-dynamicRenderingUnusedAttachments-08918
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the dynamicRenderingUnusedAttachments feature is enabled, VkRenderingInfo ::pStencilAttachment->imageView was not VK_NULL_HANDLE, and the value of VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the bound graphics pipeline was not equal to the VkFormat used to create VkRenderingInfo ::pStencilAttachment->imageView, the value of the format **must** be VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-imageView-06183
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentShadingRateAttachmentInfoKHR::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- VUID-vkCmdExecuteGeneratedCommandsEXT-imageView-06184
  If the current render pass instance was begun with vkCmdBeginRenderingKHR and VkRenderingFragmentDensityMapAttachmentInfoEXT::imageView was not VK_NULL_HANDLE, the bound graphics pipeline **must** have been created with VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-multisampledRenderToSingleSampled-07285

If the current render pass instance was begun with vkCmdBeginRenderingKHR with a VkRenderingInfo::colorAttachmentCount parameter greater than 0, then each element of the VkRenderingInfo::pColorAttachments array with a imageView not equal to VK_NULL_HANDLE **must** have been created with a sample count equal to the value of rasterizationSamples for the bound graphics pipeline

- VUID-vkCmdExecuteGeneratedCommandsEXT-multisampledRenderToSingleSampled-07286
  If VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pDepthAttachment->imageView
- VUID-vkCmdExecuteGeneratedCommandsEXT-multisampledRenderToSingleSampled-07287
  If VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, the value of rasterizationSamples for the bound graphics pipeline **must** be equal to the sample count used to create VkRenderingInfo::pStencilAttachment->imageView
- VUID-vkCmdExecuteGeneratedCommandsEXT-renderPass-06198
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, the bound pipeline **must** have been created with a VkGraphicsPipelineCreateInfo::renderPass equal to VK_NULL_HANDLE
- VUID-vkCmdExecuteGeneratedCommandsEXT-pColorAttachments-08963
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound with a fragment shader that statically writes to a color attachment, the color write mask is not zero, color writes are enabled, and the corresponding element of the VkRenderingInfo::pColorAttachments->imageView was not VK_NULL_HANDLE, then the corresponding element of VkPipelineRenderingCreateInfo::pColorAttachmentFormats used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-pDepthAttachment-08964
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, depth test is enabled, depth write is enabled, and the VkRenderingInfo::pDepthAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::depthAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-pStencilAttachment-08965
  If the current render pass instance was begun with vkCmdBeginRenderingKHR, there is a graphics pipeline bound, stencil test is enabled and the VkRenderingInfo::pStencilAttachment->imageView was not VK_NULL_HANDLE, then the VkPipelineRenderingCreateInfo::stencilAttachmentFormat used to create the pipeline **must** not be VK_FORMAT_UNDEFINED
- VUID-vkCmdExecuteGeneratedCommandsEXT-primitivesGeneratedQueryWithRasterizerDiscard-06708
  If the primitivesGeneratedQueryWithRasterizerDiscard feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, rasterization discard **must** not be enabled
- VUID-vkCmdExecuteGeneratedCommandsEXT-primitivesGeneratedQueryWithNonZeroStreams-06709

If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, the bound graphics pipeline **must** not have been created with a non-zero value in VkPipelineRasterizationStateStreamCreateInfoEXT::rasterizationStream

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07619
  If a shader object is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT dynamic state enabled, then vkCmdSetTessellationDomainOriginEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07620
  If the depthClamp feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetDepthClampEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07621
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_POLYGON_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetPolygonModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07622
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetRasterizationSamplesEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07623
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetSampleMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-alphaToCoverageEnable-08919
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and alphaToCoverageEnable was VK_TRUE in the last call to vkCmdSetAlphaToCoverageEnableEXT, then the Fragment Output Interface **must** contain a variable for the alpha Component word in Location 0 at Index 0
- VUID-vkCmdExecuteGeneratedCommandsEXT-alphaToCoverageEnable-08920
  If a shader object is bound to any graphics stage, and the most recent call to vkCmdSetAlphaToCoverageEnableEXT in the current command buffer set alphaToCoverageEnable to VK_TRUE, then the Fragment Output Interface **must** contain a

variable for the alpha Component word in Location 0 at Index 0

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07624
  If a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToCoverageEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07625
  If the alphaToOne feature is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAlphaToOneEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07626
  If the logicOp feature is enabled, a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetLogicOpEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07627
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08657
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07628
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08658
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetColorBlendEnableEXT for any attachment set that attachment's value in pColorBlendEnables to VK_TRUE, then vkCmdSetColorBlendEquationEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07629
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08659
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorWriteMaskEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07630
  If the geometryStreams feature is enabled, and a shader object is bound to the VK_SHADER_STAGE_GEOMETRY_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT dynamic state enabled, then vkCmdSetRasterizationStreamEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07631
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetConservativeRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07632
  If the VK_EXT_conservative_rasterization extension is enabled, a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of conservativeRasterizationMode is VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT, then vkCmdSetExtraPrimitiveOverestimationSizeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07633
  If the depthClipEnable feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT dynamic state, then vkCmdSetDepthClipEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07635
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-rasterizerDiscardEnable-09416
  If the VK_EXT_blend_operation_advanced extension is enabled, and a shader object is bound

to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then at least one of vkCmdSetColorBlendEquationEXT and vkCmdSetColorBlendAdvancedEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07636
  If the VK_EXT_provoking_vertex extension is enabled, a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetProvokingVertexModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07637
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic state enabled then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08666
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08667
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08668
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineRasterizationModeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07638

If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT dynamic state enabled then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08669
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to any graphics stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPolygonModeEXT in the current command buffer set polygonMode to VK_POLYGON_MODE_LINE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08670
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object is bound to the VK_SHADER_STAGE_VERTEX_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, and the most recent call to vkCmdSetPrimitiveTopologyEXT in the current command buffer set primitiveTopology to any line topology, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08671
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled, and a shader object that outputs line primitives is bound to the VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT or VK_SHADER_STAGE_GEOMETRY_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetLineStippleEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07849
  If any of the stippledRectangularLines, stippledBresenhamLines or stippledSmoothLines features are enabled and a shader object is bound to any graphics stage, or a bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_KHR dynamic state enabled, the current value of rasterizerDiscardEnable is VK_FALSE, and the current value of stippledLineEnable is VK_TRUE, then vkCmdSetLineStippleKHR **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-07639
  If the depthClipControl feature is enabled, and a shader object is bound to any graphics stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT dynamic state enabled, then vkCmdSetDepthClipNegativeOneToOneEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09650
  If the depthClampControl feature is enabled, and a shader object is bound to any graphics

stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT dynamic state enabled, and the current value of depthClampEnable is VK_TRUE, then vkCmdSetDepthClampRangeEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command

- VUID-vkCmdExecuteGeneratedCommandsEXT-pColorBlendEnables-07470
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT state enabled and the last call to vkCmdSetColorBlendEnableEXT set pColorBlendEnables for any attachment to VK_TRUE, then for those attachments in the subpass the corresponding image view's format features **must** contain VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-rasterizationSamples-07471
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current subpass does not use any color and/or depth/stencil attachments, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** follow the rules for a zero-attachment subpass
- VUID-vkCmdExecuteGeneratedCommandsEXT-samples-07472
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the VkPipelineMultisampleStateCreateInfo::rasterizationSamples parameter used to create the bound graphics pipeline
- VUID-vkCmdExecuteGeneratedCommandsEXT-samples-07473
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_MASK_EXT state and VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT states enabled, then the samples parameter in the last call to vkCmdSetSampleMaskEXT **must** be greater or equal to the rasterizationSamples parameter in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-rasterizationSamples-07474
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and neither the VK_AMD_mixed_attachment_samples nor the VK_NV_framebuffer_mixed_samples extensions are enabled, then the rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT **must** be the same as the current subpass color and/or depth/stencil attachments
- VUID-vkCmdExecuteGeneratedCommandsEXT-firstAttachment-07476
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic state enabled then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass
- VUID-vkCmdExecuteGeneratedCommandsEXT-rasterizerDiscardEnable-09417

If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorBlendEnableEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEnableEXT calls **must** specify an enable for all active color attachments in the current subpass

- VUID-vkCmdExecuteGeneratedCommandsEXT-firstAttachment-07477
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT dynamic state enabled then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdExecuteGeneratedCommandsEXT-rasterizerDiscardEnable-09418
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and both the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE and there are color attachments bound, then vkCmdSetColorBlendEquationEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendEquationEXT calls **must** specify the blend equations for all active color attachments in the current subpass where blending is enabled
- VUID-vkCmdExecuteGeneratedCommandsEXT-firstAttachment-07478
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT dynamic state enabled then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdExecuteGeneratedCommandsEXT-rasterizerDiscardEnable-09419
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, and the most recent call to vkCmdSetRasterizerDiscardEnableEXT in the current command buffer set rasterizerDiscardEnable to VK_FALSE, then vkCmdSetColorWriteMaskEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorWriteMaskEXT calls **must** specify the color write mask for all active color attachments in the current subpass
- VUID-vkCmdExecuteGeneratedCommandsEXT-firstAttachment-07479
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT dynamic state enabled then vkCmdSetColorBlendAdvancedEXT **must** have been called in the current command buffer prior to this drawing command, and the attachments specified by the firstAttachment and attachmentCount parameters of vkCmdSetColorBlendAdvancedEXT calls **must** specify the advanced blend equations for all active color attachments in the current subpass where

blending is enabled

- VUID-vkCmdExecuteGeneratedCommandsEXT-advancedBlendMaxColorAttachments-07480
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT and VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT dynamic states enabled and the last calls to vkCmdSetColorBlendEnableEXT and vkCmdSetColorBlendAdvancedEXT have enabled advanced blending, then the number of active color attachments in the current subpass **must** not exceed advancedBlendMaxColorAttachments
- VUID-vkCmdExecuteGeneratedCommandsEXT-primitivesGeneratedQueryWithNonZeroStreams-07481
  If the primitivesGeneratedQueryWithNonZeroStreams feature is not enabled and the VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT query is active, and the bound graphics pipeline was created with VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT state enabled, the last call to vkCmdSetRasterizationStreamEXT **must** have set the rasterizationStream to zero
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsPerPixel-07482
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state disabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples member of the VkPipelineMultisampleStateCreateInfo structure the bound graphics pipeline has been created with
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsPerPixel-07483
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then the sampleLocationsPerPixel member of pSampleLocationsInfo in the last call to vkCmdSetSampleLocationsEXT **must** equal the rasterizationSamples parameter of the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsEnable-07484
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, and the current subpass has a depth/stencil attachment, then that attachment **must** have been created with the VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT bit set
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsEnable-07485
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.width in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT

::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples

- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsEnable-07486
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state enabled and the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, then the sampleLocationsInfo.sampleLocationGridSize.height in the last call to vkCmdSetSampleLocationsEXT **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling rasterizationSamples
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsEnable-07487
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage, or the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT state enabled, and if sampleLocationsEnable was VK_TRUE in the last call to vkCmdSetSampleLocationsEnableEXT, the fragment shader code **must** not statically use the extended instruction InterpolateAtSample
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsEnable-07936
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.width **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.width as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsEnable-07937
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationGridSize.height **must** evenly divide VkMultisamplePropertiesEXT::sampleLocationGridSize.height as returned by vkGetPhysicalDeviceMultisamplePropertiesEXT with a samples parameter equaling the value of rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-sampleLocationsEnable-07938
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT state disabled and the VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT state enabled, and the current value of sampleLocationsEnable is VK_TRUE, then sampleLocationsInfo.sampleLocationsPerPixel **must** equal rasterizationSamples in the last call to vkCmdSetRasterizationSamplesEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-stippledLineEnable-07495

If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR, then the stippledRectangularLines feature **must** be enabled

- VUID-vkCmdExecuteGeneratedCommandsEXT-stippledLineEnable-07496
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR, then the stippledBresenhamLines feature **must** be enabled
- VUID-vkCmdExecuteGeneratedCommandsEXT-stippledLineEnable-07497
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR, then the stippledSmoothLines feature **must** be enabled
- VUID-vkCmdExecuteGeneratedCommandsEXT-stippledLineEnable-07498
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT or VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT dynamic states enabled, and if the current stippledLineEnable state is VK_TRUE and the current lineRasterizationMode state is VK_LINE_RASTERIZATION_MODE_DEFAULT_KHR, then the stippledRectangularLines feature **must** be enabled and VkPhysicalDeviceLimits::strictLines **must** be VK_TRUE
- VUID-vkCmdExecuteGeneratedCommandsEXT-conservativePointAndLineRasterization-07499
  If the bound graphics pipeline state was created with the VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT dynamic state enabled, conservativePointAndLineRasterization is not supported, and the effective primitive topology output by the last pre-rasterization shader stage is a line or point, then the conservativeRasterizationMode set by the last call to vkCmdSetConservativeRasterizationModeEXT **must** be VK_CONSERVATIVE_RASTERIZATION_MODE_DISABLED_EXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08877
  If a shader object is bound to the VK_SHADER_STAGE_FRAGMENT_BIT stage or a graphics pipeline is bound which was created with the VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT dynamic state enabled, and the current value of rasterizerDiscardEnable is VK_FALSE, then vkCmdSetAttachmentFeedbackLoopEnableEXT **must** have been called and not subsequently invalidated in the current command buffer prior to this drawing command
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08684
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the

current command buffer with pStages with an element of VK_SHADER_STAGE_VERTEX_BIT

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08685
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08686
  If there is no bound graphics pipeline, and the tessellationShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08687
  If there is no bound graphics pipeline, and the geometryShader feature is enabled, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_GEOMETRY_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08688
  If there is no bound graphics pipeline, vkCmdBindShadersEXT **must** have been called in the current command buffer with pStages with an element of VK_SHADER_STAGE_FRAGMENT_BIT
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08698
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, then all shaders created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag in the same vkCreateShadersEXT call **must** also be bound
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08699
  If any graphics shader is bound which was created with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag, any stages in between stages whose shaders which did not create a shader with the VK_SHADER_CREATE_LINK_STAGE_BIT_EXT flag as part of the same vkCreateShadersEXT call **must** not have any VkShaderEXT bound
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08878
  All bound graphics shader objects **must** have been created with identical or identically defined push constant ranges
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-08879
  All bound graphics shader objects **must** have been created with identical or identically defined arrays of descriptor set layouts
- VUID-vkCmdExecuteGeneratedCommandsEXT-pDynamicStates-08715
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpDepthAttachmentReadEXT, the depthWriteEnable parameter in the last call to vkCmdSetDepthWriteEnableEXT **must** be VK_FALSE
- VUID-vkCmdExecuteGeneratedCommandsEXT-pDynamicStates-08716
  If the bound graphics pipeline state includes a fragment shader stage, was created with VK_DYNAMIC_STATE_STENCIL_WRITE_MASK set in VkPipelineDynamicStateCreateInfo ::pDynamicStates, and the fragment shader declares the EarlyFragmentTests execution mode and uses OpStencilAttachmentReadEXT, the writeMask parameter in the last call to vkCmdSetStencilWriteMask **must** be 0

- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09116
  If a shader object is bound to any graphics stage or the bound graphics pipeline was created with VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT, and the format of any color attachment is VK_FORMAT_E5B9G9R9_UFLOAT_PACK32, the corresponding element of the pColorWriteMasks parameter of vkCmdSetColorWriteMaskEXT **must** either include all of VK_COLOR_COMPONENT_R_BIT, VK_COLOR_COMPONENT_G_BIT, and VK_COLOR_COMPONENT_B_BIT, or none of them
- VUID-vkCmdExecuteGeneratedCommandsEXT-maxFragmentDualSrcAttachments-09239
  If blending is enabled for any attachment where either the source or destination blend factors for that attachment use the secondary color input, the maximum value of Location for any output attachment statically used in the Fragment Execution Model executed by this command **must** be less than maxFragmentDualSrcAttachments
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09548
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, the value of each element of VkRenderingAttachmentLocationInfoKHR::pColorAttachmentLocations set by vkCmdSetRenderingAttachmentLocationsKHR **must** match the value set for the corresponding element in the bound pipeline
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-09549
  If the current render pass was begun with vkCmdBeginRenderingKHR, and there is no shader object bound to any graphics stage, input attachment index mappings in the bound pipeline **must** match those set for the current render pass instance via VkRenderingInputAttachmentIndexInfoKHR
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11046
  If isPreprocessed is VK_TRUE and vkGetGeneratedCommandsMemoryRequirementsEXT did not return a required size of zero then vkCmdPreprocessGeneratedCommandsEXT **must** have already been executed on the device before this command executes, and the preprocessing command **must** have used the same pGeneratedCommandsInfo content as well as the content of the input buffers it references (all except VkGeneratedCommandsInfoEXT::preprocessBuffer)
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11047
  If isPreprocessed is VK_TRUE then the indirectCommandsLayout member of pGeneratedCommandsInfo **must** have been created with the VK_INDIRECT_COMMANDS_LAYOUT_USAGE_EXPLICIT_PREPROCESS_BIT_EXT bit set
- VUID-vkCmdExecuteGeneratedCommandsEXT-indirectCommandsLayout-11141
  If the indirectCommandsLayout member of pGeneratedCommandsInfo was created with the VK_INDIRECT_COMMANDS_LAYOUT_USAGE_EXPLICIT_PREPROCESS_BIT_EXT bit set, then isPreprocessed **must** be VK_TRUE
- VUID-vkCmdExecuteGeneratedCommandsEXT-preprocessAddress-11142
  The contents of the preprocessAddress member of pGeneratedCommandsInfo **must** not have been previously used to record another vkCmdExecuteGeneratedCommandsEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11048
  If isPreprocessed is VK_TRUE then the bound descriptor sets and push constants **must** match identically with those bound during recording of the corresponding call to

vkCmdPreprocessGeneratedCommandsEXT

- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-10198
  If isPreprocessed is VK_TRUE then the conditional render state and its predicate value **must** match identically with the state and value set during execution of the corresponding call to vkCmdPreprocessGeneratedCommandsEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11049
  If isPreprocessed is VK_TRUE and the indirectCommandsLayout member of pGeneratedCommandsInfo contains a draw token, then the graphics state bound on commandBuffer **must** match identically with the graphics state bound on the stateCommandBuffer passed to vkCmdPreprocessGeneratedCommandsEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11149
  If isPreprocessed is VK_TRUE, then the queue family index of commandBuffer **must** be the same as the queue family index used to allocate the stateCommandBuffer passed to vkCmdPreprocessGeneratedCommandsEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11051
  If isPreprocessed is VK_TRUE and the indirectCommandsLayout member of pGeneratedCommandsInfo contains a dispatch token, then the compute state bound on commandBuffer **must** match identically with the compute state bound on the stateCommandBuffer passed to vkCmdPreprocessGeneratedCommandsEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11052
  If isPreprocessed is VK_TRUE and the indirectCommandsLayout member of pGeneratedCommandsInfo contains a ray tracing token, then the ray tracing state bound on commandBuffer **must** match identically with the ray tracing state bound on the stateCommandBuffer passed to vkCmdPreprocessGeneratedCommandsEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11150
  If isPreprocessed is VK_TRUE and the indirectCommandsLayout member of pGeneratedCommandsInfo contains a ray tracing token, the queue family index commandBuffer was allocated from **must** be the same queue family index used to allocate the stateCommandBuffer passed to vkCmdPreprocessGeneratedCommandsEXT
- VUID-vkCmdExecuteGeneratedCommandsEXT-indirectCommandsLayout-11053
  If the token sequence of the passed VkGeneratedCommandsInfoEXT ::indirectCommandsLayout contains a VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT token, the initial shader state of VkGeneratedCommandsInfoEXT::indirectExecutionSet **must** be bound on commandBuffer
- VUID-vkCmdExecuteGeneratedCommandsEXT-indirectCommandsLayout-11004
  If indirectCommandsLayout was created with a token sequence that contained the VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT token and indirectExecutionSet was created using VK_INDIRECT_EXECUTION_SET_INFO_TYPE_SHADER_OBJECTS_EXT, every executed VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT token **must** bind all the shader stages set in the VkIndirectCommandsExecutionSetTokenEXT::shaderStages used to create indirectCommandsLayout
- VUID-vkCmdExecuteGeneratedCommandsEXT-isPreprocessed-11055
  If isPreprocessed is VK_TRUE and the token sequence of the passed VkGeneratedCommandsInfoEXT::indirectCommandsLayout contains a

VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT token, the members of VkGeneratedCommandsInfoEXT::indirectExecutionSet accessed by this command **must** not have been modified since the preprocess buffer was generated

- VUID-vkCmdExecuteGeneratedCommandsEXT-indirectCommandsLayout-11056
  If the indirectCommandsLayout member of pGeneratedCommandsInfo contains a draw token, then the active render pass **must** not have a specified fragment density map
- VUID-vkCmdExecuteGeneratedCommandsEXT-deviceGeneratedCommandsTransformFeedback-11057
  If deviceGeneratedCommandsTransformFeedback is not supported on device, transform feedback **must** not be active
- VUID-vkCmdExecuteGeneratedCommandsEXT-indirectExecutionSet-11058
  If transform feedback is active, VkGeneratedCommandsInfoEXT::indirectExecutionSet **must** be VK_NULL_HANDLE
- VUID-vkCmdExecuteGeneratedCommandsEXT-deviceGeneratedCommands-11059
  The VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT::deviceGeneratedCommands feature **must** be enabled
- VUID-vkCmdExecuteGeneratedCommandsEXT-supportedIndirectCommandsShaderStages-11060
  The bound shader stages **must** be supported by VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::supportedIndirectCommandsShaderStages
- VUID-vkCmdExecuteGeneratedCommandsEXT-supportedIndirectCommandsShaderStages-11061
  Only stages specified in VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::supportedIndirectCommandsShaderStages **can** be set in pGeneratedCommandsInfo->shaderStages
- VUID-vkCmdExecuteGeneratedCommandsEXT-None-11062
  If a rendering pass is currently active, the view mask **must** be 0
- VUID-vkCmdExecuteGeneratedCommandsEXT-commandBuffer-11143
  commandBuffer **must** not have been created with VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT

Valid Usage (Implicit)

- VUID-vkCmdExecuteGeneratedCommandsEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdExecuteGeneratedCommandsEXT-pGeneratedCommandsInfo-parameter
  pGeneratedCommandsInfo **must** be a valid pointer to a valid VkGeneratedCommandsInfoEXT structure
- VUID-vkCmdExecuteGeneratedCommandsEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdExecuteGeneratedCommandsEXT-commandBuffer-cmdpool

The VkCommandPool that commandBuffer was allocated from **must** support graphics, or compute operations

- VUID-vkCmdExecuteGeneratedCommandsEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdExecuteGeneratedCommandsEXT-bufferlevel
  commandBuffer **must** be a primary VkCommandBuffer

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Both	Outside	Graphics Compute	Action Indirection

The VkGeneratedCommandsInfoEXT is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkGeneratedCommandsInfoEXT {
    VkStructureType                 sType;
    const void*                     pNext;
    VkShaderStageFlags              shaderStages;
    VkIndirectExecutionSetEXT       indirectExecutionSet;
    VkIndirectCommandsLayoutEXT     indirectCommandsLayout;
    VkDeviceAddress                 indirectAddress;
    VkDeviceSize                    indirectAddressSize;
    VkDeviceAddress                 preprocessAddress;
    VkDeviceSize                    preprocessSize;
    uint32_t                        maxSequenceCount;
    VkDeviceAddress                 sequenceCountAddress;
    uint32_t                        maxDrawCount;
} VkGeneratedCommandsInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderStages is the mask of shader stages used by the commands.
- indirectExecutionSet is the indirect execution set to be used for binding shaders.

- `indirectCommandsLayout` is the VkIndirectCommandsLayoutEXT that specifies the command sequence data.
- `indirectAddress` is an address that holds the indirect buffer data.
- `indirectAddressSize` is the size in bytes of indirect buffer data starting at `indirectAddress`.
- `preprocessAddress` specifies a physical address of the `VkBuffer` used for preprocessing the input data for execution. If this structure is used with vkCmdExecuteGeneratedCommandsEXT with its `isPreprocessed` set to `VK_TRUE`, then the preprocessing step is skipped but data in this address **may** still be modified. The contents and the layout of this address are opaque to applications and **must** not be modified outside functions related to device-generated commands or copied to another buffer for reuse.
- `preprocessSize` is the maximum byte size within `preprocessAddress` that is available for preprocessing.
- `maxSequenceCount` is used to determine the number of sequences to execute.
- `sequenceCountAddress` specifies an optional physical address of a single `uint32_t` value containing the requested number of sequences to execute.
- `maxDrawCount` is the maximum number of indirect draws that can be executed by any COUNT-type multi-draw indirect tokens. The draw count in the indirect buffer is clamped to this value for these token types.

If `sequenceCountAddress` is not `NULL`, then `maxSequenceCount` is the maximum number of sequences that can be executed. The actual number is `min(maxSequenceCount, *sequenceCountAddress)`. If `sequenceCountAddress` is `NULL`, then `maxSequenceCount` is the exact number of sequences to execute.

If the action command token for the layout is not a COUNT-type multi-draw indirect token, `maxDrawCount` is ignored.

Valid Usage

- VUID-VkGeneratedCommandsInfoEXT-preprocessAddress-11063
  If vkGetGeneratedCommandsMemoryRequirementsEXT returns a non-zero size, `preprocessAddress` **must** not be `NULL`
- VUID-VkGeneratedCommandsInfoEXT-preprocessAddress-11064
  `VkDeviceMemory` objects bound to the underlying buffer for `preprocessAddress` **must** have been allocated using one of the memory types allowed in the `memoryTypeBits` member of the VkMemoryRequirements structure returned by vkGetGeneratedCommandsMemoryRequirementsEXT
- VUID-VkGeneratedCommandsInfoEXT-indirectCommandsLayout-11065
  If the `indirectCommandsLayout` uses a token of `VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT`, then the `indirectExecutionSet`'s push constant layout **must** contain the `updateRange` specified in VkIndirectCommandsPushConstantTokenEXT
- VUID-VkGeneratedCommandsInfoEXT-indirectCommandsLayout-11066
  If the `indirectCommandsLayout` uses a token of `VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT`, then the `indirectExecutionSet`'s

push constant layout **must** contain the `updateRange` specified in VkIndirectCommandsPushConstantTokenEXT

- VUID-VkGeneratedCommandsInfoEXT-maxSequenceCount-11067
  `maxSequenceCount` **must** be less or equal to VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`maxIndirectSequenceCount` and VkGeneratedCommandsMemoryRequirementsInfoEXT::`maxSequencesCount` that was used to determine the `preprocessSize`
- VUID-VkGeneratedCommandsInfoEXT-sequenceCountAddress-11068
  If `sequenceCountAddress` is not `NULL`, the value contained in the address **must** be less or equal to VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT::`maxIndirectSequenceCount` and VkGeneratedCommandsMemoryRequirementsInfoEXT::`maxSequencesCount` that was used to determine the `preprocessSize`
- VUID-VkGeneratedCommandsInfoEXT-maxSequenceCount-10246
  `maxSequenceCount` **must** not be zero
- VUID-VkGeneratedCommandsInfoEXT-preprocessAddress-11069
  The underlying buffer for `preprocessAddress` **must** have the `VK_BUFFER_USAGE_2_PREPROCESS_BUFFER_BIT_EXT` bit set in its usage flag
- VUID-VkGeneratedCommandsInfoEXT-preprocessAddress-11070
  If the underlying buffer for `preprocessAddress` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkGeneratedCommandsInfoEXT-indirectCommandsLayout-11144
  If the `indirectCommandsLayout` contains a `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` token, then the descriptor and push constant layout info provided either by `pipelineLayout` or through a VkPipelineLayoutCreateInfo in `pNext` of the VkIndirectCommandsLayoutCreateInfoEXT used to create `indirectCommandsLayout` **must** be compatible with the descriptor and push constant layout info used by `indirectExecutionSet`
- VUID-VkGeneratedCommandsInfoEXT-indirectCommandsLayout-11002
  If `indirectCommandsLayout` was created with a token sequence that contained the `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` token, the shader stages used to create the initial shader state of `indirectExecutionSet` **must** equal the VkIndirectCommandsExecutionSetTokenEXT::`shaderStages` used to create `indirectCommandsLayout`
- VUID-VkGeneratedCommandsInfoEXT-preprocessSize-11071
  `preprocessSize` **must** be greater than or equal to the memory requirement's size returned by vkGetGeneratedCommandsMemoryRequirementsEXT using the matching inputs (`indirectCommandsLayout`, ...) as within this structure
- VUID-VkGeneratedCommandsInfoEXT-sequenceCountAddress-11072
  The underlying buffer for `sequenceCountAddress` **must** have the `VK_BUFFER_USAGE_2_INDIRECT_BUFFER_BIT_KHR` bit set in its usage flag
- VUID-VkGeneratedCommandsInfoEXT-sequenceCountAddress-11073
  If `sequenceCountAddress` is not `NULL`, `sequenceCountAddress` **must** be aligned to `4`
- VUID-VkGeneratedCommandsInfoEXT-indirectAddress-11074

`indirectAddress` **must** be aligned to `4`

- VUID-VkGeneratedCommandsInfoEXT-sequenceCountAddress-11075
  If the underlying buffer for `sequenceCountAddress` is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkGeneratedCommandsInfoEXT-indirectAddress-11076
  `indirectAddress` **must** not be `NULL`
- VUID-VkGeneratedCommandsInfoEXT-indirectAddressSize-11077
  `indirectAddressSize` **must** be greater than zero
- VUID-VkGeneratedCommandsInfoEXT-maxDrawCount-11078
  When not ignored, `maxDrawCount` × `maxSequenceCount` **must** be less than 2^24
- VUID-VkGeneratedCommandsInfoEXT-indirectCommandsLayout-11079
  If `indirectCommandsLayout` was created using a `VK_INDIRECT_COMMANDS_TOKEN_TYPE_VERTEX_BUFFER_EXT` token and shader objects are not bound then the bound graphics pipeline **must** have been created with `VK_DYNAMIC_STATE_VERTEX_INPUT_BINDING_STRIDE` in `pDynamicStates`
- VUID-VkGeneratedCommandsInfoEXT-indirectCommandsLayout-11083
  If the token sequence of the passed `indirectCommandsLayout` contains a `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` token, the `indirectExecutionSet` **must** not be VK_NULL_HANDLE
- VUID-VkGeneratedCommandsInfoEXT-indirectCommandsLayout-10241
  If the token sequence of the passed `indirectCommandsLayout` does not contains a `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` token, the `indirectExecutionSet` **must** be VK_NULL_HANDLE
- VUID-VkGeneratedCommandsInfoEXT-indirectExecutionSet-11080
  If `indirectExecutionSet` is VK_NULL_HANDLE, a VkGeneratedCommandsPipelineInfoEXT or VkGeneratedCommandsShaderInfoEXT **must** be included in the `pNext` chain

Valid Usage (Implicit)

- VUID-VkGeneratedCommandsInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_GENERATED_COMMANDS_INFO_EXT`
- VUID-VkGeneratedCommandsInfoEXT-shaderStages-parameter
  `shaderStages` **must** be a valid combination of VkShaderStageFlagBits values
- VUID-VkGeneratedCommandsInfoEXT-shaderStages-requiredbitmask
  `shaderStages` **must** not be `0`
- VUID-VkGeneratedCommandsInfoEXT-indirectExecutionSet-parameter
  If `indirectExecutionSet` is not VK_NULL_HANDLE, `indirectExecutionSet` **must** be a valid VkIndirectExecutionSetEXT handle
- VUID-VkGeneratedCommandsInfoEXT-indirectCommandsLayout-parameter
  `indirectCommandsLayout` **must** be a valid VkIndirectCommandsLayoutEXT handle
- VUID-VkGeneratedCommandsInfoEXT-commonparent
  Both of `indirectCommandsLayout`, and `indirectExecutionSet` that are valid handles of non-

ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Referencing the functions defined in Indirect Commands Layout, `vkCmdExecuteGeneratedCommandsEXT` behaves as:

```
uint32_t sequencesCount = sequenceCountAddress ?
      min(maxSequenceCount, sequenceCountAddress.load_uint32()) :
      maxSequenceCount;


cmdProcessAllSequences(commandBuffer, indirectExecutionSet,
                       indirectCommandsLayout, indirectAddress,
                       sequencesCount);

// The stateful commands within indirectCommandsLayout will not
// affect the state of subsequent commands in the target
// command buffer (cmd)
```

NOTE

Note

It is important to note that the affected values of all state related to the `shaderStages` used are undefined after this command. This means that e.g., if this command indirectly alters push constants, the push constant state becomes undefined.

Commands **can** be preprocessed prior execution using the following command:

```
// Provided by VK_EXT_device_generated_commands
void vkCmdPreprocessGeneratedCommandsEXT(
    VkCommandBuffer                             commandBuffer,
    const VkGeneratedCommandsInfoEXT*           pGeneratedCommandsInfo,
    VkCommandBuffer                             stateCommandBuffer);
```

- `commandBuffer` is the command buffer which does the preprocessing.
- `pGeneratedCommandsInfo` is a pointer to a VkGeneratedCommandsInfoEXT structure containing parameters affecting the preprocessing step.
- `stateCommandBuffer` is a command buffer from which to snapshot current states affecting the preprocessing step. When a graphics command action token is used, graphics state is snapshotted. When a compute action command token is used, compute state is snapshotted. When a ray tracing action command token is used, ray tracing state is snapshotted. It can be deleted at any time after this command has been recorded.

NOTE

Note

`stateCommandBuffer` access is not synchronized by the driver, meaning that this command buffer **must** not be modified between threads in an unsafe manner.

Valid Usage

- VUID-vkCmdPreprocessGeneratedCommandsEXT-pGeneratedCommandsInfo-11082
  `pGeneratedCommandsInfo`'s `indirectCommandsLayout` **must** have been created with the `VK_INDIRECT_COMMANDS_LAYOUT_USAGE_EXPLICIT_PREPROCESS_BIT_EXT` bit set
- VUID-vkCmdPreprocessGeneratedCommandsEXT-indirectCommandsLayout-11084
  If the token sequence of the passed VkGeneratedCommandsInfoEXT ::`indirectCommandsLayout` contains a `VK_INDIRECT_COMMANDS_TOKEN_TYPE_EXECUTION_SET_EXT` token, the initial shader state of VkGeneratedCommandsInfoEXT::`indirectExecutionSet` **must** be bound on `stateCommandBuffer`
- VUID-vkCmdPreprocessGeneratedCommandsEXT-stateCommandBuffer-11138
  `stateCommandBuffer` **must** be in the recording state
- VUID-vkCmdPreprocessGeneratedCommandsEXT-deviceGeneratedCommands-11087
  The `VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT`::`deviceGeneratedCommands` feature **must** be enabled
- VUID-vkCmdPreprocessGeneratedCommandsEXT-supportedIndirectCommandsShaderStages-11088
  Only stages specified in VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT ::`supportedIndirectCommandsShaderStages` **can** be set in `pGeneratedCommandsInfo->shaderStages`

Valid Usage (Implicit)

- VUID-vkCmdPreprocessGeneratedCommandsEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPreprocessGeneratedCommandsEXT-pGeneratedCommandsInfo-parameter
  `pGeneratedCommandsInfo` **must** be a valid pointer to a valid VkGeneratedCommandsInfoEXT structure
- VUID-vkCmdPreprocessGeneratedCommandsEXT-stateCommandBuffer-parameter
  `stateCommandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdPreprocessGeneratedCommandsEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdPreprocessGeneratedCommandsEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdPreprocessGeneratedCommandsEXT-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdPreprocessGeneratedCommandsEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdPreprocessGeneratedCommandsEXT-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

- VUID-vkCmdPreprocessGeneratedCommandsEXT-commonparent
  Both of `commandBuffer`, and `stateCommandBuffer` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to `stateCommandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Outside	Outside	Graphics Compute	Action

The bound descriptor sets and push constants that will be used with indirect command generation **must** already be specified on `stateCommandBuffer` at the time of preprocessing commands with vkCmdPreprocessGeneratedCommandsEXT. They **must** match the bound descriptor sets and push constants used in the execution of indirect commands with vkCmdExecuteGeneratedCommandsEXT.

If push constants for shader stages are also specified in the VkGeneratedCommandsInfoEXT ::`indirectCommandsLayout` with a `VK_INDIRECT_COMMANDS_TOKEN_TYPE_PUSH_CONSTANT_EXT` or `VK_INDIRECT_COMMANDS_TOKEN_TYPE_SEQUENCE_INDEX_EXT` token, then those values override the push constants that were previously pushed.

All state bound on `stateCommandBuffer` will be used. All state bound on `stateCommandBuffer` **must** be identical to the state bound at the time vkCmdExecuteGeneratedCommandsEXT is recorded. The queue family index `stateCommandBuffer` was allocated from **must** be the same as the queue family index of the command buffer used in vkCmdExecuteGeneratedCommandsEXT.

On some implementations, preprocessing **may** have no effect on performance.

vkCmdExecuteGeneratedCommandsEXT **may** write to the preprocess buffer, no matter the isPreprocess parameter. In this case, the implementation **must** insert appropriate synchronization automatically, which corresponds to the following pseudocode:

- Barrier
  - srcStageMask = DRAW_INDIRECT
  - srcAccesMask = 0

    - dstStageMask = COMMAND_PREPROCESS_BIT
    - dstAccessMask = COMMAND_PREPROCESS_WRITE_BIT | COMMAND_PREPROCESS_READ_BIT
- Do internal writes
- Barrier
    - srcStageMask = COMMAND_PREPROCESS_BIT
    - srcAccesMask = COMMAND_PREPROCESS_WRITE_BIT
    - dstStageMask = DRAW_INDIRECT
    - dstAccessMask = INDIRECT_COMMAND_READ_BIT
- Execute

Chapter 31. Sparse Resources

As documented in Resource Memory Association, `VkBuffer` and `VkImage` resources in Vulkan **must** be bound completely and contiguously to a single `VkDeviceMemory` object. This binding **must** be done before the resource is used, and the binding is immutable for the lifetime of the resource.

Sparse resources relax these restrictions and provide these additional features:

- Sparse resources **can** be bound non-contiguously to one or more `VkDeviceMemory` allocations.
- Sparse resources **can** be re-bound to different memory allocations over the lifetime of the resource.
- Sparse resources **can** have descriptors generated and used orthogonally with memory binding commands.

31.1. Sparse Resource Features

Sparse resources have several features that **must** be enabled explicitly at resource creation time. The features are enabled by including bits in the `flags` parameter of VkImageCreateInfo or VkBufferCreateInfo. Each feature also has one or more corresponding feature enables specified in VkPhysicalDeviceFeatures.

- The `sparseBinding` feature is the base, and provides the following capabilities:
  - Resources **can** be bound at some defined (sparse block) granularity.
  - The entire resource **must** be bound to memory before use regardless of regions actually accessed.
  - No specific mapping of image region to memory offset is defined, i.e. the location that each texel corresponds to in memory is implementation-dependent.
  - Sparse buffers have a well-defined mapping of buffer range to memory range, where an offset into a range of the buffer that is bound to a single contiguous range of memory corresponds to an identical offset within that range of memory.
  - Requested via the `VK_IMAGE_CREATE_SPARSE_BINDING_BIT` and `VK_BUFFER_CREATE_SPARSE_BINDING_BIT` bits.
  - A sparse image created using `VK_IMAGE_CREATE_SPARSE_BINDING_BIT` (but not `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`) supports all formats that non-sparse usage supports, and supports both `VK_IMAGE_TILING_OPTIMAL` and `VK_IMAGE_TILING_LINEAR` tiling.
- *Sparse Residency* builds on (and requires) the `sparseBinding` feature. It includes the following capabilities:
  - Resources do not have to be completely bound to memory before use on the device.
  - Images have a prescribed sparse image block layout, allowing specific rectangular regions of the image to be bound to specific offsets in memory allocations.
  - Consistency of access to unbound regions of the resource is defined by the absence or presence of `VkPhysicalDeviceSparseProperties`::`residencyNonResidentStrict`. If this property is present, accesses to unbound regions of the resource are well defined and behave as if the

data bound is populated with all zeros; writes are discarded. When this property is absent, accesses are considered safe, but reads will return undefined values.

  - Requested via the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT and VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT bits.
  - Sparse residency support is advertised on a finer grain via the following features:
    - The sparseResidencyBuffer feature provides support for creating VkBuffer objects with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT.
    - The sparseResidencyImage2D feature provides support for creating 2D single-sampled VkImage objects with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
    - The sparseResidencyImage3D feature provides support for creating 3D VkImage objects with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
    - The sparseResidency2Samples feature provides support for creating 2D VkImage objects with 2 samples and VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
    - The sparseResidency4Samples feature provides support for creating 2D VkImage objects with 4 samples and VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
    - The sparseResidency8Samples feature provides support for creating 2D VkImage objects with 8 samples and VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
    - The sparseResidency16Samples feature provides support for creating 2D VkImage objects with 16 samples and VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.

      Implementations supporting sparseResidencyImage2D are only **required** to support sparse 2D, single-sampled images. Support for sparse 3D and MSAA images is **optional** and **can** be enabled via sparseResidencyImage3D, sparseResidency2Samples, sparseResidency4Samples, sparseResidency8Samples, and sparseResidency16Samples.
  - A sparse image created using VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT supports all non-compressed color formats with power-of-two element size that non-sparse usage supports. Additional formats **may** also be supported and **can** be queried via vkGetPhysicalDeviceSparseImageFormatProperties. VK_IMAGE_TILING_LINEAR tiling is not supported.
- The sparseResidencyAliased feature provides the following capability that **can** be enabled per resource:

  Allows physical memory ranges to be shared between multiple locations in the same sparse resource or between multiple sparse resources, with each binding of a memory location observing a consistent interpretation of the memory contents.

  See Sparse Memory Aliasing for more information.

31.2. Sparse Buffers and Fully-Resident Images

Both VkBuffer and VkImage objects created with the VK_IMAGE_CREATE_SPARSE_BINDING_BIT or VK_BUFFER_CREATE_SPARSE_BINDING_BIT bits **can** be thought of as a linear region of address space. In the VkImage case if VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT is not used, this linear region is entirely

opaque, meaning that there is no application-visible mapping between texel location and memory offset.

Unless VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT or VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT are also used, the entire resource **must** be bound to one or more VkDeviceMemory objects before use.

31.2.1. Sparse Buffer and Fully-Resident Image Block Size

The sparse block size in bytes for sparse buffers and fully-resident images is reported as VkMemoryRequirements::alignment. alignment represents both the memory alignment requirement and the binding granularity (in bytes) for sparse resources.

31.3. Sparse Partially-Resident Buffers

VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT bit allow the buffer to be made only partially resident. Partially resident VkBuffer objects are allocated and bound identically to VkBuffer objects using only the VK_BUFFER_CREATE_SPARSE_BINDING_BIT feature. The only difference is the ability for some regions of the buffer to be unbound during device use.

31.4. Sparse Partially-Resident Images

VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT bit allow specific rectangular regions of the image called sparse image blocks to be bound to specific ranges of memory. This allows the application to manage residency at either image subresource or sparse image block granularity. Each image subresource (outside of the mip tail) starts on a sparse block boundary and has dimensions that are integer multiples of the corresponding dimensions of the sparse image block.

NOTE

Applications **can** use these types of images to control LOD based on total memory consumption. If memory pressure becomes an issue the application **can** unbind and disable specific mipmap levels of images without having to recreate resources or modify texel data of unaffected levels.

The application **can** also use this functionality to access subregions of the image in a “megatexture” fashion. The application **can** create a large image and only populate the region of the image that is currently being used in the scene.

31.4.1. Accessing Unbound Regions

The following member of VkPhysicalDeviceSparseProperties affects how data in unbound regions of sparse resources are handled by the implementation:

- residencyNonResidentStrict

If this property is not present, reads of unbound regions of the image will return undefined values. Both reads and writes are still considered *safe* and will not affect other resources or populated regions of the image.

If this property is present, all reads of unbound regions of the image will behave as if the region was bound to memory populated with all zeros; writes will be discarded.

Image operations performed on unbound memory **may** still alter some component values in the natural way for those accesses, e.g. substituting a value of one for alpha in formats that do not have an alpha component.

> Example: Reading the alpha component of an unbacked `VK_FORMAT_R8_UNORM` image will return a value of 1.0f.

See Physical Device Enumeration for instructions for retrieving physical device properties.

> **Implementor's Note**
>
> For implementations that **cannot** natively handle access to unbound regions of a resource, the implementation **may** allocate and bind memory to the unbound regions. Reads and writes to unbound regions will access the implementation-managed memory instead.
>
> Given that the values resulting from reads of unbound regions are undefined in this scenario, implementations **may** use the same physical memory for all unbound regions of multiple resources within the same process.

31.4.2. Mip Tail Regions

Sparse images created using `VK_IMAGE_CREATE_SPARSE_BINDING_BIT` (without also using `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT`) have no specific mapping of image region or image subresource to memory offset defined, so the entire image **can** be thought of as a linear opaque address region. However, images created with `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` do have a prescribed sparse image block layout, and hence each image subresource **must** start on a sparse block boundary. Within each array layer, the set of mip levels that have a smaller size than the sparse block size in bytes are grouped together into a *mip tail region*.

If the `VK_SPARSE_IMAGE_FORMAT_ALIGNED_MIP_SIZE_BIT` flag is present in the `flags` member of `VkSparseImageFormatProperties`, for the image's `format`, then any mip level which has dimensions that are not integer multiples of the corresponding dimensions of the sparse image block, and all subsequent mip levels, are also included in the mip tail region.

The following member of `VkPhysicalDeviceSparseProperties` **may** affect how the implementation places mip levels in the mip tail region:

- `residencyAlignedMipSize`

Each mip tail region is bound to memory as an opaque region (i.e. **must** be bound using a VkSparseImageOpaqueMemoryBindInfo structure) and **may** be of a size greater than or equal to the sparse block size in bytes. This size is guaranteed to be an integer multiple of the sparse block size in bytes.

An implementation **may** choose to allow each array-layer's mip tail region to be bound to memory independently or require that all array-layer's mip tail regions be treated as one. This is dictated by `VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT` in `VkSparseImageMemoryRequirements::flags`.

The following diagrams depict how `VK_SPARSE_IMAGE_FORMAT_ALIGNED_MIP_SIZE_BIT` and `VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT` alter memory usage and requirements.

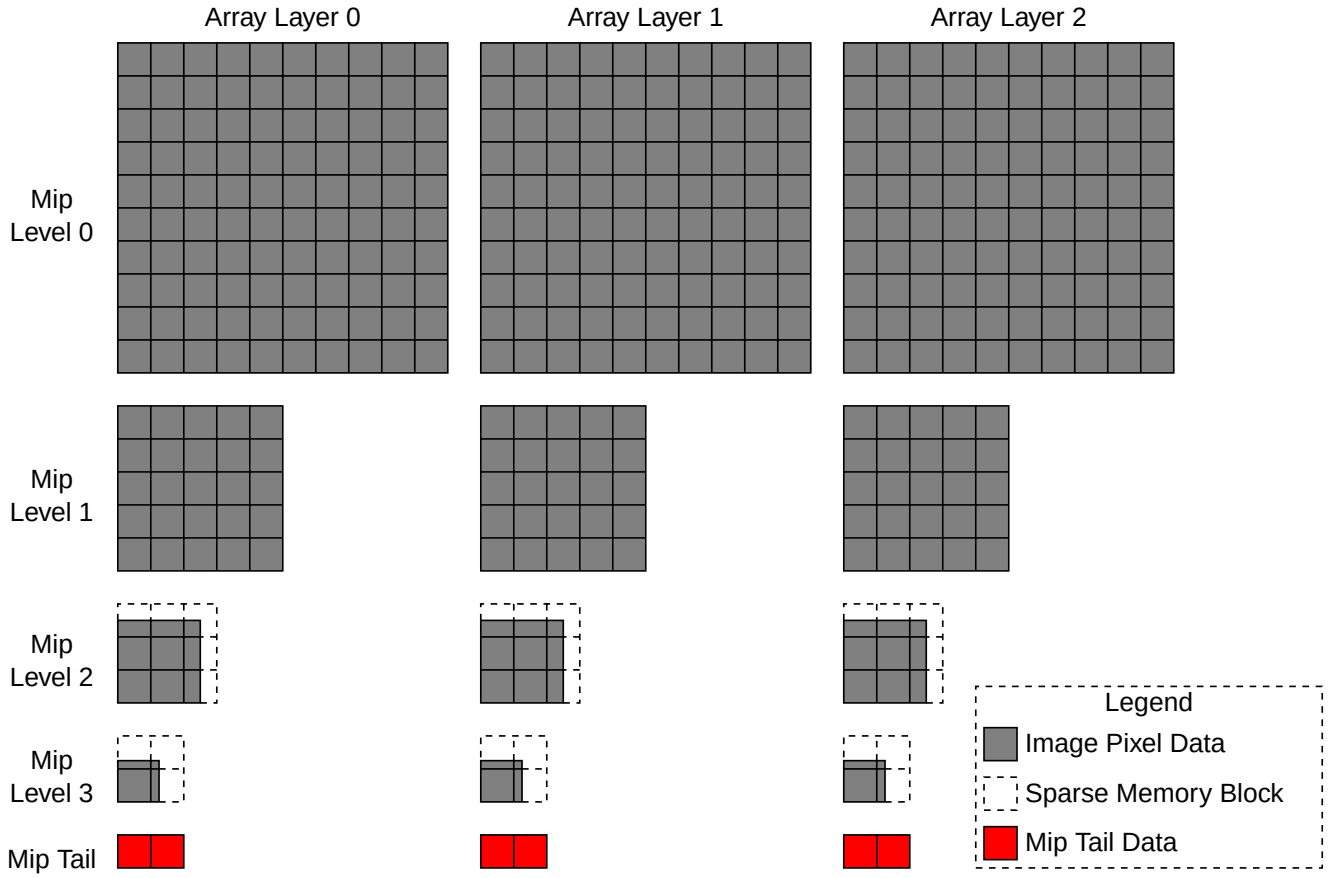


Figure 17. Sparse Image

In the absence of `VK_SPARSE_IMAGE_FORMAT_ALIGNED_MIP_SIZE_BIT` and `VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT`, each array layer contains a mip tail region containing texel data for all mip levels smaller than the sparse image block in any dimension.

Mip levels that are as large or larger than a sparse image block in all dimensions **can** be bound individually. Right-edges and bottom-edges of each level are allowed to have partially used sparse blocks. Any bound partially-used-sparse-blocks **must** still have their full sparse block size in bytes allocated in memory.

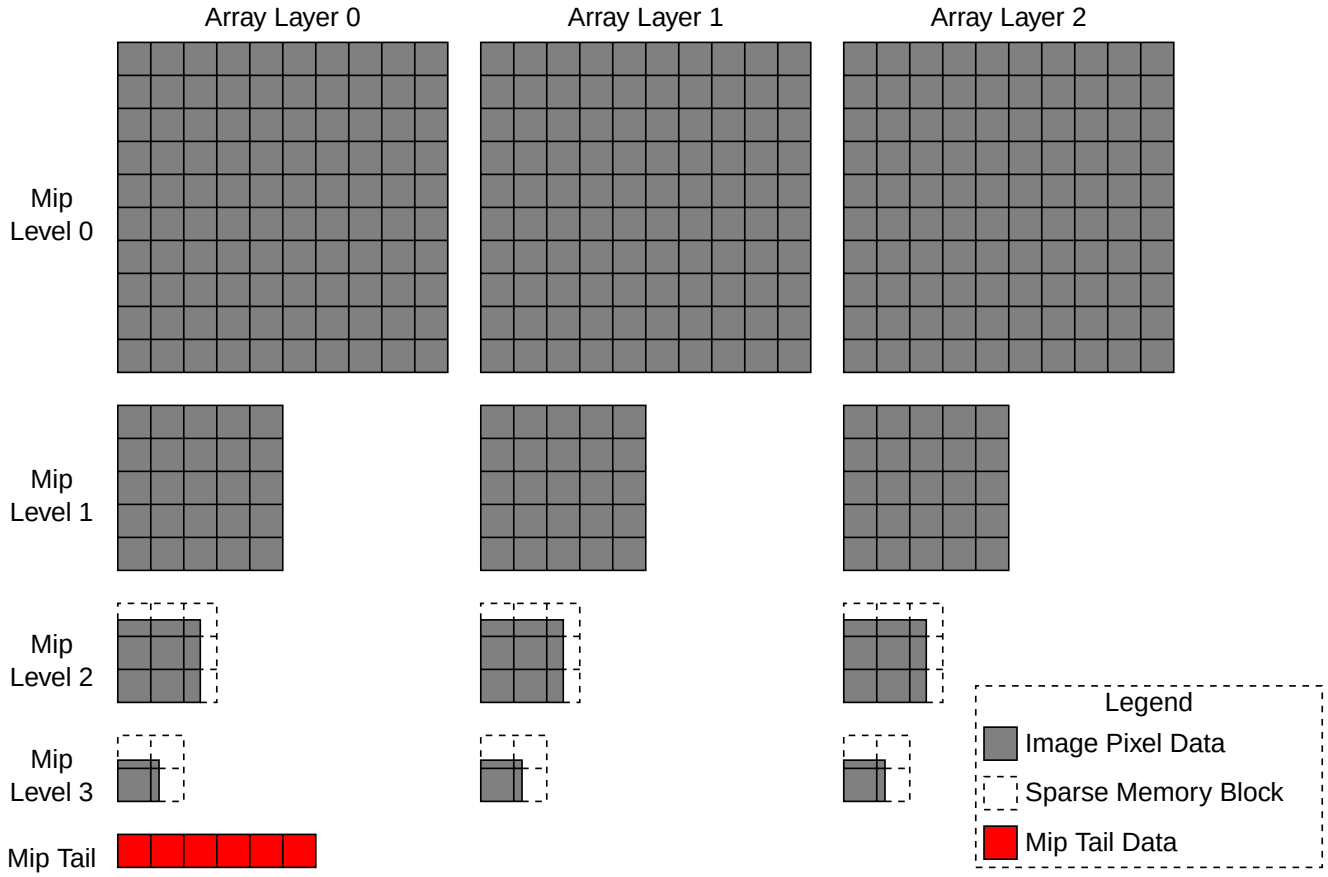


Figure 18. Sparse Image with Single Mip Tail

When `VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT` is present all array layers will share a single mip tail region.

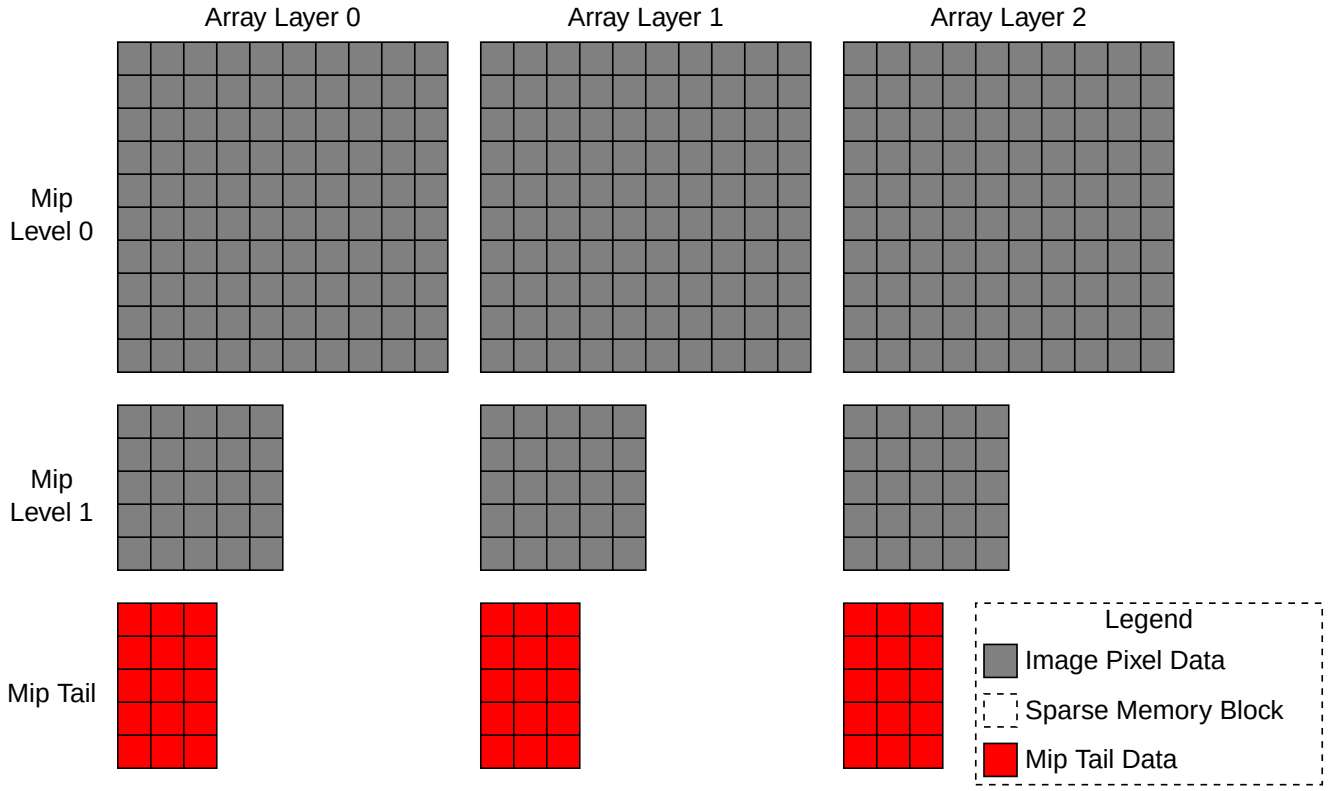


Figure 19. Sparse Image with Aligned Mip Size

NOTE The mip tail regions are presented here in 2D arrays simply for figure size reasons.

> Each mip tail is logically a single array of sparse blocks with an implementation-dependent mapping of texels or compressed texel blocks to sparse blocks.

When VK_SPARSE_IMAGE_FORMAT_ALIGNED_MIP_SIZE_BIT is present the first mip level that would contain partially used sparse blocks begins the mip tail region. This level and all subsequent levels are placed in the mip tail. Only the first N mip levels whose dimensions are an exact multiple of the sparse image block dimensions **can** be bound and unbound on a sparse block basis.

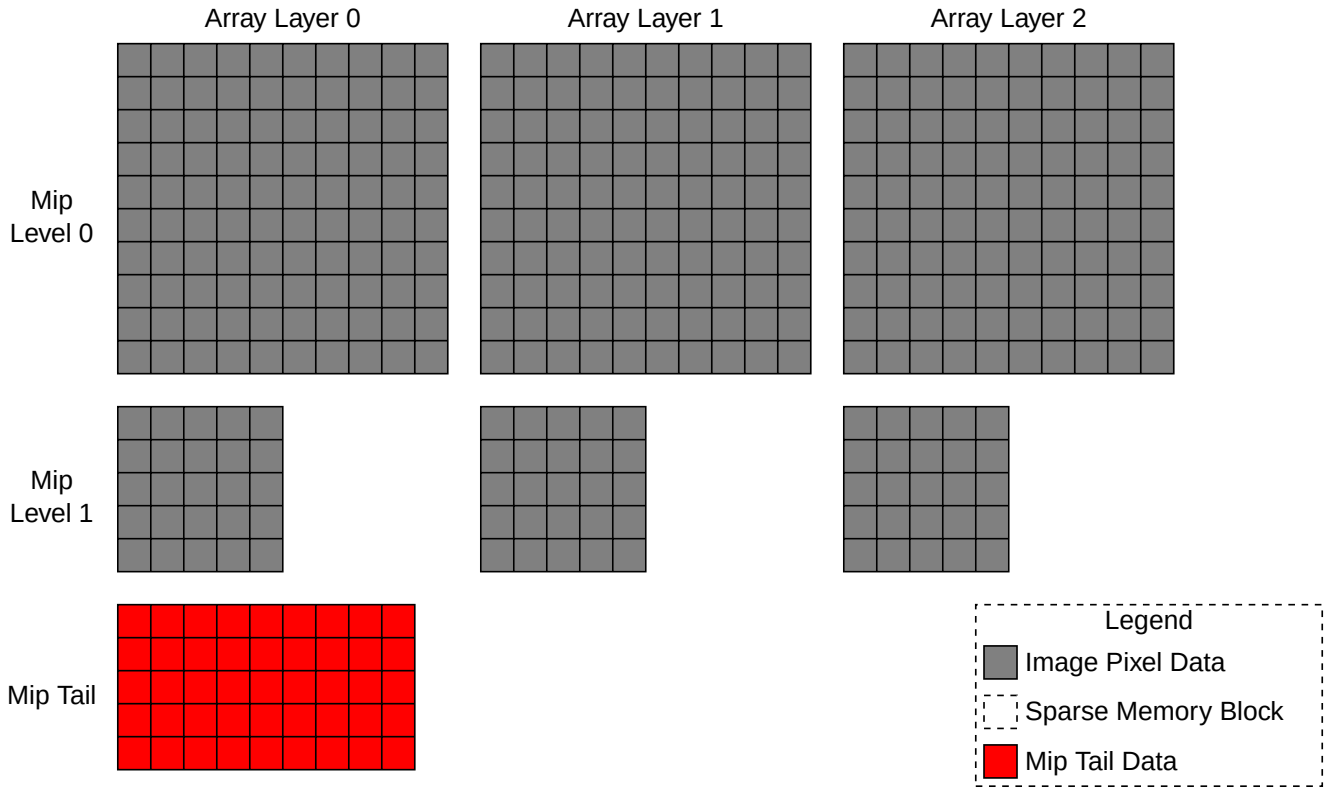


Figure 20. Sparse Image with Aligned Mip Size and Single Mip Tail

> **NOTE** The mip tail region is presented here in a 2D array simply for figure size reasons. It is logically a single array of sparse blocks with an implementation-dependent mapping of texels or compressed texel blocks to sparse blocks.

When both VK_SPARSE_IMAGE_FORMAT_ALIGNED_MIP_SIZE_BIT and VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT are present the constraints from each of these flags are in effect.

31.4.3. Standard Sparse Image Block Shapes

Standard sparse image block shapes define a standard set of dimensions for sparse image blocks that depend on the format of the image. Layout of texels or compressed texel blocks within a sparse image block is implementation-dependent. All currently defined standard sparse image block shapes are 64 KB in size.

For block-compressed formats (e.g. VK_FORMAT_BC5_UNORM_BLOCK), the texel size is the size of the compressed texel block (e.g. 128-bit for BC5) thus the dimensions of the standard sparse image block shapes apply in terms of compressed texel blocks.

NOTE For block-compressed formats, the dimensions of a sparse image block in terms of texels **can** be calculated by multiplying the sparse image block dimensions by the compressed texel block dimensions.

Table 43. Standard Sparse Image Block Shapes (Single Sample)

TEXEL SIZE (bits)	Block Shape (2D)	Block Shape (3D)
8-Bit	256 × 256 × 1	64 × 32 × 32
16-Bit	256 × 128 × 1	32 × 32 × 32
32-Bit	128 × 128 × 1	32 × 32 × 16
64-Bit	128 × 64 × 1	32 × 16 × 16
128-Bit	64 × 64 × 1	16 × 16 × 16

Table 44. Standard Sparse Image Block Shapes (MSAA)

TEXEL SIZE (bits)	Block Shape (2X)	Block Shape (4X)	Block Shape (8X)	Block Shape (16X)
8-Bit	128 × 256 × 1	128 × 128 × 1	64 × 128 × 1	64 × 64 × 1
16-Bit	128 × 128 × 1	128 × 64 × 1	64 × 64 × 1	64 × 32 × 1
32-Bit	64 × 128 × 1	64 × 64 × 1	32 × 64 × 1	32 × 32 × 1
64-Bit	64 × 64 × 1	64 × 32 × 1	32 × 32 × 1	32 × 16 × 1
128-Bit	32 × 64 × 1	32 × 32 × 1	16 × 32 × 1	16 × 16 × 1

Implementations that support the standard sparse image block shape for all formats listed in the Standard Sparse Image Block Shapes (Single Sample) and Standard Sparse Image Block Shapes (MSAA) tables **may** advertise the following `VkPhysicalDeviceSparseProperties`:

- `residencyStandard2DBlockShape`
- `residencyStandard2DMultisampleBlockShape`
- `residencyStandard3DBlockShape`

Reporting each of these features does *not* imply that all possible image types are supported as sparse. Instead, this indicates that no supported sparse image of the corresponding type will use custom sparse image block dimensions for any formats that have a corresponding standard sparse image block shape.

31.4.4. Custom Sparse Image Block Shapes

An implementation that does not support a standard image block shape for a particular sparse partially-resident image **may** choose to support a custom sparse image block shape for it instead. The dimensions of such a custom sparse image block shape are reported in `VkSparseImageFormatProperties`::`imageGranularity`. As with standard sparse image block shapes, the size in bytes of the custom sparse image block shape will be reported in `VkMemoryRequirements`::`alignment`.

Custom sparse image block dimensions are reported through `vkGetPhysicalDeviceSparseImageFormatProperties` and `vkGetImageSparseMemoryRequirements`.

An implementation **must** not support both the standard sparse image block shape and a custom

sparse image block shape for the same image. The standard sparse image block shape **must** be used if it is supported.

31.4.5. Multiple Aspects

Partially resident images are allowed to report separate sparse properties for different aspects of the image. One example is for depth/stencil images where the implementation separates the depth and stencil data into separate planes. Another reason for multiple aspects is to allow the application to manage memory allocation for implementation-private *metadata* associated with the image. See the figure below:

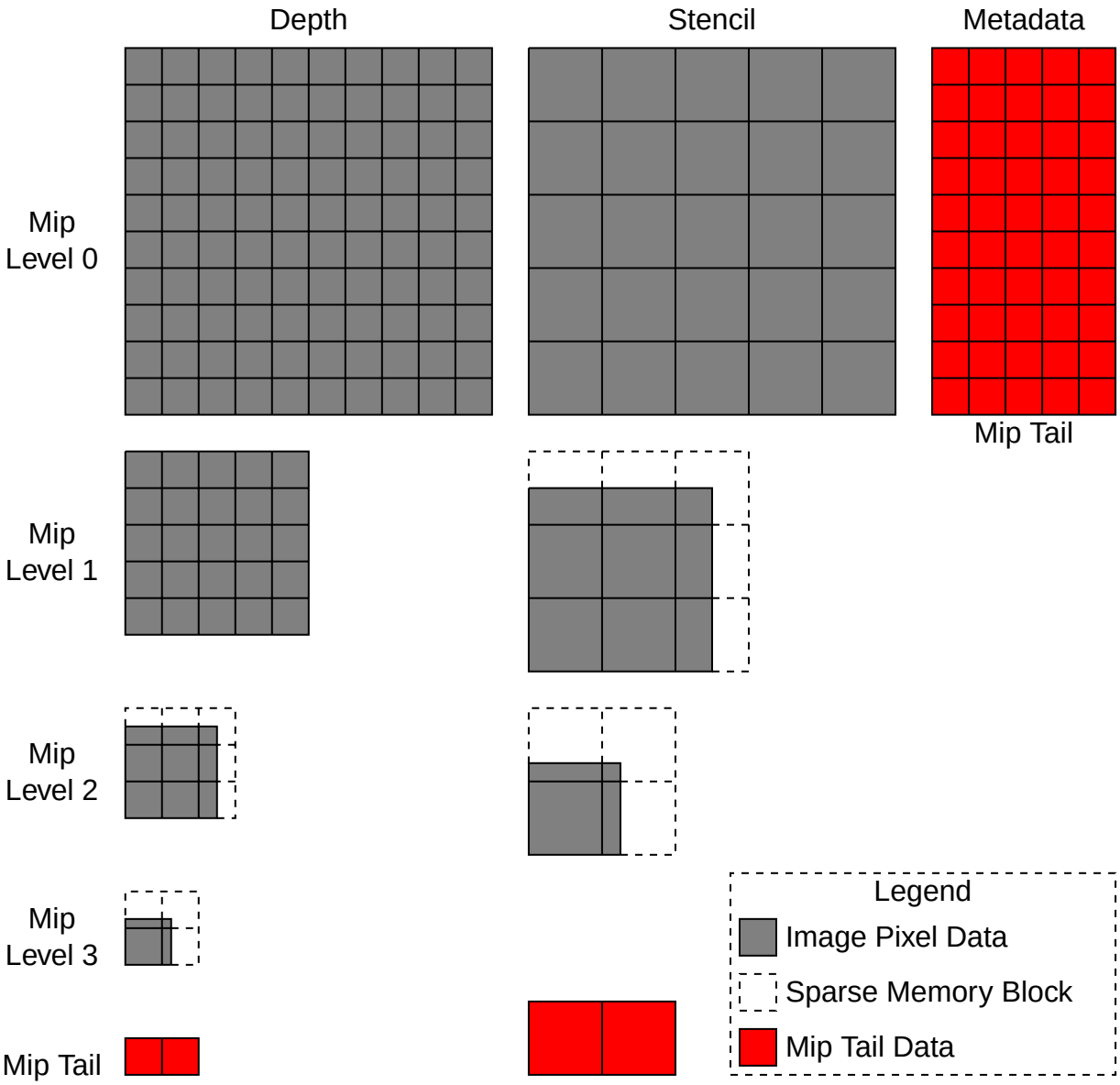


Figure 21. Multiple Aspect Sparse Image

NOTE The mip tail regions are presented here in 2D arrays simply for figure size reasons. Each mip tail is logically a single array of sparse blocks with an implementation-dependent mapping of texels or compressed texel blocks to sparse blocks.

In the figure above the depth, stencil, and metadata aspects all have unique sparse properties. The per-texel stencil data is ¼ the size of the depth data, hence the stencil sparse blocks include 4 × the number of texels. The sparse block size in bytes for all of the aspects is identical and defined by

`VkMemoryRequirements::alignment`.

Metadata

The metadata aspect of an image has the following constraints:

- All metadata is reported in the mip tail region of the metadata aspect.
- All metadata **must** be bound prior to device use of the sparse image.

31.5. Sparse Memory Aliasing

By default sparse resources have the same aliasing rules as non-sparse resources. See Memory Aliasing for more information.

`VkDevice` objects that have the `sparseResidencyAliased` feature enabled are able to use the `VK_BUFFER_CREATE_SPARSE_ALIASED_BIT` and `VK_IMAGE_CREATE_SPARSE_ALIASED_BIT` flags for resource creation. These flags allow resources to access physical memory bound into multiple locations within one or more sparse resources in a *data consistent* fashion. This means that reading physical memory from multiple aliased locations will return the same value.

Care **must** be taken when performing a write operation to aliased physical memory. Memory dependencies **must** be used to separate writes to one alias from reads or writes to another alias. Writes to aliased memory that are not properly guarded against accesses to different aliases will have undefined results for all accesses to the aliased memory.

Applications that wish to make use of data consistent sparse memory aliasing **must** abide by the following guidelines:

- All sparse resources that are bound to aliased physical memory **must** be created with the `VK_BUFFER_CREATE_SPARSE_ALIASED_BIT` / `VK_IMAGE_CREATE_SPARSE_ALIASED_BIT` flag.
- All resources that access aliased physical memory **must** interpret the memory in the same way. This implies the following:
  - Buffers and images **cannot** alias the same physical memory in a data consistent fashion. The physical memory ranges **must** be used exclusively by buffers or used exclusively by images for data consistency to be guaranteed.
  - Memory in sparse image mip tail regions **cannot** access aliased memory in a data consistent fashion.
  - Sparse images that alias the same physical memory **must** have compatible formats and be using the same sparse image block shape in order to access aliased memory in a data consistent fashion.

Failure to follow any of the above guidelines will require the application to abide by the normal, non-sparse resource aliasing rules. In this case memory **cannot** be accessed in a data consistent fashion.

NOTE	Enabling sparse resource memory aliasing **can** be a way to lower physical memory use, but it **may** reduce performance on some implementations. An application

> developer **can** test on their target HW and balance the memory / performance trade-offs measured.

31.6. Sparse Resource Implementation Guidelines (Informative)

This section is Informative. It is included to aid in implementors' understanding of sparse resources.

Device Virtual Address

The basic `sparseBinding` feature allows the resource to reserve its own device virtual address range at resource creation time rather than relying on a bind operation to set this. Without any other creation flags, no other constraints are relaxed compared to normal resources. All pages **must** be bound to physical memory before the device accesses the resource.

The `sparseResidency` features allow sparse resources to be used even when not all pages are bound to memory. Implementations that support access to unbound pages without causing a fault **may** support `residencyNonResidentStrict`.

Not faulting on access to unbound pages is not enough to support `residencyNonResidentStrict`. An implementation **must** also guarantee that reads after writes to unbound regions of the resource always return data for the read as if the memory contains zeros. Depending on any caching hierarchy of the implementation this **may** not always be possible.

Any implementation that does not fault, but does not guarantee correct read values **must** not support `residencyNonResidentStrict`.

Any implementation that **cannot** access unbound pages without causing a fault will require the implementation to bind the entire device virtual address range to physical memory. Any pages that the application does not bind to memory **may** be bound to one (or more) "`placeholder" physical page(s) allocated by the implementation. Given the following properties:

- A process **must** not access memory from another process
- Reads return undefined values

It is sufficient for each host process to allocate these placeholder pages and use them for all resources in that process. Implementations **may** allocate more often (per instance, per device, or per resource).

Binding Memory

The byte size reported in `VkMemoryRequirements`::`size` **must** be greater than or equal to the amount of physical memory **required** to fully populate the resource. Some implementations require "holes" in the device virtual address range that are never accessed. These holes **may** be included in the `size` reported for the resource.

Including or not including the device virtual address holes in the resource size will alter how

the implementation provides support for VkSparseImageOpaqueMemoryBindInfo. This operation **must** be supported for all sparse images, even ones created with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.

- If the holes are included in the size, this bind function becomes very easy. In most cases the resourceOffset is simply a device virtual address offset and the implementation can easily determine what device virtual address to bind. The cost is that the application **may** allocate more physical memory for the resource than it needs.
- If the holes are not included in the size, the application **can** allocate less physical memory than otherwise for the resource. However, in this case the implementation **must** account for the holes when mapping resourceOffset to the actual device virtual address intended to be mapped.

NOTE

If the application always uses VkSparseImageMemoryBindInfo to bind memory for the non-tail mip levels, any holes that are present in the resource size **may** never be bound.

Since VkSparseImageMemoryBindInfo uses texel locations to determine which device virtual addresses to bind, it is impossible to bind device virtual address holes with this operation.

Binding Metadata Memory

All metadata for sparse images have their own sparse properties and are embedded in the mip tail region for said properties. See the Multiaspect section for details.

Given that metadata is in a mip tail region, and the mip tail region **must** be reported as contiguous (either globally or per-array-layer), some implementations will have to resort to complicated offset → device virtual address mapping for handling VkSparseImageOpaqueMemoryBindInfo.

To make this easier on the implementation, the VK_SPARSE_MEMORY_BIND_METADATA_BIT explicitly specifies when metadata is bound with VkSparseImageOpaqueMemoryBindInfo. When this flag is not present, the resourceOffset **may** be treated as a strict device virtual address offset.

When VK_SPARSE_MEMORY_BIND_METADATA_BIT is present, the resourceOffset **must** have been derived explicitly from the imageMipTailOffset in the sparse resource properties returned for the metadata aspect. By manipulating the value returned for imageMipTailOffset, the resourceOffset does not have to correlate directly to a device virtual address offset, and **may** instead be whatever value makes it easiest for the implementation to derive the correct device virtual address.

31.7. Sparse Resource API

The APIs related to sparse resources are grouped into the following categories:

- Physical Device Features
- Physical Device Sparse Properties

- Sparse Image Format Properties
- Sparse Resource Creation
- Sparse Resource Memory Requirements
- Binding Resource Memory

31.7.1. Physical Device Features

Some sparse-resource related features are reported and enabled in VkPhysicalDeviceFeatures. These features **must** be supported and enabled on the VkDevice object before applications **can** use them. See Physical Device Features for information on how to get and set enabled device features, and for more detailed explanations of these features.

Sparse Physical Device Features

- sparseBinding: Support for creating VkBuffer and VkImage objects with the VK_BUFFER_CREATE_SPARSE_BINDING_BIT and VK_IMAGE_CREATE_SPARSE_BINDING_BIT flags, respectively.
- sparseResidencyBuffer: Support for creating VkBuffer objects with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag.
- sparseResidencyImage2D: Support for creating 2D single-sampled VkImage objects with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
- sparseResidencyImage3D: Support for creating 3D VkImage objects with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
- sparseResidency2Samples: Support for creating 2D VkImage objects with 2 samples and VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
- sparseResidency4Samples: Support for creating 2D VkImage objects with 4 samples and VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
- sparseResidency8Samples: Support for creating 2D VkImage objects with 8 samples and VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
- sparseResidency16Samples: Support for creating 2D VkImage objects with 16 samples and VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT.
- sparseResidencyAliased: Support for creating VkBuffer and VkImage objects with the VK_BUFFER_CREATE_SPARSE_ALIASED_BIT and VK_IMAGE_CREATE_SPARSE_ALIASED_BIT flags, respectively.

31.7.2. Physical Device Sparse Properties

Some features of the implementation are not possible to disable, and are reported to allow applications to alter their sparse resource usage accordingly. These read-only capabilities are reported in the VkPhysicalDeviceProperties::sparseProperties member, which is a VkPhysicalDeviceSparseProperties structure.

The VkPhysicalDeviceSparseProperties structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPhysicalDeviceSparseProperties {
    VkBool32    residencyStandard2DBlockShape;
    VkBool32    residencyStandard2DMultisampleBlockShape;
    VkBool32    residencyStandard3DBlockShape;
    VkBool32    residencyAlignedMipSize;
    VkBool32    residencyNonResidentStrict;
} VkPhysicalDeviceSparseProperties;
```

- `residencyStandard2DBlockShape` is `VK_TRUE` if the physical device will access all single-sample 2D sparse resources using the standard sparse image block shapes (based on image format), as described in the Standard Sparse Image Block Shapes (Single Sample) table. If this property is not supported the value returned in the `imageGranularity` member of the `VkSparseImageFormatProperties` structure for single-sample 2D images is not **required** to match the standard sparse image block dimensions listed in the table.
- `residencyStandard2DMultisampleBlockShape` is `VK_TRUE` if the physical device will access all multisample 2D sparse resources using the standard sparse image block shapes (based on image format), as described in the Standard Sparse Image Block Shapes (MSAA) table. If this property is not supported, the value returned in the `imageGranularity` member of the `VkSparseImageFormatProperties` structure for multisample 2D images is not **required** to match the standard sparse image block dimensions listed in the table.
- `residencyStandard3DBlockShape` is `VK_TRUE` if the physical device will access all 3D sparse resources using the standard sparse image block shapes (based on image format), as described in the Standard Sparse Image Block Shapes (Single Sample) table. If this property is not supported, the value returned in the `imageGranularity` member of the `VkSparseImageFormatProperties` structure for 3D images is not **required** to match the standard sparse image block dimensions listed in the table.
- `residencyAlignedMipSize` is `VK_TRUE` if images with mip level dimensions that are not integer multiples of the corresponding dimensions of the sparse image block **may** be placed in the mip tail. If this property is not reported, only mip levels with dimensions smaller than the `imageGranularity` member of the `VkSparseImageFormatProperties` structure will be placed in the mip tail. If this property is reported the implementation is allowed to return `VK_SPARSE_IMAGE_FORMAT_ALIGNED_MIP_SIZE_BIT` in the `flags` member of `VkSparseImageFormatProperties`, indicating that mip level dimensions that are not integer multiples of the corresponding dimensions of the sparse image block will be placed in the mip tail.
- `residencyNonResidentStrict` specifies whether the physical device **can** consistently access non-resident regions of a resource. If this property is `VK_TRUE`, access to non-resident regions of resources will be guaranteed to return values as if the resource was populated with 0; writes to non-resident regions will be discarded.

31.7.3. Sparse Image Format Properties

Given that certain aspects of sparse image support, including the sparse image block dimensions, **may** be implementation-dependent, vkGetPhysicalDeviceSparseImageFormatProperties **can** be used to query for sparse image format properties prior to resource creation. This command is used

to check whether a given set of sparse image parameters is supported and what the sparse image block shape will be.

Sparse Image Format Properties API

The VkSparseImageFormatProperties structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSparseImageFormatProperties {
    VkImageAspectFlags          aspectMask;
    VkExtent3D                  imageGranularity;
    VkSparseImageFormatFlags    flags;
} VkSparseImageFormatProperties;
```

- aspectMask is a bitmask VkImageAspectFlagBits specifying which aspects of the image the properties apply to.
- imageGranularity is the width, height, and depth of the sparse image block in texels or compressed texel blocks.
- flags is a bitmask of VkSparseImageFormatFlagBits specifying additional information about the sparse resource.

Bits which **may** be set in VkSparseImageFormatProperties::flags, specifying additional information about the sparse resource, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSparseImageFormatFlagBits {
    VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT = 0x00000001,
    VK_SPARSE_IMAGE_FORMAT_ALIGNED_MIP_SIZE_BIT = 0x00000002,
    VK_SPARSE_IMAGE_FORMAT_NONSTANDARD_BLOCK_SIZE_BIT = 0x00000004,
} VkSparseImageFormatFlagBits;
```

- VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT specifies that the image uses a single mip tail region for all array layers.
- VK_SPARSE_IMAGE_FORMAT_ALIGNED_MIP_SIZE_BIT specifies that the first mip level whose dimensions are not integer multiples of the corresponding dimensions of the sparse image block begins the mip tail region.
- VK_SPARSE_IMAGE_FORMAT_NONSTANDARD_BLOCK_SIZE_BIT specifies that the image uses non-standard sparse image block dimensions, and the imageGranularity values do not match the standard sparse image block dimensions for the given format.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkSparseImageFormatFlags;
```

VkSparseImageFormatFlags is a bitmask type for setting a mask of zero or more VkSparseImageFormatFlagBits.

vkGetPhysicalDeviceSparseImageFormatProperties returns an array of VkSparseImageFormatProperties. Each element describes properties for one set of image aspects that are bound simultaneously for a VkImage created with the provided image creation parameters. This is usually one element for each aspect in the image, but for interleaved depth/stencil images there is only one element describing the combined aspects.

```
// Provided by VK_VERSION_1_0
void vkGetPhysicalDeviceSparseImageFormatProperties(
    VkPhysicalDevice                            physicalDevice,
    VkFormat                                    format,
    VkImageType                                 type,
    VkSampleCountFlagBits                       samples,
    VkImageUsageFlags                           usage,
    VkImageTiling                               tiling,
    uint32_t*                                   pPropertyCount,
    VkSparseImageFormatProperties*              pProperties);
```

- physicalDevice is the physical device from which to query the sparse image format properties.
- format is the image format.
- type is the dimensionality of the image.
- samples is a VkSampleCountFlagBits value specifying the number of samples per texel.
- usage is a bitmask describing the intended usage of the image.
- tiling is the tiling arrangement of the texel blocks in memory.
- pPropertyCount is a pointer to an integer related to the number of sparse format properties available or queried, as described below.
- pProperties is either NULL or a pointer to an array of VkSparseImageFormatProperties structures.

If pProperties is NULL, then the number of sparse format properties available is returned in pPropertyCount. Otherwise, pPropertyCount **must** point to a variable set by the application to the number of elements in the pProperties array, and on return the variable is overwritten with the number of structures actually written to pProperties. If pPropertyCount is less than the number of sparse format properties available, at most pPropertyCount structures will be written.

If VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT is not supported for the given arguments, pPropertyCount will be zero upon return, and no data will be written to pProperties.

Multiple aspects are returned for depth/stencil images that are implemented as separate planes by the implementation. The depth and stencil data planes each have unique VkSparseImageFormatProperties data.

Depth/stencil images with depth and stencil data interleaved into a single plane will return a single VkSparseImageFormatProperties structure with the aspectMask set to VK_IMAGE_ASPECT_DEPTH_BIT | VK_IMAGE_ASPECT_STENCIL_BIT.

Valid Usage

- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-samples-01094
  `samples` **must** be a valid VkSampleCountFlagBits value that is set in `VkImageFormatProperties`::`sampleCounts` returned by `vkGetPhysicalDeviceImageFormatProperties` with `format`, `type`, `tiling`, and `usage` equal to those in this command

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-format-parameter
  `format` **must** be a valid VkFormat value
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-type-parameter
  `type` **must** be a valid VkImageType value
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-samples-parameter
  `samples` **must** be a valid VkSampleCountFlagBits value
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-usage-parameter
  `usage` **must** be a valid combination of VkImageUsageFlagBits values
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-usage-requiredbitmask
  `usage` **must** not be `0`
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-tiling-parameter
  `tiling` **must** be a valid VkImageTiling value
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkSparseImageFormatProperties structures

`vkGetPhysicalDeviceSparseImageFormatProperties2KHR` returns an array of VkSparseImageFormatProperties2. Each element describes properties for one set of image aspects that are bound simultaneously for a `VkImage` created with the provided image creation parameters. This is usually one element for each aspect in the image, but for interleaved depth/stencil images there is only one element describing the combined aspects.

```
// Provided by VK_KHR_get_physical_device_properties2
void vkGetPhysicalDeviceSparseImageFormatProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceSparseImageFormatInfo2* pFormatInfo,
    uint32_t*                                   pPropertyCount,
    VkSparseImageFormatProperties2*             pProperties);
```

- `physicalDevice` is the physical device from which to query the sparse image format properties.
- `pFormatInfo` is a pointer to a VkPhysicalDeviceSparseImageFormatInfo2 structure containing input parameters to the command.
- `pPropertyCount` is a pointer to an integer related to the number of sparse format properties available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of VkSparseImageFormatProperties2 structures.

`vkGetPhysicalDeviceSparseImageFormatProperties2KHR` behaves identically to vkGetPhysicalDeviceSparseImageFormatProperties, with the ability to return extended information by adding extending structures to the `pNext` chain of its `pProperties` parameter.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSparseImageFormatProperties2-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties2-pFormatInfo-parameter
  `pFormatInfo` **must** be a valid pointer to a valid VkPhysicalDeviceSparseImageFormatInfo2 structure
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties2-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceSparseImageFormatProperties2-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkSparseImageFormatProperties2 structures

The `VkPhysicalDeviceSparseImageFormatInfo2` structure is defined as:

```
typedef struct VkPhysicalDeviceSparseImageFormatInfo2 {
    VkStructureType          sType;
    const void*              pNext;
    VkFormat                 format;
    VkImageType              type;
    VkSampleCountFlagBits    samples;
    VkImageUsageFlags        usage;
    VkImageTiling            tiling;
} VkPhysicalDeviceSparseImageFormatInfo2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkPhysicalDeviceSparseImageFormatInfo2
VkPhysicalDeviceSparseImageFormatInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `format` is the image format.
- `type` is the dimensionality of the image.
- `samples` is a VkSampleCountFlagBits value specifying the number of samples per texel.
- `usage` is a bitmask describing the intended usage of the image.
- `tiling` is the tiling arrangement of the texel blocks in memory.

Valid Usage

- VUID-VkPhysicalDeviceSparseImageFormatInfo2-samples-01095
  `samples` **must** be a valid VkSampleCountFlagBits value that is set in `VkImageFormatProperties`::`sampleCounts` returned by `vkGetPhysicalDeviceImageFormatProperties` with `format`, `type`, `tiling`, and `usage` equal to those in this command

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceSparseImageFormatInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SPARSE_IMAGE_FORMAT_INFO_2`
- VUID-VkPhysicalDeviceSparseImageFormatInfo2-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPhysicalDeviceSparseImageFormatInfo2-format-parameter
  `format` **must** be a valid VkFormat value
- VUID-VkPhysicalDeviceSparseImageFormatInfo2-type-parameter

`type` **must** be a valid VkImageType value

- VUID-VkPhysicalDeviceSparseImageFormatInfo2-samples-parameter
  `samples` **must** be a valid VkSampleCountFlagBits value
- VUID-VkPhysicalDeviceSparseImageFormatInfo2-usage-parameter
  `usage` **must** be a valid combination of VkImageUsageFlagBits values
- VUID-VkPhysicalDeviceSparseImageFormatInfo2-usage-requiredbitmask
  `usage` **must** not be `0`
- VUID-VkPhysicalDeviceSparseImageFormatInfo2-tiling-parameter
  `tiling` **must** be a valid VkImageTiling value

The `VkSparseImageFormatProperties2` structure is defined as:

```
typedef struct VkSparseImageFormatProperties2 {
    VkStructureType                 sType;
    void*                           pNext;
    VkSparseImageFormatProperties   properties;
} VkSparseImageFormatProperties2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkSparseImageFormatProperties2 VkSparseImageFormatProperties2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `properties` is a VkSparseImageFormatProperties structure which is populated with the same values as in vkGetPhysicalDeviceSparseImageFormatProperties.

Valid Usage (Implicit)

- VUID-VkSparseImageFormatProperties2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SPARSE_IMAGE_FORMAT_PROPERTIES_2`
- VUID-VkSparseImageFormatProperties2-pNext-pNext
  `pNext` **must** be `NULL`

31.7.4. Sparse Resource Creation

Sparse resources require that one or more sparse feature flags be specified (as part of the `VkPhysicalDeviceFeatures` structure described previously in the Physical Device Features section) when calling vkCreateDevice. When the appropriate device features are enabled, the `VK_BUFFER_CREATE_SPARSE_*` and `VK_IMAGE_CREATE_SPARSE_*` flags **can** be used. See vkCreateBuffer and vkCreateImage for details of the resource creation APIs.

NOTE	Specifying VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT or VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT requires specifying VK_BUFFER_CREATE_SPARSE_BINDING_BIT or VK_IMAGE_CREATE_SPARSE_BINDING_BIT, respectively, as well. This means that resources **must** be created with the appropriate *_SPARSE_BINDING_BIT to be used with the sparse binding command (vkQueueBindSparse).

31.7.5. Sparse Resource Memory Requirements

Sparse resources have specific memory requirements related to binding sparse memory. These memory requirements are reported differently for VkBuffer objects and VkImage objects.

Buffer and Fully-Resident Images

Buffers (both fully and partially resident) and fully-resident images **can** be bound to memory using only the data from VkMemoryRequirements. For all sparse resources the VkMemoryRequirements::alignment member specifies both the binding granularity in bytes and the **required** alignment of VkDeviceMemory.

Partially Resident Images

Partially resident images have a different method for binding memory. As with buffers and fully resident images, the VkMemoryRequirements::alignment field specifies the binding granularity in bytes for the image.

Requesting sparse memory requirements for VkImage objects using vkGetImageSparseMemoryRequirements will return an array of one or more VkSparseImageMemoryRequirements structures. Each structure describes the sparse memory requirements for a group of aspects of the image.

The sparse image **must** have been created using the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag to retrieve valid sparse image memory requirements.

Sparse Image Memory Requirements

The VkSparseImageMemoryRequirements structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSparseImageMemoryRequirements {
    VkSparseImageFormatProperties    formatProperties;
    uint32_t                         imageMipTailFirstLod;
    VkDeviceSize                     imageMipTailSize;
    VkDeviceSize                     imageMipTailOffset;
    VkDeviceSize                     imageMipTailStride;
} VkSparseImageMemoryRequirements;
```

- formatProperties is a VkSparseImageFormatProperties structure specifying properties of the image format.

- `imageMipTailFirstLod` is the first mip level at which image subresources are included in the mip tail region.
- `imageMipTailSize` is the memory size (in bytes) of the mip tail region. If `formatProperties.flags` contains `VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT`, this is the size of the whole mip tail, otherwise this is the size of the mip tail of a single array layer. This value is guaranteed to be a multiple of the sparse block size in bytes.
- `imageMipTailOffset` is the opaque memory offset used with VkSparseImageOpaqueMemoryBindInfo to bind the mip tail region(s).
- `imageMipTailStride` is the offset stride between each array-layer's mip tail, if `formatProperties.flags` does not contain `VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT` (otherwise the value is undefined).

To query sparse memory requirements for an image, call:

```
// Provided by VK_VERSION_1_0
void vkGetImageSparseMemoryRequirements(
    VkDevice                                    device,
    VkImage                                     image,
    uint32_t*                                   pSparseMemoryRequirementCount,
    VkSparseImageMemoryRequirements*            pSparseMemoryRequirements);
```

- `device` is the logical device that owns the image.
- `image` is the VkImage object to get the memory requirements for.
- `pSparseMemoryRequirementCount` is a pointer to an integer related to the number of sparse memory requirements available or queried, as described below.
- `pSparseMemoryRequirements` is either `NULL` or a pointer to an array of `VkSparseImageMemoryRequirements` structures.

If `pSparseMemoryRequirements` is `NULL`, then the number of sparse memory requirements available is returned in `pSparseMemoryRequirementCount`. Otherwise, `pSparseMemoryRequirementCount` **must** point to a variable set by the application to the number of elements in the `pSparseMemoryRequirements` array, and on return the variable is overwritten with the number of structures actually written to `pSparseMemoryRequirements`. If `pSparseMemoryRequirementCount` is less than the number of sparse memory requirements available, at most `pSparseMemoryRequirementCount` structures will be written.

If the image was not created with `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` then `pSparseMemoryRequirementCount` will be zero and `pSparseMemoryRequirements` will not be written to.

NOTE

It is legal for an implementation to report a larger value in `VkMemoryRequirements::size` than would be obtained by adding together memory sizes for all `VkSparseImageMemoryRequirements` returned by `vkGetImageSparseMemoryRequirements`. This **may** occur when the implementation requires unused padding in the address range describing the resource.

Valid Usage (Implicit)

- VUID-vkGetImageSparseMemoryRequirements-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetImageSparseMemoryRequirements-image-parameter
  `image` **must** be a valid VkImage handle
- VUID-vkGetImageSparseMemoryRequirements-pSparseMemoryRequirementCount-parameter
  `pSparseMemoryRequirementCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetImageSparseMemoryRequirements-pSparseMemoryRequirements-parameter
  If the value referenced by `pSparseMemoryRequirementCount` is not `0`, and `pSparseMemoryRequirements` is not `NULL`, `pSparseMemoryRequirements` **must** be a valid pointer to an array of `pSparseMemoryRequirementCount` VkSparseImageMemoryRequirements structures
- VUID-vkGetImageSparseMemoryRequirements-image-parent
  `image` **must** have been created, allocated, or retrieved from `device`

To query sparse memory requirements for an image, call:

```
// Provided by VK_KHR_get_memory_requirements2
void vkGetImageSparseMemoryRequirements2KHR(
    VkDevice                                    device,
    const VkImageSparseMemoryRequirementsInfo2* pInfo,
    uint32_t*                                   pSparseMemoryRequirementCount,
    VkSparseImageMemoryRequirements2*           pSparseMemoryRequirements);
```

- `device` is the logical device that owns the image.
- `pInfo` is a pointer to a `VkImageSparseMemoryRequirementsInfo2` structure containing parameters required for the memory requirements query.
- `pSparseMemoryRequirementCount` is a pointer to an integer related to the number of sparse memory requirements available or queried, as described below.
- `pSparseMemoryRequirements` is either `NULL` or a pointer to an array of `VkSparseImageMemoryRequirements2` structures.

Valid Usage (Implicit)

- VUID-vkGetImageSparseMemoryRequirements2-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetImageSparseMemoryRequirements2-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkImageSparseMemoryRequirementsInfo2 structure
- VUID-vkGetImageSparseMemoryRequirements2-pSparseMemoryRequirementCount-

parameter
pSparseMemoryRequirementCount **must** be a valid pointer to a uint32_t value

- VUID-vkGetImageSparseMemoryRequirements2-pSparseMemoryRequirements-parameter
If the value referenced by pSparseMemoryRequirementCount is not 0, and pSparseMemoryRequirements is not NULL, pSparseMemoryRequirements **must** be a valid pointer to an array of pSparseMemoryRequirementCount VkSparseImageMemoryRequirements2 structures

To determine the sparse memory requirements for an image resource without creating an object, call:

```
// Provided by VK_KHR_maintenance4
void vkGetDeviceImageSparseMemoryRequirementsKHR(
    VkDevice                                    device,
    const VkDeviceImageMemoryRequirements*      pInfo,
    uint32_t*                                   pSparseMemoryRequirementCount,
    VkSparseImageMemoryRequirements2*           pSparseMemoryRequirements);
```

- device is the logical device intended to own the image.
- pInfo is a pointer to a VkDeviceImageMemoryRequirements structure containing parameters required for the memory requirements query.
- pSparseMemoryRequirementCount is a pointer to an integer related to the number of sparse memory requirements available or queried, as described below.
- pSparseMemoryRequirements is either NULL or a pointer to an array of VkSparseImageMemoryRequirements2 structures.

Valid Usage (Implicit)

- VUID-vkGetDeviceImageSparseMemoryRequirements-device-parameter
device **must** be a valid VkDevice handle
- VUID-vkGetDeviceImageSparseMemoryRequirements-pInfo-parameter
pInfo **must** be a valid pointer to a valid VkDeviceImageMemoryRequirements structure
- VUID-vkGetDeviceImageSparseMemoryRequirements-pSparseMemoryRequirementCount-parameter
pSparseMemoryRequirementCount **must** be a valid pointer to a uint32_t value
- VUID-vkGetDeviceImageSparseMemoryRequirements-pSparseMemoryRequirements-parameter
If the value referenced by pSparseMemoryRequirementCount is not 0, and pSparseMemoryRequirements is not NULL, pSparseMemoryRequirements **must** be a valid pointer to an array of pSparseMemoryRequirementCount VkSparseImageMemoryRequirements2 structures

The VkImageSparseMemoryRequirementsInfo2 structure is defined as:

```
typedef struct VkImageSparseMemoryRequirementsInfo2 {
    VkStructureType    sType;
    const void*        pNext;
    VkImage            image;
} VkImageSparseMemoryRequirementsInfo2;
```

or the equivalent

```
// Provided by VK_KHR_get_memory_requirements2
typedef VkImageSparseMemoryRequirementsInfo2 VkImageSparseMemoryRequirementsInfo2KHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- image is the image to query.

Valid Usage (Implicit)

- VUID-VkImageSparseMemoryRequirementsInfo2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_IMAGE_SPARSE_MEMORY_REQUIREMENTS_INFO_2
- VUID-VkImageSparseMemoryRequirementsInfo2-pNext-pNext
  pNext **must** be NULL
- VUID-VkImageSparseMemoryRequirementsInfo2-image-parameter
  image **must** be a valid VkImage handle

The VkSparseImageMemoryRequirements2 structure is defined as:

```
typedef struct VkSparseImageMemoryRequirements2 {
    VkStructureType                    sType;
    void*                              pNext;
    VkSparseImageMemoryRequirements    memoryRequirements;
} VkSparseImageMemoryRequirements2;
```

or the equivalent

```
// Provided by VK_KHR_get_memory_requirements2
typedef VkSparseImageMemoryRequirements2 VkSparseImageMemoryRequirements2KHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- memoryRequirements is a VkSparseImageMemoryRequirements structure describing the memory

requirements of the sparse image.

Valid Usage (Implicit)

- VUID-VkSparseImageMemoryRequirements2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SPARSE_IMAGE_MEMORY_REQUIREMENTS_2`
- VUID-VkSparseImageMemoryRequirements2-pNext-pNext
  `pNext` **must** be `NULL`

31.7.6. Binding Resource Memory

Non-sparse resources are backed by a single physical allocation prior to device use (via `vkBindImageMemory` or `vkBindBufferMemory`), and their backing **must** not be changed. On the other hand, sparse resources **can** be bound to memory non-contiguously and these bindings **can** be altered during the lifetime of the resource.

NOTE It is important to note that freeing a `VkDeviceMemory` object with `vkFreeMemory` will not cause resources (or resource regions) bound to the memory object to become unbound. Applications **must** not access resources bound to memory that has been freed.

Sparse memory bindings execute on a queue that includes the `VK_QUEUE_SPARSE_BINDING_BIT` bit. Applications **must** use synchronization primitives to guarantee that other queues do not access ranges of memory concurrently with a binding change. Applications **can** access other ranges of the same resource while a bind operation is executing.

NOTE Implementations **must** provide a guarantee that simultaneously binding sparse blocks while another queue accesses those same sparse blocks via a sparse resource **must** not access memory owned by another process or otherwise corrupt the system.

While some implementations **may** include `VK_QUEUE_SPARSE_BINDING_BIT` support in queue families that also include graphics and compute support, other implementations **may** only expose a `VK_QUEUE_SPARSE_BINDING_BIT`-only queue family. In either case, applications **must** use synchronization primitives to explicitly request any ordering dependencies between sparse memory binding operations and other graphics/compute/transfer operations, as sparse binding operations are not automatically ordered against command buffer execution, even within a single queue.

When binding memory explicitly for the `VK_IMAGE_ASPECT_METADATA_BIT` the application **must** use the `VK_SPARSE_MEMORY_BIND_METADATA_BIT` in the `VkSparseMemoryBind`::`flags` field when binding memory. Binding memory for metadata is done the same way as binding memory for the mip tail, with the addition of the `VK_SPARSE_MEMORY_BIND_METADATA_BIT` flag.

Binding the mip tail for any aspect **must** only be performed using VkSparseImageOpaqueMemoryBindInfo. If `formatProperties.flags` contains `VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT`, then it **can** be bound with a single

VkSparseMemoryBind structure, with resourceOffset = imageMipTailOffset and size = imageMipTailSize.

If formatProperties.flags does not contain VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT then the offset for the mip tail in each array layer is given as:

```
arrayMipTailOffset = imageMipTailOffset + arrayLayer * imageMipTailStride;
```

and the mip tail **can** be bound with layerCount VkSparseMemoryBind structures, each using size = imageMipTailSize and resourceOffset = arrayMipTailOffset as defined above.

Sparse memory binding is handled by the following APIs and related data structures.

Sparse Memory Binding Functions

The VkSparseMemoryBind structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSparseMemoryBind {
    VkDeviceSize               resourceOffset;
    VkDeviceSize               size;
    VkDeviceMemory             memory;
    VkDeviceSize               memoryOffset;
    VkSparseMemoryBindFlags    flags;
} VkSparseMemoryBind;
```

- resourceOffset is the offset into the resource.
- size is the size of the memory region to be bound.
- memory is the VkDeviceMemory object that the range of the resource is bound to. If memory is VK_NULL_HANDLE, the range is unbound.
- memoryOffset is the offset into the VkDeviceMemory object to bind the resource range to. If memory is VK_NULL_HANDLE, this value is ignored.
- flags is a bitmask of VkSparseMemoryBindFlagBits specifying usage of the binding operation.

The *binding range* [resourceOffset, resourceOffset + size) has different constraints based on flags. If flags contains VK_SPARSE_MEMORY_BIND_METADATA_BIT, the binding range **must** be within the mip tail region of the metadata aspect. This metadata region is defined by:

$$\text{metadataRegion} = [\text{base}, \text{base} + \text{imageMipTailSize})$$

$$\text{base} = \text{imageMipTailOffset} + \text{imageMipTailStride} \times n$$

and imageMipTailOffset, imageMipTailSize, and imageMipTailStride values are from the VkSparseImageMemoryRequirements corresponding to the metadata aspect of the image, and n is a

valid array layer index for the image,

`imageMipTailStride` is considered to be zero for aspects where `VkSparseImageMemoryRequirements::formatProperties.flags` contains `VK_SPARSE_IMAGE_FORMAT_SINGLE_MIPTAIL_BIT`.

If `flags` does not contain `VK_SPARSE_MEMORY_BIND_METADATA_BIT`, the binding range **must** be within the range [0,VkMemoryRequirements::`size`).

Valid Usage

- VUID-VkSparseMemoryBind-memory-01096
  If `memory` is not VK_NULL_HANDLE, `memory` and `memoryOffset` **must** match the memory requirements of the resource, as described in section Resource Memory Association
- VUID-VkSparseMemoryBind-resourceOffset-09491
  If the resource being bound is a `VkBuffer`, `resourceOffset`, `memoryOffset` and `size` **must** be an integer multiple of the `alignment` of the VkMemoryRequirements structure returned from a call to vkGetBufferMemoryRequirements with the buffer resource
- VUID-VkSparseMemoryBind-resourceOffset-09492
  If the resource being bound is a `VkImage`, `resourceOffset` and `memoryOffset` **must** be an integer multiple of the `alignment` of the VkMemoryRequirements structure returned from a call to vkGetImageMemoryRequirements with the image resource
- VUID-VkSparseMemoryBind-memory-01097
  If `memory` is not VK_NULL_HANDLE, `memory` **must** not have been created with a memory type that reports `VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT` bit set
- VUID-VkSparseMemoryBind-size-01098
  `size` **must** be greater than `0`
- VUID-VkSparseMemoryBind-resourceOffset-01099
  `resourceOffset` **must** be less than the size of the resource
- VUID-VkSparseMemoryBind-size-01100
  `size` **must** be less than or equal to the size of the resource minus `resourceOffset`
- VUID-VkSparseMemoryBind-memoryOffset-01101
  `memoryOffset` **must** be less than the size of `memory`
- VUID-VkSparseMemoryBind-size-01102
  `size` **must** be less than or equal to the size of `memory` minus `memoryOffset`
- VUID-VkSparseMemoryBind-memory-02730
  If `memory` was created with VkExportMemoryAllocateInfo::`handleTypes` not equal to `0`, at least one handle type it contained **must** also have been set in VkExternalMemoryBufferCreateInfo::`handleTypes` or VkExternalMemoryImageCreateInfo::`handleTypes` when the resource was created
- VUID-VkSparseMemoryBind-memory-02731
  If `memory` was created by a memory import operation, the external handle type of the imported memory **must** also have been set in VkExternalMemoryBufferCreateInfo::`handleTypes` or VkExternalMemoryImageCreateInfo::`handleTypes` when the resource was created

Valid Usage (Implicit)

- VUID-VkSparseMemoryBind-memory-parameter
  If `memory` is not VK_NULL_HANDLE, `memory` **must** be a valid VkDeviceMemory handle
- VUID-VkSparseMemoryBind-flags-parameter
  `flags` **must** be a valid combination of VkSparseMemoryBindFlagBits values

Bits which **can** be set in VkSparseMemoryBind::`flags`, specifying usage of a sparse memory binding operation, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSparseMemoryBindFlagBits {
    VK_SPARSE_MEMORY_BIND_METADATA_BIT = 0x00000001,
} VkSparseMemoryBindFlagBits;
```

- `VK_SPARSE_MEMORY_BIND_METADATA_BIT` specifies that the memory being bound is only for the metadata aspect.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkSparseMemoryBindFlags;
```

`VkSparseMemoryBindFlags` is a bitmask type for setting a mask of zero or more VkSparseMemoryBindFlagBits.

Memory is bound to `VkBuffer` objects created with the `VK_BUFFER_CREATE_SPARSE_BINDING_BIT` flag using the following structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkSparseBufferMemoryBindInfo {
    VkBuffer                     buffer;
    uint32_t                     bindCount;
    const VkSparseMemoryBind*    pBinds;
} VkSparseBufferMemoryBindInfo;
```

- `buffer` is the VkBuffer object to be bound.
- `bindCount` is the number of VkSparseMemoryBind structures in the `pBinds` array.
- `pBinds` is a pointer to an array of VkSparseMemoryBind structures.

Valid Usage (Implicit)

- VUID-VkSparseBufferMemoryBindInfo-buffer-parameter
  `buffer` **must** be a valid VkBuffer handle
- VUID-VkSparseBufferMemoryBindInfo-pBinds-parameter

`pBinds` **must** be a valid pointer to an array of `bindCount` valid VkSparseMemoryBind structures

- VUID-VkSparseBufferMemoryBindInfo-bindCount-arraylength
  `bindCount` **must** be greater than `0`

Memory is bound to opaque regions of `VkImage` objects created with the `VK_IMAGE_CREATE_SPARSE_BINDING_BIT` flag using the following structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkSparseImageOpaqueMemoryBindInfo {
    VkImage                      image;
    uint32_t                     bindCount;
    const VkSparseMemoryBind*    pBinds;
} VkSparseImageOpaqueMemoryBindInfo;
```

- `image` is the VkImage object to be bound.
- `bindCount` is the number of VkSparseMemoryBind structures in the `pBinds` array.
- `pBinds` is a pointer to an array of VkSparseMemoryBind structures.

NOTE

This structure is normally used to bind memory to fully-resident sparse images or for mip tail regions of partially resident images. However, it **can** also be used to bind memory for the entire binding range of partially resident images.

If the `pBinds`[i].flags of an element *i* of `pBinds` does not contain `VK_SPARSE_MEMORY_BIND_METADATA_BIT`, the `resourceOffset` is in the range [0, VkMemoryRequirements::`size`), This range includes data from all aspects of the image, including metadata. For most implementations this will probably mean that the `resourceOffset` is a simple device address offset within the resource. It is possible for an application to bind a range of memory that includes both resource data and metadata. However, the application would not know what part of the image the memory is used for, or if any range is being used for metadata.

If the `pBinds`[i].flags of an element *i* of `pBinds` contains `VK_SPARSE_MEMORY_BIND_METADATA_BIT`, the binding range specified **must** be within the mip tail region of the metadata aspect. In this case the `resourceOffset` is not **required** to be a simple device address offset within the resource. However, it *is* defined to be within [`imageMipTailOffset`, `imageMipTailOffset` + `imageMipTailSize`) for the metadata aspect. See VkSparseMemoryBind for the full constraints on binding region with this flag present.

Valid Usage

- VUID-VkSparseImageOpaqueMemoryBindInfo-pBinds-01103
  If the `flags` member of any element of `pBinds` contains `VK_SPARSE_MEMORY_BIND_METADATA_BIT`, the binding range defined **must** be within the mip

tail region of the metadata aspect of image

Valid Usage (Implicit)

- VUID-VkSparseImageOpaqueMemoryBindInfo-image-parameter
  image **must** be a valid VkImage handle
- VUID-VkSparseImageOpaqueMemoryBindInfo-pBinds-parameter
  pBinds **must** be a valid pointer to an array of bindCount valid VkSparseMemoryBind structures
- VUID-VkSparseImageOpaqueMemoryBindInfo-bindCount-arraylength
  bindCount **must** be greater than 0

Memory **can** be bound to sparse image blocks of VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag using the following structure:

```
// Provided by VK_VERSION_1_0
typedef struct VkSparseImageMemoryBindInfo {
    VkImage                          image;
    uint32_t                         bindCount;
    const VkSparseImageMemoryBind*   pBinds;
} VkSparseImageMemoryBindInfo;
```

- image is the VkImage object to be bound
- bindCount is the number of VkSparseImageMemoryBind structures in pBinds array
- pBinds is a pointer to an array of VkSparseImageMemoryBind structures

Valid Usage

- VUID-VkSparseImageMemoryBindInfo-subresource-01722
  The subresource.mipLevel member of each element of pBinds **must** be less than the mipLevels specified in VkImageCreateInfo when image was created
- VUID-VkSparseImageMemoryBindInfo-subresource-01723
  The subresource.arrayLayer member of each element of pBinds **must** be less than the arrayLayers specified in VkImageCreateInfo when image was created
- VUID-VkSparseImageMemoryBindInfo-subresource-01106
  The subresource.aspectMask member of each element of pBinds **must** be valid for the format specified in VkImageCreateInfo when image was created
- VUID-VkSparseImageMemoryBindInfo-image-02901
  image **must** have been created with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT set

Valid Usage (Implicit)

- VUID-VkSparseImageMemoryBindInfo-image-parameter
  `image` **must** be a valid VkImage handle
- VUID-VkSparseImageMemoryBindInfo-pBinds-parameter
  `pBinds` **must** be a valid pointer to an array of `bindCount` valid VkSparseImageMemoryBind structures
- VUID-VkSparseImageMemoryBindInfo-bindCount-arraylength
  `bindCount` **must** be greater than `0`

The `VkSparseImageMemoryBind` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkSparseImageMemoryBind {
    VkImageSubresource         subresource;
    VkOffset3D                 offset;
    VkExtent3D                 extent;
    VkDeviceMemory             memory;
    VkDeviceSize               memoryOffset;
    VkSparseMemoryBindFlags    flags;
} VkSparseImageMemoryBind;
```

- `subresource` is the image *aspect* and region of interest in the image.
- `offset` are the coordinates of the first texel within the image subresource to bind.
- `extent` is the size in texels of the region within the image subresource to bind. The extent **must** be a multiple of the sparse image block dimensions, except when binding sparse image blocks along the edge of an image subresource it **can** instead be such that any coordinate of `offset` + `extent` equals the corresponding dimensions of the image subresource.
- `memory` is the VkDeviceMemory object that the sparse image blocks of the image are bound to. If `memory` is VK_NULL_HANDLE, the sparse image blocks are unbound.
- `memoryOffset` is an offset into VkDeviceMemory object. If `memory` is VK_NULL_HANDLE, this value is ignored.
- `flags` are sparse memory binding flags.

Valid Usage

- VUID-VkSparseImageMemoryBind-memory-01104
  If the `sparseResidencyAliased` feature is not enabled, and if any other resources are bound to ranges of `memory`, the range of `memory` being bound **must** not overlap with those bound ranges
- VUID-VkSparseImageMemoryBind-memory-01105
  `memory` and `memoryOffset` **must** match the memory requirements of the calling command's `image`, as described in section Resource Memory Association

- VUID-VkSparseImageMemoryBind-offset-01107
  `offset.x` **must** be a multiple of the sparse image block width (`VkSparseImageFormatProperties`::`imageGranularity.width`) of the image
- VUID-VkSparseImageMemoryBind-extent-09388
  `extent.width` **must** be greater than `0`
- VUID-VkSparseImageMemoryBind-extent-01108
  `extent.width` **must** either be a multiple of the sparse image block width of the image, or else (`extent.width` + `offset.x`) **must** equal the width of the image subresource
- VUID-VkSparseImageMemoryBind-offset-01109
  `offset.y` **must** be a multiple of the sparse image block height (`VkSparseImageFormatProperties`::`imageGranularity.height`) of the image
- VUID-VkSparseImageMemoryBind-extent-09389
  `extent.height` **must** be greater than `0`
- VUID-VkSparseImageMemoryBind-extent-01110
  `extent.height` **must** either be a multiple of the sparse image block height of the image, or else (`extent.height` + `offset.y`) **must** equal the height of the image subresource
- VUID-VkSparseImageMemoryBind-offset-01111
  `offset.z` **must** be a multiple of the sparse image block depth (`VkSparseImageFormatProperties`::`imageGranularity.depth`) of the image
- VUID-VkSparseImageMemoryBind-extent-09390
  `extent.depth` **must** be greater than `0`
- VUID-VkSparseImageMemoryBind-extent-01112
  `extent.depth` **must** either be a multiple of the sparse image block depth of the image, or else (`extent.depth` + `offset.z`) **must** equal the depth of the image subresource
- VUID-VkSparseImageMemoryBind-memory-02732
  If `memory` was created with VkExportMemoryAllocateInfo::`handleTypes` not equal to `0`, at least one handle type it contained **must** also have been set in VkExternalMemoryImageCreateInfo::`handleTypes` when the image was created
- VUID-VkSparseImageMemoryBind-memory-02733
  If `memory` was created by a memory import operation, the external handle type of the imported memory **must** also have been set in VkExternalMemoryImageCreateInfo ::`handleTypes` when `image` was created

Valid Usage (Implicit)

- VUID-VkSparseImageMemoryBind-subresource-parameter
  `subresource` **must** be a valid VkImageSubresource structure
- VUID-VkSparseImageMemoryBind-memory-parameter
  If `memory` is not VK_NULL_HANDLE, `memory` **must** be a valid VkDeviceMemory handle
- VUID-VkSparseImageMemoryBind-flags-parameter
  `flags` **must** be a valid combination of VkSparseMemoryBindFlagBits values

To submit sparse binding operations to a queue, call:

```
// Provided by VK_VERSION_1_0
VkResult vkQueueBindSparse(
    VkQueue                                     queue,
    uint32_t                                    bindInfoCount,
    const VkBindSparseInfo*                     pBindInfo,
    VkFence                                     fence);
```

- `queue` is the queue that the sparse binding operations will be submitted to.
- `bindInfoCount` is the number of elements in the `pBindInfo` array.
- `pBindInfo` is a pointer to an array of VkBindSparseInfo structures, each specifying a sparse binding submission batch.
- `fence` is an **optional** handle to a fence to be signaled. If `fence` is not VK_NULL_HANDLE, it defines a fence signal operation.

`vkQueueBindSparse` is a queue submission command, with each batch defined by an element of `pBindInfo` as a VkBindSparseInfo structure. Batches begin execution in the order they appear in `pBindInfo`, but **may** complete out of order.

Within a batch, a given range of a resource **must** not be bound more than once. Across batches, if a range is to be bound to one allocation and offset and then to another allocation and offset, then the application **must** guarantee (usually using semaphores) that the binding operations are executed in the correct order, as well as to order binding operations against the execution of command buffer submissions.

As no operation to vkQueueBindSparse causes any pipeline stage to access memory, synchronization primitives used in this command effectively only define execution dependencies.

Additional information about fence and semaphore operation is described in the synchronization chapter.

Valid Usage

- VUID-vkQueueBindSparse-fence-01113
  If `fence` is not VK_NULL_HANDLE, `fence` **must** be unsignaled
- VUID-vkQueueBindSparse-fence-01114
  If `fence` is not VK_NULL_HANDLE, `fence` **must** not be associated with any other queue command that has not yet completed execution on that queue
- VUID-vkQueueBindSparse-pSignalSemaphores-01115
  Each element of the `pSignalSemaphores` member of each element of `pBindInfo` **must** be unsignaled when the semaphore signal operation it defines is executed on the device
- VUID-vkQueueBindSparse-pWaitSemaphores-01116
  When a semaphore wait operation referring to a binary semaphore defined by any element of the `pWaitSemaphores` member of any element of `pBindInfo` executes on `queue`,

there **must** be no other queues waiting on the same semaphore

- VUID-vkQueueBindSparse-pWaitSemaphores-03245
  All elements of the `pWaitSemaphores` member of all elements of `pBindInfo` referring to a semaphore created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_BINARY` **must** reference a semaphore signal operation that has been submitted for execution and any semaphore signal operations on which it depends **must** have also been submitted for execution

Valid Usage (Implicit)

- VUID-vkQueueBindSparse-queue-parameter
  `queue` **must** be a valid VkQueue handle
- VUID-vkQueueBindSparse-pBindInfo-parameter
  If `bindInfoCount` is not `0`, `pBindInfo` **must** be a valid pointer to an array of `bindInfoCount` valid VkBindSparseInfo structures
- VUID-vkQueueBindSparse-fence-parameter
  If `fence` is not VK_NULL_HANDLE, `fence` **must** be a valid VkFence handle
- VUID-vkQueueBindSparse-queuetype
  The `queue` **must** support sparse binding operations
- VUID-vkQueueBindSparse-commonparent
  Both of `fence`, and `queue` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `queue` **must** be externally synchronized
- Host access to `fence` **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
-	-	-	SPARSE_BINDING	-

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST

The VkBindSparseInfo structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkBindSparseInfo {
    VkStructureType                              sType;
    const void*                                  pNext;
    uint32_t                                     waitSemaphoreCount;
    const VkSemaphore*                           pWaitSemaphores;
    uint32_t                                     bufferBindCount;
    const VkSparseBufferMemoryBindInfo*          pBufferBinds;
    uint32_t                                     imageOpaqueBindCount;
    const VkSparseImageOpaqueMemoryBindInfo*     pImageOpaqueBinds;
    uint32_t                                     imageBindCount;
    const VkSparseImageMemoryBindInfo*           pImageBinds;
    uint32_t                                     signalSemaphoreCount;
    const VkSemaphore*                           pSignalSemaphores;
} VkBindSparseInfo;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- waitSemaphoreCount is the number of semaphores upon which to wait before executing the sparse binding operations for the batch.
- pWaitSemaphores is a pointer to an array of semaphores upon which to wait on before the sparse binding operations for this batch begin execution. If semaphores to wait on are provided, they define a semaphore wait operation.
- bufferBindCount is the number of sparse buffer bindings to perform in the batch.
- pBufferBinds is a pointer to an array of VkSparseBufferMemoryBindInfo structures.
- imageOpaqueBindCount is the number of opaque sparse image bindings to perform.
- pImageOpaqueBinds is a pointer to an array of VkSparseImageOpaqueMemoryBindInfo structures, indicating opaque sparse image bindings to perform.
- imageBindCount is the number of sparse image bindings to perform.
- pImageBinds is a pointer to an array of VkSparseImageMemoryBindInfo structures, indicating sparse image bindings to perform.
- signalSemaphoreCount is the number of semaphores to be signaled once the sparse binding operations specified by the structure have completed execution.
- pSignalSemaphores is a pointer to an array of semaphores which will be signaled when the sparse binding operations for this batch have completed execution. If semaphores to be signaled are provided, they define a semaphore signal operation.

Valid Usage

- VUID-VkBindSparseInfo-pWaitSemaphores-03246
  If any element of pWaitSemaphores or pSignalSemaphores was created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE then the pNext chain **must** include a VkTimelineSemaphoreSubmitInfo structure
- VUID-VkBindSparseInfo-pNext-03247
  If the pNext chain of this structure includes a VkTimelineSemaphoreSubmitInfo structure and any element of pWaitSemaphores was created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE then its waitSemaphoreValueCount member **must** equal waitSemaphoreCount
- VUID-VkBindSparseInfo-pNext-03248
  If the pNext chain of this structure includes a VkTimelineSemaphoreSubmitInfo structure and any element of pSignalSemaphores was created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE then its signalSemaphoreValueCount member **must** equal signalSemaphoreCount
- VUID-VkBindSparseInfo-pSignalSemaphores-03249
  For each element of pSignalSemaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE the corresponding element of VkTimelineSemaphoreSubmitInfo::pSignalSemaphoreValues **must** have a value greater than the current value of the semaphore when the semaphore signal operation is executed
- VUID-VkBindSparseInfo-pWaitSemaphores-03250
  For each element of pWaitSemaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE the corresponding element of VkTimelineSemaphoreSubmitInfo::pWaitSemaphoreValues **must** have a value which does not differ from the current value of the semaphore or from the value of any outstanding semaphore wait or signal operation on that semaphore by more than maxTimelineSemaphoreValueDifference
- VUID-VkBindSparseInfo-pSignalSemaphores-03251
  For each element of pSignalSemaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE the corresponding element of VkTimelineSemaphoreSubmitInfo::pSignalSemaphoreValues **must** have a value which does not differ from the current value of the semaphore or from the value of any outstanding semaphore wait or signal operation on that semaphore by more than maxTimelineSemaphoreValueDifference

Valid Usage (Implicit)

- VUID-VkBindSparseInfo-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_BIND_SPARSE_INFO
- VUID-VkBindSparseInfo-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkDeviceGroupBindSparseInfo,

VkFrameBoundaryEXT, or VkTimelineSemaphoreSubmitInfo

- VUID-VkBindSparseInfo-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkBindSparseInfo-pWaitSemaphores-parameter
  If waitSemaphoreCount is not 0, pWaitSemaphores **must** be a valid pointer to an array of waitSemaphoreCount valid VkSemaphore handles
- VUID-VkBindSparseInfo-pBufferBinds-parameter
  If bufferBindCount is not 0, pBufferBinds **must** be a valid pointer to an array of bufferBindCount valid VkSparseBufferMemoryBindInfo structures
- VUID-VkBindSparseInfo-pImageOpaqueBinds-parameter
  If imageOpaqueBindCount is not 0, pImageOpaqueBinds **must** be a valid pointer to an array of imageOpaqueBindCount valid VkSparseImageOpaqueMemoryBindInfo structures
- VUID-VkBindSparseInfo-pImageBinds-parameter
  If imageBindCount is not 0, pImageBinds **must** be a valid pointer to an array of imageBindCount valid VkSparseImageMemoryBindInfo structures
- VUID-VkBindSparseInfo-pSignalSemaphores-parameter
  If signalSemaphoreCount is not 0, pSignalSemaphores **must** be a valid pointer to an array of signalSemaphoreCount valid VkSemaphore handles
- VUID-VkBindSparseInfo-commonparent
  Both of the elements of pSignalSemaphores, and the elements of pWaitSemaphores that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

To specify the values to use when waiting for and signaling semaphores created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_TIMELINE, add a VkTimelineSemaphoreSubmitInfo structure to the pNext chain of the VkBindSparseInfo structure.

If the pNext chain of VkBindSparseInfo includes a VkDeviceGroupBindSparseInfo structure, then that structure includes device indices specifying which instance of the resources and memory are bound.

The VkDeviceGroupBindSparseInfo structure is defined as:

```
typedef struct VkDeviceGroupBindSparseInfo {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           resourceDeviceIndex;
    uint32_t           memoryDeviceIndex;
} VkDeviceGroupBindSparseInfo;
```

or the equivalent

```
// Provided by VK_KHR_device_group
typedef VkDeviceGroupBindSparseInfo VkDeviceGroupBindSparseInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `resourceDeviceIndex` is a device index indicating which instance of the resource is bound.
- `memoryDeviceIndex` is a device index indicating which instance of the memory the resource instance is bound to.

These device indices apply to all buffer and image memory binds included in the batch pointing to this structure. The semaphore waits and signals for the batch are executed only by the physical device specified by the `resourceDeviceIndex`.

If this structure is not present, `resourceDeviceIndex` and `memoryDeviceIndex` are assumed to be zero.

Valid Usage

- VUID-VkDeviceGroupBindSparseInfo-resourceDeviceIndex-01118
  `resourceDeviceIndex` and `memoryDeviceIndex` **must** both be valid device indices
- VUID-VkDeviceGroupBindSparseInfo-memoryDeviceIndex-01119
  Each memory allocation bound in this batch **must** have allocated an instance for `memoryDeviceIndex`

Valid Usage (Implicit)

- VUID-VkDeviceGroupBindSparseInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_GROUP_BIND_SPARSE_INFO`

Chapter 32. Window System Integration (WSI)

This chapter discusses the window system integration (WSI) between the Vulkan API and the various forms of displaying the results of rendering to a user. Since the Vulkan API **can** be used without displaying results, WSI is provided through the use of optional Vulkan extensions. This chapter provides an overview of WSI. See the appendix for additional details of each WSI extension, including which extensions **must** be enabled in order to use each of the functions described in this chapter.

32.1. WSI Platform

A platform is an abstraction for a window system, OS, etc. Some examples include MS Windows, Android, and Wayland. The Vulkan API **may** be integrated in a unique manner for each platform.

The Vulkan API does not define any type of platform object. Platform-specific WSI extensions are defined, each containing platform-specific functions for using WSI. Use of these extensions is guarded by preprocessor symbols as defined in the Window System-Specific Header Control appendix.

In order for an application to be compiled to use WSI with a given platform, it **must** either:

- `#define` the appropriate preprocessor symbol prior to including the `vulkan.h` header file, or
- include `vulkan_core.h` and any native platform headers, followed by the appropriate platform-specific header.

The preprocessor symbols and platform-specific headers are defined in the Window System Extensions and Headers table.

Each platform-specific extension is an instance extension. The application **must** enable instance extensions with `vkCreateInstance` before using them.

32.2. WSI Surface

Native platform surface or window objects are abstracted by surface objects, which are represented by `VkSurfaceKHR` handles:

```
// Provided by VK_KHR_surface
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkSurfaceKHR)
```

The `VK_KHR_surface` extension declares the `VkSurfaceKHR` object, and provides a function for destroying `VkSurfaceKHR` objects. Separate platform-specific extensions each provide a function for creating a `VkSurfaceKHR` object for the respective platform. From the application's perspective this is an opaque handle, just like the handles of other Vulkan objects.

NOTE On certain platforms, the Vulkan loader and ICDs **may** have conventions that treat

the handle as a pointer to a structure containing the platform-specific information about the surface. This will be described in the documentation for the loader-ICD interface, and in the `vk_icd.h` header file of the LoaderAndTools source-code repository. This does not affect the loader-layer interface; layers **may** wrap `VkSurfaceKHR` objects.

32.2.1. Android Platform

To create a `VkSurfaceKHR` object for an Android native window, call:

```
// Provided by VK_KHR_android_surface
VkResult vkCreateAndroidSurfaceKHR(
    VkInstance                                  instance,
    const VkAndroidSurfaceCreateInfoKHR*        pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSurfaceKHR*                               pSurface);
```

- `instance` is the instance to associate the surface with.
- `pCreateInfo` is a pointer to a `VkAndroidSurfaceCreateInfoKHR` structure containing parameters affecting the creation of the surface object.
- `pAllocator` is the allocator used for host memory allocated for the surface object when there is no more specific allocator available (see Memory Allocation).
- `pSurface` is a pointer to a VkSurfaceKHR handle in which the created surface object is returned.

During the lifetime of a surface created using a particular ANativeWindow handle any attempts to create another surface for the same ANativeWindow and any attempts to connect to the same ANativeWindow through other platform mechanisms will fail.

NOTE In particular, only one `VkSurfaceKHR` **can** exist at a time for a given window. Similarly, a native window **cannot** be used by both a `VkSurfaceKHR` and `EGLSurface` simultaneously.

If successful, `vkCreateAndroidSurfaceKHR` increments the ANativeWindow's reference count, and `vkDestroySurfaceKHR` will decrement it.

On Android, when a swapchain's `imageExtent` does not match the surface's `currentExtent`, the presentable images will be scaled to the surface's dimensions during presentation. `minImageExtent` is (1,1), and `maxImageExtent` is the maximum image size supported by the consumer. For the system compositor, `currentExtent` is the window size (i.e. the consumer's preferred size).

Valid Usage (Implicit)

- VUID-vkCreateAndroidSurfaceKHR-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkCreateAndroidSurfaceKHR-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkAndroidSurfaceCreateInfoKHR structure

- VUID-vkCreateAndroidSurfaceKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateAndroidSurfaceKHR-pSurface-parameter
  `pSurface` **must** be a valid pointer to a VkSurfaceKHR handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_NATIVE_WINDOW_IN_USE_KHR`

The `VkAndroidSurfaceCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_android_surface
typedef struct VkAndroidSurfaceCreateInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkAndroidSurfaceCreateFlagsKHR  flags;
    struct ANativeWindow*           window;
} VkAndroidSurfaceCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `window` is a pointer to the ANativeWindow to associate the surface with.

Valid Usage

- VUID-VkAndroidSurfaceCreateInfoKHR-window-01248
  `window` **must** point to a valid Android ANativeWindow

Valid Usage (Implicit)

- VUID-VkAndroidSurfaceCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ANDROID_SURFACE_CREATE_INFO_KHR`
- VUID-VkAndroidSurfaceCreateInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`

- VUID-VkAndroidSurfaceCreateInfoKHR-flags-zerobitmask
  `flags` **must** be `0`

To remove an unnecessary compile time dependency, an incomplete type definition of ANativeWindow is provided in the Vulkan headers:

```
// Provided by VK_KHR_android_surface
struct ANativeWindow;
```

The actual ANativeWindow type is defined in Android NDK headers.

```
// Provided by VK_KHR_android_surface
typedef VkFlags VkAndroidSurfaceCreateFlagsKHR;
```

`VkAndroidSurfaceCreateFlagsKHR` is a bitmask type for setting a mask, but is currently reserved for future use.

32.2.2. Wayland Platform

To create a `VkSurfaceKHR` object for a Wayland surface, call:

```
// Provided by VK_KHR_wayland_surface
VkResult vkCreateWaylandSurfaceKHR(
    VkInstance                                  instance,
    const VkWaylandSurfaceCreateInfoKHR*        pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSurfaceKHR*                               pSurface);
```

- `instance` is the instance to associate the surface with.
- `pCreateInfo` is a pointer to a VkWaylandSurfaceCreateInfoKHR structure containing parameters affecting the creation of the surface object.
- `pAllocator` is the allocator used for host memory allocated for the surface object when there is no more specific allocator available (see Memory Allocation).
- `pSurface` is a pointer to a VkSurfaceKHR handle in which the created surface object is returned.

Valid Usage (Implicit)

- VUID-vkCreateWaylandSurfaceKHR-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkCreateWaylandSurfaceKHR-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkWaylandSurfaceCreateInfoKHR structure
- VUID-vkCreateWaylandSurfaceKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid

VkAllocationCallbacks structure

- VUID-vkCreateWaylandSurfaceKHR-pSurface-parameter
  `pSurface` **must** be a valid pointer to a VkSurfaceKHR handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkWaylandSurfaceCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_wayland_surface
typedef struct VkWaylandSurfaceCreateInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkWaylandSurfaceCreateFlagsKHR  flags;
    struct wl_display*              display;
    struct wl_surface*              surface;
} VkWaylandSurfaceCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `display` and `surface` are pointers to the Wayland `wl_display` and `wl_surface` to associate the surface with.

Valid Usage

- VUID-VkWaylandSurfaceCreateInfoKHR-display-01304
  `display` **must** point to a valid Wayland `wl_display`
- VUID-VkWaylandSurfaceCreateInfoKHR-surface-01305
  `surface` **must** point to a valid Wayland `wl_surface`

Valid Usage (Implicit)

- VUID-VkWaylandSurfaceCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_WAYLAND_SURFACE_CREATE_INFO_KHR`
- VUID-VkWaylandSurfaceCreateInfoKHR-pNext-pNext

pNext **must** be NULL

- VUID-VkWaylandSurfaceCreateInfoKHR-flags-zerobitmask
  flags **must** be 0

On Wayland, currentExtent is the special value (0xFFFFFFFF, 0xFFFFFFFF), indicating that the surface size will be determined by the extent of a swapchain targeting the surface. Whatever the application sets a swapchain's imageExtent to will be the size of the window, after the first image is presented. minImageExtent is (1,1), and maxImageExtent is the maximum supported surface size. Any calls to vkGetPhysicalDeviceSurfacePresentModesKHR on a surface created with vkCreateWaylandSurfaceKHR are **required** to return VK_PRESENT_MODE_MAILBOX_KHR as one of the valid present modes.

Some Vulkan functions **may** send protocol over the specified wl_display connection when using a swapchain or presentable images created from a VkSurfaceKHR referring to a wl_surface. Applications **must** therefore ensure that both the wl_display and the wl_surface remain valid for the lifetime of any VkSwapchainKHR objects created from a particular wl_display and wl_surface. Also, calling vkQueuePresentKHR will result in Vulkan sending wl_surface.commit requests to the underlying wl_surface of each The wl_surface.attach, wl_surface.damage, and wl_surface.commit requests **must** be issued by the implementation during the call to vkQueuePresentKHR and **must** not be issued by the implementation outside of vkQueuePresentKHR. This ensures that any Wayland requests sent by the client after the call to vkQueuePresentKHR returns will be received by the compositor after the wl_surface.commit. Regardless of the mode of swapchain creation, a new wl_event_queue **must** be created for each successful vkCreateWaylandSurfaceKHR call, and every Wayland object created by the implementation **must** be assigned to this event queue. If the platform provides Wayland 1.11 or greater, this **must** be implemented by the use of Wayland proxy object wrappers, to avoid race conditions.

If the application wishes to synchronize any window changes with a particular frame, such requests **must** be sent to the Wayland display server prior to calling vkQueuePresentKHR.

```
// Provided by VK_KHR_wayland_surface
typedef VkFlags VkWaylandSurfaceCreateFlagsKHR;
```

VkWaylandSurfaceCreateFlagsKHR is a bitmask type for setting a mask, but is currently reserved for future use.

32.2.3. Win32 Platform

To create a VkSurfaceKHR object for a Win32 window, call:

```
// Provided by VK_KHR_win32_surface
VkResult vkCreateWin32SurfaceKHR(
    VkInstance                                  instance,
    const VkWin32SurfaceCreateInfoKHR*          pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSurfaceKHR*                               pSurface);
```

- `instance` is the instance to associate the surface with.
- `pCreateInfo` is a pointer to a `VkWin32SurfaceCreateInfoKHR` structure containing parameters affecting the creation of the surface object.
- `pAllocator` is the allocator used for host memory allocated for the surface object when there is no more specific allocator available (see Memory Allocation).
- `pSurface` is a pointer to a VkSurfaceKHR handle in which the created surface object is returned.

Valid Usage (Implicit)

- VUID-vkCreateWin32SurfaceKHR-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkCreateWin32SurfaceKHR-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkWin32SurfaceCreateInfoKHR structure
- VUID-vkCreateWin32SurfaceKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateWin32SurfaceKHR-pSurface-parameter
  `pSurface` **must** be a valid pointer to a VkSurfaceKHR handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Some Vulkan functions **may** call the `SendMessage` system API when interacting with a `VkSurfaceKHR` through a `VkSwapchainKHR`. In a multithreaded environment, calling `SendMessage` from a thread that is not the thread associated with `pCreateInfo->hwnd` will block until the application has processed the window message. Thus, applications **should** either call these Vulkan functions on the message pump thread, or make sure their message pump is actively running. Failing to do so **may** result in deadlocks.

The functions subject to this requirement are:

- vkCreateSwapchainKHR
- vkDestroySwapchainKHR
- vkAcquireNextImageKHR and vkAcquireNextImage2KHR
- vkQueuePresentKHR
- vkAcquireFullScreenExclusiveModeEXT

- vkReleaseFullScreenExclusiveModeEXT
- vkSetHdrMetadataEXT

The VkWin32SurfaceCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_win32_surface
typedef struct VkWin32SurfaceCreateInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkWin32SurfaceCreateFlagsKHR    flags;
    HINSTANCE                       hinstance;
    HWND                            hwnd;
} VkWin32SurfaceCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- hinstance is the Win32 HINSTANCE for the window to associate the surface with.
- hwnd is the Win32 HWND for the window to associate the surface with.

Valid Usage

- VUID-VkWin32SurfaceCreateInfoKHR-hinstance-01307
  hinstance **must** be a valid Win32 HINSTANCE
- VUID-VkWin32SurfaceCreateInfoKHR-hwnd-01308
  hwnd **must** be a valid Win32 HWND

Valid Usage (Implicit)

- VUID-VkWin32SurfaceCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_WIN32_SURFACE_CREATE_INFO_KHR
- VUID-VkWin32SurfaceCreateInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkWin32SurfaceCreateInfoKHR-flags-zerobitmask
  flags **must** be 0

With Win32, minImageExtent, maxImageExtent, and currentExtent **must** always equal the window size.

The currentExtent of a Win32 surface **must** have both width and height greater than 0, or both of them 0.

NOTE Due to above restrictions, it is only possible to create a new swapchain on this platform with imageExtent being equal to the current size of the window, as

reported in VkSurfaceCapabilitiesKHR::currentExtent.

The window size **may** become (0, 0) on this platform (e.g. when the window is minimized), and so a swapchain **cannot** be created until the size changes.

```
// Provided by VK_KHR_win32_surface
typedef VkFlags VkWin32SurfaceCreateFlagsKHR;
```

VkWin32SurfaceCreateFlagsKHR is a bitmask type for setting a mask, but is currently reserved for future use.

32.2.4. XCB Platform

To create a VkSurfaceKHR object for an X11 window, using the XCB client-side library, call:

```
// Provided by VK_KHR_xcb_surface
VkResult vkCreateXcbSurfaceKHR(
    VkInstance                                  instance,
    const VkXcbSurfaceCreateInfoKHR*            pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSurfaceKHR*                               pSurface);
```

- instance is the instance to associate the surface with.
- pCreateInfo is a pointer to a VkXcbSurfaceCreateInfoKHR structure containing parameters affecting the creation of the surface object.
- pAllocator is the allocator used for host memory allocated for the surface object when there is no more specific allocator available (see Memory Allocation).
- pSurface is a pointer to a VkSurfaceKHR handle in which the created surface object is returned.

Valid Usage (Implicit)

- VUID-vkCreateXcbSurfaceKHR-instance-parameter
  instance **must** be a valid VkInstance handle
- VUID-vkCreateXcbSurfaceKHR-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkXcbSurfaceCreateInfoKHR structure
- VUID-vkCreateXcbSurfaceKHR-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateXcbSurfaceKHR-pSurface-parameter
  pSurface **must** be a valid pointer to a VkSurfaceKHR handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkXcbSurfaceCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_xcb_surface
typedef struct VkXcbSurfaceCreateInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkXcbSurfaceCreateFlagsKHR      flags;
    xcb_connection_t*               connection;
    xcb_window_t                    window;
} VkXcbSurfaceCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- connection is a pointer to an xcb_connection_t to the X server.
- window is the xcb_window_t for the X11 window to associate the surface with.

Valid Usage

- VUID-VkXcbSurfaceCreateInfoKHR-connection-01310
  connection **must** point to a valid X11 xcb_connection_t
- VUID-VkXcbSurfaceCreateInfoKHR-window-01311
  window **must** be a valid X11 xcb_window_t

Valid Usage (Implicit)

- VUID-VkXcbSurfaceCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_XCB_SURFACE_CREATE_INFO_KHR
- VUID-VkXcbSurfaceCreateInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkXcbSurfaceCreateInfoKHR-flags-zerobitmask
  flags **must** be 0

With Xcb, `minImageExtent`, `maxImageExtent`, and `currentExtent` **must** always equal the window size.

The `currentExtent` of an Xcb surface **must** have both `width` and `height` greater than 0, or both of them 0.

NOTE

Due to above restrictions, it is only possible to create a new swapchain on this platform with `imageExtent` being equal to the current size of the window, as reported in VkSurfaceCapabilitiesKHR::`currentExtent`.

The window size **may** become (0, 0) on this platform (e.g. when the window is minimized), and so a swapchain **cannot** be created until the size changes.

Some Vulkan functions **may** send protocol over the specified xcb connection when using a swapchain or presentable images created from a VkSurfaceKHR referring to an xcb window. Applications **must** therefore ensure the xcb connection is available to Vulkan for the duration of any functions that manipulate such swapchains or their presentable images, and any functions that build or queue command buffers that operate on such presentable images. Specifically, applications using Vulkan with xcb-based swapchains **must**

- Avoid holding a server grab on an xcb connection while waiting for Vulkan operations to complete using a swapchain derived from a different xcb connection referring to the same X server instance. Failing to do so **may** result in deadlock.

```
// Provided by VK_KHR_xcb_surface
typedef VkFlags VkXcbSurfaceCreateFlagsKHR;
```

`VkXcbSurfaceCreateFlagsKHR` is a bitmask type for setting a mask, but is currently reserved for future use.

32.2.5. Xlib Platform

To create a `VkSurfaceKHR` object for an X11 window, using the Xlib client-side library, call:

```
// Provided by VK_KHR_xlib_surface
VkResult vkCreateXlibSurfaceKHR(
    VkInstance                                  instance,
    const VkXlibSurfaceCreateInfoKHR*           pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSurfaceKHR*                               pSurface);
```

- `instance` is the instance to associate the surface with.
- `pCreateInfo` is a pointer to a `VkXlibSurfaceCreateInfoKHR` structure containing the parameters affecting the creation of the surface object.
- `pAllocator` is the allocator used for host memory allocated for the surface object when there is no more specific allocator available (see Memory Allocation).
- `pSurface` is a pointer to a VkSurfaceKHR handle in which the created surface object is returned.

Valid Usage (Implicit)

- VUID-vkCreateXlibSurfaceKHR-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkCreateXlibSurfaceKHR-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkXlibSurfaceCreateInfoKHR structure
- VUID-vkCreateXlibSurfaceKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateXlibSurfaceKHR-pSurface-parameter
  `pSurface` **must** be a valid pointer to a VkSurfaceKHR handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkXlibSurfaceCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_xlib_surface
typedef struct VkXlibSurfaceCreateInfoKHR {
    VkStructureType                sType;
    const void*                    pNext;
    VkXlibSurfaceCreateFlagsKHR    flags;
    Display*                       dpy;
    Window                         window;
} VkXlibSurfaceCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `dpy` is a pointer to an Xlib `Display` connection to the X server.
- `window` is an Xlib `Window` to associate the surface with.

Valid Usage

- VUID-VkXlibSurfaceCreateInfoKHR-dpy-01313
  `dpy` **must** point to a valid Xlib `Display`

- VUID-VkXlibSurfaceCreateInfoKHR-window-01314
  `window` **must** be a valid Xlib `Window`

Valid Usage (Implicit)

- VUID-VkXlibSurfaceCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_XLIB_SURFACE_CREATE_INFO_KHR`
- VUID-VkXlibSurfaceCreateInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkXlibSurfaceCreateInfoKHR-flags-zerobitmask
  `flags` **must** be `0`

With Xlib, `minImageExtent`, `maxImageExtent`, and `currentExtent` **must** always equal the window size.

The `currentExtent` of an Xlib surface **must** have both `width` and `height` greater than 0, or both of them 0.

NOTE

Due to above restrictions, it is only possible to create a new swapchain on this platform with `imageExtent` being equal to the current size of the window, as reported in VkSurfaceCapabilitiesKHR::`currentExtent`.

The window size **may** become (0, 0) on this platform (e.g. when the window is minimized), and so a swapchain **cannot** be created until the size changes.

Some Vulkan functions **may** send protocol over the specified Xlib `Display` connection when using a swapchain or presentable images created from a VkSurfaceKHR referring to an Xlib window. Applications **must** therefore ensure the display connection is available to Vulkan for the duration of any functions that manipulate such swapchains or their presentable images, and any functions that build or queue command buffers that operate on such presentable images. Specifically, applications using Vulkan with Xlib-based swapchains **must**

- Avoid holding a server grab on a display connection while waiting for Vulkan operations to complete using a swapchain derived from a different display connection referring to the same X server instance. Failing to do so **may** result in deadlock.

Some implementations may require threads to implement some presentation modes so applications **must** call `XInitThreads`() before calling any other Xlib functions.

```
// Provided by VK_KHR_xlib_surface
typedef VkFlags VkXlibSurfaceCreateFlagsKHR;
```

`VkXlibSurfaceCreateFlagsKHR` is a bitmask type for setting a mask, but is currently reserved for future use.

32.2.6. Platform-Independent Information

Once created, VkSurfaceKHR objects **can** be used in this and other extensions, in particular the VK_KHR_swapchain extension.

Several WSI functions return VK_ERROR_SURFACE_LOST_KHR if the surface becomes no longer available. After such an error, the surface (and any child swapchain, if one exists) **should** be destroyed, as there is no way to restore them to a not-lost state. Applications **may** attempt to create a new VkSurfaceKHR using the same native platform window object, but whether such re-creation will succeed is platform-dependent and **may** depend on the reason the surface became unavailable. A lost surface does not otherwise cause devices to be lost.

To destroy a VkSurfaceKHR object, call:

```
// Provided by VK_KHR_surface
void vkDestroySurfaceKHR(
    VkInstance                                  instance,
    VkSurfaceKHR                                surface,
    const VkAllocationCallbacks*                pAllocator);
```

- instance is the instance used to create the surface.
- surface is the surface to destroy.
- pAllocator is the allocator used for host memory allocated for the surface object when there is no more specific allocator available (see Memory Allocation).

Destroying a VkSurfaceKHR merely severs the connection between Vulkan and the native surface, and does not imply destroying the native surface, closing a window, or similar behavior.

Valid Usage

- VUID-vkDestroySurfaceKHR-surface-01266
  All VkSwapchainKHR objects created for surface **must** have been destroyed prior to destroying surface
- VUID-vkDestroySurfaceKHR-surface-01267
  If VkAllocationCallbacks were provided when surface was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroySurfaceKHR-surface-01268
  If no VkAllocationCallbacks were provided when surface was created, pAllocator **must** be NULL

Valid Usage (Implicit)

- VUID-vkDestroySurfaceKHR-instance-parameter
  instance **must** be a valid VkInstance handle
- VUID-vkDestroySurfaceKHR-surface-parameter

If `surface` is not VK_NULL_HANDLE, `surface` **must** be a valid VkSurfaceKHR handle

- VUID-vkDestroySurfaceKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroySurfaceKHR-surface-parent
  If `surface` is a valid handle, it **must** have been created, allocated, or retrieved from `instance`

Host Synchronization

- Host access to `surface` **must** be externally synchronized

32.3. Presenting Directly to Display Devices

In some environments applications **can** also present Vulkan rendering directly to display devices without using an intermediate windowing system. This **can** be useful for embedded applications, or implementing the rendering/presentation backend of a windowing system using Vulkan. The `VK_KHR_display` extension provides the functionality necessary to enumerate display devices and create `VkSurfaceKHR` objects that target displays.

32.3.1. Display Enumeration

Displays are represented by `VkDisplayKHR` handles:

```
// Provided by VK_KHR_display
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDisplayKHR)
```

Various functions are provided for enumerating the available display devices present on a Vulkan physical device. To query information about the available displays, call:

```
// Provided by VK_KHR_display
VkResult vkGetPhysicalDeviceDisplayPropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pPropertyCount,
    VkDisplayPropertiesKHR*                     pProperties);
```

- `physicalDevice` is a physical device.
- `pPropertyCount` is a pointer to an integer related to the number of display devices available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of VkDisplayPropertiesKHR structures.

If `pProperties` is `NULL`, then the number of display devices available for `physicalDevice` is returned in `pPropertyCount`. Otherwise, `pPropertyCount` **must** point to a variable set by the application to the

number of elements in the pProperties array, and on return the variable is overwritten with the number of structures actually written to pProperties. If the value of pPropertyCount is less than the number of display devices for physicalDevice, at most pPropertyCount structures will be written, and VK_INCOMPLETE will be returned instead of VK_SUCCESS, to indicate that not all the available properties were returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceDisplayPropertiesKHR-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceDisplayPropertiesKHR-pPropertyCount-parameter
  pPropertyCount **must** be a valid pointer to a uint32_t value
- VUID-vkGetPhysicalDeviceDisplayPropertiesKHR-pProperties-parameter
  If the value referenced by pPropertyCount is not 0, and pProperties is not NULL, pProperties **must** be a valid pointer to an array of pPropertyCount VkDisplayPropertiesKHR structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkDisplayPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_display
typedef struct VkDisplayPropertiesKHR {
    VkDisplayKHR                  display;
    const char*                   displayName;
    VkExtent2D                    physicalDimensions;
    VkExtent2D                    physicalResolution;
    VkSurfaceTransformFlagsKHR    supportedTransforms;
    VkBool32                      planeReorderPossible;
    VkBool32                      persistentContent;
} VkDisplayPropertiesKHR;
```

- display is a handle that is used to refer to the display described here. This handle will be valid for the lifetime of the Vulkan instance.
- displayName is NULL or a pointer to a null-terminated UTF-8 string containing the name of the display. Generally, this will be the name provided by the display's EDID. If NULL, no suitable name is available. If not NULL, the string pointed to **must** remain accessible and unmodified as

long as `display` is valid.

- `physicalDimensions` describes the physical width and height of the visible portion of the display, in millimeters.
- `physicalResolution` describes the physical, native, or preferred resolution of the display.

NOTE	For devices which have no natural value to return here, implementations **should** return the maximum resolution supported.

- `supportedTransforms` is a bitmask of VkSurfaceTransformFlagBitsKHR describing which transforms are supported by this display.
- `planeReorderPossible` tells whether the planes on this display **can** have their z order changed. If this is `VK_TRUE`, the application **can** re-arrange the planes on this display in any order relative to each other.
- `persistentContent` tells whether the display supports self-refresh/internal buffering. If this is true, the application **can** submit persistent present operations on swapchains created against this display.

NOTE	Persistent presents **may** have higher latency, and **may** use less power when the screen content is updated infrequently, or when only a portion of the screen needs to be updated in most frames.

To query information about the available displays, call:

```
// Provided by VK_KHR_get_display_properties2
VkResult vkGetPhysicalDeviceDisplayProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pPropertyCount,
    VkDisplayProperties2KHR*                    pProperties);
```

- `physicalDevice` is a physical device.
- `pPropertyCount` is a pointer to an integer related to the number of display devices available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of `VkDisplayProperties2KHR` structures.

`vkGetPhysicalDeviceDisplayProperties2KHR` behaves similarly to vkGetPhysicalDeviceDisplayPropertiesKHR, with the ability to return extended information via chained output structures.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceDisplayProperties2KHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceDisplayProperties2KHR-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value

- VUID-vkGetPhysicalDeviceDisplayProperties2KHR-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkDisplayProperties2KHR structures

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkDisplayProperties2KHR` structure is defined as:

```
// Provided by VK_KHR_get_display_properties2
typedef struct VkDisplayProperties2KHR {
    VkStructureType           sType;
    void*                     pNext;
    VkDisplayPropertiesKHR    displayProperties;
} VkDisplayProperties2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `displayProperties` is a VkDisplayPropertiesKHR structure.

Valid Usage (Implicit)

- VUID-VkDisplayProperties2KHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_PROPERTIES_2_KHR`
- VUID-VkDisplayProperties2KHR-pNext-pNext
  `pNext` **must** be `NULL`

Acquiring and Releasing Displays

On some platforms, access to displays is limited to a single process or native driver instance. On such platforms, some or all of the displays may not be available to Vulkan if they are already in use by a native windowing system or other application.

To release a previously acquired display, call:

```
// Provided by VK_EXT_direct_mode_display
VkResult vkReleaseDisplayEXT(
    VkPhysicalDevice                            physicalDevice,
    VkDisplayKHR                                display);
```

- `physicalDevice` The physical device the display is on.
- `display` The display to release control of.

Valid Usage (Implicit)

- VUID-vkReleaseDisplayEXT-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkReleaseDisplayEXT-display-parameter
  `display` **must** be a valid VkDisplayKHR handle
- VUID-vkReleaseDisplayEXT-display-parent
  `display` **must** have been created, allocated, or retrieved from `physicalDevice`

Return Codes

Success

- `VK_SUCCESS`

Failure

None

Display Planes

Images are presented to individual planes on a display. Devices **must** support at least one plane on each display. Planes **can** be stacked and blended to composite multiple images on one display. Devices **may** support only a fixed stacking order and fixed mapping between planes and displays, or they **may** allow arbitrary application-specified stacking orders and mappings between planes and displays. To query the properties of device display planes, call:

```
// Provided by VK_KHR_display
VkResult vkGetPhysicalDeviceDisplayPlanePropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pPropertyCount,
    VkDisplayPlanePropertiesKHR*                pProperties);
```

- `physicalDevice` is a physical device.
- `pPropertyCount` is a pointer to an integer related to the number of display planes available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of `VkDisplayPlanePropertiesKHR` structures.

If pProperties is NULL, then the number of display planes available for physicalDevice is returned in pPropertyCount. Otherwise, pPropertyCount **must** point to a variable set by the application to the number of elements in the pProperties array, and on return the variable is overwritten with the number of structures actually written to pProperties. If the value of pPropertyCount is less than the number of display planes for physicalDevice, at most pPropertyCount structures will be written.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceDisplayPlanePropertiesKHR-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceDisplayPlanePropertiesKHR-pPropertyCount-parameter
  pPropertyCount **must** be a valid pointer to a uint32_t value
- VUID-vkGetPhysicalDeviceDisplayPlanePropertiesKHR-pProperties-parameter
  If the value referenced by pPropertyCount is not 0, and pProperties is not NULL, pProperties **must** be a valid pointer to an array of pPropertyCount VkDisplayPlanePropertiesKHR structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkDisplayPlanePropertiesKHR structure is defined as:

```
// Provided by VK_KHR_display
typedef struct VkDisplayPlanePropertiesKHR {
    VkDisplayKHR    currentDisplay;
    uint32_t        currentStackIndex;
} VkDisplayPlanePropertiesKHR;
```

- currentDisplay is the handle of the display the plane is currently associated with. If the plane is not currently attached to any displays, this will be VK_NULL_HANDLE.
- currentStackIndex is the current z-order of the plane. This will be between 0 and the value returned by vkGetPhysicalDeviceDisplayPlanePropertiesKHR in pPropertyCount.

To query the properties of a device's display planes, call:

```
// Provided by VK_KHR_get_display_properties2
VkResult vkGetPhysicalDeviceDisplayPlaneProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pPropertyCount,
    VkDisplayPlaneProperties2KHR*               pProperties);
```

- `physicalDevice` is a physical device.
- `pPropertyCount` is a pointer to an integer related to the number of display planes available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of `VkDisplayPlaneProperties2KHR` structures.

`vkGetPhysicalDeviceDisplayPlaneProperties2KHR` behaves similarly to vkGetPhysicalDeviceDisplayPlanePropertiesKHR, with the ability to return extended information via chained output structures.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceDisplayPlaneProperties2KHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceDisplayPlaneProperties2KHR-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceDisplayPlaneProperties2KHR-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkDisplayPlaneProperties2KHR structures

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkDisplayPlaneProperties2KHR` structure is defined as:

```
// Provided by VK_KHR_get_display_properties2
typedef struct VkDisplayPlaneProperties2KHR {
    VkStructureType                 sType;
    void*                           pNext;
    VkDisplayPlanePropertiesKHR     displayPlaneProperties;
} VkDisplayPlaneProperties2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `displayPlaneProperties` is a VkDisplayPlanePropertiesKHR structure.

Valid Usage (Implicit)

- VUID-VkDisplayPlaneProperties2KHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_PLANE_PROPERTIES_2_KHR`
- VUID-VkDisplayPlaneProperties2KHR-pNext-pNext
  `pNext` **must** be `NULL`

To determine which displays a plane is usable with, call

```
// Provided by VK_KHR_display
VkResult vkGetDisplayPlaneSupportedDisplaysKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t                                    planeIndex,
    uint32_t*                                   pDisplayCount,
    VkDisplayKHR*                               pDisplays);
```

- `physicalDevice` is a physical device.
- `planeIndex` is the plane which the application wishes to use, and **must** be in the range [0, physical device plane count - 1].
- `pDisplayCount` is a pointer to an integer related to the number of displays available or queried, as described below.
- `pDisplays` is either `NULL` or a pointer to an array of `VkDisplayKHR` handles.

If `pDisplays` is `NULL`, then the number of displays usable with the specified `planeIndex` for `physicalDevice` is returned in `pDisplayCount`. Otherwise, `pDisplayCount` **must** point to a variable set by the application to the number of elements in the `pDisplays` array, and on return the variable is overwritten with the number of handles actually written to `pDisplays`. If the value of `pDisplayCount` is less than the number of usable display-plane pairs for `physicalDevice`, at most `pDisplayCount` handles will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available pairs were returned.

Valid Usage

- VUID-vkGetDisplayPlaneSupportedDisplaysKHR-planeIndex-01249
  `planeIndex` **must** be less than the number of display planes supported by the device as determined by calling `vkGetPhysicalDeviceDisplayPlanePropertiesKHR`

Valid Usage (Implicit)

- VUID-vkGetDisplayPlaneSupportedDisplaysKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetDisplayPlaneSupportedDisplaysKHR-pDisplayCount-parameter
  `pDisplayCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetDisplayPlaneSupportedDisplaysKHR-pDisplays-parameter
  If the value referenced by `pDisplayCount` is not `0`, and `pDisplays` is not `NULL`, `pDisplays` **must** be a valid pointer to an array of `pDisplayCount` VkDisplayKHR handles

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Additional properties of displays are queried using specialized query functions.

Display Modes

Display modes are represented by `VkDisplayModeKHR` handles:

```
// Provided by VK_KHR_display
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDisplayModeKHR)
```

Each display has one or more supported modes associated with it by default. These built-in modes are queried by calling:

```
// Provided by VK_KHR_display
VkResult vkGetDisplayModePropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    VkDisplayKHR                                display,
    uint32_t*                                   pPropertyCount,
    VkDisplayModePropertiesKHR*                 pProperties);
```

- `physicalDevice` is the physical device associated with `display`.
- `display` is the display to query.
- `pPropertyCount` is a pointer to an integer related to the number of display modes available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of `VkDisplayModePropertiesKHR` structures.

If `pProperties` is `NULL`, then the number of display modes available on the specified `display` for `physicalDevice` is returned in `pPropertyCount`. Otherwise, `pPropertyCount` **must** point to a variable set by the application to the number of elements in the `pProperties` array, and on return the variable is overwritten with the number of structures actually written to `pProperties`. If the value of `pPropertyCount` is less than the number of display modes for `physicalDevice`, at most `pPropertyCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available display modes were returned.

Valid Usage (Implicit)

- VUID-vkGetDisplayModePropertiesKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetDisplayModePropertiesKHR-display-parameter
  `display` **must** be a valid VkDisplayKHR handle
- VUID-vkGetDisplayModePropertiesKHR-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetDisplayModePropertiesKHR-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkDisplayModePropertiesKHR structures
- VUID-vkGetDisplayModePropertiesKHR-display-parent
  `display` **must** have been created, allocated, or retrieved from `physicalDevice`

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkDisplayModePropertiesKHR structure is defined as:

```
// Provided by VK_KHR_display
typedef struct VkDisplayModePropertiesKHR {
    VkDisplayModeKHR              displayMode;
    VkDisplayModeParametersKHR    parameters;
} VkDisplayModePropertiesKHR;
```

- displayMode is a handle to the display mode described in this structure. This handle will be valid for the lifetime of the Vulkan instance.
- parameters is a VkDisplayModeParametersKHR structure describing the display parameters associated with displayMode.

```
// Provided by VK_KHR_display
typedef VkFlags VkDisplayModeCreateFlagsKHR;
```

VkDisplayModeCreateFlagsKHR is a bitmask type for setting a mask, but is currently reserved for future use.

To query the properties of a device's built-in display modes, call:

```
// Provided by VK_KHR_get_display_properties2
VkResult vkGetDisplayModeProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    VkDisplayKHR                                display,
    uint32_t*                                   pPropertyCount,
    VkDisplayModeProperties2KHR*                pProperties);
```

- physicalDevice is the physical device associated with display.
- display is the display to query.
- pPropertyCount is a pointer to an integer related to the number of display modes available or queried, as described below.
- pProperties is either NULL or a pointer to an array of VkDisplayModeProperties2KHR structures.

vkGetDisplayModeProperties2KHR behaves similarly to vkGetDisplayModePropertiesKHR, with the ability to return extended information via chained output structures.

Valid Usage (Implicit)

- VUID-vkGetDisplayModeProperties2KHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetDisplayModeProperties2KHR-display-parameter
  `display` **must** be a valid VkDisplayKHR handle
- VUID-vkGetDisplayModeProperties2KHR-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetDisplayModeProperties2KHR-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkDisplayModeProperties2KHR structures
- VUID-vkGetDisplayModeProperties2KHR-display-parent
  `display` **must** have been created, allocated, or retrieved from `physicalDevice`

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkDisplayModeProperties2KHR` structure is defined as:

```
// Provided by VK_KHR_get_display_properties2
typedef struct VkDisplayModeProperties2KHR {
    VkStructureType               sType;
    void*                         pNext;
    VkDisplayModePropertiesKHR    displayModeProperties;
} VkDisplayModeProperties2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `displayModeProperties` is a VkDisplayModePropertiesKHR structure.

Valid Usage (Implicit)

- VUID-VkDisplayModeProperties2KHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_MODE_PROPERTIES_2_KHR`

- VUID-VkDisplayModeProperties2KHR-pNext-pNext
  `pNext` **must** be `NULL`

The `VkDisplayModeParametersKHR` structure is defined as:

```
// Provided by VK_KHR_display
typedef struct VkDisplayModeParametersKHR {
    VkExtent2D    visibleRegion;
    uint32_t      refreshRate;
} VkDisplayModeParametersKHR;
```

- `visibleRegion` is the 2D extents of the visible region.
- `refreshRate` is a `uint32_t` that is the number of times the display is refreshed each second multiplied by 1000.

NOTE For example, a 60Hz display mode would report a `refreshRate` of 60,000.

Valid Usage

- VUID-VkDisplayModeParametersKHR-width-01990
  The `width` member of `visibleRegion` **must** be greater than `0`
- VUID-VkDisplayModeParametersKHR-height-01991
  The `height` member of `visibleRegion` **must** be greater than `0`
- VUID-VkDisplayModeParametersKHR-refreshRate-01992
  `refreshRate` **must** be greater than `0`

Additional modes **may** also be created by calling:

```
// Provided by VK_KHR_display
VkResult vkCreateDisplayModeKHR(
    VkPhysicalDevice                            physicalDevice,
    VkDisplayKHR                                display,
    const VkDisplayModeCreateInfoKHR*           pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkDisplayModeKHR*                           pMode);
```

- `physicalDevice` is the physical device associated with `display`.
- `display` is the display to create an additional mode for.
- `pCreateInfo` is a pointer to a VkDisplayModeCreateInfoKHR structure describing the new mode to create.
- `pAllocator` is the allocator used for host memory allocated for the display mode object when there is no more specific allocator available (see Memory Allocation).

- `pMode` is a pointer to a VkDisplayModeKHR handle in which the mode created is returned.

Valid Usage (Implicit)

- VUID-vkCreateDisplayModeKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkCreateDisplayModeKHR-display-parameter
  `display` **must** be a valid VkDisplayKHR handle
- VUID-vkCreateDisplayModeKHR-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkDisplayModeCreateInfoKHR structure
- VUID-vkCreateDisplayModeKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateDisplayModeKHR-pMode-parameter
  `pMode` **must** be a valid pointer to a VkDisplayModeKHR handle
- VUID-vkCreateDisplayModeKHR-display-parent
  `display` **must** have been created, allocated, or retrieved from `physicalDevice`

Host Synchronization

- Host access to `display` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_INITIALIZATION_FAILED`

The `VkDisplayModeCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_display
typedef struct VkDisplayModeCreateInfoKHR {
    VkStructureType                sType;
    const void*                    pNext;
    VkDisplayModeCreateFlagsKHR    flags;
    VkDisplayModeParametersKHR     parameters;
} VkDisplayModeCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use, and **must** be zero.
- `parameters` is a VkDisplayModeParametersKHR structure describing the display parameters to use in creating the new mode. If the parameters are not compatible with the specified display, the implementation **must** return `VK_ERROR_INITIALIZATION_FAILED`.

Valid Usage (Implicit)

- VUID-VkDisplayModeCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_MODE_CREATE_INFO_KHR`
- VUID-VkDisplayModeCreateInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDisplayModeCreateInfoKHR-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkDisplayModeCreateInfoKHR-parameters-parameter
  `parameters` **must** be a valid VkDisplayModeParametersKHR structure

Applications that wish to present directly to a display **must** select which layer, or "plane" of the display they wish to target, and a mode to use with the display. Each display supports at least one plane. The capabilities of a given mode and plane combination are determined by calling:

```
// Provided by VK_KHR_display
VkResult vkGetDisplayPlaneCapabilitiesKHR(
    VkPhysicalDevice                            physicalDevice,
    VkDisplayModeKHR                            mode,
    uint32_t                                    planeIndex,
    VkDisplayPlaneCapabilitiesKHR*              pCapabilities);
```

- `physicalDevice` is the physical device associated with the display specified by `mode`
- `mode` is the display mode the application intends to program when using the specified plane. Note this parameter also implicitly specifies a display.
- `planeIndex` is the plane which the application intends to use with the display, and is less than the number of display planes supported by the device.
- `pCapabilities` is a pointer to a VkDisplayPlaneCapabilitiesKHR structure in which the capabilities are returned.

Valid Usage (Implicit)

- VUID-vkGetDisplayPlaneCapabilitiesKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetDisplayPlaneCapabilitiesKHR-mode-parameter

`mode` **must** be a valid VkDisplayModeKHR handle

- VUID-vkGetDisplayPlaneCapabilitiesKHR-pCapabilities-parameter
  `pCapabilities` **must** be a valid pointer to a VkDisplayPlaneCapabilitiesKHR structure
- VUID-vkGetDisplayPlaneCapabilitiesKHR-mode-parent
  `mode` **must** have been created, allocated, or retrieved from `physicalDevice`

Host Synchronization

- Host access to `mode` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkDisplayPlaneCapabilitiesKHR` structure is defined as:

```
// Provided by VK_KHR_display
typedef struct VkDisplayPlaneCapabilitiesKHR {
    VkDisplayPlaneAlphaFlagsKHR    supportedAlpha;
    VkOffset2D                     minSrcPosition;
    VkOffset2D                     maxSrcPosition;
    VkExtent2D                     minSrcExtent;
    VkExtent2D                     maxSrcExtent;
    VkOffset2D                     minDstPosition;
    VkOffset2D                     maxDstPosition;
    VkExtent2D                     minDstExtent;
    VkExtent2D                     maxDstExtent;
} VkDisplayPlaneCapabilitiesKHR;
```

- `supportedAlpha` is a bitmask of VkDisplayPlaneAlphaFlagBitsKHR describing the supported alpha blending modes.
- `minSrcPosition` is the minimum source rectangle offset supported by this plane using the specified mode.
- `maxSrcPosition` is the maximum source rectangle offset supported by this plane using the specified mode. The `x` and `y` components of `maxSrcPosition` **must** each be greater than or equal to the `x` and `y` components of `minSrcPosition`, respectively.
- `minSrcExtent` is the minimum source rectangle size supported by this plane using the specified mode.

- `maxSrcExtent` is the maximum source rectangle size supported by this plane using the specified mode.
- `minDstPosition`, `maxDstPosition`, `minDstExtent`, `maxDstExtent` all have similar semantics to their corresponding `*Src*` equivalents, but apply to the output region within the mode rather than the input region within the source image. Unlike the `*Src*` offsets, `minDstPosition` and `maxDstPosition` **may** contain negative values.

The minimum and maximum position and extent fields describe the implementation limits, if any, as they apply to the specified display mode and plane. Vendors **may** support displaying a subset of a swapchain's presentable images on the specified display plane. This is expressed by returning `minSrcPosition`, `maxSrcPosition`, `minSrcExtent`, and `maxSrcExtent` values that indicate a range of possible positions and sizes which **may** be used to specify the region within the presentable images that source pixels will be read from when creating a swapchain on the specified display mode and plane.

Vendors **may** also support mapping the presentable images' content to a subset or superset of the visible region in the specified display mode. This is expressed by returning `minDstPosition`, `maxDstPosition`, `minDstExtent` and `maxDstExtent` values that indicate a range of possible positions and sizes which **may** be used to describe the region within the display mode that the source pixels will be mapped to.

Other vendors **may** support only a 1-1 mapping between pixels in the presentable images and the display mode. This **may** be indicated by returning (0,0) for `minSrcPosition`, `maxSrcPosition`, `minDstPosition`, and `maxDstPosition`, and (display mode width, display mode height) for `minSrcExtent`, `maxSrcExtent`, `minDstExtent`, and `maxDstExtent`.

The value `supportedAlpha` **must** contain at least one valid VkDisplayPlaneAlphaFlagBitsKHR bit.

These values indicate the limits of the implementation's individual fields. Not all combinations of values within the offset and extent ranges returned in `VkDisplayPlaneCapabilitiesKHR` are guaranteed to be supported. Presentation requests specifying unsupported combinations **may** fail.

To query the capabilities of a given mode and plane combination, call:

```
// Provided by VK_KHR_get_display_properties2
VkResult vkGetDisplayPlaneCapabilities2KHR(
    VkPhysicalDevice                            physicalDevice,
    const VkDisplayPlaneInfo2KHR*               pDisplayPlaneInfo,
    VkDisplayPlaneCapabilities2KHR*             pCapabilities);
```

- `physicalDevice` is the physical device associated with `pDisplayPlaneInfo`.
- `pDisplayPlaneInfo` is a pointer to a VkDisplayPlaneInfo2KHR structure describing the plane and mode.
- `pCapabilities` is a pointer to a VkDisplayPlaneCapabilities2KHR structure in which the capabilities are returned.

`vkGetDisplayPlaneCapabilities2KHR` behaves similarly to vkGetDisplayPlaneCapabilitiesKHR, with the ability to specify extended inputs via chained input structures, and to return extended

information via chained output structures.

Valid Usage (Implicit)

- VUID-vkGetDisplayPlaneCapabilities2KHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetDisplayPlaneCapabilities2KHR-pDisplayPlaneInfo-parameter
  `pDisplayPlaneInfo` **must** be a valid pointer to a valid VkDisplayPlaneInfo2KHR structure
- VUID-vkGetDisplayPlaneCapabilities2KHR-pCapabilities-parameter
  `pCapabilities` **must** be a valid pointer to a VkDisplayPlaneCapabilities2KHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkDisplayPlaneInfo2KHR` structure is defined as:

```
// Provided by VK_KHR_get_display_properties2
typedef struct VkDisplayPlaneInfo2KHR {
    VkStructureType     sType;
    const void*         pNext;
    VkDisplayModeKHR    mode;
    uint32_t            planeIndex;
} VkDisplayPlaneInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `mode` is the display mode the application intends to program when using the specified plane.

> **NOTE** This parameter also implicitly specifies a display.

- `planeIndex` is the plane which the application intends to use with the display.

The members of `VkDisplayPlaneInfo2KHR` correspond to the arguments to vkGetDisplayPlaneCapabilitiesKHR, with `sType` and `pNext` added for extensibility.

Valid Usage (Implicit)

- VUID-VkDisplayPlaneInfo2KHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_PLANE_INFO_2_KHR`
- VUID-VkDisplayPlaneInfo2KHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDisplayPlaneInfo2KHR-mode-parameter
  `mode` **must** be a valid VkDisplayModeKHR handle

Host Synchronization

- Host access to `mode` **must** be externally synchronized

The `VkDisplayPlaneCapabilities2KHR` structure is defined as:

```
// Provided by VK_KHR_get_display_properties2
typedef struct VkDisplayPlaneCapabilities2KHR {
    VkStructureType                 sType;
    void*                           pNext;
    VkDisplayPlaneCapabilitiesKHR   capabilities;
} VkDisplayPlaneCapabilities2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `capabilities` is a VkDisplayPlaneCapabilitiesKHR structure.

Valid Usage (Implicit)

- VUID-VkDisplayPlaneCapabilities2KHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_PLANE_CAPABILITIES_2_KHR`
- VUID-VkDisplayPlaneCapabilities2KHR-pNext-pNext
  `pNext` **must** be `NULL`

32.3.2. Display Control

To set the power state of a display, call:

```
// Provided by VK_EXT_display_control
VkResult vkDisplayPowerControlEXT(
    VkDevice                                    device,
    VkDisplayKHR                                display,
    const VkDisplayPowerInfoEXT*                pDisplayPowerInfo);
```

- `device` is a logical device associated with `display`.
- `display` is the display whose power state is modified.
- `pDisplayPowerInfo` is a pointer to a VkDisplayPowerInfoEXT structure specifying the new power state of `display`.

Valid Usage (Implicit)

- VUID-vkDisplayPowerControlEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDisplayPowerControlEXT-display-parameter
  `display` **must** be a valid VkDisplayKHR handle
- VUID-vkDisplayPowerControlEXT-pDisplayPowerInfo-parameter
  `pDisplayPowerInfo` **must** be a valid pointer to a valid VkDisplayPowerInfoEXT structure
- VUID-vkDisplayPowerControlEXT-commonparent
  Both of `device`, and `display` **must** have been created, allocated, or retrieved from the same VkPhysicalDevice

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

The `VkDisplayPowerInfoEXT` structure is defined as:

```
// Provided by VK_EXT_display_control
typedef struct VkDisplayPowerInfoEXT {
    VkStructureType           sType;
    const void*               pNext;
    VkDisplayPowerStateEXT    powerState;
} VkDisplayPowerInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.

- `powerState` is a VkDisplayPowerStateEXT value specifying the new power state of the display.

Valid Usage (Implicit)

- VUID-VkDisplayPowerInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_POWER_INFO_EXT`
- VUID-VkDisplayPowerInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDisplayPowerInfoEXT-powerState-parameter
  `powerState` **must** be a valid VkDisplayPowerStateEXT value

Possible values of VkDisplayPowerInfoEXT::`powerState`, specifying the new power state of a display, are:

```
// Provided by VK_EXT_display_control
typedef enum VkDisplayPowerStateEXT {
    VK_DISPLAY_POWER_STATE_OFF_EXT = 0,
    VK_DISPLAY_POWER_STATE_SUSPEND_EXT = 1,
    VK_DISPLAY_POWER_STATE_ON_EXT = 2,
} VkDisplayPowerStateEXT;
```

- `VK_DISPLAY_POWER_STATE_OFF_EXT` specifies that the display is powered down.
- `VK_DISPLAY_POWER_STATE_SUSPEND_EXT` specifies that the display is put into a low power mode, from which it **may** be able to transition back to `VK_DISPLAY_POWER_STATE_ON_EXT` more quickly than if it were in `VK_DISPLAY_POWER_STATE_OFF_EXT`. This state **may** be the same as `VK_DISPLAY_POWER_STATE_OFF_EXT`.
- `VK_DISPLAY_POWER_STATE_ON_EXT` specifies that the display is powered on.

32.3.3. Display Surfaces

A complete display configuration includes a mode, one or more display planes and any parameters describing their behavior, and parameters describing some aspects of the images associated with those planes. Display surfaces describe the configuration of a single plane within a complete display configuration. To create a `VkSurfaceKHR` object for a display plane, call:

```
// Provided by VK_KHR_display
VkResult vkCreateDisplayPlaneSurfaceKHR(
    VkInstance                                  instance,
    const VkDisplaySurfaceCreateInfoKHR*        pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSurfaceKHR*                               pSurface);
```

- `instance` is the instance corresponding to the physical device the targeted display is on.
- `pCreateInfo` is a pointer to a VkDisplaySurfaceCreateInfoKHR structure specifying which mode,

plane, and other parameters to use, as described below.

- `pAllocator` is the allocator used for host memory allocated for the surface object when there is no more specific allocator available (see Memory Allocation).
- `pSurface` is a pointer to a VkSurfaceKHR handle in which the created surface is returned.

Valid Usage (Implicit)

- VUID-vkCreateDisplayPlaneSurfaceKHR-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkCreateDisplayPlaneSurfaceKHR-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkDisplaySurfaceCreateInfoKHR structure
- VUID-vkCreateDisplayPlaneSurfaceKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateDisplayPlaneSurfaceKHR-pSurface-parameter
  `pSurface` **must** be a valid pointer to a VkSurfaceKHR handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkDisplaySurfaceCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_display
typedef struct VkDisplaySurfaceCreateInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkDisplaySurfaceCreateFlagsKHR  flags;
    VkDisplayModeKHR                displayMode;
    uint32_t                        planeIndex;
    uint32_t                        planeStackIndex;
    VkSurfaceTransformFlagBitsKHR   transform;
    float                           globalAlpha;
    VkDisplayPlaneAlphaFlagBitsKHR  alphaMode;
    VkExtent2D                      imageExtent;
} VkDisplaySurfaceCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use, and **must** be zero.
- `displayMode` is a VkDisplayModeKHR handle specifying the mode to use when displaying this surface.
- `planeIndex` is the plane on which this surface appears.
- `planeStackIndex` is the z-order of the plane.
- `transform` is a VkSurfaceTransformFlagBitsKHR value specifying the transformation to apply to images as part of the scanout operation.
- `globalAlpha` is the global alpha value. This value is ignored if `alphaMode` is not `VK_DISPLAY_PLANE_ALPHA_GLOBAL_BIT_KHR`.
- `alphaMode` is a VkDisplayPlaneAlphaFlagBitsKHR value specifying the type of alpha blending to use.
- `imageExtent` is the size of the presentable images to use with the surface.

NOTE

Creating a display surface **must** not modify the state of the displays, planes, or other resources it names. For example, it **must** not apply the specified mode to be set on the associated display. Application of display configuration occurs as a side effect of presenting to a display surface.

Valid Usage

- VUID-VkDisplaySurfaceCreateInfoKHR-planeIndex-01252
  `planeIndex` **must** be less than the number of display planes supported by the device as determined by calling `vkGetPhysicalDeviceDisplayPlanePropertiesKHR`
- VUID-VkDisplaySurfaceCreateInfoKHR-planeReorderPossible-01253
  If the `planeReorderPossible` member of the `VkDisplayPropertiesKHR` structure returned by `vkGetPhysicalDeviceDisplayPropertiesKHR` for the display corresponding to `displayMode` is `VK_TRUE` then `planeStackIndex` **must** be less than the number of display planes supported by the device as determined by calling `vkGetPhysicalDeviceDisplayPlanePropertiesKHR`; otherwise `planeStackIndex` **must** equal the `currentStackIndex` member of `VkDisplayPlanePropertiesKHR` returned by `vkGetPhysicalDeviceDisplayPlanePropertiesKHR` for the display plane corresponding to `displayMode`
- VUID-VkDisplaySurfaceCreateInfoKHR-alphaMode-01254
  If `alphaMode` is `VK_DISPLAY_PLANE_ALPHA_GLOBAL_BIT_KHR` then `globalAlpha` **must** be between `0` and `1`, inclusive
- VUID-VkDisplaySurfaceCreateInfoKHR-alphaMode-01255
  `alphaMode` **must** be one of the bits present in the `supportedAlpha` member of `VkDisplayPlaneCapabilitiesKHR` for the display plane corresponding to `displayMode`
- VUID-VkDisplaySurfaceCreateInfoKHR-transform-06740
  `transform` **must** be one of the bits present in the `supportedTransforms` member of `VkDisplayPropertiesKHR` for the display corresponding to `displayMode`
- VUID-VkDisplaySurfaceCreateInfoKHR-width-01256

The `width` and `height` members of `imageExtent` **must** be less than or equal to VkPhysicalDeviceLimits::`maxImageDimension2D`

Valid Usage (Implicit)

- VUID-VkDisplaySurfaceCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_SURFACE_CREATE_INFO_KHR`
- VUID-VkDisplaySurfaceCreateInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkDisplaySurfaceCreateInfoKHR-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkDisplaySurfaceCreateInfoKHR-displayMode-parameter
  `displayMode` **must** be a valid VkDisplayModeKHR handle
- VUID-VkDisplaySurfaceCreateInfoKHR-transform-parameter
  `transform` **must** be a valid VkSurfaceTransformFlagBitsKHR value
- VUID-VkDisplaySurfaceCreateInfoKHR-alphaMode-parameter
  `alphaMode` **must** be a valid VkDisplayPlaneAlphaFlagBitsKHR value

```
// Provided by VK_KHR_display
typedef VkFlags VkDisplaySurfaceCreateFlagsKHR;
```

`VkDisplaySurfaceCreateFlagsKHR` is a bitmask type for setting a mask, but is currently reserved for future use.

Bits which **can** be set in VkDisplaySurfaceCreateInfoKHR::`alphaMode`, specifying the type of alpha blending to use on a display, are:

```
// Provided by VK_KHR_display
typedef enum VkDisplayPlaneAlphaFlagBitsKHR {
    VK_DISPLAY_PLANE_ALPHA_OPAQUE_BIT_KHR = 0x00000001,
    VK_DISPLAY_PLANE_ALPHA_GLOBAL_BIT_KHR = 0x00000002,
    VK_DISPLAY_PLANE_ALPHA_PER_PIXEL_BIT_KHR = 0x00000004,
    VK_DISPLAY_PLANE_ALPHA_PER_PIXEL_PREMULTIPLIED_BIT_KHR = 0x00000008,
} VkDisplayPlaneAlphaFlagBitsKHR;
```

- `VK_DISPLAY_PLANE_ALPHA_OPAQUE_BIT_KHR` specifies that the source image will be treated as opaque.
- `VK_DISPLAY_PLANE_ALPHA_GLOBAL_BIT_KHR` specifies that a global alpha value **must** be specified that will be applied to all pixels in the source image.
- `VK_DISPLAY_PLANE_ALPHA_PER_PIXEL_BIT_KHR` specifies that the alpha value will be determined by the alpha component of the source image's pixels. If the source format contains no alpha values, no blending will be applied. The source alpha values are not premultiplied into the source

image's other color components.

- VK_DISPLAY_PLANE_ALPHA_PER_PIXEL_PREMULTIPLIED_BIT_KHR is equivalent to VK_DISPLAY_PLANE_ALPHA_PER_PIXEL_BIT_KHR, except the source alpha values are assumed to be premultiplied into the source image's other color components.

```
// Provided by VK_KHR_display
typedef VkFlags VkDisplayPlaneAlphaFlagsKHR;
```

VkDisplayPlaneAlphaFlagsKHR is a bitmask type for setting a mask of zero or more VkDisplayPlaneAlphaFlagBitsKHR.

32.3.4. Presenting to Headless Surfaces

Vulkan rendering can be presented to a headless surface, where the presentation operation is a no-op producing no externally-visible result.

NOTE

Because there is no real presentation target, the headless presentation engine may be extended to impose an arbitrary or customizable set of restrictions and features. This makes it a useful portable test target for applications targeting a wide range of presentation engines where the actual target presentation engines might be scarce, unavailable or otherwise undesirable or inconvenient to use for general Vulkan application development.

The usual surface query mechanisms must be used to determine the actual restrictions and features of the implementation.

To create a headless VkSurfaceKHR object, call:

```
// Provided by VK_EXT_headless_surface
VkResult vkCreateHeadlessSurfaceEXT(
    VkInstance                                  instance,
    const VkHeadlessSurfaceCreateInfoEXT*       pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSurfaceKHR*                               pSurface);
```

- instance is the instance to associate the surface with.
- pCreateInfo is a pointer to a VkHeadlessSurfaceCreateInfoEXT structure containing parameters affecting the creation of the surface object.
- pAllocator is the allocator used for host memory allocated for the surface object when there is no more specific allocator available (see Memory Allocation).
- pSurface is a pointer to a VkSurfaceKHR handle in which the created surface object is returned.

Valid Usage (Implicit)

- VUID-vkCreateHeadlessSurfaceEXT-instance-parameter

`instance` **must** be a valid VkInstance handle

- VUID-vkCreateHeadlessSurfaceEXT-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkHeadlessSurfaceCreateInfoEXT structure
- VUID-vkCreateHeadlessSurfaceEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateHeadlessSurfaceEXT-pSurface-parameter
  `pSurface` **must** be a valid pointer to a VkSurfaceKHR handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkHeadlessSurfaceCreateInfoEXT` structure is defined as:

```
// Provided by VK_EXT_headless_surface
typedef struct VkHeadlessSurfaceCreateInfoEXT {
    VkStructureType                    sType;
    const void*                        pNext;
    VkHeadlessSurfaceCreateFlagsEXT    flags;
} VkHeadlessSurfaceCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.

Valid Usage (Implicit)

- VUID-VkHeadlessSurfaceCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_HEADLESS_SURFACE_CREATE_INFO_EXT`
- VUID-VkHeadlessSurfaceCreateInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkHeadlessSurfaceCreateInfoEXT-flags-zerobitmask
  `flags` **must** be `0`

For headless surfaces, `currentExtent` is the reserved value (0xFFFFFFFF, 0xFFFFFFFF). Whatever the application sets a swapchain's `imageExtent` to will be the size of the surface, after the first image is

presented.

```
// Provided by VK_EXT_headless_surface
typedef VkFlags VkHeadlessSurfaceCreateFlagsEXT;
```

VkHeadlessSurfaceCreateFlagsEXT is a bitmask type for setting a mask, but is currently reserved for future use.

32.4. Querying for WSI Support

Not all physical devices will include WSI support. Within a physical device, not all queue families will support presentation. WSI support and compatibility **can** be determined in a platform-neutral manner (which determines support for presentation to a particular surface object) and additionally **may** be determined in platform-specific manners (which determine support for presentation on the specified physical device but do not guarantee support for presentation to a particular surface object).

To determine whether a queue family of a physical device supports presentation to a given surface, call:

```
// Provided by VK_KHR_surface
VkResult vkGetPhysicalDeviceSurfaceSupportKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t                                    queueFamilyIndex,
    VkSurfaceKHR                                surface,
    VkBool32*                                   pSupported);
```

- physicalDevice is the physical device.
- queueFamilyIndex is the queue family.
- surface is the surface.
- pSupported is a pointer to a VkBool32. VK_TRUE indicates support, and VK_FALSE indicates no support.

Valid Usage

- VUID-vkGetPhysicalDeviceSurfaceSupportKHR-queueFamilyIndex-01269 queueFamilyIndex **must** be less than pQueueFamilyPropertyCount returned by vkGetPhysicalDeviceQueueFamilyProperties for the given physicalDevice

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSurfaceSupportKHR-physicalDevice-parameter physicalDevice **must** be a valid VkPhysicalDevice handle

- VUID-vkGetPhysicalDeviceSurfaceSupportKHR-surface-parameter
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceSupportKHR-pSupported-parameter
  `pSupported` **must** be a valid pointer to a VkBool32 value
- VUID-vkGetPhysicalDeviceSurfaceSupportKHR-commonparent
  Both of `physicalDevice`, and `surface` **must** have been created, allocated, or retrieved from the same VkInstance

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

32.4.1. Android Platform

On Android, all physical devices and queue families **must** be capable of presentation with any native window. As a result there is no Android-specific query for these capabilities.

32.4.2. Wayland Platform

To determine whether a queue family of a physical device supports presentation to a Wayland compositor, call:

```
// Provided by VK_KHR_wayland_surface
VkBool32 vkGetPhysicalDeviceWaylandPresentationSupportKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t                                    queueFamilyIndex,
    struct wl_display*                          display);
```

- `physicalDevice` is the physical device.
- `queueFamilyIndex` is the queue family index.
- `display` is a pointer to the `wl_display` associated with a Wayland compositor.

This platform-specific function **can** be called prior to creating a surface.

Valid Usage

- VUID-vkGetPhysicalDeviceWaylandPresentationSupportKHR-queueFamilyIndex-01306

queueFamilyIndex **must** be less than pQueueFamilyPropertyCount returned by vkGetPhysicalDeviceQueueFamilyProperties for the given physicalDevice

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceWaylandPresentationSupportKHR-physicalDevice-parameter physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceWaylandPresentationSupportKHR-display-parameter display **must** be a valid pointer to a wl_display value

32.4.3. Win32 Platform

To determine whether a queue family of a physical device supports presentation to the Microsoft Windows desktop, call:

```
// Provided by VK_KHR_win32_surface
VkBool32 vkGetPhysicalDeviceWin32PresentationSupportKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t                                    queueFamilyIndex);
```

- physicalDevice is the physical device.
- queueFamilyIndex is the queue family index.

This platform-specific function **can** be called prior to creating a surface.

Valid Usage

- VUID-vkGetPhysicalDeviceWin32PresentationSupportKHR-queueFamilyIndex-01309 queueFamilyIndex **must** be less than pQueueFamilyPropertyCount returned by vkGetPhysicalDeviceQueueFamilyProperties for the given physicalDevice

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceWin32PresentationSupportKHR-physicalDevice-parameter physicalDevice **must** be a valid VkPhysicalDevice handle

32.4.4. XCB Platform

To determine whether a queue family of a physical device supports presentation to an X11 server, using the XCB client-side library, call:

```
// Provided by VK_KHR_xcb_surface
VkBool32 vkGetPhysicalDeviceXcbPresentationSupportKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t                                    queueFamilyIndex,
    xcb_connection_t*                           connection,
    xcb_visualid_t                              visual_id);
```

- `physicalDevice` is the physical device.
- `queueFamilyIndex` is the queue family index.
- `connection` is a pointer to an `xcb_connection_t` to the X server.
- `visual_id` is an X11 visual (`xcb_visualid_t`).

This platform-specific function **can** be called prior to creating a surface.

Valid Usage

- VUID-vkGetPhysicalDeviceXcbPresentationSupportKHR-queueFamilyIndex-01312 `queueFamilyIndex` **must** be less than `pQueueFamilyPropertyCount` returned by `vkGetPhysicalDeviceQueueFamilyProperties` for the given `physicalDevice`

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceXcbPresentationSupportKHR-physicalDevice-parameter `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceXcbPresentationSupportKHR-connection-parameter `connection` **must** be a valid pointer to an `xcb_connection_t` value

32.4.5. Xlib Platform

To determine whether a queue family of a physical device supports presentation to an X11 server, using the Xlib client-side library, call:

```
// Provided by VK_KHR_xlib_surface
VkBool32 vkGetPhysicalDeviceXlibPresentationSupportKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t                                    queueFamilyIndex,
    Display*                                    dpy,
    VisualID                                    visualID);
```

- `physicalDevice` is the physical device.
- `queueFamilyIndex` is the queue family index.
- `dpy` is a pointer to an Xlib `Display` connection to the server.

- `visualId` is an X11 visual (`VisualID`).

This platform-specific function **can** be called prior to creating a surface.

Valid Usage

- VUID-vkGetPhysicalDeviceXlibPresentationSupportKHR-queueFamilyIndex-01315 `queueFamilyIndex` **must** be less than `pQueueFamilyPropertyCount` returned by `vkGetPhysicalDeviceQueueFamilyProperties` for the given `physicalDevice`

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceXlibPresentationSupportKHR-physicalDevice-parameter `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceXlibPresentationSupportKHR-dpy-parameter `dpy` **must** be a valid pointer to a `Display` value

32.5. Surface Queries

The capabilities of a swapchain targeting a surface are the intersection of the capabilities of the WSI platform, the native window or display, and the physical device. The resulting capabilities **can** be obtained with the queries listed below in this section.

NOTE In addition to the surface capabilities as obtained by surface queries below, swapchain images are also subject to ordinary image creation limits as reported by vkGetPhysicalDeviceImageFormatProperties. As an application is instructed by the appropriate Valid Usage sections, both the surface capabilities and the image creation limits have to be satisfied whenever swapchain images are created.

32.5.1. Surface Capabilities

To query the basic capabilities of a surface, needed in order to create a swapchain, call:

```
// Provided by VK_KHR_surface
VkResult vkGetPhysicalDeviceSurfaceCapabilitiesKHR(
    VkPhysicalDevice                            physicalDevice,
    VkSurfaceKHR                                surface,
    VkSurfaceCapabilitiesKHR*                   pSurfaceCapabilities);
```

- `physicalDevice` is the physical device that will be associated with the swapchain to be created, as described for vkCreateSwapchainKHR.
- `surface` is the surface that will be associated with the swapchain.
- `pSurfaceCapabilities` is a pointer to a VkSurfaceCapabilitiesKHR structure in which the

capabilities are returned.

Valid Usage

- VUID-vkGetPhysicalDeviceSurfaceCapabilitiesKHR-surface-06523
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceCapabilitiesKHR-surface-06211
  `surface` **must** be supported by `physicalDevice`, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSurfaceCapabilitiesKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSurfaceCapabilitiesKHR-surface-parameter
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceCapabilitiesKHR-pSurfaceCapabilities-parameter
  `pSurfaceCapabilities` **must** be a valid pointer to a VkSurfaceCapabilitiesKHR structure
- VUID-vkGetPhysicalDeviceSurfaceCapabilitiesKHR-commonparent
  Both of `physicalDevice`, and `surface` **must** have been created, allocated, or retrieved from the same VkInstance

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

The `VkSurfaceCapabilitiesKHR` structure is defined as:

```
// Provided by VK_KHR_surface
typedef struct VkSurfaceCapabilitiesKHR {
    uint32_t                         minImageCount;
    uint32_t                         maxImageCount;
    VkExtent2D                       currentExtent;
    VkExtent2D                       minImageExtent;
    VkExtent2D                       maxImageExtent;
    uint32_t                         maxImageArrayLayers;
    VkSurfaceTransformFlagsKHR       supportedTransforms;
    VkSurfaceTransformFlagBitsKHR    currentTransform;
    VkCompositeAlphaFlagsKHR         supportedCompositeAlpha;
    VkImageUsageFlags                supportedUsageFlags;
} VkSurfaceCapabilitiesKHR;
```

- `minImageCount` is the minimum number of images the specified device supports for a swapchain created for the surface, and will be at least one.
- `maxImageCount` is the maximum number of images the specified device supports for a swapchain created for the surface, and will be either 0, or greater than or equal to `minImageCount`. A value of 0 means that there is no limit on the number of images, though there **may** be limits related to the total amount of memory used by presentable images.
- `currentExtent` is the current width and height of the surface, or the special value (0xFFFFFFFF, 0xFFFFFFFF) indicating that the surface size will be determined by the extent of a swapchain targeting the surface.
- `minImageExtent` contains the smallest valid swapchain extent for the surface on the specified device. The `width` and `height` of the extent will each be less than or equal to the corresponding `width` and `height` of `currentExtent`, unless `currentExtent` has the special value described above.
- `maxImageExtent` contains the largest valid swapchain extent for the surface on the specified device. The `width` and `height` of the extent will each be greater than or equal to the corresponding `width` and `height` of `minImageExtent`. The `width` and `height` of the extent will each be greater than or equal to the corresponding `width` and `height` of `currentExtent`, unless `currentExtent` has the special value described above.
- `maxImageArrayLayers` is the maximum number of layers presentable images **can** have for a swapchain created for this device and surface, and will be at least one.
- `supportedTransforms` is a bitmask of VkSurfaceTransformFlagBitsKHR indicating the presentation transforms supported for the surface on the specified device. At least one bit will be set.
- `currentTransform` is VkSurfaceTransformFlagBitsKHR value indicating the surface's current transform relative to the presentation engine's natural orientation.
- `supportedCompositeAlpha` is a bitmask of VkCompositeAlphaFlagBitsKHR, representing the alpha compositing modes supported by the presentation engine for the surface on the specified device, and at least one bit will be set. Opaque composition **can** be achieved in any alpha compositing mode by either using an image format that has no alpha component, or by ensuring that all pixels in the presentable images have an alpha value of 1.0.
- `supportedUsageFlags` is a bitmask of VkImageUsageFlagBits representing the ways the

application **can** use the presentable images of a swapchain created with VkPresentModeKHR set to VK_PRESENT_MODE_FIFO_LATEST_READY_EXT, VK_PRESENT_MODE_IMMEDIATE_KHR, VK_PRESENT_MODE_MAILBOX_KHR, VK_PRESENT_MODE_FIFO_KHR or VK_PRESENT_MODE_FIFO_RELAXED_KHR for the surface on the specified device. VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT **must** be included in the set. Implementations **may** support additional usages.

NOTE	Supported usage flags of a presentable image when using VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR or VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR presentation mode are provided by VkSharedPresentSurfaceCapabilitiesKHR::sharedPresentSupportedUsageFlags.

NOTE	Formulas such as min(N, maxImageCount) are not correct, since maxImageCount **may** be zero.

To query the basic capabilities of a surface defined by the core or extensions, call:

```
// Provided by VK_KHR_get_surface_capabilities2
VkResult vkGetPhysicalDeviceSurfaceCapabilities2KHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceSurfaceInfo2KHR*      pSurfaceInfo,
    VkSurfaceCapabilities2KHR*                  pSurfaceCapabilities);
```

- physicalDevice is the physical device that will be associated with the swapchain to be created, as described for vkCreateSwapchainKHR.
- pSurfaceInfo is a pointer to a VkPhysicalDeviceSurfaceInfo2KHR structure describing the surface and other fixed parameters that would be consumed by vkCreateSwapchainKHR.
- pSurfaceCapabilities is a pointer to a VkSurfaceCapabilities2KHR structure in which the capabilities are returned.

vkGetPhysicalDeviceSurfaceCapabilities2KHR behaves similarly to vkGetPhysicalDeviceSurfaceCapabilitiesKHR, with the ability to specify extended inputs via chained input structures, and to return extended information via chained output structures.

Valid Usage

- VUID-vkGetPhysicalDeviceSurfaceCapabilities2KHR-pSurfaceInfo-06521
  pSurfaceInfo->surface **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceCapabilities2KHR-pSurfaceInfo-06522
  pSurfaceInfo->surface **must** be supported by physicalDevice, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism
- VUID-vkGetPhysicalDeviceSurfaceCapabilities2KHR-pNext-02671
  If a VkSurfaceCapabilitiesFullScreenExclusiveEXT structure is included in the pNext chain of pSurfaceCapabilities, a VkSurfaceFullScreenExclusiveWin32InfoEXT structure **must** be included in the pNext chain of pSurfaceInfo

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSurfaceCapabilities2KHR-physicalDevice-parameter `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSurfaceCapabilities2KHR-pSurfaceInfo-parameter `pSurfaceInfo` **must** be a valid pointer to a valid VkPhysicalDeviceSurfaceInfo2KHR structure
- VUID-vkGetPhysicalDeviceSurfaceCapabilities2KHR-pSurfaceCapabilities-parameter `pSurfaceCapabilities` **must** be a valid pointer to a VkSurfaceCapabilities2KHR structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

The `VkPhysicalDeviceSurfaceInfo2KHR` structure is defined as:

```
// Provided by VK_KHR_get_surface_capabilities2
typedef struct VkPhysicalDeviceSurfaceInfo2KHR {
    VkStructureType    sType;
    const void*        pNext;
    VkSurfaceKHR       surface;
} VkPhysicalDeviceSurfaceInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `surface` is the surface that will be associated with the swapchain.

The members of `VkPhysicalDeviceSurfaceInfo2KHR` correspond to the arguments to vkGetPhysicalDeviceSurfaceCapabilitiesKHR, with `sType` and `pNext` added for extensibility.

Additional capabilities of a surface **may** be available to swapchains created with different full-screen exclusive settings - particularly if exclusive full-screen access is application controlled. These additional capabilities **can** be queried by adding a VkSurfaceFullScreenExclusiveInfoEXT structure to the `pNext` chain of this structure when used to query surface properties. Additionally, for Win32 surfaces with application controlled exclusive full-screen access, chaining a VkSurfaceFullScreenExclusiveWin32InfoEXT structure **may** also report additional surface capabilities. These additional capabilities only apply to swapchains created with the same

parameters included in the pNext chain of VkSwapchainCreateInfoKHR.

Valid Usage

- VUID-VkPhysicalDeviceSurfaceInfo2KHR-pNext-02672
  If the pNext chain includes a VkSurfaceFullScreenExclusiveInfoEXT structure with its fullScreenExclusive member set to VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT, and surface was created using vkCreateWin32SurfaceKHR, a VkSurfaceFullScreenExclusiveWin32InfoEXT structure **must** be included in the pNext chain
- VUID-VkPhysicalDeviceSurfaceInfo2KHR-surface-07919
  If surface is not VK_NULL_HANDLE, surface **must** be a valid VkSurfaceKHR handle

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceSurfaceInfo2KHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SURFACE_INFO_2_KHR
- VUID-VkPhysicalDeviceSurfaceInfo2KHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkSurfaceFullScreenExclusiveInfoEXT or VkSurfaceFullScreenExclusiveWin32InfoEXT
- VUID-VkPhysicalDeviceSurfaceInfo2KHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique

If the pNext chain of VkSwapchainCreateInfoKHR includes a VkSurfaceFullScreenExclusiveInfoEXT structure, then that structure specifies the application's preferred full-screen transition behavior.

The VkSurfaceFullScreenExclusiveInfoEXT structure is defined as:

```
// Provided by VK_EXT_full_screen_exclusive
typedef struct VkSurfaceFullScreenExclusiveInfoEXT {
    VkStructureType             sType;
    void*                       pNext;
    VkFullScreenExclusiveEXT    fullScreenExclusive;
} VkSurfaceFullScreenExclusiveInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- fullScreenExclusive is a VkFullScreenExclusiveEXT value specifying the preferred full-screen transition behavior.

If this structure is not present, fullScreenExclusive is considered to be VK_FULL_SCREEN_EXCLUSIVE_DEFAULT_EXT.

Valid Usage (Implicit)

- VUID-VkSurfaceFullScreenExclusiveInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SURFACE_FULL_SCREEN_EXCLUSIVE_INFO_EXT`
- VUID-VkSurfaceFullScreenExclusiveInfoEXT-fullScreenExclusive-parameter
  `fullScreenExclusive` **must** be a valid VkFullScreenExclusiveEXT value

Possible values of `VkSurfaceFullScreenExclusiveInfoEXT::fullScreenExclusive` are:

```
// Provided by VK_EXT_full_screen_exclusive
typedef enum VkFullScreenExclusiveEXT {
    VK_FULL_SCREEN_EXCLUSIVE_DEFAULT_EXT = 0,
    VK_FULL_SCREEN_EXCLUSIVE_ALLOWED_EXT = 1,
    VK_FULL_SCREEN_EXCLUSIVE_DISALLOWED_EXT = 2,
    VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT = 3,
} VkFullScreenExclusiveEXT;
```

- `VK_FULL_SCREEN_EXCLUSIVE_DEFAULT_EXT` specifies that the implementation **should** determine the appropriate full-screen method by whatever means it deems appropriate.
- `VK_FULL_SCREEN_EXCLUSIVE_ALLOWED_EXT` specifies that the implementation **may** use full-screen exclusive mechanisms when available. Such mechanisms **may** result in better performance and/or the availability of different presentation capabilities, but **may** require a more disruptive transition during swapchain initialization, first presentation and/or destruction.
- `VK_FULL_SCREEN_EXCLUSIVE_DISALLOWED_EXT` specifies that the implementation **should** avoid using full-screen mechanisms which rely on disruptive transitions.
- `VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT` specifies that the application will manage full-screen exclusive mode by using the vkAcquireFullScreenExclusiveModeEXT and vkReleaseFullScreenExclusiveModeEXT commands.

The `VkSurfaceFullScreenExclusiveWin32InfoEXT` structure is defined as:

```
// Provided by VK_KHR_win32_surface with VK_EXT_full_screen_exclusive
typedef struct VkSurfaceFullScreenExclusiveWin32InfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    HMONITOR           hmonitor;
} VkSurfaceFullScreenExclusiveWin32InfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `hmonitor` is the Win32 `HMONITOR` handle identifying the display to create the surface with.

NOTE If `hmonitor` is invalidated (e.g. the monitor is unplugged) during the lifetime of a

swapchain created with this structure, operations on that swapchain will return VK_ERROR_OUT_OF_DATE_KHR.

NOTE It is the responsibility of the application to change the display settings of the targeted Win32 display using the appropriate platform APIs. Such changes **may** alter the surface capabilities reported for the created surface.

Valid Usage

- VUID-VkSurfaceFullScreenExclusiveWin32InfoEXT-hmonitor-02673
  hmonitor **must** be a valid HMONITOR

Valid Usage (Implicit)

- VUID-VkSurfaceFullScreenExclusiveWin32InfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SURFACE_FULL_SCREEN_EXCLUSIVE_WIN32_INFO_EXT

The VkSurfaceCapabilities2KHR structure is defined as:

```
// Provided by VK_KHR_get_surface_capabilities2
typedef struct VkSurfaceCapabilities2KHR {
    VkStructureType             sType;
    void*                       pNext;
    VkSurfaceCapabilitiesKHR    surfaceCapabilities;
} VkSurfaceCapabilities2KHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- surfaceCapabilities is a VkSurfaceCapabilitiesKHR structure describing the capabilities of the specified surface.

Valid Usage (Implicit)

- VUID-VkSurfaceCapabilities2KHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_2_KHR
- VUID-VkSurfaceCapabilities2KHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkSharedPresentSurfaceCapabilitiesKHR, VkSurfaceCapabilitiesFullScreenExclusiveEXT, or VkSurfaceProtectedCapabilitiesKHR
- VUID-VkSurfaceCapabilities2KHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique

An application queries if a protected VkSurfaceKHR is displayable on a specific windowing system using VkSurfaceProtectedCapabilitiesKHR, which **can** be passed in pNext parameter of VkSurfaceCapabilities2KHR.

The VkSurfaceProtectedCapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_surface_protected_capabilities
typedef struct VkSurfaceProtectedCapabilitiesKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           supportsProtected;
} VkSurfaceProtectedCapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- supportsProtected specifies whether a protected swapchain created from VkPhysicalDeviceSurfaceInfo2KHR::surface for a particular windowing system **can** be displayed on screen or not. If supportsProtected is VK_TRUE, then creation of swapchains with the VK_SWAPCHAIN_CREATE_PROTECTED_BIT_KHR flag set **must** be supported for surface.

Valid Usage (Implicit)

- VUID-VkSurfaceProtectedCapabilitiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SURFACE_PROTECTED_CAPABILITIES_KHR

The VkSharedPresentSurfaceCapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_shared_presentable_image
typedef struct VkSharedPresentSurfaceCapabilitiesKHR {
    VkStructureType      sType;
    void*                pNext;
    VkImageUsageFlags    sharedPresentSupportedUsageFlags;
} VkSharedPresentSurfaceCapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- sharedPresentSupportedUsageFlags is a bitmask of VkImageUsageFlagBits representing the ways the application **can** use the shared presentable image from a swapchain created with VkPresentModeKHR set to VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR or VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR for the surface on the specified device. VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT **must** be included in the set but implementations **may** support additional usages.

Valid Usage (Implicit)

- VUID-VkSharedPresentSurfaceCapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SHARED_PRESENT_SURFACE_CAPABILITIES_KHR`

The `VkSurfaceCapabilitiesFullScreenExclusiveEXT` structure is defined as:

```
// Provided by VK_EXT_full_screen_exclusive
typedef struct VkSurfaceCapabilitiesFullScreenExclusiveEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           fullScreenExclusiveSupported;
} VkSurfaceCapabilitiesFullScreenExclusiveEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fullScreenExclusiveControlSupported` is a boolean describing whether the surface is able to make use of exclusive full-screen access.

This structure **can** be included in the `pNext` chain of VkSurfaceCapabilities2KHR to determine support for exclusive full-screen access. If `fullScreenExclusiveSupported` is `VK_FALSE`, it indicates that exclusive full-screen access is not obtainable for this surface.

Applications **must** not attempt to create swapchains with `VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT` set if `fullScreenExclusiveSupported` is `VK_FALSE`.

Valid Usage (Implicit)

- VUID-VkSurfaceCapabilitiesFullScreenExclusiveEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_FULL_SCREEN_EXCLUSIVE_EXT`

To query the basic capabilities of a surface, needed in order to create a swapchain, call:

```
// Provided by VK_EXT_display_surface_counter
VkResult vkGetPhysicalDeviceSurfaceCapabilities2EXT(
    VkPhysicalDevice                            physicalDevice,
    VkSurfaceKHR                                surface,
    VkSurfaceCapabilities2EXT*                  pSurfaceCapabilities);
```

- `physicalDevice` is the physical device that will be associated with the swapchain to be created, as described for vkCreateSwapchainKHR.
- `surface` is the surface that will be associated with the swapchain.

- `pSurfaceCapabilities` is a pointer to a VkSurfaceCapabilities2EXT structure in which the capabilities are returned.

`vkGetPhysicalDeviceSurfaceCapabilities2EXT` behaves similarly to vkGetPhysicalDeviceSurfaceCapabilitiesKHR, with the ability to return extended information by adding extending structures to the `pNext` chain of its `pSurfaceCapabilities` parameter.

Valid Usage

- VUID-vkGetPhysicalDeviceSurfaceCapabilities2EXT-surface-06523
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceCapabilities2EXT-surface-06211
  `surface` **must** be supported by `physicalDevice`, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSurfaceCapabilities2EXT-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSurfaceCapabilities2EXT-surface-parameter
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceCapabilities2EXT-pSurfaceCapabilities-parameter
  `pSurfaceCapabilities` **must** be a valid pointer to a VkSurfaceCapabilities2EXT structure
- VUID-vkGetPhysicalDeviceSurfaceCapabilities2EXT-commonparent
  Both of `physicalDevice`, and `surface` **must** have been created, allocated, or retrieved from the same VkInstance

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

The `VkSurfaceCapabilities2EXT` structure is defined as:

```
// Provided by VK_EXT_display_surface_counter
typedef struct VkSurfaceCapabilities2EXT {
    VkStructureType                  sType;
    void*                            pNext;
    uint32_t                         minImageCount;
    uint32_t                         maxImageCount;
    VkExtent2D                       currentExtent;
    VkExtent2D                       minImageExtent;
    VkExtent2D                       maxImageExtent;
    uint32_t                         maxImageArrayLayers;
    VkSurfaceTransformFlagsKHR       supportedTransforms;
    VkSurfaceTransformFlagBitsKHR    currentTransform;
    VkCompositeAlphaFlagsKHR         supportedCompositeAlpha;
    VkImageUsageFlags                supportedUsageFlags;
    VkSurfaceCounterFlagsEXT         supportedSurfaceCounters;
} VkSurfaceCapabilities2EXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `minImageCount` is the minimum number of images the specified device supports for a swapchain created for the surface, and will be at least one.
- `maxImageCount` is the maximum number of images the specified device supports for a swapchain created for the surface, and will be either 0, or greater than or equal to `minImageCount`. A value of 0 means that there is no limit on the number of images, though there **may** be limits related to the total amount of memory used by presentable images.
- `currentExtent` is the current width and height of the surface, or the special value (0xFFFFFFFF, 0xFFFFFFFF) indicating that the surface size will be determined by the extent of a swapchain targeting the surface.
- `minImageExtent` contains the smallest valid swapchain extent for the surface on the specified device. The `width` and `height` of the extent will each be less than or equal to the corresponding `width` and `height` of `currentExtent`, unless `currentExtent` has the special value described above.
- `maxImageExtent` contains the largest valid swapchain extent for the surface on the specified device. The `width` and `height` of the extent will each be greater than or equal to the corresponding `width` and `height` of `minImageExtent`. The `width` and `height` of the extent will each be greater than or equal to the corresponding `width` and `height` of `currentExtent`, unless `currentExtent` has the special value described above.
- `maxImageArrayLayers` is the maximum number of layers presentable images **can** have for a swapchain created for this device and surface, and will be at least one.
- `supportedTransforms` is a bitmask of VkSurfaceTransformFlagBitsKHR indicating the presentation transforms supported for the surface on the specified device. At least one bit will be set.
- `currentTransform` is VkSurfaceTransformFlagBitsKHR value indicating the surface's current transform relative to the presentation engine's natural orientation.
- `supportedCompositeAlpha` is a bitmask of VkCompositeAlphaFlagBitsKHR, representing the alpha

compositing modes supported by the presentation engine for the surface on the specified device, and at least one bit will be set. Opaque composition **can** be achieved in any alpha compositing mode by either using an image format that has no alpha component, or by ensuring that all pixels in the presentable images have an alpha value of 1.0.

- `supportedUsageFlags` is a bitmask of VkImageUsageFlagBits representing the ways the application **can** use the presentable images of a swapchain created with VkPresentModeKHR set to `VK_PRESENT_MODE_FIFO_LATEST_READY_EXT`, `VK_PRESENT_MODE_IMMEDIATE_KHR`, `VK_PRESENT_MODE_MAILBOX_KHR`, `VK_PRESENT_MODE_FIFO_KHR` or `VK_PRESENT_MODE_FIFO_RELAXED_KHR` for the surface on the specified device. `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT` **must** be included in the set. Implementations **may** support additional usages.
- `supportedSurfaceCounters` is a bitmask of VkSurfaceCounterFlagBitsEXT indicating the supported surface counter types.

Valid Usage

- VUID-VkSurfaceCapabilities2EXT-supportedSurfaceCounters-01246
  `supportedSurfaceCounters` **must** not include `VK_SURFACE_COUNTER_VBLANK_BIT_EXT` unless the surface queried is a display surface

Valid Usage (Implicit)

- VUID-VkSurfaceCapabilities2EXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_2_EXT`
- VUID-VkSurfaceCapabilities2EXT-pNext-pNext
  `pNext` **must** be `NULL`

Bits which **can** be set in VkSurfaceCapabilities2EXT::`supportedSurfaceCounters`, indicating supported surface counter types, are:

```
// Provided by VK_EXT_display_surface_counter
typedef enum VkSurfaceCounterFlagBitsEXT {
    VK_SURFACE_COUNTER_VBLANK_BIT_EXT = 0x00000001,
  // VK_SURFACE_COUNTER_VBLANK_EXT is a deprecated alias
    VK_SURFACE_COUNTER_VBLANK_EXT = VK_SURFACE_COUNTER_VBLANK_BIT_EXT,
} VkSurfaceCounterFlagBitsEXT;
```

- `VK_SURFACE_COUNTER_VBLANK_BIT_EXT` specifies a counter incrementing once every time a vertical blanking period occurs on the display associated with the surface.

```
// Provided by VK_EXT_display_surface_counter
typedef VkFlags VkSurfaceCounterFlagsEXT;
```

`VkSurfaceCounterFlagsEXT` is a bitmask type for setting a mask of zero or more

VkSurfaceCounterFlagBitsEXT.

Bits which **may** be set in VkSurfaceCapabilitiesKHR::supportedTransforms indicating the presentation transforms supported for the surface on the specified device, and possible values of VkSurfaceCapabilitiesKHR::currentTransform indicating the surface's current transform relative to the presentation engine's natural orientation, are:

```
// Provided by VK_KHR_surface
typedef enum VkSurfaceTransformFlagBitsKHR {
    VK_SURFACE_TRANSFORM_IDENTITY_BIT_KHR = 0x00000001,
    VK_SURFACE_TRANSFORM_ROTATE_90_BIT_KHR = 0x00000002,
    VK_SURFACE_TRANSFORM_ROTATE_180_BIT_KHR = 0x00000004,
    VK_SURFACE_TRANSFORM_ROTATE_270_BIT_KHR = 0x00000008,
    VK_SURFACE_TRANSFORM_HORIZONTAL_MIRROR_BIT_KHR = 0x00000010,
    VK_SURFACE_TRANSFORM_HORIZONTAL_MIRROR_ROTATE_90_BIT_KHR = 0x00000020,
    VK_SURFACE_TRANSFORM_HORIZONTAL_MIRROR_ROTATE_180_BIT_KHR = 0x00000040,
    VK_SURFACE_TRANSFORM_HORIZONTAL_MIRROR_ROTATE_270_BIT_KHR = 0x00000080,
    VK_SURFACE_TRANSFORM_INHERIT_BIT_KHR = 0x00000100,
} VkSurfaceTransformFlagBitsKHR;
```

- VK_SURFACE_TRANSFORM_IDENTITY_BIT_KHR specifies that image content is presented without being transformed.
- VK_SURFACE_TRANSFORM_ROTATE_90_BIT_KHR specifies that image content is rotated 90 degrees clockwise.
- VK_SURFACE_TRANSFORM_ROTATE_180_BIT_KHR specifies that image content is rotated 180 degrees clockwise.
- VK_SURFACE_TRANSFORM_ROTATE_270_BIT_KHR specifies that image content is rotated 270 degrees clockwise.
- VK_SURFACE_TRANSFORM_HORIZONTAL_MIRROR_BIT_KHR specifies that image content is mirrored horizontally.
- VK_SURFACE_TRANSFORM_HORIZONTAL_MIRROR_ROTATE_90_BIT_KHR specifies that image content is mirrored horizontally, then rotated 90 degrees clockwise.
- VK_SURFACE_TRANSFORM_HORIZONTAL_MIRROR_ROTATE_180_BIT_KHR specifies that image content is mirrored horizontally, then rotated 180 degrees clockwise.
- VK_SURFACE_TRANSFORM_HORIZONTAL_MIRROR_ROTATE_270_BIT_KHR specifies that image content is mirrored horizontally, then rotated 270 degrees clockwise.
- VK_SURFACE_TRANSFORM_INHERIT_BIT_KHR specifies that the presentation transform is not specified, and is instead determined by platform-specific considerations and mechanisms outside Vulkan.

```
// Provided by VK_KHR_display
typedef VkFlags VkSurfaceTransformFlagsKHR;
```

VkSurfaceTransformFlagsKHR is a bitmask type for setting a mask of zero or more VkSurfaceTransformFlagBitsKHR.

The supportedCompositeAlpha member is of type VkCompositeAlphaFlagBitsKHR, containing the following values:

```
// Provided by VK_KHR_surface
typedef enum VkCompositeAlphaFlagBitsKHR {
    VK_COMPOSITE_ALPHA_OPAQUE_BIT_KHR = 0x00000001,
    VK_COMPOSITE_ALPHA_PRE_MULTIPLIED_BIT_KHR = 0x00000002,
    VK_COMPOSITE_ALPHA_POST_MULTIPLIED_BIT_KHR = 0x00000004,
    VK_COMPOSITE_ALPHA_INHERIT_BIT_KHR = 0x00000008,
} VkCompositeAlphaFlagBitsKHR;
```

These values are described as follows:

- VK_COMPOSITE_ALPHA_OPAQUE_BIT_KHR: The alpha component, if it exists, of the images is ignored in the compositing process. Instead, the image is treated as if it has a constant alpha of 1.0.
- VK_COMPOSITE_ALPHA_PRE_MULTIPLIED_BIT_KHR: The alpha component, if it exists, of the images is respected in the compositing process. The non-alpha components of the image are expected to already be multiplied by the alpha component by the application.
- VK_COMPOSITE_ALPHA_POST_MULTIPLIED_BIT_KHR: The alpha component, if it exists, of the images is respected in the compositing process. The non-alpha components of the image are not expected to already be multiplied by the alpha component by the application; instead, the compositor will multiply the non-alpha components of the image by the alpha component during compositing.
- VK_COMPOSITE_ALPHA_INHERIT_BIT_KHR: The way in which the presentation engine treats the alpha component in the images is unknown to the Vulkan API. Instead, the application is responsible for setting the composite alpha blending mode using native window system commands. If the application does not set the blending mode using native window system commands, then a platform-specific default will be used.

```
// Provided by VK_KHR_surface
typedef VkFlags VkCompositeAlphaFlagsKHR;
```

VkCompositeAlphaFlagsKHR is a bitmask type for setting a mask of zero or more VkCompositeAlphaFlagBitsKHR.

32.5.2. Surface Format Support

To query the supported swapchain format-color space pairs for a surface, call:

```
// Provided by VK_KHR_surface
VkResult vkGetPhysicalDeviceSurfaceFormatsKHR(
    VkPhysicalDevice                            physicalDevice,
    VkSurfaceKHR                                surface,
    uint32_t*                                   pSurfaceFormatCount,
    VkSurfaceFormatKHR*                         pSurfaceFormats);
```

- `physicalDevice` is the physical device that will be associated with the swapchain to be created, as described for vkCreateSwapchainKHR.
- `surface` is the surface that will be associated with the swapchain.
- `pSurfaceFormatCount` is a pointer to an integer related to the number of format pairs available or queried, as described below.
- `pSurfaceFormats` is either `NULL` or a pointer to an array of `VkSurfaceFormatKHR` structures.

If `pSurfaceFormats` is `NULL`, then the number of format pairs supported for the given `surface` is returned in `pSurfaceFormatCount`. Otherwise, `pSurfaceFormatCount` **must** point to a variable set by the application to the number of elements in the `pSurfaceFormats` array, and on return the variable is overwritten with the number of structures actually written to `pSurfaceFormats`. If the value of `pSurfaceFormatCount` is less than the number of format pairs supported, at most `pSurfaceFormatCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available format pairs were returned.

The number of format pairs supported **must** be greater than or equal to 1. `pSurfaceFormats` **must** not contain an entry whose value for `format` is `VK_FORMAT_UNDEFINED`.

If `pSurfaceFormats` includes an entry whose value for `colorSpace` is `VK_COLOR_SPACE_SRGB_NONLINEAR_KHR` and whose value for `format` is a UNORM (or SRGB) format and the corresponding SRGB (or UNORM) format is a color renderable format for `VK_IMAGE_TILING_OPTIMAL`, then `pSurfaceFormats` **must** also contain an entry with the same value for `colorSpace` and `format` equal to the corresponding SRGB (or UNORM) format.

Valid Usage

- VUID-vkGetPhysicalDeviceSurfaceFormatsKHR-surface-06524
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceFormatsKHR-surface-06525
  `surface` **must** be supported by `physicalDevice`, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSurfaceFormatsKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSurfaceFormatsKHR-surface-parameter
  If `surface` is not VK_NULL_HANDLE, `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceFormatsKHR-pSurfaceFormatCount-parameter
  `pSurfaceFormatCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceSurfaceFormatsKHR-pSurfaceFormats-parameter
  If the value referenced by `pSurfaceFormatCount` is not `0`, and `pSurfaceFormats` is not `NULL`, `pSurfaceFormats` **must** be a valid pointer to an array of `pSurfaceFormatCount` VkSurfaceFormatKHR structures

- VUID-vkGetPhysicalDeviceSurfaceFormatsKHR-commonparent
  Both of `physicalDevice`, and `surface` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkInstance

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

The `VkSurfaceFormatKHR` structure is defined as:

```
// Provided by VK_KHR_surface
typedef struct VkSurfaceFormatKHR {
    VkFormat           format;
    VkColorSpaceKHR    colorSpace;
} VkSurfaceFormatKHR;
```

- `format` is a VkFormat that is compatible with the specified surface.
- `colorSpace` is a presentation VkColorSpaceKHR that is compatible with the surface.

To query the supported swapchain format tuples for a surface, call:

```
// Provided by VK_KHR_get_surface_capabilities2
VkResult vkGetPhysicalDeviceSurfaceFormats2KHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceSurfaceInfo2KHR*      pSurfaceInfo,
    uint32_t*                                   pSurfaceFormatCount,
    VkSurfaceFormat2KHR*                        pSurfaceFormats);
```

- `physicalDevice` is the physical device that will be associated with the swapchain to be created, as described for vkCreateSwapchainKHR.
- `pSurfaceInfo` is a pointer to a VkPhysicalDeviceSurfaceInfo2KHR structure describing the surface and other fixed parameters that would be consumed by vkCreateSwapchainKHR.
- `pSurfaceFormatCount` is a pointer to an integer related to the number of format tuples available or queried, as described below.
- `pSurfaceFormats` is either `NULL` or a pointer to an array of VkSurfaceFormat2KHR structures.

vkGetPhysicalDeviceSurfaceFormats2KHR behaves similarly to vkGetPhysicalDeviceSurfaceFormatsKHR, with the ability to be extended via pNext chains.

If pSurfaceFormats is NULL, then the number of format tuples supported for the given surface is returned in pSurfaceFormatCount. Otherwise, pSurfaceFormatCount **must** point to a variable set by the application to the number of elements in the pSurfaceFormats array, and on return the variable is overwritten with the number of structures actually written to pSurfaceFormats. If the value of pSurfaceFormatCount is less than the number of format tuples supported, at most pSurfaceFormatCount structures will be written, and VK_INCOMPLETE will be returned instead of VK_SUCCESS, to indicate that not all the available values were returned.

Valid Usage

- VUID-vkGetPhysicalDeviceSurfaceFormats2KHR-pSurfaceInfo-06521
  pSurfaceInfo->surface **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfaceFormats2KHR-pSurfaceInfo-06522
  pSurfaceInfo->surface **must** be supported by physicalDevice, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSurfaceFormats2KHR-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSurfaceFormats2KHR-pSurfaceInfo-parameter
  pSurfaceInfo **must** be a valid pointer to a valid VkPhysicalDeviceSurfaceInfo2KHR structure
- VUID-vkGetPhysicalDeviceSurfaceFormats2KHR-pSurfaceFormatCount-parameter
  pSurfaceFormatCount **must** be a valid pointer to a uint32_t value
- VUID-vkGetPhysicalDeviceSurfaceFormats2KHR-pSurfaceFormats-parameter
  If the value referenced by pSurfaceFormatCount is not 0, and pSurfaceFormats is not NULL, pSurfaceFormats **must** be a valid pointer to an array of pSurfaceFormatCount VkSurfaceFormat2KHR structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_SURFACE_LOST_KHR

The VkSurfaceFormat2KHR structure is defined as:

```
// Provided by VK_KHR_get_surface_capabilities2
typedef struct VkSurfaceFormat2KHR {
    VkStructureType       sType;
    void*                 pNext;
    VkSurfaceFormatKHR    surfaceFormat;
} VkSurfaceFormat2KHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- surfaceFormat is a VkSurfaceFormatKHR structure describing a format-color space pair that is compatible with the specified surface.

Valid Usage (Implicit)

- VUID-VkSurfaceFormat2KHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_SURFACE_FORMAT_2_KHR
- VUID-VkSurfaceFormat2KHR-pNext-pNext
  pNext **must** be NULL

While the format of a presentable image refers to the encoding of each pixel, the colorSpace determines how the presentation engine interprets the pixel values. A color space in this document refers to a specific color space (defined by the chromaticities of its primaries and a white point in CIE Lab), and transfer functions indicating the mapping between the image data and the colorimetry with respect to the given color space.

Possible values of VkSurfaceFormatKHR::colorSpace, specifying the color spaces that a presentation engine can accept, are:

```
// Provided by VK_KHR_surface
typedef enum VkColorSpaceKHR {
    VK_COLOR_SPACE_SRGB_NONLINEAR_KHR = 0,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_DISPLAY_P3_NONLINEAR_EXT = 1000104001,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_EXTENDED_SRGB_LINEAR_EXT = 1000104002,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_DISPLAY_P3_LINEAR_EXT = 1000104003,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_DCI_P3_NONLINEAR_EXT = 1000104004,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_BT709_LINEAR_EXT = 1000104005,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_BT709_NONLINEAR_EXT = 1000104006,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_BT2020_LINEAR_EXT = 1000104007,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_HDR10_ST2084_EXT = 1000104008,
  // Provided by VK_EXT_swapchain_colorspace
  // VK_COLOR_SPACE_DOLBYVISION_EXT is deprecated, but no reason was given in the API
XML
    VK_COLOR_SPACE_DOLBYVISION_EXT = 1000104009,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_HDR10_HLG_EXT = 1000104010,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_ADOBERGB_LINEAR_EXT = 1000104011,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_ADOBERGB_NONLINEAR_EXT = 1000104012,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_PASS_THROUGH_EXT = 1000104013,
  // Provided by VK_EXT_swapchain_colorspace
    VK_COLOR_SPACE_EXTENDED_SRGB_NONLINEAR_EXT = 1000104014,
  // VK_COLORSPACE_SRGB_NONLINEAR_KHR is a deprecated alias
    VK_COLORSPACE_SRGB_NONLINEAR_KHR = VK_COLOR_SPACE_SRGB_NONLINEAR_KHR,
  // Provided by VK_EXT_swapchain_colorspace
  // VK_COLOR_SPACE_DCI_P3_LINEAR_EXT is a deprecated alias
    VK_COLOR_SPACE_DCI_P3_LINEAR_EXT = VK_COLOR_SPACE_DISPLAY_P3_LINEAR_EXT,
} VkColorSpaceKHR;
```

- VK_COLOR_SPACE_SRGB_NONLINEAR_KHR specifies support for the images in sRGB color space, encoded according to the sRGB specification.
- VK_COLOR_SPACE_DISPLAY_P3_NONLINEAR_EXT specifies support for the images in Display-P3 color space, encoded using a Display-P3 transfer function.
- VK_COLOR_SPACE_EXTENDED_SRGB_LINEAR_EXT specifies support for the images in extended sRGB color space, encoded using a linear transfer function.
- VK_COLOR_SPACE_EXTENDED_SRGB_NONLINEAR_EXT specifies support for the images in extended sRGB color space, encoded according to the scRGB specification.

- VK_COLOR_SPACE_DISPLAY_P3_LINEAR_EXT specifies support for the images in Display-P3 color space, encoded using a linear transfer function.
- VK_COLOR_SPACE_DCI_P3_NONLINEAR_EXT specifies support for the images in DCI-P3 color space, encoded according to the DCI-P3 specification. Note that values in such an image are interpreted as XYZ encoded color data by the presentation engine.
- VK_COLOR_SPACE_BT709_LINEAR_EXT specifies support for the images in BT709 color space, encoded using a linear transfer function.
- VK_COLOR_SPACE_BT709_NONLINEAR_EXT specifies support for the images in BT709 color space, encoded according to the BT709 specification.
- VK_COLOR_SPACE_BT2020_LINEAR_EXT specifies support for the images in BT2020 color space, encoded using a linear transfer function.
- VK_COLOR_SPACE_HDR10_ST2084_EXT specifies support for the images in HDR10 (BT2020) color space, encoded according to SMPTE ST2084 Perceptual Quantizer (PQ) specification.
- VK_COLOR_SPACE_HDR10_HLG_EXT specifies support for the images in HDR10 (BT2020) color space, encoded according to the Hybrid Log Gamma (HLG) specification.
- VK_COLOR_SPACE_ADOBERGB_LINEAR_EXT specifies support for images in Adobe RGB color space, encoded using a linear transfer function.
- VK_COLOR_SPACE_ADOBERGB_NONLINEAR_EXT specifies support for the images in Adobe RGB color space, encoded according to the Adobe RGB specification (approximately Gamma 2.2).
- VK_COLOR_SPACE_PASS_THROUGH_EXT specifies that color components are used “as is”. This is intended to allow applications to supply data for color spaces not described here.

NOTE

In the initial release of the VK_KHR_surface and VK_KHR_swapchain extensions, the token VK_COLORSPACE_SRGB_NONLINEAR_KHR was used. Starting in the 2016-05-13 updates to the extension branches, matching release 1.0.13 of the core API specification, VK_COLOR_SPACE_SRGB_NONLINEAR_KHR is used instead for consistency with Vulkan naming rules. The older enum is still available for backwards compatibility.

NOTE

In older versions of this extension VK_COLOR_SPACE_DISPLAY_P3_LINEAR_EXT was misnamed VK_COLOR_SPACE_DCI_P3_LINEAR_EXT. This has been updated to indicate that it uses RGB color encoding, not XYZ. The old name is deprecated but is maintained for backwards compatibility.

NOTE

In older versions of the VK_EXT_swapchain_colorspace extension, VK_COLOR_SPACE_DOLBYVISION_EXT was exposed. The intent was to indicate the presentation engine shall decode an image using the SMPTE ST 2084 Perceptual Quantizer (PQ) EOTF, and then apply a proprietary OOTF to process the image. However, Dolby Vision profile 8.4 describes an encoding using the Hybrid Log Gamma (HLG) OETF, and there is no swapchain extension for signaling Dolby Vision metadata to be used by a proprietary OOTF. This enum is deprecated but is maintained for backwards compatibility.

NOTE

Note

For a traditional “Linear” or non-gamma transfer function color space use `VK_COLOR_SPACE_PASS_THROUGH_EXT`.

The presentation engine interprets the pixel values of the R, G, and B components as having been encoded using an appropriate transfer function. Applications **should** ensure that the appropriate transfer function has been applied. Textures Output Format Conversion requires that all implementations implicitly apply the sRGB $EOTF^{-1}$ on R, G, and B components when shaders write to an sRGB pixel format image, which is useful for sRGB color spaces. For sRGB color spaces with other pixel formats, or other non-linear color spaces, applications **can** apply the transfer function explicitly in a shader. The A channel is always interpreted as linearly encoded.

This extension defines enums for VkColorSpaceKHR that correspond to the following color spaces:

Table 45. Color Spaces and Attributes

Name	Red Primary	Green Primary	Blue Primary	White-point	Transfer function
DCI-P3	1.000, 0.000	0.000, 1.000	0.000, 0.000	0.3333, 0.3333	DCI P3
Display-P3	0.680, 0.320	0.265, 0.690	0.150, 0.060	0.3127, 0.3290 (D65)	Display-P3
BT709	0.640, 0.330	0.300, 0.600	0.150, 0.060	0.3127, 0.3290 (D65)	BT709
sRGB	0.640, 0.330	0.300, 0.600	0.150, 0.060	0.3127, 0.3290 (D65)	sRGB
extended sRGB	0.640, 0.330	0.300, 0.600	0.150, 0.060	0.3127, 0.3290 (D65)	scRGB
HDR10_ST2084	0.708, 0.292	0.170, 0.797	0.131, 0.046	0.3127, 0.3290 (D65)	ST2084 PQ
HDR10_HLG	0.708, 0.292	0.170, 0.797	0.131, 0.046	0.3127, 0.3290 (D65)	HLG
Adobe RGB	0.640, 0.330	0.210, 0.710	0.150, 0.060	0.3127, 0.3290 (D65)	Adobe RGB

The transfer functions are described in the “Transfer Functions” chapter of the Khronos Data Format Specification.

Except Display-P3 OETF, which is:

$$E = \begin{cases} 1.055 \times L^{\frac{1}{2.4}} - 0.055 & \text{for } 0.0030186 \leq L \leq 1 \\ 12.92 \times L & \text{for } 0 \leq L < 0.0030186 \end{cases}$$

where L is the linear value of a color component and E is the encoded value (as stored in the image in memory).

NOTE For most uses, the sRGB OETF is equivalent.

32.5.3. Surface Presentation Mode Support

To query the supported presentation modes for a surface, call:

```
// Provided by VK_KHR_surface
VkResult vkGetPhysicalDeviceSurfacePresentModesKHR(
    VkPhysicalDevice                            physicalDevice,
    VkSurfaceKHR                                surface,
    uint32_t*                                   pPresentModeCount,
    VkPresentModeKHR*                           pPresentModes);
```

- `physicalDevice` is the physical device that will be associated with the swapchain to be created, as described for vkCreateSwapchainKHR.
- `surface` is the surface that will be associated with the swapchain.
- `pPresentModeCount` is a pointer to an integer related to the number of presentation modes available or queried, as described below.
- `pPresentModes` is either `NULL` or a pointer to an array of VkPresentModeKHR values, indicating the supported presentation modes.

If `pPresentModes` is `NULL`, then the number of presentation modes supported for the given `surface` is returned in `pPresentModeCount`. Otherwise, `pPresentModeCount` **must** point to a variable set by the application to the number of elements in the `pPresentModes` array, and on return the variable is overwritten with the number of values actually written to `pPresentModes`. If the value of `pPresentModeCount` is less than the number of presentation modes supported, at most `pPresentModeCount` values will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available modes were returned.

Valid Usage

- VUID-vkGetPhysicalDeviceSurfacePresentModesKHR-surface-06524
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfacePresentModesKHR-surface-06525
  `surface` **must** be supported by `physicalDevice`, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSurfacePresentModesKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSurfacePresentModesKHR-surface-parameter
  If `surface` is not VK_NULL_HANDLE, `surface` **must** be a valid VkSurfaceKHR handle

- VUID-vkGetPhysicalDeviceSurfacePresentModesKHR-pPresentModeCount-parameter
  `pPresentModeCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceSurfacePresentModesKHR-pPresentModes-parameter
  If the value referenced by `pPresentModeCount` is not `0`, and `pPresentModes` is not `NULL`, `pPresentModes` **must** be a valid pointer to an array of `pPresentModeCount` VkPresentModeKHR values
- VUID-vkGetPhysicalDeviceSurfacePresentModesKHR-commonparent
  Both of `physicalDevice`, and `surface` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkInstance

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

Alternatively, to query the supported presentation modes for a surface combined with select other fixed swapchain creation parameters, call:

```
// Provided by VK_EXT_full_screen_exclusive
VkResult vkGetPhysicalDeviceSurfacePresentModes2EXT(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceSurfaceInfo2KHR*      pSurfaceInfo,
    uint32_t*                                   pPresentModeCount,
    VkPresentModeKHR*                           pPresentModes);
```

- `physicalDevice` is the physical device that will be associated with the swapchain to be created, as described for vkCreateSwapchainKHR.
- `pSurfaceInfo` is a pointer to a VkPhysicalDeviceSurfaceInfo2KHR structure describing the surface and other fixed parameters that would be consumed by vkCreateSwapchainKHR.
- `pPresentModeCount` is a pointer to an integer related to the number of presentation modes available or queried, as described below.
- `pPresentModes` is either `NULL` or a pointer to an array of VkPresentModeKHR values, indicating the supported presentation modes.

`vkGetPhysicalDeviceSurfacePresentModes2EXT` behaves similarly to vkGetPhysicalDeviceSurfacePresentModesKHR, with the ability to specify extended inputs via chained input structures.

Valid Usage

- VUID-vkGetPhysicalDeviceSurfacePresentModes2EXT-pSurfaceInfo-06521
  `pSurfaceInfo->surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDeviceSurfacePresentModes2EXT-pSurfaceInfo-06522
  `pSurfaceInfo->surface` **must** be supported by `physicalDevice`, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceSurfacePresentModes2EXT-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceSurfacePresentModes2EXT-pSurfaceInfo-parameter
  `pSurfaceInfo` **must** be a valid pointer to a valid VkPhysicalDeviceSurfaceInfo2KHR structure
- VUID-vkGetPhysicalDeviceSurfacePresentModes2EXT-pPresentModeCount-parameter
  `pPresentModeCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceSurfacePresentModes2EXT-pPresentModes-parameter
  If the value referenced by `pPresentModeCount` is not `0`, and `pPresentModes` is not `NULL`, `pPresentModes` **must** be a valid pointer to an array of `pPresentModeCount` VkPresentModeKHR values

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

Possible values of elements of the vkGetPhysicalDeviceSurfacePresentModesKHR::`pPresentModes` array, indicating the supported presentation modes for a surface, are:

```
// Provided by VK_KHR_surface
typedef enum VkPresentModeKHR {
    VK_PRESENT_MODE_IMMEDIATE_KHR = 0,
    VK_PRESENT_MODE_MAILBOX_KHR = 1,
    VK_PRESENT_MODE_FIFO_KHR = 2,
    VK_PRESENT_MODE_FIFO_RELAXED_KHR = 3,
  // Provided by VK_KHR_shared_presentable_image
    VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR = 1000111000,
  // Provided by VK_KHR_shared_presentable_image
    VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR = 1000111001,
  // Provided by VK_EXT_present_mode_fifo_latest_ready
    VK_PRESENT_MODE_FIFO_LATEST_READY_EXT = 1000361000,
} VkPresentModeKHR;
```

- VK_PRESENT_MODE_IMMEDIATE_KHR specifies that the presentation engine does not wait for a vertical blanking period to update the current image, meaning this mode **may** result in visible tearing. No internal queuing of presentation requests is needed, as the requests are applied immediately.
- VK_PRESENT_MODE_MAILBOX_KHR specifies that the presentation engine waits for the next vertical blanking period to update the current image. Tearing **cannot** be observed. An internal single-entry queue is used to hold pending presentation requests. If the queue is full when a new presentation request is received, the new request replaces the existing entry, and any images associated with the prior entry become available for reuse by the application. One request is removed from the queue and processed during each vertical blanking period in which the queue is non-empty.
- VK_PRESENT_MODE_FIFO_KHR specifies that the presentation engine waits for the next vertical blanking period to update the current image. Tearing **cannot** be observed. An internal queue is used to hold pending presentation requests. New requests are appended to the end of the queue, and one request is removed from the beginning of the queue and processed during each vertical blanking period in which the queue is non-empty. This is the only value of presentMode that is **required** to be supported.
- VK_PRESENT_MODE_FIFO_RELAXED_KHR specifies that the presentation engine generally waits for the next vertical blanking period to update the current image. If a vertical blanking period has already passed since the last update of the current image then the presentation engine does not wait for another vertical blanking period for the update, meaning this mode **may** result in visible tearing in this case. This mode is useful for reducing visual stutter with an application that will mostly present a new image before the next vertical blanking period, but may occasionally be late, and present a new image just after the next vertical blanking period. An internal queue is used to hold pending presentation requests. New requests are appended to the end of the queue, and one request is removed from the beginning of the queue and processed during or after each vertical blanking period in which the queue is non-empty.
- VK_PRESENT_MODE_FIFO_LATEST_READY_EXT specifies that the presentation engine waits for the next vertical blanking period to update the current image. Tearing **cannot** be observed. An internal queue is used to hold pending presentation requests. New requests are appended to the end of the queue. At each vertical blanking period, the presentation engine dequeues all successive requests that are ready to be presented from the beginning of the queue. The image of the last

dequeued request will be presented. The other dequeued requests will be dropped.

- VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR specifies that the presentation engine and application have concurrent access to a single image, which is referred to as a *shared presentable image*. The presentation engine is only required to update the current image after a new presentation request is received. Therefore the application **must** make a presentation request whenever an update is required. However, the presentation engine **may** update the current image at any point, meaning this mode **may** result in visible tearing.
- VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR specifies that the presentation engine and application have concurrent access to a single image, which is referred to as a *shared presentable image*. The presentation engine periodically updates the current image on its regular refresh cycle. The application is only required to make one initial presentation request, after which the presentation engine **must** update the current image without any need for further presentation requests. The application **can** indicate the image contents have been updated by making a presentation request, but this does not guarantee the timing of when it will be updated. This mode **may** result in visible tearing if rendering to the image is not timed correctly.

The supported VkImageUsageFlagBits of the presentable images of a swapchain created for a surface **may** differ depending on the presentation mode, and can be determined as per the table below:

Table 46. Presentable Image Usage Queries

Presentation mode	Image usage flags
VK_PRESENT_MODE_IMMEDIATE_KHR	VkSurfaceCapabilitiesKHR::supportedUsageFlags
VK_PRESENT_MODE_MAILBOX_KHR	VkSurfaceCapabilitiesKHR::supportedUsageFlags
VK_PRESENT_MODE_FIFO_KHR	VkSurfaceCapabilitiesKHR::supportedUsageFlags
VK_PRESENT_MODE_FIFO_RELAXED_KHR	VkSurfaceCapabilitiesKHR::supportedUsageFlags
VK_PRESENT_MODE_FIFO_LATEST_READY_EXT	VkSurfaceCapabilitiesKHR::supportedUsageFlags
VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR	VkSharedPresentSurfaceCapabilitiesKHR::sharedPresentSupportedUsageFlags
VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR	VkSharedPresentSurfaceCapabilitiesKHR::sharedPresentSupportedUsageFlags

NOTE For reference, the mode indicated by VK_PRESENT_MODE_FIFO_KHR is equivalent to the behavior of {wgl|glX|egl}SwapBuffers with a swap interval of 1, while the mode indicated by VK_PRESENT_MODE_FIFO_RELAXED_KHR is equivalent to the behavior of {wgl|glX}SwapBuffers with a swap interval of -1 (from the {WGL|GLX}_EXT_swap_control_tear extensions).

32.6. Full Screen Exclusive Control

Swapchains created with fullScreenExclusive set to VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT **must** acquire and release exclusive full-screen access explicitly, using the following commands.

To acquire exclusive full-screen access for a swapchain, call:

```
// Provided by VK_EXT_full_screen_exclusive
VkResult vkAcquireFullScreenExclusiveModeEXT(
    VkDevice                                    device,
    VkSwapchainKHR                              swapchain);
```

- `device` is the device associated with `swapchain`.
- `swapchain` is the swapchain to acquire exclusive full-screen access for.

Valid Usage

- VUID-vkAcquireFullScreenExclusiveModeEXT-swapchain-02674
  `swapchain` **must** not be in the retired state
- VUID-vkAcquireFullScreenExclusiveModeEXT-swapchain-02675
  `swapchain` **must** be a swapchain created with a VkSurfaceFullScreenExclusiveInfoEXT structure, with `fullScreenExclusive` set to `VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT`
- VUID-vkAcquireFullScreenExclusiveModeEXT-swapchain-02676
  `swapchain` **must** not currently have exclusive full-screen access

A return value of `VK_SUCCESS` indicates that the `swapchain` successfully acquired exclusive full-screen access. The swapchain will retain this exclusivity until either the application releases exclusive full-screen access with vkReleaseFullScreenExclusiveModeEXT, destroys the swapchain, or if any of the swapchain commands return `VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT` indicating that the mode was lost because of platform-specific changes.

If the swapchain was unable to acquire exclusive full-screen access to the display then `VK_ERROR_INITIALIZATION_FAILED` is returned. An application **can** attempt to acquire exclusive full-screen access again for the same swapchain even if this command fails, or if `VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT` has been returned by a swapchain command.

Valid Usage (Implicit)

- VUID-vkAcquireFullScreenExclusiveModeEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkAcquireFullScreenExclusiveModeEXT-swapchain-parameter
  `swapchain` **must** be a valid VkSwapchainKHR handle
- VUID-vkAcquireFullScreenExclusiveModeEXT-swapchain-parent
  `swapchain` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INITIALIZATION_FAILED
- VK_ERROR_SURFACE_LOST_KHR

To release exclusive full-screen access from a swapchain, call:

```
// Provided by VK_EXT_full_screen_exclusive
VkResult vkReleaseFullScreenExclusiveModeEXT(
    VkDevice                                    device,
    VkSwapchainKHR                              swapchain);
```

- device is the device associated with swapchain.
- swapchain is the swapchain to release exclusive full-screen access from.

NOTE Applications will not be able to present to swapchain after this call until exclusive full-screen access is reacquired. This is usually useful to handle when an application is minimized or otherwise intends to stop presenting for a time.

Valid Usage

- VUID-vkReleaseFullScreenExclusiveModeEXT-swapchain-02677
  swapchain **must** not be in the retired state
- VUID-vkReleaseFullScreenExclusiveModeEXT-swapchain-02678
  swapchain **must** be a swapchain created with a VkSurfaceFullScreenExclusiveInfoEXT structure, with fullScreenExclusive set to VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT

Valid Usage (Implicit)

- VUID-vkReleaseFullScreenExclusiveModeEXT-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkReleaseFullScreenExclusiveModeEXT-swapchain-parameter
  swapchain **must** be a valid VkSwapchainKHR handle
- VUID-vkReleaseFullScreenExclusiveModeEXT-swapchain-parent
  swapchain **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_SURFACE_LOST_KHR

32.7. Device Group Queries

A logical device that represents multiple physical devices **may** support presenting from images on more than one physical device, or combining images from multiple physical devices.

To query these capabilities, call:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_surface
VkResult vkGetDeviceGroupPresentCapabilitiesKHR(
    VkDevice                                    device,
    VkDeviceGroupPresentCapabilitiesKHR*        pDeviceGroupPresentCapabilities);
```

- device is the logical device.
- pDeviceGroupPresentCapabilities is a pointer to a VkDeviceGroupPresentCapabilitiesKHR structure in which the device's capabilities are returned.

Valid Usage (Implicit)

- VUID-vkGetDeviceGroupPresentCapabilitiesKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetDeviceGroupPresentCapabilitiesKHR-pDeviceGroupPresentCapabilities-parameter
  pDeviceGroupPresentCapabilities **must** be a valid pointer to a VkDeviceGroupPresentCapabilitiesKHR structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY

- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkDeviceGroupPresentCapabilitiesKHR structure is defined as:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_surface
typedef struct VkDeviceGroupPresentCapabilitiesKHR {
    VkStructureType                      sType;
    void*                                pNext;
    uint32_t                             presentMask[VK_MAX_DEVICE_GROUP_SIZE];
    VkDeviceGroupPresentModeFlagsKHR     modes;
} VkDeviceGroupPresentCapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- presentMask is an array of VK_MAX_DEVICE_GROUP_SIZE uint32_t masks, where the mask at element i is non-zero if physical device i has a presentation engine, and where bit j is set in element i if physical device i **can** present swapchain images from physical device j. If element i is non-zero, then bit i **must** be set.
- modes is a bitmask of VkDeviceGroupPresentModeFlagBitsKHR indicating which device group presentation modes are supported.

modes always has VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_BIT_KHR set.

The present mode flags are also used when presenting an image, in VkDeviceGroupPresentInfoKHR::mode.

If a device group only includes a single physical device, then modes **must** equal VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_BIT_KHR.

Valid Usage (Implicit)

- VUID-VkDeviceGroupPresentCapabilitiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_CAPABILITIES_KHR
- VUID-VkDeviceGroupPresentCapabilitiesKHR-pNext-pNext
  pNext **must** be NULL

Bits which **may** be set in VkDeviceGroupPresentCapabilitiesKHR::modes, indicating which device group presentation modes are supported, are:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_surface
typedef enum VkDeviceGroupPresentModeFlagBitsKHR {
    VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_BIT_KHR = 0x00000001,
    VK_DEVICE_GROUP_PRESENT_MODE_REMOTE_BIT_KHR = 0x00000002,
    VK_DEVICE_GROUP_PRESENT_MODE_SUM_BIT_KHR = 0x00000004,
    VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_MULTI_DEVICE_BIT_KHR = 0x00000008,
} VkDeviceGroupPresentModeFlagBitsKHR;
```

- `VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_BIT_KHR` specifies that any physical device with a presentation engine **can** present its own swapchain images.
- `VK_DEVICE_GROUP_PRESENT_MODE_REMOTE_BIT_KHR` specifies that any physical device with a presentation engine **can** present swapchain images from any physical device in its `presentMask`.
- `VK_DEVICE_GROUP_PRESENT_MODE_SUM_BIT_KHR` specifies that any physical device with a presentation engine **can** present the sum of swapchain images from any physical devices in its `presentMask`.
- `VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_MULTI_DEVICE_BIT_KHR` specifies that multiple physical devices with a presentation engine **can** each present their own swapchain images.

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_surface
typedef VkFlags VkDeviceGroupPresentModeFlagsKHR;
```

`VkDeviceGroupPresentModeFlagsKHR` is a bitmask type for setting a mask of zero or more VkDeviceGroupPresentModeFlagBitsKHR.

Some surfaces **may** not be capable of using all the device group present modes.

To query the supported device group present modes for a particular surface, call:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_surface
VkResult vkGetDeviceGroupSurfacePresentModesKHR(
    VkDevice                                    device,
    VkSurfaceKHR                                surface,
    VkDeviceGroupPresentModeFlagsKHR*           pModes);
```

- `device` is the logical device.
- `surface` is the surface.
- `pModes` is a pointer to a VkDeviceGroupPresentModeFlagsKHR in which the supported device group present modes for the surface are returned.

The modes returned by this command are not invariant, and **may** change in response to the surface being moved, resized, or occluded. These modes **must** be a subset of the modes returned by vkGetDeviceGroupPresentCapabilitiesKHR.

Valid Usage

- VUID-vkGetDeviceGroupSurfacePresentModesKHR-surface-06212
  `surface` **must** be supported by all physical devices associated with `device`, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetDeviceGroupSurfacePresentModesKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceGroupSurfacePresentModesKHR-surface-parameter
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetDeviceGroupSurfacePresentModesKHR-pModes-parameter
  `pModes` **must** be a valid pointer to a VkDeviceGroupPresentModeFlagsKHR value
- VUID-vkGetDeviceGroupSurfacePresentModesKHR-commonparent
  Both of `device`, and `surface` **must** have been created, allocated, or retrieved from the same VkInstance

Host Synchronization

- Host access to `surface` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

Alternatively, to query the supported device group presentation modes for a surface combined with select other fixed swapchain creation parameters, call:

```
// Provided by VK_EXT_full_screen_exclusive with VK_KHR_device_group or VK_VERSION_1_1
VkResult vkGetDeviceGroupSurfacePresentModes2EXT(
    VkDevice                                    device,
    const VkPhysicalDeviceSurfaceInfo2KHR*      pSurfaceInfo,
    VkDeviceGroupPresentModeFlagsKHR*           pModes);
```

- `device` is the logical device.
- `pSurfaceInfo` is a pointer to a VkPhysicalDeviceSurfaceInfo2KHR structure describing the surface and other fixed parameters that would be consumed by vkCreateSwapchainKHR.
- `pModes` is a pointer to a VkDeviceGroupPresentModeFlagsKHR in which the supported device group present modes for the surface are returned.

`vkGetDeviceGroupSurfacePresentModes2EXT` behaves similarly to vkGetDeviceGroupSurfacePresentModesKHR, with the ability to specify extended inputs via chained input structures.

Valid Usage

- VUID-vkGetDeviceGroupSurfacePresentModes2EXT-pSurfaceInfo-06213
  `pSurfaceInfo->surface` **must** be supported by all physical devices associated with `device`, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetDeviceGroupSurfacePresentModes2EXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceGroupSurfacePresentModes2EXT-pSurfaceInfo-parameter
  `pSurfaceInfo` **must** be a valid pointer to a valid VkPhysicalDeviceSurfaceInfo2KHR structure
- VUID-vkGetDeviceGroupSurfacePresentModes2EXT-pModes-parameter
  `pModes` **must** be a valid pointer to a VkDeviceGroupPresentModeFlagsKHR value

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_SURFACE_LOST_KHR`

When using `VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_MULTI_DEVICE_BIT_KHR`, the application **may** need to know which regions of the surface are used when presenting locally on each physical device. Presentation of swapchain images to this surface need only have valid contents in the regions returned by this command.

To query a set of rectangles used in presentation on the physical device, call:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_surface
VkResult vkGetPhysicalDevicePresentRectanglesKHR(
    VkPhysicalDevice                            physicalDevice,
    VkSurfaceKHR                                surface,
    uint32_t*                                   pRectCount,
    VkRect2D*                                   pRects);
```

- `physicalDevice` is the physical device.
- `surface` is the surface.
- `pRectCount` is a pointer to an integer related to the number of rectangles available or queried, as described below.
- `pRects` is either `NULL` or a pointer to an array of VkRect2D structures.

If `pRects` is `NULL`, then the number of rectangles used when presenting the given `surface` is returned in `pRectCount`. Otherwise, `pRectCount` **must** point to a variable set by the application to the number of elements in the `pRects` array, and on return the variable is overwritten with the number of structures actually written to `pRects`. If the value of `pRectCount` is less than the number of rectangles, at most `pRectCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available rectangles were returned.

The values returned by this command are not invariant, and **may** change in response to the surface being moved, resized, or occluded.

The rectangles returned by this command **must** not overlap.

Valid Usage

- VUID-vkGetPhysicalDevicePresentRectanglesKHR-surface-06523
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDevicePresentRectanglesKHR-surface-06211
  `surface` **must** be supported by `physicalDevice`, as reported by vkGetPhysicalDeviceSurfaceSupportKHR or an equivalent platform-specific mechanism

Valid Usage (Implicit)

- VUID-vkGetPhysicalDevicePresentRectanglesKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDevicePresentRectanglesKHR-surface-parameter
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-vkGetPhysicalDevicePresentRectanglesKHR-pRectCount-parameter
  `pRectCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDevicePresentRectanglesKHR-pRects-parameter
  If the value referenced by `pRectCount` is not `0`, and `pRects` is not `NULL`, `pRects` **must** be a

valid pointer to an array of pRectCount VkRect2D structures

- VUID-vkGetPhysicalDevicePresentRectanglesKHR-commonparent
  Both of physicalDevice, and surface **must** have been created, allocated, or retrieved from the same VkInstance

Host Synchronization

- Host access to surface **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

32.8. Present Wait

Applications wanting to control the pacing of the application by monitoring when presentation processes have completed to limit the number of outstanding images queued for presentation, need to have a method of being signaled during the presentation process.

Providing a mechanism which allows applications to block, waiting for a specific step of the presentation process to complete allows them to control the amount of outstanding work (and hence the potential lag in responding to user input or changes in the rendering environment).

The VK_KHR_present_wait extension allows applications to tell the presentation engine at the vkQueuePresentKHR call that it plans on waiting for presentation by passing a VkPresentIdKHR structure. The presentId passed in that structure may then be passed to a future vkWaitForPresentKHR call to cause the application to block until that presentation is finished.

32.9. WSI Swapchain

A swapchain object (a.k.a. swapchain) provides the ability to present rendering results to a surface. Swapchain objects are represented by VkSwapchainKHR handles:

```
// Provided by VK_KHR_swapchain
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkSwapchainKHR)
```

A swapchain is an abstraction for an array of presentable images that are associated with a surface.

The presentable images are represented by VkImage objects created by the platform. One image (which **can** be an array image for multiview/stereoscopic-3D surfaces) is displayed at a time, but multiple images **can** be queued for presentation. An application renders to the image, and then queues the image for presentation to the surface.

A native window **cannot** be associated with more than one non-retired swapchain at a time. Further, swapchains **cannot** be created for native windows that have a non-Vulkan graphics API surface associated with them.

NOTE

The presentation engine is an abstraction for the platform's compositor or display engine.

The presentation engine **may** be synchronous or asynchronous with respect to the application and/or logical device.

Some implementations **may** use the device's graphics queue or dedicated presentation hardware to perform presentation.

The presentable images of a swapchain are owned by the presentation engine. An application **can** acquire use of a presentable image from the presentation engine. Use of a presentable image **must** occur only after the image is returned by vkAcquireNextImageKHR, and before it is released by vkQueuePresentKHR. This includes transitioning the image layout and rendering commands.

An application **can** acquire use of a presentable image with vkAcquireNextImageKHR. After acquiring a presentable image and before modifying it, the application **must** use a synchronization primitive to ensure that the presentation engine has finished reading from the image. The application **can** then transition the image's layout, queue rendering commands to it, etc. Finally, the application presents the image with vkQueuePresentKHR, which releases the acquisition of the image.

The presentation engine controls the order in which presentable images are acquired for use by the application.

NOTE

This allows the platform to handle situations which require out-of-order return of images after presentation. At the same time, it allows the application to generate command buffers referencing all of the images in the swapchain at initialization time, rather than in its main loop.

How this all works is described below.

If a swapchain is created with presentMode set to either VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR or VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR, a single presentable image **can** be acquired, referred to as a shared presentable image. A shared presentable image **may** be concurrently accessed by the application and the presentation engine, without transitioning the image's layout after it is initially presented.

- With VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR, the presentation engine is only required to update to the latest contents of a shared presentable image after a present. The application **must** call vkQueuePresentKHR to guarantee an update. However, the presentation engine **may**

update from it at any time.

- With VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR, the presentation engine will automatically present the latest contents of a shared presentable image during every refresh cycle. The application is only required to make one initial call to vkQueuePresentKHR, after which the presentation engine will update from it without any need for further present calls. The application **can** indicate the image contents have been updated by calling vkQueuePresentKHR, but this does not guarantee the timing of when updates will occur.

The presentation engine **may** access a shared presentable image at any time after it is first presented. To avoid tearing, an application **should** coordinate access with the presentation engine. This requires presentation engine timing information through platform-specific mechanisms and ensuring that color attachment writes are made available during the portion of the presentation engine's refresh cycle they are intended for.

NOTE	The VK_KHR_shared_presentable_image extension does not provide functionality for determining the timing of the presentation engine's refresh cycles.

In order to query a swapchain's status when rendering to a shared presentable image, call:

```
// Provided by VK_KHR_shared_presentable_image
VkResult vkGetSwapchainStatusKHR(
    VkDevice                                    device,
    VkSwapchainKHR                              swapchain);
```

- device is the device associated with swapchain.
- swapchain is the swapchain to query.

Valid Usage (Implicit)

- VUID-vkGetSwapchainStatusKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetSwapchainStatusKHR-swapchain-parameter
  swapchain **must** be a valid VkSwapchainKHR handle
- VUID-vkGetSwapchainStatusKHR-swapchain-parent
  swapchain **must** have been created, allocated, or retrieved from device

Host Synchronization

- Host access to swapchain **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS
- VK_SUBOPTIMAL_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST
- VK_ERROR_OUT_OF_DATE_KHR
- VK_ERROR_SURFACE_LOST_KHR
- VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT

The possible return values for vkGetSwapchainStatusKHR **should** be interpreted as follows:

- VK_SUCCESS specifies the presentation engine is presenting the contents of the shared presentable image, as per the swapchain's VkPresentModeKHR.
- VK_SUBOPTIMAL_KHR the swapchain no longer matches the surface properties exactly, but the presentation engine is presenting the contents of the shared presentable image, as per the swapchain's VkPresentModeKHR.
- VK_ERROR_OUT_OF_DATE_KHR the surface has changed in such a way that it is no longer compatible with the swapchain.
- VK_ERROR_SURFACE_LOST_KHR the surface is no longer available.

NOTE The swapchain state **may** be cached by implementations, so applications **should** regularly call vkGetSwapchainStatusKHR when using a swapchain with VkPresentModeKHR equal to VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR.

To create a swapchain, call:

```
// Provided by VK_KHR_swapchain
VkResult vkCreateSwapchainKHR(
    VkDevice                                    device,
    const VkSwapchainCreateInfoKHR*             pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkSwapchainKHR*                             pSwapchain);
```

- device is the device to create the swapchain for.
- pCreateInfo is a pointer to a VkSwapchainCreateInfoKHR structure specifying the parameters of the created swapchain.
- pAllocator is the allocator used for host memory allocated for the swapchain object when there

is no more specific allocator available (see Memory Allocation).

- pSwapchain is a pointer to a VkSwapchainKHR handle in which the created swapchain object will be returned.

As mentioned above, if vkCreateSwapchainKHR succeeds, it will return a handle to a swapchain containing an array of at least pCreateInfo->minImageCount presentable images.

While acquired by the application, presentable images **can** be used in any way that equivalent non-presentable images **can** be used. A presentable image is equivalent to a non-presentable image created with the following VkImageCreateInfo parameters:

VkImageCreateInfo **Field**	**Value**
flags	VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT is set if VK_SWAPCHAIN_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR is set VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT and VK_IMAGE_CREATE_EXTENDED_USAGE_BIT_KHR are both set if VK_SWAPCHAIN_CREATE_MUTABLE_FORMAT_BIT_KHR is set all other bits are unset
imageType	VK_IMAGE_TYPE_2D
format	pCreateInfo->imageFormat
extent	{pCreateInfo->imageExtent.width, pCreateInfo->imageExtent.height, 1}
mipLevels	1
arrayLayers	pCreateInfo->imageArrayLayers
samples	VK_SAMPLE_COUNT_1_BIT
tiling	VK_IMAGE_TILING_OPTIMAL
usage	pCreateInfo->imageUsage
sharingMode	pCreateInfo->imageSharingMode
queueFamilyIndexCount	pCreateInfo->queueFamilyIndexCount
pQueueFamilyIndices	pCreateInfo->pQueueFamilyIndices
initialLayout	VK_IMAGE_LAYOUT_UNDEFINED

The pCreateInfo->surface **must** not be destroyed until after the swapchain is destroyed.

If oldSwapchain is VK_NULL_HANDLE, and the native window referred to by pCreateInfo->surface is already associated with a Vulkan swapchain, VK_ERROR_NATIVE_WINDOW_IN_USE_KHR **must** be returned.

If the native window referred to by pCreateInfo->surface is already associated with a non-Vulkan graphics API surface, VK_ERROR_NATIVE_WINDOW_IN_USE_KHR **must** be returned.

The native window referred to by `pCreateInfo->surface` **must** not become associated with a non-Vulkan graphics API surface before all associated Vulkan swapchains have been destroyed.

`vkCreateSwapchainKHR` will return `VK_ERROR_DEVICE_LOST` if the logical device was lost. The `VkSwapchainKHR` is a child of the `device`, and **must** be destroyed before the `device`. However, `VkSurfaceKHR` is not a child of any `VkDevice` and is not affected by the lost device. After successfully recreating a `VkDevice`, the same `VkSurfaceKHR` **can** be used to create a new `VkSwapchainKHR`, provided the previous one was destroyed.

If the `oldSwapchain` parameter of `pCreateInfo` is a valid swapchain, which has exclusive full-screen access, that access is released from `pCreateInfo->oldSwapchain`. If the command succeeds in this case, the newly created swapchain will automatically acquire exclusive full-screen access from `pCreateInfo->oldSwapchain`.

NOTE This implicit transfer is intended to avoid exiting and entering full-screen exclusive mode, which may otherwise cause unwanted visual updates to the display.

In some cases, swapchain creation **may** fail if exclusive full-screen mode is requested for application control, but for some implementation-specific reason exclusive full-screen access is unavailable for the particular combination of parameters provided. If this occurs, `VK_ERROR_INITIALIZATION_FAILED` will be returned.

NOTE In particular, it will fail if the `imageExtent` member of `pCreateInfo` does not match the extents of the monitor. Other reasons for failure may include the application not being set as high-dpi aware, or if the physical device and monitor are not compatible in this mode.

Valid Usage (Implicit)

- VUID-vkCreateSwapchainKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateSwapchainKHR-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkSwapchainCreateInfoKHR structure
- VUID-vkCreateSwapchainKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateSwapchainKHR-pSwapchain-parameter
  `pSwapchain` **must** be a valid pointer to a VkSwapchainKHR handle

Host Synchronization

- Host access to `pCreateInfo->surface` **must** be externally synchronized
- Host access to `pCreateInfo->oldSwapchain` **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST
- VK_ERROR_SURFACE_LOST_KHR
- VK_ERROR_NATIVE_WINDOW_IN_USE_KHR
- VK_ERROR_INITIALIZATION_FAILED

The VkSwapchainCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_swapchain
typedef struct VkSwapchainCreateInfoKHR {
    VkStructureType                  sType;
    const void*                      pNext;
    VkSwapchainCreateFlagsKHR        flags;
    VkSurfaceKHR                     surface;
    uint32_t                         minImageCount;
    VkFormat                         imageFormat;
    VkColorSpaceKHR                  imageColorSpace;
    VkExtent2D                       imageExtent;
    uint32_t                         imageArrayLayers;
    VkImageUsageFlags                imageUsage;
    VkSharingMode                    imageSharingMode;
    uint32_t                         queueFamilyIndexCount;
    const uint32_t*                  pQueueFamilyIndices;
    VkSurfaceTransformFlagBitsKHR    preTransform;
    VkCompositeAlphaFlagBitsKHR      compositeAlpha;
    VkPresentModeKHR                 presentMode;
    VkBool32                         clipped;
    VkSwapchainKHR                   oldSwapchain;
} VkSwapchainCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkSwapchainCreateFlagBitsKHR indicating parameters of the swapchain creation.
- surface is the surface onto which the swapchain will present images. If the creation succeeds, the swapchain becomes associated with surface.
- minImageCount is the minimum number of presentable images that the application needs. The

implementation will either create the swapchain with at least that many images, or it will fail to create the swapchain.

- `imageFormat` is a VkFormat value specifying the format the swapchain image(s) will be created with.
- `imageColorSpace` is a VkColorSpaceKHR value specifying the way the swapchain interprets image data.
- `imageExtent` is the size (in pixels) of the swapchain image(s). The behavior is platform-dependent if the image extent does not match the surface's `currentExtent` as returned by `vkGetPhysicalDeviceSurfaceCapabilitiesKHR`.

> **NOTE** On some platforms, it is normal that `maxImageExtent` **may** become `(0, 0)`, for example when the window is minimized. In such a case, it is not possible to create a swapchain due to the Valid Usage requirements .

- `imageArrayLayers` is the number of views in a multiview/stereo surface. For non-stereoscopic-3D applications, this value is 1.
- `imageUsage` is a bitmask of VkImageUsageFlagBits describing the intended usage of the (acquired) swapchain images.
- `imageSharingMode` is the sharing mode used for the image(s) of the swapchain.
- `queueFamilyIndexCount` is the number of queue families having access to the image(s) of the swapchain when `imageSharingMode` is `VK_SHARING_MODE_CONCURRENT`.
- `pQueueFamilyIndices` is a pointer to an array of queue family indices having access to the images(s) of the swapchain when `imageSharingMode` is `VK_SHARING_MODE_CONCURRENT`.
- `preTransform` is a VkSurfaceTransformFlagBitsKHR value describing the transform, relative to the presentation engine's natural orientation, applied to the image content prior to presentation. If it does not match the `currentTransform` value returned by `vkGetPhysicalDeviceSurfaceCapabilitiesKHR`, the presentation engine will transform the image content as part of the presentation operation.
- `compositeAlpha` is a VkCompositeAlphaFlagBitsKHR value indicating the alpha compositing mode to use when this surface is composited together with other surfaces on certain window systems.
- `presentMode` is the presentation mode the swapchain will use. A swapchain's present mode determines how incoming present requests will be processed and queued internally.
- `clipped` specifies whether the Vulkan implementation is allowed to discard rendering operations that affect regions of the surface that are not visible.
  - If `clipped` is `VK_TRUE`, the presentable images associated with the swapchain **may** not own all of their pixels. Pixels in the presentable images that correspond to regions of the target surface obscured by another window on the desktop, or subject to some other clipping mechanism will have undefined content when read back. Fragment shaders **may** not execute for these pixels, and thus any side effects they would have had will not occur. Setting `VK_TRUE` does not guarantee any clipping will occur, but allows more efficient presentation methods to be used on some platforms.
  - If `clipped` is `VK_FALSE`, presentable images associated with the swapchain will own all of the pixels they contain.

NOTE

Applications **should** set this value to VK_TRUE if they do not expect to read back the content of presentable images before presenting them or after reacquiring them, and if their fragment shaders do not have any side effects that require them to run for all pixels in the presentable image.

- oldSwapchain is VK_NULL_HANDLE, or the existing non-retired swapchain currently associated with surface. Providing a valid oldSwapchain **may** aid in the resource reuse, and also allows the application to still present any images that are already acquired from it.

Upon calling vkCreateSwapchainKHR with an oldSwapchain that is not VK_NULL_HANDLE, oldSwapchain is retired — even if creation of the new swapchain fails. The new swapchain is created in the non-retired state whether or not oldSwapchain is VK_NULL_HANDLE.

Upon calling vkCreateSwapchainKHR with an oldSwapchain that is not VK_NULL_HANDLE, any images from oldSwapchain that are not acquired by the application **may** be freed by the implementation, which **may** occur even if creation of the new swapchain fails. The application **can** destroy oldSwapchain to free all memory associated with oldSwapchain.

NOTE

Multiple retired swapchains **can** be associated with the same VkSurfaceKHR through multiple uses of oldSwapchain that outnumber calls to vkDestroySwapchainKHR.

After oldSwapchain is retired, the application **can** pass to vkQueuePresentKHR any images it had already acquired from oldSwapchain. E.g., an application may present an image from the old swapchain before an image from the new swapchain is ready to be presented. As usual, vkQueuePresentKHR **may** fail if oldSwapchain has entered a state that causes VK_ERROR_OUT_OF_DATE_KHR to be returned.

The application **can** continue to use a shared presentable image obtained from oldSwapchain until a presentable image is acquired from the new swapchain, as long as it has not entered a state that causes it to return VK_ERROR_OUT_OF_DATE_KHR.

Valid Usage

- VUID-VkSwapchainCreateInfoKHR-surface-01270
  surface **must** be a surface that is supported by the device as determined using vkGetPhysicalDeviceSurfaceSupportKHR
- VUID-VkSwapchainCreateInfoKHR-minImageCount-01272
  minImageCount **must** be less than or equal to the value returned in the maxImageCount member of the VkSurfaceCapabilitiesKHR structure returned by vkGetPhysicalDeviceSurfaceCapabilitiesKHR for the surface if the returned maxImageCount is not zero
- VUID-VkSwapchainCreateInfoKHR-presentMode-02839
  If presentMode is not VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR nor VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR, then minImageCount **must** be greater than or equal to the value returned in the minImageCount member of the VkSurfaceCapabilitiesKHR structure returned by vkGetPhysicalDeviceSurfaceCapabilitiesKHR for the surface

- VUID-VkSwapchainCreateInfoKHR-minImageCount-01383
  `minImageCount` **must** be `1` if `presentMode` is either `VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR` or `VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR`
- VUID-VkSwapchainCreateInfoKHR-imageFormat-01273
  `imageFormat` and `imageColorSpace` **must** match the `format` and `colorSpace` members, respectively, of one of the `VkSurfaceFormatKHR` structures returned by `vkGetPhysicalDeviceSurfaceFormatsKHR` for the surface
- VUID-VkSwapchainCreateInfoKHR-pNext-07781
  `imageExtent` **must** be between `minImageExtent` and `maxImageExtent`, inclusive, where `minImageExtent` and `maxImageExtent` are members of the `VkSurfaceCapabilitiesKHR` structure returned by `vkGetPhysicalDeviceSurfaceCapabilitiesKHR` for the surface
- VUID-VkSwapchainCreateInfoKHR-imageExtent-01689
  `imageExtent` members `width` and `height` **must** both be non-zero
- VUID-VkSwapchainCreateInfoKHR-imageArrayLayers-01275
  `imageArrayLayers` **must** be greater than `0` and less than or equal to the `maxImageArrayLayers` member of the `VkSurfaceCapabilitiesKHR` structure returned by `vkGetPhysicalDeviceSurfaceCapabilitiesKHR` for the surface
- VUID-VkSwapchainCreateInfoKHR-presentMode-01427
  If `presentMode` is `VK_PRESENT_MODE_FIFO_LATEST_READY_EXT`, `VK_PRESENT_MODE_IMMEDIATE_KHR`, `VK_PRESENT_MODE_MAILBOX_KHR`, `VK_PRESENT_MODE_FIFO_KHR` or `VK_PRESENT_MODE_FIFO_RELAXED_KHR`, `imageUsage` **must** be a subset of the supported usage flags present in the `supportedUsageFlags` member of the VkSurfaceCapabilitiesKHR structure returned by vkGetPhysicalDeviceSurfaceCapabilitiesKHR for `surface`
- VUID-VkSwapchainCreateInfoKHR-imageUsage-01384
  If `presentMode` is `VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR` or `VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR`, `imageUsage` **must** be a subset of the supported usage flags present in the `sharedPresentSupportedUsageFlags` member of the VkSharedPresentSurfaceCapabilitiesKHR structure returned by vkGetPhysicalDeviceSurfaceCapabilities2KHR for `surface`
- VUID-VkSwapchainCreateInfoKHR-imageSharingMode-01277
  If `imageSharingMode` is `VK_SHARING_MODE_CONCURRENT`, `pQueueFamilyIndices` **must** be a valid pointer to an array of `queueFamilyIndexCount` `uint32_t` values
- VUID-VkSwapchainCreateInfoKHR-imageSharingMode-01278
  If `imageSharingMode` is `VK_SHARING_MODE_CONCURRENT`, `queueFamilyIndexCount` **must** be greater than `1`
- VUID-VkSwapchainCreateInfoKHR-imageSharingMode-01428
  If `imageSharingMode` is `VK_SHARING_MODE_CONCURRENT`, each element of `pQueueFamilyIndices` **must** be unique and **must** be less than `pQueueFamilyPropertyCount` returned by either vkGetPhysicalDeviceQueueFamilyProperties or vkGetPhysicalDeviceQueueFamilyProperties2KHR for the `physicalDevice` that was used to create `device`
- VUID-VkSwapchainCreateInfoKHR-preTransform-01279
  `preTransform` **must** be one of the bits present in the `supportedTransforms` member of the

VkSurfaceCapabilitiesKHR structure returned by vkGetPhysicalDeviceSurfaceCapabilitiesKHR for the surface

- VUID-VkSwapchainCreateInfoKHR-compositeAlpha-01280
  compositeAlpha **must** be one of the bits present in the supportedCompositeAlpha member of the VkSurfaceCapabilitiesKHR structure returned by vkGetPhysicalDeviceSurfaceCapabilitiesKHR for the surface
- VUID-VkSwapchainCreateInfoKHR-presentMode-01281
  presentMode **must** be one of the VkPresentModeKHR values returned by vkGetPhysicalDeviceSurfacePresentModesKHR for the surface
- VUID-VkSwapchainCreateInfoKHR-presentModeFifoLatestReady-10161
  If the presentModeFifoLatestReady feature is not enabled, presentMode **must** not be VK_PRESENT_MODE_FIFO_LATEST_READY_EXT
- VUID-VkSwapchainCreateInfoKHR-physicalDeviceCount-01429
  If the logical device was created with VkDeviceGroupDeviceCreateInfo ::physicalDeviceCount equal to 1, flags **must** not contain VK_SWAPCHAIN_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR
- VUID-VkSwapchainCreateInfoKHR-oldSwapchain-01933
  If oldSwapchain is not VK_NULL_HANDLE, oldSwapchain **must** be a non-retired swapchain associated with native window referred to by surface
- VUID-VkSwapchainCreateInfoKHR-imageFormat-01778
  The implied image creation parameters of the swapchain **must** be supported as reported by vkGetPhysicalDeviceImageFormatProperties
- VUID-VkSwapchainCreateInfoKHR-flags-03168
  If flags contains VK_SWAPCHAIN_CREATE_MUTABLE_FORMAT_BIT_KHR then the pNext chain **must** include a VkImageFormatListCreateInfo structure with a viewFormatCount greater than zero and pViewFormats **must** have an element equal to imageFormat
- VUID-VkSwapchainCreateInfoKHR-pNext-04099
  If a VkImageFormatListCreateInfo structure was included in the pNext chain and VkImageFormatListCreateInfo::viewFormatCount is not zero then all of the formats in VkImageFormatListCreateInfo::pViewFormats **must** be compatible with the format as described in the compatibility table
- VUID-VkSwapchainCreateInfoKHR-flags-04100
  If flags does not contain VK_SWAPCHAIN_CREATE_MUTABLE_FORMAT_BIT_KHR and the pNext chain include a VkImageFormatListCreateInfo structure then VkImageFormatListCreateInfo ::viewFormatCount **must** be 0 or 1
- VUID-VkSwapchainCreateInfoKHR-flags-03187
  If flags contains VK_SWAPCHAIN_CREATE_PROTECTED_BIT_KHR, then VkSurfaceProtectedCapabilitiesKHR::supportsProtected **must** be VK_TRUE in the VkSurfaceProtectedCapabilitiesKHR structure returned by vkGetPhysicalDeviceSurfaceCapabilities2KHR for surface
- VUID-VkSwapchainCreateInfoKHR-pNext-02679
  If the pNext chain includes a VkSurfaceFullScreenExclusiveInfoEXT structure with its fullScreenExclusive member set to VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT,

and `surface` was created using vkCreateWin32SurfaceKHR, a VkSurfaceFullScreenExclusiveWin32InfoEXT structure **must** be included in the `pNext` chain

Valid Usage (Implicit)

- VUID-VkSwapchainCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR`
- VUID-VkSwapchainCreateInfoKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkDeviceGroupSwapchainCreateInfoKHR, VkImageFormatListCreateInfo, VkSurfaceFullScreenExclusiveInfoEXT, VkSurfaceFullScreenExclusiveWin32InfoEXT, or VkSwapchainCounterCreateInfoEXT
- VUID-VkSwapchainCreateInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkSwapchainCreateInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkSwapchainCreateFlagBitsKHR values
- VUID-VkSwapchainCreateInfoKHR-surface-parameter
  `surface` **must** be a valid VkSurfaceKHR handle
- VUID-VkSwapchainCreateInfoKHR-imageFormat-parameter
  `imageFormat` **must** be a valid VkFormat value
- VUID-VkSwapchainCreateInfoKHR-imageColorSpace-parameter
  `imageColorSpace` **must** be a valid VkColorSpaceKHR value
- VUID-VkSwapchainCreateInfoKHR-imageUsage-parameter
  `imageUsage` **must** be a valid combination of VkImageUsageFlagBits values
- VUID-VkSwapchainCreateInfoKHR-imageUsage-requiredbitmask
  `imageUsage` **must** not be `0`
- VUID-VkSwapchainCreateInfoKHR-imageSharingMode-parameter
  `imageSharingMode` **must** be a valid VkSharingMode value
- VUID-VkSwapchainCreateInfoKHR-preTransform-parameter
  `preTransform` **must** be a valid VkSurfaceTransformFlagBitsKHR value
- VUID-VkSwapchainCreateInfoKHR-compositeAlpha-parameter
  `compositeAlpha` **must** be a valid VkCompositeAlphaFlagBitsKHR value
- VUID-VkSwapchainCreateInfoKHR-presentMode-parameter
  `presentMode` **must** be a valid VkPresentModeKHR value
- VUID-VkSwapchainCreateInfoKHR-oldSwapchain-parameter
  If `oldSwapchain` is not VK_NULL_HANDLE, `oldSwapchain` **must** be a valid VkSwapchainKHR handle
- VUID-VkSwapchainCreateInfoKHR-commonparent
  Both of `oldSwapchain`, and `surface` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkInstance

Bits which **can** be set in VkSwapchainCreateInfoKHR::flags, specifying parameters of swapchain creation, are:

```
// Provided by VK_KHR_swapchain
typedef enum VkSwapchainCreateFlagBitsKHR {
  // Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
    VK_SWAPCHAIN_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR = 0x00000001,
  // Provided by VK_VERSION_1_1 with VK_KHR_swapchain
    VK_SWAPCHAIN_CREATE_PROTECTED_BIT_KHR = 0x00000002,
  // Provided by VK_KHR_swapchain_mutable_format
    VK_SWAPCHAIN_CREATE_MUTABLE_FORMAT_BIT_KHR = 0x00000004,
} VkSwapchainCreateFlagBitsKHR;
```

- VK_SWAPCHAIN_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR specifies that images created from the swapchain (i.e. with the swapchain member of VkImageSwapchainCreateInfoKHR set to this swapchain's handle) **must** use VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT.
- VK_SWAPCHAIN_CREATE_MUTABLE_FORMAT_BIT_KHR specifies that the images of the swapchain **can** be used to create a VkImageView with a different format than what the swapchain was created with. The list of allowed image view formats is specified by adding a VkImageFormatListCreateInfo structure to the pNext chain of VkSwapchainCreateInfoKHR. In addition, this flag also specifies that the swapchain **can** be created with usage flags that are not supported for the format the swapchain is created with but are supported for at least one of the allowed image view formats.

```
// Provided by VK_KHR_swapchain
typedef VkFlags VkSwapchainCreateFlagsKHR;
```

VkSwapchainCreateFlagsKHR is a bitmask type for setting a mask of zero or more VkSwapchainCreateFlagBitsKHR.

If the pNext chain of VkSwapchainCreateInfoKHR includes a VkDeviceGroupSwapchainCreateInfoKHR structure, then that structure includes a set of device group present modes that the swapchain **can** be used with.

The VkDeviceGroupSwapchainCreateInfoKHR structure is defined as:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
typedef struct VkDeviceGroupSwapchainCreateInfoKHR {
    VkStructureType                  sType;
    const void*                      pNext;
    VkDeviceGroupPresentModeFlagsKHR    modes;
} VkDeviceGroupSwapchainCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- `modes` is a bitfield of modes that the swapchain **can** be used with.

If this structure is not present, `modes` is considered to be `VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_BIT_KHR`.

Valid Usage (Implicit)

- VUID-VkDeviceGroupSwapchainCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEVICE_GROUP_SWAPCHAIN_CREATE_INFO_KHR`
- VUID-VkDeviceGroupSwapchainCreateInfoKHR-modes-parameter
  `modes` **must** be a valid combination of VkDeviceGroupPresentModeFlagBitsKHR values
- VUID-VkDeviceGroupSwapchainCreateInfoKHR-modes-requiredbitmask
  `modes` **must** not be `0`

If the `pNext` chain of VkSwapchainCreateInfoKHR includes a VkSurfaceFullScreenExclusiveInfoEXT structure, then that structure specifies the application's preferred full-screen presentation behavior. If this structure is not present, `fullScreenExclusive` is considered to be `VK_FULL_SCREEN_EXCLUSIVE_DEFAULT_EXT`.

To enable surface counters when creating a swapchain, add a `VkSwapchainCounterCreateInfoEXT` structure to the `pNext` chain of VkSwapchainCreateInfoKHR. `VkSwapchainCounterCreateInfoEXT` is defined as:

```
// Provided by VK_EXT_display_control
typedef struct VkSwapchainCounterCreateInfoEXT {
    VkStructureType              sType;
    const void*                  pNext;
    VkSurfaceCounterFlagsEXT     surfaceCounters;
} VkSwapchainCounterCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `surfaceCounters` is a bitmask of VkSurfaceCounterFlagBitsEXT specifying surface counters to enable for the swapchain.

Valid Usage

- VUID-VkSwapchainCounterCreateInfoEXT-surfaceCounters-01244
  The bits in `surfaceCounters` **must** be supported by VkSwapchainCreateInfoKHR::`surface`, as reported by vkGetPhysicalDeviceSurfaceCapabilities2EXT

Valid Usage (Implicit)

- VUID-VkSwapchainCounterCreateInfoEXT-sType-sType

`sType` **must** be `VK_STRUCTURE_TYPE_SWAPCHAIN_COUNTER_CREATE_INFO_EXT`

- VUID-VkSwapchainCounterCreateInfoEXT-surfaceCounters-parameter
  `surfaceCounters` **must** be a valid combination of VkSurfaceCounterFlagBitsEXT values

The requested counters become active when the first presentation command for the associated swapchain is processed by the presentation engine. To query the value of an active counter, use:

```
// Provided by VK_EXT_display_control
VkResult vkGetSwapchainCounterEXT(
    VkDevice                                    device,
    VkSwapchainKHR                              swapchain,
    VkSurfaceCounterFlagBitsEXT                 counter,
    uint64_t*                                   pCounterValue);
```

- `device` is the VkDevice associated with `swapchain`.
- `swapchain` is the swapchain from which to query the counter value.
- `counter` is a VkSurfaceCounterFlagBitsEXT value specifying the counter to query.
- `pCounterValue` will return the current value of the counter.

If a counter is not available because the swapchain is out of date, the implementation **may** return `VK_ERROR_OUT_OF_DATE_KHR`.

Valid Usage

- VUID-vkGetSwapchainCounterEXT-swapchain-01245
  One or more present commands on `swapchain` **must** have been processed by the presentation engine

Valid Usage (Implicit)

- VUID-vkGetSwapchainCounterEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetSwapchainCounterEXT-swapchain-parameter
  `swapchain` **must** be a valid VkSwapchainKHR handle
- VUID-vkGetSwapchainCounterEXT-counter-parameter
  `counter` **must** be a valid VkSurfaceCounterFlagBitsEXT value
- VUID-vkGetSwapchainCounterEXT-pCounterValue-parameter
  `pCounterValue` **must** be a valid pointer to a `uint64_t` value
- VUID-vkGetSwapchainCounterEXT-swapchain-parent
  `swapchain` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_DEVICE_LOST`
- `VK_ERROR_OUT_OF_DATE_KHR`

To destroy a swapchain object call:

```
// Provided by VK_KHR_swapchain
void vkDestroySwapchainKHR(
    VkDevice                                    device,
    VkSwapchainKHR                              swapchain,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the VkDevice associated with `swapchain`.
- `swapchain` is the swapchain to destroy.
- `pAllocator` is the allocator used for host memory allocated for the swapchain object when there is no more specific allocator available (see Memory Allocation).

The application **must** not destroy a swapchain until after completion of all outstanding operations on images that were acquired from the swapchain. `swapchain` and all associated `VkImage` handles are destroyed, and **must** not be acquired or used any more by the application. The memory of each `VkImage` will only be freed after that image is no longer used by the presentation engine. For example, if one image of the swapchain is being displayed in a window, the memory for that image **may** not be freed until the window is destroyed, or another swapchain is created for the window. Destroying the swapchain does not invalidate the parent `VkSurfaceKHR`, and a new swapchain **can** be created with it.

When a swapchain associated with a display surface is destroyed, if the image most recently presented to the display surface is from the swapchain being destroyed, then either any display resources modified by presenting images from any swapchain associated with the display surface **must** be reverted by the implementation to their state prior to the first present performed on one of these swapchains, or such resources **must** be left in their current state.

If `swapchain` has exclusive full-screen access, it is released before the swapchain is destroyed.

Valid Usage

- VUID-vkDestroySwapchainKHR-swapchain-01282
  All uses of presentable images acquired from `swapchain` **must** have completed execution
- VUID-vkDestroySwapchainKHR-swapchain-01283

If VkAllocationCallbacks were provided when swapchain was created, a compatible set of callbacks **must** be provided here

- VUID-vkDestroySwapchainKHR-swapchain-01284
  If no VkAllocationCallbacks were provided when swapchain was created, pAllocator **must** be NULL

Valid Usage (Implicit)

- VUID-vkDestroySwapchainKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkDestroySwapchainKHR-swapchain-parameter
  If swapchain is not VK_NULL_HANDLE, swapchain **must** be a valid VkSwapchainKHR handle
- VUID-vkDestroySwapchainKHR-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroySwapchainKHR-swapchain-parent
  If swapchain is a valid handle, it **must** have been created, allocated, or retrieved from device

Host Synchronization

- Host access to swapchain **must** be externally synchronized

When the VK_KHR_display_swapchain extension is enabled, multiple swapchains that share presentable images are created by calling:

```
// Provided by VK_KHR_display_swapchain
VkResult vkCreateSharedSwapchainsKHR(
    VkDevice                                    device,
    uint32_t                                    swapchainCount,
    const VkSwapchainCreateInfoKHR*             pCreateInfos,
    const VkAllocationCallbacks*                pAllocator,
    VkSwapchainKHR*                             pSwapchains);
```

- device is the device to create the swapchains for.
- swapchainCount is the number of swapchains to create.
- pCreateInfos is a pointer to an array of VkSwapchainCreateInfoKHR structures specifying the parameters of the created swapchains.
- pAllocator is the allocator used for host memory allocated for the swapchain objects when there is no more specific allocator available (see Memory Allocation).

- `pSwapchains` is a pointer to an array of VkSwapchainKHR handles in which the created swapchain objects will be returned.

`vkCreateSharedSwapchainsKHR` is similar to vkCreateSwapchainKHR, except that it takes an array of VkSwapchainCreateInfoKHR structures, and returns an array of swapchain objects.

The swapchain creation parameters that affect the properties and number of presentable images **must** match between all the swapchains. If the displays used by any of the swapchains do not use the same presentable image layout or are incompatible in a way that prevents sharing images, swapchain creation will fail with the result code `VK_ERROR_INCOMPATIBLE_DISPLAY_KHR`. If any error occurs, no swapchains will be created. Images presented to multiple swapchains **must** be re-acquired from all of them before being modified. After destroying one or more of the swapchains, the remaining swapchains and the presentable images **can** continue to be used.

Valid Usage (Implicit)

- VUID-vkCreateSharedSwapchainsKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateSharedSwapchainsKHR-pCreateInfos-parameter
  `pCreateInfos` **must** be a valid pointer to an array of `swapchainCount` valid VkSwapchainCreateInfoKHR structures
- VUID-vkCreateSharedSwapchainsKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateSharedSwapchainsKHR-pSwapchains-parameter
  `pSwapchains` **must** be a valid pointer to an array of `swapchainCount` VkSwapchainKHR handles
- VUID-vkCreateSharedSwapchainsKHR-swapchainCount-arraylength
  `swapchainCount` **must** be greater than `0`

Host Synchronization

- Host access to `pCreateInfos`[].surface **must** be externally synchronized
- Host access to `pCreateInfos`[].oldSwapchain **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

- VK_ERROR_INCOMPATIBLE_DISPLAY_KHR
- VK_ERROR_DEVICE_LOST
- VK_ERROR_SURFACE_LOST_KHR

To obtain the array of presentable images associated with a swapchain, call:

```
// Provided by VK_KHR_swapchain
VkResult vkGetSwapchainImagesKHR(
    VkDevice                                    device,
    VkSwapchainKHR                              swapchain,
    uint32_t*                                   pSwapchainImageCount,
    VkImage*                                    pSwapchainImages);
```

- device is the device associated with swapchain.
- swapchain is the swapchain to query.
- pSwapchainImageCount is a pointer to an integer related to the number of presentable images available or queried, as described below.
- pSwapchainImages is either NULL or a pointer to an array of VkImage handles.

If pSwapchainImages is NULL, then the number of presentable images for swapchain is returned in pSwapchainImageCount. Otherwise, pSwapchainImageCount **must** point to a variable set by the application to the number of elements in the pSwapchainImages array, and on return the variable is overwritten with the number of structures actually written to pSwapchainImages. If the value of pSwapchainImageCount is less than the number of presentable images for swapchain, at most pSwapchainImageCount structures will be written, and VK_INCOMPLETE will be returned instead of VK_SUCCESS, to indicate that not all the available presentable images were returned.

Valid Usage (Implicit)

- VUID-vkGetSwapchainImagesKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkGetSwapchainImagesKHR-swapchain-parameter
  swapchain **must** be a valid VkSwapchainKHR handle
- VUID-vkGetSwapchainImagesKHR-pSwapchainImageCount-parameter
  pSwapchainImageCount **must** be a valid pointer to a uint32_t value
- VUID-vkGetSwapchainImagesKHR-pSwapchainImages-parameter
  If the value referenced by pSwapchainImageCount is not 0, and pSwapchainImages is not NULL, pSwapchainImages **must** be a valid pointer to an array of pSwapchainImageCount VkImage handles
- VUID-vkGetSwapchainImagesKHR-swapchain-parent
  swapchain **must** have been created, allocated, or retrieved from device

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

NOTE	By knowing all presentable images used in the swapchain, the application can create command buffers that reference these images prior to entering its main rendering loop.

Images returned by vkGetSwapchainImagesKHR are fully backed by memory before they are passed to the application, as if they are each bound completely and contiguously to a single `VkDeviceMemory` object . All presentable images are initially in the `VK_IMAGE_LAYOUT_UNDEFINED` layout, thus before using presentable images, the application **must** transition them to a valid layout for the intended use.

Further, the lifetime of presentable images is controlled by the implementation, so applications **must** not destroy a presentable image. See vkDestroySwapchainKHR for further details on the lifetime of presentable images.

Images **can** also be created by using vkCreateImage with VkImageSwapchainCreateInfoKHR and bound to swapchain memory using vkBindImageMemory2KHR with VkBindImageMemorySwapchainInfoKHR. These images **can** be used anywhere swapchain images are used, and are useful in logical devices with multiple physical devices to create peer memory bindings of swapchain memory. These images and bindings have no effect on what memory is presented. Unlike images retrieved from `vkGetSwapchainImagesKHR`, these images **must** be destroyed with vkDestroyImage.

To acquire an available presentable image to use, and retrieve the index of that image, call:

```
// Provided by VK_KHR_swapchain
VkResult vkAcquireNextImageKHR(
    VkDevice                                    device,
    VkSwapchainKHR                              swapchain,
    uint64_t                                    timeout,
    VkSemaphore                                 semaphore,
    VkFence                                     fence,
    uint32_t*                                   pImageIndex);
```

- `device` is the device associated with `swapchain`.
- `swapchain` is the non-retired swapchain from which an image is being acquired.

- `timeout` specifies how long the function waits, in nanoseconds, if no image is available.
- `semaphore` is VK_NULL_HANDLE or a semaphore to signal.
- `fence` is VK_NULL_HANDLE or a fence to signal.
- `pImageIndex` is a pointer to a `uint32_t` in which the index of the next image to use (i.e. an index into the array of images returned by `vkGetSwapchainImagesKHR`) is returned.

Valid Usage

- VUID-vkAcquireNextImageKHR-swapchain-01285
  `swapchain` **must** not be in the retired state
- VUID-vkAcquireNextImageKHR-semaphore-01286
  If `semaphore` is not VK_NULL_HANDLE, it **must** be unsignaled
- VUID-vkAcquireNextImageKHR-semaphore-01779
  If `semaphore` is not VK_NULL_HANDLE, it **must** not have any uncompleted signal or wait operations pending
- VUID-vkAcquireNextImageKHR-fence-01287
  If `fence` is not VK_NULL_HANDLE, `fence` **must** be unsignaled
- VUID-vkAcquireNextImageKHR-fence-10066
  If `fence` is not VK_NULL_HANDLE, `fence` **must** not be associated with any other queue command that has not yet completed execution on that queue
- VUID-vkAcquireNextImageKHR-semaphore-01780
  `semaphore` and `fence` **must** not both be equal to VK_NULL_HANDLE
- VUID-vkAcquireNextImageKHR-surface-07783
  If forward progress cannot be guaranteed for the `surface` used to create the `swapchain` member of `pAcquireInfo`, `timeout` **must** not be `UINT64_MAX`
- VUID-vkAcquireNextImageKHR-semaphore-03265
  `semaphore` **must** have a VkSemaphoreType of `VK_SEMAPHORE_TYPE_BINARY`

Valid Usage (Implicit)

- VUID-vkAcquireNextImageKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkAcquireNextImageKHR-swapchain-parameter
  `swapchain` **must** be a valid VkSwapchainKHR handle
- VUID-vkAcquireNextImageKHR-semaphore-parameter
  If `semaphore` is not VK_NULL_HANDLE, `semaphore` **must** be a valid VkSemaphore handle
- VUID-vkAcquireNextImageKHR-fence-parameter
  If `fence` is not VK_NULL_HANDLE, `fence` **must** be a valid VkFence handle
- VUID-vkAcquireNextImageKHR-pImageIndex-parameter
  `pImageIndex` **must** be a valid pointer to a `uint32_t` value

- VUID-vkAcquireNextImageKHR-swapchain-parent
  `swapchain` **must** have been created, allocated, or retrieved from `device`
- VUID-vkAcquireNextImageKHR-semaphore-parent
  If `semaphore` is a valid handle, it **must** have been created, allocated, or retrieved from `device`
- VUID-vkAcquireNextImageKHR-fence-parent
  If `fence` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `swapchain` **must** be externally synchronized
- Host access to `semaphore` **must** be externally synchronized
- Host access to `fence` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`
- `VK_TIMEOUT`
- `VK_NOT_READY`
- `VK_SUBOPTIMAL_KHR`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_DEVICE_LOST`
- `VK_ERROR_OUT_OF_DATE_KHR`
- `VK_ERROR_SURFACE_LOST_KHR`
- `VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT`

If an image is acquired successfully, `vkAcquireNextImageKHR` **must** either return `VK_SUCCESS` or `VK_SUBOPTIMAL_KHR`. The implementation **may** return `VK_SUBOPTIMAL_KHR` if the swapchain no longer matches the surface properties exactly, but **can** still be used for presentation.

When successful, `vkAcquireNextImageKHR` acquires a presentable image from `swapchain` that an application **can** use, and sets `pImageIndex` to the index of that image within the swapchain. The presentation engine **may** not have finished reading from the image at the time it is acquired, so the application **must** use `semaphore` and/or `fence` to ensure that the image layout and contents are not modified until the presentation engine reads have completed. Once `vkAcquireNextImageKHR` successfully acquires an image, the semaphore signal operation referenced by `semaphore`, if not VK_NULL_HANDLE, and the fence signal operation referenced by `fence`, if not VK_NULL_HANDLE,

are submitted for execution. If vkAcquireNextImageKHR does not successfully acquire an image, semaphore and fence are unaffected. The order in which images are acquired is implementation-dependent, and **may** be different than the order the images were presented.

If timeout is zero, then vkAcquireNextImageKHR does not wait, and will either successfully acquire an image, or fail and return VK_NOT_READY if no image is available.

If the specified timeout period expires before an image is acquired, vkAcquireNextImageKHR returns VK_TIMEOUT. If timeout is UINT64_MAX, the timeout period is treated as infinite, and vkAcquireNextImageKHR will block until an image is acquired or an error occurs.

Let S be the number of images in swapchain. Let M be the value of VkSurfaceCapabilitiesKHR ::minImageCount.

vkAcquireNextImageKHR **should** not be called if the number of images that the application has currently acquired is greater than S-M. If vkAcquireNextImageKHR is called when the number of images that the application has currently acquired is less than or equal to S-M, vkAcquireNextImageKHR **must** return in finite time with an allowed VkResult code.

NOTE

Returning a result in finite time guarantees that the implementation cannot deadlock an application, or suspend its execution indefinitely with correct API usage. Acquiring too many images at once may block indefinitely, which is covered by valid usage when attempting to use UINT64_MAX. For example, a scenario here is when a compositor holds on to images which are currently being presented, and there are not any vacant images left to be acquired.

If the swapchain images no longer match native surface properties, either VK_SUBOPTIMAL_KHR or VK_ERROR_OUT_OF_DATE_KHR **must** be returned. If VK_ERROR_OUT_OF_DATE_KHR is returned, no image is acquired and attempts to present previously acquired images to the swapchain will also fail with VK_ERROR_OUT_OF_DATE_KHR. Applications need to create a new swapchain for the surface to continue presenting if VK_ERROR_OUT_OF_DATE_KHR is returned.

NOTE

VK_SUBOPTIMAL_KHR **may** happen, for example, if the platform surface has been resized but the platform is able to scale the presented images to the new size to produce valid surface updates. It is up to the application to decide whether it prefers to continue using the current swapchain in this state, or to re-create the swapchain to better match the platform surface properties.

If device loss occurs (see Lost Device) before the timeout has expired, vkAcquireNextImageKHR **must** return in finite time with either one of the allowed success codes, or VK_ERROR_DEVICE_LOST.

If semaphore is not VK_NULL_HANDLE, the semaphore **must** be unsignaled, with no signal or wait operations pending. It will become signaled when the application **can** use the image.

NOTE

Use of semaphore allows rendering operations to be recorded and submitted before the presentation engine has completed its use of the image.

If fence is not equal to VK_NULL_HANDLE, the fence **must** be unsignaled, with no signal operations pending. It will become signaled when the application **can** use the image.

NOTE

Applications **should** not rely on vkAcquireNextImageKHR blocking in order to meter their rendering speed. The implementation **may** return from this function immediately regardless of how many presentation requests are queued, and regardless of when queued presentation requests will complete relative to the call. Instead, applications **can** use fence to meter their frame generation work to match the presentation rate.

An application **must** wait until either the semaphore or fence is signaled before accessing the image's data.

NOTE

When the presentable image will be accessed by some stage S, the recommended idiom for ensuring correct synchronization is:

- The VkSubmitInfo used to submit the image layout transition for execution includes vkAcquireNextImageKHR::semaphore in its pWaitSemaphores member, with the corresponding element of pWaitDstStageMask including S.
- The synchronization command that performs any necessary image layout transition includes S in both the srcStageMask and dstStageMask.

After a successful return, the image indicated by pImageIndex and its data will be unmodified compared to when it was presented.

NOTE

Exclusive ownership of presentable images corresponding to a swapchain created with VK_SHARING_MODE_EXCLUSIVE as defined in Resource Sharing is not altered by a call to vkAcquireNextImageKHR. That means upon the first acquisition from such a swapchain presentable images are not owned by any queue family, while at subsequent acquisitions the presentable images remain owned by the queue family the image was previously presented on.

The possible return values for vkAcquireNextImageKHR depend on the timeout provided:

- VK_SUCCESS is returned if an image became available.
- VK_ERROR_SURFACE_LOST_KHR is returned if the surface becomes no longer available.
- VK_NOT_READY is returned if timeout is zero and no image was available.
- VK_TIMEOUT is returned if timeout is greater than zero and less than UINT64_MAX, and no image became available within the time allowed.
- VK_SUBOPTIMAL_KHR is returned if an image became available, and the swapchain no longer matches the surface properties exactly, but **can** still be used to present to the surface successfully.

NOTE

This **may** happen, for example, if the platform surface has been resized but the platform is able to scale the presented images to the new size to produce valid surface updates. It is up to the application to decide whether it prefers to continue using the current swapchain indefinitely or temporarily in this state, or to re-create the swapchain to better match the platform surface properties.

- VK_ERROR_OUT_OF_DATE_KHR is returned if the surface has changed in such a way that it is no longer compatible with the swapchain, and further presentation requests using the swapchain will fail. Applications **must** query the new surface properties and recreate their swapchain if they wish to continue presenting to the surface.

If the native surface and presented image sizes no longer match, presentation **may** fail . If presentation does succeed, the mapping from the presented image to the native surface is implementation-defined. It is the application's responsibility to detect surface size changes and react appropriately. If presentation fails because of a mismatch in the surface and presented image sizes, a VK_ERROR_OUT_OF_DATE_KHR error will be returned.

NOTE

For example, consider a 4x3 window/surface that gets resized to be 3x4 (taller than wider). On some window systems, the portion of the window/surface that was previously and still is visible (the 3x3 part) will contain the same contents as before, while the remaining parts of the window will have undefined contents. Other window systems **may** squash/stretch the image to fill the new window size without any undefined contents, or apply some other mapping.

To acquire an available presentable image to use, and retrieve the index of that image, call:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
VkResult vkAcquireNextImage2KHR(
    VkDevice                                    device,
    const VkAcquireNextImageInfoKHR*            pAcquireInfo,
    uint32_t*                                   pImageIndex);
```

- device is the device associated with swapchain.
- pAcquireInfo is a pointer to a VkAcquireNextImageInfoKHR structure containing parameters of the acquire.
- pImageIndex is a pointer to a uint32_t value specifying the index of the next image to use.

Valid Usage

- VUID-vkAcquireNextImage2KHR-surface-07784
  If forward progress cannot be guaranteed for the surface used to create swapchain, the timeout member of pAcquireInfo **must** not be UINT64_MAX

Valid Usage (Implicit)

- VUID-vkAcquireNextImage2KHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkAcquireNextImage2KHR-pAcquireInfo-parameter
  pAcquireInfo **must** be a valid pointer to a valid VkAcquireNextImageInfoKHR structure

- VUID-vkAcquireNextImage2KHR-pImageIndex-parameter
  `pImageIndex` **must** be a valid pointer to a `uint32_t` value

Return Codes

Success

- `VK_SUCCESS`
- `VK_TIMEOUT`
- `VK_NOT_READY`
- `VK_SUBOPTIMAL_KHR`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_DEVICE_LOST`
- `VK_ERROR_OUT_OF_DATE_KHR`
- `VK_ERROR_SURFACE_LOST_KHR`
- `VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT`

The `VkAcquireNextImageInfoKHR` structure is defined as:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
typedef struct VkAcquireNextImageInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkSwapchainKHR     swapchain;
    uint64_t           timeout;
    VkSemaphore        semaphore;
    VkFence            fence;
    uint32_t           deviceMask;
} VkAcquireNextImageInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `swapchain` is a non-retired swapchain from which an image is acquired.
- `timeout` specifies how long the function waits, in nanoseconds, if no image is available.
- `semaphore` is VK_NULL_HANDLE or a semaphore to signal.
- `fence` is VK_NULL_HANDLE or a fence to signal.
- `deviceMask` is a mask of physical devices for which the swapchain image will be ready to use when the semaphore or fence is signaled.

If vkAcquireNextImageKHR is used, the device mask is considered to include all physical devices in the logical device.

NOTE

vkAcquireNextImage2KHR signals at most one semaphore, even if the application requests waiting for multiple physical devices to be ready via the `deviceMask`. However, only a single physical device **can** wait on that semaphore, since the semaphore becomes unsignaled when the wait succeeds. For other physical devices to wait for the image to be ready, it is necessary for the application to submit semaphore signal operation(s) to that first physical device to signal additional semaphore(s) after the wait succeeds, which the other physical device(s) **can** wait upon.

Valid Usage

- VUID-VkAcquireNextImageInfoKHR-swapchain-01675
  `swapchain` **must** not be in the retired state
- VUID-VkAcquireNextImageInfoKHR-semaphore-01288
  If `semaphore` is not VK_NULL_HANDLE, it **must** be unsignaled
- VUID-VkAcquireNextImageInfoKHR-semaphore-01781
  If `semaphore` is not VK_NULL_HANDLE, it **must** not have any uncompleted signal or wait operations pending
- VUID-VkAcquireNextImageInfoKHR-fence-01289
  If `fence` is not VK_NULL_HANDLE, `fence` **must** be unsignaled
- VUID-VkAcquireNextImageInfoKHR-fence-10067
  If `fence` is not VK_NULL_HANDLE, `fence` **must** not be associated with any other queue command that has not yet completed execution on that queue
- VUID-VkAcquireNextImageInfoKHR-semaphore-01782
  `semaphore` and `fence` **must** not both be equal to VK_NULL_HANDLE
- VUID-VkAcquireNextImageInfoKHR-deviceMask-01290
  `deviceMask` **must** be a valid device mask
- VUID-VkAcquireNextImageInfoKHR-deviceMask-01291
  `deviceMask` **must** not be zero
- VUID-VkAcquireNextImageInfoKHR-semaphore-03266
  `semaphore` **must** have a VkSemaphoreType of `VK_SEMAPHORE_TYPE_BINARY`

Valid Usage (Implicit)

- VUID-VkAcquireNextImageInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ACQUIRE_NEXT_IMAGE_INFO_KHR`
- VUID-VkAcquireNextImageInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkAcquireNextImageInfoKHR-swapchain-parameter

swapchain **must** be a valid VkSwapchainKHR handle

- VUID-VkAcquireNextImageInfoKHR-semaphore-parameter
  If semaphore is not VK_NULL_HANDLE, semaphore **must** be a valid VkSemaphore handle
- VUID-VkAcquireNextImageInfoKHR-fence-parameter
  If fence is not VK_NULL_HANDLE, fence **must** be a valid VkFence handle
- VUID-VkAcquireNextImageInfoKHR-commonparent
  Each of fence, semaphore, and swapchain that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to swapchain **must** be externally synchronized
- Host access to semaphore **must** be externally synchronized
- Host access to fence **must** be externally synchronized

After queueing all rendering commands and transitioning the image to the correct layout, to queue an image for presentation, call:

```
// Provided by VK_KHR_swapchain
VkResult vkQueuePresentKHR(
    VkQueue                                     queue,
    const VkPresentInfoKHR*                     pPresentInfo);
```

- queue is a queue that is capable of presentation to the target surface's platform on the same device as the image's swapchain.
- pPresentInfo is a pointer to a VkPresentInfoKHR structure specifying parameters of the presentation.

NOTE There is no requirement for an application to present images in the same order that they were acquired - applications can arbitrarily present any image that is currently acquired.

NOTE The origin of the native orientation of the surface coordinate system is not specified in the Vulkan specification; it depends on the platform. For most platforms the origin is by default upper-left, meaning the pixel of the presented VkImage at coordinates (0,0) would appear at the upper left pixel of the platform surface (assuming VK_SURFACE_TRANSFORM_IDENTITY_BIT_KHR, and the display standing the right way up).

The result codes VK_ERROR_OUT_OF_DATE_KHR and VK_SUBOPTIMAL_KHR have the same meaning when returned by vkQueuePresentKHR as they do when returned by vkAcquireNextImageKHR. If any swapchain member of pPresentInfo was created with VK_FULL_SCREEN_EXCLUSIVE_APPLICATION_CONTROLLED_EXT, VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT will be returned if that swapchain does not have

exclusive full-screen access, possibly for implementation-specific reasons outside of the application's control. If multiple swapchains are presented, the result code is determined by applying the following rules in order:

- If the device is lost, VK_ERROR_DEVICE_LOST is returned.
- If any of the target surfaces are no longer available the error VK_ERROR_SURFACE_LOST_KHR is returned.
- If any of the presents would have a result of VK_ERROR_OUT_OF_DATE_KHR if issued separately then VK_ERROR_OUT_OF_DATE_KHR is returned.
- If any of the presents would have a result of VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT if issued separately then VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT is returned.
- If any of the presents would have a result of VK_SUBOPTIMAL_KHR if issued separately then VK_SUBOPTIMAL_KHR is returned.
- Otherwise VK_SUCCESS is returned.

Any writes to memory backing the images referenced by the pImageIndices and pSwapchains members of pPresentInfo, that are available before vkQueuePresentKHR is executed, are automatically made visible to the read access performed by the presentation engine. This automatic visibility operation for an image happens-after the semaphore signal operation, and happens-before the presentation engine accesses the image.

Presentation is a read-only operation that will not affect the content of the presentable images. Upon reacquiring the image and transitioning it away from the VK_IMAGE_LAYOUT_PRESENT_SRC_KHR layout, the contents will be the same as they were prior to transitioning the image to the present source layout and presenting it. However, if a mechanism other than Vulkan is used to modify the platform window associated with the swapchain, the content of all presentable images in the swapchain becomes undefined.

Calls to vkQueuePresentKHR **may** block, but **must** return in finite time. The processing of the presentation happens in issue order with other queue operations, but semaphores **must** be used to ensure that prior rendering and other commands in the specified queue complete before the presentation begins. The presentation command itself does not delay processing of subsequent commands on the queue. However, presentation requests sent to a particular queue are always performed in order. Exact presentation timing is controlled by the semantics of the presentation engine and native platform in use.

If an image is presented to a swapchain created from a display surface, the mode of the associated display will be updated, if necessary, to match the mode specified when creating the display surface. The mode switch and presentation of the specified image will be performed as one atomic operation.

Queueing an image for presentation defines a set of *queue operations*, including waiting on the semaphores and submitting a presentation request to the presentation engine. However, the scope of this set of queue operations does not include the actual processing of the image by the presentation engine.

If vkQueuePresentKHR fails to enqueue the corresponding set of queue operations, it **may** return

VK_ERROR_OUT_OF_HOST_MEMORY or VK_ERROR_OUT_OF_DEVICE_MEMORY. If it does, the implementation **must** ensure that the state and contents of any resources or synchronization primitives referenced is unaffected by the call or its failure.

If vkQueuePresentKHR fails in such a way that the implementation is unable to make that guarantee, the implementation **must** return VK_ERROR_DEVICE_LOST.

However, if the presentation request is rejected by the presentation engine with an error VK_ERROR_OUT_OF_DATE_KHR, VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT, or VK_ERROR_SURFACE_LOST_KHR, the set of queue operations are still considered to be enqueued and thus any semaphore wait operation specified in VkPresentInfoKHR will execute when the corresponding queue operation is complete.

vkQueuePresentKHR releases the acquisition of the images referenced by imageIndices. The queue family corresponding to the queue vkQueuePresentKHR is executed on **must** have ownership of the presented images as defined in Resource Sharing. vkQueuePresentKHR does not alter the queue family ownership, but the presented images **must** not be used again before they have been reacquired using vkAcquireNextImageKHR.

NOTE The application **can** continue to present any acquired images from a retired swapchain as long as the swapchain has not entered a state that causes vkQueuePresentKHR to return VK_ERROR_OUT_OF_DATE_KHR.

Valid Usage

- VUID-vkQueuePresentKHR-pSwapchains-01292
  Each element of pSwapchains member of pPresentInfo **must** be a swapchain that is created for a surface for which presentation is supported from queue as determined using a call to vkGetPhysicalDeviceSurfaceSupportKHR
- VUID-vkQueuePresentKHR-pSwapchains-01293
  If more than one member of pSwapchains was created from a display surface, all display surfaces referenced that refer to the same display **must** use the same display mode
- VUID-vkQueuePresentKHR-pWaitSemaphores-01294
  When a semaphore wait operation referring to a binary semaphore defined by the elements of the pWaitSemaphores member of pPresentInfo executes on queue, there **must** be no other queues waiting on the same semaphore
- VUID-vkQueuePresentKHR-pWaitSemaphores-03267
  All elements of the pWaitSemaphores member of pPresentInfo **must** be created with a VkSemaphoreType of VK_SEMAPHORE_TYPE_BINARY
- VUID-vkQueuePresentKHR-pWaitSemaphores-03268
  All elements of the pWaitSemaphores member of pPresentInfo **must** reference a semaphore signal operation that has been submitted for execution and any semaphore signal operations on which it depends **must** have also been submitted for execution

Valid Usage (Implicit)

- VUID-vkQueuePresentKHR-queue-parameter
  `queue` **must** be a valid VkQueue handle
- VUID-vkQueuePresentKHR-pPresentInfo-parameter
  `pPresentInfo` **must** be a valid pointer to a valid VkPresentInfoKHR structure

Host Synchronization

- Host access to `queue` **must** be externally synchronized
- Host access to `pPresentInfo->pWaitSemaphores`[] **must** be externally synchronized
- Host access to `pPresentInfo->pSwapchains`[] **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
-	-	-	Any	-

Return Codes

Success

- `VK_SUCCESS`
- `VK_SUBOPTIMAL_KHR`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_DEVICE_LOST`
- `VK_ERROR_OUT_OF_DATE_KHR`
- `VK_ERROR_SURFACE_LOST_KHR`
- `VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT`

The `VkPresentInfoKHR` structure is defined as:

```
// Provided by VK_KHR_swapchain
typedef struct VkPresentInfoKHR {
    VkStructureType          sType;
    const void*              pNext;
    uint32_t                 waitSemaphoreCount;
    const VkSemaphore*       pWaitSemaphores;
    uint32_t                 swapchainCount;
    const VkSwapchainKHR*    pSwapchains;
    const uint32_t*          pImageIndices;
    VkResult*                pResults;
} VkPresentInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `waitSemaphoreCount` is the number of semaphores to wait for before issuing the present request. The number **may** be zero.
- `pWaitSemaphores` is `NULL` or a pointer to an array of VkSemaphore objects with `waitSemaphoreCount` entries, and specifies the semaphores to wait for before issuing the present request.
- `swapchainCount` is the number of swapchains being presented to by this command.
- `pSwapchains` is a pointer to an array of VkSwapchainKHR objects with `swapchainCount` entries.
- `pImageIndices` is a pointer to an array of indices into the array of each swapchain's presentable images, with `swapchainCount` entries. Each entry in this array identifies the image to present on the corresponding entry in the `pSwapchains` array.
- `pResults` is a pointer to an array of VkResult typed elements with `swapchainCount` entries. Applications that do not need per-swapchain results **can** use `NULL` for `pResults`. If non-`NULL`, each entry in `pResults` will be set to the VkResult for presenting the swapchain corresponding to the same index in `pSwapchains`.

Before an application **can** present an image, the image's layout **must** be transitioned to the `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR` layout, or for a shared presentable image the `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR` layout.

NOTE

When transitioning the image to `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR` or `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR`, there is no need to delay subsequent processing, or perform any visibility operations (as vkQueuePresentKHR performs automatic visibility operations). To achieve this, the `dstAccessMask` member of the VkImageMemoryBarrier **should** be `0`, and the `dstStageMask` parameter **should** be `VK_PIPELINE_STAGE_BOTTOM_OF_PIPE_BIT`.

Valid Usage

- VUID-VkPresentInfoKHR-pSwapchain-09231
  Elements of `pSwapchain` **must** be unique
- VUID-VkPresentInfoKHR-pImageIndices-01430

Each element of pImageIndices **must** be the index of a presentable image acquired from the swapchain specified by the corresponding element of the pSwapchains array, and the presented image subresource **must** be in the VK_IMAGE_LAYOUT_PRESENT_SRC_KHR or VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR layout at the time the operation is executed on a VkDevice

- VUID-VkPresentInfoKHR-pNext-06235
  If a VkPresentIdKHR structure is included in the pNext chain, and the presentId feature is not enabled, each presentIds entry in that structure **must** be NULL

Valid Usage (Implicit)

- VUID-VkPresentInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PRESENT_INFO_KHR
- VUID-VkPresentInfoKHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkDeviceGroupPresentInfoKHR, VkDisplayPresentInfoKHR, VkFrameBoundaryEXT, VkPresentIdKHR, or VkPresentRegionsKHR
- VUID-VkPresentInfoKHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkPresentInfoKHR-pWaitSemaphores-parameter
  If waitSemaphoreCount is not 0, pWaitSemaphores **must** be a valid pointer to an array of waitSemaphoreCount valid VkSemaphore handles
- VUID-VkPresentInfoKHR-pSwapchains-parameter
  pSwapchains **must** be a valid pointer to an array of swapchainCount valid VkSwapchainKHR handles
- VUID-VkPresentInfoKHR-pImageIndices-parameter
  pImageIndices **must** be a valid pointer to an array of swapchainCount uint32_t values
- VUID-VkPresentInfoKHR-pResults-parameter
  If pResults is not NULL, pResults **must** be a valid pointer to an array of swapchainCount VkResult values
- VUID-VkPresentInfoKHR-swapchainCount-arraylength
  swapchainCount **must** be greater than 0
- VUID-VkPresentInfoKHR-commonparent
  Both of the elements of pSwapchains, and the elements of pWaitSemaphores that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

When the VK_KHR_incremental_present extension is enabled, additional fields **can** be specified that allow an application to specify that only certain rectangular regions of the presentable images of a swapchain are changed. This is an optimization hint that a presentation engine **may** use to only update the region of a surface that is actually changing. The application still **must** ensure that all pixels of a presented image contain the desired values, in case the presentation engine ignores this

hint. An application **can** provide this hint by adding a VkPresentRegionsKHR structure to the pNext chain of the VkPresentInfoKHR structure.

The VkPresentRegionsKHR structure is defined as:

```
// Provided by VK_KHR_incremental_present
typedef struct VkPresentRegionsKHR {
    VkStructureType              sType;
    const void*                  pNext;
    uint32_t                     swapchainCount;
    const VkPresentRegionKHR*    pRegions;
} VkPresentRegionsKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- swapchainCount is the number of swapchains being presented to by this command.
- pRegions is NULL or a pointer to an array of VkPresentRegionKHR elements with swapchainCount entries. If not NULL, each element of pRegions contains the region that has changed since the last present to the swapchain in the corresponding entry in the VkPresentInfoKHR::pSwapchains array.

Valid Usage

- VUID-VkPresentRegionsKHR-swapchainCount-01260
  swapchainCount **must** be the same value as VkPresentInfoKHR::swapchainCount, where VkPresentInfoKHR is included in the pNext chain of this VkPresentRegionsKHR structure

Valid Usage (Implicit)

- VUID-VkPresentRegionsKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PRESENT_REGIONS_KHR
- VUID-VkPresentRegionsKHR-pRegions-parameter
  If pRegions is not NULL, pRegions **must** be a valid pointer to an array of swapchainCount valid VkPresentRegionKHR structures
- VUID-VkPresentRegionsKHR-swapchainCount-arraylength
  swapchainCount **must** be greater than 0

For a given image and swapchain, the region to present is specified by the VkPresentRegionKHR structure, which is defined as:

```
// Provided by VK_KHR_incremental_present
typedef struct VkPresentRegionKHR {
    uint32_t                rectangleCount;
    const VkRectLayerKHR*   pRectangles;
} VkPresentRegionKHR;
```

- `rectangleCount` is the number of rectangles in `pRectangles`, or zero if the entire image has changed and should be presented.
- `pRectangles` is either `NULL` or a pointer to an array of `VkRectLayerKHR` structures. The `VkRectLayerKHR` structure is the framebuffer coordinates, plus layer, of a portion of a presentable image that has changed and **must** be presented. If non-`NULL`, each entry in `pRectangles` is a rectangle of the given image that has changed since the last image was presented to the given swapchain. The rectangles **must** be specified relative to VkSurfaceCapabilitiesKHR ::`currentTransform`, regardless of the swapchain's `preTransform`. The presentation engine will apply the `preTransform` transformation to the rectangles, along with any further transformation it applies to the image content.

Valid Usage (Implicit)

- VUID-VkPresentRegionKHR-pRectangles-parameter
  If `rectangleCount` is not `0`, and `pRectangles` is not `NULL`, `pRectangles` **must** be a valid pointer to an array of `rectangleCount` valid VkRectLayerKHR structures

The `VkRectLayerKHR` structure is defined as:

```
// Provided by VK_KHR_incremental_present
typedef struct VkRectLayerKHR {
    VkOffset2D    offset;
    VkExtent2D    extent;
    uint32_t      layer;
} VkRectLayerKHR;
```

- `offset` is the origin of the rectangle, in pixels.
- `extent` is the size of the rectangle, in pixels.
- `layer` is the layer of the image. For images with only one layer, the value of `layer` **must** be 0.

Some platforms allow the size of a surface to change, and then scale the pixels of the image to fit the surface. `VkRectLayerKHR` specifies pixels of the swapchain's image(s), which will be constant for the life of the swapchain.

Valid Usage

- VUID-VkRectLayerKHR-offset-04864
  The sum of `offset` and `extent`, after being transformed according to the `preTransform`

member of the VkSwapchainCreateInfoKHR structure, **must** be no greater than the imageExtent member of the VkSwapchainCreateInfoKHR structure passed to vkCreateSwapchainKHR

- VUID-VkRectLayerKHR-layer-01262
  layer **must** be less than the imageArrayLayers member of the VkSwapchainCreateInfoKHR structure passed to vkCreateSwapchainKHR

When the VK_KHR_display_swapchain extension is enabled, additional fields **can** be specified when presenting an image to a swapchain by setting VkPresentInfoKHR::pNext to point to a VkDisplayPresentInfoKHR structure.

The VkDisplayPresentInfoKHR structure is defined as:

```
// Provided by VK_KHR_display_swapchain
typedef struct VkDisplayPresentInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkRect2D           srcRect;
    VkRect2D           dstRect;
    VkBool32           persistent;
} VkDisplayPresentInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- srcRect is a rectangular region of pixels to present. It **must** be a subset of the image being presented. If VkDisplayPresentInfoKHR is not specified, this region will be assumed to be the entire presentable image.
- dstRect is a rectangular region within the visible region of the swapchain's display mode. If VkDisplayPresentInfoKHR is not specified, this region will be assumed to be the entire visible region of the swapchain's mode. If the specified rectangle is a subset of the display mode's visible region, content from display planes below the swapchain's plane will be visible outside the rectangle. If there are no planes below the swapchain's, the area outside the specified rectangle will be black. If portions of the specified rectangle are outside of the display's visible region, pixels mapping only to those portions of the rectangle will be discarded.
- persistent: If this is VK_TRUE, the display engine will enable buffered mode on displays that support it. This allows the display engine to stop sending content to the display until a new image is presented. The display will instead maintain a copy of the last presented image. This allows less power to be used, but **may** increase presentation latency. If VkDisplayPresentInfoKHR is not specified, persistent mode will not be used.

If the extent of the srcRect and dstRect are not equal, the presented pixels will be scaled accordingly.

Valid Usage

- VUID-VkDisplayPresentInfoKHR-srcRect-01257
  `srcRect` **must** specify a rectangular region that is a subset of the image being presented
- VUID-VkDisplayPresentInfoKHR-dstRect-01258
  `dstRect` **must** specify a rectangular region that is a subset of the `visibleRegion` parameter of the display mode the swapchain being presented uses
- VUID-VkDisplayPresentInfoKHR-persistentContent-01259
  If the `persistentContent` member of the `VkDisplayPropertiesKHR` structure returned by `vkGetPhysicalDeviceDisplayPropertiesKHR` for the display the present operation targets is `VK_FALSE`, then `persistent` **must** be `VK_FALSE`

Valid Usage (Implicit)

- VUID-VkDisplayPresentInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DISPLAY_PRESENT_INFO_KHR`

If the `pNext` chain of VkPresentInfoKHR includes a `VkDeviceGroupPresentInfoKHR` structure, then that structure includes an array of device masks and a device group present mode.

The `VkDeviceGroupPresentInfoKHR` structure is defined as:

```
// Provided by VK_VERSION_1_1 with VK_KHR_swapchain, VK_KHR_device_group with
VK_KHR_swapchain
typedef struct VkDeviceGroupPresentInfoKHR {
    VkStructureType                         sType;
    const void*                             pNext;
    uint32_t                                swapchainCount;
    const uint32_t*                         pDeviceMasks;
    VkDeviceGroupPresentModeFlagBitsKHR     mode;
} VkDeviceGroupPresentInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `swapchainCount` is zero or the number of elements in `pDeviceMasks`.
- `pDeviceMasks` is a pointer to an array of device masks, one for each element of VkPresentInfoKHR::`pSwapchains`.
- `mode` is a VkDeviceGroupPresentModeFlagBitsKHR value specifying the device group present mode that will be used for this present.

If `mode` is `VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_BIT_KHR`, then each element of `pDeviceMasks` selects which instance of the swapchain image is presented. Each element of `pDeviceMasks` **must** have exactly one bit set, and the corresponding physical device **must** have a presentation engine as

reported by VkDeviceGroupPresentCapabilitiesKHR.

If mode is VK_DEVICE_GROUP_PRESENT_MODE_REMOTE_BIT_KHR, then each element of pDeviceMasks selects which instance of the swapchain image is presented. Each element of pDeviceMasks **must** have exactly one bit set, and some physical device in the logical device **must** include that bit in its VkDeviceGroupPresentCapabilitiesKHR::presentMask.

If mode is VK_DEVICE_GROUP_PRESENT_MODE_SUM_BIT_KHR, then each element of pDeviceMasks selects which instances of the swapchain image are component-wise summed and the sum of those images is presented. If the sum in any component is outside the representable range, the value of that component is undefined. Each element of pDeviceMasks **must** have a value for which all set bits are set in one of the elements of VkDeviceGroupPresentCapabilitiesKHR::presentMask.

If mode is VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_MULTI_DEVICE_BIT_KHR, then each element of pDeviceMasks selects which instance(s) of the swapchain images are presented. For each bit set in each element of pDeviceMasks, the corresponding physical device **must** have a presentation engine as reported by VkDeviceGroupPresentCapabilitiesKHR.

If VkDeviceGroupPresentInfoKHR is not provided or swapchainCount is zero then the masks are considered to be 1. If VkDeviceGroupPresentInfoKHR is not provided, mode is considered to be VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_BIT_KHR.

Valid Usage

- VUID-VkDeviceGroupPresentInfoKHR-swapchainCount-01297
  swapchainCount **must** equal 0 or VkPresentInfoKHR::swapchainCount
- VUID-VkDeviceGroupPresentInfoKHR-mode-01298
  If mode is VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_BIT_KHR, then each element of pDeviceMasks **must** have exactly one bit set, and the corresponding element of VkDeviceGroupPresentCapabilitiesKHR::presentMask **must** be non-zero
- VUID-VkDeviceGroupPresentInfoKHR-mode-01299
  If mode is VK_DEVICE_GROUP_PRESENT_MODE_REMOTE_BIT_KHR, then each element of pDeviceMasks **must** have exactly one bit set, and some physical device in the logical device **must** include that bit in its VkDeviceGroupPresentCapabilitiesKHR::presentMask
- VUID-VkDeviceGroupPresentInfoKHR-mode-01300
  If mode is VK_DEVICE_GROUP_PRESENT_MODE_SUM_BIT_KHR, then each element of pDeviceMasks **must** have a value for which all set bits are set in one of the elements of VkDeviceGroupPresentCapabilitiesKHR::presentMask
- VUID-VkDeviceGroupPresentInfoKHR-mode-01301
  If mode is VK_DEVICE_GROUP_PRESENT_MODE_LOCAL_MULTI_DEVICE_BIT_KHR, then for each bit set in each element of pDeviceMasks, the corresponding element of VkDeviceGroupPresentCapabilitiesKHR::presentMask **must** be non-zero
- VUID-VkDeviceGroupPresentInfoKHR-pDeviceMasks-01302
  The value of each element of pDeviceMasks **must** be equal to the device mask passed in VkAcquireNextImageInfoKHR::deviceMask when the image index was last acquired
- VUID-VkDeviceGroupPresentInfoKHR-mode-01303

mode **must** have exactly one bit set, and that bit **must** have been included in VkDeviceGroupSwapchainCreateInfoKHR::modes

Valid Usage (Implicit)

- VUID-VkDeviceGroupPresentInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_INFO_KHR
- VUID-VkDeviceGroupPresentInfoKHR-pDeviceMasks-parameter
  If swapchainCount is not 0, pDeviceMasks **must** be a valid pointer to an array of swapchainCount uint32_t values
- VUID-VkDeviceGroupPresentInfoKHR-mode-parameter
  mode **must** be a valid VkDeviceGroupPresentModeFlagBitsKHR value

The VkPresentIdKHR structure is defined as:

```
// Provided by VK_KHR_present_id
typedef struct VkPresentIdKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           swapchainCount;
    const uint64_t*    pPresentIds;
} VkPresentIdKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- swapchainCount is the number of swapchains being presented to the vkQueuePresentKHR command.
- pPresentIds is NULL or a pointer to an array of uint64_t with swapchainCount entries. If not NULL, each non-zero value in pPresentIds specifies the present id to be associated with the presentation of the swapchain with the same index in the vkQueuePresentKHR call.

For applications to be able to reference specific presentation events queued by a call to vkQueuePresentKHR, an identifier needs to be associated with them. When the presentId feature is enabled, applications **can** include the VkPresentIdKHR structure in the pNext chain of the VkPresentInfoKHR structure to supply identifiers.

Each VkSwapchainKHR has a presentId associated with it. This value is initially zero when the VkSwapchainKHR is created.

When a VkPresentIdKHR structure with a non-NULL pPresentIds is included in the pNext chain of a VkPresentInfoKHR structure, each pSwapchains entry has a presentId associated in the pPresentIds array at the same index as the swapchain in the pSwapchains array. If this presentId is non-zero, then the application **can** later use this value to refer to that image presentation. A value of zero indicates that this presentation has no associated presentId. A non-zero presentId **must** be greater

than any non-zero presentId passed previously by the application for the same swapchain.

There is no requirement for any precise timing relationship between the presentation of the image to the user and the update of the presentId value, but implementations **should** make this as close as possible to the presentation of the first pixel in the new image to the user.

Valid Usage

- VUID-VkPresentIdKHR-swapchainCount-04998
  `swapchainCount` **must** be the same value as `VkPresentInfoKHR`::`swapchainCount`, where this `VkPresentIdKHR` is in the `pNext` chain of the `VkPresentInfoKHR` structure
- VUID-VkPresentIdKHR-presentIds-04999
  Each `presentIds` entry **must** be greater than any previous `presentIds` entry passed for the associated `pSwapchains` entry

Valid Usage (Implicit)

- VUID-VkPresentIdKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PRESENT_ID_KHR`
- VUID-VkPresentIdKHR-pPresentIds-parameter
  If `pPresentIds` is not `NULL`, `pPresentIds` **must** be a valid pointer to an array of `swapchainCount` `uint64_t` values
- VUID-VkPresentIdKHR-swapchainCount-arraylength
  `swapchainCount` **must** be greater than `0`

When the `presentWait` feature is enabled, an application **can** wait for an image to be presented to the user by first specifying a presentId for the target presentation by adding a `VkPresentIdKHR` structure to the `pNext` chain of the VkPresentInfoKHR structure and then waiting for that presentation to complete by calling:

```
// Provided by VK_KHR_present_wait
VkResult vkWaitForPresentKHR(
    VkDevice                                    device,
    VkSwapchainKHR                              swapchain,
    uint64_t                                    presentId,
    uint64_t                                    timeout);
```

- `device` is the device associated with `swapchain`.
- `swapchain` is the non-retired swapchain on which an image was queued for presentation.
- `presentId` is the presentation presentId to wait for.
- `timeout` is the timeout period in units of nanoseconds. `timeout` is adjusted to the closest value allowed by the implementation-dependent timeout accuracy, which **may** be substantially longer than one nanosecond, and **may** be longer than the requested period.

vkWaitForPresentKHR waits for the presentId associated with swapchain to be increased in value so that it is at least equal to presentId.

For VK_PRESENT_MODE_MAILBOX_KHR (or other present mode where images may be replaced in the presentation queue) any wait of this type associated with such an image **must** be signaled no later than a wait associated with the replacing image would be signaled.

When the presentation has completed, the presentId associated with the related pSwapchains entry will be increased in value so that it is at least equal to the value provided in the VkPresentIdKHR structure.

There is no requirement for any precise timing relationship between the presentation of the image to the user and the update of the presentId value, but implementations **should** make this as close as possible to the presentation of the first pixel in the next image being presented to the user.

The call to vkWaitForPresentKHR will block until either the presentId associated with swapchain is greater than or equal to presentId, or timeout nanoseconds passes. When the swapchain becomes OUT_OF_DATE, the call will either return VK_SUCCESS (if the image was delivered to the presentation engine and may have been presented to the user) or will return early with status VK_ERROR_OUT_OF_DATE_KHR (if the image was not presented to the user).

As an exception to the normal rules for objects which are externally synchronized, the swapchain passed to vkWaitForPresentKHR **may** be simultaneously used by other threads in calls to functions other than vkDestroySwapchainKHR. Access to the swapchain data associated with this extension **must** be atomic within the implementation.

Valid Usage

- VUID-vkWaitForPresentKHR-swapchain-04997
  swapchain **must** not be in the retired state
- VUID-vkWaitForPresentKHR-presentWait-06234
  The presentWait feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkWaitForPresentKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkWaitForPresentKHR-swapchain-parameter
  swapchain **must** be a valid VkSwapchainKHR handle
- VUID-vkWaitForPresentKHR-swapchain-parent
  swapchain **must** have been created, allocated, or retrieved from device

Host Synchronization

- Host access to swapchain **must** be externally synchronized

Return Codes

Success

- VK_SUCCESS
- VK_TIMEOUT
- VK_SUBOPTIMAL_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_DEVICE_LOST
- VK_ERROR_OUT_OF_DATE_KHR
- VK_ERROR_SURFACE_LOST_KHR
- VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT

32.10. HDR Metadata

This section describes how to improve color reproduction of content to better reproduce colors as seen on the display used to originally optimize the content.

To provide HDR metadata to an implementation, call:

```
// Provided by VK_EXT_hdr_metadata
void vkSetHdrMetadataEXT(
    VkDevice                                    device,
    uint32_t                                    swapchainCount,
    const VkSwapchainKHR*                       pSwapchains,
    const VkHdrMetadataEXT*                     pMetadata);
```

- device is the logical device where the swapchain(s) were created.
- swapchainCount is the number of swapchains included in pSwapchains.
- pSwapchains is a pointer to an array of swapchainCount VkSwapchainKHR handles.
- pMetadata is a pointer to an array of swapchainCount VkHdrMetadataEXT structures.

The metadata will be applied to the specified VkSwapchainKHR objects at the next vkQueuePresentKHR call using that VkSwapchainKHR object. The metadata will persist until a subsequent vkSetHdrMetadataEXT changes it.

Valid Usage (Implicit)

- VUID-vkSetHdrMetadataEXT-device-parameter
  device **must** be a valid VkDevice handle

- VUID-vkSetHdrMetadataEXT-pSwapchains-parameter
  `pSwapchains` **must** be a valid pointer to an array of `swapchainCount` valid VkSwapchainKHR handles
- VUID-vkSetHdrMetadataEXT-pMetadata-parameter
  `pMetadata` **must** be a valid pointer to an array of `swapchainCount` valid VkHdrMetadataEXT structures
- VUID-vkSetHdrMetadataEXT-swapchainCount-arraylength
  `swapchainCount` **must** be greater than `0`
- VUID-vkSetHdrMetadataEXT-pSwapchains-parent
  Each element of `pSwapchains` **must** have been created, allocated, or retrieved from `device`

The `VkHdrMetadataEXT` structure is defined as:

```
// Provided by VK_EXT_hdr_metadata
typedef struct VkHdrMetadataEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkXYColorEXT       displayPrimaryRed;
    VkXYColorEXT       displayPrimaryGreen;
    VkXYColorEXT       displayPrimaryBlue;
    VkXYColorEXT       whitePoint;
    float              maxLuminance;
    float              minLuminance;
    float              maxContentLightLevel;
    float              maxFrameAverageLightLevel;
} VkHdrMetadataEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `displayPrimaryRed` is a VkXYColorEXT structure specifying the red primary of the display used to optimize the content
- `displayPrimaryGreen` is a VkXYColorEXT structure specifying the green primary of the display used to optimize the content
- `displayPrimaryBlue` is a VkXYColorEXT structure specifying the blue primary of the display used to optimize the content
- `whitePoint` is a VkXYColorEXT structure specifying the white-point of the display used to optimize the content
- `maxLuminance` is the maximum luminance of the display used to optimize the content in nits
- `minLuminance` is the minimum luminance of the display used to optimize the content in nits
- `maxContentLightLevel` is the value in nits of the desired luminance for the brightest pixels in the displayed image.
- `maxFrameAverageLightLevel` is the value in nits of the average luminance of the frame which has

the brightest average luminance anywhere in the content.

If any of the above values are unknown, they **can** be set to 0.

NOTE	The meta-data provided here is intended to be used as defined in the SMPTE 2086, CTA 861.3 and CIE 15:2004 specifications. The validity and use of this data is outside the scope of Vulkan.

Valid Usage (Implicit)

- VUID-VkHdrMetadataEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_HDR_METADATA_EXT`
- VUID-VkHdrMetadataEXT-pNext-pNext
  `pNext` **must** be `NULL`

The `VkXYColorEXT` structure is defined as:

```
// Provided by VK_EXT_hdr_metadata
typedef struct VkXYColorEXT {
    float    x;
    float    y;
} VkXYColorEXT;
```

- `x` is the x chromaticity coordinate.
- `y` is the y chromaticity coordinate.

Chromaticity coordinates are as specified in CIE 15:2004 “Calculation of chromaticity coordinates” (Section 7.3) and are limited to between 0 and 1 for real colors.

Chapter 33. Deferred Host Operations

Certain Vulkan commands are inherently expensive for the host CPU to execute. It is often desirable to offload such work onto background threads, and to parallelize the work across multiple CPUs. The concept of *deferred operations* allows applications and drivers to coordinate the execution of expensive host commands using an application-managed thread pool.

The VK_KHR_deferred_host_operations extension defines the infrastructure and usage patterns for *deferrable commands*, but does not specify any commands as deferrable. This is left to additional dependent extensions. Commands **must** not be deferred unless the deferral is specifically allowed by another extension which depends on VK_KHR_deferred_host_operations. This specification will refer to such extensions as *deferral extensions*.

33.1. Requesting Deferral

When an application requests an operation deferral, the implementation **may** defer the operation. When deferral is requested and the implementation defers any operation, the implementation **must** return VK_OPERATION_DEFERRED_KHR as the success code if no errors occurred. When deferral is requested, the implementation **should** defer the operation when the workload is significant, however if the implementation chooses not to defer any of the requested operations and instead executes all of them immediately, the implementation **must** return VK_OPERATION_NOT_DEFERRED_KHR as the success code if no errors occurred.

A deferred operation is created *complete* with an initial result value of VK_SUCCESS. The deferred operation becomes *pending* when an operation has been successfully deferred with that deferred operation object.

A deferred operation is considered pending until the deferred operation completes. A pending deferred operation becomes *complete* when it has been fully executed by one or more threads. Pending deferred operations will never complete until they are *joined* by an application thread, using vkDeferredOperationJoinKHR. Applications **can** join multiple threads to the same deferred operation, enabling concurrent execution of subtasks within that operation.

The application **can** query the status of a VkDeferredOperationKHR using the vkGetDeferredOperationMaxConcurrencyKHR or vkGetDeferredOperationResultKHR commands.

Parameters to the command requesting a deferred operation **may** be accessed by the implementation at any time until the deferred operation enters the complete state. The application **must** obey the following rules while a deferred operation is pending:

- Externally synchronized parameters **must** not be accessed.
- Pointer parameters **must** not be modified (e.g. reallocated/freed).
- The contents of pointer parameters which **may** be read by the command **must** not be modified.
- The contents of pointer parameters which **may** be written by the command **must** not be read.
- Vulkan object parameters **must** not be passed as externally synchronized parameters to any other command.

When the deferred operation is complete, the application **should** call vkGetDeferredOperationResultKHR to obtain the VkResult indicating success or failure of the operation. The VkResult value returned will be one of the values that the command requesting the deferred operation is able to return. Writes to output parameters of the requesting command will happen-before the deferred operation is complete.

When a deferral is requested for a command, the implementation **may** perform memory management operations on the allocator supplied to vkCreateDeferredOperationKHR for the deferred operation object, as described in the Memory Allocation chapter. Such allocations **must** occur on the thread which requests deferral.

If an allocator was supplied for the deferred command at the time of the deferral request, then the implementation **may** perform memory management operations on this allocator during the execution of vkDeferredOperationJoinKHR. These operations **may** occur concurrently and **may** be performed by any joined thread. The application **must** ensure that the supplied allocator is able to operate correctly under these conditions.

33.2. Deferred Host Operations API

The VkDeferredOperationKHR handle is defined as:

```
// Provided by VK_KHR_deferred_host_operations
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDeferredOperationKHR)
```

This handle refers to a tracking structure which manages the execution state for a deferred command.

To construct the tracking object for a deferred command, call:

```
// Provided by VK_KHR_deferred_host_operations
VkResult vkCreateDeferredOperationKHR(
    VkDevice                                    device,
    const VkAllocationCallbacks*                pAllocator,
    VkDeferredOperationKHR*                     pDeferredOperation);
```

- `device` is the device which owns `operation`.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pDeferredOperation` is a pointer to a handle in which the created VkDeferredOperationKHR is returned.

Valid Usage (Implicit)

- VUID-vkCreateDeferredOperationKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCreateDeferredOperationKHR-pAllocator-parameter

If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure

- VUID-vkCreateDeferredOperationKHR-pDeferredOperation-parameter
`pDeferredOperation` **must** be a valid pointer to a VkDeferredOperationKHR handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

To assign a thread to a deferred operation, call:

```
// Provided by VK_KHR_deferred_host_operations
VkResult vkDeferredOperationJoinKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      operation);
```

- `device` is the device which owns `operation`.
- `operation` is the deferred operation that the calling thread should work on.

The `vkDeferredOperationJoinKHR` command will execute a portion of the deferred operation on the calling thread.

The return value will be one of the following:

- A return value of `VK_SUCCESS` indicates that `operation` is complete. The application **should** use vkGetDeferredOperationResultKHR to retrieve the result of `operation`.
- A return value of `VK_THREAD_DONE_KHR` indicates that the deferred operation is not complete, but there is no work remaining to assign to threads. Future calls to vkDeferredOperationJoinKHR are not necessary and will simply harm performance. This situation **may** occur when other threads executing vkDeferredOperationJoinKHR are about to complete `operation`, and the implementation is unable to partition the workload any further.
- A return value of `VK_THREAD_IDLE_KHR` indicates that the deferred operation is not complete, and there is no work for the thread to do at the time of the call. This situation **may** occur if the operation encounters a temporary reduction in parallelism. By returning `VK_THREAD_IDLE_KHR`, the implementation is signaling that it expects that more opportunities for parallelism will emerge as execution progresses, and that future calls to vkDeferredOperationJoinKHR **can** be beneficial. In the meantime, the application **can** perform other work on the calling thread.

Implementations **must** guarantee forward progress by enforcing the following invariants:

1. If only one thread has invoked vkDeferredOperationJoinKHR on a given operation, that thread

must execute the operation to completion and return VK_SUCCESS.

2. If multiple threads have concurrently invoked vkDeferredOperationJoinKHR on the same operation, then at least one of them **must** complete the operation and return VK_SUCCESS.

Valid Usage (Implicit)

- VUID-vkDeferredOperationJoinKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkDeferredOperationJoinKHR-operation-parameter
  operation **must** be a valid VkDeferredOperationKHR handle
- VUID-vkDeferredOperationJoinKHR-operation-parent
  operation **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS
- VK_THREAD_DONE_KHR
- VK_THREAD_IDLE_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

When a deferred operation is completed, the application **can** destroy the tracking object by calling:

```
// Provided by VK_KHR_deferred_host_operations
void vkDestroyDeferredOperationKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      operation,
    const VkAllocationCallbacks*                pAllocator);
```

- device is the device which owns operation.
- operation is the completed operation to be destroyed.
- pAllocator controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyDeferredOperationKHR-operation-03434
  If VkAllocationCallbacks were provided when operation was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyDeferredOperationKHR-operation-03435

If no VkAllocationCallbacks were provided when operation was created, pAllocator **must** be NULL

- VUID-vkDestroyDeferredOperationKHR-operation-03436
  operation **must** be completed

Valid Usage (Implicit)

- VUID-vkDestroyDeferredOperationKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkDestroyDeferredOperationKHR-operation-parameter
  If operation is not VK_NULL_HANDLE, operation **must** be a valid VkDeferredOperationKHR handle
- VUID-vkDestroyDeferredOperationKHR-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyDeferredOperationKHR-operation-parent
  If operation is a valid handle, it **must** have been created, allocated, or retrieved from device

Host Synchronization

- Host access to operation **must** be externally synchronized

To query the number of additional threads that can usefully be joined to a deferred operation, call:

```
// Provided by VK_KHR_deferred_host_operations
uint32_t vkGetDeferredOperationMaxConcurrencyKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      operation);
```

- device is the device which owns operation.
- operation is the deferred operation to be queried.

The returned value is the maximum number of threads that can usefully execute a deferred operation concurrently, reported for the state of the deferred operation at the point this command is called. This value is intended to be used to better schedule work onto available threads. Applications **can** join any number of threads to the deferred operation and expect it to eventually complete, though excessive joins **may** return VK_THREAD_DONE_KHR immediately, performing no useful work.

If operation is complete, vkGetDeferredOperationMaxConcurrencyKHR returns zero.

If operation is currently joined to any threads, the value returned by this command **may**

immediately be out of date.

If `operation` is pending, implementations **must** not return zero unless at least one thread is currently executing vkDeferredOperationJoinKHR on `operation`. If there are such threads, the implementation **should** return an estimate of the number of additional threads which it could profitably use.

Implementations **may** return 2^{32}-1 to indicate that the maximum concurrency is unknown and cannot be easily derived. Implementations **may** return values larger than the maximum concurrency available on the host CPU. In these situations, an application **should** clamp the return value rather than oversubscribing the machine.

NOTE The recommended usage pattern for applications is to query this value once, after deferral, and schedule no more than the specified number of threads to join the operation. Each time a joined thread receives `VK_THREAD_IDLE_KHR`, the application should schedule an additional join at some point in the future, but is not required to do so.

Valid Usage (Implicit)

- VUID-vkGetDeferredOperationMaxConcurrencyKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeferredOperationMaxConcurrencyKHR-operation-parameter
  `operation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkGetDeferredOperationMaxConcurrencyKHR-operation-parent
  `operation` **must** have been created, allocated, or retrieved from `device`

The `vkGetDeferredOperationResultKHR` function is defined as:

```
// Provided by VK_KHR_deferred_host_operations
VkResult vkGetDeferredOperationResultKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      operation);
```

- `device` is the device which owns `operation`.
- `operation` is the operation whose deferred result is being queried.

If no command has been deferred on `operation`, `vkGetDeferredOperationResultKHR` returns `VK_SUCCESS`.

If the deferred operation is pending, `vkGetDeferredOperationResultKHR` returns `VK_NOT_READY`.

If the deferred operation is complete, it returns the appropriate return value from the original command. This value **must** be one of the VkResult values which could have been returned by the original command if the operation had not been deferred.

Valid Usage (Implicit)

- VUID-vkGetDeferredOperationResultKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeferredOperationResultKHR-operation-parameter
  `operation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkGetDeferredOperationResultKHR-operation-parent
  `operation` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_NOT_READY`

Failure

None

Chapter 34. Acceleration Structures

34.1. Acceleration Structures

Acceleration structures are data structures used by the implementation to efficiently manage scene geometry as it is traversed during a ray tracing query. The application is responsible for managing acceleration structure objects (see Acceleration Structures), including allocation, destruction, executing builds or updates, and synchronizing resources used during ray tracing queries.

There are two types of acceleration structures, *top level acceleration structures* and *bottom level acceleration structures*.

An acceleration structure is considered to be constructed if an acceleration structure build command or copy command has been executed with the given acceleration structure as the destination.

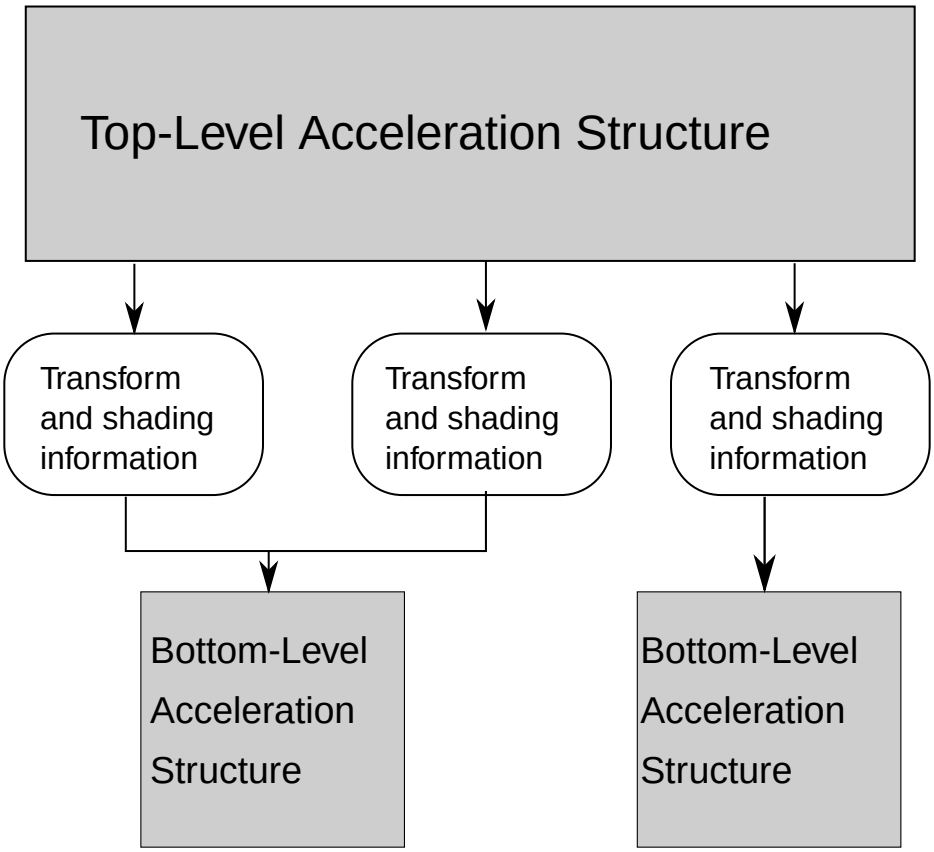


Figure 22. Acceleration Structure

Caption

The diagram shows the relationship between top and bottom level acceleration structures.

34.1.1. Geometry

Geometries refer to a triangle or axis-aligned bounding box.

34.1.2. Top Level Acceleration Structures

Opaque acceleration structure for an array of instances. The descriptor or device address referencing this is the starting point for traversal.

The top level acceleration structure takes a reference to any bottom level acceleration structure referenced by its instances. Those bottom level acceleration structure objects **must** be valid when the top level acceleration structure is accessed.

34.1.3. Bottom Level Acceleration Structures

Opaque acceleration structure for an array of geometries.

34.1.4. Acceleration Structure Update Rules

The API defines two types of operations to produce acceleration structures from geometry:

- A *build* operation is used to construct an acceleration structure.
- An *update* operation is used to modify an existing acceleration structure.

An update operation imposes certain constraints on the input, in exchange for considerably faster execution. When performing an update, the application is required to provide a full description of the acceleration structure, but is prohibited from changing anything other than instance definitions, transform matrices, and vertex or AABB positions. All other aspects of the description **must** exactly match the one from the original build.

More precisely, the application **must** not use an update operation to do any of the following:

- Change primitives or instances from *active* to *inactive*, or vice versa (as defined in Inactive Primitives and Instances).
- Change the index or vertex formats of triangle geometry.
- Change triangle geometry transform pointers from null to non-null or vice versa.
- Change the number of geometries or instances in the structure.
- Change the geometry flags for any geometry in the structure.
- Change the number of vertices or primitives for any geometry in the structure.

If the original acceleration structure was built using opacity micromaps and `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_DATA_UPDATE_EXT` was set in `flags`, the application **must** provide the corresponding micromap information to the update operation. The application is prohibited from changing anything other than the specific opacity values assigned to the triangles.

More precisely, the application **must** not use an update operation to do any of the following:

- Remove micromaps or VkOpacityMicromapSpecialIndexEXT values from a geometry which previously had them, or vice versa.
- Change between use of VkOpacityMicromapSpecialIndexEXT values and explicit micro-map

triangles.

- Change the subdivision level or format of the micromap triangle associated with any acceleration-structure triangle.

If the original acceleration structure was built using opacity micromaps and `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_UPDATE_EXT` was set in `flags`, the application **must** provide a micromap to the update operation.

If the original acceleration structure was built using opacity micromaps and neither opacity micromap update flag is set the application **must** provide the original micromap to the update operation.

34.1.5. Inactive Primitives and Instances

Acceleration structures allow the use of particular input values to signal *inactive* primitives or instances.

An *inactive* triangle is one for which the first (X) component of any vertex is NaN. If any other vertex component is NaN, and the first is not, the behavior is undefined. If the vertex format does not have a NaN representation, then all triangles are considered active.

An *inactive* instance is one whose acceleration structure reference is `0`.

An *inactive* AABB is one for which the minimum X coordinate is NaN. If any other component is NaN, and the first is not, the behavior is undefined.

In the above definitions, “NaN” refers to any type of NaN. Signaling, non-signaling, quiet, loud, or otherwise.

An inactive object is considered invisible to all rays, and **should** not be represented in the acceleration structure. Implementations **should** ensure that the presence of inactive objects does not seriously degrade traversal performance.

Inactive objects are counted in the auto-generated index sequences which are provided to shaders via `InstanceId` and `PrimitiveId` SPIR-V decorations. This allows objects in the scene to change freely between the active and inactive states, without affecting the layout of any arrays which are being indexed using the ID values.

Any transition between the active and inactive states requires a full acceleration structure rebuild. Applications **must** not perform an acceleration structure update where an object is active in the source acceleration structure but would be inactive in the destination, or vice versa.

34.1.6. Degenerate Primitives and Instances

Degenerate primitives and instances behave differently to inactive primitives and instances, and are defined as:

- triangles that have one or more vertices whose respective (X), (Y), (Z) components are identical, or have three vertices that have at least two of the (X), (Y), or (Z) components identical, therefore forming a line or point. Degenerate triangles do not generate any intersections.

- AABBs whose `minX=maxX`, `minY=maxY`, and `minZ=maxZ`. Degenerate AABBs **may** invoke the intersection shader.
- instances that reference bottom level acceleration structures that contain no active primitives. When building an acceleration structure, implementations **should** treat degenerate instances as though they are a point at the instance origin, specified by VkAccelerationStructureInstanceKHR::`transform`.

Unlike *inactive* primitives and instances, *degenerate* primitives and instances **may** transition from the degenerate to the non-degenerate state, or vice versa, when performing an acceleration structure update.

If an acceleration structure is built without `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_UPDATE_BIT_KHR` set in VkAccelerationStructureBuildGeometryInfoKHR::`flags` , degenerate primitives **may** be discarded. Primitives that are defined with the same index value for more than one vertex **can** always be discarded.

34.1.7. Building Acceleration Structures

To build acceleration structures call:

```
// Provided by VK_KHR_acceleration_structure
void vkCmdBuildAccelerationStructuresKHR(
    VkCommandBuffer                                  commandBuffer,
    uint32_t                                         infoCount,
    const VkAccelerationStructureBuildGeometryInfoKHR* pInfos,
    const VkAccelerationStructureBuildRangeInfoKHR* const* ppBuildRangeInfos);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `infoCount` is the number of acceleration structures to build. It specifies the number of the `pInfos` structures and `ppBuildRangeInfos` pointers that **must** be provided.
- `pInfos` is a pointer to an array of `infoCount` VkAccelerationStructureBuildGeometryInfoKHR structures defining the geometry used to build each acceleration structure.
- `ppBuildRangeInfos` is a pointer to an array of `infoCount` pointers to arrays of VkAccelerationStructureBuildRangeInfoKHR structures. Each `ppBuildRangeInfos`[i] is a pointer to an array of `pInfos`[i].`geometryCount` VkAccelerationStructureBuildRangeInfoKHR structures defining dynamic offsets to the addresses where geometry data is stored, as defined by `pInfos`[i].

The `vkCmdBuildAccelerationStructuresKHR` command provides the ability to initiate multiple acceleration structures builds, however there is no ordering or synchronization implied between any of the individual acceleration structure builds.

NOTE

This means that an application **cannot** build a top-level acceleration structure in the same vkCmdBuildAccelerationStructuresKHR call as the associated bottom-level or instance acceleration structures are being built. There also **cannot** be any memory aliasing between any acceleration structure memories or scratch memories being used by any of the builds.

Accesses to the acceleration structure scratch buffers as identified by the VkAccelerationStructureBuildGeometryInfoKHR::scratchData buffer device addresses **must** be synchronized with the VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stage and an access type of (VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR | VK_ACCESS_ACCELERATION_STRUCTURE_WRITE_BIT_KHR). Accesses to each VkAccelerationStructureBuildGeometryInfoKHR::srcAccelerationStructure and VkAccelerationStructureBuildGeometryInfoKHR::dstAccelerationStructure **must** be synchronized with the VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stage and an access type of VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR or VK_ACCESS_ACCELERATION_STRUCTURE_WRITE_BIT_KHR, as appropriate.

Accesses to other input buffers as identified by any used values of VkAccelerationStructureGeometryTrianglesDataKHR::vertexData, VkAccelerationStructureGeometryTrianglesDataKHR::indexData, VkAccelerationStructureGeometryTrianglesDataKHR::transformData, VkAccelerationStructureGeometryAabbsDataKHR::data, and VkAccelerationStructureGeometryInstancesDataKHR::data **must** be synchronized with the VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stage and an access type of VK_ACCESS_SHADER_READ_BIT.

Valid Usage

- VUID-vkCmdBuildAccelerationStructuresKHR-accelerationStructure-08923
  The VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure feature **must** be enabled
- VUID-vkCmdBuildAccelerationStructuresKHR-mode-04628
  The mode member of each element of pInfos **must** be a valid VkBuildAccelerationStructureModeKHR value
- VUID-vkCmdBuildAccelerationStructuresKHR-srcAccelerationStructure-04629
  If the srcAccelerationStructure member of any element of pInfos is not VK_NULL_HANDLE, the srcAccelerationStructure member **must** be a valid VkAccelerationStructureKHR handle
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-04630
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its srcAccelerationStructure member **must** not be VK_NULL_HANDLE
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03403
  The srcAccelerationStructure member of any element of pInfos **must** not be the same acceleration structure as the dstAccelerationStructure member of any other element of pInfos
- VUID-vkCmdBuildAccelerationStructuresKHR-dstAccelerationStructure-03698
  The dstAccelerationStructure member of any element of pInfos **must** not be the same acceleration structure as the dstAccelerationStructure member of any other element of pInfos
- VUID-vkCmdBuildAccelerationStructuresKHR-dstAccelerationStructure-03800

The dstAccelerationStructure member of any element of pInfos **must** be a valid VkAccelerationStructureKHR handle

- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03699
  For each element of pInfos, if its type member is VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR, its dstAccelerationStructure member **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::type equal to either VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR or VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03700
  For each element of pInfos, if its type member is VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR, its dstAccelerationStructure member **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::type equal to either VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR or VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03663
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, inactive primitives in its srcAccelerationStructure member **must** not be made active
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03664
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, active primitives in its srcAccelerationStructure member **must** not be made inactive
- VUID-vkCmdBuildAccelerationStructuresKHR-None-03407
  The dstAccelerationStructure member of any element of pInfos **must** not be referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR in any other element of pInfos
- VUID-vkCmdBuildAccelerationStructuresKHR-dstAccelerationStructure-03701
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the srcAccelerationStructure member of any other element of pInfos with a mode equal to VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresKHR-dstAccelerationStructure-03702
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the dstAccelerationStructure member of any other element of pInfos, which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresKHR-dstAccelerationStructure-03703
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the scratchData member of any element of pInfos (including the same element), which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresKHR-scratchData-03704
  The range of memory backing the scratchData member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the scratchData

member of any other element of pInfos, which is accessed by this command

- VUID-vkCmdBuildAccelerationStructuresKHR-scratchData-03705
  The range of memory backing the scratchData member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the srcAccelerationStructure member of any element of pInfos with a mode equal to VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR (including the same element), which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresKHR-dstAccelerationStructure-03706
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing any acceleration structure referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR in any other element of pInfos, which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03667
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its srcAccelerationStructure member **must** have previously been constructed with VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_UPDATE_BIT_KHR set in VkAccelerationStructureBuildGeometryInfoKHR::flags in the build
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03668
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its srcAccelerationStructure and dstAccelerationStructure members **must** either be the same VkAccelerationStructureKHR, or not have any memory aliasing
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03758
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its geometryCount member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03759
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its flags member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03760
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its type member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03761
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, its geometryType member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03762
  For each element of pInfos, if its mode member is

VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, its flags member **must** have the same value which was specified when srcAccelerationStructure was last built

- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03763
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its geometry.triangles.vertexFormat member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03764
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its geometry.triangles.maxVertex member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03765
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its geometry.triangles.indexType member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03766
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, if its geometry.triangles.transformData address was NULL when srcAccelerationStructure was last built, then it **must** be NULL
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03767
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, if its geometry.triangles.transformData address was not NULL when srcAccelerationStructure was last built, then it **must** not be NULL
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03768
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, and geometry.triangles.indexType is not VK_INDEX_TYPE_NONE_KHR, then the value of each index

referenced **must** be the same as the corresponding index value when `srcAccelerationStructure` was last built

- VUID-vkCmdBuildAccelerationStructuresKHR-primitiveCount-03769
  For each element of `pInfos`, if its `mode` member is `VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR`, then for each `VkAccelerationStructureGeometryKHR` structure referred to by its `pGeometries` or `ppGeometries` members, the `primitiveCount` member of its corresponding `VkAccelerationStructureBuildRangeInfoKHR` structure **must** have the same value which was specified when `srcAccelerationStructure` was last built
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03801
  For each element of `pInfos`[i].`pGeometries` or `pInfos`[i].`ppGeometries` with a `geometryType` of `VK_GEOMETRY_TYPE_INSTANCES_KHR`, the corresponding `ppBuildRangeInfos`[i][j].`primitiveCount` **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR ::`maxInstanceCount`
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03707
  For each element of `pInfos`, the `buffer` used to create its `dstAccelerationStructure` member **must** be bound to device memory
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03708
  For each element of `pInfos`, if its `mode` member is `VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR` the `buffer` used to create its `srcAccelerationStructure` member **must** be bound to device memory
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03709
  For each element of `pInfos`, the `buffer` used to create each acceleration structure referenced by the `geometry.instances.data` member of any element of `pGeometries` or `ppGeometries` with a `geometryType` of `VK_GEOMETRY_TYPE_INSTANCES_KHR` **must** be bound to device memory
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03671
  If `pInfos`[i].`mode` is `VK_BUILD_ACCELERATION_STRUCTURE_MODE_BUILD_KHR`, all addresses between `pInfos`[i].`scratchData.deviceAddress` and `pInfos`[i].`scratchData.deviceAddress` + N - 1 **must** be in the buffer device address range of the same buffer, where N is given by the `buildScratchSize` member of the VkAccelerationStructureBuildSizesInfoKHR structure returned from a call to vkGetAccelerationStructureBuildSizesKHR with an identical VkAccelerationStructureBuildGeometryInfoKHR structure and primitive count
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03672
  If `pInfos`[i].`mode` is `VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR`, all addresses between `pInfos`[i].`scratchData.deviceAddress` and `pInfos`[i].`scratchData.deviceAddress` + N - 1 **must** be in the buffer device address range of the same buffer, where N is given by the `updateScratchSize` member of the VkAccelerationStructureBuildSizesInfoKHR structure returned from a call to vkGetAccelerationStructureBuildSizesKHR with an identical VkAccelerationStructureBuildGeometryInfoKHR structure and primitive count
- VUID-vkCmdBuildAccelerationStructuresKHR-geometry-03673
  The buffers from which the buffer device addresses for all of the `geometry.triangles.vertexData`, `geometry.triangles.indexData`, `geometry.triangles.transformData`, `geometry.aabbs.data`, and `geometry.instances.data`

members of all pInfos[i].pGeometries and pInfos[i].ppGeometries are queried **must** have been created with the VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR usage flag

- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03674
  The buffer from which the buffer device address pInfos[i].scratchData.deviceAddress is queried **must** have been created with VK_BUFFER_USAGE_STORAGE_BUFFER_BIT usage flag
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03802
  For each element of pInfos, its scratchData.deviceAddress member **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03803
  For each element of pInfos, if scratchData.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03710
  For each element of pInfos, its scratchData.deviceAddress member **must** be a multiple of VkPhysicalDeviceAccelerationStructurePropertiesKHR::minAccelerationStructureScratchOffsetAlignment
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03804
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, geometry.triangles.vertexData.deviceAddress **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03805
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.vertexData.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03711
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, geometry.triangles.vertexData.deviceAddress **must** be aligned to the size in bytes of the smallest component of the format in vertexFormat
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03806
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.indexType is not VK_INDEX_TYPE_NONE_KHR, geometry.triangles.indexData.deviceAddress **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03807
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.indexType is not VK_INDEX_TYPE_NONE_KHR, if geometry.triangles.indexData.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03712
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, and with geometry.triangles.indexType not equal to

VK_INDEX_TYPE_NONE_KHR, geometry.triangles.indexData.deviceAddress **must** be aligned to the size in bytes of the type in indexType

- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03808
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.transformData.deviceAddress is not 0, it **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03809
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.transformData.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03810
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.transformData.deviceAddress is not 0, it **must** be aligned to 16 bytes
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03811
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR, geometry.aabbs.data.deviceAddress **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03812
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR, if geometry.aabbs.data.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03714
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR, geometry.aabbs.data.deviceAddress **must** be aligned to 8 bytes
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03715
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, if geometry.arrayOfPointers is VK_FALSE, geometry.instances.data.deviceAddress **must** be aligned to 16 bytes
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03716
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, if geometry.arrayOfPointers is VK_TRUE, geometry.instances.data.deviceAddress **must** be aligned to 8 bytes
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03717
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, if geometry.arrayOfPointers is VK_TRUE, each element of geometry.instances.data.deviceAddress in device memory **must** be aligned to 16 bytes
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03813
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, geometry.instances.data.deviceAddress **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR

- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-03814
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, if geometry.instances.data.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-06707
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, each VkAccelerationStructureInstanceKHR ::accelerationStructureReference value in geometry.instances.data.deviceAddress **must** be a valid device address containing a value obtained from vkGetAccelerationStructureDeviceAddressKHR or 0
- VUID-vkCmdBuildAccelerationStructuresKHR-commandBuffer-09547
  commandBuffer **must** not be a protected command buffer
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-10126
  For each pInfos[i], dstAccelerationStructure **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::size greater than or equal to either:
  - the memory size required by the build operation, as returned by vkGetAccelerationStructureBuildSizesKHR with pBuildInfo = pInfos[i] and with each element of the pMaxPrimitiveCounts array greater than or equal to the equivalent ppBuildRangeInfos[i][j].primitiveCount values for j in [0,pInfos[i].geometryCount) or,
  - the result of querying the corresponding VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR, if updating a compacted acceleration structure
- VUID-vkCmdBuildAccelerationStructuresKHR-ppBuildRangeInfos-03676
  Each element of ppBuildRangeInfos[i] **must** be a valid pointer to an array of pInfos [i].geometryCount VkAccelerationStructureBuildRangeInfoKHR structures

Valid Usage (Implicit)

- VUID-vkCmdBuildAccelerationStructuresKHR-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBuildAccelerationStructuresKHR-pInfos-parameter
  pInfos **must** be a valid pointer to an array of infoCount valid VkAccelerationStructureBuildGeometryInfoKHR structures
- VUID-vkCmdBuildAccelerationStructuresKHR-ppBuildRangeInfos-parameter
  ppBuildRangeInfos **must** be a valid pointer to an array of infoCount VkAccelerationStructureBuildRangeInfoKHR structures
- VUID-vkCmdBuildAccelerationStructuresKHR-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdBuildAccelerationStructuresKHR-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support compute operations

- VUID-vkCmdBuildAccelerationStructuresKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBuildAccelerationStructuresKHR-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBuildAccelerationStructuresKHR-infoCount-arraylength
  `infoCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

To build acceleration structures with some parameters sourced on the device call:

```
// Provided by VK_KHR_acceleration_structure
void vkCmdBuildAccelerationStructuresIndirectKHR(
    VkCommandBuffer                                 commandBuffer,
    uint32_t                                        infoCount,
    const VkAccelerationStructureBuildGeometryInfoKHR* pInfos,
    const VkDeviceAddress*                          pIndirectDeviceAddresses,
    const uint32_t*                                 pIndirectStrides,
    const uint32_t* const*                          ppMaxPrimitiveCounts);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `infoCount` is the number of acceleration structures to build.
- `pInfos` is a pointer to an array of `infoCount` VkAccelerationStructureBuildGeometryInfoKHR structures defining the geometry used to build each acceleration structure.
- `pIndirectDeviceAddresses` is a pointer to an array of `infoCount` buffer device addresses which point to `pInfos`[i].`geometryCount` VkAccelerationStructureBuildRangeInfoKHR structures defining dynamic offsets to the addresses where geometry data is stored, as defined by `pInfos`[i].
- `pIndirectStrides` is a pointer to an array of `infoCount` byte strides between elements of `pIndirectDeviceAddresses`.
- `ppMaxPrimitiveCounts` is a pointer to an array of `infoCount` pointers to arrays of `pInfos`

[i].geometryCount values indicating the maximum number of primitives that will be built by this command for each geometry.

Accesses to acceleration structures, scratch buffers, vertex buffers, index buffers, and instance buffers **must** be synchronized as with vkCmdBuildAccelerationStructuresKHR.

Accesses to any element of pIndirectDeviceAddresses **must** be synchronized with the VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR pipeline stage and an access type of VK_ACCESS_INDIRECT_COMMAND_READ_BIT.

Valid Usage

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-accelerationStructureIndirectBuild-03650
  The VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructureIndirectBuild feature **must** be enabled
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-mode-04628
  The mode member of each element of pInfos **must** be a valid VkBuildAccelerationStructureModeKHR value
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-srcAccelerationStructure-04629
  If the srcAccelerationStructure member of any element of pInfos is not VK_NULL_HANDLE, the srcAccelerationStructure member **must** be a valid VkAccelerationStructureKHR handle
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-04630
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its srcAccelerationStructure member **must** not be VK_NULL_HANDLE
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03403
  The srcAccelerationStructure member of any element of pInfos **must** not be the same acceleration structure as the dstAccelerationStructure member of any other element of pInfos
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-dstAccelerationStructure-03698
  The dstAccelerationStructure member of any element of pInfos **must** not be the same acceleration structure as the dstAccelerationStructure member of any other element of pInfos
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-dstAccelerationStructure-03800
  The dstAccelerationStructure member of any element of pInfos **must** be a valid VkAccelerationStructureKHR handle
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03699
  For each element of pInfos, if its type member is VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR, its dstAccelerationStructure member **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::type equal to either VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR or VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03700
  For each element of pInfos, if its type member is VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR, its dstAccelerationStructure member **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::type equal to either VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR or VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03663
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, inactive primitives in its srcAccelerationStructure member **must** not be made active
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03664
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, active primitives in its srcAccelerationStructure member **must** not be made inactive
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-None-03407
  The dstAccelerationStructure member of any element of pInfos **must** not be referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR in any other element of pInfos
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-dstAccelerationStructure-03701
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the srcAccelerationStructure member of any other element of pInfos with a mode equal to VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-dstAccelerationStructure-03702
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the dstAccelerationStructure member of any other element of pInfos, which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-dstAccelerationStructure-03703
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the scratchData member of any element of pInfos (including the same element), which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-scratchData-03704
  The range of memory backing the scratchData member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the scratchData member of any other element of pInfos, which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-scratchData-03705
  The range of memory backing the scratchData member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the srcAccelerationStructure member of any element of pInfos with a mode equal to VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR (including the same element), which is accessed by this command
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-dstAccelerationStructure-03706

The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing any acceleration structure referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR in any other element of pInfos, which is accessed by this command

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03667
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its srcAccelerationStructure member **must** have previously been constructed with VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_UPDATE_BIT_KHR set in VkAccelerationStructureBuildGeometryInfoKHR::flags in the build
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03668
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its srcAccelerationStructure and dstAccelerationStructure members **must** either be the same VkAccelerationStructureKHR, or not have any memory aliasing
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03758
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its geometryCount member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03759
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its flags member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03760
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its type member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03761
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, its geometryType member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03762
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, its flags member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03763
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its

geometry.triangles.vertexFormat member **must** have the same value which was specified when srcAccelerationStructure was last built

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03764
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its geometry.triangles.maxVertex member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03765
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its geometry.triangles.indexType member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03766
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, if its geometry.triangles.transformData address was NULL when srcAccelerationStructure was last built, then it **must** be NULL
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03767
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, if its geometry.triangles.transformData address was not NULL when srcAccelerationStructure was last built, then it **must** not be NULL
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03768
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, and geometry.triangles.indexType is not VK_INDEX_TYPE_NONE_KHR, then the value of each index referenced **must** be the same as the corresponding index value when srcAccelerationStructure was last built
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-primitiveCount-03769
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, the primitiveCount member of its corresponding VkAccelerationStructureBuildRangeInfoKHR structure **must** have the same value which was specified when srcAccelerationStructure was last built

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03801
  For each element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, the corresponding ppMaxPrimitiveCounts[i][j] **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxInstanceCount
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03707
  For each element of pInfos, the buffer used to create its dstAccelerationStructure member **must** be bound to device memory
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03708
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR the buffer used to create its srcAccelerationStructure member **must** be bound to device memory
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03709
  For each element of pInfos, the buffer used to create each acceleration structure referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR **must** be bound to device memory
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03671
  If pInfos[i].mode is VK_BUILD_ACCELERATION_STRUCTURE_MODE_BUILD_KHR, all addresses between pInfos[i].scratchData.deviceAddress and pInfos[i].scratchData.deviceAddress + N - 1 **must** be in the buffer device address range of the same buffer, where N is given by the buildScratchSize member of the VkAccelerationStructureBuildSizesInfoKHR structure returned from a call to vkGetAccelerationStructureBuildSizesKHR with an identical VkAccelerationStructureBuildGeometryInfoKHR structure and primitive count
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03672
  If pInfos[i].mode is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, all addresses between pInfos[i].scratchData.deviceAddress and pInfos[i].scratchData.deviceAddress + N - 1 **must** be in the buffer device address range of the same buffer, where N is given by the updateScratchSize member of the VkAccelerationStructureBuildSizesInfoKHR structure returned from a call to vkGetAccelerationStructureBuildSizesKHR with an identical VkAccelerationStructureBuildGeometryInfoKHR structure and primitive count
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-geometry-03673
  The buffers from which the buffer device addresses for all of the geometry.triangles.vertexData, geometry.triangles.indexData, geometry.triangles.transformData, geometry.aabbs.data, and geometry.instances.data members of all pInfos[i].pGeometries and pInfos[i].ppGeometries are queried **must** have been created with the VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR usage flag
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03674
  The buffer from which the buffer device address pInfos[i].scratchData.deviceAddress is queried **must** have been created with VK_BUFFER_USAGE_STORAGE_BUFFER_BIT usage flag
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03802
  For each element of pInfos, its scratchData.deviceAddress member **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03803

For each element of pInfos, if scratchData.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03710
  For each element of pInfos, its scratchData.deviceAddress member **must** be a multiple of VkPhysicalDeviceAccelerationStructurePropertiesKHR::minAccelerationStructureScratchOffsetAlignment
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03804
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, geometry.triangles.vertexData.deviceAddress **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03805
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.vertexData.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03711
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, geometry.triangles.vertexData.deviceAddress **must** be aligned to the size in bytes of the smallest component of the format in vertexFormat
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03806
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.indexType is not VK_INDEX_TYPE_NONE_KHR, geometry.triangles.indexData.deviceAddress **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03807
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.indexType is not VK_INDEX_TYPE_NONE_KHR, if geometry.triangles.indexData.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03712
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, and with geometry.triangles.indexType not equal to VK_INDEX_TYPE_NONE_KHR, geometry.triangles.indexData.deviceAddress **must** be aligned to the size in bytes of the type in indexType
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03808
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.transformData.deviceAddress is not 0, it **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03809
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.transformData.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a

single VkDeviceMemory object

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03810
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.transformData.deviceAddress is not 0, it **must** be aligned to 16 bytes
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03811
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR, geometry.aabbs.data.deviceAddress **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03812
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR, if geometry.aabbs.data.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03714
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR, geometry.aabbs.data.deviceAddress **must** be aligned to 8 bytes
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03715
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, if geometry.arrayOfPointers is VK_FALSE, geometry.instances.data.deviceAddress **must** be aligned to 16 bytes
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03716
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, if geometry.arrayOfPointers is VK_TRUE, geometry.instances.data.deviceAddress **must** be aligned to 8 bytes
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03717
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, if geometry.arrayOfPointers is VK_TRUE, each element of geometry.instances.data.deviceAddress in device memory **must** be aligned to 16 bytes
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03813
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, geometry.instances.data.deviceAddress **must** be a valid device address obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03814
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, if geometry.instances.data.deviceAddress is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-06707
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, each VkAccelerationStructureInstanceKHR ::accelerationStructureReference value in geometry.instances.data.deviceAddress **must** be a valid device address containing a value obtained from vkGetAccelerationStructureDeviceAddressKHR or 0

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-commandBuffer-09547
  `commandBuffer` **must** not be a protected command buffer
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pIndirectDeviceAddresses-03645
  For any element of `pIndirectDeviceAddresses`, if the buffer from which it was queried is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pIndirectDeviceAddresses-03646
  For any element of `pIndirectDeviceAddresses`[i], all device addresses between `pIndirectDeviceAddresses`[i] and `pIndirectDeviceAddresses`[i] + (`pInfos`[i].`geometryCount` × `pIndirectStrides`[i]) - 1 **must** be in the buffer device address range of the same buffer
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pIndirectDeviceAddresses-03647
  For any element of `pIndirectDeviceAddresses`, the buffer from which it was queried **must** have been created with the `VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT` bit set
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pIndirectDeviceAddresses-03648
  Each element of `pIndirectDeviceAddresses` **must** be a multiple of `4`
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pIndirectStrides-03787
  Each element of `pIndirectStrides` **must** be a multiple of `4`
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pIndirectDeviceAddresses-03651
  Each VkAccelerationStructureBuildRangeInfoKHR structure referenced by any element of `pIndirectDeviceAddresses` **must** be a valid VkAccelerationStructureBuildRangeInfoKHR structure
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-03652
  `pInfos`[i].`dstAccelerationStructure` **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::`size` greater than or equal to the memory size required by the build operation, as returned by vkGetAccelerationStructureBuildSizesKHR with `pBuildInfo` = `pInfos`[i] and `pMaxPrimitiveCounts` = `ppMaxPrimitiveCounts`[i]
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-ppMaxPrimitiveCounts-03653
  Each `ppMaxPrimitiveCounts`[i][j] **must** be greater than or equal to the `primitiveCount` value specified by the VkAccelerationStructureBuildRangeInfoKHR structure located at `pIndirectDeviceAddresses`[i] + (`j` × `pIndirectStrides`[i])

Valid Usage (Implicit)

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pInfos-parameter
  `pInfos` **must** be a valid pointer to an array of `infoCount` valid VkAccelerationStructureBuildGeometryInfoKHR structures
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pIndirectDeviceAddresses-parameter
  `pIndirectDeviceAddresses` **must** be a valid pointer to an array of `infoCount` VkDeviceAddress values

- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-pIndirectStrides-parameter
  `pIndirectStrides` **must** be a valid pointer to an array of `infoCount` `uint32_t` values
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-ppMaxPrimitiveCounts-parameter
  `ppMaxPrimitiveCounts` **must** be a valid pointer to an array of `infoCount` `uint32_t` values
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBuildAccelerationStructuresIndirectKHR-infoCount-arraylength
  `infoCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

The `VkAccelerationStructureBuildGeometryInfoKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureBuildGeometryInfoKHR {
    VkStructureType                                    sType;
    const void*                                        pNext;
    VkAccelerationStructureTypeKHR                     type;
    VkBuildAccelerationStructureFlagsKHR               flags;
    VkBuildAccelerationStructureModeKHR                mode;
    VkAccelerationStructureKHR                         srcAccelerationStructure;
    VkAccelerationStructureKHR                         dstAccelerationStructure;
    uint32_t                                           geometryCount;
    const VkAccelerationStructureGeometryKHR*          pGeometries;
    const VkAccelerationStructureGeometryKHR* const*   ppGeometries;
    VkDeviceOrHostAddressKHR                           scratchData;
} VkAccelerationStructureBuildGeometryInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `type` is a VkAccelerationStructureTypeKHR value specifying the type of acceleration structure being built.
- `flags` is a bitmask of VkBuildAccelerationStructureFlagBitsKHR specifying additional parameters of the acceleration structure.
- `mode` is a VkBuildAccelerationStructureModeKHR value specifying the type of operation to perform.
- `srcAccelerationStructure` is a pointer to an existing acceleration structure that is to be used to update the `dstAccelerationStructure` acceleration structure when `mode` is `VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR`.
- `dstAccelerationStructure` is a pointer to the target acceleration structure for the build.
- `geometryCount` specifies the number of geometries that will be built into `dstAccelerationStructure`.
- `pGeometries` is a pointer to an array of VkAccelerationStructureGeometryKHR structures.
- `ppGeometries` is a pointer to an array of pointers to VkAccelerationStructureGeometryKHR structures.
- `scratchData` is the device or host address to memory that will be used as scratch memory for the build.

Only one of `pGeometries` or `ppGeometries` **can** be a valid pointer, the other **must** be `NULL`. Each element of the non-`NULL` array describes the data used to build each acceleration structure geometry.

The index of each element of the `pGeometries` or `ppGeometries` members of VkAccelerationStructureBuildGeometryInfoKHR is used as the *geometry index* during ray traversal. The geometry index is available in ray shaders via the `RayGeometryIndexKHR` built-in, and is used to determine hit and intersection shaders executed during traversal. The geometry index is available to ray queries via the `OpRayQueryGetIntersectionGeometryIndexKHR` instruction.

Members srcAccelerationStructure and dstAccelerationStructure **may** be the same or different for an update operation (when mode is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR). If they are the same, the update happens in-place. Otherwise, the target acceleration structure is updated and the source is not modified.

Valid Usage

- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-03654
  type **must** not be VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-pGeometries-03788
  If geometryCount is not 0, exactly one of pGeometries or ppGeometries **must** be a valid pointer, the other **must** be NULL
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-03789
  If type is VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR, the geometryType member of elements of either pGeometries or ppGeometries **must** be VK_GEOMETRY_TYPE_INSTANCES_KHR
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-03790
  If type is VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR, geometryCount **must** be 1
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-03791
  If type is VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR the geometryType member of elements of either pGeometries or ppGeometries **must** not be VK_GEOMETRY_TYPE_INSTANCES_KHR
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-03792
  If type is VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR then the geometryType member of each geometry in either pGeometries or ppGeometries **must** be the same
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-03793
  If type is VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR then geometryCount **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxGeometryCount
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-03794
  If type is VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR and the geometryType member of either pGeometries or ppGeometries is VK_GEOMETRY_TYPE_AABBS_KHR, the total number of AABBs in all geometries **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxPrimitiveCount
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-03795
  If type is VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR and the geometryType member of either pGeometries or ppGeometries is VK_GEOMETRY_TYPE_TRIANGLES_KHR, the total number of triangles in all geometries **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR::maxPrimitiveCount
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-flags-03796
  If flags has the VK_BUILD_ACCELERATION_STRUCTURE_PREFER_FAST_TRACE_BIT_KHR bit set, then it **must** not have the VK_BUILD_ACCELERATION_STRUCTURE_PREFER_FAST_BUILD_BIT_KHR bit set
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-flags-07334
  If flags has the VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_UPDATE_EXT bit

set then it **must** not have the `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_DATA_UPDATE_EXT` bit set

Valid Usage (Implicit)

- VUID-VkAccelerationStructureBuildGeometryInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_BUILD_GEOMETRY_INFO_KHR`
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-type-parameter
  `type` **must** be a valid VkAccelerationStructureTypeKHR value
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkBuildAccelerationStructureFlagBitsKHR values
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-pGeometries-parameter
  If `geometryCount` is not `0`, and `pGeometries` is not `NULL`, `pGeometries` **must** be a valid pointer to an array of `geometryCount` valid VkAccelerationStructureGeometryKHR structures
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-ppGeometries-parameter
  If `geometryCount` is not `0`, and `ppGeometries` is not `NULL`, `ppGeometries` **must** be a valid pointer to an array of `geometryCount` valid pointers to valid VkAccelerationStructureGeometryKHR structures
- VUID-VkAccelerationStructureBuildGeometryInfoKHR-commonparent
  Both of `dstAccelerationStructure`, and `srcAccelerationStructure` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The `VkBuildAccelerationStructureModeKHR` enumeration is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkBuildAccelerationStructureModeKHR {
    VK_BUILD_ACCELERATION_STRUCTURE_MODE_BUILD_KHR = 0,
    VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR = 1,
} VkBuildAccelerationStructureModeKHR;
```

- `VK_BUILD_ACCELERATION_STRUCTURE_MODE_BUILD_KHR` specifies that the destination acceleration structure will be built using the specified geometries.
- `VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR` specifies that the destination acceleration structure will be built using data in a source acceleration structure, updated by the specified geometries.

The `VkDeviceOrHostAddressKHR` union is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef union VkDeviceOrHostAddressKHR {
    VkDeviceAddress    deviceAddress;
    void*              hostAddress;
} VkDeviceOrHostAddressKHR;
```

- `deviceAddress` is a buffer device address as returned by the vkGetBufferDeviceAddressKHR command.
- `hostAddress` is a host memory address.

The `VkDeviceOrHostAddressConstKHR` union is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef union VkDeviceOrHostAddressConstKHR {
    VkDeviceAddress    deviceAddress;
    const void*        hostAddress;
} VkDeviceOrHostAddressConstKHR;
```

- `deviceAddress` is a buffer device address as returned by the vkGetBufferDeviceAddressKHR command.
- `hostAddress` is a const host memory address.

The `VkAccelerationStructureGeometryKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureGeometryKHR {
    VkStructureType                            sType;
    const void*                                pNext;
    VkGeometryTypeKHR                          geometryType;
    VkAccelerationStructureGeometryDataKHR     geometry;
    VkGeometryFlagsKHR                         flags;
} VkAccelerationStructureGeometryKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `geometryType` describes which type of geometry this `VkAccelerationStructureGeometryKHR` refers to.
- `geometry` is a VkAccelerationStructureGeometryDataKHR union describing the geometry data for the relevant geometry type.
- `flags` is a bitmask of VkGeometryFlagBitsKHR values describing additional properties of how the geometry should be built.

Valid Usage (Implicit)

- VUID-VkAccelerationStructureGeometryKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_KHR`
- VUID-VkAccelerationStructureGeometryKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkAccelerationStructureGeometryKHR-geometryType-parameter
  `geometryType` **must** be a valid VkGeometryTypeKHR value
- VUID-VkAccelerationStructureGeometryKHR-triangles-parameter
  If `geometryType` is `VK_GEOMETRY_TYPE_TRIANGLES_KHR`, the `triangles` member of `geometry` **must** be a valid VkAccelerationStructureGeometryTrianglesDataKHR structure
- VUID-VkAccelerationStructureGeometryKHR-aabbs-parameter
  If `geometryType` is `VK_GEOMETRY_TYPE_AABBS_KHR`, the `aabbs` member of `geometry` **must** be a valid VkAccelerationStructureGeometryAabbsDataKHR structure
- VUID-VkAccelerationStructureGeometryKHR-instances-parameter
  If `geometryType` is `VK_GEOMETRY_TYPE_INSTANCES_KHR`, the `instances` member of `geometry` **must** be a valid VkAccelerationStructureGeometryInstancesDataKHR structure
- VUID-VkAccelerationStructureGeometryKHR-flags-parameter
  `flags` **must** be a valid combination of VkGeometryFlagBitsKHR values

The `VkAccelerationStructureGeometryDataKHR` union is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef union VkAccelerationStructureGeometryDataKHR {
    VkAccelerationStructureGeometryTrianglesDataKHR    triangles;
    VkAccelerationStructureGeometryAabbsDataKHR        aabbs;
    VkAccelerationStructureGeometryInstancesDataKHR    instances;
} VkAccelerationStructureGeometryDataKHR;
```

- `triangles` is a VkAccelerationStructureGeometryTrianglesDataKHR structure.
- `aabbs` is a VkAccelerationStructureGeometryAabbsDataKHR structure.
- `instances` is a VkAccelerationStructureGeometryInstancesDataKHR structure.

The `VkAccelerationStructureGeometryTrianglesDataKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureGeometryTrianglesDataKHR {
    VkStructureType                  sType;
    const void*                      pNext;
    VkFormat                         vertexFormat;
    VkDeviceOrHostAddressConstKHR    vertexData;
    VkDeviceSize                     vertexStride;
    uint32_t                         maxVertex;
    VkIndexType                      indexType;
    VkDeviceOrHostAddressConstKHR    indexData;
    VkDeviceOrHostAddressConstKHR    transformData;
} VkAccelerationStructureGeometryTrianglesDataKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `vertexFormat` is the VkFormat of each vertex element.
- `vertexData` is a device or host address to memory containing vertex data for this geometry.
- `maxVertex` is the number of vertices in `vertexData` minus one.
- `vertexStride` is the stride in bytes between each vertex.
- `indexType` is the VkIndexType of each index element.
- `indexData` is a device or host address to memory containing index data for this geometry.
- `transformData` is a device or host address to memory containing an optional reference to a VkTransformMatrixKHR structure describing a transformation from the space in which the vertices in this geometry are described to the space in which the acceleration structure is defined.

NOTE Unlike the stride for vertex buffers in VkVertexInputBindingDescription for graphics pipelines which must not exceed `maxVertexInputBindingStride`, `vertexStride` for acceleration structure geometry is instead restricted to being a 32-bit value.

Valid Usage

- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-vertexStride-03735
  `vertexStride` **must** be a multiple of the size in bytes of the smallest component of `vertexFormat`
- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-vertexStride-03819
  `vertexStride` **must** be less than or equal to 2^{32}-1
- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-vertexFormat-03797
  The format features of `vertexFormat` **must** contain `VK_FORMAT_FEATURE_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR`
- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-indexType-03798

indexType **must** be VK_INDEX_TYPE_UINT16, VK_INDEX_TYPE_UINT32, or VK_INDEX_TYPE_NONE_KHR

Valid Usage (Implicit)

- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_TRIANGLES_DATA_KHR
- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-pNext-pNext
  pNext **must** be NULL or a pointer to a valid instance of VkAccelerationStructureTrianglesOpacityMicromapEXT
- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-vertexFormat-parameter
  vertexFormat **must** be a valid VkFormat value
- VUID-VkAccelerationStructureGeometryTrianglesDataKHR-indexType-parameter
  indexType **must** be a valid VkIndexType value

The VkAccelerationStructureTrianglesOpacityMicromapEXT structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkAccelerationStructureTrianglesOpacityMicromapEXT {
    VkStructureType                    sType;
    void*                              pNext;
    VkIndexType                        indexType;
    VkDeviceOrHostAddressConstKHR      indexBuffer;
    VkDeviceSize                       indexStride;
    uint32_t                           baseTriangle;
    uint32_t                           usageCountsCount;
    const VkMicromapUsageEXT*          pUsageCounts;
    const VkMicromapUsageEXT* const*   ppUsageCounts;
    VkMicromapEXT                      micromap;
} VkAccelerationStructureTrianglesOpacityMicromapEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- indexType is the type of triangle indices used when indexing this micromap
- indexBuffer is the address containing the triangle indices
- indexStride is the byte stride between triangle indices
- baseTriangle is the base value added to the non-negative triangle indices
- usageCountsCount specifies the number of usage counts structures that will be used to determine the size of this micromap.
- pUsageCounts is a pointer to an array of VkMicromapUsageEXT structures.

- `ppUsageCounts` is a pointer to an array of pointers to VkMicromapUsageEXT structures.
- `micromap` is the handle to the micromap object to include in this geometry

If `VkAccelerationStructureTrianglesOpacityMicromapEXT` is included in the `pNext` chain of a VkAccelerationStructureGeometryTrianglesDataKHR structure, that geometry will reference that micromap.

For each triangle in the geometry, the acceleration structure build fetches an index from `indexBuffer` using `indexType` and `indexStride`. If that value is the unsigned cast of one of the values from VkOpacityMicromapSpecialIndexEXT then that triangle behaves as described for that special value in Ray Opacity Micromap. Otherwise that triangle uses the opacity micromap information from `micromap` at that index plus `baseTriangle`.

Only one of `pUsageCounts` or `ppUsageCounts` **can** be a valid pointer, the other **must** be `NULL`. The elements of the non-`NULL` array describe the total count used to build this geometry. For a given `format` and `subdivisionLevel` the number of triangles in this geometry matching those values after indirection and special index handling **must** be equal to the sum of matching `count` provided.

If `micromap` is VK_NULL_HANDLE, then every value read from `indexBuffer` **must** be one of the values in `VkOpacityMicromapSpecialIndexEXT`.

Valid Usage

- VUID-VkAccelerationStructureTrianglesOpacityMicromapEXT-pUsageCounts-07335
  Only one of `pUsageCounts` or `ppUsageCounts` **can** be a valid pointer, the other **must** be `NULL`

Valid Usage (Implicit)

- VUID-VkAccelerationStructureTrianglesOpacityMicromapEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_TRIANGLES_OPACITY_MICROMAP_EXT`
- VUID-VkAccelerationStructureTrianglesOpacityMicromapEXT-indexType-parameter
  `indexType` **must** be a valid VkIndexType value
- VUID-VkAccelerationStructureTrianglesOpacityMicromapEXT-pUsageCounts-parameter
  If `usageCountsCount` is not `0`, and `pUsageCounts` is not `NULL`, `pUsageCounts` **must** be a valid pointer to an array of `usageCountsCount` VkMicromapUsageEXT structures
- VUID-VkAccelerationStructureTrianglesOpacityMicromapEXT-ppUsageCounts-parameter
  If `usageCountsCount` is not `0`, and `ppUsageCounts` is not `NULL`, `ppUsageCounts` **must** be a valid pointer to an array of `usageCountsCount` valid pointers to VkMicromapUsageEXT structures
- VUID-VkAccelerationStructureTrianglesOpacityMicromapEXT-micromap-parameter
  If `micromap` is not VK_NULL_HANDLE, `micromap` **must** be a valid VkMicromapEXT handle

The `VkOpacityMicromapSpecialIndexEXT` enumeration is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef enum VkOpacityMicromapSpecialIndexEXT {
    VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_TRANSPARENT_EXT = -1,
    VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_OPAQUE_EXT = -2,
    VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_UNKNOWN_TRANSPARENT_EXT = -3,
    VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_UNKNOWN_OPAQUE_EXT = -4,
} VkOpacityMicromapSpecialIndexEXT;
```

- VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_TRANSPARENT_EXT specifies that the entire triangle is fully transparent.
- VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_OPAQUE_EXT specifies that the entire triangle is fully opaque.
- VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_UNKNOWN_TRANSPARENT_EXT specifies that the entire triangle is unknown-transparent.
- VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_UNKNOWN_OPAQUE_EXT specifies that the entire triangle is unknown-opaque.

The VkTransformMatrixKHR structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkTransformMatrixKHR {
    float    matrix[3][4];
} VkTransformMatrixKHR;
```

- matrix is a 3x4 row-major affine transformation matrix.

Valid Usage

- VUID-VkTransformMatrixKHR-matrix-03799
  The first three columns of matrix **must** define an invertible 3x3 matrix

The VkAccelerationStructureGeometryAabbsDataKHR structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureGeometryAabbsDataKHR {
    VkStructureType                  sType;
    const void*                      pNext;
    VkDeviceOrHostAddressConstKHR    data;
    VkDeviceSize                     stride;
} VkAccelerationStructureGeometryAabbsDataKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- `data` is a device or host address to memory containing VkAabbPositionsKHR structures containing position data for each axis-aligned bounding box in the geometry.
- `stride` is the stride in bytes between each entry in `data`. The stride **must** be a multiple of `8`.

Valid Usage

- VUID-VkAccelerationStructureGeometryAabbsDataKHR-stride-03545
  `stride` **must** be a multiple of `8`
- VUID-VkAccelerationStructureGeometryAabbsDataKHR-stride-03820
  `stride` **must** be less than or equal to 2^{32}-1

Valid Usage (Implicit)

- VUID-VkAccelerationStructureGeometryAabbsDataKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_AABBS_DATA_KHR`
- VUID-VkAccelerationStructureGeometryAabbsDataKHR-pNext-pNext
  `pNext` **must** be `NULL`

The `VkAabbPositionsKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAabbPositionsKHR {
    float    minX;
    float    minY;
    float    minZ;
    float    maxX;
    float    maxY;
    float    maxZ;
} VkAabbPositionsKHR;
```

- `minX` is the x position of one opposing corner of a bounding box.
- `minY` is the y position of one opposing corner of a bounding box.
- `minZ` is the z position of one opposing corner of a bounding box.
- `maxX` is the x position of the other opposing corner of a bounding box.
- `maxY` is the y position of the other opposing corner of a bounding box.
- `maxZ` is the z position of the other opposing corner of a bounding box.

Valid Usage

- VUID-VkAabbPositionsKHR-minX-03546
  `minX` **must** be less than or equal to `maxX`
- VUID-VkAabbPositionsKHR-minY-03547

`minY` **must** be less than or equal to `maxY`

- VUID-VkAabbPositionsKHR-minZ-03548
  `minZ` **must** be less than or equal to `maxZ`

The `VkAccelerationStructureGeometryInstancesDataKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureGeometryInstancesDataKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkBool32                        arrayOfPointers;
    VkDeviceOrHostAddressConstKHR   data;
} VkAccelerationStructureGeometryInstancesDataKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `arrayOfPointers` specifies whether `data` is used as an array of addresses or just an array.
- `data` is either the address of an array of device or host addresses referencing individual VkAccelerationStructureInstanceKHR structures if `arrayOfPointers` is `VK_TRUE`, or the address of an array of VkAccelerationStructureInstanceKHR structures. Addresses and VkAccelerationStructureInstanceKHR structures are tightly packed.

Valid Usage (Implicit)

- VUID-VkAccelerationStructureGeometryInstancesDataKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_INSTANCES_DATA_KHR`
- VUID-VkAccelerationStructureGeometryInstancesDataKHR-pNext-pNext
  `pNext` **must** be `NULL`

Acceleration structure instances **can** be built into top-level acceleration structures. Each acceleration structure instance is a separate entry in the top-level acceleration structure which includes all the geometry of a bottom-level acceleration structure at a transformed location. Multiple instances **can** point to the same bottom level acceleration structure.

An acceleration structure instance is defined by the structure:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureInstanceKHR {
    VkTransformMatrixKHR          transform;
    uint32_t                      instanceCustomIndex:24;
    uint32_t                      mask:8;
    uint32_t                      instanceShaderBindingTableRecordOffset:24;
    VkGeometryInstanceFlagsKHR    flags:8;
    uint64_t                      accelerationStructureReference;
} VkAccelerationStructureInstanceKHR;
```

- `transform` is a VkTransformMatrixKHR structure describing a transformation to be applied to the acceleration structure.
- `instanceCustomIndex` is a 24-bit application-specified index value accessible to ray shaders in the `InstanceCustomIndexKHR` built-in.
- `mask` is an 8-bit visibility mask for the geometry. The instance **may** only be hit if `Cull Mask & instance.mask != 0`
- `instanceShaderBindingTableRecordOffset` is a 24-bit offset used in calculating the hit shader binding table index.
- `flags` is an 8-bit mask of VkGeometryInstanceFlagBitsKHR values to apply to this instance.
- `accelerationStructureReference` is either :
  - a device address containing the value obtained from vkGetAccelerationStructureDeviceAddressKHR (used by device operations which reference acceleration structures)
  - a VkAccelerationStructureKHR object (used by host operations which reference acceleration structures).

The C language specification does not define the ordering of bit-fields, but in practice, this struct produces the correct layout with existing compilers. The intended bit pattern is for the following:

- `instanceCustomIndex` and `mask` occupy the same memory as if a single `uint32_t` was specified in their place
  - `instanceCustomIndex` occupies the 24 least significant bits of that memory
  - `mask` occupies the 8 most significant bits of that memory
- `instanceShaderBindingTableRecordOffset` and `flags` occupy the same memory as if a single `uint32_t` was specified in their place
  - `instanceShaderBindingTableRecordOffset` occupies the 24 least significant bits of that memory
  - `flags` occupies the 8 most significant bits of that memory

If a compiler produces code that diverges from that pattern, applications **must** employ another method to set values according to the correct bit pattern.

Valid Usage (Implicit)

- VUID-VkAccelerationStructureInstanceKHR-flags-parameter
  `flags` **must** be a valid combination of VkGeometryInstanceFlagBitsKHR values

Possible values of `flags` in the instance modifying the behavior of that instance are:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkGeometryInstanceFlagBitsKHR {
    VK_GEOMETRY_INSTANCE_TRIANGLE_FACING_CULL_DISABLE_BIT_KHR = 0x00000001,
    VK_GEOMETRY_INSTANCE_TRIANGLE_FLIP_FACING_BIT_KHR = 0x00000002,
    VK_GEOMETRY_INSTANCE_FORCE_OPAQUE_BIT_KHR = 0x00000004,
    VK_GEOMETRY_INSTANCE_FORCE_NO_OPAQUE_BIT_KHR = 0x00000008,
  // Provided by VK_EXT_opacity_micromap
    VK_GEOMETRY_INSTANCE_FORCE_OPACITY_MICROMAP_2_STATE_EXT = 0x00000010,
  // Provided by VK_EXT_opacity_micromap
    VK_GEOMETRY_INSTANCE_DISABLE_OPACITY_MICROMAPS_EXT = 0x00000020,
    VK_GEOMETRY_INSTANCE_TRIANGLE_FRONT_COUNTERCLOCKWISE_BIT_KHR =
VK_GEOMETRY_INSTANCE_TRIANGLE_FLIP_FACING_BIT_KHR,
} VkGeometryInstanceFlagBitsKHR;
```

- `VK_GEOMETRY_INSTANCE_TRIANGLE_FACING_CULL_DISABLE_BIT_KHR` disables face culling for this instance.
- `VK_GEOMETRY_INSTANCE_TRIANGLE_FLIP_FACING_BIT_KHR` specifies that the facing determination for geometry in this instance is inverted. Because the facing is determined in object space, an instance transform does not change the winding, but a geometry transform does.
- `VK_GEOMETRY_INSTANCE_FORCE_OPAQUE_BIT_KHR` causes this instance to act as though `VK_GEOMETRY_OPAQUE_BIT_KHR` were specified on all geometries referenced by this instance. This behavior **can** be overridden by the SPIR-V `NoOpaqueKHR` ray flag.
- `VK_GEOMETRY_INSTANCE_FORCE_NO_OPAQUE_BIT_KHR` causes this instance to act as though `VK_GEOMETRY_OPAQUE_BIT_KHR` were not specified on all geometries referenced by this instance. This behavior **can** be overridden by the SPIR-V `OpaqueKHR` ray flag.

`VK_GEOMETRY_INSTANCE_FORCE_NO_OPAQUE_BIT_KHR` and `VK_GEOMETRY_INSTANCE_FORCE_OPAQUE_BIT_KHR` **must** not be used in the same flag.

```
// Provided by VK_KHR_acceleration_structure
typedef VkFlags VkGeometryInstanceFlagsKHR;
```

`VkGeometryInstanceFlagsKHR` is a bitmask type for setting a mask of zero or more VkGeometryInstanceFlagBitsKHR.

`VkAccelerationStructureBuildRangeInfoKHR` is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureBuildRangeInfoKHR {
    uint32_t    primitiveCount;
    uint32_t    primitiveOffset;
    uint32_t    firstVertex;
    uint32_t    transformOffset;
} VkAccelerationStructureBuildRangeInfoKHR;
```

- `primitiveCount` defines the number of primitives for a corresponding acceleration structure geometry.
- `primitiveOffset` defines an offset in bytes into the memory where primitive data is defined.
- `firstVertex` is the index of the first vertex to build from for triangle geometry.
- `transformOffset` defines an offset in bytes into the memory where a transform matrix is defined.

The primitive count and primitive offset are interpreted differently depending on the VkGeometryTypeKHR used:

- For geometries of type `VK_GEOMETRY_TYPE_TRIANGLES_KHR`, `primitiveCount` is the number of triangles to be built, where each triangle is treated as 3 vertices.
  - If the geometry uses indices, `primitiveCount` × 3 indices are consumed from VkAccelerationStructureGeometryTrianglesDataKHR::`indexData`, starting at an offset of `primitiveOffset`. The value of `firstVertex` is added to the index values before fetching vertices.
  - If the geometry does not use indices, `primitiveCount` × 3 vertices are consumed from VkAccelerationStructureGeometryTrianglesDataKHR::`vertexData`, starting at an offset of `primitiveOffset` + VkAccelerationStructureGeometryTrianglesDataKHR::`vertexStride` × `firstVertex`.
  - If VkAccelerationStructureGeometryTrianglesDataKHR::`transformData` is not `NULL`, a single VkTransformMatrixKHR structure is consumed from VkAccelerationStructureGeometryTrianglesDataKHR::`transformData`, at an offset of `transformOffset`. This matrix describes a transformation from the space in which the vertices for all triangles in this geometry are described to the space in which the acceleration structure is defined.
- For geometries of type `VK_GEOMETRY_TYPE_AABBS_KHR`, `primitiveCount` is the number of axis-aligned bounding boxes. `primitiveCount` VkAabbPositionsKHR structures are consumed from VkAccelerationStructureGeometryAabbsDataKHR::`data`, starting at an offset of `primitiveOffset`.
- For geometries of type `VK_GEOMETRY_TYPE_INSTANCES_KHR`, `primitiveCount` is the number of acceleration structures. `primitiveCount` VkAccelerationStructureInstanceKHR structures are consumed from VkAccelerationStructureGeometryInstancesDataKHR::`data`, starting at an offset of `primitiveOffset`.

Valid Usage

- VUID-VkAccelerationStructureBuildRangeInfoKHR-primitiveOffset-03656

For geometries of type VK_GEOMETRY_TYPE_TRIANGLES_KHR, if the geometry uses indices, the offset primitiveOffset from VkAccelerationStructureGeometryTrianglesDataKHR::indexData **must** be a multiple of the element size of VkAccelerationStructureGeometryTrianglesDataKHR::indexType

- VUID-VkAccelerationStructureBuildRangeInfoKHR-primitiveOffset-03657
  For geometries of type VK_GEOMETRY_TYPE_TRIANGLES_KHR, if the geometry does not use indices, the offset primitiveOffset from VkAccelerationStructureGeometryTrianglesDataKHR::vertexData **must** be a multiple of the component size of VkAccelerationStructureGeometryTrianglesDataKHR::vertexFormat
- VUID-VkAccelerationStructureBuildRangeInfoKHR-transformOffset-03658
  For geometries of type VK_GEOMETRY_TYPE_TRIANGLES_KHR, the offset transformOffset from VkAccelerationStructureGeometryTrianglesDataKHR::transformData **must** be a multiple of 16
- VUID-VkAccelerationStructureBuildRangeInfoKHR-primitiveOffset-03659
  For geometries of type VK_GEOMETRY_TYPE_AABBS_KHR, the offset primitiveOffset from VkAccelerationStructureGeometryAabbsDataKHR::data **must** be a multiple of 8
- VUID-VkAccelerationStructureBuildRangeInfoKHR-primitiveOffset-03660
  For geometries of type VK_GEOMETRY_TYPE_INSTANCES_KHR, the offset primitiveOffset from VkAccelerationStructureGeometryInstancesDataKHR::data **must** be a multiple of 16

34.1.8. Copying Acceleration Structures

An additional command exists for copying acceleration structures without updating their contents. The acceleration structure object **can** be compacted in order to improve performance. Before copying, an application **must** query the size of the resulting acceleration structure.

To query acceleration structure size parameters call:

```
// Provided by VK_KHR_acceleration_structure
void vkCmdWriteAccelerationStructuresPropertiesKHR(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    accelerationStructureCount,
    const VkAccelerationStructureKHR*           pAccelerationStructures,
    VkQueryType                                 queryType,
    VkQueryPool                                 queryPool,
    uint32_t                                    firstQuery);
```

- commandBuffer is the command buffer into which the command will be recorded.
- accelerationStructureCount is the count of acceleration structures for which to query the property.
- pAccelerationStructures is a pointer to an array of existing previously built acceleration structures.
- queryType is a VkQueryType value specifying the type of queries managed by the pool.
- queryPool is the query pool that will manage the results of the query.

- `firstQuery` is the first query index within the query pool that will contain the `accelerationStructureCount` number of results.

Accesses to any of the acceleration structures listed in `pAccelerationStructures` **must** be synchronized with the `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR` pipeline stage or the `VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` pipeline stage, and an access type of `VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR`.

- If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR`, then the value written out is the number of bytes required by a compacted acceleration structure.
- If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR`, then the value written out is the number of bytes required by a serialized acceleration structure.

Valid Usage

- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-accelerationStructure-08924
  The `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure` feature **must** be enabled
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-queryPool-02493
  `queryPool` **must** have been created with a `queryType` matching `queryType`
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-queryPool-02494
  The queries identified by `queryPool` and `firstQuery` **must** be *unavailable*
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-buffer-03736
  The `buffer` used to create each acceleration structure in `pAccelerationStructures` **must** be bound to device memory
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-query-04880
  The sum of `firstQuery` plus `accelerationStructureCount` **must** be less than or equal to the number of queries in `queryPool`
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-pAccelerationStructures-04964
  All acceleration structures in `pAccelerationStructures` **must** have been built prior to the execution of this command
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-accelerationStructures-03431
  All acceleration structures in `pAccelerationStructures` **must** have been built with `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_COMPACTION_BIT_KHR` if `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR`
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-queryType-06742
  `queryType` **must** be `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR`, `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR`, `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR`, or `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR`

Valid Usage (Implicit)

- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-pAccelerationStructures-parameter
  `pAccelerationStructures` **must** be a valid pointer to an array of `accelerationStructureCount` valid VkAccelerationStructureKHR handles
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-queryType-parameter
  `queryType` **must** be a valid VkQueryType value
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-queryPool-parameter
  `queryPool` **must** be a valid VkQueryPool handle
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-accelerationStructureCount-arraylength
  `accelerationStructureCount` **must** be greater than `0`
- VUID-vkCmdWriteAccelerationStructuresPropertiesKHR-commonparent
  Each of `commandBuffer`, `queryPool`, and the elements of `pAccelerationStructures` **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

To copy an acceleration structure call:

```
// Provided by VK_KHR_acceleration_structure
void vkCmdCopyAccelerationStructureKHR(
    VkCommandBuffer                             commandBuffer,
    const VkCopyAccelerationStructureInfoKHR*   pInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pInfo` is a pointer to a VkCopyAccelerationStructureInfoKHR structure defining the copy operation.

This command copies the `pInfo->src` acceleration structure to the `pInfo->dst` acceleration structure in the manner specified by `pInfo->mode`.

Accesses to `pInfo->src` and `pInfo->dst` **must** be synchronized with the `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR` pipeline stage or the `VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` pipeline stage, and an access type of `VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR` or `VK_ACCESS_ACCELERATION_STRUCTURE_WRITE_BIT_KHR` as appropriate.

Valid Usage

- VUID-vkCmdCopyAccelerationStructureKHR-accelerationStructure-08925
  The `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure` feature **must** be enabled
- VUID-vkCmdCopyAccelerationStructureKHR-buffer-03737
  The `buffer` used to create `pInfo->src` **must** be bound to device memory
- VUID-vkCmdCopyAccelerationStructureKHR-buffer-03738
  The `buffer` used to create `pInfo->dst` **must** be bound to device memory

Valid Usage (Implicit)

- VUID-vkCmdCopyAccelerationStructureKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyAccelerationStructureKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyAccelerationStructureInfoKHR structure
- VUID-vkCmdCopyAccelerationStructureKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyAccelerationStructureKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdCopyAccelerationStructureKHR-renderpass
  This command **must** only be called outside of a render pass instance

- VUID-vkCmdCopyAccelerationStructureKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

The `VkCopyAccelerationStructureInfoKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkCopyAccelerationStructureInfoKHR {
    VkStructureType                         sType;
    const void*                             pNext;
    VkAccelerationStructureKHR              src;
    VkAccelerationStructureKHR              dst;
    VkCopyAccelerationStructureModeKHR      mode;
} VkCopyAccelerationStructureInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `src` is the source acceleration structure for the copy.
- `dst` is the target acceleration structure for the copy.
- `mode` is a VkCopyAccelerationStructureModeKHR value specifying additional operations to perform during the copy.

Valid Usage

- VUID-VkCopyAccelerationStructureInfoKHR-mode-03410
  `mode` **must** be `VK_COPY_ACCELERATION_STRUCTURE_MODE_COMPACT_KHR` or `VK_COPY_ACCELERATION_STRUCTURE_MODE_CLONE_KHR`
- VUID-VkCopyAccelerationStructureInfoKHR-src-04963
  The source acceleration structure `src` **must** have been constructed prior to the execution

of this command

- VUID-VkCopyAccelerationStructureInfoKHR-src-03411
  If `mode` is `VK_COPY_ACCELERATION_STRUCTURE_MODE_COMPACT_KHR`, `src` **must** have been constructed with `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_COMPACTION_BIT_KHR` in the build
- VUID-VkCopyAccelerationStructureInfoKHR-buffer-03718
  The `buffer` used to create `src` **must** be bound to device memory
- VUID-VkCopyAccelerationStructureInfoKHR-buffer-03719
  The `buffer` used to create `dst` **must** be bound to device memory
- VUID-VkCopyAccelerationStructureInfoKHR-dst-07791
  The range of memory backing `dst` that is accessed by this command **must** not overlap the memory backing `src` that is accessed by this command

Valid Usage (Implicit)

- VUID-VkCopyAccelerationStructureInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_ACCELERATION_STRUCTURE_INFO_KHR`
- VUID-VkCopyAccelerationStructureInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyAccelerationStructureInfoKHR-src-parameter
  `src` **must** be a valid VkAccelerationStructureKHR handle
- VUID-VkCopyAccelerationStructureInfoKHR-dst-parameter
  `dst` **must** be a valid VkAccelerationStructureKHR handle
- VUID-VkCopyAccelerationStructureInfoKHR-mode-parameter
  `mode` **must** be a valid VkCopyAccelerationStructureModeKHR value
- VUID-VkCopyAccelerationStructureInfoKHR-commonparent
  Both of `dst`, and `src` **must** have been created, allocated, or retrieved from the same VkDevice

Possible values of `mode` specifying additional operations to perform during the copy, are:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkCopyAccelerationStructureModeKHR {
    VK_COPY_ACCELERATION_STRUCTURE_MODE_CLONE_KHR = 0,
    VK_COPY_ACCELERATION_STRUCTURE_MODE_COMPACT_KHR = 1,
  // Provided by VK_KHR_acceleration_structure
    VK_COPY_ACCELERATION_STRUCTURE_MODE_SERIALIZE_KHR = 2,
  // Provided by VK_KHR_acceleration_structure
    VK_COPY_ACCELERATION_STRUCTURE_MODE_DESERIALIZE_KHR = 3,
} VkCopyAccelerationStructureModeKHR;
```

- `VK_COPY_ACCELERATION_STRUCTURE_MODE_CLONE_KHR` creates a direct copy of the acceleration structure specified in `src` into the one specified by `dst`. The `dst` acceleration structure **must** have

been created with the same parameters as `src`. If `src` contains references to other acceleration structures, `dst` will reference the same acceleration structures.

- `VK_COPY_ACCELERATION_STRUCTURE_MODE_COMPACT_KHR` creates a more compact version of an acceleration structure `src` into `dst`. The acceleration structure `dst` **must** have been created with a size at least as large as that returned by vkCmdWriteAccelerationStructuresPropertiesKHR, or vkWriteAccelerationStructuresPropertiesKHR after the build of the acceleration structure specified by `src`. If `src` contains references to other acceleration structures, `dst` will reference the same acceleration structures.
- `VK_COPY_ACCELERATION_STRUCTURE_MODE_SERIALIZE_KHR` serializes the acceleration structure to a semi-opaque format which can be reloaded on a compatible implementation.
- `VK_COPY_ACCELERATION_STRUCTURE_MODE_DESERIALIZE_KHR` deserializes the semi-opaque serialization format in the buffer to the acceleration structure.

To copy an acceleration structure to device memory call:

```
// Provided by VK_KHR_acceleration_structure
void vkCmdCopyAccelerationStructureToMemoryKHR(
    VkCommandBuffer                             commandBuffer,
    const VkCopyAccelerationStructureToMemoryInfoKHR* pInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pInfo` is an a pointer to a VkCopyAccelerationStructureToMemoryInfoKHR structure defining the copy operation.

Accesses to `pInfo->src` **must** be synchronized with the `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR` pipeline stage or the `VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` pipeline stage, and an access type of `VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR`. Accesses to the buffer indicated by `pInfo->dst.deviceAddress` **must** be synchronized with the `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR` pipeline stage or the `VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` pipeline stage, and an and an access type of `VK_ACCESS_TRANSFER_WRITE_BIT`.

This command produces the same results as vkCopyAccelerationStructureToMemoryKHR, but writes its result to a device address, and is executed on the device rather than the host. The output **may** not necessarily be bit-for-bit identical, but it can be equally used by either vkCmdCopyMemoryToAccelerationStructureKHR or vkCopyMemoryToAccelerationStructureKHR.

The defined header structure for the serialized data consists of:

- `VK_UUID_SIZE` bytes of data matching `VkPhysicalDeviceIDProperties::driverUUID`
- `VK_UUID_SIZE` bytes of data identifying the compatibility for comparison using vkGetDeviceAccelerationStructureCompatibilityKHR
- A 64-bit integer of the total size matching the value queried using `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR`

- A 64-bit integer of the deserialized size to be passed in to VkAccelerationStructureCreateInfoKHR::size
- A 64-bit integer of the count of the number of acceleration structure handles following. This value matches the value queried using VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR. This will be zero for a bottom-level acceleration structure. For top-level acceleration structures this number is implementation-dependent; the number of and ordering of the handles may not match the instance descriptions which were used to build the acceleration structure.

The corresponding handles matching the values returned by vkGetAccelerationStructureDeviceAddressKHR are tightly packed in the buffer following the count. The application is expected to store a mapping between those handles and the original application-generated bottom-level acceleration structures to provide when deserializing. The serialized data is written to the buffer (or read from the buffer) according to the host endianness.

Valid Usage

- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-accelerationStructure-08926
  The VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure feature **must** be enabled
- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-pInfo-03739
  pInfo->dst.deviceAddress **must** be a valid device address for a buffer bound to device memory
- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-pInfo-03740
  pInfo->dst.deviceAddress **must** be aligned to 256 bytes
- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-pInfo-03741
  If the buffer pointed to by pInfo->dst.deviceAddress is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-None-03559
  The buffer used to create pInfo->src **must** be bound to device memory

Valid Usage (Implicit)

- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-pInfo-parameter
  pInfo **must** be a valid pointer to a valid VkCopyAccelerationStructureToMemoryInfoKHR structure
- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support compute operations

- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyAccelerationStructureToMemoryKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkCopyAccelerationStructureToMemoryInfoKHR {
    VkStructureType                      sType;
    const void*                          pNext;
    VkAccelerationStructureKHR           src;
    VkDeviceOrHostAddressKHR             dst;
    VkCopyAccelerationStructureModeKHR   mode;
} VkCopyAccelerationStructureToMemoryInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `src` is the source acceleration structure for the copy
- `dst` is the device or host address to memory which is the target for the copy
- `mode` is a VkCopyAccelerationStructureModeKHR value specifying additional operations to perform during the copy.

Valid Usage

- VUID-VkCopyAccelerationStructureToMemoryInfoKHR-src-04959
  The source acceleration structure `src` **must** have been constructed prior to the execution of this command
- VUID-VkCopyAccelerationStructureToMemoryInfoKHR-dst-03561

The memory pointed to by `dst` **must** be at least as large as the serialization size of `src`, as reported by vkWriteAccelerationStructuresPropertiesKHR or vkCmdWriteAccelerationStructuresPropertiesKHR with a query type of `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR`

- VUID-VkCopyAccelerationStructureToMemoryInfoKHR-mode-03412
  `mode` **must** be `VK_COPY_ACCELERATION_STRUCTURE_MODE_SERIALIZE_KHR`

Valid Usage (Implicit)

- VUID-VkCopyAccelerationStructureToMemoryInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_ACCELERATION_STRUCTURE_TO_MEMORY_INFO_KHR`
- VUID-VkCopyAccelerationStructureToMemoryInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyAccelerationStructureToMemoryInfoKHR-src-parameter
  `src` **must** be a valid VkAccelerationStructureKHR handle
- VUID-VkCopyAccelerationStructureToMemoryInfoKHR-mode-parameter
  `mode` **must** be a valid VkCopyAccelerationStructureModeKHR value

To copy device memory to an acceleration structure call:

```
// Provided by VK_KHR_acceleration_structure
void vkCmdCopyMemoryToAccelerationStructureKHR(
    VkCommandBuffer                             commandBuffer,
    const VkCopyMemoryToAccelerationStructureInfoKHR* pInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pInfo` is a pointer to a VkCopyMemoryToAccelerationStructureInfoKHR structure defining the copy operation.

Accesses to `pInfo->dst` **must** be synchronized with the `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR` pipeline stage or the `VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` pipeline stage, and an access type of `VK_ACCESS_ACCELERATION_STRUCTURE_WRITE_BIT_KHR`. Accesses to the buffer indicated by `pInfo->src.deviceAddress` **must** be synchronized with the `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR` pipeline stage or the `VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` pipeline stage, and an access type of `VK_ACCESS_TRANSFER_READ_BIT`.

This command can accept acceleration structures produced by either vkCmdCopyAccelerationStructureToMemoryKHR or vkCopyAccelerationStructureToMemoryKHR.

The structure provided as input to deserialize is as described in vkCmdCopyAccelerationStructureToMemoryKHR, with any acceleration structure handles filled in with the newly-queried handles to bottom level acceleration structures created before

deserialization. These do not need to be built at deserialize time, but **must** be created.

Valid Usage

- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-accelerationStructure-08927
  The `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure` feature **must** be enabled
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-pInfo-03742
  `pInfo->src.deviceAddress` **must** be a valid device address for a buffer bound to device memory
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-pInfo-03743
  `pInfo->src.deviceAddress` **must** be aligned to `256` bytes
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-pInfo-03744
  If the buffer pointed to by `pInfo->src.deviceAddress` is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-buffer-03745
  The `buffer` used to create `pInfo->dst` **must** be bound to device memory

Valid Usage (Implicit)

- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyMemoryToAccelerationStructureInfoKHR structure
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyMemoryToAccelerationStructureKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

The `VkCopyMemoryToAccelerationStructureInfoKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkCopyMemoryToAccelerationStructureInfoKHR {
    VkStructureType                         sType;
    const void*                             pNext;
    VkDeviceOrHostAddressConstKHR           src;
    VkAccelerationStructureKHR              dst;
    VkCopyAccelerationStructureModeKHR      mode;
} VkCopyMemoryToAccelerationStructureInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `src` is the device or host address to memory containing the source data for the copy.
- `dst` is the target acceleration structure for the copy.
- `mode` is a VkCopyAccelerationStructureModeKHR value specifying additional operations to perform during the copy.

Valid Usage

- VUID-VkCopyMemoryToAccelerationStructureInfoKHR-src-04960
  The source memory pointed to by `src` **must** contain data previously serialized using vkCmdCopyAccelerationStructureToMemoryKHR, potentially modified to relocate acceleration structure references as described in that command
- VUID-VkCopyMemoryToAccelerationStructureInfoKHR-mode-03413
  `mode` **must** be `VK_COPY_ACCELERATION_STRUCTURE_MODE_DESERIALIZE_KHR`
- VUID-VkCopyMemoryToAccelerationStructureInfoKHR-pInfo-03414
  The data in `src` **must** have a format compatible with the destination physical device as returned by vkGetDeviceAccelerationStructureCompatibilityKHR
- VUID-VkCopyMemoryToAccelerationStructureInfoKHR-dst-03746
  `dst` **must** have been created with a `size` greater than or equal to that used to serialize the data in `src`

Valid Usage (Implicit)

- VUID-VkCopyMemoryToAccelerationStructureInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_MEMORY_TO_ACCELERATION_STRUCTURE_INFO_KHR`
- VUID-VkCopyMemoryToAccelerationStructureInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyMemoryToAccelerationStructureInfoKHR-dst-parameter
  `dst` **must** be a valid VkAccelerationStructureKHR handle
- VUID-VkCopyMemoryToAccelerationStructureInfoKHR-mode-parameter
  `mode` **must** be a valid VkCopyAccelerationStructureModeKHR value

To check if a serialized acceleration structure is compatible with the current device call:

```
// Provided by VK_KHR_acceleration_structure
void vkGetDeviceAccelerationStructureCompatibilityKHR(
    VkDevice                                    device,
    const VkAccelerationStructureVersionInfoKHR* pVersionInfo,
    VkAccelerationStructureCompatibilityKHR*    pCompatibility);
```

- `device` is the device to check the version against.
- `pVersionInfo` is a pointer to a VkAccelerationStructureVersionInfoKHR structure specifying version information to check against the device.
- `pCompatibility` is a pointer to a VkAccelerationStructureCompatibilityKHR value in which compatibility information is returned.

Valid Usage

- VUID-vkGetDeviceAccelerationStructureCompatibilityKHR-accelerationStructure-08928
  The `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructure` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkGetDeviceAccelerationStructureCompatibilityKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceAccelerationStructureCompatibilityKHR-pVersionInfo-parameter
  `pVersionInfo` **must** be a valid pointer to a valid VkAccelerationStructureVersionInfoKHR structure
- VUID-vkGetDeviceAccelerationStructureCompatibilityKHR-pCompatibility-parameter
  `pCompatibility` **must** be a valid pointer to a VkAccelerationStructureCompatibilityKHR value

The VkAccelerationStructureVersionInfoKHR structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkAccelerationStructureVersionInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    const uint8_t*     pVersionData;
} VkAccelerationStructureVersionInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pVersionData is a pointer to the version header of an acceleration structure as defined in vkCmdCopyAccelerationStructureToMemoryKHR

NOTE

pVersionData is a *pointer* to an array of 2×VK_UUID_SIZE uint8_t values instead of two VK_UUID_SIZE arrays as the expected use case for this member is to be pointed at the header of a previously serialized acceleration structure (via vkCmdCopyAccelerationStructureToMemoryKHR or vkCopyAccelerationStructureToMemoryKHR) that is loaded in memory. Using arrays would necessitate extra memory copies of the UUIDs.

Valid Usage (Implicit)

- VUID-VkAccelerationStructureVersionInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_VERSION_INFO_KHR
- VUID-VkAccelerationStructureVersionInfoKHR-pNext-pNext
  pNext **must** be NULL
- VUID-VkAccelerationStructureVersionInfoKHR-pVersionData-parameter
  pVersionData **must** be a valid pointer to an array of $2 \times VK_UUID_SIZE$ uint8_t values

Possible values of pCompatibility returned by vkGetDeviceAccelerationStructureCompatibilityKHR are:

```
// Provided by VK_KHR_acceleration_structure
typedef enum VkAccelerationStructureCompatibilityKHR {
    VK_ACCELERATION_STRUCTURE_COMPATIBILITY_COMPATIBLE_KHR = 0,
    VK_ACCELERATION_STRUCTURE_COMPATIBILITY_INCOMPATIBLE_KHR = 1,
} VkAccelerationStructureCompatibilityKHR;
```

- VK_ACCELERATION_STRUCTURE_COMPATIBILITY_COMPATIBLE_KHR if the pVersionData version acceleration structure is compatible with device.
- VK_ACCELERATION_STRUCTURE_COMPATIBILITY_INCOMPATIBLE_KHR if the pVersionData version acceleration structure is not compatible with device.

34.2. Host Acceleration Structure Operations

Implementations are also required to provide host implementations of the acceleration structure operations if the `accelerationStructureHostCommands` feature is enabled:

- vkBuildAccelerationStructuresKHR corresponding to vkCmdBuildAccelerationStructuresKHR
- vkCopyAccelerationStructureKHR corresponding to vkCmdCopyAccelerationStructureKHR
- vkCopyAccelerationStructureToMemoryKHR corresponding to vkCmdCopyAccelerationStructureToMemoryKHR
- vkCopyMemoryToAccelerationStructureKHR corresponding to vkCmdCopyMemoryToAccelerationStructureKHR
- vkWriteAccelerationStructuresPropertiesKHR corresponding to vkCmdWriteAccelerationStructuresPropertiesKHR

These commands are functionally equivalent to their device counterparts, except that they are executed on the host timeline, rather than being enqueued into command buffers.

All acceleration structures used by the host commands **must** be bound to host-visible memory, and all input data for acceleration structure builds **must** be referenced using host addresses instead of device addresses. Applications are not required to map acceleration structure memory when using the host commands.

NOTE

The vkBuildAccelerationStructuresKHR and vkCmdBuildAccelerationStructuresKHR **may** use different algorithms, and thus are not required to produce identical structures. The structures produced by these two commands **may** exhibit different memory footprints or traversal performance, but should strive to be similar where possible.

Apart from these details, the host and device operations are interchangeable. For example, an application **can** use vkBuildAccelerationStructuresKHR to build a structure, compact it on the device using vkCmdCopyAccelerationStructureKHR, and serialize the result using vkCopyAccelerationStructureToMemoryKHR.

NOTE

For efficient execution, acceleration structures manipulated using these commands should always be bound to host cached memory, as the implementation may need to repeatedly read and write this memory during the execution of the command.

To build acceleration structures on the host, call:

```
// Provided by VK_KHR_acceleration_structure
VkResult vkBuildAccelerationStructuresKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    uint32_t                                    infoCount,
    const VkAccelerationStructureBuildGeometryInfoKHR* pInfos,
    const VkAccelerationStructureBuildRangeInfoKHR* const* ppBuildRangeInfos);
```

- `device` is the `VkDevice` for which the acceleration structures are being built.
- `deferredOperation` is an optional VkDeferredOperationKHR to request deferral for this command.
- `infoCount` is the number of acceleration structures to build. It specifies the number of the `pInfos` structures and `ppBuildRangeInfos` pointers that **must** be provided.
- `pInfos` is a pointer to an array of `infoCount` VkAccelerationStructureBuildGeometryInfoKHR structures defining the geometry used to build each acceleration structure.
- `ppBuildRangeInfos` is a pointer to an array of `infoCount` pointers to arrays of VkAccelerationStructureBuildRangeInfoKHR structures. Each `ppBuildRangeInfos`[i] is a pointer to an array of `pInfos`[i].`geometryCount` VkAccelerationStructureBuildRangeInfoKHR structures defining dynamic offsets to the addresses where geometry data is stored, as defined by `pInfos`[i].

This command fulfills the same task as vkCmdBuildAccelerationStructuresKHR but is executed by the host.

The `vkBuildAccelerationStructuresKHR` command provides the ability to initiate multiple acceleration structures builds, however there is no ordering or synchronization implied between any of the individual acceleration structure builds.

NOTE This means that an application **cannot** build a top-level acceleration structure in the same vkBuildAccelerationStructuresKHR call as the associated bottom-level or instance acceleration structures are being built. There also **cannot** be any memory aliasing between any acceleration structure memories or scratch memories being used by any of the builds.

Valid Usage

- VUID-vkBuildAccelerationStructuresKHR-accelerationStructureHostCommands-03581
  The `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructureHostCommands` feature **must** be enabled
- VUID-vkBuildAccelerationStructuresKHR-mode-04628
  The `mode` member of each element of `pInfos` **must** be a valid VkBuildAccelerationStructureModeKHR value
- VUID-vkBuildAccelerationStructuresKHR-srcAccelerationStructure-04629
  If the `srcAccelerationStructure` member of any element of `pInfos` is not VK_NULL_HANDLE, the `srcAccelerationStructure` member **must** be a valid VkAccelerationStructureKHR handle
- VUID-vkBuildAccelerationStructuresKHR-pInfos-04630
  For each element of `pInfos`, if its `mode` member is `VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR`, its `srcAccelerationStructure` member **must** not be VK_NULL_HANDLE
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03403
  The `srcAccelerationStructure` member of any element of `pInfos` **must** not be the same acceleration structure as the `dstAccelerationStructure` member of any other element of

pInfos

- VUID-vkBuildAccelerationStructuresKHR-dstAccelerationStructure-03698
  The dstAccelerationStructure member of any element of pInfos **must** not be the same acceleration structure as the dstAccelerationStructure member of any other element of pInfos
- VUID-vkBuildAccelerationStructuresKHR-dstAccelerationStructure-03800
  The dstAccelerationStructure member of any element of pInfos **must** be a valid VkAccelerationStructureKHR handle
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03699
  For each element of pInfos, if its type member is VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR, its dstAccelerationStructure member **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::type equal to either VK_ACCELERATION_STRUCTURE_TYPE_TOP_LEVEL_KHR or VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03700
  For each element of pInfos, if its type member is VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR, its dstAccelerationStructure member **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::type equal to either VK_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL_KHR or VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03663
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, inactive primitives in its srcAccelerationStructure member **must** not be made active
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03664
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, active primitives in its srcAccelerationStructure member **must** not be made inactive
- VUID-vkBuildAccelerationStructuresKHR-None-03407
  The dstAccelerationStructure member of any element of pInfos **must** not be referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR in any other element of pInfos
- VUID-vkBuildAccelerationStructuresKHR-dstAccelerationStructure-03701
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the srcAccelerationStructure member of any other element of pInfos with a mode equal to VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, which is accessed by this command
- VUID-vkBuildAccelerationStructuresKHR-dstAccelerationStructure-03702
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the dstAccelerationStructure member of any other element of pInfos, which is accessed by this command
- VUID-vkBuildAccelerationStructuresKHR-dstAccelerationStructure-03703
  The range of memory backing the dstAccelerationStructure member of any element of

pInfos that is accessed by this command **must** not overlap the memory backing the scratchData member of any element of pInfos (including the same element), which is accessed by this command

- VUID-vkBuildAccelerationStructuresKHR-scratchData-03704
  The range of memory backing the scratchData member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the scratchData member of any other element of pInfos, which is accessed by this command
- VUID-vkBuildAccelerationStructuresKHR-scratchData-03705
  The range of memory backing the scratchData member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the srcAccelerationStructure member of any element of pInfos with a mode equal to VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR (including the same element), which is accessed by this command
- VUID-vkBuildAccelerationStructuresKHR-dstAccelerationStructure-03706
  The range of memory backing the dstAccelerationStructure member of any element of pInfos that is accessed by this command **must** not overlap the memory backing any acceleration structure referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR in any other element of pInfos, which is accessed by this command
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03667
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its srcAccelerationStructure member **must** have previously been constructed with VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_UPDATE_BIT_KHR set in VkAccelerationStructureBuildGeometryInfoKHR::flags in the build
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03668
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its srcAccelerationStructure and dstAccelerationStructure members **must** either be the same VkAccelerationStructureKHR, or not have any memory aliasing
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03758
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its geometryCount member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03759
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its flags member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03760
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, its type member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03761
  For each element of pInfos, if its mode member is

VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, its geometryType member **must** have the same value which was specified when srcAccelerationStructure was last built

- VUID-vkBuildAccelerationStructuresKHR-pInfos-03762
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, its flags member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03763
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its geometry.triangles.vertexFormat member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03764
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its geometry.triangles.maxVertex member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03765
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, its geometry.triangles.indexType member **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03766
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, if its geometry.triangles.transformData address was NULL when srcAccelerationStructure was last built, then it **must** be NULL
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03767
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, if its geometry.triangles.transformData address was not NULL when srcAccelerationStructure was last built, then it **must** not be NULL

- VUID-vkBuildAccelerationStructuresKHR-pInfos-03768
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, if geometryType is VK_GEOMETRY_TYPE_TRIANGLES_KHR, and geometry.triangles.indexType is not VK_INDEX_TYPE_NONE_KHR, then the value of each index referenced **must** be the same as the corresponding index value when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-primitiveCount-03769
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, then for each VkAccelerationStructureGeometryKHR structure referred to by its pGeometries or ppGeometries members, the primitiveCount member of its corresponding VkAccelerationStructureBuildRangeInfoKHR structure **must** have the same value which was specified when srcAccelerationStructure was last built
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03801
  For each element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, the corresponding ppBuildRangeInfos[i][j].primitiveCount **must** be less than or equal to VkPhysicalDeviceAccelerationStructurePropertiesKHR ::maxInstanceCount
- VUID-vkBuildAccelerationStructuresKHR-pInfos-10126
  For each pInfos[i], dstAccelerationStructure **must** have been created with a value of VkAccelerationStructureCreateInfoKHR::size greater than or equal to either:
  - the memory size required by the build operation, as returned by vkGetAccelerationStructureBuildSizesKHR with pBuildInfo = pInfos[i] and with each element of the pMaxPrimitiveCounts array greater than or equal to the equivalent ppBuildRangeInfos[i][j].primitiveCount values for j in [0,pInfos[i].geometryCount) or,
  - the result of querying the corresponding VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR, if updating a compacted acceleration structure
- VUID-vkBuildAccelerationStructuresKHR-ppBuildRangeInfos-03676
  Each element of ppBuildRangeInfos[i] **must** be a valid pointer to an array of pInfos[i].geometryCount VkAccelerationStructureBuildRangeInfoKHR structures
- VUID-vkBuildAccelerationStructuresKHR-deferredOperation-03678
  Any previous deferred operation that was associated with deferredOperation **must** be complete
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03722
  For each element of pInfos, the buffer used to create its dstAccelerationStructure member **must** be bound to host-visible device memory
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03723
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR the buffer used to create its srcAccelerationStructure member **must** be bound to host-visible device memory

- VUID-vkBuildAccelerationStructuresKHR-pInfos-03724
  For each element of pInfos, the buffer used to create each acceleration structure referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR **must** be bound to host-visible device memory
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03725
  If pInfos[i].mode is VK_BUILD_ACCELERATION_STRUCTURE_MODE_BUILD_KHR, all addresses between pInfos[i].scratchData.hostAddress and pInfos[i].scratchData.hostAddress + N - 1 **must** be valid host memory, where N is given by the buildScratchSize member of the VkAccelerationStructureBuildSizesInfoKHR structure returned from a call to vkGetAccelerationStructureBuildSizesKHR with an identical VkAccelerationStructureBuildGeometryInfoKHR structure and primitive count
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03726
  If pInfos[i].mode is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR, all addresses between pInfos[i].scratchData.hostAddress and pInfos[i].scratchData.hostAddress + N - 1 **must** be valid host memory, where N is given by the updateScratchSize member of the VkAccelerationStructureBuildSizesInfoKHR structure returned from a call to vkGetAccelerationStructureBuildSizesKHR with an identical VkAccelerationStructureBuildGeometryInfoKHR structure and primitive count
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03771
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, geometry.triangles.vertexData.hostAddress **must** be a valid host address
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03772
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.indexType is not VK_INDEX_TYPE_NONE_KHR, geometry.triangles.indexData.hostAddress **must** be a valid host address
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03773
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR, if geometry.triangles.transformData.hostAddress is not 0, it **must** be a valid host address
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03774
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR, geometry.aabbs.data.hostAddress **must** be a valid host address
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03775
  For each element of pInfos, the buffer used to create its dstAccelerationStructure member **must** be bound to memory that was not allocated with multiple instances
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03776
  For each element of pInfos, if its mode member is VK_BUILD_ACCELERATION_STRUCTURE_MODE_UPDATE_KHR the buffer used to create its srcAccelerationStructure member **must** be bound to memory that was not allocated with multiple instances
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03777

For each element of pInfos, the buffer used to create each acceleration structure referenced by the geometry.instances.data member of any element of pGeometries or ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR **must** be bound to memory that was not allocated with multiple instances

- VUID-vkBuildAccelerationStructuresKHR-pInfos-03778
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, geometry.instances.data.hostAddress **must** be a valid host address
- VUID-vkBuildAccelerationStructuresKHR-pInfos-03779
  For any element of pInfos[i].pGeometries or pInfos[i].ppGeometries with a geometryType of VK_GEOMETRY_TYPE_INSTANCES_KHR, each VkAccelerationStructureInstanceKHR ::accelerationStructureReference value in geometry.instances.data.hostAddress **must** be a valid VkAccelerationStructureKHR object

Valid Usage (Implicit)

- VUID-vkBuildAccelerationStructuresKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkBuildAccelerationStructuresKHR-deferredOperation-parameter
  If deferredOperation is not VK_NULL_HANDLE, deferredOperation **must** be a valid VkDeferredOperationKHR handle
- VUID-vkBuildAccelerationStructuresKHR-pInfos-parameter
  pInfos **must** be a valid pointer to an array of infoCount valid VkAccelerationStructureBuildGeometryInfoKHR structures
- VUID-vkBuildAccelerationStructuresKHR-ppBuildRangeInfos-parameter
  ppBuildRangeInfos **must** be a valid pointer to an array of infoCount VkAccelerationStructureBuildRangeInfoKHR structures
- VUID-vkBuildAccelerationStructuresKHR-infoCount-arraylength
  infoCount **must** be greater than 0
- VUID-vkBuildAccelerationStructuresKHR-deferredOperation-parent
  If deferredOperation is a valid handle, it **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS
- VK_OPERATION_DEFERRED_KHR
- VK_OPERATION_NOT_DEFERRED_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY

- VK_ERROR_OUT_OF_DEVICE_MEMORY

To copy or compact an acceleration structure on the host, call:

```
// Provided by VK_KHR_acceleration_structure
VkResult vkCopyAccelerationStructureKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    const VkCopyAccelerationStructureInfoKHR*   pInfo);
```

- device is the device which owns the acceleration structures.
- deferredOperation is an optional VkDeferredOperationKHR to request deferral for this command.
- pInfo is a pointer to a VkCopyAccelerationStructureInfoKHR structure defining the copy operation.

This command fulfills the same task as vkCmdCopyAccelerationStructureKHR but is executed by the host.

Valid Usage

- VUID-vkCopyAccelerationStructureKHR-accelerationStructureHostCommands-03582
  The VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructureHostCommands feature **must** be enabled
- VUID-vkCopyAccelerationStructureKHR-deferredOperation-03678
  Any previous deferred operation that was associated with deferredOperation **must** be complete
- VUID-vkCopyAccelerationStructureKHR-buffer-03727
  The buffer used to create pInfo->src **must** be bound to host-visible device memory
- VUID-vkCopyAccelerationStructureKHR-buffer-03728
  The buffer used to create pInfo->dst **must** be bound to host-visible device memory
- VUID-vkCopyAccelerationStructureKHR-buffer-03780
  The buffer used to create pInfo->src **must** be bound to memory that was not allocated with multiple instances
- VUID-vkCopyAccelerationStructureKHR-buffer-03781
  The buffer used to create pInfo->dst **must** be bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkCopyAccelerationStructureKHR-device-parameter
  device **must** be a valid VkDevice handle

- VUID-vkCopyAccelerationStructureKHR-deferredOperation-parameter
  If `deferredOperation` is not VK_NULL_HANDLE, `deferredOperation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkCopyAccelerationStructureKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyAccelerationStructureInfoKHR structure
- VUID-vkCopyAccelerationStructureKHR-deferredOperation-parent
  If `deferredOperation` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_OPERATION_DEFERRED_KHR`
- `VK_OPERATION_NOT_DEFERRED_KHR`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

To copy host accessible memory to an acceleration structure, call:

```
// Provided by VK_KHR_acceleration_structure
VkResult vkCopyMemoryToAccelerationStructureKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    const VkCopyMemoryToAccelerationStructureInfoKHR* pInfo);
```

- `device` is the device which owns `pInfo->dst`.
- `deferredOperation` is an optional VkDeferredOperationKHR to request deferral for this command.
- `pInfo` is a pointer to a VkCopyMemoryToAccelerationStructureInfoKHR structure defining the copy operation.

This command fulfills the same task as vkCmdCopyMemoryToAccelerationStructureKHR but is executed by the host.

This command can accept acceleration structures produced by either vkCmdCopyAccelerationStructureToMemoryKHR or vkCopyAccelerationStructureToMemoryKHR.

Valid Usage

- VUID-vkCopyMemoryToAccelerationStructureKHR-accelerationStructureHostCommands-

03583
The VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructureHostCommands feature **must** be enabled

- VUID-vkCopyMemoryToAccelerationStructureKHR-deferredOperation-03678
  Any previous deferred operation that was associated with `deferredOperation` **must** be complete
- VUID-vkCopyMemoryToAccelerationStructureKHR-pInfo-03729
  `pInfo->src.hostAddress` **must** be a valid host pointer
- VUID-vkCopyMemoryToAccelerationStructureKHR-pInfo-03750
  `pInfo->src.hostAddress` **must** be aligned to 16 bytes
- VUID-vkCopyMemoryToAccelerationStructureKHR-buffer-03730
  The `buffer` used to create `pInfo->dst` **must** be bound to host-visible device memory
- VUID-vkCopyMemoryToAccelerationStructureKHR-buffer-03782
  The `buffer` used to create `pInfo->dst` **must** be bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkCopyMemoryToAccelerationStructureKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCopyMemoryToAccelerationStructureKHR-deferredOperation-parameter
  If `deferredOperation` is not VK_NULL_HANDLE, `deferredOperation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkCopyMemoryToAccelerationStructureKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyMemoryToAccelerationStructureInfoKHR structure
- VUID-vkCopyMemoryToAccelerationStructureKHR-deferredOperation-parent
  If `deferredOperation` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- VK_SUCCESS
- VK_OPERATION_DEFERRED_KHR
- VK_OPERATION_NOT_DEFERRED_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

To copy an acceleration structure to host accessible memory, call:

```
// Provided by VK_KHR_acceleration_structure
VkResult vkCopyAccelerationStructureToMemoryKHR(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    const VkCopyAccelerationStructureToMemoryInfoKHR* pInfo);
```

- `device` is the device which owns `pInfo->src`.
- `deferredOperation` is an optional VkDeferredOperationKHR to request deferral for this command.
- `pInfo` is a pointer to a VkCopyAccelerationStructureToMemoryInfoKHR structure defining the copy operation.

This command fulfills the same task as vkCmdCopyAccelerationStructureToMemoryKHR but is executed by the host.

This command produces the same results as vkCmdCopyAccelerationStructureToMemoryKHR, but writes its result directly to a host pointer, and is executed on the host rather than the device. The output **may** not necessarily be bit-for-bit identical, but it can be equally used by either vkCmdCopyMemoryToAccelerationStructureKHR or vkCopyMemoryToAccelerationStructureKHR.

Valid Usage

- VUID-vkCopyAccelerationStructureToMemoryKHR-accelerationStructureHostCommands-03584
  The `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructureHostCommands` feature **must** be enabled
- VUID-vkCopyAccelerationStructureToMemoryKHR-deferredOperation-03678
  Any previous deferred operation that was associated with `deferredOperation` **must** be complete
- VUID-vkCopyAccelerationStructureToMemoryKHR-buffer-03731
  The `buffer` used to create `pInfo->src` **must** be bound to host-visible device memory
- VUID-vkCopyAccelerationStructureToMemoryKHR-pInfo-03732
  `pInfo->dst.hostAddress` **must** be a valid host pointer
- VUID-vkCopyAccelerationStructureToMemoryKHR-pInfo-03751
  `pInfo->dst.hostAddress` **must** be aligned to 16 bytes
- VUID-vkCopyAccelerationStructureToMemoryKHR-buffer-03783
  The `buffer` used to create `pInfo->src` **must** be bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkCopyAccelerationStructureToMemoryKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCopyAccelerationStructureToMemoryKHR-deferredOperation-parameter
  If `deferredOperation` is not VK_NULL_HANDLE, `deferredOperation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkCopyAccelerationStructureToMemoryKHR-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyAccelerationStructureToMemoryInfoKHR structure
- VUID-vkCopyAccelerationStructureToMemoryKHR-deferredOperation-parent
  If `deferredOperation` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- VK_SUCCESS
- VK_OPERATION_DEFERRED_KHR
- VK_OPERATION_NOT_DEFERRED_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

To query acceleration structure size parameters on the host, call:

```
// Provided by VK_KHR_acceleration_structure
VkResult vkWriteAccelerationStructuresPropertiesKHR(
    VkDevice                                    device,
    uint32_t                                    accelerationStructureCount,
    const VkAccelerationStructureKHR*           pAccelerationStructures,
    VkQueryType                                 queryType,
    size_t                                      dataSize,
    void*                                       pData,
    size_t                                      stride);
```

- `device` is the device which owns the acceleration structures in `pAccelerationStructures`.
- `accelerationStructureCount` is the count of acceleration structures for which to query the property.
- `pAccelerationStructures` is a pointer to an array of existing previously built acceleration structures.

- `queryType` is a VkQueryType value specifying the property to be queried.
- `dataSize` is the size in bytes of the buffer pointed to by `pData`.
- `pData` is a pointer to an application-allocated buffer where the results will be written.
- `stride` is the stride in bytes between results for individual queries within `pData`.

This command fulfills the same task as vkCmdWriteAccelerationStructuresPropertiesKHR but is executed by the host.

Valid Usage

- VUID-vkWriteAccelerationStructuresPropertiesKHR-accelerationStructureHostCommands-03585
  The `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructureHostCommands` feature **must** be enabled
- VUID-vkWriteAccelerationStructuresPropertiesKHR-pAccelerationStructures-04964
  All acceleration structures in `pAccelerationStructures` **must** have been built prior to the execution of this command
- VUID-vkWriteAccelerationStructuresPropertiesKHR-accelerationStructures-03431
  All acceleration structures in `pAccelerationStructures` **must** have been built with `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_COMPACTION_BIT_KHR` if `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR`
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-06742
  `queryType` **must** be `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR`, `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR`, `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR`, or `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR`
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-03448
  If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR`, then `stride` **must** be a multiple of the size of VkDeviceSize
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-03449
  If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR`, then `pData` **must** point to a VkDeviceSize
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-03450
  If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR`, then `stride` **must** be a multiple of the size of VkDeviceSize
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-03451
  If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR`, then `pData` **must** point to a VkDeviceSize
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-06731
  If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR`, then `stride` **must** be a multiple of the size of VkDeviceSize
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-06732
  If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR`, then `pData` **must** point to a

VkDeviceSize

- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-06733
  If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR`, then `stride` **must** be a multiple of the size of VkDeviceSize
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-06734
  If `queryType` is `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR`, then `pData` **must** point to a VkDeviceSize
- VUID-vkWriteAccelerationStructuresPropertiesKHR-dataSize-03452
  `dataSize` **must** be greater than or equal to `accelerationStructureCount`*`stride`
- VUID-vkWriteAccelerationStructuresPropertiesKHR-buffer-03733
  The `buffer` used to create each acceleration structure in `pAccelerationStructures` **must** be bound to host-visible device memory
- VUID-vkWriteAccelerationStructuresPropertiesKHR-buffer-03784
  The `buffer` used to create each acceleration structure in `pAccelerationStructures` **must** be bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkWriteAccelerationStructuresPropertiesKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkWriteAccelerationStructuresPropertiesKHR-pAccelerationStructures-parameter
  `pAccelerationStructures` **must** be a valid pointer to an array of `accelerationStructureCount` valid VkAccelerationStructureKHR handles
- VUID-vkWriteAccelerationStructuresPropertiesKHR-queryType-parameter
  `queryType` **must** be a valid VkQueryType value
- VUID-vkWriteAccelerationStructuresPropertiesKHR-pData-parameter
  `pData` **must** be a valid pointer to an array of `dataSize` bytes
- VUID-vkWriteAccelerationStructuresPropertiesKHR-accelerationStructureCount-arraylength
  `accelerationStructureCount` **must** be greater than `0`
- VUID-vkWriteAccelerationStructuresPropertiesKHR-dataSize-arraylength
  `dataSize` **must** be greater than `0`
- VUID-vkWriteAccelerationStructuresPropertiesKHR-pAccelerationStructures-parent
  Each element of `pAccelerationStructures` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Chapter 35. Micromap

35.1. Micromaps

Acceleration structures store and organize geometry for ray tracing, but in some cases it is beneficial to include some information within the geometry, particularly for triangles. A *micromap* organizes this data around a map of values corresponding to subdivided microtriangles which can be added to a triangle geometry when building a *bottom level acceleration structure*.

An *opacity micromap* is a type of micromap which stores information to control intersection opacity as described in Ray Opacity Micromap.

A micromap is considered to be constructed if a micromap build command or copy command has been executed with the given acceleration structure as the destination.

35.1.1. Building Micromaps

To build micromaps call:

```
// Provided by VK_EXT_opacity_micromap
void vkCmdBuildMicromapsEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    infoCount,
    const VkMicromapBuildInfoEXT*               pInfos);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `infoCount` is the number of micromaps to build. It specifies the number of the `pInfos` structures that **must** be provided.
- `pInfos` is a pointer to an array of `infoCount` VkMicromapBuildInfoEXT structures defining the data used to build each micromap.

The `vkCmdBuildMicromapsEXT` command provides the ability to initiate multiple micromaps builds, however there is no ordering or synchronization implied between any of the individual micromap builds.

> **NOTE** This means that there **cannot** be any memory aliasing between any micromap memories or scratch memories being used by any of the builds.

Accesses to the micromap scratch buffers as identified by the VkMicromapBuildInfoEXT::`scratchData` buffer device addresses **must** be synchronized with the `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` pipeline stage and an access type of (`VK_ACCESS_2_MICROMAP_READ_BIT_EXT` | `VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT`). Accesses to VkMicromapBuildInfoEXT::`dstMicromap` **must** be synchronized with the `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` pipeline stage and an access type of `VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT`.

Accesses to other input buffers as identified by any used values of VkMicromapBuildInfoEXT::data or VkMicromapBuildInfoEXT::triangleArray **must** be synchronized with the VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT pipeline stage and an access type of VK_ACCESS_SHADER_READ_BIT.

Valid Usage

- VUID-vkCmdBuildMicromapsEXT-pInfos-07461
  For each pInfos[i], dstMicromap **must** have been created with a value of VkMicromapCreateInfoEXT::size greater than or equal to the memory size required by the build operation, as returned by vkGetMicromapBuildSizesEXT with pBuildInfo = pInfos[i]
- VUID-vkCmdBuildMicromapsEXT-mode-07462
  The mode member of each element of pInfos **must** be a valid VkBuildMicromapModeEXT value
- VUID-vkCmdBuildMicromapsEXT-dstMicromap-07463
  The dstMicromap member of any element of pInfos **must** be a valid VkMicromapEXT handle
- VUID-vkCmdBuildMicromapsEXT-pInfos-07464
  For each element of pInfos its type member **must** match the value of VkMicromapCreateInfoEXT::type when its dstMicromap was created
- VUID-vkCmdBuildMicromapsEXT-dstMicromap-07465
  The range of memory backing the dstMicromap member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the dstMicromap member of any other element of pInfos, which is accessed by this command
- VUID-vkCmdBuildMicromapsEXT-dstMicromap-07466
  The range of memory backing the dstMicromap member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the scratchData member of any element of pInfos (including the same element), which is accessed by this command
- VUID-vkCmdBuildMicromapsEXT-scratchData-07467
  The range of memory backing the scratchData member of any element of pInfos that is accessed by this command **must** not overlap the memory backing the scratchData member of any other element of pInfos, which is accessed by this command
- VUID-vkCmdBuildMicromapsEXT-pInfos-07508
  For each element of pInfos, the buffer used to create its dstMicromap member **must** be bound to device memory
- VUID-vkCmdBuildMicromapsEXT-pInfos-07509
  If pInfos[i].mode is VK_BUILD_MICROMAP_MODE_BUILD_EXT, all addresses between pInfos[i].scratchData.deviceAddress and pInfos[i].scratchData.deviceAddress + N - 1 **must** be in the buffer device address range of the same buffer, where N is given by the buildScratchSize member of the VkMicromapBuildSizesInfoEXT structure returned from a call to vkGetMicromapBuildSizesEXT with an identical VkMicromapBuildInfoEXT structure and primitive count

- VUID-vkCmdBuildMicromapsEXT-data-07510
  The buffers from which the buffer device addresses for all of the `data` and `triangleArray` members of all `pInfos`[i] are queried **must** have been created with the `VK_BUFFER_USAGE_MICROMAP_BUILD_INPUT_READ_ONLY_BIT_EXT` usage flag
- VUID-vkCmdBuildMicromapsEXT-pInfos-07511
  For each element of `pInfos`[i] the buffer from which the buffer device address `pInfos`[i].`scratchData.deviceAddress` is queried **must** have been created with `VK_BUFFER_USAGE_STORAGE_BUFFER_BIT` usage flag
- VUID-vkCmdBuildMicromapsEXT-pInfos-07512
  For each element of `pInfos`, its `scratchData.deviceAddress`, `data.deviceAddress`, and `triangleArray.deviceAddress` members **must** be valid device addresses obtained from vkGetBufferDeviceAddressKHR
- VUID-vkCmdBuildMicromapsEXT-pInfos-07513
  For each element of `pInfos`, if `scratchData.deviceAddress`, `data.deviceAddress`, or `triangleArray.deviceAddress` is the address of a non-sparse buffer then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdBuildMicromapsEXT-pInfos-07514
  For each element of `pInfos`, its `scratchData.deviceAddress` member **must** be a multiple of VkPhysicalDeviceAccelerationStructurePropertiesKHR::`minAccelerationStructureScratchOffsetAlignment`
- VUID-vkCmdBuildMicromapsEXT-pInfos-07515
  For each element of `pInfos`, its `triangleArray.deviceAddress` and `data.deviceAddress` members **must** be a multiple of `256`

Valid Usage (Implicit)

- VUID-vkCmdBuildMicromapsEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBuildMicromapsEXT-pInfos-parameter
  `pInfos` **must** be a valid pointer to an array of `infoCount` valid VkMicromapBuildInfoEXT structures
- VUID-vkCmdBuildMicromapsEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBuildMicromapsEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdBuildMicromapsEXT-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBuildMicromapsEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBuildMicromapsEXT-infoCount-arraylength
  `infoCount` **must** be greater than `0`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

Formats which **can** be set in VkMicromapUsageEXT::`format` and VkMicromapTriangleEXT::`format` for micromap builds, are:

```
// Provided by VK_EXT_opacity_micromap
typedef enum VkOpacityMicromapFormatEXT {
    VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT = 1,
    VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT = 2,
} VkOpacityMicromapFormatEXT;
```

- `VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT` specifies that the given micromap format has one bit per subtriangle encoding either fully opaque or fully transparent.
- `VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT` specifies that the given micromap format has two bits per subtriangle encoding four modes which can be interpreted as described in ray traversal.

NOTE For compactness, these values are stored as 16-bit in some structures.

The `VkMicromapBuildInfoEXT` structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkMicromapBuildInfoEXT {
    VkStructureType                    sType;
    const void*                        pNext;
    VkMicromapTypeEXT                  type;
    VkBuildMicromapFlagsEXT            flags;
    VkBuildMicromapModeEXT             mode;
    VkMicromapEXT                      dstMicromap;
    uint32_t                           usageCountsCount;
    const VkMicromapUsageEXT*          pUsageCounts;
    const VkMicromapUsageEXT* const*   ppUsageCounts;
    VkDeviceOrHostAddressConstKHR      data;
    VkDeviceOrHostAddressKHR           scratchData;
    VkDeviceOrHostAddressConstKHR      triangleArray;
    VkDeviceSize                       triangleArrayStride;
} VkMicromapBuildInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `type` is a VkMicromapTypeEXT value specifying the type of micromap being built.
- `flags` is a bitmask of VkBuildMicromapFlagBitsEXT specifying additional parameters of the micromap.
- `mode` is a VkBuildMicromapModeEXT value specifying the type of operation to perform.
- `dstMicromap` is a pointer to the target micromap for the build.
- `usageCountsCount` specifies the number of usage counts structures that will be used to determine the size of this micromap.
- `pUsageCounts` is a pointer to an array of VkMicromapUsageEXT structures.
- `ppUsageCounts` is a pointer to an array of pointers to VkMicromapUsageEXT structures.
- `data` is the device or host address to memory which contains the data for the micromap.
- `scratchData` is the device or host address to memory that will be used as scratch memory for the build.
- `triangleArray` is the device or host address to memory containing the VkMicromapTriangleEXT data
- `triangleArrayStride` is the stride in bytes between each element of `triangleArray`

Only one of `pUsageCounts` or `ppUsageCounts` **can** be a valid pointer, the other **must** be `NULL`. The elements of the non-`NULL` array describe the total counts used to build each micromap. Each element contains a `count` which is the number of micromap triangles of that `format` and `subdivisionLevel` contained in the micromap. Multiple elements with the same `format` and `subdivisionLevel` are allowed and the total count for that `format` and `subdivisionLevel` is the sum of the `count` for each element.

Each micromap triangle refers to one element in `triangleArray` which contains the `format` and

`subdivisionLevel` for that particular triangle as well as a `dataOffset` in bytes which is the location relative to `data` where that triangle's micromap data begins. The data at `triangleArray` is laid out as a 4 byte unsigned integer for the `dataOffset` followed by a 2 byte unsigned integer for the subdivision level then a 2 byte unsigned integer for the format. In practice, compilers compile VkMicromapTriangleEXT to match this pattern.

For opacity micromaps, the data at `data` is packed as either one bit per element for `VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT` or two bits per element for `VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT` and is packed from LSB to MSB in each byte. The data at each index in those bytes is interpreted as discussed in Ray Opacity Micromap.

Valid Usage

- VUID-VkMicromapBuildInfoEXT-pUsageCounts-07516
  Only one of `pUsageCounts` or `ppUsageCounts` **can** be a valid pointer, the other **must** be `NULL`
- VUID-VkMicromapBuildInfoEXT-type-07517
  If `type` is `VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT` the `format` member of VkMicromapUsageEXT **must** be a valid value from `VkOpacityMicromapFormatEXT`
- VUID-VkMicromapBuildInfoEXT-type-07518
  If `type` is `VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT` the `format` member of VkMicromapTriangleEXT **must** be a valid value from `VkOpacityMicromapFormatEXT`

Valid Usage (Implicit)

- VUID-VkMicromapBuildInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MICROMAP_BUILD_INFO_EXT`
- VUID-VkMicromapBuildInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkMicromapBuildInfoEXT-type-parameter
  `type` **must** be a valid VkMicromapTypeEXT value
- VUID-VkMicromapBuildInfoEXT-flags-parameter
  `flags` **must** be a valid combination of VkBuildMicromapFlagBitsEXT values
- VUID-VkMicromapBuildInfoEXT-pUsageCounts-parameter
  If `usageCountsCount` is not `0`, and `pUsageCounts` is not `NULL`, `pUsageCounts` **must** be a valid pointer to an array of `usageCountsCount` VkMicromapUsageEXT structures
- VUID-VkMicromapBuildInfoEXT-ppUsageCounts-parameter
  If `usageCountsCount` is not `0`, and `ppUsageCounts` is not `NULL`, `ppUsageCounts` **must** be a valid pointer to an array of `usageCountsCount` valid pointers to VkMicromapUsageEXT structures

The `VkBuildMicromapModeEXT` enumeration is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef enum VkBuildMicromapModeEXT {
    VK_BUILD_MICROMAP_MODE_BUILD_EXT = 0,
} VkBuildMicromapModeEXT;
```

- VK_BUILD_MICROMAP_MODE_BUILD_EXT specifies that the destination micromap will be built using the specified data.

The VkMicromapUsageEXT structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkMicromapUsageEXT {
    uint32_t    count;
    uint32_t    subdivisionLevel;
    uint32_t    format;
} VkMicromapUsageEXT;
```

- count is the number of triangles in the usage format defined by the subdivisionLevel and format below in the micromap
- subdivisionLevel is the subdivision level of this usage format
- format is the format of this usage format

Valid Usage

- VUID-VkMicromapUsageEXT-format-07519
  If the VkMicromapTypeEXT of the micromap is VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT then format **must** be VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT or VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT
- VUID-VkMicromapUsageEXT-format-07520
  If the VkMicromapTypeEXT of the micromap is VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT and format is VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT then subdivisionLevel **must** be less than or equal to VkPhysicalDeviceOpacityMicromapPropertiesEXT ::maxOpacity2StateSubdivisionLevel
- VUID-VkMicromapUsageEXT-format-07521
  If the VkMicromapTypeEXT of the micromap is VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT and format is VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT then subdivisionLevel **must** be less than or equal to VkPhysicalDeviceOpacityMicromapPropertiesEXT ::maxOpacity4StateSubdivisionLevel

The format is interpreted based on the type of the micromap using it.

The VkMicromapTriangleEXT structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkMicromapTriangleEXT {
    uint32_t    dataOffset;
    uint16_t    subdivisionLevel;
    uint16_t    format;
} VkMicromapTriangleEXT;
```

- `dataOffset` is the offset in bytes of the start of the data for this triangle. This is a byte aligned value.
- `subdivisionLevel` is the subdivision level of this triangle
- `format` is the format of this triangle

Valid Usage

- VUID-VkMicromapTriangleEXT-format-07522
  If the VkMicromapTypeEXT of the micromap is `VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT` then `format` **must** be `VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT` or `VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT`
- VUID-VkMicromapTriangleEXT-format-07523
  If the VkMicromapTypeEXT of the micromap is `VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT` and `format` is `VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT` then `subdivisionLevel` **must** be less than or equal to VkPhysicalDeviceOpacityMicromapPropertiesEXT ::`maxOpacity2StateSubdivisionLevel`
- VUID-VkMicromapTriangleEXT-format-07524
  If the VkMicromapTypeEXT of the micromap is `VK_MICROMAP_TYPE_OPACITY_MICROMAP_EXT` and `format` is `VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT` then `subdivisionLevel` **must** be less than or equal to VkPhysicalDeviceOpacityMicromapPropertiesEXT ::`maxOpacity4StateSubdivisionLevel`

The `format` is interpreted based on the `type` of the micromap using it.

35.1.2. Copying Micromaps

An additional command exists for copying micromaps without updating their contents. Before copying, an application **must** query the size of the resulting micromap.

To query micromap size parameters call:

```
// Provided by VK_EXT_opacity_micromap
void vkCmdWriteMicromapsPropertiesEXT(
    VkCommandBuffer                             commandBuffer,
    uint32_t                                    micromapCount,
    const VkMicromapEXT*                        pMicromaps,
    VkQueryType                                 queryType,
    VkQueryPool                                 queryPool,
    uint32_t                                    firstQuery);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `micromapCount` is the count of micromaps for which to query the property.
- `pMicromaps` is a pointer to an array of existing previously built micromaps.
- `queryType` is a VkQueryType value specifying the type of queries managed by the pool.
- `queryPool` is the query pool that will manage the results of the query.
- `firstQuery` is the first query index within the query pool that will contain the `micromapCount` number of results.

Accesses to any of the micromaps listed in `pMicromaps` **must** be synchronized with the `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` pipeline stage and an access type of `VK_ACCESS_2_MICROMAP_READ_BIT_EXT`.

- If `queryType` is `VK_QUERY_TYPE_MICROMAP_SERIALIZATION_SIZE_EXT`, then the value written out is the number of bytes required by a serialized micromap.
- If `queryType` is `VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT`, then the value written out is the number of bytes required by a compacted micromap.

Valid Usage

- VUID-vkCmdWriteMicromapsPropertiesEXT-queryPool-07525
  `queryPool` **must** have been created with a `queryType` matching `queryType`
- VUID-vkCmdWriteMicromapsPropertiesEXT-queryPool-07526
  The queries identified by `queryPool` and `firstQuery` **must** be *unavailable*
- VUID-vkCmdWriteMicromapsPropertiesEXT-buffer-07527
  The `buffer` used to create each micromap in `pMicrmaps` **must** be bound to device memory
- VUID-vkCmdWriteMicromapsPropertiesEXT-query-07528
  The sum of `query` plus `micromapCount` **must** be less than or equal to the number of queries in `queryPool`
- VUID-vkCmdWriteMicromapsPropertiesEXT-pMicromaps-07501
  All micromaps in `pMicromaps` **must** have been constructed prior to the execution of this command
- VUID-vkCmdWriteMicromapsPropertiesEXT-pMicromaps-07502
  All micromaps in `pMicromaps` **must** have been constructed with `VK_BUILD_MICROMAP_ALLOW_COMPACTION_BIT_EXT` if `queryType` is

VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT

- VUID-vkCmdWriteMicromapsPropertiesEXT-queryType-07503
  queryType **must** be VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT or VK_QUERY_TYPE_MICROMAP_SERIALIZATION_SIZE_EXT

Valid Usage (Implicit)

- VUID-vkCmdWriteMicromapsPropertiesEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdWriteMicromapsPropertiesEXT-pMicromaps-parameter
  pMicromaps **must** be a valid pointer to an array of micromapCount valid VkMicromapEXT handles
- VUID-vkCmdWriteMicromapsPropertiesEXT-queryType-parameter
  queryType **must** be a valid VkQueryType value
- VUID-vkCmdWriteMicromapsPropertiesEXT-queryPool-parameter
  queryPool **must** be a valid VkQueryPool handle
- VUID-vkCmdWriteMicromapsPropertiesEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdWriteMicromapsPropertiesEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support compute operations
- VUID-vkCmdWriteMicromapsPropertiesEXT-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdWriteMicromapsPropertiesEXT-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdWriteMicromapsPropertiesEXT-micromapCount-arraylength
  micromapCount **must** be greater than 0
- VUID-vkCmdWriteMicromapsPropertiesEXT-commonparent
  Each of commandBuffer, queryPool, and the elements of pMicromaps **must** have been created, allocated, or retrieved from the same VkDevice

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

To copy a micromap call:

```
// Provided by VK_EXT_opacity_micromap
void vkCmdCopyMicromapEXT(
    VkCommandBuffer                             commandBuffer,
    const VkCopyMicromapInfoEXT*                pInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pInfo` is a pointer to a VkCopyMicromapInfoEXT structure defining the copy operation.

This command copies the `pInfo->src` micromap to the `pInfo->dst` micromap in the manner specified by `pInfo->mode`.

Accesses to `pInfo->src` and `pInfo->dst` **must** be synchronized with the `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` pipeline stage and an access type of `VK_ACCESS_2_MICROMAP_READ_BIT_EXT` or `VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT` as appropriate.

Valid Usage

- VUID-vkCmdCopyMicromapEXT-buffer-07529
  The `buffer` used to create `pInfo->src` **must** be bound to device memory
- VUID-vkCmdCopyMicromapEXT-buffer-07530
  The `buffer` used to create `pInfo->dst` **must** be bound to device memory

Valid Usage (Implicit)

- VUID-vkCmdCopyMicromapEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyMicromapEXT-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyMicromapInfoEXT structure
- VUID-vkCmdCopyMicromapEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyMicromapEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations

- VUID-vkCmdCopyMicromapEXT-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyMicromapEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

The `VkCopyMicromapInfoEXT` structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkCopyMicromapInfoEXT {
    VkStructureType          sType;
    const void*              pNext;
    VkMicromapEXT            src;
    VkMicromapEXT            dst;
    VkCopyMicromapModeEXT    mode;
} VkCopyMicromapInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `src` is the source micromap for the copy.
- `dst` is the target micromap for the copy.
- `mode` is a VkCopyMicromapModeEXT value specifying additional operations to perform during the copy.

Valid Usage

- VUID-VkCopyMicromapInfoEXT-mode-07531
  `mode` **must** be `VK_COPY_MICROMAP_MODE_COMPACT_EXT` or `VK_COPY_MICROMAP_MODE_CLONE_EXT`
- VUID-VkCopyMicromapInfoEXT-src-07532

The source acceleration structure `src` **must** have been constructed prior to the execution of this command

- VUID-VkCopyMicromapInfoEXT-mode-07533
  If `mode` is `VK_COPY_MICROMAP_MODE_COMPACT_EXT`, `src` **must** have been constructed with `VK_BUILD_MICROMAP_ALLOW_COMPACTION_BIT_EXT` in the build
- VUID-VkCopyMicromapInfoEXT-buffer-07534
  The `buffer` used to create `src` **must** be bound to device memory
- VUID-VkCopyMicromapInfoEXT-buffer-07535
  The `buffer` used to create `dst` **must** be bound to device memory

Valid Usage (Implicit)

- VUID-VkCopyMicromapInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_MICROMAP_INFO_EXT`
- VUID-VkCopyMicromapInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyMicromapInfoEXT-src-parameter
  `src` **must** be a valid VkMicromapEXT handle
- VUID-VkCopyMicromapInfoEXT-dst-parameter
  `dst` **must** be a valid VkMicromapEXT handle
- VUID-VkCopyMicromapInfoEXT-mode-parameter
  `mode` **must** be a valid VkCopyMicromapModeEXT value
- VUID-VkCopyMicromapInfoEXT-commonparent
  Both of `dst`, and `src` **must** have been created, allocated, or retrieved from the same VkDevice

Possible values of `mode` specifying additional operations to perform during the copy, are:

```
// Provided by VK_EXT_opacity_micromap
typedef enum VkCopyMicromapModeEXT {
    VK_COPY_MICROMAP_MODE_CLONE_EXT = 0,
    VK_COPY_MICROMAP_MODE_SERIALIZE_EXT = 1,
    VK_COPY_MICROMAP_MODE_DESERIALIZE_EXT = 2,
    VK_COPY_MICROMAP_MODE_COMPACT_EXT = 3,
} VkCopyMicromapModeEXT;
```

- `VK_COPY_MICROMAP_MODE_CLONE_EXT` creates a direct copy of the micromap specified in `src` into the one specified by `dst`. The `dst` micromap **must** have been created with the same parameters as `src`.
- `VK_COPY_MICROMAP_MODE_SERIALIZE_EXT` serializes the micromap to a semi-opaque format which can be reloaded on a compatible implementation.
- `VK_COPY_MICROMAP_MODE_DESERIALIZE_EXT` deserializes the semi-opaque serialization format in the

buffer to the micromap.

- `VK_COPY_MICROMAP_MODE_COMPACT_EXT` creates a more compact version of a micromap `src` into `dst`. The micromap `dst` **must** have been created with a size at least as large as that returned by vkCmdWriteMicromapsPropertiesEXT after the build of the micromap specified by `src`.

To copy a micromap to device memory call:

```
// Provided by VK_EXT_opacity_micromap
void vkCmdCopyMicromapToMemoryEXT(
    VkCommandBuffer                             commandBuffer,
    const VkCopyMicromapToMemoryInfoEXT*        pInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pInfo` is an a pointer to a VkCopyMicromapToMemoryInfoEXT structure defining the copy operation.

Accesses to `pInfo->src` **must** be synchronized with the `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` pipeline stage and an access type of `VK_ACCESS_2_MICROMAP_READ_BIT_EXT`. Accesses to the buffer indicated by `pInfo->dst.deviceAddress` **must** be synchronized with the `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` pipeline stage and an access type of `VK_ACCESS_TRANSFER_WRITE_BIT`.

This command produces the same results as vkCopyMicromapToMemoryEXT, but writes its result to a device address, and is executed on the device rather than the host. The output **may** not necessarily be bit-for-bit identical, but it can be equally used by either vkCmdCopyMemoryToMicromapEXT or vkCopyMemoryToMicromapEXT.

The defined header structure for the serialized data consists of:

- `VK_UUID_SIZE` bytes of data matching `VkPhysicalDeviceIDProperties`::`driverUUID`
- `VK_UUID_SIZE` bytes of data identifying the compatibility for comparison using vkGetDeviceMicromapCompatibilityEXT The serialized data is written to the buffer (or read from the buffer) according to the host endianness.

Valid Usage

- VUID-vkCmdCopyMicromapToMemoryEXT-pInfo-07536
  `pInfo->dst.deviceAddress` **must** be a valid device address for a buffer bound to device memory
- VUID-vkCmdCopyMicromapToMemoryEXT-pInfo-07537
  `pInfo->dst.deviceAddress` **must** be aligned to `256` bytes
- VUID-vkCmdCopyMicromapToMemoryEXT-pInfo-07538
  If the buffer pointed to by `pInfo->dst.deviceAddress` is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdCopyMicromapToMemoryEXT-buffer-07539
  The `buffer` used to create `pInfo->src` **must** be bound to device memory

Valid Usage (Implicit)

- VUID-vkCmdCopyMicromapToMemoryEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyMicromapToMemoryEXT-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyMicromapToMemoryInfoEXT structure
- VUID-vkCmdCopyMicromapToMemoryEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdCopyMicromapToMemoryEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdCopyMicromapToMemoryEXT-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyMicromapToMemoryEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkCopyMicromapToMemoryInfoEXT {
    VkStructureType             sType;
    const void*                 pNext;
    VkMicromapEXT               src;
    VkDeviceOrHostAddressKHR    dst;
    VkCopyMicromapModeEXT       mode;
} VkCopyMicromapToMemoryInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.

- `src` is the source micromap for the copy
- `dst` is the device or host address to memory which is the target for the copy
- `mode` is a VkCopyMicromapModeEXT value specifying additional operations to perform during the copy.

Valid Usage

- VUID-VkCopyMicromapToMemoryInfoEXT-src-07540
  The source micromap `src` **must** have been constructed prior to the execution of this command
- VUID-VkCopyMicromapToMemoryInfoEXT-dst-07541
  The memory pointed to by `dst` **must** be at least as large as the serialization size of `src`, as reported by vkWriteMicromapsPropertiesEXT or vkCmdWriteMicromapsPropertiesEXT with a query type of `VK_QUERY_TYPE_MICROMAP_SERIALIZATION_SIZE_EXT`
- VUID-VkCopyMicromapToMemoryInfoEXT-mode-07542
  `mode` **must** be `VK_COPY_MICROMAP_MODE_SERIALIZE_EXT`

Valid Usage (Implicit)

- VUID-VkCopyMicromapToMemoryInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_MICROMAP_TO_MEMORY_INFO_EXT`
- VUID-VkCopyMicromapToMemoryInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyMicromapToMemoryInfoEXT-src-parameter
  `src` **must** be a valid VkMicromapEXT handle
- VUID-VkCopyMicromapToMemoryInfoEXT-mode-parameter
  `mode` **must** be a valid VkCopyMicromapModeEXT value

To copy device memory to a micromap call:

```
// Provided by VK_EXT_opacity_micromap
void vkCmdCopyMemoryToMicromapEXT(
    VkCommandBuffer                             commandBuffer,
    const VkCopyMemoryToMicromapInfoEXT*        pInfo);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pInfo` is a pointer to a VkCopyMemoryToMicromapInfoEXT structure defining the copy operation.

Accesses to `pInfo->dst` **must** be synchronized with the `VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT` pipeline stage and an access type of `VK_ACCESS_2_MICROMAP_READ_BIT_EXT`. Accesses to the buffer indicated by `pInfo->src.deviceAddress` **must** be synchronized with the

VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT pipeline stage and an access type of VK_ACCESS_TRANSFER_READ_BIT.

This command can accept micromaps produced by either vkCmdCopyMicromapToMemoryEXT or vkCopyMicromapToMemoryEXT.

Valid Usage

- VUID-vkCmdCopyMemoryToMicromapEXT-pInfo-07543
  pInfo->src.deviceAddress **must** be a valid device address for a buffer bound to device memory
- VUID-vkCmdCopyMemoryToMicromapEXT-pInfo-07544
  pInfo->src.deviceAddress **must** be aligned to 256 bytes
- VUID-vkCmdCopyMemoryToMicromapEXT-pInfo-07545
  If the buffer pointed to by pInfo->src.deviceAddress is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdCopyMemoryToMicromapEXT-buffer-07546
  The buffer used to create pInfo->dst **must** be bound to device memory

Valid Usage (Implicit)

- VUID-vkCmdCopyMemoryToMicromapEXT-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdCopyMemoryToMicromapEXT-pInfo-parameter
  pInfo **must** be a valid pointer to a valid VkCopyMemoryToMicromapInfoEXT structure
- VUID-vkCmdCopyMemoryToMicromapEXT-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdCopyMemoryToMicromapEXT-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support compute operations
- VUID-vkCmdCopyMemoryToMicromapEXT-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdCopyMemoryToMicromapEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

The VkCopyMemoryToMicromapInfoEXT structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkCopyMemoryToMicromapInfoEXT {
    VkStructureType                  sType;
    const void*                      pNext;
    VkDeviceOrHostAddressConstKHR    src;
    VkMicromapEXT                    dst;
    VkCopyMicromapModeEXT            mode;
} VkCopyMemoryToMicromapInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- src is the device or host address to memory containing the source data for the copy.
- dst is the target micromap for the copy.
- mode is a VkCopyMicromapModeEXT value specifying additional operations to perform during the copy.

Valid Usage

- VUID-VkCopyMemoryToMicromapInfoEXT-src-07547
  The source memory pointed to by src **must** contain data previously serialized using vkCmdCopyMicromapToMemoryEXT
- VUID-VkCopyMemoryToMicromapInfoEXT-mode-07548
  mode **must** be VK_COPY_MICROMAP_MODE_DESERIALIZE_EXT
- VUID-VkCopyMemoryToMicromapInfoEXT-src-07549
  The data in src **must** have a format compatible with the destination physical device as returned by vkGetDeviceMicromapCompatibilityEXT
- VUID-VkCopyMemoryToMicromapInfoEXT-dst-07550
  dst **must** have been created with a size greater than or equal to that used to serialize the data in src

Valid Usage (Implicit)

- VUID-VkCopyMemoryToMicromapInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_COPY_MEMORY_TO_MICROMAP_INFO_EXT`
- VUID-VkCopyMemoryToMicromapInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkCopyMemoryToMicromapInfoEXT-dst-parameter
  `dst` **must** be a valid VkMicromapEXT handle
- VUID-VkCopyMemoryToMicromapInfoEXT-mode-parameter
  `mode` **must** be a valid VkCopyMicromapModeEXT value

To check if a serialized micromap is compatible with the current device call:

```
// Provided by VK_EXT_opacity_micromap
void vkGetDeviceMicromapCompatibilityEXT(
    VkDevice                                    device,
    const VkMicromapVersionInfoEXT*             pVersionInfo,
    VkAccelerationStructureCompatibilityKHR*    pCompatibility);
```

- `device` is the device to check the version against.
- `pVersionInfo` is a pointer to a VkMicromapVersionInfoEXT structure specifying version information to check against the device.
- `pCompatibility` is a pointer to a VkAccelerationStructureCompatibilityKHR value in which compatibility information is returned.

Valid Usage

- VUID-vkGetDeviceMicromapCompatibilityEXT-micromap-07551
  The `micromap` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkGetDeviceMicromapCompatibilityEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetDeviceMicromapCompatibilityEXT-pVersionInfo-parameter
  `pVersionInfo` **must** be a valid pointer to a valid VkMicromapVersionInfoEXT structure
- VUID-vkGetDeviceMicromapCompatibilityEXT-pCompatibility-parameter
  `pCompatibility` **must** be a valid pointer to a VkAccelerationStructureCompatibilityKHR value

The `VkMicromapVersionInfoEXT` structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkMicromapVersionInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    const uint8_t*     pVersionData;
} VkMicromapVersionInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pVersionData` is a pointer to the version header of a micromap as defined in vkCmdCopyMicromapToMemoryEXT

NOTE

`pVersionData` is a *pointer* to an array of 2×`VK_UUID_SIZE` `uint8_t` values instead of two `VK_UUID_SIZE` arrays as the expected use case for this member is to be pointed at the header of a previously serialized micromap (via vkCmdCopyMicromapToMemoryEXT or vkCopyMicromapToMemoryEXT) that is loaded in memory. Using arrays would necessitate extra memory copies of the UUIDs.

Valid Usage (Implicit)

- VUID-VkMicromapVersionInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MICROMAP_VERSION_INFO_EXT`
- VUID-VkMicromapVersionInfoEXT-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkMicromapVersionInfoEXT-pVersionData-parameter
  `pVersionData` **must** be a valid pointer to an array of $2 \times VK_UUID_SIZE$ `uint8_t` values

35.2. Host Micromap Operations

Implementations are also required to provide host implementations of the micromap operations if the `micromapHostCommands` feature is enabled:

- vkBuildMicromapsEXT corresponding to vkCmdBuildMicromapsEXT
- vkCopyMicromapEXT corresponding to vkCmdCopyMicromapEXT
- vkCopyMicromapToMemoryEXT corresponding to vkCmdCopyMicromapToMemoryEXT
- vkCopyMemoryToMicromapEXT corresponding to vkCmdCopyMemoryToMicromapEXT
- vkWriteMicromapsPropertiesEXT corresponding to vkCmdWriteMicromapsPropertiesEXT

These commands are functionally equivalent to their device counterparts, except that they are executed on the host timeline, rather than being enqueued into command buffers.

All micromaps used by the host commands **must** be bound to host-visible memory, and all input

data for micromap builds **must** be referenced using host addresses instead of device addresses. Applications are not required to map micromap memory when using the host commands.

NOTE

The vkBuildMicromapsEXT and vkCmdBuildMicromapsEXT **may** use different algorithms, and thus are not required to produce identical structures.

Apart from these details, the host and device operations are interchangeable.

NOTE

For efficient execution, micromaps manipulated using these commands should always be bound to host cached memory, as the implementation may need to repeatedly read and write this memory during the execution of the command.

To build micromaps on the host, call:

```
// Provided by VK_EXT_opacity_micromap
VkResult vkBuildMicromapsEXT(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    uint32_t                                    infoCount,
    const VkMicromapBuildInfoEXT*               pInfos);
```

- `device` is the `VkDevice` for which the micromaps are being built.
- `deferredOperation` is an optional VkDeferredOperationKHR to request deferral for this command.
- `infoCount` is the number of micromaps to build. It specifies the number of the `pInfos` that **must** be provided.
- `pInfos` is a pointer to an array of `infoCount` VkMicromapBuildInfoEXT structures defining the geometry used to build each micromap.

This command fulfills the same task as vkCmdBuildMicromapsEXT but is executed by the host.

The `vkBuildMicromapsEXT` command provides the ability to initiate multiple micromaps builds, however there is no ordering or synchronization implied between any of the individual micromap builds.

NOTE

This means that there **cannot** be any memory aliasing between any micromap memories or scratch memories being used by any of the builds.

Valid Usage

- VUID-vkBuildMicromapsEXT-pInfos-07461
  For each `pInfos`[i], `dstMicromap` **must** have been created with a value of VkMicromapCreateInfoEXT::`size` greater than or equal to the memory size required by the build operation, as returned by vkGetMicromapBuildSizesEXT with `pBuildInfo` = `pInfos`[i]

- VUID-vkBuildMicromapsEXT-mode-07462
  The `mode` member of each element of `pInfos` **must** be a valid VkBuildMicromapModeEXT value
- VUID-vkBuildMicromapsEXT-dstMicromap-07463
  The `dstMicromap` member of any element of `pInfos` **must** be a valid VkMicromapEXT handle
- VUID-vkBuildMicromapsEXT-pInfos-07464
  For each element of `pInfos` its `type` member **must** match the value of VkMicromapCreateInfoEXT::`type` when its `dstMicromap` was created
- VUID-vkBuildMicromapsEXT-dstMicromap-07465
  The range of memory backing the `dstMicromap` member of any element of `pInfos` that is accessed by this command **must** not overlap the memory backing the `dstMicromap` member of any other element of `pInfos`, which is accessed by this command
- VUID-vkBuildMicromapsEXT-dstMicromap-07466
  The range of memory backing the `dstMicromap` member of any element of `pInfos` that is accessed by this command **must** not overlap the memory backing the `scratchData` member of any element of `pInfos` (including the same element), which is accessed by this command
- VUID-vkBuildMicromapsEXT-scratchData-07467
  The range of memory backing the `scratchData` member of any element of `pInfos` that is accessed by this command **must** not overlap the memory backing the `scratchData` member of any other element of `pInfos`, which is accessed by this command
- VUID-vkBuildMicromapsEXT-pInfos-07552
  For each element of `pInfos`, the `buffer` used to create its `dstMicromap` member **must** be bound to host-visible device memory
- VUID-vkBuildMicromapsEXT-pInfos-07553
  For each element of `pInfos`, all referenced addresses of `pInfos`[i].`data.hostAddress` **must** be valid host memory
- VUID-vkBuildMicromapsEXT-pInfos-07554
  For each element of `pInfos`, all referenced addresses of `pInfos`[i].`triangleArray.hostAddress` **must** be valid host memory
- VUID-vkBuildMicromapsEXT-micromapHostCommands-07555
  The `VkPhysicalDeviceOpacityMicromapFeaturesEXT::micromapHostCommands` feature **must** be enabled
- VUID-vkBuildMicromapsEXT-pInfos-07556
  If `pInfos`[i].`mode` is `VK_BUILD_MICROMAP_MODE_BUILD_EXT`, all addresses between `pInfos`[i].`scratchData.hostAddress` and `pInfos`[i].`scratchData.hostAddress` + N - 1 **must** be valid host memory, where N is given by the `buildScratchSize` member of the VkMicromapBuildSizesInfoEXT structure returned from a call to vkGetMicromapBuildSizesEXT with an identical VkMicromapBuildInfoEXT structure and primitive count
- VUID-vkBuildMicromapsEXT-pInfos-07557
  For each element of `pInfos`, the `buffer` used to create its `dstMicromap` member **must** be

bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkBuildMicromapsEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkBuildMicromapsEXT-deferredOperation-parameter
  If `deferredOperation` is not VK_NULL_HANDLE, `deferredOperation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkBuildMicromapsEXT-pInfos-parameter
  `pInfos` **must** be a valid pointer to an array of `infoCount` valid VkMicromapBuildInfoEXT structures
- VUID-vkBuildMicromapsEXT-infoCount-arraylength
  `infoCount` **must** be greater than `0`
- VUID-vkBuildMicromapsEXT-deferredOperation-parent
  If `deferredOperation` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- VK_SUCCESS
- VK_OPERATION_DEFERRED_KHR
- VK_OPERATION_NOT_DEFERRED_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

To copy or compact a micromap on the host, call:

```
// Provided by VK_EXT_opacity_micromap
VkResult vkCopyMicromapEXT(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    const VkCopyMicromapInfoEXT*                pInfo);
```

- `device` is the device which owns the micromaps.
- `deferredOperation` is an optional VkDeferredOperationKHR to request deferral for this command.
- `pInfo` is a pointer to a VkCopyMicromapInfoEXT structure defining the copy operation.

This command fulfills the same task as vkCmdCopyMicromapEXT but is executed by the host.

Valid Usage

- VUID-vkCopyMicromapEXT-deferredOperation-03678
  Any previous deferred operation that was associated with `deferredOperation` **must** be complete
- VUID-vkCopyMicromapEXT-buffer-07558
  The `buffer` used to create `pInfo->src` **must** be bound to host-visible device memory
- VUID-vkCopyMicromapEXT-buffer-07559
  The `buffer` used to create `pInfo->dst` **must** be bound to host-visible device memory
- VUID-vkCopyMicromapEXT-micromapHostCommands-07560
  The `VkPhysicalDeviceOpacityMicromapFeaturesEXT::micromapHostCommands` feature **must** be enabled
- VUID-vkCopyMicromapEXT-buffer-07561
  The `buffer` used to create `pInfo->src` **must** be bound to memory that was not allocated with multiple instances
- VUID-vkCopyMicromapEXT-buffer-07562
  The `buffer` used to create `pInfo->dst` **must** be bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkCopyMicromapEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCopyMicromapEXT-deferredOperation-parameter
  If `deferredOperation` is not VK_NULL_HANDLE, `deferredOperation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkCopyMicromapEXT-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyMicromapInfoEXT structure
- VUID-vkCopyMicromapEXT-deferredOperation-parent
  If `deferredOperation` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_OPERATION_DEFERRED_KHR`
- `VK_OPERATION_NOT_DEFERRED_KHR`

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

To copy host accessible memory to a micromap, call:

```
// Provided by VK_EXT_opacity_micromap
VkResult vkCopyMemoryToMicromapEXT(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    const VkCopyMemoryToMicromapInfoEXT*        pInfo);
```

- device is the device which owns pInfo->dst.
- deferredOperation is an optional VkDeferredOperationKHR to request deferral for this command.
- pInfo is a pointer to a VkCopyMemoryToMicromapInfoEXT structure defining the copy operation.

This command fulfills the same task as vkCmdCopyMemoryToMicromapEXT but is executed by the host.

This command can accept micromaps produced by either vkCmdCopyMicromapToMemoryEXT or vkCopyMicromapToMemoryEXT.

Valid Usage

- VUID-vkCopyMemoryToMicromapEXT-deferredOperation-03678
  Any previous deferred operation that was associated with deferredOperation **must** be complete
- VUID-vkCopyMemoryToMicromapEXT-pInfo-07563
  pInfo->src.hostAddress **must** be a valid host pointer
- VUID-vkCopyMemoryToMicromapEXT-pInfo-07564
  pInfo->src.hostAddress **must** be aligned to 16 bytes
- VUID-vkCopyMemoryToMicromapEXT-buffer-07565
  The buffer used to create pInfo->dst **must** be bound to host-visible device memory
- VUID-vkCopyMemoryToMicromapEXT-micromapHostCommands-07566
  The VkPhysicalDeviceOpacityMicromapFeaturesEXT::micromapHostCommands feature **must** be enabled
- VUID-vkCopyMemoryToMicromapEXT-buffer-07567
  The buffer used to create pInfo->dst **must** be bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkCopyMemoryToMicromapEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCopyMemoryToMicromapEXT-deferredOperation-parameter
  If `deferredOperation` is not VK_NULL_HANDLE, `deferredOperation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkCopyMemoryToMicromapEXT-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyMemoryToMicromapInfoEXT structure
- VUID-vkCopyMemoryToMicromapEXT-deferredOperation-parent
  If `deferredOperation` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- VK_SUCCESS
- VK_OPERATION_DEFERRED_KHR
- VK_OPERATION_NOT_DEFERRED_KHR

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

To copy a micromap to host accessible memory, call:

```
// Provided by VK_EXT_opacity_micromap
VkResult vkCopyMicromapToMemoryEXT(
    VkDevice                                    device,
    VkDeferredOperationKHR                      deferredOperation,
    const VkCopyMicromapToMemoryInfoEXT*        pInfo);
```

- `device` is the device which owns `pInfo->src`.
- `deferredOperation` is an optional VkDeferredOperationKHR to request deferral for this command.
- `pInfo` is a pointer to a VkCopyMicromapToMemoryInfoEXT structure defining the copy operation.

This command fulfills the same task as vkCmdCopyMicromapToMemoryEXT but is executed by the host.

This command produces the same results as vkCmdCopyMicromapToMemoryEXT, but writes its

result directly to a host pointer, and is executed on the host rather than the device. The output **may** not necessarily be bit-for-bit identical, but it can be equally used by either vkCmdCopyMemoryToMicromapEXT or vkCopyMemoryToMicromapEXT.

Valid Usage

- VUID-vkCopyMicromapToMemoryEXT-deferredOperation-03678
  Any previous deferred operation that was associated with `deferredOperation` **must** be complete
- VUID-vkCopyMicromapToMemoryEXT-buffer-07568
  The `buffer` used to create `pInfo->src` **must** be bound to host-visible device memory
- VUID-vkCopyMicromapToMemoryEXT-pInfo-07569
  `pInfo->dst.hostAddress` **must** be a valid host pointer
- VUID-vkCopyMicromapToMemoryEXT-pInfo-07570
  `pInfo->dst.hostAddress` **must** be aligned to 16 bytes
- VUID-vkCopyMicromapToMemoryEXT-micromapHostCommands-07571
  The `VkPhysicalDeviceOpacityMicromapFeaturesEXT::micromapHostCommands` feature **must** be enabled
- VUID-vkCopyMicromapToMemoryEXT-buffer-07572
  The `buffer` used to create `pInfo->src` **must** be bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkCopyMicromapToMemoryEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkCopyMicromapToMemoryEXT-deferredOperation-parameter
  If `deferredOperation` is not VK_NULL_HANDLE, `deferredOperation` **must** be a valid VkDeferredOperationKHR handle
- VUID-vkCopyMicromapToMemoryEXT-pInfo-parameter
  `pInfo` **must** be a valid pointer to a valid VkCopyMicromapToMemoryInfoEXT structure
- VUID-vkCopyMicromapToMemoryEXT-deferredOperation-parent
  If `deferredOperation` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_OPERATION_DEFERRED_KHR`
- `VK_OPERATION_NOT_DEFERRED_KHR`

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

To query micromap size parameters on the host, call:

```
// Provided by VK_EXT_opacity_micromap
VkResult vkWriteMicromapsPropertiesEXT(
    VkDevice                                    device,
    uint32_t                                    micromapCount,
    const VkMicromapEXT*                        pMicromaps,
    VkQueryType                                 queryType,
    size_t                                      dataSize,
    void*                                       pData,
    size_t                                      stride);
```

- device is the device which owns the micromaps in pMicromaps.
- micromapCount is the count of micromaps for which to query the property.
- pMicromaps is a pointer to an array of existing previously built micromaps.
- queryType is a VkQueryType value specifying the property to be queried.
- dataSize is the size in bytes of the buffer pointed to by pData.
- pData is a pointer to an application-allocated buffer where the results will be written.
- stride is the stride in bytes between results for individual queries within pData.

This command fulfills the same task as vkCmdWriteMicromapsPropertiesEXT but is executed by the host.

Valid Usage

- VUID-vkWriteMicromapsPropertiesEXT-pMicromaps-07501
  All micromaps in pMicromaps **must** have been constructed prior to the execution of this command
- VUID-vkWriteMicromapsPropertiesEXT-pMicromaps-07502
  All micromaps in pMicromaps **must** have been constructed with VK_BUILD_MICROMAP_ALLOW_COMPACTION_BIT_EXT if queryType is VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT
- VUID-vkWriteMicromapsPropertiesEXT-queryType-07503
  queryType **must** be VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT or VK_QUERY_TYPE_MICROMAP_SERIALIZATION_SIZE_EXT
- VUID-vkWriteMicromapsPropertiesEXT-queryType-10071
  If queryType is VK_QUERY_TYPE_MICROMAP_SERIALIZATION_SIZE_EXT or VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT then stride **must** be a multiple of the size of VkDeviceSize

- VUID-vkWriteMicromapsPropertiesEXT-queryType-10072
  If `queryType` is `VK_QUERY_TYPE_MICROMAP_SERIALIZATION_SIZE_EXT` or `VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT` then `pData` **must** point to a VkDeviceSize
- VUID-vkWriteMicromapsPropertiesEXT-dataSize-07576
  `dataSize` **must** be greater than or equal to `micromapCount`*`stride`
- VUID-vkWriteMicromapsPropertiesEXT-buffer-07577
  The `buffer` used to create each micromap in `pMicromaps` **must** be bound to host-visible device memory
- VUID-vkWriteMicromapsPropertiesEXT-micromapHostCommands-07578
  The `VkPhysicalDeviceOpacityMicromapFeaturesEXT::micromapHostCommands` feature **must** be enabled
- VUID-vkWriteMicromapsPropertiesEXT-buffer-07579
  The `buffer` used to create each micromap in `pMicromaps` **must** be bound to memory that was not allocated with multiple instances

Valid Usage (Implicit)

- VUID-vkWriteMicromapsPropertiesEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkWriteMicromapsPropertiesEXT-pMicromaps-parameter
  `pMicromaps` **must** be a valid pointer to an array of `micromapCount` valid VkMicromapEXT handles
- VUID-vkWriteMicromapsPropertiesEXT-queryType-parameter
  `queryType` **must** be a valid VkQueryType value
- VUID-vkWriteMicromapsPropertiesEXT-pData-parameter
  `pData` **must** be a valid pointer to an array of `dataSize` bytes
- VUID-vkWriteMicromapsPropertiesEXT-micromapCount-arraylength
  `micromapCount` **must** be greater than `0`
- VUID-vkWriteMicromapsPropertiesEXT-dataSize-arraylength
  `dataSize` **must** be greater than `0`
- VUID-vkWriteMicromapsPropertiesEXT-pMicromaps-parent
  Each element of `pMicromaps` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

Chapter 36. Ray Traversal

The ray traversal process identifies and handles intersections between a ray and geometries in an acceleration structure.

Ray traversal cannot be started by a Vulkan API command directly - a shader **must** execute `OpRayQueryProceedKHR` or a pipeline trace ray instruction . When the `rayTracingPipeline` feature is enabled, `OpTraceRayKHR` **can** be used for ray tracing in a ray tracing pipeline. When the `rayQuery` feature is enabled, `OpRayQueryProceedKHR` **can** be used in any shader stage.

36.1. Ray Intersection Candidate Determination

Once tracing begins, rays are first tested against instances in a top-level acceleration structure. A ray that intersects an instance will be transformed into the space of the instance to continue traversal within that instance; therefore the transform matrix stored in the instance **must** be invertible.

In case multiple instances are intersected by a ray, the ray transformation into the space of the instance is invariant under the order in which these instances are encountered in the top-level acceleration structure.

NOTE Applying multiple forward and reverse transforms to a ray to transition from one instance to another could result in accumulated errors. Thus an implementation should behave as if the ray is transformed from the origin for each instance independently.

Next, rays are tested against geometries in a bottom-level acceleration structure to determine if a hit occurred between them, initially based only on their geometric properties (i.e. their vertices). The implementation performs similar operations to that of rasterization, but with the effective viewport determined by the parameters of the ray, and the geometry transformed into a space determined by that viewport.

The vertices of each primitive are transformed from acceleration structure space as to ray space r according to the ray origin and direction as follows:

$$\begin{pmatrix} x_r \\ y_r \\ z_r \end{pmatrix} = \begin{pmatrix} a_x^2(1-c)+c & a_x a_y(1-c)-sa_z & a_x a_z(1-c)+sa_y \\ a_x a_y(1-c)+sa_z & a_y^2(1-c)+c & a_y a_z(1-c)-sa_x \\ a_x a_z(1-c)-sa_y & a_y a_z(1-c)+sa_x & a_z^2(1-c)+c \end{pmatrix} \begin{pmatrix} x_{as}-o_x \\ y_{as}-o_y \\ z_{as}-o_z \end{pmatrix}$$

$\mathbf{a}$ is the axis of rotation from the unnormalized ray direction vector $\mathbf{d}$ to the axis vector $\mathbf{k}$:

$$\mathbf{a} = \begin{cases} \dfrac{\mathbf{d}\times\mathbf{k}}{\|\mathbf{d}\times\mathbf{k}\|} & \text{if } \|\mathbf{d}\times\mathbf{k}\| \neq 0 \\ \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} & \text{if } \|\mathbf{d}\times\mathbf{k}\| = 0 \end{cases}$$

s and c are the sine and cosine of the angle of rotation about $\mathbf{a}$ from $\mathbf{d}$ to $\mathbf{k}$:

$$c = \frac{\mathbf{d} \cdot \mathbf{k}}{\|\mathbf{d}\|}$$
$$s = \sqrt{1 - c^2}$$

$\mathbf{k}$ is the unit vector:

$$\mathbf{k} = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix}$$

$\mathbf{o}$ and $\mathbf{d}$ are the ray origin and unnormalized direction, respectively; the vector described by x_{as}, y_{as}, and z_{as} is any position in acceleration structure space; and the vector described by x_r, y_r, and z_r is the same position in ray space.

An *intersection candidate* is a unique point of intersection between a ray and a geometric primitive. For any primitive that has within its bounds a position $\mathbf{xyz_{as}}$ such that

$$\begin{aligned} x_r &= 0 \\ y_r &= 0 \\ t_{min} < -\frac{z_r}{\|\mathbf{d}\|} < t_{max} &\quad \text{if the primitive is a triangle,} \\ t_{min} \leq -\frac{z_r}{\|\mathbf{d}\|} \leq t_{max} &\quad \text{otherwise} \end{aligned}$$

(where $t = -\frac{z_r}{\|\mathbf{d}\|}$), an intersection candidate exists.

Triangle primitive bounds consist of all points on the plane formed by the three vertices and within the bounds of the edges between the vertices, subject to the watertightness constraints below. AABB primitive bounds consist of all points within an implementation-defined bound which includes the specified box.

NOTE The bounds of the AABB including all points internal to the bound implies that a ray started within the AABB will hit that AABB.

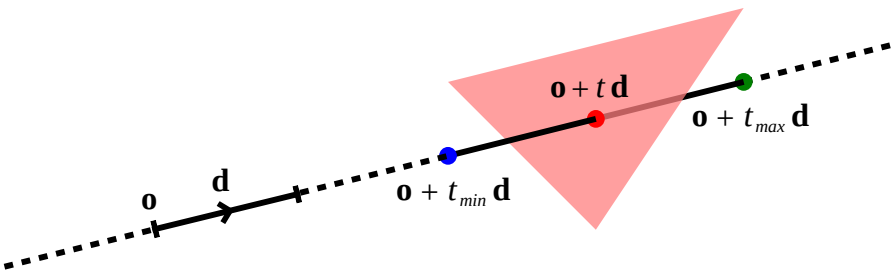


Figure 23. Ray intersection candidate

The determination of this condition is performed in an implementation specific manner, and **may** be performed with floating-point operations. Due to the complexity and number of operations involved, inaccuracies are expected, particularly as the scale of values involved begins to diverge.

Implementations **should** take efforts to maintain as much precision as possible.

NOTE

One very common case is when geometries are close to each other at some distance from the origin in acceleration structure space, where an effect similar to "z-fighting" is likely to be observed. Applications can mitigate this by ensuring their detailed geometries remain close to the origin.

Another likely case is when the origin of a ray is set to a position on a previously intersected surface, and its t_{min} is zero or near zero; an intersection may be detected on the emitting surface. This case can usually be mitigated by offsetting t_{min} slightly.

In the case of AABB geometries, implementations **may** increase their size in an acceleration structure in order to mitigate precision issues. This **may** result in false positive intersections being reported to the application.

For triangle intersection candidates, the b and c barycentric coordinates on the triangle where the above condition is met are made available to future shading. If the ray was traced with a pipeline trace ray instruction, these values are available as a vector of 2 32-bit floating-point values in the `HitAttributeKHR` storage class.

Once an intersection candidate is determined, it proceeds through the following operations, in order:

1. Ray Intersection Culling
2. Ray Intersection Confirmation
3. Ray Closest Hit Determination
4. Ray Result Determination

The sections below describe the exact details of these tests. There is no ordering guarantee between operations performed on different intersection candidates.

36.1.1. Watertightness

For a set of triangles with identical transforms, within a single instance:

- Any set of two or more triangles where all triangles have one vertex with an identical position value, that vertex is a *shared vertex*.
- Any set of two triangles with two shared vertices that were specified in the same winding order in each triangle have a *shared edge* defined by those vertices.

A *closed fan* is a set of three or more triangles where:

- All triangles in the set have the same shared vertex as one of their vertices.
- All edges that include the above vertex are shared edges.
- All above shared edges are shared by exactly two triangles from the set.
- No two triangles in the set intersect, except at shared edges.
- Every triangle in the set is joined to every other triangle in the set by a series of the above

shared edges.

Implementations **should** not double-hit or miss when a ray intersects a shared edge, or a shared vertex of a closed fan.

36.2. Ray Intersection Culling

Candidate intersections go through several phases of culling before confirmation as an actual hit. There is no particular ordering dependency between the different culling operations.

36.2.1. Ray Primitive Culling

If the `rayTraversalPrimitiveCulling` or `rayQuery` features are enabled, the `SkipTrianglesKHR` and `SkipAABBsKHR` ray flags **can** be specified when tracing a ray. `SkipTrianglesKHR` and `SkipAABBsKHR` are mutually exclusive. `SkipTrianglesKHR` is also mutually exclusive with `CullBackFacingTrianglesKHR` and `CullFrontFacingTrianglesKHR`.

If `SkipTrianglesKHR` was included in the `Ray Flags` operand of the ray trace instruction, and the intersection is with a triangle primitive, the intersection is dropped, and no further processing of this intersection occurs. If `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR` was included in the pipeline, traversal with pipeline trace ray instructions will all behave as if `SkipTrianglesKHR` was included in their `Ray Flags` operand.

If `SkipAABBsKHR` was included in the `Ray Flags` operand of the ray trace instruction, and the intersection is with an AABB primitive, the intersection is dropped, and no further processing of this intersection occurs. If `VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR` was included in the pipeline, traversal with pipeline trace ray instructions will all behave as if `SkipAABBsKHR` was included in their `Ray Flags` operand.

36.2.2. Ray Mask Culling

Instances **can** be made invisible to particular rays based on the value of VkAccelerationStructureInstanceKHR::`mask` used to add that instance to a top-level acceleration structure, and the `Cull Mask` parameter used to trace the ray.

For the instance which is intersected, if `mask` & `Cull Mask` == 0, the intersection is dropped, and no further processing occurs.

36.2.3. Ray Face Culling

As in polygon rasterization, one of the stages of ray traversal is to determine if a triangle primitive is back- or front-facing, and primitives **can** be culled based on that facing.

If the intersection candidate is with an AABB primitive, this operation is skipped.

Determination

When a ray intersects a triangle primitive, the order that vertices are specified for the polygon affects whether the ray intersects the front or back face. Front or back facing is determined in the same way as they are for rasterization, based on the sign of the polygon's area but using the ray

space coordinates instead of framebuffer coordinates. One way to compute this area is:

$$a = -\frac{1}{2}\sum_{i=0}^{n-1} x_r^i y_r^{i \oplus 1} - x_r^{i \oplus 1} y_r^i$$

where x_r^i and y_r^i are the x and y ray space coordinates of the ith vertex of the n-vertex polygon (vertices are numbered starting at zero for the purposes of this computation) and i ⊕ 1 is (i + 1) mod n.

By default, if a is negative then the intersection is with the front face of the triangle, otherwise it is with the back face. If VK_GEOMETRY_INSTANCE_TRIANGLE_FLIP_FACING_BIT_KHR is included in VkAccelerationStructureInstanceKHR::flags for the instance containing the intersected triangle, this determination is reversed. Additionally, if a is 0, the intersection candidate is treated as not intersecting with any face, irrespective of the sign.

NOTE In a left-handed coordinate system, an intersection will be with the front face of a triangle if the vertices of the triangle, as defined in index order, appear from the ray's perspective in a clockwise rotation order. VK_GEOMETRY_INSTANCE_TRIANGLE_FLIP_FACING_BIT_KHR was previously annotated as VK_GEOMETRY_INSTANCE_TRIANGLE_FRONT_COUNTERCLOCKWISE_BIT_KHR because of this.

If the ray was traced with a pipeline trace ray instruction, the HitKindKHR built-in is set to HitKindFrontFacingTriangleKHR if the intersection is with front-facing geometry, and HitKindBackFacingTriangleKHR if the intersection is with back-facing geometry, for shader stages considering this intersection.

If the ray was traced with OpRayQueryProceedKHR, OpRayQueryGetIntersectionFrontFaceKHR will return true for intersection candidates with front faces, or false for back faces.

Culling

If CullBackFacingTrianglesKHR was included in the Ray Flags parameter of the ray trace instruction, and the intersection is determined as with the back face of a triangle primitive, the intersection is dropped, and no further processing of this intersection occurs.

If CullFrontFacingTrianglesKHR was included in the Ray Flags parameter of the ray trace instruction, and the intersection is determined as with the front face of a triangle primitive, the intersection is dropped, and no further processing of this intersection occurs.

This culling is disabled if VK_GEOMETRY_INSTANCE_TRIANGLE_FACING_CULL_DISABLE_BIT_KHR was included in VkAccelerationStructureInstanceKHR::flags for the instance which the intersected geometry belongs to.

Intersection candidates that have not intersected with any face (a == 0) are unconditionally culled, irrespective of ray flags and geometry instance flags.

The CullBackFacingTrianglesKHR and CullFrontFacingTrianglesKHR Ray Flags are mutually exclusive.

36.2.4. Ray Opacity Culling

Each geometry in the acceleration structure **may** be considered either opaque or not. Opaque geometries continue through traversal as normal, whereas non-opaque geometries need to be either confirmed or discarded by shader code. Intersection candidates **can** also be culled based on their opacity.

Determination

Each individual intersection candidate is initially determined as opaque if `VK_GEOMETRY_OPAQUE_BIT_KHR` was included in the VkAccelerationStructureGeometryKHR::`flags` when the geometry it intersected with was built, otherwise it is considered non-opaque.

If the geometry includes an opacity micromap, the opacity of the intersection at this point is instead derived as described in Ray Opacity Micromap.

If the intersection candidate was generated by an intersection shader, the intersection is initially considered to have opacity matching the AABB candidate that it was generated from.

However, this opacity can be overridden when it is built into an instance. Setting `VK_GEOMETRY_INSTANCE_FORCE_OPAQUE_BIT_KHR` in VkAccelerationStructureInstanceKHR::`flags` will force all geometries in the instance to be considered opaque. Similarly, setting `VK_GEOMETRY_INSTANCE_FORCE_NO_OPAQUE_BIT_KHR` will force all geometries in the instance to be considered non-opaque.

This can again be overridden by including `OpaqueKHR` or `NoOpaqueKHR` in the `Ray Flags` parameter when tracing a ray. `OpaqueKHR` forces all geometries to behave as if they are opaque, regardless of their build parameters. Similarly, `NoOpaqueKHR` forces all geometries to behave as if they are non-opaque.

If the ray was traced with `OpRayQueryProceedKHR`, to determine the opacity of AABB intersection candidates, `OpRayQueryGetIntersectionCandidateAABBOpaqueKHR` **can** be used. This instruction will return `true` for opaque intersection candidates, and `false` for non-opaque intersection candidates.

Culling

If `CullOpaqueKHR` is included in the `Ray Flags` parameter when tracing a ray, an intersection with a geometry that is considered opaque is dropped, and no further processing occurs.

If `CullNoOpaqueKHR` is included in the `Ray Flags` parameter when tracing a ray, an intersection with a geometry that is considered non-opaque is dropped, and no further processing occurs.

The `OpaqueKHR`, `NoOpaqueKHR`, `CullOpaqueKHR`, and `CullNoOpaqueKHR` `Ray Flags` are mutually exclusive.

36.2.5. Ray Opacity Micromap

A `VK_GEOMETRY_TYPE_TRIANGLES_KHR` geometry in the acceleration structure **may** have an opacity micromap associated with it to give finer-grained opacity information.

If the intersection candidate is with a geometry with an associated opacity micromap and `VK_GEOMETRY_INSTANCE_DISABLE_OPACITY_MICROMAPS_EXT` is not set in its instance then the micromap is used to determine geometry opacity instead of the `VK_GEOMETRY_OPAQUE_BIT_KHR` flag in the geometry.

The opacity information in the micromap object is accessed using the candidate intersection u and v coordinates. The integer u and v are computed from ⌊u⌋ + ⌊v⌋, clamping ⌊u⌋ as needed to keep the sum less than or equal to 1 << subdivisionlevel. These values are mapped into a linear index with a space filling curve which is defined recursively by traversing into the sub-triangle nearest vertex 0, then the middle triangle with ordering flipped, then nearest vertex 1 then nearest vertex 2.

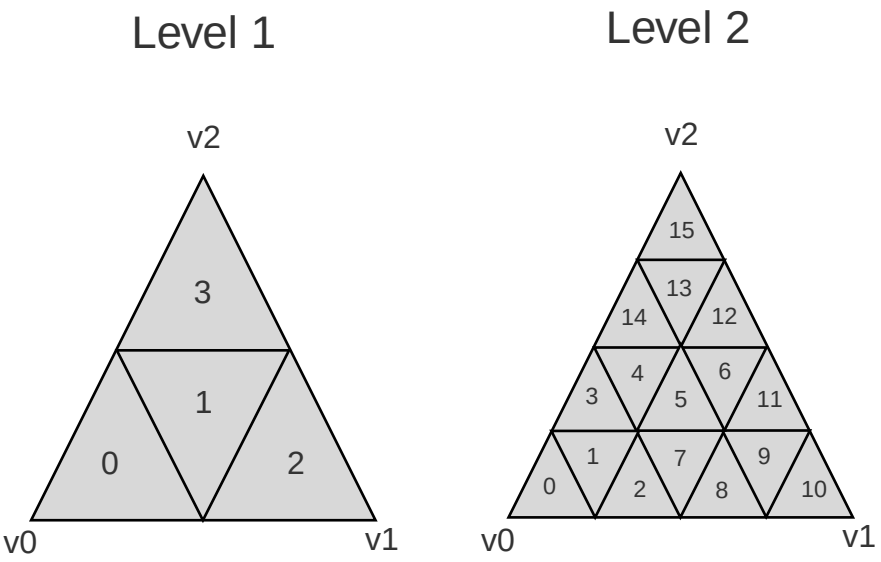


Figure 24. Example ordering for micromap data

NOTE

This encoding is spatially coherent, purely hierarchical, and allows a bit-parallel conversion between barycentric address and index values.

See the appendix for reference code implementing this mapping.

The result of the opacity micromap lookup and operations is to treat the intersection as opaque, non-opaque, or ignored. The interpretation of the values depends on `VK_GEOMETRY_INSTANCE_FORCE_OPACITY_MICROMAP_2_STATE_EXT` in the instance of the candidate intersection or `ForceOpacityMicromap2StateEXT` ray flags on the ray. If either is set, the opacity micromap information is interpreted in 2 state override mode. If the result of the micromap lookup is to treat the intersection candidate as ignored, no further processing of that candidate is done.

If the associated opacity micromap has format `VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT`, each element of the micromap is represented by a single bit at the index derived above.

If the associated opacity micromap has format `VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT`, each element is represented by a two bit value at the index derived above.

4 State value	2 State value	Special index value	2 State override	Result
0	0	`VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_TRANSPARENT_EXT`	Y	Ignored

4 State value	2 State value	Special index value	2 State override	Result
0	0	VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_TRANSPARENT_EXT	N	Ignored
1	1	VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_OPAQUE_EXT	Y	Opaque
1	1	VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_OPAQUE_EXT	N	Opaque
2		VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_UNKNOWN_TRANSPARENT_EXT	Y	Ignored
2		VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_UNKNOWN_TRANSPARENT_EXT	N	Non-opaque
3		VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_UNKNOWN_OPAQUE_EXT	Y	Opaque
3		VK_OPACITY_MICROMAP_SPECIAL_INDEX_FULLY_UNKNOWN_OPAQUE_EXT	N	Non-opaque

36.3. Ray Intersection Confirmation

Depending on the opacity of intersected geometry and whether it is a triangle or an AABB, candidate intersections are further processed to determine the eventual hit result. Candidates generated from AABB intersections run through the same confirmation process as triangle hits.

36.3.1. AABB Intersection Candidates

For an intersection candidate with an AABB geometry generated by Ray Intersection Candidate Determination, shader code is executed to determine whether any hits should be reported to the traversal infrastructure; no further processing of this intersection candidate occurs. The occurrence of an AABB intersection candidate does not guarantee the ray intersects the primitive bounds. To avoid propagating false intersections the application **should** verify the intersection candidate before reporting any hits and only report intersections within the bounds of the AABB. Reporting an intersection outside the AABB either through the implementation giving a conservative bound or reporting a t out of range is legal but **may** result in unpredictable closest hit results.

If the ray was traced with a pipeline trace ray instruction, an intersection shader is invoked from the Shader Binding Table according to the specified indexing for the intersected geometry. If this

shader calls OpReportIntersectionKHR, a new intersection candidate is generated as described below. If the intersection shader is VK_SHADER_UNUSED_KHR (which is only allowed for a zero shader group) then no further processing of the intersection candidate occurs.

Each new candidate generated as a result of this processing is a generated intersection candidate from the intersection with AABB geometry, with a t value equal to the Hit parameter of the OpReportIntersectionKHR instruction. The new generated candidate is then independently run through Ray Intersection Confirmation as a generated intersection.

If the ray was traced with OpRayQueryProceedKHR, control is returned to the shader which executed OpRayQueryProceedKHR, returning true. The resulting ray query has a candidate intersection type of RayQueryCandidateIntersectionAABBKHR. OpRayQueryGenerateIntersectionKHR **can** be called to commit a new intersection candidate with committed intersection type of RayQueryCommittedIntersectionGeneratedKHR. Further ray query processing **can** be continued by executing OpRayQueryProceedKHR with the same ray query, or intersection **can** be terminated with OpRayQueryTerminateKHR. Unlike rays traced with a pipeline trace ray instruction, candidates generated in this way skip generated intersection candidate confirmation; applications **should** make this determination before generating the intersection.

This operation **may** be executed multiple times for the same intersection candidate.

36.3.2. Triangle and Generated Intersection Candidates

For triangle and generated intersection candidates, additional shader code **may** be executed based on the intersection's opacity.

If the intersection is opaque, the candidate is immediately confirmed as a valid hit and passes to the next stage of processing.

For non-opaque intersection candidates, shader code is executed to determine whether a hit occurred or not.

If the ray was traced with a pipeline trace ray instruction, an any-hit shader is invoked from the Shader Binding Table according to the specified indexing. If this shader calls OpIgnoreIntersectionKHR, the candidate is dropped and no further processing of the candidate occurs. If the any-hit shader identified is VK_SHADER_UNUSED_KHR, the candidate is immediately confirmed as a valid hit and passes to the next stage of processing.

If the ray was traced with OpRayQueryProceedKHR, control is returned to the shader which executed OpRayQueryProceedKHR, returning true. As only triangle candidates participate in this operation with ray queries, the resulting candidate intersection type is always RayQueryCandidateIntersectionTriangleKHR. OpRayQueryConfirmIntersectionKHR **can** be called on the ray query to confirm the candidate as a hit with committed intersection type of RayQueryCommittedIntersectionTriangleKHR. Further ray query processing **can** be continued by executing OpRayQueryProceedKHR with the same ray query, or intersection **can** be terminated with OpRayQueryTerminateKHR. If OpRayQueryConfirmIntersectionKHR has not been executed, the candidate is dropped and no further processing of the candidate occurs.

This operation **may** be executed multiple times for the same intersection candidate unless VK_GEOMETRY_NO_DUPLICATE_ANY_HIT_INVOCATION_BIT_KHR was specified for the intersected geometry.

36.4. Ray Closest Hit Determination

Unless the ray was traced with the `TerminateOnFirstHitKHR` ray flag, the implementation **must** track the closest confirmed hit until all geometries have been tested and either confirmed or dropped.

After an intersection candidate is confirmed, its t value is compared to t_{max} to determine which intersection is closer, where t is the parametric distance along the ray at which the intersection occurred.

- If $t < t_{max}$, t_{max} is set to t and the candidate is set as the current closest hit.
- If $t > t_{max}$, the candidate is dropped and no further processing of that candidate occurs.
- If $t = t_{max}$, the candidate **may** be set as the current closest hit or dropped.

If `TerminateOnFirstHitKHR` was included in the `Ray Flags` used to trace the ray, once the first hit is confirmed, the ray trace is terminated.

36.5. Ray Result Determination

Once all candidates have finished processing the prior stages, or if the ray is forcibly terminated, the final result of the ray trace is determined.

If a closest hit result was identified by Ray Closest Hit Determination, a closest hit has occurred, otherwise the final result is a miss.

For rays traced with pipeline trace ray instructions which **can** invoke a closest hit shader, if a closest hit result was identified, a closest hit shader is invoked from the Shader Binding Table according to the specified indexing for the intersected geometry. Control returns to the shader that executed the pipeline trace ray instruction once this shader returns. This shader is skipped if either the ray flags included `SkipClosestHitShaderKHR`, or if the closest hit shader identified is `VK_SHADER_UNUSED_KHR`.

For rays traced with a pipeline trace ray instruction where no hit result was identified, the miss shader identified by the `Miss Index` parameter of the instruction is invoked. Control returns to the shader that executed the pipeline trace ray instruction once this shader returns. This shader is skipped if the miss shader identified is `VK_SHADER_UNUSED_KHR`.

If the ray was traced with `OpRayQueryProceedKHR`, control is returned to the shader which executed `OpRayQueryProceedKHR`, returning `false`. If a closest hit was identified by Ray Closest Hit Determination, the ray query will now have a committed intersection type of `RayQueryCommittedIntersectionGeneratedKHR` or `RayQueryCommittedIntersectionTriangleKHR`. If no closest hit was identified, the committed intersection type will be `RayQueryCommittedIntersectionNoneKHR`.

No further processing of a ray query occurs after this result is determined.

Chapter 37. Ray Tracing

Ray tracing uses a separate rendering pipeline from both the graphics and compute pipelines (see Ray Tracing Pipeline).

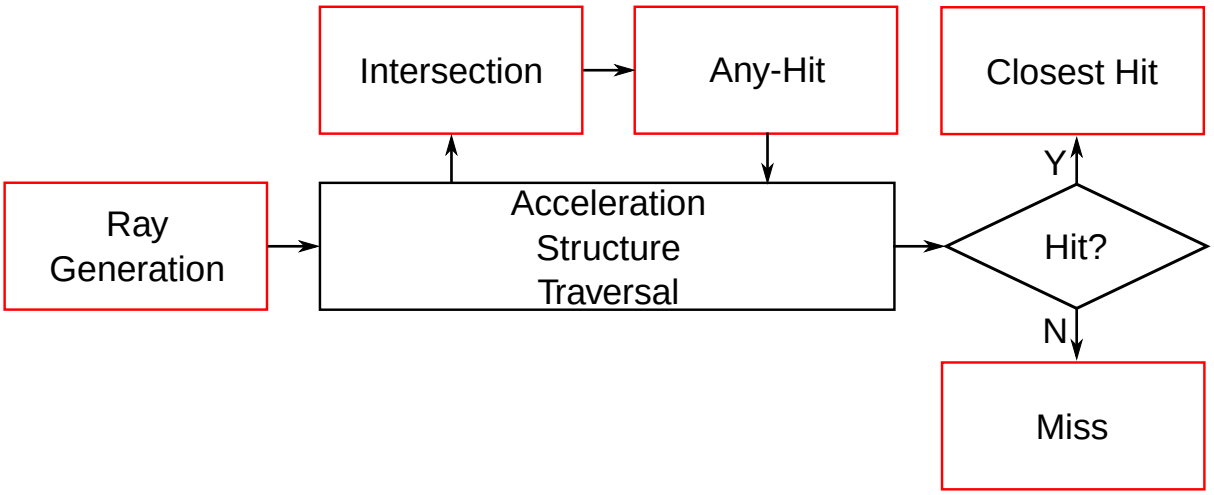


Figure 25. Ray tracing pipeline execution

Caption

Interaction between the different shader stages in the ray tracing pipeline

Within the ray tracing pipeline, a pipeline trace ray instruction **can** be called to perform a ray traversal that invokes the various ray tracing shader stages during its execution. The relationship between the ray tracing pipeline object and the geometries present in the acceleration structure traversed is passed into the ray tracing command in a VkBuffer object known as a *shader binding table*. `OpExecuteCallableKHR` can also be used in ray tracing pipelines to invoke a callable shader.

During execution, control alternates between scheduling and other operations. The scheduling functionality is implementation-specific and is responsible for workload execution. The shader stages are programmable. *Traversal*, which refers to the process of traversing acceleration structures to find potential intersections of rays with geometry, is fixed function.

The programmable portions of the pipeline are exposed in a single-ray programming model, with each invocation handling one ray at a time. Memory operations **can** be synchronized using standard memory barriers. The `Workgroup` scope and variables with a storage class of `Workgroup` **must** not be used in the ray tracing pipeline.

37.1. Shader Call Instructions

A *shader call* is an instruction which **may** cause execution to continue elsewhere by creating one or more invocations that execute a different shader stage.

The following table lists all shader call instructions and which stages each one **can** directly call.

Instruction	Intersection	Any-Hit	Closest Hit	Miss	Callable
OpTraceRayKHR	X	X	X	X	
OpReportIntersectionKHR		X			
OpExecuteCallableKHR					X

Pipeline trace ray instructions **can** be used recursively; invoked shaders **can** themselves execute pipeline trace ray instructions, to a maximum depth defined by the maxRayRecursionDepth limit.

Shaders directly invoked from the API always have a recursion depth of 0; each shader executed by a pipeline trace ray instruction has a recursion depth one higher than the recursion depth of the shader which invoked it. Applications **must** not invoke a shader with a recursion depth greater than the value of maxPipelineRayRecursionDepth specified in the pipeline.

There is no explicit recursion limit for other shader call instructions which may recurse (e.g. OpExecuteCallableKHR) but there is an upper bound determined by the stack size.

An *invocation repack instruction* is a ray tracing instruction where the implementation **may** change the set of invocations that are executing. When a repack instruction is encountered, the invocation is suspended and a new invocation begins and executes the instruction. After executing the repack instruction (which **may** result in other ray tracing shader stages executing) the new invocation ends and the original invocation is resumed, but it **may** be resumed in a different subgroup or at a different SubgroupLocalInvocationId within the same subgroup. When a subset of invocations in a subgroup execute the invocation repack instruction, those that do not execute it remain in the same subgroup at the same SubgroupLocalInvocationId.

The OpTraceRayKHR, OpReportIntersectionKHR, and OpExecuteCallableKHR instructions are invocation repack instructions.

When a ray tracing shader executes a dynamic instance of an invocation repack instruction which results in another ray tracing shader being invoked, their instructions are related by shader-call-order.

For ray tracing invocations that are shader-call-related:

- memory operations on StorageBuffer, Image, and ShaderRecordBufferKHR storage classes **can** be synchronized using the ShaderCallKHR scope.
- the CallableDataKHR, IncomingCallableDataKHR, RayPayloadKHR, HitAttributeKHR, and IncomingRayPayloadKHR storage classes are system-synchronized and no application availability and visibility operations are required.
- memory operations within a single invocation before and after the shader call instruction are ordered by program-order and do not require explicit synchronization.

37.2. Ray Tracing Commands

Ray tracing commands provoke work in the ray tracing pipeline. Ray tracing commands are recorded into a command buffer and when executed by a queue will produce work that executes

according to the bound ray tracing pipeline. A ray tracing pipeline **must** be bound to a command buffer before any ray tracing commands are recorded in that command buffer.

To dispatch ray tracing use:

```
// Provided by VK_KHR_ray_tracing_pipeline
void vkCmdTraceRaysKHR(
    VkCommandBuffer                             commandBuffer,
    const VkStridedDeviceAddressRegionKHR*      pRaygenShaderBindingTable,
    const VkStridedDeviceAddressRegionKHR*      pMissShaderBindingTable,
    const VkStridedDeviceAddressRegionKHR*      pHitShaderBindingTable,
    const VkStridedDeviceAddressRegionKHR*      pCallableShaderBindingTable,
    uint32_t                                    width,
    uint32_t                                    height,
    uint32_t                                    depth);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pRaygenShaderBindingTable` is a VkStridedDeviceAddressRegionKHR that holds the shader binding table data for the ray generation shader stage.
- `pMissShaderBindingTable` is a VkStridedDeviceAddressRegionKHR that holds the shader binding table data for the miss shader stage.
- `pHitShaderBindingTable` is a VkStridedDeviceAddressRegionKHR that holds the shader binding table data for the hit shader stage.
- `pCallableShaderBindingTable` is a VkStridedDeviceAddressRegionKHR that holds the shader binding table data for the callable shader stage.
- `width` is the width of the ray trace query dimensions.
- `height` is height of the ray trace query dimensions.
- `depth` is depth of the ray trace query dimensions.

When the command is executed, a ray generation group of `width` × `height` × `depth` rays is assembled.

Valid Usage

- VUID-vkCmdTraceRaysKHR-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdTraceRaysKHR-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdTraceRaysKHR-unnormalizedCoordinates-09635

If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's levelCount and layerCount **must** be 1

- VUID-vkCmdTraceRaysKHR-unnormalizedCoordinates-09636
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's viewType **must** be VK_IMAGE_VIEW_TYPE_1D or VK_IMAGE_VIEW_TYPE_2D
- VUID-vkCmdTraceRaysKHR-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdTraceRaysKHR-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdTraceRaysKHR-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdTraceRaysKHR-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdTraceRaysKHR-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdTraceRaysKHR-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdTraceRaysKHR-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdTraceRaysKHR-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysKHR-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the

OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT

- VUID-vkCmdTraceRaysKHR-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysKHR-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysKHR-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdTraceRaysKHR-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdTraceRaysKHR-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdTraceRaysKHR-maintenance4-08602
  If the maintenance4 feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdTraceRaysKHR-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdTraceRaysKHR-None-08606
  If the shaderObject feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdTraceRaysKHR-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound

- VUID-vkCmdTraceRaysKHR-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, VK_IMAGE_VIEW_TYPE_1D_ARRAY, VK_IMAGE_VIEW_TYPE_2D_ARRAY or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY, in any shader stage
- VUID-vkCmdTraceRaysKHR-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V OpImageSample* or OpImageSparseSample* instructions with ImplicitLod, Dref or Proj in their name, in any shader stage
- VUID-vkCmdTraceRaysKHR-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V OpImageSample* or OpImageSparseSample* instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdTraceRaysKHR-None-08607
  If the shaderObject feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdTraceRaysKHR-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysKHR-None-08612
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysKHR-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysKHR-None-08613
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysKHR-None-06550

If a bound shader accesses a VkSampler or VkImageView object that enables sampler Y'C$_B$C$_R$ conversion, that object **must** only be used with OpImageSample* or OpImageSparseSample* instructions

- VUID-vkCmdTraceRaysKHR-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler Y'C$_B$C$_R$ conversion, that object **must** not use the ConstOffset and Offset operands
- VUID-vkCmdTraceRaysKHR-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's viewType **must** match the Dim operand of the OpTypeImage as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdTraceRaysKHR-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's format and the Sampled Type operand of the OpTypeImage **must** match
- VUID-vkCmdTraceRaysKHR-OpImageWrite-08795
  If a VkImageView created with a format other than VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdTraceRaysKHR-OpImageWrite-08796
  If a VkImageView created with the format VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have four components
- VUID-vkCmdTraceRaysKHR-OpImageWrite-04469
  If a VkBufferView is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdTraceRaysKHR-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdTraceRaysKHR-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdTraceRaysKHR-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdTraceRaysKHR-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdTraceRaysKHR-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the

VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command

- VUID-vkCmdTraceRaysKHR-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdTraceRaysKHR-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdTraceRaysKHR-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdTraceRaysKHR-None-03429
  Any shader group handle referenced by this call **must** have been queried from the bound ray tracing pipeline
- VUID-vkCmdTraceRaysKHR-None-09458
  If the bound ray tracing pipeline state was created with the VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR dynamic state enabled then vkCmdSetRayTracingPipelineStackSizeKHR **must** have been called in the current command buffer prior to this trace command
- VUID-vkCmdTraceRaysKHR-maxPipelineRayRecursionDepth-03679
  This command **must** not cause a shader call instruction to be executed from a shader invocation with a recursion depth greater than the value of maxPipelineRayRecursionDepth used to create the bound ray tracing pipeline
- VUID-vkCmdTraceRaysKHR-commandBuffer-03635
  commandBuffer **must** not be a protected command buffer
- VUID-vkCmdTraceRaysKHR-size-04023
  The size member of pRayGenShaderBindingTable **must** be equal to its stride member
- VUID-vkCmdTraceRaysKHR-pRayGenShaderBindingTable-03680
  If the buffer from which pRayGenShaderBindingTable->deviceAddress was queried is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdTraceRaysKHR-pRayGenShaderBindingTable-03681
  The buffer from which the pRayGenShaderBindingTable->deviceAddress is queried **must** have been created with the VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR usage flag
- VUID-vkCmdTraceRaysKHR-pRayGenShaderBindingTable-03682
  pRayGenShaderBindingTable->deviceAddress **must** be a multiple of VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupBaseAlignment
- VUID-vkCmdTraceRaysKHR-pMissShaderBindingTable-03683
  If the buffer from which pMissShaderBindingTable->deviceAddress was queried is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory

object

- VUID-vkCmdTraceRaysKHR-pMissShaderBindingTable-03684
  The buffer from which the pMissShaderBindingTable->deviceAddress is queried **must** have been created with the VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR usage flag
- VUID-vkCmdTraceRaysKHR-pMissShaderBindingTable-03685
  pMissShaderBindingTable->deviceAddress **must** be a multiple of VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupBaseAlignment
- VUID-vkCmdTraceRaysKHR-stride-03686
  pMissShaderBindingTable->stride **must** be a multiple of VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupHandleAlignment
- VUID-vkCmdTraceRaysKHR-stride-04029
  pMissShaderBindingTable->stride **must** be less than or equal to VkPhysicalDeviceRayTracingPipelinePropertiesKHR::maxShaderGroupStride
- VUID-vkCmdTraceRaysKHR-pHitShaderBindingTable-03687
  If the buffer from which pHitShaderBindingTable->deviceAddress was queried is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdTraceRaysKHR-pHitShaderBindingTable-03688
  The buffer from which the pHitShaderBindingTable->deviceAddress is queried **must** have been created with the VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR usage flag
- VUID-vkCmdTraceRaysKHR-pHitShaderBindingTable-03689
  pHitShaderBindingTable->deviceAddress **must** be a multiple of VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupBaseAlignment
- VUID-vkCmdTraceRaysKHR-stride-03690
  pHitShaderBindingTable->stride **must** be a multiple of VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupHandleAlignment
- VUID-vkCmdTraceRaysKHR-stride-04035
  pHitShaderBindingTable->stride **must** be less than or equal to VkPhysicalDeviceRayTracingPipelinePropertiesKHR::maxShaderGroupStride
- VUID-vkCmdTraceRaysKHR-pCallableShaderBindingTable-03691
  If the buffer from which pCallableShaderBindingTable->deviceAddress was queried is non-sparse then it **must** be bound completely and contiguously to a single VkDeviceMemory object
- VUID-vkCmdTraceRaysKHR-pCallableShaderBindingTable-03692
  The buffer from which the pCallableShaderBindingTable->deviceAddress is queried **must** have been created with the VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR usage flag
- VUID-vkCmdTraceRaysKHR-pCallableShaderBindingTable-03693
  pCallableShaderBindingTable->deviceAddress **must** be a multiple of VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupBaseAlignment
- VUID-vkCmdTraceRaysKHR-stride-03694
  pCallableShaderBindingTable->stride **must** be a multiple of VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupHandleAlignment

- VUID-vkCmdTraceRaysKHR-stride-04041
  pCallableShaderBindingTable->stride **must** be less than or equal to VkPhysicalDeviceRayTracingPipelinePropertiesKHR::maxShaderGroupStride
- VUID-vkCmdTraceRaysKHR-flags-03696
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR, pHitShaderBindingTable->deviceAddress **must** not be zero
- VUID-vkCmdTraceRaysKHR-flags-03697
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR, pHitShaderBindingTable->deviceAddress **must** not be zero
- VUID-vkCmdTraceRaysKHR-flags-03511
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR, the shader group handle identified by pMissShaderBindingTable->deviceAddress **must** not be zero
- VUID-vkCmdTraceRaysKHR-flags-03512
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR, entries in the table identified by pHitShaderBindingTable->deviceAddress accessed as a result of this command in order to execute an any-hit shader **must** not be zero
- VUID-vkCmdTraceRaysKHR-flags-03513
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR, entries in the table identified by pHitShaderBindingTable->deviceAddress accessed as a result of this command in order to execute a closest hit shader **must** not be zero
- VUID-vkCmdTraceRaysKHR-flags-03514
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR, entries in the table identified by pHitShaderBindingTable->deviceAddress accessed as a result of this command in order to execute an intersection shader **must** not be zero
- VUID-vkCmdTraceRaysKHR-pHitShaderBindingTable-04735
  Any non-zero hit shader group entries in the table identified by pHitShaderBindingTable->deviceAddress accessed by this call from a geometry with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR **must** have been created with VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR
- VUID-vkCmdTraceRaysKHR-pHitShaderBindingTable-04736
  Any non-zero hit shader group entries in the table identified by pHitShaderBindingTable->deviceAddress accessed by this call from a geometry with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR **must** have been created with VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR
- VUID-vkCmdTraceRaysKHR-width-03638
  width **must** be less than or equal to VkPhysicalDeviceLimits::maxComputeWorkGroupCount[0] × VkPhysicalDeviceLimits::maxComputeWorkGroupSize[0]

- VUID-vkCmdTraceRaysKHR-height-03639
  `height` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[1] × `VkPhysicalDeviceLimits`::`maxComputeWorkGroupSize`[1]
- VUID-vkCmdTraceRaysKHR-depth-03640
  `depth` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[2] × `VkPhysicalDeviceLimits`::`maxComputeWorkGroupSize`[2]
- VUID-vkCmdTraceRaysKHR-width-03641
  `width` × `height` × `depth` **must** be less than or equal to `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`maxRayDispatchInvocationCount`

Valid Usage (Implicit)

- VUID-vkCmdTraceRaysKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdTraceRaysKHR-pRaygenShaderBindingTable-parameter
  `pRaygenShaderBindingTable` **must** be a valid pointer to a valid VkStridedDeviceAddressRegionKHR structure
- VUID-vkCmdTraceRaysKHR-pMissShaderBindingTable-parameter
  `pMissShaderBindingTable` **must** be a valid pointer to a valid VkStridedDeviceAddressRegionKHR structure
- VUID-vkCmdTraceRaysKHR-pHitShaderBindingTable-parameter
  `pHitShaderBindingTable` **must** be a valid pointer to a valid VkStridedDeviceAddressRegionKHR structure
- VUID-vkCmdTraceRaysKHR-pCallableShaderBindingTable-parameter
  `pCallableShaderBindingTable` **must** be a valid pointer to a valid VkStridedDeviceAddressRegionKHR structure
- VUID-vkCmdTraceRaysKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdTraceRaysKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdTraceRaysKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdTraceRaysKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

To dispatch ray tracing, with some parameters sourced on the device, use:

```
// Provided by VK_KHR_ray_tracing_pipeline
void vkCmdTraceRaysIndirectKHR(
    VkCommandBuffer                             commandBuffer,
    const VkStridedDeviceAddressRegionKHR*      pRaygenShaderBindingTable,
    const VkStridedDeviceAddressRegionKHR*      pMissShaderBindingTable,
    const VkStridedDeviceAddressRegionKHR*      pHitShaderBindingTable,
    const VkStridedDeviceAddressRegionKHR*      pCallableShaderBindingTable,
    VkDeviceAddress                             indirectDeviceAddress);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `pRaygenShaderBindingTable` is a VkStridedDeviceAddressRegionKHR that holds the shader binding table data for the ray generation shader stage.
- `pMissShaderBindingTable` is a VkStridedDeviceAddressRegionKHR that holds the shader binding table data for the miss shader stage.
- `pHitShaderBindingTable` is a VkStridedDeviceAddressRegionKHR that holds the shader binding table data for the hit shader stage.
- `pCallableShaderBindingTable` is a VkStridedDeviceAddressRegionKHR that holds the shader binding table data for the callable shader stage.
- `indirectDeviceAddress` is a buffer device address which is a pointer to a VkTraceRaysIndirectCommandKHR structure containing the trace ray parameters.

`vkCmdTraceRaysIndirectKHR` behaves similarly to vkCmdTraceRaysKHR except that the ray trace query dimensions are read by the device from `indirectDeviceAddress` during execution.

Valid Usage

- VUID-vkCmdTraceRaysIndirectKHR-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdTraceRaysIndirectKHR-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this

command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT

- VUID-vkCmdTraceRaysIndirectKHR-unnormalizedCoordinates-09635
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's levelCount and layerCount **must** be 1
- VUID-vkCmdTraceRaysIndirectKHR-unnormalizedCoordinates-09636
  If a VkSampler created with unnormalizedCoordinates equal to VK_TRUE is used to sample a VkImageView as a result of this command, then the image view's viewType **must** be VK_IMAGE_VIEW_TYPE_1D or VK_IMAGE_VIEW_TYPE_2D
- VUID-vkCmdTraceRaysIndirectKHR-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT
- VUID-vkCmdTraceRaysIndirectKHR-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
- VUID-vkCmdTraceRaysIndirectKHR-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdTraceRaysIndirectKHR-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdTraceRaysIndirectKHR-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdTraceRaysIndirectKHR-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdTraceRaysIndirectKHR-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdTraceRaysIndirectKHR-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain

VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT

- VUID-vkCmdTraceRaysIndirectKHR-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysIndirectKHR-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysIndirectKHR-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysIndirectKHR-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdTraceRaysIndirectKHR-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdTraceRaysIndirectKHR-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdTraceRaysIndirectKHR-maintenance4-08602
  If the maintenance4 feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdTraceRaysIndirectKHR-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdTraceRaysIndirectKHR-None-08606
  If the shaderObject feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdTraceRaysIndirectKHR-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not

have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound

- VUID-vkCmdTraceRaysIndirectKHR-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, VK_IMAGE_VIEW_TYPE_1D_ARRAY, VK_IMAGE_VIEW_TYPE_2D_ARRAY or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY, in any shader stage
- VUID-vkCmdTraceRaysIndirectKHR-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V OpImageSample* or OpImageSparseSample* instructions with ImplicitLod, Dref or Proj in their name, in any shader stage
- VUID-vkCmdTraceRaysIndirectKHR-None-08611
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V OpImageSample* or OpImageSparseSample* instructions that includes a LOD bias or any offset values, in any shader stage
- VUID-vkCmdTraceRaysIndirectKHR-None-08607
  If the shaderObject feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdTraceRaysIndirectKHR-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysIndirectKHR-None-08612
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysIndirectKHR-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the robustBufferAccess feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysIndirectKHR-None-08613
  If the robustBufferAccess feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer,

it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point

- VUID-vkCmdTraceRaysIndirectKHR-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler Y'C$_B$C$_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdTraceRaysIndirectKHR-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler Y'C$_B$C$_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdTraceRaysIndirectKHR-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdTraceRaysIndirectKHR-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match
- VUID-vkCmdTraceRaysIndirectKHR-OpImageWrite-08795
  If a VkImageView created with a format other than `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdTraceRaysIndirectKHR-OpImageWrite-08796
  If a VkImageView created with the format `VK_FORMAT_A8_UNORM_KHR` is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have four components
- VUID-vkCmdTraceRaysIndirectKHR-OpImageWrite-04469
  If a VkBufferView is accessed using `OpImageWrite` as a result of this command, then the `Type` of the `Texel` operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdTraceRaysIndirectKHR-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdTraceRaysIndirectKHR-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 32
- VUID-vkCmdTraceRaysIndirectKHR-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that instruction **must** have a `Width` of 64
- VUID-vkCmdTraceRaysIndirectKHR-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the `SampledType` of the `OpTypeImage` operand of that

instruction **must** have a `Width` of 32

- VUID-vkCmdTraceRaysIndirectKHR-sparseImageInt64Atomics-04474
  If the `sparseImageInt64Atomics` feature is not enabled, VkImage objects created with the `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command
- VUID-vkCmdTraceRaysIndirectKHR-sparseImageInt64Atomics-04475
  If the `sparseImageInt64Atomics` feature is not enabled, VkBuffer objects created with the `VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT` flag **must** not be accessed by atomic instructions through an `OpTypeImage` with a `SampledType` with a `Width` of 64 by this command
- VUID-vkCmdTraceRaysIndirectKHR-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdTraceRaysIndirectKHR-None-09600
  If a descriptor with type equal to any of `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written
- VUID-vkCmdTraceRaysIndirectKHR-None-03429
  Any shader group handle referenced by this call **must** have been queried from the bound ray tracing pipeline
- VUID-vkCmdTraceRaysIndirectKHR-None-09458
  If the bound ray tracing pipeline state was created with the `VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR` dynamic state enabled then vkCmdSetRayTracingPipelineStackSizeKHR **must** have been called in the current command buffer prior to this trace command
- VUID-vkCmdTraceRaysIndirectKHR-maxPipelineRayRecursionDepth-03679
  This command **must** not cause a shader call instruction to be executed from a shader invocation with a recursion depth greater than the value of `maxPipelineRayRecursionDepth` used to create the bound ray tracing pipeline
- VUID-vkCmdTraceRaysIndirectKHR-commandBuffer-03635
  `commandBuffer` **must** not be a protected command buffer
- VUID-vkCmdTraceRaysIndirectKHR-size-04023
  The `size` member of `pRayGenShaderBindingTable` **must** be equal to its `stride` member
- VUID-vkCmdTraceRaysIndirectKHR-pRayGenShaderBindingTable-03680
  If the buffer from which `pRayGenShaderBindingTable->deviceAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdTraceRaysIndirectKHR-pRayGenShaderBindingTable-03681
  The buffer from which the `pRayGenShaderBindingTable->deviceAddress` is queried **must** have been created with the `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` usage flag
- VUID-vkCmdTraceRaysIndirectKHR-pRayGenShaderBindingTable-03682
  `pRayGenShaderBindingTable->deviceAddress` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupBaseAlignment`

- VUID-vkCmdTraceRaysIndirectKHR-pMissShaderBindingTable-03683
  If the buffer from which `pMissShaderBindingTable->deviceAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdTraceRaysIndirectKHR-pMissShaderBindingTable-03684
  The buffer from which the `pMissShaderBindingTable->deviceAddress` is queried **must** have been created with the `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` usage flag
- VUID-vkCmdTraceRaysIndirectKHR-pMissShaderBindingTable-03685
  `pMissShaderBindingTable->deviceAddress` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupBaseAlignment`
- VUID-vkCmdTraceRaysIndirectKHR-stride-03686
  `pMissShaderBindingTable->stride` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupHandleAlignment`
- VUID-vkCmdTraceRaysIndirectKHR-stride-04029
  `pMissShaderBindingTable->stride` **must** be less than or equal to `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`maxShaderGroupStride`
- VUID-vkCmdTraceRaysIndirectKHR-pHitShaderBindingTable-03687
  If the buffer from which `pHitShaderBindingTable->deviceAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdTraceRaysIndirectKHR-pHitShaderBindingTable-03688
  The buffer from which the `pHitShaderBindingTable->deviceAddress` is queried **must** have been created with the `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` usage flag
- VUID-vkCmdTraceRaysIndirectKHR-pHitShaderBindingTable-03689
  `pHitShaderBindingTable->deviceAddress` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupBaseAlignment`
- VUID-vkCmdTraceRaysIndirectKHR-stride-03690
  `pHitShaderBindingTable->stride` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupHandleAlignment`
- VUID-vkCmdTraceRaysIndirectKHR-stride-04035
  `pHitShaderBindingTable->stride` **must** be less than or equal to `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`maxShaderGroupStride`
- VUID-vkCmdTraceRaysIndirectKHR-pCallableShaderBindingTable-03691
  If the buffer from which `pCallableShaderBindingTable->deviceAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdTraceRaysIndirectKHR-pCallableShaderBindingTable-03692
  The buffer from which the `pCallableShaderBindingTable->deviceAddress` is queried **must** have been created with the `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` usage flag
- VUID-vkCmdTraceRaysIndirectKHR-pCallableShaderBindingTable-03693
  `pCallableShaderBindingTable->deviceAddress` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupBaseAlignment`
- VUID-vkCmdTraceRaysIndirectKHR-stride-03694

pCallableShaderBindingTable->stride **must** be a multiple of VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupHandleAlignment

- VUID-vkCmdTraceRaysIndirectKHR-stride-04041
  pCallableShaderBindingTable->stride **must** be less than or equal to VkPhysicalDeviceRayTracingPipelinePropertiesKHR::maxShaderGroupStride
- VUID-vkCmdTraceRaysIndirectKHR-flags-03696
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR, pHitShaderBindingTable->deviceAddress **must** not be zero
- VUID-vkCmdTraceRaysIndirectKHR-flags-03697
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR, pHitShaderBindingTable->deviceAddress **must** not be zero
- VUID-vkCmdTraceRaysIndirectKHR-flags-03511
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR, the shader group handle identified by pMissShaderBindingTable->deviceAddress **must** not be zero
- VUID-vkCmdTraceRaysIndirectKHR-flags-03512
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR, entries in the table identified by pHitShaderBindingTable->deviceAddress accessed as a result of this command in order to execute an any-hit shader **must** not be zero
- VUID-vkCmdTraceRaysIndirectKHR-flags-03513
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR, entries in the table identified by pHitShaderBindingTable->deviceAddress accessed as a result of this command in order to execute a closest hit shader **must** not be zero
- VUID-vkCmdTraceRaysIndirectKHR-flags-03514
  If the bound ray tracing pipeline was created with flags that included VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR, entries in the table identified by pHitShaderBindingTable->deviceAddress accessed as a result of this command in order to execute an intersection shader **must** not be zero
- VUID-vkCmdTraceRaysIndirectKHR-pHitShaderBindingTable-04735
  Any non-zero hit shader group entries in the table identified by pHitShaderBindingTable->deviceAddress accessed by this call from a geometry with a geometryType of VK_GEOMETRY_TYPE_TRIANGLES_KHR **must** have been created with VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR
- VUID-vkCmdTraceRaysIndirectKHR-pHitShaderBindingTable-04736
  Any non-zero hit shader group entries in the table identified by pHitShaderBindingTable->deviceAddress accessed by this call from a geometry with a geometryType of VK_GEOMETRY_TYPE_AABBS_KHR **must** have been created with VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR
- VUID-vkCmdTraceRaysIndirectKHR-indirectDeviceAddress-03632

If the buffer from which `indirectDeviceAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object

- VUID-vkCmdTraceRaysIndirectKHR-indirectDeviceAddress-03633
  The buffer from which `indirectDeviceAddress` was queried **must** have been created with the `VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT` bit set
- VUID-vkCmdTraceRaysIndirectKHR-indirectDeviceAddress-03634
  `indirectDeviceAddress` **must** be a multiple of `4`
- VUID-vkCmdTraceRaysIndirectKHR-indirectDeviceAddress-03636
  All device addresses between `indirectDeviceAddress` and `indirectDeviceAddress` + `sizeof` (`VkTraceRaysIndirectCommandKHR`) - 1 **must** be in the buffer device address range of the same buffer
- VUID-vkCmdTraceRaysIndirectKHR-rayTracingPipelineTraceRaysIndirect-03637
  The `rayTracingPipelineTraceRaysIndirect` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCmdTraceRaysIndirectKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdTraceRaysIndirectKHR-pRaygenShaderBindingTable-parameter
  `pRaygenShaderBindingTable` **must** be a valid pointer to a valid VkStridedDeviceAddressRegionKHR structure
- VUID-vkCmdTraceRaysIndirectKHR-pMissShaderBindingTable-parameter
  `pMissShaderBindingTable` **must** be a valid pointer to a valid VkStridedDeviceAddressRegionKHR structure
- VUID-vkCmdTraceRaysIndirectKHR-pHitShaderBindingTable-parameter
  `pHitShaderBindingTable` **must** be a valid pointer to a valid VkStridedDeviceAddressRegionKHR structure
- VUID-vkCmdTraceRaysIndirectKHR-pCallableShaderBindingTable-parameter
  `pCallableShaderBindingTable` **must** be a valid pointer to a valid VkStridedDeviceAddressRegionKHR structure
- VUID-vkCmdTraceRaysIndirectKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdTraceRaysIndirectKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdTraceRaysIndirectKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdTraceRaysIndirectKHR-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

The `VkTraceRaysIndirectCommandKHR` structure is defined as:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef struct VkTraceRaysIndirectCommandKHR {
    uint32_t    width;
    uint32_t    height;
    uint32_t    depth;
} VkTraceRaysIndirectCommandKHR;
```

- `width` is the width of the ray trace query dimensions.
- `height` is height of the ray trace query dimensions.
- `depth` is depth of the ray trace query dimensions.

The members of `VkTraceRaysIndirectCommandKHR` have the same meaning as the similarly named parameters of vkCmdTraceRaysKHR.

Valid Usage

- VUID-VkTraceRaysIndirectCommandKHR-width-03638
  `width` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[0] × `VkPhysicalDeviceLimits`::`maxComputeWorkGroupSize`[0]
- VUID-VkTraceRaysIndirectCommandKHR-height-03639
  `height` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[1] × `VkPhysicalDeviceLimits`::`maxComputeWorkGroupSize`[1]
- VUID-VkTraceRaysIndirectCommandKHR-depth-03640
  `depth` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[2] × `VkPhysicalDeviceLimits`::`maxComputeWorkGroupSize`[2]
- VUID-VkTraceRaysIndirectCommandKHR-width-03641
  `width` × `height` × `depth` **must** be less than or equal to

```
    VkPhysicalDeviceRayTracingPipelinePropertiesKHR::maxRayDispatchInvocationCount
```

To dispatch ray tracing, with some parameters sourced on the device, use:

```
// Provided by VK_KHR_ray_tracing_maintenance1 with VK_KHR_ray_tracing_pipeline
void vkCmdTraceRaysIndirect2KHR(
    VkCommandBuffer                             commandBuffer,
    VkDeviceAddress                             indirectDeviceAddress);
```

- `commandBuffer` is the command buffer into which the command will be recorded.
- `indirectDeviceAddress` is a buffer device address which is a pointer to a VkTraceRaysIndirectCommand2KHR structure containing the trace ray parameters.

`vkCmdTraceRaysIndirect2KHR` behaves similarly to vkCmdTraceRaysIndirectKHR except that shader binding table parameters as well as dispatch dimensions are read by the device from `indirectDeviceAddress` during execution.

Valid Usage

- VUID-vkCmdTraceRaysIndirect2KHR-magFilter-04553
  If a VkSampler created with `magFilter` or `minFilter` equal to `VK_FILTER_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdTraceRaysIndirect2KHR-mipmapMode-04770
  If a VkSampler created with `mipmapMode` equal to `VK_SAMPLER_MIPMAP_MODE_LINEAR`, and `compareEnable` equal to `VK_FALSE` is used to sample a VkImageView as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT`
- VUID-vkCmdTraceRaysIndirect2KHR-unnormalizedCoordinates-09635
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `levelCount` and `layerCount` **must** be 1
- VUID-vkCmdTraceRaysIndirect2KHR-unnormalizedCoordinates-09636
  If a VkSampler created with `unnormalizedCoordinates` equal to `VK_TRUE` is used to sample a VkImageView as a result of this command, then the image view's `viewType` **must** be `VK_IMAGE_VIEW_TYPE_1D` or `VK_IMAGE_VIEW_TYPE_2D`
- VUID-vkCmdTraceRaysIndirect2KHR-None-06479
  If a VkImageView is sampled with depth comparison, the image view's format features **must** contain `VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT`
- VUID-vkCmdTraceRaysIndirect2KHR-None-02691
  If a VkImageView is accessed using atomic operations as a result of this command, then the image view's format features **must** contain `VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT`

- VUID-vkCmdTraceRaysIndirect2KHR-None-07888
  If a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor is accessed using atomic operations as a result of this command, then the storage texel buffer's format features **must** contain VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
- VUID-vkCmdTraceRaysIndirect2KHR-None-02692
  If a VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, then the image view's format features **must** contain VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- VUID-vkCmdTraceRaysIndirect2KHR-None-02693
  If the VK_EXT_filter_cubic extension is not enabled and any VkImageView is sampled with VK_FILTER_CUBIC_EXT as a result of this command, it **must** not have a VkImageViewType of VK_IMAGE_VIEW_TYPE_3D, VK_IMAGE_VIEW_TYPE_CUBE, or VK_IMAGE_VIEW_TYPE_CUBE_ARRAY
- VUID-vkCmdTraceRaysIndirect2KHR-filterCubic-02694
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT as a result of this command **must** have a VkImageViewType and format that supports cubic filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubic returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdTraceRaysIndirect2KHR-filterCubicMinmax-02695
  Any VkImageView being sampled with VK_FILTER_CUBIC_EXT with a reduction mode of either VK_SAMPLER_REDUCTION_MODE_MIN or VK_SAMPLER_REDUCTION_MODE_MAX as a result of this command **must** have a VkImageViewType and format that supports cubic filtering together with minmax filtering, as specified by VkFilterCubicImageViewImageFormatPropertiesEXT::filterCubicMinmax returned by vkGetPhysicalDeviceImageFormatProperties2KHR
- VUID-vkCmdTraceRaysIndirect2KHR-OpTypeImage-07027
  For any VkImageView being written as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysIndirect2KHR-OpTypeImage-07028
  For any VkImageView being read as a storage image where the image format field of the OpTypeImage is Unknown, the view's format features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysIndirect2KHR-OpTypeImage-07029
  For any VkBufferView being written as a storage texel buffer where the image format field of the OpTypeImage is Unknown, the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysIndirect2KHR-OpTypeImage-07030
  Any VkBufferView being read as a storage texel buffer where the image format field of the OpTypeImage is Unknown then the view's buffer features **must** contain VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT
- VUID-vkCmdTraceRaysIndirect2KHR-None-08600
  For each set *n* that is statically used by a bound shader, a descriptor set **must** have been bound to *n* at the same pipeline bind point, with a VkPipelineLayout that is compatible for set *n*, with the VkPipelineLayout used to create the current VkPipeline or the

VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility

- VUID-vkCmdTraceRaysIndirect2KHR-None-08601
  For each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout array used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdTraceRaysIndirect2KHR-None-10068
  For each array of resources that is used by a bound shader, the indices used to access members of the array **must** be less than the descriptor count for the identified binding in the descriptor sets used by this command
- VUID-vkCmdTraceRaysIndirect2KHR-maintenance4-08602
  If the `maintenance4` feature is not enabled, then for each push constant that is statically used by a bound shader, a push constant value **must** have been set for the same pipeline bind point, with a VkPipelineLayout that is compatible for push constants, with the VkPipelineLayout used to create the current VkPipeline or the VkDescriptorSetLayout and VkPushConstantRange arrays used to create the current VkShaderEXT , as described in Pipeline Layout Compatibility
- VUID-vkCmdTraceRaysIndirect2KHR-None-08114
  Descriptors in each bound descriptor set, specified via vkCmdBindDescriptorSets, **must** be valid as described by descriptor validity if they are statically used by a bound shader
- VUID-vkCmdTraceRaysIndirect2KHR-None-08606
  If the `shaderObject` feature is not enabled, a valid pipeline **must** be bound to the pipeline bind point used by this command
- VUID-vkCmdTraceRaysIndirect2KHR-None-08608
  If a pipeline is bound to the pipeline bind point used by this command, there **must** not have been any calls to dynamic state setting commands for any state specified statically in the VkPipeline object bound to the pipeline bind point used by this command, since that pipeline was bound
- VUID-vkCmdTraceRaysIndirect2KHR-None-08609
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used to sample from any VkImage with a VkImageView of the type `VK_IMAGE_VIEW_TYPE_3D`, `VK_IMAGE_VIEW_TYPE_CUBE`, `VK_IMAGE_VIEW_TYPE_1D_ARRAY`, `VK_IMAGE_VIEW_TYPE_2D_ARRAY` or `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY`, in any shader stage
- VUID-vkCmdTraceRaysIndirect2KHR-None-08610
  If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions with `ImplicitLod`, `Dref` or `Proj` in their name, in any shader stage
- VUID-vkCmdTraceRaysIndirect2KHR-None-08611

If the VkPipeline object bound to the pipeline bind point used by this command or any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a VkSampler object that uses unnormalized coordinates, that sampler **must** not be used with any of the SPIR-V `OpImageSample*` or `OpImageSparseSample*` instructions that includes a LOD bias or any offset values, in any shader stage

- VUID-vkCmdTraceRaysIndirect2KHR-None-08607
  If the `shaderObject` feature is enabled, either a valid pipeline **must** be bound to the pipeline bind point used by this command, or a valid combination of valid and VK_NULL_HANDLE shader objects **must** be bound to every supported shader stage corresponding to the pipeline bind point used by this command
- VUID-vkCmdTraceRaysIndirect2KHR-uniformBuffers-06935
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a uniform buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysIndirect2KHR-None-08612
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a uniform buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysIndirect2KHR-storageBuffers-06936
  If any stage of the VkPipeline object bound to the pipeline bind point used by this command accesses a storage buffer, and the `robustBufferAccess` feature is not enabled, that stage **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysIndirect2KHR-None-08613
  If the `robustBufferAccess` feature is not enabled, and any VkShaderEXT bound to a stage corresponding to the pipeline bind point used by this command accesses a storage buffer, it **must** not access values outside of the range of the buffer as specified in the descriptor set bound to the same pipeline bind point
- VUID-vkCmdTraceRaysIndirect2KHR-None-06550
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** only be used with `OpImageSample*` or `OpImageSparseSample*` instructions
- VUID-vkCmdTraceRaysIndirect2KHR-ConstOffset-06551
  If a bound shader accesses a VkSampler or VkImageView object that enables sampler $Y'C_BC_R$ conversion, that object **must** not use the `ConstOffset` and `Offset` operands
- VUID-vkCmdTraceRaysIndirect2KHR-viewType-07752
  If a VkImageView is accessed as a result of this command, then the image view's `viewType` **must** match the `Dim` operand of the `OpTypeImage` as described in Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types
- VUID-vkCmdTraceRaysIndirect2KHR-format-07753
  If a VkImageView is accessed as a result of this command, then the numeric type of the image view's `format` and the `Sampled Type` operand of the `OpTypeImage` **must** match

- VUID-vkCmdTraceRaysIndirect2KHR-OpImageWrite-08795
  If a VkImageView created with a format other than VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the image view's format
- VUID-vkCmdTraceRaysIndirect2KHR-OpImageWrite-08796
  If a VkImageView created with the format VK_FORMAT_A8_UNORM_KHR is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have four components
- VUID-vkCmdTraceRaysIndirect2KHR-OpImageWrite-04469
  If a VkBufferView is accessed using OpImageWrite as a result of this command, then the Type of the Texel operand of that instruction **must** have at least as many components as the buffer view's format
- VUID-vkCmdTraceRaysIndirect2KHR-SampledType-04470
  If a VkImageView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdTraceRaysIndirect2KHR-SampledType-04471
  If a VkImageView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdTraceRaysIndirect2KHR-SampledType-04472
  If a VkBufferView with a VkFormat that has a 64-bit component width is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 64
- VUID-vkCmdTraceRaysIndirect2KHR-SampledType-04473
  If a VkBufferView with a VkFormat that has a component width less than 64-bit is accessed as a result of this command, the SampledType of the OpTypeImage operand of that instruction **must** have a Width of 32
- VUID-vkCmdTraceRaysIndirect2KHR-sparseImageInt64Atomics-04474
  If the sparseImageInt64Atomics feature is not enabled, VkImage objects created with the VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdTraceRaysIndirect2KHR-sparseImageInt64Atomics-04475
  If the sparseImageInt64Atomics feature is not enabled, VkBuffer objects created with the VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT flag **must** not be accessed by atomic instructions through an OpTypeImage with a SampledType with a Width of 64 by this command
- VUID-vkCmdTraceRaysIndirect2KHR-None-07288
  Any shader invocation executed by this command **must** terminate
- VUID-vkCmdTraceRaysIndirect2KHR-None-09600
  If a descriptor with type equal to any of VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, or VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT is accessed as a result of this command, the image subresource identified by that descriptor **must** be in the image layout identified when the descriptor was written

- VUID-vkCmdTraceRaysIndirect2KHR-None-03429
  Any shader group handle referenced by this call **must** have been queried from the bound ray tracing pipeline
- VUID-vkCmdTraceRaysIndirect2KHR-None-09458
  If the bound ray tracing pipeline state was created with the `VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR` dynamic state enabled then vkCmdSetRayTracingPipelineStackSizeKHR **must** have been called in the current command buffer prior to this trace command
- VUID-vkCmdTraceRaysIndirect2KHR-maxPipelineRayRecursionDepth-03679
  This command **must** not cause a shader call instruction to be executed from a shader invocation with a recursion depth greater than the value of `maxPipelineRayRecursionDepth` used to create the bound ray tracing pipeline
- VUID-vkCmdTraceRaysIndirect2KHR-commandBuffer-03635
  `commandBuffer` **must** not be a protected command buffer
- VUID-vkCmdTraceRaysIndirect2KHR-indirectDeviceAddress-03632
  If the buffer from which `indirectDeviceAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-vkCmdTraceRaysIndirect2KHR-indirectDeviceAddress-03633
  The buffer from which `indirectDeviceAddress` was queried **must** have been created with the `VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT` bit set
- VUID-vkCmdTraceRaysIndirect2KHR-indirectDeviceAddress-03634
  `indirectDeviceAddress` **must** be a multiple of `4`
- VUID-vkCmdTraceRaysIndirect2KHR-indirectDeviceAddress-03636
  All device addresses between `indirectDeviceAddress` and `indirectDeviceAddress` + `sizeof` (`VkTraceRaysIndirectCommand2KHR`) - 1 **must** be in the buffer device address range of the same buffer
- VUID-vkCmdTraceRaysIndirect2KHR-rayTracingPipelineTraceRaysIndirect2-03637
  The `rayTracingPipelineTraceRaysIndirect2` feature **must** be enabled

Valid Usage (Implicit)

- VUID-vkCmdTraceRaysIndirect2KHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdTraceRaysIndirect2KHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdTraceRaysIndirect2KHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support compute operations
- VUID-vkCmdTraceRaysIndirect2KHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdTraceRaysIndirect2KHR-videocoding

This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Outside	Outside	Compute	Action

The `VkTraceRaysIndirectCommand2KHR` structure is defined as:

```
// Provided by VK_KHR_ray_tracing_maintenance1 with VK_KHR_ray_tracing_pipeline
typedef struct VkTraceRaysIndirectCommand2KHR {
    VkDeviceAddress    raygenShaderRecordAddress;
    VkDeviceSize       raygenShaderRecordSize;
    VkDeviceAddress    missShaderBindingTableAddress;
    VkDeviceSize       missShaderBindingTableSize;
    VkDeviceSize       missShaderBindingTableStride;
    VkDeviceAddress    hitShaderBindingTableAddress;
    VkDeviceSize       hitShaderBindingTableSize;
    VkDeviceSize       hitShaderBindingTableStride;
    VkDeviceAddress    callableShaderBindingTableAddress;
    VkDeviceSize       callableShaderBindingTableSize;
    VkDeviceSize       callableShaderBindingTableStride;
    uint32_t           width;
    uint32_t           height;
    uint32_t           depth;
} VkTraceRaysIndirectCommand2KHR;
```

- `raygenShaderRecordAddress` is a VkDeviceAddress of the ray generation shader binding table record used by this command.
- `raygenShaderRecordSize` is a VkDeviceSize number of bytes corresponding to the ray generation shader binding table record at base address `raygenShaderRecordAddress`.
- `missShaderBindingTableAddress` is a VkDeviceAddress of the first record in the miss shader binding table used by this command.
- `missShaderBindingTableSize` is a VkDeviceSize number of bytes corresponding to the total size of the miss shader binding table at `missShaderBindingTableAddress` that may be accessed by this

command.

- `missShaderBindingTableStride` is a VkDeviceSize number of bytes between records of the miss shader binding table.
- `hitShaderBindingTableAddress` is a VkDeviceAddress of the first record in the hit shader binding table used by this command.
- `hitShaderBindingTableSize` is a VkDeviceSize number of bytes corresponding to the total size of the hit shader binding table at `hitShaderBindingTableAddress` that may be accessed by this command.
- `hitShaderBindingTableStride` is a VkDeviceSize number of bytes between records of the hit shader binding table.
- `callableShaderBindingTableAddress` is a VkDeviceAddress of the first record in the callable shader binding table used by this command.
- `callableShaderBindingTableSize` is a VkDeviceSize number of bytes corresponding to the total size of the callable shader binding table at `callableShaderBindingTableAddress` that may be accessed by this command.
- `callableShaderBindingTableStride` is a VkDeviceSize number of bytes between records of the callable shader binding table.
- `width` is the width of the ray trace query dimensions.
- `height` is height of the ray trace query dimensions.
- `depth` is depth of the ray trace query dimensions.

The members of `VkTraceRaysIndirectCommand2KHR` have the same meaning as the similarly named parameters of vkCmdTraceRaysKHR.

Indirect shader binding table buffer parameters **must** satisfy the same memory alignment and binding requirements as their counterparts in vkCmdTraceRaysIndirectKHR and vkCmdTraceRaysKHR.

Valid Usage

- VUID-VkTraceRaysIndirectCommand2KHR-pRayGenShaderBindingTable-03680
  If the buffer from which `raygenShaderRecordAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkTraceRaysIndirectCommand2KHR-pRayGenShaderBindingTable-03681
  The buffer from which the `raygenShaderRecordAddress` is queried **must** have been created with the `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` usage flag
- VUID-VkTraceRaysIndirectCommand2KHR-pRayGenShaderBindingTable-03682
  `raygenShaderRecordAddress` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupBaseAlignment`
- VUID-VkTraceRaysIndirectCommand2KHR-pMissShaderBindingTable-03683
  If the buffer from which `missShaderBindingTableAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object

- VUID-VkTraceRaysIndirectCommand2KHR-pMissShaderBindingTable-03684
  The buffer from which the `missShaderBindingTableAddress` is queried **must** have been created with the `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` usage flag
- VUID-VkTraceRaysIndirectCommand2KHR-pMissShaderBindingTable-03685
  `missShaderBindingTableAddress` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupBaseAlignment`
- VUID-VkTraceRaysIndirectCommand2KHR-stride-03686
  `missShaderBindingTableStride` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupHandleAlignment`
- VUID-VkTraceRaysIndirectCommand2KHR-stride-04029
  `missShaderBindingTableStride` **must** be less than or equal to `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`maxShaderGroupStride`
- VUID-VkTraceRaysIndirectCommand2KHR-pHitShaderBindingTable-03687
  If the buffer from which `hitShaderBindingTableAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkTraceRaysIndirectCommand2KHR-pHitShaderBindingTable-03688
  The buffer from which the `hitShaderBindingTableAddress` is queried **must** have been created with the `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` usage flag
- VUID-VkTraceRaysIndirectCommand2KHR-pHitShaderBindingTable-03689
  `hitShaderBindingTableAddress` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupBaseAlignment`
- VUID-VkTraceRaysIndirectCommand2KHR-stride-03690
  `hitShaderBindingTableStride` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupHandleAlignment`
- VUID-VkTraceRaysIndirectCommand2KHR-stride-04035
  `hitShaderBindingTableStride` **must** be less than or equal to `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`maxShaderGroupStride`
- VUID-VkTraceRaysIndirectCommand2KHR-pCallableShaderBindingTable-03691
  If the buffer from which `callableShaderBindingTableAddress` was queried is non-sparse then it **must** be bound completely and contiguously to a single `VkDeviceMemory` object
- VUID-VkTraceRaysIndirectCommand2KHR-pCallableShaderBindingTable-03692
  The buffer from which the `callableShaderBindingTableAddress` is queried **must** have been created with the `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` usage flag
- VUID-VkTraceRaysIndirectCommand2KHR-pCallableShaderBindingTable-03693
  `callableShaderBindingTableAddress` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupBaseAlignment`
- VUID-VkTraceRaysIndirectCommand2KHR-stride-03694
  `callableShaderBindingTableStride` **must** be a multiple of `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`shaderGroupHandleAlignment`
- VUID-VkTraceRaysIndirectCommand2KHR-stride-04041
  `callableShaderBindingTableStride` **must** be less than or equal to `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`maxShaderGroupStride`

- VUID-VkTraceRaysIndirectCommand2KHR-flags-03696
  If the bound ray tracing pipeline was created with `flags` that included `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR`, `hitShaderBindingTableAddress` **must** not be zero
- VUID-VkTraceRaysIndirectCommand2KHR-flags-03697
  If the bound ray tracing pipeline was created with `flags` that included `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR`, `hitShaderBindingTableAddress` **must** not be zero
- VUID-VkTraceRaysIndirectCommand2KHR-flags-03511
  If the bound ray tracing pipeline was created with `flags` that included `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR`, the shader group handle identified by `missShaderBindingTableAddress` **must** not be zero
- VUID-VkTraceRaysIndirectCommand2KHR-flags-03512
  If the bound ray tracing pipeline was created with `flags` that included `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR`, entries in the table identified by `hitShaderBindingTableAddress` accessed as a result of this command in order to execute an any-hit shader **must** not be zero
- VUID-VkTraceRaysIndirectCommand2KHR-flags-03513
  If the bound ray tracing pipeline was created with `flags` that included `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR`, entries in the table identified by `hitShaderBindingTableAddress` accessed as a result of this command in order to execute a closest hit shader **must** not be zero
- VUID-VkTraceRaysIndirectCommand2KHR-flags-03514
  If the bound ray tracing pipeline was created with `flags` that included `VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR`, entries in the table identified by `hitShaderBindingTableAddress` accessed as a result of this command in order to execute an intersection shader **must** not be zero
- VUID-VkTraceRaysIndirectCommand2KHR-pHitShaderBindingTable-04735
  Any non-zero hit shader group entries in the table identified by `hitShaderBindingTableAddress` accessed by this call from a geometry with a `geometryType` of `VK_GEOMETRY_TYPE_TRIANGLES_KHR` **must** have been created with `VK_RAY_TRACING_SHADER_GROUP_TYPE_TRIANGLES_HIT_GROUP_KHR`
- VUID-VkTraceRaysIndirectCommand2KHR-pHitShaderBindingTable-04736
  Any non-zero hit shader group entries in the table identified by `hitShaderBindingTableAddress` accessed by this call from a geometry with a `geometryType` of `VK_GEOMETRY_TYPE_AABBS_KHR` **must** have been created with `VK_RAY_TRACING_SHADER_GROUP_TYPE_PROCEDURAL_HIT_GROUP_KHR`
- VUID-VkTraceRaysIndirectCommand2KHR-width-03638
  `width` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[0] × `VkPhysicalDeviceLimits`::`maxComputeWorkGroupSize`[0]
- VUID-VkTraceRaysIndirectCommand2KHR-height-03639
  `height` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[1] × `VkPhysicalDeviceLimits`::`maxComputeWorkGroupSize`[1]

- VUID-VkTraceRaysIndirectCommand2KHR-depth-03640
  `depth` **must** be less than or equal to `VkPhysicalDeviceLimits`::`maxComputeWorkGroupCount`[2] × `VkPhysicalDeviceLimits`::`maxComputeWorkGroupSize`[2]
- VUID-VkTraceRaysIndirectCommand2KHR-width-03641
  `width` × `height` × `depth` **must** be less than or equal to `VkPhysicalDeviceRayTracingPipelinePropertiesKHR`::`maxRayDispatchInvocationCount`

37.3. Shader Binding Table

A *shader binding table* is a resource which establishes the relationship between the ray tracing pipeline and the acceleration structures that were built for the ray tracing pipeline. It indicates the shaders that operate on each geometry in an acceleration structure. In addition, it contains the resources accessed by each shader, including indices of textures, buffer device addresses, and constants. The application allocates and manages *shader binding tables* as VkBuffer objects.

Each entry in the shader binding table consists of `shaderGroupHandleSize` bytes of data, either as queried by vkGetRayTracingShaderGroupHandlesKHR to refer to those specified shaders, or all zeros to refer to a zero shader group. A zero shader group behaves as though it is a shader group consisting entirely of `VK_SHADER_UNUSED_KHR`. The remainder of the data specified by the stride is application-visible data that can be referenced by a `ShaderRecordBufferKHR` block in the shader.

The shader binding tables to use in a ray tracing pipeline are passed to the vkCmdTraceRaysKHR, or vkCmdTraceRaysIndirectKHR commands. Shader binding tables are read-only in shaders that are executing on the ray tracing pipeline.

Shader variables identified with the `ShaderRecordBufferKHR` storage class are used to access the provided shader binding table. Such variables **must** be:

- typed as `OpTypeStruct`, or an array of this type,
- identified with a `Block` decoration, and
- laid out explicitly using the `Offset`, `ArrayStride`, and `MatrixStride` decorations as specified in Offset and Stride Assignment.

The `Offset` decoration for any member of a `Block`-decorated variable in the `ShaderRecordBufferKHR` storage class **must** not cause the space required for that variable to extend outside the range [0, `maxStorageBufferRange`).

Accesses to the shader binding table from ray tracing pipelines **must** be synchronized with the `VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR` pipeline stage and an access type of `VK_ACCESS_SHADER_READ_BIT`.

NOTE

Because different shader record buffers can be associated with the same shader, a shader variable with `ShaderRecordBufferKHR` storage class will not be dynamically uniform if different invocations of the same shader can reference different data in the shader record buffer, such as if the same shader occurs twice in the shader binding table with a different shader record buffer. In this case, indexing resources based on values in the `ShaderRecordBufferKHR` storage class, the index should be

decorated as NonUniform.

37.3.1. Indexing Rules

In order to execute the correct shaders and access the correct resources during a ray tracing dispatch, the implementation **must** be able to locate shader binding table entries at various stages of execution. This is accomplished by defining a set of indexing rules that compute shader binding table record positions relative to the buffer's base address in memory. The application **must** organize the contents of the shader binding table's memory in a way that application of the indexing rules will lead to correct records.

Ray Generation Shaders

Only one ray generation shader is executed per ray tracing dispatch.

For vkCmdTraceRaysKHR, the location of the ray generation shader is specified by the pRaygenShaderBindingTable->deviceAddress parameter — there is no indexing. All data accessed **must** be less than pRaygenShaderBindingTable->size bytes from deviceAddress. pRaygenShaderBindingTable->stride is unused, and **must** be equal to pRaygenShaderBindingTable->size.

Hit Shaders

The base for the computation of intersection, any-hit, and closest hit shader locations is the instanceShaderBindingTableRecordOffset value stored with each instance of a top-level acceleration structure (VkAccelerationStructureInstanceKHR). This value determines the beginning of the shader binding table records for a given instance.

In the following rule, geometryIndex refers to the geometry index of the intersected geometry within the instance.

The sbtRecordOffset and sbtRecordStride values are passed in as parameters to traceRayEXT() calls made in the shaders. See Section 8.19 (Ray Tracing Functions) of the OpenGL Shading Language Specification for more details. In SPIR-V, these correspond to the SBTOffset and SBTStride parameters to the pipeline trace ray instructions.

The result of this computation is then added to pHitShaderBindingTable->deviceAddress, a device address passed to vkCmdTraceRaysKHR .

For vkCmdTraceRaysKHR, the complete rule to compute a hit shader binding table record address in the pHitShaderBindingTable is:

pHitShaderBindingTable->deviceAddress + pHitShaderBindingTable->stride × (instanceShaderBindingTableRecordOffset + geometryIndex × sbtRecordStride + sbtRecordOffset)

All data accessed **must** be less than pHitShaderBindingTable->size bytes from the base address.

Miss Shaders

A miss shader is executed whenever a ray query fails to find an intersection for the given scene geometry. Multiple miss shaders **may** be executed throughout a ray tracing dispatch.

The base for the computation of miss shader locations is `pMissShaderBindingTable->deviceAddress`, a device address passed into vkCmdTraceRaysKHR .

The `missIndex` value is passed in as a parameter to `traceRayEXT`() calls made in the shaders. See Section 8.19 (Ray Tracing Functions) of the OpenGL Shading Language Specification for more details. In SPIR-V, this corresponds to the `MissIndex` parameter to the pipeline trace ray instructions.

For vkCmdTraceRaysKHR, the complete rule to compute a miss shader binding table record address in the `pMissShaderBindingTable` is:

```
pMissShaderBindingTable->deviceAddress + pMissShaderBindingTable->stride × missIndex
```

All data accessed **must** be less than `pMissShaderBindingTable->size` bytes from the base address.

Callable Shaders

A callable shader is executed when requested by a ray tracing shader. Multiple callable shaders **may** be executed throughout a ray tracing dispatch.

The base for the computation of callable shader locations is `pCallableShaderBindingTable->deviceAddress`, a device address passed into vkCmdTraceRaysKHR .

The `sbtRecordIndex` value is passed in as a parameter to `executeCallableEXT`() calls made in the shaders. See Section 8.19 (Ray Tracing Functions) of the OpenGL Shading Language Specification for more details. In SPIR-V, this corresponds to the `SBTIndex` parameter to the `OpExecuteCallableKHR` instruction.

For vkCmdTraceRaysKHR, the complete rule to compute a callable shader binding table record address in the `pCallableShaderBindingTable` is:

```
pCallableShaderBindingTable->deviceAddress + pCallableShaderBindingTable->stride × sbtRecordIndex
```

All data accessed **must** be less than `pCallableShaderBindingTable->size` bytes from the base address.

37.4. Ray Tracing Pipeline Stack

Ray tracing pipelines have a potentially large set of shaders which **may** be invoked in various call chain combinations to perform ray tracing. To store parameters for a given shader execution, an implementation **may** use a stack of data in memory. This stack **must** be sized to the sum of the stack sizes of all shaders in any call chain executed by the application.

If the stack size is not set explicitly, the stack size for a pipeline is:

rayGenStackMax + min(1, `maxPipelineRayRecursionDepth`) × max(closestHitStackMax, missStackMax, intersectionStackMax + anyHitStackMax) + max(0, `maxPipelineRayRecursionDepth` -1) × max(closestHitStackMax, missStackMax) + 2 × callableStackMax

where rayGenStackMax, closestHitStackMax, missStackMax, anyHitStackMax, intersectionStackMax, and callableStackMax are the maximum stack values queried by the respective shader stages for any shaders in any shader groups defined by the pipeline.

This stack size is potentially significant, so an application **may** want to provide a more accurate stack size after pipeline compilation. The value that the application provides is the maximum value of the sum of all shaders in a call chain across all possible call chains, taking into account any application specific knowledge about the properties of the call chains.

NOTE

For example, if an application has two types of closest hit and miss shaders that it can use but the first level of rays will only use the first kind (possibly reflection) and the second level will only use the second kind (occlusion or shadow ray, for example) then the application can compute the stack size by something similar to:

`rayGenStack` + max(`closestHit1Stack`, `miss1Stack`) + max(`closestHit2Stack`, `miss2Stack`)

This is guaranteed to be no larger than the default stack size computation which assumes that both call levels may be the larger of the two.

37.5. Ray Tracing Capture Replay

In a similar way to bufferDeviceAddressCaptureReplay, the `rayTracingPipelineShaderGroupHandleCaptureReplay` feature allows the querying of opaque data which **can** be used in a future replay.

During the capture phase, capture/replay tools are expected to query opaque data for shader group handle replay using vkGetRayTracingCaptureReplayShaderGroupHandlesKHR.

Providing the opaque data during replay, using VkRayTracingShaderGroupCreateInfoKHR ::`pShaderGroupCaptureReplayHandle` at pipeline creation time, causes the implementation to generate identical shader group handles to those in the capture phase, allowing capture/replay tools to reuse previously recorded shader binding table buffer contents or to obtain the same handles by calling vkGetRayTracingCaptureReplayShaderGroupHandlesKHR again.

Chapter 38. Video Coding

Vulkan implementations **may** expose one or more queue families supporting video coding operations. These operations are performed by recording them into a command buffer within a video coding scope, and submitting them to queues with compatible video coding capabilities.

The Vulkan video functionalities are designed to be made available through a set of APIs built on top of each other, consisting of:

- A core API providing common video coding functionalities,
- APIs providing codec-independent video decode and video encode related functionalities, respectively,
- Additional codec-specific APIs built on top of those.

This chapter details the fundamental components and operations of these.

38.1. Video Picture Resources

In the context of video coding, multidimensional arrays of image data that **can** be used as the source or target of video coding operations are referred to as *video picture resources*. They **may** store additional metadata that includes implementation-private information used during the execution of video coding operations, as discussed later.

Video picture resources are backed by VkImage objects. Individual subregions of VkImageView objects created from such resources **can** be used as decode output pictures, encode input pictures, reconstructed pictures, and/or reference pictures.

The parameters of a video picture resource are specified using a `VkVideoPictureResourceInfoKHR` structure.

The `VkVideoPictureResourceInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoPictureResourceInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkOffset2D         codedOffset;
    VkExtent2D         codedExtent;
    uint32_t           baseArrayLayer;
    VkImageView        imageViewBinding;
} VkVideoPictureResourceInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `codedOffset` is the offset in texels of the image subregion to use.
- `codedExtent` is the size in pixels of the coded image data.

- `baseArrayLayer` is the array layer of the image view specified in `imageViewBinding` to use as the video picture resource.
- `imageViewBinding` is an image view representing the video picture resource.

The image subresource referred to by such a structure is defined as the image array layer index specified in `baseArrayLayer` relative to the image subresource range the image view specified in `imageViewBinding` was created with.

The meaning of the `codedOffset` and `codedExtent` depends on the command and context the video picture resource is used in, as well as on the used video profile and corresponding codec-specific semantics, as described later.

A video picture resource is uniquely defined by the image subresource referred to by an instance of this structure, together with the `codedOffset` and `codedExtent` members that identify the image subregion within the image subresource referenced corresponding to the video picture resource according to the particular codec-specific semantics.

Accesses to image data within a video picture resource happen at the granularity indicated by VkVideoCapabilitiesKHR::`pictureAccessGranularity`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile. As a result, given an effective image subregion corresponding to a video picture resource, the actual image subregion accessed **may** be larger than that as it **may** include additional padding texels due to the picture access granularity. Any writes performed by video coding operations to such padding texels will result in undefined texel values.

Two video picture resources match if they refer to the same image subresource and they specify identical `codedOffset` and `codedExtent` values.

Valid Usage

- VUID-VkVideoPictureResourceInfoKHR-baseArrayLayer-07175
  `baseArrayLayer` **must** be less than the VkImageViewCreateInfo ::`subresourceRange.layerCount` specified when the image view `imageViewBinding` was created

Valid Usage (Implicit)

- VUID-VkVideoPictureResourceInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_PICTURE_RESOURCE_INFO_KHR`
- VUID-VkVideoPictureResourceInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkVideoPictureResourceInfoKHR-imageViewBinding-parameter
  `imageViewBinding` **must** be a valid VkImageView handle

38.2. Decoded Picture Buffer

An integral part of video coding pipelines is the reconstruction of pictures from a compressed video bitstream. A *reconstructed picture* is a video picture resource resulting from this process.

Such reconstructed pictures **can** be used as *reference pictures* in subsequent video coding operations to provide predictions of the values of samples of subsequently decoded or encoded pictures. The correct use of such reconstructed pictures as reference pictures is driven by the video compression standard, the implementation, and the application-specific use cases.

The list of reference pictures used to provide such predictions within a single video coding operation is referred to as the list of *active reference pictures*.

The *decoded picture buffer (DPB)* is an indexed data structure that maintains the set of reference pictures available to be used in video coding operations. Individual indexed entries of the DPB are referred to as the *decoded picture buffer (DPB) slots*. The range of valid DPB slot indices is between zero and `N-1`, where `N` is the capacity of the DPB. Each DPB slot **can** refer to a reference picture containing a *video frame* or **can** refer to up to two reference pictures containing the top and/or bottom *fields* that, when both present, together represent a full *video frame* .

In Vulkan, the state and the backing store of the DPB is separated as follows:

- The state of individual DPB slots is maintained by video session objects.
- The backing store of DPB slots is provided by subregions of VkImage objects used as video picture resources.

In addition, the implementation **may** also maintain opaque metadata associated with DPB slots, including:

- *Reference picture metadata* corresponding to the video picture resource associated with the DPB slot.

Such metadata **may** be stored by the implementation as part of the DPB slot state maintained by the video session, or as part of the video picture resource backing the DPB slot.

Any metadata stored in the video picture resources backing DPB slots are independent of the video session used to store it, hence such video picture resources **can** be shared with other video sessions. Correspondingly, any metadata that is dependent on the video session will always be stored as part of the DPB slot state maintained by that video session.

The responsibility of managing the DPB is split between the application and the implementation as follows:

- The application maintains the association between DPB slot indices and corresponding video picture resources.
- The implementation maintains global and per-slot opaque reference picture metadata.

In addition, the application is also responsible for managing the mapping between the codec-specific picture IDs and DPB slots, and any other codec-specific states unless otherwise specified.

38.2.1. DPB Slot States

At a given time, each DPB slot is either in *active* or *inactive* state. Initially, all DPB slots managed by a video session are in *inactive* state.

A DPB slot **can** be *activated* by using it as the target of picture reconstruction in a video coding operation with the reconstructed picture requested to be set up as a reference picture, according to the codec-specific semantics, changing its state to *active* and associating it with a *picture reference* to the reconstructed pictures.

Some video coding standards allow multiple picture references to be associated with a single DPB slot. In this case the state of the individual picture references **can** be independently updated.

> **NOTE** As an example, H.264 decoding allows associating a separate top field and bottom field picture with the same DPB slot.

As part of reference picture setup, the implementation **may** also generate reference picture metadata. Such reference picture metadata is specific to each picture reference associated with the DPB slot.

If such a video coding operation completes successfully, the activated DPB slot will have a *valid picture reference* and the reconstructed picture is associated with the DPB slot. This is true even if the DPB slot is used as the target of a picture reconstruction that only sets up a top field or bottom field reference picture and thus does not yet refer to a complete frame. However, if any data provided as input to such a video coding operation is not compliant with the video compression standard used, that video coding operation **may** complete unsuccessfully, in which case the activated DPB slot will have an *invalid picture reference*. This is true even if the DPB slot previously had a valid picture reference to a top field or bottom field reference picture, but the reconstruction of the other field corresponding to the DPB slot failed.

The application **can** use queries to get feedback about the outcome of video coding operations and use the resulting VkQueryResultStatusKHR value to determine whether the video coding operation completed successfully (result status is positive) or unsuccessfully (result status is negative).

Using a reference picture associated with a DPB slot that has an *invalid picture reference* as an active reference picture in subsequent video coding operations is legal, however, the contents of the outputs of such operations are undefined, and any DPB slots activated by such video coding operations will also have an *invalid picture reference*. This is true even if such video coding operations **may** otherwise complete successfully.

A DPB slot **can** also be *deactivated* by the application, changing its state to *inactive* and invalidating any picture references and reference picture metadata associated with the DPB slot.

If an already active DPB slot is used as the target of picture reconstruction in a video coding operation, but the decoded picture is not requested to be set up as a reference picture, according to the codec-specific semantics, no reference picture setup happens and the corresponding picture reference and reference picture metadata is invalidated within the DPB slot. If the DPB slot no longer has any associated picture references after such an operation, the DPB slot is implicitly deactivated.

If an already active DPB slot is used as the target of picture reconstruction when decoding a field picture that is not marked as reference, then the behavior is as follows:

- If the DPB slot is currently associated with a frame, then the DPB slot is deactivated.
- If the DPB slot is not currently associated with a top field picture and the decoded picture is a top field picture, or if the DPB slot is not currently associated with a bottom field picture and the decoded picture is a bottom field picture, then the other field picture association of the DPB slot, if any, is not disturbed.
- If the DPB slot is currently associated with a top field picture and the decoded picture is a top field picture, or if the DPB slot is currently associated with a bottom field picture and the decoded picture is a bottom field picture, then that picture association is invalidated, without disturbing the other field picture association, if any. If the DPB slot no longer has any associated picture references after such an operation, the DPB slot is implicitly deactivated.

A DPB slot **can** be activated with a new frame even if it is already active. In this case all previous associations of the DPB slots with reference pictures are replaced with an association with the reconstructed picture used to activate it.

If an already active DPB slot is activated with a reconstructed field picture, then the behavior is as follows:

- If the DPB slot is currently associated with a frame, then that association is replaced with an association with the reconstructed field picture used to activate it.
- If the DPB slot is not currently associated with a top field picture and the DPB slot is activated with a top field picture, or if the DPB slot is not currently associated with a bottom field picture and the DPB slot is activated with a bottom field picture, then the DPB slot is associated with the reconstructed field picture used to activate it, without disturbing the other field picture association, if any.
- If the DPB slot is currently associated with a top field picture and the DPB slot is activated with a new top field picture, or if the DPB slot is currently associated with a bottom field picture and the DPB slot is activated with a new bottom field picture, then that association is replaced with an association with the reconstructed field picture used to activate it, without disturbing the other field picture association, if any.

38.3. Video Profiles

The `VkVideoProfileInfoKHR` structure is defined as follows:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoProfileInfoKHR {
    VkStructureType                      sType;
    const void*                          pNext;
    VkVideoCodecOperationFlagBitsKHR     videoCodecOperation;
    VkVideoChromaSubsamplingFlagsKHR     chromaSubsampling;
    VkVideoComponentBitDepthFlagsKHR     lumaBitDepth;
    VkVideoComponentBitDepthFlagsKHR     chromaBitDepth;
} VkVideoProfileInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `videoCodecOperation` is a VkVideoCodecOperationFlagBitsKHR value specifying a video codec operation.
- `chromaSubsampling` is a bitmask of VkVideoChromaSubsamplingFlagBitsKHR specifying video chroma subsampling information.
- `lumaBitDepth` is a bitmask of VkVideoComponentBitDepthFlagBitsKHR specifying video luma bit depth information.
- `chromaBitDepth` is a bitmask of VkVideoComponentBitDepthFlagBitsKHR specifying video chroma bit depth information.

Video profiles are provided as input to video capability queries such as vkGetPhysicalDeviceVideoCapabilitiesKHR or vkGetPhysicalDeviceVideoFormatPropertiesKHR, as well as when creating resources to be used by video coding operations such as images, buffers, query pools, and video sessions.

The full description of a video profile is specified by an instance of this structure, and the codec-specific and auxiliary structures provided in its `pNext` chain.

When this structure is specified as an input parameter to vkGetPhysicalDeviceVideoCapabilitiesKHR, or through the `pProfiles` member of a VkVideoProfileListInfoKHR structure in the `pNext` chain of the input parameter of a query command such as vkGetPhysicalDeviceVideoFormatPropertiesKHR or vkGetPhysicalDeviceImageFormatProperties2KHR, the following error codes indicate specific causes of the failure of the query operation:

- `VK_ERROR_VIDEO_PICTURE_LAYOUT_NOT_SUPPORTED_KHR` specifies that the requested video picture layout (e.g. through the `pictureLayout` member of a VkVideoDecodeH264ProfileInfoKHR structure included in the `pNext` chain of `VkVideoProfileInfoKHR`) is not supported.
- `VK_ERROR_VIDEO_PROFILE_OPERATION_NOT_SUPPORTED_KHR` specifies that a video profile operation specified by `videoCodecOperation` is not supported.
- `VK_ERROR_VIDEO_PROFILE_FORMAT_NOT_SUPPORTED_KHR` specifies that video format parameters specified by `chromaSubsampling`, `lumaBitDepth`, or `chromaBitDepth` are not supported.
- `VK_ERROR_VIDEO_PROFILE_CODEC_NOT_SUPPORTED_KHR` specifies that the codec-specific parameters corresponding to the video codec operation are not supported.

Valid Usage

- VUID-VkVideoProfileInfoKHR-chromaSubsampling-07013
  `chromaSubsampling` **must** have a single bit set
- VUID-VkVideoProfileInfoKHR-lumaBitDepth-07014
  `lumaBitDepth` **must** have a single bit set
- VUID-VkVideoProfileInfoKHR-chromaSubsampling-07015
  If `chromaSubsampling` is not `VK_VIDEO_CHROMA_SUBSAMPLING_MONOCHROME_BIT_KHR`, then `chromaBitDepth` **must** have a single bit set
- VUID-VkVideoProfileInfoKHR-videoCodecOperation-07179
  If `videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, then the `pNext` chain **must** include a VkVideoDecodeH264ProfileInfoKHR structure
- VUID-VkVideoProfileInfoKHR-videoCodecOperation-07180
  If `videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR`, then the `pNext` chain **must** include a VkVideoDecodeH265ProfileInfoKHR structure
- VUID-VkVideoProfileInfoKHR-videoCodecOperation-09256
  If `videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR`, then the `pNext` chain **must** include a VkVideoDecodeAV1ProfileInfoKHR structure
- VUID-VkVideoProfileInfoKHR-videoCodecOperation-07181
  If `videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR`, then the `pNext` chain **must** include a VkVideoEncodeH264ProfileInfoKHR structure
- VUID-VkVideoProfileInfoKHR-videoCodecOperation-07182
  If `videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR`, then the `pNext` chain **must** include a VkVideoEncodeH265ProfileInfoKHR structure
- VUID-VkVideoProfileInfoKHR-videoCodecOperation-10262
  If `videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR`, then the `pNext` chain **must** include a VkVideoEncodeAV1ProfileInfoKHR structure

Valid Usage (Implicit)

- VUID-VkVideoProfileInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_PROFILE_INFO_KHR`
- VUID-VkVideoProfileInfoKHR-videoCodecOperation-parameter
  `videoCodecOperation` **must** be a valid VkVideoCodecOperationFlagBitsKHR value
- VUID-VkVideoProfileInfoKHR-chromaSubsampling-parameter
  `chromaSubsampling` **must** be a valid combination of VkVideoChromaSubsamplingFlagBitsKHR values
- VUID-VkVideoProfileInfoKHR-chromaSubsampling-requiredbitmask
  `chromaSubsampling` **must** not be `0`
- VUID-VkVideoProfileInfoKHR-lumaBitDepth-parameter
  `lumaBitDepth` **must** be a valid combination of VkVideoComponentBitDepthFlagBitsKHR

values

- VUID-VkVideoProfileInfoKHR-lumaBitDepth-requiredbitmask
  `lumaBitDepth` **must** not be `0`
- VUID-VkVideoProfileInfoKHR-chromaBitDepth-parameter
  `chromaBitDepth` **must** be a valid combination of VkVideoComponentBitDepthFlagBitsKHR values

Possible values of VkVideoProfileInfoKHR::`videoCodecOperation`, specifying the type of video coding operation and video compression standard used by a video profile, are:

```
// Provided by VK_KHR_video_queue
typedef enum VkVideoCodecOperationFlagBitsKHR {
    VK_VIDEO_CODEC_OPERATION_NONE_KHR = 0,
  // Provided by VK_KHR_video_encode_h264
    VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR = 0x00010000,
  // Provided by VK_KHR_video_encode_h265
    VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR = 0x00020000,
  // Provided by VK_KHR_video_decode_h264
    VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR = 0x00000001,
  // Provided by VK_KHR_video_decode_h265
    VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR = 0x00000002,
  // Provided by VK_KHR_video_decode_av1
    VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR = 0x00000004,
  // Provided by VK_KHR_video_encode_av1
    VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR = 0x00040000,
} VkVideoCodecOperationFlagBitsKHR;
```

- `VK_VIDEO_CODEC_OPERATION_NONE_KHR` specifies that no video codec operations are supported.
- `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` specifies support for H.264 decode operations.
- `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR` specifies support for H.265 decode operations.
- `VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR` specifies support for AV1 decode operations.
- `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR` specifies support for H.264 encode operations.
- `VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR` specifies support for H.265 encode operations.
- `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR` specifies support for AV1 encode operations.

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoCodecOperationFlagsKHR;
```

`VkVideoCodecOperationFlagsKHR` is a bitmask type for setting a mask of zero or more VkVideoCodecOperationFlagBitsKHR.

The video format chroma subsampling is defined with the following enums:

```
// Provided by VK_KHR_video_queue
typedef enum VkVideoChromaSubsamplingFlagBitsKHR {
    VK_VIDEO_CHROMA_SUBSAMPLING_INVALID_KHR = 0,
    VK_VIDEO_CHROMA_SUBSAMPLING_MONOCHROME_BIT_KHR = 0x00000001,
    VK_VIDEO_CHROMA_SUBSAMPLING_420_BIT_KHR = 0x00000002,
    VK_VIDEO_CHROMA_SUBSAMPLING_422_BIT_KHR = 0x00000004,
    VK_VIDEO_CHROMA_SUBSAMPLING_444_BIT_KHR = 0x00000008,
} VkVideoChromaSubsamplingFlagBitsKHR;
```

- VK_VIDEO_CHROMA_SUBSAMPLING_MONOCHROME_BIT_KHR specifies that the format is monochrome.
- VK_VIDEO_CHROMA_SUBSAMPLING_420_BIT_KHR specified that the format is 4:2:0 chroma subsampled, i.e. the two chroma components are sampled horizontally and vertically at half the sample rate of the luma component.
- VK_VIDEO_CHROMA_SUBSAMPLING_422_BIT_KHR - the format is 4:2:2 chroma subsampled, i.e. the two chroma components are sampled horizontally at half the sample rate of luma component.
- VK_VIDEO_CHROMA_SUBSAMPLING_444_BIT_KHR - the format is 4:4:4 chroma sampled, i.e. all three components of the $Y'C_BC_R$ format are sampled at the same rate, thus there is no chroma subsampling.

Chroma subsampling is described in more detail in the Chroma Reconstruction section.

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoChromaSubsamplingFlagsKHR;
```

VkVideoChromaSubsamplingFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoChromaSubsamplingFlagBitsKHR.

Possible values for the video format component bit depth are:

```
// Provided by VK_KHR_video_queue
typedef enum VkVideoComponentBitDepthFlagBitsKHR {
    VK_VIDEO_COMPONENT_BIT_DEPTH_INVALID_KHR = 0,
    VK_VIDEO_COMPONENT_BIT_DEPTH_8_BIT_KHR = 0x00000001,
    VK_VIDEO_COMPONENT_BIT_DEPTH_10_BIT_KHR = 0x00000004,
    VK_VIDEO_COMPONENT_BIT_DEPTH_12_BIT_KHR = 0x00000010,
} VkVideoComponentBitDepthFlagBitsKHR;
```

- VK_VIDEO_COMPONENT_BIT_DEPTH_8_BIT_KHR specifies a component bit depth of 8 bits.
- VK_VIDEO_COMPONENT_BIT_DEPTH_10_BIT_KHR specifies a component bit depth of 10 bits.
- VK_VIDEO_COMPONENT_BIT_DEPTH_12_BIT_KHR specifies a component bit depth of 12 bits.

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoComponentBitDepthFlagsKHR;
```

VkVideoComponentBitDepthFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoComponentBitDepthFlagBitsKHR.

Additional information about the video decode use case **can** be provided by adding a VkVideoDecodeUsageInfoKHR structure to the pNext chain of VkVideoProfileInfoKHR.

The VkVideoDecodeUsageInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_queue
typedef struct VkVideoDecodeUsageInfoKHR {
    VkStructureType                sType;
    const void*                    pNext;
    VkVideoDecodeUsageFlagsKHR     videoUsageHints;
} VkVideoDecodeUsageInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- videoUsageHints is a bitmask of VkVideoDecodeUsageFlagBitsKHR specifying hints about the intended use of the video decode profile.

Valid Usage (Implicit)

- VUID-VkVideoDecodeUsageInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_DECODE_USAGE_INFO_KHR
- VUID-VkVideoDecodeUsageInfoKHR-videoUsageHints-parameter
  videoUsageHints **must** be a valid combination of VkVideoDecodeUsageFlagBitsKHR values

The following bits **can** be specified in VkVideoDecodeUsageInfoKHR::videoUsageHints as a hint about the video decode use case:

```
// Provided by VK_KHR_video_decode_queue
typedef enum VkVideoDecodeUsageFlagBitsKHR {
    VK_VIDEO_DECODE_USAGE_DEFAULT_KHR = 0,
    VK_VIDEO_DECODE_USAGE_TRANSCODING_BIT_KHR = 0x00000001,
    VK_VIDEO_DECODE_USAGE_OFFLINE_BIT_KHR = 0x00000002,
    VK_VIDEO_DECODE_USAGE_STREAMING_BIT_KHR = 0x00000004,
} VkVideoDecodeUsageFlagBitsKHR;
```

- VK_VIDEO_DECODE_USAGE_TRANSCODING_BIT_KHR specifies that video decoding is intended to be used in conjunction with video encoding to transcode a video bitstream with the same and/or different codecs.
- VK_VIDEO_DECODE_USAGE_OFFLINE_BIT_KHR specifies that video decoding is intended to be used to consume a local video bitstream.
- VK_VIDEO_DECODE_USAGE_STREAMING_BIT_KHR specifies that video decoding is intended to be used to

consume a video bitstream received as a continuous flow over network.

NOTE	There are no restrictions on the combination of bits that **can** be specified by the application. However, applications **should** use reasonable combinations in order for the implementation to be able to select the most appropriate mode of operation for the particular use case.

```
// Provided by VK_KHR_video_decode_queue
typedef VkFlags VkVideoDecodeUsageFlagsKHR;
```

VkVideoDecodeUsageFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoDecodeUsageFlagBitsKHR.

Additional information about the video encode use case **can** be provided by adding a VkVideoEncodeUsageInfoKHR structure to the pNext chain of VkVideoProfileInfoKHR.

The VkVideoEncodeUsageInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeUsageInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkVideoEncodeUsageFlagsKHR      videoUsageHints;
    VkVideoEncodeContentFlagsKHR    videoContentHints;
    VkVideoEncodeTuningModeKHR      tuningMode;
} VkVideoEncodeUsageInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- videoUsageHints is a bitmask of VkVideoEncodeUsageFlagBitsKHR specifying hints about the intended use of the video encode profile.
- videoContentHints is a bitmask of VkVideoEncodeContentFlagBitsKHR specifying hints about the content to be encoded using the video encode profile.
- tuningMode is a VkVideoEncodeTuningModeKHR value specifying the tuning mode to use when encoding with the video profile.

Valid Usage (Implicit)

- VUID-VkVideoEncodeUsageInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_USAGE_INFO_KHR
- VUID-VkVideoEncodeUsageInfoKHR-videoUsageHints-parameter
  videoUsageHints **must** be a valid combination of VkVideoEncodeUsageFlagBitsKHR values
- VUID-VkVideoEncodeUsageInfoKHR-videoContentHints-parameter
  videoContentHints **must** be a valid combination of VkVideoEncodeContentFlagBitsKHR

values

- VUID-VkVideoEncodeUsageInfoKHR-tuningMode-parameter
  If `tuningMode` is not `0`, `tuningMode` **must** be a valid VkVideoEncodeTuningModeKHR value

The following bits **can** be specified in VkVideoEncodeUsageInfoKHR::`videoUsageHints` as a hint about the video encode use case:

```
// Provided by VK_KHR_video_encode_queue
typedef enum VkVideoEncodeUsageFlagBitsKHR {
    VK_VIDEO_ENCODE_USAGE_DEFAULT_KHR = 0,
    VK_VIDEO_ENCODE_USAGE_TRANSCODING_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_USAGE_STREAMING_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_USAGE_RECORDING_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_USAGE_CONFERENCING_BIT_KHR = 0x00000008,
} VkVideoEncodeUsageFlagBitsKHR;
```

- `VK_VIDEO_ENCODE_USAGE_TRANSCODING_BIT_KHR` specifies that video encoding is intended to be used in conjunction with video decoding to transcode a video bitstream with the same and/or different codecs.
- `VK_VIDEO_ENCODE_USAGE_STREAMING_BIT_KHR` specifies that video encoding is intended to be used to produce a video bitstream that is expected to be sent as a continuous flow over network.
- `VK_VIDEO_ENCODE_USAGE_RECORDING_BIT_KHR` specifies that video encoding is intended to be used for real-time recording for offline consumption.
- `VK_VIDEO_ENCODE_USAGE_CONFERENCING_BIT_KHR` specifies that video encoding is intended to be used in a video conferencing scenario.

NOTE

There are no restrictions on the combination of bits that **can** be specified by the application. However, applications **should** use reasonable combinations in order for the implementation to be able to select the most appropriate mode of operation for the particular use case.

```
// Provided by VK_KHR_video_encode_queue
typedef VkFlags VkVideoEncodeUsageFlagsKHR;
```

`VkVideoEncodeUsageFlagsKHR` is a bitmask type for setting a mask of zero or more VkVideoEncodeUsageFlagBitsKHR.

The following bits **can** be specified in VkVideoEncodeUsageInfoKHR::`videoContentHints` as a hint about the encoded video content:

```
// Provided by VK_KHR_video_encode_queue
typedef enum VkVideoEncodeContentFlagBitsKHR {
    VK_VIDEO_ENCODE_CONTENT_DEFAULT_KHR = 0,
    VK_VIDEO_ENCODE_CONTENT_CAMERA_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_CONTENT_DESKTOP_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_CONTENT_RENDERED_BIT_KHR = 0x00000004,
} VkVideoEncodeContentFlagBitsKHR;
```

- VK_VIDEO_ENCODE_CONTENT_CAMERA_BIT_KHR specifies that video encoding is intended to be used to encode camera content.
- VK_VIDEO_ENCODE_CONTENT_DESKTOP_BIT_KHR specifies that video encoding is intended to be used to encode desktop content.
- VK_VIDEO_ENCODE_CONTENT_RENDERED_BIT_KHR specified that video encoding is intended to be used to encode rendered (e.g. game) content.

NOTE

There are no restrictions on the combination of bits that **can** be specified by the application. However, applications **should** use reasonable combinations in order for the implementation to be able to select the most appropriate mode of operation for the particular content type.

```
// Provided by VK_KHR_video_encode_queue
typedef VkFlags VkVideoEncodeContentFlagsKHR;
```

VkVideoEncodeContentFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeContentFlagBitsKHR.

Possible video encode tuning mode values are as follows:

```
// Provided by VK_KHR_video_encode_queue
typedef enum VkVideoEncodeTuningModeKHR {
    VK_VIDEO_ENCODE_TUNING_MODE_DEFAULT_KHR = 0,
    VK_VIDEO_ENCODE_TUNING_MODE_HIGH_QUALITY_KHR = 1,
    VK_VIDEO_ENCODE_TUNING_MODE_LOW_LATENCY_KHR = 2,
    VK_VIDEO_ENCODE_TUNING_MODE_ULTRA_LOW_LATENCY_KHR = 3,
    VK_VIDEO_ENCODE_TUNING_MODE_LOSSLESS_KHR = 4,
} VkVideoEncodeTuningModeKHR;
```

- VK_VIDEO_ENCODE_TUNING_MODE_DEFAULT_KHR specifies the default tuning mode.
- VK_VIDEO_ENCODE_TUNING_MODE_HIGH_QUALITY_KHR specifies that video encoding is tuned for high quality. When using this tuning mode, the implementation **may** compromise the latency of video encoding operations to improve quality.
- VK_VIDEO_ENCODE_TUNING_MODE_LOW_LATENCY_KHR specifies that video encoding is tuned for low latency. When using this tuning mode, the implementation **may** compromise quality to increase the performance and lower the latency of video encode operations.

- VK_VIDEO_ENCODE_TUNING_MODE_ULTRA_LOW_LATENCY_KHR specifies that video encoding is tuned for ultra-low latency. When using this tuning mode, the implementation **may** compromise quality to maximize the performance and minimize the latency of video encoding operations.
- VK_VIDEO_ENCODE_TUNING_MODE_LOSSLESS_KHR specifies that video encoding is tuned for lossless encoding. When using this tuning mode, video encode operations produce lossless output.

The VkVideoProfileListInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoProfileListInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    uint32_t                        profileCount;
    const VkVideoProfileInfoKHR*    pProfiles;
} VkVideoProfileListInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- profileCount is the number of elements in the pProfiles array.
- pProfiles is a pointer to an array of VkVideoProfileInfoKHR structures.

NOTE	Video transcoding is an example of a use case that necessitates the specification of multiple profiles in various contexts.

When the application provides a video decode profile and one or more video encode profiles in the profile list, the implementation ensures that any capabilitities returned or resources created are suitable for the video transcoding use cases without the need for manual data transformations.

Valid Usage

- VUID-VkVideoProfileListInfoKHR-pProfiles-06813
  pProfiles **must** not contain more than one element whose videoCodecOperation member specifies a decode operation

Valid Usage (Implicit)

- VUID-VkVideoProfileListInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_PROFILE_LIST_INFO_KHR
- VUID-VkVideoProfileListInfoKHR-pProfiles-parameter
  If profileCount is not 0, pProfiles **must** be a valid pointer to an array of profileCount valid VkVideoProfileInfoKHR structures

38.4. Video Capabilities

38.4.1. Video Coding Capabilities

To query video coding capabilities for a specific video profile, call:

```
// Provided by VK_KHR_video_queue
VkResult vkGetPhysicalDeviceVideoCapabilitiesKHR(
    VkPhysicalDevice                            physicalDevice,
    const VkVideoProfileInfoKHR*                pVideoProfile,
    VkVideoCapabilitiesKHR*                     pCapabilities);
```

- `physicalDevice` is the physical device from which to query the video decode or encode capabilities.
- `pVideoProfile` is a pointer to a VkVideoProfileInfoKHR structure.
- `pCapabilities` is a pointer to a VkVideoCapabilitiesKHR structure in which the capabilities are returned.

If the video profile described by `pVideoProfile` is supported by the implementation, then this command returns `VK_SUCCESS` and `pCapabilities` is filled with the capabilities supported with the specified video profile. Otherwise, one of the video-profile-specific error codes are returned.

Valid Usage

- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-07183
  If `pVideoProfile->videoCodecOperation` specifies a decode operation, then the `pNext` chain of `pCapabilities` **must** include a VkVideoDecodeCapabilitiesKHR structure
- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-07184
  If `pVideoProfile->videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, then the `pNext` chain of `pCapabilities` **must** include a VkVideoDecodeH264CapabilitiesKHR structure
- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-07185
  If `pVideoProfile->videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR`, then the `pNext` chain of `pCapabilities` **must** include a VkVideoDecodeH265CapabilitiesKHR structure
- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-09257
  If `pVideoProfile->videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR`, then the `pNext` chain of `pCapabilities` **must** include a VkVideoDecodeAV1CapabilitiesKHR structure
- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-07186
  If `pVideoProfile->videoCodecOperation` specifies an encode operation, then the `pNext` chain of `pCapabilities` **must** include a VkVideoEncodeCapabilitiesKHR structure
- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-07187
  If `pVideoProfile->videoCodecOperation` is `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR`,

then the pNext chain of pCapabilities **must** include a VkVideoEncodeH264CapabilitiesKHR structure

- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-07188
  If pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the pNext chain of pCapabilities **must** include a VkVideoEncodeH265CapabilitiesKHR structure
- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-10263
  If pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the pNext chain of pCapabilities **must** include a VkVideoEncodeAV1CapabilitiesKHR structure

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pVideoProfile-parameter
  pVideoProfile **must** be a valid pointer to a valid VkVideoProfileInfoKHR structure
- VUID-vkGetPhysicalDeviceVideoCapabilitiesKHR-pCapabilities-parameter
  pCapabilities **must** be a valid pointer to a VkVideoCapabilitiesKHR structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_VIDEO_PROFILE_OPERATION_NOT_SUPPORTED_KHR
- VK_ERROR_VIDEO_PROFILE_FORMAT_NOT_SUPPORTED_KHR
- VK_ERROR_VIDEO_PICTURE_LAYOUT_NOT_SUPPORTED_KHR
- VK_ERROR_VIDEO_PROFILE_CODEC_NOT_SUPPORTED_KHR

The VkVideoCapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoCapabilitiesKHR {
    VkStructureType              sType;
    void*                        pNext;
    VkVideoCapabilityFlagsKHR    flags;
    VkDeviceSize                 minBitstreamBufferOffsetAlignment;
    VkDeviceSize                 minBitstreamBufferSizeAlignment;
    VkExtent2D                   pictureAccessGranularity;
    VkExtent2D                   minCodedExtent;
    VkExtent2D                   maxCodedExtent;
    uint32_t                     maxDpbSlots;
    uint32_t                     maxActiveReferencePictures;
    VkExtensionProperties        stdHeaderVersion;
} VkVideoCapabilitiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkVideoCapabilityFlagBitsKHR specifying capability flags.
- `minBitstreamBufferOffsetAlignment` is the minimum alignment for bitstream buffer offsets.
- `minBitstreamBufferSizeAlignment` is the minimum alignment for bitstream buffer range sizes.
- `pictureAccessGranularity` is the granularity at which image access to video picture resources happen.
- `minCodedExtent` is the minimum width and height of the coded frames.
- `maxCodedExtent` is the maximum width and height of the coded frames.
- `maxDpbSlots` is the maximum number of DPB slots supported by a single video session.
- `maxActiveReferencePictures` is the maximum number of active reference pictures a single video coding operation **can** use.
- `stdHeaderVersion` is a VkExtensionProperties structure reporting the Video Std header name and version supported for the video profile.

NOTE It is common for video compression standards to allow using all reference pictures associated with active DPB slots as active reference pictures, hence for video decode profiles the values returned in `maxDpbSlots` and `maxActiveReferencePictures` are often equal. Similarly, in case of video decode profiles supporting field pictures the value of `maxActiveReferencePictures` often equals `maxDpbSlots` × 2.

Valid Usage (Implicit)

- VUID-VkVideoCapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_CAPABILITIES_KHR`
- VUID-VkVideoCapabilitiesKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkVideoDecodeAV1CapabilitiesKHR,

VkVideoDecodeCapabilitiesKHR, VkVideoDecodeH264CapabilitiesKHR, VkVideoDecodeH265CapabilitiesKHR, VkVideoEncodeAV1CapabilitiesKHR, VkVideoEncodeAV1QuantizationMapCapabilitiesKHR, VkVideoEncodeCapabilitiesKHR, VkVideoEncodeH264CapabilitiesKHR, VkVideoEncodeH264QuantizationMapCapabilitiesKHR, VkVideoEncodeH265CapabilitiesKHR, VkVideoEncodeH265QuantizationMapCapabilitiesKHR, or VkVideoEncodeQuantizationMapCapabilitiesKHR

- VUID-VkVideoCapabilitiesKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

Bits which **can** be set in VkVideoCapabilitiesKHR::`flags` are:

```
// Provided by VK_KHR_video_queue
typedef enum VkVideoCapabilityFlagBitsKHR {
    VK_VIDEO_CAPABILITY_PROTECTED_CONTENT_BIT_KHR = 0x00000001,
    VK_VIDEO_CAPABILITY_SEPARATE_REFERENCE_IMAGES_BIT_KHR = 0x00000002,
} VkVideoCapabilityFlagBitsKHR;
```

- `VK_VIDEO_CAPABILITY_PROTECTED_CONTENT_BIT_KHR` specifies that video sessions support producing and consuming protected content.
- `VK_VIDEO_CAPABILITY_SEPARATE_REFERENCE_IMAGES_BIT_KHR` indicates that the video picture resources associated with the DPB slots of a video session **can** be backed by separate `VkImage` objects. If this capability flag is not present, then all DPB slots of a video session **must** be associated with video picture resources backed by the same `VkImage` object (e.g. using different layers of the same image).

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoCapabilityFlagsKHR;
```

`VkVideoCapabilityFlagsKHR` is a bitmask type for setting a mask of zero or more VkVideoCapabilityFlagBitsKHR.

38.4.2. Video Format Capabilities

To enumerate the supported video formats and corresponding capabilities for a specific video profile, call:

```
// Provided by VK_KHR_video_queue
VkResult vkGetPhysicalDeviceVideoFormatPropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceVideoFormatInfoKHR*   pVideoFormatInfo,
    uint32_t*                                   pVideoFormatPropertyCount,
    VkVideoFormatPropertiesKHR*                 pVideoFormatProperties);
```

- physicalDevice is the physical device from which to query the video format properties.
- pVideoFormatInfo is a pointer to a VkPhysicalDeviceVideoFormatInfoKHR structure specifying the usage and video profiles for which supported image formats and capabilities are returned.
- pVideoFormatPropertyCount is a pointer to an integer related to the number of video format properties available or queried, as described below.
- pVideoFormatProperties is a pointer to an array of VkVideoFormatPropertiesKHR structures in which supported image formats and capabilities are returned.

If pVideoFormatProperties is NULL, then the number of video format properties supported for the given physicalDevice is returned in pVideoFormatPropertyCount. Otherwise, pVideoFormatPropertyCount **must** point to a variable set by the application to the number of elements in the pVideoFormatProperties array, and on return the variable is overwritten with the number of values actually written to pVideoFormatProperties. If the value of pVideoFormatPropertyCount is less than the number of video format properties supported, at most pVideoFormatPropertyCount values will be written to pVideoFormatProperties, and VK_INCOMPLETE will be returned instead of VK_SUCCESS, to indicate that not all the available values were returned.

Video format properties are always queried with respect to a specific set of video profiles. These are specified by chaining the VkVideoProfileListInfoKHR structure to pVideoFormatInfo.

For most use cases, the images are used by a single video session and a single video profile is provided. For a use case such as video transcoding, where a decode session output image **can** be used as encode input in one or more encode sessions, multiple video profiles corresponding to the video sessions that will share the image **must** be provided.

If any of the video profiles specified via VkVideoProfileListInfoKHR::pProfiles are not supported, then this command returns one of the video-profile-specific error codes. Furthermore, if VkPhysicalDeviceVideoFormatInfoKHR::imageUsage includes any image usage flags not supported by the specified video profiles, then this command returns VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR.

This command also returns VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR if VkPhysicalDeviceVideoFormatInfoKHR::imageUsage does not include the appropriate flags as dictated by the decode capability flags returned in VkVideoDecodeCapabilitiesKHR::flags for any of the profiles specified in the VkVideoProfileListInfoKHR structure provided in the pNext chain of pVideoFormatInfo.

If the decode capability flags include VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR but not VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR, then in order to query video format properties for decode DPB and output usage, VkPhysicalDeviceVideoFormatInfoKHR::imageUsage **must** include both VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR and VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR. Otherwise, the call will fail with VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR.

If the decode capability flags include VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR but not VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR, then in order to query video format properties for decode DPB usage, VkPhysicalDeviceVideoFormatInfoKHR::imageUsage **must** include VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR, but not VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR. Otherwise, the call will fail with VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR. Similarly, to query video

format properties for decode output usage, VkPhysicalDeviceVideoFormatInfoKHR::imageUsage **must** include VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR, but not VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR. Otherwise, the call will fail with VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR.

The imageUsage member of the VkPhysicalDeviceVideoFormatInfoKHR structure specifies the expected video usage flags that the returned video formats **must** support. Correspondingly, the imageUsageFlags member of each VkVideoFormatPropertiesKHR structure returned will contain at least the same set of image usage flags.

If the implementation supports using images of a particular format in operations other than video decode/encode then the imageUsageFlags member of the corresponding VkVideoFormatPropertiesKHR structure returned will include additional image usage flags indicating that.

NOTE

For most use cases, only decode or encode related usage flags are going to be specified. For a use case such as transcode, if the image were to be shared between decode and encode session(s), then both decode and encode related usage flags **can** be set.

Multiple VkVideoFormatPropertiesKHR entries **may** be returned with the same format member with different componentMapping, imageType, or imageTiling values, as described later.

If VkPhysicalDeviceVideoFormatInfoKHR::imageUsageFlags includes VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR, multiple VkVideoFormatPropertiesKHR entries **may** be returned with the same format, componentMapping, imageType, and imageTiling member values, but different quantizationMapTexelSize returned in the VkVideoFormatQuantizationMapPropertiesKHR structure, if one is included in the VkVideoFormatPropertiesKHR::pNext chain, when the queried quantization map type supports multiple distinct quantization map texel sizes.

In addition, a different set of VkVideoFormatPropertiesKHR entries **may** be returned depending on the imageUsage member of the VkPhysicalDeviceVideoFormatInfoKHR structure, even for the same set of video profiles, for example, based on whether encode input, encode DPB, decode output, and/or decode DPB usage is requested.

The application **can** select the parameters returned in the VkVideoFormatPropertiesKHR entries and use compatible parameters when creating the input, output, and DPB images. The implementation will report all image creation and usage flags that are valid for images used with the requested video profiles but applications **should** create images only with those that are necessary for the particular use case.

Before creating an image, the application **can** obtain the complete set of supported image format features by calling vkGetPhysicalDeviceImageFormatProperties2KHR using parameters derived from the members of one of the reported VkVideoFormatPropertiesKHR entries and adding the same VkVideoProfileListInfoKHR structure to the pNext chain of VkPhysicalDeviceImageFormatInfo2.

The following applies to all VkVideoFormatPropertiesKHR entries returned by

vkGetPhysicalDeviceVideoFormatPropertiesKHR:

- vkGetPhysicalDeviceFormatProperties2KHR **must** succeed when called with VkVideoFormatPropertiesKHR::format
- If VkVideoFormatPropertiesKHR::imageTiling is VK_IMAGE_TILING_OPTIMAL, then the optimalTilingFeatures returned by vkGetPhysicalDeviceFormatProperties2KHR **must** include all format features required by the image usage flags reported in VkVideoFormatPropertiesKHR::imageUsageFlags for the format, as indicated in the Format Feature Dependent Usage Flags section.
- If VkVideoFormatPropertiesKHR::imageTiling is VK_IMAGE_TILING_LINEAR, then the linearTilingFeatures returned by vkGetPhysicalDeviceFormatProperties2KHR **must** include all format features required by the image usage flags reported in VkVideoFormatPropertiesKHR::imageUsageFlags for the format, as indicated in the Format Feature Dependent Usage Flags section.
- vkGetPhysicalDeviceImageFormatProperties2KHR **must** succeed when called with a VkPhysicalDeviceImageFormatInfo2 structure containing the following information:
  - The pNext chain including the same VkVideoProfileListInfoKHR structure used to call vkGetPhysicalDeviceVideoFormatPropertiesKHR.
  - format set to the value of VkVideoFormatPropertiesKHR::format.
  - type set to the value of VkVideoFormatPropertiesKHR::imageType.
  - tiling set to the value of VkVideoFormatPropertiesKHR::imageTiling.
  - usage set to the value of VkVideoFormatPropertiesKHR::imageUsageFlags.
  - flags set to the value of VkVideoFormatPropertiesKHR::imageCreateFlags.

The componentMapping member of VkVideoFormatPropertiesKHR defines the ordering of the Y'C_BC_R color channels from the perspective of the video codec operations specified in VkVideoProfileListInfoKHR. For example, if the implementation produces video decode output with the format VK_FORMAT_G8_B8R8_2PLANE_420_UNORM where the blue and red chrominance channels are swapped then the componentMapping member of the corresponding VkVideoFormatPropertiesKHR structure will have the following member values:

```
components.r = VK_COMPONENT_SWIZZLE_B;        // Cb component
components.g = VK_COMPONENT_SWIZZLE_IDENTITY; // Y component
components.b = VK_COMPONENT_SWIZZLE_R;        // Cr component
components.a = VK_COMPONENT_SWIZZLE_IDENTITY; // unused, defaults to 1.0
```

Valid Usage

- VUID-vkGetPhysicalDeviceVideoFormatPropertiesKHR-pNext-06812
  The pNext chain of pVideoFormatInfo **must** include a VkVideoProfileListInfoKHR structure with profileCount greater than 0

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceVideoFormatPropertiesKHR-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceVideoFormatPropertiesKHR-pVideoFormatInfo-parameter
  `pVideoFormatInfo` **must** be a valid pointer to a valid VkPhysicalDeviceVideoFormatInfoKHR structure
- VUID-vkGetPhysicalDeviceVideoFormatPropertiesKHR-pVideoFormatPropertyCount-parameter
  `pVideoFormatPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkGetPhysicalDeviceVideoFormatPropertiesKHR-pVideoFormatProperties-parameter
  If the value referenced by `pVideoFormatPropertyCount` is not `0`, and `pVideoFormatProperties` is not `NULL`, `pVideoFormatProperties` **must** be a valid pointer to an array of `pVideoFormatPropertyCount` VkVideoFormatPropertiesKHR structures

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR`
- `VK_ERROR_VIDEO_PROFILE_OPERATION_NOT_SUPPORTED_KHR`
- `VK_ERROR_VIDEO_PROFILE_FORMAT_NOT_SUPPORTED_KHR`
- `VK_ERROR_VIDEO_PICTURE_LAYOUT_NOT_SUPPORTED_KHR`
- `VK_ERROR_VIDEO_PROFILE_CODEC_NOT_SUPPORTED_KHR`

The `VkPhysicalDeviceVideoFormatInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkPhysicalDeviceVideoFormatInfoKHR {
    VkStructureType      sType;
    const void*          pNext;
    VkImageUsageFlags    imageUsage;
} VkPhysicalDeviceVideoFormatInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `imageUsage` is a bitmask of VkImageUsageFlagBits specifying the intended usage of the video images.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVideoFormatInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_FORMAT_INFO_KHR`
- VUID-VkPhysicalDeviceVideoFormatInfoKHR-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkVideoProfileListInfoKHR
- VUID-VkPhysicalDeviceVideoFormatInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPhysicalDeviceVideoFormatInfoKHR-imageUsage-parameter
  `imageUsage` **must** be a valid combination of VkImageUsageFlagBits values
- VUID-VkPhysicalDeviceVideoFormatInfoKHR-imageUsage-requiredbitmask
  `imageUsage` **must** not be `0`

The `VkVideoFormatPropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoFormatPropertiesKHR {
    VkStructureType       sType;
    void*                 pNext;
    VkFormat              format;
    VkComponentMapping    componentMapping;
    VkImageCreateFlags    imageCreateFlags;
    VkImageType           imageType;
    VkImageTiling         imageTiling;
    VkImageUsageFlags     imageUsageFlags;
} VkVideoFormatPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `format` is a VkFormat that specifies the format that **can** be used with the specified video profiles and image usages.
- `componentMapping` defines the color channel order used for the format. `format` along with `componentMapping` describe how the color channels are ordered when producing video decoder output or are expected to be ordered in video encoder input, when applicable. If the `format` reported does not require component swizzling then all members of `componentMapping` will be set to `VK_COMPONENT_SWIZZLE_IDENTITY`.
- `imageCreateFlags` is a bitmask of VkImageCreateFlagBits specifying the supported image creation flags for the format.
- `imageType` is a VkImageType that specifies the image type the format **can** be used with.

- `imageTiling` is a VkImageTiling that specifies the image tiling the format **can** be used with.
- `imageUsageFlags` is a bitmask of VkImageUsageFlagBits specifying the supported image usage flags for the format.

Valid Usage (Implicit)

- VUID-VkVideoFormatPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_FORMAT_PROPERTIES_KHR`
- VUID-VkVideoFormatPropertiesKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkVideoFormatAV1QuantizationMapPropertiesKHR, VkVideoFormatH265QuantizationMapPropertiesKHR, or VkVideoFormatQuantizationMapPropertiesKHR
- VUID-VkVideoFormatPropertiesKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

The *list of supported video format properties* for a set of image usage flags with respect to a video profile is defined as the list of VkVideoFormatPropertiesKHR structures and any structures included in its `pNext` chain, obtained by calling vkGetPhysicalDeviceVideoFormatPropertiesKHR with VkPhysicalDeviceVideoFormatInfoKHR::`imageUsage` equal to the VkImageUsageFlags in question and the VkPhysicalDeviceVideoFormatInfoKHR::`pNext` chain including a VkVideoProfileListInfoKHR structure with its `pProfiles` member containing a single array element specifying the VkVideoProfileInfoKHR structure chain describing the video profile in question.

38.5. Video Sessions

Video sessions are objects that represent and maintain the state needed to perform video decode or encode operations using a specific video profile.

In case of video encode profiles this includes the current rate control configuration and the currently set video encode quality level.

Video sessions are represented by `VkVideoSessionKHR` handles:

```
// Provided by VK_KHR_video_queue
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkVideoSessionKHR)
```

38.5.1. Creating a Video Session

To create a video session object, call:

```
// Provided by VK_KHR_video_queue
VkResult vkCreateVideoSessionKHR(
    VkDevice                                    device,
    const VkVideoSessionCreateInfoKHR*          pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkVideoSessionKHR*                          pVideoSession);
```

- `device` is the logical device that creates the video session.
- `pCreateInfo` is a pointer to a VkVideoSessionCreateInfoKHR structure containing parameters to be used to create the video session.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pVideoSession` is a pointer to a VkVideoSessionKHR handle in which the resulting video session object is returned.

The resulting video session object is said to be created with the video codec operation specified in `pCreateInfo->pVideoProfile->videoCodecOperation`.

The name and version of the codec-specific Video Std header to be used with the video session is specified by the VkExtensionProperties structure pointed to by `pCreateInfo->pStdHeaderVersion`. If a non-existent or unsupported Video Std header version is specified in `pCreateInfo->pStdHeaderVersion->specVersion`, then this command returns `VK_ERROR_VIDEO_STD_VERSION_NOT_SUPPORTED_KHR`.

Video session objects are created in *uninitialized* state. In order to transition the video session into *initial* state, the application **must** issue a vkCmdControlVideoCodingKHR command with VkVideoCodingControlInfoKHR::`flags` including `VK_VIDEO_CODING_CONTROL_RESET_BIT_KHR`.

Video session objects also maintain the state of the DPB. The number of DPB slots usable with the created video session is specified in `pCreateInfo->maxDpbSlots`, and each slot is initially in the inactive state.

Each DPB slot maintained by the created video session **can** refer to a reference picture representing a video frame.

In addition, if the `videoCodecOperation` member of the VkVideoProfileInfoKHR structure pointed to by `pCreateInfo->pVideoProfile` is `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` and the `pictureLayout` member of the VkVideoDecodeH264ProfileInfoKHR structure provided in the VkVideoProfileInfoKHR::`pNext` chain is not `VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_PROGRESSIVE_KHR`, then the created video session supports *interlaced* frames and each DPB slot maintained by the created video session **can** instead refer to separate top field and bottom field reference pictures that together **can** represent a full video frame. In this case, it is up to the application, driven by the video content, whether it associates any individual DPB slot with separate top and/or bottom field pictures or a single picture representing a full frame.

The created video session **can** be used to perform video coding operations using video frames up to the maximum size specified in `pCreateInfo->maxCodedExtent`. The minimum frame size allowed is implicitly derived from VkVideoCapabilitiesKHR::`minCodedExtent`, as returned by

vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by pCreateInfo->pVideoProfile. Accordingly, the created video session is said to be created with a minCodedExtent equal to that.

In case of video session objects created with a video encode operation, implementations **may** return the VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR error if any of the specified Video Std parameters do not adhere to the syntactic or semantic requirements of the used video compression standard, or if values derived from parameters according to the rules defined by the used video compression standard do not adhere to the capabilities of the video compression standard or the implementation.

NOTE Applications **should** not rely on the VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR error being returned by any command as a means to verify Video Std parameters, as implementations are not required to report the error in any specific set of cases.

Valid Usage (Implicit)

- VUID-vkCreateVideoSessionKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateVideoSessionKHR-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkVideoSessionCreateInfoKHR structure
- VUID-vkCreateVideoSessionKHR-pAllocator-parameter
  If pAllocator is not NULL, pAllocator **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateVideoSessionKHR-pVideoSession-parameter
  pVideoSession **must** be a valid pointer to a VkVideoSessionKHR handle

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INITIALIZATION_FAILED
- VK_ERROR_VIDEO_STD_VERSION_NOT_SUPPORTED_KHR
- VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR

The VkVideoSessionCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoSessionCreateInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    uint32_t                        queueFamilyIndex;
    VkVideoSessionCreateFlagsKHR    flags;
    const VkVideoProfileInfoKHR*    pVideoProfile;
    VkFormat                        pictureFormat;
    VkExtent2D                      maxCodedExtent;
    VkFormat                        referencePictureFormat;
    uint32_t                        maxDpbSlots;
    uint32_t                        maxActiveReferencePictures;
    const VkExtensionProperties*    pStdHeaderVersion;
} VkVideoSessionCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `queueFamilyIndex` is the index of the queue family the created video session will be used with.
- `flags` is a bitmask of VkVideoSessionCreateFlagBitsKHR specifying creation flags.
- `pVideoProfile` is a pointer to a VkVideoProfileInfoKHR structure specifying the video profile the created video session will be used with.
- `pictureFormat` is the image format the created video session will be used with. If `pVideoProfile->videoCodecOperation` specifies a decode operation, then `pictureFormat` is the image format of decode output pictures usable with the created video session. If `pVideoProfile->videoCodecOperation` specifies an encode operation, then `pictureFormat` is the image format of encode input pictures usable with the created video session.
- `maxCodedExtent` is the maximum width and height of the coded frames the created video session will be used with.
- `referencePictureFormat` is the image format of reference pictures stored in the DPB the created video session will be used with.
- `maxDpbSlots` is the maximum number of DPB Slots that **can** be used with the created video session.
- `maxActiveReferencePictures` is the maximum number of active reference pictures that **can** be used in a single video coding operation using the created video session.
- `pStdHeaderVersion` is a pointer to a VkExtensionProperties structure requesting the Video Std header version to use for the `videoCodecOperation` specified in `pVideoProfile`.

Valid Usage

- VUID-VkVideoSessionCreateInfoKHR-protectedMemory-07189
  If the `protectedMemory` feature is not enabled or if VkVideoCapabilitiesKHR::`flags` does not include `VK_VIDEO_CAPABILITY_PROTECTED_CONTENT_BIT_KHR`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by

pVideoProfile, then flags **must** not include VK_VIDEO_SESSION_CREATE_PROTECTED_CONTENT_BIT_KHR

- VUID-VkVideoSessionCreateInfoKHR-flags-08371
  If flags includes VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, then videoMaintenance1 **must** be enabled
- VUID-VkVideoSessionCreateInfoKHR-flags-10264
  If flags includes VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR, then the videoEncodeQuantizationMap feature **must** be enabled
- VUID-VkVideoSessionCreateInfoKHR-flags-10265
  If flags includes VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR, then pVideoProfile->videoCodecOperation **must** specify an encode operation
- VUID-VkVideoSessionCreateInfoKHR-flags-10266
  If flags includes VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR, then it **must** not also include VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR
- VUID-VkVideoSessionCreateInfoKHR-flags-10267
  If VkVideoEncodeCapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by pVideoProfile, then flags **must** not include VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- VUID-VkVideoSessionCreateInfoKHR-flags-10268
  If VkVideoEncodeCapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by pVideoProfile, then flags **must** not include VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR
- VUID-VkVideoSessionCreateInfoKHR-pVideoProfile-04845
  pVideoProfile **must** be a supported video profile
- VUID-VkVideoSessionCreateInfoKHR-maxDpbSlots-04847
  maxDpbSlots **must** be less than or equal to VkVideoCapabilitiesKHR::maxDpbSlots, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-maxActiveReferencePictures-04849
  maxActiveReferencePictures **must** be less than or equal to VkVideoCapabilitiesKHR::maxActiveReferencePictures, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-maxDpbSlots-04850
  If either maxDpbSlots or maxActiveReferencePictures is 0, then both **must** be 0
- VUID-VkVideoSessionCreateInfoKHR-maxCodedExtent-04851
  maxCodedExtent **must** be between VkVideoCapabilitiesKHR::minCodedExtent and VkVideoCapabilitiesKHR::maxCodedExtent, inclusive, as returned by

vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by pVideoProfile

- VUID-VkVideoSessionCreateInfoKHR-referencePictureFormat-04852
  If pVideoProfile->videoCodecOperation specifies a decode operation and maxActiveReferencePictures is greater than 0, then referencePictureFormat **must** be one of the supported decode DPB formats, as returned by vkGetPhysicalDeviceVideoFormatPropertiesKHR in VkVideoFormatPropertiesKHR::format when called with the imageUsage member of its pVideoFormatInfo parameter containing VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR, and with a VkVideoProfileListInfoKHR structure specified in the pNext chain of its pVideoFormatInfo parameter whose pProfiles member contains an element matching pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-referencePictureFormat-06814
  If pVideoProfile->videoCodecOperation specifies an encode operation and maxActiveReferencePictures is greater than 0, then referencePictureFormat **must** be one of the supported decode DPB formats, as returned by then referencePictureFormat **must** be one of the supported encode DPB formats, as returned by vkGetPhysicalDeviceVideoFormatPropertiesKHR in VkVideoFormatPropertiesKHR::format when called with the imageUsage member of its pVideoFormatInfo parameter containing VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR, and with a VkVideoProfileListInfoKHR structure specified in the pNext chain of its pVideoFormatInfo parameter whose pProfiles member contains an element matching pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-pictureFormat-04853
  If pVideoProfile->videoCodecOperation specifies a decode operation, then pictureFormat **must** be one of the supported decode output formats, as returned by vkGetPhysicalDeviceVideoFormatPropertiesKHR in VkVideoFormatPropertiesKHR::format when called with the imageUsage member of its pVideoFormatInfo parameter containing VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR, and with a VkVideoProfileListInfoKHR structure specified in the pNext chain of its pVideoFormatInfo parameter whose pProfiles member contains an element matching pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-pictureFormat-04854
  If pVideoProfile->videoCodecOperation specifies an encode operation, then pictureFormat **must** be one of the supported encode input formats, as returned by vkGetPhysicalDeviceVideoFormatPropertiesKHR in VkVideoFormatPropertiesKHR::format when called with the imageUsage member of its pVideoFormatInfo parameter containing VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR, and with a VkVideoProfileListInfoKHR structure specified in the pNext chain of its pVideoFormatInfo parameter whose pProfiles member contains an element matching pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-pStdHeaderVersion-07190
  pStdHeaderVersion->extensionName **must** match VkVideoCapabilitiesKHR::stdHeaderVersion.extensionName, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-pStdHeaderVersion-07191
  pStdHeaderVersion->specVersion **must** be less than or equal to VkVideoCapabilitiesKHR::stdHeaderVersion.specVersion, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified by pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-pVideoProfile-08251

If pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and the pNext chain of this structure includes a VkVideoEncodeH264SessionCreateInfoKHR structure, then its maxLevelIdc member **must** be less than or equal to VkVideoEncodeH264CapabilitiesKHR::maxLevelIdc, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified in pVideoProfile

- VUID-VkVideoSessionCreateInfoKHR-pVideoProfile-08252
  If pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the pNext chain of this structure includes a VkVideoEncodeH265SessionCreateInfoKHR structure, then its maxLevelIdc member **must** be less than or equal to VkVideoEncodeH265CapabilitiesKHR::maxLevelIdc, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified in pVideoProfile
- VUID-VkVideoSessionCreateInfoKHR-pVideoProfile-10269
  If pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the videoEncodeAV1 feature **must** be enabled
- VUID-VkVideoSessionCreateInfoKHR-pVideoProfile-10270
  If pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pNext chain of this structure includes a VkVideoEncodeAV1SessionCreateInfoKHR structure, then its maxLevel member **must** be less than or equal to VkVideoEncodeAV1CapabilitiesKHR::maxLevel, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified in pVideoProfile

Valid Usage (Implicit)

- VUID-VkVideoSessionCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_SESSION_CREATE_INFO_KHR
- VUID-VkVideoSessionCreateInfoKHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkVideoEncodeAV1SessionCreateInfoKHR, VkVideoEncodeH264SessionCreateInfoKHR, or VkVideoEncodeH265SessionCreateInfoKHR
- VUID-VkVideoSessionCreateInfoKHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkVideoSessionCreateInfoKHR-flags-parameter
  flags **must** be a valid combination of VkVideoSessionCreateFlagBitsKHR values
- VUID-VkVideoSessionCreateInfoKHR-pVideoProfile-parameter
  pVideoProfile **must** be a valid pointer to a valid VkVideoProfileInfoKHR structure
- VUID-VkVideoSessionCreateInfoKHR-pictureFormat-parameter
  pictureFormat **must** be a valid VkFormat value
- VUID-VkVideoSessionCreateInfoKHR-referencePictureFormat-parameter
  referencePictureFormat **must** be a valid VkFormat value
- VUID-VkVideoSessionCreateInfoKHR-pStdHeaderVersion-parameter
  pStdHeaderVersion **must** be a valid pointer to a valid VkExtensionProperties structure

Bits which **can** be set in VkVideoSessionCreateInfoKHR::flags are:

```
// Provided by VK_KHR_video_queue
typedef enum VkVideoSessionCreateFlagBitsKHR {
    VK_VIDEO_SESSION_CREATE_PROTECTED_CONTENT_BIT_KHR = 0x00000001,
  // Provided by VK_KHR_video_encode_queue
    VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_PARAMETER_OPTIMIZATIONS_BIT_KHR = 0x00000002,
  // Provided by VK_KHR_video_maintenance1
    VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR = 0x00000004,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR = 0x00000008,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR = 0x00000010,
} VkVideoSessionCreateFlagBitsKHR;
```

- VK_VIDEO_SESSION_CREATE_PROTECTED_CONTENT_BIT_KHR specifies that the video session uses protected video content.
- VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_PARAMETER_OPTIMIZATIONS_BIT_KHR specifies that the implementation is allowed to override video session parameters and other codec-specific encoding parameters to optimize video encode operations based on the use case information specified in the video profile and the used video encode quality level.

  NOTE Not specifying VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_PARAMETER_OPTIMIZATIONS_BIT_KHR does not guarantee that the implementation will not do any codec-specific parameter overrides, as certain overrides are necessary for the correct operation of the video encoder implementation due to limitations to the available encoding tools on that implementation. This flag, however, enables the implementation to apply further optimizing overrides.

- VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR specifies that queries within video coding scopes using the created video session are executed inline with video coding operations.
- VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR specifies that the video session **can** be used to encode pictures with quantization delta maps.
- VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR specifies that the video session **can** be used to encode pictures with emphasis maps.

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoSessionCreateFlagsKHR;
```

VkVideoSessionCreateFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoSessionCreateFlagBitsKHR.

38.5.2. Destroying a Video Session

To destroy a video session, call:

```
// Provided by VK_KHR_video_queue
void vkDestroyVideoSessionKHR(
    VkDevice                                    device,
    VkVideoSessionKHR                           videoSession,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the video session.
- `videoSession` is the video session to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyVideoSessionKHR-videoSession-07192
  All submitted commands that refer to `videoSession` **must** have completed execution
- VUID-vkDestroyVideoSessionKHR-videoSession-07193
  If `VkAllocationCallbacks` were provided when `videoSession` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyVideoSessionKHR-videoSession-07194
  If no `VkAllocationCallbacks` were provided when `videoSession` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyVideoSessionKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkDestroyVideoSessionKHR-videoSession-parameter
  If `videoSession` is not VK_NULL_HANDLE, `videoSession` **must** be a valid VkVideoSessionKHR handle
- VUID-vkDestroyVideoSessionKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyVideoSessionKHR-videoSession-parent
  If `videoSession` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `videoSession` **must** be externally synchronized

38.5.3. Video Session Memory Association

After creating a video session object, and before the object **can** be used to record video coding operations into command buffers using it, the application **must** allocate and bind device memory to the video session. Device memory is allocated separately (see Device Memory) and then associated with the video session.

Video sessions **may** have multiple memory bindings identified by unique unsigned integer values. Appropriate device memory **must** be bound to each such memory binding before using the video session to record command buffer commands with it.

To determine the memory requirements for a video session object, call:

```
// Provided by VK_KHR_video_queue
VkResult vkGetVideoSessionMemoryRequirementsKHR(
    VkDevice                                    device,
    VkVideoSessionKHR                           videoSession,
    uint32_t*                                   pMemoryRequirementsCount,
    VkVideoSessionMemoryRequirementsKHR*        pMemoryRequirements);
```

- `device` is the logical device that owns the video session.
- `videoSession` is the video session to query.
- `pMemoryRequirementsCount` is a pointer to an integer related to the number of memory binding requirements available or queried, as described below.
- `pMemoryRequirements` is `NULL` or a pointer to an array of VkVideoSessionMemoryRequirementsKHR structures in which the memory binding requirements of the video session are returned.

If `pMemoryRequirements` is `NULL`, then the number of memory bindings required for the video session is returned in `pMemoryRequirementsCount`. Otherwise, `pMemoryRequirementsCount` **must** point to a variable set by the application to the number of elements in the `pMemoryRequirements` array, and on return the variable is overwritten with the number of memory binding requirements actually written to `pMemoryRequirements`. If `pMemoryRequirementsCount` is less than the number of memory bindings required for the video session, then at most `pMemoryRequirementsCount` elements will be written to `pMemoryRequirements`, and `VK_INCOMPLETE` will be returned, instead of `VK_SUCCESS`, to indicate that not all required memory binding requirements were returned.

Valid Usage (Implicit)

- VUID-vkGetVideoSessionMemoryRequirementsKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetVideoSessionMemoryRequirementsKHR-videoSession-parameter
  `videoSession` **must** be a valid VkVideoSessionKHR handle
- VUID-vkGetVideoSessionMemoryRequirementsKHR-pMemoryRequirementsCount-parameter
  `pMemoryRequirementsCount` **must** be a valid pointer to a `uint32_t` value

- VUID-vkGetVideoSessionMemoryRequirementsKHR-pMemoryRequirements-parameter
  If the value referenced by `pMemoryRequirementsCount` is not `0`, and `pMemoryRequirements` is not `NULL`, `pMemoryRequirements` **must** be a valid pointer to an array of `pMemoryRequirementsCount` VkVideoSessionMemoryRequirementsKHR structures
- VUID-vkGetVideoSessionMemoryRequirementsKHR-videoSession-parent
  `videoSession` **must** have been created, allocated, or retrieved from `device`

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

None

The `VkVideoSessionMemoryRequirementsKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoSessionMemoryRequirementsKHR {
    VkStructureType         sType;
    void*                   pNext;
    uint32_t                memoryBindIndex;
    VkMemoryRequirements    memoryRequirements;
} VkVideoSessionMemoryRequirementsKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memoryBindIndex` is the index of the memory binding.
- `memoryRequirements` is a VkMemoryRequirements structure in which the requested memory binding requirements for the binding index specified by `memoryBindIndex` are returned.

Valid Usage (Implicit)

- VUID-VkVideoSessionMemoryRequirementsKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_SESSION_MEMORY_REQUIREMENTS_KHR`
- VUID-VkVideoSessionMemoryRequirementsKHR-pNext-pNext
  `pNext` **must** be `NULL`

To attach memory to a video session object, call:

```
// Provided by VK_KHR_video_queue
VkResult vkBindVideoSessionMemoryKHR(
    VkDevice                                    device,
    VkVideoSessionKHR                           videoSession,
    uint32_t                                    bindSessionMemoryInfoCount,
    const VkBindVideoSessionMemoryInfoKHR*      pBindSessionMemoryInfos);
```

- `device` is the logical device that owns the video session.
- `videoSession` is the video session to be bound with device memory.
- `bindSessionMemoryInfoCount` is the number of elements in `pBindSessionMemoryInfos`.
- `pBindSessionMemoryInfos` is a pointer to an array of `bindSessionMemoryInfoCount` VkBindVideoSessionMemoryInfoKHR structures specifying memory regions to be bound to specific memory bindings of the video session.

The valid usage statements below refer to the VkMemoryRequirements structure corresponding to a specific element of `pBindSessionMemoryInfos`, which is defined as follows:

- If the `memoryBindIndex` member of the element of `pBindSessionMemoryInfos` in question matches the `memoryBindIndex` member of one of the elements returned in `pMemoryRequirements` when vkGetVideoSessionMemoryRequirementsKHR is called with the same `videoSession` and with `pMemoryRequirementsCount` equal to `bindSessionMemoryInfoCount`, then the `memoryRequirements` member of that element of `pMemoryRequirements` is the VkMemoryRequirements structure corresponding to the element of `pBindSessionMemoryInfos` in question.
- Otherwise the element of `pBindSessionMemoryInfos` in question is said to not have a corresponding VkMemoryRequirements structure.

Valid Usage

- VUID-vkBindVideoSessionMemoryKHR-videoSession-07195
  The memory binding of `videoSession` identified by the `memoryBindIndex` member of any element of `pBindSessionMemoryInfos` **must** not already be backed by a memory object
- VUID-vkBindVideoSessionMemoryKHR-memoryBindIndex-07196
  The `memoryBindIndex` member of each element of `pBindSessionMemoryInfos` **must** be unique within `pBindSessionMemoryInfos`
- VUID-vkBindVideoSessionMemoryKHR-pBindSessionMemoryInfos-07197
  Each element of `pBindSessionMemoryInfos` **must** have a corresponding VkMemoryRequirements structure
- VUID-vkBindVideoSessionMemoryKHR-pBindSessionMemoryInfos-07198
  If an element of `pBindSessionMemoryInfos` has a corresponding VkMemoryRequirements structure, then the `memory` member of that element of `pBindSessionMemoryInfos` **must** have been allocated using one of the memory types allowed in the `memoryTypeBits` member of the corresponding VkMemoryRequirements structure
- VUID-vkBindVideoSessionMemoryKHR-pBindSessionMemoryInfos-07199
  If an element of `pBindSessionMemoryInfos` has a corresponding VkMemoryRequirements

structure, then the `memoryOffset` member of that element of `pBindSessionMemoryInfos` **must** be an integer multiple of the `alignment` member of the corresponding VkMemoryRequirements structure

- VUID-vkBindVideoSessionMemoryKHR-pBindSessionMemoryInfos-07200
  If an element of `pBindSessionMemoryInfos` has a corresponding VkMemoryRequirements structure, then the `memorySize` member of that element of `pBindSessionMemoryInfos` **must** equal the `size` member of the corresponding VkMemoryRequirements structure

Valid Usage (Implicit)

- VUID-vkBindVideoSessionMemoryKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkBindVideoSessionMemoryKHR-videoSession-parameter
  `videoSession` **must** be a valid VkVideoSessionKHR handle
- VUID-vkBindVideoSessionMemoryKHR-pBindSessionMemoryInfos-parameter
  `pBindSessionMemoryInfos` **must** be a valid pointer to an array of `bindSessionMemoryInfoCount` valid VkBindVideoSessionMemoryInfoKHR structures
- VUID-vkBindVideoSessionMemoryKHR-bindSessionMemoryInfoCount-arraylength
  `bindSessionMemoryInfoCount` **must** be greater than `0`
- VUID-vkBindVideoSessionMemoryKHR-videoSession-parent
  `videoSession` **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `videoSession` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkBindVideoSessionMemoryInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkBindVideoSessionMemoryInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           memoryBindIndex;
    VkDeviceMemory     memory;
    VkDeviceSize       memoryOffset;
    VkDeviceSize       memorySize;
} VkBindVideoSessionMemoryInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `memoryBindIndex` is the memory binding index to bind memory to.
- `memory` is the allocated device memory to be bound to the video session's memory binding with index `memoryBindIndex`.
- `memoryOffset` is the start offset of the region of `memory` which is to be bound.
- `memorySize` is the size in bytes of the region of `memory`, starting from `memoryOffset` bytes, to be bound.

Valid Usage

- VUID-VkBindVideoSessionMemoryInfoKHR-memoryOffset-07201
  `memoryOffset` **must** be less than the size of `memory`
- VUID-VkBindVideoSessionMemoryInfoKHR-memorySize-07202
  `memorySize` **must** be less than or equal to the size of `memory` minus `memoryOffset`

Valid Usage (Implicit)

- VUID-VkBindVideoSessionMemoryInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_BIND_VIDEO_SESSION_MEMORY_INFO_KHR`
- VUID-VkBindVideoSessionMemoryInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkBindVideoSessionMemoryInfoKHR-memory-parameter
  `memory` **must** be a valid VkDeviceMemory handle

38.6. Video Profile Compatibility

Resources and query pools used with a particular video session **must** be compatible with the video profile the video session was created with.

A VkBuffer is compatible with a video profile if it was created with the VkBufferCreateInfo::`pNext` chain including a VkVideoProfileListInfoKHR structure with its `pProfiles` member containing an

element matching the VkVideoProfileInfoKHR structure chain describing the video profile, and VkBufferCreateInfo::usage including at least one bit specific to video coding usage.

- VK_BUFFER_USAGE_VIDEO_DECODE_SRC_BIT_KHR
- VK_BUFFER_USAGE_VIDEO_DECODE_DST_BIT_KHR
- VK_BUFFER_USAGE_VIDEO_ENCODE_SRC_BIT_KHR
- VK_BUFFER_USAGE_VIDEO_ENCODE_DST_BIT_KHR

A VkBuffer is also compatible with a video profile if it was created with VkBufferCreateInfo::flags including VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR.

A VkImage is compatible with a video profile if it was created with the VkImageCreateInfo::pNext chain including a VkVideoProfileListInfoKHR structure with its pProfiles member containing an element matching the VkVideoProfileInfoKHR structure chain describing the video profile, and VkImageCreateInfo::usage including at least one bit specific to video coding usage.

- VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR

A VkImage is also compatible with a video profile if all of the following conditions are true for the VkImageCreateInfo structure the image was created with:

- VkImageCreateInfo::flags included VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR.
- The list of VkVideoFormatPropertiesKHR structures, obtained by calling vkGetPhysicalDeviceVideoFormatPropertiesKHR with VkPhysicalDeviceVideoFormatInfoKHR::imageUsage equal to the VkImageCreateInfo::usage the image was created with and the VkPhysicalDeviceVideoFormatInfoKHR::pNext chain including a VkVideoProfileListInfoKHR structure with its pProfiles member containing a single array element specifying the VkVideoProfileInfoKHR structure chain describing the video profile in question, contains an element for which all of the following conditions are true with respect to the VkImageCreateInfo structure the image was created with:
  - VkImageCreateInfo::format equals VkVideoFormatPropertiesKHR::format.
  - VkImageCreateInfo::flags only contains VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR and/or bits also set in VkVideoFormatPropertiesKHR::imageCreateFlags.

> **NOTE** Specifying VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR when creating decode output pictures or encode input pictures is always supported when the videoMaintenance1 feature is enabled, regardless of the

supported VkImageCreateFlags reported in VkVideoFormatPropertiesKHR::imageCreateFlags. Accordingly, implementations **should** not report VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR in VkVideoFormatPropertiesKHR::imageCreateFlags for any video format.

- VkImageCreateInfo::imageType equals VkVideoFormatPropertiesKHR::imageType.
- VkImageCreateInfo::tiling equals VkVideoFormatPropertiesKHR::imageTiling.
- VkImageCreateInfo::usage only contains bits also set in VkVideoFormatPropertiesKHR::imageUsageFlags, or VkImageCreateInfo::flags includes VK_IMAGE_CREATE_EXTENDED_USAGE_BIT.

NOTE

While some of these rules allow creating buffer or image resources that **may** be compatible with any video profile, applications **should** still prefer to include the specific video profiles the buffer or image resource is expected to be used with (through a VkVideoProfileListInfoKHR structure included in the pNext chain of the corresponding create info structure) whenever the information about the complete set of video profiles is available at resource creation time, to enable the implementation to optimize the created resource for the specific use case. In the absence of that information, the implementation **may** have to make conservative decisions about the memory requirements or representation of the resource.

A VkImageView is compatible with a video profile if the VkImage it was created from is also compatible with that video profile.

A VkQueryPool is compatible with a video profile if it was created with the VkQueryPoolCreateInfo::pNext chain including a VkVideoProfileInfoKHR structure chain describing the same video profile, and VkQueryPoolCreateInfo::queryType having one of the following values:

- VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR
- VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR

38.7. Video Session Parameters

Video session parameters objects **can** store preprocessed codec-specific parameters used with a compatible video session, and enable reducing the number of parameters needed to be provided and processed by the implementation while recording video coding operations into command buffers.

Parameters stored in such objects are *immutable* to facilitate the concurrent use of the stored parameters in multiple threads. At the same time, new parameters **can** be added to existing objects using the vkUpdateVideoSessionParametersKHR command.

In order to support concurrent use of the stored immutable parameters while also allowing the video session parameters object to be extended with new parameters, each video session parameters object maintains an *update sequence counter* that is set to 0 at object creation time and **must** be incremented by each subsequent update operation.

Certain video sequences that adhere to particular video compression standards permit updating

previously supplied parameters. If a parameter update is necessary, the application has the following options:

- Cache the set of parameters on the application side and create a new video session parameters object adding all the parameters with appropriate changes, as necessary; or
- Create a new video session parameters object providing only the updated parameters and the previously used object as the template, which ensures that parameters not specified at creation time will be copied unmodified from the template object.

The actual types of parameters that **can** be stored and the capacity for individual parameter types, and the methods of initializing, updating, and referring to individual parameters are specific to the video codec operation the video session parameters object was created with.

- For `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` these are defined in the H.264 Decode Parameter Sets section.
- For `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR` these are defined in the H.265 Decode Parameter Sets section.
- For `VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR` these are defined in the AV1 Decode Parameter Sets section.
- For `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR` these are defined in the H.264 Encode Parameter Sets section.
- For `VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR` these are defined in the H.265 Encode Parameter Sets section.
- For `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR` these are defined in the AV1 Encode Parameter Sets section.

Video session parameters objects created with an encode operation are further specialized based on the video encode quality level the video session parameters are used with, as implementations **may** apply different sets of parameter overrides depending on the used quality level. This enables implementations to store the potentially optimized set of parameters in these objects, further limiting the necessary processing required while recording video encode operations into command buffers.

Video session parameters are represented by `VkVideoSessionParametersKHR` handles:

```
// Provided by VK_KHR_video_queue
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkVideoSessionParametersKHR)
```

38.7.1. Creating Video Session Parameters

To create a video session parameters object, call:

```
// Provided by VK_KHR_video_queue
VkResult vkCreateVideoSessionParametersKHR(
    VkDevice                                    device,
    const VkVideoSessionParametersCreateInfoKHR* pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkVideoSessionParametersKHR*                pVideoSessionParameters);
```

- `device` is the logical device that creates the video session parameters object.
- `pCreateInfo` is a pointer to VkVideoSessionParametersCreateInfoKHR structure containing parameters to be used to create the video session parameters object.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pVideoSessionParameters` is a pointer to a VkVideoSessionParametersKHR handle in which the resulting video session parameters object is returned.

The resulting video session parameters object is said to be created with the video codec operation `pCreateInfo->videoSession` was created with.

Video session parameters objects created with an encode operation are always created with respect to a video encode quality level. By default, the created video session parameters objects are created with quality level zero, unless otherwise specified by including a VkVideoEncodeQualityLevelInfoKHR structure in the `pCreateInfo->pNext` chain, in which case the video session parameters object is created with the quality level specified in VkVideoEncodeQualityLevelInfoKHR::`qualityLevel`.

If `pCreateInfo->videoSessionParametersTemplate` is not `VK_NULL_HANDLE`, then it will be used as a template for constructing the new video session parameters object. This happens by first adding any parameters according to the additional creation parameters provided in the `pCreateInfo->pNext` chain, followed by adding any parameters from the template object that have a key that does not match the key of any of the already added parameters.

For video session parameters objects created with an encode operation, the template object specified in `pCreateInfo->videoSessionParametersTemplate` **must** have been created with the same video encode quality level as the newly created object.

NOTE

This means that codec-specific parameters stored in video session parameters objects **can** only be reused across different video encode quality levels by re-specifying them, as previously created video session parameters against other quality levels **cannot** be used as template because the original codec-specific parameters (before the implementation **may** have applied parameter overrides) **may** no longer be available in them for the purposes of constructing the derived object.

Video session parameters objects are only compatible with quantization maps if they are created with `pCreateInfo->flags` including `VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR`.

Video session parameters objects created with

VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR against a video session object that was created with VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR are created with a specific compatible quantization map texel size specified in the quantizationMapTexelSize member of the VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR structure included in the pNext chain of pCreateInfo.

NOTE This means that the quantization map texel size that such a video session parameters object is compatible with is fixed for the lifetime of the object. Applications have to create separate video session parameters objects to use different quantization map texel sizes with a single video session object. This is necessary because the used quantization map texel size may affect the parameter overrides the implementation has to perform and thus the final values of the used codec-specific parameters.

For video session parameters objects created with VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, the template object specified in pCreateInfo->videoSessionParametersTemplate **must** also have been created with VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR and the same compatible quantization map texel size specified in VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR::quantizationMapTexelSize.

NOTE This means that codec-specific parameters stored in video session parameters objects **can** only be reused with different quantization map texel sizes by re-specifying them, as previously created video session parameters against other quantization map texel sizes **cannot** be used as template because the original codec-specific parameters (before the implementation **may** have applied parameter overrides) **may** no longer be available in them for the purposes of constructing the derived object.

For video session parameters objects created without VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, the template object specified in pCreateInfo->videoSessionParametersTemplate **must** also have been created without VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR.

If pCreateInfo->videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the created video session parameters object will initially contain the following sets of parameter entries:

- StdVideoH264SequenceParameterSet structures representing H.264 SPS entries, as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH264SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH264SequenceParameterSet entries specified in pParametersAddInfo->pStdSPSs are added first;
  - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH264SequenceParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same

seq_parameter_set_id.

- StdVideoH264PictureParameterSet structures representing H.264 PPS entries, as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH264SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH264PictureParameterSet entries specified in pParametersAddInfo->pStdPPSs are added first;
  - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH264PictureParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same seq_parameter_set_id and pic_parameter_set_id.

If pCreateInfo->videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the created video session parameters object will initially contain the following sets of parameter entries:

- StdVideoH265VideoParameterSet structures representing H.265 VPS entries, as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH265SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH265VideoParameterSet entries specified in pParametersAddInfo->pStdVPSs are added first;
  - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265VideoParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same vps_video_parameter_set_id.
- StdVideoH265SequenceParameterSet structures representing H.265 SPS entries, as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH265SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH265SequenceParameterSet entries specified in pParametersAddInfo->pStdSPSs are added first;
  - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265SequenceParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same sps_video_parameter_set_id and sps_seq_parameter_set_id.
- StdVideoH265PictureParameterSet structures representing H.265 PPS entries, as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH265SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH265PictureParameterSet entries specified in pParametersAddInfo->pStdPPSs are added first;
  - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265PictureParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same sps_video_parameter_set_id, pps_seq_parameter_set_id, and pps_pic_parameter_set_id.

If pCreateInfo->videoSession was created with the video codec operation

VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then the created video session parameters object will contain a single AV1 sequence header represented by a StdVideoAV1SequenceHeader structure specified through the pStdSequenceHeader member of the VkVideoDecodeAV1SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain. As such video session parameters objects **can** only contain a single AV1 sequence header, it is not possible to use a previously created object as a template or subsequently update the created video session parameters object.

If pCreateInfo->videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the created video session parameters object will initially contain the following sets of parameter entries:

- StdVideoH264SequenceParameterSet structures representing H.264 SPS entries, as follows:
  - If the pParametersAddInfo member of the VkVideoEncodeH264SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH264SequenceParameterSet entries specified in pParametersAddInfo->pStdSPSs are added first;
  - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH264SequenceParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same seq_parameter_set_id.
- StdVideoH264PictureParameterSet structures representing H.264 PPS entries, as follows:
  - If the pParametersAddInfo member of the VkVideoEncodeH264SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH264PictureParameterSet entries specified in pParametersAddInfo->pStdPPSs are added first;
  - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH264PictureParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same seq_parameter_set_id and pic_parameter_set_id.

If pCreateInfo->videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the created video session parameters object will initially contain the following sets of parameter entries:

- StdVideoH265VideoParameterSet structures representing H.265 VPS entries, as follows:
  - If the pParametersAddInfo member of the VkVideoEncodeH265SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH265VideoParameterSet entries specified in pParametersAddInfo->pStdVPSs are added first;
  - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265VideoParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same vps_video_parameter_set_id.
- StdVideoH265SequenceParameterSet structures representing H.265 SPS entries, as follows:

    - If the pParametersAddInfo member of the VkVideoEncodeH265SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH265SequenceParameterSet entries specified in pParametersAddInfo->pStdSPSs are added first;
    - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265SequenceParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same sps_video_parameter_set_id and sps_seq_parameter_set_id.
- StdVideoH265PictureParameterSet structures representing H.265 PPS entries, as follows:
    - If the pParametersAddInfo member of the VkVideoEncodeH265SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain is not NULL, then the set of StdVideoH265PictureParameterSet entries specified in pParametersAddInfo->pStdPPSs are added first;
    - If pCreateInfo->videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265PictureParameterSet entry stored in it is copied to the created video session parameters object if the created object does not already contain such an entry with the same sps_video_parameter_set_id, pps_seq_parameter_set_id, and pps_pic_parameter_set_id.

If pCreateInfo->videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the created video session parameters object will contain a single AV1 sequence header specified through the members of the VkVideoEncodeAV1SessionParametersCreateInfoKHR structure provided in the pCreateInfo->pNext chain. As such video session parameters objects **can** only contain a single AV1 sequence header, it is not possible to use a previously created object as a template or subsequently update the created video session parameters object.

In case of video session parameters objects created with a video encode operation, implementations **may** return the VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR error if any of the specified Video Std parameters do not adhere to the syntactic or semantic requirements of the used video compression standard, or if values derived from parameters according to the rules defined by the used video compression standard do not adhere to the capabilities of the video compression standard or the implementation.

NOTE	Applications **should** not rely on the VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR error being returned by any command as a means to verify Video Std parameters, as implementations are not required to report the error in any specific set of cases.

Valid Usage (Implicit)

- VUID-vkCreateVideoSessionParametersKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkCreateVideoSessionParametersKHR-pCreateInfo-parameter
  pCreateInfo **must** be a valid pointer to a valid VkVideoSessionParametersCreateInfoKHR structure
- VUID-vkCreateVideoSessionParametersKHR-pAllocator-parameter

If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure

- VUID-vkCreateVideoSessionParametersKHR-pVideoSessionParameters-parameter `pVideoSessionParameters` **must** be a valid pointer to a VkVideoSessionParametersKHR handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_INITIALIZATION_FAILED`
- `VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR`

The `VkVideoSessionParametersCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoSessionParametersCreateInfoKHR {
    VkStructureType                            sType;
    const void*                                pNext;
    VkVideoSessionParametersCreateFlagsKHR     flags;
    VkVideoSessionParametersKHR                videoSessionParametersTemplate;
    VkVideoSessionKHR                          videoSession;
} VkVideoSessionParametersCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkVideoSessionParametersCreateFlagBitsKHR specifying create flags.
- `videoSessionParametersTemplate` is `VK_NULL_HANDLE` or a valid handle to a VkVideoSessionParametersKHR object used as a template for constructing the new video session parameters object.
- `videoSession` is the video session object against which the video session parameters object is going to be created.

Limiting values are defined below that are referenced by the relevant valid usage statements of this structure.

- If `videoSession` was created with the codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, then let `StdVideoH264SequenceParameterSet spsAddList[]` be the list of H.264 SPS entries to add to the created video session parameters

object, defined as follows:

  - If the pParametersAddInfo member of the VkVideoDecodeH264SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH264SequenceParameterSet entries specified in pParametersAddInfo->pStdSPSs are added to spsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH264SequenceParameterSet entry stored in it with seq_parameter_set_id not matching any of the entries already in spsAddList is added to spsAddList.

- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then let StdVideoH264PictureParameterSet ppsAddList[] be the list of H.264 PPS entries to add to the created video session parameters object, defined as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH264SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH264PictureParameterSet entries specified in pParametersAddInfo->pStdPPSs are added to ppsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH264PictureParameterSet entry stored in it with seq_parameter_set_id or pic_parameter_set_id not matching any of the entries already in ppsAddList is added to ppsAddList.
- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then let StdVideoH265VideoParameterSet vpsAddList[] be the list of H.265 VPS entries to add to the created video session parameters object, defined as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH265SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH265VideoParameterSet entries specified in pParametersAddInfo->pStdVPSs are added to vpsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265VideoParameterSet entry stored in it with vps_video_parameter_set_id not matching any of the entries already in vpsAddList is added to vpsAddList.
- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then let StdVideoH265SequenceParameterSet spsAddList[] be the list of H.265 SPS entries to add to the created video session parameters object, defined as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH265SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH265SequenceParameterSet entries specified in pParametersAddInfo->pStdSPSs are added to spsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265SequenceParameterSet entry stored in it with sps_video_parameter_set_id or sps_seq_parameter_set_id not matching any of the entries already in spsAddList is added to spsAddList.

- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then let StdVideoH265PictureParameterSet ppsAddList[] be the list of H.265 PPS entries to add to the created video session parameters object, defined as follows:
  - If the pParametersAddInfo member of the VkVideoDecodeH265SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH265PictureParameterSet entries specified in pParametersAddInfo->pStdPPSs are added to ppsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265PictureParameterSet entry stored in it with sps_video_parameter_set_id, pps_seq_parameter_set_id, or pps_pic_parameter_set_id not matching any of the entries already in ppsAddList is added to ppsAddList.
- If videoSession was created with an encode operation, then let uint32_t qualityLevel be the video encode quality level of the created video session parameters object, defined as follows:
  - If the pNext chain of this structure includes a VkVideoEncodeQualityLevelInfoKHR structure, then qualityLevel is equal to VkVideoEncodeQualityLevelInfoKHR::qualityLevel.
  - Otherwise qualityLevel is 0
- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then let StdVideoH264SequenceParameterSet spsAddList[] be the list of H.264 SPS entries to add to the created video session parameters object, defined as follows:
  - If the pParametersAddInfo member of the VkVideoEncodeH264SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH264SequenceParameterSet entries specified in pParametersAddInfo->pStdSPSs are added to spsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH264SequenceParameterSet entry stored in it with seq_parameter_set_id not matching any of the entries already in spsAddList is added to spsAddList.
- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then let StdVideoH264PictureParameterSet ppsAddList[] be the list of H.264 PPS entries to add to the created video session parameters object, defined as follows:
  - If the pParametersAddInfo member of the VkVideoEncodeH264SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH264PictureParameterSet entries specified in pParametersAddInfo->pStdPPSs are added to ppsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH264PictureParameterSet entry stored in it with seq_parameter_set_id or pic_parameter_set_id not matching any of the entries already in ppsAddList is added to ppsAddList.
- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then let StdVideoH265VideoParameterSet vpsAddList[] be the list of H.265 VPS entries to add to the created video session parameters

object, defined as follows:

  - If the pParametersAddInfo member of the VkVideoEncodeH265SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH265VideoParameterSet entries specified in pParametersAddInfo->pStdVPSs are added to vpsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265VideoParameterSet entry stored in it with vps_video_parameter_set_id not matching any of the entries already in vpsAddList is added to vpsAddList.

- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then let StdVideoH265SequenceParameterSet spsAddList[] be the list of H.265 SPS entries to add to the created video session parameters object, defined as follows:
  - If the pParametersAddInfo member of the VkVideoEncodeH265SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH265SequenceParameterSet entries specified in pParametersAddInfo->pStdSPSs are added to spsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265SequenceParameterSet entry stored in it with sps_video_parameter_set_id or sps_seq_parameter_set_id not matching any of the entries already in spsAddList is added to spsAddList.
- If videoSession was created with the codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then let StdVideoH265PictureParameterSet ppsAddList[] be the list of H.265 PPS entries to add to the created video session parameters object, defined as follows:
  - If the pParametersAddInfo member of the VkVideoEncodeH265SessionParametersCreateInfoKHR structure provided in the pNext chain is not NULL, then the set of StdVideoH265PictureParameterSet entries specified in pParametersAddInfo->pStdPPSs are added to ppsAddList;
  - If videoSessionParametersTemplate is not VK_NULL_HANDLE, then each StdVideoH265PictureParameterSet entry stored in it with sps_video_parameter_set_id, pps_seq_parameter_set_id, or pps_pic_parameter_set_id not matching any of the entries already in ppsAddList is added to ppsAddList.

Valid Usage

- VUID-VkVideoSessionParametersCreateInfoKHR-videoSessionParametersTemplate-04855
  If videoSessionParametersTemplate is not VK_NULL_HANDLE, it **must** have been created against videoSession
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSessionParametersTemplate-08310
  If videoSessionParametersTemplate is not VK_NULL_HANDLE and videoSession was created with an encode operation, then qualityLevel **must** equal the video encode quality level videoSessionParametersTemplate was created with
- VUID-VkVideoSessionParametersCreateInfoKHR-flags-10271

If flags includes VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, then videoSession **must** have been created with VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR

- VUID-VkVideoSessionParametersCreateInfoKHR-flags-10272
  If flags includes VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, then the pNext chain **must** include a VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR structure
- VUID-VkVideoSessionParametersCreateInfoKHR-flags-10273
  If flags includes VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR and videoSession was created with VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR, then the list of video format properties supported for the image usage flag VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR **must** have an element for which VkVideoFormatQuantizationMapPropertiesKHR::quantizationMapTexelSize equals the quantizationMapTexelSize member of the VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-flags-10274
  If flags includes VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR and videoSession was created with VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR, then the list of video format properties supported for the image usage flag VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR **must** have an element for which VkVideoFormatQuantizationMapPropertiesKHR::quantizationMapTexelSize equals the quantizationMapTexelSize member of the VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSessionParametersTemplate-10275
  If videoSessionParametersTemplate is not VK_NULL_HANDLE and flags includes VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, then videoSessionParametersTemplate **must** have been created with VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSessionParametersTemplate-10276
  If videoSessionParametersTemplate is not VK_NULL_HANDLE and flags includes VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, then videoSessionParametersTemplate **must** have been created with the same quantization map texel size as the one specified in the quantizationMapTexelSize member of the VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSessionParametersTemplate-10277

If videoSessionParametersTemplate is not VK_NULL_HANDLE and flags does not include VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, then videoSessionParametersTemplate **must** have been created without VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR

- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07203
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the pNext chain **must** include a VkVideoDecodeH264SessionParametersCreateInfoKHR structure
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07204
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the number of elements of spsAddList **must** be less than or equal to the maxStdSPSCount specified in the VkVideoDecodeH264SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07205
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the number of elements of ppsAddList **must** be less than or equal to the maxStdPPSCount specified in the VkVideoDecodeH264SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07206
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the pNext chain **must** include a VkVideoDecodeH265SessionParametersCreateInfoKHR structure
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07207
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the number of elements of vpsAddList **must** be less than or equal to the maxStdVPSCount specified in the VkVideoDecodeH265SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07208
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the number of elements of spsAddList **must** be less than or equal to the maxStdSPSCount specified in the VkVideoDecodeH265SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07209
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the number of elements of ppsAddList **must** be less than or equal to the maxStdPPSCount specified in the VkVideoDecodeH265SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-09258
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then videoSessionParametersTemplate **must**

be VK_NULL_HANDLE

- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-09259
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then the pNext chain **must** include a VkVideoDecodeAV1SessionParametersCreateInfoKHR structure
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07210
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the pNext chain **must** include a VkVideoEncodeH264SessionParametersCreateInfoKHR structure
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-04839
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the number of elements of spsAddList **must** be less than or equal to the maxStdSPSCount specified in the VkVideoEncodeH264SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-04840
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the number of elements of ppsAddList **must** be less than or equal to the maxStdPPSCount specified in the VkVideoEncodeH264SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-07211
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the pNext chain **must** include a VkVideoEncodeH265SessionParametersCreateInfoKHR structure
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-04841
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the number of elements of vpsAddList **must** be less than or equal to the maxStdVPSCount specified in the VkVideoEncodeH265SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-04842
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the number of elements of spsAddList **must** be less than or equal to the maxStdSPSCount specified in the VkVideoEncodeH265SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-04843
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the number of elements of ppsAddList **must** be less than or equal to the maxStdPPSCount specified in the VkVideoEncodeH265SessionParametersCreateInfoKHR structure included in the pNext chain
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-08319

If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then num_tile_columns_minus1 **must** be less than VkVideoEncodeH265CapabilitiesKHR::maxTiles.width, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile videoSession was created with, for each element of ppsAddList

- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-08320
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then num_tile_rows_minus1 **must** be less than VkVideoEncodeH265CapabilitiesKHR::maxTiles.height, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile videoSession was created with, for each element of ppsAddList
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-10278
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then videoSessionParametersTemplate **must** be VK_NULL_HANDLE
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-10279
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the pNext chain **must** include a VkVideoEncodeAV1SessionParametersCreateInfoKHR structure
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-10280
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the stdOperatingPointCount member of the VkVideoEncodeAV1SessionParametersCreateInfoKHR structure included in the pNext chain **must** be less than or equal to VkVideoEncodeAV1CapabilitiesKHR::maxOperatingPoints, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile videoSession was created with

Valid Usage (Implicit)

- VUID-VkVideoSessionParametersCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_SESSION_PARAMETERS_CREATE_INFO_KHR
- VUID-VkVideoSessionParametersCreateInfoKHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkVideoDecodeAV1SessionParametersCreateInfoKHR, VkVideoDecodeH264SessionParametersCreateInfoKHR, VkVideoDecodeH265SessionParametersCreateInfoKHR, VkVideoEncodeAV1SessionParametersCreateInfoKHR, VkVideoEncodeH264SessionParametersCreateInfoKHR, VkVideoEncodeH265SessionParametersCreateInfoKHR, VkVideoEncodeQualityLevelInfoKHR, or VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR
- VUID-VkVideoSessionParametersCreateInfoKHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique

- VUID-VkVideoSessionParametersCreateInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkVideoSessionParametersCreateFlagBitsKHR values
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSessionParametersTemplate-parameter
  If `videoSessionParametersTemplate` is not VK_NULL_HANDLE, `videoSessionParametersTemplate` **must** be a valid VkVideoSessionParametersKHR handle
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSession-parameter
  `videoSession` **must** be a valid VkVideoSessionKHR handle
- VUID-VkVideoSessionParametersCreateInfoKHR-videoSessionParametersTemplate-parent
  If `videoSessionParametersTemplate` is a valid handle, it **must** have been created, allocated, or retrieved from `videoSession`
- VUID-VkVideoSessionParametersCreateInfoKHR-commonparent
  Both of `videoSession`, and `videoSessionParametersTemplate` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

Bits which **can** be set in VkVideoSessionParametersCreateInfoKHR::`flags` are:

```
// Provided by VK_KHR_video_encode_quantization_map
typedef enum VkVideoSessionParametersCreateFlagBitsKHR {
  // Provided by VK_KHR_video_encode_quantization_map
    VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR =
0x00000001,
} VkVideoSessionParametersCreateFlagBitsKHR;
```

- `VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR` specifies that the created video session parameters object **can** be used with quantization maps.

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoSessionParametersCreateFlagsKHR;
```

`VkVideoSessionParametersCreateFlagsKHR` is a bitmask type for setting a mask of zero or more VkVideoSessionParametersCreateFlagBitsKHR.

The `VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_quantization_map
typedef struct VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkExtent2D         quantizationMapTexelSize;
} VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `quantizationMapTexelSize` specifies the quantization map texel size a video session parameters object created with `VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR` is compatible with.

Valid Usage (Implicit)

- VUID-VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_SESSION_PARAMETERS_CREATE_INFO_KHR`

38.7.2. Destroying Video Session Parameters

To destroy a video session parameters object, call:

```
// Provided by VK_KHR_video_queue
void vkDestroyVideoSessionParametersKHR(
    VkDevice                                    device,
    VkVideoSessionParametersKHR                 videoSessionParameters,
    const VkAllocationCallbacks*                pAllocator);
```

- `device` is the logical device that destroys the video session parameters object.
- `videoSessionParameters` is the video session parameters object to destroy.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyVideoSessionParametersKHR-videoSessionParameters-07212
  All submitted commands that refer to `videoSessionParameters` **must** have completed execution
- VUID-vkDestroyVideoSessionParametersKHR-videoSessionParameters-07213
  If `VkAllocationCallbacks` were provided when `videoSessionParameters` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyVideoSessionParametersKHR-videoSessionParameters-07214
  If no `VkAllocationCallbacks` were provided when `videoSessionParameters` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyVideoSessionParametersKHR-device-parameter
  `device` **must** be a valid VkDevice handle

- VUID-vkDestroyVideoSessionParametersKHR-videoSessionParameters-parameter
  If `videoSessionParameters` is not `VK_NULL_HANDLE`, `videoSessionParameters` **must** be a valid VkVideoSessionParametersKHR handle
- VUID-vkDestroyVideoSessionParametersKHR-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyVideoSessionParametersKHR-videoSessionParameters-parent
  If `videoSessionParameters` is a valid handle, it **must** have been created, allocated, or retrieved from `device`

Host Synchronization

- Host access to `videoSessionParameters` **must** be externally synchronized

38.7.3. Updating Video Session Parameters

To update video session parameters object with new parameters, call:

```
// Provided by VK_KHR_video_queue
VkResult vkUpdateVideoSessionParametersKHR(
    VkDevice                                    device,
    VkVideoSessionParametersKHR                 videoSessionParameters,
    const VkVideoSessionParametersUpdateInfoKHR* pUpdateInfo);
```

- `device` is the logical device that updates the video session parameters.
- `videoSessionParameters` is the video session parameters object to update.
- `pUpdateInfo` is a pointer to a VkVideoSessionParametersUpdateInfoKHR structure specifying the parameter update information.

After a successful call to this command, the update sequence counter of `videoSessionParameters` is changed to the value specified in `pUpdateInfo->updateSequenceCount`.

NOTE

As each update issued to a video session parameters object needs to specify the next available update sequence count value, concurrent updates of the same video session parameters object are inherently disallowed. However, recording video coding operations to command buffers referring to parameters previously added to the video session parameters object is allowed, even if there is a concurrent update in progress adding some new entries to the object.

If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` and the `pUpdateInfo->pNext` chain includes a VkVideoDecodeH264SessionParametersAddInfoKHR structure, then this command adds the following parameter entries to `videoSessionParameters`:

- The H.264 SPS entries specified in VkVideoDecodeH264SessionParametersAddInfoKHR::pStdSPSs.
- The H.264 PPS entries specified in VkVideoDecodeH264SessionParametersAddInfoKHR::pStdPPSs.

If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR and the pUpdateInfo->pNext chain includes a VkVideoDecodeH265SessionParametersAddInfoKHR structure, then this command adds the following parameter entries to videoSessionParameters:

- The H.265 VPS entries specified in VkVideoDecodeH265SessionParametersAddInfoKHR::pStdVPSs.
- The H.265 SPS entries specified in VkVideoDecodeH265SessionParametersAddInfoKHR::pStdSPSs.
- The H.265 PPS entries specified in VkVideoDecodeH265SessionParametersAddInfoKHR::pStdPPSs.

If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and the pUpdateInfo->pNext chain includes a VkVideoEncodeH264SessionParametersAddInfoKHR structure, then this command adds the following parameter entries to videoSessionParameters:

- The H.264 SPS entries specified in VkVideoEncodeH264SessionParametersAddInfoKHR::pStdSPSs.
- The H.264 PPS entries specified in VkVideoEncodeH264SessionParametersAddInfoKHR::pStdPPSs.

If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the pUpdateInfo->pNext chain includes a VkVideoEncodeH265SessionParametersAddInfoKHR structure, then this command adds the following parameter entries to videoSessionParameters:

- The H.265 VPS entries specified in VkVideoEncodeH265SessionParametersAddInfoKHR::pStdVPSs.
- The H.265 SPS entries specified in VkVideoEncodeH265SessionParametersAddInfoKHR::pStdSPSs.
- The H.265 PPS entries specified in VkVideoEncodeH265SessionParametersAddInfoKHR::pStdPPSs.

In case of video session parameters objects created with a video encode operation, implementations **may** return the VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR error if any of the specified Video Std parameters do not adhere to the syntactic or semantic requirements of the used video compression standard, or if values derived from parameters according to the rules defined by the used video compression standard do not adhere to the capabilities of the video compression standard or the implementation.

NOTE	Applications **should** not rely on the VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR

error being returned by any command as a means to verify Video Std parameters, as implementations are not required to report the error in any specific set of cases.

Valid Usage

- VUID-vkUpdateVideoSessionParametersKHR-pUpdateInfo-07215
  `pUpdateInfo->updateSequenceCount` **must** equal the current update sequence counter of `videoSessionParameters` plus one
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07216
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` and the `pNext` chain of `pUpdateInfo` includes a VkVideoDecodeH264SessionParametersAddInfoKHR structure, then `videoSessionParameters` **must** not already contain a `StdVideoH264SequenceParameterSet` entry with `seq_parameter_set_id` matching any of the elements of VkVideoDecodeH264SessionParametersAddInfoKHR::`pStdSPSs`
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07217
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, then the number of `StdVideoH264SequenceParameterSet` entries already stored in it plus the value of the `stdSPSCount` member of the VkVideoDecodeH264SessionParametersAddInfoKHR structure included in the `pUpdateInfo->pNext` chain **must** be less than or equal to the VkVideoDecodeH264SessionParametersCreateInfoKHR::`maxStdSPSCount` `videoSessionParameters` was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07218
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` and the `pNext` chain of `pUpdateInfo` includes a VkVideoDecodeH264SessionParametersAddInfoKHR structure, then `videoSessionParameters` **must** not already contain a `StdVideoH264PictureParameterSet` entry with both `seq_parameter_set_id` and `pic_parameter_set_id` matching any of the elements of VkVideoDecodeH264SessionParametersAddInfoKHR::`pStdPPSs`
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07219
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, then the number of `StdVideoH264PictureParameterSet` entries already stored in it plus the value of the `stdPPSCount` member of the VkVideoDecodeH264SessionParametersAddInfoKHR structure included in the `pUpdateInfo->pNext` chain **must** be less than or equal to the VkVideoDecodeH264SessionParametersCreateInfoKHR::`maxStdPPSCount` `videoSessionParameters` was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07220
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR` and the `pNext` chain of `pUpdateInfo` includes a VkVideoDecodeH265SessionParametersAddInfoKHR structure, then `videoSessionParameters` **must** not already contain a `StdVideoH265VideoParameterSet` entry with `vps_video_parameter_set_id` matching any of the elements of VkVideoDecodeH265SessionParametersAddInfoKHR::`pStdVPSs`

- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07221
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR`, then the number of `StdVideoH265VideoParameterSet` entries already stored in it plus the value of the `stdVPSCount` member of the VkVideoDecodeH265SessionParametersAddInfoKHR structure included in the `pUpdateInfo->pNext` chain **must** be less than or equal to the VkVideoDecodeH265SessionParametersCreateInfoKHR::`maxStdVPSCount` `videoSessionParameters` was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07222
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR` and the `pNext` chain of `pUpdateInfo` includes a VkVideoDecodeH265SessionParametersAddInfoKHR structure, then `videoSessionParameters` **must** not already contain a `StdVideoH265SequenceParameterSet` entry with both `sps_video_parameter_set_id` and `sps_seq_parameter_set_id` matching any of the elements of VkVideoDecodeH265SessionParametersAddInfoKHR::`pStdSPSs`
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07223
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR`, then the number of `StdVideoH265SequenceParameterSet` entries already stored in it plus the value of the `stdSPSCount` member of the VkVideoDecodeH265SessionParametersAddInfoKHR structure included in the `pUpdateInfo->pNext` chain **must** be less than or equal to the VkVideoDecodeH265SessionParametersCreateInfoKHR::`maxStdSPSCount` `videoSessionParameters` was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07224
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR` and the `pNext` chain of `pUpdateInfo` includes a VkVideoDecodeH265SessionParametersAddInfoKHR structure, then `videoSessionParameters` **must** not already contain a `StdVideoH265PictureParameterSet` entry with `sps_video_parameter_set_id`, `pps_seq_parameter_set_id`, and `pps_pic_parameter_set_id` all matching any of the elements of VkVideoDecodeH265SessionParametersAddInfoKHR ::`pStdPPSs`
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07225
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR`, then the number of `StdVideoH265PictureParameterSet` entries already stored in it plus the value of the `stdPPSCount` member of the VkVideoDecodeH265SessionParametersAddInfoKHR structure included in the `pUpdateInfo->pNext` chain **must** be less than or equal to the VkVideoDecodeH265SessionParametersCreateInfoKHR::`maxStdPPSCount` `videoSessionParameters` was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-09260
  `videoSessionParameters` **must** not have been created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR`
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07226
  If `videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR` and the `pNext` chain of `pUpdateInfo` includes

a VkVideoEncodeH264SessionParametersAddInfoKHR structure, then videoSessionParameters **must** not already contain a StdVideoH264SequenceParameterSet entry with seq_parameter_set_id matching any of the elements of VkVideoEncodeH264SessionParametersAddInfoKHR::pStdSPSs

- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-06441
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the number of StdVideoH264SequenceParameterSet entries already stored in it plus the value of the stdSPSCount member of the VkVideoEncodeH264SessionParametersAddInfoKHR structure included in the pUpdateInfo->pNext chain **must** be less than or equal to the VkVideoEncodeH264SessionParametersCreateInfoKHR::maxStdSPSCount videoSessionParameters was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07227
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and the pNext chain of pUpdateInfo includes a VkVideoEncodeH264SessionParametersAddInfoKHR structure, then videoSessionParameters **must** not already contain a StdVideoH264PictureParameterSet entry with both seq_parameter_set_id and pic_parameter_set_id matching any of the elements of VkVideoEncodeH264SessionParametersAddInfoKHR::pStdPPSs
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-06442
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the number of StdVideoH264PictureParameterSet entries already stored in it plus the value of the stdPPSCount member of the VkVideoEncodeH264SessionParametersAddInfoKHR structure included in the pUpdateInfo->pNext chain **must** be less than or equal to the VkVideoEncodeH264SessionParametersCreateInfoKHR::maxStdPPSCount videoSessionParameters was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07228
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the pNext chain of pUpdateInfo includes a VkVideoEncodeH265SessionParametersAddInfoKHR structure, then videoSessionParameters **must** not already contain a StdVideoH265VideoParameterSet entry with vps_video_parameter_set_id matching any of the elements of VkVideoEncodeH265SessionParametersAddInfoKHR::pStdVPSs
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-06443
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the number of StdVideoH265VideoParameterSet entries already stored in it plus the value of the stdVPSCount member of the VkVideoEncodeH265SessionParametersAddInfoKHR structure included in the pUpdateInfo->pNext chain **must** be less than or equal to the VkVideoEncodeH265SessionParametersCreateInfoKHR::maxStdVPSCount videoSessionParameters was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07229
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the pNext chain of pUpdateInfo includes

a VkVideoEncodeH265SessionParametersAddInfoKHR structure, then videoSessionParameters **must** not already contain a StdVideoH265SequenceParameterSet entry with both sps_video_parameter_set_id and sps_seq_parameter_set_id matching any of the elements of VkVideoEncodeH265SessionParametersAddInfoKHR::pStdSPSs

- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-06444
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the number of StdVideoH265SequenceParameterSet entries already stored in it plus the value of the stdSPSCount member of the VkVideoEncodeH265SessionParametersAddInfoKHR structure included in the pUpdateInfo->pNext chain **must** be less than or equal to the VkVideoEncodeH265SessionParametersCreateInfoKHR::maxStdSPSCount videoSessionParameters was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-07230
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the pNext chain of pUpdateInfo includes a VkVideoEncodeH265SessionParametersAddInfoKHR structure, then videoSessionParameters **must** not already contain a StdVideoH265PictureParameterSet entry with sps_video_parameter_set_id, pps_seq_parameter_set_id, and pps_pic_parameter_set_id all matching any of the elements of VkVideoEncodeH265SessionParametersAddInfoKHR ::pStdPPSs
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-06445
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the number of StdVideoH265PictureParameterSet entries already stored in it plus the value of the stdPPSCount member of the VkVideoEncodeH265SessionParametersAddInfoKHR structure included in the pUpdateInfo->pNext chain **must** be less than or equal to the VkVideoEncodeH265SessionParametersCreateInfoKHR::maxStdPPSCount videoSessionParameters was created with
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-08321
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the pNext chain of pUpdateInfo includes a VkVideoEncodeH265SessionParametersAddInfoKHR structure, then num_tile_columns_minus1 **must** be less than VkVideoEncodeH265CapabilitiesKHR ::maxTiles.width, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile videoSessionParameters was created with, for each element of VkVideoEncodeH265SessionParametersAddInfoKHR::pStdPPSs
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-08322
  If videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the pNext chain of pUpdateInfo includes a VkVideoEncodeH265SessionParametersAddInfoKHR structure, then num_tile_rows_minus1 **must** be less than VkVideoEncodeH265CapabilitiesKHR ::maxTiles.height, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile videoSessionParameters was created with, for each element of VkVideoEncodeH265SessionParametersAddInfoKHR::pStdPPSs
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-10281

videoSessionParameters **must** not have been created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR

Valid Usage (Implicit)

- VUID-vkUpdateVideoSessionParametersKHR-device-parameter
  device **must** be a valid VkDevice handle
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-parameter
  videoSessionParameters **must** be a valid VkVideoSessionParametersKHR handle
- VUID-vkUpdateVideoSessionParametersKHR-pUpdateInfo-parameter
  pUpdateInfo **must** be a valid pointer to a valid VkVideoSessionParametersUpdateInfoKHR structure
- VUID-vkUpdateVideoSessionParametersKHR-videoSessionParameters-parent
  videoSessionParameters **must** have been created, allocated, or retrieved from device

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR

The VkVideoSessionParametersUpdateInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoSessionParametersUpdateInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           updateSequenceCount;
} VkVideoSessionParametersUpdateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- updateSequenceCount is the new update sequence count to set for the video session parameters object.

Valid Usage (Implicit)

- VUID-VkVideoSessionParametersUpdateInfoKHR-sType-sType

sType **must** be VK_STRUCTURE_TYPE_VIDEO_SESSION_PARAMETERS_UPDATE_INFO_KHR

- VUID-VkVideoSessionParametersUpdateInfoKHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkVideoDecodeH264SessionParametersAddInfoKHR, VkVideoDecodeH265SessionParametersAddInfoKHR, VkVideoEncodeH264SessionParametersAddInfoKHR, or VkVideoEncodeH265SessionParametersAddInfoKHR
- VUID-VkVideoSessionParametersUpdateInfoKHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique

38.8. Video Coding Scope

Applications **can** record video coding commands for a video session only within a video coding scope.

To begin a video coding scope, call:

```
// Provided by VK_KHR_video_queue
void vkCmdBeginVideoCodingKHR(
    VkCommandBuffer                             commandBuffer,
    const VkVideoBeginCodingInfoKHR*            pBeginInfo);
```

- commandBuffer is the command buffer in which to record the command.
- pBeginInfo is a pointer to a VkVideoBeginCodingInfoKHR structure specifying the parameters of the video coding scope, including the video session and video session parameters object to use.

After beginning a video coding scope, the video session object specified in pBeginInfo->videoSession is *bound* to the command buffer, and the command buffer is ready to record video coding operations. Similarly, if pBeginInfo->videoSessionParameters is not VK_NULL_HANDLE, it is also *bound* to the command buffer, and video coding operations **can** refer to the codec-specific parameters stored in it.

This command also establishes the set of *bound reference picture resources* that **can** be used as reconstructed pictures or reference pictures within the video coding scope. Each element of this set consists of a video picture resource and the DPB slot index associated with it, if there is one.

The set of bound reference picture resources is immutable within a video coding scope, however, the DPB slot index associated with any of the bound reference picture resources **can** change during the video coding scope in response to video coding operations.

The VkVideoReferenceSlotInfoKHR structures provided as the elements of pBeginInfo->pReferenceSlots are interpreted by this command as follows:

- If slotIndex is non-negative and pPictureResource is not NULL, then the video picture resource defined by the VkVideoPictureResourceInfoKHR structure pointed to by pPictureResource is

added to the set of bound reference picture resources and is associated with the DPB slot index specified in `slotIndex`.

- If `slotIndex` is non-negative and `pPictureResource` is `NULL`, then the DPB slot with index `slotIndex` is deactivated by this command.
- If `slotIndex` is negative and `pPictureResource` is not `NULL`, then the video picture resource defined by the VkVideoPictureResourceInfoKHR structure pointed to by `pPictureResource` is added to the set of bound reference picture resources without an associated DPB slot. Such a picture resource **can** be subsequently used as a reconstructed picture to associate it with a DPB slot.
- If `slotIndex` is negative and `pPictureResource` is `NULL`, then the element is ignored.

NOTE

It is possible for multiple bound reference picture resources to be associated with the same DPB slot index, or for a single bound reference picture to refer to multiple separate reference pictures. For example, in case of an H.264 decode profile with interlaced frame support a single DPB slot can refer to two separate pictures for the top and bottom fields. Depending on the picture layout used by the H.264 decode profile, the following special cases **may** arise:

- If the picture layout is `VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_INTERLEAVED_LINES_BIT_KHR`, then the top and bottom field pictures are physically co-located in the same video picture resource with even scanlines corresponding to the top field and odd scanlines corresponding to the bottom field, respectively.
- If the picture layout is `VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR`, then the top and bottom field pictures are stored in separate video picture resources (in separate subregions of the same image layer, in separate layers of the same image, or in entirely separate images), hence two elements of VkVideoBeginCodingInfoKHR::`pReferenceSlots` **can** contain the same `slotIndex` but specify different video picture resources in their `pPictureResource` members.

All non-negative `slotIndex` values specified in the elements of `pBeginInfo->pReferenceSlots` **must** identify DPB slots of the video session that are in the active state at the time this command is executed on the device.

NOTE

The application does not have to specify an entry in `pBeginInfo->pReferenceSlots` corresponding to all active DPB slots of the video session, but only for those which are intended to be used in the video coding scope. This way the application can avoid any potential runtime cost associated with binding the corresponding picture resources to the command buffer.

In case of a video encode session, the application is also responsible for providing information about the current rate control state configured for the video session by including an instance of the VkVideoEncodeRateControlInfoKHR structure in the `pNext` chain of `pBeginInfo`. If no VkVideoEncodeRateControlInfoKHR is included, then the presence of an empty VkVideoEncodeRateControlInfoKHR structure is implied which indicates that the current rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR`. The specified state **must** match the

effective rate control state configured for the video session at the time the recorded command is executed on the device.

NOTE

Including an instance of the VkVideoEncodeRateControlInfoKHR structure in the pNext chain of pBeginInfo does not change the rate control state configured for the video session, but only specifies the expected rate control state configured at the time the recorded command is executed on the device which allows the implementation to have information about the configured rate control state at command buffer recording time. In order to change the current rate control state of a video session, the application has to issue an appropriate vkCmdControlVideoCodingKHR command as described in the Video Coding Control and Rate Control State sections.

Valid Usage

- VUID-vkCmdBeginVideoCodingKHR-commandBuffer-07231
  The VkCommandPool that commandBuffer was allocated from **must** support the video codec operation pBeginInfo->videoSession was created with, as returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR in VkQueueFamilyVideoPropertiesKHR::videoCodecOperations
- VUID-vkCmdBeginVideoCodingKHR-None-07232
  There **must** be no active queries
- VUID-vkCmdBeginVideoCodingKHR-commandBuffer-07233
  If commandBuffer is an unprotected command buffer and protectedNoFault is not supported, then pBeginInfo->videoSession **must** not have been created with VK_VIDEO_SESSION_CREATE_PROTECTED_CONTENT_BIT_KHR
- VUID-vkCmdBeginVideoCodingKHR-commandBuffer-07234
  If commandBuffer is a protected command buffer and protectedNoFault is not supported, then pBeginInfo->videoSession **must** have been created with VK_VIDEO_SESSION_CREATE_PROTECTED_CONTENT_BIT_KHR
- VUID-vkCmdBeginVideoCodingKHR-commandBuffer-07235
  If commandBuffer is an unprotected command buffer, protectedNoFault is not supported, and the pPictureResource member of any element of pBeginInfo->pReferenceSlots is not NULL, then pPictureResource->imageViewBinding for that element **must** not specify an image view created from a protected image
- VUID-vkCmdBeginVideoCodingKHR-commandBuffer-07236
  If commandBuffer is a protected command buffer protectedNoFault is not supported, and the pPictureResource member of any element of pBeginInfo->pReferenceSlots is not NULL, then pPictureResource->imageViewBinding for that element **must** specify an image view created from a protected image
- VUID-vkCmdBeginVideoCodingKHR-slotIndex-07239
  If the slotIndex member of any element of pBeginInfo->pReferenceSlots is not negative, then it **must** specify the index of a DPB slot that is in the active state in pBeginInfo->videoSession at the time the command is executed on the device

- VUID-vkCmdBeginVideoCodingKHR-pPictureResource-07265
  Each video picture resource specified by any non-NULL pPictureResource member specified in the elements of pBeginInfo->pReferenceSlots for which slotIndex is not negative **must** match one of the video picture resources currently associated with the DPB slot index of pBeginInfo->videoSession specified by slotIndex at the time the command is executed on the device
- VUID-vkCmdBeginVideoCodingKHR-pBeginInfo-08253
  If pBeginInfo->videoSession was created with a video encode operation and the pNext chain of pBeginInfo does not include an instance of the VkVideoEncodeRateControlInfoKHR structure, then the rate control mode configured for pBeginInfo->videoSession at the time the command is executed on the device **must** be VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR
- VUID-vkCmdBeginVideoCodingKHR-pBeginInfo-08254
  If pBeginInfo->videoSession was created with a video encode operation and the pNext chain of pBeginInfo includes an instance of the VkVideoEncodeRateControlInfoKHR structure, then it **must** match the rate control state configured for pBeginInfo->videoSession at the time the command is executed on the device
- VUID-vkCmdBeginVideoCodingKHR-pBeginInfo-08255
  If pBeginInfo->videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, the current rate control mode is not VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR or VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, and VkVideoEncodeH264CapabilitiesKHR::requiresGopRemainingFrames is VK_TRUE, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the pBeginInfo->videoSession was created with, then the pNext chain of pBeginInfo **must** include an instance of the VkVideoEncodeH264GopRemainingFrameInfoKHR with its useGopRemainingFrames member set to VK_TRUE
- VUID-vkCmdBeginVideoCodingKHR-pBeginInfo-08256
  If pBeginInfo->videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, the current rate control mode is not VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR or VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, and VkVideoEncodeH265CapabilitiesKHR::requiresGopRemainingFrames is VK_TRUE, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the pBeginInfo->videoSession was created with, then the pNext chain of pBeginInfo **must** include an instance of the VkVideoEncodeH265GopRemainingFrameInfoKHR with its useGopRemainingFrames member set to VK_TRUE
- VUID-vkCmdBeginVideoCodingKHR-pBeginInfo-10282
  If pBeginInfo->videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, the current rate control mode is not VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR or VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, and VkVideoEncodeAV1CapabilitiesKHR::requiresGopRemainingFrames is VK_TRUE, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the pBeginInfo->videoSession was created with, then the pNext chain of pBeginInfo **must** include an instance of the VkVideoEncodeAV1GopRemainingFrameInfoKHR with its

`useGopRemainingFrames` member set to `VK_TRUE`

Valid Usage (Implicit)

- VUID-vkCmdBeginVideoCodingKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBeginVideoCodingKHR-pBeginInfo-parameter
  `pBeginInfo` **must** be a valid pointer to a valid VkVideoBeginCodingInfoKHR structure
- VUID-vkCmdBeginVideoCodingKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBeginVideoCodingKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support decode, or encode operations
- VUID-vkCmdBeginVideoCodingKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdBeginVideoCodingKHR-videocoding
  This command **must** only be called outside of a video coding scope
- VUID-vkCmdBeginVideoCodingKHR-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Outside	Outside	Decode Encode	Action State

The VkVideoBeginCodingInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoBeginCodingInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkVideoBeginCodingFlagsKHR            flags;
    VkVideoSessionKHR                     videoSession;
    VkVideoSessionParametersKHR           videoSessionParameters;
    uint32_t                              referenceSlotCount;
    const VkVideoReferenceSlotInfoKHR*    pReferenceSlots;
} VkVideoBeginCodingInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `videoSession` is the video session object to be bound for the processing of the video commands.
- `videoSessionParameters` is `VK_NULL_HANDLE` or a handle of a VkVideoSessionParametersKHR object to be used for the processing of the video commands. If `VK_NULL_HANDLE`, then no video session parameters object is bound for the duration of the video coding scope.
- `referenceSlotCount` is the number of elements in the `pReferenceSlots` array.
- `pReferenceSlots` is a pointer to an array of VkVideoReferenceSlotInfoKHR structures specifying the information used to determine the set of bound reference picture resources for the video coding scope and their initial association with DPB slot indices.

Limiting values are defined below that are referenced by the relevant valid usage statements of this structure.

- Let `VkOffset2D codedOffsetGranularity` be the minimum alignment requirement for the coded offset of video picture resources. Unless otherwise defined, the value of the `x` and `y` members of `codedOffsetGranularity` are `0`.
  - If `videoSession` was created with an H.264 decode profile with a VkVideoDecodeH264ProfileInfoKHR::`pictureLayout` of `VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR`, then `codedOffsetGranularity` is equal to VkVideoDecodeH264CapabilitiesKHR::`fieldOffsetGranularity`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for that video profile.

Valid Usage

- VUID-VkVideoBeginCodingInfoKHR-videoSession-07237
  `videoSession` **must** have memory bound to all of its memory bindings returned by vkGetVideoSessionMemoryRequirementsKHR for `videoSession`
- VUID-VkVideoBeginCodingInfoKHR-slotIndex-04856
  Each non-negative VkVideoReferenceSlotInfoKHR::`slotIndex` specified in the elements of `pReferenceSlots` **must** be less than the VkVideoSessionCreateInfoKHR::`maxDpbSlots`

specified when videoSession was created

- VUID-VkVideoBeginCodingInfoKHR-pPictureResource-07238
  Each video picture resource corresponding to any non-NULL pPictureResource member specified in the elements of pReferenceSlots **must** be unique within pReferenceSlots
- VUID-VkVideoBeginCodingInfoKHR-pPictureResource-07240
  If the pPictureResource member of any element of pReferenceSlots is not NULL, then the image view specified in pPictureResource->imageViewBinding for that element **must** be compatible with the video profile videoSession was created with
- VUID-VkVideoBeginCodingInfoKHR-pPictureResource-07241
  If the pPictureResource member of any element of pReferenceSlots is not NULL, then the format of the image view specified in pPictureResource->imageViewBinding for that element **must** match the VkVideoSessionCreateInfoKHR::referencePictureFormat videoSession was created with
- VUID-VkVideoBeginCodingInfoKHR-pPictureResource-07242
  If the pPictureResource member of any element of pReferenceSlots is not NULL, then its codedOffset member **must** be an integer multiple of codedOffsetGranularity
- VUID-VkVideoBeginCodingInfoKHR-pPictureResource-07243
  If the pPictureResource member of any element of pReferenceSlots is not NULL, then its codedExtent member **must** be between minCodedExtent and maxCodedExtent, inclusive, videoSession was created with
- VUID-VkVideoBeginCodingInfoKHR-flags-07244
  If VkVideoCapabilitiesKHR::flags does not include VK_VIDEO_CAPABILITY_SEPARATE_REFERENCE_IMAGES_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile videoSession was created with, then pPictureResource->imageViewBinding of all elements of pReferenceSlots with a non-NULL pPictureResource member **must** specify image views created from the same image
- VUID-VkVideoBeginCodingInfoKHR-slotIndex-07245
  If videoSession was created with a decode operation and the slotIndex member of any element of pReferenceSlots is not negative, then the image view specified in pPictureResource->imageViewBinding for that element **must** have been created with VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR
- VUID-VkVideoBeginCodingInfoKHR-slotIndex-07246
  If videoSession was created with an encode operation and the slotIndex member of any element of pReferenceSlots is not negative, then the image view specified in pPictureResource->imageViewBinding for that element **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR
- VUID-VkVideoBeginCodingInfoKHR-videoSession-07247
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then videoSessionParameters **must** not be VK_NULL_HANDLE
- VUID-VkVideoBeginCodingInfoKHR-videoSession-07248
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then videoSessionParameters **must** not be

VK_NULL_HANDLE

- VUID-VkVideoBeginCodingInfoKHR-videoSession-09261
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then videoSessionParameters **must** not be VK_NULL_HANDLE
- VUID-VkVideoBeginCodingInfoKHR-videoSession-07249
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then videoSessionParameters **must** not be VK_NULL_HANDLE
- VUID-VkVideoBeginCodingInfoKHR-videoSession-07250
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then videoSessionParameters **must** not be VK_NULL_HANDLE
- VUID-VkVideoBeginCodingInfoKHR-videoSession-10283
  If videoSession was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then videoSessionParameters **must** not be VK_NULL_HANDLE
- VUID-VkVideoBeginCodingInfoKHR-videoSessionParameters-04857
  If videoSessionParameters is not VK_NULL_HANDLE, it **must** have been created with videoSession specified in VkVideoSessionParametersCreateInfoKHR::videoSession

Valid Usage (Implicit)

- VUID-VkVideoBeginCodingInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_BEGIN_CODING_INFO_KHR
- VUID-VkVideoBeginCodingInfoKHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkVideoEncodeAV1GopRemainingFrameInfoKHR, VkVideoEncodeAV1RateControlInfoKHR, VkVideoEncodeH264GopRemainingFrameInfoKHR, VkVideoEncodeH264RateControlInfoKHR, VkVideoEncodeH265GopRemainingFrameInfoKHR, VkVideoEncodeH265RateControlInfoKHR, or VkVideoEncodeRateControlInfoKHR
- VUID-VkVideoBeginCodingInfoKHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkVideoBeginCodingInfoKHR-flags-zerobitmask
  flags **must** be 0
- VUID-VkVideoBeginCodingInfoKHR-videoSession-parameter
  videoSession **must** be a valid VkVideoSessionKHR handle
- VUID-VkVideoBeginCodingInfoKHR-videoSessionParameters-parameter
  If videoSessionParameters is not VK_NULL_HANDLE, videoSessionParameters **must** be a valid VkVideoSessionParametersKHR handle

- VUID-VkVideoBeginCodingInfoKHR-pReferenceSlots-parameter
  If `referenceSlotCount` is not `0`, `pReferenceSlots` **must** be a valid pointer to an array of `referenceSlotCount` valid VkVideoReferenceSlotInfoKHR structures
- VUID-VkVideoBeginCodingInfoKHR-videoSessionParameters-parent
  If `videoSessionParameters` is a valid handle, it **must** have been created, allocated, or retrieved from `videoSession`
- VUID-VkVideoBeginCodingInfoKHR-commonparent
  Both of `videoSession`, and `videoSessionParameters` that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoBeginCodingFlagsKHR;
```

`VkVideoBeginCodingFlagsKHR` is a bitmask type for setting a mask, but is currently reserved for future use.

The `VkVideoReferenceSlotInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoReferenceSlotInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    int32_t                                  slotIndex;
    const VkVideoPictureResourceInfoKHR*     pPictureResource;
} VkVideoReferenceSlotInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `slotIndex` is the index of the DPB slot or a negative integer value.
- `pPictureResource` is `NULL` or a pointer to a VkVideoPictureResourceInfoKHR structure describing the video picture resource associated with the DPB slot index specified by `slotIndex`.

Valid Usage (Implicit)

- VUID-VkVideoReferenceSlotInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_REFERENCE_SLOT_INFO_KHR`
- VUID-VkVideoReferenceSlotInfoKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkVideoDecodeAV1DpbSlotInfoKHR, VkVideoDecodeH264DpbSlotInfoKHR, VkVideoDecodeH265DpbSlotInfoKHR, VkVideoEncodeAV1DpbSlotInfoKHR, VkVideoEncodeH264DpbSlotInfoKHR, or VkVideoEncodeH265DpbSlotInfoKHR
- VUID-VkVideoReferenceSlotInfoKHR-sType-unique

The `sType` value of each struct in the `pNext` chain **must** be unique

- VUID-VkVideoReferenceSlotInfoKHR-pPictureResource-parameter
  If `pPictureResource` is not `NULL`, `pPictureResource` **must** be a valid pointer to a valid VkVideoPictureResourceInfoKHR structure

To end a video coding scope, call:

```
// Provided by VK_KHR_video_queue
void vkCmdEndVideoCodingKHR(
    VkCommandBuffer                             commandBuffer,
    const VkVideoEndCodingInfoKHR*              pEndCodingInfo);
```

- `commandBuffer` is the command buffer in which to record the command.
- `pEndCodingInfo` is a pointer to a VkVideoEndCodingInfoKHR structure specifying the parameters for ending the video coding scope.

After ending a video coding scope, the video session object, the optional video session parameters object, and all reference picture resources previously bound by the corresponding vkCmdBeginVideoCodingKHR command are *unbound*.

Valid Usage

- VUID-vkCmdEndVideoCodingKHR-None-07251
  There **must** be no active queries

Valid Usage (Implicit)

- VUID-vkCmdEndVideoCodingKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdEndVideoCodingKHR-pEndCodingInfo-parameter
  `pEndCodingInfo` **must** be a valid pointer to a valid VkVideoEndCodingInfoKHR structure
- VUID-vkCmdEndVideoCodingKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdEndVideoCodingKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support decode, or encode operations
- VUID-vkCmdEndVideoCodingKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdEndVideoCodingKHR-videocoding
  This command **must** only be called inside of a video coding scope
- VUID-vkCmdEndVideoCodingKHR-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Outside	Inside	Decode Encode	Action State

The `VkVideoEndCodingInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoEndCodingInfoKHR {
    VkStructureType             sType;
    const void*                 pNext;
    VkVideoEndCodingFlagsKHR    flags;
} VkVideoEndCodingInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.

Valid Usage (Implicit)

- VUID-VkVideoEndCodingInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_END_CODING_INFO_KHR`
- VUID-VkVideoEndCodingInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkVideoEndCodingInfoKHR-flags-zerobitmask
  `flags` **must** be `0`

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoEndCodingFlagsKHR;
```

`VkVideoEndCodingFlagsKHR` is a bitmask type for setting a mask, but is currently reserved for future use.

38.9. Video Coding Control

To apply dynamic controls to the bound video session object, call:

```
// Provided by VK_KHR_video_queue
void vkCmdControlVideoCodingKHR(
    VkCommandBuffer                             commandBuffer,
    const VkVideoCodingControlInfoKHR*          pCodingControlInfo);
```

- `commandBuffer` is the command buffer in which to record the command.
- `pCodingControlInfo` is a pointer to a VkVideoCodingControlInfoKHR structure specifying the control parameters.

The control parameters provided in this call are applied to the video session at the time the command executes on the device and are in effect until a subsequent call to this command with the same video session bound changes the corresponding control parameters.

A newly created video session **must** be reset before performing video coding operations using it by including `VK_VIDEO_CODING_CONTROL_RESET_BIT_KHR` in `pCodingControlInfo->flags`. The reset operation also returns all DPB slots of the video session to the inactive state. Correspondingly, any DPB slot index associated with the bound reference picture resources is removed.

For encode sessions, the reset operation returns rate control configuration to implementation default settings and sets the video encode quality level to zero.

After video coding operations are performed using a video session, the reset operation **can** be used to return the video session to the same *initial* state as after the reset of a newly created video session. This **can** be used, for example, when different video sequences are needed to be processed with the same video session object.

If `pCodingControlInfo->flags` includes `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR`, then the command replaces the rate control configuration maintained by the video session with the configuration specified in the VkVideoEncodeRateControlInfoKHR structure included in the `pCodingControlInfo->pNext` chain.

If `pCodingControlInfo->flags` includes `VK_VIDEO_CODING_CONTROL_ENCODE_QUALITY_LEVEL_BIT_KHR`, then the command changes the current video encode quality level to the value specified in the `qualityLevel` member of the VkVideoEncodeQualityLevelInfoKHR structure included in the `pCodingControlInfo->pNext` chain.

Valid Usage

- VUID-vkCmdControlVideoCodingKHR-flags-07017
  If `pCodingControlInfo->flags` does not include `VK_VIDEO_CODING_CONTROL_RESET_BIT_KHR`, then the bound video session **must** not be in uninitialized state at the time the command is executed on the device
- VUID-vkCmdControlVideoCodingKHR-pCodingControlInfo-08243

If the bound video session was not created with an encode operation, then `pCodingControlInfo->flags` **must** not include `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR` or `VK_VIDEO_CODING_CONTROL_ENCODE_QUALITY_LEVEL_BIT_KHR`

Valid Usage (Implicit)

- VUID-vkCmdControlVideoCodingKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdControlVideoCodingKHR-pCodingControlInfo-parameter
  `pCodingControlInfo` **must** be a valid pointer to a valid VkVideoCodingControlInfoKHR structure
- VUID-vkCmdControlVideoCodingKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdControlVideoCodingKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support decode, or encode operations
- VUID-vkCmdControlVideoCodingKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdControlVideoCodingKHR-videocoding
  This command **must** only be called inside of a video coding scope
- VUID-vkCmdControlVideoCodingKHR-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Outside	Inside	Decode Encode	Action

The `VkVideoCodingControlInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_queue
typedef struct VkVideoCodingControlInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkVideoCodingControlFlagsKHR    flags;
} VkVideoCodingControlInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkVideoCodingControlFlagsKHR specifying control flags.

Valid Usage

- VUID-VkVideoCodingControlInfoKHR-flags-07018
  If `flags` includes `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR`, then the `pNext` chain **must** include a VkVideoEncodeRateControlInfoKHR structure
- VUID-VkVideoCodingControlInfoKHR-flags-08349
  If `flags` includes `VK_VIDEO_CODING_CONTROL_ENCODE_QUALITY_LEVEL_BIT_KHR`, then the `pNext` chain **must** include a VkVideoEncodeQualityLevelInfoKHR structure

Valid Usage (Implicit)

- VUID-VkVideoCodingControlInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_CODING_CONTROL_INFO_KHR`
- VUID-VkVideoCodingControlInfoKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkVideoEncodeAV1RateControlInfoKHR, VkVideoEncodeH264RateControlInfoKHR, VkVideoEncodeH265RateControlInfoKHR, VkVideoEncodeQualityLevelInfoKHR, or VkVideoEncodeRateControlInfoKHR
- VUID-VkVideoCodingControlInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkVideoCodingControlInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkVideoCodingControlFlagBitsKHR values
- VUID-VkVideoCodingControlInfoKHR-flags-requiredbitmask
  `flags` **must** not be `0`

Bits which **can** be set in VkVideoCodingControlInfoKHR::`flags`, specifying the video coding control parameters to be modified, are:

```
// Provided by VK_KHR_video_queue
typedef enum VkVideoCodingControlFlagBitsKHR {
    VK_VIDEO_CODING_CONTROL_RESET_BIT_KHR = 0x00000001,
  // Provided by VK_KHR_video_encode_queue
    VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR = 0x00000002,
  // Provided by VK_KHR_video_encode_queue
    VK_VIDEO_CODING_CONTROL_ENCODE_QUALITY_LEVEL_BIT_KHR = 0x00000004,
} VkVideoCodingControlFlagBitsKHR;
```

- VK_VIDEO_CODING_CONTROL_RESET_BIT_KHR specifies a request for the bound video session to be reset before other coding control parameters are applied.
- VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR specifies that the coding control parameters include video encode rate control parameters (see VkVideoEncodeRateControlInfoKHR).
- VK_VIDEO_CODING_CONTROL_ENCODE_QUALITY_LEVEL_BIT_KHR specifies that the coding control parameters include video encode quality level parameters (see VkVideoEncodeQualityLevelInfoKHR).

```
// Provided by VK_KHR_video_queue
typedef VkFlags VkVideoCodingControlFlagsKHR;
```

VkVideoCodingControlFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoCodingControlFlagBitsKHR.

38.10. Inline Queries

If a video session was created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, beginning queries using commands such as vkCmdBeginQuery within a video coding scope is not allowed. Instead, queries are executed inline by including an instance of the VkVideoInlineQueryInfoKHR structure in the pNext chain of the parameters of one of the video coding commands, with its queryPool member set to a valid VkQueryPool handle.

The VkVideoInlineQueryInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_maintenance1
typedef struct VkVideoInlineQueryInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkQueryPool        queryPool;
    uint32_t           firstQuery;
    uint32_t           queryCount;
} VkVideoInlineQueryInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- `queryPool` is `VK_NULL_HANDLE` or a valid handle to a VkQueryPool object that will manage the results of the queries.
- `firstQuery` is the query index within the query pool that will contain the query results for the first video coding operation. The query results of subsequent video coding operations will be contained by subsequent query indices.
- `queryCount` is the number of queries to execute.

NOTE	In practice, if `queryPool` is not `VK_NULL_HANDLE`, then `queryCount` will always have to match the number of video coding operations issued by the video coding command this structure is specified to, meaning that using inline queries in a video coding command will always execute a query for each issued video coding operation.

This structure **can** be included in the `pNext` chain of the input parameter structure of video coding commands.

- In the `pNext` chain of the `pDecodeInfo` parameter of the vkCmdDecodeVideoKHR command to execute a query for each video decode operation issued by the command.
- In the `pNext` chain of the `pEncodeInfo` parameter of the vkCmdEncodeVideoKHR command to execute a query for each video encode operation issued by the command.

Valid Usage

- VUID-VkVideoInlineQueryInfoKHR-queryPool-08372
  If `queryPool` is not `VK_NULL_HANDLE`, then `firstQuery` **must** be less than the number of queries in `queryPool`
- VUID-VkVideoInlineQueryInfoKHR-queryPool-08373
  If `queryPool` is not `VK_NULL_HANDLE`, then the sum of `firstQuery` and `queryCount` **must** be less than or equal to the number of queries in `queryPool`

Valid Usage (Implicit)

- VUID-VkVideoInlineQueryInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_INLINE_QUERY_INFO_KHR`
- VUID-VkVideoInlineQueryInfoKHR-queryPool-parameter
  If `queryPool` is not VK_NULL_HANDLE, `queryPool` **must** be a valid VkQueryPool handle

38.11. Video Decode Operations

Video decode operations consume compressed video data from a video bitstream buffer and zero or more reference pictures, and produce a *decode output picture* and an optional reconstructed picture.

NOTE	Such decode output pictures can be shared with the Decoded Picture Buffer, and can also be used as the input of video encode operations, with graphics or compute operations, or with Window System Integration APIs, depending on the capabilities of the implementation.

Video decode operations **may** access the following resources in the VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR stage:

- The source video bitstream buffer range and the image subregions corresponding to the list of active reference pictures with access VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR.
- The image subregions corresponding to the target decode output picture and reconstructed picture with access VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR.

The image subresource of each video picture resource accessed by the video decode operation is specified using a corresponding VkVideoPictureResourceInfoKHR structure. Each such image subresource **must** be in the appropriate image layout as follows:

- If the image subresource is used in the video decode operation only as decode output picture, then it **must** be in the VK_IMAGE_LAYOUT_VIDEO_DECODE_DST_KHR layout.
- If the image subresource is used in the video decode operation both as decode output picture and reconstructed picture, then it **must** be in the VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR layout.
- If the image subresource is used in the video decode operation only as reconstructed picture, then it **must** be in the VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR layout.
- If the image subresource is used in the video decode operation as a reference picture, then it **must** be in the VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR layout.

A video decode operation **may** complete unsuccessfully. In this case the decode output picture will have undefined contents. Similarly, if reference picture setup is requested, the reconstructed picture will also have undefined contents, and the activated DPB slot will have an invalid picture reference.

38.11.1. Codec-Specific Semantics

The following aspects of video decode operations are codec-specific:

- The interpretation of the contents of the source video bitstream buffer range.
- The construction and interpretation of the list of active reference pictures and the interpretation of the picture data referred to by the corresponding image subregions.
- The construction and interpretation of information related to the decode output picture and the generation of picture data to the corresponding image subregion.
- The decision on reference picture setup.
- The construction and interpretation of information related to the optional reconstructed picture and the generation of picture data to the corresponding image subregion.

These codec-specific behaviors are defined for each video codec operation separately.

- If the used video codec operation is VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the codec-specific aspects of the video decoding process are performed as defined in the H.264 Decode Operations section.
- If the used video codec operation is VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the codec-specific aspects of the video decoding process are performed as defined in the H.265 Decode Operations section.
- If the used video codec operation is VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then the codec-specific aspects of the video decoding process are performed as defined in the AV1 Decode Operations section.

38.11.2. Video Decode Operation Steps

Each video decode operation performs the following steps in the VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR stage:

1. Reads the encoded video data from the source video bitstream buffer range.
2. Performs picture reconstruction of the encoded video data according to the codec-specific semantics, applying any prediction data read from the active reference pictures in the process;
3. Writes the decoded picture data to the decode output picture, and optionally to the reconstructed picture, if one is specified and is different from the decode output picture, according to the codec-specific semantics;
4. If reference picture setup is requested, the DPB slot index specified in the reconstructed picture information is activated with the reconstructed picture.

When reconstructed picture information is provided, the specified DPB slot index is associated with the corresponding bound reference picture resource, indifferent of whether reference picture setup is requested.

38.11.3. Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR with pVideoProfile->videoCodecOperation specifying a decode operation, the VkVideoDecodeCapabilitiesKHR structure **must** be included in the pNext chain of the VkVideoCapabilitiesKHR structure to retrieve capabilities specific to video decoding.

The VkVideoDecodeCapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_queue
typedef struct VkVideoDecodeCapabilitiesKHR {
    VkStructureType                    sType;
    void*                              pNext;
    VkVideoDecodeCapabilityFlagsKHR    flags;
} VkVideoDecodeCapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- `flags` is a bitmask of VkVideoDecodeCapabilityFlagBitsKHR describing the supported video decoding capabilities.

Valid Usage (Implicit)

- VUID-VkVideoDecodeCapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_CAPABILITIES_KHR`

Bits which **may** be set in VkVideoDecodeCapabilitiesKHR::`flags`, indicating the decoding capabilities supported, are:

```
// Provided by VK_KHR_video_decode_queue
typedef enum VkVideoDecodeCapabilityFlagBitsKHR {
    VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR = 0x00000001,
    VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR = 0x00000002,
} VkVideoDecodeCapabilityFlagBitsKHR;
```

- `VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR` indicates support for using the same video picture resource as the reconstructed picture and decode output picture in a video decode operation.
- `VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR` indicates support for using distinct video picture resources as the reconstructed picture and decode output picture in a video decode operation.

NOTE Some video profiles allow using distinct video picture resources as the reconstructed picture and decode output picture in specific video decode operations even when the video decode profile does not support `VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR`. Even if the implementation only reports coincide, the decode output picture for film grain enabled frames must be a different video picture resource from the reconstructed picture because film grain is applied outside of the coding loop.

Implementations are only **required** to support one of `VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR` and `VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR`. Accordingly, applications **should** handle both cases to maximize portability.

NOTE If both `VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR` and `VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR` are supported, an application can choose to create separate images for decode DPB and decode output. E.g. in cases when linear tiling is preferred (and supported) for the decode output picture and the DPB requires optimal tiling, this avoids the need for a separate copy at the expense of additional memory bandwidth requirements during decoding.

```
// Provided by VK_KHR_video_decode_queue
typedef VkFlags VkVideoDecodeCapabilityFlagsKHR;
```

VkVideoDecodeCapabilityFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoDecodeCapabilityFlagBitsKHR.

38.11.4. Video Decode Commands

To launch video decode operations, call:

```
// Provided by VK_KHR_video_decode_queue
void vkCmdDecodeVideoKHR(
    VkCommandBuffer                             commandBuffer,
    const VkVideoDecodeInfoKHR*                 pDecodeInfo);
```

- commandBuffer is the command buffer in which to record the command.
- pDecodeInfo is a pointer to a VkVideoDecodeInfoKHR structure specifying the parameters of the video decode operations.

Each call issues one or more video decode operations. The implicit parameter opCount corresponds to the number of video decode operations issued by the command. After calling this command, the active query index of each active query is incremented by opCount.

Currently each call to this command results in the issue of a single video decode operation.

If the bound video session was created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR and the pNext chain of pDecodeInfo includes a VkVideoInlineQueryInfoKHR structure with its queryPool member specifying a valid VkQueryPool handle, then this command will execute a query for each video decode operation issued by it.

Active Reference Picture Information

The list of active reference pictures used by a video decode operation is a list of image subregions used as the source of reference picture data and related parameters, and is derived from the VkVideoReferenceSlotInfoKHR structures provided as the elements of the pDecodeInfo->pReferenceSlots array. For each element of pDecodeInfo->pReferenceSlots, one or more elements are added to the active reference picture list, as defined by the codec-specific semantics. Each element of this list contains the following information:

- The image subregion within the image subresource referred to by the video picture resource used as the reference picture.
- The DPB slot index the reference picture is associated with.
- The codec-specific reference information related to the reference picture.

Reconstructed Picture Information

Information related to the optional reconstructed picture used by a video decode operation is derived from the VkVideoReferenceSlotInfoKHR structure pointed to by pDecodeInfo-

>pSetupReferenceSlot, if not NULL, as defined by the codec-specific semantics, and consists of the following:

- The image subregion within the image subresource referred to by the video picture resource used as the reconstructed picture.
- The DPB slot index to use for picture reconstruction.
- The codec-specific reference information related to the reconstructed picture.

Specifying a valid VkVideoReferenceSlotInfoKHR structure in pDecodeInfo->pSetupReferenceSlot is always required, unless the video session was created with VkVideoSessionCreateInfoKHR::maxDpbSlot equal to zero. However, the DPB slot identified by pDecodeInfo->pSetupReferenceSlot->slotIndex is only activated with the reconstructed picture specified in pDecodeInfo->pSetupReferenceSlot->pPictureResource if reference picture setup is requested according to the codec-specific semantics.

If reconstructed picture information is specified, and pDecodeInfo->pSetupReferenceSlot->pPictureResource refers to a video picture resource different than that of the decode output picture, but reference picture setup is not requested, the contents of the video picture resource corresponding to the reconstructed picture will be undefined after the video decode operation.

NOTE	Some implementations may always output the reconstructed picture or use it as temporary storage during the video decode operation even when the reconstructed picture is not marked for future reference.

Decode Output Picture Information

Information related to the decode output picture used by a video decode operation is derived from pDecodeInfo->dstPictureResource and any codec-specific parameters provided in the pDecodeInfo->pNext chain, as defined by the codec-specific semantics, and consists of the following:

- The image subregion within the image subresource referred to by the video picture resource used as the decode output picture.
- The codec-specific picture information related to the decode output picture.

Several limiting values are defined below that are referenced by the relevant valid usage statements of this command.

- Let uint32_t activeReferencePictureCount be the size of the list of active reference pictures used by the video decode operation. Unless otherwise defined, activeReferencePictureCount is set to the value of pDecodeInfo->referenceSlotCount.
  - If the bound video session was created with an H.264 decode profile, then let activeReferencePictureCount be the value of pDecodeInfo->referenceSlotCount plus the number of elements of the pDecodeInfo->pReferenceSlots array that have a VkVideoDecodeH264DpbSlotInfoKHR structure included in their pNext chain with both pStdReferenceInfo->flags.top_field_flag and pStdReferenceInfo->flags.bottom_field_flag set.

NOTE	This means that the elements of pDecodeInfo->pReferenceSlots that include both a top and bottom field reference are counted as two separate active reference pictures, as described in the active reference picture list construction rules for H.264 decode operations.

- Let VkOffset2D codedOffsetGranularity be the minimum alignment requirement for the coded offset of video picture resources. Unless otherwise defined, the value of the x and y members of codedOffsetGranularity are 0.
  - If the bound video session was created with an H.264 decode profile with a VkVideoDecodeH264ProfileInfoKHR::pictureLayout of VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR, then codedOffsetGranularity is equal to VkVideoDecodeH264CapabilitiesKHR::fieldOffsetGranularity, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for that video profile.
- Let uint32_t dpbFrameUseCount[] be an array of size maxDpbSlots, where maxDpbSlots is the VkVideoSessionCreateInfoKHR::maxDpbSlots the bound video session was created with, with each element indicating the number of times a frame associated with the corresponding DPB slot index is referred to by the video coding operation. Let the initial value of each element of the array be 0.
  - If pDecodeInfo->pSetupReferenceSlot is not NULL, then dpbFrameUseCount[i] is incremented by one, where i equals pDecodeInfo->pSetupReferenceSlot->slotIndex. If the bound video session object was created with an H.264 decode profile, then dpbFrameUseCount[i] is decremented by one if either pStdReferenceInfo->flags.top_field_flag or pStdReferenceInfo->flags.bottom_field_flag is set in the VkVideoDecodeH264DpbSlotInfoKHR structure in the pDecodeInfo->pSetupReferenceSlot->pNext chain.
  - For each element of pDecodeInfo->pReferenceSlots, dpbFrameUseCount[i] is incremented by one, where i equals the slotIndex member of the corresponding element. If the bound video session object was created with an H.264 decode profile, then dpbFrameUseCount[i] is decremented by one if either pStdReferenceInfo->flags.top_field_flag or pStdReferenceInfo->flags.bottom_field_flag is set in the VkVideoDecodeH264DpbSlotInfoKHR structure in the pNext chain of the corresponding element of pDecodeInfo->pReferenceSlots.
- Let uint32_t dpbTopFieldUseCount[] and uint32_t dpbBottomFieldUseCount[] be arrays of size maxDpbSlots, where maxDpbSlots is the VkVideoSessionCreateInfoKHR::maxDpbSlots the bound video session was created with, with each element indicating the number of times the top field or the bottom field, respectively, associated with the corresponding DPB slot index is referred to by the video coding operation. Let the initial value of each element of the arrays be 0.
  - If the bound video session object was created with an H.264 decode profile and pDecodeInfo->pSetupReferenceSlot is not NULL, then perform the following:
    - If pStdReferenceInfo->flags.top_field_flag is set in the VkVideoDecodeH264DpbSlotInfoKHR structure in the pDecodeInfo->pSetupReferenceSlot->pNext chain, then dpbTopFieldUseCount[i] is incremented by one, where i equals pDecodeInfo->pSetupReferenceSlot->slotIndex.

        - If `pStdReferenceInfo->flags.bottom_field_flag` is set in the VkVideoDecodeH264DpbSlotInfoKHR structure in the `pDecodeInfo->pSetupReferenceSlot->pNext` chain, then `dpbBottomFieldUseCount[i]` is incremented by one, where `i` equals `pDecodeInfo->pSetupReferenceSlot->slotIndex`.
    - If the bound video session object was created with an H.264 decode profile, then perform the following for each element of `pDecodeInfo->pReferenceSlots`:
        - If `pStdReferenceInfo->flags.top_field_flag` is set in the VkVideoDecodeH264DpbSlotInfoKHR structure in the `pNext` chain of the element, then `dpbTopFieldUseCount[i]` is incremented by one, where `i` equals the `slotIndex` member of the element.
        - If `pStdReferenceInfo->flags.bottom_field_flag` is set in the VkVideoDecodeH264DpbSlotInfoKHR structure in the `pNext` chain of the element, then `dpbBottomFieldUseCount[i]` is incremented by one, where `i` equals the `slotIndex` member of the element.

Valid Usage

- VUID-vkCmdDecodeVideoKHR-None-08249
  The bound video session **must** have been created with a decode operation
- VUID-vkCmdDecodeVideoKHR-None-07011
  The bound video session **must** not be in uninitialized state at the time the command is executed on the device
- VUID-vkCmdDecodeVideoKHR-opCount-07134
  For each active query, the active query index corresponding to the query type of that query plus `opCount` **must** be less than or equal to the last activatable query index corresponding to the query type of that query plus one
- VUID-vkCmdDecodeVideoKHR-pNext-08365
  If the bound video session was created with `VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR`, and the `pNext` chain of `pDecodeInfo` includes a VkVideoInlineQueryInfoKHR structure with its `queryPool` member specifying a valid `VkQueryPool` handle, then VkVideoInlineQueryInfoKHR::queryCount **must** equal `opCount`
- VUID-vkCmdDecodeVideoKHR-pNext-08366
  If the bound video session was created with `VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR`, and the `pNext` chain of `pDecodeInfo` includes a VkVideoInlineQueryInfoKHR structure with its `queryPool` member specifying a valid `VkQueryPool` handle, then all the queries used by the command, as specified by the VkVideoInlineQueryInfoKHR structure, **must** be *unavailable*
- VUID-vkCmdDecodeVideoKHR-queryType-08367
  If the bound video session was created with `VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR`, then the `queryType` used to create the `queryPool` specified in the VkVideoInlineQueryInfoKHR structure included in the `pNext` chain of `pDecodeInfo` **must** be `VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR`
- VUID-vkCmdDecodeVideoKHR-queryPool-08368

If the bound video session was created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, then the queryPool specified in the VkVideoInlineQueryInfoKHR structure included in the pNext chain of pDecodeInfo **must** have been created with a VkVideoProfileInfoKHR structure included in the pNext chain of VkQueryPoolCreateInfo identical to the one specified in VkVideoSessionCreateInfoKHR ::pVideoProfile the bound video session was created with

- VUID-vkCmdDecodeVideoKHR-queryType-08369
  If the bound video session was created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, and the queryType used to create the queryPool specified in the VkVideoInlineQueryInfoKHR structure included in the pNext chain of pDecodeInfo is VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR, then the VkCommandPool that commandBuffer was allocated from **must** have been created with a queue family index that supports result status queries, as indicated by VkQueueFamilyQueryResultStatusPropertiesKHR::queryResultStatusSupport
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07135
  pDecodeInfo->srcBuffer **must** be compatible with the video profile the bound video session was created with
- VUID-vkCmdDecodeVideoKHR-commandBuffer-07136
  If commandBuffer is an unprotected command buffer and protectedNoFault is not supported, then pDecodeInfo->srcBuffer **must** not be a protected buffer
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07138
  pDecodeInfo->srcBufferOffset **must** be an integer multiple of VkVideoCapabilitiesKHR ::minBitstreamBufferOffsetAlignment, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07139
  pDecodeInfo->srcBufferRange **must** be an integer multiple of VkVideoCapabilitiesKHR ::minBitstreamBufferSizeAlignment, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07140
  If pDecodeInfo->pSetupReferenceSlot is not NULL and VkVideoDecodeCapabilitiesKHR::flags does not include VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then the video picture resources specified by pDecodeInfo->dstPictureResource and pDecodeInfo->pSetupReferenceSlot->pPictureResource **must** not match
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07141
  If pDecodeInfo->pSetupReferenceSlot is not NULL and none of the following is true:
  - VkVideoDecodeCapabilitiesKHR::flags includes VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
  - the bound video session was created with the video codec operation

VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR and VkVideoDecodeAV1ProfileInfoKHR::filmGrainSupport set to VK_TRUE, and film grain is enabled for the decoded picture

then the video picture resources specified by pDecodeInfo->dstPictureResource and pDecodeInfo->pSetupReferenceSlot->pPictureResource **must** match

- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07142
  pDecodeInfo->dstPictureResource.imageViewBinding **must** be compatible with the video profile the bound video session was created with
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07143
  The format of pDecodeInfo->dstPictureResource.imageViewBinding **must** match the VkVideoSessionCreateInfoKHR::pictureFormat the bound video session was created with
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07144
  pDecodeInfo->dstPictureResource.codedOffset **must** be an integer multiple of codedOffsetGranularity
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07145
  pDecodeInfo->dstPictureResource.codedExtent **must** be between minCodedExtent and maxCodedExtent, inclusive, the bound video session was created with
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07146
  pDecodeInfo->dstPictureResource.imageViewBinding **must** have been created with VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR
- VUID-vkCmdDecodeVideoKHR-commandBuffer-07147
  If commandBuffer is an unprotected command buffer and protectedNoFault is not supported, then pDecodeInfo->dstPictureResource.imageViewBinding **must** not have been created from a protected image
- VUID-vkCmdDecodeVideoKHR-commandBuffer-07148
  If commandBuffer is a protected command buffer and protectedNoFault is not supported, then pDecodeInfo->dstPictureResource.imageViewBinding **must** have been created from a protected image
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-08376
  pDecodeInfo->pSetupReferenceSlot **must** not be NULL unless the bound video session was created with VkVideoSessionCreateInfoKHR::maxDpbSlots equal to zero
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07170
  If pDecodeInfo->pSetupReferenceSlot is not NULL, then pDecodeInfo->pSetupReferenceSlot->slotIndex **must** be less than the VkVideoSessionCreateInfoKHR::maxDpbSlots specified when the bound video session was created
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07173
  If pDecodeInfo->pSetupReferenceSlot is not NULL, then pDecodeInfo->pSetupReferenceSlot->pPictureResource->codedOffset **must** be an integer multiple of codedOffsetGranularity
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07149
  If pDecodeInfo->pSetupReferenceSlot is not NULL, then pDecodeInfo->pSetupReferenceSlot->pPictureResource **must** match one of the bound reference picture resource
- VUID-vkCmdDecodeVideoKHR-activeReferencePictureCount-07150
  activeReferencePictureCount **must** be less than or equal to the

VkVideoSessionCreateInfoKHR::maxActiveReferencePictures specified when the bound video session was created

- VUID-vkCmdDecodeVideoKHR-slotIndex-07256
  The slotIndex member of each element of pDecodeInfo->pReferenceSlots **must** be less than the VkVideoSessionCreateInfoKHR::maxDpbSlots specified when the bound video session was created
- VUID-vkCmdDecodeVideoKHR-codedOffset-07257
  The codedOffset member of the VkVideoPictureResourceInfoKHR structure pointed to by the pPictureResource member of each element of pDecodeInfo->pReferenceSlots **must** be an integer multiple of codedOffsetGranularity
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07151
  The pPictureResource member of each element of pDecodeInfo->pReferenceSlots **must** match one of the bound reference picture resource associated with the DPB slot index specified in the slotIndex member of that element
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07264
  Each video picture resource corresponding to the pPictureResource member specified in the elements of pDecodeInfo->pReferenceSlots **must** be unique within pDecodeInfo->pReferenceSlots
- VUID-vkCmdDecodeVideoKHR-dpbFrameUseCount-07176
  All elements of dpbFrameUseCount **must** be less than or equal to 1
- VUID-vkCmdDecodeVideoKHR-dpbTopFieldUseCount-07177
  All elements of dpbTopFieldUseCount **must** be less than or equal to 1
- VUID-vkCmdDecodeVideoKHR-dpbBottomFieldUseCount-07178
  All elements of dpbBottomFieldUseCount **must** be less than or equal to 1
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07252
  If pDecodeInfo->pSetupReferenceSlot is NULL or pDecodeInfo->pSetupReferenceSlot->pPictureResource does not refer to the same image subresource as pDecodeInfo->dstPictureResource, then the image subresource referred to by pDecodeInfo->dstPictureResource **must** be in the VK_IMAGE_LAYOUT_VIDEO_DECODE_DST_KHR layout at the time the video decode operation is executed on the device
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07253
  If pDecodeInfo->pSetupReferenceSlot is not NULL and pDecodeInfo->pSetupReferenceSlot->pPictureResource refers to the same image subresource as pDecodeInfo->dstPictureResource, then the image subresource referred to by pDecodeInfo->dstPictureResource **must** be in the VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR layout at the time the video decode operation is executed on the device
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07254
  If pDecodeInfo->pSetupReferenceSlot is not NULL, then the image subresource referred to by pDecodeInfo->pSetupReferenceSlot->pPictureResource **must** be in the VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR layout at the time the video decode operation is executed on the device
- VUID-vkCmdDecodeVideoKHR-pPictureResource-07255
  The image subresource referred to by the pPictureResource member of each element of

pDecodeInfo->pReferenceSlots **must** be in the VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR layout at the time the video decode operation is executed on the device

- VUID-vkCmdDecodeVideoKHR-pNext-07152
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the pNext chain of pDecodeInfo **must** include a VkVideoDecodeH264PictureInfoKHR structure
- VUID-vkCmdDecodeVideoKHR-None-07258
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR but was not created with interlaced frame support, then the decode output picture **must** represent a frame
- VUID-vkCmdDecodeVideoKHR-pSliceOffsets-07153
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then all elements of the pSliceOffsets member of the VkVideoDecodeH264PictureInfoKHR structure included in the pNext chain of pDecodeInfo **must** be less than pDecodeInfo->srcBufferRange
- VUID-vkCmdDecodeVideoKHR-StdVideoH264SequenceParameterSet-07154
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH264SequenceParameterSet entry with seq_parameter_set_id matching StdVideoDecodeH264PictureInfo::seq_parameter_set_id that is provided in the pStdPictureInfo member of the VkVideoDecodeH264PictureInfoKHR structure included in the pNext chain of pDecodeInfo
- VUID-vkCmdDecodeVideoKHR-StdVideoH264PictureParameterSet-07155
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH264PictureParameterSet entry with seq_parameter_set_id and pic_parameter_set_id matching StdVideoDecodeH264PictureInfo::seq_parameter_set_id and StdVideoDecodeH264PictureInfo::pic_parameter_set_id, respectively, that are provided in the pStdPictureInfo member of the VkVideoDecodeH264PictureInfoKHR structure included in the pNext chain of pDecodeInfo
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07156
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR and pDecodeInfo->pSetupReferenceSlot is not NULL, then the pNext chain of pDecodeInfo->pSetupReferenceSlot **must** include a VkVideoDecodeH264DpbSlotInfoKHR structure
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07259
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR but was not created with interlaced frame support, and pDecodeInfo->pSetupReferenceSlot is not NULL, then the reconstructed picture **must** represent a frame
- VUID-vkCmdDecodeVideoKHR-pNext-07157
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, then the pNext chain of each element of pDecodeInfo->pReferenceSlots **must** include a VkVideoDecodeH264DpbSlotInfoKHR structure

- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07260
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` but was not created with interlaced frame support, then each active reference picture corresponding to the elements of `pDecodeInfo->pReferenceSlots` **must** represent a frame
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07261
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, `pDecodeInfo->pSetupReferenceSlot` is not `NULL`, and the decode output picture represents a frame, then the reconstructed picture **must** also represent a frame
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07262
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, `pDecodeInfo->pSetupReferenceSlot` is not `NULL`, and the decode output picture represents a top field, then the reconstructed picture **must** also represent a top field
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07263
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, `pDecodeInfo->pSetupReferenceSlot` is not `NULL`, and the decode output picture represents a bottom field, then the reconstructed picture **must** also represent a bottom field
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07266
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` and an active reference picture corresponding to any element of `pDecodeInfo->pReferenceSlots` represents a frame, then the DPB slot index of the bound video session specified by the `slotIndex` member of that element **must** be currently associated with a frame picture matching the video picture resource specified by the `pPictureResource` member of the same element at the time the command is executed on the device
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07267
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` and an active reference picture corresponding to any element of `pDecodeInfo->pReferenceSlots` represents a top field, then the DPB slot index of the bound video session specified by the `slotIndex` member of that element **must** be currently associated with a top field picture matching the video picture resource specified by the `pPictureResource` member of the same element at the time the command is executed on the device
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07268
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR` and an active reference picture corresponding to any element of `pDecodeInfo->pReferenceSlots` represents a bottom field, then the DPB slot index of the bound video session specified by the `slotIndex` member of that element **must** be currently associated with a bottom field picture matching the video picture resource specified by the `pPictureResource` member of the same element at the time the command is executed on the device
- VUID-vkCmdDecodeVideoKHR-pNext-07158

If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the pNext chain of pDecodeInfo **must** include a VkVideoDecodeH265PictureInfoKHR structure

- VUID-vkCmdDecodeVideoKHR-pSliceSegmentOffsets-07159
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then all elements of the pSliceSegmentOffsets member of the VkVideoDecodeH265PictureInfoKHR structure included in the pNext chain of pDecodeInfo **must** be less than pDecodeInfo->srcBufferRange
- VUID-vkCmdDecodeVideoKHR-StdVideoH265VideoParameterSet-07160
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH265VideoParameterSet entry with vps_video_parameter_set_id matching StdVideoDecodeH265PictureInfo::sps_video_parameter_set_id that is provided in the pStdPictureInfo member of the VkVideoDecodeH265PictureInfoKHR structure included in the pNext chain of pDecodeInfo
- VUID-vkCmdDecodeVideoKHR-StdVideoH265SequenceParameterSet-07161
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH265SequenceParameterSet entry with sps_video_parameter_set_id and sps_seq_parameter_set_id matching StdVideoDecodeH265PictureInfo::sps_video_parameter_set_id and StdVideoDecodeH265PictureInfo::pps_seq_parameter_set_id, respectively, that are provided in the pStdPictureInfo member of the VkVideoDecodeH265PictureInfoKHR structure included in the pNext chain of pDecodeInfo
- VUID-vkCmdDecodeVideoKHR-StdVideoH265PictureParameterSet-07162
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH265PictureParameterSet entry with sps_video_parameter_set_id, pps_seq_parameter_set_id, and pps_pic_parameter_set_id matching StdVideoDecodeH265PictureInfo::sps_video_parameter_set_id, StdVideoDecodeH265PictureInfo::pps_seq_parameter_set_id, and StdVideoDecodeH265PictureInfo::pps_pic_parameter_set_id, respectively, that are provided in the pStdPictureInfo member of the VkVideoDecodeH265PictureInfoKHR structure included in the pNext chain of pDecodeInfo
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-07163
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR and pDecodeInfo->pSetupReferenceSlot is not NULL, then the pNext chain of pDecodeInfo->pSetupReferenceSlot **must** include a VkVideoDecodeH265DpbSlotInfoKHR structure
- VUID-vkCmdDecodeVideoKHR-pNext-07164
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, then the pNext chain of each element of pDecodeInfo->pReferenceSlots **must** include a VkVideoDecodeH265DpbSlotInfoKHR structure
- VUID-vkCmdDecodeVideoKHR-filmGrainSupport-09248

If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR and VkVideoDecodeAV1ProfileInfoKHR ::filmGrainSupport set to VK_FALSE, then film grain **must** not be enabled for the decoded picture

- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-09249
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, pDecodeInfo->pSetupReferenceSlot is not NULL, and film grain is enabled for the decoded picture, then the video picture resources specified by pDecodeInfo->dstPictureResource and pDecodeInfo->pSetupReferenceSlot->pPictureResource **must** not match
- VUID-vkCmdDecodeVideoKHR-pNext-09250
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then the pNext chain of pDecodeInfo **must** include a VkVideoDecodeAV1PictureInfoKHR structure
- VUID-vkCmdDecodeVideoKHR-frameHeaderOffset-09251
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then the frameHeaderOffset member of the VkVideoDecodeAV1PictureInfoKHR structure included in the pNext chain of pDecodeInfo **must** be less than the minimum of pDecodeInfo->srcBufferRange
- VUID-vkCmdDecodeVideoKHR-pTileOffsets-09253
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then all elements of the pTileOffsets member of the VkVideoDecodeAV1PictureInfoKHR structure included in the pNext chain of pDecodeInfo **must** be less than pDecodeInfo->srcBufferRange
- VUID-vkCmdDecodeVideoKHR-pTileOffsets-09252
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then for each element i of the pTileOffsets and pTileSizes members of the VkVideoDecodeAV1PictureInfoKHR structure included in the pNext chain of pDecodeInfo the sum of pTileOffsets[i] and pTileSizes[i] **must** be less than or equal to pDecodeInfo->srcBufferRange
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-09254
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR and pDecodeInfo->pSetupReferenceSlot is not NULL, then the pNext chain of pDecodeInfo->pSetupReferenceSlot **must** include a VkVideoDecodeAV1DpbSlotInfoKHR structure
- VUID-vkCmdDecodeVideoKHR-pNext-09255
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then the pNext chain of each element of pDecodeInfo->pReferenceSlots **must** include a VkVideoDecodeAV1DpbSlotInfoKHR structure
- VUID-vkCmdDecodeVideoKHR-referenceNameSlotIndices-09262
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then each element of the referenceNameSlotIndices array member of the VkVideoDecodeAV1PictureInfoKHR structure included in the pNext chain of pDecodeInfo **must** either be negative or **must**

equal the slotIndex member of one of the elements of pDecodeInfo->pReferenceSlots

- VUID-vkCmdDecodeVideoKHR-slotIndex-09263
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR, then the slotIndex member of each element of pDecodeInfo->pReferenceSlots **must** equal one of the elements of the referenceNameSlotIndices array member of the VkVideoDecodeAV1PictureInfoKHR structure included in the pNext chain of pDecodeInfo

Valid Usage (Implicit)

- VUID-vkCmdDecodeVideoKHR-commandBuffer-parameter
  commandBuffer **must** be a valid VkCommandBuffer handle
- VUID-vkCmdDecodeVideoKHR-pDecodeInfo-parameter
  pDecodeInfo **must** be a valid pointer to a valid VkVideoDecodeInfoKHR structure
- VUID-vkCmdDecodeVideoKHR-commandBuffer-recording
  commandBuffer **must** be in the recording state
- VUID-vkCmdDecodeVideoKHR-commandBuffer-cmdpool
  The VkCommandPool that commandBuffer was allocated from **must** support decode operations
- VUID-vkCmdDecodeVideoKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdDecodeVideoKHR-videocoding
  This command **must** only be called inside of a video coding scope
- VUID-vkCmdDecodeVideoKHR-bufferlevel
  commandBuffer **must** be a primary VkCommandBuffer

Host Synchronization

- Host access to commandBuffer **must** be externally synchronized
- Host access to the VkCommandPool that commandBuffer was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Outside	Inside	Decode	Action

The VkVideoDecodeInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_queue
typedef struct VkVideoDecodeInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    VkVideoDecodeFlagsKHR                    flags;
    VkBuffer                                 srcBuffer;
    VkDeviceSize                             srcBufferOffset;
    VkDeviceSize                             srcBufferRange;
    VkVideoPictureResourceInfoKHR            dstPictureResource;
    const VkVideoReferenceSlotInfoKHR*       pSetupReferenceSlot;
    uint32_t                                 referenceSlotCount;
    const VkVideoReferenceSlotInfoKHR*       pReferenceSlots;
} VkVideoDecodeInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is reserved for future use.
- `srcBuffer` is the source video bitstream buffer to read the encoded bitstream from.
- `srcBufferOffset` is the starting offset in bytes from the start of `srcBuffer` to read the encoded bitstream from.
- `srcBufferRange` is the size in bytes of the encoded bitstream to decode from `srcBuffer`, starting from `srcBufferOffset`.
- `dstPictureResource` is the video picture resource to use as the decode output picture.
- `pSetupReferenceSlot` is `NULL` or a pointer to a VkVideoReferenceSlotInfoKHR structure specifying the reconstructed picture information.
- `referenceSlotCount` is the number of elements in the `pReferenceSlots` array.
- `pReferenceSlots` is `NULL` or a pointer to an array of VkVideoReferenceSlotInfoKHR structures describing the DPB slots and corresponding reference picture resources to use in this video decode operation (the set of active reference pictures).

Valid Usage

- VUID-VkVideoDecodeInfoKHR-srcBuffer-07165
  `srcBuffer` **must** have been created with `VK_BUFFER_USAGE_VIDEO_DECODE_SRC_BIT_KHR` set
- VUID-VkVideoDecodeInfoKHR-srcBufferOffset-07166
  `srcBufferOffset` **must** be less than the size of `srcBuffer`
- VUID-VkVideoDecodeInfoKHR-srcBufferRange-07167
  `srcBufferRange` **must** be less than or equal to the size of `srcBuffer` minus `srcBufferOffset`
- VUID-VkVideoDecodeInfoKHR-pSetupReferenceSlot-07168
  If `pSetupReferenceSlot` is not `NULL`, then its `slotIndex` member **must** not be negative
- VUID-VkVideoDecodeInfoKHR-pSetupReferenceSlot-07169
  If `pSetupReferenceSlot` is not `NULL`, then its `pPictureResource` **must** not be `NULL`

- VUID-VkVideoDecodeInfoKHR-slotIndex-07171
  The `slotIndex` member of each element of `pReferenceSlots` **must** not be negative
- VUID-VkVideoDecodeInfoKHR-pPictureResource-07172
  The `pPictureResource` member of each element of `pReferenceSlots` **must** not be `NULL`

Valid Usage (Implicit)

- VUID-VkVideoDecodeInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_INFO_KHR`
- VUID-VkVideoDecodeInfoKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkVideoDecodeAV1PictureInfoKHR, VkVideoDecodeH264PictureInfoKHR, VkVideoDecodeH265PictureInfoKHR, or VkVideoInlineQueryInfoKHR
- VUID-VkVideoDecodeInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkVideoDecodeInfoKHR-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkVideoDecodeInfoKHR-srcBuffer-parameter
  `srcBuffer` **must** be a valid VkBuffer handle
- VUID-VkVideoDecodeInfoKHR-dstPictureResource-parameter
  `dstPictureResource` **must** be a valid VkVideoPictureResourceInfoKHR structure
- VUID-VkVideoDecodeInfoKHR-pSetupReferenceSlot-parameter
  If `pSetupReferenceSlot` is not `NULL`, `pSetupReferenceSlot` **must** be a valid pointer to a valid VkVideoReferenceSlotInfoKHR structure
- VUID-VkVideoDecodeInfoKHR-pReferenceSlots-parameter
  If `referenceSlotCount` is not `0`, `pReferenceSlots` **must** be a valid pointer to an array of `referenceSlotCount` valid VkVideoReferenceSlotInfoKHR structures

```
// Provided by VK_KHR_video_decode_queue
typedef VkFlags VkVideoDecodeFlagsKHR;
```

`VkVideoDecodeFlagsKHR` is a bitmask type for setting a mask, but is currently reserved for future use.

38.12. H.264 Decode Operations

Video decode operations using an H.264 decode profile **can** be used to decode elementary video stream sequences compliant to the ITU-T H.264 Specification.

NOTE Refer to the Preamble for information on how the Khronos Intellectual Property Rights Policy relates to normative references to external materials not created by

> Khronos.

This process is performed according to the video decode operation steps with the codec-specific semantics defined in section 8 of the ITU-T H.264 Specification as follows:

- Syntax elements, derived values, and other parameters are applied from the following structures:
  - The `StdVideoH264SequenceParameterSet` structure corresponding to the active SPS specifying the H.264 sequence parameter set.
  - The `StdVideoH264PictureParameterSet` structure corresponding to the active PPS specifying the H.264 picture parameter set.
  - The `StdVideoDecodeH264PictureInfo` structure specifying the H.264 picture information.
  - The `StdVideoDecodeH264ReferenceInfo` structures specifying the H.264 reference information corresponding to the optional reconstructed picture and any active reference pictures.
- The contents of the provided video bitstream buffer range are interpreted as defined in the H.264 Decode Bitstream Data Access section.
- Picture data in the video picture resources corresponding to the used active reference pictures, decode output picture, and optional reconstructed picture is accessed as defined in the H.264 Decode Picture Data Access section.
- The decision on reference picture setup is made according to the parameters specified in the H.264 picture information.

If the parameters and the bitstream adhere to the syntactic and semantic requirements defined in the corresponding sections of the ITU-T H.264 Specification, as described above, and the DPB slots associated with the active reference pictures all refer to valid picture references, then the video decode operation will complete successfully. Otherwise, the video decode operation **may** complete unsuccessfully.

38.12.1. H.264 Decode Bitstream Data Access

If the target decode output picture is a frame, then the video bitstream buffer range **should** contain a VCL NAL unit comprised of the slice headers and data of a picture representing an entire frame, as defined in sections 7.3.3 and 7.3.4, and this data is interpreted as defined in sections 7.4.3 and 7.4.4 of the ITU-T H.264 Specification, respectively.

If the target decode output picture is a field, then the video bitstream buffer range should contain a VCL NAL unit comprised of the slice headers and data of a picture representing a field, as defined in sections 7.3.3 and 7.3.4, and this data is interpreted as defined in sections 7.4.3 and 7.4.4 of the ITU-T H.264 Specification, respectively.

The offsets provided in VkVideoDecodeH264PictureInfoKHR::`pSliceOffsets` **should** specify the starting offsets corresponding to each slice header within the video bitstream buffer range.

38.12.2. H.264 Decode Picture Data Access

The effective `imageOffset` and `imageExtent` corresponding to a decode output picture, reference

picture, or reconstructed picture used in video decode operations with an H.264 decode profile are defined as follows:

- imageOffset is (codedOffset.x,codedOffset.y) and imageExtent is (codedExtent.width, codedExtent.height), if the picture represents a frame.
- imageOffset is (codedOffset.x,codedOffset.y) and imageExtent is (codedExtent.width, codedExtent.height), if the picture represents a field and the picture layout of the used H.264 decode profile is VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_INTERLEAVED_LINES_BIT_KHR.
- imageOffset is (codedOffset.x,codedOffset.y) and imageExtent is (codedExtent.width, codedExtent.height / 2), if the picture represents a field and the picture layout of the used H.264 decode profile is VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR.

Where codedOffset and codedExtent are the members of the VkVideoPictureResourceInfoKHR structure corresponding to the picture.

However, accesses to image data within a video picture resource happen at the granularity indicated by VkVideoCapabilitiesKHR::pictureAccessGranularity, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile. This means that the complete image subregion accessed by video coding operations using an H.264 decode profile for the video picture resource is defined as the set of texels within the coordinate range:

([startX,endX), [startY,endY))

Where:

- startX equals imageOffset.x rounded down to the nearest integer multiple of pictureAccessGranularity.width;
- endX equals imageOffset.x + imageExtent.width rounded up to the nearest integer multiple of pictureAccessGranularity.width and clamped to the width of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;
- startY equals imageOffset.y rounded down to the nearest integer multiple of pictureAccessGranularity.height;
- endY equals imageOffset.y + imageExtent.height rounded up to the nearest integer multiple of pictureAccessGranularity.height and clamped to the height of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure.

In case of video decode operations using an H.264 decode profile, any access to a picture at the coordinates (x,y), as defined by the ITU-T H.264 Specification, is an access to the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure at the texel coordinates specified below:

- (x,y), if the accessed picture represents a frame.
- (x,y × 2), if the accessed picture represents a top field and the picture layout of the used H.264 decode profile is VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_INTERLEAVED_LINES_BIT_KHR.
- (x,y × 2 + 1), if the accessed picture represents a bottom field and the picture layout of the used H.264 decode profile is

VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_INTERLEAVED_LINES_BIT_KHR.

- (x,y), if the accessed picture represents a top field and the picture layout of the used H.264 decode profile is VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR.
- (codedOffset.x + x,codedOffset.y + y), if the accessed picture represents a bottom field and the picture layout of the used H.264 decode profile is VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR.

Where codedOffset is the member of the corresponding VkVideoPictureResourceInfoKHR structure.

38.12.3. H.264 Decode Profile

A video profile supporting H.264 video decode operations is specified by setting VkVideoProfileInfoKHR::videoCodecOperation to VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR and adding a VkVideoDecodeH264ProfileInfoKHR structure to the VkVideoProfileInfoKHR::pNext chain.

The VkVideoDecodeH264ProfileInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_h264
typedef struct VkVideoDecodeH264ProfileInfoKHR {
    VkStructureType                                 sType;
    const void*                                     pNext;
    StdVideoH264ProfileIdc                          stdProfileIdc;
    VkVideoDecodeH264PictureLayoutFlagBitsKHR       pictureLayout;
} VkVideoDecodeH264ProfileInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- stdProfileIdc is a StdVideoH264ProfileIdc value specifying the H.264 codec profile IDC, as defined in section A.2 of the ITU-T H.264 Specification.
- pictureLayout is a VkVideoDecodeH264PictureLayoutFlagBitsKHR value specifying the picture layout used by the H.264 video sequence to be decoded.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH264ProfileInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_PROFILE_INFO_KHR
- VUID-VkVideoDecodeH264ProfileInfoKHR-pictureLayout-parameter
  If pictureLayout is not 0, pictureLayout **must** be a valid VkVideoDecodeH264PictureLayoutFlagBitsKHR value

The H.264 video decode picture layout flags are defined as follows:

```
// Provided by VK_KHR_video_decode_h264
typedef enum VkVideoDecodeH264PictureLayoutFlagBitsKHR {
    VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_PROGRESSIVE_KHR = 0,
    VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_INTERLEAVED_LINES_BIT_KHR =
0x00000001,
    VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR =
0x00000002,
} VkVideoDecodeH264PictureLayoutFlagBitsKHR;
```

- VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_PROGRESSIVE_KHR specifies support for progressive content. This flag has the value 0.
- VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_INTERLEAVED_LINES_BIT_KHR specifies support for or use of a picture layout for interlaced content where all lines belonging to the top field are decoded to the even-numbered lines within the picture resource, and all lines belonging to the bottom field are decoded to the odd-numbered lines within the picture resource.
- VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR specifies support for or use of a picture layout for interlaced content where all lines belonging to a field are grouped together in a single image subregion, and the two fields comprising the frame **can** be stored in separate image subregions of the same image subresource or in separate image subresources.

```
// Provided by VK_KHR_video_decode_h264
typedef VkFlags VkVideoDecodeH264PictureLayoutFlagsKHR;
```

VkVideoDecodeH264PictureLayoutFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoDecodeH264PictureLayoutFlagBitsKHR.

38.12.4. H.264 Decode Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities for an H.264 decode profile, the VkVideoCapabilitiesKHR::pNext chain **must** include a VkVideoDecodeH264CapabilitiesKHR structure that will be filled with the profile-specific capabilities.

The VkVideoDecodeH264CapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_h264
typedef struct VkVideoDecodeH264CapabilitiesKHR {
    VkStructureType         sType;
    void*                   pNext;
    StdVideoH264LevelIdc    maxLevelIdc;
    VkOffset2D              fieldOffsetGranularity;
} VkVideoDecodeH264CapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxLevelIdc is a StdVideoH264LevelIdc value indicating the maximum H.264 level supported by

the profile, where enum constant STD_VIDEO_H264_LEVEL_IDC_<major>_<minor> identifies H.264 level <major>.<minor> as defined in section A.3 of the ITU-T H.264 Specification.

- fieldOffsetGranularity is the minimum alignment for VkVideoPictureResourceInfoKHR ::codedOffset specified for a video picture resource when using the picture layout VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH264CapabilitiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_CAPABILITIES_KHR

38.12.5. H.264 Decode Parameter Sets

Video session parameters objects created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR **can** contain the following types of parameters:

H.264 Sequence Parameter Sets (SPS)

Represented by StdVideoH264SequenceParameterSet structures and interpreted as follows:

- reserved1 and reserved2 are used only for padding purposes and are otherwise ignored;
- seq_parameter_set_id is used as the key of the SPS entry;
- level_idc is one of the enum constants STD_VIDEO_H264_LEVEL_IDC_<major>_<minor> identifying the H.264 level <major>.<minor> as defined in section A.3 of the ITU-T H.264 Specification;
- if flags.seq_scaling_matrix_present_flag is set, then the StdVideoH264ScalingLists structure pointed to by pScalingLists is interpreted as follows:
  - scaling_list_present_mask is a bitmask where bit index i corresponds to seq_scaling_list_present_flag[i] as defined in section 7.4.2.1 of the ITU-T H.264 Specification;
  - use_default_scaling_matrix_mask is a bitmask where bit index i corresponds to UseDefaultScalingMatrix4x4Flag[i], when i < 6, or corresponds to UseDefaultScalingMatrix8x8Flag[i-6], otherwise, as defined in section 7.3.2.1 of the ITU-T H.264 Specification;
  - ScalingList4x4 and ScalingList8x8 correspond to the identically named syntax elements defined in section 7.3.2.1 of the ITU-T H.264 Specification;
- if flags.vui_parameters_present_flag is set, then pSequenceParameterSetVui is a pointer to a StdVideoH264SequenceParameterSetVui structure that is interpreted as follows:
  - reserved1 is used only for padding purposes and is otherwise ignored;
  - flags.color_description_present_flag is interpreted as the value of colour_description_present_flag, as defined in section E.2.1 of the ITU-T H.264 Specification;

NOTE	The name of `colour_description_present_flag` was misspelled in the Video Std header.

  - if `flags.nal_hrd_parameters_present_flag` or `flags.vcl_hrd_parameters_present_flag` is set, then the `StdVideoH264HrdParameters` structure pointed to by `pHrdParameters` is interpreted as follows:
    - `reserved1` is used only for padding purposes and is otherwise ignored;
    - all other members of `StdVideoH264HrdParameters` are interpreted as defined in section E.2.2 of the ITU-T H.264 Specification;
  - all other members of `StdVideoH264SequenceParameterSetVui` are interpreted as defined in section E.2.1 of the ITU-T H.264 Specification;
- all other members of `StdVideoH264SequenceParameterSet` are interpreted as defined in section 7.4.2.1 of the ITU-T H.264 Specification.

H.264 Picture Parameter Sets (PPS)

Represented by `StdVideoH264PictureParameterSet` structures and interpreted as follows:

- the pair constructed from `seq_parameter_set_id` and `pic_parameter_set_id` is used as the key of the PPS entry;
- if `flags.pic_scaling_matrix_present_flag` is set, then the `StdVideoH264ScalingLists` structure pointed to by `pScalingLists` is interpreted as follows:
  - `scaling_list_present_mask` is a bitmask where bit index i corresponds to `pic_scaling_list_present_flag[i]` as defined in section 7.4.2.2 of the ITU-T H.264 Specification;
  - `use_default_scaling_matrix_mask` is a bitmask where bit index i corresponds to `UseDefaultScalingMatrix4x4Flag[i]`, when i < 6, or corresponds to `UseDefaultScalingMatrix8x8Flag[i-6]`, otherwise, as defined in section 7.3.2.2 of the ITU-T H.264 Specification;
  - `ScalingList4x4` and `ScalingList8x8` correspond to the identically named syntax elements defined in section 7.3.2.2 of the ITU-T H.264 Specification;
- all other members of `StdVideoH264PictureParameterSet` are interpreted as defined in section 7.4.2.2 of the ITU-T H.264 Specification.

When a video session parameters object is created with the codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, the VkVideoSessionParametersCreateInfoKHR::`pNext` chain **must** include a `VkVideoDecodeH264SessionParametersCreateInfoKHR` structure specifying the capacity and initial contents of the object.

The `VkVideoDecodeH264SessionParametersCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_h264
typedef struct VkVideoDecodeH264SessionParametersCreateInfoKHR {
    VkStructureType                                    sType;
    const void*                                        pNext;
    uint32_t                                           maxStdSPSCount;
    uint32_t                                           maxStdPPSCount;
    const VkVideoDecodeH264SessionParametersAddInfoKHR*    pParametersAddInfo;
} VkVideoDecodeH264SessionParametersCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxStdSPSCount` is the maximum number of H.264 SPS entries the created `VkVideoSessionParametersKHR` **can** contain.
- `maxStdPPSCount` is the maximum number of H.264 PPS entries the created `VkVideoSessionParametersKHR` **can** contain.
- `pParametersAddInfo` is `NULL` or a pointer to a VkVideoDecodeH264SessionParametersAddInfoKHR structure specifying H.264 parameters to add upon object creation.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH264SessionParametersCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_SESSION_PARAMETERS_CREATE_INFO_KHR`
- VUID-VkVideoDecodeH264SessionParametersCreateInfoKHR-pParametersAddInfo-parameter
  If `pParametersAddInfo` is not `NULL`, `pParametersAddInfo` **must** be a valid pointer to a valid VkVideoDecodeH264SessionParametersAddInfoKHR structure

The `VkVideoDecodeH264SessionParametersAddInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_h264
typedef struct VkVideoDecodeH264SessionParametersAddInfoKHR {
    VkStructureType                              sType;
    const void*                                  pNext;
    uint32_t                                     stdSPSCount;
    const StdVideoH264SequenceParameterSet*      pStdSPSs;
    uint32_t                                     stdPPSCount;
    const StdVideoH264PictureParameterSet*       pStdPPSs;
} VkVideoDecodeH264SessionParametersAddInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `stdSPSCount` is the number of elements in the `pStdSPSs` array.
- `pStdSPSs` is a pointer to an array of `StdVideoH264SequenceParameterSet` structures describing the

H.264 SPS entries to add.

- `stdPPSCount` is the number of elements in the `pStdPPSs` array.
- `pStdPPSs` is a pointer to an array of `StdVideoH264PictureParameterSet` structures describing the H.264 PPS entries to add.

This structure **can** be specified in the following places:

- In the `pParametersAddInfo` member of the VkVideoDecodeH264SessionParametersCreateInfoKHR structure specified in the `pNext` chain of VkVideoSessionParametersCreateInfoKHR used to create a video session parameters object. In this case, if the video codec operation the video session parameters object is created with is `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, then it defines the set of initial parameters to add to the created object (see Creating Video Session Parameters).
- In the `pNext` chain of VkVideoSessionParametersUpdateInfoKHR. In this case, if the video codec operation the video session parameters object to be updated was created with is `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`, then it defines the set of parameters to add to it (see Updating Video Session Parameters).

Valid Usage

- VUID-VkVideoDecodeH264SessionParametersAddInfoKHR-None-04825
  The `seq_parameter_set_id` member of each `StdVideoH264SequenceParameterSet` structure specified in the elements of `pStdSPSs` **must** be unique within `pStdSPSs`
- VUID-VkVideoDecodeH264SessionParametersAddInfoKHR-None-04826
  The pair constructed from the `seq_parameter_set_id` and `pic_parameter_set_id` members of each `StdVideoH264PictureParameterSet` structure specified in the elements of `pStdPPSs` **must** be unique within `pStdPPSs`

Valid Usage (Implicit)

- VUID-VkVideoDecodeH264SessionParametersAddInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_SESSION_PARAMETERS_ADD_INFO_KHR`
- VUID-VkVideoDecodeH264SessionParametersAddInfoKHR-pStdSPSs-parameter
  If `stdSPSCount` is not `0`, `pStdSPSs` **must** be a valid pointer to an array of `stdSPSCount` `StdVideoH264SequenceParameterSet` values
- VUID-VkVideoDecodeH264SessionParametersAddInfoKHR-pStdPPSs-parameter
  If `stdPPSCount` is not `0`, `pStdPPSs` **must** be a valid pointer to an array of `stdPPSCount` `StdVideoH264PictureParameterSet` values

38.12.6. H.264 Decoding Parameters

The `VkVideoDecodeH264PictureInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_h264
typedef struct VkVideoDecodeH264PictureInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoDecodeH264PictureInfo*     pStdPictureInfo;
    uint32_t                                 sliceCount;
    const uint32_t*                          pSliceOffsets;
} VkVideoDecodeH264PictureInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pStdPictureInfo` is a pointer to a `StdVideoDecodeH264PictureInfo` structure specifying H.264 picture information.
- `sliceCount` is the number of elements in `pSliceOffsets`.
- `pSliceOffsets` is a pointer to an array of `sliceCount` offsets specifying the start offset of the slices of the picture within the video bitstream buffer range specified in VkVideoDecodeInfoKHR.

This structure is specified in the `pNext` chain of the VkVideoDecodeInfoKHR structure passed to vkCmdDecodeVideoKHR to specify the codec-specific picture information for an H.264 decode operation.

Decode Output Picture Information

When this structure is specified in the `pNext` chain of the VkVideoDecodeInfoKHR structure passed to vkCmdDecodeVideoKHR, the information related to the decode output picture is defined as follows:

- If `pStdPictureInfo->flags.field_pic_flag` is not set, then the picture represents a frame.
- If `pStdPictureInfo->flags.field_pic_flag` is set, then the picture represents a field. Specifically:
  - If `pStdPictureInfo->flags.bottom_field_flag` is not set, then the picture represents the top field of the frame.
  - If `pStdPictureInfo->flags.bottom_field_flag` is set, then the picture represents the bottom field of the frame.
- The image subregion used is determined according to the H.264 Decode Picture Data Access section.
- The decode output picture is associated with the H.264 picture information provided in `pStdPictureInfo`.

Std Picture Information

The members of the `StdVideoDecodeH264PictureInfo` structure pointed to by `pStdPictureInfo` are interpreted as follows:

- `reserved1` and `reserved2` are used only for padding purposes and are otherwise ignored;
- `flags.is_intra` as defined in section 3.73 of the ITU-T H.264 Specification;

- `flags.is_reference` as defined in section 3.136 of the ITU-T H.264 Specification;
- `flags.complementary_field_pair` as defined in section 3.35 of the ITU-T H.264 Specification;
- `seq_parameter_set_id` and `pic_parameter_set_id` are used to identify the active parameter sets, as described below;
- all other members are interpreted as defined in section 7.4.3 of the ITU-T H.264 Specification.

Reference picture setup is controlled by the value of `StdVideoDecodeH264PictureInfo::flags.is_reference`. If it is set and a reconstructed picture is specified, then the latter is used as the target of picture reconstruction to activate the DPB slot specified in `pDecodeInfo->pSetupReferenceSlot->slotIndex`. If `StdVideoDecodeH264PictureInfo::flags.is_reference` is not set, but a reconstructed picture is specified, then the corresponding picture reference associated with the DPB slot is invalidated, as described in the DPB Slot States section.

Active Parameter Sets

The members of the `StdVideoDecodeH264PictureInfo` structure pointed to by `pStdPictureInfo` are used to select the active parameter sets to use from the bound video session parameters object, as follows:

- The *active SPS* is the SPS identified by the key specified in `StdVideoDecodeH264PictureInfo::seq_parameter_set_id`.
- The *active PPS* is the PPS identified by the key specified by the pair constructed from `StdVideoDecodeH264PictureInfo::seq_parameter_set_id` and `StdVideoDecodeH264PictureInfo::pic_parameter_set_id`.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH264PictureInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_PICTURE_INFO_KHR`
- VUID-VkVideoDecodeH264PictureInfoKHR-pStdPictureInfo-parameter
  `pStdPictureInfo` **must** be a valid pointer to a valid `StdVideoDecodeH264PictureInfo` value
- VUID-VkVideoDecodeH264PictureInfoKHR-pSliceOffsets-parameter
  `pSliceOffsets` **must** be a valid pointer to an array of `sliceCount` `uint32_t` values
- VUID-VkVideoDecodeH264PictureInfoKHR-sliceCount-arraylength
  `sliceCount` **must** be greater than `0`

The `VkVideoDecodeH264DpbSlotInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_h264
typedef struct VkVideoDecodeH264DpbSlotInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoDecodeH264ReferenceInfo*   pStdReferenceInfo;
} VkVideoDecodeH264DpbSlotInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pStdReferenceInfo` is a pointer to a `StdVideoDecodeH264ReferenceInfo` structure specifying H.264 reference information.

This structure is specified in the `pNext` chain of VkVideoDecodeInfoKHR::`pSetupReferenceSlot`, if not `NULL`, and the `pNext` chain of the elements of VkVideoDecodeInfoKHR::`pReferenceSlots` to specify the codec-specific reference picture information for an H.264 decode operation.

Active Reference Picture Information

When this structure is specified in the `pNext` chain of the elements of VkVideoDecodeInfoKHR::`pReferenceSlots`, one or two elements are added to the list of active reference pictures used by the video decode operation for each element of VkVideoDecodeInfoKHR::`pReferenceSlots` as follows:

- If neither `pStdReferenceInfo->flags.top_field_flag` nor `pStdReferenceInfo->flags.bottom_field_flag` is set, then the picture is added as a frame reference to the list of active reference pictures.
- If `pStdReferenceInfo->flags.top_field_flag` is set, then the picture is added as a top field reference to the list of active reference pictures.
- If `pStdReferenceInfo->flags.bottom_field_flag` is set, then the picture is added as a bottom field reference to the list of active reference pictures.
- For each added reference picture, the corresponding image subregion used is determined according to the H.264 Decode Picture Data Access section.
- Each added reference picture is associated with the DPB slot index specified in the `slotIndex` member of the corresponding element of VkVideoDecodeInfoKHR::`pReferenceSlots`.
- Each added reference picture is associated with the H.264 reference information provided in `pStdReferenceInfo`.

NOTE

When both the top and bottom field of an interlaced frame currently associated with a DPB slot is intended to be used as an active reference picture and both fields are stored in the same image subregion (which is the case when using `VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_INTERLEAVED_LINES_BIT_KHR` which stores the two fields at even and odd scanlines of the same image subregion), both references have to be provided through a single VkVideoReferenceSlotInfoKHR structure that has both `flags.top_field_flag` and `flags.bottom_field_flag` set in the `StdVideoDecodeH264ReferenceInfo` structure pointed to by the `pStdReferenceInfo` member of the VkVideoDecodeH264DpbSlotInfoKHR structure included in the corresponding VkVideoReferenceSlotInfoKHR structure's `pNext` chain. However, this approach can only be used when both fields are stored in the same image subregion. If that is not the case (e.g. when using `VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR` which requires separate `codedOffset` values for the two fields and also allows storing the two fields of a frame in separate image layers or entirely separate images), then a separate VkVideoReferenceSlotInfoKHR structure needs to be provided for

> referencing the two fields, each only setting one of `flags.top_field_flag` or `flags.bottom_field_flag`, and providing the appropriate video picture resource information in VkVideoReferenceSlotInfoKHR::`pPictureResource`.

Reconstructed Picture Information

When this structure is specified in the `pNext` chain of VkVideoDecodeInfoKHR::`pSetupReferenceSlot`, the information related to the reconstructed picture is defined as follows:

- If neither `pStdReferenceInfo->flags.top_field_flag` nor `pStdReferenceInfo->flags.bottom_field_flag` is set, then the picture represents a frame.
- If `pStdReferenceInfo->flags.top_field_flag` is set, then the picture represents a field, specifically, the top field of the frame.
- If `pStdReferenceInfo->flags.bottom_field_flag` is set, then the picture represents a field, specifically, the bottom field of the frame.
- The image subregion used is determined according to the H.264 Decode Picture Data Access section.
- If reference picture setup is requested, then the reconstructed picture is used to activate the DPB slot with the index specified in VkVideoDecodeInfoKHR::`pSetupReferenceSlot->slotIndex`.
- The reconstructed picture is associated with the H.264 reference information provided in `pStdReferenceInfo`.

Std Reference Information

The members of the `StdVideoDecodeH264ReferenceInfo` structure pointed to by `pStdReferenceInfo` are interpreted as follows:

- `flags.top_field_flag` is used to indicate whether the reference is used as top field reference;
- `flags.bottom_field_flag` is used to indicate whether the reference is used as bottom field reference;
- `flags.used_for_long_term_reference` is used to indicate whether the picture is marked as "used for long-term reference" as defined in section 8.2.5.1 of the ITU-T H.264 Specification;
- `flags.is_non_existing` is used to indicate whether the picture is marked as "non-existing" as defined in section 8.2.5.2 of the ITU-T H.264 Specification;
- all other members are interpreted as defined in section 8.2 of the ITU-T H.264 Specification.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH264DpbSlotInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_DPB_SLOT_INFO_KHR`
- VUID-VkVideoDecodeH264DpbSlotInfoKHR-pStdReferenceInfo-parameter
  `pStdReferenceInfo` **must** be a valid pointer to a valid `StdVideoDecodeH264ReferenceInfo` value

38.12.7. H.264 Decode Requirements

This section describes the **required** H.264 decoding capabilities for physical devices that have at least one queue family that supports the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR, as returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR in VkQueueFamilyVideoPropertiesKHR ::videoCodecOperations.

Table 47. Required Video Std Header Versions

Video Std Header Name	Version
vulkan_video_codec_h264std_decode	1.0.0

Table 48. Required Video Capabilities

Video Capability	Requirement	Requirement Type[1]
VkVideoCapabilitiesKHR		
flags	-	min
minBitstreamBufferOffsetAlignment	4096	max
minBitstreamBufferSizeAlignment	4096	max
pictureAccessGranularity	(64,64)	max
minCodedExtent	-	max
maxCodedExtent	-	min
maxDpbSlots	0	min
maxActiveReferencePictures	0	min
VkVideoDecodeCapabilitiesKHR		
flags	VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR or VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR	min
VkVideoDecodeH264CapabilitiesKHR		
maxLevelIdc	STD_VIDEO_H264_LEVEL_IDC_1_0	min
fieldOffsetGranularity	(0,0) except for profiles using VK_VIDEO_DECODE_H264_PICTURE_LAYOUT_INTERLACED_SEPARATE_PLANES_BIT_KHR	implementation-dependent

1

The **Requirement Type** column specifies the requirement is either the minimum value all

implementations **must** support, the maximum value all implementations **must** support, or the exact value all implementations **must** support. For bitmasks a minimum value is the least bits all implementations **must** set, but they **may** have additional bits set beyond this minimum.

38.13. H.265 Decode Operations

Video decode operations using an H.265 decode profile **can** be used to decode elementary video stream sequences compliant to the ITU-T H.265 Specification.

NOTE	Refer to the Preamble for information on how the Khronos Intellectual Property Rights Policy relates to normative references to external materials not created by Khronos.

This process is performed according to the video decode operation steps with the codec-specific semantics defined in section 8 of ITU-T H.265 Specification:

- Syntax elements, derived values, and other parameters are applied from the following structures:
  - The `StdVideoH265VideoParameterSet` structure corresponding to the active VPS specifying the H.265 video parameter set.
  - The `StdVideoH265SequenceParameterSet` structure corresponding to the active SPS specifying the H.265 sequence parameter set.
  - The `StdVideoH265PictureParameterSet` structure corresponding to the active PPS specifying the H.265 picture parameter set.
  - The `StdVideoDecodeH265PictureInfo` structure specifying the H.265 picture information.
  - The `StdVideoDecodeH265ReferenceInfo` structures specifying the H.265 reference information corresponding to the optional reconstructed picture and any active reference pictures.
- The contents of the provided video bitstream buffer range are interpreted as defined in the H.265 Decode Bitstream Data Access section.
- Picture data in the video picture resources corresponding to the used active reference pictures, decode output picture, and optional reconstructed picture is accessed as defined in the H.265 Decode Picture Data Access section.
- The decision on reference picture setup is made according to the parameters specified in the H.265 picture information.

If the parameters and the bitstream adhere to the syntactic and semantic requirements defined in the corresponding sections of the ITU-T H.265 Specification, as described above, and the DPB slots associated with the active reference pictures all refer to valid picture references, then the video decode operation will complete successfully. Otherwise, the video decode operation **may** complete unsuccessfully.

38.13.1. H.265 Decode Bitstream Data Access

The video bitstream buffer range **should** contain a VCL NAL unit comprised of the slice segment headers and data of a picture representing a frame, as defined in sections 7.3.6 and 7.3.8, and this

data is interpreted as defined in sections 7.4.7 and 7.4.9 of the ITU-T H.265 Specification, respectively.

The offsets provided in VkVideoDecodeH265PictureInfoKHR::pSliceSegmentOffsets **should** specify the starting offsets corresponding to each slice segment header within the video bitstream buffer range.

38.13.2. H.265 Decode Picture Data Access

Accesses to image data within a video picture resource happen at the granularity indicated by VkVideoCapabilitiesKHR::pictureAccessGranularity, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile. Accordingly, the complete image subregion of a decode output picture, reference picture, or reconstructed picture accessed by video coding operations using an H.265 decode profile is defined as the set of texels within the coordinate range:

([0,endX), [0,endY))

Where:

- endX equals codedExtent.width rounded up to the nearest integer multiple of pictureAccessGranularity.width and clamped to the width of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;
- endY equals codedExtent.height rounded up to the nearest integer multiple of pictureAccessGranularity.height and clamped to the height of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;

Where codedExtent is the member of the VkVideoPictureResourceInfoKHR structure corresponding to the picture.

In case of video decode operations using an H.265 decode profile, any access to a picture at the coordinates (x,y), as defined by the ITU-T H.265 Specification, is an access to the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure at the texel coordinates (x,y).

38.13.3. H.265 Decode Profile

A video profile supporting H.265 video decode operations is specified by setting VkVideoProfileInfoKHR::videoCodecOperation to VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR and adding a VkVideoDecodeH265ProfileInfoKHR structure to the VkVideoProfileInfoKHR::pNext chain.

The VkVideoDecodeH265ProfileInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_h265
typedef struct VkVideoDecodeH265ProfileInfoKHR {
    VkStructureType           sType;
    const void*               pNext;
    StdVideoH265ProfileIdc    stdProfileIdc;
} VkVideoDecodeH265ProfileInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `stdProfileIdc` is a `StdVideoH265ProfileIdc` value specifying the H.265 codec profile IDC, as defined in section A.3 of the ITU-T H.265 Specification.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH265ProfileInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_PROFILE_INFO_KHR`

38.13.4. H.265 Decode Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities for an H.265 decode profile, the VkVideoCapabilitiesKHR::`pNext` chain **must** include a `VkVideoDecodeH265CapabilitiesKHR` structure that will be filled with the profile-specific capabilities.

The `VkVideoDecodeH265CapabilitiesKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_h265
typedef struct VkVideoDecodeH265CapabilitiesKHR {
    VkStructureType         sType;
    void*                   pNext;
    StdVideoH265LevelIdc    maxLevelIdc;
} VkVideoDecodeH265CapabilitiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxLevelIdc` is a `StdVideoH265LevelIdc` value indicating the maximum H.265 level supported by the profile, where enum constant `STD_VIDEO_H265_LEVEL_IDC_<major>_<minor>` identifies H.265 level `<major>.<minor>` as defined in section A.4 of the ITU-T H.265 Specification.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH265CapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_CAPABILITIES_KHR`

38.13.5. H.265 Decode Parameter Sets

Video session parameters objects created with the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR` **can** contain the following types of parameters:

H.265 Video Parameter Sets (VPS)

Represented by `StdVideoH265VideoParameterSet` structures and interpreted as follows:

- `reserved1`, `reserved2`, and `reserved3` are used only for padding purposes and are otherwise ignored;
- `vps_video_parameter_set_id` is used as the key of the VPS entry;
- the `max_latency_increase_plus1`, `max_dec_pic_buffering_minus1`, and `max_num_reorder_pics` members of the `StdVideoH265DecPicBufMgr` structure pointed to by `pDecPicBufMgr` correspond to `vps_max_latency_increase_plus1`, `vps_max_dec_pic_buffering_minus1`, and `vps_max_num_reorder_pics`, respectively, as defined in section 7.4.3.1 of the ITU-T H.265 Specification;
- the `StdVideoH265HrdParameters` structure pointed to by `pHrdParameters` is interpreted as follows:
  - `reserved` is used only for padding purposes and is otherwise ignored;
  - `flags.fixed_pic_rate_general_flag` is a bitmask where bit index i corresponds to `fixed_pic_rate_general_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
  - `flags.fixed_pic_rate_within_cvs_flag` is a bitmask where bit index i corresponds to `fixed_pic_rate_within_cvs_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
  - `flags.low_delay_hrd_flag` is a bitmask where bit index i corresponds to `low_delay_hrd_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
  - if `flags.nal_hrd_parameters_present_flag` is set, then `pSubLayerHrdParametersNal` is a pointer to an array of `vps_max_sub_layers_minus1` + 1 number of `StdVideoH265SubLayerHrdParameters` structures where `vps_max_sub_layers_minus1` is the corresponding member of the encompassing `StdVideoH265VideoParameterSet` structure and each element is interpreted as follows:
    - `cbr_flag` is a bitmask where bit index i corresponds to `cbr_flag[i]` as defined in section E.3.3 of the ITU-T H.265 Specification;
    - all other members of the `StdVideoH265SubLayerHrdParameters` structure are interpreted as defined in section E.3.3 of the ITU-T H.265 Specification;
  - if `flags.vcl_hrd_parameters_present_flag` is set, then `pSubLayerHrdParametersVcl` is a pointer to an array of `vps_max_sub_layers_minus1` + 1 number of `StdVideoH265SubLayerHrdParameters` structures where `vps_max_sub_layers_minus1` is the corresponding member of the encompassing `StdVideoH265VideoParameterSet` structure and each element is interpreted as follows:
    - `cbr_flag` is a bitmask where bit index i corresponds to `cbr_flag[i]` as defined in

section E.3.3 of the ITU-T H.265 Specification;

        - all other members of the `StdVideoH265SubLayerHrdParameters` structure are interpreted as defined in section E.3.3 of the ITU-T H.265 Specification;

    - all other members of `StdVideoH265HrdParameters` are interpreted as defined in section E.3.2 of the ITU-T H.265 Specification;

- the `StdVideoH265ProfileTierLevel` structure pointed to by `pProfileTierLevel` are interpreted as follows:

    - `general_level_idc` is one of the enum constants `STD_VIDEO_H265_LEVEL_IDC_<major>_<minor>` identifying the H.265 level `<major>.<minor>` as defined in section A.4 of the ITU-T H.265 Specification;

    - all other members of `StdVideoH265ProfileTierLevel` are interpreted as defined in section 7.4.4 of the ITU-T H.265 Specification;

- all other members of `StdVideoH265VideoParameterSet` are interpreted as defined in section 7.4.3.1 of the ITU-T H.265 Specification.

H.265 Sequence Parameter Sets (SPS)

Represented by `StdVideoH265SequenceParameterSet` structures and interpreted as follows:

- `reserved1` and `reserved2` are used only for padding purposes and are otherwise ignored;
- the pair constructed from `sps_video_parameter_set_id` and `sps_seq_parameter_set_id` is used as the key of the SPS entry;
- the `StdVideoH265ProfileTierLevel` structure pointed to by `pProfileTierLevel` are interpreted as follows:

    - `general_level_idc` is one of the enum constants `STD_VIDEO_H265_LEVEL_IDC_<major>_<minor>` identifying the H.265 level `<major>.<minor>` as defined in section A.4 of the ITU-T H.265 Specification;

    - all other members of `StdVideoH265ProfileTierLevel` are interpreted as defined in section 7.4.4 of the ITU-T H.265 Specification;

- the `max_latency_increase_plus1`, `max_dec_pic_buffering_minus1`, and `max_num_reorder_pics` members of the `StdVideoH265DecPicBufMgr` structure pointed to by `pDecPicBufMgr` correspond to `sps_max_latency_increase_plus1`, `sps_max_dec_pic_buffering_minus1`, and `sps_max_num_reorder_pics`, respectively, as defined in section 7.4.3.2 of the ITU-T H.265 Specification;
- if `flags.sps_scaling_list_data_present_flag` is set, then the `StdVideoH265ScalingLists` structure pointed to by `pScalingLists` is interpreted as follows:

    - `ScalingList4x4`, `ScalingList8x8`, `ScalingList16x16`, and `ScalingList32x32` correspond to `ScalingList[0]`, `ScalingList[1]`, `ScalingList[2]`, and `ScalingList[3]`, respectively, as defined in section 7.3.4 of the ITU-T H.265 Specification;

    - `ScalingListDCCoef16x16` and `ScalingListDCCoef32x32` correspond to `scaling_list_dc_coef_minus8[0]` and `scaling_list_dc_coef_minus8[1]`, respectively, as defined in section 7.3.4 of the ITU-T H.265 Specification;

- `pShortTermRefPicSet` is a pointer to an array of `num_short_term_ref_pic_sets` number of `StdVideoH265ShortTermRefPicSet` structures where each element is interpreted as follows:
  - `reserved1`, `reserved2`, and `reserved3` are used only for padding purposes and are otherwise ignored;
  - `used_by_curr_pic_flag` is a bitmask where bit index i corresponds to `used_by_curr_pic_flag[i]` as defined in section 7.4.8 of the ITU-T H.265 Specification;
  - `use_delta_flag` is a bitmask where bit index i corresponds to `use_delta_flag[i]` as defined in section 7.4.8 of the ITU-T H.265 Specification;
  - `used_by_curr_pic_s0_flag` is a bitmask where bit index i corresponds to `used_by_curr_pic_s0_flag[i]` as defined in section 7.4.8 of the ITU-T H.265 Specification;
  - `used_by_curr_pic_s1_flag` is a bitmask where bit index i corresponds to `used_by_curr_pic_s1_flag[i]` as defined in section 7.4.8 of the ITU-T H.265 Specification;
  - all other members of `StdVideoH265ShortTermRefPicSet` are interpreted as defined in section 7.4.8 of the ITU-T H.265 Specification;
- if `flags.long_term_ref_pics_present_flag` is set then the `StdVideoH265LongTermRefPicsSps` structure pointed to by `pLongTermRefPicsSps` is interpreted as follows:
  - `used_by_curr_pic_lt_sps_flag` is a bitmask where bit index i corresponds to `used_by_curr_pic_lt_sps_flag[i]` as defined in section 7.4.3.2 of the ITU-T H.265 Specification;
  - all other members of `StdVideoH265LongTermRefPicsSps` are interpreted as defined in section 7.4.3.2 of the ITU-T H.265 Specification;
- if `flags.vui_parameters_present_flag` is set, then the `StdVideoH265SequenceParameterSetVui` structure pointed to by `pSequenceParameterSetVui` is interpreted as follows:
  - `reserved1`, `reserved2`, and `reserved3` are used only for padding purposes and are otherwise ignored;
  - the `StdVideoH265HrdParameters` structure pointed to by `pHrdParameters` is interpreted as follows:
    - `flags.fixed_pic_rate_general_flag` is a bitmask where bit index i corresponds to `fixed_pic_rate_general_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
    - `flags.fixed_pic_rate_within_cvs_flag` is a bitmask where bit index i corresponds to `fixed_pic_rate_within_cvs_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
    - `flags.low_delay_hrd_flag` is a bitmask where bit index i corresponds to `low_delay_hrd_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
    - if `flags.nal_hrd_parameters_present_flag` is set, then `pSubLayerHrdParametersNal` is a pointer to an array of `sps_max_sub_layers_minus1` + 1 number of `StdVideoH265SubLayerHrdParameters` structures where `sps_max_sub_layers_minus1` is the corresponding member of the encompassing `StdVideoH265SequenceParameterSet` structure and each element is interpreted as follows:
      - `cbr_flag` is a bitmask where bit index i corresponds to `cbr_flag[i]` as defined in

section E.3.3 of the ITU-T H.265 Specification;

- all other members of the `StdVideoH265SubLayerHrdParameters` structure are interpreted as defined in section E.3.3 of the ITU-T H.265 Specification;

- if `flags.vcl_hrd_parameters_present_flag` is set, then `pSubLayerHrdParametersVcl` is a pointer to an array of `sps_max_sub_layers_minus1 + 1` number of `StdVideoH265SubLayerHrdParameters` structures where `sps_max_sub_layers_minus1` is the corresponding member of the encompassing `StdVideoH265SequenceParameterSet` structure and each element is interpreted as follows:
  - `cbr_flag` is a bitmask where bit index i corresponds to `cbr_flag[i]` as defined in section E.3.3 of the ITU-T H.265 Specification;
  - all other members of the `StdVideoH265SubLayerHrdParameters` structure are interpreted as defined in section E.3.3 of the ITU-T H.265 Specification;
- all other members of `StdVideoH265HrdParameters` are interpreted as defined in section E.3.2 of the ITU-T H.265 Specification;

- all other members of `pSequenceParameterSetVui` are interpreted as defined in section E.3.1 of the ITU-T H.265 Specification;

- if `flags.sps_palette_predictor_initializer_present_flag` is set, then the `PredictorPaletteEntries` member of the `StdVideoH265PredictorPaletteEntries` structure pointed to by `pPredictorPaletteEntries` is interpreted as defined in section 7.4.9.13 of the ITU-T H.265 Specification;
- all other members of `StdVideoH265SequenceParameterSet` are interpreted as defined in section 7.4.3.1 of the ITU-T H.265 Specification.

H.265 Picture Parameter Sets (PPS)

Represented by `StdVideoH265PictureParameterSet` structures and interpreted as follows:

- `reserved1`, `reserved2`, and `reserved3` are used only for padding purposes and are otherwise ignored;
- the triplet constructed from `sps_video_parameter_set_id`, `pps_seq_parameter_set_id`, and `pps_pic_parameter_set_id` is used as the key of the PPS entry;
- if `flags.pps_scaling_list_data_present_flag` is set, then the `StdVideoH265ScalingLists` structure pointed to by `pScalingLists` is interpreted as follows:
  - `ScalingList4x4`, `ScalingList8x8`, `ScalingList16x16`, and `ScalingList32x32` correspond to `ScalingList[0]`, `ScalingList[1]`, `ScalingList[2]`, and `ScalingList[3]`, respectively, as defined in section 7.3.4 of the ITU-T H.265 Specification;
  - `ScalingListDCCoef16x16` and `ScalingListDCCoef32x32` correspond to `scaling_list_dc_coef_minus8[0]` and `scaling_list_dc_coef_minus8[1]`, respectively, as defined in section 7.3.4 of the ITU-T H.265 Specification;
- if `flags.pps_palette_predictor_initializer_present_flag` is set, then the `PredictorPaletteEntries` member of the `StdVideoH265PredictorPaletteEntries` structure pointed to by `pPredictorPaletteEntries` is interpreted as defined in section 7.4.9.13 of the ITU-

T H.265 Specification;

- all other members of StdVideoH265PictureParameterSet are interpreted as defined in section 7.4.3.3 of the ITU-T H.265 Specification.

When a video session parameters object is created with the codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, the VkVideoSessionParametersCreateInfoKHR::pNext chain **must** include a VkVideoDecodeH265SessionParametersCreateInfoKHR structure specifying the capacity and initial contents of the object.

The VkVideoDecodeH265SessionParametersCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_h265
typedef struct VkVideoDecodeH265SessionParametersCreateInfoKHR {
    VkStructureType                                           sType;
    const void*                                               pNext;
    uint32_t                                                  maxStdVPSCount;
    uint32_t                                                  maxStdSPSCount;
    uint32_t                                                  maxStdPPSCount;
    const VkVideoDecodeH265SessionParametersAddInfoKHR*       pParametersAddInfo;
} VkVideoDecodeH265SessionParametersCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxStdVPSCount is the maximum number of H.265 VPS entries the created VkVideoSessionParametersKHR **can** contain.
- maxStdSPSCount is the maximum number of H.265 SPS entries the created VkVideoSessionParametersKHR **can** contain.
- maxStdPPSCount is the maximum number of H.265 PPS entries the created VkVideoSessionParametersKHR **can** contain.
- pParametersAddInfo is NULL or a pointer to a VkVideoDecodeH265SessionParametersAddInfoKHR structure specifying H.265 parameters to add upon object creation.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH265SessionParametersCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_SESSION_PARAMETERS_CREATE_INFO_KHR
- VUID-VkVideoDecodeH265SessionParametersCreateInfoKHR-pParametersAddInfo-parameter
  If pParametersAddInfo is not NULL, pParametersAddInfo **must** be a valid pointer to a valid VkVideoDecodeH265SessionParametersAddInfoKHR structure

The VkVideoDecodeH265SessionParametersAddInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_h265
typedef struct VkVideoDecodeH265SessionParametersAddInfoKHR {
    VkStructureType                              sType;
    const void*                                  pNext;
    uint32_t                                     stdVPSCount;
    const StdVideoH265VideoParameterSet*         pStdVPSs;
    uint32_t                                     stdSPSCount;
    const StdVideoH265SequenceParameterSet*      pStdSPSs;
    uint32_t                                     stdPPSCount;
    const StdVideoH265PictureParameterSet*       pStdPPSs;
} VkVideoDecodeH265SessionParametersAddInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `stdVPSCount` is the number of elements in the `pStdVPSs` array.
- `pStdVPSs` is a pointer to an array of `StdVideoH265VideoParameterSet` structures describing the H.265 VPS entries to add.
- `stdSPSCount` is the number of elements in the `pStdSPSs` array.
- `pStdSPSs` is a pointer to an array of `StdVideoH265SequenceParameterSet` structures describing the H.265 SPS entries to add.
- `stdPPSCount` is the number of elements in the `pStdPPSs` array.
- `pStdPPSs` is a pointer to an array of `StdVideoH265PictureParameterSet` structures describing the H.265 PPS entries to add.

This structure **can** be specified in the following places:

- In the `pParametersAddInfo` member of the VkVideoDecodeH265SessionParametersCreateInfoKHR structure specified in the `pNext` chain of VkVideoSessionParametersCreateInfoKHR used to create a video session parameters object. In this case, if the video codec operation the video session parameters object is created with is `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR`, then it defines the set of initial parameters to add to the created object (see Creating Video Session Parameters).
- In the `pNext` chain of VkVideoSessionParametersUpdateInfoKHR. In this case, if the video codec operation the video session parameters object to be updated was created with is `VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR`, then it defines the set of parameters to add to it (see Updating Video Session Parameters).

Valid Usage

- VUID-VkVideoDecodeH265SessionParametersAddInfoKHR-None-04833
  The `vps_video_parameter_set_id` member of each `StdVideoH265VideoParameterSet` structure specified in the elements of `pStdVPSs` **must** be unique within `pStdVPSs`
- VUID-VkVideoDecodeH265SessionParametersAddInfoKHR-None-04834
  The pair constructed from the `sps_video_parameter_set_id` and `sps_seq_parameter_set_id`

members of each StdVideoH265SequenceParameterSet structure specified in the elements of pStdSPSs **must** be unique within pStdSPSs

- VUID-VkVideoDecodeH265SessionParametersAddInfoKHR-None-04835
  The triplet constructed from the sps_video_parameter_set_id, pps_seq_parameter_set_id, and pps_pic_parameter_set_id members of each StdVideoH265PictureParameterSet structure specified in the elements of pStdPPSs **must** be unique within pStdPPSs

Valid Usage (Implicit)

- VUID-VkVideoDecodeH265SessionParametersAddInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_SESSION_PARAMETERS_ADD_INFO_KHR
- VUID-VkVideoDecodeH265SessionParametersAddInfoKHR-pStdVPSs-parameter
  If stdVPSCount is not 0, pStdVPSs **must** be a valid pointer to an array of stdVPSCount StdVideoH265VideoParameterSet values
- VUID-VkVideoDecodeH265SessionParametersAddInfoKHR-pStdSPSs-parameter
  If stdSPSCount is not 0, pStdSPSs **must** be a valid pointer to an array of stdSPSCount StdVideoH265SequenceParameterSet values
- VUID-VkVideoDecodeH265SessionParametersAddInfoKHR-pStdPPSs-parameter
  If stdPPSCount is not 0, pStdPPSs **must** be a valid pointer to an array of stdPPSCount StdVideoH265PictureParameterSet values

38.13.6. H.265 Decoding Parameters

The VkVideoDecodeH265PictureInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_h265
typedef struct VkVideoDecodeH265PictureInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoDecodeH265PictureInfo*     pStdPictureInfo;
    uint32_t                                 sliceSegmentCount;
    const uint32_t*                          pSliceSegmentOffsets;
} VkVideoDecodeH265PictureInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pStdPictureInfo is a pointer to a StdVideoDecodeH265PictureInfo structure specifying H.265 picture information.
- sliceSegmentCount is the number of elements in pSliceSegmentOffsets.
- pSliceSegmentOffsets is a pointer to an array of sliceSegmentCount offsets specifying the start offset of the slice segments of the picture within the video bitstream buffer range specified in VkVideoDecodeInfoKHR.

This structure is specified in the pNext chain of the VkVideoDecodeInfoKHR structure passed to vkCmdDecodeVideoKHR to specify the codec-specific picture information for an H.265 decode operation.

Decode Output Picture Information

When this structure is specified in the pNext chain of the VkVideoDecodeInfoKHR structure passed to vkCmdDecodeVideoKHR, the information related to the decode output picture is defined as follows:

- The image subregion used is determined according to the H.265 Decode Picture Data Access section.
- The decode output picture is associated with the H.265 picture information provided in pStdPictureInfo.

Std Picture Information

The members of the StdVideoDecodeH265PictureInfo structure pointed to by pStdPictureInfo are interpreted as follows:

- reserved is used only for padding purposes and is otherwise ignored;
- flags.IrapPicFlag as defined in section 3.73 of the ITU-T H.265 Specification;
- flags.IdrPicFlag as defined in section 3.67 of the ITU-T H.265 Specification;
- flags.IsReference as defined in section 3.132 of the ITU-T H.265 Specification;
- sps_video_parameter_set_id, pps_seq_parameter_set_id, and pps_pic_parameter_set_id are used to identify the active parameter sets, as described below;
- PicOrderCntVal as defined in section 8.3.1 of the ITU-T H.265 Specification;
- NumBitsForSTRefPicSetInSlice is the number of bits used in st_ref_pic_set when short_term_ref_pic_set_sps_flag is 0, or 0 otherwise, as defined in sections 7.4.7 and 7.4.8 of the ITU-T H.265 Specification;
- NumDeltaPocsOfRefRpsIdx is the value of NumDeltaPocs[RefRpsIdx] when short_term_ref_pic_set_sps_flag is 1, or 0 otherwise, as defined in sections 7.4.7 and 7.4.8 of the ITU-T H.265 Specification;
- RefPicSetStCurrBefore, RefPicSetStCurrAfter, and RefPicSetLtCurr are interpreted as defined in section 8.3.2 of the ITU-T H.265 Specification where each element of these arrays either identifies an active reference picture using its DPB slot index or contains the value STD_VIDEO_H265_NO_REFERENCE_PICTURE to indicate “no reference picture”;
- all other members are interpreted as defined in section 8.3.2 of the ITU-T H.265 Specification.

Reference picture setup is controlled by the value of StdVideoDecodeH265PictureInfo ::flags.IsReference. If it is set and a reconstructed picture is specified, then the latter is used as the target of picture reconstruction to activate the corresponding DPB slot. If StdVideoDecodeH265PictureInfo::flags.IsReference is not set, but a reconstructed picture is specified, then the corresponding picture reference associated with the DPB slot is invalidated, as described in the DPB Slot States section.

Active Parameter Sets

The members of the StdVideoDecodeH265PictureInfo structure pointed to by pStdPictureInfo are used to select the active parameter sets to use from the bound video session parameters object, as follows:

- The *active VPS* is the VPS identified by the key specified in StdVideoDecodeH265PictureInfo::sps_video_parameter_set_id.
- The *active SPS* is the SPS identified by the key specified by the pair constructed from StdVideoDecodeH265PictureInfo::sps_video_parameter_set_id and StdVideoDecodeH265PictureInfo::pps_seq_parameter_set_id.
- The *active PPS* is the PPS identified by the key specified by the triplet constructed from StdVideoDecodeH265PictureInfo::sps_video_parameter_set_id, StdVideoDecodeH265PictureInfo::pps_seq_parameter_set_id, and StdVideoDecodeH265PictureInfo::pps_pic_parameter_set_id.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH265PictureInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_PICTURE_INFO_KHR
- VUID-VkVideoDecodeH265PictureInfoKHR-pStdPictureInfo-parameter
  pStdPictureInfo **must** be a valid pointer to a valid StdVideoDecodeH265PictureInfo value
- VUID-VkVideoDecodeH265PictureInfoKHR-pSliceSegmentOffsets-parameter
  pSliceSegmentOffsets **must** be a valid pointer to an array of sliceSegmentCount uint32_t values
- VUID-VkVideoDecodeH265PictureInfoKHR-sliceSegmentCount-arraylength
  sliceSegmentCount **must** be greater than 0

The VkVideoDecodeH265DpbSlotInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_h265
typedef struct VkVideoDecodeH265DpbSlotInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoDecodeH265ReferenceInfo*   pStdReferenceInfo;
} VkVideoDecodeH265DpbSlotInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pStdReferenceInfo is a pointer to a StdVideoDecodeH265ReferenceInfo structure specifying reference picture information described in section 8.3 of the ITU-T H.265 Specification.

This structure is specified in the pNext chain of VkVideoDecodeInfoKHR::pSetupReferenceSlot, if not NULL, and the pNext chain of the elements of VkVideoDecodeInfoKHR::pReferenceSlots to specify the codec-specific reference picture information for an H.265 decode operation.

Active Reference Picture Information

When this structure is specified in the `pNext` chain of the elements of VkVideoDecodeInfoKHR::`pReferenceSlots`, one element is added to the list of active reference pictures used by the video decode operation for each element of VkVideoDecodeInfoKHR::`pReferenceSlots` as follows:

- The image subregion used is determined according to the H.265 Decode Picture Data Access section.
- The reference picture is associated with the DPB slot index specified in the `slotIndex` member of the corresponding element of VkVideoDecodeInfoKHR::`pReferenceSlots`.
- The reference picture is associated with the H.265 reference information provided in `pStdReferenceInfo`.

Reconstructed Picture Information

When this structure is specified in the `pNext` chain of VkVideoDecodeInfoKHR::`pSetupReferenceSlot`, the information related to the reconstructed picture is defined as follows:

- The image subregion used is determined according to the H.265 Decode Picture Data Access section.
- If reference picture setup is requested, then the reconstructed picture is used to activate the DPB slot with the index specified in VkVideoDecodeInfoKHR::`pSetupReferenceSlot->slotIndex`.
- The reconstructed picture is associated with the H.265 reference information provided in `pStdReferenceInfo`.

Std Reference Information

The members of the `StdVideoDecodeH265ReferenceInfo` structure pointed to by `pStdReferenceInfo` are interpreted as follows:

- `flags.used_for_long_term_reference` is used to indicate whether the picture is marked as "used for long-term reference" as defined in section 8.3.2 of the ITU-T H.265 Specification;
- `flags.unused_for_reference` is used to indicate whether the picture is marked as "unused for reference" as defined in section 8.3.2 of the ITU-T H.265 Specification;
- all other members are interpreted as defined in section 8.3 of the ITU-T H.265 Specification.

Valid Usage (Implicit)

- VUID-VkVideoDecodeH265DpbSlotInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_DPB_SLOT_INFO_KHR`
- VUID-VkVideoDecodeH265DpbSlotInfoKHR-pStdReferenceInfo-parameter
  `pStdReferenceInfo` **must** be a valid pointer to a valid `StdVideoDecodeH265ReferenceInfo` value

38.13.7. H.265 Decode Requirements

This section describes the **required** H.265 decoding capabilities for physical devices that have at least one queue family that supports the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR, as returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR in VkQueueFamilyVideoPropertiesKHR ::videoCodecOperations.

Table 49. Required Video Std Header Versions

Video Std Header Name	Version
vulkan_video_codec_h265std_decode	1.0.0

Table 50. Required Video Capabilities

Video Capability	Requirement	Requirement Type[1]
VkVideoCapabilitiesKHR		
flags	-	min
minBitstreamBufferOffsetAlignment	4096	max
minBitstreamBufferSizeAlignment	4096	max
pictureAccessGranularity	(64,64)	max
minCodedExtent	-	max
maxCodedExtent	-	min
maxDpbSlots	0	min
maxActiveReferencePictures	0	min
VkVideoDecodeCapabilitiesKHR		
flags	VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR or VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR	min
VkVideoDecodeH265CapabilitiesKHR		
maxLevelIdc	STD_VIDEO_H265_LEVEL_IDC_1_0	min

1

The **Requirement Type** column specifies the requirement is either the minimum value all implementations **must** support, the maximum value all implementations **must** support, or the exact value all implementations **must** support. For bitmasks a minimum value is the least bits all implementations **must** set, but they **may** have additional bits set beyond this minimum.

38.14. AV1 Decode Operations

Video decode operations using an AV1 decode profile **can** be used to decode elementary video stream sequences compliant with the AV1 Specification.

> **NOTE** Refer to the Preamble for information on how the Khronos Intellectual Property Rights Policy relates to normative references to external materials not created by Khronos.

This process is performed according to the video decode operation steps with the codec-specific semantics defined in section 7 of the AV1 Specification:

- Syntax elements, derived values, and other parameters are applied from the following structures:
  - The `StdVideoAV1SequenceHeader` structure stored in the bound video session parameters object specifying the active sequence header.
  - The `StdVideoDecodeAV1PictureInfo` structure specifying the AV1 picture information.
  - The `StdVideoDecodeAV1ReferenceInfo` structures specifying the AV1 reference information corresponding to the optional reconstructed picture and any active reference pictures.
- The contents of the provided video bitstream buffer range are interpreted as defined in the AV1 Decode Bitstream Data Access section.
- Picture data in the video picture resources corresponding to the used active reference pictures, decode output picture, and optional reconstructed picture is accessed as defined in the AV1 Decode Picture Data Access section.
- The decision on reference picture setup is made according to the parameters specified in the AV1 picture information.

If the parameters and the bitstream adhere to the syntactic and semantic requirements defined in the corresponding sections of the AV1 Specification, as described above, and the DPB slots associated with the active reference pictures all refer to valid picture references, then the video decode operation will complete successfully. Otherwise, the video decode operation **may** complete unsuccessfully.

38.14.1. AV1 Decode Bitstream Data Access

The video bitstream buffer range **should** contain one or more frame OBUs, comprised of a frame header OBU and tile group OBU, that together represent an entire frame, as defined in sections 5.10, 5.9, and 5.11, and this data is interpreted as defined in sections 6.9, 6.8, and 6.10 of the AV1 Specification, respectively.

The offset specified in VkVideoDecodeAV1PictureInfoKHR::`frameHeaderOffset` **should** specify the starting offset of the frame header OBU of the frame.

> **NOTE** When the tiles of the frame are encoded into multiple tile groups, each frame OBU has a separate frame header OBU but their content is expected to match per the requirements of the AV1 Specification. Accordingly, the offset specified in

> `frameHeaderOffset` **can** be the offset of any of the otherwise identical frame header OBUs when multiple tile groups are present.

The offsets and sizes provided in VkVideoDecodeAV1PictureInfoKHR::`pTileOffsets` and VkVideoDecodeAV1PictureInfoKHR::`pTileSizes`, respectively, **should** specify the starting offsets and sizes corresponding to each tile within the video bitstream buffer range.

38.14.2. AV1 Decode Picture Data Access

Accesses to image data within a video picture resource happen at the granularity indicated by VkVideoCapabilitiesKHR::`pictureAccessGranularity`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile. Accordingly, the complete image subregion of a decode output picture, reference picture, or reconstructed picture accessed by video coding operations using an AV1 decode profile is defined as the set of texels within the coordinate range:

([0,`endX`), [0,`endY`))

Where:

- `endX` equals `codedExtent.width` rounded up to the nearest integer multiple of `pictureAccessGranularity.width` and clamped to the width of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;
- endY equals `codedExtent.height` rounded up to the nearest integer multiple of `pictureAccessGranularity.height` and clamped to the height of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;

Where `codedExtent` is the member of the VkVideoPictureResourceInfoKHR structure corresponding to the picture.

In case of video decode operations using an AV1 decode profile, any access to a picture at the coordinates (`x`,`y`), as defined by the AV1 Specification, is an access to the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure at the texel coordinates (`x`,`y`).

38.14.3. AV1 Reference Names and Semantics

Individual reference frames used in the decoding process have different semantics, as defined in section 6.10.24 of the AV1 Specification. The AV1 semantics associated with a reference picture are indicated by the corresponding enumeration constant defined in the Video Std enumeration type `StdVideoAV1ReferenceName`:

- `STD_VIDEO_AV1_REFERENCE_NAME_INTRA_FRAME` identifies the reference used for intra coding (`INTRA_FRAME`), as defined in sections 2 and 7.11.2 of the AV1 Specification.
- All other enumeration constants refer to backward or forward references used for inter coding, as defined in sections 2 and 7.11.3 of the AV1 Specification:
  - `STD_VIDEO_AV1_REFERENCE_NAME_LAST_FRAME` identifies the `LAST_FRAME` reference
  - `STD_VIDEO_AV1_REFERENCE_NAME_LAST2_FRAME` identifies the `LAST2_FRAME` reference

- `STD_VIDEO_AV1_REFERENCE_NAME_LAST3_FRAME` identifies the `LAST3_FRAME` reference
- `STD_VIDEO_AV1_REFERENCE_NAME_GOLDEN_FRAME` identifies the `GOLDEN_FRAME` reference
- `STD_VIDEO_AV1_REFERENCE_NAME_BWDREF_FRAME` identifies the `BWDREF_FRAME` reference
- `STD_VIDEO_AV1_REFERENCE_NAME_ALTREF2_FRAME` identifies the `ALTREF2_FRAME` reference
- `STD_VIDEO_AV1_REFERENCE_NAME_ALTREF_FRAME` identifies the `ALTREF_FRAME` reference

These enumeration constants are not directly used in any APIs but are used to indirectly index into certain Video Std and Vulkan API parameter arrays.

38.14.4. AV1 Decode Profile

A video profile supporting AV1 video decode operations is specified by setting VkVideoProfileInfoKHR::`videoCodecOperation` to `VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR` and adding a `VkVideoDecodeAV1ProfileInfoKHR` structure to the VkVideoProfileInfoKHR::`pNext` chain.

The `VkVideoDecodeAV1ProfileInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_av1
typedef struct VkVideoDecodeAV1ProfileInfoKHR {
    VkStructureType       sType;
    const void*           pNext;
    StdVideoAV1Profile    stdProfile;
    VkBool32              filmGrainSupport;
} VkVideoDecodeAV1ProfileInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `stdProfile` is a `StdVideoAV1Profile` value specifying the AV1 codec profile, as defined in section A.2 of the AV1 Specification.
- `filmGrainSupport` specifies whether AV1 film grain, as defined in section 7.8.3 of the AV1 Specification, **can** be used with the video profile. When this member is `VK_TRUE`, video session objects created against the video profile will be able to decode pictures that have film grain enabled.

NOTE	Enabling `filmGrainSupport` **may** increase the memory requirements of video sessions and/or video picture resources on some implementations.

Valid Usage (Implicit)

- VUID-VkVideoDecodeAV1ProfileInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_PROFILE_INFO_KHR`

38.14.5. AV1 Decode Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities for an AV1 decode profile, the VkVideoCapabilitiesKHR::pNext chain **must** include a VkVideoDecodeAV1CapabilitiesKHR structure that will be filled with the profile-specific capabilities.

The VkVideoDecodeAV1CapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_decode_av1
typedef struct VkVideoDecodeAV1CapabilitiesKHR {
    VkStructureType     sType;
    void*               pNext;
    StdVideoAV1Level    maxLevel;
} VkVideoDecodeAV1CapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxLevel is a StdVideoAV1Level value specifying the maximum AV1 level supported by the profile, as defined in section A.3 of the AV1 Specification.

Valid Usage (Implicit)

- VUID-VkVideoDecodeAV1CapabilitiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_CAPABILITIES_KHR

38.14.6. AV1 Decode Parameter Sets

Video session parameters objects created with the video codec operation VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR contain a single instance of the following parameter set:

AV1 Sequence Header

Represented by StdVideoAV1SequenceHeader structures and interpreted as follows:

- flags.reserved and reserved1 are used only for padding purposes and are otherwise ignored;
- the StdVideoAV1ColorConfig structure pointed to by pColorConfig is interpreted as follows:
  - flags.reserved and reserved1 are used only for padding purposes and are otherwise ignored;
  - all other members of StdVideoAV1ColorConfig are interpreted as defined in section 6.4.2 of the AV1 Specification;
- if flags.timing_info_present_flag is set, then the StdVideoAV1TimingInfo structure pointed to by pTimingInfo is interpreted as follows:

    - `flags.reserved` is used only for padding purposes and is otherwise ignored;
    - all other members of `StdVideoAV1TimingInfo` are interpreted as defined in section 6.4.3 of the AV1 Specification;
- all other members of `StdVideoAV1SequenceHeader` are interpreted as defined in section 6.4 of the AV1 Specification.

When a video session parameters object is created with the codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR`, the `VkVideoSessionParametersCreateInfoKHR::pNext` chain **must** include a `VkVideoDecodeAV1SessionParametersCreateInfoKHR` structure specifying the contents of the object.

The `VkVideoDecodeAV1SessionParametersCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_av1
typedef struct VkVideoDecodeAV1SessionParametersCreateInfoKHR {
    VkStructureType                    sType;
    const void*                        pNext;
    const StdVideoAV1SequenceHeader*   pStdSequenceHeader;
} VkVideoDecodeAV1SessionParametersCreateInfoKHR;
```

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pStdSequenceHeader` is a pointer to a `StdVideoAV1SequenceHeader` structure describing the AV1 sequence header entry to store in the created object.

NOTE

As AV1 video session parameters objects will only ever contain a single AV1 sequence header, this has to be specified at object creation time and such video session parameters objects cannot be updated using the `vkUpdateVideoSessionParametersKHR` command. When a new AV1 sequence header is decoded from the input video bitstream the application needs to create a new video session parameters object to store it.

Valid Usage (Implicit)

- VUID-VkVideoDecodeAV1SessionParametersCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_SESSION_PARAMETERS_CREATE_INFO_KHR`
- VUID-VkVideoDecodeAV1SessionParametersCreateInfoKHR-pStdSequenceHeader-parameter
  `pStdSequenceHeader` **must** be a valid pointer to a valid `StdVideoAV1SequenceHeader` value

38.14.7. AV1 Decoding Parameters

The `VkVideoDecodeAV1PictureInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_av1
typedef struct VkVideoDecodeAV1PictureInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoDecodeAV1PictureInfo*      pStdPictureInfo;
    int32_t                                  referenceNameSlotIndices
[VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR];
    uint32_t                                 frameHeaderOffset;
    uint32_t                                 tileCount;
    const uint32_t*                          pTileOffsets;
    const uint32_t*                          pTileSizes;
} VkVideoDecodeAV1PictureInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pStdPictureInfo` is a pointer to a `StdVideoDecodeAV1PictureInfo` structure specifying AV1 picture information.
- `referenceNameSlotIndices` is an array of seven (`VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR`, which is equal to the Video Std definition `STD_VIDEO_AV1_REFS_PER_FRAME`) signed integer values specifying the index of the DPB slot or a negative integer value for each AV1 reference name used for inter coding. In particular, the DPB slot index for the AV1 reference name `frame` is specified in `referenceNameSlotIndices`[`frame` - `STD_VIDEO_AV1_REFERENCE_NAME_LAST_FRAME`].
- `frameHeaderOffset` is the byte offset of the AV1 frame header OBU, as defined in section 5.9 of the AV1 Specification, within the video bitstream buffer range specified in VkVideoDecodeInfoKHR.
- `tileCount` is the number of elements in `pTileOffsets` and `pTileSizes`.
- `pTileOffsets` is a pointer to an array of `tileCount` integers specifying the byte offset of the tiles of the picture within the video bitstream buffer range specified in VkVideoDecodeInfoKHR.
- `pTileSizes` is a pointer to an array of `tileCount` integers specifying the byte size of the tiles of the picture within the video bitstream buffer range specified in VkVideoDecodeInfoKHR.

This structure is specified in the `pNext` chain of the VkVideoDecodeInfoKHR structure passed to vkCmdDecodeVideoKHR to specify the codec-specific picture information for an AV1 decode operation.

Decode Output Picture Information

When this structure is specified in the `pNext` chain of the VkVideoDecodeInfoKHR structure passed to vkCmdDecodeVideoKHR, the information related to the decode output picture is defined as follows:

- The image subregion used is determined according to the AV1 Decode Picture Data Access section.
- The decode output picture is associated with the AV1 picture information provided in `pStdPictureInfo`.

Std Picture Information

The members of the `StdVideoDecodeAV1PictureInfo` structure pointed to by `pStdPictureInfo` are interpreted as follows:

- `flags.reserved`, `reserved1`, and `reserved2` are used only for padding purposes and are otherwise ignored;
- `flags.apply_grain` indicates that film grain is enabled for the decoded picture, as defined in section 6.8.20 of the AV1 Specification;
- `OrderHint`, `OrderHints`, and `expectedFrameId` are interpreted as defined in section 6.8.2 of the AV1 Specification;
- the `StdVideoAV1TileInfo` structure pointed to by `pTileInfo` is interpreted as follows:
  - `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
  - `pMiColStarts` is a pointer to an array of `TileCols` number of unsigned integers that corresponds to `MiColStarts` defined in section 6.8.14 of the AV1 Specification;
  - `pMiRowStarts` is a pointer to an array of `TileRows` number of unsigned integers that corresponds to `MiRowStarts` defined in section 6.8.14 of the AV1 Specification;
  - `pWidthInSbsMinus1` is a pointer to an array of `TileCols` number of unsigned integers that corresponds to `width_in_sbs_minus_1` defined in section 6.8.14 of the AV1 Specification;
  - `pHeightInSbsMinus1` is a pointer to an array of `TileRows` number of unsigned integers that corresponds to `height_in_sbs_minus_1` defined in section 6.8.14 of the AV1 Specification;
  - all other members of `StdVideoAV1TileInfo` are interpreted as defined in section 6.8.14 of the AV1 Specification;
- the `StdVideoAV1Quantization` structure pointed to by `pQuantization` is interpreted as follows:
  - `flags.reserved` is used only for padding purposes and is otherwise ignored;
  - all other members of `StdVideoAV1Quantization` are interpreted as defined in section 6.8.11 of the AV1 Specification;
- if `flags.segmentation_enabled` is set, then the `StdVideoAV1Segmentation` structure pointed to by `pSegmentation` is interpreted as follows:
  - the elements of `FeatureEnabled` are bitmasks where bit index j of element i corresponds to `FeatureEnabled[i][j]` as defined in section 6.8.13 of the AV1 Specification;
  - `FeatureData` is interpreted as defined in section 6.8.13 of the AV1 Specification;
- the `StdVideoAV1LoopFilter` structure pointed to by `pLoopFilter` is interpreted as follows:
  - `flags.reserved` is used only for padding purposes and is otherwise ignored;
  - `update_ref_delta` is a bitmask where bit index i is interpreted as the value of `update_ref_delta` corresponding to element i of `loop_filter_ref_deltas` as defined in section 6.8.10 of the AV1 Specification;
  - `update_mode_delta` is a bitmask where bit index i is interpreted as the value of `update_mode_delta` corresponding to element i of `loop_filter_mode_deltas` as defined in section 6.8.10 of the AV1 Specification;

    - all other members of StdVideoAV1LoopFilter are interpreted as defined in section 6.8.10 of the AV1 Specification;
- if flags.enable_cdef is set in the active sequence header, then the members of the StdVideoAV1CDEF structure pointed to by pCDEF are interpreted as follows:
    - cdef_y_sec_strength and cdef_uv_sec_strength are the bitstream values of the corresponding syntax elements defined in section 5.9.19 of the AV1 Specification;
    - all other members of StdVideoAV1CDEF are interpreted as defined in section 6.10.14 of the AV1 Specification;
- the StdVideoAV1LoopRestoration structure pointed to by pLoopRestoration is interpreted as follows:
    - LoopRestorationSize[plane] is interpreted as log2(size) - 5, where size is the value of LoopRestorationSize[plane] as defined in section 6.10.15 of the AV1 Specification.
    - all other members of StdVideoAV1LoopRestoration are defined as in section 6.10.15 of the AV1 Specification;
- the members of the StdVideoAV1GlobalMotion structure provided in global_motion are interpreted as defined in section 7.10 of the AV1 Specification;
- if flags.film_grain_params_present is set in the active sequence header, then the StdVideoAV1FilmGrain structure pointed to by pFilmGrain is interpreted as follows:
    - flags.reserved is used only for padding purposes and is otherwise ignored;
    - all other members of StdVideoAV1FilmGrain are interpreted as defined in section 6.8.20 of the AV1 Specification;
- all other members are interpreted as defined in section 6.8 of the AV1 Specification.

When film grain is enabled for the decoded frame, the flags.update_grain and film_grain_params_ref_idx values specified in StdVideoAV1FilmGrain are ignored by AV1 decode operations and the load_grain_params function, as defined in section 6.8.20 of the AV1 Specification, is not executed. Instead, the application is responsible for specifying the effective film grain parameters for the frame in StdVideoAV1FilmGrain.

When film grain is enabled for the decoded frame, the application is required to specify a different decode output picture resource in VkVideoDecodeInfoKHR::dstPictureResource compared to the reconstructed picture specified in VkVideoDecodeInfoKHR::pSetupReferenceSlot->pPictureResource even if the implementation does not report support for VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR in VkVideoDecodeCapabilitiesKHR::flags for the video decode profile.

Reference picture setup is controlled by the value of StdVideoDecodeAV1PictureInfo::refresh_frame_flags. If it is not zero and a reconstructed picture is specified, then the latter is used as the target of picture reconstruction to activate the DPB slot specified in pDecodeInfo->pSetupReferenceSlot->slotIndex. If StdVideoDecodeAV1PictureInfo::refresh_frame_flags is zero, but a reconstructed picture is specified, then the corresponding picture reference associated with the DPB slot is invalidated, as described in the DPB Slot States section.

Active Parameter Sets

The *active sequence header* is the AV1 sequence header stored in the bound video session parameters object.

Valid Usage (Implicit)

- VUID-VkVideoDecodeAV1PictureInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_PICTURE_INFO_KHR`
- VUID-VkVideoDecodeAV1PictureInfoKHR-pStdPictureInfo-parameter
  `pStdPictureInfo` **must** be a valid pointer to a valid `StdVideoDecodeAV1PictureInfo` value
- VUID-VkVideoDecodeAV1PictureInfoKHR-pTileOffsets-parameter
  `pTileOffsets` **must** be a valid pointer to an array of `tileCount` `uint32_t` values
- VUID-VkVideoDecodeAV1PictureInfoKHR-pTileSizes-parameter
  `pTileSizes` **must** be a valid pointer to an array of `tileCount` `uint32_t` values
- VUID-VkVideoDecodeAV1PictureInfoKHR-tileCount-arraylength
  `tileCount` **must** be greater than `0`

The `VkVideoDecodeAV1DpbSlotInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_decode_av1
typedef struct VkVideoDecodeAV1DpbSlotInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoDecodeAV1ReferenceInfo*    pStdReferenceInfo;
} VkVideoDecodeAV1DpbSlotInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pStdReferenceInfo` is a pointer to a `StdVideoDecodeAV1ReferenceInfo` structure specifying AV1 reference information.

This structure is specified in the `pNext` chain of VkVideoDecodeInfoKHR::`pSetupReferenceSlot`, if not `NULL`, and the `pNext` chain of the elements of VkVideoDecodeInfoKHR::`pReferenceSlots` to specify the codec-specific reference picture information for an AV1 decode operation.

Active Reference Picture Information

When this structure is specified in the `pNext` chain of the elements of VkVideoDecodeInfoKHR ::`pReferenceSlots`, one element is added to the list of active reference pictures used by the video decode operation for each element of VkVideoDecodeInfoKHR::`pReferenceSlots` as follows:

- The image subregion used is determined according to the AV1 Decode Picture Data Access section.
- The reference picture is associated with the DPB slot index specified in the `slotIndex` member of the corresponding element of VkVideoDecodeInfoKHR::`pReferenceSlots`.

- The reference picture is associated with the AV1 reference information provided in pStdReferenceInfo.

Reconstructed Picture Information

When this structure is specified in the pNext chain of VkVideoDecodeInfoKHR ::pSetupReferenceSlot, the information related to the reconstructed picture is defined as follows:

- The image subregion used is determined according to the AV1 Decode Picture Data Access section.
- If reference picture setup is requested, then the reconstructed picture is used to activate the DPB slot with the index specified in VkVideoDecodeInfoKHR::pSetupReferenceSlot->slotIndex.
- The reconstructed picture is associated with the AV1 reference information provided in pStdReferenceInfo.

Std Reference Information

The members of the StdVideoDecodeAV1ReferenceInfo structure pointed to by pStdReferenceInfo are interpreted as follows:

- flags.reserved and reserved1 are used only for padding purposes and are otherwise ignored;
- flags.disable_frame_end_update_cdf is interpreted as defined in section 6.8.2 of the AV1 Specification;
- flags.segmentation_enabled is interpreted as defined in section 6.8.13 of the AV1 Specification;
- frame_type is interpreted as defined in section 6.8.2 of the AV1 Specification;

> **NOTE** The frame_type member is defined with the type uint8_t, but it takes the same values defined in the StdVideoAV1FrameType enumeration type as StdVideoDecodeAV1PictureInfo::frame_type.

- RefFrameSignBias is a bitmask where bit index i corresponds to RefFrameSignBias[i] as defined in section 6.8.2 of the AV1 Specification;
- OrderHint is interpreted as defined in section 6.8.2 of the AV1 Specification;
- SavedOrderHints is interpreted as defined in section 7.20 of the AV1 Specification.

> **NOTE** When the AV1 reference information is provided for the reconstructed picture, certain parameters (e.g. frame_type) are specified both in the AV1 picture information and in the AV1 reference information. This is necessary because unlike the AV1 picture information, which is only used for the purposes of the video decode operation in question, the AV1 reference information specified for the reconstructed picture **may** be associated with the activated DPB slot, meaning that some implementations **may** maintain it as part of the reference picture metadata corresponding to the video picture resource associated with the DPB slot. When the AV1 reference information is provided for an active reference picture, the specified parameters

correspond to the parameters specified when the DPB slot was activated (set up) with the reference picture, as usual, in order to communicate these parameters for implementations that do not maintain any subset of these parameters as part of the DPB slot's reference picture metadata.

Valid Usage (Implicit)

- VUID-VkVideoDecodeAV1DpbSlotInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_DPB_SLOT_INFO_KHR`
- VUID-VkVideoDecodeAV1DpbSlotInfoKHR-pStdReferenceInfo-parameter
  `pStdReferenceInfo` **must** be a valid pointer to a valid `StdVideoDecodeAV1ReferenceInfo` value

38.14.8. AV1 Decode Requirements

This section describes the **required** AV1 decoding capabilities for physical devices that have at least one queue family that supports the video codec operation `VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR`, as returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR in VkQueueFamilyVideoPropertiesKHR ::`videoCodecOperations`.

Table 51. Required Video Std Header Versions

Video Std Header Name	Version
`vulkan_video_codec_av1std_decode`	1.0.0

Table 52. Required Video Capabilities

Video Capability	Requirement	Requirement Type[1]
VkVideoCapabilitiesKHR		
`flags`	-	min
`minBitstreamBufferOffsetAlignment`	4096	max
`minBitstreamBufferSizeAlignment`	4096	max
`pictureAccessGranularity`	(64,64)	max
`minCodedExtent`	-	max
`maxCodedExtent`	-	min
`maxDpbSlots`	0	min
`maxActiveReferencePictures`	0	min
VkVideoDecodeCapabilitiesKHR		

Video Capability	Requirement	Requirement Type[1]
flags	VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_COINCIDE_BIT_KHR or VK_VIDEO_DECODE_CAPABILITY_DPB_AND_OUTPUT_DISTINCT_BIT_KHR	min
VkVideoDecodeAV1CapabilitiesKHR		
maxLevel	STD_VIDEO_AV1_LEVEL_2_0	min

1

The **Requirement Type** column specifies the requirement is either the minimum value all implementations **must** support, the maximum value all implementations **must** support, or the exact value all implementations **must** support. For bitmasks a minimum value is the least bits all implementations **must** set, but they **may** have additional bits set beyond this minimum.

38.15. Video Encode Operations

Video encode operations consume an *encode input picture* and zero or more reference pictures, and produce compressed video data to a video bitstream buffer and an optional reconstructed picture.

NOTE	Such encode input pictures can be used as the output of video decode operations, with graphics or compute operations, or with Window System Integration APIs, depending on the capabilities of the implementation.

Video encode operations **may** access the following resources in the VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR stage:

- The image subregions corresponding to the source encode input picture and active reference pictures with access VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR.
- The destination video bitstream buffer range and the optional reconstructed picture with access VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR.

The image subresource of each video picture resource accessed by the video encode operation is specified using a corresponding VkVideoPictureResourceInfoKHR structure. Each such image subresource **must** be in the appropriate image layout as follows:

- If the image subresource is used in the video encode operation as an encode input picture, then it **must** be in the VK_IMAGE_LAYOUT_VIDEO_ENCODE_SRC_KHR layout.
- If the image subresource is used in the video encode operation as a reconstructed picture or reference picture, then it **must** be in the VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR layout.
- If the image subresource is used in the video encode operation as a quantization map, then it **must** be in the VK_IMAGE_LAYOUT_VIDEO_ENCODE_QUANTIZATION_MAP_KHR layout.

A video encode operation **may** complete unsuccessfully. In this case the target video bitstream

buffer will have undefined contents. Similarly, if reference picture setup is requested, the reconstructed-picture will also have undefined contents, and the activated DPB slot will have an invalid picture reference.

If a video encode operation completes successfully and the codec-specific parameters provided by the application adhere to the syntactic and semantic requirements defined in the corresponding video compression standard, then the target video bitstream buffer will contain compressed video data after the execution of the video encode operation according to the respective codec-specific semantics.

38.15.1. Codec-Specific Semantics

The following aspects of video encode operations are codec-specific:

- The compressed video data written to the target video bitstream buffer range.
- The construction and interpretation of the list of active reference pictures and the interpretation of the picture data referred to by the corresponding image subregions.
- The construction and interpretation of information related to the encode input picture and the interpretation of the picture data referred to by the corresponding image subregion.
- The decision on reference picture setup.
- The construction and interpretation of information related to the optional reconstructed picture and the generation of picture data to the corresponding image subregion.
- Certain aspects of rate control.

These codec-specific behaviors are defined for each video codec operation separately.

- If the used video codec operation is `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR`, then the codec-specific aspects of the video encoding process are performed as defined in the H.264 Encode Operations section.
- If the used video codec operation is `VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR`, then the codec-specific aspects of the video encoding process are performed as defined in the H.265 Encode Operations section.
- If the used video codec operation is `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR`, then the codec-specific aspects of the video encoding process are performed as defined in the AV1 Encode Operations section.

Video Encode Parameter Overrides

Implementations supporting video encode operations for any particular video codec operation often support only a subset of the available encoding tools defined by the corresponding video compression standards. Accordingly, certain implementation-dependent limitations **may** apply to codec-specific parameters provided through the structures defined in the Video Std headers corresponding to the used video codec operation.

Exposing all of these restrictions on particular codec-specific parameter values or combinations thereof in the form of application-queryable capabilities is impractical, hence this specification allows implementations to *override* the value of any of the codec-specific parameters, unless

otherwise specified, as long as all of the following conditions are met:

- If the application-provided codec-specific parameters adhere to the syntactic and semantic requirements and rules defined by the used video compression standard, and thus would be usable to produce a video bitstream compliant with that standard, then the codec-specific parameters resulting from the process of implementation overrides **must** also adhere to the same requirements and rules, and any video bitstream produced using the overridden parameters **must** also be compliant.
- The overridden codec-specific parameter values **must** not have an impact on the codec-independent behaviors defined for video encode operations.
- The implementation **must** not override any codec-specific parameters specified to a command that **may** cause application-provided codec-specific parameters specified to subsequent commands to no longer adhere to the semantic requirements and rules defined by the used video compression standard, unless the implementation also overrides those parameters to adhere to any such requirements and rules.
- The overridden codec-specific parameter values **must** not have an impact on the codec-specific picture data access semantics.
- The overridden codec-specific parameter values **may** change the contents of the codec-specific bitstream elements produced by video encode operations or otherwise retrieved by the application (e.g. using the vkGetEncodedVideoSessionParametersKHR command) but **must** still adhere to the codec-specific semantics defined for that video codec operation, including, but not limited to, the number, type, and order of the encoded codec-specific bitstream elements.

Besides codec-specific parameter overrides performed for implementation-dependent reasons, applications **can** enable the implementation to apply additional optimizing overrides that **may** improve the efficiency or performance of video encoding operations. However, implementations **must** meet the conditions listed above even in case of such optimizing overrides.

NOTE	Unless the application opts in for optimizing overrides, implementations are not expected to override any of the codec-specific parameters, except when such overrides are necessary for the correct operation of video encoder implementation due to limitations to the available encoding tools on that implementation.

38.15.2. Video Encode Operation Steps

Each video encode operation performs the following steps in the `VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR` stage:

1. Reads the input picture data from the encode input picture;
2. Determine derived encoding quality parameters according to the codec-specific semantics and the current rate control state;
3. Compresses the input picture data according to the codec-specific semantics, applying any prediction data read from the active reference pictures and rate control restrictions in the process;
4. Writes the encoded bitstream data to the destination video bitstream buffer range;

5. Performs picture reconstruction of the encoded video data according to the codec-specific semantics, applying any prediction data read from the active reference pictures in the process, if a reconstructed picture is specified and reference picture setup is requested;
6. If reference picture setup is requested, the DPB slot index specified in the reconstructed picture information is activated with the reconstructed picture;
7. Writes the reconstructed picture data to the reconstructed picture, if one is specified, according to the codec-specific semantics.

When reconstructed picture information is provided, the specified DPB slot index is associated with the corresponding bound reference picture resource, indifferent of whether reference picture setup is requested.

38.15.3. Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR with `pVideoProfile->videoCodecOperation` specifying an encode operation, the VkVideoEncodeCapabilitiesKHR structure **must** be included in the `pNext` chain of the VkVideoCapabilitiesKHR structure to retrieve capabilities specific to video encoding.

The `VkVideoEncodeCapabilitiesKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeCapabilitiesKHR {
    VkStructureType                          sType;
    void*                                    pNext;
    VkVideoEncodeCapabilityFlagsKHR          flags;
    VkVideoEncodeRateControlModeFlagsKHR     rateControlModes;
    uint32_t                                 maxRateControlLayers;
    uint64_t                                 maxBitrate;
    uint32_t                                 maxQualityLevels;
    VkExtent2D                               encodeInputPictureGranularity;
    VkVideoEncodeFeedbackFlagsKHR            supportedEncodeFeedbackFlags;
} VkVideoEncodeCapabilitiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkVideoEncodeCapabilityFlagBitsKHR describing supported encoding features.
- `rateControlModes` is a bitmask of VkVideoEncodeRateControlModeFlagBitsKHR indicating supported rate control modes.
- `maxRateControlLayers` indicates the maximum number of rate control layers supported.
- `maxBitrate` indicates the maximum supported bitrate.
- `maxQualityLevels` indicates the number of discrete video encode quality levels supported. Implementations **must** support at least one quality level.
- `encodeInputPictureGranularity` indicates the granularity at which encode input picture data is

encoded and **may** indicate a texel granularity up to the size of the largest supported codec-specific coding block. This capability does not impose any valid usage constraints on the application, however, depending on the contents of the encode input picture, it **may** have effects on the encoded bitstream, as described in more detail below.

- `supportedEncodeFeedbackFlags` is a bitmask of VkVideoEncodeFeedbackFlagBitsKHR values specifying the supported flags for video encode feedback queries.

Implementations **must** include support for at least `VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BUFFER_OFFSET_BIT_KHR` and `VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BYTES_WRITTEN_BIT_KHR` in `supportedEncodeFeedbackFlags`.

`encodeInputPictureGranularity` provides information about the way encode input picture data is used as input to video encode operations. In particular, some implementations **may** not be able to limit the set of texels used to encode the output video bitstream to the image subregion specified in the VkVideoPictureResourceInfoKHR structure corresponding to the encode input picture (i.e. to the resolution of the image data to encode specified in its `codedExtent` member).

NOTE For example, the application requests the coded extent to be 1920x1080, but the implementation is only able to source the encode input picture data at the granularity of the codec-specific coding block size which is 16x16 pixels (or as otherwise indicated in `encodeInputPictureGranularity`). In this example, the content is horizontally aligned with the coding block size, but not vertically aligned with it. Thus encoding of the last row of coding blocks will be impacted by the contents of the input image at texel rows 1080 to 1087 (the latter being the next row which is vertically aligned with the coding block size, assuming a zero-based texel row index).

If `codedExtent` rounded up to the next integer multiple of `encodeInputPictureGranularity` is greater than the extent of the image subresource specified for the encode input picture, then the texel values corresponding to texel coordinates outside of the bounds of the image subresource **may** be undefined. However, implementations **should** use well-defined default values for such texels in order to maximize the encoding efficiency for the last coding block row/column, and/or to ensure consistent encoding results across repeated encoding of the same input content. Nonetheless, the values used for such texels **must** not have an effect on whether the video encode operation produces a compliant bitstream, and **must** not have any other effects on the encoded picture data beyond what **may** otherwise result from using these texel values as input to any compression algorithm, as defined in the used video compression standard.

NOTE While not required, it is generally a good practice for applications to make sure that the image subresource used for the encode input picture has an extent that is an integer multiple of the codec-specific coding block size (or at least `encodeInputPictureGranularity`) and that this padding area is filled with known values in order to improve encoding efficiency, portability, and reproducibility.

Valid Usage (Implicit)

- VUID-VkVideoEncodeCapabilitiesKHR-sType-sType

- `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_CAPABILITIES_KHR`

Bits which **may** be set in VkVideoEncodeCapabilitiesKHR::`flags`, indicating the encoding tools supported, are:

```
// Provided by VK_KHR_video_encode_queue
typedef enum VkVideoEncodeCapabilityFlagBitsKHR {
    VK_VIDEO_ENCODE_CAPABILITY_PRECEDING_EXTERNALLY_ENCODED_BYTES_BIT_KHR =
0x00000001,
    VK_VIDEO_ENCODE_CAPABILITY_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_DETECTION_BIT_KHR =
0x00000002,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR = 0x00000004,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR = 0x00000008,
} VkVideoEncodeCapabilityFlagBitsKHR;
```

- `VK_VIDEO_ENCODE_CAPABILITY_PRECEDING_EXTERNALLY_ENCODED_BYTES_BIT_KHR` specifies that the implementation supports the use of VkVideoEncodeInfoKHR::`precedingExternallyEncodedBytes`.
- `VK_VIDEO_ENCODE_CAPABILITY_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_DETECTION_BIT_KHR` specifies that the implementation is able to detect and report when the destination video bitstream buffer range provided by the application is not sufficiently large to fit the encoded bitstream data produced by a video encode operation by reporting the `VK_QUERY_RESULT_STATUS_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_KHR` query result status code.

> **NOTE**
>
> Some implementations **may** not be able to reliably detect insufficient bitstream buffer range conditions in all situations. Such implementations will not report support for the `VK_VIDEO_ENCODE_CAPABILITY_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_DETECTION_BIT_KHR` encode capability flag for the video profile, but **may** still report the `VK_QUERY_RESULT_STATUS_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_KHR` query result status code in certain cases. Applications **should** always check for the specific query result status code `VK_QUERY_RESULT_STATUS_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_KHR` even when this encode capability flag is not supported by the implementation for the video profile in question. However, applications **must** not assume that a different negative query result status code indicating an unsuccessful completion of a video encode operation is not the result of an insufficient bitstream buffer condition unless this encode capability flag is supported.

- `VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR` indicates support for using quantization delta maps.
- `VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR` specifies support for using emphasis maps.

```
// Provided by VK_KHR_video_encode_queue
typedef VkFlags VkVideoEncodeCapabilityFlagsKHR;
```

VkVideoEncodeCapabilityFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeCapabilityFlagBitsKHR.

38.15.4. Video Encode Quality Levels

Implementations **can** support more than one video encode quality levels for a video encode profile, which control the number and type of implementation-specific encoding tools and algorithms utilized in the encoding process.

NOTE

Generally, using higher video encode quality levels **may** produce higher quality video streams at the cost of additional processing time. However, as the final quality of an encoded picture depends on the contents of the encode input picture, the contents of the active reference pictures, the codec-specific encode parameters, and the particular implementation-specific tools used corresponding to the individual video encode quality levels, there are no guarantees that using a higher video encode quality level will always produce a higher quality encoded picture for any given set of inputs.

To query properties for a specific video encode quality level supported by a video encode profile, call:

```
// Provided by VK_KHR_video_encode_queue
VkResult vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR* pQualityLevelInfo,
    VkVideoEncodeQualityLevelPropertiesKHR*     pQualityLevelProperties);
```

- physicalDevice is the physical device to query the video encode quality level properties for.
- pQualityLevelInfo is a pointer to a VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR structure specifying the video encode profile and quality level to query properties for.
- pQualityLevelProperties is a pointer to a VkVideoEncodeQualityLevelPropertiesKHR structure in which the properties are returned.

Valid Usage

- VUID-vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR-pQualityLevelInfo-08257
  If pQualityLevelInfo->pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the pNext chain of pQualityLevelProperties **must** include a VkVideoEncodeH264QualityLevelPropertiesKHR structure
- VUID-vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR-pQualityLevelInfo-

08258
If pQualityLevelInfo->pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the pNext chain of pQualityLevelProperties **must** include a VkVideoEncodeH265QualityLevelPropertiesKHR structure

- VUID-vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR-pQualityLevelInfo-10305
If pQualityLevelInfo->pVideoProfile->videoCodecOperation is VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the pNext chain of pQualityLevelProperties **must** include a VkVideoEncodeAV1QualityLevelPropertiesKHR structure

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR-physicalDevice-parameter
physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR-pQualityLevelInfo-parameter
pQualityLevelInfo **must** be a valid pointer to a valid VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR structure
- VUID-vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR-pQualityLevelProperties-parameter
pQualityLevelProperties **must** be a valid pointer to a VkVideoEncodeQualityLevelPropertiesKHR structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_VIDEO_PROFILE_OPERATION_NOT_SUPPORTED_KHR
- VK_ERROR_VIDEO_PROFILE_FORMAT_NOT_SUPPORTED_KHR
- VK_ERROR_VIDEO_PICTURE_LAYOUT_NOT_SUPPORTED_KHR
- VK_ERROR_VIDEO_PROFILE_CODEC_NOT_SUPPORTED_KHR

The VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    const VkVideoProfileInfoKHR*    pVideoProfile;
    uint32_t                        qualityLevel;
} VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pVideoProfile` is a pointer to a VkVideoProfileInfoKHR structure specifying the video profile to query the video encode quality level properties for.
- `qualityLevel` is the video encode quality level to query properties for.

Valid Usage

- VUID-VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR-pVideoProfile-08259
  `pVideoProfile` **must** be a supported video profile
- VUID-VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR-pVideoProfile-08260
  `pVideoProfile->videoCodecOperation` **must** specify an encode operation
- VUID-VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR-qualityLevel-08261
  `qualityLevel` **must** be less than VkVideoEncodeCapabilitiesKHR::`maxQualityLevels`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile specified in `pVideoProfile`

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_QUALITY_LEVEL_INFO_KHR`
- VUID-VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR-pVideoProfile-parameter
  `pVideoProfile` **must** be a valid pointer to a valid VkVideoProfileInfoKHR structure

The `VkVideoEncodeQualityLevelPropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeQualityLevelPropertiesKHR {
    VkStructureType                           sType;
    void*                                     pNext;
    VkVideoEncodeRateControlModeFlagBitsKHR   preferredRateControlMode;
    uint32_t                                  preferredRateControlLayerCount;
} VkVideoEncodeQualityLevelPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `preferredRateControlMode` is a VkVideoEncodeRateControlModeFlagBitsKHR value indicating the preferred rate control mode to use with the video encode quality level.
- `preferredRateControlLayerCount` indicates the preferred number of rate control layers to use with the video encode quality level.

Valid Usage (Implicit)

- VUID-VkVideoEncodeQualityLevelPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUALITY_LEVEL_PROPERTIES_KHR`
- VUID-VkVideoEncodeQualityLevelPropertiesKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkVideoEncodeAV1QualityLevelPropertiesKHR, VkVideoEncodeH264QualityLevelPropertiesKHR, or VkVideoEncodeH265QualityLevelPropertiesKHR
- VUID-VkVideoEncodeQualityLevelPropertiesKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

The `VkVideoEncodeQualityLevelInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeQualityLevelInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint32_t           qualityLevel;
} VkVideoEncodeQualityLevelInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `qualityLevel` is the used video encode quality level.

This structure **can** be specified in the following places:

- In the `pNext` chain of VkVideoSessionParametersCreateInfoKHR to specify the video encode

quality level to use for a video session parameters object created for a video encode session. If no instance of this structure is included in the pNext chain of VkVideoSessionParametersCreateInfoKHR, then the video session parameters object is created with a video encode quality level of zero.

- In the pNext chain of VkVideoCodingControlInfoKHR to change the video encode quality level state of the bound video session.

Valid Usage

- VUID-VkVideoEncodeQualityLevelInfoKHR-qualityLevel-08311
  qualityLevel **must** be less than VkVideoEncodeCapabilitiesKHR::maxQualityLevels, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile

Valid Usage (Implicit)

- VUID-VkVideoEncodeQualityLevelInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUALITY_LEVEL_INFO_KHR

38.15.5. Retrieving Encoded Session Parameters

Any codec-specific parameters stored in video session parameters objects **may** need to be separately encoded and included in the final video bitstream data, depending on the used video compression standard. In such cases the application **must** call the vkGetEncodedVideoSessionParametersKHR command to retrieve the encoded parameter data from the used video session parameters object in order to be able to produce a compliant video bitstream.

NOTE

This is needed because implementations **may** have changed some of the codec-specific parameters stored in the video session parameters object, as defined in the Video Encode Parameter Overrides section. In addition, the vkGetEncodedVideoSessionParametersKHR command enables the application to retrieve the encoded parameter data without having to encode these codec-specific parameters manually.

Encoded parameter data **can** be retrieved from a video session parameters object created with a video encode operation using the command:

```
// Provided by VK_KHR_video_encode_queue
VkResult vkGetEncodedVideoSessionParametersKHR(
    VkDevice                                    device,
    const VkVideoEncodeSessionParametersGetInfoKHR* pVideoSessionParametersInfo,
    VkVideoEncodeSessionParametersFeedbackInfoKHR* pFeedbackInfo,
    size_t*                                     pDataSize,
    void*                                       pData);
```

- `device` is the logical device that owns the video session parameters object.
- `pVideoSessionParametersInfo` is a pointer to a VkVideoEncodeSessionParametersGetInfoKHR structure specifying the parameters of the encoded parameter data to retrieve.
- `pFeedbackInfo` is either `NULL` or a pointer to a VkVideoEncodeSessionParametersFeedbackInfoKHR structure in which feedback about the requested parameter data is returned.
- `pDataSize` is a pointer to a `size_t` value related to the amount of encode parameter data returned, as described below.
- `pData` is either `NULL` or a pointer to a buffer to write the encoded parameter data to.

If `pData` is `NULL`, then the size of the encoded parameter data, in bytes, that **can** be retrieved is returned in `pDataSize`. Otherwise, `pDataSize` **must** point to a variable set by the application to the size of the buffer, in bytes, pointed to by `pData`, and on return the variable is overwritten with the number of bytes actually written to `pData`. If `pDataSize` is less than the size of the encoded parameter data that **can** be retrieved, then no data will be written to `pData`, zero will be written to `pDataSize`, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that no encoded parameter data was returned.

If `pFeedbackInfo` is not `NULL` then the members of the VkVideoEncodeSessionParametersFeedbackInfoKHR structure and any additional structures included in its `pNext` chain that are applicable to the video session parameters object specified in `pVideoSessionParametersInfo->videoSessionParameters` will be filled with feedback about the requested parameter data on all successful calls to this command.

NOTE This includes the cases when `pData` is `NULL` or when `VK_INCOMPLETE` is returned by the command, and enables the application to determine whether the implementation overrode any of the requested video session parameters without actually needing to retrieve the encoded parameter data itself.

Valid Usage

- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-08359 `pVideoSessionParametersInfo->videoSessionParameters` **must** have been created with an encode operation
- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-08262 If `pVideoSessionParametersInfo->videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR`, then the `pNext` chain of `pVideoSessionParametersInfo` **must** include a VkVideoEncodeH264SessionParametersGetInfoKHR structure
- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-08263 If `pVideoSessionParametersInfo->videoSessionParameters` was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR`, then for the VkVideoEncodeH264SessionParametersGetInfoKHR structure included in the `pNext` chain of `pVideoSessionParametersInfo`, if its `writeStdSPS` member is `VK_TRUE`, then `pVideoSessionParametersInfo->videoSessionParameters` **must** contain a

StdVideoH264SequenceParameterSet entry with seq_parameter_set_id matching VkVideoEncodeH264SessionParametersGetInfoKHR::stdSPSId

- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-08264
  If pVideoSessionParametersInfo->videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then for the VkVideoEncodeH264SessionParametersGetInfoKHR structure included in the pNext chain of pVideoSessionParametersInfo, if its writeStdPPS member is VK_TRUE, then pVideoSessionParametersInfo->videoSessionParameters **must** contain a StdVideoH264PictureParameterSet entry with seq_parameter_set_id and pic_parameter_set_id matching VkVideoEncodeH264SessionParametersGetInfoKHR::stdSPSId and VkVideoEncodeH264SessionParametersGetInfoKHR::stdPPSId, respectively
- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-08265
  If pVideoSessionParametersInfo->videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the pNext chain of pVideoSessionParametersInfo **must** include a VkVideoEncodeH265SessionParametersGetInfoKHR structure
- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-08266
  If pVideoSessionParametersInfo->videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then for the VkVideoEncodeH265SessionParametersGetInfoKHR structure included in the pNext chain of pVideoSessionParametersInfo, if its writeStdVPS member is VK_TRUE, then pVideoSessionParametersInfo->videoSessionParameters **must** contain a StdVideoH265VideoParameterSet entry with vps_video_parameter_set_id matching VkVideoEncodeH265SessionParametersGetInfoKHR::stdVPSId
- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-08267
  If pVideoSessionParametersInfo->videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then for the VkVideoEncodeH265SessionParametersGetInfoKHR structure included in the pNext chain of pVideoSessionParametersInfo, if its writeStdSPS member is VK_TRUE, then pVideoSessionParametersInfo->videoSessionParameters **must** contain a StdVideoH265SequenceParameterSet entry with sps_video_parameter_set_id and sps_seq_parameter_set_id matching VkVideoEncodeH265SessionParametersGetInfoKHR::stdVPSId and VkVideoEncodeH265SessionParametersGetInfoKHR::stdSPSId, respectively
- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-08268
  If pVideoSessionParametersInfo->videoSessionParameters was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then for the VkVideoEncodeH265SessionParametersGetInfoKHR structure included in the pNext chain of pVideoSessionParametersInfo, if its writeStdPPS member is VK_TRUE, then pVideoSessionParametersInfo->videoSessionParameters **must** contain a StdVideoH265PictureParameterSet entry with sps_video_parameter_set_id, pps_seq_parameter_set_id, and pps_pic_parameter_set_id matching VkVideoEncodeH265SessionParametersGetInfoKHR::stdVPSId, VkVideoEncodeH265SessionParametersGetInfoKHR::stdSPSId, and VkVideoEncodeH265SessionParametersGetInfoKHR::stdPPSId, respectively

Valid Usage (Implicit)

- VUID-vkGetEncodedVideoSessionParametersKHR-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkGetEncodedVideoSessionParametersKHR-pVideoSessionParametersInfo-parameter
  `pVideoSessionParametersInfo` **must** be a valid pointer to a valid VkVideoEncodeSessionParametersGetInfoKHR structure
- VUID-vkGetEncodedVideoSessionParametersKHR-pFeedbackInfo-parameter
  If `pFeedbackInfo` is not `NULL`, `pFeedbackInfo` **must** be a valid pointer to a VkVideoEncodeSessionParametersFeedbackInfoKHR structure
- VUID-vkGetEncodedVideoSessionParametersKHR-pDataSize-parameter
  `pDataSize` **must** be a valid pointer to a `size_t` value
- VUID-vkGetEncodedVideoSessionParametersKHR-pData-parameter
  If the value referenced by `pDataSize` is not `0`, and `pData` is not `NULL`, `pData` **must** be a valid pointer to an array of `pDataSize` bytes

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkVideoEncodeSessionParametersGetInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeSessionParametersGetInfoKHR {
    VkStructureType                sType;
    const void*                    pNext;
    VkVideoSessionParametersKHR    videoSessionParameters;
} VkVideoEncodeSessionParametersGetInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `videoSessionParameters` is the VkVideoSessionParametersKHR object to retrieve encoded parameter data from.

Depending on the used video encode operation, additional codec-specific structures **may** need to be

included in the pNext chain of this structure to identify the specific video session parameters to retrieve encoded parameter data for, as described in the corresponding sections.

Valid Usage (Implicit)

- VUID-VkVideoEncodeSessionParametersGetInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_SESSION_PARAMETERS_GET_INFO_KHR
- VUID-VkVideoEncodeSessionParametersGetInfoKHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkVideoEncodeH264SessionParametersGetInfoKHR or VkVideoEncodeH265SessionParametersGetInfoKHR
- VUID-VkVideoEncodeSessionParametersGetInfoKHR-sType-unique
  The sType value of each struct in the pNext chain **must** be unique
- VUID-VkVideoEncodeSessionParametersGetInfoKHR-videoSessionParameters-parameter
  videoSessionParameters **must** be a valid VkVideoSessionParametersKHR handle

The VkVideoEncodeSessionParametersFeedbackInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeSessionParametersFeedbackInfoKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           hasOverrides;
} VkVideoEncodeSessionParametersFeedbackInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- hasOverrides indicates whether any of the requested parameter data were overridden by the implementation.

Depending on the used video encode operation, additional codec-specific structures **can** be included in the pNext chain of this structure to capture codec-specific feedback information about the requested parameter data, as described in the corresponding sections.

Valid Usage (Implicit)

- VUID-VkVideoEncodeSessionParametersFeedbackInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_SESSION_PARAMETERS_FEEDBACK_INFO_KHR
- VUID-VkVideoEncodeSessionParametersFeedbackInfoKHR-pNext-pNext
  Each pNext member of any structure (including this one) in the pNext chain **must** be either NULL or a pointer to a valid instance of VkVideoEncodeH264SessionParametersFeedbackInfoKHR or VkVideoEncodeH265SessionParametersFeedbackInfoKHR

- VUID-VkVideoEncodeSessionParametersFeedbackInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

38.15.6. Video Encode Commands

To launch video encode operations, call:

```
// Provided by VK_KHR_video_encode_queue
void vkCmdEncodeVideoKHR(
    VkCommandBuffer                             commandBuffer,
    const VkVideoEncodeInfoKHR*                 pEncodeInfo);
```

- `commandBuffer` is the command buffer in which to record the command.
- `pEncodeInfo` is a pointer to a VkVideoEncodeInfoKHR structure specifying the parameters of the video encode operations.

Each call issues one or more video encode operations. The implicit parameter `opCount` corresponds to the number of video encode operations issued by the command. After calling this command, the active query index of each active query is incremented by `opCount`.

Currently each call to this command results in the issue of a single video encode operation.

If the bound video session was created with `VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR` and the `pNext` chain of `pEncodeInfo` includes a VkVideoInlineQueryInfoKHR structure with its `queryPool` member specifying a valid `VkQueryPool` handle, then this command will execute a query for each video encode operation issued by it.

Active Reference Picture Information

The list of active reference pictures used by a video encode operation is a list of image subregions used as the source of reference picture data and related parameters, and is derived from the VkVideoReferenceSlotInfoKHR structures provided as the elements of the `pEncodeInfo->pReferenceSlots` array. For each element of `pEncodeInfo->pReferenceSlots`, one or more elements are added to the active reference picture list, as defined by the codec-specific semantics. Each element of this list contains the following information:

- The image subregion within the image subresource referred to by the video picture resource used as the reference picture.
- The DPB slot index the reference picture is associated with.
- The codec-specific reference information related to the reference picture.

Reconstructed Picture Information

Information related to the optional reconstructed picture used by a video encode operation is derived from the VkVideoReferenceSlotInfoKHR structure pointed to by `pEncodeInfo->pSetupReferenceSlot`, if not `NULL`, as defined by the codec-specific semantics, and consists of the following:

- The image subregion within the image subresource referred to by the video picture resource used as the reconstructed picture.
- The DPB slot index to use for picture reconstruction.
- The codec-specific reference information related to the reconstructed picture.

Specifying a valid VkVideoReferenceSlotInfoKHR structure in `pEncodeInfo->pSetupReferenceSlot` is always required, unless the video session was created with VkVideoSessionCreateInfoKHR::`maxDpbSlot` equal to zero. However, the DPB slot identified by `pEncodeInfo->pSetupReferenceSlot->slotIndex` is only activated with the reconstructed picture specified in `pEncodeInfo->pSetupReferenceSlot->pPictureResource` if reference picture setup is requested according to the codec-specific semantics.

If reconstructed picture information is specified, but reference picture setup is not requested, according to the codec-specific semantics, the contents of the video picture resource corresponding to the reconstructed picture will be undefined after the video encode operation.

NOTE	Some implementations may always output the reconstructed picture or use it as temporary storage during the video encode operation even when the reconstructed picture is not marked for future reference.

Encode Input Picture Information

Information related to the encode input picture used by a video encode operation is derived from `pEncodeInfo->srcPictureResource` and any codec-specific parameters provided in the `pEncodeInfo->pNext` chain, as defined by the codec-specific semantics, and consists of the following:

- The image subregion within the image subresource referred to by the video picture resource used as the encode input picture.
- The codec-specific picture information related to the encoded picture.

Several limiting values are defined below that are referenced by the relevant valid usage statements of this command.

- Let `uint32_t activeReferencePictureCount` be the size of the list of active reference pictures used by the video encode operation. Unless otherwise defined, `activeReferencePictureCount` is set to the value of `pEncodeInfo->referenceSlotCount`.
- Let `VkOffset2D codedOffsetGranularity` be the minimum alignment requirement for the coded offset of video picture resources. Unless otherwise defined, the value of the `x` and `y` members of `codedOffsetGranularity` are `0`.
- Let `uint32_t dpbFrameUseCount[]` be an array of size `maxDpbSlots`, where `maxDpbSlots` is the VkVideoSessionCreateInfoKHR::`maxDpbSlots` the bound video session was created with, with each element indicating the number of times a frame associated with the corresponding DPB slot index is referred to by the video coding operation. Let the initial value of each element of the array be `0`.
  - If `pEncodeInfo->pSetupReferenceSlot` is not `NULL`, then `dpbFrameUseCount[i]` is incremented by one, where `i` equals `pEncodeInfo->pSetupReferenceSlot->slotIndex`.

    - For each element of `pEncodeInfo->pReferenceSlots`, `dpbFrameUseCount[i]` is incremented by one, where `i` equals the `slotIndex` member of the corresponding element.
- If there is a bound video session parameters object created with `VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR`, then let `VkExtent2D quantizationMapTexelSize` be the quantization map texel size the bound video session parameters object was created with.
- Let `VkExtent2D maxCodingBlockSize` be the maximum codec-specific coding block size that **may** be used by the video encode operation.
    - If the bound video session object was created with an H.264 encode profile, then let `maxCodingBlockSize` be equal to the size of an H.264 macroblock, i.e. `{16,16}`.
    - If the bound video session object was created with an H.265 encode profile, then let `maxCodingBlockSize` be equal to the maximum H.265 coding block size that **may** be used by the video encode operation derived as the maximum of the CTB sizes corresponding to the VkVideoEncodeH265CtbSizeFlagBitsKHR bits set in VkVideoEncodeH265CapabilitiesKHR::`ctbSizes`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with.
    - If the bound video session object was created with an AV1 encode profile, then let `maxCodingBlockSize` be equal to the maximum AV1 superblock size that **may** be used by the video encode operation derived as the maximum of the superblock sizes corresponding to the VkVideoEncodeAV1SuperblockSizeFlagBitsKHR bits set in VkVideoEncodeAV1CapabilitiesKHR::`superblockSizes`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with.
    - Otherwise, `maxCodingBlockSize` is undefined.
- If `maxCodingBlockSize` is defined, then let `VkExtent2D minCodingBlockExtent` be the coded extent of the encode input picture expressed in terms of codec-specific coding blocks, assuming the maximum size of such coding blocks, as defined by `maxCodingBlockSize`, calculated from the value of the `codedExtent` member of `pEncodeInfo->srcPictureResource` as follows:
    - `minCodingBlockExtent.width` = (`codedExtent.width` `maxCodingBlockSize.width` - 1) / `maxCodingBlockSize.width`
    - `minCodingBlockExtent.height` = (`codedExtent.height` `maxCodingBlockSize.height` - 1) / `maxCodingBlockSize.height`
- If the bound video session object was created with an H.264 encode profile, then:
    - Let `StdVideoH264PictureType h264PictureType` be the picture type of the encoded picture set to the value of `pStdPictureInfo->primary_pic_type` specified in the VkVideoEncodeH264PictureInfoKHR structure included in the `pEncodeInfo->pNext` chain.
    - Let `StdVideoH264PictureType h264L0PictureTypes[]` and `StdVideoH264PictureType h264L1PictureTypes[]` be the picture types of the reference pictures in the L0 and L1 reference lists, respectively. If `pStdPictureInfo->pRefLists` specified in the VkVideoEncodeH264PictureInfoKHR structure included in the `pEncodeInfo->pNext` chain is not `NULL`, then for each reference index specified in the elements of the `pStdPictureInfo->pRefLists->RefPicList0` and `pStdPictureInfo->pRefLists->RefPicList1` arrays, if the reference index is not `STD_VIDEO_H264_NO_REFERENCE_PICTURE`, `pStdReferenceInfo-`

>primary_pic_type is added to h264L0PictureTypes or h264L1PictureTypes, respectively, where pStdReferenceInfo is the member of the VkVideoEncodeH264DpbSlotInfoKHR structure included in the pNext chain of the element of pEncodeInfo->pReferenceSlots for which slotIndex equals the reference index in question.

- If the bound video session object was created with an H.265 encode profile, then:
  - Let StdVideoH265PictureType h265PictureType be the picture type of the encoded picture set to the value of pStdPictureInfo->pic_type specified in the VkVideoEncodeH265PictureInfoKHR structure included in the pEncodeInfo->pNext chain.
  - Let StdVideoH265PictureType h265L0PictureTypes[] and StdVideoH265PictureType h265L1PictureTypes[] be the picture types of the reference pictures in the L0 and L1 reference lists, respectively. If pStdPictureInfo->pRefLists specified in the VkVideoEncodeH265PictureInfoKHR structure included in the pEncodeInfo->pNext chain is not NULL, then for each reference index specified in the elements of the pStdPictureInfo->pRefLists->RefPicList0 and pStdPictureInfo->pRefLists->RefPicList1 arrays, if the reference index is not STD_VIDEO_H265_NO_REFERENCE_PICTURE, pStdReferenceInfo->pic_type is added to h265L0PictureTypes or h265L1PictureTypes, respectively, where pStdReferenceInfo is the member of the VkVideoEncodeH265DpbSlotInfoKHR structure included in the pNext chain of the element of pEncodeInfo->pReferenceSlots for which slotIndex equals the reference index in question.
- If the bound video session object was created with an AV1 encode profile, then:
  - If the primaryReferenceCdfOnly member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pEncodeInfo->pNext chain is set to VK_TRUE, then let int32_t cdfOnlyReferenceIndex be the value of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo->primary_ref_frame.
  - Otherwise let int32_t cdfOnlyReferenceIndex be -1.

Valid Usage

- VUID-vkCmdEncodeVideoKHR-None-08250
  The bound video session **must** have been created with an encode operation
- VUID-vkCmdEncodeVideoKHR-None-07012
  The bound video session **must** not be in uninitialized state at the time the command is executed on the device
- VUID-vkCmdEncodeVideoKHR-None-08318
  The bound video session parameters object **must** have been created with the currently set video encode quality level for the bound video session at the time the command is executed on the device
- VUID-vkCmdEncodeVideoKHR-opCount-07174
  For each active query, the active query index corresponding to the query type of that query plus opCount **must** be less than or equal to the last activatable query index corresponding to the query type of that query plus one
- VUID-vkCmdEncodeVideoKHR-pNext-08360
  If the bound video session was created with

VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, and the pNext chain of pEncodeInfo includes a VkVideoInlineQueryInfoKHR structure with its queryPool member specifying a valid VkQueryPool handle, then VkVideoInlineQueryInfoKHR::queryCount **must** equal opCount

- VUID-vkCmdEncodeVideoKHR-pNext-08361
  If the bound video session was created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, and the pNext chain of pEncodeInfo includes a VkVideoInlineQueryInfoKHR structure with its queryPool member specifying a valid VkQueryPool handle, then all the queries used by the command, as specified by the VkVideoInlineQueryInfoKHR structure, **must** be *unavailable*
- VUID-vkCmdEncodeVideoKHR-queryType-08362
  If the bound video session was created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, then the queryType used to create the queryPool specified in the VkVideoInlineQueryInfoKHR structure included in the pNext chain of pEncodeInfo **must** be VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR or VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR
- VUID-vkCmdEncodeVideoKHR-queryPool-08363
  If the bound video session was created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, then the queryPool specified in the VkVideoInlineQueryInfoKHR structure included in the pNext chain of pEncodeInfo **must** have been created with a VkVideoProfileInfoKHR structure included in the pNext chain of VkQueryPoolCreateInfo identical to the one specified in VkVideoSessionCreateInfoKHR ::pVideoProfile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-queryType-08364
  If the bound video session was created with VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR, and the queryType used to create the queryPool specified in the VkVideoInlineQueryInfoKHR structure included in the pNext chain of pEncodeInfo is VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR, then the VkCommandPool that commandBuffer was allocated from **must** have been created with a queue family index that supports result status queries, as indicated by VkQueueFamilyQueryResultStatusPropertiesKHR::queryResultStatusSupport
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08201
  pEncodeInfo->dstBuffer **must** be compatible with the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-commandBuffer-08202
  If commandBuffer is an unprotected command buffer and protectedNoFault is not supported, then pEncodeInfo->dstBuffer **must** not be a protected buffer
- VUID-vkCmdEncodeVideoKHR-commandBuffer-08203
  If commandBuffer is a protected command buffer and protectedNoFault is not supported, then pEncodeInfo->dstBuffer **must** be a protected buffer
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08204
  pEncodeInfo->dstBufferOffset **must** be an integer multiple of VkVideoCapabilitiesKHR ::minBitstreamBufferOffsetAlignment, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with

- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08205
  pEncodeInfo->dstBufferRange **must** be an integer multiple of VkVideoCapabilitiesKHR::minBitstreamBufferSizeAlignment, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08206
  pEncodeInfo->srcPictureResource.imageViewBinding **must** be compatible with the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08207
  The format of pEncodeInfo->srcPictureResource.imageViewBinding **must** match the VkVideoSessionCreateInfoKHR::pictureFormat the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08208
  pEncodeInfo->srcPictureResource.codedOffset **must** be an integer multiple of codedOffsetGranularity
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08209
  pEncodeInfo->srcPictureResource.codedExtent **must** be between minCodedExtent and maxCodedExtent, inclusive, the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08210
  pEncodeInfo->srcPictureResource.imageViewBinding **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR
- VUID-vkCmdEncodeVideoKHR-commandBuffer-08211
  If commandBuffer is an unprotected command buffer and protectedNoFault is not supported, then pEncodeInfo->srcPictureResource.imageViewBinding **must** not have been created from a protected image
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08377
  pEncodeInfo->pSetupReferenceSlot **must** not be NULL unless the bound video session was created with VkVideoSessionCreateInfoKHR::maxDpbSlots equal to zero
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08213
  If pEncodeInfo->pSetupReferenceSlot is not NULL, then pEncodeInfo->pSetupReferenceSlot->slotIndex **must** be less than the VkVideoSessionCreateInfoKHR::maxDpbSlots specified when the bound video session was created
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08214
  If pEncodeInfo->pSetupReferenceSlot is not NULL, then pEncodeInfo->pSetupReferenceSlot->pPictureResource->codedOffset **must** be an integer multiple of codedOffsetGranularity
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08215
  If pEncodeInfo->pSetupReferenceSlot is not NULL, then pEncodeInfo->pSetupReferenceSlot->pPictureResource **must** match one of the bound reference picture resource
- VUID-vkCmdEncodeVideoKHR-activeReferencePictureCount-08216
  activeReferencePictureCount **must** be less than or equal to the VkVideoSessionCreateInfoKHR::maxActiveReferencePictures specified when the bound video session was created
- VUID-vkCmdEncodeVideoKHR-slotIndex-08217
  The slotIndex member of each element of pEncodeInfo->pReferenceSlots **must** be less than

the VkVideoSessionCreateInfoKHR::maxDpbSlots specified when the bound video session was created

- VUID-vkCmdEncodeVideoKHR-codedOffset-08218
  The codedOffset member of the VkVideoPictureResourceInfoKHR structure pointed to by the pPictureResource member of each element of pEncodeInfo->pReferenceSlots **must** be an integer multiple of codedOffsetGranularity
- VUID-vkCmdEncodeVideoKHR-pPictureResource-08219
  The pPictureResource member of each element of pEncodeInfo->pReferenceSlots **must** match one of the bound reference picture resource associated with the DPB slot index specified in the slotIndex member of that element
- VUID-vkCmdEncodeVideoKHR-pPictureResource-08220
  Each video picture resource corresponding to the pPictureResource member specified in the elements of pEncodeInfo->pReferenceSlots **must** be unique within pEncodeInfo->pReferenceSlots
- VUID-vkCmdEncodeVideoKHR-dpbFrameUseCount-08221
  All elements of dpbFrameUseCount **must** be less than or equal to 1
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08222
  The image subresource referred to by pEncodeInfo->srcPictureResource **must** be in the VK_IMAGE_LAYOUT_VIDEO_ENCODE_SRC_KHR layout at the time the video encode operation is executed on the device
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08223
  If pEncodeInfo->pSetupReferenceSlot is not NULL, then the image subresource referred to by pEncodeInfo->pSetupReferenceSlot->pPictureResource **must** be in the VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR layout at the time the video encode operation is executed on the device
- VUID-vkCmdEncodeVideoKHR-pPictureResource-08224
  The image subresource referred to by the pPictureResource member of each element of pEncodeInfo->pReferenceSlots **must** be in the VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR layout at the time the video encode operation is executed on the device
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10306
  If pEncodeInfo->flags includes VK_VIDEO_ENCODE_WITH_QUANTIZATION_DELTA_MAP_BIT_KHR, then the bound video session **must** have been created with VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10307
  If pEncodeInfo->flags includes VK_VIDEO_ENCODE_WITH_EMPHASIS_MAP_BIT_KHR, then the bound video session **must** have been created with VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10308
  If the current rate control mode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR or VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, then pEncodeInfo->flags **must** not include VK_VIDEO_ENCODE_WITH_EMPHASIS_MAP_BIT_KHR
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10309
  If pEncodeInfo->flags includes VK_VIDEO_ENCODE_WITH_QUANTIZATION_DELTA_MAP_BIT_KHR or

VK_VIDEO_ENCODE_WITH_EMPHASIS_MAP_BIT_KHR, then the pNext chain of pEncodeInfo **must** include a VkVideoEncodeQuantizationMapInfoKHR structure with its quantizationMap member specifying a valid VkImageView handle

- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10310
  If pEncodeInfo->flags includes VK_VIDEO_ENCODE_WITH_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_VIDEO_ENCODE_WITH_EMPHASIS_MAP_BIT_KHR, then the VkImageView specified by the quantizationMap member of the VkVideoEncodeQuantizationMapInfoKHR structure included in the pNext chain **must** be compatible with the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10311
  If pEncodeInfo->flags includes VK_VIDEO_ENCODE_WITH_QUANTIZATION_DELTA_MAP_BIT_KHR, then the VkImageView specified by the quantizationMap member of the VkVideoEncodeQuantizationMapInfoKHR structure included in the pNext chain of pEncodeInfo **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10312
  If pEncodeInfo->flags includes VK_VIDEO_ENCODE_WITH_EMPHASIS_MAP_BIT_KHR, then the VkImageView specified by the quantizationMap member of the VkVideoEncodeQuantizationMapInfoKHR structure included in the pNext chain of pEncodeInfo **must** have been created with VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR
- VUID-vkCmdEncodeVideoKHR-pNext-10313
  If an instance of the VkVideoEncodeQuantizationMapInfoKHR structure is included in the pNext chain of pEncodeInfo, its quantizationMap member is not VK_NULL_HANDLE, commandBuffer is an unprotected command buffer, and protectedNoFault is not supported, then VkVideoEncodeQuantizationMapInfoKHR::quantizationMap **must** not have been created from a protected image
- VUID-vkCmdEncodeVideoKHR-pNext-10314
  If an instance of the VkVideoEncodeQuantizationMapInfoKHR structure is included in the pNext chain of pEncodeInfo and its quantizationMap member is not VK_NULL_HANDLE, then the image subresource range referenced by quantizationMap **must** be in the VK_IMAGE_LAYOUT_VIDEO_ENCODE_QUANTIZATION_MAP_KHR layout at the time the video encode operation is executed on the device
- VUID-vkCmdEncodeVideoKHR-pNext-10315
  If an instance of the VkVideoEncodeQuantizationMapInfoKHR structure is included in the pNext chain of pEncodeInfo and its quantizationMap member is not VK_NULL_HANDLE, and there is a bound video session parameters object, then the bound video session parameters object **must** have been created with VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR
- VUID-vkCmdEncodeVideoKHR-pNext-10316
  If an instance of the VkVideoEncodeQuantizationMapInfoKHR structure is included in the pNext chain of pEncodeInfo, its quantizationMap member is not VK_NULL_HANDLE, and there is a bound video session parameters object created with VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, then quantizationMapExtent **must** equal ⌈pEncodeInfo->srcPictureResource.codedExtent /

quantizationMapTexelSize

- VUID-vkCmdEncodeVideoKHR-pNext-08225
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the pNext chain of pEncodeInfo **must** include a VkVideoEncodeH264PictureInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-StdVideoH264SequenceParameterSet-08226
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH264SequenceParameterSet entry with seq_parameter_set_id matching StdVideoEncodeH264PictureInfo::seq_parameter_set_id that is provided in the pStdPictureInfo member of the VkVideoEncodeH264PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-StdVideoH264PictureParameterSet-08227
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH264PictureParameterSet entry with seq_parameter_set_id and pic_parameter_set_id matching StdVideoEncodeH264PictureInfo::seq_parameter_set_id and StdVideoEncodeH264PictureInfo::pic_parameter_set_id, respectively, that are provided in the pStdPictureInfo member of the VkVideoEncodeH264PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08228
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and pEncodeInfo->pSetupReferenceSlot is not NULL, then the pNext chain of pEncodeInfo->pSetupReferenceSlot **must** include a VkVideoEncodeH264DpbSlotInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-pNext-08229
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the pNext chain of each element of pEncodeInfo->pReferenceSlots **must** include a VkVideoEncodeH264DpbSlotInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-constantQp-08269
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and the current rate control mode is not VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, then VkVideoEncodeH264NaluSliceInfoKHR::constantQp **must** be zero for each element of the pNaluSliceEntries member of the VkVideoEncodeH264PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-constantQp-08270
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and the current rate control mode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, then VkVideoEncodeH264NaluSliceInfoKHR::constantQp **must** be between VkVideoEncodeH264CapabilitiesKHR::minQp and VkVideoEncodeH264CapabilitiesKHR::maxQp, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, for each element of the pNaluSliceEntries member

of the VkVideoEncodeH264PictureInfoKHR structure included in the pNext chain of pEncodeInfo

- VUID-vkCmdEncodeVideoKHR-constantQp-08271
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and VkVideoEncodeH264CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_H264_CAPABILITY_PER_SLICE_CONSTANT_QP_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then VkVideoEncodeH264NaluSliceInfoKHR::constantQp **must** have the same value for each element of the pNaluSliceEntries member of the VkVideoEncodeH264PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-naluSliceEntryCount-08302
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then the naluSliceEntryCount member of the VkVideoEncodeH264PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** be less than or equal to minCodingBlockExtent.width multiplied by minCodingBlockExtent.height
- VUID-vkCmdEncodeVideoKHR-naluSliceEntryCount-08312
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and VkVideoEncodeH264CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_H264_CAPABILITY_ROW_UNALIGNED_SLICE_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then the naluSliceEntryCount member of the VkVideoEncodeH264PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** be less than or equal to minCodingBlockExtent.height
- VUID-vkCmdEncodeVideoKHR-pNext-08352
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, the pNext chain of pEncodeInfo includes a VkVideoEncodeH264PictureInfoKHR structure, and pEncodeInfo->referenceSlotCount is greater than zero, then VkVideoEncodeH264PictureInfoKHR::pStdPictureInfo->pRefLists **must** not be NULL
- VUID-vkCmdEncodeVideoKHR-pNext-08339
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, the pNext chain of pEncodeInfo includes a VkVideoEncodeH264PictureInfoKHR structure, and VkVideoEncodeH264PictureInfoKHR::pStdPictureInfo->pRefLists is not NULL, then each element of the RefPicList0 and RefPicList1 array members of the StdVideoEncodeH264ReferenceListsInfo structure pointed to by VkVideoEncodeH264PictureInfoKHR::pStdPictureInfo->pRefLists **must** either be STD_VIDEO_H264_NO_REFERENCE_PICTURE or **must** equal the slotIndex member of one of the elements of pEncodeInfo->pReferenceSlots
- VUID-vkCmdEncodeVideoKHR-pNext-08353
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, the pNext chain of pEncodeInfo includes a VkVideoEncodeH264PictureInfoKHR structure, and pEncodeInfo->referenceSlotCount is greater than zero, then the slotIndex member of each element of pEncodeInfo->pReferenceSlots **must** equal one of the elements of the RefPicList0 or RefPicList1 array

members of the StdVideoEncodeH264ReferenceListsInfo structure pointed to by VkVideoEncodeH264PictureInfoKHR::pStdPictureInfo->pRefLists

- VUID-vkCmdEncodeVideoKHR-maxPPictureL0ReferenceCount-08340
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and VkVideoEncodeH264CapabilitiesKHR::maxPPictureL0ReferenceCount is zero, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then h264PictureType and each element of h264L0PictureTypes and h264L1PictureTypes **must** not be STD_VIDEO_H264_PICTURE_TYPE_P
- VUID-vkCmdEncodeVideoKHR-maxBPictureL0ReferenceCount-08341
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and VkVideoEncodeH264CapabilitiesKHR::maxBPictureL0ReferenceCount and VkVideoEncodeH264CapabilitiesKHR::maxL1ReferenceCount are both zero, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then h264PictureType and each element of h264L0PictureTypes and h264L1PictureTypes **must** not be STD_VIDEO_H264_PICTURE_TYPE_B
- VUID-vkCmdEncodeVideoKHR-flags-08342
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and VkVideoEncodeH264CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_H264_CAPABILITY_B_FRAME_IN_L0_LIST_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then each element of h264L0PictureTypes **must** not be STD_VIDEO_H264_PICTURE_TYPE_B
- VUID-vkCmdEncodeVideoKHR-flags-08343
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and VkVideoEncodeH264CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_H264_CAPABILITY_B_FRAME_IN_L1_LIST_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then each element of h264L1PictureTypes **must** not be STD_VIDEO_H264_PICTURE_TYPE_B
- VUID-vkCmdEncodeVideoKHR-pNext-08230
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the pNext chain of pEncodeInfo **must** include a VkVideoEncodeH265PictureInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-StdVideoH265VideoParameterSet-08231
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH265VideoParameterSet entry with vps_video_parameter_set_id matching StdVideoEncodeH265PictureInfo::sps_video_parameter_set_id that is provided in the pStdPictureInfo member of the VkVideoEncodeH265PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-StdVideoH265SequenceParameterSet-08232
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the bound video session parameters

object **must** contain a StdVideoH265SequenceParameterSet entry with sps_video_parameter_set_id and sps_seq_parameter_set_id matching StdVideoEncodeH265PictureInfo::sps_video_parameter_set_id and StdVideoEncodeH265PictureInfo::pps_seq_parameter_set_id, respectively, that are provided in the pStdPictureInfo member of the VkVideoEncodeH265PictureInfoKHR structure included in the pNext chain of pEncodeInfo

- VUID-vkCmdEncodeVideoKHR-StdVideoH265PictureParameterSet-08233
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the bound video session parameters object **must** contain a StdVideoH265PictureParameterSet entry with sps_video_parameter_set_id, pps_seq_parameter_set_id, and pps_pic_parameter_set_id matching StdVideoEncodeH265PictureInfo::sps_video_parameter_set_id, StdVideoEncodeH265PictureInfo::pps_seq_parameter_set_id, and StdVideoEncodeH265PictureInfo::pps_pic_parameter_set_id, respectively, that are provided in the pStdPictureInfo member of the VkVideoEncodeH265PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-08234
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and pEncodeInfo->pSetupReferenceSlot is not NULL, then the pNext chain of pEncodeInfo->pSetupReferenceSlot **must** include a VkVideoEncodeH265DpbSlotInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-pNext-08235
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the pNext chain of each element of pEncodeInfo->pReferenceSlots **must** include a VkVideoEncodeH265DpbSlotInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-constantQp-08272
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the current rate control mode is not VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, then VkVideoEncodeH265NaluSliceSegmentInfoKHR::constantQp **must** be zero for each element of the pNaluSliceSegmentEntries member of the VkVideoEncodeH265PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-constantQp-08273
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and the current rate control mode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, then VkVideoEncodeH265NaluSliceSegmentInfoKHR::constantQp **must** be between VkVideoEncodeH265CapabilitiesKHR::minQp and VkVideoEncodeH265CapabilitiesKHR::maxQp, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, for each element of the pNaluSliceSegmentEntries member of the VkVideoEncodeH265PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-constantQp-08274
  If the bound video session was created with the video codec operation

VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and VkVideoEncodeH265CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_H265_CAPABILITY_PER_SLICE_SEGMENT_CONSTANT_QP_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then VkVideoEncodeH265NaluSliceSegmentInfoKHR::constantQp **must** have the same value for each element of the pNaluSliceSegmentEntries member of the VkVideoEncodeH264PictureInfoKHR structure included in the pNext chain of pEncodeInfo

- VUID-vkCmdEncodeVideoKHR-naluSliceSegmentEntryCount-08307
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then the naluSliceSegmentEntryCount member of the VkVideoEncodeH265PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** be less than or equal to minCodingBlockExtent.width multiplied by minCodingBlockExtent.height
- VUID-vkCmdEncodeVideoKHR-naluSliceSegmentEntryCount-08313
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and VkVideoEncodeH265CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_H265_CAPABILITY_ROW_UNALIGNED_SLICE_SEGMENT_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then the naluSliceSegmentEntryCount member of the VkVideoEncodeH265PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** be less than or equal to minCodingBlockExtent.height
- VUID-vkCmdEncodeVideoKHR-pNext-08354
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, the pNext chain of pEncodeInfo includes a VkVideoEncodeH265PictureInfoKHR structure, and pEncodeInfo->referenceSlotCount is greater than zero, then VkVideoEncodeH265PictureInfoKHR::pStdPictureInfo->pRefLists **must** not be NULL
- VUID-vkCmdEncodeVideoKHR-pNext-08344
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, the pNext chain of pEncodeInfo includes a VkVideoEncodeH265PictureInfoKHR structure, and VkVideoEncodeH265PictureInfoKHR::pStdPictureInfo->pRefLists is not NULL, then each element of the RefPicList0 and RefPicList1 array members of the StdVideoEncodeH265ReferenceListsInfo structure pointed to by VkVideoEncodeH265PictureInfoKHR::pStdPictureInfo->pRefLists **must** either be STD_VIDEO_H265_NO_REFERENCE_PICTURE or **must** equal the slotIndex member of one of the elements of pEncodeInfo->pReferenceSlots
- VUID-vkCmdEncodeVideoKHR-pNext-08355
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, the pNext chain of pEncodeInfo includes a VkVideoEncodeH265PictureInfoKHR structure, and pEncodeInfo->referenceSlotCount is greater than zero, then the slotIndex member of each element of pEncodeInfo->pReferenceSlots **must** equal one of the elements of the RefPicList0 or RefPicList1 array members of the StdVideoEncodeH265ReferenceListsInfo structure pointed to by VkVideoEncodeH265PictureInfoKHR::pStdPictureInfo->pRefLists

- VUID-vkCmdEncodeVideoKHR-maxPPictureL0ReferenceCount-08345
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and VkVideoEncodeH265CapabilitiesKHR::maxPPictureL0ReferenceCount is zero, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then h265PictureType and each element of h265L0PictureTypes and h265L1PictureTypes **must** not be STD_VIDEO_H265_PICTURE_TYPE_P
- VUID-vkCmdEncodeVideoKHR-maxBPictureL0ReferenceCount-08346
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and VkVideoEncodeH265CapabilitiesKHR::maxBPictureL0ReferenceCount and VkVideoEncodeH265CapabilitiesKHR::maxL1ReferenceCount are both zero, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then h265PictureType and each element of h265L0PictureTypes and h265L1PictureTypes **must** not be STD_VIDEO_H265_PICTURE_TYPE_B
- VUID-vkCmdEncodeVideoKHR-flags-08347
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and VkVideoEncodeH265CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_H265_CAPABILITY_B_FRAME_IN_L0_LIST_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then each element of h265L0PictureTypes **must** not be STD_VIDEO_H264_PICTURE_TYPE_B
- VUID-vkCmdEncodeVideoKHR-flags-08348
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and VkVideoEncodeH265CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_H265_CAPABILITY_B_FRAME_IN_L1_LIST_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then each element of h265L1PictureTypes **must** not be STD_VIDEO_H265_PICTURE_TYPE_B
- VUID-vkCmdEncodeVideoKHR-pNext-10317
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the pNext chain of pEncodeInfo **must** include a VkVideoEncodeAV1PictureInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10318
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and pEncodeInfo->pSetupReferenceSlot is not NULL, then the pNext chain of pEncodeInfo->pSetupReferenceSlot **must** include a VkVideoEncodeAV1DpbSlotInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-pNext-10319
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the pNext chain of each element of pEncodeInfo->pReferenceSlots **must** include a VkVideoEncodeAV1DpbSlotInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-constantQIndex-10320
  If the bound video session was created with the video codec operation

VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the current rate control mode is not VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, then the constantQIndex member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** be zero

- VUID-vkCmdEncodeVideoKHR-constantQIndex-10321
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the current rate control mode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, then the constantQIndex member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** be between VkVideoEncodeAV1CapabilitiesKHR::minQIndex and VkVideoEncodeAV1CapabilitiesKHR::maxQIndex, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-flags-10322
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and VkVideoEncodeAV1CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_AV1_CAPABILITY_FRAME_SIZE_OVERRIDE_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo->flags.frame_size_override_flag **must** be zero for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-flags-10323
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and VkVideoEncodeAV1CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_AV1_CAPABILITY_FRAME_SIZE_OVERRIDE_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then pEncodeInfo->srcPictureResource.codedExtent.width **must** equal StdVideoAV1SequenceHeader::max_frame_width_minus_1 + 1 of the active AV1 sequence header
- VUID-vkCmdEncodeVideoKHR-flags-10324
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and VkVideoEncodeAV1CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_AV1_CAPABILITY_FRAME_SIZE_OVERRIDE_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then pEncodeInfo->srcPictureResource.codedExtent.height **must** equal StdVideoAV1SequenceHeader::max_frame_height_minus_1 + 1 of the active AV1 sequence header
- VUID-vkCmdEncodeVideoKHR-flags-10325
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and VkVideoEncodeAV1CapabilitiesKHR::flags does not include VK_VIDEO_ENCODE_AV1_CAPABILITY_MOTION_VECTOR_SCALING_BIT_KHR, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then for each element i of pEncodeInfo->pReferenceSlots pEncodeInfo->pReferenceSlots[i].pPictureResource->codedExtent **must** match pEncodeInfo->srcPictureResource.codedExtent

- VUID-vkCmdEncodeVideoKHR-predictionMode-10326
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the predictionMode member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_INTRA_ONLY_KHR, then VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo->frame_type **must** be STD_VIDEO_AV1_FRAME_TYPE_KEY or STD_VIDEO_AV1_FRAME_TYPE_INTRA_ONLY
- VUID-vkCmdEncodeVideoKHR-pStdPictureInfo-10327
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and pStdPictureInfo->frame_type for the pStdPictureInfo member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is STD_VIDEO_AV1_FRAME_TYPE_KEY or STD_VIDEO_AV1_FRAME_TYPE_INTRA_ONLY, then VkVideoEncodeAV1PictureInfoKHR::predictionMode **must** be VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_INTRA_ONLY_KHR
- VUID-vkCmdEncodeVideoKHR-maxSingleReferenceCount-10328
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and VkVideoEncodeAV1CapabilitiesKHR::maxSingleReferenceCount is zero, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then the predictionMode member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** not be VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_SINGLE_REFERENCE_KHR
- VUID-vkCmdEncodeVideoKHR-predictionMode-10329
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the predictionMode member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_SINGLE_REFERENCE_KHR, then there **must** be at least one non-negative element of VkVideoEncodeAV1PictureInfoKHR::referenceNameSlotIndices with element index i that does not equal cdfOnlyReferenceIndex and for which bit index i is set in VkVideoEncodeAV1CapabilitiesKHR::singleReferenceNameMask, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-maxUnidirectionalCompoundReferenceCount-10330
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and VkVideoEncodeAV1CapabilitiesKHR::maxUnidirectionalCompoundReferenceCount is zero, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then the predictionMode member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** not be VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_UNIDIRECTIONAL_COMPOUND_KHR
- VUID-vkCmdEncodeVideoKHR-predictionMode-10331
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the predictionMode member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_UNIDIRECTIONAL_COMPOUND_KHR, then there **must** be at least two non-negative elements of VkVideoEncodeAV1PictureInfoKHR

::referenceNameSlotIndices with element indices i and j where (i,j) ∈ {(0,1),(0,2),(0,3),(4,6)}, such that neither element equals cdfOnlyReferenceIndex and for which bit indices i and j are set in VkVideoEncodeAV1CapabilitiesKHR::unidirectionalCompoundReferenceNameMask, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with

- VUID-vkCmdEncodeVideoKHR-maxBidirectionalCompoundReferenceCount-10332
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and VkVideoEncodeAV1CapabilitiesKHR::maxBidirectionalCompoundReferenceCount is zero, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with, then the predictionMode member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** not be VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_BIDIRECTIONAL_COMPOUND_KHR
- VUID-vkCmdEncodeVideoKHR-predictionMode-10333
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the predictionMode member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_BIDIRECTIONAL_COMPOUND_KHR, then there **must** be at least two non-negative elements of VkVideoEncodeAV1PictureInfoKHR::referenceNameSlotIndices with element indices i ∈ {0,1,2,3} and j ∈ {4,5,6}, respectively, such that neither element equals cdfOnlyReferenceIndex, and for which bit indices i and j are set in VkVideoEncodeAV1CapabilitiesKHR::bidirectionalCompoundReferenceNameMask, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-referenceNameSlotIndices-10334
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then each element of the referenceNameSlotIndices array member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** either be negative or **must** equal the slotIndex member of one of the elements of pEncodeInfo->pReferenceSlots
- VUID-vkCmdEncodeVideoKHR-slotIndex-10335
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then the slotIndex member of each element of pEncodeInfo->pReferenceSlots **must** equal one of the elements of the referenceNameSlotIndices array member of the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-pExtensionHeader-10336
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pExtensionHeader member of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is not NULL, then pExtensionHeader->temporal_id **must** be less than VkVideoEncodeAV1CapabilitiesKHR::maxTemporalLayerCount, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with

- VUID-vkCmdEncodeVideoKHR-pExtensionHeader-10337
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pExtensionHeader member of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is not NULL, then pExtensionHeader->spatial_id **must** be less than VkVideoEncodeAV1CapabilitiesKHR::maxSpatialLayerCount, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10338
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and pEncodeInfo->pSetupReferenceSlot is not NULL, then the pExtensionHeader member of VkVideoEncodeAV1DpbSlotInfoKHR::pStdReferenceInfo for the VkVideoEncodeAV1DpbSlotInfoKHR structure included in the pNext chain of pEncodeInfo->pSetupReferenceSlot and the pExtensionHeader member of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo **must** both be NULL or not NULL
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10339
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, pEncodeInfo->pSetupReferenceSlot is not NULL, and the pExtensionHeader member of VkVideoEncodeAV1DpbSlotInfoKHR::pStdReferenceInfo for the VkVideoEncodeAV1DpbSlotInfoKHR structure included in the pNext chain of pEncodeInfo->pSetupReferenceSlot is not NULL, then pExtensionHeader->temporal_id **must** equal pStdPictureInfo->pExtensionHeader->temporal_id in the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-10340
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, pEncodeInfo->pSetupReferenceSlot is not NULL, and the pExtensionHeader member of VkVideoEncodeAV1DpbSlotInfoKHR::pStdReferenceInfo for the VkVideoEncodeAV1DpbSlotInfoKHR structure included in the pNext chain of pEncodeInfo->pSetupReferenceSlot is not NULL, then pExtensionHeader->spatial_id **must** equal pStdPictureInfo->pExtensionHeader->spatial_id in the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo
- VUID-vkCmdEncodeVideoKHR-pExtensionHeader-10341
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pExtensionHeader member of VkVideoEncodeAV1DpbSlotInfoKHR::pStdReferenceInfo for any of the VkVideoEncodeAV1DpbSlotInfoKHR structures included in the pNext chain of any element of pEncodeInfo->pReferenceSlots is not NULL, then pExtensionHeader->temporal_id **must** be less than VkVideoEncodeAV1CapabilitiesKHR::maxTemporalLayerCount, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pExtensionHeader-10342
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pExtensionHeader member of

VkVideoEncodeAV1DpbSlotInfoKHR::pStdReferenceInfo for any of the VkVideoEncodeAV1DpbSlotInfoKHR structures included in the pNext chain of any element of pEncodeInfo->pReferenceSlots is not NULL, then pExtensionHeader->spatial_id **must** be less than VkVideoEncodeAV1CapabilitiesKHR::maxSpatialLayerCount, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with

- VUID-vkCmdEncodeVideoKHR-pTileInfo-10343
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pTileInfo member of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is not NULL, then pTileInfo->TileCols **must** be greater than 0
- VUID-vkCmdEncodeVideoKHR-pTileInfo-10344
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pTileInfo member of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is not NULL, then pTileInfo->TileRows **must** be greater than 0
- VUID-vkCmdEncodeVideoKHR-pTileInfo-10345
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pTileInfo member of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is not NULL, then pTileInfo->TileCols **must** be less than or equal to VkVideoEncodeAV1CapabilitiesKHR::maxTiles.width, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pTileInfo-10346
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pTileInfo member of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is not NULL, then pTileInfo->TileRows **must** be less than or equal to VkVideoEncodeAV1CapabilitiesKHR::maxTiles.height, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pTileInfo-10347
  If the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and the pTileInfo member of VkVideoEncodeAV1PictureInfoKHR::pStdPictureInfo for the VkVideoEncodeAV1PictureInfoKHR structure included in the pNext chain of pEncodeInfo is not NULL, then ⌊pEncodeInfo->srcPictureResource.codedExtent.width / pTileInfo->TileCols⌋ **must** be between VkVideoEncodeAV1CapabilitiesKHR::minTileSize.width and VkVideoEncodeAV1CapabilitiesKHR::maxTileSize.width, inclusive, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with

- VUID-vkCmdEncodeVideoKHR-pTileInfo-10348
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR` and the `pTileInfo` member of VkVideoEncodeAV1PictureInfoKHR::`pStdPictureInfo` for the VkVideoEncodeAV1PictureInfoKHR structure included in the `pNext` chain of `pEncodeInfo` is not `NULL`, then □`pEncodeInfo->srcPictureResource.codedExtent.height` / `pTileInfo->TileRows`□ **must** be between VkVideoEncodeAV1CapabilitiesKHR::`minTileSize.height` and VkVideoEncodeAV1CapabilitiesKHR::`maxTileSize.height`, inclusive, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile the bound video session was created with
- VUID-vkCmdEncodeVideoKHR-pStdPictureInfo-10349
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR`, then VkVideoEncodeAV1PictureInfoKHR::`pStdPictureInfo->flags.segmentation_enabled` for the VkVideoEncodeAV1PictureInfoKHR structure included in the `pNext` chain of `pEncodeInfo` **must** be zero
- VUID-vkCmdEncodeVideoKHR-pStdPictureInfo-10350
  If the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR`, then VkVideoEncodeAV1PictureInfoKHR::`pStdPictureInfo->pSegmentation` for the VkVideoEncodeAV1PictureInfoKHR structure included in the `pNext` chain of `pEncodeInfo` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkCmdEncodeVideoKHR-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdEncodeVideoKHR-pEncodeInfo-parameter
  `pEncodeInfo` **must** be a valid pointer to a valid VkVideoEncodeInfoKHR structure
- VUID-vkCmdEncodeVideoKHR-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdEncodeVideoKHR-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support encode operations
- VUID-vkCmdEncodeVideoKHR-renderpass
  This command **must** only be called outside of a render pass instance
- VUID-vkCmdEncodeVideoKHR-videocoding
  This command **must** only be called inside of a video coding scope
- VUID-vkCmdEncodeVideoKHR-bufferlevel
  `commandBuffer` **must** be a primary `VkCommandBuffer`

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally

synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary	Outside	Inside	Encode	Action

The VkVideoEncodeInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeInfoKHR {
    VkStructureType                       sType;
    const void*                           pNext;
    VkVideoEncodeFlagsKHR                 flags;
    VkBuffer                              dstBuffer;
    VkDeviceSize                          dstBufferOffset;
    VkDeviceSize                          dstBufferRange;
    VkVideoPictureResourceInfoKHR         srcPictureResource;
    const VkVideoReferenceSlotInfoKHR*    pSetupReferenceSlot;
    uint32_t                              referenceSlotCount;
    const VkVideoReferenceSlotInfoKHR*    pReferenceSlots;
    uint32_t                              precedingExternallyEncodedBytes;
} VkVideoEncodeInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is a pointer to a structure extending this structure.
- flags is a bitmask of VkVideoEncodeFlagBitsKHR indicating video encode command flags.
- dstBuffer is the destination video bitstream buffer to write the encoded bitstream to.
- dstBufferOffset is the starting offset in bytes from the start of dstBuffer to write the encoded bitstream to.
- dstBufferRange is the maximum bitstream size in bytes that **can** be written to dstBuffer, starting from dstBufferOffset.
- srcPictureResource is the video picture resource to use as the encode input picture.
- pSetupReferenceSlot is NULL or a pointer to a VkVideoReferenceSlotInfoKHR structure specifying the reconstructed picture information.
- referenceSlotCount is the number of elements in the pReferenceSlots array.
- pReferenceSlots is NULL or a pointer to an array of VkVideoReferenceSlotInfoKHR structures describing the DPB slots and corresponding reference picture resources to use in this video encode operation (the set of active reference pictures).
- precedingExternallyEncodedBytes is the number of bytes externally encoded by the application to

the video bitstream and is used to update the internal state of the implementation's rate control algorithm to account for the bitrate budget consumed by these externally encoded bytes.

Valid Usage

- VUID-VkVideoEncodeInfoKHR-dstBuffer-08236
  `dstBuffer` **must** have been created with `VK_BUFFER_USAGE_VIDEO_ENCODE_DST_BIT_KHR` set
- VUID-VkVideoEncodeInfoKHR-dstBufferOffset-08237
  `dstBufferOffset` **must** be less than the size of `dstBuffer`
- VUID-VkVideoEncodeInfoKHR-dstBufferRange-08238
  `dstBufferRange` **must** be less than or equal to the size of `dstBuffer` minus `dstBufferOffset`
- VUID-VkVideoEncodeInfoKHR-pSetupReferenceSlot-08239
  If `pSetupReferenceSlot` is not `NULL`, then its `slotIndex` member **must** not be negative
- VUID-VkVideoEncodeInfoKHR-pSetupReferenceSlot-08240
  If `pSetupReferenceSlot` is not `NULL`, then its `pPictureResource` **must** not be `NULL`
- VUID-VkVideoEncodeInfoKHR-slotIndex-08241
  The `slotIndex` member of each element of `pReferenceSlots` **must** not be negative
- VUID-VkVideoEncodeInfoKHR-pPictureResource-08242
  The `pPictureResource` member of each element of `pReferenceSlots` **must** not be `NULL`

Valid Usage (Implicit)

- VUID-VkVideoEncodeInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_INFO_KHR`
- VUID-VkVideoEncodeInfoKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkVideoEncodeAV1PictureInfoKHR, VkVideoEncodeH264PictureInfoKHR, VkVideoEncodeH265PictureInfoKHR, VkVideoEncodeQuantizationMapInfoKHR, or VkVideoInlineQueryInfoKHR
- VUID-VkVideoEncodeInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkVideoEncodeInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkVideoEncodeFlagBitsKHR values
- VUID-VkVideoEncodeInfoKHR-dstBuffer-parameter
  `dstBuffer` **must** be a valid VkBuffer handle
- VUID-VkVideoEncodeInfoKHR-srcPictureResource-parameter
  `srcPictureResource` **must** be a valid VkVideoPictureResourceInfoKHR structure
- VUID-VkVideoEncodeInfoKHR-pSetupReferenceSlot-parameter
  If `pSetupReferenceSlot` is not `NULL`, `pSetupReferenceSlot` **must** be a valid pointer to a valid VkVideoReferenceSlotInfoKHR structure
- VUID-VkVideoEncodeInfoKHR-pReferenceSlots-parameter

If `referenceSlotCount` is not `0`, `pReferenceSlots` **must** be a valid pointer to an array of `referenceSlotCount` valid VkVideoReferenceSlotInfoKHR structures

Bits which **can** be set in VkVideoEncodeInfoKHR::`flags`, specifying video encode flags, are:

```
// Provided by VK_KHR_video_encode_quantization_map
typedef enum VkVideoEncodeFlagBitsKHR {
  // Provided by VK_KHR_video_encode_quantization_map
    VK_VIDEO_ENCODE_WITH_QUANTIZATION_DELTA_MAP_BIT_KHR = 0x00000001,
  // Provided by VK_KHR_video_encode_quantization_map
    VK_VIDEO_ENCODE_WITH_EMPHASIS_MAP_BIT_KHR = 0x00000002,
} VkVideoEncodeFlagBitsKHR;
```

- `VK_VIDEO_ENCODE_WITH_QUANTIZATION_DELTA_MAP_BIT_KHR` specifies the use of a quantization delta map in the issued video encode operations.
- `VK_VIDEO_ENCODE_WITH_EMPHASIS_MAP_BIT_KHR` specifies the use of an emphasis map in the issued video encode operations.

```
// Provided by VK_KHR_video_encode_queue
typedef VkFlags VkVideoEncodeFlagsKHR;
```

VkVideoEncodeFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeFlagBitsKHR.

38.16. Video Encode Rate Control

The size of the encoded bitstream data produced by video encode operations is a function of the following set of constraints:

- The capabilities of the compression algorithms defined and employed by the used video compression standard;
- Restrictions imposed by the selected video profile according to the rules defined by the used video compression standard;
- Further restrictions imposed by the capabilities supported by the implementation for the selected video profile;
- The image data in the encode input picture and the set of active reference pictures (as these affect the effectiveness of the compression algorithms employed by the video encode operations);
- The set of codec-specific and codec-independent encoding parameters provided by the application.

These also inherently define the set of decoder capabilities required for reconstructing and processing the picture data in the encoded bitstream.

Video coding uses *bitrate* as the quantitative metric associated with encoded bitstream data size which expresses the rate at which video bitstream data **can** be transferred or processed, measured in number of bits per second. This bitrate is both a function of the encoded bitstream data size of the encoded pictures as well as the *frame rate* used by the video sequence.

Rate control algorithms are used by video encode operations to enable adjusting encoding parameters to achieve a target bitrate, or otherwise directly or indirectly control the bitrate of the generated video bitstream data. These algorithms are usually not defined by the used video compression standard, although some video compression standards do provide non-normative guidelines for implementations.

Accordingly, this specification does not mandate implementations to produce identical encoded bitstream data outputs in response to video encode operations, however, it does define a set of codec-independent and codec-specific parameters that enable the application to control the behavior of the rate control algorithms supported by the implementation. Some of these parameters guarantee certain implementation behavior while others provide guidance for implementations to apply various rate control heuristics.

NOTE

Applications need to make sure that they configure rate control parameters appropriately and that they follow the promises made to the implementation through parameters providing guidance for the implementation's rate control algorithms and heuristics in order to be able to get the desired rate control behavior and to be able to hit the set bitrate targets. In addition, the behavior of rate control may also differ across implementations even if the capabilities of the used video profile match between those implementations. This may happen due to implementations applying different rate control algorithms or heuristics internally, and thus even the same set of guidance parameter values may have different effects on the rate control behavior across implementations.

38.16.1. Rate Control Modes

After a video session is reset to the initial state, the default behavior and parameters of video encode rate control are entirely implementation-dependent and the application **cannot** affect the bitrate or quality parameters of the encoded bitstream data produced by video encode operations unless the application changes the rate control configuration of the video session, as described in the Video Coding Control section.

For each supported video profile, the implementation **may** expose a set of *rate control modes* that are available for use by the application when encoding bitstreams targeting that video profile. These modes allow using different rate control algorithms that fall into one of the following two categories:

1. Per-operation rate control
2. Stream-level rate control

In case of *per-operation rate control*, the bitrate of the generated video bitstream data is indirectly controlled by quality, size, or other encoding parameters specified by the application for each individual video encode operation.

In case of *stream-level rate control*, the application **can** directly specify target bitrates besides other encoding parameters to control the behavior of the rate control algorithm used by the implementation across multiple video encode operations.

The rate control modes are defined with the following enums:

```
// Provided by VK_KHR_video_encode_queue
typedef enum VkVideoEncodeRateControlModeFlagBitsKHR {
    VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR = 0,
    VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_RATE_CONTROL_MODE_CBR_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_RATE_CONTROL_MODE_VBR_BIT_KHR = 0x00000004,
} VkVideoEncodeRateControlModeFlagBitsKHR;
```

- `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR` specifies the use of implementation-specific rate control.
- `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR` specifies that rate control is disabled and the application will specify per-operation rate control parameters controlling the encoding quality. In this mode implementations will encode pictures independently of the output bitrate of prior video encode operations.
  - When using an H.264 encode profile, implementations will use the QP value specified in VkVideoEncodeH264NaluSliceInfoKHR::`constantQp` to control the quality of the encoded picture.
  - When using an H.265 encode profile, implementations will use the QP value specified in VkVideoEncodeH265NaluSliceSegmentInfoKHR::`constantQp` to control the quality of the encoded picture.
  - When using an AV1 encode profile, implementations will use the quantizer index value specified in VkVideoEncodeAV1PictureInfoKHR::`constantQIndex` to control the quality of the encoded picture.
- `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_CBR_BIT_KHR` specifies the use of constant bitrate (CBR) rate control mode. In this mode the implementation will attempt to produce the encoded bitstream at a constant bitrate while conforming to the constraints of other rate control parameters.
- `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_VBR_BIT_KHR` specifies the use of variable bitrate (VBR) rate control mode. In this mode the implementation will produce the encoded bitstream at a variable bitrate according to the constraints of other rate control parameters.

```
// Provided by VK_KHR_video_encode_queue
typedef VkFlags VkVideoEncodeRateControlModeFlagsKHR;
```

`VkVideoEncodeRateControlModeFlagsKHR` is a bitmask type for setting a mask of zero or more VkVideoEncodeRateControlModeFlagBitsKHR.

38.16.2. Leaky Bucket Model

Video encoding implementations use the *leaky bucket model* for stream-level rate control. The leaky bucket is a concept referring to the interface between the video encoder and the consumer (for example, a network connection), where the video encoder produces encoded bitstream data corresponding to the encoded pictures and adds them in the leaky bucket while its content are drained by the consumer.

Analogously, a similar leaky bucket is considered to exist at the input interface of a video decoder, into which encoded bitstream data is continuously added and is subsequently consumed by the video decoder. It is desirable to avoid overflowing or underflowing this leaky bucked because:

- In case of an underflow, the video decoder will be unable to consume encoded bitstream data in order to decode pictures (and optionally display them).
- In case of an overflow, the leaky bucket will be unable to accommodate more encoded bitstream data and such data **may** need to be thrown away, leading to the loss of the corresponding encoded pictures.

These requirements **can** be satisfied by imposing various constraints on the encoder-side leaky bucket to avoid its overflow or underflow, depending on the used rate control algorithm and codec parameters. However, enumerating these constraints is outside the scope of this specification.

The term *virtual buffer* is often used as an alternative to refer to the leaky bucket.

This virtual buffer model is defined by the following parameters:

- The bitrate (`R`) at which the encoded bitstream is expected to be processed.
- The size (`B`) of the virtual buffer.
- The initial occupancy (`F`) of the virtual buffer.

In this model the virtual buffer is used to smooth out fluctuations in the bitrate of the encoded bitstream over time without experiencing buffer overflow or underflow, as long as the bitrate of the encoded stream does not diverge from the target bitrate for extended periods of time.

This buffering **may** inherently impose a processing delay, as the goal of the model is to enable decoders maintain a consistent processing rate of an encoded bitstream with varying data rate.

The initial or start-up delay (`D`) is computed as:

```
D = F / R
```

NOTE	Applications need to configure the virtual buffer with sufficient size to avoid or minimize buffer overflows and underflows while also keeping it small enough to meet their latency goals.

38.16.3. Rate Control Layers

Some video compression standards and video profiles allow associating encoded pictures with specific *video coding layers*. The name, identification, and semantics associated with such video coding layers are defined by the corresponding video compression standards.

Analogously, stream-level rate control **can** be configured to use one or more *rate control layers*:

- When a single rate control layer is configured, it is applied to all encoded pictures, regardless of the picture's video coding layer. In this case the distribution of the available bitrate budget across video coding layers is implementation-dependent.
- When multiple rate control layers are configured, each rate control layer is applied to the corresponding video coding layer, i.e. only across encoded pictures pertaining to the corresponding video coding layer.

Individual rate control layers are identified using *layer indices* between zero and `N-1`, where `N` is the number of active rate control layers.

Rate control layers are only applicable when using stream-level rate control modes.

38.16.4. Rate Control State

Rate control state is maintained by the implementation in the video session objects and its parameters are specified using an instance of the `VkVideoEncodeRateControlInfoKHR` structure. The complete rate control state of a video session is defined by the following set of parameters:

- The values of the members of the VkVideoEncodeRateControlInfoKHR structure used to configure the rate control state.
- The values of the members of any VkVideoEncodeRateControlLayerInfoKHR structures specified in VkVideoEncodeRateControlInfoKHR::`pLayers` used to configure the state of individual rate control layers.
- If the video session was created with an H.264 encode profile:
  - The values of the members of the VkVideoEncodeH264RateControlInfoKHR structure, if one is specified in the `pNext` chain of the VkVideoEncodeRateControlInfoKHR used to configure the rate control state.
  - The values of the members of any VkVideoEncodeH264RateControlLayerInfoKHR structures included in the `pNext` chain of a VkVideoEncodeRateControlLayerInfoKHR structure used to configure the state of a rate control layer.
- If the video session was created with an H.265 encode profile:
  - The values of the members of the VkVideoEncodeH265RateControlInfoKHR structure, if one is specified in the `pNext` chain of the VkVideoEncodeRateControlInfoKHR used to configure the rate control state.
  - The values of the members of any VkVideoEncodeH265RateControlLayerInfoKHR structures included in the `pNext` chain of a VkVideoEncodeRateControlLayerInfoKHR structure used to configure the state of a rate control layer.
- If the video session was created with an AV1 encode profile:

- The values of the members of the VkVideoEncodeAV1RateControlInfoKHR structure, if one is specified in the pNext chain of the VkVideoEncodeRateControlInfoKHR used to configure the rate control state.
- The values of the members of any VkVideoEncodeAV1RateControlLayerInfoKHR structures included in the pNext chain of a VkVideoEncodeRateControlLayerInfoKHR structure used to configure the state of a rate control layer.

Two rate control states match if all the parameters listed above match between them.

The VkVideoEncodeRateControlInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeRateControlInfoKHR {
    VkStructureType                                 sType;
    const void*                                     pNext;
    VkVideoEncodeRateControlFlagsKHR                flags;
    VkVideoEncodeRateControlModeFlagBitsKHR         rateControlMode;
    uint32_t                                        layerCount;
    const VkVideoEncodeRateControlLayerInfoKHR*     pLayers;
    uint32_t                                        virtualBufferSizeInMs;
    uint32_t                                        initialVirtualBufferSizeInMs;
} VkVideoEncodeRateControlInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is reserved for future use.
- rateControlMode is a VkVideoEncodeRateControlModeFlagBitsKHR value specifying the rate control mode.
- layerCount specifies the number of rate control layers to use.
- pLayers is a pointer to an array of layerCount VkVideoEncodeRateControlLayerInfoKHR structures, each specifying the rate control configuration of the corresponding rate control layer.
- virtualBufferSizeInMs is the size in milliseconds of the virtual buffer used by the implementation's rate control algorithm for the leaky bucket model, with respect to the average bitrate of the stream calculated by summing the values of the averageBitrate members of the elements of the pLayers array.
- initialVirtualBufferSizeInMs is the initial occupancy in milliseconds of the virtual buffer used by the implementation's rate control algorithm for the leaky bucket model.

If layerCount is zero then the values of virtualBufferSizeInMs and initialVirtualBufferSizeInMs are ignored.

This structure **can** be specified in the following places:

- In the pNext chain of VkVideoBeginCodingInfoKHR to specify the current rate control state expected to be configured when beginning a video coding scope.

- In the pNext chain of VkVideoCodingControlInfoKHR to change the rate control configuration of the bound video session.

Including this structure in the pNext chain of VkVideoCodingControlInfoKHR and including VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR in VkVideoCodingControlInfoKHR::flags enables updating the rate control configuration of the bound video session. This replaces the entire rate control configuration of the bound video session and **may** reset the state of all enabled rate control layers to an initial state according to the codec-specific rate control semantics defined in the corresponding sections listed below.

When layerCount is greater than one, multiple rate control layers are configured, and each rate control layer is applied to the corresponding video coding layer identified by the index of the corresponding element of pLayer.

- If the video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then this index specifies the H.264 temporal layer ID of the video coding layer the rate control layer is applied to.
- If the video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, then this index specifies the H.265 temporal ID of the video coding layer the rate control layer is applied to.
- If the video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, then this index specifies the AV1 temporal ID of the temporal layer the rate control layer is applied to.

Additional structures providing codec-specific rate control parameters **can** be included in the pNext chain of VkVideoCodingControlInfoKHR depending on the video profile the bound video session was created. For further details see:

- Video Coding Control
- H.264 Encode Rate Control
- H.265 Encode Rate Control
- AV1 Encode Rate Control

The new rate control configuration takes effect when the corresponding vkCmdControlVideoCodingKHR is executed on the device, and only impacts video encode operations that follow in execution order.

Valid Usage

- VUID-VkVideoEncodeRateControlInfoKHR-rateControlMode-08248
  If rateControlMode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR or VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, then layerCount **must** be 0
- VUID-VkVideoEncodeRateControlInfoKHR-rateControlMode-08275
  If rateControlMode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_CBR_BIT_KHR or VK_VIDEO_ENCODE_RATE_CONTROL_MODE_VBR_BIT_KHR, then layerCount **must** be greater than 0
- VUID-VkVideoEncodeRateControlInfoKHR-rateControlMode-08244

If rateControlMode is not VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR, then it **must** specify one of the bits included in VkVideoEncodeCapabilitiesKHR::rateControlModes, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile

- VUID-VkVideoEncodeRateControlInfoKHR-layerCount-08245
  layerCount member **must** be less than or equal to VkVideoEncodeCapabilitiesKHR ::maxRateControlLayers, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeRateControlInfoKHR-pLayers-08276
  For each element of pLayers, its averageBitrate member **must** be between 1 and VkVideoEncodeCapabilitiesKHR::maxBitrate, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeRateControlInfoKHR-pLayers-08277
  For each element of pLayers, its maxBitrate member **must** be between 1 and VkVideoEncodeCapabilitiesKHR::maxBitrate, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeRateControlInfoKHR-rateControlMode-08356
  If rateControlMode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_CBR_BIT_KHR, then for each element of pLayers, its averageBitrate member **must** equal its maxBitrate member
- VUID-VkVideoEncodeRateControlInfoKHR-rateControlMode-08278
  If rateControlMode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_VBR_BIT_KHR, then for each element of pLayers, its averageBitrate member **must** be less than or equal to its maxBitrate member
- VUID-VkVideoEncodeRateControlInfoKHR-layerCount-08357
  If layerCount is not zero, then virtualBufferSizeInMs **must** be greater than zero
- VUID-VkVideoEncodeRateControlInfoKHR-layerCount-08358
  If layerCount is not zero, then initialVirtualBufferSizeInMs **must** be less than or equal to virtualBufferSizeInMs
- VUID-VkVideoEncodeRateControlInfoKHR-videoCodecOperation-07022
  If the videoCodecOperation of the used video profile is VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, the pNext chain this structure is included in also includes an instance of the VkVideoEncodeH264RateControlInfoKHR structure, and layerCount is greater than 1, then layerCount **must** equal VkVideoEncodeH264RateControlInfoKHR::temporalLayerCount
- VUID-VkVideoEncodeRateControlInfoKHR-videoCodecOperation-07025
  If the videoCodecOperation of the used video profile is VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, the pNext chain this structure is included in also includes an instance of the VkVideoEncodeH265RateControlInfoKHR structure, and layerCount is greater than 1, then layerCount **must** equal VkVideoEncodeH265RateControlInfoKHR::subLayerCount
- VUID-VkVideoEncodeRateControlInfoKHR-videoCodecOperation-10351
  If the videoCodecOperation of the used video profile is VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, the pNext chain this structure is included in also includes an instance of the VkVideoEncodeAV1RateControlInfoKHR structure, and layerCount is greater than 1, then layerCount **must** equal

VkVideoEncodeAV1RateControlInfoKHR::temporalLayerCount

Valid Usage (Implicit)

- VUID-VkVideoEncodeRateControlInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_RATE_CONTROL_INFO_KHR
- VUID-VkVideoEncodeRateControlInfoKHR-flags-zerobitmask
  flags **must** be 0
- VUID-VkVideoEncodeRateControlInfoKHR-rateControlMode-parameter
  If rateControlMode is not 0, rateControlMode **must** be a valid VkVideoEncodeRateControlModeFlagBitsKHR value
- VUID-VkVideoEncodeRateControlInfoKHR-pLayers-parameter
  If layerCount is not 0, pLayers **must** be a valid pointer to an array of layerCount valid VkVideoEncodeRateControlLayerInfoKHR structures

```
// Provided by VK_KHR_video_encode_queue
typedef VkFlags VkVideoEncodeRateControlFlagsKHR;
```

VkVideoEncodeRateControlFlagsKHR is a bitmask type for setting a mask, but currently reserved for future use.

Rate Control Layer State

The configuration of individual rate control layers is specified using an instance of the VkVideoEncodeRateControlLayerInfoKHR structure.

The VkVideoEncodeRateControlLayerInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_queue
typedef struct VkVideoEncodeRateControlLayerInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    uint64_t           averageBitrate;
    uint64_t           maxBitrate;
    uint32_t           frameRateNumerator;
    uint32_t           frameRateDenominator;
} VkVideoEncodeRateControlLayerInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is a pointer to a structure extending this structure.
- averageBitrate is the average bitrate to be targeted by the implementation's rate control algorithm.
- maxBitrate is the peak bitrate to be targeted by the implementation's rate control algorithm.

- `frameRateNumerator` is the numerator of the frame rate assumed by the implementation's rate control algorithm.
- `frameRateDenominator` is the denominator of the frame rate assumed by the implementation's rate control algorithm.

NOTE	The ability of the implementation's rate control algorithm to be able to match the requested average and/or peak bitrates **may** be limited by the set of other codec-independent and codec-specific rate control parameters specified by the application, the input content, as well as the application conforming to the rate control guidance provided to the implementation, as described earlier.

Additional structures providing codec-specific rate control parameters **can** be included in the `pNext` chain of `VkVideoEncodeRateControlLayerInfoKHR` depending on the video profile the bound video session was created with. For further details see:

- Video Coding Control
- H.264 Encode Rate Control
- H.265 Encode Rate Control
- AV1 Encode Rate Control

Valid Usage

- VUID-VkVideoEncodeRateControlLayerInfoKHR-frameRateNumerator-08350
  `frameRateNumerator` **must** be greater than zero
- VUID-VkVideoEncodeRateControlLayerInfoKHR-frameRateDenominator-08351
  `frameRateDenominator` **must** be greater than zero

Valid Usage (Implicit)

- VUID-VkVideoEncodeRateControlLayerInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_RATE_CONTROL_LAYER_INFO_KHR`
- VUID-VkVideoEncodeRateControlLayerInfoKHR-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkVideoEncodeAV1RateControlLayerInfoKHR, VkVideoEncodeH264RateControlLayerInfoKHR, or VkVideoEncodeH265RateControlLayerInfoKHR
- VUID-VkVideoEncodeRateControlLayerInfoKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

38.17. Video Encode Quantization Maps

Quantization maps are VkImage objects that are used in video encode operations to control the relative quantization parameter values across the encoded picture. Each texel in the quantization

map controls the relative quantization parameter values used to encode the corresponding rectangular block of texels in the encode input picture.

The size of the rectangular block of texels each quantization map texel covers is referred to as the *quantization map texel size*.

The extent of the image subresource used as a quantization map when encoding a picture with a coded extent of (`width`,`height`) thus has to be at least (⌈`width` / `texelSize.width`⌉, ⌈`height` / `texelSize.height`⌉), where `texelSize` is the used quantization map texel size.

In particular, the quantization map texel at location (`x`,`y`) contains relative quantization parameter values used when encoding the `texelSize` sized rectangular block of the encode input picture starting at the texel location (`x` × `texelSize.width`, `y` × `texelSize.height`).

The quantization map texel size does not always match the size of the codec-specific coding blocks used during encoding. Furthermore, some video compression standards allow the size of the codec-specific coding blocks to vary across the encoded picture. In order to accommodate for such mismatches between the granularity at which quantization parameters are stored in quantization maps and the granularity at which they are applied to codec-specific coding blocks during encoding, the following mapping rules are applied to define the quantization map texel value corresponding to a given codec-specific coding block with a size (`width`,`height`) at the texel location (`x`,`y`) in the encode input picture:

- If the size of the codec-specific coding block matches the used quantization map texel size, then the fetched quantization map value corresponding to the codec-specific coding block is the texel value at the texel location (`x` / `texelSize.width`, `y` / `texelSize.height`).
- If the size of the codec-specific coding block is smaller than the used quantization map texel size, then the fetched quantization map value corresponding to the codec-specific coding block is the texel value at the texel location (⌊`x` / `texelSize.width`⌋, ⌊`y` / `texelSize.height`⌋).
- If the size of the codec-specific coding block is larger than the used quantization map texel size, then the fetched quantization map value corresponding to the codec-specific coding block **may** be any value determined as the linear interpolation of the quantization map texel values in the subregion starting at texel location (`x` / `texelSize.width`,`y` / `texelSize.height`) with a size (⌈`width` / `texelSize.width`⌉, ⌈`height` / `texelSize.height`⌉).

The actual control parameters stored in the quantization map depend on its type. This specification supports the following types of quantization maps:

- Quantization delta maps
- Emphasis maps

38.17.1. Quantization Delta Maps

Quantization delta maps contain values that directly affect the codec-specific quantization parameter values used to encode the corresponding block of the encode input picture.

Quantization delta maps **can** be used in conjunction with any rate control mode, including `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`.

Due to their codec-specific nature, they are described in more detail in the corresponding codec-specific section for video encode operations that support them. In particular:

- The behavior of quantization delta maps used with an H.264 encode profile is described in the H.264 Encode Quantization and H.264 QP Delta Maps sections.
- The behavior of quantization delta maps used with an H.265 encode profile is described in the H.265 Encode Quantization and H.265 QP Delta Maps sections.
- The behavior of quantization delta maps used with an AV1 encode profile is described in the AV1 Encode Quantization and AV1 Quantizer Index Delta Maps sections.

This specification does not support quantization delta maps for any other video encode operation.

38.17.2. Emphasis Maps

Emphasis maps contain values that indirectly affect the codec-specific quantization parameter values used to encode the corresponding block of the encode input picture.

The texels of emphasis maps contain values that provide input to the encoder implementation about the relative importance (emphasis) of regions of the encoded pictures in order to enable the implementation's rate control algorithm to allocate more bitrate budget for regions of the encoded picture with higher emphasis values than to those with lower emphasis values.

Emphasis maps **can** only be used when the current rate control mode configured for the video session is not `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR` or `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`.

As these emphasis values only control the otherwise implementation-specific behavior of the used rate control algorithm, this specification does not impose additional restrictions on implementations beyond the ones outlined in the corresponding codec-specific sections describing quantization behavior:

- The behavior of emphasis maps used with an H.264 encode profile is described in the H.264 Encode Quantization section.
- The behavior of emphasis maps used with an H.265 encode profile is described in the H.265 Encode Quantization section.
- The behavior of emphasis maps used with an AV1 encode profile is described in the AV1 Encode Quantization section.

This specification does not support emphasis maps for any other video encode operation.

Emphasis maps always have single channel unsigned normalized integer formats and implementations are **required** to support the `VK_FORMAT_R8_UNORM` format for emphasis maps, as reported in VkVideoFormatPropertiesKHR::`format`, when the video encode profile supports `VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR`.

38.17.3. Quantization Map Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR with `pVideoProfile->videoCodecOperation`

specifying an encode operation, the VkVideoEncodeQuantizationMapCapabilitiesKHR structure **can** be included in the pNext chain of the VkVideoCapabilitiesKHR structure to retrieve capabilities specific to video encode quantization maps.

The VkVideoEncodeQuantizationMapCapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_quantization_map
typedef struct VkVideoEncodeQuantizationMapCapabilitiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkExtent2D         maxQuantizationMapExtent;
} VkVideoEncodeQuantizationMapCapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxQuantizationMapExtent indicates the maximum supported width and height of quantization maps.

Valid Usage (Implicit)

- VUID-VkVideoEncodeQuantizationMapCapabilitiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_CAPABILITIES_KHR

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities of an H.264 encode profile, the VkVideoEncodeH264QuantizationMapCapabilitiesKHR structure **can** be included in the pNext chain of the VkVideoCapabilitiesKHR structure to retrieve additional video encode quantization map capabilities specific to H.264 encode profiles.

The VkVideoEncodeH264QuantizationMapCapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264 with VK_KHR_video_encode_quantization_map
typedef struct VkVideoEncodeH264QuantizationMapCapabilitiesKHR {
    VkStructureType    sType;
    void*              pNext;
    int32_t            minQpDelta;
    int32_t            maxQpDelta;
} VkVideoEncodeH264QuantizationMapCapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- minQpDelta indicates the minimum QP delta value supported for H.264 QP delta maps.
- maxQpDelta indicates the maximum QP delta value supported for H.264 QP delta maps.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264QuantizationMapCapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_QUANTIZATION_MAP_CAPABILITIES_KHR`

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities of an H.265 encode profile, the `VkVideoEncodeH265QuantizationMapCapabilitiesKHR` structure **can** be included in the `pNext` chain of the VkVideoCapabilitiesKHR structure to retrieve additional video encode quantization map capabilities specific to H.265 encode profiles.

The `VkVideoEncodeH265QuantizationMapCapabilitiesKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h265 with VK_KHR_video_encode_quantization_map
typedef struct VkVideoEncodeH265QuantizationMapCapabilitiesKHR {
    VkStructureType    sType;
    void*              pNext;
    int32_t            minQpDelta;
    int32_t            maxQpDelta;
} VkVideoEncodeH265QuantizationMapCapabilitiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `minQpDelta` indicates the minimum QP delta value supported for H.265 QP delta maps.
- `maxQpDelta` indicates the maximum QP delta value supported for H.265 QP delta maps.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265QuantizationMapCapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_QUANTIZATION_MAP_CAPABILITIES_KHR`

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities of an AV1 encode profile, the `VkVideoEncodeAV1QuantizationMapCapabilitiesKHR` structure **can** be included in the `pNext` chain of the VkVideoCapabilitiesKHR structure to retrieve additional video encode quantization map capabilities specific to AV1 encode profiles.

The `VkVideoEncodeAV1QuantizationMapCapabilitiesKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_av1 with VK_KHR_video_encode_quantization_map
typedef struct VkVideoEncodeAV1QuantizationMapCapabilitiesKHR {
    VkStructureType    sType;
    void*              pNext;
    int32_t            minQIndexDelta;
    int32_t            maxQIndexDelta;
} VkVideoEncodeAV1QuantizationMapCapabilitiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `minQIndexDelta` indicates the minimum quantizer index delta value supported for AV1 quantizer index delta maps.
- `maxQIndexDelta` indicates the maximum quantizer index delta value supported for AV1 quantizer index delta maps.

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1QuantizationMapCapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_QUANTIZATION_MAP_CAPABILITIES_KHR`

38.17.4. Quantization Map Format Properties

When calling vkGetPhysicalDeviceVideoFormatPropertiesKHR, the VkVideoFormatQuantizationMapPropertiesKHR structure **can** be included in the `pNext` chain of the VkVideoFormatPropertiesKHR structure to retrieve video format properties specific to video encode quantization maps.

The `VkVideoFormatQuantizationMapPropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_quantization_map
typedef struct VkVideoFormatQuantizationMapPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkExtent2D         quantizationMapTexelSize;
} VkVideoFormatQuantizationMapPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `quantizationMapTexelSize` indicates the quantization map texel size of the video format, i.e. the number of pixels corresponding to each quantization map texel.

The values returned in this structure are only defined if the allowed image usage flags returned in VkVideoFormatPropertiesKHR::`imageUsageFlags` for this video format include `VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR` or `VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR`.

Implementations **may** support multiple quantization map texel sizes for a particular video format which is indicated by vkGetPhysicalDeviceVideoFormatPropertiesKHR returning multiple entries with different `quantizationMapTexelSize` values.

Valid Usage (Implicit)

- VUID-VkVideoFormatQuantizationMapPropertiesKHR-sType-sType

sType **must** be VK_STRUCTURE_TYPE_VIDEO_FORMAT_QUANTIZATION_MAP_PROPERTIES_KHR

When calling vkGetPhysicalDeviceVideoFormatPropertiesKHR, the VkVideoFormatH265QuantizationMapPropertiesKHR structure **can** be included in the pNext chain of the VkVideoFormatPropertiesKHR structure to retrieve video format properties specific to video encode quantization maps used with an H.265 encode profile.

The VkVideoFormatH265QuantizationMapPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265 with VK_KHR_video_encode_quantization_map
typedef struct VkVideoFormatH265QuantizationMapPropertiesKHR {
    VkStructureType                    sType;
    void*                              pNext;
    VkVideoEncodeH265CtbSizeFlagsKHR   compatibleCtbSizes;
} VkVideoFormatH265QuantizationMapPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- compatibleCtbSizes is a bitmask of VkVideoEncodeH265CtbSizeFlagBitsKHR indicating the CTB sizes that quantization maps using this video format are compatible with.

NOTE

The value of compatibleCtbSizes does not limit the use of the specific quantization map format, but does limit the implementation in being able to encode pictures with CTB sizes not included in compatibleCtbSizes but otherwise supported by the used video profile, as indicated by VkVideoEncodeH265CapabilitiesKHR::ctbSizes. In particular, using smaller quantization map texel sizes may prevent implementations from encoding with larger CTB sizes which may have a negative impact on the efficiency of the encoder.

The values returned in this structure are only defined if the allowed image usage flags returned in VkVideoFormatPropertiesKHR::imageUsageFlags for this video format include VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR.

Valid Usage (Implicit)

- VUID-VkVideoFormatH265QuantizationMapPropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_FORMAT_H265_QUANTIZATION_MAP_PROPERTIES_KHR

When calling vkGetPhysicalDeviceVideoFormatPropertiesKHR, the VkVideoFormatAV1QuantizationMapPropertiesKHR structure **can** be included in the pNext chain of the VkVideoFormatPropertiesKHR structure to retrieve video format properties specific to video encode quantization maps used with an AV1 encode profile.

The VkVideoFormatAV1QuantizationMapPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1 with VK_KHR_video_encode_quantization_map
typedef struct VkVideoFormatAV1QuantizationMapPropertiesKHR {
    VkStructureType                          sType;
    void*                                    pNext;
    VkVideoEncodeAV1SuperblockSizeFlagsKHR   compatibleSuperblockSizes;
} VkVideoFormatAV1QuantizationMapPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- compatibleSuperblockSizes is a bitmask of VkVideoEncodeAV1SuperblockSizeFlagBitsKHR indicating the AV1 superblock sizes that quantization maps using this video format are compatible with.

NOTE

The value of compatibleSuperblockSizes does not limit the use of the specific quantization map format, but does limit the implementation in being able to encode pictures with superblock sizes not included in compatibleSuperblockSizes but otherwise supported by the used video profile, as indicated by VkVideoEncodeAV1CapabilitiesKHR::superblockSizes. In particular, using smaller quantization map texel sizes may prevent implementations from encoding with larger superblock sizes which may have a negative impact on the efficiency of the encoder.

The values returned in this structure are only defined if the allowed image usage flags returned in VkVideoFormatPropertiesKHR::imageUsageFlags for this video format include VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR.

Valid Usage (Implicit)

- VUID-VkVideoFormatAV1QuantizationMapPropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_FORMAT_AV1_QUANTIZATION_MAP_PROPERTIES_KHR

38.17.5. Encoding with Quantization Maps

The VkVideoEncodeQuantizationMapInfoKHR structure **can** be included in the pNext chain of the VkVideoEncodeInfoKHR structure passed to the vkCmdEncodeVideoKHR command to specify the quantization map used by the issued video encode operations.

The VkVideoEncodeQuantizationMapInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_quantization_map
typedef struct VkVideoEncodeQuantizationMapInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkImageView        quantizationMap;
    VkExtent2D         quantizationMapExtent;
} VkVideoEncodeQuantizationMapInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `quantizationMap` specifies the image view to use as the quantization map.
- `quantizationMapExtent` specifies the extent of the image subregion of `quantizationMap` to use as the quantization map starting at offset (0,0).

Valid Usage

- VUID-VkVideoEncodeQuantizationMapInfoKHR-quantizationMapExtent-10352
  `quantizationMapExtent.width` **must** be less than or equal to the width of `quantizationMap`
- VUID-VkVideoEncodeQuantizationMapInfoKHR-quantizationMapExtent-10353
  `quantizationMapExtent.height` **must** be less than or equal to the height of `quantizationMap`

Valid Usage (Implicit)

- VUID-VkVideoEncodeQuantizationMapInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_INFO_KHR`
- VUID-VkVideoEncodeQuantizationMapInfoKHR-quantizationMap-parameter
  If `quantizationMap` is not VK_NULL_HANDLE, `quantizationMap` **must** be a valid VkImageView handle

38.18. H.264 Encode Operations

Video encode operations using an H.264 encode profile **can** be used to encode elementary video stream sequences compliant to the ITU-T H.264 Specification.

NOTE Refer to the Preamble for information on how the Khronos Intellectual Property Rights Policy relates to normative references to external materials not created by Khronos.

This process is performed according to the video encode operation steps with the codec-specific semantics defined in section 8 of the ITU-T H.264 Specification as follows:

- Syntax elements, derived values, and other parameters are applied from the following structures:

    - The StdVideoH264SequenceParameterSet structure corresponding to the active SPS specifying the H.264 sequence parameter set.
    - The StdVideoH264PictureParameterSet structure corresponding to the active PPS specifying the H.264 picture parameter set.
    - The StdVideoEncodeH264PictureInfo structure specifying the H.264 picture information.
    - The StdVideoEncodeH264SliceHeader structures specifying the H.264 slice header parameters for each encoded H.264 slice.
    - The StdVideoEncodeH264ReferenceInfo structures specifying the H.264 reference information corresponding to the optional reconstructed picture and any active reference pictures.
- The encoded bitstream data is written to the destination video bitstream buffer range as defined in the H.264 Encode Bitstream Data Access section.
- Picture data in the video picture resources corresponding to the used encode input picture, active reference pictures, and optional reconstructed picture is accessed as defined in the H.264 Encode Picture Data Access section.
- The decision on reference picture setup is made according to the parameters specified in the H.264 picture information.

If the parameters adhere to the syntactic and semantic requirements defined in the corresponding sections of the ITU-T H.264 Specification, as described above, and the DPB slots associated with the active reference pictures all refer to valid picture references, then the video encode operation will complete successfully. Otherwise, the video encode operation **may** complete unsuccessfully.

38.18.1. H.264 Encode Parameter Overrides

Implementations **may** override, unless otherwise specified, any of the H.264 encode parameters specified in the following Video Std structures:

- StdVideoH264SequenceParameterSet
- StdVideoH264PictureParameterSet
- StdVideoEncodeH264PictureInfo
- StdVideoEncodeH264SliceHeader
- StdVideoEncodeH264ReferenceInfo

All such H.264 encode parameter overrides **must** fulfill the conditions defined in the Video Encode Parameter Overrides section.

In addition, implementations **must** not override any of the following H.264 encode parameters:

- StdVideoEncodeH264PictureInfo::primary_pic_type
- StdVideoEncodeH264SliceHeader::slice_type

In case of H.264 encode parameters stored in video session parameters objects, applications need to use the vkGetEncodedVideoSessionParametersKHR command to determine whether any implementation overrides happened. If the query indicates that implementation overrides were applied, then the application needs to retrieve and use the encoded H.264 parameter sets in the

bitstream in order to be able to produce a compliant H.264 video bitstream using the H.264 encode parameters stored in the video session parameters object.

In case of any H.264 encode parameters stored in the encoded bitstream produced by video encode operations, if the implementation supports the VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR video encode feedback query flag, the application **can** use such queries to retrieve feedback about whether any implementation overrides have been applied to those H.264 encode parameters.

38.18.2. H.264 Encode Bitstream Data Access

Each video encode operation writes one or more VCL NAL units comprising of slice headers and data of the encoded picture, in the format defined in sections 7.3.3 and 7.3.4, according to the semantics defined in sections 7.4.3 and 7.4.4 of the ITU-T H.264 Specification, respectively. The number of VCL NAL units written is specified by VkVideoEncodeH264PictureInfoKHR ::naluSliceEntryCount.

In addition, if VkVideoEncodeH264PictureInfoKHR::generatePrefixNalu is VK_TRUE for the video encode operation, then an additional prefix NAL unit is written before each VCL NAL unit corresponding to individual slices in the format defined in section 7.3.2.12, according to the semantics defined in section 7.4.2.12 of the ITU-T H.264 Specification, respectively.

38.18.3. H.264 Encode Picture Data Access

Accesses to image data within a video picture resource happen at the granularity indicated by VkVideoCapabilitiesKHR::pictureAccessGranularity, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile. Accordingly, the complete image subregion of a encode input picture, reference picture, or reconstructed picture accessed by video coding operations using an H.264 encode profile is defined as the set of texels within the coordinate range:

([0,endX), [0,endY))

Where:

- endX equals codedExtent.width rounded up to the nearest integer multiple of pictureAccessGranularity.width and clamped to the width of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;
- endY equals codedExtent.height rounded up to the nearest integer multiple of pictureAccessGranularity.height and clamped to the height of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;

Where codedExtent is the member of the VkVideoPictureResourceInfoKHR structure corresponding to the picture.

In case of video encode operations using an H.264 encode profile, any access to a picture at the coordinates (x,y), as defined by the ITU-T H.264 Specification, is an access to the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure at the texel

coordinates (x,y).

Implementations **may** choose not to access some or all texels within particular reference pictures available to a video encode operation (e.g. due to video encode parameter overrides restricting the effective set of used reference pictures, or if the encoding algorithm chooses not to use certain subregions of the reference picture data for sample prediction).

38.18.4. H.264 Frame, Picture, and Slice

H.264 pictures are partitioned into slices, as defined in section 6.3 of the ITU-T H.264 Specification.

For the purposes of this specification, the H.264 slices comprising a picture are referred to as the *picture partitions* of the picture.

Video encode operations using an H.264 encode profile **can** encode slices of different types, as defined in section 7.4.3 of the ITU-T H.264 Specification, by specifying the corresponding enumeration constant value in `StdVideoEncodeH264SliceHeader`::`slice_type` in the H.264 slice header parameters from the Video Std enumeration type `StdVideoH264SliceType`:

- `STD_VIDEO_H264_SLICE_TYPE_P` indicates that the slice is a *P slice* as defined in section 3.109 of the ITU-T H.264 Specification.
- `STD_VIDEO_H264_SLICE_TYPE_B` indicates that the slice is a *B slice* as defined in section 3.9 of the ITU-T H.264 Specification.
- `STD_VIDEO_H264_SLICE_TYPE_I` indicates that the slice is an *I slice* as defined in section 3.66 of the ITU-T H.264 Specification.

Pictures constructed from such slices **can** be of different types, as defined in section 7.4.2.4 of the ITU-T H.264 Specification. Video encode operations using an H.264 encode profile **can** encode pictures of a specific type by specifying the corresponding enumeration constant value in `StdVideoEncodeH264PictureInfo`::`primary_pic_type` in the H.264 picture information from the Video Std enumeration type `StdVideoH264PictureType`:

- `STD_VIDEO_H264_PICTURE_TYPE_P` indicates that the picture is a *P picture*. A frame consisting of a P picture is also referred to as a *P frame*.
- `STD_VIDEO_H264_PICTURE_TYPE_B` indicates that the picture is a *B picture*. A frame consisting of a B picture is also referred to as a *B frame*.
- `STD_VIDEO_H264_PICTURE_TYPE_I` indicates that the picture is an *I picture*. A frame consisting of an I picture is also referred to as an *I frame*.
- `STD_VIDEO_H264_PICTURE_TYPE_IDR` indicates that the picture is a special type of I picture called an *IDR picture* as defined in section 3.69 of the ITU-T H.264 Specification. A frame consisting of an IDR picture is also referred to as an *IDR frame*.

38.18.5. H.264 Coding Blocks

H.264 encode supports a single type of coding block called a *macroblock*, as defined in section 3.84 of the ITU-T H.264 Specification.

38.18.6. H.264 Encode Profile

A video profile supporting H.264 video encode operations is specified by setting VkVideoProfileInfoKHR::videoCodecOperation to VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR and adding a VkVideoEncodeH264ProfileInfoKHR structure to the VkVideoProfileInfoKHR::pNext chain.

The VkVideoEncodeH264ProfileInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264ProfileInfoKHR {
    VkStructureType           sType;
    const void*               pNext;
    StdVideoH264ProfileIdc    stdProfileIdc;
} VkVideoEncodeH264ProfileInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- stdProfileIdc is a StdVideoH264ProfileIdc value specifying the H.264 codec profile IDC, as defined in section A.2 of the ITU-T H.264 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264ProfileInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_PROFILE_INFO_KHR

38.18.7. H.264 Encode Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities for an H.264 encode profile, the VkVideoCapabilitiesKHR::pNext chain **must** include a VkVideoEncodeH264CapabilitiesKHR structure that will be filled with the profile-specific capabilities.

The VkVideoEncodeH264CapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264CapabilitiesKHR {
    VkStructureType                          sType;
    void*                                    pNext;
    VkVideoEncodeH264CapabilityFlagsKHR      flags;
    StdVideoH264LevelIdc                     maxLevelIdc;
    uint32_t                                 maxSliceCount;
    uint32_t                                 maxPPictureL0ReferenceCount;
    uint32_t                                 maxBPictureL0ReferenceCount;
    uint32_t                                 maxL1ReferenceCount;
    uint32_t                                 maxTemporalLayerCount;
    VkBool32                                 expectDyadicTemporalLayerPattern;
    int32_t                                  minQp;
    int32_t                                  maxQp;
    VkBool32                                 prefersGopRemainingFrames;
    VkBool32                                 requiresGopRemainingFrames;
    VkVideoEncodeH264StdFlagsKHR             stdSyntaxFlags;
} VkVideoEncodeH264CapabilitiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkVideoEncodeH264CapabilityFlagBitsKHR indicating supported H.264 encoding capabilities.
- `maxLevelIdc` is a `StdVideoH264LevelIdc` value indicating the maximum H.264 level supported by the profile, where enum constant `STD_VIDEO_H264_LEVEL_IDC_<major>_<minor>` identifies H.264 level `<major>.<minor>` as defined in section A.3 of the ITU-T H.264 Specification.
- `maxSliceCount` indicates the maximum number of slices that **can** be encoded for a single picture. Further restrictions **may** apply to the number of slices that **can** be encoded for a single picture depending on other capabilities and codec-specific rules.
- `maxPPictureL0ReferenceCount` indicates the maximum number of reference pictures the implementation supports in the reference list L0 for P pictures.

> **NOTE** As implementations **may** override the reference lists, `maxPPictureL0ReferenceCount` does not limit the number of elements that the application **can** specify in the L0 reference list for P pictures. However, if `maxPPictureL0ReferenceCount` is zero, then the use of P pictures is not allowed.

- `maxBPictureL0ReferenceCount` indicates the maximum number of reference pictures the implementation supports in the reference list L0 for B pictures.
- `maxL1ReferenceCount` indicates the maximum number of reference pictures the implementation supports in the reference list L1 if encoding of B pictures is supported.

> **NOTE** As implementations **may** override the reference lists, `maxBPictureL0ReferenceCount` and `maxL1ReferenceCount` does not limit the number of elements that the application **can** specify in the L0 and L1 reference lists for B

> pictures. However, if maxBPictureL0ReferenceCount and maxL1ReferenceCount are both zero, then the use of B pictures is not allowed.

- maxTemporalLayerCount indicates the maximum number of H.264 temporal layers supported by the implementation.
- expectDyadicTemporalLayerPattern indicates that the implementation's rate control algorithms expect the application to use a dyadic temporal layer pattern when encoding multiple temporal layers.
- minQp indicates the minimum QP value supported.
- maxQp indicates the maximum QP value supported.
- prefersGopRemainingFrames indicates that the implementation's rate control algorithm prefers the application to specify the number of frames of each type remaining in the current group of pictures when beginning a video coding scope.
- requiresGopRemainingFrames indicates that the implementation's rate control algorithm requires the application to specify the number of frames of each type remaining in the current group of pictures when beginning a video coding scope.
- stdSyntaxFlags is a bitmask of VkVideoEncodeH264StdFlagBitsKHR indicating capabilities related to H.264 syntax elements.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264CapabilitiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_CAPABILITIES_KHR

Bits which **may** be set in VkVideoEncodeH264CapabilitiesKHR::flags, indicating the H.264 encoding capabilities supported, are:

```
// Provided by VK_KHR_video_encode_h264
typedef enum VkVideoEncodeH264CapabilityFlagBitsKHR {
    VK_VIDEO_ENCODE_H264_CAPABILITY_HRD_COMPLIANCE_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_H264_CAPABILITY_PREDICTION_WEIGHT_TABLE_GENERATED_BIT_KHR =
0x00000002,
    VK_VIDEO_ENCODE_H264_CAPABILITY_ROW_UNALIGNED_SLICE_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_H264_CAPABILITY_DIFFERENT_SLICE_TYPE_BIT_KHR = 0x00000008,
    VK_VIDEO_ENCODE_H264_CAPABILITY_B_FRAME_IN_L0_LIST_BIT_KHR = 0x00000010,
    VK_VIDEO_ENCODE_H264_CAPABILITY_B_FRAME_IN_L1_LIST_BIT_KHR = 0x00000020,
    VK_VIDEO_ENCODE_H264_CAPABILITY_PER_PICTURE_TYPE_MIN_MAX_QP_BIT_KHR = 0x00000040,
    VK_VIDEO_ENCODE_H264_CAPABILITY_PER_SLICE_CONSTANT_QP_BIT_KHR = 0x00000080,
    VK_VIDEO_ENCODE_H264_CAPABILITY_GENERATE_PREFIX_NALU_BIT_KHR = 0x00000100,
  // Provided by VK_KHR_video_encode_h264 with VK_KHR_video_encode_quantization_map
    VK_VIDEO_ENCODE_H264_CAPABILITY_MB_QP_DIFF_WRAPAROUND_BIT_KHR = 0x00000200,
} VkVideoEncodeH264CapabilityFlagBitsKHR;
```

- VK_VIDEO_ENCODE_H264_CAPABILITY_HRD_COMPLIANCE_BIT_KHR specifies whether the implementation

may be able to generate HRD compliant bitstreams if any of the `nal_hrd_parameters_present_flag` or `vcl_hrd_parameters_present_flag` members of `StdVideoH264SpsVuiFlags` are set to `1` in the active SPS.

- `VK_VIDEO_ENCODE_H264_CAPABILITY_PREDICTION_WEIGHT_TABLE_GENERATED_BIT_KHR` specifies that if `StdVideoH264PpsFlags`::`weighted_pred_flag` is set to `1` or `StdVideoH264PictureParameterSet`::`weighted_bipred_idc` is set to `STD_VIDEO_H264_WEIGHTED_BIPRED_IDC_EXPLICIT` in the active PPS when encoding a P picture or B picture, respectively, then the implementation is able to internally decide syntax for `pred_weight_table`, as defined in section 7.4.3.2 of the ITU-T H.264 Specification, and the application is not **required** to provide a weight table in the H.264 slice header parameters.
- `VK_VIDEO_ENCODE_H264_CAPABILITY_ROW_UNALIGNED_SLICE_BIT_KHR` specifies that each slice in a frame with multiple slices may begin or finish at any offset in a macroblock row. If not supported, all slices in the frame **must** begin at the start of a macroblock row (and hence each slice **must** finish at the end of a macroblock row).
- `VK_VIDEO_ENCODE_H264_CAPABILITY_DIFFERENT_SLICE_TYPE_BIT_KHR` specifies that when a frame is encoded with multiple slices, the implementation allows encoding each slice with a different `StdVideoEncodeH264SliceHeader`::`slice_type` specified in the H.264 slice header parameters. If not supported, all slices of the frame **must** be encoded with the same `slice_type` which corresponds to the picture type of the frame.
- `VK_VIDEO_ENCODE_H264_CAPABILITY_B_FRAME_IN_L0_LIST_BIT_KHR` specifies support for using a B frame as L0 reference, as specified in `StdVideoEncodeH264ReferenceListsInfo`::`RefPicList0` in the H.264 picture information.
- `VK_VIDEO_ENCODE_H264_CAPABILITY_B_FRAME_IN_L1_LIST_BIT_KHR` specifies support for using a B frame as L1 reference, as specified in `StdVideoEncodeH264ReferenceListsInfo`::`RefPicList1` in the H.264 picture information.
- `VK_VIDEO_ENCODE_H264_CAPABILITY_PER_PICTURE_TYPE_MIN_MAX_QP_BIT_KHR` specifies support for specifying different QP values in the members of VkVideoEncodeH264QpKHR.
- `VK_VIDEO_ENCODE_H264_CAPABILITY_PER_SLICE_CONSTANT_QP_BIT_KHR` specifies support for specifying different constant QP values for each slice.
- `VK_VIDEO_ENCODE_H264_CAPABILITY_GENERATE_PREFIX_NALU_BIT_KHR` specifies support for generating prefix NAL units by setting VkVideoEncodeH264PictureInfoKHR::`generatePrefixNalu` to `VK_TRUE`.
- `VK_VIDEO_ENCODE_H264_CAPABILITY_MB_QP_DIFF_WRAPAROUND_BIT_KHR` indicates support for wraparound during the calculation of the QP values of subsequently encoded macroblocks, as defined in equation 7-37 of the ITU-T H.264 Specification. If not supported, equation 7-37 of the ITU-T H.264 Specification is effectively reduced to the following:

  $QP_Y = QP_{Y,PREV} + mb_qp_delta$

  NOTE The effect of this is that the maximum QP difference across subsequent macroblocks is limited to the $[-(26 + QpBdOffset_Y / 2), 25 + QpBdOffset_Y / 2]$ range.

```
// Provided by VK_KHR_video_encode_h264
typedef VkFlags VkVideoEncodeH264CapabilityFlagsKHR;
```

VkVideoEncodeH264CapabilityFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeH264CapabilityFlagBitsKHR.

Bits which **may** be set in VkVideoEncodeH264CapabilitiesKHR::stdSyntaxFlags, indicating the capabilities related to the H.264 syntax elements, are:

```
// Provided by VK_KHR_video_encode_h264
typedef enum VkVideoEncodeH264StdFlagBitsKHR {
    VK_VIDEO_ENCODE_H264_STD_SEPARATE_COLOR_PLANE_FLAG_SET_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_H264_STD_QPPRIME_Y_ZERO_TRANSFORM_BYPASS_FLAG_SET_BIT_KHR =
0x00000002,
    VK_VIDEO_ENCODE_H264_STD_SCALING_MATRIX_PRESENT_FLAG_SET_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_H264_STD_CHROMA_QP_INDEX_OFFSET_BIT_KHR = 0x00000008,
    VK_VIDEO_ENCODE_H264_STD_SECOND_CHROMA_QP_INDEX_OFFSET_BIT_KHR = 0x00000010,
    VK_VIDEO_ENCODE_H264_STD_PIC_INIT_QP_MINUS26_BIT_KHR = 0x00000020,
    VK_VIDEO_ENCODE_H264_STD_WEIGHTED_PRED_FLAG_SET_BIT_KHR = 0x00000040,
    VK_VIDEO_ENCODE_H264_STD_WEIGHTED_BIPRED_IDC_EXPLICIT_BIT_KHR = 0x00000080,
    VK_VIDEO_ENCODE_H264_STD_WEIGHTED_BIPRED_IDC_IMPLICIT_BIT_KHR = 0x00000100,
    VK_VIDEO_ENCODE_H264_STD_TRANSFORM_8X8_MODE_FLAG_SET_BIT_KHR = 0x00000200,
    VK_VIDEO_ENCODE_H264_STD_DIRECT_SPATIAL_MV_PRED_FLAG_UNSET_BIT_KHR = 0x00000400,
    VK_VIDEO_ENCODE_H264_STD_ENTROPY_CODING_MODE_FLAG_UNSET_BIT_KHR = 0x00000800,
    VK_VIDEO_ENCODE_H264_STD_ENTROPY_CODING_MODE_FLAG_SET_BIT_KHR = 0x00001000,
    VK_VIDEO_ENCODE_H264_STD_DIRECT_8X8_INFERENCE_FLAG_UNSET_BIT_KHR = 0x00002000,
    VK_VIDEO_ENCODE_H264_STD_CONSTRAINED_INTRA_PRED_FLAG_SET_BIT_KHR = 0x00004000,
    VK_VIDEO_ENCODE_H264_STD_DEBLOCKING_FILTER_DISABLED_BIT_KHR = 0x00008000,
    VK_VIDEO_ENCODE_H264_STD_DEBLOCKING_FILTER_ENABLED_BIT_KHR = 0x00010000,
    VK_VIDEO_ENCODE_H264_STD_DEBLOCKING_FILTER_PARTIAL_BIT_KHR = 0x00020000,
    VK_VIDEO_ENCODE_H264_STD_SLICE_QP_DELTA_BIT_KHR = 0x00080000,
    VK_VIDEO_ENCODE_H264_STD_DIFFERENT_SLICE_QP_DELTA_BIT_KHR = 0x00100000,
} VkVideoEncodeH264StdFlagBitsKHR;
```

- VK_VIDEO_ENCODE_H264_STD_SEPARATE_COLOR_PLANE_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264SpsFlags::separate_colour_plane_flag in the SPS when that value is 1.
- VK_VIDEO_ENCODE_H264_STD_QPPRIME_Y_ZERO_TRANSFORM_BYPASS_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264SpsFlags::qpprime_y_zero_transform_bypass_flag in the SPS when that value is 1.
- VK_VIDEO_ENCODE_H264_STD_SCALING_MATRIX_PRESENT_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided values for StdVideoH264SpsFlags::seq_scaling_matrix_present_flag in the SPS and StdVideoH264PpsFlags::pic_scaling_matrix_present_flag in the PPS when any of those values are 1.
- VK_VIDEO_ENCODE_H264_STD_CHROMA_QP_INDEX_OFFSET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264PictureParameterSet::chroma_qp_index_offset in the PPS when that value is non-zero.
- VK_VIDEO_ENCODE_H264_STD_SECOND_CHROMA_QP_INDEX_OFFSET_BIT_KHR specifies whether the implementation supports using the application-provided value for

StdVideoH264PictureParameterSet::second_chroma_qp_index_offset in the PPS when that value is non-zero.

- VK_VIDEO_ENCODE_H264_STD_PIC_INIT_QP_MINUS26_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264PictureParameterSet::pic_init_qp_minus26 in the PPS when that value is non-zero.
- VK_VIDEO_ENCODE_H264_STD_WEIGHTED_PRED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264PpsFlags::weighted_pred_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H264_STD_WEIGHTED_BIPRED_IDC_EXPLICIT_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264PictureParameterSet::weighted_bipred_idc in the PPS when that value is STD_VIDEO_H264_WEIGHTED_BIPRED_IDC_EXPLICIT.
- VK_VIDEO_ENCODE_H264_STD_WEIGHTED_BIPRED_IDC_IMPLICIT_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264PictureParameterSet::weighted_bipred_idc in the PPS when that value is STD_VIDEO_H264_WEIGHTED_BIPRED_IDC_IMPLICIT.
- VK_VIDEO_ENCODE_H264_STD_TRANSFORM_8X8_MODE_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264PpsFlags::transform_8x8_mode_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H264_STD_DIRECT_SPATIAL_MV_PRED_FLAG_UNSET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeH264SliceHeaderFlags::direct_spatial_mv_pred_flag in the H.264 slice header parameters when that value is 0.
- VK_VIDEO_ENCODE_H264_STD_ENTROPY_CODING_MODE_FLAG_UNSET_BIT_KHR specifies whether the implementation supports CAVLC entropy coding, as defined in section 9.2 of the ITU-T H.264 Specification, and thus supports using the application-provided value for StdVideoH264PpsFlags::entropy_coding_mode_flag in the PPS when that value is 0.
- VK_VIDEO_ENCODE_H264_STD_ENTROPY_CODING_MODE_FLAG_SET_BIT_KHR specifies whether the implementation supports CABAC entropy coding, as defined in section 9.3 of the ITU-T H.264 Specification, and thus supports using the application-provided value for StdVideoH264PpsFlags::entropy_coding_mode_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H264_STD_DIRECT_8X8_INFERENCE_FLAG_UNSET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264SpsFlags::direct_8x8_inference_flag in the SPS when that value is 0.
- VK_VIDEO_ENCODE_H264_STD_CONSTRAINED_INTRA_PRED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH264PpsFlags::constrained_intra_pred_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H264_STD_DEBLOCKING_FILTER_DISABLED_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeH264SliceHeader::disable_deblocking_filter_idc in the H.264 slice header parameters when that value is STD_VIDEO_H264_DISABLE_DEBLOCKING_FILTER_IDC_DISABLED.
- VK_VIDEO_ENCODE_H264_STD_DEBLOCKING_FILTER_ENABLED_BIT_KHR specifies whether the implementation supports using the application-provided value for

StdVideoEncodeH264SliceHeader::disable_deblocking_filter_idc in the H.264 slice header parameters when that value is STD_VIDEO_H264_DISABLE_DEBLOCKING_FILTER_IDC_ENABLED.

- VK_VIDEO_ENCODE_H264_STD_DEBLOCKING_FILTER_PARTIAL_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeH264SliceHeader::disable_deblocking_filter_idc in the H.264 slice header parameters when that value is STD_VIDEO_H264_DISABLE_DEBLOCKING_FILTER_IDC_PARTIAL.
- VK_VIDEO_ENCODE_H264_STD_SLICE_QP_DELTA_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeH264SliceHeader::slice_qp_delta in the H.264 slice header parameters when that value is identical across the slices of the encoded frame.
- VK_VIDEO_ENCODE_H264_STD_DIFFERENT_SLICE_QP_DELTA_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeH264SliceHeader::slice_qp_delta in the H.264 slice header parameters when that value is different across the slices of the encoded frame.

These capability flags provide information to the application about specific H.264 syntax element values that the implementation supports without having to override them and do not otherwise restrict the values that the application **can** specify for any of the mentioned H.264 syntax elements.

```
// Provided by VK_KHR_video_encode_h264
typedef VkFlags VkVideoEncodeH264StdFlagsKHR;
```

VkVideoEncodeH264StdFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeH264StdFlagBitsKHR.

38.18.8. H.264 Encode Quality Level Properties

When calling vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR with pVideoProfile->videoCodecOperation specified as VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, the VkVideoEncodeH264QualityLevelPropertiesKHR structure **must** be included in the pNext chain of the VkVideoEncodeQualityLevelPropertiesKHR structure to retrieve additional video encode quality level properties specific to H.264 encoding.

The VkVideoEncodeH264QualityLevelPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264QualityLevelPropertiesKHR {
    VkStructureType                          sType;
    void*                                    pNext;
    VkVideoEncodeH264RateControlFlagsKHR     preferredRateControlFlags;
    uint32_t                                 preferredGopFrameCount;
    uint32_t                                 preferredIdrPeriod;
    uint32_t                                 preferredConsecutiveBFrameCount;
    uint32_t                                 preferredTemporalLayerCount;
    VkVideoEncodeH264QpKHR                   preferredConstantQp;
    uint32_t                                 preferredMaxL0ReferenceCount;
    uint32_t                                 preferredMaxL1ReferenceCount;
    VkBool32                                 preferredStdEntropyCodingModeFlag;
} VkVideoEncodeH264QualityLevelPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `preferredRateControlFlags` is a bitmask of VkVideoEncodeH264RateControlFlagBitsKHR values indicating the preferred flags to use for VkVideoEncodeH264RateControlInfoKHR::`flags`.
- `preferredGopFrameCount` indicates the preferred value to use for VkVideoEncodeH264RateControlInfoKHR::`gopFrameCount`.
- `preferredIdrPeriod` indicates the preferred value to use for VkVideoEncodeH264RateControlInfoKHR::`idrPeriod`.
- `preferredConsecutiveBFrameCount` indicates the preferred value to use for VkVideoEncodeH264RateControlInfoKHR::`consecutiveBFrameCount`.
- `preferredTemporalLayerCount` indicates the preferred value to use for VkVideoEncodeH264RateControlInfoKHR::`temporalLayerCount`.
- `preferredConstantQp` indicates the preferred values to use for VkVideoEncodeH264NaluSliceInfoKHR::`constantQp` for each picture type when using rate control mode `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`.
- `preferredMaxL0ReferenceCount` indicates the preferred maximum number of reference pictures to use in the reference list L0.
- `preferredMaxL1ReferenceCount` indicates the preferred maximum number of reference pictures to use in the reference list L1.
- `preferredStdEntropyCodingModeFlag` indicates the preferred value to use for `entropy_coding_mode_flag` in `StdVideoH264PpsFlags`.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264QualityLevelPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_QUALITY_LEVEL_PROPERTIES_KHR`

38.18.9. H.264 Encode Session

Additional parameters can be specified when creating a video session with an H.264 encode profile by including an instance of the VkVideoEncodeH264SessionCreateInfoKHR structure in the pNext chain of VkVideoSessionCreateInfoKHR.

The VkVideoEncodeH264SessionCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264SessionCreateInfoKHR {
    VkStructureType         sType;
    const void*             pNext;
    VkBool32                useMaxLevelIdc;
    StdVideoH264LevelIdc    maxLevelIdc;
} VkVideoEncodeH264SessionCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- useMaxLevelIdc indicates whether the value of maxLevelIdc should be used by the implementation. When it is VK_FALSE, the implementation ignores the value of maxLevelIdc and uses the value of VkVideoEncodeH264CapabilitiesKHR::maxLevelIdc, as reported by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile.
- maxLevelIdc is a StdVideoH264LevelIdc value specifying the upper bound on the H.264 level for the video bitstreams produced by the created video session, where enum constant STD_VIDEO_H264_LEVEL_IDC_<major>_<minor> identifies H.264 level <major>.<minor> as defined in section A.3 of the ITU-T H.264 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264SessionCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_CREATE_INFO_KHR

38.18.10. H.264 Encode Parameter Sets

Video session parameters objects created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR **can** contain the following types of parameters:

H.264 Sequence Parameter Sets (SPS)

Represented by StdVideoH264SequenceParameterSet structures and interpreted as follows:

- reserved1 and reserved2 are used only for padding purposes and are otherwise ignored;
- seq_parameter_set_id is used as the key of the SPS entry;
- level_idc is one of the enum constants STD_VIDEO_H264_LEVEL_IDC_<major>_<minor> identifying

the H.264 level `<major>.<minor>` as defined in section A.3 of the ITU-T H.264 Specification;

- if `flags.seq_scaling_matrix_present_flag` is set, then the `StdVideoH264ScalingLists` structure pointed to by `pScalingLists` is interpreted as follows:
  - `scaling_list_present_mask` is a bitmask where bit index i corresponds to `seq_scaling_list_present_flag[i]` as defined in section 7.4.2.1 of the ITU-T H.264 Specification;
  - `use_default_scaling_matrix_mask` is a bitmask where bit index i corresponds to `UseDefaultScalingMatrix4x4Flag[i]`, when i < 6, or corresponds to `UseDefaultScalingMatrix8x8Flag[i-6]`, otherwise, as defined in section 7.3.2.1 of the ITU-T H.264 Specification;
  - `ScalingList4x4` and `ScalingList8x8` correspond to the identically named syntax elements defined in section 7.3.2.1 of the ITU-T H.264 Specification;
- if `flags.vui_parameters_present_flag` is set, then `pSequenceParameterSetVui` is a pointer to a `StdVideoH264SequenceParameterSetVui` structure that is interpreted as follows:
  - `reserved1` is used only for padding purposes and is otherwise ignored;
  - `flags.color_description_present_flag` is interpreted as the value of `colour_description_present_flag`, as defined in section E.2.1 of the ITU-T H.264 Specification;

    NOTE The name of `colour_description_present_flag` was misspelled in the Video Std header.

  - if `flags.nal_hrd_parameters_present_flag` or `flags.vcl_hrd_parameters_present_flag` is set, then the `StdVideoH264HrdParameters` structure pointed to by `pHrdParameters` is interpreted as follows:
    - `reserved1` is used only for padding purposes and is otherwise ignored;
    - all other members of `StdVideoH264HrdParameters` are interpreted as defined in section E.2.2 of the ITU-T H.264 Specification;
  - all other members of `StdVideoH264SequenceParameterSetVui` are interpreted as defined in section E.2.1 of the ITU-T H.264 Specification;
- all other members of `StdVideoH264SequenceParameterSet` are interpreted as defined in section 7.4.2.1 of the ITU-T H.264 Specification.

H.264 Picture Parameter Sets (PPS)

Represented by `StdVideoH264PictureParameterSet` structures and interpreted as follows:

- the pair constructed from `seq_parameter_set_id` and `pic_parameter_set_id` is used as the key of the PPS entry;
- if `flags.pic_scaling_matrix_present_flag` is set, then the `StdVideoH264ScalingLists` structure pointed to by `pScalingLists` is interpreted as follows:
  - `scaling_list_present_mask` is a bitmask where bit index i corresponds to

pic_scaling_list_present_flag[i] as defined in section 7.4.2.2 of the ITU-T H.264 Specification;

- use_default_scaling_matrix_mask is a bitmask where bit index i corresponds to UseDefaultScalingMatrix4x4Flag[i], when i < 6, or corresponds to UseDefaultScalingMatrix8x8Flag[i-6], otherwise, as defined in section 7.3.2.2 of the ITU-T H.264 Specification;
- ScalingList4x4 and ScalingList8x8 correspond to the identically named syntax elements defined in section 7.3.2.2 of the ITU-T H.264 Specification;

- all other members of StdVideoH264PictureParameterSet are interpreted as defined in section 7.4.2.2 of the ITU-T H.264 Specification.

Implementations **may** override any of these parameters according to the semantics defined in the Video Encode Parameter Overrides section before storing the resulting H.264 parameter sets into the video session parameters object. Applications need to use the vkGetEncodedVideoSessionParametersKHR command to determine whether any implementation overrides happened and to retrieve the encoded H.264 parameter sets in order to be able to produce a compliant H.264 video bitstream.

Such H.264 parameter set overrides **may** also have cascading effects on the implementation overrides applied to the encoded bitstream produced by video encode operations. If the implementation supports the VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR video encode feedback query flag, then the application **can** use such queries to retrieve feedback about whether any implementation overrides have been applied to the encoded bitstream.

When a video session parameters object is created with the codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, the VkVideoSessionParametersCreateInfoKHR::pNext chain **must** include a VkVideoEncodeH264SessionParametersCreateInfoKHR structure specifying the capacity and initial contents of the object.

The VkVideoEncodeH264SessionParametersCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264SessionParametersCreateInfoKHR {
    VkStructureType                                 sType;
    const void*                                     pNext;
    uint32_t                                        maxStdSPSCount;
    uint32_t                                        maxStdPPSCount;
    const VkVideoEncodeH264SessionParametersAddInfoKHR*    pParametersAddInfo;
} VkVideoEncodeH264SessionParametersCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxStdSPSCount is the maximum number of H.264 SPS entries the created VkVideoSessionParametersKHR **can** contain.
- maxStdPPSCount is the maximum number of H.264 PPS entries the created VkVideoSessionParametersKHR **can** contain.

- pParametersAddInfo is NULL or a pointer to a VkVideoEncodeH264SessionParametersAddInfoKHR structure specifying H.264 parameters to add upon object creation.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264SessionParametersCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_CREATE_INFO_KHR
- VUID-VkVideoEncodeH264SessionParametersCreateInfoKHR-pParametersAddInfo-parameter
  If pParametersAddInfo is not NULL, pParametersAddInfo **must** be a valid pointer to a valid VkVideoEncodeH264SessionParametersAddInfoKHR structure

The VkVideoEncodeH264SessionParametersAddInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264SessionParametersAddInfoKHR {
    VkStructureType                              sType;
    const void*                                  pNext;
    uint32_t                                     stdSPSCount;
    const StdVideoH264SequenceParameterSet*      pStdSPSs;
    uint32_t                                     stdPPSCount;
    const StdVideoH264PictureParameterSet*       pStdPPSs;
} VkVideoEncodeH264SessionParametersAddInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- stdSPSCount is the number of elements in the pStdSPSs array.
- pStdSPSs is a pointer to an array of StdVideoH264SequenceParameterSet structures describing the H.264 SPS entries to add.
- stdPPSCount is the number of elements in the pStdPPSs array.
- pStdPPSs is a pointer to an array of StdVideoH264PictureParameterSet structures describing the H.264 PPS entries to add.

This structure **can** be specified in the following places:

- In the pParametersAddInfo member of the VkVideoEncodeH264SessionParametersCreateInfoKHR structure specified in the pNext chain of VkVideoSessionParametersCreateInfoKHR used to create a video session parameters object. In this case, if the video codec operation the video session parameters object is created with is VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then it defines the set of initial parameters to add to the created object (see Creating Video Session Parameters).
- In the pNext chain of VkVideoSessionParametersUpdateInfoKHR. In this case, if the video codec operation the video session parameters object to be updated was created with is VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR, then it defines the set of parameters to add to it

(see Updating Video Session Parameters).

Valid Usage

- VUID-VkVideoEncodeH264SessionParametersAddInfoKHR-None-04837
  The `seq_parameter_set_id` member of each `StdVideoH264SequenceParameterSet` structure specified in the elements of `pStdSPSs` **must** be unique within `pStdSPSs`
- VUID-VkVideoEncodeH264SessionParametersAddInfoKHR-None-04838
  The pair constructed from the `seq_parameter_set_id` and `pic_parameter_set_id` members of each `StdVideoH264PictureParameterSet` structure specified in the elements of `pStdPPSs` **must** be unique within `pStdPPSs`

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264SessionParametersAddInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_ADD_INFO_KHR`
- VUID-VkVideoEncodeH264SessionParametersAddInfoKHR-pStdSPSs-parameter
  If `stdSPSCount` is not `0`, and `pStdSPSs` is not `NULL`, `pStdSPSs` **must** be a valid pointer to an array of `stdSPSCount` `StdVideoH264SequenceParameterSet` values
- VUID-VkVideoEncodeH264SessionParametersAddInfoKHR-pStdPPSs-parameter
  If `stdPPSCount` is not `0`, and `pStdPPSs` is not `NULL`, `pStdPPSs` **must** be a valid pointer to an array of `stdPPSCount` `StdVideoH264PictureParameterSet` values

The `VkVideoEncodeH264SessionParametersGetInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264SessionParametersGetInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           writeStdSPS;
    VkBool32           writeStdPPS;
    uint32_t           stdSPSId;
    uint32_t           stdPPSId;
} VkVideoEncodeH264SessionParametersGetInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `writeStdSPS` indicates whether the encoded H.264 sequence parameter set identified by `stdSPSId` is requested to be retrieved.
- `writeStdPPS` indicates whether the encoded H.264 picture parameter set identified by the pair constructed from `stdSPSId` and `stdPPSId` is requested to be retrieved.
- `stdSPSId` specifies the H.264 sequence parameter set ID used to identify the retrieved H.264 sequence and/or picture parameter set(s).

- `stdPPSId` specifies the H.264 picture parameter set ID used to identify the retrieved H.264 picture parameter set when `writeStdPPS` is `VK_TRUE`.

When this structure is specified in the `pNext` chain of the VkVideoEncodeSessionParametersGetInfoKHR structure passed to vkGetEncodedVideoSessionParametersKHR, the command will write encoded parameter data to the output buffer in the following order:

1. The H.264 sequence parameter set identified by `stdSPSId`, if `writeStdSPS` is `VK_TRUE`.
2. The H.264 picture parameter set identified by the pair constructed from `stdSPSId` and `stdPPSId`, if `writeStdPPS` is `VK_TRUE`.

Valid Usage

- VUID-VkVideoEncodeH264SessionParametersGetInfoKHR-writeStdSPS-08279
  At least one of `writeStdSPS` and `writeStdPPS` **must** be `VK_TRUE`

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264SessionParametersGetInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_GET_INFO_KHR`

The `VkVideoEncodeH264SessionParametersFeedbackInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264SessionParametersFeedbackInfoKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           hasStdSPSOverrides;
    VkBool32           hasStdPPSOverrides;
} VkVideoEncodeH264SessionParametersFeedbackInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `hasStdSPSOverrides` indicates whether any of the parameters of the requested H.264 sequence parameter set, if one was requested via VkVideoEncodeH264SessionParametersGetInfoKHR ::`writeStdSPS`, were overridden by the implementation.
- `hasStdPPSOverrides` indicates whether any of the parameters of the requested H.264 picture parameter set, if one was requested via VkVideoEncodeH264SessionParametersGetInfoKHR ::`writeStdPPS`, were overridden by the implementation.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264SessionParametersFeedbackInfoKHR-sType-sType

sType must be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_FEEDBACK_INFO_KHR

38.18.11. H.264 Encoding Parameters

The VkVideoEncodeH264PictureInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264PictureInfoKHR {
    VkStructureType                               sType;
    const void*                                   pNext;
    uint32_t                                      naluSliceEntryCount;
    const VkVideoEncodeH264NaluSliceInfoKHR*      pNaluSliceEntries;
    const StdVideoEncodeH264PictureInfo*          pStdPictureInfo;
    VkBool32                                      generatePrefixNalu;
} VkVideoEncodeH264PictureInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- naluSliceEntryCount is the number of elements in pNaluSliceEntries.
- pNaluSliceEntries is a pointer to an array of naluSliceEntryCount VkVideoEncodeH264NaluSliceInfoKHR structures specifying the parameters of the individual H.264 slices to encode for the input picture.
- pStdPictureInfo is a pointer to a StdVideoEncodeH264PictureInfo structure specifying H.264 picture information.
- generatePrefixNalu controls whether prefix NALUs are generated before slice NALUs into the target bitstream, as defined in sections 7.3.2.12 and 7.4.2.12 of the ITU-T H.264 Specification.

This structure is specified in the pNext chain of the VkVideoEncodeInfoKHR structure passed to vkCmdEncodeVideoKHR to specify the codec-specific picture information for an H.264 encode operation.

Encode Input Picture Information

When this structure is specified in the pNext chain of the VkVideoEncodeInfoKHR structure passed to vkCmdEncodeVideoKHR, the information related to the encode input picture is defined as follows:

- The image subregion used is determined according to the H.264 Encode Picture Data Access section.
- The encode input picture is associated with the H.264 picture information provided in pStdPictureInfo.

Std Picture Information

The members of the StdVideoEncodeH264PictureInfo structure pointed to by pStdPictureInfo are interpreted as follows:

- `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
- `flags.IdrPicFlag` as defined in section 7.4.1 of the ITU-T H.264 Specification;
- `flags.is_reference` as defined in section 3.136 of the ITU-T H.264 Specification;
- `seq_parameter_set_id` and `pic_parameter_set_id` are used to identify the active parameter sets, as described below;
- `primary_pic_type` as defined in section 7.4.2 of the ITU-T H.264 Specification;
- `PicOrderCnt` as defined in section 8.2 of the ITU-T H.264 Specification;
- `temporal_id` as defined in section G.7.4.1.1 of the ITU-T H.264 Specification;
- if `pRefLists` is not `NULL`, then it is a pointer to a `StdVideoEncodeH264ReferenceListsInfo` structure that is interpreted as follows:
  - `flags.reserved` is used only for padding purposes and is otherwise ignored;
  - `ref_pic_list_modification_flag_l0` and `ref_pic_list_modification_flag_l1` as defined in section 7.4.3.1 of the ITU-T H.264 Specification;
  - `num_ref_idx_l0_active_minus1` and `num_ref_idx_l1_active_minus1` as defined in section 7.4.3 of the ITU-T H.264 Specification;
  - `RefPicList0` and `RefPicList1` as defined in section 8.2.4 of the ITU-T H.264 Specification where each element of these arrays either identifies an active reference picture using its DPB slot index or contains the value `STD_VIDEO_H264_NO_REFERENCE_PICTURE` to indicate “no reference picture”;
  - if `refList0ModOpCount` is not zero, then `pRefList0ModOperations` is a pointer to an array of `refList0ModOpCount` number of `StdVideoEncodeH264RefListModEntry` structures specifying the modification parameters for the reference list L0 as defined in section 7.4.3.1 of the ITU-T H.264 Specification;
  - if `refList1ModOpCount` is not zero, then `pRefList1ModOperations` is a pointer to an array of `refList1ModOpCount` number of `StdVideoEncodeH264RefListModEntry` structures specifying the modification parameters for the reference list L1 as defined in section 7.4.3.1 of the ITU-T H.264 Specification;
  - if `refPicMarkingOpCount` is not zero, then `refPicMarkingOperations` is a pointer to an array of `refPicMarkingOpCount` number of `StdVideoEncodeH264RefPicMarkingEntry` structures specifying the reference picture marking parameters as defined in section 7.4.3.3 of the ITU-T H.264 Specification;
- all other members are interpreted as defined in section 7.4.3 of the ITU-T H.264 Specification.

Reference picture setup is controlled by the value of `StdVideoEncodeH264PictureInfo::flags.is_reference`. If it is set and a reconstructed picture is specified, then the latter is used as the target of picture reconstruction to activate the DPB slot specified in `pEncodeInfo->pSetupReferenceSlot->slotIndex`. If `StdVideoEncodeH264PictureInfo::flags.is_reference` is not set, but a reconstructed picture is specified, then the corresponding picture reference associated with the DPB slot is invalidated, as described in the DPB Slot States section.

Active Parameter Sets

The members of the `StdVideoEncodeH264PictureInfo` structure pointed to by `pStdPictureInfo` are

used to select the active parameter sets to use from the bound video session parameters object, as follows:

- The *active SPS* is the SPS identified by the key specified in StdVideoEncodeH264PictureInfo::seq_parameter_set_id.
- The *active PPS* is the PPS identified by the key specified by the pair constructed from StdVideoEncodeH264PictureInfo::seq_parameter_set_id and StdVideoEncodeH264PictureInfo::pic_parameter_set_id.

H.264 encoding uses *explicit weighted sample prediction* for a slice, as defined in section 8.4.2.3 of the ITU-T H.264 Specification, if any of the following conditions are true for the active PPS and the pStdSliceHeader member of the corresponding element of pNaluSliceEntries:

- pStdSliceHeader->slice_type is STD_VIDEO_H264_SLICE_TYPE_P and weighted_pred_flag is enabled in the active PPS.
- pStdSliceHeader->slice_type is STD_VIDEO_H264_SLICE_TYPE_B and weighted_bipred_idc in the active PPS equals STD_VIDEO_H264_WEIGHTED_BIPRED_IDC_EXPLICIT.

Valid Usage

- VUID-VkVideoEncodeH264PictureInfoKHR-naluSliceEntryCount-08301
  naluSliceEntryCount **must** be between 1 and VkVideoEncodeH264CapabilitiesKHR::maxSliceCount, inclusive, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeH264PictureInfoKHR-flags-08304
  If VkVideoEncodeH264CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H264_CAPABILITY_GENERATE_PREFIX_NALU_BIT_KHR, then generatePrefixNalu **must** be VK_FALSE
- VUID-VkVideoEncodeH264PictureInfoKHR-flags-08314
  If VkVideoEncodeH264CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H264_CAPABILITY_PREDICTION_WEIGHT_TABLE_GENERATED_BIT_KHR and the slice corresponding to any element of pNaluSliceEntries uses explicit weighted sample prediction, then VkVideoEncodeH264NaluSliceInfoKHR::pStdSliceHeader->pWeightTable **must** not be NULL for that element of pNaluSliceEntries
- VUID-VkVideoEncodeH264PictureInfoKHR-flags-08315
  If VkVideoEncodeH264CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H264_CAPABILITY_DIFFERENT_SLICE_TYPE_BIT_KHR, then VkVideoEncodeH264NaluSliceInfoKHR::pStdSliceHeader->slice_type **must** be identical for all elements of pNaluSliceEntries

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264PictureInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_PICTURE_INFO_KHR`
- VUID-VkVideoEncodeH264PictureInfoKHR-pNaluSliceEntries-parameter
  `pNaluSliceEntries` **must** be a valid pointer to an array of `naluSliceEntryCount` valid VkVideoEncodeH264NaluSliceInfoKHR structures
- VUID-VkVideoEncodeH264PictureInfoKHR-pStdPictureInfo-parameter
  `pStdPictureInfo` **must** be a valid pointer to a valid `StdVideoEncodeH264PictureInfo` value
- VUID-VkVideoEncodeH264PictureInfoKHR-naluSliceEntryCount-arraylength
  `naluSliceEntryCount` **must** be greater than `0`

The VkVideoEncodeH264NaluSliceInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264NaluSliceInfoKHR {
    VkStructureType                         sType;
    const void*                             pNext;
    int32_t                                 constantQp;
    const StdVideoEncodeH264SliceHeader*    pStdSliceHeader;
} VkVideoEncodeH264NaluSliceInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `constantQp` is the QP to use for the slice if the current rate control mode configured for the video session is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`.
- `pStdSliceHeader` is a pointer to a `StdVideoEncodeH264SliceHeader` structure specifying H.264 slice header parameters for the slice.

Std Slice Header Parameters

The members of the `StdVideoEncodeH264SliceHeader` structure pointed to by `pStdSliceHeader` are interpreted as follows:

- `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
- if `pWeightTable` is not `NULL`, then it is a pointer to a `StdVideoEncodeH264WeightTable` that is interpreted as follows:
  - `flags.reserved` is used only for padding purposes and is otherwise ignored;
  - all other members of `StdVideoEncodeH264WeightTable` are interpreted as defined in section 7.4.3.2 of the ITU-T H.264 Specification;
- all other members are interpreted as defined in section 7.4.3 of the ITU-T H.264 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264NaluSliceInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_NALU_SLICE_INFO_KHR`
- VUID-VkVideoEncodeH264NaluSliceInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkVideoEncodeH264NaluSliceInfoKHR-pStdSliceHeader-parameter
  `pStdSliceHeader` **must** be a valid pointer to a valid `StdVideoEncodeH264SliceHeader` value

The VkVideoEncodeH264DpbSlotInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264DpbSlotInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoEncodeH264ReferenceInfo*   pStdReferenceInfo;
} VkVideoEncodeH264DpbSlotInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pStdReferenceInfo` is a pointer to a `StdVideoEncodeH264ReferenceInfo` structure specifying H.264 reference information.

This structure is specified in the `pNext` chain of VkVideoEncodeInfoKHR::`pSetupReferenceSlot`, if not `NULL`, and the `pNext` chain of the elements of VkVideoEncodeInfoKHR::`pReferenceSlots` to specify the codec-specific reference picture information for an H.264 encode operation.

Active Reference Picture Information

When this structure is specified in the `pNext` chain of the elements of VkVideoEncodeInfoKHR ::`pReferenceSlots`, one element is added to the list of active reference pictures used by the video encode operation for each element of VkVideoEncodeInfoKHR::`pReferenceSlots` as follows:

- The image subregion used is determined according to the H.264 Encode Picture Data Access section.
- The reference picture is associated with the DPB slot index specified in the `slotIndex` member of the corresponding element of VkVideoEncodeInfoKHR::`pReferenceSlots`.
- The reference picture is associated with the H.264 reference information provided in `pStdReferenceInfo`.

Reconstructed Picture Information

When this structure is specified in the `pNext` chain of VkVideoEncodeInfoKHR ::`pSetupReferenceSlot`, the information related to the reconstructed picture is defined as follows:

- The image subregion used is determined according to the H.264 Encode Picture Data Access

section.

- If reference picture setup is requested, then the reconstructed picture is used to activate the DPB slot with the index specified in VkVideoEncodeInfoKHR::pSetupReferenceSlot->slotIndex.
- The reconstructed picture is associated with the H.264 reference information provided in pStdReferenceInfo.

Std Reference Information

The members of the StdVideoEncodeH264ReferenceInfo structure pointed to by pStdReferenceInfo are interpreted as follows:

- flags.reserved is used only for padding purposes and is otherwise ignored;
- flags.used_for_long_term_reference is used to indicate whether the picture is marked as "used for long-term reference" as defined in section 8.2.5.1 of the ITU-T H.264 Specification;
- primary_pic_type as defined in section 7.4.2 of the ITU-T H.264 Specification;
- long_term_pic_num and long_term_frame_idx as defined in section 7.4.3 of the ITU-T H.264 Specification;
- temporal_id as defined in section G.7.4.1.1 of the ITU-T H.264 Specification;
- all other members are interpreted as defined in section 8.2 of the ITU-T H.264 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264DpbSlotInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_DPB_SLOT_INFO_KHR
- VUID-VkVideoEncodeH264DpbSlotInfoKHR-pStdReferenceInfo-parameter
  pStdReferenceInfo **must** be a valid pointer to a valid StdVideoEncodeH264ReferenceInfo value

38.18.12. H.264 Encode Rate Control

Group of Pictures

In case of H.264 encoding it is common practice to follow a regular pattern of different picture types in display order when encoding subsequent frames. This pattern is referred to as the *group of pictures* (GOP).

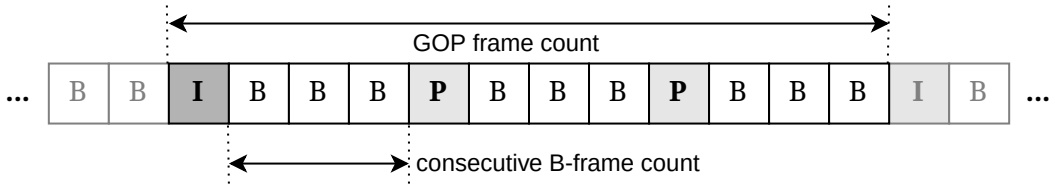
A regular GOP is defined by the following parameters:

- The number of frames in the GOP;
- The number of consecutive B frames between I and/or P frames in display order.

GOPs are further classified as *open* and *closed* GOPs.

Frame types in an open GOP follow each other in display order according to the following algorithm:

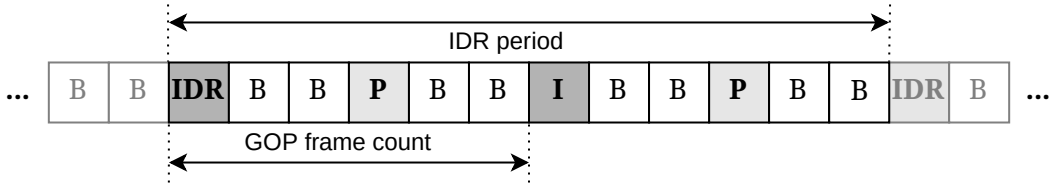
1. The first frame is always an I frame.
2. This is followed by a number of consecutive B frames, as defined above.
3. If the number of frames in the GOP is not reached yet, then the next frame is a P frame and the algorithm continues from step 2.



Text is not SVG - cannot display

Figure 26. H.264 open GOP

In case of a closed GOP, an IDR frame is used at a certain period.



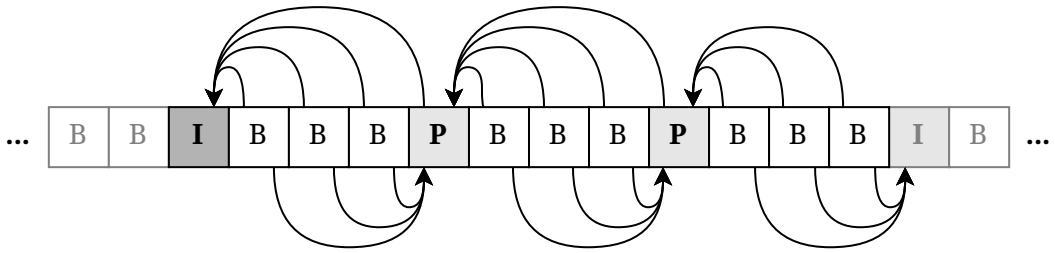
Text is not SVG - cannot display

Figure 27. H.264 closed GOP

It is also typical for H.264 encoding to use specific reference picture usage patterns across the frames of the GOP. The two most common reference patterns used are as follows:

Flat Reference Pattern

- Each P frame uses the last non-B frame, in display order, as reference.
- Each B frame uses the last non-B frame, in display order, as its forward reference, and uses the next non-B frame, in display order, as its backward reference.

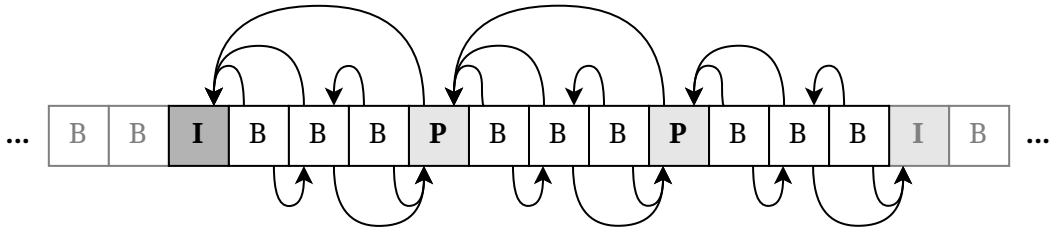


Text is not SVG - cannot display

Figure 28. H.264 flat reference pattern

Dyadic Reference Pattern

- Each P frame uses the last non-B frame, in display order, as reference.
- The following algorithm is applied to the sequence of consecutive B frames between I and/or P frames in display order:
  1. The B frame in the middle of this sequence uses the frame preceding the sequence as its forward reference, and uses the frame following the sequence as its backward reference.
  2. The algorithm is executed recursively for the following frame sequences:
     - The B frames of the original sequence preceding the frame in the middle, if any.
     - The B frames of the original sequence following the frame in the middle, if any.



Text is not SVG - cannot display

Figure 29. H.264 dyadic reference pattern

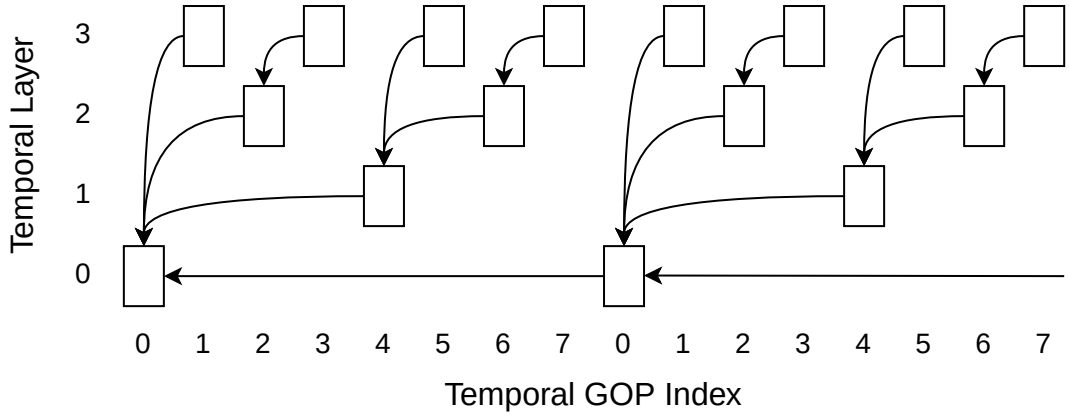
The application **can** provide guidance to the implementation's rate control algorithm about the structure of the GOP used by the application. Any such guidance about the GOP and its structure does not mandate that specific GOP structure to be used by the application, as the picture type of individual encoded pictures is still application-controlled, however, any deviation from the provided guidance **may** result in undesired rate control behavior including, but not limited, to the implementation not being able to conform to the expected average or target bitrates, or other rate control parameters specified by the application.

When an H.264 encode session is used to encode multiple temporal layers, it is also common practice to follow a regular pattern for the H.264 temporal ID for the encoded pictures in display order when encoding subsequent frames. This pattern is referred to as the *temporal GOP*. The most

common temporal layer pattern used is as follows:

Dyadic Temporal Layer Pattern

- The number of frames in the temporal GOP is 2^{n-1}, where n is the number of temporal layers.
- The i^{th} frame in the temporal GOP uses temporal ID t, if and only if the index of the least significant bit set in i equals n-t-1, except for the first frame, which is the only frame in the temporal GOP using temporal ID zero.
- The i^{th} frame in the temporal GOP uses the r^{th} frame as reference, where r is calculated from i by clearing the least significant bit set in it, except for the first frame in the temporal GOP, which uses the first frame of the previous temporal GOP, if any, as reference.



Text is not SVG - cannot display

Figure 30. H.264 dyadic temporal layer pattern

NOTE	Multi-layer rate control and multi-layer coding are typically used for streaming cases where low latency is expected, hence B pictures with backward prediction are usually not used.

The `VkVideoEncodeH264RateControlInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264RateControlInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    VkVideoEncodeH264RateControlFlagsKHR     flags;
    uint32_t                                 gopFrameCount;
    uint32_t                                 idrPeriod;
    uint32_t                                 consecutiveBFrameCount;
    uint32_t                                 temporalLayerCount;
} VkVideoEncodeH264RateControlInfoKHR;
```

- `sType` is a `VkStructureType` value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkVideoEncodeH264RateControlFlagBitsKHR specifying H.264 rate control flags.
- `gopFrameCount` is the number of frames within a group of pictures (GOP) intended to be used by the application. If it is 0, the rate control algorithm **may** assume an implementation-dependent GOP length. If it is `UINT32_MAX`, the GOP length is treated as infinite.
- `idrPeriod` is the interval, in terms of number of frames, between two IDR frames (see IDR period). If it is 0, the rate control algorithm **may** assume an implementation-dependent IDR period. If it is `UINT32_MAX`, the IDR period is treated as infinite.
- `consecutiveBFrameCount` is the number of consecutive B frames between I and/or P frames within the GOP.
- `temporalLayerCount` specifies the number of H.264 temporal layers that the application intends to use.

When an instance of this structure is included in the `pNext` chain of the VkVideoCodingControlInfoKHR structure passed to the vkCmdControlVideoCodingKHR command, and VkVideoCodingControlInfoKHR::`flags` includes `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR`, the parameters in this structure are used as guidance for the implementation's rate control algorithm (see Video Coding Control).

If `flags` includes `VK_VIDEO_ENCODE_H264_RATE_CONTROL_ATTEMPT_HRD_COMPLIANCE_BIT_KHR`, then the rate control state is reset to an initial state to meet HRD compliance requirements. Otherwise the new rate control state **may** be applied without a reset depending on the implementation and the specified rate control parameters.

NOTE It would be possible to infer the picture type to be used when encoding a frame, on the basis of the values provided for `consecutiveBFrameCount`, `idrPeriod`, and `gopFrameCount`, but this inferred picture type will not be used by implementations to override the picture type provided to the video encode operation.

Valid Usage

- VUID-VkVideoEncodeH264RateControlInfoKHR-flags-08280
  If VkVideoEncodeH264CapabilitiesKHR::`flags`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include `VK_VIDEO_ENCODE_H264_CAPABILITY_HRD_COMPLIANCE_BIT_KHR`, then `flags` **must** not contain `VK_VIDEO_ENCODE_H264_RATE_CONTROL_ATTEMPT_HRD_COMPLIANCE_BIT_KHR`
- VUID-VkVideoEncodeH264RateControlInfoKHR-flags-08281
  If `flags` contains `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR` or `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR`, then it **must** also contain `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REGULAR_GOP_BIT_KHR`
- VUID-VkVideoEncodeH264RateControlInfoKHR-flags-08282
  If `flags` contains `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR`, then it **must** not also contain `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR`

- VUID-VkVideoEncodeH264RateControlInfoKHR-flags-08283
  If `flags` contains `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REGULAR_GOP_BIT_KHR`, then `gopFrameCount` **must** be greater than `0`
- VUID-VkVideoEncodeH264RateControlInfoKHR-idrPeriod-08284
  If `idrPeriod` is not `0`, then it **must** be greater than or equal to `gopFrameCount`
- VUID-VkVideoEncodeH264RateControlInfoKHR-consecutiveBFrameCount-08285
  If `consecutiveBFrameCount` is not `0`, then it **must** be less than `gopFrameCount`

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264RateControlInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_RATE_CONTROL_INFO_KHR`
- VUID-VkVideoEncodeH264RateControlInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkVideoEncodeH264RateControlFlagBitsKHR values

Bits which **can** be set in VkVideoEncodeH264RateControlInfoKHR::`flags`, specifying H.264 rate control flags, are:

```
// Provided by VK_KHR_video_encode_h264
typedef enum VkVideoEncodeH264RateControlFlagBitsKHR {
    VK_VIDEO_ENCODE_H264_RATE_CONTROL_ATTEMPT_HRD_COMPLIANCE_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_H264_RATE_CONTROL_REGULAR_GOP_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_H264_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_H264_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR = 0x00000008,
    VK_VIDEO_ENCODE_H264_RATE_CONTROL_TEMPORAL_LAYER_PATTERN_DYADIC_BIT_KHR =
0x00000010,
} VkVideoEncodeH264RateControlFlagBitsKHR;
```

- `VK_VIDEO_ENCODE_H264_RATE_CONTROL_ATTEMPT_HRD_COMPLIANCE_BIT_KHR` specifies that rate control **should** attempt to produce an HRD compliant bitstream, as defined in annex C of the ITU-T H.264 Specification.
- `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REGULAR_GOP_BIT_KHR` specifies that the application intends to use a regular GOP structure according to the parameters specified in the `gopFrameCount`, `idrPeriod`, and `consecutiveBFrameCount` members of the VkVideoEncodeH264RateControlInfoKHR structure.
- `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR` specifies that the application intends to follow a flat reference pattern in the GOP.
- `VK_VIDEO_ENCODE_H264_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR` specifies that the application intends to follow a dyadic reference pattern in the GOP.
- `VK_VIDEO_ENCODE_H264_RATE_CONTROL_TEMPORAL_LAYER_PATTERN_DYADIC_BIT_KHR` specifies that the application intends to follow a dyadic temporal layer pattern.

```
// Provided by VK_KHR_video_encode_h264
typedef VkFlags VkVideoEncodeH264RateControlFlagsKHR;
```

VkVideoEncodeH264RateControlFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeH264RateControlFlagBitsKHR.

Rate Control Layers

The VkVideoEncodeH264RateControlLayerInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264RateControlLayerInfoKHR {
    VkStructureType                  sType;
    const void*                      pNext;
    VkBool32                         useMinQp;
    VkVideoEncodeH264QpKHR           minQp;
    VkBool32                         useMaxQp;
    VkVideoEncodeH264QpKHR           maxQp;
    VkBool32                         useMaxFrameSize;
    VkVideoEncodeH264FrameSizeKHR    maxFrameSize;
} VkVideoEncodeH264RateControlLayerInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- useMinQp indicates whether the QP values determined by rate control will be clamped to the lower bounds on the QP values specified in minQp.
- minQp specifies the lower bounds on the QP values, for each picture type, that the implementation's rate control algorithm will use when useMinQp is VK_TRUE.
- useMaxQp indicates whether the QP values determined by rate control will be clamped to the upper bounds on the QP values specified in maxQp.
- maxQp specifies the upper bounds on the QP values, for each picture type, that the implementation's rate control algorithm will use when useMaxQp is VK_TRUE.
- useMaxFrameSize indicates whether the implementation's rate control algorithm **should** use the values specified in maxFrameSize as the upper bounds on the encoded frame size for each picture type.
- maxFrameSize specifies the upper bounds on the encoded frame size, for each picture type, when useMaxFrameSize is VK_TRUE.

When used, the values in minQp and maxQp guarantee that the effective QP values used by the implementation will respect those lower and upper bounds, respectively. However, limiting the range of QP values that the implementation is able to use will also limit the capabilities of the implementation's rate control algorithm to comply to other constraints. In particular, the implementation **may** not be able to comply to the following:

- The average and/or peak bitrate values to be used for the encoded bitstream specified in the

`averageBitrate` and `maxBitrate` members of the VkVideoEncodeRateControlLayerInfoKHR structure.

- The upper bounds on the encoded frame size, for each picture type, specified in the `maxFrameSize` member of `VkVideoEncodeH264RateControlLayerInfoKHR`.

NOTE	In general, applications need to configure rate control parameters appropriately in order to be able to get the desired rate control behavior, as described in the Video Encode Rate Control section.

When an instance of this structure is included in the `pNext` chain of a VkVideoEncodeRateControlLayerInfoKHR structure specified in one of the elements of the `pLayers` array member of the VkVideoEncodeRateControlInfoKHR structure passed to the vkCmdControlVideoCodingKHR command, VkVideoCodingControlInfoKHR::`flags` includes `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR`, and the bound video session was created with the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR`, it specifies the H.264-specific rate control parameters of the rate control layer corresponding to that element of `pLayers`.

Valid Usage

- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-useMinQp-08286
  If `useMinQp` is `VK_TRUE`, then the `qpI`, `qpP`, and `qpB` members of `minQp` **must** all be between VkVideoEncodeH264CapabilitiesKHR::`minQp` and VkVideoEncodeH264CapabilitiesKHR::`maxQp`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-useMaxQp-08287
  If `useMaxQp` is `VK_TRUE`, then the `qpI`, `qpP`, and `qpB` members of `maxQp` **must** all be between VkVideoEncodeH264CapabilitiesKHR::`minQp` and VkVideoEncodeH264CapabilitiesKHR::`maxQp`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-useMinQp-08288
  If `useMinQp` is `VK_TRUE` and VkVideoEncodeH264CapabilitiesKHR::`flags`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include `VK_VIDEO_ENCODE_H264_CAPABILITY_PER_PICTURE_TYPE_MIN_MAX_QP_BIT_KHR`, then the `qpI`, `qpP`, and `qpB` members of `minQp` **must** all specify the same value
- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-useMaxQp-08289
  If `useMaxQp` is `VK_TRUE` and VkVideoEncodeH264CapabilitiesKHR::`flags`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include `VK_VIDEO_ENCODE_H264_CAPABILITY_PER_PICTURE_TYPE_MIN_MAX_QP_BIT_KHR`, then the `qpI`, `qpP`, and `qpB` members of `maxQp` **must** all specify the same value
- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-useMinQp-08374
  If `useMinQp` and `useMaxQp` are both `VK_TRUE`, then the `qpI`, `qpP`, and `qpB` members of `minQp` **must** all be less than or equal to the respective members of `maxQp`

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_RATE_CONTROL_LAYER_INFO_KHR`
- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-minQp-parameter
  `minQp` **must** be a valid VkVideoEncodeH264QpKHR structure
- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-maxQp-parameter
  `maxQp` **must** be a valid VkVideoEncodeH264QpKHR structure
- VUID-VkVideoEncodeH264RateControlLayerInfoKHR-maxFrameSize-parameter
  `maxFrameSize` **must** be a valid VkVideoEncodeH264FrameSizeKHR structure

The `VkVideoEncodeH264QpKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264QpKHR {
    int32_t    qpI;
    int32_t    qpP;
    int32_t    qpB;
} VkVideoEncodeH264QpKHR;
```

- `qpI` is the QP to be used for I pictures.
- `qpP` is the QP to be used for P pictures.
- `qpB` is the QP to be used for B pictures.

The `VkVideoEncodeH264FrameSizeKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264FrameSizeKHR {
    uint32_t    frameISize;
    uint32_t    framePSize;
    uint32_t    frameBSize;
} VkVideoEncodeH264FrameSizeKHR;
```

- `frameISize` is the size in bytes to be used for I pictures.
- `framePSize` is the size in bytes to be used for P pictures.
- `frameBSize` is the size in bytes to be used for B pictures.

GOP Remaining Frames

Besides session level rate control configuration, the application **can** specify the number of frames per frame type remaining in the group of pictures (GOP).

The `VkVideoEncodeH264GopRemainingFrameInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h264
typedef struct VkVideoEncodeH264GopRemainingFrameInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           useGopRemainingFrames;
    uint32_t           gopRemainingI;
    uint32_t           gopRemainingP;
    uint32_t           gopRemainingB;
} VkVideoEncodeH264GopRemainingFrameInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `useGopRemainingFrames` indicates whether the implementation's rate control algorithm **should** use the values specified in `gopRemainingI`, `gopRemainingP`, and `gopRemainingB`. If `useGopRemainingFrames` is `VK_FALSE`, then the values of `gopRemainingI`, `gopRemainingP`, and `gopRemainingB` are ignored.
- `gopRemainingI` specifies the number of I frames the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the video encode operation.
- `gopRemainingP` specifies the number of P frames the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the video encode operation.
- `gopRemainingB` specifies the number of B frames the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the video encode operation.

Setting `useGopRemainingFrames` to `VK_TRUE` and including this structure in the `pNext` chain of VkVideoBeginCodingInfoKHR is only mandatory if the VkVideoEncodeH264CapabilitiesKHR ::`requiresGopRemainingFrames` reported for the used video profile is `VK_TRUE`. However, implementations **may** use these remaining frame counts, when specified, even when it is not required. In particular, when the application does not use a regular GOP structure, these values **may** provide additional guidance for the implementation's rate control algorithm.

The VkVideoEncodeH264CapabilitiesKHR::`prefersGopRemainingFrames` capability is also used to indicate that the implementation's rate control algorithm **may** operate more accurately if the application specifies the remaining frame counts using this structure.

As with other rate control guidance values, if the effective order and number of frames encoded by the application are not in line with the remaining frame counts specified in this structure at any given point, then the behavior of the implementation's rate control algorithm **may** deviate from the one expected by the application.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH264GopRemainingFrameInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_GOP_REMAINING_FRAME_INFO_KHR`

H.264 QP Delta Maps

Quantization delta maps used with an H.264 encode profile are referred to as *QP delta maps* and their texels contain integer values representing QP delta values that are applied in the process of determining the quantization parameters of the encoded picture.

Accordingly, H.264 QP delta maps always have single channel integer formats, as reported in VkVideoFormatPropertiesKHR::`format`.

When the rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`, the QP delta values are added to the per slice constant QP values that, in effect, enable the application to explicitly control the used QP values at the granularity of the used quantization map texel size.

For all other rate control modes, the QP delta values **can** be used to offset the QP values that the rate control algorithm would otherwise produce.

H.264 Encode Quantization

Performing H.264 encode operations involves the process of assigning QP values to individual H.264 macroblocks. This process depends on the used rate control mode, as well as other encode and rate control parameters, as described below:

- If the configured rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR`, then the QP value is initialized by the implementation-specific default rate control algorithm.
  - If the video encode operation is issued with a quantization delta map, the QP delta value corresponding to the macroblock, as fetched from the quantization map, is added to the previously determined QP value. If the fetched QP delta value falls outside the supported QP delta value range reported in the `minQpDelta` and `maxQpDelta` members of VkVideoEncodeH264QuantizationMapCapabilitiesKHR, then the QP value used for the macroblock becomes undefined.
- If the configured rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`, then the QP value is initialized from the constant QP value specified for the H.264 slice the macroblock is part of.
  - If the video encode operation is issued with a quantization delta map, the QP delta value corresponding to the macroblock, as fetched from the quantization map, is added to the previously determined QP value. If the fetched QP delta value falls outside the supported QP delta value range reported in the `minQpDelta` and `maxQpDelta` members of VkVideoEncodeH264QuantizationMapCapabilitiesKHR, then the QP value used for the macroblock becomes undefined.
- If the configured rate control mode is not `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR` or `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`, then the QP value is initialized by the corresponding rate control algorithm.
  - If the video encode operation is issued with a quantization delta map, the QP delta value corresponding to the macroblock, as fetched from the quantization map, is added to the previously determined QP value. If the fetched QP delta value falls outside the supported QP delta value range reported in the `minQpDelta` and `maxQpDelta` members of VkVideoEncodeH264QuantizationMapCapabilitiesKHR, then the QP value used for the

macroblock becomes undefined.

- If the video encode operation is issued with an emphasis map, the rate control will adjust the QP value based on the emphasis value corresponding to the macroblock, as fetched from the quantization map, according to the following equation:

  $QP_{new} = f(QP_{prev},e)$

  Where QP_{new} is the resulting QP value, QP_{prev} is the previously determined QP value, e is the emphasis value corresponding to the macroblock, and f is an implementation-defined function for which the following implication is true:

  $e_1 < e_2 \Rightarrow f(QP,e_1) \geq f(QP,e_2)$

  > **NOTE** This means that lower emphasis values will result in higher QP values, whereas higher emphasis values will result in lower QP values, but the function is not strictly decreasing with respect to the input emphasis value for a given input QP value.

- If clamping to minimum QP values is enabled in the applied rate control layer, then the QP value is clamped to the corresponding minimum QP value.
- If clamping to maximum QP values is enabled in the applied rate control layer, then the QP value is clamped to the corresponding maximum QP value.

- If `VK_VIDEO_ENCODE_H264_CAPABILITY_MB_QP_DIFF_WRAPAROUND_BIT_KHR` is not supported, then the determined QP value is clamped in such a way that the `mb_qp_delta` value of the encoded macroblock complies to the modified version of equation 7-37 of the ITU-T H.264 Specification.

  > **NOTE** The effect of this is that the maximum QP difference across subsequent macroblocks is limited to the $[-(26 + QpBdOffset_Y / 2), 25 + QpBdOffset_Y / 2]$ range and only has an observable change in behavior when the video encode operation is issued with a QP delta map.

- In all cases, the final QP value is clamped to the QP value range supported by the video profile, as reported in the `minQp` and `maxQp` members of VkVideoEncodeH264CapabilitiesKHR.

38.18.13. H.264 Encode Requirements

This section described the **required** H.264 encoding capabilities for physical devices that have at least one queue family that supports the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR`, as returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR in VkQueueFamilyVideoPropertiesKHR ::`videoCodecOperations`.

Table 53. Required Video Std Header Versions

Video Std Header Name	Version
`vulkan_video_codec_h264std_encode`	1.0.0

Table 54. Required Video Capabilities

Video Capability	Requirement	Requirement Type[1]
VkVideoCapabilitiesKHR		
flags	-	min
minBitstreamBufferOffsetAlignment	4096	max
minBitstreamBufferSizeAlignment	4096	max
pictureAccessGranularity	(64,64)	max
minCodedExtent	-	max
maxCodedExtent	-	min
maxDpbSlots	0	min
maxActiveReferencePictures	0	min
VkVideoEncodeCapabilitiesKHR		
flags	-	min
rateControlModes	-	min
maxBitrate	64000	min
maxQualityLevels	1	min
encodeInputPictureGranularity	(64,64)	max
supportedEncodeFeedbackFlags	VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BUFFER_OFFSET_BIT_KHR VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BYTES_WRITTEN_BIT_KHR	min
VkVideoEncodeH264CapabilitiesKHR		
flags	-	min
maxLevelIdc	STD_VIDEO_H264_LEVEL_IDC_1_0	min
maxSliceCount	1	min
maxPPictureL0ReferenceCount	0	min
maxBPictureL0ReferenceCount	0	min
maxL1ReferenceCount	0	min
maxTemporalLayerCount	1	min
expectDyadicTemporalLayerPattern	-	implementation-dependent
minQp	-	max
maxQp	-	min
prefersGopRemainingFrames	-	implementation-dependent

Video Capability	Requirement	Requirement Type[1]
`requiresGopRemainingFrames`	-	implementation-dependent
`stdSyntaxFlags`	-	min
VkVideoEncodeQuantizationMapCapabilitiesKHR		
`maxQuantizationMapExtent`	-[2]	min
VkVideoEncodeH264QuantizationMapCapabilitiesKHR		
`minQpDelta`	-[3]	max
`maxQpDelta`	-[3]	min

1 The **Requirement Type** column specifies the requirement is either the minimum value all implementations **must** support, the maximum value all implementations **must** support, or the exact value all implementations **must** support. For bitmasks a minimum value is the least bits all implementations **must** set, but they **may** have additional bits set beyond this minimum.

2 If VkVideoCapabilitiesKHR::`flags` includes `VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR` or `VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR`, then the `width` and `height` members of `maxQuantizationMapExtent` **must** be greater than zero.

3 If VkVideoCapabilitiesKHR::`flags` includes `VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR`, then `maxQpDelta` **must** be greater than `minQpDelta`.

38.19. H.265 Encode Operations

Video encode operations using an H.265 encode profile **can** be used to encode elementary video stream sequences compliant to the ITU-T H.265 Specification.

> **NOTE** Refer to the Preamble for information on how the Khronos Intellectual Property Rights Policy relates to normative references to external materials not created by Khronos.

This process is performed according to the video encode operation steps with the codec-specific semantics defined in section 8 of the ITU-T H.265 Specification as follows:

- Syntax elements, derived values, and other parameters are applied from the following structures:
  - The `StdVideoH265VideoParameterSet` structure corresponding to the active VPS specifying the

H.265 video parameter set.

  - The `StdVideoH265SequenceParameterSet` structure corresponding to the active SPS specifying the H.265 sequence parameter set.
  - The `StdVideoH265PictureParameterSet` structure corresponding to the active PPS specifying the H.265 picture parameter set.
  - The `StdVideoEncodeH265PictureInfo` structure specifying the H.265 picture information.
  - The `StdVideoEncodeH265SliceSegmentHeader` structures specifying the H.265 slice segment header parameters for each encoded H.265 slice segment.
  - The `StdVideoEncodeH265ReferenceInfo` structures specifying the H.265 reference information corresponding to the optional reconstructed picture and any active reference pictures.

- The encoded bitstream data is written to the destination video bitstream buffer range as defined in the H.265 Encode Bitstream Data Access section.
- Picture data in the video picture resources corresponding to the used encode input picture, active reference pictures, and optional reconstructed picture is accessed as defined in the H.265 Encode Picture Data Access section.
- The decision on reference picture setup is made according to the parameters specified in the H.265 picture information.

If the parameters adhere to the syntactic and semantic requirements defined in the corresponding sections of the ITU-T H.265 Specification, as described above, and the DPB slots associated with the active reference pictures all refer to valid picture references, then the video encode operation will complete successfully. Otherwise, the video encode operation **may** complete unsuccessfully.

38.19.1. H.265 Encode Parameter Overrides

Implementations **may** override, unless otherwise specified, any of the H.265 encode parameters specified in the following Video Std structures:

- `StdVideoH265VideoParameterSet`
- `StdVideoH265SequenceParameterSet`
- `StdVideoH265PictureParameterSet`
- `StdVideoEncodeH265PictureInfo`
- `StdVideoEncodeH265SliceSegmentHeader`
- `StdVideoEncodeH265ReferenceInfo`

All such H.265 encode parameter overrides **must** fulfill the conditions defined in the Video Encode Parameter Overrides section.

In addition, implementations **must** not override any of the following H.265 encode parameters:

- `StdVideoEncodeH265PictureInfo`::`pic_type`
- `StdVideoEncodeH265SliceSegmentHeader`::`slice_type`

In case of a video session parameters object created with

VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, the following H.265 SPS and PPS parameters **may** be overridden by the implementation according to the quantization map texel size the video session parameters object was created with:

- StdVideoH265SequenceParameterSet::log2_min_luma_coding_block_size_minus3
- StdVideoH265SequenceParameterSet::log2_diff_max_min_luma_coding_block_size
- StdVideoH265SequenceParameterSet::log2_min_pcm_luma_coding_block_size_minus3
- StdVideoH265SequenceParameterSet::log2_diff_max_min_pcm_luma_coding_block_size
- StdVideoH265PictureParameterSet::diff_cu_qp_delta_depth

This **may** be necessary in order to limit the set of H.265 coding unit and coding tree unit sizes used during picture encoding to those that are supported by the implementation when using the specific quantization map texel size.

In case of H.265 encode parameters stored in video session parameters objects, applications need to use the vkGetEncodedVideoSessionParametersKHR command to determine whether any implementation overrides happened. If the query indicates that implementation overrides were applied, then the application needs to retrieve and use the encoded H.265 parameter sets in the bitstream in order to be able to produce a compliant H.265 video bitstream using the H.265 encode parameters stored in the video session parameters object.

In case of any H.265 encode parameters stored in the encoded bitstream produced by video encode operations, if the implementation supports the VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR video encode feedback query flag, the application **can** use such queries to retrieve feedback about whether any implementation overrides have been applied to those H.265 encode parameters.

38.19.2. H.265 Encode Bitstream Data Access

Each video encode operation writes one or more VCL NAL units comprising of slice segment headers and data of the encoded picture, in the format defined in sections 7.3.6 and 7.3.8, according to the semantics defined in sections 7.4.7 and 7.4.9 of the ITU-T H.265 Specification, respectively. The number of VCL NAL units written is specified by VkVideoEncodeH265PictureInfoKHR::naluSliceSegmentEntryCount.

38.19.3. H.265 Encode Picture Data Access

Accesses to image data within a video picture resource happen at the granularity indicated by VkVideoCapabilitiesKHR::pictureAccessGranularity, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile. Accordingly, the complete image subregion of a encode input picture, reference picture, or reconstructed picture accessed by video coding operations using an H.265 encode profile is defined as the set of texels within the coordinate range:

([0,endX), [0,endY))

Where:

- endX equals codedExtent.width rounded up to the nearest integer multiple of pictureAccessGranularity.width and clamped to the width of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;
- endY equals codedExtent.height rounded up to the nearest integer multiple of pictureAccessGranularity.height and clamped to the height of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;

Where codedExtent is the member of the VkVideoPictureResourceInfoKHR structure corresponding to the picture.

In case of video encode operations using an H.265 encode profile, any access to a picture at the coordinates (x,y), as defined by the ITU-T H.265 Specification, is an access to the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure at the texel coordinates (x,y).

Implementations **may** choose not to access some or all texels within particular reference pictures available to a video encode operation (e.g. due to video encode parameter overrides restricting the effective set of used reference pictures, or if the encoding algorithm chooses not to use certain subregions of the reference picture data for sample prediction).

38.19.4. H.265 Frame, Picture, Slice Segments, and Tiles

H.265 pictures consist of one or more slices, slice segments, and tiles, as defined in section 6.3.1 of the ITU-T H.265 Specification.

For the purposes of this specification, the H.265 slice segments comprising a picture are referred to as the *picture partitions* of the picture.

Video encode operations using an H.265 encode profile **can** encode slice segments of different types, as defined in section 7.4.7.1 of the ITU-T H.265 Specification, by specifying the corresponding enumeration constant value in StdVideoEncodeH265SliceSegmentHeader::slice_type in the H.265 slice segment header parameters from the Video Std enumeration type StdVideoH265SliceType:

- STD_VIDEO_H265_SLICE_TYPE_B indicates that the slice segment is part of a *B slice* as defined in section 3.12 of the ITU-T H.265 Specification.
- STD_VIDEO_H265_SLICE_TYPE_P indicates that the slice segment is part of a *P slice* as defined in section 3.111 of the ITU-T H.265 Specification.
- STD_VIDEO_H265_SLICE_TYPE_I indicates that the slice segment is part of an *I slice* as defined in section 3.74 of the ITU-T H.265 Specification.

Pictures constructed from such slice segments **can** be of different types, as defined in section 7.4.3.5 of the ITU-T H.265 Specification. Video encode operations using an H.265 encode profile **can** encode pictures of a specific type by specifying the corresponding enumeration constant value in StdVideoEncodeH265PictureInfo::pic_type in the H.265 picture information from the Video Std enumeration type StdVideoH265PictureType:

- STD_VIDEO_H265_PICTURE_TYPE_P indicates that the picture is a *P picture*. A frame consisting of a P picture is also referred to as a *P frame*.

- STD_VIDEO_H265_PICTURE_TYPE_B indicates that the picture is a *B picture*. A frame consisting of a B picture is also referred to as a *B frame*.
- STD_VIDEO_H265_PICTURE_TYPE_I indicates that the picture is an *I picture*. A frame consisting of an I picture is also referred to as an *I frame*.
- STD_VIDEO_H265_PICTURE_TYPE_IDR indicates that the picture is a special type of I picture called an *IDR picture* as defined in section 3.67 of the ITU-T H.265 Specification. A frame consisting of an IDR picture is also referred to as an *IDR frame*.

38.19.5. H.265 Coding Blocks

H.265 encode supports two types of coding blocks:

- *Coding tree unit*, as defined in section 3.35 of the ITU-T H.265 Specification.
- *Coding unit*, as defined in section 3.36 of the ITU-T H.265 Specification.

38.19.6. H.265 Encode Profile

A video profile supporting H.265 video encode operations is specified by setting VkVideoProfileInfoKHR::videoCodecOperation to VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR and adding a VkVideoEncodeH265ProfileInfoKHR structure to the VkVideoProfileInfoKHR::pNext chain.

The VkVideoEncodeH265ProfileInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265ProfileInfoKHR {
    VkStructureType           sType;
    const void*               pNext;
    StdVideoH265ProfileIdc    stdProfileIdc;
} VkVideoEncodeH265ProfileInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- stdProfileIdc is a StdVideoH265ProfileIdc value specifying the H.265 codec profile IDC, as defined in section A.3 of the ITU-T H.265 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265ProfileInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_PROFILE_INFO_KHR

38.19.7. H.265 Encode Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities for an H.265 encode profile, the VkVideoCapabilitiesKHR::pNext chain **must** include a VkVideoEncodeH265CapabilitiesKHR structure that will be filled with the profile-specific capabilities.

The VkVideoEncodeH265CapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265CapabilitiesKHR {
    VkStructureType                                 sType;
    void*                                           pNext;
    VkVideoEncodeH265CapabilityFlagsKHR             flags;
    StdVideoH265LevelIdc                            maxLevelIdc;
    uint32_t                                        maxSliceSegmentCount;
    VkExtent2D                                      maxTiles;
    VkVideoEncodeH265CtbSizeFlagsKHR                ctbSizes;
    VkVideoEncodeH265TransformBlockSizeFlagsKHR     transformBlockSizes;
    uint32_t                                        maxPPictureL0ReferenceCount;
    uint32_t                                        maxBPictureL0ReferenceCount;
    uint32_t                                        maxL1ReferenceCount;
    uint32_t                                        maxSubLayerCount;
    VkBool32
expectDyadicTemporalSubLayerPattern;
    int32_t                                         minQp;
    int32_t                                         maxQp;
    VkBool32                                        prefersGopRemainingFrames;
    VkBool32                                        requiresGopRemainingFrames;
    VkVideoEncodeH265StdFlagsKHR                    stdSyntaxFlags;
} VkVideoEncodeH265CapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkVideoEncodeH265CapabilityFlagBitsKHR indicating supported H.265 encoding capabilities.
- maxLevelIdc is a StdVideoH265LevelIdc value indicating the maximum H.265 level supported by the profile, where enum constant STD_VIDEO_H265_LEVEL_IDC_<major>_<minor> identifies H.265 level <major>.<minor> as defined in section A.4 of the ITU-T H.265 Specification.
- maxSliceSegmentCount indicates the maximum number of slice segments that **can** be encoded for a single picture. Further restrictions **may** apply to the number of slice segments that **can** be encoded for a single picture depending on other capabilities and codec-specific rules.
- maxTiles indicates the maximum number of H.265 tile columns and rows, as defined in sections 3.175 and 3.176 of the ITU-T H.265 Specification that **can** be encoded for a single picture. Further restrictions **may** apply to the number of H.265 tiles that **can** be encoded for a single picture depending on other capabilities and codec-specific rules.
- ctbSizes is a bitmask of VkVideoEncodeH265CtbSizeFlagBitsKHR describing the supported CTB sizes.
- transformBlockSizes is a bitmask of VkVideoEncodeH265TransformBlockSizeFlagBitsKHR describing the supported transform block sizes.
- maxPPictureL0ReferenceCount indicates the maximum number of reference pictures the implementation supports in the reference list L0 for P pictures.

NOTE	As implementations **may** override the reference lists, `maxPPictureL0ReferenceCount` does not limit the number of elements that the application **can** specify in the L0 reference list for P pictures. However, if `maxPPictureL0ReferenceCount` is zero, then the use of P pictures is not allowed. In case of H.265 encoding, pictures **can** be encoded using only forward prediction even if P pictures are not supported, as the ITU-T H.265 Specification supports *generalized P & B frames* (also known as low delay B frames) whereas B frames **can** refer to past frames through both the L0 and L1 reference lists.

- `maxBPictureL0ReferenceCount` indicates the maximum number of reference pictures the implementation supports in the reference list L0 for B pictures.
- `maxL1ReferenceCount` indicates the maximum number of reference pictures the implementation supports in the reference list L1 if encoding of B pictures is supported.

NOTE	As implementations **may** override the reference lists, `maxBPictureL0ReferenceCount` and `maxL1ReferenceCount` does not limit the number of elements that the application **can** specify in the L0 and L1 reference lists for B pictures. However, if `maxBPictureL0ReferenceCount` and `maxL1ReferenceCount` are both zero, then the use of B pictures is not allowed.

- `maxSubLayerCount` indicates the maximum number of H.265 sub-layers supported by the implementation.
- `expectDyadicTemporalSubLayerPattern` indicates that the implementation's rate control algorithms expect the application to use a dyadic temporal sub-layer pattern when encoding multiple temporal sub-layers.
- `minQp` indicates the minimum QP value supported.
- `maxQp` indicates the maximum QP value supported.
- `prefersGopRemainingFrames` indicates that the implementation's rate control algorithm prefers the application to specify the number of frames of each type remaining in the current group of pictures when beginning a video coding scope.
- `requiresGopRemainingFrames` indicates that the implementation's rate control algorithm requires the application to specify the number of frames of each type remaining in the current group of pictures when beginning a video coding scope.
- `stdSyntaxFlags` is a bitmask of VkVideoEncodeH265StdFlagBitsKHR indicating capabilities related to H.265 syntax elements.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265CapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_CAPABILITIES_KHR`

Bits which **may** be set in VkVideoEncodeH265CapabilitiesKHR::`flags`, indicating the H.265 encoding capabilities supported, are:

```
// Provided by VK_KHR_video_encode_h265
typedef enum VkVideoEncodeH265CapabilityFlagBitsKHR {
    VK_VIDEO_ENCODE_H265_CAPABILITY_HRD_COMPLIANCE_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_H265_CAPABILITY_PREDICTION_WEIGHT_TABLE_GENERATED_BIT_KHR =
0x00000002,
    VK_VIDEO_ENCODE_H265_CAPABILITY_ROW_UNALIGNED_SLICE_SEGMENT_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_H265_CAPABILITY_DIFFERENT_SLICE_SEGMENT_TYPE_BIT_KHR = 0x00000008,
    VK_VIDEO_ENCODE_H265_CAPABILITY_B_FRAME_IN_L0_LIST_BIT_KHR = 0x00000010,
    VK_VIDEO_ENCODE_H265_CAPABILITY_B_FRAME_IN_L1_LIST_BIT_KHR = 0x00000020,
    VK_VIDEO_ENCODE_H265_CAPABILITY_PER_PICTURE_TYPE_MIN_MAX_QP_BIT_KHR = 0x00000040,
    VK_VIDEO_ENCODE_H265_CAPABILITY_PER_SLICE_SEGMENT_CONSTANT_QP_BIT_KHR =
0x00000080,
    VK_VIDEO_ENCODE_H265_CAPABILITY_MULTIPLE_TILES_PER_SLICE_SEGMENT_BIT_KHR =
0x00000100,
    VK_VIDEO_ENCODE_H265_CAPABILITY_MULTIPLE_SLICE_SEGMENTS_PER_TILE_BIT_KHR =
0x00000200,
  // Provided by VK_KHR_video_encode_h265 with VK_KHR_video_encode_quantization_map
    VK_VIDEO_ENCODE_H265_CAPABILITY_CU_QP_DIFF_WRAPAROUND_BIT_KHR = 0x00000400,
} VkVideoEncodeH265CapabilityFlagBitsKHR;
```

- VK_VIDEO_ENCODE_H265_CAPABILITY_HRD_COMPLIANCE_BIT_KHR specifies whether the implementation **may** be able to generate HRD compliant bitstreams if any of the nal_hrd_parameters_present_flag, vcl_hrd_parameters_present_flag, or sub_pic_hrd_params_present_flag members of StdVideoH265HrdFlags are set to 1 in the HRD parameters of the active VPS or active SPS, or if StdVideoH265SpsVuiFlags ::vui_hrd_parameters_present_flag is set to 1 in the active SPS.
- VK_VIDEO_ENCODE_H265_CAPABILITY_PREDICTION_WEIGHT_TABLE_GENERATED_BIT_KHR specifies that if the weighted_pred_flag or the weighted_bipred_flag member of StdVideoH265PpsFlags is set to 1 in the active PPS when encoding a P picture or B picture, respectively, then the implementation is able to internally decide syntax for pred_weight_table, as defined in section 7.4.7.3 of the ITU-T H.265 Specification, and the application is not **required** to provide a weight table in the H.265 slice segment header parameters.
- VK_VIDEO_ENCODE_H265_CAPABILITY_ROW_UNALIGNED_SLICE_SEGMENT_BIT_KHR specifies that each slice segment in a frame with a single or multiple tiles per slice may begin or finish at any offset in a CTB row. If not supported, all slice segments in such a frame **must** begin at the start of a CTB row (and hence each slice segment **must** finish at the end of a CTB row). Also indicates that each slice segment in a frame with multiple slices per tile may begin or finish at any offset within the enclosing tile's CTB row. If not supported, slice segments in such a frame **must** begin at the start of the enclosing tile's CTB row (and hence each slice segment **must** finish at the end of the enclosing tile's CTB row).
- VK_VIDEO_ENCODE_H265_CAPABILITY_DIFFERENT_SLICE_SEGMENT_TYPE_BIT_KHR specifies that when a frame is encoded with multiple slice segments, the implementation allows encoding each slice segment with a different StdVideoEncodeH265SliceSegmentHeader::slice_type specified in the H.265 slice segment header parameters. If not supported, all slice segments of the frame **must** be encoded with the same slice_type which corresponds to the picture type of the frame.
- VK_VIDEO_ENCODE_H265_CAPABILITY_B_FRAME_IN_L0_LIST_BIT_KHR specifies support for using a B

frame as L0 reference, as specified in StdVideoEncodeH265ReferenceListsInfo::RefPicList0 in the H.265 picture information.

- VK_VIDEO_ENCODE_H265_CAPABILITY_B_FRAME_IN_L1_LIST_BIT_KHR specifies support for using a B frame as L1 reference, as specified in StdVideoEncodeH265ReferenceListsInfo::RefPicList1 in the H.265 picture information.
- VK_VIDEO_ENCODE_H265_CAPABILITY_PER_PICTURE_TYPE_MIN_MAX_QP_BIT_KHR specifies support for specifying different QP values in the members of VkVideoEncodeH265QpKHR.
- VK_VIDEO_ENCODE_H265_CAPABILITY_PER_SLICE_SEGMENT_CONSTANT_QP_BIT_KHR specifies support for specifying different constant QP values for each slice segment.
- VK_VIDEO_ENCODE_H265_CAPABILITY_MULTIPLE_TILES_PER_SLICE_SEGMENT_BIT_KHR specifies whether encoding multiple tiles per slice segment, as defined in section 6.3.1 of the ITU-T H.265 Specification, is supported. If this capability flag is not present, then the implementation is only able to encode a single tile for each slice segment.
- VK_VIDEO_ENCODE_H265_CAPABILITY_MULTIPLE_SLICE_SEGMENTS_PER_TILE_BIT_KHR specifies whether encoding multiple slice segments per tile, as defined in section 6.3.1 of the ITU-T H.265 Specification, is supported. If this capability flag is not present, then the implementation is only able to encode a single slice segment for each tile.
- VK_VIDEO_ENCODE_H265_CAPABILITY_CU_QP_DIFF_WRAPAROUND_BIT_KHR indicates support for wraparound during the calculation of the QP values of subsequently encoded coding units, as defined in section 7.4.9.14 of the ITU-T H.265 Specification. If not supported, equation 8-283 of the ITU-T H.265 Specification is effectively reduced to the following:

  $Qp_Y = qP_{Y_PRED} + \texttt{CuQpDeltaVal}$

  NOTE The effect of this is that the maximum QP difference across subsequent coding units is limited to the $[-(26 + QpBdOffset_Y / 2), 25 + QpBdOffset_Y / 2]$ range.

```
// Provided by VK_KHR_video_encode_h265
typedef VkFlags VkVideoEncodeH265CapabilityFlagsKHR;
```

VkVideoEncodeH265CapabilityFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeH265CapabilityFlagBitsKHR.

Bits which **may** be set in VkVideoEncodeH265CapabilitiesKHR::stdSyntaxFlags, indicating the capabilities related to the H.265 syntax elements, are:

```
// Provided by VK_KHR_video_encode_h265
typedef enum VkVideoEncodeH265StdFlagBitsKHR {
    VK_VIDEO_ENCODE_H265_STD_SEPARATE_COLOR_PLANE_FLAG_SET_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_H265_STD_SAMPLE_ADAPTIVE_OFFSET_ENABLED_FLAG_SET_BIT_KHR =
0x00000002,
    VK_VIDEO_ENCODE_H265_STD_SCALING_LIST_DATA_PRESENT_FLAG_SET_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_H265_STD_PCM_ENABLED_FLAG_SET_BIT_KHR = 0x00000008,
    VK_VIDEO_ENCODE_H265_STD_SPS_TEMPORAL_MVP_ENABLED_FLAG_SET_BIT_KHR = 0x00000010,
    VK_VIDEO_ENCODE_H265_STD_INIT_QP_MINUS26_BIT_KHR = 0x00000020,
    VK_VIDEO_ENCODE_H265_STD_WEIGHTED_PRED_FLAG_SET_BIT_KHR = 0x00000040,
    VK_VIDEO_ENCODE_H265_STD_WEIGHTED_BIPRED_FLAG_SET_BIT_KHR = 0x00000080,
    VK_VIDEO_ENCODE_H265_STD_LOG2_PARALLEL_MERGE_LEVEL_MINUS2_BIT_KHR = 0x00000100,
    VK_VIDEO_ENCODE_H265_STD_SIGN_DATA_HIDING_ENABLED_FLAG_SET_BIT_KHR = 0x00000200,
    VK_VIDEO_ENCODE_H265_STD_TRANSFORM_SKIP_ENABLED_FLAG_SET_BIT_KHR = 0x00000400,
    VK_VIDEO_ENCODE_H265_STD_TRANSFORM_SKIP_ENABLED_FLAG_UNSET_BIT_KHR = 0x00000800,
    VK_VIDEO_ENCODE_H265_STD_PPS_SLICE_CHROMA_QP_OFFSETS_PRESENT_FLAG_SET_BIT_KHR =
0x00001000,
    VK_VIDEO_ENCODE_H265_STD_TRANSQUANT_BYPASS_ENABLED_FLAG_SET_BIT_KHR = 0x00002000,
    VK_VIDEO_ENCODE_H265_STD_CONSTRAINED_INTRA_PRED_FLAG_SET_BIT_KHR = 0x00004000,
    VK_VIDEO_ENCODE_H265_STD_ENTROPY_CODING_SYNC_ENABLED_FLAG_SET_BIT_KHR =
0x00008000,
    VK_VIDEO_ENCODE_H265_STD_DEBLOCKING_FILTER_OVERRIDE_ENABLED_FLAG_SET_BIT_KHR =
0x00010000,
    VK_VIDEO_ENCODE_H265_STD_DEPENDENT_SLICE_SEGMENTS_ENABLED_FLAG_SET_BIT_KHR =
0x00020000,
    VK_VIDEO_ENCODE_H265_STD_DEPENDENT_SLICE_SEGMENT_FLAG_SET_BIT_KHR = 0x00040000,
    VK_VIDEO_ENCODE_H265_STD_SLICE_QP_DELTA_BIT_KHR = 0x00080000,
    VK_VIDEO_ENCODE_H265_STD_DIFFERENT_SLICE_QP_DELTA_BIT_KHR = 0x00100000,
} VkVideoEncodeH265StdFlagBitsKHR;
```

- VK_VIDEO_ENCODE_H265_STD_SEPARATE_COLOR_PLANE_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265SpsFlags ::separate_colour_plane_flag in the SPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_SAMPLE_ADAPTIVE_OFFSET_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265SpsFlags ::sample_adaptive_offset_enabled_flag in the SPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_SCALING_LIST_DATA_PRESENT_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for the scaling_list_enabled_flag and sps_scaling_list_data_present_flag members of StdVideoH265SpsFlags in the SPS, and the application-provided value for StdVideoH265PpsFlags ::pps_scaling_list_data_present_flag in the PPS when those values are 1.
- VK_VIDEO_ENCODE_H265_STD_PCM_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265SpsFlags::pcm_enable_flag in the SPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_SPS_TEMPORAL_MVP_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265SpsFlags ::sps_temporal_mvp_enabled_flag in the SPS when that value is 1.

- VK_VIDEO_ENCODE_H265_STD_INIT_QP_MINUS26_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PictureParameterSet ::init_qp_minus26 in the PPS when that value is non-zero.
- VK_VIDEO_ENCODE_H265_STD_WEIGHTED_PRED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::weighted_pred_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_WEIGHTED_BIPRED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::weighted_bipred_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_LOG2_PARALLEL_MERGE_LEVEL_MINUS2_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PictureParameterSet::log2_parallel_merge_level_minus2 in the PPS when that value is non-zero.
- VK_VIDEO_ENCODE_H265_STD_SIGN_DATA_HIDING_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::sign_data_hiding_enabled_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_TRANSFORM_SKIP_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::transform_skip_enabled_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_TRANSFORM_SKIP_ENABLED_FLAG_UNSET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::transform_skip_enabled_flag in the PPS when that value is 0.
- VK_VIDEO_ENCODE_H265_STD_PPS_SLICE_CHROMA_QP_OFFSETS_PRESENT_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags::pps_slice_chroma_qp_offsets_present_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_TRANSQUANT_BYPASS_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::transquant_bypass_enabled_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_CONSTRAINED_INTRA_PRED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::constrained_intra_pred_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_ENTROPY_CODING_SYNC_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::entropy_coding_sync_enabled_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_DEBLOCKING_FILTER_OVERRIDE_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags::deblocking_filter_override_enabled_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_DEPENDENT_SLICE_SEGMENTS_ENABLED_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoH265PpsFlags ::dependent_slice_segments_enabled_flag in the PPS when that value is 1.
- VK_VIDEO_ENCODE_H265_STD_DEPENDENT_SLICE_SEGMENT_FLAG_SET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeH265SliceSegmentHeader::dependent_slice_segment_flag in the H.265 slice segment

header parameters when that value is 1.

- VK_VIDEO_ENCODE_H265_STD_SLICE_QP_DELTA_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeH265SliceSegmentHeader::slice_qp_delta in the H.265 slice segment header parameters when that value is identical across the slice segments of the encoded frame.
- VK_VIDEO_ENCODE_H265_STD_DIFFERENT_SLICE_QP_DELTA_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeH265SliceSegmentHeader::slice_qp_delta in the H.265 slice segment header parameters when that value is different across the slice segments of the encoded frame.

These capability flags provide information to the application about specific H.265 syntax element values that the implementation supports without having to override them and do not otherwise restrict the values that the application **can** specify for any of the mentioned H.265 syntax elements.

```
// Provided by VK_KHR_video_encode_h265
typedef VkFlags VkVideoEncodeH265StdFlagsKHR;
```

VkVideoEncodeH265StdFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeH265StdFlagBitsKHR.

Bits which **may** be set in VkVideoEncodeH265CapabilitiesKHR::ctbSizes, indicating the CTB sizes supported by the implementation, are:

```
// Provided by VK_KHR_video_encode_h265
typedef enum VkVideoEncodeH265CtbSizeFlagBitsKHR {
    VK_VIDEO_ENCODE_H265_CTB_SIZE_16_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_H265_CTB_SIZE_32_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_H265_CTB_SIZE_64_BIT_KHR = 0x00000004,
} VkVideoEncodeH265CtbSizeFlagBitsKHR;
```

- VK_VIDEO_ENCODE_H265_CTB_SIZE_16_BIT_KHR specifies that a CTB size of 16x16 is supported.
- VK_VIDEO_ENCODE_H265_CTB_SIZE_32_BIT_KHR specifies that a CTB size of 32x32 is supported.
- VK_VIDEO_ENCODE_H265_CTB_SIZE_64_BIT_KHR specifies that a CTB size of 64x64 is supported.

```
// Provided by VK_KHR_video_encode_h265
typedef VkFlags VkVideoEncodeH265CtbSizeFlagsKHR;
```

VkVideoEncodeH265CtbSizeFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeH265CtbSizeFlagBitsKHR.

Implementations **must** support at least one of VkVideoEncodeH265CtbSizeFlagBitsKHR.

Bits which **may** be set in VkVideoEncodeH265CapabilitiesKHR::transformBlockSizes, indicating the transform block sizes supported by the implementation, are:

```
// Provided by VK_KHR_video_encode_h265
typedef enum VkVideoEncodeH265TransformBlockSizeFlagBitsKHR {
    VK_VIDEO_ENCODE_H265_TRANSFORM_BLOCK_SIZE_4_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_H265_TRANSFORM_BLOCK_SIZE_8_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_H265_TRANSFORM_BLOCK_SIZE_16_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_H265_TRANSFORM_BLOCK_SIZE_32_BIT_KHR = 0x00000008,
} VkVideoEncodeH265TransformBlockSizeFlagBitsKHR;
```

- VK_VIDEO_ENCODE_H265_TRANSFORM_BLOCK_SIZE_4_BIT_KHR specifies that a transform block size of 4x4 is supported.
- VK_VIDEO_ENCODE_H265_TRANSFORM_BLOCK_SIZE_8_BIT_KHR specifies that a transform block size of 8x8 is supported.
- VK_VIDEO_ENCODE_H265_TRANSFORM_BLOCK_SIZE_16_BIT_KHR specifies that a transform block size of 16x16 is supported.
- VK_VIDEO_ENCODE_H265_TRANSFORM_BLOCK_SIZE_32_BIT_KHR specifies that a transform block size of 32x32 is supported.

```
// Provided by VK_KHR_video_encode_h265
typedef VkFlags VkVideoEncodeH265TransformBlockSizeFlagsKHR;
```

VkVideoEncodeH265TransformBlockSizeFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeH265TransformBlockSizeFlagBitsKHR.

Implementations **must** support at least one of VkVideoEncodeH265TransformBlockSizeFlagBitsKHR.

38.19.8. H.265 Encode Quality Level Properties

When calling vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR with pVideoProfile->videoCodecOperation specified as VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, the VkVideoEncodeH265QualityLevelPropertiesKHR structure **must** be included in the pNext chain of the VkVideoEncodeQualityLevelPropertiesKHR structure to retrieve additional video encode quality level properties specific to H.265 encoding.

The VkVideoEncodeH265QualityLevelPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265QualityLevelPropertiesKHR {
    VkStructureType                          sType;
    void*                                    pNext;
    VkVideoEncodeH265RateControlFlagsKHR     preferredRateControlFlags;
    uint32_t                                 preferredGopFrameCount;
    uint32_t                                 preferredIdrPeriod;
    uint32_t                                 preferredConsecutiveBFrameCount;
    uint32_t                                 preferredSubLayerCount;
    VkVideoEncodeH265QpKHR                   preferredConstantQp;
    uint32_t                                 preferredMaxL0ReferenceCount;
    uint32_t                                 preferredMaxL1ReferenceCount;
} VkVideoEncodeH265QualityLevelPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `preferredRateControlFlags` is a bitmask of VkVideoEncodeH265RateControlFlagBitsKHR values indicating the preferred flags to use for VkVideoEncodeH265RateControlInfoKHR::`flags`.
- `preferredGopFrameCount` indicates the preferred value to use for VkVideoEncodeH265RateControlInfoKHR::`gopFrameCount`.
- `preferredIdrPeriod` indicates the preferred value to use for VkVideoEncodeH265RateControlInfoKHR::`idrPeriod`.
- `preferredConsecutiveBFrameCount` indicates the preferred value to use for VkVideoEncodeH265RateControlInfoKHR::`consecutiveBFrameCount`.
- `preferredSubLayerCount` indicates the preferred value to use for VkVideoEncodeH265RateControlInfoKHR::`subLayerCount`.
- `preferredConstantQp` indicates the preferred values to use for VkVideoEncodeH265NaluSliceSegmentInfoKHR::`constantQp` for each picture type when using rate control mode `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`.
- `preferredMaxL0ReferenceCount` indicates the preferred maximum number of reference pictures to use in the reference list L0.
- `preferredMaxL1ReferenceCount` indicates the preferred maximum number of reference pictures to use in the reference list L1.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265QualityLevelPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_QUALITY_LEVEL_PROPERTIES_KHR`

38.19.9. H.265 Encode Session

Additional parameters can be specified when creating a video session with an H.265 encode profile by including an instance of the VkVideoEncodeH265SessionCreateInfoKHR structure in the `pNext`

chain of VkVideoSessionCreateInfoKHR.

The VkVideoEncodeH265SessionCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265SessionCreateInfoKHR {
    VkStructureType         sType;
    const void*             pNext;
    VkBool32                useMaxLevelIdc;
    StdVideoH265LevelIdc    maxLevelIdc;
} VkVideoEncodeH265SessionCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- useMaxLevelIdc indicates whether the value of maxLevelIdc should be used by the implementation. When it is VK_FALSE, the implementation ignores the value of maxLevelIdc and uses the value of VkVideoEncodeH265CapabilitiesKHR::maxLevelIdc, as reported by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile.
- maxLevelIdc is a StdVideoH265LevelIdc value specifying the upper bound on the H.265 level for the video bitstreams produced by the created video session, where enum constant STD_VIDEO_H265_LEVEL_IDC_<major>_<minor> identifies H.265 level <major>.<minor> as defined in section A.4 of the ITU-T H.265 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265SessionCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_CREATE_INFO_KHR

38.19.10. H.265 Encode Parameter Sets

Video session parameters objects created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR **can** contain the following types of parameters:

H.265 Video Parameter Sets (VPS)

Represented by StdVideoH265VideoParameterSet structures and interpreted as follows:

- reserved1, reserved2, and reserved3 are used only for padding purposes and are otherwise ignored;
- vps_video_parameter_set_id is used as the key of the VPS entry;
- the max_latency_increase_plus1, max_dec_pic_buffering_minus1, and max_num_reorder_pics members of the StdVideoH265DecPicBufMgr structure pointed to by pDecPicBufMgr correspond to vps_max_latency_increase_plus1, vps_max_dec_pic_buffering_minus1, and vps_max_num_reorder_pics, respectively, as defined in section 7.4.3.1 of the ITU-T H.265

Specification;

- the `StdVideoH265HrdParameters` structure pointed to by `pHrdParameters` is interpreted as follows:
  - `reserved` is used only for padding purposes and is otherwise ignored;
  - `flags.fixed_pic_rate_general_flag` is a bitmask where bit index i corresponds to `fixed_pic_rate_general_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
  - `flags.fixed_pic_rate_within_cvs_flag` is a bitmask where bit index i corresponds to `fixed_pic_rate_within_cvs_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
  - `flags.low_delay_hrd_flag` is a bitmask where bit index i corresponds to `low_delay_hrd_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
  - if `flags.nal_hrd_parameters_present_flag` is set, then `pSubLayerHrdParametersNal` is a pointer to an array of `vps_max_sub_layers_minus1 + 1` number of `StdVideoH265SubLayerHrdParameters` structures where `vps_max_sub_layers_minus1` is the corresponding member of the encompassing `StdVideoH265VideoParameterSet` structure and each element is interpreted as follows:
    - `cbr_flag` is a bitmask where bit index i corresponds to `cbr_flag[i]` as defined in section E.3.3 of the ITU-T H.265 Specification;
    - all other members of the `StdVideoH265SubLayerHrdParameters` structure are interpreted as defined in section E.3.3 of the ITU-T H.265 Specification;
  - if `flags.vcl_hrd_parameters_present_flag` is set, then `pSubLayerHrdParametersVcl` is a pointer to an array of `vps_max_sub_layers_minus1 + 1` number of `StdVideoH265SubLayerHrdParameters` structures where `vps_max_sub_layers_minus1` is the corresponding member of the encompassing `StdVideoH265VideoParameterSet` structure and each element is interpreted as follows:
    - `cbr_flag` is a bitmask where bit index i corresponds to `cbr_flag[i]` as defined in section E.3.3 of the ITU-T H.265 Specification;
    - all other members of the `StdVideoH265SubLayerHrdParameters` structure are interpreted as defined in section E.3.3 of the ITU-T H.265 Specification;
  - all other members of `StdVideoH265HrdParameters` are interpreted as defined in section E.3.2 of the ITU-T H.265 Specification;
- the `StdVideoH265ProfileTierLevel` structure pointed to by `pProfileTierLevel` are interpreted as follows:
  - `general_level_idc` is one of the enum constants `STD_VIDEO_H265_LEVEL_IDC_<major>_<minor>` identifying the H.265 level `<major>.<minor>` as defined in section A.4 of the ITU-T H.265 Specification;
  - all other members of `StdVideoH265ProfileTierLevel` are interpreted as defined in section 7.4.4 of the ITU-T H.265 Specification;
- all other members of `StdVideoH265VideoParameterSet` are interpreted as defined in section 7.4.3.1 of the ITU-T H.265 Specification.

H.265 Sequence Parameter Sets (SPS)

Represented by `StdVideoH265SequenceParameterSet` structures and interpreted as follows:

- `reserved1` and `reserved2` are used only for padding purposes and are otherwise ignored;
- the pair constructed from `sps_video_parameter_set_id` and `sps_seq_parameter_set_id` is used as the key of the SPS entry;
- the `StdVideoH265ProfileTierLevel` structure pointed to by `pProfileTierLevel` are interpreted as follows:
  - `general_level_idc` is one of the enum constants `STD_VIDEO_H265_LEVEL_IDC_<major>_<minor>` identifying the H.265 level `<major>.<minor>` as defined in section A.4 of the ITU-T H.265 Specification;
  - all other members of `StdVideoH265ProfileTierLevel` are interpreted as defined in section 7.4.4 of the ITU-T H.265 Specification;
- the `max_latency_increase_plus1`, `max_dec_pic_buffering_minus1`, and `max_num_reorder_pics` members of the `StdVideoH265DecPicBufMgr` structure pointed to by `pDecPicBufMgr` correspond to `sps_max_latency_increase_plus1`, `sps_max_dec_pic_buffering_minus1`, and `sps_max_num_reorder_pics`, respectively, as defined in section 7.4.3.2 of the ITU-T H.265 Specification;
- if `flags.sps_scaling_list_data_present_flag` is set, then the `StdVideoH265ScalingLists` structure pointed to by `pScalingLists` is interpreted as follows:
  - `ScalingList4x4`, `ScalingList8x8`, `ScalingList16x16`, and `ScalingList32x32` correspond to `ScalingList[0]`, `ScalingList[1]`, `ScalingList[2]`, and `ScalingList[3]`, respectively, as defined in section 7.3.4 of the ITU-T H.265 Specification;
  - `ScalingListDCCoef16x16` and `ScalingListDCCoef32x32` correspond to `scaling_list_dc_coef_minus8[0]` and `scaling_list_dc_coef_minus8[1]`, respectively, as defined in section 7.3.4 of the ITU-T H.265 Specification;
- `pShortTermRefPicSet` is a pointer to an array of `num_short_term_ref_pic_sets` number of `StdVideoH265ShortTermRefPicSet` structures where each element is interpreted as follows:
  - `reserved1`, `reserved2`, and `reserved3` are used only for padding purposes and are otherwise ignored;
  - `used_by_curr_pic_flag` is a bitmask where bit index i corresponds to `used_by_curr_pic_flag[i]` as defined in section 7.4.8 of the ITU-T H.265 Specification;
  - `use_delta_flag` is a bitmask where bit index i corresponds to `use_delta_flag[i]` as defined in section 7.4.8 of the ITU-T H.265 Specification;
  - `used_by_curr_pic_s0_flag` is a bitmask where bit index i corresponds to `used_by_curr_pic_s0_flag[i]` as defined in section 7.4.8 of the ITU-T H.265 Specification;
  - `used_by_curr_pic_s1_flag` is a bitmask where bit index i corresponds to `used_by_curr_pic_s1_flag[i]` as defined in section 7.4.8 of the ITU-T H.265 Specification;
  - all other members of `StdVideoH265ShortTermRefPicSet` are interpreted as defined in section 7.4.8 of the ITU-T H.265 Specification;

- if `flags.long_term_ref_pics_present_flag` is set then the `StdVideoH265LongTermRefPicsSps` structure pointed to by `pLongTermRefPicsSps` is interpreted as follows:
  - `used_by_curr_pic_lt_sps_flag` is a bitmask where bit index i corresponds to `used_by_curr_pic_lt_sps_flag[i]` as defined in section 7.4.3.2 of the ITU-T H.265 Specification;
  - all other members of `StdVideoH265LongTermRefPicsSps` are interpreted as defined in section 7.4.3.2 of the ITU-T H.265 Specification;
- if `flags.vui_parameters_present_flag` is set, then the `StdVideoH265SequenceParameterSetVui` structure pointed to by `pSequenceParameterSetVui` is interpreted as follows:
  - `reserved1`, `reserved2`, and `reserved3` are used only for padding purposes and are otherwise ignored;
  - the `StdVideoH265HrdParameters` structure pointed to by `pHrdParameters` is interpreted as follows:
    - `flags.fixed_pic_rate_general_flag` is a bitmask where bit index i corresponds to `fixed_pic_rate_general_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
    - `flags.fixed_pic_rate_within_cvs_flag` is a bitmask where bit index i corresponds to `fixed_pic_rate_within_cvs_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
    - `flags.low_delay_hrd_flag` is a bitmask where bit index i corresponds to `low_delay_hrd_flag[i]` as defined in section E.3.2 of the ITU-T H.265 Specification;
    - if `flags.nal_hrd_parameters_present_flag` is set, then `pSubLayerHrdParametersNal` is a pointer to an array of `sps_max_sub_layers_minus1` + 1 number of `StdVideoH265SubLayerHrdParameters` structures where `sps_max_sub_layers_minus1` is the corresponding member of the encompassing `StdVideoH265SequenceParameterSet` structure and each element is interpreted as follows:
      - `cbr_flag` is a bitmask where bit index i corresponds to `cbr_flag[i]` as defined in section E.3.3 of the ITU-T H.265 Specification;
      - all other members of the `StdVideoH265SubLayerHrdParameters` structure are interpreted as defined in section E.3.3 of the ITU-T H.265 Specification;
    - if `flags.vcl_hrd_parameters_present_flag` is set, then `pSubLayerHrdParametersVcl` is a pointer to an array of `sps_max_sub_layers_minus1` + 1 number of `StdVideoH265SubLayerHrdParameters` structures where `sps_max_sub_layers_minus1` is the corresponding member of the encompassing `StdVideoH265SequenceParameterSet` structure and each element is interpreted as follows:
      - `cbr_flag` is a bitmask where bit index i corresponds to `cbr_flag[i]` as defined in section E.3.3 of the ITU-T H.265 Specification;
      - all other members of the `StdVideoH265SubLayerHrdParameters` structure are interpreted as defined in section E.3.3 of the ITU-T H.265 Specification;
    - all other members of `StdVideoH265HrdParameters` are interpreted as defined in section E.3.2 of the ITU-T H.265 Specification;

    - all other members of `pSequenceParameterSetVui` are interpreted as defined in section E.3.1 of the ITU-T H.265 Specification;
- if `flags.sps_palette_predictor_initializer_present_flag` is set, then the `PredictorPaletteEntries` member of the `StdVideoH265PredictorPaletteEntries` structure pointed to by `pPredictorPaletteEntries` is interpreted as defined in section 7.4.9.13 of the ITU-T H.265 Specification;
- all other members of `StdVideoH265SequenceParameterSet` are interpreted as defined in section 7.4.3.1 of the ITU-T H.265 Specification.

H.265 Picture Parameter Sets (PPS)

Represented by `StdVideoH265PictureParameterSet` structures and interpreted as follows:

- `reserved1`, `reserved2`, and `reserved3` are used only for padding purposes and are otherwise ignored;
- the triplet constructed from `sps_video_parameter_set_id`, `pps_seq_parameter_set_id`, and `pps_pic_parameter_set_id` is used as the key of the PPS entry;
- if `flags.pps_scaling_list_data_present_flag` is set, then the `StdVideoH265ScalingLists` structure pointed to by `pScalingLists` is interpreted as follows:
    - `ScalingList4x4`, `ScalingList8x8`, `ScalingList16x16`, and `ScalingList32x32` correspond to `ScalingList[0]`, `ScalingList[1]`, `ScalingList[2]`, and `ScalingList[3]`, respectively, as defined in section 7.3.4 of the ITU-T H.265 Specification;
    - `ScalingListDCCoef16x16` and `ScalingListDCCoef32x32` correspond to `scaling_list_dc_coef_minus8[0]` and `scaling_list_dc_coef_minus8[1]`, respectively, as defined in section 7.3.4 of the ITU-T H.265 Specification;
- if `flags.pps_palette_predictor_initializer_present_flag` is set, then the `PredictorPaletteEntries` member of the `StdVideoH265PredictorPaletteEntries` structure pointed to by `pPredictorPaletteEntries` is interpreted as defined in section 7.4.9.13 of the ITU-T H.265 Specification;
- all other members of `StdVideoH265PictureParameterSet` are interpreted as defined in section 7.4.3.3 of the ITU-T H.265 Specification.

Implementations **may** override any of these parameters according to the semantics defined in the Video Encode Parameter Overrides section before storing the resulting H.265 parameter sets into the video session parameters object. Applications need to use the vkGetEncodedVideoSessionParametersKHR command to determine whether any implementation overrides happened and to retrieve the encoded H.265 parameter sets in order to be able to produce a compliant H.265 video bitstream.

Such H.265 parameter set overrides **may** also have cascading effects on the implementation overrides applied to the encoded bitstream produced by video encode operations. If the implementation supports the `VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR` video encode feedback query flag, then the application **can** use such queries to retrieve feedback about whether any implementation overrides have been applied to the encoded bitstream.

When a video session parameters object is created with the codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, the VkVideoSessionParametersCreateInfoKHR::pNext chain **must** include a VkVideoEncodeH265SessionParametersCreateInfoKHR structure specifying the capacity and initial contents of the object.

The VkVideoEncodeH265SessionParametersCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265SessionParametersCreateInfoKHR {
    VkStructureType                                       sType;
    const void*                                           pNext;
    uint32_t                                              maxStdVPSCount;
    uint32_t                                              maxStdSPSCount;
    uint32_t                                              maxStdPPSCount;
    const VkVideoEncodeH265SessionParametersAddInfoKHR*   pParametersAddInfo;
} VkVideoEncodeH265SessionParametersCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxStdVPSCount is the maximum number of H.265 VPS entries the created VkVideoSessionParametersKHR **can** contain.
- maxStdSPSCount is the maximum number of H.265 SPS entries the created VkVideoSessionParametersKHR **can** contain.
- maxStdPPSCount is the maximum number of H.265 PPS entries the created VkVideoSessionParametersKHR **can** contain.
- pParametersAddInfo is NULL or a pointer to a VkVideoEncodeH265SessionParametersAddInfoKHR structure specifying H.265 parameters to add upon object creation.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265SessionParametersCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_CREATE_INFO_KHR
- VUID-VkVideoEncodeH265SessionParametersCreateInfoKHR-pParametersAddInfo-parameter
  If pParametersAddInfo is not NULL, pParametersAddInfo **must** be a valid pointer to a valid VkVideoEncodeH265SessionParametersAddInfoKHR structure

The VkVideoEncodeH265SessionParametersAddInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265SessionParametersAddInfoKHR {
    VkStructureType                              sType;
    const void*                                  pNext;
    uint32_t                                     stdVPSCount;
    const StdVideoH265VideoParameterSet*         pStdVPSs;
    uint32_t                                     stdSPSCount;
    const StdVideoH265SequenceParameterSet*      pStdSPSs;
    uint32_t                                     stdPPSCount;
    const StdVideoH265PictureParameterSet*       pStdPPSs;
} VkVideoEncodeH265SessionParametersAddInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `stdVPSCount` is the number of elements in the `pStdVPSs` array.
- `pStdVPSs` is a pointer to an array of `StdVideoH265VideoParameterSet` structures describing the H.265 VPS entries to add.
- `stdSPSCount` is the number of elements in the `pStdSPSs` array.
- `pStdSPSs` is a pointer to an array of `StdVideoH265SequenceParameterSet` structures describing the H.265 SPS entries to add.
- `stdPPSCount` is the number of elements in the `pStdPPSs` array.
- `pStdPPSs` is a pointer to an array of `StdVideoH265PictureParameterSet` structures describing the H.265 PPS entries to add.

This structure **can** be specified in the following places:

- In the `pParametersAddInfo` member of the VkVideoEncodeH265SessionParametersCreateInfoKHR structure specified in the `pNext` chain of VkVideoSessionParametersCreateInfoKHR used to create a video session parameters object. In this case, if the video codec operation the video session parameters object is created with is `VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR`, then it defines the set of initial parameters to add to the created object (see Creating Video Session Parameters).
- In the `pNext` chain of VkVideoSessionParametersUpdateInfoKHR. In this case, if the video codec operation the video session parameters object to be updated was created with is `VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR`, then it defines the set of parameters to add to it (see Updating Video Session Parameters).

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265SessionParametersAddInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_ADD_INFO_KHR`
- VUID-VkVideoEncodeH265SessionParametersAddInfoKHR-pStdVPSs-parameter
  If `stdVPSCount` is not `0`, and `pStdVPSs` is not `NULL`, `pStdVPSs` **must** be a valid pointer to an array of `stdVPSCount` `StdVideoH265VideoParameterSet` values

- VUID-VkVideoEncodeH265SessionParametersAddInfoKHR-pStdSPSs-parameter
  If `stdSPSCount` is not `0`, and `pStdSPSs` is not `NULL`, `pStdSPSs` **must** be a valid pointer to an array of `stdSPSCount` `StdVideoH265SequenceParameterSet` values
- VUID-VkVideoEncodeH265SessionParametersAddInfoKHR-pStdPPSs-parameter
  If `stdPPSCount` is not `0`, and `pStdPPSs` is not `NULL`, `pStdPPSs` **must** be a valid pointer to an array of `stdPPSCount` `StdVideoH265PictureParameterSet` values

Valid Usage

- VUID-VkVideoEncodeH265SessionParametersAddInfoKHR-None-06438
  The `vps_video_parameter_set_id` member of each `StdVideoH265VideoParameterSet` structure specified in the elements of `pStdVPSs` **must** be unique within `pStdVPSs`
- VUID-VkVideoEncodeH265SessionParametersAddInfoKHR-None-06439
  The pair constructed from the `sps_video_parameter_set_id` and `sps_seq_parameter_set_id` members of each `StdVideoH265SequenceParameterSet` structure specified in the elements of `pStdSPSs` **must** be unique within `pStdSPSs`
- VUID-VkVideoEncodeH265SessionParametersAddInfoKHR-None-06440
  The triplet constructed from the `sps_video_parameter_set_id`, `pps_seq_parameter_set_id`, and `pps_pic_parameter_set_id` members of each `StdVideoH265PictureParameterSet` structure specified in the elements of `pStdPPSs` **must** be unique within `pStdPPSs`

The `VkVideoEncodeH265SessionParametersGetInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265SessionParametersGetInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           writeStdVPS;
    VkBool32           writeStdSPS;
    VkBool32           writeStdPPS;
    uint32_t           stdVPSId;
    uint32_t           stdSPSId;
    uint32_t           stdPPSId;
} VkVideoEncodeH265SessionParametersGetInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `writeStdVPS` indicates whether the encoded H.265 video parameter set identified by `stdVPSId` is requested to be retrieved.
- `writeStdSPS` indicates whether the encoded H.265 sequence parameter set identified by the pair constructed from `stdVPSId` and `stdSPSId` is requested to be retrieved.
- `writeStdPPS` indicates whether the encoded H.265 picture parameter set identified by the triplet constructed from `stdVPSId`, `stdSPSId`, and `stdPPSId` is requested to be retrieved.

- `stdVPSId` specifies the H.265 video parameter set ID used to identify the retrieved H.265 video, sequence, and/or picture parameter set(s).
- `stdSPSId` specifies the H.265 sequence parameter set ID used to identify the retrieved H.265 sequence and/or picture parameter set(s) when `writeStdSPS` and/or `writeStdPPS` is `VK_TRUE`.
- `stdPPSId` specifies the H.265 picture parameter set ID used to identify the retrieved H.265 picture parameter set when `writeStdPPS` is `VK_TRUE`.

When this structure is specified in the `pNext` chain of the VkVideoEncodeSessionParametersGetInfoKHR structure passed to vkGetEncodedVideoSessionParametersKHR, the command will write encoded parameter data to the output buffer in the following order:

1. The H.265 video parameter set identified by `stdVPSId`, if `writeStdVPS` is `VK_TRUE`.
2. The H.265 sequence parameter set identified by the pair constructed from `stdVPSId` and `stdSPSId`, if `writeStdSPS` is `VK_TRUE`.
3. The H.265 picture parameter set identified by the triplet constructed from `stdVPSId`, `stdSPSId`, and `stdPPSId`, if `writeStdPPS` is `VK_TRUE`.

Valid Usage

- VUID-VkVideoEncodeH265SessionParametersGetInfoKHR-writeStdVPS-08290
  At least one of `writeStdVPS`, `writeStdSPS`, and `writeStdPPS` **must** be `VK_TRUE`

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265SessionParametersGetInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_GET_INFO_KHR`

The `VkVideoEncodeH265SessionParametersFeedbackInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265SessionParametersFeedbackInfoKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           hasStdVPSOverrides;
    VkBool32           hasStdSPSOverrides;
    VkBool32           hasStdPPSOverrides;
} VkVideoEncodeH265SessionParametersFeedbackInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `hasStdVPSOverrides` indicates whether any of the parameters of the requested H.265 video parameter set, if one was requested via VkVideoEncodeH265SessionParametersGetInfoKHR

::writeStdVPS, were overridden by the implementation.

- hasStdSPSOverrides indicates whether any of the parameters of the requested H.265 sequence parameter set, if one was requested via VkVideoEncodeH265SessionParametersGetInfoKHR ::writeStdSPS, were overridden by the implementation.
- hasStdPPSOverrides indicates whether any of the parameters of the requested H.265 picture parameter set, if one was requested via VkVideoEncodeH265SessionParametersGetInfoKHR ::writeStdPPS, were overridden by the implementation.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265SessionParametersFeedbackInfoKHR-sType-sType sType must be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_FEEDBACK_INFO_KHR

38.19.11. H.265 Encoding Parameters

The VkVideoEncodeH265PictureInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265PictureInfoKHR {
    VkStructureType                                   sType;
    const void*                                       pNext;
    uint32_t                                          naluSliceSegmentEntryCount;
    const VkVideoEncodeH265NaluSliceSegmentInfoKHR*   pNaluSliceSegmentEntries;
    const StdVideoEncodeH265PictureInfo*              pStdPictureInfo;
} VkVideoEncodeH265PictureInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- naluSliceSegmentEntryCount is the number of elements in pNaluSliceSegmentEntries.
- pNaluSliceSegmentEntries is a pointer to an array of naluSliceSegmentEntryCount VkVideoEncodeH265NaluSliceSegmentInfoKHR structures specifying the parameters of the individual H.265 slice segments to encode for the input picture.
- pStdPictureInfo is a pointer to a StdVideoEncodeH265PictureInfo structure specifying H.265 picture information.

This structure is specified in the pNext chain of the VkVideoEncodeInfoKHR structure passed to vkCmdEncodeVideoKHR to specify the codec-specific picture information for an H.265 encode operation.

Encode Input Picture Information

When this structure is specified in the pNext chain of the VkVideoEncodeInfoKHR structure passed to vkCmdEncodeVideoKHR, the information related to the encode input picture is defined as follows:

- The image subregion used is determined according to the H.265 Encode Picture Data Access section.
- The encode input picture is associated with the H.265 picture information provided in `pStdPictureInfo`.

Std Picture Information

The members of the `StdVideoEncodeH265PictureInfo` structure pointed to by `pStdPictureInfo` are interpreted as follows:

- `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
- `flags.is_reference` as defined in section 3.132 of the ITU-T H.265 Specification;
- `flags.IrapPicFlag` as defined in section 3.73 of the ITU-T H.265 Specification;
- `flags.used_for_long_term_reference` is used to indicate whether the picture is marked as "used for long-term reference" as defined in section 8.3.2 of the ITU-T H.265 Specification;
- `flags.discardable_flag` and `cross_layer_bla_flag` as defined in section F.7.4.7.1 of the ITU-T H.265 Specification;
- `pic_type` as defined in section 7.4.3.5 of the ITU-T H.265 Specification;
- `sps_video_parameter_set_id`, `pps_seq_parameter_set_id`, and `pps_pic_parameter_set_id` are used to identify the active parameter sets, as described below;
- `PicOrderCntVal` as defined in section 8.3.1 of the ITU-T H.265 Specification;
- `TemporalId` as defined in section 7.4.2.2 of the ITU-T H.265 Specification;
- if `pRefLists` is not `NULL`, then it is a pointer to a `StdVideoEncodeH265ReferenceListsInfo` structure that is interpreted as follows:
  - `flags.reserved` is used only for padding purposes and is otherwise ignored;
  - `ref_pic_list_modification_flag_l0` and `ref_pic_list_modification_flag_l1` as defined in section 7.4.7.2 of the ITU-T H.265 Specification;
  - `num_ref_idx_l0_active_minus1` and `num_ref_idx_l1_active_minus1` as defined in section 7.4.7.1 of the ITU-T H.265 Specification;
  - `RefPicList0` and `RefPicList1` as defined in section 8.3.4 of the ITU-T H.265 Specification where each element of these arrays either identifies an active reference picture using its DPB slot index or contains the value `STD_VIDEO_H265_NO_REFERENCE_PICTURE` to indicate "no reference picture";
  - `list_entry_l0` and `list_entry_l1` as defined in section 7.4.7.2 of the ITU-T H.265 Specification;
- if `flags.short_term_ref_pic_set_sps_flag` is set, then the `StdVideoH265ShortTermRefPicSet` structure pointed to by `pShortTermRefPicSet` is interpreted as defined for the elements of the `pShortTermRefPicSet` array specified in H.265 sequence parameter sets.
- if `flags.long_term_ref_pics_present_flag` is set in the active SPS, then the `StdVideoEncodeH265LongTermRefPics` structure pointed to by `pLongTermRefPics` is interpreted as follows:
  - `used_by_curr_pic_lt_flag` is a bitmask where bit index i corresponds to

used_by_curr_pic_lt_flag[i] as defined in section 7.4.7.1 of the ITU-T H.265 Specification;

- all other members of StdVideoEncodeH265LongTermRefPics are interpreted as defined in section 7.4.7.1 of the ITU-T H.265 Specification;

- all other members are interpreted as defined in section 7.4.7.1 of the ITU-T H.265 Specification.

Reference picture setup is controlled by the value of StdVideoEncodeH265PictureInfo::flags.is_reference. If it is set and a reconstructed picture is specified, then the latter is used as the target of picture reconstruction to activate the DPB slot specified in pEncodeInfo->pSetupReferenceSlot->slotIndex. If StdVideoEncodeH265PictureInfo::flags.is_reference is not set, but a reconstructed picture is specified, then the corresponding picture reference associated with the DPB slot is invalidated, as described in the DPB Slot States section.

Active Parameter Sets

The members of the StdVideoEncodeH265PictureInfo structure pointed to by pStdPictureInfo are used to select the active parameter sets to use from the bound video session parameters object, as follows:

- The *active VPS* is the VPS identified by the key specified in StdVideoEncodeH265PictureInfo::sps_video_parameter_set_id.
- The *active SPS* is the SPS identified by the key specified by the pair constructed from StdVideoEncodeH265PictureInfo::sps_video_parameter_set_id and StdVideoEncodeH265PictureInfo::pps_seq_parameter_set_id.
- The *active PPS* is the PPS identified by the key specified by the triplet constructed from StdVideoEncodeH265PictureInfo::sps_video_parameter_set_id, StdVideoEncodeH265PictureInfo::pps_seq_parameter_set_id, and StdVideoEncodeH265PictureInfo::pps_pic_parameter_set_id.

H.265 encoding uses *explicit weighted sample prediction* for a slice segment, as defined in section 8.5.3.3.4 of the ITU-T H.265 Specification, if any of the following conditions are true for the active PPS and the pStdSliceSegmentHeader member of the corresponding element of pNaluSliceSegmentEntries:

- pStdSliceSegmentHeader->slice_type is STD_VIDEO_H265_SLICE_TYPE_P and weighted_pred_flag is enabled in the active PPS.
- pStdSliceSegmentHeader->slice_type is STD_VIDEO_H265_SLICE_TYPE_B and weighted_bipred_flag is enabled in the active PPS.

The number of H.265 tiles, as defined in section 3.174 of the ITU-T H.265 Specification, is derived from the num_tile_columns_minus1 and num_tile_rows_minus1 members of the active PPS as follows:

$$(\texttt{num_tile_columns_minus1} + 1) \times (\texttt{num_tile_rows_minus1} + 1)$$

Valid Usage

- VUID-VkVideoEncodeH265PictureInfoKHR-naluSliceSegmentEntryCount-08306
  naluSliceSegmentEntryCount **must** be between 1 and VkVideoEncodeH265CapabilitiesKHR

::maxSliceSegmentCount, inclusive, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile

- VUID-VkVideoEncodeH265PictureInfoKHR-flags-08323
  If VkVideoEncodeH265CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H265_CAPABILITY_MULTIPLE_TILES_PER_SLICE_SEGMENT_BIT_KHR, then naluSliceSegmentEntryCount **must** be greater than or equal to the number of H.265 tiles in the picture
- VUID-VkVideoEncodeH265PictureInfoKHR-flags-08324
  If VkVideoEncodeH265CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H265_CAPABILITY_MULTIPLE_SLICE_SEGMENTS_PER_TILE_BIT_KHR, then naluSliceSegmentEntryCount **must** be less than or equal to the number of H.265 tiles in the picture
- VUID-VkVideoEncodeH265PictureInfoKHR-flags-08316
  If VkVideoEncodeH265CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H265_CAPABILITY_PREDICTION_WEIGHT_TABLE_GENERATED_BIT_KHR and the slice segment corresponding to any element of pNaluSliceSegmentEntries uses explicit weighted sample prediction, then VkVideoEncodeH265NaluSliceSegmentInfoKHR::pStdSliceSegmentHeader->pWeightTable **must** not be NULL for that element of pNaluSliceSegmentEntries
- VUID-VkVideoEncodeH265PictureInfoKHR-flags-08317
  If VkVideoEncodeH265CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H265_CAPABILITY_DIFFERENT_SLICE_SEGMENT_TYPE_BIT_KHR, then VkVideoEncodeH265NaluSliceSegmentInfoKHR::pStdSliceSegmentHeader->slice_type **must** be identical for all elements of pNaluSliceSegmentEntries

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265PictureInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_PICTURE_INFO_KHR
- VUID-VkVideoEncodeH265PictureInfoKHR-pNaluSliceSegmentEntries-parameter
  pNaluSliceSegmentEntries **must** be a valid pointer to an array of naluSliceSegmentEntryCount valid VkVideoEncodeH265NaluSliceSegmentInfoKHR structures
- VUID-VkVideoEncodeH265PictureInfoKHR-pStdPictureInfo-parameter
  pStdPictureInfo **must** be a valid pointer to a valid StdVideoEncodeH265PictureInfo value
- VUID-VkVideoEncodeH265PictureInfoKHR-naluSliceSegmentEntryCount-arraylength
  naluSliceSegmentEntryCount **must** be greater than 0

The VkVideoEncodeH265NaluSliceSegmentInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265NaluSliceSegmentInfoKHR {
    VkStructureType                                 sType;
    const void*                                     pNext;
    int32_t                                         constantQp;
    const StdVideoEncodeH265SliceSegmentHeader*     pStdSliceSegmentHeader;
} VkVideoEncodeH265NaluSliceSegmentInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `constantQp` is the QP to use for the slice segment if the current rate control mode configured for the video session is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`.
- `pStdSliceSegmentHeader` is a pointer to a `StdVideoEncodeH265SliceSegmentHeader` structure specifying H.265 slice segment header parameters for the slice segment.

Std Slice Segment Header Parameters

The members of the `StdVideoEncodeH265SliceSegmentHeader` structure pointed to by `pStdSliceSegmentHeader` are interpreted as follows:

- `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
- if `pWeightTable` is not `NULL`, then it is a pointer to a `StdVideoEncodeH265WeightTable` that is interpreted as follows:
  - `flags.luma_weight_l0_flag`, `flags.chroma_weight_l0_flag`, `flags.luma_weight_l1_flag`, and `flags.chroma_weight_l1_flag` are bitmasks where bit index i corresponds to `luma_weight_l0_flag[i]`, `chroma_weight_l0_flag[i]`, `luma_weight_l1_flag[i]`, and `chroma_weight_l1_flag[i]`, respectively, as defined in section 7.4.7.3 of the ITU-T H.265 Specification;
  - all other members of `StdVideoEncodeH265WeightTable` are interpreted as defined in section 7.4.7.3 of the ITU-T H.265 Specification;
- all other members are interpreted as defined in section 7.4.7.1 of the ITU-T H.265 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265NaluSliceSegmentInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_NALU_SLICE_SEGMENT_INFO_KHR`
- VUID-VkVideoEncodeH265NaluSliceSegmentInfoKHR-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkVideoEncodeH265NaluSliceSegmentInfoKHR-pStdSliceSegmentHeader-parameter
  `pStdSliceSegmentHeader` **must** be a valid pointer to a valid `StdVideoEncodeH265SliceSegmentHeader` value

The VkVideoEncodeH265DpbSlotInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265DpbSlotInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoEncodeH265ReferenceInfo*   pStdReferenceInfo;
} VkVideoEncodeH265DpbSlotInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pStdReferenceInfo` is a pointer to a `StdVideoEncodeH265ReferenceInfo` structure specifying H.265 reference information.

This structure is specified in the `pNext` chain of VkVideoEncodeInfoKHR::`pSetupReferenceSlot`, if not `NULL`, and the `pNext` chain of the elements of VkVideoEncodeInfoKHR::`pReferenceSlots` to specify the codec-specific reference picture information for an H.265 encode operation.

Active Reference Picture Information

When this structure is specified in the `pNext` chain of the elements of VkVideoEncodeInfoKHR::`pReferenceSlots`, one element is added to the list of active reference pictures used by the video encode operation for each element of VkVideoEncodeInfoKHR::`pReferenceSlots` as follows:

- The image subregion used is determined according to the H.265 Encode Picture Data Access section.
- The reference picture is associated with the DPB slot index specified in the `slotIndex` member of the corresponding element of VkVideoEncodeInfoKHR::`pReferenceSlots`.
- The reference picture is associated with the H.265 reference information provided in `pStdReferenceInfo`.

Reconstructed Picture Information

When this structure is specified in the `pNext` chain of VkVideoEncodeInfoKHR::`pSetupReferenceSlot`, the information related to the reconstructed picture is defined as follows:

- The image subregion used is determined according to the H.265 Encode Picture Data Access section.
- If reference picture setup is requested, then the reconstructed picture is used to activate the DPB slot with the index specified in VkVideoEncodeInfoKHR::`pSetupReferenceSlot->slotIndex`.
- The reconstructed picture is associated with the H.265 reference information provided in `pStdReferenceInfo`.

Std Reference Information

The members of the `StdVideoEncodeH265ReferenceInfo` structure pointed to by `pStdReferenceInfo` are interpreted as follows:

- `flags.reserved` is used only for padding purposes and is otherwise ignored;
- `flags.used_for_long_term_reference` is used to indicate whether the picture is marked as "used for long-term reference" as defined in section 8.3.2 of the ITU-T H.265 Specification;
- `flags.unused_for_reference` is used to indicate whether the picture is marked as "unused for reference" as defined in section 8.3.2 of the ITU-T H.265 Specification;
- `pic_type` as defined in section 7.4.3.5 of the ITU-T H.265 Specification;
- `PicOrderCntVal` as defined in section 8.3.1 of the ITU-T H.265 Specification;
- `TemporalId` as defined in section 7.4.2.2 of the ITU-T H.265 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265DpbSlotInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_DPB_SLOT_INFO_KHR`
- VUID-VkVideoEncodeH265DpbSlotInfoKHR-pStdReferenceInfo-parameter
  `pStdReferenceInfo` **must** be a valid pointer to a valid `StdVideoEncodeH265ReferenceInfo` value

38.19.12. H.265 Encode Rate Control

Group of Pictures

In case of H.265 encoding it is common practice to follow a regular pattern of different picture types in display order when encoding subsequent frames. This pattern is referred to as the *group of pictures* (GOP).

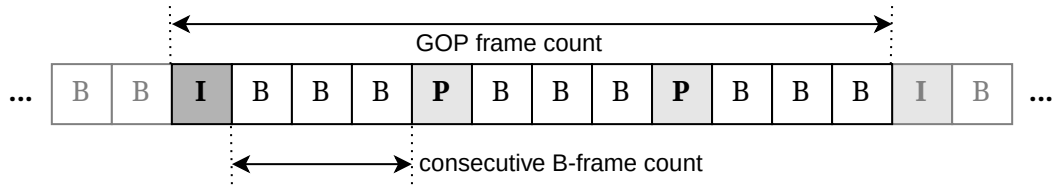
A regular GOP is defined by the following parameters:

- The number of frames in the GOP;
- The number of consecutive B frames between I and/or P frames in display order.

GOPs are further classified as *open* and *closed* GOPs.

Frame types in an open GOP follow each other in display order according to the following algorithm:

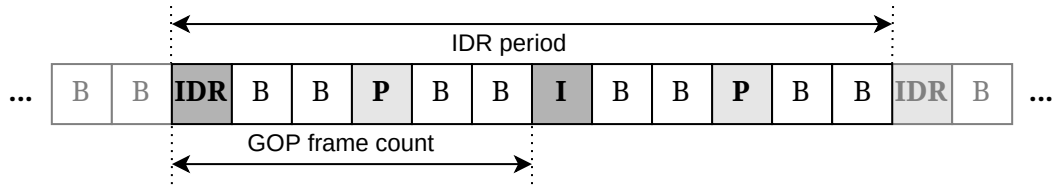
1. The first frame is always an I frame.
2. This is followed by a number of consecutive B frames, as defined above.
3. If the number of frames in the GOP is not reached yet, then the next frame is a P frame and the algorithm continues from step 2.



Text is not SVG - cannot display

Figure 31. H.265 open GOP

In case of a closed GOP, an IDR frame is used at a certain period.



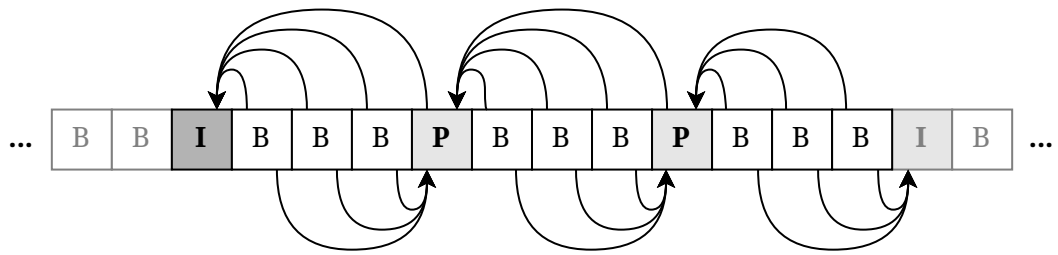
Text is not SVG - cannot display

Figure 32. H.265 closed GOP

It is also typical for H.265 encoding to use specific reference picture usage patterns across the frames of the GOP. The two most common reference patterns used are as follows:

Flat Reference Pattern

- Each P frame uses the last non-B frame, in display order, as reference.
- Each B frame uses the last non-B frame, in display order, as its forward reference, and uses the next non-B frame, in display order, as its backward reference.



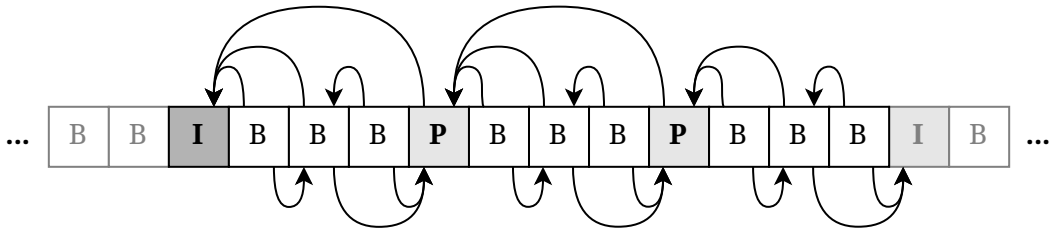
Text is not SVG - cannot display

Figure 33. H.265 flat reference pattern

Dyadic Reference Pattern

- Each P frame uses the last non-B frame, in display order, as reference.

- The following algorithm is applied to the sequence of consecutive B frames between I and/or P frames in display order:
  1. The B frame in the middle of this sequence uses the frame preceding the sequence as its forward reference, and uses the frame following the sequence as its backward reference.
  2. The algorithm is executed recursively for the following frame sequences:
     - The B frames of the original sequence preceding the frame in the middle, if any.
     - The B frames of the original sequence following the frame in the middle, if any.



Text is not SVG - cannot display

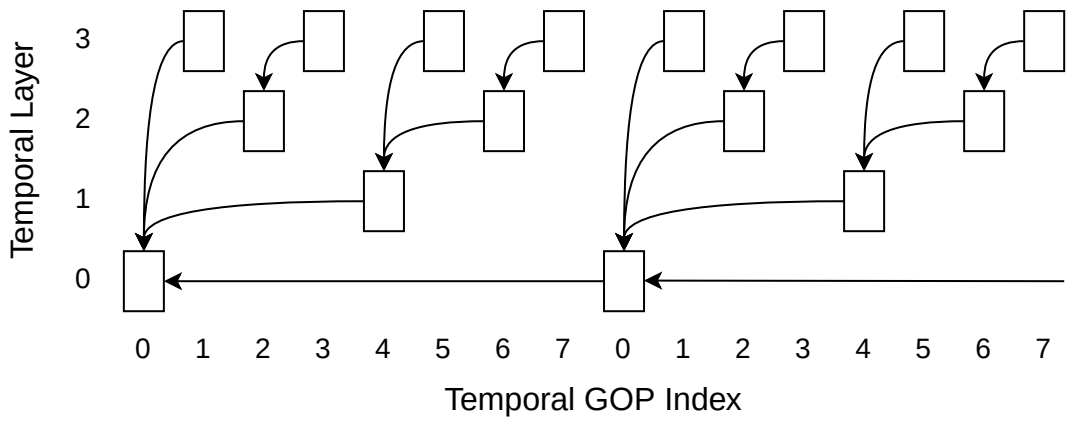
Figure 34. H.265 dyadic reference pattern

The application **can** provide guidance to the implementation's rate control algorithm about the structure of the GOP used by the application. Any such guidance about the GOP and its structure does not mandate that specific GOP structure to be used by the application, as the picture type of individual encoded pictures is still application-controlled, however, any deviation from the provided guidance **may** result in undesired rate control behavior including, but not limited, to the implementation not being able to conform to the expected average or target bitrates, or other rate control parameters specified by the application.

When an H.265 encode session is used to encode multiple temporal sub-layers, it is also common practice to follow a regular pattern for the H.265 temporal ID for the encoded pictures in display order when encoding subsequent frames. This pattern is referred to as the *temporal GOP*. The most common temporal layer pattern used is as follows:

Dyadic Temporal Sub-Layer Pattern

- The number of frames in the temporal GOP is 2^{n-1}, where n is the number of temporal sub-layers.
- The i^{th} frame in the temporal GOP uses temporal ID t, if and only if the index of the least significant bit set in i equals n-t-1, except for the first frame, which is the only frame in the temporal GOP using temporal ID zero.
- The i^{th} frame in the temporal GOP uses the r^{th} frame as reference, where r is calculated from i by clearing the least significant bit set in it, except for the first frame in the temporal GOP, which uses the first frame of the previous temporal GOP, if any, as reference.



Text is not SVG - cannot display

Figure 35. H.265 dyadic temporal sub-layer pattern

NOTE	Multi-layer rate control and multi-layer coding are typically used for streaming cases where low latency is expected, hence B pictures with backward prediction are usually not used.

The `VkVideoEncodeH265RateControlInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265RateControlInfoKHR {
    VkStructureType                           sType;
    const void*                               pNext;
    VkVideoEncodeH265RateControlFlagsKHR      flags;
    uint32_t                                  gopFrameCount;
    uint32_t                                  idrPeriod;
    uint32_t                                  consecutiveBFrameCount;
    uint32_t                                  subLayerCount;
} VkVideoEncodeH265RateControlInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkVideoEncodeH265RateControlFlagBitsKHR specifying H.265 rate control flags.
- `gopFrameCount` is the number of frames within a group of pictures (GOP) intended to be used by the application. If it is 0, the rate control algorithm **may** assume an implementation-dependent GOP length. If it is `UINT32_MAX`, the GOP length is treated as infinite.
- `idrPeriod` is the interval, in terms of number of frames, between two IDR frames (see IDR period). If it is 0, the rate control algorithm **may** assume an implementation-dependent IDR period. If it is `UINT32_MAX`, the IDR period is treated as infinite.
- `consecutiveBFrameCount` is the number of consecutive B frames between I and/or P frames within

the GOP.

- `subLayerCount` specifies the number of H.265 sub-layers that the application intends to use.

When an instance of this structure is included in the `pNext` chain of the VkVideoCodingControlInfoKHR structure passed to the vkCmdControlVideoCodingKHR command, and VkVideoCodingControlInfoKHR::`flags` includes `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR`, the parameters in this structure are used as guidance for the implementation's rate control algorithm (see Video Coding Control).

If `flags` includes `VK_VIDEO_ENCODE_H265_RATE_CONTROL_ATTEMPT_HRD_COMPLIANCE_BIT_KHR`, then the rate control state is reset to an initial state to meet HRD compliance requirements. Otherwise the new rate control state **may** be applied without a reset depending on the implementation and the specified rate control parameters.

NOTE

It would be possible to infer the picture type to be used when encoding a frame, on the basis of the values provided for `consecutiveBFrameCount`, `idrPeriod`, and `gopFrameCount`, but this inferred picture type will not be used by implementations to override the picture type provided to the video encode operation.

Valid Usage

- VUID-VkVideoEncodeH265RateControlInfoKHR-flags-08291
  If VkVideoEncodeH265CapabilitiesKHR::`flags`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include `VK_VIDEO_ENCODE_H265_CAPABILITY_HRD_COMPLIANCE_BIT_KHR`, then `flags` **must** not contain `VK_VIDEO_ENCODE_H265_RATE_CONTROL_ATTEMPT_HRD_COMPLIANCE_BIT_KHR`
- VUID-VkVideoEncodeH265RateControlInfoKHR-flags-08292
  If `flags` contains `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR` or `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR`, then it **must** also contain `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REGULAR_GOP_BIT_KHR`
- VUID-VkVideoEncodeH265RateControlInfoKHR-flags-08293
  If `flags` contains `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR`, then it **must** not also contain `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR`
- VUID-VkVideoEncodeH265RateControlInfoKHR-flags-08294
  If `flags` contains `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REGULAR_GOP_BIT_KHR`, then `gopFrameCount` **must** be greater than `0`
- VUID-VkVideoEncodeH265RateControlInfoKHR-idrPeriod-08295
  If `idrPeriod` is not `0`, then it **must** be greater than or equal to `gopFrameCount`
- VUID-VkVideoEncodeH265RateControlInfoKHR-consecutiveBFrameCount-08296
  If `consecutiveBFrameCount` is not `0`, then it **must** be less than `gopFrameCount`

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265RateControlInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_RATE_CONTROL_INFO_KHR`
- VUID-VkVideoEncodeH265RateControlInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkVideoEncodeH265RateControlFlagBitsKHR values

Bits which **can** be set in VkVideoEncodeH265RateControlInfoKHR::`flags`, specifying H.265 rate control flags, are:

```
// Provided by VK_KHR_video_encode_h265
typedef enum VkVideoEncodeH265RateControlFlagBitsKHR {
    VK_VIDEO_ENCODE_H265_RATE_CONTROL_ATTEMPT_HRD_COMPLIANCE_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_H265_RATE_CONTROL_REGULAR_GOP_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_H265_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_H265_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR = 0x00000008,
    VK_VIDEO_ENCODE_H265_RATE_CONTROL_TEMPORAL_SUB_LAYER_PATTERN_DYADIC_BIT_KHR =
0x00000010,
} VkVideoEncodeH265RateControlFlagBitsKHR;
```

- `VK_VIDEO_ENCODE_H265_RATE_CONTROL_ATTEMPT_HRD_COMPLIANCE_BIT_KHR` specifies that rate control **should** attempt to produce an HRD compliant bitstream, as defined in annex C of the ITU-T H.265 Specification.
- `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REGULAR_GOP_BIT_KHR` specifies that the application intends to use a regular GOP structure according to the parameters specified in the `gopFrameCount`, `idrPeriod`, and `consecutiveBFrameCount` members of the VkVideoEncodeH265RateControlInfoKHR structure.
- `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR` specifies that the application intends to follow a flat reference pattern in the GOP.
- `VK_VIDEO_ENCODE_H265_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR` specifies that the application intends to follow a dyadic reference pattern in the GOP.
- `VK_VIDEO_ENCODE_H265_RATE_CONTROL_TEMPORAL_SUB_LAYER_PATTERN_DYADIC_BIT_KHR` specifies that the application intends to follow a dyadic temporal sub-layer pattern.

```
// Provided by VK_KHR_video_encode_h265
typedef VkFlags VkVideoEncodeH265RateControlFlagsKHR;
```

`VkVideoEncodeH265RateControlFlagsKHR` is a bitmask type for setting a mask of zero or more VkVideoEncodeH265RateControlFlagBitsKHR.

Rate Control Layers

The `VkVideoEncodeH265RateControlLayerInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265RateControlLayerInfoKHR {
    VkStructureType                  sType;
    const void*                      pNext;
    VkBool32                         useMinQp;
    VkVideoEncodeH265QpKHR           minQp;
    VkBool32                         useMaxQp;
    VkVideoEncodeH265QpKHR           maxQp;
    VkBool32                         useMaxFrameSize;
    VkVideoEncodeH265FrameSizeKHR    maxFrameSize;
} VkVideoEncodeH265RateControlLayerInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `useMinQp` indicates whether the QP values determined by rate control will be clamped to the lower bounds on the QP values specified in `minQp`.
- `minQp` specifies the lower bounds on the QP values, for each picture type, that the implementation's rate control algorithm will use when `useMinQp` is `VK_TRUE`.
- `useMaxQp` indicates whether the QP values determined by rate control will be clamped to the upper bounds on the QP values specified in `maxQp`.
- `maxQp` specifies the upper bounds on the QP values, for each picture type, that the implementation's rate control algorithm will use when `useMaxQp` is `VK_TRUE`.
- `useMaxFrameSize` indicates whether the implementation's rate control algorithm **should** use the values specified in `maxFrameSize` as the upper bounds on the encoded frame size for each picture type.
- `maxFrameSize` specifies the upper bounds on the encoded frame size, for each picture type, when `useMaxFrameSize` is `VK_TRUE`.

When used, the values in `minQp` and `maxQp` guarantee that the effective QP values used by the implementation will respect those lower and upper bounds, respectively. However, limiting the range of QP values that the implementation is able to use will also limit the capabilities of the implementation's rate control algorithm to comply to other constraints. In particular, the implementation **may** not be able to comply to the following:

- The average and/or peak bitrate values to be used for the encoded bitstream specified in the `averageBitrate` and `maxBitrate` members of the VkVideoEncodeRateControlLayerInfoKHR structure.
- The upper bounds on the encoded frame size, for each picture type, specified in the `maxFrameSize` member of `VkVideoEncodeH265RateControlLayerInfoKHR`.

NOTE	In general, applications need to configure rate control parameters appropriately in order to be able to get the desired rate control behavior, as described in the Video Encode Rate Control section.

When an instance of this structure is included in the `pNext` chain of a

VkVideoEncodeRateControlLayerInfoKHR structure specified in one of the elements of the pLayers array member of the VkVideoEncodeRateControlInfoKHR structure passed to the vkCmdControlVideoCodingKHR command, VkVideoCodingControlInfoKHR::flags includes VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR, and the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR, it specifies the H.265-specific rate control parameters of the rate control layer corresponding to that element of pLayers.

Valid Usage

- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-useMinQp-08297
  If useMinQp is VK_TRUE, then the qpI, qpP, and qpB members of minQp **must** all be between VkVideoEncodeH265CapabilitiesKHR::minQp and VkVideoEncodeH265CapabilitiesKHR::maxQp, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-useMaxQp-08298
  If useMaxQp is VK_TRUE, then the qpI, qpP, and qpB members of maxQp **must** all be between VkVideoEncodeH265CapabilitiesKHR::minQp and VkVideoEncodeH265CapabilitiesKHR::maxQp, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-useMinQp-08299
  If useMinQp is VK_TRUE and VkVideoEncodeH265CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H265_CAPABILITY_PER_PICTURE_TYPE_MIN_MAX_QP_BIT_KHR, then the qpI, qpP, and qpB members of minQp **must** all specify the same value
- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-useMaxQp-08300
  If useMaxQp is VK_TRUE and VkVideoEncodeH265CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_H265_CAPABILITY_PER_PICTURE_TYPE_MIN_MAX_QP_BIT_KHR, then the qpI, qpP, and qpB members of maxQp **must** all specify the same value
- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-useMinQp-08375
  If useMinQp and useMaxQp are both VK_TRUE, then the qpI, qpP, and qpB members of minQp **must** all be less than or equal to the respective members of maxQp

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_RATE_CONTROL_LAYER_INFO_KHR
- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-minQp-parameter
  minQp **must** be a valid VkVideoEncodeH265QpKHR structure
- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-maxQp-parameter
  maxQp **must** be a valid VkVideoEncodeH265QpKHR structure
- VUID-VkVideoEncodeH265RateControlLayerInfoKHR-maxFrameSize-parameter

maxFrameSize **must** be a valid VkVideoEncodeH265FrameSizeKHR structure

The VkVideoEncodeH265QpKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265QpKHR {
    int32_t    qpI;
    int32_t    qpP;
    int32_t    qpB;
} VkVideoEncodeH265QpKHR;
```

- qpI is the QP to be used for I pictures.
- qpP is the QP to be used for P pictures.
- qpB is the QP to be used for B pictures.

The VkVideoEncodeH265FrameSizeKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265FrameSizeKHR {
    uint32_t    frameISize;
    uint32_t    framePSize;
    uint32_t    frameBSize;
} VkVideoEncodeH265FrameSizeKHR;
```

- frameISize is the size in bytes to be used for I frames.
- framePSize is the size in bytes to be used for P frames.
- frameBSize is the size in bytes to be used for B frames.

GOP Remaining Frames

Besides session level rate control configuration, the application **can** specify the number of frames per frame type remaining in the group of pictures (GOP).

The VkVideoEncodeH265GopRemainingFrameInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_h265
typedef struct VkVideoEncodeH265GopRemainingFrameInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           useGopRemainingFrames;
    uint32_t           gopRemainingI;
    uint32_t           gopRemainingP;
    uint32_t           gopRemainingB;
} VkVideoEncodeH265GopRemainingFrameInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `useGopRemainingFrames` indicates whether the implementation's rate control algorithm **should** use the values specified in `gopRemainingI`, `gopRemainingP`, and `gopRemainingB`. If `useGopRemainingFrames` is `VK_FALSE`, then the values of `gopRemainingI`, `gopRemainingP`, and `gopRemainingB` are ignored.
- `gopRemainingI` specifies the number of I frames the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the video encode operation.
- `gopRemainingP` specifies the number of P frames the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the video encode operation.
- `gopRemainingB` specifies the number of B frames the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the video encode operation.

Setting `useGopRemainingFrames` to `VK_TRUE` and including this structure in the `pNext` chain of VkVideoBeginCodingInfoKHR is only mandatory if the VkVideoEncodeH265CapabilitiesKHR::`requiresGopRemainingFrames` reported for the used video profile is `VK_TRUE`. However, implementations **may** use these remaining frame counts, when specified, even when it is not required. In particular, when the application does not use a regular GOP structure, these values **may** provide additional guidance for the implementation's rate control algorithm.

The VkVideoEncodeH265CapabilitiesKHR::`prefersGopRemainingFrames` capability is also used to indicate that the implementation's rate control algorithm **may** operate more accurately if the application specifies the remaining frame counts using this structure.

As with other rate control guidance values, if the effective order and number of frames encoded by the application are not in line with the remaining frame counts specified in this structure at any given point, then the behavior of the implementation's rate control algorithm **may** deviate from the one expected by the application.

Valid Usage (Implicit)

- VUID-VkVideoEncodeH265GopRemainingFrameInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_GOP_REMAINING_FRAME_INFO_KHR`

H.265 QP Delta Maps

Quantization delta maps used with an H.265 encode profile are referred to as *QP delta maps* and their texels contain integer values representing QP delta values that are applied in the process of determining the quantization parameters of the encoded picture.

Accordingly, H.265 QP delta maps always have single channel integer formats, as reported in VkVideoFormatPropertiesKHR::`format`.

When the rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`, the QP delta values are added to the per slice segment constant QP values that, in effect, enable the application to explicitly control the used QP values at the granularity of the used quantization map texel size.

For all other rate control modes, the QP delta values **can** be used to offset the QP values that the rate control algorithm would otherwise produce.

H.265 Encode Quantization

Performing H.265 encode operations involves the process of assigning QP values to individual H.265 coding units. This process depends on the used rate control mode, as well as other encode and rate control parameters, as described below:

- If the configured rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR`, then the QP value is initialized by the implementation-specific default rate control algorithm.
  - If the video encode operation is issued with a quantization delta map, the QP delta value corresponding to the coding unit, as fetched from the quantization map, is added to the previously determined QP value. If the fetched QP delta value falls outside the supported QP delta value range reported in the `minQpDelta` and `maxQpDelta` members of VkVideoEncodeH265QuantizationMapCapabilitiesKHR, then the QP value used for the coding unit becomes undefined.
- If the configured rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`, then the QP value is initialized from the constant QP value specified for the H.265 slice segment the coding unit is part of.
  - If the video encode operation is issued with a quantization delta map, the QP delta value corresponding to the coding unit, as fetched from the quantization map, is added to the previously determined QP value. If the fetched QP delta value falls outside the supported QP delta value range reported in the `minQpDelta` and `maxQpDelta` members of VkVideoEncodeH265QuantizationMapCapabilitiesKHR, then the QP value used for the coding unit becomes undefined.
- If the configured rate control mode is not `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR` or `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`, then the QP value is initialized by the corresponding rate control algorithm.
  - If the video encode operation is issued with a quantization delta map, the QP delta value corresponding to the coding unit, as fetched from the quantization map, is added to the previously determined QP value. If the fetched QP delta value falls outside the supported QP delta value range reported in the `minQpDelta` and `maxQpDelta` members of VkVideoEncodeH265QuantizationMapCapabilitiesKHR, then the QP value used for the coding unit becomes undefined.
  - If the video encode operation is issued with an emphasis map, the rate control will adjust the QP value based on the emphasis value corresponding to the coding unit, as fetched from the quantization map, according to the following equation:

    $QP_{new} = f(QP_{prev},e)$

    Where QP_{new} is the resulting QP value, QP_{prev} is the previously determined QP value, e is the emphasis value corresponding to the coding unit, and f is an implementation-defined function for which the following implication is true:

    $e_1 < e_2 \Rightarrow f(QP,e_1) \geq f(QP,e_2)$

NOTE	This means that lower emphasis values will result in higher QP values, whereas higher emphasis values will result in lower QP values, but the function is not strictly decreasing with respect to the input emphasis value for a given input QP value.

    - If clamping to minimum QP values is enabled in the applied rate control layer, then the QP value is clamped to the corresponding minimum QP value.
    - If clamping to maximum QP values is enabled in the applied rate control layer, then the QP value is clamped to the corresponding maximum QP value.
- If `VK_VIDEO_ENCODE_H265_CAPABILITY_CU_QP_DIFF_WRAPAROUND_BIT_KHR` is not supported, then the determined QP value is clamped in such a way that the `CuQpDeltaVal` value of the encoded coding unit complies to the modified version of equation 8-283 of the ITU-T H.265 Specification.

NOTE	The effect of this is that the maximum QP difference across subsequent coding units is limited to the [-(26 + $QpBdOffset_Y$ / 2), 25 + $QpBdOffset_Y$ / 2] range and only has an observable change in behavior when the video encode operation is issued with a QP delta map.

- In all cases, the final QP value is clamped to the QP value range supported by the video profile, as reported in the `minQp` and `maxQp` members of VkVideoEncodeH265CapabilitiesKHR.

38.19.13. H.265 Encode Requirements

This section described the **required** H.265 encoding capabilities for physical devices that have at least one queue family that supports the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR`, as returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR in VkQueueFamilyVideoPropertiesKHR ::`videoCodecOperations`.

Table 55. Required Video Std Header Versions

Video Std Header Name	Version
`vulkan_video_codec_h265std_encode`	1.0.0

Table 56. Required Video Capabilities

Video Capability	Requirement	Requirement Type[1]
VkVideoCapabilitiesKHR		
`flags`	-	min
`minBitstreamBufferOffsetAlignment`	4096	max
`minBitstreamBufferSizeAlignment`	4096	max
`pictureAccessGranularity`	(64,64)	max
`minCodedExtent`	-	max
`maxCodedExtent`	-	min

Video Capability	Requirement	Requirement Type[1]
maxDpbSlots	0	min
maxActiveReferencePictures	0	min
VkVideoEncodeCapabilitiesKHR		
flags	-	min
rateControlModes	-	min
maxBitrate	128000	min
maxQualityLevels	1	min
encodeInputPictureGranularity	(64,64)	max
supportedEncodeFeedbackFlags	VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BUFFER_OFFSET_BIT_KHR VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BYTES_WRITTEN_BIT_KHR	min
VkVideoEncodeH265CapabilitiesKHR		
flags	-	min
maxLevelIdc	STD_VIDEO_H265_LEVEL_IDC_1_0	min
maxSliceSegmentCount	1	min
maxTiles	(1,1)	min
ctbSizes	at least one bit set	implementation-dependent
transformBlockSizes	at least one bit set	implementation-dependent
maxPPictureL0ReferenceCount	0	min
maxBPictureL0ReferenceCount	0	min
maxL1ReferenceCount	0	min
maxSubLayerCount	1	min
expectDyadicTemporalSubLayerPattern	-	implementation-dependent
minQp	-	max
maxQp	-	min
prefersGopRemainingFrames	-	implementation-dependent
requiresGopRemainingFrames	-	implementation-dependent

Video Capability	Requirement	Requirement Type[1]
stdSyntaxFlags	-	min
VkVideoEncodeQuantizationMapCapabilitiesKHR		
maxQuantizationMapExtent	-[2]	min
VkVideoEncodeH265QuantizationMapCapabilitiesKHR		
minQpDelta	-[3]	max
maxQpDelta	-[3]	min

1

The **Requirement Type** column specifies the requirement is either the minimum value all implementations **must** support, the maximum value all implementations **must** support, or the exact value all implementations **must** support. For bitmasks a minimum value is the least bits all implementations **must** set, but they **may** have additional bits set beyond this minimum.

2

If VkVideoCapabilitiesKHR::flags includes VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR, then the width and height members of maxQuantizationMapExtent **must** be greater than zero.

3

If VkVideoCapabilitiesKHR::flags includes VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR, then maxQpDelta **must** be greater than minQpDelta.

38.20. AV1 Encode Operations

Video encode operations using an AV1 encode profile **can** be used to encode elementary video stream sequences compliant with the AV1 Specification.

NOTE Refer to the Preamble for information on how the Khronos Intellectual Property Rights Policy relates to normative references to external materials not created by Khronos.

This process is performed according to the video encode operation steps with the codec-specific semantics defined in section 7 of the AV1 Specification:

- Syntax elements, derived values, and other parameters are applied from the following structures:
  - The StdVideoAV1SequenceHeader structure, the optional StdVideoEncodeAV1DecoderModelInfo structure, and the optional array of StdVideoEncodeAV1OperatingPointInfo structures stored in the bound video session parameters object specifying the active sequence header.

    - The `StdVideoEncodeAV1PictureInfo` structure specifying the AV1 picture information.
    - The `StdVideoEncodeAV1ReferenceInfo` structures specifying the AV1 reference information corresponding to the optional reconstructed picture and any active reference pictures.
    - The encoded bitstream data is written to the destination video bitstream buffer range as defined in the AV1 Encode Bitstream Data Access section.
    - Picture data in the video picture resources corresponding to the used encode input picture, active reference pictures, and optional reconstructed picture is accessed as defined in the AV1 Encode Picture Data Access section.
- The decision on reference picture setup is made according to the parameters specified in the AV1 picture information.

If the parameters adhere to the syntactic and semantic requirements defined in the corresponding sections of the AV1 Specification, as described above, and the DPB slots associated with the active reference pictures all refer to valid picture references, then the video encode operation will complete successfully. Otherwise, the video encode operation **may** complete unsuccessfully.

38.20.1. AV1 Encode Parameter Overrides

Implementations **may** override, unless otherwise specified, any of the AV1 encode parameters specified in the following Video Std structures:

- `StdVideoAV1SequenceHeader`
- `StdVideoEncodeAV1DecoderModelInfo`
- `StdVideoEncodeAV1OperatingPointInfo`
- `StdVideoEncodeAV1PictureInfo`
- `StdVideoEncodeAV1ReferenceInfo`

All such AV1 encode parameter overrides **must** fulfill the conditions defined in the Video Encode Parameter Overrides section.

In addition, implementations **must** not override any of the following AV1 encode parameters:

- the following parameters specified in `StdVideoAV1SequenceHeader`:
    - `flags.still_picture`
    - `flags.enable_order_hint`
    - `flags.frame_id_numbers_present_flag`
    - `flags.film_grain_params_present`
    - `flags.timing_info_present_flag`
    - `flags.initial_display_delay_present_flag`
    - `delta_frame_id_length_minus_2`
    - `additional_frame_id_length_minus_1`
    - `order_hint_bits_minus_1`

- the following parameters specified in the StdVideoAV1ColorConfig structure pointed to by StdVideoAV1SequenceHeader::pColorConfig:
  - flags.mono_chrome
  - flags.color_range
  - BitDepth
  - subsampling_x
  - subsampling_y
  - color_primaries
  - transfer_characteristics
  - matrix_coefficients
  - chroma_sample_position
- the following parameters specified in the StdVideoAV1TimingInfo structure pointed to by StdVideoAV1SequenceHeader::pTimingInfo:
  - flags.equal_picture_interval
  - num_units_in_display_tick
  - time_scale
  - num_ticks_per_picture_minus_1
- the parameters specified in StdVideoEncodeAV1DecoderModelInfo
- the parameters specified in StdVideoEncodeAV1OperatingPointInfo
- the following parameters specified in StdVideoEncodeAV1PictureInfo:
  - flags.show_frame
  - flags.showable_frame
  - frame_type
  - frame_presentation_time
  - current_frame_id
  - order_hint
  - refresh_frame_flags
  - render_width_minus_1
  - render_height_minus_1
  - ref_order_hint
  - ref_frame_idx
  - delta_frame_id_minus_1
- the following parameters specified in the StdVideoEncodeAV1ExtensionHeader structure pointed to by StdVideoEncodeAV1PictureInfo::pExtensionHeader when VkVideoEncodeAV1PictureInfoKHR::generateObuExtensionHeader is set to VK_TRUE:
  - temporal_id

- spatial_id

If VkVideoEncodeAV1PictureInfoKHR::primaryReferenceCdfOnly is set to VK_TRUE for a video encode operation, the implementation will not override StdVideoEncodeAV1PictureInfo::primary_ref_frame.

NOTE

Implementations supporting the VK_VIDEO_ENCODE_AV1_STD_PRIMARY_REF_FRAME_BIT_KHR AV1 syntax element capability reported in VkVideoEncodeAV1CapabilitiesKHR::stdSyntaxFlag also guarantee the use of the application-specified value for StdVideoEncodeAV1PictureInfo::primary_ref_frame. While support for VK_VIDEO_ENCODE_AV1_STD_PRIMARY_REF_FRAME_BIT_KHR guarantees that the implementation will not override the application-specified primary_ref_frame, it does not mandate the implementation to use the reference picture indicated by primary_ref_frame for sample prediction as implementations can always decide to use only a subset of the application-specified reference pictures for sample prediction. This means that implementations supporting VK_VIDEO_ENCODE_AV1_STD_PRIMARY_REF_FRAME_BIT_KHR may end up using the reference picture indicated by primary_ref_frame only for CDF data reference even if the application did not set VkVideoEncodeAV1PictureInfoKHR::primaryReferenceCdfOnly to VK_TRUE.

If VkVideoEncodeAV1CapabilitiesKHR::codedPictureAlignment is not equal to {8,8} for the used video profile, implementations will override the coded picture's resolution and parameters related to the width and height in the following manner:

- Let w and h be the codedExtent.width and codedExtent.height of the VkVideoPictureResourceInfoKHR structure corresponding to the encode input picture, rounded up to the nearest integer multiple of 8.
- Let aW and aH be w and h rounded up to the nearest integer multiple of codedPictureAlignment.width and codedPictureAlignment.height respectively.
- If w equals aW, no override will occur. Otherwise the coded width will be aW.
- If h equals aH, no override will occur. Otherwise the coded height will be aH.

NOTE

The AV1 specification codes all resolutions to an 8x8 alignment, but supports unaligned resolutions through implicit cropping. Thus, if the original coded extent, aligned to 8x8, meets the implementation required alignment, no override needs to occur. Otherwise, the implementation cannot code the requested coded extent, so the final resolution in the bitstream is overridden to be aligned to the implementation required alignment.

For example, consider an implementation that is only able to output bitstreams that are 16x16 aligned (as indicated by VkVideoEncodeAV1CapabilitiesKHR::codedPictureAlignment). If an application requests the coded extent to be 1920x1080, the resulting bitstream will be 1920x1088. On the other hand, a request of 1920x1082 will result in no override, since the 8x8 alignment of this resolution (1920x1088) is 16x16 aligned.

In case of a video session parameters object created with VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR, the following AV1 sequence header parameters **may** be overridden by the implementation according to the quantization map texel size the video session parameters object was created with:

- StdVideoAV1SequenceHeader::flags.use_128x128_superblock

This **may** be necessary to change the AV1 superblock size used during encoding to be compatible with the used quantization map texel size.

In case of AV1 encode parameters stored in video session parameters objects, applications need to use the vkGetEncodedVideoSessionParametersKHR command to determine whether any implementation overrides happened. If the query indicates that implementation overrides were applied, then the application needs to retrieve and use the encoded AV1 sequence header in the bitstream in order to be able to produce a compliant AV1 video bitstream using the AV1 encode parameters stored in the video session parameters object.

In case of any AV1 encode parameters stored in the encoded bitstream produced by video encode operations, if the implementation supports the VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR video encode feedback query flag, the application **can** use such queries to retrieve feedback about whether any implementation overrides have been applied to those AV1 encode parameters.

38.20.2. AV1 Encode Bitstream Data Access

Each video encode operation writes either:

- A single OBU with obu_type equal to OBU_FRAME comprising of the frame header and tile data of the encoded picture, or
- An OBU with obu_type equal to OBU_FRAME_HEADER encapsulating the frame header of the encoded picture, followed by one or more OBUs with obu_type equal to OBU_TILE_GROUP comprising of the tile data of the encoded picture.

In addition, if VkVideoEncodeAV1PictureInfoKHR::generateObuExtensionHeader is set to VK_TRUE for the video encode operation, then OBU extension headers are included in the generated bitstream as defined in sections 5.3.1, 5.3.2, and 5.3.3 of the AV1 Specification.

38.20.3. AV1 Encode Picture Data Access

Accesses to image data within a video picture resource happen at the granularity indicated by VkVideoCapabilitiesKHR::pictureAccessGranularity, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile. Accordingly, the complete image subregion of a encode input picture, reference picture, or reconstructed picture accessed by video coding operations using an AV1 encode profile is defined as the set of texels within the coordinate range:

([0,endX),[0,endY))

Where:

- endX equals codedExtent.width rounded up to the nearest integer multiple of pictureAccessGranularity.width and clamped to the width of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;
- endY equals codedExtent.height rounded up to the nearest integer multiple of pictureAccessGranularity.height and clamped to the height of the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure;

Where codedExtent is the member of the VkVideoPictureResourceInfoKHR structure corresponding to the picture.

In case of video encode operations using an AV1 encode profile, any access to a picture at the coordinates (x,y), as defined by the AV1 Specification, is an access to the image subresource referred to by the corresponding VkVideoPictureResourceInfoKHR structure at the texel coordinates (x,y).

Implementations **may** choose not to access some or all texels within particular reference pictures available to a video encode operation (e.g. due to video encode parameter overrides restricting the effective set of used reference pictures, or if the encoding algorithm chooses not to use certain subregions of the reference picture data for sample prediction).

38.20.4. AV1 Reference Names and Semantics

Individual reference frames used in the encoding process have different semantics, as defined in section 6.10.24 of the AV1 Specification. The AV1 semantics associated with a reference picture is indicated by the corresponding enumeration constant defined in the Video Std enumeration type StdVideoAV1ReferenceName:

- STD_VIDEO_AV1_REFERENCE_NAME_INTRA_FRAME identifies the reference used for intra coding (INTRA_FRAME), as defined in sections 2 and 7.11.2 of the AV1 Specification.
- All other enumeration constants refer to backward or forward references used for inter coding, as defined in sections 2 and 7.11.3 of the AV1 Specification:
  - STD_VIDEO_AV1_REFERENCE_NAME_LAST_FRAME identifies the LAST_FRAME reference
  - STD_VIDEO_AV1_REFERENCE_NAME_LAST2_FRAME identifies the LAST2_FRAME reference
  - STD_VIDEO_AV1_REFERENCE_NAME_LAST3_FRAME identifies the LAST3_FRAME reference
  - STD_VIDEO_AV1_REFERENCE_NAME_GOLDEN_FRAME identifies the GOLDEN_FRAME reference
  - STD_VIDEO_AV1_REFERENCE_NAME_BWDREF_FRAME identifies the BWDREF_FRAME reference
  - STD_VIDEO_AV1_REFERENCE_NAME_ALTREF2_FRAME identifies the ALTREF2_FRAME reference
  - STD_VIDEO_AV1_REFERENCE_NAME_ALTREF_FRAME identifies the ALTREF_FRAME reference

These enumeration constants are not directly used in any APIs but are used to indirectly index into certain Video Std and Vulkan API parameter arrays.

38.20.5. AV1 Prediction Modes

AV1 encoding supports multiple types of prediction modes, as described in section 6.10.24 of the

AV1 Specification.

Possible AV1 encode prediction modes are as follows:

```
// Provided by VK_KHR_video_encode_av1
typedef enum VkVideoEncodeAV1PredictionModeKHR {
    VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_INTRA_ONLY_KHR = 0,
    VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_SINGLE_REFERENCE_KHR = 1,
    VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_UNIDIRECTIONAL_COMPOUND_KHR = 2,
    VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_BIDIRECTIONAL_COMPOUND_KHR = 3,
} VkVideoEncodeAV1PredictionModeKHR;
```

- VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_INTRA_ONLY_KHR specifies the use of *intra-only prediction mode*, used when encoding AV1 frames of type STD_VIDEO_AV1_FRAME_TYPE_KEY or STD_VIDEO_AV1_FRAME_TYPE_INTRA_ONLY.
- VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_SINGLE_REFERENCE_KHR specifies the use of *single reference prediction mode*, used when encoding AV1 frames of type STD_VIDEO_AV1_FRAME_TYPE_INTER or STD_VIDEO_AV1_FRAME_TYPE_SWITCH with reference_select, as defined in section 6.8.23 of the AV1 Specification, equal to 0. When using this prediction mode, the application **must** specify a reference picture for at least one AV1 reference name in VkVideoEncodeAV1PictureInfoKHR::referenceNameSlotIndices that is supported by the implementation, as reported in VkVideoEncodeAV1CapabilitiesKHR::singleReferenceNameMask.
- VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_UNIDIRECTIONAL_COMPOUND_KHR specifies the use of *unidirectional compound prediction mode*, used when encoding AV1 frames of type STD_VIDEO_AV1_FRAME_TYPE_INTER or STD_VIDEO_AV1_FRAME_TYPE_SWITCH with reference_select, as defined in section 6.8.23 of the AV1 Specification, equal to 1, and both reference names used for prediction are from the same reference frame group, as defined in section 6.10.24 of the AV1 Specification. When using this prediction mode, the application **must** specify a reference picture for at least two AV1 reference names in VkVideoEncodeAV1PictureInfoKHR::referenceNameSlotIndices that is supported by the implementation, as reported in VkVideoEncodeAV1CapabilitiesKHR::unidirectionalCompoundReferenceNameMask, where those two reference names are one of the allowed pairs of reference names, as defined in section 5.11.25 of the AV1 Specification, listed below:
  - LAST_FRAME and LAST2_FRAME,
  - LAST_FRAME and LAST3_FRAME,
  - LAST_FRAME and GOLDEN_FRAME, or
  - BWDREF_FRAME and ALTREF_FRAME.
- VK_VIDEO_ENCODE_AV1_PREDICTION_MODE_BIDIRECTIONAL_COMPOUND_KHR specifies the use of *bidirectional compound prediction mode*, used when encoding AV1 frames of type STD_VIDEO_AV1_FRAME_TYPE_INTER or STD_VIDEO_AV1_FRAME_TYPE_SWITCH with reference_select, as defined in section 6.8.23 of the AV1 Specification, equal to 1, and the two reference names used for prediction are from different reference frame groups, as defined in section 6.10.24 of the AV1 Specification. When using this prediction mode, the application **must** specify a reference picture for at least one AV1 reference name from each reference frame group in VkVideoEncodeAV1PictureInfoKHR::referenceNameSlotIndices that is supported by the

implementation, as reported in VkVideoEncodeAV1CapabilitiesKHR::bidirectionalCompoundReferenceNameMask.

The effective prediction mode used to encode individual AV1 mode info blocks **may** use simpler prediction modes than the one set by the application for the frame, as allowed by the AV1 Specification, in particular:

- Frames encoded with single reference prediction mode **may** contain mode info blocks encoded with intra-only prediction mode.
- Frames encoded with unidirectional compound prediction mode **may** contain mode info blocks encoded with intra-only or single reference prediction mode.
- Frames encoded with bidirectional compound prediction mode **may** contain mode info blocks encoded with intra-only, single reference, or unidirectional compound prediction mode.

38.20.6. AV1 Coding Blocks

AV1 encode supports two types of coding blocks, as defined in section 2 of the AV1 Specification:

- *Superblock*.
- *Mode info block*.

38.20.7. AV1 Encode Profile

A video profile supporting AV1 video encode operations is specified by setting VkVideoProfileInfoKHR::videoCodecOperation to VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR and adding a VkVideoEncodeAV1ProfileInfoKHR structure to the VkVideoProfileInfoKHR::pNext chain.

The VkVideoEncodeAV1ProfileInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1ProfileInfoKHR {
    VkStructureType       sType;
    const void*           pNext;
    StdVideoAV1Profile    stdProfile;
} VkVideoEncodeAV1ProfileInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- stdProfile is a StdVideoAV1Profile value specifying the AV1 codec profile, as defined in section A.2 of the AV1 Specification.

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1ProfileInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_PROFILE_INFO_KHR

38.20.8. AV1 Encode Capabilities

When calling vkGetPhysicalDeviceVideoCapabilitiesKHR to query the capabilities for an AV1 encode profile, the VkVideoCapabilitiesKHR::pNext chain **must** include a VkVideoEncodeAV1CapabilitiesKHR structure that will be filled with the profile-specific capabilities.

The VkVideoEncodeAV1CapabilitiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1CapabilitiesKHR {
    VkStructureType                           sType;
    void*                                     pNext;
    VkVideoEncodeAV1CapabilityFlagsKHR        flags;
    StdVideoAV1Level                          maxLevel;
    VkExtent2D                                codedPictureAlignment;
    VkExtent2D                                maxTiles;
    VkExtent2D                                minTileSize;
    VkExtent2D                                maxTileSize;
    VkVideoEncodeAV1SuperblockSizeFlagsKHR    superblockSizes;
    uint32_t                                  maxSingleReferenceCount;
    uint32_t                                  singleReferenceNameMask;
    uint32_t                                  maxUnidirectionalCompoundReferenceCount;
    uint32_t
maxUnidirectionalCompoundGroup1ReferenceCount;
    uint32_t                                  unidirectionalCompoundReferenceNameMask;
    uint32_t                                  maxBidirectionalCompoundReferenceCount;
    uint32_t
maxBidirectionalCompoundGroup1ReferenceCount;
    uint32_t
maxBidirectionalCompoundGroup2ReferenceCount;
    uint32_t                                  bidirectionalCompoundReferenceNameMask;
    uint32_t                                  maxTemporalLayerCount;
    uint32_t                                  maxSpatialLayerCount;
    uint32_t                                  maxOperatingPoints;
    uint32_t                                  minQIndex;
    uint32_t                                  maxQIndex;
    VkBool32                                  prefersGopRemainingFrames;
    VkBool32                                  requiresGopRemainingFrames;
    VkVideoEncodeAV1StdFlagsKHR               stdSyntaxFlags;
} VkVideoEncodeAV1CapabilitiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkVideoEncodeAV1CapabilityFlagBitsKHR indicating supported AV1 encoding capabilities.
- maxLevel is a StdVideoAV1Level value indicating the maximum AV1 level supported by the profile, as defined in section A.3 of the AV1 Specification.
- codedPictureAlignment indicates the alignment at which the implementation will code pictures.

This capability does not impose any valid usage constraints on the application. However, depending on the codedExtent of the encode input picture resource, this capability **may** result in a change of the resolution of the encoded picture, as described in more detail below.

- maxTiles indicates the maximum number of AV1 tile columns and rows the implementation supports.
- minTileSize indicates the minimum extent of individual AV1 tiles the implementation supports.
- maxTileSize indicates the maximum extent of individual AV1 tiles the implementation supports.
- superblockSizes is a bitmask of VkVideoEncodeAV1SuperblockSizeFlagBitsKHR values indicating the supported AV1 superblock sizes.
- maxSingleReferenceCount indicates the maximum number of reference pictures the implementation supports when using single reference prediction mode.
- singleReferenceNameMask is a bitmask of supported AV1 reference names when using single reference prediction mode.
- maxUnidirectionalCompoundReferenceCount indicates the maximum number of reference pictures the implementation supports when using unidirectional compound prediction mode.
- maxUnidirectionalCompoundGroup1ReferenceCount indicates the maximum number of reference pictures the implementation supports when using unidirectional compound prediction mode from reference frame group 1, as defined in section 6.10.24 of the AV1 Specification.
- unidirectionalCompoundReferenceNameMask is a bitmask of supported AV1 reference names when using unidirectional compound prediction mode.
- maxBidirectionalCompoundReferenceCount indicates the maximum number of reference pictures the implementation supports when using bidirectional compound prediction mode.
- maxBidirectionalCompoundGroup1ReferenceCount indicates the maximum number of reference pictures the implementation supports when using bidirectional compound prediction mode from reference frame group 1, as defined in section 6.10.24 of the AV1 Specification.
- maxBidirectionalCompoundGroup2ReferenceCount indicates the maximum number of reference pictures the implementation supports when using bidirectional compound prediction mode from reference frame group 2, as defined in section 6.10.24 of the AV1 Specification.
- bidirectionalCompoundReferenceNameMask is a bitmask of supported AV1 reference names when using bidirectional compound prediction mode.
- maxTemporalLayerCount indicates the maximum number of AV1 temporal layers supported by the implementation.
- maxSpatialLayerCount indicates the maximum number of AV1 spatial layers supported by the implementation.
- maxOperatingPoints indicates the maximum number of AV1 operating points supported by the implementation.
- minQIndex indicates the minimum quantizer index value supported.
- maxQIndex indicates the maximum quantizer index value supported.
- prefersGopRemainingFrames indicates that the implementation's rate control algorithm prefers the application to specify the number of frames in each AV1 rate control group remaining in the

current group of pictures when beginning a video coding scope.

- `requiresGopRemainingFrames` indicates that the implementation's rate control algorithm requires the application to specify the number of frames in each AV1 rate control group remaining in the current group of pictures when beginning a video coding scope.
- `stdSyntaxFlags` is a bitmask of VkVideoEncodeAV1StdFlagBitsKHR indicating capabilities related to AV1 syntax elements.

`singleReferenceNameMask`, `unidirectionalCompoundReferenceNameMask`, and `bidirectionalCompoundReferenceNameMask` are encoded such that when bit index i is set, it indicates support for the AV1 reference name `STD_VIDEO_AV1_REFERENCE_NAME_LAST_FRAME` + i.

NOTE These masks indicate which elements of the `referenceNameSlotIndices` member of VkVideoEncodeAV1PictureInfoKHR are supported to be used by the implementation. It is important to note that both the bits of these masks and the elements of `referenceNameSlotIndices` are indexed such that the first value specifies the support bit and DPB slot index, respectively, for the AV1 reference name `STD_VIDEO_AV1_REFERENCE_NAME_LAST_FRAME` (i.e. there is no bit or element for `STD_VIDEO_AV1_REFERENCE_NAME_INTRA_FRAME`).

`codedPictureAlignment` provides information about implementation limitations to encode arbitrary resolutions. In particular, some implementations **may** not be able to generate bitstreams aligned to the requirements of the AV1 Specification (8x8). In such cases, the implementation **may** override the width and height of the bitstream, in order to produce a bitstream compliant to the AV1 Specification. If such an override occurs, the encoded resolution of the coded picture is enlargened, with the texel values used for the texel coordinates outside of the bounds of the `codedExtent` of the encode input picture resource being first governed by the rules regarding the encode input picture granularity. Any texel values outside of the region described by the encode input picture granularity are implementation-defined. Implementations **should** use well-defined values to minimize impact on the produced encoded content.

NOTE This capability does not impose additional application requirements. However, these overrides change the effective resolution of the bitstream and add padding pixels. Applications sensitive to such overrides **can** use this capability and the corresponding override behavior to compute the cropping needed to reproduce the original input of the encoding and transmit it in a side channel (i.e. by using cropping fields available in a container). Additionally, applications **can** explicitly consider this alignment in their coded extent, to avoid implementation-defined texel values being included in the encoded content.

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1CapabilitiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_CAPABILITIES_KHR`

Bits which **may** be set in VkVideoEncodeAV1CapabilitiesKHR::`flags`, indicating the AV1 encoding capabilities supported, are:

```
// Provided by VK_KHR_video_encode_av1
typedef enum VkVideoEncodeAV1CapabilityFlagBitsKHR {
    VK_VIDEO_ENCODE_AV1_CAPABILITY_PER_RATE_CONTROL_GROUP_MIN_MAX_Q_INDEX_BIT_KHR =
0x00000001,
    VK_VIDEO_ENCODE_AV1_CAPABILITY_GENERATE_OBU_EXTENSION_HEADER_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_AV1_CAPABILITY_PRIMARY_REFERENCE_CDF_ONLY_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_AV1_CAPABILITY_FRAME_SIZE_OVERRIDE_BIT_KHR = 0x00000008,
    VK_VIDEO_ENCODE_AV1_CAPABILITY_MOTION_VECTOR_SCALING_BIT_KHR = 0x00000010,
} VkVideoEncodeAV1CapabilityFlagBitsKHR;
```

- VK_VIDEO_ENCODE_AV1_CAPABILITY_PER_RATE_CONTROL_GROUP_MIN_MAX_Q_INDEX_BIT_KHR indicates support for specifying different quantizer index values in the members of VkVideoEncodeAV1QIndexKHR.
- VK_VIDEO_ENCODE_AV1_CAPABILITY_GENERATE_OBU_EXTENSION_HEADER_BIT_KHR indicates support for generating OBU extension headers, as defined in section 5.3.3 of the AV1 Specification.
- VK_VIDEO_ENCODE_AV1_CAPABILITY_PRIMARY_REFERENCE_CDF_ONLY_BIT_KHR indicates support for using the primary reference frame indicated by the value of StdVideoEncodeAV1PictureInfo ::primary_ref_frame in the AV1 picture information only for CDF data reference, as defined in section 6.8.2 of the AV1 Specification.
- VK_VIDEO_ENCODE_AV1_CAPABILITY_FRAME_SIZE_OVERRIDE_BIT_KHR indicates support for encoding a picture with a frame size different from the maximum frame size defined in the active AV1 sequence header. If this capability is not supported, then frame_size_override_flag **must** not be set in the AV1 picture information of the encoded frame and the coded extent of the encode input picture **must** match the maximum coded extent allowed by the active AV1 sequence header, i.e. (max_frame_width_minus_1 + 1, max_frame_height_minus_1 + 1).
- VK_VIDEO_ENCODE_AV1_CAPABILITY_MOTION_VECTOR_SCALING_BIT_KHR indicates support for motion vector scaling, as defined in section 7.11.3.3 of the AV1 Specification. If this capability is not supported, then the coded extent of all active reference pictures **must** match the coded extent of the encode input picture. This capability **may** only be supported by a video profile when VK_VIDEO_ENCODE_AV1_CAPABILITY_FRAME_SIZE_OVERRIDE_BIT_KHR is also supported.

```
// Provided by VK_KHR_video_encode_av1
typedef VkFlags VkVideoEncodeAV1CapabilityFlagsKHR;
```

VkVideoEncodeAV1CapabilityFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeAV1CapabilityFlagBitsKHR.

Bits which **may** be set in VkVideoEncodeAV1CapabilitiesKHR::stdSyntaxFlags, indicating the capabilities related to the AV1 syntax elements, are:

```
// Provided by VK_KHR_video_encode_av1
typedef enum VkVideoEncodeAV1StdFlagBitsKHR {
    VK_VIDEO_ENCODE_AV1_STD_UNIFORM_TILE_SPACING_FLAG_SET_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_AV1_STD_SKIP_MODE_PRESENT_UNSET_BIT_KHR = 0x00000002,
    VK_VIDEO_ENCODE_AV1_STD_PRIMARY_REF_FRAME_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_AV1_STD_DELTA_Q_BIT_KHR = 0x00000008,
} VkVideoEncodeAV1StdFlagBitsKHR;
```

- VK_VIDEO_ENCODE_AV1_STD_UNIFORM_TILE_SPACING_FLAG_SET_BIT_KHR indicates whether the implementation supports using the application-provided value for StdVideoAV1TileInfoFlags::uniform_tile_spacing_flag in the AV1 tile parameters when that value is 1, indifferent of the coded extent of the encode input picture and the number of tile columns and rows requested in the TileCols and TileRows members of StdVideoAV1TileInfo.
- VK_VIDEO_ENCODE_AV1_STD_SKIP_MODE_PRESENT_UNSET_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeAV1PictureInfoFlags::skip_mode_present when that value is 0.
- VK_VIDEO_ENCODE_AV1_STD_PRIMARY_REF_FRAME_BIT_KHR specifies whether the implementation supports using the application-provided value for StdVideoEncodeAV1PictureInfo::primary_ref_frame.
- VK_VIDEO_ENCODE_AV1_STD_DELTA_Q_BIT_KHR specifies whether the implementation supports using the application-provided values for the DeltaQYDc, DeltaQUDc, DeltaQUAc, DeltaQVDc, and DeltaQVAc members of StdVideoAV1Quantization.

These capability flags provide information to the application about specific AV1 syntax element values that the implementation supports without having to override them and do not otherwise restrict the values that the application **can** specify for any of the mentioned AV1 syntax elements.

```
// Provided by VK_KHR_video_encode_av1
typedef VkFlags VkVideoEncodeAV1StdFlagsKHR;
```

VkVideoEncodeAV1StdFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeAV1StdFlagBitsKHR.

Bits which **may** be set in VkVideoEncodeAV1CapabilitiesKHR::superblockSizes, indicating the superblock sizes supported by the implementation, are:

```
// Provided by VK_KHR_video_encode_av1
typedef enum VkVideoEncodeAV1SuperblockSizeFlagBitsKHR {
    VK_VIDEO_ENCODE_AV1_SUPERBLOCK_SIZE_64_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_AV1_SUPERBLOCK_SIZE_128_BIT_KHR = 0x00000002,
} VkVideoEncodeAV1SuperblockSizeFlagBitsKHR;
```

- VK_VIDEO_ENCODE_AV1_SUPERBLOCK_SIZE_64_BIT_KHR specifies that a superblock size of 64x64 is supported.

- VK_VIDEO_ENCODE_AV1_SUPERBLOCK_SIZE_128_BIT_KHR specifies that a superblock size of 128x128 is supported.

```
// Provided by VK_KHR_video_encode_av1
typedef VkFlags VkVideoEncodeAV1SuperblockSizeFlagsKHR;
```

VkVideoEncodeAV1SuperblockSizeFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeAV1SuperblockSizeFlagBitsKHR.

Implementations **must** support at least one of VkVideoEncodeAV1SuperblockSizeFlagBitsKHR.

38.20.9. AV1 Encode Quality Level Properties

When calling vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR with pVideoProfile->videoCodecOperation specified as VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, the VkVideoEncodeAV1QualityLevelPropertiesKHR structure **must** be included in the pNext chain of the VkVideoEncodeQualityLevelPropertiesKHR structure to retrieve additional video encode quality level properties specific to AV1 encoding.

The VkVideoEncodeAV1QualityLevelPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1QualityLevelPropertiesKHR {
    VkStructureType                       sType;
    void*                                 pNext;
    VkVideoEncodeAV1RateControlFlagsKHR   preferredRateControlFlags;
    uint32_t                              preferredGopFrameCount;
    uint32_t                              preferredKeyFramePeriod;
    uint32_t                              preferredConsecutiveBipredictiveFrameCount;
    uint32_t                              preferredTemporalLayerCount;
    VkVideoEncodeAV1QIndexKHR             preferredConstantQIndex;
    uint32_t                              preferredMaxSingleReferenceCount;
    uint32_t                              preferredSingleReferenceNameMask;
    uint32_t
preferredMaxUnidirectionalCompoundReferenceCount;
    uint32_t
preferredMaxUnidirectionalCompoundGroup1ReferenceCount;
    uint32_t
preferredUnidirectionalCompoundReferenceNameMask;
    uint32_t
preferredMaxBidirectionalCompoundReferenceCount;
    uint32_t
preferredMaxBidirectionalCompoundGroup1ReferenceCount;
    uint32_t
preferredMaxBidirectionalCompoundGroup2ReferenceCount;
    uint32_t
preferredBidirectionalCompoundReferenceNameMask;
} VkVideoEncodeAV1QualityLevelPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `preferredRateControlFlags` is a bitmask of VkVideoEncodeAV1RateControlFlagBitsKHR values indicating the preferred flags to use for VkVideoEncodeAV1RateControlInfoKHR::`flags`.
- `preferredGopFrameCount` indicates the preferred value to use for VkVideoEncodeAV1RateControlInfoKHR::`gopFrameCount`.
- `preferredKeyFramePeriod` indicates the preferred value to use for VkVideoEncodeAV1RateControlInfoKHR::`keyFramePeriod`.
- `preferredConsecutiveBipredictiveFrameCount` indicates the preferred value to use for VkVideoEncodeAV1RateControlInfoKHR::`consecutiveBipredictiveFrameCount`.
- `preferredTemporalLayerCount` indicates the preferred value to use for VkVideoEncodeAV1RateControlInfoKHR::`temporalLayerCount`.
- `preferredConstantQIndex` indicates the preferred value to use for VkVideoEncodeAV1PictureInfoKHR::`constantQIndex` when using rate control mode `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`.
- `preferredMaxSingleReferenceCount` indicates the preferred maximum number of reference pictures to use with single reference prediction mode.
- `preferredSingleReferenceNameMask` is a bitmask of preferred AV1 reference names when using single reference prediction mode.
- `preferredMaxUnidirectionalCompoundReferenceCount` indicates the preferred maximum number of reference pictures to use with unidirectional compound prediction mode.
- `preferredMaxUnidirectionalCompoundGroup1ReferenceCount` indicates the preferred maximum number of reference pictures to use with unidirectional compound prediction mode from reference frame group 1, as defined in section 6.10.24 of the AV1 Specification.
- `preferredUnidirectionalCompoundReferenceNameMask` is a bitmask of preferred AV1 reference names when using unidirectional compound prediction mode.
- `preferredMaxBidirectionalCompoundReferenceCount` indicates the preferred maximum number of reference pictures to use with bidirectional compound prediction mode.
- `preferredMaxBidirectionalCompoundGroup1ReferenceCount` indicates the preferred maximum number of reference pictures to use with bidirectional compound prediction mode from reference frame group 1, as defined in section 6.10.24 of the AV1 Specification.
- `preferredMaxBidirectionalCompoundGroup2ReferenceCount` indicates the preferred maximum number of reference pictures to use with bidirectional compound prediction mode from reference frame group 2, as defined in section 6.10.24 of the AV1 Specification.
- `preferredBidirectionalCompoundReferenceNameMask` is a bitmask of preferred AV1 reference names when using bidirectional compound prediction mode.

`preferredSingleReferenceNameMask`, `preferredUnidirectionalCompoundReferenceNameMask`, and `preferredBidirectionalCompoundReferenceNameMask` are encoded such that when bit index i is set, it indicates preference for using the AV1 reference name `STD_VIDEO_AV1_REFERENCE_NAME_LAST_FRAME` + i.

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1QualityLevelPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_QUALITY_LEVEL_PROPERTIES_KHR`

38.20.10. AV1 Encode Session

Additional parameters can be specified when creating a video session with an AV1 encode profile by including an instance of the VkVideoEncodeAV1SessionCreateInfoKHR structure in the `pNext` chain of VkVideoSessionCreateInfoKHR.

The `VkVideoEncodeAV1SessionCreateInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1SessionCreateInfoKHR {
    VkStructureType     sType;
    const void*         pNext;
    VkBool32            useMaxLevel;
    StdVideoAV1Level    maxLevel;
} VkVideoEncodeAV1SessionCreateInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `useMaxLevel` indicates whether the value of `maxLevel` should be used by the implementation. When it is set to `VK_FALSE`, the implementation ignores the value of `maxLevel` and uses the value of VkVideoEncodeAV1CapabilitiesKHR::`maxLevel`, as reported by vkGetPhysicalDeviceVideoCapabilitiesKHR for the video profile.
- `maxLevel` is a `StdVideoAV1Level` value specifying the upper bound on the AV1 level for the video bitstreams produced by the created video session.

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1SessionCreateInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_SESSION_CREATE_INFO_KHR`

38.20.11. AV1 Encode Parameter Sets

Video session parameters objects created with the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR` contain a single instance of the following parameter set:

AV1 Sequence Header

Represented by `StdVideoAV1SequenceHeader` structures and interpreted as follows:

- `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
- the `StdVideoAV1ColorConfig` structure pointed to by `pColorConfig` is interpreted as follows:
  - `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
  - all other members of `StdVideoAV1ColorConfig` are interpreted as defined in section 6.4.2 of the AV1 Specification;
- if `flags.timing_info_present_flag` is set, then the `StdVideoAV1TimingInfo` structure pointed to by `pTimingInfo` is interpreted as follows:
  - `flags.reserved` is used only for padding purposes and is otherwise ignored;
  - all other members of `StdVideoAV1TimingInfo` are interpreted as defined in section 6.4.3 of the AV1 Specification;
- all other members of `StdVideoAV1SequenceHeader` are interpreted as defined in section 6.4 of the AV1 Specification.

When `StdVideoAV1SequenceHeader`::`flags.timing_info_present_flag` is set, the AV1 sequence header **can** be amended with AV1 decoder model information, represented by a `StdVideoEncodeAV1DecoderModelInfo` structure and interpreted as follows:

- `reserved1` is used only for padding purposes and is otherwise ignored;
- all other members of `StdVideoEncodeAV1DecoderModelInfo` are interpreted as defined in section 6.4.4 of the AV1 Specification.

When `StdVideoAV1SequenceHeader`::`flags.reduced_still_picture_header` is not set, the AV1 sequence header **can** be amended with AV1 operating point information, represented by an array of `StdVideoEncodeAV1OperatingPointInfo` structures and interpreted as follows:

- `flags.reserved` is used only for padding purposes and is otherwise ignored;
- all other members of `StdVideoEncodeAV1OperatingPointInfo` are interpreted as the corresponding element of the respective arrays defined in section 6.4 of the AV1 Specification.

Implementations **may** override any of these parameters according to the semantics defined in the Video Encode Parameter Overrides section before storing the resulting AV1 sequence header into the video session parameters object. Applications need to use the vkGetEncodedVideoSessionParametersKHR command to determine whether any implementation overrides happened and to retrieve the encoded AV1 sequence header in order to be able to produce a compliant AV1 video bitstream.

The encoded AV1 sequence header retrieved using the vkGetEncodedVideoSessionParametersKHR command is encoded as a single OBU with `obu_type` equal to `OBU_SEQUENCE_HEADER`, as defined in section 5.3 of the AV1 Specification.

Such AV1 sequence header overrides **may** also have cascading effects on the implementation overrides applied to the encoded bitstream produced by video encode operations. If the

implementation supports the VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR video encode feedback query flag, then the application **can** use such queries to retrieve feedback about whether any implementation overrides have been applied to the encoded bitstream.

When a video session parameters object is created with the codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, the VkVideoSessionParametersCreateInfoKHR::pNext chain **must** include a VkVideoEncodeAV1SessionParametersCreateInfoKHR structure specifying the contents of the object.

The VkVideoEncodeAV1SessionParametersCreateInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1SessionParametersCreateInfoKHR {
    VkStructureType                                 sType;
    const void*                                     pNext;
    const StdVideoAV1SequenceHeader*                pStdSequenceHeader;
    const StdVideoEncodeAV1DecoderModelInfo*        pStdDecoderModelInfo;
    uint32_t                                        stdOperatingPointCount;
    const StdVideoEncodeAV1OperatingPointInfo*      pStdOperatingPoints;
} VkVideoEncodeAV1SessionParametersCreateInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pStdSequenceHeader is a pointer to a StdVideoAV1SequenceHeader structure describing parameters of the AV1 sequence header entry to store in the created object.
- pStdDecoderModelInfo is NULL or a pointer to a StdVideoEncodeAV1DecoderModelInfo structure specifying the AV1 decoder model information to store in the created object.
- stdOperatingPointCount is the number of elements in the pStdOperatingPoints array.
- pStdOperatingPoints is NULL or a pointer to an array of stdOperatingPointCount number of StdVideoEncodeAV1OperatingPointInfo structures specifying the AV1 operating point information to store in the created object. Each element i specifies the parameter values corresponding to element i of the syntax elements defined in section 6.4 of the AV1 Specification.

Valid Usage

- VUID-VkVideoEncodeAV1SessionParametersCreateInfoKHR-pStdSequenceHeader-10288
  pStdSequenceHeader->flags.film_grain_params_present **must** be zero

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1SessionParametersCreateInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_SESSION_PARAMETERS_CREATE_INFO_KHR
- VUID-VkVideoEncodeAV1SessionParametersCreateInfoKHR-pStdSequenceHeader-parameter

pStdSequenceHeader **must** be a valid pointer to a valid StdVideoAV1SequenceHeader value

- VUID-VkVideoEncodeAV1SessionParametersCreateInfoKHR-pStdDecoderModelInfo-parameter
  If pStdDecoderModelInfo is not NULL, pStdDecoderModelInfo **must** be a valid pointer to a valid StdVideoEncodeAV1DecoderModelInfo value
- VUID-VkVideoEncodeAV1SessionParametersCreateInfoKHR-pStdOperatingPoints-parameter
  If stdOperatingPointCount is not 0, and pStdOperatingPoints is not NULL, pStdOperatingPoints **must** be a valid pointer to an array of stdOperatingPointCount StdVideoEncodeAV1OperatingPointInfo values

38.20.12. AV1 Encoding Parameters

The VkVideoEncodeAV1PictureInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1PictureInfoKHR {
    VkStructureType                         sType;
    const void*                             pNext;
    VkVideoEncodeAV1PredictionModeKHR       predictionMode;
    VkVideoEncodeAV1RateControlGroupKHR     rateControlGroup;
    uint32_t                                constantQIndex;
    const StdVideoEncodeAV1PictureInfo*     pStdPictureInfo;
    int32_t                                 referenceNameSlotIndices
[VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR];
    VkBool32                                primaryReferenceCdfOnly;
    VkBool32                                generateObuExtensionHeader;
} VkVideoEncodeAV1PictureInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- predictionMode specifies the AV1 prediction mode to use for the encoded frame.
- rateControlGroup specifies the AV1 rate control group to use for the encoded frame when the current rate control mode is not VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR. Otherwise it is ignored.
- constantQIndex is the quantizer index to use for the encoded frame if the current rate control mode configured for the video session is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR.
- pStdPictureInfo is a pointer to a StdVideoEncodeAV1PictureInfo structure specifying AV1 picture information.
- referenceNameSlotIndices is an array of seven (VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR, which is equal to the Video Std definition STD_VIDEO_AV1_REFS_PER_FRAME) signed integer values specifying the index of the DPB slot or a negative integer value for each AV1 reference name used for inter coding. In particular, the DPB slot index for the AV1 reference name frame is specified in referenceNameSlotIndices[frame - STD_VIDEO_AV1_REFERENCE_NAME_LAST_FRAME].

- `primaryReferenceCdfOnly` controls whether the primary reference frame indicated by the value of `pStdPictureInfo->primary_ref_frame` is used only for CDF data reference, as defined in sections 6.8.2 of the AV1 Specification. If set to `VK_TRUE`, then the primary reference frame's picture data will not be used for sample prediction.
- `generateObuExtensionHeader` controls whether OBU extension headers are generated into the target bitstream, as defined in sections 5.3.1, 5.3.2, and 5.3.3 of the AV1 Specification.

This structure is specified in the `pNext` chain of the VkVideoEncodeInfoKHR structure passed to vkCmdEncodeVideoKHR to specify the codec-specific picture information for an AV1 encode operation.

Encode Input Picture Information

When this structure is specified in the `pNext` chain of the VkVideoEncodeInfoKHR structure passed to vkCmdEncodeVideoKHR, the information related to the encode input picture is defined as follows:

- The image subregion used is determined according to the AV1 Encode Picture Data Access section.
- The encode input picture is associated with the AV1 picture information provided in `pStdPictureInfo`.

Std Picture Information

The members of the `StdVideoEncodeAV1PictureInfo` structure pointed to by `pStdPictureInfo` are interpreted as follows:

- `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
- `pSegmentation` **must** be `NULL`

> **NOTE** AV1 segmentation is currently not supported in video encode operations. Accordingly, the application needs to set `flags.segmentation_enabled` to `0` and `pSegmentation` to `NULL`.

- `pTileInfo` is `NULL` or a pointer to a `StdVideoAV1TileInfo` structure specifying AV1 tile parameters;
- the `StdVideoAV1Quantization` structure pointed to by `pQuantization` is interpreted as follows:
  - `flags.reserved` is used only for padding purposes and is otherwise ignored;
  - all other members of `StdVideoAV1Quantization` are interpreted as defined in section 6.8.11 of the AV1 Specification;
- the `StdVideoAV1LoopFilter` structure pointed to by `pLoopFilter` is interpreted as follows:
  - `flags.reserved` is used only for padding purposes and is otherwise ignored;
  - `update_ref_delta` is a bitmask where bit index i is interpreted as the value of `update_ref_delta` corresponding to element i of `loop_filter_ref_deltas` as defined in section 6.8.10 of the AV1 Specification;
  - `update_mode_delta` is a bitmask where bit index i is interpreted as the value of `update_mode_delta` corresponding to element i of `loop_filter_mode_deltas` as defined in

section 6.8.10 of the AV1 Specification;

  - all other members of StdVideoAV1LoopFilter are interpreted as defined in section 6.8.10 of the AV1 Specification;

- if flags.enable_cdef is set in the active sequence header, then the members of the StdVideoAV1CDEF structure pointed to by pCDEF are interpreted as follows:

  - cdef_y_sec_strength and cdef_uv_sec_strength are the bitstream values of the corresponding syntax elements defined in section 5.9.19 of the AV1 Specification;

  - all other members of StdVideoAV1CDEF are interpreted as defined in section 6.10.14 of the AV1 Specification;

- if flags.UsesLr is set in the active sequence header, then the StdVideoAV1LoopRestoration structure pointed to by pLoopRestoration is interpreted as follows:

  - LoopRestorationSize[plane] is interpreted as log2(size) - 5, where size is the value of LoopRestorationSize[plane] as defined in section 6.10.15 of the AV1 Specification;

  - all other members of StdVideoAV1LoopRestoration are defined as in section 6.10.15 of the AV1 Specification;

- the members of the StdVideoAV1GlobalMotion structure provided in global_motion are interpreted as defined in section 7.10 of the AV1 Specification;

- pExtensionHeader is NULL or a pointer to a StdVideoEncodeAV1ExtensionHeader structure whose temporal_id and spatial_id members specify the temporal and spatial layer ID of the reference frame, respectively (these IDs are encoded into the OBU extension header if VkVideoEncodeAV1PictureInfoKHR::generateObuExtensionHeader is set to VK_TRUE for the encode operation);

- if flags.buffer_removal_time_present_flag is set, then pBufferRemovalTimes is a pointer to an array of N number of unsigned integer values specifying the elements of the buffer_removal_time array, as defined in section 6.8.2 of the AV1 Specification, where N is the number of operating points specified for the active sequence header through VkVideoEncodeAV1SessionParametersCreateInfoKHR::stdOperatingPointCount;

- all other members are interpreted as defined in section 6.8 of the AV1 Specification.

Reference picture setup is controlled by the value of StdVideoEncodeAV1PictureInfo::refresh_frame_flags. If it is not zero and a reconstructed picture is specified, then the latter is used as the target of picture reconstruction to activate the DPB slot specified in pEncodeInfo->pSetupReferenceSlot->slotIndex. If StdVideoEncodeAV1PictureInfo::refresh_frame_flags is zero, but a reconstructed picture is specified, then the corresponding picture reference associated with the DPB slot is invalidated, as described in the DPB Slot States section.

Std Tile Parameters

Specifying AV1 tile parameters is optional. If StdVideoEncodeAV1PictureInfo::pTileInfo is NULL, then the implementation determines the values of AV1 tile parameters defined in section 6.8.14 of the AV1 Specification in an implementation-dependent manner. If StdVideoEncodeAV1PictureInfo::pTileInfo is not NULL, then the members of the StdVideoAV1TileInfo structure pointed to by StdVideoEncodeAV1PictureInfo::pTileInfo are interpreted as follows:

- `flags.reserved` and `reserved1` are used only for padding purposes and are otherwise ignored;
- `TileCols` and `TileRows` specify the number of tile columns and tile rows as defined in section 6.8.14 of the AV1 Specification;
- `tile_size_bytes_minus_1` is ignored, as its value, as defined in section 6.8.14 of the AV1 Specification, is determined as the result of the encoding process;
- `pMiColStarts` and `pMiRowStarts` are ignored, as the elements of the `MiColStarts` and `MiRowStarts` arrays defined in section 6.8.14 of the AV1 Specification are determined by the implementation based on the tile widths and heights determined by the implementation or specified through the `pWidthInSbsMinus1` and `pHeightInSbsMinus1` arrays, respectively;
- `pWidthInSbsMinus1` is `NULL` or a pointer to an array of `TileCols` number of unsigned integers that corresponds to `width_in_sbs_minus_1` defined in section 6.8.14 of the AV1 Specification;
- `pHeightInSbsMinus1` is `NULL` or is a pointer to an array of `TileRows` number of unsigned integers that corresponds to `height_in_sbs_minus_1` defined in section 6.8.14 of the AV1 Specification;
- all other members of `StdVideoAV1TileInfo` are interpreted as defined in section 6.8.14 of the AV1 Specification.

If `flags.uniform_tile_spacing_flag` is set, then `pWidthInSbsMinus1` and `pHeightInSbsMinus1` are ignored.

If `flags.uniform_tile_spacing_flag` is not set and `pWidthInSbsMinus1` is `NULL`, then the width of individual tile columns is determined in an implementation-dependent manner.

If `flags.uniform_tile_spacing_flag` is not set and `pHeightInSbsMinus1` is `NULL`, then the height of individual tile rows is determined in an implementation-dependent manner.

NOTE

In general, implementations are expected to respect the application-specified AV1 tile parameters. However, as implementations may have restrictions on the combination of tile column and row counts, and tile widths and heights with respect to the extent of the encoded frame beyond the restrictions specified in the AV1 Specification and this specification (through video profile capabilities), certain parameter combinations may require the implementation to override them in order to conform to such implementation-specific limitations.

Active Parameter Sets

The *active sequence header* is the AV1 sequence header stored in the bound video session parameters object.

Valid Usage

- VUID-VkVideoEncodeAV1PictureInfoKHR-flags-10289
  If VkVideoEncodeAV1CapabilitiesKHR::`flags`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include `VK_VIDEO_ENCODE_AV1_CAPABILITY_PRIMARY_REFERENCE_CDF_ONLY_BIT_KHR`, then `primaryReferenceCdfOnly` **must** be `VK_FALSE`
- VUID-VkVideoEncodeAV1PictureInfoKHR-primaryReferenceCdfOnly-10290

If primaryReferenceCdfOnly is set to VK_TRUE, then pStdPictureInfo->primary_ref_frame **must** be less than VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR

- VUID-VkVideoEncodeAV1PictureInfoKHR-pStdPictureInfo-10291
  If pStdPictureInfo->primary_ref_frame is less than VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR, then referenceNameSlotIndices [pStdPictureInfo->primary_ref_frame] **must** not be negative
- VUID-VkVideoEncodeAV1PictureInfoKHR-flags-10292
  If VkVideoEncodeAV1CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_AV1_CAPABILITY_GENERATE_OBU_EXTENSION_HEADER_BIT_KHR, then generateObuExtensionHeader **must** be VK_FALSE
- VUID-VkVideoEncodeAV1PictureInfoKHR-generateObuExtensionHeader-10293
  If generateObuExtensionHeader is set to VK_TRUE, then pStdPictureInfo->pExtensionHeader **must** not be NULL

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1PictureInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_PICTURE_INFO_KHR
- VUID-VkVideoEncodeAV1PictureInfoKHR-predictionMode-parameter
  predictionMode **must** be a valid VkVideoEncodeAV1PredictionModeKHR value
- VUID-VkVideoEncodeAV1PictureInfoKHR-rateControlGroup-parameter
  rateControlGroup **must** be a valid VkVideoEncodeAV1RateControlGroupKHR value
- VUID-VkVideoEncodeAV1PictureInfoKHR-pStdPictureInfo-parameter
  pStdPictureInfo **must** be a valid pointer to a valid StdVideoEncodeAV1PictureInfo value

The VkVideoEncodeAV1DpbSlotInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1DpbSlotInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    const StdVideoEncodeAV1ReferenceInfo*    pStdReferenceInfo;
} VkVideoEncodeAV1DpbSlotInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pStdReferenceInfo is a pointer to a StdVideoEncodeAV1ReferenceInfo structure specifying AV1 reference information.

This structure is specified in the pNext chain of VkVideoEncodeInfoKHR::pSetupReferenceSlot, if not NULL, and the pNext chain of the elements of VkVideoEncodeInfoKHR::pReferenceSlots to specify the codec-specific reference picture information for an AV1 encode operation.

Active Reference Picture Information

When this structure is specified in the pNext chain of the elements of VkVideoEncodeInfoKHR::pReferenceSlots, one element is added to the list of active reference pictures used by the video encode operation for each element of VkVideoEncodeInfoKHR::pReferenceSlots as follows:

- The image subregion used is determined according to the AV1 Encode Picture Data Access section.
- The reference picture is associated with the DPB slot index specified in the slotIndex member of the corresponding element of VkVideoEncodeInfoKHR::pReferenceSlots.
- The reference picture is associated with the AV1 reference information provided in pStdReferenceInfo.

Reconstructed Picture Information

When this structure is specified in the pNext chain of VkVideoEncodeInfoKHR::pSetupReferenceSlot, the information related to the reconstructed picture is defined as follows:

- The image subregion used is determined according to the AV1 Encode Picture Data Access section.
- If reference picture setup is requested, then the reconstructed picture is used to activate the DPB slot with the index specified in VkVideoEncodeInfoKHR::pSetupReferenceSlot->slotIndex.
- The reconstructed picture is associated with the AV1 reference information provided in pStdReferenceInfo.

Std Reference Information

The members of the StdVideoEncodeAV1ReferenceInfo structure pointed to by pStdReferenceInfo are interpreted as follows:

- flags.reserved and reserved1 are used only for padding purposes and are otherwise ignored;
- flags.disable_frame_end_update_cdf is interpreted as defined in section 6.8.2 of the AV1 Specification;
- flags.segmentation_enabled is interpreted as defined in section 6.8.13 of the AV1 Specification;
- RefFrameId is interpreted as the element of the RefFrameId array defined in section 6.8.2 of the AV1 Specification corresponding to the reference frame;
- frame_type is interpreted as defined in section 6.8.2 of the AV1 Specification;
- OrderHint is interpreted as defined in section 6.8.2 of the AV1 Specification;
- pExtensionHeader is NULL or a pointer to a StdVideoEncodeAV1ExtensionHeader structure whose temporal_id and spatial_id members specify the temporal and spatial layer ID of the reference frame, respectively.

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1DpbSlotInfoKHR-sType-sType

sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_DPB_SLOT_INFO_KHR

- VUID-VkVideoEncodeAV1DpbSlotInfoKHR-pStdReferenceInfo-parameter pStdReferenceInfo **must** be a valid pointer to a valid StdVideoEncodeAV1ReferenceInfo value

38.20.13. AV1 Encode Rate Control

Group of Pictures

In case of AV1 encoding it is common practice to follow a regular pattern of frame types and prediction directions in display order when encoding subsequent frames. This pattern is referred to as the *group of pictures* (GOP).

The AV1 Specification, unlike some other video compression standards, does not restrict the direction in display order of the referenced frames based on the used frame type or AV1 prediction mode. Accordingly, this specification introduces the concept of *rate control groups* for which the application **can** specify separate rate control configuration parameters. When encoding a frame, the application specifies the rate control group the encoded frame belongs to through a VkVideoEncodeAV1RateControlGroupKHR value in VkVideoEncodeAV1PictureInfoKHR::rateControlGroup. This value is then used by the implementation's rate control algorithm to determine which rate control configuration parameters apply to it.

Possible AV1 encode rate control groups are as follows:

```
// Provided by VK_KHR_video_encode_av1
typedef enum VkVideoEncodeAV1RateControlGroupKHR {
    VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_INTRA_KHR = 0,
    VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_PREDICTIVE_KHR = 1,
    VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR = 2,
} VkVideoEncodeAV1RateControlGroupKHR;
```

- VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_INTRA_KHR **should** be specified when encoding AV1 frames that use intra-only prediction (e.g. when encoding AV1 frames of type STD_VIDEO_AV1_FRAME_TYPE_KEY or STD_VIDEO_AV1_FRAME_TYPE_INTRA_ONLY).
- VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_PREDICTIVE_KHR **should** be specified when encoding AV1 frames that only have forward references in display order.
- VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR **should** be specified when encoding AV1 frames that have backward references in display order.

NOTE While the application can specify any rate control group for any frame, indifferent of the frame type, prediction mode, or prediction direction, specifying a rate control group that does not reflect the prediction direction used by the encoded frame may result in unexpected behavior of the implementation's rate control algorithm.

A regular GOP is defined by the following parameters:

- The number of frames in the GOP;

- The number of consecutive frames encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR` between frames encoded with other rate control groups in display order.

GOPs are further classified as *open* and *closed* GOPs.

Frame types in an open GOP follow each other in display order according to the following algorithm:

1. The first frame is always a frame encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_INTRA_KHR`.
2. This is followed by a number of consecutive frames encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR`.
3. If the number of frames in the GOP is not reached yet, then the next frame is a frame encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_PREDICTIVE_KHR` and the algorithm continues from step 2.

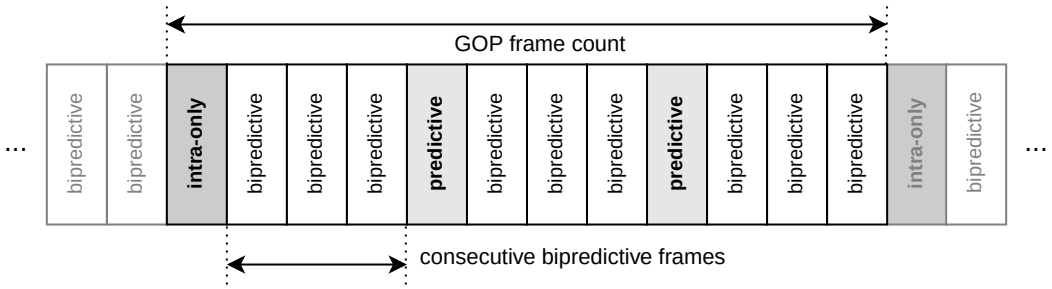


Figure 36. AV1 open GOP

In case of a closed GOP, a frame with the AV1 frame type `STD_VIDEO_AV1_FRAME_TYPE_KEY` is used at a certain period.

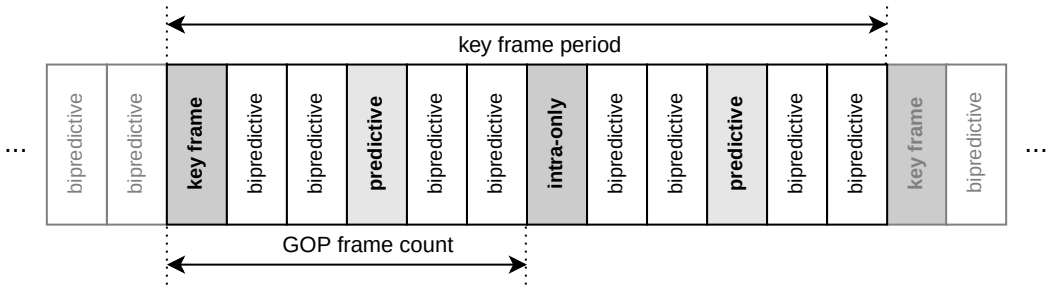


Figure 37. AV1 closed GOP

It is also typical for AV1 encoding to use specific reference picture usage patterns across the frames

of the GOP. The two most common reference patterns used are as follows:

Flat Reference Pattern

- Each frame encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_PREDICTIVE_KHR` refers to the last frame that was not encoded using `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR`, in display order, as its forward reference.
- Each frame encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR` refers to the last frame that was not encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR`, in display order, as its forward reference, and refers to the next frame that was not encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR`, in display order, as its backward reference.

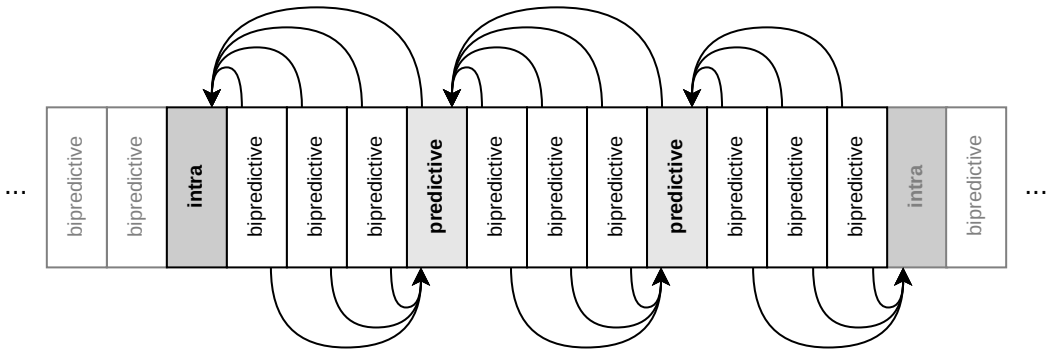


Figure 38. AV1 flat reference pattern

Dyadic Reference Pattern

- Each frame encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_PREDICTIVE_KHR` refers to the last frame that was not encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR`, in display order, as its forward reference.
- The following algorithm is applied to the sequence of consecutive frames encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR` between frames using other rate control groups in display order:
  1. The frame in the middle of this sequence uses the frame preceding the sequence as its forward reference, and uses the frame following the sequence as its backward reference.
  2. The algorithm is executed recursively for the following frame sequences:
     - The frames of the original sequence preceding the frame in the middle, if any.
     - The frames of the original sequence following the frame in the middle, if any.

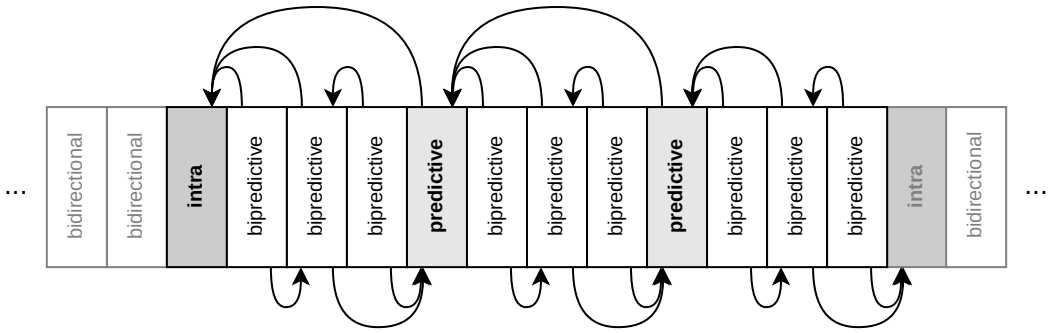


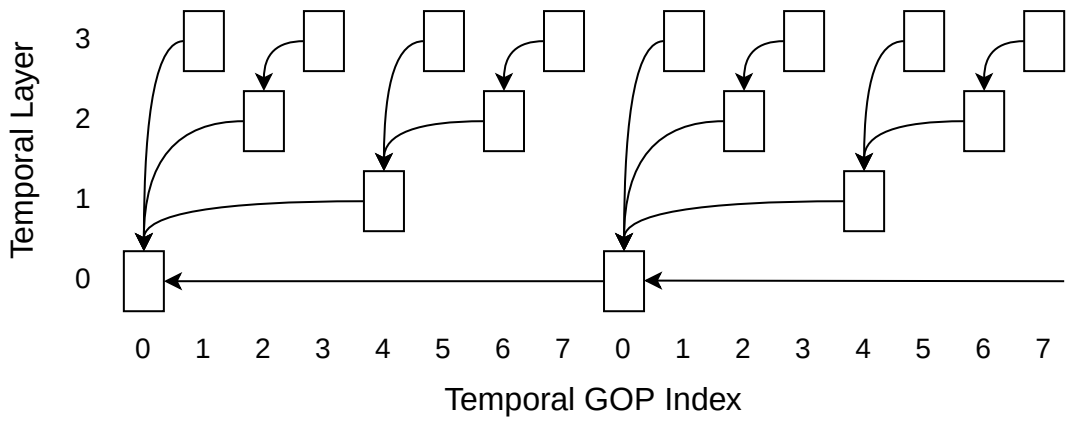
Figure 39. AV1 dyadic reference pattern

The application **can** provide guidance to the implementation's rate control algorithm about the structure of the GOP used by the application. Any such guidance about the GOP and its structure does not mandate that specific GOP structure to be used by the application, as the frame type and the selected rate control group is still application-controlled, however, any deviation from the provided guidance **may** result in undesired rate control behavior including, but not limited, to the implementation not being able to conform to the expected average or target bitrates, or other rate control parameters specified by the application.

When an AV1 encode session is used to encode multiple temporal layers, it is also common practice to follow a regular pattern for the AV1 temporal ID for the encoded frames in display order when encoding subsequent frames. This pattern is referred to as the *temporal GOP*. The most common temporal layer pattern used is as follows:

Dyadic Temporal Layer Pattern

- The number of frames in the temporal GOP is 2^{n-1}, where n is the number of temporal layers.
- The i^{th} frame in the temporal GOP uses temporal ID t, if and only if the index of the least significant bit set in i equals n-t-1, except for the first frame, which is the only frame in the temporal GOP using temporal ID zero.
- The i^{th} frame in the temporal GOP uses the r^{th} frame as reference, where r is calculated from i by clearing the least significant bit set in it, except for the first frame in the temporal GOP, which uses the first frame of the previous temporal GOP, if any, as reference.



Text is not SVG - cannot display

Figure 40. AV1 dyadic temporal layer pattern

NOTE Multi-layer rate control and multi-layer coding are typically used for streaming cases where low latency is expected, hence frames usually do not use backward references in display order.

The `VkVideoEncodeAV1RateControlInfoKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1RateControlInfoKHR {
    VkStructureType                          sType;
    const void*                              pNext;
    VkVideoEncodeAV1RateControlFlagsKHR      flags;
    uint32_t                                 gopFrameCount;
    uint32_t                                 keyFramePeriod;
    uint32_t                                 consecutiveBipredictiveFrameCount;
    uint32_t                                 temporalLayerCount;
} VkVideoEncodeAV1RateControlInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is a bitmask of VkVideoEncodeAV1RateControlFlagBitsKHR specifying AV1 rate control flags.
- `gopFrameCount` is the number of frames within a group of pictures (GOP) intended to be used by the application. If it is set to 0, the rate control algorithm **may** assume an implementation-dependent GOP length. If it is set to `UINT32_MAX`, the GOP length is treated as infinite.
- `keyFramePeriod` is the interval, in terms of number of frames, between two frames with the AV1 frame type `STD_VIDEO_AV1_FRAME_TYPE_KEY` (see key frame period). If it is set to 0, the rate control algorithm **may** assume an implementation-dependent key frame period. If it is set to `UINT32_MAX`, the key frame period is treated as infinite.

- `consecutiveBipredictiveFrameCount` is the number of consecutive frames encoded with `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR` between frames encoded with other rate control groups within the GOP.
- `temporalLayerCount` specifies the number of AV1 temporal layers that the application intends to use.

When an instance of this structure is included in the `pNext` chain of the VkVideoCodingControlInfoKHR structure passed to the vkCmdControlVideoCodingKHR command, and VkVideoCodingControlInfoKHR::`flags` includes `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR`, the parameters in this structure are used as guidance for the implementation's rate control algorithm (see Video Coding Control).

Valid Usage

- VUID-VkVideoEncodeAV1RateControlInfoKHR-flags-10294
  If `flags` contains `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR` or `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR`, then it **must** also contain `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REGULAR_GOP_BIT_KHR`
- VUID-VkVideoEncodeAV1RateControlInfoKHR-flags-10295
  If `flags` contains `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR`, then it **must** not also contain `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR`
- VUID-VkVideoEncodeAV1RateControlInfoKHR-flags-10296
  If `flags` contains `VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REGULAR_GOP_BIT_KHR`, then `gopFrameCount` **must** be greater than `0`
- VUID-VkVideoEncodeAV1RateControlInfoKHR-keyFramePeriod-10297
  If `keyFramePeriod` is not `0`, then it **must** be greater than or equal to `gopFrameCount`
- VUID-VkVideoEncodeAV1RateControlInfoKHR-consecutiveBipredictiveFrameCount-10298
  If `consecutiveBipredictiveFrameCount` is not `0`, then it **must** be less than `gopFrameCount`
- VUID-VkVideoEncodeAV1RateControlInfoKHR-temporalLayerCount-10299
  `temporalLayerCount` **must** be less than or equal to VkVideoEncodeAV1CapabilitiesKHR::`maxTemporalLayerCount`, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1RateControlInfoKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_RATE_CONTROL_INFO_KHR`
- VUID-VkVideoEncodeAV1RateControlInfoKHR-flags-parameter
  `flags` **must** be a valid combination of VkVideoEncodeAV1RateControlFlagBitsKHR values

Bits which **can** be set in VkVideoEncodeAV1RateControlInfoKHR::`flags`, specifying AV1 rate control flags, are:

```
// Provided by VK_KHR_video_encode_av1
typedef enum VkVideoEncodeAV1RateControlFlagBitsKHR {
    VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REGULAR_GOP_BIT_KHR = 0x00000001,
    VK_VIDEO_ENCODE_AV1_RATE_CONTROL_TEMPORAL_LAYER_PATTERN_DYADIC_BIT_KHR =
0x00000002,
    VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR = 0x00000004,
    VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR = 0x00000008,
} VkVideoEncodeAV1RateControlFlagBitsKHR;
```

- VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REGULAR_GOP_BIT_KHR specifies that the application intends to use a regular GOP structure according to the parameters specified in the gopFrameCount and keyFramePeriod members of the VkVideoEncodeAV1RateControlInfoKHR structure.
- VK_VIDEO_ENCODE_AV1_RATE_CONTROL_TEMPORAL_LAYER_PATTERN_DYADIC_BIT_KHR specifies that the application intends to follow a dyadic temporal layer pattern.
- VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REFERENCE_PATTERN_FLAT_BIT_KHR specifies that the application intends to follow a flat reference pattern in the GOP.
- VK_VIDEO_ENCODE_AV1_RATE_CONTROL_REFERENCE_PATTERN_DYADIC_BIT_KHR specifies that the application intends to follow a dyadic reference pattern in the GOP.

```
// Provided by VK_KHR_video_encode_av1
typedef VkFlags VkVideoEncodeAV1RateControlFlagsKHR;
```

VkVideoEncodeAV1RateControlFlagsKHR is a bitmask type for setting a mask of zero or more VkVideoEncodeAV1RateControlFlagBitsKHR.

Rate Control Layers

The VkVideoEncodeAV1RateControlLayerInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1RateControlLayerInfoKHR {
    VkStructureType                 sType;
    const void*                     pNext;
    VkBool32                        useMinQIndex;
    VkVideoEncodeAV1QIndexKHR       minQIndex;
    VkBool32                        useMaxQIndex;
    VkVideoEncodeAV1QIndexKHR       maxQIndex;
    VkBool32                        useMaxFrameSize;
    VkVideoEncodeAV1FrameSizeKHR    maxFrameSize;
} VkVideoEncodeAV1RateControlLayerInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- useMinQIndex indicates whether the quantizer index values determined by rate control will be

clamped to the lower bounds on the quantizer index values specified in minQIndex.

- minQIndex specifies the lower bounds on the quantizer index values, for each rate control group, that the implementation's rate control algorithm will use when useMinQIndex is set to VK_TRUE.
- useMaxQIndex indicates whether the quantizer index values determined by rate control will be clamped to the upper bounds on the quantizer index values specified in maxQIndex.
- maxQIndex specifies the upper bounds on the quantizer index values, for each rate control group, that the implementation's rate control algorithm will use when useMaxQIndex is set to VK_TRUE.
- useMaxFrameSize indicates whether the implementation's rate control algorithm **should** use the values specified in maxFrameSize as the upper bounds on the encoded frame size for each rate control group.
- maxFrameSize specifies the upper bounds on the encoded frame size, for each rate control group, when useMaxFrameSize is set to VK_TRUE.

When used, the values in minQIndex and maxQIndex guarantee that the effective quantizer index values used by the implementation will respect those lower and upper bounds, respectively. However, limiting the range of quantizer index values that the implementation is able to use will also limit the capabilities of the implementation's rate control algorithm to comply to other constraints. In particular, the implementation **may** not be able to comply to the following:

- The average and/or peak bitrate values to be used for the encoded bitstream specified in the averageBitrate and maxBitrate members of the VkVideoEncodeRateControlLayerInfoKHR structure.
- The upper bounds on the encoded frame size, for each rate control group, specified in the maxFrameSize member of VkVideoEncodeAV1RateControlLayerInfoKHR.

NOTE	In general, applications need to configure rate control parameters appropriately in order to be able to get the desired rate control behavior, as described in the Video Encode Rate Control section.

When an instance of this structure is included in the pNext chain of a VkVideoEncodeRateControlLayerInfoKHR structure specified in one of the elements of the pLayers array member of the VkVideoEncodeRateControlInfoKHR structure passed to the vkCmdControlVideoCodingKHR command, VkVideoCodingControlInfoKHR::flags includes VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR, and the bound video session was created with the video codec operation VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR, it specifies the AV1-specific rate control parameters of the rate control layer corresponding to that element of pLayers.

Valid Usage

- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-useMinQIndex-10300
  If useMinQIndex is VK_TRUE, then the intraQIndex, predictiveQIndex, and bipredictiveQIndex members of minQIndex **must** all be between VkVideoEncodeAV1CapabilitiesKHR::minQIndex and VkVideoEncodeAV1CapabilitiesKHR::maxQIndex, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-useMinQIndex-10301

If useMinQIndex is VK_TRUE and VkVideoEncodeAV1CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_AV1_CAPABILITY_PER_RATE_CONTROL_GROUP_MIN_MAX_Q_INDEX_BIT_KHR, then the intraQIndex, predictiveQIndex, and bipredictiveQIndex members of minQIndex **must** all specify the same value

- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-useMaxQIndex-10302
  If useMaxQIndex is VK_TRUE, then the intraQIndex, predictiveQIndex, and bipredictiveQIndex members of maxQIndex **must** all be between VkVideoEncodeAV1CapabilitiesKHR::minQIndex and VkVideoEncodeAV1CapabilitiesKHR::maxQIndex, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile
- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-useMaxQIndex-10303
  If useMaxQIndex is VK_TRUE and VkVideoEncodeAV1CapabilitiesKHR::flags, as returned by vkGetPhysicalDeviceVideoCapabilitiesKHR for the used video profile, does not include VK_VIDEO_ENCODE_AV1_CAPABILITY_PER_RATE_CONTROL_GROUP_MIN_MAX_Q_INDEX_BIT_KHR, then the intraQIndex, predictiveQIndex, and bipredictiveQIndex members of maxQIndex **must** all specify the same value
- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-useMinQIndex-10304
  If useMinQIndex and useMaxQIndex are both VK_TRUE, then the intraQIndex, predictiveQIndex, and bipredictiveQIndex members of minQIndex **must** all be less than or equal to the respective members of maxQIndex

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_RATE_CONTROL_LAYER_INFO_KHR
- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-minQIndex-parameter
  minQIndex **must** be a valid VkVideoEncodeAV1QIndexKHR structure
- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-maxQIndex-parameter
  maxQIndex **must** be a valid VkVideoEncodeAV1QIndexKHR structure
- VUID-VkVideoEncodeAV1RateControlLayerInfoKHR-maxFrameSize-parameter
  maxFrameSize **must** be a valid VkVideoEncodeAV1FrameSizeKHR structure

The VkVideoEncodeAV1QIndexKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1QIndexKHR {
    uint32_t    intraQIndex;
    uint32_t    predictiveQIndex;
    uint32_t    bipredictiveQIndex;
} VkVideoEncodeAV1QIndexKHR;
```

- intraQIndex is the quantizer index to be used for frames encoded with VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_INTRA_KHR.

- predictiveQIndex is the quantizer index to be used for frames encoded with VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_PREDICTIVE_KHR.
- bipredictiveQIndex is the quantizer index to be used for frames encoded with VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR.

The VkVideoEncodeAV1FrameSizeKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1FrameSizeKHR {
    uint32_t    intraFrameSize;
    uint32_t    predictiveFrameSize;
    uint32_t    bipredictiveFrameSize;
} VkVideoEncodeAV1FrameSizeKHR;
```

- intraFrameSize is the size in bytes to be used for frames encoded with VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_INTRA_KHR.
- predictiveFrameSize is the size in bytes to be used for frames encoded with VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_PREDICTIVE_KHR.
- bipredictiveFrameSize is the size in bytes to be used for frames encoded with VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR.

GOP Remaining Frames

Besides session level rate control configuration, the application **can** specify the number of frames per frame type remaining in the group of pictures (GOP).

The VkVideoEncodeAV1GopRemainingFrameInfoKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkVideoEncodeAV1GopRemainingFrameInfoKHR {
    VkStructureType    sType;
    const void*        pNext;
    VkBool32           useGopRemainingFrames;
    uint32_t           gopRemainingIntra;
    uint32_t           gopRemainingPredictive;
    uint32_t           gopRemainingBipredictive;
} VkVideoEncodeAV1GopRemainingFrameInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- useGopRemainingFrames indicates whether the implementation's rate control algorithm **should** use the values specified in gopRemainingIntra, gopRemainingPredictive, and gopRemainingBipredictive. If useGopRemainingFrames is VK_FALSE, then the values of gopRemainingIntra, gopRemainingPredictive, and gopRemainingBipredictive are ignored.
- gopRemainingIntra specifies the number of frames encoded with

VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_INTRA_KHR the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the next video encode operation.

- gopRemainingPredictive specifies the number of frames encoded with VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_PREDICTIVE_KHR the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the next video encode operation.
- gopRemainingBipredictive specifies the number of frames encoded with VK_VIDEO_ENCODE_AV1_RATE_CONTROL_GROUP_BIPREDICTIVE_KHR the implementation's rate control algorithm **should** assume to be remaining in the GOP prior to executing the next video encode operation.

Setting useGopRemainingFrames to VK_TRUE and including this structure in the pNext chain of VkVideoBeginCodingInfoKHR is only mandatory if the VkVideoEncodeAV1CapabilitiesKHR ::requiresGopRemainingFrames reported for the used video profile is VK_TRUE. However, implementations **may** use these remaining frame counts, when specified, even when it is not required. In particular, when the application does not use a regular GOP structure, these values **may** provide additional guidance for the implementation's rate control algorithm.

The VkVideoEncodeAV1CapabilitiesKHR::prefersGopRemainingFrames capability is also used to indicate that the implementation's rate control algorithm **may** operate more accurately if the application specifies the remaining frame counts using this structure.

As with other rate control guidance values, if the effective order and number of frames encoded by the application are not in line with the remaining frame counts specified in this structure at any given point, then the behavior of the implementation's rate control algorithm **may** deviate from the one expected by the application.

Valid Usage (Implicit)

- VUID-VkVideoEncodeAV1GopRemainingFrameInfoKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_GOP_REMAINING_FRAME_INFO_KHR

AV1 Quantizer Index Delta Maps

Quantization delta maps used with an AV1 encode profile are referred to as *quantizer index delta maps* and their texels contain integer values representing quantizer index delta values that are applied in the process of determining the quantizer indices of the encoded picture.

Accordingly, AV1 quantizer index delta maps always have single channel integer formats, as reported in VkVideoFormatPropertiesKHR::format.

When the rate control mode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR, the quantizer index delta values are added to the constant quantizer index value that, in effect, enable the application to explicitly control the used quantizer index values at the granularity of the used quantization map texel size.

For all other rate control modes, the quantizer index delta values **can** be used to offset the

quantizer index values that the rate control algorithm would otherwise produce.

AV1 Encode Quantization

Performing AV1 encode operations involves the process of assigning quantizer index values to individual AV1 mode info blocks. This process depends on the used rate control mode, as well as other encode and rate control parameters, as described below:

- If the configured rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR`, then the quantizer index value is initialized by the implementation-specific default rate control algorithm.
  - If the video encode operation is issued with a quantization delta map, the quantizer index delta value corresponding to the mode info block, as fetched from the quantization map, is added to the previously determined quantizer index value. If the fetched quantizer index delta value falls outside the supported quantizer index delta value range reported in the `minQIndexDelta` and `maxQIndexDelta` members of VkVideoEncodeAV1QuantizationMapCapabilitiesKHR, then the quantizer index value used for the mode info block becomes undefined.
- If the configured rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`, then the quantizer index value is initialized from the constant quantizer index value specified for the encoded frame.
  - If the video encode operation is issued with a quantization delta map, the quantizer index delta value corresponding to the mode info block, as fetched from the quantization map, is added to the previously determined quantizer index value. If the fetched quantizer index delta value falls outside the supported quantizer index delta value range reported in the `minQIndexDelta` and `maxQIndexDelta` members of VkVideoEncodeAV1QuantizationMapCapabilitiesKHR, then the quantizer index value used for the mode info block becomes undefined.
- If the configured rate control mode is not `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR` or `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`, then the quantizer index value is initialized by the corresponding rate control algorithm.
  - If the video encode operation is issued with a quantization delta map, the quantizer index delta value corresponding to the mode info block, as fetched from the quantization map, is added to the previously determined quantizer index value. If the fetched quantizer index delta value falls outside the supported quantizer index delta value range reported in the `minQIndexDelta` and `maxQIndexDelta` members of VkVideoEncodeAV1QuantizationMapCapabilitiesKHR, then the quantizer index value used for the mode info block becomes undefined.
  - If the video encode operation is issued with an emphasis map, the rate control will adjust the quantizer index value based on the emphasis value corresponding to the mode info block, as fetched from the quantization map, according to the following equation:

    $QIndex_{new} = f(QIndex_{prev}, e)$

    Where $QIndex_{new}$ is the resulting quantizer index value, $QIndex_{prev}$ is the previously determined quantizer index value, e is the emphasis value corresponding to the

macroblock, and f is an implementation-defined function for which the following implication is true:

$e_1 < e_2 \Rightarrow f(QIndex,e_1) \geq f(QIndex,e_2)$

> **NOTE** This means that lower emphasis values will result in higher quantizer index values, whereas higher emphasis values will result in lower quantizer index values, but the function is not strictly decreasing with respect to the input emphasis value for a given input quantizer index value.

  - If clamping to minimum quantizer index values is enabled in the applied rate control layer, then the quantizer index value is clamped to the corresponding minimum quantizer index value.
  - If clamping to maximum quantizer index values is enabled in the applied rate control layer, then the quantizer index value is clamped to the corresponding maximum quantizer index value.
- In all cases, the final quantizer index value is clamped to the minimum and maximum quantizer index values supported by the video profile.

38.20.14. AV1 Encode Requirements

This section described the **required** AV1 encoding capabilities for physical devices that have at least one queue family that supports the video codec operation `VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR`, as returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR in VkQueueFamilyVideoPropertiesKHR ::`videoCodecOperations`.

Table 57. Required Video Std Header Versions

Video Std Header Name	Version
`vulkan_video_codec_av1std_encode`	1.0.0

Table 58. Required Video Capabilities

Video Capability	Requirement	Requirement Type[1]
VkVideoCapabilitiesKHR		
`flags`	-	min
`minBitstreamBufferOffsetAlignment`	4096	max
`minBitstreamBufferSizeAlignment`	4096	max
`pictureAccessGranularity`	(64,64)	max
`minCodedExtent`	-	max
`maxCodedExtent`	-	min
`maxDpbSlots`	0	min
`maxActiveReferencePictures`	0	min

Video Capability	Requirement	Requirement Type[1]
VkVideoEncodeCapabilitiesKHR		
flags	-	min
rateControlModes	-	min
maxBitrate	5529600	min
maxQualityLevels	1	min
encodeInputPictureGranularity	(64,64)	max
supportedEncodeFeedbackFlags	VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BUFFER_OFFSET_BIT_KHR VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_BYTES_WRITTEN_BIT_KHR	min
VkVideoEncodeAV1CapabilitiesKHR		
flags	-	min
codedPictureAlignment	(8,8)	min
maxTiles	1	min
minTileSize	-	max
maxTileSize	-	min
maxLevel	STD_VIDEO_AV1_LEVEL_2_0	min
superblockSizes	at least one bit set	implementation-dependent
maxSingleReferenceCount	0	min
singleReferenceNameMask	- [2]	min
maxUnidirectionalCompoundReferenceCount	0 [3]	min
maxUnidirectionalCompoundGroup1ReferenceCount	0 [3,4]	min
unidirectionalCompoundReferenceNameMask	- [2]	min
maxBidirectionalCompoundReferenceCount	0 [3]	min
maxBidirectionalCompoundGroup1ReferenceCount	0 [3,5]	min
maxBidirectionalCompoundGroup2ReferenceCount	0 [3,5]	min
bidirectionalCompoundReferenceNameMask	- [2]	min
maxTemporalLayerCount	1	min
maxSpatialLayerCount	1	min
maxOperatingPoints	0	min
minQIndex	-	max

Video Capability	Requirement	Requirement Type[1]
maxQIndex	-	min
prefersGopRemainingFrames	-	implementation-dependent
requiresGopRemainingFrames	-	implementation-dependent
stdSyntaxFlags	-	min
VkVideoEncodeQuantizationMapCapabilitiesKHR		
maxQuantizationMapExtent	-[6]	min
VkVideoEncodeAV1QuantizationMapCapabilitiesKHR		
minQIndexDelta	-[7]	max
maxQIndexDelta	-[7]	min

1

The **Requirement Type** column specifies the requirement is either the minimum value all implementations **must** support, the maximum value all implementations **must** support, or the exact value all implementations **must** support. For bitmasks a minimum value is the least bits all implementations **must** set, but they **may** have additional bits set beyond this minimum.

2

These masks **must** only have bits set in the least significant VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR bits (bit index i indicates support for the AV1 reference name STD_VIDEO_AV1_REFERENCE_NAME_LAST_FRAME + i when using the corresponding AV1 prediction mode), and **must** have at least as many bits set in any *ReferenceNameMask capability as the value of the corresponding max*ReferenceCount capability.

3

If greater than zero, it **must** be at least 2.

4

maxUnidirectionalCompoundGroup1ReferenceCount **must** be less than or equal to maxUnidirectionalCompoundReferenceCount

5

The sum of maxBidirectionalCompoundGroup1ReferenceCount and maxBidirectionalCompoundGroup2ReferenceCount **must** be greater than or equal to maxBidirectionalCompoundReferenceCount

6

If VkVideoCapabilitiesKHR::flags includes VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR or VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR, then the width and height members of maxQuantizationMapExtent **must** be greater than zero.

7

If VkVideoCapabilitiesKHR::flags includes VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR, then maxQIndexDelta **must** be greater than minQIndexDelta.

Chapter 39. Extending Vulkan

New functionality **may** be added to Vulkan via either new extensions or new versions of the core, or new versions of an extension in some cases.

This chapter describes how Vulkan is versioned, how compatibility is affected between different versions, and compatibility rules that are followed by the Vulkan Working Group.

39.1. Instance and Device Functionality

Commands that enumerate instance properties, or that accept a VkInstance object as a parameter, are considered instance-level functionality.

Commands that dispatch from a VkDevice object or a child object of a VkDevice, or take any of them as a parameter, are considered device-level functionality. Types defined by a device extension are also considered device-level functionality.

Commands that dispatch from VkPhysicalDevice, or accept a VkPhysicalDevice object as a parameter, are considered either instance-level or device-level functionality depending if the functionality is specified by an instance extension or device extension respectively.

Additionally, commands that enumerate physical device properties are considered device-level functionality.

NOTE Vulkan 1.0 initially specified new physical device enumeration functionality as instance-level, requiring it to be included in an instance extension. As the capabilities of device-level functionality require discovery via physical device enumeration, this led to the situation where many device extensions required an instance extension as well. To alleviate this extra work, `VK_KHR_get_physical_device_properties2` (and subsequently Vulkan 1.1) redefined device-level functionality to include physical device enumeration.

39.2. Core Versions

The Vulkan Specification is regularly updated with bug fixes and clarifications. Occasionally new functionality is added to the core and at some point it is expected that there will be a desire to perform a large, breaking change to the API. In order to indicate to developers how and when these changes are made to the specification, and to provide a way to identify each set of changes, the Vulkan API maintains a version number.

39.2.1. Version Numbers

The Vulkan version number comprises four parts indicating the variant, major, minor and patch version of the Vulkan API Specification.

The *variant* indicates the variant of the Vulkan API supported by the implementation. This is always 0 for the Vulkan API.

NOTE	A non-zero variant indicates the API is a variant of the Vulkan API and applications will typically need to be modified to run against it. The variant field was a later addition to the version number, added in version 1.2.175 of the Specification. As Vulkan uses variant 0, this change is fully backwards compatible with the previous version number format for Vulkan implementations. New version number macros have been added for this change and the old macros deprecated. For existing applications using the older format and macros, an implementation with non-zero variant will decode as a very high Vulkan version. The high version number should be detectable by applications performing suitable version checking.

The *major version* indicates a significant change in the API, which will encompass a wholly new version of the specification.

The *minor version* indicates the incorporation of new functionality into the core specification.

The *patch version* indicates bug fixes, clarifications, and language improvements have been incorporated into the specification.

Compatibility guarantees made about versions of the API sharing any of the same version numbers are documented in Core Versions

The version number is used in several places in the API. In each such use, the version numbers are packed into a 32-bit integer as follows:

- The variant is a 3-bit integer packed into bits 31-29.
- The major version is a 7-bit integer packed into bits 28-22.
- The minor version number is a 10-bit integer packed into bits 21-12.
- The patch version number is a 12-bit integer packed into bits 11-0.

`VK_API_VERSION_VARIANT` extracts the API variant number from a packed version number:

```
// Provided by VK_VERSION_1_0
#define VK_API_VERSION_VARIANT(version) ((uint32_t)(version) >> 29U)
```

`VK_API_VERSION_MAJOR` extracts the API major version number from a packed version number:

```
// Provided by VK_VERSION_1_0
#define VK_API_VERSION_MAJOR(version) (((uint32_t)(version) >> 22U) & 0x7FU)
```

`VK_VERSION_MAJOR` extracts the API major version number from a packed version number:

```
// Provided by VK_VERSION_1_0
// VK_VERSION_MAJOR is deprecated, but no reason was given in the API XML
// DEPRECATED: This define is deprecated. VK_API_VERSION_MAJOR should be used instead.
#define VK_VERSION_MAJOR(version) ((uint32_t)(version) >> 22U)
```

VK_API_VERSION_MINOR extracts the API minor version number from a packed version number:

```
// Provided by VK_VERSION_1_0
#define VK_API_VERSION_MINOR(version) (((uint32_t)(version) >> 12U) & 0x3FFU)
```

VK_VERSION_MINOR extracts the API minor version number from a packed version number:

```
// Provided by VK_VERSION_1_0
// VK_VERSION_MINOR is deprecated, but no reason was given in the API XML
// DEPRECATED: This define is deprecated. VK_API_VERSION_MINOR should be used instead.
#define VK_VERSION_MINOR(version) (((uint32_t)(version) >> 12U) & 0x3FFU)
```

VK_API_VERSION_PATCH extracts the API patch version number from a packed version number:

```
// Provided by VK_VERSION_1_0
#define VK_API_VERSION_PATCH(version) ((uint32_t)(version) & 0xFFFU)
```

VK_VERSION_PATCH extracts the API patch version number from a packed version number:

```
// Provided by VK_VERSION_1_0
// VK_VERSION_PATCH is deprecated, but no reason was given in the API XML
// DEPRECATED: This define is deprecated. VK_API_VERSION_PATCH should be used instead.
#define VK_VERSION_PATCH(version) ((uint32_t)(version) & 0xFFFU)
```

VK_MAKE_API_VERSION constructs an API version number.

```
// Provided by VK_VERSION_1_0
#define VK_MAKE_API_VERSION(variant, major, minor, patch) \
    ((((uint32_t)(variant)) << 29U) | (((uint32_t)(major)) << 22U) |
(((uint32_t)(minor)) << 12U) | ((uint32_t)(patch)))
```

- variant is the variant number.
- major is the major version number.
- minor is the minor version number.
- patch is the patch version number.

VK_MAKE_VERSION constructs an API version number.

```
// Provided by VK_VERSION_1_0
// VK_MAKE_VERSION is deprecated, but no reason was given in the API XML
// DEPRECATED: This define is deprecated. VK_MAKE_API_VERSION should be used instead.
#define VK_MAKE_VERSION(major, minor, patch) \
    ((((uint32_t)(major)) << 22U) | (((uint32_t)(minor)) << 12U) |
((uint32_t)(patch)))
```

- `major` is the major version number.
- `minor` is the minor version number.
- `patch` is the patch version number.

`VK_API_VERSION_1_0` returns the API version number for Vulkan 1.0.0.

```
// Provided by VK_VERSION_1_0
// Vulkan 1.0 version number
#define VK_API_VERSION_1_0 VK_MAKE_API_VERSION(0, 1, 0, 0)// Patch version should
always be set to 0
```

39.2.2. Querying Version Support

NOTE

In Vulkan 1.0, there is no mechanism to detect the separate versions of instance-level and device-level functionality supported. However, the `vkEnumerateInstanceVersion` command was added in Vulkan 1.1 to determine the supported version of instance-level functionality - querying for this function via vkGetInstanceProcAddr will return `NULL` on implementations that only support Vulkan 1.0 functionality. For more information on this, please refer to the Vulkan 1.1 specification.

The version of device-level functionality can be queried by calling vkGetPhysicalDeviceProperties or vkGetPhysicalDeviceProperties2KHR, and is returned in VkPhysicalDeviceProperties::`apiVersion`, encoded as described in Version Numbers.

39.3. Layers

When a layer is enabled, it inserts itself into the call chain for Vulkan commands the layer is interested in. Layers **can** be used for a variety of tasks that extend the base behavior of Vulkan beyond what is required by the specification - such as call logging, tracing, validation, or providing additional extensions.

NOTE

For example, an implementation is not expected to check that the value of enums used by the application fall within allowed ranges. Instead, a validation layer would do those checks and flag issues. This avoids a performance penalty during production use of the application because those layers would not be enabled in production.

NOTE

Vulkan layers **may** wrap object handles (i.e. return a different handle value to the application than that generated by the implementation). This is generally discouraged, as it increases the probability of incompatibilities with new extensions. The validation layers wrap handles in order to track the proper use and destruction of each object. See the "Architecture of the Vulkan Loader Interfaces" document for additional information.

To query the available layers, call:

```
// Provided by VK_VERSION_1_0
VkResult vkEnumerateInstanceLayerProperties(
    uint32_t*                                   pPropertyCount,
    VkLayerProperties*                          pProperties);
```

- `pPropertyCount` is a pointer to an integer related to the number of layer properties available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of VkLayerProperties structures.

If `pProperties` is `NULL`, then the number of layer properties available is returned in `pPropertyCount`. Otherwise, `pPropertyCount` **must** point to a variable set by the application to the number of elements in the `pProperties` array, and on return the variable is overwritten with the number of structures actually written to `pProperties`. If `pPropertyCount` is less than the number of layer properties available, at most `pPropertyCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available properties were returned.

The list of available layers may change at any time due to actions outside of the Vulkan implementation, so two calls to `vkEnumerateInstanceLayerProperties` with the same parameters **may** return different results, or retrieve different `pPropertyCount` values or `pProperties` contents. Once an instance has been created, the layers enabled for that instance will continue to be enabled and valid for the lifetime of that instance, even if some of them become unavailable for future instances.

Valid Usage (Implicit)

- VUID-vkEnumerateInstanceLayerProperties-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkEnumerateInstanceLayerProperties-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkLayerProperties structures

Return Codes

Success

- `VK_SUCCESS`

- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkLayerProperties structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkLayerProperties {
    char        layerName[VK_MAX_EXTENSION_NAME_SIZE];
    uint32_t    specVersion;
    uint32_t    implementationVersion;
    char        description[VK_MAX_DESCRIPTION_SIZE];
} VkLayerProperties;
```

- layerName is an array of VK_MAX_EXTENSION_NAME_SIZE char containing a null-terminated UTF-8 string which is the name of the layer. Use this name in the ppEnabledLayerNames array passed in the VkInstanceCreateInfo structure to enable this layer for an instance.
- specVersion is the Vulkan version the layer was written to, encoded as described in Version Numbers.
- implementationVersion is the version of this layer. It is an integer, increasing with backward compatible changes.
- description is an array of VK_MAX_DESCRIPTION_SIZE char containing a null-terminated UTF-8 string which provides additional details that **can** be used by the application to identify the layer.

VK_MAX_EXTENSION_NAME_SIZE is the length in char values of an array containing a layer or extension name string, as returned in VkLayerProperties::layerName, VkExtensionProperties::extensionName, and other queries.

```
#define VK_MAX_EXTENSION_NAME_SIZE        256U
```

VK_MAX_DESCRIPTION_SIZE is the length in char values of an array containing a string with additional descriptive information about a query, as returned in VkLayerProperties::description and other queries.

```
#define VK_MAX_DESCRIPTION_SIZE           256U
```

To enable a layer, the name of the layer **should** be added to the ppEnabledLayerNames member of VkInstanceCreateInfo when creating a VkInstance.

Loader implementations **may** provide mechanisms outside the Vulkan API for enabling specific layers. Layers enabled through such a mechanism are *implicitly enabled*, while layers enabled by including the layer name in the ppEnabledLayerNames member of VkInstanceCreateInfo are *explicitly*

enabled. Implicitly enabled layers are loaded before explicitly enabled layers, such that implicitly enabled layers are closer to the application, and explicitly enabled layers are closer to the driver. Except where otherwise specified, implicitly enabled and explicitly enabled layers differ only in the way they are enabled, and the order in which they are loaded. Explicitly enabling a layer that is implicitly enabled results in this layer being loaded as an implicitly enabled layer; it has no additional effect.

39.3.1. Device Layer Deprecation

Previous versions of this specification distinguished between instance and device layers. Instance layers were only able to intercept commands that operate on `VkInstance` and `VkPhysicalDevice`, except they were not able to intercept vkCreateDevice. Device layers were enabled for individual devices when they were created, and could only intercept commands operating on that device or its child objects.

Device-only layers are now deprecated, and this specification no longer distinguishes between instance and device layers. Layers are enabled during instance creation, and are able to intercept all commands operating on that instance or any of its child objects. At the time of deprecation there were no known device-only layers and no compelling reason to create one.

In order to maintain compatibility with implementations released prior to device-layer deprecation, applications **should** still enumerate and enable device layers. The behavior of `vkEnumerateDeviceLayerProperties` and valid usage of the `ppEnabledLayerNames` member of VkDeviceCreateInfo maximizes compatibility with applications written to work with the previous requirements.

To enumerate device layers, call:

```
// Provided by VK_VERSION_1_0
VkResult vkEnumerateDeviceLayerProperties(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pPropertyCount,
    VkLayerProperties*                          pProperties);
```

- `physicalDevice` is the physical device that will be queried.
- `pPropertyCount` is a pointer to an integer related to the number of layer properties available or queried.
- `pProperties` is either `NULL` or a pointer to an array of VkLayerProperties structures.

If `pProperties` is `NULL`, then the number of layer properties available is returned in `pPropertyCount`. Otherwise, `pPropertyCount` **must** point to a variable set by the application to the number of elements in the `pProperties` array, and on return the variable is overwritten with the number of structures actually written to `pProperties`. If `pPropertyCount` is less than the number of layer properties available, at most `pPropertyCount` structures will be written, and `VK_INCOMPLETE` will be returned instead of `VK_SUCCESS`, to indicate that not all the available properties were returned.

The list of layers enumerated by `vkEnumerateDeviceLayerProperties` **must** be exactly the sequence of layers enabled for the instance. The members of `VkLayerProperties` for each enumerated layer **must**

be the same as the properties when the layer was enumerated by `vkEnumerateInstanceLayerProperties`.

NOTE	Due to platform details on Android, `vkEnumerateDeviceLayerProperties` may be called with `physicalDevice` equal to `NULL` during layer discovery. This behavior will only be observed by layer implementations, and not the underlying Vulkan driver.

Valid Usage (Implicit)

- VUID-vkEnumerateDeviceLayerProperties-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkEnumerateDeviceLayerProperties-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkEnumerateDeviceLayerProperties-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkLayerProperties structures

Return Codes

Success

- `VK_SUCCESS`
- `VK_INCOMPLETE`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `ppEnabledLayerNames` and `enabledLayerCount` members of VkDeviceCreateInfo are deprecated and their values **must** be ignored by implementations. However, for compatibility, only an empty list of layers or a list that exactly matches the sequence enabled at instance creation time are valid, and validation layers **should** issue diagnostics for other cases.

Regardless of the enabled layer list provided in VkDeviceCreateInfo, the sequence of layers active for a device will be exactly the sequence of layers enabled when the parent instance was created.

39.4. Extensions

Extensions **may** define new Vulkan commands, structures, and enumerants. For compilation purposes, the interfaces defined by registered extensions, including new structures and enumerants as well as function pointer types for new commands, are defined in the Khronos-supplied `vulkan_core.h` together with the core API. However, commands defined by extensions **may** not be available for static linking - in which case function pointers to these commands **should** be queried at runtime as described in Command Function Pointers. Extensions **may** be provided by layers as well as by a Vulkan implementation.

Because extensions **may** extend or change the behavior of the Vulkan API, extension authors **should** add support for their extensions to the Khronos validation layers. This is especially important for new commands whose parameters have been wrapped by the validation layers. See the "Architecture of the Vulkan Loader Interfaces" document for additional information.

NOTE

To enable an instance extension, the name of the extension **can** be added to the `ppEnabledExtensionNames` member of `VkInstanceCreateInfo` when creating a `VkInstance`.

To enable a device extension, the name of the extension **can** be added to the `ppEnabledExtensionNames` member of `VkDeviceCreateInfo` when creating a `VkDevice`.

Physical-Device-Level functionality does not have any enabling mechanism and **can** be used as long as the `VkPhysicalDevice` supports the device extension as determined by `vkEnumerateDeviceExtensionProperties`.

Enabling an extension (with no further use of that extension) does not change the behavior of functionality exposed by the core Vulkan API or any other extension, other than making valid the use of the commands, enums and structures defined by that extension.

Valid Usage sections for individual commands and structures do not currently contain which extensions have to be enabled in order to make their use valid, although they might do so in the future. It is defined only in the Valid Usage for Extensions section.

39.4.1. Instance Extensions

Instance extensions add new instance-level functionality to the API, outside of the core specification.

To query the available instance extensions, call:

```
// Provided by VK_VERSION_1_0
VkResult vkEnumerateInstanceExtensionProperties(
    const char*                                 pLayerName,
    uint32_t*                                   pPropertyCount,
    VkExtensionProperties*                      pProperties);
```

- `pLayerName` is either `NULL` or a pointer to a null-terminated UTF-8 string naming the layer to retrieve extensions from.
- `pPropertyCount` is a pointer to an integer related to the number of extension properties available or queried, as described below.
- `pProperties` is either `NULL` or a pointer to an array of `VkExtensionProperties` structures.

When `pLayerName` parameter is `NULL`, only extensions provided by the Vulkan implementation or by implicitly enabled layers are returned. When `pLayerName` is the name of a layer, the instance extensions provided by that layer are returned.

If pProperties is NULL, then the number of extensions properties available is returned in pPropertyCount. Otherwise, pPropertyCount **must** point to a variable set by the application to the number of elements in the pProperties array, and on return the variable is overwritten with the number of structures actually written to pProperties. If pPropertyCount is less than the number of extension properties available, at most pPropertyCount structures will be written, and VK_INCOMPLETE will be returned instead of VK_SUCCESS, to indicate that not all the available properties were returned.

Because the list of available layers may change externally between calls to vkEnumerateInstanceExtensionProperties, two calls may retrieve different results if a pLayerName is available in one call but not in another. The extensions supported by a layer may also change between two calls, e.g. if the layer implementation is replaced by a different version between those calls.

Implementations **must** not advertise any pair of extensions that cannot be enabled together due to behavioral differences, or any extension that cannot be enabled against the advertised version.

Valid Usage (Implicit)

- VUID-vkEnumerateInstanceExtensionProperties-pLayerName-parameter
  If pLayerName is not NULL, pLayerName **must** be a null-terminated UTF-8 string
- VUID-vkEnumerateInstanceExtensionProperties-pPropertyCount-parameter
  pPropertyCount **must** be a valid pointer to a uint32_t value
- VUID-vkEnumerateInstanceExtensionProperties-pProperties-parameter
  If the value referenced by pPropertyCount is not 0, and pProperties is not NULL, pProperties **must** be a valid pointer to an array of pPropertyCount VkExtensionProperties structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_LAYER_NOT_PRESENT

39.4.2. Device Extensions

Device extensions add new device-level functionality to the API, outside of the core specification.

To query the extensions available to a given physical device, call:

```
// Provided by VK_VERSION_1_0
VkResult vkEnumerateDeviceExtensionProperties(
    VkPhysicalDevice                            physicalDevice,
    const char*                                 pLayerName,
    uint32_t*                                   pPropertyCount,
    VkExtensionProperties*                      pProperties);
```

- `physicalDevice` is the physical device that will be queried.
- `pLayerName` is either `NULL` or a pointer to a null-terminated UTF-8 string naming the layer to retrieve extensions from.
- `pPropertyCount` is a pointer to an integer related to the number of extension properties available or queried, and is treated in the same fashion as the vkEnumerateInstanceExtensionProperties ::`pPropertyCount` parameter.
- `pProperties` is either `NULL` or a pointer to an array of VkExtensionProperties structures.

When `pLayerName` parameter is `NULL`, only extensions provided by the Vulkan implementation or by implicitly enabled layers are returned. When `pLayerName` is the name of a layer, the device extensions provided by that layer are returned.

Implementations **must** not advertise any pair of extensions that cannot be enabled together due to behavioral differences, or any extension that cannot be enabled against the advertised version.

If the `VK_KHR_ray_tracing_pipeline` extension is advertised as supported by this query, the `VK_KHR_pipeline_library` extension **must** also be supported.

NOTE	Due to platform details on Android, `vkEnumerateDeviceExtensionProperties` may be called with `physicalDevice` equal to `NULL` during layer discovery. This behavior will only be observed by layer implementations, and not the underlying Vulkan driver.

Valid Usage (Implicit)

- VUID-vkEnumerateDeviceExtensionProperties-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkEnumerateDeviceExtensionProperties-pLayerName-parameter
  If `pLayerName` is not `NULL`, `pLayerName` **must** be a null-terminated UTF-8 string
- VUID-vkEnumerateDeviceExtensionProperties-pPropertyCount-parameter
  `pPropertyCount` **must** be a valid pointer to a `uint32_t` value
- VUID-vkEnumerateDeviceExtensionProperties-pProperties-parameter
  If the value referenced by `pPropertyCount` is not `0`, and `pProperties` is not `NULL`, `pProperties` **must** be a valid pointer to an array of `pPropertyCount` VkExtensionProperties structures

Return Codes

Success

- VK_SUCCESS
- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_LAYER_NOT_PRESENT

The VkExtensionProperties structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkExtensionProperties {
    char        extensionName[VK_MAX_EXTENSION_NAME_SIZE];
    uint32_t    specVersion;
} VkExtensionProperties;
```

- extensionName is an array of VK_MAX_EXTENSION_NAME_SIZE char containing a null-terminated UTF-8 string which is the name of the extension.
- specVersion is the version of this extension. It is an integer, incremented with backward compatible changes.

Accessing Device-Level Functionality From a VkPhysicalDevice

Some device extensions also add support for physical-device-level functionality. Physical-device-level functionality **can** be used, if the required extension is supported as advertised by vkEnumerateDeviceExtensionProperties for a given VkPhysicalDevice.

Accessing Device-Level Functionality From a VkDevice

For commands that are dispatched from a VkDevice, or from a child object of a VkDevice, device extensions **must** be enabled in vkCreateDevice.

39.5. Extension Dependencies

Some extensions are dependent on other extensions, or on specific core API versions, to function. To enable extensions with dependencies, any *required extensions* **must** also be enabled through the same API mechanisms when creating an instance with vkCreateInstance or a device with vkCreateDevice. Each extension which has such dependencies documents them in the appendix summarizing that extension.

If an extension is supported (as queried by vkEnumerateInstanceExtensionProperties or vkEnumerateDeviceExtensionProperties), then *required extensions* of that extension **must** also be

supported for the same instance or physical device.

Any device extension that has an instance extension dependency that is not enabled by vkCreateInstance is considered to be unsupported, hence it **must** not be returned by vkEnumerateDeviceExtensionProperties for any VkPhysicalDevice child of the instance. Instance extensions do not have dependencies on device extensions.

If a required extension has been promoted to another extension or to a core API version, then as a *general* rule, the dependency is also satisfied by the promoted extension or core version. This will be true so long as any features required by the original extension are also required or enabled by the promoted extension or core version. However, in some cases an extension is promoted while making some of its features optional in the promoted extension or core version. In this case, the dependency **may** not be satisfied. The only way to be certain is to look at the descriptions of the original dependency and the promoted version in the Layers & Extensions and Core Revisions appendices.

NOTE

There is metadata in `vk.xml` describing some aspects of promotion, especially `requires`, `promotedto` and `deprecatedby` attributes of `<extension>` tags. However, the metadata does not yet fully describe this scenario. In the future, we may extend the XML schema to describe the full set of extensions and versions satisfying a dependency. As discussed in more detail for Promotion below, when an extension is promoted it does not mean that a mechanical substitution of an extension API by the corresponding promoted API will work in exactly the same fashion; be supported at runtime; or even exist.

39.6. Compatibility Guarantees (Informative)

This section is marked as informal as there is no binding responsibility on implementations of the Vulkan API - these guarantees are however a contract between the Vulkan Working Group and developers using this Specification.

39.6.1. Core Versions

Each of the major, minor, and patch versions of the Vulkan specification provide different compatibility guarantees.

Patch Versions

A difference in the patch version indicates that a set of bug fixes or clarifications have been made to the Specification. Informative enums returned by Vulkan commands that will not affect the runtime behavior of a valid application may be added in a patch version (e.g. VkVendorId).

The specification's patch version is strictly increasing for a given major version of the specification; any change to a specification as described above will result in the patch version being increased by 1. Patch versions are applied to all minor versions, even if a given minor version is not affected by the provoking change.

Specifications with different patch versions but the same major and minor version are *fully*

compatible with each other - such that a valid application written against one will work with an implementation of another.

NOTE	If a patch version includes a bug fix or clarification that could have a significant impact on developer expectations, these will be highlighted in the change log. Generally the Vulkan Working Group tries to avoid these kinds of changes, instead fixing them in either an extension or core version.

Minor Versions

Changes in the minor version of the specification indicate that new functionality has been added to the core specification. This will usually include new interfaces in the header, and **may** also include behavior changes and bug fixes. Core functionality **may** be deprecated in a minor version, but will not be obsoleted or removed.

The specification's minor version is strictly increasing for a given major version of the specification; any change to a specification as described above will result in the minor version being increased by 1. Changes that can be accommodated in a patch version will not increase the minor version.

Specifications with a lower minor version are *backwards compatible* with an implementation of a specification with a higher minor version for core functionality and extensions issued with the KHR vendor tag. Vendor and multi-vendor extensions are not guaranteed to remain functional across minor versions, though in general they are with few exceptions - see Obsoletion for more information.

Major Versions

A difference in the major version of specifications indicates a large set of changes which will likely include interface changes, behavioral changes, removal of deprecated functionality, and the modification, addition, or replacement of other functionality.

The specification's major version is monotonically increasing; any change to the specification as described above will result in the major version being increased. Changes that can be accommodated in a patch or minor version will not increase the major version.

The Vulkan Working Group intends to only issue a new major version of the Specification in order to realize significant improvements to the Vulkan API that will necessarily require breaking compatibility.

A new major version will likely include a wholly new version of the specification to be issued - which could include an overhaul of the versioning semantics for the minor and patch versions. The patch and minor versions of a specification are therefore not meaningful across major versions. If a major version of the specification includes similar versioning semantics, it is expected that the patch and the minor version will be reset to 0 for that major version.

39.6.2. Extensions

A KHR extension **must** be able to be enabled alongside any other KHR extension, and for any minor

or patch version of the core Specification beyond the minimum version it requires. A multi-vendor extension **should** be able to be enabled alongside any KHR extension or other multi-vendor extension, and for any minor or patch version of the core Specification beyond the minimum version it requires. A vendor extension **should** be able to be enabled alongside any KHR extension, multi-vendor extension, or other vendor extension from the same vendor, and for any minor or patch version of the core Specification beyond the minimum version it requires. A vendor extension **may** be able to be enabled alongside vendor extensions from another vendor.

The one other exception to this is if a vendor or multi-vendor extension is made obsolete by either a core version or another extension, which will be highlighted in the extension appendix.

Promotion

Extensions, or features of an extension, **may** be promoted to a new core version of the API, or a newer extension which an equal or greater number of implementors are in favor of.

NOTE Structure and enumerated types defined in extensions that were promoted to core in a later version of Vulkan are defined in terms of the equivalent Vulkan core interfaces in that version and its successors. This affects the Vulkan Specification, the Vulkan header files, and the corresponding XML Registry.

When extension functionality is promoted, minor changes **may** be introduced, limited to the following:

- Naming
- Non-intrusive parameter changes
- Feature advertisement/enablement
- Combining structure parameters into larger structures
- Author ID suffixes changed or removed

NOTE If extension functionality is promoted, there is no guarantee of direct compatibility, however it should require little effort to port code from the original feature to the promoted one.

The Vulkan Working Group endeavors to ensure that larger changes are marked as either deprecated or obsoleted as appropriate, and can do so retroactively if necessary.

Extensions that are promoted are listed as being promoted in their extension appendices, with reference to where they were promoted to.

When an extension is promoted, any backwards compatibility aliases which exist in the extension will **not** be promoted.

NOTE As a hypothetical example, if the `VK_KHR_surface` extension were promoted to part of a future core version, the `VK_COLOR_SPACE_SRGB_NONLINEAR_KHR` token defined by that extension would be promoted to `VK_COLOR_SPACE_SRGB_NONLINEAR`. However, the

> `VK_COLORSPACE_SRGB_NONLINEAR_KHR` token aliases `VK_COLOR_SPACE_SRGB_NONLINEAR_KHR`. The `VK_COLORSPACE_SRGB_NONLINEAR_KHR` would not be promoted, because it is a backwards compatibility alias that exists only due to a naming mistake when the extension was initially published.

Deprecation

Extensions **may** be marked as deprecated when the intended use cases either become irrelevant or can be solved in other ways. Generally, a new feature will become available to solve the use case in another extension or core version of the API, but it is not guaranteed.

> **NOTE** Features that are intended to replace deprecated functionality have no guarantees of compatibility, and applications may require drastic modification in order to make use of the new features.

Extensions that are deprecated are listed as being deprecated in their extension appendices, with an explanation of the deprecation and any features that are relevant.

Obsoletion

Occasionally, an extension will be marked as obsolete if a new version of the core API or a new extension is fundamentally incompatible with it. An obsoleted extension **must** not be used with the extension or core version that obsoleted it.

Extensions that are obsoleted are listed as being obsoleted in their extension appendices, with reference to what they were obsoleted by.

Aliases

When an extension is promoted or deprecated by a newer feature, some or all of its functionality **may** be replicated into the newer feature. Rather than duplication of all the documentation and definitions, the specification instead identifies the identical commands and types as *aliases* of one another. Each alias is mentioned together with the definition it aliases, with the older aliases marked as “equivalents”. Each alias of the same command has identical behavior, and each alias of the same type has identical meaning - they can be used interchangeably in an application with no compatibility issues.

> **NOTE** For promoted types, the aliased extension type is semantically identical to the new core type. The C99 headers simply `typedef` the older aliases to the promoted types.
>
> For promoted command aliases, however, there are two separate command definitions, due to the fact that the C99 ABI has no way to alias command definitions without resorting to macros. Calling either command will produce identical behavior within the bounds of the specification, and should still invoke the same path in the implementation. Debug tools may use separate commands with different debug behavior; to write the appropriate command name to an output log, for instance.

Special Use Extensions

Some extensions exist only to support a specific purpose or specific class of application. These are referred to as “special use extensions”. Use of these extensions in applications not meeting the special use criteria is not recommended.

Special use cases are restricted, and only those defined below are used to describe extensions:

Table 59. Extension Special Use Cases

Special Use	XML Tag	Full Description
CAD support	cadsupport	Extension is intended to support specialized functionality used by CAD/CAM applications.
D3D support	d3demulation	Extension is intended to support D3D emulation layers, and applications ported from D3D, by adding functionality specific to D3D.
Developer tools	devtools	Extension is intended to support developer tools such as capture-replay libraries.
Debugging tools	debugging	Extension is intended for use by applications when debugging.
OpenGL / ES support	glemulation	Extension is intended to support OpenGL and/or OpenGL ES emulation layers, and applications ported from those APIs, by adding functionality specific to those APIs.

Special use extensions are identified in the metadata for each such extension in the Layers & Extensions appendix, using the name in the “Special Use” column above.

Special use extensions are also identified in `vk.xml` with the short name in “XML Tag” column above, as described in the “API Extensions (`extension` tag)” section of the registry schema documentation.

Chapter 40. Features

Features advertise additional functionality which **can** be enabled in the API. If a feature is *supported* by a physical device, it **must** be *enabled* when creating a VkDevice in order to use its associated functionality with that VkDevice. If a feature is *unsupported*, that functionality **cannot** be used with that VkDevice.

NOTE

Features are reported via the basic VkPhysicalDeviceFeatures structure, as well as the extensible structure `VkPhysicalDeviceFeatures2`, which was added in the `VK_KHR_get_physical_device_properties2` extension and included in Vulkan 1.1. When new features are added in future Vulkan versions or extensions, each extension **should** introduce one new feature structure, if needed. This structure **can** be added to the `pNext` chain of the `VkPhysicalDeviceFeatures2` structure.

For convenience, new core versions of Vulkan **may** introduce new unified feature structures for features promoted from extensions. At the same time, the extension's original feature structure (if any) is also promoted to the core API, and is an alias of the extension's structure. This results in multiple names for the same feature: in the original extension's feature structure and the promoted structure alias, in the unified feature structure. When a feature was implicitly supported and enabled in the extension, but an explicit name was added during promotion, then the extension itself acts as an alias for the feature as listed in the table below.

All aliases of the same feature in the core API **must** be reported consistently: either all **must** be reported as supported, or none of them. When a promoted extension is available, any corresponding feature aliases **must** be supported.

Table 60. Extension Feature Aliases

Extension	Feature(s)

To query supported features, call:

```
// Provided by VK_VERSION_1_0
void vkGetPhysicalDeviceFeatures(
    VkPhysicalDevice                            physicalDevice,
    VkPhysicalDeviceFeatures*                   pFeatures);
```

- `physicalDevice` is the physical device from which to query the supported features.
- `pFeatures` is a pointer to a VkPhysicalDeviceFeatures structure in which the physical device features are returned. For each feature, a value of `VK_TRUE` specifies that the feature is supported on this physical device, and `VK_FALSE` specifies that the feature is not supported.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceFeatures-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle

- VUID-vkGetPhysicalDeviceFeatures-pFeatures-parameter
  `pFeatures` **must** be a valid pointer to a VkPhysicalDeviceFeatures structure

Fine-grained features used by a logical device **must** be enabled at `VkDevice` creation time. If a feature is enabled that the physical device does not support, `VkDevice` creation will fail and return `VK_ERROR_FEATURE_NOT_PRESENT`.

The fine-grained features are enabled by passing a pointer to the `VkPhysicalDeviceFeatures` structure via the `pEnabledFeatures` member of the VkDeviceCreateInfo structure that is passed into the `vkCreateDevice` call. If a member of `pEnabledFeatures` is `VK_TRUE` or `VK_FALSE`, then the device will be created with the indicated feature enabled or disabled, respectively. Features **can** also be enabled by using the VkPhysicalDeviceFeatures2 structure.

If an application wishes to enable all features supported by a device, it **can** simply pass in the `VkPhysicalDeviceFeatures` structure that was previously returned by `vkGetPhysicalDeviceFeatures`. To disable an individual feature, the application **can** set the desired member to `VK_FALSE` in the same structure. Setting `pEnabledFeatures` to `NULL` and not including a VkPhysicalDeviceFeatures2 in the `pNext` chain of VkDeviceCreateInfo is equivalent to setting all members of the structure to `VK_FALSE`.

NOTE	Some features, such as `robustBufferAccess`, **may** incur a runtime performance cost. Application writers **should** carefully consider the implications of enabling all supported features.

To query supported features defined by the core or extensions, call:

```
// Provided by VK_KHR_get_physical_device_properties2
void vkGetPhysicalDeviceFeatures2KHR(
    VkPhysicalDevice                            physicalDevice,
    VkPhysicalDeviceFeatures2*                  pFeatures);
```

- `physicalDevice` is the physical device from which to query the supported features.
- `pFeatures` is a pointer to a VkPhysicalDeviceFeatures2 structure in which the physical device features are returned.

Each structure in `pFeatures` and its `pNext` chain contains members corresponding to fine-grained features. Each structure in `pFeatures` and its `pNext` chain contains VkBool32 members corresponding to fine-grained features. Each such member is returned with a `VK_TRUE` value indicating that feature is supported on this physical device, or a `VK_FALSE` value indicating it is unsupported.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceFeatures2-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceFeatures2-pFeatures-parameter
  `pFeatures` **must** be a valid pointer to a VkPhysicalDeviceFeatures2 structure

The VkPhysicalDeviceFeatures2 structure is defined as:

```
typedef struct VkPhysicalDeviceFeatures2 {
    VkStructureType             sType;
    void*                       pNext;
    VkPhysicalDeviceFeatures    features;
} VkPhysicalDeviceFeatures2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkPhysicalDeviceFeatures2 VkPhysicalDeviceFeatures2KHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- features is a VkPhysicalDeviceFeatures structure describing the fine-grained features of the Vulkan 1.0 API.

The pNext chain of this structure is used to extend the structure with features defined by extensions. This structure **can** be used in vkGetPhysicalDeviceFeatures2KHR or **can** be included in the pNext chain of a VkDeviceCreateInfo structure, in which case it controls which features are enabled in lieu of pEnabledFeatures.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFeatures2-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2

The VkPhysicalDeviceFeatures structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPhysicalDeviceFeatures {
    VkBool32    robustBufferAccess;
    VkBool32    fullDrawIndexUint32;
    VkBool32    imageCubeArray;
    VkBool32    independentBlend;
    VkBool32    geometryShader;
    VkBool32    tessellationShader;
    VkBool32    sampleRateShading;
    VkBool32    dualSrcBlend;
    VkBool32    logicOp;
    VkBool32    multiDrawIndirect;
    VkBool32    drawIndirectFirstInstance;
    VkBool32    depthClamp;
    VkBool32    depthBiasClamp;
```

```
    VkBool32    fillModeNonSolid;
    VkBool32    depthBounds;
    VkBool32    wideLines;
    VkBool32    largePoints;
    VkBool32    alphaToOne;
    VkBool32    multiViewport;
    VkBool32    samplerAnisotropy;
    VkBool32    textureCompressionETC2;
    VkBool32    textureCompressionASTC_LDR;
    VkBool32    textureCompressionBC;
    VkBool32    occlusionQueryPrecise;
    VkBool32    pipelineStatisticsQuery;
    VkBool32    vertexPipelineStoresAndAtomics;
    VkBool32    fragmentStoresAndAtomics;
    VkBool32    shaderTessellationAndGeometryPointSize;
    VkBool32    shaderImageGatherExtended;
    VkBool32    shaderStorageImageExtendedFormats;
    VkBool32    shaderStorageImageMultisample;
    VkBool32    shaderStorageImageReadWithoutFormat;
    VkBool32    shaderStorageImageWriteWithoutFormat;
    VkBool32    shaderUniformBufferArrayDynamicIndexing;
    VkBool32    shaderSampledImageArrayDynamicIndexing;
    VkBool32    shaderStorageBufferArrayDynamicIndexing;
    VkBool32    shaderStorageImageArrayDynamicIndexing;
    VkBool32    shaderClipDistance;
    VkBool32    shaderCullDistance;
    VkBool32    shaderFloat64;
    VkBool32    shaderInt64;
    VkBool32    shaderInt16;
    VkBool32    shaderResourceResidency;
    VkBool32    shaderResourceMinLod;
    VkBool32    sparseBinding;
    VkBool32    sparseResidencyBuffer;
    VkBool32    sparseResidencyImage2D;
    VkBool32    sparseResidencyImage3D;
    VkBool32    sparseResidency2Samples;
    VkBool32    sparseResidency4Samples;
    VkBool32    sparseResidency8Samples;
    VkBool32    sparseResidency16Samples;
    VkBool32    sparseResidencyAliased;
    VkBool32    variableMultisampleRate;
    VkBool32    inheritedQueries;
} VkPhysicalDeviceFeatures;
```

This structure describes the following features:

- `robustBufferAccess` specifies that accesses to buffers are bounds-checked against the range of the buffer descriptor (as determined by `VkDescriptorBufferInfo`::`range`, VkBufferViewCreateInfo::`range`, or the size of the buffer). Out of bounds accesses **must** not cause application termination, and the effects of shader loads, stores, and atomics **must**

conform to an implementation-dependent behavior as described below.

- A buffer access is considered to be out of bounds if any of the following are true:
  - The pointer was formed by `OpImageTexelPointer` and the coordinate is less than zero or greater than or equal to the number of whole elements in the bound range.
  - The pointer was not formed by `OpImageTexelPointer` and the object pointed to is not wholly contained within the bound range. This includes accesses performed via *variable pointers* where the buffer descriptor being accessed cannot be statically determined. Uninitialized pointers and pointers equal to `OpConstantNull` are treated as pointing to a zero-sized object, so all accesses through such pointers are considered to be out of bounds. Buffer accesses through buffer device addresses are not bounds-checked.
  - If the `VkPhysicalDeviceCooperativeMatrixFeaturesKHR::cooperativeMatrixRobustBufferAccess` feature is not enabled, then accesses using `OpCooperativeMatrixLoadKHR` and `OpCooperativeMatrixStoreKHR` **may** not be bounds-checked.

    > **NOTE** If a SPIR-V `OpLoad` instruction loads a structure and the tail end of the structure is out of bounds, then all members of the structure are considered out of bounds even if the members at the end are not statically used.

  - If the `robustBufferAccess2` feature is not enabled and any buffer access is determined to be out of bounds, then any other access of the same type (load, store, or atomic) to the same buffer that accesses an address less than 16 bytes away from the out of bounds address **may** also be considered out of bounds.
  - If the access is a load that reads from the same memory locations as a prior store in the same shader invocation, with no other intervening accesses to the same memory locations in that shader invocation, then the result of the load **may** be the value stored by the store instruction, even if the access is out of bounds. If the load is `Volatile`, then an out of bounds load **must** return the appropriate out of bounds value.
- Accesses to descriptors written with a VK_NULL_HANDLE resource or view are not considered to be out of bounds. Instead, each type of descriptor access defines a specific behavior for accesses to a null descriptor.
- Out-of-bounds buffer loads will return any of the following values:
  - If the access is to a uniform buffer and the `robustBufferAccess2` feature is enabled, loads of offsets between the end of the descriptor range and the end of the descriptor range rounded up to a multiple of `robustUniformBufferAccessSizeAlignment` bytes **must** return either zero values or the contents of the memory at the offset being loaded. Loads of offsets past the descriptor range rounded up to a multiple of `robustUniformBufferAccessSizeAlignment` bytes **must** return zero values.
  - If the access is to a storage buffer and the `robustBufferAccess2` feature is enabled, loads of offsets between the end of the descriptor range and the end of the descriptor range rounded up to a multiple of `robustStorageBufferAccessSizeAlignment` bytes **must** return either zero values or the contents of the memory at the offset being loaded. Loads of offsets past the descriptor range rounded up to a multiple of

`robustStorageBufferAccessSizeAlignment` bytes **must** return zero values. Similarly, stores to addresses between the end of the descriptor range and the end of the descriptor range rounded up to a multiple of `robustStorageBufferAccessSizeAlignment` bytes **may** be discarded.

- Non-atomic accesses to storage buffers that are a multiple of 32 bits **may** be decomposed into 32-bit accesses that are individually bounds-checked.
- If the access is to an index buffer and the `robustBufferAccess2` feature is enabled, zero values **must** be returned.
- If the access is to a uniform texel buffer or storage texel buffer and the `robustBufferAccess2` feature is enabled, zero values **must** be returned, and then Conversion to RGBA is applied based on the buffer view's format.
- Values from anywhere within the memory range(s) bound to the buffer (possibly including bytes of memory past the end of the buffer, up to the end of the bound range).
- Zero values, or (0,0,0,x) vectors for vector reads where x is a valid value represented in the type of the vector components and **may** be any of:
  - 0, 1, or the maximum representable positive integer value, for signed or unsigned integer components
  - 0.0 or 1.0, for floating-point components

◦ Out-of-bounds writes **may** modify values within the memory range(s) bound to the buffer, but **must** not modify any other memory.

- If the `robustBufferAccess2` feature is enabled, out of bounds writes **must** not modify any memory.

◦ Out-of-bounds atomics **may** modify values within the memory range(s) bound to the buffer, but **must** not modify any other memory, and return an undefined value.

- If the `robustBufferAccess2` feature is enabled, out of bounds atomics **must** not modify any memory, and return an undefined value.

◦ If the `robustBufferAccess2` feature is not enabled, vertex input attributes are considered out of bounds if the offset of the attribute in the bound vertex buffer range plus the size of the attribute is greater than either:

- `vertexBufferRangeSize`, if `bindingStride` == 0; or
- (`vertexBufferRangeSize` - (`vertexBufferRangeSize` % `bindingStride`))

  where `vertexBufferRangeSize` is the byte size of the memory range bound to the vertex buffer binding and `bindingStride` is the byte stride of the corresponding vertex input binding. Further, if any vertex input attribute using a specific vertex input binding is out of bounds, then all vertex input attributes using that vertex input binding for that vertex shader invocation are considered out of bounds.

- If a vertex input attribute is out of bounds, it will be assigned one of the following values:
  - Values from anywhere within the memory range(s) bound to the buffer, converted according to the format of the attribute.

        - Zero values, format converted according to the format of the attribute.
        - Zero values, or (0,0,0,x) vectors, as described above.
    - If the `robustBufferAccess2` feature is enabled, vertex input attributes are considered out of bounds if the offset of the attribute in the bound vertex buffer range plus the size of the attribute is greater than the byte size of the memory range bound to the vertex buffer binding.
        - If a vertex input attribute is out of bounds, the raw data extracted are zero values, and missing G, B, or A components are filled with (0,0,1).
    - If `robustBufferAccess` is not enabled, applications **must** not perform out of bounds accesses .
- `fullDrawIndexUint32` specifies the full 32-bit range of indices is supported for indexed draw calls when using a VkIndexType of `VK_INDEX_TYPE_UINT32`. `maxDrawIndexedIndexValue` is the maximum index value that **may** be used (aside from the primitive restart index, which is always 2^{32}-1 when the VkIndexType is `VK_INDEX_TYPE_UINT32`). If this feature is supported, `maxDrawIndexedIndexValue` **must** be 2^{32}-1; otherwise it **must** be no smaller than 2^{24}-1. See `maxDrawIndexedIndexValue`.
- `imageCubeArray` specifies whether image views with a VkImageViewType of `VK_IMAGE_VIEW_TYPE_CUBE_ARRAY` **can** be created, and that the corresponding `SampledCubeArray` and `ImageCubeArray` SPIR-V capabilities **can** be used in shader code.
- `independentBlend` specifies whether the `VkPipelineColorBlendAttachmentState` settings are controlled independently per-attachment. If this feature is not enabled, the `VkPipelineColorBlendAttachmentState` settings for all color attachments **must** be identical. Otherwise, a different `VkPipelineColorBlendAttachmentState` **can** be provided for each bound color attachment.
- `geometryShader` specifies whether geometry shaders are supported. If this feature is not enabled, the `VK_SHADER_STAGE_GEOMETRY_BIT` and `VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT` enum values **must** not be used. This also specifies whether shader modules **can** declare the `Geometry` capability.
- `tessellationShader` specifies whether tessellation control and evaluation shaders are supported. If this feature is not enabled, the `VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT`, `VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT`, `VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT`, `VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT`, and `VK_STRUCTURE_TYPE_PIPELINE_TESSELLATION_STATE_CREATE_INFO` enum values **must** not be used. This also specifies whether shader modules **can** declare the `Tessellation` capability.
- `sampleRateShading` specifies whether Sample Shading and multisample interpolation are supported. If this feature is not enabled, the `sampleShadingEnable` member of the VkPipelineMultisampleStateCreateInfo structure **must** be `VK_FALSE` and the `minSampleShading` member is ignored. This also specifies whether shader modules **can** declare the `SampleRateShading` capability.
- `dualSrcBlend` specifies whether blend operations which take two sources are supported. If this feature is not enabled, the `VK_BLEND_FACTOR_SRC1_COLOR`, `VK_BLEND_FACTOR_ONE_MINUS_SRC1_COLOR`, `VK_BLEND_FACTOR_SRC1_ALPHA`, and `VK_BLEND_FACTOR_ONE_MINUS_SRC1_ALPHA` enum values **must** not be used as source or destination blending factors. See Dual-Source Blending.
- `logicOp` specifies whether logic operations are supported. If this feature is not enabled, the

`logicOpEnable` member of the VkPipelineColorBlendStateCreateInfo structure **must** be `VK_FALSE`, and the `logicOp` member is ignored.

- `multiDrawIndirect` specifies whether multiple draw indirect is supported. If this feature is not enabled, the `drawCount` parameter to the vkCmdDrawIndirect and vkCmdDrawIndexedIndirect commands **must** be 0 or 1. The `maxDrawIndirectCount` member of the `VkPhysicalDeviceLimits` structure **must** also be 1 if this feature is not supported. See `maxDrawIndirectCount`.
- `drawIndirectFirstInstance` specifies whether indirect drawing calls support the `firstInstance` parameter. If this feature is not enabled, the `firstInstance` member of all `VkDrawIndirectCommand` and `VkDrawIndexedIndirectCommand` structures that are provided to the vkCmdDrawIndirect and vkCmdDrawIndexedIndirect commands **must** be 0.
- `depthClamp` specifies whether depth clamping is supported. If this feature is not enabled, the `depthClampEnable` member of the VkPipelineRasterizationStateCreateInfo structure **must** be `VK_FALSE`. Otherwise, setting `depthClampEnable` to `VK_TRUE` will enable depth clamping.
- `depthBiasClamp` specifies whether depth bias clamping is supported. If this feature is not enabled, the `depthBiasClamp` member of the VkPipelineRasterizationStateCreateInfo structure **must** be 0.0 unless the `VK_DYNAMIC_STATE_DEPTH_BIAS` dynamic state is enabled, in which case the `depthBiasClamp` parameter to vkCmdSetDepthBias **must** be 0.0.
- `fillModeNonSolid` specifies whether point and wireframe fill modes are supported. If this feature is not enabled, the `VK_POLYGON_MODE_POINT` and `VK_POLYGON_MODE_LINE` enum values **must** not be used.
- `depthBounds` specifies whether depth bounds tests are supported. If this feature is not enabled, the `depthBoundsTestEnable` member of the VkPipelineDepthStencilStateCreateInfo structure **must** be `VK_FALSE` unless the `VK_DYNAMIC_STATE_DEPTH_BOUNDS_TEST_ENABLE` dynamic state is enabled, in which case the `depthBoundsTestEnable` parameter to vkCmdSetDepthBoundsTestEnableEXT **must** be `VK_FALSE`. When `depthBoundsTestEnable` is `VK_FALSE`, the `minDepthBounds` and `maxDepthBounds` members of the VkPipelineDepthStencilStateCreateInfo structure are ignored.
- `wideLines` specifies whether lines with width other than 1.0 are supported. If this feature is not enabled, the `lineWidth` member of the VkPipelineRasterizationStateCreateInfo structure **must** be 1.0 unless the `VK_DYNAMIC_STATE_LINE_WIDTH` dynamic state is enabled, in which case the `lineWidth` parameter to vkCmdSetLineWidth **must** be 1.0. When this feature is supported, the range and granularity of supported line widths are indicated by the `lineWidthRange` and `lineWidthGranularity` members of the `VkPhysicalDeviceLimits` structure, respectively.
- `largePoints` specifies whether points with size greater than 1.0 are supported. If this feature is not enabled, only a point size of 1.0 written by a shader is supported. The range and granularity of supported point sizes are indicated by the `pointSizeRange` and `pointSizeGranularity` members of the `VkPhysicalDeviceLimits` structure, respectively.
- `alphaToOne` specifies whether the implementation is able to replace the alpha value of the fragment shader color output in the Multisample Coverage fragment operation. If this feature is not enabled, then the `alphaToOneEnable` member of the VkPipelineMultisampleStateCreateInfo structure **must** be `VK_FALSE`. Otherwise setting `alphaToOneEnable` to `VK_TRUE` will enable alpha-to-one behavior.
- `multiViewport` specifies whether more than one viewport is supported. If this feature is not enabled:

    - The `viewportCount` and `scissorCount` members of the VkPipelineViewportStateCreateInfo structure **must** be 1.
    - The `firstViewport` and `viewportCount` parameters to the vkCmdSetViewport command **must** be 0 and 1, respectively.
    - The `firstScissor` and `scissorCount` parameters to the vkCmdSetScissor command **must** be 0 and 1, respectively.
- `samplerAnisotropy` specifies whether anisotropic filtering is supported. If this feature is not enabled, the `anisotropyEnable` member of the VkSamplerCreateInfo structure **must** be `VK_FALSE`.
- `textureCompressionETC2` specifies whether all of the ETC2 and EAC compressed texture formats are supported. If this feature is enabled, then the `VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT`, `VK_FORMAT_FEATURE_BLIT_SRC_BIT` and `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT` features **must** be supported in `optimalTilingFeatures` for the following formats:
    - `VK_FORMAT_ETC2_R8G8B8_UNORM_BLOCK`
    - `VK_FORMAT_ETC2_R8G8B8_SRGB_BLOCK`
    - `VK_FORMAT_ETC2_R8G8B8A1_UNORM_BLOCK`
    - `VK_FORMAT_ETC2_R8G8B8A1_SRGB_BLOCK`
    - `VK_FORMAT_ETC2_R8G8B8A8_UNORM_BLOCK`
    - `VK_FORMAT_ETC2_R8G8B8A8_SRGB_BLOCK`
    - `VK_FORMAT_EAC_R11_UNORM_BLOCK`
    - `VK_FORMAT_EAC_R11_SNORM_BLOCK`
    - `VK_FORMAT_EAC_R11G11_UNORM_BLOCK`
    - `VK_FORMAT_EAC_R11G11_SNORM_BLOCK`

    To query for additional properties, or if the feature is not enabled, vkGetPhysicalDeviceFormatProperties and vkGetPhysicalDeviceImageFormatProperties **can** be used to check for supported properties of individual formats as normal.
- `textureCompressionASTC_LDR` specifies whether all of the ASTC LDR compressed texture formats are supported. If this feature is enabled, then the `VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT`, `VK_FORMAT_FEATURE_BLIT_SRC_BIT` and `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT` features **must** be supported in `optimalTilingFeatures` for the following formats:
    - `VK_FORMAT_ASTC_4x4_UNORM_BLOCK`
    - `VK_FORMAT_ASTC_4x4_SRGB_BLOCK`
    - `VK_FORMAT_ASTC_5x4_UNORM_BLOCK`
    - `VK_FORMAT_ASTC_5x4_SRGB_BLOCK`
    - `VK_FORMAT_ASTC_5x5_UNORM_BLOCK`
    - `VK_FORMAT_ASTC_5x5_SRGB_BLOCK`
    - `VK_FORMAT_ASTC_6x5_UNORM_BLOCK`
    - `VK_FORMAT_ASTC_6x5_SRGB_BLOCK`

- VK_FORMAT_ASTC_6x6_UNORM_BLOCK
- VK_FORMAT_ASTC_6x6_SRGB_BLOCK
- VK_FORMAT_ASTC_8x5_UNORM_BLOCK
- VK_FORMAT_ASTC_8x5_SRGB_BLOCK
- VK_FORMAT_ASTC_8x6_UNORM_BLOCK
- VK_FORMAT_ASTC_8x6_SRGB_BLOCK
- VK_FORMAT_ASTC_8x8_UNORM_BLOCK
- VK_FORMAT_ASTC_8x8_SRGB_BLOCK
- VK_FORMAT_ASTC_10x5_UNORM_BLOCK
- VK_FORMAT_ASTC_10x5_SRGB_BLOCK
- VK_FORMAT_ASTC_10x6_UNORM_BLOCK
- VK_FORMAT_ASTC_10x6_SRGB_BLOCK
- VK_FORMAT_ASTC_10x8_UNORM_BLOCK
- VK_FORMAT_ASTC_10x8_SRGB_BLOCK
- VK_FORMAT_ASTC_10x10_UNORM_BLOCK
- VK_FORMAT_ASTC_10x10_SRGB_BLOCK
- VK_FORMAT_ASTC_12x10_UNORM_BLOCK
- VK_FORMAT_ASTC_12x10_SRGB_BLOCK
- VK_FORMAT_ASTC_12x12_UNORM_BLOCK
- VK_FORMAT_ASTC_12x12_SRGB_BLOCK

  To query for additional properties, or if the feature is not enabled, vkGetPhysicalDeviceFormatProperties and vkGetPhysicalDeviceImageFormatProperties **can** be used to check for supported properties of individual formats as normal.

- textureCompressionBC specifies whether all of the BC compressed texture formats are supported. If this feature is enabled, then the VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT, VK_FORMAT_FEATURE_BLIT_SRC_BIT and VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT features **must** be supported in optimalTilingFeatures for the following formats:
  - VK_FORMAT_BC1_RGB_UNORM_BLOCK
  - VK_FORMAT_BC1_RGB_SRGB_BLOCK
  - VK_FORMAT_BC1_RGBA_UNORM_BLOCK
  - VK_FORMAT_BC1_RGBA_SRGB_BLOCK
  - VK_FORMAT_BC2_UNORM_BLOCK
  - VK_FORMAT_BC2_SRGB_BLOCK
  - VK_FORMAT_BC3_UNORM_BLOCK
  - VK_FORMAT_BC3_SRGB_BLOCK

    - VK_FORMAT_BC4_UNORM_BLOCK
    - VK_FORMAT_BC4_SNORM_BLOCK
    - VK_FORMAT_BC5_UNORM_BLOCK
    - VK_FORMAT_BC5_SNORM_BLOCK
    - VK_FORMAT_BC6H_UFLOAT_BLOCK
    - VK_FORMAT_BC6H_SFLOAT_BLOCK
    - VK_FORMAT_BC7_UNORM_BLOCK
    - VK_FORMAT_BC7_SRGB_BLOCK

    To query for additional properties, or if the feature is not enabled, vkGetPhysicalDeviceFormatProperties and vkGetPhysicalDeviceImageFormatProperties **can** be used to check for supported properties of individual formats as normal.

- occlusionQueryPrecise specifies whether occlusion queries returning actual sample counts are supported. Occlusion queries are created in a VkQueryPool by specifying the queryType of VK_QUERY_TYPE_OCCLUSION in the VkQueryPoolCreateInfo structure which is passed to vkCreateQueryPool. If this feature is enabled, queries of this type **can** enable VK_QUERY_CONTROL_PRECISE_BIT in the flags parameter to vkCmdBeginQuery. If this feature is not supported, the implementation supports only boolean occlusion queries. When any samples are passed, boolean queries will return a non-zero result value, otherwise a result value of zero is returned. When this feature is enabled and VK_QUERY_CONTROL_PRECISE_BIT is set, occlusion queries will report the actual number of samples passed.
- pipelineStatisticsQuery specifies whether the pipeline statistics queries are supported. If this feature is not enabled, queries of type VK_QUERY_TYPE_PIPELINE_STATISTICS **cannot** be created, and none of the VkQueryPipelineStatisticFlagBits bits **can** be set in the pipelineStatistics member of the VkQueryPoolCreateInfo structure.
- vertexPipelineStoresAndAtomics specifies whether storage buffers and images support stores and atomic operations in the vertex, tessellation, and geometry shader stages. If this feature is not enabled, all storage image, storage texel buffer, and storage buffer variables used by these stages in shader modules **must** be decorated with the NonWritable decoration (or the readonly memory qualifier in GLSL).
- fragmentStoresAndAtomics specifies whether storage buffers and images support stores and atomic operations in the fragment shader stage. If this feature is not enabled, all storage image, storage texel buffer, and storage buffer variables used by the fragment stage in shader modules **must** be decorated with the NonWritable decoration (or the readonly memory qualifier in GLSL).
- shaderTessellationAndGeometryPointSize specifies whether the PointSize built-in decoration is available in the tessellation control, tessellation evaluation, and geometry shader stages. If this feature is not enabled, members decorated with the PointSize built-in decoration **must** not be read from or written to and all points written from a tessellation or geometry shader will have a size of 1.0. This also specifies whether shader modules **can** declare the TessellationPointSize capability for tessellation control and evaluation shaders, or if the shader modules **can** declare the GeometryPointSize capability for geometry shaders. An implementation supporting this feature **must** also support one or both of the tessellationShader or geometryShader features.

- `shaderImageGatherExtended` specifies whether the extended set of image gather instructions are available in shader code. If this feature is not enabled, the `OpImage*Gather` instructions do not support the `Offset` and `ConstOffsets` operands. This also specifies whether shader modules **can** declare the `ImageGatherExtended` capability.
- `shaderStorageImageExtendedFormats` specifies whether all the “storage image extended formats” below are supported; if this feature is supported, then the `VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT` **must** be supported in `optimalTilingFeatures` for the following formats:
  - `VK_FORMAT_R16G16_SFLOAT`
  - `VK_FORMAT_B10G11R11_UFLOAT_PACK32`
  - `VK_FORMAT_R16_SFLOAT`
  - `VK_FORMAT_R16G16B16A16_UNORM`
  - `VK_FORMAT_A2B10G10R10_UNORM_PACK32`
  - `VK_FORMAT_R16G16_UNORM`
  - `VK_FORMAT_R8G8_UNORM`
  - `VK_FORMAT_R16_UNORM`
  - `VK_FORMAT_R8_UNORM`
  - `VK_FORMAT_R16G16B16A16_SNORM`
  - `VK_FORMAT_R16G16_SNORM`
  - `VK_FORMAT_R8G8_SNORM`
  - `VK_FORMAT_R16_SNORM`
  - `VK_FORMAT_R8_SNORM`
  - `VK_FORMAT_R16G16_SINT`
  - `VK_FORMAT_R8G8_SINT`
  - `VK_FORMAT_R16_SINT`
  - `VK_FORMAT_R8_SINT`
  - `VK_FORMAT_A2B10G10R10_UINT_PACK32`
  - `VK_FORMAT_R16G16_UINT`
  - `VK_FORMAT_R8G8_UINT`
  - `VK_FORMAT_R16_UINT`
  - `VK_FORMAT_R8_UINT`

NOTE

`shaderStorageImageExtendedFormats` feature only adds a guarantee of format support, which is specified for the whole physical device. Therefore enabling or disabling the feature via vkCreateDevice has no practical effect.

To query for additional properties, or if the feature is not supported, vkGetPhysicalDeviceFormatProperties and vkGetPhysicalDeviceImageFormatProperties **can** be used to check for supported properties of individual formats, as usual rules allow.

> VK_FORMAT_R32G32_UINT, VK_FORMAT_R32G32_SINT, and VK_FORMAT_R32G32_SFLOAT from StorageImageExtendedFormats SPIR-V capability, are already covered by core Vulkan mandatory format support.

- shaderStorageImageMultisample specifies whether multisampled storage images are supported. If this feature is not enabled, images that are created with a usage that includes VK_IMAGE_USAGE_STORAGE_BIT **must** be created with samples equal to VK_SAMPLE_COUNT_1_BIT. This also specifies whether shader modules **can** declare the StorageImageMultisample and ImageMSArray capabilities.
- shaderStorageImageReadWithoutFormat specifies whether storage images and storage texel buffers require a format qualifier to be specified when reading. shaderStorageImageReadWithoutFormat applies only to formats listed in the storage without format list.
- shaderStorageImageWriteWithoutFormat specifies whether storage images and storage texel buffers require a format qualifier to be specified when writing. shaderStorageImageWriteWithoutFormat applies only to formats listed in the storage without format list.
- shaderUniformBufferArrayDynamicIndexing specifies whether arrays of uniform buffers **can** be indexed by integer expressions that are dynamically uniform within the invocation group in shader code. If this feature is not enabled, resources with a descriptor type of VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER or VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC **must** be indexed only by constant integral expressions when aggregated into arrays in shader code. This also specifies whether shader modules **can** declare the UniformBufferArrayDynamicIndexing capability.
- shaderSampledImageArrayDynamicIndexing specifies whether arrays of samplers or sampled images **can** be indexed by integer expressions that are dynamically uniform within the invocation group in shader code. If this feature is not enabled, resources with a descriptor type of VK_DESCRIPTOR_TYPE_SAMPLER, VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, or VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE **must** be indexed only by constant integral expressions when aggregated into arrays in shader code. This also specifies whether shader modules **can** declare the SampledImageArrayDynamicIndexing capability.
- shaderStorageBufferArrayDynamicIndexing specifies whether arrays of storage buffers **can** be indexed by integer expressions that are dynamically uniform within the invocation group in shader code. If this feature is not enabled, resources with a descriptor type of VK_DESCRIPTOR_TYPE_STORAGE_BUFFER or VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC **must** be indexed only by constant integral expressions when aggregated into arrays in shader code. This also specifies whether shader modules **can** declare the StorageBufferArrayDynamicIndexing capability.
- shaderStorageImageArrayDynamicIndexing specifies whether arrays of storage images **can** be indexed by integer expressions that are dynamically uniform within the invocation group in shader code. If this feature is not enabled, resources with a descriptor type of VK_DESCRIPTOR_TYPE_STORAGE_IMAGE **must** be indexed only by constant integral expressions when aggregated into arrays in shader code. This also specifies whether shader modules **can** declare the StorageImageArrayDynamicIndexing capability.
- shaderClipDistance specifies whether clip distances are supported in shader code. If this feature is not enabled, any members decorated with the ClipDistance built-in decoration **must** not be

read from or written to in shader modules. This also specifies whether shader modules **can** declare the `ClipDistance` capability.

- `shaderCullDistance` specifies whether cull distances are supported in shader code. If this feature is not enabled, any members decorated with the `CullDistance` built-in decoration **must** not be read from or written to in shader modules. This also specifies whether shader modules **can** declare the `CullDistance` capability.
- `shaderFloat64` specifies whether 64-bit floats (doubles) are supported in shader code. If this feature is not enabled, 64-bit floating-point types **must** not be used in shader code. This also specifies whether shader modules **can** declare the `Float64` capability. Declaring and using 64-bit floats is enabled for all storage classes that SPIR-V allows with the `Float64` capability.
- `shaderInt64` specifies whether 64-bit integers (signed and unsigned) are supported in shader code. If this feature is not enabled, 64-bit integer types **must** not be used in shader code. This also specifies whether shader modules **can** declare the `Int64` capability. Declaring and using 64-bit integers is enabled for all storage classes that SPIR-V allows with the `Int64` capability.
- `shaderInt16` specifies whether 16-bit integers (signed and unsigned) are supported in shader code. If this feature is not enabled, 16-bit integer types **must** not be used in shader code. This also specifies whether shader modules **can** declare the `Int16` capability. However, this only enables a subset of the storage classes that SPIR-V allows for the `Int16` SPIR-V capability: Declaring and using 16-bit integers in the `Private`, `Workgroup` (for non-Block variables), and `Function` storage classes is enabled, while declaring them in the interface storage classes (e.g., `UniformConstant`, `Uniform`, `StorageBuffer`, `Input`, `Output`, and `PushConstant`) is not enabled.
- `shaderResourceResidency` specifies whether image operations that return resource residency information are supported in shader code. If this feature is not enabled, the `OpImageSparse*` instructions **must** not be used in shader code. This also specifies whether shader modules **can** declare the `SparseResidency` capability. The feature requires at least one of the `sparseResidency*` features to be supported.
- `shaderResourceMinLod` specifies whether image operations specifying the minimum resource LOD are supported in shader code. If this feature is not enabled, the `MinLod` image operand **must** not be used in shader code. This also specifies whether shader modules **can** declare the `MinLod` capability.
- `sparseBinding` specifies whether resource memory **can** be managed at opaque sparse block level instead of at the object level. If this feature is not enabled, resource memory **must** be bound only on a per-object basis using the vkBindBufferMemory and vkBindImageMemory commands. In this case, buffers and images **must** not be created with `VK_BUFFER_CREATE_SPARSE_BINDING_BIT` and `VK_IMAGE_CREATE_SPARSE_BINDING_BIT` set in the `flags` member of the VkBufferCreateInfo and VkImageCreateInfo structures, respectively. Otherwise resource memory **can** be managed as described in Sparse Resource Features.
- `sparseResidencyBuffer` specifies whether the device **can** access partially resident buffers. If this feature is not enabled, buffers **must** not be created with `VK_BUFFER_CREATE_SPARSE_RESIDENCY_BIT` set in the `flags` member of the VkBufferCreateInfo structure.
- `sparseResidencyImage2D` specifies whether the device **can** access partially resident 2D images with 1 sample per pixel. If this feature is not enabled, images with an `imageType` of `VK_IMAGE_TYPE_2D` and `samples` set to `VK_SAMPLE_COUNT_1_BIT` **must** not be created with `VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT` set in the `flags` member of the VkImageCreateInfo

structure.

- sparseResidencyImage3D specifies whether the device **can** access partially resident 3D images. If this feature is not enabled, images with an imageType of VK_IMAGE_TYPE_3D **must** not be created with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT set in the flags member of the VkImageCreateInfo structure.
- sparseResidency2Samples specifies whether the physical device **can** access partially resident 2D images with 2 samples per pixel. If this feature is not enabled, images with an imageType of VK_IMAGE_TYPE_2D and samples set to VK_SAMPLE_COUNT_2_BIT **must** not be created with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT set in the flags member of the VkImageCreateInfo structure.
- sparseResidency4Samples specifies whether the physical device **can** access partially resident 2D images with 4 samples per pixel. If this feature is not enabled, images with an imageType of VK_IMAGE_TYPE_2D and samples set to VK_SAMPLE_COUNT_4_BIT **must** not be created with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT set in the flags member of the VkImageCreateInfo structure.
- sparseResidency8Samples specifies whether the physical device **can** access partially resident 2D images with 8 samples per pixel. If this feature is not enabled, images with an imageType of VK_IMAGE_TYPE_2D and samples set to VK_SAMPLE_COUNT_8_BIT **must** not be created with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT set in the flags member of the VkImageCreateInfo structure.
- sparseResidency16Samples specifies whether the physical device **can** access partially resident 2D images with 16 samples per pixel. If this feature is not enabled, images with an imageType of VK_IMAGE_TYPE_2D and samples set to VK_SAMPLE_COUNT_16_BIT **must** not be created with VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT set in the flags member of the VkImageCreateInfo structure.
- sparseResidencyAliased specifies whether the physical device **can** correctly access data aliased into multiple locations. If this feature is not enabled, the VK_BUFFER_CREATE_SPARSE_ALIASED_BIT and VK_IMAGE_CREATE_SPARSE_ALIASED_BIT enum values **must** not be used in flags members of the VkBufferCreateInfo and VkImageCreateInfo structures, respectively.
- variableMultisampleRate specifies whether all pipelines that will be bound to a command buffer during a subpass which uses no attachments **must** have the same value for VkPipelineMultisampleStateCreateInfo::rasterizationSamples. If set to VK_TRUE, the implementation supports variable multisample rates in a subpass which uses no attachments. If set to VK_FALSE, then all pipelines bound in such a subpass **must** have the same multisample rate. This has no effect in situations where a subpass uses any attachments.
- inheritedQueries specifies whether a secondary command buffer **may** be executed while a query is active.

The VkPhysicalDeviceVariablePointersFeatures structure is defined as:

```
typedef struct VkPhysicalDeviceVariablePointersFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           variablePointersStorageBuffer;
    VkBool32           variablePointers;
} VkPhysicalDeviceVariablePointersFeatures;
```

or the equivalent

```
// Provided by VK_KHR_variable_pointers
typedef VkPhysicalDeviceVariablePointersFeatures
VkPhysicalDeviceVariablePointersFeaturesKHR;
```

```
// Provided by VK_KHR_variable_pointers
typedef VkPhysicalDeviceVariablePointersFeatures
VkPhysicalDeviceVariablePointerFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `variablePointersStorageBuffer` specifies whether the implementation supports the SPIR-V `VariablePointersStorageBuffer` capability. When this feature is not enabled, shader modules **must** not declare the `SPV_KHR_variable_pointers` extension or the `VariablePointersStorageBuffer` capability.
- `variablePointers` specifies whether the implementation supports the SPIR-V `VariablePointers` capability. When this feature is not enabled, shader modules **must** not declare the `VariablePointers` capability.

If the `VkPhysicalDeviceVariablePointersFeatures` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceVariablePointersFeatures` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage

- VUID-VkPhysicalDeviceVariablePointersFeatures-variablePointers-01431
  If `variablePointers` is enabled then `variablePointersStorageBuffer` **must** also be enabled

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVariablePointersFeatures-sType-sType

sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VARIABLE_POINTERS_FEATURES

The VkPhysicalDeviceMultiviewFeatures structure is defined as:

```
typedef struct VkPhysicalDeviceMultiviewFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           multiview;
    VkBool32           multiviewGeometryShader;
    VkBool32           multiviewTessellationShader;
} VkPhysicalDeviceMultiviewFeatures;
```

or the equivalent

```
// Provided by VK_KHR_multiview
typedef VkPhysicalDeviceMultiviewFeatures VkPhysicalDeviceMultiviewFeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- multiview specifies whether the implementation supports multiview rendering within a render pass. If this feature is not enabled, the view mask of each subpass **must** always be zero.
- multiviewGeometryShader specifies whether the implementation supports multiview rendering within a render pass, with geometry shaders. If this feature is not enabled, then a pipeline compiled against a subpass with a non-zero view mask **must** not include a geometry shader.
- multiviewTessellationShader specifies whether the implementation supports multiview rendering within a render pass, with tessellation shaders. If this feature is not enabled, then a pipeline compiled against a subpass with a non-zero view mask **must** not include any tessellation shaders.

If the VkPhysicalDeviceMultiviewFeatures structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceMultiviewFeatures **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage

- VUID-VkPhysicalDeviceMultiviewFeatures-multiviewGeometryShader-00580
  If multiviewGeometryShader is enabled then multiview **must** also be enabled
- VUID-VkPhysicalDeviceMultiviewFeatures-multiviewTessellationShader-00581
  If multiviewTessellationShader is enabled then multiview **must** also be enabled

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMultiviewFeatures-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_FEATURES`

The VkPhysicalDeviceShaderAtomicFloatFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_shader_atomic_float
typedef struct VkPhysicalDeviceShaderAtomicFloatFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderBufferFloat32Atomics;
    VkBool32           shaderBufferFloat32AtomicAdd;
    VkBool32           shaderBufferFloat64Atomics;
    VkBool32           shaderBufferFloat64AtomicAdd;
    VkBool32           shaderSharedFloat32Atomics;
    VkBool32           shaderSharedFloat32AtomicAdd;
    VkBool32           shaderSharedFloat64Atomics;
    VkBool32           shaderSharedFloat64AtomicAdd;
    VkBool32           shaderImageFloat32Atomics;
    VkBool32           shaderImageFloat32AtomicAdd;
    VkBool32           sparseImageFloat32Atomics;
    VkBool32           sparseImageFloat32AtomicAdd;
} VkPhysicalDeviceShaderAtomicFloatFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `shaderBufferFloat32Atomics` indicates whether shaders **can** perform 32-bit floating-point load, store and exchange atomic operations on storage buffers.
- `shaderBufferFloat32AtomicAdd` indicates whether shaders **can** perform 32-bit floating-point add atomic operations on storage buffers.
- `shaderBufferFloat64Atomics` indicates whether shaders **can** perform 64-bit floating-point load, store and exchange atomic operations on storage buffers.
- `shaderBufferFloat64AtomicAdd` indicates whether shaders **can** perform 64-bit floating-point add atomic operations on storage buffers.
- `shaderSharedFloat32Atomics` indicates whether shaders **can** perform 32-bit floating-point load, store and exchange atomic operations on shared memory.
- `shaderSharedFloat32AtomicAdd` indicates whether shaders **can** perform 32-bit floating-point add atomic operations on shared memory.
- `shaderSharedFloat64Atomics` indicates whether shaders **can** perform 64-bit floating-point load, store and exchange atomic operations on shared memory.

- shaderSharedFloat64AtomicAdd indicates whether shaders **can** perform 64-bit floating-point add atomic operations on shared memory.
- shaderImageFloat32Atomics indicates whether shaders **can** perform 32-bit floating-point load, store and exchange atomic image operations.
- shaderImageFloat32AtomicAdd indicates whether shaders **can** perform 32-bit floating-point add atomic image operations.
- sparseImageFloat32Atomics indicates whether 32-bit floating-point load, store and exchange atomic operations **can** be used on sparse images.
- sparseImageFloat32AtomicAdd indicates whether 32-bit floating-point add atomic operations **can** be used on sparse images.

If the VkPhysicalDeviceShaderAtomicFloatFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderAtomicFloatFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderAtomicFloatFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_FLOAT_FEATURES_EXT

The VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT structure is defined as:

```
// Provided by VK_EXT_shader_atomic_float2
typedef struct VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderBufferFloat16Atomics;
    VkBool32           shaderBufferFloat16AtomicAdd;
    VkBool32           shaderBufferFloat16AtomicMinMax;
    VkBool32           shaderBufferFloat32AtomicMinMax;
    VkBool32           shaderBufferFloat64AtomicMinMax;
    VkBool32           shaderSharedFloat16Atomics;
    VkBool32           shaderSharedFloat16AtomicAdd;
    VkBool32           shaderSharedFloat16AtomicMinMax;
    VkBool32           shaderSharedFloat32AtomicMinMax;
    VkBool32           shaderSharedFloat64AtomicMinMax;
    VkBool32           shaderImageFloat32AtomicMinMax;
    VkBool32           sparseImageFloat32AtomicMinMax;
} VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.
- shaderBufferFloat16Atomics indicates whether shaders **can** perform 16-bit floating-point load, store, and exchange atomic operations on storage buffers.
- shaderBufferFloat16AtomicAdd indicates whether shaders **can** perform 16-bit floating-point add atomic operations on storage buffers.
- shaderBufferFloat16AtomicMinMax indicates whether shaders **can** perform 16-bit floating-point min and max atomic operations on storage buffers.
- shaderBufferFloat32AtomicMinMax indicates whether shaders **can** perform 32-bit floating-point min and max atomic operations on storage buffers.
- shaderBufferFloat64AtomicMinMax indicates whether shaders **can** perform 64-bit floating-point min and max atomic operations on storage buffers.
- shaderSharedFloat16Atomics indicates whether shaders **can** perform 16-bit floating-point load, store and exchange atomic operations on shared memory.
- shaderSharedFloat16AtomicAdd indicates whether shaders **can** perform 16-bit floating-point add atomic operations on shared memory.
- shaderSharedFloat16AtomicMinMax indicates whether shaders **can** perform 16-bit floating-point min and max atomic operations on shared memory.
- shaderSharedFloat32AtomicMinMax indicates whether shaders **can** perform 32-bit floating-point min and max atomic operations on shared memory.
- shaderSharedFloat64AtomicMinMax indicates whether shaders **can** perform 64-bit floating-point min and max atomic operations on shared memory.
- shaderImageFloat32AtomicMinMax indicates whether shaders **can** perform 32-bit floating-point min and max atomic image operations.
- sparseImageFloat32AtomicMinMax indicates whether 32-bit floating-point min and max atomic operations **can** be used on sparse images.

If the VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_FLOAT_2_FEATURES_EXT

The VkPhysicalDeviceShaderAtomicInt64Features structure is defined as:

```
typedef struct VkPhysicalDeviceShaderAtomicInt64Features {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderBufferInt64Atomics;
    VkBool32           shaderSharedInt64Atomics;
} VkPhysicalDeviceShaderAtomicInt64Features;
```

or the equivalent

```
// Provided by VK_KHR_shader_atomic_int64
typedef VkPhysicalDeviceShaderAtomicInt64Features
VkPhysicalDeviceShaderAtomicInt64FeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `shaderBufferInt64Atomics` indicates whether shaders **can** perform 64-bit unsigned and signed integer atomic operations on buffers.
- `shaderSharedInt64Atomics` indicates whether shaders **can** perform 64-bit unsigned and signed integer atomic operations on shared memory.

If the `VkPhysicalDeviceShaderAtomicInt64Features` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceShaderAtomicInt64Features` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderAtomicInt64Features-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_INT64_FEATURES`

The VkPhysicalDeviceShaderImageAtomicInt64FeaturesEXT structure is defined as:

```
// Provided by VK_EXT_shader_image_atomic_int64
typedef struct VkPhysicalDeviceShaderImageAtomicInt64FeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderImageInt64Atomics;
    VkBool32           sparseImageInt64Atomics;
} VkPhysicalDeviceShaderImageAtomicInt64FeaturesEXT;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderImageInt64Atomics indicates whether shaders **can** support 64-bit unsigned and signed integer atomic operations on images.
- sparseImageInt64Atomics indicates whether 64-bit integer atomics **can** be used on sparse images.

If the VkPhysicalDeviceShaderAtomicInt64FeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderAtomicInt64FeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderImageAtomicInt64FeaturesEXT-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_IMAGE_ATOMIC_INT64_FEATURES_EXT

The VkPhysicalDevice8BitStorageFeatures structure is defined as:

```
typedef struct VkPhysicalDevice8BitStorageFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           storageBuffer8BitAccess;
    VkBool32           uniformAndStorageBuffer8BitAccess;
    VkBool32           storagePushConstant8;
} VkPhysicalDevice8BitStorageFeatures;
```

or the equivalent

```
// Provided by VK_KHR_8bit_storage
typedef VkPhysicalDevice8BitStorageFeatures VkPhysicalDevice8BitStorageFeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- storageBuffer8BitAccess indicates whether objects in the StorageBuffer, ShaderRecordBufferKHR, or PhysicalStorageBuffer storage class with the Block decoration **can** have 8-bit integer members. If this feature is not enabled, 8-bit integer members **must** not be used in such objects. This also indicates whether shader modules **can** declare the StorageBuffer8BitAccess capability.
- uniformAndStorageBuffer8BitAccess indicates whether objects in the Uniform storage class with the Block decoration **can** have 8-bit integer members. If this feature is not enabled, 8-bit integer

members **must** not be used in such objects. This also indicates whether shader modules **can** declare the `UniformAndStorageBuffer8BitAccess` capability.

- `storagePushConstant8` indicates whether objects in the `PushConstant` storage class **can** have 8-bit integer members. If this feature is not enabled, 8-bit integer members **must** not be used in such objects. This also indicates whether shader modules **can** declare the `StoragePushConstant8` capability.

If the `VkPhysicalDevice8BitStorageFeatures` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDevice8BitStorageFeatures` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevice8BitStorageFeatures-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_8BIT_STORAGE_FEATURES`

The VkPhysicalDevice16BitStorageFeatures structure is defined as:

```
typedef struct VkPhysicalDevice16BitStorageFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           storageBuffer16BitAccess;
    VkBool32           uniformAndStorageBuffer16BitAccess;
    VkBool32           storagePushConstant16;
    VkBool32           storageInputOutput16;
} VkPhysicalDevice16BitStorageFeatures;
```

or the equivalent

```
// Provided by VK_KHR_16bit_storage
typedef VkPhysicalDevice16BitStorageFeatures VkPhysicalDevice16BitStorageFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `storageBuffer16BitAccess` specifies whether objects in the `StorageBuffer`, `ShaderRecordBufferKHR`, or `PhysicalStorageBuffer` storage class with the `Block` decoration **can** have 16-bit integer and 16-bit floating-point members. If this feature is not enabled, 16-bit integer or 16-bit floating-point members **must** not be used in such objects. This also specifies whether shader modules **can** declare the `StorageBuffer16BitAccess` capability.
- `uniformAndStorageBuffer16BitAccess` specifies whether objects in the `Uniform` storage class with the `Block` decoration **can** have 16-bit integer and 16-bit floating-point members. If this feature is

not enabled, 16-bit integer or 16-bit floating-point members **must** not be used in such objects. This also specifies whether shader modules **can** declare the `UniformAndStorageBuffer16BitAccess` capability.

- `storagePushConstant16` specifies whether objects in the `PushConstant` storage class **can** have 16-bit integer and 16-bit floating-point members. If this feature is not enabled, 16-bit integer or floating-point members **must** not be used in such objects. This also specifies whether shader modules **can** declare the `StoragePushConstant16` capability.
- `storageInputOutput16` specifies whether objects in the `Input` and `Output` storage classes **can** have 16-bit integer and 16-bit floating-point members. If this feature is not enabled, 16-bit integer or 16-bit floating-point members **must** not be used in such objects. This also specifies whether shader modules **can** declare the `StorageInputOutput16` capability.

If the `VkPhysicalDevice16BitStorageFeatures` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDevice16BitStorageFeatures` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevice16BitStorageFeatures-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_16BIT_STORAGE_FEATURES`

The `VkPhysicalDeviceShaderFloat16Int8Features` structure is defined as:

```
typedef struct VkPhysicalDeviceShaderFloat16Int8Features {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderFloat16;
    VkBool32           shaderInt8;
} VkPhysicalDeviceShaderFloat16Int8Features;
```

or the equivalent

```
// Provided by VK_KHR_shader_float16_int8
typedef VkPhysicalDeviceShaderFloat16Int8Features
VkPhysicalDeviceShaderFloat16Int8FeaturesKHR;
```

```
// Provided by VK_KHR_shader_float16_int8
typedef VkPhysicalDeviceShaderFloat16Int8Features
VkPhysicalDeviceFloat16Int8FeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.
- shaderFloat16 indicates whether 16-bit floats (halfs) are supported in shader code. This also indicates whether shader modules **can** declare the Float16 capability. However, this only enables a subset of the storage classes that SPIR-V allows for the Float16 SPIR-V capability: Declaring and using 16-bit floats in the Private, Workgroup (for non-Block variables), and Function storage classes is enabled, while declaring them in the interface storage classes (e.g., UniformConstant, Uniform, StorageBuffer, Input, Output, and PushConstant) is not enabled.
- shaderInt8 indicates whether 8-bit integers (signed and unsigned) are supported in shader code. This also indicates whether shader modules **can** declare the Int8 capability. However, this only enables a subset of the storage classes that SPIR-V allows for the Int8 SPIR-V capability: Declaring and using 8-bit integers in the Private, Workgroup (for non-Block variables), and Function storage classes is enabled, while declaring them in the interface storage classes (e.g., UniformConstant, Uniform, StorageBuffer, Input, Output, and PushConstant) is not enabled.

If the VkPhysicalDeviceShaderFloat16Int8Features structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderFloat16Int8Features **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderFloat16Int8Features-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT16_INT8_FEATURES

The VkPhysicalDeviceShaderClockFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_shader_clock
typedef struct VkPhysicalDeviceShaderClockFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderSubgroupClock;
    VkBool32           shaderDeviceClock;
} VkPhysicalDeviceShaderClockFeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderSubgroupClock indicates whether shaders **can** perform Subgroup scoped clock reads.
- shaderDeviceClock indicates whether shaders **can** perform Device scoped clock reads.

If the VkPhysicalDeviceShaderClockFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderClockFeaturesKHR

can also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderClockFeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_CLOCK_FEATURES_KHR

The VkPhysicalDeviceSamplerYcbcrConversionFeatures structure is defined as:

```
typedef struct VkPhysicalDeviceSamplerYcbcrConversionFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           samplerYcbcrConversion;
} VkPhysicalDeviceSamplerYcbcrConversionFeatures;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkPhysicalDeviceSamplerYcbcrConversionFeatures
VkPhysicalDeviceSamplerYcbcrConversionFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- samplerYcbcrConversion specifies whether the implementation supports sampler $Y'C_BC_R$ conversion. If samplerYcbcrConversion is VK_FALSE, sampler $Y'C_BC_R$ conversion is not supported, and samplers using sampler $Y'C_BC_R$ conversion **must** not be used.

If the VkPhysicalDeviceSamplerYcbcrConversionFeatures structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceSamplerYcbcrConversionFeatures **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceSamplerYcbcrConversionFeatures-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SAMPLER_YCBCR_CONVERSION_FEATURES

The VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_blend_operation_advanced
typedef struct VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           advancedBlendCoherentOperations;
} VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `advancedBlendCoherentOperations` specifies whether blending using advanced blend operations is guaranteed to execute atomically and in primitive order. If this is `VK_TRUE`, `VK_ACCESS_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT` is treated the same as `VK_ACCESS_COLOR_ATTACHMENT_READ_BIT`, and advanced blending needs no additional synchronization over basic blending. If this is `VK_FALSE`, then memory dependencies are required to guarantee order between two advanced blending operations that occur on the same sample.

If the `VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BLEND_OPERATION_ADVANCED_FEATURES_EXT`

The `VkPhysicalDeviceVertexAttributeDivisorFeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_vertex_attribute_divisor
typedef struct VkPhysicalDeviceVertexAttributeDivisorFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           vertexAttributeInstanceRateDivisor;
    VkBool32           vertexAttributeInstanceRateZeroDivisor;
} VkPhysicalDeviceVertexAttributeDivisorFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `vertexAttributeInstanceRateDivisor` specifies whether vertex attribute fetching may be repeated

in the case of instanced rendering.

- `vertexAttributeInstanceRateZeroDivisor` specifies whether a zero value for `VkVertexInputBindingDivisorDescriptionEXT::divisor` is supported.

If the `VkPhysicalDeviceVertexAttributeDivisorFeaturesKHR` structure is included in the `pNext` chain of the `VkPhysicalDeviceFeatures2` structure passed to `vkGetPhysicalDeviceFeatures2KHR`, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceVertexAttributeDivisorFeaturesKHR` **can** also be used in the `pNext` chain of `VkDeviceCreateInfo` to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVertexAttributeDivisorFeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_ATTRIBUTE_DIVISOR_FEATURES_KHR`

The `VkPhysicalDeviceASTCDecodeFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_astc_decode_mode
typedef struct VkPhysicalDeviceASTCDecodeFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           decodeModeSharedExponent;
} VkPhysicalDeviceASTCDecodeFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `decodeModeSharedExponent` indicates whether the implementation supports decoding ASTC compressed formats to `VK_FORMAT_E5B9G9R9_UFLOAT_PACK32` internal precision.

If the `VkPhysicalDeviceASTCDecodeFeaturesEXT` structure is included in the `pNext` chain of the `VkPhysicalDeviceFeatures2` structure passed to `vkGetPhysicalDeviceFeatures2KHR`, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceASTCDecodeFeaturesEXT` **can** also be used in the `pNext` chain of `VkDeviceCreateInfo` to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceASTCDecodeFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ASTC_DECODE_FEATURES_EXT`

The `VkPhysicalDeviceTransformFeedbackFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_transform_feedback
typedef struct VkPhysicalDeviceTransformFeedbackFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           transformFeedback;
    VkBool32           geometryStreams;
} VkPhysicalDeviceTransformFeedbackFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `transformFeedback` indicates whether the implementation supports transform feedback and shader modules **can** declare the `TransformFeedback` capability.
- `geometryStreams` indicates whether the implementation supports the `GeometryStreams` SPIR-V capability.

If the `VkPhysicalDeviceTransformFeedbackFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceTransformFeedbackFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceTransformFeedbackFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TRANSFORM_FEEDBACK_FEATURES_EXT`

The `VkPhysicalDeviceVulkanMemoryModelFeatures` structure is defined as:

```
typedef struct VkPhysicalDeviceVulkanMemoryModelFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           vulkanMemoryModel;
    VkBool32           vulkanMemoryModelDeviceScope;
    VkBool32           vulkanMemoryModelAvailabilityVisibilityChains;
} VkPhysicalDeviceVulkanMemoryModelFeatures;
```

or the equivalent

```
// Provided by VK_KHR_vulkan_memory_model
typedef VkPhysicalDeviceVulkanMemoryModelFeatures
VkPhysicalDeviceVulkanMemoryModelFeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- vulkanMemoryModel indicates whether shader modules **can** declare the VulkanMemoryModel capability.
- vulkanMemoryModelDeviceScope indicates whether the Vulkan Memory Model can use Device scope synchronization. This also indicates whether shader modules **can** declare the VulkanMemoryModelDeviceScope capability.
- vulkanMemoryModelAvailabilityVisibilityChains indicates whether the Vulkan Memory Model can use availability and visibility chains with more than one element.

If the VkPhysicalDeviceVulkanMemoryModelFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceVulkanMemoryModelFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVulkanMemoryModelFeatures-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_MEMORY_MODEL_FEATURES

The VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_compute_shader_derivatives
typedef struct VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           computeDerivativeGroupQuads;
    VkBool32           computeDerivativeGroupLinear;
} VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- computeDerivativeGroupQuads indicates that the implementation supports the ComputeDerivativeGroupQuadsKHR SPIR-V capability.
- computeDerivativeGroupLinear indicates that the implementation supports the ComputeDerivativeGroupLinearKHR SPIR-V capability.

See Quad shader scope for more information.

If the VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR. structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR. **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COMPUTE_SHADER_DERIVATIVES_FEATURES_KHR

The VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_fragment_shader_barycentric
typedef struct VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           fragmentShaderBarycentric;
} VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- fragmentShaderBarycentric indicates that the implementation supports the BaryCoordKHR and BaryCoordNoPerspKHR SPIR-V fragment shader built-ins and supports the PerVertexKHR SPIR-V decoration on fragment shader input variables.

See Barycentric Interpolation for more information.

If the VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_BARYCENTRIC_FEATURES_KHR

The VkPhysicalDeviceFragmentDensityMapFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_fragment_density_map
typedef struct VkPhysicalDeviceFragmentDensityMapFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           fragmentDensityMap;
    VkBool32           fragmentDensityMapDynamic;
    VkBool32           fragmentDensityMapNonSubsampledImages;
} VkPhysicalDeviceFragmentDensityMapFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fragmentDensityMap` specifies whether the implementation supports render passes with a fragment density map attachment. If this feature is not enabled and the `pNext` chain of VkRenderPassCreateInfo includes a VkRenderPassFragmentDensityMapCreateInfoEXT structure, `fragmentDensityMapAttachment` **must** be `VK_ATTACHMENT_UNUSED`.
- `fragmentDensityMapDynamic` specifies whether the implementation supports dynamic fragment density map image views. If this feature is not enabled, `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DYNAMIC_BIT_EXT` **must** not be included in VkImageViewCreateInfo::`flags`.
- `fragmentDensityMapNonSubsampledImages` specifies whether the implementation supports regular non-subsampled image attachments with fragment density map render passes. If this feature is not enabled, render passes with a fragment density map attachment **must** only have subsampled attachments bound.

If the `VkPhysicalDeviceFragmentDensityMapFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceFragmentDensityMapFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentDensityMapFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_FEATURES_EXT`

The `VkPhysicalDeviceFragmentDensityMap2FeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_fragment_density_map2
typedef struct VkPhysicalDeviceFragmentDensityMap2FeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           fragmentDensityMapDeferred;
} VkPhysicalDeviceFragmentDensityMap2FeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fragmentDensityMapDeferred` specifies whether the implementation supports deferred reads of fragment density map image views. If this feature is not enabled, `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DEFERRED_BIT_EXT` **must** not be included in `VkImageViewCreateInfo::flags`.

If the `VkPhysicalDeviceFragmentDensityMap2FeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceFragmentDensityMap2FeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentDensityMap2FeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_2_FEATURES_EXT`

The `VkPhysicalDeviceUniformBufferStandardLayoutFeatures` structure is defined as:

```
typedef struct VkPhysicalDeviceUniformBufferStandardLayoutFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           uniformBufferStandardLayout;
} VkPhysicalDeviceUniformBufferStandardLayoutFeatures;
```

or the equivalent

```
// Provided by VK_KHR_uniform_buffer_standard_layout
typedef VkPhysicalDeviceUniformBufferStandardLayoutFeatures
VkPhysicalDeviceUniformBufferStandardLayoutFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.
- uniformBufferStandardLayout indicates that the implementation supports the same layouts for uniform buffers as for storage and other kinds of buffers. See Standard Buffer Layout.

If the VkPhysicalDeviceUniformBufferStandardLayoutFeatures structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceUniformBufferStandardLayoutFeatures **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceUniformBufferStandardLayoutFeatures-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_UNIFORM_BUFFER_STANDARD_LAYOUT_FEATURES

The VkPhysicalDeviceDepthClipEnableFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_depth_clip_enable
typedef struct VkPhysicalDeviceDepthClipEnableFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           depthClipEnable;
} VkPhysicalDeviceDepthClipEnableFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- depthClipEnable indicates that the implementation supports setting the depth clipping operation explicitly via the VkPipelineRasterizationDepthClipStateCreateInfoEXT pipeline state. Otherwise depth clipping is only enabled when VkPipelineRasterizationStateCreateInfo::depthClampEnable is VK_FALSE.

If the VkPhysicalDeviceDepthClipEnableFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceDepthClipEnableFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDepthClipEnableFeaturesEXT-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLIP_ENABLE_FEATURES_EXT

The VkPhysicalDeviceMemoryPriorityFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_memory_priority
typedef struct VkPhysicalDeviceMemoryPriorityFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           memoryPriority;
} VkPhysicalDeviceMemoryPriorityFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- memoryPriority indicates that the implementation supports memory priorities specified at memory allocation time via VkMemoryPriorityAllocateInfoEXT.

If the VkPhysicalDeviceMemoryPriorityFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceMemoryPriorityFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMemoryPriorityFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PRIORITY_FEATURES_EXT

The VkPhysicalDeviceBufferDeviceAddressFeatures structure is defined as:

```
typedef struct VkPhysicalDeviceBufferDeviceAddressFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           bufferDeviceAddress;
    VkBool32           bufferDeviceAddressCaptureReplay;
    VkBool32           bufferDeviceAddressMultiDevice;
} VkPhysicalDeviceBufferDeviceAddressFeatures;
```

or the equivalent

```
// Provided by VK_KHR_buffer_device_address
typedef VkPhysicalDeviceBufferDeviceAddressFeatures
VkPhysicalDeviceBufferDeviceAddressFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `bufferDeviceAddress` indicates that the implementation supports accessing buffer memory in shaders as storage buffers via an address queried from vkGetBufferDeviceAddressKHR.
- `bufferDeviceAddressCaptureReplay` indicates that the implementation supports saving and reusing buffer and device addresses, e.g. for trace capture and replay.
- `bufferDeviceAddressMultiDevice` indicates that the implementation supports the `bufferDeviceAddress` , `rayTracingPipeline` and `rayQuery` features for logical devices created with multiple physical devices. If this feature is not supported, buffer and acceleration structure addresses **must** not be queried on a logical device created with more than one physical device.

NOTE	`bufferDeviceAddressMultiDevice` exists to allow certain legacy platforms to be able to support `bufferDeviceAddress` without needing to support shared GPU virtual addresses for multi-device configurations.

See vkGetBufferDeviceAddressKHR for more information.

If the `VkPhysicalDeviceBufferDeviceAddressFeatures` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceBufferDeviceAddressFeatures` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceBufferDeviceAddressFeatures-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BUFFER_DEVICE_ADDRESS_FEATURES`

The `VkPhysicalDeviceImagelessFramebufferFeatures` structure is defined as:

```
typedef struct VkPhysicalDeviceImagelessFramebufferFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           imagelessFramebuffer;
} VkPhysicalDeviceImagelessFramebufferFeatures;
```

or the equivalent

```
// Provided by VK_KHR_imageless_framebuffer
typedef VkPhysicalDeviceImagelessFramebufferFeatures
VkPhysicalDeviceImagelessFramebufferFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `imagelessFramebuffer` indicates that the implementation supports specifying the image view for attachments at render pass begin time via VkRenderPassAttachmentBeginInfo.

If the `VkPhysicalDeviceImagelessFramebufferFeatures` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceImagelessFramebufferFeatures` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceImagelessFramebufferFeatures-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGELESS_FRAMEBUFFER_FEATURES`

The `VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_fragment_shader_interlock
typedef struct VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           fragmentShaderSampleInterlock;
    VkBool32           fragmentShaderPixelInterlock;
    VkBool32           fragmentShaderShadingRateInterlock;
} VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `fragmentShaderSampleInterlock` indicates that the implementation supports the `FragmentShaderSampleInterlockEXT` SPIR-V capability.
- `fragmentShaderPixelInterlock` indicates that the implementation supports the `FragmentShaderPixelInterlockEXT` SPIR-V capability.
- `fragmentShaderShadingRateInterlock` indicates that the implementation supports the `FragmentShaderShadingRateInterlockEXT` SPIR-V capability.

If the `VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT-sType-sType sType must be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_INTERLOCK_FEATURES_EXT

The VkPhysicalDeviceCooperativeMatrixFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_cooperative_matrix
typedef struct VkPhysicalDeviceCooperativeMatrixFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           cooperativeMatrix;
    VkBool32           cooperativeMatrixRobustBufferAccess;
} VkPhysicalDeviceCooperativeMatrixFeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- cooperativeMatrix indicates that the implementation supports the CooperativeMatrixKHR SPIR-V capability.
- cooperativeMatrixRobustBufferAccess indicates that the implementation supports robust buffer access for SPIR-V OpCooperativeMatrixLoadKHR and OpCooperativeMatrixStoreKHR instructions.

If the VkPhysicalDeviceCooperativeMatrixFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceCooperativeMatrixFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceCooperativeMatrixFeaturesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COOPERATIVE_MATRIX_FEATURES_KHR

The VkPhysicalDeviceYcbcrImageArraysFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_ycbcr_image_arrays
typedef struct VkPhysicalDeviceYcbcrImageArraysFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           ycbcrImageArrays;
} VkPhysicalDeviceYcbcrImageArraysFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `ycbcrImageArrays` indicates that the implementation supports creating images with a format that requires $Y'C_BC_R$ conversion and has multiple array layers.

If the `VkPhysicalDeviceYcbcrImageArraysFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceYcbcrImageArraysFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceYcbcrImageArraysFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_YCBCR_IMAGE_ARRAYS_FEATURES_EXT`

The `VkPhysicalDeviceTimelineSemaphoreFeatures` structure is defined as:

```
typedef struct VkPhysicalDeviceTimelineSemaphoreFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           timelineSemaphore;
} VkPhysicalDeviceTimelineSemaphoreFeatures;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkPhysicalDeviceTimelineSemaphoreFeatures
VkPhysicalDeviceTimelineSemaphoreFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `timelineSemaphore` indicates whether semaphores created with a VkSemaphoreType of `VK_SEMAPHORE_TYPE_TIMELINE` are supported.

If the `VkPhysicalDeviceTimelineSemaphoreFeatures` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceTimelineSemaphoreFeatures` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceTimelineSemaphoreFeatures-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_FEATURES`

The `VkPhysicalDeviceIndexTypeUint8FeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_index_type_uint8
typedef struct VkPhysicalDeviceIndexTypeUint8FeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           indexTypeUint8;
} VkPhysicalDeviceIndexTypeUint8FeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `indexTypeUint8` indicates that `VK_INDEX_TYPE_UINT8_KHR` can be used with vkCmdBindIndexBuffer2KHR and vkCmdBindIndexBuffer.

If the `VkPhysicalDeviceIndexTypeUint8FeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceIndexTypeUint8FeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceIndexTypeUint8FeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_INDEX_TYPE_UINT8_FEATURES_KHR`

The `VkPhysicalDevicePrimitiveTopologyListRestartFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_primitive_topology_list_restart
typedef struct VkPhysicalDevicePrimitiveTopologyListRestartFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           primitiveTopologyListRestart;
    VkBool32           primitiveTopologyPatchListRestart;
} VkPhysicalDevicePrimitiveTopologyListRestartFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.
- primitiveTopologyListRestart indicates that list type primitives, VK_PRIMITIVE_TOPOLOGY_POINT_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, VK_PRIMITIVE_TOPOLOGY_LINE_LIST_WITH_ADJACENCY and VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST_WITH_ADJACENCY, **can** use the primitive restart index value in index buffers.
- primitiveTopologyPatchListRestart indicates that the VK_PRIMITIVE_TOPOLOGY_PATCH_LIST topology **can** use the primitive restart index value in index buffers.

If the VkPhysicalDevicePrimitiveTopologyListRestartFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDevicePrimitiveTopologyListRestartFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePrimitiveTopologyListRestartFeaturesEXT-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRIMITIVE_TOPOLOGY_LIST_RESTART_FEATURES_EXT

The VkPhysicalDeviceSeparateDepthStencilLayoutsFeatures structure is defined as:

```
typedef struct VkPhysicalDeviceSeparateDepthStencilLayoutsFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           separateDepthStencilLayouts;
} VkPhysicalDeviceSeparateDepthStencilLayoutsFeatures;
```

or the equivalent

```
// Provided by VK_KHR_separate_depth_stencil_layouts
typedef VkPhysicalDeviceSeparateDepthStencilLayoutsFeatures
VkPhysicalDeviceSeparateDepthStencilLayoutsFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- separateDepthStencilLayouts indicates whether the implementation supports a VkImageMemoryBarrier for a depth/stencil image with only one of VK_IMAGE_ASPECT_DEPTH_BIT or VK_IMAGE_ASPECT_STENCIL_BIT set, and whether VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL, VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL, VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL, or VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL can be used.

If the VkPhysicalDeviceSeparateDepthStencilLayoutsFeatures structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceSeparateDepthStencilLayoutsFeatures **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceSeparateDepthStencilLayoutsFeatures-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SEPARATE_DEPTH_STENCIL_LAYOUTS_FEATURES

The VkPhysicalDevicePipelineExecutablePropertiesFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_executable_properties
typedef struct VkPhysicalDevicePipelineExecutablePropertiesFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           pipelineExecutableInfo;
} VkPhysicalDevicePipelineExecutablePropertiesFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pipelineExecutableInfo indicates that the implementation supports reporting properties and statistics about the pipeline executables associated with a compiled pipeline.

If the VkPhysicalDevicePipelineExecutablePropertiesFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDevicePipelineExecutablePropertiesFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePipelineExecutablePropertiesFeaturesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_EXECUTABLE_PROPERTIES_FEATURES_KHR

The VkPhysicalDeviceAttachmentFeedbackLoopDynamicStateFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_attachment_feedback_loop_dynamic_state
typedef struct VkPhysicalDeviceAttachmentFeedbackLoopDynamicStateFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           attachmentFeedbackLoopDynamicState;
} VkPhysicalDeviceAttachmentFeedbackLoopDynamicStateFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `attachmentFeedbackLoopDynamicState` specifies whether dynamic feedback loops are supported.

If the `VkPhysicalDeviceAttachmentFeedbackLoopDynamicStateFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceAttachmentFeedbackLoopDynamicStateFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceAttachmentFeedbackLoopDynamicStateFeaturesEXT-sType-sType `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ATTACHMENT_FEEDBACK_LOOP_DYNAMIC_STATE_FEATURES_EXT`

The `VkPhysicalDeviceLegacyVertexAttributesFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_legacy_vertex_attributes
typedef struct VkPhysicalDeviceLegacyVertexAttributesFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           legacyVertexAttributes;
} VkPhysicalDeviceLegacyVertexAttributesFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `legacyVertexAttributes` specifies whether compatibility features for vertex attributes are supported when using dynamic vertex input state.

If the `VkPhysicalDeviceLegacyVertexAttributesFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceLegacyVertexAttributesFeaturesEXT` **can** also be used in the `pNext` chain of

VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceLegacyVertexAttributesFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LEGACY_VERTEX_ATTRIBUTES_FEATURES_EXT`

The `VkPhysicalDeviceLineRasterizationFeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_line_rasterization
typedef struct VkPhysicalDeviceLineRasterizationFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           rectangularLines;
    VkBool32           bresenhamLines;
    VkBool32           smoothLines;
    VkBool32           stippledRectangularLines;
    VkBool32           stippledBresenhamLines;
    VkBool32           stippledSmoothLines;
} VkPhysicalDeviceLineRasterizationFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `rectangularLines` indicates whether the implementation supports rectangular line rasterization.
- `bresenhamLines` indicates whether the implementation supports Bresenham-style line rasterization.
- `smoothLines` indicates whether the implementation supports smooth line rasterization.
- `stippledRectangularLines` indicates whether the implementation supports stippled line rasterization with `VK_LINE_RASTERIZATION_MODE_RECTANGULAR_KHR` lines.
- `stippledBresenhamLines` indicates whether the implementation supports stippled line rasterization with `VK_LINE_RASTERIZATION_MODE_BRESENHAM_KHR` lines.
- `stippledSmoothLines` indicates whether the implementation supports stippled line rasterization with `VK_LINE_RASTERIZATION_MODE_RECTANGULAR_SMOOTH_KHR` lines.

If the `VkPhysicalDeviceLineRasterizationFeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceLineRasterizationFeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceLineRasterizationFeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LINE_RASTERIZATION_FEATURES_KHR`

The `VkPhysicalDeviceAccelerationStructureFeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkPhysicalDeviceAccelerationStructureFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           accelerationStructure;
    VkBool32           accelerationStructureCaptureReplay;
    VkBool32           accelerationStructureIndirectBuild;
    VkBool32           accelerationStructureHostCommands;
    VkBool32           descriptorBindingAccelerationStructureUpdateAfterBind;
} VkPhysicalDeviceAccelerationStructureFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `accelerationStructure` indicates whether the implementation supports the acceleration structure functionality. See Acceleration Structures.
- `accelerationStructureCaptureReplay` indicates whether the implementation supports saving and reusing acceleration structure device addresses, e.g. for trace capture and replay.
- `accelerationStructureIndirectBuild` indicates whether the implementation supports indirect acceleration structure build commands, e.g. vkCmdBuildAccelerationStructuresIndirectKHR.
- `accelerationStructureHostCommands` indicates whether the implementation supports host side acceleration structure commands, e.g. vkBuildAccelerationStructuresKHR, vkCopyAccelerationStructureKHR, vkCopyAccelerationStructureToMemoryKHR, vkCopyMemoryToAccelerationStructureKHR, vkWriteAccelerationStructuresPropertiesKHR.
- `descriptorBindingAccelerationStructureUpdateAfterBind` indicates whether the implementation supports updating acceleration structure descriptors after a set is bound. If this feature is not enabled, `VK_DESCRIPTOR_BINDING_UPDATE_AFTER_BIND_BIT` **must** not be used with `VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR`.

If the `VkPhysicalDeviceAccelerationStructureFeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceAccelerationStructureFeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceAccelerationStructureFeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ACCELERATION_STRUCTURE_FEATURES_KHR`

The `VkPhysicalDeviceRayTracingPipelineFeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef struct VkPhysicalDeviceRayTracingPipelineFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           rayTracingPipeline;
    VkBool32           rayTracingPipelineShaderGroupHandleCaptureReplay;
    VkBool32           rayTracingPipelineShaderGroupHandleCaptureReplayMixed;
    VkBool32           rayTracingPipelineTraceRaysIndirect;
    VkBool32           rayTraversalPrimitiveCulling;
} VkPhysicalDeviceRayTracingPipelineFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `rayTracingPipeline` indicates whether the implementation supports the ray tracing pipeline functionality. See Ray Tracing.
- `rayTracingPipelineShaderGroupHandleCaptureReplay` indicates whether the implementation supports saving and reusing shader group handles, e.g. for trace capture and replay.
- `rayTracingPipelineShaderGroupHandleCaptureReplayMixed` indicates whether the implementation supports reuse of shader group handles being arbitrarily mixed with creation of non-reused shader group handles. If this is `VK_FALSE`, all reused shader group handles **must** be specified before any non-reused handles **may** be created.
- `rayTracingPipelineTraceRaysIndirect` indicates whether the implementation supports indirect ray tracing commands, e.g. vkCmdTraceRaysIndirectKHR.
- `rayTraversalPrimitiveCulling` indicates whether the implementation supports primitive culling during ray traversal.

If the `VkPhysicalDeviceRayTracingPipelineFeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceRayTracingPipelineFeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage

- VUID-VkPhysicalDeviceRayTracingPipelineFeaturesKHR-

rayTracingPipelineShaderGroupHandleCaptureReplayMixed-03575
If rayTracingPipelineShaderGroupHandleCaptureReplayMixed is VK_TRUE, rayTracingPipelineShaderGroupHandleCaptureReplay **must** also be VK_TRUE

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceRayTracingPipelineFeaturesKHR-sType-sType
sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_PIPELINE_FEATURES_KHR

The VkPhysicalDeviceRayQueryFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_ray_query
typedef struct VkPhysicalDeviceRayQueryFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           rayQuery;
} VkPhysicalDeviceRayQueryFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- rayQuery indicates whether the implementation supports ray query (OpRayQueryProceedKHR) functionality.

If the VkPhysicalDeviceRayQueryFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceRayQueryFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceRayQueryFeaturesKHR-sType-sType
sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_QUERY_FEATURES_KHR

The VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR structure is defined as:

```
// Provided by VK_KHR_ray_tracing_maintenance1
typedef struct VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           rayTracingMaintenance1;
    VkBool32           rayTracingPipelineTraceRaysIndirect2;
} VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `rayTracingMaintenance1` indicates that the implementation supports the following:
  - The `CullMaskKHR` SPIR-V builtin using the `SPV_KHR_ray_cull_mask` SPIR-V extension.
  - Additional acceleration structure property queries: `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR` and `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR`.
  - A new access flag `VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR`.
  - A new pipeline stage flag bit `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR`
- `rayTracingPipelineTraceRaysIndirect2` indicates whether the implementation supports the extended indirect ray tracing command vkCmdTraceRaysIndirect2KHR.

If the `VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR-sType-sType `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_MAINTENANCE_1_FEATURES_KHR`

The `VkPhysicalDeviceVideoEncodeQuantizationMapFeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_video_encode_quantization_map
typedef struct VkPhysicalDeviceVideoEncodeQuantizationMapFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           videoEncodeQuantizationMap;
} VkPhysicalDeviceVideoEncodeQuantizationMapFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `videoEncodeQuantizationMap` indicates that the implementation supports video encode quantization maps.

NOTE Support for `videoEncodeQuantizationMap` does not indicate that all video encode

profiles support quantization maps. Support for quantization maps for any specific video encode profile is subject to video-profile-specific capabilities.

If the VkPhysicalDeviceVideoEncodeQuantizationMapFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceVideoEncodeQuantizationMapFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVideoEncodeQuantizationMapFeaturesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_QUANTIZATION_MAP_FEATURES_KHR

The VkPhysicalDeviceVideoMaintenance1FeaturesKHR structure is defined as:

```
// Provided by VK_KHR_video_maintenance1
typedef struct VkPhysicalDeviceVideoMaintenance1FeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           videoMaintenance1;
} VkPhysicalDeviceVideoMaintenance1FeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- videoMaintenance1 indicates that the implementation supports the following:
  - The new buffer creation flag VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR.
  - The new image creation flag VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR.
  - The new video session creation flag VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR.

If the VkPhysicalDeviceVideoMaintenance1FeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceVideoMaintenance1FeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVideoMaintenance1FeaturesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_MAINTENANCE_1_FEATURES_KHR

The VkPhysicalDeviceVideoEncodeAV1FeaturesKHR structure is defined as:

```
// Provided by VK_KHR_video_encode_av1
typedef struct VkPhysicalDeviceVideoEncodeAV1FeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           videoEncodeAV1;
} VkPhysicalDeviceVideoEncodeAV1FeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- videoEncodeAV1 indicates that the implementation supports AV1 encode operations.

If the VkPhysicalDeviceVideoEncodeAV1FeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceVideoEncodeAV1FeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVideoEncodeAV1FeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_AV1_FEATURES_KHR

The VkPhysicalDeviceExtendedDynamicState3FeaturesEXT structure is defined as:

```
// Provided by VK_EXT_extended_dynamic_state3
typedef struct VkPhysicalDeviceExtendedDynamicState3FeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           extendedDynamicState3TessellationDomainOrigin;
    VkBool32           extendedDynamicState3DepthClampEnable;
    VkBool32           extendedDynamicState3PolygonMode;
    VkBool32           extendedDynamicState3RasterizationSamples;
    VkBool32           extendedDynamicState3SampleMask;
    VkBool32           extendedDynamicState3AlphaToCoverageEnable;
    VkBool32           extendedDynamicState3AlphaToOneEnable;
    VkBool32           extendedDynamicState3LogicOpEnable;
    VkBool32           extendedDynamicState3ColorBlendEnable;
    VkBool32           extendedDynamicState3ColorBlendEquation;
    VkBool32           extendedDynamicState3ColorWriteMask;
    VkBool32           extendedDynamicState3RasterizationStream;
    VkBool32           extendedDynamicState3ConservativeRasterizationMode;
    VkBool32           extendedDynamicState3ExtraPrimitiveOverestimationSize;
    VkBool32           extendedDynamicState3DepthClipEnable;
    VkBool32           extendedDynamicState3SampleLocationsEnable;
    VkBool32           extendedDynamicState3ColorBlendAdvanced;
    VkBool32           extendedDynamicState3ProvokingVertexMode;
    VkBool32           extendedDynamicState3LineRasterizationMode;
    VkBool32           extendedDynamicState3LineStippleEnable;
    VkBool32           extendedDynamicState3DepthClipNegativeOneToOne;
    VkBool32           extendedDynamicState3ViewportWScalingEnable;
    VkBool32           extendedDynamicState3ViewportSwizzle;
    VkBool32           extendedDynamicState3CoverageToColorEnable;
    VkBool32           extendedDynamicState3CoverageToColorLocation;
    VkBool32           extendedDynamicState3CoverageModulationMode;
    VkBool32           extendedDynamicState3CoverageModulationTableEnable;
    VkBool32           extendedDynamicState3CoverageModulationTable;
    VkBool32           extendedDynamicState3CoverageReductionMode;
    VkBool32           extendedDynamicState3RepresentativeFragmentTestEnable;
    VkBool32           extendedDynamicState3ShadingRateImageEnable;
} VkPhysicalDeviceExtendedDynamicState3FeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `extendedDynamicState3TessellationDomainOrigin` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT`
- `extendedDynamicState3DepthClampEnable` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT`

- `extendedDynamicState3PolygonMode` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_POLYGON_MODE_EXT`
- `extendedDynamicState3RasterizationSamples` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT`
- `extendedDynamicState3SampleMask` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_SAMPLE_MASK_EXT`
- `extendedDynamicState3AlphaToCoverageEnable` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT`
- `extendedDynamicState3AlphaToOneEnable` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT`
- `extendedDynamicState3LogicOpEnable` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT`
- `extendedDynamicState3ColorBlendEnable` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT`
- `extendedDynamicState3ColorBlendEquation` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT`
- `extendedDynamicState3ColorWriteMask` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT`
- `extendedDynamicState3RasterizationStream` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT`
- `extendedDynamicState3ConservativeRasterizationMode` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT`
- `extendedDynamicState3ExtraPrimitiveOverestimationSize` indicates that the implementation supports the following dynamic state:
  - `VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT`
- `extendedDynamicState3DepthClipEnable` indicates that the implementation supports the following dynamic state:

    - VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT
- extendedDynamicState3SampleLocationsEnable indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT
- extendedDynamicState3ColorBlendAdvanced indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT
- extendedDynamicState3ProvokingVertexMode indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT
- extendedDynamicState3LineRasterizationMode indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT
- extendedDynamicState3LineStippleEnable indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT
- extendedDynamicState3DepthClipNegativeOneToOne indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT
- extendedDynamicState3ViewportWScalingEnable indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_VIEWPORT_W_SCALING_ENABLE_NV
- extendedDynamicState3ViewportSwizzle indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_VIEWPORT_SWIZZLE_NV
- extendedDynamicState3CoverageToColorEnable indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_COVERAGE_TO_COLOR_ENABLE_NV
- extendedDynamicState3CoverageToColorLocation indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_COVERAGE_TO_COLOR_LOCATION_NV
- extendedDynamicState3CoverageModulationMode indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_COVERAGE_MODULATION_MODE_NV
- extendedDynamicState3CoverageModulationTableEnable indicates that the implementation supports the following dynamic state:
    - VK_DYNAMIC_STATE_COVERAGE_MODULATION_TABLE_ENABLE_NV
- extendedDynamicState3CoverageModulationTable indicates that the implementation supports the

following dynamic state:

  - VK_DYNAMIC_STATE_COVERAGE_MODULATION_TABLE_NV

- extendedDynamicState3CoverageReductionMode indicates that the implementation supports the following dynamic state:

  - VK_DYNAMIC_STATE_COVERAGE_REDUCTION_MODE_NV

- extendedDynamicState3RepresentativeFragmentTestEnable indicates that the implementation supports the following dynamic state:

  - VK_DYNAMIC_STATE_REPRESENTATIVE_FRAGMENT_TEST_ENABLE_NV

- extendedDynamicState3ShadingRateImageEnable indicates that the implementation supports the following dynamic state:

  - VK_DYNAMIC_STATE_SHADING_RATE_IMAGE_ENABLE_NV

If the VkPhysicalDeviceExtendedDynamicState3FeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceExtendedDynamicState3FeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceExtendedDynamicState3FeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTENDED_DYNAMIC_STATE_3_FEATURES_EXT

The VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           deviceGeneratedCommands;
    VkBool32           dynamicGeneratedPipelineLayout;
} VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- deviceGeneratedCommands indicates whether the implementation supports functionality to generate commands on the device.
- dynamicGeneratedPipelineLayout indicates the implementation allows the pipelineLayout member of VkIndirectCommandsLayoutCreateInfoEXT to be VK_NULL_HANDLE and VkPipelineLayoutCreateInfo **can** be chained off those structures' pNext instead.

If the VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_GENERATED_COMMANDS_FEATURES_EXT

The VkPhysicalDeviceDeviceMemoryReportFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_device_memory_report
typedef struct VkPhysicalDeviceDeviceMemoryReportFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           deviceMemoryReport;
} VkPhysicalDeviceDeviceMemoryReportFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- deviceMemoryReport indicates whether the implementation supports the ability to register device memory report callbacks.

If the VkPhysicalDeviceDeviceMemoryReportFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceDeviceMemoryReportFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDeviceMemoryReportFeaturesEXT-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_MEMORY_REPORT_FEATURES_EXT

The VkPhysicalDeviceGlobalPriorityQueryFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_global_priority
typedef struct VkPhysicalDeviceGlobalPriorityQueryFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           globalPriorityQuery;
} VkPhysicalDeviceGlobalPriorityQueryFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `globalPriorityQuery` indicates whether the implementation supports the ability to query global queue priorities.

If the `VkPhysicalDeviceGlobalPriorityQueryFeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceGlobalPriorityQueryFeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceGlobalPriorityQueryFeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GLOBAL_PRIORITY_QUERY_FEATURES_KHR`

The `VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeatures` structure is defined as:

```
typedef struct VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderZeroInitializeWorkgroupMemory;
} VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeatures;
```

or the equivalent

```
// Provided by VK_KHR_zero_initialize_workgroup_memory
typedef VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeatures
VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.

- shaderZeroInitializeWorkgroupMemory specifies whether the implementation supports initializing a variable in Workgroup storage class.

If the VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeatures structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeatures **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeatures-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ZERO_INITIALIZE_WORKGROUP_MEMORY_FEATURES

The VkPhysicalDeviceShaderSubgroupUniformControlFlowFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_shader_subgroup_uniform_control_flow
typedef struct VkPhysicalDeviceShaderSubgroupUniformControlFlowFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderSubgroupUniformControlFlow;
} VkPhysicalDeviceShaderSubgroupUniformControlFlowFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderSubgroupUniformControlFlow specifies whether the implementation supports the shader execution mode SubgroupUniformControlFlowKHR

If the VkPhysicalDeviceShaderSubgroupUniformControlFlowFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderSubgroupUniformControlFlowFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderSubgroupUniformControlFlowFeaturesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_UNIFORM_CONTROL_FLOW_FEATURES_KHR

The VkPhysicalDeviceRobustness2FeaturesEXT structure is defined as:

```
// Provided by VK_EXT_robustness2
typedef struct VkPhysicalDeviceRobustness2FeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           robustBufferAccess2;
    VkBool32           robustImageAccess2;
    VkBool32           nullDescriptor;
} VkPhysicalDeviceRobustness2FeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `robustBufferAccess2` indicates whether buffer accesses are tightly bounds-checked against the range of the descriptor. Uniform buffers **must** be bounds-checked to the range of the descriptor, where the range is rounded up to a multiple of `robustUniformBufferAccessSizeAlignment`. Storage buffers **must** be bounds-checked to the range of the descriptor, where the range is rounded up to a multiple of `robustStorageBufferAccessSizeAlignment`. Out of bounds buffer loads will return zero values, and image load, sample, and atomic operations from texel buffers will have (0,0,1) values inserted for missing G, B, or A components based on the format.
- `robustImageAccess2` indicates whether image accesses are tightly bounds-checked against the dimensions of the image view. Out of bounds image load, sample, and atomic operations from images will return zero values, with (0,0,1) values inserted for missing G, B, or A components based on the format.
- `nullDescriptor` indicates whether descriptors **can** be written with a VK_NULL_HANDLE resource or view, which are considered valid to access and act as if the descriptor were bound to nothing.

If the `VkPhysicalDeviceRobustness2FeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceRobustness2FeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage

- VUID-VkPhysicalDeviceRobustness2FeaturesEXT-robustBufferAccess2-04000
  If `robustBufferAccess2` is enabled then `robustBufferAccess` **must** also be enabled

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceRobustness2FeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ROBUSTNESS_2_FEATURES_EXT`

The `VkPhysicalDeviceShaderTerminateInvocationFeatures` structure is defined as:

```
typedef struct VkPhysicalDeviceShaderTerminateInvocationFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderTerminateInvocation;
} VkPhysicalDeviceShaderTerminateInvocationFeatures;
```

or the equivalent

```
// Provided by VK_KHR_shader_terminate_invocation
typedef VkPhysicalDeviceShaderTerminateInvocationFeatures
VkPhysicalDeviceShaderTerminateInvocationFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `shaderTerminateInvocation` specifies whether the implementation supports SPIR-V modules that use the `SPV_KHR_terminate_invocation` extension.

If the `VkPhysicalDeviceShaderTerminateInvocationFeatures` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceShaderTerminateInvocationFeatures` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderTerminateInvocationFeatures-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TERMINATE_INVOCATION_FEATURES`

The `VkPhysicalDeviceCustomBorderColorFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_custom_border_color
typedef struct VkPhysicalDeviceCustomBorderColorFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           customBorderColors;
    VkBool32           customBorderColorWithoutFormat;
} VkPhysicalDeviceCustomBorderColorFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.

- customBorderColors indicates that the implementation supports providing a borderColor value with one of the following values at sampler creation time:
  - VK_BORDER_COLOR_FLOAT_CUSTOM_EXT
  - VK_BORDER_COLOR_INT_CUSTOM_EXT
- customBorderColorWithoutFormat indicates that explicit formats are not required for custom border colors and the value of the format member of the VkSamplerCustomBorderColorCreateInfoEXT structure **may** be VK_FORMAT_UNDEFINED. If this feature bit is not set, applications **must** provide the VkFormat of the image view(s) being sampled by this sampler in the format member of the VkSamplerCustomBorderColorCreateInfoEXT structure.

If the VkPhysicalDeviceCustomBorderColorFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceCustomBorderColorFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceCustomBorderColorFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CUSTOM_BORDER_COLOR_FEATURES_EXT

The VkPhysicalDeviceBorderColorSwizzleFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_border_color_swizzle
typedef struct VkPhysicalDeviceBorderColorSwizzleFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           borderColorSwizzle;
    VkBool32           borderColorSwizzleFromImage;
} VkPhysicalDeviceBorderColorSwizzleFeaturesEXT;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- borderColorSwizzle indicates that defined values are returned by sampled image operations when used with a sampler that uses a VK_BORDER_COLOR_FLOAT_OPAQUE_BLACK, VK_BORDER_COLOR_INT_OPAQUE_BLACK, VK_BORDER_COLOR_FLOAT_CUSTOM_EXT, or VK_BORDER_COLOR_INT_CUSTOM_EXT borderColor and an image view that uses a non-identity component mapping, when either borderColorSwizzleFromImage is enabled or the VkSamplerBorderColorComponentMappingCreateInfoEXT is specified.
- borderColorSwizzleFromImage indicates that the implementation will return the correct border color values from sampled image operations under the conditions expressed above, without the

application having to specify the border color component mapping when creating the sampler object. If this feature bit is not set, applications **can** chain a VkSamplerBorderColorComponentMappingCreateInfoEXT structure when creating samplers for use with image views that do not have an identity swizzle and, when those samplers are combined with image views using the same component mapping, sampled image operations that use opaque black or custom border colors will return the correct border color values.

If the VkPhysicalDeviceBorderColorSwizzleFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceBorderColorSwizzleFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceBorderColorSwizzleFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BORDER_COLOR_SWIZZLE_FEATURES_EXT

The VkPhysicalDevicePortabilitySubsetFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_portability_subset
typedef struct VkPhysicalDevicePortabilitySubsetFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           constantAlphaColorBlendFactors;
    VkBool32           events;
    VkBool32           imageViewFormatReinterpretation;
    VkBool32           imageViewFormatSwizzle;
    VkBool32           imageView2DOn3DImage;
    VkBool32           multisampleArrayImage;
    VkBool32           mutableComparisonSamplers;
    VkBool32           pointPolygons;
    VkBool32           samplerMipLodBias;
    VkBool32           separateStencilMaskRef;
    VkBool32           shaderSampleRateInterpolationFunctions;
    VkBool32           tessellationIsolines;
    VkBool32           tessellationPointMode;
    VkBool32           triangleFans;
    VkBool32           vertexAttributeAccessBeyondStride;
} VkPhysicalDevicePortabilitySubsetFeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- constantAlphaColorBlendFactors indicates whether this implementation supports constant *alpha* Blend Factors used as source or destination *color* Blending.

- `events` indicates whether this implementation supports synchronization using Events.
- `imageViewFormatReinterpretation` indicates whether this implementation supports a `VkImageView` being created with a texel format containing a different number of components, or a different number of bits in each component, than the texel format of the underlying `VkImage`.
- `imageViewFormatSwizzle` indicates whether this implementation supports remapping format components using VkImageViewCreateInfo::`components`.
- `imageView2DOn3DImage` indicates whether this implementation supports a `VkImage` being created with the `VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT` flag set, permitting a 2D or 2D array image view to be created on a 3D `VkImage`.
- `multisampleArrayImage` indicates whether this implementation supports a `VkImage` being created as a 2D array with multiple samples per texel.
- `mutableComparisonSamplers` indicates whether this implementation allows descriptors with comparison samplers to be updated.
- `pointPolygons` indicates whether this implementation supports Rasterization using a *point* Polygon Mode.
- `samplerMipLodBias` indicates whether this implementation supports setting a mipmap LOD bias value when creating a sampler.
- `separateStencilMaskRef` indicates whether this implementation supports separate front and back Stencil Test reference values.
- `shaderSampleRateInterpolationFunctions` indicates whether this implementation supports fragment shaders which use the `InterpolationFunction` capability and the extended instructions `InterpolateAtCentroid`, `InterpolateAtOffset`, and `InterpolateAtSample` from the `GLSL.std.450` extended instruction set. This member is only meaningful if the `sampleRateShading` feature is supported.
- `tessellationIsolines` indicates whether this implementation supports isoline output from the Tessellation stage of a graphics pipeline. This member is only meaningful if the `tessellationShader` feature is supported.
- `tessellationPointMode` indicates whether this implementation supports point output from the Tessellation stage of a graphics pipeline. This member is only meaningful if the `tessellationShader` feature is supported.
- `triangleFans` indicates whether this implementation supports Triangle Fans primitive topology.
- `vertexAttributeAccessBeyondStride` indicates whether this implementation supports accessing a vertex input attribute beyond the stride of the corresponding vertex input binding.

If the `VkPhysicalDevicePortabilitySubsetFeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDevicePortabilitySubsetFeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePortabilitySubsetFeaturesKHR-sType-sType

sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PORTABILITY_SUBSET_FEATURES_KHR

The VkPhysicalDevicePerformanceQueryFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_performance_query
typedef struct VkPhysicalDevicePerformanceQueryFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           performanceCounterQueryPools;
    VkBool32           performanceCounterMultipleQueryPools;
} VkPhysicalDevicePerformanceQueryFeaturesKHR;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- performanceCounterQueryPools indicates whether the implementation supports performance counter query pools.
- performanceCounterMultipleQueryPools indicates whether the implementation supports using multiple performance query pools in a primary command buffer and secondary command buffers executed within it.

If the VkPhysicalDevicePerformanceQueryFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDevicePerformanceQueryFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePerformanceQueryFeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PERFORMANCE_QUERY_FEATURES_KHR

The VkPhysicalDeviceDepthClipControlFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_depth_clip_control
typedef struct VkPhysicalDeviceDepthClipControlFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           depthClipControl;
} VkPhysicalDeviceDepthClipControlFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.
- depthClipControl indicates that the implementation supports setting VkPipelineViewportDepthClipControlCreateInfoEXT::negativeOneToOne to VK_TRUE.

If the VkPhysicalDeviceDepthClipControlFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceDepthClipControlFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDepthClipControlFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLIP_CONTROL_FEATURES_EXT

The VkPhysicalDeviceDepthClampControlFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_depth_clamp_control
typedef struct VkPhysicalDeviceDepthClampControlFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           depthClampControl;
} VkPhysicalDeviceDepthClampControlFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- depthClampControl indicates that the implementation supports setting VkPipelineViewportDepthClampControlCreateInfoEXT::depthClampMode to VK_DEPTH_CLAMP_MODE_USER_DEFINED_RANGE_EXT.

If the VkPhysicalDeviceDepthClampControlFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceDepthClampControlFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDepthClampControlFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLAMP_CONTROL_FEATURES_EXT

The VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_workgroup_memory_explicit_layout
typedef struct VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           workgroupMemoryExplicitLayout;
    VkBool32           workgroupMemoryExplicitLayoutScalarBlockLayout;
    VkBool32           workgroupMemoryExplicitLayout8BitAccess;
    VkBool32           workgroupMemoryExplicitLayout16BitAccess;
} VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `workgroupMemoryExplicitLayout` indicates whether the implementation supports the SPIR-V `WorkgroupMemoryExplicitLayoutKHR` capability.
- `workgroupMemoryExplicitLayoutScalarBlockLayout` indicates whether the implementation supports scalar alignment for laying out Workgroup Blocks.
- `workgroupMemoryExplicitLayout8BitAccess` indicates whether objects in the `Workgroup` storage class with the `Block` decoration **can** have 8-bit integer members. If this feature is not enabled, 8-bit integer members **must** not be used in such objects. This also indicates whether shader modules **can** declare the `WorkgroupMemoryExplicitLayout8BitAccessKHR` capability.
- `workgroupMemoryExplicitLayout16BitAccess` indicates whether objects in the `Workgroup` storage class with the `Block` decoration **can** have 16-bit integer and 16-bit floating-point members. If this feature is not enabled, 16-bit integer or 16-bit floating-point members **must** not be used in such objects. This also indicates whether shader modules **can** declare the `WorkgroupMemoryExplicitLayout16BitAccessKHR` capability.

If the `VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR-sType-sType `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_WORKGROUP_MEMORY_EXPLICIT_LAYOUT_FEATURES_KHR`

The `VkPhysicalDeviceSynchronization2Features` structure is defined as:

```
typedef struct VkPhysicalDeviceSynchronization2Features {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           synchronization2;
} VkPhysicalDeviceSynchronization2Features;
```

or the equivalent

```
// Provided by VK_KHR_synchronization2
typedef VkPhysicalDeviceSynchronization2Features
VkPhysicalDeviceSynchronization2FeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- synchronization2 indicates whether the implementation supports the new set of synchronization commands introduced in VK_KHR_synchronization2.

If the VkPhysicalDeviceSynchronization2Features structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceSynchronization2Features **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceSynchronization2Features-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SYNCHRONIZATION_2_FEATURES

The VkPhysicalDeviceVertexInputDynamicStateFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_vertex_input_dynamic_state
typedef struct VkPhysicalDeviceVertexInputDynamicStateFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           vertexInputDynamicState;
} VkPhysicalDeviceVertexInputDynamicStateFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- `vertexInputDynamicState` indicates that the implementation supports the following dynamic states:
  - `VK_DYNAMIC_STATE_VERTEX_INPUT_EXT`

If the `VkPhysicalDeviceVertexInputDynamicStateFeaturesEXT` structure is included in the `pNext` chain of the `VkPhysicalDeviceFeatures2` structure passed to `vkGetPhysicalDeviceFeatures2KHR`, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceVertexInputDynamicStateFeaturesEXT` **can** also be used in the `pNext` chain of `VkDeviceCreateInfo` to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVertexInputDynamicStateFeaturesEXT-sType-sType `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_INPUT_DYNAMIC_STATE_FEATURES_EXT`

The `VkPhysicalDevicePrimitivesGeneratedQueryFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_primitives_generated_query
typedef struct VkPhysicalDevicePrimitivesGeneratedQueryFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           primitivesGeneratedQuery;
    VkBool32           primitivesGeneratedQueryWithRasterizerDiscard;
    VkBool32           primitivesGeneratedQueryWithNonZeroStreams;
} VkPhysicalDevicePrimitivesGeneratedQueryFeaturesEXT;
```

This structure describes the following features:

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `primitivesGeneratedQuery` indicates whether the implementation supports the `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT` query type.
- `primitivesGeneratedQueryWithRasterizerDiscard` indicates whether the implementation supports this query when rasterization discard is enabled.
- `primitivesGeneratedQueryWithNonZeroStreams` indicates whether the implementation supports this query with a non-zero index in `vkCmdBeginQueryIndexedEXT`.

If the `VkPhysicalDevicePrimitivesGeneratedQueryFeaturesEXT` structure is included in the `pNext` chain of the `VkPhysicalDeviceFeatures2` structure passed to `vkGetPhysicalDeviceFeatures2KHR`, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDevicePrimitivesGeneratedQueryFeaturesEXT` **can** also be used in the `pNext` chain of `VkDeviceCreateInfo` to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePrimitivesGeneratedQueryFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRIMITIVES_GENERATED_QUERY_FEATURES_EXT`

The `VkPhysicalDeviceFragmentShadingRateFeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_fragment_shading_rate
typedef struct VkPhysicalDeviceFragmentShadingRateFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           pipelineFragmentShadingRate;
    VkBool32           primitiveFragmentShadingRate;
    VkBool32           attachmentFragmentShadingRate;
} VkPhysicalDeviceFragmentShadingRateFeaturesKHR;
```

This structure describes the following features:

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pipelineFragmentShadingRate` indicates that the implementation supports the pipeline fragment shading rate.
- `primitiveFragmentShadingRate` indicates that the implementation supports the primitive fragment shading rate.
- `attachmentFragmentShadingRate` indicates that the implementation supports the attachment fragment shading rate.

If the `VkPhysicalDeviceFragmentShadingRateFeaturesKHR` structure is included in the `pNext` chain of the `VkPhysicalDeviceFeatures2` structure passed to `vkGetPhysicalDeviceFeatures2KHR`, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceFragmentShadingRateFeaturesKHR` **can** also be used in the `pNext` chain of `VkDeviceCreateInfo` to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentShadingRateFeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_FEATURES_KHR`

The `VkPhysicalDeviceColorWriteEnableFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_color_write_enable
typedef struct VkPhysicalDeviceColorWriteEnableFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           colorWriteEnable;
} VkPhysicalDeviceColorWriteEnableFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `colorWriteEnable` indicates that the implementation supports the dynamic state `VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT`.

If the `VkPhysicalDeviceColorWriteEnableFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceColorWriteEnableFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceColorWriteEnableFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COLOR_WRITE_ENABLE_FEATURES_EXT`

The `VkPhysicalDeviceProvokingVertexFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_provoking_vertex
typedef struct VkPhysicalDeviceProvokingVertexFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           provokingVertexLast;
    VkBool32           transformFeedbackPreservesProvokingVertex;
} VkPhysicalDeviceProvokingVertexFeaturesEXT;
```

This structure describes the following features:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `provokingVertexLast` indicates whether the implementation supports the `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT` provoking vertex mode for flat shading.
- `transformFeedbackPreservesProvokingVertex` indicates that the order of vertices within each primitive written by transform feedback will preserve the provoking vertex. This does not apply to triangle fan primitives when `transformFeedbackPreservesTriangleFanProvokingVertex` is

VK_FALSE. transformFeedbackPreservesProvokingVertex **must** be VK_FALSE when the VK_EXT_transform_feedback extension is not supported.

If the VkPhysicalDeviceProvokingVertexFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceProvokingVertexFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

When VkPhysicalDeviceProvokingVertexFeaturesEXT is in the pNext chain of VkDeviceCreateInfo but the transformFeedback feature is not enabled, the value of transformFeedbackPreservesProvokingVertex is ignored.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceProvokingVertexFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROVOKING_VERTEX_FEATURES_EXT

The VkPhysicalDeviceMultiDrawFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_multi_draw
typedef struct VkPhysicalDeviceMultiDrawFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           multiDraw;
} VkPhysicalDeviceMultiDrawFeaturesEXT;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- multiDraw indicates that the implementation supports vkCmdDrawMultiEXT and vkCmdDrawMultiIndexedEXT.

If the VkPhysicalDeviceMultiDrawFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceMultiDrawFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMultiDrawFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTI_DRAW_FEATURES_EXT

The VkPhysicalDeviceOpacityMicromapFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkPhysicalDeviceOpacityMicromapFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           micromap;
    VkBool32           micromapCaptureReplay;
    VkBool32           micromapHostCommands;
} VkPhysicalDeviceOpacityMicromapFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `micromap` indicates whether the implementation supports the micromap array feature.
- `micromapCaptureReplay` indicates whether the implementation supports capture and replay of addresses for micromap arrays.
- `micromapHostCommands` indicates whether the implementation supports host side micromap array commands.

If the `VkPhysicalDeviceOpacityMicromapFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceOpacityMicromapFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceOpacityMicromapFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_OPACITY_MICROMAP_FEATURES_EXT`

The `VkPhysicalDevicePresentIdFeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_present_id
typedef struct VkPhysicalDevicePresentIdFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           presentId;
} VkPhysicalDevicePresentIdFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `presentId` indicates that the implementation supports specifying present ID values in the

VkPresentIdKHR extension to the VkPresentInfoKHR struct.

If the VkPhysicalDevicePresentIdFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDevicePresentIdFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePresentIdFeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_ID_FEATURES_KHR

The VkPhysicalDevicePresentWaitFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_present_wait
typedef struct VkPhysicalDevicePresentWaitFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           presentWait;
} VkPhysicalDevicePresentWaitFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- presentWait indicates that the implementation supports vkWaitForPresentKHR.

If the VkPhysicalDevicePresentWaitFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDevicePresentWaitFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePresentWaitFeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_WAIT_FEATURES_KHR

The VkPhysicalDeviceHostImageCopyFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkPhysicalDeviceHostImageCopyFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           hostImageCopy;
} VkPhysicalDeviceHostImageCopyFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `hostImageCopy` indicates that the implementation supports copying from host memory to images using the vkCopyMemoryToImageEXT command, copying from images to host memory using the vkCopyImageToMemoryEXT command, and copying between images using the vkCopyImageToImageEXT command.

If the `VkPhysicalDeviceHostImageCopyFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceHostImageCopyFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceHostImageCopyFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_HOST_IMAGE_COPY_FEATURES_EXT`

The `VkPhysicalDeviceShaderIntegerDotProductFeatures` structure is defined as:

```
typedef struct VkPhysicalDeviceShaderIntegerDotProductFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderIntegerDotProduct;
} VkPhysicalDeviceShaderIntegerDotProductFeatures;
```

or the equivalent

```
// Provided by VK_KHR_shader_integer_dot_product
typedef VkPhysicalDeviceShaderIntegerDotProductFeatures
VkPhysicalDeviceShaderIntegerDotProductFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.
- shaderIntegerDotProduct specifies whether shader modules **can** declare the DotProductInputAllKHR, DotProductInput4x8BitKHR, DotProductInput4x8BitPackedKHR and DotProductKHR capabilities.

If the VkPhysicalDeviceShaderIntegerDotProductFeatures structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderIntegerDotProductFeatures **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderIntegerDotProductFeatures-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_FEATURES

The VkPhysicalDeviceMaintenance4Features structure is defined as:

```
typedef struct VkPhysicalDeviceMaintenance4Features {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           maintenance4;
} VkPhysicalDeviceMaintenance4Features;
```

or the equivalent

```
// Provided by VK_KHR_maintenance4
typedef VkPhysicalDeviceMaintenance4Features VkPhysicalDeviceMaintenance4FeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maintenance4 indicates that the implementation supports the following:
  - The application **may** destroy a VkPipelineLayout object immediately after using it to create another object.
  - LocalSizeId **can** be used as an alternative to LocalSize to specify the local workgroup size with specialization constants.
  - Images created with identical creation parameters will always have the same alignment requirements.
  - The size memory requirement of a buffer or image is never greater than that of another buffer or image created with a greater or equal size.

- Push constants do not have to be initialized before they are dynamically accessed.
- The interface matching rules allow a larger output vector to match with a smaller input vector, with additional values being discarded.

If the VkPhysicalDeviceMaintenance4Features structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceMaintenance4Features **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance4Features-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_FEATURES

The VkPhysicalDeviceMaintenance5FeaturesKHR structure is defined as:

```
// Provided by VK_KHR_maintenance5
typedef struct VkPhysicalDeviceMaintenance5FeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           maintenance5;
} VkPhysicalDeviceMaintenance5FeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maintenance5 indicates that the implementation supports the following:
  - The ability to expose support for the optional format VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR.
  - The ability to expose support for the optional format VK_FORMAT_A8_UNORM_KHR.
  - A property to indicate that multisample coverage operations are performed after sample counting in EarlyFragmentTests mode.
  - Creating a VkBufferView with a subset of the associated VkBuffer usage using VkBufferUsageFlags2CreateInfoKHR.
  - A new function vkCmdBindIndexBuffer2KHR, allowing a range of memory to be bound as an index buffer.
  - vkGetDeviceProcAddr will return NULL for function pointers of core functions for versions higher than the version requested by the application.
  - vkCmdBindVertexBuffers2EXT supports using VK_WHOLE_SIZE in the pSizes parameter.
  - If PointSize is not written, a default value of 1.0 is used for the size of points.
  - VkShaderModuleCreateInfo **can** be added as a chained structure to pipeline creation via VkPipelineShaderStageCreateInfo, rather than having to create a shader module.

- A function vkGetRenderingAreaGranularityKHR to query the optimal render area for a dynamic rendering instance.
- A property to indicate that depth/stencil texturing operations with VK_COMPONENT_SWIZZLE_ONE have defined behavior.
- vkGetDeviceImageSubresourceLayoutKHR allows an application to perform a vkGetImageSubresourceLayout query without having to create an image.
- VK_REMAINING_ARRAY_LAYERS as the layerCount member of VkImageSubresourceLayers.
- A property to indicate whether PointSize controls the final rasterization of polygons if polygon mode is VK_POLYGON_MODE_POINT.
- Two properties to indicate the non-strict line rasterization algorithm used.
- Two new flags words VkPipelineCreateFlagBits2KHR and VkBufferUsageFlagBits2KHR.
- Physical-device-level functions **can** now be called with any value in the valid range for a type beyond the defined enumerants, such that applications can avoid checking individual features, extensions, or versions before querying supported properties of a particular enumerant.
- Copies between images of any type are allowed, with 1D images treated as 2D images with a height of 1.

If the VkPhysicalDeviceMaintenance5FeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceMaintenance5FeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance5FeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_5_FEATURES_KHR

The VkPhysicalDeviceMaintenance6FeaturesKHR structure is defined as:

```
// Provided by VK_KHR_maintenance6
typedef struct VkPhysicalDeviceMaintenance6FeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           maintenance6;
} VkPhysicalDeviceMaintenance6FeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maintenance6 indicates that the implementation supports the following:

- VK_NULL_HANDLE **can** be used when binding an index buffer
- VkBindMemoryStatusKHR **can** be included in the pNext chain of the VkBindBufferMemoryInfo and VkBindImageMemoryInfo structures, enabling applications to retrieve VkResult values for individual memory binding operations.
- VkPhysicalDeviceMaintenance6PropertiesKHR::blockTexelViewCompatibleMultipleLayers property to indicate that the implementation supports creating image views with VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT where the layerCount member of subresourceRange is greater than 1.
- VkPhysicalDeviceMaintenance6PropertiesKHR::maxCombinedImageSamplerDescriptorCount property which indicates the maximum descriptor size required for any format that requires a sampler $Y'C_BC_R$ conversion supported by the implementation.
- A VkPhysicalDeviceMaintenance6PropertiesKHR::fragmentShadingRateClampCombinerInputs property which indicates whether the implementation clamps the inputs to fragment shading rate combiner operations.

If the VkPhysicalDeviceMaintenance6FeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceMaintenance6FeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance6FeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_6_FEATURES_KHR

The VkPhysicalDeviceMaintenance7FeaturesKHR structure is defined as:

```
// Provided by VK_KHR_maintenance7
typedef struct VkPhysicalDeviceMaintenance7FeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           maintenance7;
} VkPhysicalDeviceMaintenance7FeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maintenance7 indicates that the implementation supports the following:
  - The VK_RENDERING_CONTENTS_INLINE_BIT_KHR and VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_KHR flags **can** be used to record commands in render pass instances both inline and in secondary command buffers executed with vkCmdExecuteCommands for dynamic rendering and legacy render passes

respectively.

- Querying information regarding the underlying devices in environments where the Vulkan implementation is provided through layered implementations. This is done by chaining VkPhysicalDeviceLayeredApiPropertiesListKHR to VkPhysicalDeviceProperties2.
- New limits which indicate the maximum total count of dynamic uniform buffers and dynamic storage buffers that **can** be included in a pipeline layout.
- 32-bit timestamp queries **must** wrap on overflow
- A property that indicates whether a fragment shading rate attachment can have a size that is too small to cover a specified render area.
- A property that indicates support for writing to one aspect of a depth/stencil attachment without performing a read-modify-write operation on the other aspect

If the VkPhysicalDeviceMaintenance7FeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceMaintenance7FeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance7FeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_7_FEATURES_KHR

The VkPhysicalDeviceDynamicRenderingFeatures structure is defined as:

```
typedef struct VkPhysicalDeviceDynamicRenderingFeatures {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           dynamicRendering;
} VkPhysicalDeviceDynamicRenderingFeatures;
```

or the equivalent

```
// Provided by VK_KHR_dynamic_rendering
typedef VkPhysicalDeviceDynamicRenderingFeatures
VkPhysicalDeviceDynamicRenderingFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- dynamicRendering specifies that the implementation supports dynamic render pass instances using the vkCmdBeginRenderingKHR command.

If the VkPhysicalDeviceDynamicRenderingFeatures structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceDynamicRenderingFeatures **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDynamicRenderingFeatures-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_FEATURES

The VkPhysicalDeviceImageViewMinLodFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_image_view_min_lod
typedef struct VkPhysicalDeviceImageViewMinLodFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           minLod;
} VkPhysicalDeviceImageViewMinLodFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- minLod indicates whether the implementation supports clamping the minimum LOD value during Image Level(s) Selection, Texel Gathering and Integer Texel Coordinate Operations with a given VkImageView by VkImageViewMinLodCreateInfoEXT::minLod.

If the VkPhysicalDeviceImageViewMinLodFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceImageViewMinLodFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceImageViewMinLodFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_VIEW_MIN_LOD_FEATURES_EXT

The VkPhysicalDeviceAttachmentFeedbackLoopLayoutFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_attachment_feedback_loop_layout
typedef struct VkPhysicalDeviceAttachmentFeedbackLoopLayoutFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           attachmentFeedbackLoopLayout;
} VkPhysicalDeviceAttachmentFeedbackLoopLayoutFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `attachmentFeedbackLoopLayout` indicates whether the implementation supports using `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT` image layout for images created with `VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT`.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceAttachmentFeedbackLoopLayoutFeaturesEXT-sType-sType `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ATTACHMENT_FEEDBACK_LOOP_LAYOUT_FEATURES_EXT`

The `VkPhysicalDeviceNestedCommandBufferFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_nested_command_buffer
typedef struct VkPhysicalDeviceNestedCommandBufferFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           nestedCommandBuffer;
    VkBool32           nestedCommandBufferRendering;
    VkBool32           nestedCommandBufferSimultaneousUse;
} VkPhysicalDeviceNestedCommandBufferFeaturesEXT;
```

This structure describes the following features:

- `nestedCommandBuffer` indicates the implementation supports nested command buffers, which allows Secondary Command Buffers to execute other Secondary Command Buffers.
- `nestedCommandBufferRendering` indicates that it is valid to call vkCmdExecuteCommands inside a Secondary Command Buffer recorded with `VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT`.
- `nestedCommandBufferSimultaneousUse` indicates that the implementation supports nested command buffers with command buffers that are recorded with `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT`.

If the `VkPhysicalDeviceNestedCommandBufferFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported.

VkPhysicalDeviceNestedCommandBufferFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceNestedCommandBufferFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NESTED_COMMAND_BUFFER_FEATURES_EXT

The VkPhysicalDevicePipelineBinaryFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPhysicalDevicePipelineBinaryFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           pipelineBinaries;
} VkPhysicalDevicePipelineBinaryFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pipelineBinaries indicates that the implementation supports Pipeline Binaries.

If the VkPhysicalDevicePipelineBinaryFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDevicePipelineBinaryFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePipelineBinaryFeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_BINARY_FEATURES_KHR

The VkPhysicalDeviceImage2DViewOf3DFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_image_2d_view_of_3d
typedef struct VkPhysicalDeviceImage2DViewOf3DFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           image2DViewOf3D;
    VkBool32           sampler2DViewOf3D;
} VkPhysicalDeviceImage2DViewOf3DFeaturesEXT;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- image2DViewOf3D indicates that the implementation supports using a 2D view of a 3D image in a descriptor of type VK_DESCRIPTOR_TYPE_STORAGE_IMAGE if the image is created using VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT.
- sampler2DViewOf3D indicates that the implementation supports using a 2D view of a 3D image in a descriptor of type VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER if the image is created using VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT.

If the VkPhysicalDeviceImage2DViewOf3DFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceImage2DViewOf3DFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceImage2DViewOf3DFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_2D_VIEW_OF_3D_FEATURES_EXT

The VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_ray_tracing_position_fetch
typedef struct VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           rayTracingPositionFetch;
} VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- rayTracingPositionFetch indicates that the implementation supports fetching the object space vertex positions of a hit triangle.

If the VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_POSITION_FETCH_FEATURES_KHR`

The `VkPhysicalDeviceShaderFloatControls2FeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_shader_float_controls2
typedef struct VkPhysicalDeviceShaderFloatControls2FeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderFloatControls2;
} VkPhysicalDeviceShaderFloatControls2FeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `shaderFloatControls2` specifies whether shader modules **can** declare the `FloatControls2` capability.

If the `VkPhysicalDeviceShaderFloatControls2FeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceShaderFloatControls2FeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderFloatControls2FeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT_CONTROLS_2_FEATURES_KHR`

The `VkPhysicalDeviceNonSeamlessCubeMapFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_non_seamless_cube_map
typedef struct VkPhysicalDeviceNonSeamlessCubeMapFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           nonSeamlessCubeMap;
} VkPhysicalDeviceNonSeamlessCubeMapFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.
- nonSeamlessCubeMap indicates that the implementation supports VK_SAMPLER_CREATE_NON_SEAMLESS_CUBE_MAP_BIT_EXT.

If the VkPhysicalDeviceNonSeamlessCubeMapFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceNonSeamlessCubeMapFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceNonSeamlessCubeMapFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NON_SEAMLESS_CUBE_MAP_FEATURES_EXT

The VkPhysicalDeviceShaderModuleIdentifierFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_shader_module_identifier
typedef struct VkPhysicalDeviceShaderModuleIdentifierFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderModuleIdentifier;
} VkPhysicalDeviceShaderModuleIdentifierFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderModuleIdentifier indicates whether the implementation supports querying an identifier of a VkShaderModule or VkShaderModuleCreateInfo structure, and creating pipelines from identifiers only.

If the VkPhysicalDeviceShaderModuleIdentifierFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderModuleIdentifierFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderModuleIdentifierFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MODULE_IDENTIFIER_FEATURES_EXT

The VkPhysicalDeviceShaderTileImageFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_shader_tile_image
typedef struct VkPhysicalDeviceShaderTileImageFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderTileImageColorReadAccess;
    VkBool32           shaderTileImageDepthReadAccess;
    VkBool32           shaderTileImageStencilReadAccess;
} VkPhysicalDeviceShaderTileImageFeaturesEXT;
```

The members of the VkPhysicalDeviceShaderTileImageFeaturesEXT structure describe the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderTileImageColorReadAccess indicates that the implementation supports the TileImageColorReadAccessEXT SPIR-V capability.
- shaderTileImageDepthReadAccess indicates that the implementation supports the TileImageDepthReadAccessEXT SPIR-V capability.
- shaderTileImageStencilReadAccess indicates that the implementation supports the TileImageStencilReadAccessEXT SPIR-V capability.

If the VkPhysicalDeviceShaderTileImageFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderTileImageFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderTileImageFeaturesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TILE_IMAGE_FEATURES_EXT

The VkPhysicalDeviceDepthBiasControlFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_depth_bias_control
typedef struct VkPhysicalDeviceDepthBiasControlFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           depthBiasControl;
    VkBool32           leastRepresentableValueForceUnormRepresentation;
    VkBool32           floatRepresentation;
    VkBool32           depthBiasExact;
} VkPhysicalDeviceDepthBiasControlFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `depthBiasControl` indicates whether the implementation supports the `vkCmdSetDepthBias2EXT` command and the `VkDepthBiasRepresentationInfoEXT` structure.
- `leastRepresentableValueForceUnormRepresentation` indicates whether the implementation supports using the `VK_DEPTH_BIAS_REPRESENTATION_LEAST_REPRESENTABLE_VALUE_FORCE_UNORM_EXT` depth bias representation.
- `floatRepresentation` indicates whether the implementation supports using the `VK_DEPTH_BIAS_REPRESENTATION_FLOAT_EXT` depth bias representation.
- `depthBiasExact` indicates whether the implementation supports forcing depth bias to not be scaled to ensure a minimum resolvable difference using `VkDepthBiasRepresentationInfoEXT` ::`depthBiasExact`.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDepthBiasControlFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_BIAS_CONTROL_FEATURES_EXT`

The `VkPhysicalDeviceShaderObjectFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_shader_object
typedef struct VkPhysicalDeviceShaderObjectFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderObject;
} VkPhysicalDeviceShaderObjectFeaturesEXT;
```

This structure describes the following feature:

- `shaderObject` indicates whether the implementation supports shader objects.

If the `VkPhysicalDeviceShaderObjectFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceShaderObjectFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderObjectFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_OBJECT_FEATURES_EXT`

The `VkPhysicalDeviceFrameBoundaryFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_frame_boundary
typedef struct VkPhysicalDeviceFrameBoundaryFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           frameBoundary;
} VkPhysicalDeviceFrameBoundaryFeaturesEXT;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `frameBoundary` indicates whether the implementation supports frame boundary information.

If the `VkPhysicalDeviceFrameBoundaryFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceFrameBoundaryFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFrameBoundaryFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAME_BOUNDARY_FEATURES_EXT`

The `VkPhysicalDevicePresentModeFifoLatestReadyFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_present_mode_fifo_latest_ready
typedef struct VkPhysicalDevicePresentModeFifoLatestReadyFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           presentModeFifoLatestReady;
} VkPhysicalDevicePresentModeFifoLatestReadyFeaturesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `presentModeFifoLatestReady` specifies whether the implementation supports the `VK_PRESENT_MODE_FIFO_LATEST_READY_EXT` present mode.

If the `VkPhysicalDevicePresentModeFifoLatestReadyFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDevicePresentModeFifoLatestReadyFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePresentModeFifoLatestReadyFeaturesEXT-sType-sType sType must be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_MODE_FIFO_LATEST_READY_FEATURES_EXT

The VkPhysicalDeviceDynamicRenderingUnusedAttachmentsFeaturesEXT structure is defined as:

```
// Provided by VK_EXT_dynamic_rendering_unused_attachments
typedef struct VkPhysicalDeviceDynamicRenderingUnusedAttachmentsFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           dynamicRenderingUnusedAttachments;
} VkPhysicalDeviceDynamicRenderingUnusedAttachmentsFeaturesEXT;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- dynamicRenderingUnusedAttachments indicates that the implementation supports binding graphics pipelines within a render pass instance where any pipeline VkPipelineRenderingCreateInfo::pColorAttachmentFormats element with a format other than VK_FORMAT_UNDEFINED is allowed with a corresponding VkRenderingInfo::pColorAttachments element with an imageView equal to VK_NULL_HANDLE, or any pipeline VkPipelineRenderingCreateInfo::pColorAttachmentFormats element with a VK_FORMAT_UNDEFINED format is allowed with a corresponding VkRenderingInfo::pColorAttachments element with a non-VK_NULL_HANDLE imageView. Also a VkPipelineRenderingCreateInfo::depthAttachmentFormat other than VK_FORMAT_UNDEFINED is allowed with a VK_NULL_HANDLE VkRenderingInfo::pDepthAttachment, or a VkPipelineRenderingCreateInfo::depthAttachmentFormat of VK_FORMAT_UNDEFINED is allowed with a non-VK_NULL_HANDLE VkRenderingInfo::pDepthAttachment. Also a VkPipelineRenderingCreateInfo::stencilAttachmentFormat other than VK_FORMAT_UNDEFINED is allowed with a VK_NULL_HANDLE VkRenderingInfo::pStencilAttachment, or a VkPipelineRenderingCreateInfo::stencilAttachmentFormat of VK_FORMAT_UNDEFINED is allowed with a non-VK_NULL_HANDLE VkRenderingInfo::pStencilAttachment. Any writes to a VkRenderingInfo::pColorAttachments, VkRenderingInfo::pDepthAttachment, or VkRenderingInfo::pStencilAttachment with VK_NULL_HANDLE are discarded.

If the VkPhysicalDeviceDynamicRenderingUnusedAttachmentsFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceDynamicRenderingUnusedAttachmentsFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDynamicRenderingUnusedAttachmentsFeaturesEXT-sType-sType sType must be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_UNUSED_ATTACHMENTS_FEATURES_EXT

The VkPhysicalDeviceShaderMaximalReconvergenceFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_shader_maximal_reconvergence
typedef struct VkPhysicalDeviceShaderMaximalReconvergenceFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderMaximalReconvergence;
} VkPhysicalDeviceShaderMaximalReconvergenceFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderMaximalReconvergence specifies whether the implementation supports the shader execution mode MaximallyReconvergesKHR

If the VkPhysicalDevicePrivateDataFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDevicePrivateDataFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderMaximalReconvergenceFeaturesKHR-sType-sType sType must be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MAXIMAL_RECONVERGENCE_FEATURES_KHR

The VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_shader_subgroup_rotate
typedef struct VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderSubgroupRotate;
    VkBool32           shaderSubgroupRotateClustered;
} VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR;
```

- sType is a VkStructureType value identifying this structure.

- pNext is NULL or a pointer to a structure extending this structure.
- shaderSubgroupRotate specifies whether shader modules **can** declare the GroupNonUniformRotateKHR capability.
- shaderSubgroupRotateClustered specifies whether shader modules **can** use the ClusterSize operand to OpGroupNonUniformRotateKHR.

If the VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_ROTATE_FEATURES_KHR

The VkPhysicalDeviceShaderExpectAssumeFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_shader_expect_assume
typedef struct VkPhysicalDeviceShaderExpectAssumeFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderExpectAssume;
} VkPhysicalDeviceShaderExpectAssumeFeaturesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderExpectAssume specifies whether shader modules **can** declare the ExpectAssumeKHR capability.

If the VkPhysicalDeviceShaderExpectAssumeFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderExpectAssumeFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderExpectAssumeFeaturesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_EXPECT_ASSUME_FEATURES_KHR

The VkPhysicalDeviceDynamicRenderingLocalReadFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_dynamic_rendering_local_read
typedef struct VkPhysicalDeviceDynamicRenderingLocalReadFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           dynamicRenderingLocalRead;
} VkPhysicalDeviceDynamicRenderingLocalReadFeaturesKHR;
```

This structure describes the following feature:

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `dynamicRenderingLocalRead` specifies that the implementation supports local reads inside dynamic render pass instances using the vkCmdBeginRenderingKHR command.

If the `VkPhysicalDeviceDynamicRenderingLocalReadFeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceDynamicRenderingLocalReadFeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDynamicRenderingLocalReadFeaturesKHR-sType-sType `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_LOCAL_READ_FEATURES_KHR`

The `VkPhysicalDeviceShaderQuadControlFeaturesKHR` structure is defined as:

```
// Provided by VK_KHR_shader_quad_control
typedef struct VkPhysicalDeviceShaderQuadControlFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderQuadControl;
} VkPhysicalDeviceShaderQuadControlFeaturesKHR;
```

This structure describes the following features:

- `shaderQuadControl` indicates whether the implementation supports shaders with the `QuadControlKHR` capability.

If the `VkPhysicalDeviceShaderQuadControlFeaturesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceShaderQuadControlFeaturesKHR` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderQuadControlFeaturesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_QUAD_CONTROL_FEATURES_KHR`

The `VkPhysicalDeviceMapMemoryPlacedFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_map_memory_placed
typedef struct VkPhysicalDeviceMapMemoryPlacedFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           memoryMapPlaced;
    VkBool32           memoryMapRangePlaced;
    VkBool32           memoryUnmapReserve;
} VkPhysicalDeviceMapMemoryPlacedFeaturesEXT;
```

This structure describes the following features:

- `memoryMapPlaced` indicates that the implementation supports placing memory maps at application-specified virtual addresses.
- `memoryMapRangePlaced` indicates that the implementation supports placing memory maps of a subrange of a memory object at application-specified virtual addresses.
- `memoryUnmapReserve` indicates that the implementation supports leaving the memory range reserved when unmapping a memory object.

If the `VkPhysicalDeviceMapMemoryPlacedFeaturesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. `VkPhysicalDeviceMapMemoryPlacedFeaturesEXT` **can** also be used in the `pNext` chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMapMemoryPlacedFeaturesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAP_MEMORY_PLACED_FEATURES_EXT`

The `VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT` structure is defined as:

```
// Provided by VK_EXT_shader_replicated_composites
typedef struct VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderReplicatedComposites;
} VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderReplicatedComposites specifies whether shader modules **can** declare the ReplicatedCompositesEXT capability.

If the VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_REPLICATED_COMPOSITES_FEATURES_EXT

The VkPhysicalDeviceShaderRelaxedExtendedInstructionFeaturesKHR structure is defined as:

```
// Provided by VK_KHR_shader_relaxed_extended_instruction
typedef struct VkPhysicalDeviceShaderRelaxedExtendedInstructionFeaturesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           shaderRelaxedExtendedInstruction;
} VkPhysicalDeviceShaderRelaxedExtendedInstructionFeaturesKHR;
```

This structure describes the following feature:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderRelaxedExtendedInstruction specifies whether the implementation supports SPIR-V modules that use the SPV_KHR_relaxed_extended_instruction extension.

If the VkPhysicalDeviceShaderRelaxedExtendedInstructionFeaturesKHR structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceShaderRelaxedExtendedInstructionFeaturesKHR **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderRelaxedExtendedInstructionFeaturesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_RELAXED_EXTENDED_INSTRUCTION_FEATURES_KHR

40.1. Feature Requirements

All Vulkan graphics implementations **must** support the following features:

- If Vulkan 1.0 is supported, `robustBufferAccess` **must** be supported if `VK_KHR_portability_subset` is not advertised
- If `VK_KHR_dynamic_rendering` is supported, `dynamicRendering` **must** be supported
- If `VK_KHR_multiview` is supported, `multiview` **must** be supported
- If `VK_KHR_shader_float16_int8` is supported, `shaderFloat16`, or `shaderInt8` **must** be supported
- If `VK_KHR_16bit_storage` is supported, `storageBuffer16BitAccess` **must** be supported
- If `VK_KHR_imageless_framebuffer` is supported, `imagelessFramebuffer` **must** be supported
- If `VK_KHR_performance_query` is supported, `performanceCounterQueryPools` **must** be supported
- If `VK_KHR_variable_pointers` is supported, `variablePointersStorageBuffer` **must** be supported
- If `VK_KHR_sampler_ycbcr_conversion` is supported, `samplerYcbcrConversion` **must** be supported
- If `VK_KHR_shader_subgroup_extended_types` is supported, `shaderSubgroupExtendedTypes` **must** be supported
- If `VK_KHR_8bit_storage` is supported, `storageBuffer8BitAccess` **must** be supported
- If `VK_KHR_shader_atomic_int64` is supported, `shaderBufferInt64Atomics` **must** be supported
- If `VK_KHR_shader_clock` is supported, `shaderSubgroupClock` **must** be supported
- If `VK_KHR_global_priority` is supported, `globalPriorityQuery` **must** be supported
- If `VK_KHR_timeline_semaphore` is supported, `timelineSemaphore` **must** be supported
- If `VK_KHR_vulkan_memory_model` is supported, `vulkanMemoryModel` **must** be supported
- If `VK_KHR_shader_terminate_invocation` is supported, `shaderTerminateInvocation` **must** be supported
- If `VK_KHR_fragment_shading_rate` is supported, `pipelineFragmentShadingRate` **must** be supported
- If `VK_KHR_dynamic_rendering_local_read` is supported, `dynamicRenderingLocalRead` **must** be supported
- If `VK_KHR_shader_quad_control` is supported, `shaderQuadControl` **must** be supported
- If `VK_KHR_separate_depth_stencil_layouts` is supported, `separateDepthStencilLayouts` **must** be supported
- If `VK_KHR_present_wait` is supported, `presentWait` **must** be supported
- If `VK_KHR_uniform_buffer_standard_layout` is supported, `uniformBufferStandardLayout` **must** be supported
- If `VK_KHR_buffer_device_address` is supported, `bufferDeviceAddress` **must** be supported
- If `VK_KHR_pipeline_executable_properties` is supported, `pipelineExecutableInfo` **must** be supported
- If `VK_KHR_shader_integer_dot_product` is supported, `shaderIntegerDotProduct` **must** be supported
- If `VK_KHR_present_id` is supported, `presentId` **must** be supported

- If VK_KHR_synchronization2 is supported, synchronization2 **must** be supported
- If VK_KHR_fragment_shader_barycentric is supported, fragmentShaderBarycentric **must** be supported
- If VK_KHR_shader_subgroup_uniform_control_flow is supported, shaderSubgroupUniformControlFlow **must** be supported
- If VK_KHR_zero_initialize_workgroup_memory is supported, shaderZeroInitializeWorkgroupMemory **must** be supported
- If VK_KHR_workgroup_memory_explicit_layout is supported, workgroupMemoryExplicitLayout **must** be supported
- If VK_KHR_ray_tracing_maintenance1 is supported, rayTracingMaintenance1 **must** be supported
- If VK_KHR_maintenance4 is supported, maintenance4 **must** be supported
- If VK_KHR_shader_subgroup_rotate is supported, shaderSubgroupRotate **must** be supported
- If VK_KHR_shader_maximal_reconvergence is supported, shaderMaximalReconvergence **must** be supported
- If VK_KHR_maintenance5 is supported, maintenance5 **must** be supported
- If VK_KHR_ray_tracing_position_fetch is supported, rayTracingPositionFetch **must** be supported
- If VK_KHR_pipeline_binary is supported, pipelineBinaries **must** be supported
- If VK_KHR_cooperative_matrix is supported, cooperativeMatrix **must** be supported
- If VK_KHR_compute_shader_derivatives is supported, computeDerivativeGroupLinear **must** be supported
- If VK_KHR_video_encode_av1 is supported, videoEncodeAV1 **must** be supported
- If VK_KHR_video_maintenance1 is supported, videoMaintenance1 **must** be supported
- If VK_KHR_vertex_attribute_divisor is supported, vertexAttributeInstanceRateDivisor **must** be supported
- If VK_KHR_shader_float_controls2 is supported, shaderFloatControls2 **must** be supported
- If VK_KHR_index_type_uint8 is supported, indexTypeUint8 **must** be supported
- If VK_KHR_line_rasterization is supported, rectangularLines, bresenhamLines, smoothLines, stippledRectangularLines, stippledBresenhamLines, or stippledSmoothLines **must** be supported
- If VK_KHR_shader_expect_assume is supported, shaderExpectAssume **must** be supported
- If VK_KHR_maintenance6 is supported, maintenance6 **must** be supported
- If VK_KHR_video_encode_quantization_map is supported, videoEncodeQuantizationMap **must** be supported
- If VK_KHR_shader_relaxed_extended_instruction is supported, shaderRelaxedExtendedInstruction **must** be supported
- If VK_KHR_maintenance7 is supported, maintenance7 **must** be supported
- If VK_EXT_transform_feedback is supported, transformFeedback **must** be supported
- If VK_EXT_depth_clip_enable is supported, depthClipEnable **must** be supported

- If VK_EXT_fragment_density_map is supported, fragmentDensityMap **must** be supported
- If VK_EXT_shader_image_atomic_int64 is supported, the following features **must** be supported:
  - shaderInt64
  - shaderImageInt64Atomics
- If VK_EXT_memory_priority is supported, memoryPriority **must** be supported
- If VK_EXT_fragment_shader_interlock is supported, fragmentShaderSampleInterlock, fragmentShaderPixelInterlock, or fragmentShaderShadingRateInterlock **must** be supported
- If VK_EXT_ycbcr_image_arrays is supported, ycbcrImageArrays **must** be supported
- If VK_EXT_provoking_vertex is supported, provokingVertexLast **must** be supported
- If VK_EXT_shader_atomic_float is supported, the following features **must** be supported:
  - shaderBufferFloat32Atomics, shaderBufferFloat32AtomicAdd, shaderBufferFloat64Atomics, shaderBufferFloat64AtomicAdd, shaderSharedFloat32Atomics, shaderSharedFloat32AtomicAdd, shaderSharedFloat64Atomics, shaderSharedFloat64AtomicAdd, shaderImageFloat32Atomics, or shaderImageFloat32AtomicAdd
  - shaderImageFloat32Atomics if sparseImageFloat32Atomics is supported
  - shaderImageFloat32AtomicAdd if sparseImageFloat32AtomicAdd is supported
- If VK_EXT_host_image_copy is supported, hostImageCopy **must** be supported
- If VK_EXT_map_memory_placed is supported, memoryMapPlaced **must** be supported
- If VK_EXT_shader_atomic_float2 is supported, the following features **must** be supported:
  - shaderBufferFloat16Atomics, shaderBufferFloat16AtomicAdd, shaderBufferFloat16AtomicMinMax, shaderBufferFloat32AtomicMinMax, shaderBufferFloat64AtomicMinMax, shaderSharedFloat16Atomics, shaderSharedFloat16AtomicAdd, shaderSharedFloat16AtomicMinMax, shaderSharedFloat32AtomicMinMax, shaderSharedFloat64AtomicMinMax, or shaderImageFloat32AtomicMinMax
  - shaderImageFloat32AtomicMinMax if sparseImageFloat32AtomicMinMax is supported
- If VK_EXT_depth_bias_control is supported, depthBiasControl **must** be supported
- If VK_EXT_device_memory_report is supported, deviceMemoryReport **must** be supported
- If VK_EXT_robustness2 is supported, robustBufferAccess2, robustImageAccess2, or nullDescriptor **must** be supported
- If VK_EXT_custom_border_color is supported, customBorderColors **must** be supported
- If VK_EXT_fragment_density_map2 is supported, fragmentDensityMapDeferred **must** be supported
- If VK_EXT_attachment_feedback_loop_layout is supported, attachmentFeedbackLoopLayout **must** be supported
- If VK_EXT_vertex_input_dynamic_state is supported, vertexInputDynamicState **must** be supported
- If VK_EXT_depth_clip_control is supported, depthClipControl **must** be supported
- If VK_EXT_primitive_topology_list_restart is supported, primitiveTopologyListRestart **must** be supported
- If VK_EXT_present_mode_fifo_latest_ready is supported, presentModeFifoLatestReady **must** be

supported

- If VK_EXT_frame_boundary is supported, frameBoundary **must** be supported
- If VK_EXT_color_write_enable is supported, colorWriteEnable **must** be supported
- If VK_EXT_primitives_generated_query is supported, primitivesGeneratedQuery **must** be supported
- If VK_EXT_image_view_min_lod is supported, minLod **must** be supported
- If VK_EXT_multi_draw is supported, multiDraw **must** be supported
- If VK_EXT_image_2d_view_of_3d is supported, image2DViewOf3D **must** be supported
- If VK_EXT_shader_tile_image is supported, shaderTileImageColorReadAccess **must** be supported
- If VK_EXT_opacity_micromap is supported, micromap **must** be supported
- If VK_EXT_border_color_swizzle is supported, borderColorSwizzle **must** be supported
- If VK_EXT_non_seamless_cube_map is supported, nonSeamlessCubeMap **must** be supported
- If VK_EXT_nested_command_buffer is supported, nestedCommandBuffer **must** be supported
- If VK_EXT_extended_dynamic_state3 is supported, the following features **must** be supported:
  - tessellationShader if extendedDynamicState3TessellationDomainOrigin is supported
  - depthClamp if extendedDynamicState3DepthClampEnable is supported
  - fillModeNonSolid if extendedDynamicState3PolygonMode is supported
  - alphaToOne if extendedDynamicState3AlphaToOneEnable is supported
  - logicOp if extendedDynamicState3LogicOpEnable is supported
  - geometryStreams if VK_EXT_transform_feedback and extendedDynamicState3RasterizationStream are supported
- If VK_EXT_shader_module_identifier is supported, shaderModuleIdentifier **must** be supported
- If VK_EXT_shader_object is supported, shaderObject **must** be supported
- If VK_EXT_legacy_vertex_attributes is supported, legacyVertexAttributes **must** be supported
- If VK_EXT_dynamic_rendering_unused_attachments is supported, dynamicRenderingUnusedAttachments **must** be supported
- If VK_EXT_attachment_feedback_loop_dynamic_state is supported, attachmentFeedbackLoopDynamicState **must** be supported
- If VK_EXT_shader_replicated_composites is supported, shaderReplicatedComposites **must** be supported
- If VK_EXT_device_generated_commands is supported, deviceGeneratedCommands **must** be supported
- If VK_EXT_depth_clamp_control is supported, depthClampControl **must** be supported
- If VK_KHR_acceleration_structure is supported, the following features **must** be supported:
  - accelerationStructure
  - bufferDeviceAddress
  - descriptorBindingAccelerationStructureUpdateAfterBind
  - descriptorIndexing if Vulkan 1.2 is supported

- If VK_KHR_ray_tracing_pipeline is supported, the following features **must** be supported:
  - rayTracingPipeline
  - rayTracingPipelineTraceRaysIndirect
  - rayTraversalPrimitiveCulling if VK_KHR_ray_query is supported
- If VK_KHR_ray_query is supported, rayQuery **must** be supported
- VK_EXT_transform_feedback, if the extendedDynamicState3RasterizationStream feature is supported.
- VK_EXT_conservative_rasterization extension, if the extendedDynamicState3ConservativeRasterizationMode feature is supported.
- VK_EXT_conservative_rasterization extension, if the extendedDynamicState3ExtraPrimitiveOverestimationSize feature is supported.
- VK_EXT_sample_locations extension, if the extendedDynamicState3SampleLocationsEnable feature is supported.
- VK_EXT_blend_operation_advanced extension, if the extendedDynamicState3ColorBlendAdvanced feature is supported.
- VK_EXT_provoking_vertex, if the extendedDynamicState3ProvokingVertexMode feature is supported.
- VK_KHR_line_rasterization or VK_EXT_line_rasterization extension, if the extendedDynamicState3LineRasterizationMode feature is supported.
- VK_KHR_line_rasterization or VK_EXT_line_rasterization extension, if the extendedDynamicState3LineStippleEnable feature is supported.
- VK_EXT_depth_clip_control, if the extendedDynamicState3DepthClipNegativeOneToOne feature is supported.

All other features defined in the Specification are **optional**.

Chapter 41. Limits

Limits are implementation-dependent minimums, maximums, and other device characteristics that an application **may** need to be aware of.

NOTE

Limits are reported via the basic VkPhysicalDeviceLimits structure as well as the extensible structure VkPhysicalDeviceProperties2, which was added in VK_KHR_get_physical_device_properties2 and included in Vulkan 1.1. When limits are added in future Vulkan versions or extensions, each extension **should** introduce one new limit structure, if needed. This structure **can** be added to the pNext chain of the VkPhysicalDeviceProperties2 structure.

The VkPhysicalDeviceLimits structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkPhysicalDeviceLimits {
    uint32_t              maxImageDimension1D;
    uint32_t              maxImageDimension2D;
    uint32_t              maxImageDimension3D;
    uint32_t              maxImageDimensionCube;
    uint32_t              maxImageArrayLayers;
    uint32_t              maxTexelBufferElements;
    uint32_t              maxUniformBufferRange;
    uint32_t              maxStorageBufferRange;
    uint32_t              maxPushConstantsSize;
    uint32_t              maxMemoryAllocationCount;
    uint32_t              maxSamplerAllocationCount;
    VkDeviceSize          bufferImageGranularity;
    VkDeviceSize          sparseAddressSpaceSize;
    uint32_t              maxBoundDescriptorSets;
    uint32_t              maxPerStageDescriptorSamplers;
    uint32_t              maxPerStageDescriptorUniformBuffers;
    uint32_t              maxPerStageDescriptorStorageBuffers;
    uint32_t              maxPerStageDescriptorSampledImages;
    uint32_t              maxPerStageDescriptorStorageImages;
    uint32_t              maxPerStageDescriptorInputAttachments;
    uint32_t              maxPerStageResources;
    uint32_t              maxDescriptorSetSamplers;
    uint32_t              maxDescriptorSetUniformBuffers;
    uint32_t              maxDescriptorSetUniformBuffersDynamic;
    uint32_t              maxDescriptorSetStorageBuffers;
    uint32_t              maxDescriptorSetStorageBuffersDynamic;
    uint32_t              maxDescriptorSetSampledImages;
    uint32_t              maxDescriptorSetStorageImages;
    uint32_t              maxDescriptorSetInputAttachments;
    uint32_t              maxVertexInputAttributes;
    uint32_t              maxVertexInputBindings;
    uint32_t              maxVertexInputAttributeOffset;
    uint32_t              maxVertexInputBindingStride;
```

```
    uint32_t              maxVertexOutputComponents;
    uint32_t              maxTessellationGenerationLevel;
    uint32_t              maxTessellationPatchSize;
    uint32_t              maxTessellationControlPerVertexInputComponents;
    uint32_t              maxTessellationControlPerVertexOutputComponents;
    uint32_t              maxTessellationControlPerPatchOutputComponents;
    uint32_t              maxTessellationControlTotalOutputComponents;
    uint32_t              maxTessellationEvaluationInputComponents;
    uint32_t              maxTessellationEvaluationOutputComponents;
    uint32_t              maxGeometryShaderInvocations;
    uint32_t              maxGeometryInputComponents;
    uint32_t              maxGeometryOutputComponents;
    uint32_t              maxGeometryOutputVertices;
    uint32_t              maxGeometryTotalOutputComponents;
    uint32_t              maxFragmentInputComponents;
    uint32_t              maxFragmentOutputAttachments;
    uint32_t              maxFragmentDualSrcAttachments;
    uint32_t              maxFragmentCombinedOutputResources;
    uint32_t              maxComputeSharedMemorySize;
    uint32_t              maxComputeWorkGroupCount[3];
    uint32_t              maxComputeWorkGroupInvocations;
    uint32_t              maxComputeWorkGroupSize[3];
    uint32_t              subPixelPrecisionBits;
    uint32_t              subTexelPrecisionBits;
    uint32_t              mipmapPrecisionBits;
    uint32_t              maxDrawIndexedIndexValue;
    uint32_t              maxDrawIndirectCount;
    float                 maxSamplerLodBias;
    float                 maxSamplerAnisotropy;
    uint32_t              maxViewports;
    uint32_t              maxViewportDimensions[2];
    float                 viewportBoundsRange[2];
    uint32_t              viewportSubPixelBits;
    size_t                minMemoryMapAlignment;
    VkDeviceSize          minTexelBufferOffsetAlignment;
    VkDeviceSize          minUniformBufferOffsetAlignment;
    VkDeviceSize          minStorageBufferOffsetAlignment;
    int32_t               minTexelOffset;
    uint32_t              maxTexelOffset;
    int32_t               minTexelGatherOffset;
    uint32_t              maxTexelGatherOffset;
    float                 minInterpolationOffset;
    float                 maxInterpolationOffset;
    uint32_t              subPixelInterpolationOffsetBits;
    uint32_t              maxFramebufferWidth;
    uint32_t              maxFramebufferHeight;
    uint32_t              maxFramebufferLayers;
    VkSampleCountFlags    framebufferColorSampleCounts;
    VkSampleCountFlags    framebufferDepthSampleCounts;
    VkSampleCountFlags    framebufferStencilSampleCounts;
    VkSampleCountFlags    framebufferNoAttachmentsSampleCounts;
```

```
    uint32_t              maxColorAttachments;
    VkSampleCountFlags    sampledImageColorSampleCounts;
    VkSampleCountFlags    sampledImageIntegerSampleCounts;
    VkSampleCountFlags    sampledImageDepthSampleCounts;
    VkSampleCountFlags    sampledImageStencilSampleCounts;
    VkSampleCountFlags    storageImageSampleCounts;
    uint32_t              maxSampleMaskWords;
    VkBool32              timestampComputeAndGraphics;
    float                 timestampPeriod;
    uint32_t              maxClipDistances;
    uint32_t              maxCullDistances;
    uint32_t              maxCombinedClipAndCullDistances;
    uint32_t              discreteQueuePriorities;
    float                 pointSizeRange[2];
    float                 lineWidthRange[2];
    float                 pointSizeGranularity;
    float                 lineWidthGranularity;
    VkBool32              strictLines;
    VkBool32              standardSampleLocations;
    VkDeviceSize          optimalBufferCopyOffsetAlignment;
    VkDeviceSize          optimalBufferCopyRowPitchAlignment;
    VkDeviceSize          nonCoherentAtomSize;
} VkPhysicalDeviceLimits;
```

The `VkPhysicalDeviceLimits` are properties of the physical device. These are available in the `limits` member of the VkPhysicalDeviceProperties structure which is returned from vkGetPhysicalDeviceProperties.

- `maxImageDimension1D` is the largest dimension (`width`) that is guaranteed to be supported for all images created with an `imageType` of `VK_IMAGE_TYPE_1D`. Some combinations of image parameters (format, usage, etc.) **may** allow support for larger dimensions, which **can** be queried using vkGetPhysicalDeviceImageFormatProperties.
- `maxImageDimension2D` is the largest dimension (`width` or `height`) that is guaranteed to be supported for all images created with an `imageType` of `VK_IMAGE_TYPE_2D` and without `VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT` set in `flags`. Some combinations of image parameters (format, usage, etc.) **may** allow support for larger dimensions, which **can** be queried using vkGetPhysicalDeviceImageFormatProperties.
- `maxImageDimension3D` is the largest dimension (`width`, `height`, or `depth`) that is guaranteed to be supported for all images created with an `imageType` of `VK_IMAGE_TYPE_3D`. Some combinations of image parameters (format, usage, etc.) **may** allow support for larger dimensions, which **can** be queried using vkGetPhysicalDeviceImageFormatProperties.
- `maxImageDimensionCube` is the largest dimension (`width` or `height`) that is guaranteed to be supported for all images created with an `imageType` of `VK_IMAGE_TYPE_2D` and with `VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT` set in `flags`. Some combinations of image parameters (format, usage, etc.) **may** allow support for larger dimensions, which **can** be queried using vkGetPhysicalDeviceImageFormatProperties.
- `maxImageArrayLayers` is the maximum number of layers (`arrayLayers`) for an image.

- `maxTexelBufferElements` is the maximum number of addressable texels for a buffer view created on a buffer which was created with the `VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT` or `VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT` set in the `usage` member of the VkBufferCreateInfo structure.
- `maxUniformBufferRange` is the maximum value that **can** be specified in the `range` member of a VkDescriptorBufferInfo structure passed to vkUpdateDescriptorSets for descriptors of type `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER` or `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC`.
- `maxStorageBufferRange` is the maximum value that **can** be specified in the `range` member of a VkDescriptorBufferInfo structure passed to vkUpdateDescriptorSets for descriptors of type `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER` or `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC`.
- `maxPushConstantsSize` is the maximum size, in bytes, of the pool of push constant memory. For each of the push constant ranges indicated by the `pPushConstantRanges` member of the VkPipelineLayoutCreateInfo structure, (`offset` + `size`) **must** be less than or equal to this limit.
- `maxMemoryAllocationCount` is the maximum number of device memory allocations, as created by vkAllocateMemory, which **can** simultaneously exist.
- `maxSamplerAllocationCount` is the maximum number of sampler objects, as created by vkCreateSampler, which **can** simultaneously exist on a device.
- `bufferImageGranularity` is the granularity, in bytes, at which buffer or linear image resources, and optimal image resources **can** be bound to adjacent offsets in the same `VkDeviceMemory` object without aliasing. See Buffer-Image Granularity for more details.
- `sparseAddressSpaceSize` is the total amount of address space available, in bytes, for sparse memory resources. This is an upper bound on the sum of the sizes of all sparse resources, regardless of whether any memory is bound to them.
- `maxBoundDescriptorSets` is the maximum number of descriptor sets that **can** be simultaneously used by a pipeline. All `DescriptorSet` decorations in shader modules **must** have a value less than `maxBoundDescriptorSets`. See Descriptor Sets.
- `maxPerStageDescriptorSamplers` is the maximum number of samplers that **can** be accessible to a single shader stage in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_SAMPLER` or `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` count against this limit. A descriptor is accessible to a shader stage when the `stageFlags` member of the `VkDescriptorSetLayoutBinding` structure has the bit for that shader stage set. See Sampler and Combined Image Sampler.
- `maxPerStageDescriptorUniformBuffers` is the maximum number of uniform buffers that **can** be accessible to a single shader stage in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER` or `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` count against this limit. A descriptor is accessible to a shader stage when the `stageFlags` member of the `VkDescriptorSetLayoutBinding` structure has the bit for that shader stage set. See Uniform Buffer and Dynamic Uniform Buffer.
- `maxPerStageDescriptorStorageBuffers` is the maximum number of storage buffers that **can** be accessible to a single shader stage in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER` or `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` count against this limit. A descriptor is accessible to a pipeline shader stage when the `stageFlags` member of the `VkDescriptorSetLayoutBinding` structure has the bit for that shader stage set. See Storage Buffer and Dynamic Storage Buffer.

- `maxPerStageDescriptorSampledImages` is the maximum number of sampled images that **can** be accessible to a single shader stage in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, or `VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER` count against this limit. A descriptor is accessible to a pipeline shader stage when the `stageFlags` member of the `VkDescriptorSetLayoutBinding` structure has the bit for that shader stage set. See Combined Image Sampler, Sampled Image, and Uniform Texel Buffer.
- `maxPerStageDescriptorStorageImages` is the maximum number of storage images that **can** be accessible to a single shader stage in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER` count against this limit. A descriptor is accessible to a pipeline shader stage when the `stageFlags` member of the `VkDescriptorSetLayoutBinding` structure has the bit for that shader stage set. See Storage Image, and Storage Texel Buffer.
- `maxPerStageDescriptorInputAttachments` is the maximum number of input attachments that **can** be accessible to a single shader stage in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` count against this limit. A descriptor is accessible to a pipeline shader stage when the `stageFlags` member of the `VkDescriptorSetLayoutBinding` structure has the bit for that shader stage set. These are only supported for the fragment stage. See Input Attachment.
- `maxPerStageResources` is the maximum number of resources that **can** be accessible to a single shader stage in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, `VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER`, `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER`, `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER`, `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER`, `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC`, `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC`, or `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` count against this limit. For the fragment shader stage the framebuffer color attachments also count against this limit.
- `maxDescriptorSetSamplers` is the maximum number of samplers that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_SAMPLER` or `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` count against this limit. See Sampler and Combined Image Sampler.
- `maxDescriptorSetUniformBuffers` is the maximum number of uniform buffers that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER` or `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` count against this limit. See Uniform Buffer and Dynamic Uniform Buffer.
- `maxDescriptorSetUniformBuffersDynamic` is the maximum number of dynamic uniform buffers that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` count against this limit. See Dynamic Uniform Buffer.
- `maxDescriptorSetStorageBuffers` is the maximum number of storage buffers that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER` or `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` count against this limit. See Storage Buffer and Dynamic Storage Buffer.
- `maxDescriptorSetStorageBuffersDynamic` is the maximum number of dynamic storage buffers that

can be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` count against this limit. See Dynamic Storage Buffer.

- `maxDescriptorSetSampledImages` is the maximum number of sampled images that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE`, or `VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER` count against this limit. See Combined Image Sampler, Sampled Image, and Uniform Texel Buffer.
- `maxDescriptorSetStorageImages` is the maximum number of storage images that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, or `VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER` count against this limit. See Storage Image, and Storage Texel Buffer.
- `maxDescriptorSetInputAttachments` is the maximum number of input attachments that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT` count against this limit. See Input Attachment.
- `maxVertexInputAttributes` is the maximum number of vertex input attributes that **can** be specified for a graphics pipeline. These are described in the array of `VkVertexInputAttributeDescription` structures that are provided at graphics pipeline creation time via the `pVertexAttributeDescriptions` member of the VkPipelineVertexInputStateCreateInfo structure. See Vertex Attributes and Vertex Input Description.
- `maxVertexInputBindings` is the maximum number of vertex buffers that **can** be specified for providing vertex attributes to a graphics pipeline. These are described in the array of `VkVertexInputBindingDescription` structures that are provided at graphics pipeline creation time via the `pVertexBindingDescriptions` member of the VkPipelineVertexInputStateCreateInfo structure. The `binding` member of `VkVertexInputBindingDescription` **must** be less than this limit. See Vertex Input Description.
- `maxVertexInputAttributeOffset` is the maximum vertex input attribute offset that **can** be added to the vertex input binding stride. The `offset` member of the `VkVertexInputAttributeDescription` structure **must** be less than or equal to this limit. See Vertex Input Description.
- `maxVertexInputBindingStride` is the maximum vertex input binding stride that **can** be specified in a vertex input binding. The `stride` member of the `VkVertexInputBindingDescription` structure **must** be less than or equal to this limit. See Vertex Input Description.
- `maxVertexOutputComponents` is the maximum number of components of output variables which **can** be output by a vertex shader. See Vertex Shaders.
- `maxTessellationGenerationLevel` is the maximum tessellation generation level supported by the fixed-function tessellation primitive generator. See Tessellation.
- `maxTessellationPatchSize` is the maximum patch size, in vertices, of patches that **can** be processed by the tessellation control shader and tessellation primitive generator. The `patchControlPoints` member of the VkPipelineTessellationStateCreateInfo structure specified at pipeline creation time and the value provided in the `OutputVertices` execution mode of shader modules **must** be less than or equal to this limit. See Tessellation.
- `maxTessellationControlPerVertexInputComponents` is the maximum number of components of input variables which **can** be provided as per-vertex inputs to the tessellation control shader stage.
- `maxTessellationControlPerVertexOutputComponents` is the maximum number of components of

per-vertex output variables which **can** be output from the tessellation control shader stage.

- `maxTessellationControlPerPatchOutputComponents` is the maximum number of components of per-patch output variables which **can** be output from the tessellation control shader stage.
- `maxTessellationControlTotalOutputComponents` is the maximum total number of components of per-vertex and per-patch output variables which **can** be output from the tessellation control shader stage.
- `maxTessellationEvaluationInputComponents` is the maximum number of components of input variables which **can** be provided as per-vertex inputs to the tessellation evaluation shader stage.
- `maxTessellationEvaluationOutputComponents` is the maximum number of components of per-vertex output variables which **can** be output from the tessellation evaluation shader stage.
- `maxGeometryShaderInvocations` is the maximum invocation count supported for instanced geometry shaders. The value provided in the `Invocations` execution mode of shader modules **must** be less than or equal to this limit. See Geometry Shading.
- `maxGeometryInputComponents` is the maximum number of components of input variables which **can** be provided as inputs to the geometry shader stage.
- `maxGeometryOutputComponents` is the maximum number of components of output variables which **can** be output from the geometry shader stage.
- `maxGeometryOutputVertices` is the maximum number of vertices which **can** be emitted by any geometry shader.
- `maxGeometryTotalOutputComponents` is the maximum total number of components of output variables, across all emitted vertices, which **can** be output from the geometry shader stage.
- `maxFragmentInputComponents` is the maximum number of components of input variables which **can** be provided as inputs to the fragment shader stage.
- `maxFragmentOutputAttachments` is the maximum number of output attachments which **can** be written to by the fragment shader stage.
- `maxFragmentDualSrcAttachments` is the maximum number of output attachments which **can** be written to by the fragment shader stage when blending is enabled and one of the dual source blend modes is in use. See Dual-Source Blending and `dualSrcBlend`.
- `maxFragmentCombinedOutputResources` is the total number of storage buffers, storage images, and output `Location` decorated color attachments (described in Fragment Output Interface) which **can** be used in the fragment shader stage.
- `maxComputeSharedMemorySize` is the maximum total storage size, in bytes, available for variables declared with the `Workgroup` storage class in shader modules (or with the `shared` storage qualifier in GLSL) in the compute shader stage.
- `maxComputeWorkGroupCount`[3] is the maximum number of local workgroups that **can** be dispatched by a single dispatching command. These three values represent the maximum number of local workgroups for the X, Y, and Z dimensions, respectively. The workgroup count parameters to the dispatching commands **must** be less than or equal to the corresponding limit. See Dispatching Commands.
- `maxComputeWorkGroupInvocations` is the maximum total number of compute shader invocations in

a single local workgroup. The product of the X, Y, and Z sizes, as specified by the `LocalSize` or `LocalSizeId` execution mode in shader modules or by the object decorated by the `WorkgroupSize` decoration, **must** be less than or equal to this limit.

- `maxComputeWorkGroupSize`[3] is the maximum size of a local compute workgroup, per dimension. These three values represent the maximum local workgroup size in the X, Y, and Z dimensions, respectively. The `x`, `y`, and `z` sizes, as specified by the `LocalSize` or `LocalSizeId` execution mode or by the object decorated by the `WorkgroupSize` decoration in shader modules, **must** be less than or equal to the corresponding limit.
- `subPixelPrecisionBits` is the number of bits of subpixel precision in framebuffer coordinates x_f and y_f. See Rasterization.
- `subTexelPrecisionBits` is the number of bits of precision in the division along an axis of an image used for minification and magnification filters. $2^{\texttt{subTexelPrecisionBits}}$ is the actual number of divisions along each axis of the image represented. Sub-texel values calculated during image sampling will snap to these locations when generating the filtered results.
- `mipmapPrecisionBits` is the number of bits of division that the LOD calculation for mipmap fetching get snapped to when determining the contribution from each mip level to the mip filtered results. $2^{\texttt{mipmapPrecisionBits}}$ is the actual number of divisions.
- `maxDrawIndexedIndexValue` is the maximum index value that **can** be used for indexed draw calls when using 32-bit indices. This excludes the primitive restart index value of 0xFFFFFFFF. See `fullDrawIndexUint32`.
- `maxDrawIndirectCount` is the maximum draw count that is supported for indirect drawing calls. See `multiDrawIndirect`.
- `maxSamplerLodBias` is the maximum absolute sampler LOD bias. The sum of the `mipLodBias` member of the VkSamplerCreateInfo structure and the `Bias` operand of image sampling operations in shader modules (or 0 if no `Bias` operand is provided to an image sampling operation) are clamped to the range [-`maxSamplerLodBias`,+`maxSamplerLodBias`]. See [samplers-mipLodBias].
- `maxSamplerAnisotropy` is the maximum degree of sampler anisotropy. The maximum degree of anisotropic filtering used for an image sampling operation is the minimum of the `maxAnisotropy` member of the VkSamplerCreateInfo structure and this limit. See [samplers-maxAnisotropy].
- `maxViewports` is the maximum number of active viewports. The `viewportCount` member of the VkPipelineViewportStateCreateInfo structure that is provided at pipeline creation **must** be less than or equal to this limit.
- `maxViewportDimensions`[2] are the maximum viewport dimensions in the X (width) and Y (height) dimensions, respectively. The maximum viewport dimensions **must** be greater than or equal to the largest image which **can** be created and used as a framebuffer attachment. See Controlling the Viewport.
- `viewportBoundsRange`[2] is the [minimum, maximum] range that the corners of a viewport **must** be contained in. This range **must** be at least [-2 × `size`, 2 × `size` - 1], where `size` = max(`maxViewportDimensions`[0], `maxViewportDimensions`[1]). See Controlling the Viewport.

NOTE	The intent of the `viewportBoundsRange` limit is to allow a maximum sized viewport to be arbitrarily shifted relative to the output target as long as at least

> some portion intersects. This would give a bounds limit of [-`size` + 1, 2 × `size` - 1] which would allow all possible non-empty-set intersections of the output target and the viewport. Since these numbers are typically powers of two, picking the signed number range using the smallest possible number of bits ends up with the specified range.

- `viewportSubPixelBits` is the number of bits of subpixel precision for viewport bounds. The subpixel precision that floating-point viewport bounds are interpreted at is given by this limit.
- `minMemoryMapAlignment` is the minimum **required** alignment, in bytes, of host visible memory allocations within the host address space. When mapping a memory allocation with vkMapMemory, subtracting `offset` bytes from the returned pointer will always produce an integer multiple of this limit. See Host Access to Device Memory Objects. The value **must** be a power of two.
- `minTexelBufferOffsetAlignment` is the minimum **required** alignment, in bytes, for the `offset` member of the VkBufferViewCreateInfo structure for texel buffers. The value **must** be a power of two. VkBufferViewCreateInfo::`offset` **must** be a multiple of this value.
- `minUniformBufferOffsetAlignment` is the minimum **required** alignment, in bytes, for the `offset` member of the `VkDescriptorBufferInfo` structure for uniform buffers. When a descriptor of type `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER` or `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` is updated, the `offset` **must** be an integer multiple of this limit. Similarly, dynamic offsets for uniform buffers **must** be multiples of this limit. The value **must** be a power of two.
- `minStorageBufferOffsetAlignment` is the minimum **required** alignment, in bytes, for the `offset` member of the `VkDescriptorBufferInfo` structure for storage buffers. When a descriptor of type `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER` or `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` is updated, the `offset` **must** be an integer multiple of this limit. Similarly, dynamic offsets for storage buffers **must** be multiples of this limit. The value **must** be a power of two.
- `minTexelOffset` is the minimum offset value for the `ConstOffset` image operand of any of the `OpImageSample*` or `OpImageFetch*` image instructions.
- `maxTexelOffset` is the maximum offset value for the `ConstOffset` image operand of any of the `OpImageSample*` or `OpImageFetch*` image instructions.
- `minTexelGatherOffset` is the minimum offset value for the `Offset`, `ConstOffset`, or `ConstOffsets` image operands of any of the `OpImage*Gather` image instructions.
- `maxTexelGatherOffset` is the maximum offset value for the `Offset`, `ConstOffset`, or `ConstOffsets` image operands of any of the `OpImage*Gather` image instructions.
- `minInterpolationOffset` is the base minimum (inclusive) negative offset value for the `Offset` operand of the `InterpolateAtOffset` extended instruction.
- `maxInterpolationOffset` is the base maximum (inclusive) positive offset value for the `Offset` operand of the `InterpolateAtOffset` extended instruction.
- `subPixelInterpolationOffsetBits` is the number of fractional bits that the `x` and `y` offsets to the `InterpolateAtOffset` extended instruction **may** be rounded to as fixed-point values.
- `maxFramebufferWidth` is the maximum width for a framebuffer. The `width` member of the VkFramebufferCreateInfo structure **must** be less than or equal to this limit.
- `maxFramebufferHeight` is the maximum height for a framebuffer. The `height` member of the

VkFramebufferCreateInfo structure **must** be less than or equal to this limit.

- `maxFramebufferLayers` is the maximum layer count for a layered framebuffer. The `layers` member of the VkFramebufferCreateInfo structure **must** be less than or equal to this limit.
- `framebufferColorSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the color sample counts that are supported for all framebuffer color attachments with floating- or fixed-point formats. There is no limit specifying the color sample counts that are supported for all color attachments with integer formats.
- `framebufferDepthSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the supported depth sample counts for all framebuffer depth/stencil attachments, when the format includes a depth component.
- `framebufferStencilSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the supported stencil sample counts for all framebuffer depth/stencil attachments, when the format includes a stencil component.
- `framebufferNoAttachmentsSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the supported sample counts for a subpass which uses no attachments.
- `maxColorAttachments` is the maximum number of color attachments that **can** be used by a subpass in a render pass. The `colorAttachmentCount` member of the `VkSubpassDescription` or `VkSubpassDescription2` structure **must** be less than or equal to this limit.
- `sampledImageColorSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the sample counts supported for all 2D images created with `VK_IMAGE_TILING_OPTIMAL`, `usage` containing `VK_IMAGE_USAGE_SAMPLED_BIT`, and a non-integer color format.
- `sampledImageIntegerSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the sample counts supported for all 2D images created with `VK_IMAGE_TILING_OPTIMAL`, `usage` containing `VK_IMAGE_USAGE_SAMPLED_BIT`, and an integer color format.
- `sampledImageDepthSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the sample counts supported for all 2D images created with `VK_IMAGE_TILING_OPTIMAL`, `usage` containing `VK_IMAGE_USAGE_SAMPLED_BIT`, and a depth format.
- `sampledImageStencilSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the sample counts supported for all 2D images created with `VK_IMAGE_TILING_OPTIMAL`, `usage` containing `VK_IMAGE_USAGE_SAMPLED_BIT`, and a stencil format.
- `storageImageSampleCounts` is a bitmask[1] of VkSampleCountFlagBits indicating the sample counts supported for all 2D images created with `VK_IMAGE_TILING_OPTIMAL`, and `usage` containing `VK_IMAGE_USAGE_STORAGE_BIT`.
- `maxSampleMaskWords` is the maximum number of array elements of a variable decorated with the `SampleMask` built-in decoration.
- `timestampComputeAndGraphics` specifies support for timestamps on all graphics and compute queues. If this limit is `VK_TRUE`, all queues that advertise the `VK_QUEUE_GRAPHICS_BIT` or `VK_QUEUE_COMPUTE_BIT` in the `VkQueueFamilyProperties`::`queueFlags` support `VkQueueFamilyProperties`::`timestampValidBits` of at least 36. See Timestamp Queries.
- `timestampPeriod` is the number of nanoseconds **required** for a timestamp query to be incremented by 1. See Timestamp Queries.
- `maxClipDistances` is the maximum number of clip distances that **can** be used in a single shader

stage. The size of any array declared with the ClipDistance built-in decoration in a shader module **must** be less than or equal to this limit.

- maxCullDistances is the maximum number of cull distances that **can** be used in a single shader stage. The size of any array declared with the CullDistance built-in decoration in a shader module **must** be less than or equal to this limit.
- maxCombinedClipAndCullDistances is the maximum combined number of clip and cull distances that **can** be used in a single shader stage. The sum of the sizes of all arrays declared with the ClipDistance and CullDistance built-in decoration used by a single shader stage in a shader module **must** be less than or equal to this limit.
- discreteQueuePriorities is the number of discrete priorities that **can** be assigned to a queue based on the value of each member of VkDeviceQueueCreateInfo::pQueuePriorities. This **must** be at least 2, and levels **must** be spread evenly over the range, with at least one level at 1.0, and another at 0.0. See Queue Priority.
- pointSizeRange[2] is the range [minimum,maximum] of supported sizes for points. Values written to variables decorated with the PointSize built-in decoration are clamped to this range.
- lineWidthRange[2] is the range [minimum,maximum] of supported widths for lines. Values specified by the lineWidth member of the VkPipelineRasterizationStateCreateInfo or the lineWidth parameter to vkCmdSetLineWidth are clamped to this range.
- pointSizeGranularity is the granularity of supported point sizes. Not all point sizes in the range defined by pointSizeRange are supported. This limit specifies the granularity (or increment) between successive supported point sizes.
- lineWidthGranularity is the granularity of supported line widths. Not all line widths in the range defined by lineWidthRange are supported. This limit specifies the granularity (or increment) between successive supported line widths.
- strictLines specifies whether lines are rasterized according to the preferred method of rasterization. If set to VK_FALSE, lines **may** be rasterized under a relaxed set of rules. If set to VK_TRUE, lines are rasterized as per the strict definition. See Basic Line Segment Rasterization.
- standardSampleLocations specifies whether rasterization uses the standard sample locations as documented in Multisampling. If set to VK_TRUE, the implementation uses the documented sample locations. If set to VK_FALSE, the implementation **may** use different sample locations.
- optimalBufferCopyOffsetAlignment is the optimal buffer offset alignment in bytes for vkCmdCopyBufferToImage2KHR, vkCmdCopyBufferToImage, vkCmdCopyImageToBuffer2KHR, and vkCmdCopyImageToBuffer. This value is also the optimal host memory offset alignment in bytes for vkCopyMemoryToImageEXT and vkCopyImageToMemoryEXT. The per texel alignment requirements are enforced, but applications **should** use the optimal alignment for optimal performance and power use. The value **must** be a power of two.
- optimalBufferCopyRowPitchAlignment is the optimal buffer row pitch alignment in bytes for vkCmdCopyBufferToImage2KHR, vkCmdCopyBufferToImage, vkCmdCopyImageToBuffer2KHR, and vkCmdCopyImageToBuffer. This value is also the optimal host memory row pitch alignment in bytes for vkCopyMemoryToImageEXT and vkCopyImageToMemoryEXT. Row pitch is the number of bytes between texels with the same X coordinate in adjacent rows (Y coordinates differ by one). The per texel alignment requirements are enforced, but applications **should** use the optimal alignment for optimal performance and power use. The value **must** be a power of

two.

- `nonCoherentAtomSize` is the size and alignment in bytes that bounds concurrent access to host-mapped device memory. The value **must** be a power of two.

1

For all bitmasks of VkSampleCountFlagBits, the sample count limits defined above represent the minimum supported sample counts for each image type. Individual images **may** support additional sample counts, which are queried using vkGetPhysicalDeviceImageFormatProperties as described in Supported Sample Counts.

Bits which **may** be set in the sample count limits returned by VkPhysicalDeviceLimits, as well as in other queries and structures representing image sample counts, are:

```
// Provided by VK_VERSION_1_0
typedef enum VkSampleCountFlagBits {
    VK_SAMPLE_COUNT_1_BIT = 0x00000001,
    VK_SAMPLE_COUNT_2_BIT = 0x00000002,
    VK_SAMPLE_COUNT_4_BIT = 0x00000004,
    VK_SAMPLE_COUNT_8_BIT = 0x00000008,
    VK_SAMPLE_COUNT_16_BIT = 0x00000010,
    VK_SAMPLE_COUNT_32_BIT = 0x00000020,
    VK_SAMPLE_COUNT_64_BIT = 0x00000040,
} VkSampleCountFlagBits;
```

- `VK_SAMPLE_COUNT_1_BIT` specifies an image with one sample per pixel.
- `VK_SAMPLE_COUNT_2_BIT` specifies an image with 2 samples per pixel.
- `VK_SAMPLE_COUNT_4_BIT` specifies an image with 4 samples per pixel.
- `VK_SAMPLE_COUNT_8_BIT` specifies an image with 8 samples per pixel.
- `VK_SAMPLE_COUNT_16_BIT` specifies an image with 16 samples per pixel.
- `VK_SAMPLE_COUNT_32_BIT` specifies an image with 32 samples per pixel.
- `VK_SAMPLE_COUNT_64_BIT` specifies an image with 64 samples per pixel.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkSampleCountFlags;
```

`VkSampleCountFlags` is a bitmask type for setting a mask of zero or more VkSampleCountFlagBits.

The `VkPhysicalDevicePushDescriptorPropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_push_descriptor
typedef struct VkPhysicalDevicePushDescriptorPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxPushDescriptors;
} VkPhysicalDevicePushDescriptorPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxPushDescriptors` is the maximum number of descriptors that **can** be used in a descriptor set layout created with `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR` set.

If the `VkPhysicalDevicePushDescriptorPropertiesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePushDescriptorPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PUSH_DESCRIPTOR_PROPERTIES_KHR`

The `VkPhysicalDeviceMultiviewProperties` structure is defined as:

```
typedef struct VkPhysicalDeviceMultiviewProperties {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxMultiviewViewCount;
    uint32_t           maxMultiviewInstanceIndex;
} VkPhysicalDeviceMultiviewProperties;
```

or the equivalent

```
// Provided by VK_KHR_multiview
typedef VkPhysicalDeviceMultiviewProperties VkPhysicalDeviceMultiviewPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxMultiviewViewCount` is one greater than the maximum view index that **can** be used in a subpass.
- `maxMultiviewInstanceIndex` is the maximum valid value of instance index allowed to be generated by a drawing command recorded within a subpass of a multiview render pass instance.

If the VkPhysicalDeviceMultiviewProperties structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMultiviewProperties-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_PROPERTIES

The VkPhysicalDeviceFloatControlsProperties structure is defined as:

```
typedef struct VkPhysicalDeviceFloatControlsProperties {
    VkStructureType                      sType;
    void*                                pNext;
    VkShaderFloatControlsIndependence    denormBehaviorIndependence;
    VkShaderFloatControlsIndependence    roundingModeIndependence;
    VkBool32                             shaderSignedZeroInfNanPreserveFloat16;
    VkBool32                             shaderSignedZeroInfNanPreserveFloat32;
    VkBool32                             shaderSignedZeroInfNanPreserveFloat64;
    VkBool32                             shaderDenormPreserveFloat16;
    VkBool32                             shaderDenormPreserveFloat32;
    VkBool32                             shaderDenormPreserveFloat64;
    VkBool32                             shaderDenormFlushToZeroFloat16;
    VkBool32                             shaderDenormFlushToZeroFloat32;
    VkBool32                             shaderDenormFlushToZeroFloat64;
    VkBool32                             shaderRoundingModeRTEFloat16;
    VkBool32                             shaderRoundingModeRTEFloat32;
    VkBool32                             shaderRoundingModeRTEFloat64;
    VkBool32                             shaderRoundingModeRTZFloat16;
    VkBool32                             shaderRoundingModeRTZFloat32;
    VkBool32                             shaderRoundingModeRTZFloat64;
} VkPhysicalDeviceFloatControlsProperties;
```

or the equivalent

```
// Provided by VK_KHR_shader_float_controls
typedef VkPhysicalDeviceFloatControlsProperties
VkPhysicalDeviceFloatControlsPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- denormBehaviorIndependence is a VkShaderFloatControlsIndependence value indicating whether, and how, denorm behavior can be set independently for different bit widths.
- roundingModeIndependence is a VkShaderFloatControlsIndependence value indicating whether, and how, rounding modes can be set independently for different bit widths.

- shaderSignedZeroInfNanPreserveFloat16 is a boolean value indicating whether sign of a zero, Nans and ±∞ **can** be preserved in 16-bit floating-point computations. It also indicates whether the SignedZeroInfNanPreserve execution mode **can** be used for 16-bit floating-point types.
- shaderSignedZeroInfNanPreserveFloat32 is a boolean value indicating whether sign of a zero, Nans and ±∞ **can** be preserved in 32-bit floating-point computations. It also indicates whether the SignedZeroInfNanPreserve execution mode **can** be used for 32-bit floating-point types.
- shaderSignedZeroInfNanPreserveFloat64 is a boolean value indicating whether sign of a zero, Nans and ±∞ **can** be preserved in 64-bit floating-point computations. It also indicates whether the SignedZeroInfNanPreserve execution mode **can** be used for 64-bit floating-point types.
- shaderDenormPreserveFloat16 is a boolean value indicating whether denormals **can** be preserved in 16-bit floating-point computations. It also indicates whether the DenormPreserve execution mode **can** be used for 16-bit floating-point types.
- shaderDenormPreserveFloat32 is a boolean value indicating whether denormals **can** be preserved in 32-bit floating-point computations. It also indicates whether the DenormPreserve execution mode **can** be used for 32-bit floating-point types.
- shaderDenormPreserveFloat64 is a boolean value indicating whether denormals **can** be preserved in 64-bit floating-point computations. It also indicates whether the DenormPreserve execution mode **can** be used for 64-bit floating-point types.
- shaderDenormFlushToZeroFloat16 is a boolean value indicating whether denormals **can** be flushed to zero in 16-bit floating-point computations. It also indicates whether the DenormFlushToZero execution mode **can** be used for 16-bit floating-point types.
- shaderDenormFlushToZeroFloat32 is a boolean value indicating whether denormals **can** be flushed to zero in 32-bit floating-point computations. It also indicates whether the DenormFlushToZero execution mode **can** be used for 32-bit floating-point types.
- shaderDenormFlushToZeroFloat64 is a boolean value indicating whether denormals **can** be flushed to zero in 64-bit floating-point computations. It also indicates whether the DenormFlushToZero execution mode **can** be used for 64-bit floating-point types.
- shaderRoundingModeRTEFloat16 is a boolean value indicating whether an implementation supports the round-to-nearest-even rounding mode for 16-bit floating-point arithmetic and conversion instructions. It also indicates whether the RoundingModeRTE execution mode **can** be used for 16-bit floating-point types.
- shaderRoundingModeRTEFloat32 is a boolean value indicating whether an implementation supports the round-to-nearest-even rounding mode for 32-bit floating-point arithmetic and conversion instructions. It also indicates whether the RoundingModeRTE execution mode **can** be used for 32-bit floating-point types.
- shaderRoundingModeRTEFloat64 is a boolean value indicating whether an implementation supports the round-to-nearest-even rounding mode for 64-bit floating-point arithmetic and conversion instructions. It also indicates whether the RoundingModeRTE execution mode **can** be used for 64-bit floating-point types.
- shaderRoundingModeRTZFloat16 is a boolean value indicating whether an implementation supports the round-towards-zero rounding mode for 16-bit floating-point arithmetic and conversion instructions. It also indicates whether the RoundingModeRTZ execution mode **can** be used for 16-bit floating-point types.

- shaderRoundingModeRTZFloat32 is a boolean value indicating whether an implementation supports the round-towards-zero rounding mode for 32-bit floating-point arithmetic and conversion instructions. It also indicates whether the RoundingModeRTZ execution mode **can** be used for 32-bit floating-point types.
- shaderRoundingModeRTZFloat64 is a boolean value indicating whether an implementation supports the round-towards-zero rounding mode for 64-bit floating-point arithmetic and conversion instructions. It also indicates whether the RoundingModeRTZ execution mode **can** be used for 64-bit floating-point types.

If the VkPhysicalDeviceFloatControlsProperties structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFloatControlsProperties-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FLOAT_CONTROLS_PROPERTIES

Values which **may** be returned in the denormBehaviorIndependence and roundingModeIndependence fields of VkPhysicalDeviceFloatControlsProperties are:

```
typedef enum VkShaderFloatControlsIndependence {
    VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_32_BIT_ONLY = 0,
    VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_ALL = 1,
    VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_NONE = 2,
  // Provided by VK_KHR_shader_float_controls
    VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_32_BIT_ONLY_KHR =
VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_32_BIT_ONLY,
  // Provided by VK_KHR_shader_float_controls
    VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_ALL_KHR =
VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_ALL,
  // Provided by VK_KHR_shader_float_controls
    VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_NONE_KHR =
VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_NONE,
} VkShaderFloatControlsIndependence;
```

or the equivalent

```
// Provided by VK_KHR_shader_float_controls
typedef VkShaderFloatControlsIndependence VkShaderFloatControlsIndependenceKHR;
```

- VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_32_BIT_ONLY specifies that shader float controls for 32-bit floating-point **can** be set independently; other bit widths **must** be set identically to each other.
- VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_ALL specifies that shader float controls for all bit widths **can** be set independently.

- VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_NONE specifies that shader float controls for all bit widths **must** be set identically.

The VkPhysicalDeviceDiscardRectanglePropertiesEXT structure is defined as:

```
// Provided by VK_EXT_discard_rectangles
typedef struct VkPhysicalDeviceDiscardRectanglePropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxDiscardRectangles;
} VkPhysicalDeviceDiscardRectanglePropertiesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxDiscardRectangles is the maximum number of active discard rectangles that **can** be specified.

If the VkPhysicalDeviceDiscardRectanglePropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDiscardRectanglePropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DISCARD_RECTANGLE_PROPERTIES_EXT

The VkPhysicalDeviceSampleLocationsPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_sample_locations
typedef struct VkPhysicalDeviceSampleLocationsPropertiesEXT {
    VkStructureType       sType;
    void*                 pNext;
    VkSampleCountFlags    sampleLocationSampleCounts;
    VkExtent2D            maxSampleLocationGridSize;
    float                 sampleLocationCoordinateRange[2];
    uint32_t              sampleLocationSubPixelBits;
    VkBool32              variableSampleLocations;
} VkPhysicalDeviceSampleLocationsPropertiesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- sampleLocationSampleCounts is a bitmask of VkSampleCountFlagBits indicating the sample counts supporting custom sample locations.
- maxSampleLocationGridSize is the maximum size of the pixel grid in which sample locations **can** vary that is supported for all sample counts in sampleLocationSampleCounts.

- sampleLocationCoordinateRange[2] is the range of supported sample location coordinates.
- sampleLocationSubPixelBits is the number of bits of subpixel precision for sample locations.
- variableSampleLocations specifies whether the sample locations used by all pipelines that will be bound to a command buffer during a subpass **must** match. If set to VK_TRUE, the implementation supports variable sample locations in a subpass. If set to VK_FALSE, then the sample locations **must** stay constant in each subpass.

If the VkPhysicalDeviceSampleLocationsPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceSampleLocationsPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SAMPLE_LOCATIONS_PROPERTIES_EXT

The VkPhysicalDeviceLegacyVertexAttributesPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_legacy_vertex_attributes
typedef struct VkPhysicalDeviceLegacyVertexAttributesPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           nativeUnalignedPerformance;
} VkPhysicalDeviceLegacyVertexAttributesPropertiesEXT;
```

This structure describes the following features:

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- nativeUnalignedPerformance specifies whether unaligned vertex fetches do not incur significant performance penalties as compared to aligned fetches.

If the VkPhysicalDeviceLegacyVertexAttributesPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceFeatures2 structure passed to vkGetPhysicalDeviceFeatures2KHR, it is filled in to indicate whether each corresponding feature is supported. VkPhysicalDeviceLegacyVertexAttributesPropertiesEXT **can** also be used in the pNext chain of VkDeviceCreateInfo to selectively enable these features.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceLegacyVertexAttributesPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LEGACY_VERTEX_ATTRIBUTES_PROPERTIES_EXT

The VkPhysicalDeviceExternalMemoryHostPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_external_memory_host
typedef struct VkPhysicalDeviceExternalMemoryHostPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkDeviceSize       minImportedHostPointerAlignment;
} VkPhysicalDeviceExternalMemoryHostPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `minImportedHostPointerAlignment` is the minimum **required** alignment, in bytes, for the base address and size of host pointers that **can** be imported to a Vulkan memory object. The value **must** be a power of two.

If the `VkPhysicalDeviceExternalMemoryHostPropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceExternalMemoryHostPropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_MEMORY_HOST_PROPERTIES_EXT`

The `VkPhysicalDevicePointClippingProperties` structure is defined as:

```
typedef struct VkPhysicalDevicePointClippingProperties {
    VkStructureType            sType;
    void*                      pNext;
    VkPointClippingBehavior    pointClippingBehavior;
} VkPhysicalDevicePointClippingProperties;
```

or the equivalent

```
// Provided by VK_KHR_maintenance2
typedef VkPhysicalDevicePointClippingProperties
VkPhysicalDevicePointClippingPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pointClippingBehavior` is a VkPointClippingBehavior value specifying the point clipping behavior supported by the implementation.

If the `VkPhysicalDevicePointClippingProperties` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in

with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePointClippingProperties-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_POINT_CLIPPING_PROPERTIES`

The `VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_blend_operation_advanced
typedef struct VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           advancedBlendMaxColorAttachments;
    VkBool32           advancedBlendIndependentBlend;
    VkBool32           advancedBlendNonPremultipliedSrcColor;
    VkBool32           advancedBlendNonPremultipliedDstColor;
    VkBool32           advancedBlendCorrelatedOverlap;
    VkBool32           advancedBlendAllOperations;
} VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `advancedBlendMaxColorAttachments` is one greater than the highest color attachment index that **can** be used in a subpass, for a pipeline that uses an advanced blend operation.
- `advancedBlendIndependentBlend` specifies whether advanced blend operations **can** vary per-attachment.
- `advancedBlendNonPremultipliedSrcColor` specifies whether the source color **can** be treated as non-premultiplied. If this is `VK_FALSE`, then VkPipelineColorBlendAdvancedStateCreateInfoEXT ::`srcPremultiplied` **must** be `VK_TRUE`.
- `advancedBlendNonPremultipliedDstColor` specifies whether the destination color **can** be treated as non-premultiplied. If this is `VK_FALSE`, then VkPipelineColorBlendAdvancedStateCreateInfoEXT ::`dstPremultiplied` **must** be `VK_TRUE`.
- `advancedBlendCorrelatedOverlap` specifies whether the overlap mode **can** be treated as correlated. If this is `VK_FALSE`, then VkPipelineColorBlendAdvancedStateCreateInfoEXT ::`blendOverlap` **must** be `VK_BLEND_OVERLAP_UNCORRELATED_EXT`.
- `advancedBlendAllOperations` specifies whether all advanced blend operation enums are supported. See the valid usage of VkPipelineColorBlendAttachmentState.

If the `VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT-sType-sType sType must be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BLEND_OPERATION_ADVANCED_PROPERTIES_EXT

The VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_vertex_attribute_divisor
typedef struct VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxVertexAttribDivisor;
    VkBool32           supportsNonZeroFirstInstance;
} VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxVertexAttribDivisor is the maximum value of the number of instances that will repeat the value of vertex attribute data when instanced rendering is enabled.
- supportsNonZeroFirstInstance specifies whether a non-zero value for the firstInstance parameter of drawing commands is supported when VkVertexInputBindingDivisorDescriptionKHR::divisor is not 1.

If the VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR-sType-sType sType must be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_ATTRIBUTE_DIVISOR_PROPERTIES_KHR

The VkPhysicalDeviceMaintenance3Properties structure is defined as:

```
typedef struct VkPhysicalDeviceMaintenance3Properties {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxPerSetDescriptors;
    VkDeviceSize       maxMemoryAllocationSize;
} VkPhysicalDeviceMaintenance3Properties;
```

or the equivalent

```
// Provided by VK_KHR_maintenance3
typedef VkPhysicalDeviceMaintenance3Properties
VkPhysicalDeviceMaintenance3PropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxPerSetDescriptors` is a maximum number of descriptors (summed over all descriptor types) in a single descriptor set that is guaranteed to satisfy any implementation-dependent constraints on the size of a descriptor set itself. Applications **can** query whether a descriptor set that goes beyond this limit is supported using vkGetDescriptorSetLayoutSupportKHR.
- `maxMemoryAllocationSize` is the maximum size of a memory allocation that **can** be created, even if there is more space available in the heap.

If the `VkPhysicalDeviceMaintenance3Properties` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance3Properties-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_3_PROPERTIES`

The `VkPhysicalDeviceMaintenance4Properties` structure is defined as:

```
typedef struct VkPhysicalDeviceMaintenance4Properties {
    VkStructureType    sType;
    void*              pNext;
    VkDeviceSize       maxBufferSize;
} VkPhysicalDeviceMaintenance4Properties;
```

or the equivalent

```
// Provided by VK_KHR_maintenance4
typedef VkPhysicalDeviceMaintenance4Properties
VkPhysicalDeviceMaintenance4PropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxBufferSize` is the maximum size `VkBuffer` that **can** be created.

If the `VkPhysicalDeviceMaintenance4Properties` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in

with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance4Properties-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_PROPERTIES`

The `VkPhysicalDeviceMaintenance5PropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance5
typedef struct VkPhysicalDeviceMaintenance5PropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           earlyFragmentMultisampleCoverageAfterSampleCounting;
    VkBool32           earlyFragmentSampleMaskTestBeforeSampleCounting;
    VkBool32           depthStencilSwizzleOneSupport;
    VkBool32           polygonModePointSize;
    VkBool32           nonStrictSinglePixelWideLinesUseParallelogram;
    VkBool32           nonStrictWideLinesUseParallelogram;
} VkPhysicalDeviceMaintenance5PropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `earlyFragmentMultisampleCoverageAfterSampleCounting` is a boolean value indicating whether the fragment shading and multisample coverage operations are performed after sample counting for fragment shaders with `EarlyFragmentTests` execution mode.
- `earlyFragmentSampleMaskTestBeforeSampleCounting` is a boolean value indicating whether the sample mask test operation is performed before sample counting for fragment shaders using the `EarlyFragmentTests` execution mode.
- `depthStencilSwizzleOneSupport` is a boolean indicating that depth/stencil texturing operations with `VK_COMPONENT_SWIZZLE_ONE` have defined behavior.
- `polygonModePointSize` is a boolean value indicating whether the point size of the final rasterization of polygons with `VK_POLYGON_MODE_POINT` is controlled by `PointSize`.
- `nonStrictSinglePixelWideLinesUseParallelogram` is a boolean value indicating whether non-strict lines with a width of 1.0 are rasterized as parallelograms or using Bresenham's algorithm.
- `nonStrictWideLinesUseParallelogram` is a boolean value indicating whether non-strict lines with a width greater than 1.0 are rasterized as parallelograms or using Bresenham's algorithm.

If the `VkPhysicalDeviceMaintenance5PropertiesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance5PropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_5_PROPERTIES_KHR`

The `VkPhysicalDeviceMaintenance6PropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance6
typedef struct VkPhysicalDeviceMaintenance6PropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           blockTexelViewCompatibleMultipleLayers;
    uint32_t           maxCombinedImageSamplerDescriptorCount;
    VkBool32           fragmentShadingRateClampCombinerInputs;
} VkPhysicalDeviceMaintenance6PropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `blockTexelViewCompatibleMultipleLayers` is a boolean value indicating that an implementation supports creating image views with `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT` where the `layerCount` member of `subresourceRange` is greater than `1`.
- `maxCombinedImageSamplerDescriptorCount` is the maximum number of combined image sampler descriptors that the implementation uses to access any of the formats that require a sampler $Y'C_BC_R$ conversion supported by the implementation.
- `fragmentShadingRateClampCombinerInputs` is a boolean value indicating that an implementation clamps the inputs to combiner operations.

If the `VkPhysicalDeviceMaintenance6PropertiesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance6PropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_6_PROPERTIES_KHR`

The `VkPhysicalDeviceMaintenance7PropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance7
typedef struct VkPhysicalDeviceMaintenance7PropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           robustFragmentShadingRateAttachmentAccess;
    VkBool32           separateDepthStencilAttachmentAccess;
    uint32_t           maxDescriptorSetTotalUniformBuffersDynamic;
    uint32_t           maxDescriptorSetTotalStorageBuffersDynamic;
    uint32_t           maxDescriptorSetTotalBuffersDynamic;
    uint32_t           maxDescriptorSetUpdateAfterBindTotalUniformBuffersDynamic;
    uint32_t           maxDescriptorSetUpdateAfterBindTotalStorageBuffersDynamic;
    uint32_t           maxDescriptorSetUpdateAfterBindTotalBuffersDynamic;
} VkPhysicalDeviceMaintenance7PropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `robustFragmentShadingRateAttachmentAccess` indicates whether the scaled size of a fragment shading rate attachment **can** be less than the size of the render area. If `robustFragmentShadingRateAttachmentAccess` is `VK_FALSE`, the size of the attachment multiplied by the texel size **must** be greater than or equal to the size of the render area. If it is `VK_TRUE` and the fragment shading rate attachment was created with VkImageSubresourceRange::`baseMipLevel` equal to 0, the scaled size **can** be smaller than the render area, and shading rates for missing texels are defined by texel replacement for invalid texels.
- `separateDepthStencilAttachmentAccess` indicates support for writing to one aspect of a depth/stencil attachment without performing read-modify-write operations on the other aspect. If this property is `VK_TRUE`, writes to one aspect **must** not result in read-modify-write operations on the other aspect. If `VK_FALSE`, writes to one aspect **may** result in writes to the other aspect as defined by render pass load operations, render pass store operations and render pass resolve operations.
- `maxDescriptorSetTotalUniformBuffersDynamic` is the maximum total count of dynamic uniform buffers that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` count against this limit. Only descriptors in descriptor set layouts created without the `VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT` bit set count against this limit. See Dynamic Uniform Buffer.
- `maxDescriptorSetTotalStorageBuffersDynamic` is the maximum total count of dynamic storage buffers that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` count against this limit. Only descriptors in descriptor set layouts created without the `VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT` bit set count against this limit. See Dynamic Storage Buffer.
- `maxDescriptorSetTotalBuffersDynamic` is the maximum total count of dynamic uniform buffers and storage buffers that **can** be included in a pipeline layout. Descriptors with a type of `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC` or `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC` count against this limit. Only descriptors in descriptor set layouts created without the

VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT bit set count against this limit.

- maxDescriptorSetUpdateAfterBindTotalUniformBuffersDynamic is similar to maxDescriptorSetTotalUniformBuffersDynamic but counts descriptors from descriptor sets created with or without the VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT bit set.
- maxDescriptorSetUpdateAfterBindTotalStorageBuffersDynamic is similar to maxDescriptorSetTotalStorageBuffersDynamic but counts descriptors from descriptor sets created with or without the VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT bit set.
- maxDescriptorSetUpdateAfterBindTotalBuffersDynamic is similar to maxDescriptorSetTotalBuffersDynamic but counts descriptors from descriptor sets created with or without the VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT bit set. While an application **can** allocate dynamic storage buffer descriptors from a pool created with the VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT, bindings for these descriptors **must** not be present in any descriptor set layout that includes bindings created with VK_DESCRIPTOR_BINDING_UPDATE_AFTER_BIND_BIT.

If the VkPhysicalDeviceMaintenance7PropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMaintenance7PropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_7_PROPERTIES_KHR

The VkPhysicalDeviceLayeredApiPropertiesListKHR structure is defined as:

```
// Provided by VK_KHR_maintenance7
typedef struct VkPhysicalDeviceLayeredApiPropertiesListKHR {
    VkStructureType                               sType;
    void*                                         pNext;
    uint32_t                                      layeredApiCount;
    VkPhysicalDeviceLayeredApiPropertiesKHR*      pLayeredApis;
} VkPhysicalDeviceLayeredApiPropertiesListKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- layeredApiCount is an integer related to the number of layered implementations underneath the Vulkan physical device, as described below.
- pLayeredApis is a pointer to an array of VkPhysicalDeviceLayeredApiPropertiesKHR in which information regarding the layered implementations underneath the Vulkan physical device are returned.

If pLayeredApis is NULL, then the number of layered implementations that are underneath the top-most Vulkan physical device (i.e. the one returned by vkGetPhysicalDeviceProperties2KHR) is

returned in `layeredApiCount`. Otherwise, `layeredApiCount` **must** be set by the application to the number of elements in the `pLayeredApis` array, and on return the variable is overwritten with the number of values actually written to `pLayeredApis`. If the value of `layeredApiCount` is less than the number of layered implementations underneath the Vulkan physical device, at most `layeredApiCount` values will be written to `pLayeredApis`. An implementation that is not a layer will return 0 in `layeredApiCount`.

In the presence of multiple layered implementations, each element of `pLayeredApis` corresponds to an API implementation that is implemented on top of the API at the previous index. If there are layered implementations underneath a non-Vulkan implementation, they may not be visible in this query as the corresponding APIs may lack such a query.

If the `VkPhysicalDeviceLayeredApiPropertiesListKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceLayeredApiPropertiesListKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_PROPERTIES_LIST_KHR`
- VUID-VkPhysicalDeviceLayeredApiPropertiesListKHR-pLayeredApis-parameter
  If `layeredApiCount` is not `0`, and `pLayeredApis` is not `NULL`, `pLayeredApis` **must** be a valid pointer to an array of `layeredApiCount` VkPhysicalDeviceLayeredApiPropertiesKHR structures

The `VkPhysicalDeviceLayeredApiPropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance7
typedef struct VkPhysicalDeviceLayeredApiPropertiesKHR {
    VkStructureType                    sType;
    void*                              pNext;
    uint32_t                           vendorID;
    uint32_t                           deviceID;
    VkPhysicalDeviceLayeredApiKHR      layeredAPI;
    char                               deviceName[VK_MAX_PHYSICAL_DEVICE_NAME_SIZE];
} VkPhysicalDeviceLayeredApiPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `vendorID` is a unique identifier for the vendor of the layered implementation.
- `deviceID` is a unique identifier for the layered implementation among devices available from the vendor.
- `layeredAPI` is a VkPhysicalDeviceLayeredApiKHR specifying the API implemented by the layered implementation.
- `deviceName` is an array of `VK_MAX_PHYSICAL_DEVICE_NAME_SIZE` `char` containing a null-terminated

UTF-8 string which is the name of the device.

If `layeredAPI` is `VK_PHYSICAL_DEVICE_LAYERED_API_VULKAN_KHR`, additional Vulkan-specific information can be queried by including the VkPhysicalDeviceLayeredApiVulkanPropertiesKHR structure in the `pNext` chain. Otherwise if such a structure is included in the `pNext` chain, it is ignored.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceLayeredApiPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_PROPERTIES_KHR`
- VUID-VkPhysicalDeviceLayeredApiPropertiesKHR-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkPhysicalDeviceLayeredApiVulkanPropertiesKHR
- VUID-VkPhysicalDeviceLayeredApiPropertiesKHR-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

The list of possible API implementations of a layered implementation underneath the Vulkan physical device, as returned in VkPhysicalDeviceLayeredApiPropertiesKHR::`layeredAPI`, are:

```
// Provided by VK_KHR_maintenance7
typedef enum VkPhysicalDeviceLayeredApiKHR {
    VK_PHYSICAL_DEVICE_LAYERED_API_VULKAN_KHR = 0,
    VK_PHYSICAL_DEVICE_LAYERED_API_D3D12_KHR = 1,
    VK_PHYSICAL_DEVICE_LAYERED_API_METAL_KHR = 2,
    VK_PHYSICAL_DEVICE_LAYERED_API_OPENGL_KHR = 3,
    VK_PHYSICAL_DEVICE_LAYERED_API_OPENGLES_KHR = 4,
} VkPhysicalDeviceLayeredApiKHR;
```

- `VK_PHYSICAL_DEVICE_LAYERED_API_VULKAN_KHR` - the device implements the Vulkan API.
- `VK_PHYSICAL_DEVICE_LAYERED_API_D3D12_KHR` - the device implements the D3D12 API.
- `VK_PHYSICAL_DEVICE_LAYERED_API_METAL_KHR` - the device implements the Metal API.
- `VK_PHYSICAL_DEVICE_LAYERED_API_OPENGL_KHR` - the device implements the OpenGL API.
- `VK_PHYSICAL_DEVICE_LAYERED_API_OPENGLES_KHR` - the device implements the OpenGL ES API.

The `VkPhysicalDeviceLayeredApiVulkanPropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_maintenance7
typedef struct VkPhysicalDeviceLayeredApiVulkanPropertiesKHR {
    VkStructureType                sType;
    void*                          pNext;
    VkPhysicalDeviceProperties2    properties;
} VkPhysicalDeviceLayeredApiVulkanPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `properties` is a VkPhysicalDeviceProperties2 in which properties of the underlying layered Vulkan implementation are returned.

The implementation **must** zero-fill the contents of `properties.properties.limits` and `properties.properties.sparseProperties`.

Valid Usage

- VUID-VkPhysicalDeviceLayeredApiVulkanPropertiesKHR-pNext-10011
  Only VkPhysicalDeviceDriverProperties and VkPhysicalDeviceIDProperties are allowed in the `pNext` chain of `properties`

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceLayeredApiVulkanPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_VULKAN_PROPERTIES_KHR`

The `VkPhysicalDeviceConservativeRasterizationPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_conservative_rasterization
typedef struct VkPhysicalDeviceConservativeRasterizationPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    float              primitiveOverestimationSize;
    float              maxExtraPrimitiveOverestimationSize;
    float              extraPrimitiveOverestimationSizeGranularity;
    VkBool32           primitiveUnderestimation;
    VkBool32           conservativePointAndLineRasterization;
    VkBool32           degenerateTrianglesRasterized;
    VkBool32           degenerateLinesRasterized;
    VkBool32           fullyCoveredFragmentShaderInputVariable;
    VkBool32           conservativeRasterizationPostDepthCoverage;
} VkPhysicalDeviceConservativeRasterizationPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `primitiveOverestimationSize` is the size in pixels the generating primitive is increased at each of its edges during conservative rasterization overestimation mode. Even with a size of 0.0, conservative rasterization overestimation rules still apply and if any part of the pixel rectangle is covered by the generating primitive, fragments are generated for the entire pixel. However implementations **may** make the pixel coverage area even more conservative by increasing the size of the generating primitive.
- `maxExtraPrimitiveOverestimationSize` is the maximum size in pixels of extra overestimation the

implementation supports in the pipeline state. A value of 0.0 means the implementation does not support any additional overestimation of the generating primitive during conservative rasterization. A value above 0.0 allows the application to further increase the size of the generating primitive during conservative rasterization overestimation.

- `extraPrimitiveOverestimationSizeGranularity` is the granularity of extra overestimation that can be specified in the pipeline state between 0.0 and `maxExtraPrimitiveOverestimationSize` inclusive. A value of 0.0 means the implementation can use the smallest representable non-zero value in the screen space pixel fixed-point grid.
- `primitiveUnderestimation` is `VK_TRUE` if the implementation supports the `VK_CONSERVATIVE_RASTERIZATION_MODE_UNDERESTIMATE_EXT` conservative rasterization mode in addition to `VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT`. Otherwise the implementation only supports `VK_CONSERVATIVE_RASTERIZATION_MODE_OVERESTIMATE_EXT`.
- `conservativePointAndLineRasterization` is `VK_TRUE` if the implementation supports conservative rasterization of point and line primitives as well as triangle primitives. Otherwise the implementation only supports triangle primitives.
- `degenerateTrianglesRasterized` is `VK_FALSE` if the implementation culls primitives generated from triangles that become zero area after they are quantized to the fixed-point rasterization pixel grid. `degenerateTrianglesRasterized` is `VK_TRUE` if these primitives are not culled and the provoking vertex attributes and depth value are used for the fragments. The primitive area calculation is done on the primitive generated from the clipped triangle if applicable. Zero area primitives are backfacing and the application **can** enable backface culling if desired.
- `degenerateLinesRasterized` is `VK_FALSE` if the implementation culls lines that become zero length after they are quantized to the fixed-point rasterization pixel grid. `degenerateLinesRasterized` is `VK_TRUE` if zero length lines are not culled and the provoking vertex attributes and depth value are used for the fragments.
- `fullyCoveredFragmentShaderInputVariable` is `VK_TRUE` if the implementation supports the SPIR-V builtin fragment shader input variable `FullyCoveredEXT` specifying that conservative rasterization is enabled and the fragment area is fully covered by the generating primitive.
- `conservativeRasterizationPostDepthCoverage` is `VK_TRUE` if the implementation supports conservative rasterization with the `PostDepthCoverage` execution mode enabled. Otherwise the `PostDepthCoverage` execution mode **must** not be used when conservative rasterization is enabled.

If the `VkPhysicalDeviceConservativeRasterizationPropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceConservativeRasterizationPropertiesEXT-sType-sType `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CONSERVATIVE_RASTERIZATION_PROPERTIES_EXT`

The `VkPhysicalDeviceFragmentDensityMapPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_fragment_density_map
typedef struct VkPhysicalDeviceFragmentDensityMapPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkExtent2D         minFragmentDensityTexelSize;
    VkExtent2D         maxFragmentDensityTexelSize;
    VkBool32           fragmentDensityInvocations;
} VkPhysicalDeviceFragmentDensityMapPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `minFragmentDensityTexelSize` is the minimum fragment density texel size.
- `maxFragmentDensityTexelSize` is the maximum fragment density texel size.
- `fragmentDensityInvocations` specifies whether the implementation **may** invoke additional fragment shader invocations for each covered sample.

If the `VkPhysicalDeviceFragmentDensityMapPropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentDensityMapPropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_PROPERTIES_EXT`

The `VkPhysicalDeviceFragmentDensityMap2PropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_fragment_density_map2
typedef struct VkPhysicalDeviceFragmentDensityMap2PropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           subsampledLoads;
    VkBool32           subsampledCoarseReconstructionEarlyAccess;
    uint32_t           maxSubsampledArrayLayers;
    uint32_t           maxDescriptorSetSubsampledSamplers;
} VkPhysicalDeviceFragmentDensityMap2PropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `subsampledLoads` specifies if performing image data read with load operations on subsampled attachments will be resampled to the fragment density of the render pass
- `subsampledCoarseReconstructionEarlyAccess` specifies if performing image data read with samplers created with `flags` containing

VK_SAMPLER_CREATE_SUBSAMPLED_COARSE_RECONSTRUCTION_BIT_EXT in fragment shader will trigger additional reads during VK_PIPELINE_STAGE_VERTEX_SHADER_BIT

- maxSubsampledArrayLayers is the maximum number of VkImageView array layers for usages supporting subsampled samplers
- maxDescriptorSetSubsampledSamplers is the maximum number of subsampled samplers that **can** be included in a VkPipelineLayout

If the VkPhysicalDeviceFragmentDensityMap2PropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentDensityMap2PropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_2_PROPERTIES_EXT

The VkPhysicalDeviceDepthStencilResolveProperties structure is defined as:

```
typedef struct VkPhysicalDeviceDepthStencilResolveProperties {
    VkStructureType       sType;
    void*                 pNext;
    VkResolveModeFlags    supportedDepthResolveModes;
    VkResolveModeFlags    supportedStencilResolveModes;
    VkBool32              independentResolveNone;
    VkBool32              independentResolve;
} VkPhysicalDeviceDepthStencilResolveProperties;
```

or the equivalent

```
// Provided by VK_KHR_depth_stencil_resolve
typedef VkPhysicalDeviceDepthStencilResolveProperties
VkPhysicalDeviceDepthStencilResolvePropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- supportedDepthResolveModes is a bitmask of VkResolveModeFlagBits indicating the set of supported depth resolve modes. VK_RESOLVE_MODE_SAMPLE_ZERO_BIT **must** be included in the set but implementations **may** support additional modes.
- supportedStencilResolveModes is a bitmask of VkResolveModeFlagBits indicating the set of supported stencil resolve modes. VK_RESOLVE_MODE_SAMPLE_ZERO_BIT **must** be included in the set but implementations **may** support additional modes. VK_RESOLVE_MODE_AVERAGE_BIT **must** not be included in the set.
- independentResolveNone is VK_TRUE if the implementation supports setting the depth and stencil

resolve modes to different values when one of those modes is VK_RESOLVE_MODE_NONE. Otherwise the implementation only supports setting both modes to the same value.

- independentResolve is VK_TRUE if the implementation supports all combinations of the supported depth and stencil resolve modes, including setting either depth or stencil resolve mode to VK_RESOLVE_MODE_NONE. An implementation that supports independentResolve **must** also support independentResolveNone.

If the VkPhysicalDeviceDepthStencilResolveProperties structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDepthStencilResolveProperties-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_STENCIL_RESOLVE_PROPERTIES

The VkPhysicalDevicePerformanceQueryPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_performance_query
typedef struct VkPhysicalDevicePerformanceQueryPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           allowCommandBufferQueryCopies;
} VkPhysicalDevicePerformanceQueryPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- allowCommandBufferQueryCopies is VK_TRUE if the performance query pools are allowed to be used with vkCmdCopyQueryPoolResults.

If the VkPhysicalDevicePerformanceQueryPropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePerformanceQueryPropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PERFORMANCE_QUERY_PROPERTIES_KHR

The VkPhysicalDeviceTransformFeedbackPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_transform_feedback
typedef struct VkPhysicalDeviceTransformFeedbackPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxTransformFeedbackStreams;
    uint32_t           maxTransformFeedbackBuffers;
    VkDeviceSize       maxTransformFeedbackBufferSize;
    uint32_t           maxTransformFeedbackStreamDataSize;
    uint32_t           maxTransformFeedbackBufferDataSize;
    uint32_t           maxTransformFeedbackBufferDataStride;
    VkBool32           transformFeedbackQueries;
    VkBool32           transformFeedbackStreamsLinesTriangles;
    VkBool32           transformFeedbackRasterizationStreamSelect;
    VkBool32           transformFeedbackDraw;
} VkPhysicalDeviceTransformFeedbackPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxTransformFeedbackStreams` is the maximum number of vertex streams that can be output from geometry shaders declared with the `GeometryStreams` capability. If the implementation does not support `VkPhysicalDeviceTransformFeedbackFeaturesEXT::geometryStreams` then `maxTransformFeedbackStreams` **must** be `1`.
- `maxTransformFeedbackBuffers` is the maximum number of transform feedback buffers that can be bound for capturing shader outputs from the last pre-rasterization shader stage.
- `maxTransformFeedbackBufferSize` is the maximum size that can be specified when binding a buffer for transform feedback in vkCmdBindTransformFeedbackBuffersEXT.
- `maxTransformFeedbackStreamDataSize` is the maximum amount of data in bytes for each vertex that captured to one or more transform feedback buffers associated with a specific vertex stream.
- `maxTransformFeedbackBufferDataSize` is the maximum amount of data in bytes for each vertex that can be captured to a specific transform feedback buffer.
- `maxTransformFeedbackBufferDataStride` is the maximum stride between each capture of vertex data to the buffer.
- `transformFeedbackQueries` is `VK_TRUE` if the implementation supports the `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT` query type. `transformFeedbackQueries` is `VK_FALSE` if queries of this type **cannot** be created.
- `transformFeedbackStreamsLinesTriangles` is `VK_TRUE` if the implementation supports the geometry shader `OpExecutionMode` of `OutputLineStrip` and `OutputTriangleStrip` in addition to `OutputPoints` when more than one vertex stream is output. If `transformFeedbackStreamsLinesTriangles` is `VK_FALSE` the implementation only supports an `OpExecutionMode` of `OutputPoints` when more than one vertex stream is output from the geometry shader.
- `transformFeedbackRasterizationStreamSelect` is `VK_TRUE` if the implementation supports the `GeometryStreams` SPIR-V capability and the application can use VkPipelineRasterizationStateStreamCreateInfoEXT to modify which vertex stream output is

used for rasterization. Otherwise vertex stream `0` **must** always be used for rasterization.

- `transformFeedbackDraw` is `VK_TRUE` if the implementation supports the vkCmdDrawIndirectByteCountEXT function otherwise the function **must** not be called.

If the `VkPhysicalDeviceTransformFeedbackPropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceTransformFeedbackPropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TRANSFORM_FEEDBACK_PROPERTIES_EXT`

The `VkPhysicalDeviceAccelerationStructurePropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_acceleration_structure
typedef struct VkPhysicalDeviceAccelerationStructurePropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    uint64_t           maxGeometryCount;
    uint64_t           maxInstanceCount;
    uint64_t           maxPrimitiveCount;
    uint32_t           maxPerStageDescriptorAccelerationStructures;
    uint32_t           maxPerStageDescriptorUpdateAfterBindAccelerationStructures;
    uint32_t           maxDescriptorSetAccelerationStructures;
    uint32_t           maxDescriptorSetUpdateAfterBindAccelerationStructures;
    uint32_t           minAccelerationStructureScratchOffsetAlignment;
} VkPhysicalDeviceAccelerationStructurePropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxGeometryCount` is the maximum number of geometries in the bottom level acceleration structure.
- `maxInstanceCount` is the maximum number of instances in the top level acceleration structure.
- `maxPrimitiveCount` is the maximum number of triangles or AABBs in all geometries in the bottom level acceleration structure.
- `maxPerStageDescriptorAccelerationStructures` is the maximum number of acceleration structure bindings that **can** be accessible to a single shader stage in a pipeline layout. Descriptor bindings with a descriptor type of `VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR` count against this limit. Only descriptor bindings in descriptor set layouts created without the `VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT` bit set count against this limit.
- `maxPerStageDescriptorUpdateAfterBindAccelerationStructures` is similar to `maxPerStageDescriptorAccelerationStructures` but counts descriptor bindings from descriptor sets created with or without the `VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT`

bit set.

- maxDescriptorSetAccelerationStructures is the maximum number of acceleration structure descriptors that **can** be included in descriptor bindings in a pipeline layout across all pipeline shader stages and descriptor set numbers. Descriptor bindings with a descriptor type of VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR count against this limit. Only descriptor bindings in descriptor set layouts created without the VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT bit set count against this limit.
- maxDescriptorSetUpdateAfterBindAccelerationStructures is similar to maxDescriptorSetAccelerationStructures but counts descriptor bindings from descriptor sets created with or without the VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT bit set.
- minAccelerationStructureScratchOffsetAlignment is the minimum **required** alignment, in bytes, for scratch data passed in to an acceleration structure build command. The value **must** be a power of two.

Due to the fact that the geometry, instance, and primitive counts are specified at acceleration structure creation as 32-bit values, maxGeometryCount, maxInstanceCount, and maxPrimitiveCount **must** not exceed 2^{32}-1.

If the VkPhysicalDeviceAccelerationStructurePropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceAccelerationStructurePropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ACCELERATION_STRUCTURE_PROPERTIES_KHR

The VkPhysicalDeviceRayTracingPipelinePropertiesKHR structure is defined as:

```
// Provided by VK_KHR_ray_tracing_pipeline
typedef struct VkPhysicalDeviceRayTracingPipelinePropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           shaderGroupHandleSize;
    uint32_t           maxRayRecursionDepth;
    uint32_t           maxShaderGroupStride;
    uint32_t           shaderGroupBaseAlignment;
    uint32_t           shaderGroupHandleCaptureReplaySize;
    uint32_t           maxRayDispatchInvocationCount;
    uint32_t           shaderGroupHandleAlignment;
    uint32_t           maxRayHitAttributeSize;
} VkPhysicalDeviceRayTracingPipelinePropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.

- shaderGroupHandleSize is the size in bytes of the shader header.
- maxRayRecursionDepth is the maximum number of levels of ray recursion allowed in a trace command.
- maxShaderGroupStride is the maximum stride in bytes allowed between shader groups in the shader binding table.
- shaderGroupBaseAlignment is the **required** alignment in bytes for the base of the shader binding table.
- shaderGroupHandleCaptureReplaySize is the number of bytes for the information required to do capture and replay for shader group handles.
- maxRayDispatchInvocationCount is the maximum number of ray generation shader invocations which **may** be produced by a single vkCmdTraceRaysIndirectKHR or vkCmdTraceRaysKHR command.
- shaderGroupHandleAlignment is the **required** alignment in bytes for each entry in a shader binding table. The value **must** be a power of two.
- maxRayHitAttributeSize is the maximum size in bytes for a ray attribute structure

If the VkPhysicalDeviceRayTracingPipelinePropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceRayTracingPipelinePropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_PIPELINE_PROPERTIES_KHR

The VkPhysicalDeviceCooperativeMatrixPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_cooperative_matrix
typedef struct VkPhysicalDeviceCooperativeMatrixPropertiesKHR {
    VkStructureType       sType;
    void*                 pNext;
    VkShaderStageFlags    cooperativeMatrixSupportedStages;
} VkPhysicalDeviceCooperativeMatrixPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- cooperativeMatrixSupportedStages is a bitfield of VkShaderStageFlagBits describing the shader stages that cooperative matrix instructions are supported in. cooperativeMatrixSupportedStages will have the VK_SHADER_STAGE_COMPUTE_BIT bit set if any of the physical device's queues support VK_QUEUE_COMPUTE_BIT.

cooperativeMatrixSupportedStages **must** not have any bits other than VK_SHADER_STAGE_COMPUTE_BIT set.

If the VkPhysicalDeviceCooperativeMatrixPropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceCooperativeMatrixPropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COOPERATIVE_MATRIX_PROPERTIES_KHR

The VkPhysicalDeviceTimelineSemaphoreProperties structure is defined as:

```
typedef struct VkPhysicalDeviceTimelineSemaphoreProperties {
    VkStructureType    sType;
    void*              pNext;
    uint64_t           maxTimelineSemaphoreValueDifference;
} VkPhysicalDeviceTimelineSemaphoreProperties;
```

or the equivalent

```
// Provided by VK_KHR_timeline_semaphore
typedef VkPhysicalDeviceTimelineSemaphoreProperties
VkPhysicalDeviceTimelineSemaphorePropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- maxTimelineSemaphoreValueDifference indicates the maximum difference allowed by the implementation between the current value of a timeline semaphore and any pending signal or wait operations.

If the VkPhysicalDeviceTimelineSemaphoreProperties structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceTimelineSemaphoreProperties-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_PROPERTIES

The VkPhysicalDeviceLineRasterizationPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_line_rasterization
typedef struct VkPhysicalDeviceLineRasterizationPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           lineSubPixelPrecisionBits;
} VkPhysicalDeviceLineRasterizationPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `lineSubPixelPrecisionBits` is the number of bits of subpixel precision in framebuffer coordinates x_f and y_f when rasterizing line segments.

If the `VkPhysicalDeviceLineRasterizationPropertiesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceLineRasterizationPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LINE_RASTERIZATION_PROPERTIES_KHR`

The `VkPhysicalDeviceRobustness2PropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_robustness2
typedef struct VkPhysicalDeviceRobustness2PropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkDeviceSize       robustStorageBufferAccessSizeAlignment;
    VkDeviceSize       robustUniformBufferAccessSizeAlignment;
} VkPhysicalDeviceRobustness2PropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `robustStorageBufferAccessSizeAlignment` is the number of bytes that the range of a storage buffer descriptor is rounded up to when used for bounds-checking when the `robustBufferAccess2` feature is enabled. This value **must** be either 1 or 4.
- `robustUniformBufferAccessSizeAlignment` is the number of bytes that the range of a uniform buffer descriptor is rounded up to when used for bounds-checking when the `robustBufferAccess2` feature is enabled. This value **must** be a power of two in the range [1, 256].

If the `VkPhysicalDeviceRobustness2PropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceRobustness2PropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ROBUSTNESS_2_PROPERTIES_EXT`

The `VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_device_generated_commands
typedef struct VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT {
    VkStructureType                         sType;
    void*                                   pNext;
    uint32_t                                maxIndirectPipelineCount;
    uint32_t                                maxIndirectShaderObjectCount;
    uint32_t                                maxIndirectSequenceCount;
    uint32_t                                maxIndirectCommandsTokenCount;
    uint32_t                                maxIndirectCommandsTokenOffset;
    uint32_t                                maxIndirectCommandsIndirectStride;
    VkIndirectCommandsInputModeFlagsEXT     supportedIndirectCommandsInputModes;
    VkShaderStageFlags                      supportedIndirectCommandsShaderStages;
    VkShaderStageFlags
supportedIndirectCommandsShaderStagesPipelineBinding;
    VkShaderStageFlags
supportedIndirectCommandsShaderStagesShaderBinding;
    VkBool32                                deviceGeneratedCommandsTransformFeedback;
    VkBool32
deviceGeneratedCommandsMultiDrawIndirectCount;
} VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxIndirectPipelineCount` is the maximum number of pipelines passed to vkCreateIndirectExecutionSetEXT.
- `maxIndirectShaderObjectCount` is the maximum number of shader objects passed to vkCreateIndirectExecutionSetEXT. If this value is zero, binding shader objects indirectly is not supported.
- `maxIndirectSequenceCount` is the maximum number of sequences in VkGeneratedCommandsInfoEXT and in VkGeneratedCommandsMemoryRequirementsInfoEXT.
- `maxIndirectCommandsTokenCount` is the maximum number of tokens in VkIndirectCommandsLayoutCreateInfoEXT.
- `maxIndirectCommandsTokenOffset` is the maximum offset in VkIndirectCommandsLayoutTokenEXT.
- `maxIndirectCommandsIndirectStride` is the maximum stream stride in VkIndirectCommandsLayoutCreateInfoEXT.
- `supportedIndirectCommandsInputModes` indicates the supported input modes.

- supportedIndirectCommandsShaderStages indicates the stages which **can** be used to generate indirect commands. Implementations are required to support, at minimum: VK_SHADER_STAGE_VERTEX_BIT, VK_SHADER_STAGE_FRAGMENT_BIT, VK_SHADER_STAGE_COMPUTE_BIT.
- supportedIndirectCommandsShaderStagesPipelineBinding indicates the stages which **can** be used within indirect execution sets for indirectly binding shader stages using pipelines.
- supportedIndirectCommandsShaderStagesShaderBinding indicates the stages which **can** be used within indirect execution sets for indirectly binding shader stages using shader objects.
- deviceGeneratedCommandsTransformFeedback indicates whether the implementation supports interactions with VK_EXT_transform_feedback for pipelines not created with VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT.
- deviceGeneratedCommandsMultiDrawIndirectCount indicates whether the implementation supports COUNT variants of multi-draw indirect tokens.

If the VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_GENERATED_COMMANDS_PROPERTIES_EXT

The VkPhysicalDevicePortabilitySubsetPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_portability_subset
typedef struct VkPhysicalDevicePortabilitySubsetPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           minVertexInputBindingStrideAlignment;
} VkPhysicalDevicePortabilitySubsetPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- minVertexInputBindingStrideAlignment indicates the minimum alignment for vertex input strides. VkVertexInputBindingDescription::stride **must** be a multiple of, and at least as large as, this value. The value **must** be a power of two.

If the VkPhysicalDevicePortabilitySubsetPropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePortabilitySubsetPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PORTABILITY_SUBSET_PROPERTIES_KHR`

The `VkPhysicalDeviceFragmentShadingRatePropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_fragment_shading_rate
typedef struct VkPhysicalDeviceFragmentShadingRatePropertiesKHR {
    VkStructureType          sType;
    void*                    pNext;
    VkExtent2D               minFragmentShadingRateAttachmentTexelSize;
    VkExtent2D               maxFragmentShadingRateAttachmentTexelSize;
    uint32_t                 maxFragmentShadingRateAttachmentTexelSizeAspectRatio;
    VkBool32                 primitiveFragmentShadingRateWithMultipleViewports;
    VkBool32                 layeredShadingRateAttachments;
    VkBool32                 fragmentShadingRateNonTrivialCombinerOps;
    VkExtent2D               maxFragmentSize;
    uint32_t                 maxFragmentSizeAspectRatio;
    uint32_t                 maxFragmentShadingRateCoverageSamples;
    VkSampleCountFlagBits    maxFragmentShadingRateRasterizationSamples;
    VkBool32                 fragmentShadingRateWithShaderDepthStencilWrites;
    VkBool32                 fragmentShadingRateWithSampleMask;
    VkBool32                 fragmentShadingRateWithShaderSampleMask;
    VkBool32                 fragmentShadingRateWithConservativeRasterization;
    VkBool32                 fragmentShadingRateWithFragmentShaderInterlock;
    VkBool32                 fragmentShadingRateWithCustomSampleLocations;
    VkBool32                 fragmentShadingRateStrictMultiplyCombiner;
} VkPhysicalDeviceFragmentShadingRatePropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `minFragmentShadingRateAttachmentTexelSize` indicates minimum supported width and height of the portion of the framebuffer corresponding to each texel in a fragment shading rate attachment. Each value **must** be less than or equal to the values in `maxFragmentShadingRateAttachmentTexelSize`. Each value **must** be a power-of-two. It **must** be (0,0) if the `attachmentFragmentShadingRate` feature is not supported.
- `maxFragmentShadingRateAttachmentTexelSize` indicates maximum supported width and height of the portion of the framebuffer corresponding to each texel in a fragment shading rate attachment. Each value **must** be greater than or equal to the values in `minFragmentShadingRateAttachmentTexelSize`. Each value **must** be a power-of-two. It **must** be (0,0) if the `attachmentFragmentShadingRate` feature is not supported.
- `maxFragmentShadingRateAttachmentTexelSizeAspectRatio` indicates the maximum ratio between the width and height of the portion of the framebuffer corresponding to each texel in a fragment shading rate attachment. `maxFragmentShadingRateAttachmentTexelSizeAspectRatio` **must**

be a power-of-two value, and **must** be less than or equal to max(maxFragmentShadingRateAttachmentTexelSize.width / minFragmentShadingRateAttachmentTexelSize.height, maxFragmentShadingRateAttachmentTexelSize.height / minFragmentShadingRateAttachmentTexelSize.width). It **must** be 0 if the attachmentFragmentShadingRate feature is not supported.

- primitiveFragmentShadingRateWithMultipleViewports specifies whether the primitive fragment shading rate **can** be used when multiple viewports are used. If this value is VK_FALSE, only a single viewport **must** be used, and applications **must** not write to the ViewportIndex built-in when setting PrimitiveShadingRateKHR. It **must** be VK_FALSE if the geometryShader feature is not supported, or if the primitiveFragmentShadingRate feature is not supported.
- layeredShadingRateAttachments specifies whether a shading rate attachment image view **can** be created with multiple layers. If this value is VK_FALSE, when creating an image view with a usage that includes VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR, layerCount **must** be 1. It **must** be VK_FALSE if the multiview feature, or the geometryShader feature is not supported, or if the attachmentFragmentShadingRate feature is not supported.
- fragmentShadingRateNonTrivialCombinerOps specifies whether VkFragmentShadingRateCombinerOpKHR enums other than VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR or VK_FRAGMENT_SHADING_RATE_COMBINER_OP_REPLACE_KHR **can** be used. It **must** be VK_FALSE unless either the primitiveFragmentShadingRate or attachmentFragmentShadingRate feature is supported.
- maxFragmentSize indicates the maximum supported width and height of a fragment. Its width and height members **must** both be power-of-two values. This limit is purely informational, and is not validated.
- maxFragmentSizeAspectRatio indicates the maximum ratio between the width and height of a fragment. maxFragmentSizeAspectRatio **must** be a power-of-two value, and **must** be less than or equal to the maximum of the width and height members of maxFragmentSize. This limit is purely informational, and is not validated.
- maxFragmentShadingRateCoverageSamples specifies the maximum number of coverage samples supported in a single fragment. maxFragmentShadingRateCoverageSamples **must** be less than or equal to the product of the width and height members of maxFragmentSize, and the sample count reported by maxFragmentShadingRateRasterizationSamples. maxFragmentShadingRateCoverageSamples **must** be less than or equal to maxSampleMaskWords × 32 if fragmentShadingRateWithShaderSampleMask is supported. This limit is purely informational, and is not validated.
- maxFragmentShadingRateRasterizationSamples is a VkSampleCountFlagBits value specifying the maximum sample rate supported when a fragment covers multiple pixels. This limit is purely informational, and is not validated.
- fragmentShadingRateWithShaderDepthStencilWrites specifies whether the implementation supports writing FragDepth or FragStencilRefEXT from a fragment shader for multi-pixel fragments. If this value is VK_FALSE, writing to those built-ins will clamp the fragment shading rate to (1,1).
- fragmentShadingRateWithSampleMask specifies whether the implementation supports setting valid bits of VkPipelineMultisampleStateCreateInfo::pSampleMask to 0 for multi-pixel fragments. If this

value is VK_FALSE, zeroing valid bits in the sample mask will clamp the fragment shading rate to (1,1).

- fragmentShadingRateWithShaderSampleMask specifies whether the implementation supports reading or writing SampleMask for multi-pixel fragments. If this value is VK_FALSE, using that built-in will clamp the fragment shading rate to (1,1).
- fragmentShadingRateWithConservativeRasterization specifies whether conservative rasterization is supported for multi-pixel fragments. It **must** be VK_FALSE if VK_EXT_conservative_rasterization is not supported. If this value is VK_FALSE, using conservative rasterization will clamp the fragment shading rate to (1,1).
- fragmentShadingRateWithFragmentShaderInterlock specifies whether fragment shader interlock is supported for multi-pixel fragments. It **must** be VK_FALSE if VK_EXT_fragment_shader_interlock is not supported. If this value is VK_FALSE, using fragment shader interlock will clamp the fragment shading rate to (1,1).
- fragmentShadingRateWithCustomSampleLocations specifies whether custom sample locations are supported for multi-pixel fragments. It **must** be VK_FALSE if VK_EXT_sample_locations is not supported. If this value is VK_FALSE, using custom sample locations will clamp the fragment shading rate to (1,1).
- fragmentShadingRateStrictMultiplyCombiner specifies whether VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MUL_KHR accurately performs a multiplication or not. Implementations where this value is VK_FALSE will instead combine rates with an addition. If fragmentShadingRateNonTrivialCombinerOps is VK_FALSE, implementations **must** report this as VK_FALSE. If fragmentShadingRateNonTrivialCombinerOps is VK_TRUE, implementations **should** report this as VK_TRUE.

NOTE

Multiplication of the combiner rates using the fragment width/height in linear space is equivalent to an addition of those values in log2 space. Some implementations inadvertently implemented an addition in linear space due to unclear requirements originating outside of this specification. This resulted in fragmentShadingRateStrictMultiplyCombiner being added. Fortunately, this only affects situations where a rate of 1 in either dimension is combined with another rate of 1. All other combinations result in the exact same result as if multiplication was performed in linear space due to the clamping logic, and the fact that both the sum and product of 2 and 2 are equal. In many cases, this limit will not affect the correct operation of applications.

If the VkPhysicalDeviceFragmentShadingRatePropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

These properties are related to fragment shading rates.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentShadingRatePropertiesKHR-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_PROPERTIES_KHR

The VkPhysicalDeviceCustomBorderColorPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_custom_border_color
typedef struct VkPhysicalDeviceCustomBorderColorPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxCustomBorderColorSamplers;
} VkPhysicalDeviceCustomBorderColorPropertiesEXT;
```

- maxCustomBorderColorSamplers indicates the maximum number of samplers with custom border colors which **can** simultaneously exist on a device.

If the VkPhysicalDeviceCustomBorderColorPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceCustomBorderColorPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CUSTOM_BORDER_COLOR_PROPERTIES_EXT

The VkPhysicalDeviceProvokingVertexPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_provoking_vertex
typedef struct VkPhysicalDeviceProvokingVertexPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           provokingVertexModePerPipeline;
    VkBool32           transformFeedbackPreservesTriangleFanProvokingVertex;
} VkPhysicalDeviceProvokingVertexPropertiesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- provokingVertexModePerPipeline indicates whether the implementation supports graphics pipelines with different provoking vertex modes within the same render pass instance.
- transformFeedbackPreservesTriangleFanProvokingVertex indicates whether the implementation can preserve the provoking vertex order when writing triangle fan vertices to transform feedback.

If the VkPhysicalDeviceProvokingVertexPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceProvokingVertexPropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROVOKING_VERTEX_PROPERTIES_EXT`

The `VkPhysicalDeviceHostImageCopyPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkPhysicalDeviceHostImageCopyPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           copySrcLayoutCount;
    VkImageLayout*     pCopySrcLayouts;
    uint32_t           copyDstLayoutCount;
    VkImageLayout*     pCopyDstLayouts;
    uint8_t            optimalTilingLayoutUUID[VK_UUID_SIZE];
    VkBool32           identicalMemoryTypeRequirements;
} VkPhysicalDeviceHostImageCopyPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `copySrcLayoutCount` is an integer related to the number of image layouts for host copies from images available or queried, as described below.
- `pCopySrcLayouts` is a pointer to an array of VkImageLayout in which supported image layouts for use with host copy operations from images are returned.
- `copyDstLayoutCount` is an integer related to the number of image layouts for host copies to images available or queried, as described below.
- `pCopyDstLayouts` is a pointer to an array of VkImageLayout in which supported image layouts for use with host copy operations to images are returned.
- `optimalTilingLayoutUUID` is an array of `VK_UUID_SIZE` `uint8_t` values representing a universally unique identifier for the implementation's swizzling layout of images created with `VK_IMAGE_TILING_OPTIMAL`.
- `identicalMemoryTypeRequirements` indicates that specifying the `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT` flag in VkImageCreateInfo::`usage` does not affect the memory type requirements of the image.

If the `VkPhysicalDeviceHostImageCopyPropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

If `pCopyDstLayouts` is `NULL`, then the number of image layouts that are supported in VkCopyMemoryToImageInfoEXT::`dstImageLayout` and VkCopyImageToImageInfoEXT::`dstImageLayout` is returned in `copyDstLayoutCount`. Otherwise, `copyDstLayoutCount` **must** be set by the application to the number of elements in the `pCopyDstLayouts` array, and on return the variable

is overwritten with the number of values actually written to pCopyDstLayouts. If the value of copyDstLayoutCount is less than the number of image layouts that are supported, at most copyDstLayoutCount values will be written to pCopyDstLayouts. The implementation **must** include the VK_IMAGE_LAYOUT_GENERAL image layout in pCopyDstLayouts.

If pCopySrcLayouts is NULL, then the number of image layouts that are supported in VkCopyImageToMemoryInfoEXT::srcImageLayout and VkCopyImageToImageInfoEXT::srcImageLayout is returned in copySrcLayoutCount. Otherwise, copySrcLayoutCount **must** be set by the application to the number of elements in the pCopySrcLayouts array, and on return the variable is overwritten with the number of values actually written to pCopySrcLayouts. If the value of copySrcLayoutCount is less than the number of image layouts that are supported, at most copySrcLayoutCount values will be written to pCopySrcLayouts. The implementation **must** include the VK_IMAGE_LAYOUT_GENERAL image layout in pCopySrcLayouts.

The optimalTilingLayoutUUID value can be used to ensure compatible data layouts when using the VK_HOST_IMAGE_COPY_MEMCPY_EXT flag in vkCopyMemoryToImageEXT and vkCopyImageToMemoryEXT.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceHostImageCopyPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_HOST_IMAGE_COPY_PROPERTIES_EXT
- VUID-VkPhysicalDeviceHostImageCopyPropertiesEXT-pCopySrcLayouts-parameter
  If copySrcLayoutCount is not 0, and pCopySrcLayouts is not NULL, pCopySrcLayouts **must** be a valid pointer to an array of copySrcLayoutCount VkImageLayout values
- VUID-VkPhysicalDeviceHostImageCopyPropertiesEXT-pCopyDstLayouts-parameter
  If copyDstLayoutCount is not 0, and pCopyDstLayouts is not NULL, pCopyDstLayouts **must** be a valid pointer to an array of copyDstLayoutCount VkImageLayout values

The VkPhysicalDeviceMultiDrawPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_multi_draw
typedef struct VkPhysicalDeviceMultiDrawPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxMultiDrawCount;
} VkPhysicalDeviceMultiDrawPropertiesEXT;
```

The members of the VkPhysicalDeviceMultiDrawPropertiesEXT structure describe the following features:

- maxMultiDrawCount indicates the maximum number of draw calls which **can** be batched into a single multidraw.

If the VkPhysicalDeviceMultiDrawPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in

with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMultiDrawPropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTI_DRAW_PROPERTIES_EXT`

The `VkPhysicalDeviceNestedCommandBufferPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_nested_command_buffer
typedef struct VkPhysicalDeviceNestedCommandBufferPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxCommandBufferNestingLevel;
} VkPhysicalDeviceNestedCommandBufferPropertiesEXT;
```

The members of the `VkPhysicalDeviceNestedCommandBufferPropertiesEXT` structure describe the following features:

- `maxCommandBufferNestingLevel` indicates the maximum nesting level of calls to vkCmdExecuteCommands from Secondary Command Buffers. A `maxCommandBufferNestingLevel` of `UINT32_MAX` means there is no limit to the nesting level.

If the `VkPhysicalDeviceNestedCommandBufferPropertiesEXT` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceNestedCommandBufferPropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NESTED_COMMAND_BUFFER_PROPERTIES_EXT`

The `VkPhysicalDeviceFragmentShaderBarycentricPropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_fragment_shader_barycentric
typedef struct VkPhysicalDeviceFragmentShaderBarycentricPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           triStripVertexOrderIndependentOfProvokingVertex;
} VkPhysicalDeviceFragmentShaderBarycentricPropertiesKHR;
```

- When the provoking vertex mode is `VK_PROVOKING_VERTEX_MODE_LAST_VERTEX_EXT`, and the primitive order is odd in a triangle strip, the ordering of vertices is defined in last vertex table. `triStripVertexOrderIndependentOfProvokingVertex` equal to `VK_TRUE` indicates that the implementation ignores this and uses the vertex order defined by

VK_PROVOKING_VERTEX_MODE_FIRST_VERTEX_EXT instead.

If the VkPhysicalDeviceFragmentShaderBarycentricPropertiesKHR structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceFragmentShaderBarycentricPropertiesKHR-sType-sType sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_BARYCENTRIC_PROPERTIES_KHR

The VkPhysicalDeviceShaderModuleIdentifierPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_shader_module_identifier
typedef struct VkPhysicalDeviceShaderModuleIdentifierPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint8_t            shaderModuleIdentifierAlgorithmUUID[VK_UUID_SIZE];
} VkPhysicalDeviceShaderModuleIdentifierPropertiesEXT;
```

The members of the VkPhysicalDeviceShaderModuleIdentifierPropertiesEXT structure describe the following:

- shaderModuleIdentifierAlgorithmUUID is an array of VK_UUID_SIZE uint8_t values which uniquely represents the algorithm used to compute an identifier in vkGetShaderModuleIdentifierEXT and vkGetShaderModuleCreateInfoIdentifierEXT. Implementations **should** not change this value in different driver versions if the algorithm used to compute an identifier is the same.

NOTE

The algorithm UUID may be the same in different ICDs if the algorithms are guaranteed to produce the same results. This may happen in driver stacks which support different kinds of hardware with shared code.

Khronos' conformance testing can not guarantee that shaderModuleIdentifierAlgorithmUUID values are actually unique, so implementors should make their own best efforts to ensure that their UUID is unlikely to conflict with other implementations which may use a different algorithm. In particular, hard-coded values which easily conflict, such as all-0 bits, **should** never be used. Hard-coded values are acceptable if best effort is ensured that the value will not accidentally conflict.

If the VkPhysicalDeviceShaderModuleIdentifierPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderModuleIdentifierPropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MODULE_IDENTIFIER_PROPERTIES_EXT`

The `VkPhysicalDeviceExtendedDynamicState3PropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_extended_dynamic_state3
typedef struct VkPhysicalDeviceExtendedDynamicState3PropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           dynamicPrimitiveTopologyUnrestricted;
} VkPhysicalDeviceExtendedDynamicState3PropertiesEXT;
```

- `dynamicPrimitiveTopologyUnrestricted` indicates that the implementation allows `vkCmdSetPrimitiveTopologyEXT` to use a different primitive topology class to the one specified in the active graphics pipeline.

If the `VkPhysicalDeviceExtendedDynamicState3PropertiesEXT` structure is included in the `pNext` chain of the `VkPhysicalDeviceProperties2` structure passed to `vkGetPhysicalDeviceProperties2KHR`, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceExtendedDynamicState3PropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTENDED_DYNAMIC_STATE_3_PROPERTIES_EXT`

The `VkPhysicalDeviceOpacityMicromapPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_opacity_micromap
typedef struct VkPhysicalDeviceOpacityMicromapPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           maxOpacity2StateSubdivisionLevel;
    uint32_t           maxOpacity4StateSubdivisionLevel;
} VkPhysicalDeviceOpacityMicromapPropertiesEXT;
```

- `sType` is a `VkStructureType` value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxOpacity2StateSubdivisionLevel` is the maximum allowed `subdivisionLevel` when `format` is `VK_OPACITY_MICROMAP_FORMAT_2_STATE_EXT`
- `maxOpacity4StateSubdivisionLevel` is the maximum allowed `subdivisionLevel` when `format` is

VK_OPACITY_MICROMAP_FORMAT_4_STATE_EXT

If the VkPhysicalDeviceOpacityMicromapPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceOpacityMicromapPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_OPACITY_MICROMAP_PROPERTIES_EXT

The VkPhysicalDeviceShaderObjectPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_shader_object
typedef struct VkPhysicalDeviceShaderObjectPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    uint8_t            shaderBinaryUUID[VK_UUID_SIZE];
    uint32_t           shaderBinaryVersion;
} VkPhysicalDeviceShaderObjectPropertiesEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- shaderBinaryUUID is an array of VK_UUID_SIZE uint8_t values representing a universally unique identifier for one or more implementations whose shader binaries are guaranteed to be compatible with each other.
- shaderBinaryVersion is an unsigned integer incremented to represent backwards compatible differences between implementations with the same shaderBinaryUUID.

The purpose and usage of the values of this structure are described in greater detail in Binary Shader Compatibility.

If the VkPhysicalDeviceShaderObjectPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceShaderObjectPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_OBJECT_PROPERTIES_EXT

The VkPhysicalDevicePipelineBinaryPropertiesKHR structure is defined as:

```
// Provided by VK_KHR_pipeline_binary
typedef struct VkPhysicalDevicePipelineBinaryPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           pipelineBinaryInternalCache;
    VkBool32           pipelineBinaryInternalCacheControl;
    VkBool32           pipelineBinaryPrefersInternalCache;
    VkBool32           pipelineBinaryPrecompiledInternalCache;
    VkBool32           pipelineBinaryCompressedData;
} VkPhysicalDevicePipelineBinaryPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `pipelineBinaryInternalCache` specifies that the implementation maintains a pipeline cache internal to the implementation. If this is `VK_TRUE`, applications **can** create pipeline binaries with only a pipeline create info, and in this case, an implementation **may** be able to create a pipeline binary directly without application needing to capture the binary itself.
- `pipelineBinaryInternalCacheControl` specifies whether the driver's internal cache **can** be disabled. If this property is `VK_TRUE` VkDevicePipelineBinaryInternalCacheControlKHR ::`disableInternalCache` **can** be used to disable the driver's internal cache, allowing an application to take full control of both memory and disk usage.
- `pipelineBinaryPrefersInternalCache` specifies that the implementation prefers to maintain an internal cache, and applications **should** not store pipeline binaries in their own on-disk caches to avoid increased on-disk storage requirements. Applications are encouraged to only store pipeline keys instead, and aim to create pipeline binaries from key alone on subsequent runs of the application.
- `pipelineBinaryPrecompiledInternalCache` specifies that the implementation **may** have pipeline binaries in its internal cache, which is populated without the application ever having generated that pipeline itself. Applications **can** attempt to create binaries without extracting pipeline binary data from the pipeline prior for a set of pipeline keys, including from previous runs of the application.
- `pipelineBinaryCompressedData` specifies that the binary data is already compressed and so applications **should** not attempt to compress it.

NOTE

These properties tend to be platform specific and may change depending on external configuration which is outside the scope of this specification. These properties are intended to guide applications when implementations have dedicated caching solutions available. In particular, if the `pipelineBinaryPrefersInternalCache` limit is exposed, relying on the internal cache may provide some advantage compared to an application-specific solution. An application with its own dedicated solution may still use its own caching system even with this limit exposed.

If the `VkPhysicalDevicePipelineBinaryPropertiesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in

with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDevicePipelineBinaryPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_BINARY_PROPERTIES_KHR`

The `VkPhysicalDeviceComputeShaderDerivativesPropertiesKHR` structure is defined as:

```
// Provided by VK_KHR_compute_shader_derivatives
typedef struct VkPhysicalDeviceComputeShaderDerivativesPropertiesKHR {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           meshAndTaskShaderDerivatives;
} VkPhysicalDeviceComputeShaderDerivativesPropertiesKHR;
```

The members of the `VkPhysicalDeviceComputeShaderDerivativesPropertiesKHR` structure describe the following:

- `meshAndTaskShaderDerivatives` indicates whether the mesh and task shader stages support the `ComputeDerivativeGroupQuadsKHR` and `ComputeDerivativeGroupLinearKHR` SPIR-V capabilities.

If the `VkPhysicalDeviceComputeShaderDerivativesPropertiesKHR` structure is included in the `pNext` chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceComputeShaderDerivativesPropertiesKHR-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COMPUTE_SHADER_DERIVATIVES_PROPERTIES_KHR`

The `VkPhysicalDeviceMapMemoryPlacedPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_map_memory_placed
typedef struct VkPhysicalDeviceMapMemoryPlacedPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkDeviceSize       minPlacedMemoryMapAlignment;
} VkPhysicalDeviceMapMemoryPlacedPropertiesEXT;
```

The members of the `VkPhysicalDeviceMapMemoryPlacedPropertiesEXT` structure describe the following:

- `minPlacedMemoryMapAlignment` is the minimum alignment required for memory object offsets and virtual address ranges when using placed memory mapping.

If the VkPhysicalDeviceMapMemoryPlacedPropertiesEXT structure is included in the pNext chain of the VkPhysicalDeviceProperties2 structure passed to vkGetPhysicalDeviceProperties2KHR, it is filled in with each corresponding implementation-dependent property.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceMapMemoryPlacedPropertiesEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAP_MEMORY_PLACED_PROPERTIES_EXT

41.1. Limit Requirements

The following table specifies the **required** minimum/maximum for all Vulkan graphics implementations. Where a limit corresponds to a fine-grained device feature which is **optional**, the feature name is listed with two **required** limits, one when the feature is supported and one when it is not supported. If an implementation supports a feature, the limits reported are the same whether or not the feature is enabled.

Table 61. Required Limit Types

Type	Limit	Feature
uint32_t	maxImageDimension1D	-
uint32_t	maxImageDimension2D	-
uint32_t	maxImageDimension3D	-
uint32_t	maxImageDimensionCube	-
uint32_t	maxImageArrayLayers	-
uint32_t	maxTexelBufferElements	-
uint32_t	maxUniformBufferRange	-
uint32_t	maxStorageBufferRange	-
uint32_t	maxPushConstantsSize	-
uint32_t	maxMemoryAllocationCount	-
uint32_t	maxSamplerAllocationCount	-
VkDeviceSize	bufferImageGranularity	-
VkDeviceSize	sparseAddressSpaceSize	sparseBinding
uint32_t	maxBoundDescriptorSets	-
uint32_t	maxPerStageDescriptorSamplers	-
uint32_t	maxPerStageDescriptorUniformBuffers	-
uint32_t	maxPerStageDescriptorStorageBuffers	-
uint32_t	maxPerStageDescriptorSampledImages	-
uint32_t	maxPerStageDescriptorStorageImages	-

Type	Limit	Feature
uint32_t	maxPerStageDescriptorInputAttachments	-
uint32_t	maxPerStageResources	-
uint32_t	maxDescriptorSetSamplers	-
uint32_t	maxDescriptorSetUniformBuffers	-
uint32_t	maxDescriptorSetUniformBuffersDynamic	-
uint32_t	maxDescriptorSetStorageBuffers	-
uint32_t	maxDescriptorSetStorageBuffersDynamic	-
uint32_t	maxDescriptorSetSampledImages	-
uint32_t	maxDescriptorSetStorageImages	-
uint32_t	maxDescriptorSetInputAttachments	-
uint32_t	maxVertexInputAttributes	-
uint32_t	maxVertexInputBindings	-
uint32_t	maxVertexInputAttributeOffset	-
uint32_t	maxVertexInputBindingStride	-
uint32_t	maxVertexOutputComponents	-
uint32_t	maxTessellationGenerationLevel	tessellationShader
uint32_t	maxTessellationPatchSize	tessellationShader
uint32_t	maxTessellationControlPerVertexInputComponents	tessellationShader
uint32_t	maxTessellationControlPerVertexOutputComponents	tessellationShader
uint32_t	maxTessellationControlPerPatchOutputComponents	tessellationShader
uint32_t	maxTessellationControlTotalOutputComponents	tessellationShader
uint32_t	maxTessellationEvaluationInputComponents	tessellationShader
uint32_t	maxTessellationEvaluationOutputComponents	tessellationShader
uint32_t	maxGeometryShaderInvocations	geometryShader
uint32_t	maxGeometryInputComponents	geometryShader
uint32_t	maxGeometryOutputComponents	geometryShader
uint32_t	maxGeometryOutputVertices	geometryShader
uint32_t	maxGeometryTotalOutputComponents	geometryShader
uint32_t	maxFragmentInputComponents	-
uint32_t	maxFragmentOutputAttachments	-
uint32_t	maxFragmentDualSrcAttachments	dualSrcBlend
uint32_t	maxFragmentCombinedOutputResources	-
uint32_t	maxComputeSharedMemorySize	-
3 × uint32_t	maxComputeWorkGroupCount	-
uint32_t	maxComputeWorkGroupInvocations	-

Type	Limit	Feature
3 × uint32_t	maxComputeWorkGroupSize	-
uint32_t	subPixelPrecisionBits	-
uint32_t	subTexelPrecisionBits	-
uint32_t	mipmapPrecisionBits	-
uint32_t	maxDrawIndexedIndexValue	fullDrawIndexUint32
uint32_t	maxDrawIndirectCount	multiDrawIndirect
float	maxSamplerLodBias	-
float	maxSamplerAnisotropy	samplerAnisotropy
uint32_t	maxViewports	multiViewport
2 × uint32_t	maxViewportDimensions	-
2 × float	viewportBoundsRange	-
uint32_t	viewportSubPixelBits	-
size_t	minMemoryMapAlignment	-
VkDeviceSize	minTexelBufferOffsetAlignment	-
VkDeviceSize	minUniformBufferOffsetAlignment	-
VkDeviceSize	minStorageBufferOffsetAlignment	-
int32_t	minTexelOffset	-
uint32_t	maxTexelOffset	-
int32_t	minTexelGatherOffset	shaderImageGatherExtended
uint32_t	maxTexelGatherOffset	shaderImageGatherExtended
float	minInterpolationOffset	sampleRateShading
float	maxInterpolationOffset	sampleRateShading
uint32_t	subPixelInterpolationOffsetBits	sampleRateShading
uint32_t	maxFramebufferWidth	-
uint32_t	maxFramebufferHeight	-
uint32_t	maxFramebufferLayers	-
VkSampleCountFlags	framebufferColorSampleCounts	-
VkSampleCountFlags	framebufferDepthSampleCounts	-
VkSampleCountFlags	framebufferStencilSampleCounts	-
VkSampleCountFlags	framebufferNoAttachmentsSampleCounts	-
uint32_t	maxColorAttachments	-

Type	Limit	Feature
VkSampleCountFlags	sampledImageColorSampleCounts	-
VkSampleCountFlags	sampledImageIntegerSampleCounts	-
VkSampleCountFlags	sampledImageDepthSampleCounts	-
VkSampleCountFlags	sampledImageStencilSampleCounts	-
VkSampleCountFlags	storageImageSampleCounts	shaderStorageImageMultisample
uint32_t	maxSampleMaskWords	-
VkBool32	timestampComputeAndGraphics	-
float	timestampPeriod	-
uint32_t	maxClipDistances	shaderClipDistance
uint32_t	maxCullDistances	shaderCullDistance
uint32_t	maxCombinedClipAndCullDistances	shaderCullDistance
uint32_t	discreteQueuePriorities	-
2 × float	pointSizeRange	largePoints
2 × float	lineWidthRange	wideLines
float	pointSizeGranularity	largePoints
float	lineWidthGranularity	wideLines
VkBool32	strictLines	-
VkBool32	standardSampleLocations	-
VkDeviceSize	optimalBufferCopyOffsetAlignment	-
VkDeviceSize	optimalBufferCopyRowPitchAlignment	-
VkDeviceSize	nonCoherentAtomSize	-
uint32_t	maxDiscardRectangles	VK_EXT_discard_rectangles
VkDeviceSize	maxBufferSize	maintenance4
float	primitiveOverestimationSize	VK_EXT_conservative_rasterization
VkBool32	maxExtraPrimitiveOverestimationSize	VK_EXT_conservative_rasterization
float	extraPrimitiveOverestimationSizeGranularity	VK_EXT_conservative_rasterization
VkBool32	degenerateTriangleRasterized	VK_EXT_conservative_rasterization
float	degenerateLinesRasterized	VK_EXT_conservative_rasterization

Type	Limit	Feature
VkBool32	fullyCoveredFragmentShaderInputVariable	VK_EXT_conservative_rasterization
VkBool32	conservativeRasterizationPostDepthCoverage	VK_EXT_conservative_rasterization
uint32_t	maxVertexAttribDivisor	VK_KHR_vertex_attribute_divisor
uint32_t	maxTransformFeedbackStreams	VK_EXT_transform_feedback
uint32_t	maxTransformFeedbackBuffers	VK_EXT_transform_feedback
VkDeviceSize	maxTransformFeedbackBufferSize	VK_EXT_transform_feedback
uint32_t	maxTransformFeedbackStreamDataSize	VK_EXT_transform_feedback
uint32_t	maxTransformFeedbackBufferDataSize	VK_EXT_transform_feedback
uint32_t	maxTransformFeedbackBufferDataStride	VK_EXT_transform_feedback
VkBool32	transformFeedbackQueries	VK_EXT_transform_feedback
VkBool32	transformFeedbackStreamsLinesTriangles	VK_EXT_transform_feedback
VkBool32	transformFeedbackRasterizationStreamSelect	VK_EXT_transform_feedback
VkBool32	transformFeedbackDraw	VK_EXT_transform_feedback
VkExtent2D	minFragmentDensityTexelSize	fragmentDensityMap
VkExtent2D	maxFragmentDensityTexelSize	fragmentDensityMap
VkBool32	fragmentDensityInvocations	fragmentDensityMap
VkBool32	subsampledLoads	VK_EXT_fragment_density_map2
VkBool32	subsampledCoarseReconstructionEarlyAccess	VK_EXT_fragment_density_map2
uint32_t	maxSubsampledArrayLayers	VK_EXT_fragment_density_map2
uint32_t	maxDescriptorSetSubsampledSamplers	VK_EXT_fragment_density_map2
uint32_t	maxGeometryCount	VK_NV_ray_tracing, VK_KHR_acceleration_structure
uint32_t	maxInstanceCount	VK_NV_ray_tracing, VK_KHR_acceleration_structure
uint32_t	shaderGroupHandleSize	VK_NV_ray_tracing, VK_KHR_ray_tracing_pipeline
uint32_t	maxShaderGroupStride	VK_NV_ray_tracing, VK_KHR_ray_tracing_pipeline
uint32_t	shaderGroupBaseAlignment	VK_NV_ray_tracing, VK_KHR_ray_tracing_pipeline
uint32_t	maxPerStageDescriptorAccelerationStructures	VK_KHR_acceleration_structure

Type	Limit	Feature
uint32_t	maxPerStageDescriptorUpdateAfterBindAccelerationStructures	VK_KHR_acceleration_structure
uint32_t	maxDescriptorSetAccelerationStructures	VK_NV_ray_tracing, VK_KHR_acceleration_structure
uint32_t	maxDescriptorSetUpdateAfterBindAccelerationStructures	VK_KHR_acceleration_structure
uint32_t	minAccelerationStructureScratchOffsetAlignment	VK_KHR_acceleration_structure
uint32_t	maxRayRecursionDepth	VK_KHR_ray_tracing_pipeline
uint32_t	shaderGroupHandleCaptureReplaySize	VK_KHR_ray_tracing_pipeline
uint32_t	maxRayDispatchInvocationCount	VK_KHR_ray_tracing_pipeline
uint32_t	shaderGroupHandleAlignment	VK_KHR_ray_tracing_pipeline
uint32_t	maxRayHitAttributeSize	VK_KHR_ray_tracing_pipeline
uint64_t	maxTimelineSemaphoreValueDifference	timelineSemaphore
uint32_t	lineSubPixelPrecisionBits	VK_KHR_line_rasterization, VK_EXT_line_rasterization
uint32_t	maxCustomBorderColorSamplers	VK_EXT_custom_border_color
VkDeviceSize	robustStorageBufferAccessSizeAlignment	VK_EXT_robustness2
VkDeviceSize	robustUniformBufferAccessSizeAlignment	VK_EXT_robustness2
2 × uint32_t	minFragmentShadingRateAttachmentTexelSize	attachmentFragmentShadingRate
2 × uint32_t	maxFragmentShadingRateAttachmentTexelSize	attachmentFragmentShadingRate
uint32_t	maxFragmentShadingRateAttachmentTexelSizeAspectRatio	attachmentFragmentShadingRate
VkBool32	primitiveFragmentShadingRateWithMultipleViewports	primitiveFragmentShadingRate
VkBool32	layeredShadingRateAttachments	attachmentFragmentShadingRate
VkBool32	fragmentShadingRateNonTrivialCombinerOps	pipelineFragmentShadingRate
2 × uint32_t	maxFragmentSize	pipelineFragmentShadingRate
uint32_t	maxFragmentSizeAspectRatio	pipelineFragmentShadingRate
uint32_t	maxFragmentShadingRateCoverageSamples	pipelineFragmentShadingRate
VkSampleCountFlagBits	maxFragmentShadingRateRasterizationSamples	pipelineFragmentShadingRate
VkBool32	fragmentShadingRateWithShaderDepthStencilWrites	pipelineFragmentShadingRate
VkBool32	fragmentShadingRateWithSampleMask	pipelineFragmentShadingRate
VkBool32	fragmentShadingRateWithShaderSampleMask	pipelineFragmentShadingRate

Type	Limit	Feature
VkBool32	fragmentShadingRateWithConservativeRasterization	pipelineFragmentShadingRate
VkBool32	fragmentShadingRateWithFragmentShaderInterlock	pipelineFragmentShadingRate
VkBool32	fragmentShadingRateWithCustomSampleLocations	pipelineFragmentShadingRate
VkBool32	fragmentShadingRateStrictMultiplyCombiner	pipelineFragmentShadingRate
VkBool32	triStripVertexOrderIndependentOfProvokingVertex	-
VkBool32	dynamicPrimitiveTopologyUnrestricted	VK_EXT_extended_dynamic_state3
uint32_t	maxOpacity2StateSubdivisionLevel	VK_EXT_opacity_micromap
uint32_t	maxOpacity4StateSubdivisionLevel	VK_EXT_opacity_micromap
uint32_t	maxIndirectShaderObjectCount	shaderObject
VkBool32	separateDepthStencilAttachmentAccess	maintenance7
uint32_t	maxDescriptorSetTotalUniformBuffersDynamic	maintenance7
uint32_t	maxDescriptorSetTotalStorageBuffersDynamic	maintenance7
uint32_t	maxDescriptorSetTotalBuffersDynamic	maintenance7
uint32_t	maxDescriptorSetUpdateAfterBindTotalUniformBuffersDynamic	maintenance7
uint32_t	maxDescriptorSetUpdateAfterBindTotalStorageBuffersDynamic	maintenance7
uint32_t	maxDescriptorSetUpdateAfterBindTotalBuffersDynamic	maintenance7

Table 62. Required Limits

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
maxImageDimension1D	-	4096	min
maxImageDimension2D	-	4096	min
maxImageDimension3D	-	256	min
maxImageDimensionCube	-	4096	min
maxImageArrayLayers	-	256	min
maxTexelBufferElements	-	65536	min
maxUniformBufferRange	-	16384	min
maxStorageBufferRange	-	2^{27}	min
maxPushConstantsSize	-	128	min
maxMemoryAllocationCount	-	4096	min
maxSamplerAllocationCount	-	4000	min
bufferImageGranularity	-	131072	max

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
sparseAddressSpaceSize	0	2^{31}	min
maxBoundDescriptorSets	-	4	min
maxPerStageDescriptorSamplers	-	16	min
maxPerStageDescriptorUniformBuffers	-	12	min
maxPerStageDescriptorStorageBuffers	-	4	min
maxPerStageDescriptorSampledImages	-	16	min
maxPerStageDescriptorStorageImages	-	4	min
maxPerStageDescriptorInputAttachments	-	4	min
maxPerStageResources	-	128 [2]	min
maxDescriptorSetSamplers	-	96 [8]	min, $n \times$ PerStage
maxDescriptorSetUniformBuffers	-	72 [8]	min, $n \times$ PerStage
maxDescriptorSetUniformBuffersDynamic	-	8	min
maxDescriptorSetStorageBuffers	-	24 [8]	min, $n \times$ PerStage
maxDescriptorSetStorageBuffersDynamic	-	4	min
maxDescriptorSetTotalUniformBuffersDynamic	-	maxDescriptorSetUniformBuffersDynamic	min
maxDescriptorSetTotalStorageBuffersDynamic	-	maxDescriptorSetStorageBuffersDynamic	min
maxDescriptorSetTotalBuffersDynamic	-	maxDescriptorSetUniformBuffersDynamic + maxDescriptorSetStorageBuffersDynamic	min
maxDescriptorSetSampledImages	-	96 [8]	min, $n \times$ PerStage
maxDescriptorSetStorageImages	-	24 [8]	min, $n \times$ PerStage
maxDescriptorSetInputAttachments	-	4	min
maxVertexInputAttributes	-	16	min
maxVertexInputBindings	-	16 [10]	min
maxVertexInputAttributeOffset	-	2047	min
maxVertexInputBindingStride	-	2048	min

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
maxVertexOutputComponents	-	64	min
maxTessellationGenerationLevel	0	64	min
maxTessellationPatchSize	0	32	min
maxTessellationControlPerVertexInputComponents	0	64	min
maxTessellationControlPerVertexOutputComponents	0	64	min
maxTessellationControlPerPatchOutputComponents	0	120	min
maxTessellationControlTotalOutputComponents	0	2048	min
maxTessellationEvaluationInputComponents	0	64	min
maxTessellationEvaluationOutputComponents	0	64	min
maxGeometryShaderInvocations	0	32	min
maxGeometryInputComponents	0	64	min
maxGeometryOutputComponents	0	64	min
maxGeometryOutputVertices	0	256	min
maxGeometryTotalOutputComponents	0	1024	min
maxFragmentInputComponents	-	64	min
maxFragmentOutputAttachments	-	4	min
maxFragmentDualSrcAttachments	0	1	min
maxFragmentCombinedOutputResources	-	4	min
maxComputeSharedMemorySize	-	16384	min
maxComputeWorkGroupCount	-	(65535,65535,65535)	min
maxComputeWorkGroupInvocations	-	128	min
maxComputeWorkGroupSize	-	(128,128,64)	min
subPixelPrecisionBits	-	4	min
subTexelPrecisionBits	-	4	min
mipmapPrecisionBits	-	4	min
maxDrawIndexedIndexValue	2^{24}-1	2^{32}-1	min
maxDrawIndirectCount	1	2^{16}-1	min
maxSamplerLodBias	-	2	min
maxSamplerAnisotropy	1	16	min
maxViewports	1	16	min
maxViewportDimensions	-	(4096,4096) [3]	min
viewportBoundsRange	-	(-8192,8191) [4]	(max,min)

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
viewportSubPixelBits	-	0	min
minMemoryMapAlignment	-	64	min
minTexelBufferOffsetAlignment	-	256	max
minUniformBufferOffsetAlignment	-	256	max
minStorageBufferOffsetAlignment	-	256	max
minTexelOffset	-	-8	max
maxTexelOffset	-	7	min
minTexelGatherOffset	0	-8	max
maxTexelGatherOffset	0	7	min
minInterpolationOffset	0.0	-0.5 [5]	max
maxInterpolationOffset	0.0	0.5 - (1 ULP) [5]	min
subPixelInterpolationOffsetBits	0	4 [5]	min
maxFramebufferWidth	-	4096	min
maxFramebufferHeight	-	4096	min
maxFramebufferLayers	-	256	min
framebufferColorSampleCounts	-	(VK_SAMPLE_COUNT_1_BIT | VK_SAMPLE_COUNT_4_BIT)	min
framebufferDepthSampleCounts	-	(VK_SAMPLE_COUNT_1_BIT | VK_SAMPLE_COUNT_4_BIT)	min
framebufferStencilSampleCounts	-	(VK_SAMPLE_COUNT_1_BIT | VK_SAMPLE_COUNT_4_BIT)	min
framebufferNoAttachmentsSampleCounts	-	(VK_SAMPLE_COUNT_1_BIT | VK_SAMPLE_COUNT_4_BIT)	min
maxColorAttachments	-	4	min
sampledImageColorSampleCounts	-	(VK_SAMPLE_COUNT_1_BIT | VK_SAMPLE_COUNT_4_BIT)	min
sampledImageIntegerSampleCounts	-	VK_SAMPLE_COUNT_1_BIT	min

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
sampledImageDepthSampleCounts	-	(VK_SAMPLE_COUNT_1_BIT | VK_SAMPLE_COUNT_4_BIT)	min
sampledImageStencilSampleCounts	-	(VK_SAMPLE_COUNT_1_BIT | VK_SAMPLE_COUNT_4_BIT)	min
storageImageSampleCounts	VK_SAMPLE_COUNT_1_BIT	(VK_SAMPLE_COUNT_1_BIT | VK_SAMPLE_COUNT_4_BIT)	min
maxSampleMaskWords	-	1	min
timestampComputeAndGraphics	-	-	implementation-dependent
timestampPeriod	-	-	duration
maxClipDistances	0	8	min
maxCullDistances	0	8	min
maxCombinedClipAndCullDistances	0	8	min
discreteQueuePriorities	-	2	min
pointSizeRange	(1.0,1.0)	(1.0,64.0 - ULP) [6]	(max,min)
lineWidthRange	(1.0,1.0)	(1.0,8.0 - ULP) [7]	(max,min)
pointSizeGranularity	0.0	1.0 [6]	max, fixed point increment
lineWidthGranularity	0.0	1.0 [7]	max, fixed point increment
strictLines	-	-	implementation-dependent
standardSampleLocations	-	-	implementation-dependent
optimalBufferCopyOffsetAlignment	-	-	recommendation
optimalBufferCopyRowPitchAlignment	-	-	recommendation
nonCoherentAtomSize	-	256	max
maxPushDescriptors	-	32	min

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
maxMultiviewViewCount	-	6	min
maxMultiviewInstanceIndex	-	2^{27}-1	min
maxDiscardRectangles	0	4	min
sampleLocationSampleCounts	-	VK_SAMPLE_COUNT_4_BIT	min
maxSampleLocationGridSize	-	(1,1)	min
sampleLocationCoordinateRange	-	(0.0, 0.9375)	(max,min)
sampleLocationSubPixelBits	-	4	min
variableSampleLocations	-	false	implementation-dependent
nativeUnalignedPerformance	-	false	implementation-dependent
minImportedHostPointerAlignment	-	65536	max
advancedBlendMaxColorAttachments	-	1	min
advancedBlendIndependentBlend	-	false	implementation-dependent
advancedBlendNonPremultipliedSrcColor	-	false	implementation-dependent
advancedBlendNonPremultipliedDstColor	-	false	implementation-dependent
advancedBlendCorrelatedOverlap	-	false	implementation-dependent
advancedBlendAllOperations	-	false	implementation-dependent
maxPerSetDescriptors	-	1024	min
maxMemoryAllocationSize	-	2^{30}	min
maxBufferSize	-	2^{30}	min
primitiveOverestimationSize	-	0.0	min
maxExtraPrimitiveOverestimationSize	-	0.0	min
extraPrimitiveOverestimationSizeGranularity	-	0.0	min
primitiveUnderestimation	-	false	implementation-dependent
conservativePointAndLineRasterization	-	false	implementation-dependent
degenerateTrianglesRasterized	-	false	implementation-dependent

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
degenerateLinesRasterized	-	false	implementation-dependent
fullyCoveredFragmentShaderInputVariable	-	false	implementation-dependent
conservativeRasterizationPostDepthCoverage	-	false	implementation-dependent
maxVertexAttribDivisor	-	2^{16}-1	min
maxTransformFeedbackStreams	-	1	min
maxTransformFeedbackBuffers	-	1	min
maxTransformFeedbackBufferSize	-	2^{27}	min
maxTransformFeedbackStreamDataSize	-	512	min
maxTransformFeedbackBufferDataSize	-	512	min
maxTransformFeedbackBufferDataStride	-	512	min
transformFeedbackQueries	-	false	implementation-dependent
transformFeedbackStreamsLinesTriangles	-	false	implementation-dependent
transformFeedbackRasterizationStreamSelect	-	false	implementation-dependent
transformFeedbackDraw	-	false	implementation-dependent
minFragmentDensityTexelSize	-	(1,1)	min
maxFragmentDensityTexelSize	-	(1,1)	min
fragmentDensityInvocations	-	-	implementation-dependent
subsampledLoads	true	false	implementation-dependent
subsampledCoarseReconstructionEarlyAccess	false	false	implementation-dependent
maxSubsampledArrayLayers	2	2	min
maxDescriptorSetSubsampledSamplers	1	1	min
VkPhysicalDeviceRayTracingPipelinePropertiesKHR::shaderGroupHandleSize	-	32	exact
VkPhysicalDeviceRayTracingPipelinePropertiesKHR::maxRayRecursionDepth	-	1	min
maxShaderGroupStride	-	4096	min

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
shaderGroupBaseAlignment	-	64	max
maxGeometryCount	-	2^{24}-1	min
maxInstanceCount	-	2^{24}-1	min
maxPrimitiveCount	-	2^{29}-1	min
maxPerStageDescriptorAccelerationStructures	-	16	min
maxPerStageDescriptorUpdateAfterBindAccelerationStructures	-	500000 [9]	min
maxDescriptorSetAccelerationStructures	-	16	min
maxDescriptorSetUpdateAfterBindAccelerationStructures	-	500000 [9]	min
minAccelerationStructureScratchOffsetAlignment	-	256	max
shaderGroupHandleCaptureReplaySize	-	64	max
maxRayDispatchInvocationCount	-	2^{30}	min
shaderGroupHandleAlignment	-	32	max
maxRayHitAttributeSize	-	32	min
maxTimelineSemaphoreValueDifference	-	2^{31}-1	min
lineSubPixelPrecisionBits	-	4	min
maxIndirectSequenceCount	-	2^{20}	min
maxIndirectCommandsTokenCount	-	16	min
maxIndirectCommandsTokenOffset	-	2047	min
maxIndirectPipelineCount	-	2^{12}	min
deviceGeneratedCommandsTransformFeedback	-	false	implementation-dependent
deviceGeneratedCommandsMultiDrawIndirectCount	-	false	implementation-dependent
maxIndirectShaderObjectCount	0	2^{12}	implementation-dependent
maxIndirectCommandsIndirectStride	-	2048	min
supportedIndirectCommandsInputModes	-	VK_INDIRECT_COMMANDS_INPUT_MODE_VULKAN_INDEX_BUFFER_EXT	min

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
supportedIndirectCommandsShaderStages	-	(VK_SHADER_STAGE_COMPUTE_BIT | VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT)	min
supportedIndirectCommandsShaderStagesPipelineBinding	-	0	min
supportedIndirectCommandsShaderStagesShaderBinding	-	0	min
maxCustomBorderColorSamplers	-	32	min
robustStorageBufferAccessSizeAlignment	-	4	max
robustUniformBufferAccessSizeAlignment	-	256	max
minFragmentShadingRateAttachmentTexelSize	(0,0)	(32,32)	max
maxFragmentShadingRateAttachmentTexelSize	(0,0)	(8,8)	min
maxFragmentShadingRateAttachmentTexelSizeAspectRatio	0	1	min
primitiveFragmentShadingRateWithMultipleViewports	false	false	implementation-dependent
layeredShadingRateAttachments	false	false	implementation-dependent
fragmentShadingRateNonTrivialCombinerOps	-	false	implementation-dependent
maxFragmentSize	-	(2,2)	min
maxFragmentSizeAspectRatio	-	2	min
maxFragmentShadingRateCoverageSamples	-	16	min
maxFragmentShadingRateRasterizationSamples	-	VK_SAMPLE_COUNT_4_BIT	min
fragmentShadingRateWithShaderDepthStencilWrites	-	false	implementation-dependent
fragmentShadingRateWithSampleMask	-	false	implementation-dependent
fragmentShadingRateWithShaderSampleMask	-	false	implementation-dependent
fragmentShadingRateWithConservativeRasterization	-	false	implementation-dependent
fragmentShadingRateWithFragmentShaderInterlock	-	false	implementation-dependent

Limit	Unsupported Limit	Supported Limit	Limit Type[1]
fragmentShadingRateWithCustomSampleLocations	-	false	implementation-dependent
fragmentShadingRateStrictMultiplyCombiner	-	false	implementation-dependent
maxMultiDrawCount	-	1024	min
maxCommandBufferNestingLevel	-	1	min
triStripVertexOrderIndependentOfProvokingVertex	-	false	implementation-dependent
dynamicPrimitiveTopologyUnrestricted	-	-	implementation-dependent
maxOpacity2StateSubdivisionLevel	-	3	min
maxOpacity4StateSubdivisionLevel	-	3	min
minPlacedMemoryMapAlignment	-	65536	max
separateDepthStencilAttachmentAccess	false	-	implementation-dependent

1

The **Limit Type** column specifies the limit is either the minimum limit all implementations **must** support, the maximum limit all implementations **must** support, or the exact value all implementations **must** support. For bitmasks a minimum limit is the least bits all implementations **must** set, but they **may** have additional bits set beyond this minimum.

2

The maxPerStageResources **must** be at least the smallest of the following:

- the sum of the maxPerStageDescriptorUniformBuffers, maxPerStageDescriptorStorageBuffers, maxPerStageDescriptorSampledImages, maxPerStageDescriptorStorageImages, maxPerStageDescriptorInputAttachments, maxColorAttachments limits, or
- 128.

It **may** not be possible to reach this limit in every stage.

3

See maxViewportDimensions for the **required** relationship to other limits.

4

See viewportBoundsRange for the **required** relationship to other limits.

5

The values minInterpolationOffset and maxInterpolationOffset describe the closed interval of supported interpolation offsets: [minInterpolationOffset, maxInterpolationOffset]. The ULP is determined by subPixelInterpolationOffsetBits. If subPixelInterpolationOffsetBits is 4, this

provides increments of $(1/2^4) = 0.0625$, and thus the range of supported interpolation offsets would be [-0.5, 0.4375].

6

The point size ULP is determined by `pointSizeGranularity`. If the `pointSizeGranularity` is 0.125, the range of supported point sizes **must** be at least [1.0, 63.875].

7

The line width ULP is determined by `lineWidthGranularity`. If the `lineWidthGranularity` is 0.0625, the range of supported line widths **must** be at least [1.0, 7.9375].

8

The minimum `maxDescriptorSet*` limit is *n* times the corresponding *specification* minimum `maxPerStageDescriptor*` limit, where *n* is the number of shader stages supported by the VkPhysicalDevice. If all shader stages are supported, $n = 6$ (vertex, tessellation control, tessellation evaluation, geometry, fragment, compute).

10

If the `VK_KHR_portability_subset` extension is enabled, the required minimum value of `maxVertexInputBindings` is 8.

41.2. Additional Multisampling Capabilities

To query additional multisampling capabilities which **may** be supported for a specific sample count, beyond the minimum capabilities described for Limits above, call:

```
// Provided by VK_EXT_sample_locations
void vkGetPhysicalDeviceMultisamplePropertiesEXT(
    VkPhysicalDevice                            physicalDevice,
    VkSampleCountFlagBits                       samples,
    VkMultisamplePropertiesEXT*                 pMultisampleProperties);
```

- `physicalDevice` is the physical device from which to query the additional multisampling capabilities.
- `samples` is a VkSampleCountFlagBits value specifying the sample count to query capabilities for.
- `pMultisampleProperties` is a pointer to a VkMultisamplePropertiesEXT structure in which information about additional multisampling capabilities specific to the sample count is returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceMultisamplePropertiesEXT-physicalDevice-parameter `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceMultisamplePropertiesEXT-samples-parameter `samples` **must** be a valid VkSampleCountFlagBits value

- VUID-vkGetPhysicalDeviceMultisamplePropertiesEXT-pMultisampleProperties-parameter
  `pMultisampleProperties` **must** be a valid pointer to a VkMultisamplePropertiesEXT structure

The `VkMultisamplePropertiesEXT` structure is defined as

```
// Provided by VK_EXT_sample_locations
typedef struct VkMultisamplePropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkExtent2D         maxSampleLocationGridSize;
} VkMultisamplePropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `maxSampleLocationGridSize` is the maximum size of the pixel grid in which sample locations **can** vary.

Valid Usage (Implicit)

- VUID-VkMultisamplePropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_MULTISAMPLE_PROPERTIES_EXT`
- VUID-VkMultisamplePropertiesEXT-pNext-pNext
  `pNext` **must** be `NULL`

If the sample count for which additional multisampling capabilities are requested using `vkGetPhysicalDeviceMultisamplePropertiesEXT` is set in `sampleLocationSampleCounts` the `width` and `height` members of `VkMultisamplePropertiesEXT`::`maxSampleLocationGridSize` **must** be greater than or equal to the corresponding members of `maxSampleLocationGridSize`, respectively, otherwise both members **must** be `0`.

Chapter 42. Formats

Supported buffer and image formats **may** vary across implementations. A minimum set of format features are guaranteed, but others **must** be explicitly queried before use to ensure they are supported by the implementation.

The features for the set of formats (VkFormat) supported by the implementation are queried individually using the vkGetPhysicalDeviceFormatProperties command.

42.1. Format Definition

The following image formats **can** be passed to, and **may** be returned from Vulkan commands. The memory required to store each format is discussed with that format, and also summarized in the Representation and Texel Block Size section and the Compatible formats table.

```
// Provided by VK_VERSION_1_0
typedef enum VkFormat {
    VK_FORMAT_UNDEFINED = 0,
    VK_FORMAT_R4G4_UNORM_PACK8 = 1,
    VK_FORMAT_R4G4B4A4_UNORM_PACK16 = 2,
    VK_FORMAT_B4G4R4A4_UNORM_PACK16 = 3,
    VK_FORMAT_R5G6B5_UNORM_PACK16 = 4,
    VK_FORMAT_B5G6R5_UNORM_PACK16 = 5,
    VK_FORMAT_R5G5B5A1_UNORM_PACK16 = 6,
    VK_FORMAT_B5G5R5A1_UNORM_PACK16 = 7,
    VK_FORMAT_A1R5G5B5_UNORM_PACK16 = 8,
    VK_FORMAT_R8_UNORM = 9,
    VK_FORMAT_R8_SNORM = 10,
    VK_FORMAT_R8_USCALED = 11,
    VK_FORMAT_R8_SSCALED = 12,
    VK_FORMAT_R8_UINT = 13,
    VK_FORMAT_R8_SINT = 14,
    VK_FORMAT_R8_SRGB = 15,
    VK_FORMAT_R8G8_UNORM = 16,
    VK_FORMAT_R8G8_SNORM = 17,
    VK_FORMAT_R8G8_USCALED = 18,
    VK_FORMAT_R8G8_SSCALED = 19,
    VK_FORMAT_R8G8_UINT = 20,
    VK_FORMAT_R8G8_SINT = 21,
    VK_FORMAT_R8G8_SRGB = 22,
    VK_FORMAT_R8G8B8_UNORM = 23,
    VK_FORMAT_R8G8B8_SNORM = 24,
    VK_FORMAT_R8G8B8_USCALED = 25,
    VK_FORMAT_R8G8B8_SSCALED = 26,
    VK_FORMAT_R8G8B8_UINT = 27,
    VK_FORMAT_R8G8B8_SINT = 28,
    VK_FORMAT_R8G8B8_SRGB = 29,
    VK_FORMAT_B8G8R8_UNORM = 30,
    VK_FORMAT_B8G8R8_SNORM = 31,
```

```
    VK_FORMAT_B8G8R8_USCALED = 32,
    VK_FORMAT_B8G8R8_SSCALED = 33,
    VK_FORMAT_B8G8R8_UINT = 34,
    VK_FORMAT_B8G8R8_SINT = 35,
    VK_FORMAT_B8G8R8_SRGB = 36,
    VK_FORMAT_R8G8B8A8_UNORM = 37,
    VK_FORMAT_R8G8B8A8_SNORM = 38,
    VK_FORMAT_R8G8B8A8_USCALED = 39,
    VK_FORMAT_R8G8B8A8_SSCALED = 40,
    VK_FORMAT_R8G8B8A8_UINT = 41,
    VK_FORMAT_R8G8B8A8_SINT = 42,
    VK_FORMAT_R8G8B8A8_SRGB = 43,
    VK_FORMAT_B8G8R8A8_UNORM = 44,
    VK_FORMAT_B8G8R8A8_SNORM = 45,
    VK_FORMAT_B8G8R8A8_USCALED = 46,
    VK_FORMAT_B8G8R8A8_SSCALED = 47,
    VK_FORMAT_B8G8R8A8_UINT = 48,
    VK_FORMAT_B8G8R8A8_SINT = 49,
    VK_FORMAT_B8G8R8A8_SRGB = 50,
    VK_FORMAT_A8B8G8R8_UNORM_PACK32 = 51,
    VK_FORMAT_A8B8G8R8_SNORM_PACK32 = 52,
    VK_FORMAT_A8B8G8R8_USCALED_PACK32 = 53,
    VK_FORMAT_A8B8G8R8_SSCALED_PACK32 = 54,
    VK_FORMAT_A8B8G8R8_UINT_PACK32 = 55,
    VK_FORMAT_A8B8G8R8_SINT_PACK32 = 56,
    VK_FORMAT_A8B8G8R8_SRGB_PACK32 = 57,
    VK_FORMAT_A2R10G10B10_UNORM_PACK32 = 58,
    VK_FORMAT_A2R10G10B10_SNORM_PACK32 = 59,
    VK_FORMAT_A2R10G10B10_USCALED_PACK32 = 60,
    VK_FORMAT_A2R10G10B10_SSCALED_PACK32 = 61,
    VK_FORMAT_A2R10G10B10_UINT_PACK32 = 62,
    VK_FORMAT_A2R10G10B10_SINT_PACK32 = 63,
    VK_FORMAT_A2B10G10R10_UNORM_PACK32 = 64,
    VK_FORMAT_A2B10G10R10_SNORM_PACK32 = 65,
    VK_FORMAT_A2B10G10R10_USCALED_PACK32 = 66,
    VK_FORMAT_A2B10G10R10_SSCALED_PACK32 = 67,
    VK_FORMAT_A2B10G10R10_UINT_PACK32 = 68,
    VK_FORMAT_A2B10G10R10_SINT_PACK32 = 69,
    VK_FORMAT_R16_UNORM = 70,
    VK_FORMAT_R16_SNORM = 71,
    VK_FORMAT_R16_USCALED = 72,
    VK_FORMAT_R16_SSCALED = 73,
    VK_FORMAT_R16_UINT = 74,
    VK_FORMAT_R16_SINT = 75,
    VK_FORMAT_R16_SFLOAT = 76,
    VK_FORMAT_R16G16_UNORM = 77,
    VK_FORMAT_R16G16_SNORM = 78,
    VK_FORMAT_R16G16_USCALED = 79,
    VK_FORMAT_R16G16_SSCALED = 80,
    VK_FORMAT_R16G16_UINT = 81,
    VK_FORMAT_R16G16_SINT = 82,
```

```
    VK_FORMAT_R16G16_SFLOAT = 83,
    VK_FORMAT_R16G16B16_UNORM = 84,
    VK_FORMAT_R16G16B16_SNORM = 85,
    VK_FORMAT_R16G16B16_USCALED = 86,
    VK_FORMAT_R16G16B16_SSCALED = 87,
    VK_FORMAT_R16G16B16_UINT = 88,
    VK_FORMAT_R16G16B16_SINT = 89,
    VK_FORMAT_R16G16B16_SFLOAT = 90,
    VK_FORMAT_R16G16B16A16_UNORM = 91,
    VK_FORMAT_R16G16B16A16_SNORM = 92,
    VK_FORMAT_R16G16B16A16_USCALED = 93,
    VK_FORMAT_R16G16B16A16_SSCALED = 94,
    VK_FORMAT_R16G16B16A16_UINT = 95,
    VK_FORMAT_R16G16B16A16_SINT = 96,
    VK_FORMAT_R16G16B16A16_SFLOAT = 97,
    VK_FORMAT_R32_UINT = 98,
    VK_FORMAT_R32_SINT = 99,
    VK_FORMAT_R32_SFLOAT = 100,
    VK_FORMAT_R32G32_UINT = 101,
    VK_FORMAT_R32G32_SINT = 102,
    VK_FORMAT_R32G32_SFLOAT = 103,
    VK_FORMAT_R32G32B32_UINT = 104,
    VK_FORMAT_R32G32B32_SINT = 105,
    VK_FORMAT_R32G32B32_SFLOAT = 106,
    VK_FORMAT_R32G32B32A32_UINT = 107,
    VK_FORMAT_R32G32B32A32_SINT = 108,
    VK_FORMAT_R32G32B32A32_SFLOAT = 109,
    VK_FORMAT_R64_UINT = 110,
    VK_FORMAT_R64_SINT = 111,
    VK_FORMAT_R64_SFLOAT = 112,
    VK_FORMAT_R64G64_UINT = 113,
    VK_FORMAT_R64G64_SINT = 114,
    VK_FORMAT_R64G64_SFLOAT = 115,
    VK_FORMAT_R64G64B64_UINT = 116,
    VK_FORMAT_R64G64B64_SINT = 117,
    VK_FORMAT_R64G64B64_SFLOAT = 118,
    VK_FORMAT_R64G64B64A64_UINT = 119,
    VK_FORMAT_R64G64B64A64_SINT = 120,
    VK_FORMAT_R64G64B64A64_SFLOAT = 121,
    VK_FORMAT_B10G11R11_UFLOAT_PACK32 = 122,
    VK_FORMAT_E5B9G9R9_UFLOAT_PACK32 = 123,
    VK_FORMAT_D16_UNORM = 124,
    VK_FORMAT_X8_D24_UNORM_PACK32 = 125,
    VK_FORMAT_D32_SFLOAT = 126,
    VK_FORMAT_S8_UINT = 127,
    VK_FORMAT_D16_UNORM_S8_UINT = 128,
    VK_FORMAT_D24_UNORM_S8_UINT = 129,
    VK_FORMAT_D32_SFLOAT_S8_UINT = 130,
    VK_FORMAT_BC1_RGB_UNORM_BLOCK = 131,
    VK_FORMAT_BC1_RGB_SRGB_BLOCK = 132,
    VK_FORMAT_BC1_RGBA_UNORM_BLOCK = 133,
```

```
    VK_FORMAT_BC1_RGBA_SRGB_BLOCK = 134,
    VK_FORMAT_BC2_UNORM_BLOCK = 135,
    VK_FORMAT_BC2_SRGB_BLOCK = 136,
    VK_FORMAT_BC3_UNORM_BLOCK = 137,
    VK_FORMAT_BC3_SRGB_BLOCK = 138,
    VK_FORMAT_BC4_UNORM_BLOCK = 139,
    VK_FORMAT_BC4_SNORM_BLOCK = 140,
    VK_FORMAT_BC5_UNORM_BLOCK = 141,
    VK_FORMAT_BC5_SNORM_BLOCK = 142,
    VK_FORMAT_BC6H_UFLOAT_BLOCK = 143,
    VK_FORMAT_BC6H_SFLOAT_BLOCK = 144,
    VK_FORMAT_BC7_UNORM_BLOCK = 145,
    VK_FORMAT_BC7_SRGB_BLOCK = 146,
    VK_FORMAT_ETC2_R8G8B8_UNORM_BLOCK = 147,
    VK_FORMAT_ETC2_R8G8B8_SRGB_BLOCK = 148,
    VK_FORMAT_ETC2_R8G8B8A1_UNORM_BLOCK = 149,
    VK_FORMAT_ETC2_R8G8B8A1_SRGB_BLOCK = 150,
    VK_FORMAT_ETC2_R8G8B8A8_UNORM_BLOCK = 151,
    VK_FORMAT_ETC2_R8G8B8A8_SRGB_BLOCK = 152,
    VK_FORMAT_EAC_R11_UNORM_BLOCK = 153,
    VK_FORMAT_EAC_R11_SNORM_BLOCK = 154,
    VK_FORMAT_EAC_R11G11_UNORM_BLOCK = 155,
    VK_FORMAT_EAC_R11G11_SNORM_BLOCK = 156,
    VK_FORMAT_ASTC_4x4_UNORM_BLOCK = 157,
    VK_FORMAT_ASTC_4x4_SRGB_BLOCK = 158,
    VK_FORMAT_ASTC_5x4_UNORM_BLOCK = 159,
    VK_FORMAT_ASTC_5x4_SRGB_BLOCK = 160,
    VK_FORMAT_ASTC_5x5_UNORM_BLOCK = 161,
    VK_FORMAT_ASTC_5x5_SRGB_BLOCK = 162,
    VK_FORMAT_ASTC_6x5_UNORM_BLOCK = 163,
    VK_FORMAT_ASTC_6x5_SRGB_BLOCK = 164,
    VK_FORMAT_ASTC_6x6_UNORM_BLOCK = 165,
    VK_FORMAT_ASTC_6x6_SRGB_BLOCK = 166,
    VK_FORMAT_ASTC_8x5_UNORM_BLOCK = 167,
    VK_FORMAT_ASTC_8x5_SRGB_BLOCK = 168,
    VK_FORMAT_ASTC_8x6_UNORM_BLOCK = 169,
    VK_FORMAT_ASTC_8x6_SRGB_BLOCK = 170,
    VK_FORMAT_ASTC_8x8_UNORM_BLOCK = 171,
    VK_FORMAT_ASTC_8x8_SRGB_BLOCK = 172,
    VK_FORMAT_ASTC_10x5_UNORM_BLOCK = 173,
    VK_FORMAT_ASTC_10x5_SRGB_BLOCK = 174,
    VK_FORMAT_ASTC_10x6_UNORM_BLOCK = 175,
    VK_FORMAT_ASTC_10x6_SRGB_BLOCK = 176,
    VK_FORMAT_ASTC_10x8_UNORM_BLOCK = 177,
    VK_FORMAT_ASTC_10x8_SRGB_BLOCK = 178,
    VK_FORMAT_ASTC_10x10_UNORM_BLOCK = 179,
    VK_FORMAT_ASTC_10x10_SRGB_BLOCK = 180,
    VK_FORMAT_ASTC_12x10_UNORM_BLOCK = 181,
    VK_FORMAT_ASTC_12x10_SRGB_BLOCK = 182,
    VK_FORMAT_ASTC_12x12_UNORM_BLOCK = 183,
    VK_FORMAT_ASTC_12x12_SRGB_BLOCK = 184,
```

```
    VK_FORMAT_G8B8G8R8_422_UNORM = 1000156000,
    VK_FORMAT_B8G8R8G8_422_UNORM = 1000156001,
    VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM = 1000156002,
    VK_FORMAT_G8_B8R8_2PLANE_420_UNORM = 1000156003,
    VK_FORMAT_G8_B8_R8_3PLANE_422_UNORM = 1000156004,
    VK_FORMAT_G8_B8R8_2PLANE_422_UNORM = 1000156005,
    VK_FORMAT_G8_B8_R8_3PLANE_444_UNORM = 1000156006,
    VK_FORMAT_R10X6_UNORM_PACK16 = 1000156007,
    VK_FORMAT_R10X6G10X6_UNORM_2PACK16 = 1000156008,
    VK_FORMAT_R10X6G10X6B10X6A10X6_UNORM_4PACK16 = 1000156009,
    VK_FORMAT_G10X6B10X6G10X6R10X6_422_UNORM_4PACK16 = 1000156010,
    VK_FORMAT_B10X6G10X6R10X6G10X6_422_UNORM_4PACK16 = 1000156011,
    VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16 = 1000156012,
    VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16 = 1000156013,
    VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16 = 1000156014,
    VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16 = 1000156015,
    VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16 = 1000156016,
    VK_FORMAT_R12X4_UNORM_PACK16 = 1000156017,
    VK_FORMAT_R12X4G12X4_UNORM_2PACK16 = 1000156018,
    VK_FORMAT_R12X4G12X4B12X4A12X4_UNORM_4PACK16 = 1000156019,
    VK_FORMAT_G12X4B12X4G12X4R12X4_422_UNORM_4PACK16 = 1000156020,
    VK_FORMAT_B12X4G12X4R12X4G12X4_422_UNORM_4PACK16 = 1000156021,
    VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16 = 1000156022,
    VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16 = 1000156023,
    VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16 = 1000156024,
    VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16 = 1000156025,
    VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16 = 1000156026,
    VK_FORMAT_G16B16G16R16_422_UNORM = 1000156027,
    VK_FORMAT_B16G16R16G16_422_UNORM = 1000156028,
    VK_FORMAT_G16_B16_R16_3PLANE_420_UNORM = 1000156029,
    VK_FORMAT_G16_B16R16_2PLANE_420_UNORM = 1000156030,
    VK_FORMAT_G16_B16_R16_3PLANE_422_UNORM = 1000156031,
    VK_FORMAT_G16_B16R16_2PLANE_422_UNORM = 1000156032,
    VK_FORMAT_G16_B16_R16_3PLANE_444_UNORM = 1000156033,
  // Provided by VK_KHR_maintenance5
    VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR = 1000470000,
  // Provided by VK_KHR_maintenance5
    VK_FORMAT_A8_UNORM_KHR = 1000470001,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G8B8G8R8_422_UNORM_KHR = VK_FORMAT_G8B8G8R8_422_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_B8G8R8G8_422_UNORM_KHR = VK_FORMAT_B8G8R8G8_422_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM_KHR = VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G8_B8R8_2PLANE_420_UNORM_KHR = VK_FORMAT_G8_B8R8_2PLANE_420_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G8_B8_R8_3PLANE_422_UNORM_KHR = VK_FORMAT_G8_B8_R8_3PLANE_422_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G8_B8R8_2PLANE_422_UNORM_KHR = VK_FORMAT_G8_B8R8_2PLANE_422_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
```

```
    VK_FORMAT_G8_B8_R8_3PLANE_444_UNORM_KHR = VK_FORMAT_G8_B8_R8_3PLANE_444_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_R10X6_UNORM_PACK16_KHR = VK_FORMAT_R10X6_UNORM_PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_R10X6G10X6_UNORM_2PACK16_KHR = VK_FORMAT_R10X6G10X6_UNORM_2PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_R10X6G10X6B10X6A10X6_UNORM_4PACK16_KHR =
VK_FORMAT_R10X6G10X6B10X6A10X6_UNORM_4PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G10X6B10X6G10X6R10X6_422_UNORM_4PACK16_KHR =
VK_FORMAT_G10X6B10X6G10X6R10X6_422_UNORM_4PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_B10X6G10X6R10X6G10X6_422_UNORM_4PACK16_KHR =
VK_FORMAT_B10X6G10X6R10X6G10X6_422_UNORM_4PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16_KHR =
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16_KHR =
VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16_KHR =
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16_KHR =
VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16_KHR =
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_R12X4_UNORM_PACK16_KHR = VK_FORMAT_R12X4_UNORM_PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_R12X4G12X4_UNORM_2PACK16_KHR = VK_FORMAT_R12X4G12X4_UNORM_2PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_R12X4G12X4B12X4A12X4_UNORM_4PACK16_KHR =
VK_FORMAT_R12X4G12X4B12X4A12X4_UNORM_4PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G12X4B12X4G12X4R12X4_422_UNORM_4PACK16_KHR =
VK_FORMAT_G12X4B12X4G12X4R12X4_422_UNORM_4PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_B12X4G12X4R12X4G12X4_422_UNORM_4PACK16_KHR =
VK_FORMAT_B12X4G12X4R12X4G12X4_422_UNORM_4PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16_KHR =
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16_KHR =
VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16_KHR =
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16,
```

```
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16_KHR =
VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16_KHR =
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G16B16G16R16_422_UNORM_KHR = VK_FORMAT_G16B16G16R16_422_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_B16G16R16G16_422_UNORM_KHR = VK_FORMAT_B16G16R16G16_422_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G16_B16_R16_3PLANE_420_UNORM_KHR =
VK_FORMAT_G16_B16_R16_3PLANE_420_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G16_B16R16_2PLANE_420_UNORM_KHR = VK_FORMAT_G16_B16R16_2PLANE_420_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G16_B16_R16_3PLANE_422_UNORM_KHR =
VK_FORMAT_G16_B16_R16_3PLANE_422_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G16_B16R16_2PLANE_422_UNORM_KHR = VK_FORMAT_G16_B16R16_2PLANE_422_UNORM,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_G16_B16_R16_3PLANE_444_UNORM_KHR =
VK_FORMAT_G16_B16_R16_3PLANE_444_UNORM,
} VkFormat;
```

- VK_FORMAT_UNDEFINED specifies that the format is not specified.
- VK_FORMAT_R4G4_UNORM_PACK8 specifies a two-component, 8-bit packed unsigned normalized format that has a 4-bit R component in bits 4..7, and a 4-bit G component in bits 0..3.
- VK_FORMAT_R4G4B4A4_UNORM_PACK16 specifies a four-component, 16-bit packed unsigned normalized format that has a 4-bit R component in bits 12..15, a 4-bit G component in bits 8..11, a 4-bit B component in bits 4..7, and a 4-bit A component in bits 0..3.
- VK_FORMAT_B4G4R4A4_UNORM_PACK16 specifies a four-component, 16-bit packed unsigned normalized format that has a 4-bit B component in bits 12..15, a 4-bit G component in bits 8..11, a 4-bit R component in bits 4..7, and a 4-bit A component in bits 0..3.
- VK_FORMAT_R5G6B5_UNORM_PACK16 specifies a three-component, 16-bit packed unsigned normalized format that has a 5-bit R component in bits 11..15, a 6-bit G component in bits 5..10, and a 5-bit B component in bits 0..4.
- VK_FORMAT_B5G6R5_UNORM_PACK16 specifies a three-component, 16-bit packed unsigned normalized format that has a 5-bit B component in bits 11..15, a 6-bit G component in bits 5..10, and a 5-bit R component in bits 0..4.
- VK_FORMAT_R5G5B5A1_UNORM_PACK16 specifies a four-component, 16-bit packed unsigned normalized format that has a 5-bit R component in bits 11..15, a 5-bit G component in bits 6..10, a 5-bit B component in bits 1..5, and a 1-bit A component in bit 0.
- VK_FORMAT_B5G5R5A1_UNORM_PACK16 specifies a four-component, 16-bit packed unsigned normalized format that has a 5-bit B component in bits 11..15, a 5-bit G component in bits 6..10, a 5-bit R component in bits 1..5, and a 1-bit A component in bit 0.

- VK_FORMAT_A1R5G5B5_UNORM_PACK16 specifies a four-component, 16-bit packed unsigned normalized format that has a 1-bit A component in bit 15, a 5-bit R component in bits 10..14, a 5-bit G component in bits 5..9, and a 5-bit B component in bits 0..4.
- VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR specifies a four-component, 16-bit packed unsigned normalized format that has a 1-bit A component in bit 15, a 5-bit B component in bits 10..14, a 5-bit G component in bits 5..9, and a 5-bit R component in bits 0..4.
- VK_FORMAT_A8_UNORM_KHR specifies a one-component, 8-bit unsigned normalized format that has a single 8-bit A component.
- VK_FORMAT_R8_UNORM specifies a one-component, 8-bit unsigned normalized format that has a single 8-bit R component.
- VK_FORMAT_R8_SNORM specifies a one-component, 8-bit signed normalized format that has a single 8-bit R component.
- VK_FORMAT_R8_USCALED specifies a one-component, 8-bit unsigned scaled integer format that has a single 8-bit R component.
- VK_FORMAT_R8_SSCALED specifies a one-component, 8-bit signed scaled integer format that has a single 8-bit R component.
- VK_FORMAT_R8_UINT specifies a one-component, 8-bit unsigned integer format that has a single 8-bit R component.
- VK_FORMAT_R8_SINT specifies a one-component, 8-bit signed integer format that has a single 8-bit R component.
- VK_FORMAT_R8_SRGB specifies a one-component, 8-bit unsigned normalized format that has a single 8-bit R component stored with sRGB nonlinear encoding.
- VK_FORMAT_R8G8_UNORM specifies a two-component, 16-bit unsigned normalized format that has an 8-bit R component in byte 0, and an 8-bit G component in byte 1.
- VK_FORMAT_R8G8_SNORM specifies a two-component, 16-bit signed normalized format that has an 8-bit R component in byte 0, and an 8-bit G component in byte 1.
- VK_FORMAT_R8G8_USCALED specifies a two-component, 16-bit unsigned scaled integer format that has an 8-bit R component in byte 0, and an 8-bit G component in byte 1.
- VK_FORMAT_R8G8_SSCALED specifies a two-component, 16-bit signed scaled integer format that has an 8-bit R component in byte 0, and an 8-bit G component in byte 1.
- VK_FORMAT_R8G8_UINT specifies a two-component, 16-bit unsigned integer format that has an 8-bit R component in byte 0, and an 8-bit G component in byte 1.
- VK_FORMAT_R8G8_SINT specifies a two-component, 16-bit signed integer format that has an 8-bit R component in byte 0, and an 8-bit G component in byte 1.
- VK_FORMAT_R8G8_SRGB specifies a two-component, 16-bit unsigned normalized format that has an 8-bit R component stored with sRGB nonlinear encoding in byte 0, and an 8-bit G component stored with sRGB nonlinear encoding in byte 1.
- VK_FORMAT_R8G8B8_UNORM specifies a three-component, 24-bit unsigned normalized format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, and an 8-bit B component in byte 2.
- VK_FORMAT_R8G8B8_SNORM specifies a three-component, 24-bit signed normalized format that has

an 8-bit R component in byte 0, an 8-bit G component in byte 1, and an 8-bit B component in byte 2.

- VK_FORMAT_R8G8B8_USCALED specifies a three-component, 24-bit unsigned scaled format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, and an 8-bit B component in byte 2.
- VK_FORMAT_R8G8B8_SSCALED specifies a three-component, 24-bit signed scaled format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, and an 8-bit B component in byte 2.
- VK_FORMAT_R8G8B8_UINT specifies a three-component, 24-bit unsigned integer format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, and an 8-bit B component in byte 2.
- VK_FORMAT_R8G8B8_SINT specifies a three-component, 24-bit signed integer format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, and an 8-bit B component in byte 2.
- VK_FORMAT_R8G8B8_SRGB specifies a three-component, 24-bit unsigned normalized format that has an 8-bit R component stored with sRGB nonlinear encoding in byte 0, an 8-bit G component stored with sRGB nonlinear encoding in byte 1, and an 8-bit B component stored with sRGB nonlinear encoding in byte 2.
- VK_FORMAT_B8G8R8_UNORM specifies a three-component, 24-bit unsigned normalized format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, and an 8-bit R component in byte 2.
- VK_FORMAT_B8G8R8_SNORM specifies a three-component, 24-bit signed normalized format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, and an 8-bit R component in byte 2.
- VK_FORMAT_B8G8R8_USCALED specifies a three-component, 24-bit unsigned scaled format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, and an 8-bit R component in byte 2.
- VK_FORMAT_B8G8R8_SSCALED specifies a three-component, 24-bit signed scaled format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, and an 8-bit R component in byte 2.
- VK_FORMAT_B8G8R8_UINT specifies a three-component, 24-bit unsigned integer format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, and an 8-bit R component in byte 2.
- VK_FORMAT_B8G8R8_SINT specifies a three-component, 24-bit signed integer format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, and an 8-bit R component in byte 2.
- VK_FORMAT_B8G8R8_SRGB specifies a three-component, 24-bit unsigned normalized format that has an 8-bit B component stored with sRGB nonlinear encoding in byte 0, an 8-bit G component stored with sRGB nonlinear encoding in byte 1, and an 8-bit R component stored with sRGB nonlinear encoding in byte 2.
- VK_FORMAT_R8G8B8A8_UNORM specifies a four-component, 32-bit unsigned normalized format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, an 8-bit B component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_R8G8B8A8_SNORM specifies a four-component, 32-bit signed normalized format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, an 8-bit B component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_R8G8B8A8_USCALED specifies a four-component, 32-bit unsigned scaled format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, an 8-bit B component in byte 2, and an 8-bit A component in byte 3.

- VK_FORMAT_R8G8B8A8_SSCALED specifies a four-component, 32-bit signed scaled format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, an 8-bit B component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_R8G8B8A8_UINT specifies a four-component, 32-bit unsigned integer format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, an 8-bit B component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_R8G8B8A8_SINT specifies a four-component, 32-bit signed integer format that has an 8-bit R component in byte 0, an 8-bit G component in byte 1, an 8-bit B component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_R8G8B8A8_SRGB specifies a four-component, 32-bit unsigned normalized format that has an 8-bit R component stored with sRGB nonlinear encoding in byte 0, an 8-bit G component stored with sRGB nonlinear encoding in byte 1, an 8-bit B component stored with sRGB nonlinear encoding in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_B8G8R8A8_UNORM specifies a four-component, 32-bit unsigned normalized format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, an 8-bit R component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_B8G8R8A8_SNORM specifies a four-component, 32-bit signed normalized format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, an 8-bit R component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_B8G8R8A8_USCALED specifies a four-component, 32-bit unsigned scaled format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, an 8-bit R component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_B8G8R8A8_SSCALED specifies a four-component, 32-bit signed scaled format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, an 8-bit R component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_B8G8R8A8_UINT specifies a four-component, 32-bit unsigned integer format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, an 8-bit R component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_B8G8R8A8_SINT specifies a four-component, 32-bit signed integer format that has an 8-bit B component in byte 0, an 8-bit G component in byte 1, an 8-bit R component in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_B8G8R8A8_SRGB specifies a four-component, 32-bit unsigned normalized format that has an 8-bit B component stored with sRGB nonlinear encoding in byte 0, an 8-bit G component stored with sRGB nonlinear encoding in byte 1, an 8-bit R component stored with sRGB nonlinear encoding in byte 2, and an 8-bit A component in byte 3.
- VK_FORMAT_A8B8G8R8_UNORM_PACK32 specifies a four-component, 32-bit packed unsigned normalized format that has an 8-bit A component in bits 24..31, an 8-bit B component in bits 16..23, an 8-bit G component in bits 8..15, and an 8-bit R component in bits 0..7.
- VK_FORMAT_A8B8G8R8_SNORM_PACK32 specifies a four-component, 32-bit packed signed normalized format that has an 8-bit A component in bits 24..31, an 8-bit B component in bits 16..23, an 8-bit G component in bits 8..15, and an 8-bit R component in bits 0..7.
- VK_FORMAT_A8B8G8R8_USCALED_PACK32 specifies a four-component, 32-bit packed unsigned scaled integer format that has an 8-bit A component in bits 24..31, an 8-bit B component in bits 16..23,

an 8-bit G component in bits 8..15, and an 8-bit R component in bits 0..7.

- VK_FORMAT_A8B8G8R8_SSCALED_PACK32 specifies a four-component, 32-bit packed signed scaled integer format that has an 8-bit A component in bits 24..31, an 8-bit B component in bits 16..23, an 8-bit G component in bits 8..15, and an 8-bit R component in bits 0..7.
- VK_FORMAT_A8B8G8R8_UINT_PACK32 specifies a four-component, 32-bit packed unsigned integer format that has an 8-bit A component in bits 24..31, an 8-bit B component in bits 16..23, an 8-bit G component in bits 8..15, and an 8-bit R component in bits 0..7.
- VK_FORMAT_A8B8G8R8_SINT_PACK32 specifies a four-component, 32-bit packed signed integer format that has an 8-bit A component in bits 24..31, an 8-bit B component in bits 16..23, an 8-bit G component in bits 8..15, and an 8-bit R component in bits 0..7.
- VK_FORMAT_A8B8G8R8_SRGB_PACK32 specifies a four-component, 32-bit packed unsigned normalized format that has an 8-bit A component in bits 24..31, an 8-bit B component stored with sRGB nonlinear encoding in bits 16..23, an 8-bit G component stored with sRGB nonlinear encoding in bits 8..15, and an 8-bit R component stored with sRGB nonlinear encoding in bits 0..7.
- VK_FORMAT_A2R10G10B10_UNORM_PACK32 specifies a four-component, 32-bit packed unsigned normalized format that has a 2-bit A component in bits 30..31, a 10-bit R component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit B component in bits 0..9.
- VK_FORMAT_A2R10G10B10_SNORM_PACK32 specifies a four-component, 32-bit packed signed normalized format that has a 2-bit A component in bits 30..31, a 10-bit R component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit B component in bits 0..9.
- VK_FORMAT_A2R10G10B10_USCALED_PACK32 specifies a four-component, 32-bit packed unsigned scaled integer format that has a 2-bit A component in bits 30..31, a 10-bit R component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit B component in bits 0..9.
- VK_FORMAT_A2R10G10B10_SSCALED_PACK32 specifies a four-component, 32-bit packed signed scaled integer format that has a 2-bit A component in bits 30..31, a 10-bit R component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit B component in bits 0..9.
- VK_FORMAT_A2R10G10B10_UINT_PACK32 specifies a four-component, 32-bit packed unsigned integer format that has a 2-bit A component in bits 30..31, a 10-bit R component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit B component in bits 0..9.
- VK_FORMAT_A2R10G10B10_SINT_PACK32 specifies a four-component, 32-bit packed signed integer format that has a 2-bit A component in bits 30..31, a 10-bit R component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit B component in bits 0..9.
- VK_FORMAT_A2B10G10R10_UNORM_PACK32 specifies a four-component, 32-bit packed unsigned normalized format that has a 2-bit A component in bits 30..31, a 10-bit B component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit R component in bits 0..9.
- VK_FORMAT_A2B10G10R10_SNORM_PACK32 specifies a four-component, 32-bit packed signed normalized format that has a 2-bit A component in bits 30..31, a 10-bit B component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit R component in bits 0..9.
- VK_FORMAT_A2B10G10R10_USCALED_PACK32 specifies a four-component, 32-bit packed unsigned scaled integer format that has a 2-bit A component in bits 30..31, a 10-bit B component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit R component in bits 0..9.
- VK_FORMAT_A2B10G10R10_SSCALED_PACK32 specifies a four-component, 32-bit packed signed scaled

integer format that has a 2-bit A component in bits 30..31, a 10-bit B component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit R component in bits 0..9.

- VK_FORMAT_A2B10G10R10_UINT_PACK32 specifies a four-component, 32-bit packed unsigned integer format that has a 2-bit A component in bits 30..31, a 10-bit B component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit R component in bits 0..9.
- VK_FORMAT_A2B10G10R10_SINT_PACK32 specifies a four-component, 32-bit packed signed integer format that has a 2-bit A component in bits 30..31, a 10-bit B component in bits 20..29, a 10-bit G component in bits 10..19, and a 10-bit R component in bits 0..9.
- VK_FORMAT_R16_UNORM specifies a one-component, 16-bit unsigned normalized format that has a single 16-bit R component.
- VK_FORMAT_R16_SNORM specifies a one-component, 16-bit signed normalized format that has a single 16-bit R component.
- VK_FORMAT_R16_USCALED specifies a one-component, 16-bit unsigned scaled integer format that has a single 16-bit R component.
- VK_FORMAT_R16_SSCALED specifies a one-component, 16-bit signed scaled integer format that has a single 16-bit R component.
- VK_FORMAT_R16_UINT specifies a one-component, 16-bit unsigned integer format that has a single 16-bit R component.
- VK_FORMAT_R16_SINT specifies a one-component, 16-bit signed integer format that has a single 16-bit R component.
- VK_FORMAT_R16_SFLOAT specifies a one-component, 16-bit signed floating-point format that has a single 16-bit R component.
- VK_FORMAT_R16G16_UNORM specifies a two-component, 32-bit unsigned normalized format that has a 16-bit R component in bytes 0..1, and a 16-bit G component in bytes 2..3.
- VK_FORMAT_R16G16_SNORM specifies a two-component, 32-bit signed normalized format that has a 16-bit R component in bytes 0..1, and a 16-bit G component in bytes 2..3.
- VK_FORMAT_R16G16_USCALED specifies a two-component, 32-bit unsigned scaled integer format that has a 16-bit R component in bytes 0..1, and a 16-bit G component in bytes 2..3.
- VK_FORMAT_R16G16_SSCALED specifies a two-component, 32-bit signed scaled integer format that has a 16-bit R component in bytes 0..1, and a 16-bit G component in bytes 2..3.
- VK_FORMAT_R16G16_UINT specifies a two-component, 32-bit unsigned integer format that has a 16-bit R component in bytes 0..1, and a 16-bit G component in bytes 2..3.
- VK_FORMAT_R16G16_SINT specifies a two-component, 32-bit signed integer format that has a 16-bit R component in bytes 0..1, and a 16-bit G component in bytes 2..3.
- VK_FORMAT_R16G16_SFLOAT specifies a two-component, 32-bit signed floating-point format that has a 16-bit R component in bytes 0..1, and a 16-bit G component in bytes 2..3.
- VK_FORMAT_R16G16B16_UNORM specifies a three-component, 48-bit unsigned normalized format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, and a 16-bit B component in bytes 4..5.
- VK_FORMAT_R16G16B16_SNORM specifies a three-component, 48-bit signed normalized format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, and a 16-bit B

component in bytes 4..5.

- VK_FORMAT_R16G16B16_USCALED specifies a three-component, 48-bit unsigned scaled integer format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, and a 16-bit B component in bytes 4..5.
- VK_FORMAT_R16G16B16_SSCALED specifies a three-component, 48-bit signed scaled integer format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, and a 16-bit B component in bytes 4..5.
- VK_FORMAT_R16G16B16_UINT specifies a three-component, 48-bit unsigned integer format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, and a 16-bit B component in bytes 4..5.
- VK_FORMAT_R16G16B16_SINT specifies a three-component, 48-bit signed integer format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, and a 16-bit B component in bytes 4..5.
- VK_FORMAT_R16G16B16_SFLOAT specifies a three-component, 48-bit signed floating-point format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, and a 16-bit B component in bytes 4..5.
- VK_FORMAT_R16G16B16A16_UNORM specifies a four-component, 64-bit unsigned normalized format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, a 16-bit B component in bytes 4..5, and a 16-bit A component in bytes 6..7.
- VK_FORMAT_R16G16B16A16_SNORM specifies a four-component, 64-bit signed normalized format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, a 16-bit B component in bytes 4..5, and a 16-bit A component in bytes 6..7.
- VK_FORMAT_R16G16B16A16_USCALED specifies a four-component, 64-bit unsigned scaled integer format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, a 16-bit B component in bytes 4..5, and a 16-bit A component in bytes 6..7.
- VK_FORMAT_R16G16B16A16_SSCALED specifies a four-component, 64-bit signed scaled integer format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, a 16-bit B component in bytes 4..5, and a 16-bit A component in bytes 6..7.
- VK_FORMAT_R16G16B16A16_UINT specifies a four-component, 64-bit unsigned integer format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, a 16-bit B component in bytes 4..5, and a 16-bit A component in bytes 6..7.
- VK_FORMAT_R16G16B16A16_SINT specifies a four-component, 64-bit signed integer format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, a 16-bit B component in bytes 4..5, and a 16-bit A component in bytes 6..7.
- VK_FORMAT_R16G16B16A16_SFLOAT specifies a four-component, 64-bit signed floating-point format that has a 16-bit R component in bytes 0..1, a 16-bit G component in bytes 2..3, a 16-bit B component in bytes 4..5, and a 16-bit A component in bytes 6..7.
- VK_FORMAT_R32_UINT specifies a one-component, 32-bit unsigned integer format that has a single 32-bit R component.
- VK_FORMAT_R32_SINT specifies a one-component, 32-bit signed integer format that has a single 32-bit R component.
- VK_FORMAT_R32_SFLOAT specifies a one-component, 32-bit signed floating-point format that has a

single 32-bit R component.

- VK_FORMAT_R32G32_UINT specifies a two-component, 64-bit unsigned integer format that has a 32-bit R component in bytes 0..3, and a 32-bit G component in bytes 4..7.
- VK_FORMAT_R32G32_SINT specifies a two-component, 64-bit signed integer format that has a 32-bit R component in bytes 0..3, and a 32-bit G component in bytes 4..7.
- VK_FORMAT_R32G32_SFLOAT specifies a two-component, 64-bit signed floating-point format that has a 32-bit R component in bytes 0..3, and a 32-bit G component in bytes 4..7.
- VK_FORMAT_R32G32B32_UINT specifies a three-component, 96-bit unsigned integer format that has a 32-bit R component in bytes 0..3, a 32-bit G component in bytes 4..7, and a 32-bit B component in bytes 8..11.
- VK_FORMAT_R32G32B32_SINT specifies a three-component, 96-bit signed integer format that has a 32-bit R component in bytes 0..3, a 32-bit G component in bytes 4..7, and a 32-bit B component in bytes 8..11.
- VK_FORMAT_R32G32B32_SFLOAT specifies a three-component, 96-bit signed floating-point format that has a 32-bit R component in bytes 0..3, a 32-bit G component in bytes 4..7, and a 32-bit B component in bytes 8..11.
- VK_FORMAT_R32G32B32A32_UINT specifies a four-component, 128-bit unsigned integer format that has a 32-bit R component in bytes 0..3, a 32-bit G component in bytes 4..7, a 32-bit B component in bytes 8..11, and a 32-bit A component in bytes 12..15.
- VK_FORMAT_R32G32B32A32_SINT specifies a four-component, 128-bit signed integer format that has a 32-bit R component in bytes 0..3, a 32-bit G component in bytes 4..7, a 32-bit B component in bytes 8..11, and a 32-bit A component in bytes 12..15.
- VK_FORMAT_R32G32B32A32_SFLOAT specifies a four-component, 128-bit signed floating-point format that has a 32-bit R component in bytes 0..3, a 32-bit G component in bytes 4..7, a 32-bit B component in bytes 8..11, and a 32-bit A component in bytes 12..15.
- VK_FORMAT_R64_UINT specifies a one-component, 64-bit unsigned integer format that has a single 64-bit R component.
- VK_FORMAT_R64_SINT specifies a one-component, 64-bit signed integer format that has a single 64-bit R component.
- VK_FORMAT_R64_SFLOAT specifies a one-component, 64-bit signed floating-point format that has a single 64-bit R component.
- VK_FORMAT_R64G64_UINT specifies a two-component, 128-bit unsigned integer format that has a 64-bit R component in bytes 0..7, and a 64-bit G component in bytes 8..15.
- VK_FORMAT_R64G64_SINT specifies a two-component, 128-bit signed integer format that has a 64-bit R component in bytes 0..7, and a 64-bit G component in bytes 8..15.
- VK_FORMAT_R64G64_SFLOAT specifies a two-component, 128-bit signed floating-point format that has a 64-bit R component in bytes 0..7, and a 64-bit G component in bytes 8..15.
- VK_FORMAT_R64G64B64_UINT specifies a three-component, 192-bit unsigned integer format that has a 64-bit R component in bytes 0..7, a 64-bit G component in bytes 8..15, and a 64-bit B component in bytes 16..23.
- VK_FORMAT_R64G64B64_SINT specifies a three-component, 192-bit signed integer format that has a

64-bit R component in bytes 0..7, a 64-bit G component in bytes 8..15, and a 64-bit B component in bytes 16..23.

- VK_FORMAT_R64G64B64_SFLOAT specifies a three-component, 192-bit signed floating-point format that has a 64-bit R component in bytes 0..7, a 64-bit G component in bytes 8..15, and a 64-bit B component in bytes 16..23.
- VK_FORMAT_R64G64B64A64_UINT specifies a four-component, 256-bit unsigned integer format that has a 64-bit R component in bytes 0..7, a 64-bit G component in bytes 8..15, a 64-bit B component in bytes 16..23, and a 64-bit A component in bytes 24..31.
- VK_FORMAT_R64G64B64A64_SINT specifies a four-component, 256-bit signed integer format that has a 64-bit R component in bytes 0..7, a 64-bit G component in bytes 8..15, a 64-bit B component in bytes 16..23, and a 64-bit A component in bytes 24..31.
- VK_FORMAT_R64G64B64A64_SFLOAT specifies a four-component, 256-bit signed floating-point format that has a 64-bit R component in bytes 0..7, a 64-bit G component in bytes 8..15, a 64-bit B component in bytes 16..23, and a 64-bit A component in bytes 24..31.
- VK_FORMAT_B10G11R11_UFLOAT_PACK32 specifies a three-component, 32-bit packed unsigned floating-point format that has a 10-bit B component in bits 22..31, an 11-bit G component in bits 11..21, an 11-bit R component in bits 0..10. See Unsigned 10-Bit Floating-Point Numbers and Unsigned 11-Bit Floating-Point Numbers.
- VK_FORMAT_E5B9G9R9_UFLOAT_PACK32 specifies a three-component, 32-bit packed unsigned floating-point format that has a 5-bit shared exponent in bits 27..31, a 9-bit B component mantissa in bits 18..26, a 9-bit G component mantissa in bits 9..17, and a 9-bit R component mantissa in bits 0..8.
- VK_FORMAT_D16_UNORM specifies a one-component, 16-bit unsigned normalized format that has a single 16-bit depth component.
- VK_FORMAT_X8_D24_UNORM_PACK32 specifies a two-component, 32-bit format that has 24 unsigned normalized bits in the depth component and, **optionally**, 8 bits that are unused.
- VK_FORMAT_D32_SFLOAT specifies a one-component, 32-bit signed floating-point format that has 32 bits in the depth component.
- VK_FORMAT_S8_UINT specifies a one-component, 8-bit unsigned integer format that has 8 bits in the stencil component.
- VK_FORMAT_D16_UNORM_S8_UINT specifies a two-component, 24-bit format that has 16 unsigned normalized bits in the depth component and 8 unsigned integer bits in the stencil component.
- VK_FORMAT_D24_UNORM_S8_UINT specifies a two-component, 32-bit packed format that has 8 unsigned integer bits in the stencil component, and 24 unsigned normalized bits in the depth component.
- VK_FORMAT_D32_SFLOAT_S8_UINT specifies a two-component format that has 32 signed float bits in the depth component and 8 unsigned integer bits in the stencil component. There are **optionally** 24 bits that are unused.
- VK_FORMAT_BC1_RGB_UNORM_BLOCK specifies a three-component, block-compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGB texel data. This format has no alpha and is considered opaque.
- VK_FORMAT_BC1_RGB_SRGB_BLOCK specifies a three-component, block-compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGB texel

data with sRGB nonlinear encoding. This format has no alpha and is considered opaque.

- VK_FORMAT_BC1_RGBA_UNORM_BLOCK specifies a four-component, block-compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGB texel data, and provides 1 bit of alpha.
- VK_FORMAT_BC1_RGBA_SRGB_BLOCK specifies a four-component, block-compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGB texel data with sRGB nonlinear encoding, and provides 1 bit of alpha.
- VK_FORMAT_BC2_UNORM_BLOCK specifies a four-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data with the first 64 bits encoding alpha values followed by 64 bits encoding RGB values.
- VK_FORMAT_BC2_SRGB_BLOCK specifies a four-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data with the first 64 bits encoding alpha values followed by 64 bits encoding RGB values with sRGB nonlinear encoding.
- VK_FORMAT_BC3_UNORM_BLOCK specifies a four-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data with the first 64 bits encoding alpha values followed by 64 bits encoding RGB values.
- VK_FORMAT_BC3_SRGB_BLOCK specifies a four-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data with the first 64 bits encoding alpha values followed by 64 bits encoding RGB values with sRGB nonlinear encoding.
- VK_FORMAT_BC4_UNORM_BLOCK specifies a one-component, block-compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized red texel data.
- VK_FORMAT_BC4_SNORM_BLOCK specifies a one-component, block-compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of signed normalized red texel data.
- VK_FORMAT_BC5_UNORM_BLOCK specifies a two-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RG texel data with the first 64 bits encoding red values followed by 64 bits encoding green values.
- VK_FORMAT_BC5_SNORM_BLOCK specifies a two-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of signed normalized RG texel data with the first 64 bits encoding red values followed by 64 bits encoding green values.
- VK_FORMAT_BC6H_UFLOAT_BLOCK specifies a three-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned floating-point RGB texel data.
- VK_FORMAT_BC6H_SFLOAT_BLOCK specifies a three-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of signed floating-point RGB texel data.
- VK_FORMAT_BC7_UNORM_BLOCK specifies a four-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_BC7_SRGB_BLOCK specifies a four-component, block-compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.

- VK_FORMAT_ETC2_R8G8B8_UNORM_BLOCK specifies a three-component, ETC2 compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGB texel data. This format has no alpha and is considered opaque.
- VK_FORMAT_ETC2_R8G8B8_SRGB_BLOCK specifies a three-component, ETC2 compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGB texel data with sRGB nonlinear encoding. This format has no alpha and is considered opaque.
- VK_FORMAT_ETC2_R8G8B8A1_UNORM_BLOCK specifies a four-component, ETC2 compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGB texel data, and provides 1 bit of alpha.
- VK_FORMAT_ETC2_R8G8B8A1_SRGB_BLOCK specifies a four-component, ETC2 compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGB texel data with sRGB nonlinear encoding, and provides 1 bit of alpha.
- VK_FORMAT_ETC2_R8G8B8A8_UNORM_BLOCK specifies a four-component, ETC2 compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data with the first 64 bits encoding alpha values followed by 64 bits encoding RGB values.
- VK_FORMAT_ETC2_R8G8B8A8_SRGB_BLOCK specifies a four-component, ETC2 compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data with the first 64 bits encoding alpha values followed by 64 bits encoding RGB values with sRGB nonlinear encoding applied.
- VK_FORMAT_EAC_R11_UNORM_BLOCK specifies a one-component, ETC2 compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized red texel data.
- VK_FORMAT_EAC_R11_SNORM_BLOCK specifies a one-component, ETC2 compressed format where each 64-bit compressed texel block encodes a 4×4 rectangle of signed normalized red texel data.
- VK_FORMAT_EAC_R11G11_UNORM_BLOCK specifies a two-component, ETC2 compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RG texel data with the first 64 bits encoding red values followed by 64 bits encoding green values.
- VK_FORMAT_EAC_R11G11_SNORM_BLOCK specifies a two-component, ETC2 compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of signed normalized RG texel data with the first 64 bits encoding red values followed by 64 bits encoding green values.
- VK_FORMAT_ASTC_4x4_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_4x4_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 4×4 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_5x4_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 5×4 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_5x4_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 5×4 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.

- VK_FORMAT_ASTC_5x5_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 5×5 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_5x5_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 5×5 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_6x5_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 6×5 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_6x5_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 6×5 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_6x6_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 6×6 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_6x6_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 6×6 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_8x5_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes an 8×5 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_8x5_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes an 8×5 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_8x6_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes an 8×6 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_8x6_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes an 8×6 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_8x8_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes an 8×8 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_8x8_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes an 8×8 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_10x5_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 10×5 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_10x5_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 10×5 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.

- VK_FORMAT_ASTC_10x6_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 10×6 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_10x6_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 10×6 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_10x8_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 10×8 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_10x8_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 10×8 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_10x10_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 10×10 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_10x10_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 10×10 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_12x10_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 12×10 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_12x10_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 12×10 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_ASTC_12x12_UNORM_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 12×12 rectangle of unsigned normalized RGBA texel data.
- VK_FORMAT_ASTC_12x12_SRGB_BLOCK specifies a four-component, ASTC compressed format where each 128-bit compressed texel block encodes a 12×12 rectangle of unsigned normalized RGBA texel data with sRGB nonlinear encoding applied to the RGB components.
- VK_FORMAT_G8B8G8R8_422_UNORM specifies a four-component, 32-bit format containing a pair of G components, an R component, and a B component, collectively encoding a 2×1 rectangle of unsigned normalized RGB texel data. One G value is present at each *i* coordinate, with the B and R values shared across both G values and thus recorded at half the horizontal resolution of the image. This format has an 8-bit G component for the even *i* coordinate in byte 0, an 8-bit B component in byte 1, an 8-bit G component for the odd *i* coordinate in byte 2, and an 8-bit R component in byte 3. This format only supports images with a width that is a multiple of two. For the purposes of the constraints on copy extents, this format is treated as a compressed format with a 2×1 compressed texel block.
- VK_FORMAT_B8G8R8G8_422_UNORM specifies a four-component, 32-bit format containing a pair of G components, an R component, and a B component, collectively encoding a 2×1 rectangle of unsigned normalized RGB texel data. One G value is present at each *i* coordinate, with the B and R values shared across both G values and thus recorded at half the horizontal resolution of the

image. This format has an 8-bit B component in byte 0, an 8-bit G component for the even i coordinate in byte 1, an 8-bit R component in byte 2, and an 8-bit G component for the odd i coordinate in byte 3. This format only supports images with a width that is a multiple of two. For the purposes of the constraints on copy extents, this format is treated as a compressed format with a 2×1 compressed texel block.

- VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM specifies an unsigned normalized *multi-planar format* that has an 8-bit G component in plane 0, an 8-bit B component in plane 1, and an 8-bit R component in plane 2. The horizontal and vertical dimensions of the R and B planes are halved relative to the image dimensions, and each R and B component is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$ and $\lfloor j_G \times 0.5 \rfloor = j_B = j_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane. This format only supports images with a width and height that is a multiple of two.
- VK_FORMAT_G8_B8R8_2PLANE_420_UNORM specifies an unsigned normalized *multi-planar format* that has an 8-bit G component in plane 0, and a two-component, 16-bit BR plane 1 consisting of an 8-bit B component in byte 0 and an 8-bit R component in byte 1. The horizontal and vertical dimensions of the BR plane are halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$ and $\lfloor j_G \times 0.5 \rfloor = j_B = j_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, and VK_IMAGE_ASPECT_PLANE_1_BIT for the BR plane. This format only supports images with a width and height that is a multiple of two.
- VK_FORMAT_G8_B8_R8_3PLANE_422_UNORM specifies an unsigned normalized *multi-planar format* that has an 8-bit G component in plane 0, an 8-bit B component in plane 1, and an 8-bit R component in plane 2. The horizontal dimension of the R and B plane is halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane. This format only supports images with a width that is a multiple of two.
- VK_FORMAT_G8_B8R8_2PLANE_422_UNORM specifies an unsigned normalized *multi-planar format* that has an 8-bit G component in plane 0, and a two-component, 16-bit BR plane 1 consisting of an 8-bit B component in byte 0 and an 8-bit R component in byte 1. The horizontal dimension of the BR plane is halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, and VK_IMAGE_ASPECT_PLANE_1_BIT for the BR plane. This format only supports images with a width that is a multiple of two.
- VK_FORMAT_G8_B8_R8_3PLANE_444_UNORM specifies an unsigned normalized *multi-planar format* that has an 8-bit G component in plane 0, an 8-bit B component in plane 1, and an 8-bit R component in plane 2. Each plane has the same dimensions and each R, G and B component contributes to a single texel. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane.

- VK_FORMAT_R10X6_UNORM_PACK16 specifies a one-component, 16-bit unsigned normalized format that has a single 10-bit R component in the top 10 bits of a 16-bit word, with the bottom 6 bits unused.
- VK_FORMAT_R10X6G10X6_UNORM_2PACK16 specifies a two-component, 32-bit unsigned normalized format that has a 10-bit R component in the top 10 bits of the word in bytes 0..1, and a 10-bit G component in the top 10 bits of the word in bytes 2..3, with the bottom 6 bits of each word unused.
- VK_FORMAT_R10X6G10X6B10X6A10X6_UNORM_4PACK16 specifies a four-component, 64-bit unsigned normalized format that has a 10-bit R component in the top 10 bits of the word in bytes 0..1, a 10-bit G component in the top 10 bits of the word in bytes 2..3, a 10-bit B component in the top 10 bits of the word in bytes 4..5, and a 10-bit A component in the top 10 bits of the word in bytes 6..7, with the bottom 6 bits of each word unused.
- VK_FORMAT_G10X6B10X6G10X6R10X6_422_UNORM_4PACK16 specifies a four-component, 64-bit format containing a pair of G components, an R component, and a B component, collectively encoding a 2×1 rectangle of unsigned normalized RGB texel data. One G value is present at each i coordinate, with the B and R values shared across both G values and thus recorded at half the horizontal resolution of the image. This format has a 10-bit G component for the even i coordinate in the top 10 bits of the word in bytes 0..1, a 10-bit B component in the top 10 bits of the word in bytes 2..3, a 10-bit G component for the odd i coordinate in the top 10 bits of the word in bytes 4..5, and a 10-bit R component in the top 10 bits of the word in bytes 6..7, with the bottom 6 bits of each word unused. This format only supports images with a width that is a multiple of two. For the purposes of the constraints on copy extents, this format is treated as a compressed format with a 2×1 compressed texel block.
- VK_FORMAT_B10X6G10X6R10X6G10X6_422_UNORM_4PACK16 specifies a four-component, 64-bit format containing a pair of G components, an R component, and a B component, collectively encoding a 2×1 rectangle of unsigned normalized RGB texel data. One G value is present at each i coordinate, with the B and R values shared across both G values and thus recorded at half the horizontal resolution of the image. This format has a 10-bit B component in the top 10 bits of the word in bytes 0..1, a 10-bit G component for the even i coordinate in the top 10 bits of the word in bytes 2..3, a 10-bit R component in the top 10 bits of the word in bytes 4..5, and a 10-bit G component for the odd i coordinate in the top 10 bits of the word in bytes 6..7, with the bottom 6 bits of each word unused. This format only supports images with a width that is a multiple of two. For the purposes of the constraints on copy extents, this format is treated as a compressed format with a 2×1 compressed texel block.
- VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 10-bit G component in the top 10 bits of each 16-bit word of plane 0, a 10-bit B component in the top 10 bits of each 16-bit word of plane 1, and a 10-bit R component in the top 10 bits of each 16-bit word of plane 2, with the bottom 6 bits of each word unused. The horizontal and vertical dimensions of the R and B planes are halved relative to the image dimensions, and each R and B component is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$ and $\lfloor j_G \times 0.5 \rfloor = j_B = j_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane. This format only supports images with a width and height that is a multiple of two.

- VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 10-bit G component in the top 10 bits of each 16-bit word of plane 0, and a two-component, 32-bit BR plane 1 consisting of a 10-bit B component in the top 10 bits of the word in bytes 0..1, and a 10-bit R component in the top 10 bits of the word in bytes 2..3, with the bottom 6 bits of each word unused. The horizontal and vertical dimensions of the BR plane are halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$ and $\lfloor j_G \times 0.5 \rfloor = j_B = j_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, and VK_IMAGE_ASPECT_PLANE_1_BIT for the BR plane. This format only supports images with a width and height that is a multiple of two.
- VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 10-bit G component in the top 10 bits of each 16-bit word of plane 0, a 10-bit B component in the top 10 bits of each 16-bit word of plane 1, and a 10-bit R component in the top 10 bits of each 16-bit word of plane 2, with the bottom 6 bits of each word unused. The horizontal dimension of the R and B plane is halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane. This format only supports images with a width that is a multiple of two.
- VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 10-bit G component in the top 10 bits of each 16-bit word of plane 0, and a two-component, 32-bit BR plane 1 consisting of a 10-bit B component in the top 10 bits of the word in bytes 0..1, and a 10-bit R component in the top 10 bits of the word in bytes 2..3, with the bottom 6 bits of each word unused. The horizontal dimension of the BR plane is halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, and VK_IMAGE_ASPECT_PLANE_1_BIT for the BR plane. This format only supports images with a width that is a multiple of two.
- VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 10-bit G component in the top 10 bits of each 16-bit word of plane 0, a 10-bit B component in the top 10 bits of each 16-bit word of plane 1, and a 10-bit R component in the top 10 bits of each 16-bit word of plane 2, with the bottom 6 bits of each word unused. Each plane has the same dimensions and each R, G and B component contributes to a single texel. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane.
- VK_FORMAT_R12X4_UNORM_PACK16 specifies a one-component, 16-bit unsigned normalized format that has a single 12-bit R component in the top 12 bits of a 16-bit word, with the bottom 4 bits unused.
- VK_FORMAT_R12X4G12X4_UNORM_2PACK16 specifies a two-component, 32-bit unsigned normalized format that has a 12-bit R component in the top 12 bits of the word in bytes 0..1, and a 12-bit G component in the top 12 bits of the word in bytes 2..3, with the bottom 4 bits of each word unused.

- VK_FORMAT_R12X4G12X4B12X4A12X4_UNORM_4PACK16 specifies a four-component, 64-bit unsigned normalized format that has a 12-bit R component in the top 12 bits of the word in bytes 0..1, a 12-bit G component in the top 12 bits of the word in bytes 2..3, a 12-bit B component in the top 12 bits of the word in bytes 4..5, and a 12-bit A component in the top 12 bits of the word in bytes 6..7, with the bottom 4 bits of each word unused.
- VK_FORMAT_G12X4B12X4G12X4R12X4_422_UNORM_4PACK16 specifies a four-component, 64-bit format containing a pair of G components, an R component, and a B component, collectively encoding a 2×1 rectangle of unsigned normalized RGB texel data. One G value is present at each i coordinate, with the B and R values shared across both G values and thus recorded at half the horizontal resolution of the image. This format has a 12-bit G component for the even i coordinate in the top 12 bits of the word in bytes 0..1, a 12-bit B component in the top 12 bits of the word in bytes 2..3, a 12-bit G component for the odd i coordinate in the top 12 bits of the word in bytes 4..5, and a 12-bit R component in the top 12 bits of the word in bytes 6..7, with the bottom 4 bits of each word unused. This format only supports images with a width that is a multiple of two. For the purposes of the constraints on copy extents, this format is treated as a compressed format with a 2×1 compressed texel block.
- VK_FORMAT_B12X4G12X4R12X4G12X4_422_UNORM_4PACK16 specifies a four-component, 64-bit format containing a pair of G components, an R component, and a B component, collectively encoding a 2×1 rectangle of unsigned normalized RGB texel data. One G value is present at each i coordinate, with the B and R values shared across both G values and thus recorded at half the horizontal resolution of the image. This format has a 12-bit B component in the top 12 bits of the word in bytes 0..1, a 12-bit G component for the even i coordinate in the top 12 bits of the word in bytes 2..3, a 12-bit R component in the top 12 bits of the word in bytes 4..5, and a 12-bit G component for the odd i coordinate in the top 12 bits of the word in bytes 6..7, with the bottom 4 bits of each word unused. This format only supports images with a width that is a multiple of two. For the purposes of the constraints on copy extents, this format is treated as a compressed format with a 2×1 compressed texel block.
- VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 12-bit G component in the top 12 bits of each 16-bit word of plane 0, a 12-bit B component in the top 12 bits of each 16-bit word of plane 1, and a 12-bit R component in the top 12 bits of each 16-bit word of plane 2, with the bottom 4 bits of each word unused. The horizontal and vertical dimensions of the R and B planes are halved relative to the image dimensions, and each R and B component is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$ and $\lfloor j_G \times 0.5 \rfloor = j_B = j_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane. This format only supports images with a width and height that is a multiple of two.
- VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 12-bit G component in the top 12 bits of each 16-bit word of plane 0, and a two-component, 32-bit BR plane 1 consisting of a 12-bit B component in the top 12 bits of the word in bytes 0..1, and a 12-bit R component in the top 12 bits of the word in bytes 2..3, with the bottom 4 bits of each word unused. The horizontal and vertical dimensions of the BR plane are halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$ and $\lfloor j_G \times 0.5 \rfloor = j_B = j_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using

VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, and VK_IMAGE_ASPECT_PLANE_1_BIT for the BR plane. This format only supports images with a width and height that is a multiple of two.

- VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 12-bit G component in the top 12 bits of each 16-bit word of plane 0, a 12-bit B component in the top 12 bits of each 16-bit word of plane 1, and a 12-bit R component in the top 12 bits of each 16-bit word of plane 2, with the bottom 4 bits of each word unused. The horizontal dimension of the R and B plane is halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane. This format only supports images with a width that is a multiple of two.
- VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 12-bit G component in the top 12 bits of each 16-bit word of plane 0, and a two-component, 32-bit BR plane 1 consisting of a 12-bit B component in the top 12 bits of the word in bytes 0..1, and a 12-bit R component in the top 12 bits of the word in bytes 2..3, with the bottom 4 bits of each word unused. The horizontal dimension of the BR plane is halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, and VK_IMAGE_ASPECT_PLANE_1_BIT for the BR plane. This format only supports images with a width that is a multiple of two.
- VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16 specifies an unsigned normalized *multi-planar format* that has a 12-bit G component in the top 12 bits of each 16-bit word of plane 0, a 12-bit B component in the top 12 bits of each 16-bit word of plane 1, and a 12-bit R component in the top 12 bits of each 16-bit word of plane 2, with the bottom 4 bits of each word unused. Each plane has the same dimensions and each R, G and B component contributes to a single texel. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane.
- VK_FORMAT_G16B16G16R16_422_UNORM specifies a four-component, 64-bit format containing a pair of G components, an R component, and a B component, collectively encoding a 2×1 rectangle of unsigned normalized RGB texel data. One G value is present at each *i* coordinate, with the B and R values shared across both G values and thus recorded at half the horizontal resolution of the image. This format has a 16-bit G component for the even *i* coordinate in the word in bytes 0..1, a 16-bit B component in the word in bytes 2..3, a 16-bit G component for the odd *i* coordinate in the word in bytes 4..5, and a 16-bit R component in the word in bytes 6..7. This format only supports images with a width that is a multiple of two. For the purposes of the constraints on copy extents, this format is treated as a compressed format with a 2×1 compressed texel block.
- VK_FORMAT_B16G16R16G16_422_UNORM specifies a four-component, 64-bit format containing a pair of G components, an R component, and a B component, collectively encoding a 2×1 rectangle of unsigned normalized RGB texel data. One G value is present at each *i* coordinate, with the B and R values shared across both G values and thus recorded at half the horizontal resolution of the image. This format has a 16-bit B component in the word in bytes 0..1, a 16-bit G component for the even *i* coordinate in the word in bytes 2..3, a 16-bit R component in the word in bytes 4..5, and a 16-bit G component for the odd *i* coordinate in the word in bytes 6..7. This format only

supports images with a width that is a multiple of two. For the purposes of the constraints on copy extents, this format is treated as a compressed format with a 2×1 compressed texel block.

- VK_FORMAT_G16_B16_R16_3PLANE_420_UNORM specifies an unsigned normalized *multi-planar format* that has a 16-bit G component in each 16-bit word of plane 0, a 16-bit B component in each 16-bit word of plane 1, and a 16-bit R component in each 16-bit word of plane 2. The horizontal and vertical dimensions of the R and B planes are halved relative to the image dimensions, and each R and B component is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$ and $\lfloor j_G \times 0.5 \rfloor = j_B = j_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane. This format only supports images with a width and height that is a multiple of two.
- VK_FORMAT_G16_B16R16_2PLANE_420_UNORM specifies an unsigned normalized *multi-planar format* that has a 16-bit G component in each 16-bit word of plane 0, and a two-component, 32-bit BR plane 1 consisting of a 16-bit B component in the word in bytes 0..1, and a 16-bit R component in the word in bytes 2..3. The horizontal and vertical dimensions of the BR plane are halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$ and $\lfloor j_G \times 0.5 \rfloor = j_B = j_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, and VK_IMAGE_ASPECT_PLANE_1_BIT for the BR plane. This format only supports images with a width and height that is a multiple of two.
- VK_FORMAT_G16_B16_R16_3PLANE_422_UNORM specifies an unsigned normalized *multi-planar format* that has a 16-bit G component in each 16-bit word of plane 0, a 16-bit B component in each 16-bit word of plane 1, and a 16-bit R component in each 16-bit word of plane 2. The horizontal dimension of the R and B plane is halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane. This format only supports images with a width that is a multiple of two.
- VK_FORMAT_G16_B16R16_2PLANE_422_UNORM specifies an unsigned normalized *multi-planar format* that has a 16-bit G component in each 16-bit word of plane 0, and a two-component, 32-bit BR plane 1 consisting of a 16-bit B component in the word in bytes 0..1, and a 16-bit R component in the word in bytes 2..3. The horizontal dimension of the BR plane is halved relative to the image dimensions, and each R and B value is shared with the G components for which $\lfloor i_G \times 0.5 \rfloor = i_B = i_R$. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, and VK_IMAGE_ASPECT_PLANE_1_BIT for the BR plane. This format only supports images with a width that is a multiple of two.
- VK_FORMAT_G16_B16_R16_3PLANE_444_UNORM specifies an unsigned normalized *multi-planar format* that has a 16-bit G component in each 16-bit word of plane 0, a 16-bit B component in each 16-bit word of plane 1, and a 16-bit R component in each 16-bit word of plane 2. Each plane has the same dimensions and each R, G and B component contributes to a single texel. The location of each plane when this image is in linear layout can be determined via vkGetImageSubresourceLayout, using VK_IMAGE_ASPECT_PLANE_0_BIT for the G plane, VK_IMAGE_ASPECT_PLANE_1_BIT for the B plane, and VK_IMAGE_ASPECT_PLANE_2_BIT for the R plane.

42.1.1. Compatible Formats of Planes of Multi-Planar Formats

Individual planes of multi-planar formats are size-compatible with single-plane color formats if they occupy the same number of bits per texel block, and are compatible with those formats if they have the same block extent.

In the following table, individual planes of a *multi-planar* format are compatible with the format listed against the relevant plane index for that multi-planar format, and any format compatible with the listed single-plane format according to Format Compatibility Classes. These planes are also size-compatible with any format that is size-compatible with the listed single-plane format.

Table 63. Plane Format Compatibility Table

Plane	Compatible format for plane	Width relative to the width *w* of the plane with the largest dimensions	Height relative to the height *h* of the plane with the largest dimensions
VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM			
0	VK_FORMAT_R8_UNORM	w	h
1	VK_FORMAT_R8_UNORM	w/2	h/2
2	VK_FORMAT_R8_UNORM	w/2	h/2
VK_FORMAT_G8_B8R8_2PLANE_420_UNORM			
0	VK_FORMAT_R8_UNORM	w	h
1	VK_FORMAT_R8G8_UNORM	w/2	h/2
VK_FORMAT_G8_B8_R8_3PLANE_422_UNORM			
0	VK_FORMAT_R8_UNORM	w	h
1	VK_FORMAT_R8_UNORM	w/2	h
2	VK_FORMAT_R8_UNORM	w/2	h
VK_FORMAT_G8_B8R8_2PLANE_422_UNORM			
0	VK_FORMAT_R8_UNORM	w	h
1	VK_FORMAT_R8G8_UNORM	w/2	h
VK_FORMAT_G8_B8_R8_3PLANE_444_UNORM			
0	VK_FORMAT_R8_UNORM	w	h
1	VK_FORMAT_R8_UNORM	w	h
2	VK_FORMAT_R8_UNORM	w	h
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16			
0	VK_FORMAT_R10X6_UNORM_PACK16	w	h
1	VK_FORMAT_R10X6_UNORM_PACK16	w/2	h/2
2	VK_FORMAT_R10X6_UNORM_PACK16	w/2	h/2
VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16			
0	VK_FORMAT_R10X6_UNORM_PACK16	w	h

Plane	Compatible format for plane	Width relative to the width w of the plane with the largest dimensions	Height relative to the height h of the plane with the largest dimensions
1	VK_FORMAT_R10X6G10X6_UNORM_2PACK16	w/2	h/2
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16			
0	VK_FORMAT_R10X6_UNORM_PACK16	w	h
1	VK_FORMAT_R10X6_UNORM_PACK16	w/2	h
2	VK_FORMAT_R10X6_UNORM_PACK16	w/2	h
VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16			
0	VK_FORMAT_R10X6_UNORM_PACK16	w	h
1	VK_FORMAT_R10X6G10X6_UNORM_2PACK16	w/2	h
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16			
0	VK_FORMAT_R10X6_UNORM_PACK16	w	h
1	VK_FORMAT_R10X6_UNORM_PACK16	w	h
2	VK_FORMAT_R10X6_UNORM_PACK16	w	h
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16			
0	VK_FORMAT_R12X4_UNORM_PACK16	w	h
1	VK_FORMAT_R12X4_UNORM_PACK16	w/2	h/2
2	VK_FORMAT_R12X4_UNORM_PACK16	w/2	h/2
VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16			
0	VK_FORMAT_R12X4_UNORM_PACK16	w	h
1	VK_FORMAT_R12X4G12X4_UNORM_2PACK16	w/2	h/2
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16			
0	VK_FORMAT_R12X4_UNORM_PACK16	w	h
1	VK_FORMAT_R12X4_UNORM_PACK16	w/2	h
2	VK_FORMAT_R12X4_UNORM_PACK16	w/2	h
VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16			
0	VK_FORMAT_R12X4_UNORM_PACK16	w	h
1	VK_FORMAT_R12X4G12X4_UNORM_2PACK16	w/2	h
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16			
0	VK_FORMAT_R12X4_UNORM_PACK16	w	h
1	VK_FORMAT_R12X4_UNORM_PACK16	w	h
2	VK_FORMAT_R12X4_UNORM_PACK16	w	h
VK_FORMAT_G16_B16_R16_3PLANE_420_UNORM			
0	VK_FORMAT_R16_UNORM	w	h

Plane	Compatible format for plane	Width relative to the width *w* of the plane with the largest dimensions	Height relative to the height *h* of the plane with the largest dimensions
1	VK_FORMAT_R16_UNORM	w/2	h/2
2	VK_FORMAT_R16_UNORM	w/2	h/2
VK_FORMAT_G16_B16R16_2PLANE_420_UNORM			
0	VK_FORMAT_R16_UNORM	w	h
1	VK_FORMAT_R16G16_UNORM	w/2	h/2
VK_FORMAT_G16_B16_R16_3PLANE_422_UNORM			
0	VK_FORMAT_R16_UNORM	w	h
1	VK_FORMAT_R16_UNORM	w/2	h
2	VK_FORMAT_R16_UNORM	w/2	h
VK_FORMAT_G16_B16R16_2PLANE_422_UNORM			
0	VK_FORMAT_R16_UNORM	w	h
1	VK_FORMAT_R16G16_UNORM	w/2	h
VK_FORMAT_G16_B16_R16_3PLANE_444_UNORM			
0	VK_FORMAT_R16_UNORM	w	h
1	VK_FORMAT_R16_UNORM	w	h
2	VK_FORMAT_R16_UNORM	w	h

42.1.2. Multi-planar Format Image Aspect

When using VkImageAspectFlagBits to select a plane of a multi-planar format, the following are the valid options:

- Two planes
  - VK_IMAGE_ASPECT_PLANE_0_BIT
  - VK_IMAGE_ASPECT_PLANE_1_BIT
- Three planes
  - VK_IMAGE_ASPECT_PLANE_0_BIT
  - VK_IMAGE_ASPECT_PLANE_1_BIT
  - VK_IMAGE_ASPECT_PLANE_2_BIT

42.1.3. Packed Formats

For the purposes of address alignment when accessing buffer memory containing vertex attribute or texel data, the following formats are considered *packed* - components of the texels or attributes are stored in bitfields packed into one or more 8-, 16-, or 32-bit fundamental data type.

- Packed into 8-bit data types:
  - VK_FORMAT_R4G4_UNORM_PACK8
- Packed into 16-bit data types:
  - VK_FORMAT_R4G4B4A4_UNORM_PACK16
  - VK_FORMAT_B4G4R4A4_UNORM_PACK16
  - VK_FORMAT_R5G6B5_UNORM_PACK16
  - VK_FORMAT_B5G6R5_UNORM_PACK16
  - VK_FORMAT_R5G5B5A1_UNORM_PACK16
  - VK_FORMAT_B5G5R5A1_UNORM_PACK16
  - VK_FORMAT_A1R5G5B5_UNORM_PACK16
  - VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR
  - VK_FORMAT_R10X6_UNORM_PACK16
  - VK_FORMAT_R10X6G10X6_UNORM_2PACK16
  - VK_FORMAT_R10X6G10X6B10X6A10X6_UNORM_4PACK16
  - VK_FORMAT_G10X6B10X6G10X6R10X6_422_UNORM_4PACK16
  - VK_FORMAT_B10X6G10X6R10X6G10X6_422_UNORM_4PACK16
  - VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16
  - VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16
  - VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16
  - VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16
  - VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16
  - VK_FORMAT_R12X4_UNORM_PACK16
  - VK_FORMAT_R12X4G12X4_UNORM_2PACK16
  - VK_FORMAT_R12X4G12X4B12X4A12X4_UNORM_4PACK16
  - VK_FORMAT_G12X4B12X4G12X4R12X4_422_UNORM_4PACK16
  - VK_FORMAT_B12X4G12X4R12X4G12X4_422_UNORM_4PACK16
  - VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16
  - VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16
  - VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16
  - VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16
  - VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16
- Packed into 32-bit data types:
  - VK_FORMAT_A8B8G8R8_UNORM_PACK32
  - VK_FORMAT_A8B8G8R8_SNORM_PACK32
  - VK_FORMAT_A8B8G8R8_USCALED_PACK32

- VK_FORMAT_A8B8G8R8_SSCALED_PACK32
- VK_FORMAT_A8B8G8R8_UINT_PACK32
- VK_FORMAT_A8B8G8R8_SINT_PACK32
- VK_FORMAT_A8B8G8R8_SRGB_PACK32
- VK_FORMAT_A2R10G10B10_UNORM_PACK32
- VK_FORMAT_A2R10G10B10_SNORM_PACK32
- VK_FORMAT_A2R10G10B10_USCALED_PACK32
- VK_FORMAT_A2R10G10B10_SSCALED_PACK32
- VK_FORMAT_A2R10G10B10_UINT_PACK32
- VK_FORMAT_A2R10G10B10_SINT_PACK32
- VK_FORMAT_A2B10G10R10_UNORM_PACK32
- VK_FORMAT_A2B10G10R10_SNORM_PACK32
- VK_FORMAT_A2B10G10R10_USCALED_PACK32
- VK_FORMAT_A2B10G10R10_SSCALED_PACK32
- VK_FORMAT_A2B10G10R10_UINT_PACK32
- VK_FORMAT_A2B10G10R10_SINT_PACK32
- VK_FORMAT_B10G11R11_UFLOAT_PACK32
- VK_FORMAT_E5B9G9R9_UFLOAT_PACK32
- VK_FORMAT_X8_D24_UNORM_PACK32

42.1.4. Identification of Formats

A “format” is represented by a single enum value. The name of a format is usually built up by using the following pattern:

```
VK_FORMAT_{component-format|compression-scheme}_{numeric-format}
```

The component-format indicates either the size of the R, G, B, and A components (if they are present) in the case of a color format, or the size of the depth (D) and stencil (S) components (if they are present) in the case of a depth/stencil format (see below). An X indicates a component that is unused, but **may** be present for padding.

Table 64. Interpretation of Numeric Format

Numeric format	Type-Declaration instructions	Numeric type	Description
`UNORM`	OpTypeFloat	floating-point	The components are unsigned normalized values in the range [0,1]
`SNORM`	OpTypeFloat	floating-point	The components are signed normalized values in the range [-1,1]
`USCALED`	OpTypeFloat	floating-point	The components are unsigned integer values that get converted to floating-point in the range [0,2^n-1]
`SSCALED`	OpTypeFloat	floating-point	The components are signed integer values that get converted to floating-point in the range [-2^{n-1},2^{n-1}-1]
`UINT`	OpTypeInt	unsigned integer	The components are unsigned integer values in the range [0,2^n-1]
`SINT`	OpTypeInt	signed integer	The components are signed integer values in the range [-2^{n-1},2^{n-1}-1]
`UFLOAT`	OpTypeFloat	floating-point	The components are unsigned floating-point numbers (used by packed, shared exponent, and some compressed formats)
`SFLOAT`	OpTypeFloat	floating-point	The components are signed floating-point numbers
`SRGB`	OpTypeFloat	floating-point	The R, G, and B components are unsigned normalized values that represent values using sRGB nonlinear encoding, while the A component (if one exists) is a regular unsigned normalized value
n is the number of bits in the component.			

The suffix `_PACKnn` indicates that the format is packed into an underlying type with `nn` bits. The suffix `_mPACKnn` is a short-hand that indicates that the format has `m` groups of components (which may or may not be stored in separate *planes*) that are each packed into an underlying type with `nn` bits.

The suffix `_BLOCK` indicates that the format is a block-compressed format, with the representation of multiple pixels encoded interdependently within a region.

Table 65. Interpretation of Compression Scheme

Compression scheme	Description
`BC`	Block Compression. See Block-Compressed Image Formats.
`ETC2`	Ericsson Texture Compression. See ETC Compressed Image Formats.
`EAC`	ETC2 Alpha Compression. See ETC Compressed Image Formats.

Compression scheme	Description
ASTC	Adaptive Scalable Texture Compression (LDR Profile). See ASTC Compressed Image Formats.

For *multi-planar* images, the components in separate *planes* are separated by underscores, and the number of planes is indicated by the addition of a `_2PLANE` or `_3PLANE` suffix. Similarly, the separate aspects of depth-stencil formats are separated by underscores, although these are not considered separate planes. Formats are suffixed by `_422` to indicate that planes other than the first are reduced in size by a factor of two horizontally or that the R and B values appear at half the horizontal frequency of the G values, `_420` to indicate that planes other than the first are reduced in size by a factor of two both horizontally and vertically, and `_444` for consistency to indicate that all three planes of a three-planar image are the same size.

NOTE No common format has a single plane containing both R and B components but does not store these components at reduced horizontal resolution.

42.1.5. Representation and Texel Block Size

Color formats **must** be represented in memory in exactly the form indicated by the format's name. This means that promoting one format to another with more bits per component and/or additional components **must** not occur for color formats. Depth/stencil formats have more relaxed requirements as discussed below.

Each format has a *texel block size*, the number of bytes used to store one *texel block* (a single addressable element of an uncompressed image, or a single compressed block of a compressed image). The texel block size for each format is shown in the Compatible formats table.

The representation of non-packed formats is that the first component specified in the name of the format is in the lowest memory addresses and the last component specified is in the highest memory addresses. See Byte mappings for non-packed/compressed color formats. The in-memory ordering of bytes within a component is determined by the host endianness.

Table 66. Byte Mappings for Non-Packed/Compressed Color Formats

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	← Byte
R																VK_FORMAT_R8_*
R	G															VK_FORMAT_R8G8_*
R	G	B														VK_FORMAT_R8G8B8_*
B	G	R														VK_FORMAT_B8G8R8_*
R	G	B	A													VK_FORMAT_R8G8B8A8_*
B	G	R	A													VK_FORMAT_B8G8R8A8_*
A																VK_FORMAT_A8_UNORM_KHR
G_0	B	G_1	R													VK_FORMAT_G8B8G8R8_422_UNORM
B	G_0	R	G_1													VK_FORMAT_B8G8R8G8_422_UNORM

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	← Byte
R																VK_FORMAT_R16_*
R		G														VK_FORMAT_R16G16_*
R		G		B												VK_FORMAT_R16G16B16_*
R		G		B		A										VK_FORMAT_R16G16B16A16_*
G_0		B		G_1		R										VK_FORMAT_G10X6B10X6G10X6R10X6_4PACK16_422_UNORM VK_FORMAT_G12X4B12X4G12X4R12X4_4PACK16_422_UNORM VK_FORMAT_G16B16G16R16_UNORM
B		G_0		R		G_1										VK_FORMAT_B10X6G10X6R10X6G10X6_4PACK16_422_UNORM VK_FORMAT_B12X4G12X4R12X4G12X4_4PACK16_422_UNORM VK_FORMAT_B16G16R16G16_422_UNORM
R																VK_FORMAT_R32_*
R				G												VK_FORMAT_R32G32_*
R				G				B								VK_FORMAT_R32G32B32_*
R				G				B				A				VK_FORMAT_R32G32B32A32_*
R																VK_FORMAT_R64_*
R								G								VK_FORMAT_R64G64_*
VK_FORMAT_R64G64B64_* **as** VK_FORMAT_R64G64_* **but with B in bytes 16-23**																
VK_FORMAT_R64G64B64A64_* **as** VK_FORMAT_R64G64B64_* **but with A in bytes 24-31**																

Packed formats store multiple components within one underlying type. The bit representation is that the first component specified in the name of the format is in the most-significant bits and the last component specified is in the least-significant bits of the underlying type. The in-memory ordering of bytes comprising the underlying type is determined by the host endianness.

Table 67. Bit Mappings for Packed 8-Bit Formats

Bit							
7	6	5	4	3	2	1	0
VK_FORMAT_R4G4_UNORM_PACK8							
R				G			
3	2	1	0	3	2	1	0

Table 68. Bit Mappings for Packed 16-Bit Formats

Bit															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VK_FORMAT_R4G4B4A4_UNORM_PACK16															
R				G				B				A			
3	2	1	0	3	2	1	0	3	2	1	0	3	2	1	0
VK_FORMAT_B4G4R4A4_UNORM_PACK16															
B				G				R				A			

Bit															
3	2	1	0	3	2	1	0	3	2	1	0	3	2	1	0
VK_FORMAT_R5G6B5_UNORM_PACK16															
R				G						B					
4	3	2	1	0	5	4	3	2	1	0	4	3	2	1	0
VK_FORMAT_B5G6R5_UNORM_PACK16															
B				G						R					
4	3	2	1	0	5	4	3	2	1	0	4	3	2	1	0
VK_FORMAT_R5G5B5A1_UNORM_PACK16															
R				G						B					A
4	3	2	1	0	4	3	2	1	0	4	3	2	1	0	0
VK_FORMAT_B5G5R5A1_UNORM_PACK16															
B				G						R					A
4	3	2	1	0	4	3	2	1	0	4	3	2	1	0	0
VK_FORMAT_A1R5G5B5_UNORM_PACK16															
A	R					G					B				
0	4	3	2	1	0	4	3	2	1	0	4	3	2	1	0
VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR															
A	B					G					R				
0	4	3	2	1	0	4	3	2	1	0	4	3	2	1	0
VK_FORMAT_R10X6_UNORM_PACK16															
R										X					
9	8	7	6	5	4	3	2	1	0	5	4	3	2	1	0
VK_FORMAT_R12X4_UNORM_PACK16															
R										X					
11	10	9	8	7	6	5	4	3	2	1	0	3	2	1	0

Table 69. Bit Mappings for Packed 32-Bit Formats

Bit																															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VK_FORMAT_A8B8G8R8_*_PACK32																															
A								B								G								R							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
VK_FORMAT_A2R10G10B10_*_PACK32																															
A				R								G								B											
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0
VK_FORMAT_A2B10G10R10_*_PACK32																															
A				B								G								R											
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0
VK_FORMAT_B10G11R11_UFLOAT_PACK32																															

Bit																															
B										G										R											
9	8	7	6	5	4	3	2	1	0	10	9	8	7	6	5	4	3	2	1	0	10	9	8	7	6	5	4	3	2	1	0
VK_FORMAT_E5B9G9R9_UFLOAT_PACK32																															
E					B							G								R										
4	3	2	1	0	8	7	6	5	4	3	2	1	0	8	7	6	5	4	3	2	1	0	8	7	6	5	4	3	2	1	0
VK_FORMAT_X8_D24_UNORM_PACK32																															
X								D																							
7	6	5	4	3	2	1	0	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

42.1.6. Depth/Stencil Formats

Depth/stencil formats are considered opaque and need not be stored in the exact number of bits per texel or component ordering indicated by the format enum. However, implementations **must** not substitute a different depth or stencil precision than is described in the format (e.g. D16 **must** not be implemented as D24 or D32).

42.1.7. Format Compatibility Classes

Uncompressed color formats are *compatible* with each other if they occupy the same number of bits per texel block as long as neither or both are alpha formats (e.g., VK_FORMAT_A8_UNORM_KHR) . Compressed color formats are compatible with each other if the only difference between them is the numeric format of the uncompressed pixels. Each depth/stencil format is only compatible with itself. In the following table, all the formats in the same row are compatible. Each format has a defined *texel block extent* specifying how many texels each texel block represents in each dimension.

Table 70. Compatible Formats

Class, Texel Block Size, Texel Block Extent, # Texels/Block	**Formats**
8-bit Block size 1 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R4G4_UNORM_PACK8, VK_FORMAT_R8_UNORM, VK_FORMAT_R8_SNORM, VK_FORMAT_R8_USCALED, VK_FORMAT_R8_SSCALED, VK_FORMAT_R8_UINT, VK_FORMAT_R8_SINT, VK_FORMAT_R8_SRGB

Class, Texel Block Size, Texel Block Extent, # Texels/Block	**Formats**
16-bit Block size 2 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR, VK_FORMAT_R10X6_UNORM_PACK16, VK_FORMAT_R12X4_UNORM_PACK16, VK_FORMAT_R4G4B4A4_UNORM_PACK16, VK_FORMAT_B4G4R4A4_UNORM_PACK16, VK_FORMAT_R5G6B5_UNORM_PACK16, VK_FORMAT_B5G6R5_UNORM_PACK16, VK_FORMAT_R5G5B5A1_UNORM_PACK16, VK_FORMAT_B5G5R5A1_UNORM_PACK16, VK_FORMAT_A1R5G5B5_UNORM_PACK16, VK_FORMAT_R8G8_UNORM, VK_FORMAT_R8G8_SNORM, VK_FORMAT_R8G8_USCALED, VK_FORMAT_R8G8_SSCALED, VK_FORMAT_R8G8_UINT, VK_FORMAT_R8G8_SINT, VK_FORMAT_R8G8_SRGB, VK_FORMAT_R16_UNORM, VK_FORMAT_R16_SNORM, VK_FORMAT_R16_USCALED, VK_FORMAT_R16_SSCALED, VK_FORMAT_R16_UINT, VK_FORMAT_R16_SINT, VK_FORMAT_R16_SFLOAT
8-bit alpha Block size 1 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_A8_UNORM_KHR
24-bit Block size 3 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R8G8B8_UNORM, VK_FORMAT_R8G8B8_SNORM, VK_FORMAT_R8G8B8_USCALED, VK_FORMAT_R8G8B8_SSCALED, VK_FORMAT_R8G8B8_UINT, VK_FORMAT_R8G8B8_SINT, VK_FORMAT_R8G8B8_SRGB, VK_FORMAT_B8G8R8_UNORM, VK_FORMAT_B8G8R8_SNORM, VK_FORMAT_B8G8R8_USCALED, VK_FORMAT_B8G8R8_SSCALED, VK_FORMAT_B8G8R8_UINT, VK_FORMAT_B8G8R8_SINT, VK_FORMAT_B8G8R8_SRGB

Class, Texel Block Size, Texel Block Extent, # Texels/Block	Formats
32-bit Block size 4 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R10X6G10X6_UNORM_2PACK16, VK_FORMAT_R12X4G12X4_UNORM_2PACK16, VK_FORMAT_R8G8B8A8_UNORM, VK_FORMAT_R8G8B8A8_SNORM, VK_FORMAT_R8G8B8A8_USCALED, VK_FORMAT_R8G8B8A8_SSCALED, VK_FORMAT_R8G8B8A8_UINT, VK_FORMAT_R8G8B8A8_SINT, VK_FORMAT_R8G8B8A8_SRGB, VK_FORMAT_B8G8R8A8_UNORM, VK_FORMAT_B8G8R8A8_SNORM, VK_FORMAT_B8G8R8A8_USCALED, VK_FORMAT_B8G8R8A8_SSCALED, VK_FORMAT_B8G8R8A8_UINT, VK_FORMAT_B8G8R8A8_SINT, VK_FORMAT_B8G8R8A8_SRGB, VK_FORMAT_A8B8G8R8_UNORM_PACK32, VK_FORMAT_A8B8G8R8_SNORM_PACK32, VK_FORMAT_A8B8G8R8_USCALED_PACK32, VK_FORMAT_A8B8G8R8_SSCALED_PACK32, VK_FORMAT_A8B8G8R8_UINT_PACK32, VK_FORMAT_A8B8G8R8_SINT_PACK32, VK_FORMAT_A8B8G8R8_SRGB_PACK32, VK_FORMAT_A2R10G10B10_UNORM_PACK32, VK_FORMAT_A2R10G10B10_SNORM_PACK32, VK_FORMAT_A2R10G10B10_USCALED_PACK32, VK_FORMAT_A2R10G10B10_SSCALED_PACK32, VK_FORMAT_A2R10G10B10_UINT_PACK32, VK_FORMAT_A2R10G10B10_SINT_PACK32, VK_FORMAT_A2B10G10R10_UNORM_PACK32, VK_FORMAT_A2B10G10R10_SNORM_PACK32, VK_FORMAT_A2B10G10R10_USCALED_PACK32, VK_FORMAT_A2B10G10R10_SSCALED_PACK32, VK_FORMAT_A2B10G10R10_UINT_PACK32, VK_FORMAT_A2B10G10R10_SINT_PACK32, VK_FORMAT_R16G16_UNORM, VK_FORMAT_R16G16_SNORM, VK_FORMAT_R16G16_USCALED, VK_FORMAT_R16G16_SSCALED, VK_FORMAT_R16G16_UINT, VK_FORMAT_R16G16_SINT, VK_FORMAT_R16G16_SFLOAT, VK_FORMAT_R32_UINT, VK_FORMAT_R32_SINT, VK_FORMAT_R32_SFLOAT, VK_FORMAT_B10G11R11_UFLOAT_PACK32, VK_FORMAT_E5B9G9R9_UFLOAT_PACK32

Class, Texel Block Size, Texel Block Extent, # Texels/Block	**Formats**
48-bit Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R16G16B16_UNORM, VK_FORMAT_R16G16B16_SNORM, VK_FORMAT_R16G16B16_USCALED, VK_FORMAT_R16G16B16_SSCALED, VK_FORMAT_R16G16B16_UINT, VK_FORMAT_R16G16B16_SINT, VK_FORMAT_R16G16B16_SFLOAT
64-bit Block size 8 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R16G16B16A16_UNORM, VK_FORMAT_R16G16B16A16_SNORM, VK_FORMAT_R16G16B16A16_USCALED, VK_FORMAT_R16G16B16A16_SSCALED, VK_FORMAT_R16G16B16A16_UINT, VK_FORMAT_R16G16B16A16_SINT, VK_FORMAT_R16G16B16A16_SFLOAT, VK_FORMAT_R32G32_UINT, VK_FORMAT_R32G32_SINT, VK_FORMAT_R32G32_SFLOAT, VK_FORMAT_R64_UINT, VK_FORMAT_R64_SINT, VK_FORMAT_R64_SFLOAT
96-bit Block size 12 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R32G32B32_UINT, VK_FORMAT_R32G32B32_SINT, VK_FORMAT_R32G32B32_SFLOAT
128-bit Block size 16 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R32G32B32A32_UINT, VK_FORMAT_R32G32B32A32_SINT, VK_FORMAT_R32G32B32A32_SFLOAT, VK_FORMAT_R64G64_UINT, VK_FORMAT_R64G64_SINT, VK_FORMAT_R64G64_SFLOAT
192-bit Block size 24 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R64G64B64_UINT, VK_FORMAT_R64G64B64_SINT, VK_FORMAT_R64G64B64_SFLOAT
256-bit Block size 32 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R64G64B64A64_UINT, VK_FORMAT_R64G64B64A64_SINT, VK_FORMAT_R64G64B64A64_SFLOAT

Class, Texel Block Size, Texel Block Extent, # Texels/Block	**Formats**
D16 Block size 2 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_D16_UNORM
D24 Block size 4 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_X8_D24_UNORM_PACK32
D32 Block size 4 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_D32_SFLOAT
S8 Block size 1 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_S8_UINT
D16S8 Block size 3 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_D16_UNORM_S8_UINT
D24S8 Block size 4 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_D24_UNORM_S8_UINT
D32S8 Block size 5 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_D32_SFLOAT_S8_UINT
BC1_RGB Block size 8 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_BC1_RGB_UNORM_BLOCK, VK_FORMAT_BC1_RGB_SRGB_BLOCK
BC1_RGBA Block size 8 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_BC1_RGBA_UNORM_BLOCK, VK_FORMAT_BC1_RGBA_SRGB_BLOCK
BC2 Block size 16 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_BC2_UNORM_BLOCK, VK_FORMAT_BC2_SRGB_BLOCK

Class, Texel Block Size, Texel Block Extent, # Texels/Block	**Formats**
BC3 Block size 16 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_BC3_UNORM_BLOCK, VK_FORMAT_BC3_SRGB_BLOCK
BC4 Block size 8 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_BC4_UNORM_BLOCK, VK_FORMAT_BC4_SNORM_BLOCK
BC5 Block size 16 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_BC5_UNORM_BLOCK, VK_FORMAT_BC5_SNORM_BLOCK
BC6H Block size 16 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_BC6H_UFLOAT_BLOCK, VK_FORMAT_BC6H_SFLOAT_BLOCK
BC7 Block size 16 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_BC7_UNORM_BLOCK, VK_FORMAT_BC7_SRGB_BLOCK
ETC2_RGB Block size 8 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_ETC2_R8G8B8_UNORM_BLOCK, VK_FORMAT_ETC2_R8G8B8_SRGB_BLOCK
ETC2_RGBA Block size 8 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_ETC2_R8G8B8A1_UNORM_BLOCK, VK_FORMAT_ETC2_R8G8B8A1_SRGB_BLOCK
ETC2_EAC_RGBA Block size 16 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_ETC2_R8G8B8A8_UNORM_BLOCK, VK_FORMAT_ETC2_R8G8B8A8_SRGB_BLOCK
EAC_R Block size 8 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_EAC_R11_UNORM_BLOCK, VK_FORMAT_EAC_R11_SNORM_BLOCK
EAC_RG Block size 16 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_EAC_R11G11_UNORM_BLOCK, VK_FORMAT_EAC_R11G11_SNORM_BLOCK

Class, Texel Block Size, Texel Block Extent, # Texels/Block	**Formats**
ASTC_4x4 Block size 16 byte 4x4x1 block extent 16 texel/block	VK_FORMAT_ASTC_4x4_UNORM_BLOCK, VK_FORMAT_ASTC_4x4_SRGB_BLOCK
ASTC_5x4 Block size 16 byte 5x4x1 block extent 20 texel/block	VK_FORMAT_ASTC_5x4_UNORM_BLOCK, VK_FORMAT_ASTC_5x4_SRGB_BLOCK
ASTC_5x5 Block size 16 byte 5x5x1 block extent 25 texel/block	VK_FORMAT_ASTC_5x5_UNORM_BLOCK, VK_FORMAT_ASTC_5x5_SRGB_BLOCK
ASTC_6x5 Block size 16 byte 6x5x1 block extent 30 texel/block	VK_FORMAT_ASTC_6x5_UNORM_BLOCK, VK_FORMAT_ASTC_6x5_SRGB_BLOCK
ASTC_6x6 Block size 16 byte 6x6x1 block extent 36 texel/block	VK_FORMAT_ASTC_6x6_UNORM_BLOCK, VK_FORMAT_ASTC_6x6_SRGB_BLOCK
ASTC_8x5 Block size 16 byte 8x5x1 block extent 40 texel/block	VK_FORMAT_ASTC_8x5_UNORM_BLOCK, VK_FORMAT_ASTC_8x5_SRGB_BLOCK
ASTC_8x6 Block size 16 byte 8x6x1 block extent 48 texel/block	VK_FORMAT_ASTC_8x6_UNORM_BLOCK, VK_FORMAT_ASTC_8x6_SRGB_BLOCK
ASTC_8x8 Block size 16 byte 8x8x1 block extent 64 texel/block	VK_FORMAT_ASTC_8x8_UNORM_BLOCK, VK_FORMAT_ASTC_8x8_SRGB_BLOCK
ASTC_10x5 Block size 16 byte 10x5x1 block extent 50 texel/block	VK_FORMAT_ASTC_10x5_UNORM_BLOCK, VK_FORMAT_ASTC_10x5_SRGB_BLOCK
ASTC_10x6 Block size 16 byte 10x6x1 block extent 60 texel/block	VK_FORMAT_ASTC_10x6_UNORM_BLOCK, VK_FORMAT_ASTC_10x6_SRGB_BLOCK

Class, Texel Block Size, Texel Block Extent, # Texels/Block	**Formats**
ASTC_10x8 Block size 16 byte 10x8x1 block extent 80 texel/block	VK_FORMAT_ASTC_10x8_UNORM_BLOCK, VK_FORMAT_ASTC_10x8_SRGB_BLOCK
ASTC_10x10 Block size 16 byte 10x10x1 block extent 100 texel/block	VK_FORMAT_ASTC_10x10_UNORM_BLOCK, VK_FORMAT_ASTC_10x10_SRGB_BLOCK
ASTC_12x10 Block size 16 byte 12x10x1 block extent 120 texel/block	VK_FORMAT_ASTC_12x10_UNORM_BLOCK, VK_FORMAT_ASTC_12x10_SRGB_BLOCK
ASTC_12x12 Block size 16 byte 12x12x1 block extent 144 texel/block	VK_FORMAT_ASTC_12x12_UNORM_BLOCK, VK_FORMAT_ASTC_12x12_SRGB_BLOCK
32-bit G8B8G8R8 Block size 4 byte 2x1x1 block extent 1 texel/block	VK_FORMAT_G8B8G8R8_422_UNORM
32-bit B8G8R8G8 Block size 4 byte 2x1x1 block extent 1 texel/block	VK_FORMAT_B8G8R8G8_422_UNORM
8-bit 3-plane 420 Block size 3 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM
8-bit 2-plane 420 Block size 3 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G8_B8R8_2PLANE_420_UNORM
8-bit 3-plane 422 Block size 3 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G8_B8_R8_3PLANE_422_UNORM
8-bit 2-plane 422 Block size 3 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G8_B8R8_2PLANE_422_UNORM

Class, Texel Block Size, Texel Block Extent, # Texels/Block	Formats
8-bit 3-plane 444 Block size 3 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G8_B8_R8_3PLANE_444_UNORM
64-bit R10G10B10A10 Block size 8 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R10X6G10X6B10X6A10X6_UNORM_4PACK16
64-bit G10B10G10R10 Block size 8 byte 2x1x1 block extent 1 texel/block	VK_FORMAT_G10X6B10X6G10X6R10X6_422_UNORM_4PACK16
64-bit B10G10R10G10 Block size 8 byte 2x1x1 block extent 1 texel/block	VK_FORMAT_B10X6G10X6R10X6G10X6_422_UNORM_4PACK16
10-bit 3-plane 420 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16
10-bit 2-plane 420 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16
10-bit 3-plane 422 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16
10-bit 2-plane 422 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16
10-bit 3-plane 444 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16
64-bit R12G12B12A12 Block size 8 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_R12X4G12X4B12X4A12X4_UNORM_4PACK16

Class, Texel Block Size, Texel Block Extent, # Texels/Block	Formats
64-bit G12B12G12R12 Block size 8 byte 2x1x1 block extent 1 texel/block	VK_FORMAT_G12X4B12X4G12X4R12X4_422_UNORM_4PACK16
64-bit B12G12R12G12 Block size 8 byte 2x1x1 block extent 1 texel/block	VK_FORMAT_B12X4G12X4R12X4G12X4_422_UNORM_4PACK16
12-bit 3-plane 420 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16
12-bit 2-plane 420 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16
12-bit 3-plane 422 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16
12-bit 2-plane 422 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16
12-bit 3-plane 444 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16
64-bit G16B16G16R16 Block size 8 byte 2x1x1 block extent 1 texel/block	VK_FORMAT_G16B16G16R16_422_UNORM
64-bit B16G16R16G16 Block size 8 byte 2x1x1 block extent 1 texel/block	VK_FORMAT_B16G16R16G16_422_UNORM
16-bit 3-plane 420 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G16_B16_R16_3PLANE_420_UNORM

Class, Texel Block Size, Texel Block Extent, # Texels/Block	Formats
16-bit 2-plane 420 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G16_B16R16_2PLANE_420_UNORM
16-bit 3-plane 422 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G16_B16_R16_3PLANE_422_UNORM
16-bit 2-plane 422 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G16_B16R16_2PLANE_422_UNORM
16-bit 3-plane 444 Block size 6 byte 1x1x1 block extent 1 texel/block	VK_FORMAT_G16_B16_R16_3PLANE_444_UNORM

Size Compatibility

Color formats with the same texel block size are considered *size-compatible* as long as neither or both are alpha formats (e.g., VK_FORMAT_A8_UNORM_KHR). If two size-compatible formats have different block extents (i.e. for compressed formats), then an image with size A × B × C in one format with a block extent of a × b × c can be represented as an image with size X × Y × Z in the other format with block extent x × y × z at the ratio between the block extents for each format, where

$$\lceil A/a \rceil = \lceil X/x \rceil$$

$$\lceil B/b \rceil = \lceil Y/y \rceil$$

$$\lceil C/c \rceil = \lceil Z/z \rceil$$

NOTE For example, a 7x3 image in the VK_FORMAT_ASTC_8x5_UNORM_BLOCK format can be represented as a 1x1 VK_FORMAT_R64G64_UINT image.

Images created with the VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT flag can have size-compatible views created from them to enable access via different size-compatible formats. Image views created in this way will be sized to match the expectations of the block extents noted above.

Copy operations are able to copy between size-compatible formats in different resources to enable manipulation of data in different formats. The extent used in these copy operations always matches

the source image, and is resized to the expectations of the block extents noted above for the destination image.

42.2. Format Properties

To query supported format features which are properties of the physical device, call:

```
// Provided by VK_VERSION_1_0
void vkGetPhysicalDeviceFormatProperties(
    VkPhysicalDevice                            physicalDevice,
    VkFormat                                    format,
    VkFormatProperties*                         pFormatProperties);
```

- `physicalDevice` is the physical device from which to query the format properties.
- `format` is the format whose properties are queried.
- `pFormatProperties` is a pointer to a VkFormatProperties structure in which physical device properties for `format` are returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceFormatProperties-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceFormatProperties-format-parameter
  `format` **must** be a valid VkFormat value
- VUID-vkGetPhysicalDeviceFormatProperties-pFormatProperties-parameter
  `pFormatProperties` **must** be a valid pointer to a VkFormatProperties structure

The `VkFormatProperties` structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkFormatProperties {
    VkFormatFeatureFlags    linearTilingFeatures;
    VkFormatFeatureFlags    optimalTilingFeatures;
    VkFormatFeatureFlags    bufferFeatures;
} VkFormatProperties;
```

- `linearTilingFeatures` is a bitmask of VkFormatFeatureFlagBits specifying features supported by images created with a `tiling` parameter of `VK_IMAGE_TILING_LINEAR`.
- `optimalTilingFeatures` is a bitmask of VkFormatFeatureFlagBits specifying features supported by images created with a `tiling` parameter of `VK_IMAGE_TILING_OPTIMAL`.
- `bufferFeatures` is a bitmask of VkFormatFeatureFlagBits specifying features supported by buffers.

NOTE	If no format feature flags are supported, the format itself is not supported, and images of that format cannot be created.

If `format` is a block-compressed format, then `bufferFeatures` **must** not support any features for the format.

If `format` is not a multi-plane format then `linearTilingFeatures` and `optimalTilingFeatures` **must** not contain `VK_FORMAT_FEATURE_DISJOINT_BIT`.

Bits which **can** be set in the VkFormatProperties features `linearTilingFeatures`, `optimalTilingFeatures`, VkDrmFormatModifierPropertiesEXT::`drmFormatModifierTilingFeatures`, and `bufferFeatures` are:

```
// Provided by VK_VERSION_1_0
typedef enum VkFormatFeatureFlagBits {
    VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT = 0x00000001,
    VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT = 0x00000002,
    VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT = 0x00000004,
    VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT = 0x00000008,
    VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT = 0x00000010,
    VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT = 0x00000020,
    VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT = 0x00000040,
    VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT = 0x00000080,
    VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT = 0x00000100,
    VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT = 0x00000200,
    VK_FORMAT_FEATURE_BLIT_SRC_BIT = 0x00000400,
    VK_FORMAT_FEATURE_BLIT_DST_BIT = 0x00000800,
    VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT = 0x00001000,
    VK_FORMAT_FEATURE_TRANSFER_SRC_BIT = 0x00004000,
    VK_FORMAT_FEATURE_TRANSFER_DST_BIT = 0x00008000,
    VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT = 0x00020000,
    VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT = 0x00040000,

VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT =
0x00080000,

VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT =
0x00100000,

VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEA
BLE_BIT = 0x00200000,
    VK_FORMAT_FEATURE_DISJOINT_BIT = 0x00400000,
    VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT = 0x00800000,
  // Provided by VK_KHR_video_decode_queue
    VK_FORMAT_FEATURE_VIDEO_DECODE_OUTPUT_BIT_KHR = 0x02000000,
  // Provided by VK_KHR_video_decode_queue
    VK_FORMAT_FEATURE_VIDEO_DECODE_DPB_BIT_KHR = 0x04000000,
  // Provided by VK_KHR_acceleration_structure
    VK_FORMAT_FEATURE_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR = 0x20000000,
  // Provided by VK_EXT_filter_cubic
```

```
    VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT = 0x00002000,
  // Provided by VK_EXT_fragment_density_map
    VK_FORMAT_FEATURE_FRAGMENT_DENSITY_MAP_BIT_EXT = 0x01000000,
  // Provided by VK_KHR_fragment_shading_rate
    VK_FORMAT_FEATURE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR = 0x40000000,
  // Provided by VK_KHR_video_encode_queue
    VK_FORMAT_FEATURE_VIDEO_ENCODE_INPUT_BIT_KHR = 0x08000000,
  // Provided by VK_KHR_video_encode_queue
    VK_FORMAT_FEATURE_VIDEO_ENCODE_DPB_BIT_KHR = 0x10000000,
  // Provided by VK_KHR_maintenance1
    VK_FORMAT_FEATURE_TRANSFER_SRC_BIT_KHR = VK_FORMAT_FEATURE_TRANSFER_SRC_BIT,
  // Provided by VK_KHR_maintenance1
    VK_FORMAT_FEATURE_TRANSFER_DST_BIT_KHR = VK_FORMAT_FEATURE_TRANSFER_DST_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT_KHR =
VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT_KHR =
VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion

VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT_KH
R =
VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion

VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT_KH
R =
VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion

VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEA
BLE_BIT_KHR =
VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEA
BLE_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_FEATURE_DISJOINT_BIT_KHR = VK_FORMAT_FEATURE_DISJOINT_BIT,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT_KHR =
VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT,
} VkFormatFeatureFlagBits;
```

These values all have the same meaning as the equivalently named values for VkFormatFeatureFlags2 and **may** be set in `linearTilingFeatures`, `optimalTilingFeatures`, and VkDrmFormatModifierPropertiesEXT::`drmFormatModifierTilingFeatures`, specifying that the features are supported by images or image views or sampler $Y'C_BC_R$ conversion objects created with the queried vkGetPhysicalDeviceFormatProperties::`format`:

- `VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT` specifies that an image view **can** be sampled from.
- `VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT` specifies that an image view **can** be used as a storage

image.

- VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT specifies that an image view **can** be used as storage image that supports atomic operations.
- VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT specifies that an image view **can** be used as a framebuffer color attachment and as an input attachment.
- VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT specifies that an image view **can** be used as a framebuffer color attachment that supports blending.
- VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT specifies that an image view **can** be used as a framebuffer depth/stencil attachment and as an input attachment.
- VK_FORMAT_FEATURE_BLIT_SRC_BIT specifies that an image **can** be used as srcImage for the vkCmdBlitImage2KHR and vkCmdBlitImage commands.
- VK_FORMAT_FEATURE_BLIT_DST_BIT specifies that an image **can** be used as dstImage for the vkCmdBlitImage2KHR and vkCmdBlitImage commands.
- VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT specifies that if VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT is also set, an image view **can** be used with a sampler that has either of magFilter or minFilter set to VK_FILTER_LINEAR, or mipmapMode set to VK_SAMPLER_MIPMAP_MODE_LINEAR. If VK_FORMAT_FEATURE_BLIT_SRC_BIT is also set, an image can be used as the srcImage to vkCmdBlitImage2KHR and vkCmdBlitImage with a filter of VK_FILTER_LINEAR. This bit **must** only be exposed for formats that also support the VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT or VK_FORMAT_FEATURE_BLIT_SRC_BIT.

  If the format being queried is a depth/stencil format, this bit only specifies that the depth aspect (not the stencil aspect) of an image of this format supports linear filtering, and that linear filtering of the depth aspect is supported whether depth compare is enabled in the sampler or not. Where depth comparison is supported it **may** be linear filtered whether this bit is present or not, but where this bit is not present the filtered value **may** be computed in an implementation-dependent manner which differs from the normal rules of linear filtering. The resulting value **must** be in the range [0,1] and **should** be proportional to, or a weighted average of, the number of comparison passes or failures.

- VK_FORMAT_FEATURE_TRANSFER_SRC_BIT specifies that an image **can** be used as a source image for copy commands. If the application apiVersion is Vulkan 1.0 and VK_KHR_maintenance1 is not supported, VK_FORMAT_FEATURE_TRANSFER_SRC_BIT is implied to be set when the format feature flag is not 0.
- VK_FORMAT_FEATURE_TRANSFER_DST_BIT specifies that an image **can** be used as a destination image for copy commands and clear commands. If the application apiVersion is Vulkan 1.0 and VK_KHR_maintenance1 is not supported, VK_FORMAT_FEATURE_TRANSFER_DST_BIT is implied to be set when the format feature flag is not 0.
- VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT specifies that VkImage **can** be used with a sampler that has either of magFilter or minFilter set to VK_FILTER_CUBIC_EXT, or be the source image for a blit with filter set to VK_FILTER_CUBIC_EXT. This bit **must** only be exposed for formats that also support the VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT. If the format being queried is a depth/stencil format, this only specifies that the depth aspect is cubic filterable.
- VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT specifies that an application **can** define a

sampler $Y'C_BC_R$ conversion using this format as a source, and that an image of this format **can** be used with a VkSamplerYcbcrConversionCreateInfo `xChromaOffset` and/or `yChromaOffset` of `VK_CHROMA_LOCATION_MIDPOINT`. Otherwise both `xChromaOffset` and `yChromaOffset` **must** be `VK_CHROMA_LOCATION_COSITED_EVEN`. If a format does not incorporate chroma downsampling (it is not a "422" or "420" format) but the implementation supports sampler $Y'C_BC_R$ conversion for this format, the implementation **must** set `VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT`.

- `VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT` specifies that an application **can** define a sampler $Y'C_BC_R$ conversion using this format as a source, and that an image of this format **can** be used with a VkSamplerYcbcrConversionCreateInfo `xChromaOffset` and/or `yChromaOffset` of `VK_CHROMA_LOCATION_COSITED_EVEN`. Otherwise both `xChromaOffset` and `yChromaOffset` **must** be `VK_CHROMA_LOCATION_MIDPOINT`. If neither `VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT` nor `VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT` is set, the application **must** not define a sampler $Y'C_BC_R$ conversion using this format as a source.
- `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT` specifies that an application **can** define a sampler $Y'C_BC_R$ conversion using this format as a source with `chromaFilter` set to `VK_FILTER_LINEAR`.
- `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT` specifies that the format can have different chroma, min, and mag filters.
- `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT` specifies that reconstruction is explicit, as described in Chroma Reconstruction. If this bit is not present, reconstruction is implicit by default.
- `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEABLE_BIT` specifies that reconstruction **can** be forcibly made explicit by setting VkSamplerYcbcrConversionCreateInfo::`forceExplicitReconstruction` to `VK_TRUE`. If the format being queried supports `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT` it **must** also support `VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEABLE_BIT`.
- `VK_FORMAT_FEATURE_DISJOINT_BIT` specifies that a multi-planar image **can** have the `VK_IMAGE_CREATE_DISJOINT_BIT` set during image creation. An implementation **must** not set `VK_FORMAT_FEATURE_DISJOINT_BIT` for *single-plane formats*.
- `VK_FORMAT_FEATURE_FRAGMENT_DENSITY_MAP_BIT_EXT` specifies that an image view **can** be used as a fragment density map attachment.
- `VK_FORMAT_FEATURE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR` specifies that an image view **can** be used as a fragment shading rate attachment. An implementation **must** not set this feature for formats with a numeric format other than `UINT`, or set it as a buffer feature.
- `VK_FORMAT_FEATURE_VIDEO_DECODE_OUTPUT_BIT_KHR` specifies that an image view with this format **can** be used as a decode output picture in video decode operations.
- `VK_FORMAT_FEATURE_VIDEO_DECODE_DPB_BIT_KHR` specifies that an image view with this format **can** be used as an output reconstructed picture or an input reference picture in video decode operations.
- `VK_FORMAT_FEATURE_VIDEO_ENCODE_INPUT_BIT_KHR` specifies that an image view with this format **can**

be used as an encode input picture in video encode operations.

- VK_FORMAT_FEATURE_VIDEO_ENCODE_DPB_BIT_KHR specifies that an image view with this format **can** be used as an output reconstructed picture or an input reference picture in video encode operations.

> **NOTE**
>
> Specific video profiles **may** have additional restrictions on the format and other image creation parameters corresponding to image views used by video coding operations that **can** be enumerated using the vkGetPhysicalDeviceVideoFormatPropertiesKHR command.

The following bits **may** be set in bufferFeatures, specifying that the features are supported by buffers or buffer views created with the queried vkGetPhysicalDeviceFormatProperties::format:

- VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT specifies that the format **can** be used to create a buffer view that **can** be bound to a VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER descriptor.
- VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT specifies that the format **can** be used to create a buffer view that **can** be bound to a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor.
- VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT specifies that atomic operations are supported on VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER with this format.
- VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT specifies that the format **can** be used as a vertex attribute format (VkVertexInputAttributeDescription::format).
- VK_FORMAT_FEATURE_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR specifies that the format **can** be used as the vertex format when creating an acceleration structure (VkAccelerationStructureGeometryTrianglesDataKHR::vertexFormat). This format **can** also be used as the vertex format in host memory when doing host acceleration structure builds.

> **NOTE**
>
> VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT and VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT are only intended to be advertised for single-component formats, since SPIR-V atomic operations require a scalar type.

```
// Provided by VK_VERSION_1_0
typedef VkFlags VkFormatFeatureFlags;
```

VkFormatFeatureFlags is a bitmask type for setting a mask of zero or more VkFormatFeatureFlagBits.

To query supported format features which are properties of the physical device, call:

```
// Provided by VK_KHR_get_physical_device_properties2
void vkGetPhysicalDeviceFormatProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    VkFormat                                    format,
    VkFormatProperties2*                        pFormatProperties);
```

- `physicalDevice` is the physical device from which to query the format properties.
- `format` is the format whose properties are queried.
- `pFormatProperties` is a pointer to a VkFormatProperties2 structure in which physical device properties for `format` are returned.

`vkGetPhysicalDeviceFormatProperties2KHR` behaves similarly to vkGetPhysicalDeviceFormatProperties, with the ability to return extended information in a `pNext` chain of output structures.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceFormatProperties2-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceFormatProperties2-format-parameter
  `format` **must** be a valid VkFormat value
- VUID-vkGetPhysicalDeviceFormatProperties2-pFormatProperties-parameter
  `pFormatProperties` **must** be a valid pointer to a VkFormatProperties2 structure

The `VkFormatProperties2` structure is defined as:

```
typedef struct VkFormatProperties2 {
    VkStructureType       sType;
    void*                 pNext;
    VkFormatProperties    formatProperties;
} VkFormatProperties2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkFormatProperties2 VkFormatProperties2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `formatProperties` is a VkFormatProperties structure describing features supported by the requested format.

Valid Usage (Implicit)

- VUID-VkFormatProperties2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_2`
- VUID-VkFormatProperties2-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkDrmFormatModifierPropertiesList2EXT,

VkDrmFormatModifierPropertiesListEXT, or VkFormatProperties3

- VUID-VkFormatProperties2-sType-unique
  The sType value of each struct in the pNext chain **must** be unique

To obtain the list of Linux DRM format modifiers compatible with a VkFormat, add a VkDrmFormatModifierPropertiesListEXT structure to the pNext chain of VkFormatProperties2.

The VkDrmFormatModifierPropertiesListEXT structure is defined as:

```
// Provided by VK_EXT_image_drm_format_modifier
typedef struct VkDrmFormatModifierPropertiesListEXT {
    VkStructureType                        sType;
    void*                                  pNext;
    uint32_t                               drmFormatModifierCount;
    VkDrmFormatModifierPropertiesEXT*      pDrmFormatModifierProperties;
} VkDrmFormatModifierPropertiesListEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- drmFormatModifierCount is an inout parameter related to the number of modifiers compatible with the format, as described below.
- pDrmFormatModifierProperties is either NULL or a pointer to an array of VkDrmFormatModifierPropertiesEXT structures.

If pDrmFormatModifierProperties is NULL, then the function returns in drmFormatModifierCount the number of modifiers compatible with the queried format. Otherwise, the application **must** set drmFormatModifierCount to the length of the array pDrmFormatModifierProperties; the function will write at most drmFormatModifierCount elements to the array, and will return in drmFormatModifierCount the number of elements written.

Among the elements in array pDrmFormatModifierProperties, each returned drmFormatModifier **must** be unique.

Valid Usage (Implicit)

- VUID-VkDrmFormatModifierPropertiesListEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DRM_FORMAT_MODIFIER_PROPERTIES_LIST_EXT

The VkDrmFormatModifierPropertiesEXT structure describes properties of a VkFormat when that format is combined with a Linux DRM format modifier. These properties, like those of VkFormatProperties2, are independent of any particular image.

The VkDrmFormatModifierPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_image_drm_format_modifier
typedef struct VkDrmFormatModifierPropertiesEXT {
    uint64_t                drmFormatModifier;
    uint32_t                drmFormatModifierPlaneCount;
    VkFormatFeatureFlags    drmFormatModifierTilingFeatures;
} VkDrmFormatModifierPropertiesEXT;
```

- drmFormatModifier is a *Linux DRM format modifier*.
- drmFormatModifierPlaneCount is the number of *memory planes* in any image created with format and drmFormatModifier. An image's *memory planecount* is distinct from its *format planecount*, as explained below.
- drmFormatModifierTilingFeatures is a bitmask of VkFormatFeatureFlagBits that are supported by any image created with format and drmFormatModifier.

The returned drmFormatModifierTilingFeatures **must** contain at least one bit.

The implementation **must** not return DRM_FORMAT_MOD_INVALID in drmFormatModifier.

An image's *memory planecount* (as returned by drmFormatModifierPlaneCount) is distinct from its *format planecount* (in the sense of multi-planar $Y'C_BC_R$ formats). In VkImageAspectFlags, each VK_IMAGE_ASPECT_MEMORY_PLANE_i_BIT_EXT represents a *memory plane* and each VK_IMAGE_ASPECT_PLANE_i_BIT a *format plane*.

An image's set of *format planes* is an ordered partition of the image's **content** into separable groups of format components. The ordered partition is encoded in the name of each VkFormat. For example, VK_FORMAT_G8_B8R8_2PLANE_420_UNORM contains two *format planes*; the first plane contains the green component and the second plane contains the blue component and red component. If the format name does not contain PLANE, then the format contains a single plane; for example, VK_FORMAT_R8G8B8A8_UNORM. Some commands, such as vkCmdCopyBufferToImage, do not operate on all format components in the image, but instead operate only on the *format planes* explicitly chosen by the application and operate on each *format plane* independently.

An image's set of *memory planes* is an ordered partition of the image's **memory** rather than the image's **content**. Each *memory plane* is a contiguous range of memory. The union of an image's *memory planes* is not necessarily contiguous.

If an image is linear, then the partition is the same for *memory planes* and for *format planes*. Therefore, if the returned drmFormatModifier is DRM_FORMAT_MOD_LINEAR, then drmFormatModifierPlaneCount **must** equal the *format planecount*, and drmFormatModifierTilingFeatures **must** be identical to the VkFormatProperties2 ::linearTilingFeatures returned in the same pNext chain.

If an image is non-linear, then the partition of the image's **memory** into *memory planes* is implementation-specific and **may** be unrelated to the partition of the image's **content** into *format planes*. For example, consider an image whose format is VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM, tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, whose drmFormatModifier is not DRM_FORMAT_MOD_LINEAR, and flags lacks VK_IMAGE_CREATE_DISJOINT_BIT. The image has 3 *format planes*, and commands such vkCmdCopyBufferToImage act on each *format plane* independently as

if the data of each *format plane* were separable from the data of the other planes. In a straightforward implementation, the implementation **may** store the image's content in 3 adjacent *memory planes* where each *memory plane* corresponds exactly to a *format plane*. However, the implementation **may** also store the image's content in a single *memory plane* where all format components are combined using an implementation-private block-compressed format; or the implementation **may** store the image's content in a collection of 7 adjacent *memory planes* using an implementation-private sharding technique. Because the image is non-linear and non-disjoint, the implementation has much freedom when choosing the image's placement in memory.

The *memory planecount* applies to function parameters and structures only when the API specifies an explicit requirement on `drmFormatModifierPlaneCount`. In all other cases, the *memory planecount* is ignored.

The list of Linux DRM format modifiers compatible with a VkFormat **can** be obtained by adding a VkDrmFormatModifierPropertiesList2EXT structure to the `pNext` chain of VkFormatProperties2.

The VkDrmFormatModifierPropertiesList2EXT structure is defined as:

```
// Provided by VK_EXT_image_drm_format_modifier with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
typedef struct VkDrmFormatModifierPropertiesList2EXT {
    VkStructureType                        sType;
    void*                                  pNext;
    uint32_t                               drmFormatModifierCount;
    VkDrmFormatModifierProperties2EXT*     pDrmFormatModifierProperties;
} VkDrmFormatModifierPropertiesList2EXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `drmFormatModifierCount` is an inout parameter related to the number of modifiers compatible with the `format`, as described below.
- `pDrmFormatModifierProperties` is either `NULL` or a pointer to an array of VkDrmFormatModifierProperties2EXT structures.

If `pDrmFormatModifierProperties` is `NULL`, the number of modifiers compatible with the queried `format` is returned in `drmFormatModifierCount`. Otherwise, the application **must** set `drmFormatModifierCount` to the length of the array `pDrmFormatModifierProperties`; the function will write at most `drmFormatModifierCount` elements to the array, and will return in `drmFormatModifierCount` the number of elements written.

Among the elements in array `pDrmFormatModifierProperties`, each returned `drmFormatModifier` **must** be unique.

Among the elements in array `pDrmFormatModifierProperties`, the bits reported in `drmFormatModifierTilingFeatures` **must** include the bits reported in the corresponding element of `VkDrmFormatModifierPropertiesListEXT`::`pDrmFormatModifierProperties`.

Valid Usage (Implicit)

- VUID-VkDrmFormatModifierPropertiesList2EXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DRM_FORMAT_MODIFIER_PROPERTIES_LIST_2_EXT`

The VkDrmFormatModifierProperties2EXT structure describes properties of a VkFormat when that format is combined with a Linux DRM format modifier. These properties, like those of VkFormatProperties2, are independent of any particular image.

The VkDrmFormatModifierPropertiesEXT structure is defined as:

```
// Provided by VK_EXT_image_drm_format_modifier with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
typedef struct VkDrmFormatModifierProperties2EXT {
    uint64_t                drmFormatModifier;
    uint32_t                drmFormatModifierPlaneCount;
    VkFormatFeatureFlags2   drmFormatModifierTilingFeatures;
} VkDrmFormatModifierProperties2EXT;
```

- `drmFormatModifier` is a *Linux DRM format modifier*.
- `drmFormatModifierPlaneCount` is the number of *memory planes* in any image created with `format` and `drmFormatModifier`. An image's *memory planecount* is distinct from its *format planecount*, as explained below.
- `drmFormatModifierTilingFeatures` is a bitmask of VkFormatFeatureFlagBits2 that are supported by any image created with `format` and `drmFormatModifier`.

To query supported format extended features which are properties of the physical device, add VkFormatProperties3 structure to the `pNext` chain of VkFormatProperties2.

The VkFormatProperties3 structure is defined as:

```
typedef struct VkFormatProperties3 {
    VkStructureType         sType;
    void*                   pNext;
    VkFormatFeatureFlags2   linearTilingFeatures;
    VkFormatFeatureFlags2   optimalTilingFeatures;
    VkFormatFeatureFlags2   bufferFeatures;
} VkFormatProperties3;
```

or the equivalent

```
// Provided by VK_KHR_format_feature_flags2
typedef VkFormatProperties3 VkFormatProperties3KHR;
```

- `linearTilingFeatures` is a bitmask of VkFormatFeatureFlagBits2 specifying features supported

by images created with a `tiling` parameter of `VK_IMAGE_TILING_LINEAR`.

- `optimalTilingFeatures` is a bitmask of VkFormatFeatureFlagBits2 specifying features supported by images created with a `tiling` parameter of `VK_IMAGE_TILING_OPTIMAL`.
- `bufferFeatures` is a bitmask of VkFormatFeatureFlagBits2 specifying features supported by buffers.

The bits reported in `linearTilingFeatures`, `optimalTilingFeatures` and `bufferFeatures` **must** include the bits reported in the corresponding fields of `VkFormatProperties2`::`formatProperties`.

Valid Usage (Implicit)

- VUID-VkFormatProperties3-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_3`

Bits which **can** be set in the VkFormatProperties3 features `linearTilingFeatures`, `optimalTilingFeatures`, and `bufferFeatures` are:

```
// Flag bits for VkFormatFeatureFlagBits2
typedef VkFlags64 VkFormatFeatureFlagBits2;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_BIT =
0x00000001ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_BIT_KHR =
0x00000001ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_STORAGE_IMAGE_BIT =
0x00000002ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_STORAGE_IMAGE_BIT_KHR =
0x00000002ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_STORAGE_IMAGE_ATOMIC_BIT =
0x00000004ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_STORAGE_IMAGE_ATOMIC_BIT_KHR
= 0x00000004ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_UNIFORM_TEXEL_BUFFER_BIT =
0x00000008ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_UNIFORM_TEXEL_BUFFER_BIT_KHR
= 0x00000008ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_STORAGE_TEXEL_BUFFER_BIT =
0x00000010ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_STORAGE_TEXEL_BUFFER_BIT_KHR
= 0x00000010ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_STORAGE_TEXEL_BUFFER_ATOMIC_BIT = 0x00000020ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_STORAGE_TEXEL_BUFFER_ATOMIC_BIT_KHR = 0x00000020ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_VERTEX_BUFFER_BIT =
0x00000040ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_VERTEX_BUFFER_BIT_KHR =
0x00000040ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_COLOR_ATTACHMENT_BIT =
```

```
0x00000080ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_COLOR_ATTACHMENT_BIT_KHR =
0x00000080ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_COLOR_ATTACHMENT_BLEND_BIT =
0x00000100ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_COLOR_ATTACHMENT_BLEND_BIT_KHR = 0x00000100ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_DEPTH_STENCIL_ATTACHMENT_BIT
= 0x00000200ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_DEPTH_STENCIL_ATTACHMENT_BIT_KHR = 0x00000200ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_BLIT_SRC_BIT =
0x00004400ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_BLIT_SRC_BIT_KHR =
0x00004400ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_BLIT_DST_BIT =
0x00008800ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_BLIT_DST_BIT_KHR =
0x00008800ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_FILTER_LINEAR_BIT = 0x00001000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_FILTER_LINEAR_BIT_KHR = 0x00001000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_FILTER_CUBIC_BIT = 0x00002000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT = 0x00002000ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_TRANSFER_SRC_BIT =
0x00004000ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_TRANSFER_SRC_BIT_KHR =
0x00004000ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_TRANSFER_DST_BIT =
0x00008000ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_TRANSFER_DST_BIT_KHR =
0x00008000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_FILTER_MINMAX_BIT = 0x00010000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_FILTER_MINMAX_BIT_KHR = 0x00010000ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_MIDPOINT_CHROMA_SAMPLES_BIT
= 0x00020000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_MIDPOINT_CHROMA_SAMPLES_BIT_KHR = 0x00020000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT = 0x00040000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT_KHR =
0x00040000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT
= 0x00080000ULL;
```

```
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT_
KHR = 0x00080000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT
= 0x00100000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT_
KHR = 0x00100000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORC
EABLE_BIT = 0x00200000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORC
EABLE_BIT_KHR = 0x00200000ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_DISJOINT_BIT =
0x00400000ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_DISJOINT_BIT_KHR =
0x00400000ULL;
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_COSITED_CHROMA_SAMPLES_BIT =
0x00800000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_COSITED_CHROMA_SAMPLES_BIT_KHR = 0x00800000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT = 0x80000000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT_KHR = 0x80000000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT = 0x100000000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT_KHR = 0x100000000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT = 0x200000000ULL;
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT_KHR = 0x200000000ULL;
// Provided by VK_KHR_video_decode_queue with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_VIDEO_DECODE_OUTPUT_BIT_KHR
= 0x02000000ULL;
// Provided by VK_KHR_video_decode_queue with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_VIDEO_DECODE_DPB_BIT_KHR =
0x04000000ULL;
// Provided by VK_KHR_acceleration_structure with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR = 0x20000000ULL;
// Provided by VK_EXT_fragment_density_map with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_FRAGMENT_DENSITY_MAP_BIT_EXT
= 0x01000000ULL;
```

```
// Provided by VK_KHR_fragment_shading_rate with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR = 0x40000000ULL;
// Provided by VK_EXT_host_image_copy
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_HOST_IMAGE_TRANSFER_BIT_EXT
= 0x400000000000ULL;
// Provided by VK_KHR_video_encode_queue with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_VIDEO_ENCODE_INPUT_BIT_KHR =
0x08000000ULL;
// Provided by VK_KHR_video_encode_queue with VK_KHR_format_feature_flags2 or
VK_VERSION_1_3
static const VkFormatFeatureFlagBits2 VK_FORMAT_FEATURE_2_VIDEO_ENCODE_DPB_BIT_KHR =
0x10000000ULL;
// Provided by VK_KHR_video_encode_quantization_map
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR = 0x2000000000000ULL;
// Provided by VK_KHR_video_encode_quantization_map
static const VkFormatFeatureFlagBits2
VK_FORMAT_FEATURE_2_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR = 0x4000000000000ULL;
```

or the equivalent

```
// Provided by VK_KHR_format_feature_flags2
typedef VkFormatFeatureFlagBits2 VkFormatFeatureFlagBits2KHR;
```

The following bits **may** be set in `linearTilingFeatures` and `optimalTilingFeatures`, specifying that the features are supported by images or image views or sampler $Y'C_BC_R$ conversion objects created with the queried vkGetPhysicalDeviceFormatProperties2KHR::`format`:

- `VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_BIT` specifies that an image view **can** be sampled from.
- `VK_FORMAT_FEATURE_2_STORAGE_IMAGE_BIT` specifies that an image view **can** be used as a storage image.
- `VK_FORMAT_FEATURE_2_STORAGE_IMAGE_ATOMIC_BIT` specifies that an image view **can** be used as storage image that supports atomic operations.
- `VK_FORMAT_FEATURE_2_COLOR_ATTACHMENT_BIT` specifies that an image view **can** be used as a framebuffer color attachment and as an input attachment.
- `VK_FORMAT_FEATURE_2_COLOR_ATTACHMENT_BLEND_BIT` specifies that an image view **can** be used as a framebuffer color attachment that supports blending.
- `VK_FORMAT_FEATURE_2_DEPTH_STENCIL_ATTACHMENT_BIT` specifies that an image view **can** be used as a framebuffer depth/stencil attachment and as an input attachment.
- `VK_FORMAT_FEATURE_2_BLIT_SRC_BIT` specifies that an image **can** be used as the `srcImage` for vkCmdBlitImage2KHR and vkCmdBlitImage.
- `VK_FORMAT_FEATURE_2_BLIT_DST_BIT` specifies that an image **can** be used as the `dstImage` for

vkCmdBlitImage2KHR and vkCmdBlitImage.

- VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_FILTER_LINEAR_BIT specifies that if VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_BIT is also set, an image view **can** be used with a sampler that has either of magFilter or minFilter set to VK_FILTER_LINEAR, or mipmapMode set to VK_SAMPLER_MIPMAP_MODE_LINEAR. If VK_FORMAT_FEATURE_2_BLIT_SRC_BIT is also set, an image can be used as the srcImage for vkCmdBlitImage2KHR and vkCmdBlitImage with a filter of VK_FILTER_LINEAR. This bit **must** only be exposed for formats that also support the VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_BIT or VK_FORMAT_FEATURE_2_BLIT_SRC_BIT.

  If the format being queried is a depth/stencil format, this bit only specifies that the depth aspect (not the stencil aspect) of an image of this format supports linear filtering. Where depth comparison is supported it **may** be linear filtered whether this bit is present or not, but where this bit is not present the filtered value **may** be computed in an implementation-dependent manner which differs from the normal rules of linear filtering. The resulting value **must** be in the range [0,1] and **should** be proportional to, or a weighted average of, the number of comparison passes or failures.

- VK_FORMAT_FEATURE_2_TRANSFER_SRC_BIT specifies that an image **can** be used as a source image for copy commands.
- VK_FORMAT_FEATURE_2_TRANSFER_DST_BIT specifies that an image **can** be used as a destination image for copy commands and clear commands.
- VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_FILTER_CUBIC_BIT specifies that VkImage **can** be used with a sampler that has either of magFilter or minFilter set to VK_FILTER_CUBIC_EXT, or be the source image for a blit with filter set to VK_FILTER_CUBIC_EXT. This bit **must** only be exposed for formats that also support the VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_BIT. If the format being queried is a depth/stencil format, this only specifies that the depth aspect is cubic filterable.
- VK_FORMAT_FEATURE_2_MIDPOINT_CHROMA_SAMPLES_BIT specifies that an application **can** define a sampler $Y'C_BC_R$ conversion using this format as a source, and that an image of this format **can** be used with a VkSamplerYcbcrConversionCreateInfo xChromaOffset and/or yChromaOffset of VK_CHROMA_LOCATION_MIDPOINT. Otherwise both xChromaOffset and yChromaOffset **must** be VK_CHROMA_LOCATION_COSITED_EVEN. If a format does not incorporate chroma downsampling (it is not a “422” or “420” format) but the implementation supports sampler $Y'C_BC_R$ conversion for this format, the implementation **must** set VK_FORMAT_FEATURE_2_MIDPOINT_CHROMA_SAMPLES_BIT.
- VK_FORMAT_FEATURE_2_COSITED_CHROMA_SAMPLES_BIT specifies that an application **can** define a sampler $Y'C_BC_R$ conversion using this format as a source, and that an image of this format **can** be used with a VkSamplerYcbcrConversionCreateInfo xChromaOffset and/or yChromaOffset of VK_CHROMA_LOCATION_COSITED_EVEN. Otherwise both xChromaOffset and yChromaOffset **must** be VK_CHROMA_LOCATION_MIDPOINT. If neither VK_FORMAT_FEATURE_2_COSITED_CHROMA_SAMPLES_BIT nor VK_FORMAT_FEATURE_2_MIDPOINT_CHROMA_SAMPLES_BIT is set, the application **must** not define a sampler $Y'C_BC_R$ conversion using this format as a source.
- VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT specifies that an application **can** define a sampler $Y'C_BC_R$ conversion using this format as a source with chromaFilter set to VK_FILTER_LINEAR.
- VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT specifies that the format can have different chroma, min, and mag filters.

- VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT specifies that reconstruction is explicit, as described in Chroma Reconstruction. If this bit is not present, reconstruction is implicit by default.
- VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEABLE_BIT specifies that reconstruction **can** be forcibly made explicit by setting VkSamplerYcbcrConversionCreateInfo::forceExplicitReconstruction to VK_TRUE. If the format being queried supports VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT it **must** also support VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEABLE_BIT.
- VK_FORMAT_FEATURE_2_DISJOINT_BIT specifies that a multi-planar image **can** have the VK_IMAGE_CREATE_DISJOINT_BIT set during image creation. An implementation **must** not set VK_FORMAT_FEATURE_2_DISJOINT_BIT for *single-plane formats*.
- VK_FORMAT_FEATURE_2_FRAGMENT_DENSITY_MAP_BIT_EXT specifies that an image view **can** be used as a fragment density map attachment.
- VK_FORMAT_FEATURE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR specifies that an image view **can** be used as a fragment shading rate attachment. An implementation **must** not set this feature for formats with a numeric format other than UINT, or set it as a buffer feature.
- VK_FORMAT_FEATURE_2_VIDEO_DECODE_OUTPUT_BIT_KHR specifies that an image view with this format **can** be used as a decode output picture in video decode operations.
- VK_FORMAT_FEATURE_2_VIDEO_DECODE_DPB_BIT_KHR specifies that an image view with this format **can** be used as an output reconstructed picture or an input reference picture in video decode operations.
- VK_FORMAT_FEATURE_2_VIDEO_ENCODE_INPUT_BIT_KHR specifies that an image view with this format **can** be used as an encode input picture in video encode operations.
- VK_FORMAT_FEATURE_2_VIDEO_ENCODE_DPB_BIT_KHR specifies that an image view with this format **can** be used as an output reconstructed picture or an input reference picture in video encode operations.

> **NOTE** Specific video profiles **may** have additional restrictions on the format and other image creation parameters corresponding to image views used by video coding operations that **can** be enumerated using the vkGetPhysicalDeviceVideoFormatPropertiesKHR command.

- VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT specifies that image views or buffer views created with this format **can** be used as storage images or storage texel buffers respectively for read operations without specifying a format.
- VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT specifies that image views or buffer views created with this format **can** be used as storage images or storage texel buffers respectively for write operations without specifying a format.
- VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT specifies that image views created with this format **can** be used for depth comparison performed by OpImage*Dref* instructions.

- VK_FORMAT_FEATURE_2_HOST_IMAGE_TRANSFER_BIT_EXT specifies that an image **can** be created with VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT.
- VK_FORMAT_FEATURE_2_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR specifies that an image view with this format **can** be used as a quantization delta map in video encode operations.
- VK_FORMAT_FEATURE_2_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR specifies that an image view with this format **can** be used as an emphasis map in video encode operations.

NOTE	Video encode quantization maps have additional restrictions specific to the video profile they are used with that **can** be enumerated using the vkGetPhysicalDeviceVideoFormatPropertiesKHR command.

The following bits **may** be set in bufferFeatures, specifying that the features are supported by buffers or buffer views created with the queried vkGetPhysicalDeviceFormatProperties2KHR ::format:

- VK_FORMAT_FEATURE_2_UNIFORM_TEXEL_BUFFER_BIT specifies that the format **can** be used to create a buffer view that **can** be bound to a VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER descriptor.
- VK_FORMAT_FEATURE_2_STORAGE_TEXEL_BUFFER_BIT specifies that the format **can** be used to create a buffer view that **can** be bound to a VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER descriptor.
- VK_FORMAT_FEATURE_2_STORAGE_TEXEL_BUFFER_ATOMIC_BIT specifies that atomic operations are supported on VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER with this format.
- VK_FORMAT_FEATURE_2_VERTEX_BUFFER_BIT specifies that the format **can** be used as a vertex attribute format (VkVertexInputAttributeDescription::format).
- VK_FORMAT_FEATURE_2_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR specifies that the format **can** be used as the vertex format when creating an acceleration structure (VkAccelerationStructureGeometryTrianglesDataKHR::vertexFormat). This format **can** also be used as the vertex format in host memory when doing host acceleration structure builds.
- VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT specifies that buffer views created with this format **can** be used as storage texel buffers for read operations without specifying a format.
- VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT specifies that buffer views created with this format **can** be used as storage texel buffers for write operations without specifying a format.

```
typedef VkFlags64 VkFormatFeatureFlags2;
```

or the equivalent

```
// Provided by VK_KHR_format_feature_flags2
typedef VkFormatFeatureFlags2 VkFormatFeatureFlags2KHR;
```

VkFormatFeatureFlags2 is a bitmask type for setting a mask of zero or more VkFormatFeatureFlagBits2.

42.2.1. Potential Format Features

Some valid usage conditions depend on the format features supported by a VkImage whose VkImageTiling is unknown. In such cases the exact VkFormatFeatureFlagBits supported by the VkImage cannot be determined, so the valid usage conditions are expressed in terms of the *potential format features* of the VkImage format.

The *potential format features* of a VkFormat are defined as follows:

- The union of VkFormatFeatureFlagBits and VkFormatFeatureFlagBits2, supported when the VkImageTiling is `VK_IMAGE_TILING_OPTIMAL`, `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`, or `VK_IMAGE_TILING_LINEAR`

42.3. Required Format Support

Implementations **must** support at least the following set of features on the listed formats. For images, these features **must** be supported for every VkImageType (including arrayed and cube variants) unless otherwise noted. These features are supported on existing formats without needing to advertise an extension or needing to explicitly enable them. Support for additional functionality beyond the requirements listed here is queried using the vkGetPhysicalDeviceFormatProperties command.

NOTE Unless otherwise excluded below, the required formats are supported for all VkImageCreateFlags values as long as those flag values are otherwise allowed.

The following tables show which feature bits **must** be supported for each format. Formats that are required to support `VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT` **must** also support `VK_FORMAT_FEATURE_TRANSFER_SRC_BIT` and `VK_FORMAT_FEATURE_TRANSFER_DST_BIT`.

Table 71. Key for Format Feature Tables

□	This feature **must** be supported on the named format
†	This feature **must** be supported on at least some of the named formats, with more information in the table where the symbol appears
‡	This feature **must** be supported with some caveats or preconditions, with more information in the table where the symbol appears
§	This feature **must** be supported with some caveats or preconditions, with more information in the table where the symbol appears

Table 72. Feature Bits in `optimalTilingFeatures`

`VK_FORMAT_FEATURE_TRANSFER_SRC_BIT`
`VK_FORMAT_FEATURE_TRANSFER_DST_BIT`
`VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT`
`VK_FORMAT_FEATURE_BLIT_SRC_BIT`

VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT
VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT
VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT
VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT
VK_FORMAT_FEATURE_BLIT_DST_BIT
VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT
VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT

Table 73. Feature Bits in `bufferFeatures`

VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT
VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT
VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT
VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT

Table 74. Mandatory Format Support: Sub-Byte Components

VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT													
VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT												✓	✓
VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT											✓	✓	✓
VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT										✓	✓	✓	✓
VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT									✓	✓	✓	✓	✓
VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT								✓	✓	✓	✓	✓	✓
VK_FORMAT_FEATURE_BLIT_DST_BIT							✓	✓	✓	✓	✓	✓	✓
VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT						✓	✓	✓	✓	✓	✓	✓	✓
VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT					✓	✓	✓	✓	✓	✓	✓	✓	✓
VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT				✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT			✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
VK_FORMAT_FEATURE_BLIT_SRC_BIT		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Format													
VK_FORMAT_UNDEFINED													
VK_FORMAT_R4G4_UNORM_PACK8													
VK_FORMAT_R4G4B4A4_UNORM_PACK16													
VK_FORMAT_B4G4R4A4_UNORM_PACK16	✓	✓	✓										
VK_FORMAT_R5G6B5_UNORM_PACK16	✓	✓	✓			✓	✓	✓					
VK_FORMAT_B5G6R5_UNORM_PACK16													
VK_FORMAT_R5G5B5A1_UNORM_PACK16													
VK_FORMAT_B5G5R5A1_UNORM_PACK16													
VK_FORMAT_A1R5G5B5_UNORM_PACK16	✓	✓	✓			✓	✓	✓					
VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR													

Table 75. Mandatory Format Support: 1-3 Byte-Sized Components

Format	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT	
VK_FORMAT_R8_UNORM	◻	◻	◻	◻‡		◻	◻	◻		◻	◻			
VK_FORMAT_R8_SNORM	◻	◻	◻	◻‡						◻	◻			
VK_FORMAT_R8_USCALED														
VK_FORMAT_R8_SSCALED														
VK_FORMAT_R8_UINT	◻	◻		◻‡		◻	◻			◻	◻			
VK_FORMAT_R8_SINT	◻	◻		◻‡		◻	◻			◻	◻			
VK_FORMAT_R8_SRGB														
VK_FORMAT_R8G8_UNORM	◻	◻	◻	◻‡		◻	◻	◻		◻	◻			
VK_FORMAT_R8G8_SNORM	◻	◻	◻	◻‡						◻	◻			
VK_FORMAT_R8G8_USCALED														
VK_FORMAT_R8G8_SSCALED														
VK_FORMAT_R8G8_UINT	◻	◻		◻‡		◻	◻			◻	◻			
VK_FORMAT_R8G8_SINT	◻	◻		◻‡		◻	◻			◻	◻			
VK_FORMAT_R8G8_SRGB														
VK_FORMAT_R8G8B8_UNORM														
VK_FORMAT_R8G8B8_SNORM														
VK_FORMAT_R8G8B8_USCALED														
VK_FORMAT_R8G8B8_SSCALED														
VK_FORMAT_R8G8B8_UINT														
VK_FORMAT_R8G8B8_SINT														
VK_FORMAT_R8G8B8_SRGB														
VK_FORMAT_B8G8R8_UNORM														
VK_FORMAT_B8G8R8_SNORM														

VK_FORMAT_B8G8R8_USCALED													
VK_FORMAT_B8G8R8_SSCALED													
VK_FORMAT_B8G8R8_UINT													
VK_FORMAT_B8G8R8_SINT													
VK_FORMAT_B8G8R8_SRGB													
VK_FORMAT_A8_UNORM_KHR													
Format features marked with ‡ **must** be supported for optimalTilingFeatures if the VkPhysicalDevice supports the shaderStorageImageExtendedFormats feature.													

Table 76. Mandatory Format Support: 4 Byte-Sized Components

Format	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT	
VK_FORMAT_R8G8B8A8_UNORM	✓	✓	✓	✓		✓	✓	✓		✓	✓	✓		
VK_FORMAT_R8G8B8A8_SNORM	✓	✓	✓	✓						✓	✓	✓		
VK_FORMAT_R8G8B8A8_USCALED														
VK_FORMAT_R8G8B8A8_SSCALED														
VK_FORMAT_R8G8B8A8_UINT	✓	✓		✓		✓	✓			✓	✓	✓		
VK_FORMAT_R8G8B8A8_SINT	✓	✓		✓		✓	✓			✓	✓	✓		
VK_FORMAT_R8G8B8A8_SRGB	✓	✓	✓			✓	✓	✓						
VK_FORMAT_B8G8R8A8_UNORM	✓	✓	✓			✓	✓	✓		✓	✓			
VK_FORMAT_B8G8R8A8_SNORM														
VK_FORMAT_B8G8R8A8_USCALED														
VK_FORMAT_B8G8R8A8_SSCALED														
VK_FORMAT_B8G8R8A8_UINT														
VK_FORMAT_B8G8R8A8_SINT														
VK_FORMAT_B8G8R8A8_SRGB	✓	✓	✓			✓	✓	✓						
VK_FORMAT_A8B8G8R8_UNORM_PACK32	✓	✓	✓			✓	✓	✓		✓	✓	✓		
VK_FORMAT_A8B8G8R8_SNORM_PACK32	✓	✓	✓							✓	✓	✓		
VK_FORMAT_A8B8G8R8_USCALED_PACK32														
VK_FORMAT_A8B8G8R8_SSCALED_PACK32														
VK_FORMAT_A8B8G8R8_UINT_PACK32	✓	✓				✓	✓			✓	✓	✓		
VK_FORMAT_A8B8G8R8_SINT_PACK32	✓	✓				✓	✓			✓	✓	✓		
VK_FORMAT_A8B8G8R8_SRGB_PACK32	✓	✓	✓			✓	✓	✓						

Table 77. Mandatory Format Support: 10- and 12-Bit Components

VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT	
Format													
VK_FORMAT_A2R10G10B10_UNORM_PACK32													
VK_FORMAT_A2R10G10B10_SNORM_PACK32													
VK_FORMAT_A2R10G10B10_USCALED_PACK32													
VK_FORMAT_A2R10G10B10_SSCALED_PACK32													
VK_FORMAT_A2R10G10B10_UINT_PACK32													
VK_FORMAT_A2R10G10B10_SINT_PACK32													
VK_FORMAT_A2B10G10R10_UNORM_PACK32				‡									
VK_FORMAT_A2B10G10R10_SNORM_PACK32													
VK_FORMAT_A2B10G10R10_USCALED_PACK32													
VK_FORMAT_A2B10G10R10_SSCALED_PACK32													
VK_FORMAT_A2B10G10R10_UINT_PACK32				‡									
VK_FORMAT_A2B10G10R10_SINT_PACK32													
VK_FORMAT_R10X6_UNORM_PACK16													
VK_FORMAT_R10X6G10X6_UNORM_2PACK16													
VK_FORMAT_R12X4_UNORM_PACK16													
VK_FORMAT_R12X4G12X4_UNORM_2PACK16													

Format features marked with ‡ **must** be supported for `optimalTilingFeatures` if the `VkPhysicalDevice` supports the `shaderStorageImageExtendedFormats` feature.

Table 78. Mandatory Format Support: 16-bit Components

Format	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
VK_FORMAT_R16_UNORM				‡						◻			
VK_FORMAT_R16_SNORM				‡						◻			
VK_FORMAT_R16_USCALED													
VK_FORMAT_R16_SSCALED													
VK_FORMAT_R16_UINT	◻	◻		‡		◻	◻			◻	◻		
VK_FORMAT_R16_SINT	◻	◻		‡		◻	◻			◻	◻		
VK_FORMAT_R16_SFLOAT	◻	◻	◻	‡		◻	◻	◻		◻	◻		
VK_FORMAT_R16G16_UNORM				‡						◻			
VK_FORMAT_R16G16_SNORM				‡						◻			
VK_FORMAT_R16G16_USCALED													
VK_FORMAT_R16G16_SSCALED													
VK_FORMAT_R16G16_UINT	◻	◻		‡		◻	◻			◻	◻		
VK_FORMAT_R16G16_SINT	◻	◻		‡		◻	◻			◻	◻		
VK_FORMAT_R16G16_SFLOAT	◻	◻	◻	‡	§	◻	◻	◻		◻	◻	§	§
VK_FORMAT_R16G16B16_UNORM													
VK_FORMAT_R16G16B16_SNORM													
VK_FORMAT_R16G16B16_USCALED													
VK_FORMAT_R16G16B16_SSCALED													
VK_FORMAT_R16G16B16_UINT													
VK_FORMAT_R16G16B16_SINT													
VK_FORMAT_R16G16B16_SFLOAT													
VK_FORMAT_R16G16B16A16_UNORM				‡						◻			
VK_FORMAT_R16G16B16A16_SNORM				‡						◻			

VK_FORMAT_R16G16B16A16_USCALED													
VK_FORMAT_R16G16B16A16_SSCALED													
VK_FORMAT_R16G16B16A16_UINT	□	□		□		□	□			□	□	□	
VK_FORMAT_R16G16B16A16_SINT	□	□		□		□	□			□	□	□	
VK_FORMAT_R16G16B16A16_SFLOAT	□	□	□	□	§	□	□	□		□	□	□	§

Format features marked with ‡ **must** be supported for `optimalTilingFeatures` if the `VkPhysicalDevice` supports the `shaderStorageImageExtendedFormats` feature.

Table 79. Mandatory Format Support: 32-bit Components

VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	**Format**
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32_UINT
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32_SINT
✓	✓	✓	✓	†	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32_SFLOAT
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32G32_UINT
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32G32_SINT
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32G32_SFLOAT
✓													VK_FORMAT_R32G32B32_UINT
✓													VK_FORMAT_R32G32B32_SINT
✓													VK_FORMAT_R32G32B32_SFLOAT
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32G32B32A32_UINT
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32G32B32A32_SINT
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	VK_FORMAT_R32G32B32A32_SFLOAT

Format features marked with † **must** be supported for `optimalTilingFeatures` if the `VkPhysicalDevice` supports the `shaderImageFloat32Atomics` or the `shaderImageFloat32AtomicAdd` or the `shaderImageFloat32AtomicMinMax` feature.

Table 80. Mandatory Format Support: 64-bit/uneven Components

	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
Format													
VK_FORMAT_R64_UINT				†	†								
VK_FORMAT_R64_SINT				†	†								
VK_FORMAT_R64_SFLOAT													
VK_FORMAT_R64G64_UINT													
VK_FORMAT_R64G64_SINT													
VK_FORMAT_R64G64_SFLOAT													
VK_FORMAT_R64G64B64_UINT													
VK_FORMAT_R64G64B64_SINT													
VK_FORMAT_R64G64B64_SFLOAT													
VK_FORMAT_R64G64B64A64_UINT													
VK_FORMAT_R64G64B64A64_SINT													
VK_FORMAT_R64G64B64A64_SFLOAT													
VK_FORMAT_B10G11R11_UFLOAT_PACK32				‡									
VK_FORMAT_E5B9G9R9_UFLOAT_PACK32													

Format features marked with ‡ **must** be supported for `optimalTilingFeatures` if the `VkPhysicalDevice` supports the `shaderStorageImageExtendedFormats` feature.

If the `shaderImageInt64Atomics` feature is supported, `VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT` and `VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT` **must** be advertised in `optimalTilingFeatures` for both `VK_FORMAT_R64_UINT` and `VK_FORMAT_R64_SINT`.

Table 81. Mandatory Format Support: Depth/Stencil With `VkImageType VK_IMAGE_TYPE_2D`

Format	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
VK_FORMAT_D16_UNORM	◻	◻							◻				
VK_FORMAT_X8_D24_UNORM_PACK32									†				
VK_FORMAT_D32_SFLOAT	◻	◻							†				
VK_FORMAT_S8_UINT													
VK_FORMAT_D16_UNORM_S8_UINT													
VK_FORMAT_D24_UNORM_S8_UINT									†				
VK_FORMAT_D32_SFLOAT_S8_UINT									†				
VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT feature **must** be supported for at least one of VK_FORMAT_X8_D24_UNORM_PACK32 and VK_FORMAT_D32_SFLOAT, and **must** be supported for at least one of VK_FORMAT_D24_UNORM_S8_UINT and VK_FORMAT_D32_SFLOAT_S8_UINT.													
`bufferFeatures` **must** not support any features for these formats													

Table 82. Mandatory Format Support: BC Compressed Formats With `VkImageType` `VK_IMAGE_TYPE_2D` *and* `VK_IMAGE_TYPE_3D`

Format	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT	
VK_FORMAT_BC1_RGB_UNORM_BLOCK	†	†	†											
VK_FORMAT_BC1_RGB_SRGB_BLOCK	†	†	†											
VK_FORMAT_BC1_RGBA_UNORM_BLOCK	†	†	†											
VK_FORMAT_BC1_RGBA_SRGB_BLOCK	†	†	†											
VK_FORMAT_BC2_UNORM_BLOCK	†	†	†											
VK_FORMAT_BC2_SRGB_BLOCK	†	†	†											
VK_FORMAT_BC3_UNORM_BLOCK	†	†	†											
VK_FORMAT_BC3_SRGB_BLOCK	†	†	†											
VK_FORMAT_BC4_UNORM_BLOCK	†	†	†											
VK_FORMAT_BC4_SNORM_BLOCK	†	†	†											
VK_FORMAT_BC5_UNORM_BLOCK	†	†	†											
VK_FORMAT_BC5_SNORM_BLOCK	†	†	†											
VK_FORMAT_BC6H_UFLOAT_BLOCK	†	†	†											
VK_FORMAT_BC6H_SFLOAT_BLOCK	†	†	†											
VK_FORMAT_BC7_UNORM_BLOCK	†	†	†											
VK_FORMAT_BC7_SRGB_BLOCK	†	†	†											

The `VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT`, `VK_FORMAT_FEATURE_BLIT_SRC_BIT` and `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT` features **must** be supported in `optimalTilingFeatures` for all the formats in at least one of: this table, Mandatory Format Support: ETC2 and EAC Compressed Formats With `VkImageType` `VK_IMAGE_TYPE_2D`, or Mandatory Format Support: ASTC LDR Compressed Formats With `VkImageType` `VK_IMAGE_TYPE_2D`.

Table 83. Mandatory Format Support: ETC2 and EAC Compressed Formats With `VkImageType` `VK_IMAGE_TYPE_2D`

Format	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT	
VK_FORMAT_ETC2_R8G8B8_UNORM_BLOCK	†	†	†											
VK_FORMAT_ETC2_R8G8B8_SRGB_BLOCK	†	†	†											
VK_FORMAT_ETC2_R8G8B8A1_UNORM_BLOCK	†	†	†											
VK_FORMAT_ETC2_R8G8B8A1_SRGB_BLOCK	†	†	†											
VK_FORMAT_ETC2_R8G8B8A8_UNORM_BLOCK	†	†	†											
VK_FORMAT_ETC2_R8G8B8A8_SRGB_BLOCK	†	†	†											
VK_FORMAT_EAC_R11_UNORM_BLOCK	†	†	†											
VK_FORMAT_EAC_R11_SNORM_BLOCK	†	†	†											
VK_FORMAT_EAC_R11G11_UNORM_BLOCK	†	†	†											
VK_FORMAT_EAC_R11G11_SNORM_BLOCK	†	†	†											

The `VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT`, `VK_FORMAT_FEATURE_BLIT_SRC_BIT` and `VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT` features **must** be supported in `optimalTilingFeatures` for all the formats in at least one of: this table, Mandatory Format Support: BC Compressed Formats With `VkImageType` `VK_IMAGE_TYPE_2D` and `VK_IMAGE_TYPE_3D`, or Mandatory Format Support: ASTC LDR Compressed Formats With `VkImageType` `VK_IMAGE_TYPE_2D`.

Table 84. Mandatory Format Support: ASTC LDR Compressed Formats With `VkImageType VK_IMAGE_TYPE_2D`

Format	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_ATOMIC_BIT
VK_FORMAT_ASTC_4x4_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_4x4_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_5x4_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_5x4_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_5x5_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_5x5_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_6x5_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_6x5_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_6x6_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_6x6_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_8x5_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_8x5_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_8x6_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_8x6_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_8x8_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_8x8_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_10x5_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_10x5_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_10x6_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_10x6_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_10x8_UNORM_BLOCK	†	†	†										

VK_FORMAT_ASTC_10x8_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_10x10_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_10x10_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_12x10_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_12x10_SRGB_BLOCK	†	†	†										
VK_FORMAT_ASTC_12x12_UNORM_BLOCK	†	†	†										
VK_FORMAT_ASTC_12x12_SRGB_BLOCK	†	†	†										
The VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT, VK_FORMAT_FEATURE_BLIT_SRC_BIT and VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT features **must** be supported in optimalTilingFeatures for all the formats in at least one of: this table, Mandatory Format Support: BC Compressed Formats With VkImageType VK_IMAGE_TYPE_2D and VK_IMAGE_TYPE_3D, or Mandatory Format Support: ETC2 and EAC Compressed Formats With VkImageType VK_IMAGE_TYPE_2D.													

If cubic filtering is supported, VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT **must** be supported for the following image view types:

- VK_IMAGE_VIEW_TYPE_2D
- VK_IMAGE_VIEW_TYPE_2D_ARRAY

for the following formats:

- VK_FORMAT_R4G4_UNORM_PACK8
- VK_FORMAT_R4G4B4A4_UNORM_PACK16
- VK_FORMAT_B4G4R4A4_UNORM_PACK16
- VK_FORMAT_R5G6B5_UNORM_PACK16
- VK_FORMAT_B5G6R5_UNORM_PACK16
- VK_FORMAT_R5G5B5A1_UNORM_PACK16
- VK_FORMAT_B5G5R5A1_UNORM_PACK16
- VK_FORMAT_A1R5G5B5_UNORM_PACK16
- VK_FORMAT_R8_UNORM
- VK_FORMAT_R8_SNORM
- VK_FORMAT_R8_SRGB
- VK_FORMAT_R8G8_UNORM
- VK_FORMAT_R8G8_SNORM
- VK_FORMAT_R8G8_SRGB
- VK_FORMAT_R8G8B8_UNORM
- VK_FORMAT_R8G8B8_SNORM
- VK_FORMAT_R8G8B8_SRGB
- VK_FORMAT_B8G8R8_UNORM

- VK_FORMAT_B8G8R8_SNORM
- VK_FORMAT_B8G8R8_SRGB
- VK_FORMAT_R8G8B8A8_UNORM
- VK_FORMAT_R8G8B8A8_SNORM
- VK_FORMAT_R8G8B8A8_SRGB
- VK_FORMAT_B8G8R8A8_UNORM
- VK_FORMAT_B8G8R8A8_SNORM
- VK_FORMAT_B8G8R8A8_SRGB
- VK_FORMAT_A8B8G8R8_UNORM_PACK32
- VK_FORMAT_A8B8G8R8_SNORM_PACK32
- VK_FORMAT_A8B8G8R8_SRGB_PACK32

If ETC compressed formats are supported, VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT **must** be supported for the following image view types:

- VK_IMAGE_VIEW_TYPE_2D
- VK_IMAGE_VIEW_TYPE_2D_ARRAY

for the following additional formats:

- VK_FORMAT_ETC2_R8G8B8_UNORM_BLOCK
- VK_FORMAT_ETC2_R8G8B8_SRGB_BLOCK
- VK_FORMAT_ETC2_R8G8B8A1_UNORM_BLOCK
- VK_FORMAT_ETC2_R8G8B8A1_SRGB_BLOCK
- VK_FORMAT_ETC2_R8G8B8A8_UNORM_BLOCK
- VK_FORMAT_ETC2_R8G8B8A8_SRGB_BLOCK

If cubic filtering is supported for any other formats, the following image view types **must** be supported for those formats:

- VK_IMAGE_VIEW_TYPE_2D
- VK_IMAGE_VIEW_TYPE_2D_ARRAY

To be used with VkImageView with subresourceRange.aspectMask equal to VK_IMAGE_ASPECT_COLOR_BIT, sampler $Y'C_BC_R$ conversion **must** be enabled for the following formats:

Table 85. Formats Requiring Sampler $Y'C_BC_R$ Conversion for VK_IMAGE_ASPECT_COLOR_BIT Image Views

Format	Planes	VK_FORMAT_FEATURE_DISJOINT_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_TRANSFER_SRC_BIT	VK_FORMAT_FEATURE_TRANSFER_DST_BIT	VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT	VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEABLE_BIT
VK_FORMAT_G8B8G8R8_422_UNORM	1										
VK_FORMAT_B8G8R8G8_422_UNORM	1										
VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM	3		†	†	†	†					
VK_FORMAT_G8_B8R8_2PLANE_420_UNORM	2		†	†	†	†					
VK_FORMAT_G8_B8_R8_3PLANE_422_UNORM	3										
VK_FORMAT_G8_B8R8_2PLANE_422_UNORM	2										
VK_FORMAT_G8_B8_R8_3PLANE_444_UNORM	3										
VK_FORMAT_R10X6G10X6B10X6A10X6_UNORM_4PACK16	1										
VK_FORMAT_G10X6B10X6G10X6R10X6_422_UNORM_4PACK16	1										
VK_FORMAT_B10X6G10X6R10X6G10X6_422_UNORM_4PACK16	1										
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16	3										
VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16	2										
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16	3										
VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16	2										
VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16	3										
VK_FORMAT_R12X4G12X4B12X4A12X4_UNORM_4PACK16	1										
VK_FORMAT_G12X4B12X4G12X4R12X4_422_UNORM_4PACK16	1										
VK_FORMAT_B12X4G12X4R12X4G12X4_422_UNORM_4PACK16	1										
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16	3										
VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16	2										
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16	3										
VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16	2										
VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16	3										

VK_FORMAT_G16B16G16R16_422_UNORM	1										
VK_FORMAT_B16G16R16G16_422_UNORM	1										
VK_FORMAT_G16_B16_R16_3PLANE_420_UNORM	3										
VK_FORMAT_G16_B16R16_2PLANE_420_UNORM	2										
VK_FORMAT_G16_B16_R16_3PLANE_422_UNORM	3										
VK_FORMAT_G16_B16R16_2PLANE_422_UNORM	2										
VK_FORMAT_G16_B16_R16_3PLANE_444_UNORM	3										
Format features marked † **must** be supported for optimalTilingFeatures with VkImageType VK_IMAGE_TYPE_2D if the VkPhysicalDevice supports the VkPhysicalDeviceSamplerYcbcrConversionFeatures feature.											

Implementations are not required to support the VK_IMAGE_CREATE_SPARSE_BINDING_BIT, VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT, or VK_IMAGE_CREATE_SPARSE_ALIASED_BIT VkImageCreateFlags for the above formats that require sampler $Y'C_BC_R$ conversion. To determine whether the implementation supports sparse image creation flags with these formats use vkGetPhysicalDeviceImageFormatProperties or vkGetPhysicalDeviceImageFormatProperties2KHR.

VK_FORMAT_FEATURE_FRAGMENT_DENSITY_MAP_BIT_EXT **must** be supported for the following formats if the fragmentDensityMap feature is enabled:

- VK_FORMAT_R8G8_UNORM

VK_FORMAT_FEATURE_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR **must** be supported in bufferFeatures for the following formats if the accelerationStructure feature is supported:

- VK_FORMAT_R32G32_SFLOAT
- VK_FORMAT_R32G32B32_SFLOAT
- VK_FORMAT_R16G16_SFLOAT
- VK_FORMAT_R16G16B16A16_SFLOAT
- VK_FORMAT_R16G16_SNORM
- VK_FORMAT_R16G16B16A16_SNORM

VK_FORMAT_FEATURE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR **must** be supported for the following formats if the attachmentFragmentShadingRate feature is supported:

- VK_FORMAT_R8_UINT

If the VK_EXT_host_image_copy extension is supported and VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT is supported in optimalTilingFeatures or linearTilingFeatures for a color format, VK_FORMAT_FEATURE_2_HOST_IMAGE_TRANSFER_BIT_EXT **must** also be supported in optimalTilingFeatures or linearTilingFeatures respectively.

42.3.1. Formats Without Shader Storage Format

The device-level features for using a storage image or a storage texel buffer with an image format of `Unknown`, `shaderStorageImageReadWithoutFormat` and `shaderStorageImageWriteWithoutFormat`, only apply to the following formats:

- `VK_FORMAT_R8G8B8A8_UNORM`
- `VK_FORMAT_R8G8B8A8_SNORM`
- `VK_FORMAT_R8G8B8A8_UINT`
- `VK_FORMAT_R8G8B8A8_SINT`
- `VK_FORMAT_R32_UINT`
- `VK_FORMAT_R32_SINT`
- `VK_FORMAT_R32_SFLOAT`
- `VK_FORMAT_R32G32_UINT`
- `VK_FORMAT_R32G32_SINT`
- `VK_FORMAT_R32G32_SFLOAT`
- `VK_FORMAT_R32G32B32A32_UINT`
- `VK_FORMAT_R32G32B32A32_SINT`
- `VK_FORMAT_R32G32B32A32_SFLOAT`
- `VK_FORMAT_R16G16B16A16_UINT`
- `VK_FORMAT_R16G16B16A16_SINT`
- `VK_FORMAT_R16G16B16A16_SFLOAT`
- `VK_FORMAT_R16G16_SFLOAT`
- `VK_FORMAT_B10G11R11_UFLOAT_PACK32`
- `VK_FORMAT_R16_SFLOAT`
- `VK_FORMAT_R16G16B16A16_UNORM`
- `VK_FORMAT_A2B10G10R10_UNORM_PACK32`
- `VK_FORMAT_R16G16_UNORM`
- `VK_FORMAT_R8G8_UNORM`
- `VK_FORMAT_R16_UNORM`
- `VK_FORMAT_R8_UNORM`
- `VK_FORMAT_R16G16B16A16_SNORM`
- `VK_FORMAT_R16G16_SNORM`
- `VK_FORMAT_R8G8_SNORM`
- `VK_FORMAT_R16_SNORM`
- `VK_FORMAT_R8_SNORM`
- `VK_FORMAT_R16G16_SINT`

- VK_FORMAT_R8G8_SINT
- VK_FORMAT_R16_SINT
- VK_FORMAT_R8_SINT
- VK_FORMAT_A2B10G10R10_UINT_PACK32
- VK_FORMAT_R16G16_UINT
- VK_FORMAT_R8G8_UINT
- VK_FORMAT_R16_UINT
- VK_FORMAT_R8_UINT
- VK_FORMAT_A8_UNORM_KHR

NOTE	This list of formats is the union of required storage formats from Required Format Support section and formats listed in shaderStorageImageExtendedFormats.

An implementation that supports VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT for any format from the given list of formats and supports shaderStorageImageReadWithoutFormat **must** support VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT for that same format if Vulkan 1.3 or the VK_KHR_format_feature_flags2 extension is supported.

An implementation that supports VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT for any format from the given list of formats and supports shaderStorageImageWriteWithoutFormat **must** support VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT for that same format if Vulkan 1.3 or the VK_KHR_format_feature_flags2 extension is supported.

An implementation that does not support either of VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT or VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT for a format **must** not report support for VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT or VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT for that format if it is not listed in the SPIR-V and Vulkan Image Format Compatibility table.

NOTE	Some older implementations do not follow this restriction. They report support for formats as storage images even though they do not support access without the Format qualifier and there is no matching Format token. Such images cannot be either read from or written to. Drivers which pass Vulkan conformance test suite version 1.3.9.0, or any subsequent version will conform to the requirement above.

42.3.2. Depth Comparison Format Support

If Vulkan 1.3 or the VK_KHR_format_feature_flags2 extension is supported, a depth/stencil format with a depth component supporting VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT **must** support VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT.

42.3.3. Format Feature Dependent Usage Flags

Certain resource usage flags depend on support for the corresponding format feature flag for the format in question. The following tables list the VkBufferUsageFlagBits and VkImageUsageFlagBits that have such dependencies, and the format feature flags they depend on. Additional restrictions, including, but not limited to, further required format feature flags specific to the particular use of the resource **may** apply, as described in the respective sections of this specification.

Table 86. Format Feature Dependent Buffer Usage Flags

Buffer usage flag	Required format feature flag
VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_UNIFORM_TEXEL_BUFFER_BIT
VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT	VK_FORMAT_FEATURE_STORAGE_TEXEL_BUFFER_BIT
VK_BUFFER_USAGE_VERTEX_BUFFER_BIT	VK_FORMAT_FEATURE_VERTEX_BUFFER_BIT

Table 87. Format Feature Dependent Image Usage Flags

Image usage flag	Required format feature flag
VK_IMAGE_USAGE_SAMPLED_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT
VK_IMAGE_USAGE_STORAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT
VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT
VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT or VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR	VK_FORMAT_FEATURE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR	VK_FORMAT_FEATURE_VIDEO_DECODE_OUTPUT_BIT_KHR
VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR	VK_FORMAT_FEATURE_VIDEO_DECODE_DPB_BIT_KHR
VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR	VK_FORMAT_FEATURE_VIDEO_ENCODE_INPUT_BIT_KHR
VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR	VK_FORMAT_FEATURE_VIDEO_ENCODE_DPB_BIT_KHR
VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR	VK_FORMAT_FEATURE_2_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR	VK_FORMAT_FEATURE_2_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR

Chapter 43. Additional Capabilities

This chapter describes additional capabilities beyond the minimum capabilities described in the Limits and Formats chapters, including:

- Additional Image Capabilities
- Additional Buffer Capabilities
- Optional Semaphore Capabilities
- Optional Fence Capabilities
- Timestamp Calibration Capabilities

43.1. Additional Image Capabilities

Additional image capabilities, such as larger dimensions or additional sample counts for certain image types, or additional capabilities for *linear* tiling format images, are described in this section.

To query additional capabilities specific to image types, call:

```
// Provided by VK_VERSION_1_0
VkResult vkGetPhysicalDeviceImageFormatProperties(
    VkPhysicalDevice                            physicalDevice,
    VkFormat                                    format,
    VkImageType                                 type,
    VkImageTiling                               tiling,
    VkImageUsageFlags                           usage,
    VkImageCreateFlags                          flags,
    VkImageFormatProperties*                    pImageFormatProperties);
```

- `physicalDevice` is the physical device from which to query the image capabilities.
- `format` is a VkFormat value specifying the image format, corresponding to VkImageCreateInfo ::`format`.
- `type` is a VkImageType value specifying the image type, corresponding to VkImageCreateInfo ::`imageType`.
- `tiling` is a VkImageTiling value specifying the image tiling, corresponding to VkImageCreateInfo::`tiling`.
- `usage` is a bitmask of VkImageUsageFlagBits specifying the intended usage of the image, corresponding to VkImageCreateInfo::`usage`.
- `flags` is a bitmask of VkImageCreateFlagBits specifying additional parameters of the image, corresponding to VkImageCreateInfo::`flags`.
- `pImageFormatProperties` is a pointer to a VkImageFormatProperties structure in which capabilities are returned.

The `format`, `type`, `tiling`, `usage`, and `flags` parameters correspond to parameters that would be

consumed by vkCreateImage (as members of VkImageCreateInfo).

If format is not a supported image format, or if the combination of format, type, tiling, usage, and flags is not supported for images, then vkGetPhysicalDeviceImageFormatProperties returns VK_ERROR_FORMAT_NOT_SUPPORTED.

The limitations on an image format that are reported by vkGetPhysicalDeviceImageFormatProperties have the following property: if usage1 and usage2 of type VkImageUsageFlags are such that the bits set in usage1 are a subset of the bits set in usage2, and flags1 and flags2 of type VkImageCreateFlags are such that the bits set in flags1 are a subset of the bits set in flags2, then the limitations for usage1 and flags1 **must** be no more strict than the limitations for usage2 and flags2, for all values of format, type, and tiling.

If the VK_EXT_host_image_copy extension is supported, usage includes VK_IMAGE_USAGE_SAMPLED_BIT, and flags does not include either of VK_IMAGE_CREATE_SPARSE_BINDING_BIT, VK_IMAGE_CREATE_SPARSE_RESIDENCY_BIT, or VK_IMAGE_CREATE_SPARSE_ALIASED_BIT, then the result of calls to vkGetPhysicalDeviceImageFormatProperties with identical parameters except for the inclusion of VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT in usage **must** be identical.

Valid Usage

- VUID-vkGetPhysicalDeviceImageFormatProperties-tiling-02248
  tiling **must** not be VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT. (Use vkGetPhysicalDeviceImageFormatProperties2KHR instead)

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceImageFormatProperties-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceImageFormatProperties-format-parameter
  format **must** be a valid VkFormat value
- VUID-vkGetPhysicalDeviceImageFormatProperties-type-parameter
  type **must** be a valid VkImageType value
- VUID-vkGetPhysicalDeviceImageFormatProperties-tiling-parameter
  tiling **must** be a valid VkImageTiling value
- VUID-vkGetPhysicalDeviceImageFormatProperties-usage-parameter
  usage **must** be a valid combination of VkImageUsageFlagBits values
- VUID-vkGetPhysicalDeviceImageFormatProperties-usage-requiredbitmask
  usage **must** not be 0
- VUID-vkGetPhysicalDeviceImageFormatProperties-flags-parameter
  flags **must** be a valid combination of VkImageCreateFlagBits values
- VUID-vkGetPhysicalDeviceImageFormatProperties-pImageFormatProperties-parameter
  pImageFormatProperties **must** be a valid pointer to a VkImageFormatProperties structure

Return Codes

Success

- VK_SUCCESS

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY
- VK_ERROR_FORMAT_NOT_SUPPORTED

The VkImageFormatProperties structure is defined as:

```
// Provided by VK_VERSION_1_0
typedef struct VkImageFormatProperties {
    VkExtent3D            maxExtent;
    uint32_t              maxMipLevels;
    uint32_t              maxArrayLayers;
    VkSampleCountFlags    sampleCounts;
    VkDeviceSize          maxResourceSize;
} VkImageFormatProperties;
```

- maxExtent are the maximum image dimensions. See the Allowed Extent Values section below for how these values are constrained by type.
- maxMipLevels is the maximum number of mipmap levels. maxMipLevels **must** be equal to the number of levels in the complete mipmap chain based on the maxExtent.width, maxExtent.height, and maxExtent.depth, except when one of the following conditions is true, in which case it **may** instead be 1:
  - vkGetPhysicalDeviceImageFormatProperties::tiling was VK_IMAGE_TILING_LINEAR
  - VkPhysicalDeviceImageFormatInfo2::tiling was VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT
  - the VkPhysicalDeviceImageFormatInfo2::pNext chain included a VkPhysicalDeviceExternalImageFormatInfo structure with a handle type included in the handleTypes member for which mipmap image support is not required
  - image format is one of the formats that require a sampler $Y'C_BC_R$ conversion
  - flags contains VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT
- maxArrayLayers is the maximum number of array layers. maxArrayLayers **must** be no less than VkPhysicalDeviceLimits::maxImageArrayLayers, except when one of the following conditions is true, in which case it **may** instead be 1:
  - tiling is VK_IMAGE_TILING_LINEAR
  - tiling is VK_IMAGE_TILING_OPTIMAL and type is VK_IMAGE_TYPE_3D
  - format is one of the formats that require a sampler $Y'C_BC_R$ conversion
- If tiling is VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT, then maxArrayLayers **must** not be 0.

- `sampleCounts` is a bitmask of VkSampleCountFlagBits specifying all the supported sample counts for this image as described below.
- `maxResourceSize` is an upper bound on the total image size in bytes, inclusive of all image subresources. Implementations **may** have an address space limit on total size of a resource, which is advertised by this property. `maxResourceSize` **must** be at least 2^{31}.

NOTE	There is no mechanism to query the size of an image before creating it, to compare that size against `maxResourceSize`. If an application attempts to create an image that exceeds this limit, the creation will fail and vkCreateImage will return `VK_ERROR_OUT_OF_DEVICE_MEMORY`. While the advertised limit **must** be at least 2^{31}, it **may** not be possible to create an image that approaches that size, particularly for `VK_IMAGE_TYPE_1D`.

If the combination of parameters to `vkGetPhysicalDeviceImageFormatProperties` is not supported by the implementation for use in vkCreateImage, then all members of `VkImageFormatProperties` will be filled with zero.

NOTE	Filling `VkImageFormatProperties` with zero for unsupported formats is an exception to the usual rule that output structures have undefined contents on error. This exception was unintentional, but is preserved for backwards compatibility.

To query additional capabilities specific to image types, call:

```
// Provided by VK_KHR_get_physical_device_properties2
VkResult vkGetPhysicalDeviceImageFormatProperties2KHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceImageFormatInfo2*     pImageFormatInfo,
    VkImageFormatProperties2*                   pImageFormatProperties);
```

- `physicalDevice` is the physical device from which to query the image capabilities.
- `pImageFormatInfo` is a pointer to a VkPhysicalDeviceImageFormatInfo2 structure describing the parameters that would be consumed by vkCreateImage.
- `pImageFormatProperties` is a pointer to a VkImageFormatProperties2 structure in which capabilities are returned.

`vkGetPhysicalDeviceImageFormatProperties2KHR` behaves similarly to vkGetPhysicalDeviceImageFormatProperties, with the ability to return extended information in a `pNext` chain of output structures.

If the `pNext` chain of `pImageFormatInfo` includes a VkVideoProfileListInfoKHR structure with a `profileCount` member greater than `0`, then this command returns format capabilities specific to image types used in conjunction with the specified video profiles. In this case, this command will return one of the video-profile-specific error codes if any of the profiles specified via VkVideoProfileListInfoKHR::`pProfiles` are not supported. Furthermore, if VkPhysicalDeviceImageFormatInfo2::`usage` includes any image usage flag not supported by the specified video profiles, then this command returns `VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR`.

Valid Usage

- VUID-vkGetPhysicalDeviceImageFormatProperties2-pNext-09004
  If the `pNext` chain of `pImageFormatProperties` includes a VkHostImageCopyDevicePerformanceQueryEXT structure, `pImageFormatInfo->usage` **must** contain `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT`

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceImageFormatProperties2-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceImageFormatProperties2-pImageFormatInfo-parameter
  `pImageFormatInfo` **must** be a valid pointer to a valid VkPhysicalDeviceImageFormatInfo2 structure
- VUID-vkGetPhysicalDeviceImageFormatProperties2-pImageFormatProperties-parameter
  `pImageFormatProperties` **must** be a valid pointer to a VkImageFormatProperties2 structure

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`
- `VK_ERROR_FORMAT_NOT_SUPPORTED`
- `VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR`
- `VK_ERROR_VIDEO_PROFILE_OPERATION_NOT_SUPPORTED_KHR`
- `VK_ERROR_VIDEO_PROFILE_FORMAT_NOT_SUPPORTED_KHR`
- `VK_ERROR_VIDEO_PICTURE_LAYOUT_NOT_SUPPORTED_KHR`
- `VK_ERROR_VIDEO_PROFILE_CODEC_NOT_SUPPORTED_KHR`

The `VkPhysicalDeviceImageFormatInfo2` structure is defined as:

```
typedef struct VkPhysicalDeviceImageFormatInfo2 {
    VkStructureType       sType;
    const void*           pNext;
    VkFormat              format;
    VkImageType           type;
    VkImageTiling         tiling;
    VkImageUsageFlags     usage;
    VkImageCreateFlags    flags;
} VkPhysicalDeviceImageFormatInfo2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkPhysicalDeviceImageFormatInfo2 VkPhysicalDeviceImageFormatInfo2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure. The `pNext` chain of `VkPhysicalDeviceImageFormatInfo2` is used to provide additional image parameters to `vkGetPhysicalDeviceImageFormatProperties2KHR`.
- `format` is a VkFormat value indicating the image format, corresponding to VkImageCreateInfo::`format`.
- `type` is a VkImageType value indicating the image type, corresponding to VkImageCreateInfo::`imageType`.
- `tiling` is a VkImageTiling value indicating the image tiling, corresponding to VkImageCreateInfo::`tiling`.
- `usage` is a bitmask of VkImageUsageFlagBits indicating the intended usage of the image, corresponding to VkImageCreateInfo::`usage`.
- `flags` is a bitmask of VkImageCreateFlagBits indicating additional parameters of the image, corresponding to VkImageCreateInfo::`flags`.

The members of `VkPhysicalDeviceImageFormatInfo2` correspond to the arguments to vkGetPhysicalDeviceImageFormatProperties, with `sType` and `pNext` added for extensibility.

Valid Usage

- VUID-VkPhysicalDeviceImageFormatInfo2-tiling-02249
  `tiling` **must** be `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT` if and only if the `pNext` chain includes VkPhysicalDeviceImageDrmFormatModifierInfoEXT
- VUID-VkPhysicalDeviceImageFormatInfo2-tiling-02313
  If `tiling` is `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT` and `flags` contains `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT`, then the `pNext` chain **must** include a VkImageFormatListCreateInfo structure with non-zero `viewFormatCount`

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceImageFormatInfo2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_FORMAT_INFO_2`
- VUID-VkPhysicalDeviceImageFormatInfo2-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkImageFormatListCreateInfo, VkPhysicalDeviceExternalImageFormatInfo, VkPhysicalDeviceImageDrmFormatModifierInfoEXT, VkPhysicalDeviceImageViewImageFormatInfoEXT, or VkVideoProfileListInfoKHR
- VUID-VkPhysicalDeviceImageFormatInfo2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPhysicalDeviceImageFormatInfo2-format-parameter
  `format` **must** be a valid VkFormat value
- VUID-VkPhysicalDeviceImageFormatInfo2-type-parameter
  `type` **must** be a valid VkImageType value
- VUID-VkPhysicalDeviceImageFormatInfo2-tiling-parameter
  `tiling` **must** be a valid VkImageTiling value
- VUID-VkPhysicalDeviceImageFormatInfo2-usage-parameter
  `usage` **must** be a valid combination of VkImageUsageFlagBits values
- VUID-VkPhysicalDeviceImageFormatInfo2-usage-requiredbitmask
  `usage` **must** not be `0`
- VUID-VkPhysicalDeviceImageFormatInfo2-flags-parameter
  `flags` **must** be a valid combination of VkImageCreateFlagBits values

The `VkImageFormatProperties2` structure is defined as:

```
typedef struct VkImageFormatProperties2 {
    VkStructureType             sType;
    void*                       pNext;
    VkImageFormatProperties     imageFormatProperties;
} VkImageFormatProperties2;
```

or the equivalent

```
// Provided by VK_KHR_get_physical_device_properties2
typedef VkImageFormatProperties2 VkImageFormatProperties2KHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure. The `pNext` chain of `VkImageFormatProperties2` is used to allow the specification of additional capabilities to be returned from `vkGetPhysicalDeviceImageFormatProperties2KHR`.

- `imageFormatProperties` is a VkImageFormatProperties structure in which capabilities are returned.

If the combination of parameters to `vkGetPhysicalDeviceImageFormatProperties2KHR` is not supported by the implementation for use in vkCreateImage, then all members of `imageFormatProperties` will be filled with zero.

NOTE Filling `imageFormatProperties` with zero for unsupported formats is an exception to the usual rule that output structures have undefined contents on error. This exception was unintentional, but is preserved for backwards compatibility. This exception only applies to `imageFormatProperties`, not `sType`, `pNext`, or any structures chained from `pNext`.

Valid Usage (Implicit)

- VUID-VkImageFormatProperties2-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_IMAGE_FORMAT_PROPERTIES_2`
- VUID-VkImageFormatProperties2-pNext-pNext
  Each `pNext` member of any structure (including this one) in the `pNext` chain **must** be either `NULL` or a pointer to a valid instance of VkExternalImageFormatProperties, VkFilterCubicImageViewImageFormatPropertiesEXT, VkHostImageCopyDevicePerformanceQueryEXT, or VkSamplerYcbcrConversionImageFormatProperties
- VUID-VkImageFormatProperties2-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique

To determine the image capabilities compatible with an external memory handle type, add a VkPhysicalDeviceExternalImageFormatInfo structure to the `pNext` chain of the VkPhysicalDeviceImageFormatInfo2 structure and a `VkExternalImageFormatProperties` structure to the `pNext` chain of the VkImageFormatProperties2 structure.

The `VkPhysicalDeviceExternalImageFormatInfo` structure is defined as:

```
typedef struct VkPhysicalDeviceExternalImageFormatInfo {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExternalMemoryHandleTypeFlagBits    handleType;
} VkPhysicalDeviceExternalImageFormatInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkPhysicalDeviceExternalImageFormatInfo
VkPhysicalDeviceExternalImageFormatInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleType` is a VkExternalMemoryHandleTypeFlagBits value specifying the memory handle type that will be used with the memory associated with the image.

If `handleType` is 0, vkGetPhysicalDeviceImageFormatProperties2KHR will behave as if VkPhysicalDeviceExternalImageFormatInfo was not present, and VkExternalImageFormatProperties will be ignored.

If `handleType` is not compatible with the `format`, `type`, `tiling`, `usage`, and `flags` specified in VkPhysicalDeviceImageFormatInfo2, then vkGetPhysicalDeviceImageFormatProperties2KHR returns `VK_ERROR_FORMAT_NOT_SUPPORTED`.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceExternalImageFormatInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_IMAGE_FORMAT_INFO`
- VUID-VkPhysicalDeviceExternalImageFormatInfo-handleType-parameter
  If `handleType` is not `0`, `handleType` **must** be a valid VkExternalMemoryHandleTypeFlagBits value

Possible values of VkPhysicalDeviceExternalImageFormatInfo::`handleType`, specifying an external memory handle type, are:

```
typedef enum VkExternalMemoryHandleTypeFlagBits {
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT = 0x00000001,
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT = 0x00000002,
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT = 0x00000004,
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT = 0x00000008,
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT = 0x00000010,
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT = 0x00000020,
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT = 0x00000040,
  // Provided by VK_EXT_external_memory_dma_buf
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_DMA_BUF_BIT_EXT = 0x00000200,
  // Provided by VK_EXT_external_memory_host
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT = 0x00000080,
  // Provided by VK_EXT_external_memory_host
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT = 0x00000100,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT_KHR =
VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR =
VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT_KHR =
VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT_KHR =
VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT_KHR =
VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT_KHR =
VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT_KHR =
VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT,
} VkExternalMemoryHandleTypeFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkExternalMemoryHandleTypeFlagBits VkExternalMemoryHandleTypeFlagBitsKHR;
```

- VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT specifies a POSIX file descriptor handle that has only limited valid usage outside of Vulkan and other compatible APIs. It **must** be compatible with the POSIX system calls dup, dup2, close, and the non-standard system call dup3. Additionally, it **must** be transportable over a socket using an SCM_RIGHTS control message. It owns a reference to the underlying memory resource represented by its Vulkan memory object.
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT specifies an NT handle that has only limited

valid usage outside of Vulkan and other compatible APIs. It **must** be compatible with the functions DuplicateHandle, CloseHandle, CompareObjectHandles, GetHandleInformation, and SetHandleInformation. It owns a reference to the underlying memory resource represented by its Vulkan memory object.

- VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT specifies a global share handle that has only limited valid usage outside of Vulkan and other compatible APIs. It is not compatible with any native APIs. It does not own a reference to the underlying memory resource represented by its Vulkan memory object, and will therefore become invalid when all Vulkan memory objects associated with it are destroyed.
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT specifies an NT handle returned by IDXGIResource1::CreateSharedHandle referring to a Direct3D 10 or 11 texture resource. It owns a reference to the memory used by the Direct3D resource.
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT specifies a global share handle returned by IDXGIResource::GetSharedHandle referring to a Direct3D 10 or 11 texture resource. It does not own a reference to the underlying Direct3D resource, and will therefore become invalid when all Vulkan memory objects and Direct3D resources associated with it are destroyed.
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT specifies an NT handle returned by ID3D12Device::CreateSharedHandle referring to a Direct3D 12 heap resource. It owns a reference to the resources used by the Direct3D heap.
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT specifies an NT handle returned by ID3D12Device::CreateSharedHandle referring to a Direct3D 12 committed resource. It owns a reference to the memory used by the Direct3D resource.
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT specifies a host pointer returned by a host memory allocation command. It does not own a reference to the underlying memory resource, and will therefore become invalid if the host memory is freed.
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT specifies a host pointer to *host mapped foreign memory*. It does not own a reference to the underlying memory resource, and will therefore become invalid if the foreign memory is unmapped or otherwise becomes no longer available.
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_DMA_BUF_BIT_EXT is a file descriptor for a Linux dma_buf. It owns a reference to the underlying memory resource represented by its Vulkan memory object.

Some external memory handle types can only be shared within the same underlying physical device and/or the same driver version, as defined in the following table:

Table 88. External Memory Handle Types Compatibility

Handle type	VkPhysicalDeviceIDProperties::driverUUID	VkPhysicalDeviceIDProperties::deviceUUID
VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT	Must match	Must match
VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT	Must match	Must match
VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT	Must match	Must match
VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT	Must match	Must match
VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT	Must match	Must match
VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT	Must match	Must match
VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT	Must match	Must match
VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT	No restriction	No restriction
VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT	No restriction	No restriction
VK_EXTERNAL_MEMORY_HANDLE_TYPE_DMA_BUF_BIT_EXT	No restriction	No restriction

NOTE The above table does not restrict the drivers and devices with which VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT and VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT **may** be shared, as these handle types inherently mean memory that does not come from the same device, as they import memory from the host or a foreign device, respectively.

NOTE Even though the above table does not restrict the drivers and devices with which VK_EXTERNAL_MEMORY_HANDLE_TYPE_DMA_BUF_BIT_EXT **may** be shared, query mechanisms exist in the Vulkan API that prevent the import of incompatible dma-bufs (such as vkGetMemoryFdPropertiesKHR) and that prevent incompatible usage of dma-bufs (such as VkPhysicalDeviceExternalBufferInfo and VkPhysicalDeviceExternalImageFormatInfo).

```
typedef VkFlags VkExternalMemoryHandleTypeFlags;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkExternalMemoryHandleTypeFlags VkExternalMemoryHandleTypeFlagsKHR;
```

VkExternalMemoryHandleTypeFlags is a bitmask type for setting a mask of zero or more VkExternalMemoryHandleTypeFlagBits.

The VkExternalImageFormatProperties structure is defined as:

```
typedef struct VkExternalImageFormatProperties {
    VkStructureType               sType;
    void*                         pNext;
    VkExternalMemoryProperties    externalMemoryProperties;
} VkExternalImageFormatProperties;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkExternalImageFormatProperties VkExternalImageFormatPropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- externalMemoryProperties is a VkExternalMemoryProperties structure specifying various capabilities of the external handle type when used with the specified image creation parameters.

Valid Usage (Implicit)

- VUID-VkExternalImageFormatProperties-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_EXTERNAL_IMAGE_FORMAT_PROPERTIES

The VkExternalMemoryProperties structure is defined as:

```
typedef struct VkExternalMemoryProperties {
    VkExternalMemoryFeatureFlags       externalMemoryFeatures;
    VkExternalMemoryHandleTypeFlags    exportFromImportedHandleTypes;
    VkExternalMemoryHandleTypeFlags    compatibleHandleTypes;
} VkExternalMemoryProperties;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkExternalMemoryProperties VkExternalMemoryPropertiesKHR;
```

- `externalMemoryFeatures` is a bitmask of VkExternalMemoryFeatureFlagBits specifying the features of `handleType`.
- `exportFromImportedHandleTypes` is a bitmask of VkExternalMemoryHandleTypeFlagBits specifying which types of imported handle `handleType` **can** be exported from.
- `compatibleHandleTypes` is a bitmask of VkExternalMemoryHandleTypeFlagBits specifying handle types which **can** be specified at the same time as `handleType` when creating an image compatible with external memory.

`compatibleHandleTypes` **must** include at least `handleType`. Inclusion of a handle type in `compatibleHandleTypes` does not imply the values returned in VkImageFormatProperties2 will be the same when VkPhysicalDeviceExternalImageFormatInfo::`handleType` is set to that type. The application is responsible for querying the capabilities of all handle types intended for concurrent use in a single image and intersecting them to obtain the compatible set of capabilities.

Bits which **may** be set in VkExternalMemoryProperties::`externalMemoryFeatures`, specifying features of an external memory handle type, are:

```
typedef enum VkExternalMemoryFeatureFlagBits {
    VK_EXTERNAL_MEMORY_FEATURE_DEDICATED_ONLY_BIT = 0x00000001,
    VK_EXTERNAL_MEMORY_FEATURE_EXPORTABLE_BIT = 0x00000002,
    VK_EXTERNAL_MEMORY_FEATURE_IMPORTABLE_BIT = 0x00000004,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_FEATURE_DEDICATED_ONLY_BIT_KHR =
VK_EXTERNAL_MEMORY_FEATURE_DEDICATED_ONLY_BIT,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_FEATURE_EXPORTABLE_BIT_KHR =
VK_EXTERNAL_MEMORY_FEATURE_EXPORTABLE_BIT,
  // Provided by VK_KHR_external_memory_capabilities
    VK_EXTERNAL_MEMORY_FEATURE_IMPORTABLE_BIT_KHR =
VK_EXTERNAL_MEMORY_FEATURE_IMPORTABLE_BIT,
} VkExternalMemoryFeatureFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkExternalMemoryFeatureFlagBits VkExternalMemoryFeatureFlagBitsKHR;
```

- `VK_EXTERNAL_MEMORY_FEATURE_DEDICATED_ONLY_BIT` specifies that images or buffers created with the specified parameters and handle type **must** use the mechanisms defined by VkMemoryDedicatedRequirements and VkMemoryDedicatedAllocateInfo to create (or import) a dedicated allocation for the image or buffer.
- `VK_EXTERNAL_MEMORY_FEATURE_EXPORTABLE_BIT` specifies that handles of this type **can** be exported from Vulkan memory objects.
- `VK_EXTERNAL_MEMORY_FEATURE_IMPORTABLE_BIT` specifies that handles of this type **can** be imported as Vulkan memory objects.

Because their semantics in external APIs roughly align with that of an image or buffer with a dedicated allocation in Vulkan, implementations are **required** to report VK_EXTERNAL_MEMORY_FEATURE_DEDICATED_ONLY_BIT for the following external handle types:

- VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT
- VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT

Implementations **must** not report VK_EXTERNAL_MEMORY_FEATURE_DEDICATED_ONLY_BIT for images or buffers with external handle type VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT, or VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT.

```
typedef VkFlags VkExternalMemoryFeatureFlags;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkExternalMemoryFeatureFlags VkExternalMemoryFeatureFlagsKHR;
```

VkExternalMemoryFeatureFlags is a bitmask type for setting a mask of zero or more VkExternalMemoryFeatureFlagBits.

To query the image capabilities that are compatible with a Linux DRM format modifier, set VkPhysicalDeviceImageFormatInfo2::tiling to VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT and add a VkPhysicalDeviceImageDrmFormatModifierInfoEXT structure to the pNext chain of VkPhysicalDeviceImageFormatInfo2.

The VkPhysicalDeviceImageDrmFormatModifierInfoEXT structure is defined as:

```
// Provided by VK_EXT_image_drm_format_modifier
typedef struct VkPhysicalDeviceImageDrmFormatModifierInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    uint64_t           drmFormatModifier;
    VkSharingMode      sharingMode;
    uint32_t           queueFamilyIndexCount;
    const uint32_t*    pQueueFamilyIndices;
} VkPhysicalDeviceImageDrmFormatModifierInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- drmFormatModifier is the image's *Linux DRM format modifier*, corresponding to VkImageDrmFormatModifierExplicitCreateInfoEXT::modifier or to VkImageDrmFormatModifierListCreateInfoEXT::pModifiers.
- sharingMode specifies how the image will be accessed by multiple queue families.

- `queueFamilyIndexCount` is the number of entries in the `pQueueFamilyIndices` array.
- `pQueueFamilyIndices` is a pointer to an array of queue families that will access the image. It is ignored if `sharingMode` is not `VK_SHARING_MODE_CONCURRENT`.

If the `drmFormatModifier` is incompatible with the parameters specified in VkPhysicalDeviceImageFormatInfo2 and its `pNext` chain, then vkGetPhysicalDeviceImageFormatProperties2KHR returns `VK_ERROR_FORMAT_NOT_SUPPORTED`. The implementation **must** support the query of any `drmFormatModifier`, including unknown and invalid modifier values.

Valid Usage

- VUID-VkPhysicalDeviceImageDrmFormatModifierInfoEXT-sharingMode-02314
  If `sharingMode` is `VK_SHARING_MODE_CONCURRENT`, then `pQueueFamilyIndices` **must** be a valid pointer to an array of `queueFamilyIndexCount` `uint32_t` values
- VUID-VkPhysicalDeviceImageDrmFormatModifierInfoEXT-sharingMode-02315
  If `sharingMode` is `VK_SHARING_MODE_CONCURRENT`, then `queueFamilyIndexCount` **must** be greater than `1`
- VUID-VkPhysicalDeviceImageDrmFormatModifierInfoEXT-sharingMode-02316
  If `sharingMode` is `VK_SHARING_MODE_CONCURRENT`, each element of `pQueueFamilyIndices` **must** be unique and **must** be less than the `pQueueFamilyPropertyCount` returned by vkGetPhysicalDeviceQueueFamilyProperties2KHR for the `physicalDevice` that was used to create `device`

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceImageDrmFormatModifierInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_DRM_FORMAT_MODIFIER_INFO_EXT`
- VUID-VkPhysicalDeviceImageDrmFormatModifierInfoEXT-sharingMode-parameter
  `sharingMode` **must** be a valid VkSharingMode value

To determine the number of combined image samplers required to support a multi-planar format, add VkSamplerYcbcrConversionImageFormatProperties to the `pNext` chain of the VkImageFormatProperties2 structure in a call to `vkGetPhysicalDeviceImageFormatProperties2KHR`.

The `VkSamplerYcbcrConversionImageFormatProperties` structure is defined as:

```
typedef struct VkSamplerYcbcrConversionImageFormatProperties {
    VkStructureType    sType;
    void*              pNext;
    uint32_t           combinedImageSamplerDescriptorCount;
} VkSamplerYcbcrConversionImageFormatProperties;
```

or the equivalent

```
// Provided by VK_KHR_sampler_ycbcr_conversion
typedef VkSamplerYcbcrConversionImageFormatProperties
VkSamplerYcbcrConversionImageFormatPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `combinedImageSamplerDescriptorCount` is the number of combined image sampler descriptors that the implementation uses to access the format.

Valid Usage (Implicit)

- VUID-VkSamplerYcbcrConversionImageFormatProperties-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_IMAGE_FORMAT_PROPERTIES`

`combinedImageSamplerDescriptorCount` is a number between 1 and the number of planes in the format. A descriptor set layout binding with immutable $Y'C_BC_R$ conversion samplers will have a maximum `combinedImageSamplerDescriptorCount` which is the maximum across all formats supported by its samplers of the `combinedImageSamplerDescriptorCount` for each format. Descriptor sets with that layout will internally use that maximum `combinedImageSamplerDescriptorCount` descriptors for each descriptor in the binding. This expanded number of descriptors will be consumed from the descriptor pool when a descriptor set is allocated, and counts towards the `maxDescriptorSetSamplers`, `maxDescriptorSetSampledImages`, `maxPerStageDescriptorSamplers`, and `maxPerStageDescriptorSampledImages` limits.

NOTE

All descriptors in a binding use the same maximum `combinedImageSamplerDescriptorCount` descriptors to allow implementations to use a uniform stride for dynamic indexing of the descriptors in the binding.

For example, consider a descriptor set layout binding with two descriptors and immutable samplers for multi-planar formats that have `VkSamplerYcbcrConversionImageFormatProperties::combinedImageSamplerDescriptorCount` values of `2` and `3` respectively. There are two descriptors in the binding and the maximum `combinedImageSamplerDescriptorCount` is `3`, so descriptor sets with this layout consume `6` descriptors from the descriptor pool. To create a descriptor pool that allows allocating four descriptor sets with this layout, `descriptorCount` must be at least `24`.

Instead of querying all the potential formats that the application might use in the descriptor layout, the application **can** use the VkPhysicalDeviceMaintenance6PropertiesKHR::`maxCombinedImageSamplerDescriptorCount` property to determine the maximum descriptor size that will accommodate any and all formats that require a sampler $Y'C_BC_R$ conversion supported by the implementation.

To query if using `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT` has a negative impact on device performance when accessing an image, add `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT` to

VkPhysicalDeviceImageFormatInfo2::usage, and add a VkHostImageCopyDevicePerformanceQueryEXT structure to the pNext chain of a VkImageFormatProperties2 structure passed to vkGetPhysicalDeviceImageFormatProperties2KHR. This structure is defined as:

```
// Provided by VK_EXT_host_image_copy
typedef struct VkHostImageCopyDevicePerformanceQueryEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           optimalDeviceAccess;
    VkBool32           identicalMemoryLayout;
} VkHostImageCopyDevicePerformanceQueryEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- optimalDeviceAccess returns VK_TRUE if use of host image copy has no adverse effect on device access performance, compared to an image that is created with exact same creation parameters, and bound to the same VkDeviceMemory, except that VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT is replaced with VK_IMAGE_USAGE_TRANSFER_SRC_BIT and VK_IMAGE_USAGE_TRANSFER_DST_BIT.
- identicalMemoryLayout returns VK_TRUE if use of host image copy has no impact on memory layout compared to an image that is created with exact same creation parameters, and bound to the same VkDeviceMemory, except that VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT is replaced with VK_IMAGE_USAGE_TRANSFER_SRC_BIT and VK_IMAGE_USAGE_TRANSFER_DST_BIT.

The implementation **may** return VK_FALSE in optimalDeviceAccess if identicalMemoryLayout is VK_FALSE. If identicalMemoryLayout is VK_TRUE, optimalDeviceAccess **must** be VK_TRUE.

The implementation **may** return VK_TRUE in optimalDeviceAccess while identicalMemoryLayout is VK_FALSE. In this situation, any device performance impact **should** not be measurable.

If VkPhysicalDeviceImageFormatInfo2::format is a block-compressed format and vkGetPhysicalDeviceImageFormatProperties2KHR returns VK_SUCCESS, the implementation **must** return VK_TRUE in optimalDeviceAccess.

NOTE Applications can make use of optimalDeviceAccess to determine their resource copying strategy. If a resource is expected to be accessed more on device than on the host, and the implementation considers the resource sub-optimally accessed, it is likely better to use device copies instead.

NOTE Layout not being identical yet still considered optimal for device access could happen if the implementation has different memory layout patterns, some of which are easier to access on the host.

NOTE The most practical reason for optimalDeviceAccess to be VK_FALSE is that host image access may disable framebuffer compression where it would otherwise have been enabled. This represents far more efficient host image access since no compression algorithm is required to read or write to the image, but it would impact device

access performance. Some implementations may only set `optimalDeviceAccess` to `VK_FALSE` if certain conditions are met, such as specific image usage flags or creation flags.

Valid Usage (Implicit)

- VUID-VkHostImageCopyDevicePerformanceQueryEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_HOST_IMAGE_COPY_DEVICE_PERFORMANCE_QUERY_EXT`

To determine if cubic filtering can be used with a given image format and a given image view type add a VkPhysicalDeviceImageViewImageFormatInfoEXT structure to the `pNext` chain of the VkPhysicalDeviceImageFormatInfo2 structure, and a VkFilterCubicImageViewImageFormatPropertiesEXT structure to the `pNext` chain of the VkImageFormatProperties2 structure.

The `VkPhysicalDeviceImageViewImageFormatInfoEXT` structure is defined as:

```
// Provided by VK_EXT_filter_cubic
typedef struct VkPhysicalDeviceImageViewImageFormatInfoEXT {
    VkStructureType    sType;
    void*              pNext;
    VkImageViewType    imageViewType;
} VkPhysicalDeviceImageViewImageFormatInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `imageViewType` is a VkImageViewType value specifying the type of the image view.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceImageViewImageFormatInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_VIEW_IMAGE_FORMAT_INFO_EXT`
- VUID-VkPhysicalDeviceImageViewImageFormatInfoEXT-imageViewType-parameter
  `imageViewType` **must** be a valid VkImageViewType value

The `VkFilterCubicImageViewImageFormatPropertiesEXT` structure is defined as:

```
// Provided by VK_EXT_filter_cubic
typedef struct VkFilterCubicImageViewImageFormatPropertiesEXT {
    VkStructureType    sType;
    void*              pNext;
    VkBool32           filterCubic;
    VkBool32           filterCubicMinmax;
} VkFilterCubicImageViewImageFormatPropertiesEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `filterCubic` tells if image format, image type and image view type **can** be used with cubic filtering. This field is set by the implementation. An application-specified value is ignored.
- `filterCubicMinmax` tells if image format, image type and image view type **can** be used with cubic filtering and minmax filtering. This field is set by the implementation. An application-specified value is ignored.

Valid Usage (Implicit)

- VUID-VkFilterCubicImageViewImageFormatPropertiesEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_FILTER_CUBIC_IMAGE_VIEW_IMAGE_FORMAT_PROPERTIES_EXT`

Valid Usage

- VUID-VkFilterCubicImageViewImageFormatPropertiesEXT-pNext-02627
  If the `pNext` chain of the VkImageFormatProperties2 structure includes a VkFilterCubicImageViewImageFormatPropertiesEXT structure, the `pNext` chain of the VkPhysicalDeviceImageFormatInfo2 structure **must** include a VkPhysicalDeviceImageViewImageFormatInfoEXT structure with an `imageViewType` that is compatible with `imageType`

43.1.1. Supported Sample Counts

`vkGetPhysicalDeviceImageFormatProperties` returns a bitmask of VkSampleCountFlagBits in `sampleCounts` specifying the supported sample counts for the image parameters.

`sampleCounts` will be `VK_SAMPLE_COUNT_1_BIT` if at least one of the following conditions is true:

- `tiling` is `VK_IMAGE_TILING_LINEAR`
- `type` is not `VK_IMAGE_TYPE_2D`
- `flags` contains `VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT`
- Neither the `VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT` flag nor the `VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT` flag in `VkFormatProperties::optimalTilingFeatures` returned by vkGetPhysicalDeviceFormatProperties is set

- VkPhysicalDeviceExternalImageFormatInfo::handleType is an external handle type for which multisampled image support is not required.
- format is one of the formats that require a sampler $Y'C_BC_R$ conversion
- usage contains VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- usage contains VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT

Otherwise, the bits set in sampleCounts will be the sample counts supported for the specified values of usage and format. For each bit set in usage, the supported sample counts relate to the limits in VkPhysicalDeviceLimits as follows:

- If usage includes VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT and format is a floating- or fixed-point color format, a superset of VkPhysicalDeviceLimits::framebufferColorSampleCounts
- If usage includes VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT, and format includes a depth component, a superset of VkPhysicalDeviceLimits::framebufferDepthSampleCounts
- If usage includes VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT, and format includes a stencil component, a superset of VkPhysicalDeviceLimits::framebufferStencilSampleCounts
- If usage includes VK_IMAGE_USAGE_SAMPLED_BIT, and format includes a color component, a superset of VkPhysicalDeviceLimits::sampledImageColorSampleCounts
- If usage includes VK_IMAGE_USAGE_SAMPLED_BIT, and format includes a depth component, a superset of VkPhysicalDeviceLimits::sampledImageDepthSampleCounts
- If usage includes VK_IMAGE_USAGE_SAMPLED_BIT, and format is an integer format, a superset of VkPhysicalDeviceLimits::sampledImageIntegerSampleCounts
- If usage includes VK_IMAGE_USAGE_STORAGE_BIT, a superset of VkPhysicalDeviceLimits::storageImageSampleCounts

If multiple bits are set in usage, sampleCounts will be the intersection of the per-usage values described above.

If none of the bits described above are set in usage, then there is no corresponding limit in VkPhysicalDeviceLimits. In this case, sampleCounts **must** include at least VK_SAMPLE_COUNT_1_BIT.

43.1.2. Allowed Extent Values Based on Image Type

Implementations **may** support extent values larger than the required minimum/maximum values for certain types of images. VkImageFormatProperties::maxExtent for each type is subject to the constraints below.

NOTE	Implementations **must** support images with dimensions up to the required minimum/maximum values for all types of images. It follows that the query for additional capabilities **must** return extent values that are at least as large as the required values.

For VK_IMAGE_TYPE_1D:

- maxExtent.width ≥ VkPhysicalDeviceLimits::maxImageDimension1D

- maxExtent.height = 1
- maxExtent.depth = 1

For VK_IMAGE_TYPE_2D when flags does not contain VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT:

- maxExtent.width ≥ VkPhysicalDeviceLimits::maxImageDimension2D
- maxExtent.height ≥ VkPhysicalDeviceLimits::maxImageDimension2D
- maxExtent.depth = 1

For VK_IMAGE_TYPE_2D when flags contains VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT:

- maxExtent.width ≥ VkPhysicalDeviceLimits::maxImageDimensionCube
- maxExtent.height ≥ VkPhysicalDeviceLimits::maxImageDimensionCube
- maxExtent.depth = 1

For VK_IMAGE_TYPE_3D:

- maxExtent.width ≥ VkPhysicalDeviceLimits::maxImageDimension3D
- maxExtent.height ≥ VkPhysicalDeviceLimits::maxImageDimension3D
- maxExtent.depth ≥ VkPhysicalDeviceLimits::maxImageDimension3D

43.2. Additional Buffer Capabilities

To query the external handle types supported by buffers, call:

```
// Provided by VK_KHR_external_memory_capabilities
void vkGetPhysicalDeviceExternalBufferPropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceExternalBufferInfo*   pExternalBufferInfo,
    VkExternalBufferProperties*                 pExternalBufferProperties);
```

- physicalDevice is the physical device from which to query the buffer capabilities.
- pExternalBufferInfo is a pointer to a VkPhysicalDeviceExternalBufferInfo structure describing the parameters that would be consumed by vkCreateBuffer.
- pExternalBufferProperties is a pointer to a VkExternalBufferProperties structure in which capabilities are returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceExternalBufferProperties-physicalDevice-parameter physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceExternalBufferProperties-pExternalBufferInfo-parameter pExternalBufferInfo **must** be a valid pointer to a valid VkPhysicalDeviceExternalBufferInfo structure

- VUID-vkGetPhysicalDeviceExternalBufferProperties-pExternalBufferProperties-parameter
  pExternalBufferProperties **must** be a valid pointer to a VkExternalBufferProperties structure

The VkPhysicalDeviceExternalBufferInfo structure is defined as:

```
typedef struct VkPhysicalDeviceExternalBufferInfo {
    VkStructureType                       sType;
    const void*                           pNext;
    VkBufferCreateFlags                   flags;
    VkBufferUsageFlags                    usage;
    VkExternalMemoryHandleTypeFlagBits    handleType;
} VkPhysicalDeviceExternalBufferInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkPhysicalDeviceExternalBufferInfo VkPhysicalDeviceExternalBufferInfoKHR;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkBufferCreateFlagBits describing additional parameters of the buffer, corresponding to VkBufferCreateInfo::flags.
- usage is a bitmask of VkBufferUsageFlagBits describing the intended usage of the buffer, corresponding to VkBufferCreateInfo::usage.
- handleType is a VkExternalMemoryHandleTypeFlagBits value specifying the memory handle type that will be used with the memory associated with the buffer.

Only usage flags representable in VkBufferUsageFlagBits are returned in this structure's usage. If the pNext chain includes a VkBufferUsageFlags2CreateInfoKHR structure, all usage flags of the buffer are returned in VkBufferUsageFlags2CreateInfoKHR::usage.

Valid Usage

- VUID-VkPhysicalDeviceExternalBufferInfo-None-09499
  If the pNext chain does not include a VkBufferUsageFlags2CreateInfoKHR structure, usage **must** be a valid combination of VkBufferUsageFlagBits values
- VUID-VkPhysicalDeviceExternalBufferInfo-None-09500
  If the pNext chain does not include a VkBufferUsageFlags2CreateInfoKHR structure, usage **must** not be 0

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceExternalBufferInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_BUFFER_INFO`
- VUID-VkPhysicalDeviceExternalBufferInfo-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkBufferUsageFlags2CreateInfoKHR
- VUID-VkPhysicalDeviceExternalBufferInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPhysicalDeviceExternalBufferInfo-flags-parameter
  `flags` **must** be a valid combination of VkBufferCreateFlagBits values
- VUID-VkPhysicalDeviceExternalBufferInfo-handleType-parameter
  `handleType` **must** be a valid VkExternalMemoryHandleTypeFlagBits value

The `VkExternalBufferProperties` structure is defined as:

```
typedef struct VkExternalBufferProperties {
    VkStructureType              sType;
    void*                        pNext;
    VkExternalMemoryProperties   externalMemoryProperties;
} VkExternalBufferProperties;
```

or the equivalent

```
// Provided by VK_KHR_external_memory_capabilities
typedef VkExternalBufferProperties VkExternalBufferPropertiesKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `externalMemoryProperties` is a VkExternalMemoryProperties structure specifying various capabilities of the external handle type when used with the specified buffer creation parameters.

Valid Usage (Implicit)

- VUID-VkExternalBufferProperties-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXTERNAL_BUFFER_PROPERTIES`
- VUID-VkExternalBufferProperties-pNext-pNext
  `pNext` **must** be `NULL`

43.3. Optional Semaphore Capabilities

Semaphores **may** support import and export of their payload to external handles. To query the external handle types supported by semaphores, call:

```
// Provided by VK_KHR_external_semaphore_capabilities
void vkGetPhysicalDeviceExternalSemaphorePropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceExternalSemaphoreInfo* pExternalSemaphoreInfo,
    VkExternalSemaphoreProperties*              pExternalSemaphoreProperties);
```

- `physicalDevice` is the physical device from which to query the semaphore capabilities.
- `pExternalSemaphoreInfo` is a pointer to a VkPhysicalDeviceExternalSemaphoreInfo structure describing the parameters that would be consumed by vkCreateSemaphore.
- `pExternalSemaphoreProperties` is a pointer to a VkExternalSemaphoreProperties structure in which capabilities are returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceExternalSemaphoreProperties-physicalDevice-parameter
  `physicalDevice` **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceExternalSemaphoreProperties-pExternalSemaphoreInfo-parameter
  `pExternalSemaphoreInfo` **must** be a valid pointer to a valid VkPhysicalDeviceExternalSemaphoreInfo structure
- VUID-vkGetPhysicalDeviceExternalSemaphoreProperties-pExternalSemaphoreProperties-parameter
  `pExternalSemaphoreProperties` **must** be a valid pointer to a VkExternalSemaphoreProperties structure

The `VkPhysicalDeviceExternalSemaphoreInfo` structure is defined as:

```
typedef struct VkPhysicalDeviceExternalSemaphoreInfo {
    VkStructureType                         sType;
    const void*                             pNext;
    VkExternalSemaphoreHandleTypeFlagBits   handleType;
} VkPhysicalDeviceExternalSemaphoreInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore_capabilities
typedef VkPhysicalDeviceExternalSemaphoreInfo
VkPhysicalDeviceExternalSemaphoreInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleType` is a VkExternalSemaphoreHandleTypeFlagBits value specifying the external semaphore handle type for which capabilities will be returned.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceExternalSemaphoreInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_SEMAPHORE_INFO`
- VUID-VkPhysicalDeviceExternalSemaphoreInfo-pNext-pNext
  `pNext` **must** be `NULL` or a pointer to a valid instance of VkSemaphoreTypeCreateInfo
- VUID-VkPhysicalDeviceExternalSemaphoreInfo-sType-unique
  The `sType` value of each struct in the `pNext` chain **must** be unique
- VUID-VkPhysicalDeviceExternalSemaphoreInfo-handleType-parameter
  `handleType` **must** be a valid VkExternalSemaphoreHandleTypeFlagBits value

Bits which **may** be set in VkPhysicalDeviceExternalSemaphoreInfo::`handleType`, specifying an external semaphore handle type, are:

```
typedef enum VkExternalSemaphoreHandleTypeFlagBits {
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT = 0x00000001,
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT = 0x00000002,
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT = 0x00000004,
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT = 0x00000008,
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT = 0x00000010,
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D11_FENCE_BIT =
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT_KHR =
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR =
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT_KHR =
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT_KHR =
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT_KHR =
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT,
} VkExternalSemaphoreHandleTypeFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore_capabilities
typedef VkExternalSemaphoreHandleTypeFlagBits
VkExternalSemaphoreHandleTypeFlagBitsKHR;
```

- VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT specifies a POSIX file descriptor handle that has only limited valid usage outside of Vulkan and other compatible APIs. It **must** be compatible with the POSIX system calls dup, dup2, close, and the non-standard system call dup3. Additionally, it **must** be transportable over a socket using an SCM_RIGHTS control message. It owns a reference to the underlying synchronization primitive represented by its Vulkan semaphore object.
- VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT specifies an NT handle that has only limited valid usage outside of Vulkan and other compatible APIs. It **must** be compatible with the functions DuplicateHandle, CloseHandle, CompareObjectHandles, GetHandleInformation, and SetHandleInformation. It owns a reference to the underlying synchronization primitive represented by its Vulkan semaphore object.
- VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT specifies a global share handle that has only limited valid usage outside of Vulkan and other compatible APIs. It is not compatible with any native APIs. It does not own a reference to the underlying synchronization primitive represented by its Vulkan semaphore object, and will therefore become invalid when all Vulkan semaphore objects associated with it are destroyed.
- VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT specifies an NT handle returned by ID3D12Device::CreateSharedHandle referring to a Direct3D 12 fence, or ID3D11Device5::CreateFence referring to a Direct3D 11 fence. It owns a reference to the underlying synchronization primitive associated with the Direct3D fence.
- VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D11_FENCE_BIT is an alias of VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT with the same meaning. It is provided for convenience and code clarity when interacting with D3D11 fences.
- VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT specifies a POSIX file descriptor handle to a Linux Sync File or Android Fence object. It can be used with any native API accepting a valid sync file or fence as input. It owns a reference to the underlying synchronization primitive associated with the file descriptor. Implementations which support importing this handle type **must** accept any type of sync or fence FD supported by the native system they are running on.

NOTE

Handles of type VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT generated by the implementation may represent either Linux Sync Files or Android Fences at the implementation's discretion. Applications **should** only use operations defined for both types of file descriptors, unless they know via means external to Vulkan the type of the file descriptor, or are prepared to deal with the system-defined operation failures resulting from using the wrong type.

Some external semaphore handle types can only be shared within the same underlying physical device and/or the same driver version, as defined in the following table:

Table 89. External Semaphore Handle Types Compatibility

Handle type	VkPhysicalDeviceIDProperties::driverUUID	VkPhysicalDeviceIDProperties::deviceUUID
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT	Must match	Must match
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT	Must match	Must match
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT	Must match	Must match
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT	Must match	Must match
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT	No restriction	No restriction
VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_ZIRCON_EVENT_BIT_FUCHSIA	No restriction	No restriction

```
typedef VkFlags VkExternalSemaphoreHandleTypeFlags;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore_capabilities
typedef VkExternalSemaphoreHandleTypeFlags VkExternalSemaphoreHandleTypeFlagsKHR;
```

VkExternalSemaphoreHandleTypeFlags is a bitmask type for setting a mask of zero or more VkExternalSemaphoreHandleTypeFlagBits.

The VkExternalSemaphoreProperties structure is defined as:

```
typedef struct VkExternalSemaphoreProperties {
    VkStructureType                       sType;
    void*                                 pNext;
    VkExternalSemaphoreHandleTypeFlags    exportFromImportedHandleTypes;
    VkExternalSemaphoreHandleTypeFlags    compatibleHandleTypes;
    VkExternalSemaphoreFeatureFlags       externalSemaphoreFeatures;
} VkExternalSemaphoreProperties;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore_capabilities
typedef VkExternalSemaphoreProperties VkExternalSemaphorePropertiesKHR;
```

- sType is a VkStructureType value identifying this structure.

- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `exportFromImportedHandleTypes` is a bitmask of VkExternalSemaphoreHandleTypeFlagBits specifying which types of imported handle `handleType` **can** be exported from.
- `compatibleHandleTypes` is a bitmask of VkExternalSemaphoreHandleTypeFlagBits specifying handle types which **can** be specified at the same time as `handleType` when creating a semaphore.
- `externalSemaphoreFeatures` is a bitmask of VkExternalSemaphoreFeatureFlagBits describing the features of `handleType`.

If `handleType` is not supported by the implementation, then VkExternalSemaphoreProperties ::`externalSemaphoreFeatures` will be zero.

Valid Usage (Implicit)

- VUID-VkExternalSemaphoreProperties-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXTERNAL_SEMAPHORE_PROPERTIES`
- VUID-VkExternalSemaphoreProperties-pNext-pNext
  `pNext` **must** be `NULL`

Bits which **may** be set in VkExternalSemaphoreProperties::`externalSemaphoreFeatures`, specifying the features of an external semaphore handle type, are:

```
typedef enum VkExternalSemaphoreFeatureFlagBits {
    VK_EXTERNAL_SEMAPHORE_FEATURE_EXPORTABLE_BIT = 0x00000001,
    VK_EXTERNAL_SEMAPHORE_FEATURE_IMPORTABLE_BIT = 0x00000002,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_EXTERNAL_SEMAPHORE_FEATURE_EXPORTABLE_BIT_KHR =
VK_EXTERNAL_SEMAPHORE_FEATURE_EXPORTABLE_BIT,
  // Provided by VK_KHR_external_semaphore_capabilities
    VK_EXTERNAL_SEMAPHORE_FEATURE_IMPORTABLE_BIT_KHR =
VK_EXTERNAL_SEMAPHORE_FEATURE_IMPORTABLE_BIT,
} VkExternalSemaphoreFeatureFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore_capabilities
typedef VkExternalSemaphoreFeatureFlagBits VkExternalSemaphoreFeatureFlagBitsKHR;
```

- `VK_EXTERNAL_SEMAPHORE_FEATURE_EXPORTABLE_BIT` specifies that handles of this type **can** be exported from Vulkan semaphore objects.
- `VK_EXTERNAL_SEMAPHORE_FEATURE_IMPORTABLE_BIT` specifies that handles of this type **can** be imported as Vulkan semaphore objects.

```
typedef VkFlags VkExternalSemaphoreFeatureFlags;
```

or the equivalent

```
// Provided by VK_KHR_external_semaphore_capabilities
typedef VkExternalSemaphoreFeatureFlags VkExternalSemaphoreFeatureFlagsKHR;
```

VkExternalSemaphoreFeatureFlags is a bitmask type for setting a mask of zero or more VkExternalSemaphoreFeatureFlagBits.

43.4. Optional Fence Capabilities

Fences **may** support import and export of their payload to external handles. To query the external handle types supported by fences, call:

```
// Provided by VK_KHR_external_fence_capabilities
void vkGetPhysicalDeviceExternalFencePropertiesKHR(
    VkPhysicalDevice                            physicalDevice,
    const VkPhysicalDeviceExternalFenceInfo*    pExternalFenceInfo,
    VkExternalFenceProperties*                  pExternalFenceProperties);
```

- physicalDevice is the physical device from which to query the fence capabilities.
- pExternalFenceInfo is a pointer to a VkPhysicalDeviceExternalFenceInfo structure describing the parameters that would be consumed by vkCreateFence.
- pExternalFenceProperties is a pointer to a VkExternalFenceProperties structure in which capabilities are returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceExternalFenceProperties-physicalDevice-parameter physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceExternalFenceProperties-pExternalFenceInfo-parameter pExternalFenceInfo **must** be a valid pointer to a valid VkPhysicalDeviceExternalFenceInfo structure
- VUID-vkGetPhysicalDeviceExternalFenceProperties-pExternalFenceProperties-parameter pExternalFenceProperties **must** be a valid pointer to a VkExternalFenceProperties structure

The VkPhysicalDeviceExternalFenceInfo structure is defined as:

```
typedef struct VkPhysicalDeviceExternalFenceInfo {
    VkStructureType                       sType;
    const void*                           pNext;
    VkExternalFenceHandleTypeFlagBits     handleType;
} VkPhysicalDeviceExternalFenceInfo;
```

or the equivalent

```
// Provided by VK_KHR_external_fence_capabilities
typedef VkPhysicalDeviceExternalFenceInfo VkPhysicalDeviceExternalFenceInfoKHR;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `handleType` is a VkExternalFenceHandleTypeFlagBits value specifying an external fence handle type for which capabilities will be returned.

NOTE

Handles of type `VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT` generated by the implementation may represent either Linux Sync Files or Android Fences at the implementation's discretion. Applications **should** only use operations defined for both types of file descriptors, unless they know via means external to Vulkan the type of the file descriptor, or are prepared to deal with the system-defined operation failures resulting from using the wrong type.

Valid Usage (Implicit)

- VUID-VkPhysicalDeviceExternalFenceInfo-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_FENCE_INFO`
- VUID-VkPhysicalDeviceExternalFenceInfo-pNext-pNext
  `pNext` **must** be `NULL`
- VUID-VkPhysicalDeviceExternalFenceInfo-handleType-parameter
  `handleType` **must** be a valid VkExternalFenceHandleTypeFlagBits value

Bits which **may** be set in

- VkPhysicalDeviceExternalFenceInfo::`handleType`
- VkExternalFenceProperties::`exportFromImportedHandleTypes`
- VkExternalFenceProperties::`compatibleHandleTypes`

indicate external fence handle types, and are:

```
typedef enum VkExternalFenceHandleTypeFlagBits {
    VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_FD_BIT = 0x00000001,
    VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT = 0x00000002,
    VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT = 0x00000004,
    VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT = 0x00000008,
  // Provided by VK_KHR_external_fence_capabilities
    VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_FD_BIT_KHR =
VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_FD_BIT,
  // Provided by VK_KHR_external_fence_capabilities
    VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR =
VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT,
  // Provided by VK_KHR_external_fence_capabilities
    VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT_KHR =
VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT,
  // Provided by VK_KHR_external_fence_capabilities
    VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT_KHR =
VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT,
} VkExternalFenceHandleTypeFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_external_fence_capabilities
typedef VkExternalFenceHandleTypeFlagBits VkExternalFenceHandleTypeFlagBitsKHR;
```

- VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_FD_BIT specifies a POSIX file descriptor handle that has only limited valid usage outside of Vulkan and other compatible APIs. It **must** be compatible with the POSIX system calls dup, dup2, close, and the non-standard system call dup3. Additionally, it **must** be transportable over a socket using an SCM_RIGHTS control message. It owns a reference to the underlying synchronization primitive represented by its Vulkan fence object.
- VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT specifies an NT handle that has only limited valid usage outside of Vulkan and other compatible APIs. It **must** be compatible with the functions DuplicateHandle, CloseHandle, CompareObjectHandles, GetHandleInformation, and SetHandleInformation. It owns a reference to the underlying synchronization primitive represented by its Vulkan fence object.
- VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT specifies a global share handle that has only limited valid usage outside of Vulkan and other compatible APIs. It is not compatible with any native APIs. It does not own a reference to the underlying synchronization primitive represented by its Vulkan fence object, and will therefore become invalid when all Vulkan fence objects associated with it are destroyed.
- VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT specifies a POSIX file descriptor handle to a Linux Sync File or Android Fence. It can be used with any native API accepting a valid sync file or fence as input. It owns a reference to the underlying synchronization primitive associated with the file descriptor. Implementations which support importing this handle type **must** accept any type of sync or fence FD supported by the native system they are running on.

Some external fence handle types can only be shared within the same underlying physical device and/or the same driver version, as defined in the following table:

Table 90. External Fence Handle Types Compatibility

Handle type	VkPhysicalDeviceIDProperties::driverUUID	VkPhysicalDeviceIDProperties::deviceUUID
VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_FD_BIT	Must match	Must match
VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT	Must match	Must match
VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT	Must match	Must match
VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT	No restriction	No restriction

```
typedef VkFlags VkExternalFenceHandleTypeFlags;
```

or the equivalent

```
// Provided by VK_KHR_external_fence_capabilities
typedef VkExternalFenceHandleTypeFlags VkExternalFenceHandleTypeFlagsKHR;
```

VkExternalFenceHandleTypeFlags is a bitmask type for setting a mask of zero or more VkExternalFenceHandleTypeFlagBits.

The VkExternalFenceProperties structure is defined as:

```
typedef struct VkExternalFenceProperties {
    VkStructureType                 sType;
    void*                           pNext;
    VkExternalFenceHandleTypeFlags  exportFromImportedHandleTypes;
    VkExternalFenceHandleTypeFlags  compatibleHandleTypes;
    VkExternalFenceFeatureFlags     externalFenceFeatures;
} VkExternalFenceProperties;
```

or the equivalent

```
// Provided by VK_KHR_external_fence_capabilities
typedef VkExternalFenceProperties VkExternalFencePropertiesKHR;
```

- exportFromImportedHandleTypes is a bitmask of VkExternalFenceHandleTypeFlagBits indicating which types of imported handle handleType **can** be exported from.
- compatibleHandleTypes is a bitmask of VkExternalFenceHandleTypeFlagBits specifying handle types which **can** be specified at the same time as handleType when creating a fence.

- `externalFenceFeatures` is a bitmask of VkExternalFenceFeatureFlagBits indicating the features of `handleType`.

If `handleType` is not supported by the implementation, then VkExternalFenceProperties ::`externalFenceFeatures` will be zero.

Valid Usage (Implicit)

- VUID-VkExternalFenceProperties-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_EXTERNAL_FENCE_PROPERTIES`
- VUID-VkExternalFenceProperties-pNext-pNext
  `pNext` **must** be `NULL`

Bits which **may** be set in VkExternalFenceProperties::`externalFenceFeatures`, indicating features of a fence external handle type, are:

```
typedef enum VkExternalFenceFeatureFlagBits {
    VK_EXTERNAL_FENCE_FEATURE_EXPORTABLE_BIT = 0x00000001,
    VK_EXTERNAL_FENCE_FEATURE_IMPORTABLE_BIT = 0x00000002,
  // Provided by VK_KHR_external_fence_capabilities
    VK_EXTERNAL_FENCE_FEATURE_EXPORTABLE_BIT_KHR =
VK_EXTERNAL_FENCE_FEATURE_EXPORTABLE_BIT,
  // Provided by VK_KHR_external_fence_capabilities
    VK_EXTERNAL_FENCE_FEATURE_IMPORTABLE_BIT_KHR =
VK_EXTERNAL_FENCE_FEATURE_IMPORTABLE_BIT,
} VkExternalFenceFeatureFlagBits;
```

or the equivalent

```
// Provided by VK_KHR_external_fence_capabilities
typedef VkExternalFenceFeatureFlagBits VkExternalFenceFeatureFlagBitsKHR;
```

- `VK_EXTERNAL_FENCE_FEATURE_EXPORTABLE_BIT` specifies handles of this type **can** be exported from Vulkan fence objects.
- `VK_EXTERNAL_FENCE_FEATURE_IMPORTABLE_BIT` specifies handles of this type **can** be imported to Vulkan fence objects.

```
typedef VkFlags VkExternalFenceFeatureFlags;
```

or the equivalent

```
// Provided by VK_KHR_external_fence_capabilities
typedef VkExternalFenceFeatureFlags VkExternalFenceFeatureFlagsKHR;
```

VkExternalFenceFeatureFlags is a bitmask type for setting a mask of zero or more VkExternalFenceFeatureFlagBits.

43.5. Timestamp Calibration Capabilities

To query the set of time domains for which a physical device supports timestamp calibration, call:

```
// Provided by VK_KHR_calibrated_timestamps
VkResult vkGetPhysicalDeviceCalibrateableTimeDomainsKHR(
    VkPhysicalDevice                            physicalDevice,
    uint32_t*                                   pTimeDomainCount,
    VkTimeDomainKHR*                            pTimeDomains);
```

- physicalDevice is the physical device from which to query the set of calibrateable time domains.
- pTimeDomainCount is a pointer to an integer related to the number of calibrateable time domains available or queried, as described below.
- pTimeDomains is either NULL or a pointer to an array of VkTimeDomainKHR values, indicating the supported calibrateable time domains.

If pTimeDomains is NULL, then the number of calibrateable time domains supported for the given physicalDevice is returned in pTimeDomainCount. Otherwise, pTimeDomainCount **must** point to a variable set by the application to the number of elements in the pTimeDomains array, and on return the variable is overwritten with the number of values actually written to pTimeDomains. If the value of pTimeDomainCount is less than the number of calibrateable time domains supported, at most pTimeDomainCount values will be written to pTimeDomains, and VK_INCOMPLETE will be returned instead of VK_SUCCESS, to indicate that not all the available time domains were returned.

Valid Usage (Implicit)

- VUID-vkGetPhysicalDeviceCalibrateableTimeDomainsKHR-physicalDevice-parameter
  physicalDevice **must** be a valid VkPhysicalDevice handle
- VUID-vkGetPhysicalDeviceCalibrateableTimeDomainsKHR-pTimeDomainCount-parameter
  pTimeDomainCount **must** be a valid pointer to a uint32_t value
- VUID-vkGetPhysicalDeviceCalibrateableTimeDomainsKHR-pTimeDomains-parameter
  If the value referenced by pTimeDomainCount is not 0, and pTimeDomains is not NULL, pTimeDomains **must** be a valid pointer to an array of pTimeDomainCount VkTimeDomainKHR values

Return Codes

Success

- VK_SUCCESS

- VK_INCOMPLETE

Failure

- VK_ERROR_OUT_OF_HOST_MEMORY
- VK_ERROR_OUT_OF_DEVICE_MEMORY

Chapter 44. Debugging

To aid developers in tracking down errors in the application's use of Vulkan, particularly in combination with an external debugger or profiler, *debugging extensions* may be available.

The VkObjectType enumeration defines values, each of which corresponds to a specific Vulkan handle type. These values **can** be used to associate debug information with a particular type of object through one or more extensions.

```
// Provided by VK_VERSION_1_0
typedef enum VkObjectType {
    VK_OBJECT_TYPE_UNKNOWN = 0,
    VK_OBJECT_TYPE_INSTANCE = 1,
    VK_OBJECT_TYPE_PHYSICAL_DEVICE = 2,
    VK_OBJECT_TYPE_DEVICE = 3,
    VK_OBJECT_TYPE_QUEUE = 4,
    VK_OBJECT_TYPE_SEMAPHORE = 5,
    VK_OBJECT_TYPE_COMMAND_BUFFER = 6,
    VK_OBJECT_TYPE_FENCE = 7,
    VK_OBJECT_TYPE_DEVICE_MEMORY = 8,
    VK_OBJECT_TYPE_BUFFER = 9,
    VK_OBJECT_TYPE_IMAGE = 10,
    VK_OBJECT_TYPE_EVENT = 11,
    VK_OBJECT_TYPE_QUERY_POOL = 12,
    VK_OBJECT_TYPE_BUFFER_VIEW = 13,
    VK_OBJECT_TYPE_IMAGE_VIEW = 14,
    VK_OBJECT_TYPE_SHADER_MODULE = 15,
    VK_OBJECT_TYPE_PIPELINE_CACHE = 16,
    VK_OBJECT_TYPE_PIPELINE_LAYOUT = 17,
    VK_OBJECT_TYPE_RENDER_PASS = 18,
    VK_OBJECT_TYPE_PIPELINE = 19,
    VK_OBJECT_TYPE_DESCRIPTOR_SET_LAYOUT = 20,
    VK_OBJECT_TYPE_SAMPLER = 21,
    VK_OBJECT_TYPE_DESCRIPTOR_POOL = 22,
    VK_OBJECT_TYPE_DESCRIPTOR_SET = 23,
    VK_OBJECT_TYPE_FRAMEBUFFER = 24,
    VK_OBJECT_TYPE_COMMAND_POOL = 25,
    VK_OBJECT_TYPE_SAMPLER_YCBCR_CONVERSION = 1000156000,
    VK_OBJECT_TYPE_DESCRIPTOR_UPDATE_TEMPLATE = 1000085000,
  // Provided by VK_KHR_surface
    VK_OBJECT_TYPE_SURFACE_KHR = 1000000000,
  // Provided by VK_KHR_swapchain
    VK_OBJECT_TYPE_SWAPCHAIN_KHR = 1000001000,
  // Provided by VK_KHR_display
    VK_OBJECT_TYPE_DISPLAY_KHR = 1000002000,
  // Provided by VK_KHR_display
    VK_OBJECT_TYPE_DISPLAY_MODE_KHR = 1000002001,
  // Provided by VK_KHR_video_queue
    VK_OBJECT_TYPE_VIDEO_SESSION_KHR = 1000023000,
  // Provided by VK_KHR_video_queue
```

```
    VK_OBJECT_TYPE_VIDEO_SESSION_PARAMETERS_KHR = 1000023001,
  // Provided by VK_EXT_debug_utils
    VK_OBJECT_TYPE_DEBUG_UTILS_MESSENGER_EXT = 1000128000,
  // Provided by VK_KHR_acceleration_structure
    VK_OBJECT_TYPE_ACCELERATION_STRUCTURE_KHR = 1000150000,
  // Provided by VK_EXT_validation_cache
    VK_OBJECT_TYPE_VALIDATION_CACHE_EXT = 1000160000,
  // Provided by VK_KHR_deferred_host_operations
    VK_OBJECT_TYPE_DEFERRED_OPERATION_KHR = 1000268000,
  // Provided by VK_EXT_opacity_micromap
    VK_OBJECT_TYPE_MICROMAP_EXT = 1000396000,
  // Provided by VK_EXT_shader_object
    VK_OBJECT_TYPE_SHADER_EXT = 1000482000,
  // Provided by VK_KHR_pipeline_binary
    VK_OBJECT_TYPE_PIPELINE_BINARY_KHR = 1000483000,
  // Provided by VK_EXT_device_generated_commands
    VK_OBJECT_TYPE_INDIRECT_COMMANDS_LAYOUT_EXT = 1000572000,
  // Provided by VK_EXT_device_generated_commands
    VK_OBJECT_TYPE_INDIRECT_EXECUTION_SET_EXT = 1000572001,
  // Provided by VK_KHR_descriptor_update_template
    VK_OBJECT_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_KHR =
VK_OBJECT_TYPE_DESCRIPTOR_UPDATE_TEMPLATE,
  // Provided by VK_KHR_sampler_ycbcr_conversion
    VK_OBJECT_TYPE_SAMPLER_YCBCR_CONVERSION_KHR =
VK_OBJECT_TYPE_SAMPLER_YCBCR_CONVERSION,
} VkObjectType;
```

Table 91. `VkObjectType` and Vulkan Handle Relationship

VkObjectType	Vulkan Handle Type
VK_OBJECT_TYPE_UNKNOWN	Unknown/Undefined Handle
VK_OBJECT_TYPE_INSTANCE	VkInstance
VK_OBJECT_TYPE_PHYSICAL_DEVICE	VkPhysicalDevice
VK_OBJECT_TYPE_DEVICE	VkDevice
VK_OBJECT_TYPE_QUEUE	VkQueue
VK_OBJECT_TYPE_SEMAPHORE	VkSemaphore
VK_OBJECT_TYPE_COMMAND_BUFFER	VkCommandBuffer
VK_OBJECT_TYPE_FENCE	VkFence
VK_OBJECT_TYPE_DEVICE_MEMORY	VkDeviceMemory
VK_OBJECT_TYPE_BUFFER	VkBuffer
VK_OBJECT_TYPE_IMAGE	VkImage
VK_OBJECT_TYPE_EVENT	VkEvent
VK_OBJECT_TYPE_QUERY_POOL	VkQueryPool
VK_OBJECT_TYPE_BUFFER_VIEW	VkBufferView

VkObjectType	Vulkan Handle Type
VK_OBJECT_TYPE_IMAGE_VIEW	VkImageView
VK_OBJECT_TYPE_SHADER_MODULE	VkShaderModule
VK_OBJECT_TYPE_PIPELINE_CACHE	VkPipelineCache
VK_OBJECT_TYPE_PIPELINE_LAYOUT	VkPipelineLayout
VK_OBJECT_TYPE_RENDER_PASS	VkRenderPass
VK_OBJECT_TYPE_PIPELINE	VkPipeline
VK_OBJECT_TYPE_DESCRIPTOR_SET_LAYOUT	VkDescriptorSetLayout
VK_OBJECT_TYPE_SAMPLER	VkSampler
VK_OBJECT_TYPE_DESCRIPTOR_POOL	VkDescriptorPool
VK_OBJECT_TYPE_DESCRIPTOR_SET	VkDescriptorSet
VK_OBJECT_TYPE_FRAMEBUFFER	VkFramebuffer
VK_OBJECT_TYPE_COMMAND_POOL	VkCommandPool
VK_OBJECT_TYPE_SAMPLER_YCBCR_CONVERSION	VkSamplerYcbcrConversion
VK_OBJECT_TYPE_DESCRIPTOR_UPDATE_TEMPLATE	VkDescriptorUpdateTemplate
VK_OBJECT_TYPE_SURFACE_KHR	VkSurfaceKHR
VK_OBJECT_TYPE_SWAPCHAIN_KHR	VkSwapchainKHR
VK_OBJECT_TYPE_DISPLAY_KHR	VkDisplayKHR
VK_OBJECT_TYPE_DISPLAY_MODE_KHR	VkDisplayModeKHR
VK_OBJECT_TYPE_VIDEO_SESSION_KHR	VkVideoSessionKHR
VK_OBJECT_TYPE_VIDEO_SESSION_PARAMETERS_KHR	VkVideoSessionParametersKHR
VK_OBJECT_TYPE_DEBUG_UTILS_MESSENGER_EXT	VkDebugUtilsMessengerEXT
VK_OBJECT_TYPE_ACCELERATION_STRUCTURE_KHR	VkAccelerationStructureKHR
VK_OBJECT_TYPE_VALIDATION_CACHE_EXT	VkValidationCacheEXT
VK_OBJECT_TYPE_DEFERRED_OPERATION_KHR	VkDeferredOperationKHR
VK_OBJECT_TYPE_INDIRECT_COMMANDS_LAYOUT_EXT	VkIndirectCommandsLayoutEXT
VK_OBJECT_TYPE_INDIRECT_EXECUTION_SET_EXT	VkIndirectExecutionSetEXT
VK_OBJECT_TYPE_MICROMAP_EXT	VkMicromapEXT
VK_OBJECT_TYPE_SHADER_EXT	VkShaderEXT

If this Specification was generated with any such extensions included, they will be described in the remainder of this chapter.

44.1. Debug Utilities

Vulkan provides flexible debugging utilities for debugging an application.

The Object Debug Annotation section describes how to associate either a name or binary data with a specific Vulkan object.

The Queue Labels section describes how to annotate and group the work submitted to a queue.

The Command Buffer Labels section describes how to associate logical elements of the scene with commands in a VkCommandBuffer.

The Debug Messengers section describes how to create debug messenger objects associated with an application supplied callback to capture debug messages from a variety of Vulkan components.

44.1.1. Object Debug Annotation

It can be useful for an application to provide its own content relative to a specific Vulkan object.

The following commands allow application developers to associate application-defined information with Vulkan objects. These commands are device-level commands but they **may** reference instance-level objects (such as VkInstance) and physical device-level objects (such as VkPhysicalDevice) with a few restrictions: * The data for the corresponding object **may** still be available after the VkDevice used in the corresponding API call to set it is destroyed, but access to this data is not guaranteed and should be avoided. * Subsequent calls to change the data of the same object across multiple VkDevice objects, **may** result in the data being changed to the most recent version for all VkDevice objects and not just the VkDevice used in the most recent API call.

Object Naming

An object can be given an application-defined name by calling:

```
// Provided by VK_EXT_debug_utils
VkResult vkSetDebugUtilsObjectNameEXT(
    VkDevice                                    device,
    const VkDebugUtilsObjectNameInfoEXT*        pNameInfo);
```

- device is the device that is associated with the named object passed in via objectHandle.
- pNameInfo is a pointer to a VkDebugUtilsObjectNameInfoEXT structure specifying parameters of the name to set on the object.

Valid Usage

- VUID-vkSetDebugUtilsObjectNameEXT-pNameInfo-02587
  pNameInfo->objectType **must** not be VK_OBJECT_TYPE_UNKNOWN
- VUID-vkSetDebugUtilsObjectNameEXT-pNameInfo-02588
  pNameInfo->objectHandle **must** not be VK_NULL_HANDLE

- VUID-vkSetDebugUtilsObjectNameEXT-pNameInfo-07872
  If `pNameInfo->objectHandle` is the valid handle of an instance-level object, the VkDevice identified by `device` **must** be a descendent of the same VkInstance as the object identified by `pNameInfo->objectHandle`
- VUID-vkSetDebugUtilsObjectNameEXT-pNameInfo-07873
  If `pNameInfo->objectHandle` is the valid handle of a physical-device-level object, the VkDevice identified by `device` **must** be a descendant of the same VkPhysicalDevice as the object identified by `pNameInfo->objectHandle`
- VUID-vkSetDebugUtilsObjectNameEXT-pNameInfo-07874
  If `pNameInfo->objectHandle` is the valid handle of a device-level object, that object **must** be a descendent of the VkDevice identified by `device`

Valid Usage (Implicit)

- VUID-vkSetDebugUtilsObjectNameEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkSetDebugUtilsObjectNameEXT-pNameInfo-parameter
  `pNameInfo` **must** be a valid pointer to a valid VkDebugUtilsObjectNameInfoEXT structure

Host Synchronization

- Host access to `pNameInfo->objectHandle` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`
- `VK_ERROR_OUT_OF_DEVICE_MEMORY`

The `VkDebugUtilsObjectNameInfoEXT` structure is defined as:

```
// Provided by VK_EXT_debug_utils
typedef struct VkDebugUtilsObjectNameInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkObjectType       objectType;
    uint64_t           objectHandle;
    const char*        pObjectName;
} VkDebugUtilsObjectNameInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `objectType` is a VkObjectType specifying the type of the object to be named.
- `objectHandle` is the object to be named.
- `pObjectName` is either `NULL` or a null-terminated UTF-8 string specifying the name to apply to `objectHandle`.

Applications **may** change the name associated with an object simply by calling `vkSetDebugUtilsObjectNameEXT` again with a new string. If `pObjectName` is either `NULL` or an empty string, then any previously set name is removed.

Valid Usage

- VUID-VkDebugUtilsObjectNameInfoEXT-objectType-02589
  If `objectType` is `VK_OBJECT_TYPE_UNKNOWN`, `objectHandle` **must** not be VK_NULL_HANDLE
- VUID-VkDebugUtilsObjectNameInfoEXT-objectType-02590
  If `objectType` is not `VK_OBJECT_TYPE_UNKNOWN`, `objectHandle` **must** be VK_NULL_HANDLE or a valid Vulkan handle of the type associated with `objectType` as defined in the `VkObjectType` and Vulkan Handle Relationship table

Valid Usage (Implicit)

- VUID-VkDebugUtilsObjectNameInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT`
- VUID-VkDebugUtilsObjectNameInfoEXT-objectType-parameter
  `objectType` **must** be a valid VkObjectType value
- VUID-VkDebugUtilsObjectNameInfoEXT-pObjectName-parameter
  If `pObjectName` is not `NULL`, `pObjectName` **must** be a null-terminated UTF-8 string

Object Data Association

In addition to setting a name for an object, debugging and validation layers **may** have uses for additional binary data on a per-object basis that have no other place in the Vulkan API.

For example, a `VkShaderModule` could have additional debugging data attached to it to aid in offline shader tracing.

Additional data can be attached to an object by calling `vkSetDebugUtilsObjectTagEXT` as defined below.

```
// Provided by VK_EXT_debug_utils
VkResult vkSetDebugUtilsObjectTagEXT(
    VkDevice                                    device,
    const VkDebugUtilsObjectTagInfoEXT*         pTagInfo);
```

- `device` is the device that created the object.
- `pTagInfo` is a pointer to a VkDebugUtilsObjectTagInfoEXT structure specifying parameters of the tag to attach to the object.

Valid Usage

- VUID-vkSetDebugUtilsObjectTagEXT-pNameInfo-07875
  If `pNameInfo->objectHandle` is the valid handle of an instance-level object, the VkDevice identified by `device` **must** be a descendent of the same VkInstance as the object identified by `pNameInfo->objectHandle`
- VUID-vkSetDebugUtilsObjectTagEXT-pNameInfo-07876
  If `pNameInfo->objectHandle` is the valid handle of a physical-device-level object, the VkDevice identified by `device` **must** be a descendant of the same VkPhysicalDevice as the object identified by `pNameInfo->objectHandle`
- VUID-vkSetDebugUtilsObjectTagEXT-pNameInfo-07877
  If `pNameInfo->objectHandle` is the valid handle of a device-level object, that object **must** be a descendent of the VkDevice identified by `device`

Valid Usage (Implicit)

- VUID-vkSetDebugUtilsObjectTagEXT-device-parameter
  `device` **must** be a valid VkDevice handle
- VUID-vkSetDebugUtilsObjectTagEXT-pTagInfo-parameter
  `pTagInfo` **must** be a valid pointer to a valid VkDebugUtilsObjectTagInfoEXT structure

Host Synchronization

- Host access to `pTagInfo->objectHandle` **must** be externally synchronized

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

- VK_ERROR_OUT_OF_DEVICE_MEMORY

The VkDebugUtilsObjectTagInfoEXT structure is defined as:

```
// Provided by VK_EXT_debug_utils
typedef struct VkDebugUtilsObjectTagInfoEXT {
    VkStructureType    sType;
    const void*        pNext;
    VkObjectType       objectType;
    uint64_t           objectHandle;
    uint64_t           tagName;
    size_t             tagSize;
    const void*        pTag;
} VkDebugUtilsObjectTagInfoEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- objectType is a VkObjectType specifying the type of the object to be named.
- objectHandle is the object to be tagged.
- tagName is a numerical identifier of the tag.
- tagSize is the number of bytes of data to attach to the object.
- pTag is a pointer to an array of tagSize bytes containing the data to be associated with the object.

The tagName parameter gives a name or identifier to the type of data being tagged. This can be used by debugging layers to easily filter for only data that can be used by that implementation.

Valid Usage

- VUID-VkDebugUtilsObjectTagInfoEXT-objectType-01908
  objectType **must** not be VK_OBJECT_TYPE_UNKNOWN
- VUID-VkDebugUtilsObjectTagInfoEXT-objectHandle-01910
  objectHandle **must** be a valid Vulkan handle of the type associated with objectType as defined in the VkObjectType and Vulkan Handle Relationship table

Valid Usage (Implicit)

- VUID-VkDebugUtilsObjectTagInfoEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEBUG_UTILS_OBJECT_TAG_INFO_EXT
- VUID-VkDebugUtilsObjectTagInfoEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkDebugUtilsObjectTagInfoEXT-objectType-parameter
  objectType **must** be a valid VkObjectType value

- VUID-VkDebugUtilsObjectTagInfoEXT-pTag-parameter
  `pTag` **must** be a valid pointer to an array of `tagSize` bytes
- VUID-VkDebugUtilsObjectTagInfoEXT-tagSize-arraylength
  `tagSize` **must** be greater than `0`

44.1.2. Queue Labels

All Vulkan work is submitted using queues. It is possible for an application to use multiple queues, each containing multiple command buffers, when performing work. It can be useful to identify which queue, or even where in a queue, something has occurred.

To begin identifying a region using a debug label inside a queue, you may use the vkQueueBeginDebugUtilsLabelEXT command.

Then, when the region of interest has passed, you may end the label region using vkQueueEndDebugUtilsLabelEXT.

Additionally, a single debug label may be inserted at any time using vkQueueInsertDebugUtilsLabelEXT.

A queue debug label region is opened by calling:

```
// Provided by VK_EXT_debug_utils
void vkQueueBeginDebugUtilsLabelEXT(
    VkQueue                                     queue,
    const VkDebugUtilsLabelEXT*                 pLabelInfo);
```

- `queue` is the queue in which to start a debug label region.
- `pLabelInfo` is a pointer to a VkDebugUtilsLabelEXT structure specifying parameters of the label region to open.

Valid Usage (Implicit)

- VUID-vkQueueBeginDebugUtilsLabelEXT-queue-parameter
  `queue` **must** be a valid VkQueue handle
- VUID-vkQueueBeginDebugUtilsLabelEXT-pLabelInfo-parameter
  `pLabelInfo` **must** be a valid pointer to a valid VkDebugUtilsLabelEXT structure

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
-	-	-	Any	-

The VkDebugUtilsLabelEXT structure is defined as:

```
// Provided by VK_EXT_debug_utils
typedef struct VkDebugUtilsLabelEXT {
    VkStructureType    sType;
    const void*        pNext;
    const char*        pLabelName;
    float              color[4];
} VkDebugUtilsLabelEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- pLabelName is a pointer to a null-terminated UTF-8 string containing the name of the label.
- color is an optional RGBA color value that can be associated with the label. A particular implementation **may** choose to ignore this color value. The values contain RGBA values in order, in the range 0.0 to 1.0. If all elements in color are 0.0, then it is ignored.

Valid Usage (Implicit)

- VUID-VkDebugUtilsLabelEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEBUG_UTILS_LABEL_EXT
- VUID-VkDebugUtilsLabelEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkDebugUtilsLabelEXT-pLabelName-parameter
  pLabelName **must** be a null-terminated UTF-8 string

A queue debug label region is closed by calling:

```
// Provided by VK_EXT_debug_utils
void vkQueueEndDebugUtilsLabelEXT(
    VkQueue                                     queue);
```

- queue is the queue in which a debug label region should be closed.

The calls to vkQueueBeginDebugUtilsLabelEXT and vkQueueEndDebugUtilsLabelEXT **must** be matched and balanced.

Valid Usage

- VUID-vkQueueEndDebugUtilsLabelEXT-None-01911
  There **must** be an outstanding vkQueueBeginDebugUtilsLabelEXT command prior to the vkQueueEndDebugUtilsLabelEXT on the queue

Valid Usage (Implicit)

- VUID-vkQueueEndDebugUtilsLabelEXT-queue-parameter
  `queue` **must** be a valid VkQueue handle

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
-	-	-	Any	-

A single label can be inserted into a queue by calling:

```
// Provided by VK_EXT_debug_utils
void vkQueueInsertDebugUtilsLabelEXT(
    VkQueue                                     queue,
    const VkDebugUtilsLabelEXT*                 pLabelInfo);
```

- `queue` is the queue into which a debug label will be inserted.
- `pLabelInfo` is a pointer to a VkDebugUtilsLabelEXT structure specifying parameters of the label to insert.

Valid Usage (Implicit)

- VUID-vkQueueInsertDebugUtilsLabelEXT-queue-parameter
  `queue` **must** be a valid VkQueue handle
- VUID-vkQueueInsertDebugUtilsLabelEXT-pLabelInfo-parameter
  `pLabelInfo` **must** be a valid pointer to a valid VkDebugUtilsLabelEXT structure

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
-	-	-	Any	-

44.1.3. Command Buffer Labels

Typical Vulkan applications will submit many command buffers in each frame, with each command buffer containing a large number of individual commands. Being able to logically annotate regions of command buffers that belong together as well as hierarchically subdivide the

frame is important to a developer's ability to navigate the commands viewed holistically.

To identify the beginning of a debug label region in a command buffer, vkCmdBeginDebugUtilsLabelEXT **can** be used as defined below.

To indicate the end of a debug label region in a command buffer, vkCmdEndDebugUtilsLabelEXT **can** be used.

To insert a single command buffer debug label inside of a command buffer, vkCmdInsertDebugUtilsLabelEXT **can** be used as defined below.

A command buffer debug label region can be opened by calling:

```
// Provided by VK_EXT_debug_utils
void vkCmdBeginDebugUtilsLabelEXT(
    VkCommandBuffer                             commandBuffer,
    const VkDebugUtilsLabelEXT*                 pLabelInfo);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `pLabelInfo` is a pointer to a VkDebugUtilsLabelEXT structure specifying parameters of the label region to open.

Valid Usage (Implicit)

- VUID-vkCmdBeginDebugUtilsLabelEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdBeginDebugUtilsLabelEXT-pLabelInfo-parameter
  `pLabelInfo` **must** be a valid pointer to a valid VkDebugUtilsLabelEXT structure
- VUID-vkCmdBeginDebugUtilsLabelEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdBeginDebugUtilsLabelEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdBeginDebugUtilsLabelEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	Action State

A command buffer label region can be closed by calling:

```
// Provided by VK_EXT_debug_utils
void vkCmdEndDebugUtilsLabelEXT(
    VkCommandBuffer                             commandBuffer);
```

- `commandBuffer` is the command buffer into which the command is recorded.

An application **may** open a debug label region in one command buffer and close it in another, or otherwise split debug label regions across multiple command buffers or multiple queue submissions. When viewed from the linear series of submissions to a single queue, the calls to vkCmdBeginDebugUtilsLabelEXT and vkCmdEndDebugUtilsLabelEXT **must** be matched and balanced.

There **can** be problems reporting command buffer debug labels during the recording process because command buffers **may** be recorded out of sequence with the resulting execution order. Since the recording order **may** be different, a solitary command buffer **may** have an inconsistent view of the debug label regions by itself. Therefore, if an issue occurs during the recording of a command buffer, and the environment requires returning debug labels, the implementation **may** return only those labels it is aware of. This is true even if the implementation is aware of only the debug labels within the command buffer being actively recorded.

Valid Usage

- VUID-vkCmdEndDebugUtilsLabelEXT-commandBuffer-01912
  There **must** be an outstanding `vkCmdBeginDebugUtilsLabelEXT` command prior to the `vkCmdEndDebugUtilsLabelEXT` on the queue that `commandBuffer` is submitted to
- VUID-vkCmdEndDebugUtilsLabelEXT-commandBuffer-01913
  If `commandBuffer` is a secondary command buffer, there **must** be an outstanding `vkCmdBeginDebugUtilsLabelEXT` command recorded to `commandBuffer` that has not previously been ended by a call to `vkCmdEndDebugUtilsLabelEXT`

Valid Usage (Implicit)

- VUID-vkCmdEndDebugUtilsLabelEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle

- VUID-vkCmdEndDebugUtilsLabelEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state
- VUID-vkCmdEndDebugUtilsLabelEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdEndDebugUtilsLabelEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	Action State

A single debug label can be inserted into a command buffer by calling:

```
// Provided by VK_EXT_debug_utils
void vkCmdInsertDebugUtilsLabelEXT(
    VkCommandBuffer                             commandBuffer,
    const VkDebugUtilsLabelEXT*                 pLabelInfo);
```

- `commandBuffer` is the command buffer into which the command is recorded.
- `pInfo` is a pointer to a VkDebugUtilsLabelEXT structure specifying parameters of the label to insert.

Valid Usage (Implicit)

- VUID-vkCmdInsertDebugUtilsLabelEXT-commandBuffer-parameter
  `commandBuffer` **must** be a valid VkCommandBuffer handle
- VUID-vkCmdInsertDebugUtilsLabelEXT-pLabelInfo-parameter
  `pLabelInfo` **must** be a valid pointer to a valid VkDebugUtilsLabelEXT structure
- VUID-vkCmdInsertDebugUtilsLabelEXT-commandBuffer-recording
  `commandBuffer` **must** be in the recording state

- VUID-vkCmdInsertDebugUtilsLabelEXT-commandBuffer-cmdpool
  The `VkCommandPool` that `commandBuffer` was allocated from **must** support graphics, or compute operations
- VUID-vkCmdInsertDebugUtilsLabelEXT-videocoding
  This command **must** only be called outside of a video coding scope

Host Synchronization

- Host access to `commandBuffer` **must** be externally synchronized
- Host access to the `VkCommandPool` that `commandBuffer` was allocated from **must** be externally synchronized

Command Properties

Command Buffer Levels	Render Pass Scope	Video Coding Scope	Supported Queue Types	Command Type
Primary Secondary	Both	Outside	Graphics Compute	Action

44.1.4. Debug Messengers

Vulkan allows an application to register multiple callbacks with any Vulkan component wishing to report debug information. Some callbacks may log the information to a file, others may cause a debug break point or other application-defined behavior. A primary producer of callback messages are the validation layers. An application **can** register callbacks even when no validation layers are enabled, but they will only be called for the Vulkan loader and, if implemented, other layer and driver events.

A `VkDebugUtilsMessengerEXT` is a messenger object which handles passing along debug messages to a provided debug callback.

```
// Provided by VK_EXT_debug_utils
VK_DEFINE_NON_DISPATCHABLE_HANDLE(VkDebugUtilsMessengerEXT)
```

The debug messenger will provide detailed feedback on the application's use of Vulkan when events of interest occur. When an event of interest does occur, the debug messenger will submit a debug message to the debug callback that was provided during its creation. Additionally, the debug messenger is responsible with filtering out debug messages that the callback is not interested in and will only provide desired debug messages.

A debug messenger triggers a debug callback with a debug message when an event of interest occurs. To create a debug messenger which will trigger a debug callback, call:

```
// Provided by VK_EXT_debug_utils
VkResult vkCreateDebugUtilsMessengerEXT(
    VkInstance                                  instance,
    const VkDebugUtilsMessengerCreateInfoEXT*   pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkDebugUtilsMessengerEXT*                   pMessenger);
```

- `instance` is the instance the messenger will be used with.
- `pCreateInfo` is a pointer to a VkDebugUtilsMessengerCreateInfoEXT structure containing the callback pointer, as well as defining conditions under which this messenger will trigger the callback.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.
- `pMessenger` is a pointer to a VkDebugUtilsMessengerEXT handle in which the created object is returned.

Valid Usage (Implicit)

- VUID-vkCreateDebugUtilsMessengerEXT-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkCreateDebugUtilsMessengerEXT-pCreateInfo-parameter
  `pCreateInfo` **must** be a valid pointer to a valid VkDebugUtilsMessengerCreateInfoEXT structure
- VUID-vkCreateDebugUtilsMessengerEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkCreateDebugUtilsMessengerEXT-pMessenger-parameter
  `pMessenger` **must** be a valid pointer to a VkDebugUtilsMessengerEXT handle

Return Codes

Success

- `VK_SUCCESS`

Failure

- `VK_ERROR_OUT_OF_HOST_MEMORY`

The application **must** ensure that vkCreateDebugUtilsMessengerEXT is not executed in parallel with any Vulkan command that is also called with `instance` or child of `instance` as the dispatchable argument.

The definition of `VkDebugUtilsMessengerCreateInfoEXT` is:

```
// Provided by VK_EXT_debug_utils
typedef struct VkDebugUtilsMessengerCreateInfoEXT {
    VkStructureType                         sType;
    const void*                             pNext;
    VkDebugUtilsMessengerCreateFlagsEXT     flags;
    VkDebugUtilsMessageSeverityFlagsEXT     messageSeverity;
    VkDebugUtilsMessageTypeFlagsEXT         messageType;
    PFN_vkDebugUtilsMessengerCallbackEXT    pfnUserCallback;
    void*                                   pUserData;
} VkDebugUtilsMessengerCreateInfoEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is `0` and is reserved for future use.
- `messageSeverity` is a bitmask of VkDebugUtilsMessageSeverityFlagBitsEXT specifying which severity of event(s) will cause this callback to be called.
- `messageType` is a bitmask of VkDebugUtilsMessageTypeFlagBitsEXT specifying which type of event(s) will cause this callback to be called.
- `pfnUserCallback` is the application callback function to call.
- `pUserData` is user data to be passed to the callback.

For each `VkDebugUtilsMessengerEXT` that is created the `VkDebugUtilsMessengerCreateInfoEXT::messageSeverity` and `VkDebugUtilsMessengerCreateInfoEXT::messageType` determine when that `VkDebugUtilsMessengerCreateInfoEXT::pfnUserCallback` is called. The process to determine if the user's `pfnUserCallback` is triggered when an event occurs is as follows:

1. The implementation will perform a bitwise AND of the event's VkDebugUtilsMessageSeverityFlagBitsEXT with the `messageSeverity` provided during creation of the VkDebugUtilsMessengerEXT object.
    a. If the value is 0, the message is skipped.
2. The implementation will perform bitwise AND of the event's VkDebugUtilsMessageTypeFlagBitsEXT with the `messageType` provided during the creation of the VkDebugUtilsMessengerEXT object.
    a. If the value is 0, the message is skipped.
3. The callback will trigger a debug message for the current event

The callback will come directly from the component that detected the event, unless some other layer intercepts the calls for its own purposes (filter them in a different way, log to a system error log, etc.).

An application **can** receive multiple callbacks if multiple `VkDebugUtilsMessengerEXT` objects are created. A callback will always be executed in the same thread as the originating Vulkan call.

A callback **can** be called from multiple threads simultaneously (if the application is making Vulkan

calls from multiple threads).

Valid Usage (Implicit)

- VUID-VkDebugUtilsMessengerCreateInfoEXT-sType-sType
  `sType` **must** be `VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT`
- VUID-VkDebugUtilsMessengerCreateInfoEXT-flags-zerobitmask
  `flags` **must** be `0`
- VUID-VkDebugUtilsMessengerCreateInfoEXT-messageSeverity-parameter
  `messageSeverity` **must** be a valid combination of VkDebugUtilsMessageSeverityFlagBitsEXT values
- VUID-VkDebugUtilsMessengerCreateInfoEXT-messageSeverity-requiredbitmask
  `messageSeverity` **must** not be `0`
- VUID-VkDebugUtilsMessengerCreateInfoEXT-messageType-parameter
  `messageType` **must** be a valid combination of VkDebugUtilsMessageTypeFlagBitsEXT values
- VUID-VkDebugUtilsMessengerCreateInfoEXT-messageType-requiredbitmask
  `messageType` **must** not be `0`
- VUID-VkDebugUtilsMessengerCreateInfoEXT-pfnUserCallback-parameter
  `pfnUserCallback` **must** be a valid PFN_vkDebugUtilsMessengerCallbackEXT value

```
// Provided by VK_EXT_debug_utils
typedef VkFlags VkDebugUtilsMessengerCreateFlagsEXT;
```

`VkDebugUtilsMessengerCreateFlagsEXT` is a bitmask type for setting a mask, but is currently reserved for future use.

Bits which **can** be set in VkDebugUtilsMessengerCreateInfoEXT::`messageSeverity`, specifying event severities which cause a debug messenger to call the callback, are:

```
// Provided by VK_EXT_debug_utils
typedef enum VkDebugUtilsMessageSeverityFlagBitsEXT {
    VK_DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT = 0x00000001,
    VK_DEBUG_UTILS_MESSAGE_SEVERITY_INFO_BIT_EXT = 0x00000010,
    VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT = 0x00000100,
    VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT = 0x00001000,
} VkDebugUtilsMessageSeverityFlagBitsEXT;
```

- `VK_DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT` specifies the most verbose output indicating all diagnostic messages from the Vulkan loader, layers, and drivers should be captured.
- `VK_DEBUG_UTILS_MESSAGE_SEVERITY_INFO_BIT_EXT` specifies an informational message such as resource details that may be handy when debugging an application.
- `VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT` specifies use of Vulkan that **may** expose an

application bug. Such cases may not be immediately harmful, such as a fragment shader outputting to a location with no attachment. Other cases **may** point to behavior that is almost certainly bad when unintended such as using an image whose memory has not been filled. In general if you see a warning but you know that the behavior is intended/desired, then simply ignore the warning.

- VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT specifies that the application has violated a valid usage condition of the specification.

NOTE

The values of VkDebugUtilsMessageSeverityFlagBitsEXT are sorted based on severity. The higher the flag value, the more severe the message. This allows for simple boolean operation comparisons when looking at VkDebugUtilsMessageSeverityFlagBitsEXT values.

For example:

```
    if (messageSeverity >=
VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT) {
        // Do something for warnings and errors
    }
```

In addition, space has been left between the enums to allow for later addition of new severities in between the existing values.

```
// Provided by VK_EXT_debug_utils
typedef VkFlags VkDebugUtilsMessageSeverityFlagsEXT;
```

VkDebugUtilsMessageSeverityFlagsEXT is a bitmask type for setting a mask of zero or more VkDebugUtilsMessageSeverityFlagBitsEXT.

Bits which **can** be set in VkDebugUtilsMessengerCreateInfoEXT::messageType, specifying event types which cause a debug messenger to call the callback, are:

```
// Provided by VK_EXT_debug_utils
typedef enum VkDebugUtilsMessageTypeFlagBitsEXT {
    VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT = 0x00000001,
    VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT = 0x00000002,
    VK_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT = 0x00000004,
} VkDebugUtilsMessageTypeFlagBitsEXT;
```

- VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT specifies that some general event has occurred. This is typically a non-specification, non-performance event.
- VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT specifies that something has occurred during validation against the Vulkan specification that may indicate invalid behavior.
- VK_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT specifies a potentially non-optimal use of Vulkan, e.g. using vkCmdClearColorImage when setting VkAttachmentDescription::loadOp to

VK_ATTACHMENT_LOAD_OP_CLEAR would have worked.

```
// Provided by VK_EXT_debug_utils
typedef VkFlags VkDebugUtilsMessageTypeFlagsEXT;
```

VkDebugUtilsMessageTypeFlagsEXT is a bitmask type for setting a mask of zero or more VkDebugUtilsMessageTypeFlagBitsEXT.

The prototype for the VkDebugUtilsMessengerCreateInfoEXT::pfnUserCallback function implemented by the application is:

```
// Provided by VK_EXT_debug_utils
typedef VkBool32 (VKAPI_PTR *PFN_vkDebugUtilsMessengerCallbackEXT)(
    VkDebugUtilsMessageSeverityFlagBitsEXT           messageSeverity,
    VkDebugUtilsMessageTypeFlagsEXT                  messageTypes,
    const VkDebugUtilsMessengerCallbackDataEXT*      pCallbackData,
    void*                                            pUserData);
```

- messageSeverity specifies the VkDebugUtilsMessageSeverityFlagBitsEXT that triggered this callback.
- messageTypes is a bitmask of VkDebugUtilsMessageTypeFlagBitsEXT specifying which type of event(s) triggered this callback.
- pCallbackData contains all the callback related data in the VkDebugUtilsMessengerCallbackDataEXT structure.
- pUserData is the user data provided when the VkDebugUtilsMessengerEXT was created.

The callback returns a VkBool32, which is interpreted in a layer-specified manner. The application **should** always return VK_FALSE. The VK_TRUE value is reserved for use in layer development.

Valid Usage

- VUID-PFN_vkDebugUtilsMessengerCallbackEXT-None-04769
  The callback **must** not make calls to any Vulkan commands

The definition of VkDebugUtilsMessengerCallbackDataEXT is:

```
// Provided by VK_EXT_debug_utils
typedef struct VkDebugUtilsMessengerCallbackDataEXT {
    VkStructureType                                 sType;
    const void*                                     pNext;
    VkDebugUtilsMessengerCallbackDataFlagsEXT       flags;
    const char*                                     pMessageIdName;
    int32_t                                         messageIdNumber;
    const char*                                     pMessage;
    uint32_t                                        queueLabelCount;
    const VkDebugUtilsLabelEXT*                     pQueueLabels;
    uint32_t                                        cmdBufLabelCount;
    const VkDebugUtilsLabelEXT*                     pCmdBufLabels;
    uint32_t                                        objectCount;
    const VkDebugUtilsObjectNameInfoEXT*            pObjects;
} VkDebugUtilsMessengerCallbackDataEXT;
```

- `sType` is a VkStructureType value identifying this structure.
- `pNext` is `NULL` or a pointer to a structure extending this structure.
- `flags` is `0` and is reserved for future use.
- `pMessageIdName` is `NULL` or a null-terminated UTF-8 string that identifies the particular message ID that is associated with the provided message. If the message corresponds to a validation layer message, then this string may contain the portion of the Vulkan specification that is believed to have been violated.
- `messageIdNumber` is the ID number of the triggering message. If the message corresponds to a validation layer message, then this number is related to the internal number associated with the message being triggered.
- `pMessage` is a null-terminated UTF-8 string detailing the trigger conditions.
- `queueLabelCount` is a count of items contained in the `pQueueLabels` array.
- `pQueueLabels` is `NULL` or a pointer to an array of VkDebugUtilsLabelEXT active in the current `VkQueue` at the time the callback was triggered. Refer to Queue Labels for more information.
- `cmdBufLabelCount` is a count of items contained in the `pCmdBufLabels` array.
- `pCmdBufLabels` is `NULL` or a pointer to an array of VkDebugUtilsLabelEXT active in the current `VkCommandBuffer` at the time the callback was triggered. Refer to Command Buffer Labels for more information.
- `objectCount` is a count of items contained in the `pObjects` array.
- `pObjects` is a pointer to an array of VkDebugUtilsObjectNameInfoEXT objects related to the detected issue. The array is roughly in order or importance, but the 0th element is always guaranteed to be the most important object for this message.

NOTE	This structure should only be considered valid during the lifetime of the triggered callback.

Since adding queue and command buffer labels behaves like pushing and popping onto a stack, the

order of both pQueueLabels and pCmdBufLabels is based on the order the labels were defined. The result is that the first label in either pQueueLabels or pCmdBufLabels will be the first defined (and therefore the oldest) while the last label in each list will be the most recent.

NOTE

pQueueLabels will only be non-NULL if one of the objects in pObjects can be related directly to a defined VkQueue which has had one or more labels associated with it.

Likewise, pCmdBufLabels will only be non-NULL if one of the objects in pObjects can be related directly to a defined VkCommandBuffer which has had one or more labels associated with it. Additionally, while command buffer labels allow for beginning and ending across different command buffers, the debug messaging framework **cannot** guarantee that labels in pCmdBufLables will contain those defined outside of the associated command buffer. This is partially due to the fact that the association of one command buffer with another may not have been defined at the time the debug message is triggered.

Valid Usage (Implicit)

- VUID-VkDebugUtilsMessengerCallbackDataEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CALLBACK_DATA_EXT
- VUID-VkDebugUtilsMessengerCallbackDataEXT-pNext-pNext
  pNext **must** be NULL
- VUID-VkDebugUtilsMessengerCallbackDataEXT-flags-zerobitmask
  flags **must** be 0
- VUID-VkDebugUtilsMessengerCallbackDataEXT-pMessageIdName-parameter
  If pMessageIdName is not NULL, pMessageIdName **must** be a null-terminated UTF-8 string
- VUID-VkDebugUtilsMessengerCallbackDataEXT-pMessage-parameter
  If pMessage is not NULL, pMessage **must** be a null-terminated UTF-8 string
- VUID-VkDebugUtilsMessengerCallbackDataEXT-pQueueLabels-parameter
  If queueLabelCount is not 0, pQueueLabels **must** be a valid pointer to an array of queueLabelCount valid VkDebugUtilsLabelEXT structures
- VUID-VkDebugUtilsMessengerCallbackDataEXT-pCmdBufLabels-parameter
  If cmdBufLabelCount is not 0, pCmdBufLabels **must** be a valid pointer to an array of cmdBufLabelCount valid VkDebugUtilsLabelEXT structures
- VUID-VkDebugUtilsMessengerCallbackDataEXT-pObjects-parameter
  If objectCount is not 0, pObjects **must** be a valid pointer to an array of objectCount valid VkDebugUtilsObjectNameInfoEXT structures

```
// Provided by VK_EXT_debug_utils
typedef VkFlags VkDebugUtilsMessengerCallbackDataFlagsEXT;
```

VkDebugUtilsMessengerCallbackDataFlagsEXT is a bitmask type for setting a mask, but is currently reserved for future use.

To intentionally submit a debug message, call:

```
// Provided by VK_EXT_debug_utils
void vkSubmitDebugUtilsMessageEXT(
    VkInstance                                  instance,
    VkDebugUtilsMessageSeverityFlagBitsEXT      messageSeverity,
    VkDebugUtilsMessageTypeFlagsEXT             messageTypes,
    const VkDebugUtilsMessengerCallbackDataEXT* pCallbackData);
```

- `instance` is the debug stream's VkInstance.
- `messageSeverity` is a VkDebugUtilsMessageSeverityFlagBitsEXT value specifying the severity of this event/message.
- `messageTypes` is a bitmask of VkDebugUtilsMessageTypeFlagBitsEXT specifying which type of event(s) to identify with this message.
- `pCallbackData` contains all the callback related data in the VkDebugUtilsMessengerCallbackDataEXT structure.

The call will propagate through the layers and generate callback(s) as indicated by the message's flags. The parameters are passed on to the callback in addition to the `pUserData` value that was defined at the time the messenger was registered.

Valid Usage

- VUID-vkSubmitDebugUtilsMessageEXT-objectType-02591
  The `objectType` member of each element of `pCallbackData->pObjects` **must** not be `VK_OBJECT_TYPE_UNKNOWN`

Valid Usage (Implicit)

- VUID-vkSubmitDebugUtilsMessageEXT-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkSubmitDebugUtilsMessageEXT-messageSeverity-parameter
  `messageSeverity` **must** be a valid VkDebugUtilsMessageSeverityFlagBitsEXT value
- VUID-vkSubmitDebugUtilsMessageEXT-messageTypes-parameter
  `messageTypes` **must** be a valid combination of VkDebugUtilsMessageTypeFlagBitsEXT values
- VUID-vkSubmitDebugUtilsMessageEXT-messageTypes-requiredbitmask
  `messageTypes` **must** not be `0`
- VUID-vkSubmitDebugUtilsMessageEXT-pCallbackData-parameter
  `pCallbackData` **must** be a valid pointer to a valid VkDebugUtilsMessengerCallbackDataEXT structure

To destroy a `VkDebugUtilsMessengerEXT` object, call:

```
// Provided by VK_EXT_debug_utils
void vkDestroyDebugUtilsMessengerEXT(
    VkInstance                                  instance,
    VkDebugUtilsMessengerEXT                    messenger,
    const VkAllocationCallbacks*                pAllocator);
```

- `instance` is the instance where the callback was created.
- `messenger` is the VkDebugUtilsMessengerEXT object to destroy. `messenger` is an externally synchronized object and **must** not be used on more than one thread at a time. This means that `vkDestroyDebugUtilsMessengerEXT` **must** not be called when a callback is active.
- `pAllocator` controls host memory allocation as described in the Memory Allocation chapter.

Valid Usage

- VUID-vkDestroyDebugUtilsMessengerEXT-messenger-01915
  If `VkAllocationCallbacks` were provided when `messenger` was created, a compatible set of callbacks **must** be provided here
- VUID-vkDestroyDebugUtilsMessengerEXT-messenger-01916
  If no `VkAllocationCallbacks` were provided when `messenger` was created, `pAllocator` **must** be `NULL`

Valid Usage (Implicit)

- VUID-vkDestroyDebugUtilsMessengerEXT-instance-parameter
  `instance` **must** be a valid VkInstance handle
- VUID-vkDestroyDebugUtilsMessengerEXT-messenger-parameter
  If `messenger` is not VK_NULL_HANDLE, `messenger` **must** be a valid VkDebugUtilsMessengerEXT handle
- VUID-vkDestroyDebugUtilsMessengerEXT-pAllocator-parameter
  If `pAllocator` is not `NULL`, `pAllocator` **must** be a valid pointer to a valid VkAllocationCallbacks structure
- VUID-vkDestroyDebugUtilsMessengerEXT-messenger-parent
  If `messenger` is a valid handle, it **must** have been created, allocated, or retrieved from `instance`

Host Synchronization

- Host access to `messenger` **must** be externally synchronized

The application **must** ensure that vkDestroyDebugUtilsMessengerEXT is not executed in parallel with any Vulkan command that is also called with `instance` or child of `instance` as the dispatchable

argument.

44.2. Frame Boundary

The VkFrameBoundaryEXT structure is defined as:

```
// Provided by VK_EXT_frame_boundary
typedef struct VkFrameBoundaryEXT {
    VkStructureType            sType;
    const void*                pNext;
    VkFrameBoundaryFlagsEXT    flags;
    uint64_t                   frameID;
    uint32_t                   imageCount;
    const VkImage*             pImages;
    uint32_t                   bufferCount;
    const VkBuffer*            pBuffers;
    uint64_t                   tagName;
    size_t                     tagSize;
    const void*                pTag;
} VkFrameBoundaryEXT;
```

- sType is a VkStructureType value identifying this structure.
- pNext is NULL or a pointer to a structure extending this structure.
- flags is a bitmask of VkFrameBoundaryFlagBitsEXT that can flag the last submission of a frame identifier.
- frameID is the frame identifier.
- imageCount is the number of images that store frame results.
- pImages is a pointer to an array of VkImage objects with imageCount entries.
- bufferCount is the number of buffers the store the frame results.
- pBuffers is a pointer to an array of VkBuffer objects with bufferCount entries.
- tagName is a numerical identifier for tag data.
- tagSize is the number of bytes of tag data.
- pTag is a pointer to an array of tagSize bytes containing tag data.

The application **can** associate frame boundary information to a queue submission call by adding a VkFrameBoundaryEXT structure to the pNext chain of queue submission, VkPresentInfoKHR, or VkBindSparseInfo.

NOTE

The frame identifier is used to associate one or more queue submissions to a frame. It is meant to be unique within a frame lifetime, i.e. it is possible (though not recommended) to reuse frame identifiers, as long as any two frames that may have overlapping queue submissions (as in the example above) use different frame identifiers.

> Since the concept of frame is application-dependent, there is no way to validate the use of frame identifier. It is good practice to use a monotonically increasing counter as the frame identifier and not reuse identifiers between frames.

The pImages and pBuffers arrays contain a list of images and buffers which store the "end result" of the frame. As the concept of frame is application-dependent, not all frames **may** produce their results in images or buffers, yet this is a sufficiently common case to be handled by VkFrameBoundaryEXT. Note that no extra information, such as image layout is being provided, since the images are meant to be used by tools which would already be tracking this required information. Having the possibility of passing a list of end-result images makes VkFrameBoundaryEXT as expressive as vkQueuePresentKHR, which is often the default frame boundary delimiter.

The application **can** also associate arbitrary extra information via tag data using tagName, tagSize and pTag. This extra information is typically tool-specific.

Valid Usage (Implicit)

- VUID-VkFrameBoundaryEXT-sType-sType
  sType **must** be VK_STRUCTURE_TYPE_FRAME_BOUNDARY_EXT
- VUID-VkFrameBoundaryEXT-flags-parameter
  flags **must** be a valid combination of VkFrameBoundaryFlagBitsEXT values
- VUID-VkFrameBoundaryEXT-pImages-parameter
  If imageCount is not 0, and pImages is not NULL, pImages **must** be a valid pointer to an array of imageCount valid VkImage handles
- VUID-VkFrameBoundaryEXT-pBuffers-parameter
  If bufferCount is not 0, and pBuffers is not NULL, pBuffers **must** be a valid pointer to an array of bufferCount valid VkBuffer handles
- VUID-VkFrameBoundaryEXT-pTag-parameter
  If tagSize is not 0, and pTag is not NULL, pTag **must** be a valid pointer to an array of tagSize bytes
- VUID-VkFrameBoundaryEXT-commonparent
  Both of the elements of pBuffers, and the elements of pImages that are valid handles of non-ignored parameters **must** have been created, allocated, or retrieved from the same VkDevice

The bit which **can** be set in VkFrameBoundaryEXT::flags is:

```
// Provided by VK_EXT_frame_boundary
typedef enum VkFrameBoundaryFlagBitsEXT {
    VK_FRAME_BOUNDARY_FRAME_END_BIT_EXT = 0x00000001,
} VkFrameBoundaryFlagBitsEXT;
```

- VK_FRAME_BOUNDARY_FRAME_END_BIT_EXT specifies that this queue submission is the last one for this frame, i.e. once this queue submission has terminated, then the work for this frame is

completed.

Note that in the presence of timeline semaphores, the last queue submission might not be the last one to be submitted, as timeline semaphores allow for wait-before-signal submissions. In the context of frame boundary, the queue submission that should be done flagged as the last one is the one that is meant to be executed last, even if it **may** not be the last one to be submitted.

```
// Provided by VK_EXT_frame_boundary
typedef VkFlags VkFrameBoundaryFlagsEXT;
```

VkFrameBoundaryFlagsEXT is a bitmask type for setting a mask of zero or more VkFrameBoundaryFlagBitsEXT.

Appendix A: Vulkan Environment for SPIR-V

Shaders for Vulkan are defined by the Khronos SPIR-V Specification as well as the Khronos SPIR-V Extended Instructions for GLSL Specification. This appendix defines additional SPIR-V requirements applying to Vulkan shaders.

Versions and Formats

A Vulkan 1.0 implementation **must** support the 1.0 version of SPIR-V and the 1.0 version of the SPIR-V Extended Instructions for GLSL. If the VK_KHR_spirv_1_4 extension is enabled, the implementation **must** additionally support the 1.4 version of SPIR-V.

A SPIR-V module passed into vkCreateShaderModule is interpreted as a series of 32-bit words in host endianness, with literal strings packed as described in section 2.2 of the SPIR-V Specification. The first few words of the SPIR-V module **must** be a magic number and a SPIR-V version number, as described in section 2.3 of the SPIR-V Specification.

Capabilities

The table below lists the set of SPIR-V capabilities that **may** be supported in Vulkan implementations. The application **must** not use any of these capabilities in SPIR-V passed to vkCreateShaderModule unless one of the following conditions is met for the VkDevice specified in the `device` parameter of vkCreateShaderModule:

- The corresponding field in the table is blank.
- Any corresponding Vulkan feature is enabled.
- Any corresponding Vulkan extension is enabled.
- Any corresponding Vulkan property is supported.
- The corresponding core version is supported (as returned by VkPhysicalDeviceProperties::`apiVersion`).

Table 92. List of SPIR-V Capabilities and Corresponding Vulkan Features, Extensions, or Core Version

SPIR-V `OpCapability`	**Vulkan feature, extension, or core version**
`Matrix`	VK_VERSION_1_0
`Shader`	VK_VERSION_1_0
`InputAttachment`	VK_VERSION_1_0
`Sampled1D`	VK_VERSION_1_0

SPIR-V `OpCapability` **Vulkan feature, extension, or core version**
`Image1D` VK_VERSION_1_0
`SampledBuffer` VK_VERSION_1_0
`ImageBuffer` VK_VERSION_1_0
`ImageQuery` VK_VERSION_1_0
`DerivativeControl` VK_VERSION_1_0
`Geometry` `VkPhysicalDeviceFeatures::geometryShader`
`Tessellation` `VkPhysicalDeviceFeatures::tessellationShader`
`Float64` `VkPhysicalDeviceFeatures::shaderFloat64`
`Int64` `VkPhysicalDeviceFeatures::shaderInt64`

`Int64Atomics`
`VkPhysicalDeviceVulkan12Features::shaderBufferInt64Atomics`
`VkPhysicalDeviceVulkan12Features::shaderSharedInt64Atomics`
`VkPhysicalDeviceShaderImageAtomicInt64FeaturesEXT::shaderImageInt64Atomics`

`AtomicFloat16AddEXT`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderBufferFloat16AtomicAdd`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderSharedFloat16AtomicAdd`

`AtomicFloat32AddEXT`
`VkPhysicalDeviceShaderAtomicFloatFeaturesEXT::shaderBufferFloat32AtomicAdd`
`VkPhysicalDeviceShaderAtomicFloatFeaturesEXT::shaderSharedFloat32AtomicAdd`
`VkPhysicalDeviceShaderAtomicFloatFeaturesEXT::shaderImageFloat32AtomicAdd`

`AtomicFloat64AddEXT`
`VkPhysicalDeviceShaderAtomicFloatFeaturesEXT::shaderBufferFloat64AtomicAdd`
`VkPhysicalDeviceShaderAtomicFloatFeaturesEXT::shaderSharedFloat64AtomicAdd`

`AtomicFloat16MinMaxEXT`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderBufferFloat16AtomicMinMax`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderSharedFloat16AtomicMinMax`

`AtomicFloat32MinMaxEXT`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderBufferFloat32AtomicMinMax`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderSharedFloat32AtomicMinMax`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderImageFloat32AtomicMinMax`

SPIR-V `OpCapability` **Vulkan feature, extension, or core version**

`AtomicFloat64MinMaxEXT`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderBufferFloat64AtomicMinMax`
`VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::shaderSharedFloat64AtomicMinMax`

`Int64ImageEXT` `VkPhysicalDeviceShaderImageAtomicInt64FeaturesEXT::shaderImageInt64Atomics`
`Int16` `VkPhysicalDeviceFeatures::shaderInt16`
`TessellationPointSize` `VkPhysicalDeviceFeatures::shaderTessellationAndGeometryPointSize`
`GeometryPointSize` `VkPhysicalDeviceFeatures::shaderTessellationAndGeometryPointSize`
`ImageGatherExtended` `VkPhysicalDeviceFeatures::shaderImageGatherExtended`
`StorageImageMultisample` `VkPhysicalDeviceFeatures::shaderStorageImageMultisample`
`UniformBufferArrayDynamicIndexing` `VkPhysicalDeviceFeatures::shaderUniformBufferArrayDynamicIndexing`
`SampledImageArrayDynamicIndexing` `VkPhysicalDeviceFeatures::shaderSampledImageArrayDynamicIndexing`
`StorageBufferArrayDynamicIndexing` `VkPhysicalDeviceFeatures::shaderStorageBufferArrayDynamicIndexing`
`StorageImageArrayDynamicIndexing` `VkPhysicalDeviceFeatures::shaderStorageImageArrayDynamicIndexing`
`ClipDistance` `VkPhysicalDeviceFeatures::shaderClipDistance`
`CullDistance` `VkPhysicalDeviceFeatures::shaderCullDistance`
`ImageCubeArray` `VkPhysicalDeviceFeatures::imageCubeArray`
`SampleRateShading` `VkPhysicalDeviceFeatures::sampleRateShading`
`SparseResidency` `VkPhysicalDeviceFeatures::shaderResourceResidency`
`MinLod` `VkPhysicalDeviceFeatures::shaderResourceMinLod`
`SampledCubeArray` `VkPhysicalDeviceFeatures::imageCubeArray`

SPIR-V `OpCapability` **Vulkan feature, extension, or core version**
`ImageMSArray` `VkPhysicalDeviceFeatures::shaderStorageImageMultisample`
`StorageImageExtendedFormats` `VK_VERSION_1_0`
`InterpolationFunction` `VkPhysicalDeviceFeatures::sampleRateShading`

`StorageImageReadWithoutFormat`
`VkPhysicalDeviceFeatures::shaderStorageImageReadWithoutFormat`
`VK_KHR_format_feature_flags2`

`StorageImageWriteWithoutFormat`
`VkPhysicalDeviceFeatures::shaderStorageImageWriteWithoutFormat`
`VK_KHR_format_feature_flags2`

`MultiViewport` `VkPhysicalDeviceFeatures::multiViewport`
`DrawParameters` `VK_KHR_shader_draw_parameters`
`MultiView` `VkPhysicalDeviceMultiviewFeatures::multiview`
`DeviceGroup` `VK_KHR_device_group`
`VariablePointersStorageBuffer` `VkPhysicalDeviceVariablePointersFeatures::variablePointersStorageBuffer`
`VariablePointers` `VkPhysicalDeviceVariablePointersFeatures::variablePointers`
`ShaderClockKHR` `VK_KHR_shader_clock`
`StencilExportEXT` `VK_EXT_shader_stencil_export`
`StorageBuffer16BitAccess` `VkPhysicalDevice16BitStorageFeatures::storageBuffer16BitAccess`
`UniformAndStorageBuffer16BitAccess` `VkPhysicalDevice16BitStorageFeatures::uniformAndStorageBuffer16BitAccess`
`StoragePushConstant16` `VkPhysicalDevice16BitStorageFeatures::storagePushConstant16`
`StorageInputOutput16` `VkPhysicalDevice16BitStorageFeatures::storageInputOutput16`
`SampleMaskPostDepthCoverage` `VK_EXT_post_depth_coverage`

SPIR-V OpCapability **Vulkan feature, extension, or core version**
FragmentFullyCoveredEXT VK_EXT_conservative_rasterization
Float16 VkPhysicalDeviceVulkan12Features::shaderFloat16
Int8 VkPhysicalDeviceVulkan12Features::shaderInt8
StorageBuffer8BitAccess VkPhysicalDeviceVulkan12Features::storageBuffer8BitAccess
UniformAndStorageBuffer8BitAccess VkPhysicalDeviceVulkan12Features::uniformAndStorageBuffer8BitAccess
StoragePushConstant8 VkPhysicalDeviceVulkan12Features::storagePushConstant8
VulkanMemoryModel VkPhysicalDeviceVulkan12Features::vulkanMemoryModel
VulkanMemoryModelDeviceScope VkPhysicalDeviceVulkan12Features::vulkanMemoryModelDeviceScope
DenormPreserve VkPhysicalDeviceVulkan12Properties::shaderDenormPreserveFloat16 VkPhysicalDeviceVulkan12Properties::shaderDenormPreserveFloat32 VkPhysicalDeviceVulkan12Properties::shaderDenormPreserveFloat64
DenormFlushToZero VkPhysicalDeviceVulkan12Properties::shaderDenormFlushToZeroFloat16 VkPhysicalDeviceVulkan12Properties::shaderDenormFlushToZeroFloat32 VkPhysicalDeviceVulkan12Properties::shaderDenormFlushToZeroFloat64
SignedZeroInfNanPreserve VkPhysicalDeviceVulkan12Properties::shaderSignedZeroInfNanPreserveFloat16 VkPhysicalDeviceVulkan12Properties::shaderSignedZeroInfNanPreserveFloat32 VkPhysicalDeviceVulkan12Properties::shaderSignedZeroInfNanPreserveFloat64
RoundingModeRTE VkPhysicalDeviceVulkan12Properties::shaderRoundingModeRTEFloat16 VkPhysicalDeviceVulkan12Properties::shaderRoundingModeRTEFloat32 VkPhysicalDeviceVulkan12Properties::shaderRoundingModeRTEFloat64
RoundingModeRTZ VkPhysicalDeviceVulkan12Properties::shaderRoundingModeRTZFloat16 VkPhysicalDeviceVulkan12Properties::shaderRoundingModeRTZFloat32 VkPhysicalDeviceVulkan12Properties::shaderRoundingModeRTZFloat64
ComputeDerivativeGroupQuadsKHR VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR::computeDerivativeGroupQuads

SPIR-V OpCapability **Vulkan feature, extension, or core version**
ComputeDerivativeGroupLinearKHR VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR::computeDerivativeGroupLinear
RayTracingKHR VkPhysicalDeviceRayTracingPipelineFeaturesKHR::rayTracingPipeline
RayQueryKHR VkPhysicalDeviceRayQueryFeaturesKHR::rayQuery
RayTraversalPrimitiveCullingKHR VkPhysicalDeviceRayTracingPipelineFeaturesKHR::rayTraversalPrimitiveCulling VkPhysicalDeviceRayQueryFeaturesKHR::rayQuery
RayCullMaskKHR VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR::rayTracingMaintenance1
TransformFeedback VkPhysicalDeviceTransformFeedbackFeaturesEXT::transformFeedback
GeometryStreams VkPhysicalDeviceTransformFeedbackFeaturesEXT::geometryStreams
FragmentDensityEXT VkPhysicalDeviceFragmentDensityMapFeaturesEXT::fragmentDensityMap
PhysicalStorageBufferAddresses VkPhysicalDeviceVulkan12Features::bufferDeviceAddress
FragmentShaderSampleInterlockEXT VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT::fragmentShaderSampleInterlock
FragmentShaderPixelInterlockEXT VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT::fragmentShaderPixelInterlock
FragmentShaderShadingRateInterlockEXT VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT::fragmentShaderShadingRateInterlock
FragmentShadingRateKHR VkPhysicalDeviceFragmentShadingRateFeaturesKHR::pipelineFragmentShadingRate VkPhysicalDeviceFragmentShadingRateFeaturesKHR::primitiveFragmentShadingRate VkPhysicalDeviceFragmentShadingRateFeaturesKHR::attachmentFragmentShadingRate
WorkgroupMemoryExplicitLayoutKHR VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR ::workgroupMemoryExplicitLayout
WorkgroupMemoryExplicitLayout8BitAccessKHR VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR ::workgroupMemoryExplicitLayout8BitAccess
WorkgroupMemoryExplicitLayout16BitAccessKHR VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR ::workgroupMemoryExplicitLayout16BitAccess

SPIR-V OpCapability **Vulkan feature, extension, or core version**
DotProductInputAll VkPhysicalDeviceVulkan13Features::shaderIntegerDotProduct VkPhysicalDeviceShaderIntegerDotProductFeaturesKHR::shaderIntegerDotProduct
DotProductInput4x8Bit VkPhysicalDeviceVulkan13Features::shaderIntegerDotProduct VkPhysicalDeviceShaderIntegerDotProductFeaturesKHR::shaderIntegerDotProduct
DotProductInput4x8BitPacked VkPhysicalDeviceVulkan13Features::shaderIntegerDotProduct VkPhysicalDeviceShaderIntegerDotProductFeaturesKHR::shaderIntegerDotProduct
DotProduct VkPhysicalDeviceVulkan13Features::shaderIntegerDotProduct VkPhysicalDeviceShaderIntegerDotProductFeaturesKHR::shaderIntegerDotProduct
FragmentBarycentricKHR VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR::fragmentShaderBarycentric
RayTracingOpacityMicromapEXT VK_EXT_opacity_micromap
RayTracingPositionFetchKHR VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR::rayTracingPositionFetch
RayQueryPositionFetchKHR VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR::rayTracingPositionFetch
TileImageColorReadAccessEXT VkPhysicalDeviceShaderTileImageFeaturesEXT::shaderTileImageColorReadAccess
TileImageDepthReadAccessEXT VkPhysicalDeviceShaderTileImageFeaturesEXT::shaderTileImageDepthReadAccess
TileImageStencilReadAccessEXT VkPhysicalDeviceShaderTileImageFeaturesEXT::shaderTileImageStencilReadAccess
CooperativeMatrixKHR VkPhysicalDeviceCooperativeMatrixFeaturesKHR::cooperativeMatrix
GroupNonUniformRotateKHR VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR::shaderSubgroupRotate
ExpectAssumeKHR VkPhysicalDeviceShaderExpectAssumeFeaturesKHR::shaderExpectAssume
FloatControls2 VkPhysicalDeviceShaderFloatControls2FeaturesKHR::shaderFloatControls2
QuadControlKHR VkPhysicalDeviceShaderQuadControlFeaturesKHR::shaderQuadControl
ReplicatedCompositesEXT VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT::shaderReplicatedComposites

The application **must** not pass a SPIR-V module containing any of the following to vkCreateShaderModule:

- any `OpCapability` not listed above,
- an unsupported capability, or
- a capability which corresponds to a Vulkan feature or extension which has not been enabled.

SPIR-V Extensions

The following table lists SPIR-V extensions that implementations **may** support. The application **must** not pass a SPIR-V module to vkCreateShaderModule that uses the following SPIR-V extensions unless one of the following conditions is met for the VkDevice specified in the `device` parameter of vkCreateShaderModule:

- Any corresponding Vulkan extension is enabled.
- The corresponding core version is supported (as returned by VkPhysicalDeviceProperties ::`apiVersion`).

Table 93. List of SPIR-V Extensions and Corresponding Vulkan Extensions or Core Version

SPIR-V `OpExtension`	**Vulkan extension or core version**
`SPV_KHR_variable_pointers`	`VK_KHR_variable_pointers`
`SPV_KHR_shader_draw_parameters`	`VK_KHR_shader_draw_parameters`
`SPV_KHR_8bit_storage`	`VK_KHR_8bit_storage`
`SPV_KHR_16bit_storage`	`VK_KHR_16bit_storage`
`SPV_KHR_shader_clock`	`VK_KHR_shader_clock`
`SPV_KHR_float_controls`	`VK_KHR_shader_float_controls`
`SPV_KHR_storage_buffer_storage_class`	`VK_KHR_storage_buffer_storage_class`
`SPV_KHR_post_depth_coverage`	`VK_EXT_post_depth_coverage`
`SPV_EXT_shader_stencil_export`	`VK_EXT_shader_stencil_export`
`SPV_KHR_vulkan_memory_model`	`VK_KHR_vulkan_memory_model`

SPIR-V `OpExtension` **Vulkan extension or core version**
`SPV_KHR_ray_tracing` `VK_KHR_ray_tracing_pipeline`
`SPV_KHR_ray_query` `VK_KHR_ray_query`
`SPV_KHR_ray_cull_mask` `VK_KHR_ray_tracing_maintenance1`
`SPV_EXT_fragment_invocation_density` `VK_EXT_fragment_density_map`
`SPV_KHR_physical_storage_buffer` `VK_KHR_buffer_device_address`
`SPV_EXT_fragment_shader_interlock` `VK_EXT_fragment_shader_interlock`
`SPV_KHR_fragment_shading_rate` `VK_KHR_fragment_shading_rate`
`SPV_KHR_non_semantic_info` `VK_KHR_shader_non_semantic_info`
`SPV_EXT_shader_image_int64` `VK_EXT_shader_image_atomic_int64`
`SPV_KHR_terminate_invocation` `VK_KHR_shader_terminate_invocation`
`SPV_KHR_multiview` `VK_KHR_multiview`
`SPV_KHR_workgroup_memory_explicit_layout` `VK_KHR_workgroup_memory_explicit_layout`
`SPV_EXT_shader_atomic_float_add` `VK_EXT_shader_atomic_float`
`SPV_KHR_fragment_shader_barycentric` `VK_KHR_fragment_shader_barycentric`
`SPV_KHR_subgroup_uniform_control_flow` `VK_KHR_shader_subgroup_uniform_control_flow`
`SPV_EXT_shader_atomic_float_min_max` `VK_EXT_shader_atomic_float2`
`SPV_EXT_shader_atomic_float16_add` `VK_EXT_shader_atomic_float2`
`SPV_EXT_fragment_fully_covered` `VK_EXT_conservative_rasterization`
`SPV_KHR_integer_dot_product` `VK_KHR_shader_integer_dot_product`

SPIR-V `OpExtension` **Vulkan extension or core version**
`SPV_KHR_device_group` `VK_KHR_device_group`
`SPV_KHR_ray_tracing_position_fetch` `VK_KHR_ray_tracing_position_fetch`
`SPV_EXT_shader_tile_image` `VK_EXT_shader_tile_image`
`SPV_EXT_opacity_micromap` `VK_EXT_opacity_micromap`
`SPV_KHR_cooperative_matrix` `VK_KHR_cooperative_matrix`
`SPV_KHR_maximal_reconvergence` `VK_KHR_shader_maximal_reconvergence`
`SPV_KHR_subgroup_rotate` `VK_KHR_shader_subgroup_rotate`
`SPV_KHR_expect_assume` `VK_KHR_shader_expect_assume`
`SPV_KHR_float_controls2` `VK_KHR_shader_float_controls2`
`SPV_KHR_quad_control` `VK_KHR_shader_quad_control`
`SPV_KHR_compute_shader_derivatives` `VK_KHR_compute_shader_derivatives`
`SPV_EXT_replicated_composites` `VK_EXT_shader_replicated_composites`
`SPV_KHR_relaxed_extended_instruction` `VK_KHR_shader_relaxed_extended_instruction`

Validation Rules Within a Module

A SPIR-V module passed to vkCreateShaderModule **must** conform to the following rules:

Standalone SPIR-V Validation

The following rules **can** be validated with only the SPIR-V module itself. They do not depend on knowledge of the implementation and its capabilities or knowledge of runtime information, such as enabled features.

Valid Usage

- VUID-StandaloneSpirv-None-04633

Every entry point **must** have no return value and accept no arguments

- VUID-StandaloneSpirv-None-04634
  The static function-call graph for an entry point **must** not contain cycles; that is, static recursion is not allowed
- VUID-StandaloneSpirv-None-04635
  The `Logical` or `PhysicalStorageBuffer64` addressing model **must** be selected
- VUID-StandaloneSpirv-None-04636
  `Scope` for execution **must** be limited to `Workgroup` or `Subgroup`
- VUID-StandaloneSpirv-None-04637
  If the `Scope` for execution is `Workgroup`, then it **must** only be used in the task, mesh, tessellation control, or compute `Execution Model`
- VUID-StandaloneSpirv-None-04638
  `Scope` for memory **must** be limited to `Device`, `QueueFamily`, `Workgroup`, `ShaderCallKHR`, `Subgroup`, or `Invocation`
- VUID-StandaloneSpirv-ExecutionModel-07320
  If the `Execution Model` is `TessellationControl`, and the `MemoryModel` is `GLSL450`, the `Scope` for memory **must** not be `Workgroup`
- VUID-StandaloneSpirv-None-07321
  If the `Scope` for memory is `Workgroup`, then it **must** only be used in the task, mesh, tessellation control, or compute `Execution Model`
- VUID-StandaloneSpirv-None-04640
  If the `Scope` for memory is `ShaderCallKHR`, then it **must** only be used in ray generation, intersection, closest hit, any-hit, miss, and callable `Execution Model`
- VUID-StandaloneSpirv-None-04641
  If the `Scope` for memory is `Invocation`, then memory semantics **must** be `None`
- VUID-StandaloneSpirv-None-04642
  `Scope` for group operations **must** be limited to `Subgroup`
- VUID-StandaloneSpirv-SubgroupVoteKHR-07951
  If none of the `SubgroupVoteKHR`, `GroupNonUniform`, or `SubgroupBallotKHR` capabilities are declared, `Scope` for memory **must** not be `Subgroup`
- VUID-StandaloneSpirv-None-04643
  `Storage Class` **must** be limited to `UniformConstant`, `Input`, `Uniform`, `Output`, `Workgroup`, `Private`, `Function`, `PushConstant`, `Image`, `StorageBuffer`, `RayPayloadKHR`, `IncomingRayPayloadKHR`, `HitAttributeKHR`, `CallableDataKHR`, `IncomingCallableDataKHR`, `ShaderRecordBufferKHR`, `PhysicalStorageBuffer`, or `TileImageEXT`
- VUID-StandaloneSpirv-None-04644
  If the `Storage Class` is `Output`, then it **must** not be used in the `GlCompute`, `RayGenerationKHR`, `IntersectionKHR`, `AnyHitKHR`, `ClosestHitKHR`, `MissKHR`, or `CallableKHR Execution Model`
- VUID-StandaloneSpirv-None-04645
  If the `Storage Class` is `Workgroup`, then it **must** only be used in the task, mesh, or compute `Execution Model`
- VUID-StandaloneSpirv-None-08720

If the Storage Class is TileImageEXT, then it **must** only be used in the fragment execution model

- VUID-StandaloneSpirv-OpAtomicStore-04730
  OpAtomicStore **must** not use Acquire, AcquireRelease, or SequentiallyConsistent memory semantics
- VUID-StandaloneSpirv-OpAtomicLoad-04731
  OpAtomicLoad **must** not use Release, AcquireRelease, or SequentiallyConsistent memory semantics
- VUID-StandaloneSpirv-OpMemoryBarrier-04732
  OpMemoryBarrier **must** use one of Acquire, Release, AcquireRelease, or SequentiallyConsistent memory semantics
- VUID-StandaloneSpirv-OpMemoryBarrier-04733
  OpMemoryBarrier **must** include at least one Storage Class
- VUID-StandaloneSpirv-OpControlBarrier-04650
  If the semantics for OpControlBarrier includes one of Acquire, Release, AcquireRelease, or SequentiallyConsistent memory semantics, then it **must** include at least one Storage Class
- VUID-StandaloneSpirv-OpVariable-04651
  Any OpVariable with an Initializer operand **must** have Output, Private, Function, or Workgroup as its Storage Class operand
- VUID-StandaloneSpirv-OpVariable-04734
  Any OpVariable with an Initializer operand and Workgroup as its Storage Class operand **must** use OpConstantNull as the initializer
- VUID-StandaloneSpirv-OpReadClockKHR-04652
  Scope for OpReadClockKHR **must** be limited to Subgroup or Device
- VUID-StandaloneSpirv-OriginLowerLeft-04653
  The OriginLowerLeft Execution Mode **must** not be used; fragment entry points **must** declare OriginUpperLeft
- VUID-StandaloneSpirv-PixelCenterInteger-04654
  The PixelCenterInteger Execution Mode **must** not be used (pixels are always centered at half-integer coordinates)
- VUID-StandaloneSpirv-UniformConstant-04655
  Any variable in the UniformConstant Storage Class **must** be typed as either OpTypeImage, OpTypeSampler, OpTypeSampledImage, OpTypeAccelerationStructureKHR, or an array of one of these types
- VUID-StandaloneSpirv-Uniform-06807
  Any variable in the Uniform or StorageBuffer Storage Class **must** be typed as OpTypeStruct or an array of this type
- VUID-StandaloneSpirv-PushConstant-06808
  Any variable in the PushConstant Storage Class **must** be typed as OpTypeStruct
- VUID-StandaloneSpirv-OpTypeImage-04656
  OpTypeImage **must** declare a scalar 32-bit float, 64-bit integer, or 32-bit integer type for the "Sampled Type" (RelaxedPrecision **can** be applied to a sampling instruction and to the

variable holding the result of a sampling instruction)

- VUID-StandaloneSpirv-OpTypeImage-04657
  `OpTypeImage` **must** have a “Sampled” operand of 1 (sampled image) or 2 (storage image)
- VUID-StandaloneSpirv-OpTypeSampledImage-06671
  `OpTypeSampledImage` **must** have a `OpTypeImage` with a “Sampled” operand of 1 (sampled image)
- VUID-StandaloneSpirv-Image-04965
  The SPIR-V Type of the `Image Format` operand of an `OpTypeImage` **must** match the `Sampled Type`, as defined in Image Format and Type Matching
- VUID-StandaloneSpirv-OpImageTexelPointer-04658
  If an `OpImageTexelPointer` is used in an atomic operation, the image type of the `image` parameter to `OpImageTexelPointer` **must** have an image format of `R64i`, `R64ui`, `R32f`, `R32i`, or `R32ui`
- VUID-StandaloneSpirv-OpImageQuerySizeLod-04659
  `OpImageQuerySizeLod`, `OpImageQueryLod`, and `OpImageQueryLevels` **must** only consume an “Image” operand whose type has its “Sampled” operand set to 1
- VUID-StandaloneSpirv-OpTypeImage-09638
  An `OpTypeImage` **must** not have a “Dim” operand of `Rect`
- VUID-StandaloneSpirv-OpTypeImage-06214
  An `OpTypeImage` with a “Dim” operand of `SubpassData` **must** have an “Arrayed” operand of 0 (non-arrayed) and a “Sampled” operand of 2 (storage image)
- VUID-StandaloneSpirv-SubpassData-04660
  The (u,v) coordinates used for a `SubpassData` **must** be the <id> of a constant vector (0,0)
- VUID-StandaloneSpirv-OpTypeImage-06924
  Objects of types `OpTypeImage`, `OpTypeSampler`, `OpTypeSampledImage`, `OpTypeAccelerationStructureKHR`, and arrays of these types **must** not be stored to or modified
- VUID-StandaloneSpirv-Uniform-06925
  Any variable in the `Uniform Storage Class` decorated as `Block` **must** not be stored to or modified
- VUID-StandaloneSpirv-Offset-04663
  Image operand `Offset` **must** only be used with `OpImage*Gather` instructions
- VUID-StandaloneSpirv-Offset-04865
  Any image instruction which uses an `Offset`, `ConstOffset`, or `ConstOffsets` image operand, **must** only consume a “Sampled Image” operand whose type has its “Sampled” operand set to 1
- VUID-StandaloneSpirv-OpImageGather-04664
  The “Component” operand of `OpImageGather`, and `OpImageSparseGather` **must** be the <id> of a constant instruction
- VUID-StandaloneSpirv-OpImage-04777
  `OpImage*Dref*` instructions **must** not consume an image whose `Dim` is 3D
- VUID-StandaloneSpirv-None-04667

Structure types **must** not contain opaque types

- VUID-StandaloneSpirv-BuiltIn-04668
  Any `BuiltIn` decoration not listed in Built-In Variables **must** not be used
- VUID-StandaloneSpirv-OpEntryPoint-09658
  For a given `OpEntryPoint`, any `BuiltIn` decoration **must** not be used more than once by the `Input` interface
- VUID-StandaloneSpirv-OpEntryPoint-09659
  For a given `OpEntryPoint`, any `BuiltIn` decoration **must** not be used more than once by the `Output` interface
- VUID-StandaloneSpirv-Location-06672
  The `Location` or `Component` decorations **must** only be used with the `Input`, `Output`, `RayPayloadKHR`, `IncomingRayPayloadKHR`, `HitAttributeKHR`, `HitObjectAttributeNV`, `CallableDataKHR`, `IncomingCallableDataKHR`, or `ShaderRecordBufferKHR` storage classes
- VUID-StandaloneSpirv-Location-04915
  The `Location` or `Component` decorations **must** not be used with `BuiltIn`
- VUID-StandaloneSpirv-Location-04916
  The `Location` decorations **must** be used on user-defined variables
- VUID-StandaloneSpirv-Location-04917
  If a user-defined variable is not a pointer to a `Block` decorated `OpTypeStruct`, then the `OpVariable` **must** have a `Location` decoration
- VUID-StandaloneSpirv-Location-04918
  If a user-defined variable has a `Location` decoration, and the variable is a pointer to a `OpTypeStruct`, then the members of that structure **must** not have `Location` decorations
- VUID-StandaloneSpirv-Location-04919
  If a user-defined variable does not have a `Location` decoration, and the variable is a pointer to a `Block` decorated `OpTypeStruct`, then each member of the struct **must** have a `Location` decoration
- VUID-StandaloneSpirv-Component-04920
  The `Component` decoration value **must** not be greater than 3
- VUID-StandaloneSpirv-Component-04921
  If the `Component` decoration is used on an `OpVariable` that has a `OpTypeVector` type with a `Component Type` with a `Width` that is less than or equal to 32, the sum of its `Component Count` and the `Component` decoration value **must** be less than or equal to 4
- VUID-StandaloneSpirv-Component-04922
  If the `Component` decoration is used on an `OpVariable` that has a `OpTypeVector` type with a `Component Type` with a `Width` that is equal to 64, the sum of two times its `Component Count` and the `Component` decoration value **must** be less than or equal to 4
- VUID-StandaloneSpirv-Component-04923
  The `Component` decorations value **must** not be 1 or 3 for scalar or two-component 64-bit data types
- VUID-StandaloneSpirv-Component-04924
  The `Component` decorations **must** not be used with any type that is not a scalar or vector, or

an array of such a type

- VUID-StandaloneSpirv-Component-07703
  The `Component` decorations **must** not be used for a 64-bit vector type with more than two components
- VUID-StandaloneSpirv-Input-09557
  The pointers of any `Input` or `Output` Interface user-defined variables **must** not contain any `PhysicalStorageBuffer` `Storage` `Class` pointers
- VUID-StandaloneSpirv-GLSLShared-04669
  The `GLSLShared` and `GLSLPacked` decorations **must** not be used
- VUID-StandaloneSpirv-Flat-04670
  The `Flat`, `NoPerspective`, `Sample`, and `Centroid` decorations **must** only be used on variables with the `Output` or `Input` `Storage` `Class`
- VUID-StandaloneSpirv-Flat-06201
  The `Flat`, `NoPerspective`, `Sample`, and `Centroid` decorations **must** not be used on variables with the `Output` storage class in a fragment shader
- VUID-StandaloneSpirv-Flat-06202
  The `Flat`, `NoPerspective`, `Sample`, and `Centroid` decorations **must** not be used on variables with the `Input` storage class in a vertex shader
- VUID-StandaloneSpirv-PerVertexKHR-06777
  The `PerVertexKHR` decoration **must** only be used on variables with the `Input` `Storage` `Class` in a fragment shader
- VUID-StandaloneSpirv-Flat-04744
  Any variable with integer or double-precision floating-point type and with `Input` `Storage` `Class` in a fragment shader, **must** be decorated `Flat`
- VUID-StandaloneSpirv-ViewportRelativeNV-04672
  The `ViewportRelativeNV` decoration **must** only be used on a variable decorated with `Layer` in the vertex, tessellation evaluation, or geometry shader stages
- VUID-StandaloneSpirv-ViewportRelativeNV-04673
  The `ViewportRelativeNV` decoration **must** not be used unless a variable decorated with one of `ViewportIndex` or `ViewportMaskNV` is also statically used by the same `OpEntryPoint`
- VUID-StandaloneSpirv-ViewportMaskNV-04674
  The `ViewportMaskNV` and `ViewportIndex` decorations **must** not both be statically used by one or more `OpEntryPoint`'s that form the pre-rasterization shader stages of a graphics pipeline
- VUID-StandaloneSpirv-FPRoundingMode-04675
  Rounding modes other than round-to-nearest-even and round-towards-zero **must** not be used for the `FPRoundingMode` decoration
- VUID-StandaloneSpirv-Invariant-04677
  Variables decorated with `Invariant` and variables with structure types that have any members decorated with `Invariant` **must** be in the `Output` or `Input` `Storage` `Class`, `Invariant` used on an `Input` `Storage` `Class` variable or structure member has no effect
- VUID-StandaloneSpirv-VulkanMemoryModel-04678
  If the `VulkanMemoryModel` capability is not declared, the `Volatile` decoration **must** be used

on any variable declaration that includes one of the SMIDNV, WarpIDNV, SubgroupSize, SubgroupLocalInvocationId, SubgroupEqMask, SubgroupGeMask, SubgroupGtMask, SubgroupLeMask, or SubgroupLtMask BuiltIn decorations when used in the ray generation, closest hit, miss, intersection, or callable shaders, or with the RayTmaxKHR Builtin decoration when used in an intersection shader

- VUID-StandaloneSpirv-VulkanMemoryModel-04679
  If the VulkanMemoryModel capability is declared, the OpLoad instruction **must** use the Volatile memory semantics when it accesses into any variable that includes one of the SMIDNV, WarpIDNV, SubgroupSize, SubgroupLocalInvocationId, SubgroupEqMask, SubgroupGeMask, SubgroupGtMask, SubgroupLeMask, or SubgroupLtMask BuiltIn decorations when used in the ray generation, closest hit, miss, intersection, or callable shaders, or with the RayTmaxKHR Builtin decoration when used in an intersection shader
- VUID-StandaloneSpirv-OpTypeRuntimeArray-04680
  OpTypeRuntimeArray **must** only be used for:
  - the last member of a Block-decorated OpTypeStruct in StorageBuffer or PhysicalStorageBuffer storage Storage Class
  - BufferBlock-decorated OpTypeStruct in the Uniform storage Storage Class
  - the outermost dimension of an arrayed variable in the StorageBuffer, Uniform, or UniformConstant storage Storage Class
  - variables in the NodePayloadAMDX storage Storage Class when the CoalescingAMDX Execution Mode is specified
- VUID-StandaloneSpirv-Function-04681
  A type *T* that is an array sized with a specialization constant **must** neither be, nor be contained in, the type *T2* of a variable *V*, unless either: a) *T* is equal to *T2*, b) *V* is declared in the Function, or Private Storage Class, c) *V* is a non-Block variable in the Workgroup Storage Class, or d) *V* is an interface variable with an additional level of arrayness, as described in interface matching, and *T* is the member type of the array type *T2*
- VUID-StandaloneSpirv-OpControlBarrier-04682
  If OpControlBarrier is used in ray generation, intersection, any-hit, closest hit, miss, fragment, vertex, tessellation evaluation, or geometry shaders, the execution Scope **must** be Subgroup
- VUID-StandaloneSpirv-LocalSize-06426
  For each compute shader entry point, either a LocalSize or LocalSizeId Execution Mode, or an object decorated with the WorkgroupSize decoration **must** be specified
- VUID-StandaloneSpirv-DerivativeGroupQuadsNV-04684
  For compute shaders using the DerivativeGroupQuadsNV execution mode, the first two dimensions of the local workgroup size **must** be a multiple of two
- VUID-StandaloneSpirv-DerivativeGroupLinearNV-04778
  For compute shaders using the DerivativeGroupLinearNV execution mode, the product of the dimensions of the local workgroup size **must** be a multiple of four
- VUID-StandaloneSpirv-DerivativeGroupQuadsKHR-10151
  For compute, mesh, or task shaders using the DerivativeGroupQuadsKHR execution mode, the first two dimensions of the local workgroup size **must** be a multiple of two

- VUID-StandaloneSpirv-DerivativeGroupLinearKHR-10152
  For compute, mesh, or task shaders using the `DerivativeGroupLinearKHR` execution mode, the product of the dimensions of the local workgroup size **must** be a multiple of four
- VUID-StandaloneSpirv-OpGroupNonUniformBallotBitCount-04685
  If `OpGroupNonUniformBallotBitCount` is used, the group operation **must** be limited to `Reduce`, `InclusiveScan`, or `ExclusiveScan`
- VUID-StandaloneSpirv-None-04686
  The *Pointer* operand of all atomic instructions **must** have a `Storage Class` limited to `Uniform`, `Workgroup`, `Image`, `StorageBuffer`, `PhysicalStorageBuffer`, or `TaskPayloadWorkgroupEXT`
- VUID-StandaloneSpirv-Offset-04687
  Output variables or block members decorated with `Offset` that have a 64-bit type, or a composite type containing a 64-bit type, **must** specify an `Offset` value aligned to a 8 byte boundary
- VUID-StandaloneSpirv-Offset-04689
  The size of any output block containing any member decorated with `Offset` that is a 64-bit type **must** be a multiple of 8
- VUID-StandaloneSpirv-Offset-04690
  The first member of an output block specifying a `Offset` decoration **must** specify a `Offset` value that is aligned to an 8 byte boundary if that block contains any member decorated with `Offset` and is a 64-bit type
- VUID-StandaloneSpirv-Offset-04691
  Output variables or block members decorated with `Offset` that have a 32-bit type, or a composite type contains a 32-bit type, **must** specify an `Offset` value aligned to a 4 byte boundary
- VUID-StandaloneSpirv-Offset-04692
  Output variables, blocks or block members decorated with `Offset` **must** only contain base types that have components that are either 32-bit or 64-bit in size
- VUID-StandaloneSpirv-Offset-04716
  Only variables or block members in the output interface decorated with `Offset` **can** be captured for transform feedback, and those variables or block members **must** also be decorated with `XfbBuffer` and `XfbStride`, or inherit `XfbBuffer` and `XfbStride` decorations from a block containing them
- VUID-StandaloneSpirv-XfbBuffer-04693
  All variables or block members in the output interface of the entry point being compiled decorated with a specific `XfbBuffer` value **must** all be decorated with identical `XfbStride` values
- VUID-StandaloneSpirv-Stream-04694
  If any variables or block members in the output interface of the entry point being compiled are decorated with `Stream`, then all variables belonging to the same `XfbBuffer` **must** specify the same `Stream` value
- VUID-StandaloneSpirv-XfbBuffer-04696
  For any two variables or block members in the output interface of the entry point being compiled with the same `XfbBuffer` value, the ranges determined by the `Offset` decoration

and the size of the type **must** not overlap

- VUID-StandaloneSpirv-XfbBuffer-04697
  All block members in the output interface of the entry point being compiled that are in the same block and have a declared or inherited `XfbBuffer` decoration **must** specify the same `XfbBuffer` value
- VUID-StandaloneSpirv-RayPayloadKHR-04698
  `RayPayloadKHR Storage Class` **must** only be used in ray generation, closest hit or miss shaders
- VUID-StandaloneSpirv-IncomingRayPayloadKHR-04699
  `IncomingRayPayloadKHR Storage Class` **must** only be used in closest hit, any-hit, or miss shaders
- VUID-StandaloneSpirv-IncomingRayPayloadKHR-04700
  There **must** be at most one variable with the `IncomingRayPayloadKHR Storage Class` in the input interface of an entry point
- VUID-StandaloneSpirv-HitAttributeKHR-04701
  `HitAttributeKHR Storage Class` **must** only be used in intersection, any-hit, or closest hit shaders
- VUID-StandaloneSpirv-HitAttributeKHR-04702
  There **must** be at most one variable with the `HitAttributeKHR Storage Class` in the input interface of an entry point
- VUID-StandaloneSpirv-HitAttributeKHR-04703
  A variable with `HitAttributeKHR Storage Class` **must** only be written to in an intersection shader
- VUID-StandaloneSpirv-CallableDataKHR-04704
  `CallableDataKHR Storage Class` **must** only be used in ray generation, closest hit, miss, and callable shaders
- VUID-StandaloneSpirv-IncomingCallableDataKHR-04705
  `IncomingCallableDataKHR Storage Class` **must** only be used in callable shaders
- VUID-StandaloneSpirv-IncomingCallableDataKHR-04706
  There **must** be at most one variable with the `IncomingCallableDataKHR Storage Class` in the input interface of an entry point
- VUID-StandaloneSpirv-ShaderRecordBufferKHR-07119
  `ShaderRecordBufferKHR Storage Class` **must** only be used in ray generation, intersection, any-hit, closest hit, callable, or miss shaders
- VUID-StandaloneSpirv-Base-07650
  The `Base` operand of `OpPtrAccessChain` **must** have a storage class of `Workgroup`, `StorageBuffer`, or `PhysicalStorageBuffer`
- VUID-StandaloneSpirv-Base-07651
  If the `Base` operand of `OpPtrAccessChain` has a `Workgroup Storage Class`, then the `VariablePointers` capability **must** be declared
- VUID-StandaloneSpirv-Base-07652
  If the `Base` operand of `OpPtrAccessChain` has a `StorageBuffer Storage Class`, then the

VariablePointers or VariablePointersStorageBuffer capability **must** be declared

- VUID-StandaloneSpirv-PhysicalStorageBuffer64-04708
  If the PhysicalStorageBuffer64 addressing model is enabled, all instructions that support memory access operands and that use a physical pointer **must** include the Aligned operand
- VUID-StandaloneSpirv-PhysicalStorageBuffer64-04709
  If the PhysicalStorageBuffer64 addressing model is enabled, any access chain instruction that accesses into a RowMajor matrix **must** only be used as the Pointer operand to OpLoad or OpStore
- VUID-StandaloneSpirv-PhysicalStorageBuffer64-04710
  If the PhysicalStorageBuffer64 addressing model is enabled, OpConvertUToPtr and OpConvertPtrToU **must** use an integer type whose Width is 64
- VUID-StandaloneSpirv-OpTypeForwardPointer-04711
  OpTypeForwardPointer **must** have a Storage Class of PhysicalStorageBuffer
- VUID-StandaloneSpirv-None-04745
  All block members in a variable with a Storage Class of PushConstant declared as an array **must** only be accessed by dynamically uniform indices
- VUID-StandaloneSpirv-OpVariable-06673
  There **must** not be more than one OpVariable in the PushConstant Storage Class listed in the Interface for each OpEntryPoint
- VUID-StandaloneSpirv-OpEntryPoint-06674
  Each OpEntryPoint **must** not statically use more than one OpVariable in the PushConstant Storage Class
- VUID-StandaloneSpirv-OpEntryPoint-08721
  Each OpEntryPoint **must** not have more than one Input variable assigned the same Component word inside a Location slot, either explicitly or implicitly
- VUID-StandaloneSpirv-OpEntryPoint-08722
  Each OpEntryPoint **must** not have more than one Output variable assigned the same Component word inside a Location slot, either explicitly or implicitly
- VUID-StandaloneSpirv-Result-04780
  The Result Type operand of any OpImageRead or OpImageSparseRead instruction **must** be a vector of four components
- VUID-StandaloneSpirv-Base-04781
  The Base operand of any OpBitCount, OpBitReverse, OpBitFieldInsert, OpBitFieldSExtract, or OpBitFieldUExtract instruction **must** be a 32-bit integer scalar or a vector of 32-bit integers
- VUID-StandaloneSpirv-PushConstant-06675
  Any variable in the PushConstant or StorageBuffer storage class **must** be decorated as Block
- VUID-StandaloneSpirv-Uniform-06676
  Any variable in the Uniform Storage Class **must** be decorated as Block or BufferBlock
- VUID-StandaloneSpirv-UniformConstant-06677
  Any variable in the UniformConstant, StorageBuffer, or Uniform Storage Class **must** be

decorated with DescriptorSet and Binding

- VUID-StandaloneSpirv-InputAttachmentIndex-06678
  Variables decorated with InputAttachmentIndex **must** be in the UniformConstant Storage Class
- VUID-StandaloneSpirv-DescriptorSet-06491
  If a variable is decorated by DescriptorSet or Binding, the Storage Class **must** correspond to an entry in Shader Resource and Storage Class Correspondence
- VUID-StandaloneSpirv-Input-06778
  Variables with a Storage Class of Input in a fragment shader stage that are decorated with PerVertexKHR **must** be declared as arrays
- VUID-StandaloneSpirv-MeshEXT-07102
  The module **must** not contain both an entry point that uses the TaskEXT or MeshEXT Execution Model and an entry point that uses the TaskNV or MeshNV Execution Model
- VUID-StandaloneSpirv-MeshEXT-07106
  In mesh shaders using the MeshEXT Execution Model OpSetMeshOutputsEXT **must** be called before any outputs are written
- VUID-StandaloneSpirv-MeshEXT-07107
  In mesh shaders using the MeshEXT Execution Model all variables declared as output **must** not be read from
- VUID-StandaloneSpirv-MeshEXT-07108
  In mesh shaders using the MeshEXT Execution Model for OpSetMeshOutputsEXT instructions, the “Vertex Count” and “Primitive Count” operands **must** not depend on ViewIndex
- VUID-StandaloneSpirv-MeshEXT-07109
  In mesh shaders using the MeshEXT Execution Model variables decorated with PrimitivePointIndicesEXT, PrimitiveLineIndicesEXT, or PrimitiveTriangleIndicesEXT declared as an array **must** not be accessed by indices that depend on ViewIndex
- VUID-StandaloneSpirv-MeshEXT-07110
  In mesh shaders using the MeshEXT Execution Model any values stored in variables decorated with PrimitivePointIndicesEXT, PrimitiveLineIndicesEXT, or PrimitiveTriangleIndicesEXT **must** not depend on ViewIndex
- VUID-StandaloneSpirv-MeshEXT-07111
  In mesh shaders using the MeshEXT Execution Model variables in workgroup or private Storage Class declared as or containing a composite type **must** not be accessed by indices that depend on ViewIndex
- VUID-StandaloneSpirv-MeshEXT-07330
  In mesh shaders using the MeshEXT Execution Model the OutputVertices Execution Mode **must** be greater than 0
- VUID-StandaloneSpirv-MeshEXT-07331
  In mesh shaders using the MeshEXT Execution Model the OutputPrimitivesEXT Execution Mode **must** be greater than 0
- VUID-StandaloneSpirv-Input-07290
  Variables with a Storage Class of Input or Output and a type of OpTypeBool **must** be decorated with the BuiltIn decoration

- VUID-StandaloneSpirv-TileImageEXT-08723
  The tile image variable declarations **must** obey the constraints on the `TileImageEXT` `Storage Class` and the `Location` decoration described in Fragment Tile Image Interface
- VUID-StandaloneSpirv-None-08724
  The `TileImageEXT` `Storage Class` **must** only be used for declaring tile image variables
- VUID-StandaloneSpirv-Pointer-08973
  The `Storage Class` of the `Pointer` operand to `OpCooperativeMatrixLoadKHR` or `OpCooperativeMatrixStoreKHR` **must** be limited to `Workgroup`, `StorageBuffer`, or `PhysicalStorageBuffer`
- VUID-StandaloneSpirv-UniformBufferArrayDynamicIndexing-10127
  If the `UniformBufferArrayDynamicIndexing` capability is not declared, and an instruction accesses memory through a uniform buffer, the uniform buffer through which that memory is accessed **must** be determined by constant integral expressions
- VUID-StandaloneSpirv-SampledImageArrayDynamicIndexing-10128
  If the `SampledImageArrayDynamicIndexing` capability is not declared, and an instruction accesses memory through a sampled image or sampler, the sampled image or sampler through which that memory is accessed **must** be determined by constant integral expressions
- VUID-StandaloneSpirv-StorageBufferArrayDynamicIndexing-10129
  If the `StorageBufferArrayDynamicIndexing` capability is not declared, and an instruction accesses memory through a storage buffer, the storage buffer through which that memory is accessed **must** be determined by constant integral expressions
- VUID-StandaloneSpirv-StorageImageArrayDynamicIndexing-10130
  If the `StorageImageArrayDynamicIndexing` capability is not declared, and an instruction accesses memory through a storage image, the storage image through which that memory is accessed **must** be determined by constant integral expressions
- VUID-StandaloneSpirv-InputAttachmentArrayDynamicIndexing-10131
  If the `InputAttachmentArrayDynamicIndexing` capability is not declared, and an instruction accesses memory through an input attachment, the input attachmnet through which that memory is accessed **must** be determined by constant integral expressions
- VUID-StandaloneSpirv-UniformTexelBufferArrayDynamicIndexing-10132
  If the `UniformTexelBufferArrayDynamicIndexing` capability is not declared, and an instruction accesses memory through a uniform texel buffer, the uniform texel buffer through which that memory is accessed **must** be determined by constant integral expressions
- VUID-StandaloneSpirv-StorageTexelBufferArrayDynamicIndexing-10133
  If the `StorageTexelBufferArrayDynamicIndexing` capability is not declared, and an instruction accesses memory through a storage texel buffer, the storage texel buffer through which that memory is accessed **must** be determined by constant integral expressions

Runtime SPIR-V Validation

The following rules **must** be validated at runtime. These rules depend on knowledge of the implementation and its capabilities and knowledge of runtime information, such as enabled features.

Valid Usage

- VUID-RuntimeSpirv-vulkanMemoryModel-06265
  If the `vulkanMemoryModel` feature is enabled and the `vulkanMemoryModelDeviceScope` feature is not enabled, `Device` memory scope **must** not be used
- VUID-RuntimeSpirv-vulkanMemoryModel-06266
  If the `vulkanMemoryModel` feature is not enabled, `QueueFamily` memory scope **must** not be used
- VUID-RuntimeSpirv-shaderSubgroupClock-06267
  If the `shaderSubgroupClock` feature is not enabled, the `Subgroup` scope **must** not be used for `OpReadClockKHR`
- VUID-RuntimeSpirv-shaderDeviceClock-06268
  If the `shaderDeviceClock` feature is not enabled, the `Device` scope **must** not be used for `OpReadClockKHR`
- VUID-RuntimeSpirv-None-09558
  If the `dynamicRenderingLocalRead` feature is not enabled, any variable created with a "Type" of `OpTypeImage` that has a "Dim" operand of `SubpassData` **must** be decorated with `InputAttachmentIndex`
- VUID-RuntimeSpirv-OpTypeImage-09644
  Any variable declared as an `OpTypeArray` where the `Element Type` is an `OpTypeImage` with a "Dim" operand of `SubpassData` **must** be decorated with `InputAttachmentIndex`
- VUID-RuntimeSpirv-shaderStorageImageWriteWithoutFormat-07956
  If the VK_KHR_format_feature_flags2 extension is not enabled, and the `shaderStorageImageWriteWithoutFormat` feature is not enabled, any variable created with a "Type" of `OpTypeImage` that has a "Sampled" operand of 2 and an "Image Format" operand of `Unknown` **must** be decorated with `NonWritable`
- VUID-RuntimeSpirv-shaderStorageImageReadWithoutFormat-07957
  If the VK_KHR_format_feature_flags2 extension is not enabled, and the `shaderStorageImageReadWithoutFormat` feature is not enabled, any variable created with a "Type" of `OpTypeImage` that has a "Sampled" operand of 2 and an "Image Format" operand of `Unknown` **must** be decorated with `NonReadable`
- VUID-RuntimeSpirv-OpImageWrite-07112
  `OpImageWrite` to any `Image` whose `Image Format` is not `Unknown` **must** have the `Texel` operand contain at least as many components as the corresponding VkFormat as given in the SPIR-V Image Format compatibility table
- VUID-RuntimeSpirv-Location-06272
  The sum of `Location` and the number of locations the variable it decorates consumes **must** be less than or equal to the value for the matching `Execution Model` defined in Shader

Input and Output Locations

- VUID-RuntimeSpirv-Location-06428
  The maximum number of storage buffers, storage images, and output `Location` decorated color attachments written to in the `Fragment` `Execution` `Model` **must** be less than or equal to `maxFragmentCombinedOutputResources`
- VUID-RuntimeSpirv-None-06278
  `shaderBufferInt64Atomics` **must** be enabled for 64-bit integer atomic operations to be supported on a *Pointer* with a `Storage` `Class` of `StorageBuffer` or `Uniform`
- VUID-RuntimeSpirv-None-06279
  `shaderSharedInt64Atomics` **must** be enabled for 64-bit integer atomic operations to be supported on a *Pointer* with a `Storage` `Class` of `Workgroup`
- VUID-RuntimeSpirv-None-06284
  `shaderBufferFloat32Atomics`, or `shaderBufferFloat32AtomicAdd`, or `shaderBufferFloat64Atomics`, or `shaderBufferFloat64AtomicAdd`, or `shaderBufferFloat16Atomics`, or `shaderBufferFloat16AtomicAdd`, or `shaderBufferFloat16AtomicMinMax`, or `shaderBufferFloat32AtomicMinMax`, or `shaderBufferFloat64AtomicMinMax`, **must** be enabled for floating-point atomic operations to be supported on a *Pointer* with a `Storage` `Class` of `StorageBuffer`
- VUID-RuntimeSpirv-None-06285
  `shaderSharedFloat32Atomics`, or `shaderSharedFloat32AtomicAdd`, or `shaderSharedFloat64Atomics`, or `shaderSharedFloat64AtomicAdd`, or `shaderSharedFloat16Atomics`, or `shaderSharedFloat16AtomicAdd`, or `shaderSharedFloat16AtomicMinMax`, or `shaderSharedFloat32AtomicMinMax`, or `shaderSharedFloat64AtomicMinMax`, **must** be enabled for floating-point atomic operations to be supported on a *Pointer* with a `Storage` `Class` of `Workgroup`
- VUID-RuntimeSpirv-None-06286
  `shaderImageFloat32Atomics`, or `shaderImageFloat32AtomicAdd`, or `shaderImageFloat32AtomicMinMax`, **must** be enabled for 32-bit floating-point atomic operations to be supported on a *Pointer* with a `Storage` `Class` of `Image`
- VUID-RuntimeSpirv-None-06287
  `sparseImageFloat32Atomics`, or `sparseImageFloat32AtomicAdd`, or `sparseImageFloat32AtomicMinMax`, **must** be enabled for 32-bit floating-point atomics to be supported on sparse images
- VUID-RuntimeSpirv-None-06288
  `shaderImageInt64Atomics` **must** be enabled for 64-bit integer atomic operations to be supported on a *Pointer* with a `Storage` `Class` of `Image`
- VUID-RuntimeSpirv-denormBehaviorIndependence-06289
  If `denormBehaviorIndependence` is `VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_32_BIT_ONLY`, then the entry point **must** use the same denormals `Execution` `Mode` for both 16-bit and 64-bit floating-point types
- VUID-RuntimeSpirv-denormBehaviorIndependence-06290
  If `denormBehaviorIndependence` is `VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_NONE`, then the entry point **must** use the same denormals `Execution` `Mode` for all floating-point types

- VUID-RuntimeSpirv-roundingModeIndependence-06291
  If `roundingModeIndependence` is `VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_32_BIT_ONLY`, then the entry point **must** use the same rounding `Execution Mode` for both 16-bit and 64-bit floating-point types
- VUID-RuntimeSpirv-roundingModeIndependence-06292
  If `roundingModeIndependence` is `VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_NONE`, then the entry point **must** use the same rounding `Execution Mode` for all floating-point types
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat16-06293
  If `shaderSignedZeroInfNanPreserveFloat16` is `VK_FALSE`, then `SignedZeroInfNanPreserve` for 16-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat32-06294
  If `shaderSignedZeroInfNanPreserveFloat32` is `VK_FALSE`, then `SignedZeroInfNanPreserve` for 32-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat64-06295
  If `shaderSignedZeroInfNanPreserveFloat64` is `VK_FALSE`, then `SignedZeroInfNanPreserve` for 64-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderDenormPreserveFloat16-06296
  If `shaderDenormPreserveFloat16` is `VK_FALSE`, then `DenormPreserve` for 16-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderDenormPreserveFloat32-06297
  If `shaderDenormPreserveFloat32` is `VK_FALSE`, then `DenormPreserve` for 32-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderDenormPreserveFloat64-06298
  If `shaderDenormPreserveFloat64` is `VK_FALSE`, then `DenormPreserve` for 64-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderDenormFlushToZeroFloat16-06299
  If `shaderDenormFlushToZeroFloat16` is `VK_FALSE`, then `DenormFlushToZero` for 16-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderDenormFlushToZeroFloat32-06300
  If `shaderDenormFlushToZeroFloat32` is `VK_FALSE`, then `DenormFlushToZero` for 32-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderDenormFlushToZeroFloat64-06301
  If `shaderDenormFlushToZeroFloat64` is `VK_FALSE`, then `DenormFlushToZero` for 64-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderRoundingModeRTEFloat16-06302
  If `shaderRoundingModeRTEFloat16` is `VK_FALSE`, then `RoundingModeRTE` for 16-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderRoundingModeRTEFloat32-06303
  If `shaderRoundingModeRTEFloat32` is `VK_FALSE`, then `RoundingModeRTE` for 32-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderRoundingModeRTEFloat64-06304
  If `shaderRoundingModeRTEFloat64` is `VK_FALSE`, then `RoundingModeRTE` for 64-bit floating-point type **must** not be used

- VUID-RuntimeSpirv-shaderRoundingModeRTZFloat16-06305
  If `shaderRoundingModeRTZFloat16` is `VK_FALSE`, then `RoundingModeRTZ` for 16-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderRoundingModeRTZFloat32-06306
  If `shaderRoundingModeRTZFloat32` is `VK_FALSE`, then `RoundingModeRTZ` for 32-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderRoundingModeRTZFloat64-06307
  If `shaderRoundingModeRTZFloat64` is `VK_FALSE`, then `RoundingModeRTZ` for 64-bit floating-point type **must** not be used
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat16-09559
  If `shaderSignedZeroInfNanPreserveFloat16` is `VK_FALSE` then any `FPFastMathDefault` execution mode with a type of 16-bit float **must** include the `NSZ`, `NotInf`, and `NotNaN` flags
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat16-09560
  If `shaderSignedZeroInfNanPreserveFloat16` is `VK_FALSE` then any `FPFastMathMode` decoration on an instruction with result type or any operand type that includes a 16-bit float **must** include the `NSZ`, `NotInf`, and `NotNaN` flags
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat32-09561
  If `shaderSignedZeroInfNanPreserveFloat32` is `VK_FALSE` then any `FPFastMathDefault` execution mode with a type of 32-bit float **must** include the `NSZ`, `NotInf`, and `NotNaN` flags
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat32-09562
  If `shaderSignedZeroInfNanPreserveFloat32` is `VK_FALSE` then any `FPFastMathMode` decoration on an instruction with result type or any operand type that includes a 32-bit float **must** include the `NSZ`, `NotInf`, and `NotNaN` flags
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat64-09563
  If `shaderSignedZeroInfNanPreserveFloat64` is `VK_FALSE` then any `FPFastMathDefault` execution mode with a type of 64-bit float **must** include the `NSZ`, `NotInf`, and `NotNaN` flags
- VUID-RuntimeSpirv-shaderSignedZeroInfNanPreserveFloat64-09564
  If `shaderSignedZeroInfNanPreserveFloat64` is `VK_FALSE` then any `FPFastMathMode` decoration on an instruction with result type or any operand type that includes a 64-bit float **must** include the `NSZ`, `NotInf`, and `NotNaN` flags
- VUID-RuntimeSpirv-Offset-06308
  The `Offset` plus size of the type of each variable, in the output interface of the entry point being compiled, decorated with `XfbBuffer` **must** not be greater than VkPhysicalDeviceTransformFeedbackPropertiesEXT::`maxTransformFeedbackBufferDataSize`
- VUID-RuntimeSpirv-XfbBuffer-06309
  For any given `XfbBuffer` value, define the buffer data size to be smallest number of bytes such that, for all outputs decorated with the same `XfbBuffer` value, the size of the output interface variable plus the `Offset` is less than or equal to the buffer data size. For a given `Stream`, the sum of all the buffer data sizes for all buffers writing to that stream the **must** not exceed VkPhysicalDeviceTransformFeedbackPropertiesEXT::`maxTransformFeedbackStreamDataSize`
- VUID-RuntimeSpirv-OpEmitStreamVertex-06310
  The Stream value to `OpEmitStreamVertex` and `OpEndStreamPrimitive` **must** be less than

VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackStreams

- VUID-RuntimeSpirv-transformFeedbackStreamsLinesTriangles-06311
  If the geometry shader emits to more than one vertex stream and VkPhysicalDeviceTransformFeedbackPropertiesEXT::transformFeedbackStreamsLinesTriangles is VK_FALSE, then Execution Mode **must** be OutputPoints
- VUID-RuntimeSpirv-Stream-06312
  The stream number value to Stream **must** be less than VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackStreams
- VUID-RuntimeSpirv-XfbStride-06313
  The XFB Stride value to XfbStride **must** be less than or equal to VkPhysicalDeviceTransformFeedbackPropertiesEXT::maxTransformFeedbackBufferDataStride
- VUID-RuntimeSpirv-PhysicalStorageBuffer64-06314
  If the PhysicalStorageBuffer64 addressing model is enabled any load or store through a physical pointer type **must** be aligned to a multiple of the size of the largest scalar type in the pointed-to type
- VUID-RuntimeSpirv-PhysicalStorageBuffer64-06315
  If the PhysicalStorageBuffer64 addressing model is enabled the pointer value of a memory access instruction **must** be at least as aligned as specified by the Aligned memory access operand
- VUID-RuntimeSpirv-OpTypeCooperativeMatrixKHR-10163
  For OpTypeCooperativeMatrixKHR, the component type, scope, number of rows, and number of columns **must** match one of the matrices in any of the supported VkCooperativeMatrixPropertiesKHR, where
  - for Use of MatrixA, the number of rows must match VkCooperativeMatrixPropertiesKHR::MSize and the number of columns must match VkCooperativeMatrixPropertiesKHR::KSize and the type **must** match VkCooperativeMatrixPropertiesKHR::AType
  - for Use of MatrixB, the number of rows must match VkCooperativeMatrixPropertiesKHR::KSize and the number of columns must match VkCooperativeMatrixPropertiesKHR::NSize and the type **must** match VkCooperativeMatrixPropertiesKHR::BType
  - for Use of MatrixAccumulator, the number of rows must match VkCooperativeMatrixPropertiesKHR::MSize and the number of columns must match VkCooperativeMatrixPropertiesKHR::NSize and the type **must** match VkCooperativeMatrixPropertiesKHR::CType or VkCooperativeMatrixPropertiesKHR::ResultType
- VUID-RuntimeSpirv-OpCooperativeMatrixMulAddKHR-10060
  For OpCooperativeMatrixMulAddKHR, the operands **must** match a supported VkCooperativeMatrixPropertiesKHR, such that:
  - The type of A **must** have Rows match VkCooperativeMatrixPropertiesKHR::MSize, Columns match VkCooperativeMatrixPropertiesKHR::KSize, Use be MatrixAKHR, and ComponentType match VkCooperativeMatrixPropertiesKHR::AType.

- The type of B **must** have Rows match VkCooperativeMatrixPropertiesKHR::KSize, Columns match VkCooperativeMatrixPropertiesKHR::NSize, Use be MatrixBKHR, and ComponentType match VkCooperativeMatrixPropertiesKHR::BType.
- The type of C **must** have Rows match VkCooperativeMatrixPropertiesKHR::MSize, Columns match VkCooperativeMatrixPropertiesKHR::NSize, Use be MatrixAccumulatorKHR, and ComponentType match VkCooperativeMatrixPropertiesKHR::CType.
- The type of Result **must** have Rows match VkCooperativeMatrixPropertiesKHR::MSize, Columns match VkCooperativeMatrixPropertiesKHR::NSize, Use be MatrixAccumulatorKHR, and ComponentType match VkCooperativeMatrixPropertiesKHR::ResultType.
- If and only if VkCooperativeMatrixPropertiesKHR::AType is a signed integer type, MatrixASignedComponents **must** be used.
- If and only if VkCooperativeMatrixPropertiesKHR::BType is a signed integer type, MatrixBSignedComponents **must** be used.
- If and only if VkCooperativeMatrixPropertiesKHR::CType is a signed integer type, MatrixCSignedComponents **must** be used.
- If and only if VkCooperativeMatrixPropertiesKHR::ResultType is a signed integer type, MatrixResultSignedComponents **must** be used.
- If and only if VkCooperativeMatrixPropertiesKHR::saturatingAccumulation is VK_TRUE, SaturatingAccumulationKHR **must** be used.
- If and only if VkCooperativeMatrixPropertiesKHR::saturatingAccumulation is VK_FALSE, SaturatingAccumulationKHR **must** not be used.
- The scope of all cooperative matrix operands **must** match VkCooperativeMatrixPropertiesKHR::scope.

- VUID-RuntimeSpirv-cooperativeMatrixSupportedStages-08985
  OpTypeCooperativeMatrixKHR and OpCooperativeMatrix* instructions **must** not be used in shader stages not included in VkPhysicalDeviceCooperativeMatrixPropertiesKHR::cooperativeMatrixSupportedStages
- VUID-RuntimeSpirv-DescriptorSet-06323
  DescriptorSet and Binding decorations **must** obey the constraints on Storage Class, type, and descriptor type described in DescriptorSet and Binding Assignment
- VUID-RuntimeSpirv-OpCooperativeMatrixLoadKHR-08986
  For OpCooperativeMatrixLoadKHR and OpCooperativeMatrixStoreKHR instructions, the Pointer and Stride operands **must** be aligned to at least the lesser of 16 bytes or the natural alignment of a row or column (depending on ColumnMajor) of the matrix (where the natural alignment is the number of columns/rows multiplied by the component size)
- VUID-RuntimeSpirv-shaderSampleRateInterpolationFunctions-06325
  If the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::shaderSampleRateInterpolationFunctions is VK_FALSE, then GLSL.std.450 fragment interpolation functions are not supported by the implementation and OpCapability **must** not be InterpolationFunction

- VUID-RuntimeSpirv-tessellationShader-06326
  If the tessellationShader feature is enabled, and the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::tessellationIsolines is VK_FALSE, then OpExecutionMode **must** not be IsoLines
- VUID-RuntimeSpirv-tessellationShader-06327
  If the tessellationShader feature is enabled, and the VK_KHR_portability_subset extension is enabled, and VkPhysicalDevicePortabilitySubsetFeaturesKHR::tessellationPointMode is VK_FALSE, then OpExecutionMode **must** not be PointMode
- VUID-RuntimeSpirv-storageBuffer8BitAccess-06328
  If storageBuffer8BitAccess is VK_FALSE, then objects containing an 8-bit integer element **must** not have Storage Class of StorageBuffer, ShaderRecordBufferKHR, or PhysicalStorageBuffer
- VUID-RuntimeSpirv-uniformAndStorageBuffer8BitAccess-06329
  If uniformAndStorageBuffer8BitAccess is VK_FALSE, then objects in the Uniform Storage Class with the Block decoration **must** not have an 8-bit integer member
- VUID-RuntimeSpirv-storagePushConstant8-06330
  If storagePushConstant8 is VK_FALSE, then objects containing an 8-bit integer element **must** not have Storage Class of PushConstant
- VUID-RuntimeSpirv-storageBuffer16BitAccess-06331
  If storageBuffer16BitAccess is VK_FALSE, then objects containing 16-bit integer or 16-bit floating-point elements **must** not have Storage Class of StorageBuffer, ShaderRecordBufferKHR, or PhysicalStorageBuffer
- VUID-RuntimeSpirv-uniformAndStorageBuffer16BitAccess-06332
  If uniformAndStorageBuffer16BitAccess is VK_FALSE, then objects in the Uniform Storage Class with the Block decoration **must** not have 16-bit integer or 16-bit floating-point members
- VUID-RuntimeSpirv-storagePushConstant16-06333
  If storagePushConstant16 is VK_FALSE, then objects containing 16-bit integer or 16-bit floating-point elements **must** not have Storage Class of PushConstant
- VUID-RuntimeSpirv-storageInputOutput16-06334
  If storageInputOutput16 is VK_FALSE, then objects containing 16-bit integer or 16-bit floating-point elements **must** not have Storage Class of Input or Output
- VUID-RuntimeSpirv-None-06337
  shaderBufferFloat16Atomics, or shaderBufferFloat16AtomicAdd, or shaderBufferFloat16AtomicMinMax, or shaderSharedFloat16Atomics, or shaderSharedFloat16AtomicAdd, or shaderSharedFloat16AtomicMinMax **must** be enabled for 16-bit floating-point atomic operations
- VUID-RuntimeSpirv-None-06338
  shaderBufferFloat32Atomics, or shaderBufferFloat32AtomicAdd, or shaderSharedFloat32Atomics, or shaderSharedFloat32AtomicAdd, or shaderImageFloat32Atomics, or shaderImageFloat32AtomicAdd or shaderBufferFloat32AtomicMinMax, or shaderSharedFloat32AtomicMinMax, or shaderImageFloat32AtomicMinMax **must** be enabled for 32-bit floating-point atomic operations

- VUID-RuntimeSpirv-None-06339
  `shaderBufferFloat64Atomics`, or `shaderBufferFloat64AtomicAdd`, or `shaderSharedFloat64Atomics`, or `shaderSharedFloat64AtomicAdd`, or `shaderBufferFloat64AtomicMinMax`, or `shaderSharedFloat64AtomicMinMax`, **must** be enabled for 64-bit floating-point atomic operations
- VUID-RuntimeSpirv-NonWritable-06340
  If the `fragmentStoresAndAtomics` feature is not enabled, then all storage image, storage texel buffer, and storage buffer variables in the fragment stage **must** be decorated with the `NonWritable` decoration
- VUID-RuntimeSpirv-NonWritable-06341
  If the `vertexPipelineStoresAndAtomics` feature is not enabled, then all storage image, storage texel buffer, and storage buffer variables in the vertex, tessellation, and geometry stages **must** be decorated with the `NonWritable` decoration
- VUID-RuntimeSpirv-None-06342
  If `subgroupQuadOperationsInAllStages` is `VK_FALSE`, then quad subgroup operations **must** not be used except for in fragment and compute stages
- VUID-RuntimeSpirv-Offset-06344
  The first element of the `Offset` operand of `InterpolateAtOffset` **must** be greater than or equal to:
  $frag_{width}$ × `minInterpolationOffset`
  where $frag_{width}$ is the width of the current fragment in pixels
- VUID-RuntimeSpirv-Offset-06345
  The first element of the `Offset` operand of `InterpolateAtOffset` **must** be less than or equal to
  $frag_{width}$ × (`maxInterpolationOffset` + ULP) - ULP
  where $frag_{width}$ is the width of the current fragment in pixels and ULP = 1 / 2^`subPixelInterpolationOffsetBits`^
- VUID-RuntimeSpirv-Offset-06346
  The second element of the `Offset` operand of `InterpolateAtOffset` **must** be greater than or equal to
  $frag_{height}$ × `minInterpolationOffset`
  where $frag_{height}$ is the height of the current fragment in pixels
- VUID-RuntimeSpirv-Offset-06347
  The second element of the `Offset` operand of `InterpolateAtOffset` **must** be less than or equal to
  $frag_{height}$ × (`maxInterpolationOffset` + ULP) - ULP
  where $frag_{height}$ is the height of the current fragment in pixels and ULP = 1 / 2^`subPixelInterpolationOffsetBits`^
- VUID-RuntimeSpirv-OpRayQueryInitializeKHR-06348
  For `OpRayQueryInitializeKHR` instructions, all components of the `RayOrigin` and `RayDirection` operands **must** be finite floating-point values
- VUID-RuntimeSpirv-OpRayQueryInitializeKHR-06349
  For `OpRayQueryInitializeKHR` instructions, the `RayTmin` and `RayTmax` operands **must** be non-negative floating-point values

- VUID-RuntimeSpirv-OpRayQueryInitializeKHR-06350
  For OpRayQueryInitializeKHR instructions, the RayTmin operand **must** be less than or equal to the RayTmax operand
- VUID-RuntimeSpirv-OpRayQueryInitializeKHR-06351
  For OpRayQueryInitializeKHR instructions, RayOrigin, RayDirection, RayTmin, and RayTmax operands **must** not contain NaNs
- VUID-RuntimeSpirv-OpRayQueryInitializeKHR-06352
  For OpRayQueryInitializeKHR instructions, Acceleration Structure **must** be an acceleration structure built as a top-level acceleration structure
- VUID-RuntimeSpirv-OpRayQueryInitializeKHR-06889
  For OpRayQueryInitializeKHR instructions, the Rayflags operand **must** not contain both SkipTrianglesKHR and SkipAABBsKHR
- VUID-RuntimeSpirv-OpRayQueryInitializeKHR-06890
  For OpRayQueryInitializeKHR instructions, the Rayflags operand **must** not contain more than one of SkipTrianglesKHR, CullBackFacingTrianglesKHR, and CullFrontFacingTrianglesKHR
- VUID-RuntimeSpirv-OpRayQueryInitializeKHR-06891
  For OpRayQueryInitializeKHR instructions, the Rayflags operand **must** not contain more than one of OpaqueKHR, NoOpaqueKHR, CullOpaqueKHR, and CullNoOpaqueKHR
- VUID-RuntimeSpirv-OpRayQueryGenerateIntersectionKHR-06353
  For OpRayQueryGenerateIntersectionKHR instructions, Hit T **must** satisfy the condition RayTmin ≤ Hit T ≤ RayTmax, where RayTmin is equal to the value returned by OpRayQueryGetRayTMinKHR with the same ray query object, and RayTmax is equal to the value of OpRayQueryGetIntersectionTKHR for the current committed intersection with the same ray query object
- VUID-RuntimeSpirv-flags-08761
  For OpRayQueryGetIntersectionTriangleVertexPositionsKHR instructions, Acceleration Structure **must** have been built with VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DATA_ACCESS_KHR in flags
- VUID-RuntimeSpirv-OpTraceRayKHR-06355
  For OpTraceRayKHR instructions, all components of the RayOrigin and RayDirection operands **must** be finite floating-point values
- VUID-RuntimeSpirv-OpTraceRayKHR-06356
  For OpTraceRayKHR instructions, the RayTmin and RayTmax operands **must** be non-negative floating-point values
- VUID-RuntimeSpirv-OpTraceRayKHR-06552
  For OpTraceRayKHR instructions, the Rayflags operand **must** not contain both SkipTrianglesKHR and SkipAABBsKHR
- VUID-RuntimeSpirv-OpTraceRayKHR-06892
  For OpTraceRayKHR instructions, the Rayflags operand **must** not contain more than one of SkipTrianglesKHR, CullBackFacingTrianglesKHR, and CullFrontFacingTrianglesKHR
- VUID-RuntimeSpirv-OpTraceRayKHR-06893
  For OpTraceRayKHR instructions, the Rayflags operand **must** not contain more than one of

OpaqueKHR, NoOpaqueKHR, CullOpaqueKHR, and CullNoOpaqueKHR

- VUID-RuntimeSpirv-OpTraceRayKHR-06553
  For OpTraceRayKHR instructions, if the Rayflags operand contains SkipTrianglesKHR, the pipeline **must** not have been created with VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR set
- VUID-RuntimeSpirv-OpTraceRayKHR-06554
  For OpTraceRayKHR instructions, if the Rayflags operand contains SkipAABBsKHR, the pipeline **must** not have been created with VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR set
- VUID-RuntimeSpirv-OpTraceRayKHR-06357
  For OpTraceRayKHR instructions, the RayTmin operand **must** be less than or equal to the RayTmax operand
- VUID-RuntimeSpirv-OpTraceRayKHR-06358
  For OpTraceRayKHR instructions, RayOrigin, RayDirection, RayTmin, and RayTmax operands **must** not contain NaNs
- VUID-RuntimeSpirv-OpTraceRayKHR-06359
  For OpTraceRayKHR instructions, Acceleration Structure **must** be an acceleration structure built as a top-level acceleration structure
- VUID-RuntimeSpirv-OpReportIntersectionKHR-06998
  The value of the “Hit Kind” operand of OpReportIntersectionKHR **must** be in the range [0,127]
- VUID-RuntimeSpirv-x-06429
  In compute shaders using the GLCompute Execution Model the x size in LocalSize or LocalSizeId **must** be less than or equal to VkPhysicalDeviceLimits ::maxComputeWorkGroupSize[0]
- VUID-RuntimeSpirv-y-06430
  In compute shaders using the GLCompute Execution Model the y size in LocalSize or LocalSizeId **must** be less than or equal to VkPhysicalDeviceLimits ::maxComputeWorkGroupSize[1]
- VUID-RuntimeSpirv-z-06431
  In compute shaders using the GLCompute Execution Model the z size in LocalSize or LocalSizeId **must** be less than or equal to VkPhysicalDeviceLimits ::maxComputeWorkGroupSize[2]
- VUID-RuntimeSpirv-x-06432
  In compute shaders using the GLCompute Execution Model the product of x size, y size, and z size in LocalSize or LocalSizeId **must** be less than or equal to VkPhysicalDeviceLimits ::maxComputeWorkGroupInvocations
- VUID-RuntimeSpirv-LocalSizeId-06434
  If Execution Mode LocalSizeId is used, maintenance4 **must** be enabled
- VUID-RuntimeSpirv-maintenance4-06817
  If the maintenance4 feature is not enabled, any OpTypeVector output interface variables **must** not have a higher Component Count than a matching OpTypeVector input interface variable

- VUID-RuntimeSpirv-OpEntryPoint-08743
  Any user-defined variables shared between the OpEntryPoint of two shader stages, and declared with Input as its Storage Class for the subsequent shader stage, **must** have all Location slots and Component words declared in the preceding shader stage's OpEntryPoint with Output as the Storage Class
- VUID-RuntimeSpirv-OpEntryPoint-07754
  Any user-defined variables between the OpEntryPoint of two shader stages **must** have the same type and width for each Component
- VUID-RuntimeSpirv-OpVariable-08746
  Any OpVariable, Block-decorated OpTypeStruct, or Block-decorated OpTypeStruct members shared between the OpEntryPoint of two shader stages **must** have matching decorations as defined in interface matching
- VUID-RuntimeSpirv-Workgroup-06530
  The sum of size in bytes for variables and padding in the Workgroup Storage Class in the GLCompute Execution Model **must** be less than or equal to maxComputeSharedMemorySize
- VUID-RuntimeSpirv-shaderZeroInitializeWorkgroupMemory-06372
  If the shaderZeroInitializeWorkgroupMemory feature is not enabled, any OpVariable with Workgroup as its Storage Class **must** not have an Initializer operand
- VUID-RuntimeSpirv-OpImage-06376
  If an OpImage*Gather operation has an image operand of Offset, ConstOffset, or ConstOffsets the offset value **must** be greater than or equal to minTexelGatherOffset
- VUID-RuntimeSpirv-OpImage-06377
  If an OpImage*Gather operation has an image operand of Offset, ConstOffset, or ConstOffsets the offset value **must** be less than or equal to maxTexelGatherOffset
- VUID-RuntimeSpirv-OpImageSample-06435
  If an OpImageSample* or OpImageFetch* operation has an image operand of ConstOffset then the offset value **must** be greater than or equal to minTexelOffset
- VUID-RuntimeSpirv-OpImageSample-06436
  If an OpImageSample* or OpImageFetch* operation has an image operand of ConstOffset then the offset value **must** be less than or equal to maxTexelOffset
- VUID-RuntimeSpirv-samples-08725
  If an OpTypeImage has an MS operand 0, its bound image **must** have been created with VkImageCreateInfo::samples as VK_SAMPLE_COUNT_1_BIT
- VUID-RuntimeSpirv-samples-08726
  If an OpTypeImage has an MS operand 1, its bound image **must** not have been created with VkImageCreateInfo::samples as VK_SAMPLE_COUNT_1_BIT
- VUID-RuntimeSpirv-SubgroupUniformControlFlowKHR-06379
  The Execution Mode SubgroupUniformControlFlowKHR **must** not be applied to an entry point unless the shaderSubgroupUniformControlFlow feature is enabled, the corresponding shader stage bit is set in subgroupSupportedStages, and the entry point does not execute any *invocation repack instructions*
- VUID-RuntimeSpirv-OpEntryPoint-08727
  Each OpEntryPoint **must** not have more than one variable decorated with

InputAttachmentIndex per image aspect of the attachment image bound to it, either explicitly or implicitly as described by input attachment interface

- VUID-RuntimeSpirv-shaderTileImageColorReadAccess-08728
  If the shaderTileImageColorReadAccess feature is not enabled, OpColorAttachmentReadEXT operation **must** not be used
- VUID-RuntimeSpirv-shaderTileImageDepthReadAccess-08729
  If the shaderTileImageDepthReadAccess feature is not enabled, OpDepthAttachmentReadEXT operation **must** not be used
- VUID-RuntimeSpirv-shaderTileImageStencilReadAccess-08730
  If the shaderTileImageStencilReadAccess feature is not enabled, OpStencilAttachmentReadEXT operation **must** not be used
- VUID-RuntimeSpirv-minSampleShading-08731
  If sample shading is enabled and minSampleShading is 1.0, the sample operand of any OpColorAttachmentReadEXT, OpDepthAttachmentReadEXT, or OpStencilAttachmentReadEXT operation **must** evaluate to the value of the coverage index for any given fragment invocation
- VUID-RuntimeSpirv-minSampleShading-08732
  If sample shading is enabled and any of the OpColorAttachmentReadEXT, OpDepthAttachmentReadEXT, or OpStencilAttachmentReadEXT operations are used, then minSampleShading **must** be 1.0
- VUID-RuntimeSpirv-MeshEXT-09218
  In mesh shaders using the MeshEXT or MeshNV Execution Model and the OutputPoints Execution Mode, if the maintenance5 feature is not enabled, and if the number of output points is greater than 0, a PointSize decorated variable **must** be written to for each output point
- VUID-RuntimeSpirv-maintenance5-09190
  If the maintenance5 feature is enabled and a PointSize decorated variable is written to, all execution paths **must** write to a PointSize decorated variable
- VUID-RuntimeSpirv-MaximallyReconvergesKHR-09565
  The execution mode MaximallyReconvergesKHR **must** not be applied to an entry point unless the entry point does not execute any *invocation repack instructions*
- VUID-RuntimeSpirv-shaderSubgroupRotateClustered-09566
  If shaderSubgroupRotateClustered is VK_FALSE, then the ClusterSize operand to OpGroupNonUniformRotateKHR **must** not be used
- VUID-RuntimeSpirv-protectedNoFault-09645
  If protectedNoFault is not supported, the Storage Class of the PhysicalStorageBuffer **must** not be used if the buffer being accessed is protected
- VUID-RuntimeSpirv-meshAndTaskShaderDerivatives-10153
  If meshAndTaskShaderDerivatives is VK_FALSE, the DerivativeGroupLinearKHR and DerivativeGroupQuadsKHR execution modes **must** not be used in the MeshEXT, MeshNV, TaskEXT, or TaskNV Execution Model

Precision and Operation of SPIR-V Instructions

The following rules apply to half, single, and double-precision floating-point instructions:

- Positive and negative infinities and positive and negative zeros are generated as dictated by IEEE 754, but subject to the precisions allowed in the table below.
- Signaling NaNs are not required to be generated and exceptions are never raised. Signaling NaN **may** be converted to quiet NaNs values by any floating-point instruction.
- The set of operations `OpPhi`, `OpSelect`, `OpFunctionCall`, `OpReturnValue`, `OpVectorExtractDynamic`, `OpVectorInsertDynamic`, `OpVectorShuffle`, `OpCompositeConstruct`, `OpCompositeExtract`, `OpCompositeInsert`, `OpTranspose`, `OpCopyObject`, `OpCopyLogical`, `OpCopyMemory`, `OpGroupNonUniformRotateKHR`, `OpCooperativeMatrixLoadKHR`, `OpCooperativeMatrixStoreKHR`, `OpAtomicLoad`, `OpAtomicStore`, `OpAtomicExchange`, `OpStore`, and `OpLoad` are referred to as *bit-preserving operations*.
- The floating-point environment used for an instruction can be determined as follows:
  - If the SPIR-V specifies it explicitly using the `FPFastMath` decoration or `FPFastMathDefault Execution Mode` then that is used.
  - If the environment is not specified in the SPIR-V then it is determined as follows:
    - If the operation is not decorated `NoContraction` then the flags `AllowContract`, `AllowReassoc`, `AllowRecip`, and `AllowTransform` are assumed.
    - If any of the following conditions are true then the flags `NSZ`, `NotInf`, and `NotNaN` are assumed:
      - The entry point does not use the `Execution Mode SignedZeroInfNanPreserve` with a bit-width corresponding to one of the operands or to the result type.
      - The operation is not a bit-preserving operation and is not one of `OpFConvert`, `OpFNegate`, `OpFAdd`, `OpFSub`, `OpFMul`, `OpFDiv`, `OpIsNan`, or `OpIsInf`.
      - The operation is an `OpLoad` from the `Input Storage Class` in the fragment shader stage.
- All bit-preserving operations and the following instructions **must** not flush denormalized values: `OpConstant`, `OpConstantComposite`, `OpSpecConstant`, `OpSpecConstantComposite`, and `OpBitcast`.
- Denormalized values are supported.
  - By default, any half, single, or double-precision denormalized value input into a shader or potentially generated by any instruction (except those listed above) or any extended instructions for GLSL in a shader **may** be flushed to zero.
  - If the entry point is declared with the `DenormFlushToZero Execution Mode` then for the affected instructions the denormalized result **must** be flushed to zero and the denormalized operands **may** be flushed to zero. Denormalized values obtained via unpacking an integer into a vector of values with smaller bit width and interpreting those values as floating-point numbers **must** be flushed to zero.
  - The following core SPIR-V instructions **must** respect the `DenormFlushToZero Execution Mode`: `OpSpecConstantOp` (with opcode `OpFConvert`), `OpFConvert`, `OpFNegate`, `OpFAdd`, `OpFSub`, `OpFMul`, `OpFDiv`, `OpFRem`, `OpFMod`, `OpVectorTimesScalar`, `OpMatrixTimesScalar`, `OpVectorTimesMatrix`, `OpMatrixTimesVector`, `OpMatrixTimesMatrix`, `OpOuterProduct`, `OpDot`; and the following extended

instructions for GLSL: Round, RoundEven, Trunc, FAbs, Floor, Ceil, Fract, Radians, Degrees, Sin, Cos, Tan, Asin, Acos, Atan, Sinh, Cosh, Tanh, Asinh, Acosh, Atanh, Atan2, Pow, Exp, Log, Exp2, Log2, Sqrt, InverseSqrt, Determinant, MatrixInverse, Modf, ModfStruct, FMin, FMax, FClamp, FMix, Step, SmoothStep, Fma, UnpackHalf2x16, Length, Distance, Cross, Normalize, FaceForward, Reflect, Refract, NMin, NMax, and NClamp.

- The following core SPIR-V instructions **must** respect the DenormPreserve Execution Mode: OpSpecConstantOp, OpFConvert, OpFNegate, OpFAdd, OpFSub, OpFMul, OpVectorTimesScalar, OpMatrixTimesScalar, OpVectorTimesMatrix, OpMatrixTimesVector, OpMatrixTimesMatrix, OpOuterProduct, OpDot, OpFOrdEqual, OpFUnordEqual, OpFOrdNotEqual, OpFUnordNotEqual, OpFOrdLessThan, OpFUnordLessThan, OpFOrdGreaterThan, OpFUnordGreaterThan, OpFOrdLessThanEqual, OpFUnordLessThanEqual, OpFOrdGreaterThanEqual, OpFUnordGreaterThanEqual, OpGroupNonUniformAllEqual; and the following extended instructions for GLSL: FAbs, FSign, Radians, Degrees, FMin, FMax, FClamp, FMix, Fma, PackHalf2x16, PackDouble2x32, UnpackHalf2x16, UnpackDouble2x32, NMin, NMax, and NClamp.

The precision of double-precision instructions is at least that of single precision.

The precision of individual operations is defined in Precision of Individual Operations. Subject to the constraints below, however, implementations **may** reorder or combine operations, resulting in expressions exhibiting different precisions than might be expected from the constituent operations.

Evaluation of Expressions

Implementations **may** rearrange floating-point operations using any of the mathematical properties governing the expressions in precise arithmetic, even where the floating- point operations do not share these properties. This includes, but is not limited to, associativity and distributivity, and **may** involve a different number of rounding steps than would occur if the operations were not rearranged. In shaders that use the SignedZeroInfNanPreserve Execution Mode the values **must** be preserved if they are generated after any rearrangement but the Execution Mode does not change which rearrangements are valid. This rearrangement **can** be prevented for particular operations by using the NoContraction decoration.

For example, in the absence of the NoContraction decoration implementations are allowed to implement a + b - a and $\frac{a \times b}{a}$ as b. The SignedZeroInfNanPreserve does not prevent these transformations, even though they may overflow to infinity or NaN when evaluated in floating-point.

NOTE

If the NoContraction decoration is applied then operations may not be rearranged, so, for example, a + a - a must account for possible overflow to infinity. If infinities are not preserved then the expression may be replaced with a, since the replacement is exact when overflow does not occur and infinities may be replaced with undefined values. If both NoContraction and SignedZeroInfNanPreserve are used then the result must be infinity for sufficiently large a.

Precision of Individual Operations

The precision of individual operations is defined either in terms of rounding (correctly rounded), as an error bound in ULP, or as inherited from a formula as follows:

Correct Result

Operations that are described as returning the "correct result" will return the infinitely precise result which, due to the nature of the operation, will not need rounding.

Correctly Rounded

Operations described as "correctly rounded" will return the infinitely precise result, x, rounded so as to be representable in floating-point. If the entry point is declared with the `RoundingModeRTE` or the `RoundingModeRTZ Execution Mode` then this is done according to IEEE 754 "roundTiesToEven" or "roundTowardZero" rounding directions, respectively. Otherwise, they are rounded with implementation-defined rounding mode.

Correctly Rounded with Implementation-Defined Rounding Mode

Operations described as "correctly rounded with implementation-defined rounding mode" will return the infinitely precise result, x, rounded so as to be representable in floating-point. If x is exactly representable then x will be returned. Otherwise, either the floating-point value closest to and no less than x or the value closest to and no greater than x will be returned. Which value is chosen is implementation-defined.

ULP

Where an error bound of n ULP (units in the last place) is given, for an operation with infinitely precise result x the value returned **must** be in the range [x - n × ulp(x), x + n × ulp(x)]. The function ulp(x) is defined as follows:

> If there exist non-equal, finite floating-point numbers a and b such that a ≤ x ≤ b then ulp(x) is the minimum possible distance between such numbers, $ulp(x) = \min_{a,\ b}|b - a|$. If such numbers do not exist then ulp(x) is defined to be the difference between the two non-equal, finite floating-point numbers nearest to x.

Where the range of allowed return values includes any value of magnitude larger than that of the largest representable finite floating-point number, operations **may**, additionally, return either an infinity of the appropriate sign or the finite number with the largest magnitude of the appropriate sign. If the infinitely precise result of the operation is not mathematically defined then the value returned is undefined.

Inherited From ...

Where an operation's precision is described as being inherited from a formula, the result returned **must** be at least as accurate as the result of computing an approximation to x using a formula equivalent to the given formula applied to the supplied inputs. Specifically, the formula given may be transformed using the mathematical associativity, commutativity and distributivity of the operators involved to yield an equivalent formula. The SPIR-V precision rules, when applied to each such formula and the given input values, define a range of permitted values. If NaN is one of the permitted values then the operation may return any result, otherwise let the largest permitted value in any of the ranges be F_{max} and the smallest be F_{min}. The operation **must** return a value in the range [x - E, x + E] where $E = \max(|x - F_{min}|, |x - F_{max}|)$. If the entry point is declared with the `DenormFlushToZero` execution mode, then any intermediate denormal value(s) while evaluating the formula **may** be flushed to zero. Denormal final results **must** be flushed to zero. If the entry point is declared with the `DenormPreserve Execution Mode`, then denormals **must** be preserved throughout the

formula.

For half- (16 bit) and single- (32 bit) precision instructions, precisions are **required** to be at least as follows:

Table 94. Precision of Core SPIR-V Instructions

Instruction	Single precision, unless decorated with RelaxedPrecision	Half precision
OpFNegate	Correct result.	
OpFAdd	Correctly rounded.	
OpFSub	Correctly rounded.	
OpFMul, OpVectorTimesScalar, OpMatrixTimesScalar	Correctly rounded.	
OpMatrixTimesVector	Inherited from $\sum_{j=0}^{n-1} M_{ij} \times v_j$.	
OpVectorTimesMatrix	Inherited from $\sum_{j=0}^{n-1} v_j \times M_{ji}$.	
OpMatrixTimesMatrix	Inherited from $\sum_{k=0}^{n-1} M_{ik} \times N_{kj}$.	
OpOuterProduct	Correctly rounded.	
OpDot(x, y)	Inherited from $\sum_{i=0}^{n-1} x_i \times y_i$.	
OpIsNan, OpIsInf	Correct result.	
OpFOrdEqual, OpFUnordEqual	Correct result.	
OpFOrdNotEqual, OpFUnordNotEqual	Correct result.	
OpFOrdLessThan, OpFUnordLessThan	Correct result.	
OpFOrdGreaterThan, OpFUnordGreaterThan	Correct result.	
OpFOrdLessThanEqual, OpFUnordLessThanEqual	Correct result.	
OpFOrdGreaterThanEqual, OpFUnordGreaterThanEqual	Correct result.	
OpFDiv(x,y)	2.5 ULP for $\|y\| = 0$ or $\|y\|$ in the range $[2^{-126}, 2^{126}]$.	2.5 ULP for $\|y\| = 0$ or $\|y\|$ in the range $[2^{-14}, 2^{14}]$.
OpFRem(x,y)	Inherited from x - y × trunc(x/y).	
OpFMod(x,y)	Inherited from x - y × floor(x/y).	
OpQuantizeToF16	Correctly rounded with implementation defined rounding mode.	
conversions between types	Correctly rounded.	

NOTE The OpFRem and OpFMod instructions use cheap approximations of remainder, and the error can be large due to the discontinuity in trunc() and floor(). This can produce mathematically unexpected results in some cases, such as FMod(x,x) computing x

rather than 0, and can also cause the result to have a different sign than the infinitely precise result.

Table 95. Precision of GLSL.std.450 Instructions

Instruction	Single precision, unless decorated with RelaxedPrecision	Half precision
fma()	Inherited from OpFMul followed by OpFAdd.	
exp(x), exp2(x)	$3 + 2 \times \lvert x \rvert$ ULP.	$1 + 2 \times \lvert x \rvert$ ULP.
log(), log2()	3 ULP outside the range $[0.5, 2.0]$. Absolute error $< 2^{-21}$ inside the range $[0.5, 2.0]$.	3 ULP outside the range $[0.5, 2.0]$. Absolute error $< 2^{-7}$ inside the range $[0.5, 2.0]$.
pow(x, y)	Inherited from exp2(y × log2(x)).	
sqrt()	Inherited from 1.0 / inversesqrt().	
inversesqrt()	2 ULP.	
radians(x)	Inherited from $x \times C_{\pi_180}$, where C_{π_180} is a correctly rounded approximation to $\frac{\pi}{180}$.	
degrees(x)	Inherited from $x \times C_{180_\pi}$, where C_{180_π} is a correctly rounded approximation to $\frac{180}{\pi}$.	
sin()	Absolute error $\leq 2^{-11}$ inside the range $[-\pi, \pi]$.	Absolute error $\leq 2^{-7}$ inside the range $[-\pi, \pi]$.
cos()	Absolute error $\leq 2^{-11}$ inside the range $[-\pi, \pi]$.	Absolute error $\leq 2^{-7}$ inside the range $[-\pi, \pi]$.
tan()	Inherited from $\frac{\sin()}{\cos()}$.	
asin(x)	Inherited from $\text{atan2}(x, sqrt(1.0 - x \times x))$.	
acos(x)	Inherited from $\text{atan2}(sqrt(1.0 - x \times x), x)$.	
atan(), atan2()	4096 ULP	5 ULP.
sinh(x)	Inherited from $(\exp(x) - \exp(-x)) \times 0.5$.	
cosh(x)	Inherited from $(\exp(x) + \exp(-x)) \times 0.5$.	
tanh()	Inherited from $\frac{\sinh()}{\cosh()}$.	
asinh(x)	Inherited from $\log(x + sqrt(x \times x + 1.0))$.	
acosh(x)	Inherited from $\log(x + sqrt(x \times x - 1.0))$.	
atanh(x)	Inherited from $\log(\frac{1.0 + x}{1.0 - x}) \times 0.5$.	
frexp()	Correct result.	
ldexp()	Correctly rounded.	
length(x)	Inherited from $sqrt(dot(x, x))$.	
distance(x, y)	Inherited from $length(x - y)$.	

Instruction	Single precision, unless decorated with RelaxedPrecision	Half precision
cross()	Inherited from OpFSub(OpFMul, OpFMul).	
normalize(x)	Inherited from $x \times inversesqrt(dot(x, x))$.	
faceforward(N, I, NRef)	Inherited from dot(NRef, I) < 0.0 ? N : -N.	
reflect(x, y)	Inherited from x - 2.0 × dot(y, x) × y.	
refract(I, N, eta)	Inherited from k < 0.0 ? 0.0 : eta × I - (eta × dot(N, I) + sqrt(k)) × N, where k = 1 - eta × eta × (1.0 - dot(N, I) × dot(N, I)).	
round	Correctly rounded.	
roundEven	Correctly rounded.	
trunc	Correctly rounded.	
fabs	Correct result.	
fsign	Correct result.	
floor	Correctly rounded.	
ceil	Correctly rounded.	
fract	Correctly rounded.	
modf	Correctly rounded.	
fmin	Correct result.	
fmax	Correct result.	
fclamp	Correct result.	
fmix(x, y, a)	Inherited from $x \times (1.0 - a) + y \times a$.	
step	Correctly rounded.	
smoothStep(edge0, edge1, x)	Inherited from $t \times t \times (3.0 - 2.0 \times t)$, where $t = clamp(\frac{x - edge0}{edge1 - edge0}, 0.0, 1.0)$.	
nmin	Correct result.	
nmax	Correct result.	
nclamp	Correct result.	
packHalf2x16	Correctly rounded with implementation defined rounding mode.	

GLSL.std.450 extended instructions specifically defined in terms of the above instructions inherit the above errors. GLSL.std.450 extended instructions not listed above and not defined in terms of the above have undefined precision.

For the OpSRem and OpSMod instructions, if either operand is negative the result is undefined.

NOTE While the OpSRem and OpSMod instructions are supported by the Vulkan environment, they require non-negative values and thus do not enable additional functionality

> beyond what OpUMod provides.

OpCooperativeMatrixMulAddKHR performs its operations in an implementation-dependent order and internal precision.

Signedness of SPIR-V Image Accesses

SPIR-V associates a signedness with all integer image accesses. This is required in certain parts of the SPIR-V and the Vulkan image access pipeline to ensure defined results. The signedness is determined from a combination of the access instruction's Image Operands and the underlying image's Sampled Type as follows:

1. If the instruction's Image Operands contains the SignExtend operand then the access is signed.
2. If the instruction's Image Operands contains the ZeroExtend operand then the access is unsigned.
3. Otherwise, the image accesses signedness matches that of the Sampled Type of the OpTypeImage being accessed.

Image Format and Type Matching

When specifying the Image Format of an OpTypeImage, the converted bit width and type, as shown in the table below, **must** match the Sampled Type. The signedness **must** match the signedness of any access to the image.

> **NOTE** Formatted accesses are always converted from a shader readable type to the resource's format or vice versa via Format Conversion for reads and Texel Output Format Conversion for writes. As such, the bit width and format below do not necessarily match 1:1 with what might be expected for some formats.

For a given Image Format, the Sampled Type **must** be the type described in the *Type* column of the below table, with its Literal Width set to that in the *Bit Width* column. Every access that is made to the image **must** have a signedness equal to that in the *Signedness* column (where applicable).

Image Format	Type-Declaration instructions	Bit Width	Signedness
Unknown	Any	Any	Any

Image Format	Type-Declaration instructions	Bit Width	Signedness
Rgba32f	OpTypeFloat	32	N/A
Rg32f			
R32f			
Rgba16f			
Rg16f			
R16f			
Rgba16			
Rg16			
R16			
Rgba16Snorm			
Rg16Snorm			
R16Snorm			
Rgb10A2			
R11fG11fB10f			
Rgba8			
Rg8			
R8			
Rgba8Snorm			
Rg8Snorm			
R8Snorm			

Image Format	Type-Declaration instructions	Bit Width	Signedness
Rgba32i	OpTypeInt	32	1
Rg32i			
R32i			
Rgba16i			
Rg16i			
R16i			
Rgba8i			
Rg8i			
R8i			
Rgba32ui			0
Rg32ui			
R32ui			
Rgba16ui			
Rg16ui			
R16ui			
Rgb10a2ui			
Rgba8ui			
Rg8ui			
R8ui			
R64i	OpTypeInt	64	1
R64ui			0

The *SPIR-V Type* is defined by an instruction in SPIR-V, declared with the Type-Declaration Instruction, Bit Width, and Signedness from above.

Compatibility Between SPIR-V Image Dimensions and Vulkan ImageView Types

SPIR-V `Image Dim` values are compatible with VkImageView `viewType` values as defined below:

Table 96. SPIR-V and Vulkan ImageView Dimension Compatibility

SPIR-V Image Dim	Compatible Vulkan ImageView viewTypes
1D	VK_IMAGE_VIEW_TYPE_1D, VK_IMAGE_VIEW_TYPE_1D_ARRAY
2D	VK_IMAGE_VIEW_TYPE_2D, VK_IMAGE_VIEW_TYPE_2D_ARRAY
3D	VK_IMAGE_VIEW_TYPE_3D
Cube	VK_IMAGE_VIEW_TYPE_CUBE, VK_IMAGE_VIEW_TYPE_CUBE_ARRAY

Compatibility Between SPIR-V Image Formats and Vulkan Formats

SPIR-V `Image Format` values are compatible with VkFormat values as defined below:

Table 97. SPIR-V and Vulkan Image Format Compatibility

SPIR-V Image Format	Compatible Vulkan Format
`Unknown`	Any
`R8`	`VK_FORMAT_R8_UNORM`
`R8Snorm`	`VK_FORMAT_R8_SNORM`
`R8ui`	`VK_FORMAT_R8_UINT`
`R8i`	`VK_FORMAT_R8_SINT`
`Rg8`	`VK_FORMAT_R8G8_UNORM`
`Rg8Snorm`	`VK_FORMAT_R8G8_SNORM`
`Rg8ui`	`VK_FORMAT_R8G8_UINT`
`Rg8i`	`VK_FORMAT_R8G8_SINT`
`Rgba8`	`VK_FORMAT_R8G8B8A8_UNORM`
`Rgba8Snorm`	`VK_FORMAT_R8G8B8A8_SNORM`
`Rgba8ui`	`VK_FORMAT_R8G8B8A8_UINT`
`Rgba8i`	`VK_FORMAT_R8G8B8A8_SINT`
`Rgb10A2`	`VK_FORMAT_A2B10G10R10_UNORM_PACK32`
`Rgb10a2ui`	`VK_FORMAT_A2B10G10R10_UINT_PACK32`
`R16`	`VK_FORMAT_R16_UNORM`
`R16Snorm`	`VK_FORMAT_R16_SNORM`
`R16ui`	`VK_FORMAT_R16_UINT`
`R16i`	`VK_FORMAT_R16_SINT`
`R16f`	`VK_FORMAT_R16_SFLOAT`
`Rg16`	`VK_FORMAT_R16G16_UNORM`
`Rg16Snorm`	`VK_FORMAT_R16G16_SNORM`
`Rg16ui`	`VK_FORMAT_R16G16_UINT`
`Rg16i`	`VK_FORMAT_R16G16_SINT`
`Rg16f`	`VK_FORMAT_R16G16_SFLOAT`
`Rgba16`	`VK_FORMAT_R16G16B16A16_UNORM`
`Rgba16Snorm`	`VK_FORMAT_R16G16B16A16_SNORM`
`Rgba16ui`	`VK_FORMAT_R16G16B16A16_UINT`
`Rgba16i`	`VK_FORMAT_R16G16B16A16_SINT`
`Rgba16f`	`VK_FORMAT_R16G16B16A16_SFLOAT`
`R32ui`	`VK_FORMAT_R32_UINT`
`R32i`	`VK_FORMAT_R32_SINT`
`R32f`	`VK_FORMAT_R32_SFLOAT`

SPIR-V Image Format	Compatible Vulkan Format
Rg32ui	VK_FORMAT_R32G32_UINT
Rg32i	VK_FORMAT_R32G32_SINT
Rg32f	VK_FORMAT_R32G32_SFLOAT
Rgba32ui	VK_FORMAT_R32G32B32A32_UINT
Rgba32i	VK_FORMAT_R32G32B32A32_SINT
Rgba32f	VK_FORMAT_R32G32B32A32_SFLOAT
R64ui	VK_FORMAT_R64_UINT
R64i	VK_FORMAT_R64_SINT
R11fG11fB10f	VK_FORMAT_B10G11R11_UFLOAT_PACK32

Ray Query Precision and Operation

The values returned by OpRayQueryGetIntersectionTriangleVertexPositionsKHR are transformed by the geometry transform, which is performed at standard floating-point precision, but without a specifically defined order of floating-point operations to perform the matrix multiplication.

Appendix B: Memory Model

NOTE This memory model describes synchronizations provided by all implementations; however, some of the synchronizations defined require extra features to be supported by the implementation. See VkPhysicalDeviceVulkanMemoryModelFeatures.

Agent

Operation is a general term for any task that is executed on the system.

NOTE An operation is by definition something that is executed. Thus if an instruction is skipped due to control flow, it does not constitute an operation.

Each operation is executed by a particular *agent*. Possible agents include each shader invocation, each host thread, and each fixed-function stage of the pipeline.

Memory Location

A *memory location* identifies unique storage for 8 bits of data. Memory operations access a *set of memory locations* consisting of one or more memory locations at a time, e.g. an operation accessing a 32-bit integer in memory would read/write a set of four memory locations. Memory operations that access whole aggregates **may** access any padding bytes between elements or members, but no padding bytes at the end of the aggregate. Two sets of memory locations *overlap* if the intersection of their sets of memory locations is non-empty. A memory operation **must** not affect memory at a memory location not within its set of memory locations.

Memory locations for buffers and images are explicitly allocated in VkDeviceMemory objects, and are implicitly allocated for SPIR-V variables in each shader invocation.

Variables with `Workgroup` storage class that point to a block-decorated type share a set of memory locations.

Allocation

The values stored in newly allocated memory locations are determined by a SPIR-V variable's initializer, if present, or else are undefined. At the time an allocation is created there have been no memory operations to any of its memory locations. The initialization is not considered to be a memory operation.

NOTE For tessellation control shader output variables, a consequence of initialization not being considered a memory operation is that some implementations may need to insert a barrier between the initialization of the output variables and any reads of those variables.

Memory Operation

For an operation A and memory location M:

- A *reads* M if and only if the data stored in M is an input to A.
- A *writes* M if and only if the data output from A is stored to M.
- A *accesses* M if and only if it either reads or writes (or both) M.

NOTE	A write whose value is the same as what was already in those memory locations is still considered to be a write and has all the same effects.

Reference

A *reference* is an object that a particular agent **can** use to access a set of memory locations. On the host, a reference is a host virtual address. On the device, a reference is:

- The descriptor that a variable is bound to, for variables in Image, Uniform, or StorageBuffer storage classes. If the variable is an array (or array of arrays, etc.) then each element of the array **may** be a unique reference.
- The address range for a buffer in `PhysicalStorageBuffer` storage class, where the base of the address range is queried with vkGetBufferDeviceAddressKHR and the length of the range is the size of the buffer.
- A single common reference for all variables with `Workgroup` storage class that point to a block-decorated type.
- The variable itself for non-block-decorated type variables in `Workgroup` storage class.
- The variable itself for variables in other storage classes.

Two memory accesses through distinct references **may** require availability and visibility operations as defined below.

Program-Order

A *dynamic instance* of an instruction is defined in SPIR-V (https://registry.khronos.org/spir-v/specs/unified1/SPIRV.html#DynamicInstance) as a way of referring to a particular execution of a static instruction. Program-order is an ordering on dynamic instances of instructions executed by a single shader invocation:

- (Basic block): If instructions A and B are in the same basic block, and A is listed in the module before B, then the n'th dynamic instance of A is program-ordered before the n'th dynamic instance of B.
- (Branch): The dynamic instance of a branch or switch instruction is program-ordered before the dynamic instance of the OpLabel instruction to which it transfers control.
- (Call entry): The dynamic instance of an `OpFunctionCall` instruction is program-ordered before the dynamic instances of the `OpFunctionParameter` instructions and the body of the called

function.

- (Call exit): The dynamic instance of the instruction following an `OpFunctionCall` instruction is program-ordered after the dynamic instance of the return instruction executed by the called function.
- (Transitive Closure): If dynamic instance A of any instruction is program-ordered before dynamic instance B of any instruction and B is program-ordered before dynamic instance C of any instruction then A is program-ordered before C.
- (Complete definition): No other dynamic instances are program-ordered.

For instructions executed on the host, the source language defines the program-order relation (e.g. as “sequenced-before”).

Shader Call Related

Shader-call-related is an equivalence relation on invocations defined as the symmetric and transitive closure of:

- A is shader-call-related to B if A is created by an shader call instruction executed by B.

Shader Call Order

Shader-call-order is a partial order on dynamic instances of instructions executed by invocations that are shader-call-related:

- (Program order): If dynamic instance A is program-ordered before B, then A is shader-call-ordered before B.
- (Shader call entry): If A is a dynamic instance of an shader call instruction and B is a dynamic instance executed by an invocation that is created by A, then A is shader-call-ordered before B.
- (Shader call exit): If A is a dynamic instance of an shader call instruction, B is the next dynamic instance executed by the same invocation, and C is a dynamic instance executed by an invocation that is created by A, then C is shader-call-ordered before B.
- (Transitive closure): If A is shader-call-ordered-before B and B is shader-call-ordered-before C, then A is shader-call-ordered-before C.
- (Complete definition): No other dynamic instances are shader-call-ordered.

Scope

Atomic and barrier instructions include scopes which identify sets of shader invocations that **must** obey the requested ordering and atomicity rules of the operation, as defined below.

The various scopes are described in detail in the Shaders chapter.

Atomic Operation

An *atomic operation* on the device is any SPIR-V operation whose name begins with `OpAtomic`. An atomic operation on the host is any operation performed with an std::atomic typed object.

Each atomic operation has a memory scope and a semantics. Informally, the scope determines which other agents it is atomic with respect to, and the semantics constrains its ordering against other memory accesses. Device atomic operations have explicit scopes and semantics. Each host atomic operation implicitly uses the `CrossDevice` scope, and uses a memory semantics equivalent to a C++ std::memory_order value of relaxed, acquire, release, acq_rel, or seq_cst.

Two atomic operations A and B are *potentially-mutually-ordered* if and only if all of the following are true:

- They access the same set of memory locations.
- They use the same reference.
- A is in the instance of B's memory scope.
- B is in the instance of A's memory scope.
- A and B are not the same operation (irreflexive).

Two atomic operations A and B are *mutually-ordered* if and only if they are potentially-mutually-ordered and any of the following are true:

- A and B are both device operations.
- A and B are both host operations.

NOTE	If two atomic operations are not mutually-ordered, and if their sets of memory locations overlap, then each **must** be synchronized against the other as if they were non-atomic operations.

Scoped Modification Order

For a given atomic write A, all atomic writes that are mutually-ordered with A occur in an order known as A's *scoped modification order*. A's scoped modification order relates no other operations.

NOTE	Invocations outside the instance of A's memory scope **may** observe the values at A's set of memory locations becoming visible to it in an order that disagrees with the scoped modification order.

NOTE	It is valid to have non-atomic operations or atomics in a different scope instance to the same set of memory locations, as long as they are synchronized against each other as if they were non-atomic (if they are not, it is treated as a data race). That means this definition of A's scoped modification order could include atomic operations that occur much later, after intervening non-atomics. That is a bit non-intuitive, but it helps to keep this definition simple and non-circular.

Memory Semantics

Non-atomic memory operations, by default, **may** be observed by one agent in a different order than they were written by another agent.

Atomics and some synchronization operations include *memory semantics*, which are flags that constrain the order in which other memory accesses (including non-atomic memory accesses and availability and visibility operations) performed by the same agent **can** be observed by other agents, or **can** observe accesses by other agents.

Device instructions that include semantics are `OpAtomic*`, `OpControlBarrier`, `OpMemoryBarrier`, and `OpMemoryNamedBarrier`. Host instructions that include semantics are some std::atomic methods and memory fences.

SPIR-V supports the following memory semantics:

- Relaxed: No constraints on order of other memory accesses.
- Acquire: A memory read with this semantic performs an *acquire operation*. A memory barrier with this semantic is an *acquire barrier*.
- Release: A memory write with this semantic performs a *release operation*. A memory barrier with this semantic is a *release barrier*.
- AcquireRelease: A memory read-modify-write operation with this semantic performs both an acquire operation and a release operation, and inherits the limitations on ordering from both of those operations. A memory barrier with this semantic is both a release and acquire barrier.

NOTE	SPIR-V does not support “consume” semantics on the device.

The memory semantics operand also includes *storage class semantics* which indicate which storage classes are constrained by the synchronization. SPIR-V storage class semantics include:

- UniformMemory
- WorkgroupMemory
- ImageMemory
- OutputMemory

Each SPIR-V memory operation accesses a single storage class. Semantics in synchronization operations can include a combination of storage classes.

The UniformMemory storage class semantic applies to accesses to memory in the PhysicalStorageBuffer, `ShaderRecordBufferKHR`, Uniform and StorageBuffer storage classes. The WorkgroupMemory storage class semantic applies to accesses to memory in the Workgroup storage class. The ImageMemory storage class semantic applies to accesses to memory in the Image storage class. The OutputMemory storage class semantic applies to accesses to memory in the Output storage class.

NOTE	Informally, these constraints limit how memory operations can be reordered, and these limits apply not only to the order of accesses as performed in the agent that

executes the instruction, but also to the order the effects of writes become visible to all other agents within the same instance of the instruction's memory scope.

NOTE Release and acquire operations in different threads **can** act as synchronization operations, to guarantee that writes that happened before the release are visible after the acquire. (This is not a formal definition, just an Informative forward reference.)

NOTE The OutputMemory storage class semantic is only useful in tessellation control shaders, which is the only execution model where output variables are shared between invocations.

The memory semantics operand **can** also include availability and visibility flags, which apply availability and visibility operations as described in availability and visibility. The availability/visibility flags are:

- MakeAvailable: Semantics **must** be Release or AcquireRelease. Performs an availability operation before the release operation or barrier.
- MakeVisible: Semantics **must** be Acquire or AcquireRelease. Performs a visibility operation after the acquire operation or barrier.

The specifics of these operations are defined in Availability and Visibility Semantics.

Host atomic operations **may** support a different list of memory semantics and synchronization operations, depending on the host architecture and source language.

Release Sequence

After an atomic operation A performs a release operation on a set of memory locations M, the *release sequence headed by A* is the longest continuous subsequence of A's scoped modification order that consists of:

- the atomic operation A as its first element
- atomic read-modify-write operations on M by any agent

NOTE The atomics in the last bullet **must** be mutually-ordered with A by virtue of being in A's scoped modification order.

NOTE This intentionally omits "atomic writes to M performed by the same agent that performed A", which is present in the corresponding C++ definition.

Synchronizes-With

Synchronizes-with is a relation between operations, where each operation is either an atomic operation or a memory barrier (aka fence on the host).

If A and B are atomic operations, then A synchronizes-with B if and only if all of the following are true:

- A performs a release operation
- B performs an acquire operation
- A and B are mutually-ordered
- B reads a value written by A or by an operation in the release sequence headed by A

`OpControlBarrier`, `OpMemoryBarrier`, and `OpMemoryNamedBarrier` are *memory barrier* instructions in SPIR-V.

If A is a release barrier and B is an atomic operation that performs an acquire operation, then A synchronizes-with B if and only if all of the following are true:

- there exists an atomic write X (with any memory semantics)
- A is program-ordered before X
- X and B are mutually-ordered
- B reads a value written by X or by an operation in the release sequence headed by X
    - If X is relaxed, it is still considered to head a hypothetical release sequence for this rule
- A and B are in the instance of each other's memory scopes
- X's storage class is in A's semantics.

If A is an atomic operation that performs a release operation and B is an acquire barrier, then A synchronizes-with B if and only if all of the following are true:

- there exists an atomic read X (with any memory semantics)
- X is program-ordered before B
- X and A are mutually-ordered
- X reads a value written by A or by an operation in the release sequence headed by A
- A and B are in the instance of each other's memory scopes
- X's storage class is in B's semantics.

If A is a release barrier and B is an acquire barrier, then A synchronizes-with B if all of the following are true:

- there exists an atomic write X (with any memory semantics)
- A is program-ordered before X
- there exists an atomic read Y (with any memory semantics)
- Y is program-ordered before B
- X and Y are mutually-ordered
- Y reads the value written by X or by an operation in the release sequence headed by X
    - If X is relaxed, it is still considered to head a hypothetical release sequence for this rule

- A and B are in the instance of each other's memory scopes
- X's and Y's storage class is in A's and B's semantics.
    - NOTE: X and Y **must** have the same storage class, because they are mutually ordered.

If A is a release barrier, B is an acquire barrier, and C is a control barrier (where A **can** equal C, and B **can** equal C), then A synchronizes-with B if all of the following are true:

- A is program-ordered before (or equals) C
- C is program-ordered before (or equals) B
- A and B are in the instance of each other's memory scopes
- A and B are in the instance of C's execution scope

> **NOTE** This is similar to the barrier-barrier synchronization above, but with a control barrier filling the role of the relaxed atomics.

Let F be an ordering of fragment shader invocations, such that invocation F_1 is ordered before invocation F_2 if and only if F_1 and F_2 overlap as described in Fragment Shader Interlock and F_1 executes the interlocked code before F_2.

If A is an `OpEndInvocationInterlockEXT` instruction and B is an `OpBeginInvocationInterlockEXT` instruction, then A synchronizes-with B if the agent that executes A is ordered before the agent that executes B in F. A and B are both considered to have `FragmentInterlock` memory scope and semantics of UniformMemory and ImageMemory, and A is considered to have Release semantics and B is considered to have Acquire semantics.

> **NOTE** `OpBeginInvocationInterlockEXT` and `OpBeginInvocationInterlockEXT` do not perform implicit availability or visibility operations. Usually, shaders using fragment shader interlock will declare the relevant resources as `coherent` to get implicit per-instruction availability and visibility operations.

If A is a release barrier and B is an acquire barrier, then A synchronizes-with B if all of the following are true:

- A is shader-call-ordered-before B
- A and B are in the instance of each other's memory scopes

No other release and acquire barriers synchronize-with each other.

System-Synchronizes-With

System-synchronizes-with is a relation between arbitrary operations on the device or host. Certain operations system-synchronize-with each other, which informally means the first operation occurs before the second and that the synchronization is performed without using application-visible memory accesses.

If there is an execution dependency between two operations A and B, then the operation in the first

synchronization scope system-synchronizes-with the operation in the second synchronization scope.

NOTE	This covers all Vulkan synchronization primitives, including device operations executing before a synchronization primitive is signaled, wait operations happening before subsequent device operations, signal operations happening before host operations that wait on them, and host operations happening before vkQueueSubmit. The list is spread throughout the synchronization chapter, and is not repeated here.

System-synchronizes-with implicitly includes all storage class semantics and has `CrossDevice` scope.

If A system-synchronizes-with B, we also say A is *system-synchronized-before* B and B is *system-synchronized-after* A.

Private vs. Non-Private

By default, non-atomic memory operations are treated as *private*, meaning such a memory operation is not intended to be used for communication with other agents. Memory operations with the NonPrivatePointer/NonPrivateTexel bit set are treated as *non-private*, and are intended to be used for communication with other agents.

More precisely, for private memory operations to be Location-Ordered between distinct agents requires using system-synchronizes-with rather than shader-based synchronization. Private memory operations still obey program-order.

Atomic operations are always considered non-private.

Inter-Thread-Happens-Before

Let SC be a non-empty set of storage class semantics. Then (using template syntax) operation A *inter-thread-happens-before*<SC> operation B if and only if any of the following is true:

- A system-synchronizes-with B
- A synchronizes-with B, and both A and B have all of SC in their semantics
- A is an operation on memory in a storage class in SC or that has all of SC in its semantics, B is a release barrier or release atomic with all of SC in its semantics, and A is program-ordered before B
- A is an acquire barrier or acquire atomic with all of SC in its semantics, B is an operation on memory in a storage class in SC or that has all of SC in its semantics, and A is program-ordered before B
- A and B are both host operations and A inter-thread-happens-before B as defined in the host language specification
- A inter-thread-happens-before<SC> some X and X inter-thread-happens-before<SC> B

Happens-Before

Operation A *happens-before* operation B if and only if any of the following is true:

- A is program-ordered before B
- A inter-thread-happens-before<SC> B for some set of storage classes SC

Happens-after is defined similarly.

NOTE	Unlike C++, happens-before is not always sufficient for a write to be visible to a read. Additional availability and visibility operations **may** be required for writes to be visible-to other memory accesses.

NOTE	Happens-before is not transitive, but each of program-order and inter-thread-happens-before<SC> are transitive. These can be thought of as covering the "single-threaded" case and the "multi-threaded" case, and it is not necessary (and not valid) to form chains between the two.

Availability and Visibility

Availability and *visibility* are states of a write operation, which (informally) track how far the write has permeated the system, i.e. which agents and references are able to observe the write. Availability state is per *memory domain*. Visibility state is per (agent,reference) pair. Availability and visibility states are per-memory location for each write.

Memory domains are named according to the agents whose memory accesses use the domain. Domains used by shader invocations are organized hierarchically into multiple smaller memory domains which correspond to the different scopes. Each memory domain is considered the *dual* of a scope, and vice versa. The memory domains defined in Vulkan include:

- *host* - accessible by host agents
- *device* - accessible by all device agents for a particular device
- *shader* - accessible by shader agents for a particular device, corresponding to the `Device` scope
- *queue family instance* - accessible by shader agents in a single queue family, corresponding to the `QueueFamily` scope.
- *fragment interlock instance* - accessible by fragment shader agents that overlap, corresponding to the `FragmentInterlock` scope.
- *shader call instance* - accessible by shader agents that are shader-call-related, corresponding to the `ShaderCallKHR` scope.
- *workgroup instance* - accessible by shader agents in the same workgroup, corresponding to the `Workgroup` scope.
- *subgroup instance* - accessible by shader agents in the same subgroup, corresponding to the `Subgroup` scope.

The memory domains are nested in the order listed above, except for shader call instance domain,

with memory domains later in the list nested in the domains earlier in the list. The shader call instance domain is at an implementation-dependent location in the list, and is nested according to that location. The shader call instance domain is not broader than the queue family instance domain.

NOTE

Memory domains do not correspond to storage classes or device-local and host-local VkDeviceMemory allocations, rather they indicate whether a write can be made visible only to agents in the same subgroup, same workgroup, overlapping fragment shader invocation, shader-call-related ray tracing invocation, in any shader invocation, or anywhere on the device, or host. The shader, queue family instance, fragment interlock instance, shader call instance, workgroup instance, and subgroup instance domains are only used for shader-based availability/visibility operations, in other cases writes can be made available from/visible to the shader via the device domain.

Availability operations, *visibility operations*, and *memory domain operations* alter the state of the write operations that happen-before them, and which are included in their *source scope* to be available or visible to their *destination scope*.

- For an availability operation, the source scope is a set of (agent,reference,memory location) tuples, and the destination scope is a set of memory domains.
- For a memory domain operation, the source scope is a memory domain and the destination scope is a memory domain.
- For a visibility operation, the source scope is a set of memory domains and the destination scope is a set of (agent,reference,memory location) tuples.

How the scopes are determined depends on the specific operation. Availability and memory domain operations expand the set of memory domains to which the write is available. Visibility operations expand the set of (agent,reference,memory location) tuples to which the write is visible.

Recall that availability and visibility states are per-memory location, and let W be a write operation to one or more locations performed by agent A via reference R. Let L be one of the locations written. (W,L) (the write W to L), is initially not available to any memory domain and only visible to (A,R,L). An availability operation AV that happens-after W and that includes (A,R,L) in its source scope makes (W,L) *available* to the memory domains in its destination scope.

A memory domain operation DOM that happens-after AV and for which (W,L) is available in the source scope makes (W,L) available in the destination memory domain.

A visibility operation VIS that happens-after AV (or DOM) and for which (W,L) is available in any domain in the source scope makes (W,L) *visible* to all (agent,reference,L) tuples included in its destination scope.

If write W_2 happens-after W, and their sets of memory locations overlap, then W will not be available/visible to all agents/references for those memory locations that overlap (and future AV/DOM/VIS ops cannot revive W's write to those locations).

Availability, memory domain, and visibility operations are treated like other non-atomic memory

accesses for the purpose of memory semantics, meaning they can be ordered by release-acquire sequences or memory barriers.

An *availability chain* is a sequence of availability operations to increasingly broad memory domains, where element N+1 of the chain is performed in the dual scope instance of the destination memory domain of element N and element N happens-before element N+1. An example is an availability operation with destination scope of the workgroup instance domain that happens-before an availability operation to the shader domain performed by an invocation in the same workgroup. An availability chain AVC that happens-after W and that includes (A,R,L) in the source scope makes (W,L) *available* to the memory domains in its final destination scope. An availability chain with a single element is just the availability operation.

Similarly, a *visibility chain* is a sequence of visibility operations from increasingly narrow memory domains, where element N of the chain is performed in the dual scope instance of the source memory domain of element N+1 and element N happens-before element N+1. An example is a visibility operation with source scope of the shader domain that happens-before a visibility operation with source scope of the workgroup instance domain performed by an invocation in the same workgroup. A visibility chain VISC that happens-after AVC (or DOM) and for which (W,L) is available in any domain in the source scope makes (W,L) *visible* to all (agent,reference,L) tuples included in its final destination scope. A visibility chain with a single element is just the visibility operation.

Availability, Visibility, and Domain Operations

The following operations generate availability, visibility, and domain operations. When multiple availability/visibility/domain operations are described, they are system-synchronized-with each other in the order listed.

An operation that performs a memory dependency generates:

- If the source access mask includes `VK_ACCESS_HOST_WRITE_BIT`, then the dependency includes a memory domain operation from host domain to device domain.
- An availability operation with source scope of all writes in the first access scope of the dependency and a destination scope of the device domain.
- A visibility operation with source scope of the device domain and destination scope of the second access scope of the dependency.
- If the destination access mask includes `VK_ACCESS_HOST_READ_BIT` or `VK_ACCESS_HOST_WRITE_BIT`, then the dependency includes a memory domain operation from device domain to host domain.

vkFlushMappedMemoryRanges performs an availability operation, with a source scope of (agents,references) = (all host threads, all mapped memory ranges passed to the command), and destination scope of the host domain.

vkInvalidateMappedMemoryRanges performs a visibility operation, with a source scope of the host domain and a destination scope of (agents,references) = (all host threads, all mapped memory ranges passed to the command).

vkQueueSubmit performs a memory domain operation from host to device, and a visibility

operation with source scope of the device domain and destination scope of all agents and references on the device.

Availability and Visibility Semantics

A memory barrier or atomic operation via agent A that includes MakeAvailable in its semantics performs an availability operation whose source scope includes agent A and all references in the storage classes in that instruction's storage class semantics, and all memory locations, and whose destination scope is a set of memory domains selected as specified below. The implicit availability operation is program-ordered between the barrier or atomic and all other operations program-ordered before the barrier or atomic.

A memory barrier or atomic operation via agent A that includes MakeVisible in its semantics performs a visibility operation whose source scope is a set of memory domains selected as specified below, and whose destination scope includes agent A and all references in the storage classes in that instruction's storage class semantics, and all memory locations. The implicit visibility operation is program-ordered between the barrier or atomic and all other operations program-ordered after the barrier or atomic.

The memory domains are selected based on the memory scope of the instruction as follows:

- `Device` scope uses the shader domain
- `QueueFamily` scope uses the queue family instance domain
- `FragmentInterlock` scope uses the fragment interlock instance domain
- `ShaderCallKHR` scope uses the shader call instance domain
- `Workgroup` scope uses the workgroup instance domain
- `Subgroup` uses the subgroup instance domain
- `Invocation` perform no availability/visibility operations.

When an availability operation performed by an agent A includes a memory domain D in its destination scope, where D corresponds to scope instance S, it also includes the memory domains that correspond to each smaller scope instance S' that is a subset of S and that includes A. Similarly for visibility operations.

Per-Instruction Availability and Visibility Semantics

A memory write instruction that includes MakePointerAvailable, or an image write instruction that includes MakeTexelAvailable, performs an availability operation whose source scope includes the agent and reference used to perform the write and the memory locations written by the instruction, and whose destination scope is a set of memory domains selected by the Scope operand specified in Availability and Visibility Semantics. The implicit availability operation is program-ordered between the write and all other operations program-ordered after the write.

A memory read instruction that includes MakePointerVisible, or an image read instruction that includes MakeTexelVisible, performs a visibility operation whose source scope is a set of memory domains selected by the Scope operand as specified in Availability and Visibility Semantics, and

whose destination scope includes the agent and reference used to perform the read and the memory locations read by the instruction. The implicit visibility operation is program-ordered between read and all other operations program-ordered before the read.

NOTE Although reads with per-instruction visibility only perform visibility ops from the shader or fragment interlock instance or shader call instance or workgroup instance or subgroup instance domain, they will also see writes that were made visible via the device domain, i.e. those writes previously performed by non-shader agents and made visible via API commands.

NOTE It is expected that all invocations in a subgroup execute on the same processor with the same path to memory, and thus availability and visibility operations with subgroup scope can be expected to be “free”.

Location-Ordered

Let X and Y be memory accesses to overlapping sets of memory locations M, where X != Y. Let (A_X,R_X) be the agent and reference used for X, and (A_Y,R_Y) be the agent and reference used for Y. For now, let “→” denote happens-before and “$\rightarrow^{rcpo}$” denote the reflexive closure of program-ordered before.

If D_1 and D_2 are different memory domains, then let DOM(D_1,D_2) be a memory domain operation from D_1 to D_2. Otherwise, let DOM(D,D) be a placeholder such that X→DOM(D,D)→Y if and only if X→Y.

X is *location-ordered* before Y for a location L in M if and only if any of the following is true:

- A_X == A_Y and R_X == R_Y and X→Y
  - NOTE: this case means no availability/visibility ops are required when it is the same (agent,reference).
- X is a read, both X and Y are non-private, and X→Y
- X is a read, and X (transitively) system-synchronizes with Y
- If R_X == R_Y and A_X and A_Y access a common memory domain D (e.g. are in the same workgroup instance if D is the workgroup instance domain), and both X and Y are non-private:
  - X is a write, Y is a write, AVC(A_X,R_X,D,L) is an availability chain making (X,L) available to domain D, and X→^{rcpo}AVC(A_X,R_X,D,L)→Y
  - X is a write, Y is a read, AVC(A_X,R_X,D,L) is an availability chain making (X,L) available to domain D, VISC(A_Y,R_Y,D,L) is a visibility chain making writes to L available in domain D visible to Y, and X→^{rcpo}AVC(A_X,R_X,D,L)→VISC(A_Y,R_Y,D,L)→^{rcpo}Y
  - If VkPhysicalDeviceVulkanMemoryModelFeatures::`vulkanMemoryModelAvailabilityVisibilityChains` is `VK_FALSE`, then AVC and VISC **must** each only have a single element in the chain, in each sub-bullet above.
- Let D_X and D_Y each be either the device domain or the host domain, depending on whether A_X and A_Y execute on the device or host:
  - X is a write and Y is a write, and X→AV(A_X,R_X,D_X,L)→DOM(D_X,D_Y)→Y

- X is a write and Y is a read, and $X \rightarrow AV(A_X,R_X,D_X,L) \rightarrow DOM(D_X,D_Y) \rightarrow VIS(A_Y,R_Y,D_Y,L) \rightarrow Y$

NOTE	The final bullet (synchronization through device/host domain) requires API-level synchronization operations, since the device/host domains are not accessible via shader instructions. And “device domain” is not to be confused with “device scope”, which synchronizes through the “shader domain”.

Data Race

Let X and Y be operations that access overlapping sets of memory locations M, where X != Y, and at least one of X and Y is a write, and X and Y are not mutually-ordered atomic operations. If there does not exist a location-ordered relation between X and Y for each location in M, then there is a *data race*.

Applications **must** ensure that no data races occur during the execution of their application.

NOTE	Data races can only occur due to instructions that are actually executed. For example, an instruction skipped due to control flow must not contribute to a data race.

Visible-To

Let X be a write and Y be a read whose sets of memory locations overlap, and let M be the set of memory locations that overlap. Let M_2 be a non-empty subset of M. Then X is *visible-to* Y for memory locations M_2 if and only if all of the following are true:

- X is location-ordered before Y for each location L in M_2.
- There does not exist another write Z to any location L in M_2 such that X is location-ordered before Z for location L and Z is location-ordered before Y for location L.

If X is visible-to Y, then Y reads the value written by X for locations M_2.

NOTE	It is possible for there to be a write between X and Y that overwrites a subset of the memory locations, but the remaining memory locations (M_2) will still be visible-to Y.

Acyclicity

Reads-from is a relation between operations, where the first operation is a write, the second operation is a read, and the second operation reads the value written by the first operation. *From-reads* is a relation between operations, where the first operation is a read, the second operation is a write, and the first operation reads a value written earlier than the second operation in the second operation’s scoped modification order or location order (or the first operation reads from the initial value, and the second operation is any write to the same locations).

Then the implementation **must** guarantee that no cycles exist in the union of the following relations:

- location-ordered
- scoped modification order (over all atomic writes)
- reads-from
- from-reads

NOTE	This is a “consistency” axiom, which informally guarantees that sequences of operations cannot violate causality.

Scoped Modification Order Coherence

Let A and B be mutually-ordered atomic operations, where A is location-ordered before B. Then the following rules are a consequence of acyclicity:

- If A and B are both reads and A does not read the initial value, then the write that A takes its value from **must** be earlier in its own scoped modification order than (or the same as) the write that B takes its value from (no cycles between location-order, reads-from, and from-reads).
- If A is a read and B is a write and A does not read the initial value, then A **must** take its value from a write earlier than B in B’s scoped modification order (no cycles between location-order, scope modification order, and reads-from).
- If A is a write and B is a read, then B **must** take its value from A or a write later than A in A’s scoped modification order (no cycles between location-order, scoped modification order, and from-reads).
- If A and B are both writes, then A **must** be earlier than B in A’s scoped modification order (no cycles between location-order and scoped modification order).
- If A is a write and B is a read-modify-write and B reads the value written by A, then B comes immediately after A in A’s scoped modification order (no cycles between scoped modification order and from-reads).

Shader I/O

If a shader invocation A in a shader stage other than `Vertex` performs a memory read operation X from an object in storage class `CallableDataKHR`, `IncomingCallableDataKHR`, `RayPayloadKHR`, `HitAttributeKHR`, `IncomingRayPayloadKHR`, or `Input`, then X is system-synchronized-after all writes to the corresponding `CallableDataKHR`, `IncomingCallableDataKHR`, `RayPayloadKHR`, `HitAttributeKHR`, `IncomingRayPayloadKHR`, or `Output` storage variable(s) in the shader invocation(s) that contribute to generating invocation A, and those writes are all visible-to X.

NOTE	It is not necessary for the upstream shader invocations to have completed execution, they only need to have generated the output that is being read.

Deallocation

A call to vkFreeMemory **must** happen-after all memory operations on all memory locations in that VkDeviceMemory object.

NOTE	Normally, device memory operations in a given queue are synchronized with vkFreeMemory by having a host thread wait on a fence signaled by that queue, and the wait happens-before the call to vkFreeMemory on the host.

The deallocation of SPIR-V variables is managed by the system and happens-after all operations on those variables.

Descriptions (Informative)

This subsection offers more easily understandable consequences of the memory model for app/compiler developers.

Let SC be the storage class(es) specified by a release or acquire operation or barrier.

- An atomic write with release semantics must not be reordered against any read or write to SC that is program-ordered before it (regardless of the storage class the atomic is in).
- An atomic read with acquire semantics must not be reordered against any read or write to SC that is program-ordered after it (regardless of the storage class the atomic is in).
- Any write to SC program-ordered after a release barrier must not be reordered against any read or write to SC program-ordered before that barrier.
- Any read from SC program-ordered before an acquire barrier must not be reordered against any read or write to SC program-ordered after the barrier.

A control barrier (even if it has no memory semantics) must not be reordered against any memory barriers.

This memory model allows memory accesses with and without availability and visibility operations, as well as atomic operations, all to be performed on the same memory location. This is critical to allow it to reason about memory that is reused in multiple ways, e.g. across the lifetime of different shader invocations or draw calls. While GLSL (and legacy SPIR-V) applies the “coherent” decoration to variables (for historical reasons), this model treats each memory access instruction as having optional implicit availability/visibility operations. GLSL to SPIR-V compilers should map all (non-atomic) operations on a coherent variable to Make{Pointer,Texel}{Available}{Visible} flags in this model.

Atomic operations implicitly have availability/visibility operations, and the scope of those operations is taken from the atomic operation’s scope.

Tessellation Output Ordering

For SPIR-V that uses the Vulkan Memory Model, the `OutputMemory` storage class is used to synchronize accesses to tessellation control output variables. For legacy SPIR-V that does not enable the Vulkan Memory Model via `OpMemoryModel`, tessellation outputs can be ordered using a control barrier with no particular memory scope or semantics, as defined below.

Let X and Y be memory operations performed by shader invocations A_X and A_Y. Operation X is *tessellation-output-ordered* before operation Y if and only if all of the following are true:

- There is a dynamic instance of an `OpControlBarrier` instruction C such that X is program-ordered before C in A_X and C is program-ordered before Y in A_Y.
- A_X and A_Y are in the same instance of C's execution scope.

If shader invocations A_X and A_Y in the `TessellationControl` execution model execute memory operations X and Y, respectively, on the `Output` storage class, and X is tessellation-output-ordered before Y with a scope of `Workgroup`, then X is location-ordered before Y, and if X is a write and Y is a read then X is visible-to Y.

Cooperative Matrix Memory Access

For each dynamic instance of a cooperative matrix load instruction (`OpCooperativeMatrixLoadKHR`), some implementation-dependent invocation(s) within the instance of the matrix's scope perform a non-atomic load from each memory location that is defined to be accessed by the instruction.

For each memory location accessed by a dynamic instance of a cooperative matrix store instruction (`OpCooperativeMatrixStoreKHR`), a single implementation-dependent invocation within the instance of the matrix's scope performs a non-atomic store to that memory location.

Appendix C: Compressed Image Formats

The compressed texture formats used by Vulkan are described in the specifically identified sections of the Khronos Data Format Specification, version 1.3.

Unless otherwise described, the quantities encoded in these compressed formats are treated as normalized, unsigned values.

Those formats listed as sRGB-encoded have in-memory representations of R, G and B components which are nonlinearly-encoded as R', G', and B'; any alpha component is unchanged. As part of filtering, the nonlinear R', G', and B' values are converted to linear R, G, and B components; any alpha component is unchanged. The conversion between linear and nonlinear encoding is performed as described in the “KHR_DF_TRANSFER_SRGB” section of the Khronos Data Format Specification.

Block-Compressed Image Formats

BC1, BC2 and BC3 formats are described in “S3TC Compressed Texture Image Formats” chapter of the Khronos Data Format Specification. BC4 and BC5 are described in the “RGTC Compressed Texture Image Formats” chapter. BC6H and BC7 are described in the “BPTC Compressed Texture Image Formats” chapter.

Table 98. Mapping of Vulkan BC Formats to Descriptions

VkFormat	Khronos Data Format Specification description
Formats described in the “S3TC Compressed Texture Image Formats” chapter	
VK_FORMAT_BC1_RGB_UNORM_BLOCK	BC1 with no alpha
VK_FORMAT_BC1_RGB_SRGB_BLOCK	BC1 with no alpha, sRGB-encoded
VK_FORMAT_BC1_RGBA_UNORM_BLOCK	BC1 with alpha
VK_FORMAT_BC1_RGBA_SRGB_BLOCK	BC1 with alpha, sRGB-encoded
VK_FORMAT_BC2_UNORM_BLOCK	BC2
VK_FORMAT_BC2_SRGB_BLOCK	BC2, sRGB-encoded
VK_FORMAT_BC3_UNORM_BLOCK	BC3
VK_FORMAT_BC3_SRGB_BLOCK	BC3, sRGB-encoded
Formats described in the “RGTC Compressed Texture Image Formats” chapter	
VK_FORMAT_BC4_UNORM_BLOCK	BC4 unsigned
VK_FORMAT_BC4_SNORM_BLOCK	BC4 signed
VK_FORMAT_BC5_UNORM_BLOCK	BC5 unsigned
VK_FORMAT_BC5_SNORM_BLOCK	BC5 signed
Formats described in the “BPTC Compressed Texture Image Formats” chapter	
VK_FORMAT_BC6H_UFLOAT_BLOCK	BC6H (unsigned version)
VK_FORMAT_BC6H_SFLOAT_BLOCK	BC6H (signed version)
VK_FORMAT_BC7_UNORM_BLOCK	BC7
VK_FORMAT_BC7_SRGB_BLOCK	BC7, sRGB-encoded

ETC Compressed Image Formats

The following formats are described in the “ETC2 Compressed Texture Image Formats” chapter of the Khronos Data Format Specification.

Table 99. Mapping of Vulkan ETC Formats to Descriptions

VkFormat	Khronos Data Format Specification description
VK_FORMAT_ETC2_R8G8B8_UNORM_BLOCK	RGB ETC2
VK_FORMAT_ETC2_R8G8B8_SRGB_BLOCK	RGB ETC2 with sRGB encoding
VK_FORMAT_ETC2_R8G8B8A1_UNORM_BLOCK	RGB ETC2 with punch-through alpha
VK_FORMAT_ETC2_R8G8B8A1_SRGB_BLOCK	RGB ETC2 with punch-through alpha and sRGB
VK_FORMAT_ETC2_R8G8B8A8_UNORM_BLOCK	RGBA ETC2
VK_FORMAT_ETC2_R8G8B8A8_SRGB_BLOCK	RGBA ETC2 with sRGB encoding
VK_FORMAT_EAC_R11_UNORM_BLOCK	Unsigned R11 EAC
VK_FORMAT_EAC_R11_SNORM_BLOCK	Signed R11 EAC
VK_FORMAT_EAC_R11G11_UNORM_BLOCK	Unsigned RG11 EAC
VK_FORMAT_EAC_R11G11_SNORM_BLOCK	Signed RG11 EAC

ASTC Compressed Image Formats

ASTC formats are described in the “ASTC Compressed Texture Image Formats” chapter of the Khronos Data Format Specification.

Table 100. Mapping of Vulkan ASTC Formats to Descriptions

VkFormat	Compressed texel block dimensions	Requested mode
VK_FORMAT_ASTC_4x4_UNORM_BLOCK	4 × 4	Linear LDR
VK_FORMAT_ASTC_4x4_SRGB_BLOCK	4 × 4	sRGB
VK_FORMAT_ASTC_5x4_UNORM_BLOCK	5 × 4	Linear LDR
VK_FORMAT_ASTC_5x4_SRGB_BLOCK	5 × 4	sRGB
VK_FORMAT_ASTC_5x5_UNORM_BLOCK	5 × 5	Linear LDR
VK_FORMAT_ASTC_5x5_SRGB_BLOCK	5 × 5	sRGB
VK_FORMAT_ASTC_6x5_UNORM_BLOCK	6 × 5	Linear LDR
VK_FORMAT_ASTC_6x5_SRGB_BLOCK	6 × 5	sRGB
VK_FORMAT_ASTC_6x6_UNORM_BLOCK	6 × 6	Linear LDR
VK_FORMAT_ASTC_6x6_SRGB_BLOCK	6 × 6	sRGB
VK_FORMAT_ASTC_8x5_UNORM_BLOCK	8 × 5	Linear LDR
VK_FORMAT_ASTC_8x5_SRGB_BLOCK	8 × 5	sRGB
VK_FORMAT_ASTC_8x6_UNORM_BLOCK	8 × 6	Linear LDR
VK_FORMAT_ASTC_8x6_SRGB_BLOCK	8 × 6	sRGB
VK_FORMAT_ASTC_8x8_UNORM_BLOCK	8 × 8	Linear LDR
VK_FORMAT_ASTC_8x8_SRGB_BLOCK	8 × 8	sRGB
VK_FORMAT_ASTC_10x5_UNORM_BLOCK	10 × 5	Linear LDR
VK_FORMAT_ASTC_10x5_SRGB_BLOCK	10 × 5	sRGB
VK_FORMAT_ASTC_10x6_UNORM_BLOCK	10 × 6	Linear LDR
VK_FORMAT_ASTC_10x6_SRGB_BLOCK	10 × 6	sRGB
VK_FORMAT_ASTC_10x8_UNORM_BLOCK	10 × 8	Linear LDR
VK_FORMAT_ASTC_10x8_SRGB_BLOCK	10 × 8	sRGB
VK_FORMAT_ASTC_10x10_UNORM_BLOCK	10 × 10	Linear LDR
VK_FORMAT_ASTC_10x10_SRGB_BLOCK	10 × 10	sRGB
VK_FORMAT_ASTC_12x10_UNORM_BLOCK	12 × 10	Linear LDR
VK_FORMAT_ASTC_12x10_SRGB_BLOCK	12 × 10	sRGB
VK_FORMAT_ASTC_12x12_UNORM_BLOCK	12 × 12	Linear LDR

VkFormat	Compressed texel block dimensions	Requested mode
VK_FORMAT_ASTC_12x12_SRGB_BLOCK	12 × 12	sRGB

ASTC textures containing any HDR blocks **should** not be passed into the API using an sRGB or UNORM texture format.

NOTE An HDR block in a texture passed using a LDR UNORM format will return the appropriate ASTC error color if the implementation supports only the ASTC LDR profile, but may result in either the error color or a decompressed HDR color if the implementation supports HDR decoding.

ASTC Decode Mode

If the VK_EXT_astc_decode_mode extension is enabled, the decode mode is determined as follows:

Table 101. Mapping of Vulkan ASTC Decoding Format to ASTC Decoding Modes

VkFormat	Decoding mode
VK_FORMAT_R16G16B16A16_SFLOAT	decode_float16
VK_FORMAT_R8G8B8A8_UNORM	decode_unorm8
VK_FORMAT_E5B9G9R9_UFLOAT_PACK32	decode_rgb9e5

Otherwise, the ASTC decode mode is decode_float16.

Note that an implementation **may** use HDR mode when linear LDR mode is requested unless the decode mode is decode_unorm8.

Appendix D: Core Revisions (Informative)

New minor versions of the Vulkan API are defined periodically by the Khronos Vulkan Working Group. These consist of some amount of additional functionality added to the core API, potentially including both new functionality and functionality promoted from extensions.

Vulkan Version 1.0

Vulkan Version 1.0 was the initial release of the Vulkan API.

New Macros

- VK_API_VERSION
- VK_API_VERSION_1_0
- VK_API_VERSION_MAJOR
- VK_API_VERSION_MINOR
- VK_API_VERSION_PATCH
- VK_API_VERSION_VARIANT
- VK_DEFINE_HANDLE
- VK_DEFINE_NON_DISPATCHABLE_HANDLE
- VK_HEADER_VERSION
- VK_HEADER_VERSION_COMPLETE
- VK_MAKE_API_VERSION
- VK_MAKE_VERSION
- VK_NULL_HANDLE
- VK_USE_64_BIT_PTR_DEFINES
- VK_VERSION_MAJOR
- VK_VERSION_MINOR
- VK_VERSION_PATCH

New Base Types

- VkBool32
- VkDeviceAddress
- VkDeviceSize
- VkFlags
- VkSampleMask

New Object Types

- VkBuffer
- VkBufferView
- VkCommandBuffer
- VkCommandPool
- VkDescriptorPool
- VkDescriptorSet
- VkDescriptorSetLayout
- VkDevice
- VkDeviceMemory
- VkEvent
- VkFence
- VkFramebuffer
- VkImage
- VkImageView
- VkInstance
- VkPhysicalDevice
- VkPipeline
- VkPipelineCache
- VkPipelineLayout
- VkQueryPool
- VkQueue
- VkRenderPass
- VkSampler
- VkSemaphore
- VkShaderModule

New Commands

- vkAllocateCommandBuffers
- vkAllocateDescriptorSets
- vkAllocateMemory
- vkBeginCommandBuffer
- vkBindBufferMemory
- vkBindImageMemory

- vkCmdBeginQuery
- vkCmdBeginRenderPass
- vkCmdBindDescriptorSets
- vkCmdBindIndexBuffer
- vkCmdBindPipeline
- vkCmdBindVertexBuffers
- vkCmdBlitImage
- vkCmdClearAttachments
- vkCmdClearColorImage
- vkCmdClearDepthStencilImage
- vkCmdCopyBuffer
- vkCmdCopyBufferToImage
- vkCmdCopyImage
- vkCmdCopyImageToBuffer
- vkCmdCopyQueryPoolResults
- vkCmdDispatch
- vkCmdDispatchIndirect
- vkCmdDraw
- vkCmdDrawIndexed
- vkCmdDrawIndexedIndirect
- vkCmdDrawIndirect
- vkCmdEndQuery
- vkCmdEndRenderPass
- vkCmdExecuteCommands
- vkCmdFillBuffer
- vkCmdNextSubpass
- vkCmdPipelineBarrier
- vkCmdPushConstants
- vkCmdResetEvent
- vkCmdResetQueryPool
- vkCmdResolveImage
- vkCmdSetBlendConstants
- vkCmdSetDepthBias
- vkCmdSetDepthBounds
- vkCmdSetEvent

- vkCmdSetLineWidth
- vkCmdSetScissor
- vkCmdSetStencilCompareMask
- vkCmdSetStencilReference
- vkCmdSetStencilWriteMask
- vkCmdSetViewport
- vkCmdUpdateBuffer
- vkCmdWaitEvents
- vkCmdWriteTimestamp
- vkCreateBuffer
- vkCreateBufferView
- vkCreateCommandPool
- vkCreateComputePipelines
- vkCreateDescriptorPool
- vkCreateDescriptorSetLayout
- vkCreateDevice
- vkCreateEvent
- vkCreateFence
- vkCreateFramebuffer
- vkCreateGraphicsPipelines
- vkCreateImage
- vkCreateImageView
- vkCreateInstance
- vkCreatePipelineCache
- vkCreatePipelineLayout
- vkCreateQueryPool
- vkCreateRenderPass
- vkCreateSampler
- vkCreateSemaphore
- vkCreateShaderModule
- vkDestroyBuffer
- vkDestroyBufferView
- vkDestroyCommandPool
- vkDestroyDescriptorPool
- vkDestroyDescriptorSetLayout

- vkDestroyDevice
- vkDestroyEvent
- vkDestroyFence
- vkDestroyFramebuffer
- vkDestroyImage
- vkDestroyImageView
- vkDestroyInstance
- vkDestroyPipeline
- vkDestroyPipelineCache
- vkDestroyPipelineLayout
- vkDestroyQueryPool
- vkDestroyRenderPass
- vkDestroySampler
- vkDestroySemaphore
- vkDestroyShaderModule
- vkDeviceWaitIdle
- vkEndCommandBuffer
- vkEnumerateDeviceExtensionProperties
- vkEnumerateDeviceLayerProperties
- vkEnumerateInstanceExtensionProperties
- vkEnumerateInstanceLayerProperties
- vkEnumeratePhysicalDevices
- vkFlushMappedMemoryRanges
- vkFreeCommandBuffers
- vkFreeDescriptorSets
- vkFreeMemory
- vkGetBufferMemoryRequirements
- vkGetDeviceMemoryCommitment
- vkGetDeviceProcAddr
- vkGetDeviceQueue
- vkGetEventStatus
- vkGetFenceStatus
- vkGetImageMemoryRequirements
- vkGetImageSparseMemoryRequirements
- vkGetImageSubresourceLayout

- vkGetInstanceProcAddr
- vkGetPhysicalDeviceFeatures
- vkGetPhysicalDeviceFormatProperties
- vkGetPhysicalDeviceImageFormatProperties
- vkGetPhysicalDeviceMemoryProperties
- vkGetPhysicalDeviceProperties
- vkGetPhysicalDeviceQueueFamilyProperties
- vkGetPhysicalDeviceSparseImageFormatProperties
- vkGetPipelineCacheData
- vkGetQueryPoolResults
- vkGetRenderAreaGranularity
- vkInvalidateMappedMemoryRanges
- vkMapMemory
- vkMergePipelineCaches
- vkQueueBindSparse
- vkQueueSubmit
- vkQueueWaitIdle
- vkResetCommandBuffer
- vkResetCommandPool
- vkResetDescriptorPool
- vkResetEvent
- vkResetFences
- vkSetEvent
- vkUnmapMemory
- vkUpdateDescriptorSets
- vkWaitForFences

New Structures

- VkAllocationCallbacks
- VkApplicationInfo
- VkAttachmentDescription
- VkAttachmentReference
- VkBaseInStructure
- VkBaseOutStructure
- VkBindSparseInfo

- VkBufferCopy
- VkBufferCreateInfo
- VkBufferImageCopy
- VkBufferMemoryBarrier
- VkBufferViewCreateInfo
- VkClearAttachment
- VkClearDepthStencilValue
- VkClearRect
- VkCommandBufferAllocateInfo
- VkCommandBufferBeginInfo
- VkCommandBufferInheritanceInfo
- VkCommandPoolCreateInfo
- VkComponentMapping
- VkComputePipelineCreateInfo
- VkCopyDescriptorSet
- VkDescriptorBufferInfo
- VkDescriptorImageInfo
- VkDescriptorPoolCreateInfo
- VkDescriptorPoolSize
- VkDescriptorSetAllocateInfo
- VkDescriptorSetLayoutBinding
- VkDescriptorSetLayoutCreateInfo
- VkDeviceCreateInfo
- VkDeviceQueueCreateInfo
- VkDispatchIndirectCommand
- VkDrawIndexedIndirectCommand
- VkDrawIndirectCommand
- VkEventCreateInfo
- VkExtensionProperties
- VkExtent2D
- VkExtent3D
- VkFenceCreateInfo
- VkFormatProperties
- VkFramebufferCreateInfo
- VkGraphicsPipelineCreateInfo

- VkImageBlit
- VkImageCopy
- VkImageCreateInfo
- VkImageFormatProperties
- VkImageMemoryBarrier
- VkImageResolve
- VkImageSubresource
- VkImageSubresourceLayers
- VkImageSubresourceRange
- VkImageViewCreateInfo
- VkInstanceCreateInfo
- VkLayerProperties
- VkMappedMemoryRange
- VkMemoryAllocateInfo
- VkMemoryBarrier
- VkMemoryHeap
- VkMemoryRequirements
- VkMemoryType
- VkOffset2D
- VkOffset3D
- VkPhysicalDeviceFeatures
- VkPhysicalDeviceLimits
- VkPhysicalDeviceMemoryProperties
- VkPhysicalDeviceProperties
- VkPhysicalDeviceSparseProperties
- VkPipelineCacheCreateInfo
- VkPipelineCacheHeaderVersionOne
- VkPipelineColorBlendAttachmentState
- VkPipelineColorBlendStateCreateInfo
- VkPipelineDepthStencilStateCreateInfo
- VkPipelineDynamicStateCreateInfo
- VkPipelineInputAssemblyStateCreateInfo
- VkPipelineMultisampleStateCreateInfo
- VkPipelineRasterizationStateCreateInfo
- VkPipelineShaderStageCreateInfo

- VkPipelineTessellationStateCreateInfo
- VkPipelineVertexInputStateCreateInfo
- VkPipelineViewportStateCreateInfo
- VkPushConstantRange
- VkQueryPoolCreateInfo
- VkQueueFamilyProperties
- VkRect2D
- VkRenderPassBeginInfo
- VkRenderPassCreateInfo
- VkSamplerCreateInfo
- VkSemaphoreCreateInfo
- VkSparseBufferMemoryBindInfo
- VkSparseImageFormatProperties
- VkSparseImageMemoryBind
- VkSparseImageMemoryBindInfo
- VkSparseImageMemoryRequirements
- VkSparseImageOpaqueMemoryBindInfo
- VkSparseMemoryBind
- VkSpecializationInfo
- VkSpecializationMapEntry
- VkStencilOpState
- VkSubmitInfo
- VkSubpassDependency
- VkSubpassDescription
- VkSubresourceLayout
- VkVertexInputAttributeDescription
- VkVertexInputBindingDescription
- VkViewport
- VkWriteDescriptorSet
- Extending VkBindDescriptorSetsInfoKHR, VkPushConstantsInfoKHR, VkPushDescriptorSetInfoKHR, VkPushDescriptorSetWithTemplateInfoKHR, VkSetDescriptorBufferOffsetsInfoEXT, VkBindDescriptorBufferEmbeddedSamplersInfoEXT, VkIndirectCommandsLayoutCreateInfoEXT:
  - VkPipelineLayoutCreateInfo
- Extending VkPipelineShaderStageCreateInfo:

    - VkShaderModuleCreateInfo

New Unions

- VkClearColorValue
- VkClearValue

New Function Pointers

- PFN_vkAllocationFunction
- PFN_vkFreeFunction
- PFN_vkInternalAllocationNotification
- PFN_vkInternalFreeNotification
- PFN_vkReallocationFunction
- PFN_vkVoidFunction

New Enums

- VkAccessFlagBits
- VkAttachmentDescriptionFlagBits
- VkAttachmentLoadOp
- VkAttachmentStoreOp
- VkBlendFactor
- VkBlendOp
- VkBorderColor
- VkBufferCreateFlagBits
- VkBufferUsageFlagBits
- VkColorComponentFlagBits
- VkCommandBufferLevel
- VkCommandBufferResetFlagBits
- VkCommandBufferUsageFlagBits
- VkCommandPoolCreateFlagBits
- VkCommandPoolResetFlagBits
- VkCompareOp
- VkComponentSwizzle
- VkCullModeFlagBits
- VkDependencyFlagBits
- VkDescriptorPoolCreateFlagBits

- VkDescriptorSetLayoutCreateFlagBits
- VkDescriptorType
- VkDynamicState
- VkEventCreateFlagBits
- VkFenceCreateFlagBits
- VkFilter
- VkFormat
- VkFormatFeatureFlagBits
- VkFramebufferCreateFlagBits
- VkFrontFace
- VkImageAspectFlagBits
- VkImageCreateFlagBits
- VkImageLayout
- VkImageTiling
- VkImageType
- VkImageUsageFlagBits
- VkImageViewCreateFlagBits
- VkImageViewType
- VkIndexType
- VkInstanceCreateFlagBits
- VkInternalAllocationType
- VkLogicOp
- VkMemoryHeapFlagBits
- VkMemoryMapFlagBits
- VkMemoryPropertyFlagBits
- VkObjectType
- VkPhysicalDeviceType
- VkPipelineBindPoint
- VkPipelineCacheHeaderVersion
- VkPipelineCreateFlagBits
- VkPipelineShaderStageCreateFlagBits
- VkPipelineStageFlagBits
- VkPolygonMode
- VkPrimitiveTopology
- VkQueryControlFlagBits

- VkQueryPipelineStatisticFlagBits
- VkQueryResultFlagBits
- VkQueryType
- VkQueueFlagBits
- VkRenderPassCreateFlagBits
- VkResult
- VkSampleCountFlagBits
- VkSamplerAddressMode
- VkSamplerCreateFlagBits
- VkSamplerMipmapMode
- VkShaderStageFlagBits
- VkSharingMode
- VkSparseImageFormatFlagBits
- VkSparseMemoryBindFlagBits
- VkStencilFaceFlagBits
- VkStencilOp
- VkStructureType
- VkSubpassContents
- VkSubpassDescriptionFlagBits
- VkSystemAllocationScope
- VkVendorId
- VkVertexInputRate

New Bitmasks

- VkAccessFlags
- VkAttachmentDescriptionFlags
- VkBufferCreateFlags
- VkBufferUsageFlags
- VkBufferViewCreateFlags
- VkColorComponentFlags
- VkCommandBufferResetFlags
- VkCommandBufferUsageFlags
- VkCommandPoolCreateFlags
- VkCommandPoolResetFlags
- VkCullModeFlags

- VkDependencyFlags
- VkDescriptorPoolCreateFlags
- VkDescriptorPoolResetFlags
- VkDescriptorSetLayoutCreateFlags
- VkDeviceCreateFlags
- VkDeviceQueueCreateFlags
- VkEventCreateFlags
- VkFenceCreateFlags
- VkFormatFeatureFlags
- VkFramebufferCreateFlags
- VkImageAspectFlags
- VkImageCreateFlags
- VkImageUsageFlags
- VkImageViewCreateFlags
- VkInstanceCreateFlags
- VkMemoryHeapFlags
- VkMemoryMapFlags
- VkMemoryPropertyFlags
- VkPipelineCacheCreateFlags
- VkPipelineColorBlendStateCreateFlags
- VkPipelineCreateFlags
- VkPipelineDepthStencilStateCreateFlags
- VkPipelineDynamicStateCreateFlags
- VkPipelineInputAssemblyStateCreateFlags
- VkPipelineLayoutCreateFlags
- VkPipelineMultisampleStateCreateFlags
- VkPipelineRasterizationStateCreateFlags
- VkPipelineShaderStageCreateFlags
- VkPipelineStageFlags
- VkPipelineTessellationStateCreateFlags
- VkPipelineVertexInputStateCreateFlags
- VkPipelineViewportStateCreateFlags
- VkQueryControlFlags
- VkQueryPipelineStatisticFlags
- VkQueryPoolCreateFlags

- VkQueryResultFlags
- VkQueueFlags
- VkRenderPassCreateFlags
- VkSampleCountFlags
- VkSamplerCreateFlags
- VkSemaphoreCreateFlags
- VkShaderModuleCreateFlags
- VkShaderStageFlags
- VkSparseImageFormatFlags
- VkSparseMemoryBindFlags
- VkStencilFaceFlags
- VkSubpassDescriptionFlags

New Headers

- `vk_platform`

New Enum Constants

- `VK_ATTACHMENT_UNUSED`
- `VK_FALSE`
- `VK_LOD_CLAMP_NONE`
- `VK_MAX_DESCRIPTION_SIZE`
- `VK_MAX_EXTENSION_NAME_SIZE`
- `VK_MAX_MEMORY_HEAPS`
- `VK_MAX_MEMORY_TYPES`
- `VK_MAX_PHYSICAL_DEVICE_NAME_SIZE`
- `VK_QUEUE_FAMILY_IGNORED`
- `VK_REMAINING_ARRAY_LAYERS`
- `VK_REMAINING_MIP_LEVELS`
- `VK_SUBPASS_EXTERNAL`
- `VK_TRUE`
- `VK_UUID_SIZE`
- `VK_WHOLE_SIZE`

Appendix E: Layers & Extensions (Informative)

Extensions to the Vulkan API **can** be defined by authors, groups of authors, and the Khronos Vulkan Working Group. In order not to compromise the readability of the Vulkan Specification, the core Specification does not incorporate most extensions. The online Registry of extensions is available at URL

https://registry.khronos.org/vulkan/

and allows generating versions of the Specification incorporating different extensions.

Authors creating extensions and layers **must** follow the mandatory procedures described in the Vulkan Documentation and Extensions document when creating extensions and layers.

The remainder of this appendix documents a set of extensions chosen when this document was built. Versions of the Specification published in the Registry include:

- Core API + mandatory extensions required of all Vulkan implementations.
- Core API + all registered and published Khronos (`KHR`) extensions.
- Core API + all registered and published extensions.

Extensions are grouped as Khronos `KHR`, multivendor `EXT`, and then alphabetically by author ID. Within each group, extensions are listed in alphabetical order by their name.

Extension Dependencies

Extensions which have dependencies on specific core versions or on other extensions will list such dependencies.

For core versions, the specified version **must** be supported at runtime. All extensions implicitly require support for Vulkan 1.0.

For a device extension, use of any device-level functionality defined by that extension requires that any extensions that extension depends on be enabled.

For any extension, use of any instance-level functionality defined by that extension requires only that any extensions that extension depends on be supported at runtime.

Extension Interactions

Some extensions define APIs which are only supported when other extensions or core versions are supported at runtime. Such interactions are noted as “API Interactions”.

List of Current Extensions

- VK_KHR_16bit_storage (promoted to core)
- VK_KHR_8bit_storage (promoted to core)
- VK_KHR_acceleration_structure
- VK_KHR_android_surface
- VK_KHR_bind_memory2 (promoted to core)
- VK_KHR_buffer_device_address (promoted to core)
- VK_KHR_calibrated_timestamps
- VK_KHR_compute_shader_derivatives
- VK_KHR_cooperative_matrix
- VK_KHR_copy_commands2 (promoted to core)
- VK_KHR_create_renderpass2 (promoted to core)
- VK_KHR_dedicated_allocation (promoted to core)
- VK_KHR_deferred_host_operations
- VK_KHR_depth_stencil_resolve (promoted to core)
- VK_KHR_descriptor_update_template (promoted to core)
- VK_KHR_device_group (promoted to core)
- VK_KHR_device_group_creation (promoted to core)
- VK_KHR_display
- VK_KHR_display_swapchain
- VK_KHR_draw_indirect_count (promoted to core)
- VK_KHR_driver_properties (promoted to core)
- VK_KHR_dynamic_rendering (promoted to core)
- VK_KHR_dynamic_rendering_local_read
- VK_KHR_external_fence (promoted to core)
- VK_KHR_external_fence_capabilities (promoted to core)
- VK_KHR_external_fence_fd
- VK_KHR_external_fence_win32
- VK_KHR_external_memory (promoted to core)
- VK_KHR_external_memory_capabilities (promoted to core)
- VK_KHR_external_memory_fd
- VK_KHR_external_memory_win32
- VK_KHR_external_semaphore (promoted to core)
- VK_KHR_external_semaphore_capabilities (promoted to core)

- VK_KHR_external_semaphore_fd
- VK_KHR_external_semaphore_win32
- VK_KHR_format_feature_flags2 (promoted to core)
- VK_KHR_fragment_shader_barycentric
- VK_KHR_fragment_shading_rate
- VK_KHR_get_display_properties2
- VK_KHR_get_memory_requirements2 (promoted to core)
- VK_KHR_get_physical_device_properties2 (promoted to core)
- VK_KHR_get_surface_capabilities2
- VK_KHR_global_priority
- VK_KHR_image_format_list (promoted to core)
- VK_KHR_imageless_framebuffer (promoted to core)
- VK_KHR_incremental_present
- VK_KHR_index_type_uint8
- VK_KHR_line_rasterization
- VK_KHR_load_store_op_none
- VK_KHR_maintenance1 (promoted to core)
- VK_KHR_maintenance2 (promoted to core)
- VK_KHR_maintenance3 (promoted to core)
- VK_KHR_maintenance4 (promoted to core)
- VK_KHR_maintenance5
- VK_KHR_maintenance6
- VK_KHR_maintenance7
- VK_KHR_map_memory2
- VK_KHR_multiview (promoted to core)
- VK_KHR_performance_query
- VK_KHR_pipeline_binary
- VK_KHR_pipeline_executable_properties
- VK_KHR_pipeline_library
- VK_KHR_portability_enumeration
- VK_KHR_present_id
- VK_KHR_present_wait
- VK_KHR_push_descriptor
- VK_KHR_ray_query
- VK_KHR_ray_tracing_maintenance1

- VK_KHR_ray_tracing_pipeline
- VK_KHR_ray_tracing_position_fetch
- VK_KHR_relaxed_block_layout (promoted to core)
- VK_KHR_sampler_mirror_clamp_to_edge (promoted to core)
- VK_KHR_sampler_ycbcr_conversion (promoted to core)
- VK_KHR_separate_depth_stencil_layouts (promoted to core)
- VK_KHR_shader_atomic_int64 (promoted to core)
- VK_KHR_shader_clock
- VK_KHR_shader_draw_parameters (promoted to core)
- VK_KHR_shader_expect_assume
- VK_KHR_shader_float16_int8 (promoted to core)
- VK_KHR_shader_float_controls (promoted to core)
- VK_KHR_shader_float_controls2
- VK_KHR_shader_integer_dot_product (promoted to core)
- VK_KHR_shader_maximal_reconvergence
- VK_KHR_shader_non_semantic_info (promoted to core)
- VK_KHR_shader_quad_control
- VK_KHR_shader_relaxed_extended_instruction
- VK_KHR_shader_subgroup_extended_types (promoted to core)
- VK_KHR_shader_subgroup_rotate
- VK_KHR_shader_subgroup_uniform_control_flow
- VK_KHR_shader_terminate_invocation (promoted to core)
- VK_KHR_shared_presentable_image
- VK_KHR_spirv_1_4 (promoted to core)
- VK_KHR_storage_buffer_storage_class (promoted to core)
- VK_KHR_surface
- VK_KHR_surface_protected_capabilities
- VK_KHR_swapchain
- VK_KHR_swapchain_mutable_format
- VK_KHR_synchronization2 (promoted to core)
- VK_KHR_timeline_semaphore (promoted to core)
- VK_KHR_uniform_buffer_standard_layout (promoted to core)
- VK_KHR_variable_pointers (promoted to core)
- VK_KHR_vertex_attribute_divisor
- VK_KHR_video_decode_av1

- VK_KHR_video_decode_h264
- VK_KHR_video_decode_h265
- VK_KHR_video_decode_queue
- VK_KHR_video_encode_av1
- VK_KHR_video_encode_h264
- VK_KHR_video_encode_h265
- VK_KHR_video_encode_quantization_map
- VK_KHR_video_encode_queue
- VK_KHR_video_maintenance1
- VK_KHR_video_queue
- VK_KHR_vulkan_memory_model (promoted to core)
- VK_KHR_wayland_surface
- VK_KHR_win32_keyed_mutex
- VK_KHR_win32_surface
- VK_KHR_workgroup_memory_explicit_layout
- VK_KHR_xcb_surface
- VK_KHR_xlib_surface
- VK_KHR_zero_initialize_workgroup_memory (promoted to core)
- VK_EXT_astc_decode_mode
- VK_EXT_attachment_feedback_loop_dynamic_state
- VK_EXT_attachment_feedback_loop_layout
- VK_EXT_blend_operation_advanced
- VK_EXT_border_color_swizzle
- VK_EXT_color_write_enable
- VK_EXT_conservative_rasterization
- VK_EXT_custom_border_color
- VK_EXT_debug_utils
- VK_EXT_depth_bias_control
- VK_EXT_depth_clamp_control
- VK_EXT_depth_clip_control
- VK_EXT_depth_clip_enable
- VK_EXT_depth_range_unrestricted
- VK_EXT_device_generated_commands
- VK_EXT_device_memory_report
- VK_EXT_direct_mode_display

- VK_EXT_discard_rectangles
- VK_EXT_display_control
- VK_EXT_display_surface_counter
- VK_EXT_dynamic_rendering_unused_attachments
- VK_EXT_extended_dynamic_state3
- VK_EXT_external_memory_acquire_unmodified
- VK_EXT_external_memory_dma_buf
- VK_EXT_external_memory_host
- VK_EXT_filter_cubic
- VK_EXT_fragment_density_map
- VK_EXT_fragment_density_map2
- VK_EXT_fragment_shader_interlock
- VK_EXT_frame_boundary
- VK_EXT_full_screen_exclusive
- VK_EXT_hdr_metadata
- VK_EXT_headless_surface
- VK_EXT_host_image_copy
- VK_EXT_image_2d_view_of_3d
- VK_EXT_image_drm_format_modifier
- VK_EXT_image_view_min_lod
- VK_EXT_layer_settings
- VK_EXT_legacy_vertex_attributes
- VK_EXT_map_memory_placed
- VK_EXT_memory_budget
- VK_EXT_memory_priority
- VK_EXT_metal_objects
- VK_EXT_multi_draw
- VK_EXT_nested_command_buffer
- VK_EXT_non_seamless_cube_map
- VK_EXT_opacity_micromap
- VK_EXT_pci_bus_info
- VK_EXT_physical_device_drm
- VK_EXT_post_depth_coverage
- VK_EXT_present_mode_fifo_latest_ready
- VK_EXT_primitive_topology_list_restart

- VK_EXT_primitives_generated_query
- VK_EXT_provoking_vertex
- VK_EXT_queue_family_foreign
- VK_EXT_robustness2
- VK_EXT_sample_locations
- VK_EXT_shader_atomic_float
- VK_EXT_shader_atomic_float2
- VK_EXT_shader_image_atomic_int64
- VK_EXT_shader_module_identifier
- VK_EXT_shader_object
- VK_EXT_shader_replicated_composites
- VK_EXT_shader_stencil_export
- VK_EXT_shader_tile_image
- VK_EXT_swapchain_colorspace
- VK_EXT_transform_feedback
- VK_EXT_validation_cache
- VK_EXT_vertex_input_dynamic_state
- VK_EXT_ycbcr_image_arrays

VK_KHR_16bit_storage

Name String

`VK_KHR_16bit_storage`

Extension Type

Device extension

Registered Extension Number

84

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
and
VK_KHR_storage_buffer_storage_class
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_16bit_storage

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jan-Harald Fredriksen [GitHub]janharaldfredriksen-arm

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_shader_16bit_storage`

Contributors

- Alexander Galazin, ARM
- Jan-Harald Fredriksen, ARM
- Joerg Wagner, ARM

- Neil Henning, Codeplay
- Jeff Bolz, Nvidia
- Daniel Koch, Nvidia
- David Neto, Google
- John Kessenich, Google

Description

The `VK_KHR_16bit_storage` extension allows use of 16-bit types in shader input and output interfaces, and push constant blocks. This extension introduces several new optional features which map to SPIR-V capabilities and allow access to 16-bit data in `Block`-decorated objects in the `Uniform` and the `StorageBuffer` storage classes, and objects in the `PushConstant` storage class. This extension allows 16-bit variables to be declared and used as user-defined shader inputs and outputs but does not change location assignment and component assignment rules.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. However, if Vulkan 1.1 is supported and this extension is not, the `storageBuffer16BitAccess` capability is optional. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevice16BitStorageFeaturesKHR

New Enum Constants

- `VK_KHR_16BIT_STORAGE_EXTENSION_NAME`
- `VK_KHR_16BIT_STORAGE_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_16BIT_STORAGE_FEATURES_KHR`

New SPIR-V Capabilities

- `StorageBuffer16BitAccess`
- `UniformAndStorageBuffer16BitAccess`
- `StoragePushConstant16`
- `StorageInputOutput16`

Version History

- Revision 1, 2017-03-23 (Alexander Galazin)
  - Initial draft

VK_KHR_8bit_storage

Name String

`VK_KHR_8bit_storage`

Extension Type

Device extension

Registered Extension Number

178

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
and
VK_KHR_storage_buffer_storage_class
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_8bit_storage

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Alexander Galazin [GitHub]alegal-arm

Other Extension Metadata

Last Modified Date

2018-02-05

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_shader_16bit_storage`

IP Status

No known IP claims.

Contributors

- Alexander Galazin, Arm

Description

The `VK_KHR_8bit_storage` extension allows use of 8-bit types in uniform and storage buffers, and

push constant blocks. This extension introduces several new optional features which map to SPIR-V capabilities and allow access to 8-bit data in `Block`-decorated objects in the `Uniform` and the `StorageBuffer` storage classes, and objects in the `PushConstant` storage class.

The `StorageBuffer8BitAccess` capability **must** be supported by all implementations of this extension. The other capabilities are optional.

Promotion to Vulkan 1.2

Vulkan APIs in this extension are included in core Vulkan 1.2, with the KHR suffix omitted. However, if Vulkan 1.2 is supported and this extension is not, the `StorageBuffer8BitAccess` capability is optional. External interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevice8BitStorageFeaturesKHR

New Enum Constants

- `VK_KHR_8BIT_STORAGE_EXTENSION_NAME`
- `VK_KHR_8BIT_STORAGE_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_8BIT_STORAGE_FEATURES_KHR`

New SPIR-V Capabilities

- `StorageBuffer8BitAccess`
- `UniformAndStorageBuffer8BitAccess`
- `StoragePushConstant8`

Version History

- Revision 1, 2018-02-05 (Alexander Galazin)
  - Initial draft

VK_KHR_acceleration_structure

Name String

`VK_KHR_acceleration_structure`

Extension Type

Device extension

Registered Extension Number

151

Revision

13

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1
and
`VK_EXT_descriptor_indexing`
and
VK_KHR_buffer_device_address
or
Vulkan Version 1.2
and
VK_KHR_deferred_host_operations

API Interactions

- Interacts with VK_VERSION_1_2
- Interacts with VK_VERSION_1_3
- Interacts with VK_EXT_debug_report
- Interacts with VK_KHR_format_feature_flags2

Contact

- Daniel Koch [GitHub]dgkoch

Other Extension Metadata

Last Modified Date

2021-09-30

Contributors

- Samuel Bourasseau, Adobe
- Matthäus Chajdas, AMD
- Greg Grebe, AMD
- Nicolai Hähnle, AMD
- Tobias Hector, AMD
- Dave Oldcorn, AMD
- Skyler Saleh, AMD
- Mathieu Robart, Arm
- Marius Bjorge, Arm

- Tom Olson, Arm
- Sebastian Tafuri, EA
- Henrik Rydgard, Embark
- Juan Cañada, Epic Games
- Patrick Kelly, Epic Games
- Yuriy O'Donnell, Epic Games
- Michael Doggett, Facebook/Oculus
- Ricardo Garcia, Igalia
- Andrew Garrard, Imagination
- Don Scorgie, Imagination
- Dae Kim, Imagination
- Joshua Barczak, Intel
- Slawek Grajewski, Intel
- Jeff Bolz, NVIDIA
- Pascal Gautron, NVIDIA
- Daniel Koch, NVIDIA
- Christoph Kubisch, NVIDIA
- Ashwin Lele, NVIDIA
- Robert Stepinski, NVIDIA
- Martin Stich, NVIDIA
- Nuno Subtil, NVIDIA
- Eric Werness, NVIDIA
- Jon Leech, Khronos
- Jeroen van Schijndel, OTOY
- Juul Joosten, OTOY
- Alex Bourd, Qualcomm
- Roman Larionov, Qualcomm
- David McAllister, Qualcomm
- Lewis Gordon, Samsung
- Ralph Potter, Samsung
- Jasper Bekkers, Traverse Research
- Jesse Barker, Unity
- Baldur Karlsson, Valve

Description

In order to be efficient, rendering techniques such as ray tracing need a quick way to identify which primitives may be intersected by a ray traversing the geometries. Acceleration structures are the most common way to represent the geometry spatially sorted, in order to quickly identify such potential intersections.

This extension adds new functionalities:

- Acceleration structure objects and build commands
- Structures to describe geometry inputs to acceleration structure builds
- Acceleration structure copy commands

New Object Types

- VkAccelerationStructureKHR

New Commands

- vkBuildAccelerationStructuresKHR
- vkCmdBuildAccelerationStructuresIndirectKHR
- vkCmdBuildAccelerationStructuresKHR
- vkCmdCopyAccelerationStructureKHR
- vkCmdCopyAccelerationStructureToMemoryKHR
- vkCmdCopyMemoryToAccelerationStructureKHR
- vkCmdWriteAccelerationStructuresPropertiesKHR
- vkCopyAccelerationStructureKHR
- vkCopyAccelerationStructureToMemoryKHR
- vkCopyMemoryToAccelerationStructureKHR
- vkCreateAccelerationStructureKHR
- vkDestroyAccelerationStructureKHR
- vkGetAccelerationStructureBuildSizesKHR
- vkGetAccelerationStructureDeviceAddressKHR
- vkGetDeviceAccelerationStructureCompatibilityKHR
- vkWriteAccelerationStructuresPropertiesKHR

New Structures

- VkAabbPositionsKHR
- VkAccelerationStructureBuildGeometryInfoKHR
- VkAccelerationStructureBuildRangeInfoKHR
- VkAccelerationStructureBuildSizesInfoKHR

- VkAccelerationStructureCreateInfoKHR
- VkAccelerationStructureDeviceAddressInfoKHR
- VkAccelerationStructureGeometryAabbsDataKHR
- VkAccelerationStructureGeometryInstancesDataKHR
- VkAccelerationStructureGeometryKHR
- VkAccelerationStructureGeometryTrianglesDataKHR
- VkAccelerationStructureInstanceKHR
- VkAccelerationStructureVersionInfoKHR
- VkCopyAccelerationStructureInfoKHR
- VkCopyAccelerationStructureToMemoryInfoKHR
- VkCopyMemoryToAccelerationStructureInfoKHR
- VkTransformMatrixKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceAccelerationStructureFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceAccelerationStructurePropertiesKHR
- Extending VkWriteDescriptorSet:
  - VkWriteDescriptorSetAccelerationStructureKHR

New Unions

- VkAccelerationStructureGeometryDataKHR
- VkDeviceOrHostAddressConstKHR
- VkDeviceOrHostAddressKHR

New Enums

- VkAccelerationStructureBuildTypeKHR
- VkAccelerationStructureCompatibilityKHR
- VkAccelerationStructureCreateFlagBitsKHR
- VkAccelerationStructureTypeKHR
- VkBuildAccelerationStructureFlagBitsKHR
- VkBuildAccelerationStructureModeKHR
- VkCopyAccelerationStructureModeKHR
- VkGeometryFlagBitsKHR
- VkGeometryInstanceFlagBitsKHR
- VkGeometryTypeKHR

New Bitmasks

- VkAccelerationStructureCreateFlagsKHR
- VkBuildAccelerationStructureFlagsKHR
- VkGeometryFlagsKHR
- VkGeometryInstanceFlagsKHR

New Enum Constants

- VK_KHR_ACCELERATION_STRUCTURE_EXTENSION_NAME
- VK_KHR_ACCELERATION_STRUCTURE_SPEC_VERSION
- Extending VkAccessFlagBits:
  - VK_ACCESS_ACCELERATION_STRUCTURE_READ_BIT_KHR
  - VK_ACCESS_ACCELERATION_STRUCTURE_WRITE_BIT_KHR
- Extending VkBufferUsageFlagBits:
  - VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR
  - VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR
- Extending VkCopyAccelerationStructureModeKHR:
  - VK_COPY_ACCELERATION_STRUCTURE_MODE_DESERIALIZE_KHR
  - VK_COPY_ACCELERATION_STRUCTURE_MODE_SERIALIZE_KHR
- Extending VkDescriptorType:
  - VK_DESCRIPTOR_TYPE_ACCELERATION_STRUCTURE_KHR
- Extending VkFormatFeatureFlagBits:
  - VK_FORMAT_FEATURE_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR
- Extending VkIndexType:
  - VK_INDEX_TYPE_NONE_KHR
- Extending VkObjectType:
  - VK_OBJECT_TYPE_ACCELERATION_STRUCTURE_KHR
- Extending VkPipelineStageFlagBits:
  - VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR
- Extending VkQueryType:
  - VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR
  - VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_SIZE_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_BUILD_GEOMETRY_INFO_KHR
  - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_BUILD_SIZES_INFO_KHR
  - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_CREATE_INFO_KHR

    - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_DEVICE_ADDRESS_INFO_KHR
    - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_AABBS_DATA_KHR
    - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_INSTANCES_DATA_KHR
    - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_KHR
    - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_GEOMETRY_TRIANGLES_DATA_KHR
    - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_VERSION_INFO_KHR
    - VK_STRUCTURE_TYPE_COPY_ACCELERATION_STRUCTURE_INFO_KHR
    - VK_STRUCTURE_TYPE_COPY_ACCELERATION_STRUCTURE_TO_MEMORY_INFO_KHR
    - VK_STRUCTURE_TYPE_COPY_MEMORY_TO_ACCELERATION_STRUCTURE_INFO_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ACCELERATION_STRUCTURE_FEATURES_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ACCELERATION_STRUCTURE_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET_ACCELERATION_STRUCTURE_KHR

If VK_EXT_debug_report is supported:

- Extending VkDebugReportObjectTypeEXT:
    - VK_DEBUG_REPORT_OBJECT_TYPE_ACCELERATION_STRUCTURE_KHR_EXT

If VK_KHR_format_feature_flags2 or Vulkan Version 1.3 is supported:

- Extending VkFormatFeatureFlagBits2:
    - VK_FORMAT_FEATURE_2_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR

Issues

(1) How does this extension differ from VK_NV_ray_tracing?

DISCUSSION:

The following is a summary of the main functional differences between VK_KHR_acceleration_structure and VK_NV_ray_tracing:

- added acceleration structure serialization / deserialization (VK_COPY_ACCELERATION_STRUCTURE_MODE_SERIALIZE_KHR, VK_COPY_ACCELERATION_STRUCTURE_MODE_DESERIALIZE_KHR, vkCmdCopyAccelerationStructureToMemoryKHR, vkCmdCopyMemoryToAccelerationStructureKHR)
- document inactive primitives and instances
- added VkPhysicalDeviceAccelerationStructureFeaturesKHR structure
- added indirect and batched acceleration structure builds (vkCmdBuildAccelerationStructuresIndirectKHR)
- added host acceleration structure commands
- reworked geometry structures so they could be better shared between device, host, and indirect

builds

- explicitly made VkAccelerationStructureKHR use device addresses
- added acceleration structure compatibility check function (vkGetDeviceAccelerationStructureCompatibilityKHR)
- add parameter for requesting memory requirements for host and/or device build
- added format feature for acceleration structure build vertex formats (VK_FORMAT_FEATURE_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR)

(3) What are the changes between the public provisional (VK_KHR_ray_tracing v8) release and the internal provisional (VK_KHR_ray_tracing v9) release?

- added geometryFlags to VkAccelerationStructureCreateGeometryTypeInfoKHR (later reworked to obsolete this)
- added minAccelerationStructureScratchOffsetAlignment property to VkPhysicalDeviceRayTracingPropertiesKHR
- fix naming and return enum from vkGetDeviceAccelerationStructureCompatibilityKHR
  - renamed VkAccelerationStructureVersionKHR to VkAccelerationStructureVersionInfoKHR
  - renamed VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_VERSION_KHR to VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_VERSION_INFO_KHR
  - removed VK_ERROR_INCOMPATIBLE_VERSION_KHR
  - added VkAccelerationStructureCompatibilityKHR enum
  - remove return value from vkGetDeviceAccelerationStructureCompatibilityKHR and added return enum parameter
- Require Vulkan 1.1
- added creation time capture and replay flags
  - added VkAccelerationStructureCreateFlagBitsKHR and VkAccelerationStructureCreateFlagsKHR
  - renamed the flags member of VkAccelerationStructureCreateInfoKHR to buildFlags (later removed) and added the createFlags member
- change vkCmdBuildAccelerationStructuresIndirectKHR to use buffer device address for indirect parameter
- make VK_KHR_deferred_host_operations an interaction instead of a required extension (later went back on this)
- renamed VkAccelerationStructureBuildOffsetInfoKHR to VkAccelerationStructureBuildRangeInfoKHR
  - renamed the ppOffsetInfos parameter of vkCmdBuildAccelerationStructuresKHR to ppBuildRangeInfos
- Re-unify geometry description between build and create
  - remove VkAccelerationStructureCreateGeometryTypeInfoKHR and VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_CREATE_GEOMETRY_TYPE_INFO_KHR

    - added VkAccelerationStructureCreateSizeInfoKHR structure (later removed)
    - change type of the pGeometryInfos member of VkAccelerationStructureCreateInfoKHR from VkAccelerationStructureCreateGeometryTypeInfoKHR to VkAccelerationStructureGeometryKHR (later removed)
    - added pCreateSizeInfos member to VkAccelerationStructureCreateInfoKHR (later removed)
- Fix ppGeometries ambiguity, add pGeometries
    - remove geometryArrayOfPointers member of VkAccelerationStructureBuildGeometryInfoKHR
    - disambiguate two meanings of ppGeometries by explicitly adding pGeometries to the VkAccelerationStructureBuildGeometryInfoKHR structure and require one of them be NULL
- added nullDescriptor support for acceleration structures
- changed the update member of VkAccelerationStructureBuildGeometryInfoKHR from a bool to the mode VkBuildAccelerationStructureModeKHR enum which allows future extensibility in update types
- Clarify deferred host ops for pipeline creation
    - VkDeferredOperationKHR is now a top-level parameter for vkBuildAccelerationStructuresKHR, vkCreateRayTracingPipelinesKHR, vkCopyAccelerationStructureToMemoryKHR, vkCopyAccelerationStructureKHR, and vkCopyMemoryToAccelerationStructureKHR
    - removed VkDeferredOperationInfoKHR structure
    - change deferred host creation/return parameter behavior such that the implementation can modify such parameters until the deferred host operation completes
    - VK_KHR_deferred_host_operations is required again
- Change acceleration structure build to always be sized
    - de-alias VkAccelerationStructureMemoryRequirementsTypeNV and VkAccelerationStructureMemoryRequirementsTypeKHR, and remove VkAccelerationStructureMemoryRequirementsTypeKHR
    - add vkGetAccelerationStructureBuildSizesKHR command and VkAccelerationStructureBuildSizesInfoKHR structure and VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_BUILD_SIZES_INFO_KHR enum to query sizes for acceleration structures and scratch storage
    - move size queries for scratch space to vkGetAccelerationStructureBuildSizesKHR
    - remove compactedSize, buildFlags, maxGeometryCount, pGeometryInfos, pCreateSizeInfos members of VkAccelerationStructureCreateInfoKHR and add the size member
    - add maxVertex member to VkAccelerationStructureGeometryTrianglesDataKHR structure
    - remove VkAccelerationStructureCreateSizeInfoKHR structure

(4) What are the changes between the internal provisional (VK_KHR_ray_tracing v9) release and the final (VK_KHR_acceleration_structure v11) release?

- refactor VK_KHR_ray_tracing into 3 extensions, enabling implementation flexibility and

decoupling ray query support from ray pipelines:

  - VK_KHR_acceleration_structure (for acceleration structure operations)
  - VK_KHR_ray_tracing_pipeline (for ray tracing pipeline and shader stages)
  - VK_KHR_ray_query (for ray queries in existing shader stages)

- clarify buffer usage flags for ray tracing
  - VK_BUFFER_USAGE_RAY_TRACING_BIT_NV is left alone in VK_NV_ray_tracing (required on scratch and instanceData)
  - VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR is added as an alias of VK_BUFFER_USAGE_RAY_TRACING_BIT_NV in VK_KHR_ray_tracing_pipeline and is required on shader binding table buffers
  - VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR is added in VK_KHR_acceleration_structure for all vertex, index, transform, aabb, and instance buffer data referenced by device build commands
  - VK_BUFFER_USAGE_STORAGE_BUFFER_BIT is used for scratchData
- add max primitive counts (ppMaxPrimitiveCounts) to vkCmdBuildAccelerationStructuresIndirectKHR
- Allocate acceleration structures from VkBuffers and add a mode to constrain the device address
  - de-alias VkBindAccelerationStructureMemoryInfoNV and vkBindAccelerationStructureMemoryNV, and remove VkBindAccelerationStructureMemoryInfoKHR, VkAccelerationStructureMemoryRequirementsInfoKHR, and vkGetAccelerationStructureMemoryRequirementsKHR
  - acceleration structures now take a VkBuffer and offset at creation time for memory placement
  - add a new VK_BUFFER_USAGE_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR buffer usage for such buffers
  - add a new VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR acceleration structure type for layering
- move VK_GEOMETRY_TYPE_INSTANCES_KHR to main enum instead of being added via extension
- make build commands more consistent - all now build multiple acceleration structures and are named plurally (vkCmdBuildAccelerationStructuresIndirectKHR, vkCmdBuildAccelerationStructuresKHR, vkBuildAccelerationStructuresKHR)
- add interactions with VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT for acceleration structures, including a new feature (descriptorBindingAccelerationStructureUpdateAfterBind) and 3 new properties (maxPerStageDescriptorAccelerationStructures, maxPerStageDescriptorUpdateAfterBindAccelerationStructures, maxDescriptorSetUpdateAfterBindAccelerationStructures)
- extension is no longer provisional
- define synchronization requirements for builds, traces, and copies
- define synchronization requirements for AS build inputs and indirect build buffer

(5) What is `VK_ACCELERATION_STRUCTURE_TYPE_GENERIC_KHR` for?

RESOLVED: It is primarily intended for API layering. In DXR, the acceleration structure is basically just a buffer in a special layout, and you do not know at creation time whether it will be used as a top or bottom level acceleration structure. We thus added a generic acceleration structure type whose type is unknown at creation time, but is specified at build time instead. Applications which are written directly for Vulkan should not use it.

Version History

- Revision 1, 2019-12-05 (Members of the Vulkan Ray Tracing TSG)
  - Internal revisions (forked from VK_NV_ray_tracing)
- Revision 2, 2019-12-20 (Daniel Koch, Eric Werness)
  - Add const version of DeviceOrHostAddress (!3515)
  - Add VU to clarify that only handles in the current pipeline are valid (!3518)
  - Restore some missing VUs and add in-place update language (#1902, !3522)
  - rename VkAccelerationStructureInstanceKHR member from accelerationStructure to accelerationStructureReference to better match its type (!3523)
  - Allow VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS for pipeline creation if shader group handles cannot be reused (!3523)
  - update documentation for the VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS error code and add missing documentation for new return codes from VK_KHR_deferred_host_operations (!3523)
  - list new query types for VK_KHR_ray_tracing (!3523)
  - Fix VU statements for VkAccelerationStructureGeometryKHR referring to correct union members and update to use more current wording (!3523)
- Revision 3, 2020-01-10 (Daniel Koch, Jon Leech, Christoph Kubisch)
  - Fix 'instance of' and 'that/which contains/defines' markup issues (!3528)
  - factor out VK_KHR_pipeline_library as stand-alone extension (!3540)
  - Resolve Vulkan-hpp issues (!3543)
    - add missing require for VkGeometryInstanceFlagsKHR
    - de-alias VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_CREATE_INFO_NV since the KHR structure is no longer equivalent
    - add len to pDataSize attribute for vkWriteAccelerationStructuresPropertiesKHR
- Revision 4, 2020-01-23 (Daniel Koch, Eric Werness)
  - Improve vkWriteAccelerationStructuresPropertiesKHR, add return value and VUs (#1947)
  - Clarify language to allow multiple raygen shaders (#1959)
  - Various editorial feedback (!3556)
  - Add language to help deal with looped self-intersecting fans (#1901)

- Change vkCmdTraceRays{,Indirect}KHR args to pointers (!3559)
- Add scratch address validation language (#1941, !3551)
- Fix definition and add hierarchy information for shader call scope (#1977, !3571)

- Revision 5, 2020-02-04 (Eric Werness, Jeff Bolz, Daniel Koch)
  - remove vestigial accelerationStructureUUID (!3582)
  - update definition of repack instructions and improve memory model interactions (#1910, #1913, !3584)
  - Fix wrong sType for VkPhysicalDeviceRayTracingFeaturesKHR (#1988)
  - Use provisional SPIR-V capabilities (#1987)
  - require rayTraversalPrimitiveCulling if rayQuery is supported (#1927)
  - Miss shaders do not have object parameters (!3592)
  - Fix missing required types in XML (!3592)
  - clarify matching conditions for update (!3592)
  - add goal that host and device builds be similar (!3592)
  - clarify that `maxPrimitiveCount` limit should apply to triangles and AABBs (!3592)
  - Require alignment for instance arrayOfPointers (!3592)
  - Zero is a valid value for instance flags (!3592)
  - Add some alignment VUs that got lost in refactoring (!3592)
  - Recommend TMin epsilon rather than culling (!3592)
  - Get angle from dot product not cross product (!3592)
  - Clarify that AH can access the payload and attributes (!3592)
  - Match DXR behavior for inactive primitive definition (!3592)
  - Use a more generic term than degenerate for inactive to avoid confusion (!3592)
- Revision 6, 2020-02-20 (Daniel Koch)
  - fix some dangling NV references (#1996)
  - rename VkCmdTraceRaysIndirectCommandKHR to VkTraceRaysIndirectCommandKHR (!3607)
  - update contributor list (!3611)
  - use uint64_t instead of VkAccelerationStructureReferenceKHR in VkAccelerationStructureInstanceKHR (#2004)
- Revision 7, 2020-02-28 (Tobias Hector)
  - remove HitTKHR SPIR-V builtin (spirv/spirv-extensions#7)
- Revision 8, 2020-03-06 (Tobias Hector, Dae Kim, Daniel Koch, Jeff Bolz, Eric Werness)
  - explicitly state that Tmax is updated when new closest intersection is accepted (#2020,!3536)
  - Made references to min and max t values consistent (!3644)

    - finish enumerating differences relative to VK_NV_ray_tracing in issues (1) and (2) (#1974,!3642)
    - fix formatting in some math equations (!3642)
    - Restrict the Hit Kind operand of `OpReportIntersectionKHR` to 7-bits (spirv/spirv-extensions#8,!3646)
    - Say ray tracing '**should**' be watertight (#2008,!3631)
    - Clarify memory requirements for ray tracing buffers (#2005,!3649)
    - Add callable size limits (#1997,!3652)
- Revision 9, 2020-04-15 (Eric Werness, Daniel Koch, Tobias Hector, Joshua Barczak)
    - Add geometry flags to acceleration structure creation (!3672)
    - add build scratch memory alignment (minAccelerationStructureScratchOffsetAlignment) (#2065,!3725)
    - fix naming and return enum from vkGetDeviceAccelerationStructureCompatibilityKHR (#2051,!3726)
    - require SPIR-V 1.4 (#2096,!3777)
    - added creation time capture/replay flags (#2104,!3774)
    - require Vulkan 1.1 (#2133,!3806)
    - use device addresses instead of VkBuffers for ray tracing commands (#2074,!3815)
    - add interactions with Vulkan 1.2 and VK_KHR_vulkan_memory_model (#2133,!3830)
    - make VK_KHR_pipeline_library an interaction instead of required (#2045,#2108,!3830)
    - make VK_KHR_deferred_host_operations an interaction instead of required (#2045,!3830)
    - removed maxCallableSize and added explicit stack size management for ray pipelines (#1997,!3817,!3772,!3844)
    - improved documentation for VkAccelerationStructureVersionInfoKHR (#2135,3835)
    - rename VkAccelerationStructureBuildOffsetInfoKHR to VkAccelerationStructureBuildRangeInfoKHR (#2058,!3754)
    - Re-unify geometry description between build and create (!3754)
    - Fix ppGeometries ambiguity, add pGeometries (#2032,!3811)
    - add interactions with VK_EXT_robustness2 and allow nullDescriptor support for acceleration structures (#1920,!3848)
    - added future extensibility for AS updates (#2114,!3849)
    - Fix VU for dispatchrays and add a limit on the size of the full grid (#2160,!3851)
    - Add shaderGroupHandleAlignment property (#2180,!3875)
    - Clarify deferred host ops for pipeline creation (#2067,!3813)
    - Change acceleration structure build to always be sized (#2131,#2197,#2198,!3854,!3883,!3880)
- Revision 10, 2020-07-03 (Mathieu Robart, Daniel Koch, Eric Werness, Tobias Hector)

  - Decomposition of the specification, from VK_KHR_ray_tracing to VK_KHR_acceleration_structure (#1918,!3912)
  - clarify buffer usage flags for ray tracing (#2181,!3939)
  - add max primitive counts to build indirect command (#2233,!3944)
  - Allocate acceleration structures from VkBuffers and add a mode to constrain the device address (#2131,!3936)
  - Move VK_GEOMETRY_TYPE_INSTANCES_KHR to main enum (#2243,!3952)
  - make build commands more consistent (#2247,!3958)
  - add interactions with UPDATE_AFTER_BIND (#2128,!3986)
  - correct and expand build command VUs (!4020)
  - fix copy command VUs (!4018)
  - added various alignment requirements (#2229,!3943)
  - fix valid usage for arrays of geometryCount items (#2198,!4010)
  - define what is allowed to change on RTAS updates and relevant VUs (#2177,!3961)
- Revision 11, 2020-11-12 (Eric Werness, Josh Barczak, Daniel Koch, Tobias Hector)
  - de-alias NV and KHR acceleration structure types and associated commands (#2271,!4035)
  - specify alignment for host copy commands (#2273,!4037)
  - document `VK_FORMAT_FEATURE_ACCELERATION_STRUCTURE_VERTEX_BUFFER_BIT_KHR`
  - specify that acceleration structures are non-linear (#2289,!4068)
  - add several missing VUs for strides, vertexFormat, and indexType (#2315,!4069)
  - restore VUs for VkAccelerationStructureBuildGeometryInfoKHR (#2337,!4098)
  - ban multi-instance memory for host operations (#2324,!4102)
  - allow dstAccelerationStructure to be null for vkGetAccelerationStructureBuildSizesKHR (#2330,!4111)
  - more build VU cleanup (#2138,#4130)
  - specify host endianness for AS serialization (#2261,!4136)
  - add invertible transform matrix VU (#1710,!4140)
  - require geometryCount to be 1 for TLAS builds (!4145)
  - improved validity conditions for build addresses (#4142)
  - add single statement SPIR-V VUs, build limit VUs (!4158)
  - document limits for vertex and aabb strides (#2390,!4184)
  - specify that `VK_PIPELINE_STAGE_ACCELERATION_STRUCTURE_BUILD_BIT_KHR` applies to AS copies (#2382,#4173)
  - define sync for AS build inputs and indirect buffer (#2407,!4208)
- Revision 12, 2021-08-06 (Samuel Bourasseau)
  - rename VK_GEOMETRY_INSTANCE_TRIANGLE_FRONT_COUNTERCLOCKWISE_BIT_KHR to

VK_GEOMETRY_INSTANCE_TRIANGLE_FLIP_FACING_BIT_KHR (keep previous as alias).

    - Clarify description and add note.

- Revision 13, 2021-09-30 (Jon Leech)
    - Add interaction with `VK_KHR_format_feature_flags2` to `vk.xml`

VK_KHR_android_surface

Name String

`VK_KHR_android_surface`

Extension Type

Instance extension

Registered Extension Number

9

Revision

6

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- Jesse Hall [GitHub]critsec

Other Extension Metadata

Last Modified Date

2016-01-14

IP Status

No known IP claims.

Contributors

- Patrick Doane, Blizzard
- Faith Ekstrand, Intel
- Ian Elliott, LunarG
- Courtney Goeltzenleuchter, LunarG
- Jesse Hall, Google
- James Jones, NVIDIA
- Antoine Labour, Google

- Jon Leech, Khronos
- David Mao, AMD
- Norbert Nopper, Freescale
- Alon Or-bach, Samsung
- Daniel Rakos, AMD
- Graham Sellers, AMD
- Ray Smith, ARM
- Jeff Vigil, Qualcomm
- Chia-I Wu, LunarG

Description

The `VK_KHR_android_surface` extension is an instance extension. It provides a mechanism to create a VkSurfaceKHR object (defined by the `VK_KHR_surface` extension) that refers to an ANativeWindow, Android's native surface type. The ANativeWindow represents the producer endpoint of any buffer queue, regardless of consumer endpoint. Common consumer endpoints for `ANativeWindows` are the system window compositor, video encoders, and application-specific compositors importing the images through a `SurfaceTexture`.

New Base Types

- ANativeWindow

New Commands

- vkCreateAndroidSurfaceKHR

New Structures

- VkAndroidSurfaceCreateInfoKHR

New Bitmasks

- VkAndroidSurfaceCreateFlagsKHR

New Enum Constants

- `VK_KHR_ANDROID_SURFACE_EXTENSION_NAME`
- `VK_KHR_ANDROID_SURFACE_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_ANDROID_SURFACE_CREATE_INFO_KHR`

Issues

1) Does Android need a way to query for compatibility between a particular physical device (and queue family?) and a specific Android display?

RESOLVED: No. Currently on Android, any physical device is expected to be able to present to the system compositor, and all queue families must support the necessary image layout transitions and synchronization operations.

Version History

- Revision 1, 2015-09-23 (Jesse Hall)
  - Initial draft.
- Revision 2, 2015-10-26 (Ian Elliott)
  - Renamed from VK_EXT_KHR_android_surface to VK_KHR_android_surface.
- Revision 3, 2015-11-03 (Daniel Rakos)
  - Added allocation callbacks to surface creation function.
- Revision 4, 2015-11-10 (Jesse Hall)
  - Removed VK_ERROR_INVALID_ANDROID_WINDOW_KHR.
- Revision 5, 2015-11-28 (Daniel Rakos)
  - Updated the surface create function to take a pCreateInfo structure.
- Revision 6, 2016-01-14 (James Jones)
  - Moved VK_ERROR_NATIVE_WINDOW_IN_USE_KHR from the VK_KHR_android_surface to the VK_KHR_surface extension.

VK_KHR_bind_memory2

Name String

`VK_KHR_bind_memory2`

Extension Type

Device extension

Registered Extension Number

158

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Tobias Hector [GitHub]tobski

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA
- Tobias Hector, Imagination Technologies

Description

This extension provides versions of vkBindBufferMemory and vkBindImageMemory that allow multiple bindings to be performed at once, and are extensible.

This extension also introduces `VK_IMAGE_CREATE_ALIAS_BIT_KHR`, which allows "identical" images that alias the same memory to interpret the contents consistently, even across image layout changes.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkBindBufferMemory2KHR
- vkBindImageMemory2KHR

New Structures

- VkBindBufferMemoryInfoKHR
- VkBindImageMemoryInfoKHR

New Enum Constants

- `VK_KHR_BIND_MEMORY_2_EXTENSION_NAME`
- `VK_KHR_BIND_MEMORY_2_SPEC_VERSION`
- Extending VkImageCreateFlagBits:
  - `VK_IMAGE_CREATE_ALIAS_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_INFO_KHR`

- VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_INFO_KHR

Version History

- Revision 1, 2017-05-19 (Tobias Hector)
  - Pulled bind memory functions into their own extension

VK_KHR_buffer_device_address

Name String

VK_KHR_buffer_device_address

Extension Type

Device extension

Registered Extension Number

258

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
and
VK_KHR_device_group
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_physical_storage_buffer

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2019-06-24

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_buffer_reference` and `GL_EXT_buffer_reference2` and `GL_EXT_buffer_reference_uvec2`

Contributors

- Jeff Bolz, NVIDIA
- Neil Henning, AMD
- Tobias Hector, AMD
- Faith Ekstrand, Intel
- Baldur Karlsson, Valve
- Jan-Harald Fredriksen, Arm

Description

This extension allows the application to query a 64-bit buffer device address value for a buffer, which can be used to access the buffer memory via the `PhysicalStorageBuffer` storage class in the `GL_EXT_buffer_reference` GLSL extension and `SPV_KHR_physical_storage_buffer` SPIR-V extension.

Another way to describe this extension is that it adds “pointers to buffer memory in shaders”. By calling vkGetBufferDeviceAddressKHR with a `VkBuffer`, it will return a VkDeviceAddress value which represents the address of the start of the buffer.

vkGetBufferOpaqueCaptureAddressKHR and vkGetDeviceMemoryOpaqueCaptureAddressKHR allow opaque addresses for buffers and memory objects to be queried for the current process. A trace capture and replay tool can then supply these addresses to be used at replay time to match the addresses used when the trace was captured. To enable tools to insert these queries, new memory allocation flags must be specified for memory objects that will be bound to buffers accessed via the `PhysicalStorageBuffer` storage class. **Note that this mechanism is intended only to support capture/replay tools, and is not recommended for use in other applications.**

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. However, if Vulkan 1.2 is supported and this extension is not, the `bufferDeviceAddress` feature is optional. The original type, enum, and command names are still available as aliases of the core functionality.

Promotion to Vulkan 1.3

Support for the `bufferDeviceAddress` feature is mandatory in Vulkan 1.3, regardless of whether this extension is supported.

New Commands

- vkGetBufferDeviceAddressKHR
- vkGetBufferOpaqueCaptureAddressKHR
- vkGetDeviceMemoryOpaqueCaptureAddressKHR

New Structures

- VkBufferDeviceAddressInfoKHR
- VkDeviceMemoryOpaqueCaptureAddressInfoKHR
- Extending VkBufferCreateInfo:
  - VkBufferOpaqueCaptureAddressCreateInfoKHR
- Extending VkMemoryAllocateInfo:
  - VkMemoryOpaqueCaptureAddressAllocateInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceBufferDeviceAddressFeaturesKHR

New Enum Constants

- `VK_KHR_BUFFER_DEVICE_ADDRESS_EXTENSION_NAME`
- `VK_KHR_BUFFER_DEVICE_ADDRESS_SPEC_VERSION`
- Extending VkBufferCreateFlagBits:
  - `VK_BUFFER_CREATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_KHR`
- Extending VkBufferUsageFlagBits:
  - `VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT_KHR`
- Extending VkMemoryAllocateFlagBits:
  - `VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT_KHR`
  - `VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_CAPTURE_REPLAY_BIT_KHR`
- Extending VkResult:
  - `VK_ERROR_INVALID_OPAQUE_CAPTURE_ADDRESS_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_BUFFER_DEVICE_ADDRESS_INFO_KHR`
  - `VK_STRUCTURE_TYPE_BUFFER_OPAQUE_CAPTURE_ADDRESS_CREATE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_DEVICE_MEMORY_OPAQUE_CAPTURE_ADDRESS_INFO_KHR`
  - `VK_STRUCTURE_TYPE_MEMORY_OPAQUE_CAPTURE_ADDRESS_ALLOCATE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BUFFER_DEVICE_ADDRESS_FEATURES_KHR`

New SPIR-V Capabilities

- `PhysicalStorageBufferAddresses`

Version History

- Revision 1, 2019-06-24 (Jan-Harald Fredriksen)
  - Internal revisions based on VK_EXT_buffer_device_address

VK_KHR_calibrated_timestamps

Name String

VK_KHR_calibrated_timestamps

Extension Type

Device extension

Registered Extension Number

544

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Daniel Rakos [GitHub]aqnuep

Extension Proposal

VK_EXT_calibrated_timestamps

Other Extension Metadata

Last Modified Date

2023-07-12

IP Status

No known IP claims.

Contributors

- Matthaeus G. Chajdas, AMD
- Alan Harrison, AMD
- Derrick Owens, AMD
- Daniel Rakos, RasterGrid
- Faith Ekstrand, Intel
- Keith Packard, Valve

Description

This extension provides an interface to query calibrated timestamps obtained quasi simultaneously

from two time domains.

New Commands

- vkGetCalibratedTimestampsKHR
- vkGetPhysicalDeviceCalibrateableTimeDomainsKHR

New Structures

- VkCalibratedTimestampInfoKHR

New Enums

- VkTimeDomainKHR

New Enum Constants

- `VK_KHR_CALIBRATED_TIMESTAMPS_EXTENSION_NAME`
- `VK_KHR_CALIBRATED_TIMESTAMPS_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_CALIBRATED_TIMESTAMP_INFO_KHR`

Version History

- Revision 1, 2023-07-12 (Daniel Rakos)
  - Initial draft.

VK_KHR_compute_shader_derivatives

Name String

`VK_KHR_compute_shader_derivatives`

Extension Type

Device extension

Registered Extension Number

512

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2

SPIR-V Dependencies

- SPV_KHR_compute_shader_derivatives

Contact

- Jean-Noe Morissette [GitHub]MagicPoncho

Extension Proposal

VK_KHR_compute_shader_derivatives

Other Extension Metadata

Last Modified Date

2024-06-26

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension requires `SPV_KHR_compute_shader_derivatives`
- This extension provides API support for `GL_KHR_compute_shader_derivatives`

Contributors

- Jean-Noe Morissette, Epic Games
- Daniel Koch, NVIDIA
- Pat Brown, NVIDIA
- Stu Smith, AMD
- Jan-Harald Fredriksen, Arm
- Tobias Hector, AMD
- Ralph Potter, Samsung
- Pan Gao, Huawei
- Samuel (Sheng-Wen) Huang, MediaTek
- Graeme Leese, Broadcom
- Hans-Kristian Arntzen, Valve
- Matthew Netsh, Qualcomm

Description

This extension adds Vulkan support for the `SPV_KHR_compute_shader_derivatives` SPIR-V extension.

The SPIR-V extension provides two new execution modes, both of which allow execution models with defined workgroups to use built-ins that evaluate derivatives explicitly or implicitly. Derivatives will be computed via differencing over a 2x2 group of shader invocations. The `DerivativeGroupQuadsKHR` execution mode assembles shader invocations into 2x2 groups, where each group has x and y coordinates of the local invocation ID of the form (2m+{0,1}, 2n+{0,1}). The `DerivativeGroupLinearKHR` execution mode assembles shader invocations into 2x2 groups, where

each group has local invocation index values of the form 4m+{0,1,2,3}.

The new execution modes are supported in compute shaders and optionally (see meshAndTaskShaderDerivatives) in mesh and task shaders.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceComputeShaderDerivativesFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceComputeShaderDerivativesPropertiesKHR

New Enum Constants

- VK_KHR_COMPUTE_SHADER_DERIVATIVES_EXTENSION_NAME
- VK_KHR_COMPUTE_SHADER_DERIVATIVES_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COMPUTE_SHADER_DERIVATIVES_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COMPUTE_SHADER_DERIVATIVES_PROPERTIES_KHR

New SPIR-V Capability

- ComputeDerivativeGroupQuadsKHR
- ComputeDerivativeGroupLinearKHR

Examples

None.

Version History

- Revision 1, 2023-02-27 (Jean-Noe Morissette)
  - Initial draft
  - Add properties and clarify mesh and task support (Daniel Koch)

VK_KHR_cooperative_matrix

Name String

VK_KHR_cooperative_matrix

Extension Type

Device extension

Registered Extension Number

507

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_cooperative_matrix

Contact

- Kevin Petit [GitHub]kpet

Extension Proposal

VK_KHR_cooperative_matrix

Other Extension Metadata

Last Modified Date

2023-05-03

Interactions and External Dependencies

- This extension provides API support for `GLSL_KHR_cooperative_matrix`

Contributors

- Jeff Bolz, NVIDIA
- Markus Tavenrath, NVIDIA
- Daniel Koch, NVIDIA
- Kevin Petit, Arm Ltd.
- Boris Zanin, AMD

Description

This extension adds support for using cooperative matrix types in SPIR-V. Cooperative matrix types are medium-sized matrices that are primarily supported in compute shaders, where the storage for the matrix is spread across all invocations in some scope (usually a subgroup) and those invocations cooperate to efficiently perform matrix multiplies.

Cooperative matrix types are defined by the `SPV_KHR_cooperative_matrix` SPIR-V extension and can be used with the `GLSL_KHR_cooperative_matrix` GLSL extension.

This extension includes support for enumerating the matrix types and dimensions that are supported by the implementation.

New Commands

- vkGetPhysicalDeviceCooperativeMatrixPropertiesKHR

New Structures

- VkCooperativeMatrixPropertiesKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceCooperativeMatrixFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceCooperativeMatrixPropertiesKHR

New Enums

- VkComponentTypeKHR
- VkScopeKHR

New Enum Constants

- `VK_KHR_COOPERATIVE_MATRIX_EXTENSION_NAME`
- `VK_KHR_COOPERATIVE_MATRIX_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_COOPERATIVE_MATRIX_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COOPERATIVE_MATRIX_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COOPERATIVE_MATRIX_PROPERTIES_KHR`

New SPIR-V Capabilities

- CooperativeMatrixKHR

Issues

Version History

- Revision 2, 2023-05-03 (Kevin Petit)
  - First KHR revision
- Revision 1, 2019-02-05 (Jeff Bolz)
  - NVIDIA vendor extension

VK_KHR_copy_commands2

Name String

`VK_KHR_copy_commands2`

Extension Type

Device extension

Registered Extension Number

338

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Matthew Netsch [GitHub]mnetsch

Other Extension Metadata

Last Modified Date

2020-07-06

Contributors

- Jeff Leger, Qualcomm
- Tobias Hector, AMD
- Jan-Harald Fredriksen, ARM
- Tom Olson, ARM

Description

This extension provides extensible versions of the Vulkan buffer and image copy commands. The new commands are functionally identical to the core commands, except that their copy parameters are specified using extensible structures that can be used to pass extension-specific information.

The following extensible copy commands are introduced with this extension: vkCmdCopyBuffer2KHR, vkCmdCopyImage2KHR, vkCmdCopyBufferToImage2KHR, vkCmdCopyImageToBuffer2KHR, vkCmdBlitImage2KHR, and vkCmdResolveImage2KHR. Each command contains an `*Info2KHR` structure parameter that includes `sType`/`pNext` members. Lower level structures describing each region to be copied are also extended with `sType`/`pNext` members.

New Commands

- vkCmdBlitImage2KHR
- vkCmdCopyBuffer2KHR
- vkCmdCopyBufferToImage2KHR
- vkCmdCopyImage2KHR
- vkCmdCopyImageToBuffer2KHR
- vkCmdResolveImage2KHR

New Structures

- VkBlitImageInfo2KHR
- VkBufferCopy2KHR
- VkBufferImageCopy2KHR
- VkCopyBufferInfo2KHR
- VkCopyBufferToImageInfo2KHR
- VkCopyImageInfo2KHR
- VkCopyImageToBufferInfo2KHR
- VkImageBlit2KHR
- VkImageCopy2KHR
- VkImageResolve2KHR
- VkResolveImageInfo2KHR

New Enum Constants

- `VK_KHR_COPY_COMMANDS_2_EXTENSION_NAME`
- `VK_KHR_COPY_COMMANDS_2_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_BLIT_IMAGE_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_BUFFER_COPY_2_KHR`
  - `VK_STRUCTURE_TYPE_BUFFER_IMAGE_COPY_2_KHR`
  - `VK_STRUCTURE_TYPE_COPY_BUFFER_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_COPY_BUFFER_TO_IMAGE_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_COPY_IMAGE_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_COPY_IMAGE_TO_BUFFER_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_IMAGE_BLIT_2_KHR`
  - `VK_STRUCTURE_TYPE_IMAGE_COPY_2_KHR`
  - `VK_STRUCTURE_TYPE_IMAGE_RESOLVE_2_KHR`

- `VK_STRUCTURE_TYPE_RESOLVE_IMAGE_INFO_2_KHR`

Promotion to Vulkan 1.3

Vulkan APIs in this extension are included in core Vulkan 1.3, with the KHR suffix omitted. External interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

Version History

- Revision 1, 2020-07-06 (Jeff Leger)
  - Internal revisions

VK_KHR_create_renderpass2

Name String

`VK_KHR_create_renderpass2`

Extension Type

Device extension

Registered Extension Number

110

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_multiview
and
VK_KHR_maintenance2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Tobias Hector [GitHub]tobias

Other Extension Metadata

Last Modified Date

2018-02-07

Contributors

- Tobias Hector
- Jeff Bolz

Description

This extension provides a new command to create render passes in a way that can be easily extended by other extensions through the substructures of render pass creation. The Vulkan 1.0 render pass creation sub-structures do not include `sType`/`pNext` members. Additionally, the render pass begin/next/end commands have been augmented with new extensible structures for passing additional subpass information.

The VkRenderPassMultiviewCreateInfo and VkInputAttachmentAspectReference structures that extended the original VkRenderPassCreateInfo are not accepted into the new creation functions, and instead their parameters are folded into this extension as follows:

- Elements of VkRenderPassMultiviewCreateInfo::`pViewMasks` are now specified in VkSubpassDescription2KHR::`viewMask`.
- Elements of VkRenderPassMultiviewCreateInfo::`pViewOffsets` are now specified in VkSubpassDependency2KHR::`viewOffset`.
- VkRenderPassMultiviewCreateInfo::`correlationMaskCount` and VkRenderPassMultiviewCreateInfo::`pCorrelationMasks` are directly specified in VkRenderPassCreateInfo2KHR.
- VkInputAttachmentAspectReference::`aspectMask` is now specified in the relevant input attachment reference in VkAttachmentReference2KHR::`aspectMask`

The details of these mappings are explained fully in the new structures.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkCmdBeginRenderPass2KHR
- vkCmdEndRenderPass2KHR
- vkCmdNextSubpass2KHR
- vkCreateRenderPass2KHR

New Structures

- VkAttachmentDescription2KHR
- VkAttachmentReference2KHR
- VkRenderPassCreateInfo2KHR
- VkSubpassBeginInfoKHR

- VkSubpassDependency2KHR
- VkSubpassDescription2KHR
- VkSubpassEndInfoKHR

New Enum Constants

- VK_KHR_CREATE_RENDERPASS_2_EXTENSION_NAME
- VK_KHR_CREATE_RENDERPASS_2_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_2_KHR
  - VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_2_KHR
  - VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO_2_KHR
  - VK_STRUCTURE_TYPE_SUBPASS_BEGIN_INFO_KHR
  - VK_STRUCTURE_TYPE_SUBPASS_DEPENDENCY_2_KHR
  - VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_2_KHR
  - VK_STRUCTURE_TYPE_SUBPASS_END_INFO_KHR

Version History

- Revision 1, 2018-02-07 (Tobias Hector)
  - Internal revisions

VK_KHR_dedicated_allocation

Name String

VK_KHR_dedicated_allocation

Extension Type

Device extension

Registered Extension Number

128

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_memory_requirements2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA
- Faith Ekstrand, Intel

Description

This extension enables resources to be bound to a dedicated allocation, rather than suballocated. For any particular resource, applications **can** query whether a dedicated allocation is recommended, in which case using a dedicated allocation **may** improve the performance of access to that resource. Normal device memory allocations must support multiple resources per allocation, memory aliasing and sparse binding, which could interfere with some optimizations. Applications should query the implementation for when a dedicated allocation **may** be beneficial by adding a `VkMemoryDedicatedRequirementsKHR` structure to the `pNext` chain of the `VkMemoryRequirements2` structure passed as the `pMemoryRequirements` parameter of a call to `vkGetBufferMemoryRequirements2KHR` or `vkGetImageMemoryRequirements2KHR`. Certain external handle types and external images or buffers **may** also depend on dedicated allocations on implementations that associate image or buffer metadata with OS-level memory objects.

This extension adds a two small structures to memory requirements querying and memory allocation: a new structure that flags whether an image/buffer should have a dedicated allocation, and a structure indicating the image or buffer that an allocation will be bound to.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkMemoryAllocateInfo:
  - VkMemoryDedicatedAllocateInfoKHR
- Extending VkMemoryRequirements2:
  - VkMemoryDedicatedRequirementsKHR

New Enum Constants

- VK_KHR_DEDICATED_ALLOCATION_EXTENSION_NAME
- VK_KHR_DEDICATED_ALLOCATION_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_MEMORY_DEDICATED_ALLOCATE_INFO_KHR
  - VK_STRUCTURE_TYPE_MEMORY_DEDICATED_REQUIREMENTS_KHR

Examples

```
    // Create an image with a dedicated allocation based on the
    // implementation's preference

    VkImageCreateInfo imageCreateInfo =
    {
        // Image creation parameters
    };

    VkImage image;
    VkResult result = vkCreateImage(
        device,
        &imageCreateInfo,
        NULL,               // pAllocator
        &image);

    VkMemoryDedicatedRequirementsKHR dedicatedRequirements =
    {
        .sType = VK_STRUCTURE_TYPE_MEMORY_DEDICATED_REQUIREMENTS_KHR,
        .pNext = NULL,
    };

    VkMemoryRequirements2 memoryRequirements =
    {
        .sType = VK_STRUCTURE_TYPE_MEMORY_REQUIREMENTS_2,
        .pNext = &dedicatedRequirements,
    };

    const VkImageMemoryRequirementsInfo2 imageRequirementsInfo =
    {
        .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_REQUIREMENTS_INFO_2,
        .pNext = NULL,
        .image = image
    };

    vkGetImageMemoryRequirements2(
        device,
        &imageRequirementsInfo,
        &memoryRequirements);
```

```
    if (dedicatedRequirements.prefersDedicatedAllocation) {
        // Allocate memory with VkMemoryDedicatedAllocateInfoKHR::image
        // pointing to the image we are allocating the memory for

        VkMemoryDedicatedAllocateInfoKHR dedicatedInfo =
        {
            .sType = VK_STRUCTURE_TYPE_MEMORY_DEDICATED_ALLOCATE_INFO_KHR,
            .pNext = NULL,
            .image = image,
            .buffer = VK_NULL_HANDLE,
        };

        VkMemoryAllocateInfo memoryAllocateInfo =
        {
            .sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO,
            .pNext = &dedicatedInfo,
            .allocationSize = memoryRequirements.size,
            .memoryTypeIndex = FindMemoryTypeIndex(memoryRequirements.memoryTypeBits),
        };

        VkDeviceMemory memory;
        vkAllocateMemory(
            device,
            &memoryAllocateInfo,
            NULL,               // pAllocator
            &memory);

        // Bind the image to the memory

        vkBindImageMemory(
            device,
            image,
            memory,
            0);
    } else {
        // Take the normal memory sub-allocation path
    }
```

Version History

- Revision 1, 2017-02-27 (James Jones)
  - Copy content from VK_NV_dedicated_allocation
  - Add some references to external object interactions to the overview.
- Revision 2, 2017-03-27 (Faith Ekstrand)
  - Rework the extension to be query-based
- Revision 3, 2017-07-31 (Faith Ekstrand)
  - Clarify that memory objects allocated with VkMemoryDedicatedAllocateInfoKHR can only

have the specified resource bound and no others.

VK_KHR_deferred_host_operations

Name String

`VK_KHR_deferred_host_operations`

Extension Type

Device extension

Registered Extension Number

269

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Josh Barczak [GitHub]jbarczak

Other Extension Metadata

Last Modified Date

2020-11-12

IP Status

No known IP claims.

Contributors

- Joshua Barczak, Intel
- Jeff Bolz, NVIDIA
- Daniel Koch, NVIDIA
- Slawek Grajewski, Intel
- Tobias Hector, AMD
- Yuriy O'Donnell, Epic
- Eric Werness, NVIDIA
- Baldur Karlsson, Valve
- Jesse Barker, Unity
- Contributors to VK_KHR_acceleration_structure, VK_KHR_ray_tracing_pipeline

Description

The VK_KHR_deferred_host_operations extension defines the infrastructure and usage patterns for deferrable commands, but does not specify any commands as deferrable. This is left to additional dependent extensions. Commands **must** not be deferred unless the deferral is specifically allowed by another extension which depends on VK_KHR_deferred_host_operations.

New Object Types

- VkDeferredOperationKHR

New Commands

- vkCreateDeferredOperationKHR
- vkDeferredOperationJoinKHR
- vkDestroyDeferredOperationKHR
- vkGetDeferredOperationMaxConcurrencyKHR
- vkGetDeferredOperationResultKHR

New Enum Constants

- VK_KHR_DEFERRED_HOST_OPERATIONS_EXTENSION_NAME
- VK_KHR_DEFERRED_HOST_OPERATIONS_SPEC_VERSION
- Extending VkObjectType:
  - VK_OBJECT_TYPE_DEFERRED_OPERATION_KHR
- Extending VkResult:
  - VK_OPERATION_DEFERRED_KHR
  - VK_OPERATION_NOT_DEFERRED_KHR
  - VK_THREAD_DONE_KHR
  - VK_THREAD_IDLE_KHR

Code Examples

The following examples will illustrate the concept of deferrable operations using a hypothetical example. The command vkDoSomethingExpensive denotes a deferrable command.

The following example illustrates how a vulkan application might request deferral of an expensive operation:

```
// create a deferred operation
VkDeferredOperationKHR hOp;
VkResult result = vkCreateDeferredOperationKHR(device, pCallbacks, &hOp);
assert(result == VK_SUCCESS);

result = vkDoSomethingExpensive(device, hOp, ...);
```

```
assert( result == VK_OPERATION_DEFERRED_KHR );

// operation was deferred.  Execute it asynchronously
std::async::launch(
    [ hOp ] ( )
    {
        vkDeferredOperationJoinKHR(device, hOp);

        result = vkGetDeferredOperationResultKHR(device, hOp);

        // deferred operation is now complete.  'result' indicates success or failure

        vkDestroyDeferredOperationKHR(device, hOp, pCallbacks);
    }
);
```

The following example illustrates extracting concurrency from a single deferred operation:

```
// create a deferred operation
VkDeferredOperationKHR hOp;
VkResult result = vkCreateDeferredOperationKHR(device, pCallbacks, &hOp);
assert(result == VK_SUCCESS);

result = vkDoSomethingExpensive(device, hOp, ...);
assert( result == VK_OPERATION_DEFERRED_KHR );

// Query the maximum amount of concurrency and clamp to the desired maximum
uint32_t numLaunches = std::min(vkGetDeferredOperationMaxConcurrencyKHR(device, hOp),
maxThreads);

std::vector<std::future<void> > joins;

for (uint32_t i = 0; i < numLaunches; i++) {
  joins.emplace_back(std::async::launch(
    [ hOp ] ( )
    {
        vkDeferredOperationJoinKHR(device, hOp);
                // in a job system, a return of VK_THREAD_IDLE_KHR should queue
another
                // job, but it is not functionally required
    }
  ));
}

for (auto &f : joins) {
  f.get();
}

result = vkGetDeferredOperationResultKHR(device, hOp);
```

```
// deferred operation is now complete.  'result' indicates success or failure

vkDestroyDeferredOperationKHR(device, hOp, pCallbacks);
```

The following example shows a subroutine which guarantees completion of a deferred operation, in the presence of multiple worker threads, and returns the result of the operation.

```
VkResult FinishDeferredOperation(VkDeferredOperationKHR hOp)
{
    // Attempt to join the operation until the implementation indicates that we should
stop

    VkResult result = vkDeferredOperationJoinKHR(device, hOp);
    while( result == VK_THREAD_IDLE_KHR )
    {
        std::this_thread::yield();
        result = vkDeferredOperationJoinKHR(device, hOp);
    }

    switch( result )
    {
    case VK_SUCCESS:
        {
            // deferred operation has finished.  Query its result.
            result = vkGetDeferredOperationResultKHR(device, hOp);
        }
        break;

    case VK_THREAD_DONE_KHR:
        {
            // deferred operation is being wrapped up by another thread
            //  wait for that thread to finish
            do
            {
                std::this_thread::yield();
                result = vkGetDeferredOperationResultKHR(device, hOp);
            } while( result == VK_NOT_READY );
        }
        break;

    default:
        assert(false); // other conditions are illegal.
        break;
    }

    return result;
}
```

Issues

1. Should this extension have a VkPhysicalDevice*FeaturesKHR structure?

RESOLVED: No. This extension does not add any functionality on its own and requires a dependent extension to actually enable functionality and thus there is no value in adding a feature structure. If necessary, any dependent extension could add a feature boolean if it wanted to indicate that it is adding optional deferral support.

Version History

- Revision 1, 2019-12-05 (Josh Barczak, Daniel Koch)
  - Initial draft.
- Revision 2, 2020-03-06 (Daniel Koch, Tobias Hector)
  - Add missing VK_OBJECT_TYPE_DEFERRED_OPERATION_KHR enum
  - fix sample code
  - Clarified deferred operation parameter lifetimes (#2018,!3647)
- Revision 3, 2020-05-15 (Josh Barczak)
  - Clarify behavior of vkGetDeferredOperationMaxConcurrencyKHR, allowing it to return 0 if the operation is complete (#2036,!3850)
- Revision 4, 2020-11-12 (Tobias Hector, Daniel Koch)
  - Remove VkDeferredOperationInfoKHR and change return value semantics when deferred host operations are in use (#2067,3813)
  - clarify return value of vkGetDeferredOperationResultKHR (#2339,!4110)

VK_KHR_depth_stencil_resolve

Name String

`VK_KHR_depth_stencil_resolve`

Extension Type

Device extension

Registered Extension Number

200

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_create_renderpass2
or

Vulkan Version 1.2

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Jan-Harald Fredriksen [GitHub]janharald

Other Extension Metadata

Last Modified Date

2018-04-09

Contributors

- Jan-Harald Fredriksen, Arm
- Andrew Garrard, Samsung Electronics
- Soowan Park, Samsung Electronics
- Jeff Bolz, NVIDIA
- Daniel Rakos, AMD

Description

This extension adds support for automatically resolving multisampled depth/stencil attachments in a subpass in a similar manner as for color attachments.

Multisampled color attachments can be resolved at the end of a subpass by specifying `pResolveAttachments` entries corresponding to the `pColorAttachments` array entries. This does not allow for a way to map the resolve attachments to the depth/stencil attachment. The vkCmdResolveImage command does not allow for depth/stencil images. While there are other ways to resolve the depth/stencil attachment, they can give sub-optimal performance. Extending the `VkSubpassDescription2` in this extension allows an application to add a `pDepthStencilResolveAttachment`, that is similar to the color `pResolveAttachments`, that the `pDepthStencilAttachment` can be resolved into.

Depth and stencil samples are resolved to a single value based on the resolve mode. The set of possible resolve modes is defined in the VkResolveModeFlagBits enum. The `VK_RESOLVE_MODE_SAMPLE_ZERO_BIT` mode is the only mode that is required of all implementations (that support the extension or support Vulkan 1.2 or higher). Some implementations may also support averaging (the same as color sample resolve) or taking the minimum or maximum sample, which may be more suitable for depth/stencil resolve.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceDepthStencilResolvePropertiesKHR
- Extending VkSubpassDescription2:
  - VkSubpassDescriptionDepthStencilResolveKHR

New Enums

- VkResolveModeFlagBitsKHR

New Bitmasks

- VkResolveModeFlagsKHR

New Enum Constants

- VK_KHR_DEPTH_STENCIL_RESOLVE_EXTENSION_NAME
- VK_KHR_DEPTH_STENCIL_RESOLVE_SPEC_VERSION
- Extending VkResolveModeFlagBits:
  - VK_RESOLVE_MODE_AVERAGE_BIT_KHR
  - VK_RESOLVE_MODE_MAX_BIT_KHR
  - VK_RESOLVE_MODE_MIN_BIT_KHR
  - VK_RESOLVE_MODE_NONE_KHR
  - VK_RESOLVE_MODE_SAMPLE_ZERO_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_STENCIL_RESOLVE_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_SUBPASS_DESCRIPTION_DEPTH_STENCIL_RESOLVE_KHR

Version History

- Revision 1, 2018-04-09 (Jan-Harald Fredriksen)
  - Initial revision

VK_KHR_descriptor_update_template

Name String

VK_KHR_descriptor_update_template

Extension Type

Device extension

Registered Extension Number

86

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

API Interactions

- Interacts with VK_EXT_debug_report
- Interacts with VK_KHR_push_descriptor

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Markus Tavenrath [GitHub]mtavenrath

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Interactions and External Dependencies

- Interacts with `VK_KHR_push_descriptor`

Contributors

- Jeff Bolz, NVIDIA
- Michael Worcester, Imagination Technologies

Description

Applications may wish to update a fixed set of descriptors in a large number of descriptor sets very frequently, i.e. during initialization phase or if it is required to rebuild descriptor sets for each frame. For those cases it is also not unlikely that all information required to update a single descriptor set is stored in a single struct. This extension provides a way to update a fixed set of descriptors in a single VkDescriptorSet with a pointer to an application-defined data structure describing the new descriptors.

Promotion to Vulkan 1.1

vkCmdPushDescriptorSetWithTemplateKHR is included as an interaction with `VK_KHR_push_descriptor`. If Vulkan 1.1 and `VK_KHR_push_descriptor` are supported, this is included by `VK_KHR_push_descriptor`.

The base functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Object Types

- VkDescriptorUpdateTemplateKHR

New Commands

- vkCreateDescriptorUpdateTemplateKHR
- vkDestroyDescriptorUpdateTemplateKHR
- vkUpdateDescriptorSetWithTemplateKHR

If VK_KHR_push_descriptor is supported:

- vkCmdPushDescriptorSetWithTemplateKHR

New Structures

- VkDescriptorUpdateTemplateCreateInfoKHR
- VkDescriptorUpdateTemplateEntryKHR

New Enums

- VkDescriptorUpdateTemplateTypeKHR

New Bitmasks

- VkDescriptorUpdateTemplateCreateFlagsKHR

New Enum Constants

- `VK_KHR_DESCRIPTOR_UPDATE_TEMPLATE_EXTENSION_NAME`
- `VK_KHR_DESCRIPTOR_UPDATE_TEMPLATE_SPEC_VERSION`
- Extending VkDescriptorUpdateTemplateType:
  - `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_DESCRIPTOR_SET_KHR`
- Extending VkObjectType:
  - `VK_OBJECT_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_CREATE_INFO_KHR`

If `VK_EXT_debug_report` is supported:

- Extending `VkDebugReportObjectTypeEXT`:
  - `VK_DEBUG_REPORT_OBJECT_TYPE_DESCRIPTOR_UPDATE_TEMPLATE_KHR_EXT`

If VK_KHR_push_descriptor is supported:

- Extending VkDescriptorUpdateTemplateType:
  - `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`

Version History

- Revision 1, 2016-01-11 (Markus Tavenrath)
  - Initial draft

VK_KHR_device_group

Name String

`VK_KHR_device_group`

Extension Type

Device extension

Registered Extension Number

61

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_device_group_creation

API Interactions

- Interacts with VK_KHR_bind_memory2
- Interacts with VK_KHR_surface
- Interacts with VK_KHR_swapchain

SPIR-V Dependencies

- SPV_KHR_device_group

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2017-10-10

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA
- Tobias Hector, Imagination Technologies

Description

This extension provides functionality to use a logical device that consists of multiple physical devices, as created with the `VK_KHR_device_group_creation` extension. A device group can allocate memory across the subdevices, bind memory from one subdevice to a resource on another subdevice, record command buffers where some work executes on an arbitrary subset of the subdevices, and potentially present a swapchain image from one or more subdevices.

Promotion to Vulkan 1.1

The following enums, types and commands are included as interactions with `VK_KHR_swapchain`:

- `VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_CAPABILITIES_KHR`
- `VK_STRUCTURE_TYPE_IMAGE_SWAPCHAIN_CREATE_INFO_KHR`
- `VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_SWAPCHAIN_INFO_KHR`
- `VK_STRUCTURE_TYPE_ACQUIRE_NEXT_IMAGE_INFO_KHR`
- `VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_INFO_KHR`
- `VK_STRUCTURE_TYPE_DEVICE_GROUP_SWAPCHAIN_CREATE_INFO_KHR`
- `VK_SWAPCHAIN_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR`
- VkDeviceGroupPresentModeFlagBitsKHR
- VkDeviceGroupPresentCapabilitiesKHR
- VkImageSwapchainCreateInfoKHR
- VkBindImageMemorySwapchainInfoKHR
- VkAcquireNextImageInfoKHR
- VkDeviceGroupPresentInfoKHR
- VkDeviceGroupSwapchainCreateInfoKHR
- vkGetDeviceGroupPresentCapabilitiesKHR
- vkGetDeviceGroupSurfacePresentModesKHR
- vkGetPhysicalDevicePresentRectanglesKHR
- vkAcquireNextImage2KHR

If Vulkan 1.1 and `VK_KHR_swapchain` are supported, these are included by `VK_KHR_swapchain`.

The base functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkCmdDispatchBaseKHR
- vkCmdSetDeviceMaskKHR
- vkGetDeviceGroupPeerMemoryFeaturesKHR

If VK_KHR_surface is supported:

- vkGetDeviceGroupPresentCapabilitiesKHR
- vkGetDeviceGroupSurfacePresentModesKHR
- vkGetPhysicalDevicePresentRectanglesKHR

If VK_KHR_swapchain is supported:

- vkAcquireNextImage2KHR

New Structures

- Extending VkBindSparseInfo:
  - VkDeviceGroupBindSparseInfoKHR
- Extending VkCommandBufferBeginInfo:
  - VkDeviceGroupCommandBufferBeginInfoKHR
- Extending VkMemoryAllocateInfo:
  - VkMemoryAllocateFlagsInfoKHR
- Extending VkRenderPassBeginInfo, VkRenderingInfo:
  - VkDeviceGroupRenderPassBeginInfoKHR
- Extending VkSubmitInfo:
  - VkDeviceGroupSubmitInfoKHR

If VK_KHR_bind_memory2 is supported:

- Extending VkBindBufferMemoryInfo:
  - VkBindBufferMemoryDeviceGroupInfoKHR
- Extending VkBindImageMemoryInfo:
  - VkBindImageMemoryDeviceGroupInfoKHR

If VK_KHR_surface is supported:

- VkDeviceGroupPresentCapabilitiesKHR

If VK_KHR_swapchain is supported:

- VkAcquireNextImageInfoKHR
- Extending VkBindImageMemoryInfo:

    - VkBindImageMemorySwapchainInfoKHR
- Extending VkImageCreateInfo:
    - VkImageSwapchainCreateInfoKHR
- Extending VkPresentInfoKHR:
    - VkDeviceGroupPresentInfoKHR
- Extending VkSwapchainCreateInfoKHR:
    - VkDeviceGroupSwapchainCreateInfoKHR

New Enums

- VkMemoryAllocateFlagBitsKHR
- VkPeerMemoryFeatureFlagBitsKHR

If VK_KHR_surface is supported:

- VkDeviceGroupPresentModeFlagBitsKHR

New Bitmasks

- VkMemoryAllocateFlagsKHR
- VkPeerMemoryFeatureFlagsKHR

If VK_KHR_surface is supported:

- VkDeviceGroupPresentModeFlagsKHR

New Enum Constants

- `VK_KHR_DEVICE_GROUP_EXTENSION_NAME`
- `VK_KHR_DEVICE_GROUP_SPEC_VERSION`
- Extending VkDependencyFlagBits:
    - `VK_DEPENDENCY_DEVICE_GROUP_BIT_KHR`
- Extending VkMemoryAllocateFlagBits:
    - `VK_MEMORY_ALLOCATE_DEVICE_MASK_BIT_KHR`
- Extending VkPeerMemoryFeatureFlagBits:
    - `VK_PEER_MEMORY_FEATURE_COPY_DST_BIT_KHR`
    - `VK_PEER_MEMORY_FEATURE_COPY_SRC_BIT_KHR`
    - `VK_PEER_MEMORY_FEATURE_GENERIC_DST_BIT_KHR`
    - `VK_PEER_MEMORY_FEATURE_GENERIC_SRC_BIT_KHR`
- Extending VkPipelineCreateFlagBits:
    - `VK_PIPELINE_CREATE_DISPATCH_BASE_KHR`
    - `VK_PIPELINE_CREATE_VIEW_INDEX_FROM_DEVICE_INDEX_BIT_KHR`

- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_DEVICE_GROUP_BIND_SPARSE_INFO_KHR
  - VK_STRUCTURE_TYPE_DEVICE_GROUP_COMMAND_BUFFER_BEGIN_INFO_KHR
  - VK_STRUCTURE_TYPE_DEVICE_GROUP_RENDER_PASS_BEGIN_INFO_KHR
  - VK_STRUCTURE_TYPE_DEVICE_GROUP_SUBMIT_INFO_KHR
  - VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_FLAGS_INFO_KHR

If VK_KHR_bind_memory2 is supported:

- Extending VkImageCreateFlagBits:
  - VK_IMAGE_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_BIND_BUFFER_MEMORY_DEVICE_GROUP_INFO_KHR
  - VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_DEVICE_GROUP_INFO_KHR

If VK_KHR_surface is supported:

- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_CAPABILITIES_KHR

If VK_KHR_swapchain is supported:

- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_ACQUIRE_NEXT_IMAGE_INFO_KHR
  - VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_SWAPCHAIN_INFO_KHR
  - VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_INFO_KHR
  - VK_STRUCTURE_TYPE_DEVICE_GROUP_SWAPCHAIN_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_IMAGE_SWAPCHAIN_CREATE_INFO_KHR
- Extending VkSwapchainCreateFlagBitsKHR:
  - VK_SWAPCHAIN_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR

New Built-in Variables

- DeviceIndex

New SPIR-V Capabilities

- DeviceGroup

Version History

- Revision 1, 2016-10-19 (Jeff Bolz)
  - Internal revisions

- Revision 2, 2017-05-19 (Tobias Hector)
  - Removed extended memory bind functions to VK_KHR_bind_memory2, added dependency on that extension, and device-group-specific structs for those functions.
- Revision 3, 2017-10-06 (Ian Elliott)
  - Corrected Vulkan 1.1 interactions with the WSI extensions. All Vulkan 1.1 WSI interactions are with the VK_KHR_swapchain extension.
- Revision 4, 2017-10-10 (Jeff Bolz)
  - Rename “SFR” bits and structure members to use the phrase “split instance bind regions”.

VK_KHR_device_group_creation

Name String

`VK_KHR_device_group_creation`

Extension Type

Instance extension

Registered Extension Number

71

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2016-10-19

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA

Description

This extension provides instance-level commands to enumerate groups of physical devices, and to create a logical device from a subset of one of those groups. Such a logical device can then be used with new features in the VK_KHR_device_group extension.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkEnumeratePhysicalDeviceGroupsKHR

New Structures

- VkPhysicalDeviceGroupPropertiesKHR
- Extending VkDeviceCreateInfo:
  - VkDeviceGroupDeviceCreateInfoKHR

New Enum Constants

- VK_KHR_DEVICE_GROUP_CREATION_EXTENSION_NAME
- VK_KHR_DEVICE_GROUP_CREATION_SPEC_VERSION
- VK_MAX_DEVICE_GROUP_SIZE_KHR
- Extending VkMemoryHeapFlagBits:
  - VK_MEMORY_HEAP_MULTI_INSTANCE_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_DEVICE_GROUP_DEVICE_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GROUP_PROPERTIES_KHR

Examples

```
    VkDeviceCreateInfo devCreateInfo = { VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO };
    // (not shown) fill out devCreateInfo as usual.
    uint32_t deviceGroupCount = 0;
    VkPhysicalDeviceGroupPropertiesKHR *props = NULL;

    // Query the number of device groups
    vkEnumeratePhysicalDeviceGroupsKHR(g_vkInstance, &deviceGroupCount, NULL);

    // Allocate and initialize structures to query the device groups
    props = (VkPhysicalDeviceGroupPropertiesKHR *)malloc(deviceGroupCount*sizeof
(VkPhysicalDeviceGroupPropertiesKHR));
    for (i = 0; i < deviceGroupCount; ++i) {
```

```
        props[i].sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GROUP_PROPERTIES_KHR;
        props[i].pNext = NULL;
    }
    vkEnumeratePhysicalDeviceGroupsKHR(g_vkInstance, &deviceGroupCount, props);

    // If the first device group has more than one physical device. create
    // a logical device using all of the physical devices.
    VkDeviceGroupDeviceCreateInfoKHR deviceGroupInfo = {
VK_STRUCTURE_TYPE_DEVICE_GROUP_DEVICE_CREATE_INFO_KHR };
    if (props[0].physicalDeviceCount > 1) {
        deviceGroupInfo.physicalDeviceCount = props[0].physicalDeviceCount;
        deviceGroupInfo.pPhysicalDevices = props[0].physicalDevices;
        devCreateInfo.pNext = &deviceGroupInfo;
    }

    vkCreateDevice(props[0].physicalDevices[0], &devCreateInfo, NULL, &g_vkDevice);
    free(props);
```

Version History

- Revision 1, 2016-10-19 (Jeff Bolz)
    - Internal revisions

VK_KHR_display

Name String

`VK_KHR_display`

Extension Type

Instance extension

Registered Extension Number

3

Revision

23

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- James Jones [GitHub]cubanismo
- Norbert Nopper [GitHub]FslNopper

Other Extension Metadata

Last Modified Date

2017-03-13

IP Status

No known IP claims.

Contributors

- James Jones, NVIDIA
- Norbert Nopper, Freescale
- Jeff Vigil, Qualcomm
- Daniel Rakos, AMD

Description

This extension provides the API to enumerate displays and available modes on a given device.

New Object Types

- VkDisplayKHR
- VkDisplayModeKHR

New Commands

- vkCreateDisplayModeKHR
- vkCreateDisplayPlaneSurfaceKHR
- vkGetDisplayModePropertiesKHR
- vkGetDisplayPlaneCapabilitiesKHR
- vkGetDisplayPlaneSupportedDisplaysKHR
- vkGetPhysicalDeviceDisplayPlanePropertiesKHR
- vkGetPhysicalDeviceDisplayPropertiesKHR

New Structures

- VkDisplayModeCreateInfoKHR
- VkDisplayModeParametersKHR
- VkDisplayModePropertiesKHR
- VkDisplayPlaneCapabilitiesKHR
- VkDisplayPlanePropertiesKHR
- VkDisplayPropertiesKHR
- VkDisplaySurfaceCreateInfoKHR

New Enums

- VkDisplayPlaneAlphaFlagBitsKHR

New Bitmasks

- VkDisplayModeCreateFlagsKHR
- VkDisplayPlaneAlphaFlagsKHR
- VkDisplaySurfaceCreateFlagsKHR
- VkSurfaceTransformFlagsKHR

New Enum Constants

- `VK_KHR_DISPLAY_EXTENSION_NAME`
- `VK_KHR_DISPLAY_SPEC_VERSION`
- Extending VkObjectType:
  - `VK_OBJECT_TYPE_DISPLAY_KHR`
  - `VK_OBJECT_TYPE_DISPLAY_MODE_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_DISPLAY_MODE_CREATE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_DISPLAY_SURFACE_CREATE_INFO_KHR`

Issues

1) Which properties of a mode should be fixed in the mode information vs. settable in some other function when setting the mode? E.g., do we need to double the size of the mode pool to include both stereo and non-stereo modes? YUV and RGB scanout even if they both take RGB input images? BGR vs. RGB input? etc.

RESOLVED: Many modern displays support at most a handful of resolutions and timings natively. Other “modes” are expected to be supported using scaling hardware on the display engine or GPU. Other properties, such as rotation and mirroring should not require duplicating hardware modes just to express all combinations. Further, these properties may be implemented on a per-display or per-overlay granularity.

To avoid the exponential growth of modes as mutable properties are added, as was the case with `EGLConfig`/WGL pixel formats/`GLXFBConfig`, this specification should separate out hardware properties and configurable state into separate objects. Modes and overlay planes will express capabilities of the hardware, while a separate structure will allow applications to configure scaling, rotation, mirroring, color keys, LUT values, alpha masks, etc. for a given swapchain independent of the mode in use. Constraints on these settings will be established by properties of the immutable objects.

Note the resolution of this issue may affect issue 5 as well.

2) What properties of a display itself are useful?

RESOLVED: This issue is too broad. It was meant to prompt general discussion, but resolving this issue amounts to completing this specification. All interesting properties should be included. The issue will remain as a placeholder since removing it would make it hard to parse existing discussion notes that refer to issues by number.

3) How are multiple overlay planes within a display or mode enumerated?

RESOLVED: They are referred to by an index. Each display will report the number of overlay planes it contains.

4) Should swapchains be created relative to a mode or a display?

RESOLVED: When using this extension, swapchains are created relative to a mode and a plane. The mode implies the display object the swapchain will present to. If the specified mode is not the display's current mode, the new mode will be applied when the first image is presented to the swapchain, and the default operating system mode, if any, will be restored when the swapchain is destroyed.

5) Should users query generic ranges from displays and construct their own modes explicitly using those constraints rather than querying a fixed set of modes (Most monitors only have one real "mode" these days, even though many support relatively arbitrary scaling, either on the monitor side or in the GPU display engine, making "modes" something of a relic/compatibility construct).

RESOLVED: Expose both. Display information structures will expose a set of predefined modes, as well as any attributes necessary to construct a customized mode.

6) Is it fine if we return the display and display mode handles in the structure used to query their properties?

RESOLVED: Yes.

7) Is there a possibility that not all displays of a device work with all of the present queues of a device? If yes, how do we determine which displays work with which present queues?

RESOLVED: No known hardware has such limitations, but determining such limitations is supported automatically using the existing `VK_KHR_surface` and `VK_KHR_swapchain` query mechanisms.

8) Should all presentation need to be done relative to an overlay plane, or can a display mode + display be used alone to target an output?

RESOLVED: Require specifying a plane explicitly.

9) Should displays have an associated window system display, such as an `HDC` or `Display*`?

RESOLVED: No. Displays are independent of any windowing system in use on the system. Further, neither `HDC` nor `Display*` refer to a physical display object.

10) Are displays queried from a physical GPU or from a device instance?

RESOLVED: Developers prefer to query modes directly from the physical GPU so they can use display information as an input to their device selection algorithms prior to device creation. This

avoids the need to create placeholder device instances to enumerate displays.

This preference must be weighed against the extra initialization that must be done by driver vendors prior to device instance creation to support this usage.

11) Should displays and/or modes be dispatchable objects? If functions are to take displays, overlays, or modes as their first parameter, they must be dispatchable objects as defined in Khronos bug 13529. If they are not added to the list of dispatchable objects, functions operating on them must take some higher-level object as their first parameter. There is no performance case against making them dispatchable objects, but they would be the first extension objects to be dispatchable.

RESOLVED: Do not make displays or modes dispatchable. They will dispatch based on their associated physical device.

12) Should hardware cursor capabilities be exposed?

RESOLVED: Defer. This could be a separate extension on top of the base WSI specs.

13) How many display objects should be enumerated for "tiled" display devices? There are ongoing design discussions among lower-level display API authors regarding how to expose displays if they are one physical display device to an end user, but may internally be implemented as two side-by-side displays using the same display engine (and sometimes cabling) resources as two physically separate display devices.

RESOLVED: Tiled displays will appear as a single display object in this API.

14) Should the raw EDID data be included in the display information?

RESOLVED: No. A future extension could be added which reports the EDID if necessary. This may be complicated by the outcome of issue 13.

15) Should min and max scaling factor capabilities of overlays be exposed?

RESOLVED: Yes. This is exposed indirectly by allowing applications to query the min/max position and extent of the source and destination regions from which image contents are fetched by the display engine when using a particular mode and overlay pair.

16) Should devices be able to expose planes that can be moved between displays? If so, how?

RESOLVED: Yes. Applications can determine which displays a given plane supports using vkGetDisplayPlaneSupportedDisplaysKHR.

17) Should there be a way to destroy display modes? If so, does it support destroying “built in” modes?

RESOLVED: Not in this extension. A future extension could add this functionality.

18) What should the lifetime of display and built-in display mode objects be?

RESOLVED: The lifetime of the instance. These objects cannot be destroyed. A future extension may be added to expose a way to destroy these objects and/or support display hotplug.

19) Should persistent mode for smart panels be enabled/disabled at swapchain creation time, or on a per-present basis.

RESOLVED: On a per-present basis.

Examples

NOTE	The example code for the `VK_KHR_display` and `VK_KHR_display_swapchain` extensions was removed from the appendix after revision 1.0.43. The display enumeration example code was ported to the cube demo that is shipped with the official Khronos SDK, and is being kept up-to-date in that location (see: https://github.com/KhronosGroup/Vulkan-Tools/blob/main/cube/cube.c).

Version History

- Revision 1, 2015-02-24 (James Jones)
  - Initial draft
- Revision 2, 2015-03-12 (Norbert Nopper)
  - Added overlay enumeration for a display.
- Revision 3, 2015-03-17 (Norbert Nopper)
  - Fixed typos and namings as discussed in Bugzilla.
  - Reordered and grouped functions.
  - Added functions to query count of display, mode and overlay.
  - Added native display handle, which may be needed on some platforms to create a native Window.
- Revision 4, 2015-03-18 (Norbert Nopper)
  - Removed primary and virtualPostion members (see comment of James Jones in Bugzilla).
  - Added native overlay handle to information structure.
  - Replaced , with ; in struct.
- Revision 6, 2015-03-18 (Daniel Rakos)
  - Added WSI extension suffix to all items.
  - Made the whole API more “Vulkanish”.
  - Replaced all functions with a single vkGetDisplayInfoKHR function to better match the rest of the API.
  - Made the display, display mode, and overlay objects be first class objects, not subclasses of VkBaseObject as they do not support the common functions anyways.
  - Renamed *Info structures to *Properties.
  - Removed overlayIndex field from VkOverlayProperties as there is an implicit index already as a result of moving to a “Vulkanish” API.
  - Displays are not get through device, but through physical GPU to match the rest of the

Vulkan API. Also this is something ISVs explicitly requested.

    - Added issue (6) and (7).
- Revision 7, 2015-03-25 (James Jones)
    - Added an issues section
    - Added rotation and mirroring flags
- Revision 8, 2015-03-25 (James Jones)
    - Combined the duplicate issues sections introduced in last change.
    - Added proposed resolutions to several issues.
- Revision 9, 2015-04-01 (Daniel Rakos)
    - Rebased extension against Vulkan 0.82.0
- Revision 10, 2015-04-01 (James Jones)
    - Added issues (10) and (11).
    - Added more straw-man issue resolutions, and cleaned up the proposed resolution for issue (4).
    - Updated the rotation and mirroring enums to have proper bitmask semantics.
- Revision 11, 2015-04-15 (James Jones)
    - Added proposed resolution for issues (1) and (2).
    - Added issues (12), (13), (14), and (15)
    - Removed pNativeHandle field from overlay structure.
    - Fixed small compilation errors in example code.
- Revision 12, 2015-07-29 (James Jones)
    - Rewrote the guts of the extension against the latest WSI swapchain specifications and the latest Vulkan API.
    - Address overlay planes by their index rather than an object handle and refer to them as “planes” rather than “overlays” to make it slightly clearer that even a display with no “overlays” still has at least one base “plane” that images can be displayed on.
    - Updated most of the issues.
    - Added an “extension type” section to the specification header.
    - Reused the VK_EXT_KHR_surface surface transform enumerations rather than redefining them here.
    - Updated the example code to use the new semantics.
- Revision 13, 2015-08-21 (Ian Elliott)
    - Renamed this extension and all of its enumerations, types, functions, etc. This makes it compliant with the proposed standard for Vulkan extensions.
    - Switched from “revision” to “version”, including use of the VK_MAKE_VERSION macro in the header file.
- Revision 14, 2015-09-01 (James Jones)

    - Restore single-field revision number.
- Revision 15, 2015-09-08 (James Jones)
    - Added alpha flags enum.
    - Added premultiplied alpha support.
- Revision 16, 2015-09-08 (James Jones)
    - Added description section to the spec.
    - Added issues 16 - 18.
- Revision 17, 2015-10-02 (James Jones)
    - Planes are now a property of the entire device rather than individual displays. This allows planes to be moved between multiple displays on devices that support it.
    - Added a function to create a VkSurfaceKHR object describing a display plane and mode to align with the new per-platform surface creation conventions.
    - Removed detailed mode timing data. It was agreed that the mode extents and refresh rate are sufficient for current use cases. Other information could be added back in as an extension if it is needed in the future.
    - Added support for smart/persistent/buffered display devices.
- Revision 18, 2015-10-26 (Ian Elliott)
    - Renamed from VK_EXT_KHR_display to VK_KHR_display.
- Revision 19, 2015-11-02 (James Jones)
    - Updated example code to match revision 17 changes.
- Revision 20, 2015-11-03 (Daniel Rakos)
    - Added allocation callbacks to creation functions.
- Revision 21, 2015-11-10 (Jesse Hall)
    - Added VK_DISPLAY_PLANE_ALPHA_OPAQUE_BIT_KHR, and use VkDisplayPlaneAlphaFlagBitsKHR for VkDisplayPlanePropertiesKHR::alphaMode instead of VkDisplayPlaneAlphaFlagsKHR, since it only represents one mode.
    - Added reserved flags bitmask to VkDisplayPlanePropertiesKHR.
    - Use VkSurfaceTransformFlagBitsKHR instead of obsolete VkSurfaceTransformKHR.
    - Renamed vkGetDisplayPlaneSupportedDisplaysKHR parameters for clarity.
- Revision 22, 2015-12-18 (James Jones)
    - Added missing “planeIndex” parameter to vkGetDisplayPlaneSupportedDisplaysKHR()
- Revision 23, 2017-03-13 (James Jones)
    - Closed all remaining issues. The specification and implementations have been shipping with the proposed resolutions for some time now.
    - Removed the sample code and noted it has been integrated into the official Vulkan SDK cube demo.

VK_KHR_display_swapchain

Name String

`VK_KHR_display_swapchain`

Extension Type

Device extension

Registered Extension Number

4

Revision

10

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_swapchain
and
VK_KHR_display

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2017-03-13

IP Status

No known IP claims.

Contributors

- James Jones, NVIDIA
- Jeff Vigil, Qualcomm
- Jesse Hall, Google

Description

This extension provides an API to create a swapchain directly on a device's display without any underlying window system.

New Commands

- vkCreateSharedSwapchainsKHR

New Structures

- Extending VkPresentInfoKHR:
  - VkDisplayPresentInfoKHR

New Enum Constants

- `VK_KHR_DISPLAY_SWAPCHAIN_EXTENSION_NAME`
- `VK_KHR_DISPLAY_SWAPCHAIN_SPEC_VERSION`
- Extending VkResult:
  - `VK_ERROR_INCOMPATIBLE_DISPLAY_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_DISPLAY_PRESENT_INFO_KHR`

Issues

1) Should swapchains sharing images each hold a reference to the images, or should it be up to the application to destroy the swapchains and images in an order that avoids the need for reference counting?

RESOLVED: Take a reference. The lifetime of presentable images is already complex enough.

2) Should the `srcRect` and `dstRect` parameters be specified as part of the presentation command, or at swapchain creation time?

RESOLVED: As part of the presentation command. This allows moving and scaling the image on the screen without the need to respecify the mode or create a new swapchain and presentable images.

3) Should `srcRect` and `dstRect` be specified as rects, or separate offset/extent values?

RESOLVED: As rects. Specifying them separately might make it easier for hardware to expose support for one but not the other, but in such cases applications must just take care to obey the reported capabilities and not use non-zero offsets or extents that require scaling, as appropriate.

4) How can applications create multiple swapchains that use the same images?

RESOLVED: By calling vkCreateSharedSwapchainsKHR.

An earlier resolution used vkCreateSwapchainKHR, chaining multiple VkSwapchainCreateInfoKHR structures through `pNext`. In order to allow each swapchain to also allow other extension structs, a level of indirection was used: VkSwapchainCreateInfoKHR::`pNext` pointed to a different structure, which had both `sType` and `pNext` members for additional extensions, and also had a pointer to the next VkSwapchainCreateInfoKHR structure. The number of swapchains to be created could only be found by walking this linked list of alternating structures, and the `pSwapchains` out parameter was reinterpreted to be an array of VkSwapchainKHR handles.

Another option considered was a method to specify a “shared” swapchain when creating a new swapchain, such that groups of swapchains using the same images could be built up one at a time. This was deemed unusable because drivers need to know all of the displays an image will be used

on when determining which internal formats and layouts to use for that image.

Examples

NOTE	The example code for the `VK_KHR_display` and `VK_KHR_display_swapchain` extensions was removed from the appendix after revision 1.0.43. The display swapchain creation example code was ported to the cube demo that is shipped with the official Khronos SDK, and is being kept up-to-date in that location (see: https://github.com/KhronosGroup/Vulkan-Tools/blob/main/cube/cube.c).

Version History

- Revision 1, 2015-07-29 (James Jones)
  - Initial draft
- Revision 2, 2015-08-21 (Ian Elliott)
  - Renamed this extension and all of its enumerations, types, functions, etc. This makes it compliant with the proposed standard for Vulkan extensions.
  - Switched from “revision” to “version”, including use of the VK_MAKE_VERSION macro in the header file.
- Revision 3, 2015-09-01 (James Jones)
  - Restore single-field revision number.
- Revision 4, 2015-09-08 (James Jones)
  - Allow creating multiple swapchains that share the same images using a single call to vkCreateSwapchainKHR().
- Revision 5, 2015-09-10 (Alon Or-bach)
  - Removed underscores from SWAP_CHAIN in two enums.
- Revision 6, 2015-10-02 (James Jones)
  - Added support for smart panels/buffered displays.
- Revision 7, 2015-10-26 (Ian Elliott)
  - Renamed from VK_EXT_KHR_display_swapchain to VK_KHR_display_swapchain.
- Revision 8, 2015-11-03 (Daniel Rakos)
  - Updated sample code based on the changes to VK_KHR_swapchain.
- Revision 9, 2015-11-10 (Jesse Hall)
  - Replaced VkDisplaySwapchainCreateInfoKHR with vkCreateSharedSwapchainsKHR, changing resolution of issue #4.
- Revision 10, 2017-03-13 (James Jones)
  - Closed all remaining issues. The specification and implementations have been shipping with the proposed resolutions for some time now.
  - Removed the sample code and noted it has been integrated into the official Vulkan SDK cube demo.

VK_KHR_draw_indirect_count

Name String

`VK_KHR_draw_indirect_count`

Extension Type

Device extension

Registered Extension Number

170

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2017-08-25

IP Status

No known IP claims.

Contributors

- Matthaeus G. Chajdas, AMD
- Derrick Owens, AMD
- Graham Sellers, AMD
- Daniel Rakos, AMD
- Dominik Witczak, AMD
- Piers Daniell, NVIDIA

Description

This extension is based on the `VK_AMD_draw_indirect_count` extension. This extension allows an application to source the number of draws for indirect drawing calls from a buffer.

Applications might want to do culling on the GPU via a compute shader prior to drawing. This enables the application to generate an arbitrary number of drawing commands and execute them without host intervention.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. However, if Vulkan 1.2 is supported and this extension is not, the commands vkCmdDrawIndirectCountKHR and vkCmdDrawIndexedIndirectCountKHR are optional. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkCmdDrawIndexedIndirectCountKHR
- vkCmdDrawIndirectCountKHR

New Enum Constants

- `VK_KHR_DRAW_INDIRECT_COUNT_EXTENSION_NAME`
- `VK_KHR_DRAW_INDIRECT_COUNT_SPEC_VERSION`

Version History

- Revision 1, 2017-08-25 (Piers Daniell)
  - Initial draft based on VK_AMD_draw_indirect_count

VK_KHR_driver_properties

Name String

`VK_KHR_driver_properties`

Extension Type

Device extension

Registered Extension Number

197

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Daniel Rakos [GitHub]drakos-amd

Other Extension Metadata

Last Modified Date

2018-04-11

IP Status

No known IP claims.

Contributors

- Baldur Karlsson
- Matthaeus G. Chajdas, AMD
- Piers Daniell, NVIDIA
- Alexander Galazin, Arm
- Jesse Hall, Google
- Daniel Rakos, AMD

Description

This extension provides a new physical device query which allows retrieving information about the driver implementation, allowing applications to determine which physical device corresponds to which particular vendor's driver, and which conformance test suite version the driver implementation is compliant with.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- VkConformanceVersionKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceDriverPropertiesKHR

New Enums

- VkDriverIdKHR

New Enum Constants

- `VK_KHR_DRIVER_PROPERTIES_EXTENSION_NAME`

- VK_KHR_DRIVER_PROPERTIES_SPEC_VERSION
- VK_MAX_DRIVER_INFO_SIZE_KHR
- VK_MAX_DRIVER_NAME_SIZE_KHR
- Extending VkDriverId:
  - VK_DRIVER_ID_AMD_OPEN_SOURCE_KHR
  - VK_DRIVER_ID_AMD_PROPRIETARY_KHR
  - VK_DRIVER_ID_ARM_PROPRIETARY_KHR
  - VK_DRIVER_ID_BROADCOM_PROPRIETARY_KHR
  - VK_DRIVER_ID_GGP_PROPRIETARY_KHR
  - VK_DRIVER_ID_GOOGLE_SWIFTSHADER_KHR
  - VK_DRIVER_ID_IMAGINATION_PROPRIETARY_KHR
  - VK_DRIVER_ID_INTEL_OPEN_SOURCE_MESA_KHR
  - VK_DRIVER_ID_INTEL_PROPRIETARY_WINDOWS_KHR
  - VK_DRIVER_ID_MESA_RADV_KHR
  - VK_DRIVER_ID_NVIDIA_PROPRIETARY_KHR
  - VK_DRIVER_ID_QUALCOMM_PROPRIETARY_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DRIVER_PROPERTIES_KHR

Version History

- Revision 1, 2018-04-11 (Daniel Rakos)
  - Internal revisions

VK_KHR_dynamic_rendering

Name String

VK_KHR_dynamic_rendering

Extension Type

Device extension

Registered Extension Number

45

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
VK_KHR_depth_stencil_resolve
or
Vulkan Version 1.2

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Tobias Hector [GitHub]tobski

Extension Proposal

VK_KHR_dynamic_rendering

Other Extension Metadata

Last Modified Date

2021-10-06

Contributors

- Tobias Hector, AMD
- Arseny Kapoulkine, Roblox
- François Duranleau, Gameloft
- Stuart Smith, AMD
- Hai Nguyen, Google
- Jean-François Roy, Google
- Jeff Leger, Qualcomm
- Jan-Harald Fredriksen, Arm
- Piers Daniell, Nvidia
- James Fitzpatrick, Imagination
- Piotr Byszewski, Mobica
- Jesse Hall, Google
- Mike Blumenkrantz, Valve

Description

This extension allows applications to create single-pass render pass instances without needing to create render pass objects or framebuffers. Dynamic render passes can also span across multiple primary command buffers, rather than relying on secondary command buffers.

This extension also incorporates `VK_ATTACHMENT_STORE_OP_NONE_KHR` from

VK_QCOM_render_pass_store_ops, enabling applications to avoid unnecessary synchronization when an attachment is not written during a render pass.

New Commands

- vkCmdBeginRenderingKHR
- vkCmdEndRenderingKHR

New Structures

- VkRenderingAttachmentInfoKHR
- VkRenderingInfoKHR
- Extending VkCommandBufferInheritanceInfo:
  - VkCommandBufferInheritanceRenderingInfoKHR
- Extending VkGraphicsPipelineCreateInfo:
  - VkPipelineRenderingCreateInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceDynamicRenderingFeaturesKHR

New Enums

- VkRenderingFlagBitsKHR

New Bitmasks

- VkRenderingFlagsKHR

New Enum Constants

- VK_KHR_DYNAMIC_RENDERING_EXTENSION_NAME
- VK_KHR_DYNAMIC_RENDERING_SPEC_VERSION
- Extending VkAttachmentStoreOp:
  - VK_ATTACHMENT_STORE_OP_NONE_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_RENDERING_INFO_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_RENDERING_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO_KHR
  - VK_STRUCTURE_TYPE_RENDERING_INFO_KHR

Promotion to Vulkan 1.3

Vulkan APIs in this extension are included in core Vulkan 1.3, with the KHR suffix omitted. External

interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

Version History

- Revision 1, 2021-10-06 (Tobias Hector)
  - Initial revision

VK_KHR_dynamic_rendering_local_read

Name String

`VK_KHR_dynamic_rendering_local_read`

Extension Type

Device extension

Registered Extension Number

233

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_dynamic_rendering
or
Vulkan Version 1.3

Contact

- Tobias Hector [GitHub]tobski

Extension Proposal

VK_KHR_dynamic_rendering_local_read

Other Extension Metadata

Last Modified Date

2023-11-03

Contributors

- Tobias Hector, AMD
- Hans-Kristian Arntzen, Valve
- Connor Abbott, Valve
- Pan Gao, Huawei
- Lionel Landwerlin, Intel

- Shahbaz Youssefi, Google
- Alyssa Rosenzweig, Valve
- Jan-Harald Fredriksen, Arm
- Mike Blumenkrantz, Valve
- Graeme Leese, Broadcom
- Piers Daniell, Nvidia
- Stuart Smith, AMD
- Daniel Story, Nintendo
- James Fitzpatrick, Imagination
- Piotr Byszewski, Mobica
- Spencer Fricke, LunarG
- Tom Olson, Arm
- Michal Pietrasiuk, Intel
- Matthew Netsch, Qualcomm
- Marty Johnson, Khronos
- Wyvern Wang, Huawei
- Jeff Bolz, Nvidia
- Samuel (Sheng-Wen) Huang, MediaTek

Description

This extension enables reads from attachments and resources written by previous fragment shaders within a dynamic render pass.

New Commands

- vkCmdSetRenderingAttachmentLocationsKHR
- vkCmdSetRenderingInputAttachmentIndicesKHR

New Structures

- Extending VkGraphicsPipelineCreateInfo, VkCommandBufferInheritanceInfo:
  - VkRenderingAttachmentLocationInfoKHR
  - VkRenderingInputAttachmentIndexInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceDynamicRenderingLocalReadFeaturesKHR

New Enum Constants

- `VK_KHR_DYNAMIC_RENDERING_LOCAL_READ_EXTENSION_NAME`

- VK_KHR_DYNAMIC_RENDERING_LOCAL_READ_SPEC_VERSION
- Extending VkImageLayout:
  - VK_IMAGE_LAYOUT_RENDERING_LOCAL_READ_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_LOCAL_READ_FEATURES_KHR
  - VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_LOCATION_INFO_KHR
  - VK_STRUCTURE_TYPE_RENDERING_INPUT_ATTACHMENT_INDEX_INFO_KHR

Version History

- Revision 1, 2023-11-03 (Tobias Hector)
  - Initial revision

VK_KHR_external_fence

Name String

VK_KHR_external_fence

Extension Type

Device extension

Registered Extension Number

114

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_fence_capabilities

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jesse Hall [GitHub]critsec

Other Extension Metadata

Last Modified Date

2017-05-08

IP Status

No known IP claims.

Contributors

- Jesse Hall, Google
- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Cass Everitt, Oculus
- Contributors to VK_KHR_external_semaphore

Description

An application using external memory may wish to synchronize access to that memory using fences. This extension enables an application to create fences from which non-Vulkan handles that reference the underlying synchronization primitive can be exported.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkFenceCreateInfo:
  - VkExportFenceCreateInfoKHR

New Enums

- VkFenceImportFlagBitsKHR

New Bitmasks

- VkFenceImportFlagsKHR

New Enum Constants

- VK_KHR_EXTERNAL_FENCE_EXTENSION_NAME
- VK_KHR_EXTERNAL_FENCE_SPEC_VERSION
- Extending VkFenceImportFlagBits:
  - VK_FENCE_IMPORT_TEMPORARY_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_EXPORT_FENCE_CREATE_INFO_KHR

Issues

This extension borrows concepts, semantics, and language from VK_KHR_external_semaphore. That extension's issues apply equally to this extension.

Version History

- Revision 1, 2017-05-08 (Jesse Hall)
  - Initial revision

VK_KHR_external_fence_capabilities

Name String

`VK_KHR_external_fence_capabilities`

Extension Type

Instance extension

Registered Extension Number

113

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jesse Hall [GitHub]critsec

Other Extension Metadata

Last Modified Date

2017-05-08

IP Status

No known IP claims.

Contributors

- Jesse Hall, Google
- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Cass Everitt, Oculus
- Contributors to `VK_KHR_external_semaphore_capabilities`

Description

An application may wish to reference device fences in multiple Vulkan logical devices or instances, in multiple processes, and/or in multiple APIs. This extension provides a set of capability queries and handle definitions that allow an application to determine what types of “external” fence handles an implementation supports for a given set of use cases.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkGetPhysicalDeviceExternalFencePropertiesKHR

New Structures

- VkExternalFencePropertiesKHR
- VkPhysicalDeviceExternalFenceInfoKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceIDPropertiesKHR

New Enums

- VkExternalFenceFeatureFlagBitsKHR
- VkExternalFenceHandleTypeFlagBitsKHR

New Bitmasks

- VkExternalFenceFeatureFlagsKHR
- VkExternalFenceHandleTypeFlagsKHR

New Enum Constants

- `VK_KHR_EXTERNAL_FENCE_CAPABILITIES_EXTENSION_NAME`
- `VK_KHR_EXTERNAL_FENCE_CAPABILITIES_SPEC_VERSION`
- `VK_LUID_SIZE_KHR`
- Extending VkExternalFenceFeatureFlagBits:
  - `VK_EXTERNAL_FENCE_FEATURE_EXPORTABLE_BIT_KHR`
  - `VK_EXTERNAL_FENCE_FEATURE_IMPORTABLE_BIT_KHR`
- Extending VkExternalFenceHandleTypeFlagBits:
  - `VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_FD_BIT_KHR`
  - `VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR`
  - `VK_EXTERNAL_FENCE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT_KHR`

    - VK_EXTERNAL_FENCE_HANDLE_TYPE_SYNC_FD_BIT_KHR
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_EXTERNAL_FENCE_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_FENCE_INFO_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ID_PROPERTIES_KHR

Version History

- Revision 1, 2017-05-08 (Jesse Hall)
    - Initial version

VK_KHR_external_fence_fd

Name String

VK_KHR_external_fence_fd

Extension Type

Device extension

Registered Extension Number

116

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_fence
or
Vulkan Version 1.1

Contact

- Jesse Hall [GitHub]critsec

Other Extension Metadata

Last Modified Date

2017-05-08

IP Status

No known IP claims.

Contributors

- Jesse Hall, Google

- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Cass Everitt, Oculus
- Contributors to VK_KHR_external_semaphore_fd

Description

An application using external memory may wish to synchronize access to that memory using fences. This extension enables an application to export fence payload to and import fence payload from POSIX file descriptors.

New Commands

- vkGetFenceFdKHR
- vkImportFenceFdKHR

New Structures

- VkFenceGetFdInfoKHR
- VkImportFenceFdInfoKHR

New Enum Constants

- VK_KHR_EXTERNAL_FENCE_FD_EXTENSION_NAME
- VK_KHR_EXTERNAL_FENCE_FD_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_FENCE_GET_FD_INFO_KHR
  - VK_STRUCTURE_TYPE_IMPORT_FENCE_FD_INFO_KHR

Issues

This extension borrows concepts, semantics, and language from VK_KHR_external_semaphore_fd. That extension's issues apply equally to this extension.

Version History

- Revision 1, 2017-05-08 (Jesse Hall)
  - Initial revision

VK_KHR_external_fence_win32

Name String

VK_KHR_external_fence_win32

Extension Type

Device extension

Registered Extension Number

115

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_fence

Contact

- Jesse Hall [GitHub]critsec

Other Extension Metadata

Last Modified Date

2017-05-08

IP Status

No known IP claims.

Contributors

- Jesse Hall, Google
- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Cass Everitt, Oculus
- Contributors to `VK_KHR_external_semaphore_win32`

Description

An application using external memory may wish to synchronize access to that memory using fences. This extension enables an application to export fence payload to and import fence payload from Windows handles.

New Commands

- vkGetFenceWin32HandleKHR
- vkImportFenceWin32HandleKHR

New Structures

- VkFenceGetWin32HandleInfoKHR
- VkImportFenceWin32HandleInfoKHR
- Extending VkFenceCreateInfo:

- VkExportFenceWin32HandleInfoKHR

New Enum Constants

- VK_KHR_EXTERNAL_FENCE_WIN32_EXTENSION_NAME
- VK_KHR_EXTERNAL_FENCE_WIN32_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_EXPORT_FENCE_WIN32_HANDLE_INFO_KHR
  - VK_STRUCTURE_TYPE_FENCE_GET_WIN32_HANDLE_INFO_KHR
  - VK_STRUCTURE_TYPE_IMPORT_FENCE_WIN32_HANDLE_INFO_KHR

Issues

This extension borrows concepts, semantics, and language from VK_KHR_external_semaphore_win32. That extension's issues apply equally to this extension.

1) Should D3D12 fence handle types be supported, like they are for semaphores?

RESOLVED: No. Doing so would require extending the fence signal and wait operations to provide values to signal / wait for, like VkD3D12FenceSubmitInfoKHR does. A D3D12 fence can be signaled by importing it into a VkSemaphore instead of a VkFence, and applications can check status or wait on the D3D12 fence using non-Vulkan APIs. The convenience of being able to do these operations on VkFence objects does not justify the extra API complexity.

Version History

- Revision 1, 2017-05-08 (Jesse Hall)
  - Initial revision

VK_KHR_external_memory

Name String

VK_KHR_external_memory

Extension Type

Device extension

Registered Extension Number

73

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_memory_capabilities
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-10-20

IP Status

No known IP claims.

Interactions and External Dependencies

- Interacts with `VK_KHR_dedicated_allocation`.
- Interacts with `VK_NV_dedicated_allocation`.

Contributors

- Faith Ekstrand, Intel
- Ian Elliott, Google
- Jesse Hall, Google
- Tobias Hector, Imagination Technologies
- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Matthew Netsch, Qualcomm Technologies, Inc.
- Daniel Rakos, AMD
- Carsten Rohde, NVIDIA
- Ray Smith, ARM
- Lina Versace, Google

Description

An application may wish to reference device memory in multiple Vulkan logical devices or instances, in multiple processes, and/or in multiple APIs. This extension enables an application to export non-Vulkan handles from Vulkan memory objects such that the underlying resources can be referenced outside the scope of the Vulkan logical device that created them.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkBufferCreateInfo:
  - VkExternalMemoryBufferCreateInfoKHR
- Extending VkImageCreateInfo:
  - VkExternalMemoryImageCreateInfoKHR
- Extending VkMemoryAllocateInfo:
  - VkExportMemoryAllocateInfoKHR

New Enum Constants

- `VK_KHR_EXTERNAL_MEMORY_EXTENSION_NAME`
- `VK_KHR_EXTERNAL_MEMORY_SPEC_VERSION`
- `VK_QUEUE_FAMILY_EXTERNAL_KHR`
- Extending VkResult:
  - `VK_ERROR_INVALID_EXTERNAL_HANDLE_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_EXPORT_MEMORY_ALLOCATE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_BUFFER_CREATE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_IMAGE_CREATE_INFO_KHR`

Issues

1) How do applications correlate two physical devices across process or Vulkan instance boundaries?

RESOLVED: New device ID fields have been introduced by `VK_KHR_external_memory_capabilities`. These fields, combined with the existing VkPhysicalDeviceProperties::`driverVersion` field can be used to identify compatible devices across processes, drivers, and APIs. VkPhysicalDeviceProperties::`pipelineCacheUUID` is not sufficient for this purpose because despite its description in the specification, it need only identify a unique pipeline cache format in practice. Multiple devices may be able to use the same pipeline cache data, and hence it would be desirable for all of them to have the same pipeline cache UUID. However, only the same concrete physical device can be used when sharing memory, so an actual unique device ID was introduced. Further, the pipeline cache UUID was specific to Vulkan, but correlation with other, non-extensible APIs is required to enable interoperation with those APIs.

2) If memory objects are shared between processes and APIs, is this considered aliasing according to the rules outlined in the Memory Aliasing section?

RESOLVED: Yes. Applications must take care to obey all restrictions imposed on aliased resources when using memory across multiple Vulkan instances or other APIs.

3) Are new image layouts or metadata required to specify image layouts and layout transitions compatible with non-Vulkan APIs, or with other instances of the same Vulkan driver?

RESOLVED: Separate instances of the same Vulkan driver running on the same GPU should have identical internal layout semantics, so applications have the tools they need to ensure views of images are consistent between the two instances. Other APIs will fall into two categories: Those that are Vulkan- compatible, and those that are Vulkan-incompatible. Vulkan-incompatible APIs will require the image to be in the GENERAL layout whenever they are accessing them.

Note this does not attempt to address cross-device transitions, nor transitions to engines on the same device which are not visible within the Vulkan API. Both of these are beyond the scope of this extension.

4) Is a new barrier flag or operation of some type needed to prepare external memory for handoff to another Vulkan instance or API and/or receive it from another instance or API?

RESOLVED: Yes. Some implementations need to perform additional cache management when transitioning memory between address spaces and other APIs, instances, or processes which may operate in a separate address space. Options for defining this transition include:

- A new structure that can be added to the `pNext` list in VkMemoryBarrier, VkBufferMemoryBarrier, and VkImageMemoryBarrier.
- A new bit in VkAccessFlags that can be set to indicate an “external” access.
- A new bit in VkDependencyFlags
- A new special queue family that represents an “external” queue.

A new structure has the advantage that the type of external transition can be described in as much detail as necessary. However, there is not currently a known need for anything beyond differentiating between external and internal accesses, so this is likely an over-engineered solution. The access flag bit has the advantage that it can be applied at buffer, image, or global granularity, and semantically it maps pretty well to the operation being described. Additionally, the API already includes `VK_ACCESS_MEMORY_READ_BIT` and `VK_ACCESS_MEMORY_WRITE_BIT` which appear to be intended for this purpose. However, there is no obvious pipeline stage that would correspond to an external access, and therefore no clear way to use `VK_ACCESS_MEMORY_READ_BIT` or `VK_ACCESS_MEMORY_WRITE_BIT`. VkDependencyFlags and VkPipelineStageFlags operate at command granularity rather than image or buffer granularity, which would make an entire pipeline barrier an internal→external or external→internal barrier. This may not be a problem in practice, but seems like the wrong scope. Another downside of VkDependencyFlags is that it lacks inherent directionality: there are no `src` and `dst` variants of it in the barrier or dependency description semantics, so two bits might need to be added to describe both internal→external and external→internal transitions. Transitioning a resource to a special queue family corresponds well with the operation of transitioning to a separate Vulkan instance, in that both operations ideally include scheduling a barrier on both sides of the transition: Both the releasing and the acquiring queue or process. Using a special queue family requires adding an additional reserved queue family index. Re-using `VK_QUEUE_FAMILY_IGNORED` would have left it unclear how to transition a concurrent usage resource

from one process to another, since the semantics would have likely been equivalent to the currently-ignored transition of `VK_QUEUE_FAMILY_IGNORED` → `VK_QUEUE_FAMILY_IGNORED`. Fortunately, creating a new reserved queue family index is not invasive.

Based on the above analysis, the approach of transitioning to a special "external" queue family was chosen.

5) Do internal driver memory arrangements and/or other internal driver image properties need to be exported and imported when sharing images across processes or APIs.

RESOLVED: Some vendors claim this is necessary on their implementations, but it was determined that the security risks of allowing opaque metadata to be passed from applications to the driver were too high. Therefore, implementations which require metadata will need to associate it with the objects represented by the external handles, and rely on the dedicated allocation mechanism to associate the exported and imported memory objects with a single image or buffer.

6) Most prior interoperation and cross-process sharing APIs have been based on image-level sharing. Should Vulkan sharing be based on memory-object sharing or image sharing?

RESOLVED: These extensions have assumed memory-level sharing is the correct granularity. Vulkan is a lower-level API than most prior APIs, and as such attempts to closely align with to the underlying primitives of the hardware and system-level drivers it abstracts. In general, the resource that holds the backing store for both images and buffers of various types is memory. Images and buffers are merely metadata containing brief descriptions of the layout of bits within that memory.

Because memory object-based sharing is aligned with the overall Vulkan API design, it enables the full range of Vulkan capabilities with external objects. External memory can be used as backing for sparse images, for example, whereas such usage would be awkward at best with a sharing mechanism based on higher-level primitives such as images. Further, aligning the mechanism with the API in this way provides some hope of trivial compatibility with future API enhancements. If new objects backed by memory objects are added to the API, they too can be used across processes with minimal additions to the base external memory APIs.

Earlier APIs implemented interop at a higher level, and this necessitated entirely separate sharing APIs for images and buffers. To co-exist and interoperate with those APIs, the Vulkan external sharing mechanism must accommodate their model. However, if it can be agreed that memory-based sharing is the more desirable and forward-looking design, legacy interoperation constraints can be considered another reason to favor memory-based sharing: while native and legacy driver primitives that may be used to implement sharing may not be as low-level as the API here suggests, raw memory is still the least common denominator among the types. Image-based sharing can be cleanly derived from a set of base memory- object sharing APIs with minimal effort, whereas image-based sharing does not generalize well to buffer or raw-memory sharing. Therefore, following the general Vulkan design principle of minimalism, it is better to expose interopability with image-based native and external primitives via the memory sharing API, and place sufficient limits on their usage to ensure they can be used only as backing for equivalent Vulkan images. This provides a consistent API for applications regardless of which platform or external API they are targeting, which makes development of multi-API and multi-platform applications simpler.

7) Should Vulkan define a common external handle type and provide Vulkan functions to facilitate cross-process sharing of such handles rather than relying on native handles to define the external objects?

RESOLVED: No. Cross-process sharing of resources is best left to native platforms. There are myriad security and extensibility issues with such a mechanism, and attempting to re-solve all those issues within Vulkan does not align with Vulkan's purpose as a graphics API. If desired, such a mechanism could be built as a layer or helper library on top of the opaque native handle defined in this family of extensions.

8) Must implementations provide additional guarantees about state implicitly included in memory objects for those memory objects that may be exported?

RESOLVED: Implementations must ensure that sharing memory objects does not transfer any information between the exporting and importing instances and APIs other than that required to share the data contained in the memory objects explicitly shared. As specific examples, data from previously freed memory objects that used the same underlying physical memory, and data from memory objects using adjacent physical memory must not be visible to applications importing an exported memory object.

9) Must implementations validate external handles the application provides as inputs to memory import operations?

RESOLVED: Implementations must return an error to the application if the provided memory handle cannot be used to complete the requested import operation. However, implementations need not validate handles are of the exact type specified by the application.

Version History

- Revision 1, 2016-10-20 (James Jones)
  - Initial version

VK_KHR_external_memory_capabilities

Name String

`VK_KHR_external_memory_capabilities`

Extension Type

Instance extension

Registered Extension Number

72

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-10-17

IP Status

No known IP claims.

Interactions and External Dependencies

- Interacts with `VK_KHR_dedicated_allocation`.
- Interacts with `VK_NV_dedicated_allocation`.

Contributors

- Ian Elliott, Google
- Jesse Hall, Google
- James Jones, NVIDIA

Description

An application may wish to reference device memory in multiple Vulkan logical devices or instances, in multiple processes, and/or in multiple APIs. This extension provides a set of capability queries and handle definitions that allow an application to determine what types of “external” memory handles an implementation supports for a given set of use cases.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkGetPhysicalDeviceExternalBufferPropertiesKHR

New Structures

- VkExternalBufferPropertiesKHR
- VkExternalMemoryPropertiesKHR

- VkPhysicalDeviceExternalBufferInfoKHR
- Extending VkImageFormatProperties2:
  - VkExternalImageFormatPropertiesKHR
- Extending VkPhysicalDeviceImageFormatInfo2:
  - VkPhysicalDeviceExternalImageFormatInfoKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceIDPropertiesKHR

New Enums

- VkExternalMemoryFeatureFlagBitsKHR
- VkExternalMemoryHandleTypeFlagBitsKHR

New Bitmasks

- VkExternalMemoryFeatureFlagsKHR
- VkExternalMemoryHandleTypeFlagsKHR

New Enum Constants

- `VK_KHR_EXTERNAL_MEMORY_CAPABILITIES_EXTENSION_NAME`
- `VK_KHR_EXTERNAL_MEMORY_CAPABILITIES_SPEC_VERSION`
- `VK_LUID_SIZE_KHR`
- Extending VkExternalMemoryFeatureFlagBits:
  - `VK_EXTERNAL_MEMORY_FEATURE_DEDICATED_ONLY_BIT_KHR`
  - `VK_EXTERNAL_MEMORY_FEATURE_EXPORTABLE_BIT_KHR`
  - `VK_EXTERNAL_MEMORY_FEATURE_IMPORTABLE_BIT_KHR`
- Extending VkExternalMemoryHandleTypeFlagBits:
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_BIT_KHR`
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D11_TEXTURE_KMT_BIT_KHR`
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_HEAP_BIT_KHR`
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_D3D12_RESOURCE_BIT_KHR`
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT_KHR`
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR`
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_EXTERNAL_BUFFER_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_EXTERNAL_IMAGE_FORMAT_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_BUFFER_INFO_KHR`

- VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_IMAGE_FORMAT_INFO_KHR
- VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ID_PROPERTIES_KHR

Issues

1) Why do so many external memory capabilities need to be queried on a per-memory-handle-type basis?

PROPOSED RESOLUTION: This is because some handle types are based on OS-native objects that have far more limited capabilities than the very generic Vulkan memory objects. Not all memory handle types can name memory objects that support 3D images, for example. Some handle types cannot even support the deferred image and memory binding behavior of Vulkan and require specifying the image when allocating or importing the memory object.

2) Do the VkExternalImageFormatPropertiesKHR and VkExternalBufferPropertiesKHR structs need to include a list of memory type bits that support the given handle type?

PROPOSED RESOLUTION: No. The memory types that do not support the handle types will simply be filtered out of the results returned by vkGetImageMemoryRequirements and vkGetBufferMemoryRequirements when a set of handle types was specified at image or buffer creation time.

3) Should the non-opaque handle types be moved to their own extension?

PROPOSED RESOLUTION: Perhaps. However, defining the handle type bits does very little and does not require any platform-specific types on its own, and it is easier to maintain the bitfield values in a single extension for now. Presumably more handle types could be added by separate extensions though, and it would be midly weird to have some platform-specific ones defined in the core spec and some in extensions

4) Do we need a D3D11_TILEPOOL type?

PROPOSED RESOLUTION: No. This is technically possible, but the synchronization is awkward. D3D11 surfaces must be synchronized using shared mutexes, and these synchronization primitives are shared by the entire memory object, so D3D11 shared allocations divided among multiple buffer and image bindings may be difficult to synchronize.

5) Should the Windows 7-compatible handle types be named “KMT” handles or “GLOBAL_SHARE” handles?

PROPOSED RESOLUTION: KMT, simply because it is more concise.

6) How do applications identify compatible devices and drivers across instance, process, and API boundaries when sharing memory?

PROPOSED RESOLUTION: New device properties are exposed that allow applications to correctly correlate devices and drivers. A device and driver UUID that must both match to ensure sharing compatibility between two Vulkan instances, or a Vulkan instance and an extensible external API are added. To allow correlating with Direct3D devices, a device LUID is added that corresponds to a DXGI adapter LUID. A driver ID is not needed for Direct3D because mismatched driver component

versions are not currently supported on the Windows OS. Should support for such configurations be introduced at the OS level, further Vulkan extensions would be needed to correlate userspace component builds.

Version History

- Revision 1, 2016-10-17 (James Jones)
  - Initial version

VK_KHR_external_memory_fd

Name String

`VK_KHR_external_memory_fd`

Extension Type

Device extension

Registered Extension Number

75

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_memory
or
Vulkan Version 1.1

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-10-21

IP Status

No known IP claims.

Contributors

- James Jones, NVIDIA
- Jeff Juliano, NVIDIA

Description

An application may wish to reference device memory in multiple Vulkan logical devices or instances, in multiple processes, and/or in multiple APIs. This extension enables an application to export POSIX file descriptor handles from Vulkan memory objects and to import Vulkan memory objects from POSIX file descriptor handles exported from other Vulkan memory objects or from similar resources in other APIs.

New Commands

- vkGetMemoryFdKHR
- vkGetMemoryFdPropertiesKHR

New Structures

- VkMemoryFdPropertiesKHR
- VkMemoryGetFdInfoKHR
- Extending VkMemoryAllocateInfo:
  - VkImportMemoryFdInfoKHR

New Enum Constants

- `VK_KHR_EXTERNAL_MEMORY_FD_EXTENSION_NAME`
- `VK_KHR_EXTERNAL_MEMORY_FD_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_IMPORT_MEMORY_FD_INFO_KHR`
  - `VK_STRUCTURE_TYPE_MEMORY_FD_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_MEMORY_GET_FD_INFO_KHR`

Issues

1) Does the application need to close the file descriptor returned by vkGetMemoryFdKHR?

RESOLVED: Yes, unless it is passed back in to a driver instance to import the memory. A successful get call transfers ownership of the file descriptor to the application, and a successful import transfers it back to the driver. Destroying the original memory object will not close the file descriptor or remove its reference to the underlying memory resource associated with it.

2) Do drivers ever need to expose multiple file descriptors per memory object?

RESOLVED: No. This would indicate there are actually multiple memory objects, rather than a single memory object.

3) How should the valid size and memory type for POSIX file descriptor memory handles created outside of Vulkan be specified?

RESOLVED: The valid memory types are queried directly from the external handle. The size will be

specified by future extensions that introduce such external memory handle types.

Version History

- Revision 1, 2016-10-21 (James Jones)
  - Initial revision

VK_KHR_external_memory_win32

Name String

`VK_KHR_external_memory_win32`

Extension Type

Device extension

Registered Extension Number

74

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_memory
or
Vulkan Version 1.1

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-10-21

IP Status

No known IP claims.

Contributors

- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Carsten Rohde, NVIDIA

Description

An application may wish to reference device memory in multiple Vulkan logical devices or instances, in multiple processes, and/or in multiple APIs. This extension enables an application to export Windows handles from Vulkan memory objects and to import Vulkan memory objects from Windows handles exported from other Vulkan memory objects or from similar resources in other APIs.

New Commands

- vkGetMemoryWin32HandleKHR
- vkGetMemoryWin32HandlePropertiesKHR

New Structures

- VkMemoryGetWin32HandleInfoKHR
- VkMemoryWin32HandlePropertiesKHR
- Extending VkMemoryAllocateInfo:
  - VkExportMemoryWin32HandleInfoKHR
  - VkImportMemoryWin32HandleInfoKHR

New Enum Constants

- `VK_KHR_EXTERNAL_MEMORY_WIN32_EXTENSION_NAME`
- `VK_KHR_EXTERNAL_MEMORY_WIN32_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_EXPORT_MEMORY_WIN32_HANDLE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_IMPORT_MEMORY_WIN32_HANDLE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_MEMORY_GET_WIN32_HANDLE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_MEMORY_WIN32_HANDLE_PROPERTIES_KHR`

Issues

1) Do applications need to call `CloseHandle`() on the values returned from vkGetMemoryWin32HandleKHR when `handleType` is `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR`?

RESOLVED: Yes. A successful get call transfers ownership of the handle to the application. Destroying the memory object will not destroy the handle or the handle's reference to the underlying memory resource. Unlike file descriptor opaque handles, win32 opaque handle ownership can not be transferred back to a driver by an import operation.

2) Should the language regarding KMT/Windows 7 handles be moved to a separate extension so that it can be deprecated over time?

RESOLVED: No. Support for them can be deprecated by drivers if they choose, by no longer

returning them in the supported handle types of the instance level queries.

3) How should the valid size and memory type for windows memory handles created outside of Vulkan be specified?

RESOLVED: The valid memory types are queried directly from the external handle. The size is determined by the associated image or buffer memory requirements for external handle types that require dedicated allocations, and by the size specified when creating the object from which the handle was exported for other external handle types.

Version History

- Revision 1, 2016-10-21 (James Jones)
  - Initial revision

VK_KHR_external_semaphore

Name String

`VK_KHR_external_semaphore`

Extension Type

Device extension

Registered Extension Number

78

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_semaphore_capabilities

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-10-21

IP Status

No known IP claims.

Contributors

- Faith Ekstrand, Intel
- Jesse Hall, Google
- Tobias Hector, Imagination Technologies
- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Matthew Netsch, Qualcomm Technologies, Inc.
- Ray Smith, ARM
- Lina Versace, Google

Description

An application using external memory may wish to synchronize access to that memory using semaphores. This extension enables an application to create semaphores from which non-Vulkan handles that reference the underlying synchronization primitive can be exported.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkSemaphoreCreateInfo:
  - VkExportSemaphoreCreateInfoKHR

New Enums

- VkSemaphoreImportFlagBitsKHR

New Bitmasks

- VkSemaphoreImportFlagsKHR

New Enum Constants

- `VK_KHR_EXTERNAL_SEMAPHORE_EXTENSION_NAME`
- `VK_KHR_EXTERNAL_SEMAPHORE_SPEC_VERSION`
- Extending VkSemaphoreImportFlagBits:
  - `VK_SEMAPHORE_IMPORT_TEMPORARY_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_CREATE_INFO_KHR`

Issues

1) Should there be restrictions on what side effects can occur when waiting on imported semaphores that are in an invalid state?

RESOLVED: Yes. Normally, validating such state would be the responsibility of the application, and the implementation would be free to enter an undefined state if valid usage rules were violated. However, this could cause security concerns when using imported semaphores, as it would require the importing application to trust the exporting application to ensure the state is valid. Requiring this level of trust is undesirable for many potential use cases.

2) Must implementations validate external handles the application provides as input to semaphore state import operations?

RESOLVED: Implementations must return an error to the application if the provided semaphore state handle cannot be used to complete the requested import operation. However, implementations need not validate handles are of the exact type specified by the application.

Version History

- Revision 1, 2016-10-21 (James Jones)
  - Initial revision

VK_KHR_external_semaphore_capabilities

Name String

`VK_KHR_external_semaphore_capabilities`

Extension Type

Instance extension

Registered Extension Number

77

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-10-20

IP Status

No known IP claims.

Contributors

- Jesse Hall, Google
- James Jones, NVIDIA
- Jeff Juliano, NVIDIA

Description

An application may wish to reference device semaphores in multiple Vulkan logical devices or instances, in multiple processes, and/or in multiple APIs. This extension provides a set of capability queries and handle definitions that allow an application to determine what types of "external" semaphore handles an implementation supports for a given set of use cases.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkGetPhysicalDeviceExternalSemaphorePropertiesKHR

New Structures

- VkExternalSemaphorePropertiesKHR
- VkPhysicalDeviceExternalSemaphoreInfoKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceIDPropertiesKHR

New Enums

- VkExternalSemaphoreFeatureFlagBitsKHR
- VkExternalSemaphoreHandleTypeFlagBitsKHR

New Bitmasks

- VkExternalSemaphoreFeatureFlagsKHR

- VkExternalSemaphoreHandleTypeFlagsKHR

New Enum Constants

- VK_KHR_EXTERNAL_SEMAPHORE_CAPABILITIES_EXTENSION_NAME
- VK_KHR_EXTERNAL_SEMAPHORE_CAPABILITIES_SPEC_VERSION
- VK_LUID_SIZE_KHR
- Extending VkExternalSemaphoreFeatureFlagBits:
  - VK_EXTERNAL_SEMAPHORE_FEATURE_EXPORTABLE_BIT_KHR
  - VK_EXTERNAL_SEMAPHORE_FEATURE_IMPORTABLE_BIT_KHR
- Extending VkExternalSemaphoreHandleTypeFlagBits:
  - VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_D3D12_FENCE_BIT_KHR
  - VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT_KHR
  - VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR
  - VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT_KHR
  - VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_SYNC_FD_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_EXTERNAL_SEMAPHORE_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_SEMAPHORE_INFO_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ID_PROPERTIES_KHR

Version History

- Revision 1, 2016-10-20 (James Jones)
  - Initial revision

VK_KHR_external_semaphore_fd

Name String

VK_KHR_external_semaphore_fd

Extension Type

Device extension

Registered Extension Number

80

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_semaphore
or
Vulkan Version 1.1

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-10-21

IP Status

No known IP claims.

Contributors

- Jesse Hall, Google
- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Carsten Rohde, NVIDIA

Description

An application using external memory may wish to synchronize access to that memory using semaphores. This extension enables an application to export semaphore payload to and import semaphore payload from POSIX file descriptors.

New Commands

- vkGetSemaphoreFdKHR
- vkImportSemaphoreFdKHR

New Structures

- VkImportSemaphoreFdInfoKHR
- VkSemaphoreGetFdInfoKHR

New Enum Constants

- `VK_KHR_EXTERNAL_SEMAPHORE_FD_EXTENSION_NAME`
- `VK_KHR_EXTERNAL_SEMAPHORE_FD_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_IMPORT_SEMAPHORE_FD_INFO_KHR`
  - `VK_STRUCTURE_TYPE_SEMAPHORE_GET_FD_INFO_KHR`

Issues

1) Does the application need to close the file descriptor returned by vkGetSemaphoreFdKHR?

RESOLVED: Yes, unless it is passed back in to a driver instance to import the semaphore. A successful get call transfers ownership of the file descriptor to the application, and a successful import transfers it back to the driver. Destroying the original semaphore object will not close the file descriptor or remove its reference to the underlying semaphore resource associated with it.

Version History

- Revision 1, 2016-10-21 (Jesse Hall)
  - Initial revision

VK_KHR_external_semaphore_win32

Name String

`VK_KHR_external_semaphore_win32`

Extension Type

Device extension

Registered Extension Number

79

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_semaphore

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-10-21

IP Status

No known IP claims.

Contributors

- James Jones, NVIDIA
- Jeff Juliano, NVIDIA

- Carsten Rohde, NVIDIA

Description

An application using external memory may wish to synchronize access to that memory using semaphores. This extension enables an application to export semaphore payload to and import semaphore payload from Windows handles.

New Commands

- vkGetSemaphoreWin32HandleKHR
- vkImportSemaphoreWin32HandleKHR

New Structures

- VkImportSemaphoreWin32HandleInfoKHR
- VkSemaphoreGetWin32HandleInfoKHR
- Extending VkSemaphoreCreateInfo:
  - VkExportSemaphoreWin32HandleInfoKHR
- Extending VkSubmitInfo:
  - VkD3D12FenceSubmitInfoKHR

New Enum Constants

- `VK_KHR_EXTERNAL_SEMAPHORE_WIN32_EXTENSION_NAME`
- `VK_KHR_EXTERNAL_SEMAPHORE_WIN32_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_D3D12_FENCE_SUBMIT_INFO_KHR`
  - `VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_WIN32_HANDLE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_IMPORT_SEMAPHORE_WIN32_HANDLE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_SEMAPHORE_GET_WIN32_HANDLE_INFO_KHR`

Issues

1) Do applications need to call `CloseHandle`() on the values returned from vkGetSemaphoreWin32HandleKHR when `handleType` is `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR`?

RESOLVED: Yes. A successful get call transfers ownership of the handle to the application. Destroying the semaphore object will not destroy the handle or the handle's reference to the underlying semaphore resource. Unlike file descriptor opaque handles, win32 opaque handle ownership can not be transferred back to a driver by an import operation.

2) Should the language regarding KMT/Windows 7 handles be moved to a separate extension so that it can be deprecated over time?

RESOLVED: No. Support for them can be deprecated by drivers if they choose, by no longer returning them in the supported handle types of the instance level queries.

3) Should applications be allowed to specify additional object attributes for shared handles?

RESOLVED: Yes. Applications will be allowed to provide similar attributes to those they would to any other handle creation API.

4) How do applications communicate the desired fence values to use with `D3D12_FENCE`-based Vulkan semaphores?

RESOLVED: There are a couple of options. The values for the signaled and reset states could be communicated up front when creating the object and remain static for the life of the Vulkan semaphore, or they could be specified using auxiliary structures when submitting semaphore signal and wait operations, similar to what is done with the keyed mutex extensions. The latter is more flexible and consistent with the keyed mutex usage, but the former is a much simpler API.

Since Vulkan tends to favor flexibility and consistency over simplicity, a new structure specifying D3D12 fence acquire and release values is added to the vkQueueSubmit function.

Version History

- Revision 1, 2016-10-21 (James Jones)
  - Initial revision

VK_KHR_format_feature_flags2

Name String

`VK_KHR_format_feature_flags2`

Extension Type

Device extension

Registered Extension Number

361

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Lionel Landwerlin [GitHub]llandwerlin

Other Extension Metadata

Last Modified Date

2021-07-01

IP Status

No known IP claims.

Contributors

- Lionel Landwerlin, Intel
- Faith Ekstrand, Intel
- Tobias Hector, AMD
- Spencer Fricke, Samsung Electronics
- Graeme Leese, Broadcom
- Jan-Harald Fredriksen, ARM

Description

This extension adds a new VkFormatFeatureFlagBits2KHR 64bits format feature flag type to extend the existing VkFormatFeatureFlagBits which is limited to 31 flags. At the time of this writing 29 bits of VkFormatFeatureFlagBits are already used.

Because VkFormatProperties2 is already defined to extend the Vulkan 1.0 vkGetPhysicalDeviceFormatProperties command, this extension defines a new VkFormatProperties3KHR to extend the VkFormatProperties.

On top of replicating all the bits from VkFormatFeatureFlagBits, VkFormatFeatureFlagBits2KHR adds the following bits :

- VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT_KHR and VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT_KHR specify that an implementation supports reading and writing, respectively, a given VkFormat through storage operations without specifying the format in the shader.
- VK_FORMAT_FEATURE_2_SAMPLED_IMAGE_DEPTH_COMPARISON_BIT_KHR specifies that an implementation supports depth comparison performed by OpImage*Dref* instructions on a given VkFormat. Previously the result of executing a OpImage*Dref* instruction on an image view, where the format was not one of the depth/stencil formats with a depth component, was undefined. This bit clarifies on which formats such instructions can be used.

Prior to version 2 of this extension, implementations exposing the shaderStorageImageReadWithoutFormat and shaderStorageImageWriteWithoutFormat features may not report VK_FORMAT_FEATURE_2_STORAGE_READ_WITHOUT_FORMAT_BIT_KHR and VK_FORMAT_FEATURE_2_STORAGE_WRITE_WITHOUT_FORMAT_BIT_KHR in VkFormatProperties3KHR ::bufferFeatures. Despite this, buffer reads/writes are supported as intended by the original

features.

New Structures

- Extending VkFormatProperties2:
  - VkFormatProperties3KHR

New Enums

- VkFormatFeatureFlagBits2KHR

New Bitmasks

- VkFormatFeatureFlags2KHR

New Enum Constants

- `VK_KHR_FORMAT_FEATURE_FLAGS_2_EXTENSION_NAME`
- `VK_KHR_FORMAT_FEATURE_FLAGS_2_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_3_KHR`

Promotion to Vulkan 1.3

Vulkan APIs in this extension are included in core Vulkan 1.3, with the KHR suffix omitted. External interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

Version History

- Revision 2, 2022-07-20 (Lionel Landwerlin)
  - Clarify that VK_FORMAT_FEATURE_2_STORAGE_(READ|WRITE)_WITHOUT_FORMAT_BIT also apply to buffer views.
- Revision 1, 2020-07-21 (Lionel Landwerlin)
  - Initial draft

VK_KHR_fragment_shader_barycentric

Name String

`VK_KHR_fragment_shader_barycentric`

Extension Type

Device extension

Registered Extension Number

323

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_fragment_shader_barycentric

Contact

- Stu Smith

Extension Proposal

VK_KHR_fragment_shader_barycentric

Other Extension Metadata

Last Modified Date

2022-03-10

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_fragment_shader_barycentric`

Contributors

- Stu Smith, AMD
- Tobias Hector, AMD
- Graeme Leese, Broadcom
- Jan-Harald Fredriksen, Arm
- Slawek Grajewski, Intel
- Pat Brown, NVIDIA
- Hans-Kristian Arntzen, Valve
- Contributors to the VK_NV_fragment_shader_barycentric specification

Description

This extension is based on the `VK_NV_fragment_shader_barycentric` extension, and adds support for the following SPIR-V extension in Vulkan:

- `SPV_KHR_fragment_shader_barycentric`

The extension provides access to three additional fragment shader variable decorations in SPIR-V:

- `PerVertexKHR`, which indicates that a fragment shader input will not have interpolated values, but instead must be accessed with an extra array index that identifies one of the vertices of the primitive producing the fragment
- `BaryCoordKHR`, which indicates that the variable is a three-component floating-point vector holding barycentric weights for the fragment produced using perspective interpolation
- `BaryCoordNoPerspKHR`, which indicates that the variable is a three-component floating-point vector holding barycentric weights for the fragment produced using linear interpolation

When using GLSL source-based shader languages, the following variables from `GL_EXT_fragment_shader_barycentric` map to these SPIR-V built-in decorations:

- `in vec3 gl_BaryCoordEXT;` → `BaryCoordKHR`
- `in vec3 gl_BaryCoordNoPerspEXT;` → `BaryCoordNoPerspKHR`

GLSL variables declared using the `pervertexEXT` GLSL qualifier are expected to be decorated with `PerVertexKHR` in SPIR-V.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceFragmentShaderBarycentricFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceFragmentShaderBarycentricPropertiesKHR

New Enum Constants

- `VK_KHR_FRAGMENT_SHADER_BARYCENTRIC_EXTENSION_NAME`
- `VK_KHR_FRAGMENT_SHADER_BARYCENTRIC_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_BARYCENTRIC_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_BARYCENTRIC_PROPERTIES_KHR`

New Built-In Variables

- `BaryCoordKHR`
- `BaryCoordNoPerspKHR`

New SPIR-V Decorations

- `PerVertexKHR`

New SPIR-V Capabilities

- `FragmentBarycentricKHR`

Issues

1) What are the interactions with MSAA and how are `BaryCoordKHR` and `BaryCoordNoPerspKHR` interpolated?

RESOLVED: The inputs decorated with `BaryCoordKHR` or `BaryCoordNoPerspKHR` **may** also be decorated with the `Centroid` or `Sample` qualifiers to specify interpolation, like any other fragment shader input. If the `shaderSampleRateInterpolationFunctions` feature is enabled, the extended instructions InterpolateAtCentroid, InterpolateAtOffset, and InterpolateAtSample from the GLSL.std.450 **may** also be used with inputs decorated with `BaryCoordKHR` or `BaryCoordNoPerspKHR`.

Version History

- Revision 1, 2022-03-10 (Stu Smith)
  - Initial revision

VK_KHR_fragment_shading_rate

Name String

`VK_KHR_fragment_shading_rate`

Extension Type

Device extension

Registered Extension Number

227

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
VK_KHR_create_renderpass2
or
Vulkan Version 1.2

API Interactions

- Interacts with VK_VERSION_1_3
- Interacts with VK_KHR_dynamic_rendering
- Interacts with VK_KHR_format_feature_flags2

SPIR-V Dependencies

- SPV_KHR_fragment_shading_rate

Contact

- Tobias Hector [GitHub]tobski

Extension Proposal

VK_KHR_fragment_shading_rate

Other Extension Metadata

Last Modified Date

2021-09-30

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_fragment_shading_rate`

Contributors

- Tobias Hector, AMD
- Guennadi Riguer, AMD
- Matthaeus Chajdas, AMD
- Pat Brown, Nvidia
- Matthew Netsch, Qualcomm
- Slawomir Grajewski, Intel
- Jan-Harald Fredriksen, Arm
- Jeff Bolz, Nvidia
- Arseny Kapoulkine, Roblox
- Contributors to the VK_NV_shading_rate_image specification
- Contributors to the VK_EXT_fragment_density_map specification

Description

This extension adds the ability to change the rate at which fragments are shaded. Rather than the usual single fragment invocation for each pixel covered by a primitive, multiple pixels can be shaded by a single fragment shader invocation.

Up to three methods are available to the application to change the fragment shading rate:

- Pipeline Fragment Shading Rate, which allows the specification of a rate per-draw.
- Primitive Fragment Shading Rate, which allows the specification of a rate per primitive, specified during shading.
- Attachment Fragment Shading Rate, which allows the specification of a rate per-region of the framebuffer, specified in a specialized image attachment.

Additionally, these rates can all be specified and combined in order to adjust the overall detail in

the image at each point.

This functionality can be used to focus shading efforts where higher levels of detail are needed in some parts of a scene compared to others. This can be particularly useful in high resolution rendering, or for XR contexts.

This extension also adds support for the `SPV_KHR_fragment_shading_rate` extension which enables setting the primitive fragment shading rate, and allows querying the final shading rate from a fragment shader.

New Commands

- vkCmdSetFragmentShadingRateKHR
- vkGetPhysicalDeviceFragmentShadingRatesKHR

New Structures

- VkPhysicalDeviceFragmentShadingRateKHR
- Extending VkGraphicsPipelineCreateInfo:
  - VkPipelineFragmentShadingRateStateCreateInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceFragmentShadingRateFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceFragmentShadingRatePropertiesKHR
- Extending VkSubpassDescription2:
  - VkFragmentShadingRateAttachmentInfoKHR

If Vulkan Version 1.3 or VK_KHR_dynamic_rendering is supported:

- Extending VkRenderingInfo:
  - VkRenderingFragmentShadingRateAttachmentInfoKHR

New Enums

- VkFragmentShadingRateCombinerOpKHR

New Enum Constants

- `VK_KHR_FRAGMENT_SHADING_RATE_EXTENSION_NAME`
- `VK_KHR_FRAGMENT_SHADING_RATE_SPEC_VERSION`
- Extending VkAccessFlagBits:
  - `VK_ACCESS_FRAGMENT_SHADING_RATE_ATTACHMENT_READ_BIT_KHR`
- Extending VkDynamicState:
  - `VK_DYNAMIC_STATE_FRAGMENT_SHADING_RATE_KHR`

- Extending VkFormatFeatureFlagBits:
  - VK_FORMAT_FEATURE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- Extending VkImageLayout:
  - VK_IMAGE_LAYOUT_FRAGMENT_SHADING_RATE_ATTACHMENT_OPTIMAL_KHR
- Extending VkImageUsageFlagBits:
  - VK_IMAGE_USAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- Extending VkPipelineStageFlagBits:
  - VK_PIPELINE_STAGE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_FRAGMENT_SHADING_RATE_ATTACHMENT_INFO_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_FRAGMENT_SHADING_RATE_STATE_CREATE_INFO_KHR

If VK_KHR_format_feature_flags2 or Vulkan Version 1.3 is supported:

- Extending VkFormatFeatureFlagBits2:
  - VK_FORMAT_FEATURE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR

If Vulkan Version 1.3 or VK_KHR_dynamic_rendering is supported:

- Extending VkPipelineCreateFlagBits:
  - VK_PIPELINE_CREATE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
  - VK_PIPELINE_RASTERIZATION_STATE_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_INFO_KHR

Version History

- Revision 1, 2020-05-06 (Tobias Hector)
  - Initial revision
- Revision 2, 2021-09-30 (Jon Leech)
  - Add interaction with VK_KHR_format_feature_flags2 to vk.xml

VK_KHR_get_display_properties2

Name String

VK_KHR_get_display_properties2

Extension Type

Instance extension

Registered Extension Number

122

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_display

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2017-02-21

IP Status

No known IP claims.

Contributors

- Ian Elliott, Google
- James Jones, NVIDIA

Description

This extension provides new queries for device display properties and capabilities that can be easily extended by other extensions, without introducing any further queries. This extension can be considered the `VK_KHR_display` equivalent of the `VK_KHR_get_physical_device_properties2` extension.

New Commands

- vkGetDisplayModeProperties2KHR
- vkGetDisplayPlaneCapabilities2KHR
- vkGetPhysicalDeviceDisplayPlaneProperties2KHR
- vkGetPhysicalDeviceDisplayProperties2KHR

New Structures

- VkDisplayModeProperties2KHR
- VkDisplayPlaneCapabilities2KHR

- VkDisplayPlaneInfo2KHR
- VkDisplayPlaneProperties2KHR
- VkDisplayProperties2KHR

New Enum Constants

- VK_KHR_GET_DISPLAY_PROPERTIES_2_EXTENSION_NAME
- VK_KHR_GET_DISPLAY_PROPERTIES_2_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_DISPLAY_MODE_PROPERTIES_2_KHR
  - VK_STRUCTURE_TYPE_DISPLAY_PLANE_CAPABILITIES_2_KHR
  - VK_STRUCTURE_TYPE_DISPLAY_PLANE_INFO_2_KHR
  - VK_STRUCTURE_TYPE_DISPLAY_PLANE_PROPERTIES_2_KHR
  - VK_STRUCTURE_TYPE_DISPLAY_PROPERTIES_2_KHR

Issues

1) What should this extension be named?

RESOLVED: VK_KHR_get_display_properties2. Other alternatives:

- VK_KHR_display2
- One extension, combined with VK_KHR_surface_capabilites2.

2) Should extensible input structs be added for these new functions:

RESOLVED:

- vkGetPhysicalDeviceDisplayProperties2KHR: No. The only current input is a VkPhysicalDevice. Other inputs would not make sense.
- vkGetPhysicalDeviceDisplayPlaneProperties2KHR: No. The only current input is a VkPhysicalDevice. Other inputs would not make sense.
- vkGetDisplayModeProperties2KHR: No. The only current inputs are a VkPhysicalDevice and a VkDisplayModeKHR. Other inputs would not make sense.

3) Should additional display query functions be extended?

RESOLVED:

- vkGetDisplayPlaneSupportedDisplaysKHR: No. Extensions should instead extend vkGetDisplayPlaneCapabilitiesKHR().

Version History

- Revision 1, 2017-02-21 (James Jones)
  - Initial draft.

VK_KHR_get_memory_requirements2

Name String

`VK_KHR_get_memory_requirements2`

Extension Type

Device extension

Registered Extension Number

147

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Faith Ekstrand [GitHub]gfxstrand

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Contributors

- Faith Ekstrand, Intel
- Jeff Bolz, NVIDIA
- Jesse Hall, Google

Description

This extension provides new queries for memory requirements of images and buffers that can be easily extended by other extensions, without introducing any additional commands. The Vulkan 1.0 VkMemoryRequirements and VkSparseImageMemoryRequirements structures do not include `sType` and `pNext` members. This extension wraps them in new structures with these members, so an application can query a chain of memory requirements structures by constructing the chain and letting the implementation fill them in. A new command is added for each `vkGet*MemoryRequrements` command in core Vulkan 1.0.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkGetBufferMemoryRequirements2KHR
- vkGetImageMemoryRequirements2KHR
- vkGetImageSparseMemoryRequirements2KHR

New Structures

- VkBufferMemoryRequirementsInfo2KHR
- VkImageMemoryRequirementsInfo2KHR
- VkImageSparseMemoryRequirementsInfo2KHR
- VkMemoryRequirements2KHR
- VkSparseImageMemoryRequirements2KHR

New Enum Constants

- `VK_KHR_GET_MEMORY_REQUIREMENTS_2_EXTENSION_NAME`
- `VK_KHR_GET_MEMORY_REQUIREMENTS_2_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_BUFFER_MEMORY_REQUIREMENTS_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_IMAGE_MEMORY_REQUIREMENTS_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_IMAGE_SPARSE_MEMORY_REQUIREMENTS_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_MEMORY_REQUIREMENTS_2_KHR`
  - `VK_STRUCTURE_TYPE_SPARSE_IMAGE_MEMORY_REQUIREMENTS_2_KHR`

Version History

- Revision 1, 2017-03-23 (Faith Ekstrand)
  - Internal revisions

VK_KHR_get_physical_device_properties2

Name String

`VK_KHR_get_physical_device_properties2`

Extension Type

Instance extension

Registered Extension Number

60

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA
- Ian Elliott, Google

Description

This extension provides new queries for device features, device properties, and format properties that can be easily extended by other extensions, without introducing any further queries. The Vulkan 1.0 feature/limit/formatproperty structures do not include `sType`/`pNext` members. This extension wraps them in new structures with `sType`/`pNext` members, so an application can query a chain of feature/limit/formatproperty structures by constructing the chain and letting the implementation fill them in. A new command is added for each `vkGetPhysicalDevice*` command in core Vulkan 1.0. The new feature structure (and a `pNext` chain of extending structures) can also be passed in to device creation to enable features.

This extension also allows applications to use the physical-device components of device extensions before vkCreateDevice is called.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkGetPhysicalDeviceFeatures2KHR
- vkGetPhysicalDeviceFormatProperties2KHR
- vkGetPhysicalDeviceImageFormatProperties2KHR
- vkGetPhysicalDeviceMemoryProperties2KHR
- vkGetPhysicalDeviceProperties2KHR
- vkGetPhysicalDeviceQueueFamilyProperties2KHR
- vkGetPhysicalDeviceSparseImageFormatProperties2KHR

New Structures

- VkFormatProperties2KHR
- VkImageFormatProperties2KHR
- VkPhysicalDeviceImageFormatInfo2KHR
- VkPhysicalDeviceMemoryProperties2KHR
- VkPhysicalDeviceProperties2KHR
- VkPhysicalDeviceSparseImageFormatInfo2KHR
- VkQueueFamilyProperties2KHR
- VkSparseImageFormatProperties2KHR
- Extending VkDeviceCreateInfo:
  - VkPhysicalDeviceFeatures2KHR

New Enum Constants

- VK_KHR_GET_PHYSICAL_DEVICE_PROPERTIES_2_EXTENSION_NAME
- VK_KHR_GET_PHYSICAL_DEVICE_PROPERTIES_2_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_FORMAT_PROPERTIES_2_KHR
  - VK_STRUCTURE_TYPE_IMAGE_FORMAT_PROPERTIES_2_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_FORMAT_INFO_2_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PROPERTIES_2_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROPERTIES_2_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SPARSE_IMAGE_FORMAT_INFO_2_KHR
  - VK_STRUCTURE_TYPE_QUEUE_FAMILY_PROPERTIES_2_KHR
  - VK_STRUCTURE_TYPE_SPARSE_IMAGE_FORMAT_PROPERTIES_2_KHR

Examples

```
// Get features with a hypothetical future extension.
VkHypotheticalExtensionFeaturesKHR hypotheticalFeatures =
{
    .sType = VK_STRUCTURE_TYPE_HYPOTHETICAL_FEATURES_KHR,
    .pNext = NULL,
};

VkPhysicalDeviceFeatures2KHR features =
{
    .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2_KHR,
    .pNext = &hypotheticalFeatures,
};

// After this call, features and hypotheticalFeatures have been filled out.
vkGetPhysicalDeviceFeatures2KHR(physicalDevice, &features);

// Properties/limits can be chained and queried similarly.

// Enable some features:
VkHypotheticalExtensionFeaturesKHR enabledHypotheticalFeatures =
{
    .sType = VK_STRUCTURE_TYPE_HYPOTHETICAL_FEATURES_KHR,
    .pNext = NULL,
};

VkPhysicalDeviceFeatures2KHR enabledFeatures =
{
    .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2_KHR,
    .pNext = &enabledHypotheticalFeatures,
};

enabledFeatures.features.xyz = VK_TRUE;
enabledHypotheticalFeatures.abc = VK_TRUE;

VkDeviceCreateInfo deviceCreateInfo =
{
    .sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO,
    .pNext = &enabledFeatures,
    ...
    .pEnabledFeatures = NULL,
};

VkDevice device;
vkCreateDevice(physicalDevice, &deviceCreateInfo, NULL, &device);
```

Version History

- Revision 1, 2016-09-12 (Jeff Bolz)

    - Internal revisions
- Revision 2, 2016-11-02 (Ian Elliott)
    - Added ability for applications to use the physical-device components of device extensions before vkCreateDevice is called.

VK_KHR_get_surface_capabilities2

Name String

`VK_KHR_get_surface_capabilities2`

Extension Type

Instance extension

Registered Extension Number

120

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2017-02-27

IP Status

No known IP claims.

Contributors

- Ian Elliott, Google
- James Jones, NVIDIA
- Alon Or-bach, Samsung

Description

This extension provides new queries for device surface capabilities that can be easily extended by other extensions, without introducing any further queries. This extension can be considered the `VK_KHR_surface` equivalent of the `VK_KHR_get_physical_device_properties2` extension.

New Commands

- vkGetPhysicalDeviceSurfaceCapabilities2KHR
- vkGetPhysicalDeviceSurfaceFormats2KHR

New Structures

- VkPhysicalDeviceSurfaceInfo2KHR
- VkSurfaceCapabilities2KHR
- VkSurfaceFormat2KHR

New Enum Constants

- `VK_KHR_GET_SURFACE_CAPABILITIES_2_EXTENSION_NAME`
- `VK_KHR_GET_SURFACE_CAPABILITIES_2_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SURFACE_INFO_2_KHR`
  - `VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_2_KHR`
  - `VK_STRUCTURE_TYPE_SURFACE_FORMAT_2_KHR`

Issues

1) What should this extension be named?

RESOLVED: `VK_KHR_get_surface_capabilities2`. Other alternatives:

- `VK_KHR_surface2`
- One extension, combining a separate display-specific query extension.

2) Should additional WSI query functions be extended?

RESOLVED:

- vkGetPhysicalDeviceSurfaceCapabilitiesKHR: Yes. The need for this motivated the extension.
- vkGetPhysicalDeviceSurfaceSupportKHR: No. Currently only has boolean output. Extensions should instead extend vkGetPhysicalDeviceSurfaceCapabilities2KHR.
- vkGetPhysicalDeviceSurfaceFormatsKHR: Yes.
- vkGetPhysicalDeviceSurfacePresentModesKHR: No. Recent discussion concluded this introduced too much variability for applications to deal with. Extensions should instead extend vkGetPhysicalDeviceSurfaceCapabilities2KHR.
- vkGetPhysicalDeviceXlibPresentationSupportKHR: Not in this extension.
- vkGetPhysicalDeviceXcbPresentationSupportKHR: Not in this extension.
- vkGetPhysicalDeviceWaylandPresentationSupportKHR: Not in this extension.
- vkGetPhysicalDeviceWin32PresentationSupportKHR: Not in this extension.

Version History

- Revision 1, 2017-02-27 (James Jones)
  - Initial draft.

VK_KHR_global_priority

Name String

`VK_KHR_global_priority`

Extension Type

Device extension

Registered Extension Number

189

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Tobias Hector [GitHub]tobski

Other Extension Metadata

Last Modified Date

2021-10-22

Contributors

- Tobias Hector, AMD
- Contributors to `VK_EXT_global_priority`
- Contributors to `VK_EXT_global_priority_query`

Description

In Vulkan, users can specify device-scope queue priorities. In some cases it may be useful to extend this concept to a system-wide scope. This device extension allows applications to query the global queue priorities supported by a queue family, and then set a priority when creating queues. The default queue priority is `VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_EXT`.

Implementations can report which global priority levels are treated differently by the

implementation. It is intended primarily for use in system integration along with certain platform-specific priority enforcement rules.

The driver implementation will attempt to skew hardware resource allocation in favor of the higher-priority task. Therefore, higher-priority work may retain similar latency and throughput characteristics even if the system is congested with lower priority work.

The global priority level of a queue shall take precedence over the per-process queue priority (`VkDeviceQueueCreateInfo::pQueuePriorities`).

Abuse of this feature may result in starving the rest of the system from hardware resources. Therefore, the driver implementation may deny requests to acquire a priority above the default priority (`VK_QUEUE_GLOBAL_PRIORITY_MEDIUM_EXT`) if the caller does not have sufficient privileges. In this scenario `VK_ERROR_NOT_PERMITTED_EXT` is returned.

The driver implementation may fail the queue allocation request if resources required to complete the operation have been exhausted (either by the same process or a different process). In this scenario `VK_ERROR_INITIALIZATION_FAILED` is returned.

New Structures

- Extending VkDeviceQueueCreateInfo:
  - VkDeviceQueueGlobalPriorityCreateInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceGlobalPriorityQueryFeaturesKHR
- Extending VkQueueFamilyProperties2:
  - VkQueueFamilyGlobalPriorityPropertiesKHR

New Enums

- VkQueueGlobalPriorityKHR

New Enum Constants

- `VK_KHR_GLOBAL_PRIORITY_EXTENSION_NAME`
- `VK_KHR_GLOBAL_PRIORITY_SPEC_VERSION`
- `VK_MAX_GLOBAL_PRIORITY_SIZE_KHR`
- Extending VkResult:
  - `VK_ERROR_NOT_PERMITTED_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_DEVICE_QUEUE_GLOBAL_PRIORITY_CREATE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_GLOBAL_PRIORITY_QUERY_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_QUEUE_FAMILY_GLOBAL_PRIORITY_PROPERTIES_KHR`

Issues

1) Can we additionally query whether a caller is permitted to acquire a specific global queue priority in this extension?

RESOLVED: No. Whether a caller has enough privilege goes with the OS, and the Vulkan driver cannot really guarantee that the privilege will not change in between this query and the actual queue creation call.

2) If more than 1 queue using global priority is requested, is there a good way to know which queue is failing the device creation?

RESOLVED: No. There is not a good way at this moment, and it is also not quite actionable for the applications to know that because the information may not be accurate. Queue creation can fail because of runtime constraints like insufficient privilege or lack of resource, and the failure is not necessarily tied to that particular queue configuration requested.

Version History

- Revision 1, 2021-10-22 (Tobias Hector)
  - Initial draft

VK_KHR_image_format_list

Name String

`VK_KHR_image_format_list`

Extension Type

Device extension

Registered Extension Number

148

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Faith Ekstrand [GitHub]gfxstrand

Other Extension Metadata

Last Modified Date

2017-03-20

IP Status

No known IP claims.

Contributors

- Faith Ekstrand, Intel
- Jan-Harald Fredriksen, ARM
- Jeff Bolz, NVIDIA
- Jeff Leger, Qualcomm
- Neil Henning, Codeplay

Description

On some implementations, setting the `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` on image creation can cause access to that image to perform worse than an equivalent image created without `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` because the implementation does not know what view formats will be paired with the image.

This extension allows an application to provide the list of all formats that **can** be used with an image when it is created. The implementation may then be able to create a more efficient image that supports the subset of formats required by the application without having to support all formats in the format compatibility class of the image format.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkImageCreateInfo, VkSwapchainCreateInfoKHR, VkPhysicalDeviceImageFormatInfo2:
  - VkImageFormatListCreateInfoKHR

New Enum Constants

- `VK_KHR_IMAGE_FORMAT_LIST_EXTENSION_NAME`
- `VK_KHR_IMAGE_FORMAT_LIST_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_IMAGE_FORMAT_LIST_CREATE_INFO_KHR`

Version History

- Revision 1, 2017-03-20 (Faith Ekstrand)
  - Initial revision

VK_KHR_imageless_framebuffer

Name String

`VK_KHR_imageless_framebuffer`

Extension Type

Device extension

Registered Extension Number

109

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
and
VK_KHR_maintenance2
or
Vulkan Version 1.1
and
VK_KHR_image_format_list
or
Vulkan Version 1.2

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Tobias Hector [GitHub]tobias

Other Extension Metadata

Last Modified Date

2018-12-14

Contributors

- Tobias Hector
- Graham Wihlidal

Description

This extension allows framebuffers to be created without the need for creating images first, allowing more flexibility in how they are used, and avoiding the need for many of the confusing compatibility rules.

Framebuffers are now created with a small amount of additional metadata about the image views that will be used in VkFramebufferAttachmentsCreateInfoKHR, and the actual image views are provided at render pass begin time via VkRenderPassAttachmentBeginInfoKHR.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- VkFramebufferAttachmentImageInfoKHR
- Extending VkFramebufferCreateInfo:
  - VkFramebufferAttachmentsCreateInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceImagelessFramebufferFeaturesKHR
- Extending VkRenderPassBeginInfo:
  - VkRenderPassAttachmentBeginInfoKHR

New Enum Constants

- `VK_KHR_IMAGELESS_FRAMEBUFFER_EXTENSION_NAME`
- `VK_KHR_IMAGELESS_FRAMEBUFFER_SPEC_VERSION`
- Extending VkFramebufferCreateFlagBits:
  - `VK_FRAMEBUFFER_CREATE_IMAGELESS_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENTS_CREATE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_FRAMEBUFFER_ATTACHMENT_IMAGE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGELESS_FRAMEBUFFER_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_RENDER_PASS_ATTACHMENT_BEGIN_INFO_KHR`

Version History

- Revision 1, 2018-12-14 (Tobias Hector)
  - Internal revisions

VK_KHR_incremental_present

Name String

`VK_KHR_incremental_present`

Extension Type

Device extension

Registered Extension Number

85

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_swapchain

Contact

- Ian Elliott [GitHub]ianelliottus

Other Extension Metadata

Last Modified Date

2016-11-02

IP Status

No known IP claims.

Contributors

- Ian Elliott, Google
- Jesse Hall, Google
- Alon Or-bach, Samsung
- James Jones, NVIDIA
- Daniel Rakos, AMD
- Ray Smith, ARM
- Mika Isojarvi, Google
- Jeff Juliano, NVIDIA
- Jeff Bolz, NVIDIA

Description

This device extension extends vkQueuePresentKHR, from the `VK_KHR_swapchain` extension, allowing

an application to specify a list of rectangular, modified regions of each image to present. This should be used in situations where an application is only changing a small portion of the presentable images within a swapchain, since it enables the presentation engine to avoid wasting time presenting parts of the surface that have not changed.

This extension is leveraged from the `EGL_KHR_swap_buffers_with_damage` extension.

New Structures

- VkPresentRegionKHR
- VkRectLayerKHR
- Extending VkPresentInfoKHR:
  - VkPresentRegionsKHR

New Enum Constants

- `VK_KHR_INCREMENTAL_PRESENT_EXTENSION_NAME`
- `VK_KHR_INCREMENTAL_PRESENT_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PRESENT_REGIONS_KHR`

Issues

1) How should we handle steroescopic-3D swapchains? We need to add a layer for each rectangle. One approach is to create another struct containing the VkRect2D plus layer, and have VkPresentRegionsKHR point to an array of that struct. Another approach is to have two parallel arrays, `pRectangles` and `pLayers`, where `pRectangles`[i] and `pLayers`[i] must be used together. Which approach should we use, and if the array of a new structure, what should that be called?

RESOLVED: Create a new structure, which is a VkRect2D plus a layer, and will be called VkRectLayerKHR.

2) Where is the origin of the VkRectLayerKHR?

RESOLVED: The upper left corner of the presentable image(s) of the swapchain, per the definition of framebuffer coordinates.

3) Does the rectangular region, VkRectLayerKHR, specify pixels of the swapchain's image(s), or of the surface?

RESOLVED: Of the image(s). Some presentation engines may scale the pixels of a swapchain's image(s) to the size of the surface. The size of the swapchain's image(s) will be consistent, where the size of the surface may vary over time.

4) What if all of the rectangles for a given swapchain contain a width and/or height of zero?

RESOLVED: The application is indicating that no pixels changed since the last present. The presentation engine may use such a hint and not update any pixels for the swapchain. However, all

other semantics of vkQueuePresentKHR must still be honored, including waiting for semaphores to signal.

5) When the swapchain is created with VkSwapchainCreateInfoKHR::preTransform set to a value other than VK_SURFACE_TRANSFORM_IDENTITY_BIT_KHR, should the rectangular region, VkRectLayerKHR, be transformed to align with the preTransform?

RESOLVED: No. The rectangular region in VkRectLayerKHR should not be transformed. As such, it may not align with the extents of the swapchain's image(s). It is the responsibility of the presentation engine to transform the rectangular region. This matches the behavior of the Android presentation engine, which set the precedent.

Version History

- Revision 1, 2016-11-02 (Ian Elliott)
  - Internal revisions
- Revision 2, 2021-03-18 (Ian Elliott)
  - Clarified alignment of rectangles for presentation engines that support transformed swapchains.

VK_KHR_index_type_uint8

Name String

VK_KHR_index_type_uint8

Extension Type

Device extension

Registered Extension Number

534

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2023-06-06

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA

Description

This extension allows `uint8_t` indices to be used with vkCmdBindIndexBuffer.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceIndexTypeUint8FeaturesKHR

New Enum Constants

- `VK_KHR_INDEX_TYPE_UINT8_EXTENSION_NAME`
- `VK_KHR_INDEX_TYPE_UINT8_SPEC_VERSION`
- Extending VkIndexType:
  - `VK_INDEX_TYPE_UINT8_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_INDEX_TYPE_UINT8_FEATURES_KHR`

Version History

- Revision 1, 2023-06-06 (Piers Daniell)
  - Internal revisions

VK_KHR_line_rasterization

Name String

`VK_KHR_line_rasterization`

Extension Type

Device extension

Registered Extension Number

535

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2023-06-08

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA
- Allen Jensen, NVIDIA
- Faith Ekstrand, Intel

Description

This extension adds some line rasterization features that are commonly used in CAD applications and supported in other APIs like OpenGL. Bresenham-style line rasterization is supported, smooth rectangular lines (coverage to alpha) are supported, and stippled lines are supported for all three line rasterization modes.

New Commands

- vkCmdSetLineStippleKHR

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceLineRasterizationFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceLineRasterizationPropertiesKHR
- Extending VkPipelineRasterizationStateCreateInfo:
  - VkPipelineRasterizationLineStateCreateInfoKHR

New Enums

- VkLineRasterizationModeKHR

New Enum Constants

- VK_KHR_LINE_RASTERIZATION_EXTENSION_NAME
- VK_KHR_LINE_RASTERIZATION_SPEC_VERSION
- Extending VkDynamicState:
  - VK_DYNAMIC_STATE_LINE_STIPPLE_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LINE_RASTERIZATION_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LINE_RASTERIZATION_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_LINE_STATE_CREATE_INFO_KHR

Issues

1) Do we need to support Bresenham-style and smooth lines with more than one rasterization sample? i.e. the equivalent of glDisable(GL_MULTISAMPLE) in OpenGL when the framebuffer has more than one sample?

RESOLVED: Yes. For simplicity, Bresenham line rasterization carries forward a few restrictions from OpenGL, such as not supporting per-sample shading, alpha to coverage, or alpha to one.

Version History

- Revision 1, 2019-05-09 (Jeff Bolz)
  - Initial draft

VK_KHR_load_store_op_none

Name String

VK_KHR_load_store_op_none

Extension Type

Device extension

Registered Extension Number

527

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Shahbaz Youssefi [GitHub]syoussefi

Extension Proposal

VK_KHR_load_store_op_none

Other Extension Metadata

Last Modified Date

2023-05-16

Contributors

- Shahbaz Youssefi, Google
- Bill Licea-Kane, Qualcomm Technologies, Inc.
- Tobias Hector, AMD

Description

This extension provides `VK_ATTACHMENT_LOAD_OP_NONE_KHR` and `VK_ATTACHMENT_STORE_OP_NONE_KHR`, which are identically promoted from the `VK_EXT_load_store_op_none` extension.

New Enum Constants

- `VK_KHR_LOAD_STORE_OP_NONE_EXTENSION_NAME`
- `VK_KHR_LOAD_STORE_OP_NONE_SPEC_VERSION`
- Extending VkAttachmentLoadOp:
  - `VK_ATTACHMENT_LOAD_OP_NONE_KHR`
- Extending VkAttachmentStoreOp:
  - `VK_ATTACHMENT_STORE_OP_NONE_KHR`

NOTE	While `VK_ATTACHMENT_STORE_OP_NONE` is part of Vulkan 1.3, this extension was not promoted to core either in whole or in part. This functionality was promoted from `VK_KHR_dynamic_rendering`.

Version History

- Revision 1, 2023-05-16 (Shahbaz Youssefi)
  - Initial revision, based on VK_EXT_load_store_op_none.

VK_KHR_maintenance1

Name String

`VK_KHR_maintenance1`

Extension Type

Device extension

Registered Extension Number

70

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2018-03-13

Contributors

- Dan Ginsburg, Valve
- Daniel Koch, NVIDIA
- Daniel Rakos, AMD
- Jan-Harald Fredriksen, ARM
- Faith Ekstrand, Intel
- Jeff Bolz, NVIDIA
- Jesse Hall, Google
- John Kessenich, Google
- Michael Worcester, Imagination Technologies
- Neil Henning, Codeplay Software Ltd.
- Piers Daniell, NVIDIA
- Slawomir Grajewski, Intel
- Tobias Hector, Imagination Technologies
- Tom Olson, ARM

Description

VK_KHR_maintenance1 adds a collection of minor features that were intentionally left out or overlooked from the original Vulkan 1.0 release.

The new features are as follows:

- Allow 2D and 2D array image views to be created from 3D images, which can then be used as color framebuffer attachments. This allows applications to render to slices of a 3D image.
- Support vkCmdCopyImage between 2D array layers and 3D slices. This extension allows copying from layers of a 2D array image to slices of a 3D image and vice versa.
- Allow negative height to be specified in the VkViewport::height field to perform y-inversion of the clip-space to framebuffer-space transform. This allows apps to avoid having to use gl_Position.y = -gl_Position.y in shaders also targeting other APIs.
- Allow implementations to express support for doing just transfers and clears of image formats that they otherwise support no other format features for. This is done by adding new format feature flags VK_FORMAT_FEATURE_TRANSFER_SRC_BIT_KHR and VK_FORMAT_FEATURE_TRANSFER_DST_BIT_KHR.
- Support vkCmdFillBuffer on transfer-only queues. Previously vkCmdFillBuffer was defined to only work on command buffers allocated from command pools which support graphics or compute queues. It is now allowed on queues that just support transfer operations.
- Fix the inconsistency of how error conditions are returned between the vkCreateGraphicsPipelines and vkCreateComputePipelines functions and the vkAllocateDescriptorSets and vkAllocateCommandBuffers functions.
- Add new VK_ERROR_OUT_OF_POOL_MEMORY_KHR error so implementations can give a more precise reason for vkAllocateDescriptorSets failures.
- Add a new command vkTrimCommandPoolKHR which gives the implementation an opportunity to release any unused command pool memory back to the system.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkTrimCommandPoolKHR

New Bitmasks

- VkCommandPoolTrimFlagsKHR

New Enum Constants

- VK_KHR_MAINTENANCE1_EXTENSION_NAME
- VK_KHR_MAINTENANCE1_SPEC_VERSION

- VK_KHR_MAINTENANCE_1_EXTENSION_NAME
- VK_KHR_MAINTENANCE_1_SPEC_VERSION
- Extending VkFormatFeatureFlagBits:
  - VK_FORMAT_FEATURE_TRANSFER_DST_BIT_KHR
  - VK_FORMAT_FEATURE_TRANSFER_SRC_BIT_KHR
- Extending VkImageCreateFlagBits:
  - VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT_KHR
- Extending VkResult:
  - VK_ERROR_OUT_OF_POOL_MEMORY_KHR

Issues

1. Are viewports with zero height allowed?

   RESOLVED: Yes, although they have low utility.

Version History

- Revision 1, 2016-10-26 (Piers Daniell)
  - Internal revisions
- Revision 2, 2018-03-13 (Jon Leech)
  - Add issue for zero-height viewports

VK_KHR_maintenance2

Name String

VK_KHR_maintenance2

Extension Type

Device extension

Registered Extension Number

118

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Michael Worcester [GitHub]michaelworcester

Other Extension Metadata

Last Modified Date

2017-09-05

Contributors

- Michael Worcester, Imagination Technologies
- Stuart Smith, Imagination Technologies
- Jeff Bolz, NVIDIA
- Daniel Koch, NVIDIA
- Jan-Harald Fredriksen, ARM
- Daniel Rakos, AMD
- Neil Henning, Codeplay
- Piers Daniell, NVIDIA

Description

`VK_KHR_maintenance2` adds a collection of minor features that were intentionally left out or overlooked from the original Vulkan 1.0 release.

The new features are as follows:

- Allow the application to specify which aspect of an input attachment might be read for a given subpass.
- Allow implementations to express the clipping behavior of points.
- Allow creating images with usage flags that may not be supported for the base image's format, but are supported for image views of the image that have a different but compatible format.
- Allow creating uncompressed image views of compressed images.
- Allow the application to select between an upper-left and lower-left origin for the tessellation domain space.
- Adds two new image layouts for depth stencil images to allow either the depth or stencil aspect to be read-only while the other aspect is writable.

Input Attachment Specification

Input attachment specification allows an application to specify which aspect of a multi-aspect image (e.g. a depth/stencil format) will be accessed via a `subpassLoad` operation.

On some implementations there **may** be a performance penalty if the implementation does not

know (at vkCreateRenderPass time) which aspect(s) of multi-aspect images **can** be accessed as input attachments.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- VkInputAttachmentAspectReferenceKHR
- Extending VkImageViewCreateInfo:
  - VkImageViewUsageCreateInfoKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDevicePointClippingPropertiesKHR
- Extending VkPipelineTessellationStateCreateInfo:
  - VkPipelineTessellationDomainOriginStateCreateInfoKHR
- Extending VkRenderPassCreateInfo:
  - VkRenderPassInputAttachmentAspectCreateInfoKHR

New Enums

- VkPointClippingBehaviorKHR
- VkTessellationDomainOriginKHR

New Enum Constants

- `VK_KHR_MAINTENANCE2_EXTENSION_NAME`
- `VK_KHR_MAINTENANCE2_SPEC_VERSION`
- `VK_KHR_MAINTENANCE_2_EXTENSION_NAME`
- `VK_KHR_MAINTENANCE_2_SPEC_VERSION`
- Extending VkImageCreateFlagBits:
  - `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT_KHR`
  - `VK_IMAGE_CREATE_EXTENDED_USAGE_BIT_KHR`
- Extending VkImageLayout:
  - `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_STENCIL_READ_ONLY_OPTIMAL_KHR`
  - `VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_STENCIL_ATTACHMENT_OPTIMAL_KHR`
- Extending VkPointClippingBehavior:
  - `VK_POINT_CLIPPING_BEHAVIOR_ALL_CLIP_PLANES_KHR`
  - `VK_POINT_CLIPPING_BEHAVIOR_USER_CLIP_PLANES_ONLY_KHR`
- Extending VkStructureType:

    - VK_STRUCTURE_TYPE_IMAGE_VIEW_USAGE_CREATE_INFO_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_POINT_CLIPPING_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_PIPELINE_TESSELLATION_DOMAIN_ORIGIN_STATE_CREATE_INFO_KHR
    - VK_STRUCTURE_TYPE_RENDER_PASS_INPUT_ATTACHMENT_ASPECT_CREATE_INFO_KHR
- Extending VkTessellationDomainOrigin:
    - VK_TESSELLATION_DOMAIN_ORIGIN_LOWER_LEFT_KHR
    - VK_TESSELLATION_DOMAIN_ORIGIN_UPPER_LEFT_KHR

Input Attachment Specification Example

Consider the case where a render pass has two subpasses and two attachments.

Attachment 0 has the format VK_FORMAT_D24_UNORM_S8_UINT, attachment 1 has some color format.

Subpass 0 writes to attachment 0, subpass 1 reads only the depth information from attachment 0 (using inputAttachmentRead) and writes to attachment 1.

```
    VkInputAttachmentAspectReferenceKHR references[] = {
        {
            .subpass = 1,
            .inputAttachmentIndex = 0,
            .aspectMask = VK_IMAGE_ASPECT_DEPTH_BIT
        }
    };

    VkRenderPassInputAttachmentAspectCreateInfoKHR specifyAspects = {
        .sType =
VK_STRUCTURE_TYPE_RENDER_PASS_INPUT_ATTACHMENT_ASPECT_CREATE_INFO_KHR,
        .pNext = NULL,
        .aspectReferenceCount = 1,
        .pAspectReferences = references
    };


    VkRenderPassCreateInfo createInfo = {
        ...
        .pNext = &specifyAspects,
        ...
    };

    vkCreateRenderPass(...);
```

Issues

1) What is the default tessellation domain origin?

RESOLVED: Vulkan 1.0 originally inadvertently documented a lower-left origin, but the

conformance tests and all implementations implemented an upper-left origin. This extension adds a control to select between lower-left (for compatibility with OpenGL) and upper-left, and we retroactively fix unextended Vulkan to have a default of an upper-left origin.

Version History

- Revision 1, 2017-04-28

VK_KHR_maintenance3

Name String

`VK_KHR_maintenance3`

Extension Type

Device extension

Registered Extension Number

169

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2017-09-05

Contributors

- Jeff Bolz, NVIDIA

Description

`VK_KHR_maintenance3` adds a collection of minor features that were intentionally left out or overlooked from the original Vulkan 1.0 release.

The new features are as follows:

- A limit on the maximum number of descriptors that are supported in a single descriptor set layout. Some implementations have a limit on the total size of descriptors in a set, which cannot be expressed in terms of the limits in Vulkan 1.0.
- A limit on the maximum size of a single memory allocation. Some platforms have kernel interfaces that limit the maximum size of an allocation.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkGetDescriptorSetLayoutSupportKHR

New Structures

- VkDescriptorSetLayoutSupportKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceMaintenance3PropertiesKHR

New Enum Constants

- `VK_KHR_MAINTENANCE3_EXTENSION_NAME`
- `VK_KHR_MAINTENANCE3_SPEC_VERSION`
- `VK_KHR_MAINTENANCE_3_EXTENSION_NAME`
- `VK_KHR_MAINTENANCE_3_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_SUPPORT_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_3_PROPERTIES_KHR`

Version History

- Revision 1, 2017-08-22

VK_KHR_maintenance4

Name String

`VK_KHR_maintenance4`

Extension Type

Device extension

Registered Extension Number

414

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2021-10-25

Interactions and External Dependencies

- Requires SPIR-V 1.2 for `LocalSizeId`

Contributors

- Lionel Duc, NVIDIA
- Faith Ekstrand, Intel
- Spencer Fricke, Samsung
- Tobias Hector, AMD
- Lionel Landwerlin, Intel
- Graeme Leese, Broadcom
- Tom Olson, Arm
- Stu Smith, AMD
- Yiwei Zhang, Google

Description

`VK_KHR_maintenance4` adds a collection of minor features, none of which would warrant an entire extension of their own.

The new features are as follows:

- Allow the application to destroy their VkPipelineLayout object immediately after it was used to create another object. It is no longer necessary to keep its handle valid while the created object

is in use.

- Add a new `maxBufferSize` implementation-defined limit for the maximum size `VkBuffer` that **can** be created.
- Add support for the SPIR-V 1.2 `LocalSizeId` execution mode, which can be used as an alternative to `LocalSize` to specify the local workgroup size with specialization constants.
- Add a guarantee that images created with identical creation parameters will always have the same alignment requirements.
- Add new vkGetDeviceBufferMemoryRequirementsKHR, vkGetDeviceImageMemoryRequirementsKHR, and vkGetDeviceImageSparseMemoryRequirementsKHR to allow the application to query the image memory requirements without having to create an image object and query it.
- Relax the requirement that push constants must be initialized before they are dynamically accessed.
- Relax the interface matching rules to allow a larger output vector to match with a smaller input vector, with additional values being discarded.
- Add a guarantee for buffer memory requirement that the size memory requirement is never greater than the result of aligning create size with the alignment memory requirement.

New Commands

- vkGetDeviceBufferMemoryRequirementsKHR
- vkGetDeviceImageMemoryRequirementsKHR
- vkGetDeviceImageSparseMemoryRequirementsKHR

New Structures

- VkDeviceBufferMemoryRequirementsKHR
- VkDeviceImageMemoryRequirementsKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceMaintenance4FeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceMaintenance4PropertiesKHR

New Enum Constants

- `VK_KHR_MAINTENANCE_4_EXTENSION_NAME`
- `VK_KHR_MAINTENANCE_4_SPEC_VERSION`
- Extending VkImageAspectFlagBits:
  - `VK_IMAGE_ASPECT_NONE_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_DEVICE_BUFFER_MEMORY_REQUIREMENTS_KHR`

- VK_STRUCTURE_TYPE_DEVICE_IMAGE_MEMORY_REQUIREMENTS_KHR
- VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_FEATURES_KHR
- VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_4_PROPERTIES_KHR

Promotion to Vulkan 1.3

Vulkan APIs in this extension are included in core Vulkan 1.3, with the KHR suffix omitted. External interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

Issues

None.

Version History

- Revision 1, 2021-08-18 (Piers Daniell)
  - Internal revisions
- Revision 2, 2021-10-25 (Yiwei Zhang)
  - More guarantees on buffer memory requirements

VK_KHR_maintenance5

Name String

VK_KHR_maintenance5

Extension Type

Device extension

Registered Extension Number

471

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1
and
VK_KHR_dynamic_rendering
or
Vulkan Version 1.3

API Interactions

- Interacts with VK_VERSION_1_1

- Interacts with VK_VERSION_1_2
- Interacts with VK_VERSION_1_3
- Interacts with VK_EXT_attachment_feedback_loop_layout
- Interacts with VK_EXT_buffer_device_address
- Interacts with VK_EXT_conditional_rendering
- Interacts with VK_EXT_descriptor_buffer
- Interacts with VK_EXT_fragment_density_map
- Interacts with VK_EXT_graphics_pipeline_library
- Interacts with VK_EXT_opacity_micromap
- Interacts with VK_EXT_pipeline_creation_cache_control
- Interacts with VK_EXT_pipeline_protected_access
- Interacts with VK_EXT_transform_feedback
- Interacts with VK_KHR_acceleration_structure
- Interacts with VK_KHR_buffer_device_address
- Interacts with VK_KHR_device_group
- Interacts with VK_KHR_dynamic_rendering
- Interacts with VK_KHR_fragment_shading_rate
- Interacts with VK_KHR_pipeline_executable_properties
- Interacts with VK_KHR_pipeline_library
- Interacts with VK_KHR_ray_tracing_pipeline
- Interacts with VK_KHR_video_decode_queue
- Interacts with VK_KHR_video_encode_queue
- Interacts with VK_NV_device_generated_commands
- Interacts with VK_NV_displacement_micromap
- Interacts with VK_NV_ray_tracing
- Interacts with VK_NV_ray_tracing_motion_blur

Contact

- Stu Smith [GitHub]stu-s

Extension Proposal

VK_KHR_maintenance5

Other Extension Metadata

Last Modified Date

2023-05-02

Interactions and External Dependencies

Contributors

- Stu Smith, AMD
- Tobias Hector, AMD
- Shahbaz Youssefi, Google
- Slawomir Cygan, Intel
- Lionel Landwerlin, Intel
- James Fitzpatrick, Imagination Technologies
- Andrew Garrard, Imagination Technologies
- Ralph Potter, Samsung
- Pan Gao, Huawei
- Jan-Harald Fredriksen, ARM
- Jon Leech, Khronos
- Mike Blumenkrantz, Valve

Description

`VK_KHR_maintenance5` adds a collection of minor features, none of which would warrant an entire extension of their own.

The new features are as follows:

- A new `VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR` format
- A new `VK_FORMAT_A8_UNORM_KHR` format
- A property to indicate that multisample coverage operations are performed after sample counting in EarlyFragmentTests mode
- Relax VkBufferView creation requirements by allowing subsets of the associated VkBuffer usage using `VkBufferUsageFlags2CreateInfoKHR`
- A new command vkCmdBindIndexBuffer2KHR, allowing a range of memory to be bound as an index buffer
- vkGetDeviceProcAddr must return `NULL` for supported core functions beyond the version requested by the application.
- A property to indicate that the sample mask test is performed after sample counting in EarlyFragmentTests mode
- `vkCmdBindVertexBuffers2` now supports using `VK_WHOLE_SIZE` in the `pSizes` parameter.
- A default size of 1.0 is used if `PointSize` is not written
- Shader modules are deprecated - applications can now pass VkShaderModuleCreateInfo as a chained struct to pipeline creation via VkPipelineShaderStageCreateInfo
- A function vkGetRenderingAreaGranularityKHR to query the optimal render area for a dynamic rendering instance.

- A property to indicate that depth/stencil texturing operations with VK_COMPONENT_SWIZZLE_ONE have defined behavior
- Add vkGetImageSubresourceLayout2KHR and a new function vkGetDeviceImageSubresourceLayoutKHR to allow the application to query the image memory layout without having to create an image object and query it.
- Allow VK_REMAINING_ARRAY_LAYERS as the layerCount member of VkImageSubresourceLayers
- Adds stronger guarantees for propagation of VK_ERROR_DEVICE_LOST return values
- A property to indicate whether PointSize controls the final rasterization of polygons if polygon mode is VK_POLYGON_MODE_POINT
- Two properties to indicate the non-strict line rasterization algorithm used
- Two new flags words VkPipelineCreateFlagBits2KHR and VkBufferUsageFlagBits2KHR
- Physical-device-level functions can now be called with any value in the valid range for a type beyond the defined enumerants, such that applications can avoid checking individual features, extensions, or versions before querying supported properties of a particular enumerant.
- Clarification that copies between images of any type are allowed, treating 1D images as 2D images with a height of 1.

New Commands

- vkCmdBindIndexBuffer2KHR
- vkGetDeviceImageSubresourceLayoutKHR
- vkGetImageSubresourceLayout2KHR
- vkGetRenderingAreaGranularityKHR

New Structures

- VkDeviceImageSubresourceInfoKHR
- VkImageSubresource2KHR
- VkRenderingAreaInfoKHR
- VkSubresourceLayout2KHR
- Extending VkBufferViewCreateInfo, VkBufferCreateInfo, VkPhysicalDeviceExternalBufferInfo, VkDescriptorBufferBindingInfoEXT:
  - VkBufferUsageFlags2CreateInfoKHR
- Extending VkComputePipelineCreateInfo, VkGraphicsPipelineCreateInfo, VkRayTracingPipelineCreateInfoNV, VkRayTracingPipelineCreateInfoKHR:
  - VkPipelineCreateFlags2CreateInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceMaintenance5FeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceMaintenance5PropertiesKHR

New Enums

- VkBufferUsageFlagBits2KHR
- VkPipelineCreateFlagBits2KHR

New Bitmasks

- VkBufferUsageFlags2KHR
- VkPipelineCreateFlags2KHR

New Enum Constants

- VK_KHR_MAINTENANCE_5_EXTENSION_NAME
- VK_KHR_MAINTENANCE_5_SPEC_VERSION
- Extending VkFormat:
  - VK_FORMAT_A1B5G5R5_UNORM_PACK16_KHR
  - VK_FORMAT_A8_UNORM_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_BUFFER_USAGE_FLAGS_2_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_DEVICE_IMAGE_SUBRESOURCE_INFO_KHR
  - VK_STRUCTURE_TYPE_IMAGE_SUBRESOURCE_2_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_5_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_5_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_CREATE_FLAGS_2_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_RENDERING_AREA_INFO_KHR
  - VK_STRUCTURE_TYPE_SUBRESOURCE_LAYOUT_2_KHR

If VK_KHR_dynamic_rendering or Vulkan Version 1.3 and VK_EXT_fragment_density_map is supported:

- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT

If VK_KHR_dynamic_rendering or Vulkan Version 1.3 and VK_KHR_fragment_shading_rate is supported:

- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_RENDERING_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR

If VK_EXT_attachment_feedback_loop_layout is supported:

- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT

    - VK_PIPELINE_CREATE_2_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT

If VK_EXT_conditional_rendering is supported:

- Extending VkBufferUsageFlagBits2KHR:
    - VK_BUFFER_USAGE_2_CONDITIONAL_RENDERING_BIT_EXT

If VK_EXT_descriptor_buffer is supported:

- Extending VkBufferUsageFlagBits2KHR:
    - VK_BUFFER_USAGE_2_PUSH_DESCRIPTORS_DESCRIPTOR_BUFFER_BIT_EXT
    - VK_BUFFER_USAGE_2_RESOURCE_DESCRIPTOR_BUFFER_BIT_EXT
    - VK_BUFFER_USAGE_2_SAMPLER_DESCRIPTOR_BUFFER_BIT_EXT
- Extending VkPipelineCreateFlagBits2KHR:
    - VK_PIPELINE_CREATE_2_DESCRIPTOR_BUFFER_BIT_EXT

If VK_EXT_graphics_pipeline_library is supported:

- Extending VkPipelineCreateFlagBits2KHR:
    - VK_PIPELINE_CREATE_2_LINK_TIME_OPTIMIZATION_BIT_EXT
    - VK_PIPELINE_CREATE_2_RETAIN_LINK_TIME_OPTIMIZATION_INFO_BIT_EXT

If VK_EXT_opacity_micromap is supported:

- Extending VkBufferUsageFlagBits2KHR:
    - VK_BUFFER_USAGE_2_MICROMAP_BUILD_INPUT_READ_ONLY_BIT_EXT
    - VK_BUFFER_USAGE_2_MICROMAP_STORAGE_BIT_EXT
- Extending VkPipelineCreateFlagBits2KHR:
    - VK_PIPELINE_CREATE_2_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT

If VK_EXT_pipeline_protected_access is supported:

- Extending VkPipelineCreateFlagBits2KHR:
    - VK_PIPELINE_CREATE_2_NO_PROTECTED_ACCESS_BIT_EXT
    - VK_PIPELINE_CREATE_2_PROTECTED_ACCESS_ONLY_BIT_EXT

If VK_EXT_transform_feedback is supported:

- Extending VkBufferUsageFlagBits2KHR:
    - VK_BUFFER_USAGE_2_TRANSFORM_FEEDBACK_BUFFER_BIT_EXT
    - VK_BUFFER_USAGE_2_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT

If VK_KHR_acceleration_structure is supported:

- Extending VkBufferUsageFlagBits2KHR:

    - VK_BUFFER_USAGE_2_ACCELERATION_STRUCTURE_BUILD_INPUT_READ_ONLY_BIT_KHR
    - VK_BUFFER_USAGE_2_ACCELERATION_STRUCTURE_STORAGE_BIT_KHR

If VK_KHR_pipeline_executable_properties is supported:

- Extending VkPipelineCreateFlagBits2KHR:
    - VK_PIPELINE_CREATE_2_CAPTURE_INTERNAL_REPRESENTATIONS_BIT_KHR
    - VK_PIPELINE_CREATE_2_CAPTURE_STATISTICS_BIT_KHR

If VK_KHR_pipeline_library is supported:

- Extending VkPipelineCreateFlagBits2KHR:
    - VK_PIPELINE_CREATE_2_LIBRARY_BIT_KHR

If VK_KHR_ray_tracing_pipeline is supported:

- Extending VkBufferUsageFlagBits2KHR:
    - VK_BUFFER_USAGE_2_SHADER_BINDING_TABLE_BIT_KHR
- Extending VkPipelineCreateFlagBits2KHR:
    - VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR
    - VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR
    - VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR
    - VK_PIPELINE_CREATE_2_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR
    - VK_PIPELINE_CREATE_2_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR
    - VK_PIPELINE_CREATE_2_RAY_TRACING_SKIP_AABBS_BIT_KHR
    - VK_PIPELINE_CREATE_2_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR

If VK_KHR_video_decode_queue is supported:

- Extending VkBufferUsageFlagBits2KHR:
    - VK_BUFFER_USAGE_2_VIDEO_DECODE_DST_BIT_KHR
    - VK_BUFFER_USAGE_2_VIDEO_DECODE_SRC_BIT_KHR

If VK_KHR_video_encode_queue is supported:

- Extending VkBufferUsageFlagBits2KHR:
    - VK_BUFFER_USAGE_2_VIDEO_ENCODE_DST_BIT_KHR
    - VK_BUFFER_USAGE_2_VIDEO_ENCODE_SRC_BIT_KHR

If VK_NV_device_generated_commands is supported:

- Extending VkPipelineCreateFlagBits2KHR:
    - VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_NV

If VK_NV_displacement_micromap is supported:

- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_RAY_TRACING_DISPLACEMENT_MICROMAP_BIT_NV

If VK_NV_ray_tracing is supported:

- Extending VkBufferUsageFlagBits2KHR:
  - VK_BUFFER_USAGE_2_RAY_TRACING_BIT_NV
- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_DEFER_COMPILE_BIT_NV

If VK_NV_ray_tracing_motion_blur is supported:

- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_RAY_TRACING_ALLOW_MOTION_BIT_NV

If Vulkan Version 1.1 or VK_KHR_device_group is supported:

- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_DISPATCH_BASE_BIT_KHR
  - VK_PIPELINE_CREATE_2_VIEW_INDEX_FROM_DEVICE_INDEX_BIT_KHR

If Vulkan Version 1.2 or VK_KHR_buffer_device_address or VK_EXT_buffer_device_address is supported:

- Extending VkBufferUsageFlagBits2KHR:
  - VK_BUFFER_USAGE_2_SHADER_DEVICE_ADDRESS_BIT_KHR

If Vulkan Version 1.3 or VK_EXT_pipeline_creation_cache_control is supported:

- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_EARLY_RETURN_ON_FAILURE_BIT_KHR
  - VK_PIPELINE_CREATE_2_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT_KHR

Issues

None.

Version History

- Revision 1, 2022-12-12 (Stu Smith)
  - Initial revision

VK_KHR_maintenance6

Name String

`VK_KHR_maintenance6`

Extension Type

Device extension

Registered Extension Number

546

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1

API Interactions

- Interacts with VK_EXT_descriptor_buffer
- Interacts with VK_KHR_push_descriptor

Contact

- Jon Leech [GitHub]oddhack

Extension Proposal

VK_KHR_maintenance6

Other Extension Metadata

Last Modified Date

2023-08-03

Interactions and External Dependencies

- Interacts with `VK_EXT_robustness2`

Contributors

- Jon Leech, Khronos
- Stu Smith, AMD
- Mike Blumenkrantz, Valve
- Ralph Potter, Samsung
- James Fitzpatrick, Imagination Technologies
- Piers Daniell, NVIDIA
- Daniel Story, Nintendo

Description

VK_KHR_maintenance6 adds a collection of minor features, none of which would warrant an entire extension of their own.

The new features are as follows:

- VkBindMemoryStatusKHR may be included in the `pNext` chain of VkBindBufferMemoryInfo and VkBindImageMemoryInfo, allowing applications to identify individual resources for which memory binding failed during calls to vkBindBufferMemory2KHR and vkBindImageMemory2KHR.
- A new property `fragmentShadingRateClampCombinerInputs` to indicate if an implementation clamps the inputs to fragment shading rate combiner operations.
- VK_NULL_HANDLE is allowed to be used when binding an index buffer, instead of a valid VkBuffer handle. When the `nullDescriptor` feature is enabled, every index fetched results in a value of zero.
- A new property `maxCombinedImageSamplerDescriptorCount` to indicate the maximum number of descriptors needed for any of the formats that require a sampler $Y'C_BC_R$ conversion supported by the implementation.
- A new property `blockTexelViewCompatibleMultipleLayers` indicating whether `VK_IMAGE_CREATE_BLOCK_TEXEL_VIEW_COMPATIBLE_BIT` is allowed to be used with `layerCount` > 1
- `pNext` extensible *2 versions of all descriptor binding commands.

New Commands

- vkCmdBindDescriptorSets2KHR
- vkCmdPushConstants2KHR

If `VK_EXT_descriptor_buffer` is supported:

- vkCmdBindDescriptorBufferEmbeddedSamplers2EXT
- vkCmdSetDescriptorBufferOffsets2EXT

If VK_KHR_push_descriptor is supported:

- vkCmdPushDescriptorSet2KHR
- vkCmdPushDescriptorSetWithTemplate2KHR

New Structures

- VkBindDescriptorSetsInfoKHR
- VkPushConstantsInfoKHR
- Extending VkBindBufferMemoryInfo, VkBindImageMemoryInfo:
  - VkBindMemoryStatusKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:

    - VkPhysicalDeviceMaintenance6FeaturesKHR
- Extending VkPhysicalDeviceProperties2:
    - VkPhysicalDeviceMaintenance6PropertiesKHR

If VK_EXT_descriptor_buffer is supported:

- VkBindDescriptorBufferEmbeddedSamplersInfoEXT
- VkSetDescriptorBufferOffsetsInfoEXT

If VK_KHR_push_descriptor is supported:

- VkPushDescriptorSetInfoKHR
- VkPushDescriptorSetWithTemplateInfoKHR

New Enum Constants

- VK_KHR_MAINTENANCE_6_EXTENSION_NAME
- VK_KHR_MAINTENANCE_6_SPEC_VERSION
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_BIND_DESCRIPTOR_SETS_INFO_KHR
    - VK_STRUCTURE_TYPE_BIND_MEMORY_STATUS_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_6_FEATURES_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_6_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_PUSH_CONSTANTS_INFO_KHR

If VK_EXT_descriptor_buffer is supported:

- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_BIND_DESCRIPTOR_BUFFER_EMBEDDED_SAMPLERS_INFO_EXT
    - VK_STRUCTURE_TYPE_SET_DESCRIPTOR_BUFFER_OFFSETS_INFO_EXT

If VK_KHR_push_descriptor is supported:

- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PUSH_DESCRIPTOR_SET_INFO_KHR
    - VK_STRUCTURE_TYPE_PUSH_DESCRIPTOR_SET_WITH_TEMPLATE_INFO_KHR

Issues

None.

Version History

- Revision 1, 2023-08-01 (Jon Leech)

- Initial revision

VK_KHR_maintenance7

Name String

`VK_KHR_maintenance7`

Extension Type

Device extension

Registered Extension Number

563

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1

Contact

- Mike Blumenkrantz [GitHub]zmike

Extension Proposal

VK_KHR_maintenance7

Other Extension Metadata

Last Modified Date

2024-01-30

Interactions and External Dependencies

Contributors

- Mike Blumenkrantz, Valve
- Hans-Kristian Arntzen, Valve
- Pan Gao, Huawei
- Tobias Hector, AMD
- Jon Leech, Khronos
- Daniel Story, Nintendo
- Shahbaz Youssefi, Google
- Yiwei Zhang, Google
- Matthew Netsch, Qualcomm

Description

VK_KHR_maintenance7 adds a collection of minor features, none of which would warrant an entire extension of their own.

The proposed new features are as follows:

- Add a property query to determine if a framebuffer writes to depth or stencil aspect does not trigger a write access in the sibling aspect. For example, this allows sampling stencil aspect as a texture while rendering to the sibling depth attachment and vice-versa given appropriate image layouts.
- Add a way to query information regarding the underlying devices in environments where the Vulkan implementation is provided through layered implementations. For example, running on Mesa/Venus, driver ID is returned as `VK_DRIVER_ID_MESA_VENUS`, but it can be necessary to know what the real driver under the hood is. The new VkPhysicalDeviceLayeredApiPropertiesKHR structure can be used to gather information regarding layers underneath the top-level physical device.
- Promote `VK_RENDERING_CONTENTS_INLINE_BIT_EXT` and `VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_EXT` to KHR
- Add a limit to report the maximum total count of dynamic uniform buffers and dynamic storage buffers that can be included in a pipeline layout.
- Require that for an unsigned integer query, the 32-bit result value **must** be equal to the 32 least significant bits of the equivalent 64-bit result value.
- Add query for robust access support when using fragment shading rate attachments

New Structures

- VkPhysicalDeviceLayeredApiPropertiesKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceMaintenance7FeaturesKHR
- Extending VkPhysicalDeviceLayeredApiPropertiesKHR:
  - VkPhysicalDeviceLayeredApiVulkanPropertiesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceLayeredApiPropertiesListKHR
  - VkPhysicalDeviceMaintenance7PropertiesKHR

New Enums

- VkPhysicalDeviceLayeredApiKHR

New Enum Constants

- `VK_KHR_MAINTENANCE_7_EXTENSION_NAME`
- `VK_KHR_MAINTENANCE_7_SPEC_VERSION`

- Extending VkRenderingFlagBits:
  - `VK_RENDERING_CONTENTS_INLINE_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_PROPERTIES_LIST_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LAYERED_API_VULKAN_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_7_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAINTENANCE_7_PROPERTIES_KHR`
- Extending VkSubpassContents:
  - `VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_KHR`

Issues

None.

Version History

- Revision 1, 2024-01-30 (Jon Leech)
  - Initial revision

VK_KHR_map_memory2

Name String

`VK_KHR_map_memory2`

Extension Type

Device extension

Registered Extension Number

272

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Faith Ekstrand [GitHub]gfxstrand

Extension Proposal

VK_KHR_map_memory2

Other Extension Metadata

Last Modified Date

2023-03-14

Interactions and External Dependencies

- None

Contributors

- Faith Ekstrand, Collabora
- Tobias Hector, AMD

Description

This extension provides extensible versions of the Vulkan memory map and unmap commands. The new commands are functionally identical to the core commands, except that their parameters are specified using extensible structures that can be used to pass extension-specific information.

New Commands

- vkMapMemory2KHR
- vkUnmapMemory2KHR

New Structures

- VkMemoryMapInfoKHR
- VkMemoryUnmapInfoKHR

New Enums

- VkMemoryUnmapFlagBitsKHR

New Bitmasks

- VkMemoryUnmapFlagsKHR

New Enum Constants

- `VK_KHR_MAP_MEMORY_2_EXTENSION_NAME`
- `VK_KHR_MAP_MEMORY_2_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_MEMORY_MAP_INFO_KHR`
  - `VK_STRUCTURE_TYPE_MEMORY_UNMAP_INFO_KHR`

Version History

- Revision 0, 2022-08-03 (Faith Ekstrand)
  - Internal revisions

- Revision 1, 2023-03-14
  - Public release

VK_KHR_multiview

Name String

`VK_KHR_multiview`

Extension Type

Device extension

Registered Extension Number

54

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_multiview

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2016-10-28

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_multiview`

Contributors

- Jeff Bolz, NVIDIA

Description

This extension has the same goal as the OpenGL ES `GL_OVR_multiview` extension. Multiview is a rendering technique originally designed for VR where it is more efficient to record a single set of commands to be executed with slightly different behavior for each “view”.

It includes a concise way to declare a render pass with multiple views, and gives implementations freedom to render the views in the most efficient way possible. This is done with a multiview configuration specified during render pass creation with the VkRenderPassMultiviewCreateInfo passed into VkRenderPassCreateInfo::`pNext`.

This extension enables the use of the `SPV_KHR_multiview` shader extension, which adds a new `ViewIndex` built-in type that allows shaders to control what to do for each view. If using GLSL there is also the `GL_EXT_multiview` extension that introduces a `highp int gl_ViewIndex;` built-in variable for vertex, tessellation, geometry, and fragment shaders.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceMultiviewFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceMultiviewPropertiesKHR
- Extending VkRenderPassCreateInfo:
  - VkRenderPassMultiviewCreateInfoKHR

New Enum Constants

- `VK_KHR_MULTIVIEW_EXTENSION_NAME`
- `VK_KHR_MULTIVIEW_SPEC_VERSION`
- Extending VkDependencyFlagBits:
  - `VK_DEPENDENCY_VIEW_LOCAL_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTIVIEW_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_RENDER_PASS_MULTIVIEW_CREATE_INFO_KHR`

New Built-In Variables

- `ViewIndex`

New SPIR-V Capabilities

- `MultiView`

Version History

- Revision 1, 2016-10-28 (Jeff Bolz)
  - Internal revisions

VK_KHR_performance_query

Name String

`VK_KHR_performance_query`

Extension Type

Device extension

Registered Extension Number

117

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- Developer tools

Contact

- Alon Or-bach [GitHub]alonorbach

Other Extension Metadata

Last Modified Date

2019-10-08

IP Status

No known IP claims.

Contributors

- Jesse Barker, Unity Technologies
- Kenneth Benzie, Codeplay

- Jan-Harald Fredriksen, ARM
- Jeff Leger, Qualcomm
- Jesse Hall, Google
- Tobias Hector, AMD
- Neil Henning, Codeplay
- Baldur Karlsson
- Lionel Landwerlin, Intel
- Peter Lohrmann, AMD
- Alon Or-bach, Samsung
- Daniel Rakos, AMD
- Niklas Smedberg, Unity Technologies
- Igor Ostrowski, Intel

Description

The `VK_KHR_performance_query` extension adds a mechanism to allow querying of performance counters for use in applications and by profiling tools.

Each queue family **may** expose counters that **can** be enabled on a queue of that family. We extend VkQueryType to add a new query type for performance queries, and chain a structure on VkQueryPoolCreateInfo to specify the performance queries to enable.

New Commands

- vkAcquireProfilingLockKHR
- vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR
- vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR
- vkReleaseProfilingLockKHR

New Structures

- VkAcquireProfilingLockInfoKHR
- VkPerformanceCounterDescriptionKHR
- VkPerformanceCounterKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePerformanceQueryFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDevicePerformanceQueryPropertiesKHR
- Extending VkQueryPoolCreateInfo:
  - VkQueryPoolPerformanceCreateInfoKHR

- Extending VkSubmitInfo, VkSubmitInfo2:
  - VkPerformanceQuerySubmitInfoKHR

New Unions

- VkPerformanceCounterResultKHR

New Enums

- VkAcquireProfilingLockFlagBitsKHR
- VkPerformanceCounterDescriptionFlagBitsKHR
- VkPerformanceCounterScopeKHR
- VkPerformanceCounterStorageKHR
- VkPerformanceCounterUnitKHR

New Bitmasks

- VkAcquireProfilingLockFlagsKHR
- VkPerformanceCounterDescriptionFlagsKHR

New Enum Constants

- `VK_KHR_PERFORMANCE_QUERY_EXTENSION_NAME`
- `VK_KHR_PERFORMANCE_QUERY_SPEC_VERSION`
- Extending VkQueryType:
  - `VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_ACQUIRE_PROFILING_LOCK_INFO_KHR`
  - `VK_STRUCTURE_TYPE_PERFORMANCE_COUNTER_DESCRIPTION_KHR`
  - `VK_STRUCTURE_TYPE_PERFORMANCE_COUNTER_KHR`
  - `VK_STRUCTURE_TYPE_PERFORMANCE_QUERY_SUBMIT_INFO_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PERFORMANCE_QUERY_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PERFORMANCE_QUERY_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_QUERY_POOL_PERFORMANCE_CREATE_INFO_KHR`

Issues

1) Should this extension include a mechanism to begin a query in command buffer *A* and end the query in command buffer *B*?

RESOLVED No - queries are tied to command buffer creation and thus have to be encapsulated within a single command buffer.

2) Should this extension include a mechanism to begin and end queries globally on the queue, not using the existing command buffer commands?

RESOLVED No - for the same reasoning as the resolution of 1).

3) Should this extension expose counters that require multiple passes?

RESOLVED Yes - users should re-submit a command buffer with the same commands in it multiple times, specifying the pass to count as the query parameter in VkPerformanceQuerySubmitInfoKHR.

4) How to handle counters across parallel workloads?

RESOLVED In the spirit of Vulkan, a counter description flag `VK_PERFORMANCE_COUNTER_DESCRIPTION_CONCURRENTLY_IMPACTED_BIT_KHR` denotes that the accuracy of a counter result is affected by parallel workloads.

5) How to handle secondary command buffers?

RESOLVED Secondary command buffers inherit any counter pass index specified in the parent primary command buffer. Note: this is no longer an issue after change from issue 10 resolution

6) What commands does the profiling lock have to be held for?

RESOLVED For any command buffer that is being queried with a performance query pool, the profiling lock **must** be held while that command buffer is in the *recording*, *executable*, or *pending state*.

7) Should we support vkCmdCopyQueryPoolResults?

RESOLVED Yes.

8) Should we allow performance queries to interact with multiview?

RESOLVED Yes, but the performance queries must be performed once for each pass per view.

9) Should a `queryCount > 1` be usable for performance queries?

RESOLVED Yes. Some vendors will have costly performance counter query pool creation, and would rather if a certain set of counters were to be used multiple times that a `queryCount > 1` can be used to amortize the instantiation cost.

10) Should we introduce an indirect mechanism to set the counter pass index?

RESOLVED Specify the counter pass index at submit time instead, to avoid requiring re-recording of command buffers when multiple counter passes are needed.

Examples

The following example shows how to find what performance counters a queue family supports, setup a query pool to record these performance counters, how to add the query pool to the command buffer to record information, and how to get the results from the query pool.

```
// A previously created physical device
VkPhysicalDevice physicalDevice;

// One of the queue families our device supports
uint32_t queueFamilyIndex;

uint32_t counterCount;

// Get the count of counters supported
vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR(
  physicalDevice,
  queueFamilyIndex,
  &counterCount,
  NULL,
  NULL);

VkPerformanceCounterKHR* counters =
  malloc(sizeof(VkPerformanceCounterKHR) * counterCount);
VkPerformanceCounterDescriptionKHR* counterDescriptions =
  malloc(sizeof(VkPerformanceCounterDescriptionKHR) * counterCount);

// Get the counters supported
vkEnumeratePhysicalDeviceQueueFamilyPerformanceQueryCountersKHR(
  physicalDevice,
  queueFamilyIndex,
  &counterCount,
  counters,
  counterDescriptions);

// Try to enable the first 8 counters
uint32_t enabledCounters[8];

const uint32_t enabledCounterCount = min(counterCount, 8));

for (uint32_t i = 0; i < enabledCounterCount; i++) {
  enabledCounters[i] = i;
}

// A previously created device that had the performanceCounterQueryPools feature
// set to VK_TRUE
VkDevice device;

VkQueryPoolPerformanceCreateInfoKHR performanceQueryCreateInfo = {
  .sType = VK_STRUCTURE_TYPE_QUERY_POOL_PERFORMANCE_CREATE_INFO_KHR,
  .pNext = NULL,

  // Specify the queue family that this performance query is performed on
  .queueFamilyIndex = queueFamilyIndex,

  // The number of counters to enable
```

```
  .counterIndexCount = enabledCounterCount,

  // The array of indices of counters to enable
  .pCounterIndices = enabledCounters
};


// Get the number of passes our counters will require.
uint32_t numPasses;

vkGetPhysicalDeviceQueueFamilyPerformanceQueryPassesKHR(
  physicalDevice,
  &performanceQueryCreateInfo,
  &numPasses);

VkQueryPoolCreateInfo queryPoolCreateInfo = {
  .sType = VK_STRUCTURE_TYPE_QUERY_POOL_CREATE_INFO,
  .pNext = &performanceQueryCreateInfo,
  .flags = 0,
  // Using our new query type here
  .queryType = VK_QUERY_TYPE_PERFORMANCE_QUERY_KHR,
  .queryCount = 1,
  .pipelineStatistics = 0
};

VkQueryPool queryPool;

VkResult result = vkCreateQueryPool(
  device,
  &queryPoolCreateInfo,
  NULL,
  &queryPool);

assert(VK_SUCCESS == result);

// A queue from queueFamilyIndex
VkQueue queue;

// A command buffer we want to record counters on
VkCommandBuffer commandBuffer;

VkCommandBufferBeginInfo commandBufferBeginInfo = {
  .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO,
  .pNext = NULL,
  .flags = 0,
  .pInheritanceInfo = NULL
};

VkAcquireProfilingLockInfoKHR lockInfo = {
  .sType = VK_STRUCTURE_TYPE_ACQUIRE_PROFILING_LOCK_INFO_KHR,
  .pNext = NULL,
```

```
  .flags = 0,
  .timeout = UINT64_MAX // Wait forever for the lock
};

// Acquire the profiling lock before we record command buffers
// that will use performance queries

result = vkAcquireProfilingLockKHR(device, &lockInfo);

assert(VK_SUCCESS == result);

result = vkBeginCommandBuffer(commandBuffer, &commandBufferBeginInfo);

assert(VK_SUCCESS == result);

vkCmdResetQueryPool(
  commandBuffer,
  queryPool,
  0,
  1);

vkCmdBeginQuery(
  commandBuffer,
  queryPool,
  0,
  0);

// Perform the commands you want to get performance information on
// ...

// Perform a barrier to ensure all previous commands were complete before
// ending the query
vkCmdPipelineBarrier(commandBuffer,
  VK_PIPELINE_STAGE_BOTTOM_OF_PIPE_BIT,
  VK_PIPELINE_STAGE_BOTTOM_OF_PIPE_BIT,
  0,
  0,
  NULL,
  0,
  NULL,
  0,
  NULL);

vkCmdEndQuery(
  commandBuffer,
  queryPool,
  0);

result = vkEndCommandBuffer(commandBuffer);

assert(VK_SUCCESS == result);
```

```
for (uint32_t counterPass = 0; counterPass < numPasses; counterPass++) {

  VkPerformanceQuerySubmitInfoKHR performanceQuerySubmitInfo = {
    VK_STRUCTURE_TYPE_PERFORMANCE_QUERY_SUBMIT_INFO_KHR,
    NULL,
    counterPass
  };


  // Submit the command buffer and wait for its completion
  // ...
}

// Release the profiling lock after the command buffer is no longer in the
// pending state.
vkReleaseProfilingLockKHR(device);

result = vkResetCommandBuffer(commandBuffer, 0);

assert(VK_SUCCESS == result);

// Create an array to hold the results of all counters
VkPerformanceCounterResultKHR* recordedCounters = malloc(
  sizeof(VkPerformanceCounterResultKHR) * enabledCounterCount);

result = vkGetQueryPoolResults(
  device,
  queryPool,
  0,
  1,
  sizeof(VkPerformanceCounterResultKHR) * enabledCounterCount,
  recordedCounters,
  sizeof(VkPerformanceCounterResultKHR) * enabledCounterCount,
  NULL);

// recordedCounters is filled with our counters, we will look at one for posterity
switch (counters[0].storage) {
  case VK_PERFORMANCE_COUNTER_STORAGE_INT32:
    // use recordCounters[0].int32 to get at the counter result!
    break;
  case VK_PERFORMANCE_COUNTER_STORAGE_INT64:
    // use recordCounters[0].int64 to get at the counter result!
    break;
  case VK_PERFORMANCE_COUNTER_STORAGE_UINT32:
    // use recordCounters[0].uint32 to get at the counter result!
    break;
  case VK_PERFORMANCE_COUNTER_STORAGE_UINT64:
    // use recordCounters[0].uint64 to get at the counter result!
    break;
  case VK_PERFORMANCE_COUNTER_STORAGE_FLOAT32:
```

```
    // use recordCounters[0].float32 to get at the counter result!
    break;
  case VK_PERFORMANCE_COUNTER_STORAGE_FLOAT64:
    // use recordCounters[0].float64 to get at the counter result!
    break;
}
```

Version History

- Revision 1, 2019-10-08

VK_KHR_pipeline_binary

Name String

VK_KHR_pipeline_binary

Extension Type

Device extension

Registered Extension Number

484

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_maintenance5

Contact

- Stu Smith [GitHub]stu-s

Extension Proposal

VK_KHR_pipeline_binary

Other Extension Metadata

Last Modified Date

2024-07-01

Contributors

- Stu Smith, AMD
- Tobias Hector, AMD
- Alan Harrison, AMD
- Maciej Jesionowski, AMD

- Younggwan Kim, Arm
- Jan-Harald Fredriksen, Arm
- Ting Wei, Arm
- Chris Glover, Google
- Shahbaz Youssefi, Google
- Jakub Kuderski, Google
- Piotr Byszewski, Mobica
- Piers Daniell, NVIDIA
- Ralph Potter, Samsung
- Matthew Netsch, Qualcomm
- Hans-Kristian Arntzen, Valve
- Samuel Pitoiset, Valve
- Tatsuyuki Ishi, Valve

Description

This extension provides a method to obtain binary data associated with individual pipelines such that applications can manage caching themselves instead of using VkPipelineCache objects.

New Object Types

- VkPipelineBinaryKHR

New Commands

- vkCreatePipelineBinariesKHR
- vkDestroyPipelineBinaryKHR
- vkGetPipelineBinaryDataKHR
- vkGetPipelineKeyKHR
- vkReleaseCapturedPipelineDataKHR

New Structures

- VkPipelineBinaryCreateInfoKHR
- VkPipelineBinaryDataInfoKHR
- VkPipelineBinaryDataKHR
- VkPipelineBinaryHandlesInfoKHR
- VkPipelineBinaryKeyKHR
- VkPipelineBinaryKeysAndDataKHR
- VkPipelineCreateInfoKHR

- VkReleaseCapturedPipelineDataInfoKHR
- Extending VkDeviceCreateInfo:
  - VkDevicePipelineBinaryInternalCacheControlKHR
- Extending VkGraphicsPipelineCreateInfo, VkComputePipelineCreateInfo, VkRayTracingPipelineCreateInfoKHR:
  - VkPipelineBinaryInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePipelineBinaryFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDevicePipelineBinaryPropertiesKHR

New Enum Constants

- VK_KHR_PIPELINE_BINARY_EXTENSION_NAME
- VK_KHR_PIPELINE_BINARY_SPEC_VERSION
- VK_MAX_PIPELINE_BINARY_KEY_SIZE_KHR
- Extending VkObjectType:
  - VK_OBJECT_TYPE_PIPELINE_BINARY_KHR
- Extending VkPipelineCreateFlagBits2KHR:
  - VK_PIPELINE_CREATE_2_CAPTURE_DATA_BIT_KHR
- Extending VkResult:
  - VK_ERROR_NOT_ENOUGH_SPACE_KHR
  - VK_PIPELINE_BINARY_MISSING_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_DEVICE_PIPELINE_BINARY_INTERNAL_CACHE_CONTROL_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_BINARY_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_BINARY_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_BINARY_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_BINARY_DATA_INFO_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_BINARY_HANDLES_INFO_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_BINARY_INFO_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_BINARY_KEY_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_RELEASE_CAPTURED_PIPELINE_DATA_INFO_KHR

Version History

- Revision 1, 2021-12-10 (Chris Glover)
  - Initial draft.

VK_KHR_pipeline_executable_properties

Name String

`VK_KHR_pipeline_executable_properties`

Extension Type

Device extension

Registered Extension Number

270

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- Developer tools

Contact

- Faith Ekstrand [GitHub]gfxstrand

Other Extension Metadata

Last Modified Date

2019-05-28

IP Status

No known IP claims.

Interactions and External Dependencies

Contributors

- Faith Ekstrand, Intel
- Ian Romanick, Intel
- Kenneth Graunke, Intel
- Baldur Karlsson, Valve

- Jesse Hall, Google
- Jeff Bolz, Nvidia
- Piers Daniel, Nvidia
- Tobias Hector, AMD
- Jan-Harald Fredriksen, ARM
- Tom Olson, ARM
- Daniel Koch, Nvidia
- Spencer Fricke, Samsung

Description

When a pipeline is created, its state and shaders are compiled into zero or more device-specific executables, which are used when executing commands against that pipeline. This extension adds a mechanism to query properties and statistics about the different executables produced by the pipeline compilation process. This is intended to be used by debugging and performance tools to allow them to provide more detailed information to the user. Certain compile time shader statistics provided through this extension may be useful to developers for debugging or performance analysis.

New Commands

- vkGetPipelineExecutableInternalRepresentationsKHR
- vkGetPipelineExecutablePropertiesKHR
- vkGetPipelineExecutableStatisticsKHR

New Structures

- VkPipelineExecutableInfoKHR
- VkPipelineExecutableInternalRepresentationKHR
- VkPipelineExecutablePropertiesKHR
- VkPipelineExecutableStatisticKHR
- VkPipelineInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePipelineExecutablePropertiesFeaturesKHR

New Unions

- VkPipelineExecutableStatisticValueKHR

New Enums

- VkPipelineExecutableStatisticFormatKHR

New Enum Constants

- `VK_KHR_PIPELINE_EXECUTABLE_PROPERTIES_EXTENSION_NAME`
- `VK_KHR_PIPELINE_EXECUTABLE_PROPERTIES_SPEC_VERSION`
- Extending VkPipelineCreateFlagBits:
  - `VK_PIPELINE_CREATE_CAPTURE_INTERNAL_REPRESENTATIONS_BIT_KHR`
  - `VK_PIPELINE_CREATE_CAPTURE_STATISTICS_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PIPELINE_EXECUTABLE_PROPERTIES_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_INTERNAL_REPRESENTATION_KHR`
  - `VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_PROPERTIES_KHR`
  - `VK_STRUCTURE_TYPE_PIPELINE_EXECUTABLE_STATISTIC_KHR`
  - `VK_STRUCTURE_TYPE_PIPELINE_INFO_KHR`

Issues

1) What should we call the pieces of the pipeline which are produced by the compilation process and about which you can query properties and statistics?

RESOLVED: Call them “executables”. The name “binary” was used in early drafts of the extension but it was determined that “pipeline binary” could have a fairly broad meaning (such as a binary serialized form of an entire pipeline) and was too big of a namespace for the very specific needs of this extension.

Version History

- Revision 1, 2019-05-28 (Faith Ekstrand)
  - Initial draft

VK_KHR_pipeline_library

Name String

`VK_KHR_pipeline_library`

Extension Type

Device extension

Registered Extension Number

291

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Christoph Kubisch [GitHub]pixeljetstream

Other Extension Metadata

Last Modified Date

2020-01-08

IP Status

No known IP claims.

Contributors

- See contributors to `VK_KHR_ray_tracing_pipeline`

Description

A pipeline library is a special pipeline that cannot be bound, instead it defines a set of shaders and shader groups which can be linked into other pipelines. This extension defines the infrastructure for pipeline libraries, but does not specify the creation or usage of pipeline libraries. This is left to additional dependent extensions.

New Structures

- Extending VkGraphicsPipelineCreateInfo:
  - VkPipelineLibraryCreateInfoKHR

New Enum Constants

- `VK_KHR_PIPELINE_LIBRARY_EXTENSION_NAME`
- `VK_KHR_PIPELINE_LIBRARY_SPEC_VERSION`
- Extending VkPipelineCreateFlagBits:
  - `VK_PIPELINE_CREATE_LIBRARY_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PIPELINE_LIBRARY_CREATE_INFO_KHR`

Version History

- Revision 1, 2020-01-08 (Christoph Kubisch)
  - Initial draft.

VK_KHR_portability_enumeration

Name String

`VK_KHR_portability_enumeration`

Extension Type

Instance extension

Registered Extension Number

395

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Charles Giessen [GitHub]charles-lunarg

Other Extension Metadata

Last Modified Date

2021-06-02

IP Status

No known IP claims.

Interactions and External Dependencies

- Interacts with `VK_KHR_portability_subset`

Contributors

- Lenny Komow, LunarG
- Charles Giessen, LunarG

Description

This extension allows applications to control whether devices that expose the `VK_KHR_portability_subset` extension are included in the results of physical device enumeration. Since devices which support the `VK_KHR_portability_subset` extension are not fully conformant Vulkan implementations, the Vulkan loader does not report those devices unless the application explicitly asks for them. This prevents applications which may not be aware of non-conformant devices from accidentally using them, as any device which supports the `VK_KHR_portability_subset` extension mandates that the extension must be enabled if that device is used.

This extension is implemented in the loader.

New Enum Constants

- `VK_KHR_PORTABILITY_ENUMERATION_EXTENSION_NAME`
- `VK_KHR_PORTABILITY_ENUMERATION_SPEC_VERSION`
- Extending VkInstanceCreateFlagBits:
  - `VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR`

Version History

- Revision 1, 2021-06-02 (Lenny Komow)
  - Initial version

VK_KHR_present_id

Name String

`VK_KHR_present_id`

Extension Type

Device extension

Registered Extension Number

295

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_swapchain
and
VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Keith Packard [GitHub]keithp

Other Extension Metadata

Last Modified Date

2019-05-15

IP Status

No known IP claims.

Contributors

- Keith Packard, Valve
- Ian Elliott, Google
- Alon Or-bach, Samsung

Description

This device extension allows an application that uses the `VK_KHR_swapchain` extension to provide an identifier for present operations on a swapchain. An application **can** use this to reference specific present operations in other extensions.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePresentIdFeaturesKHR
- Extending VkPresentInfoKHR:
  - VkPresentIdKHR

New Enum Constants

- `VK_KHR_PRESENT_ID_EXTENSION_NAME`
- `VK_KHR_PRESENT_ID_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_ID_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PRESENT_ID_KHR`

Issues

None.

Examples

Version History

- Revision 1, 2019-05-15 (Keith Packard)
  - Initial version

VK_KHR_present_wait

Name String

`VK_KHR_present_wait`

Extension Type

Device extension

Registered Extension Number

249

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_swapchain
and
VK_KHR_present_id

Contact

- Keith Packard [GitHub]keithp

Other Extension Metadata

Last Modified Date

2019-05-15

IP Status

No known IP claims.

Contributors

- Keith Packard, Valve
- Ian Elliott, Google
- Tobias Hector, AMD
- Daniel Stone, Collabora

Description

This device extension allows an application that uses the `VK_KHR_swapchain` extension to wait for present operations to complete. An application can use this to monitor and control the pacing of the application by managing the number of outstanding images yet to be presented.

New Commands

- vkWaitForPresentKHR

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePresentWaitFeaturesKHR

New Enum Constants

- `VK_KHR_PRESENT_WAIT_EXTENSION_NAME`
- `VK_KHR_PRESENT_WAIT_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_WAIT_FEATURES_KHR`

Issues

1) When does the wait finish?

RESOLVED. The wait will finish when the present is visible to the user. There is no requirement for any precise timing relationship between the presentation of the image to the user, but implementations **should** signal the wait as close as possible to the presentation of the first pixel in the new image to the user.

2) Should this use fences or other existing synchronization mechanism.

RESOLVED. Because display and rendering are often implemented in separate drivers, this extension will provide a separate synchronization API.

3) Should this extension share present identification with other extensions?

RESOLVED. Yes. A new extension, VK_KHR_present_id, should be created to provide a shared structure for presentation identifiers.

4) What happens when presentations complete out of order wrt calls to vkQueuePresent? This could happen if the semaphores for the presentations were ready out of order.

OPTION A: Require that when a PresentId is set that the driver ensure that images are always presented in the order of calls to vkQueuePresent.

OPTION B: Finish both waits when the earliest present completes. This will complete the later present wait earlier than the actual presentation. This should be the easiest to implement as the driver need only track the largest present ID completed. This is also the 'natural' consequence of interpreting the existing vkWaitForPresentKHR specificationn.

OPTION C: Finish both waits when both have completed. This will complete the earlier presentation later than the actual presentation time. This is allowed by the current specification as there is no precise timing requirement for when the presentId value is updated. This requires slightly more complexity in the driver as it will need to track all outstanding presentId values.

Examples

Version History

- Revision 1, 2019-02-19 (Keith Packard)
  - Initial version

VK_KHR_push_descriptor

Name String

`VK_KHR_push_descriptor`

Extension Type

Device extension

Registered Extension Number

81

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

API Interactions

- Interacts with VK_VERSION_1_1
- Interacts with VK_KHR_descriptor_update_template

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2017-09-12

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA
- Michael Worcester, Imagination Technologies

Description

This extension allows descriptors to be written into the command buffer, while the implementation is responsible for managing their memory. Push descriptors may enable easier porting from older APIs and in some cases can be more efficient than writing descriptors into descriptor sets.

New Commands

- vkCmdPushDescriptorSetKHR

If Vulkan Version 1.1 or VK_KHR_descriptor_update_template is supported:

- vkCmdPushDescriptorSetWithTemplateKHR

New Structures

- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDevicePushDescriptorPropertiesKHR

New Enum Constants

- `VK_KHR_PUSH_DESCRIPTOR_EXTENSION_NAME`
- `VK_KHR_PUSH_DESCRIPTOR_SPEC_VERSION`
- Extending VkDescriptorSetLayoutCreateFlagBits:
  - `VK_DESCRIPTOR_SET_LAYOUT_CREATE_PUSH_DESCRIPTOR_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PUSH_DESCRIPTOR_PROPERTIES_KHR`

If Vulkan Version 1.1 or VK_KHR_descriptor_update_template is supported:

- Extending VkDescriptorUpdateTemplateType:
  - `VK_DESCRIPTOR_UPDATE_TEMPLATE_TYPE_PUSH_DESCRIPTORS_KHR`

Version History

- Revision 1, 2016-10-15 (Jeff Bolz)
  - Internal revisions
- Revision 2, 2017-09-12 (Tobias Hector)
  - Added interactions with Vulkan 1.1

VK_KHR_ray_query

Name String

`VK_KHR_ray_query`

Extension Type

Device extension

Registered Extension Number

349

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_spirv_1_4
and
VK_KHR_acceleration_structure

SPIR-V Dependencies

- SPV_KHR_ray_query

Contact

- Daniel Koch [GitHub]dgkoch

Other Extension Metadata

Last Modified Date

2020-11-12

Interactions and External Dependencies

- This extension provides API support for `GLSL_EXT_ray_query`

Contributors

- Matthäus Chajdas, AMD
- Greg Grebe, AMD
- Nicolai Hähnle, AMD
- Tobias Hector, AMD
- Dave Oldcorn, AMD
- Skyler Saleh, AMD
- Mathieu Robart, Arm
- Marius Bjorge, Arm
- Tom Olson, Arm
- Sebastian Tafuri, EA
- Henrik Rydgard, Embark
- Juan Cañada, Epic Games
- Patrick Kelly, Epic Games
- Yuriy O'Donnell, Epic Games
- Michael Doggett, Facebook/Oculus
- Andrew Garrard, Imagination
- Don Scorgie, Imagination

- Dae Kim, Imagination
- Joshua Barczak, Intel
- Slawek Grajewski, Intel
- Jeff Bolz, NVIDIA
- Pascal Gautron, NVIDIA
- Daniel Koch, NVIDIA
- Christoph Kubisch, NVIDIA
- Ashwin Lele, NVIDIA
- Robert Stepinski, NVIDIA
- Martin Stich, NVIDIA
- Nuno Subtil, NVIDIA
- Eric Werness, NVIDIA
- Jon Leech, Khronos
- Jeroen van Schijndel, OTOY
- Juul Joosten, OTOY
- Alex Bourd, Qualcomm
- Roman Larionov, Qualcomm
- David McAllister, Qualcomm
- Spencer Fricke, Samsung
- Lewis Gordon, Samsung
- Ralph Potter, Samsung
- Jasper Bekkers, Traverse Research
- Jesse Barker, Unity
- Baldur Karlsson, Valve

Description

Rasterization has been the dominant method to produce interactive graphics, but increasing performance of graphics hardware has made ray tracing a viable option for interactive rendering. Being able to integrate ray tracing with traditional rasterization makes it easier for applications to incrementally add ray traced effects to existing applications or to do hybrid approaches with rasterization for primary visibility and ray tracing for secondary queries.

Ray queries are available to all shader types, including graphics, compute and ray tracing pipelines. Ray queries are not able to launch additional shaders, instead returning traversal results to the calling shader.

This extension adds support for the following SPIR-V extension in Vulkan:

- `SPV_KHR_ray_query`

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceRayQueryFeaturesKHR

New Enum Constants

- VK_KHR_RAY_QUERY_EXTENSION_NAME
- VK_KHR_RAY_QUERY_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_QUERY_FEATURES_KHR

New SPIR-V Capabilities

- RayQueryKHR
- RayTraversalPrimitiveCullingKHR

Sample Code

Example of ray query in a GLSL shader, illustrating how to use ray queries to determine whether a given position (at ray origin) is in shadow or not, by tracing a ray towards the light, and checking for any intersections with geometry occluding the light.

```
rayQueryEXT rq;

rayQueryInitializeEXT(rq, accStruct, gl_RayFlagsTerminateOnFirstHitEXT, cullMask,
origin, tMin, direction, tMax);

// Traverse the acceleration structure and store information about the first
intersection (if any)
rayQueryProceedEXT(rq);

if (rayQueryGetIntersectionTypeEXT(rq, true) ==
gl_RayQueryCommittedIntersectionNoneEXT) {
    // Not in shadow
}
```

Issues

(1) What are the changes between the public provisional (VK_KHR_ray_tracing v8) release and the final (VK_KHR_acceleration_structure v11 / VK_KHR_ray_query v1) release?

- refactor VK_KHR_ray_tracing into 3 extensions, enabling implementation flexibility and decoupling ray query support from ray pipelines:
  - VK_KHR_acceleration_structure (for acceleration structure operations)
  - VK_KHR_ray_tracing_pipeline (for ray tracing pipeline and shader stages)

    - VK_KHR_ray_query (for ray queries in existing shader stages)
- Update SPIRV capabilities to use RayQueryKHR
- extension is no longer provisional

Version History

- Revision 1, 2020-11-12 (Mathieu Robart, Daniel Koch, Andrew Garrard)
    - Decomposition of the specification, from VK_KHR_ray_tracing to VK_KHR_ray_query (#1918,!3912)
    - update to use RayQueryKHR SPIR-V capability
    - add numerical limits for ray parameters (#2235,!3960)
    - relax formula for ray intersection candidate determination (#2322,!4080)
    - restrict traces to TLAS (#2239,!4141)
    - require HitT to be in ray interval for OpRayQueryGenerateIntersectionKHR (#2359,!4146)
    - add ray query shader stages for AS read bit (#2407,!4203)

VK_KHR_ray_tracing_maintenance1

Name String

VK_KHR_ray_tracing_maintenance1

Extension Type

Device extension

Registered Extension Number

387

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_acceleration_structure

API Interactions

- Interacts with VK_VERSION_1_3
- Interacts with VK_EXT_device_generated_commands
- Interacts with VK_KHR_ray_tracing_pipeline
- Interacts with VK_KHR_synchronization2

SPIR-V Dependencies

- SPV_KHR_ray_cull_mask

Contact

- Daniel Koch [GitHub]dgkoch

Other Extension Metadata

Last Modified Date

2022-02-21

Interactions and External Dependencies

- This extension provides API support for `GLSL_EXT_ray_cull_mask`
- Interacts with `VK_KHR_ray_tracing_pipeline`
- Interacts with `VK_KHR_synchronization2`

Contributors

- Stu Smith, AMD
- Tobias Hector, AMD
- Marius Bjorge, Arm
- Tom Olson, Arm
- Yuriy O'Donnell, Epic Games
- Yunpeng Zhu, Huawei
- Andrew Garrard, Imagination
- Dae Kim, Imagination
- Joshua Barczak, Intel
- Lionel Landwerlin, Intel
- Daniel Koch, NVIDIA
- Eric Werness, NVIDIA
- Spencer Fricke, Samsung

Description

`VK_KHR_ray_tracing_maintenance1` adds a collection of minor ray tracing features, none of which would warrant an entire extension of their own.

The new features are as follows:

- Adds support for the `SPV_KHR_ray_cull_mask` SPIR-V extension in Vulkan. This extension provides access to built-in `CullMaskKHR` shader variable which contains the value of the `OpTrace* Cull Mask` parameter. This new shader variable is accessible in the intersection, any-hit, closest hit and miss shader stages.
- Adds support for a new pipeline stage and access mask built on top of `VK_KHR_synchronization2`:
  - `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR` to specify execution of acceleration structure copy commands

    - VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR to specify read access to a shader binding table in any shader pipeline stage
- Adds two new acceleration structure query parameters:
    - VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR to query the acceleration structure size on the device timeline
    - VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR to query the number of bottom level acceleration structure pointers for serialization
- Adds an optional new indirect ray tracing dispatch command, vkCmdTraceRaysIndirect2KHR, which sources the shader binding table parameters as well as the dispatch dimensions from the device. The rayTracingPipelineTraceRaysIndirect2 feature indicates whether this functionality is supported.

New Commands

If VK_KHR_ray_tracing_pipeline is supported:

- vkCmdTraceRaysIndirect2KHR

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
    - VkPhysicalDeviceRayTracingMaintenance1FeaturesKHR

If VK_KHR_ray_tracing_pipeline is supported:

- VkTraceRaysIndirectCommand2KHR

New Enum Constants

- VK_KHR_RAY_TRACING_MAINTENANCE_1_EXTENSION_NAME
- VK_KHR_RAY_TRACING_MAINTENANCE_1_SPEC_VERSION
- Extending VkQueryType:
    - VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SERIALIZATION_BOTTOM_LEVEL_POINTERS_KHR
    - VK_QUERY_TYPE_ACCELERATION_STRUCTURE_SIZE_KHR
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_MAINTENANCE_1_FEATURES_KHR

If VK_KHR_synchronization2 or Vulkan Version 1.3 and VK_KHR_ray_tracing_pipeline is supported:

- Extending VkAccessFlagBits2:
    - VK_ACCESS_2_SHADER_BINDING_TABLE_READ_BIT_KHR

If VK_EXT_device_generated_commands is supported:

- Extending VkIndirectCommandsTokenTypeEXT:

    - VK_INDIRECT_COMMANDS_TOKEN_TYPE_TRACE_RAYS2_EXT

If VK_KHR_synchronization2 or Vulkan Version 1.3 is supported:

- Extending VkPipelineStageFlagBits2:
    - VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_COPY_BIT_KHR

New Built-In Variables

- CullMaskKHR

New SPIR-V Capabilities

- RayCullMaskKHR

Issues

None Yet!

Version History

- Revision 1, 2022-02-21 (Members of the Vulkan Ray Tracing TSG)
    - internal revisions

VK_KHR_ray_tracing_pipeline

Name String

VK_KHR_ray_tracing_pipeline

Extension Type

Device extension

Registered Extension Number

348

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_spirv_1_4
and
VK_KHR_acceleration_structure

API Interactions

- Interacts with VK_KHR_ray_query

SPIR-V Dependencies

- SPV_KHR_ray_tracing

Contact

- Daniel Koch [GitHub]dgkoch

Other Extension Metadata

Last Modified Date

2020-11-12

Interactions and External Dependencies

- This extension provides API support for `GLSL_EXT_ray_tracing`
- This extension interacts with Vulkan 1.2 and `VK_KHR_vulkan_memory_model`, adding the shader-call-related relation of invocations, shader-call-order partial order of dynamic instances of instructions, and the `ShaderCallKHR` scope.
- This extension interacts with `VK_KHR_pipeline_library`, enabling pipeline libraries to be used with ray tracing pipelines and enabling usage of VkRayTracingPipelineInterfaceCreateInfoKHR.

Contributors

- Matthäus Chajdas, AMD
- Greg Grebe, AMD
- Nicolai Hähnle, AMD
- Tobias Hector, AMD
- Dave Oldcorn, AMD
- Skyler Saleh, AMD
- Mathieu Robart, Arm
- Marius Bjorge, Arm
- Tom Olson, Arm
- Sebastian Tafuri, EA
- Henrik Rydgard, Embark
- Juan Cañada, Epic Games
- Patrick Kelly, Epic Games
- Yuriy O'Donnell, Epic Games
- Michael Doggett, Facebook/Oculus
- Andrew Garrard, Imagination
- Don Scorgie, Imagination
- Dae Kim, Imagination
- Joshua Barczak, Intel

- Slawek Grajewski, Intel
- Jeff Bolz, NVIDIA
- Pascal Gautron, NVIDIA
- Daniel Koch, NVIDIA
- Christoph Kubisch, NVIDIA
- Ashwin Lele, NVIDIA
- Robert Stepinski, NVIDIA
- Martin Stich, NVIDIA
- Nuno Subtil, NVIDIA
- Eric Werness, NVIDIA
- Jon Leech, Khronos
- Jeroen van Schijndel, OTOY
- Juul Joosten, OTOY
- Alex Bourd, Qualcomm
- Roman Larionov, Qualcomm
- David McAllister, Qualcomm
- Spencer Fricke, Samsung
- Lewis Gordon, Samsung
- Ralph Potter, Samsung
- Jasper Bekkers, Traverse Research
- Jesse Barker, Unity
- Baldur Karlsson, Valve

Description

Rasterization has been the dominant method to produce interactive graphics, but increasing performance of graphics hardware has made ray tracing a viable option for interactive rendering. Being able to integrate ray tracing with traditional rasterization makes it easier for applications to incrementally add ray traced effects to existing applications or to do hybrid approaches with rasterization for primary visibility and ray tracing for secondary queries.

To enable ray tracing, this extension adds a few different categories of new functionality:

- A new ray tracing pipeline type with new shader domains: ray generation, intersection, any-hit, closest hit, miss, and callable
- A shader binding indirection table to link shader groups with acceleration structure items
- Ray tracing commands which initiate the ray pipeline traversal and invocation of the various new shader domains depending on which traversal conditions are met

This extension adds support for the following SPIR-V extension in Vulkan:

- SPV_KHR_ray_tracing

New Commands

- vkCmdSetRayTracingPipelineStackSizeKHR
- vkCmdTraceRaysIndirectKHR
- vkCmdTraceRaysKHR
- vkCreateRayTracingPipelinesKHR
- vkGetRayTracingCaptureReplayShaderGroupHandlesKHR
- vkGetRayTracingShaderGroupHandlesKHR
- vkGetRayTracingShaderGroupStackSizeKHR

New Structures

- VkRayTracingPipelineCreateInfoKHR
- VkRayTracingPipelineInterfaceCreateInfoKHR
- VkRayTracingShaderGroupCreateInfoKHR
- VkStridedDeviceAddressRegionKHR
- VkTraceRaysIndirectCommandKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceRayTracingPipelineFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceRayTracingPipelinePropertiesKHR

New Enums

- VkRayTracingShaderGroupTypeKHR
- VkShaderGroupShaderKHR

New Enum Constants

- VK_KHR_RAY_TRACING_PIPELINE_EXTENSION_NAME
- VK_KHR_RAY_TRACING_PIPELINE_SPEC_VERSION
- VK_SHADER_UNUSED_KHR
- Extending VkBufferUsageFlagBits:
  - VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR
- Extending VkDynamicState:
  - VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR
- Extending VkPipelineBindPoint:
  - VK_PIPELINE_BIND_POINT_RAY_TRACING_KHR

- Extending VkPipelineCreateFlagBits:
  - VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_ANY_HIT_SHADERS_BIT_KHR
  - VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_CLOSEST_HIT_SHADERS_BIT_KHR
  - VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_INTERSECTION_SHADERS_BIT_KHR
  - VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_MISS_SHADERS_BIT_KHR
  - VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR
  - VK_PIPELINE_CREATE_RAY_TRACING_SKIP_AABBS_BIT_KHR
  - VK_PIPELINE_CREATE_RAY_TRACING_SKIP_TRIANGLES_BIT_KHR
- Extending VkPipelineStageFlagBits:
  - VK_PIPELINE_STAGE_RAY_TRACING_SHADER_BIT_KHR
- Extending VkShaderStageFlagBits:
  - VK_SHADER_STAGE_ANY_HIT_BIT_KHR
  - VK_SHADER_STAGE_CALLABLE_BIT_KHR
  - VK_SHADER_STAGE_CLOSEST_HIT_BIT_KHR
  - VK_SHADER_STAGE_INTERSECTION_BIT_KHR
  - VK_SHADER_STAGE_MISS_BIT_KHR
  - VK_SHADER_STAGE_RAYGEN_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_PIPELINE_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_PIPELINE_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_RAY_TRACING_PIPELINE_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_RAY_TRACING_PIPELINE_INTERFACE_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_RAY_TRACING_SHADER_GROUP_CREATE_INFO_KHR

New or Modified Built-In Variables

- LaunchIdKHR
- LaunchSizeKHR
- WorldRayOriginKHR
- WorldRayDirectionKHR
- ObjectRayOriginKHR
- ObjectRayDirectionKHR
- RayTminKHR
- RayTmaxKHR
- InstanceCustomIndexKHR
- InstanceId

- `ObjectToWorldKHR`
- `WorldToObjectKHR`
- `HitKindKHR`
- `IncomingRayFlagsKHR`
- `RayGeometryIndexKHR`
- (modified)`PrimitiveId`

New SPIR-V Capabilities

- `RayTracingKHR`
- `RayTraversalPrimitiveCullingKHR`

Issues

(1) How does this extension differ from VK_NV_ray_tracing?

DISCUSSION:

The following is a summary of the main functional differences between VK_KHR_ray_tracing_pipeline and VK_NV_ray_tracing:

- added support for indirect ray tracing (vkCmdTraceRaysIndirectKHR)
- uses SPV_KHR_ray_tracing instead of SPV_NV_ray_tracing
  - refer to KHR SPIR-V enums instead of NV SPIR-V enums (which are functionally equivalent and aliased to the same values).
  - added `RayGeometryIndexKHR` built-in
- removed vkCompileDeferredNV compilation functionality and replaced with deferred host operations interactions for ray tracing
- added VkPhysicalDeviceRayTracingPipelineFeaturesKHR structure
- extended VkPhysicalDeviceRayTracingPipelinePropertiesKHR structure
  - renamed `maxRecursionDepth` to `maxRayRecursionDepth` and it has a minimum of 1 instead of 31
  - require `shaderGroupHandleSize` to be 32 bytes
  - added `maxRayDispatchInvocationCount`, `shaderGroupHandleAlignment` and `maxRayHitAttributeSize`
- reworked geometry structures so they could be better shared between device, host, and indirect builds
- changed SBT parameters to a structure and added size (VkStridedDeviceAddressRegionKHR)
- add parameter for requesting memory requirements for host and/or device build
- added pipeline library support for ray tracing
- added watertightness guarantees
- added no-null-shader pipeline flags (`VK_PIPELINE_CREATE_RAY_TRACING_NO_NULL_*_SHADERS_BIT_KHR`)

- added memory model interactions with ray tracing and define how subgroups work and can be repacked

(3) What are the changes between the public provisional (VK_KHR_ray_tracing v8) release and the internal provisional (VK_KHR_ray_tracing v9) release?

- Require Vulkan 1.1 and SPIR-V 1.4
- Added interactions with Vulkan 1.2 and `VK_KHR_vulkan_memory_model`
- added creation time capture and replay flags
  - added `VK_PIPELINE_CREATE_RAY_TRACING_SHADER_GROUP_HANDLE_CAPTURE_REPLAY_BIT_KHR` to VkPipelineCreateFlagBits
- replace `VkStridedBufferRegionKHR` with VkStridedDeviceAddressRegionKHR and change vkCmdTraceRaysKHR, vkCmdTraceRaysIndirectKHR, to take these for the shader binding table and use device addresses instead of buffers.
- require the shader binding table buffers to have the `VK_BUFFER_USAGE_RAY_TRACING_BIT_KHR` set
- make `VK_KHR_pipeline_library` an interaction instead of required extension
- rename the `libraries` member of VkRayTracingPipelineCreateInfoKHR to `pLibraryInfo` and make it a pointer
- make `VK_KHR_deferred_host_operations` an interaction instead of a required extension (later went back on this)
- added explicit stack size management for ray tracing pipelines
  - removed the `maxCallableSize` member of VkRayTracingPipelineInterfaceCreateInfoKHR
  - added the `pDynamicState` member to VkRayTracingPipelineCreateInfoKHR
  - added `VK_DYNAMIC_STATE_RAY_TRACING_PIPELINE_STACK_SIZE_KHR` dynamic state for ray tracing pipelines
  - added vkGetRayTracingShaderGroupStackSizeKHR and vkCmdSetRayTracingPipelineStackSizeKHR commands
  - added VkShaderGroupShaderKHR enum
- Added `maxRayDispatchInvocationCount` limit to VkPhysicalDeviceRayTracingPipelinePropertiesKHR
- Added `shaderGroupHandleAlignment` property to VkPhysicalDeviceRayTracingPipelinePropertiesKHR
- Added `maxRayHitAttributeSize` property to VkPhysicalDeviceRayTracingPipelinePropertiesKHR
- Clarify deferred host ops for pipeline creation
  - VkDeferredOperationKHR is now a top-level parameter for vkCreateRayTracingPipelinesKHR
  - removed `VkDeferredOperationInfoKHR` structure
  - change deferred host creation/return parameter behavior such that the implementation can modify such parameters until the deferred host operation completes
  - `VK_KHR_deferred_host_operations` is required again

(4) What are the changes between the internal provisional (VK_KHR_ray_tracing v9) release and the final (VK_KHR_acceleration_structure v11 / VK_KHR_ray_tracing_pipeline v1) release?

- refactor VK_KHR_ray_tracing into 3 extensions, enabling implementation flexibility and decoupling ray query support from ray pipelines:
  - `VK_KHR_acceleration_structure` (for acceleration structure operations)
  - `VK_KHR_ray_tracing_pipeline` (for ray tracing pipeline and shader stages)
  - `VK_KHR_ray_query` (for ray queries in existing shader stages)
- Require `Volatile` for the following builtins in the ray generation, closest hit, miss, intersection, and callable shader stages:
  - `SubgroupSize`, `SubgroupLocalInvocationId`, `SubgroupEqMask`, `SubgroupGeMask`, `SubgroupGtMask`, `SubgroupLeMask`, `SubgroupLtMask`
  - `SMIDNV`, `WarpIDNV`
- clarify buffer usage flags for ray tracing
  - `VK_BUFFER_USAGE_SHADER_BINDING_TABLE_BIT_KHR` is added as an alias of `VK_BUFFER_USAGE_RAY_TRACING_BIT_NV` and is required on shader binding table buffers
  - `VK_BUFFER_USAGE_STORAGE_BUFFER_BIT` is used in `VK_KHR_acceleration_structure` for `scratchData`
- rename `maxRecursionDepth` to `maxRayPipelineRecursionDepth` (pipeline creation) and `maxRayRecursionDepth` (limit) to reduce confusion
- Add queryable `maxRayHitAttributeSize` limit and rename members of VkRayTracingPipelineInterfaceCreateInfoKHR to `maxPipelineRayPayloadSize` and `maxPipelineRayHitAttributeSize` for clarity
- Update SPIRV capabilities to use `RayTracingKHR`
- extension is no longer provisional
- define synchronization requirements for indirect trace rays and indirect buffer

(5) This extension adds gl_InstanceID for the intersection, any-hit, and closest hit shaders, but in KHR_vulkan_glsl, gl_InstanceID is replaced with gl_InstanceIndex. Which should be used for Vulkan in this extension?

RESOLVED: This extension uses gl_InstanceID and maps it to `InstanceId` in SPIR-V. It is acknowledged that this is different than other shader stages in Vulkan. There are two main reasons for the difference here:

- symmetry with gl_PrimitiveID which is also available in these shaders
- there is no “baseInstance” relevant for these shaders, and so ID makes it more obvious that this is zero-based.

(6) Why is `VK_KHR_pipeline_library` an interaction instead of a required dependency, particularly when the “Feature Requirements” section says it is required to be supported anyhow?

RESOLVED: If the `VK_KHR_pipeline_library` extensino were a required dependency, then every application would need to enable the extension whether or not they actually want to use the

pipeline library functionality. Developers found this to be annoying and unfriendly behavior. We do wish to require all **implementations** to support it though, and thus it is listed in the feature requirements section.

Sample Code

Example ray generation GLSL shader

```
#version 450 core
#extension GL_EXT_ray_tracing : require
layout(set = 0, binding = 0, rgba8) uniform image2D image;
layout(set = 0, binding = 1) uniform accelerationStructureEXT as;
layout(location = 0) rayPayloadEXT float payload;

void main()
{
   vec4 col = vec4(0, 0, 0, 1);

   vec3 origin = vec3(float(gl_LaunchIDEXT.x)/float(gl_LaunchSizeEXT.x), float
(gl_LaunchIDEXT.y)/float(gl_LaunchSizeEXT.y), 1.0);
   vec3 dir = vec3(0.0, 0.0, -1.0);

   traceRayEXT(as, 0, 0xff, 0, 1, 0, origin, 0.0, dir, 1000.0, 0);

   col.y = payload;

   imageStore(image, ivec2(gl_LaunchIDEXT.xy), col);
}
```

Version History

- Revision 1, 2020-11-12 (Mathieu Robart, Daniel Koch, Eric Werness, Tobias Hector)
  - Decomposition of the specification, from VK_KHR_ray_tracing to VK_KHR_ray_tracing_pipeline (#1918,!3912)
  - require certain subgroup and sm_shader_builtin shader builtins to be decorated as volatile in the ray generation, closest hit, miss, intersection, and callable stages (#1924,!3903,!3954)
  - clarify buffer usage flags for ray tracing (#2181,!3939)
  - rename maxRecursionDepth to maxRayPipelineRecursionDepth and maxRayRecursionDepth (#2203,!3937)
  - add queryable maxRayHitAttributeSize and rename members of VkRayTracingPipelineInterfaceCreateInfoKHR (#2102,!3966)
  - update to use `RayTracingKHR` SPIR-V capability
  - add VUs for matching hit group type against geometry type (#2245,!3994)
  - require `RayTMaxKHR` be volatile in intersection shaders (#2268,!4030)
  - add numerical limits for ray parameters (#2235,!3960)

- fix SBT indexing rules for device addresses (#2308,!4079)
- relax formula for ray intersection candidate determination (#2322,!4080)
- add more details on `ShaderRecordBufferKHR` variables (#2230,!4083)
- clarify valid bits for `InstanceCustomIndexKHR` (GLSL/GLSL#19,!4128)
- allow at most one `IncomingRayPayloadKHR`, `IncomingCallableDataKHR`, and `HitAttributeKHR` (!4129)
- add minimum for maxShaderGroupStride (#2353,!4131)
- require VK_KHR_pipeline_library extension to be supported (#2348,!4135)
- clarify meaning of 'geometry index' (#2272,!4137)
- restrict traces to TLAS (#2239,!4141)
- add note about maxPipelineRayPayloadSize (#2383,!4172)
- do not require raygen shader in pipeline libraries (!4185)
- define sync for indirect trace rays and indirect buffer (#2407,!4208)

VK_KHR_ray_tracing_position_fetch

Name String

`VK_KHR_ray_tracing_position_fetch`

Extension Type

Device extension

Registered Extension Number

482

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_acceleration_structure

SPIR-V Dependencies

- SPV_KHR_ray_tracing_position_fetch

Contact

- Eric Werness

Extension Proposal

VK_KHR_ray_tracing_position_fetch

Other Extension Metadata

Last Modified Date

2023-02-17

Interactions and External Dependencies

- This extension provides API support for `GLSL_EXT_ray_tracing_position_fetch`
- Interacts with `VK_KHR_ray_tracing_pipeline`
- Interacts with `VK_KHR_ray_query`

Contributors

- Eric Werness, NVIDIA
- Stu Smith, AMD
- Yuriy O'Donnell, Epic Games
- Ralph Potter, Samsung
- Joshua Barczak, Intel
- Lionel Landwerlin, Intel
- Andrew Garrard, Imagination Technologies
- Alex Bourd, Qualcomm
- Yunpeng Zhu, Huawei Technologies
- Marius Bjorge, Arm
- Daniel Koch, NVIDIA

Description

`VK_KHR_ray_tracing_position_fetch` adds the ability to fetch the vertex positions in the shader from a hit triangle as stored in the acceleration structure.

An application adds `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DATA_ACCESS_KHR` to the acceleration structure at build time. Then, if the hit is a triangle geometry, the shader (any-hit or closest hit for ray pipelines or using ray query) **can** fetch the three, three-component vertex positions in object space, of the triangle which was hit.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceRayTracingPositionFetchFeaturesKHR

New Enum Constants

- `VK_KHR_RAY_TRACING_POSITION_FETCH_EXTENSION_NAME`
- `VK_KHR_RAY_TRACING_POSITION_FETCH_SPEC_VERSION`
- Extending VkBuildAccelerationStructureFlagBitsKHR:

    - VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DATA_ACCESS_KHR
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_RAY_TRACING_POSITION_FETCH_FEATURES_KHR

New Built-In Variables

- HitTriangleVertexPositionsKHR

New SPIR-V Capabilities

- RayTracingPositionFetchKHR
- RayQueryPositionFetchKHR

Issues

None Yet!

Version History

- Revision 1, 2023-02-17 (Eric Werness)
    - internal revisions

VK_KHR_relaxed_block_layout

Name String

VK_KHR_relaxed_block_layout

Extension Type

Device extension

Registered Extension Number

145

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- John Kessenich [GitHub]johnkslang

Other Extension Metadata

Last Modified Date

2017-03-26

IP Status

No known IP claims.

Contributors

- John Kessenich, Google

Description

The `VK_KHR_relaxed_block_layout` extension allows implementations to indicate they can support more variation in block `Offset` decorations. For example, placing a vector of three floats at an offset of 16×N + 4.

See Offset and Stride Assignment for details.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Enum Constants

- `VK_KHR_RELAXED_BLOCK_LAYOUT_EXTENSION_NAME`
- `VK_KHR_RELAXED_BLOCK_LAYOUT_SPEC_VERSION`

Version History

- Revision 1, 2017-03-26 (JohnK)

VK_KHR_sampler_mirror_clamp_to_edge

Name String

`VK_KHR_sampler_mirror_clamp_to_edge`

Extension Type

Device extension

Registered Extension Number

15

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Tobias Hector [GitHub]tobski

Other Extension Metadata

Last Modified Date

2019-08-17

Contributors

- Tobias Hector, Imagination Technologies
- Jon Leech, Khronos

Description

`VK_KHR_sampler_mirror_clamp_to_edge` extends the set of sampler address modes to include an additional mode (`VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE`) that effectively uses a texture map twice as large as the original image in which the additional half of the new image is a mirror image of the original image.

This new mode relaxes the need to generate images whose opposite edges match by using the original image to generate a matching "mirror image". This mode allows the texture to be mirrored only once in the negative s, t, and r directions.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2. However, if Vulkan 1.2 is supported and this extension is not, the VkSamplerAddressMode `VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE` is optional. Since the original extension did not use an author suffix on the enum `VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE`, it is used by both core and extension implementations.

New Enum Constants

- `VK_KHR_SAMPLER_MIRROR_CLAMP_TO_EDGE_EXTENSION_NAME`
- `VK_KHR_SAMPLER_MIRROR_CLAMP_TO_EDGE_SPEC_VERSION`
- Extending VkSamplerAddressMode:
  - `VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE`
  - `VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE_KHR`

Example

Creating a sampler with the new address mode in each dimension

```
    VkSamplerCreateInfo createInfo =
    {
        .sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO,
        // Other members set to application-desired values
    };

    createInfo.addressModeU = VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE;
    createInfo.addressModeV = VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE;
    createInfo.addressModeW = VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE;

    VkSampler sampler;
    VkResult result = vkCreateSampler(
        device,
        &createInfo,
        &sampler);
```

Issues

1) Why are both KHR and core versions of the `VK_SAMPLER_ADDRESS_MODE_MIRROR_CLAMP_TO_EDGE` token present?

RESOLVED: This functionality was intended to be required in Vulkan 1.0. We realized shortly before public release that not all implementations could support it, and moved the functionality into an optional extension, but did not apply the KHR extension suffix. Adding a KHR-suffixed alias of the non-suffixed enum has been done to comply with our own naming rules.

In a related change, before spec revision 1.1.121 this extension was hardwiring into the spec Makefile so it was always included with the Specification, even in the core-only versions. This has now been reverted, and it is treated as any other extension.

Version History

- Revision 1, 2016-02-16 (Tobias Hector)
  - Initial draft
- Revision 2, 2019-08-14 (Jon Leech)
  - Add KHR-suffixed alias of non-suffixed enum.
- Revision 3, 2019-08-17 (Jon Leech)
  - Add an issue explaining the reason for the extension API not being suffixed with KHR.

VK_KHR_sampler_ycbcr_conversion

Name String

`VK_KHR_sampler_ycbcr_conversion`

Extension Type

Device extension

Registered Extension Number

157

Revision

14

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_maintenance1
and
VK_KHR_bind_memory2
and
VK_KHR_get_memory_requirements2
and
VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

API Interactions

- Interacts with VK_EXT_debug_report

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Andrew Garrard [GitHub]fluppeteer

Other Extension Metadata

Last Modified Date

2017-08-11

IP Status

No known IP claims.

Contributors

- Andrew Garrard, Samsung Electronics
- Tobias Hector, Imagination Technologies
- James Jones, NVIDIA
- Daniel Koch, NVIDIA
- Daniel Rakos, AMD
- Romain Guy, Google
- Jesse Hall, Google
- Tom Cooksey, ARM Ltd

- Jeff Leger, Qualcomm Technologies, Inc
- Jan-Harald Fredriksen, ARM Ltd
- Jan Outters, Samsung Electronics
- Alon Or-bach, Samsung Electronics
- Michael Worcester, Imagination Technologies
- Jeff Bolz, NVIDIA
- Tony Zlatinski, NVIDIA
- Matthew Netsch, Qualcomm Technologies, Inc

Description

The use of $Y'C_BC_R$ sampler conversion is an area in 3D graphics not used by most Vulkan developers. It is mainly used for processing inputs from video decoders and cameras. The use of the extension assumes basic knowledge of $Y'C_BC_R$ concepts.

This extension provides the ability to perform specified color space conversions during texture sampling operations for the $Y'C_BC_R$ color space natively. It also adds a selection of multi-planar formats, image aspect plane, and the ability to bind memory to the planes of an image collectively or separately.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted. However, if Vulkan 1.1 is supported and this extension is not, the `samplerYcbcrConversion` capability is optional. The original type, enum, and command names are still available as aliases of the core functionality.

New Object Types

- VkSamplerYcbcrConversionKHR

New Commands

- vkCreateSamplerYcbcrConversionKHR
- vkDestroySamplerYcbcrConversionKHR

New Structures

- VkSamplerYcbcrConversionCreateInfoKHR
- Extending VkBindImageMemoryInfo:
  - VkBindImagePlaneMemoryInfoKHR
- Extending VkImageFormatProperties2:
  - VkSamplerYcbcrConversionImageFormatPropertiesKHR
- Extending VkImageMemoryRequirementsInfo2:

    - VkImagePlaneMemoryRequirementsInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
    - VkPhysicalDeviceSamplerYcbcrConversionFeaturesKHR
- Extending VkSamplerCreateInfo, VkImageViewCreateInfo:
    - VkSamplerYcbcrConversionInfoKHR

New Enums

- VkChromaLocationKHR
- VkSamplerYcbcrModelConversionKHR
- VkSamplerYcbcrRangeKHR

New Enum Constants

- VK_KHR_SAMPLER_YCBCR_CONVERSION_EXTENSION_NAME
- VK_KHR_SAMPLER_YCBCR_CONVERSION_SPEC_VERSION
- Extending VkChromaLocation:
    - VK_CHROMA_LOCATION_COSITED_EVEN_KHR
    - VK_CHROMA_LOCATION_MIDPOINT_KHR
- Extending VkFormat:
    - VK_FORMAT_B10X6G10X6R10X6G10X6_422_UNORM_4PACK16_KHR
    - VK_FORMAT_B12X4G12X4R12X4G12X4_422_UNORM_4PACK16_KHR
    - VK_FORMAT_B16G16R16G16_422_UNORM_KHR
    - VK_FORMAT_B8G8R8G8_422_UNORM_KHR
    - VK_FORMAT_G10X6B10X6G10X6R10X6_422_UNORM_4PACK16_KHR
    - VK_FORMAT_G10X6_B10X6R10X6_2PLANE_420_UNORM_3PACK16_KHR
    - VK_FORMAT_G10X6_B10X6R10X6_2PLANE_422_UNORM_3PACK16_KHR
    - VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_420_UNORM_3PACK16_KHR
    - VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_422_UNORM_3PACK16_KHR
    - VK_FORMAT_G10X6_B10X6_R10X6_3PLANE_444_UNORM_3PACK16_KHR
    - VK_FORMAT_G12X4B12X4G12X4R12X4_422_UNORM_4PACK16_KHR
    - VK_FORMAT_G12X4_B12X4R12X4_2PLANE_420_UNORM_3PACK16_KHR
    - VK_FORMAT_G12X4_B12X4R12X4_2PLANE_422_UNORM_3PACK16_KHR
    - VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_420_UNORM_3PACK16_KHR
    - VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_422_UNORM_3PACK16_KHR
    - VK_FORMAT_G12X4_B12X4_R12X4_3PLANE_444_UNORM_3PACK16_KHR
    - VK_FORMAT_G16B16G16R16_422_UNORM_KHR

    - VK_FORMAT_G16_B16R16_2PLANE_420_UNORM_KHR
    - VK_FORMAT_G16_B16R16_2PLANE_422_UNORM_KHR
    - VK_FORMAT_G16_B16_R16_3PLANE_420_UNORM_KHR
    - VK_FORMAT_G16_B16_R16_3PLANE_422_UNORM_KHR
    - VK_FORMAT_G16_B16_R16_3PLANE_444_UNORM_KHR
    - VK_FORMAT_G8B8G8R8_422_UNORM_KHR
    - VK_FORMAT_G8_B8R8_2PLANE_420_UNORM_KHR
    - VK_FORMAT_G8_B8R8_2PLANE_422_UNORM_KHR
    - VK_FORMAT_G8_B8_R8_3PLANE_420_UNORM_KHR
    - VK_FORMAT_G8_B8_R8_3PLANE_422_UNORM_KHR
    - VK_FORMAT_G8_B8_R8_3PLANE_444_UNORM_KHR
    - VK_FORMAT_R10X6G10X6B10X6A10X6_UNORM_4PACK16_KHR
    - VK_FORMAT_R10X6G10X6_UNORM_2PACK16_KHR
    - VK_FORMAT_R10X6_UNORM_PACK16_KHR
    - VK_FORMAT_R12X4G12X4B12X4A12X4_UNORM_4PACK16_KHR
    - VK_FORMAT_R12X4G12X4_UNORM_2PACK16_KHR
    - VK_FORMAT_R12X4_UNORM_PACK16_KHR
- Extending VkFormatFeatureFlagBits:
    - VK_FORMAT_FEATURE_COSITED_CHROMA_SAMPLES_BIT_KHR
    - VK_FORMAT_FEATURE_DISJOINT_BIT_KHR
    - VK_FORMAT_FEATURE_MIDPOINT_CHROMA_SAMPLES_BIT_KHR
    - VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_BIT_KHR
    - VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_CHROMA_RECONSTRUCTION_EXPLICIT_FORCEABLE_BIT_KHR
    - VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_LINEAR_FILTER_BIT_KHR
    - VK_FORMAT_FEATURE_SAMPLED_IMAGE_YCBCR_CONVERSION_SEPARATE_RECONSTRUCTION_FILTER_BIT_KHR
- Extending VkImageAspectFlagBits:
    - VK_IMAGE_ASPECT_PLANE_0_BIT_KHR
    - VK_IMAGE_ASPECT_PLANE_1_BIT_KHR
    - VK_IMAGE_ASPECT_PLANE_2_BIT_KHR
- Extending VkImageCreateFlagBits:
    - VK_IMAGE_CREATE_DISJOINT_BIT_KHR
- Extending VkObjectType:
    - VK_OBJECT_TYPE_SAMPLER_YCBCR_CONVERSION_KHR
- Extending VkSamplerYcbcrModelConversion:

    - VK_SAMPLER_YCBCR_MODEL_CONVERSION_RGB_IDENTITY_KHR
    - VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_2020_KHR
    - VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_601_KHR
    - VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_709_KHR
    - VK_SAMPLER_YCBCR_MODEL_CONVERSION_YCBCR_IDENTITY_KHR
- Extending VkSamplerYcbcrRange:
    - VK_SAMPLER_YCBCR_RANGE_ITU_FULL_KHR
    - VK_SAMPLER_YCBCR_RANGE_ITU_NARROW_KHR
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_BIND_IMAGE_PLANE_MEMORY_INFO_KHR
    - VK_STRUCTURE_TYPE_IMAGE_PLANE_MEMORY_REQUIREMENTS_INFO_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SAMPLER_YCBCR_CONVERSION_FEATURES_KHR
    - VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_CREATE_INFO_KHR
    - VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_IMAGE_FORMAT_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_SAMPLER_YCBCR_CONVERSION_INFO_KHR

If VK_EXT_debug_report is supported:

- Extending VkDebugReportObjectTypeEXT:
    - VK_DEBUG_REPORT_OBJECT_TYPE_SAMPLER_YCBCR_CONVERSION_EXT
    - VK_DEBUG_REPORT_OBJECT_TYPE_SAMPLER_YCBCR_CONVERSION_KHR_EXT

Version History

- Revision 1, 2017-01-24 (Andrew Garrard)
    - Initial draft
- Revision 2, 2017-01-25 (Andrew Garrard)
    - After initial feedback
- Revision 3, 2017-01-27 (Andrew Garrard)
    - Higher bit depth formats, renaming, swizzle
- Revision 4, 2017-02-22 (Andrew Garrard)
    - Added query function, formats as RGB, clarifications
- Revision 5, 2017-04-?? (Andrew Garrard)
    - Simplified query and removed output conversions
- Revision 6, 2017-04-24 (Andrew Garrard)
    - Tidying, incorporated new image query, restored transfer functions
- Revision 7, 2017-04-25 (Andrew Garrard)

    - Added cosited option/midpoint requirement for formats, “bypassConversion”
- Revision 8, 2017-04-25 (Andrew Garrard)
    - Simplified further
- Revision 9, 2017-04-27 (Andrew Garrard)
    - Disjoint no more
- Revision 10, 2017-04-28 (Andrew Garrard)
    - Restored disjoint
- Revision 11, 2017-04-29 (Andrew Garrard)
    - Now Ycbcr conversion, and KHR
- Revision 12, 2017-06-06 (Andrew Garrard)
    - Added conversion to image view creation
- Revision 13, 2017-07-13 (Andrew Garrard)
    - Allowed cosited-only chroma samples for formats
- Revision 14, 2017-08-11 (Andrew Garrard)
    - Reflected quantization changes in BT.2100-1

VK_KHR_separate_depth_stencil_layouts

Name String

`VK_KHR_separate_depth_stencil_layouts`

Extension Type

Device extension

Registered Extension Number

242

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
VK_KHR_create_renderpass2
or
Vulkan Version 1.2

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2019-06-25

Contributors

- Daniel Koch, NVIDIA
- Jeff Bolz, NVIDIA
- Jesse Barker, Unity
- Tobias Hector, AMD

Description

This extension allows image memory barriers for depth/stencil images to have just one of the `VK_IMAGE_ASPECT_DEPTH_BIT` or `VK_IMAGE_ASPECT_STENCIL_BIT` aspect bits set, rather than require both. This allows their layouts to be set independently. To support depth/stencil images with different layouts for the depth and stencil aspects, the depth/stencil attachment interface has been updated to support a separate layout for stencil.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkAttachmentDescription2:
  - VkAttachmentDescriptionStencilLayoutKHR
- Extending VkAttachmentReference2:
  - VkAttachmentReferenceStencilLayoutKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceSeparateDepthStencilLayoutsFeaturesKHR

New Enum Constants

- `VK_KHR_SEPARATE_DEPTH_STENCIL_LAYOUTS_EXTENSION_NAME`
- `VK_KHR_SEPARATE_DEPTH_STENCIL_LAYOUTS_SPEC_VERSION`
- Extending VkImageLayout:
  - `VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL_KHR`

    - VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL_KHR
    - VK_IMAGE_LAYOUT_STENCIL_ATTACHMENT_OPTIMAL_KHR
    - VK_IMAGE_LAYOUT_STENCIL_READ_ONLY_OPTIMAL_KHR
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_ATTACHMENT_DESCRIPTION_STENCIL_LAYOUT_KHR
    - VK_STRUCTURE_TYPE_ATTACHMENT_REFERENCE_STENCIL_LAYOUT_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SEPARATE_DEPTH_STENCIL_LAYOUTS_FEATURES_KHR

Version History

- Revision 1, 2019-06-25 (Piers Daniell)
    - Internal revisions

VK_KHR_shader_atomic_int64

Name String

VK_KHR_shader_atomic_int64

Extension Type

Device extension

Registered Extension Number

181

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Aaron Hagan [GitHub]ahagan

Other Extension Metadata

Last Modified Date

2018-07-05

Interactions and External Dependencies

- This extension provides API support for `GL_ARB_gpu_shader_int64` and `GL_EXT_shader_atomic_int64`

Contributors

- Aaron Hagan, AMD
- Daniel Rakos, AMD
- Jeff Bolz, NVIDIA
- Neil Henning, Codeplay

Description

This extension advertises the SPIR-V **Int64Atomics** capability for Vulkan, which allows a shader to contain 64-bit atomic operations on signed and unsigned integers. The supported operations include OpAtomicMin, OpAtomicMax, OpAtomicAnd, OpAtomicOr, OpAtomicXor, OpAtomicAdd, OpAtomicExchange, and OpAtomicCompareExchange.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. However, if Vulkan 1.2 is supported and this extension is not, the `shaderBufferInt64Atomics` capability is optional. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderAtomicInt64FeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_ATOMIC_INT64_EXTENSION_NAME`
- `VK_KHR_SHADER_ATOMIC_INT64_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_INT64_FEATURES_KHR`

New SPIR-V Capabilities

- `Int64Atomics`

Version History

- Revision 1, 2018-07-05 (Aaron Hagan)
  - Internal revisions

VK_KHR_shader_clock

Name String

`VK_KHR_shader_clock`

Extension Type

Device extension

Registered Extension Number

182

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_shader_clock

Contact

- Aaron Hagan [GitHub]ahagan

Other Extension Metadata

Last Modified Date

2019-4-25

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_ARB_shader_clock` and `GL_EXT_shader_realtime_clock`

Contributors

- Aaron Hagan, AMD
- Daniel Koch, NVIDIA

Description

This extension advertises the SPIR-V `ShaderClockKHR` capability for Vulkan, which allows a shader to query a real-time or monotonically incrementing counter at the subgroup level or across the device level. The two valid SPIR-V scopes for `OpReadClockKHR` are `Subgroup` and `Device`.

When using GLSL source-based shading languages, the `clockRealtime*EXT`() timing functions map to the `OpReadClockKHR` instruction with a scope of `Device`, and the `clock*ARB`() timing functions map to the `OpReadClockKHR` instruction with a scope of `Subgroup`.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderClockFeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_CLOCK_EXTENSION_NAME`
- `VK_KHR_SHADER_CLOCK_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_CLOCK_FEATURES_KHR`

New SPIR-V Capabilities

- `ShaderClockKHR`

Version History

- Revision 1, 2019-4-25 (Aaron Hagan)
  - Initial revision

VK_KHR_shader_draw_parameters

Name String

`VK_KHR_shader_draw_parameters`

Extension Type

Device extension

Registered Extension Number

64

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

SPIR-V Dependencies

- SPV_KHR_shader_draw_parameters

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Daniel Koch [GitHub]dgkoch

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_ARB_shader_draw_parameters`

Contributors

- Daniel Koch, NVIDIA Corporation
- Jeff Bolz, NVIDIA
- Daniel Rakos, AMD
- Jan-Harald Fredriksen, ARM
- John Kessenich, Google
- Stuart Smith, IMG

Description

This extension adds support for the following SPIR-V extension in Vulkan:

- `SPV_KHR_shader_draw_parameters`

The extension provides access to three additional built-in shader variables in Vulkan:

- `BaseInstance`, containing the `firstInstance` parameter passed to drawing commands,
- `BaseVertex`, containing the `firstVertex` or `vertexOffset` parameter passed to drawing commands, and
- `DrawIndex`, containing the index of the draw call currently being processed from an indirect drawing call.

When using GLSL source-based shader languages, the following variables from `GL_ARB_shader_draw_parameters` can map to these SPIR-V built-in decorations:

- `in int gl_BaseInstanceARB;` → `BaseInstance`,
- `in int gl_BaseVertexARB;` → `BaseVertex`, and
- `in int gl_DrawIDARB;` → `DrawIndex`.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1. However, the `shaderDrawParameters` feature bit was added to distinguish whether it is actually available or not.

New Enum Constants

- `VK_KHR_SHADER_DRAW_PARAMETERS_EXTENSION_NAME`
- `VK_KHR_SHADER_DRAW_PARAMETERS_SPEC_VERSION`

New Built-In Variables

- `BaseInstance`
- `BaseVertex`
- `DrawIndex`

New SPIR-V Capabilities

- `DrawParameters`

Issues

1) Is this the same functionality as `GL_ARB_shader_draw_parameters`?

RESOLVED: It is actually a superset, as it also adds in support for arrayed drawing commands.

In GL for `GL_ARB_shader_draw_parameters`, `gl_BaseVertexARB` holds the integer value passed to the parameter to the command that resulted in the current shader invocation. In the case where the command has no `baseVertex` parameter, the value of `gl_BaseVertexARB` is zero. This means that `gl_BaseVertexARB` = `baseVertex` (for `glDrawElements` commands with `baseVertex`) or 0. In particular there are no `glDrawArrays` commands that take a `baseVertex` parameter.

Now in Vulkan, we have `BaseVertex` = `vertexOffset` (for indexed drawing commands) or `firstVertex` (for arrayed drawing commands), and so Vulkan's version is really a superset of GL functionality.

Version History

- Revision 1, 2016-10-05 (Daniel Koch)
  - Internal revisions

VK_KHR_shader_expect_assume

Name String

`VK_KHR_shader_expect_assume`

Extension Type

Device extension

Registered Extension Number

545

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_expect_assume

Contact

- Kevin Petit [GitHub]kpet

Extension Proposal

VK_KHR_shader_expect_assume

Other Extension Metadata

Last Modified Date

2023-12-06

IP Status

No known IP claims.

Contributors

- Kevin Petit, Arm
- Tobias Hector, AMD
- James Fitzpatrick, Imagination Technologies

Description

This extension allows the use of the `SPV_KHR_expect_assume` extension in SPIR-V shader modules which enables SPIR-V producers to provide optimization hints to the Vulkan implementation.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderExpectAssumeFeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_EXPECT_ASSUME_EXTENSION_NAME`
- `VK_KHR_SHADER_EXPECT_ASSUME_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_EXPECT_ASSUME_FEATURES_KHR`

New SPIR-V Capabilities

- ExpectAssumeKHR

Version History

- Revision 1, 2023-12-06 (Kevin Petit)
  - Initial revision

VK_KHR_shader_float16_int8

Name String

`VK_KHR_shader_float16_int8`

Extension Type

Device extension

Registered Extension Number

83

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Alexander Galazin [GitHub]alegal-arm

Other Extension Metadata

Last Modified Date

2018-03-07

Interactions and External Dependencies

- This extension interacts with VK_KHR_8bit_storage
- This extension interacts with VK_KHR_16bit_storage
- This extension interacts with VK_KHR_shader_float_controls
- This extension provides API support for GL_EXT_shader_explicit_arithmetic_types

IP Status

No known IP claims.

Contributors

- Alexander Galazin, Arm
- Jan-Harald Fredriksen, Arm
- Jeff Bolz, NVIDIA
- Graeme Leese, Broadcom
- Daniel Rakos, AMD

Description

The VK_KHR_shader_float16_int8 extension allows use of 16-bit floating-point types and 8-bit integer types in shaders for arithmetic operations.

It introduces two new optional features shaderFloat16 and shaderInt8 which directly map to the Float16 and the Int8 SPIR-V capabilities. The VK_KHR_shader_float16_int8 extension also specifies precision requirements for half-precision floating-point SPIR-V operations. This extension does not enable use of 8-bit integer types or 16-bit floating-point types in any shader input and output interfaces and therefore does not supersede the VK_KHR_8bit_storage or VK_KHR_16bit_storage extensions.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. However, if Vulkan 1.2 is supported and this extension is not, both the shaderFloat16 and shaderInt8 capabilities are optional. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceFloat16Int8FeaturesKHR
  - VkPhysicalDeviceShaderFloat16Int8FeaturesKHR

New Enum Constants

- VK_KHR_SHADER_FLOAT16_INT8_EXTENSION_NAME
- VK_KHR_SHADER_FLOAT16_INT8_SPEC_VERSION

- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FLOAT16_INT8_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT16_INT8_FEATURES_KHR`

Version History

- Revision 1, 2018-03-07 (Alexander Galazin)
  - Initial draft

VK_KHR_shader_float_controls

Name String

`VK_KHR_shader_float_controls`

Extension Type

Device extension

Registered Extension Number

198

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_float_controls

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Alexander Galazin [GitHub]alegal-arm

Other Extension Metadata

Last Modified Date

2018-09-11

IP Status

No known IP claims.

Contributors

- Alexander Galazin, Arm
- Jan-Harald Fredriksen, Arm
- Jeff Bolz, NVIDIA
- Graeme Leese, Broadcom
- Daniel Rakos, AMD

Description

The `VK_KHR_shader_float_controls` extension enables efficient use of floating-point computations through the ability to query and override the implementation's default behavior for rounding modes, denormals, signed zero, and infinity.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceFloatControlsPropertiesKHR

New Enums

- VkShaderFloatControlsIndependenceKHR

New Enum Constants

- `VK_KHR_SHADER_FLOAT_CONTROLS_EXTENSION_NAME`
- `VK_KHR_SHADER_FLOAT_CONTROLS_SPEC_VERSION`
- Extending VkShaderFloatControlsIndependence:
  - `VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_32_BIT_ONLY_KHR`
  - `VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_ALL_KHR`
  - `VK_SHADER_FLOAT_CONTROLS_INDEPENDENCE_NONE_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FLOAT_CONTROLS_PROPERTIES_KHR`

New SPIR-V Capabilities

- `DenormPreserve`
- `DenormFlushToZero`
- `SignedZeroInfNanPreserve`
- `RoundingModeRTE`

- `RoundingModeRTZ`

Issues

1) Which instructions must flush denorms?

RESOLVED: Only floating-point conversion, floating-point arithmetic, floating-point relational (except `OpIsNaN`, `OpIsInf`), and floating-point GLSL.std.450 extended instructions must flush denormals.

2) What is the denorm behavior for intermediate results?

RESOLVED: When a SPIR-V instruction is implemented as a sequence of other instructions:

- in the `DenormFlushToZero` execution mode, the intermediate instructions may flush denormals, the final result of the sequence **must** not be denormal.
- in the `DenormPreserve` execution mode, denormals must be preserved throughout the whole sequence.

3) Do denorm and rounding mode controls apply to `OpSpecConstantOp`?

RESOLVED: Yes, except when the opcode is `OpQuantizeToF16`.

4) The SPIR-V specification says that `OpConvertFToU` and `OpConvertFToS` unconditionally round towards zero. Do the rounding mode controls specified through the execution modes apply to them?

RESOLVED: No, these instructions unconditionally round towards zero.

5) Do any of the "Pack" GLSL.std.450 instructions count as conversion instructions and have the rounding mode applied?

RESOLVED: No, only instructions listed in "section 3.32.11. Conversion Instructions" of the SPIR-V specification count as conversion instructions.

6) When using inf/nan-ignore mode, what is expected of `OpIsNan` and `OpIsInf`?

RESOLVED: These instructions must always accurately detect inf/nan if it is passed to them.

Version 4 API Incompatibility

The original versions of `VK_KHR_shader_float_controls` shipped with booleans named "separateDenormSettings" and "separateRoundingModeSettings", which at first glance could have indicated "they can all be set independently, or not". However the spec language as written indicated that the 32-bit value could always be set independently, and only the 16- and 64-bit controls needed to be the same if these values were `VK_FALSE`.

As a result of this slight disparity, and lack of test coverage for this facet of the extension, we ended up with two different behaviors in the wild, where some implementations worked as written, and others worked based on the naming. As these are hard limits in hardware with reasons for exposure as written, it was not possible to standardize on a single way to make this work within the

existing API.

No known users of this part of the extension exist in the wild, and as such the Vulkan WG took the unusual step of retroactively changing the once boolean value into a tri-state enum, breaking source compatibility. This was however done in such a way as to retain ABI compatibility, in case any code using this did exist; with the numerical values 0 and 1 retaining their original specified meaning, and a new value signifying the additional “all need to be set together” state. If any applications exist today, compiled binaries will continue to work as written in most cases, but will need changes before the code can be recompiled.

Version History

- Revision 4, 2019-06-18 (Tobias Hector)
  - Modified settings restrictions, see Version 4 API incompatibility
- Revision 3, 2018-09-11 (Alexander Galazin)
  - Minor restructuring
- Revision 2, 2018-04-17 (Alexander Galazin)
  - Added issues and resolutions
- Revision 1, 2018-04-11 (Alexander Galazin)
  - Initial draft

VK_KHR_shader_float_controls2

Name String

`VK_KHR_shader_float_controls2`

Extension Type

Device extension

Registered Extension Number

529

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1
and
VK_KHR_shader_float_controls

SPIR-V Dependencies

- SPV_KHR_float_controls2

Contact

- Graeme Leese [GitHub]gnl21

Extension Proposal

VK_KHR_shader_float_controls2

Other Extension Metadata

Last Modified Date

2023-05-16

Interactions and External Dependencies

- This extension requires `SPV_KHR_float_controls2`.

Contributors

- Graeme Leese, Broadcom

Description

This extension enables use of the more expressive fast floating-point math flags in the SPV_KHR_float_controls2 extension. These flags give finer- grained control over which optimizations compilers may apply, potentially speeding up execution while retaining correct results.

The extension also adds control over the fast-math modes to the GLSL extended instruction set, making these operations more consistent with SPIR-V and allowing their use in situations where floating-point conformance is important.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderFloatControls2FeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_FLOAT_CONTROLS_2_EXTENSION_NAME`
- `VK_KHR_SHADER_FLOAT_CONTROLS_2_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT_CONTROLS_2_FEATURES_KHR`

New SPIR-V Capabilities

- FloatControls2

Version History

- Revision 1, 2023-05-16 (Graeme Leese)
  - Initial draft

VK_KHR_shader_integer_dot_product

Name String

`VK_KHR_shader_integer_dot_product`

Extension Type

Device extension

Registered Extension Number

281

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_integer_dot_product

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Kevin Petit [GitHub]kpet

Extension Proposal

VK_KHR_shader_integer_dot_product

Other Extension Metadata

Last Modified Date

2021-06-16

Interactions and External Dependencies

- This extension interacts with `VK_KHR_shader_float16_int8`.

IP Status

No known IP claims.

Contributors

- Kévin Petit, Arm Ltd.
- Jeff Bolz, NVidia
- Spencer Fricke, Samsung

- Jesse Hall, Google
- John Kessenich, Google
- Graeme Leese, Broadcom
- Einar Hov, Arm Ltd.
- Stuart Brady, Arm Ltd.
- Pablo Cascon, Arm Ltd.
- Tobias Hector, AMD
- Jeff Leger, Qualcomm
- Ruihao Zhang, Qualcomm
- Pierre Boudier, NVidia
- Jon Leech, The Khronos Group
- Tom Olson, Arm Ltd.

Description

This extension adds support for the integer dot product SPIR-V instructions defined in SPV_KHR_integer_dot_product. These instructions are particularly useful for neural network inference and training but find uses in other general-purpose compute applications as well.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderIntegerDotProductFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceShaderIntegerDotProductPropertiesKHR

New Enum Constants

- `VK_KHR_SHADER_INTEGER_DOT_PRODUCT_EXTENSION_NAME`
- `VK_KHR_SHADER_INTEGER_DOT_PRODUCT_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_INTEGER_DOT_PRODUCT_PROPERTIES_KHR`

Promotion to Vulkan 1.3

Vulkan APIs in this extension are included in core Vulkan 1.3, with the KHR suffix omitted. External interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

New SPIR-V Capabilities

- `DotProductInputAllKHR`
- `DotProductInput4x8BitKHR`
- `DotProductInput4x8BitPackedKHR`
- `DotProductKHR`

Version History

- Revision 1, 2021-06-16 (Kévin Petit)
  - Initial revision

VK_KHR_shader_maximal_reconvergence

Name String

`VK_KHR_shader_maximal_reconvergence`

Extension Type

Device extension

Registered Extension Number

435

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_maximal_reconvergence

Contact

- Alan Baker [GitHub]alan-baker

Extension Proposal

VK_KHR_shader_maximal_reconvergence

Other Extension Metadata

Last Modified Date

2021-11-12

IP Status

No known IP claims.

Interactions and External Dependencies

- Requires SPIR-V 1.3.
- This extension requires `SPV_KHR_maximal_reconvergence`

Contributors

- Alan Baker, Google

Description

This extension allows the use of the `SPV_KHR_maximal_reconvergence` SPIR-V extension in shader modules. `SPV_KHR_maximal_reconvergence` provides stronger guarantees that diverged subgroups will reconverge. These guarantees should match shader author intuition about divergence and reconvergence of invocations based on the structure of the code in the HLL.

Developers should utilize this extension if they require stronger guarantees about reconvergence than either the core spec or SPV_KHR_subgroup_uniform_control_flow. This extension will define the rules that govern how invocations diverge and reconverge in a way that should match developer intuition. It allows robust programs to be written relying on subgroup operations and other tangled instructions.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderMaximalReconvergenceFeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_MAXIMAL_RECONVERGENCE_EXTENSION_NAME`
- `VK_KHR_SHADER_MAXIMAL_RECONVERGENCE_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MAXIMAL_RECONVERGENCE_FEATURES_KHR`

Version History

- Revision 1, 2021-11-12 (Alan Baker)
  - Internal draft version

VK_KHR_shader_non_semantic_info

Name String

`VK_KHR_shader_non_semantic_info`

Extension Type

Device extension

Registered Extension Number

294

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

SPIR-V Dependencies

- SPV_KHR_non_semantic_info

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Baldur Karlsson [GitHub]baldurk

Other Extension Metadata

Last Modified Date

2019-10-16

IP Status

No known IP claims.

Contributors

- Baldur Karlsson, Valve

Description

This extension allows the use of the `SPV_KHR_non_semantic_info` extension in SPIR-V shader modules.

New Enum Constants

- `VK_KHR_SHADER_NON_SEMANTIC_INFO_EXTENSION_NAME`
- `VK_KHR_SHADER_NON_SEMANTIC_INFO_SPEC_VERSION`

Promotion to Vulkan 1.3

Functionality in this extension is included in core Vulkan 1.3. Because the extension has no API controlling its functionality, this results only in a change to the SPIR-V Extensions table.

Version History

- Revision 1, 2019-10-16 (Baldur Karlsson)
  - Initial revision

VK_KHR_shader_quad_control

Name String

`VK_KHR_shader_quad_control`

Extension Type

Device extension

Registered Extension Number

236

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1
and
VK_KHR_vulkan_memory_model
and
VK_KHR_shader_maximal_reconvergence

SPIR-V Dependencies

- SPV_KHR_quad_control

Contact

- Tobias Hector [GitHub]tobski

Extension Proposal

VK_KHR_shader_quad_control

Other Extension Metadata

Last Modified Date

2023-11-01

IP Status

No known IP claims.

Contributors

- Tobias Hector, AMD
- Bill Licea-Kane, Qualcomm
- Graeme Leese, Broadcom
- Jan-Harald Fredriksen, Arm
- Nicolai Hähnle, AMD

- Jeff Bolz, NVidia
- Alan Baker, Google
- Hans-Kristian Arntzen, Valve

Description

This extension adds new quad any/all operations, requires that derivatives are well-defined in quad-uniform control flow, and adds the ability to require helper invocations participate in group operations.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderQuadControlFeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_QUAD_CONTROL_EXTENSION_NAME`
- `VK_KHR_SHADER_QUAD_CONTROL_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_QUAD_CONTROL_FEATURES_KHR`

New SPIR-V Capabilities

- QuadControlKHR

Version History

- Revision 1, 2023-11-01 (Tobias Hector)
  - Initial draft

VK_KHR_shader_relaxed_extended_instruction

Name String

`VK_KHR_shader_relaxed_extended_instruction`

Extension Type

Device extension

Registered Extension Number

559

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

SPIR-V Dependencies

- SPV_KHR_relaxed_extended_instruction

Contact

- Nathan Gauër [GitHub]Keenuts

Extension Proposal

VK_KHR_shader_relaxed_extended_instruction

Other Extension Metadata

Last Modified Date

2024-01-24

IP Status

No known IP claims.

Contributors

- Alan Baker, Google LLC
- Nathan Gauër, Google LLC

Description

This extension allows the use of the `SPV_KHR_relaxed_extended_instruction` extension in SPIR-V shader modules.

It adds a new SPIR-V instruction, which allows some usage of forward references in non-semantic instruction sets. This extensions interacts with the `SPV_KHR_non_semantic_info` extension, hence with `VK_KHR_shader_non_semantic_info`.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderRelaxedExtendedInstructionFeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_RELAXED_EXTENDED_INSTRUCTION_EXTENSION_NAME`
- `VK_KHR_SHADER_RELAXED_EXTENDED_INSTRUCTION_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_RELAXED_EXTENDED_INSTRUCTION_FEATURES_KHR`

Version History

- Revision 1, 2024-01-24 (Nathan Gauër)

- Initial revision

VK_KHR_shader_subgroup_extended_types

Name String

`VK_KHR_shader_subgroup_extended_types`

Extension Type

Device extension

Registered Extension Number

176

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Neil Henning [GitHub]sheredom

Other Extension Metadata

Last Modified Date

2019-01-08

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GLSL_EXT_shader_subgroup_extended_types`

Contributors

- Jeff Bolz, NVIDIA
- Jan-Harald Fredriksen, Arm
- Neil Henning, AMD
- Daniel Koch, NVIDIA
- Jeff Leger, Qualcomm
- Graeme Leese, Broadcom
- David Neto, Google

- Daniel Rakos, AMD

Description

This extension enables the Non Uniform Group Operations in SPIR-V to support 8-bit integer, 16-bit integer, 64-bit integer, 16-bit floating-point, and vectors of these types.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderSubgroupExtendedTypesFeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_SUBGROUP_EXTENDED_TYPES_EXTENSION_NAME`
- `VK_KHR_SHADER_SUBGROUP_EXTENDED_TYPES_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_EXTENDED_TYPES_FEATURES_KHR`

Version History

- Revision 1, 2019-01-08 (Neil Henning)
  - Initial draft

VK_KHR_shader_subgroup_rotate

Name String

`VK_KHR_shader_subgroup_rotate`

Extension Type

Device extension

Registered Extension Number

417

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

None

SPIR-V Dependencies

- SPV_KHR_subgroup_rotate

Contact

- Kevin Petit [GitHub]kpet

Extension Proposal

VK_KHR_shader_subgroup_rotate

Last Modified Date

2024-01-29

IP Status

No known IP claims.

Contributors

- Kévin Petit, Arm Ltd.
- Tobias Hector, AMD
- John Leech, Khronos
- Matthew Netsch, Qualcomm
- Jan-Harald Fredriksen, Arm Ltd.
- Graeme Leese, Broadcom
- Tom Olson, Arm Ltd.
- Spencer Fricke, LunarG Inc.

This extension adds support for the subgroup rotate instruction defined in SPV_KHR_subgroup_rotate.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderSubgroupRotateFeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_SUBGROUP_ROTATE_EXTENSION_NAME`
- `VK_KHR_SHADER_SUBGROUP_ROTATE_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_ROTATE_FEATURES_KHR`
- Extending `VkSubgroupFeatureFlagBits`:
  - `VK_SUBGROUP_FEATURE_ROTATE_BIT_KHR`
  - `VK_SUBGROUP_FEATURE_ROTATE_CLUSTERED_BIT_KHR`

New SPIR-V Capabilities

- GroupNonUniformRotateKHR

Version History

- Revision 2, 2024-01-29 (Kévin Petit)
  - Add `VK_SUBGROUP_FEATURE_ROTATE_BIT_KHR` and `VK_SUBGROUP_FEATURE_ROTATE_CLUSTERED_BIT_KHR`
- Revision 1, 2023-06-20 (Kévin Petit)
  - Initial revision

VK_KHR_shader_subgroup_uniform_control_flow

Name String

`VK_KHR_shader_subgroup_uniform_control_flow`

Extension Type

Device extension

Registered Extension Number

324

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_subgroup_uniform_control_flow

Contact

- Alan Baker [GitHub]alan-baker

Other Extension Metadata

Last Modified Date

2020-08-27

IP Status

No known IP claims.

Interactions and External Dependencies

- Requires SPIR-V 1.3.
- This extension provides API support for `GL_EXT_subgroupuniform_qualifier`

Contributors

- Alan Baker, Google
- Jeff Bolz, NVIDIA

Description

This extension allows the use of the `SPV_KHR_subgroup_uniform_control_flow` SPIR-V extension in shader modules. `SPV_KHR_subgroup_uniform_control_flow` provides stronger guarantees that diverged subgroups will reconverge.

Developers should utilize this extension if they use subgroup operations to reduce the work performed by a uniform subgroup. This extension will guarantee that uniform subgroup will reconverge in the same manner as invocation groups (see “Uniform Control Flow” in the Khronos SPIR-V Specification).

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderSubgroupUniformControlFlowFeaturesKHR

New Enum Constants

- `VK_KHR_SHADER_SUBGROUP_UNIFORM_CONTROL_FLOW_EXTENSION_NAME`
- `VK_KHR_SHADER_SUBGROUP_UNIFORM_CONTROL_FLOW_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_SUBGROUP_UNIFORM_CONTROL_FLOW_FEATURES_KHR`

Version History

- Revision 1, 2020-08-27 (Alan Baker)
  - Internal draft version

VK_KHR_shader_terminate_invocation

Name String

`VK_KHR_shader_terminate_invocation`

Extension Type

Device extension

Registered Extension Number

216

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_terminate_invocation

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Jesse Hall [GitHub]critsec

Other Extension Metadata

Last Modified Date

2020-08-11

IP Status

No known IP claims.

Contributors

- Alan Baker, Google
- Jeff Bolz, NVIDIA
- Jesse Hall, Google
- Ralph Potter, Samsung
- Tom Olson, Arm

Description

This extension adds Vulkan support for the `SPV_KHR_terminate_invocation` SPIR-V extension. That SPIR-V extension provides a new instruction, `OpTerminateInvocation`, which causes a shader invocation to immediately terminate and sets the coverage of shaded samples to `0`; only previously executed instructions will have observable effects. The `OpTerminateInvocation` instruction, along with the `OpDemoteToHelperInvocation` instruction from the `VK_EXT_shader_demote_to_helper_invocation` extension, together replace the `OpKill` instruction, which could behave like either of these instructions. `OpTerminateInvocation` provides the behavior required by the GLSL `discard` statement, and should be used when available by GLSL compilers and applications that need the GLSL `discard` behavior.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:

- VkPhysicalDeviceShaderTerminateInvocationFeaturesKHR

New Enum Constants

- VK_KHR_SHADER_TERMINATE_INVOCATION_EXTENSION_NAME
- VK_KHR_SHADER_TERMINATE_INVOCATION_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TERMINATE_INVOCATION_FEATURES_KHR

Promotion to Vulkan 1.3

Vulkan APIs in this extension are included in core Vulkan 1.3, with the KHR suffix omitted. External interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

Version History

- Revision 1, 2020-08-11 (Jesse Hall)

VK_KHR_shared_presentable_image

Name String

VK_KHR_shared_presentable_image

Extension Type

Device extension

Registered Extension Number

112

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_swapchain
and
VK_KHR_get_surface_capabilities2
and
VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Alon Or-bach [GitHub]alonorbach

Other Extension Metadata

Last Modified Date

2017-03-20

IP Status

No known IP claims.

Contributors

- Alon Or-bach, Samsung Electronics
- Ian Elliott, Google
- Jesse Hall, Google
- Pablo Ceballos, Google
- Chris Forbes, Google
- Jeff Juliano, NVIDIA
- James Jones, NVIDIA
- Daniel Rakos, AMD
- Tobias Hector, Imagination Technologies
- Graham Connor, Imagination Technologies
- Michael Worcester, Imagination Technologies
- Cass Everitt, Oculus
- Johannes Van Waveren, Oculus

Description

This extension extends `VK_KHR_swapchain` to enable creation of a shared presentable image. This allows the application to use the image while the presention engine is accessing it, in order to reduce the latency between rendering and presentation.

New Commands

- vkGetSwapchainStatusKHR

New Structures

- Extending VkSurfaceCapabilities2KHR:
  - VkSharedPresentSurfaceCapabilitiesKHR

New Enum Constants

- `VK_KHR_SHARED_PRESENTABLE_IMAGE_EXTENSION_NAME`
- `VK_KHR_SHARED_PRESENTABLE_IMAGE_SPEC_VERSION`
- Extending VkImageLayout:

    - VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR
- Extending VkPresentModeKHR:
    - VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR
    - VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_SHARED_PRESENT_SURFACE_CAPABILITIES_KHR

Issues

1) Should we allow a Vulkan WSI swapchain to toggle between normal usage and shared presentation usage?

RESOLVED: No. WSI swapchains are typically recreated with new properties instead of having their properties changed. This can also save resources, assuming that fewer images are needed for shared presentation, and assuming that most VR applications do not need to switch between normal and shared usage.

2) Should we have a query for determining how the presentation engine refresh is triggered?

RESOLVED: Yes. This is done via which presentation modes a surface supports.

3) Should the object representing a shared presentable image be an extension of a VkSwapchainKHR or a separate object?

RESOLVED: Extension of a swapchain due to overlap in creation properties and to allow common functionality between shared and normal presentable images and swapchains.

4) What should we call the extension and the new structures it creates?

RESOLVED: Shared presentable image / shared present.

5) Should the `minImageCount` and `presentMode` values of the VkSwapchainCreateInfoKHR be ignored, or required to be compatible values?

RESOLVED: `minImageCount` must be 1, and `presentMode` should be set to either `VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR` or `VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR`.

6) What should the layout of the shared presentable image be?

RESOLVED: After acquiring the shared presentable image, the application must transition it to the `VK_IMAGE_LAYOUT_SHARED_PRESENT_KHR` layout prior to it being used. After this initial transition, any image usage that was requested during swapchain creation **can** be performed on the image without layout transitions being performed.

7) Do we need a new API for the trigger to refresh new content?

RESOLVED: vkQueuePresentKHR to act as API to trigger a refresh, as will allow combination with other compatible extensions to vkQueuePresentKHR.

8) How should an application detect a VK_ERROR_OUT_OF_DATE_KHR error on a swapchain using the VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR present mode?

RESOLVED: Introduce vkGetSwapchainStatusKHR to allow applications to query the status of a swapchain using a shared presentation mode.

9) What should subsequent calls to vkQueuePresentKHR for VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR swapchains be defined to do?

RESOLVED: State that implementations may use it as a hint for updated content.

10) Can the ownership of a shared presentable image be transferred to a different queue?

RESOLVED: No. It is not possible to transfer ownership of a shared presentable image obtained from a swapchain created using VK_SHARING_MODE_EXCLUSIVE after it has been presented.

11) How should vkQueueSubmit behave if a command buffer uses an image from a VK_ERROR_OUT_OF_DATE_KHR swapchain?

RESOLVED: vkQueueSubmit is expected to return the VK_ERROR_DEVICE_LOST error.

12) Can Vulkan provide any guarantee on the order of rendering, to enable beam chasing?

RESOLVED: This could be achieved via use of render passes to ensure strip rendering.

Version History

- Revision 1, 2017-03-20 (Alon Or-bach)
  - Internal revisions

VK_KHR_spirv_1_4

Name String

VK_KHR_spirv_1_4

Extension Type

Device extension

Registered Extension Number

237

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1
and

VK_KHR_shader_float_controls

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Jesse Hall [GitHub]critsec

Other Extension Metadata

Last Modified Date

2019-04-01

IP Status

No known IP claims.

Interactions and External Dependencies

- Requires SPIR-V 1.4.

Contributors

- Alexander Galazin, Arm
- David Neto, Google
- Jesse Hall, Google
- John Kessenich, Google
- Neil Henning, AMD
- Tom Olson, Arm

Description

This extension allows the use of SPIR-V 1.4 shader modules. SPIR-V 1.4's new features primarily make it an easier target for compilers from high-level languages, rather than exposing new hardware functionality.

SPIR-V 1.4 incorporates features that are also available separately as extensions. SPIR-V 1.4 shader modules do not need to enable those extensions with the `OpExtension` opcode, since they are integral parts of SPIR-V 1.4.

SPIR-V 1.4 introduces new floating-point execution mode capabilities, also available via `SPV_KHR_float_controls`. Implementations are not required to support all of these new capabilities; support can be queried using VkPhysicalDeviceFloatControlsPropertiesKHR from the `VK_KHR_shader_float_controls` extension.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Enum Constants

- VK_KHR_SPIRV_1_4_EXTENSION_NAME
- VK_KHR_SPIRV_1_4_SPEC_VERSION

Issues

1. Should we have an extension specific to this SPIR-V version, or add a version-generic query for SPIR-V version? SPIR-V 1.4 does not need any other API changes.

RESOLVED: Just expose SPIR-V 1.4.

Most new SPIR-V versions introduce optionally-required capabilities or have implementation-defined limits, and would need more API and specification changes specific to that version to make them available in Vulkan. While we could expose the parts of a new SPIR-V version that do not need accompanying changes generically, we will still end up writing extensions specific to each version for the remaining parts. Thus the generic mechanism will not reduce future spec-writing effort. In addition, making it clear which parts of a future version are supported by the generic mechanism and which cannot be used without specific support would be difficult to get right ahead of time.

2. Can different stages of the same pipeline use shaders with different SPIR-V versions?

RESOLVED: Yes.

Mixing SPIR-V versions 1.0-1.3 in the same pipeline has not been disallowed, so it would be inconsistent to disallow mixing 1.4 with previous versions. SPIR-V 1.4 does not introduce anything that should cause new difficulties here.

3. Must Vulkan extensions corresponding to SPIR-V extensions that were promoted to core in 1.4 be enabled in order to use that functionality in a SPIR-V 1.4 module?

RESOLVED: No, with caveats.

The SPIR-V 1.4 module does not need to declare the SPIR-V extensions, since the functionality is now part of core, so there is no need to enable the Vulkan extension that allows SPIR-V modules to declare the SPIR-V extension. However, when the functionality that is now core in SPIR-V 1.4 is optionally supported, the query for support is provided by a Vulkan extension, and that query can only be used if the extension is enabled.

This applies to any SPIR-V version; specifically for SPIR-V 1.4 this only applies to the functionality from SPV_KHR_float_controls, which was made available in Vulkan by VK_KHR_shader_float_controls. Even though the extension was promoted in SPIR-V 1.4, the capabilities are still optional in implementations that support VK_KHR_spirv_1_4.

A SPIR-V 1.4 module does not need to enable SPV_KHR_float_controls in order to use the capabilities, so if the application has *a priori* knowledge that the implementation supports the capabilities, it does not need to enable VK_KHR_shader_float_controls. However, if it does not have this knowledge and has to query for support at runtime, it must enable VK_KHR_shader_float_controls in order to use VkPhysicalDeviceFloatControlsPropertiesKHR.

Version History

- Revision 1, 2019-04-01 (Jesse Hall)
  - Internal draft versions

VK_KHR_storage_buffer_storage_class

Name String

`VK_KHR_storage_buffer_storage_class`

Extension Type

Device extension

Registered Extension Number

132

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

SPIR-V Dependencies

- SPV_KHR_storage_buffer_storage_class

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Alexander Galazin [GitHub]alegal-arm

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Contributors

- Alexander Galazin, ARM
- David Neto, Google

Description

This extension adds support for the following SPIR-V extension in Vulkan:

- SPV_KHR_storage_buffer_storage_class

This extension provides a new SPIR-V StorageBuffer storage class. A Block-decorated object in this class is equivalent to a BufferBlock-decorated object in the Uniform storage class.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1.

New Enum Constants

- VK_KHR_STORAGE_BUFFER_STORAGE_CLASS_EXTENSION_NAME
- VK_KHR_STORAGE_BUFFER_STORAGE_CLASS_SPEC_VERSION

Version History

- Revision 1, 2017-03-23 (Alexander Galazin)
  - Initial draft

VK_KHR_surface

Name String

VK_KHR_surface

Extension Type

Instance extension

Registered Extension Number

1

Revision

25

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- James Jones [GitHub]cubanismo
- Ian Elliott [GitHub]ianelliottus

Other Extension Metadata

Last Modified Date

2016-08-25

IP Status

No known IP claims.

Contributors

- Patrick Doane, Blizzard
- Ian Elliott, LunarG
- Jesse Hall, Google
- James Jones, NVIDIA
- David Mao, AMD
- Norbert Nopper, Freescale
- Alon Or-bach, Samsung
- Daniel Rakos, AMD
- Graham Sellers, AMD
- Jeff Vigil, Qualcomm
- Chia-I Wu, LunarG
- Faith Ekstrand, Intel

Description

The `VK_KHR_surface` extension is an instance extension. It introduces VkSurfaceKHR objects, which abstract native platform surface or window objects for use with Vulkan. It also provides a way to determine whether a queue family in a physical device supports presenting to particular surface.

Separate extensions for each platform provide the mechanisms for creating VkSurfaceKHR objects, but once created they may be used in this and other platform-independent extensions, in particular the `VK_KHR_swapchain` extension.

New Object Types

- VkSurfaceKHR

New Commands

- vkDestroySurfaceKHR
- vkGetPhysicalDeviceSurfaceCapabilitiesKHR
- vkGetPhysicalDeviceSurfaceFormatsKHR
- vkGetPhysicalDeviceSurfacePresentModesKHR
- vkGetPhysicalDeviceSurfaceSupportKHR

New Structures

- VkSurfaceCapabilitiesKHR
- VkSurfaceFormatKHR

New Enums

- VkColorSpaceKHR
- VkCompositeAlphaFlagBitsKHR
- VkPresentModeKHR
- VkSurfaceTransformFlagBitsKHR

New Bitmasks

- VkCompositeAlphaFlagsKHR

New Enum Constants

- VK_KHR_SURFACE_EXTENSION_NAME
- VK_KHR_SURFACE_SPEC_VERSION
- Extending VkObjectType:
  - VK_OBJECT_TYPE_SURFACE_KHR
- Extending VkResult:
  - VK_ERROR_NATIVE_WINDOW_IN_USE_KHR
  - VK_ERROR_SURFACE_LOST_KHR

Examples

NOTE	The example code for the VK_KHR_surface and VK_KHR_swapchain extensions was removed from the appendix after revision 1.0.29. This WSI example code was ported to the cube demo that is shipped with the official Khronos SDK, and is being kept up-to-date in that location (see: https://github.com/KhronosGroup/Vulkan-Tools/blob/main/cube/cube.c).

Issues

1) Should this extension include a method to query whether a physical device supports presenting to a specific window or native surface on a given platform?

RESOLVED: Yes. Without this, applications would need to create a device instance to determine whether a particular window can be presented to. Knowing that a device supports presentation to a platform in general is not sufficient, as a single machine might support multiple seats, or instances of the platform that each use different underlying physical devices. Additionally, on some platforms, such as the X Window System, different drivers and devices might be used for different windows depending on which section of the desktop they exist on.

2) Should the vkGetPhysicalDeviceSurfaceCapabilitiesKHR, vkGetPhysicalDeviceSurfaceFormatsKHR, and vkGetPhysicalDeviceSurfacePresentModesKHR functions be in this extension and operate on physical devices, rather than being in VK_KHR_swapchain (i.e. device extension) and being dependent on VkDevice?

RESOLVED: Yes. While it might be useful to depend on `VkDevice` (and therefore on enabled extensions and features) for the queries, Vulkan was released only with the VkPhysicalDevice versions. Many cases can be resolved by a Valid Usage statement, and/or by a separate `pNext` chain version of the query struct specific to a given extension or parameters, via extensible versions of the queries: vkGetPhysicalDeviceSurfacePresentModes2EXT, vkGetPhysicalDeviceSurfaceCapabilities2KHR, and vkGetPhysicalDeviceSurfaceFormats2KHR.

3) Should Vulkan support Xlib or XCB as the API for accessing the X Window System platform?

RESOLVED: Both. XCB is a more modern and efficient API, but Xlib usage is deeply ingrained in many applications and likely will remain in use for the foreseeable future. Not all drivers necessarily need to support both, but including both as options in the core specification will probably encourage support, which should in turn ease adoption of the Vulkan API in older codebases. Additionally, the performance improvements possible with XCB likely will not have a measurable impact on the performance of Vulkan presentation and other minimal window system interactions defined here.

4) Should the GBM platform be included in the list of platform enums?

RESOLVED: Deferred, and will be addressed with a platform-specific extension to be written in the future.

Version History

- Revision 1, 2015-05-20 (James Jones)
  - Initial draft, based on LunarG KHR spec, other KHR specs, patches attached to bugs.
- Revision 2, 2015-05-22 (Ian Elliott)
  - Created initial Description section.
  - Removed query for whether a platform requires the use of a queue for presentation, since it was decided that presentation will always be modeled as being part of the queue.
  - Fixed typos and other minor mistakes.
- Revision 3, 2015-05-26 (Ian Elliott)
  - Improved the Description section.
- Revision 4, 2015-05-27 (James Jones)
  - Fixed compilation errors in example code.
- Revision 5, 2015-06-01 (James Jones)
  - Added issues 1 and 2 and made related spec updates.
- Revision 6, 2015-06-01 (James Jones)
  - Merged the platform type mappings table previously removed from VK_KHR_swapchain with the platform description table in this spec.
  - Added issues 3 and 4 documenting choices made when building the initial list of native platforms supported.
- Revision 7, 2015-06-11 (Ian Elliott)

    - Updated table 1 per input from the KHR TSG.
    - Updated issue 4 (GBM) per discussion with Daniel Stone. He will create a platform-specific extension sometime in the future.
- Revision 8, 2015-06-17 (James Jones)
    - Updated enum-extending values using new convention.
    - Fixed the value of VK_SURFACE_PLATFORM_INFO_TYPE_SUPPORTED_KHR.
- Revision 9, 2015-06-17 (James Jones)
    - Rebased on Vulkan API version 126.
- Revision 10, 2015-06-18 (James Jones)
    - Marked issues 2 and 3 resolved.
- Revision 11, 2015-06-23 (Ian Elliott)
    - Examples now show use of function pointers for extension functions.
    - Eliminated extraneous whitespace.
- Revision 12, 2015-07-07 (Daniel Rakos)
    - Added error section describing when each error is expected to be reported.
    - Replaced the term “queue node index” with “queue family index” in the spec as that is the agreed term to be used in the latest version of the core header and spec.
    - Replaced bool32_t with VkBool32.
- Revision 13, 2015-08-06 (Daniel Rakos)
    - Updated spec against latest core API header version.
- Revision 14, 2015-08-20 (Ian Elliott)
    - Renamed this extension and all of its enumerations, types, functions, etc. This makes it compliant with the proposed standard for Vulkan extensions.
    - Switched from “revision” to “version”, including use of the VK_MAKE_VERSION macro in the header file.
    - Did miscellaneous cleanup, etc.
- Revision 15, 2015-08-20 (Ian Elliott—porting a 2015-07-29 change from James Jones)
    - Moved the surface transform enums here from VK_WSI_swapchain so they could be reused by VK_WSI_display.
- Revision 16, 2015-09-01 (James Jones)
    - Restore single-field revision number.
- Revision 17, 2015-09-01 (James Jones)
    - Fix example code compilation errors.
- Revision 18, 2015-09-26 (Jesse Hall)
    - Replaced VkSurfaceDescriptionKHR with the VkSurfaceKHR object, which is created via layered extensions. Added VkDestroySurfaceKHR.

- Revision 19, 2015-09-28 (Jesse Hall)
  - Renamed from VK_EXT_KHR_swapchain to VK_EXT_KHR_surface.
- Revision 20, 2015-09-30 (Jeff Vigil)
  - Add error result VK_ERROR_SURFACE_LOST_KHR.
- Revision 21, 2015-10-15 (Daniel Rakos)
  - Updated the resolution of issue #2 and include the surface capability queries in this extension.
  - Renamed SurfaceProperties to SurfaceCapabilities as it better reflects that the values returned are the capabilities of the surface on a particular device.
  - Other minor cleanup and consistency changes.
- Revision 22, 2015-10-26 (Ian Elliott)
  - Renamed from VK_EXT_KHR_surface to VK_KHR_surface.
- Revision 23, 2015-11-03 (Daniel Rakos)
  - Added allocation callbacks to vkDestroySurfaceKHR.
- Revision 24, 2015-11-10 (Jesse Hall)
  - Removed VkSurfaceTransformKHR. Use VkSurfaceTransformFlagBitsKHR instead.
  - Rename VkSurfaceCapabilitiesKHR member maxImageArraySize to maxImageArrayLayers.
- Revision 25, 2016-01-14 (James Jones)
  - Moved VK_ERROR_NATIVE_WINDOW_IN_USE_KHR from the VK_KHR_android_surface to the VK_KHR_surface extension.
- 2016-08-23 (Ian Elliott)
  - Update the example code, to not have so many characters per line, and to split out a new example to show how to obtain function pointers.
- 2016-08-25 (Ian Elliott)
  - A note was added at the beginning of the example code, stating that it will be removed from future versions of the appendix.

VK_KHR_surface_protected_capabilities

Name String

`VK_KHR_surface_protected_capabilities`

Extension Type

Instance extension

Registered Extension Number

240

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1
and
VK_KHR_get_surface_capabilities2

Contact

- Sandeep Shinde [GitHub]sashinde

Other Extension Metadata

Last Modified Date

2018-12-18

IP Status

No known IP claims.

Contributors

- Sandeep Shinde, NVIDIA
- James Jones, NVIDIA
- Daniel Koch, NVIDIA

Description

This extension extends VkSurfaceCapabilities2KHR, providing applications a way to query whether swapchains **can** be created with the `VK_SWAPCHAIN_CREATE_PROTECTED_BIT_KHR` flag set.

Vulkan 1.1 added (optional) support for protect memory and protected resources including buffers (`VK_BUFFER_CREATE_PROTECTED_BIT`), images (`VK_IMAGE_CREATE_PROTECTED_BIT`), and swapchains (`VK_SWAPCHAIN_CREATE_PROTECTED_BIT_KHR`). However, on implementations which support multiple windowing systems, not all window systems **may** be able to provide a protected display path.

This extension provides a way to query if a protected swapchain created for a surface (and thus a specific windowing system) **can** be displayed on screen. It extends the existing VkSurfaceCapabilities2KHR structure with a new VkSurfaceProtectedCapabilitiesKHR structure from which the application **can** obtain information about support for protected swapchain creation through vkGetPhysicalDeviceSurfaceCapabilities2KHR.

New Structures

- Extending VkSurfaceCapabilities2KHR:
  - VkSurfaceProtectedCapabilitiesKHR

New Enum Constants

- `VK_KHR_SURFACE_PROTECTED_CAPABILITIES_EXTENSION_NAME`

- VK_KHR_SURFACE_PROTECTED_CAPABILITIES_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_SURFACE_PROTECTED_CAPABILITIES_KHR

Version History

- Revision 1, 2018-12-18 (Sandeep Shinde, Daniel Koch)
  - Internal revisions.

VK_KHR_swapchain

Name String

VK_KHR_swapchain

Extension Type

Device extension

Registered Extension Number

2

Revision

70

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

API Interactions

- Interacts with VK_VERSION_1_1

Contact

- James Jones [GitHub]cubanismo
- Ian Elliott [GitHub]ianelliottus

Other Extension Metadata

Last Modified Date

2017-10-06

IP Status

No known IP claims.

Interactions and External Dependencies

- Interacts with Vulkan 1.1

Contributors

- Patrick Doane, Blizzard
- Ian Elliott, LunarG
- Jesse Hall, Google
- Mathias Heyer, NVIDIA
- James Jones, NVIDIA
- David Mao, AMD
- Norbert Nopper, Freescale
- Alon Or-bach, Samsung
- Daniel Rakos, AMD
- Graham Sellers, AMD
- Jeff Vigil, Qualcomm
- Chia-I Wu, LunarG
- Faith Ekstrand, Intel
- Matthaeus G. Chajdas, AMD
- Ray Smith, ARM

Description

The `VK_KHR_swapchain` extension is the device-level companion to the `VK_KHR_surface` extension. It introduces VkSwapchainKHR objects, which provide the ability to present rendering results to a surface.

New Object Types

- VkSwapchainKHR

New Commands

- vkAcquireNextImageKHR
- vkCreateSwapchainKHR
- vkDestroySwapchainKHR
- vkGetSwapchainImagesKHR
- vkQueuePresentKHR

If Vulkan Version 1.1 is supported:

- vkAcquireNextImage2KHR
- vkGetDeviceGroupPresentCapabilitiesKHR
- vkGetDeviceGroupSurfacePresentModesKHR
- vkGetPhysicalDevicePresentRectanglesKHR

New Structures

- VkPresentInfoKHR
- VkSwapchainCreateInfoKHR

If Vulkan Version 1.1 is supported:

- VkAcquireNextImageInfoKHR
- VkDeviceGroupPresentCapabilitiesKHR
- Extending VkBindImageMemoryInfo:
  - VkBindImageMemorySwapchainInfoKHR
- Extending VkImageCreateInfo:
  - VkImageSwapchainCreateInfoKHR
- Extending VkPresentInfoKHR:
  - VkDeviceGroupPresentInfoKHR
- Extending VkSwapchainCreateInfoKHR:
  - VkDeviceGroupSwapchainCreateInfoKHR

New Enums

- VkSwapchainCreateFlagBitsKHR

If Vulkan Version 1.1 is supported:

- VkDeviceGroupPresentModeFlagBitsKHR

New Bitmasks

- VkSwapchainCreateFlagsKHR

If Vulkan Version 1.1 is supported:

- VkDeviceGroupPresentModeFlagsKHR

New Enum Constants

- `VK_KHR_SWAPCHAIN_EXTENSION_NAME`
- `VK_KHR_SWAPCHAIN_SPEC_VERSION`
- Extending VkImageLayout:
  - `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR`
- Extending VkObjectType:
  - `VK_OBJECT_TYPE_SWAPCHAIN_KHR`
- Extending VkResult:
  - `VK_ERROR_OUT_OF_DATE_KHR`

    - VK_SUBOPTIMAL_KHR
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PRESENT_INFO_KHR
    - VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR

If Vulkan Version 1.1 is supported:

- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_ACQUIRE_NEXT_IMAGE_INFO_KHR
    - VK_STRUCTURE_TYPE_BIND_IMAGE_MEMORY_SWAPCHAIN_INFO_KHR
    - VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_CAPABILITIES_KHR
    - VK_STRUCTURE_TYPE_DEVICE_GROUP_PRESENT_INFO_KHR
    - VK_STRUCTURE_TYPE_DEVICE_GROUP_SWAPCHAIN_CREATE_INFO_KHR
    - VK_STRUCTURE_TYPE_IMAGE_SWAPCHAIN_CREATE_INFO_KHR
- Extending VkSwapchainCreateFlagBitsKHR:
    - VK_SWAPCHAIN_CREATE_PROTECTED_BIT_KHR
    - VK_SWAPCHAIN_CREATE_SPLIT_INSTANCE_BIND_REGIONS_BIT_KHR

Issues

1) Does this extension allow the application to specify the memory backing of the presentable images?

RESOLVED: No. Unlike standard images, the implementation will allocate the memory backing of the presentable image.

2) What operations are allowed on presentable images?

RESOLVED: This is determined by the image usage flags specified when creating the presentable image's swapchain.

3) Does this extension support MSAA presentable images?

RESOLVED: No. Presentable images are always single-sampled. Multi-sampled rendering must use regular images. To present the rendering results the application must manually resolve the multi-sampled image to a single-sampled presentable image prior to presentation.

4) Does this extension support stereo/multi-view presentable images?

RESOLVED: Yes. The number of views associated with a presentable image is determined by the `imageArrayLayers` specified when creating a swapchain. All presentable images in a given swapchain use the same array size.

5) Are the layers of stereo presentable images half-sized?

RESOLVED: No. The image extents always match those requested by the application.

6) Do the “present” and “acquire next image” commands operate on a queue? If not, do they need to include explicit semaphore objects to interlock them with queue operations?

RESOLVED: The present command operates on a queue. The image ownership operation it represents happens in order with other operations on the queue, so no explicit semaphore object is required to synchronize its actions.

Applications may want to acquire the next image in separate threads from those in which they manage their queue, or in multiple threads. To make such usage easier, the acquire next image command takes a semaphore to signal as a method of explicit synchronization. The application must later queue a wait for this semaphore before queuing execution of any commands using the image.

7) Does vkAcquireNextImageKHR block if no images are available?

RESOLVED: The command takes a timeout parameter. Special values for the timeout are 0, which makes the call a non-blocking operation, and `UINT64_MAX`, which blocks indefinitely. Values in between will block for up to the specified time. The call will return when an image becomes available or an error occurs. It may, but is not required to, return before the specified timeout expires if the swapchain becomes out of date.

8) Can multiple presents be queued using one vkQueuePresentKHR call?

RESOLVED: Yes. VkPresentInfoKHR contains a list of swapchains and corresponding image indices that will be presented. When supported, all presentations queued with a single vkQueuePresentKHR call will be applied atomically as one operation. The same swapchain must not appear in the list more than once. Later extensions may provide applications stronger guarantees of atomicity for such present operations, and/or allow them to query whether atomic presentation of a particular group of swapchains is possible.

9) How do the presentation and acquire next image functions notify the application the targeted surface has changed?

RESOLVED: Two new result codes are introduced for this purpose:

- `VK_SUBOPTIMAL_KHR` - Presentation will still succeed, subject to the window resize behavior, but the swapchain is no longer configured optimally for the surface it targets. Applications should query updated surface information and recreate their swapchain at the next convenient opportunity.
- `VK_ERROR_OUT_OF_DATE_KHR` - Failure. The swapchain is no longer compatible with the surface it targets. The application must query updated surface information and recreate the swapchain before presentation will succeed.

These can be returned by both vkAcquireNextImageKHR and vkQueuePresentKHR.

10) Does the vkAcquireNextImageKHR command return a semaphore to the application via an output parameter, or accept a semaphore to signal from the application as an object handle parameter?

RESOLVED: Accept a semaphore to signal as an object handle. This avoids the need to specify

whether the application must destroy the semaphore or whether it is owned by the swapchain, and if the latter, what its lifetime is and whether it can be reused for other operations once it is received from vkAcquireNextImageKHR.

11) What types of swapchain queuing behavior should be exposed? Options include swap interval specification, mailbox/most recent vs. FIFO queue management, targeting specific vertical blank intervals or absolute times for a given present operation, and probably others. For some of these, whether they are specified at swapchain creation time or as per-present parameters needs to be decided as well.

RESOLVED: The base swapchain extension will expose 3 possible behaviors (of which, FIFO will always be supported):

- Immediate present: Does not wait for vertical blanking period to update the current image, likely resulting in visible tearing. No internal queue is used. Present requests are applied immediately.
- Mailbox queue: Waits for the next vertical blanking period to update the current image. No tearing should be observed. An internal single-entry queue is used to hold pending presentation requests. If the queue is full when a new presentation request is received, the new request replaces the existing entry, and any images associated with the prior entry become available for reuse by the application.
- FIFO queue: Waits for the next vertical blanking period to update the current image. No tearing should be observed. An internal queue containing `numSwapchainImages` - 1 entries is used to hold pending presentation requests. New requests are appended to the end of the queue, and one request is removed from the beginning of the queue and processed during each vertical blanking period in which the queue is non-empty

Not all surfaces will support all of these modes, so the modes supported will be returned using a surface information query. All surfaces must support the FIFO queue mode. Applications must choose one of these modes up front when creating a swapchain. Switching modes can be accomplished by recreating the swapchain.

12) Can `VK_PRESENT_MODE_MAILBOX_KHR` provide non-blocking guarantees for vkAcquireNextImageKHR? If so, what is the proper criteria?

RESOLVED: Yes. The difficulty is not immediately obvious here. Naively, if at least 3 images are requested, mailbox mode should always have an image available for the application if the application does not own any images when the call to vkAcquireNextImageKHR was made. However, some presentation engines may have more than one “current” image, and would still need to block in some cases. The right requirement appears to be that if the application allocates the surface’s minimum number of images + 1 then it is guaranteed non-blocking behavior when it does not currently own any images.

13) Is there a way to create and initialize a new swapchain for a surface that has generated a `VK_SUBOPTIMAL_KHR` return code while still using the old swapchain?

RESOLVED: Not as part of this specification. This could be useful to allow the application to create an “optimal” replacement swapchain and rebuild all its command buffers using it in a background thread at a low priority while continuing to use the “suboptimal” swapchain in the main thread. It

could probably use the same “atomic replace” semantics proposed for recreating direct-to-device swapchains without incurring a mode switch. However, after discussion, it was determined some platforms probably could not support concurrent swapchains for the same surface though, so this will be left out of the base KHR extensions. A future extension could add this for platforms where it is supported.

14) Should there be a special value for VkSurfaceCapabilitiesKHR::`maxImageCount` to indicate there are no practical limits on the number of images in a swapchain?

RESOLVED: Yes. There will often be cases where there is no practical limit to the number of images in a swapchain other than the amount of available resources (i.e., memory) in the system. Trying to derive a hard limit from things like memory size is prone to failure. It is better in such cases to leave it to applications to figure such soft limits out via trial/failure iterations.

15) Should there be a special value for VkSurfaceCapabilitiesKHR::`currentExtent` to indicate the size of the platform surface is undefined?

RESOLVED: Yes. On some platforms (Wayland, for example), the surface size is defined by the images presented to it rather than the other way around.

16) Should there be a special value for VkSurfaceCapabilitiesKHR::`maxImageExtent` to indicate there is no practical limit on the surface size?

RESOLVED: No. It seems unlikely such a system would exist. 0 could be used to indicate the platform places no limits on the extents beyond those imposed by Vulkan for normal images, but this query could just as easily return those same limits, so a special “unlimited” value does not seem useful for this field.

17) How should surface rotation and mirroring be exposed to applications? How do they specify rotation and mirroring transforms applied prior to presentation?

RESOLVED: Applications can query both the supported and current transforms of a surface. Both are specified relative to the device’s “natural” display rotation and direction. The supported transforms indicate which orientations the presentation engine accepts images in. For example, a presentation engine that does not support transforming surfaces as part of presentation, and which is presenting to a surface that is displayed with a 90-degree rotation, would return only one supported transform bit: `VK_SURFACE_TRANSFORM_ROTATE_90_BIT_KHR`. Applications must transform their rendering by the transform they specify when creating the swapchain in `preTransform` field.

18) Can surfaces ever not support `VK_MIRROR_NONE`? Can they support vertical and horizontal mirroring simultaneously? Relatedly, should `VK_MIRROR_NONE`[_BIT] be zero, or bit one, and should applications be allowed to specify multiple pre and current mirror transform bits, or exactly one?

RESOLVED: Since some platforms may not support presenting with a transform other than the native window’s current transform, and prerotation/mirroring are specified relative to the device’s natural rotation and direction, rather than relative to the surface’s current rotation and direction, it is necessary to express lack of support for no mirroring. To allow this, the `MIRROR_NONE` enum must occupy a bit in the flags. Since `MIRROR_NONE` must be a bit in the bitmask rather than a bitmask with no values set, allowing more than one bit to be set in the bitmask would make it possible to describe undefined transforms such as `VK_MIRROR_NONE_BIT | VK_MIRROR_HORIZONTAL_BIT`, or a

transform that includes both “no mirroring” and “horizontal mirroring” simultaneously. Therefore, it is desirable to allow specifying all supported mirroring transforms using only one bit. The question then becomes, should there be a `VK_MIRROR_HORIZONTAL_AND_VERTICAL_BIT` to represent a simultaneous horizontal and vertical mirror transform? However, such a transform is equivalent to a 180 degree rotation, so presentation engines and applications that wish to support or use such a transform can express it through rotation instead. Therefore, 3 exclusive bits are sufficient to express all needed mirroring transforms.

19) Should support for sRGB be required?

RESOLVED: In the advent of UHD and HDR display devices, proper color space information is vital to the display pipeline represented by the swapchain. The application can discover the supported format/color-space pairs and select a pair most suited to its rendering needs. Currently only the sRGB color space is supported, future extensions may provide support for more color spaces. See issues 23 and 24.

20) Is there a mechanism to modify or replace an existing swapchain with one targeting the same surface?

RESOLVED: Yes. This is described above in the text.

21) Should there be a way to set prerotation and mirroring using native APIs when presenting using a Vulkan swapchain?

RESOLVED: Yes. The transforms that can be expressed in this extension are a subset of those possible on native platforms. If a platform exposes a method to specify the transform of presented images for a given surface using native methods and exposes more transforms or other properties for surfaces than Vulkan supports, it might be impossible, difficult, or inconvenient to set some of those properties using Vulkan KHR extensions and some using the native interfaces. To avoid overwriting properties set using native commands when presenting using a Vulkan swapchain, the application can set the pretransform to “inherit”, in which case the current native properties will be used, or if none are available, a platform-specific default will be used. Platforms that do not specify a reasonable default or do not provide native mechanisms to specify such transforms should not include the inherit bits in the `supportedTransforms` bitmask they return in VkSurfaceCapabilitiesKHR.

22) Should the content of presentable images be clipped by objects obscuring their target surface?

RESOLVED: Applications can choose which behavior they prefer. Allowing the content to be clipped could enable more efficient presentation methods on some platforms, but some applications might rely on the content of presentable images to perform techniques such as partial updates or motion blurs.

23) What is the purpose of specifying a VkColorSpaceKHR along with VkFormat when creating a swapchain?

RESOLVED: While Vulkan itself is color space agnostic (e.g. even the meaning of R, G, B and A can be freely defined by the rendering application), the swapchain eventually will have to present the images on a display device with specific color reproduction characteristics. If any color space transformations are necessary before an image can be displayed, the color space of the presented

image must be known to the swapchain. A swapchain will only support a restricted set of color format and -space pairs. This set can be discovered via vkGetPhysicalDeviceSurfaceFormatsKHR. As it can be expected that most display devices support the sRGB color space, at least one format/color-space pair has to be exposed, where the color space is VK_COLOR_SPACE_SRGB_NONLINEAR_KHR.

24) How are sRGB formats and the sRGB color space related?

RESOLVED: While Vulkan exposes a number of SRGB texture formats, using such formats does not guarantee working in a specific color space. It merely means that the hardware can directly support applying the non-linear transfer functions defined by the sRGB standard color space when reading from or writing to images of those formats. Still, it is unlikely that a swapchain will expose a *_SRGB format along with any color space other than VK_COLOR_SPACE_SRGB_NONLINEAR_KHR.

On the other hand, non-*_SRGB formats will be very likely exposed in pair with a SRGB color space. This means, the hardware will not apply any transfer function when reading from or writing to such images, yet they will still be presented on a device with sRGB display characteristics. In this case the application is responsible for applying the transfer function, for instance by using shader math.

25) How are the lifetimes of surfaces and swapchains targeting them related?

RESOLVED: A surface must outlive any swapchains targeting it. A VkSurfaceKHR owns the binding of the native window to the Vulkan driver.

26) How can the application control the way the alpha component of swapchain images is treated by the presentation engine during compositing?

RESOLVED: We should add new enum values to allow the application to negotiate with the presentation engine on how to treat image alpha values during the compositing process. Since not all platforms can practically control this through the Vulkan driver, a value of VK_COMPOSITE_ALPHA_INHERIT_BIT_KHR is provided like for surface transforms.

27) Is vkCreateSwapchainKHR the right function to return VK_ERROR_NATIVE_WINDOW_IN_USE_KHR, or should the various platform-specific VkSurfaceKHR factory functions catch this error earlier?

RESOLVED: For most platforms, the VkSurfaceKHR structure is a simple container holding the data that identifies a native window or other object representing a surface on a particular platform. For the surface factory functions to return this error, they would likely need to register a reference on the native objects with the native display server somehow, and ensure no other such references exist. Surfaces were not intended to be that heavyweight.

Swapchains are intended to be the objects that directly manipulate native windows and communicate with the native presentation mechanisms. Swapchains will already need to communicate with the native display server to negotiate allocation and/or presentation of presentable images for a native surface. Therefore, it makes more sense for swapchain creation to be the point at which native object exclusivity is enforced. Platforms may choose to enforce further restrictions on the number of VkSurfaceKHR objects that may be created for the same native window if such a requirement makes sense on a particular platform, but a global requirement is only sensible at the swapchain level.

Examples

NOTE	The example code for the `VK_KHR_surface` and `VK_KHR_swapchain` extensions was removed from the appendix after revision 1.0.29. This WSI example code was ported to the cube demo that is shipped with the official Khronos SDK, and is being kept up-to-date in that location (see: https://github.com/KhronosGroup/Vulkan-Tools/blob/main/cube/cube.c).

Version History

- Revision 1, 2015-05-20 (James Jones)
  - Initial draft, based on LunarG KHR spec, other KHR specs, patches attached to bugs.
- Revision 2, 2015-05-22 (Ian Elliott)
  - Made many agreed-upon changes from 2015-05-21 KHR TSG meeting. This includes using only a queue for presentation, and having an explicit function to acquire the next image.
  - Fixed typos and other minor mistakes.
- Revision 3, 2015-05-26 (Ian Elliott)
  - Improved the Description section.
  - Added or resolved issues that were found in improving the Description. For example, pSurfaceDescription is used consistently, instead of sometimes using pSurface.
- Revision 4, 2015-05-27 (James Jones)
  - Fixed some grammatical errors and typos
  - Filled in the description of imageUseFlags when creating a swapchain.
  - Added a description of swapInterval.
  - Replaced the paragraph describing the order of operations on a queue for image ownership and presentation.
- Revision 5, 2015-05-27 (James Jones)
  - Imported relevant issues from the (abandoned) vk_wsi_persistent_swapchain_images extension.
  - Added issues 6 and 7, regarding behavior of the acquire next image and present commands with respect to queues.
  - Updated spec language and examples to align with proposed resolutions to issues 6 and 7.
- Revision 6, 2015-05-27 (James Jones)
  - Added issue 8, regarding atomic presentation of multiple swapchains
  - Updated spec language and examples to align with proposed resolution to issue 8.
- Revision 7, 2015-05-27 (James Jones)
  - Fixed compilation errors in example code, and made related spec fixes.
- Revision 8, 2015-05-27 (James Jones)
  - Added issue 9, and the related VK_SUBOPTIMAL_KHR result code.

    - Renamed VK_OUT_OF_DATE_KHR to VK_ERROR_OUT_OF_DATE_KHR.
- Revision 9, 2015-05-27 (James Jones)
    - Added inline proposed resolutions (marked with [JRJ]) to some XXX questions/issues. These should be moved to the issues section in a subsequent update if the proposals are adopted.
- Revision 10, 2015-05-28 (James Jones)
    - Converted vkAcquireNextImageKHR back to a non-queue operation that uses a VkSemaphore object for explicit synchronization.
    - Added issue 10 to determine whether vkAcquireNextImageKHR generates or returns semaphores, or whether it operates on a semaphore provided by the application.
- Revision 11, 2015-05-28 (James Jones)
    - Marked issues 6, 7, and 8 resolved.
    - Renamed VkSurfaceCapabilityPropertiesKHR to VkSurfacePropertiesKHR to better convey the mutable nature of the information it contains.
- Revision 12, 2015-05-28 (James Jones)
    - Added issue 11 with a proposed resolution, and the related issue 12.
    - Updated various sections of the spec to match the proposed resolution to issue 11.
- Revision 13, 2015-06-01 (James Jones)
    - Moved some structures to VK_EXT_KHR_swap_chain to resolve the specification's issues 1 and 2.
- Revision 14, 2015-06-01 (James Jones)
    - Added code for example 4 demonstrating how an application might make use of the two different present and acquire next image KHR result codes.
    - Added issue 13.
- Revision 15, 2015-06-01 (James Jones)
    - Added issues 14 - 16 and related spec language.
    - Fixed some spelling errors.
    - Added language describing the meaningful return values for vkAcquireNextImageKHR and vkQueuePresentKHR.
- Revision 16, 2015-06-02 (James Jones)
    - Added issues 17 and 18, as well as related spec language.
    - Removed some erroneous text added by mistake in the last update.
- Revision 17, 2015-06-15 (Ian Elliott)
    - Changed special value from "-1" to "0" so that the data types can be unsigned.
- Revision 18, 2015-06-15 (Ian Elliott)
    - Clarified the values of VkSurfacePropertiesKHR::minImageCount and the timeout parameter of the vkAcquireNextImageKHR function.
- Revision 19, 2015-06-17 (James Jones)

    - Misc. cleanup. Removed resolved inline issues and fixed typos.
    - Fixed clarification of VkSurfacePropertiesKHR::minImageCount made in version 18.
    - Added a brief “Image Ownership” definition to the list of terms used in the spec.
- Revision 20, 2015-06-17 (James Jones)
    - Updated enum-extending values using new convention.
- Revision 21, 2015-06-17 (James Jones)
    - Added language describing how to use VK_IMAGE_LAYOUT_PRESENT_SOURCE_KHR.
    - Cleaned up an XXX comment regarding the description of which queues vkQueuePresentKHR can be used on.
- Revision 22, 2015-06-17 (James Jones)
    - Rebased on Vulkan API version 126.
- Revision 23, 2015-06-18 (James Jones)
    - Updated language for issue 12 to read as a proposed resolution.
    - Marked issues 11, 12, 13, 16, and 17 resolved.
    - Temporarily added links to the relevant bugs under the remaining unresolved issues.
    - Added issues 19 and 20 as well as proposed resolutions.
- Revision 24, 2015-06-19 (Ian Elliott)
    - Changed special value for VkSurfacePropertiesKHR::currentExtent back to “-1” from “0”. This value will never need to be unsigned, and “0” is actually a legal value.
- Revision 25, 2015-06-23 (Ian Elliott)
    - Examples now show use of function pointers for extension functions.
    - Eliminated extraneous whitespace.
- Revision 26, 2015-06-25 (Ian Elliott)
    - Resolved Issues 9 & 10 per KHR TSG meeting.
- Revision 27, 2015-06-25 (James Jones)
    - Added oldSwapchain member to VkSwapchainCreateInfoKHR.
- Revision 28, 2015-06-25 (James Jones)
    - Added the “inherit” bits to the rotation and mirroring flags and the associated issue 21.
- Revision 29, 2015-06-25 (James Jones)
    - Added the “clipped” flag to VkSwapchainCreateInfoKHR, and the associated issue 22.
    - Specified that presenting an image does not modify it.
- Revision 30, 2015-06-25 (James Jones)
    - Added language to the spec that clarifies the behavior of vkCreateSwapchainKHR() when the oldSwapchain field of VkSwapchainCreateInfoKHR is not NULL.
- Revision 31, 2015-06-26 (Ian Elliott)

    - Example of new VkSwapchainCreateInfoKHR members, “oldSwapchain” and “clipped”.
    - Example of using VkSurfacePropertiesKHR::{min|max}ImageCount to set VkSwapchainCreateInfoKHR::minImageCount.
    - Rename vkGetSurfaceInfoKHR()'s 4th parameter to “pDataSize”, for consistency with other functions.
    - Add macro with C-string name of extension (just to header file).
- Revision 32, 2015-06-26 (James Jones)
    - Minor adjustments to the language describing the behavior of “oldSwapchain”
    - Fixed the version date on my previous two updates.
- Revision 33, 2015-06-26 (Jesse Hall)
    - Add usage flags to VkSwapchainCreateInfoKHR
- Revision 34, 2015-06-26 (Ian Elliott)
    - Rename vkQueuePresentKHR()'s 2nd parameter to “pPresentInfo”, for consistency with other functions.
- Revision 35, 2015-06-26 (Faith Ekstrand)
    - Merged the VkRotationFlagBitsKHR and VkMirrorFlagBitsKHR enums into a single VkSurfaceTransformFlagBitsKHR enum.
- Revision 36, 2015-06-26 (Faith Ekstrand)
    - Added a VkSurfaceTransformKHR enum that is not a bitmask. Each value in VkSurfaceTransformKHR corresponds directly to one of the bits in VkSurfaceTransformFlagBitsKHR so transforming from one to the other is easy. Having a separate enum means that currentTransform and preTransform are now unambiguous by definition.
- Revision 37, 2015-06-29 (Ian Elliott)
    - Corrected one of the signatures of vkAcquireNextImageKHR, which had the last two parameters switched from what it is elsewhere in the specification and header files.
- Revision 38, 2015-06-30 (Ian Elliott)
    - Corrected a typo in description of the vkGetSwapchainInfoKHR() function.
    - Corrected a typo in header file comment for VkPresentInfoKHR::sType.
- Revision 39, 2015-07-07 (Daniel Rakos)
    - Added error section describing when each error is expected to be reported.
    - Replaced bool32_t with VkBool32.
- Revision 40, 2015-07-10 (Ian Elliott)
    - Updated to work with version 138 of the `vulkan.h` header. This includes declaring the VkSwapchainKHR type using the new VK_DEFINE_NONDISP_HANDLE macro, and no longer extending VkObjectType (which was eliminated).
- Revision 41 2015-07-09 (Mathias Heyer)
    - Added color space language.

- Revision 42, 2015-07-10 (Daniel Rakos)
  - Updated query mechanism to reflect the convention changes done in the core spec.
  - Removed “queue” from the name of VK_STRUCTURE_TYPE_QUEUE_PRESENT_INFO_KHR to be consistent with the established naming convention.
  - Removed reference to the no longer existing VkObjectType enum.
- Revision 43, 2015-07-17 (Daniel Rakos)
  - Added support for concurrent sharing of swapchain images across queue families.
  - Updated sample code based on recent changes
- Revision 44, 2015-07-27 (Ian Elliott)
  - Noted that support for VK_PRESENT_MODE_FIFO_KHR is required. That is ICDs may optionally support IMMEDIATE and MAILBOX, but must support FIFO.
- Revision 45, 2015-08-07 (Ian Elliott)
  - Corrected a typo in spec file (type and variable name had wrong case for the imageColorSpace member of the VkSwapchainCreateInfoKHR struct).
  - Corrected a typo in header file (last parameter in PFN_vkGetSurfacePropertiesKHR was missing “KHR” at the end of type: VkSurfacePropertiesKHR).
- Revision 46, 2015-08-20 (Ian Elliott)
  - Renamed this extension and all of its enumerations, types, functions, etc. This makes it compliant with the proposed standard for Vulkan extensions.
  - Switched from “revision” to “version”, including use of the VK_MAKE_VERSION macro in the header file.
  - Made improvements to several descriptions.
  - Changed the status of several issues from PROPOSED to RESOLVED, leaving no unresolved issues.
  - Resolved several TODOs, did miscellaneous cleanup, etc.
- Revision 47, 2015-08-20 (Ian Elliott—porting a 2015-07-29 change from James Jones)
  - Moved the surface transform enums to VK_WSI_swapchain so they could be reused by VK_WSI_display.
- Revision 48, 2015-09-01 (James Jones)
  - Various minor cleanups.
- Revision 49, 2015-09-01 (James Jones)
  - Restore single-field revision number.
- Revision 50, 2015-09-01 (James Jones)
  - Update Example #4 to include code that illustrates how to use the oldSwapchain field.
- Revision 51, 2015-09-01 (James Jones)
  - Fix example code compilation errors.
- Revision 52, 2015-09-08 (Matthaeus G. Chajdas)

    - Corrected a typo.
- Revision 53, 2015-09-10 (Alon Or-bach)
    - Removed underscore from SWAP_CHAIN left in VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR.
- Revision 54, 2015-09-11 (Jesse Hall)
    - Described the execution and memory coherence requirements for image transitions to and from VK_IMAGE_LAYOUT_PRESENT_SOURCE_KHR.
- Revision 55, 2015-09-11 (Ray Smith)
    - Added errors for destroying and binding memory to presentable images
- Revision 56, 2015-09-18 (James Jones)
    - Added fence argument to vkAcquireNextImageKHR
    - Added example of how to meter a host thread based on presentation rate.
- Revision 57, 2015-09-26 (Jesse Hall)
    - Replace VkSurfaceDescriptionKHR with VkSurfaceKHR.
    - Added issue 25 with agreed resolution.
- Revision 58, 2015-09-28 (Jesse Hall)
    - Renamed from VK_EXT_KHR_device_swapchain to VK_EXT_KHR_swapchain.
- Revision 59, 2015-09-29 (Ian Elliott)
    - Changed vkDestroySwapchainKHR() to return void.
- Revision 60, 2015-10-01 (Jeff Vigil)
    - Added error result VK_ERROR_SURFACE_LOST_KHR.
- Revision 61, 2015-10-05 (Faith Ekstrand)
    - Added the VkCompositeAlpha enum and corresponding structure fields.
- Revision 62, 2015-10-12 (Daniel Rakos)
    - Added VK_PRESENT_MODE_FIFO_RELAXED_KHR.
- Revision 63, 2015-10-15 (Daniel Rakos)
    - Moved surface capability queries to VK_EXT_KHR_surface.
- Revision 64, 2015-10-26 (Ian Elliott)
    - Renamed from VK_EXT_KHR_swapchain to VK_KHR_swapchain.
- Revision 65, 2015-10-28 (Ian Elliott)
    - Added optional pResult member to VkPresentInfoKHR, so that per-swapchain results can be obtained from vkQueuePresentKHR().
- Revision 66, 2015-11-03 (Daniel Rakos)
    - Added allocation callbacks to create and destroy functions.
    - Updated resource transition language.

    - Updated sample code.
- Revision 67, 2015-11-10 (Jesse Hall)
    - Add reserved flags bitmask to VkSwapchainCreateInfoKHR.
    - Modify naming and member ordering to match API style conventions, and so the VkSwapchainCreateInfoKHR image property members mirror corresponding VkImageCreateInfo members but with an 'image' prefix.
    - Make VkPresentInfoKHR::pResults non-const; it is an output array parameter.
    - Make pPresentInfo parameter to vkQueuePresentKHR const.
- Revision 68, 2016-04-05 (Ian Elliott)
    - Moved the “validity” include for vkAcquireNextImage to be in its proper place, after the prototype and list of parameters.
    - Clarified language about presentable images, including how they are acquired, when applications can and cannot use them, etc. As part of this, removed language about “ownership” of presentable images, and replaced it with more-consistent language about presentable images being “acquired” by the application.
- 2016-08-23 (Ian Elliott)
    - Update the example code, to use the final API command names, to not have so many characters per line, and to split out a new example to show how to obtain function pointers. This code is more similar to the LunarG “cube” demo program.
- 2016-08-25 (Ian Elliott)
    - A note was added at the beginning of the example code, stating that it will be removed from future versions of the appendix.
- Revision 69, 2017-09-07 (Tobias Hector)
    - Added interactions with Vulkan 1.1
- Revision 70, 2017-10-06 (Ian Elliott)
    - Corrected interactions with Vulkan 1.1

VK_KHR_swapchain_mutable_format

Name String

`VK_KHR_swapchain_mutable_format`

Extension Type

Device extension

Registered Extension Number

201

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_swapchain
and
VK_KHR_maintenance2
or
Vulkan Version 1.1
and
VK_KHR_image_format_list
or
Vulkan Version 1.2

Contact

- Daniel Rakos [GitHub]drakos-amd

Other Extension Metadata

Last Modified Date

2018-03-28

IP Status

No known IP claims.

Contributors

- Faith Ekstrand, Intel
- Jan-Harald Fredriksen, ARM
- Jesse Hall, Google
- Daniel Rakos, AMD
- Ray Smith, ARM

Description

This extension allows processing of swapchain images as different formats to that used by the window system, which is particularly useful for switching between sRGB and linear RGB formats.

It adds a new swapchain creation flag that enables creating image views from presentable images with a different format than the one used to create the swapchain.

New Enum Constants

- `VK_KHR_SWAPCHAIN_MUTABLE_FORMAT_EXTENSION_NAME`
- `VK_KHR_SWAPCHAIN_MUTABLE_FORMAT_SPEC_VERSION`
- Extending VkSwapchainCreateFlagBitsKHR:
  - `VK_SWAPCHAIN_CREATE_MUTABLE_FORMAT_BIT_KHR`

Issues

1) Are there any new capabilities needed?

RESOLVED: No. It is expected that all implementations exposing this extension support swapchain image format mutability.

2) Do we need a separate `VK_SWAPCHAIN_CREATE_EXTENDED_USAGE_BIT_KHR`?

RESOLVED: No. This extension requires `VK_KHR_maintenance2` and presentable images of swapchains created with `VK_SWAPCHAIN_CREATE_MUTABLE_FORMAT_BIT_KHR` are created internally in a way equivalent to specifying both `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` and `VK_IMAGE_CREATE_EXTENDED_USAGE_BIT_KHR`.

3) Do we need a separate structure to allow specifying an image format list for swapchains?

RESOLVED: No. We simply use the same VkImageFormatListCreateInfoKHR structure introduced by `VK_KHR_image_format_list`. The structure is required to be included in the `pNext` chain of VkSwapchainCreateInfoKHR for swapchains created with `VK_SWAPCHAIN_CREATE_MUTABLE_FORMAT_BIT_KHR`.

Version History

- Revision 1, 2018-03-28 (Daniel Rakos)
  - Internal revisions.

VK_KHR_synchronization2

Name String

`VK_KHR_synchronization2`

Extension Type

Device extension

Registered Extension Number

315

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

API Interactions

- Interacts with VK_EXT_blend_operation_advanced

- Interacts with VK_EXT_conditional_rendering
- Interacts with VK_EXT_device_generated_commands
- Interacts with VK_EXT_fragment_density_map
- Interacts with VK_EXT_mesh_shader
- Interacts with VK_EXT_transform_feedback
- Interacts with VK_KHR_acceleration_structure
- Interacts with VK_KHR_fragment_shading_rate
- Interacts with VK_KHR_ray_tracing_pipeline
- Interacts with VK_NV_device_generated_commands
- Interacts with VK_NV_mesh_shader
- Interacts with VK_NV_ray_tracing
- Interacts with VK_NV_shading_rate_image

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Tobias Hector [GitHub]tobski

Other Extension Metadata

Last Modified Date

2020-12-03

Interactions and External Dependencies

- Interacts with `VK_KHR_create_renderpass2`

Contributors

- Tobias Hector

Description

This extension modifies the original core synchronization APIs to simplify the interface and improve usability of these APIs. It also adds new pipeline stage and access flag types that extend into the 64-bit range, as we have run out within the 32-bit range. The new flags are identical to the old values within the 32-bit range, with new stages and bits beyond that.

Pipeline stages and access flags are now specified together in memory barrier structures, making the connection between the two more obvious. Additionally, scoping the pipeline stages into the barrier structs allows the use of the `MEMORY_READ` and `MEMORY_WRITE` flags without sacrificing precision. The per-stage access flags should be used to disambiguate specific accesses in a given stage or set of stages - for instance, between uniform reads and sampling operations.

Layout transitions have been simplified as well; rather than requiring a different set of layouts for depth/stencil/color attachments, there are generic `VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR` and `VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR` layouts which are contextually applied based on the image

format. For example, for a depth format image, VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR is equivalent to VK_IMAGE_LAYOUT_DEPTH_READ_ONLY_OPTIMAL_KHR. VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR also functionally replaces VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL.

Events are now more efficient, because they include memory dependency information when you set them on the device. Previously, this information was only known when waiting on an event, so the dependencies could not be satisfied until the wait occurred. That sometimes meant stalling the pipeline when the wait occurred. The new API provides enough information for implementations to satisfy these dependencies in parallel with other tasks.

Queue submission has been changed to wrap command buffers and semaphores in extensible structures, which incorporate changes from Vulkan 1.1, VK_KHR_device_group, and VK_KHR_timeline_semaphore. This also adds a pipeline stage to the semaphore signal operation, mirroring the existing pipeline stage specification for wait operations.

Other miscellaneous changes include:

- Events can now be specified as interacting only with the device, allowing more efficient access to the underlying object.
- Image memory barriers that do not perform an image layout transition can be specified by setting oldLayout equal to newLayout.
    - E.g. the old and new layout can both be set to VK_IMAGE_LAYOUT_UNDEFINED, without discarding data in the image.
- Queue family ownership transfer parameters are simplified in some cases.
- Extensions with commands or functions with a VkPipelineStageFlags or VkPipelineStageFlagBits parameter have had those APIs replaced with equivalents using VkPipelineStageFlags2KHR.
- The new event and barrier interfaces are now more extensible for future changes.
- Relevant pipeline stage masks can now be specified as empty with the new VK_PIPELINE_STAGE_NONE_KHR and VK_PIPELINE_STAGE_2_NONE_KHR values.
- VkMemoryBarrier2KHR can be chained to VkSubpassDependency2, overriding the original 32-bit stage and access masks.

New Base Types

- VkFlags64

New Commands

- vkCmdPipelineBarrier2KHR
- vkCmdResetEvent2KHR
- vkCmdSetEvent2KHR
- vkCmdWaitEvents2KHR
- vkCmdWriteTimestamp2KHR
- vkQueueSubmit2KHR

New Structures

- VkBufferMemoryBarrier2KHR
- VkCommandBufferSubmitInfoKHR
- VkDependencyInfoKHR
- VkImageMemoryBarrier2KHR
- VkSemaphoreSubmitInfoKHR
- VkSubmitInfo2KHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceSynchronization2FeaturesKHR
- Extending VkSubpassDependency2:
  - VkMemoryBarrier2KHR

New Enums

- VkAccessFlagBits2KHR
- VkPipelineStageFlagBits2KHR
- VkSubmitFlagBitsKHR

New Bitmasks

- VkAccessFlags2KHR
- VkPipelineStageFlags2KHR
- VkSubmitFlagsKHR

New Enum Constants

- `VK_KHR_SYNCHRONIZATION_2_EXTENSION_NAME`
- `VK_KHR_SYNCHRONIZATION_2_SPEC_VERSION`
- Extending VkAccessFlagBits:
  - `VK_ACCESS_NONE_KHR`
- Extending VkEventCreateFlagBits:
  - `VK_EVENT_CREATE_DEVICE_ONLY_BIT_KHR`
- Extending VkImageLayout:
  - `VK_IMAGE_LAYOUT_ATTACHMENT_OPTIMAL_KHR`
  - `VK_IMAGE_LAYOUT_READ_ONLY_OPTIMAL_KHR`
- Extending VkPipelineStageFlagBits:
  - `VK_PIPELINE_STAGE_NONE_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2_KHR`

    - VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO_KHR
    - VK_STRUCTURE_TYPE_DEPENDENCY_INFO_KHR
    - VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2_KHR
    - VK_STRUCTURE_TYPE_MEMORY_BARRIER_2_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SYNCHRONIZATION_2_FEATURES_KHR
    - VK_STRUCTURE_TYPE_SEMAPHORE_SUBMIT_INFO_KHR
    - VK_STRUCTURE_TYPE_SUBMIT_INFO_2_KHR

If VK_EXT_blend_operation_advanced is supported:

- Extending VkAccessFlagBits2:
    - VK_ACCESS_2_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT

If VK_EXT_conditional_rendering is supported:

- Extending VkAccessFlagBits2:
    - VK_ACCESS_2_CONDITIONAL_RENDERING_READ_BIT_EXT
- Extending VkPipelineStageFlagBits2:
    - VK_PIPELINE_STAGE_2_CONDITIONAL_RENDERING_BIT_EXT

If VK_EXT_device_generated_commands is supported:

- Extending VkAccessFlagBits2:
    - VK_ACCESS_2_COMMAND_PREPROCESS_READ_BIT_EXT
    - VK_ACCESS_2_COMMAND_PREPROCESS_WRITE_BIT_EXT
- Extending VkPipelineStageFlagBits2:
    - VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_EXT

If VK_EXT_fragment_density_map is supported:

- Extending VkAccessFlagBits2:
    - VK_ACCESS_2_FRAGMENT_DENSITY_MAP_READ_BIT_EXT
- Extending VkPipelineStageFlagBits2:
    - VK_PIPELINE_STAGE_2_FRAGMENT_DENSITY_PROCESS_BIT_EXT

If VK_EXT_mesh_shader is supported:

- Extending VkPipelineStageFlagBits2:
    - VK_PIPELINE_STAGE_2_MESH_SHADER_BIT_EXT
    - VK_PIPELINE_STAGE_2_TASK_SHADER_BIT_EXT

If VK_EXT_transform_feedback is supported:

- Extending VkAccessFlagBits2:
  - VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT
  - VK_ACCESS_2_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT
  - VK_ACCESS_2_TRANSFORM_FEEDBACK_WRITE_BIT_EXT
- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_TRANSFORM_FEEDBACK_BIT_EXT

If VK_KHR_acceleration_structure is supported:

- Extending VkAccessFlagBits2:
  - VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR
  - VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_KHR
- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_KHR

If VK_KHR_fragment_shading_rate is supported:

- Extending VkAccessFlagBits2:
  - VK_ACCESS_2_FRAGMENT_SHADING_RATE_ATTACHMENT_READ_BIT_KHR
- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_KHR

If VK_KHR_ray_tracing_pipeline is supported:

- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_KHR

If VK_NV_device_generated_commands is supported:

- Extending VkAccessFlagBits2:
  - VK_ACCESS_2_COMMAND_PREPROCESS_READ_BIT_NV
  - VK_ACCESS_2_COMMAND_PREPROCESS_WRITE_BIT_NV
- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_COMMAND_PREPROCESS_BIT_NV

If VK_NV_mesh_shader is supported:

- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_MESH_SHADER_BIT_NV
  - VK_PIPELINE_STAGE_2_TASK_SHADER_BIT_NV

If VK_NV_ray_tracing is supported:

- Extending VkAccessFlagBits2:
  - `VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_NV`
  - `VK_ACCESS_2_ACCELERATION_STRUCTURE_WRITE_BIT_NV`
- Extending VkPipelineStageFlagBits2:
  - `VK_PIPELINE_STAGE_2_ACCELERATION_STRUCTURE_BUILD_BIT_NV`
  - `VK_PIPELINE_STAGE_2_RAY_TRACING_SHADER_BIT_NV`

If `VK_NV_shading_rate_image` is supported:

- Extending VkAccessFlagBits2:
  - `VK_ACCESS_2_SHADING_RATE_IMAGE_READ_BIT_NV`
- Extending VkPipelineStageFlagBits2:
  - `VK_PIPELINE_STAGE_2_SHADING_RATE_IMAGE_BIT_NV`

Promotion to Vulkan 1.3

Vulkan APIs in this extension are included in core Vulkan 1.3, with the KHR suffix omitted. External interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

Examples

See https://github.com/KhronosGroup/Vulkan-Docs/wiki/Synchronization-Examples

Version History

- Revision 1, 2020-12-03 (Tobias Hector)
  - Internal revisions

VK_KHR_timeline_semaphore

Name String

`VK_KHR_timeline_semaphore`

Extension Type

Device extension

Registered Extension Number

208

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Faith Ekstrand [GitHub]gfxstrand

Other Extension Metadata

Last Modified Date

2019-06-12

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension interacts with `VK_KHR_external_semaphore_capabilities`
- This extension interacts with `VK_KHR_external_semaphore`
- This extension interacts with `VK_KHR_external_semaphore_win32`

Contributors

- Jeff Bolz, NVIDIA
- Yuriy O'Donnell, Epic Games
- Faith Ekstrand, Intel
- Jesse Hall, Google
- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Daniel Rakos, AMD
- Ray Smith, Arm

Description

This extension introduces a new type of semaphore that has an integer payload identifying a point in a timeline. Such timeline semaphores support the following operations:

- Host query - A host operation that allows querying the payload of the timeline semaphore.
- Host wait - A host operation that allows a blocking wait for a timeline semaphore to reach a specified value.
- Host signal - A host operation that allows advancing the timeline semaphore to a specified value.
- Device wait - A device operation that allows waiting for a timeline semaphore to reach a

specified value.

- Device signal - A device operation that allows advancing the timeline semaphore to a specified value.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Commands

- vkGetSemaphoreCounterValueKHR
- vkSignalSemaphoreKHR
- vkWaitSemaphoresKHR

New Structures

- VkSemaphoreSignalInfoKHR
- VkSemaphoreWaitInfoKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceTimelineSemaphoreFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceTimelineSemaphorePropertiesKHR
- Extending VkSemaphoreCreateInfo, VkPhysicalDeviceExternalSemaphoreInfo:
  - VkSemaphoreTypeCreateInfoKHR
- Extending VkSubmitInfo, VkBindSparseInfo:
  - VkTimelineSemaphoreSubmitInfoKHR

New Enums

- VkSemaphoreTypeKHR
- VkSemaphoreWaitFlagBitsKHR

New Bitmasks

- VkSemaphoreWaitFlagsKHR

New Enum Constants

- `VK_KHR_TIMELINE_SEMAPHORE_EXTENSION_NAME`
- `VK_KHR_TIMELINE_SEMAPHORE_SPEC_VERSION`
- Extending VkSemaphoreType:
  - `VK_SEMAPHORE_TYPE_BINARY_KHR`

    - VK_SEMAPHORE_TYPE_TIMELINE_KHR
- Extending VkSemaphoreWaitFlagBits:
    - VK_SEMAPHORE_WAIT_ANY_BIT_KHR
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_FEATURES_KHR
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TIMELINE_SEMAPHORE_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_SEMAPHORE_SIGNAL_INFO_KHR
    - VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO_KHR
    - VK_STRUCTURE_TYPE_SEMAPHORE_WAIT_INFO_KHR
    - VK_STRUCTURE_TYPE_TIMELINE_SEMAPHORE_SUBMIT_INFO_KHR

Issues

1) Do we need a new object type for this?

RESOLVED: No, we just introduce a new type of semaphore object, as VK_KHR_external_semaphore_win32 already uses semaphores as the destination for importing D3D12 fence objects, which are semantically close/identical to the proposed synchronization primitive.

2) What type of payload the new synchronization primitive has?

RESOLVED: A 64-bit unsigned integer that can only be set to strictly increasing values by signal operations and is not changed by wait operations.

3) Does the new synchronization primitive have the same signal-before-wait requirement as the existing semaphores do?

RESOLVED: No. Timeline semaphores support signaling and waiting entirely asynchronously. It is the responsibility of the application to avoid deadlock.

4) Does the new synchronization primitive allow resetting its payload?

RESOLVED: No, allowing the payload value to “go backwards” is problematic. Applications looking for reset behavior should create a new instance of the synchronization primitive instead.

5) How do we enable host waits on the synchronization primitive?

RESOLVED: Both a non-blocking query of the current payload value of the synchronization primitive, and a blocking wait operation are provided.

6) How do we enable device waits and signals on the synchronization primitive?

RESOLVED: Similar to VK_KHR_external_semaphore_win32, this extension introduces a new structure that can be chained to VkSubmitInfo to specify the values signaled semaphores should be set to, and the values waited semaphores need to reach.

7) Can the new synchronization primitive be used to synchronize presentation and swapchain

image acquisition operations?

RESOLVED: Some implementations may have problems with supporting that directly, thus it is not allowed in this extension.

8) Do we want to support external sharing of the new synchronization primitive type?

RESOLVED: Yes. Timeline semaphore specific external sharing capabilities can be queried using vkGetPhysicalDeviceExternalSemaphorePropertiesKHR by chaining the new VkSemaphoreTypeCreateInfoKHR structure to its `pExternalSemaphoreInfo` structure. This allows having a different set of external semaphore handle types supported for timeline semaphores vs. binary semaphores.

9) Do we need to add a host signal operation for the new synchronization primitive type?

RESOLVED: Yes. This helps in situations where one host thread submits a workload but another host thread has the information on when the workload is ready to be executed.

10) How should the new synchronization primitive interact with the ordering requirements of the original `VkSemaphore`?

RESOLVED: Prior to calling any command which **may** cause a wait operation on a binary semaphore, the application **must** ensure that the semaphore signal operation that has been submitted for execution and any semaphore signal operations on which it depends (if any) **must** have also been submitted for execution.

11) Should we have separate feature bits for different sub-features of timeline semaphores?

RESOLVED: No. The only feature which cannot be supported universally is timeline semaphore import/export. For import/export, the application is already required to query available external handle types via vkGetPhysicalDeviceExternalSemaphorePropertiesKHR and provide the semaphore type by adding a VkSemaphoreTypeCreateInfoKHR structure to the `pNext` chain of VkPhysicalDeviceExternalSemaphoreInfo so no new feature bit is required.

Version History

- Revision 1, 2018-05-10 (Faith Ekstrand)
  - Initial version
- Revision 2, 2019-06-12 (Faith Ekstrand)
  - Added an initialValue parameter to timeline semaphore creation

VK_KHR_uniform_buffer_standard_layout

Name String

`VK_KHR_uniform_buffer_standard_layout`

Extension Type

Device extension

Registered Extension Number

254

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Graeme Leese [GitHub]gnl21

Other Extension Metadata

Last Modified Date

2019-01-25

Contributors

- Graeme Leese, Broadcom
- Jeff Bolz, NVIDIA
- Tobias Hector, AMD
- Faith Ekstrand, Intel
- Neil Henning, AMD

Description

This extension enables tighter array and struct packing to be used with uniform buffers.

It modifies the alignment rules for uniform buffers, allowing for tighter packing of arrays and structures. This allows, for example, the std430 layout, as defined in GLSL to be supported in uniform buffers.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:

    - VkPhysicalDeviceUniformBufferStandardLayoutFeaturesKHR

New Enum Constants

- `VK_KHR_UNIFORM_BUFFER_STANDARD_LAYOUT_EXTENSION_NAME`
- `VK_KHR_UNIFORM_BUFFER_STANDARD_LAYOUT_SPEC_VERSION`
- Extending VkStructureType:
    - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_UNIFORM_BUFFER_STANDARD_LAYOUT_FEATURES_KHR`

Version History

- Revision 1, 2019-01-25 (Graeme Leese)
    - Initial draft

VK_KHR_variable_pointers

Name String

`VK_KHR_variable_pointers`

Extension Type

Device extension

Registered Extension Number

121

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
and
VK_KHR_storage_buffer_storage_class
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_variable_pointers

Deprecation State

- *Promoted* to Vulkan 1.1

Contact

- Jesse Hall [GitHub]critsec

Other Extension Metadata

Last Modified Date

2017-09-05

IP Status

No known IP claims.

Contributors

- John Kessenich, Google
- Neil Henning, Codeplay
- David Neto, Google
- Daniel Koch, Nvidia
- Graeme Leese, Broadcom
- Weifeng Zhang, Qualcomm
- Stephen Clarke, Imagination Technologies
- Faith Ekstrand, Intel
- Jesse Hall, Google

Description

The `VK_KHR_variable_pointers` extension allows implementations to indicate their level of support for the `SPV_KHR_variable_pointers` SPIR-V extension. The SPIR-V extension allows shader modules to use invocation-private pointers into uniform and/or storage buffers, where the pointer values can be dynamic and non-uniform.

The `SPV_KHR_variable_pointers` extension introduces two capabilities. The first, `VariablePointersStorageBuffer`, **must** be supported by all implementations of this extension. The second, `VariablePointers`, is optional.

Promotion to Vulkan 1.1

All functionality in this extension is included in core Vulkan 1.1, with the KHR suffix omitted, however support for the `variablePointersStorageBuffer` feature is made optional. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceVariablePointerFeaturesKHR
  - VkPhysicalDeviceVariablePointersFeaturesKHR

New Enum Constants

- `VK_KHR_VARIABLE_POINTERS_EXTENSION_NAME`

- VK_KHR_VARIABLE_POINTERS_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VARIABLE_POINTERS_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VARIABLE_POINTER_FEATURES_KHR

New SPIR-V Capabilities

- VariablePointers
- VariablePointersStorageBuffer

Issues

1) Do we need an optional property for the SPIR-V VariablePointersStorageBuffer capability or should it be mandatory when this extension is advertised?

RESOLVED: Add it as a distinct feature, but make support mandatory. Adding it as a feature makes the extension easier to include in a future core API version. In the extension, the feature is mandatory, so that presence of the extension guarantees some functionality. When included in a core API version, the feature would be optional.

2) Can support for these capabilities vary between shader stages?

RESOLVED: No, if the capability is supported in any stage it must be supported in all stages.

3) Should the capabilities be features or limits?

RESOLVED: Features, primarily for consistency with other similar extensions.

Version History

- Revision 1, 2017-03-14 (Jesse Hall and John Kessenich)
  - Internal revisions

VK_KHR_vertex_attribute_divisor

Name String

VK_KHR_vertex_attribute_divisor

Extension Type

Device extension

Registered Extension Number

526

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Shahbaz Youssefi [GitHub]syoussefi

Extension Proposal

VK_KHR_vertex_attribute_divisor

Other Extension Metadata

Last Modified Date

2023-09-20

IP Status

No known IP claims.

Contributors

- Shahbaz Youssefi, Google
- Contributors to `VK_EXT_vertex_attribute_divisor`

Description

This extension is based on the `VK_EXT_vertex_attribute_divisor` extension. The only difference is the new property `supportsNonZeroFirstInstance`, which indicates support for non-zero values in `firstInstance`. This allows the extension to be supported on implementations that have traditionally only supported OpenGL ES.

New Structures

- VkVertexInputBindingDivisorDescriptionKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceVertexAttributeDivisorFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceVertexAttributeDivisorPropertiesKHR
- Extending VkPipelineVertexInputStateCreateInfo:
  - VkPipelineVertexInputDivisorStateCreateInfoKHR

New Enum Constants

- `VK_KHR_VERTEX_ATTRIBUTE_DIVISOR_EXTENSION_NAME`

- VK_KHR_VERTEX_ATTRIBUTE_DIVISOR_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_ATTRIBUTE_DIVISOR_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_ATTRIBUTE_DIVISOR_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_DIVISOR_STATE_CREATE_INFO_KHR

Version History

- Revision 1, 2023-09-20 (Shahbaz Youssefi)
  - First Version, based on VK_EXT_vertex_attribute_divisor

VK_KHR_video_decode_av1

Name String

VK_KHR_video_decode_av1

Extension Type

Device extension

Registered Extension Number

513

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_decode_queue

Contact

- Daniel Rakos [GitHub]aqnuep

Extension Proposal

VK_KHR_video_decode_av1

Other Extension Metadata

Last Modified Date

2024-01-02

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD

- Benjamin Cheng, AMD
- Ho Hin Lau, AMD
- Lynne Iribarren, Independent
- David Airlie, Red Hat, Inc.
- Ping Liu, Intel
- Srinath Kumarapuram, NVIDIA
- Vassili Nikolaev, NVIDIA
- Tony Zlatinski, NVIDIA
- Charlie Turner, Igalia
- Daniel Almeida, Collabora
- Nicolas Dufresne, Collabora
- Daniel Rakos, RasterGrid

Description

This extension builds upon the `VK_KHR_video_decode_queue` extension by adding support for decoding elementary video stream sequences compliant with the AV1 video compression standard.

New Structures

- Extending VkVideoCapabilitiesKHR:
  - VkVideoDecodeAV1CapabilitiesKHR
- Extending VkVideoDecodeInfoKHR:
  - VkVideoDecodeAV1PictureInfoKHR
- Extending VkVideoProfileInfoKHR, VkQueryPoolCreateInfo:
  - VkVideoDecodeAV1ProfileInfoKHR
- Extending VkVideoReferenceSlotInfoKHR:
  - VkVideoDecodeAV1DpbSlotInfoKHR
- Extending VkVideoSessionParametersCreateInfoKHR:
  - VkVideoDecodeAV1SessionParametersCreateInfoKHR

New Enum Constants

- `VK_KHR_VIDEO_DECODE_AV1_EXTENSION_NAME`
- `VK_KHR_VIDEO_DECODE_AV1_SPEC_VERSION`
- `VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_CAPABILITIES_KHR`
  - `VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_DPB_SLOT_INFO_KHR`

    - VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_PICTURE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_PROFILE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_DECODE_AV1_SESSION_PARAMETERS_CREATE_INFO_KHR
- Extending VkVideoCodecOperationFlagBitsKHR:
    - VK_VIDEO_CODEC_OPERATION_DECODE_AV1_BIT_KHR

Version History

- Revision 1, 2024-01-02 (Daniel Rakos)
    - Internal revisions

VK_KHR_video_decode_h264

Name String

VK_KHR_video_decode_h264

Extension Type

Device extension

Registered Extension Number

41

Revision

9

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_decode_queue

Contact

- peter.fang@amd.com

Extension Proposal

VK_KHR_video_decode_h264

Other Extension Metadata

Last Modified Date

2023-12-05

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD

- Chunbo Chen, Intel
- HoHin Lau, AMD
- Jake Beju, AMD
- Peter Fang, AMD
- Ping Liu, Intel
- Srinath Kumarapuram, NVIDIA
- Tony Zlatinski, NVIDIA
- Daniel Rakos, RasterGrid

Description

This extension builds upon the VK_KHR_video_decode_queue extension by adding support for decoding elementary video stream sequences compliant with the H.264/AVC video compression standard.

NOTE	This extension was promoted to KHR from the provisional extension VK_EXT_video_decode_h264.

New Structures

- Extending VkVideoCapabilitiesKHR:
  - VkVideoDecodeH264CapabilitiesKHR
- Extending VkVideoDecodeInfoKHR:
  - VkVideoDecodeH264PictureInfoKHR
- Extending VkVideoProfileInfoKHR, VkQueryPoolCreateInfo:
  - VkVideoDecodeH264ProfileInfoKHR
- Extending VkVideoReferenceSlotInfoKHR:
  - VkVideoDecodeH264DpbSlotInfoKHR
- Extending VkVideoSessionParametersCreateInfoKHR:
  - VkVideoDecodeH264SessionParametersCreateInfoKHR
- Extending VkVideoSessionParametersUpdateInfoKHR:
  - VkVideoDecodeH264SessionParametersAddInfoKHR

New Enums

- VkVideoDecodeH264PictureLayoutFlagBitsKHR

New Bitmasks

- VkVideoDecodeH264PictureLayoutFlagsKHR

New Enum Constants

- `VK_KHR_VIDEO_DECODE_H264_EXTENSION_NAME`
- `VK_KHR_VIDEO_DECODE_H264_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_CAPABILITIES_KHR`
  - `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_DPB_SLOT_INFO_KHR`
  - `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_PICTURE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_PROFILE_INFO_KHR`
  - `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_SESSION_PARAMETERS_ADD_INFO_KHR`
  - `VK_STRUCTURE_TYPE_VIDEO_DECODE_H264_SESSION_PARAMETERS_CREATE_INFO_KHR`
- Extending VkVideoCodecOperationFlagBitsKHR:
  - `VK_VIDEO_CODEC_OPERATION_DECODE_H264_BIT_KHR`

Version History

- Revision 1, 2018-6-11 (Peter Fang)
  - Initial draft
- Revision 2, March 29 2021 (Tony Zlatinski)
  - Spec and API Updates
- Revision 3, August 1 2021 (Srinath Kumarapuram)
  - Rename `VkVideoDecodeH264FieldLayoutFlagsEXT` to `VkVideoDecodeH264PictureLayoutFlagsEXT`, `VkVideoDecodeH264FieldLayoutFlagBitsEXT` to `VkVideoDecodeH264PictureLayoutFlagBitsEXT` (along with the names of enumerants it defines), and `VkVideoDecodeH264ProfileEXT.fieldLayout` to `VkVideoDecodeH264ProfileEXT.pictureLayout`, following Vulkan naming conventions.
- Revision 4, 2022-03-16 (Ahmed Abdelkhalek)
  - Relocate Std header version reporting/requesting from this extension to VK_KHR_video_queue extension.
  - Remove the now empty VkVideoDecodeH264SessionCreateInfoEXT.
- Revision 5, 2022-03-31 (Ahmed Abdelkhalek)
  - Use type StdVideoH264Level for VkVideoDecodeH264Capabilities.maxLevel
- Revision 6, 2022-08-09 (Daniel Rakos)
  - Rename `VkVideoDecodeH264ProfileEXT` to `VkVideoDecodeH264ProfileInfoEXT`
  - Rename `VkVideoDecodeH264MvcEXT` to `VkVideoDecodeH264MvcInfoEXT`
- Revision 7, 2022-09-18 (Daniel Rakos)
  - Change type of `VkVideoDecodeH264ProfileInfoEXT::pictureLayout` to `VkVideoDecodeH264PictureLayoutFlagBitsEXT`

    - Remove MVC support and related `VkVideoDecodeH264MvcInfoEXT` structure
    - Rename `spsStdCount`, `pSpsStd`, `ppsStdCount`, and `pPpsStd` to `stdSPSCount`, `pStdSPSs`, `stdPPSCount`, and `pStdPPSs`, respectively, in `VkVideoDecodeH264SessionParametersAddInfoEXT`
    - Rename `maxSpsStdCount` and `maxPpsStdCount` to `maxStdSPSCount` and `maxStdPPSCount`, respectively, in `VkVideoDecodeH264SessionParametersCreateInfoEXT`
    - Rename `slicesCount` and `pSlicesDataOffsets` to `sliceCount` and `pSliceOffsets`, respectively, in `VkVideoDecodeH264PictureInfoEXT`
- Revision 8, 2022-09-29 (Daniel Rakos)
    - Change extension from `EXT` to `KHR`
    - Extension is no longer provisional
- Revision 9, 2023-12-05 (Daniel Rakos)
    - Condition reference picture setup based on the value of `StdVideoDecodeH264PictureInfo::flags.is_reference`

VK_KHR_video_decode_h265

Name String

`VK_KHR_video_decode_h265`

Extension Type

Device extension

Registered Extension Number

188

Revision

8

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_decode_queue

Contact

- peter.fang@amd.com

Extension Proposal

VK_KHR_video_decode_h265

Other Extension Metadata

Last Modified Date

2023-12-05

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- HoHin Lau, AMD
- Jake Beju, AMD
- Peter Fang, AMD
- Ping Liu, Intel
- Srinath Kumarapuram, NVIDIA
- Tony Zlatinski, NVIDIA
- Daniel Rakos, RasterGrid

Description

This extension builds upon the `VK_KHR_video_decode_queue` extension by adding support for decoding elementary video stream sequences compliant with the H.265/HEVC video compression standard.

NOTE	This extension was promoted to `KHR` from the provisional extension `VK_EXT_video_decode_h265`.

New Structures

- Extending VkVideoCapabilitiesKHR:
  - VkVideoDecodeH265CapabilitiesKHR
- Extending VkVideoDecodeInfoKHR:
  - VkVideoDecodeH265PictureInfoKHR
- Extending VkVideoProfileInfoKHR, VkQueryPoolCreateInfo:
  - VkVideoDecodeH265ProfileInfoKHR
- Extending VkVideoReferenceSlotInfoKHR:
  - VkVideoDecodeH265DpbSlotInfoKHR
- Extending VkVideoSessionParametersCreateInfoKHR:
  - VkVideoDecodeH265SessionParametersCreateInfoKHR
- Extending VkVideoSessionParametersUpdateInfoKHR:
  - VkVideoDecodeH265SessionParametersAddInfoKHR

New Enum Constants

- `VK_KHR_VIDEO_DECODE_H265_EXTENSION_NAME`
- `VK_KHR_VIDEO_DECODE_H265_SPEC_VERSION`
- Extending VkStructureType:

    - VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_CAPABILITIES_KHR
    - VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_DPB_SLOT_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_PICTURE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_PROFILE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_SESSION_PARAMETERS_ADD_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_DECODE_H265_SESSION_PARAMETERS_CREATE_INFO_KHR
- Extending VkVideoCodecOperationFlagBitsKHR:
    - VK_VIDEO_CODEC_OPERATION_DECODE_H265_BIT_KHR

Version History

- Revision 1, 2018-6-11 (Peter Fang)
    - Initial draft
- Revision 1.6, March 29 2021 (Tony Zlatinski)
    - Spec and API updates.
- Revision 2, 2022-03-16 (Ahmed Abdelkhalek)
    - Relocate Std header version reporting/requesting from this extension to VK_KHR_video_queue extension.
    - Remove the now empty VkVideoDecodeH265SessionCreateInfoEXT.
- Revision 3, 2022-03-31 (Ahmed Abdelkhalek)
    - Use type StdVideoH265Level for VkVideoDecodeH265Capabilities.maxLevel
- Revision 4, 2022-08-09 (Daniel Rakos)
    - Rename VkVideoDecodeH265ProfileEXT to VkVideoDecodeH265ProfileInfoEXT
- Revision 5, 2022-09-18 (Daniel Rakos)
    - Rename vpsStdCount, pVpsStd, spsStdCount, pSpsStd, ppsStdCount, and pPpsStd to stdVPSCount, pStdVPSs, stdSPSCount, pStdSPSs, stdPPSCount, and pStdPPSs, respectively, in VkVideoDecodeH265SessionParametersAddInfoEXT
    - Rename maxVpsStdCount, maxSpsStdCount, and maxPpsStdCount to maxStdVPSCount, maxStdSPSCount and maxStdPPSCount, respectively, in VkVideoDecodeH265SessionParametersCreateInfoEXT
    - Rename slicesCount and pSlicesDataOffsets to sliceCount and pSliceOffsets, respectively, in VkVideoDecodeH265PictureInfoEXT
- Revision 6, 2022-11-14 (Daniel Rakos)
    - Rename slice to sliceSegment in the APIs for better clarity
- Revision 7, 2022-11-14 (Daniel Rakos)
    - Change extension from EXT to KHR
    - Extension is no longer provisional
- Revision 8, 2023-12-05 (Daniel Rakos)

- Condition reference picture setup based on the value of `StdVideoDecodeH265PictureInfo::flags.IsReference`

VK_KHR_video_decode_queue

Name String

`VK_KHR_video_decode_queue`

Extension Type

Device extension

Registered Extension Number

25

Revision

8

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_queue
and
VK_KHR_synchronization2
or
Vulkan Version 1.3

API Interactions

- Interacts with VK_VERSION_1_3
- Interacts with VK_KHR_format_feature_flags2

Contact

- jake.beju@amd.com

Extension Proposal

VK_KHR_video_decode_queue

Other Extension Metadata

Last Modified Date

2023-12-05

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- Jake Beju, AMD

- Olivier Lapicque, NVIDIA
- Peter Fang, AMD
- Piers Daniell, NVIDIA
- Srinath Kumarapuram, NVIDIA
- Tony Zlatinski, NVIDIA
- Daniel Rakos, RasterGrid

Description

This extension builds upon the `VK_KHR_video_queue` extension by adding common APIs specific to video decoding and thus enabling implementations to expose queue families supporting video decode operations.

More specifically, it adds video decode specific capabilities and a new command buffer command that allows recording video decode operations against a video session.

This extension is to be used in conjunction with other codec specific video decode extensions that enable decoding video sequences of specific video compression standards.

New Commands

- vkCmdDecodeVideoKHR

New Structures

- VkVideoDecodeInfoKHR
- Extending VkVideoCapabilitiesKHR:
  - VkVideoDecodeCapabilitiesKHR
- Extending VkVideoProfileInfoKHR, VkQueryPoolCreateInfo:
  - VkVideoDecodeUsageInfoKHR

New Enums

- VkVideoDecodeCapabilityFlagBitsKHR
- VkVideoDecodeUsageFlagBitsKHR

New Bitmasks

- VkVideoDecodeCapabilityFlagsKHR
- VkVideoDecodeFlagsKHR
- VkVideoDecodeUsageFlagsKHR

New Enum Constants

- `VK_KHR_VIDEO_DECODE_QUEUE_EXTENSION_NAME`

- VK_KHR_VIDEO_DECODE_QUEUE_SPEC_VERSION
- Extending VkAccessFlagBits2:
  - VK_ACCESS_2_VIDEO_DECODE_READ_BIT_KHR
  - VK_ACCESS_2_VIDEO_DECODE_WRITE_BIT_KHR
- Extending VkBufferUsageFlagBits:
  - VK_BUFFER_USAGE_VIDEO_DECODE_DST_BIT_KHR
  - VK_BUFFER_USAGE_VIDEO_DECODE_SRC_BIT_KHR
- Extending VkFormatFeatureFlagBits:
  - VK_FORMAT_FEATURE_VIDEO_DECODE_DPB_BIT_KHR
  - VK_FORMAT_FEATURE_VIDEO_DECODE_OUTPUT_BIT_KHR
- Extending VkImageLayout:
  - VK_IMAGE_LAYOUT_VIDEO_DECODE_DPB_KHR
  - VK_IMAGE_LAYOUT_VIDEO_DECODE_DST_KHR
  - VK_IMAGE_LAYOUT_VIDEO_DECODE_SRC_KHR
- Extending VkImageUsageFlagBits:
  - VK_IMAGE_USAGE_VIDEO_DECODE_DPB_BIT_KHR
  - VK_IMAGE_USAGE_VIDEO_DECODE_DST_BIT_KHR
  - VK_IMAGE_USAGE_VIDEO_DECODE_SRC_BIT_KHR
- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_VIDEO_DECODE_BIT_KHR
- Extending VkQueueFlagBits:
  - VK_QUEUE_VIDEO_DECODE_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_VIDEO_DECODE_CAPABILITIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_DECODE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_DECODE_USAGE_INFO_KHR

If VK_KHR_format_feature_flags2 or Vulkan Version 1.3 is supported:

- Extending VkFormatFeatureFlagBits2:
  - VK_FORMAT_FEATURE_2_VIDEO_DECODE_DPB_BIT_KHR
  - VK_FORMAT_FEATURE_2_VIDEO_DECODE_OUTPUT_BIT_KHR

Version History

- Revision 1, 2018-6-11 (Peter Fang)
  - Initial draft

- Revision 1.5, Nov 09 2018 (Tony Zlatinski)
  - API Updates
- Revision 1.6, Jan 08 2020 (Tony Zlatinski)
  - API unify with the video_encode_queue spec
- Revision 1.7, March 29 2021 (Tony Zlatinski)
  - Spec and API updates.
- Revision 2, September 30 2021 (Jon Leech)
  - Add interaction with `VK_KHR_format_feature_flags2` to `vk.xml`
- Revision 3, 2022-02-25 (Ahmed Abdelkhalek)
  - Add VkVideoDecodeCapabilitiesKHR with new flags to report support for decode DPB and output coinciding in the same image, or in distinct images.
- Revision 4, 2022-03-31 (Ahmed Abdelkhalek)
  - Remove redundant VkVideoDecodeInfoKHR.coded{Offset|Extent}
- Revision 5, 2022-07-18 (Daniel Rakos)
  - Remove `VkVideoDecodeFlagBitsKHR` as it contains no defined flags for now
- Revision 6, 2022-08-12 (Daniel Rakos)
  - Add VkVideoDecodeUsageInfoKHR structure and related flags
- Revision 7, 2022-09-29 (Daniel Rakos)
  - Extension is no longer provisional
- Revision 8, 2023-12-05 (Daniel Rakos)
  - Require the specification of a reconstructed picture in all cases, except when the video session was created with no DPB slots to match shipping implementations
  - Make DPB slot activation behavior codec-specific to continue allowing application control over reference picture setup now that a reconstructed picture is always mandatory

VK_KHR_video_encode_av1

Name String

`VK_KHR_video_encode_av1`

Extension Type

Device extension

Registered Extension Number

514

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_encode_queue

Contact

- Daniel Rakos [GitHub]aqnuep

Extension Proposal

VK_KHR_video_encode_av1

Other Extension Metadata

Last Modified Date

2024-09-23

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- Benjamin Cheng, AMD
- Ho Hin Lau, AMD
- Lynne Iribarren, Independent
- David Airlie, Red Hat, Inc.
- Ping Liu, Intel
- Srinath Kumarapuram, NVIDIA
- Vassili Nikolaev, NVIDIA
- Tony Zlatinski, NVIDIA
- Konda Raju, NVIDIA
- Charlie Turner, Igalia
- Daniel Almeida, Collabora
- Nicolas Dufresne, Collabora
- Daniel Rakos, RasterGrid

Description

This extension builds upon the `VK_KHR_video_encode_queue` extension by adding support for encoding elementary video stream sequences compliant with the AV1 video compression standard.

New Structures

- VkVideoEncodeAV1FrameSizeKHR

- VkVideoEncodeAV1QIndexKHR
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceVideoEncodeAV1FeaturesKHR
- Extending VkVideoBeginCodingInfoKHR:
  - VkVideoEncodeAV1GopRemainingFrameInfoKHR
- Extending VkVideoCapabilitiesKHR:
  - VkVideoEncodeAV1CapabilitiesKHR
- Extending VkVideoCodingControlInfoKHR, VkVideoBeginCodingInfoKHR:
  - VkVideoEncodeAV1RateControlInfoKHR
- Extending VkVideoEncodeInfoKHR:
  - VkVideoEncodeAV1PictureInfoKHR
- Extending VkVideoEncodeQualityLevelPropertiesKHR:
  - VkVideoEncodeAV1QualityLevelPropertiesKHR
- Extending VkVideoEncodeRateControlLayerInfoKHR:
  - VkVideoEncodeAV1RateControlLayerInfoKHR
- Extending VkVideoProfileInfoKHR, VkQueryPoolCreateInfo:
  - VkVideoEncodeAV1ProfileInfoKHR
- Extending VkVideoReferenceSlotInfoKHR:
  - VkVideoEncodeAV1DpbSlotInfoKHR
- Extending VkVideoSessionCreateInfoKHR:
  - VkVideoEncodeAV1SessionCreateInfoKHR
- Extending VkVideoSessionParametersCreateInfoKHR:
  - VkVideoEncodeAV1SessionParametersCreateInfoKHR

New Enums

- VkVideoEncodeAV1CapabilityFlagBitsKHR
- VkVideoEncodeAV1PredictionModeKHR
- VkVideoEncodeAV1RateControlFlagBitsKHR
- VkVideoEncodeAV1RateControlGroupKHR
- VkVideoEncodeAV1StdFlagBitsKHR
- VkVideoEncodeAV1SuperblockSizeFlagBitsKHR

New Bitmasks

- VkVideoEncodeAV1CapabilityFlagsKHR
- VkVideoEncodeAV1RateControlFlagsKHR

- VkVideoEncodeAV1StdFlagsKHR
- VkVideoEncodeAV1SuperblockSizeFlagsKHR

New Enum Constants

- VK_KHR_VIDEO_ENCODE_AV1_EXTENSION_NAME
- VK_KHR_VIDEO_ENCODE_AV1_SPEC_VERSION
- VK_MAX_VIDEO_AV1_REFERENCES_PER_FRAME_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_AV1_FEATURES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_CAPABILITIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_DPB_SLOT_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_GOP_REMAINING_FRAME_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_PICTURE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_PROFILE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_QUALITY_LEVEL_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_RATE_CONTROL_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_RATE_CONTROL_LAYER_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_SESSION_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_SESSION_PARAMETERS_CREATE_INFO_KHR
- Extending VkVideoCodecOperationFlagBitsKHR:
  - VK_VIDEO_CODEC_OPERATION_ENCODE_AV1_BIT_KHR

Version History

- Revision 1, 2024-09-23 (Daniel Rakos)
  - Internal revisions

VK_KHR_video_encode_h264

Name String

VK_KHR_video_encode_h264

Extension Type

Device extension

Registered Extension Number

39

Revision

14

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_encode_queue

Contact

- Ahmed Abdelkhalek [GitHub]aabdelkh

Extension Proposal

VK_KHR_video_encode_h264

Other Extension Metadata

Last Modified Date

2023-12-05

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- George Hao, AMD
- Jake Beju, AMD
- Peter Fang, AMD
- Ping Liu, Intel
- Srinath Kumarapuram, NVIDIA
- Tony Zlatinski, NVIDIA
- Ravi Chaudhary, NVIDIA
- Yang Liu, AMD
- Daniel Rakos, RasterGrid
- Aidan Fabius, Core Avionics & Industrial Inc.
- Lynne Iribarren, Independent

Description

This extension builds upon the `VK_KHR_video_encode_queue` extension by adding support for encoding elementary video stream sequences compliant with the H.264/AVC video compression standard.

NOTE	This extension was promoted to `KHR` from the provisional extension `VK_EXT_video_encode_h264`.

New Structures

- VkVideoEncodeH264FrameSizeKHR
- VkVideoEncodeH264NaluSliceInfoKHR
- VkVideoEncodeH264QpKHR
- Extending VkVideoBeginCodingInfoKHR:
  - VkVideoEncodeH264GopRemainingFrameInfoKHR
- Extending VkVideoCapabilitiesKHR:
  - VkVideoEncodeH264CapabilitiesKHR
- Extending VkVideoCodingControlInfoKHR, VkVideoBeginCodingInfoKHR:
  - VkVideoEncodeH264RateControlInfoKHR
- Extending VkVideoEncodeInfoKHR:
  - VkVideoEncodeH264PictureInfoKHR
- Extending VkVideoEncodeQualityLevelPropertiesKHR:
  - VkVideoEncodeH264QualityLevelPropertiesKHR
- Extending VkVideoEncodeRateControlLayerInfoKHR:
  - VkVideoEncodeH264RateControlLayerInfoKHR
- Extending VkVideoEncodeSessionParametersFeedbackInfoKHR:
  - VkVideoEncodeH264SessionParametersFeedbackInfoKHR
- Extending VkVideoEncodeSessionParametersGetInfoKHR:
  - VkVideoEncodeH264SessionParametersGetInfoKHR
- Extending VkVideoProfileInfoKHR, VkQueryPoolCreateInfo:
  - VkVideoEncodeH264ProfileInfoKHR
- Extending VkVideoReferenceSlotInfoKHR:
  - VkVideoEncodeH264DpbSlotInfoKHR
- Extending VkVideoSessionCreateInfoKHR:
  - VkVideoEncodeH264SessionCreateInfoKHR
- Extending VkVideoSessionParametersCreateInfoKHR:
  - VkVideoEncodeH264SessionParametersCreateInfoKHR
- Extending VkVideoSessionParametersUpdateInfoKHR:
  - VkVideoEncodeH264SessionParametersAddInfoKHR

New Enums

- VkVideoEncodeH264CapabilityFlagBitsKHR
- VkVideoEncodeH264RateControlFlagBitsKHR
- VkVideoEncodeH264StdFlagBitsKHR

New Bitmasks

- VkVideoEncodeH264CapabilityFlagsKHR
- VkVideoEncodeH264RateControlFlagsKHR
- VkVideoEncodeH264StdFlagsKHR

New Enum Constants

- VK_KHR_VIDEO_ENCODE_H264_EXTENSION_NAME
- VK_KHR_VIDEO_ENCODE_H264_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_CAPABILITIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_DPB_SLOT_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_GOP_REMAINING_FRAME_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_NALU_SLICE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_PICTURE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_PROFILE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_QUALITY_LEVEL_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_RATE_CONTROL_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_RATE_CONTROL_LAYER_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_ADD_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_FEEDBACK_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_SESSION_PARAMETERS_GET_INFO_KHR
- Extending VkVideoCodecOperationFlagBitsKHR:
  - VK_VIDEO_CODEC_OPERATION_ENCODE_H264_BIT_KHR

Version History

- Revision 0, 2018-7-23 (Ahmed Abdelkhalek)
  - Initial draft
- Revision 0.5, 2020-02-13 (Tony Zlatinski)
  - General Spec cleanup
  - Added DPB structures
  - Change the VCL frame encode structure
  - Added a common Non-VCL Picture Paramarameters structure
- Revision 1, 2021-03-29 (Tony Zlatinski)

    - Spec and API updates
- Revision 2, August 1 2021 (Srinath Kumarapuram)
    - Rename `VkVideoEncodeH264CapabilitiesFlagsEXT` to `VkVideoEncodeH264CapabilityFlagsEXT` and `VkVideoEncodeH264CapabilitiesFlagsEXT` to `VkVideoEncodeH264CapabilityFlagsEXT`, following Vulkan naming conventions.
- Revision 3, 2021-12-08 (Ahmed Abdelkhalek)
    - Rate control updates
- Revision 4, 2022-02-04 (Ahmed Abdelkhalek)
    - Align VkVideoEncodeH264VclFrameInfoEXT structure to similar one in VK_EXT_video_encode_h265 extension
- Revision 5, 2022-02-10 (Ahmed Abdelkhalek)
    - Updates to encode capability interface
- Revision 6, 2022-03-16 (Ahmed Abdelkhalek)
    - Relocate Std header version reporting/requesting from this extension to VK_KHR_video_queue extension.
    - Remove redundant maxPictureSizeInMbs from VkVideoEncodeH264SessionCreateInfoEXT.
    - Remove the now empty VkVideoEncodeH264SessionCreateInfoEXT.
- Revision 7, 2022-04-06 (Ahmed Abdelkhalek)
    - Add capability flag to report support to use B frame in L1 reference list.
    - Add capability flag to report support for disabling SPS direct_8x8_inference_flag.
- Revision 8, 2022-07-18 (Daniel Rakos)
    - Replace `VkVideoEncodeH264RateControlStructureFlagBitsEXT` bit enum with `VkVideoEncodeH264RateControlStructureEXT` enum
    - Rename `VkVideoEncodeH264ProfileEXT` to `VkVideoEncodeH264ProfileInfoEXT`
    - Rename `VkVideoEncodeH264ReferenceListsEXT` to `VkVideoEncodeH264ReferenceListsInfoEXT`
    - Rename `VkVideoEncodeH264EmitPictureParametersEXT` to `VkVideoEncodeH264EmitPictureParametersInfoEXT`
    - Rename `VkVideoEncodeH264NaluSliceEXT` to `VkVideoEncodeH264NaluSliceInfoEXT`
- Revision 9, 2022-09-18 (Daniel Rakos)
    - Rename `spsStdCount`, `pSpsStd`, `ppsStdCount`, and `pPpsStd` to `stdSPSCount`, `pStdSPSs`, `stdPPSCount`, and `pStdPPSs`, respectively, in `VkVideoEncodeH264SessionParametersAddInfoEXT`
    - Rename `maxSpsStdCount` and `maxPpsStdCount` to `maxStdSPSCount` and `maxStdPPSCount`, respectively, in `VkVideoEncodeH264SessionParametersCreateInfoEXT`
- Revision 10, 2023-03-06 (Daniel Rakos)
    - Removed `VkVideoEncodeH264EmitPictureParametersInfoEXT`
    - Changed member types in `VkVideoEncodeH264CapabilitiesEXT` and `VkVideoEncodeH264ReferenceListsInfoEXT` from `uint8_t` to `uint32_t`

- Changed the type of `VkVideoEncodeH264RateControlInfoEXT::temporalLayerCount` and `VkVideoEncodeH264RateControlLayerInfoEXT::temporalLayerId` from `uint8_t` to `uint32_t`
- Removed `VkVideoEncodeH264InputModeFlagsEXT` and `VkVideoEncodeH264OutputModeFlagsEXT` as we only support frame-in-frame-out mode for now
- Rename `pCurrentPictureInfo` in `VkVideoEncodeH264VclFrameInfoEXT` to `pStdPictureInfo`
- Rename `pSliceHeaderStd` in `VkVideoEncodeH264NaluSliceInfoEXT` to `pStdSliceHeader`
- Rename `pReferenceFinalLists` in `VkVideoEncodeH264VclFrameInfoEXT` and `VkVideoEncodeH264NaluSliceInfoEXT` to `pStdReferenceFinalLists`
- Removed the `slotIndex` member of `VkVideoEncodeH264DpbSlotInfoEXT` and changed it to be chained to `VkVideoReferenceSlotInfoKHR`
- Replaced `VkVideoEncodeH264ReferenceListsInfoEXT` with the new Video Std header structure `StdVideoEncodeH264ReferenceLists` that also includes data previously part of the now removed `StdVideoEncodeH264RefMemMgmtCtrlOperations` structure
- Added new capability flag `VK_VIDEO_ENCODE_H264_CAPABILITY_DIFFERENT_REFERENCE_FINAL_LISTS_BIT_EXT`

- Revision 11, 2023-05-22 (Daniel Rakos)
  - Renamed `VkVideoEncodeH264VclFrameInfoEXT` to `VkVideoEncodeH264PictureInfoEXT`
  - Added `VkVideoEncodeH264PictureInfoEXT::generatePrefixNalu` and `VK_VIDEO_ENCODE_H264_CAPABILITY_GENERATE_PREFIX_NALU_BIT_EXT` to enable the generation of H.264 prefix NALUs when supported by the implementation
  - Removed `VkVideoEncodeH264RateControlLayerInfoEXT::temporalLayerId`
  - Added `expectDyadicTemporalLayerPattern` capability
  - Added the `VkVideoEncodeH264SessionParametersGetInfoEXT` structure to identify the H.264 parameter sets to retrieve encoded parameter data for, and the `VkVideoEncodeH264SessionParametersFeedbackInfoEXT` structure to retrieve H.264 parameter set override information when using the new `vkGetEncodedVideoSessionParametersKHR` command
  - Added `VkVideoEncodeH264NaluSliceInfoEXT::constantQp` to specify per-slice constant QP when rate control mode is `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR`
  - Added `VkVideoEncodeH264QualityLevelPropertiesEXT` for retrieving H.264 specific quality level recommendations
  - Replaced `VkVideoEncodeH264RateControlStructureEXT` enum with the flags type `VkVideoEncodeH264RateControlFlagsEXT` and bits defined in `VkVideoEncodeH264RateControlFlagBitsEXT` and added HRD compliance flag
  - Removed `useInitialRcQp` and `initialRcQp` members of `VkVideoEncodeH264RateControlLayerInfoEXT`
  - Added `prefersGopRemainingFrames` and `requiresGopRemainingFrames`, and the new `VkVideoEncodeH264GopRemainingFrameInfoEXT` structure to allow specifying remaining frames of each type in the rate control GOP
  - Added `maxTemporalLayers`, `maxQp`, and `minQp` capabilities

- Added `maxLevelIdc` capability and new `VkVideoEncodeH264SessionCreateInfoEXT` structure to specify upper bounds on the H.264 level of the produced video bitstream
- Moved capability flags specific to codec syntax restrictions from `VkVideoEncodeH264CapabilityFlagsEXT` to the new `VkVideoEncodeH264StdFlagsEXT` which is now included as a separate `stdSyntaxFlags` member in `VkVideoEncodeH264CapabilitiesEXT`
- Removed codec syntax override values from `VkVideoEncodeH264CapabilitiesEXT`
- Removed `VkVideoEncodeH264NaluSliceInfoEXT::mbCount` and `VK_VIDEO_ENCODE_H264_CAPABILITY_SLICE_MB_COUNT_BIT_EXT`
- Replaced `VK_VIDEO_ENCODE_H264_CAPABILITY_MULTIPLE_SLICES_PER_FRAME_BIT_EXT` with the new `maxSliceCount` capability
- Removed capability flag `VK_VIDEO_ENCODE_H264_CAPABILITY_DIFFERENT_REFERENCE_FINAL_LISTS_BIT_EXT` and removed `pStdReferenceFinalLists` members from the `VkVideoEncodeH264PictureInfoEXT` and `VkVideoEncodeH264NaluSliceInfoEXT` structures as reference lists info is now included in `pStdPictureInfo`
- Added capability flag `VK_VIDEO_ENCODE_H264_CAPABILITY_B_FRAME_IN_L0_LIST_BIT_EXT`

- Revision 12, 2023-07-19 (Daniel Rakos)
  - Added video std capability flags `VK_VIDEO_ENCODE_H264_STD_SLICE_QP_DELTA_BIT_EXT` and `VK_VIDEO_ENCODE_H264_STD_DIFFERENT_SLICE_QP_DELTA_BIT_EXT`
  - Fixed optionality of the array members of `VkVideoEncodeH264SessionParametersAddInfoEXT`
  - Fixed optionality of `VkVideoEncodeH264RateControlInfoEXT::flags`
- Revision 13, 2023-09-04 (Daniel Rakos)
  - Change extension from `EXT` to `KHR`
  - Extension is no longer provisional
- Revision 14, 2023-12-05 (Daniel Rakos)
  - Condition reference picture setup based on the value of `StdVideoEncodeH264PictureInfo::flags.is_reference`

VK_KHR_video_encode_h265

Name String

`VK_KHR_video_encode_h265`

Extension Type

Device extension

Registered Extension Number

40

Revision

14

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_encode_queue

Contact

- Ahmed Abdelkhalek [GitHub]aabdelkh

Extension Proposal

VK_KHR_video_encode_h265

Other Extension Metadata

Last Modified Date

2023-12-05

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- George Hao, AMD
- Jake Beju, AMD
- Chunbo Chen, Intel
- Ping Liu, Intel
- Srinath Kumarapuram, NVIDIA
- Tony Zlatinski, NVIDIA
- Ravi Chaudhary, NVIDIA
- Daniel Rakos, RasterGrid
- Aidan Fabius, Core Avionics & Industrial Inc.
- Lynne Iribarren, Independent

Description

This extension builds upon the `VK_KHR_video_encode_queue` extension by adding support for encoding elementary video stream sequences compliant with the H.265/HEVC video compression standard.

NOTE	This extension was promoted to `KHR` from the provisional extension `VK_EXT_video_encode_h265`.

New Structures

- VkVideoEncodeH265FrameSizeKHR

- VkVideoEncodeH265NaluSliceSegmentInfoKHR
- VkVideoEncodeH265QpKHR
- Extending VkVideoBeginCodingInfoKHR:
  - VkVideoEncodeH265GopRemainingFrameInfoKHR
- Extending VkVideoCapabilitiesKHR:
  - VkVideoEncodeH265CapabilitiesKHR
- Extending VkVideoCodingControlInfoKHR, VkVideoBeginCodingInfoKHR:
  - VkVideoEncodeH265RateControlInfoKHR
- Extending VkVideoEncodeInfoKHR:
  - VkVideoEncodeH265PictureInfoKHR
- Extending VkVideoEncodeQualityLevelPropertiesKHR:
  - VkVideoEncodeH265QualityLevelPropertiesKHR
- Extending VkVideoEncodeRateControlLayerInfoKHR:
  - VkVideoEncodeH265RateControlLayerInfoKHR
- Extending VkVideoEncodeSessionParametersFeedbackInfoKHR:
  - VkVideoEncodeH265SessionParametersFeedbackInfoKHR
- Extending VkVideoEncodeSessionParametersGetInfoKHR:
  - VkVideoEncodeH265SessionParametersGetInfoKHR
- Extending VkVideoProfileInfoKHR, VkQueryPoolCreateInfo:
  - VkVideoEncodeH265ProfileInfoKHR
- Extending VkVideoReferenceSlotInfoKHR:
  - VkVideoEncodeH265DpbSlotInfoKHR
- Extending VkVideoSessionCreateInfoKHR:
  - VkVideoEncodeH265SessionCreateInfoKHR
- Extending VkVideoSessionParametersCreateInfoKHR:
  - VkVideoEncodeH265SessionParametersCreateInfoKHR
- Extending VkVideoSessionParametersUpdateInfoKHR:
  - VkVideoEncodeH265SessionParametersAddInfoKHR

New Enums

- VkVideoEncodeH265CapabilityFlagBitsKHR
- VkVideoEncodeH265CtbSizeFlagBitsKHR
- VkVideoEncodeH265RateControlFlagBitsKHR
- VkVideoEncodeH265StdFlagBitsKHR
- VkVideoEncodeH265TransformBlockSizeFlagBitsKHR

New Bitmasks

- VkVideoEncodeH265CapabilityFlagsKHR
- VkVideoEncodeH265CtbSizeFlagsKHR
- VkVideoEncodeH265RateControlFlagsKHR
- VkVideoEncodeH265StdFlagsKHR
- VkVideoEncodeH265TransformBlockSizeFlagsKHR

New Enum Constants

- VK_KHR_VIDEO_ENCODE_H265_EXTENSION_NAME
- VK_KHR_VIDEO_ENCODE_H265_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_CAPABILITIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_DPB_SLOT_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_GOP_REMAINING_FRAME_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_NALU_SLICE_SEGMENT_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_PICTURE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_PROFILE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_QUALITY_LEVEL_PROPERTIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_RATE_CONTROL_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_RATE_CONTROL_LAYER_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_ADD_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_FEEDBACK_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_SESSION_PARAMETERS_GET_INFO_KHR
- Extending VkVideoCodecOperationFlagBitsKHR:
  - VK_VIDEO_CODEC_OPERATION_ENCODE_H265_BIT_KHR

Version History

- Revision 0, 2019-11-14 (Ahmed Abdelkhalek)
  - Initial draft
- Revision 0.5, 2020-02-13 (Tony Zlatinski)
  - General Spec cleanup
  - Added DPB structures
  - Change the VCL frame encode structure

    - Added a common Non-VCL Picture Paramarameters structure
- Revision 2, Oct 10 2021 (Srinath Kumarapuram)
    - Vulkan Video Encode h.265 update and spec edits
- Revision 3, 2021-12-08 (Ahmed Abdelkhalek)
    - Rate control updates
- Revision 4, 2022-01-11 (Ahmed Abdelkhalek)
    - Replace occurrences of “slice” by “slice segment” and rename structures/enums to reflect this.
- Revision 5, 2022-02-10 (Ahmed Abdelkhalek)
    - Updates to encode capability interface
- Revision 6, 2022-03-16 (Ahmed Abdelkhalek)
    - Relocate Std header version reporting/requesting from this extension to VK_KHR_video_queue extension.
    - Remove the now empty VkVideoEncodeH265SessionCreateInfoEXT.
- Revision 7, 2022-03-24 (Ahmed Abdelkhalek)
    - Add capability flags to report support to disable transform skip and support to use B frame in L1 reference list.
- Revision 8, 2022-07-18 (Daniel Rakos)
    - Replace `VkVideoEncodeH265RateControlStructureFlagBitsEXT` bit enum with `VkVideoEncodeH265RateControlStructureEXT` enum
    - Rename `VkVideoEncodeH265ProfileEXT` to `VkVideoEncodeH265ProfileInfoEXT`
    - Rename `VkVideoEncodeH265ReferenceListsEXT` to `VkVideoEncodeH265ReferenceListsInfoEXT`
    - Rename `VkVideoEncodeH265EmitPictureParametersEXT` to `VkVideoEncodeH265EmitPictureParametersInfoEXT`
    - Rename `VkVideoEncodeH265NaluSliceSegmentEXT` to `VkVideoEncodeH265NaluSliceSegmentInfoEXT`
- Revision 9, 2022-09-18 (Daniel Rakos)
    - Rename `vpsStdCount`, `pVpsStd`, `spsStdCount`, `pSpsStd`, `ppsStdCount`, and `pPpsStd` to `stdVPSCount`, `pStdVPSs`, `stdSPSCount`, `pStdSPSs`, `stdPPSCount`, and `pStdPPSs`, respectively, in `VkVideoEncodeH265SessionParametersAddInfoEXT`
    - Rename `maxVpsStdCount`, `maxSpsStdCount`, and `maxPpsStdCount` to `maxStdVPSCount`, `maxStdSPSCount` and `maxStdPPSCount`, respectively, in `VkVideoEncodeH265SessionParametersCreateInfoEXT`
- Revision 10, 2023-03-06 (Daniel Rakos)
    - Removed `VkVideoEncodeH265EmitPictureParametersInfoEXT`
    - Changed member types in `VkVideoEncodeH265CapabilitiesEXT` and `VkVideoEncodeH265ReferenceListsInfoEXT` from `uint8_t` to `uint32_t`
    - Changed the type of `VkVideoEncodeH265RateControlInfoEXT::subLayerCount` and `VkVideoEncodeH265RateControlLayerInfoEXT::temporalId` from `uint8_t` to `uint32_t`

- Removed VkVideoEncodeH265InputModeFlagsEXT and VkVideoEncodeH265OutputModeFlagsEXT as we only support frame-in-frame-out mode for now
- Rename pCurrentPictureInfo in VkVideoEncodeH265VclFrameInfoEXT to pStdPictureInfo
- Rename pSliceSegmentHeaderStd in VkVideoEncodeH265NaluSliceSegmentInfoEXT to pStdSliceSegmentHeader
- Rename pReferenceFinalLists in VkVideoEncodeH265VclFrameInfoEXT and VkVideoEncodeH265NaluSliceSegmentInfoEXT to pStdReferenceFinalLists
- Removed the slotIndex member of VkVideoEncodeH265DpbSlotInfoEXT and changed it to be chained to VkVideoReferenceSlotInfoKHR
- Replaced VkVideoEncodeH265ReferenceListsInfoEXT with the new Video Std header structure StdVideoEncodeH265ReferenceLists
- Added new capability flag VK_VIDEO_ENCODE_H265_CAPABILITY_DIFFERENT_REFERENCE_FINAL_LISTS_BIT_EXT

- Revision 11, 2023-05-26 (Daniel Rakos)
  - Renamed VkVideoEncodeH265VclFrameInfoEXT to VkVideoEncodeH265PictureInfoEXT
  - Removed VkVideoEncodeH265RateControlLayerInfoEXT::temporalId
  - Added expectDyadicTemporalSubLayerPattern capability
  - Added the VkVideoEncodeH265SessionParametersGetInfoEXT structure to identify the H.265 parameter sets to retrieve encoded parameter data for, and the VkVideoEncodeH265SessionParametersFeedbackInfoEXT structure to retrieve H.265 parameter set override information when using the new vkGetEncodedVideoSessionParametersKHR command
  - Added VkVideoEncodeH265NaluSliceSegmentInfoEXT::constantQp to specify per-slice segment constant QP when rate control mode is VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DISABLED_BIT_KHR
  - Added VkVideoEncodeH265QualityLevelPropertiesEXT for retrieving H.265 specific quality level recommendations
  - Replaced VkVideoEncodeH265RateControlStructureEXT enum with the flags type VkVideoEncodeH265RateControlFlagsEXT and bits defined in VkVideoEncodeH265RateControlFlagBitsEXT and added HRD compliance flag
  - Removed useInitialRcQp and initialRcQp members of VkVideoEncodeH265RateControlLayerInfoEXT
  - Added prefersGopRemainingFrames and requiresGopRemainingFrames, and the new VkVideoEncodeH265GopRemainingFrameInfoEXT structure to allow specifying remaining frames of each type in the rate control GOP
  - Renamed maxSubLayersCount capability to maxSubLayerCount
  - Added maxQp, and minQp capabilities
  - Added maxLevelIdc capability and new VkVideoEncodeH265SessionCreateInfoEXT structure to specify upper bounds on the H.265 level of the produced video bitstream
  - Moved capability flags specific to codec syntax restrictions from VkVideoEncodeH265CapabilityFlagsEXT to the new VkVideoEncodeH265StdFlagsEXT which is now

included as a separate `stdSyntaxFlags` member in `VkVideoEncodeH265CapabilitiesEXT`

- Added `std` prefix to codec syntax capabilities in `VkVideoEncodeH265CapabilitiesEXT`
- Removed `VkVideoEncodeH265NaluSliceSegmentInfoEXT::ctbCount` and `VK_VIDEO_ENCODE_H265_CAPABILITY_SLICE_SEGMENT_CTB_COUNT_BIT_EXT`
- Replaced `VK_VIDEO_ENCODE_H265_CAPABILITY_MULTIPLE_SLICE_SEGMENTS_PER_FRAME_BIT_EXT` with the new `maxSliceSegmentCount` capability
- Added `maxTiles` capability
- Removed codec syntax min/max capabilities from `VkVideoEncodeH265CapabilitiesEXT`
- Removed capability flag `VK_VIDEO_ENCODE_H265_CAPABILITY_DIFFERENT_REFERENCE_FINAL_LISTS_BIT_EXT` and removed `pStdReferenceFinalLists` members from the `VkVideoEncodeH265PictureInfoEXT` and `VkVideoEncodeH265NaluSliceSegmentInfoEXT` structures as reference lists info is now included in `pStdPictureInfo`
- Added capability flag `VK_VIDEO_ENCODE_H265_CAPABILITY_B_FRAME_IN_L0_LIST_BIT_EXT`

- Revision 12, 2023-07-19 (Daniel Rakos)
  - Added video std capability flags `VK_VIDEO_ENCODE_H265_STD_SLICE_QP_DELTA_BIT_EXT` and `VK_VIDEO_ENCODE_H265_STD_DIFFERENT_SLICE_QP_DELTA_BIT_EXT`
  - Fixed optionality of the array members of `VkVideoEncodeH265SessionParametersAddInfoEXT`
  - Fixed optionality of `VkVideoEncodeH265RateControlInfoEXT::flags`
- Revision 13, 2023-09-04 (Daniel Rakos)
  - Change extension from `EXT` to `KHR`
  - Extension is no longer provisional
- Revision 14, 2023-12-05 (Daniel Rakos)
  - Condition reference picture setup based on the value of `StdVideoEncodeH265PictureInfo::flags.is_reference`

VK_KHR_video_encode_quantization_map

Name String

`VK_KHR_video_encode_quantization_map`

Extension Type

Device extension

Registered Extension Number

554

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_encode_queue
and
VK_KHR_format_feature_flags2

API Interactions

- Interacts with VK_KHR_video_encode_av1
- Interacts with VK_KHR_video_encode_h264
- Interacts with VK_KHR_video_encode_h265

Contact

- Ahmed Abdelkhalek [GitHub]aabdelkh

Extension Proposal

VK_KHR_video_encode_quantization_map

Other Extension Metadata

Last Modified Date

2024-09-23

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- Benjamin Cheng, AMD
- Srinath Kumarapuram, NVIDIA
- Tony Zlatinski, NVIDIA
- Ping Liu, Intel
- Daniel Rakos, RasterGrid
- Lynne Iribarren, Independent

Description

This extension builds upon the `VK_KHR_video_encode_queue` extension by enabling fine grained control of codec-specific quantization parameters in video encode operations.

More specifically, it adds support for quantization maps:

- Quantization delta maps to directly control the relative value of quantization parameter values on a per-block basis for all rate control modes (including when rate control is disabled).
- Emphasis maps to indirectly control the relative quantization parameter values on a per-block basis when rate control is not disabled and the rate control mode is not configured to the implementation-defined default mode.

This extension is to be used in conjunction with other codec specific video encode extensions that specify the codec specific quantization parameters these maps control.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceVideoEncodeQuantizationMapFeaturesKHR
- Extending VkVideoCapabilitiesKHR:
  - VkVideoEncodeQuantizationMapCapabilitiesKHR
- Extending VkVideoEncodeInfoKHR:
  - VkVideoEncodeQuantizationMapInfoKHR
- Extending VkVideoFormatPropertiesKHR:
  - VkVideoFormatQuantizationMapPropertiesKHR
- Extending VkVideoSessionParametersCreateInfoKHR:
  - VkVideoEncodeQuantizationMapSessionParametersCreateInfoKHR

If VK_KHR_video_encode_av1 is supported:

- Extending VkVideoCapabilitiesKHR:
  - VkVideoEncodeAV1QuantizationMapCapabilitiesKHR
- Extending VkVideoFormatPropertiesKHR:
  - VkVideoFormatAV1QuantizationMapPropertiesKHR

If VK_KHR_video_encode_h264 is supported:

- Extending VkVideoCapabilitiesKHR:
  - VkVideoEncodeH264QuantizationMapCapabilitiesKHR

If VK_KHR_video_encode_h265 is supported:

- Extending VkVideoCapabilitiesKHR:
  - VkVideoEncodeH265QuantizationMapCapabilitiesKHR
- Extending VkVideoFormatPropertiesKHR:
  - VkVideoFormatH265QuantizationMapPropertiesKHR

New Enums

- VkVideoEncodeFlagBitsKHR
- VkVideoSessionParametersCreateFlagBitsKHR

New Enum Constants

- `VK_KHR_VIDEO_ENCODE_QUANTIZATION_MAP_EXTENSION_NAME`

- VK_KHR_VIDEO_ENCODE_QUANTIZATION_MAP_SPEC_VERSION
- Extending VkFormatFeatureFlagBits2:
  - VK_FORMAT_FEATURE_2_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR
  - VK_FORMAT_FEATURE_2_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- Extending VkImageLayout:
  - VK_IMAGE_LAYOUT_VIDEO_ENCODE_QUANTIZATION_MAP_KHR
- Extending VkImageUsageFlagBits:
  - VK_IMAGE_USAGE_VIDEO_ENCODE_EMPHASIS_MAP_BIT_KHR
  - VK_IMAGE_USAGE_VIDEO_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_QUANTIZATION_MAP_FEATURES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_CAPABILITIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUANTIZATION_MAP_SESSION_PARAMETERS_CREATE_INFO_KHR
  - VK_STRUCTURE_TYPE_VIDEO_FORMAT_QUANTIZATION_MAP_PROPERTIES_KHR
- Extending VkVideoEncodeCapabilityFlagBitsKHR:
  - VK_VIDEO_ENCODE_CAPABILITY_EMPHASIS_MAP_BIT_KHR
  - VK_VIDEO_ENCODE_CAPABILITY_QUANTIZATION_DELTA_MAP_BIT_KHR
- Extending VkVideoEncodeFlagBitsKHR:
  - VK_VIDEO_ENCODE_WITH_EMPHASIS_MAP_BIT_KHR
  - VK_VIDEO_ENCODE_WITH_QUANTIZATION_DELTA_MAP_BIT_KHR
- Extending VkVideoSessionCreateFlagBitsKHR:
  - VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_EMPHASIS_MAP_BIT_KHR
  - VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_QUANTIZATION_DELTA_MAP_BIT_KHR
- Extending VkVideoSessionParametersCreateFlagBitsKHR:
  - VK_VIDEO_SESSION_PARAMETERS_CREATE_QUANTIZATION_MAP_COMPATIBLE_BIT_KHR

If VK_KHR_video_encode_av1 is supported:

- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_AV1_QUANTIZATION_MAP_CAPABILITIES_KHR
  - VK_STRUCTURE_TYPE_VIDEO_FORMAT_AV1_QUANTIZATION_MAP_PROPERTIES_KHR

If VK_KHR_video_encode_h264 is supported:

- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_VIDEO_ENCODE_H264_QUANTIZATION_MAP_CAPABILITIES_KHR

- Extending VkVideoEncodeH264CapabilityFlagBitsKHR:
  - `VK_VIDEO_ENCODE_H264_CAPABILITY_MB_QP_DIFF_WRAPAROUND_BIT_KHR`

If VK_KHR_video_encode_h265 is supported:

- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_VIDEO_ENCODE_H265_QUANTIZATION_MAP_CAPABILITIES_KHR`
  - `VK_STRUCTURE_TYPE_VIDEO_FORMAT_H265_QUANTIZATION_MAP_PROPERTIES_KHR`
- Extending VkVideoEncodeH265CapabilityFlagBitsKHR:
  - `VK_VIDEO_ENCODE_H265_CAPABILITY_CU_QP_DIFF_WRAPAROUND_BIT_KHR`

Version History

- Revision 1, 2024-08-21 (Daniel Rakos)
  - Internal revisions
- Revision 2, 2024-09-23 (Daniel Rakos)
  - Added interactions with VK_KHR_video_encode_av1

VK_KHR_video_encode_queue

Name String

`VK_KHR_video_encode_queue`

Extension Type

Device extension

Registered Extension Number

300

Revision

12

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_queue
and
VK_KHR_synchronization2
or
Vulkan Version 1.3

API Interactions

- Interacts with VK_VERSION_1_3
- Interacts with VK_KHR_format_feature_flags2

Contact

- Ahmed Abdelkhalek [GitHub]aabdelkh

Extension Proposal

VK_KHR_video_encode_queue

Other Extension Metadata

Last Modified Date

2023-12-05

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- Damien Kessler, NVIDIA
- George Hao, AMD
- Jake Beju, AMD
- Peter Fang, AMD
- Piers Daniell, NVIDIA
- Srinath Kumarapuram, NVIDIA
- Thomas J. Meier, NVIDIA
- Tony Zlatinski, NVIDIA
- Ravi Chaudhary, NVIDIA
- Yang Liu, AMD
- Daniel Rakos, RasterGrid
- Ping Liu, Intel
- Aidan Fabius, Core Avionics & Industrial Inc.
- Lynne Iribarren, Independent

Description

This extension builds upon the `VK_KHR_video_queue` extension by adding common APIs specific to video encoding and thus enabling implementations to expose queue families supporting video encode operations.

More specifically, it adds video encode specific capabilities and a new command buffer command that allows recording video encode operations against a video session.

This extension is to be used in conjunction with other codec specific video encode extensions that enable encoding video sequences of specific video compression standards.

New Commands

- vkCmdEncodeVideoKHR
- vkGetEncodedVideoSessionParametersKHR
- vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR

New Structures

- VkPhysicalDeviceVideoEncodeQualityLevelInfoKHR
- VkVideoEncodeInfoKHR
- VkVideoEncodeQualityLevelPropertiesKHR
- VkVideoEncodeRateControlLayerInfoKHR
- VkVideoEncodeSessionParametersFeedbackInfoKHR
- VkVideoEncodeSessionParametersGetInfoKHR
- Extending VkQueryPoolCreateInfo:
  - VkQueryPoolVideoEncodeFeedbackCreateInfoKHR
- Extending VkVideoCapabilitiesKHR:
  - VkVideoEncodeCapabilitiesKHR
- Extending VkVideoCodingControlInfoKHR, VkVideoBeginCodingInfoKHR:
  - VkVideoEncodeRateControlInfoKHR
- Extending VkVideoCodingControlInfoKHR, VkVideoSessionParametersCreateInfoKHR:
  - VkVideoEncodeQualityLevelInfoKHR
- Extending VkVideoProfileInfoKHR, VkQueryPoolCreateInfo:
  - VkVideoEncodeUsageInfoKHR

New Enums

- VkVideoEncodeCapabilityFlagBitsKHR
- VkVideoEncodeContentFlagBitsKHR
- VkVideoEncodeFeedbackFlagBitsKHR
- VkVideoEncodeRateControlModeFlagBitsKHR
- VkVideoEncodeTuningModeKHR
- VkVideoEncodeUsageFlagBitsKHR

New Bitmasks

- VkVideoEncodeCapabilityFlagsKHR
- VkVideoEncodeContentFlagsKHR
- VkVideoEncodeFeedbackFlagsKHR
- VkVideoEncodeFlagsKHR

- VkVideoEncodeRateControlFlagsKHR
- VkVideoEncodeRateControlModeFlagsKHR
- VkVideoEncodeUsageFlagsKHR

New Enum Constants

- VK_KHR_VIDEO_ENCODE_QUEUE_EXTENSION_NAME
- VK_KHR_VIDEO_ENCODE_QUEUE_SPEC_VERSION
- Extending VkAccessFlagBits2:
  - VK_ACCESS_2_VIDEO_ENCODE_READ_BIT_KHR
  - VK_ACCESS_2_VIDEO_ENCODE_WRITE_BIT_KHR
- Extending VkBufferUsageFlagBits:
  - VK_BUFFER_USAGE_VIDEO_ENCODE_DST_BIT_KHR
  - VK_BUFFER_USAGE_VIDEO_ENCODE_SRC_BIT_KHR
- Extending VkFormatFeatureFlagBits:
  - VK_FORMAT_FEATURE_VIDEO_ENCODE_DPB_BIT_KHR
  - VK_FORMAT_FEATURE_VIDEO_ENCODE_INPUT_BIT_KHR
- Extending VkImageLayout:
  - VK_IMAGE_LAYOUT_VIDEO_ENCODE_DPB_KHR
  - VK_IMAGE_LAYOUT_VIDEO_ENCODE_DST_KHR
  - VK_IMAGE_LAYOUT_VIDEO_ENCODE_SRC_KHR
- Extending VkImageUsageFlagBits:
  - VK_IMAGE_USAGE_VIDEO_ENCODE_DPB_BIT_KHR
  - VK_IMAGE_USAGE_VIDEO_ENCODE_DST_BIT_KHR
  - VK_IMAGE_USAGE_VIDEO_ENCODE_SRC_BIT_KHR
- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_VIDEO_ENCODE_BIT_KHR
- Extending VkQueryResultStatusKHR:
  - VK_QUERY_RESULT_STATUS_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_KHR
- Extending VkQueryType:
  - VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR
- Extending VkQueueFlagBits:
  - VK_QUEUE_VIDEO_ENCODE_BIT_KHR
- Extending VkResult:
  - VK_ERROR_INVALID_VIDEO_STD_PARAMETERS_KHR
- Extending VkStructureType:

    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_ENCODE_QUALITY_LEVEL_INFO_KHR
    - VK_STRUCTURE_TYPE_QUERY_POOL_VIDEO_ENCODE_FEEDBACK_CREATE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_CAPABILITIES_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUALITY_LEVEL_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_QUALITY_LEVEL_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_RATE_CONTROL_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_RATE_CONTROL_LAYER_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_SESSION_PARAMETERS_FEEDBACK_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_SESSION_PARAMETERS_GET_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_ENCODE_USAGE_INFO_KHR
- Extending VkVideoCodingControlFlagBitsKHR:
    - VK_VIDEO_CODING_CONTROL_ENCODE_QUALITY_LEVEL_BIT_KHR
    - VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR
- Extending VkVideoSessionCreateFlagBitsKHR:
    - VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_PARAMETER_OPTIMIZATIONS_BIT_KHR

If VK_KHR_format_feature_flags2 or Vulkan Version 1.3 is supported:

- Extending VkFormatFeatureFlagBits2:
    - VK_FORMAT_FEATURE_2_VIDEO_ENCODE_DPB_BIT_KHR
    - VK_FORMAT_FEATURE_2_VIDEO_ENCODE_INPUT_BIT_KHR

Version History

- Revision 1, 2018-07-23 (Ahmed Abdelkhalek)
    - Initial draft
- Revision 1.1, 10/29/2019 (Tony Zlatinski)
    - Updated the reserved spec tokens and renamed VkVideoEncoderKHR to VkVideoSessionKHR
- Revision 1.6, Jan 08 2020 (Tony Zlatinski)
    - API unify with the video_decode_queue spec
- Revision 2, March 29 2021 (Tony Zlatinski)
    - Spec and API updates.
- Revision 3, 2021-09-30 (Jon Leech)
    - Add interaction with VK_KHR_format_feature_flags2 to vk.xml
- Revision 4, 2022-02-10 (Ahmed Abdelkhalek)
    - Updates to encode capability interface

- Revision 5, 2022-03-31 (Ahmed Abdelkhalek)
  - Remove redundant VkVideoEncodeInfoKHR.codedExtent
- Revision 6, 2022-07-18 (Daniel Rakos)
  - Remove `VkVideoEncodeRateControlFlagBitsKHR` and `VkVideoEncodeFlagBitsKHR` as they contain no defined flags for now
  - Add `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_BIT_KHR` and `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_LAYER_BIT_KHR` to indicate rate control and rate control layer change requests, respectively, in video coding control operations
- Revision 7, 2022-08-12 (Daniel Rakos)
  - Add VkVideoEncodeUsageInfoKHR structure and related flags
- Revision 8, 2023-03-06 (Daniel Rakos)
  - Replace `VK_QUERY_TYPE_VIDEO_ENCODE_BITSTREAM_BUFFER_RANGE_KHR` queries with more generic `VK_QUERY_TYPE_VIDEO_ENCODE_FEEDBACK_KHR` queries that can be extended in the future with more feedback values
  - Rename `dstBitstreamBuffer`, `dstBitstreamBufferOffset`, and `dstBitstreamBufferMaxRange` in `VkVideoEncodeInfoKHR` to `dstBuffer`, `dstBufferOffset`, and `dstBufferRange`, respectively, for consistency with the naming convention in the video decode extensions
  - Change the type of `rateControlLayerCount` and `qualityLevelCount` in `VkVideoEncodeCapabilitiesKHR` from `uint8_t` to `uint32_t` and rename them to `maxRateControlLayers` and `maxQualityLevels`, respectively
  - Change the type of `averageBitrate` and `maxBitrate` in `VkVideoEncodeRateControlLayerInfoKHR`` from `uint32_t` to `uint64_t`
  - Fixed the definition of rate control flag bits and added the new `VK_VIDEO_ENCODE_RATE_CONTROL_MODE_DEFAULT_KHR` constant to indicate implementation-specific automatic rate control
  - Change the type of `VkVideoEncodeRateControlInfoKHR::layerCount` from `uint8_t` to `uint32_t`
  - Rename `pLayerConfigs` to `pLayers` in `VkVideoEncodeRateControlInfoKHR`
- Revision 9, 2023-03-28 (Daniel Rakos)
  - Removed `VK_VIDEO_CODING_CONTROL_ENCODE_RATE_CONTROL_LAYER_BIT_KHR` and the ability to change the state of individual rate control layers
  - Added new `VK_VIDEO_ENCODE_FEEDBACK_BITSTREAM_HAS_OVERRIDES_BIT_KHR` flag to video encode feedback queries
  - Added new video session create flag `VK_VIDEO_SESSION_CREATE_ALLOW_ENCODE_PARAMETER_OPTIMIZATIONS_BIT_KHR` to opt-in to video session and encoding parameter optimizations
  - Added the `vkGetEncodedVideoSessionParametersKHR` command to enable retrieving encoded video session parameter data
  - Moved `virtualBufferSizeInMs` and `initialVirtualBufferSizeInMs` from `VkVideoEncodeRateControlLayerInfoKHR` to `VkVideoEncodeRateControlInfoKHR`
  - Added `maxBitrate` capability

    - Renamed `inputImageDataFillAlignment` capability to `encodeInputPictureGranularity` to better reflect its purpose
    - Added new `vkGetPhysicalDeviceVideoEncodeQualityLevelPropertiesKHR` command and related structures to enable querying recommended settings for video encode quality levels
    - Added `VK_VIDEO_CODING_CONTROL_ENCODE_QUALITY_LEVEL_BIT_KHR` flag and `VkVideoEncodeQualityLevelInfoKHR` structure to allow controlling video encode quality level and removed `qualityLevel` from the encode operation parameters
- Revision 10, 2023-07-19 (Daniel Rakos)
    - Added `VK_QUERY_RESULT_STATUS_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_KHR` query result status code and the related capability flag `VK_VIDEO_ENCODE_CAPABILITY_INSUFFICIENT_BITSTREAM_BUFFER_RANGE_DETECTION_BIT_KHR`
- Revision 11, 2023-09-04 (Daniel Rakos)
    - Extension is no longer provisional
- Revision 12, 2023-12-05 (Daniel Rakos)
    - Require the specification of a reconstructed picture in all cases, except when the video session was created with no DPB slots to match shipping implementations
    - Make DPB slot activation behavior codec-specific to continue allowing application control over reference picture setup now that a reconstructed picture is always mandatory

VK_KHR_video_maintenance1

Name String

`VK_KHR_video_maintenance1`

Extension Type

Device extension

Registered Extension Number

516

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_video_queue

Contact

- Daniel Rakos [GitHub]aqnuep

Extension Proposal

VK_KHR_video_maintenance1

Other Extension Metadata

Last Modified Date

2023-07-27

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- Aidan Fabius, Core Avionics & Industrial Inc.
- Ping Liu, Intel
- Lynne Iribarren, Independent
- Srinath Kumarapuram, NVIDIA
- Tony Zlatinski, NVIDIA
- Daniel Rakos, RasterGrid

Description

`VK_KHR_video_maintenance1` adds a collection of minor video coding features, none of which would warrant an entire extension of their own.

The new features are as follows:

- Allow creating buffers that can be used in video coding operations, independent of the used video profile, using the new buffer creation flag `VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR`.
- Allow creating images that can be used as decode output or encode input pictures, independent of the used video profile, using the new image creation flag `VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR`.
- Allow specifying queries used by video coding operations as part of the video coding command parameters, instead of using begin/end query when the video session is created using the new video session creation flag `VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR`.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceVideoMaintenance1FeaturesKHR
- Extending VkVideoDecodeInfoKHR, VkVideoEncodeInfoKHR:
  - VkVideoInlineQueryInfoKHR

New Enum Constants

- `VK_KHR_VIDEO_MAINTENANCE_1_EXTENSION_NAME`
- `VK_KHR_VIDEO_MAINTENANCE_1_SPEC_VERSION`

- Extending VkBufferCreateFlagBits:
  - `VK_BUFFER_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR`
- Extending VkImageCreateFlagBits:
  - `VK_IMAGE_CREATE_VIDEO_PROFILE_INDEPENDENT_BIT_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_MAINTENANCE_1_FEATURES_KHR`
  - `VK_STRUCTURE_TYPE_VIDEO_INLINE_QUERY_INFO_KHR`
- Extending VkVideoSessionCreateFlagBitsKHR:
  - `VK_VIDEO_SESSION_CREATE_INLINE_QUERIES_BIT_KHR`

Version History

- Revision 1, 2023-07-27 (Daniel Rakos)
  - internal revisions

VK_KHR_video_queue

Name String

`VK_KHR_video_queue`

Extension Type

Device extension

Registered Extension Number

24

Revision

8

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.1
and
VK_KHR_synchronization2
or
Vulkan Version 1.3

Contact

- Tony Zlatinski [GitHub]tzlatinski

Extension Proposal

VK_KHR_video_queue

Other Extension Metadata

Last Modified Date

2022-09-29

IP Status

No known IP claims.

Contributors

- Ahmed Abdelkhalek, AMD
- George Hao, AMD
- Jake Beju, AMD
- Piers Daniell, NVIDIA
- Srinath Kumarapuram, NVIDIA
- Tobias Hector, AMD
- Tony Zlatinski, NVIDIA
- Daniel Rakos, RasterGrid

Description

This extension provides common APIs to enable exposing queue families with support for video codec operations by introducing the following new object types and related functionalities:

- Video session objects that represent and maintain the state needed to perform video codec operations.
- Video session parameters objects that act as a container for codec specific parameters.

In addition, it also introduces query commands that allow applications to determine video coding related capabilities, and command buffer commands that enable recording video coding operations against a video session.

This extension is to be used in conjunction with other extensions that enable specific video coding operations.

New Object Types

- VkVideoSessionKHR
- VkVideoSessionParametersKHR

New Commands

- vkBindVideoSessionMemoryKHR
- vkCmdBeginVideoCodingKHR
- vkCmdControlVideoCodingKHR
- vkCmdEndVideoCodingKHR

- vkCreateVideoSessionKHR
- vkCreateVideoSessionParametersKHR
- vkDestroyVideoSessionKHR
- vkDestroyVideoSessionParametersKHR
- vkGetPhysicalDeviceVideoCapabilitiesKHR
- vkGetPhysicalDeviceVideoFormatPropertiesKHR
- vkGetVideoSessionMemoryRequirementsKHR
- vkUpdateVideoSessionParametersKHR

New Structures

- VkBindVideoSessionMemoryInfoKHR
- VkPhysicalDeviceVideoFormatInfoKHR
- VkVideoBeginCodingInfoKHR
- VkVideoCapabilitiesKHR
- VkVideoCodingControlInfoKHR
- VkVideoEndCodingInfoKHR
- VkVideoFormatPropertiesKHR
- VkVideoPictureResourceInfoKHR
- VkVideoReferenceSlotInfoKHR
- VkVideoSessionCreateInfoKHR
- VkVideoSessionMemoryRequirementsKHR
- VkVideoSessionParametersCreateInfoKHR
- VkVideoSessionParametersUpdateInfoKHR
- Extending VkPhysicalDeviceImageFormatInfo2, VkPhysicalDeviceVideoFormatInfoKHR, VkImageCreateInfo, VkBufferCreateInfo:
  - VkVideoProfileListInfoKHR
- Extending VkQueryPoolCreateInfo:
  - VkVideoProfileInfoKHR
- Extending VkQueueFamilyProperties2:
  - VkQueueFamilyQueryResultStatusPropertiesKHR
  - VkQueueFamilyVideoPropertiesKHR

New Enums

- VkQueryResultStatusKHR
- VkVideoCapabilityFlagBitsKHR
- VkVideoChromaSubsamplingFlagBitsKHR

- VkVideoCodecOperationFlagBitsKHR
- VkVideoCodingControlFlagBitsKHR
- VkVideoComponentBitDepthFlagBitsKHR
- VkVideoSessionCreateFlagBitsKHR

New Bitmasks

- VkVideoBeginCodingFlagsKHR
- VkVideoCapabilityFlagsKHR
- VkVideoChromaSubsamplingFlagsKHR
- VkVideoCodecOperationFlagsKHR
- VkVideoCodingControlFlagsKHR
- VkVideoComponentBitDepthFlagsKHR
- VkVideoEndCodingFlagsKHR
- VkVideoSessionCreateFlagsKHR
- VkVideoSessionParametersCreateFlagsKHR

New Enum Constants

- `VK_KHR_VIDEO_QUEUE_EXTENSION_NAME`
- `VK_KHR_VIDEO_QUEUE_SPEC_VERSION`
- Extending VkObjectType:
  - `VK_OBJECT_TYPE_VIDEO_SESSION_KHR`
  - `VK_OBJECT_TYPE_VIDEO_SESSION_PARAMETERS_KHR`
- Extending VkQueryResultFlagBits:
  - `VK_QUERY_RESULT_WITH_STATUS_BIT_KHR`
- Extending VkQueryType:
  - `VK_QUERY_TYPE_RESULT_STATUS_ONLY_KHR`
- Extending VkResult:
  - `VK_ERROR_IMAGE_USAGE_NOT_SUPPORTED_KHR`
  - `VK_ERROR_VIDEO_PICTURE_LAYOUT_NOT_SUPPORTED_KHR`
  - `VK_ERROR_VIDEO_PROFILE_CODEC_NOT_SUPPORTED_KHR`
  - `VK_ERROR_VIDEO_PROFILE_FORMAT_NOT_SUPPORTED_KHR`
  - `VK_ERROR_VIDEO_PROFILE_OPERATION_NOT_SUPPORTED_KHR`
  - `VK_ERROR_VIDEO_STD_VERSION_NOT_SUPPORTED_KHR`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_BIND_VIDEO_SESSION_MEMORY_INFO_KHR`

    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VIDEO_FORMAT_INFO_KHR
    - VK_STRUCTURE_TYPE_QUEUE_FAMILY_QUERY_RESULT_STATUS_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_QUEUE_FAMILY_VIDEO_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_VIDEO_BEGIN_CODING_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_CAPABILITIES_KHR
    - VK_STRUCTURE_TYPE_VIDEO_CODING_CONTROL_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_END_CODING_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_FORMAT_PROPERTIES_KHR
    - VK_STRUCTURE_TYPE_VIDEO_PICTURE_RESOURCE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_PROFILE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_PROFILE_LIST_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_REFERENCE_SLOT_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_SESSION_CREATE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_SESSION_MEMORY_REQUIREMENTS_KHR
    - VK_STRUCTURE_TYPE_VIDEO_SESSION_PARAMETERS_CREATE_INFO_KHR
    - VK_STRUCTURE_TYPE_VIDEO_SESSION_PARAMETERS_UPDATE_INFO_KHR

Version History

- Revision 0.1, 2019-11-21 (Tony Zlatinski)
    - Initial draft
- Revision 0.2, 2019-11-27 (Tony Zlatinski)
    - Make vulkan video core common between decode and encode
- Revision 1, March 29 2021 (Tony Zlatinski)
    - Spec and API updates.
- Revision 2, August 1 2021 (Srinath Kumarapuram)
    - Rename VkVideoCapabilitiesFlagBitsKHR to VkVideoCapabilityFlagBitsKHR (along with the names of enumerants it defines) and VkVideoCapabilitiesFlagsKHR to VkVideoCapabilityFlagsKHR, following Vulkan naming conventions.
- Revision 3, 2022-03-16 (Ahmed Abdelkhalek)
    - Relocate Std header version reporting/requesting from codec-operation specific extensions to this extension.
    - Make Std header versions codec-operation specific instead of only codec-specific.
- Revision 4, 2022-05-30 (Daniel Rakos)
    - Refactor the video format query APIs and related language
    - Extend VkResult with video-specific error codes
- Revision 5, 2022-08-11 (Daniel Rakos)

    - Add VkVideoSessionParametersCreateFlagsKHR
    - Remove VkVideoCodingQualityPresetFlagsKHR
    - Rename VkQueueFamilyQueryResultStatusProperties2KHR to VkQueueFamilyQueryResultStatusPropertiesKHR
    - Rename VkVideoQueueFamilyProperties2KHR to VkQueueFamilyVideoPropertiesKHR
    - Rename VkVideoProfileKHR to VkVideoProfileInfoKHR
    - Rename VkVideoProfilesKHR to VkVideoProfileListInfoKHR
    - Rename VkVideoGetMemoryPropertiesKHR to VkVideoSessionMemoryRequirementsKHR
    - Rename VkVideoBindMemoryKHR to VkBindVideoSessionMemoryInfoKHR
    - Fix pNext constness of VkPhysicalDeviceVideoFormatInfoKHR and VkVideoSessionMemoryRequirementsKHR
    - Fix incorrectly named value enums in bit enum types VkVideoCodecOperationFlagBitsKHR and VkVideoChromaSubsamplingFlagBitsKHR
    - Remove unnecessary default values from VkVideoSessionCreateFlagBitsKHR and VkVideoCodingControlFlagBitsKHR
    - Eliminate nested pointer in VkVideoSessionMemoryRequirementsKHR
    - Rename VkVideoPictureResourceKHR to VkVideoPictureResourceInfoKHR
    - Rename VkVideoReferenceSlotKHR to VkVideoReferenceSlotInfoKHR
- Revision 6, 2022-09-18 (Daniel Rakos)
    - Rename the maxReferencePicturesSlotsCount and maxReferencePicturesActiveCount fields of VkVideoCapabilitiesKHR and VkVideoSessionCreateInfoKHR to maxDpbSlots and maxActiveReferencePictures, respectively, to clarify their meaning
    - Rename capabilityFlags to flags in VkVideoCapabilitiesKHR
    - Rename videoPictureExtentGranularity to pictureAccessGranularity in VkVideoCapabilitiesKHR
    - Rename minExtent and maxExtent to minCodedExtent and maxCodedExtent, respectively, in VkVideoCapabilitiesKHR
    - Rename referencePicturesFormat to referencePictureFormat in VkVideoSessionCreateInfoKHR
- Revision 7, 2022-09-26 (Daniel Rakos)
    - Change type of VkVideoReferenceSlotInfoKHR::slotIndex from int8_t to int32_t
- Revision 8, 2022-09-29 (Daniel Rakos)
    - Extension is no longer provisional

VK_KHR_vulkan_memory_model

Name String

VK_KHR_vulkan_memory_model

Extension Type

Device extension

Registered Extension Number

212

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_vulkan_memory_model

Deprecation State

- *Promoted* to Vulkan 1.2

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2018-12-10

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA
- Alan Baker, Google
- Tobias Hector, AMD
- David Neto, Google
- Robert Simpson, Qualcomm Technologies, Inc.
- Brian Sumner, AMD

Description

The VK_KHR_vulkan_memory_model extension allows use of the features guarded by the `VulkanMemoryModel`, `VulkanMemoryModelDeviceScope`, and `VulkanMemoryModelAvailabilityVisibilityChains` capabilities in shader modules. The Vulkan

Memory Model formally defines how to synchronize memory accesses to the same memory locations performed by multiple shader invocations.

NOTE	Version 3 of the spec added a member (vulkanMemoryModelAvailabilityVisibilityChains) to VkPhysicalDeviceVulkanMemoryModelFeaturesKHR, which is an incompatible change from version 2.

Promotion to Vulkan 1.2

All functionality in this extension is included in core Vulkan 1.2, with the KHR suffix omitted. However, if Vulkan 1.2 is supported and this extension is not, the vulkanMemoryModel capability is optional. The original type, enum, and command names are still available as aliases of the core functionality.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceVulkanMemoryModelFeaturesKHR

New Enum Constants

- VK_KHR_VULKAN_MEMORY_MODEL_EXTENSION_NAME
- VK_KHR_VULKAN_MEMORY_MODEL_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_MEMORY_MODEL_FEATURES_KHR

New SPIR-V Capabilities

- VulkanMemoryModelKHR

Version History

- Revision 1, 2018-06-24 (Jeff Bolz)
  - Initial draft
- Revision 3, 2018-12-10 (Jeff Bolz)
  - Add vulkanMemoryModelAvailabilityVisibilityChains member to the VkPhysicalDeviceVulkanMemoryModelFeaturesKHR structure.

VK_KHR_wayland_surface

Name String

VK_KHR_wayland_surface

Extension Type

Instance extension

Registered Extension Number

7

Revision

6

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- Jesse Hall [GitHub]critsec
- Ian Elliott [GitHub]ianelliottus

Other Extension Metadata

Last Modified Date

2015-11-28

IP Status

No known IP claims.

Contributors

- Patrick Doane, Blizzard
- Faith Ekstrand, Intel
- Ian Elliott, LunarG
- Courtney Goeltzenleuchter, LunarG
- Jesse Hall, Google
- James Jones, NVIDIA
- Antoine Labour, Google
- Jon Leech, Khronos
- David Mao, AMD
- Norbert Nopper, Freescale
- Alon Or-bach, Samsung
- Daniel Rakos, AMD
- Graham Sellers, AMD
- Ray Smith, ARM
- Jeff Vigil, Qualcomm
- Chia-I Wu, LunarG

Description

The VK_KHR_wayland_surface extension is an instance extension. It provides a mechanism to create a VkSurfaceKHR object (defined by the VK_KHR_surface extension) that refers to a Wayland wl_surface, as well as a query to determine support for rendering to a Wayland compositor.

New Commands

- vkCreateWaylandSurfaceKHR
- vkGetPhysicalDeviceWaylandPresentationSupportKHR

New Structures

- VkWaylandSurfaceCreateInfoKHR

New Bitmasks

- VkWaylandSurfaceCreateFlagsKHR

New Enum Constants

- VK_KHR_WAYLAND_SURFACE_EXTENSION_NAME
- VK_KHR_WAYLAND_SURFACE_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_WAYLAND_SURFACE_CREATE_INFO_KHR

Issues

1) Does Wayland need a way to query for compatibility between a particular physical device and a specific Wayland display? This would be a more general query than vkGetPhysicalDeviceSurfaceSupportKHR: if the Wayland-specific query returned VK_TRUE for a (VkPhysicalDevice, struct wl_display*) pair, then the physical device could be assumed to support presentation to any VkSurfaceKHR for surfaces on the display.

RESOLVED: Yes. vkGetPhysicalDeviceWaylandPresentationSupportKHR was added to address this issue.

2) Should we require surfaces created with vkCreateWaylandSurfaceKHR to support the VK_PRESENT_MODE_MAILBOX_KHR present mode?

RESOLVED: Yes. Wayland is an inherently mailbox window system and mailbox support is required for some Wayland compositor interactions to work as expected. While handling these interactions may be possible with VK_PRESENT_MODE_FIFO_KHR, it is much more difficult to do without deadlock and requiring all Wayland applications to be able to support implementations which only support VK_PRESENT_MODE_FIFO_KHR would be an onerous restriction on application developers.

Version History

- Revision 1, 2015-09-23 (Jesse Hall)

    - Initial draft, based on the previous contents of VK_EXT_KHR_swapchain (later renamed VK_EXT_KHR_surface).
- Revision 2, 2015-10-02 (James Jones)
    - Added vkGetPhysicalDeviceWaylandPresentationSupportKHR() to resolve issue #1.
    - Adjusted wording of issue #1 to match the agreed-upon solution.
    - Renamed “window” parameters to “surface” to match Wayland conventions.
- Revision 3, 2015-10-26 (Ian Elliott)
    - Renamed from VK_EXT_KHR_wayland_surface to VK_KHR_wayland_surface.
- Revision 4, 2015-11-03 (Daniel Rakos)
    - Added allocation callbacks to vkCreateWaylandSurfaceKHR.
- Revision 5, 2015-11-28 (Daniel Rakos)
    - Updated the surface create function to take a pCreateInfo structure.
- Revision 6, 2017-02-08 (Faith Ekstrand)
    - Added the requirement that implementations support `VK_PRESENT_MODE_MAILBOX_KHR`.
    - Added wording about interactions between vkQueuePresentKHR and the Wayland requests sent to the compositor.

VK_KHR_win32_keyed_mutex

Name String

`VK_KHR_win32_keyed_mutex`

Extension Type

Device extension

Registered Extension Number

76

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_memory_win32

Contact

- Carsten Rohde [GitHub]crohde

Other Extension Metadata

Last Modified Date

2016-10-21

IP Status

No known IP claims.

Contributors

- James Jones, NVIDIA
- Jeff Juliano, NVIDIA
- Carsten Rohde, NVIDIA

Description

Applications that wish to import Direct3D 11 memory objects into the Vulkan API may wish to use the native keyed mutex mechanism to synchronize access to the memory between Vulkan and Direct3D. This extension provides a way for an application to access the keyed mutex associated with an imported Vulkan memory object when submitting command buffers to a queue.

New Structures

- Extending VkSubmitInfo, VkSubmitInfo2:
  - VkWin32KeyedMutexAcquireReleaseInfoKHR

New Enum Constants

- `VK_KHR_WIN32_KEYED_MUTEX_EXTENSION_NAME`
- `VK_KHR_WIN32_KEYED_MUTEX_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_WIN32_KEYED_MUTEX_ACQUIRE_RELEASE_INFO_KHR`

Version History

- Revision 1, 2016-10-21 (James Jones)
  - Initial revision

VK_KHR_win32_surface

Name String

`VK_KHR_win32_surface`

Extension Type

Instance extension

Registered Extension Number

10

Revision

6

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- Jesse Hall [GitHub]critsec
- Ian Elliott [GitHub]ianelliottus

Other Extension Metadata

Last Modified Date

2017-04-24

IP Status

No known IP claims.

Contributors

- Patrick Doane, Blizzard
- Faith Ekstrand, Intel
- Ian Elliott, LunarG
- Courtney Goeltzenleuchter, LunarG
- Jesse Hall, Google
- James Jones, NVIDIA
- Antoine Labour, Google
- Jon Leech, Khronos
- David Mao, AMD
- Norbert Nopper, Freescale
- Alon Or-bach, Samsung
- Daniel Rakos, AMD
- Graham Sellers, AMD
- Ray Smith, ARM
- Jeff Vigil, Qualcomm
- Chia-I Wu, LunarG

Description

The `VK_KHR_win32_surface` extension is an instance extension. It provides a mechanism to create a

VkSurfaceKHR object (defined by the VK_KHR_surface extension) that refers to a Win32 HWND, as well as a query to determine support for rendering to the windows desktop.

New Commands

- vkCreateWin32SurfaceKHR
- vkGetPhysicalDeviceWin32PresentationSupportKHR

New Structures

- VkWin32SurfaceCreateInfoKHR

New Bitmasks

- VkWin32SurfaceCreateFlagsKHR

New Enum Constants

- VK_KHR_WIN32_SURFACE_EXTENSION_NAME
- VK_KHR_WIN32_SURFACE_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_WIN32_SURFACE_CREATE_INFO_KHR

Issues

1) Does Win32 need a way to query for compatibility between a particular physical device and a specific screen? Compatibility between a physical device and a window generally only depends on what screen the window is on. However, there is not an obvious way to identify a screen without already having a window on the screen.

RESOLVED: No. While it may be useful, there is not a clear way to do this on Win32. However, a method was added to query support for presenting to the windows desktop as a whole.

2) If a native window object (HWND) is used by one graphics API, and then is later used by a different graphics API (one of which is Vulkan), can these uses interfere with each other?

RESOLVED: Yes.

Uses of a window object by multiple graphics APIs results in undefined behavior. Such behavior may succeed when using one Vulkan implementation but fail when using a different Vulkan implementation. Potential failures include:

- Creating then destroying a flip presentation model DXGI swapchain on a window object can prevent vkCreateSwapchainKHR from succeeding on the same window object.
- Creating then destroying a VkSwapchainKHR on a window object can prevent creation of a bitblt model DXGI swapchain on the same window object.
- Creating then destroying a VkSwapchainKHR on a window object can effectively SetPixelFormat to a different format than the format chosen by an OpenGL application.

- Creating then destroying a VkSwapchainKHR on a window object on one VkPhysicalDevice can prevent vkCreateSwapchainKHR from succeeding on the same window object, but on a different VkPhysicalDevice that is associated with a different Vulkan ICD.

In all cases the problem can be worked around by creating a new window object.

Technical details include:

- Creating a DXGI swapchain over a window object can alter the object for the remainder of its lifetime. The alteration persists even after the DXGI swapchain has been destroyed. This alteration can make it impossible for a conformant Vulkan implementation to create a VkSwapchainKHR over the same window object. Mention of this alteration can be found in the remarks section of the MSDN documentation for `DXGI_SWAP_EFFECT`.
- Calling GDI's `SetPixelFormat` (needed by OpenGL's WGL layer) on a window object alters the object for the remainder of its lifetime. The MSDN documentation for `SetPixelFormat` explains that a window object's pixel format can be set only one time.
- Creating a VkSwapchainKHR over a window object can alter the object for its remaining lifetime. Either of the above alterations may occur as a side effect of vkCreateSwapchainKHR.

Version History

- Revision 1, 2015-09-23 (Jesse Hall)
  - Initial draft, based on the previous contents of VK_EXT_KHR_swapchain (later renamed VK_EXT_KHR_surface).
- Revision 2, 2015-10-02 (James Jones)
  - Added presentation support query for win32 desktops.
- Revision 3, 2015-10-26 (Ian Elliott)
  - Renamed from VK_EXT_KHR_win32_surface to VK_KHR_win32_surface.
- Revision 4, 2015-11-03 (Daniel Rakos)
  - Added allocation callbacks to vkCreateWin32SurfaceKHR.
- Revision 5, 2015-11-28 (Daniel Rakos)
  - Updated the surface create function to take a pCreateInfo structure.
- Revision 6, 2017-04-24 (Jeff Juliano)
  - Add issue 2 addressing reuse of a native window object in a different Graphics API, or by a different Vulkan ICD.

VK_KHR_workgroup_memory_explicit_layout

Name String

`VK_KHR_workgroup_memory_explicit_layout`

Extension Type

Device extension

Registered Extension Number

337

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_KHR_workgroup_memory_explicit_layout

Contact

- Caio Marcelo de Oliveira Filho [GitHub]cmarcelo

Other Extension Metadata

Last Modified Date

2020-06-01

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_shared_memory_block`

Contributors

- Caio Marcelo de Oliveira Filho, Intel
- Jeff Bolz, NVIDIA
- Graeme Leese, Broadcom
- Faith Ekstrand, Intel
- Daniel Koch, NVIDIA

Description

This extension adds Vulkan support for the `SPV_KHR_workgroup_memory_explicit_layout` SPIR-V extension, which allows shaders to explicitly define the layout of `Workgroup` storage class memory and create aliases between variables from that storage class in a compute shader.

The aliasing feature allows different “views” on the same data, so the shader can bulk copy data from another storage class using one type (e.g. an array of large vectors), and then use the data with a more specific type. It also enables reducing the amount of workgroup memory consumed by allowing the shader to alias data whose lifetimes do not overlap.

The explicit layout support and some form of aliasing is also required for layering OpenCL on top of Vulkan.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceWorkgroupMemoryExplicitLayoutFeaturesKHR

New Enum Constants

- `VK_KHR_WORKGROUP_MEMORY_EXPLICIT_LAYOUT_EXTENSION_NAME`
- `VK_KHR_WORKGROUP_MEMORY_EXPLICIT_LAYOUT_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_WORKGROUP_MEMORY_EXPLICIT_LAYOUT_FEATURES_KHR`

New SPIR-V Capabilities

- `WorkgroupMemoryExplicitLayoutKHR`
- `WorkgroupMemoryExplicitLayout8BitAccessKHR`
- `WorkgroupMemoryExplicitLayout16BitAccessKHR`

Version History

- Revision 1, 2020-06-01 (Caio Marcelo de Oliveira Filho)
  - Initial version

VK_KHR_xcb_surface

Name String

`VK_KHR_xcb_surface`

Extension Type

Instance extension

Registered Extension Number

6

Revision

6

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- Jesse Hall [GitHub]critsec
- Ian Elliott [GitHub]ianelliottus

Other Extension Metadata

Last Modified Date

2015-11-28

IP Status

No known IP claims.

Contributors

- Patrick Doane, Blizzard
- Faith Ekstrand, Intel
- Ian Elliott, LunarG
- Courtney Goeltzenleuchter, LunarG
- Jesse Hall, Google
- James Jones, NVIDIA
- Antoine Labour, Google
- Jon Leech, Khronos
- David Mao, AMD
- Norbert Nopper, Freescale
- Alon Or-bach, Samsung
- Daniel Rakos, AMD
- Graham Sellers, AMD
- Ray Smith, ARM
- Jeff Vigil, Qualcomm
- Chia-I Wu, LunarG

Description

The `VK_KHR_xcb_surface` extension is an instance extension. It provides a mechanism to create a VkSurfaceKHR object (defined by the `VK_KHR_surface` extension) that refers to an X11 `Window`, using the XCB client-side library, as well as a query to determine support for rendering via XCB.

New Commands

- vkCreateXcbSurfaceKHR
- vkGetPhysicalDeviceXcbPresentationSupportKHR

New Structures

- VkXcbSurfaceCreateInfoKHR

New Bitmasks

- VkXcbSurfaceCreateFlagsKHR

New Enum Constants

- `VK_KHR_XCB_SURFACE_EXTENSION_NAME`
- `VK_KHR_XCB_SURFACE_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_XCB_SURFACE_CREATE_INFO_KHR`

Issues

1) Does XCB need a way to query for compatibility between a particular physical device and a specific screen? This would be a more general query than vkGetPhysicalDeviceSurfaceSupportKHR: If it returned `VK_TRUE`, then the physical device could be assumed to support presentation to any window on that screen.

RESOLVED: Yes, this is needed for toolkits that want to create a VkDevice before creating a window. To ensure the query is reliable, it must be made against a particular X visual rather than the screen in general.

Version History

- Revision 1, 2015-09-23 (Jesse Hall)
  - Initial draft, based on the previous contents of VK_EXT_KHR_swapchain (later renamed VK_EXT_KHR_surface).
- Revision 2, 2015-10-02 (James Jones)
  - Added presentation support query for an (xcb_connection_t*, xcb_visualid_t) pair.
  - Removed “root” parameter from CreateXcbSurfaceKHR(), as it is redundant when a window on the same screen is specified as well.
  - Adjusted wording of issue #1 and added agreed upon resolution.
- Revision 3, 2015-10-14 (Ian Elliott)
  - Removed “root” parameter from CreateXcbSurfaceKHR() in one more place.
- Revision 4, 2015-10-26 (Ian Elliott)
  - Renamed from VK_EXT_KHR_xcb_surface to VK_KHR_xcb_surface.
- Revision 5, 2015-10-23 (Daniel Rakos)
  - Added allocation callbacks to vkCreateXcbSurfaceKHR.
- Revision 6, 2015-11-28 (Daniel Rakos)
  - Updated the surface create function to take a pCreateInfo structure.

VK_KHR_xlib_surface

Name String

`VK_KHR_xlib_surface`

Extension Type

Instance extension

Registered Extension Number

5

Revision

6

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- Jesse Hall [GitHub]critsec
- Ian Elliott [GitHub]ianelliottus

Other Extension Metadata

Last Modified Date

2015-11-28

IP Status

No known IP claims.

Contributors

- Patrick Doane, Blizzard
- Faith Ekstrand, Intel
- Ian Elliott, LunarG
- Courtney Goeltzenleuchter, LunarG
- Jesse Hall, Google
- James Jones, NVIDIA
- Antoine Labour, Google
- Jon Leech, Khronos
- David Mao, AMD
- Norbert Nopper, Freescale
- Alon Or-bach, Samsung

- Daniel Rakos, AMD
- Graham Sellers, AMD
- Ray Smith, ARM
- Jeff Vigil, Qualcomm
- Chia-I Wu, LunarG

Description

The VK_KHR_xlib_surface extension is an instance extension. It provides a mechanism to create a VkSurfaceKHR object (defined by the VK_KHR_surface extension) that refers to an X11 Window, using the Xlib client-side library, as well as a query to determine support for rendering via Xlib.

New Commands

- vkCreateXlibSurfaceKHR
- vkGetPhysicalDeviceXlibPresentationSupportKHR

New Structures

- VkXlibSurfaceCreateInfoKHR

New Bitmasks

- VkXlibSurfaceCreateFlagsKHR

New Enum Constants

- VK_KHR_XLIB_SURFACE_EXTENSION_NAME
- VK_KHR_XLIB_SURFACE_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_XLIB_SURFACE_CREATE_INFO_KHR

Issues

1) Does X11 need a way to query for compatibility between a particular physical device and a specific screen? This would be a more general query than vkGetPhysicalDeviceSurfaceSupportKHR; if it returned VK_TRUE, then the physical device could be assumed to support presentation to any window on that screen.

RESOLVED: Yes, this is needed for toolkits that want to create a VkDevice before creating a window. To ensure the query is reliable, it must be made against a particular X visual rather than the screen in general.

Version History

- Revision 1, 2015-09-23 (Jesse Hall)
  - Initial draft, based on the previous contents of VK_EXT_KHR_swapchain (later renamed

VK_EXT_KHR_surface).

- Revision 2, 2015-10-02 (James Jones)
  - Added presentation support query for (Display*, VisualID) pair.
  - Removed “root” parameter from CreateXlibSurfaceKHR(), as it is redundant when a window on the same screen is specified as well.
  - Added appropriate X errors.
  - Adjusted wording of issue #1 and added agreed upon resolution.
- Revision 3, 2015-10-14 (Ian Elliott)
  - Renamed this extension from VK_EXT_KHR_x11_surface to VK_EXT_KHR_xlib_surface.
- Revision 4, 2015-10-26 (Ian Elliott)
  - Renamed from VK_EXT_KHR_xlib_surface to VK_KHR_xlib_surface.
- Revision 5, 2015-11-03 (Daniel Rakos)
  - Added allocation callbacks to vkCreateXlibSurfaceKHR.
- Revision 6, 2015-11-28 (Daniel Rakos)
  - Updated the surface create function to take a pCreateInfo structure.

VK_KHR_zero_initialize_workgroup_memory

Name String

`VK_KHR_zero_initialize_workgroup_memory`

Extension Type

Device extension

Registered Extension Number

326

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Deprecation State

- *Promoted* to Vulkan 1.3

Contact

- Alan Baker [GitHub]alan-baker

Other Extension Metadata

Last Modified Date

2020-11-18

IP Status

No known IP claims.

Contributors

- Alan Baker, Google
- Jeff Bolz, Nvidia
- Faith Ekstrand, Intel

Description

This extension allows the use of a null constant initializer on shader Workgroup memory variables, allowing implementations to expose any special hardware or instructions they may have. Zero initialization is commonly used by applications running untrusted content (e.g. web browsers) as way of defeating memory-scraping attacks.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceZeroInitializeWorkgroupMemoryFeaturesKHR

New Enum Constants

- `VK_KHR_ZERO_INITIALIZE_WORKGROUP_MEMORY_EXTENSION_NAME`
- `VK_KHR_ZERO_INITIALIZE_WORKGROUP_MEMORY_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ZERO_INITIALIZE_WORKGROUP_MEMORY_FEATURES_KHR`

Promotion to Vulkan 1.3

Vulkan APIs in this extension are included in core Vulkan 1.3, with the KHR suffix omitted. External interactions defined by this extension, such as SPIR-V token names, retain their original names. The original Vulkan API names are still available as aliases of the core functionality.

Version History

- Revision 1, 2020-11-18 (Alan Baker)
  - Internal draft version

VK_EXT_astc_decode_mode

Name String

`VK_EXT_astc_decode_mode`

Extension Type

Device extension

Registered Extension Number

68

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Jan-Harald Fredriksen [GitHub]janharaldfredriksen-arm

Other Extension Metadata

Last Modified Date

2018-08-07

Contributors

- Jan-Harald Fredriksen, Arm

Description

The existing specification requires that low dynamic range (LDR) ASTC textures are decompressed to FP16 values per component. In many cases, decompressing LDR textures to a lower precision intermediate result gives acceptable image quality. Source material for LDR textures is typically authored as 8-bit UNORM values, so decoding to FP16 values adds little value. On the other hand, reducing precision of the decoded result reduces the size of the decompressed data, potentially improving texture cache performance and saving power.

The goal of this extension is to enable this efficiency gain on existing ASTC texture data. This is achieved by giving the application the ability to select the intermediate decoding precision.

Three decoding options are provided:

- Decode to `VK_FORMAT_R16G16B16A16_SFLOAT` precision: This is the default, and matches the required behavior in the core API.
- Decode to `VK_FORMAT_R8G8B8A8_UNORM` precision: This is provided as an option in LDR mode.
- Decode to `VK_FORMAT_E5B9G9R9_UFLOAT_PACK32` precision: This is provided as an option in both LDR and HDR mode. In this mode, negative values cannot be represented and are clamped to zero. The alpha component is ignored, and the results are as if alpha was 1.0. This decode mode is

optional and support can be queried via the physical device properties.

New Structures

- Extending VkImageViewCreateInfo:
  - VkImageViewASTCDecodeModeEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceASTCDecodeFeaturesEXT

New Enum Constants

- VK_EXT_ASTC_DECODE_MODE_EXTENSION_NAME
- VK_EXT_ASTC_DECODE_MODE_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_IMAGE_VIEW_ASTC_DECODE_MODE_EXT
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ASTC_DECODE_FEATURES_EXT

Issues

1) Are implementations allowed to decode at a higher precision than what is requested?

```
RESOLUTION: No.
If we allow this, then this extension could be exposed on all
implementations that support ASTC.
But developers would have no way of knowing what precision was actually
used, and thus whether the image quality is sufficient at reduced
precision.
```

2) Should the decode mode be image view state and/or sampler state?

```
RESOLUTION: Image view state only.
Some implementations treat the different decode modes as different
texture formats.
```

Example

Create an image view that decodes to VK_FORMAT_R8G8B8A8_UNORM precision:

```
    VkImageViewASTCDecodeModeEXT decodeMode =
    {
        .sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_ASTC_DECODE_MODE_EXT,
        .pNext = NULL,
        .decodeMode = VK_FORMAT_R8G8B8A8_UNORM
    };
```

```
    VkImageViewCreateInfo createInfo =
    {
        .sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO,
        .pNext = &decodeMode,
        // flags, image, viewType set to application-desired values
        .format = VK_FORMAT_ASTC_8x8_UNORM_BLOCK,
        // components, subresourceRange set to application-desired values
    };

    VkImageView imageView;
    VkResult result = vkCreateImageView(
        device,
        &createInfo,
        NULL,
        &imageView);
```

Version History

- Revision 1, 2018-08-07 (Jan-Harald Fredriksen)
  - Initial revision

VK_EXT_attachment_feedback_loop_dynamic_state

Name String

VK_EXT_attachment_feedback_loop_dynamic_state

Extension Type

Device extension

Registered Extension Number

525

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
VK_EXT_attachment_feedback_loop_layout

Special Uses

- OpenGL / ES support
- D3D support

Contact

- Mike Blumenkrantz [GitHub]zmike

Extension Proposal

VK_EXT_attachment_feedback_loop_dynamic_state

Other Extension Metadata

Last Modified Date

2023-04-28

IP Status

No known IP claims.

Contributors

- Mike Blumenkrantz, Valve
- Daniel Story, Nintendo
- Stu Smith, AMD
- Samuel Pitoiset, Valve
- Ricardo Garcia, Igalia

Description

This extension adds support for setting attachment feedback loops dynamically on command buffers.

New Commands

- vkCmdSetAttachmentFeedbackLoopEnableEXT

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceAttachmentFeedbackLoopDynamicStateFeaturesEXT

New Enum Constants

- `VK_EXT_ATTACHMENT_FEEDBACK_LOOP_DYNAMIC_STATE_EXTENSION_NAME`
- `VK_EXT_ATTACHMENT_FEEDBACK_LOOP_DYNAMIC_STATE_SPEC_VERSION`
- Extending VkDynamicState:
  - `VK_DYNAMIC_STATE_ATTACHMENT_FEEDBACK_LOOP_ENABLE_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ATTACHMENT_FEEDBACK_LOOP_DYNAMIC_STATE_FEATURES_EXT`

Version History

- Revision 1, 2023-04-28 (Mike Blumenkrantz)
  - Initial revision

VK_EXT_attachment_feedback_loop_layout

Name String

`VK_EXT_attachment_feedback_loop_layout`

Extension Type

Device extension

Registered Extension Number

340

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Uses

- OpenGL / ES support
- D3D support

Contact

- Joshua Ashton [GitHub]Joshua-Ashton

Extension Proposal

VK_EXT_attachment_feedback_loop_layout

Other Extension Metadata

Last Modified Date

2022-04-04

IP Status

No known IP claims.

Contributors

- Joshua Ashton, Valve
- Faith Ekstrand, Collabora

- Bas Nieuwenhuizen, Google
- Samuel Iglesias Gonsálvez, Igalia
- Ralph Potter, Samsung
- Jan-Harald Fredriksen, Arm
- Ricardo Garcia, Igalia

Description

This extension adds a new image layout, `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT`, which allows applications to have an image layout in which they are able to both render to and sample/fetch from the same subresource of an image in a given render pass.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceAttachmentFeedbackLoopLayoutFeaturesEXT

New Enum Constants

- `VK_EXT_ATTACHMENT_FEEDBACK_LOOP_LAYOUT_EXTENSION_NAME`
- `VK_EXT_ATTACHMENT_FEEDBACK_LOOP_LAYOUT_SPEC_VERSION`
- Extending VkDependencyFlagBits:
  - `VK_DEPENDENCY_FEEDBACK_LOOP_BIT_EXT`
- Extending VkImageLayout:
  - `VK_IMAGE_LAYOUT_ATTACHMENT_FEEDBACK_LOOP_OPTIMAL_EXT`
- Extending VkImageUsageFlagBits:
  - `VK_IMAGE_USAGE_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT`
- Extending VkPipelineCreateFlagBits:
  - `VK_PIPELINE_CREATE_COLOR_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT`
  - `VK_PIPELINE_CREATE_DEPTH_STENCIL_ATTACHMENT_FEEDBACK_LOOP_BIT_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ATTACHMENT_FEEDBACK_LOOP_LAYOUT_FEATURES_EXT`

Version History

- Revision 2, 2022-04-04 (Joshua Ashton)
  - Renamed from VALVE to EXT.
- Revision 1, 2021-03-09 (Joshua Ashton)
  - Initial draft.

VK_EXT_blend_operation_advanced

Name String

VK_EXT_blend_operation_advanced

Extension Type

Device extension

Registered Extension Number

149

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2017-06-12

Contributors

- Jeff Bolz, NVIDIA

Description

This extension adds a number of “advanced” blending operations that **can** be used to perform new color blending operations, many of which are more complex than the standard blend modes provided by unextended Vulkan. This extension requires different styles of usage, depending on the level of hardware support and the enabled features:

- If VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT::advancedBlendCoherentOperations is VK_FALSE, the new blending operations are supported, but a memory dependency **must** separate each advanced blend operation on a given sample. VK_ACCESS_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT is used to synchronize reads using advanced blend operations.
- If VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT::advancedBlendCoherentOperations is VK_TRUE, advanced blend operations obey primitive order just like basic blend operations.

In unextended Vulkan, the set of blending operations is limited, and **can** be expressed very simply.

The VK_BLEND_OP_MIN and VK_BLEND_OP_MAX blend operations simply compute component-wise minimums or maximums of source and destination color components. The VK_BLEND_OP_ADD, VK_BLEND_OP_SUBTRACT, and VK_BLEND_OP_REVERSE_SUBTRACT modes multiply the source and destination colors by source and destination factors and either add the two products together or subtract one from the other. This limited set of operations supports many common blending operations but precludes the use of more sophisticated transparency and blending operations commonly available in many dedicated imaging APIs.

This extension provides a number of new "advanced" blending operations. Unlike traditional blending operations using VK_BLEND_OP_ADD, these blending equations do not use source and destination factors specified by VkBlendFactor. Instead, each blend operation specifies a complete equation based on the source and destination colors. These new blend operations are used for both RGB and alpha components; they **must** not be used to perform separate RGB and alpha blending (via different values of color and alpha VkBlendOp).

These blending operations are performed using premultiplied colors, where RGB colors **can** be considered premultiplied or non-premultiplied by alpha, according to the srcPremultiplied and dstPremultiplied members of VkPipelineColorBlendAdvancedStateCreateInfoEXT. If a color is considered non-premultiplied, the (R,G,B) color components are multiplied by the alpha component prior to blending. For non-premultiplied color components in the range [0,1], the corresponding premultiplied color component would have values in the range [0 × A, 1 × A].

Many of these advanced blending equations are formulated where the result of blending source and destination colors with partial coverage have three separate contributions: from the portions covered by both the source and the destination, from the portion covered only by the source, and from the portion covered only by the destination. The blend parameter VkPipelineColorBlendAdvancedStateCreateInfoEXT::blendOverlap **can** be used to specify a correlation between source and destination pixel coverage. If set to VK_BLEND_OVERLAP_CONJOINT_EXT, the source and destination are considered to have maximal overlap, as would be the case if drawing two objects on top of each other. If set to VK_BLEND_OVERLAP_DISJOINT_EXT, the source and destination are considered to have minimal overlap, as would be the case when rendering a complex polygon tessellated into individual non-intersecting triangles. If set to VK_BLEND_OVERLAP_UNCORRELATED_EXT, the source and destination coverage are assumed to have no spatial correlation within the pixel.

In addition to the coherency issues on implementations not supporting advancedBlendCoherentOperations, this extension has several limitations worth noting. First, the new blend operations have a limit on the number of color attachments they **can** be used with, as indicated by VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT ::advancedBlendMaxColorAttachments. Additionally, blending precision **may** be limited to 16-bit floating-point, which **may** result in a loss of precision and dynamic range for framebuffer formats with 32-bit floating-point components, and in a loss of precision for formats with 12- and 16-bit signed or unsigned normalized integer components.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceBlendOperationAdvancedFeaturesEXT

- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceBlendOperationAdvancedPropertiesEXT
- Extending VkPipelineColorBlendStateCreateInfo:
  - VkPipelineColorBlendAdvancedStateCreateInfoEXT

New Enums

- VkBlendOverlapEXT

New Enum Constants

- VK_EXT_BLEND_OPERATION_ADVANCED_EXTENSION_NAME
- VK_EXT_BLEND_OPERATION_ADVANCED_SPEC_VERSION
- Extending VkAccessFlagBits:
  - VK_ACCESS_COLOR_ATTACHMENT_READ_NONCOHERENT_BIT_EXT
- Extending VkBlendOp:
  - VK_BLEND_OP_BLUE_EXT
  - VK_BLEND_OP_COLORBURN_EXT
  - VK_BLEND_OP_COLORDODGE_EXT
  - VK_BLEND_OP_CONTRAST_EXT
  - VK_BLEND_OP_DARKEN_EXT
  - VK_BLEND_OP_DIFFERENCE_EXT
  - VK_BLEND_OP_DST_ATOP_EXT
  - VK_BLEND_OP_DST_EXT
  - VK_BLEND_OP_DST_IN_EXT
  - VK_BLEND_OP_DST_OUT_EXT
  - VK_BLEND_OP_DST_OVER_EXT
  - VK_BLEND_OP_EXCLUSION_EXT
  - VK_BLEND_OP_GREEN_EXT
  - VK_BLEND_OP_HARDLIGHT_EXT
  - VK_BLEND_OP_HARDMIX_EXT
  - VK_BLEND_OP_HSL_COLOR_EXT
  - VK_BLEND_OP_HSL_HUE_EXT
  - VK_BLEND_OP_HSL_LUMINOSITY_EXT
  - VK_BLEND_OP_HSL_SATURATION_EXT
  - VK_BLEND_OP_INVERT_EXT
  - VK_BLEND_OP_INVERT_OVG_EXT

    - VK_BLEND_OP_INVERT_RGB_EXT
    - VK_BLEND_OP_LIGHTEN_EXT
    - VK_BLEND_OP_LINEARBURN_EXT
    - VK_BLEND_OP_LINEARDODGE_EXT
    - VK_BLEND_OP_LINEARLIGHT_EXT
    - VK_BLEND_OP_MINUS_CLAMPED_EXT
    - VK_BLEND_OP_MINUS_EXT
    - VK_BLEND_OP_MULTIPLY_EXT
    - VK_BLEND_OP_OVERLAY_EXT
    - VK_BLEND_OP_PINLIGHT_EXT
    - VK_BLEND_OP_PLUS_CLAMPED_ALPHA_EXT
    - VK_BLEND_OP_PLUS_CLAMPED_EXT
    - VK_BLEND_OP_PLUS_DARKER_EXT
    - VK_BLEND_OP_PLUS_EXT
    - VK_BLEND_OP_RED_EXT
    - VK_BLEND_OP_SCREEN_EXT
    - VK_BLEND_OP_SOFTLIGHT_EXT
    - VK_BLEND_OP_SRC_ATOP_EXT
    - VK_BLEND_OP_SRC_EXT
    - VK_BLEND_OP_SRC_IN_EXT
    - VK_BLEND_OP_SRC_OUT_EXT
    - VK_BLEND_OP_SRC_OVER_EXT
    - VK_BLEND_OP_VIVIDLIGHT_EXT
    - VK_BLEND_OP_XOR_EXT
    - VK_BLEND_OP_ZERO_EXT
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BLEND_OPERATION_ADVANCED_FEATURES_EXT
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BLEND_OPERATION_ADVANCED_PROPERTIES_EXT
    - VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_ADVANCED_STATE_CREATE_INFO_EXT

Issues

None.

Version History

- Revision 1, 2017-06-12 (Jeff Bolz)

    - Internal revisions
- Revision 2, 2017-06-12 (Jeff Bolz)
    - Internal revisions

VK_EXT_border_color_swizzle

Name String

`VK_EXT_border_color_swizzle`

Extension Type

Device extension

Registered Extension Number

412

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_EXT_custom_border_color

Special Uses

- OpenGL / ES support
- D3D support

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2021-10-12

IP Status

No known IP claims.

Contributors

- Graeme Leese, Broadcom
- Jan-Harald Fredriksen, Arm
- Ricardo Garcia, Igalia
- Shahbaz Youssefi, Google
- Stu Smith, AMD

Description

After the publication of VK_EXT_custom_border_color, it was discovered that some implementations had undefined behavior when combining a sampler that uses a custom border color with image views whose component mapping is not the identity mapping.

Since VK_EXT_custom_border_color has already shipped, this new extension VK_EXT_border_color_swizzle was created to define the interaction between custom border colors and non-identity image view swizzles, and provide a work-around for implementations that must pre-swizzle the sampler border color to match the image view component mapping it is combined with.

This extension also defines the behavior between samplers with an opaque black border color and image views with a non-identity component swizzle, which was previously left undefined.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceBorderColorSwizzleFeaturesEXT
- Extending VkSamplerCreateInfo:
  - VkSamplerBorderColorComponentMappingCreateInfoEXT

New Enum Constants

- `VK_EXT_BORDER_COLOR_SWIZZLE_EXTENSION_NAME`
- `VK_EXT_BORDER_COLOR_SWIZZLE_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_BORDER_COLOR_SWIZZLE_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_SAMPLER_BORDER_COLOR_COMPONENT_MAPPING_CREATE_INFO_EXT`

Issues

None.

Version History

- Revision 1, 2021-10-12 (Piers Daniell)
  - Internal revisions.

VK_EXT_color_write_enable

Name String

`VK_EXT_color_write_enable`

Extension Type

Device extension

Registered Extension Number

382

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Sharif Elcott [GitHub]selcott

Other Extension Metadata

Last Modified Date

2020-02-25

IP Status

No known IP claims.

Contributors

- Sharif Elcott, Google
- Tobias Hector, AMD
- Piers Daniell, NVIDIA

Description

This extension allows for selectively enabling and disabling writes to output color attachments via a pipeline dynamic state.

The intended use cases for this new state are mostly identical to those of colorWriteMask, such as selectively disabling writes to avoid feedback loops between subpasses or bandwidth savings for unused outputs. By making the state dynamic, one additional benefit is the ability to reduce pipeline counts and pipeline switching via shaders that write a superset of the desired data of which subsets are selected dynamically. The reason for a new state, colorWriteEnable, rather than making colorWriteMask dynamic is that, on many implementations, the more flexible per-component semantics of the colorWriteMask state cannot be made dynamic in a performant manner.

New Commands

- vkCmdSetColorWriteEnableEXT

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceColorWriteEnableFeaturesEXT
- Extending VkPipelineColorBlendStateCreateInfo:
  - VkPipelineColorWriteCreateInfoEXT

New Enum Constants

- VK_EXT_COLOR_WRITE_ENABLE_EXTENSION_NAME
- VK_EXT_COLOR_WRITE_ENABLE_SPEC_VERSION
- Extending VkDynamicState:
  - VK_DYNAMIC_STATE_COLOR_WRITE_ENABLE_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_COLOR_WRITE_ENABLE_FEATURES_EXT
  - VK_STRUCTURE_TYPE_PIPELINE_COLOR_WRITE_CREATE_INFO_EXT

Version History

- Revision 1, 2020-01-25 (Sharif Elcott)
  - Internal revisions

VK_EXT_conservative_rasterization

Name String

VK_EXT_conservative_rasterization

Extension Type

Device extension

Registered Extension Number

102

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_EXT_fragment_fully_covered

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2020-06-09

Interactions and External Dependencies

- This extension requires `SPV_EXT_fragment_fully_covered` if the `VkPhysicalDeviceConservativeRasterizationPropertiesEXT::fullyCoveredFragmentShaderInputVariable` feature is used.
- This extension requires `SPV_KHR_post_depth_coverage`if the `VkPhysicalDeviceConservativeRasterizationPropertiesEXT::conservativeRasterizationPostDepthCoverage` feature is used.
- This extension provides API support for `GL_NV_conservative_raster_underestimation` if the `VkPhysicalDeviceConservativeRasterizationPropertiesEXT::fullyCoveredFragmentShaderInputVariable` feature is used.

Contributors

- Daniel Koch, NVIDIA
- Daniel Rakos, AMD
- Jeff Bolz, NVIDIA
- Slawomir Grajewski, Intel
- Stu Smith, Imagination Technologies

Description

This extension adds a new rasterization mode called conservative rasterization. There are two modes of conservative rasterization; overestimation and underestimation.

When overestimation is enabled, if any part of the primitive, including its edges, covers any part of the rectangular pixel area, including its sides, then a fragment is generated with all coverage samples turned on. This extension allows for some variation in implementations by accounting for differences in overestimation, where the generating primitive size is increased at each of its edges by some sub-pixel amount to further increase conservative pixel coverage. Implementations can allow the application to specify an extra overestimation beyond the base overestimation the implementation already does. It also allows implementations to either cull degenerate primitives or rasterize them.

When underestimation is enabled, fragments are only generated if the rectangular pixel area is fully covered by the generating primitive. If supported by the implementation, when a pixel rectangle is fully covered the fragment shader input variable builtin called FullyCoveredEXT is set to true. The shader variable works in either overestimation or underestimation mode.

Implementations can process degenerate triangles and lines by either discarding them or generating conservative fragments for them. Degenerate triangles are those that end up with zero area after the rasterizer quantizes them to the fixed-point pixel grid. Degenerate lines are those with zero length after quantization.

New Structures

- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceConservativeRasterizationPropertiesEXT
- Extending VkPipelineRasterizationStateCreateInfo:
  - VkPipelineRasterizationConservativeStateCreateInfoEXT

New Enums

- VkConservativeRasterizationModeEXT

New Bitmasks

- VkPipelineRasterizationConservativeStateCreateFlagsEXT

New Enum Constants

- `VK_EXT_CONSERVATIVE_RASTERIZATION_EXTENSION_NAME`
- `VK_EXT_CONSERVATIVE_RASTERIZATION_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CONSERVATIVE_RASTERIZATION_PROPERTIES_EXT`
  - `VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_CONSERVATIVE_STATE_CREATE_INFO_EXT`

New Built-In Variables

- FullyCoveredEXT

New SPIR-V Capabilities

- FragmentFullyCoveredEXT

Version History

- Revision 1.1, 2020-09-06 (Piers Daniell)
  - Add missing SPIR-V and GLSL dependencies.
- Revision 1, 2017-08-28 (Piers Daniell)
  - Internal revisions

VK_EXT_custom_border_color

Name String

`VK_EXT_custom_border_color`

Extension Type

Device extension

Registered Extension Number

288

Revision

12

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Uses

- OpenGL / ES support
- D3D support

Contact

- Liam Middlebrook [GitHub]liam-middlebrook

Other Extension Metadata

Last Modified Date

2020-04-16

IP Status

No known IP claims.

Contributors

- Joshua Ashton, Valve
- Hans-Kristian Arntzen, Valve
- Philip Rebohle, Valve
- Liam Middlebrook, NVIDIA
- Jeff Bolz, NVIDIA
- Tobias Hector, AMD
- Faith Ekstrand, Intel
- Spencer Fricke, Samsung Electronics
- Graeme Leese, Broadcom

- Jesse Hall, Google
- Jan-Harald Fredriksen, ARM
- Tom Olson, ARM
- Stuart Smith, Imagination Technologies
- Donald Scorgie, Imagination Technologies
- Alex Walters, Imagination Technologies
- Peter Quayle, Imagination Technologies

Description

This extension provides cross-vendor functionality to specify a custom border color for use when the sampler address mode `VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER` is used.

To create a sampler which uses a custom border color set VkSamplerCreateInfo::`borderColor` to one of:

- `VK_BORDER_COLOR_FLOAT_CUSTOM_EXT`
- `VK_BORDER_COLOR_INT_CUSTOM_EXT`

When `VK_BORDER_COLOR_FLOAT_CUSTOM_EXT` or `VK_BORDER_COLOR_INT_CUSTOM_EXT` is used, applications must provide a VkSamplerCustomBorderColorCreateInfoEXT in the `pNext` chain for VkSamplerCreateInfo.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceCustomBorderColorFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceCustomBorderColorPropertiesEXT
- Extending VkSamplerCreateInfo:
  - VkSamplerCustomBorderColorCreateInfoEXT

New Enum Constants

- `VK_EXT_CUSTOM_BORDER_COLOR_EXTENSION_NAME`
- `VK_EXT_CUSTOM_BORDER_COLOR_SPEC_VERSION`
- Extending VkBorderColor:
  - `VK_BORDER_COLOR_FLOAT_CUSTOM_EXT`
  - `VK_BORDER_COLOR_INT_CUSTOM_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CUSTOM_BORDER_COLOR_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_CUSTOM_BORDER_COLOR_PROPERTIES_EXT`

    - VK_STRUCTURE_TYPE_SAMPLER_CUSTOM_BORDER_COLOR_CREATE_INFO_EXT

Issues

1) Should VkClearColorValue be used for the border color value, or should we have our own struct/union? Do we need to specify the type of the input values for the components? This is more of a concern if VkClearColorValue is used here because it provides a union of float,int,uint types.

RESOLVED: Will reuse existing VkClearColorValue structure in order to easily take advantage of float,int,uint borderColor types.

2) For hardware which supports a limited number of border colors what happens if that number is exceeded? Should this be handled by the driver unbeknownst to the application? In Revision 1 we had solved this issue using a new Object type, however that may have lead to additional system resource consumption which would otherwise not be required.

RESOLVED: Added VkPhysicalDeviceCustomBorderColorPropertiesEXT::maxCustomBorderColorSamplers for tracking implementation-specific limit, and Valid Usage statement handling overflow.

3) Should this be supported for immutable samplers at all, or by a feature bit? Some implementations may not be able to support custom border colors on immutable samplers — is it worthwhile enabling this to work on them for implementations that can support it, or forbidding it entirely.

RESOLVED: Samplers created with a custom border color are forbidden from being immutable. This resolves concerns for implementations where the custom border color is an index to a LUT instead of being directly embedded into sampler state.

4) Should UINT and SINT (unsigned integer and signed integer) border color types be separated or should they be combined into one generic INT (integer) type?

RESOLVED: Separating these does not make much sense as the existing fixed border color types do not have this distinction, and there is no reason in hardware to do so. This separation would also create unnecessary work and considerations for the application.

Version History

- Revision 1, 2019-10-10 (Joshua Ashton)
    - Internal revisions.
- Revision 2, 2019-10-11 (Liam Middlebrook)
    - Remove VkCustomBorderColor object and associated functions
    - Add issues concerning HW limitations for custom border color count
- Revision 3, 2019-10-12 (Joshua Ashton)
    - Re-expose the limits for the maximum number of unique border colors
    - Add extra details about border color tracking
    - Fix typos
- Revision 4, 2019-10-12 (Joshua Ashton)

    - Changed maxUniqueCustomBorderColors to a uint32_t from a VkDeviceSize
- Revision 5, 2019-10-14 (Liam Middlebrook)
    - Added features bit
- Revision 6, 2019-10-15 (Joshua Ashton)
    - Type-ize VK_BORDER_COLOR_CUSTOM
    - Fix const-ness on `pNext` of VkSamplerCustomBorderColorCreateInfoEXT
- Revision 7, 2019-11-26 (Liam Middlebrook)
    - Renamed maxUniqueCustomBorderColors to maxCustomBorderColors
- Revision 8, 2019-11-29 (Joshua Ashton)
    - Renamed borderColor member of VkSamplerCustomBorderColorCreateInfoEXT to customBorderColor
- Revision 9, 2020-02-19 (Joshua Ashton)
    - Renamed maxCustomBorderColors to maxCustomBorderColorSamplers
- Revision 10, 2020-02-21 (Joshua Ashton)
    - Added format to VkSamplerCustomBorderColorCreateInfoEXT and feature bit
- Revision 11, 2020-04-07 (Joshua Ashton)
    - Dropped UINT/SINT border color differences, consolidated types
- Revision 12, 2020-04-16 (Joshua Ashton)
    - Renamed VK_BORDER_COLOR_CUSTOM_FLOAT_EXT to VK_BORDER_COLOR_FLOAT_CUSTOM_EXT for consistency

VK_EXT_debug_utils

Name String

`VK_EXT_debug_utils`

Extension Type

Instance extension

Registered Extension Number

129

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

None

Special Use

- Debugging tools

Contact

- Mark Young [GitHub]marky-lunarg

Other Extension Metadata

Last Modified Date

2020-04-03

Revision

2

IP Status

No known IP claims.

Dependencies

- This extension is written against version 1.0 of the Vulkan API.
- Requires VkObjectType

Contributors

- Mark Young, LunarG
- Baldur Karlsson
- Ian Elliott, Google
- Courtney Goeltzenleuchter, Google
- Karl Schultz, LunarG
- Mark Lobodzinski, LunarG
- Mike Schuchardt, LunarG
- Jaakko Konttinen, AMD
- Dan Ginsburg, Valve Software
- Rolando Olivares, Epic Games
- Dan Baker, Oxide Games
- Kyle Spagnoli, NVIDIA
- Jon Ashburn, LunarG
- Piers Daniell, NVIDIA

Description

Due to the nature of the Vulkan interface, there is very little error information available to the developer and application. By using the `VK_EXT_debug_utils` extension, developers **can** obtain more information. When combined with validation layers, even more detailed feedback on the application's use of Vulkan will be provided.

This extension provides the following capabilities:

- The ability to create a debug messenger which will pass along debug messages to an application supplied callback.
- The ability to identify specific Vulkan objects using a name or tag to improve tracking.
- The ability to identify specific sections within a `VkQueue` or `VkCommandBuffer` using labels to aid organization and offline analysis in external tools.

The main difference between this extension and `VK_EXT_debug_report` and `VK_EXT_debug_marker` is that those extensions use `VkDebugReportObjectTypeEXT` to identify objects. This extension uses the core VkObjectType in place of `VkDebugReportObjectTypeEXT`. The primary reason for this move is that no future object type handle enumeration values will be added to `VkDebugReportObjectTypeEXT` since the creation of VkObjectType.

In addition, this extension combines the functionality of both `VK_EXT_debug_report` and `VK_EXT_debug_marker` by allowing object name and debug markers (now called labels) to be returned to the application's callback function. This should assist in clarifying the details of a debug message including: what objects are involved and potentially which location within a VkQueue or VkCommandBuffer the message occurred.

New Object Types

- VkDebugUtilsMessengerEXT

New Commands

- vkCmdBeginDebugUtilsLabelEXT
- vkCmdEndDebugUtilsLabelEXT
- vkCmdInsertDebugUtilsLabelEXT
- vkCreateDebugUtilsMessengerEXT
- vkDestroyDebugUtilsMessengerEXT
- vkQueueBeginDebugUtilsLabelEXT
- vkQueueEndDebugUtilsLabelEXT
- vkQueueInsertDebugUtilsLabelEXT
- vkSetDebugUtilsObjectNameEXT
- vkSetDebugUtilsObjectTagEXT
- vkSubmitDebugUtilsMessageEXT

New Structures

- VkDebugUtilsLabelEXT
- VkDebugUtilsMessengerCallbackDataEXT
- VkDebugUtilsObjectTagInfoEXT
- Extending VkInstanceCreateInfo:

    - VkDebugUtilsMessengerCreateInfoEXT
- Extending VkPipelineShaderStageCreateInfo:
    - VkDebugUtilsObjectNameInfoEXT

New Function Pointers

- PFN_vkDebugUtilsMessengerCallbackEXT

New Enums

- VkDebugUtilsMessageSeverityFlagBitsEXT
- VkDebugUtilsMessageTypeFlagBitsEXT

New Bitmasks

- VkDebugUtilsMessageSeverityFlagsEXT
- VkDebugUtilsMessageTypeFlagsEXT
- VkDebugUtilsMessengerCallbackDataFlagsEXT
- VkDebugUtilsMessengerCreateFlagsEXT

New Enum Constants

- `VK_EXT_DEBUG_UTILS_EXTENSION_NAME`
- `VK_EXT_DEBUG_UTILS_SPEC_VERSION`
- Extending VkObjectType:
    - `VK_OBJECT_TYPE_DEBUG_UTILS_MESSENGER_EXT`
- Extending VkStructureType:
    - `VK_STRUCTURE_TYPE_DEBUG_UTILS_LABEL_EXT`
    - `VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CALLBACK_DATA_EXT`
    - `VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT`
    - `VK_STRUCTURE_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT`
    - `VK_STRUCTURE_TYPE_DEBUG_UTILS_OBJECT_TAG_INFO_EXT`

Examples

Example 1

`VK_EXT_debug_utils` allows an application to register multiple callbacks with any Vulkan component wishing to report debug information. Some callbacks may log the information to a file, others may cause a debug break point or other application-defined behavior. An application **can** register callbacks even when no validation layers are enabled, but they will only be called for loader and, if implemented, driver events.

To capture events that occur while creating or destroying an instance an application **can** link a

VkDebugUtilsMessengerCreateInfoEXT structure to the pNext element of the VkInstanceCreateInfo structure given to vkCreateInstance.

Example uses: Create three callback objects. One will log errors and warnings to the debug console using Windows OutputDebugString. The second will cause the debugger to break at that callback when an error happens and the third will log warnings to stdout.

```
    extern VkInstance instance;
    VkResult res;
    VkDebugUtilsMessengerEXT cb1, cb2, cb3;

    // Must call extension functions through a function pointer:
    PFN_vkCreateDebugUtilsMessengerEXT pfnCreateDebugUtilsMessengerEXT =
(PFN_vkCreateDebugUtilsMessengerEXT)vkGetInstanceProcAddr(instance,
"vkCreateDebugUtilsMessengerEXT");
    PFN_vkDestroyDebugUtilsMessengerEXT pfnDestroyDebugUtilsMessengerEXT =
(PFN_vkDestroyDebugUtilsMessengerEXT)vkGetInstanceProcAddr(instance,
"vkDestroyDebugUtilsMessengerEXT");

    VkDebugUtilsMessengerCreateInfoEXT callback1 = {
        .sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT,
        .pNext = NULL,
        .flags = 0,
        .messageSeverity = VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT |
                           VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT,
        .messageType= VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT |
                      VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT,
        .pfnUserCallback = myOutputDebugString,
        .pUserData = NULL
    };
    res = pfnCreateDebugUtilsMessengerEXT(instance, &callback1, NULL, &cb1);
    if (res != VK_SUCCESS) {
       // Do error handling for VK_ERROR_OUT_OF_MEMORY
    }

    callback1.messageSeverity = VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT;
    callback1.pfnUserCallback = myDebugBreak;
    callback1.pUserData = NULL;
    res = pfnCreateDebugUtilsMessengerEXT(instance, &callback1, NULL, &cb2);
    if (res != VK_SUCCESS) {
       // Do error handling for VK_ERROR_OUT_OF_MEMORY
    }

    VkDebugUtilsMessengerCreateInfoEXT callback3 = {
        .sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT,
        .pNext = NULL,
        .flags = 0,
        .messageSeverity = VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT,
        .messageType = VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT |
                       VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT,
```

```
        .pfnUserCallback = mystdOutLogger,
        .pUserData = NULL
    };
    res = pfnCreateDebugUtilsMessengerEXT(instance, &callback3, NULL, &cb3);
    if (res != VK_SUCCESS) {
       // Do error handling for VK_ERROR_OUT_OF_MEMORY
    }

    ...

    // Remove callbacks when cleaning up
    pfnDestroyDebugUtilsMessengerEXT(instance, cb1, NULL);
    pfnDestroyDebugUtilsMessengerEXT(instance, cb2, NULL);
    pfnDestroyDebugUtilsMessengerEXT(instance, cb3, NULL);
```

Example 2

Associate a name with an image, for easier debugging in external tools or with validation layers that can print a friendly name when referring to objects in error messages.

```
    extern VkInstance instance;
    extern VkDevice device;
    extern VkImage image;

    // Must call extension functions through a function pointer:
    PFN_vkSetDebugUtilsObjectNameEXT pfnSetDebugUtilsObjectNameEXT =
(PFN_vkSetDebugUtilsObjectNameEXT)vkGetInstanceProcAddr(instance,
"vkSetDebugUtilsObjectNameEXT");

    // Set a name on the image
    const VkDebugUtilsObjectNameInfoEXT imageNameInfo =
    {
        .sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT,
        .pNext = NULL,
        .objectType = VK_OBJECT_TYPE_IMAGE,
        .objectHandle = (uint64_t)image,
        .pObjectName = "Brick Diffuse Texture",
    };

    pfnSetDebugUtilsObjectNameEXT(device, &imageNameInfo);

    // A subsequent error might print:
    //   Image 'Brick Diffuse Texture' (0xc0dec0dedeadbeef) is used in a
    //   command buffer with no memory bound to it.
```

Example 3

Annotating regions of a workload with naming information so that offline analysis tools can display a more usable visualization of the commands submitted.

```
    extern VkInstance instance;
    extern VkCommandBuffer commandBuffer;

    // Must call extension functions through a function pointer:
    PFN_vkQueueBeginDebugUtilsLabelEXT pfnQueueBeginDebugUtilsLabelEXT =
(PFN_vkQueueBeginDebugUtilsLabelEXT)vkGetInstanceProcAddr(instance,
"vkQueueBeginDebugUtilsLabelEXT");
    PFN_vkQueueEndDebugUtilsLabelEXT pfnQueueEndDebugUtilsLabelEXT =
(PFN_vkQueueEndDebugUtilsLabelEXT)vkGetInstanceProcAddr(instance,
"vkQueueEndDebugUtilsLabelEXT");
    PFN_vkCmdBeginDebugUtilsLabelEXT pfnCmdBeginDebugUtilsLabelEXT =
(PFN_vkCmdBeginDebugUtilsLabelEXT)vkGetInstanceProcAddr(instance,
"vkCmdBeginDebugUtilsLabelEXT");
    PFN_vkCmdEndDebugUtilsLabelEXT pfnCmdEndDebugUtilsLabelEXT =
(PFN_vkCmdEndDebugUtilsLabelEXT)vkGetInstanceProcAddr(instance,
"vkCmdEndDebugUtilsLabelEXT");
    PFN_vkCmdInsertDebugUtilsLabelEXT pfnCmdInsertDebugUtilsLabelEXT =
(PFN_vkCmdInsertDebugUtilsLabelEXT)vkGetInstanceProcAddr(instance,
"vkCmdInsertDebugUtilsLabelEXT");

    // Describe the area being rendered
    const VkDebugUtilsLabelEXT houseLabel =
    {
        .sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_LABEL_EXT,
        .pNext = NULL,
        .pLabelName = "Brick House",
        .color = { 1.0f, 0.0f, 0.0f, 1.0f },
    };

    // Start an annotated group of calls under the 'Brick House' name
    pfnCmdBeginDebugUtilsLabelEXT(commandBuffer, &houseLabel);
    {
        // A mutable structure for each part being rendered
        VkDebugUtilsLabelEXT housePartLabel =
        {
            .sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_LABEL_EXT,
            .pNext = NULL,
            .pLabelName = NULL,
            .color = { 0.0f, 0.0f, 0.0f, 0.0f },
        };

        // Set the name and insert the marker
        housePartLabel.pLabelName = "Walls";
        pfnCmdInsertDebugUtilsLabelEXT(commandBuffer, &housePartLabel);

        // Insert the drawcall for the walls
        vkCmdDrawIndexed(commandBuffer, 1000, 1, 0, 0, 0);

        // Insert a recursive region for two sets of windows
        housePartLabel.pLabelName = "Windows";
```

```
        pfnCmdBeginDebugUtilsLabelEXT(commandBuffer, &housePartLabel);
        {
            vkCmdDrawIndexed(commandBuffer, 75, 6, 1000, 0, 0);
            vkCmdDrawIndexed(commandBuffer, 100, 2, 1450, 0, 0);
        }
        pfnCmdEndDebugUtilsLabelEXT(commandBuffer);

        housePartLabel.pLabelName = "Front Door";
        pfnCmdInsertDebugUtilsLabelEXT(commandBuffer, &housePartLabel);

        vkCmdDrawIndexed(commandBuffer, 350, 1, 1650, 0, 0);

        housePartLabel.pLabelName = "Roof";
        pfnCmdInsertDebugUtilsLabelEXT(commandBuffer, &housePartLabel);

        vkCmdDrawIndexed(commandBuffer, 500, 1, 2000, 0, 0);
    }
    // End the house annotation started above
    pfnCmdEndDebugUtilsLabelEXT(commandBuffer);

    // Do other work

    vkEndCommandBuffer(commandBuffer);

    // Describe the queue being used
    const VkDebugUtilsLabelEXT queueLabel =
    {
        .sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_LABEL_EXT,
        .pNext = NULL,
        .pLabelName = "Main Render Work",
        .color = { 0.0f, 1.0f, 0.0f, 1.0f },
    };

    // Identify the queue label region
    pfnQueueBeginDebugUtilsLabelEXT(queue, &queueLabel);

    // Submit the work for the main render thread
    const VkCommandBuffer cmd_bufs[] = {commandBuffer};
    VkSubmitInfo submit_info =
    {
        .sType = VK_STRUCTURE_TYPE_SUBMIT_INFO,
        .pNext = NULL,
        .waitSemaphoreCount = 0,
        .pWaitSemaphores = NULL,
        .pWaitDstStageMask = NULL,
        .commandBufferCount = 1,
        .pCommandBuffers = cmd_bufs,
        .signalSemaphoreCount = 0,
        .pSignalSemaphores = NULL
    };
    vkQueueSubmit(queue, 1, &submit_info, fence);
```

```
    // End the queue label region
    pfnQueueEndDebugUtilsLabelEXT(queue);
```

Issues

1) Should we just name this extension `VK_EXT_debug_report2`

RESOLVED: No. There is enough additional changes to the structures to break backwards compatibility. So, a new name was decided that would not indicate any interaction with the previous extension.

2) Will validation layers immediately support all the new features.

RESOLVED: Not immediately. As one can imagine, there is a lot of work involved with converting the validation layer logging over to the new functionality. Basic logging, as seen in the origin `VK_EXT_debug_report` extension will be made available immediately. However, adding the labels and object names will take time. Since the priority for Khronos at this time is to continue focusing on Valid Usage statements, it may take a while before the new functionality is fully exposed.

3) If the validation layers will not expose the new functionality immediately, then what is the point of this extension?

RESOLVED: We needed a replacement for `VK_EXT_debug_report` because the `VkDebugReportObjectTypeEXT` enumeration will no longer be updated and any new objects will need to be debugged using the new functionality provided by this extension.

4) Should this extension be split into two separate parts (1 extension that is an instance extension providing the callback functionality, and another device extension providing the general debug marker and annotation functionality)?

RESOLVED: No, the functionality for this extension is too closely related. If we did split up the extension, where would the structures and enums live, and how would you define that the device behavior in the instance extension is really only valid if the device extension is enabled, and the functionality is passed in. It is cleaner to just define this all as an instance extension, plus it allows the application to enable all debug functionality provided with one enable string during vkCreateInstance.

Version History

- Revision 1, 2017-09-14 (Mark Young and all listed Contributors)
    - Initial draft, based on `VK_EXT_debug_report` and `VK_EXT_debug_marker` in addition to previous feedback supplied from various companies including Valve, Epic, and Oxide games.
- Revision 2, 2020-04-03 (Mark Young and Piers Daniell)
    - Updated to allow either `NULL` or an empty string to be passed in for `pObjectName` in `VkDebugUtilsObjectNameInfoEXT`, because the loader and various drivers support `NULL` already.

VK_EXT_depth_bias_control

Name String

`VK_EXT_depth_bias_control`

Extension Type

Device extension

Registered Extension Number

284

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- D3D support

Contact

- Joshua Ashton [GitHub]Joshua-Ashton

Extension Proposal

VK_EXT_depth_bias_control

Other Extension Metadata

Last Modified Date

2023-02-15

IP Status

No known IP claims.

Contributors

- Joshua Ashton, VALVE
- Hans-Kristian Arntzen, VALVE
- Mike Blumenkrantz, VALVE
- Georg Lehmann, VALVE
- Piers Daniell, NVIDIA
- Lionel Landwerlin, INTEL
- Tobias Hector, AMD

- Ricardo Garcia, IGALIA
- Jan-Harald Fredriksen, ARM
- Shahbaz Youssefi, GOOGLE
- Tom Olson, ARM

Description

This extension adds a new structure, `VkDepthBiasRepresentationInfoEXT`, that can be added to a `pNext` chain of `VkPipelineRasterizationStateCreateInfo` and allows setting the scaling and representation of depth bias for a pipeline.

This state can also be set dynamically by using the new structure mentioned above in combination with the new `vkCmdSetDepthBias2EXT` command.

New Commands

- vkCmdSetDepthBias2EXT

New Structures

- VkDepthBiasInfoEXT
- Extending VkDepthBiasInfoEXT, VkPipelineRasterizationStateCreateInfo:
  - VkDepthBiasRepresentationInfoEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceDepthBiasControlFeaturesEXT

New Enums

- VkDepthBiasRepresentationEXT

New Enum Constants

- `VK_EXT_DEPTH_BIAS_CONTROL_EXTENSION_NAME`
- `VK_EXT_DEPTH_BIAS_CONTROL_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_DEPTH_BIAS_INFO_EXT`
  - `VK_STRUCTURE_TYPE_DEPTH_BIAS_REPRESENTATION_INFO_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_BIAS_CONTROL_FEATURES_EXT`

Version History

- Revision 1, 2022-09-22 (Joshua Ashton)
  - Initial draft.

VK_EXT_depth_clamp_control

Name String

`VK_EXT_depth_clamp_control`

Extension Type

Device extension

Registered Extension Number

583

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Jules Blok [GitHub]jules

Extension Proposal

VK_EXT_depth_clamp_control

Other Extension Metadata

Last Modified Date

2024-07-15

Contributors

- Jules Blok, Independent

Description

This extension allows the application to control the viewport depth clamp range separately from the viewport `minDepth` and `maxDepth`. This gives the ability for the application to restrict depth values to an application-defined range rather than the viewport depth range.

It can be used to set a smaller or larger clamping range than the viewport depth range without affecting the depth mapping of the viewport transform.

New Commands

- vkCmdSetDepthClampRangeEXT

New Structures

- VkDepthClampRangeEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceDepthClampControlFeaturesEXT
- Extending VkPipelineViewportStateCreateInfo:
  - VkPipelineViewportDepthClampControlCreateInfoEXT

New Enums

- VkDepthClampModeEXT

New Enum Constants

- `VK_EXT_DEPTH_CLAMP_CONTROL_EXTENSION_NAME`
- `VK_EXT_DEPTH_CLAMP_CONTROL_SPEC_VERSION`
- Extending VkDynamicState:
  - `VK_DYNAMIC_STATE_DEPTH_CLAMP_RANGE_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLAMP_CONTROL_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_DEPTH_CLAMP_CONTROL_CREATE_INFO_EXT`

Issues

1) Should the depth clamp range be a per-viewport parameter?

RESOLVED: No. Because the depth clamp range was previously defined to be equal to the viewport depth range, conformant runtimes are already handling the depth clamp range as a per-viewport parameter. However because of complexities from interactions with multiple viewports a per-viewport clamp range is left to a future extensions if a use case arises.

2) Should this pipeline state be dynamic?

RESOLVED: Yes. Since the viewport depth range can already be a dynamic state conformant runtimes are already able to handle the depth clamp range as a dynamic state.

3) Can the depth clamp range be ignored when depth clamping is disabled?

RESOLVED: Yes. This extension overrides the clamping range used only when depth clamping is enabled. The alternative would be highly unintuitive. As a consequence the VK_EXT_depth_clip_enable extension is required if depth clipping is desired in combination with this extension.

Version History

- Revision 1, 2024-02-13 (Jules Blok)

- Initial draft

VK_EXT_depth_clip_control

Name String

`VK_EXT_depth_clip_control`

Extension Type

Device extension

Registered Extension Number

356

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- OpenGL / ES support

Contact

- Shahbaz Youssefi [GitHub]syoussefi

Other Extension Metadata

Last Modified Date

2021-11-09

Contributors

- Spencer Fricke, Samsung Electronics
- Shahbaz Youssefi, Google
- Ralph Potter, Samsung Electronics

Description

This extension allows the application to use the OpenGL depth range in NDC, i.e. with depth in range [-1, 1], as opposed to Vulkan's default of [0, 1]. The purpose of this extension is to allow efficient layering of OpenGL over Vulkan, by avoiding emulation in the pre-rasterization shader stages. This emulation, which effectively duplicates gl_Position but with a different depth value, costs ALU and consumes shader output components that the implementation may not have to spare to meet OpenGL minimum requirements.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceDepthClipControlFeaturesEXT
- Extending VkPipelineViewportStateCreateInfo:
  - VkPipelineViewportDepthClipControlCreateInfoEXT

New Enum Constants

- `VK_EXT_DEPTH_CLIP_CONTROL_EXTENSION_NAME`
- `VK_EXT_DEPTH_CLIP_CONTROL_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLIP_CONTROL_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_DEPTH_CLIP_CONTROL_CREATE_INFO_EXT`

Issues

1) Should this extension include an origin control option to match GL_LOWER_LEFT found in ARB_clip_control?

RESOLVED: No. The fix for porting over the origin is a simple y-axis flip. The depth clip control is a much harder problem to solve than what this extension is aimed to solve. Adding an equivalent to GL_LOWER_LEFT would require more testing.

2) Should this pipeline state be dynamic?

RESOLVED: Yes. The purpose of this extension is to emulate the OpenGL depth range, which is expected to be globally fixed (in case of OpenGL ES) or very infrequently changed (with `glClipControl` in OpenGL).

3) Should the control provided in this extension be an enum that could be extended in the future?

RESOLVED: No. It is highly unlikely that the depth range is changed to anything other than [0, 1] in the future. Should that happen a new extension will be required to extend such an enum, and that extension might as well add a new struct to chain to VkPipelineViewportStateCreateInfo::`pNext` instead.

Version History

- Revision 0, 2020-10-01 (Spencer Fricke)
  - Internal revisions
- Revision 1, 2020-11-26 (Shahbaz Youssefi)
  - Language fixes

VK_EXT_depth_clip_enable

Name String

`VK_EXT_depth_clip_enable`

Extension Type

Device extension

Registered Extension Number

103

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- D3D support

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2018-12-20

Contributors

- Daniel Rakos, AMD
- Henri Verbeet, CodeWeavers
- Jeff Bolz, NVIDIA
- Philip Rebohle, DXVK
- Tobias Hector, AMD

Description

This extension allows the depth clipping operation, that is normally implicitly controlled by VkPipelineRasterizationStateCreateInfo::`depthClampEnable`, to instead be controlled explicitly by VkPipelineRasterizationDepthClipStateCreateInfoEXT::`depthClipEnable`.

This is useful for translating DX content which assumes depth clamping is always enabled, but depth clip can be controlled by the DepthClipEnable rasterization state (D3D12_RASTERIZER_DESC).

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceDepthClipEnableFeaturesEXT
- Extending VkPipelineRasterizationStateCreateInfo:
  - VkPipelineRasterizationDepthClipStateCreateInfoEXT

New Bitmasks

- VkPipelineRasterizationDepthClipStateCreateFlagsEXT

New Enum Constants

- VK_EXT_DEPTH_CLIP_ENABLE_EXTENSION_NAME
- VK_EXT_DEPTH_CLIP_ENABLE_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEPTH_CLIP_ENABLE_FEATURES_EXT
  - VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_DEPTH_CLIP_STATE_CREATE_INFO_EXT

Version History

- Revision 1, 2018-12-20 (Piers Daniell)
  - Internal revisions

VK_EXT_depth_range_unrestricted

Name String

VK_EXT_depth_range_unrestricted

Extension Type

Device extension

Registered Extension Number

14

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2017-06-22

Contributors

- Daniel Koch, NVIDIA
- Jeff Bolz, NVIDIA

Description

This extension removes the VkViewport `minDepth` and `maxDepth` restrictions that the values must be between `0.0` and `1.0`, inclusive. It also removes the same restriction on VkPipelineDepthStencilStateCreateInfo `minDepthBounds` and `maxDepthBounds`. Finally it removes the restriction on the `depth` value in VkClearDepthStencilValue.

New Enum Constants

- `VK_EXT_DEPTH_RANGE_UNRESTRICTED_EXTENSION_NAME`
- `VK_EXT_DEPTH_RANGE_UNRESTRICTED_SPEC_VERSION`

Issues

1) How do VkViewport `minDepth` and `maxDepth` values outside of the `0.0` to `1.0` range interact with Primitive Clipping?

RESOLVED: The behavior described in Primitive Clipping still applies. If depth clamping is disabled the depth values are still clipped to $0 \le z_c \le w_c$ before the viewport transform. If depth clamping is enabled the above equation is ignored and the depth values are instead clamped to the VkViewport `minDepth` and `maxDepth` values, which in the case of this extension can be outside of the `0.0` to `1.0` range.

2) What happens if a resulting depth fragment is outside of the `0.0` to `1.0` range and the depth buffer is fixed-point rather than floating-point?

RESOLVED: This situation can also arise without this extension (when fragment shaders replace depth values, for example), and this extension does not change the behavior, which is defined in the Depth Test section of the Fragment Operations chapter.

Version History

- Revision 1, 2017-06-22 (Piers Daniell)
  - Internal revisions

VK_EXT_device_generated_commands

Name String

`VK_EXT_device_generated_commands`

Extension Type

Device extension

Registered Extension Number

573

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_buffer_device_address
and
VK_KHR_maintenance5

API Interactions

- Interacts with VK_EXT_shader_object

Contact

- Mike Blumenkrantz [GitHub]zmike

Extension Proposal

VK_EXT_device_generated_commands

Other Extension Metadata

Last Modified Date

2024-02-23

Interactions and External Dependencies

- This extension requires Vulkan 1.1
- This extension requires `VK_EXT_buffer_device_address` or `VK_KHR_buffer_device_address` or Vulkan 1.2 for the ability to bind vertex and index buffers on the device.
- This extension requires `VK_KHR_maintenance5` for the ability to use VkPipelineCreateFlags2KHR.
- This extension interacts with `VK_NV_mesh_shader`. If the latter extension is not supported, remove the command tokens to initiate NV mesh tasks drawing in this extension.
- This extension interacts with `VK_EXT_mesh_shader`. If the latter extension is not supported, remove the command tokens to initiate EXT mesh tasks drawing in this extension.
- This extension interacts with `VK_KHR_ray_tracing_pipeline`. If the latter extension is not supported, remove the command tokens to initiate ray tracing in this extension.
- This extension interacts with `VK_EXT_shader_object`. If the latter extension is not supported, remove references to shader objects in this extension.

Contributors

- Mike Blumenkrantz, VALVE
- Hans-Kristian Arntzen, VALVE
- Jan-Harald Fredriksen, ARM
- Spencer Fricke, LunarG
- Ricardo Garcia, Igalia
- Tobias Hector, AMD
- Baldur Karlsson, VALVE
- Christoph Kubisch, NVIDIA
- Lionel Landwerlin, INTEL
- Jon Leech, Khronos
- Ting Wei, ARM
- Ken Shanyi Zhang, AMD
- Faith Ekstrand, Collabora
- Vikram Kushwaha, NVIDIA
- Connor Abbott, VALVE
- Samuel Pitoiset, VALVE

Description

This extension allows the device to generate a number of commands for command buffers. It provides a subset of functionality from both `VK_NV_device_generated_commands` and `VK_NV_device_generated_commands_compute` as well as some new features.

When rendering a large number of objects, the device can be leveraged to implement a number of critical functions, like updating matrices, or implementing occlusion culling, frustum culling, front to back sorting, etc. Implementing those on the device does not require any special extension, since an application is free to define its own data structures, and just process them using shaders.

To render objects which have been processed on the device, Vulkan has several ways to perform indirect rendering, from the most basic `vkCmdDrawIndirect` with one indirect draw to `vkCmdDrawIndirectCount` which supports multiple indirect draws batched together, with a way to determine number of draws at device execution time.

However, if rendering state needs to change between the indirect draws, then unextended Vulkan forces the application to speculatively record a prohibitive number of redundant indirect commands covering all possible state combinations - which could end up processing nothing after culling - or read back the processed stream and issue graphics command from the host. For very large scenes, the synchronization overhead and cost to generate the command buffer can become the bottleneck. This extension allows an application to generate a device side stream of state changes and commands, and convert it efficiently into a command buffer without having to read it back to the host.

Furthermore, it allows incremental changes to such command buffers by manipulating only partial sections of a command stream — for example pipeline and shader object bindings. Unextended Vulkan requires re-creation of entire command buffers in such a scenario, or updates synchronized on the host.

The intended usage for this extension is for the application to:

- create `VkBuffer` objects and retrieve physical addresses from them via vkGetBufferDeviceAddressKHR
- create a `VkIndirectExecutionSetEXT` for the ability to change shaders on the device.
- create a VkIndirectCommandsLayoutEXT, which lists the VkIndirectCommandsTokenTypeEXT it wants to dynamically execute as an atomic command sequence. This step likely involves some internal device code compilation, since the intent is for the GPU to generate the command buffer based on the layout.
- fill the input stream buffers with the data for each of the inputs it needs. Each input is an array that will be filled with token-dependent data.
- set up a preprocess `VkBuffer` that uses memory according to the information retrieved via vkGetGeneratedCommandsMemoryRequirementsEXT.
- optionally preprocess the generated content using vkCmdPreprocessGeneratedCommandsEXT, for example on an asynchronous compute queue, or for the purpose of re-using the data in multiple executions.
- call vkCmdExecuteGeneratedCommandsEXT to create and execute the actual device commands for all sequences based on the inputs provided.

For each draw in a sequence, the following can be specified:

- a number of vertex buffer bindings
- a different index buffer, with an optional dynamic offset and index type
- a number of different push constants
- updates to bound shader stages

For each dispatch in a sequence, the following can be specified:

- a number of different push constants
- updates to bound shader stages

For each trace rays in a sequence, the following can be specified:

- a number of different push constants
- updates to bound shader stages

While the GPU can be faster than a CPU to generate the commands, it will not happen asynchronously to the device, therefore the primary use case is generating “less” total work (occlusion culling, classification to use specialized shaders, etc.).

New Object Types

- VkIndirectCommandsLayoutEXT
- VkIndirectExecutionSetEXT

New Commands

- vkCmdExecuteGeneratedCommandsEXT
- vkCmdPreprocessGeneratedCommandsEXT
- vkCreateIndirectCommandsLayoutEXT
- vkCreateIndirectExecutionSetEXT
- vkDestroyIndirectCommandsLayoutEXT
- vkDestroyIndirectExecutionSetEXT
- vkGetGeneratedCommandsMemoryRequirementsEXT
- vkUpdateIndirectExecutionSetPipelineEXT
- vkUpdateIndirectExecutionSetShaderEXT

New Structures

- VkBindIndexBufferIndirectCommandEXT
- VkBindVertexBufferIndirectCommandEXT
- VkDrawIndirectCountIndirectCommandEXT
- VkGeneratedCommandsInfoEXT
- VkGeneratedCommandsMemoryRequirementsInfoEXT
- VkIndirectCommandsExecutionSetTokenEXT
- VkIndirectCommandsIndexBufferTokenEXT
- VkIndirectCommandsLayoutCreateInfoEXT
- VkIndirectCommandsLayoutTokenEXT
- VkIndirectCommandsPushConstantTokenEXT
- VkIndirectCommandsVertexBufferTokenEXT
- VkIndirectExecutionSetCreateInfoEXT
- VkIndirectExecutionSetPipelineInfoEXT
- VkIndirectExecutionSetShaderInfoEXT
- VkIndirectExecutionSetShaderLayoutInfoEXT
- VkWriteIndirectExecutionSetPipelineEXT
- Extending VkGeneratedCommandsInfoEXT, VkGeneratedCommandsMemoryRequirementsInfoEXT:
  - VkGeneratedCommandsPipelineInfoEXT

    - VkGeneratedCommandsShaderInfoEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
    - VkPhysicalDeviceDeviceGeneratedCommandsFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
    - VkPhysicalDeviceDeviceGeneratedCommandsPropertiesEXT

If VK_EXT_shader_object is supported:

- VkWriteIndirectExecutionSetShaderEXT

New Unions

- VkIndirectCommandsTokenDataEXT
- VkIndirectExecutionSetInfoEXT

New Enums

- VkIndirectCommandsInputModeFlagBitsEXT
- VkIndirectCommandsLayoutUsageFlagBitsEXT
- VkIndirectCommandsTokenTypeEXT
- VkIndirectExecutionSetInfoTypeEXT

New Bitmasks

- VkIndirectCommandsInputModeFlagsEXT
- VkIndirectCommandsLayoutUsageFlagsEXT

New Enum Constants

- `VK_EXT_DEVICE_GENERATED_COMMANDS_EXTENSION_NAME`
- `VK_EXT_DEVICE_GENERATED_COMMANDS_SPEC_VERSION`
- Extending VkAccessFlagBits:
    - `VK_ACCESS_COMMAND_PREPROCESS_READ_BIT_EXT`
    - `VK_ACCESS_COMMAND_PREPROCESS_WRITE_BIT_EXT`
- Extending VkBufferUsageFlagBits2KHR:
    - `VK_BUFFER_USAGE_2_PREPROCESS_BUFFER_BIT_EXT`
- Extending VkObjectType:
    - `VK_OBJECT_TYPE_INDIRECT_COMMANDS_LAYOUT_EXT`
    - `VK_OBJECT_TYPE_INDIRECT_EXECUTION_SET_EXT`
- Extending VkPipelineCreateFlagBits2KHR:
    - `VK_PIPELINE_CREATE_2_INDIRECT_BINDABLE_BIT_EXT`

- Extending VkPipelineStageFlagBits:
  - `VK_PIPELINE_STAGE_COMMAND_PREPROCESS_BIT_EXT`
- Extending VkShaderCreateFlagBitsEXT:
  - `VK_SHADER_CREATE_INDIRECT_BINDABLE_BIT_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_GENERATED_COMMANDS_INFO_EXT`
  - `VK_STRUCTURE_TYPE_GENERATED_COMMANDS_MEMORY_REQUIREMENTS_INFO_EXT`
  - `VK_STRUCTURE_TYPE_GENERATED_COMMANDS_PIPELINE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_GENERATED_COMMANDS_SHADER_INFO_EXT`
  - `VK_STRUCTURE_TYPE_INDIRECT_COMMANDS_LAYOUT_CREATE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_INDIRECT_COMMANDS_LAYOUT_TOKEN_EXT`
  - `VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_CREATE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_PIPELINE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_SHADER_INFO_EXT`
  - `VK_STRUCTURE_TYPE_INDIRECT_EXECUTION_SET_SHADER_LAYOUT_INFO_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_GENERATED_COMMANDS_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_GENERATED_COMMANDS_PROPERTIES_EXT`
  - `VK_STRUCTURE_TYPE_WRITE_INDIRECT_EXECUTION_SET_PIPELINE_EXT`
  - `VK_STRUCTURE_TYPE_WRITE_INDIRECT_EXECUTION_SET_SHADER_EXT`

Example Code

TODO

Version History

- Revision 1, 2024-02-23 (Mike Blumenkrantz)
  - Initial version

VK_EXT_device_memory_report

Name String

`VK_EXT_device_memory_report`

Extension Type

Device extension

Registered Extension Number

285

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- Developer tools

Contact

- Yiwei Zhang [GitHub]zhangyiwei

Other Extension Metadata

Last Modified Date

2021-01-06

IP Status

No known IP claims.

Contributors

- Yiwei Zhang, Google
- Jesse Hall, Google

Description

This device extension allows registration of device memory event callbacks upon device creation, so that applications or middleware can obtain detailed information about memory usage and how memory is associated with Vulkan objects. This extension exposes the actual underlying device memory usage, including allocations that are not normally visible to the application, such as memory consumed by vkCreateGraphicsPipelines. It is intended primarily for use by debug tooling rather than for production applications.

New Structures

- VkDeviceMemoryReportCallbackDataEXT
- Extending VkDeviceCreateInfo:
  - VkDeviceDeviceMemoryReportCreateInfoEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceDeviceMemoryReportFeaturesEXT

New Function Pointers

- PFN_vkDeviceMemoryReportCallbackEXT

New Enums

- VkDeviceMemoryReportEventTypeEXT

New Bitmasks

- VkDeviceMemoryReportFlagsEXT

New Enum Constants

- `VK_EXT_DEVICE_MEMORY_REPORT_EXTENSION_NAME`
- `VK_EXT_DEVICE_MEMORY_REPORT_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_DEVICE_DEVICE_MEMORY_REPORT_CREATE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_DEVICE_MEMORY_REPORT_CALLBACK_DATA_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DEVICE_MEMORY_REPORT_FEATURES_EXT`

Issues

1) Should this be better expressed as an extension to VK_EXT_debug_utils and its general-purpose messenger construct?

RESOLVED: No. The intended lifecycle is quite different. We want to make this extension tied to the device's lifecycle. Each ICD just handles its own implementation of this extension, and this extension will only be directly exposed from the ICD. So we can avoid the extra implementation complexity used to accommodate the flexibility of `VK_EXT_debug_utils` extension.

2) Can we extend and use the existing internal allocation callbacks instead of adding the new callback structure in this extension?

RESOLVED: No. Our memory reporting layer that combines this information with other memory information it collects directly (e.g. bindings of resources to VkDeviceMemory) would have to intercept all entry points that take a VkAllocationCallbacks parameter and inject its own `pfnInternalAllocation` and `pfnInternalFree`. That may be doable for the extensions we know about, but not for ones we do not. The proposal would work fine in the face of most unknown extensions. But even for ones we know about, since apps can provide a different set of callbacks and userdata and those can be retained by the driver and used later (esp. for pool object, but not just those), we would have to dynamically allocate the interception trampoline every time. That is getting to be an unreasonably large amount of complexity and (possibly) overhead.

We are interested in both alloc/free and import/unimport. The latter is fairly important for tracking (and avoiding double-counting) of swapchain images (still true with "native swapchains" based on external memory) and media/camera interop. Though we might be able to handle this with additional VkInternalAllocationType values, for import/export we do want to be able to tie this to

the external resource, which is one thing that the `memoryObjectId` is for.

The internal alloc/free callbacks are not extensible except via new VkInternalAllocationType values. The VkDeviceMemoryReportCallbackDataEXT in this extension is extensible. That was deliberate: there is a real possibility we will want to get extra information in the future. As one example, currently this reports only physical allocations, but we believe there are interesting cases for tracking how populated that VA region is.

The callbacks are clearly specified as only callable within the context of a call from the application into Vulkan. We believe there are some cases where drivers can allocate device memory asynchronously. This was one of the sticky issues that derailed the internal device memory allocation reporting design (which is essentially what this extension is trying to do) leading up to 1.0.

VkAllocationCallbacks is described in a section called “Host memory” and the intro to it is very explicitly about host memory. The other callbacks are all inherently about host memory. But this extension is very focused on device memory.

3) Should the callback be reporting which heap is used?

RESOLVED: Yes. It is important for non-UMA systems to have all the device memory allocations attributed to the corresponding device memory heaps. For internally-allocated device memory, `heapIndex` will always correspond to an advertised heap, rather than having a magic value indicating a non-advertised heap. Drivers can advertise heaps that do not have any corresponding memory types if they need to.

4) Should we use an array of callback for the layers to intercept instead of chaining multiple of the VkDeviceDeviceMemoryReportCreateInfoEXT structures in the `pNext` of VkDeviceCreateInfo?

RESOLVED No. The pointer to the VkDeviceDeviceMemoryReportCreateInfoEXT structure itself is const and you cannot just cast it away. Thus we cannot update the callback array inside the structure. In addition, we cannot drop this `pNext` chain either, so making a copy of this whole structure does not work either.

5) Should we track bulk allocations shared among multiple objects?

RESOLVED No. Take the shader heap as an example. Some implementations will let multiple VkPipeline objects share the same shader heap. We are not asking the implementation to report `VK_OBJECT_TYPE_PIPELINE` along with a VK_NULL_HANDLE for this bulk allocation. Instead, this bulk allocation is considered as a layer below what this extension is interested in. Later, when the actual VkPipeline objects are created by suballocating from the bulk allocation, we ask the implementation to report the valid handles of the VkPipeline objects along with the actual suballocated sizes and different `memoryObjectId`.

6) Can we require the callbacks to be always called in the same thread with the Vulkan commands?

RESOLVED No. Some implementations might choose to multiplex work from multiple application threads into a single backend thread and perform JIT allocations as a part of that flow. Since this behavior is theoretically legit, we cannot require the callbacks to be always called in the same thread with the Vulkan commands, and the note is to remind the applications to handle this case

properly.

7) Should we add an additional “allocation failed” event type with things like size and heap index reported?

RESOLVED Yes. This fits in well with the callback infrastructure added in this extension, and implementation touches the same code and has the same overheads as the rest of the extension. It could help debugging things like getting a `VK_ERROR_OUT_OF_HOST_MEMORY` error when ending a command buffer. Right now the allocation failure could have happened anywhere during recording, and a callback would be really useful to understand where and why.

Version History

- Revision 1, 2020-08-26 (Yiwei Zhang)
  - Initial version
- Revision 2, 2021-01-06 (Yiwei Zhang)
  - Minor description update

VK_EXT_direct_mode_display

Name String

`VK_EXT_direct_mode_display`

Extension Type

Instance extension

Registered Extension Number

89

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_display

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-12-13

IP Status

No known IP claims.

Contributors

- Pierre Boudier, NVIDIA
- James Jones, NVIDIA
- Damien Leone, NVIDIA
- Pierre-Loup Griffais, Valve
- Liam Middlebrook, NVIDIA

Description

This is extension, along with related platform extensions, allows applications to take exclusive control of displays associated with a native windowing system. This is especially useful for virtual reality applications that wish to hide HMDs (head mounted displays) from the native platform's display management system, desktop, and/or other applications.

New Commands

- vkReleaseDisplayEXT

New Enum Constants

- VK_EXT_DIRECT_MODE_DISPLAY_EXTENSION_NAME
- VK_EXT_DIRECT_MODE_DISPLAY_SPEC_VERSION

Issues

1) Should this extension and its related platform-specific extensions leverage VK_KHR_display, or provide separate equivalent interfaces.

RESOLVED: Use VK_KHR_display concepts and objects. VK_KHR_display can be used to enumerate all displays on the system, including those attached to/in use by a window system or native platform, but VK_KHR_display_swapchain will fail to create a swapchain on in-use displays. This extension and its platform-specific children will allow applications to grab in-use displays away from window systems and/or native platforms, allowing them to be used with VK_KHR_display_swapchain.

2) Are separate calls needed to acquire displays and enable direct mode?

RESOLVED: No, these operations happen in one combined command. Acquiring a display puts it into direct mode.

Version History

- Revision 1, 2016-12-13 (James Jones)
  - Initial draft

VK_EXT_discard_rectangles

Name String

`VK_EXT_discard_rectangles`

Extension Type

Device extension

Registered Extension Number

100

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2023-01-18

Interactions and External Dependencies

- Interacts with `VK_KHR_device_group`
- Interacts with Vulkan 1.1

Contributors

- Daniel Koch, NVIDIA
- Jeff Bolz, NVIDIA

Description

This extension provides additional orthogonally aligned “discard rectangles” specified in framebuffer-space coordinates that restrict rasterization of all points, lines and triangles.

From zero to an implementation-dependent limit (specified by `maxDiscardRectangles`) number of discard rectangles can be operational at once. When one or more discard rectangles are active, rasterized fragments can either survive if the fragment is within any of the operational discard rectangles (`VK_DISCARD_RECTANGLE_MODE_INCLUSIVE_EXT` mode) or be rejected if the fragment is within any of the operational discard rectangles (`VK_DISCARD_RECTANGLE_MODE_EXCLUSIVE_EXT` mode).

These discard rectangles operate orthogonally to the existing scissor test functionality. The discard

rectangles can be different for each physical device in a device group by specifying the device mask and setting discard rectangle dynamic state.

Version 2 of this extension introduces new dynamic states VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT and VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT, and the corresponding functions vkCmdSetDiscardRectangleEnableEXT and vkCmdSetDiscardRectangleModeEXT. Applications that use these dynamic states must ensure the implementation advertises at least specVersion 2 of this extension.

New Commands

- vkCmdSetDiscardRectangleEXT
- vkCmdSetDiscardRectangleEnableEXT
- vkCmdSetDiscardRectangleModeEXT

New Structures

- Extending VkGraphicsPipelineCreateInfo:
  - VkPipelineDiscardRectangleStateCreateInfoEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceDiscardRectanglePropertiesEXT

New Enums

- VkDiscardRectangleModeEXT

New Bitmasks

- VkPipelineDiscardRectangleStateCreateFlagsEXT

New Enum Constants

- VK_EXT_DISCARD_RECTANGLES_EXTENSION_NAME
- VK_EXT_DISCARD_RECTANGLES_SPEC_VERSION
- Extending VkDynamicState:
  - VK_DYNAMIC_STATE_DISCARD_RECTANGLE_ENABLE_EXT
  - VK_DYNAMIC_STATE_DISCARD_RECTANGLE_EXT
  - VK_DYNAMIC_STATE_DISCARD_RECTANGLE_MODE_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DISCARD_RECTANGLE_PROPERTIES_EXT
  - VK_STRUCTURE_TYPE_PIPELINE_DISCARD_RECTANGLE_STATE_CREATE_INFO_EXT

Version History

- Revision 2, 2023-01-18 (Piers Daniell)

    - Add dynamic states for discard rectangle enable/disable and mode.
- Revision 1, 2016-12-22 (Piers Daniell)
    - Internal revisions

VK_EXT_display_control

Name String

`VK_EXT_display_control`

Extension Type

Device extension

Registered Extension Number

92

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_EXT_display_surface_counter
and
VK_KHR_swapchain

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-12-13

IP Status

No known IP claims.

Contributors

- Pierre Boudier, NVIDIA
- James Jones, NVIDIA
- Damien Leone, NVIDIA
- Pierre-Loup Griffais, Valve
- Daniel Vetter, Intel

Description

This extension defines a set of utility functions for use with the VK_KHR_display and VK_KHR_display_swapchain extensions.

New Commands

- vkDisplayPowerControlEXT
- vkGetSwapchainCounterEXT
- vkRegisterDeviceEventEXT
- vkRegisterDisplayEventEXT

New Structures

- VkDeviceEventInfoEXT
- VkDisplayEventInfoEXT
- VkDisplayPowerInfoEXT
- Extending VkSwapchainCreateInfoKHR:
  - VkSwapchainCounterCreateInfoEXT

New Enums

- VkDeviceEventTypeEXT
- VkDisplayEventTypeEXT
- VkDisplayPowerStateEXT

New Enum Constants

- VK_EXT_DISPLAY_CONTROL_EXTENSION_NAME
- VK_EXT_DISPLAY_CONTROL_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_DEVICE_EVENT_INFO_EXT
  - VK_STRUCTURE_TYPE_DISPLAY_EVENT_INFO_EXT
  - VK_STRUCTURE_TYPE_DISPLAY_POWER_INFO_EXT
  - VK_STRUCTURE_TYPE_SWAPCHAIN_COUNTER_CREATE_INFO_EXT

Issues

1) Should this extension add an explicit “WaitForVsync” API or a fence signaled at vsync that the application can wait on?

RESOLVED: A fence. A separate API could later be provided that allows exporting the fence to a native object that could be inserted into standard run loops on POSIX and Windows systems.

2) Should callbacks be added for a vsync event, or in general to monitor events in Vulkan?

RESOLVED: No, fences should be used. Some events are generated by interrupts which are managed in the kernel. In order to use a callback provided by the application, drivers would need to have the userspace driver spawn threads that would wait on the kernel event, and hence the callbacks could be difficult for the application to synchronize with its other work given they would arrive on a foreign thread.

3) Should vblank or scanline events be exposed?

RESOLVED: Vblank events. Scanline events could be added by a separate extension, but the latency of processing an interrupt and waking up a userspace event is high enough that the accuracy of a scanline event would be rather low. Further, per-scanline interrupts are not supported by all hardware.

Version History

- Revision 1, 2016-12-13 (James Jones)
  - Initial draft

VK_EXT_display_surface_counter

Name String

`VK_EXT_display_surface_counter`

Extension Type

Instance extension

Registered Extension Number

91

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_display

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2016-12-13

IP Status

No known IP claims.

Contributors

- Pierre Boudier, NVIDIA
- James Jones, NVIDIA
- Damien Leone, NVIDIA
- Pierre-Loup Griffais, Valve
- Daniel Vetter, Intel

Description

This extension defines a vertical blanking period counter associated with display surfaces. It provides a mechanism to query support for such a counter from a VkSurfaceKHR object.

New Commands

- vkGetPhysicalDeviceSurfaceCapabilities2EXT

New Structures

- VkSurfaceCapabilities2EXT

New Enums

- VkSurfaceCounterFlagBitsEXT

New Bitmasks

- VkSurfaceCounterFlagsEXT

New Enum Constants

- `VK_EXT_DISPLAY_SURFACE_COUNTER_EXTENSION_NAME`
- `VK_EXT_DISPLAY_SURFACE_COUNTER_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES2_EXT`
  - `VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_2_EXT`

Version History

- Revision 1, 2016-12-13 (James Jones)
  - Initial draft

VK_EXT_dynamic_rendering_unused_attachments

Name String

`VK_EXT_dynamic_rendering_unused_attachments`

Extension Type

Device extension

Registered Extension Number

500

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
VK_KHR_dynamic_rendering
or
Vulkan Version 1.3

Contact

- Piers Daniell [GitHub]pdaniell-nv

Extension Proposal

VK_EXT_dynamic_rendering_unused_attachments

Other Extension Metadata

Last Modified Date

2023-05-22

IP Status

No known IP claims.

Contributors

- Daniel Story, Nintendo
- Hans-Kristian Arntzen, Valve
- Jan-Harald Fredriksen, Arm
- James Fitzpatrick, Imagination Technologies
- Pan Gao, Huawei Technologies
- Ricardo Garcia, Igalia

- Stu Smith, AMD

Description

This extension lifts some restrictions in the `VK_KHR_dynamic_rendering` extension to allow render pass instances and bound pipelines within those render pass instances to have an unused attachment specified in one but not the other. It also allows pipelines to use different formats in a render pass as long the attachment is NULL.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceDynamicRenderingUnusedAttachmentsFeaturesEXT

New Enum Constants

- `VK_EXT_DYNAMIC_RENDERING_UNUSED_ATTACHMENTS_EXTENSION_NAME`
- `VK_EXT_DYNAMIC_RENDERING_UNUSED_ATTACHMENTS_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DYNAMIC_RENDERING_UNUSED_ATTACHMENTS_FEATURES_EXT`

Issues

None.

Version History

- Revision 1, 2023-05-22 (Piers Daniell)
  - Internal revisions

VK_EXT_extended_dynamic_state3

Name String

`VK_EXT_extended_dynamic_state3`

Extension Type

Device extension

Registered Extension Number

456

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

API Interactions

- Interacts with VK_VERSION_1_1
- Interacts with VK_EXT_blend_operation_advanced
- Interacts with VK_EXT_conservative_rasterization
- Interacts with VK_EXT_depth_clip_control
- Interacts with VK_EXT_depth_clip_enable
- Interacts with VK_EXT_line_rasterization
- Interacts with VK_EXT_provoking_vertex
- Interacts with VK_EXT_sample_locations
- Interacts with VK_EXT_transform_feedback
- Interacts with VK_KHR_maintenance2
- Interacts with VK_NV_clip_space_w_scaling
- Interacts with VK_NV_coverage_reduction_mode
- Interacts with VK_NV_fragment_coverage_to_color
- Interacts with VK_NV_framebuffer_mixed_samples
- Interacts with VK_NV_representative_fragment_test
- Interacts with VK_NV_shading_rate_image
- Interacts with VK_NV_viewport_swizzle
- Interacts with VkPhysicalDeviceExtendedDynamicState3FeaturesEXT::extendedDynamicState3AlphaToOneEnable
- Interacts with VkPhysicalDeviceExtendedDynamicState3FeaturesEXT::extendedDynamicState3DepthClampEnable
- Interacts with VkPhysicalDeviceExtendedDynamicState3FeaturesEXT::extendedDynamicState3LogicOpEnable
- Interacts with VkPhysicalDeviceExtendedDynamicState3FeaturesEXT::extendedDynamicState3PolygonMode
- Interacts with VkPhysicalDeviceExtendedDynamicState3FeaturesEXT::extendedDynamicState3RasterizationStream
- Interacts with

VkPhysicalDeviceExtendedDynamicState3FeaturesEXT::extendedDynamicState3Tessellation DomainOrigin

Contact

- Piers Daniell [GitHub]pdaniell-nv

Extension Proposal

VK_EXT_extended_dynamic_state3

Other Extension Metadata

Last Modified Date

2022-09-02

IP Status

No known IP claims.

Contributors

- Daniel Story, Nintendo
- Jamie Madill, Google
- Jan-Harald Fredriksen, Arm
- Faith Ekstrand, Collabora
- Mike Blumenkrantz, Valve
- Ricardo Garcia, Igalia
- Samuel Pitoiset, Valve
- Shahbaz Youssefi, Google
- Stu Smith, AMD
- Tapani Pälli, Intel

Description

This extension adds almost all of the remaining pipeline state as dynamic state to help applications further reduce the number of monolithic pipelines they need to create and bind.

New Commands

- vkCmdSetAlphaToCoverageEnableEXT
- vkCmdSetAlphaToOneEnableEXT
- vkCmdSetColorBlendEnableEXT
- vkCmdSetColorBlendEquationEXT
- vkCmdSetColorWriteMaskEXT
- vkCmdSetDepthClampEnableEXT
- vkCmdSetLogicOpEnableEXT

- vkCmdSetPolygonModeEXT
- vkCmdSetRasterizationSamplesEXT
- vkCmdSetSampleMaskEXT

If VK_EXT_blend_operation_advanced is supported:

- vkCmdSetColorBlendAdvancedEXT

If VK_EXT_conservative_rasterization is supported:

- vkCmdSetConservativeRasterizationModeEXT
- vkCmdSetExtraPrimitiveOverestimationSizeEXT

If VK_EXT_depth_clip_control is supported:

- vkCmdSetDepthClipNegativeOneToOneEXT

If VK_EXT_depth_clip_enable is supported:

- vkCmdSetDepthClipEnableEXT

If `VK_EXT_line_rasterization` is supported:

- vkCmdSetLineRasterizationModeEXT
- vkCmdSetLineStippleEnableEXT

If VK_EXT_provoking_vertex is supported:

- vkCmdSetProvokingVertexModeEXT

If VK_EXT_sample_locations is supported:

- vkCmdSetSampleLocationsEnableEXT

If VK_EXT_transform_feedback is supported:

- vkCmdSetRasterizationStreamEXT

If VK_KHR_maintenance2 or Vulkan Version 1.1 is supported:

- vkCmdSetTessellationDomainOriginEXT

If `VK_NV_clip_space_w_scaling` is supported:

- vkCmdSetViewportWScalingEnableNV

If `VK_NV_coverage_reduction_mode` is supported:

- vkCmdSetCoverageReductionModeNV

If `VK_NV_fragment_coverage_to_color` is supported:

- vkCmdSetCoverageToColorEnableNV
- vkCmdSetCoverageToColorLocationNV

If VK_NV_framebuffer_mixed_samples is supported:

- vkCmdSetCoverageModulationModeNV
- vkCmdSetCoverageModulationTableEnableNV
- vkCmdSetCoverageModulationTableNV

If VK_NV_representative_fragment_test is supported:

- vkCmdSetRepresentativeFragmentTestEnableNV

If VK_NV_shading_rate_image is supported:

- vkCmdSetShadingRateImageEnableNV

If VK_NV_viewport_swizzle is supported:

- vkCmdSetViewportSwizzleNV

New Structures

- VkColorBlendAdvancedEXT
- VkColorBlendEquationEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceExtendedDynamicState3FeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceExtendedDynamicState3PropertiesEXT

New Enum Constants

- VK_EXT_EXTENDED_DYNAMIC_STATE_3_EXTENSION_NAME
- VK_EXT_EXTENDED_DYNAMIC_STATE_3_SPEC_VERSION
- Extending VkDynamicState:
  - VK_DYNAMIC_STATE_ALPHA_TO_COVERAGE_ENABLE_EXT
  - VK_DYNAMIC_STATE_ALPHA_TO_ONE_ENABLE_EXT
  - VK_DYNAMIC_STATE_COLOR_BLEND_ENABLE_EXT
  - VK_DYNAMIC_STATE_COLOR_BLEND_EQUATION_EXT
  - VK_DYNAMIC_STATE_COLOR_WRITE_MASK_EXT
  - VK_DYNAMIC_STATE_DEPTH_CLAMP_ENABLE_EXT
  - VK_DYNAMIC_STATE_LOGIC_OP_ENABLE_EXT
  - VK_DYNAMIC_STATE_POLYGON_MODE_EXT

    - VK_DYNAMIC_STATE_RASTERIZATION_SAMPLES_EXT
    - VK_DYNAMIC_STATE_SAMPLE_MASK_EXT
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTENDED_DYNAMIC_STATE_3_FEATURES_EXT
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTENDED_DYNAMIC_STATE_3_PROPERTIES_EXT

If VK_EXT_blend_operation_advanced is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_COLOR_BLEND_ADVANCED_EXT

If VK_EXT_conservative_rasterization is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_CONSERVATIVE_RASTERIZATION_MODE_EXT
    - VK_DYNAMIC_STATE_EXTRA_PRIMITIVE_OVERESTIMATION_SIZE_EXT

If VK_EXT_depth_clip_control is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_DEPTH_CLIP_NEGATIVE_ONE_TO_ONE_EXT

If VK_EXT_depth_clip_enable is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_DEPTH_CLIP_ENABLE_EXT

If VK_EXT_line_rasterization is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_LINE_RASTERIZATION_MODE_EXT
    - VK_DYNAMIC_STATE_LINE_STIPPLE_ENABLE_EXT

If VK_EXT_provoking_vertex is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_PROVOKING_VERTEX_MODE_EXT

If VK_EXT_sample_locations is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_ENABLE_EXT

If VK_EXT_transform_feedback is supported:

- Extending VkDynamicState:

    - VK_DYNAMIC_STATE_RASTERIZATION_STREAM_EXT

If VK_KHR_maintenance2 or Vulkan Version 1.1 is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_TESSELLATION_DOMAIN_ORIGIN_EXT

If VK_NV_clip_space_w_scaling is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_VIEWPORT_W_SCALING_ENABLE_NV

If VK_NV_coverage_reduction_mode is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_COVERAGE_REDUCTION_MODE_NV

If VK_NV_fragment_coverage_to_color is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_COVERAGE_TO_COLOR_ENABLE_NV
    - VK_DYNAMIC_STATE_COVERAGE_TO_COLOR_LOCATION_NV

If VK_NV_framebuffer_mixed_samples is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_COVERAGE_MODULATION_MODE_NV
    - VK_DYNAMIC_STATE_COVERAGE_MODULATION_TABLE_ENABLE_NV
    - VK_DYNAMIC_STATE_COVERAGE_MODULATION_TABLE_NV

If VK_NV_representative_fragment_test is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_REPRESENTATIVE_FRAGMENT_TEST_ENABLE_NV

If VK_NV_shading_rate_image is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_SHADING_RATE_IMAGE_ENABLE_NV

If VK_NV_viewport_swizzle is supported:

- Extending VkDynamicState:
    - VK_DYNAMIC_STATE_VIEWPORT_SWIZZLE_NV

Issues

1) What about the VkPipelineMultisampleStateCreateInfo state `sampleShadingEnable` and `minSampleShading`?

UNRESOLVED

- `sampleShadingEnable` and `minSampleShading` are required when compiling the fragment shader, and it is not meaningful to set them dynamically since they always need to match the fragment shader state, so this hardware state may as well just come from the pipeline with the fragment shader.

Version History

- Revision 2, 2022-07-18 (Piers Daniell)
  - Added rasterizationSamples
- Revision 1, 2022-05-18 (Piers Daniell)
  - Internal revisions

VK_EXT_external_memory_acquire_unmodified

Name String

`VK_EXT_external_memory_acquire_unmodified`

Extension Type

Device extension

Registered Extension Number

454

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_memory
or
Vulkan Version 1.1

Contact

- Lina Versace [GitHub]linyaa-kiwi

Extension Proposal

VK_EXT_external_memory_acquire_unmodified

Other Extension Metadata

Last Modified Date

2023-03-09

Contributors

- Lina Versace, Google
- Chia-I Wu, Google
- James Jones, NVIDIA
- Yiwei Zhang, Google

Description

A memory barrier **may** have a performance penalty when acquiring ownership of a subresource range from an external queue family. This extension provides API that **may** reduce the performance penalty if ownership of the subresource range was previously released to the external queue family and if the resource's memory has remained unmodified between the release and acquire operations.

New Structures

- Extending VkBufferMemoryBarrier, VkBufferMemoryBarrier2, VkImageMemoryBarrier, VkImageMemoryBarrier2:
  - VkExternalMemoryAcquireUnmodifiedEXT

New Enum Constants

- `VK_EXT_EXTERNAL_MEMORY_ACQUIRE_UNMODIFIED_EXTENSION_NAME`
- `VK_EXT_EXTERNAL_MEMORY_ACQUIRE_UNMODIFIED_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_ACQUIRE_UNMODIFIED_EXT`

Version History

- Revision 1, 2023-03-09 (Lina Versace)
  - Initial revision

VK_EXT_external_memory_dma_buf

Name String

`VK_EXT_external_memory_dma_buf`

Extension Type

Device extension

Registered Extension Number

126

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_memory_fd

Contact

- Lina Versace [GitHub]linyaa-kiwi

Other Extension Metadata

Last Modified Date

2017-10-10

IP Status

No known IP claims.

Contributors

- Lina Versace, Google
- James Jones, NVIDIA
- Faith Ekstrand, Intel

Description

A `dma_buf` is a type of file descriptor, defined by the Linux kernel, that allows sharing memory across kernel device drivers and across processes. This extension enables applications to import a `dma_buf` as VkDeviceMemory, to export VkDeviceMemory as a `dma_buf`, and to create VkBuffer objects that **can** be bound to that memory.

New Enum Constants

- `VK_EXT_EXTERNAL_MEMORY_DMA_BUF_EXTENSION_NAME`
- `VK_EXT_EXTERNAL_MEMORY_DMA_BUF_SPEC_VERSION`
- Extending VkExternalMemoryHandleTypeFlagBits:
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_DMA_BUF_BIT_EXT`

Issues

1) How does the application, when creating a VkImage that it intends to bind to `dma_buf` VkDeviceMemory containing an externally produced image, specify the memory layout (such as row pitch and DRM format modifier) of the VkImage? In other words, how does the application

achieve behavior comparable to that provided by `EGL_EXT_image_dma_buf_import` and `EGL_EXT_image_dma_buf_import_modifiers`?

RESOLVED: Features comparable to those in `EGL_EXT_image_dma_buf_import` and `EGL_EXT_image_dma_buf_import_modifiers` will be provided by an extension layered atop this one.

2) Without the ability to specify the memory layout of external `dma_buf` images, how is this extension useful?

RESOLVED: This extension provides exactly one new feature: the ability to import/export between `dma_buf` and VkDeviceMemory. This feature, together with features provided by `VK_KHR_external_memory_fd`, is sufficient to bind a VkBuffer to `dma_buf`.

Version History

- Revision 1, 2017-10-10 (Lina Versace)
  - Squashed internal revisions

VK_EXT_external_memory_host

Name String

`VK_EXT_external_memory_host`

Extension Type

Device extension

Registered Extension Number

179

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_memory
or
Vulkan Version 1.1

Contact

- Daniel Rakos [GitHub]drakos-amd

Other Extension Metadata

Last Modified Date

2017-11-10

IP Status

No known IP claims.

Contributors

- Jaakko Konttinen, AMD
- David Mao, AMD
- Daniel Rakos, AMD
- Tobias Hector, Imagination Technologies
- Faith Ekstrand, Intel
- James Jones, NVIDIA

Description

This extension enables an application to import host allocations and host mapped foreign device memory to Vulkan memory objects.

New Commands

- vkGetMemoryHostPointerPropertiesEXT

New Structures

- VkMemoryHostPointerPropertiesEXT
- Extending VkMemoryAllocateInfo:
  - VkImportMemoryHostPointerInfoEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceExternalMemoryHostPropertiesEXT

New Enum Constants

- `VK_EXT_EXTERNAL_MEMORY_HOST_EXTENSION_NAME`
- `VK_EXT_EXTERNAL_MEMORY_HOST_SPEC_VERSION`
- Extending VkExternalMemoryHandleTypeFlagBits:
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_ALLOCATION_BIT_EXT`
  - `VK_EXTERNAL_MEMORY_HANDLE_TYPE_HOST_MAPPED_FOREIGN_MEMORY_BIT_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_IMPORT_MEMORY_HOST_POINTER_INFO_EXT`
  - `VK_STRUCTURE_TYPE_MEMORY_HOST_POINTER_PROPERTIES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTERNAL_MEMORY_HOST_PROPERTIES_EXT`

Issues

1) What memory type has to be used to import host pointers?

RESOLVED: Depends on the implementation. Applications have to use the new vkGetMemoryHostPointerPropertiesEXT command to query the supported memory types for a particular host pointer. The reported memory types may include memory types that come from a memory heap that is otherwise not usable for regular memory object allocation and thus such a heap's size may be zero.

2) Can the application still access the contents of the host allocation after importing?

RESOLVED: Yes. However, usual synchronization requirements apply.

3) Can the application free the host allocation?

RESOLVED: No, it violates valid usage conditions. Using the memory object imported from a host allocation that is already freed thus results in undefined behavior.

4) Is vkMapMemory expected to return the same host address which was specified when importing it to the memory object?

RESOLVED: No. Implementations are allowed to return the same address but it is not required. Some implementations might return a different virtual mapping of the allocation, although the same physical pages will be used.

5) Is there any limitation on the alignment of the host pointer and/or size?

RESOLVED: Yes. Both the address and the size have to be an integer multiple of `minImportedHostPointerAlignment`. In addition, some platforms and foreign devices may have additional restrictions.

6) Can the same host allocation be imported multiple times into a given physical device?

RESOLVED: No, at least not guaranteed by this extension. Some platforms do not allow locking the same physical pages for device access multiple times, so attempting to do it may result in undefined behavior.

7) Does this extension support exporting the new handle type?

RESOLVED: No.

8) Should we include the possibility to import host mapped foreign device memory using this API?

RESOLVED: Yes, through a separate handle type. Implementations are still allowed to support only one of the handle types introduced by this extension by not returning import support for a particular handle type as returned in VkExternalMemoryPropertiesKHR.

Version History

- Revision 1, 2017-11-10 (Daniel Rakos)
  - Internal revisions

VK_EXT_filter_cubic

Name String

VK_EXT_filter_cubic

Extension Type

Device extension

Registered Extension Number

171

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Matthew Netsch [GitHub]mnetsch

Other Extension Metadata

Last Modified Date

2019-12-13

Contributors

- Bill Licea-Kane, Qualcomm Technologies, Inc.
- Andrew Garrard, Samsung
- Daniel Koch, NVIDIA
- Donald Scorgie, Imagination Technologies
- Graeme Leese, Broadcom
- Jan-Harald Fredriksen, ARM
- Jeff Leger, Qualcomm Technologies, Inc.
- Tobias Hector, AMD
- Tom Olson, ARM
- Stuart Smith, Imagination Technologies

Description

VK_EXT_filter_cubic extends VK_IMG_filter_cubic.

It documents cubic filtering of other image view types. It adds new structures that **can** be added to

the pNext chain of VkPhysicalDeviceImageFormatInfo2 and VkImageFormatProperties2 that **can** be used to determine which image types and which image view types support cubic filtering.

New Structures

- Extending VkImageFormatProperties2:
  - VkFilterCubicImageViewImageFormatPropertiesEXT
- Extending VkPhysicalDeviceImageFormatInfo2:
  - VkPhysicalDeviceImageViewImageFormatInfoEXT

New Enum Constants

- VK_EXT_FILTER_CUBIC_EXTENSION_NAME
- VK_EXT_FILTER_CUBIC_SPEC_VERSION
- Extending VkFilter:
  - VK_FILTER_CUBIC_EXT
- Extending VkFormatFeatureFlagBits:
  - VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_CUBIC_BIT_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_FILTER_CUBIC_IMAGE_VIEW_IMAGE_FORMAT_PROPERTIES_EXT
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_VIEW_IMAGE_FORMAT_INFO_EXT

Version History

- Revision 3, 2019-12-13 (wwlk)
  - Delete requirement to cubic filter the formats USCALED_PACKED32, SSCALED_PACKED32, UINT_PACK32, and SINT_PACK32 (cut/paste error)
- Revision 2, 2019-06-05 (wwlk)
  - Clarify 1D optional
- Revision 1, 2019-01-24 (wwlk)
  - Initial version

VK_EXT_fragment_density_map

Name String

VK_EXT_fragment_density_map

Extension Type

Device extension

Registered Extension Number

219

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

API Interactions

- Interacts with VK_VERSION_1_3
- Interacts with VK_KHR_dynamic_rendering
- Interacts with VK_KHR_format_feature_flags2

SPIR-V Dependencies

- SPV_EXT_fragment_invocation_density

Contact

- Matthew Netsch [GitHub]mnetsch

Other Extension Metadata

Last Modified Date

2021-09-30

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_fragment_invocation_density`

Contributors

- Matthew Netsch, Qualcomm Technologies, Inc.
- Robert VanReenen, Qualcomm Technologies, Inc.
- Jonathan Wicks, Qualcomm Technologies, Inc.
- Tate Hornbeck, Qualcomm Technologies, Inc.
- Sam Holmes, Qualcomm Technologies, Inc.
- Jeff Leger, Qualcomm Technologies, Inc.
- Jan-Harald Fredriksen, ARM
- Jeff Bolz, NVIDIA
- Pat Brown, NVIDIA
- Daniel Rakos, AMD
- Piers Daniell, NVIDIA

Description

This extension allows an application to specify areas of the render target where the fragment shader may be invoked fewer times. These fragments are broadcasted out to multiple pixels to cover the render target.

The primary use of this extension is to reduce workloads in areas where lower quality may not be perceived such as the distorted edges of a lens or the periphery of a user's gaze.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceFragmentDensityMapFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceFragmentDensityMapPropertiesEXT
- Extending VkRenderPassCreateInfo, VkRenderPassCreateInfo2:
  - VkRenderPassFragmentDensityMapCreateInfoEXT

If Vulkan Version 1.3 or VK_KHR_dynamic_rendering is supported:

- Extending VkRenderingInfo:
  - VkRenderingFragmentDensityMapAttachmentInfoEXT

New Enum Constants

- `VK_EXT_FRAGMENT_DENSITY_MAP_EXTENSION_NAME`
- `VK_EXT_FRAGMENT_DENSITY_MAP_SPEC_VERSION`
- Extending VkAccessFlagBits:
  - `VK_ACCESS_FRAGMENT_DENSITY_MAP_READ_BIT_EXT`
- Extending VkFormatFeatureFlagBits:
  - `VK_FORMAT_FEATURE_FRAGMENT_DENSITY_MAP_BIT_EXT`
- Extending VkImageCreateFlagBits:
  - `VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT`
- Extending VkImageLayout:
  - `VK_IMAGE_LAYOUT_FRAGMENT_DENSITY_MAP_OPTIMAL_EXT`
- Extending VkImageUsageFlagBits:
  - `VK_IMAGE_USAGE_FRAGMENT_DENSITY_MAP_BIT_EXT`
- Extending VkImageViewCreateFlagBits:
  - `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DYNAMIC_BIT_EXT`
- Extending VkPipelineStageFlagBits:
  - `VK_PIPELINE_STAGE_FRAGMENT_DENSITY_PROCESS_BIT_EXT`

- Extending VkSamplerCreateFlagBits:
  - VK_SAMPLER_CREATE_SUBSAMPLED_BIT_EXT
  - VK_SAMPLER_CREATE_SUBSAMPLED_COARSE_RECONSTRUCTION_BIT_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_FEATURES_EXT
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_PROPERTIES_EXT
  - VK_STRUCTURE_TYPE_RENDER_PASS_FRAGMENT_DENSITY_MAP_CREATE_INFO_EXT

If VK_KHR_format_feature_flags2 or Vulkan Version 1.3 is supported:

- Extending VkFormatFeatureFlagBits2:
  - VK_FORMAT_FEATURE_2_FRAGMENT_DENSITY_MAP_BIT_EXT

If Vulkan Version 1.3 or VK_KHR_dynamic_rendering is supported:

- Extending VkPipelineCreateFlagBits:
  - VK_PIPELINE_CREATE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
  - VK_PIPELINE_RASTERIZATION_STATE_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_RENDERING_FRAGMENT_DENSITY_MAP_ATTACHMENT_INFO_EXT

New or Modified Built-In Variables

- FragInvocationCountEXT
- FragSizeEXT

New SPIR-V Capabilities

- FragmentDensityEXT

Version History

- Revision 1, 2018-09-25 (Matthew Netsch)
  - Initial version
- Revision 2, 2021-09-30 (Jon Leech)
  - Add interaction with VK_KHR_format_feature_flags2 to vk.xml

VK_EXT_fragment_density_map2

Name String

VK_EXT_fragment_density_map2

Extension Type

Device extension

Registered Extension Number

333

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_EXT_fragment_density_map

Contact

- Matthew Netsch [GitHub]mnetsch

Other Extension Metadata

Last Modified Date

2020-06-16

Interactions and External Dependencies

- Interacts with Vulkan 1.1

Contributors

- Matthew Netsch, Qualcomm Technologies, Inc.
- Jonathan Tinkham, Qualcomm Technologies, Inc.
- Jonathan Wicks, Qualcomm Technologies, Inc.
- Jan-Harald Fredriksen, ARM

Description

This extension adds additional features and properties to `VK_EXT_fragment_density_map` in order to reduce fragment density map host latency as well as improved queries for subsampled sampler implementation-dependent behavior.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceFragmentDensityMap2FeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceFragmentDensityMap2PropertiesEXT

New Enum Constants

- `VK_EXT_FRAGMENT_DENSITY_MAP_2_EXTENSION_NAME`
- `VK_EXT_FRAGMENT_DENSITY_MAP_2_SPEC_VERSION`

- Extending VkImageViewCreateFlagBits:
  - `VK_IMAGE_VIEW_CREATE_FRAGMENT_DENSITY_MAP_DEFERRED_BIT_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_2_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_DENSITY_MAP_2_PROPERTIES_EXT`

Version History

- Revision 1, 2020-06-16 (Matthew Netsch)
  - Initial version

VK_EXT_fragment_shader_interlock

Name String

`VK_EXT_fragment_shader_interlock`

Extension Type

Device extension

Registered Extension Number

252

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_EXT_fragment_shader_interlock

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2019-05-02

Interactions and External Dependencies

- This extension provides API support for `GL_ARB_fragment_shader_interlock`

Contributors

- Daniel Koch, NVIDIA
- Graeme Leese, Broadcom
- Jan-Harald Fredriksen, Arm
- Faith Ekstrand, Intel
- Jeff Bolz, NVIDIA
- Ruihao Zhang, Qualcomm
- Slawomir Grajewski, Intel
- Spencer Fricke, Samsung

Description

This extension adds support for the `FragmentShaderPixelInterlockEXT`, `FragmentShaderSampleInterlockEXT`, and `FragmentShaderShadingRateInterlockEXT` capabilities from the `SPV_EXT_fragment_shader_interlock` extension to Vulkan.

Enabling these capabilities provides a critical section for fragment shaders to avoid overlapping pixels being processed at the same time, and certain guarantees about the ordering of fragment shader invocations of fragments of overlapping pixels.

This extension can be useful for algorithms that need to access per-pixel data structures via shader loads and stores. Algorithms using this extension can access per-pixel data structures in critical sections without other invocations accessing the same per-pixel data. Additionally, the ordering guarantees are useful for cases where the API ordering of fragments is meaningful. For example, applications may be able to execute programmable blending operations in the fragment shader, where the destination buffer is read via image loads and the final value is written via image stores.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceFragmentShaderInterlockFeaturesEXT

New Enum Constants

- `VK_EXT_FRAGMENT_SHADER_INTERLOCK_EXTENSION_NAME`
- `VK_EXT_FRAGMENT_SHADER_INTERLOCK_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADER_INTERLOCK_FEATURES_EXT`

New SPIR-V Capabilities

- `FragmentShaderInterlockEXT`
- `FragmentShaderPixelInterlockEXT`
- `FragmentShaderShadingRateInterlockEXT`

Version History

- Revision 1, 2019-05-24 (Piers Daniell)
  - Internal revisions

VK_EXT_frame_boundary

Name String

`VK_EXT_frame_boundary`

Extension Type

Device extension

Registered Extension Number

376

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- James Fitzpatrick [GitHub]jamesfitzpatrick

Extension Proposal

VK_EXT_frame_boundary

Other Extension Metadata

Last Modified Date

2023-06-14

Contributors

- James Fitzpatrick, Imagination Technologies
- Hugues Evrard, Google
- Melih Yasin Yalcin, Google
- Andrew Garrard, Imagination Technologies
- Jan-Harald Fredriksen, Arm
- Vassili Nikolaev, NVIDIA
- Ting Wei, Huawei

Description

VK_EXT_frame_boundary is a device extension that helps **tools** (such as debuggers) to group queue submissions per frames in non-trivial scenarios, typically when vkQueuePresentKHR is not a relevant frame boundary delimiter.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceFrameBoundaryFeaturesEXT
- Extending VkSubmitInfo, VkSubmitInfo2, VkPresentInfoKHR, VkBindSparseInfo:
  - VkFrameBoundaryEXT

New Enums

- VkFrameBoundaryFlagBitsEXT

New Bitmasks

- VkFrameBoundaryFlagsEXT

New Enum Constants

- `VK_EXT_FRAME_BOUNDARY_EXTENSION_NAME`
- `VK_EXT_FRAME_BOUNDARY_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_FRAME_BOUNDARY_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAME_BOUNDARY_FEATURES_EXT`

Version History

- Revision 0, 2022-01-14 (Hugues Evard)
  - Initial proposal
- Revision 1, 2023-06-14 (James Fitzpatrick)
  - Initial draft

VK_EXT_full_screen_exclusive

Name String

`VK_EXT_full_screen_exclusive`

Extension Type

Device extension

Registered Extension Number

256

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
VK_KHR_surface
and
VK_KHR_get_surface_capabilities2
and
VK_KHR_swapchain

API Interactions

- Interacts with VK_VERSION_1_1
- Interacts with VK_KHR_device_group
- Interacts with VK_KHR_win32_surface

Contact

- James Jones [GitHub]cubanismo

Other Extension Metadata

Last Modified Date

2019-03-12

IP Status

No known IP claims.

Interactions and External Dependencies

- Interacts with Vulkan 1.1
- Interacts with `VK_KHR_device_group`
- Interacts with `VK_KHR_win32_surface`

Contributors

- Hans-Kristian Arntzen, ARM
- Slawomir Grajewski, Intel
- Tobias Hector, AMD
- James Jones, NVIDIA
- Daniel Rakos, AMD
- Jeff Juliano, NVIDIA

- Joshua Schnarr, NVIDIA
- Aaron Hagan, AMD

Description

This extension allows applications to set the policy for swapchain creation and presentation mechanisms relating to full-screen access. Implementations may be able to acquire exclusive access to a particular display for an application window that covers the whole screen. This can increase performance on some systems by bypassing composition, however it can also result in disruptive or expensive transitions in the underlying windowing system when a change occurs.

Applications can choose between explicitly disallowing or allowing this behavior, letting the implementation decide, or managing this mode of operation directly using the new vkAcquireFullScreenExclusiveModeEXT and vkReleaseFullScreenExclusiveModeEXT commands.

New Commands

- vkAcquireFullScreenExclusiveModeEXT
- vkGetPhysicalDeviceSurfacePresentModes2EXT
- vkReleaseFullScreenExclusiveModeEXT

If VK_KHR_device_group or Vulkan Version 1.1 is supported:

- vkGetDeviceGroupSurfacePresentModes2EXT

New Structures

- Extending VkPhysicalDeviceSurfaceInfo2KHR, VkSwapchainCreateInfoKHR:
  - VkSurfaceFullScreenExclusiveInfoEXT
- Extending VkSurfaceCapabilities2KHR:
  - VkSurfaceCapabilitiesFullScreenExclusiveEXT

If VK_KHR_win32_surface is supported:

- Extending VkPhysicalDeviceSurfaceInfo2KHR, VkSwapchainCreateInfoKHR:
  - VkSurfaceFullScreenExclusiveWin32InfoEXT

New Enums

- VkFullScreenExclusiveEXT

New Enum Constants

- `VK_EXT_FULL_SCREEN_EXCLUSIVE_EXTENSION_NAME`
- `VK_EXT_FULL_SCREEN_EXCLUSIVE_SPEC_VERSION`
- Extending VkResult:
  - `VK_ERROR_FULL_SCREEN_EXCLUSIVE_MODE_LOST_EXT`

- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_SURFACE_CAPABILITIES_FULL_SCREEN_EXCLUSIVE_EXT`
  - `VK_STRUCTURE_TYPE_SURFACE_FULL_SCREEN_EXCLUSIVE_INFO_EXT`

If VK_KHR_win32_surface is supported:

- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_SURFACE_FULL_SCREEN_EXCLUSIVE_WIN32_INFO_EXT`

Issues

1) What should the extension & flag be called?

RESOLVED: VK_EXT_full_screen_exclusive.

Other options considered (prior to the app-controlled mode) were:

- VK_EXT_smooth_fullscreen_transition
- VK_EXT_fullscreen_behavior
- VK_EXT_fullscreen_preference
- VK_EXT_fullscreen_hint
- VK_EXT_fast_fullscreen_transition
- VK_EXT_avoid_fullscreen_exclusive

2) Do we need more than a boolean toggle?

RESOLVED: Yes.

Using an enum with default/allowed/disallowed/app-controlled enables applications to accept driver default behavior, specifically override it in either direction without implying the driver is ever required to use full-screen exclusive mechanisms, or manage this mode explicitly.

3) Should this be a KHR or EXT extension?

RESOLVED: EXT, in order to allow it to be shipped faster.

4) Can the fullscreen hint affect the surface capabilities, and if so, should the hint also be specified as input when querying the surface capabilities?

RESOLVED: Yes on both accounts.

While the hint does not guarantee a particular fullscreen mode will be used when the swapchain is created, it can sometimes imply particular modes will NOT be used. If the driver determines that it will opt-out of using a particular mode based on the policy, and knows it can only support certain capabilities if that mode is used, it would be confusing at best to the application to report those capabilities in such cases. Not allowing implementations to report this state to applications could result in situations where applications are unable to determine why swapchain creation fails when they specify certain hint values, which could result in never- terminating surface creation loops.

5) Should full-screen be one word or two?

RESOLVED: Two words.

"Fullscreen" is not in my dictionary, and web searches did not turn up definitive proof that it is a colloquially accepted compound word. Documentation for the corresponding Windows API mechanisms dithers. The text consistently uses a hyphen, but none-the-less, there is a SetFullscreenState method in the DXGI swapchain object. Given this inconclusive external guidance, it is best to adhere to the Vulkan style guidelines and avoid inventing new compound words.

Version History

- Revision 4, 2019-03-12 (Tobias Hector)
  - Added application-controlled mode, and related functions
  - Tidied up appendix
- Revision 3, 2019-01-03 (James Jones)
  - Renamed to VK_EXT_full_screen_exclusive
  - Made related adjustments to the tri-state enumerant names.
- Revision 2, 2018-11-27 (James Jones)
  - Renamed to VK_KHR_fullscreen_behavior
  - Switched from boolean flag to tri-state enum
- Revision 1, 2018-11-06 (James Jones)
  - Internal revision

VK_EXT_hdr_metadata

Name String

`VK_EXT_hdr_metadata`

Extension Type

Device extension

Registered Extension Number

106

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_swapchain

Contact

- Courtney Goeltzenleuchter [GitHub]courtney-g

Other Extension Metadata

Last Modified Date

2024-03-26

IP Status

No known IP claims.

Contributors

- Courtney Goeltzenleuchter, Google
- Sebastian Wick, Red Hat Inc.
- Tobias Hector, AMD

Description

This extension defines two new structures and a function to assign SMPTE (the Society of Motion Picture and Television Engineers) 2086 metadata and CTA (Consumer Technology Association) 861.3 metadata to a swapchain.

SMPTE 2086 metadata defines the color volume of the display on which the content was optimized for viewing and includes the color primaries, white point, and luminance range. When such content is reproduced on another display, this metadata can be used by the presentation engine to improve processing of images. For instance, values in the image can first be clamped to the color volume described in the metadata, and then what remains can be remapped to the color volume of the presentation engine.

CTA 861.3 metadata additionally includes the maximum intended luminance for the content and the maximum average light level across frames.

This extension does not define exactly how this metadata is used, however, it simply provides a mechanism to provide it to the presentation engine. Presentation engines may process the image based on the metadata before displaying it, resulting in the image being modified outside of Vulkan. For example, the clamping of colors in the image to the color volume may change those values in the image itself.

The metadata does not override or otherwise influence the color space and color encoding.

New Commands

- vkSetHdrMetadataEXT

New Structures

- VkHdrMetadataEXT
- VkXYColorEXT

New Enum Constants

- `VK_EXT_HDR_METADATA_EXTENSION_NAME`
- `VK_EXT_HDR_METADATA_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_HDR_METADATA_EXT`

Issues

1) Do we need a query function for the currently specified metadata?

No, Vulkan does not provide queries for state that the application can track on its own.

2) Should we specify default metadata if not specified by the application?

No, the metadata is optional and the absence of the metadata is well-defined.

Version History

- Revision 1, 2016-12-27 (Courtney Goeltzenleuchter)
  - Initial version
- Revision 2, 2018-12-19 (Courtney Goeltzenleuchter)
  - Correct implicit validity for VkHdrMetadataEXT structure
- Revision 3, 2024-03-26 (Tobias Hector & Sebastian Wick)
  - Clarifications and removal of erroneous "reference monitor" term

VK_EXT_headless_surface

Name String

`VK_EXT_headless_surface`

Extension Type

Instance extension

Registered Extension Number

257

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- Lisa Wu [GitHub]chengtianww

Other Extension Metadata

Last Modified Date

2019-03-21

IP Status

No known IP claims.

Contributors

- Ray Smith, Arm

Description

The `VK_EXT_headless_surface` extension is an instance extension. It provides a mechanism to create VkSurfaceKHR objects independently of any window system or display device. The presentation operation for a swapchain created from a headless surface is by default a no-op, resulting in no externally-visible result.

Because there is no real presentation target, future extensions can layer on top of the headless surface to introduce arbitrary or customizable sets of restrictions or features. These could include features like saving to a file or restrictions to emulate a particular presentation target.

This functionality is expected to be useful for application and driver development because it allows any platform to expose an arbitrary or customizable set of restrictions and features of a presentation engine. This makes it a useful portable test target for applications targeting a wide range of presentation engines where the actual target presentation engines might be scarce, unavailable or otherwise undesirable or inconvenient to use for general Vulkan application development.

New Commands

- vkCreateHeadlessSurfaceEXT

New Structures

- VkHeadlessSurfaceCreateInfoEXT

New Bitmasks

- VkHeadlessSurfaceCreateFlagsEXT

New Enum Constants

- `VK_EXT_HEADLESS_SURFACE_EXTENSION_NAME`
- `VK_EXT_HEADLESS_SURFACE_SPEC_VERSION`
- Extending VkStructureType:

- VK_STRUCTURE_TYPE_HEADLESS_SURFACE_CREATE_INFO_EXT

Version History

- Revision 1, 2019-03-21 (Ray Smith)
  - Initial draft

VK_EXT_host_image_copy

Name String

VK_EXT_host_image_copy

Extension Type

Device extension

Registered Extension Number

271

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
VK_KHR_copy_commands2
and
VK_KHR_format_feature_flags2
or
Vulkan Version 1.3

Contact

- Shahbaz Youssefi [GitHub]syoussefi

Extension Proposal

VK_EXT_host_image_copy

Other Extension Metadata

Last Modified Date

2023-04-26

Contributors

- Shahbaz Youssefi, Google

- Faith Ekstrand, Collabora
- Hans-Kristian Arntzen, Valve
- Piers Daniell, NVIDIA
- Jan-Harald Fredriksen, Arm
- James Fitzpatrick, Imagination
- Daniel Story, Nintendo

Description

This extension allows applications to copy data between host memory and images on the host processor, without staging the data through a GPU-accessible buffer. This removes the need to allocate and manage the buffer and its associated memory. On some architectures it may also eliminate an extra copy operation. This extension additionally allows applications to copy data between images on the host.

To support initializing a new image in preparation for a host copy, it is now possible to transition a new image to `VK_IMAGE_LAYOUT_GENERAL` or other host-copyable layouts via vkTransitionImageLayoutEXT. Additionally, it is possible to perform copies that preserve the swizzling layout of the image by using the `VK_HOST_IMAGE_COPY_MEMCPY_EXT` flag. In that case, the memory size needed for copies to or from a buffer can be retrieved by chaining VkSubresourceHostMemcpySizeEXT to `pLayout` in vkGetImageSubresourceLayout2EXT.

New Commands

- vkCopyImageToImageEXT
- vkCopyImageToMemoryEXT
- vkCopyMemoryToImageEXT
- vkGetImageSubresourceLayout2EXT
- vkTransitionImageLayoutEXT

New Structures

- VkCopyImageToImageInfoEXT
- VkCopyImageToMemoryInfoEXT
- VkCopyMemoryToImageInfoEXT
- VkHostImageLayoutTransitionInfoEXT
- VkImageSubresource2EXT
- VkImageToMemoryCopyEXT
- VkMemoryToImageCopyEXT
- VkSubresourceLayout2EXT
- Extending VkImageFormatProperties2:
  - VkHostImageCopyDevicePerformanceQueryEXT

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceHostImageCopyFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceHostImageCopyPropertiesEXT
- Extending VkSubresourceLayout2KHR:
  - VkSubresourceHostMemcpySizeEXT

New Enums

- VkHostImageCopyFlagBitsEXT

New Bitmasks

- VkHostImageCopyFlagsEXT

New Enum Constants

- `VK_EXT_HOST_IMAGE_COPY_EXTENSION_NAME`
- `VK_EXT_HOST_IMAGE_COPY_SPEC_VERSION`
- Extending VkFormatFeatureFlagBits2:
  - `VK_FORMAT_FEATURE_2_HOST_IMAGE_TRANSFER_BIT_EXT`
- Extending VkImageUsageFlagBits:
  - `VK_IMAGE_USAGE_HOST_TRANSFER_BIT_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_COPY_IMAGE_TO_IMAGE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_COPY_IMAGE_TO_MEMORY_INFO_EXT`
  - `VK_STRUCTURE_TYPE_COPY_MEMORY_TO_IMAGE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_HOST_IMAGE_COPY_DEVICE_PERFORMANCE_QUERY_EXT`
  - `VK_STRUCTURE_TYPE_HOST_IMAGE_LAYOUT_TRANSITION_INFO_EXT`
  - `VK_STRUCTURE_TYPE_IMAGE_TO_MEMORY_COPY_EXT`
  - `VK_STRUCTURE_TYPE_MEMORY_TO_IMAGE_COPY_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_HOST_IMAGE_COPY_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_HOST_IMAGE_COPY_PROPERTIES_EXT`
  - `VK_STRUCTURE_TYPE_SUBRESOURCE_HOST_MEMCPY_SIZE_EXT`

Issues

1) When uploading data to an image, the data is usually loaded from disk. Why not have the application load the data directly into a `VkDeviceMemory` bound to a buffer (instead of host memory), and use vkCmdCopyBufferToImage? The same could be done when downloading data from an image.

RESOLVED: This may not always be possible. Complicated Vulkan applications such as game engines often have decoupled subsystems for streaming data and rendering. It may be unreasonable to require the streaming subsystem to coordinate with the rendering subsystem to allocate memory on its behalf, especially as Vulkan may not be the only API supported by the engine. In emulation layers, the image data is necessarily provided by the application in host memory, so an optimization as suggested is not possible. Most importantly, the device memory may not be mappable by an application, but still accessible to the driver.

2) Are `optimalBufferCopyOffsetAlignment` and `optimalBufferCopyRowPitchAlignment` applicable to host memory as well with the functions introduced by this extension? Or should there be new limits?

RESOLVED: No alignment requirements for the host memory pointer.

3) Should there be granularity requirements for image offsets and extents?

RESOLVED: No granularity requirements, i.e. a granularity of 1 pixel (for non-compressed formats) and 1 texel block (for compressed formats) is assumed.

4) How should the application deal with layout transitions before or after copying to or from images?

RESOLVED: An existing issue with linear images is that when emulating other APIs, it is impossible to know when to transition them as they are written to by the host and then used bindlessly. The copy operations in this extension are affected by the same limitation. A new command is thus introduced by this extension to address this problem by allowing the host to perform an image layout transition between a handful of layouts.

Version History

- Revision 0, 2021-01-20 (Faith Ekstrand)
  - Initial idea and xml
- Revision 1, 2023-04-26 (Shahbaz Youssefi)
  - Initial revision

VK_EXT_image_2d_view_of_3d

Name String

`VK_EXT_image_2d_view_of_3d`

Extension Type

Device extension

Registered Extension Number

394

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_maintenance1
and
VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- OpenGL / ES support

Contact

- Mike Blumenkrantz [GitHub]zmike

Other Extension Metadata

Last Modified Date

2022-02-22

IP Status

No known IP claims.

Contributors

- Mike Blumenkrantz, Valve
- Piers Daniell, NVIDIA
- Spencer Fricke, Samsung
- Ricardo Garcia, Igalia
- Graeme Leese, Broadcom
- Ralph Potter, Samsung
- Stu Smith, AMD
- Shahbaz Youssefi, Google
- Alex Walters, Imagination

Description

This extension allows a single slice of a 3D image to be used as a 2D view in image descriptors, matching both the functionality of glBindImageTexture in OpenGL with the `layer` parameter set to true and 2D view binding provided by the extension EGL_KHR_gl_texture_3D_image. It is primarily intended to support GL emulation.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:

- VkPhysicalDeviceImage2DViewOf3DFeaturesEXT

New Enum Constants

- VK_EXT_IMAGE_2D_VIEW_OF_3D_EXTENSION_NAME
- VK_EXT_IMAGE_2D_VIEW_OF_3D_SPEC_VERSION
- Extending VkImageCreateFlagBits:
  - VK_IMAGE_CREATE_2D_VIEW_COMPATIBLE_BIT_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_2D_VIEW_OF_3D_FEATURES_EXT

Version History

- Revision 1, 2022-03-25 (Mike Blumenkrantz)
  - Internal revisions

VK_EXT_image_drm_format_modifier

Name String

VK_EXT_image_drm_format_modifier

Extension Type

Device extension

Registered Extension Number

159

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_bind_memory2
and
VK_KHR_get_physical_device_properties2
and
VK_KHR_sampler_ycbcr_conversion
or
Vulkan Version 1.1
and
VK_KHR_image_format_list
or
Vulkan Version 1.2

API Interactions

- Interacts with VK_VERSION_1_3
- Interacts with VK_KHR_format_feature_flags2

Contact

- Lina Versace [GitHub]linyaa-kiwi

Other Extension Metadata

Last Modified Date

2021-09-30

IP Status

No known IP claims.

Contributors

- Antoine Labour, Google
- Bas Nieuwenhuizen, Google
- Lina Versace, Google
- James Jones, NVIDIA
- Faith Ekstrand, Intel
- Jőrg Wagner, ARM
- Kristian Høgsberg Kristensen, Google
- Ray Smith, ARM

Description

This extension provides the ability to use *DRM format modifiers* with images, enabling Vulkan to better integrate with the Linux ecosystem of graphics, video, and display APIs.

Its functionality closely overlaps with `EGL_EXT_image_dma_buf_import_modifiers`[2] and `EGL_MESA_image_dma_buf_export`[3]. Unlike the EGL extensions, this extension does not require the use of a specific handle type (such as a dma_buf) for external memory and provides more explicit control of image creation.

Introduction to DRM Format Modifiers

A *DRM format modifier* is a 64-bit, vendor-prefixed, semi-opaque unsigned integer. Most *modifiers* represent a concrete, vendor-specific tiling format for images. Some exceptions are `DRM_FORMAT_MOD_LINEAR` (which is not vendor-specific); `DRM_FORMAT_MOD_NONE` (which is an alias of `DRM_FORMAT_MOD_LINEAR` due to historical accident); and `DRM_FORMAT_MOD_INVALID` (which does not represent a tiling format). The *modifier's* vendor prefix consists of the 8 most significant bits. The canonical list of *modifiers* and vendor prefixes is found in `drm_fourcc.h` in the Linux kernel source. The other dominant source of *modifiers* are vendor kernel trees.

One goal of *modifiers* in the Linux ecosystem is to enumerate for each vendor a reasonably sized set

of tiling formats that are appropriate for images shared across processes, APIs, and/or devices, where each participating component may possibly be from different vendors. A non-goal is to enumerate all tiling formats supported by all vendors. Some tiling formats used internally by vendors are inappropriate for sharing; no *modifiers* should be assigned to such tiling formats.

Modifier values typically do not *describe* memory layouts. More precisely, a *modifier*'s lower 56 bits usually have no structure. Instead, modifiers *name* memory layouts; they name a small set of vendor-preferred layouts for image sharing. As a consequence, in each vendor namespace the modifier values are often sequentially allocated starting at 1.

Each *modifier* is usually supported by a single vendor and its name matches the pattern `{VENDOR}_FORMAT_MOD_*` or `DRM_FORMAT_MOD_{VENDOR}_*`. Examples are `I915_FORMAT_MOD_X_TILED` and `DRM_FORMAT_MOD_BROADCOM_VC4_T_TILED`. An exception is `DRM_FORMAT_MOD_LINEAR`, which is supported by most vendors.

Many APIs in Linux use *modifiers* to negotiate and specify the memory layout of shared images. For example, a Wayland compositor and Wayland client may, by relaying *modifiers* over the Wayland protocol `zwp_linux_dmabuf_v1`, negotiate a vendor-specific tiling format for a shared `wl_buffer`. The client may allocate the underlying memory for the `wl_buffer` with GBM, providing the chosen *modifier* to `gbm_bo_create_with_modifiers`. The client may then import the `wl_buffer` into Vulkan for producing image content, providing the resource's dma_buf to VkImportMemoryFdInfoKHR and its *modifier* to VkImageDrmFormatModifierExplicitCreateInfoEXT. The compositor may then import the `wl_buffer` into OpenGL for sampling, providing the resource's dma_buf and *modifier* to `eglCreateImage`. The compositor may also bypass OpenGL and submit the `wl_buffer` directly to the kernel's display API, providing the dma_buf and *modifier* through `drm_mode_fb_cmd2`.

Format Translation

Modifier-capable APIs often pair *modifiers* with DRM formats, which are defined in `drm_fourcc.h`. However, `VK_EXT_image_drm_format_modifier` uses VkFormat instead of DRM formats. The application must convert between VkFormat and DRM format when it sends or receives a DRM format to or from an external API.

The mapping from VkFormat to DRM format is lossy. Therefore, when receiving a DRM format from an external API, often the application must use information from the external API to accurately map the DRM format to a VkFormat. For example, DRM formats do not distinguish between RGB and sRGB (as of 2018-03-28); external information is required to identify the image's color space.

The mapping between VkFormat and DRM format is also incomplete. For some DRM formats there exist no corresponding Vulkan format, and for some Vulkan formats there exist no corresponding DRM format.

Usage Patterns

Three primary usage patterns are intended for this extension:

- **Negotiation.** The application negotiates with *modifier*-aware, external components to determine sets of image creation parameters supported among all components.

  In the Linux ecosystem, the negotiation usually assumes the image is a 2D, single-sampled, non-

mipmapped, non-array image; this extension permits that assumption but does not require it. The result of the negotiation usually resembles a set of tuples such as *(drmFormat, drmFormatModifier)*, where each participating component supports all tuples in the set.

Many details of this negotiation - such as the protocol used during negotiation, the set of image creation parameters expressible in the protocol, and how the protocol chooses which process and which API will create the image - are outside the scope of this specification.

In this extension, vkGetPhysicalDeviceFormatProperties2KHR with VkDrmFormatModifierPropertiesListEXT serves a primary role during the negotiation, and vkGetPhysicalDeviceImageFormatProperties2KHR with VkPhysicalDeviceImageDrmFormatModifierInfoEXT serves a secondary role.

- **Import.** The application imports an image with a *modifier*.

  In this pattern, the application receives from an external source the image's memory and its creation parameters, which are often the result of the negotiation described above. Some image creation parameters are implicitly defined by the external source; for example, `VK_IMAGE_TYPE_2D` is often assumed. Some image creation parameters are usually explicit, such as the image's `format`, `drmFormatModifier`, and `extent`; and each plane's `offset` and `rowPitch`.

  Before creating the image, the application first verifies that the physical device supports the received creation parameters by querying vkGetPhysicalDeviceFormatProperties2KHR with VkDrmFormatModifierPropertiesListEXT and vkGetPhysicalDeviceImageFormatProperties2KHR with VkPhysicalDeviceImageDrmFormatModifierInfoEXT. Then the application creates the image by chaining VkImageDrmFormatModifierExplicitCreateInfoEXT and VkExternalMemoryImageCreateInfo onto VkImageCreateInfo.

- **Export.** The application creates an image and allocates its memory. Then the application exports to *modifier*-aware consumers the image's memory handles; its creation parameters; its *modifier*; and the `offset`, `size`, and `rowPitch` of each *memory plane*.

  In this pattern, the Vulkan device is the authority for the image; it is the allocator of the image's memory and the decider of the image's creation parameters. When choosing the image's creation parameters, the application usually chooses a tuple *(format, drmFormatModifier)* from the result of the negotiation described above. The negotiation's result often contains multiple tuples that share the same format but differ in their *modifier*. In this case, the application should defer the choice of the image's *modifier* to the Vulkan implementation by providing all such *modifiers* to VkImageDrmFormatModifierListCreateInfoEXT::`pDrmFormatModifiers`; and the implementation should choose from `pDrmFormatModifiers` the optimal *modifier* in consideration with the other image parameters.

  The application creates the image by chaining VkImageDrmFormatModifierListCreateInfoEXT and VkExternalMemoryImageCreateInfo onto VkImageCreateInfo. The protocol and APIs by which the application will share the image with external consumers will likely determine the value of VkExternalMemoryImageCreateInfo::`handleTypes`. The implementation chooses for the image an optimal *modifier* from VkImageDrmFormatModifierListCreateInfoEXT ::`pDrmFormatModifiers`. The application then queries the implementation-chosen *modifier* with

vkGetImageDrmFormatModifierPropertiesEXT, and queries the memory layout of each plane with vkGetImageSubresourceLayout.

The application then allocates the image's memory with VkMemoryAllocateInfo, adding chained extending structures for external memory; binds it to the image; and exports the memory, for example, with vkGetMemoryFdKHR.

Finally, the application sends the image's creation parameters, its *modifier*, its per-plane memory layout, and the exported memory handle to the external consumers. The details of how the application transmits this information to external consumers is outside the scope of this specification.

Prior Art

Extension `EGL_EXT_image_dma_buf_import`[1] introduced the ability to create an `EGLImage` by importing for each plane a dma_buf, offset, and row pitch.

Later, extension `EGL_EXT_image_dma_buf_import_modifiers`[2] introduced the ability to query which combination of formats and *modifiers* the implementation supports and to specify *modifiers* during creation of the `EGLImage`.

Extension `EGL_MESA_image_dma_buf_export`[3] is the inverse of `EGL_EXT_image_dma_buf_import_modifiers`.

The Linux kernel modesetting API (KMS), when configuring the display's framebuffer with `struct drm_mode_fb_cmd2`[4], allows one to specify the framebuffer's *modifier* as well as a per-plane memory handle, offset, and row pitch.

GBM, a graphics buffer manager for Linux, allows creation of a `gbm_bo` (that is, a graphics *buffer object*) by importing data similar to that in `EGL_EXT_image_dma_buf_import_modifiers`[1]; and symmetrically allows exporting the same data from the `gbm_bo`. See the references to *modifier* and *plane* in `gbm.h`[5].

New Commands

- vkGetImageDrmFormatModifierPropertiesEXT

New Structures

- VkDrmFormatModifierPropertiesEXT
- VkImageDrmFormatModifierPropertiesEXT
- Extending VkFormatProperties2:
  - VkDrmFormatModifierPropertiesListEXT
- Extending VkImageCreateInfo:
  - VkImageDrmFormatModifierExplicitCreateInfoEXT
  - VkImageDrmFormatModifierListCreateInfoEXT
- Extending VkPhysicalDeviceImageFormatInfo2:
  - VkPhysicalDeviceImageDrmFormatModifierInfoEXT

If VK_KHR_format_feature_flags2 or Vulkan Version 1.3 is supported:

- VkDrmFormatModifierProperties2EXT
- Extending VkFormatProperties2:
  - VkDrmFormatModifierPropertiesList2EXT

New Enum Constants

- VK_EXT_IMAGE_DRM_FORMAT_MODIFIER_EXTENSION_NAME
- VK_EXT_IMAGE_DRM_FORMAT_MODIFIER_SPEC_VERSION
- Extending VkImageAspectFlagBits:
  - VK_IMAGE_ASPECT_MEMORY_PLANE_0_BIT_EXT
  - VK_IMAGE_ASPECT_MEMORY_PLANE_1_BIT_EXT
  - VK_IMAGE_ASPECT_MEMORY_PLANE_2_BIT_EXT
  - VK_IMAGE_ASPECT_MEMORY_PLANE_3_BIT_EXT
- Extending VkImageTiling:
  - VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT
- Extending VkResult:
  - VK_ERROR_INVALID_DRM_FORMAT_MODIFIER_PLANE_LAYOUT_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_DRM_FORMAT_MODIFIER_PROPERTIES_LIST_EXT
  - VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_EXPLICIT_CREATE_INFO_EXT
  - VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_LIST_CREATE_INFO_EXT
  - VK_STRUCTURE_TYPE_IMAGE_DRM_FORMAT_MODIFIER_PROPERTIES_EXT
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_DRM_FORMAT_MODIFIER_INFO_EXT

If VK_KHR_format_feature_flags2 or Vulkan Version 1.3 is supported:

- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_DRM_FORMAT_MODIFIER_PROPERTIES_LIST_2_EXT

Issues

1) Should this extension define a single DRM format modifier per VkImage? Or define one per plane?

+

RESOLVED: There exists a single DRM format modifier per VkImage.

DISCUSSION: Prior art, such as EGL_EXT_image_dma_buf_import_modifiers[2], struct drm_mode_fb_cmd2[4], and struct gbm_import_fd_modifier_data[5], allows defining one *modifier* per plane. However, developers of the GBM and kernel APIs concede it was a mistake. Beginning in Linux 4.10, the

kernel requires that the application provide the same DRM format *modifier* for each plane. (See Linux commit bae781b259269590109e8a4a8227331362b88212). And GBM provides an entry point, `gbm_bo_get_modifier`, for querying the *modifier* of the image but does not provide one to query the modifier of individual planes.

2) When creating an image with VkImageDrmFormatModifierExplicitCreateInfoEXT, which is typically used when *importing* an image, should the application explicitly provide the size of each plane?

+

RESOLVED: No. The application **must** not provide the size. To enforce this, the API requires that VkImageDrmFormatModifierExplicitCreateInfoEXT::`pPlaneLayouts->size` **must** be 0.

DISCUSSION: Prior art, such as `EGL_EXT_image_dma_buf_import_modifiers`[2], `struct drm_mode_fb_cmd2`[4], and `struct gbm_import_fd_modifier_data`[5], omits from the API the size of each plane. Instead, the APIs infer each plane's size from the import parameters, which include the image's pixel format and a dma_buf, offset, and row pitch for each plane.

However, Vulkan differs from EGL and GBM with regards to image creation in the following ways:

Differences in Image Creation

- **Undedicated allocation by default.** When importing or exporting a set of dma_bufs as an `EGLImage` or `gbm_bo`, common practice mandates that each dma_buf's memory be dedicated (in the sense of `VK_KHR_dedicated_allocation`) to the image (though not necessarily dedicated to a single plane). In particular, neither the GBM documentation nor the EGL extension specifications explicitly state this requirement, but in light of common practice this is likely due to under-specification rather than intentional omission. In contrast, `VK_EXT_image_drm_format_modifier` permits, but does not require, the implementation to require dedicated allocations for images created with `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`.
- **Separation of image creation and memory allocation.** When importing a set of dma_bufs as an `EGLImage` or `gbm_bo`, EGL and GBM create the image resource and bind it to memory (the dma_bufs) simultaneously. This allows EGL and GBM to query each dma_buf's size during image creation. In Vulkan, image creation and memory allocation are independent unless a dedicated allocation is used (as in `VK_KHR_dedicated_allocation`). Therefore, without requiring dedicated allocation, Vulkan cannot query the size of each dma_buf (or other external handle) when calculating the image's memory layout. Even if dedication allocation were required, Vulkan cannot calculate the image's memory layout until after the image is bound to its dma_ufs.

The above differences complicate the potential inference of plane size in Vulkan. Consider the following problematic cases:

Problematic Plane Size Calculations

- **Padding.** Some plane of the image may require implementation-dependent padding.
- **Metadata.** For some *modifiers*, the image may have a metadata plane which requires a non-trivial calculation to determine its size.
- **Mipmapped, array, and 3D images.** The implementation may support

`VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT` for images whose `mipLevels`, `arrayLayers`, or `depth` is greater than 1. For such images with certain *modifiers*, the calculation of each plane's size may be non-trivial.

However, an application-provided plane size solves none of the above problems.

For simplicity, consider an external image with a single memory plane. The implementation is obviously capable calculating the image's size when its tiling is `VK_IMAGE_TILING_OPTIMAL`. Likewise, any reasonable implementation is capable of calculating the image's size when its tiling uses a supported *modifier*.

Suppose that the external image's size is smaller than the implementation-calculated size. If the application provided the external image's size to vkCreateImage, the implementation would observe the mismatched size and recognize its inability to comprehend the external image's layout (unless the implementation used the application-provided size to select a refinement of the tiling layout indicated by the *modifier*, which is strongly discouraged). The implementation would observe the conflict, and reject image creation with `VK_ERROR_INVALID_DRM_FORMAT_MODIFIER_PLANE_LAYOUT_EXT`. On the other hand, if the application did not provide the external image's size to vkCreateImage, then the application would observe after calling vkGetImageMemoryRequirements that the external image's size is less than the size required by the implementation. The application would observe the conflict and refuse to bind the `VkImage` to the external memory. In both cases, the result is explicit failure.

Suppose that the external image's size is larger than the implementation-calculated size. If the application provided the external image's size to vkCreateImage, for reasons similar to above the implementation would observe the mismatched size and recognize its inability to comprehend the image data residing in the extra size. The implementation, however, must assume that image data resides in the entire size provided by the application. The implementation would observe the conflict and reject image creation with `VK_ERROR_INVALID_DRM_FORMAT_MODIFIER_PLANE_LAYOUT_EXT`. On the other hand, if the application did not provide the external image's size to vkCreateImage, then the application would observe after calling vkGetImageMemoryRequirements that the external image's size is larger than the implementation-usable size. The application would observe the conflict and refuse to bind the `VkImage` to the external memory. In both cases, the result is explicit failure.

Therefore, an application-provided size provides no benefit, and this extension should not require it. This decision renders VkSubresourceLayout::`size` an unused field during image creation, and thus introduces a risk that implementations may require applications to submit sideband creation parameters in the unused field. To prevent implementations from relying on sideband data, this extension *requires* the application to set `size` to 0.

References

1. `EGL_EXT_image_dma_buf_import`
2. `EGL_EXT_image_dma_buf_import_modifiers`
3. `EGL_MESA_image_dma_buf_export`
4. `struct drm_mode_fb_cmd2`
5. `gbm.h`

Version History

- Revision 1, 2018-08-29 (Lina Versace)
  - First stable revision
- Revision 2, 2021-09-30 (Jon Leech)
  - Add interaction with `VK_KHR_format_feature_flags2` to `vk.xml`

VK_EXT_image_view_min_lod

Name String

`VK_EXT_image_view_min_lod`

Extension Type

Device extension

Registered Extension Number

392

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Joshua Ashton [GitHub]Joshua-Ashton

Other Extension Metadata

Last Modified Date

2021-11-09

IP Status

No known IP claims.

Contributors

- Joshua Ashton, Valve
- Hans-Kristian Arntzen, Valve
- Samuel Iglesias Gonsalvez, Igalia
- Tobias Hector, AMD
- Faith Ekstrand, Intel

- Tom Olson, ARM

Description

This extension allows applications to clamp the minimum LOD value during Image Level(s) Selection, Texel Gathering and Integer Texel Coordinate Operations with a given VkImageView by VkImageViewMinLodCreateInfoEXT::`minLod`.

This extension may be useful to restrict a VkImageView to only mips which have been uploaded, and the use of fractional `minLod` can be useful for smoothly introducing new mip levels when using linear mipmap filtering.

New Structures

- Extending VkImageViewCreateInfo:
  - VkImageViewMinLodCreateInfoEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceImageViewMinLodFeaturesEXT

New Enum Constants

- `VK_EXT_IMAGE_VIEW_MIN_LOD_EXTENSION_NAME`
- `VK_EXT_IMAGE_VIEW_MIN_LOD_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_IMAGE_VIEW_MIN_LOD_CREATE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_IMAGE_VIEW_MIN_LOD_FEATURES_EXT`

Version History

- Revision 1, 2021-07-06 (Joshua Ashton)
  - Initial version

VK_EXT_layer_settings

Name String

`VK_EXT_layer_settings`

Extension Type

Instance extension

Registered Extension Number

497

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Christophe Riccio [GitHub]christophe

Extension Proposal

VK_EXT_layer_settings

Other Extension Metadata

Last Modified Date

2023-09-23

IP Status

No known IP claims.

Contributors

- Christophe Riccio, LunarG
- Mark Lobodzinski, LunarG
- Charles Giessen, LunarG
- Spencer Fricke, LunarG
- Juan Ramos, LunarG
- Daniel Rakos, RasterGrid
- Shahbaz Youssefi, Google
- Lina Versace, Google
- Bill Hollings, The Brenwill Workshop
- Jon Leech, Khronos
- Tom Olson, Arm

Description

This extension provides a mechanism for configuring programmatically through the Vulkan API the behavior of layers.

This extension provides the VkLayerSettingsCreateInfoEXT struct that can be included in the `pNext` chain of the VkInstanceCreateInfo structure passed as the `pCreateInfo` parameter of vkCreateInstance.

The structure contains an array of VkLayerSettingEXT structure values that configure specific features of layers.

NOTE	The VK_EXT_layer_settings extension subsumes all the functionality provided in the VK_EXT_validation_flags extension and the VK_EXT_validation_features extension.

New Structures

- VkLayerSettingEXT
- Extending VkInstanceCreateInfo:
  - VkLayerSettingsCreateInfoEXT

New Enums

- VkLayerSettingTypeEXT

New Enum Constants

- VK_EXT_LAYER_SETTINGS_EXTENSION_NAME
- VK_EXT_LAYER_SETTINGS_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_LAYER_SETTINGS_CREATE_INFO_EXT

Example

One example usage of VK_EXT_layer_settings is as implemented by the Vulkan Profiles layer.

It allows the profiles layer tests used by the profiles layer C.I. to programmatically configure the layer for each test without affecting the C.I. environment, allowing to run multiple tests concurrently.

```
const char* profile_file_data = JSON_TEST_FILES_PATH "VP_KHR_roadmap_2022.json";
const char* profile_name_data = "VP_KHR_roadmap_2022";
VkBool32 emulate_portability_data = VK_TRUE;
const char* simulate_capabilities[] = {
    "SIMULATE_API_VERSION_BIT",
    "SIMULATE_FEATURES_BIT",
    "SIMULATE_PROPERTIES_BIT",
    "SIMULATE_EXTENSIONS_BIT",
    "SIMULATE_FORMATS_BIT",
    "SIMULATE_QUEUE_FAMILY_PROPERTIES_BIT"
};
const char* debug_reports[] = {
    "DEBUG_REPORT_ERROR_BIT",
    "DEBUG_REPORT_WARNING_BIT",
    "DEBUG_REPORT_NOTIFICATION_BIT",
    "DEBUG_REPORT_DEBUG_BIT"
};

const VkLayerSettingEXT settings[] = {
     {kLayerName, kLayerSettingsProfileFile, VK_LAYER_SETTING_TYPE_STRING_EXT, 1,
```

```
&profile_file_data},
     {kLayerName, kLayerSettingsProfileName, VK_LAYER_SETTING_TYPE_STRING_EXT, 1,
&profile_name_data},
     {kLayerName, kLayerSettingsEmulatePortability, VK_LAYER_SETTING_TYPE_BOOL32_EXT,
1, &emulate_portability_data},
     {kLayerName, kLayerSettingsSimulateCapabilities,
VK_LAYER_SETTING_TYPE_STRING_EXT,
        static_cast<uint32_t>(std::size(simulate_capabilities)),
simulate_capabilities},
     {kLayerName, kLayerSettingsDebugReports, VK_LAYER_SETTING_TYPE_STRING_EXT,
        static_cast<uint32_t>(std::size(debug_reports)), debug_reports}
};

const VkLayerSettingsCreateInfoEXT layer_settings_create_info{
    VK_STRUCTURE_TYPE_LAYER_SETTINGS_CREATE_INFO_EXT, nullptr,
    static_cast<uint32_t>(std::size(settings)), settings};

VkInstanceCreateInfo inst_create_info = {};
...
inst_create_info.pNext = &layer_settings_create_info;
vkCreateInstance(&inst_create_info, nullptr, &_instances);
```

Issues

- How should application developers figure out the list of available settings?

This extension does not provide a reflection API for layer settings. Layer settings are described in each layer JSON manifest and the documentation of each layer which implements this extension.

Version History

- Revision 1, 2020-06-17 (Mark Lobodzinski)
  - Initial revision for Validation layer internal usages
- Revision 2, 2023-09-26 (Christophe Riccio)
  - Refactor APIs for any layer usages and public release

VK_EXT_legacy_vertex_attributes

Name String

`VK_EXT_legacy_vertex_attributes`

Extension Type

Device extension

Registered Extension Number

496

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_EXT_vertex_input_dynamic_state

Special Use

- OpenGL / ES support

Contact

- Mike Blumenkrantz [GitHub]zmike

Extension Proposal

VK_EXT_legacy_vertex_attributes

Other Extension Metadata

Last Modified Date

2024-02-23

IP Status

No known IP claims.

Contributors

- Mike Blumenkrantz, Valve
- Piers Daniell, NVIDIA
- Spencer Fricke, LunarG
- Alyssa Rosenzweig, Valve

Description

This extension adds support for legacy features of (non-64-bit) vertex attributes as found in OpenGL:

- Vertex attributes loaded from arbitrary buffer alignments
- Vertex attributes using arbitrary strides
- Vertex attributes where the component data type of the binding does not match the component numeric type of the shader input

These features are only usable with dynamic vertex input. Unaligned loads of vertex attributes may incur performance penalties, indicated with a property.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:

    - VkPhysicalDeviceLegacyVertexAttributesFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
    - VkPhysicalDeviceLegacyVertexAttributesPropertiesEXT

New Enum Constants

- VK_EXT_LEGACY_VERTEX_ATTRIBUTES_EXTENSION_NAME
- VK_EXT_LEGACY_VERTEX_ATTRIBUTES_SPEC_VERSION
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LEGACY_VERTEX_ATTRIBUTES_FEATURES_EXT
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_LEGACY_VERTEX_ATTRIBUTES_PROPERTIES_EXT

Issues

1.) Should implementations convert float/integer values?

RESOLVED: No. When fetching an integer data type from float values or float data types from integer values, the resulting shader values are implementation-dependent.

Version History

- Revision 1, 2024-02-16 (Mike Blumenkrantz)
    - Initial revision

VK_EXT_map_memory_placed

Name String

VK_EXT_map_memory_placed

Extension Type

Device extension

Registered Extension Number

273

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_map_memory2

Contact

- Faith Ekstrand [GitHub]gfxstrand

Extension Proposal

VK_EXT_map_memory_placed

Other Extension Metadata

Last Modified Date

2023-03-21

IP Status

No known IP claims.

Interactions and External Dependencies

- Depends on apitext:VK_KHR_map_memory2
- Interacts with apitext:VK_EXT_external_memory_host

Contributors

- Faith Ekstrand, Collabora
- Tobias Hector, AMD
- James Jones, NVIDIA
- Georg Lehmann, Valve
- Derek Lesho, Codeweavers

Description

This extension allows an application to request that vkMapMemory2KHR attempt to place the memory map at a particular virtual address.

New Structures

- Extending VkMemoryMapInfoKHR:
  - VkMemoryMapPlacedInfoEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceMapMemoryPlacedFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceMapMemoryPlacedPropertiesEXT

New Enum Constants

- `VK_EXT_MAP_MEMORY_PLACED_EXTENSION_NAME`
- `VK_EXT_MAP_MEMORY_PLACED_SPEC_VERSION`
- Extending VkMemoryMapFlagBits:
  - `VK_MEMORY_MAP_PLACED_BIT_EXT`
- Extending VkMemoryUnmapFlagBitsKHR:

    - VK_MEMORY_UNMAP_RESERVE_BIT_EXT
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_MEMORY_MAP_PLACED_INFO_EXT
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAP_MEMORY_PLACED_FEATURES_EXT
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MAP_MEMORY_PLACED_PROPERTIES_EXT

Version History

- Revision 0, 2024-01-14 (Faith Ekstrand)
    - Internal revisions

VK_EXT_memory_budget

Name String

VK_EXT_memory_budget

Extension Type

Device extension

Registered Extension Number

238

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2018-10-08

Contributors

- Jeff Bolz, NVIDIA
- Jeff Juliano, NVIDIA

Description

While running a Vulkan application, other processes on the machine might also be attempting to use the same device memory, which can pose problems. This extension adds support for querying the amount of memory used and the total memory budget for a memory heap. The values returned by this query are implementation-dependent and can depend on a variety of factors including operating system and system load.

The VkPhysicalDeviceMemoryBudgetPropertiesEXT::`heapBudget` values can be used as a guideline for how much total memory from each heap the **current process** can use at any given time, before allocations may start failing or causing performance degradation. The values may change based on other activity in the system that is outside the scope and control of the Vulkan implementation.

The VkPhysicalDeviceMemoryBudgetPropertiesEXT::`heapUsage` will display the **current process** estimated heap usage.

With this information, the idea is for an application at some interval (once per frame, per few seconds, etc) to query `heapBudget` and `heapUsage`. From here the application can notice if it is over budget and decide how it wants to handle the memory situation (free it, move to host memory, changing mipmap levels, etc). This extension is designed to be used in concert with `VK_EXT_memory_priority` to help with this part of memory management.

New Structures

- Extending VkPhysicalDeviceMemoryProperties2:
  - VkPhysicalDeviceMemoryBudgetPropertiesEXT

New Enum Constants

- `VK_EXT_MEMORY_BUDGET_EXTENSION_NAME`
- `VK_EXT_MEMORY_BUDGET_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_BUDGET_PROPERTIES_EXT`

Version History

- Revision 1, 2018-10-08 (Jeff Bolz)
  - Initial revision

VK_EXT_memory_priority

Name String

`VK_EXT_memory_priority`

Extension Type

Device extension

Registered Extension Number

239

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Jeff Bolz [GitHub]jeffbolznv

Other Extension Metadata

Last Modified Date

2018-10-08

Contributors

- Jeff Bolz, NVIDIA
- Jeff Juliano, NVIDIA

Description

This extension adds a `priority` value specified at memory allocation time. On some systems with both device-local and non-device-local memory heaps, the implementation may transparently move memory from one heap to another when a heap becomes full (for example, when the total memory used across all processes exceeds the size of the heap). In such a case, this priority value may be used to determine which allocations are more likely to remain in device-local memory.

New Structures

- Extending VkMemoryAllocateInfo:
  - VkMemoryPriorityAllocateInfoEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceMemoryPriorityFeaturesEXT

New Enum Constants

- `VK_EXT_MEMORY_PRIORITY_EXTENSION_NAME`
- `VK_EXT_MEMORY_PRIORITY_SPEC_VERSION`
- Extending VkStructureType:

- VK_STRUCTURE_TYPE_MEMORY_PRIORITY_ALLOCATE_INFO_EXT
- VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PRIORITY_FEATURES_EXT

Version History

- Revision 1, 2018-10-08 (Jeff Bolz)
  - Initial revision

VK_EXT_metal_objects

Name String

VK_EXT_metal_objects

Extension Type

Device extension

Registered Extension Number

312

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Bill Hollings [GitHub]billhollings

Extension Proposal

VK_EXT_metal_objects

Other Extension Metadata

Last Modified Date

2024-04-04

IP Status

No known IP claims.

Contributors

- Bill Hollings, The Brenwill Workshop Ltd.
- Dzmitry Malyshau, Mozilla Corp.

Description

In a Vulkan implementation that is layered on top of Metal on Apple device platforms, this extension provides the ability to import and export the underlying Metal objects associated with specific Vulkan objects.

As detailed in the extension proposal document, this extension adds one new Vulkan command, vkExportMetalObjectsEXT, to export underlying Metal objects from Vulkan objects, and supports importing the appropriate existing Metal objects when creating Vulkan objects of types VkDeviceMemory, VkImage, VkSemaphore, and VkEvent,

The intent is that this extension will be advertised and supported only on implementations that are layered on top of Metal on Apple device platforms.

New Base Types

- IOSurfaceRef
- MTLBuffer_id
- MTLCommandQueue_id
- MTLDevice_id
- MTLSharedEvent_id
- MTLTexture_id

New Commands

- vkExportMetalObjectsEXT

New Structures

- VkExportMetalObjectsInfoEXT
- Extending VkExportMetalObjectsInfoEXT:
  - VkExportMetalBufferInfoEXT
  - VkExportMetalCommandQueueInfoEXT
  - VkExportMetalDeviceInfoEXT
  - VkExportMetalIOSurfaceInfoEXT
  - VkExportMetalSharedEventInfoEXT
  - VkExportMetalTextureInfoEXT
- Extending VkImageCreateInfo:
  - VkImportMetalIOSurfaceInfoEXT
  - VkImportMetalTextureInfoEXT
- Extending VkInstanceCreateInfo, VkMemoryAllocateInfo, VkImageCreateInfo, VkImageViewCreateInfo, VkBufferViewCreateInfo, VkSemaphoreCreateInfo, VkEventCreateInfo:

    - VkExportMetalObjectCreateInfoEXT
- Extending VkMemoryAllocateInfo:
    - VkImportMetalBufferInfoEXT
- Extending VkSemaphoreCreateInfo, VkEventCreateInfo:
    - VkImportMetalSharedEventInfoEXT

New Enums

- VkExportMetalObjectTypeFlagBitsEXT

New Bitmasks

- VkExportMetalObjectTypeFlagsEXT

New Enum Constants

- `VK_EXT_METAL_OBJECTS_EXTENSION_NAME`
- `VK_EXT_METAL_OBJECTS_SPEC_VERSION`
- Extending VkStructureType:
    - `VK_STRUCTURE_TYPE_EXPORT_METAL_BUFFER_INFO_EXT`
    - `VK_STRUCTURE_TYPE_EXPORT_METAL_COMMAND_QUEUE_INFO_EXT`
    - `VK_STRUCTURE_TYPE_EXPORT_METAL_DEVICE_INFO_EXT`
    - `VK_STRUCTURE_TYPE_EXPORT_METAL_IO_SURFACE_INFO_EXT`
    - `VK_STRUCTURE_TYPE_EXPORT_METAL_OBJECTS_INFO_EXT`
    - `VK_STRUCTURE_TYPE_EXPORT_METAL_OBJECT_CREATE_INFO_EXT`
    - `VK_STRUCTURE_TYPE_EXPORT_METAL_SHARED_EVENT_INFO_EXT`
    - `VK_STRUCTURE_TYPE_EXPORT_METAL_TEXTURE_INFO_EXT`
    - `VK_STRUCTURE_TYPE_IMPORT_METAL_BUFFER_INFO_EXT`
    - `VK_STRUCTURE_TYPE_IMPORT_METAL_IO_SURFACE_INFO_EXT`
    - `VK_STRUCTURE_TYPE_IMPORT_METAL_SHARED_EVENT_INFO_EXT`
    - `VK_STRUCTURE_TYPE_IMPORT_METAL_TEXTURE_INFO_EXT`

Issues

None.

Version History

- Revision 1, 2022-05-28 (Bill Hollings)
    - Initial draft.
    - Incorporated feedback from review by the Vulkan Working Group. Renamed many structures, moved import/export of MTLBuffer to VkDeviceMemory, added export of

MTLSharedEvent, added import of MTLSharedEvent for VkSemaphore and VkEvent, and changed used bit mask fields to individual bit fields to simplify Valid Usage rules.

- Revision 2, 2024-04-04 (Bill Hollings)
  - Add an `__unsafe_unretained` ownership qualifier to all Metal object declarations, to support Automatic Reference Counting (ARC) on Apple devices.

VK_EXT_multi_draw

Name String

`VK_EXT_multi_draw`

Extension Type

Device extension

Registered Extension Number

393

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Mike Blumenkrantz [GitHub]zmike

Other Extension Metadata

Last Modified Date

2021-05-19

Interactions and External Dependencies

- Interacts with Vulkan 1.1.
- Interacts with `VK_KHR_shader_draw_parameters`.

IP Status

No known IP claims.

Contributors

- Mike Blumenkrantz, VALVE
- Piers Daniell, NVIDIA
- Faith Ekstrand, INTEL

- Spencer Fricke, SAMSUNG
- Ricardo Garcia, IGALIA
- Jon Leech, KHRONOS
- Stu Smith, AMD

Description

Processing multiple draw commands in sequence incurs measurable overhead within drivers due to repeated state checks and updates during dispatch. This extension enables passing the entire sequence of draws directly to the driver in order to avoid any such overhead, using an array of VkMultiDrawInfoEXT or VkMultiDrawIndexedInfoEXT structs with `vkCmdDrawMultiEXT` or `vkCmdDrawMultiIndexedEXT`, respectively. These functions could be used any time multiple draw commands are being recorded without any state changes between them in order to maximize performance.

New Commands

- vkCmdDrawMultiEXT
- vkCmdDrawMultiIndexedEXT

New Structures

- VkMultiDrawIndexedInfoEXT
- VkMultiDrawInfoEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceMultiDrawFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceMultiDrawPropertiesEXT

New Enum Constants

- `VK_EXT_MULTI_DRAW_EXTENSION_NAME`
- `VK_EXT_MULTI_DRAW_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTI_DRAW_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MULTI_DRAW_PROPERTIES_EXT`

New or Modified Built-In Variables

- (modified)`DrawIndex`

Version History

- Revision 1, 2021-01-20 (Mike Blumenkrantz)

- Initial version

VK_EXT_nested_command_buffer

Name String

`VK_EXT_nested_command_buffer`

Extension Type

Device extension

Registered Extension Number

452

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2023-09-18

Contributors

- Daniel Story, Nintendo
- Peter Kohaut, NVIDIA
- Shahbaz Youssefi, Google
- Slawomir Grajewski, Intel
- Stu Smith, AMD

Description

With core Vulkan it is not legal to call vkCmdExecuteCommands when recording a secondary command buffer. This extension relaxes that restriction, allowing secondary command buffers to execute other secondary command buffers.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceNestedCommandBufferFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceNestedCommandBufferPropertiesEXT

New Enum Constants

- `VK_EXT_NESTED_COMMAND_BUFFER_EXTENSION_NAME`
- `VK_EXT_NESTED_COMMAND_BUFFER_SPEC_VERSION`
- Extending VkRenderingFlagBits:
  - `VK_RENDERING_CONTENTS_INLINE_BIT_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NESTED_COMMAND_BUFFER_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NESTED_COMMAND_BUFFER_PROPERTIES_EXT`
- Extending VkSubpassContents:
  - `VK_SUBPASS_CONTENTS_INLINE_AND_SECONDARY_COMMAND_BUFFERS_EXT`

Issues

1) The Command Buffer Levels property for the Vulkan commands comes from the `cmdbufferlevel` attribute in `vk.xml` for the command, and it is currently not possible to modify this attribute based on whether an extension is enabled. For this extension we want the `cmdbufferlevel` attribute for vkCmdExecuteCommands to be `primary,secondary` when this extension is enabled and `primary` otherwise.

RESOLVED: The `cmdbufferlevel` attribute for vkCmdExecuteCommands has been changed to `primary,secondary` and a new VUID added to prohibit recording this command in a secondary command buffer unless this extension is enabled.

Version History

- Revision 1, 2023-09-18 (Piers Daniell)
  - Internal revisions

VK_EXT_non_seamless_cube_map

Name String

`VK_EXT_non_seamless_cube_map`

Extension Type

Device extension

Registered Extension Number

423

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Uses

- D3D support
- OpenGL / ES support

Contact

- Georg Lehmann [GitHub]DadSchoorse

Extension Proposal

VK_EXT_non_seamless_cube_map

Other Extension Metadata

Last Modified Date

2021-09-04

IP Status

No known IP claims.

Contributors

- Georg Lehmann

Description

This extension provides functionality to disable cube map edge handling on a per sampler level which matches the behavior of other graphics APIs.

This extension may be useful for building translation layers for those APIs or for porting applications that rely on this cube map behavior.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceNonSeamlessCubeMapFeaturesEXT

New Enum Constants

- `VK_EXT_NON_SEAMLESS_CUBE_MAP_EXTENSION_NAME`
- `VK_EXT_NON_SEAMLESS_CUBE_MAP_SPEC_VERSION`
- Extending VkSamplerCreateFlagBits:
  - `VK_SAMPLER_CREATE_NON_SEAMLESS_CUBE_MAP_BIT_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_NON_SEAMLESS_CUBE_MAP_FEATURES_EXT`

Version History

- Revision 1, 2021-09-04 (Georg Lehmann)
  - First Version

VK_EXT_opacity_micromap

Name String

`VK_EXT_opacity_micromap`

Extension Type

Device extension

Registered Extension Number

397

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_acceleration_structure
and
VK_KHR_synchronization2
or
Vulkan Version 1.3

SPIR-V Dependencies

- SPV_EXT_opacity_micromap

Contact

- Christoph Kubisch [GitHub]pixeljetstream
- Eric Werness

Extension Proposal

VK_EXT_opacity_micromap

Other Extension Metadata

Last Modified Date

2022-08-24

Interactions and External Dependencies

- This extension provides API support for `GLSL_EXT_opacity_micromap`

Contributors

- Christoph Kubisch, NVIDIA
- Eric Werness, NVIDIA
- Josh Barczak, Intel
- Stu Smith, AMD

Description

When adding transparency to a ray traced scene, an application can choose between further tessellating the geometry or using an any-hit shader to allow the ray through specific parts of the geometry. These options have the downside of either significantly increasing memory consumption or adding runtime overhead to run shader code in the middle of traversal, respectively.

This extension adds the ability to add an *opacity micromap* to geometry when building an acceleration structure. The opacity micromap compactly encodes opacity information which can be read by the implementation to mark parts of triangles as opaque or transparent. The format is externally visible to allow the application to compress its internal geometry and surface representations into the compressed format ahead of time. The compressed format subdivides each triangle into a set of subtriangles, each of which can be assigned either two or four opacity values. These opacity values can control if a ray hitting that subtriangle is treated as an opaque hit, complete miss, or possible hit, depending on the controls described in Ray Opacity Micromap.

This extension provides:

- a VkMicromapEXT structure to store the micromap,
- functions similar to acceleration structure build functions to build the opacity micromap array, and
- a structure to extend VkAccelerationStructureGeometryTrianglesDataKHR to attach a micromap to the geometry of the acceleration structure.

New Object Types

- VkMicromapEXT

New Commands

- vkBuildMicromapsEXT
- vkCmdBuildMicromapsEXT
- vkCmdCopyMemoryToMicromapEXT

- vkCmdCopyMicromapEXT
- vkCmdCopyMicromapToMemoryEXT
- vkCmdWriteMicromapsPropertiesEXT
- vkCopyMemoryToMicromapEXT
- vkCopyMicromapEXT
- vkCopyMicromapToMemoryEXT
- vkCreateMicromapEXT
- vkDestroyMicromapEXT
- vkGetDeviceMicromapCompatibilityEXT
- vkGetMicromapBuildSizesEXT
- vkWriteMicromapsPropertiesEXT

New Structures

- VkCopyMemoryToMicromapInfoEXT
- VkCopyMicromapInfoEXT
- VkCopyMicromapToMemoryInfoEXT
- VkMicromapBuildInfoEXT
- VkMicromapBuildSizesInfoEXT
- VkMicromapCreateInfoEXT
- VkMicromapTriangleEXT
- VkMicromapUsageEXT
- VkMicromapVersionInfoEXT
- Extending VkAccelerationStructureGeometryTrianglesDataKHR:
  - VkAccelerationStructureTrianglesOpacityMicromapEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceOpacityMicromapFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceOpacityMicromapPropertiesEXT

New Enums

- VkBuildMicromapFlagBitsEXT
- VkBuildMicromapModeEXT
- VkCopyMicromapModeEXT
- VkMicromapCreateFlagBitsEXT
- VkMicromapTypeEXT

- VkOpacityMicromapFormatEXT
- VkOpacityMicromapSpecialIndexEXT

New Bitmasks

- VkBuildMicromapFlagsEXT
- VkMicromapCreateFlagsEXT

New Enum Constants

- VK_EXT_OPACITY_MICROMAP_EXTENSION_NAME
- VK_EXT_OPACITY_MICROMAP_SPEC_VERSION
- Extending VkAccessFlagBits2:
  - VK_ACCESS_2_MICROMAP_READ_BIT_EXT
  - VK_ACCESS_2_MICROMAP_WRITE_BIT_EXT
- Extending VkBufferUsageFlagBits:
  - VK_BUFFER_USAGE_MICROMAP_BUILD_INPUT_READ_ONLY_BIT_EXT
  - VK_BUFFER_USAGE_MICROMAP_STORAGE_BIT_EXT
- Extending VkBuildAccelerationStructureFlagBitsKHR:
  - VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_DISABLE_OPACITY_MICROMAPS_EXT
  - VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_DATA_UPDATE_EXT
  - VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_OPACITY_MICROMAP_UPDATE_EXT
- Extending VkGeometryInstanceFlagBitsKHR:
  - VK_GEOMETRY_INSTANCE_DISABLE_OPACITY_MICROMAPS_EXT
  - VK_GEOMETRY_INSTANCE_FORCE_OPACITY_MICROMAP_2_STATE_EXT
- Extending VkObjectType:
  - VK_OBJECT_TYPE_MICROMAP_EXT
- Extending VkPipelineCreateFlagBits:
  - VK_PIPELINE_CREATE_RAY_TRACING_OPACITY_MICROMAP_BIT_EXT
- Extending VkPipelineStageFlagBits2:
  - VK_PIPELINE_STAGE_2_MICROMAP_BUILD_BIT_EXT
- Extending VkQueryType:
  - VK_QUERY_TYPE_MICROMAP_COMPACTED_SIZE_EXT
  - VK_QUERY_TYPE_MICROMAP_SERIALIZATION_SIZE_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_ACCELERATION_STRUCTURE_TRIANGLES_OPACITY_MICROMAP_EXT
  - VK_STRUCTURE_TYPE_COPY_MEMORY_TO_MICROMAP_INFO_EXT

- VK_STRUCTURE_TYPE_COPY_MICROMAP_INFO_EXT
- VK_STRUCTURE_TYPE_COPY_MICROMAP_TO_MEMORY_INFO_EXT
- VK_STRUCTURE_TYPE_MICROMAP_BUILD_INFO_EXT
- VK_STRUCTURE_TYPE_MICROMAP_BUILD_SIZES_INFO_EXT
- VK_STRUCTURE_TYPE_MICROMAP_CREATE_INFO_EXT
- VK_STRUCTURE_TYPE_MICROMAP_VERSION_INFO_EXT
- VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_OPACITY_MICROMAP_FEATURES_EXT
- VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_OPACITY_MICROMAP_PROPERTIES_EXT

Reference Code

```
uint32_t BarycentricsToSpaceFillingCurveIndex(float u, float v, uint32_t level)
{
    u = clamp(u, 0.0f, 1.0f);
    v = clamp(v, 0.0f, 1.0f);

    uint32_t iu, iv, iw;

    // Quantize barycentric coordinates
    float fu = u * (1u << level);
    float fv = v * (1u << level);

    iu = (uint32_t)fu;
    iv = (uint32_t)fv;

    float uf = fu - float(iu);
    float vf = fv - float(iv);

    if (iu >= (1u << level)) iu = (1u << level) - 1u;
    if (iv >= (1u << level)) iv = (1u << level) - 1u;

    uint32_t iuv = iu + iv;

    if (iuv >= (1u << level))
        iu -= iuv - (1u << level) + 1u;

    iw = ~(iu + iv);

    if (uf + vf >= 1.0f && iuv < (1u << level) - 1u) --iw;

    uint32_t b0 = ~(iu ^ iw);
    b0 &= ((1u << level) - 1u);
    uint32_t t = (iu ^ iv) & b0;

    uint32_t f = t;
    f ^= f >> 1u;
    f ^= f >> 2u;
```

```
    f ^= f >> 4u;
    f ^= f >> 8u;
    uint32_t b1 = ((f ^ iu) & ~b0) | t;

    // Interleave bits
    b0 = (b0 | (b0 << 8u)) & 0x00ff00ffu;
    b0 = (b0 | (b0 << 4u)) & 0x0f0f0f0fu;
    b0 = (b0 | (b0 << 2u)) & 0x33333333u;
    b0 = (b0 | (b0 << 1u)) & 0x55555555u;
    b1 = (b1 | (b1 << 8u)) & 0x00ff00ffu;
    b1 = (b1 | (b1 << 4u)) & 0x0f0f0f0fu;
    b1 = (b1 | (b1 << 2u)) & 0x33333333u;
    b1 = (b1 | (b1 << 1u)) & 0x55555555u;

    return b0 | (b1 << 1u);
}
```

Issues

(1) Is the build actually similar to an acceleration structure build?

- Resolved: The build should be much lighter-weight than an acceleration structure build, but the infrastructure is similar enough that it makes sense to keep the concepts compatible.

(2) Why does VkMicromapUsageEXT not have type/pNext?

- Resolved: There can be a very large number of these structures, so doubling the size of these can be significant memory consumption. Also, an application may be loading these directly from a file which is more compatible with it being a flat structure. The including structures are extensible and are probably a more suitable place to add extensibility.

(3) Why is there a SPIR-V extension?

- Resolved: There is a ray flag. To be consistent with how the existing ray tracing extensions work that ray flag needs its own extension.

(4) Should there be indirect micromap build?

- Resolved: Not for now. There is more in-depth usage metadata required and it seems less likely that something like a GPU culling system would need to change the counts for a micromap.

(5) Should micromaps have a micromap device address?

- Resolved: There is no need right now (can just use the handle) but that is a bit different from acceleration structures, though the two are not completely parallel in their usage.

(6) Why are the alignment requirements defined as a mix of hardcoded values and caps?

- Resolved: This is most parallel with the definition of `VK_KHR_acceleration_structure` and maintaining commonality makes it easier for applications to share memory.

Version History

- Revision 2, 2022-06-22 (Eric Werness)
  - EXTify and clean up for discussion
- Revision 1, 2022-01-01 (Eric Werness)
  - Initial revision

VK_EXT_pci_bus_info

Name String

`VK_EXT_pci_bus_info`

Extension Type

Device extension

Registered Extension Number

213

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Matthaeus G. Chajdas [GitHub]anteru

Other Extension Metadata

Last Modified Date

2018-12-10

IP Status

No known IP claims.

Contributors

- Matthaeus G. Chajdas, AMD
- Daniel Rakos, AMD

Description

This extension adds a new query to obtain PCI bus information about a physical device.

Not all physical devices have PCI bus information, either due to the device not being connected to the system through a PCI interface or due to platform specific restrictions and policies. Thus this extension is only expected to be supported by physical devices which can provide the information.

As a consequence, applications should always check for the presence of the extension string for each individual physical device for which they intend to issue the new query for and should not have any assumptions about the availability of the extension on any given platform.

New Structures

- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDevicePCIBusInfoPropertiesEXT

New Enum Constants

- `VK_EXT_PCI_BUS_INFO_EXTENSION_NAME`
- `VK_EXT_PCI_BUS_INFO_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PCI_BUS_INFO_PROPERTIES_EXT`

Version History

- Revision 2, 2018-12-10 (Daniel Rakos)
  - Changed all members of the new structure to have the uint32_t type
- Revision 1, 2018-10-11 (Daniel Rakos)
  - Initial revision

VK_EXT_physical_device_drm

Name String

`VK_EXT_physical_device_drm`

Extension Type

Device extension

Registered Extension Number

354

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2

or
Vulkan Version 1.1

Contact

- Simon Ser [GitHub]emersion

Other Extension Metadata

Last Modified Date

2021-06-09

IP Status

No known IP claims.

Contributors

- Simon Ser

Description

This extension provides new facilities to query DRM properties for physical devices, enabling users to match Vulkan physical devices with DRM nodes on Linux.

Its functionality closely overlaps with `EGL_EXT_device_drm`[1]. Unlike the EGL extension, this extension does not expose a string containing the name of the device file and instead exposes device minor numbers.

DRM defines multiple device node types. Each physical device may have one primary node and one render node associated. Physical devices may have no primary node (e.g. if the device does not have a display subsystem), may have no render node (e.g. if it is a software rendering engine), or may have neither (e.g. if it is a software rendering engine without a display subsystem).

To query DRM properties for a physical device, chain VkPhysicalDeviceDrmPropertiesEXT to VkPhysicalDeviceProperties2.

New Structures

- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceDrmPropertiesEXT

New Enum Constants

- `VK_EXT_PHYSICAL_DEVICE_DRM_EXTENSION_NAME`
- `VK_EXT_PHYSICAL_DEVICE_DRM_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_DRM_PROPERTIES_EXT`

References

1. `EGL_EXT_device_drm`

Version History

- Revision 1, 2021-06-09
  - First stable revision

VK_EXT_post_depth_coverage

Name String

`VK_EXT_post_depth_coverage`

Extension Type

Device extension

Registered Extension Number

156

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

SPIR-V Dependencies

- SPV_KHR_post_depth_coverage

Contact

- Daniel Koch [GitHub]dgkoch

Other Extension Metadata

Last Modified Date

2017-07-17

Interactions and External Dependencies

- This extension provides API support for `GL_ARB_post_depth_coverage` and `GL_EXT_post_depth_coverage`

Contributors

- Jeff Bolz, NVIDIA

Description

This extension adds support for the following SPIR-V extension in Vulkan:

- `SPV_KHR_post_depth_coverage`

which allows the fragment shader to control whether values in the `SampleMask` built-in input variable reflect the coverage after early depth and stencil tests are applied.

This extension adds a new `PostDepthCoverage` execution mode under the `SampleMaskPostDepthCoverage` capability. When this mode is specified along with `EarlyFragmentTests`, the value of an input variable decorated with the `SampleMask` built-in reflects the coverage after the early fragment tests are applied. Otherwise, it reflects the coverage before the depth and stencil tests.

When using GLSL source-based shading languages, the `post_depth_coverage` layout qualifier from GL_ARB_post_depth_coverage or GL_EXT_post_depth_coverage maps to the `PostDepthCoverage` execution mode.

New Enum Constants

- `VK_EXT_POST_DEPTH_COVERAGE_EXTENSION_NAME`
- `VK_EXT_POST_DEPTH_COVERAGE_SPEC_VERSION`

New SPIR-V Capabilities

- `SampleMaskPostDepthCoverage`

Version History

- Revision 1, 2017-07-17 (Daniel Koch)
  - Internal revisions

VK_EXT_present_mode_fifo_latest_ready

Name String

`VK_EXT_present_mode_fifo_latest_ready`

Extension Type

Device extension

Registered Extension Number

362

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_swapchain

Contact

- Lionel Duc [GitHub]nvlduc

Extension Proposal

VK_EXT_present_mode_fifo_latest_ready

Other Extension Metadata

Last Modified Date

2024-05-28

IP Status

No known IP claims.

Contributors

- James Jones, NVIDIA
- Lionel Duc, NVIDIA

Description

This device extension adds a new present mode, `VK_PRESENT_MODE_FIFO_LATEST_READY_EXT`.

This tear-free present mode behaves much like `VK_PRESENT_MODE_FIFO_KHR`, except that each vertical blanking period dequeues consecutive present requests until the latest ready is found to update the current image.

While this seems similar in concept to `VK_PRESENT_MODE_MAILBOX_KHR`, the fundamental difference is that the processing of the present requests is done during vblank. From the application perspective, this means for example, that in a flip-based model, a single vblank **may** cause multiple swapchain images to be released at once, while `VK_PRESENT_MODE_MAILBOX_KHR` **may** continuously be releasing images as new requests become ready.

This additional present mode is useful when using a time-based present API.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePresentModeFifoLatestReadyFeaturesEXT

New Enum Constants

- `VK_EXT_PRESENT_MODE_FIFO_LATEST_READY_EXTENSION_NAME`
- `VK_EXT_PRESENT_MODE_FIFO_LATEST_READY_SPEC_VERSION`
- Extending VkPresentModeKHR:
  - `VK_PRESENT_MODE_FIFO_LATEST_READY_EXT`

- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRESENT_MODE_FIFO_LATEST_READY_FEATURES_EXT`

Version History

- Revision 1, 2024-05-28 (Lionel Duc)
  - Internal revisions

VK_EXT_primitive_topology_list_restart

Name String

`VK_EXT_primitive_topology_list_restart`

Extension Type

Device extension

Registered Extension Number

357

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- OpenGL / ES support

Contact

- Shahbaz Youssefi [GitHub]syoussefi

Other Extension Metadata

Last Modified Date

2021-01-11

IP Status

No known IP claims.

Contributors

- Courtney Goeltzenleuchter, Google
- Shahbaz Youssefi, Google

Description

This extension allows list primitives to use the primitive restart index value. This provides a more efficient implementation when layering OpenGL functionality on Vulkan by avoiding emulation which incurs data copies.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePrimitiveTopologyListRestartFeaturesEXT

New Enum Constants

- `VK_EXT_PRIMITIVE_TOPOLOGY_LIST_RESTART_EXTENSION_NAME`
- `VK_EXT_PRIMITIVE_TOPOLOGY_LIST_RESTART_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRIMITIVE_TOPOLOGY_LIST_RESTART_FEATURES_EXT`

Version History

- Revision 0, 2020-09-14 (Courtney Goeltzenleuchter)
  - Internal revisions
- Revision 1, 2021-01-11 (Shahbaz Youssefi)
  - Add the `primitiveTopologyPatchListRestart` feature
  - Internal revisions

VK_EXT_primitives_generated_query

Name String

`VK_EXT_primitives_generated_query`

Extension Type

Device extension

Registered Extension Number

383

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_EXT_transform_feedback

Special Use

- OpenGL / ES support

Contact

- Shahbaz Youssefi [GitHub]syoussefi

Extension Proposal

VK_EXT_primitives_generated_query

Other Extension Metadata

Last Modified Date

2022-01-24

Contributors

- Shahbaz Youssefi, Google
- Piers Daniell, NVIDIA
- Faith Ekstrand, Collabora
- Jan-Harald Fredriksen, Arm

Description

This extension adds support for a new query type to match OpenGL's `GL_PRIMITIVES_GENERATED` to support layering.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePrimitivesGeneratedQueryFeaturesEXT

New Enum Constants

- `VK_EXT_PRIMITIVES_GENERATED_QUERY_EXTENSION_NAME`
- `VK_EXT_PRIMITIVES_GENERATED_QUERY_SPEC_VERSION`
- Extending VkQueryType:
  - `VK_QUERY_TYPE_PRIMITIVES_GENERATED_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PRIMITIVES_GENERATED_QUERY_FEATURES_EXT`

Issues

1) Can the query from `VK_EXT_transform_feedback` be used instead?

RESOLVED: No. While the query from VK_EXT_transform_feedback can produce the same results as in this extension, it is only available while transform feedback is active. The OpenGL `GL_PRIMITIVES_GENERATED` query is independent from transform feedback. Emulation through artificial transform feedback is unnecessarily inefficient.

2) Can VK_QUERY_PIPELINE_STATISTIC_CLIPPING_INVOCATIONS_BIT be used instead?

RESOLVED: It could, but we prefer the extension for simplicity. Vulkan requires that only one query be active at a time. If both the GL_PRIMITIVES_GENERATED and the GL_CLIPPING_INPUT_PRIMITIVES_ARB queries need to be simultaneously enabled, emulation of both through VK_QUERY_PIPELINE_STATISTIC_CLIPPING_INVOCATIONS_BIT is inconvenient.

3) On some hardware, this query cannot be implemented if VkPipelineRasterizationStateCreateInfo::rasterizerDiscardEnable is enabled. How will this be handled?

RESOLVED: A feature flag is exposed by this extension for this. On said hardware, the GL implementation disables rasterizer-discard and achieves the same effect through other means. It will not be able to do the same in Vulkan due to lack of state information. A feature flag is exposed by this extension so the OpenGL implementation on top of Vulkan would be able to implement a similar workaround.

4) On some hardware, this query cannot be implemented for non-zero query indices. How will this be handled?

RESOLVED: A feature flag is exposed by this extension for this. If this feature is not present, the query from VK_EXT_transform_feedback can be used to the same effect.

5) How is the interaction of this extension with transformFeedbackRasterizationStreamSelect handled?

RESOLVED: Disallowed for non-zero streams. In OpenGL, the rasterization stream is always stream zero.

Version History

- Revision 1, 2021-06-23 (Shahbaz Youssefi)
  - Internal revisions

VK_EXT_provoking_vertex

Name String

VK_EXT_provoking_vertex

Extension Type

Device extension

Registered Extension Number

255

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Use

- OpenGL / ES support

Contact

- Jesse Hall [GitHub]jessehall

Other Extension Metadata

Last Modified Date

2021-02-22

IP Status

No known IP claims.

Contributors

- Alexis Hétu, Google
- Bill Licea-Kane, Qualcomm
- Daniel Koch, Nvidia
- Jamie Madill, Google
- Jan-Harald Fredriksen, Arm
- Faith Ekstrand, Intel
- Jeff Bolz, Nvidia
- Jeff Leger, Qualcomm
- Jesse Hall, Google
- Jörg Wagner, Arm
- Matthew Netsch, Qualcomm
- Mike Blumenkrantz, Valve
- Piers Daniell, Nvidia
- Tobias Hector, AMD

Description

This extension allows changing the provoking vertex convention between Vulkan's default convention (first vertex) and OpenGL's convention (last vertex).

This extension is intended for use by API-translation layers that implement APIs like OpenGL on top

of Vulkan, and need to match the source API's provoking vertex convention. Applications using Vulkan directly should use Vulkan's default convention.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceProvokingVertexFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceProvokingVertexPropertiesEXT
- Extending VkPipelineRasterizationStateCreateInfo:
  - VkPipelineRasterizationProvokingVertexStateCreateInfoEXT

New Enums

- VkProvokingVertexModeEXT

New Enum Constants

- `VK_EXT_PROVOKING_VERTEX_EXTENSION_NAME`
- `VK_EXT_PROVOKING_VERTEX_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROVOKING_VERTEX_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROVOKING_VERTEX_PROPERTIES_EXT`
  - `VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_PROVOKING_VERTEX_STATE_CREATE_INFO_EXT`

Issues

1) At what granularity should this state be set?

RESOLVED: At pipeline bind, with an optional per-render pass restriction.

The most natural place to put this state is in the graphics pipeline object. Some implementations require it to be known when creating the pipeline, and pipeline state is convenient for implementing OpenGL 3.2's glProvokingVertex, which can change the state between draw calls. However, some implementations can only change it approximately render pass granularity. To accommodate both, provoking vertex will be pipeline state, but implementations can require that only one mode is used within a render pass instance; the render pass's mode is chosen implicitly when the first pipeline is bound.

2) Does the provoking vertex mode affect the order that vertices are written to transform feedback buffers?

RESOLVED: Yes, to enable layered implementations of OpenGL and D3D.

All of OpenGL, OpenGL ES, and Direct3D 11 require that vertices are written to transform feedback buffers such that flat-shaded attributes have the same value when drawing the contents of the

transform feedback buffer as they did in the original drawing when the transform feedback buffer was written (assuming the provoking vertex mode has not changed, in APIs that support more than one mode).

Version History

- Revision 1, (1c) 2021-02-22 (Jesse Hall)
  - Added VkPhysicalDeviceProvokingVertexPropertiesEXT::transformFeedbackPreservesTriangleFanProvokingVertex to accommodate implementations that cannot change the transform feedback vertex order for triangle fans.
- Revision 1, (1b) 2020-06-14 (Jesse Hall)
  - Added VkPhysicalDeviceProvokingVertexFeaturesEXT::transformFeedbackPreservesProvokingVertex and required that transform feedback write vertices so as to preserve the provoking vertex of each primitive.
- Revision 1, (1a) 2019-10-23 (Jesse Hall)
  - Initial draft, based on a proposal by Alexis Hétu

VK_EXT_queue_family_foreign

Name String

`VK_EXT_queue_family_foreign`

Extension Type

Device extension

Registered Extension Number

127

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_external_memory
or
Vulkan Version 1.1

Contact

- Lina Versace [GitHub]linyaa-kiwi

Other Extension Metadata

Last Modified Date

2017-11-01

IP Status

No known IP claims.

Contributors

- Lina Versace, Google
- James Jones, NVIDIA
- Faith Ekstrand, Intel
- Jesse Hall, Google
- Daniel Rakos, AMD
- Ray Smith, ARM

Description

This extension defines a special queue family, `VK_QUEUE_FAMILY_FOREIGN_EXT`, which can be used to transfer ownership of resources backed by external memory to foreign, external queues. This is similar to `VK_QUEUE_FAMILY_EXTERNAL_KHR`, defined in `VK_KHR_external_memory`. The key differences between the two are:

- The queues represented by `VK_QUEUE_FAMILY_EXTERNAL_KHR` must share the same physical device and the same driver version as the current VkInstance. `VK_QUEUE_FAMILY_FOREIGN_EXT` has no such restrictions. It can represent devices and drivers from other vendors, and can even represent non-Vulkan-capable devices.
- All resources backed by external memory support `VK_QUEUE_FAMILY_EXTERNAL_KHR`. Support for `VK_QUEUE_FAMILY_FOREIGN_EXT` is more restrictive.
- Applications should expect transitions to/from `VK_QUEUE_FAMILY_FOREIGN_EXT` to be more expensive than transitions to/from `VK_QUEUE_FAMILY_EXTERNAL_KHR`.

New Enum Constants

- `VK_EXT_QUEUE_FAMILY_FOREIGN_EXTENSION_NAME`
- `VK_EXT_QUEUE_FAMILY_FOREIGN_SPEC_VERSION`
- `VK_QUEUE_FAMILY_FOREIGN_EXT`

Version History

- Revision 1, 2017-11-01 (Lina Versace)
  - Squashed internal revisions

VK_EXT_robustness2

Name String

`VK_EXT_robustness2`

Extension Type

Device extension

Registered Extension Number

287

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Liam Middlebrook [GitHub]liam-middlebrook

Other Extension Metadata

Last Modified Date

2020-01-29

IP Status

No known IP claims.

Contributors

- Liam Middlebrook, NVIDIA
- Jeff Bolz, NVIDIA

Description

This extension adds stricter requirements for how out of bounds reads and writes are handled. Most accesses **must** be tightly bounds-checked, out of bounds writes **must** be discarded, out of bound reads **must** return zero. Rather than allowing multiple possible (0,0,0,x) vectors, the out of bounds values are treated as zero, and then missing components are inserted based on the format as described in Conversion to RGBA and vertex input attribute extraction.

These additional requirements **may** be expensive on some implementations, and should only be enabled when truly necessary.

This extension also adds support for “null descriptors”, where VK_NULL_HANDLE **can** be used

instead of a valid handle. Accesses to null descriptors have well-defined behavior, and do not rely on robustness.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceRobustness2FeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceRobustness2PropertiesEXT

New Enum Constants

- VK_EXT_ROBUSTNESS_2_EXTENSION_NAME
- VK_EXT_ROBUSTNESS_2_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ROBUSTNESS_2_FEATURES_EXT
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ROBUSTNESS_2_PROPERTIES_EXT

Issues

1. Why do VkPhysicalDeviceRobustness2PropertiesEXT::robustUniformBufferAccessSizeAlignment and VkPhysicalDeviceRobustness2PropertiesEXT::robustStorageBufferAccessSizeAlignment exist?

RESOLVED: Some implementations cannot efficiently tightly bounds-check all buffer accesses. Rather, the size of the bound range is padded to some power of two multiple, up to 256 bytes for uniform buffers and up to 4 bytes for storage buffers, and that padded size is bounds-checked. This is sufficient to implement D3D-like behavior, because D3D only allows binding whole uniform buffers or ranges that are a multiple of 256 bytes, and D3D raw and structured buffers only support 32-bit accesses.

Examples

None.

Version History

- Revision 1, 2019-11-01 (Jeff Bolz, Liam Middlebrook)
  - Initial draft

VK_EXT_sample_locations

Name String

VK_EXT_sample_locations

Extension Type

Device extension

Registered Extension Number

144

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Daniel Rakos [GitHub]drakos-amd

Other Extension Metadata

Last Modified Date

2017-08-02

Contributors

- Mais Alnasser, AMD
- Matthaeus G. Chajdas, AMD
- Maciej Jesionowski, AMD
- Daniel Rakos, AMD
- Slawomir Grajewski, Intel
- Jeff Bolz, NVIDIA
- Bill Licea-Kane, Qualcomm

Description

This extension allows an application to modify the locations of samples within a pixel used in rasterization. Additionally, it allows applications to specify different sample locations for each pixel in a group of adjacent pixels, which **can** increase antialiasing quality (particularly if a custom resolve shader is used that takes advantage of these different locations).

It is common for implementations to optimize the storage of depth values by storing values that **can** be used to reconstruct depth at each sample location, rather than storing separate depth values for each sample. For example, the depth values from a single triangle **may** be represented using plane equations. When the depth value for a sample is needed, it is automatically evaluated at the sample location. Modifying the sample locations causes the reconstruction to no longer evaluate the same depth values as when the samples were originally generated, thus the depth aspect of a depth/stencil attachment **must** be cleared before rendering to it using different sample locations.

Some implementations **may** need to evaluate depth image values while performing image layout transitions. To accommodate this, instances of the VkSampleLocationsInfoEXT structure **can** be specified for each situation where an explicit or automatic layout transition has to take place. VkSampleLocationsInfoEXT **can** be chained from VkImageMemoryBarrier structures to provide sample locations for layout transitions performed by vkCmdWaitEvents and vkCmdPipelineBarrier calls, and VkRenderPassSampleLocationsBeginInfoEXT **can** be chained from VkRenderPassBeginInfo to provide sample locations for layout transitions performed implicitly by a render pass instance.

New Commands

- vkCmdSetSampleLocationsEXT
- vkGetPhysicalDeviceMultisamplePropertiesEXT

New Structures

- VkAttachmentSampleLocationsEXT
- VkMultisamplePropertiesEXT
- VkSampleLocationEXT
- VkSubpassSampleLocationsEXT
- Extending VkImageMemoryBarrier, VkImageMemoryBarrier2:
  - VkSampleLocationsInfoEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceSampleLocationsPropertiesEXT
- Extending VkPipelineMultisampleStateCreateInfo:
  - VkPipelineSampleLocationsStateCreateInfoEXT
- Extending VkRenderPassBeginInfo:
  - VkRenderPassSampleLocationsBeginInfoEXT

New Enum Constants

- `VK_EXT_SAMPLE_LOCATIONS_EXTENSION_NAME`
- `VK_EXT_SAMPLE_LOCATIONS_SPEC_VERSION`
- Extending VkDynamicState:
  - `VK_DYNAMIC_STATE_SAMPLE_LOCATIONS_EXT`
- Extending VkImageCreateFlagBits:
  - `VK_IMAGE_CREATE_SAMPLE_LOCATIONS_COMPATIBLE_DEPTH_BIT_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_MULTISAMPLE_PROPERTIES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SAMPLE_LOCATIONS_PROPERTIES_EXT`
  - `VK_STRUCTURE_TYPE_PIPELINE_SAMPLE_LOCATIONS_STATE_CREATE_INFO_EXT`

- VK_STRUCTURE_TYPE_RENDER_PASS_SAMPLE_LOCATIONS_BEGIN_INFO_EXT
- VK_STRUCTURE_TYPE_SAMPLE_LOCATIONS_INFO_EXT

Version History

- Revision 1, 2017-08-02 (Daniel Rakos)
  - Internal revisions

VK_EXT_shader_atomic_float

Name String

VK_EXT_shader_atomic_float

Extension Type

Device extension

Registered Extension Number

261

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

API Interactions

- Interacts with VkPhysicalDeviceShaderAtomicFloatFeaturesEXT::sparseImageFloat32AtomicAdd
- Interacts with VkPhysicalDeviceShaderAtomicFloatFeaturesEXT::sparseImageFloat32Atomics

SPIR-V Dependencies

- SPV_EXT_shader_atomic_float_add

Contact

- Vikram Kushwaha [GitHub]vkushwaha-nv

Other Extension Metadata

Last Modified Date

2020-07-15

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_shader_atomic_float`

Contributors

- Vikram Kushwaha, NVIDIA
- Jeff Bolz, NVIDIA

Description

This extension allows a shader to contain floating-point atomic operations on buffer, workgroup, and image memory. It also advertises the SPIR-V `AtomicFloat32AddEXT` and `AtomicFloat64AddEXT` capabilities that allows atomic addition on floating-points numbers. The supported operations include `OpAtomicFAddEXT`, `OpAtomicExchange`, `OpAtomicLoad` and `OpAtomicStore`.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderAtomicFloatFeaturesEXT

New Enum Constants

- `VK_EXT_SHADER_ATOMIC_FLOAT_EXTENSION_NAME`
- `VK_EXT_SHADER_ATOMIC_FLOAT_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_FLOAT_FEATURES_EXT`

New SPIR-V Capabilities

- `AtomicFloat32AddEXT`
- `AtomicFloat64AddEXT`

Version History

- Revision 1, 2020-07-15 (Vikram Kushwaha)
  - Internal revisions

VK_EXT_shader_atomic_float2

Name String

`VK_EXT_shader_atomic_float2`

Extension Type

Device extension

Registered Extension Number

274

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_EXT_shader_atomic_float

API Interactions

- Interacts with VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT::sparseImageFloat32AtomicMinMax

SPIR-V Dependencies

- SPV_EXT_shader_atomic_float16_add
- SPV_EXT_shader_atomic_float_min_max

Contact

- Faith Ekstrand [GitHub]gfxstrand

Other Extension Metadata

Last Modified Date

2020-08-14

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GLSL_EXT_shader_atomic_float2`

Contributors

- Faith Ekstrand, Intel

Description

This extension allows a shader to perform 16-bit floating-point atomic operations on buffer and workgroup memory as well as floating-point atomic minimum and maximum operations on buffer, workgroup, and image memory. It advertises the SPIR-V `AtomicFloat16AddEXT` capability which allows atomic add operations on 16-bit floating-point numbers and the SPIR-V `AtomicFloat16MinMaxEXT`, `AtomicFloat32MinMaxEXT` and `AtomicFloat64MinMaxEXT` capabilities which allow atomic minimum and maximum operations on floating-point numbers. The supported operations include `OpAtomicFAddEXT`, `OpAtomicFMinEXT` and `OpAtomicFMaxEXT`.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderAtomicFloat2FeaturesEXT

New Enum Constants

- VK_EXT_SHADER_ATOMIC_FLOAT_2_EXTENSION_NAME
- VK_EXT_SHADER_ATOMIC_FLOAT_2_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_ATOMIC_FLOAT_2_FEATURES_EXT

Issues

1) Should this extension add support for 16-bit image atomics?

RESOLVED: No. While Vulkan supports creating storage images with VK_FORMAT_R16_SFLOAT and doing load and store on them, the data in the shader has a 32-bit representation. Vulkan currently has no facility for even basic reading or writing such images using 16-bit float values in the shader. Adding such functionality would be required before 16-bit image atomics would make sense and is outside the scope of this extension.

New SPIR-V Capabilities

- AtomicFloat32MinMaxEXT
- AtomicFloat32MinMaxEXT
- AtomicFloat32MinMaxEXT
- AtomicFloat64MinMaxEXT

Version History

- Revision 1, 2020-08-14 (Faith Ekstrand)
  - Internal revisions

VK_EXT_shader_image_atomic_int64

Name String

VK_EXT_shader_image_atomic_int64

Extension Type

Device extension

Registered Extension Number

235

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

SPIR-V Dependencies

- SPV_EXT_shader_image_int64

Contact

- Tobias Hector [GitHub]tobski

Other Extension Metadata

Last Modified Date

2020-07-14

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GLSL_EXT_shader_image_int64`

Contributors

- Matthaeus Chajdas, AMD
- Graham Wihlidal, Epic Games
- Tobias Hector, AMD
- Jeff Bolz, Nvidia
- Faith Ekstrand, Intel

Description

This extension extends existing 64-bit integer atomic support to enable these operations on images as well.

When working with large 2- or 3-dimensional data sets (e.g. rasterization or screen-space effects), image accesses are generally more efficient than equivalent buffer accesses. This extension allows applications relying on 64-bit integer atomics in this manner to quickly improve performance with only relatively minor code changes.

64-bit integer atomic support is guaranteed for optimally tiled images with the `VK_FORMAT_R64_UINT` and `VK_FORMAT_R64_SINT` formats.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderImageAtomicInt64FeaturesEXT

New Enum Constants

- `VK_EXT_SHADER_IMAGE_ATOMIC_INT64_EXTENSION_NAME`
- `VK_EXT_SHADER_IMAGE_ATOMIC_INT64_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_IMAGE_ATOMIC_INT64_FEATURES_EXT`

Version History

- Revision 1, 2020-07-14 (Tobias Hector)
  - Initial draft

VK_EXT_shader_module_identifier

Name String

`VK_EXT_shader_module_identifier`

Extension Type

Device extension

Registered Extension Number

463

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
`VK_EXT_pipeline_creation_cache_control`
or
Vulkan Version 1.3

Contact

- Hans-Kristian Arntzen [GitHub]HansKristian-Work

Extension Proposal

VK_EXT_shader_module_identifier

Other Extension Metadata

Last Modified Date

2022-05-16

IP Status

No known IP claims.

Contributors

- Hans-Kristian Arntzen, Valve
- Ricardo Garcia, Igalia
- Piers Daniell, NVIDIA
- Jan-Harald Fredriksen, Arm
- Tom Olson, Arm
- Faith Ekstrand, Collabora

Description

Some applications generate SPIR-V code at runtime. When pipeline caches are primed, either explicitly through e.g. VkPipelineCache mechanisms, or implicitly through driver managed caches, having to re-generate SPIR-V modules is redundant. SPIR-V modules could be cached on disk by an application, but the extra disk size requirement might be prohibitive in some use cases.

This extension adds the ability for an application to query a small identifier associated with a VkShaderModule. On subsequent runs of the application, the same identifier **can** be provided in lieu of a VkShaderModule object. A pipeline creation call with such a module **may** succeed if a pipeline could be created without invoking compilation, and information inside the SPIR-V module is not required by the implementation.

`VK_PIPELINE_CREATE_FAIL_ON_PIPELINE_COMPILE_REQUIRED_BIT` **must** be used if only the identifier is provided, and this use case is intended to work like a non-blocking, speculative compile. Applications **can** fallback as necessary.

The main motivation for identifying the module itself and not the entire pipeline is that pipeline identifiers change when a driver is updated, but module identifiers are expected to be stable for any particular driver implementation. This approach is helpful for shader pre-compilation systems which can prime pipeline caches ahead of time. When on-disk pipeline caches are updated, the same shader identifiers could lead to a pipeline cache hit.

New Commands

- vkGetShaderModuleCreateInfoIdentifierEXT
- vkGetShaderModuleIdentifierEXT

New Structures

- VkShaderModuleIdentifierEXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderModuleIdentifierFeaturesEXT
- Extending VkPhysicalDeviceProperties2:

    - VkPhysicalDeviceShaderModuleIdentifierPropertiesEXT
- Extending VkPipelineShaderStageCreateInfo:
    - VkPipelineShaderStageModuleIdentifierCreateInfoEXT

New Enum Constants

- VK_EXT_SHADER_MODULE_IDENTIFIER_EXTENSION_NAME
- VK_EXT_SHADER_MODULE_IDENTIFIER_SPEC_VERSION
- VK_MAX_SHADER_MODULE_IDENTIFIER_SIZE_EXT
- Extending VkStructureType:
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MODULE_IDENTIFIER_FEATURES_EXT
    - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_MODULE_IDENTIFIER_PROPERTIES_EXT
    - VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_MODULE_IDENTIFIER_CREATE_INFO_EXT
    - VK_STRUCTURE_TYPE_SHADER_MODULE_IDENTIFIER_EXT

Version History

- Revision 1, 2022-03-16 (Hans-Kristian Arntzen)
    - Initial draft

VK_EXT_shader_object

Name String

VK_EXT_shader_object

Extension Type

Device extension

Registered Extension Number

483

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1
and
VK_KHR_dynamic_rendering
or
Vulkan Version 1.3

API Interactions

- Interacts with VK_VERSION_1_1
- Interacts with VK_VERSION_1_3
- Interacts with VK_EXT_blend_operation_advanced
- Interacts with VK_EXT_conservative_rasterization
- Interacts with VK_EXT_depth_clamp_control
- Interacts with VK_EXT_depth_clip_control
- Interacts with VK_EXT_depth_clip_enable
- Interacts with VK_EXT_fragment_density_map
- Interacts with VK_EXT_line_rasterization
- Interacts with VK_EXT_mesh_shader
- Interacts with VK_EXT_provoking_vertex
- Interacts with VK_EXT_sample_locations
- Interacts with VK_EXT_subgroup_size_control
- Interacts with VK_EXT_transform_feedback
- Interacts with VK_KHR_device_group
- Interacts with VK_KHR_fragment_shading_rate
- Interacts with VK_NV_clip_space_w_scaling
- Interacts with VK_NV_coverage_reduction_mode
- Interacts with VK_NV_fragment_coverage_to_color
- Interacts with VK_NV_framebuffer_mixed_samples
- Interacts with VK_NV_mesh_shader
- Interacts with VK_NV_representative_fragment_test
- Interacts with VK_NV_shading_rate_image
- Interacts with VK_NV_viewport_swizzle

Contact

- Daniel Story [GitHub]daniel-story

Extension Proposal

VK_EXT_shader_object

Other Extension Metadata

Last Modified Date

2023-03-30

Interactions and External Dependencies

- Interacts with `VK_EXT_extended_dynamic_state`

- Interacts with VK_EXT_extended_dynamic_state2
- Interacts with VK_EXT_extended_dynamic_state3
- Interacts with VK_EXT_vertex_input_dynamic_state

IP Status

No known IP claims.

Contributors

- Piers Daniell, NVIDIA
- Sandy Jamieson, Nintendo
- Žiga Markuš, LunarG
- Tobias Hector, AMD
- Alex Walters, Imagination
- Shahbaz Youssefi, Google
- Ralph Potter, Samsung
- Jan-Harald Fredriksen, ARM
- Connor Abott, Valve
- Arseny Kapoulkine, Roblox
- Patrick Doane, Activision
- Jeff Leger, Qualcomm
- Stu Smith, AMD
- Chris Glover, Google
- Ricardo Garcia, Igalia
- Faith Ekstrand, Collabora
- Timur Kristóf, Valve
- Constantine Shablya, Collabora
- Daniel Koch, NVIDIA
- Alyssa Rosenzweig, Collabora
- Mike Blumenkrantz, Valve
- Samuel Pitoiset, Valve
- Qun Lin, AMD
- Spencer Fricke, LunarG
- Soroush Faghihi Kashani, Imagination

Description

This extension introduces a new VkShaderEXT object type which represents a single compiled shader stage. Shader objects provide a more flexible alternative to VkPipeline objects, which may be helpful in certain use cases.

New Object Types

- VkShaderEXT

New Commands

- vkCmdBindShadersEXT
- vkCmdBindVertexBuffers2EXT
- vkCmdSetAlphaToCoverageEnableEXT
- vkCmdSetAlphaToOneEnableEXT
- vkCmdSetColorBlendEnableEXT
- vkCmdSetColorBlendEquationEXT
- vkCmdSetColorWriteMaskEXT
- vkCmdSetCullModeEXT
- vkCmdSetDepthBiasEnableEXT
- vkCmdSetDepthBoundsTestEnableEXT
- vkCmdSetDepthClampEnableEXT
- vkCmdSetDepthCompareOpEXT
- vkCmdSetDepthTestEnableEXT
- vkCmdSetDepthWriteEnableEXT
- vkCmdSetFrontFaceEXT
- vkCmdSetLogicOpEXT
- vkCmdSetLogicOpEnableEXT
- vkCmdSetPatchControlPointsEXT
- vkCmdSetPolygonModeEXT
- vkCmdSetPrimitiveRestartEnableEXT
- vkCmdSetPrimitiveTopologyEXT
- vkCmdSetRasterizationSamplesEXT
- vkCmdSetRasterizerDiscardEnableEXT
- vkCmdSetSampleMaskEXT
- vkCmdSetScissorWithCountEXT
- vkCmdSetStencilOpEXT
- vkCmdSetStencilTestEnableEXT
- vkCmdSetTessellationDomainOriginEXT
- vkCmdSetVertexInputEXT
- vkCmdSetViewportWithCountEXT
- vkCreateShadersEXT

- vkDestroyShaderEXT
- vkGetShaderBinaryDataEXT

If VK_EXT_blend_operation_advanced is supported:

- vkCmdSetColorBlendAdvancedEXT

If VK_EXT_conservative_rasterization is supported:

- vkCmdSetConservativeRasterizationModeEXT
- vkCmdSetExtraPrimitiveOverestimationSizeEXT

If VK_EXT_depth_clamp_control is supported:

- vkCmdSetDepthClampRangeEXT

If VK_EXT_depth_clip_control is supported:

- vkCmdSetDepthClipNegativeOneToOneEXT

If VK_EXT_depth_clip_enable is supported:

- vkCmdSetDepthClipEnableEXT

If `VK_EXT_line_rasterization` is supported:

- vkCmdSetLineRasterizationModeEXT
- vkCmdSetLineStippleEnableEXT

If VK_EXT_provoking_vertex is supported:

- vkCmdSetProvokingVertexModeEXT

If VK_EXT_sample_locations is supported:

- vkCmdSetSampleLocationsEnableEXT

If VK_EXT_transform_feedback is supported:

- vkCmdSetRasterizationStreamEXT

If `VK_NV_clip_space_w_scaling` is supported:

- vkCmdSetViewportWScalingEnableNV

If `VK_NV_coverage_reduction_mode` is supported:

- vkCmdSetCoverageReductionModeNV

If `VK_NV_fragment_coverage_to_color` is supported:

- vkCmdSetCoverageToColorEnableNV

- vkCmdSetCoverageToColorLocationNV

If VK_NV_framebuffer_mixed_samples is supported:

- vkCmdSetCoverageModulationModeNV
- vkCmdSetCoverageModulationTableEnableNV
- vkCmdSetCoverageModulationTableNV

If VK_NV_representative_fragment_test is supported:

- vkCmdSetRepresentativeFragmentTestEnableNV

If VK_NV_shading_rate_image is supported:

- vkCmdSetShadingRateImageEnableNV

If VK_NV_viewport_swizzle is supported:

- vkCmdSetViewportSwizzleNV

New Structures

- VkColorBlendAdvancedEXT
- VkColorBlendEquationEXT
- VkShaderCreateInfoEXT
- VkVertexInputAttributeDescription2EXT
- VkVertexInputBindingDescription2EXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderObjectFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceShaderObjectPropertiesEXT
- Extending VkPipelineShaderStageCreateInfo, VkShaderCreateInfoEXT:
  - VkShaderRequiredSubgroupSizeCreateInfoEXT

New Enums

- VkShaderCodeTypeEXT
- VkShaderCreateFlagBitsEXT

New Bitmasks

- VkShaderCreateFlagsEXT

New Enum Constants

- VK_EXT_SHADER_OBJECT_EXTENSION_NAME

- VK_EXT_SHADER_OBJECT_SPEC_VERSION
- Extending VkObjectType:
  - VK_OBJECT_TYPE_SHADER_EXT
- Extending VkResult:
  - VK_ERROR_INCOMPATIBLE_SHADER_BINARY_EXT
  - VK_INCOMPATIBLE_SHADER_BINARY_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_OBJECT_FEATURES_EXT
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_OBJECT_PROPERTIES_EXT
  - VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT
  - VK_STRUCTURE_TYPE_SHADER_REQUIRED_SUBGROUP_SIZE_CREATE_INFO_EXT
  - VK_STRUCTURE_TYPE_VERTEX_INPUT_ATTRIBUTE_DESCRIPTION_2_EXT
  - VK_STRUCTURE_TYPE_VERTEX_INPUT_BINDING_DESCRIPTION_2_EXT

If VK_EXT_fragment_density_map is supported:

- Extending VkShaderCreateFlagBitsEXT:
  - VK_SHADER_CREATE_FRAGMENT_DENSITY_MAP_ATTACHMENT_BIT_EXT

If VK_EXT_mesh_shader or VK_NV_mesh_shader is supported:

- Extending VkShaderCreateFlagBitsEXT:
  - VK_SHADER_CREATE_NO_TASK_SHADER_BIT_EXT

If VK_EXT_subgroup_size_control or Vulkan Version 1.3 is supported:

- Extending VkShaderCreateFlagBitsEXT:
  - VK_SHADER_CREATE_ALLOW_VARYING_SUBGROUP_SIZE_BIT_EXT
  - VK_SHADER_CREATE_REQUIRE_FULL_SUBGROUPS_BIT_EXT

If VK_KHR_device_group or Vulkan Version 1.1 is supported:

- Extending VkShaderCreateFlagBitsEXT:
  - VK_SHADER_CREATE_DISPATCH_BASE_BIT_EXT

If VK_KHR_fragment_shading_rate is supported:

- Extending VkShaderCreateFlagBitsEXT:
  - VK_SHADER_CREATE_FRAGMENT_SHADING_RATE_ATTACHMENT_BIT_EXT

Examples

Example 1

Create linked pair of vertex and fragment shaders.

```
// Logical device created with the shaderObject feature enabled
VkDevice device;

// SPIR-V shader code for a vertex shader, along with its size in bytes
void* pVertexSpirv;
size_t vertexSpirvSize;

// SPIR-V shader code for a fragment shader, along with its size in bytes
void* pFragmentSpirv;
size_t fragmentSpirvSize;

// Descriptor set layout compatible with the shaders
VkDescriptorSetLayout descriptorSetLayout;

VkShaderCreateInfoEXT shaderCreateInfos[2] =
{
    {
        .sType = VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT,
        .pNext = NULL,
        .flags = VK_SHADER_CREATE_LINK_STAGE_BIT_EXT,
        .stage = VK_SHADER_STAGE_VERTEX_BIT,
        .nextStage = VK_SHADER_STAGE_FRAGMENT_BIT,
        .codeType = VK_SHADER_CODE_TYPE_SPIRV_EXT,
        .codeSize = vertexSpirvSize,
        .pCode = pVertexSpirv,
        .pName = "main",
        .setLayoutCount = 1,
        .pSetLayouts = &descriptorSetLayout;
        .pushConstantRangeCount = 0,
        .pPushConstantRanges = NULL,
        .pSpecializationInfo = NULL
    },
    {
        .sType = VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT,
        .pNext = NULL,
        .flags = VK_SHADER_CREATE_LINK_STAGE_BIT_EXT,
        .stage = VK_SHADER_STAGE_FRAGMENT_BIT,
        .nextStage = 0,
        .codeType = VK_SHADER_CODE_TYPE_SPIRV_EXT,
        .codeSize = fragmentSpirvSize,
        .pCode = pFragmentSpirv,
        .pName = "main",
        .setLayoutCount = 1,
        .pSetLayouts = &descriptorSetLayout;
        .pushConstantRangeCount = 0,
        .pPushConstantRanges = NULL,
        .pSpecializationInfo = NULL
    }
```

```
};

VkResult result;
VkShaderEXT shaders[2];

result = vkCreateShadersEXT(device, 2, &shaderCreateInfos, NULL, shaders);
if (result != VK_SUCCESS)
{
    // Handle error
}
```

Later, during command buffer recording, bind the linked shaders and draw.

```
// Command buffer in the recording state
VkCommandBuffer commandBuffer;

// Vertex and fragment shader objects created above
VkShaderEXT shaders[2];

// Assume vertex buffers, descriptor sets, etc. have been bound, and existing
// state setting commands have been called to set all required state

const VkShaderStageFlagBits stages[2] =
{
    VK_SHADER_STAGE_VERTEX_BIT,
    VK_SHADER_STAGE_FRAGMENT_BIT
};

// Bind linked shaders
vkCmdBindShadersEXT(commandBuffer, 2, stages, shaders);

// Equivalent to the previous line. Linked shaders can be bound one at a time,
// in any order:
// vkCmdBindShadersEXT(commandBuffer, 1, &stages[1], &shaders[1]);
// vkCmdBindShadersEXT(commandBuffer, 1, &stages[0], &shaders[0]);

// The above is sufficient to draw if the device was created with the
// tessellationShader and geometryShader features disabled. Otherwise, since
// those stages should not execute, vkCmdBindShadersEXT() must be called at
// least once with each of their stages in pStages before drawing:

const VkShaderStageFlagBits unusedStages[3] =
{
    VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT,
    VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT,
    VK_SHADER_STAGE_GEOMETRY_BIT
};

// NULL pShaders is equivalent to an array of stageCount VK_NULL_HANDLE values,
// meaning no shaders are bound to those stages, and that any previously bound
```

```
// shaders are unbound
vkCmdBindShadersEXT(commandBuffer, 3, unusedStages, NULL);

// Graphics shader objects may only be used to draw inside dynamic render pass
// instances begun with vkCmdBeginRendering(), assume one has already been begun

// Draw a triangle
vkCmdDraw(commandBuffer, 3, 1, 0, 0);
```

Example 2

Create unlinked vertex, geometry, and fragment shaders.

```
// Logical device created with the shaderObject feature enabled
VkDevice device;

// SPIR-V shader code for vertex shaders, along with their sizes in bytes
void* pVertexSpirv[2];
size_t vertexSpirvSize[2];

// SPIR-V shader code for a geometry shader, along with its size in bytes
void pGeometrySpirv;
size_t geometrySpirvSize;

// SPIR-V shader code for fragment shaders, along with their sizes in bytes
void* pFragmentSpirv[2];
size_t fragmentSpirvSize[2];

// Descriptor set layout compatible with the shaders
VkDescriptorSetLayout descriptorSetLayout;

VkShaderCreateInfoEXT shaderCreateInfos[5] =
{
    // Stage order does not matter
    {
        .sType = VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT,
        .pNext = NULL,
        .flags = 0,
        .stage = VK_SHADER_STAGE_GEOMETRY_BIT,
        .nextStage = VK_SHADER_STAGE_FRAGMENT_BIT,
        .codeType = VK_SHADER_CODE_TYPE_SPIRV_EXT,
        .codeSize = pGeometrySpirv,
        .pCode = geometrySpirvSize,
        .pName = "main",
        .setLayoutCount = 1,
        .pSetLayouts = &descriptorSetLayout;
        .pushConstantRangeCount = 0,
        .pPushConstantRanges = NULL,
        .pSpecializationInfo = NULL
    },
```

```
    {
        .sType = VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT,
        .pNext = NULL,
        .flags = 0,
        .stage = VK_SHADER_STAGE_VERTEX_BIT,
        .nextStage = VK_SHADER_STAGE_GEOMETRY_BIT,
        .codeType = VK_SHADER_CODE_TYPE_SPIRV_EXT,
        .codeSize = vertexSpirvSize[0],
        .pCode = pVertexSpirv[0],
        .pName = "main",
        .setLayoutCount = 1,
        .pSetLayouts = &descriptorSetLayout;
        .pushConstantRangeCount = 0,
        .pPushConstantRanges = NULL,
        .pSpecializationInfo = NULL
    },
    {
        .sType = VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT,
        .pNext = NULL,
        .flags = 0,
        .stage = VK_SHADER_STAGE_FRAGMENT_BIT,
        .nextStage = 0,
        .codeType = VK_SHADER_CODE_TYPE_SPIRV_EXT,
        .codeSize = fragmentSpirvSize[0],
        .pCode = pFragmentSpirv[0],
        .pName = "main",
        .setLayoutCount = 1,
        .pSetLayouts = &descriptorSetLayout;
        .pushConstantRangeCount = 0,
        .pPushConstantRanges = NULL,
        .pSpecializationInfo = NULL
    },
    {
        .sType = VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT,
        .pNext = NULL,
        .flags = 0,
        .stage = VK_SHADER_STAGE_FRAGMENT_BIT,
        .nextStage = 0,
        .codeType = VK_SHADER_CODE_TYPE_SPIRV_EXT,
        .codeSize = fragmentSpirvSize[1],
        .pCode = pFragmentSpirv[1],
        .pName = "main",
        .setLayoutCount = 1,
        .pSetLayouts = &descriptorSetLayout;
        .pushConstantRangeCount = 0,
        .pPushConstantRanges = NULL,
        .pSpecializationInfo = NULL
    },
    {
        .sType = VK_STRUCTURE_TYPE_SHADER_CREATE_INFO_EXT,
        .pNext = NULL,
```

```
        .flags = 0,
        .stage = VK_SHADER_STAGE_VERTEX_BIT,
        // Suppose we want this vertex shader to be able to be followed by
        // either a geometry shader or fragment shader:
        .nextStage = VK_SHADER_STAGE_GEOMETRY_BIT | VK_SHADER_STAGE_FRAGMENT_BIT,
        .codeType = VK_SHADER_CODE_TYPE_SPIRV_EXT,
        .codeSize = vertexSpirvSize[1],
        .pCode = pVertexSpirv[1],
        .pName = "main",
        .setLayoutCount = 1,
        .pSetLayouts = &descriptorSetLayout;
        .pushConstantRangeCount = 0,
        .pPushConstantRanges = NULL,
        .pSpecializationInfo = NULL
    }
};

VkResult result;
VkShaderEXT shaders[5];

result = vkCreateShadersEXT(device, 5, &shaderCreateInfos, NULL, shaders);
if (result != VK_SUCCESS)
{
    // Handle error
}
```

Later, during command buffer recording, bind the linked shaders in different combinations and draw.

```
// Command buffer in the recording state
VkCommandBuffer commandBuffer;

// Vertex, geometry, and fragment shader objects created above
VkShaderEXT shaders[5];

// Assume vertex buffers, descriptor sets, etc. have been bound, and existing
// state setting commands have been called to set all required state

const VkShaderStageFlagBits stages[3] =
{
    // Any order is allowed
    VK_SHADER_STAGE_FRAGMENT_BIT,
    VK_SHADER_STAGE_VERTEX_BIT,
    VK_SHADER_STAGE_GEOMETRY_BIT,
};

VkShaderEXT bindShaders[3] =
{
    shaders[2], // FS
    shaders[1], // VS
```

```
    shaders[0]  // GS
};

// Bind unlinked shaders
vkCmdBindShadersEXT(commandBuffer, 3, stages, bindShaders);

// Assume the tessellationShader feature is disabled, so vkCmdBindShadersEXT()
// need not have been called with either tessellation stage

// Graphics shader objects may only be used to draw inside dynamic render pass
// instances begun with vkCmdBeginRendering(), assume one has already been begun

// Draw a triangle
vkCmdDraw(commandBuffer, 3, 1, 0, 0);

// Bind a different unlinked fragment shader
const VkShaderStageFlagBits fragmentStage = VK_SHADER_STAGE_FRAGMENT_BIT;
vkCmdBindShadersEXT(commandBuffer, 1, &fragmentStage, &shaders[3]);

// Draw another triangle
vkCmdDraw(commandBuffer, 3, 1, 0, 0);

// Bind a different unlinked vertex shader
const VkShaderStageFlagBits vertexStage = VK_SHADER_STAGE_VERTEX_BIT;
vkCmdBindShadersEXT(commandBuffer, 1, &vertexStage, &shaders[4]);

// Draw another triangle
vkCmdDraw(commandBuffer, 3, 1, 0, 0);
```

Version History

- Revision 1, 2023-03-30 (Daniel Story)
  - Initial draft

VK_EXT_shader_replicated_composites

Name String

`VK_EXT_shader_replicated_composites`

Extension Type

Device extension

Registered Extension Number

565

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

SPIR-V Dependencies

- SPV_EXT_replicated_composites

Contact

- Kevin Petit [GitHub]kpet

Extension Proposal

VK_EXT_shader_replicated_composites

Last Modified Date

2024-02-08

IP Status

No known IP claims.

Contributors

- Kévin Petit, Arm Ltd.
- Jeff Bolz, NVIDIA
- Piers Daniell, NVIDIA

This extension adds support for creating composites from a single value in SPIR-V modules, as defined by SPV_EXT_replicated_composites.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderReplicatedCompositesFeaturesEXT

New Enum Constants

- `VK_EXT_SHADER_REPLICATED_COMPOSITES_EXTENSION_NAME`
- `VK_EXT_SHADER_REPLICATED_COMPOSITES_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_REPLICATED_COMPOSITES_FEATURES_EXT`

New SPIR-V Capabilities

- ReplicatedCompositesEXT

Version History

- Revision 1, 2024-02-08 (Kévin Petit)

- Initial revision

VK_EXT_shader_stencil_export

Name String

`VK_EXT_shader_stencil_export`

Extension Type

Device extension

Registered Extension Number

141

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

SPIR-V Dependencies

- SPV_EXT_shader_stencil_export

Contact

- Dominik Witczak [GitHub]dominikwitczakamd

Other Extension Metadata

Last Modified Date

2017-07-19

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_ARB_shader_stencil_export`

Contributors

- Dominik Witczak, AMD
- Daniel Rakos, AMD
- Rex Xu, AMD

Description

This extension adds support for the SPIR-V extension `SPV_EXT_shader_stencil_export`, providing a mechanism whereby a shader may generate the stencil reference value per invocation. When

stencil testing is enabled, this allows the test to be performed against the value generated in the shader.

New Enum Constants

- `VK_EXT_SHADER_STENCIL_EXPORT_EXTENSION_NAME`
- `VK_EXT_SHADER_STENCIL_EXPORT_SPEC_VERSION`

Version History

- Revision 1, 2017-07-19 (Dominik Witczak)
  - Initial draft

VK_EXT_shader_tile_image

Name String

`VK_EXT_shader_tile_image`

Extension Type

Device extension

Registered Extension Number

396

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

Vulkan Version 1.3

SPIR-V Dependencies

- SPV_EXT_shader_tile_image

Contact

- Jan-Harald Fredriksen [GitHub]janharaldfredriksen-arm

Extension Proposal

VK_EXT_shader_tile_image

Other Extension Metadata

Last Modified Date

2023-03-23

IP Status

No known IP claims.

Interactions and External Dependencies

- This extension provides API support for `GL_EXT_shader_tile_image`

Contributors

- Sandeep Kakarlapudi, Arm
- Jan-Harald Fredriksen, Arm
- James Fitzpatrick, Imagination
- Andrew Garrard, Imagination
- Jeff Leger, Qualcomm
- Huilong Wang, Huawei
- Graeme Leese, Broadcom
- Hans-Kristian Arntzen, Valve
- Tobias Hector, AMD
- Jeff Bolz, NVIDIA
- Shahbaz Youssefi, Google

Description

This extension allows fragment shader invocations to read color, depth and stencil values at their pixel location in rasterization order. The functionality is only available when using dynamic render passes introduced by VK_KHR_dynamic_rendering. Example use cases are programmable blending and deferred shading.

See fragment shader tile image reads for more information.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceShaderTileImageFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceShaderTileImagePropertiesEXT

New Enum Constants

- `VK_EXT_SHADER_TILE_IMAGE_EXTENSION_NAME`
- `VK_EXT_SHADER_TILE_IMAGE_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TILE_IMAGE_FEATURES_EXT`
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_TILE_IMAGE_PROPERTIES_EXT`

Issues

None.

Examples

Color read example.

```
layout( location = 0 /* aliased to color attachment 0 */ ) tileImageEXT highp
attachmentEXT color0;
layout( location = 1 /* aliased to color attachment 1 */ ) tileImageEXT highp
attachmentEXT color1;

layout( location = 0 ) out vec4 fragColor;

void main()
{
    vec4 value = colorAttachmentReadEXT(color0) + colorAttachmentReadEXT(color1);
    fragColor = value;
}
```

Depth & Stencil read example.

```
void main()
{
    // read sample 0: works for non-MSAA or MSAA targets
    highp float last_depth = depthAttachmentReadEXT();
    lowp uint last_stencil = stencilAttachmentReadEXT();

    //..
}
```

Version History

- Revision 1, 2023-03-23 (Sandeep Kakarlapudi)
  - Initial version

VK_EXT_swapchain_colorspace

Name String

`VK_EXT_swapchain_colorspace`

Extension Type

Instance extension

Registered Extension Number

105

Revision

5

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_surface

Contact

- Courtney Goeltzenleuchter [GitHub]courtney-g

Other Extension Metadata

Last Modified Date

2019-04-26

IP Status

No known IP claims.

Contributors

- Courtney Goeltzenleuchter, Google

Description

This extension expands VkColorSpaceKHR to add support for most standard color spaces beyond VK_COLOR_SPACE_SRGB_NONLINEAR_KHR. This extension also adds support for VK_COLOR_SPACE_PASS_THROUGH_EXT which allows applications to use color spaces not explicitly enumerated in VkColorSpaceKHR.

New Enum Constants

- VK_EXT_SWAPCHAIN_COLOR_SPACE_EXTENSION_NAME
- VK_EXT_SWAPCHAIN_COLOR_SPACE_SPEC_VERSION
- Extending VkColorSpaceKHR:
  - VK_COLOR_SPACE_ADOBERGB_LINEAR_EXT
  - VK_COLOR_SPACE_ADOBERGB_NONLINEAR_EXT
  - VK_COLOR_SPACE_BT2020_LINEAR_EXT
  - VK_COLOR_SPACE_BT709_LINEAR_EXT
  - VK_COLOR_SPACE_BT709_NONLINEAR_EXT
  - VK_COLOR_SPACE_DCI_P3_LINEAR_EXT
  - VK_COLOR_SPACE_DCI_P3_NONLINEAR_EXT
  - VK_COLOR_SPACE_DISPLAY_P3_LINEAR_EXT
  - VK_COLOR_SPACE_DISPLAY_P3_NONLINEAR_EXT
  - VK_COLOR_SPACE_DOLBYVISION_EXT
  - VK_COLOR_SPACE_EXTENDED_SRGB_LINEAR_EXT

    - VK_COLOR_SPACE_EXTENDED_SRGB_NONLINEAR_EXT
    - VK_COLOR_SPACE_HDR10_HLG_EXT
    - VK_COLOR_SPACE_HDR10_ST2084_EXT
    - VK_COLOR_SPACE_PASS_THROUGH_EXT

Issues

1) Does the spec need to specify which kinds of image formats support the color spaces?

RESOLVED: Pixel format is independent of color space (though some color spaces really want / need floating-point color components to be useful). Therefore, do not plan on documenting what formats support which color spaces. An application **can** call vkGetPhysicalDeviceSurfaceFormatsKHR to query what a particular implementation supports.

2) How does application determine if HW supports appropriate transfer function for a color space?

RESOLVED: Extension indicates that implementation **must** not do the OETF encoding if it is not sRGB. That responsibility falls to the application shaders. Any other native OETF / EOTF functions supported by an implementation can be described by separate extension.

Version History

- Revision 1, 2016-12-27 (Courtney Goeltzenleuchter)
    - Initial version
- Revision 2, 2017-01-19 (Courtney Goeltzenleuchter)
    - Add pass through and multiple options for BT2020.
    - Clean up some issues with equations not displaying properly.
- Revision 3, 2017-06-23 (Courtney Goeltzenleuchter)
    - Add extended sRGB non-linear enum.
- Revision 4, 2019-04-26 (Graeme Leese)
    - Clarify color space transfer function usage.
    - Refer to normative definitions in the Data Format Specification.
    - Clarify DCI-P3 and Display P3 usage.
- Revision 5, 2024-03-16 (Zehui Lin)
    - Fix interchanged concepts of EOTF and OETF.
    - Clarify that the presentation engine can accept the color spaces.

VK_EXT_transform_feedback

Name String

VK_EXT_transform_feedback

Extension Type

Device extension

Registered Extension Number

29

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Special Uses

- OpenGL / ES support
- D3D support
- Developer tools

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2018-10-09

Contributors

- Baldur Karlsson, Valve
- Boris Zanin, Mobica
- Daniel Rakos, AMD
- Donald Scorgie, Imagination
- Henri Verbeet, CodeWeavers
- Jan-Harald Fredriksen, Arm
- Faith Ekstrand, Intel
- Jeff Bolz, NVIDIA
- Jesse Barker, Unity
- Jesse Hall, Google
- Pierre-Loup Griffais, Valve
- Philip Rebohle, DXVK

- Ruihao Zhang, Qualcomm
- Samuel Pitoiset, Valve
- Slawomir Grajewski, Intel
- Stu Smith, Imagination Technologies

Description

This extension adds transform feedback to the Vulkan API by exposing the SPIR-V `TransformFeedback` and `GeometryStreams` capabilities to capture vertex, tessellation or geometry shader outputs to one or more buffers. It adds API functionality to bind transform feedback buffers to capture the primitives emitted by the graphics pipeline from SPIR-V outputs decorated for transform feedback. The transform feedback capture can be paused and resumed by way of storing and retrieving a byte counter. The captured data can be drawn again where the vertex count is derived from the byte counter without CPU intervention. If the implementation is capable, a vertex stream other than zero can be rasterized.

All these features are designed to match the full capabilities of OpenGL core transform feedback functionality and beyond. Many of the features are optional to allow base OpenGL ES GPUs to also implement this extension.

The primary purpose of the functionality exposed by this extension is to support translation layers from other 3D APIs. This functionality is not considered forward looking, and is not expected to be promoted to a KHR extension or to core Vulkan. Unless this is needed for translation, it is recommended that developers use alternative techniques of using the GPU to process and capture vertex data.

New Commands

- vkCmdBeginQueryIndexedEXT
- vkCmdBeginTransformFeedbackEXT
- vkCmdBindTransformFeedbackBuffersEXT
- vkCmdDrawIndirectByteCountEXT
- vkCmdEndQueryIndexedEXT
- vkCmdEndTransformFeedbackEXT

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceTransformFeedbackFeaturesEXT
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDeviceTransformFeedbackPropertiesEXT
- Extending VkPipelineRasterizationStateCreateInfo:
  - VkPipelineRasterizationStateStreamCreateInfoEXT

New Bitmasks

- VkPipelineRasterizationStateStreamCreateFlagsEXT

New Enum Constants

- VK_EXT_TRANSFORM_FEEDBACK_EXTENSION_NAME
- VK_EXT_TRANSFORM_FEEDBACK_SPEC_VERSION
- Extending VkAccessFlagBits:
  - VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT
  - VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT
  - VK_ACCESS_TRANSFORM_FEEDBACK_WRITE_BIT_EXT
- Extending VkBufferUsageFlagBits:
  - VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_BUFFER_BIT_EXT
  - VK_BUFFER_USAGE_TRANSFORM_FEEDBACK_COUNTER_BUFFER_BIT_EXT
- Extending VkPipelineStageFlagBits:
  - VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT
- Extending VkQueryType:
  - VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TRANSFORM_FEEDBACK_FEATURES_EXT
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_TRANSFORM_FEEDBACK_PROPERTIES_EXT
  - VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_STREAM_CREATE_INFO_EXT

Issues

1) Should we include pause/resume functionality?

RESOLVED: Yes, this is needed to ease layering other APIs which have this functionality. To pause use vkCmdEndTransformFeedbackEXT and provide valid buffer handles in the pCounterBuffers array and offsets in the pCounterBufferOffsets array for the implementation to save the resume points. Then to resume use vkCmdBeginTransformFeedbackEXT with the previous pCounterBuffers and pCounterBufferOffsets values. Between the pause and resume there needs to be a memory barrier for the counter buffers with a source access of VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_WRITE_BIT_EXT at pipeline stage VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT to a destination access of VK_ACCESS_TRANSFORM_FEEDBACK_COUNTER_READ_BIT_EXT at pipeline stage VK_PIPELINE_STAGE_TRANSFORM_FEEDBACK_BIT_EXT.

2) How does this interact with multiview?

RESOLVED: Transform feedback cannot be made active in a render pass with multiview enabled.

3) How should queries be done?

RESOLVED: There is a new query type `VK_QUERY_TYPE_TRANSFORM_FEEDBACK_STREAM_EXT`. A query pool created with this type will capture 2 integers - numPrimitivesWritten and numPrimitivesNeeded - for the specified vertex stream output from the last pre-rasterization shader stage. The vertex stream output queried is zero by default, but can be specified with the new `vkCmdBeginQueryIndexedEXT` and `vkCmdEndQueryIndexedEXT` commands.

Version History

- Revision 1, 2018-10-09 (Piers Daniell)
  - Internal revisions

VK_EXT_validation_cache

Name String

`VK_EXT_validation_cache`

Extension Type

Device extension

Registered Extension Number

161

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Contact

- Cort Stratton [GitHub]cdwfs

Other Extension Metadata

Last Modified Date

2017-08-29

IP Status

No known IP claims.

Contributors

- Cort Stratton, Google
- Chris Forbes, Google

Description

This extension provides a mechanism for caching the results of potentially expensive internal validation operations across multiple runs of a Vulkan application. At the core is the VkValidationCacheEXT object type, which is managed similarly to the existing VkPipelineCache.

The new struct VkShaderModuleValidationCacheCreateInfoEXT can be included in the `pNext` chain at vkCreateShaderModule time. It contains a VkValidationCacheEXT to use when validating the VkShaderModule.

New Object Types

- VkValidationCacheEXT

New Commands

- vkCreateValidationCacheEXT
- vkDestroyValidationCacheEXT
- vkGetValidationCacheDataEXT
- vkMergeValidationCachesEXT

New Structures

- VkValidationCacheCreateInfoEXT
- Extending VkShaderModuleCreateInfo, VkPipelineShaderStageCreateInfo:
  - VkShaderModuleValidationCacheCreateInfoEXT

New Enums

- VkValidationCacheHeaderVersionEXT

New Bitmasks

- VkValidationCacheCreateFlagsEXT

New Enum Constants

- `VK_EXT_VALIDATION_CACHE_EXTENSION_NAME`
- `VK_EXT_VALIDATION_CACHE_SPEC_VERSION`
- Extending VkObjectType:
  - `VK_OBJECT_TYPE_VALIDATION_CACHE_EXT`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_SHADER_MODULE_VALIDATION_CACHE_CREATE_INFO_EXT`
  - `VK_STRUCTURE_TYPE_VALIDATION_CACHE_CREATE_INFO_EXT`

Version History

- Revision 1, 2017-08-29 (Cort Stratton)
  - Initial draft

VK_EXT_vertex_input_dynamic_state

Name String

`VK_EXT_vertex_input_dynamic_state`

Extension Type

Device extension

Registered Extension Number

353

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2020-08-21

IP Status

No known IP claims.

Contributors

- Jeff Bolz, NVIDIA
- Spencer Fricke, Samsung
- Stu Smith, AMD

Description

One of the states that contributes to the combinatorial explosion of pipeline state objects that need to be created, is the vertex input binding and attribute descriptions. By allowing them to be dynamic applications may reduce the number of pipeline objects they need to create.

This extension adds dynamic state support for what is normally static state in VkPipelineVertexInputStateCreateInfo.

New Commands

- vkCmdSetVertexInputEXT

New Structures

- VkVertexInputAttributeDescription2EXT
- VkVertexInputBindingDescription2EXT
- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceVertexInputDynamicStateFeaturesEXT

New Enum Constants

- VK_EXT_VERTEX_INPUT_DYNAMIC_STATE_EXTENSION_NAME
- VK_EXT_VERTEX_INPUT_DYNAMIC_STATE_SPEC_VERSION
- Extending VkDynamicState:
  - VK_DYNAMIC_STATE_VERTEX_INPUT_EXT
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VERTEX_INPUT_DYNAMIC_STATE_FEATURES_EXT
  - VK_STRUCTURE_TYPE_VERTEX_INPUT_ATTRIBUTE_DESCRIPTION_2_EXT
  - VK_STRUCTURE_TYPE_VERTEX_INPUT_BINDING_DESCRIPTION_2_EXT

Version History

- Revision 2, 2020-11-05 (Piers Daniell)
  - Make VkVertexInputBindingDescription2EXT extensible
  - Add new VkVertexInputAttributeDescription2EXT struct for the pVertexAttributeDescriptions parameter to vkCmdSetVertexInputEXT so it is also extensible
- Revision 1, 2020-08-21 (Piers Daniell)
  - Internal revisions

VK_EXT_ycbcr_image_arrays

Name String

VK_EXT_ycbcr_image_arrays

Extension Type

Device extension

Registered Extension Number

253

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_sampler_ycbcr_conversion
or
Vulkan Version 1.1

Contact

- Piers Daniell [GitHub]pdaniell-nv

Other Extension Metadata

Last Modified Date

2019-01-15

Contributors

- Piers Daniell, NVIDIA

Description

This extension allows images of a format that requires $Y'C_BC_R$ conversion to be created with multiple array layers, which is otherwise restricted.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDeviceYcbcrImageArraysFeaturesEXT

New Enum Constants

- `VK_EXT_YCBCR_IMAGE_ARRAYS_EXTENSION_NAME`
- `VK_EXT_YCBCR_IMAGE_ARRAYS_SPEC_VERSION`
- Extending VkStructureType:
  - `VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_YCBCR_IMAGE_ARRAYS_FEATURES_EXT`

Version History

- Revision 1, 2019-01-15 (Piers Daniell)
  - Initial revision

List of Provisional Extensions

- VK_KHR_portability_subset

VK_KHR_portability_subset

Name String

`VK_KHR_portability_subset`

Extension Type

Device extension

Registered Extension Number

164

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

VK_KHR_get_physical_device_properties2
or
Vulkan Version 1.1

- **This is a *provisional* extension and must be used with caution. See the description of provisional header files for enablement and stability details.**

Contact

- Bill Hollings [GitHub]billhollings

Other Extension Metadata

Last Modified Date

2020-07-21

IP Status

No known IP claims.

Contributors

- Bill Hollings, The Brenwill Workshop Ltd.
- Daniel Koch, NVIDIA
- Dzmitry Malyshau, Mozilla
- Chip Davis, CodeWeavers
- Dan Ginsburg, Valve
- Mike Weiblen, LunarG
- Neil Trevett, NVIDIA
- Alexey Knyazev, Independent

Description

The VK_KHR_portability_subset extension allows a non-conformant Vulkan implementation to be built on top of another non-Vulkan graphics API, and identifies differences between that implementation and a fully-conformant native Vulkan implementation.

This extension provides Vulkan implementations with the ability to mark otherwise-required capabilities as unsupported, or to establish additional properties and limits that the application should adhere to in order to guarantee portable behavior and operation across platforms, including platforms where Vulkan is not natively supported.

The goal of this specification is to document, and make queryable, capabilities which are required to be supported by a fully-conformant Vulkan 1.0 implementation, but may be optional for an implementation of the Vulkan 1.0 Portability Subset.

The intent is that this extension will be advertised only on implementations of the Vulkan 1.0 Portability Subset, and not on conformant implementations of Vulkan 1.0. Fully-conformant Vulkan implementations provide all the required capabilities, and so will not provide this extension. Therefore, the existence of this extension can be used to determine that an implementation is likely not fully conformant with the Vulkan spec.

If this extension is supported by the Vulkan implementation, the application must enable this extension.

This extension defines several new structures that can be chained to the existing structures used by certain standard Vulkan calls, in order to query for non-conformant portable behavior.

New Structures

- Extending VkPhysicalDeviceFeatures2, VkDeviceCreateInfo:
  - VkPhysicalDevicePortabilitySubsetFeaturesKHR
- Extending VkPhysicalDeviceProperties2:
  - VkPhysicalDevicePortabilitySubsetPropertiesKHR

New Enum Constants

- VK_KHR_PORTABILITY_SUBSET_EXTENSION_NAME
- VK_KHR_PORTABILITY_SUBSET_SPEC_VERSION
- Extending VkStructureType:
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PORTABILITY_SUBSET_FEATURES_KHR
  - VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PORTABILITY_SUBSET_PROPERTIES_KHR

Issues

None.

Version History

- Revision 1, 2020-07-21 (Bill Hollings)
  - Initial draft.

List of Deprecated Extensions

- VK_EXT_load_store_op_none

VK_EXT_load_store_op_none

Name String

`VK_EXT_load_store_op_none`

Extension Type

Device extension

Registered Extension Number

401

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

None

Deprecation State

- *Promoted* to VK_KHR_load_store_op_none extension

Contact

- Shahbaz Youssefi [GitHub]syoussefi

Other Extension Metadata

Last Modified Date

2021-06-06

Contributors

- Shahbaz Youssefi, Google
- Bill Licea-Kane, Qualcomm Technologies, Inc.
- Tobias Hector, AMD

Description

This extension incorporates `VK_ATTACHMENT_STORE_OP_NONE_EXT` from `VK_QCOM_render_pass_store_ops`, enabling applications to avoid unnecessary synchronization when an attachment is not written during a render pass.

Additionally, `VK_ATTACHMENT_LOAD_OP_NONE_EXT` is introduced to avoid unnecessary synchronization when an attachment is not used during a render pass at all. In combination with `VK_ATTACHMENT_STORE_OP_NONE_EXT`, this is useful as an alternative to preserve attachments in applications that cannot decide if an attachment will be used in a render pass until after the necessary pipelines have been created.

Promotion to VK_KHR_load_store_op_none

All functionality in this extension is included in VK_KHR_load_store_op_none, with the suffix changed to KHR. The original enum names are still available as aliases of the KHR functionality.

New Enum Constants

- VK_EXT_LOAD_STORE_OP_NONE_EXTENSION_NAME
- VK_EXT_LOAD_STORE_OP_NONE_SPEC_VERSION
- Extending VkAttachmentLoadOp:
  - VK_ATTACHMENT_LOAD_OP_NONE_EXT
- Extending VkAttachmentStoreOp:
  - VK_ATTACHMENT_STORE_OP_NONE_EXT

> **NOTE** While VK_ATTACHMENT_STORE_OP_NONE is part of Vulkan 1.3, this extension was not promoted to core either in whole or in part. This functionality was promoted from VK_KHR_dynamic_rendering.

Version History

- Revision 1, 2021-06-06 (Shahbaz Youssefi)
  - Initial revision, based on VK_QCOM_render_pass_store_ops.
  - Added VK_ATTACHMENT_LOAD_OP_NONE_EXT.

Appendix F: API Boilerplate

This appendix defines Vulkan API features that are infrastructure required for a complete functional description of Vulkan, but do not logically belong elsewhere in the Specification.

Vulkan Header Files

Vulkan is defined as an API in the C99 language. Khronos provides a corresponding set of header files for applications using the API, which may be used in either C or C++ code. The interface descriptions in the specification are the same as the interfaces defined in these header files, and both are derived from the `vk.xml` XML API Registry, which is the canonical machine-readable description of the Vulkan API. The Registry, scripts used for processing it into various forms, and documentation of the registry schema are available as described at https://registry.khronos.org/vulkan/#apiregistry .

Language bindings for other languages can be defined using the information in the Specification and the Registry. Khronos does not provide any such bindings, but third-party developers have created some additional bindings.

Vulkan Combined API Header `vulkan.h` (Informative)

Applications normally will include the header `vulkan.h`. In turn, `vulkan.h` always includes the following headers:

- `vk_platform.h`, defining platform-specific macros and headers.
- `vulkan_core.h`, defining APIs for the Vulkan core and all registered extensions *other* than window system-specific and provisional extensions, which are included in separate header files.

In addition, specific preprocessor macros defined at the time `vulkan.h` is included cause header files for the corresponding window system-specific and provisional interfaces to be included, as described below.

Vulkan Platform-Specific Header `vk_platform.h` (Informative)

Platform-specific macros and interfaces are defined in `vk_platform.h`. These macros are used to control platform-dependent behavior, and their exact definitions are under the control of specific platforms and Vulkan implementations.

Platform-Specific Calling Conventions

On many platforms the following macros are empty strings, causing platform- and compiler-specific default calling conventions to be used.

`VKAPI_ATTR` is a macro placed before the return type in Vulkan API function declarations. This macro controls calling conventions for C++11 and GCC/Clang-style compilers.

`VKAPI_CALL` is a macro placed after the return type in Vulkan API function declarations. This macro controls calling conventions for MSVC-style compilers.

VKAPI_PTR is a macro placed between the '(' and '*' in Vulkan API function pointer declarations. This macro also controls calling conventions, and typically has the same definition as VKAPI_ATTR or VKAPI_CALL, depending on the compiler.

With these macros, a Vulkan function declaration takes the form of:

```
VKAPI_ATTR <return_type> VKAPI_CALL <command_name>(<command_parameters>);
```

Additionally, a Vulkan function pointer type declaration takes the form of:

```
typedef <return_type> (VKAPI_PTR *PFN_<command_name>)(<command_parameters>);
```

Platform-Specific Header Control

If the VK_NO_STDINT_H macro is defined by the application at compile time, extended integer types used by the Vulkan API, such as uint8_t, **must** also be defined by the application. Otherwise, the Vulkan headers will not compile. If VK_NO_STDINT_H is not defined, the system <stdint.h> is used to define these types. There is a fallback path when Microsoft Visual Studio version 2008 and earlier versions are detected at compile time.

If the VK_NO_STDDEF_H macro is defined by the application at compile time, size_t, **must** also be defined by the application. Otherwise, the Vulkan headers will not compile. If VK_NO_STDDEF_H is not defined, the system <stddef.h> is used to define this type.

Vulkan Core API Header vulkan_core.h

Applications that do not make use of window system-specific extensions may simply include vulkan_core.h instead of vulkan.h, although there is usually no reason to do so. In addition to the Vulkan API, vulkan_core.h also defines a small number of C preprocessor macros that are described below.

Vulkan Header File Version Number

VK_HEADER_VERSION is the version number of the vulkan_core.h header. This value is kept synchronized with the patch version of the released Specification.

```
// Provided by VK_VERSION_1_0
// Version of this file
#define VK_HEADER_VERSION 302
```

VK_HEADER_VERSION_COMPLETE is the complete version number of the vulkan_core.h header, comprising the major, minor, and patch versions. The major/minor values are kept synchronized with the complete version of the released Specification. This value is intended for use by automated tools to identify exactly which version of the header was used during their generation.

Applications should not use this value as their VkApplicationInfo::apiVersion. Instead applications

should explicitly select a specific fixed major/minor API version using, for example, one of the VK_API_VERSION_*_* values.

```
// Provided by VK_VERSION_1_0
// Complete version of this file
#define VK_HEADER_VERSION_COMPLETE VK_MAKE_API_VERSION(0, 1, 3, VK_HEADER_VERSION)
```

VK_API_VERSION is now commented out of vulkan_core.h and **cannot** be used.

```
// Provided by VK_VERSION_1_0
// DEPRECATED: This define has been removed. Specific version defines (e.g.
VK_API_VERSION_1_0), or the VK_MAKE_VERSION macro, should be used instead.
//#define VK_API_VERSION VK_MAKE_API_VERSION(0, 1, 0, 0) // Patch version should
always be set to 0
```

Vulkan Handle Macros

VK_DEFINE_HANDLE defines a dispatchable handle type.

```
// Provided by VK_VERSION_1_0

#define VK_DEFINE_HANDLE(object) typedef struct object##_T* object;
```

- object is the name of the resulting C type.

The only dispatchable handle types are those related to device and instance management, such as VkDevice.

VK_DEFINE_NON_DISPATCHABLE_HANDLE defines a non-dispatchable handle type.

```
// Provided by VK_VERSION_1_0

#ifndef VK_DEFINE_NON_DISPATCHABLE_HANDLE
    #if (VK_USE_64_BIT_PTR_DEFINES==1)
        #define VK_DEFINE_NON_DISPATCHABLE_HANDLE(object) typedef struct object##_T
*object;
    #else
        #define VK_DEFINE_NON_DISPATCHABLE_HANDLE(object) typedef uint64_t object;
    #endif
#endif
```

- object is the name of the resulting C type.

Most Vulkan handle types, such as VkBuffer, are non-dispatchable.

NOTE The vulkan_core.h header allows the VK_DEFINE_NON_DISPATCHABLE_HANDLE

> and VK_NULL_HANDLE definitions to be overridden by the application. If VK_DEFINE_NON_DISPATCHABLE_HANDLE is already defined when vulkan_core.h is compiled, the default definitions for VK_DEFINE_NON_DISPATCHABLE_HANDLE and VK_NULL_HANDLE are skipped. This allows the application to define a binary-compatible custom handle which **may** provide more type-safety or other features needed by the application. Applications **must** not define handles in a way that is not binary compatible - where binary compatibility is platform dependent.

VK_NULL_HANDLE is a reserved value representing a non-valid object handle. It may be passed to and returned from Vulkan commands only when specifically allowed.

```
// Provided by VK_VERSION_1_0

#ifndef VK_DEFINE_NON_DISPATCHABLE_HANDLE
    #if (VK_USE_64_BIT_PTR_DEFINES==1)
        #if (defined(__cplusplus) && (__cplusplus >= 201103L)) || (defined(_MSVC_LANG)
&& (_MSVC_LANG >= 201103L))
            #define VK_NULL_HANDLE nullptr
        #else
            #define VK_NULL_HANDLE ((void*)0)
        #endif
    #else
        #define VK_NULL_HANDLE 0ULL
    #endif
#endif
#ifndef VK_NULL_HANDLE
    #define VK_NULL_HANDLE 0
#endif
```

VK_USE_64_BIT_PTR_DEFINES defines whether the default non-dispatchable handles are declared using either a 64-bit pointer type or a 64-bit unsigned integer type.

VK_USE_64_BIT_PTR_DEFINES is set to '1' to use a 64-bit pointer type or any other value to use a 64-bit unsigned integer type.

```
// Provided by VK_VERSION_1_0

#ifndef VK_USE_64_BIT_PTR_DEFINES
    #if defined(__LP64__) || defined(_WIN64) || (defined(__x86_64__) &&
!defined(__ILP32__) ) || defined(_M_X64) || defined(__ia64) || defined (_M_IA64) ||
defined(__aarch64__) || defined(__powerpc64__) || (defined(__riscv) && __riscv_xlen ==
64)
        #define VK_USE_64_BIT_PTR_DEFINES 1
    #else
        #define VK_USE_64_BIT_PTR_DEFINES 0
    #endif
#endif
```

NOTE

The `vulkan_core.h` header allows the `VK_USE_64_BIT_PTR_DEFINES` definition to be overridden by the application. This allows the application to select either a 64-bit pointer type or a 64-bit unsigned integer type for non-dispatchable handles in the case where the predefined preprocessor check does not identify the desired configuration.

NOTE

This macro was introduced starting with the Vulkan 1.2.174 headers, and its availability can be checked at compile time by requiring `VK_HEADER_VERSION >= 174`.

It is not available if you are using older headers, such as may be shipped with an older Vulkan SDK. Developers requiring this functionality may wish to include a copy of the current Vulkan headers with their project in this case.

Window System-Specific Header Control (Informative)

To use a Vulkan extension supporting a platform-specific window system, header files for that window system **must** be included at compile time, or platform-specific types **must** be forward-declared. The Vulkan header files are unable to determine whether or not an external header is available at compile time, so platform-specific extensions are provided in separate headers from the core API and platform-independent extensions, allowing applications to decide which ones they need to be defined and how the external headers are included.

Extensions dependent on particular sets of platform headers, or that forward-declare platform-specific types, are declared in a header named for that platform. Before including these platform-specific Vulkan headers, applications **must** include both `vulkan_core.h` and any external native headers the platform extensions depend on.

As a convenience for applications that do not need the flexibility of separate platform-specific Vulkan headers, `vulkan.h` includes `vulkan_core.h`, and then conditionally includes platform-specific Vulkan headers and the external headers they depend on. Applications control which platform-specific headers are included by #defining macros before including `vulkan.h`.

The correspondence between platform-specific extensions, external headers they require, the platform-specific header which declares them, and the preprocessor macros which enable inclusion by `vulkan.h` are shown in the following table.

Table 102. Window System Extensions and Headers

Extension Name	Window System Name	Platform-specific Header	Required External Headers	Controlling `vulkan.h` Macro
`VK_KHR_android_surface`	Android	`vulkan_android.h`	None	`VK_USE_PLATFORM_ANDROID_KHR`
`VK_KHR_wayland_surface`	Wayland	`vulkan_wayland.h`	`<wayland-client.h>`	`VK_USE_PLATFORM_WAYLAND_KHR`

Extension Name	Window System Name	Platform-specific Header	Required External Headers	Controlling vulkan.h Macro
VK_KHR_win32_surface, VK_KHR_external_memory_win32, VK_KHR_win32_keyed_mutex, VK_KHR_external_semaphore_win32, VK_KHR_external_fence_win32, VK_NV_external_memory_win32, VK_NV_win32_keyed_mutex	Microsoft Windows	vulkan_win32.h	<windows.h>	VK_USE_PLATFORM_WIN32_KHR
VK_KHR_xcb_surface	X11 Xcb	vulkan_xcb.h	<xcb/xcb.h>	VK_USE_PLATFORM_XCB_KHR
VK_KHR_xlib_surface	X11 Xlib	vulkan_xlib.h	<X11/Xlib.h>	VK_USE_PLATFORM_XLIB_KHR
VK_EXT_directfb_surface	DirectFB	vulkan_directfb.h	<directfb/directfb.h>	VK_USE_PLATFORM_DIRECTFB_EXT
VK_EXT_acquire_xlib_display	X11 XRAndR	vulkan_xlib_xrandr.h	<X11/Xlib.h>, <X11/extensions/Xrandr.h>	VK_USE_PLATFORM_XLIB_XRANDR_EXT
VK_GGP_stream_descriptor_surface, VK_GGP_frame_token	Google Games Platform	vulkan_ggp.h	<ggp_c/vulkan_types.h>	VK_USE_PLATFORM_GGP
VK_MVK_ios_surface	iOS	vulkan_ios.h	None	VK_USE_PLATFORM_IOS_MVK
VK_MVK_macos_surface	macOS	vulkan_macos.h	None	VK_USE_PLATFORM_MACOS_MVK
VK_NN_vi_surface	VI	vulkan_vi.h	None	VK_USE_PLATFORM_VI_NN
VK_FUCHSIA_imagepipe_surface	Fuchsia	vulkan_fuchsia.h	<zircon/types.h>	VK_USE_PLATFORM_FUCHSIA
VK_EXT_metal_surface	Metal on CoreAnimation	vulkan_metal.h	None	VK_USE_PLATFORM_METAL_EXT
VK_QNX_screen_surface	QNX Screen	vulkan_screen.h	<screen/screen.h>	VK_USE_PLATFORM_SCREEN_QNX

NOTE This section describes the purpose of the headers independently of the specific underlying functionality of the window system extensions themselves. Each extension name will only link to a description of that extension when viewing a specification built with that extension included.

Provisional Extension Header Control (Informative)

Provisional extensions **should** not be used in production applications. The functionality defined by such extensions **may** change in ways that break backwards compatibility between revisions, and before final release of a non-provisional version of that extension.

Provisional extensions are defined in a separate *provisional header*, `vulkan_beta.h`, allowing applications to decide whether or not to include them. The mechanism is similar to window system-specific headers: before including `vulkan_beta.h`, applications **must** include `vulkan_core.h`.

NOTE Sometimes a provisional extension will include a subset of its interfaces in `vulkan_core.h`. This may occur if the provisional extension is promoted from an existing vendor or EXT extension and some of the existing interfaces are defined as aliases of the provisional extension interfaces. All other interfaces of that provisional extension which are not aliased will be included in `vulkan_beta.h`.

As a convenience for applications, `vulkan.h` conditionally includes `vulkan_beta.h`. Applications **can** control inclusion of `vulkan_beta.h` by #defining the macro `VK_ENABLE_BETA_EXTENSIONS` before including `vulkan.h`.

NOTE Starting in version 1.2.171 of the Specification, all provisional enumerants are protected by the macro `VK_ENABLE_BETA_EXTENSIONS`. Applications needing to use provisional extensions must always define this macro, even if they are explicitly including `vulkan_beta.h`. This is a minor change to behavior, affecting only provisional extensions.

NOTE This section describes the purpose of the provisional header independently of the specific provisional extensions which are contained in that header at any given time. The extension appendices for provisional extensions note their provisional status, and link back to this section for more information. Provisional extensions are intended to provide early access for bleeding-edge developers, with the understanding that extension interfaces may change in response to developer feedback. Provisional extensions are very likely to eventually be updated and released as non-provisional extensions, but there is no guarantee this will happen, or how long it will take if it does happen.

Video Std Headers

Performing video coding operations usually involves the application having to provide various parameters, data structures, or other syntax elements specific to the particular video compression standard used, and the associated semantics are covered by the specification of those.

The interface descriptions of these are available in the header files derived from the `video.xml` XML file, which is the canonical machine-readable description of data structures and enumerations that are associated with the externally-provided video compression standards.

Table 103. Video Std Headers

Video Std Header Name	Description	Header File	Related Extensions
vulkan_video_codecs_common	Codec-independent common definitions	<vk_video/vulkan_video_codecs_common.h>	-
vulkan_video_codec_h264std	ITU-T H.264 common definitions	<vk_video/vulkan_video_codec_h264std.h>	VK_KHR_video_decode_h264, VK_KHR_video_encode_h264
vulkan_video_codec_h264std_decode	ITU-T H.264 decode-specific definitions	<vk_video/vulkan_video_codec_h264std_decode.h>	VK_KHR_video_decode_h264
vulkan_video_codec_h264std_encode	ITU-T H.264 encode-specific definitions	<vk_video/vulkan_video_codec_h264std_encode.h>	VK_KHR_video_encode_h264
vulkan_video_codec_h265std	ITU-T H.265 common definitions	<vk_video/vulkan_video_codec_h265std.h>	VK_KHR_video_decode_h265, VK_KHR_video_encode_h265
vulkan_video_codec_h265std_decode	ITU-T H.265 decode-specific definitions	<vk_video/vulkan_video_codec_h265std_decode.h>	VK_KHR_video_decode_h265
vulkan_video_codec_h265std_encode	ITU-T H.265 encode-specific definitions	<vk_video/vulkan_video_codec_h265std_encode.h>	VK_KHR_video_encode_h265
vulkan_video_codec_av1std	AV1 common definitions	<vk_video/vulkan_video_codec_av1std.h>	VK_KHR_video_decode_av1
vulkan_video_codec_av1std_decode	AV1 decode-specific definitions	<vk_video/vulkan_video_codec_av1std_decode.h>	VK_KHR_video_decode_av1
vulkan_video_codec_av1std_encode	AV1 encode-specific definitions	<vk_video/vulkan_video_codec_av1std_encode.h>	VK_KHR_video_encode_av1

Appendix G: Invariance

The Vulkan specification is not pixel exact. It therefore does not guarantee an exact match between images produced by different Vulkan implementations. However, the specification does specify exact matches, in some cases, for images produced by the same implementation. The purpose of this appendix is to identify and provide justification for those cases that require exact matches.

Repeatability

The obvious and most fundamental case is repeated issuance of a series of Vulkan commands. For any given Vulkan and framebuffer state vector, and for any Vulkan command, the resulting Vulkan and framebuffer state **must** be identical whenever the command is executed on that initial Vulkan and framebuffer state. This repeatability requirement does not apply when using shaders containing side effects (image and buffer variable stores and atomic operations), because these memory operations are not guaranteed to be processed in a defined order.

One purpose of repeatability is avoidance of visual artifacts when a double-buffered scene is redrawn. If rendering is not repeatable, swapping between two buffers rendered with the same command sequence **may** result in visible changes in the image. Such false motion is distracting to the viewer. Another reason for repeatability is testability.

Repeatability, while important, is a weak requirement. Given only repeatability as a requirement, two scenes rendered with one (small) polygon changed in position might differ at every pixel. Such a difference, while within the law of repeatability, is certainly not within its spirit. Additional invariance rules are desirable to ensure useful operation.

Multi-Pass Algorithms

Invariance is necessary for a whole set of useful multi-pass algorithms. Such algorithms render multiple times, each time with a different Vulkan mode vector, to eventually produce a result in the framebuffer. Examples of these algorithms include:

- “Erasing” a primitive from the framebuffer by redrawing it, either in a different color or using the XOR logical operation.
- Using stencil operations to compute capping planes.

Invariance Rules

For a given Vulkan device:

Rule 1 *For any given Vulkan and framebuffer state vector, and for any given Vulkan command, the resulting Vulkan and framebuffer state **must** be identical each time the command is executed on that initial Vulkan and framebuffer state.*

Rule 2 *Changes to the following state values have no side effects (the use of any other state value is not affected by the change):*

Required:

- *Color and depth/stencil attachment contents*
- *Scissor parameters (other than enable)*
- *Write masks (color, depth, stencil)*
- *Clear values (color, depth, stencil)*

Strongly suggested:

- *Stencil parameters (other than enable)*
- *Depth test parameters (other than enable)*
- *Blend parameters (other than enable)*
- *Logical operation parameters (other than enable)*

Corollary 1 *Fragment generation is invariant with respect to the state values listed in Rule 2.*

Rule 3 *The arithmetic of each per-fragment operation is invariant except with respect to parameters that directly control it.*

Corollary 2 *Images rendered into different color attachments of the same framebuffer, either simultaneously or separately using the same command sequence, are pixel identical.*

Rule 4 *Identical pipelines will produce the same result when run multiple times with the same input. The wording "Identical pipelines" means VkPipeline objects that have been created with identical SPIR-V binaries and identical state, which are then used by commands executed using the same Vulkan state vector. Invariance is relaxed for shaders with side effects, such as performing stores or atomics.*

Rule 5 *All fragment shaders that either conditionally or unconditionally assign* `FragCoord.z` *to* `FragDepth` *are depth-invariant with respect to each other, for those fragments where the assignment to* `FragDepth` *actually is done.*

If a sequence of Vulkan commands specifies primitives to be rendered with shaders containing side effects (image and buffer variable stores and atomic operations), invariance rules are relaxed. In particular, rule 1, corollary 2, and rule 4 do not apply in the presence of shader side effects.

The following weaker versions of rules 1 and 4 apply to Vulkan commands involving shader side effects:

Rule 6 *For any given Vulkan and framebuffer state vector, and for any given Vulkan command, the contents of any framebuffer state not directly or indirectly affected by results of shader image or buffer variable stores or atomic operations* ***must*** *be identical each time the command is executed on that initial Vulkan and framebuffer state.*

Rule 7 *Identical pipelines will produce the same result when run multiple times with the same input as long as:*

- *shader invocations do not use image atomic operations;*

- *no framebuffer memory is written to more than once by image stores, unless all such stores write the same value; and*
- *no shader invocation, or other operation performed to process the sequence of commands, reads memory written to by an image store.*

NOTE

The OpenGL specification has the following invariance rule: Consider a primitive p' obtained by translating a primitive p through an offset (x, y) in window coordinates, where x and y are integers. As long as neither p' nor p is clipped, it **must** be the case that each fragment f' produced from p' is identical to a corresponding fragment f from p except that the center of f' is offset by (x, y) from the center of f.

This rule does not apply to Vulkan and is an intentional difference from OpenGL.

When any sequence of Vulkan commands triggers shader invocations that perform image stores or atomic operations, and subsequent Vulkan commands read the memory written by those shader invocations, these operations **must** be explicitly synchronized.

Tessellation Invariance

When using a pipeline containing tessellation evaluation shaders, the fixed-function tessellation primitive generator consumes the input patch specified by an application and emits a new set of primitives. The following invariance rules are intended to provide repeatability guarantees. Additionally, they are intended to allow an application with a carefully crafted tessellation evaluation shader to ensure that the sets of triangles generated for two adjacent patches have identical vertices along shared patch edges, avoiding “cracks” caused by minor differences in the positions of vertices along shared edges.

Rule 1 *When processing two patches with identical outer and inner tessellation levels, the tessellation primitive generator will emit an identical set of point, line, or triangle primitives as long as the pipeline used to process the patch primitives has tessellation evaluation shaders specifying the same tessellation mode, spacing, vertex order, and point mode decorations. Two sets of primitives are considered identical if and only if they contain the same number and type of primitives and the generated tessellation coordinates for the vertex numbered m of the primitive numbered n are identical for all values of m and n.*

Rule 2 *The set of vertices generated along the outer edge of the subdivided primitive in triangle and quad tessellation, and the tessellation coordinates of each, depend only on the corresponding outer tessellation level and the spacing decorations in the tessellation shaders of the pipeline.*

Rule 3 *The set of vertices generated when subdividing any outer primitive edge is always symmetric. For triangle tessellation, if the subdivision generates a vertex with tessellation coordinates of the form (0, x, 1-x), (x, 0, 1-x), or (x, 1-x, 0), it will also generate a vertex with coordinates of exactly (0, 1-x, x), (1-x, 0, x), or (1-x, x, 0), respectively. For quad tessellation, if the subdivision generates a vertex with coordinates of (x, 0) or (0, x), it will also generate a vertex with coordinates of exactly (1-x, 0) or (0, 1-x), respectively. For isoline tessellation, if it generates vertices at (0, x) and (1, x) where x is not zero, it will also generate vertices at exactly (0, 1-x) and (1, 1-x), respectively.*

Rule 4 *The set of vertices generated when subdividing outer edges in triangular and quad tessellation*

must *be independent of the specific edge subdivided, given identical outer tessellation levels and spacing. For example, if vertices at (x, 1 - x, 0) and (1-x, x, 0) are generated when subdividing the w = 0 edge in triangular tessellation, vertices* ***must*** *be generated at (x, 0, 1-x) and (1-x, 0, x) when subdividing an otherwise identical v = 0 edge. For quad tessellation, if vertices at (x, 0) and (1-x, 0) are generated when subdividing the v = 0 edge, vertices* ***must*** *be generated at (0, x) and (0, 1-x) when subdividing an otherwise identical u = 0 edge.*

Rule 5 *When processing two patches that are identical in all respects enumerated in rule 1 except for vertex order, the set of triangles generated for triangle and quad tessellation* ***must*** *be identical except for vertex and triangle order. For each triangle n1 produced by processing the first patch, there* ***must*** *be a triangle n2 produced when processing the second patch each of whose vertices has the same tessellation coordinates as one of the vertices in n1.*

Rule 6 *When processing two patches that are identical in all respects enumerated in rule 1 other than matching outer tessellation levels and/or vertex order, the set of interior triangles generated for triangle and quad tessellation* ***must*** *be identical in all respects except for vertex and triangle order. For each interior triangle n1 produced by processing the first patch, there* ***must*** *be a triangle n2 produced when processing the second patch each of whose vertices has the same tessellation coordinates as one of the vertices in n1. A triangle produced by the tessellator is considered an interior triangle if none of its vertices lie on an outer edge of the subdivided primitive.*

Rule 7 *For quad and triangle tessellation, the set of triangles connecting an inner and outer edge depends only on the inner and outer tessellation levels corresponding to that edge and the spacing decorations.*

Rule 8 *The value of all defined components of* `TessCoord` *will be in the range [0, 1]. Additionally, for any defined component x of* `TessCoord`*, the results of computing 1.0-x in a tessellation evaluation shader will be exact. If any floating-point values in the range [0, 1] fail to satisfy this property, such values* ***must*** *not be used as tessellation coordinate components.*

Appendix H: Lexicon

This appendix defines terms, abbreviations, and API prefixes used in the Specification.

Glossary

The terms defined in this section are used consistently throughout the Specification and may be used with or without capitalization.

Accessible (Descriptor Binding)

A descriptor binding is accessible to a shader stage if that stage is included in the `stageFlags` of the descriptor binding. Descriptors using that binding **can** only be used by stages in which they are accessible.

Acquire Operation (Resource)

An operation that acquires ownership of an image subresource or buffer range.

Active (Transform Feedback)

Transform feedback is made active after vkCmdBeginTransformFeedbackEXT executes and remains active until vkCmdEndTransformFeedbackEXT executes. While transform feedback is active, data written to variables in the output interface of the last pre-rasterization shader stage of the graphics pipeline are captured to the bound transform feedback buffers if those variables are decorated for transform feedback.

Adjacent Vertex

A vertex in an adjacency primitive topology that is not part of a given primitive, but is accessible in geometry shaders.

Active Object (Ray Tracing)

A primitive or instance in a ray tracing acceleration structure which has a corresponding ID, and is not *inactive* (meaning that it is visible to rays).

Advanced Blend Operation

Blending performed using one of the blend operation enums introduced by the `VK_EXT_blend_operation_advanced` extension. See Advanced Blending Operations.

Alias (API type/command)

An identical definition of another API type/command with the same behavior but a different name.

Aliased Range (Memory)

A range of a device memory allocation that is bound to multiple resources simultaneously.

Allocation Scope

An association of a host memory allocation to a parent object or command, where the allocation's lifetime ends before or at the same time as the parent object is freed or destroyed, or during the parent command.

API command

Any command defined in the Vulkan specification. These entry points all have a vk prefix.

Aspect (Image)

Some image types contain multiple kinds (called “aspects”) of data for each pixel, where each aspect is used in a particular way by the pipeline and **may** be stored differently or separately from other aspects. For example, the color components of an image format make up the color aspect of the image, and **can** be used as a framebuffer color attachment. Some operations, like depth testing, operate only on specific aspects of an image.

Attachment (Render Pass)

A zero-based integer index name used in render pass creation to refer to a framebuffer attachment that is accessed by one or more subpasses. The index also refers to an attachment description which includes information about the properties of the image view that will later be attached.

Availability Operation

An operation that causes the values generated by specified memory write accesses to become available for future access.

Available

A state of values written to memory that allows them to be made visible.

Axis-aligned Bounding Box

A box bounding a region in space defined by extents along each axis and thus representing a box where each edge is aligned to one of the major axes.

Back-Facing

See Facingness.

Backward Prediction

Sample prediction performed during video decode and encode operations based on reference pictures that occur temporally (in display order) after the current picture.

Backward Reference

A reference picture that occurs temporally (in display order) after the current picture.

Batch

A single structure submitted to a queue as part of a queue submission command, describing a set of queue operations to execute.

Backwards Compatibility

A given version of the API is backwards compatible with an earlier version if an application, relying only on valid behavior and functionality defined by the earlier specification, is able to correctly run against each version without any modification. This assumes no active attempt by that application to not run when it detects a different version.

Binary Semaphore

A semaphore with a boolean payload indicating whether the semaphore is signaled or unsignaled. Represented by a VkSemaphore object created with a semaphore type of `VK_SEMAPHORE_TYPE_BINARY` .

Binding (Memory)

An association established between a range of a resource object and a range of a memory object. These associations determine the memory locations affected by operations performed on elements of a resource object. Memory bindings are established using the vkBindBufferMemory command for non-sparse buffer objects, using the vkBindImageMemory command for non-sparse image objects , and using the vkQueueBindSparse command for sparse resources .

Blend Constant

Four floating-point (RGBA) values used as an input to blending.

Blending

Arithmetic operations between a fragment color value and a value in a color attachment that produce a final color value to be written to the attachment.

Buffer

A resource that represents a linear array of data in device memory. Represented by a VkBuffer object.

Buffer Device Address

A 64-bit value used in a shader to access buffer memory through the `PhysicalStorageBuffer` storage class.

Buffer View

An object that represents a range of a specific buffer, and state controlling how the contents are interpreted. Represented by a VkBufferView object.

Built-In Variable

A variable decorated in a shader, where the decoration makes the variable take values provided by the execution environment or values that are generated by fixed-function pipeline stages.

Built-In Interface Block

A block defined in a shader containing only variables decorated with built-in decorations, and is used to match against other shader stages.

Clip Coordinates

The homogeneous coordinate space in which vertex positions (`Position` decoration) are written by pre-rasterization shader stages.

Clip Distance

A built-in output from pre-rasterization shader stages defining a clip half-space against which the primitive is clipped.

Clip Volume

The intersection of the view volume with all clip half-spaces.

Coding Block (Video)

An element of a video picture, comprising a rectangular block of texel values that are coded as a single unit in video coding. Video coding blocks have a corresponding coding block size (width and height) defining the dimensions of these elements in units of texels. These dimensions are usually codec-specific and a single video compression standard **may** support more than one coding block type and size. Video compression standards **may** use multiple coding block types to organize coding blocks of different kinds and sizes into hierarchies.

Color Attachment

A subpass attachment point, or image view, that is the target of fragment color outputs and blending.

Color Renderable Format

A VkFormat where `VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT` is set in one of the following, depending on the image's tiling:

- VkFormatProperties::`linearTilingFeatures`
- VkFormatProperties::`optimalTilingFeatures`
- VkDrmFormatModifierPropertiesEXT::`drmFormatModifierTilingFeatures`

Combined Image Sampler

A descriptor type that includes both a sampled image and a sampler.

Command Buffer

An object that records commands to be submitted to a queue. Represented by a VkCommandBuffer object.

Command Buffer Nesting Level

The Command Buffer Nesting Level of a secondary command buffer is equal to the maximum nesting level of all secondary command buffers executed by that command buffer plus one, where a secondary command buffer that executes no other secondary command buffers has a nesting level of zero.

Command Pool

An object that command buffer memory is allocated from, and that owns that memory. Command pools aid multithreaded performance by enabling different threads to use different allocators, without internal synchronization on each use. Represented by a VkCommandPool object.

Compatible Allocator

When allocators are compatible, allocations from each allocator **can** be freed by the other allocator.

Compatible Image Formats

When formats are compatible, images created with one of the formats **can** have image views

created from it using any of the compatible formats. Also see *Size-Compatible Image Formats*.

Compatible Queues

Queues within a queue family. Compatible queues have identical properties.

Complete Mipmap Chain

The entire set of mip levels that can be provided for an image, from the largest application-specified mip level size down to the *minimum mip level size*. See Image Mip Level Sizing.

Completed Operation

A deferred operation whose corresponding command has been executed to completion. See Deferred Host Operations

Component (Format)

A distinct part of a format. Color components are represented with `R`, `G`, `B`, and `A`. Depth and stencil components are represented with `D` and `S`. Formats **can** have multiple instances of the same component. Some formats have other notations such as `E` or `X` which are not considered a component of the format.

Compressed Texel Block

An element of an image having a block-compressed format, comprising a rectangular block of texel values that are encoded as a single value in memory. Compressed texel blocks of a particular block-compressed format have a corresponding width, height, and depth defining the dimensions of these elements in units of texels, and a size in bytes of the encoding in memory.

Constant Integral Expressions

A SPIR-V constant instruction whose type is `OpTypeInt`. See *Constant Instruction* in section 2.2.1 “Instructions” of the Khronos SPIR-V Specification.

Cooperative Matrix

A SPIR-V type where the storage for and computations performed on the matrix are spread across a set of invocations such as a subgroup.

Coverage Index

The index of a sample in the coverage mask.

Coverage Mask

A bitfield associated with a fragment representing the samples that were determined to be covered based on the result of rasterization, and then subsequently modified by fragment operations or the fragment shader.

Cull Distance

A built-in output from pre-rasterization shader stages defining a cull half-space where the primitive is rejected if all vertices have a negative value for the same cull distance.

Cull Volume

The intersection of the view volume with all cull half-spaces.

Decode Output Picture

A video picture resource used to store the result of a video decode operation.

Decoded Picture Buffer

An indexed set of reference pictures used by a video session. Abbreviated as DPB.

Decoded Picture Buffer Slot

An entry within a DPB that **can** be associated with a particular reference picture.

Decoded Picture Buffer Slot Index

The index of a DPB slot within its encompassing DPB.

Decoration (SPIR-V)

Auxiliary information such as built-in variables, stream numbers, invariance, interpolation type, relaxed precision, etc., added to variables or structure-type members through decorations.

Deferrable Command

A command which allows deferred execution of host-side work. See Deferred Host Operations.

Deferrable Operation

A single logical item of host-side work which can be deferred. Represented by the VkDeferredOperationKHR object. See Deferred Host Operations.

Deprecated (feature)

A feature is deprecated if it is no longer recommended as the correct or best way to achieve its intended purpose.

Depth/Stencil Attachment

A subpass attachment point, or image view, that is the target of depth and/or stencil test operations and writes.

Depth/Stencil Format

A VkFormat that includes depth and/or stencil components.

Depth/Stencil Image (or ImageView)

A VkImage (or VkImageView) with a depth/stencil format.

Depth/Stencil Resolve Attachment

A subpass attachment point, or image view, that is the target of a multisample resolve operation from the corresponding depth/stencil attachment at the end of the subpass.

Derivative Group

A set of compute, mesh, task, or fragment shader invocations that cooperate to compute derivatives, including implicit derivatives for sampled image operations.

Descriptor

Information about a resource or resource view written into a descriptor set that is used to access the resource or view from a shader.

Descriptor Binding

An entry in a descriptor set layout corresponding to zero or more descriptors of a single descriptor type in a set. Defined by a VkDescriptorSetLayoutBinding structure.

Descriptor Pool

An object that descriptor sets are allocated from, and that owns the storage of those descriptor sets. Descriptor pools aid multithreaded performance by enabling different threads to use different allocators, without internal synchronization on each use. Represented by a VkDescriptorPool object.

Descriptor Set

An object that resource descriptors are written into via the API, and that **can** be bound to a command buffer such that the descriptors contained within it **can** be accessed from shaders. Represented by a VkDescriptorSet object.

Descriptor Set Layout

An object defining the set of resources (types and counts) and their relative arrangement (in the binding namespace) within a descriptor set. Used when allocating descriptor sets and when creating pipeline layouts. Represented by a VkDescriptorSetLayout object.

Device

The processor(s) and execution environment that perform tasks requested by the application via the Vulkan API.

Device Group

A set of physical devices that support accessing each other's memory and recording a single command buffer that **can** be executed on all the physical devices.

Device Index

A zero-based integer that identifies one physical device from a logical device. A device index is valid if it is less than the number of physical devices in the logical device.

Device Mask

A bitmask where each bit represents one device index. A device mask value is valid if every bit that is set in the mask is at a bit position that is less than the number of physical devices in the logical device.

Device Memory

Memory accessible to the device. Represented by a VkDeviceMemory object.

Device-Level Command

Any command that is dispatched from a logical device, or from a child object of a logical device.

Device-Level Functionality

All device-level commands and objects, and their structures, enumerated types, and enumerants. Additionally, physical-device-level functionality defined by a device extension is also considered device-level functionality.

Device-Level Object

Logical device objects and their child objects. For example, VkDevice, VkQueue, and VkCommandBuffer objects are device-level objects.

Device-Local Memory

Memory that is connected to the device, and **may** be more performant for device access than host-local memory.

Direct Drawing Commands

Drawing commands that take all their parameters as direct arguments to the command (and not sourced via structures in buffer memory as the *indirect drawing commands*). Includes vkCmdDrawMultiIndexedEXT, vkCmdDrawMultiEXT, vkCmdDraw, and vkCmdDrawIndexed.

Disjoint

Disjoint planes are *image planes* to which memory is bound independently.
A *disjoint image* consists of multiple *disjoint planes*, and is created with the `VK_IMAGE_CREATE_DISJOINT_BIT` bit set.

Dispatchable Command

A non-global command. The first argument to each dispatchable command is a dispatchable handle type.

Dispatchable Handle

A handle of a pointer handle type which **may** be used by layers as part of intercepting API commands.

Dispatching Commands

Commands that provoke work using a compute pipeline, such as vkCmdDispatch. See Dispatching Commands.

Drawing Commands

Commands that provoke work using a graphics pipeline. Includes vkCmdDraw, vkCmdDrawIndexed, vkCmdDrawIndirectCountKHR, vkCmdDrawIndexedIndirectCountKHR, vkCmdDrawMultiIndexedEXT, vkCmdDrawMultiEXT, vkCmdDrawIndirect, and vkCmdDrawIndexedIndirect.

Duration (Command)

The *duration* of a Vulkan command refers to the interval between calling the command and its return to the caller.

Dynamic Storage Buffer

A storage buffer whose offset is specified each time the storage buffer is bound to a command buffer via a descriptor set.

Dynamic Uniform Buffer

A uniform buffer whose offset is specified each time the uniform buffer is bound to a command buffer via a descriptor set.

Dynamically Uniform

See *Dynamically Uniform* in section 2.2 “Terms” of the Khronos SPIR-V Specification.

Encode Input Picture

A video picture resource used as the input of a video encode operation.

Element

Arrays are composed of multiple elements, where each element exists at a unique index within that array. Used primarily to describe data passed to or returned from the Vulkan API.

Explicitly-Enabled Layer

A layer enabled by the application by adding it to the enabled layer list in vkCreateInstance or vkCreateDevice.

Event

A synchronization primitive that is signaled when execution of previous commands completes through a specified set of pipeline stages. Events can be waited on by the device and polled by the host. Represented by a VkEvent object.

Executable State (Command Buffer)

A command buffer that has ended recording commands and **can** be executed. See also Initial State and Recording State.

Execution Dependency

A dependency that guarantees that certain pipeline stages’ work for a first set of commands has completed execution before certain pipeline stages’ work for a second set of commands begins execution. This is accomplished via pipeline barriers, subpass dependencies, events, or implicit ordering operations.

Execution Dependency Chain

A sequence of execution dependencies that transitively act as a single execution dependency.

Explicit chroma reconstruction

An implementation of sampler $Y'C_BC_R$ conversion which reconstructs reduced-resolution chroma samples to luma resolution and then separately performs texture sample interpolation. This is distinct from an implicit implementation, which incorporates chroma sample reconstruction into texture sample interpolation.

Extension Scope

The set of objects and commands that **can** be affected by an extension. Extensions are either device scope or instance scope.

Extending Structure

A structure type which may appear in the `pNext` *chain* of another structure, extending the functionality of the other structure. Extending structures may be defined by either core API versions or extensions.

External Handle

A resource handle which has meaning outside of a specific Vulkan device or its parent instance. External handles **may** be used to share resources between multiple Vulkan devices in different instances, or between Vulkan and other APIs. Some external handle types correspond to platform-defined handles, in which case the resource **may** outlive any particular Vulkan device or instance and **may** be transferred between processes, or otherwise manipulated via functionality defined by the platform for that handle type.

External synchronization

A type of synchronization **required** of the application, where parameters defined to be externally synchronized **must** not be used simultaneously in multiple threads.

Facingness (Polygon)

A classification of a polygon as either front-facing or back-facing, depending on the orientation (winding order) of its vertices.

Facingness (Fragment)

A fragment is either front-facing or back-facing, depending on the primitive it was generated from. If the primitive was a polygon (regardless of polygon mode), the fragment inherits the facingness of the polygon. All other fragments are front-facing.

Fence

A synchronization primitive that is signaled when a set of batches or sparse binding operations complete execution on a queue. Fences **can** be waited on by the host. Represented by a VkFence object.

Field (Video)

Possibly discontinuous subregions of a frame. Frames **may** consist of two fields, a top field and a bottom field.

Flat Shading

A property of a vertex attribute that causes the value from a single vertex (the provoking vertex) to be used for all vertices in a primitive, and for interpolation of that attribute to return that single value unaltered.

Format Features

A set of features from VkFormatFeatureFlagBits that a VkFormat is capable of using for various commands. The list is determined by factors such as VkImageTiling.

Forward Prediction

Sample prediction performed during video decode and encode operations based on reference pictures that occur temporally (in display order) before the current picture.

Forward Reference

A reference picture that occurs temporally (in display order) before the current picture.

Fragment

A rectangular framebuffer region with associated data produced by rasterization and processed

by fragment operations including the fragment shader.

Fragment Area

The width and height, in pixels, of a fragment.

Fragment Density

The ratio of fragments per framebuffer area in the x and y direction.

Fragment Density Texel Size

The (w,h) framebuffer region in pixels that each texel in a fragment density map applies to.

Fragment Input Attachment Interface

Variables with `UniformConstant` storage class and a decoration of `InputAttachmentIndex` that are statically used by a fragment shader's entry point, which receive values from input attachments.

Fragment Output Interface

A fragment shader entry point's variables with `Output` storage class, which output to color and/or depth/stencil attachments.

Frame (Video)

A multi-dimensional array of luma samples and an optional multi-dimensional array of chroma samples.

Fragment Tile Image Interface

A fragment shader entry point's variables with `TileImageEXT` storage class and a decoration of `Location`, which are used to read values from color attachments.

Framebuffer

A collection of image views and a set of dimensions that, in conjunction with a render pass, define the inputs and outputs used by drawing commands. Represented by a VkFramebuffer object.

Framebuffer Attachment

One of the image views used in a framebuffer.

Framebuffer Coordinates

A coordinate system in which adjacent pixels' coordinates differ by 1 in x and/or y, with (0,0) in the upper left corner and pixel centers at half-integers.

Framebuffer-Space

Operating with respect to framebuffer coordinates.

Framebuffer-Local

A framebuffer-local dependency guarantees that only for a single framebuffer region, the first set of operations happens-before the second set of operations.

Framebuffer-Global

A framebuffer-global dependency guarantees that for all framebuffer regions, the first set of

operations happens-before the second set of operations.

Framebuffer Region

A framebuffer region is a set of sample (x, y, layer, sample) coordinates that is a subset of the entire framebuffer.

Front-Facing

See Facingness.

Full Compatibility

A given version of the API is fully compatible with another version if an application, relying only on valid behavior and functionality defined by either of those specifications, is able to correctly run against each version without any modification. This assumes no active attempt by that application to not run when it detects a different version.

Global Command

A Vulkan command for which the first argument is not a dispatchable handle type.

Global Workgroup

A collection of local workgroups dispatched by a single dispatching command.

Handle

An opaque integer or pointer value used to refer to a Vulkan object. Each object type has a unique handle type.

Happen-after, happens-after

A transitive, irreflexive and antisymmetric ordering relation between operations. An execution dependency with a source of **A** and a destination of **B** enforces that **B** happens-after **A**. The inverse relation of happens-before.

Happen-before, happens-before

A transitive, irreflexive and antisymmetric ordering relation between operations. An execution dependency with a source of **A** and a destination of **B** enforces that **A** happens-before **B**. The inverse relation of happens-after.

Helper Invocation

A fragment shader invocation that is created solely for the purposes of evaluating derivatives for use in non-helper fragment shader invocations, and which does not have side effects.

Host

The processor(s) and execution environment that the application runs on, and that the Vulkan API is exposed on.

Host Mapped Device Memory

Device memory that is mapped for host access using vkMapMemory.

Host Mapped Foreign Memory

Memory owned by a foreign device that is mapped for host access.

Host Memory

Memory not accessible to the device, used to store implementation data structures.

Host-Accessible Subresource

A buffer, or a linear image subresource in either the `VK_IMAGE_LAYOUT_PREINITIALIZED` or `VK_IMAGE_LAYOUT_GENERAL` layout. Host-accessible subresources have a well-defined addressing scheme which can be used by the host.

Host-Local Memory

Memory that is not local to the device, and **may** be less performant for device access than device-local memory.

Host-Visible Memory

Device memory that **can** be mapped on the host and **can** be read and written by the host.

ICD

Installable Client Driver. An ICD is represented as a VkPhysicalDevice.

Identically Defined Objects

Objects of the same type where all arguments to their creation or allocation functions, with the exception of `pAllocator`, are

1. Vulkan handles which refer to the same object or
2. identical scalar or enumeration values or
3. Host pointers which point to an array of values or structures which also satisfy these three constraints.

Image

A resource that represents a multi-dimensional formatted interpretation of device memory. Represented by a VkImage object.

Image Subresource

A specific mipmap level, layer, and set of aspects of an image.

Image Subresource Range

A set of image subresources that are contiguous mipmap levels and layers.

Image View

An object that represents an image subresource range of a specific image, and state controlling how the contents are interpreted. Represented by a VkImageView object.

Immutable Sampler

A sampler descriptor provided at descriptor set layout creation time for a specific binding. This sampler is then used for that binding in all descriptor sets allocated with the layout, and it **cannot** be changed.

Implicit chroma reconstruction

An implementation of sampler $Y'C_BC_R$ conversion which reconstructs the reduced-resolution

chroma samples directly at the sample point, as part of the normal texture sampling operation. This is distinct from an *explicit chroma reconstruction* implementation, which reconstructs the reduced-resolution chroma samples to the resolution of the luma samples, then filters the result as part of texture sample interpolation.

Implicitly-Enabled Layer

A layer enabled by a loader-defined mechanism outside the Vulkan API, rather than explicitly by the application during instance or device creation.

Inactive Object (Ray Tracing)

A primitive or instance in a ray tracing acceleration structure which has a corresponding ID, but which will never report an intersection with any ray.

Index Buffer

A buffer bound via vkCmdBindIndexBuffer which is the source of index values used to fetch vertex attributes for a vkCmdDrawIndexed or vkCmdDrawIndexedIndirect command.

Indexed Drawing Commands

Drawing commands which use an *index buffer* as the source of index values used to fetch vertex attributes for a drawing command. Includes vkCmdDrawIndexed, vkCmdDrawIndexedIndirectCountKHR, vkCmdDrawMultiIndexedEXT, and vkCmdDrawIndexedIndirect.

Indirect Commands

Drawing or dispatching commands that source some of their parameters from structures in buffer memory. Includes vkCmdDrawIndirect, vkCmdDrawIndexedIndirect, vkCmdDrawIndirectCountKHR, vkCmdDrawIndexedIndirectCountKHR, and vkCmdDispatchIndirect.

Indirect Drawing Commands

Drawing commands that source some of their parameters from structures in buffer memory. Includes vkCmdDrawIndirect, vkCmdDrawIndirectCountKHR, vkCmdDrawIndexedIndirectCountKHR, and vkCmdDrawIndexedIndirect.

Initial State (Command Buffer)

A command buffer that has not begun recording commands. See also Recording State and Executable State.

Input Attachment

A descriptor type that represents an image view, and supports unfiltered read-only access in a shader, only at the fragment's location in the view.

Instance

The top-level Vulkan object, which represents the application's connection to the implementation. Represented by a VkInstance object.

Instance-Level Command

Any command that is dispatched from an instance, or from a child object of an instance, except

for physical devices and their children.

Instance-Level Functionality

All instance-level commands and objects, and their structures, enumerated types, and enumerants.

Instance-Level Object

High-level Vulkan objects, which are not physical devices, nor children of physical devices. For example, VkInstance is an instance-level object.

Instance (Memory)

In a logical device representing more than one physical device, some device memory allocations have the requested amount of memory allocated multiple times, once for each physical device in a device mask. Each such replicated allocation is an instance of the device memory.

Instance (Resource)

In a logical device representing more than one physical device, buffer and image resources exist on all physical devices but **can** be bound to memory differently on each. Each such replicated resource is an instance of the resource.

Inter Block (Video)

A video coding block that uses inter prediction.

Inter Prediction (Video)

Sample prediction derived from reference picture samples or metadata.

Internal Synchronization

A type of synchronization **required** of the implementation, where parameters not defined to be externally synchronized **may** require internal mutexing to avoid multithreaded race conditions.

Intra Block (Video)

A video coding block that only uses intra prediction.

Intra Prediction (Video)

Sample prediction that is not derived from reference picture samples or metadata.

Invocation (Shader)

A single execution of an entry point in a SPIR-V module. For example, a single vertex's execution of a vertex shader or a single fragment's execution of a fragment shader.

Invocation Group

A set of shader invocations that are executed in parallel and that **must** execute the same control flow path in order for control flow to be considered dynamically uniform.

Invocation Repack Instruction

A ray tracing instruction where the implementation **may** change the set of invocations that are executing.

Join (Deferred Host Operations)

The act of instructing a thread to participate in the execution of a deferred operation. See Deferred Host Operations.

Linear Resource

A resource is *linear* if it is one of the following:

- a VkBuffer
- a VkImage created with `VK_IMAGE_TILING_LINEAR`
- a VkImage created with `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT` and whose Linux DRM format modifier is `DRM_FORMAT_MOD_LINEAR`

Because a VkAccelerationStructureKHR resource does not have memory bound to it directly, it is considered neither linear nor non-linear. However, the VkBuffer on which a VkAccelerationStructureKHR resource is placed is a linear resource.

A resource is *non-linear* if it is one of the following:

- a VkImage created with `VK_IMAGE_TILING_OPTIMAL`
- a VkImage created with `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT` and whose Linux DRM format modifier is not `DRM_FORMAT_MOD_LINEAR`

Linux DRM Format Modifier

A 64-bit, vendor-prefixed, semi-opaque unsigned integer describing vendor-specific details of an image's memory layout. In Linux graphics APIs, *modifiers* are commonly used to specify the memory layout of externally shared images. An image has a *modifier* if and only if it is created with `tiling` equal to `VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT`. For more details, refer to the appendix for extension `VK_EXT_image_drm_format_modifier`.

Local Workgroup

A collection of compute shader invocations invoked by a single dispatching command, which share data via `WorkgroupLocal` variables and can synchronize with each other.

Logical Device

An object that represents the application's interface to the physical device. The logical device is the parent of most Vulkan objects. Represented by a VkDevice object.

Logical Operation

Bitwise operations between a fragment color value and a value in a color attachment, that produce a final color value to be written to the attachment.

Lost Device

A state that a logical device **may** be in as a result of unrecoverable implementation errors, or other exceptional conditions.

Mappable

See Host-Visible Memory.

Memory Dependency

A memory dependency is an execution dependency which includes availability and visibility operations such that:

- The first set of operations happens-before the availability operation
- The availability operation happens-before the visibility operation
- The visibility operation happens-before the second set of operations

Memory Domain

A memory domain is an abstract place to which memory writes are made available by availability operations and memory domain operations. The memory domains correspond to the set of agents that the write **can** then be made visible to. The memory domains are *host*, *device*, *shader*, *workgroup instance* (for workgroup instance there is a unique domain for each compute workgroup) and *subgroup instance* (for subgroup instance there is a unique domain for each subgroup).

Memory Domain Operation

An operation that makes the writes that are available to one memory domain available to another memory domain.

Memory Heap

A region of memory from which device memory allocations **can** be made.

Memory Type

An index used to select a set of memory properties (e.g. mappable, cached) for a device memory allocation.

Minimum Mip Level Size

The smallest size that is permitted for a mip level. For conventional images this is 1x1x1. See Image Mip Level Sizing.

Mip Tail Region

The set of mipmap levels of a sparse residency texture that are too small to fill a sparse block, and that **must** all be bound to memory collectively and opaquely.

Multi-planar

A *multi-planar format* (or “planar format”) is an image format consisting of more than one *plane*, identifiable with a `_2PLANE` or `_3PLANE` component to the format name and listed in Formats Requiring Sampler $Y'C_BC_R$ Conversion for `VK_IMAGE_ASPECT_COLOR_BIT` Image Views. A *multi-planar image* (or “planar image”) is an image of a multi-planar format.

Nested Command Buffers

A nested command buffer is a secondary command buffer that is executed by another secondary command buffer, which may itself execute other secondary command buffers.

Non-Dispatchable Handle

A handle of an integer handle type. Handle values **may** not be unique, even for two objects of the same type.

Non-Indexed Drawing Commands

Drawing commands for which the vertex attributes are sourced in linear order from the vertex input attributes for a drawing command (i.e. they do not use an *index buffer*). Includes vkCmdDraw, vkCmdDrawIndirectCountKHR, vkCmdDrawMultiEXT, and vkCmdDrawIndirect.

Normalized

A value that is interpreted as being in the range [0,1] as a result of being implicitly divided by some other value.

Normalized Device Coordinates

A coordinate space after perspective division is applied to clip coordinates, and before the viewport transformation converts them to framebuffer coordinates.

Obsoleted (feature)

A feature is obsolete if it can no longer be used.

Opaque Capture Address

A 64-bit value representing the device address of a buffer or memory object that is expected to be used by trace capture/replay tools in combination with the `bufferDeviceAddress` feature.

Overlapped Range (Aliased Range)

The aliased range of a device memory allocation that intersects a given image subresource of an image or range of a buffer.

Ownership (Resource)

If an entity (e.g. a queue family) has ownership of a resource, access to that resource is well-defined for access by that entity.

Packed Format

A format whose components are stored as a single texel block in memory, with their relative locations defined within that element.

Payload

Importable or exportable reference to the internal data of an object in Vulkan.

Peer Memory

An instance of memory corresponding to a different physical device than the physical device performing the memory access, in a logical device that represents multiple physical devices.

Physical Device

An object that represents a single device in the system. Represented by a VkPhysicalDevice object.

Physical-Device-Level Command

Any command that is dispatched from a physical device.

Physical-Device-Level Functionality

All physical-device-level commands and objects, and their structures, enumerated types, and

enumerants.

Physical-Device-Level Object

Physical device objects. For example, VkPhysicalDevice is a physical-device-level object.

Picture Partition (Video)

Collective term that refers to a subregion of a video picture resource that **may** be decoded or encoded independently according to the partitioning scheme defined in the corresponding video compression standard. Picture partitions **may** be non-rectangular.

Pipeline

An object controlling how graphics or compute work is executed on the device. A pipeline includes one or more shaders, as well as state controlling any non-programmable stages of the pipeline. Represented by a VkPipeline object.

Pipeline Barrier

An execution and/or memory dependency recorded as an explicit command in a command buffer, that forms a dependency between the previous and subsequent commands.

Pipeline Cache

An object that **can** be used to collect and retrieve information from pipelines as they are created, and **can** be populated with previously retrieved information in order to accelerate pipeline creation. Represented by a VkPipelineCache object.

Pipeline Layout

An object defining the set of resources (via a collection of descriptor set layouts) and push constants used by pipelines that are created using the layout. Used when creating a pipeline and when binding descriptor sets and setting push constant values. Represented by a VkPipelineLayout object.

Pipeline Library

A pipeline that cannot be directly used, instead defining a set of shaders and shader groups which will be linked into other pipelines.

Pipeline Stage

A logically independent execution unit that performs some of the operations defined by an action command.

Pipeline Trace Ray Instruction

A ray tracing instruction which traces a ray into an acceleration structure when using ray tracing pipelines. One of:

- `OpTraceRayKHR`

`pNext` Chain

A set of structures chained together through their `pNext` members.

Planar

See *multi-planar*.

Plane

An *image plane* is part of the representation of an image, containing a subset of the color components necessary to represent the texels in the image and with a contiguous mapping of coordinates to bound memory. Most images consist only of a single plane, but some formats spread the components across multiple image planes. The host-accessible properties of each image plane are accessible for a linear layout using vkGetImageSubresourceLayout. If a multi-planar image is created with the `VK_IMAGE_CREATE_DISJOINT_BIT` bit set, the image is described as *disjoint*, and its planes are therefore bound to memory independently.

Point Sampling (Rasterization)

A rule that determines whether a fragment sample location is covered by a polygon primitive by testing whether the sample location is in the interior of the polygon in framebuffer-space, or on the boundary of the polygon according to the tie-breaking rules.

Potential Format Features

The union of all VkFormatFeatureFlagBits that the implementation supports for a specified VkFormat, over all supported image tilings.

Pre-rasterization

Operations that execute before rasterization, and any state associated with those operations.

Presentable image

A `VkImage` object obtained from a `VkSwapchainKHR` used to present to a `VkSurfaceKHR` object.

Preserve Attachment

One of a list of attachments in a subpass description that is not read or written by the subpass, but that is read or written on earlier and later subpasses and whose contents **must** be preserved through this subpass.

Primary Command Buffer

A command buffer that **can** execute secondary command buffers, and **can** be submitted directly to a queue.

Primitive Topology

State controlling how vertices are assembled into primitives, e.g. as lists of triangles, strips of lines, etc.

Promoted (feature)

A feature from an older extension is considered promoted if it is made available as part of a new core version or newer extension with wider support.

Provisional

A feature is released provisionally in order to get wider feedback on the functionality before it is finalized. Provisional features may change in ways that break backwards compatibility, and thus are not recommended for use in production applications.

Provoking Vertex

The vertex in a primitive from which flat shaded attribute values are taken. This is generally the

"first" vertex in the primitive, and depends on the primitive topology.

Push Constants

A small bank of values writable via the API and accessible in shaders. Push constants allow the application to set values used in shaders without creating buffers or modifying and binding descriptor sets for each update.

Push Constant Interface

The set of variables with `PushConstant` storage class that are statically used by a shader entry point, and which receive values from push constant commands.

Push Descriptors

Descriptors that are written directly into a command buffer rather than into a descriptor set. Push descriptors allow the application to set descriptors used in shaders without allocating or modifying descriptor sets for each update.

Descriptor Update Template

An object specifying a mapping from descriptor update information in host memory to elements in a descriptor set, which helps enable more efficient descriptor set updates.

Query Pool

An object containing a number of query entries and their associated state and results. Represented by a VkQueryPool object.

Queue

An object that executes command buffers and sparse binding operations on a device. Represented by a VkQueue object.

Queue Family

A set of queues that have common properties and support the same functionality, as advertised in VkQueueFamilyProperties.

Queue Operation

A unit of work to be executed by a specific queue on a device, submitted via a queue submission command. Each queue submission command details the specific queue operations that occur as a result of calling that command. Queue operations typically include work that is specific to each command, and synchronization tasks.

Queue Submission

Zero or more batches and an optional fence to be signaled, passed to a command for execution on a queue. See the Devices and Queues chapter for more information.

Ray Tracing Command

Commands that provoke work using a ray tracing pipeline. Includes vkCmdTraceRaysKHR, and vkCmdTraceRaysIndirectKHR .

Reconstructed Picture

A video picture resource reconstructed from a compressed bitstream using video decode or

encode operations that **can** be used as a reference picture by future video decode or encode operations with the same video session.

Recording State (Command Buffer)

A command buffer that is ready to record commands. See also Initial State and Executable State.

Reference Picture

A video picture resource used by video decode and encode operations to provide predictions of the values of samples in the subsequently decoded or encoded pictures.

Reference Picture Metadata

Opaque state associated with a DPB slot, maintained by a video session.

Release Operation (Resource)

An operation that releases ownership of an image subresource or buffer range.

Render Pass

An object that represents a set of framebuffer attachments and phases of rendering using those attachments. Represented by a VkRenderPass object.

Render Pass Instance

A use of a render pass in a command buffer.

Required Extensions

Extensions that **must** be enabled alongside extensions dependent on them (see Extension Dependencies).

Reset (Command Buffer)

Resetting a command buffer discards any previously recorded commands and puts a command buffer in the initial state.

Residency Code

An integer value returned by sparse image instructions, indicating whether any sparse unbound texels were accessed.

Resolve Attachment

A subpass attachment point, or image view, that is the target of a multisample resolve operation from the corresponding color attachment at the end of the subpass.

Retired Swapchain

A swapchain that has been used as the `oldSwapchain` parameter to vkCreateSwapchainKHR. Images cannot be acquired from a retired swapchain, however images that were acquired (but not presented) before the swapchain was retired **can** be presented.

Sample Index

The index of a sample within a single set of samples.

Sample Shading

Invoking the fragment shader multiple times per fragment, with the covered samples partitioned among the invocations.

Sampled Image

A descriptor type that represents an image view, and supports filtered (sampled) and unfiltered read-only access in a shader.

Sampler

An object containing state controlling how sampled image data is sampled (or filtered) when accessed in a shader. Also a descriptor type describing the object. Represented by a VkSampler object.

Secondary Command Buffer

A command buffer that **can** be executed by a primary command buffer, and **must** not be submitted directly to a queue.

Self-Dependency

A subpass dependency from a subpass to itself, i.e. with `srcSubpass` equal to `dstSubpass`. A self-dependency is not automatically performed during a render pass instance, rather a subset of it **can** be performed via vkCmdPipelineBarrier during the subpass.

Semaphore

A synchronization primitive that supports signal and wait operations, and **can** be used to synchronize operations within a queue or across queues. Represented by a VkSemaphore object.

Shader

Instructions selected (via an entry point) from a shader module, which are executed in a shader stage.

Shader Call

An instruction which **may** cause execution to continue in a different shader stage.

Shader Code

A stream of instructions used to describe the operation of a shader.

Shader Group

A set of Shader Stages that are part of a VkPipeline containing multiple of such sets. This allows the device to make use of all the shader groups from the bound pipeline independently.

Shader Module

A collection of shader code, potentially including several functions and entry points, that is used to create shaders in pipelines. Represented by a VkShaderModule object.

Shader Stage

A stage of the graphics or compute pipeline that executes shader code.

Shading Rate

The ratio of the number of fragment shader invocations generated in a fully covered framebuffer region to the size (in pixels) of that region.

Shared presentable image

A presentable image created from a swapchain with VkPresentModeKHR set to either VK_PRESENT_MODE_SHARED_DEMAND_REFRESH_KHR or VK_PRESENT_MODE_SHARED_CONTINUOUS_REFRESH_KHR.

Side Effect

A store to memory or atomic operation on memory from a shader invocation.

Single-plane format

A format that is not *multi-planar*.

Size-Compatible Image Formats

When a compressed image format and an uncompressed image format are size-compatible, it means that the texel block size of the uncompressed format **must** equal the texel block size of the compressed format.

Sparse Block

An element of a sparse resource that can be independently bound to memory. Sparse blocks of a particular sparse resource have a corresponding size in bytes that they use in the bound memory.

Sparse Image Block

A sparse block in a sparse partially-resident image. In addition to the sparse block size in bytes, sparse image blocks have a corresponding width, height, and depth defining the dimensions of these elements in units of texels or compressed texel blocks, the latter being used in case of sparse images having a block-compressed format.

Sparse Unbound Texel

A texel read from a region of a sparse texture that does not have memory bound to it.

Static Use

An object in a shader is statically used by a shader entry point if any function in the entry point's call tree contains an instruction using the object. A reference in the entry point's interface list does not constitute a static use. Static use is used to constrain the set of descriptors used by a shader entry point.

Storage Buffer

A descriptor type that represents a buffer, and supports reads, writes, and atomics in a shader.

Storage Image

A descriptor type that represents an image view, and supports unfiltered loads, stores, and atomics in a shader.

Storage Texel Buffer

A descriptor type that represents a buffer view, and supports unfiltered, formatted reads, writes,

and atomics in a shader.

Subpass

A phase of rendering within a render pass, that reads and writes a subset of the attachments.

Subpass Dependency

An execution and/or memory dependency between two subpasses described as part of render pass creation, and automatically performed between subpasses in a render pass instance. A subpass dependency limits the overlap of execution of the pair of subpasses, and **can** provide guarantees of memory coherence between accesses in the subpasses.

Subpass Description

Lists of attachment indices for input attachments, color attachments, depth/stencil attachment, resolve attachments, depth/stencil resolve, and preserve attachments used by the subpass in a render pass.

Subset (Self-Dependency)

A subset of a self-dependency is a pipeline barrier performed during the subpass of the self-dependency, and whose stage masks and access masks each contain a subset of the bits set in the identically named mask in the self-dependency.

Texel Block

A single addressable element of an image with an uncompressed VkFormat, or a single compressed block of an image with a compressed VkFormat.

Texel Block Size

The size (in bytes) used to store a texel block of a compressed or uncompressed image.

Texel Coordinate System

One of three coordinate systems (normalized, unnormalized, integer) defining how texel coordinates are interpreted in an image or a specific mipmap level of an image.

Tile Image

A per-tile view of a framebuffer attachment. If the `VK_EXT_shader_tile_image` extension is enabled, the framebuffer is considered to be divided into tiles.

Timeline Semaphore

A semaphore with a strictly increasing 64-bit unsigned integer payload indicating whether the semaphore is signaled with respect to a particular reference value. Represented by a VkSemaphore object created with a semaphore type of `VK_SEMAPHORE_TYPE_TIMELINE`.

Uniform Texel Buffer

A descriptor type that represents a buffer view, and supports unfiltered, formatted, read-only access in a shader.

Uniform Buffer

A descriptor type that represents a buffer, and supports read-only access in a shader.

Units in the Last Place (ULP)

A measure of floating-point error loosely defined as the smallest representable step in a floating-point format near a given value. For the precise definition see Precision and Operation of SPIR-V instructions or Jean-Michel Muller, “On the definition of ulp(x)”, RR-5504, INRIA. Other sources may also use the term “unit of least precision”.

Unnormalized

A value that is interpreted according to its conventional interpretation, and is not normalized.

User-Defined Variable Interface

A shader entry point’s variables with `Input` or `Output` storage class that are not built-in variables.

Vertex Input Attribute

A graphics pipeline resource that produces input values for the vertex shader by reading data from a vertex input binding and converting it to the attribute’s format.

Vertex Stream

A vertex stream is where the last pre-rasterization shader stages outputs vertex data, which then goes to the rasterizer, is captured to a transform feedback buffer, or both. Geometry shaders **can** emit primitives to multiple independent vertex streams. Each vertex emitted by the geometry shader is directed at one of the vertex streams.

Validation Cache

An object that **can** be used to collect and retrieve validation results from the validation layers, and **can** be populated with previously retrieved results in order to accelerate the validation process. Represented by a VkValidationCacheEXT object.

Variable-Sized Descriptor Binding

A descriptor binding whose size will be specified when a descriptor set is allocated using this layout.

Vertex Input Binding

A graphics pipeline resource that is bound to a buffer and includes state that affects addressing calculations within that buffer.

Vertex Input Interface

A vertex shader entry point’s variables with `Input` storage class, which receive values from vertex input attributes.

Video Bitstream Buffer

A resource that represents a linear array of data in device memory storing encoded video data. Represented by a VkBuffer object.

Video Coding Scope

A series of subsequent commands recorded into a command buffer starting with a vkCmdBeginVideoCodingKHR command and ending with a vkCmdEndVideoCodingKHR command that encompasses a set of video decode or encode operations.

Video Coding Operations

Any operations recorded into a command buffer within a video coding scope, including video decode and encode operations.

Video Decode Operation

An operation consuming data from a video bitstream buffer and zero or more reference pictures, and producing data to a decode output picture and an optional reconstructed picture.

Video Encode Operation

An operation consuming data from an encode input picture and zero or more reference pictures, and producing data to a video bitstream buffer and an optional reconstructed picture.

Video Picture Resource

A resource that represents a multi-dimensional formatted interpretation of device memory to be used with a video session as a decode output picture, encode input picture, reconstructed picture, and/or reference picture. It **may** contain metadata associated with a particular video session it is used with. Represented by a VkImage object and referred to using VkImageView objects created from it.

Video Session

A resource that represents and maintains the state needed to perform video decode or encode operations. Represented by a VkVideoSessionKHR object.

Video Session Parameters

A resource that stores preprocessed codec-specific parameters used with a compatible video session in video codec operations. Represented by a VkVideoSessionParametersKHR object.

Video Transcoding

The process of using the outputs of video decoding operations as inputs in video encoding operations.

View Mask

When multiview is enabled, a view mask is a property of a subpass controlling which views the rendering commands are broadcast to.

View Volume

A subspace in homogeneous coordinates, corresponding to post-projection x and y values between -1 and +1, and z values between 0 and +1.

Viewport Transformation

A transformation from normalized device coordinates to framebuffer coordinates, based on a viewport rectangle and depth range.

Visibility Operation

An operation that causes available values to become visible to specified memory accesses.

Visible

A state of values written to memory that allows them to be accessed by a set of operations.

Common Abbreviations

The abbreviations and acronyms defined in this section are sometimes used in the Specification and the API where they are considered clear and commonplace.

Src
: Source

Dst
: Destination

Min
: Minimum

Max
: Maximum

Rect
: Rectangle

Info
: Information

LOD
: Level of Detail

Log
: Logarithm

ID
: Identifier

UUID
: Universally Unique Identifier

Op
: Operation

R
: Red color component

G
: Green color component

B
: Blue color component

A

Alpha color component

RTZ

Round towards zero

RTE

Round to nearest even

Video-Specific Abbreviations

The following abbreviations and acronyms are used in the context of video decode and encode operations to refer to commonly used video compression terms in their usual abbreviated form:

AVC

Advanced Video Coding

Bipred

Bidirectional Prediction

CABAC

Context-Adaptive Binary Arithmetic Coding

CAVLC

Context-Adaptive Variable-Length Coding

CBR

Constant Bit Rate

CTB

Coding Tree Block

Diff

Difference

DPB

Decoded Picture Buffer

GOP

Group Of Pictures

HDR

High Dynamic Range

HEVC

High Efficiency Video Coding

HRD
: Hypothetical Reference Decoder

IDC
: Indicator

IDR
: Instantaneous Decoder Refresh

MB
: Macroblock

MV
: Motion Vector

NALU
: Network Abstraction Layer Unit

OBU
: Open Bitstream Unit

PCM
: Pulse-Code Modulation

Pic
: Picture

Pred
: Prediction

PPS
: Picture Parameter Set

QP
: Quantization Parameter

RC
: Rate Control

SPS
: Sequence Parameter Set

Std
: Standard

VBR
: Variable Bit Rate

VCL

Video Coding Layer

VPS

Video Parameter Set

Prefixes

Prefixes are used in the API to denote specific semantic meaning of Vulkan names, or as a label to avoid name clashes, and are explained here:

VK/Vk/vk

Vulkan namespace
All types, commands, enumerants and defines in this specification are prefixed with these two characters.

PFN/pfn

Function Pointer
Denotes that a type is a function pointer, or that a variable is of a pointer type.

p

Pointer
Variable is a pointer.

vkCmd

Commands that record commands in command buffers
These API commands do not result in immediate processing on the device. Instead, they record the requested action in a command buffer for execution when the command buffer is submitted to a queue.

s

Structure
Used to denote the `VK_STRUCTURE_TYPE*` member of each structure in `sType`

Appendix I: Credits (Informative)

Vulkan 1.3 is the result of contributions from many people and companies participating in the Khronos Vulkan Working Group, as well as input from the Vulkan Advisory Panel.

Members of the Working Group, including the company that they represented at the time of their most recent contribution, are listed in the following section. Some specific contributions made by individuals are listed together with their name.

Working Group Contributors to Vulkan

- Aaron Greig, Codeplay Software Ltd. (version 1.1)
- Aaron Hagan, AMD (version 1.1)
- Adam Jackson, Red Hat (versions 1.0, 1.1)
- Adam Śmigielski, Mobica (version 1.0)
- Aditi Verma, Qualcomm (version 1.3)
- Ahmed Abdelkhalek, AMD (version 1.3)
- Aidan Fabius, Core Avionics & Industrial Inc. (version 1.2)
- Alan Baker, Google (versions 1.1, 1.2, 1.3)
- Alan Ward, Google (versions 1.1, 1.2)
- Alejandro Piñeiro, Igalia (version 1.1)
- Alex Bourd, Qualcomm Technologies, Inc. (versions 1.0, 1.1)
- Alex Crabb, Caster Communications (versions 1.2, 1.3)
- Alex Walters, Imagination Technologies (versions 1.2, 1.3)
- Alexander Galazin, Arm (versions 1.0, 1.1, 1.2, 1.3)
- Alexey Sachkov, Intel (version 1.3)
- Allan MacKinnon, Google (version 1.3)
- Allen Hux, Intel (version 1.0)
- Alon Or-bach, Google (versions 1.0, 1.1, 1.2, 1.3) (WSI technical sub-group chair)
- Anastasia Stulova, Arm (versions 1.2, 1.3)
- Andreas Vasilakis, Think Silicon (version 1.2)
- Andres Gomez, Igalia (version 1.1)
- Andrew Cox, Samsung Electronics (version 1.0)
- Andrew Ellem, Google (version 1.3)
- Andrew Garrard, Imagination Technologies (versions 1.0, 1.1, 1.2, 1.3) (format wrangler)
- Andrew Poole, Samsung Electronics (version 1.0)
- Andrew Rafter, Samsung Electronics (version 1.0)

- Andrew Richards, Codeplay Software Ltd. (version 1.0)
- Andrew Woloszyn, Google (versions 1.0, 1.1)
- Ann Thorsnes, Khronos (versions 1.2, 1.3)
- Antoine Labour, Google (versions 1.0, 1.1)
- Aras Pranckevičius, Unity Technologies (version 1.0)
- Arseny Kapoulkine, Roblox (version 1.3)
- Ashwin Kolhe, NVIDIA (version 1.0)
- Baldur Karlsson, Valve Software (versions 1.1, 1.2, 1.3)
- Barthold Lichtenbelt, NVIDIA (version 1.1)
- Bas Nieuwenhuizen, Google (versions 1.1, 1.2)
- Ben Bowman, Imagination Technologies (version 1.0)
- Benj Lipchak, Unknown (version 1.0)
- Bill Hollings, Brenwill (versions 1.0, 1.1, 1.2, 1.3)
- Bill Licea-Kane, Qualcomm Technologies, Inc. (versions 1.0, 1.1)
- Blaine Kohl, Khronos (versions 1.2, 1.3)
- Bob Fraser, Google (version 1.3)
- Boris Zanin, Mobica (versions 1.2, 1.3)
- Brent E. Insko, Intel (version 1.0)
- Brian Ellis, Qualcomm Technologies, Inc. (version 1.0)
- Brian Paul, VMware (versions 1.2, 1.3)
- Caio Marcelo de Oliveira Filho, Intel (versions 1.2, 1.3)
- Cass Everitt, Oculus VR (versions 1.0, 1.1)
- Cemil Azizoglu, Canonical (version 1.0)
- Lina Versace, Google (versions 1.0, 1.1, 1.2)
- Chang-Hyo Yu, Samsung Electronics (version 1.0)
- Charles Giessen, LunarG (version 1.3)
- Chia-I Wu, LunarG (version 1.0)
- Chris Frascati, Qualcomm Technologies, Inc. (version 1.0)
- Chris Glover, Google (version 1.3)
- Christian Forfang, Arm (version 1.3)
- Christoph Kubisch, NVIDIA (version 1.3)
- Christophe Riccio, Unity Technologies (versions 1.0, 1.1)
- Cody Northrop, LunarG (version 1.0)
- Colin Riley, AMD (version 1.1)
- Cort Stratton, Google (versions 1.1, 1.2)

- Courtney Goeltzenleuchter, Google (versions 1.0, 1.1, 1.3)
- Craig Davies, Huawei (version 1.2)
- Dae Kim, Imagination Technologies (version 1.1)
- Damien Leone, NVIDIA (version 1.0)
- Dan Baker, Oxide Games (versions 1.0, 1.1)
- Dan Ginsburg, Valve Software (versions 1.0, 1.1, 1.2, 1.3)
- Daniel Johnston, Intel (versions 1.0, 1.1)
- Daniel Koch, NVIDIA (versions 1.0, 1.1, 1.2, 1.3)
- Daniel Rakos, AMD (versions 1.0, 1.1, 1.2, 1.3)
- Daniel Stone, Collabora (versions 1.1, 1.2)
- Daniel Vetter, Intel (version 1.2)
- David Airlie, Red Hat (versions 1.0, 1.1, 1.2, 1.3)
- David Mao, AMD (versions 1.0, 1.2)
- David Miller, Miller & Mattson (versions 1.0, 1.1) (Vulkan reference card)
- David Neto, Google (versions 1.0, 1.1, 1.2, 1.3)
- David Pankratz, Huawei (version 1.3)
- David Wilkinson, AMD (version 1.2)
- David Yu, Pixar (version 1.0)
- Dejan Mircevski, Google (version 1.1)
- Diego Novillo, Google (version 1.3)
- Dimitris Georgakakis, Think Silicon (version 1.3)
- Dominik Witczak, AMD (versions 1.0, 1.1, 1.3)
- Donald Scorgie, Imagination Technologies (version 1.2)
- Dzmitry Malyshau, Mozilla (versions 1.1, 1.2, 1.3)
- Ed Hutchins, Oculus (version 1.2)
- Emily Stearns, Khronos (versions 1.2, 1.3)
- François Duranleau, Gameloft (version 1.3)
- Frank (LingJun) Chen, Qualcomm Technologies, Inc. (version 1.0)
- Fred Liao, Mediatek (version 1.0)
- Gabe Dagani, Freescale (version 1.0)
- Gabor Sines, AMD (version 1.2)
- Graeme Leese, Broadcom (versions 1.0, 1.1, 1.2, 1.3)
- Graham Connor, Imagination Technologies (version 1.0)
- Graham Sellers, AMD (versions 1.0, 1.1)
- Graham Wihlidal, Electronic Arts (version 1.3)

- Greg Fischer, LunarG (version 1.1)
- Gregory Grebe, AMD (version 1.3)
- Hai Nguyen, Google (versions 1.2, 1.3)
- Hans-Kristian Arntzen, Valve Software (versions 1.1, 1.2, 1.3)
- Henri Verbeet, Codeweavers (version 1.2)
- Wyvern Wang, Huawei (version 1.3)
- Hwanyong Lee, Kyungpook National University (version 1.0)
- Iago Toral, Igalia (versions 1.1, 1.2)
- Ian Elliott, Google (versions 1.0, 1.1, 1.2)
- Ian Romanick, Intel (versions 1.0, 1.1, 1.3)
- Ivan Briano, Intel (version 1.3)
- James Fitzpatrick, Imagination (version 1.3)
- James Hughes, Oculus VR (version 1.0)
- James Jones, NVIDIA (versions 1.0, 1.1, 1.2, 1.3)
- James Riordon, Khronos (versions 1.2, 1.3)
- Jamie Madill, Google (version 1.3)
- Jan Hermes, Continental Corporation (versions 1.0, 1.1)
- Jan-Harald Fredriksen, Arm (versions 1.0, 1.1, 1.2, 1.3)
- Faith Ekstrand, Intel (versions 1.0, 1.1, 1.2, 1.3)
- Jean-François Roy, Google (versions 1.1, 1.2, 1.3)
- Jeff Bolz, NVIDIA (versions 1.0, 1.1, 1.2, 1.3)
- Jeff Juliano, NVIDIA (versions 1.0, 1.1, 1.2)
- Jeff Leger, Qualcomm Technologies, Inc. (versions 1.1, 1.3)
- Jeff Phillips, Khronos (version 1.3)
- Jeff Vigil, Samsung Electronics (versions 1.0, 1.1, 1.2, 1.3)
- Jens Owen, Google (versions 1.0, 1.1)
- Jeremy Hayes, LunarG (version 1.0)
- Jesse Barker, Unity Technologies (versions 1.0, 1.1, 1.2, 1.3)
- Jesse Hall, Google (versions 1.0, 1.1, 1.2, 1.3)
- Joe Davis, Samsung Electronics (version 1.1)
- Johannes van Waveren, Oculus VR (versions 1.0, 1.1)
- John Anthony, Arm (version 1.2, 1.3)
- John Kessenich, Google (versions 1.0, 1.1, 1.2, 1.3) (SPIR-V and GLSL for Vulkan spec author)
- John McDonald, Valve Software (versions 1.0, 1.1, 1.2, 1.3)
- John Zulauf, LunarG (versions 1.1, 1.2, 1.3)

- Jon Ashburn, LunarG (version 1.0)
- Jon Leech, Independent (versions 1.0, 1.1, 1.2, 1.3) (XML toolchain, normative language, release wrangler)
- Jonas Gustavsson, Samsung Electronics (versions 1.0, 1.1)
- Jonas Meyer, Epic Games (versions 1.2, 1.3)
- Jonathan Hamilton, Imagination Technologies (version 1.0)
- Jordan Justen, Intel (version 1.1)
- Joshua Ashton, Valve Software (version 1.3)
- Jungwoo Kim, Samsung Electronics (versions 1.0, 1.1)
- Jörg Wagner, Arm (version 1.1)
- Kalle Raita, Google (version 1.1)
- Karen Ghavam, LunarG (versions 1.1, 1.2, 1.3)
- Karl Schultz, LunarG (versions 1.1, 1.2)
- Kathleen Mattson, Khronos (versions 1.0, 1.1, 1.2)
- Kaye Mason, Google (version 1.2)
- Keith Packard, Valve (version 1.2)
- Kenneth Benzie, Codeplay Software Ltd. (versions 1.0, 1.1)
- Kenneth Russell, Google (version 1.1)
- Kerch Holt, NVIDIA (versions 1.0, 1.1)
- Kevin O'Neil, AMD (version 1.1)
- Kevin Petit, Arm (version 1.3)
- Kris Rose, Khronos (versions 1.2, 1.3)
- Kristian Kristensen, Intel (versions 1.0, 1.1)
- Krzysztof Iwanicki, Samsung Electronics (version 1.0)
- Larry Seiler, Intel (version 1.0)
- Laura Shubel, Caster Communications (version 1.3)
- Lauri Ilola, Nokia (version 1.1)
- Lei Zhang, Google (version 1.2)
- Lenny Komow, LunarG (versions 1.1, 1.2)
- Liam Middlebrook, NVIDIA (version 1.3)
- Lionel Landwerlin, Intel (versions 1.1, 1.2)
- Lisie Aartsen, Khronos (version 1.3)
- Liz Maitral, Khronos (version 1.2)
- Lou Kramer, AMD (version 1.3)
- Lutz Latta, Lucasfilm (version 1.0)

- Maciej Jesionowski, AMD (version 1.1)
- Mais Alnasser, AMD (version 1.1)
- Marcin Kantoch, AMD (version 1.3)
- Marcin Rogucki, Mobica (version 1.1)
- Maria Rovatsou, Codeplay Software Ltd. (version 1.0)
- Mariusz Merecki, Intel (version 1.3)
- Mark Bellamy, Arm (version 1.2, 1.3)
- Mark Callow, Independent (versions 1.0, 1.1, 1.2, 1.3)
- Mark Kilgard, NVIDIA (versions 1.1, 1.2)
- Mark Lobodzinski, LunarG (versions 1.0, 1.1, 1.2)
- Mark Young, LunarG (versions 1.1, 1.3)
- Markus Tavenrath, NVIDIA (version 1.1)
- Marty Johnson, Khronos (version 1.3)
- Mateusz Przybylski, Intel (version 1.0)
- Mathias Heyer, NVIDIA (versions 1.0, 1.1)
- Mathias Schott, NVIDIA (versions 1.0, 1.1)
- Mathieu Robart, Arm (version 1.2)
- Matt Netsch, Qualcomm Technologies, Inc. (version 1.1)
- Matthew Rusch, NVIDIA (version 1.3)
- Matthäus Chajdas, AMD (versions 1.1, 1.2, 1.3)
- Maurice Ribble, Qualcomm Technologies, Inc. (versions 1.0, 1.1)
- Maxim Lukyanov, Samsung Electronics (version 1.0)
- Michael Blumenkrantz, Self (version 1.3)
- Michael Lentine, Google (version 1.0)
- Michael O’Hara, AMD (version 1.1)
- Michael Phillip, Samsung Electronics (version 1.2)
- Michael Wong, Codeplay Software Ltd. (version 1.1)
- Michael Worcester, Imagination Technologies (versions 1.0, 1.1)
- Michal Pietrasiuk, Intel (versions 1.0, 1.3)
- Mika Isojarvi, Google (versions 1.0, 1.1)
- Mike Schuchardt, LunarG (versions 1.1, 1.2)
- Mike Stroyan, LunarG (version 1.0)
- Mike Weiblen, LunarG (versions 1.1, 1.2, 1.3)
- Minyoung Son, Samsung Electronics (version 1.0)
- Mitch Singer, AMD (versions 1.0, 1.1, 1.2, 1.3)

- Mythri Venugopal, Samsung Electronics (version 1.0)
- Naveen Leekha, Google (version 1.0)
- Neil Henning, AMD (versions 1.0, 1.1, 1.2, 1.3)
- Neil Hickey, Arm (version 1.2)
- Neil Trevett, NVIDIA (versions 1.0, 1.1, 1.2, 1.3)
- Nick Penwarden, Epic Games (version 1.0)
- Nicolai Hähnle, AMD (version 1.1)
- Niklas Smedberg, Unity Technologies (version 1.0)
- Norbert Nopper, Independent (versions 1.0, 1.1)
- Nuno Subtil, NVIDIA (versions 1.1, 1.2, 1.3)
- Pat Brown, NVIDIA (version 1.0)
- Patrick Cozzi, Independent (version 1.1)
- Patrick Doane, Blizzard Entertainment (version 1.0)
- Peter Lohrmann, AMD (versions 1.0, 1.2)
- Petros Bantolas, Imagination Technologies (version 1.1)
- Philip Rebohle, Valve Software (version 1.3)
- Pierre Boudier, NVIDIA (versions 1.0, 1.1, 1.2, 1.3)
- Pierre-Loup Griffais, Valve Software (versions 1.0, 1.1, 1.2, 1.3)
- Piers Daniell, NVIDIA (versions 1.0, 1.1, 1.2, 1.3)
- Ping Liu, Intel (version 1.3)
- Piotr Bialecki, Intel (version 1.0)
- Piotr Byszewski, Mobica (version 1.3)
- Prabindh Sundareson, Samsung Electronics (version 1.0)
- Pyry Haulos, Google (versions 1.0, 1.1) (Vulkan conformance test subcommittee chair)
- Rachel Bradshaw, Caster Communications (version 1.3)
- Rajeev Rao, Qualcomm (version 1.2)
- Ralph Potter, Samsung Electronics (versions 1.1, 1.2, 1.3)
- Raun Krisch, Samsung Electronics (version 1.3)
- Ray Smith, Arm (versions 1.0, 1.1, 1.2)
- Ricardo Garcia, Igalia (version 1.3)
- Richard Huddy, Samsung Electronics (versions 1.2, 1.3)
- Rob Barris, NVIDIA (version 1.1)
- Rob Stepinski, Transgaming (version 1.0)
- Robert Simpson, Qualcomm Technologies, Inc. (versions 1.0, 1.1, 1.3)
- Rolando Caloca Olivares, Epic Games (versions 1.0, 1.1, 1.2, 1.3)

- Ronan Keryell, Xilinx (version 1.3)
- Roy Ju, Mediatek (version 1.0)
- Rufus Hamade, Imagination Technologies (version 1.0)
- Ruihao Zhang, Qualcomm Technologies, Inc. (versions 1.1, 1.2, 1.3)
- Samuel (Sheng-Wen) Huang, Mediatek (version 1.3)
- Samuel Iglesias Gonsalvez, Igalia (version 1.3)
- Sascha Willems, Self (version 1.3)
- Sean Ellis, Arm (version 1.0)
- Sean Harmer, KDAB Group (versions 1.0, 1.1)
- Shannon Woods, Google (versions 1.0, 1.1, 1.2, 1.3)
- Slawomir Cygan, Intel (versions 1.0, 1.1, 1.3)
- Slawomir Grajewski, Intel (versions 1.0, 1.1, 1.3)
- Sorel Bosan, AMD (version 1.1)
- Spencer Fricke, Samsung Electronics (versions 1.2, 1.3)
- Stefanus Du Toit, Google (version 1.0)
- Stephen Huang, Mediatek (version 1.1)
- Steve Hill, Broadcom (versions 1.0, 1.2)
- Steve Viggers, Core Avionics & Industrial Inc. (versions 1.0, 1.2)
- Steve Winston, Holochip (version 1.3)
- Stuart Smith, AMD (versions 1.0, 1.1, 1.2, 1.3)
- Sujeevan Rajayogam, Google (version 1.3)
- Tilmann Scheller, Samsung Electronics (version 1.1)
- Theresa Foley, Intel (version 1.0)
- Tim Lewis, Khronos (version 1.3)
- Timo Suoranta, AMD (version 1.0)
- Timothy Lottes, AMD (versions 1.0, 1.1)
- Tobias Hector, AMD (versions 1.0, 1.1, 1.2, 1.3) (validity language and toolchain)
- Tobin Ehlis, LunarG (version 1.0)
- Tom Olson, Arm (versions 1.0, 1.1, 1.2, 1.3) (Working Group chair)
- Tomasz Bednarz, Independent (version 1.1)
- Tomasz Kubale, Intel (version 1.0)
- Tony Barbour, LunarG (versions 1.0, 1.1, 1.2)
- Tony Zlatinski, NVIDIA (version 1.3)
- Victor Eruhimov, Unknown (version 1.1)
- Vikram Kushwaha, NVIDIA (version 1.3)

- Vincent Hindriksen, Stream HPC (versions 1.2, 1.3)
- Wasim Abbas, Arm (version 1.3)
- Wayne Lister, Imagination Technologies (version 1.0)
- Wolfgang Engel, Unknown (version 1.1)
- Yanjun Zhang, VeriSilicon (versions 1.0, 1.1, 1.2, 1.3)
- Yunxing Zhu, Huawei (version 1.3)

Other Credits

The Vulkan Advisory Panel members provided important real-world usage information and advice that helped guide design decisions.

The wider Vulkan community have provided useful feedback, questions and specification changes that have helped improve the quality of the Specification via GitHub.

Administrative support to the Working Group for Vulkan 1.1, 1.2, and 1.3 was provided by Khronos staff including Ann Thorsnes, Blaine Kohl, Dominic Agoro-Ombaka, Emily Stearns, Jeff Phillips, Lisie Aartsen, Liz Maitral, Marty Johnson, Tim Lewis, and Xiao-Yu CHENG; and by Alex Crabb, Laura Shubel, and Rachel Bradshaw of Caster Communications.

Administrative support for Vulkan 1.0 was provided by Andrew Riegel, Elizabeth Riegel, Glenn Fredericks, Kathleen Mattson and Michelle Clark of Gold Standard Group.

Technical support was provided by James Riordon, site administration of Khronos.org and OpenGL.org.